Tcllib

Handbuchseiten (deutsch)

2026-10-27

Inhalt

apps **15**
dtplite 15
mkdoc 21
nns 23
nnsd 25
nnslog 26
page 27
pt 34
tcldocstrip 46
devdoc **49**
tcl_community_communication 49
tcllib_devguide 52
tcllib_installer 68
tcllib_license 74
tcllib_releasemgr 75
tcllib_sources 76
aes **77**
aes 78
amazon-s3 **80**
S3 80
xsxp 105
asn **108**
asn 108
base32 **117**
base32 117
base32core 119
base32hex 121
base64 **123**
ascii85 123
base64 125
uuencode 128
yencode 131
bee **134**
bee 134
bench **140**
bench 140
bench_intro 145
bench_lang_intro 146
bench_lang_spec 148
bench_read 150
bench_wcsv 152
bench_wtext 153
bibtex **154**
bibtex 154

blowfish **156**
blowfish 156
cache **159**
async 159
clay **161**
clay 161
clock **176**
iso8601 176
rfc2822 178
cmdline **178**
cmdline 178
comm **182**
comm 183
comm_wire 200
control **203**
control 203
coroutine **205**
coro_auto 205
tcllib_coroutine 206
counter **209**
counter 209
crc **212**
cksum 212
crc16 214
crc32 219
sum 222
cron **223**
cron 223
csv **227**
csv 227
debug **231**
debug 231
debug_caller 235
debug_heartbeat 236
debug_timestamp 237
defer **238**
defer 238
des **240**
des 240
tcldes 243
tcldesjr 244
dicttool **244**
dicttool 244
dns **246**
tcllib_dns 246
tcllib_ip 251

docstrip **258**
docstrip 258
docstrip_util 266
doctools **276**
changelog 276
cvs 278
docidx 280
docidx_intro 285
docidx_lang_cmdref 287
docidx_lang_faq 289
docidx_lang_intro 290
docidx_lang_syntax 293
docidx_plugin_apiref 295
doctoc 301
doctoc_intro 306
doctoc_lang_cmdref 308
doctoc_lang_faq 310
doctoc_lang_intro 311
doctoc_lang_syntax 316
doctoc_plugin_apiref 318
doctools 324
doctools_intro 331
doctools_lang_cmdref 333
doctools_lang_faq 341
doctools_lang_intro 342
doctools_lang_syntax 352
doctools_plugin_apiref 354
mpexpand 362
doctools2base **363**
html_cssdefaults 363
nroff_manmacros 364
tcl_parse 365
tcllib_msgcat 368
doctools2idx **369**
export_docidx 369
idx_container 372
idx_export 378
idx_export_html 384
idx_export_json 389
idx_export_nroff 393
idx_export_text 396
idx_export_wiki 398
idx_import 401
idx_import_json 408
idx_introduction 412
idx_msgcat_c 415

idx_msgcat_de 416
idx_msgcat_en 417
idx_msgcat_fr 418
idx_parse 419
idx_structure 423
import_docidx 427
doctools2toc 429
export_doctoc 429
import_doctoc 433
toc_container 436
toc_export 444
toc_export_html 450
toc_export_json 455
toc_export_nroff 459
toc_export_text 463
toc_export_wiki 466
toc_import 469
toc_import_json 477
toc_introduction 481
toc_msgcat_c 484
toc_msgcat_de 485
toc_msgcat_en 486
toc_msgcat_fr 487
toc_parse 488
toc_structure 493
dtplite 497
pkg_dtplite 497
fileutil 498
fileutil 498
multi 506
multiop 507
paths 514
traverse 515
ftp 519
ftp 519
ftp_geturl 525
ftpd 526
ftpd 526
fumagic 530
cfront 530
cgen 532
filetypes 533
rtcore 534
generator 537
generator 537
gpx 545

gpx 545
grammar_aycock 548
aycock 548
grammar_fa 550
dacceptor 550
dexec 552
fa 554
faop 564
grammar_me 571
gasm 571
me_ast 577
me_cpu 579
me_cpucore 584
me_intro 590
me_tcl 591
me_util 597
me_vm 598
grammar_peg 607
peg 607
peg_interp 615
hook 617
hook 617
html 622
html 622
htmlparse 630
htmlparse 630
http 634
autoproxy 634
httpd 639
httpd 639
ident 652
ident 652
imap4 653
imap4 653
inifile 662
ini 662
interp 664
deleg_method 664
deleg_proc 665
tcllib_interp 666
irc 667
irc 667
picoirc 671
javascript 674
javascript 674
jpeg 676

jpeg 676
json 679
json 679
json_write 682
lambda 684
lambda 684
lazyset 688
lazyset 688
ldap 690
ldap 690
ldapx 701
log 713
log 713
logger 718
loggerAppender 725
loggerUtils 726
map 729
map_geocode_nominatim 729
map_slippy 731
map_slippy_cache 740
map_slippy_fetcher 741
mapproj 742
mapproj 742
markdown 749
markdown 749
math 751
bigfloat 751
bignum 760
calculus 765
changepoint 774
combinatorics 776
constants 781
decimal 784
exact 788
figurate 792
filtergen 798
fourier 800
fuzzy 803
interpolate 805
linalg 810
machineparameters 826
math 830
math_geometry 832
numtheory 846
optimize 850
pca 856

polynomials 859
probopt 862
qcomplex 866
quasirandom 870
rational_funcs 872
roman 875
romberg 876
special 881
statistics 889
symdiff 927
trig 928
md4 932
md4 932
md5 935
md5 935
md5crypt 938
md5crypt 938
mime 939
mime 940
smtp 945
mkdoc 948
mkdoc 948
multiplexer 951
multiplexer 951
namespacex 953
namespacex 953
ncgi 956
ncgi 956
nettool 961
nettool 961
nmea 964
nmea 964
nns 967
nns_auto 967
nns_client 969
nns_common 974
nns_intro 975
nns_protocol 977
nns_server 980
nntp 982
nntp 982
ntp 988
ntp_time 988
oauth 990
oauth 990
oometa 995

oometa 995
ooutil 998
ooutil 998
otp 1001
otp 1001
page 1003
page_intro 1003
page_pluginmgr 1004
page_util_flow 1015
page_util_norm_lemon 1017
page_util_norm_peg 1018
page_util_peg 1020
page_util_quote 1022
pki 1023
pki 1023
pluginmgr 1029
pluginmgr 1029
png 1035
png 1035
pop3 1038
pop3 1038
pop3d 1042
pop3d 1042
pop3d_dbox 1046
pop3d_udb 1048
practcl 1050
practcl 1050
processman 1079
processman 1079
profiler 1081
profiler 1081
pt 1083
pt_astree 1083
pt_cparam_config_critcl 1088
pt_cparam_config_tea 1089
pt_from_api 1090
pt_introduction 1098
pt_json_language 1100
pt_param 1108
pt_parse_peg 1118
pt_parser_api 1120
pt_peg_container 1127
pt_peg_container_peg 1137
pt_peg_export 1138
pt_peg_export_container 1145
pt_peg_export_json 1153

pt_peg_export_peg 1162
pt_peg_from_container 1172
pt_peg_from_json 1172
pt_peg_from_peg 1181
pt_peg_import 1189
pt_peg_import_container 1197
pt_peg_import_json 1198
pt_peg_import_peg 1206
pt_peg_interp 1215
pt_peg_introduction 1221
pt_peg_language 1224
pt_peg_op 1233
pt_peg_to_container 1236
pt_peg_to_cparam 1244
pt_peg_to_json 1253
pt_peg_to_param 1262
pt_peg_to_peg 1286
pt_peg_to_tclparam 1295
pt_pegrammar 1303
pt_pexpr_op 1310
pt_pexpression 1314
pt_pgen 1320
pt_rdengine 1324
pt_tclparam_config_nx 1339
pt_tclparam_config_snit 1341
pt_tclparam_config_tcloo 1342
pt_to_api 1343
pt_util 1351

rc4 **1352**
rc4 1352

rcs **1354**
rcs 1354

report **1358**
report 1358

rest **1365**
rest 1365

ripemd **1375**
ripemd128 1375
ripemd160 1378

sasl **1380**
gtoken 1380
ntlm 1382
sasl 1382
scram 1388

sha1 **1389**
sha1 1389

sha256 1392
simulation 1395
annealing 1395
montecarlo 1399
simulation_random 1402
smtpd 1406
smtpd 1406
snit 1411
snit 1411
snitfaq 1452
soundex 1513
soundex 1513
stooop 1514
stooop 1514
switched 1517
string 1522
token 1522
token_shell 1526
stringprep 1529
stringprep 1529
stringprep_data 1532
unicode 1533
unicode_data 1536
struct 1536
disjointset 1536
graph 1541
graph1 1557
graphops 1563
matrix 1583
pool 1592
prioqueue 1598
queue 1600
record 1602
skiplist 1609
stack 1610
struct_list 1612
struct_map 1624
struct_set 1625
struct_tree 1628
struct_tree1 1639
tar 1643
tar 1643
tepam 1647
tepam_argument_dialogbox 1647
tepam_doc_gen 1663
tepam_introduction 1671

tepam_procedure 1679
term 1703
ansi_cattr 1703
ansi_cctrl 1707
ansi_cmacros 1714
ansi_code 1715
ansi_ctrlu 1716
ansi_send 1718
imenu 1725
ipager 1728
receive 1731
term 1732
term_bind 1733
term_send 1735
textutil 1736
adjust 1736
expander 1741
patch 1748
repeat 1750
tabify 1751
textutil 1753
textutil_split 1766
textutil_string 1768
trim 1770
wcswidth 1773
tie 1775
tie 1775
tie_std 1782
tiff 1783
tiff 1783
tool 1787
meta 1787
tool 1790
tool_dict_ensemble 1795
transfer 1796
connect 1796
copyops 1800
ddest 1802
dsource 1805
receiver 1808
tqueue 1812
transmitter 1815
treeql 1820
treeql 1820
try 1830
tcllib_fhome 1830

tcllib_throw 1831
tcllib_try 1832
udpcluster 1834
udpcluster 1834
uev 1835
uevent 1835
uevent_onidle 1839
units 1840
units 1840
uri 1846
uri 1846
urn-scheme 1853
uuid 1854
uuid 1854
valtype 1855
cc_amex 1855
cc_discover 1857
cc_mastercard 1859
cc_visa 1861
ean13 1863
iban 1864
imei 1866
isbn 1868
luhn 1870
luhn5 1872
usnpi 1873
valtype_common 1875
verhoeff 1877
virtchannel_base 1879
cat 1879
facade 1880
halfpipe 1881
nullzero 1883
randseed 1884
std 1885
tcllib_fifo 1886
tcllib_fifo2 1887
tcllib_memchan 1888
tcllib_null 1889
tcllib_random 1890
tcllib_string 1891
tcllib_variable 1892
tcllib_zero 1893
textwindow 1894
virtchannel_core 1894
core 1894

events 1896
transformcore 1898
virtchannel_transform 1899
adler32 1899
hex 1900
identity 1901
limitsize 1902
observe 1903
rot 1904
spacer 1905
tcllib_zlib 1906
vt_base64 1907
vt_counter 1908
vt_crc32 1910
vt_otp 1911
websocket 1912
websocket 1912
wip 1920
wip 1920
yaml 1926
huddle 1926
yaml 1937
zip 1940
decode 1940
encode 1943
mkzip 1944
Lizenz 1946
Lizenzbedingungen license 1947

apps

dtplite

Stichwörter: dtplite, HTML, TMML, conversion, docidx, doctoc, doctools, manpage, markup, nroff

Name

dtplite – Ein schlanker DocTools-Markup-Prozessor

Übersicht

dtplite -o *output* ?*options*? *format inputfile*
dtplite validate *inputfile*
dtplite -o *output* ?*options*? *format inputdirectory*
dtplite -merge -o *output* ?*options*? *format inputdirectory*

Beschreibung

Die in diesem Dokument beschriebene Anwendung **dtplite** ist der Nachfolger des äußerst einfachen **mpexpand**. In ihrer Funktionalität ist sie vom doctools-Prozessor **dtp** beeinflusst und damit wesentlich leistungsfähiger als **mpexpand**, und dennoch genauso benutzerfreundlich; auf jeden Fall einfacher als **dtp** mit seiner Vielzahl an Unterbefehlen und Optionen.

dtplite basiert, ebenso wie die beiden anderen Prozessoren, auf dem Paket **doctools**.

Anwendungsfälle

dtplite wurde unter Berücksichtigung der folgenden drei Anwendungsfälle entwickelt.

1. Validierung eines einzelnen Dokuments, d. h. Überprüfung, ob es im gültigen doctools-Format verfasst wurde. Dieser Modus kann auch verwendet werden, um eine vorläufige Version der formatierten Ausgabe für ein einzelnes Dokument zu erhalten, die in einem Browser, mit nroff usw. angezeigt werden kann, wodurch eine Korrekturlesung der Formatierung ermöglicht wird.
2. Erstellung der formatierten Dokumentation für ein einzelnes Paket, d. h. aller Manpages sowie einer Tabelle mit Inhaltsverzeichnis und einem Index mit Schlüsselwörtern.
3. Eine Erweiterung der bisherigen Vorgehensweise, eine Methode zur einfachen Erstellung eines Dokumentationsbaums für mehrere Pakete und insbesondere einer einheitlichen Tabelle mit Inhaltsverzeichnis und Index für Schlüsselwörter.

Darüber hinaus möchten wir auch die vom HTML-Formatierer bereitgestellten Anpassungsfunktionen nutzen. Es ist zwar nicht das einzige Format, das die Anwendung generieren können sollte, doch gehen wir davon aus, dass es am häufigsten verwendet wird, und es ist eines der wenigen, die Anpassungsmöglichkeiten bieten.

Wir ermöglichen es dem Aufrufer, eine Kopfzeile, eine Fußzeile, ein Stylesheet sowie Daten für eine Navigationsleiste am oberen Rand des generierten Dokuments festzulegen. Zwar können alle diese Elemente festgelegt werden, sofern die Formatierungs-Engine einen entsprechenden Parameter bereitstellt (siehe Abschnitt Optionen), doch unterliegen die beiden letztgenannten einer internen Verarbeitung, die sie spezifisch für HTML macht.

Befehlszeile

dtplite -o *output* ?options? *format inputfile*

Dies ist die Form für den Anwendungsfall [1]. Die *options* werden später im Abschnitt

Optionen erläutert.

path *output* (in)

Dieses Argument legt fest, wohin das generierte Dokument geschrieben werden soll. Es kann sich dabei um den Pfad zu einer Datei oder einem Verzeichnis handeln oder um "**-**". Der letzte Wert bewirkt, dass die Anwendung das generierte Dokument in "**stdout**" schreibt.

Wenn der *output* nicht existiert, muss [file dirname $output] vorhanden sein und es muss sich um ein beschreibbares Verzeichnis handeln. Das generierte Dokument wird in eine Datei in diesem Verzeichnis geschrieben, wobei der Name dieser Datei aus *inputfile*, *format* und dem für die Option **-ext** angegebenen Wert (sofern vorhanden) abgeleitet wird.

(path|handle) *format* (in)

Dieses Argument legt fest, welche Formatierungs-Engine bei der Verarbeitung der Eingabe verwendet werden soll, und bestimmt somit das Format des erzeugten Dokuments. Die von der Anwendung unterstützten Möglichkeiten finden Sie im Abschnitt Formate.

path *inputfile* (in)

Dieses Argument legt den Pfad zu der zu verarbeitenden Datei fest. Die Datei muss vorhanden und lesbar sein und im *doctools*-Format vorliegen.

dtplite validate *inputfile*

Dies ist eine vereinfachte Form für den Anwendungsfall [1]. Das Format "validate" erzeugt keinerlei Ausgabe; es werden lediglich Syntaxprüfungen durchgeführt. Daher ist die Angabe einer Ausgabedatei oder anderer Optionen nicht erforderlich und wird weggelassen.

dtplite -o *output* ?options? *format inputdirectory*

Dies ist die Form für den Anwendungsfall [2]. Es unterscheidet sich von der Form für den Anwendungsfall [1] dadurch, dass die Eingabedokumente über ein Verzeichnis statt über eine Datei angegeben werden. Die übrigen Argumente sind identisch, mit Ausnahme von *output*, das nun den Pfad zu einem vorhandenen und beschreibbaren Verzeichnis angeben muss.

Bei den Eingabedokumenten handelt es sich um alle Dateien im *inputdirectory* oder in einem seiner Unterverzeichnisse, die von **fileutil::fileType** als Dateien im *doctools*-Format erkannt wurden.

dtplite -merge -o *output* ?options? *format inputdirectory*

Dies ist die Form für den Anwendungsfall [3]. Der einzige Unterschied zur Form für den Anwendungsfall [2] besteht in der zusätzlichen Option **-merge**.

Bei jedem solchen Aufruf werden die generierten Dokumente, die aus dem Prozess der Eingabedokumente unter *inputdirectory* oder einem seiner Unterverzeichnisse stammen, mit den Dateien unter *output* zusammengeführt. Auf diese Weise ist es möglich, die einheitliche Dokumentation für eine beliebige Anzahl von Paketen schrittweise aufzubauen. Beachten Sie, dass alle Pakete zweimal durchlaufen werden müssen, um vollständig korrekte Querverweise zu erhalten (bei Formaten, die diese unterstützen).

Optionen

In diesem Abschnitt werden alle Optionen beschrieben, die dem Benutzer der Anwendung zur Verfügung stehen, mit Ausnahme der Optionen **-o** und **-merge**. Diese beiden wurden bereits im

Abschnitt Befehlszeile beschrieben.

-exclude string
Diese Option legt ein Ausschlussmuster (Glob-Muster) fest. Alle Dateien, die als zu verarbeitende Manpages identifiziert wurden und dem Ausschlussmuster entsprechen, werden ignoriert. Die Option kann mehrfach angegeben werden, wobei jede Angabe ein weiteres Muster zur Liste der Ausschlüsse hinzufügt.

-ext string
Wenn der Name einer Ausgabedatei aus dem Namen einer Eingabedatei abgeleitet werden muss, wird standardmäßig der Name des *format* als Erweiterung verwendet. Diese Option hebt dies jedoch auf und erzwingt die Verwendung von *string* als Erweiterung. Diese Option wird ignoriert, wenn der Name der Ausgabedatei vollständig über die Option **-o** festgelegt ist.

Bei mehrfacher Verwendung ist nur die letzte Definition maßgeblich.

-header file
Diese Option kann genau dann verwendet werden, wenn die Auswahl das *format* ausgewählt hat, das einen Engine-Parameter namens "header" bereitstellt. Sie übernimmt den Inhalt der angegebenen Datei und weist ihn diesem Parameter zu, damit die Engine ihn entsprechend nutzen kann. Die HTML-Engine fügt den Text unmittelbar nach dem Tag ein. Sind Navigationsschaltflächen vorhanden (siehe Option **-nav** weiter unten), wird der für diese generierte HTML-Code vor der endgültigen Zuweisung an den Parameter an die hier stammenden Header-Daten angehängt.

Bei mehrfacher Verwendung ist nur die letzte Definition relevant.

-footer file
Wie **-header**, mit folgenden Unterschieden: Alle Navigationsschaltflächen werden ignoriert, der entsprechende erforderliche Engine-Parameter hat den Namen "footer" und die Daten werden unmittelbar vor dem Tag eingefügt.

Bei mehrfacher Verwendung ist nur die letzte Definition relevant.

-style file
Diese Option kann genau dann verwendet werden, wenn die Auswahl das *format* ausgewählt hat, das einen Engine-Parameter mit dem Namen "meta" bereitstellt. Wenn dies der Fall ist, wird ein HTML-Codeabschnitt generiert, der die *file* als Stylesheet für das generierte Dokument deklariert, und dieser wird dem Parameter zugewiesen. Die HTML-Engine fügt diesen Code unmittelbar nach dem Tag in das Dokument ein.

Bei der Verarbeitung eines Eingabeverzeichnisses wird die Stylesheet-Datei in das Ausgabeverzeichnis kopiert, und der generierte HTML-Code verweist auf diese Kopie, um das Ergebnis in sich geschlossener zu gestalten. Bei der Verarbeitung einer Eingabedatei gibt es keinen Speicherort, an den das Stylesheet kopiert werden könnte, weshalb einfach wie angegeben auf die Datei verwiesen wird.

Bei mehrfacher Verwendung ist nur die letzte Definition relevant.

-toc path
Diese Option legt eine Doctoc-Datei fest, die für das Inhaltsverzeichnis verwendet werden

soll, anstatt ein eigenes zu erstellen.

Bei mehrfacher Verwendung ist nur die letzte Definition maßgeblich.

-pre+toc label path|text
siehe unten ...

-post+toc label path|text
Diese Option legt zusätzliche Doctoc-Dateien (oder Texte) fest, die in der Navigationsleiste verwendet werden sollen.

Die Positionierung und die Handhabung bei mehrfacher Verwendung entsprechen den Optionen **-prenav** und **-postnav**; siehe unten.

-nav label url
siehe unten ...

-prenav label url
Legt eine Navigationsschaltfläche mit der anzuzeigenden *label* und der *url* fest, auf die verwiesen wird. Diese Option kann genau dann verwendet werden, wenn das ausgewählte *format* einen Engine-Parameter namens "header" bereitstellt. Der dafür erzeugte HTML-Code wird an die Daten aus der Option **-header** angehängt, bevor er in die erzeugten Dokumente eingefügt wird.

Bei mehrfacher Verwendung werden alle Angaben gesammelt und eine Navigationsleiste erzeugt, die erste Angabe am linken Rand und die letzte am rechten.

Die URL kann relativ sein. In diesem Fall wird davon ausgegangen, dass sie relativ zu den Hauptdateien (Inhaltsverzeichnis und Index für Schlüsselwörter) ist, und sie wird durch eine Transformation so umgewandelt, dass alle anderen Verweise weiterhin korrekt funktionieren.

-postnav label url
Legt eine Navigationsschaltfläche mit der anzuzeigenden *label* und der *url* fest, auf die verwiesen wird. Diese Option kann nur dann verwendet werden, wenn das ausgewählte *format* einen Engine-Parameter namens "header" bereitstellt. Der dafür erzeugte HTML-Code wird an die Daten aus der Option **-header** angehängt, bevor er in die erzeugten Dokumente eingefügt wird.

Bei mehrfacher Verwendung werden alle Angaben gesammelt und eine Navigationsleiste erzeugt, die letzte Angabe am rechten Rand und die erste am linken.

Die URL kann relativ sein. In diesem Fall wird davon ausgegangen, dass sie relativ zu den Hauptdateien (Inhaltsverzeichnis und Index für Schlüsselwörter) ist, und sie unterliegt einer Transformation, damit alle anderen Verweise weiterhin korrekt funktionieren.

Formate

Zunächst wird das Argument *format* als Pfad zu einer TCL-Datei interpretiert, die den Code für die angeforderte Formatierungs-Engine enthält. Das Argument wird genau dann als Name eines der unten aufgeführten vordefinierten Formate interpretiert, wenn der Pfad nicht existiert.

Beachten Sie eine Einschränkung: Wenn die Interpretation des Formats als Pfad zu dem

Tcl-Skript, das die Engine implementiert, erfolgreich war, muss dieses Skript nicht nur die Engine-API für doctools, d. h. *doctools_api*, sondern auch die APIs für *doctoc_api* und *docidx_api* implementieren. Andernfalls schlägt die Erstellung einer Tabelle mit Inhaltsverzeichnis und einem Index für Schlüsselwörter fehl.

Liste der vordefinierten Formate, d. h. derjenigen, die vom Paket **doctools** bereitgestellt werden:

nroff
: Der Prozessor erzeugt eine *roff-Ausgabe, das Standardformat für Unix-Manpages.

html
: Der Prozessor erzeugt HTML-Ausgabe, die in Webbrowsern verwendet und dort angezeigt wird. Diese Engine ist derzeit die einzige, die die verschiedenen Engine-Parameter bereitstellt, die für die zusätzliche Anpassung der Ausgabe erforderlich sind.

tmml
: Der Prozessor erzeugt TMML-Ausgaben, die Tcl Manpage Markup Language, eine Ableitung von XML.

latex
: Der Prozessor erzeugt eine LaTeX-Ausgabe.

wiki
: Der Prozessor erzeugt Wiki-Markup, wie es **wikit** versteht.

list
: Der Prozessor extrahiert die von **manpage_begin** bereitgestellten Informationen. Dieses Format wird intern verwendet, um die Metadaten zu extrahieren, aus denen sowohl das Inhaltsverzeichnis als auch der Index für Schlüsselwörter abgeleitet werden.

null
: Der Prozessor erzeugt keine Ausgabe. Dies entspricht dem Fehlercode **validate**.

Verzeichnisstrukturen

In diesem Abschnitt beschreiben wir die Verzeichnisstrukturen, die von der Anwendung unter *output* bei der Verarbeitung aller Dokumente in einem *inputdirectory* erzeugt werden. Mit anderen Worten: Dies ist nur für die Anwendungsfälle [2] und [3] relevant.

[2]
: Wenn ein einzelner Satz von Eingabedokumenten verarbeitet wird, wird die folgende Verzeichnisstruktur erstellt. Die verwendete Erweiterung dient der Ausgabe im HTML-Format; dies ist jedoch für die Struktur nicht relevant und wurde lediglich verwendet, um korrekte Namen der Dateien zu gewährleisten.

```
output/
    toc.html
    index.html
    files/
        path/to/FOO.html
```

Die letzte Zeile in den Beispielen zeigt das Dokument, das für eine Datei namens FOO generiert wurde, die sich unter folgendem Pfad befindet:

```
inputdirectory/path/to/FOO
```

[3]

Wenn viele Pakete zu einem einheitlichen Dokumentensatz zusammengeführt werden, ist die erzeugte Verzeichnisstruktur etwas tiefer gegliedert:

```
    output
        .toc
        .idx
        .tocdoc
        .idxdoc
        .xrf
        toc.html
        index.html
        FOO1/
            ...
        FOO2/
            toc.html
            files/
                path/to/BAR.html
```

Jedes der Verzeichnisse FOO1, ... enthält die für das Paket FOO1, ... generierten Dokumente und folgt der für den Anwendungsfall [2] dargestellten Struktur. Die einzige Ausnahme besteht darin, dass kein paketbezogener Index vorhanden ist.

Die Dateien "*.toc*", "*.idx*" und "*.xrf*" enthalten den internen Status der gesamten Ausgabe und werden beim nächsten Aufruf gelesen und aktualisiert. Ihr Inhalt wird nicht dokumentiert. Löschen Sie diese Dateien, wenn alle für die Ausgabe gewünschten Pakete verarbeitet wurden, d. h. wenn die Ausgabe vollständig ist.

Die Dateien "*.tocdoc*" und "*.idxdoc*" sind Zwischendateien im doctoc- bzw. docidx-Markup, die das Hauptinhaltsverzeichnis und den Index für die Dokumentensammlung vor deren Konvertierung in das gewählte Ausgabeformat enthalten. Sie bleiben an Ort und Stelle, d. h. sie werden nicht gelöscht, um als Beispiele für die Markup-Formate von doctoc und docidx zu dienen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2004-2013 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

mkdoc

Stichwörter: mkdoc

Name

mkdoc – Anwendung zum Extrahieren und Konvertieren von Quellcode-Dokumentation

Übersicht

mkdoc --help
mkdoc --version
mkdoc --license
mkdoc *input output* ?*--css cssfile*?

Beschreibung

Dieses Dokument beschreibt **mkdoc**, eine Anwendung zum Extrahieren von Dokumentation, die in Quellcode-Dateien eingebettet ist, seien es "*.tcl*"-Dateien oder andere.

Befehlszeile

mkdoc --help
Die Anwendung gibt eine kurze Hilfe auf die Standardausgabe aus und beendet sich.

mkdoc --version
Die Anwendung gibt ihre Versionsnummer auf der Standardausgabe aus und wird beendet.

mkdoc --license
Die Anwendung gibt ihre Lizenz auf der Standardausgabe aus und wird beendet.

mkdoc *input output* ?--css *cssfile*?
Die Anwendung liest die Datei *input* ein, extrahiert die darin enthaltene Dokumentation und schreibt sie in die Datei *output*.

Wenn die Ausgabedatei nicht die Endung "*.md*" hat, wird die extrahierte Dokumentation vor dem Schreiben in HTML konvertiert.

Wenn die Generierung und das Schreiben von HTML erfolgen, kann das Standard-CSS-Stylesheet überschrieben werden, indem über die Option **--css** der Pfad zu einem benutzerdefinierten Stylesheet festgelegt wird.

Wenn die Eingabedatei eine "*.md*"-Datei ist, wird erwartet, dass sie Markdown im Originalformat enthält, anstatt in Code eingebettetes Markdown.

Wenn die Datei hingegen als Code betrachtet wird, wird erwartet, dass die Dokumentation in allen Zeilen enthalten ist, die mit dem Marker **#'** beginnen. Bei Skriptsprachen wie Tcl bedeutet das Zeichen **#** dieses Markers, dass die Dokumentation in den entsprechend gekennzeichneten Kommentaren enthalten ist. Bei anderen Sprachen müssen der Marker und die Dokumentation möglicherweise in mehrzeilige Kommentare eingebettet werden.

Beispiele

```
    # Create HTML manual for a CPP file using a custom style sheet
    mkdoc sample.cpp sample.html --css manual.css

    # Extract the documentation from code as simple Markdown, ready to
    be processed
    # further, for example with pandoc, or similar
    mkdoc sample.cpp sample.md

    # Convert a Markdown file to HTML
    mkdoc sample.md sample.html
```

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese dem Autor des Pakets. Bitte teilen Sie ihm auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation mit.

Copyright am Code

Art der BSD-Lizenz:

Die folgenden Bedingungen gelten für alle Dateien, die mit der Software in Zusammenhang stehen, sofern in einzelnen Dateien nicht ausdrücklich etwas anderes angegeben ist.

Die Autoren erteilen hiermit die Erlaubnis, diese Software und ihre Dokumentation für jeden beliebigen Zweck zu nutzen, zu vervielfältigen, zu ändern, zu verbreiten und zu lizenzieren, vorausgesetzt, dass die vorhandenen Copyright-Hinweise in allen Kopien beibehalten werden und dass dieser Hinweis bei jeder Weitergabe wörtlich übernommen wird. Für keine der zulässigen Verwendungszwecke ist eine schriftliche Vereinbarung, eine Lizenz oder die Zahlung von Lizenzgebühren erforderlich. Änderungen an dieser Software können durch ihre Autoren urheberrechtlich geschützt sein und müssen nicht den hier beschriebenen Lizenzbedingungen entsprechen, sofern die neuen Bedingungen auf der ersten Seite jeder Datei, für die sie gelten, deutlich angegeben sind.

In keinem Fall haften die Autoren oder Vertreiber gegenüber Dritten für direkte, indirekte, besondere, zufällige oder Folgeschäden, die sich aus der Nutzung dieser Software, ihrer Dokumentation oder daraus abgeleiteter Werke ergeben, selbst wenn die Autoren auf die Möglichkeit solcher Schäden hingewiesen wurden.

Die Autoren und Vertreiber schließen ausdrücklich jegliche Gewährleistung aus, einschließlich, aber nicht beschränkt auf die stillschweigenden Gewährleistungen der Marktgängigkeit, der Eignung für einen bestimmten Zweck und der Nichtverletzung von Rechten Dritter. Diese Software wird "wie besehen" bereitgestellt, und die Autoren und Vertreiber sind nicht verpflichtet, Wartung, Support, Updates, Erweiterungen oder Modifikationen bereitzustellen.

EINGESCHRÄNKTE RECHTE: Die Nutzung, Vervielfältigung oder Offenlegung durch die Regierung unterliegt den Einschränkungen, die in Unterabsatz (c) (1) (ii) der Klausel über Rechte an technischen Daten und Computersoftware gemäß DFARS 252.227-7013 und FAR 52.227-19 festgelegt sind.

Kategorie
Textverarbeitung

Copyright

```
Copyright (c) 2019-2022, Detlef Groth <detlef(at)dgroth(dot)de>
```


nns
Stichwörter: nns, application, client, name service

Name
nns – Name Service Facility, Befehlszeilen-Client-Anwendung

Übersicht
nns bind ?*-host host*? ?*-port port*? *name data*
nns search ?*-host host*? ?*-port port*? ?**-continuous**? ?*pattern*?
nns ident ?*-host host*? ?*-port port*?
nns who

Beschreibung
Bitte lesen Sie zunächst *Namensdienst-Funktion, Einleitung*.

Die in diesem Dokument beschriebene Anwendung **nns** ist ein einfacher Befehlszeilen-Client für die Nano-Namensdienstfunktion, die von den Tcllib-Paketen **nameserv** und **nameserv::server** bereitgestellt wird. Darüber hinaus dienen die Quelltexte der Anwendung auch als Beispiele für die Verwendung des Client-Pakets **nameserv**. Alle Funktionen eines Clients werden abgedeckt, von der Konfiguration über die Registrierung von Namen bis hin zur Suche.

Diese Namensdienstfunktion hat nichts mit dem *Domain Name System* des Internets zu tun, das auch als *DNS* bekannt ist. Wenn Sie nach einem Paket suchen, das sich damit befasst, sehen Sie sich bitte eines der Pakete **dns** oder **resolv** an, die beide ebenfalls in Tcllib zu finden sind.

Anwendungsfälle
nns wurde unter Berücksichtigung der folgenden zwei Hauptanwendungsfälle entwickelt.

1. Registrierung eines Namens-Daten-Paares im Namensdienst.
2. Durchsuchen des Namensdienstes nach Einträgen, die einem Glob-Muster entsprechen.

Darüber hinaus möchten wir auch den Client identifizieren und Informationen über den Namensdienst abrufen können.

Befehlszeile
nns bind ?-host *host*? ?-port *port*? *name data*

Diese Form registriert das Paar *name*/*data* im angegebenen Namensdienst. In dieser Form wird der Befehl *nicht* beendet, damit die Registrierung bestehen bleibt. Der Benutzer muss ihn ausdrücklich beenden, entweder durch ein Signal oder über die Jobsteuerung der verwendeten Shell. Der Prozess übersteht insbesondere den Verlust der Verbindung zum Namensdienst und stellt das Paar *name*/*data* wieder her, wenn die Verbindung wiederhergestellt ist.

Die Optionen zur Angabe des Namensdienstes werden später im Abschnitt Optionen erläutert.

nns search ?-host *host*? ?-port *port*? ?-continuous? ?*pattern*?

Diese Form durchsucht den angegebenen Namensdienst nach Einträgen, die dem Glob-*pattern* entsprechen, und gibt diese über stdout aus, wobei jeder Eintrag in einer eigenen Zeile steht. Wenn kein Muster festgelegt wird, wird standardmäßig * verwendet, das mit allem übereinstimmt.

Die Optionen zur Angabe des Namensdienstes werden später im Abschnitt Optionen erläutert.

Wenn die Option **-continuous** festgelegt wird, beendet der Client nach Durchführung der Suche den Vorgang nicht, sondern beginnt, den Dienst kontinuierlich auf Änderungen an der Menge der übereinstimmenden Einträge zu überwachen und die Anzeige entsprechend zu aktualisieren, sobald Änderungen eintreffen. In dieser Form übersteht er insbesondere auch den Verlust der Verbindung zum Namensdienst und nimmt die Suche wieder auf, wenn die Verbindung wiederhergestellt ist.

nns ident ?-host *host*? ?-port *port*?

Diese Form fragt den angegebenen Namensdienst nach der Version und den Funktionen des von ihm unterstützten Namensdienstprotokolls ab und gibt die Ergebnisse über stdout aus.

Die Optionen zur Angabe des Namensdienstes werden später im Abschnitt Optionen erläutert.

nns who

Diese Form gibt den Namen, die Version und die Protokollversion der Anwendung auf stdout aus.

Optionen

In diesem Abschnitt werden alle Optionen beschrieben, die dem Benutzer der Anwendung zur Verfügung stehen

-host name|ipaddress

Wenn diese Option nicht angegeben wird, wird standardmäßig "**localhost**" verwendet. Sie legt den Namen oder die IP-Adresse des Hosts fest, auf dem der anzusprechende Namensdienst ausgeführt wird.

-port number

Wenn diese Option nicht angegeben wird, wird standardmäßig "**38573**" verwendet. Sie legt den TCP-Port fest, auf dem der anzufragende Namensdienst auf Anfragen wartet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nnsd

Stichwörter: nnsd, application, name service, server

Name

nnsd – Name Service Facility, Serveranwendung für die Befehlszeile

Übersicht

nnsd ?*-localonly flag*? ?*-port port*?

Beschreibung

Bitte lesen Sie zunächst *Namensdienst-Funktion, Einleitung*.

Die in diesem Dokument beschriebene Anwendung "nns" ist ein einfacher Befehlszeilenserver für die Nano-Namensdienstfunktion, die von den Tcllib-Paketen "**nameserv**" und "**nameserv::server**" bereitgestellt wird. Darüber hinaus dienen die Quellcodes der Anwendung auch als Beispiele für die Verwendung des Serverpakets **nameserv::server**.

Diese Namensdienstfunktion hat nichts mit dem *Domain Name System* des Internets zu tun, das auch als *DNS* bekannt ist. Wenn Sie nach einem Paket suchen, das sich damit befasst, sehen Sie sich bitte eines der Pakete **dns** oder **resolv** an, die beide ebenfalls in Tcllib zu finden sind.

Anwendungsfälle

nnsd wurde unter Berücksichtigung des folgenden Hauptanwendungsfalls entwickelt.

1. Einen Nano-Namensdienst auf einem beliebigen Host ausführen.

Befehlszeile

nnsd ?-localonly *flag*? ?-port *port*?

Der Befehl konfiguriert einen Server gemäß den angegebenen Optionen und startet ihn. Der Befehl beendet sich nicht von selbst, da er die Namensdienst-Datenbank vollständig im Arbeitsspeicher hält. Der Benutzer muss ihn explizit beenden, entweder durch Senden eines Signals oder über die Job-Steuerungsfunktionen der verwendeten Shell.

Die Optionen zur Konfiguration des Namensdienstes werden im Abschnitt Optionen erläutert.

Optionen

In diesem Abschnitt werden alle Optionen beschrieben, die dem Benutzer der Anwendung zur Verfügung stehen

-localonly bool

Wenn diese Option nicht festgelegt wird, gilt standardmäßig die Einstellung **true**, d. h., es werden nur lokale Verbindungen akzeptiert. Der Server akzeptiert Fernverbindungen, d. h. Verbindungen von anderen Hosts, genau dann, wenn diese Option auf **false** konfiguriert ist.

-port number

Wenn diese Option nicht angegeben wird, wird standardmäßig "**38573**" verwendet. Sie legt den TCP-Port fest, auf dem der Server auf Anfragen warten soll.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

nnslog

Stichwörter: nnslog, application, client, name service

Name

nnslog – Name Service Facility, Client-Anwendung zur Protokollierung über die Befehlszeile

Übersicht

nnslog ?*-host host*? ?*-port port*?

Beschreibung

Bitte lesen Sie zunächst *Namensdienst-Funktion, Einleitung*.

Die in diesem Dokument beschriebene Anwendung, **nnslog**, ist ein einfacher Befehlszeilen-Client für die Nano-Namensdienstfunktion, die von den Tcllib-Paketen **nameserv** und **nameserv::server** bereitgestellt wird.

Im Wesentlichen wird damit "nns search -continuous *" umgesetzt, allerdings mit einer anderen Ausgabeformatierung. Anstatt den aktuellen Inhalt des Servers fortlaufend im Terminal anzuzeigen, protokolliert es lediglich alle empfangenen Hinzufügungs- und Entfernungsereignisse in der Datei "**stdout**".

Diese Namensdienstfunktion hat nichts mit dem *Domain Name System* des Internets zu tun, das auch als *DNS* bekannt ist. Wenn Sie nach einem Paket suchen, das sich damit befasst, sehen Sie sich bitte eines der Pakete **dns** oder **resolv** an, die beide ebenfalls in Tcllib zu finden sind.

Anwendungsfälle

nnslog wurde unter Berücksichtigung des folgenden Hauptanwendungsfalls entwickelt.

1. Überwachung des Namensdienstes auf alle Änderungen und Protokollierung dieser Änderungen in einem Textterminal.

Befehlszeile

nnslog ?-host *host*? ?-port *port*?

Der Befehl stellt eine Verbindung zum angegebenen Namensdienst her, richtet eine Suche nach allen Änderungen ein und gibt anschließend alle empfangenen Ereignisse auf der Standardausgabe aus, wobei jedes Ereignis in einer eigenen Zeile steht. Der Befehl wird erst beendet, wenn er vom Benutzer explizit beendet wird. Er übersteht insbesondere den Verlust der Verbindung zum Namensdienst und nimmt die Suche sowie die Protokollierung wieder auf, sobald die Verbindung wiederhergestellt ist.

Die Optionen zur Angabe des Namensdienstes werden später im Abschnitt Optionen erläutert.

Optionen

In diesem Abschnitt werden alle Optionen beschrieben, die dem Benutzer der Anwendung zur Verfügung stehen

-host name|ipaddress

Wenn diese Option nicht angegeben wird, wird standardmäßig "**localhost**" verwendet. Sie legt den Namen oder die IP-Adresse des Hosts fest, auf dem der anzusprechende Namensdienst ausgeführt wird.

-port number

Wenn diese Option nicht angegeben wird, wird standardmäßig "**38573**" verwendet. Sie legt den TCP-Port fest, auf dem der anzufragende Namensdienst auf Anfragen wartet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page

Stichwörter: page, parser generator, text processing

Name

Seite – Parser-Generator

Übersicht

page ?*options...*? ?*input ?output*?

Beschreibung

Die in diesem Dokument beschriebene Anwendung, **page**, ist tatsächlich nicht nur ein Parser-Generator, wie der Name vermuten lässt, sondern ein generisches Werkzeug zur Durchführung beliebiger Transformationen an Texten.

Seine Vielseitigkeit beruht auf der Verwendung von *Plugins* zum Lesen, zur Transformation und zum Schreiben von Daten, und die von Tcllib bereitgestellte vordefinierte Sammlung von Plugins dient der Generierung von rekursiven Abstiegsparsern mit Memoisierung (auch bekannt als *Packrat-Parser*) aus Grammatik-Spezifikationen (*Parsing Expression Grammars*).

page baut auf dem Paket **page::pluginmgr** auf und bündelt dessen Funktionalität in einer kommandozeilenbasierten Anwendung. Alle anderen **page::***-Pakete sind Plugins und/oder Unterstützungspakete für die Erstellung von Parsern. Die Parser selbst basieren auf den Paketen **grammar::peg**, **grammar::peg::interp** und **grammar::mengine**.

Befehlszeile

page ?*options*...? ?*input* ?*output*??

Dies ist die allgemeine Form zum Aufruf von **page**. Die Anwendung liest den Inhalt der Datei *input*, führt einen Prozess unter Berücksichtigung der festgelegten *options* durch und schreibt das Ergebnis anschließend in die Datei *output*.

Wenn *input* die Zeichenfolge **-** ist, werden die zu verarbeitenden Daten statt aus einer Datei aus **stdin** gelesen. Analog dazu wird das Ergebnis in die Datei **stdout** statt in eine Datei geschrieben, wenn *output* die Zeichenfolge **-** ist. Fehlt eine Angabe für die Ausgabe oder Eingabe, geht die Anwendung von "**-**" aus.

Die detaillierten Spezifikationen der unterstützten *options* finden Sie im Abschnitt Optionen.

path *input* (in)

Dieses Argument legt den Pfad zu der Datei fest, die von der Anwendung verarbeitet werden soll, oder "**-**". Der letzte Wert bewirkt, dass die Anwendung den Text aus der Datei "**stdin**" liest. Andernfalls muss die Datei vorhanden und lesbar sein. Wenn das Argument fehlt, wird "**-**" angenommen.

path *output* (in)

Dieses Argument legt fest, wohin der generierte Text geschrieben werden soll. Es kann sich dabei um den Pfad zu einer Datei oder um "**-**" handeln. Der letztgenannte Wert bewirkt, dass die Anwendung das generierte Dokument in "**stdout**" schreibt.

Wenn die Datei *output* nicht existiert, muss [file dirname $output] vorhanden sein und ein beschreibbares Verzeichnis darstellen, da die Anwendung die Datei, in die geschrieben werden soll, selbst anlegt.

Fehlt das Argument, wird **-** angenommen.

Betrieb

... Lesen ... Transformation ... Schreiben – Plugins – Pipeline ...

Optionen

In diesem Abschnitt werden alle Optionen beschrieben, die dem Benutzer der Anwendung zur Verfügung stehen. Die Optionen werden stets der Reihe nach verarbeitet. Das heißt, wenn sowohl **--help** als auch **--version** angegeben sind, hat die zuerst gefundene Option Vorrang.

Unbekannte Optionen, die vor einer der Optionen **-rd**, **-wr** oder **-tr** festgelegt werden, führen dazu, dass der Prozess mit einer Fehlermeldung abgebrochen wird. Unbekannte Optionen, die zwischen diesen Optionen oder nach der letzten davon stehen, werden so behandelt, als würden sie stets ein einzelnes Argument erwarten, und werden der letzten vor ihnen stehenden Plugin-Option zugeordnet. Sie werden erst geprüft, nachdem alle relevanten Plugins – und damit die von ihnen unterstützten Optionen – bekannt sind. Das heißt, solche unbekannten Optionen führen genau dann zu einem Fehler, wenn die Plugin-Option, der sie zugeordnet sind, diese nicht versteht und nicht durch eine nachfolgende Plugin-Option ersetzt wurde.

Die Standardoptionen kommen genau dann zur Anwendung, wenn die Befehlszeile keinerlei Optionen enthielt. Sie richten die Anwendung als PEG-basierten Parsergenerator ein. Die genaue Liste der Optionen lautet

```
-c peg
```

Und nun die unterstützten Optionen und ihre Argumente, sofern vorhanden:

--help
: siehe unten ...

-h
: siehe unten ...

-?
: Wird eine dieser Optionen in der Befehlszeile gefunden, werden alle davor oder danach stehenden Argumente ignoriert. Die Anwendung gibt eine kurze Beschreibung der erkannten Optionen aus und wird beendet.

--version
: siehe unten ...

-V
: Wird eine dieser Optionen in der Befehlszeile gefunden, werden alle davor oder danach folgenden Argumente ignoriert. Die Anwendung gibt ihre eigene Versionsnummer aus und wird beendet.

-P
: Diese Option weist die Anwendung an, während des Einlesens der Eingabe eine visuelle Rückmeldung zu aktivieren.

-T
: Diese Option weist die Anwendung an, während des Einlesens der Eingabe Statistiken zu erfassen und diese nach Abschluss des Einlesevorgangs, jedoch vor Beginn des Prozesses, auszugeben.

-D
: Diese Option weist die Anwendung an, die Protokollierung in der "Safe"-Basis zu aktivieren, um Probleme mit Plugins zu beheben.

-r parser
: siehe unten ...

-rd parser

siehe unten ...

--reader parser
Diese Optionen legen fest, welches Plugin die Anwendung zum Einlesen von *input* verwenden soll. Wenn die Optionen mehrfach angegeben werden, wird die zuletzt angegebene Option verwendet.

-w generator
siehe unten ...

-wr generator
siehe unten ...

--writer generator
Diese Optionen legen fest, welches Plugin die Anwendung zur Erstellung und zum Schreiben von *output* verwenden soll. Wenn die Optionen mehrfach angegeben werden, wird die zuletzt angegebene Option verwendet.

-t process
siehe unten ...

-tr process
siehe unten ...

--transform process
Diese Optionen legen ein Plugin fest, das auf die Eingabe angewendet werden soll. Im Gegensatz zu Lese- und Schreiboperationen überschreibt jede Verwendung die vorherigen Verwendungen *nicht*, sondern fügt jedes ausgewählte Plugin einer Liste von Transformationen hinzu – entweder am Anfang oder am Ende –, je nachdem, welche der beiden Optionen **-p** oder **-a** zuletzt verwendet wurde. Standardmäßig werden die neuen Transformationen angehängt.

-a
siehe unten ...

--append
Diese Optionen signalisieren der Anwendung, dass alle nachfolgenden Transformationen am Ende der Transformationsliste hinzugefügt werden sollen.

-p
siehe unten ...

--prepend
Diese Optionen teilen der Anwendung mit, dass alle nachfolgenden Transformationen am Anfang der Transformationsliste eingefügt werden sollen.

--reset
Diese Option weist die Anwendung an, die Liste der Transformationen zu löschen. Dies ist erforderlich, um die verwendeten Standardtransformationen zu entfernen.

-c file
siehe unten ...

--configuration file
Diese Option bewirkt, dass die Anwendung eine Konfigurationsdatei und/oder ein Plugin lädt. Dabei handelt es sich um ein Plugin, das im Wesentlichen einen vordefinierten Satz von Befehlszeilenoptionen bereitstellt. Diese werden genau so verarbeitet, als wären sie anstelle der Option und ihrer Argumente angegeben worden. Das bedeutet, dass unbekannte

Optionen, die am Anfang der Konfigurationsdatei gefunden werden, dem letzten Plugin zugeordnet werden, selbst wenn dieses Plugin vor der Konfigurationsdatei selbst angegeben wurde. Umgekehrt können unbekannte Optionen, die nach der Konfigurationsdatei folgen, einem in der Datei angegebenen Plugin zugeordnet werden.

Wenn es sich bei dem Argument um eine Datei handelt, die nicht als Plugin geladen werden kann, geht die Anwendung davon aus, dass ihr Inhalt eine Liste von Optionen und deren Argumenten ist, getrennt durch Leerzeichen, Tabulatoren und Zeilenumbrüche. Optionen und Argumente, die Leerzeichen enthalten, können durch doppelte Anführungszeichen (") und einfache Anführungszeichen (') in Anführungszeichen gesetzt werden. Das Anführungszeichen kann innerhalb einer in Anführungszeichen gesetzten Zeichenfolge durch Verdopplung angegeben werden. Zeilenumbrüche in einer in Anführungszeichen gesetzten Zeichenfolge werden unverändert akzeptiert.

Plugins

page nutzt vier verschiedene Arten von Plugins, nämlich: Reader, Writer, Transformationen und Konfigurationen. An dieser Stelle geben wir lediglich eine grundlegende Einleitung in deren Verwendung aus **page**. Die genauen APIs, die den Plugins zur Verfügung gestellt werden und von diesen erwartet werden, finden Sie in der Dokumentation zu **page::pluginmgr** – für diejenigen, die eigene Plugins erstellen möchten.

Plugins werden als Argumente für die Optionen **-r**, **-w**, **-t**, **-c** und deren entsprechende längere Formen festgelegt. Weitere Informationen finden Sie im Abschnitt Optionen.

Jedes dieser Argumente wird zunächst als Name einer Datei behandelt, und diese Datei wird als Plugin geladen. Wenn es jedoch keine Datei mit diesem Namen gibt, wird das Argument in den Namen eines Pakets umgewandelt, und dieses Paket wird anschließend geladen. Für jeden Plugin-Typ durchsucht die Paketverwaltung nicht nur die regulären Pfade, sondern auch eine Reihe von anwendungs- und typ-spezifischen Pfaden. Eine Auflistung aller Pfade und ihrer Quellen finden Sie im Abschnitt Speicherorte für Plugins.

-c *name*

Konfigurationen. Der Name des Pakets für das Plugin *name* lautet "page::config::*name*".

Wir verfügen über ein vordefiniertes Plugin:

peg

Es richtet die Anwendung als Parser-Generator ein, der Parsing-Ausdrucksgrammatiken akzeptiert und einen Packrat-Parser in Tcl erstellt. Die tatsächlich festgelegten Argumente lauten:

```
--reset
--append
--reader    peg
--transform reach
--transform use
--writer    me
```

-r *name*

Sehr geehrte Leserinnen und Leser. Der Name des Pakets für das Plugin *name* lautet

"page::reader::*name*".

Wir verfügen über fünf vordefinierte Plugins:

peg
: Interpretiert die Eingabe als Parsing-Ausdruck-Grammatik (*PEG*) und generiert dafür eine Baumdarstellung. Sowohl die Syntax von PEGs als auch die Struktur der Baumdarstellung werden in den jeweiligen Manpages erläutert.

hb
: Interpretiert die Eingabe als Tcl-Code, wie er vom Writer-Plugin *hb* generiert wird, und erzeugt dessen Baumdarstellung.

ser
: Interpretiert die Eingabe als Serialisierung eines PEG, wie sie vom Writer-Plugin *ser* unter Verwendung des Pakets **grammar::peg** generiert wurde.

lemon
: Interpretiert die Eingabe als Grammatikbeschreibung, wie sie der von Richard Hipp entwickelte Parsergenerator *LEMON* versteht, und erzeugt daraus eine Baumdarstellung. Sowohl die Eingabesyntax als auch die Struktur der Baumdarstellung sind in den jeweiligen Handbuchseiten erklärt.

treeser
: Der Interpreter interpretiert die Eingabe als Serialisierung eines **struct::tree**. Sie wird als solche validiert, jedoch nicht darüber hinaus. Es wird *nicht* davon ausgegangen, dass es sich um die Baumdarstellung einer Grammatik handelt.

-w *name*
: Autoren. Der Name des Pakets für das Plugin *name* lautet "page::writer::*name*".

Wir verfügen über acht vordefinierte Plugins:

identity
: Schreibt die eingehenden Daten einfach unverändert weiter, ohne Änderungen vorzunehmen. Dies eignet sich gut zur Überprüfung des Rohdatenergebnisses eines Lesers oder einer Transformation.

null
: Erzeugt nichts und ignoriert die eingehende Datenstruktur.

tree
: Geht davon aus, dass es sich bei der eingehenden Datenstruktur um ein "**struct::tree**" handelt, und generiert eine eingerückte textuelle Darstellung aller Knoten, ihrer übergeordneten Beziehungen sowie ihrer Attributinformationen.

peg
: Geht davon aus, dass die eingehende Datenstruktur eine Baumdarstellung einer *PEG* oder einer anderen Grammatik ist, und gibt sie als PEG aus. Das Ergebnis ist übersichtlich formatiert und teilweise vereinfacht (Zeichenketten als Zeichenfolgen). Im Wesentlichen handelt es sich um einen "Pretty Printer", der jedoch auch dazu verwendet werden kann, eine kanonische Darstellung der Eingabegrammatik zu erhalten.

tpc
: Geht davon aus, dass es sich bei der eingehenden Datenstruktur um eine Baumdarstellung einer *PEG* oder einer anderen Grammatik handelt, und generiert Tcl-Code, der ein Paket definiert, welches ein **grammar::peg**-Objekt enthält, das die Grammatik umfasst, wenn

dieses in einen Interpreter geladen wird.

hb
Dies ähnelt dem Writer-Plugin *tpc*, schreibt jedoch nur die Anweisungen, die statische Ausdrücke und Grammatikregeln definieren. Der Code, der das Ergebnis zu einem Paket macht, wird weggelassen.

ser
Geht davon aus, dass es sich bei der eingehenden Datenstruktur um eine Baumdarstellung einer *PEG* oder einer anderen Grammatik handelt, führt eine Transformation auf der internen Ebene in ein **grammar::peg**-Objekt durch und schreibt deren Serialisierung aus.

me
Geht davon aus, dass es sich bei der eingehenden Datenstruktur um eine Baumdarstellung einer *PEG* oder einer anderen Grammatik handelt, und generiert Tcl-Code zur Definition eines Pakets, das einen rekursiven Abstiegsparser mit Memoization implementiert, der auf der vom Paket **grammar::mengine** bereitgestellten Match-Engine (ME) basiert.

-t *name*
Transformers. Der Name des Pakets für das Plugin *name* lautet "page::transform::*name*".

Wir verfügen über zwei vordefinierte Plugins:

reach
Es wird davon ausgegangen, dass die eingehende Datenstruktur eine Baumdarstellung einer *PEG* oder einer anderen Grammatik ist. Es wird ermittelt, welche nichtterminalen Symbole und Regeln vom Startsymbol bzw. Ausdruck aus erreichbar sind. Alle nichtterminalen Symbole, die nicht erreicht wurden, werden entfernt.

use
Es wird davon ausgegangen, dass die eingehende Datenstruktur eine Baumdarstellung einer *PEG* oder einer anderen Grammatik ist. Es wird ermittelt, welche nichtterminalen Symbole und Regeln *endliche* Folgen von terminalen Symbolen erzeugen können (im Sinne einer kontextfreien Grammatik). Alle nichtterminalen Symbole, die in diesem Sinne als nicht nützlich erachtet wurden, werden entfernt.

Speicherorte für Plugins

Die anwendungsspezifischen Pfade, die von **page** durchsucht werden, sind entweder folgende oder stammen aus:

1. Das Verzeichnis "*~/.page/plugin*"
2. Die Variable *PAGE_PLUGINS*
3. Der Registrierungseintrag *HKEY_LOCAL_MACHINE\SOFTWARE\PAGE\PLUGINS*
4. Der Registrierungseintrag *HKEY_CURRENT_USER\SOFTWARE\PAGE\PLUGINS*

Die von **page** durchsuchten typspezifischen Pfade sind entweder:

1. Das Verzeichnis "*~/.page/plugin/*"
2. Die Umgebungsvariable *PAGE__PLUGINS*
3. Der Registrierungseintrag *HKEY_LOCAL_MACHINE\SOFTWARE\PAGE\\PLUGINS*
4. Der Registrierungseintrag *HKEY_CURRENT_USER\SOFTWARE\PAGE\\PLUGINS*

Dabei ist der Platzhalter stets einer der unten aufgeführten Werte, in korrekter Großschreibung.

1. reader

2. writer
3. transform
4. config

Die Registrierungseinträge gelten speziell für die Windows(tm)-Plattform; alle anderen Plattformen ignorieren sie.

Der Inhalt sowohl der Umgebungsvariablen als auch der Registrierungseinträge wird als Liste von Pfaden interpretiert, wobei die Elemente entweder durch einen Doppelpunkt (Unix) oder ein Semikolon (Windows) voneinander getrennt sind.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Page-Parsergenerator

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt

Stichwörter: pt, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching , parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt – Anwendung für Parser-Tools

Übersicht

package require Tcl 8.5 9
pt generate *resultformat* ?*options...*? *resultfile inputformat inputfile*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Dokument beschreibt **pt**, die Hauptanwendung des Moduls, einen *Parser-Generator*. Es richtet sich an Personen, die einen Parser für eine von ihnen entwickelte Sprache erstellen möchten. Sollten Sie stattdessen die Anwendung anpassen wollen, finden Sie die grundlegenden Informationen im Abschnitt Interne Abläufe der Anwendung.

Es befindet sich in der Benutzeranwendungsschicht der Parser-Tools.

Bild: arch_user_app

Befehlszeile

pt generate *resultformat ?options...? resultfile inputformat inputfile*

Dieser Unterbefehl der Anwendung liest die in der Datei *inputfile* im Format *inputformat* gespeicherte Parsing-Expression-Grammatik ein, wandelt sie gemäß den vom Benutzer festgelegten (formatspezifischen) Optionen in das *resultformat* um und speichert das Ergebnis in der Datei *resultfile*.

Die Datei *inputfile* muss vorhanden sein, während die Datei *resultfile* angelegt werden kann, wobei ein bereits vorhandener Inhalt überschrieben wird. Fehlende Verzeichnisse im Pfad zur Datei *resultfile* werden ebenfalls angelegt.

Die genaue Form des Ergebnisses sowie die von den bekannten Ergebnisformaten unterstützten Optionen werden in den folgenden Abschnitten dieses Dokuments erläutert; die nachstehende Liste dient als Index, der die Namen der Formate den entsprechenden Abschnitten zuordnet. In alphabetischer Reihenfolge:

c
: Ein *resultformat*. Siehe Abschnitt C-Parser.

container
: Ein *resultformat*. Siehe Abschnitt Grammatik-Container.

critcl
: Ein *resultformat*. Siehe Abschnitt In Tcl eingebetteter C-Parser.

json
: Ein *input*- und *resultformat*. Siehe Abschnitt JSON-Austausch von Grammatiken.

oo
: Ein *resultformat*. Siehe Abschnitt TclOO-Parser.

peg
: Ein *input*- und *resultformat*. Siehe Abschnitt PEG-Spezifikationssprache.

snit
: Ein *resultformat*. Siehe Abschnitt Snit-Parser.

Von den sieben möglichen Ergebnissen sind vier reine Parser (**c**, **critcl**, **oo**, und **snit**), eines (**container**) stellt Code bereit, der in Verbindung mit einem generischen Parser (auch als Grammatik-Interpreter bezeichnet) verwendet werden kann, und die letzten beiden (**json** und **peg**) erfüllen eine Doppelfunktion als Eingabeformate und ermöglichen die Transformation von Grammatiken zum Austausch, zur Neuformatierung und Ähnlichem.

Die erstellten Parser lassen sich in drei Kategorien einteilen:

```
+ --- C ---> critcl, c
|
+ --- specialized -+
|                  |
---+                  + --- Tcl -> snit, oo
|
+ --- interpreted (Tcl) ------> container
```

Specialized parsers implemented in C

Die schnellsten Parser werden bei Verwendung der Ergebnisformate **c** und **critcl** erstellt. Ersteres gibt den rohen C-Code für den Parser zurück, während letzteres diesen mithilfe von *CriTcl* in ein Tcl-Paket einbindet.

Dadurch ist Letzteres wesentlich einfacher zu verwenden als Ersteres. Andererseits lässt sich Ersteres durch eine Vielzahl von Optionen an die Anforderungen der Benutzer anpassen, was beispielsweise die Verwendung des Parsers außerhalb einer Tcl-Umgebung ermöglicht – eine Funktion, die das Format **critcl** nicht unterstützt. Daher ist das Format **c** für fortgeschrittene Benutzer oder Benutzer mit besonderen Anforderungen gedacht.

Ein Nachteil aller Parser in diesem Abschnitt ist die Notwendigkeit, sie über einen C-Compiler zu kompilieren, um sie tatsächlich ausführbar zu machen. Nicht jeder verfügt über die dafür erforderlichen Werkzeuge. Die Parser im nächsten Abschnitt richten sich an Personen, die solchen Einschränkungen unterliegen.

Specialized parsers implemented in Tcl

Da die Parser in diesem Abschnitt in Tcl implementiert sind, arbeiten sie deutlich langsamer als alle Beispiele aus dem vorherigen Abschnitt. Andererseits ermöglicht dies ihren Einsatz in reinen Tcl-Umgebungen oder in Umgebungen, in denen nur eine begrenzte Anzahl von Paketen zulässig ist. Im letzteren Fall ist es von Vorteil, sich für die Einbindung der C-basierten Laufzeitunterstützung (siehe Hinweise unten) in die Umgebung einzusetzen, um die Auswirkungen von Tcl auf die Geschwindigkeit dieser Parser zu verringern.

Die relevanten Formate sind **snit** und **oo**. Beide place ihr Ergebnis in einem Tcl-Paket, das jeweils ein **snit::type** bzw. TclOO **class** enthält.

Von der unterstützenden Laufzeitumgebung, bei der es sich um das Paket **pt::rde** handelt, muss der Benutzer nichts weiter wissen, als dass sie existiert und dass die Parser davon abhängig sind. Kenntnisse über die von der Laufzeitumgebung für die Nutzung durch die Parser exportierte API werden von den Benutzern der Parser *nicht* benötigt.

Interpreted parsing implemented in Tcl

Die letzte Kategorie: die Grammatikinterpretation. Das bedeutet, dass ein Interpreter für Parsing-Ausdruck-Grammatiken die Beschreibung der Grammatik, anhand derer die Eingabe analysiert werden soll, heranzieht und diese als Leitfaden für den Analyse-Prozess verwendet. Dies ist die langsamste der verfügbaren Optionen, da der Interpreter die konfigurierte Grammatik kontinuierlich durchlaufen muss, während die spezialisierten Parser aus den vorangegangenen Abschnitten das relevante Wissen über die Grammatik bereits fest integriert haben.

Der Einsatz der Interpretation ist nur dann sinnvoll, wenn die Grammatik für bestimmte Eingaben vom Benutzer interaktiv geändert werden kann, da die Interpretation nach jeder Änderung eine schnelle Anpassung ermöglicht, während die vorherigen Methoden die Generierung eines völlig neuen Parsers erfordern, was nicht so schnell ist. Ist die zu verwendende Grammatik hingegen feststehend, sind die vorherigen Methoden weitaus vorteilhafter, da die Zeit für die Generierung des Parsers im Vergleich zur Laufzeit des Parser-Codes verschwindend gering ist.

Das entsprechende Ergebnisformat ist **container**. Es generiert (schnell) Grammatikbeschreibungen (anstelle eines vollständigen Parsers), die der vom Grammatik-Interpreter von ParserTools erwarteten API entsprechen. Letzterer wird durch das Paket **pt::peg::interp** bereitgestellt.

Alle von **critcl**, **snit** und **oo** generierten Parser sowie der Grammatik-Interpreter nutzen eine gemeinsame API für den Zugriff auf die eigentliche Parsing-Funktionalität, wodurch sie alle über eine hohe Kompatibilität verfügen. Dies wird im Spezifikationsdokument *Parser-API* beschrieben.

PEG-Spezifikationssprache

peg, eine Sprache zur Spezifikation von Parsing-Expression-Grammatiken, soll sowohl für Menschen lesbar als auch schreibbar sein, dabei jedoch streng genug, um eine maschinelle Verarbeitung zu ermöglichen. Wie jede Computersprache. Sie wurde definiert, um das Verfassen einer Grammatik-Spezifikation zu vereinfachen – etwas, wofür sich die anderen in den Parser-Tools vorhandenen Formate nicht eignen.

Wenn Sie eine Einleitung zur Sprache oder deren formale Spezifikation einsehen möchten, lesen Sie bitte das *PEG-Sprach-Tutorial*.

Wenn dieses Format als Ergebnisformat verwendet wird, unterstützt es die folgenden Optionen:

-file string
: Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
: Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
: Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-template string
: Der Wert dieser Option ist eine Zeichenkette, in die der generierte Text und die Werte der anderen Optionen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mit den unten aufgeführten Platzhaltern angegeben werden. Der Standardwert lautet "**@code@**".

  @user@
  : Ist durch den Wert der Option **-user** zu ersetzen.

  @format@
  : Ist durch die Konstante **PEG** zu ersetzen.

  @file@
  : Ist durch den Wert der Option **-file** zu ersetzen.

  @name@
  : Ist durch den Wert der Option **-name** zu ersetzen.

  @code@
  : Wird durch den generierten Text ersetzt.

JSON-Austausch von Grammatiken

Das Format **json** für Parsing Expression Grammars wurde als Datenaustauschformat geschrieben, das nicht an Tcl gebunden ist. Es wurde definiert, um den Austausch von Grammatiken mit PackRat/PEG-basierten Parsergeneratoren für andere Sprachen zu ermöglichen.

Die formale Spezifikation des JSON-Grammatik-Austauschformats finden Sie in *The JSON Grammar Exchange Format*.

Wenn dieses Format als Ergebnisformat verwendet wird, unterstützt es die folgenden Optionen:

-file string
: Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
: Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
: Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-indented boolean
: Wenn diese Option aktiviert ist, teilt das System das generierte JSON-Objekt in Zeilen auf und rückt es entsprechend seiner inneren Struktur ein, wobei jeder Schlüssel eines Dictionarys in einer eigenen Zeile steht.

  Ist die Option nicht aktiviert (Standardeinstellung), wird das gesamte JSON-Objekt in einer einzigen Zeile ausgegeben, wobei der Abstand zwischen allen Elementen so gering wie möglich gehalten wird.

-aligned boolean
: Wenn diese Option aktiviert ist, sorgt das System dafür, dass die Werte der Schlüssel in einem Dictionary vertikal zueinander ausgerichtet sind, um einen ansprechenden Effekt wie bei einer Tabelle zu erzielen. Damit dies funktioniert, muss zudem die Option **-indented** gesetzt sein.

  Wenn die Option nicht gesetzt ist (Standardeinstellung), wird die Ausgabe gemäß dem Wert von **indented** formatiert, ohne dass versucht wird, die Werte der Dictionary-Schlüssel auszurichten.

In Tcl eingebetteter C-Parser

Das Format **critcl** ist ausführbarer Code, ein Parser für die Grammatik. Es handelt sich um ein Tcl-Paket, dessen eigentliche Parser-Implementierung in C geschrieben und über das Paket **critcl** in Tcl eingebettet ist.

Dieses Ergebnisformat unterstützt die folgenden Optionen:

-file string
: Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet

unknown.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-class string
Der Wert dieser Option ist der Name der zu generierenden Klasse ohne führende Doppelpunkte. Der Standardwert lautet **CLASS**.

Bei einem einfachen Wert wie **X** ohne Doppelpunkte, beispielsweise Klasse, lautet der Parser-Befehl **X**::**X**. Bei einem Wert mit Namespace wie **X::Y** lautet der Parser-Befehl hingegen **X::Y**.

-package string
Der Wert dieser Option ist der Name des zu erstellenden Pakets. Der Standardwert lautet **PACKAGE**.

-version string
Der Wert dieser Option gibt die Version des zu erstellenden Pakets an. Der Standardwert lautet **1**.

C-Parser

Das Format **c** ist ausführbarer Code, nämlich ein Parser für die Grammatik. Die Parser-Implementierung ist in C geschrieben und lässt sich mithilfe einer Vielzahl von Optionen an die Bedürfnisse der Benutzer anpassen.

Das Format **critcl** ist beispielsweise als vordefinierte Konfiguration dieser Optionen auf Basis des Generators für **c** implementiert.

Dieses Ergebnisformat unterstützt die folgenden Optionen:

-file string
Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-template string
Der Wert dieser Option ist eine Zeichenkette, in die der generierte Text und die übrigen Konfigurationseinstellungen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mithilfe der unten aufgeführten Platzhalter angegeben werden. Der Standardwert lautet "**@code@**".
@user@

Ist durch den Wert der Option **-user** zu ersetzen.

@format@
Ist durch die Konstante **C/PARAM** zu ersetzen.

@file@
Ist durch den Wert der Option **-file** zu ersetzen.

@name@
Ist durch den Wert der Option **-name** zu ersetzen.

@code@
Wird durch den generierten Tcl-Code ersetzt.

Die folgenden Optionen sind insofern besonders, als sie im generierten Code vorkommen und dort ebenfalls ersetzt werden.

@statedecl@
Ist durch den Wert der Option **state-decl** zu ersetzen.

@stateref@
Ist durch den Wert der Option **state-ref** zu ersetzen.

@strings@
Ist durch den Wert der Option **string-varname** zu ersetzen.

@self@
Ist durch den Wert der Option **self-command** zu ersetzen.

@def@
Ist durch den Wert der Option **fun-qualifier** zu ersetzen.

@ns@
Ist durch den Wert der Option **namespace** zu ersetzen.

@main@
Ist durch den Wert der Option **main** zu ersetzen.

@prelude@
Ist durch den Wert der Option **prelude** zu ersetzen.

-state-decl string
Eine C-Zeichenkette, die die Argumentdeklaration darstellt, die in den generierten Parsing-Funktionen verwendet werden soll, um auf den Parsing-Zustand zu verweisen. Im Wesentlichen handelt es sich dabei um den Typ und den Namen des Arguments. Der Standardwert ist die Zeichenkette **RDE_PARAM p**.

-state-ref string
Eine C-Zeichenkette mit dem Argumentnamen, mit dem die erzeugten Parsing-Funktionen auf den Parsing-Zustand verweisen. Der Standardwert ist die Zeichenkette **p**.

-self-command string
Eine C-Zeichenkette, die die Referenz angibt, die erforderlich ist, um die generierte Parser-Funktion (Methoden ...) von einer anderen Parser-Funktion aus aufzurufen, entsprechend dem gewählten Rahmen (Template). Der Standardwert ist die leere Zeichenkette.

-fun-qualifier string
Eine C-Zeichenkette mit den Attributen der erzeugten Funktionen (Methoden ...) gemäß dem gewählten Rahmen (Template). Der Standardwert lautet **static**.

-namespace string
Der Name des C-Namespaces, in dem sich die Parser-Funktionen (Methoden usw.) befinden sollen, oder ein allgemeines Präfix, das den Funktionsnamen vorangestellt werden soll. Der Standardwert ist die leere Zeichenkette.

-main string
Der Name der Hauptfunktion (Methode, ...), die vom ausgewählten Rahmen (Template) aufgerufen werden soll, um mit der Analyse der Eingabe zu beginnen. Der Standardwert ist **__main**.

-string-varname string
Der Name der Variablen, die für die Tabelle der Zeichenfolgen verwendet wird, auf die der generierte Parser zurückgreift, d. h. Fehlermeldungen, Symbolnamen usw. Der Standardwert lautet **p_string**.

-prelude string
Ein Codeausschnitt, der am Anfang jeder generierten Parsing-Funktion eingefügt werden soll. Der Standardwert ist die leere Zeichenkette.

-indent integer
Die Anzahl der Zeichen, um die jede Zeile des generierten Codes eingerückt werden soll. Der Standardwert lautet **0**.

-comments boolean
Ein Flag, das die Generierung von Code-Kommentaren steuert, die den ursprünglichen Parsing-Ausdruck enthalten, für den eine Parsing-Funktion vorgesehen ist. Der Standardwert lautet **on**.

Snit-Parser

Das Format **snit** ist ausführbarer Code, ein Parser für die Grammatik. Es handelt sich um ein Tcl-Paket, das ein **snit::type** enthält, d. h. eine Klasse, deren Instanzen Parser für die Eingabegrammatik sind.

Dieses Ergebnisformat unterstützt die folgenden Optionen:

-file string
Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-class string
Der Wert dieser Option ist der Name der zu generierenden Klasse ohne führende Doppelpunkte. Hinweis: Dieser Wert dient gleichzeitig als Name des zu generierenden Pakets, wenn die Option **-package** nicht festgelegt ist (siehe unten). Der Standardwert lautet **CLASS** und gilt, wenn weder die Option **-class** noch die Option **-package** festgelegt wurden.

-package string

Der Wert dieser Option ist der Name des zu generierenden Pakets ohne führende Doppelpunkte. Hinweis: Dieser Wert dient gleichzeitig als Name der zu generierenden Klasse, wenn die Option **-class** nicht festgelegt wurde (siehe oben). Der Standardwert lautet **PACKAGE** und gilt, wenn weder die Option **-package** noch die Option **-class** festgelegt wurden.

-version string
Der Wert dieser Option gibt die Version des zu erstellenden Pakets an. Der Standardwert lautet **1**.

TclOO-Parser

Das Format **oo** ist ausführbarer Code, ein Parser für die Grammatik. Es handelt sich um ein Tcl-Paket, das eine Klasse **TclOO** enthält, deren Instanzen Parser für die Eingabegrammatik sind.

Dieses Ergebnisformat unterstützt die folgenden Optionen:

-file string
Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-class string
Der Wert dieser Option ist der Name der zu generierenden Klasse ohne führende Doppelpunkte. Hinweis: Dieser Wert dient gleichzeitig als Name des zu generierenden Pakets, wenn die Option **-package** nicht festgelegt ist (siehe unten). Der Standardwert lautet **CLASS** und gilt, wenn weder die Option **-class** noch die Option **-package** festgelegt wurden.

-package string
Der Wert dieser Option ist der Name des zu generierenden Pakets ohne führende Doppelpunkte. Hinweis: Dieser Wert dient gleichzeitig als Name der zu generierenden Klasse, wenn die Option **-class** nicht festgelegt wurde (siehe oben). Der Standardwert lautet **PACKAGE** und gilt, wenn weder die Option **-package** noch die Option **-class** festgelegt wurden.

-version string
Der Wert dieser Option gibt die Version des zu erstellenden Pakets an. Der Standardwert lautet **1**.

Grammatik-Container

Das Format **container** ist eine weitere Form, Parsing Expression Grammars zu beschreiben. Zwar sind Daten in diesem Format ausführbar, sie stellen jedoch keinen Parser für die Grammatik dar. Sie müssen stets in Verbindung mit dem Paket **pt::peg::interp**, einem Grammatik-Interpreter, verwendet werden.

Das Format stellt Grammatiken mittels einer **snit::type** dar, d. h. einer Klasse, deren Instanzen API-kompatibel zu den Instanzen des Pakets **pt::peg::container** sind und die mit der jeweiligen

Grammatik vorbelegt sind.

Dieses Ergebnisformat unterstützt die folgenden Optionen:

-file string
Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-mode bulk|incremental
Der Wert dieser Option legt fest, welche Methoden von **pt::peg::container**-Instanzen zur Festlegung der Grammatik verwendet werden, d. h. zum Vorladen in den Container. Es gibt zwei zulässige Werte, die im Folgenden aufgeführt sind. Der Standardwert ist **bulk**.

bulk
In diesem Modus werden die Methoden **start**, **add**, **modes** und **rules** verwendet, um die Grammatik pauschal festzulegen, d. h. als eine Menge von nichtterminalen Symbolen sowie zwei Dictionarys, die die Symbole ihren semantischen Modi und Parsing-Ausdrücken zuordnen.

Dieser Modus ist der Standardmodus.

incremental
In diesem Modus werden die Methoden **start**, **add**, **mode** und **rule** verwendet, um die Grammatik stückweise festzulegen, wobei jedes Nichtterminal über einen eigenen Block von Definitionsbefehlen verfügt.

-template string
Der Wert dieser Option ist eine Zeichenkette, in die der generierte Code und die übrigen Konfigurationseinstellungen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mithilfe der unten aufgeführten Platzhalter angegeben werden. Der Standardwert lautet "**@code@**".

@user@
Ist durch den Wert der Option **-user** zu ersetzen.

@format@
Ist durch die Konstante **CONTAINER** zu ersetzen.

@file@
Ist durch den Wert der Option **-file** zu ersetzen.

@name@
Ist durch den Wert der Option **-name** zu ersetzen.

@mode@
Ist durch den Wert der Option **-mode** zu ersetzen.

@code@
Wird durch den generierten Code ersetzt.

Beispiel

In diesem Abschnitt arbeiten wir ein vollständiges Beispiel durch, beginnend mit einer PEG-Grammatik und endend mit der Ausführung des daraus generierten Parsers auf einer Eingabe, wobei wir der in der folgenden Abbildung dargestellten Gliederung folgen:

Bild: flow

Unsere Grammatik, von der angenommen wird, dass sie in der Datei "*calculator.peg*" gespeichert ist, lautet wie folgt:

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Daraus erstellen wir einen SNIT-basierten Parser mithilfe von

```
pt generate snit calculator.tcl -class calculator -name calculator peg
    calculator.peg
```

Damit liegen uns nun das Parser-Paket und die Klasse vor, die in die Datei "*calculator.tcl*" geschrieben wurden. Unter der Annahme, dass dieses Paket anschließend ordnungsgemäß an einem Ort installiert wird, an dem Tcl es finden kann, können wir diese Klasse nun über ein Skript wie das folgende verwenden:

```
    package require calculator

    lassign $argv input
    set channel [open $input r]

    set parser [calculator]
    set ast [$parser parse $channel]
    $parser destroy
    close $channel

    ... now process the returned abstract syntax tree ...
```

wobei der in der Variable gespeicherte abstrakte Syntaxbaum wie folgt aussehen wird:

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
```

```
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

unter der Annahme, dass die Eingabedatei und der Kanal den Text enthielten

```
 120+5
```

Eine anschaulichere Darstellung des Baums wäre

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                          |            |
|                          +- Digit 2 2 | 0
|                                       |
Expression 0 4 --- Factor 0 4 -+------------------------------ AddOp 3
    3 | +
|                                       |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

Unabhängig davon liegt es an dieser Stelle in der Verantwortung der Benutzerin, mit dem Baum zu arbeiten, um das von ihr angestrebte Ziel zu erreichen. Das heißt, ihn zu analysieren, durch eine Transformation umzuwandeln usw. Das Paket **pt::ast** dürfte dabei hilfreich sein, da es Befehle bereitstellt, mit denen sich solche AST-Strukturen auf verschiedene Weise durchlaufen lassen.

Wichtig ist: Die hier verwendeten Parser geben eine Datenstruktur zurück, die die Struktur der Eingabe gemäß der dem Parser zugrunde liegenden Grammatik abbildet. Während des Parsing-Prozesses gibt es *keine* Rückruffunktionen, d. h. keine *Parsing-Aktionen*, wie sie bei den meisten anderen Parsern zu finden sind.

Kehren wir zum letzten Codeausschnitt zurück, in dem der Parser für eine bestimmte Eingabe ausgeführt wird: Beachten Sie, wie die Parser-Instanz der festgelegten *Parser-API* folgt.

Interne Abläufe

Dieser Abschnitt richtet sich an Nutzer der Anwendung, die diese ändern oder erweitern möchten. Nutzer, die sich ausschließlich für die Erstellung von Parsern interessieren, können diesen Abschnitt überspringen.

Die Hauptfunktionalität der Anwendung ist im Paket **pt::pgen** zusammengefasst. Bitte lesen Sie dieses Paket, um weitere Informationen zu erhalten.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcldocstrip

Stichwörter: tcldocstrip, .dtx, LaTeX, conversion, docstrip, documentation, literate programming, markup, source

Name

tcldocstrip – Tcl-basierter Docstrip-Prozessor

Übersicht

tcldocstrip *output* ?*options*? *input* ?*guards*?
tcldocstrip ?*options*? *output* (?*options*? *input guards*)...
tcldocstrip -guards *input*

Beschreibung

Die in diesem Dokument beschriebene Anwendung, **tcldocstrip**, ist eine Weiterentwicklung von **docstrip**, einem einfachen Werkzeug zur literaten Programmierung für LaTeX.

tcldocstrip basiert auf dem Paket **docstrip**.

Anwendungsfälle

tcldocstrip wurde unter Berücksichtigung der folgenden drei Anwendungsfälle entwickelt.

1. Konvertierung einer einzelnen Eingabedatei gemäß den aufgeführten Bedingungen in die bereinigte Ausgabe. Hiermit wird der einfachste Fall behandelt, bei dem eine Reihe von Bedingungen ein einzelnes Dokument in einer einzigen Eingabedatei festlegen.
2. Das Zusammenfügen, d. h. die Zusammenstellung einer Ausgabe aus mehreren Schutzgruppen in einer bestimmten Reihenfolge und möglicherweise aus verschiedenen Dateien. Dies ist der zweite häufige Fall. Ein Dokument, das sich über mehrere Eingaben und/oder über verschiedene Schutzgruppen erstreckt.
3. Extraktion und Auflistung aller eindeutigen Ausdrücke und Schutzmechanismen, die innerhalb eines Dokuments verwendet werden, um einer Person, die das betreffende Dokument nicht verfasst hat, dabei zu helfen, sich mit diesem vertraut zu machen.

Befehlszeile

tcldocstrip *output* ?options? *input* ?*guards*?

Dies ist die Form für den Anwendungsfall [1]. Sie wandelt die Datei *input* gemäß den angegebenen *guards* und Optionen um. Das Ergebnis wird in die genannte Datei *output* geschrieben. Die Verwendung der Zeichenfolge **-** als Name der Ausgabedatei signalisiert, dass das Ergebnis in die Datei **stdout** geschrieben werden soll. Die Guards sind dokumentspezifisch und müssen dem Aufrufer bekannt sein. Die *options* werden später im Abschnitt Optionen erläutert.

path *output* (in)

Dieses Argument legt fest, wohin das generierte Dokument geschrieben werden soll. Es kann sich dabei um den Pfad zu einer Datei oder einem Verzeichnis handeln oder um "**-**". Der letzte Wert bewirkt, dass die Anwendung das generierte Dokument in "**stdout**" speichert.

Wenn der *output* nicht existiert, muss "[file dirname $output]" vorhanden sein und es muss sich um ein beschreibbares Verzeichnis handeln.

path *inputfile* (in)

Dieses Argument legt den Pfad zu der zu verarbeitenden Datei fest. Die Datei muss vorhanden und lesbar sein sowie im *docstrip*-Format vorliegen.

tcldocstrip ?options? *output* (?options? *input guards*)...

Dies ist die Form für den Anwendungsfall [2]. Sie unterscheidet sich von der Form für den Anwendungsfall [1] dadurch, dass Optionen vor der Ausgabedatei angegeben werden können, die allgemein gelten, und dass mehr als eine Eingabedatei angegeben werden kann, wobei jede über eigene, eingabespezifische Optionen und Guards verfügt.

Sie extrahiert Daten aus den verschiedenen Dateien *input* gemäß den angegebenen *options* und *guards* und schreibt das Ergebnis in die angegebene Datei *output*, und zwar in der Reihenfolge, in der sie in der Befehlszeile angegeben wurden. Vor der Ausgabedatei angegebene Optionen sind globale Einstellungen, während die vor jeder Eingabedatei angegebenen Optionen nur für diese jeweilige Eingabedatei gelten. Nicht angegebene Werte werden aus den globalen Einstellungen oder den Standardwerten übernommen. Bei der Form [1] führt die Verwendung der Zeichenfolge **-** als Ausgabe dazu, dass die Anwendung auf stdout schreibt. Die Verwendung der Zeichenfolge**.** für eine Eingabedatei signalisiert, dass die letzte Eingabedatei erneut verwendet werden soll. Dies ermöglicht die Zusammenstellung der Ausgabe aus einer Eingabedatei unter Verwendung mehrerer unterschiedlicher Schutzsätze, ohne dass jedes Mal der vollständige Name der Datei angegeben werden muss.

tcldocstrip -guards *input*

Dies ist die Form für den Anwendungsfall [3]. Es ermittelt die Guard-Bedingungen sowie die eindeutigen Guard-Ausdrücke, die im angegebenen *input*-Dokument verwendet werden. Die gefundenen Zeichenfolgen werden in die Standardausgabe geschrieben, wobei pro Zeile eine Zeichenfolge ausgegeben wird.

Optionen

In diesem Abschnitt werden alle Optionen beschrieben, die dem Benutzer der Anwendung zur Verfügung stehen, mit Ausnahme der Option **-guards**. Diese Option wurde bereits im Abschnitt

Befehlszeile beschrieben.

-metaprefix string
Diese Option wird vom Befehl **docstrip::extract** übernommen, der vom Paket **docstrip** bereitgestellt wird.

Sie legt die Zeichenkette fest, durch die das Präfix "%%" einer Metakommentarzeile ersetzt wird. Der Standardwert ist "%%". Bei Tcl-Code wäre dies typischerweise "#".

-onerror mode
Diese Option wird vom Befehl **docstrip::extract** übernommen, der im Paket **docstrip** bereitgestellt wird.

Sie legt fest, welche Maßnahme ergriffen wird, wenn ein Formatfehler im zu verarbeitenden *text* erkannt wird. Die Einstellungsmöglichkeiten sind:

ignore
Ignoriert den Fehler und fährt fort, als wäre nichts geschehen.

puts
Schreibt eine Fehlermeldung nach **stderr** und setzt die Verarbeitung fort.

throw
Es wird ein Fehler ausgelöst. **::errorCode** ist auf eine Liste gesetzt, deren erstes Element **DOCSTRIP** ist, deren zweites Element die Fehlerart angibt und deren drittes Element die Zeilennummer ist, in der der Fehler festgestellt wurde. Dies ist die Standardeinstellung.

-trimlines bool
Diese Option wird vom Befehl **docstrip::extract** übernommen, der vom Paket **docstrip** bereitgestellt wird.

Legt fest, ob *Leerzeichen* am Zeilenende vor dem Prozess der Zeile entfernt werden sollen. Der Standardwert ist **true**.

-preamble text
siehe unten ...

-postamble text
siehe unten ...

-nopreamble
siehe unten ...

-nopostamble
Die Optionen -no*amble deaktivieren die Vor- und Nachschriften der Datei vollständig, während die Optionen -*amble den *user*-Teil der Vor- und Nachschriften der Datei festlegen. Dieser Teil kann leer sein; in diesem Fall werden nur die Standardteile angezeigt. Dies ist die Standardeinstellung.

Sind Präambeln aktiviert, werden sie vor dem eigentlichen Inhalt einer generierten Datei eingefügt. Ebenso werden Postambeln, sofern sie aktiviert sind, nach dem eigentlichen Inhalt einer generierten Datei eingefügt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden

oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *docstrip* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright
```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

devdoc

tcl_community_communication
Stichwörter: tcl_community_communication

Name
tcl_community_communication – Tcl-Community – Höflicher Umgang miteinander

Beschreibung
Die Tcl-Community begrüßt Beiträge von allen, die die Entwicklung folgender Bereiche vorantreiben möchten:

- Die Sprache Tcl
- Von Tcl abgeleitete Sprachen
- Tcl-bezogene Bibliotheken
- Tcl-Erweiterungen
- Externe Projekte, die Tcl integrieren:

Wir begrüßen Beiträge von jedermann. Wir machen keinen Unterschied hinsichtlich Geschlecht, ethnischer Zugehörigkeit, Religion, kulturellem Hintergrund, kybernetischer Natur und sonstigen demografischen Merkmalen sowie persönlicher politischer Ansichten.

Eine Gemeinschaft steht und fällt mit der Kommunikation. Und gelegentlich ist unsere Kommunikation mit Mustern gespickt, die harsch, unfreundlich, abweisend und/oder auf andere Weise unfreundlich sind. Als ehrenamtliche Gemeinschaft sind wir auf jede Hilfe angewiesen, die wir bekommen können. Daher bitten wir alle Mitwirkenden, sich in den Diskussionen der Tcl-Community bewusst darum zu bemühen, auf eine Weise zu kommunizieren, die einladend ist. Auf eine Weise, die freundlich ist. Auf eine Weise, die, mit einem Wort: gütig ist.

Diese Leitlinien enthalten konkrete Vorschläge, wie dieses Ziel erreicht werden kann.

Bitte beachten Sie: Im weiteren Verlauf dieses Dokuments können sich alle Verweise auf "Menschen", "Personen", "jeder" oder "jemand" auf jedes empfindungsfähige Wesen beziehen, nicht nur auf körperliche Mitglieder der Spezies Homo sapiens.

Wir sind ein Zufluchtsort, kein Clubhaus

Die Tcl-Community ist ein Zusammenschluss von Amateuren und Profis, die Tools programmieren, testen und nutzen. Unsere Community steht allen offen. Es gibt keine Absperrseile. Es gibt keinen Türsteher. Es gibt keine geheimen Handzeichen. Jedes empfindungsfähige Wesen, das sich in unsere Mitte begibt, ist willkommen. Sollte jemand jemals gebeten werden, die Gemeinschaft zu verlassen, so geschieht dies ausschließlich, weil er oder sie den reibungslosen Ablauf der Gemeinschaft stört.

Wir bewerten Ideen, nicht Personen

Eine gute Idee kann von jedem kommen, ganz gleich, wie kurz er schon bei uns ist. Eine schlechte Idee kann von jedem kommen, ganz gleich, wie lange oder wie kurz er schon bei uns ist. Wir beurteilen ein Konzept danach, wie es einer genauen Prüfung hinsichtlich Logik, Umsetzung und Regressionstests standhält. Wir beurteilen Ideen nicht danach, wer die Idee zuerst hatte, wer der Idee zustimmt oder wer sie ablehnt.

Allen mit Respekt begegnen

Jeder verdient jederzeit Respekt und Höflichkeit.

Personen mit den Namen ansprechen, die sie selbst verwenden.

Wenn Sie aus grammatikalischen Anforderungen das Geschlecht einer Person angeben müssen, respektieren Sie bitte deren Präferenzen hinsichtlich ihrer Geschlechtsidentität. Sollten Sie sich hinsichtlich des Geschlechts einer Person unsicher sein, fragen Sie nach. Sollte jemand Ihr Geschlecht erraten müssen und dabei falsch liegen, korrigieren Sie die Person bitte und nehmen Sie dies nicht persönlich.

Gegenüber anderen Teilnehmern keinen harschen Ton anschlagen.

Bitte führen Sie keine persönlichen Angriffe gegen irgendjemanden durch (unabhängig davon, ob es sich um einen Teilnehmer handelt oder nicht).

Kritisieren Sie Anweisungen und Handlungen, niemals Personen.

Nichts persönlich nehmen

Im Zweifelsfall sollten Sie von den besten Absichten der Menschen ausgehen. Kritik an Ihren Anweisungen ist kein persönlicher Angriff auf Sie.

Personen, nicht Menschengruppen

Stereotypen sind in vielerlei Hinsicht ein wenig hilfreiches Mittel. Sie sind in der Regel zu stark vereinfacht. Meistens sind sie schlichtweg falsch. Und selbst wenn sie "richtig" wären, sind sie absolut nicht geeignet, um die Fähigkeiten, Motivationen oder Eignung einer Person zu beurteilen.

Bitte verwenden Sie sie nicht in der Kommunikation innerhalb der Tcl-Community.

Fehler passieren

Das menschliche Dasein besteht aus einer Abfolge von Versuchen und Fehlern. Fortschritt bedeutet, dass wir einen Versuch mehr haben als Fehler. Sich zu irren oder einen Fehler zu begehen, ist der Normalzustand des Menschen. Akzeptieren Sie die Fehler Ihrer Mitmenschen und seien Sie sich bewusst, dass auch Sie fehlbar sind.

Bei dem bleiben, was gesagt wurde

Bitte reagieren Sie auf das, was die Menschen tatsächlich sagen. Wir sind alle einzigartige Individuen, aber keiner von uns kann Gedanken lesen. Wenn Sie feststellen, dass Sie auf das reagieren, was Sie sich vorstellen, was jemand denkt, ist die Wahrscheinlichkeit groß, dass Sie sich irren.

Wenn Sie jemanden kritisieren müssen, beschränken Sie sich auf Dinge, die diese Person tatsächlich getan hat. Kritisieren Sie niemals etwas, von dem Sie vermuten, dass die Person es getan hat. Oder sich vorstellen, dass sie es getan hat. Oder etwas, das jemand, der bestimmte Eigenschaften mit ihr teilt, in der Vergangenheit getan hat.

Beschränken Sie Diskussionen über Themen, die nicht mit Tcl zu tun haben, auf das, was sachlich und ohne Emotionen oder Wertungen dargelegt werden kann.

Wenn es schwierig wird, nicht eskalieren

Sollten Sie das Gefühl haben, persönlich angegriffen oder beleidigt zu werden, verhalten Sie sich großmütig. In einem öffentlichen Forum zurückzuschlagen, wird die Situation nur verschlimmern. Sprechen Sie die Angelegenheit in einem privaten Schriftwechsel an. Seien Sie höflich. Bringen Sie Ihre Gefühle zum Ausdruck, geben Sie jedoch einen Hinweis darauf, dass es sich um Ihre Gefühle handelt. Suchen Sie beim Schreiben nach einer Möglichkeit, die Wogen zu glätten. Und wenn Sie Zweifel haben, lassen Sie Ihren Brief erst einmal über Nacht ruhen, bevor Sie auf "Senden" klicken. Und selbst wenn Sie keine Zweifel haben, lassen Sie ihn danach noch einen weiteren Tag ruhen.

Wenn Sie Zeuge eines laufenden Streits sind, bitten Sie die beiden Parteien höflich, die Angelegenheit in einem privateren Rahmen zu klären.

Das letzte Wort ist: Es tut mir leid

Sollte es doch einmal zu einem persönlichen Argument kommen, entschuldigen Sie sich als Erster. Eine Entschuldigung bedeutet kein Eingeständnis in einer logischen Frage. Sie räumt lediglich ein, dass die Diskussion an einem bestimmten Punkt entweder die Logik oder die Anstandsregeln der Gemeinschaft – oder beides – außer Acht gelassen hat. Kehren Sie zum Thema zurück, sobald sich die Gemüter wieder beruhigt haben.

Niemand führt Buch

Es gibt keinen Preis dafür, Recht zu haben. Es entstehen keine Kosten, wenn man Unrecht hat. Eine aufdringliche Verkaufsmasche wird Ihre Idee nicht weiterbringen – genauso wenig wie ein logisches Argument. Sie bewerben sich nicht um ein politisches Amt. Dies ist kein Debattierclub. Wenn Sie feststellen, dass Sie eine Diskussion über den Punkt hinaus fortsetzen, an dem ein Thema noch logisch erörtert werden kann, hören Sie auf.

Kein Missionieren

Die Tcl-Community ist kein Ort, um für Ihr bevorzugtes Betriebssystem, Ihre politische Einstellung, Ihre Religion, Ihr Marketingkonzept oder Ihr Wirtschaftsmodell zu werben. Punkt.

(Und sollten Sie diese Themen dennoch zur Sprache bringen, müssen Sie damit rechnen, dass Ihr gewähltes Thema logisch diskutiert wird – und die Chancen stehen gut, dass dies nicht gerade positiv ausfällt.)

Die Gemeinschaft respektieren

Wenn die Gemeinschaft eine Entscheidung über das weitere Vorgehen getroffen hat, hören Sie bitte auf zu streiten.

Wenn sich jemand darüber beschwert, wie Sie Ihren Ausdruck für Ihre Ideen verwenden, hören Sie zu.

> Wenn Ihre Worte andere verletzen, hören Sie damit auf. Es gibt kein Maß an "Recht haben", das es wiedergutmachen könnte, wenn jemand unsere Gemeinschaft verlässt, weil er sich beleidigt, bedroht oder ignoriert gefühlt hat.

Indem wir diese Richtlinien befolgen, stärken wir unsere Gemeinschaft, fördern mehr Engagement für unsere Projekte, und unsere Diskussionen werden freundlicher verlaufen und leichter zu Ergebnissen führen.

Vielen Dank.

Unterzeichner

- Sean "the Hypnotoad" Woods
- Andreas Kupries

Autoren

Hauptautor
Sean "the Hypnotoad" Woods

Leichte Bearbeitung
Andreas Kupries

tcllib_devguide

Stichwörter: tcllib_devguide

Name

tcllib_devguide – Tcllib – Das Entwicklerhandbuch

Übersicht

Module *name code-action doc-action example-action*
Application *name*
Exclude *name*

Beschreibung

Willkommen bei Tcllib, der Tcl-Standardbibliothek. Beachten Sie, dass Tcllib selbst kein Paket ist. Es handelt sich um eine Sammlung von (teilweise unabhängigen) *Tcl*-Paketen, die Hilfsfunktionen bereitstellen, die für eine große Zahl von Tcl-Programmierern nützlich sind.

Dieses Dokument dient als Leitfaden für Entwickler, die an Tcllib arbeiten, d. h. für Betreuer, die Fehler beheben, den Funktionsumfang der Bibliothek erweitern usw.

Bitte lesen Sie

1. *Tcllib – So erhalten Sie die Quelltexte* und
2. *Tcllib – Das Installationshandbuch*

zunächst, wenn Sie dies noch nicht getan haben.

Wir gehen hier davon aus, dass die Quelltexte bereits in einem Verzeichnis Ihrer Wahl verfügbar sind und dass Sie nicht nur wissen, wie man sie kompiliert und installiert, sondern auch über alle notwendigen Voraussetzungen verfügen, um dies tatsächlich zu tun. Insbesondere die Anleitung zu den Quelltexten erläutert auch, welches Quellcode-Verwaltungssystem verwendet wird, wo Sie es finden, wie Sie es einrichten usw.

Verpflichtungen

Mitwirkender

Als Mitwirkender bei Tcllib verpflichten Sie sich zu Folgendem:

1. Behalten Sie die in *Tcl Community – Freundliche Kommunikation* festgehaltenen Richtlinien im Hinterkopf. Der wichtigste Punkt, den Sie daraus mitnehmen sollten, ist, *freundlich miteinander umzugehen*.
2. Ihre Beiträge werden unter einer BSD-/MIT-Lizenz veröffentlicht. Einzelheiten finden Sie unter *Tcllib – Lizenz*.

Beiträge können Sie leisten, indem Sie Tickets in unseren Tracker eingeben, Patches, Code-Bundles oder Code-Zweige zur Einbindung bereitstellen oder Beiträge in den Tcllib-bezogenen Mailinglisten veröffentlichen.

Betreuer

Wenn Sie ein oder mehrere Pakete zur vollständigen Aufnahme in Tcllib beisteuern, verpflichten Sie sich dazu,

1. Behalten Sie die in *Tcl Community – Freundliche Kommunikation* (wie jeder Mitwirkende) festgehaltenen Richtlinien im Hinterkopf. Der wichtigste Punkt, den Sie daraus mitnehmen sollten, ist, *freundlich miteinander umzugehen*.
2. Ihre Pakete werden unter einer BSD-/MIT-Lizenz verbreitet. Einzelheiten finden Sie unter *Tcllib – Lizenz*
3. Wartung der neuen Pakete über einen Zeitraum von zwei Jahren gemäß den folgenden Regeln und Zuständigkeiten:
   1. Ein Betreuer kann nach Ablauf der vorgeschriebenen Frist nach eigenem Ermessen zurücktreten.
   2. Ein Betreuer kann vor Ablauf der vorgeschriebenen Amtszeit zurücktreten, sofern ein Nachfolger sofort zur Verfügung steht und sich bereit erklärt hat, die verbleibende Amtszeit sowie seine eigene vorgeschriebene Amtszeit (siehe unten) zu übernehmen.
   3. Wenn Sie Ihr Amt niederlegen, ohne dass ein Nachfolger die Betreuung übernimmt, müssen die betreffenden Pakete mit dem Kennzeichen **unmaintained** versehen werden.
   4. Wenn für ein Paket ein Ersatzbetreuer hinzugezogen wird, wird dieses (erneut) mit der Kennzeichnung **maintained** versehen.

      Ein Ersatzbetreuer unterliegt denselben Regeln wie der ursprüngliche Betreuer, mit der Ausnahme, dass die vorgeschriebene Betreuungsdauer auf ein Jahr verkürzt wird.
   5. Für jedes **unmaintained**-Paket kann ein Mitwirkender, der daran interessiert ist, dessen Betreuer zu werden, diese Rolle übernehmen, indem er das Paket unter Angabe seines Namens und seiner Kontaktdaten als **maintained** kennzeichnet und sich zur Einhaltung der Regeln für einen neuen Betreuer verpflichtet (siehe vorheriger Punkt).
   6. Bei jedem bereits vorhandenen **maintained**-Paket kann ein Mitwirkender, der daran interessiert ist, Mitbetreuer zu werden, diese Rolle mit Zustimmung des bzw. der bestehenden Betreuer übernehmen, wobei er sich zur Einhaltung der Regeln für einen neuen Betreuer verpflichtet (siehe die beiden vorangegangenen Punkte).

```
Zu den Aufgaben eines Betreuers gehören:

1. Beobachtung des Ticket-Trackers von Tcllib hinsichtlich
    Fehlermeldungen, Fehlerbehebungen und Funktionswünschen im
    Zusammenhang mit den neuen Paketen.

2. Prüfung der oben genannten Tickets, deren Ablehnung oder Bearbeitung

3. Abstimmung und Austausch mit dem Einreicher des Tickets während der
    Entwicklung und/oder Umsetzung von Fehlerbehebungen.
```

1. Befolgen Sie die Verzweigungen und Arbeitsablauf dieses Leitfadens.

Verzweigungen und Arbeitsablauf

Paketabhängigkeiten

Was Pakete und die Abhängigkeiten zwischen ihnen betrifft, nimmt Tcllib eine mittlere Position zwischen zwei Extremen ein:

1. Auf der einen Seite eine Gruppe stark miteinander verknüpfter Pakete, die in der Regel von einem einzigen Autor für ein einzelnes Projekt erstellt wurden. Betrachtet man meine (Andreas Kupries) eigenen Arbeiten, so sind Beispiele hierfür Marpa, CRIMP, Kinetcl usw.

   Bei jeder Änderung kümmert sich der Autor des Projekts um alle Anpassungen, die sich aus etwaigen durch diese Änderung verursachten Inkompatibilitäten im System ergeben.
2. Auf der anderen Seite steht die Welt der halbunabhängigen Projekte zahlreicher verschiedener Autoren, in der die Autoren zwar wissen, von welchen Paketen ihre eigenen Entwicklungen abhängen, jedoch in der Regel nicht wissen, wer sonst noch von diesen Paketen abhängig ist.

   Das Beste, was ein Autor tun kann, der eine (inkompatible) Änderung an seinem Projekt vornimmt, ist erstens, solche Änderungen in irgendeiner Form anzukündigen, und zweitens, mithilfe der Versionsverwaltung den Code vor und nach der Änderung zu unterscheiden.

   Es obliegt dann der Community, sich anzupassen, sei es durch die Aktualisierung der eigenen Projekte auf die neue Version oder durch das Festhalten an der alten.

Wie bereits erwähnt, befindet sich Tcllib genau in der Mitte dieses Spektrums.

Während wir als Betreuer nicht alle Nutzer der Tcllib-Pakete kennen können und uns daher in dieser Hinsicht auf die in Punkt 2 oben angesprochenen Mechanismen verlassen müssen, stellen die Abhängigkeiten zwischen den in Tcllib enthaltenen Paketen eine andere Sache dar.

Da wir gemeinsam für die Nutzbarkeit von Tcllib in seiner Gesamtheit gegenüber der Außenwelt verantwortlich sind, ist es unsere Pflicht, auch auf Tcllib-Pakete zu achten, die wir nicht direkt betreuen, wenn diese von Paketen abhängen, für deren Betreuung wir zuständig sind. Dies kann so einfach sein wie die Abstimmung mit den Betreuern der betroffenen Pakete. Es kann aber auch erfordern, dass wir entscheiden, wie wir betroffene Pakete ohne Betreuer anpassen – d. h. sie so ändern, dass sie unser geändertes Paket ordnungsgemäß nutzen, oder sie so ändern, dass sie ordnungsgemäß von der unveränderten Version unseres Pakets abhängen.

Hinweise: Das oben Genannte ist nicht nur eine lästige Pflicht, sondern auch eine Chance. Zusätzliche Erkenntnisse lassen sich gewinnen, wenn wir uns zwingen, unser Paket und die geplanten Änderungen aus einer Außenperspektive zu betrachten und die Auswirkungen

unseres Handelns auf andere im Allgemeinen sowie auf abhängige Pakete im Besonderen zu berücksichtigen.

Trunk

Die Verwaltung und Nutzung von Zweigen ist ein wichtiger Bestandteil der Arbeit mit einem *verteilten Versionskontrollsystem* (*DVCS*) wie fossil.

Bei Tcllib ist der Hauptzweig der Sammlung *trunk*. In *git* würde dieser Zweig *master* heißen, und genau dies ist im GitHub-Mirror von Tcllib der Fall.

Um das Debugging angemessen zu unterstützen, *muss jeder einzelne Commit* in diesem Zweig *die gesamte Testsuite* der Sammlung bestehen. Die Verwendung der Bisektionsmethode zur Ermittlung des Zeitpunkts, zu dem ein Problem aufgetreten ist, ist ein Beispiel für eine Maßnahme, die durch diese Constraint vereinfacht wird.

Dies ist Teil unserer gemeinsamen Verantwortung für die allgemeine Nutzbarkeit von Tcllib gegenüber der Außenwelt. Da *fossil* über keinen Mechanismus verfügt, um diese Bedingung durchzusetzen, beruht dies auf dem Vertrauensprinzip seitens der Entwickler und Betreuer.

Um Ihnen die Arbeit zu erleichtern, enthält Tcllib ein Tool ("*sak.tcl*"), das eine Reihe von Befehlen zur Unterstützung bereitstellt. Diese Befehle werden in den folgenden Abschnitten dieses Leitfadens erläutert.

Zwar ist es möglich und zulässig, direkt in den Trunk zu Committen, doch beachten Sie bitte die oben genannte Constraint bezüglich der Testsuite sowie die nachfolgenden Hinweise zu weiteren möglichen Problemen bei einem Commit.

Zweige

Angesichts der Constraints, die für den *trunk*-Zweig des Repositorys gelten, wird (dringend) empfohlen, alle Entwicklungsarbeiten, die über geringfügige Änderungen hinausgehen, in einem anderen Zweig als dem *trunk*-Zweig durchzuführen.

Außerhalb des Trunks ist es Entwicklern gestattet, Zwischenzustände ihrer Arbeit, die noch Fehler enthalten, zu Committen. Erst am Ende eines Entwicklungszyklus, wenn der betreffende Branch als bereit für die Zusammenführung angesehen wird, ist es erforderlich, alle Validierungen durchzuführen, um sicherzustellen, dass die bevorstehende Zusammenführung zu einem fehlerfreien Commit im Trunk führt.

Hinweis: Eine Überprüfung durch einen zweiten Entwickler ist zwar keine zwingende Anforderung für das Zusammenführen eines Zweigs, es ist jedoch empfehlenswert, eine solche unabhängige Meinung einzuholen, um die Arbeit einer Gegenprüfung zu unterziehen.

Außerdem wurde empfohlen, jedem neuen Zweig einen Namen zu geben, der dabei hilft, weitere Details darüber zu ermitteln. Beispiele für geeignete Angaben, die in einen Zweignamen aufgenommen werden sollten, wären

- (Spitz-)Name des Entwicklers
- Ticket-Hash/Referenz
- Ein oder zwei Schlüsselwörter, die auf das Werk zutreffen
- ...

... Darüber hinaus sind die meisten Entwicklungszweige zwar wahrscheinlich von relativ kurzer Dauer, doch gibt es kein Verbot, auch länger bestehende Zweige anzulegen. Die Ersteller sollten sich jedoch bewusst sein, dass solche Zweige umso stärker vom Stammzweig abweichen, je

länger sie ohne Zusammenführung bestehen bleiben, was mit einem entsprechenden Anstieg des Aufwands einhergeht, der entweder für die Zusammenführung aus dem Zweig in den Stammzweig oder umgekehrt aufgewendet werden muss.

Arbeiten mit Zweigen

In der Hoffnung, gute Arbeitspraktiken zu fördern, folgen nun einige Beispiele, die Verzweigungen erzeugen, sowie die dazugehörigen "fossil"-Befehle (Sequenzen).

Awareness

Bei der Entwicklung müssen wir uns stets des Kontexts unserer Arbeit bewusst sein. Auf welchem Zweig befinden wir uns? Welche Dateien haben wir geändert? Welche neuen Dateien sind dem Repository noch nicht bekannt? Was ist auf dem Server geschehen, seit wir unseren Checkout verwendet haben? Die Antworten auf diese Fragen sind besonders wichtig, wenn Sie einen langfristig genutzten Checkout verwenden und nach einer längeren Pause wieder darauf zurückkommen.

Befehle zur Beantwortung solcher Fragen sind:

fossil pull
: Führen Sie alle Änderungen durch, die seit dem letzten Abruf oder der letzten Synchronisierung vom Remote-Repository vorgenommen wurden. Dies muss zuerst erfolgen, bevor Sie einen der folgenden Befehle ausführen.

  Selbst wenn sich der Commit in unserem Checkout auf den gewünschten Zweig bezieht, könnten durch am Remote-Repository vorgenommene Commits die Situation hinter unserem Rücken verändert haben.

fossil info | grep tags
: siehe unten ...

fossil branch list | grep '*'
: Zwei verschiedene Möglichkeiten, den Zweig zu ermitteln, auf dem sich unser Checkout befindet.

fossil timeline
: Was haben wir (und andere) in letzter Zeit unternommen?

  Achtung, diese Informationen sind höchstwahrscheinlich veraltet, und zwar umso mehr, je länger wir diesen Checkout nicht mehr genutzt haben. Führen Sie zunächst **fossil pull** aus, um die aktuellsten Informationen aus dem Remote-Repository des Projekts abzurufen.

fossil timeline current
: Setzt den Commit, auf dem unser Checkout basiert, an den Anfang der Zeitleiste.

fossil changes
: Listet die Dateien auf, die wir im Vergleich zu dem Commit, auf dem der Checkout basiert, geändert haben.

fossil extra
: Listet die Dateien auf, die sich in unserem Checkout befinden, die dem Repository jedoch nicht bekannt sind. Dabei kann es sich um Überbleibsel unserer Arbeit handeln oder um etwas, das wir noch nicht mit **fossil add** in das Repository übernommen haben.

Clean checkouts

Achten Sie darauf, wo Sie sich befinden (siehe erste Definition).

Für nahezu alle der nachfolgend aufgeführten Vorgehensweisen ist ein sauberer Checkout zumindest wünschenswert, oft sogar erforderlich. Um zu überprüfen, ob ein Checkout sauber ist, führen Sie folgenden Befehl aus:

```
fossil changes
fossil extra
```

Wie Sie vorgehen sollten, wenn nicht festgeschriebene Änderungen jeglicher Art festgestellt werden, hängt vom jeweiligen Kontext ab und liegt außerhalb des Geltungsbereichs dieses Leitfadens.

Starting a new branch

Achten Sie darauf, wo Sie sich befinden (siehe erste Definition).

Stellen Sie sicher, dass Ihr Checkout sauber ist (siehe zweite Definition). Dies ist *eine Anforderung*.

In den meisten Fällen sollten Sie sich auf dem Zweig *trunk* befinden und den neuesten Commit dieses Zweigs verwenden. Um dorthin zu gelangen, verwenden Sie

```
fossil pull
fossil update trunk
```

Wenn Sie einen anderen Zweig als Ausgangspunkt für die bevorstehenden Arbeiten wünschen, ersetzen Sie *trunk* in den obigen Befehlen durch den Namen dieses Zweigs.

Nachdem die Basis nun festgelegt ist, stehen uns zwei Möglichkeiten zur Erstellung des neuen Zweigs zur Verfügung, die jeweils unterschiedliche Vor- und Nachteile aufweisen. Der einfachere Weg besteht darin,

```
fossil branch new NAME_OF_NEW_BRANCH
```

und mit der Entwicklung beginnen. Der Vorteil dabei ist, dass Sie nicht vergessen können, den Zweig anzulegen. Die Nachteile bestehen darin, dass wir einen Commit im Zweig haben, der gegenüber dem Ausgangspunkt unverändert ist, und dass wir drastische Maßnahmen wie das Ausblenden oder Ignorieren ergreifen müssen, um den Commit zu entfernen, sollten wir beschließen, die Arbeit vor dem ersten tatsächlichen Commit in diesem Zweig aufzugeben.

Die andere Möglichkeit, den Zweig zu erstellen, besteht darin, mit der Entwicklung zu beginnen und dann beim ersten Commit die Option **--branch** zu verwenden, um **fossil** mitzuteilen, dass wir nun einen Zweig starten. Führen Sie also folgenden Befehl aus:

```
fossil commit --branch NAME_OF_NEW_BRANCH ...
```

wobei… für beliebige weitere Optionen steht, die zur Angabe der Commit-Meldung, der zu übergebenden Dateien usw. verwendet werden.

Die (Vor-) und Nachteile sind nun umgekehrt.

Wir haben keinen überflüssigen Commit, sondern nur das, was tatsächlich entwickelt wurde. Die Arbeit bleibt verborgen, bis wir unseren ersten Commit durchführen.

Es kann vorkommen, dass wir vergessen, **--branch NAME_OF_NEW_BRANCH** zu verwenden, und dieses Versehen dann über die Fossil-Weboberfläche korrigieren müssen (mir sind derzeit keine Möglichkeiten bekannt, dies über die Befehlszeile zu bewerkstelligen, obwohl eine "magische Formel" wie **fossil tag create** möglicherweise funktionieren könnte).

Es ist hilfreich, stets aufmerksam zu bleiben, beispielsweise indem man vor jedem Commit überprüft, ob man sich auf dem gewünschten Branch befindet.

Merging a branch into trunk

Achten Sie darauf, wo Sie sich befinden (siehe erste Definition).

Stellen Sie sicher, dass Ihr Checkout sauber ist (siehe zweite Definition). In der vollständigen Abfolge (Zickzack) ist dies aufgrund der Zusammenführung aus dem Trunk *eine Anforderung*. In der kürzeren Abfolge ist es lediglich wünschenswert. Dennoch halte ich es für eine gute Angewohnheit, den Checkout vor größeren Vorgängen sauber zu halten.

Die vollständige Abfolge mit durchgehenden Überprüfungen dient dazu,

1. Überprüfen Sie den Checkout, d. h. den letzten Commit in Ihrem Branch. Führen Sie die vollständige Testsuite sowie weitere Validierungen durch und beheben Sie alle aufgetretenen Probleme.
2. Fügen Sie den aktuellen Stand des *Trunk* ein (siehe folgende Definition).
3. Führen Sie die Validierung des Checkouts erneut durch. Die eingehenden Änderungen aus dem Trunk könnten inzwischen zu Fehlern geführt haben. Nehmen Sie gegebenenfalls die erforderlichen Korrekturen vor.
4. Führen Sie nun die Zusammenführung in den Trunk durch, indem Sie Folgendes verwenden:

   ```
   fossil update trunk
   fossil merge --integrate YOUR_BRANCH
   ```

5. Zu diesem Zeitpunkt sollte sich der Checkout im gleichen Zustand befinden wie am Ende des obigen Punktes (3), da wir etwaige Probleme mit dem Trunk bereits behoben haben. Daher reicht ein einfaches

   ```
   fossil commit ...
   ```

6. Dies sollte nun ausreichen, um den Merge zurückzuentfernen und den Branch zu schließen (aufgrund des **--integrate**, den wir beim Merge verwendet haben).

   Wer besonders vorsichtig ist, kann den Checkout vor dem Commit ein drittes Mal validieren.

Ich bezeichne dies als *Zick-Zack-Zusammenführung*, da die Pfeile in der Zeitleiste so aussehen: Bei der ersten Zusammenführung verlaufen sie vom Trunk zum Feature-Zweig und bei der abschließenden Zusammenführung wieder zurück.

Wer weniger paranoid ist, kann das tun, was ich als *einfache Zusammenführung* bezeichne, bei der Schritt (2) hinter Schritt (4) verschoben und Schritt (3) vollständig übersprungen wird. Die daraus resultierende kürzere Abfolge lautet

1. Validierung
2. In den Trunk zusammenführen
3. Erneut validieren
4. In den Trunk einpflegen

Der letzte Schritt nach einer Zickzack- oder einer einfachen Zusammenführung besteht darin,

```
fossil sync
```

Dadurch werden unsere Änderungen auf dem Remote-Server gespeichert, und wir erhalten zudem alle weiteren Änderungen, die während unseres Merge-Vorgangs vorgenommen wurden. Dies ermöglicht es uns insbesondere zu überprüfen, ob es zu einem Wettlauf mit einem anderen Entwickler gekommen ist, der zu einer Aufspaltung des Trunks geführt hat.

In einem solchen Fall sollten wir uns mit dem anderen Entwickler abstimmen, wer die Aufspaltung behebt, um sicherzustellen, dass es nicht erneut zu einem Wettlauf kommt.

Merging from trunk

Achten Sie darauf, wo Sie sich befinden (siehe erste Definition).

Stellen Sie sicher, dass Ihr Checkout sauber ist (siehe zweite Definition). Dies ist *eine Anforderung*.

In den meisten Fällen möchten Sie den neuesten Commit des Zweigs *trunk* (oder eines anderen Ursprungs) importieren. Um diesen zu erhalten, verwenden Sie

```
fossil pull
```

Nachdem dies erledigt ist, können wir diesen Commit nun mit folgendem Befehl in unseren aktuellen Zweig importieren:

```
fossil merge trunk
```

Auch wenn **fossil** keine Konflikte meldet, ist es ratsam zu überprüfen, ob der Vorgang die neuen und/oder geänderten Funktionen, an denen wir gerade arbeiten, beeinträchtigt hat.

Nachdem wir eine korrekte Zusammenführung erstellt haben, speichern wir den Zustand mit

```
fossil commit ...
```

bevor die Entwicklung fortgesetzt wird.

Versionsnummern

In Tcllib müssen alle Änderungen an einem Paket mit einer Erhöhung der Versionsnummern einhergehen. Welcher Teil erhöht wird (Patch-Level, Minor- oder Major-Version), hängt von der Art der vorgenommenen Änderung ab. Bei mehreren Änderungen in einem Commit "gewinnt" diejenige mit der höchsten Versionsnummer.

Wenn man in einem Entwicklungszweig arbeitet, kann die Versionsänderung bis zum Zeitpunkt des Zusammenführens aufgeschoben werden; dann muss sie alle Änderungen in diesem Zweig abdecken.

Nachfolgend finden Sie eine Liste der verschiedenen Arten von Änderungen und der damit verbundenen Versionserhöhungen:

D - documentation

Keine Erhöhung

T - testsuite

Keine Erhöhung

B - bugfix
Patch-Level

I - implementation tweak
Patch-Level

P - performance tweak
Patch-Level

E - backward-compatible extension
Minor-Version

API - incompatible change
Major-Version

Beachten Sie, dass ein Commit, das eine Versionserhöhung enthält, in seiner Commit-Meldung sowohl die neue Versionsnummer als auch die Art der Änderung, die dazu geführt hat, angeben muss.

Beachten Sie ferner, dass die Versionsnummer eines Pakets derzeit an drei Stellen vorhanden ist. Bei einer Versionserhöhung müssen alle diese Stellen aktualisiert werden:

1. Die Implementierung des Pakets.
2. Der Paket-Index ("*pkgIndex.tcl*")
3. Die Paketdokumentation.

Der Befehl "*sak.tcl*" **validate version** hilft dabei, Abweichungen zwischen den beiden erstgenannten zu erkennen. Auch alle anderen **validate**-Methoden sind für jeden Entwickler von Interesse. Rufen Sie ihn auf mit

```
sak.tcl help validate
```

um deren Dokumentation einzusehen.

Aufbau im Überblick

Hauptverzeichnisse

Die wichtigsten Verzeichnisse im Toplevel-Fenster von "Tcllib", die für Entwickler von Interesse sind, lauten:

"*modules*"
Jedes Verzeichnis steht für ein oder mehrere Pakete. Im letzteren Fall stehen die Pakete in der Regel in irgendeiner Weise miteinander in Zusammenhang. Beispiele hierfür sind "*base64*", "*math*" und "*struct*", wobei die Beziehungen zwischen den Paketen in dem jeweiligen Verzeichnis von locker (base64) bis eng (math) reichen.

"*apps*"
Dieses Verzeichnis enthält alle installierbaren Anwendungen samt ihrer Dokumentation. Es gibt derzeit keine Unterverzeichnisse in diesem Verzeichnis.

"*examples*"
Jedes Verzeichnis "*foo*" enthält eine oder mehrere Beispiele für die Pakete in "*modules/foo*". Diese Beispiele sind in der Regel noch nicht ausgereift genug, um für eine Installation in Betracht gezogen zu werden.

Weitere Verzeichnisse

"*config*"
Dieses Verzeichnis enthält Dateien zur Unterstützung des Unix-Build-Systems, d. h. "*configure* " und "*Makefile.in*".

"*devdoc*"
Dieses Verzeichnis enthält die DocTools-Quelldateien für die allgemeine Dokumentation, wie beispielsweise dieses Dokument und die dazugehörigen Anleitungen.

"*embedded*"
Dieses Verzeichnis enthält die gesamte Dokumentation, die für *HTML* formatiert und so gestaltet ist, dass sie sich nahtlos in die von Fossil für das Repository generierte Website einfügt.

Es handelt sich hierbei um die Dokumentation, auf die Sie vom Tcllib-Stammverzeichnis aus zugreifen können und die im Repository als "*embedded/index.md*" vorliegt.

"*idoc*"
Dieses Verzeichnis enthält die gesamte Dokumentation, formatiert für *nroff* und *HTML*, wobei letztere ohne jegliche Formatierung vorliegt. Dies ist die Dokumentation, die installiert wird.

"*support*"
Dieses Verzeichnis enthält die Quellcodes der internen Pakete und Dienstprogramme, die bei der Implementierung der Skripte bzw. Tools "*installer.tcl*" und "*sak.tcl*" verwendet werden.

Dateien auf oberster Ebene

"*aclocal.m4*"
siehe unten ...

"*configure*"
siehe unten ...

"*configure.in*"
siehe unten ...

"*Makefile.in*"
Diese vier Dateien bilden das Unix-Build-System, das auf dem Skript "*installer.tcl*" aufbaut.

"*installer.tcl*"
Das auf Tcl basierende Installationsskript bzw. -tool.

"*project.shed*"
Konfiguration für das Build-System **PracTcl** von *Sean Wood*.

"*sak.tcl*"
Dies ist das wichtigste Werkzeug für Entwickler und Release-Manager, das *Schweizer Taschenmesser* für Verwaltungsaufgaben im Zusammenhang mit der Sammlung.

"*ChangeLog*"
Das Protokoll der Änderungen am globalen Support, wenn die Quelltexte noch in *CVS* verwaltet wurden. Mit der Umstellung auf das SCM *fossil* nicht mehr relevant.

"*license.terms*"
Die Lizenz im einfachen ASCII-Format. Siehe auch *Tcllib – Lizenz* für die übersichtliche Darstellung. Der Text ist identisch.

"*README.md*"
siehe unten ...

"***.github/CONTRIBUTING.md***"
: siehe unten ...

"***.github/ISSUE_TEMPLATE.md***"
: siehe unten ...

"***.github/PULL_REQUEST_TEMPLATE.md***"
: Diese im Markdown-Format verfassten Dokumente werden vom GitHub-Mirror dieser Quellen verwendet und angezeigt, wobei die Nutzer auf die offizielle Website und die Issue-Tracker weitergeleitet werden.

"***DESCRIPTION.txt***"
: siehe unten ...

"***STATUS***"
: siehe unten ...

"***tcllib.spec***"
: siehe unten ...

"***tcllib.tap***"
: siehe unten ...

"***tcllib.yml***"
: ????

Dateitypen

Die gängigsten Dateiformate, geordnet nach Erweiterung, sind:

"***.tcl***"
: Tcl-Code für ein Paket, eine Anwendung oder Beispiele.

"***.man***"
: Mit Doctools formatierte Dokumentation, in der Regel für ein Paket.

"***.test***"
: Testsuite für ein Paket oder einen Teil davon. Basierend auf **tcltest**.

"***.bench***"
: Leistungs-Benchmarks für ein Paket oder einen Teil davon. Basierend auf "*modules/bench*".

"***.pcx***"
: Syntaxregeln für *TclDevKit*s **tclchecker**. Durch die Anwendung dieser Regeln kann der Prüfer die Verwendung von Befehlen eines Tcllib-Pakets **foo** validieren, ohne die "*.tcl*"-Dateien, in denen diese implementiert sind, durchsuchen zu müssen.

Tools für Testsuiten

Die Testsuiten in Tcllib basieren auf dem Tcl-Standardtestpaket **tcltest** sowie auf Dienstprogrammen, die sich im Verzeichnis "*modules/devtools*" befinden.

Die Entwickler von Tcllib rufen die Testsuiten über die Methode **test run** des Tools "*sak.tcl*" auf, während andere Methoden von **test** Verwaltungsfunktionen bereitstellen, beispielsweise das Festlegen einer Liste der zu verwendenden Standard-Tcl-Shells.

Die Testsuiten eines bestimmten Moduls aufrufen

Rufen Sie entweder

```
./sak.tcl test run foo
```

or

```
./sak.tcl test run modules/foo
```

um die Testsuiten aufzurufen, die sich in einem bestimmten Modul "*foo*" befinden.

Die Testsuiten aller Module aufrufen

Rufen Sie das Tool ohne Modulnamen auf, d. h.

```
./sak.tcl test run
```

um die Testsuiten aller Module auszuführen.

Detaillierte Testprotokolle

In allen bisherigen Beispielen gibt der Testausführer eine Kombination aus Fortschrittsanzeige und Testsuite-Protokoll an die Standardausgabe aus, wobei für jedes Modul nur die Tests angezeigt werden, die bestanden oder fehlgeschlagen sind, und am Ende in einer Zusammenfassung angegeben wird, wie viele davon jeweils vorhanden sind.

Um ein detailliertes Protokoll zu erhalten, müssen Sie den Testausführer mit zusätzlichen Optionen aufrufen.

Zum einen:

```
 ./sak.tcl test run --log LOG foo
```

Zwar wird dabei wie zuvor derselbe kurze Protokollauszug auf dem Terminal angezeigt, doch wird zudem ein detailliertes Protokoll in die Datei "*LOG.log*" geschrieben und Auszüge in weitere Dateien ("*LOG.summary*", "*LOG.failures*" usw.) ausgegeben.

Für zwei Personen:

```
./sak.tcl test run -v foo
```

Dadurch wird das detaillierte Protokoll anstelle des Kurzprotokolls in die Standardausgabe geschrieben.

Unabhängig von der Form enthält das detaillierte Protokoll eine Auflistung aller ausgeführten Tests, der fehlgeschlagenen Tests sowie der Art und Weise, wie diese fehlgeschlagen sind (erwartete Ergebnisse im Vergleich zu den tatsächlichen Ergebnissen).

Auswahl der Shell

Standardmäßig verwendet der Testausführer alle über **test add** festgelegten Tcl-Shells, um die festgelegten Testsuiten aufzurufen, sofern solche vorhanden sind. Wenn keine angegeben sind, greift er auf die Tcl-Shell zurück, die zur Ausführung des Tools selbst verwendet wird.

Verwenden Sie die Option **--shell**, um die zu verwendende Tcl-Shell explizit anzugeben, beispielsweise

```
./sak.tcl test run --shell /path/to/tclsh ...
```

Hilfe

Rufen Sie das Tool wie folgt auf:

```
./sak.tcl help test
```

um die ausführliche Hilfe zu allen Methoden von **test** sowie die zugehörigen Optionen einzusehen.

Dokumentationswerkzeuge

Das Standardformat für die Dokumentation von Paketen und anderen Elementen in Tcllib ist *doctools*. Die dazugehörigen Pakete sind Teil von Tcllib; siehe die Verzeichnisse "*modules/doctools*" und "*modules/dtplite*". Letzteres ist ein Paket, wobei die eigentliche Anwendung "*apps/dtplite*" einen schlanken Wrapper darum bildet.

Tcllib-Entwickler erhalten über die Methode **doc** des Tools "*sak.tcl*" Zugriff darauf; dabei handelt es sich um einen weiteren (internen) Wrapper um das Paket "*modules/dtplite*".

Dokumentation für ein bestimmtes Modul erstellen

Rufen Sie entweder

```
./sak.tcl doc html foo
```

or

```
./sak.tcl doc html modules/foo
```

um HTML für die Dokumentation im Modul "*foo*" zu generieren. Anstelle von **html** kann hier natürlich jedes andere unterstützte Format verwendet werden.

Die generierte, formatierte Dokumentation wird im Verzeichnis "*doc*" im aktuellen Arbeitsverzeichnis platziert.

Dokumentation für alle Module erstellen

Rufen Sie das Tool ohne Modulnamen auf, d. h.

```
./sak.tcl doc html
```

um HTML-Code für die Dokumentation aller Module zu generieren. Anstelle von **html** kann hier natürlich jedes andere unterstützte Format verwendet werden.

Die generierte, formatierte Dokumentation wird im Verzeichnis "*doc*" im aktuellen Arbeitsverzeichnis platziert.

Verfügbare Ausgabeformate, Hilfe

Rufen Sie das Tool wie folgt auf:

```
./sak.tcl help doc
```

um die gesamte Liste der unterstützten Ausgabeformate anzuzeigen, die generiert werden können.

Validierung ohne Ausgabe

Hinweis: Bitte beachten Sie das spezielle Format **validate**.

Wenn Sie diesen Wert als Namen des zu generierenden Formats verwenden, wird das Tool dazu veranlasst, lediglich zu prüfen, ob die Dokumentation syntaktisch korrekt ist, ohne dabei eine tatsächliche Ausgabe zu erzeugen.

Rufen Sie es entweder wie folgt auf:

```
./sak.tcl doc validate (modules/)foo
```

oder

```
./sak.tcl doc validate
```

um entweder die Pakete eines bestimmten Moduls oder alle Pakete zu überprüfen.

Hinweise zum Erstellen einer Testsuite

Während in den vorangegangenen Abschnitten die Ausführung der Testsuiten für ein Modul und die darin enthaltenen Pakete behandelt wurde, hat dies keinerlei Bedeutung, wenn das betreffende Modul überhaupt keine Testsuiten enthält.

Dieser Abschnitt bietet eine sehr grundlegende Einleitung zu möglichen Methoden zum Erstellen von Tests und Testsuiten.

Zunächst gibt es sogenannte "Routinetests". Diese dienen dazu, absolut grundlegende Annahmen zu überprüfen, die unter keinen Umständen fehlschlagen dürfen.

Beispielsweise für einen Befehl FOO, der zwei Argumente erwartet: drei Tests, bei denen er mit null, einem bzw. drei Argumenten aufgerufen wird. Dabei wird grundsätzlich geprüft, ob der Befehl fehlschlägt, wenn er nicht genügend oder zu viele Argumente erhält.

Danach folgen die Tests, die anhand unseres Wissens über den Befehl dessen Eigenschaften und Annahmen überprüfen. Einige Beispiele, die auf den im Rahmen des "Google Summer of Code 2009" hinzugefügten Graphoperationen basieren, sind:

- Der Befehl "BellmanFord" in struct::graph::ops nimmt einen *Startknoten* als Argument entgegen, und dieser Knoten sollte ein Knoten des Graphen sein. Dies entspricht einem Testfall, der das Verhalten überprüft, wenn der angegebene Knoten kein Knoten des Graphen ist.

  Dies führt in der Implementierung häufig dazu, dass Code geschrieben wird, der diese Annahme explizit überprüft und einen nachvollziehbaren Fehler auslöst, anstatt zuzulassen, dass der Algorithmus später auf seltsame, nicht deterministische Weise fehlschlägt.

  Es ist jedoch nicht immer möglich, solche Überprüfungen durchzuführen. Das Argument "graph" ist beispielsweise selbst ein Befehl, und obwohl wir erwarten, dass es eine bestimmte Schnittstelle aufweist, d. h. eine Reihe von Unterbefehlen, auch bekannt als Methoden, können wir nicht überprüfen, ob es über diese verfügt, es sei denn, wir versuchen tatsächlich, sie zu verwenden. Dies wird ohnehin vom Algorithmus erledigt, sodass eine explizite Überprüfung lediglich einen Mehraufwand darstellt, auf den wir verzichten können.
- Soweit ich mich erinnere, besteht eines der charakteristischen Merkmale von Bellman-Ford und/oder Johnson darin, dass sie mit negativen Gewichten umgehen können. Dijkstra hingegen erfordert positive Gewichte.

  Daraus ergeben sich (mindestens) drei Testfälle … ein Graph mit ausschließlich positiven Gewichten, einer mit ausschließlich negativen Gewichten und einer mit einer Mischung aus positiven und negativen Gewichten. Wenn man weiter darüber nachdenkt: Kann der Algorithmus auch das Gewicht **0** verarbeiten? Ein weiterer Testfall – oder mehrere –, wenn wir Null mit positiven und negativen Gewichten mischen.
- Bei den beiden Algorithmen, über die wir derzeit nachdenken, geht es um Abstände zwischen Knoten, wobei der Abstand "unendlich" sein kann, d. h., Knoten haben möglicherweise keine Verbindung zueinander. Das bedeutet, dass gute Testfälle folgende sind:

1. Stark verbundener Graph
2. Verbundener Graph
3. Nicht zusammenhängender Graph.

```
An den Extremen der Kategorien "stark verbunden" und "nicht verbunden"
    finden wir die vollständig verbundenen Graphen und Graphen ohne
    Kanten, die nur Knoten enthalten, d. h. vollständig unverbunden
    sind.
```

- Soweit ich mich erinnere, nehmen beide Algorithmen gewichtete Kanten entgegen und füllen einen Standardwert ein, falls Kanten im Eingabegraphen ungewichtet bleiben.

  Daraus ergeben sich zudem drei Testfälle:
  1. Graph mit allen Kanten und expliziten Gewichten.
  2. Ein Graph ohne jegliche Gewichte.
  3. Graph mit einer Mischung aus gewichteten und ungewichteten Graphen.

Was oben anhand von Beispielen beschrieben wurde, wird als *Black-Box*-Test bezeichnet. Testfälle werden auf der Grundlage des Wissens des Entwicklers über die Eigenschaften des Algorithmus und dessen Eingaben entworfen und geschrieben, ohne Bezug auf eine bestimmte Implementierung zu nehmen.

Darüber hinaus bildet das *White-Box*-Testen eine Ergänzung zum *Black-Box*-Testen. Dabei kennen wir die Implementierung des Algorithmus, untersuchen diese und entwerfen unsere Tests so, dass sie den Code durch alle möglichen Pfade der Implementierung führen. Überall dort, wo eine Entscheidung getroffen wird, verfügen wir über einen Testfall, der eine bestimmte Richtung der Entscheidung erzwingt – und zwar für alle möglichen Kombinationen und Richtungen. Dabei kann es leicht zu einer kombinatorischen Explosion bei der Anzahl der benötigten Tests kommen.

In der Praxis hoffe ich oft, dass die von mir durchgeführten Black-Box-Tests ausreichen, um alle Pfade abzudecken, sodass White-Box-Tests entfallen.

Das oben Gesagte sollte ausreichen, um deutlich zu machen, dass das Schreiben von Tests für einen Algorithmus mindestens genauso viel Zeit in Anspruch nimmt wie das Programmieren des Algorithmus selbst – oft sogar noch mehr Zeit. Viel mehr Zeit. Siehe hierzu beispielsweise auch http://sqlite.org/testing.html, einen Bericht darüber, wie die SQLite-Datenbank-Engine getestet wird. Ein weiterer interessanter Artikel könnte https://www.researchgate.net/publication/298896236 sein. Obwohl er sich auf einen bestimmten numerischen Algorithmus bezieht, zeigt er dennoch, dass selbst ein auf den ersten Blick einfacher Algorithmus zu einer unglaublichen Anzahl von Tests führen kann.

Eine interessante Verbindung besteht zur Dokumentation. Einerseits sind die Eigenschaften, die im Rahmen von Black-Box-Tests überprüft werden, genau jene Eigenschaften, die in der Man-Seite des Algorithmus dokumentiert sein sollten. Und umgekehrt bietet die Dokumentation der Eigenschaften eines Algorithmus eine gute Grundlage für die Gestaltung der Black-Box-Tests.

In der Praxis werden Testfälle und Dokumentation oft gleichzeitig verfasst, wobei sie sich gegenseitig beeinflussen. Und das eigentliche Verfassen von Testfällen ist eine Mischung aus Black-Box- und White-Box-Ansatz, was möglicherweise bereits während des Schreibens der Tests Einfluss auf die Implementierung nimmt. Ein Beispiel hierfür ist das Verfassen eines Tests für eine Bedingung wie *Startknoten nicht im Eingabegraphen*, der als Erinnerung dient, eine Überprüfung dieser Bedingung in den Code einzubauen.

Installationswerkzeuge

Ein letzter Punkt, den man berücksichtigen sollte, wenn ein neues Paket zur Sammlung hinzukommt, ist die Installation.

Die *Verwendung* des Skripts "*installer.tcl*" ist in *Tcllib – Anleitung für Installateure* dokumentiert.

Hier beschreiben wir, wie Sie das genannte Installationsprogramm so erweitern können, dass es neue Pakete und/oder Anwendungen installieren kann.

In den meisten Fällen muss lediglich eine einzige Datei geändert werden, nämlich die Datei "*support/installation/modules.tcl*", die je nach zu installierendem Modul und Anwendung jeweils einen Befehl enthält.

Die entsprechenden Befehle lauten:

Module *name code-action doc-action example-action*

Installieren Sie die Pakete des Moduls *name*, die sich unter "*modules/name*" befinden.

Die *code-action* ist für die Installation der Pakete und ihres Indexes zuständig. Das System stellt derzeit Folgendes bereit:

_tcl
: Kopiert alle im Verzeichnis "*modules/name*" gefundenen "*.tcl*"-Dateien in das Installationsverzeichnis.

_tcr
: Kopieren Sie ebenso wie bei **_tcl** die "*.tcl*"-Dateien, die sich in den Unterverzeichnissen von "*modules/Name*" befinden.

_tci
: Nennen Sie die Datei "**_tcl**" und kopieren Sie außerdem die Datei "*tclIndex.tcl*".

_msg
: Als **_tcl**, und kopieren Sie bitte auch das Unterverzeichnis "*msgs*".

_doc
: Geben Sie "**_tcl**" ein und kopieren Sie zudem das Unterverzeichnis "*mpformats*".

_tex
: Als **_tcl** und kopieren Sie zudem die "*.tex*"-Dateien.

Die *doc-action* ist für die Installation der Paketdokumentation zuständig. Das System stellt derzeit Folgendes bereit:

_null
: Es liegt keine Dokumentation vor; bitte unternehmen Sie nichts.

_man
: Verarbeiten Sie die im Verzeichnis "*modules/name*" befindlichen "*.man*"-Dateien und installieren Sie die Ergebnisse (nroff und/oder HTML) an dem dem Installationsprogramm angegebenen Speicherort.

  Hierbei handelt es sich eigentlich um eine Ausweichlösung; normalerweise verwendet das Installationsprogramm die vorgefertigte, formatierte Dokumentation, die sich unter "*idoc*" befindet.

Die *example-action* ist für die Installation der Beispiele zuständig. Das System stellt derzeit Folgendes bereit:

_null
Es sind keine Beispiele verfügbar; bitte unternehmen Sie nichts.

_exa
Kopieren Sie das Verzeichnis "*examples/Name*" rekursiv in den Installationsordner für die Beispiele.

Application *name*
Installieren Sie die Anwendung mit dem Namen *name*, die sich im Ordner "*apps*" befindet.

Exclude *name*
Dieser Befehl teilt dem Installationsprogramm mit, welche der aufgeführten Module *nicht* installiert werden sollen. Das heißt, er nennt die veralteten Module von Tcllib.

Nur wenn die oben genannten Aktionen für das neue Modul nicht geeignet sind, muss eine zweite Datei geändert werden: "*support/installation/actions.tcl*".

Diese Datei enthält die Implementierungen der verfügbaren Aktionen und ist der Ort, an dem jede benutzerdefinierte Aktion hinzugefügt werden muss, die zur Bewältigung der besonderen Umstände des Moduls erforderlich ist.

tcllib_installer

Stichwörter: tcllib_installer

Name

tcllib_install_guide – Tcllib – Anleitung für Installateure

Beschreibung

Willkommen bei Tcllib, der Tcl-Standardbibliothek. Beachten Sie, dass Tcllib selbst kein Paket ist. Es handelt sich um eine Sammlung von (teilweise unabhängigen) *Tcl*-Paketen, die Hilfsfunktionen bereitstellen, die für eine große Zahl von Tcl-Programmierern nützlich sind.

Dieses Dokument richtet sich an alle, die die in Tcllib enthaltenen Pakete kompilieren und installieren möchten – sei es für sich selbst oder für andere.

Darüber hinaus bieten wir Folgendes an:

1. *Tcllib – Das Entwicklerhandbuch*.

Für Entwickler, die selbst an den Paketen arbeiten möchten.

Bitte lesen Sie zunächst *Tcllib – So erhalten Sie die Quellcodes*, falls das noch nicht geschehen ist. Hier wird davon ausgegangen, dass die Quelltexte bereits in einem Verzeichnis der Wahl vorliegen.

Voraussetzungen

Bevor Tcllib kompiliert und verwendet werden kann, müssen einige Voraussetzungen installiert werden. Diese sind:

1. Die Skriptsprache Tcl. Weitere Informationen finden Sie unter Tcl.
2. Optional das Paket Critcl (C-Einbettung) für Tcl. Einzelheiten finden Sie unter **CriTcl**.

Diese Liste geht davon aus, dass der Rechner, auf dem Tcllib installiert werden soll, im Wesentlichen leer ist. Wenn Teile der unten aufgeführten Abhängigkeiten bereits installiert sind, können die entsprechenden Schritte natürlich übersprungen werden. Es wird jedoch weiterhin

empfohlen, die entsprechenden Abschnitte zu lesen, um sicherzustellen, dass die dort genannten Abhängigkeiten tatsächlich installiert sind.

Tcl

Da wir eine Reihe von Tcl-Paketen und -Anwendungen installieren, dürfte es ziemlich offensichtlich sein, dass eine funktionierende Installation von Tcl selbst erforderlich ist, und ich werde nicht weiter darauf eingehen.

Wählen Sie aus den zahlreichen Möglichkeiten diejenige aus, mit der Sie am besten zurechtkommen, solange sie mindestens Tcl 8.2 oder höher bereitstellt. Dabei kann es sich um eine Tcl-Installation handeln, die von Ihrer Betriebssystemdistribution bereitgestellt wird, von einem distributionsunabhängigen Anbieter stammt oder von Ihnen selbst erstellt wurde.

Beachten Sie, dass die Pakete in Tcllib mittlerweile die Versionen 8.4, 8.5 und sogar 8.6 erfordern. Ältere Tcl-Versionen können solche Pakete nicht verwenden. Der Versuch, sie zu nutzen, führt zu Fehlern der Art *Paket nicht gefunden*, da ihre Indexdateien diese Pakete in Kernversionen, die sie nicht unterstützen, nicht registrieren.

Ich selbst habe während der Entwicklung die ActiveState's ActiveTcl 8.5-Distribution verwendet (und tue dies nach wie vor), da ich mit dieser am besten vertraut bin.

(Offenlegung: Ich, Andreas Kupries, war bis 2016 bei ActiveState beschäftigt und habe dort die Pflege von ActiveTcl und TclDevKit übernommen).. Derzeit bin ich bei SUSE Software Canada ULC tätig, allerdings nicht in Bereichen, die mit Tcl zu tun haben.

Diese Distribution finden Sie unter http://www.activestate.com/activetcl. Laden Sie das Archiv von ActiveTcl 8.5 (oder höher) für Ihre Plattform herunter und installieren Sie es gemäß den Anweisungen von ActiveState.

Für diejenigen, die Tcl selbst kompilieren und installieren möchten, finden Sie die entsprechenden Quelldateien unter

Tcl
: http://core.tcl-lang.org/tcl/

zusammen mit den erforderlichen Anweisungen zur Kompilierung.

Wenn Probleme beim Kompilieren, Installieren oder Verwenden von Tcl auftreten, reichen Sie bitte ein Ticket bei *Tcl* oder beim Anbieter Ihrer Distribution ein, *nicht* bei *Tcllib*.

Critcl

Das Tool **critcl** ist eine *optionale* Abhängigkeit.

Dies ist nur erforderlich, wenn Sie versuchen, die C-basierten *Beschleuniger* für eine Reihe von Paketen zu erstellen, wie unter Critcl und Beschleuniger erläutert.

Vorsicht: Die C-Beschleuniger werden *standardmäßig gebaut*.

Unter Unix sucht das Build-System von Tcllib (**sak.tcl**) unter dem Namen **critcl**.

Unter Windows hingegen ist die Suche komplexer. Das Build-System sucht zunächst nach einer geeigneten Anwendung **critcl.exe**. Wird diese nicht gefunden, sucht es stattdessen nach einer Kombination aus Interpreter (**tclkitsh.exe**, **tclsh.exe**) und Starkit (**critcl.kit**, **critcl**). *Beachten Sie*, dass die Auswahl des Starkits über die Umgebungsvariable überschrieben werden kann.

Für Tcllib 1.21 und ältere Versionen ist Critcl Version 2 oder höher eine Anforderung.

Tcllib 2.0 und neuere Versionen haben hingegen die Anforderung an Critcl Version 3.3 oder

höher.

Critcl stellt diese Veröffentlichungen und die zugehörigen Quellen bereit.

Es kann jeder Zweig des Repositorys verwendet werden (wenn nicht das vorgefertigte "starkit" oder "starpack" zum Einsatz kommt), wobei die Verwendung des stabilen Zweigs *master* empfohlen wird.

Unter der oben genannten URL finden Sie zudem eine Anleitung zum Kompilieren und Installieren von Critcl, einschließlich einer Liste der erforderlichen Abhängigkeiten.

Die Anleitung dazu wird hier nicht wiederholt. Wenn bei dieser Anleitung Probleme auftreten, reichen Sie bitte ein Ticket für das Projekt *Critcl* ein, nicht für Tcllib.

Bau- und Installationsanleitung

Da es sich bei Tcllib im Wesentlichen um eine Sammlung von Paketen handelt, die in reinem Tcl geschrieben sind, entspricht das Erstellen derselben der Installation. Die Ausnahmen hiervon werden später in einem eigenen Unterabschnitt, Critcl und Beschleuniger, behandelt.

Zuvor gilt jedoch der Standardfall, der sich je nach Plattform durch wesentliche Unterschiede in der Anleitung auszeichnet, d. h. *Unix*-like im Vergleich zu *Windows*.

In Bezug auf Letzteres gibt es zudem einen Hinweis: Es ist möglich, eine *Unix*-like-Umgebung mithilfe von Projekten wie *MSYS*, *Cygwin* und anderen einzurichten. In diesem Fall hat der Benutzer die Wahl, welche Anweisungen er befolgen möchte.

Unabhängig von der Umgebung oder der Plattform muss ein geeignetes *Tcl* installiert sein, und dessen **tclsh** sollte (*Unix*) im Suchpfad liegen oder mit "*.tcl*"-Dateien (*Windows*) verknüpft sein.

Installation unter Unix

In *Unix*-like-Umgebungen wird Tcllib mit dem Standard-Dateisatz ausgeliefert, um

```
./configure
make install
# or just
make
```

eine geeignete Methode zur Installation. Hierbei handelt es sich um eine standardmäßige, nicht-interaktive Installation, bei der automatisch ermittelt wird, wo alles platziert werden soll, d. h. Pakete, Anwendungen und die Manpages.

Dies umfasst das Tcllibc-Paket. Daher ist eine Installation von Critcl erforderlich.

Um nur bestimmte Teile des Projekts zu installieren, verwenden Sie

```
make install-binaries      # Tcllibc. Requires an installation of
  Critcl.
make install-tcl           # Tcl packages and applications.
make install-libraries     # Tcl packages alone.
make install-applications  # Applications alone.
make install-doc           # Nroff manpages.
```

Für ein grafisches Installationsprogramm stattdessen:

```
./installer.tcl
```

Beachten Sie jedoch, dass dieses Installationsprogramm lediglich die Tcl-Pakete, Anwendungen und die Dokumentation abdeckt. Die binäre Tcllibc deckt es *nicht* ab.

Installation unter Windows

In einer Windows-Umgebung steht das Skript "**installer.tcl**" zur Durchführung der Installation zur Verfügung.

Wenn die "*.tcl*"-Dateien dem gewünschten **tclsh** zugeordnet sind, reicht ein Doppelklick bzw. das Öffnen von **installer.tcl** aus, um das Programm im grafischen Modus zu starten. Dies setzt voraus, dass *Tk* ebenfalls installiert und verfügbar ist.

Ohne *Tk* besteht die einzige Möglichkeit, das Installationsprogramm aufzurufen, darin, ein DOS-Fenster zu öffnen, d. h. **cmd.exe**, und anschließend folgenden Befehl auszuführen:

```
./installer.tcl
```

darin. *Beachten Sie*, dass dieses Installationsprogramm ausschließlich die Tcl-Pakete, Anwendungen und die Dokumentation verwaltet. Die binäre Tcllibc wird *nicht* berücksichtigt.

Critcl und Beschleuniger

Zwar besteht der Großteil von Tcllib aus Paketen, die in reinem Tcl geschrieben sind, doch gibt es auch eine Reihe von Paketen, denen sogenannte *Beschleuniger* zugeordnet sind.

Es handelt sich hierbei um auf **critcl** basierende C-Pakete, deren Verwendung die Leistung der Pakete, die sie nutzen, steigert. Sie werden zwar unter Unix standardmäßig kompiliert (siehe *make install*), sind jedoch nicht erforderlich, sondern *optional*.

Beim Erstellen der Beschleuniger wird die normalerweise optionale Abhängigkeit von **critcl** zur Anforderung. Wenn diese gemäß den nachstehenden Anweisungen erstellt werden, werden sie zudem ebenfalls installiert.

Um Tcllib mit den Beschleunigern in einer Unix-ähnlichen Umgebung zu kompilieren und zu installieren, führen Sie folgenden Befehl aus:

```
./configure
make
# or
make install
# or
make install-binaries
```

Das zugrunde liegende Tool ist "*sak.tcl*" im Toplevel-Verzeichnis von Tcllib, und das Ziel "**make install-binaries**" ist lediglich eine Hülle um

```
./sak.tcl critcl
```

Rufen Sie daher in einer Windows-Umgebung stattdessen folgenden Befehl auf:

```
./sak.tcl critcl
./installer.tcl
```

aus einem DOS-Fenster heraus, d. h. **cmd.exe**.

Vorsicht: Die durch diese Befehlsfolge erzeugte Verzeichnisstruktur unterscheidet sich von der durch **make install** und dessen Varianten erzeugten.

make install erstellt zwei Verzeichnisse für Pakete, eines für Tcllib und das andere für Tcllibc.

Die oben genannte Folge legt "Tcllibc" hingegen in ein Unterverzeichnis des Tcllib-Paketverzeichnisses.

Werkzeuge

Das Herzstück des Build-Systems von Tcllib ist das Skript "*installer.tcl*", das sich im Toplevel-Verzeichnis eines Checkouts oder einer Version befindet.

Die den Entwicklern auf Unix-ähnlichen Systemen zur Verfügung stehende Einrichtung

```
        configure ; make install
```

ist lediglich eine Hülle um dieses Skript. Um über die in dieser Hülle festgelegten Standards hinauszugehen, ist es erforderlich, dieses Skript direkt aufzurufen.

Auf Windows-Systemen ist die direkte Verwendung die einzige Möglichkeit, das Programm aufzurufen.

Um grundlegende Hilfe zu erhalten, rufen Sie das Programm wie folgt auf:

```
        ./installer.tcl -help
```

Dadurch wird eine kurze Liste aller verfügbaren Optionen auf den Standard-Ausgabekanal ausgegeben.

Die Befehle, die den verschiedenen *install*-Zielen im *Makefile.in* für Unix zugeordnet sind, können ebenfalls als weitere Beispiele für die Verwendung dieses Tools herangezogen werden.

Das Installationsprogramm kann im GUI- und im CLI-Modus ausgeführt werden. Standardmäßig wählt es den Modus automatisch aus, wenn das Tcl-Paket **Tk** verwendet werden kann oder nicht. Mit der Option **-no-gui** können Sie den CLI-Modus erzwingen.

Beachten Sie, dass Optionen über die Befehlszeile festgelegt werden können, selbst wenn das Installationsprogramm am Ende den GUI-Modus wählt. In diesem Fall bestimmen die fest vorgegebenen Standardwerte und die Optionen, welche Daten dem Benutzer zur Bearbeitung angezeigt werden.

Das Installationsprogramm führt eine Auswahl der Standardeinstellungen für die Speicherorte von Paketen, Beispielen und der Dokumentation sowie für das Format der Dokumentation durch. Der Benutzer kann diese Standardeinstellungen über die grafische Benutzeroberfläche oder durch Angabe zusätzlicher Optionen überschreiben. Die Standardeinstellungen hängen von der erkannten Plattform (Unix/Windows) sowie von der ausführbaren Datei "**tclsh**" ab, die zum Ausführen des Installationsprogramms verwendet wird.

Optionen

-help
: Zeigt die Liste der hier erläuterten Optionen auf dem Standardausgabekanal an und beendet das Programm.

+excluded
: Veraltete Pakete in die Installation einbeziehen.

-no-gui
: Den Befehlszeilenbetrieb des Installationsprogramms erzwingen

-no-wait

Im CLI-Modus fordert das Installationsprogramm den Benutzer standardmäßig auf, zu bestätigen, dass die gewählte Konfiguration (Zielpfade, zu installierende Komponenten) korrekt ist, bevor es weitere Schritte ausführt. Mit dieser Option überspringt das Installationsprogramm diese Abfrage und beginnt sofort mit der Installation.

-app-path *path*
siehe unten ...

-example-path *path*
siehe unten ...

-html-path *path*
siehe unten ...

-nroff-path *path*
siehe unten ...

-pkg-path *path*
Legt die Zielpfade für die Anwendungen, Beispiele, die HTML-Dokumentation, die nroff-Manpages und die Pakete fest. Die Standardwerte leiten sich aus dem Speicherort der Datei "**tclsh**" ab, die zum Ausführen des Installationsprogramms verwendet wurde.

-dry-run
siehe unten ...

-simulate
Führt das Installationsprogramm aus, ohne die Zielverzeichnisse zu ändern.

-apps
siehe unten ...

-no-apps
siehe unten ...

-examples
siehe unten ...

-no-examples
siehe unten ...

-pkgs
siehe unten ...

-no-pkgs
siehe unten ...

-html
siehe unten ...

-no-html
siehe unten ...

-nroff
siehe unten ...

-no-nroff
Schaltet die Installation von Anwendungen, Beispielen, Paketen, der HTML-Dokumentation und der nroff-Manpages aus oder ein.

Anwendungen, Beispiele und Pakete werden standardmäßig installiert.

Unter Windows wird die HTML-Dokumentation standardmäßig installiert.

Unter Unix werden die nroff-Manpages standardmäßig installiert.

tcllib_license

Stichwörter: tcllib_license

Name

tcllib_license – Tcllib – Lizenz

Beschreibung

Willkommen bei Tcllib, der Tcl-Standardbibliothek. Beachten Sie, dass Tcllib selbst kein Paket ist. Es handelt sich um eine Sammlung von (teilweise unabhängigen) *Tcl*-Paketen, die Hilfsfunktionen bereitstellen, die für eine große Zahl von Tcl-Programmierern nützlich sind.

Die Sammlung unterliegt der BSD-Lizenz.

Lizenz

Das Copyright an dieser Software liegt bei Ajuba Solutions und anderen Parteien. Die folgenden Bedingungen gelten für alle mit der Software verbundenen Dateien, sofern in einzelnen Dateien nicht ausdrücklich etwas anderes angegeben ist.

Die Autoren erteilen hiermit die Erlaubnis, diese Software und ihre Dokumentation für jeden beliebigen Zweck zu nutzen, zu vervielfältigen, zu ändern, zu verbreiten und zu lizenzieren, vorausgesetzt, dass die vorhandenen Copyright-Hinweise in allen Kopien beibehalten werden und dass dieser Hinweis bei jeder Weitergabe wörtlich übernommen wird. Für keine der zulässigen Verwendungszwecke ist eine schriftliche Vereinbarung, eine Lizenz oder die Zahlung von Lizenzgebühren erforderlich. Änderungen an dieser Software können durch ihre Autoren urheberrechtlich geschützt sein und müssen nicht den hier beschriebenen Lizenzbedingungen entsprechen, sofern die neuen Bedingungen auf der ersten Seite jeder Datei, für die sie gelten, deutlich angegeben sind.

IN KEINEM FALL HAFTEN DIE AUTOREN ODER VERTRIEBSPARTNER GEGENÜBER IRGENDEINER PARTEI FÜR DIREKTE, INDIREKTE, BESONDERE, NEBEN- ODER FOLGESCHÄDEN, DIE SICH AUS DER NUTZUNG DIESER SOFTWARE, IHRER DOKUMENTATION ODER ETWAIGER DAVON ABGELEITETER WERKE ERGEBEN, SELBST WENN DIE AUTOREN AUF DIE MÖGLICHKEIT SOLCHER SCHÄDEN HINGEWIESEN WURDEN.

DIE AUTOREN UND VERTRIEBSPARTNER SCHLIESSEN AUSDRÜCKLICH JEGLICHE GEWÄHRLEISTUNG AUS, EINSCHLIESSLICH, ABER NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDEN GEWÄHRLEISTUNGEN DER MARKTGÄNGIGKEIT, DER EIGNUNG FÜR EINEN BESTIMMTEN ZWECK UND DER NICHTVERLETZUNG VON RECHTEN DRITTER. DIESE SOFTWARE WIRD "WIE BESEHEN" BEREITGESTELLT, UND DIE AUTOREN UND VERTRIEBSPARTNER SIND NICHT VERPFLICHTET, WARTUNG, SUPPORT, AKTUALISIERUNGEN, VERBESSERUNGEN ODER ÄNDERUNGEN ANZUBIETEN.

NUTZUNG DURCH BEHÖRDEN: Wenn Sie diese Software im Auftrag der US-Regierung erwerben, stehen der Regierung lediglich "eingeschränkte Rechte" an der Software und der zugehörigen Dokumentation zu, wie in den Federal Acquisition Regulations (FARs) in Klausel 52.227.19 (c) (2) definiert. Wenn Sie die Software im Auftrag des Verteidigungsministeriums erwerben, wird die

Software als "kommerzielle Computersoftware" eingestuft, und die Regierung verfügt lediglich über "eingeschränkte Rechte" gemäß der Definition in Klausel 252.227-7013 (c) (1) der DFARs. Ungeachtet des Vorstehenden gewähren die Autoren der US-Regierung und anderen in ihrem Namen handelnden Stellen die Erlaubnis, die Software gemäß den in dieser Lizenz festgelegten Bedingungen zu nutzen und zu verbreiten.

tcllib_releasemgr

Stichwörter: tcllib_releasemgr

Name

tcllib_releasemgr – Tcllib – Leitfaden für Release-Manager

Beschreibung

Willkommen bei Tcllib, der Tcl-Standardbibliothek. Beachten Sie, dass Tcllib selbst kein Paket ist. Es handelt sich um eine Sammlung von (teilweise unabhängigen) *Tcl*-Paketen, die Hilfsfunktionen bereitstellen, die für eine große Zahl von Tcl-Programmierern nützlich sind.

Dieses Dokument richtet sich an den Release-Manager von Tcllib, dessen Stellvertreter sowie an alle weiteren Personen, die an der Erstellung einer offiziellen Version von Tcllib zur Verbreitung interessiert sind.

Bitte lesen Sie zunächst *Tcllib – So erhalten Sie die Quellcodes*, falls das noch nicht geschehen ist. Hier wird davon ausgegangen, dass die Quelltexte bereits in einem Verzeichnis der Wahl vorliegen.

Werkzeuge

Das Skript "*sak.tcl*" im Toplevel-Verzeichnis eines Tcllib-Checkouts ist das einzige Werkzeug, das der Release-Manager zur Ausführung seiner Aufgaben verwendet.

Die wichtigsten Befehle, die verwendet werden sollen, sind

```
    sak.tcl validate
    sak.tcl test run
    sak.tcl review
    sak.tcl readme
    sak.tcl localdoc
    sak.tcl release
```

Weitere Einzelheiten finden Sie in den Erläuterungen zu den verschiedenen Aufgaben.

Aufgaben

Einen Release Candidate starten

Aufgabe: Einen Kandidaten für die Veröffentlichung freigeben

Den Kandidaten fertigstellen

Aufgaben: Testen, Validierung und Prüfen, ob die Version zur Veröffentlichung geeignet ist; Testsuiten korrigieren, ggf. Pakete korrigieren; Dokumentation neu generieren; sich mit den Paketbetreuern bezüglich der Korrekturen abstimmen; Wichtig: Die Pakete durchgehen und die Änderungen seit der letzten Version klassifizieren, um eine übersichtliche README-Datei zu erstellen.

Offiziell machen

Aufgabe: Veröffentlichung abschließen, Kandidatenversion offiziell machen

Das Release verteilen

Nach dem Release muss es veröffentlicht und die Öffentlichkeit über seine Existenz informiert werden.

1. Über http://core.tcl-lang.org/tcllib/eventedit ein Fossil-Ereignis für das Release anlegen.

   Als Vorlage dient ein bestehendes Ereignis.
2. Aktualisieren Sie eine Reihe von Webseiten:
   1. Startseite
   2. Downloads
   3. Frühere Veröffentlichungen
   4. http://www.tcl-lang.org/home/release.txt
   5. http://www.tcl-lang.org/software/tcllib/*.tml
   6. http://wiki.tcl-lang.org/page/Tcllib

1. Der erste Speicherort verweist auf die Datei "*embedded/index.md*" im Repository selbst und kann daher im Rahmen des Release-Prozesses bearbeitet werden. Hier wird der Verweis auf das neue Fossil-Ereignis als aktuelle Version hinzugefügt.

   Die nächsten beiden Speicherorte befinden sich im Fossil-tcllib-Wiki und erfordern Administrator- oder Wiki-Schreibrechte für http://core.tcl-lang.org/tcllib.

   Die letzten beiden Speicherorte erfordern SSH-Zugriff auf http://www.tcl-lang.org sowie die Berechtigung, Dateien im Webbereich zu bearbeiten.
2. ***TODO*** Mailinglisten und andere Plätze, an die Sie Hinweise senden können.

tcllib_sources

Stichwörter: tcllib_sources

Name

tcllib_sources – Tcllib – So erhalten Sie die Quellcodes

Beschreibung

Willkommen bei Tcllib, der Tcl-Standardbibliothek. Beachten Sie, dass Tcllib selbst kein Paket ist. Es handelt sich um eine Sammlung von (teilweise unabhängigen) *Tcl*-Paketen, die Hilfsfunktionen bereitstellen, die für eine große Zahl von Tcl-Programmierern nützlich sind.

Dieses Dokument richtet sich an alle, die sich entweder lediglich den Quellcode von Tcllib ansehen, die Pakete kompilieren oder diese erweitern und anpassen möchten.

Für alle, die die Pakete bauen oder entwickeln, gibt es darüber hinaus:

1. *Tcllib - The Installer's Guide*.
2. *Tcllib – Das Entwicklerhandbuch*,

in dieser Reihenfolge.

Fundort der Quellen

Das offizielle Repository für Tcllib finden Sie unter http://core.tcl-lang.org/tcllib. Dieses Repository wird von Fossil SCM verwaltet.

Archive für "HEAD" und Veröffentlichungen abrufen

Am einfachsten geht dies, indem man das offizielle Repository aufruft und den Links im Abschnitt *Releases* oben folgt, direkt unterhalb des Eingabefelds für die Suche in der Dokumentation des Pakets.

Abruf beliebiger Commits

Bei allem, was über den Head-Status und Veröffentlichungen hinausgeht, ist der Prozess etwas aufwendiger.

Wenn die Commit-ID (Commit-Hash) **((ID))** der betreffenden Revision bereits bekannt ist, lassen sich Links zu den gewünschten Archiven in folgender Form bilden:

```
    https://core.tcl-lang.org/tcllib/tarball/((ID))/Tcl+Library+Source+
    Code.tar.gz
    https://core.tcl-lang.org/tcllib/zip/((ID))/Tcl+Library+Source+Code
    .zip
```

Beachten Sie, dass für "**((ID))**" auch Zweignamen verwendet werden können. Das ergibt Archive mit der aktuellsten Revision des genannten Zweigs.

Der Teil der URL nach "**((ID))**" ist der Name der zurückgegebenen Datei und kann nach Bedarf angepasst werden.

Ohne bekannte Commit-ID dauert der Prozess erneut länger:

1. Das offizielle Repository aufrufen.
2. Den Anmeldelink bzw. die Anmeldeschaltfläche in der oberen rechten Ecke der Seite suchen.
3. Als "anonymous" anmelden, mit dem halbzufälligen Passwort aus dem Captcha.
4. Die "Timeline" über den Link bzw. die Schaltfläche in der Mitte der Navigationsleiste öffnen.
5. Die gewünschte Revision auswählen.
6. Dem Link zur Seite mit den Einzelheiten folgen.
7. Auf dieser Seite den Link "ZIP" oder "Tarball" wählen, um eine Kopie dieser Revision im gewünschten Format zu erhalten.

Quellcodeverwaltung

Die Quellen werden mit Fossil SCM verwaltet. Binärdateien für gängige Plattformen liegen auf der Download-Seite.

Mit diesem Tool lässt sich die vollständige Geschichte wie folgt abrufen:

```
    fossil clone  http://core.tcl-lang.org/tcllib  tcllib.fossil
```

gefolgt von

```
    mkdir tcllib
    cd tcllib
    fossil open ../tcllib.fossil
```

um einen Checkout des Trunk-Heads durchzuführen.

aes

aes

Stichwörter: aes, block cipher, data integrity, encryption, security

Name

aes – Implementierung der AES-Blockverschlüsselung

Übersicht

package require Tcl 8.5 9
package require aes ?**1.2.2**?
::aes::aes ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | ?**--**? *data*]
::aes::Init *mode keydata iv*
::aes::Encrypt *Key data*
::aes::Decrypt *Key data*
::aes::Reset *Key iv*
::aes::Final *Key*

Beschreibung

Dies ist eine Tcl-Implementierung des Advanced Encryption Standard (AES), wie er vom US-amerikanischen National Institute of Standards and Technology veröffentlicht wurde [1]. AES ist eine 128-Bit-Blockverschlüsselung mit einer variablen Schlüsselgröße von 128, 192 oder 256 Bit. Diese Implementierung unterstützt die Modi CBC und ECB.

Befehle

::aes::aes ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | ?*--*? *data*]

Wendet den Algorithmus **aes** entweder auf die durch das Argument bereitgestellten Daten oder auf die aus dem Kanal *-in* gelesenen Daten an. Wenn ein *-out*-Kanal angegeben wird, wird das Ergebnis in diesen Kanal geschrieben.

Die Option *-key* muss angegeben werden. Dieser Parameter akzeptiert eine binäre Zeichenkette mit einer Länge von entweder 16, 24 oder 32 Byte und dient zur Generierung des Schlüsselschemas.

Die Optionen *-mode* und *-dir* sind optional und sind standardmäßig auf den CBC-Modus bzw. auf encrypt gesetzt. Der Initialisierungsvektor *-iv* erwartet ein binäres Argument mit einer Länge von 16 Byte, das standardmäßig aus lauter Nullen besteht. Weitere Informationen zu den verfügbaren Modi und deren Verwendung finden Sie unter Betriebsarten.

Da AES eine 128-Bit-Blockverschlüsselung ist, müssen die Daten in Einheiten bereitgestellt werden, die ein Vielfaches von 16 Byte sind.

Programmierschnittstelle

Der interne Zustand wird in einer undurchsichtigen Struktur verwaltet, die von der Funktion **Init** zurückgegeben wird. Im CBC-Modus wird ein bestimmter, von der Eingabe abhängiger Zustand verwaltet, der durch den Aufruf der Funktion **Reset** mit einem neuen Wert für den

Initialisierungsvektor zurückgesetzt werden kann. Im ECB-Modus wird der Zustand durch die Eingabe nicht beeinflusst.

::aes::Init *mode keydata iv*
: Erstellt ein neues AES-Schlüsselschema unter Verwendung der angegebenen Schlüsseldaten und des vorgegebenen Initialisierungsvektors. Der Initialisierungsvektor ist für den CBC-Modus von Bedeutung; im ECB-Modus wird er nicht verwendet. Weitere Informationen zu den Verschlüsselungsmodi finden Sie unter Betriebsarten.

::aes::Encrypt *Key data*
: Verschlüsselt die bereitgestellten Daten mit einem vorbereiteten Schlüssel aus **Init**. Das Argument "data" sollte ein binäres Array sein, dessen Größe ein Vielfaches der AES-Blockgröße von 16 Byte beträgt. Das Ergebnis ist ein binäres Array, dessen Größe der der Eingabedaten entspricht.

::aes::Decrypt *Key data*
: Hinweis: Der gleiche Schlüssel kann sowohl zum Verschlüsseln als auch zum Entschlüsseln von Daten verwendet werden, sofern der Initialisierungsvektor für den CBC-Modus entsprechend zurückgesetzt wird.

::aes::Reset *Key iv*
: Setzt den Initialisierungsvektor zurück. Dies ermöglicht es dem Programmierer, einen Schlüssel wiederzuverwenden und den Aufwand für die Neugenerierung des Schlüsselschemas zu vermeiden, wenn dieselben Schlüsseldaten mehrfach verwendet werden.

::aes::Final *Key*
: Diese Funktion sollte aufgerufen werden, um die mit *Key* verbundenen Ressourcen freizugeben. Sobald diese Funktion aufgerufen wurde, darf der Schlüssel nicht mehr verwendet werden.

Betriebsarten

Cipher Block Chaining (CBC)
: Im CBC-Modus wird die Ausgabe der Verschlüsselung des letzten Blocks zur Bearbeitung des aktuellen Blocks herangezogen. Zur Verarbeitung des ersten Blocks wird ein Initialisierungsvektor verwendet, dessen Größe der Chiffreblockgröße entspricht. Der Initialisierungsvektor sollte zufällig ausgewählt und als erster Block der Ausgabe übertragen werden. Fehler bei der Verschlüsselung wirken sich auf den aktuellen Block und den nächsten Block aus; danach korrigiert sich die Verschlüsselung von selbst. CBC ist der am häufigsten verwendete Modus in der Softwareverschlüsselung und der Standardbetriebsmodus dieses Moduls.

Electronic Code Book (ECB)
: Der ECB-Modus ist der Grundmodus aller Blockchiffren. Jeder Block wird unabhängig verschlüsselt, sodass identischer Klartext bei der Verschlüsselung mit demselben Schlüssel zu identischem Ergebnis führt. Etwaige Verschlüsselungsfehler wirken sich nur auf einen einzelnen Block aus. *Dieser Modus ist anfällig für Angriffe mit bekanntem Klartext.*

Beispiele

```
% set nil_block [string repeat \\0 16]
% aes::aes -hex -mode cbc -dir encrypt -key $nil_block $nil_block
66e94bd4ef8a2c3b884cfa59ca342b2e
```

```
set Key [aes::Init cbc $sixteen_bytes_key_data $sixteen_byte_iv]
append ciphertext [aes::Encrypt $Key $plaintext]
append ciphertext [aes::Encrypt $Key $additional_plaintext]
aes::Final $Key
```

Literatur

1. "Advanced Encryption Standard", Federal Information Processing Standards Publication 197, 2001 (http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf)

Autoren

Thorsten Schloermann, Pat Thoyts

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *aes* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
Copyright (c) 2012-2014, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

amazon-s3

S3

Stichwörter: S3, amazon, cloud

Name

S3 – Schnittstelle zum Amazon S3-Webdienst

Übersicht

package require Tcl 8.5 9
package require S3 ?**1.0.5**?
package require sha1 1.1

package require md5 2.1
package require base64 2.4
package require xsxp 1.1
S3::Configure ?*-reset boolean*? ?*-retries integer*? ?*-accesskeyid idstring*? ?*-secretaccesskey idstring*? ?*-service-access-point FQDN*? ?*-use-tls boolean*? ?*-default-compare always|never|exists|missing|newer|date|checksum|different*? ?*-default-separator string*? ?*-default-acl private|public-read|public-read-write|authenticated-read|keep|calc*? ?*-default-bucket bucketname*?
S3::SuggestBucket ?*name*?
S3::REST *dict*
S3::ListAllMyBuckets ?*-blocking boolean*? ?*-parse-xml xmlstring*? ?*-result-type REST|xml|pxml|dict|names|owner*?
S3::PutBucket ?*-bucket bucketname*? ?*-blocking boolean*? ?*-acl {}|private|public-read|public-read-write|authenticated-read*?
S3::DeleteBucket ?*-bucket bucketname*? ?*-blocking boolean*?
S3::GetBucket ?*-bucket bucketname*? ?*-blocking boolean*? ?*-parse-xml xmlstring*? ?*-max-count integer*? ?*-prefix prefixstring*? ?*-delimiter delimiterstring*? ?*-result-type REST|xml|pxml|names|dict*?
S3::Put ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-file filename*? ?*-content contentstring*? ?*-acl private|public-read|public-read-write|authenticated-read|calc|keep*? ?*-content-type contenttypestring*? ?*-x-amz-meta-* metadatatext*? ?*-compare comparemode*?
S3::Get ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-compare comparemode*? ?*-file filename*? ?*-content contentvarname*? ?*-timestamp aws|now*? ?*-headers headervarname*?
S3::Head ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-dict dictvarname*? ?*-headers headersvarname*? ?*-status statusvarname*?
S3::GetAcl ?*-blocking boolean*? ?*-bucket bucketname*? **-resource** *resourcename* ?*-result-type REST|xml|pxml*?
S3::PutAcl ?*-blocking boolean*? ?*-bucket bucketname*? **-resource** *resourcename* ?*-acl new-acl*?
S3::Delete ?*-bucket bucketname*? **-resource** *resourcename* ?*-blocking boolean*? ?*-status statusvar*?
S3::Push ?*-bucket bucketname*? **-directory** *directoryname* ?*-prefix prefixstring*? ?*-compare comparemode*? ?*-x-amz-meta-* metastring*? ?*-acl aclcode*? ?*-delete boolean*? ?*-error throw|break|continue*? ?*-progress scriptprefix*?
S3::Pull ?*-bucket bucketname*? **-directory** *directoryname* ?*-prefix prefixstring*? ?*-blocking boolean*? ?*-compare comparemode*? ?*-delete boolean*? ?*-timestamp aws|now*? ?*-error throw|break|continue*? ?*-progress scriptprefix*?
S3::Toss ?*-bucket bucketname*? **-prefix** *prefixstring* ?*-blocking boolean*? ?*-error throw|break|continue*? ?*-progress scriptprefix*?

Beschreibung

Dieses Paket ermöglicht den Zugriff auf den Webdienst "Simple Storage Solution" von Amazon.

Zusammenfassend lässt sich sagen, dass Amazon Simple Storage Solution einen kostenpflichtigen Webdienst bereitstellt, der die Online-Speicherung beliebiger Daten als "Ressourcen" in "Buckets" ermöglicht. Weitere Informationen zu diesem System finden Sie unter http://www.amazonaws.com/. Der Zugriff auf den Dienst erfolgt über HTTP (SOAP oder REST). Ein

Großteil dieser Dokumentation ist für Sie kaum nachvollziehbar, wenn Sie mit den Begriffen und Funktionen des Amazon S3-Dienstes nicht vertraut sind.

Dieses Paket stellt Dienste zum Lesen und Schreiben der Datenelemente über die REST-Schnittstelle bereit. Zudem bietet es einige übergeordnete Operationen. Weitere Pakete derselben Distribution bieten noch mehr Funktionen.

Copyright 2006 Darren New. Alle Rechte vorbehalten. ES WERDEN KEINERLEI GEWÄHRLEISTUNGEN JEGLICHER ART ÜBERNOMMEN. DAS KOPIEREN ODER DIE NUTZUNG BEFREIT DEN AUTOR IN JEDER HINSICHT VON JEGLICHER HAFTUNG. Diese Software wird unter im Wesentlichen denselben Bedingungen wie Tcl lizenziert. Die Bedingungen finden Sie in der Datei "LICENSE.txt".

Fehlermeldung

Die Fehlermeldung dieses Pakets nutzt $errorCode, um detailliertere Informationen zum Vorfall zu liefern, anstatt lediglich einen Fehler auszugeben. Jeder vom S3-Paket abgefangene Fehler (und es wird versucht, alle abzufangen) kehrt mit einem $errorCode zurück, bei dem es sich um eine Liste mit mindestens drei Elementen handelt. In allen Fällen lautet das erste Element "S3". Das zweite Element nimmt einen von sechs Werten an, wobei dieses Element den Wert des dritten und der nachfolgenden Elemente bestimmt. S3::REST löst keinen Fehler aus, sondern gibt ein Dictionary zurück, in dem die Schlüssel "error", "errorInfo" und "errorCode" gesetzt sind. Dies ermöglicht eine zuverlässige Nutzung im Hintergrund. Die möglichen zweiten Elemente sind folgende:

usage
: Das Paket wurde falsch verwendet. Beispielsweise wurde ein Befehl aufgerufen, der eine Konfiguration der Bibliothek erfordert, bevor diese konfiguriert wurde, oder es wurde eine ungültige Kombination von Optionen angegeben. Das dritte Element von $errorCode enthält den Namen des Parameters, der falsch war. Das vierte Element enthält in der Regel die Argumente, die tatsächlich an die auslösende Prozedur übergeben wurden, es sei denn, der Verwendungsfehler beschränkt sich nicht auf eine einzelne Prozedur.

local
: Auf dem lokalen System ist ein Vorfall aufgetreten, der zu einem Fehler geführt hat. Beispielsweise wurde eine Anfrage zum Hoch- oder Herunterladen einer Datei gestellt, doch die Dateiberechtigungen haben diesen Zugriff verweigert. Das dritte Element von $errorCode ist das ursprüngliche $errorCode.

socket
: Es ist ein Problem mit dem Socket aufgetreten. Dieser wurde vorzeitig geschlossen, oder es wurde ein anderer Fehler bei der Kommunikation mit Amazon festgestellt. Das dritte Element von $errorCode ist das ursprüngliche $errorCode, oder manchmal die Meldung von fcopy, oder ...?

remote
: Der Amazon Web Service hat im HTTP-Header einen Fehlercode außerhalb des 2xx-Bereichs zurückgegeben. Mit anderen Worten: Alles verlief wie dokumentiert, außer dass in diesem speziellen Fall dokumentiert war, dass es nicht funktionieren würde. Das dritte Element ist das von **::S3::REST** zurückgegebene Dictionary. Hinweis: S3::REST selbst löst diesen Fehler niemals aus, sondern gibt lediglich das Dictionary zurück. Die meisten übergeordneten Befehle lösen aus Gründen der Benutzerfreundlichkeit einen Fehler aus, sofern nicht durch ein

Argument angegeben wird, dass dies nicht geschehen soll. Wenn etwas als "löst keinen S3-Remote-Fehler aus" dokumentiert ist, bedeutet dies, dass ein Statuswert zurückgegeben wird, anstatt einen Fehler auszulösen, falls Amazon einen HTTP-Statuscode außerhalb des 2xx-Bereichs zurückgibt.

notyet
Der Benutzer hat sich an die Dokumentation gehalten, doch der Autor ist noch nicht dazu gekommen, diese Funktion zu implementieren. (Derzeit fallen lediglich die TLS-Unterstützung und erweiterte Berechtigungen in diese Kategorie, ebenso wie der Befehl S3::Acl.)

xml
Der Dienst hat ungültiges XML oder XML mit einem unerwarteten Schema zurückgegeben. Bei übergeordneten Befehlen, die Dienst-XML als Eingabe zum Parsen akzeptieren, kann dieser Fehler ebenfalls ausgelöst werden.

Befehle

Dieses Paket bietet mehrere unterschiedliche Komplexitätsstufen.

- Die unterste Ebene nimmt lediglich die an den Dienst zu übermittelnden Argumente entgegen, sendet diese, ruft das Ergebnis ab und stellt es dem Aufrufer zur Verfügung. *Hinweis:* Diese Ebene ermöglicht sowohl synchrone als auch ereignisgesteuerte Verarbeitung. Sie ist abhängig von den Paketen MD5, SHA1 und base64 aus Tcllib (verfügbar unter http://core.tcl.tk/tcllib/). Hinweis: **S3::Configure** ist erforderlich, damit **S3::REST** aufgrund des Authentifizierungsteils funktioniert; daher haben wir dies in die "unterste Ebene" integriert.
- Die nächste Ebene wertet die Ergebnisse der Aufrufe aus und ermöglicht so Funktionen wie das Hochladen nur geänderter Dateien, die Synchronisierung von Verzeichnissen und Ähnliches. Diese Ebene ist sowohl vom Paket "**TclXML**" als auch vom enthaltenen Paket "**xsxp**" abhängig. Diese Pakete werden mit package require geladen, wenn diese komplexeren Routinen aufgerufen werden, sodass nichts fehlschlägt, wenn sie nicht korrekt installiert sind.
- Ebenfalls enthalten ist ein separates Programm, das die Bibliothek nutzt. Es enthält Code zum Parsen von $argv0 und $argv über die Befehlszeile, wodurch der Aufruf als tclkit usw. möglich ist. (Noch nicht implementiert.)
- Ein weiteres eigenständiges Programm bietet eine grafische Benutzeroberfläche, die Funktionen zum Ziehen und Ablegen sowie ähnliche Funktionen ermöglicht. (Noch nicht implementiert.)
- Auf diesem Paket basiert auch das Programm "OddJob". Es handelt sich um ein eigenständiges Programm, das die Verteilung von Rechenaufgaben über den Webdienst "Elastic Compute Cloud" von Amazon ermöglicht.

Das Ziel besteht darin, zumindest die untersten Schichten in reinem Tcl zu implementieren, wobei ausschließlich Komponenten aus allgemein verfügbaren Quellen, wie beispielsweise Tcllib, verwendet werden sollen.

Befehle auf niedriger Ebene

Für diese Befehle sind keine weiteren Pakete erforderlich, die nicht oben aufgeführt sind. Sie kommunizieren direkt mit dem Dienst oder sind Hilfs- bzw. Konfigurationsroutinen. Beachten Sie, dass das Paket "xsxp" zur Unterstützung dieses Pakets entwickelt wurde; es sollte daher überall dort verfügbar sein, wo Sie dieses Paket bezogen haben.

S3::Configure ?-reset *boolean*? ?-retries *integer*? ?-accesskeyid *idstring*? ?

-secretaccesskey *idstring*? ?-service-access-point *FQDN*? ?-use-tls *boolean*? ? -default-compare *always*|*never*|*exists*|*missing*|*newer*|*date*|*checksum*|*different*? ? -default-separator *string*? ?-default-acl *private*|*public-read*|*public-read-write*|*authenticated-read*|*keep*|*calc*? ?-default-bucket *bucketname*?

Es gibt einen Befehl für die Konfiguration, nämlich **S3::Configure**. Wird er ohne Argumente aufgerufen, gibt er ein Dictionary mit Schlüssel-Wert-Paaren zurück, in dem alle aktuellen Einstellungen aufgeführt sind. Wird er mit einem Argument aufgerufen, gibt er den Wert dieses einzelnen Arguments zurück. Wenn der Befehl mit zwei oder mehr Argumenten aufgerufen wird, müssen diese paarweise übergeben werden, und die Änderungen werden der Reihe nach übernommen. Pro Interpreter gibt es nur einen Satz an Informationen zur Konfiguration.

Folgende Optionen werden akzeptiert:

-reset *boolean*

Standardmäßig false. Wenn der Wert true ist, werden alle bisherigen Änderungen sowie alle Änderungen, die im Rahmen desselben Aufrufs vor der Option reset vorgenommen wurden, auf die Standardwerte zurückgesetzt.

-retries *integer*

Der Standardwert beträgt 3. Wenn Amazon einen 500-Fehler zurückgibt, wird nach einer exponentiellen Backoff-Verzögerung so oft ein erneuter Versuch unternommen, bevor schließlich der 500-Fehler ausgelöst wird. Dies gilt für jeden Aufruf von **S3::REST** durch übergeordnete Befehle, jedoch nicht für **S3::REST** selbst. Das heißt, **S3::REST** gibt stets den HTTP-Status 500 zurück, wenn dieser empfangen wird. Funktionen wie **S3::Put** wiederholen den PUT-Aufruf und wiederholen zudem die GET- und HEAD-Aufrufe, die zum Vergleich der Inhalte verwendet werden. Wenn Sie diesen Wert auf 0 setzen, werden Wiederholungsversuche und die damit verbundenen Verzögerungen verhindert. Darüber hinaus werden Socket-Fehler (d. h. Fehler, deren "errorCode" mit "S3 socket" beginnt) nach einer Wartezeit ebenfalls erneut versucht.

-accesskeyid *idstring*

siehe unten ...

-secretaccesskey *idstring*

Beide sind standardmäßig auf eine leere Zeichenfolge gesetzt. Diese müssen vor der Ausführung von Aufrufen festgelegt werden. Dies ist Ihre S3-ID. Sobald ein Konto registriert ist, wird http://www.amazonaws.com/ aufgerufen, eine Anmeldung durchgeführt, die Schaltfläche "Your Web Services Account" gewählt, "AWS Access Identifiers" ausgewählt, und die Zugriffsschlüssel-ID sowie die geheimen Zugriffsschlüssel werden angezeigt. Alle **S3::REST**-Aufrufe werden authentifiziert. Die Schuld für die unglückliche Namenswahl liegt bei Amazon.

-service-access-point *FQDN*

Standardmäßig ist "s3.amazonaws.com" eingestellt. Dies ist der vollqualifizierte Name des Servers, der bei Aufrufen von **S3::REST** kontaktiert werden soll. Sie sollten diese Einstellung wahrscheinlich nie ändern müssen, es sei denn, jemand anderes implementiert einen kompatiblen Dienst oder Sie möchten Tests durchführen, indem Sie die Bibliothek auf Ihren eigenen Dienst verweisen.

-slop-seconds *integer*

Wenn Datumsangaben zwischen Amazon und dem lokalen Rechner verglichen werden, werden zwei Datumsangaben, die innerhalb dieser Anzahl von Sekunden voneinander liegen, als identisch betrachtet. Dies ist nützlich für die Korrektur von Uhrenabweichungen, die Berücksichtigung von Verarbeitungsverzögerungen und Ähnliches.

-use-tls *boolean*
Standardwert ist "false". Diese Funktion ist noch nicht implementiert. Wenn "true" aktiviert ist, baut **S3::REST** eine TLS-Verbindung zu Amazon auf. Wenn "false" aktiviert ist, werden unverschlüsselte Verbindungen verwendet.

-bucket-prefix *string*
Die Standardeinstellung lautet "TclS3". Diese Zeichenfolge wird von **S3::SuggestBucketName** verwendet, wenn diesem Befehl eine leere Zeichenfolge als Argument übergeben wird. Sie dient dazu, verschiedene Anwendungen zu unterscheiden, die den Amazon-Dienst nutzen. Ihre Anwendung sollte diesen Wert stets festlegen, um zu verhindern, dass es zu Konflikten mit den Buckets anderer Nutzer von Amazon S3 oder mit anderen Buckets desselben Nutzers kommt.

-default-compare *always*|*never*|*exists*|*missing*|*newer*|*date*|*checksum*|*different*
Die Standardeinstellung lautet "always". Wenn für -compare kein Wert in **S3::Put**, **S3::Get** oder **S3::Delete** festgelegt wird, wird dieser Vergleich herangezogen. Eine Beschreibung der Bedeutung steht in den jeweiligen Befehlen.

-default-separator *string*
Standardmäßig ist "/" eingestellt. Diese Option wird derzeit nicht verwendet. Es könnte sinnvoll sein, dies für **S3::Push** und **S3::Pull** zu verwenden, doch es wäre problematisch, wenn Ressourcen Schrägstriche in ihren Namen enthalten dürften, die keine Verzeichnisse kennzeichnen. Daher hat diese Einstellung derzeit keine Auswirkung.

-default-acl *private*|*public-read*|*public-read-write*|*authenticated-read*|*keep*|*calc*
Standardmäßig ist dies eine leere Zeichenkette. Wenn bei **S3::Put** oder **S3::Push** kein -acl-Argument angegeben wird, wird diese Zeichenkette verwendet (als x-amz-acl-Header, sofern nicht "keep" oder "calc" angegeben ist). Wenn dieser ebenfalls leer ist, wird kein "x-amz-acl"-Header generiert. Dies wird von **S3::REST** *nicht* verwendet.

-default-bucket *bucketname*
Wenn für **S3::GetBucket**, **S3::PutBucket**, **S3::Get**, **S3::Put**, **S3::Head**, **S3::Acl**, **S3::Delete**, **S3::Push**, **S3::Pull** oder **S3::Toss** kein Bucket angegeben wird und diese Variable keine leere Zeichenkette (und nicht lediglich "/") ist, wird dieser Wert für den Bucket verwendet. Dies ist nützlich, wenn ein Programm eine große Anzahl von Ressourcenoperationen innerhalb eines einzelnen Buckets durchführt.

S3::SuggestBucket ?*name*?
Der Befehl **S3::SuggestBucket** akzeptiert eine optionale Zeichenfolge als Präfix und gibt einen gültigen Bucket zurück, der das Argument *name* sowie die Access Key ID enthält. Dadurch wird der Name für den Eigentümer und die Anwendung eindeutig (vorausgesetzt, die Anwendung wählt ein geeignetes *name*-Argument aus). Wenn kein Name angegeben wird, wird der Name aus **S3::Configure** *-bucket-prefix* verwendet. Ist auch dieser leer (was nicht der Standard ist), wird ein Fehler ausgelöst.

S3::REST *dict*
Der Befehl **S3::REST** nimmt ein Dictionary als Argument entgegen und gibt ein Dictionary

zurück. Das zurückgegebene Dictionary enthält dieselben Schlüssel wie das Eingabedictionary sowie zusätzliche Schlüssel als Ergebnis. Das Vorhandensein oder Fehlen von Schlüsseln im Eingabe-Dictionary kann das Verhalten der Routine steuern. Sie löst niemals direkt einen Fehler aus, fügt jedoch bei Bedarf die Schlüssel "error", "errorInfo" und "errorCode" hinzu. Einige Schlüssel sind erforderlich, andere optional. Die Routine kann je nach Vorhandensein von **resultvar** im Dictionary entweder im blockierenden oder im nicht-blockierenden Modus ausgeführt werden. Dazu müssen die Einträge *-accesskeyid* und *-secretaccesskey* vor dem Aufruf über **S3::Configure** konfiguriert werden.

Folgende Eingabeschlüssel sind möglich:

verb *GET*|*PUT*|*DELETE*|*HEAD*
: Dieses Pflichtfeld gibt das zu verwendende Verb an.

resource *string*
: Dieses Pflichtfeld gibt die Ressource an, auf die zugegriffen werden soll. Ein führendes "/" wird hinzugefügt, sofern es nicht bereits vorhanden ist. Bei Bedarf wird die Angabe für Sie durch URL-Kodierung bearbeitet. Ein bereits URL-kodierter Ressourcenname ist nicht anzugeben.

?rtype *torrent*|*acl*?
: Dies deutet darauf hin, dass eine Torrent- oder ACL-Ressource manipuliert wird. Dies gehört nicht in den Schlüssel **resource**, da sonst das Trennzeichen "?" durch URL-Kodierung verändert wird.

?parameters *dict*?
: Dieses optionale Dictionary enthält Parameter, die der URL für die Transaktion hinzugefügt werden. Die Schlüssel müssen in der korrekten Groß-/Kleinschreibung angegeben werden (was in der Amazon-Dokumentation verwirrend ist), und die Werte müssen gültig sein. Es kann sich um ein leeres Dictionary handeln oder es kann vollständig weggelassen werden, wenn keine Parameter gewünscht sind. Es wird keine weitere Fehlerprüfung der Parameter durchgeführt.

?headers *dict*?
: Dieses optionale Dictionary enthält Header, die der HTTP-Anfrage hinzugefügt werden sollen. Die Schlüssel müssen in *Kleinbuchstaben* geschrieben sein, damit die Authentifizierung funktioniert. Die Werte dürfen keine eingebetteten Zeilenumbrüche oder Wagenrückläufe enthalten. Dies ist in erster Linie für das Hinzufügen von x-amz-*-Headern nützlich. Da die Authentifizierung von **S3::REST** berechnet wird, wird dieser Header hier nicht hinzugefügt. Da content-type einen eigenen Schlüssel erhält, wird auch dieser Header hier nicht hinzugefügt.

?inbody *contentstring*?
: Dieses optionale Element gibt, wenn es angegeben wird, den zu sendenden Inhalt an. Es wird mit einer binären Übertragungskodierung gesendet, wobei nur die unteren Bytes verwendet werden. zu verwenden ist [encoding convertto utf-8], wenn es sich bei der Zeichenkette um eine UTF-8-Zeichenkette handelt. Dies wird in einem einzigen Durchgang geschrieben. Im nicht-blockierenden Modus kann ein besonders großes **inbody** den Schreib-Socket blockieren.

?infile *filename*?
: Dieser optionale Parameter gibt – wenn er angegeben wird und **inbody** nicht angegeben ist – den Namen der Datei an, aus der der Hauptteil der HTTP-Nachricht zusammengestellt

wird. Die Datei wird zum Lesen geöffnet und schrittweise per [fcopy] übertragen, sodass es im nicht-blockierenden Modus nicht zu einer Blockierung kommen sollte, selbst wenn die Datei sehr groß ist. Die Datei wird im Binärmodus übertragen, sodass die Bytes auf Ihrer Festplatte mit den Bytes in Ihrer Ressource übereinstimmen. Aufgrund von HTTP-Einschränkungen muss es möglich sein, [file size] für diese Datei zu verwenden, um die Größe zu Beginn der Transaktion zu ermitteln.

?S3chan *channel*?
Dieser optionale Parameter gibt, sofern angegeben, den bereits geöffneten Socket an, über den die Transaktion abgewickelt werden soll. Wird er nicht angegeben, wird eine Verbindung zu dem über **S3::Configure** angegebenen Service-Zugangspunkt hergestellt, bei dem es sich in der Regel um s3.amazonaws.com handelt. Wenn dieser Parameter angegeben wird, wird der Kanal am Ende der Transaktion nicht geschlossen.

?outchan *channel*?
Dieses optionale Element gibt, wenn angegeben, den bereits geöffneten Kanal an, in den der von S3 zurückgegebene Hauptteil geschrieben werden soll. Das heißt: Um eine große Ressource abzurufen, wird eine Datei geöffnet, der Übersetzungsmodus festgelegt und der Kanal als Wert des Schlüssels outchan übergeben. Die Ausgabe wird stückweise in den Kanal geschrieben, damit der Speicher nicht unnötig belegt wird. Der Kanal wird am Ende der Transaktion nicht geschlossen.

?resultvar *varname*?
Dieser optionale Parameter gibt, wenn angegeben, an, dass **S3::REST** im nicht-blockierenden Modus ausgeführt werden soll. Der *varname* muss in Bezug auf Namespaces vollständig qualifiziert sein und darf nicht lokal zu einer Prozedur sein. Sofern angegeben, wird das Ergebnis des Aufrufs von **S3::REST** dieser Variable zugewiesen, sobald alle Vorgänge abgeschlossen sind; trace oder vwait zeigen an, wann dies geschehen ist. Wenn dieser Schlüssel nicht angegeben wird, gibt **S3::REST** das Ergebnis einfach zurück, und es werden innerhalb dieses Aufrufs keine Aufrufe an die Ereignisschleife ausgelöst.

?throwsocket *throw*|*return*?
Dieser optionale Eintrag gibt, sofern vorhanden, an, dass **S3::REST** einen Fehler auslösen soll, wenn throwmode auf throw gesetzt ist und ein Socket-Fehler auftritt. Er gibt an, dass **S3::REST** den Fehlercode im zurückgegebenen Dictionary zurückgeben soll, wenn ein Socket-Fehler auftritt und dieser Parameter auf return gesetzt ist. Wenn **throwsocket** auf *return* gesetzt ist oder wenn der Aufruf nicht blockierend ist, werden ein Socket-Fehler (d. h. ein Fehler, dessen Fehlercode mit "S3 socket" beginnt) sowie **errorInfo** und **errorCode** im Dictionary unter **error** zurückgegeben. Wenn ein Vordergrundaufruf durchgeführt wird (d. h., **resultvar** wird nicht angegeben) und diese Option nicht angegeben oder auf *throw* gesetzt ist, wird stattdessen **error** aufgerufen.

Sobald der Aufruf von **S3::REST** abgeschlossen ist, gibt das Programm ein neues Dict zurück, entweder in der *resultvar* oder als Ergebnis der Ausführung. Dieses Dict ist eine Kopie des ursprünglichen Dicts, dem die Ergebnisse als neue Schlüssel hinzugefügt wurden. Folgende neue Schlüssel sind möglich:

error *errorstring*
siehe unten ...

errorInfo *errorstring*

siehe unten ...

errorCode ***errorstring***
Wenn ein Fehler erkannt wird, werden diese drei Schlüssel im Ergebnis gesetzt. Beachten Sie, dass **S3::REST** einen HTTP-Rückgabecode, der nicht im 2XX-Bereich liegt, *nicht* als Fehler betrachtet. Der Wert **errorCode** wird gemäß der Beschreibung unter Fehlermeldung formatiert. Sind diese vorhanden, sind andere hier beschriebene Schlüssel möglicherweise nicht vorhanden.

httpstatus ***threedigits***
Der dreistellige Code aus der HTTP-Transaktion. 2XX für einen erfolgreichen Vorgang, 5XX für einen Serverfehler usw.

httpmessage ***text***
Die Textangabe nach dem Statuscode. "OK" oder "Forbidden" oder Ähnliches.

outbody ***contentstring***
Wenn *outchan* nicht festgelegt wurde, enthält dieser Schlüssel einen Verweis auf den (unverschlüsselten) Inhalt des zurückgegebenen Hauptteils. Wenn Amazon einen Fehler zurückgibt (d. h. der HTTP-Status ist kein 2XX-Wert), wird die Fehlermeldung in **outbody** gespeichert oder gegebenenfalls in **outchan** geschrieben.

outheaders ***dict***
Dieser Schlüssel enthält ein Dictionary der von Amazon zurückgegebenen Header. Die Schlüssel sind stets in Kleinbuchstaben geschrieben. Er ist vor allem nützlich, um gegebenenfalls die "x-amz-meta-*"-Header zu finden, obwohl auch Angaben wie "last-modified" und "content-type" hilfreich sind. Die Schlüssel dieses Dictionary sind stets in Kleinbuchstaben geschrieben. Sowohl Schlüssel als auch Werte werden von überflüssigen Leerzeichen bereinigt.

Befehle auf hoher Ebene

Die Routinen in diesem Abschnitt nutzen alle einen oder mehrere Aufrufe von **S3::REST**, um ihre Aufgaben auszuführen, und analysieren und verwalten anschließend die Daten auf praktische Weise. Sofern nicht anders angegeben, lösen alle diese Befehle Fehler aus, wie unter Fehlermeldung beschrieben.

In allen diesen Befehlen werden alle Argumente als Name-Wert-Paare in beliebiger Reihenfolge angegeben. Alle Argumentnamen beginnen mit einem Bindestrich.

Es gibt einige Optionen, die vielen Befehlen gemeinsam sind, und diese gemeinsamen Optionen werden hier beschrieben.

-blocking ***boolean***
Wenn dieser Parameter angegeben und auf false gesetzt ist, erfolgen alle Aufrufe von **S3:REST** nicht-blockierend, und intern rufen diese Routinen [vwait] auf, um die Ergebnisse abzurufen. Mit anderen Worten: Diese Routinen geben denselben Wert zurück, führen jedoch Ereignisschleifen aus, während sie auf Amazon warten.

-parse-xml ***xmlstring***
Wenn dieses Argument angegeben wird, überspringt die Routine die eigentliche Kommunikation mit Amazon und verhält sich stattdessen so, als wäre die übergebene XML-Zeichenkette als Hauptteil des Aufrufs zurückgegeben worden. Da einige dieser Routinen die Rückgabe von Daten in verschiedenen Formaten ermöglichen, kann dieses Argument

verwendet werden, um vorhandenes XML zu analysieren und die benötigten Informationen zu extrahieren. Es ist zudem für Tests hilfreich.

-bucket *bucketname*

Nahezu jeder übergeordnete Befehl muss wissen, in welchem Bucket sich die Ressourcen befinden. Diese Option legt dies fest. (Lediglich der Befehl zum Auflisten der verfügbaren Buckets benötigt diesen Parameter nicht.) Der Wert muss nicht URL-kodiert sein, selbst wenn er Sonderzeichen oder Nicht-ASCII-Zeichen enthält. Er darf führende oder nachgestellte Leerzeichen enthalten oder auch nicht – die Befehle normalisieren den Bucket. Wird dieser Parameter nicht angegeben, wird der Wert aus **S3::Configure -default-bucket** übernommen, sofern diese Zeichenfolge nicht leer ist. Hinweis: Leerzeichen und Schrägstriche werden stets an beiden Enden entfernt und der verbleibende Teil muss einen gültigen Bucket ergeben.

-resource *resourcename*

Hiermit wird die gewünschte Ressource innerhalb des Buckets festgelegt. Sie kann mit einem Schrägstrich beginnen oder auch nicht – beide Fälle werden berücksichtigt. Eine URL-Kodierung ist nicht erforderlich, selbst wenn sie Sonderzeichen oder Nicht-ASCII-Zeichen enthält.

-compare *always*|*never*|*exists*|*missing*|*newer*|*date*|*checksum*|*different*

Wenn Befehle Ressourcen in Dateien oder Dateien in Ressourcen kopieren, kann der Aufrufer festlegen, dass der Kopiervorgang übersprungen werden soll, falls der Inhalt identisch ist. Dieses Argument legt die Bedingungen fest, unter denen die Dateien kopiert werden sollen. Wird es nicht übergeben, wird das Ergebnis von **S3::Configure -default-compare** verwendet, dessen Standardwert wiederum "always" ist. Die verschiedenen Werte haben folgende Bedeutung:

always

Die Daten werden stets kopiert. Dies ist die Standardeinstellung.

never

Die Daten werden niemals kopiert. Dies ist im Wesentlichen eine leere Operation, außer in den Fällen **S3::Push** und **S3::Pull**, in denen das Flag -delete möglicherweise eine Rolle spielt.

exists

Die Daten werden nur kopiert, wenn das Ziel bereits vorhanden ist.

missing

Die Daten werden nur kopiert, wenn das Ziel noch nicht vorhanden ist.

newer

Die Daten werden kopiert, wenn das Ziel fehlt oder wenn das Datum der Quelle um mindestens **S3::Configure -slop-seconds** Sekunden neuer ist als das Datum des Ziels. Wenn die Quelle Amazon ist, wird das Datum aus dem Last-Modified-Header entnommen. Wenn die Quelle lokal ist, wird der mtime-Zeitstempel der Datei herangezogen. Wenn die Quelldaten nicht als Datei, sondern als Zeichenfolge festgelegt sind, wird der aktuelle Zeitpunkt über [clock seconds] verwendet.

date

Ähnlich wie *newer*, nur dass kopiert wird, wenn das Datum neuer *oder* älter ist.

checksum

Die MD5-Prüfsumme der lokalen Datei oder Zeichenkette wird berechnet, bei Amazon der

eTag der Ressource abgefragt und die Daten werden kopiert, falls diese voneinander abweichen. Die Daten werden ebenfalls kopiert, falls das Ziel fehlt. Hinweis: Dies kann bei großen lokalen Dateien langsam sein, sofern nicht die C-Version der MD5-Unterstützung verfügbar ist.

different

Die Daten werden kopiert, wenn das Ziel nicht vorhanden ist. Wenn das Ziel vorhanden ist und ein tatsächlicher Dateiname (anstelle einer Inhaltszeichenfolge) festgelegt wurde und das Datum der Datei vom Datum der Ressource abweicht, werden die Daten kopiert. Wenn die Daten als Inhaltszeichenfolge bereitgestellt werden, wird das "Datum" als "jetzt" behandelt, sodass es sich wahrscheinlich immer unterscheiden wird, es sei denn, der Wert für slop-seconds ist groß. Sind die Datumsangaben identisch, werden die MD5-Prüfsummen verglichen, und die Daten werden kopiert, wenn sich die Prüfsummen unterscheiden.

Hinweis: "newer" und "date" berücksichtigen den Inhalt nicht, "checksum" berücksichtigt die Datumsangaben nicht, "different" prüft beides.

S3::ListAllMyBuckets ?-blocking *boolean*? ?-parse-xml *xmlstring*? ?-result-type *REST*|*xml*|*pxml*|*dict*|*names*|*owner*?

Diese Routine führt einen GET-Aufruf beim Amazon S3-Dienst durch, der so definiert ist, dass er eine Liste der Buckets gibt, die dem durch den Autorisierungsheader identifizierten Konto gehören. (Die Schuld an den unsinnigen Namen liegt bei Amazon.)

-blocking *boolean*

Die Standarddefinition finden Sie weiter oben.

-parse-xml *xmlstring*

Die Standarddefinition finden Sie weiter oben.

-result-type *REST*

Das von **S3::REST** zurückgegebene Dictionary ist der Wert von **S3::ListAllMyBuckets**. In diesem Fall löst ein HTTP-Status, der nicht im 2XX-Bereich liegt, keinen Fehler aus. Dies lässt sich nicht mit *-parse-xml* kombinieren.

-result-type *xml*

Als Ergebnis wird das rohe XML des Hauptteils zurückgegeben (ohne jegliche Kodierung).

-result-type *pxml*

Es gibt den von xsxp::parse geparsten XML-Code des Hauptteils zurück.

-result-type *dict*

Es wird ein Dictionary mit interessanten Abschnitten des XML-Dokuments zurückgegeben. Das Dictionary enthält die folgenden Schlüssel:

Owner/ID

Die Amazon AWS-ID (im Hexadezimalformat) des Eigentümers des Buckets.

Owner/DisplayName

Der Name der Amazon AWS-ID.

Bucket/Name

Eine Liste mit Namen, jeweils einer pro Bucket.

Bucket/CreationDate

Eine Liste von Datumsangaben, jeweils eine pro Bucket, in derselben Reihenfolge wie Bucket/Name, im ISO-Format (wie von Amazon zurückgegeben).

-result-type *names*
Es wird eine Liste mit Bucket-Namen zurückgegeben, wobei alle anderen Informationen entfernt wurden. Dies ist der Standard-Ergebnistyp für diesen Befehl.

-result-type *owner*
Es wird eine Liste mit zwei Elementen zurückgegeben. Das erste Element ist die ID des Eigentümers, das zweite ist der Anzeigename des Eigentümers.

S3::PutBucket ?-bucket *bucketname*? ?-blocking *boolean*? ?-acl *{}*|*private*|*public-read*|*public-read-write*|*authenticated-read*?
Dieser Befehl erstellt einen Bucket, falls dieser noch nicht vorhanden ist. Bucket-Namen sind global eindeutig; daher kann es vorkommen, dass Sie von Amazon eine "Forbidden"-Fehlermeldung erhalten, auch wenn Sie den Bucket in **S3::ListAllMyBuckets** nicht sehen können. In **S3::SuggestBucket** finden Sie Möglichkeiten, dieses Risiko zu minimieren. Der Header "x-amz-acl" stammt aus der Option **-acl** oder, wenn nicht festgelegt, aus **S3::Configure -default-acl**.

S3::DeleteBucket ?-bucket *bucketname*? ?-blocking *boolean*?
Dieser Befehl löscht einen Bucket, sofern dieser leer ist und Sie über die entsprechenden Berechtigungen verfügen. Beachten Sie, dass die Liste der Buckets bei Amazon eine globale Ressource ist, die eine weitreichende Synchronisierung erfordert. Wenn Sie einen Bucket löschen, kann es einige Minuten (oder Stunden) dauern, bis Sie ihn neu erstellen können; bis dahin werden "Conflict"-Fehlermeldungen angezeigt.

S3::GetBucket ?-bucket *bucketname*? ?-blocking *boolean*? ?-parse-xml *xmlstring*? ?-max-count *integer*? ?-prefix *prefixstring*? ?-delimiter *delimiterstring*? ?-result-type *REST*|*xml*|*pxml*|*names*|*dict*?
Hiermit wird der Inhalt eines Buckets aufgelistet. Das heißt, es wird eine Verzeichnisauflistung der Ressourcen innerhalb eines Buckets gegeben, ohne dass dabei Benutzerdaten übertragen werden.

-bucket *bucketname*
Das übliche "Bucket"-Argument.

-blocking *boolean*
Das übliche Blockierungsargument.

-parse-xml *xmlstring*
Das Standardargument für "parse-xml".

-max-count *integer*
Wenn dieser Wert angegeben wird, entspricht er der maximalen Anzahl der Datensätze, die zurückgegeben werden. Wird kein Wert angegeben, wird der Code so lange wiederholt, bis alle Datensätze gefunden wurden. Nicht kompatibel mit -parse-xml. Hinweis: Wenn dieser Wert angegeben wird, erfolgt nur ein Aufruf von **S3::REST**. Andernfalls werden so viele Aufrufe durchgeführt, bis die Liste vollständig abgearbeitet ist, wobei die Ergebnisse im Arbeitsspeicher zwischengespeichert werden. Seien Sie daher vorsichtig, falls Sie mit sehr großen Datensätzen arbeiten.

-prefix *prefixstring*
Wenn vorhanden, beschränkt dies die Auflistung auf Ressourcen mit einem bestimmten Präfix. Ein eventuell vorhandenes führendes "/" wird entfernt.

-delimiter *delimiterstring*

Wenn angegeben, legt es ein Trennzeichen für die Auflistung fest. Durch die Angabe dieses Trennzeichens werden mehrere Ressourcen zu einem Eintrag zusammengefasst, so als ob S3 Verzeichnisse unterstützen würde. Weitere Informationen finden Sie in der Amazon-Dokumentation.

-result-type *REST*|*xml*|*pxml*|*names*|*dict*

Dies gibt das Format des vom Befehl zurückgegebenen Ergebnisses an.

REST

Wenn *-max-count* festgelegt wird, gibt **S3::REST** das Dictionary zurück. Wenn *-max-count* nicht festgelegt ist, gibt es eine Liste aller Dictionarys zurück, die durch einen oder mehrere Aufrufe von **S3::REST** zurückgegeben wurden.

xml

Wenn *-max-count* festgelegt wird, wird der von **S3::REST** zurückgegebene Hauptteil zurückgegeben. Wenn *-max-count* nicht festgelegt wird, wird eine Liste aller Hauptteile zurückgegeben, die von einem oder mehreren Aufrufen von **S3::REST** zurückgegeben wurden.

pxml

Wenn *-max-count* festgelegt wird, wird der von **S3::REST** zurückgegebene Hauptteil durch xsxp::parse geleitet und anschließend zurückgegeben. Wenn *-max-count* nicht festgelegt ist, wird jeder der von einem oder mehreren Aufrufen von **S3::REST** zurückgegebenen Hauptteile durch xsxp::parse geleitet und die Liste dann zurückgegeben.

names

Gibt eine Liste aller Namen zurück, die entweder in den Feldern "Contents/Key" oder in den Feldern "CommonPrefixes/Prefix" gefunden wurden. Wenn weder *-delimiter* noch *-max-count* festgelegt sind, gibt diese Funktion eine Liste aller Ressourcen mit dem festgelegten *-prefix* zurück.

dict

Gibt ein Dictionary zurück. (Es wird immer nur ein Dictionary zurückgegeben, auch wenn *-max-count* nicht festgelegt wurde.) Das Dictionary enthält folgende Schlüssel:

Name

Der Name des Buckets (aus dem letzten Aufruf von **S3::REST**).

Prefix

Aus dem letzten Aufruf von **S3::REST**.

Marker

Aus dem letzten Aufruf von **S3::REST**.

MaxKeys

Aus dem letzten Aufruf von **S3::REST**.

IsTruncated

Ausgehend vom letzten Aufruf von **S3::REST**; daher stets "false", wenn *-max-count* nicht angegeben ist.

NextMarker

Wird immer bereitgestellt, wenn "IsTruncated" auf "true" gesetzt ist, und berechnet, falls Amazon diesen Wert nicht bereitstellt. Kann leer sein, wenn "IsTruncated" auf "false" gesetzt ist.

Key

Eine Liste der Namen der Ressourcen im Bucket, die den Einschränkungen *-prefix* und *-delimiter* entsprechen.

LastModified
Eine Liste der Zeitangaben der Ressourcen im Bucket, in derselben Reihenfolge wie Key, in dem von Amazon zurückgegebenen Format. (Das heißt, sie wird nicht in Sekunden seit der Epoche umgewandelt.)

ETag
Eine Liste von Entity-Tags (auch bekannt als MD5-Prüfsummen) in derselben Reihenfolge wie "Key".

Size
Eine Liste der Größen der Ressourcen in Byte, in derselben Reihenfolge wie "Key".

Owner/ID
Eine Liste der Eigentümer der Ressourcen im Bucket, in derselben Reihenfolge wie "Key".

Owner/DisplayName
Eine Liste der Eigentümer der Ressourcen im Bucket, in derselben Reihenfolge wie bei Key. Dabei handelt es sich um die Anzeigenamen.

CommonPrefixes/Prefix
Eine Liste von Präfixen, die mehreren Entitäten gemeinsam sind. Diese ist nur vorhanden, wenn *-delimiter* angegeben wurde.

S3::Put ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ?-file *filename*? ?-content *contentstring*? ?-acl *private|public-read|public-read-write|authenticated-read|calc|keep*? ?-content-type *contenttypestring*? ?-x-amz-meta-* *metadatatext*? ?-compare *comparemode*?

Dieser Befehl sendet Daten zur Speicherung an eine Ressource auf den Servern von Amazon unter Verwendung des HTTP-PUT-Befehls. Er gibt den Wert 0 zurück, wenn der Modus **-compare** die Übertragung verhindert hat, den Wert 1, wenn die Übertragung erfolgreich war, oder löst einen Fehler aus, wenn die Übertragung versucht wurde, aber fehlgeschlagen ist. 5XX-Fehler des Servers und S3-Socket-Fehler werden gemäß den Einstellungen von **S3:Configure -retries** erneut versucht, bevor ein Fehler ausgelöst wird; andere Fehler lösen sofort einen Fehler aus.

-bucket
Hiermit wird der Bucket festgelegt, in den die Ressource geschrieben wird. Führende und/oder abschließende Schrägstriche werden für Sie entfernt, ebenso wie Leerzeichen.

-resource
Dies ist der vollständige Name der Ressource innerhalb des Buckets. Ein einzelner führender Schrägstrich wird entfernt, ein abschließender Schrägstrich jedoch nicht. Leerzeichen werden nicht entfernt.

-blocking
Das Standard-Blockierungsflag.

-file
Wenn dies festgelegt wird, muss der *filename* vorhanden und lesbar sein und darf keine Sonderdatei oder Verzeichnisdatei sein. [file size] muss für diese Datei gelten und darf sich

während der gesamten Laufzeit des Aufrufs nicht ändern. Der Standard-Content-Type wird anhand des Namens und/oder des Inhalts der Datei berechnet. Die Angabe dieser Option führt zu einem Fehler, wenn gleichzeitig **-content** angegeben ist; es muss jedoch mindestens eine der Optionen **-file** oder **-content** angegeben werden. (Die Datei darf nicht vorhanden oder nicht lesbar sein, wenn **-compare** *never* festgelegt ist.)

-content
Wenn dies festgelegt wird, wird der *contentstring* als Hauptteil der Ressource gesendet. Der Content-Type ist standardmäßig "application/octet-string". Es werden nur die unteren Bytes gesendet; daher sind Nicht-ASCII-Zeichen bei Bedarf vor der Übergabe an diese Routine entsprechend zu kodieren (z. B. mit [encoding convertto utf-8]). Die Angabe dieser Option führt zu einem Fehler, wenn gleichzeitig **-file** angegeben ist; es muss jedoch mindestens eine der Optionen **-file** oder **-content** angegeben werden.

-acl
Wenn nicht anders angegeben, ist der Standardwert "**S3::Configure -default-acl**". Hiermit wird der "x-amz-acl"-Header bei der PUT-Operation gesetzt. Wenn der Wert *calc* angegeben wird, wird der x-amz-acl-Header auf der Grundlage der Ein- und Ausgabeberechtigungen der hochzuladenden Datei berechnet; die Angabe von *calc* und **-content** führt zu einem Fehler. Wenn der Wert *keep* angegeben wird, wird die ACL der Ressource vor dem PUT-Vorgang ausgelesen (oder der Standardwert verwendet, falls die Ressource nicht existiert) und anschließend nach dem PUT-Vorgang wieder auf den ursprünglichen Wert zurückgesetzt (sofern sie existierte). Es tritt ein Fehler auf, wenn die Ressource zwar erfolgreich geschrieben wurde, die beibehaltene ACL anschließend jedoch nicht angewendet werden kann. Dies sollte niemals vorkommen. *Hinweis: calc* ist derzeit noch nicht vollständig implementiert.

-x-amz-meta-*
Beginnt ein Header mit "-x-amz-meta-", wird sein Inhalt dem PUT-Befehl hinzugefügt, um als Metadaten zusammen mit der Ressource gespeichert zu werden. Auch hier erfolgt keine Kodierung, und die Metadaten sollten keine Zeichen wie Zeilenumbrüche, Wagenrückläufe usw. enthalten. Am besten halten Sie sich an einfache ASCII-Zeichenfolgen oder passen die Bibliothek an mehreren Plätzen entsprechend an.

-content-type
Dies überschreibt den von **-file** berechneten Content-Type oder legt den Content-Type für **-content** fest.

-compare
Dies ist das Standardargument für den Vergleichsmodus. **S3::Put** gibt den Wert 1 zurück, wenn die Daten kopiert wurden, bzw. den Wert 0, wenn die Daten aufgrund des Vergleichsmodus übersprungen wurden, was darauf hinweist, dass sie übersprungen werden sollten.

S3::Get ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ? -compare *comparemode*? ?-file *filename*? ?-content *contentvarname*? ?-timestamp *aws|now*? ?-headers *headervarname*?

Dieser Befehl ruft Daten von einer Ressource auf den S3-Servern von Amazon ab und verwendet dabei den HTTP-GET-Befehl. Er gibt den Wert 0 zurück, wenn der Modus **-compare** die Übertragung verhindert hat, den Wert 1, wenn die Übertragung erfolgreich war, oder löst einen Fehler aus, wenn die Übertragung versucht wurde, aber fehlgeschlagen ist. 5XX-Fehler

des Servers und S3-Socket-Fehler werden gemäß den Einstellungen von **S3:Configure** erneut versucht, bevor ein Fehler ausgelöst wird; andere Fehler werden sofort ausgelöst. Hinweis: Die Authentifizierung erfolgt stets als der über **S3::Configure -accesskeyid** konfigurierte Benutzer. Für nicht authentifizierte GET-Anfragen dient das Tcllib-Paket http.

-bucket
Hiermit wird der Bucket festgelegt, aus dem die Ressource gelesen wird. Führende und/oder abschließende Schrägstriche werden für Sie entfernt, ebenso wie Leerzeichen.

-resource
Dies ist der vollständige Name der Ressource innerhalb des Buckets. Ein einzelner führender Schrägstrich wird entfernt, ein abschließender Schrägstrich jedoch nicht. Leerzeichen werden nicht entfernt.

-blocking
Das Standard-Blockierungsflag.

-file
Wenn dies festgelegt wird, wird der Inhalt der Ressource schrittweise in diese Datei eingelesen, ohne ihn zuvor vollständig in den Arbeitsspeicher zu laden. Das übergeordnete Verzeichnis muss bereits vorhanden sein. Wenn die Datei bereits existiert, muss sie beschreibbar sein. Wird während des Prozesses ein Fehler ausgelöst und die Datei existierte bereits, kann sie überschrieben werden. Wird während des Prozesses ein Fehler ausgelöst und die Datei existierte noch nicht, werden alle bereits geschriebenen Teile gelöscht. Die Angabe dieser Option führt zu einem Fehler, wenn gleichzeitig **-content** angegeben ist; es muss jedoch mindestens eine der Optionen **-file** oder **-content** angegeben werden.

-timestamp
Dies gilt nur in Verbindung mit **-file**. Es kann als *now* oder *aws* angegeben werden. Die Standardeinstellung ist *now*. Bei *now* wird das Änderungsdatum der Datei dem System überlassen. Wenn *aws* aktiv ist, wird die Mtime der Datei so gesetzt, dass sie mit dem "Last-Modified"-Header der Ressource übereinstimmt, wodurch die beiden Werte für **-compare** *date* oder **-compare** *newer* entsprechend synchronisiert werden.

-content
Wenn dies festgelegt wird, ist *contentvarname* eine Variable im Geltungsbereich des Aufrufers (nicht unbedingt global), die den Wert des Hauptteils der Ressource entgegennimmt. Es erfolgt keine Kodierung; wenn die Ressource (beispielsweise) eine UTF-8-Bytefolge darstellt, liefert [encoding convertfrom utf-8] eine gültige UTF-8-Zeichenkette. Wenn dies festgelegt wird, wird **-compare** ignoriert, es sei denn, es handelt sich um *never*; in diesem Fall erfolgt keine Zuweisung an *contentvarname*. Die Angabe dieses Parameters ist ein Fehler, wenn gleichzeitig **-file** angegeben ist; es muss jedoch mindestens einer der Parameter **-file** oder **-content** angegeben werden.

-compare
Dies ist das Standardargument für den Vergleichsmodus. **S3::Get** gibt den Wert 1 zurück, wenn die Daten kopiert wurden, oder den Wert 0, wenn die Daten aufgrund des Vergleichsmodus übersprungen wurden, was darauf hinweist, dass sie übersprungen werden sollten.

-headers
Wenn dies festgelegt wird, werden die beim Abruf erzeugten Header als Dictionary in der

angegebenen Variable gespeichert. Dazu gehören Content-Type- und x-amz-meta-*-Header sowie die üblichen HTTP-Header, die x-amz-id-Debugging-Header und so weiter. Wenn keine Datei abgerufen wird (aufgrund von **-compare** oder anderen Fehlern), erfolgt keine Zuweisung an diese Variable.

S3::Head ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ?-dict *dictvarname*? ?-headers *headersvarname*? ?-status *statusvarname*?

Dieser Befehl fordert den HEAD-Befehl von der Ressource an. Er gibt zurück, ob als Ergebnis der Anfrage ein 2XX-Code zurückgegeben wurde, und löst dabei niemals einen S3-Remote-Fehler aus. Das heißt, wenn dieser Befehl den Wert 1 zurückgibt, ist die Ressource vorhanden und zugänglich. Wenn der Befehl den Wert 0 zurückgibt, ist ein Fehler aufgetreten; Einzelheiten können dem Ergebnis **-status** entnommen werden.

-bucket
Hiermit wird der Bucket festgelegt, aus dem die Ressource gelesen wird. Führende und/oder abschließende Schrägstriche werden für Sie entfernt, ebenso wie Leerzeichen.

-resource
Dies ist der vollständige Name der Ressource innerhalb des Buckets. Ein einzelner führender Schrägstrich wird entfernt, ein abschließender Schrägstrich jedoch nicht. Leerzeichen werden nicht entfernt.

-blocking
Das Standard-Blockierungsflag.

-dict
Wenn angegeben, wird das aus dem Aufruf von **S3::REST** resultierende Dictionary der angegebenen (nicht unbedingt globalen) Variablen im Geltungsbereich des Aufrufers zugewiesen.

-headers
Wenn angegeben, wird das Dictionary der Header aus dem Ergebnis den angegebenen (nicht unbedingt globalen) Variablen im Geltungsbereich des Aufrufers zugewiesen.

-status
Wenn eine Variable festgelegt wird, wird der angegebenen (nicht unbedingt globalen) Variable im Gültigkeitsbereich des Aufrufers eine Liste mit zwei Elementen zugewiesen. Das erste Element ist der dreistellige HTTP-Statuscode, während das zweite Element die HTTP-Meldung ist (z. B. "OK" oder "Forbidden").

S3::GetAcl ?-blocking *boolean*? ?-bucket *bucketname*? -resource *resourcename* ?-result-type *REST*|*xml*|*pxml*?

Dieser Befehl ruft die ACL der angegebenen Ressource ab oder löst einen Fehler aus, falls diese nicht verfügbar ist.

-blocking *boolean*
Die Standarddefinition finden Sie weiter oben.

-bucket
Hiermit wird der Bucket festgelegt, aus dem die Ressource gelesen wird. Führende und/oder abschließende Schrägstriche werden für Sie entfernt, ebenso wie Leerzeichen.

-resource
Dies ist der vollständige Name der Ressource innerhalb des Buckets. Ein einzelner führender Schrägstrich wird entfernt, ein abschließender Schrägstrich jedoch nicht.

Leerzeichen werden nicht entfernt.

-parse-xml *xml*
Das XML aus einem früheren GetACL-Aufruf kann übergeben werden, um in eine Dictionary-Form geparst zu werden. In diesem Fall muss -result-type den Typ "pxml" oder "dict" haben.

-result-type *REST*
Das von **S3::REST** zurückgegebene Dictionary ist der Wert von **S3::GetAcl**. In diesem Fall löst ein HTTP-Status, der nicht der Klasse 2XX angehört, keinen Fehler aus.

-result-type *xml*
Als Ergebnis wird das rohe XML des Hauptteils zurückgegeben (ohne jegliche Kodierung).

-result-type *pxml*
Es gibt den von xsxp::parse geparsten XML-Code des Hauptteils zurück.

-result-type *dict*
Diese Funktion ruft die ACL ab, wertet sie aus und gibt ein Dictionary mit zwei Elementen zurück.

Das erste Element hat den Schlüssel "owner", dessen Wert die kanonische ID des Eigentümers der Ressource ist.

Das zweite Element hat den Schlüssel "acl", dessen Wert ein Dictionary ist. Jeder Schlüssel in diesem Dictionary entspricht einer der Amazon-Berechtigungen, nämlich "READ", "WRITE", "READ_ACP", "WRITE_ACP" oder "FULL_CONTROL". Jeder Wert eines Schlüssels ist eine Liste kanonischer IDs oder Gruppen-URLs, die über diese Berechtigung verfügen. Die Elemente sind in der Liste nicht in einer bestimmten Reihenfolge angeordnet, und es sind nicht unbedingt alle Schlüssel vorhanden. Anzeigenamen werden nicht zurückgegeben, da sie nicht besonders nützlich sind; bei Bedarf liefert pxml diese.

S3::PutAcl ?-blocking *boolean*? ?-bucket *bucketname*? -resource *resourcename* ?-acl *new-acl*?
Damit wird die ACL für die angegebene Ressource festgelegt. Die Funktion gibt das in die ACL geschriebene XML zurück oder löst einen Fehler aus, falls ein Fehler aufgetreten ist.

-blocking *boolean*
Die Standarddefinition finden Sie weiter oben.

-bucket
Hiermit wird der Bucket festgelegt, aus dem die Ressource gelesen wird. Führende und/oder abschließende Schrägstriche werden für Sie entfernt, ebenso wie Leerzeichen.

-resource
Dies ist der vollständige Name der Ressource innerhalb des Buckets. Ein einzelner führender Schrägstrich wird entfernt, ein abschließender Schrägstrich jedoch nicht. Leerzeichen werden nicht entfernt.

-owner
Wenn dieser Wert angegeben wird, wird davon ausgegangen, dass er mit dem Eigentümer der Ressource übereinstimmt. Andernfalls muss möglicherweise ein GET-Aufruf an die Ressource gesendet werden, um den Eigentümer zu ermitteln. Wenn Sie den Eigentümer bereits kennen (beispielsweise aus einem Aufruf von **S3::GetAcl**), können Sie den Wert

des Schlüssels "owner" als Wert dieser Option übergeben; dieser wird dann bei der Erstellung des XML-Dokuments verwendet.

-acl
Wenn diese Option festgelegt wird, liefert sie die ACL, die der Aufrufer in die Ressource schreiben möchte. Wird sie nicht festgelegt oder ist sie leer, wird der Wert aus **S3::Configure -default-acl** übernommen. Die ACL wird mit einem PUT-Befehl in die Ressource "?acl" geschrieben.

Wenn der an diese Option übergebene Wert mit "<" beginnt, wird er als Hauptteil interpretiert, der per PUT an die ACL-Ressource gesendet werden soll.

Stimmt der Wert mit einem der Standard-Amazon-x-amz-acl-Header überein (d. h. einer vordefinierten Zugriffsrichtlinie), wird dieser Header in XML übersetzt und anschließend angewendet. Die vordefinierten Zugriffsrichtlinien lauten "private", "public-read", "public-read-write" und "authenticated-read" (in Kleinbuchstaben).

Andernfalls wird davon ausgegangen, dass der Wert ein Dictionary ist, das als "acl"-Untereintrag innerhalb des von **S3::GetAcl -result-type dict** zurückgegebenen Dictionarys formatiert ist. Der entsprechende Wert wird generiert und auf die Ressource angewendet. Hinweis: Ein Wert, der "//" enthält, wird als Gruppe angesehen, ein Wert, der "@" enthält, wird als "AmazonCustomerByEmail" angesehen und der Wert ansonsten als kanonische Amazon-ID angesehen.

Hinweis: Sie können den Eigentümer nicht ändern; daher führt der Aufruf von "GetAcl" für eine Ressource, die einem bestimmten Benutzer gehört, und die anschließende Anwendung über "PutAcl" auf eine Ressource, die einem anderen Benutzer gehört, möglicherweise nicht genau zu dem Ergebnis, das Sie erwarten.

S3::Delete ?-bucket *bucketname*? -resource *resourcename* ?-blocking *boolean*? ? -status *statusvar*?
Dieser Befehl löscht die angegebene Ressource aus dem angegebenen Bucket. Er gibt den Wert 1 zurück, wenn die Ressource erfolgreich gelöscht wurde, andernfalls den Wert 0. Er gibt den Wert 0 zurück, anstatt einen S3-Remote-Fehler auszulösen.

-bucket
Hiermit wird der Bucket festgelegt, aus dem die Ressource gelöscht werden soll. Führende und/oder abschließende Schrägstriche werden für Sie entfernt, ebenso wie Leerzeichen.

-resource
Dies ist der vollständige Name der Ressource innerhalb des Buckets. Ein einzelner führender Schrägstrich wird entfernt, ein abschließender Schrägstrich jedoch nicht. Leerzeichen werden nicht entfernt.

-blocking
Das Standard-Blockierungsflag.

-status
Wenn angegeben, wird die angegebene (nicht unbedingt globale) Variable im Gültigkeitsbereich des Aufrufers auf eine Liste mit zwei Elementen gesetzt. Das erste Element ist der dreistellige HTTP-Statuscode. Das zweite Element ist die HTTP-Meldung (z.

B. "OK" oder "Forbidden"). Hinweis: Das DELETE-Ergebnis von Amazon ist bei Erfolg 204; dieser Code weist darauf hin, dass der zurückgegebene Hauptteil keinen Inhalt enthält.

S3::Push ?-bucket *bucketname*? -directory *directoryname* ?-prefix *prefixstring*? ? -compare *comparemode*? ?-x-amz-meta-* *metastring*? ?-acl *aclcode*? ?-delete *boolean* ? ?-error *throw|break|continue*? ?-progress *scriptprefix*?

Hierbei wird ein lokales Verzeichnis mit einem Remote-Bucket synchronisiert, indem die Unterschiede mithilfe von **S3::Put** übertragen werden. Zu beachten ist, dass Änderungen, die im Bucket, jedoch nicht lokal vorgenommen wurden, verloren gehen können. Daher handelt es sich hierbei nicht um eine allgemeine Primitive für die bidirektionale Synchronisation. (Siehe hierzu **S3::Sync**.) Zudem ist zu beachten, dass bei Ressourcennamen die Groß- und Kleinschreibung beachtet wird; daher kann eine Änderung der Groß-/Kleinschreibung einer Datei auf einem Windows-Rechner zu ansonsten unnötigen Übertragungen führen. Ferner werden nur reguläre Dateien berücksichtigt; Geräte, Pipes, symbolische Links und Verzeichnisse werden daher nicht kopiert.

-bucket

Hiermit wird der Name des Buckets festgelegt, in den die Daten übertragen werden sollen.

-directory

Hiermit wird der Name des lokalen Verzeichnisses angegeben, aus dem die Dateien entnommen werden sollen. Es muss vorhanden sein, über [glob] lesbar sein und so weiter. Wenn nur einige der darin enthaltenen Dateien lesbar sind, führt **S3::Push** die lesbaren Dateien mit dem Befehl "PUT" aus und gibt in seinen Ergebnissen die Liste der Dateien zurück, die nicht geöffnet werden konnten.

-prefix

Hiermit wird der Name festgelegt, der allen Ressourcen hinzugefügt wird. Das heißt, er entspricht dem Remote-Äquivalent von **-directory**. Wenn kein Präfix angegeben wird, wird das Stammverzeichnis des Buckets als Remote-Verzeichnis behandelt. Beispiele sollen dies verdeutlichen.

```
S3::Push -bucket test -directory /tmp/xyz -prefix hello/world
```

In diesem Beispiel wird die Datei "/tmp/xyz/pdq.html" auf den Servern von Amazon unter der Adresse "http://s3.amazonaws.com/test/hello/world/pdq.html" gespeichert. Ebenso wird "/tmp/xyz/abc/def/Hello" auf den Servern von Amazon als "http://s3.amazonaws.com/test/hello/world/abc/def/Hello" gespeichert. Ohne die Option **-prefix** würde "/tmp/xyz/pdq.html" als "http://s3.amazonaws.com/test/pdq.html" gespeichert werden.

-blocking

Dies ist die übliche Blockierungsoption.

-compare

Sofern vorhanden, wird dieser Wert bei jedem Aufruf von **S3::Put** übergeben. Wenn kein Wert festgelegt ist, wird selbstverständlich **S3::Configure -default-compare** verwendet.

-x-amz-meta-*

Wenn dieser Wert vorhanden ist, wird er an jeden Aufruf von **S3::Put** übergeben. Alle kopierten Dateien verfügen über dieselben Metadaten.

-acl

Sofern vorhanden, wird dieser Wert bei jedem Aufruf von **S3::Put** übergeben.

-delete

Der Standardwert ist "false". Wenn der Wert "true" ist, werden Ressourcen im Zielverzeichnis, die nicht im Quellverzeichnis vorhanden sind, mit **S3::Delete** gelöscht. Da nur reguläre Dateien berücksichtigt werden, verhindert das Vorhandensein eines symbolischen Links, einer Pipe, eines Geräts oder eines Verzeichnisses im lokalen Quellverzeichnis *nicht* das Löschen einer Remote-Ressource mit einem entsprechenden Namen.

-error

Diese Option steuert das Verhalten von **S3::Push**, falls **S3::Put** einen Fehler auslöst. Hinweis: Fehler, die im lokalen Dateisystem oder beim Einlesen der Ressourcenliste im Remote-Bucket auftreten, lösen stets einen Fehler aus. Diese Option ermöglicht die Steuerung von "teilweisen" Fehlern, bei denen einige Dateien kopiert wurden, andere jedoch nicht. **S3::Delete** wird stets abgeschlossen, wobei Fehler lediglich im Rückgabewert protokolliert werden.

throw

Der Fehler wird mit demselben Fehlercode erneut ausgelöst.

break

Die Verarbeitung endet, ohne dass ein Fehler ausgelöst wird; der Fehler wird im Rückgabewert erfasst, und der Befehl gibt normal zurück. Die Aufrufe von **S3::Delete** werden nicht gestartet.

continue

Dies ist die Standardeinstellung. Die Verarbeitung wird fortgesetzt, ohne dass ein Fehler ausgelöst wird; der Fehler wird im Rückgabewert vermerkt, und es wird mit der nächsten zu kopierenden Datei im lokalen Verzeichnis fortgefahren.

-progress

Wenn dies festgelegt wird und das angegebene Skriptpräfix nicht leer ist, wird das angegebene Skriptpräfix an verschiedenen Stellen der Verarbeitung mehrmals im Kontext des Aufrufers mit zusätzlichen Argumenten aufgerufen. Dies ermöglicht eine Fortschrittsanzeige, ohne die Verarbeitung in den Hintergrund zu verlegen. Das angegebene Präfix wird mit zusätzlichen Argumenten aufgerufen, wobei das erste zusätzliche Argument angibt, über welchen Teil des Prozesses berichtet wird. Das Präfix wird zunächst mit *args* als erstem zusätzlichen Argument und einem Dictionary, das die normalisierten Argumente des Aufrufs von **S3::Push** darstellt, als zweitem zusätzlichen Argument aufgerufen. Anschließend wird das Präfix mit *local* als erstem zusätzlichen Argument und einer Liste der zu berücksichtigenden Dateisuffixe als zweitem Argument aufgerufen. Danach wird das Präfix mit *remote* als erstem zusätzlichen Argument und einer Liste der im Remote-Bucket vorhandenen Dateisuffixe als zweitem zusätzlichen Argument aufgerufen. Anschließend wird für jede Datei in der lokalen Liste das Präfix mit *start* als erstem Zusatzargument und dem gemeinsamen Suffix als zweitem Zusatzargument aufgerufen. Wenn **S3::Put** für diese Datei zurückgibt, wird das Präfix mit *copy* als erstem zusätzlichen Argument, dem gemeinsamen Suffix als zweitem zusätzlichen Argument und einem dritten Argument aufgerufen, das "copied" ist (wenn **S3::Put** die Ressource gesendet hat), "skipped" (falls **S3::Put** auf Grundlage von **-compare** entschieden hat, dies nicht zu tun) oder den Fehlercode, den **S3::Put** aufgrund unerwarteter Fehler

ausgelöst hat (in diesem Fall ist das dritte Argument eine Liste, die mit "S3" beginnt). Wenn alle Dateien übertragen wurden, kann das Präfix nullmal oder mehrmals mit *delete* als erstem zusätzlichen Argument und dem Suffix der zu löschenden Ressource als zweitem zusätzlichen Argument aufgerufen werden, wobei das dritte Argument entweder eine leere Zeichenkette ist (falls das Löschen erfolgreich war) oder der Fehlercode von **S3::Delete**, falls es fehlgeschlagen ist. Schließlich wird das Präfix mit *finished* als erstem zusätzlichen Argument und dem Wert als zweitem zusätzlichen Argument aufgerufen.

Das Ergebnis dieses Befehls ist ein Dictionary. Die Schlüssel sind die Suffixe (d. h. der gemeinsame Teil des Pfads nach "**-directory**" und "**-prefix**"), während die Werte je nach Fall entweder "copied", "skipped" (wenn **-compare** angibt, dass die Datei nicht kopiert werden soll) oder gegebenenfalls der von **S3::Put** ausgelöste Fehlercode. Wenn **-delete** true ist, können zudem Einträge für Suffixe mit dem Wert "deleted" oder "notdeleted" vorhanden sein, die angeben, ob der versuchte Vorgang **S3::Delete** erfolgreich war oder nicht. Das Rückgaberesultat enthält ein weiteres Schlüssel-Wert-Paar, dessen Schlüssel die leere Zeichenkette ist und dessen Wert ein verschachteltes Dictionary ist. Zu den Schlüsseln dieses verschachtelten Dictionaries gehören "filescopied" (die Anzahl der erfolgreich kopierten Dateien), "bytescopied" (die Anzahl der Datenbytes in den kopierten Dateien, ohne Header, Metadaten, usw.), "compareskipped" (die Anzahl der Dateien, die aufgrund des Modus **-compare** nicht kopiert wurden), "errorskipped" (die Anzahl der Dateien, die aufgrund ausgelöster Fehler nicht kopiert wurden), "filesdeleted" (die Anzahl der Ressourcen, die gelöscht wurden, da lokal keine entsprechenden Dateien vorhanden waren, oder 0, falls **-delete** auf "false" gesetzt ist) sowie "filesnotdeleted" (die Anzahl der Ressourcen, deren Löschung versucht wurde, jedoch fehlschlug).

Hinweis: Dies ist derzeit etwas ineffizient implementiert. Es wird zunächst die Bucket-Liste (einschließlich Zeitstempel und eTags) abgerufen, anschließend wird **S3::Put** aufgerufen, das wiederum HEAD verwendet, um die Zeitstempel und eTags erneut zu ermitteln. Eine Korrektur dieses Problems ohne API-Änderung ist für ein zukünftiges Upgrade geplant.

S3::Pull ?-bucket *bucketname*? -directory *directoryname* ?-prefix *prefixstring*? ?-blocking *boolean*? ?-compare *comparemode*? ?-delete *boolean*? ?-timestamp *aws*|*now*? ?-error *throw*|*break*|*continue*? ?-progress *scriptprefix*?

Dadurch wird ein Remote-Bucket mit einem lokalen Verzeichnis synchronisiert, indem die Unterschiede mithilfe von **S3::Get** abgerufen werden. Sollten lokal Änderungen vorgenommen worden sein, die im Bucket nicht vorhanden sind, können diese Unterschiede verloren gehen. Es handelt sich hierbei nicht um einen allgemeinen Mechanismus zur bidirektionalen Synchronisation. (Siehe hierzu **S3::Sync**.) Dabei werden bei Bedarf Verzeichnisse angelegt; neue Verzeichnisse werden mit Standardberechtigungen erstellt. Beachten Sie, dass bei Ressourcennamen die Groß- und Kleinschreibung beachtet werden muss; daher kann eine Änderung der Groß-/Kleinschreibung einer Datei auf einem Windows-Rechner zu ansonsten unnötigen Übertragungen führen. Versuchen Sie außerdem, keine Daten in Ressourcen zu speichern, deren Namen mit einem Schrägstrich enden oder die Präfixe von Ressourcen sind, die ansonsten mit einem Schrägstrich beginnen würden; d. h., verwenden Sie diese Funktion nicht, wenn Sie Daten in Ressourcen speichern, deren Namen lokal Verzeichnisse sein müssen.

Hinweis: Dies ist derzeit etwas ineffizient implementiert. Es ruft die Bucket-Liste

(einschließlich Zeitstempel und eTags) ab und ruft dann **S3::Get** auf, das HEAD verwendet, um die Zeitstempel und eTags erneut zu ermitteln. Eine Korrektur dieses Problems ohne API-Änderung ist für ein zukünftiges Upgrade geplant.

-bucket
Hiermit wird der Name des Buckets angegeben, aus dem Daten abgerufen werden sollen.

-directory
Hiermit wird der Name des lokalen Verzeichnisses angegeben, in das Dateien geschrieben werden sollen. Es muss vorhanden sein, über [glob] lesbar sein, Schreibzugriff für die Erstellung von Dateien bieten und so weiter. Wenn nur einige der darin enthaltenen Dateien beschreibbar sind, ruft **S3::Pull** die beschreibbaren Dateien mit GET ab und gibt in seinen Ergebnissen die Liste der Dateien zurück, die nicht geöffnet werden konnten.

-prefix
Das Präfix der Ressourcen, die für den Abruf in Betracht gezogen werden. Weitere Einzelheiten, Beispiele usw. finden Sie unter **S3::Push**. (Natürlich dient **S3::Pull** dem Lesen und nicht dem Schreiben, doch das Präfix wird auf ähnliche Weise behandelt.)

-blocking
Dies ist die übliche Blockierungsoption.

-compare
Dieser Wert wird an jeden Aufruf von **S3::Get** übergeben, sofern er angegeben ist. Ist dies nicht der Fall, wird selbstverständlich **S3::Configure -default-compare** verwendet.

-timestamp
Dieser Wert wird, wenn er angegeben ist, bei jedem Aufruf von **S3::Get** übergeben.

-delete
Wenn diese Option festgelegt ist und den Wert true annimmt, werden Dateien, die im **-directory** vorhanden sind, aber nicht im **-prefix** enthalten sind, gelöscht, nachdem alle Ressourcen kopiert wurden. Zudem werden leere Verzeichnisse (mit Ausnahme des obersten **-directory**) gelöscht, da Amazon S3 das Konzept eines leeren Verzeichnisses nicht kennt.

-error
Eine Beschreibung dieser Option finden Sie unter **S3::Push**.

-progress
Eine Beschreibung dieser Option finden Sie unter **S3::Push**. Der Unterschied besteht darin, dass lokale Verzeichnisse mit einem abschließenden Schrägstrich angegeben werden können, um zu kennzeichnen, dass es sich um Verzeichnisse handelt.

Der Wert dieses Befehls ist ein Dictionary. Er entspricht in Form und Bedeutung der Beschreibung des Rückgabewerts von **S3::Push**. Der einzige Unterschied besteht darin, dass Verzeichnisse, deren Name mit einem abschließenden Schrägstrich endet, ebenfalls enthalten sein können, sofern sie leer sind und gelöscht werden.

S3::Toss ?-bucket *bucketname*? -prefix *prefixstring* ?-blocking *boolean*? ?-error *throw|break|continue*? ?-progress *scriptprefix*?

Dadurch werden einige oder alle Ressourcen innerhalb eines Buckets gelöscht. Würde Amazon echte Verzeichnisse implementieren, würde dies als "rekursives Löschen" gelten.

-bucket
Der Bucket, aus dem Ressourcen gelöscht werden sollen.

-blocking
Die übliche Blockierungsoption.

-prefix
Das Präfix für die zu löschenden Ressourcen. Alle Ressourcen, deren Name mit dieser Zeichenfolge beginnt, werden gelöscht. Diese Angabe ist erforderlich. Um den gesamten Inhalt des Buckets zu löschen, geben Sie als Präfix eine leere Zeichenfolge an.

-error
Wenn der Wert "throw" ist, gibt **S3::Toss** alle auftretenden Fehler erneut weiter. Wenn der Wert "break" ist, gibt **S3::Toss** nach dem ersten Fehler einen normalen Rückgabewert zurück und protokolliert diesen Fehler im Rückgabewert. Wenn der Wert "continue" (Standardeinstellung) ist, fährt **S3::Toss** fort und listet alle Fehler im Rückgabewert auf.

-progress
Wenn dies festgelegt ist und es sich nicht um eine leere Zeichenkette handelt, wird das Skriptpräfix im Kontext des Aufrufers mehrmals aufgerufen, wobei jeweils zusätzliche Argumente angehängt werden. Zunächst wird es so aufgerufen, dass das erste zusätzliche Argument *args* und das zweite die verarbeitete Liste der Argumente für **S3::Toss** ist. Anschließend wird es mit *remote* als erstem zusätzlichen Argument und der Liste der zu löschenden Suffixe im Bucket als zweitem zusätzlichen Argument aufgerufen. Anschließend wird es aufgerufen, wobei das erste zusätzliche Argument *delete* ist, das zweite zusätzliche Argument das gelöschte Suffix und das dritte zusätzliche Argument "deleted" oder "notdeleted", je nachdem, ob **S3::Delete** einen Fehler ausgelöst hat. Schließlich wird das Skript-Präfix mit dem ersten zusätzlichen Argument "finished" und dem zweiten zusätzlichen Argument, dem Wert der Rückgabe, aufgerufen.

Der Rückgabewert ist ein Dictionary. Die Schlüssel sind die Suffixe der Dateien, die **S3::Toss** zu löschen versucht hat, und deren Werte entweder die Zeichenfolge "deleted" oder "notdeleted" sind. Zudem gibt es ein weiteres Schlüssel-Wert-Paar, dessen Schlüssel die leere Zeichenkette ist und dessen Wert ein eingebettetes Dictionary ist. Zu den Schlüsseln dieses eingebetteten Dictionarys gehören "filesdeleted" und "filesnotdeleted", die jeweils ganzzahlige Werte enthalten.

Einschränkungen

- Die MD5-Prüfung mit reinem Tcl ist langsam. Wenn Sie Dateien im Megabyte-Bereich verarbeiten, sollten Sie sicherstellen, dass eine Binärunterstützung verfügbar ist.
- Die Befehle **S3::Pull** und **S3::Push** rufen eine Verzeichnisauflistung ab, die Zeitstempel und MD5-Hashes enthält, und rufen anschließend **S3::Get** und **S3::Put** auf. Wenn ein komplexer **-compare**-Modus festgelegt wird, rufen **S3::Get** und **S3::Put** für jede Datei eine HEAD-Operation auf, um die Zeitstempel und MD5-Hashes jeder Ressource erneut abzurufen. Es ist zu erwarten, dass dieses Problem in einer zukünftigen Version dieses Pakets ohne Änderungen an der API behoben wird.
- Die Befehle **S3::Pull** und **S3::Push** rufen eine Verzeichnisauflistung ab, ohne **-max-count** zu verwenden. Das gesamte Verzeichnis wird auf einmal in den Arbeitsspeicher geladen. Bei sehr großen Buckets könnte dies zu einem Leistungsproblem führen. Der Autor hat derzeit nicht vor, dieses Verhalten zu ändern. Willkommen bei Open Source.
- **S3::Sync** wurde bislang weder konzipiert noch implementiert. Die Absicht besteht darin, Änderungen synchron zu halten, sodass Änderungen sowohl am Bucket als auch am lokalen

Verzeichnis vorgenommen und anschließend durch **S3::Sync** zusammengeführt werden können.
- Auch **-compare** *calc* ist noch nicht vollständig implementiert. Dies liegt in erster Linie daran, dass Windows keine geeignete Methode zur Unterscheidung zwischen lokalen Dateien mit den Berechtigungen "public-read" und "public-read-write" bereitstellt. Unterstützung bei der Klärung der TWAPI-Spezifikationen in diesem Zusammenhang wäre willkommen. Auch die U**X-Semantik lässt sich nur schwer direkt abbilden. Einzelheiten stehen im Quellcode. Hinweis: Es gibt keine Tests für calc, da diese Funktion noch nicht fertiggestellt ist.
- Die HTTP-Verarbeitung wird innerhalb der Bibliothek implementiert, anstatt ein "echtes" HTTP-Paket zu verwenden. Daher werden mehrzeilige Header (noch) nicht korrekt verarbeitet. Fügen Sie keine Wagenrückläufe oder Zeilenvorschübe in x-amz-meta-*-Header, Content-Type-Werte usw. ein. Der Autor geht derzeit nicht davon aus, dies zu verbessern.
- Intern sind **S3::Push**, **S3::Pull** und **S3::Toss** einander sehr ähnlich und sollten überarbeitet werden.
- Die Idee, mit **-compare** *never* und **-delete** *true* Dateien zu löschen, die an einer Stelle gelöscht wurden, an der anderen jedoch noch nicht, ohne dabei geänderte Dateien zu kopieren, ist noch nicht getestet worden.

Anwendungsempfehlungen

So rufen Sie ein "Verzeichnis" aus einem Bucket ab, nehmen Änderungen vor und speichern es wieder zurück:

```
file mkdir ./tempfiles
S3::Pull -bucket sample -prefix of/interest -directory ./tempfiles \
  -timestamp aws
do_my_process ./tempfiles other arguments
S3::Push -bucket sample -prefix of/interest -directory ./tempfiles \
  -compare newer -delete true
```

Um Dateien lokal zu löschen, die aus S3 gelöscht wurden, ohne dabei sonstige Änderungen an den Dateien vorzunehmen:

```
S3::Pull -bucket sample -prefix of/interest -directory ./myfiles \
  -compare never -delete true
```

Zukünftige Entwicklungen

Der Autor beabsichtigt, neben der Fertigstellung der noch unvollständigen Funktionen an mehreren weiteren Projekten im Zusammenhang mit diesem Paket zu arbeiten.

Zunächst einmal ist ein Befehlszeilenprogramm nützlich, mit dem sich Buckets durchsuchen und Dateien aus Shell-Skripten und von der Befehlszeile übertragen lassen.

Zweitens wäre ein GUI-basiertes Programm nützlich, das eine visuelle Bearbeitung von Bucket- und Ressourcenbäumen ermöglicht, ähnlich wie der Windows Explorer.

Drittens ein Befehlszeilenprogramm (und möglicherweise auch ein GUI-basiertes Programm) namens "OddJob", das S3 nutzt, um die Rechenaufgaben zwischen mehreren Servern, auf denen OddJob läuft, zu synchronisieren. Es wird ein S3-Bucket mit einer Reihe von auszuführenden Skripten eingerichtet, und das OddJob-Programm kann auf mehreren Rechnern aufgerufen werden, um die Skripte auf allen Rechnern auszuführen, wobei jeder Rechner nach Abschluss

einer Aufgabe zur nächsten noch nicht gestarteten Aufgabe übergeht. Das Programm befindet sich noch in der Entwicklungsphase und ist in erster Linie für den Einsatz auf der Amazon Elastic Compute Cloud vorgesehen.

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *amazon-s3* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

2006,2008 Darren New. All Rights Reserved. See LICENSE.TXT for terms.

xsxp

Stichwörter: xsxp, dom, parser, xml

Name

xsxp – eXtremely Simple Xml Parser

Übersicht

package require Tcl 8.5 9
package require xsxp 1.1
package require xml
xsxp::parse *xml*
xsxp::fetch *pxml path* ?*part*?
xsxp::fetchall *pxml_list path* ?*part*?
xsxp::only *pxml tagname*
xsxp::prettyprint *pxml* ?*chan*?

Beschreibung

Dieses Paket bietet eine einfache Schnittstelle, um XML in eine Liste aus reinen Werten zu parsen. Es stellt zudem Zugriffsfunktionen bereit, um bestimmte Untertags zu extrahieren, ähnlich wie beim DOM-Zugriff. Dieses Paket wurde für das Amazon-S3-Zugriffspaket von Darren New geschrieben und wird dort verwendet.

Das ist zwar ziemlich dürftig, aber ich benötigte so etwas für S3, und damals funktionierte TclDOM aufgrund von Problemen mit den Versionsnummern nicht mit dem neuen Tcl 8.5.

Zudem handelt es sich hierbei um eine reine Wertimplementierung. So gibt es beispielsweise im Falle eines ausgelösten Fehlers nichts aufzuräumen. Dies vereinfacht den Code bei ausreichend kleinen XML-Dokumenten, was durch Amazons S3 gewährleistet wird.

Copyright 2006 Darren New. Alle Rechte vorbehalten. ES WERDEN KEINERLEI GEWÄHRLEISTUNGEN JEGLICHER ART ÜBERNOMMEN. DAS KOPIEREN ODER DIE NUTZUNG BEFREIT DEN AUTOR IN JEDER HINSICHT VON JEGLICHER HAFTUNG. Diese Software wird unter im Wesentlichen denselben Bedingungen wie Tcl lizenziert. Die Bedingungen finden Sie in der Datei "LICENSE.txt".

Befehle

Das Paket implementiert fünf relativ einfache Prozeduren. Eine parst, eine dient der Fehlersuche, und die übrigen extrahieren verschiedene Teile des analysierten Dokuments zur weiteren Verarbeitung.

xsxp::parse *xml*

Diese Funktion analysiert ein XML-Dokument (unter Verwendung des Standardmoduls "xml" der tcllib in einer SAX-ähnlichen Weise) und erstellt eine Datenstruktur, die bei erfolgreicher Analyse zurückgegeben wird. Der Rückgabewert wird im Folgenden als "pxml" oder "parsed xml" bezeichnet. Die Liste besteht aus zwei oder mehr Elementen:
- Das erste Element ist der Name des Tags.
- Das zweite Element ist eine als Array-Get formatierte Liste von Schlüssel-Wert-Paaren. Die Schlüssel sind Attributnamen und die Werte sind Attributwerte. Diese Liste ist leer, wenn das Tag keine Attribute enthält.
- Die Elemente vom dritten bis zum letzten sind die Kindknoten des Knotens, sofern vorhanden. Jeder Kindknoten ist rekursiv ein pxml.
- Hinweis: Wenn das nullte Element, d. h. der Tag-Name, "%PCDATA" lautet, sind die Attribute leer und das dritte Element enthält den Text des Elements. Wenn der Inhalt eines Elements zudem ausschließlich aus PCDATA besteht, hat es nur einen Kindknoten, und alle PCDATA-Elemente werden miteinander verkettet. Mit anderen Worten: Dieser Parser eignet

sich nur bedingt für XML mit Elementen, die sowohl Kind-Tags als auch PCDATA enthalten. Da Amazon S3 dies nicht tut (und dies im Übrigen auch bei den meisten Anwendungen von XML, bei denen XML eine ungeeignete Wahl ist, nicht der Fall ist), stellt dies wahrscheinlich keine gravierende Einschränkung dar.

xsxp::fetch *pxml path* ?*part*?

pxml ist ein geparstes XML, wie es von xsxp::parse zurückgegeben wird. *path* ist eine Liste von Element-Tag-Namen. Jedes Element ist der Name eines zu suchenden Kindknotens, optional gefolgt von einem Rautezeichen ("#") und einer Ziffernfolge. Eine leere Liste oder ein anfängliches leeres Element wählt *pxml* aus. Wenn kein Rautezeichen vorhanden ist, verhält es sich so, als wäre "#0" an dieses Element angehängt worden. (Zusätzlich zu einer Liste können Schrägstriche Unterteile trennen, wo dies zweckmäßig ist.)

Ein Element von *path* durchsucht die Kindknoten auf der angegebenen Ebene nach der n-ten Instanz eines Kindknotens, dessen Tag mit dem Teil des Elements vor dem Rautezeichen übereinstimmt. Wenn ein Element lediglich aus "#" gefolgt von Ziffern besteht, wird der entsprechend indizierte Kindknoten ausgewählt, unabhängig von den Tags der Kindknoten. Daher wählt ein Element von "#3" stets den vierten Kindknoten des betreffenden Knotens aus.

part ist standardmäßig auf "%ALL" gesetzt. Es kann einer der folgenden Begriffe sein, wobei die Groß- und Kleinschreibung beachtet werden muss:

%ALL
: gibt das gesamte ausgewählte Element zurück.

%TAGNAME
: Gibt **lindex** 0 des ausgewählten Elements zurück.

%ATTRIBUTES
: Gibt den Index 1 des ausgewählten Elements zurück.

%CHILDREN
: Gibt den Bereich von lrange 2 bis zum Ende des ausgewählten Elements zurück, wodurch eine Liste von Elementen zurückgegeben wird.

%PCDATA
: Gibt eine Verkettung aller Inhalte der direkten Kindknoten dieses Knotens zurück, deren Tag %PCDATA lautet. Es wird ein Fehler ausgelöst, wenn keine solchen Kindknoten gefunden werden. Das heißt, part=%PCDATA bedeutet, dass der Textinhalt dieses Knotens zurückgegeben wird, nicht jedoch der seiner Kindknoten.

%PCDATA?
: verhält sich wie %PCDATA, gibt jedoch eine leere Zeichenfolge zurück, wenn keine PCDATA-Daten gefunden werden.

Um beispielsweise den ersten fett gesetzten Text aus dem fünften Absatz des body der HTML-Datei abzurufen,

```
xsxp::fetch $pxml {body p#4 b} %PCDATA
```

xsxp::fetchall *pxml_list path* ?*part*?

Diese Funktion durchläuft jedes PXML in *pxml_list* (die eine Liste von PXMLs sein muss), wählt den angegebenen Pfad daraus aus, erstellt eine neue Liste mit den ausgewählten Daten und

gibt diese neue Liste zurück.

Beispielsweise könnte *pxml_list* die %CHILDREN eines bestimmten Elements sein, und die Parameter *path* und *part* könnten aus jedem Kindknoten ein Unterelement auswählen, an dem Interesse besteht.

xsxp::only *pxml tagname*
Diese Funktion durchläuft die direkten Kindknoten von *pxml* und wählt nur diejenigen aus, deren Tag *tagname* lautet. Sie gibt eine Liste der übereinstimmenden Elemente zurück.

xsxp::prettyprint *pxml* ?*chan*?
Dadurch wird eine übersichtliche Darstellung von *pxml* an *chan* (standardmäßig stdout) ausgegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *amazon-s3* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright

```
2006 Darren New. All Rights Reserved.
```

asn

asn

Stichwörter: asn, ber, cer, der, internet, protocol, x.208, x.209

Name

asn – ASN.1-BER-Kodierer/Dekodierer für die Kodierung und Dekodierung

Übersicht

package require Tcl 8.5 9
package require asn ?**0.8.5**?
::asn::asnSequence ?*evalue* ...?
::asn::asnSequenceFromList *elist*
::asn::asnSet ?*evalue* ...?
::asn::asnSetFromList *elist*
::asn::asnApplicationConstr *appNumber* ?*evalue* ...?
::asn::asnApplication *appNumber data*
::asn::asnChoice *appNumber* ?*evalue* ...?

::asn::asnChoiceConstr *appNumber* ?*evalue* ...?
::asn::asnInteger *number*
::asn::asnEnumeration *number*
::asn::asnBoolean *bool*
::asn::asnContext *context data*
::asn::asnContextConstr *context* ?*evalue* ...?
::asn::asnObjectIdentifier *idlist*
::asn::asnUTCTime *utcstring*
::asn::asnNull
::asn::asnBitString *string*
::asn::asnOctetString *string*
::asn::asnNumericString *string*
::asn::asnPrintableString *string*
::asn::asnIA5String *string*
::asn::asnBMPString *string*
::asn::asnUTF8String *string*
::asn::asnString *string*
::asn::defaultStringType ?*type*?
::asn::asnPeekByte *dataVar byteVar*
::asn::asnGetLength *dataVar lengthVar*
::asn::asnGetResponse *chan dataVar*
::asn::asnGetInteger *dataVar intVar*
::asn::asnGetEnumeration *dataVar enumVar*
::asn::asnGetOctetString *dataVar stringVar*
::asn::asnGetString *dataVar stringVar* ?*typeVar*?
::asn::asnGetNumericString *dataVar stringVar*
::asn::asnGetPrintableString *dataVar stringVar*
::asn::asnGetIA5String *dataVar stringVar*
::asn::asnGetBMPString *dataVar stringVar*
::asn::asnGetUTF8String *dataVar stringVar*
::asn::asnGetUTCTime *dataVar utcVar*
::asn::asnGetBitString *dataVar bitsVar*
::asn::asnGetObjectIdentifier *dataVar oidVar*
::asn::asnGetBoolean *dataVar boolVar*
::asn::asnGetNull *dataVar*
::asn::asnGetSequence *dataVar sequenceVar*
::asn::asnGetSet *dataVar setVar*
::asn::asnGetApplication *dataVar appNumberVar* ?*contentVar*? ?*encodingTypeVar*?
::asn::asnGetContext *dataVar contextNumberVar* ?*contentVar*? ?*encodingTypeVar*?
::asn::asnPeekTag *dataVar tagVar tagtypeVar constrVar*
::asn::asnTag *tagnumber* ?*class*? ?*tagstyle*?
::asn::asnRetag *dataVar newTag*

Beschreibung

Das Paket **asn** stellt *teilweise* Dekodier- und Kodierbefehle für BER-kodierte ASN.1-Daten bereit. Es kann auch zur Dekodierung von DER verwendet werden, bei dem es sich um eine

eingeschränkte Teilmenge von BER handelt.

ASN.1 ist ein Standard für die *Abstract Syntax Notation*, und BER sind die dazugehörigen *Basic Kodierungen*.

Weitere Informationen zu dieser Norm finden Sie unter http://asn1.elibel.tm.fr/en/standards/index.htm.

Siehe auch http://luca.ntop.org/Teaching/Appunti/asn1.html für *A Layman's Guide to a Subset of ASN.1, BER, and DER*, einen technischen Hinweis der RSA Laboratories von Burton S. Kaliski Jr. (Überarbeitet am 1. November 1993). Eine Textversion dieses Hinweises ist Teil der Modulquellen und sollte von jedem Implementierer gelesen werden.

API

Encoder

::asn::asnSequence ?*evalue ...*?
Nimmt null oder mehrere kodierte Werte entgegen, packt diese zu einer ASN-Sequenz und gibt deren kodierte binäre Form zurück.

::asn::asnSequenceFromList *elist*
Nimmt eine Liste kodierter Werte entgegen, packt diese zu einer ASN-Sequenz und gibt deren kodierte Binärform zurück.

::asn::asnSet ?*evalue ...*?
Nimmt null oder mehrere kodierte Werte entgegen, packt sie zu einem ASN-Set zusammen und gibt dessen kodierte binäre Form zurück.

::asn::asnSetFromList *elist*
Nimmt eine Liste kodierter Werte entgegen, packt diese zu einem ASN-Set zusammen und gibt dessen kodierte binäre Form zurück.

::asn::asnApplicationConstr *appNumber* ?*evalue ...*?
Nimmt null oder mehrere kodierte Werte entgegen, packt diese in ein ASN-Anwendungskonstrukt und gibt dessen kodierte Binärform zurück.

::asn::asnApplication *appNumber data*
Nimmt einen einzelnen kodierten Wert *data*, packt ihn in ein ASN-Anwendungskonstrukt und gibt dessen kodierte binäre Form zurück.

::asn::asnChoice *appNumber* ?*evalue ...*?
Nimmt null oder mehrere kodierte Werte entgegen, packt diese in ein ASN-Choice-Konstrukt und gibt dessen kodierte binäre Form zurück.

::asn::asnChoiceConstr *appNumber* ?*evalue ...*?
Nimmt null oder mehrere kodierte Werte entgegen, packt diese in ein ASN-Choice-Konstrukt und gibt dessen kodierte binäre Form zurück.

::asn::asnInteger *number*
Gibt die kodierte Form der angegebenen Ganzzahl *number* zurück.

::asn::asnEnumeration *number*
Gibt die kodierte Form der angegebenen Enumerations-ID *number* zurück.

::asn::asnBoolean *bool*
Gibt die kodierte Form des angegebenen booleschen Werts *bool* zurück.

::asn::asnContext *context data*
Nimmt einen kodierten Wert entgegen und packt ihn zu einem konstruierten Wert mit dem

Anwendungs-Tag, der *context*-Nummer.

::asn::asnContextConstr *context ?evalue ...?*
Nimmt null oder mehrere kodierte Werte entgegen und packt diese zu einem zusammengesetzten Wert mit einem Anwendungs-Tag, der *context*-Nummer, zusammen.

::asn::asnObjectIdentifier *idlist*
Nimmt eine Liste mit mindestens zwei Ganzzahlen entgegen, die einen Objektkennungswert (OID) beschreiben, und gibt den kodierten Wert zurück.

::asn::asnUTCTime *utcstring*
Gibt die kodierte Form der angegebenen UTC-Zeitangabe zurück.

::asn::asnNull
Gibt die NULL-Kodierung zurück.

::asn::asnBitString *string*
Gibt die kodierte Form der angegebenen *string* zurück.

::asn::asnOctetString *string*
Gibt die kodierte Form der angegebenen *string* zurück.

::asn::asnNumericString *string*
Gibt den als ASN.1-NumericString kodierten *string* zurück. Löst einen Fehler aus, wenn der *string* andere Zeichen als Dezimalziffern und Leerzeichen enthält.

::asn::asnPrintableString *string*
Gibt die Kodierung von *string* als ASN.1-PrintableString zurück. Löst einen Fehler aus, wenn *string* Zeichen enthält, die für den Datentyp "Printable String" nicht zulässig sind. Die zulässigen Zeichen sind A–Z, a–z, 0–9, Leerzeichen, Apostroph, Doppelpunkt, Klammern, Pluszeichen, Minuszeichen, Komma, Punkt, Schrägstrich, Fragezeichen und das Gleichheitszeichen.

::asn::asnIA5String *string*
Gibt den als ASN.1 IA5String kodierten *string* zurück. Löst einen Fehler aus, wenn der *string* Zeichen enthält, die außerhalb des US-ASCII-Zeichenbereichs liegen.

::asn::asnBMPString *string*
Gibt die als ASN.1-Zeichenkette der Basic Multilingual Plane durchführte Kodierung der *string* zurück (die im Wesentlichen Big-Endian-UCS2 entspricht).

::asn::asnUTF8String *string*
Gibt die als UTF-8-Kodierung kodierte *string* zurück. Es gibt Hinweise darauf, dass einige ältere Anwendungen, wie beispielsweise die Windows CryptoAPI, mit UTF-8-Zeichenketten möglicherweise nicht korrekt funktionieren. Am sichersten ist es daher, BMPStrings zu verwenden.

::asn::asnString *string*
Gibt eine kodierte Form von *string* zurück, wobei der möglichst restriktivste ASN.1-Zeichenkettentyp ausgewählt wird. Wenn die Zeichenkette Nicht-ASCII-Zeichen enthält, stehen mehrere Zeichenkettentypen zur Auswahl. Siehe **::asn::defaultStringType**.

::asn::defaultStringType ?*type*?
Wählt den Zeichenkettentyp aus, der für die Kodierung von Nicht-ASCII-Zeichenketten verwendet werden soll. Gibt den aktuellen Standardwert zurück, wenn die Funktion ohne Argument aufgerufen wird. Wird das Argument *type* übergeben, sollte es entweder **UTF8** oder **BMP** lauten, um jeweils UTF8String oder BMPString auszuwählen.

Decoder

Allgemeine Hinweise:

1. Nahezu alle Getter-Befehle (Decoder) nehmen zwei Argumente entgegen. Bei diesen Argumenten handelt es sich um Namen von Variablen, *mit Ausnahme* von **::asn::asnGetResponse**. Die Variable *dataVar* enthält die zu dekodierenden, kodierten ASN-Werte. Wenn ein Wert-Getter (Dekoder) aufgerufen wird, wird der Wert in der zweiten Variablen gespeichert, und die verbleibende Eingabe wird wieder in der ersten Variable *dataVar* gespeichert.
2. Beim Abrufen (Dekodieren) wird zunächst die Variable *dataVar* geändert, bevor der dekodierte Wert in der zweiten Variablen gespeichert wird. Das bedeutet: Wenn sich beide Argumente auf dieselbe Variable beziehen, enthält diese nach dem Aufruf stets den dekodierten Wert und nicht den Rest der Eingabe.

::asn::asnPeekByte *dataVar byteVar*

Gibt eine Kopie des ersten Bytes der Daten zurück, ohne *dataVar* zu verändern. Dies kann verwendet werden, um auf implizite Tags zu prüfen.

::asn::asnGetLength *dataVar lengthVar*

Dekodiert die Längenangaben für einen Block von BER-Daten. Der Tag muss zuvor aus den Daten entfernt worden sein.

::asn::asnGetResponse *chan dataVar*

Liest eine kodierte ASN-*Sequenz* aus dem Kanal *chan* ein und speichert sie in der Variable *dataVar*.

::asn::asnGetInteger *dataVar intVar*

Dekodiert den ganzzahligen Wert, der am Anfang der in der Variable *dataVar* gespeicherten Daten kodiert ist. Die Ganzzahl wird in der Variable *intVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für den nächsten zu dekodierenden Wert bereit sind. Wenn der Wert am Anfang keine kodierte Ganzzahl ist, wird ein Fehler ausgelöst.

::asn::asnGetEnumeration *dataVar enumVar*

Dekodiert den Wert der Enumerations-ID, der am Anfang der in der Variable *dataVar* gespeicherten Daten kodiert ist. Die Enumerations-ID wird in der Variable *enumVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Wenn der Wert am Anfang keine kodierte Enumerations-ID ist, wird ein Fehler ausgelöst.

::asn::asnGetOctetString *dataVar stringVar*

Dekodiert den Wert der Zeichenfolge, der am Anfang der in der Variable *dataVar* gespeicherten Daten durch Kodierung kodiert ist. Die Zeichenfolge wird in der Variable *stringVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für den nächsten zu dekodierenden Wert bereit sind. Wenn der Wert am Anfang keine kodierte Zeichenkette ist, wird ein Fehler ausgelöst.

::asn::asnGetString *dataVar stringVar ?typeVar?*

Dekodiert den lesbaren Zeichenfolgenwert, der am Anfang der in der Variable *dataVar* gespeicherten Daten durch Kodierung kodiert ist. Die Zeichenfolge wird in der Variable *stringVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Wenn der Wert am Anfang keine kodierte Zeichenkette ist, wird ein Fehler ausgelöst.

Hierbei handelt es sich um eine Komfortfunktion, die in der Lage ist, alle unterstützten ASN.1-Zeichenkettentypen automatisch zu unterscheiden und den Eingabewert entsprechend zu konvertieren.

Die typspezifischen Konvertierungsbefehle finden Sie weiter unten unter **::asn::asnGetPrintableString**, **::asnGetIA5String** usw.

Wenn das optionale dritte Argument *typeVar* angegeben wird, wird der Typ der eingehenden Zeichenkette in der damit benannten Variable gespeichert.

Die Funktion löst den Fehler "Ungültiger Name des Befehls asnGetSome**UnsupportedString**" aus, wenn auf den nicht unterstützten Zeichenfolgen-Typ **Unsupported** gestoßen wird. Wenn es nötig ist, können Sie die entsprechende Funktion "asn::asnGetSome**UnsupportedString**" in Ihrer Anwendung erstellen.

::asn::asnGetNumericString *dataVar stringVar*

Dekodiert den numerischen Zeichenfolgenwert, der am Anfang der in der Variable *dataVar* gespeicherten Daten durch Kodierung kodiert ist. Die numerische Zeichenfolge wird in der Variable *stringVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für den nächsten zu dekodierenden Wert bereit sind. Wenn der Wert am Anfang keine kodierte numerische Zeichenkette ist, wird ein Fehler ausgelöst.

::asn::asnGetPrintableString *dataVar stringVar*

Dekodiert den druckbaren Zeichenfolgenwert, der am Anfang der in der Variable *dataVar* gespeicherten Daten kodiert ist. Die druckbare Zeichenfolge wird in der Variable *stringVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für den nächsten zu dekodierenden Wert bereit sind. Wenn der Wert am Anfang keine kodierte druckbare Zeichenkette ist, wird ein Fehler ausgelöst.

::asn::asnGetIA5String *dataVar stringVar*

Dekodiert den IA5-Zeichenfolgenwert (US-ASCII), der am Anfang der in der Variable *dataVar* gespeicherten Daten kodiert ist. Die IA5-Zeichenkette wird in der Variable *stringVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Wenn der Wert am Anfang keine kodierte IA5-Zeichenkette ist, wird ein Fehler ausgelöst.

::asn::asnGetBMPString *dataVar stringVar*

Dekodiert den BMP-Zeichenfolgenwert (Zwei-Byte-Unicode – UCS2), der am Anfang der in der Variable *dataVar* gespeicherten Daten kodiert ist. Die BMP-Zeichenkette wird in eine korrekte Tcl-Zeichenkette umgewandelt, die anschließend in der Variable *stringVar* gespeichert wird; die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Wenn der Wert am Anfang keine kodierte BMP-Zeichenkette ist, wird ein Fehler ausgelöst.

::asn::asnGetUTF8String *dataVar stringVar*

Dekodiert den UTF-8-Zeichenfolgenwert, der am Anfang der in der Variablen *dataVar* gespeicherten Daten durch Kodierung erzeugt wurde. Die UTF-8-Zeichenkette wird in eine korrekte Tcl-Zeichenkette umgewandelt, die anschließend in der Variable *stringVar* gespeichert wird; die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit ist. Wenn der Wert am Anfang

keine kodierte UTF-8-Zeichenkette ist, wird ein Fehler ausgelöst.

::asn::asnGetUTCTime *dataVar utcVar*

Dekodiert den am Anfang der in der Variable *dataVar* gespeicherten Daten durchführenden Prozess der Kodierung des UTC-Zeitwerts. Der UTC-Zeitwert wird in der Variable *utcVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Der in *utcVar* gespeicherte Wert ist eine Zeichenkette; um einen tatsächlichen Zeitwert zu erhalten, verwenden Sie den Befehl **clock scan**. Wenn der Wert am Anfang kein kodierter UTC-Zeitwert ist, wird ein Fehler ausgelöst.

::asn::asnGetBitString *dataVar bitsVar*

Dekodiert den Wert der Bitzeichenfolge, der am Anfang der in der Variable *dataVar* gespeicherten Daten kodiert ist. Die Bitzeichenfolge wird in der Variable *bitsVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für den nächsten zu dekodierenden Wert bereit sind. Der in *bitsVar* gespeicherte Wert ist eine Zeichenkette, die ausschließlich die Zeichen **"0"** und **"1"** enthält. Wenn der Wert am Anfang keine kodierte Bitfolge ist, wird ein Fehler ausgelöst.

::asn::asnGetObjectIdentifier *dataVar oidVar*

Dekodiert den Wert des Objektidentifikators (OID), der am Anfang der in der Variablen *dataVar* gespeicherten Daten kodiert ist. Die OID wird in der Variablen *oidVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Der in *oidVar* gespeicherte Wert ist eine Liste von Ganzzahlen. Wenn der Wert am Anfang kein kodierter OID ist, wird ein Fehler ausgelöst.

::asn::asnGetBoolean *dataVar boolVar*

Dekodiert den am Anfang der in der Variablen *dataVar* gespeicherten Daten durchgeführten Booleschen Wert. Der Boolesche Wert wird in der Variablen *boolVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Der in *boolVar* gespeicherte Wert lautet entweder **0** oder **1**. Wenn der Wert am Anfang kein kodierter Boolescher Wert ist, wird ein Fehler ausgelöst.

::asn::asnGetNull *dataVar*

Dekodiert den am Anfang der in der Variable *dataVar* gespeicherten Daten kodierten NULL-Wert und entfernt die dekodierten Bytes vom Anfang der Daten in *dataVar*, sodass diese für die Dekodierung des nächsten Werts bereit sind. Wenn der Wert am Anfang kein kodierter NULL-Wert ist, wird ein Fehler ausgelöst.

::asn::asnGetSequence *dataVar sequenceVar*

Dekodiert den am Anfang der in der Variable *dataVar* gespeicherten Daten durch Kodierung erzeugte ASN-Sequenzwert. Die ASN-Sequenz wird in der Variable *sequenceVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass der nächste Wert dekodiert werden kann. Wenn der Wert am Anfang keine kodierte ASN-Sequenz ist, wird ein Fehler ausgelöst.

Der Wert in *sequenceVar* ist binär kodiert und muss gemäß den Definitionen der Sequenz mithilfe der anderen "asnGet..."-Befehle des Dekodierers weiter dekodiert werden.

::asn::asnGetSet *dataVar setVar*

Dekodiert den am Anfang der in der Variable *dataVar* gespeicherten Daten durchführenden

Prozess der Kodierung des ASN-Sets. Das ASN-Set wird in der Variable *setVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Wenn der Wert am Anfang kein kodierter ASN-Satz ist, wird ein Fehler ausgelöst.

Der Wert in *setVar* ist binär kodiert und muss gemäß den Definitionen der Sequenz mithilfe der anderen "asnGet..."-Befehle des Dekodierers weiter dekodiert werden.

::asn::asnGetApplication *dataVar appNumberVar ?contentVar? ?encodingTypeVar?*

Dekodiert den Wert des ASN-Anwendungskonstrukts, der am Anfang der in der Variablen *dataVar* gespeicherten Daten kodiert ist. Die ID des ASN-Anwendungskonstrukts wird in der Variablen *appNumberVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass der nächste Wert dekodiert werden kann. Wenn der Wert am Anfang kein kodiertes ASN-Anwendungskonstrukt ist, wird ein Fehler ausgelöst.

Wenn eine *contentVar* festgelegt ist, place alle mit dem ASN-Anwendungskonstrukt verbundenen Daten in der genannten Variable in binärer Form, die mit den Dekodierbefehlen dieses Pakets verarbeitet werden kann.

Wenn eine *encodingTypeVar* festgelegt wird, wird diese Variable auf **1** gesetzt, wenn es sich um eine konstruierte Kodierung handelt, und auf **0**, wenn es sich um eine primitive Kodierung handelt.

Ohne die optionalen Argumente obliegt es dem Aufrufer, den Rest des ASN-Anwendungskonstrukts auf der Grundlage der von diesem Befehl abgerufenen ID mithilfe der Decodierbefehle des Pakets zu decodieren.

::asn::asnGetContext *dataVar contextNumberVar ?contentVar? ?encodingTypeVar?*

Dekodiert das ASN-Kontext-Tag-Konstrukt, das am Anfang der in der Variable *dataVar* gespeicherten Daten durchgeführt wird. Die ID des ASN-Kontext-Tag-Konstrukts wird in der Variable *contextNumberVar* gespeichert, und die dekodierten Bytes werden vom Anfang der Daten in *dataVar* entfernt, sodass diese für die Dekodierung des nächsten Werts bereit sind. Wenn der Wert am Anfang kein kodiertes ASN-Kontext-Tag-Konstrukt ist, wird ein Fehler ausgelöst.

Wenn eine *contentVar* festgelegt wird, so place alle mit dem ASN-Kontext-Tag-Konstrukt verbundenen Daten in der genannten Variable in binärer Form, die mit den Paketbefehlen verarbeitet werden kann.

Wenn eine *encodingTypeVar* festgelegt wird, wird diese Variable auf **1** gesetzt, wenn es sich um eine konstruierte Kodierung handelt, und auf **0**, wenn es sich um eine primitive Kodierung handelt.

Ohne die optionalen Argumente obliegt es dem Aufrufer, den Rest des ASN-Kontext-Tag-Konstrukts auf der Grundlage der von diesem Befehl abgerufenen ID mithilfe der Decoder-Befehle des Pakets zu dekodieren.

Umgang mit Tags

Wenn Sie mit ASN.1 arbeiten, müssen Sie häufig getaggte Werte dekodieren, deren Tag sich vom universellen Tag eines Typs unterscheidet. In diesen Fällen müssen Sie, um den Wert zu dekodieren, das Tag durch das universelle Tag des Typs ersetzen. Um einen getaggten Wert zu dekodieren, verwenden Sie **::asn::asnRetag**, um das Tag in einen geeigneten Typ umzuwandeln, sodass Sie einen der Dekodierer für primitive Werte nutzen können. Zu diesem Zweck stellt das Paket drei Hilfsfunktionen bereit:

::asn::asnPeekTag *dataVar tagVar tagtypeVar constrVar*
: Mit dem Befehl **::asn::asnPeekTag** können Sie einen Blick auf die Daten werfen und den Wert des Tags entschlüsseln, ohne ihn aus den Daten zu entfernen. Die Variable *tagVar* wird auf die Tag-Nummer gesetzt. Die Variable *tagtypeVar* wird auf die Klasse des Tags gesetzt, die einer der folgenden Werte sein kann: **UNIVERSAL**, **CONTEXT**, **APPLICATION** oder **PRIVATE**. Die Variable *constrVar* wird auf 1 gesetzt, wenn das Tag für einen zusammengesetzten Wert steht, oder auf 0 für einen primitiven Wert. Der Befehl gibt die Länge des Tags zurück.

::asn::asnTag *tagnumber* ?*class*? ?*tagstyle*?
: Der Befehl **::asn::asnTag** dient zum Erstellen eines Tag-Werts. Die *tagnumber* legt die Nummer des Tags fest. Wenn angegeben, muss die *class* des Tags einer der folgenden Werte sein: **UNIVERSAL**, **CONTEXT**, **APPLICATION** oder **PRIVATE**; oder lediglich der erste Buchstabe als Abkürzung (**U**, **C**, **A** oder **P**). Die Standardklasse ist **UNIVERSAL**. Wenn angegeben, muss der *tagstyle* bei einer konstruierten Kodierung entweder **C** oder **1** lauten, bei einer primitiven Kodierung hingegen **P** oder **0**. Der Standardwert ist **P**. (Die Verwendung von **1** und **0** erleichtert die Nutzung dieses Befehls mit den von **::asn::asnPeekTag** gegebenen Werten.)

::asn::asnRetag *dataVar newTag*
: Ersetzt das Tag am Anfang der Daten in *dataVar* durch *newTag*. Das neue Tag kann mit dem Befehl **::asn::asnTag** erstellt werden.

Beispiele

Beispiele für die Verwendung dieses Pakets finden Sie in der Implementierung des Pakets **ldap**.

Wenn Text kodiert wird, ist **::asn::asnUTF8String** am besten zu *vermeiden*, da es in seltenen Fällen möglicherweise nicht korrekt funktioniert. Für reines 7-Bit-ASCII ist die Kodierung mit **::asn::asnIA5String** am einfachsten. Außerhalb des 7-Bit-ASCII-Bereichs wandeln Sie Tcl-Zeichenketten in UCS2 (d. h. utf-16le) um und führen Sie die Kodierung mit **::asn::asnBMPString** durch. Verwenden Sie zum Dekodieren beliebiger Zeichenfolgen **::asn::asnGetString**; denken Sie jedoch daran, jede UCS2-Zeichenfolge anschließend wieder in eine Tcl-Zeichenfolge zurückzuwandeln.

```
# Encode
set eflag [::asn::asnBoolean 1]
set etotal [::asn::asnInteger 734]
set line [encoding convertto utf-16le "€2 ÷ 3 → €⅔ ✓"]
set eline [::asn::asnBMPString $line]
set enote [::asn::asnIA5String "7-bit ASCII ~^"]
set encoded [::asn::asnSequence $eflag $etotal $eline $enote]
# Decode
::asn::asnGetSequence encoded sequence
::asn::asnGetBoolean sequence flag
```

```
::asn::asnGetInteger sequence total
::asn::asnGetString sequence bmp_line
set line [encoding convertfrom utf-16le $bmp_line]
::asn::asnGetString sequence note
puts "flag=$flag total=$total line='$line' note='$note'"
=>
flag=1 total=734 line='€2 ÷ 3 → €⅔ ✓' note='7-bit ASCII ~^'
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *asn* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2004 Jochen Loewer <loewerj@web.de>
Copyright (c) 2004-2011 Michael Schlenker <mic42@users.sourceforge.net>
```


base32

base32

Stichwörter: base32, rfc3548, rfc4648

Name

base32 – Standard-Kodierung nach base32

Übersicht

package require Tcl 8.5 9
package require base32::core ?**0.2**?
package require base32 ?**0.2**?
::base32::encode *bstring*
::base32::decode *estring*

Beschreibung

Dieses Paket stellt Befehle zur Kodierung und Dekodierung von Binärzeichenfolgen in die bzw. aus der Standard-Base32-Kodierung bereit, wie in RFC 4648 *(sowie in der älteren* RFC 3548)* festgelegt.*

API

::base32::encode *bstring*

Gibt als Ergebnis eine Base32-kodierte Version der Binärzeichenfolge *bstring* zurück.

Das Ergebnis wird möglicherweise am Ende mit den Zeichen "**=**" durch Innenabstand aufgefüllt, um sicherzustellen, dass die Länge des Ergebnisses ein Vielfaches von acht ist.

Der Encoder löst einen Fehler aus, wenn *bstring* weder eine Binärzeichenfolge noch eine Zeichenfolge ist, die ausschließlich aus 7-Bit-ASCII-Zeichen besteht.

::base32::decode *estring*

Gibt eine Binärzeichenkette zurück, die aus dem *estring* mittels Base32-Dekodierung gewonnen wurde.

Beachten Sie, dass der Encoder zwar ausschließlich Großbuchstaben erzeugt, dieser Dekoder jedoch nicht zwischen Groß- und Kleinschreibung unterscheidet, d. h., er ist toleranter.

Der Dekoder kann bei ungültiger Eingabe einen Fehler auslösen. Beispielsweise wenn:
- Die Eingabe enthält Zeichen, die für Base32-kodierte Daten nicht zulässig sind;
- Die Länge der Eingabe ist kein Vielfaches von acht;
- Der Innenabstand erscheint in der Mitte und nicht am Ende der Eingabe;
- Die Länge des Innenabstands beträgt nicht genau ein, drei, vier oder sechs Zeichen.

Es ist auch möglich, dass eine ungültige Eingabe *keinen* Fehler auslöst – beispielsweise reiner Text, der keine der oben aufgeführten Eigenschaften aufweist.

Codetabelle

Die zur Konvertierung von 5-Bit-Sequenzen verwendete Codetabelle ist nachstehend aufgeführt, wobei links die numerische Kennung der Bitsequenzen und rechts das zur Kodierung verwendete Zeichen angegeben ist. Hinweis: Die Zeichen "0" und "1" werden bei der Kodierung *nicht* verwendet, um Verwechslungen mit "O", "o" und "l" (L) zu vermeiden.

```
 0 A    9 J   18 S   27 3
 1 B   10 K   19 T   28 4
 2 C   11 L   20 U   29 5
 3 D   12 M   21 V   30 6
 4 E   13 N   22 W   31 7
 5 F   14 O   23 X
 6 G   15 P   24 Y
 7 H   16 Q   25 Z
 8 I   17 R   26 2
```

Beispiele

Dieses Beispiel veranschaulicht, wie eine Tcl-Zeichenkette in Base32 kodiert und aus Base32 dekodiert wird, wobei berücksichtigt wird, dass die Base32-Befehle mit binären Zeichenketten arbeiten.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc32 [::base32::encode $bytes]
set dec32 [::base32::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32='$enc32' [expr {$UTF8_LINE eq $line}]"
=>
base32='Z2KMHNZA4KAJYVLUMYWTRYUATUQOFGM64KCKI===' 1
```

Wenn die ursprüngliche Zeichenkette aus 7-Bit-ASCII-Zeichen besteht, sind die Konvertierungen in und aus Rohbytes mithilfe des integrierten Befehls encoding nicht erforderlich. Beispiele:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set enc32 [::base32::encode $ASCII_LINE]
set dec32 [::base32::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32='$enc32' [expr {$ASCII_LINE eq $line}]"
=>
base32='EEQDOLLCNF2CAQKTINEUSID3PZPH2===' 1
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *base32* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright

```
Public domain
```

base32core

Stichwörter: base32core, base32

Name

base32::core – Erweiterung der grundlegenden Base32-Zuordnungen

Übersicht

package require Tcl 8.5 9
package require base32::core ?**0.2**?
::base32::core::define *map forwvar backwvar ivar*
::base32::core::valid *string pattern mvar*

Beschreibung

Dieses Paket stellt allgemeine Befehle zur Erstellung vollständiger Base32-Zuordnungen bereit, ausgehend von einer grundlegenden Zuordnung, die lediglich die Codes und die zugehörigen Zeichen auflistet. Die hier erstellten vollständigen Zuordnungen – sowohl die reguläre als auch die inverse – bilden Bitfolgen ab und werden von diesen zurückabgebildet und berücksichtigen zudem die Teilzuordnungen am Ende einer Zeichenkette.

Dies ist im Wesentlichen ein internes Paket für Implementierer von Base32-Kodierern und -Dekodierern. Reguläre Benutzer benötigen dieses Paket nicht.

API

::base32::core::define *map forwvar backwvar ivar*

Dieser Befehl berechnet vollständige Vorwärts- und Rückwärtsabbildungen (Inverse) aus der Basis-*map* und speichert diese in den Variablen, die durch *forwvar* bzw. *backwvar* benannt sind. Außerdem erstellt er ein Regexp-Muster zur Erkennung ungültiger Zeichen in einer vermeintlich Base32-kodierten Eingabe und speichert dieses in der durch *ivar* benannten Variablen.

::base32::core::valid *string pattern mvar*

Dieser Befehl prüft, ob die Eingabe *string* eine gültige Base32-kodierte Zeichenkette ist, und zwar anhand des von **::base32::core::define** erzeugten *pattern* ungültiger Zeichen sowie einiger weiterer allgemeiner Regeln.

Das Ergebnis des Befehls ist ein boolescher Flag. Sein Wert lautet **True** bei einem gültigen *string* und andernfalls **False**. Im letzteren Fall wird eine Fehlermeldung, die das Problem mit der Eingabe beschreibt, in der durch *mvar* benannten Variablen gespeichert. Die Variable wird nicht verändert, wenn die Eingabe als gültig befunden wurde.

Die vom Befehl überprüften Regeln gehen über die Ablehnung ungültiger Zeichen hinaus und lauten wie folgt:

1. Die Länge der Eingabe ist kein Vielfaches von acht,
2. Der Innenabstand erscheint nicht am Ende der Eingabe, sondern in der Mitte,
3. Der Innenabstand hat keine Länge von sechs, vier, drei oder einem Zeichen,

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *base32* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright

```
Public domain
```

base32hex

Stichwörter: base32hex, base32, hex, rfc4648

Name

base32::hex – Base32-Kodierung mit erweiterter Hexadezimaldarstellung

Übersicht

package require Tcl 8.5 9
package require base32::core ?**0.2**?
package require base32::hex ?**0.2**?
::base32::hex::encode *bstring*
::base32::hex::decode *estring*

Beschreibung

Dieses Paket enthält Befehle zum Kodieren und Dekodieren von Binärzeichenfolgen in die bzw. aus der erweiterten Hex-Base32-Kodierung, wie in RFC 4648 festgelegt.

(Beachten Sie, dass der in Tcl integrierte Befehl binary eine einfache Kodierung und Dekodierung von Bytes in und aus Hexadezimalziffernpaaren ermöglicht.)

API

::base32::hex::encode *bstring*

Gibt als Ergebnis eine erweiterte, in der Hexadezimalbasis 32 kodierte Version der Binärzeichenfolge *bstring* zurück.

Das Ergebnis wird möglicherweise am Ende mit den Zeichen **=** durch Padding aufgefüllt, um sicherzustellen, dass die Länge des Ergebnisses ein Vielfaches von acht ist.

Der Encoder löst einen Fehler aus, wenn *bstring* weder eine Binärzeichenfolge noch eine Zeichenfolge ist, die ausschließlich aus 7-Bit-ASCII-Zeichen besteht.

::base32::hex::decode *estring*

Gibt eine binäre Zeichenkette zurück, die aus dem *estring* in der erweiterten Hex-Base32-Kodierung dekodiert wurde.

Hinweis: Der Encoder erzeugt zwar ausschließlich Großbuchstaben, dieser Decoder unterscheidet jedoch nicht zwischen Groß- und Kleinschreibung, d. h., er ist toleranter.

Der Decoder kann bei ungültiger Eingabe einen Fehler auslösen. Beispielsweise wenn:
- Die Eingabe enthält Zeichen, die für erweiterte, in der Base-32-Kodierung kodierte Daten nicht zulässig sind;
- Die Länge der Eingabe ist kein Vielfaches von acht;
- Der Innenabstand erscheint in der Mitte und nicht am Ende der Eingabe;
- Die Länge des Innenabstands beträgt nicht genau ein, drei, vier oder sechs Zeichen.

Es ist auch möglich, dass eine ungültige Eingabe *keinen* Fehler auslöst – beispielsweise reiner Text, der keine der oben aufgeführten Eigenschaften aufweist.

Codetabelle

Die Codetabelle zur Umwandlung von 5-Bit-Sequenzen ist nachstehend aufgeführt, wobei links

die numerische Kennung der Bitsequenzen und rechts das zur Kodierung verwendete Zeichen angegeben ist. Das wesentliche Merkmal der erweiterten Hexadezimal-Zuordnung besteht darin, dass die ersten 16 Codes den Ziffern und Hexadezimalzeichen zugeordnet sind.

```
    0 0    9 9        18 I   27 R
    1 1   10 A        19 J   28 S
    2 2   11 B        20 K   29 T
    3 3   12 C        21 L   30 U
    4 4   13 D        22 M   31 V
    5 5   14 E        23 N
    6 6   15 F        24 O
    7 7        16 G   25 P
    8 8        17 H   26 Q
```

Beispiele

Dieses Beispiel veranschaulicht, wie eine Tcl-Zeichenkette in das erweiterte Hex-Base32-Format kodiert und wieder daraus dekodiert wird, wobei zu berücksichtigen ist, dass die Befehle für das erweiterte Hex-Base32-Format mit binären Zeichenketten arbeiten.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞𝕫"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc32 [::base32::hex::encode $bytes]
set dec32 [::base32::hex::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32::hex='$enc32' [expr {$UTF8_LINE eq $line}]"
=>
base32::hex='PQAC7DP0SA09OLBKCOMJHOK0JKGE56CUSA2A8===' 1
```

Wenn es sich bei der ursprünglichen Zeichenkette um 7-Bit-ASCII handelt, sind die Konvertierungen in und aus Rohbytes mithilfe des Befehls **encoding** nicht erforderlich. Beispiel:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set enc32 [::base32::hex::encode $ASCII_LINE]
set dec32 [::base32::hex::decode $enc32]
set line [encoding convertfrom utf-8 $dec32]
puts "base32::hex='$enc32' [expr {$ASCII_LINE eq $line}]"
=>
base32::hex='44G3EBB2D5Q20GAJ8D4KI83RFPF7Q===' 1
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *base32* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

Copyright
```
Public domain
```

base64

ascii85
Stichwörter: ascii85, encoding

Name
ascii85 – Binärdaten mit ascii85 kodieren/dekodieren

Übersicht
package require Tcl 8.5 9
package require ascii85 ?**1.1**?
::ascii85::encode ?*-maxlen maxlen*? ?*-wrapchar wrapchar*? *bstring*
::ascii85::decode *estring*

Beschreibung
Dieses Paket enthält Befehle zum Kodieren und Dekodieren von Binärzeichenfolgen in die bzw. aus der ASCII85-Kodierung. (Diese Kodierung ist auch als Base85 bekannt.)

API
::ascii85::encode ?-maxlen *maxlen*? ?-wrapchar *wrapchar*? *bstring*
Gibt als Ergebnis eine ASCII85-kodierte Version der Binärzeichenkette *bstring* zurück.

Die Ergebniszeichenkette enthält nach jeweils *maxlen* Zeichen der Ausgabe das Zeichen *wrapchar*. Der Standardwert für *wrapchar* ist ein Zeilenumbruch, der Standardwert für *maxlen* ist 76. Um einen Zeilenumbruch zu verhindern, setzen Sie *maxlen* auf **0**.

Der Befehl löst einen Fehler aus, wenn *maxlen* keine ganze Zahl oder negativ ist oder wenn *bstring* weder eine Binärzeichenkette noch eine Zeichenkette ist, die ausschließlich aus 7-Bit-ASCII-Zeichen besteht.

::ascii85::decode *estring*
Gibt eine Binärzeichenkette zurück, die aus *estring* ASCII85-dekodiert wurde. Alle Leerzeichen (Leerzeichen, Tabulatoren, Zeilenumbrüche) in *estring* werden ignoriert.

Beispiele
Dieses Beispiel zeigt, wie Sie eine Tcl-Zeichenkette in ascii85 kodiert und aus ascii85 dekodiert, wobei berücksichtigt wird, dass die ascii85-Befehle mit binären Zeichenketten arbeiten.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞𝕫"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc85 [::ascii85::encode $bytes]
set dec85 [::ascii85::decode $enc85]
set line [encoding convertfrom utf-8 $dec85]
puts "ascii85='$enc85' [expr {$UTF8_LINE eq $line}]"
=>
ascii85='cBQ5U+Q@pA<HMh)39#IZ+QAf\ie4*' 1
```

Wenn die ursprüngliche Zeichenkette aus 7-Bit-ASCII-Zeichen besteht, sind die Konvertierungen in und aus Rohbytes mithilfe des integrierten Befehls **encoding** nicht erforderlich. Beispiel:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set enc85 [::ascii85::encode $ASCII_LINE]
set dec85 [::ascii85::decode $enc85]
set line [encoding convertfrom utf-8 $dec85]
puts "ascii85='$enc85' [expr {$ASCII_LINE eq $line}]"
=>
ascii85='+Wr]q@VKp,5uU-B8K`A/?@;' 1
```

Ein Zeilenumbruch kann verhindert werden, sodass keine Leerzeichen eingefügt werden:

```
ascii85::encode [string repeat xyz 24]
=>
G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H
    ?a^]G
^4U[H$X^\H?a^]
ascii85::encode -maxlen 0 [string repeat xyz 24]
=>
G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H?a^]G^4U[H$X^\H
    ?a^]G^4U[H$X^\H?a^]
```

Literatur

1. http://en.wikipedia.org/wiki/Ascii85
2. "Postscript Language Reference Manual", 3. Auflage, Seite 131.
   http://www.adobe.com/devnet/postscript/pdfs/PLRM.pdf

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *base64* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright

```
Copyright (c) 2010, Emiliano Gavilán
```

base64

Stichwörter: base64, encoding

Name

base64 – Binärdaten in Base64 kodieren/dekodieren

Übersicht

package require Tcl 8.5 9
package require base64 ?**2.6.1**?
::base64::encode ?*-maxlen maxlen*? ?*-wrapchar wrapchar*? *bstring*
::base64::decode *estring*

Beschreibung

Dieses Paket enthält Befehle zum Kodieren und Dekodieren von Binärzeichenfolgen in die bzw. aus der Standard-Base64-Kodierung, wie in RFC 4648 festgelegt.

Es wird empfohlen, bei neuem Code die integrierten binary *Unterbefehle zur Kodierung und Dekodierung von Base64 (eingeführt in Tcl 8.6) den Befehlen dieses Pakets vorzuziehen.*

Achtung: Abweichungen im Dekodierungsverhalten

Mit Tcl 8.6 wurde eine integrierte Unterstützung für die Base64-Kodierung und -Dekodierung eingeführt, die über Unterbefehle des Befehls binary erfolgt:

```
    binary encode base64
    binary decode base64
```

Beachten Sie, dass diese Unterbefehle zwar dieselben Signaturen wie die von diesem Paket bereitgestellten Befehle haben, die Decoder jedoch *nicht verhaltenskompatibel* sind.

Der integrierte Unterbefehl binary decode base64 akzeptiert die Option **-strict**, mit der der Benutzer zwischen dem strengen und dem nicht-strengen Modus wählen kann. Im strengen Modus werden ungültige Zeichen und unvollständige Daten am Ende der Eingabe als Fehler gemeldet. Im nicht-strengen Modus werden sie ignoriert. (Siehe RFC 2045.)

Die in diesem Paket bereitgestellten Implementierungen bieten keine Möglichkeit, die Strenge zu steuern. Stattdessen werden strenge und nicht-strenge Werte gemischt. Unvollständige Daten am Ende der Eingabe werden als Fehler gemeldet, ungültige Zeichen werden jedoch ignoriert.

Standardmäßig führt der Encoder dieses Pakets nach jeweils 76 Zeichen in der Ausgabe einen Zeilenumbruch durch. Der integrierte Unterbefehl binary encode base64 ist standardmäßig so eingestellt, dass kein Zeilenumbruch erfolgt, unterstützt jedoch den Zeilenumbruch mithilfe der Option **-maxlen**.

API

::base64::encode ?-maxlen *maxlen*? ?-wrapchar *wrapchar*? *bstring*

Gibt als Ergebnis eine Base64-kodierte Version der Binärzeichenkette *bstring* zurück.

Die Ergebniszeichenkette enthält nach jeweils *maxlen* Zeichen der Ausgabe das Zeichen *wrapchar*. Der Standardwert für *wrapchar* ist ein Zeilenumbruch, der Standardwert für *maxlen* ist 76. Um einen Zeilenumbruch zu verhindern, setzen Sie *maxlen* auf **0**.

Der Befehl löst einen Fehler aus, wenn *maxlen* keine ganze Zahl oder negativ ist oder wenn *bstring* weder eine Binärzeichenfolge noch eine Zeichenfolge ist, die ausschließlich aus 7-Bit-ASCII-Zeichen besteht.

::base64::decode *estring*

Gibt eine Binärzeichenkette zurück, die aus *estring* Base64-dekodiert wurde. Ungültige Zeichen oder Leerzeichen (Leerzeichen, Tabulatoren, Zeilenumbrüche) in *estring* werden ignoriert.

Hinweise zur Umsetzung

Dieses Paket enthält drei verschiedene Implementierungen für die Base64-Kodierung und -Dekodierung und wählt je nach der Umgebung, in der es ausgeführt wird, eine davon aus.

Alle drei Implementierungen verhalten sich gleich. Siehe auch Achtung: Abweichungen im Dekodierungsverhalten am Anfang dieses Dokuments.

1. Wenn Tcl 8.6 oder höher erkannt wird, werden die Befehle mithilfe der zu diesem Zeitpunkt verfügbaren integrierten Befehle implementiert.
2. Wenn die Erweiterung Trf geladen werden kann, werden die Befehle mithilfe ihrer Befehle umgesetzt.
3. Wenn keine der beiden oben genannten Möglichkeiten in Frage kommt, wird eine reine Tcl-Implementierung verwendet. Diese ist deutlich langsamer.

Beispiele

Dieses Beispiel zeigt, wie Sie eine Tcl-Zeichenkette in Base64 kodiert und aus Base64 dekodiert, wobei zu berücksichtigen ist, dass die Base64-Befehle mit binären Zeichenketten arbeiten.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞𝕫"
# Legacy (this package)
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc64 [::base64::encode $bytes]
set dec64 [::base64::decode $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$UTF8_LINE eq $line}]"
=>
base64='zpTDtyDigJxVdGYtOOKAnSDimZ7ihKQ=' 1
# Modern (using built-in binary subcommands)
set bytes [encoding convertto utf-8 $UTF8_LINE]
set enc64 [binary encode base64 $bytes]
set dec64 [binary decode base64 $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$UTF8_LINE eq $line}]"
=>
base64='zpTDtyDigJxVdGYtOOKAnSDimZ7ihKQ=' 1
```

Wenn die ursprüngliche Zeichenkette aus 7-Bit-ASCII-Zeichen besteht, sind die Konvertierungen in und aus Rohbytes mithilfe des integrierten Befehls encoding nicht erforderlich. Beispiele:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
# Legacy (this package)
set enc64 [::base64::encode $ASCII_LINE]
set dec64 [::base64::decode $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$ASCII_LINE eq $line}]"
=>
base64='ISA3LWJpdCBBU0NJSSB7fl59' 1
# Modern (using built-in binary subcommands)
set enc64 [binary encode base64 $ASCII_LINE]
set dec64 [binary decode base64 $enc64]
set line [encoding convertfrom utf-8 $dec64]
puts "base64='$enc64' [expr {$ASCII_LINE eq $line}]"
=>
base64='ISA3LWJpdCBBU0NJSSB7fl59' 1
```

Ein Zeilenumbruch kann verhindert werden, sodass keine Leerzeichen eingefügt werden:

```
base64::encode [string repeat xyz 20]
=>
eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6
eHl6eHl6eHl6
base64::encode -maxlen 0 [string repeat xyz 20]
=>
eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl6eHl
    6
```

Hinweis: Der integrierte Unterbefehl binary encode base64 standardmäßig einfügt keine Zeilenumbrüche, dies jedoch tut, wenn die Option **-maxlen** verwendet wird.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *base64* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright


```
Copyright (c) 2000, Eric Melski
Copyright (c) 2001, Miguel Sofer
```


uuencode

Stichwörter: uuencode, encoding

Name

uuencode – Binäre Daten im UU-Format kodieren/dekodieren

Übersicht

package require Tcl 8.5 9
package require uuencode ?**1.1.6**?
::uuencode::encode *bstring*
::uuencode::decode *estring*
::uuencode::uuencode ?*-name string*? ?*-mode octal*? (**-file** *filename* | ?**--**? *bstring*)
::uuencode::uudecode (**-file** *filename* | ?**--**? *estring*)

Beschreibung

Dieses Paket enthält eine reine Tcl-Implementierung der Befehle **uuencode(1p)** https://www.man7.org/linux/man-pages/man1/uuencode.1p.html und **uudecode(1p)** https://www.man7.org/linux/man-pages/man1/uudecode.1p.html. Bei dieser Kodierung werden Binärdaten in druckbare 7-Bit-ASCII-Zeichen gepackt.

API

::uuencode::encode *bstring*

Gibt als Ergebnis eine uuencode-kodierte Version der Binärzeichenkette *bstring* zurück.

Alle übergebenen Bytes werden kodiert, auch wenn sie die maximale Zeilenlänge von uuencode überschreiten. Wenn die Anzahl der Eingabebytes kein Vielfaches von 3 ist, wird die Zeichenkette mit zusätzlichen **\x00**-NUL-Bytes aufgefüllt.

Der Befehl löst einen Fehler aus, wenn *bstring* weder eine Binärzeichenkette noch eine Zeichenkette ist, die ausschließlich aus 7-Bit-ASCII-Zeichen besteht.

::uuencode::decode *estring*
Gibt eine Binärzeichenkette zurück, die aus dem *estring* mit uuencode-Decodierung entschlüsselt wurde, einschließlich etwaiger Padding-NUL-Bytes **\x00** am Ende.

::uuencode::uuencode ?-name *string*? ?-mode *octal*? (-file *filename* | ?--? *bstring*)
Gibt als Ergebnis eine umgeschlossene, uuencodierte Version der Daten im durch **-file** *Dateinamen* oder durch die Binärzeichenfolge *bstring* festgelegten Dateinamen zurück.

Jedes umgeschlossene, uuencodierte Ergebnis besteht aus einem Header, einem Hauptteil und einem Footer. Der Header ist eine Zeile, die mit **begin** beginnt, gefolgt von einem vorgeschlagenen Dateimodus und einem vorgeschlagenen Dateinamen. Der Hauptteil besteht aus Zeilen mit uuencodierten Daten, denen jeweils ein Zeichen für die Zeilenlänge vorangestellt ist. Der Footer ist **end** in einer eigenen Zeile. Siehe **uuencode(1)** https://www.man7.org/linux/man-pages/man1/uuencode.1p.html.

::uuencode::uudecode (-file *filename* | ?--? *estring*)
Gibt eine Liste aus Listen mit jeweils drei Elementen zurück, bestehend aus Dateiname, Dateimodus und uuencode-dekodierten Bytes, und zwar aus der durch **-file** angegebenen Datei *filename* oder aus dem *estring*.

Optionen

-file *filename*
Bewirkt, dass die Befehle **uuencode** oder **uudecode** ihre Daten aus der angegebenen Datei lesen, anstatt einen binären Zeichenfolgenparameter zu verwenden.

-name *string*
Die uuencodierte Daten-Kopfzeile enthält den vorgeschlagenen Dateinamen, der beim Entpacken der Daten verwendet werden soll. Verwenden Sie diese Option, um den Standardwert "data.dat" zu ändern.

-mode *octal*
Die uuencodierte Datenkopfzeile enthält ein empfohlenes Bitmuster für die Zugriffsrechte, das als Oktalzahl ausgedrückt wird. Um die Standardeinstellung 0644 zu ändern, können Sie diese Option festlegen. Beispielsweise wäre 0755 für eine ausführbare Datei geeignet. Siehe **chmod(1)** https://www.man7.org/linux/man-pages/man1/chmod.1.html.

Beispiele

In den Beispielen wird der Unterbefehl **string trimright** verwendet, um etwaige **\x00** -NUL-Bytes zu entfernen, die möglicherweise während der Kodierung hinzugefügt wurden. *Hinweis: Wenn die ursprünglichen Binärzeichenfolgen möglicherweise mit* **\x00**-NUL-Bytes *enden, ist ein ausgefeilterer Ansatz erforderlich.*

Dieses Beispiel zeigt, wie Sie eine Tcl-Zeichenkette mit "uuencode" und "uudecode" verarbeitet, wobei zu berücksichtigen ist, dass die "uuencode"-Befehle mit binären Zeichenketten arbeiten.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞𝕫"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set encuu [::uuencode::encode $bytes]
set decuu [string trimright [::uuencode::decode $encuu] \x00]
set line [encoding convertfrom utf-8 $decuu]
puts "uuencode='$encuu' [expr {$UTF8_LINE eq $line}]"
=>
uuencode='SI3#MR#B@)Q5=&8M..*`G2#BF9[BA*0`' 1
```

Wenn die ursprüngliche Zeichenkette aus 7-Bit-ASCII-Zeichen besteht, sind die Konvertierungen in und aus Rohbytes mithilfe des integrierten Befehls **encoding** nicht erforderlich. Beispiel:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set encuu [::uuencode::encode $ASCII_LINE]
set decuu [string trimright [::uuencode::decode $encuu] \x00]
set line [encoding convertfrom utf-8 $decuu]
puts "uuencode='$encuu' [expr {$ASCII_LINE eq $line}]"
=>
uuencode='(2`W+6)I="!!4T-)22![?EY]' 1
```

Dieses Beispiel zeigt, wie Sie eine uuencodierte Byte-Zeichenkette erstellt und in eine Datei schreibt.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞𝕫"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set encuu [::uuencode::uuencode -name test.uu $bytes]
writeFile test.uu binary $encuu
puts $encuu
=>
begin 644 test.uu
7SI3#MR#B@)Q5=&8M..*`G2#BF9[BA*0`
`
end
```

Dieses Beispiel zeigt, wie Sie eine Datei liest, die eine oder mehrere uuencodierte Dateien enthält.

```
foreach lst [::uuencode::uudecode -file test.uu] {
    lassign $lst name mode decuu
    set decuu [string trimright $decuu \x00]
    set line [encoding convertfrom utf-8 $decuu]
    puts "name=$name mode=$mode line='$line'"
}
=>
name=test.uu mode=644 line='Δ÷ "Utf-8" ♞𝕫'
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *base64* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

Copyright
```
Copyright (c) 2002, Pat Thoyts
```

yencode
Stichwörter: yencode, encoding, yEnc, ydecode

Name
yencode – Binäre Daten im Y-Format kodieren/dekodieren

Übersicht
package require Tcl 8.5 9
package require yencode ?**1.1.4**?
::yencode::encode *bstring*
::yencode::decode *estring*
::yencode::yencode ?*-name string*? ?*-line integer*? ?*-crc32 boolean*? (**-file** *filename* | ?**--**? *bstring*)
::yencode::ydecode (**-file** *filename* | ?**--**? *estring*)

Beschreibung
Dieses Paket enthält eine rein auf Tcl basierende Implementierung der yEncode-Dateikodierung, die für Usenet-Nachrichten verwendet wird. Diese Kodierung packt Binärdaten in ein Format, das eine 8-Bit-kompatible Übertragungsschicht erfordert, dabei jedoch Sonderzeichen der *NNTP* -Posting-Protokolle escapt. Einzelheiten zur Kodierung finden Sie unter http://www.yenc.org.

API
::yencode::encode *bstring*
Gibt als Ergebnis eine yEncoded-Version der Binärzeichenfolge *bstring* zurück.

Der Befehl löst einen Fehler aus, wenn *bstring* weder eine Binärzeichenfolge noch eine Zeichenfolge ist, die ausschließlich aus 7-Bit-ASCII-Zeichen besteht.

::yencode::decode *estring*
Gibt eine Binärzeichenfolge zurück, die aus dem *estring* mittels yEncode-Dekodierung dekodiert wurde.

::yencode::yencode ?-name *string*? ?-line *integer*? ?-crc32 *boolean*? (-file *filename* | ?--? *bstring*)
Gibt als Ergebnis eine "wrapped" yEncoded-Version der Daten im durch **-file** *Dateinamen* oder durch die Binärzeichenfolge *bstring* festgelegten Dateinamen zurück.

Jedes "wrapped" yEncoded-Ergebnis besteht aus einem Header, einem Body und einem Trailer. Der Header ist eine Zeile, die mit **=ybegin** beginnt, gefolgt von einer Zeilenlänge in der Form **line=***length*, der Größe der Originaldaten (in Byte) in der Form **size=***size* sowie dem Name der Originaldatei in der Form **name=***filename*, alles durch Leerzeichen getrennt. Der Hauptteil besteht aus den yEncoded-Daten. Der Trailer ist eine Zeile, die mit **=yend** beginnt, gefolgt von der Größe der Originaldaten in der Form **size=***size* und optional (aber empfohlen) einem CRC32-Prüfsummenwert in der Form **crc32=***checksum*, alles durch Leerzeichen getrennt.

::yencode::ydecode (-file *filename* | ?--? *estring*)

Gibt eine Liste aus Listen mit jeweils drei Elementen zurück, bestehend aus Dateiname, Dateigröße und yEncode-dekodierten Bytes, entweder aus der durch **-file** angegebenen Datei *filename* oder aus dem *estring*.

Optionen

-file *filename*

Bewirkt, dass die Befehle **yencode** oder **ydecode** ihre Daten aus der angegebenen Datei lesen, anstatt einen binären Zeichenfolgenparameter zu verwenden.

-name *string*

Die yEncoded-Daten-Kopfzeile enthält den ursprünglichen Dateinamen, der beim Entpacken der Daten verwendet werden soll. Verwenden Sie diese Option, um den Standardwert "default.bin" zu ändern.

-line *integer*

Die yEncoded-Kopfzeile legt die für die Kodierung verwendete Zeilenlänge fest, deren Standardwert 128 beträgt. Verwenden Sie diese Option, um eine andere Zeilenlänge festzulegen. Hinweis: NNTP legt eine Begrenzung der Zeilenlänge auf 1000 Zeichen fest und einige Systeme haben bei mehr als 255 Zeichen pro Zeile Probleme.

-crc32 *boolean*

Wenn die Kodierung dieses Pakets durchgeführt wird, fügt es gemäß der yEncode-Spezifikation einen CRC-Wert (Cyclic Redundancy Check) in den Trailer ein. Dies lässt sich verhindern, indem Sie diese Option verwendet und für *boolean* einen falschen Wert übergibt, z. B. **0**.

Beispiele

Die yEncode-Daten werden in den Beispielen nicht gezeigt, weil es nur Rohbytes sind.

Dieses Beispiel zeigt, wie Sie eine Tcl-Zeichenkette mit "yEncode" kodiert und wieder dekodiert, wobei zu berücksichtigen ist, dass die "yEncode"-Befehle mit binären Zeichenketten arbeiten.

```
const UTF8_LINE "Δ÷ "Utf-8" ♞ℤ"
set bytes [encoding convertto utf-8 $UTF8_LINE]
set ency [::yencode::encode $bytes]
set decy [::yencode::decode $ency]
set line [encoding convertfrom utf-8 $decy]
puts "[expr {$UTF8_LINE eq $line}]"
=>
1
```

Wenn die ursprüngliche Zeichenkette aus 7-Bit-ASCII-Zeichen besteht, sind die Konvertierungen in und aus Rohbytes mithilfe des integrierten Befehls **encoding** nicht erforderlich. Beispiel:

```
const ASCII_LINE "! 7-bit ASCII {~^}"
set ency [::yencode::encode $ASCII_LINE]
set decy [::yencode::decode $ency]
set line [encoding convertfrom utf-8 $decy]
puts "[expr {$ASCII_LINE eq $line}]"
=>
1
```

Dieses Beispiel zeigt, wie Sie eine yEncoded-Byte-Zeichenkette erstellt und in eine Datei schreibt.

```
set bytes [encoding convertto utf-8 $UTF8_LINE]
set ency [::yencode::yencode -name test.y $bytes]
writeFile test.y binary $ency
puts $ency
=>
=ybegin line=128 size=23 name=test.y
    … raw binary data elided …
=yend size=23 crc32=fc01071
```

Dieses Beispiel zeigt, wie Sie eine Datei liest, die eine oder mehrere yEncoded-Dateien enthält.

```
foreach lst [::yencode::ydecode $ency] {
    lassign $lst name size decy
    set line [encoding convertfrom utf-8 $decy]
    puts "name=$name size=$size line='$line'"
}
=>
name=test.y size=23 line='Δ÷ "Utf-8" ♞ℤ'
```

Literatur

1. http://www.yenc.org/yenc-draft.1.3.txt

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *base64* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright

```
Copyright (c) 2002, Pat Thoyts
```

bee

bee

Stichwörter: bee, BitTorrent, serialization, torrent

Name

bee – Encoder/Decoder für das BitTorrent-Serialisierungsformat

Übersicht

package require Tcl 8.5 9
package require bee ?**0.3**?
::bee::encodeString *string*
::bee::encodeNumber *integer*
::bee::encodeListArgs ?*beevalue* ...?
::bee::encodeList *listofbees*
::bee::encodeDictArgs ?*key beevalue* ...?
::bee::encodeDict *dictofbees*
::bee::decode *beestring* ?*endVar*? ?*start*?
::bee::decodeIndices *beestring* ?*endVar*? ?*start*?
::bee::decodeChannel *chan* **-command** *on_decode* ?**-exact**? ?*-prefix data*?
on_decode value *handle value*
on_decode error *handle message*
on_decode eof *handle*
::bee::decodeCancel *handle*
::bee::decodePush *handle string*

Beschreibung

Dieses Paket stellt Befehle zur Kodierung und Dekodierung von Daten im Bencode-Format (ausgesprochen "Bee-Encode") bereit, dem Serialisierungsformat des BitTorrent-Protokolls, das für Daten und Nachrichten verwendet wird.

API

Encoder

Dieses Paket stellt für jeden Werttyp (Zeichenkette und Ganzzahl) einen Encoder-Befehl sowie für jeden Sammlungstyp (Liste und Dictionary) jeweils zwei Befehle bereit, von denen der eine eine vollständige Tcl-Sammlung zur Kodierung entgegennimmt und der andere dieselben Daten als Argumente entgegennimmt.

::bee::encodeString *string*
Gibt die Bee-Kodierung der *string* zurück.

::bee::encodeNumber *integer*
Gibt die Bee-Kodierung der Zahl *integer* zurück.

Dieser Befehl löst einen Fehler aus, wenn *integer* keine ganze Zahl ist.

::bee::encodeListArgs ?*beevalue ...*?
Gibt eine bee-kodierte Liste der angegebenen null oder mehr bee-kodierten *beevalue*s zurück.

::bee::encodeList *listofbees*
Gibt eine bee-kodierte Liste der angegebenen Tcl-Liste *listofbees* mit bee-kodierten Werten zurück.

::bee::encodeDictArgs ?*key beevalue ...*?
Gibt ein bee-kodiertes Dictionary zurück, das aus den angegebenen null oder mehr Paaren aus Tcl-Zeichenkettenschlüsseln und bee-kodierten *beevalue*s besteht.

Dieser Befehl bee-kodiert jeden Zeichenkettenschlüssel selbst.

::bee::encodeDict *dictofbees*
Gibt ein bee-kodiertes Dictionary des angegebenen Tcl-Dictionarys *dictofbees* zurück, dessen Schlüssel Tcl-Zeichenketten und dessen Werte bee-kodierte Werte sein müssen.

Dieser Befehl bee-kodiert jeden Zeichenkettenschlüssel selbst.

Decoder

Dieses Paket stellt Befehle zur Dekodierung von "bee"-kodierten Zeichenfolgen sowie von über einen Kanal empfangenen "bee"-kodierten Daten bereit.

Die Befehle zum Dekodieren einer "bee"-kodierten Zeichenkette setzen voraus, dass die Zeichenkette einen oder mehrere vollständige "bee"-kodierte Werte enthält (die ihrerseits verschachtelte "bee"-kodierte Listen und Wörterbücher enthalten können).

Der Befehl zur inkrementellen Dekodierung von Bee-Werten, die über einen Kanal eingehen, arbeitet asynchron und stellt die vollständig dekodierten Werte über eine Rückruffunktion bereit.

::bee::decode *beestring* ?*endVar*? ?*start*?
Gibt einen aus dem *beestring* dekodierten Wert zurück, beginnend bei der Indexposition *start*, sofern angegeben – andernfalls bei der Standardposition **0** – und setzt *endVar*, sofern angegeben.

Wenn der "bee"-kodierte Wert eine Zeichenkette oder eine Ganzzahl ist, gibt es die entsprechende Tcl-Zeichenkette bzw. Ganzzahl zurück.

Wenn der "bee"-kodierte Wert eine Sammlung (eine "bee"-Liste oder ein "bee"-Dict) ist, gibt es die entsprechende Tcl-Liste bzw. ein Tcl-Dict zurück. In beiden Fällen werden alle enthaltenen Werte (einschließlich verschachtelter Bee-Listen und Bee-Dictionarys) vollständig bee-dekodiert, sodass das Ergebnis eine Tcl-Liste oder ein Tcl-Dictionary ist, in der bzw. dem alle verschachtelten Werte entsprechend vollständig in Tcl-Listen, Tcl-Dictionarys, Zeichenketten oder Ganzzahlen bee-dekodiert sind.

Wenn die Option *endVar* angegeben wird, handelt es sich dabei um den Namen einer Variablen, in der die Indexposition des ersten Zeichens in *beestring* gespeichert wird, das auf den gerade dekodierten Wert *folgt*, d. h. die Startposition des nächsten zu dekodierenden Werts. Dies ist nützlich, wenn der *beestring* zwei oder mehr miteinander verkettete "bee"-kodierte Werte enthält (im Gegensatz zu einer einzelnen "bee"-kodierten Liste von

"bee"-kodierten Werten). Siehe das letzte Beispiel im Abschnitt Beispiele.

Der optionale Parameter *start* ist die Indexposition, ab der der nächste "bee"-kodierte Wert gelesen und dekodiert wird.

::bee::decodeIndices *beestring* ?*endVar*? ?*start*?

Gibt eine Liste zurück, die auf dem "bee"-dekodierten Wert aus *beestring* basiert, beginnend bei der Indexposition *start*, sofern angegeben – andernfalls bei der Standardposition **0** – und setzt *endVar*, sofern angegeben. Die Bedeutung und Verwendung von *endVar* und *start* entsprechen genau denen von **::bee::decode**.

Die Elemente der zurückgegebenen Liste hängen vom Typ des "bee"-dekodierten Werts ab. In allen Fällen ist das erste Element jedoch ein Typname (einer der folgenden: **string**, **integer**, **list**, **dict**), während das zweite und dritte Element die Start- und End-Index-Positionen im *beestring* für den "bee"-kodierten Wert darstellen. Bei Bee-Listen und Bee-Dictionarys gibt es ein viertes Element, das Details zu den darin enthaltenen "bee"-Werten liefert.

Formal lauten die Ergebnislisten für die verschiedenen Typen von "bee"-Werten wie folgt:

string

Eine Liste mit drei Elementen:
- Eine konstante Zeichenfolge, die den Typ des Werts angibt; bei Zeichenfolgen lautet sie **string**.
- Die Indexposition (>= *0*) des ersten Zeichens des "bee"-Werts im angegebenen *beestring* .
- Die Indexposition (>= *0*) des letzten Zeichens des "bee"-Werts im angegebenen *beestring*.

Jede Ergebnisliste beginnt bei jedem Bee-Wert mit diesen drei Elementen.

integer

Eine Liste, die die konstante Zeichenkette **integer** sowie – genau wie bei Zeichenketten – die Start- und End-Index-Positionen des "bee"-Werts im *beestring* enthält.

list

Eine Liste, die die konstante Zeichenkette **list** sowie – genau wie bei Zeichenketten – die Start- und End-Index-Positionen des "bee"-Werts im *beestring* enthält.

Darüber hinaus verfügt die Liste über ein viertes Element, das seinerseits die hier beschriebenen Index-Daten für alle Elemente der "bee"-Liste enthält.

dictionary

Eine Liste, die die konstante Zeichenkette **dict** sowie – genau wie bei Zeichenketten – die Start- und End-Index-Positionen des Bee-Werts im *beestring* enthält.

Darüber hinaus hat das Dictionary ein viertes Element. Dieses Element ist ein Dictionary, das von jedem Zeichenkettenschlüssel des Bee-Dictionarys auf eine Liste aus zwei Elementen verweist, wobei jedes dieser Elemente wiederum eine Liste aus zwei Elementen enthält. Die Liste des ersten Elements enthält die Indexpositionen für den Schlüssel, und die Liste des zweiten Elements enthält die Indexpositionen für den Wert, auf den der Schlüssel verweist. Diese Struktur ist die einzige, die nicht nur Indexpositionen, sondern auch tatsächliche Werte aus dem *beestring* enthält. Die Indexpositionen jedes Schlüssels

sind zwar eindeutig, d. h. als Schlüssel verwendbar, doch ist die Navigation darin nicht einfach, wenn ein bestimmtes Element gesucht wird. Die Verwendung der tatsächlichen Schlüssel erleichtert die Navigation erheblich, weshalb sie bereitgestellt werden.

::bee::decodeChannel *chan* -command *on_decode* ?-exact? ?-prefix *data*?

Dieser Befehl erstellt einen Decoder für eine Reihe von Bee-Werten, die über den Kanal *chan* eingehen, und gibt ein Decoder-Handle zurück. Jedes Dekodierungsereignis wird an den erforderlichen Rückrufbefehl *on_decode* gemeldet.

Das zurückgegebene Handle kann verwendet werden, um Daten in den Decoder zu schieben oder den Decoder abzubrechen.

Der Versuch, einen weiteren Bee-Decoder auf dem Kanal *chan* zu erstellen, während ein bestehender Bee-Decoder noch aktiv ist, löst einen Fehler aus.

-command

Der durch die *erforderliche* Option **-command** festgelegte Befehl *on_decode* dient dazu, dekodierte Werte, Fehler und das EOF des Kanals zu melden.

Die Rückruffunktion wird auf globaler Ebene des Interpreters mit zwei oder drei Argumenten ausgeführt. Die möglichen Signaturen der Rückruffunktion lauten:

on_decode value *handle value*

Der Decoder hat einen Bee-Wert empfangen und erfolgreich dekodiert. Das Format des entsprechenden Tcl-*value* entspricht dem von **::bee::decode** zurückgegebenen Format. Der Kanal ist weiterhin geöffnet und das Decoder-Handle ist gültig. Die Rückruffunktion kann daher Daten in den Decoder schieben oder den Decoder abbrechen.

on_decode error *handle message*

Der Decoder hat einen Fehler festgestellt, bei dem es sich nicht um EOF handelt. Beispielsweise einen ungültigen Bee-Wert. Die *message* liefert Einzelheiten zum Fehler. Das Decoder-Handle befindet sich im gleichen Zustand wie bei EOF, d. h. es ist ungültig und kann nicht verwendet werden. Der Kanal bleibt jedoch offen.

on_decode eof *handle*

Der Decoder hat am Kanal *chan* das EOF erreicht. Danach erfolgen keine weiteren Aufrufe der Rückruffunktion. Der Kanal war zum Zeitpunkt des Aufrufs bereits geschlossen, und das *handle* ist ungültig und kann nicht verwendet werden.

-exact

Standardmäßig geht der Decoder davon aus, dass alle Daten des Kanals aus "bee"-Werten bestehen, und liest pro Ereignis so viel wie möglich ein, ohne dabei die Grenzen zwischen den "bee"-Werten zu berücksichtigen. Das bedeutet: Wenn der Kanal nach einer Reihe von "bee"-Werten Nicht-"bee"-Daten enthält, kann der Anfang dieser Nicht-"bee"-Daten verloren gehen, da er bereits vom Decoder gelesen, aber noch nicht verarbeitet wurde.

In solchen Fällen wird die Option **-exact** verwendet. Wenn diese Option festgelegt wird, liest der Decoder keine Zeichen mehr, die über den aktuell verarbeiteten "bee"-Wert hinausgehen, sodass alle Nicht-"bee"-Daten im Kanal verbleiben und nach dem Schließen des Decoders weiterverarbeitet werden können.

-prefix

Wenn diese Option festgelegt wird, wird ihr Wert als Anfang eines Bee-Werts angesehen

und zur Initialisierung des internen Puffers des Decoders verwendet. Diese Funktion ist erforderlich, wenn der Ersteller des Decoders Daten aus dem Kanal gelesen hat, um zu entscheiden, ob der Decoder erstellt werden soll oder nicht. Ohne diese Option gingen diese Anfangsdaten bei der Dekodierung verloren.

::bee::decodeCancel *handle*
Dieser Befehl bricht den durch **::bee::decodeChannel** eingerichteten und durch das Handle *handle* repräsentierten Decoder ab.

::bee::decodePush *handle string*
Dieser Befehl hängt die *string* an den internen Puffer des Decoders an. Er ist das Laufzeitäquivalent der Option **-prefix** des Befehls **::bee::decodeChannel**. Mit diesem Befehl lassen sich Daten in den Decoder zurückschieben, wenn die Rückruffunktion **on_decode value** Daten aus dem Kanal verwendet hat, um zu bestimmen, ob ein weiterer Bee-Wert dekodiert werden soll oder nicht.

Formatdefinition

Daten im Bencode-Serialisierungsformat setzen sich aus zwei Grundformen und zwei Sammlungsformen zusammen. Die Grundformen sind Zeichenketten und Ganzzahlen, die Sammlungen sind Listen und Dictionarys.

String *S*
Eine Zeichenkette *S* der Länge *L* wird durch die Zeichenkette "*L*:*S*" kodiert, wobei die Länge in Textform angegeben wird. So würde beispielsweise die Zeichenkette **"Bencode format"** als **"14:Bencode format"** kodiert.

Integer *N*
Eine Ganzzahl *N* wird durch die Zeichenfolge "**i***N***e**" kodiert. Beispielsweise würde die Ganzzahl **-482** als **"i-482e"** kodiert.

List *v1 ... vn*
Eine Liste der Werte *v1* bis *vn* wird durch die Zeichenfolge "**l***BV1*...*BVn***e**" kodiert, wobei "BV**i**" die Bee-Kodierung des Werts "v**i**" ist.

Dict *k1 -> v1 ...*
Ein Dictionary, das den Zeichenkettenschlüssel *k***i** dem Wert *v***i** zuordnet, für **i** in **1** ... **n**, wird durch die Zeichenfolge "**d***BK***i***BV***i**...**e**" kodiert, für i in **1** ... **n**, wobei "BK**i**" die Bee-Kodierung der Schlüsselzeichenfolge "k**i**" ist und "BV**i**" die Bee-Kodierung des Werts "v**i**" ist.

Hinweis: Die Bee-Kodierung behält die Reihenfolge der Schlüssel in der Eingabe nicht bei, sondern speichert sie in schlüsselsortierter Reihenfolge.

Der Typ jedes kodierten Elements lässt sich unmittelbar am ersten Zeichen seiner Kodierung erkennen:

i
Ganzzahl.

l
Liste.

d
Dictionary.

[0-9]

Zeichenkette.

Indem Ganzzahlen **i** vorangestellt wird, stellt das Format sicher, dass sie sich von Zeichenketten unterscheiden, die alle mit einer Ziffer beginnen.

Beispiele

Dieses Beispiel zeigt, wie einzelne Zeichenketten und Ganzzahlen bee-kodiert und bee-dekodiert werden.

```
const LABEL "Δ÷ "Utf-8" ♞ℤ"
set labelbeeEnc [::bee::encodeString $LABEL]
set labelbeeDec [::bee::decode $labelbeeEnc]
puts "labelbeeEnc='$labelbeeEnc' eq LABEL → [expr {$labelbeeDec eq
    $LABEL}]"
set xbeeEnc [::bee::encodeNumber 834]
set ybeeEnc [::bee::encodeNumber -172]
set ybeeDec [::bee::decode $ybeeEnc]
puts "ybeeEnc='$ybeeEnc' == -172 → [expr {$ybeeDec == -172}]"
=>
labelbeeEnc='13:Δ÷ "Utf-8" ♞ℤ' eq LABEL → 1
ybeeEnc='i-172e' == -172 → 1
```

Dieses Beispiel zeigt, wie sich Sammlungen bee-kodierter Werte bee-kodieren lassen.

```
set pointbeeEnc [::bee::encodeList [list $xbeeEnc $ybeeEnc]]
set placebeeEnc [::bee::encodeDictArgs label $labelbeeEnc point
    $pointbeeEnc]
puts "pointbeeEnc='$pointbeeEnc'"
puts "placebeeEnc='$placebeeEnc'"
=>
pointbeeEnc='li834ei-172ee'
placebeeEnc='d5:label13:Δ÷ "Utf-8" ♞ℤ5:pointli834ei-172eee'
```

Dieses Beispiel zeigt, wie sich Sammlungen bee-kodierter Werte bee-dekodieren lassen.

```
set placebeeDec [::bee::decode $placebeeEnc]
set label [dict get $placebeeDec label]
puts "label='$label' eq LABEL → [expr {$label eq $LABEL}]"
lassign [dict get $placebeeDec point] x y
puts "x=$x == 834 → [expr {$x == 834}] • y=$y == -172 → [expr {$y ==
    -172}]"
=>
label='Δ÷ "Utf-8" ♞ℤ' eq LABEL → 1
x=834 == 834 → 1 • y=-172 == -172 → 1
```

Dieses Beispiel zeigt, wie mit dem Befehl **::bee::decodeIndices** bee-dekodiert wird.

```
set labelbeeDecInd [::bee::decodeIndices $labelbeeEnc]
lassign $labelbeeDecInd type i j
set label [::bee::decode [string range $labelbeeEnc $i $j]]
puts "labelbeeDecInd type=$type i=$i j=$j label='$label'"
=>
labelbeeDecInd type=string i=0 j=15 label='Δ÷ "Utf-8" ♞𝕫'
```

Dieses Beispiel zeigt, wie eine Folge von *einzelnen* bee-kodierten Werten zunächst bee-kodiert und anschließend wieder bee-dekodiert wird.

```
set stream [::bee::encodeString "€⅔•"][::bee::encodeNumber
    -91][::bee::encodeString "α-ω"][::bee::encodeNumber 68]
for {set i 0} {$i < [string length $stream]} {} {
    lappend items [::bee::decode $stream i $i]
}
puts "stream='$stream' items={$items} size=[llength $items]"
=>
stream='3:€⅔•i-91e3:α-ωi68e' items={€⅔• -91 α-ω 68} size=4
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bee* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench

bench

Stichwörter: bench, benchmark, merging, normalization, performance, testing

Name

bench – Verarbeitung von Benchmark-Suiten

Übersicht

package require Tcl 8.5 9
package require bench ?**0.6**?

::bench::locate *pattern paths*
::bench::run ?*option value...*? *interp_list file...*
::bench::versions *interp_list*
::bench::del *bench_result column*
::bench::edit *bench_result column newvalue*
::bench::merge *bench_result...*
::bench::norm *bench_result column*
::bench::out::raw *bench_result*

Beschreibung

Dieses Paket enthält Befehle zur Ausführung von Benchmarks, die in der Bench-Sprache geschrieben sind, sowie zur Verarbeitung der bei dieser Ausführung erzeugten Ergebnisse.

Ein Leser, der sich für die Bench-Sprache selbst interessiert, beginnt mit der *Einleitung in die Bench-Sprache* und geht von dort zur formalen *Spezifikation der Bench-Sprache* weiter.

Öffentliche API

Durchführung des Benchmark-Tests

::bench::locate *pattern paths*

Dieser Befehl ermittelt Tcl-Interpreter und gibt eine Liste mit deren Pfaden zurück. Er durchsucht die vom Aufrufer angegebene Liste der *paths* unter Verwendung des Glob-*pattern*.

Der Befehl löst symbolische Links auf, um die tatsächlichen ausführbaren Dateien zu finden, die dem Muster entsprechen. Nur Interpreter, die als ausführbar markiert sind und auf der aktuellen Plattform tatsächlich ausführbar sind, werden in das Ergebnis aufgenommen.

::bench::run ?*option value*...? *interp_list file*...

Dieser Befehl führt die in der Dateimenge deklarierten Benchmarks einmal je über die *interp_list* angegebenen Tcl-Interpreter und gemäß der durch die Optionen festgelegten Konfiguration aus und gibt anschließend die kumulierten Zeitmessungsergebnisse zurück. Das Format dieses Ergebnisses wird im Abschnitt Ergebnisformat beschrieben.

Es wird davon ausgegangen, dass der Inhalt der Dateien in der Bench-Sprache verfasst ist.

Die verfügbaren Optionen sind

-errors *flag*

Das Argument ist ein boolescher Wert. Wenn dieser Wert gesetzt ist, werden Fehler in Benchmarks an den Befehl weitergeleitet, wodurch die Ausführung des Benchmarks abgebrochen wird. Andernfalls werden sie über einen speziellen Ergebniscode im Zeitmessungsergebnis erfasst. Standardmäßig erfolgt eine Weiterleitung und ein Abbruch.

-threads *n*

Das Argument ist ein nicht-negativer Wert für die Anzahl der Threads, die bei der Ausführung der Benchmarks verwendet werden sollen. Der Standardwert ist **0**, was bedeutet, dass keine Threads verwendet werden.

-match *pattern*

Das Argument ist ein Glob-Muster. Es werden nur Benchmarks ausgeführt, deren Beschreibung mit dem Muster übereinstimmt. Der Standardwert ist die leere Zeichenkette, wodurch alle Benchmarks ausgeführt werden.

-rmatch *pattern*
Das Argument ist ein Muster für einen regulären Ausdruck. Es werden nur Benchmarks ausgeführt, deren Beschreibung mit dem Muster übereinstimmt. Der Standardwert ist die leere Zeichenfolge, wodurch alle Benchmarks ausgeführt werden.

-iters *n*
Das Argument ist eine positive ganze Zahl, die die maximale Anzahl an Iterationen für einen beliebigen Benchmark angibt. Der Standardwert lautet **1000**. Einzelne Benchmarks können diesen Wert überschreiben.

-pkgdir *path*
Das Argument ist ein Pfad zu einem vorhandenen, lesbaren Verzeichnis. Es können mehrere Pfade angegeben werden; verwenden Sie dazu die Option einfach mehrmals, wobei Sie jedes Mal einen der zu verwendenden Pfade angeben.

Wenn keine Pfade angegeben wurden, verhält sich das System wie bisher. Wenn ein oder mehrere Pfade festgelegt werden, beispielsweise **N**, wird jeder der festgelegten Interpreter **N**-mal mit einem der festgelegten Pfade aufgerufen. Der ausgewählte Pfad wird in die Variable **auto_path** des Interpreters übernommen, wodurch diese bestimmte Versionen eines Pakets finden können.

Auf diese Weise ermöglicht die Verwendung von **-pkgdir** dem Benutzer, mehrere verschiedene Versionen eines Pakets im Vergleich zu einem oder mehreren Interpretern zu testen.

Hinweis: Eine leere Zeichenkette ist als Pfad zulässig und bewirkt, dass das System die angegebenen Interpreter mit einem unveränderten **auto_path** ausführt – sofern das betreffende Paket dort ebenfalls verfügbar ist.

::bench::versions *interp_list*
Dieser Befehl nimmt eine Liste von Tcl-Interpretern entgegen, die durch ihren Pfad identifiziert werden, und gibt ein Dictionary zurück, das die Interpreter ihren Versionen zuordnet. Interpreter, die nicht tatsächlich ausführbar sind oder bei der Abfrage einen Fehler zurückgeben, werden nicht in das Ergebnis aufgenommen. Das bedeutet, dass das Ergebnis möglicherweise weniger Interpreter enthält als in der Eingabeliste vorhanden sind.

Der Befehl verwendet den integrierten Befehl **info patchlevel**, um die Version jedes Interpreters zu ermitteln.

Ergebnismanipulation

::bench::del *bench_result column*
Dieser Befehl entfernt eine Spalte, d. h. alle Benchmark-Ergebnisse für einen bestimmten Tcl-Interpreter, aus dem angegebenen Benchmark-Ergebnis und gibt das geänderte Ergebnis zurück.

Die Benchmark-Ergebnisse liegen in dem in Abschnitt Ergebnisformat beschriebenen Format vor.

Die Spalten werden durch eine ganzzahlige Nummer identifiziert.

::bench::edit *bench_result column newvalue*

Dieser Befehl benennt eine Spalte im angegebenen Benchmark-Ergebnis um und gibt das geänderte Ergebnis zurück. Das bedeutet, dass der Pfad zum Tcl-Interpreter in der angegebenen Spalte in eine beliebige Zeichenfolge geändert wird.

Die Benchmark-Ergebnisse liegen in dem im Abschnitt Ergebnisformat beschriebenen Format vor.

Die Spalte wird durch eine ganzzahlige Nummer gekennzeichnet.

::bench::merge *bench_result*...

Dieser Befehl nimmt ein oder mehrere Benchmark-Ergebnisse entgegen, fasst diese zu einem einzigen großen Ergebnis zusammen und gibt dieses als Ergebnis zurück.

Alle Benchmark-Ergebnisse liegen in dem im Abschnitt Ergebnisformat beschriebenen Format vor.

::bench::norm *bench_result column*

Dieser Befehl normalisiert die Zeitmessungsergebnisse anhand des angegebenen Benchmark-Ergebnisses und gibt das modifizierte Ergebnis zurück. Das bedeutet, dass die Zellwerte keine Zeitangaben mehr sind, sondern Faktoren, die angeben, um wie viel schneller oder langsamer die Ausführung im Vergleich zur Basislinie war.

Die Basislinie, anhand derer der Befehl die Normalisierung vornimmt, sind die Zeitmessungsergebnisse in der ausgewählten Spalte. Das bedeutet, dass nach der Normalisierung die Werte in dieser Spalte alle den Wert **1** annehmen, da diese Benchmarks weder schneller noch langsamer als die Basislinie sind.

Ein Faktor kleiner als **1** weist auf einen Benchmark hin, der schneller als die Basislinie war, während ein Faktor größer als **1** auf eine langsamere Ausführung hinweist.

Die Benchmark-Ergebnisse liegen in dem im Abschnitt Ergebnisformat beschriebenen Format vor.

Die Spalten werden durch eine ganze Zahl gekennzeichnet.

::bench::out::raw *bench_result*

Dieser Befehl formatiert das angegebene Benchmark-Ergebnis für die Ausgabe in eine Datei, einen Socket usw. Dieser spezielle Befehl führt keinerlei Formatierung durch, sondern leitet die Eingabe unverändert weiter.

Weitere Formatierungsstile finden Sie in den Paketen **bench::out::text** und **bench::out::csv** , die Befehle zur Formatierung von Benchmark-Ergebnissen für die menschliche Lesbarkeit bzw. als CSV-Daten zum Import in Tabellenkalkulationsprogramme bereitstellen.

Um Benchmark-Ergebnisse aus Dateien, Sockets usw. einzulesen, nutzen Sie bitte das Paket **bench::in** und die darin enthaltenen Befehle.

Ergebnisformat

Nach der Ausführung einer Reihe von Benchmarks gibt dieses Paket als Rohergebnis ein Tcl-Dictionary zurück, das alle relevanten Informationen enthält. Das Dictionary ist eine kompakte Darstellung – d. h. eine Serialisierung – einer zweidimensionalen Tabelle, deren Spalten Tcl-Interpreter und deren Zeilen Benchmarks enthalten. Die Zellen der Tabelle enthalten die Zeitmessungsergebnisse. Die Tcl-Interpreter (Spalten) werden anhand ihrer Pfade identifiziert. Die Benchmarks (Zeilen) werden anhand ihrer Beschreibung identifiziert.

Die möglichen Schlüssel sind allesamt gültige Tcl-Listen mit zwei oder drei Elementen und weisen eine der folgenden Formen auf:

{interp *}
: Die Schlüssel, die diesem Glob-Muster entsprechen, enthalten Informationen zu allen Tcl-Interpretern, die zur Ausführung der Benchmarks verwendet wurden. Das zweite Element des Schlüssels ist der Pfad zum Interpreter.

  Der zugehörige Wert ist die Version des Tcl-Interpreters.

{desc *}
: Die Schlüssel, die diesem Glob-Muster entsprechen, erfassen die Informationen zu allen Benchmarks, die in der ausgeführten Benchmark-Suite gefunden wurden. Das zweite Element des Schlüssels ist die Beschreibung des Benchmarks, die eindeutig sein muss.

  Der zugehörige Wert ist irrelevant und wird auf die leere Zeichenkette gesetzt.

{usec * *}
: Die Schlüssel, die diesem Glob-Muster entsprechen, erfassen die Leistungsdaten, d. h. die Zeitmessungsergebnisse. Das zweite Element des Schlüssels ist die Beschreibung des Benchmarks, das dritte Element der Pfad des Tcl-Interpreters, mit dem dieser ausgeführt wurde.

  Der zugehörige Wert ist entweder einer von mehreren speziellen Ergebniscodes oder die Zeit, die für die Ausführung des Benchmarks benötigt wurde, in Mikrosekunden. Die möglichen speziellen Ergebniscodes sind

  ERR
  : Der Benchmark konnte nicht ausgeführt werden; es trat ein Tcl-Fehler auf.

  BAD_RES
  : Der Benchmark konnte zwar ausgeführt werden, das Ergebnis aus seinem Rumpf entsprach jedoch nicht den angegebenen Erwartungen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bench* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Benchmark-Werkzeuge

Copyright

```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench_intro
Stichwörter: bench_intro, bench language, benchmark, performance, testing

Name
bench_intro – Einleitung zu "bench"

Beschreibung
Der Begriff *bench* (kurz für *benchmark tools*) bezeichnet eine Reihe miteinander verbundener, aber dennoch unterschiedlicher Bestandteile, die zusammenwirken, um die einfache Erstellung und Ausführung von Leistungstestsuiten – auch als Benchmarks bezeichnet – zu ermöglichen. Dazu gehören

1. Eine auf Tcl basierende Sprache zur Definition von Testfällen. Ein Testfall wird durch einen Tcl-Befehl dargestellt, der die verschiedenen zur Ausführung erforderlichen Bestandteile festlegt, wie z. B. die Vorbereitungs- und Aufräumschritte sowie die zu testenden Befehle usw.
2. Ein Paket, das die Möglichkeit bietet, in dieser Sprache geschriebene Testfälle auszuführen.

Welche der ausführlicheren Dokumente für den Leser dieser Einleitung relevant sind, hängt von seiner Rolle im Benchmarking-Prozess ab.

1. Ein *Autor* von Benchmarks muss die Bench-Sprache selbst verstehen. Ein Anfänger im Bereich Bench liest zunächst die eher informell verfasste *Einleitung in die Bench-Sprache*. Nachdem er diese verinnerlicht hat, sollte die formale *Spezifikation der Bench-Sprache* verständlich werden. Ein mit Bench erfahrener Autor benötigt dieses letzte Dokument möglicherweise nur gelegentlich, um die Erinnerung aufzufrischen.
2. Ein *Benutzer* von Benchmark-Suiten, die in der Sprache *bench* geschrieben sind, muss wissen, welche Werkzeuge zur Verfügung stehen. Auf der untersten Ebene befindet sich das Paket **bench**, das die grundlegenden Funktionen zum Lesen und Ausführen von Dateien mit in der Bench-Sprache geschriebenen Benchmarks sowie zur Bearbeitung der Benchmark-Ergebnisse bereitstellt.

Historische Hinweise
Dieses Modul und dieses Paket basieren auf der Anwendung **tclbench** von Jeff Hobbs zur Leistungsmessung des Tcl-Kerns sowie auf deren Vorgänger "*runbench.tcl*".

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bench* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Benchmark-Werkzeuge

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench_lang_intro
Stichwörter: bench_lang_intro, bench language, benchmark, examples, performance, testing

Name
bench_lang_intro – Einleitung in die Bench-Sprache

Beschreibung
Dieses Dokument bietet anhand zahlreicher Beispiele eine informelle Einleitung in Version 1 der Bench-Sprache. Nach der Lektüre dieses Dokuments sollte ein Benchmark-Entwickler in der Lage sein, die formale *Spezifikation der Bench-Sprache* zu verstehen.

Grundlagen
Im weitesten Sinne ist die *Bench-Sprache* im Wesentlichen Tcl, ergänzt um eine Reihe von Befehlen zur Unterstützung der Deklaration von Benchmarks. Ein in dieser Sprache verfasstes Dokument ist ein Tcl-Skript und weist dieselbe Syntax auf.

Einfache Benchmarks
Einer der einfachsten Benchmarks, die in bench geschrieben werden können, ist

```
bench -desc LABEL -body {
    set a b
}
```

Dieser Code deklariert einen Benchmark namens **LABEL**, der die Zeit misst, die benötigt wird, um einer Variable einen Wert zuzuweisen. Der Tcl-Code, der diese Zuweisung durchführt, ist der **-body** des Benchmarks.

Vor- und Nachbearbeitung
Unser nächstes Beispiel veranschaulicht, wie *Initialisierungs*- und *Aufräum-*Code deklariert wird, d. h. Code, der Informationen für die Verwendung des **-body** berechnet und der diese Ressourcen nach Abschluss der Messung wieder freigibt. Es handelt sich dabei um den **-pre**- bzw. den **-post**-Rumpf.

In unserem Beispiel, das direkt aus der Benchmark-Suite des Tcllib-Pakets **aes** stammt, erstellt der konkrete Initialisierungscode das Schlüsselschema, das vom Verschlüsselungsbefehl verwendet wird, dessen Geschwindigkeit gemessen wird, und der Aufräumcode gibt alle Ressourcen frei, die an dieses Schlüsselschema gebunden sind.

```
bench -desc "AES-${len} ECB encryption core" -pre {
    set key [aes::Init ecb $k $i]
} -body {
    aes::Encrypt $key $p
} -post {
    aes::Final $key
}
```

Fortgeschrittene Vor- und Nachbearbeitung

Unser letztes Beispiel befasst sich erneut mit Initialisierungs- und Aufräumcode. Um den Unterschied zu der im letzten Abschnitt behandelten regulären Initialisierung und dem Aufräumvorgang zu erkennen, ist es notwendig, etwas mehr darüber zu wissen, wie bench die Geschwindigkeit des **-body** tatsächlich misst.

Anstatt den **-body** nur einmal auszuführen, führt das System den **-body** tatsächlich mehrere hundert Mal aus und gibt anschließend den Durchschnitt der ermittelten Ausführungszeiten zurück. Dies geschieht, um Umwelteinflüsse wie die Systemauslastung so weit wie möglich aus dem Ergebnis zu eliminieren, wobei sich Ausreißer im Durchschnitt gegenseitig aufheben.

Der Nachteil dieser Vorgehensweise besteht darin, dass wir bei der Messung von Operationen, die nicht idempotent sind, höchstwahrscheinlich nicht die Zeit für die gewünschte Operation messen, sondern die Zeit für den Zustand bzw. die Zustände, in denen sich das System nach der ersten Iteration befindet – eine Mischung aus Faktoren, die für uns nicht von Interesse sind.

Sollten wir beispielsweise die Zeit messen wollen, die benötigt wird, um ein Element in eine Menge aufzunehmen, wobei sich das Element noch nicht in der Menge befindet und die Menge bestimmte Eigenschaften aufweist – wie beispielsweise, dass es sich um ein gemeinsam genutztes Tcl_Obj handelt –, dann wird in der ersten Iteration die dafür benötigte Zeit gemessen. *Allerdings* messen alle nachfolgenden Iterationen die Zeit für die Aufnahme eines Elements, das sich bereits in der Menge befindet, und das Tcl_Obj, das die Menge enthält, wird ebenfalls nicht mehr gemeinsam genutzt. Letztendlich werden die für die mehreren hundert Iterationen in diesem Zustand gemessenen Zeiten die Zeit aus der ersten Iteration bei weitem überwiegen – der einzigen, die tatsächlich das gemessen hat, was wir wollten.

Die erweiterten Initialisierungs- und Bereinigungscodes, nämlich der **-ipre**- bzw. der **-ipost** -Rumpf, dienen genau der Lösung dieses Problems. Während die regulären Initialisierungs- und Bereinigungscodes vor und nach der gesamten Iterationsreihe ausgeführt werden, werden die erweiterten Codes vor und nach jeder einzelnen Iteration des Rumpfes ausgeführt, ohne selbst gemessen zu werden. Dadurch können sie das System genau in den Zustand versetzen, den der Rumpf messen möchte.

Unser Beispiel, das direkt aus der Benchmark-Suite des Tcllib-Pakets **struct::set** stammt, entspricht genau dem Beispiel, das oben verwendet wurde, um die Notwendigkeit der erweiterten Initialisierung und Bereinigung zu veranschaulichen. Der konkrete Initialisierungscode erstellt eine Variable, die auf eine Menge mit bestimmten Eigenschaften verweist (die Menge verfügt über eine Zeichenfolgendarstellung, die gemeinsam genutzt wird), was sich auf die Geschwindigkeit des Befehls **include** auswirkt, und der Bereinigungscode gibt die durch diese Initialisierung erstellten temporären Variablen frei.

```
bench -desc "set include, missing <SC> x$times $n" -ipre {
    set A $sx($times,$n)
    set B $A
} -body {
    struct::set include A x
} -ipost {
    unset A B
}
```

Weiterführende Literatur

Nachdem der Leser – von dem angenommen wird, dass er ein *Entwickler* von Benchmarks ist – dieses Dokument verinnerlicht hat, sollte er ausreichend gerüstet sein, um die formale *Spezifikation der Bench-Sprache* zu verstehen. Diese dient zudem als detaillierte Spezifikation und Spickzettel für alle verfügbaren Befehle und deren Syntax.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bench* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Benchmark-Werkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


bench_lang_spec

Stichwörter: bench_lang_spec, bench language, benchmark, performance, specification, testing

Name

bench_lang_spec – Spezifikation der Bench-Sprache

Übersicht

bench_rm *path...*
bench_tmpfile
bench *options...*

Beschreibung

Dieses Dokument legt sowohl die Namen als auch die Syntax aller Befehle fest, die zusammen die "Bench-Sprache", Version 1, bilden. Da dieses Dokument als Nachschlagewerk gedacht ist,

sind die Befehle in alphabetischer Reihenfolge aufgeführt, und die Beschreibungen sind relativ kurz gehalten. Anfängern wird empfohlen, zunächst die eher informell verfasste *Einleitung in die Bench-Sprache* zu lesen.

Befehle

bench_rm *path*...
Dieser Befehl löscht die als Argumente festgelegten Dateien ohne Rückmeldung und gibt anschließend die leere Zeichenkette als Ergebnis zurück. Der Befehl gilt als *vertrauenswürdig*; es wird nicht überprüft, ob sich die festgelegten Dateien außerhalb des eingeschränkten Bereichs befinden, in dem die Benchmarks ausgeführt werden.

bench_tmpfile
Dieser Befehl gibt den Pfad zu einer benchspezifischen, eindeutigen temporären Datei zurück. Die Eindeutigkeit bedeutet, dass mehrere Aufrufe unterschiedliche Pfade zurückgeben. Der Pfad kann zwar aus früheren Ausführungen stammen, der Befehl selbst erstellt jedoch *keine* Dateien.

Der Speicherort der temporären Dateien ist plattformabhängig:

Unix und unbestimmte Plattform
"*/tmp*"

Windows
$TEMP

Alles andere
Das aktuelle Arbeitsverzeichnis.

bench *options*...
Dieser Befehl deklariert einen einzelnen Benchmark. Sein Ergebnis ist die leere Zeichenkette. Alle Bestandteile des Benchmarks werden über Optionen und deren Werte deklariert. Die Optionen können in beliebiger Reihenfolge angegeben werden. Folgende Optionen werden akzeptiert:

-body script
Das Argument dieser Option definiert den Rumpf des Benchmarks, d. h. das Tcl-Skript, dessen Leistung gemessen werden soll. Diese Option und **-desc** sind die beiden obligatorischen Bestandteile jedes Benchmarks.

-desc msg
Das Argument dieser Option gibt den Namen des Benchmarks an. Dieser muss eindeutig sein, da sonst die Zeitmessdaten verschiedener Benchmarks miteinander vermischt werden.

Achtung! Diese Anforderung wird bei der Ausführung der Benchmarks nicht überprüft, und das System erzeugt stillschweigend fehlerhafte Daten. Diese Option und **-body** sind die beiden erforderlichen Bestandteile jedes Benchmarks.

-ipost script
Das Argument dieser Option deklariert ein Skript, das unmittelbar *nach* jeder Iteration des Rumpfes ausgeführt wird. Seine Aufgabe besteht darin, die vom Rumpf oder vom **-ipre** -Rumpf erstellten Ressourcen freizugeben, die nicht in die nächste Iteration übernommen werden sollen.

-ipre script
Das Argument dieser Option deklariert ein Skript, das unmittelbar *vor* jeder Iteration des Rumpfes ausgeführt wird. Seine Aufgabe besteht darin, den vom Rumpf erwarteten Systemzustand herzustellen, damit die richtige Messung erfolgt.

-iterations num
Das Argument dieser Option legt fest, wie oft der **-body** des Benchmarks maximal ausgeführt werden soll. Während der Ausführung werden dieser Wert und die globale maximale Anzahl an Iterationen verglichen, wobei der kleinere der beiden Werte verwendet wird.

Diese Option sollte nur für Benchmarks verwendet werden, bei denen zu erwarten ist oder bekannt ist, dass ein Durchlauf viel Zeit in Anspruch nimmt. Das heißt, die Anzahl der Durchläufe sollte reduziert werden, um die Gesamtzeit für die Ausführung des gesamten Benchmarks in überschaubaren Grenzen zu halten.

-post script
Das Argument dieser Option deklariert ein Skript, das *nach* der Ausführung aller Iterationen des Rumpfes ausgeführt wird. Seine Aufgabe besteht darin, die vom Rumpf oder vom **-pre** -Rumpf erstellten Ressourcen freizugeben.

-pre script
Das Argument dieser Option deklariert ein Skript, das *vor* der Ausführung einer der Iterationen des Rumpfes ausgeführt wird. Seine Aufgabe besteht darin, alle Ressourcen bereitzustellen, die der Rumpf benötigt, um fehlerfrei ausgeführt zu werden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bench* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Benchmark-Werkzeuge

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

bench_read

Stichwörter: bench_read, benchmark, csv, formatting, human readable, parsing, performance, reading, testing, text

Name

bench::in – bench::in – Lesen von Benchmark-Ergebnissen

Übersicht

package require Tcl 8.5 9
package require csv
package require bench::in ?**0.2**?
::bench::in::read *file*

Beschreibung

Dieses Paket enthält einen Befehl zum Einlesen von Benchmark-Ergebnissen aus Dateien, Sockets usw.

Leser, die sich für die Erstellung, die Verarbeitung oder das Schreiben solcher Ergebnisse interessieren, sollten stattdessen den Artikel *bench – Verarbeitung von Benchmark-Suiten* lesen.

Wenn die Bench-Sprache selbst Ihr eigentliches Interesse ist, beginnen Sie bitte mit der *Einleitung in die Bench-Sprache* und fahren Sie anschließend mit der formalen *Spezifikation der Bench-Sprache* fort.

Öffentliche API

::bench::in::read *file*

Dieser Befehl liest ein Benchmark-Ergebnis aus der angegebenen *file* ein und gibt es als Ergebnis zurück. Der Befehl erkennt die drei Formate, die von den folgenden Befehlen erstellt werden:

bench::out::raw
Bereitgestellt durch das Paket **bench**.

bench::out::csv
Bereitgestellt durch das Paket **bench::out::csv**.

bench::out::text
Bereitgestellt durch das Paket **bench::out::text**.

und erkennt automatisch, welches Format in der Eingabedatei verwendet wird.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bench* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Benchmark-Werkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

bench_wcsv

Stichwörter: bench_wcsv, benchmark, csv, formatting, performance, testing

Name

bench::out::csv – bench::out::csv – Aufbereitung von Benchmark-Ergebnissen als CSV

Übersicht

package require Tcl 8.5 9
package require bench::out::csv ?**0.1.3**?
::bench::out::csv *bench_result*

Beschreibung

Dieses Paket enthält Befehle zur Aufbereitung von Benchmark-Ergebnissen in eine CSV-Tabelle, die in Tabellenkalkulationsprogramme importiert werden kann.

Leser, die sich für die Erstellung oder Verarbeitung solcher Ergebnisse interessieren, sollten stattdessen *bench – Verarbeitung von Benchmark-Suiten* lesen.

Wenn die Bench-Sprache selbst Ihr eigentliches Interesse ist, beginnen Sie bitte mit der *Einleitung in die Bench-Sprache* und fahren Sie anschließend mit der formalen *Spezifikation der Bench-Sprache* fort.

Öffentliche API

::bench::out::csv *bench_result*

Dieser Befehl formatiert das angegebene Benchmark-Ergebnis für die Ausgabe in eine Datei, einen Socket usw. Dieser spezielle Befehl erzeugt CSV-Daten, die in Tabellenkalkulationsprogramme importiert werden können.

Weitere Formatierungsoptionen finden Sie in den Paketen **bench** und **bench::out::text**, die Befehle zur Formatierung von Benchmark-Ergebnissen im Rohformat bzw. zur besseren Lesbarkeit für den Menschen bereitstellen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bench* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Benchmark-Werkzeuge

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

bench_wtext

Stichwörter: bench_wtext, benchmark, formatting, human readable, performance, testing, text

Name

bench::out::text – bench::out::text – Aufbereitung von Benchmark-Ergebnissen als für Menschen lesbarer Text

Übersicht

package require Tcl 8.5 9
package require bench::out::text ?**0.1.3**?
::bench::out::text *bench_result*

Beschreibung

Dieses Paket enthält Befehle zur Aufbereitung von Benchmark-Ergebnissen in menschenlesbaren Text.

Leser, die sich für die Erstellung oder Verarbeitung solcher Ergebnisse interessieren, sollten stattdessen *bench – Verarbeitung von Benchmark-Suiten* lesen.

Wenn die Bench-Sprache selbst Ihr eigentliches Interesse ist, beginnen Sie bitte mit der *Einleitung in die Bench-Sprache* und fahren Sie anschließend mit der formalen *Spezifikation der Bench-Sprache* fort.

Öffentliche API

::bench::out::text *bench_result*

Dieser Befehl formatiert das angegebene Benchmark-Ergebnis für die Ausgabe in eine Datei, einen Socket usw. Dieser spezielle Befehl erzeugt für Menschen lesbaren Text.

Weitere Formatierungsstile finden Sie in den Paketen **bench** und **bench::out::csv**, die Befehle zur Formatierung von Benchmark-Ergebnissen im Rohformat bzw. als importierbare CSV-Daten bereitstellen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bench* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Benchmark-Werkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

bibtex

bibtex

Stichwörter: bibtex, bibliography, parsing, text processing

Name

bibtex – BibTeX-Dateien parsen

Übersicht

package require Tcl 8.5 9
package require bibtex ?**0.8**?
::bibtex::parse ?*options*? ?*text*?
::bibtex::parse *text*
::bibtex::parse ?*-command cmd*? **-channel** *chan*
::bibtex::parse ?*-recordcommand recordcmd*? ?*-preamblecommand preamblecmd*? ?*-stringcommand stringcmd*? ?*-commentcommand commentcmd*? ?*-progresscommand progresscmd*? ?*-casesensitivestrings bool*? (*text* | **-channel** *chan*)
::bibtex::wait *token*
::bibtex::destroy *token*
::bibtex::addStrings *token stringdict*

Beschreibung

Dieses Paket enthält Befehle zum Parsen von Literaturverzeichnissen im BibTeX-Format.

::bibtex::parse ?*options*? ?*text*?
Dies ist die allgemeine Form des Befehls zum Parsen eines Literaturverzeichnisses. Je nach den Optionen beim Aufruf gibt er entweder ein Token für den Parser oder die geparsten Einträge des eingegebenen Literaturverzeichnisses zurück. Statt sofort zu parsen und ein vorgegebenes Format zurückzugeben, kann der Befehl auch in einen ereignisbasierten Parse-Stil wechseln. Alle relevanten Einträge der Eingabe werden dann über Rückrufbefehle gemeldet, im Stil von SAX.

::bibtex::parse *text*
In dieser Form nimmt der Befehl an, dass der angegebene *text* ein Literaturverzeichnis im BibTeX-Format ist, parst ihn und gibt eine Liste mit einem Element je Datensatz im Literaturverzeichnis zurück. Kommentare, String-Definitionen, Präambeln usw. erscheinen im Ergebnis nicht. Jedes Element ist eine Liste aus Datensatztyp, Bibliographieschlüssel und Datensatzdaten, in dieser Reihenfolge. Die Datensatzdaten sind ein Dictionary; seine Schlüssel sind die Schlüssel des Datensatzes, mit den zugehörigen Werten.

::bibtex::parse ?-command *cmd*? -channel *chan*
In dieser Form liest der Befehl das Literaturverzeichnis vom angegebenen Tcl-Kanal *chan* und gibt anschließend dieselbe Datenstruktur zurück wie oben beschrieben.

Ist die Option **-command** angegeben, ist das Ergebnis stattdessen ein Handle des Parsers, und die gesamte Verarbeitung erfolgt inkrementell im Hintergrund. Ist die Eingabe vollständig gelesen, wird die Rückruffunktion *cmd* mit dem Ergebnis des Parsens aufgerufen. Die Rückruffunktion ist so definiert
cmd *token parseresult*

Das Parsergebnis weist – bei den einfacheren Formen des Parsers – die oben erläuterte Struktur auf.

Hinweis: Der Parser schließt den Kanal *nicht*, nachdem er ihn vollständig gelesen hat. Das bleibt Aufgabe des Nutzers des Parsers.

::bibtex::parse ?-recordcommand *recordcmd*? ?-preamblecommand *preamblecmd*? ?-stringcommand *stringcmd*? ?-commentcommand *commentcmd*? ?-progresscommand *progresscmd*? ?-casesensitivestrings *bool*? (*text* | -channel *chan*)

Dies ist die Form des Parsers auf der niedrigsten Ebene. Zurückgegeben wird ein Handle für den Parser. Während der Verarbeitung ruft er die angegebenen Rückrufbefehle für jede im Literaturverzeichnis gefundene Art von Daten auf.

Die Verarbeitung läuft inkrementell im Hintergrund, genau dann und nur dann, wenn ein Tcl-Kanal *chan* angegeben ist. Bei einem *text* erfolgt die Verarbeitung sofort, und alle Rückruffunktionen werden aufgerufen, bevor der Befehl selbst zurückkehrt.

Die Rückruffunktionen, also **cmd*, sind sämtlich Befehlspräfixe und werden mit zusätzlichen Argumenten am Ende aufgerufen. Die Bedeutung der Argumente hängt von der Rückruffunktion ab und wird unten erklärt. Das erste Argument ist immer das Handle des Parsers, der die Rückruffunktion aufruft.

-casesensitivestrings

Diese Option nimmt einen booleschen Wert entgegen. Ist sie gesetzt, unterscheidet die Verarbeitung der String-Makros Groß- und Kleinschreibung. Voreingestellt ist die Verarbeitung ohne diese Unterscheidung.

recordcmd *token type key recorddict*

Diese Rückruffunktion wird aufgerufen, wenn der Parser einen Datensatz des Literaturverzeichnisses in der Eingabe erkennt. Die Argumente sind der Datensatztyp, der Bibliographieschlüssel des Datensatzes und ein Dictionary mit den Schlüsseln und Werten, die den Datensatz beschreiben. Alle dem Parser bekannten String-Makros sind bereits expandiert.

preamblecmd *token preambletext*

Diese Rückruffunktion wird aufgerufen, wenn der Parser einen @preamble-Block in der Eingabe erkennt. Das einzige weitere Argument ist der Text im Präambel-Block. Standardmäßig werden solche Einträge ignoriert.

stringcmd *token stringdict*

Diese Rückruffunktion wird aufgerufen, wenn der Parser eine Makrodefinition auf @string-Basis in der Eingabe erkennt. Das Argument ist ein Dictionary mit den Makronamen als Schlüsseln und den Ersetzungszeichenketten als Werten. Standardmäßig werden solche Definitionen dem Zustand des Parsers hinzugefügt, zur Verwendung in späteren Datensätzen.

commentcmd *token commenttext*

Diese Rückruffunktion wird aufgerufen, wenn der Parser einen Kommentar in der Eingabe erkennt. Das einzige weitere Argument ist der Kommentartext. Standardmäßig werden solche Einträge ignoriert.

progresscmd *token percent*

Diese Rückruffunktion wird während der Verarbeitung aufgerufen, um dem Benutzer den bisherigen Fortschritt mitzuteilen. Ihr Argument ist der Prozentsatz der verarbeiteten

Daten, als ganze Zahl zwischen **0** und **100**. Bei inkrementeller Verarbeitung ist der Prozentsatz immer **-1**, weil die Gesamtzahl der Einträge vorher nicht bekannt ist.

::bibtex::wait *token*
Dieser Befehl wartet, bis der durch *token* dargestellte Parser fertig ist, und kehrt dann zurück. Das zurückgegebene Ergebnis ist die leere Zeichenkette.

::bibtex::destroy *token*
Dieser Befehl räumt den gesamten internen Zustand auf, der zum Parser mit dem Handle *token* gehört, und zerstört ihn damit. Der Befehl kann aus den Rückruffunktionen des Parsers heraus aufgerufen werden, um die Verarbeitung zu beenden.

::bibtex::addStrings *token stringdict*
Dieser Befehl fügt die im Dictionary *stringdict* gespeicherten Makrodefinitionen dem Parser mit dem Handle *token* hinzu.

Die Schlüssel des Dictionarys sind die Makronamen, die Werte ihre Ersetzungszeichenketten. Dieser Befehl hat die passende Signatur, um als **-stringcommand**-Rückruffunktion bei einem Aufruf von **::bibtex::parse** zu dienen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *bibtex* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright


```
Copyright (c) 2005 for documentation, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


blowfish

blowfish

Stichwörter: blowfish, block cipher, cryptography, encryption, security

Name

blowfish – Implementierung der Blowfish-Blockverschlüsselung

Übersicht

package require Tcl 8.5 9
package require blowfish ?**1.0.6**?

::blowfish::blowfish ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-out channel*? ?*-chunksize size*? ?*-pad padchar*? [*-in channel* | ?--? *data*]
::blowfish::Init *mode keydata iv*
::blowfish::Encrypt *Key data*
::blowfish::Decrypt *Key data*
::blowfish::Reset *Key iv*
::blowfish::Final *Key*

Beschreibung

Dieses Paket ist eine Implementierung des von Bruce Schneier entwickelten Blowfish-Algorithmus [1] in Tcl. Blowfish ist eine 64-Bit-Blockverschlüsselung, die für den schnellen Betrieb auf 32-Bit-Architekturen ausgelegt ist und eine variable Schlüssellänge unterstützt. Diese Implementierung unterstützt die Blowfish-Verschlüsselung im ECB- und CBC-Modus.

Befehle

::blowfish::blowfish ?*-mode [ecb|cbc]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-out channel*? ?*-chunksize size*? ?*-pad padchar*? [*-in channel* | ?--? *data*]

Wendet den Algorithmus **blowfish** entweder auf die durch das Argument bereitgestellten Daten oder auf die aus dem Kanal *-in* gelesenen Daten an. Wenn ein *-out*-Kanal angegeben wird, wird das Ergebnis in diesen Kanal geschrieben.

Die Option *-key* muss angegeben werden. Dieser Parameter nimmt eine binäre Zeichenkette variabler Länge entgegen und dient zur Generierung des **blowfish**-Schlüsselschemas. Beachten Sie, dass die Erstellung eines Schlüsselschemas in Blowfish ein recht rechenintensiver Vorgang ist; daher lohnt es sich, den Schlüssel nach Möglichkeit wiederzuverwenden. Siehe **Reset**.

Die Optionen *-mode* und *-dir* sind optional und sind standardmäßig auf den CBC-Modus bzw. encrypt gesetzt. Der Initialisierungsvektor *-iv* erwartet ein 8-Byte-Binärargument, dessen Standardwert aus 8 Nullen besteht. Weitere Informationen zu den verfügbaren Modi und deren Verwendung finden Sie unter Betriebsarten.

Blowfish ist eine 64-Bit-Blockverschlüsselung. Das bedeutet, dass die Daten in Einheiten bereitgestellt werden müssen, die ein Vielfaches von 8 Byte sind. Der Befehl *blowfish* fügt standardmäßig bei Bedarf Nul-Zeichen hinzu, um die Eingabedaten auf ein Vielfaches von 8 Byte aufzufüllen. Die Befehle der Programmier-API fügen niemals Innenabstände hinzu, sondern lösen stattdessen einen Fehler aus, wenn die Eingabe kein Vielfaches der Blockgröße ist. Mit der Option *-pad* können Sie den Innenabstand ändern oder die Innenabstände deaktivieren, wenn die leere Zeichenkette als Argument übergeben wird.

Programmierschnittstelle

::blowfish::Init *mode keydata iv*

Erstellt eine neue Blowfish-Schlüsselplanung unter Verwendung der angegebenen Schlüsseldaten und des vorgegebenen Initialisierungsvektors. Der Initialisierungsvektor wird im ECB-Modus nicht verwendet, ist jedoch für den CBC-Modus von Bedeutung. Weitere

Informationen zu den Verschlüsselungsmodi finden Sie unter Betriebsarten.

::blowfish::Encrypt *Key data*
Verschlüsselt die bereitgestellten Daten mit einem vorbereiteten Schlüssel aus **Init**. Das Argument "data" sollte ein binäres Array sein, dessen Größe ein Vielfaches der Blockgröße von 8 Byte beträgt. Das Ergebnis ist ein binäres Array, dessen Größe der der Eingabe der verschlüsselten Daten entspricht.

::blowfish::Decrypt *Key data*
Hinweis: Der gleiche Schlüssel kann sowohl zum Verschlüsseln als auch zum Entschlüsseln von Daten verwendet werden, sofern der Initialisierungsvektor für den CBC-Modus entsprechend zurückgesetzt wird.

::blowfish::Reset *Key iv*
Setzt den Initialisierungsvektor zurück. Dies ermöglicht es dem Programmierer, einen Schlüssel wiederzuverwenden und den Aufwand für die Neugenerierung des Schlüsselschemas zu vermeiden, wenn dieselben Schlüsseldaten mehrfach verwendet werden.

::blowfish::Final *Key*
Diese Funktion sollte aufgerufen werden, um die mit *Key* verbundenen Ressourcen freizugeben. Sobald diese Funktion aufgerufen wurde, darf der Schlüssel nicht mehr verwendet werden.

Betriebsarten

Electronic Code Book (ECB)

Der ECB-Modus ist der Grundmodus aller Blockchiffren. Jeder Block wird unabhängig verschlüsselt, sodass identischer Klartext bei der Verschlüsselung mit demselben Schlüssel zu identischem Ergebnis führt. Etwaige Verschlüsselungsfehler wirken sich zwar nur auf einen einzelnen Block aus, dieser ist jedoch anfällig für Angriffe mit bekanntem Klartext.

Cipher Block Chaining (CBC)

Im CBC-Modus wird die Ausgabe der Verschlüsselung des letzten Blocks zur Beeinflussung des aktuellen Blocks herangezogen. Für die Verarbeitung des ersten Blocks wird ein Initialisierungsvektor verwendet, dessen Größe der Chiffreblockgröße entspricht. Der Initialisierungsvektor sollte zufällig ausgewählt und als erster Block der Ausgabe übertragen werden. Fehler bei der Verschlüsselung wirken sich auf den aktuellen Block und den nächsten Block aus; danach korrigiert sich die Chiffre von selbst. CBC ist der am häufigsten verwendete Modus bei der Softwareverschlüsselung.

Beispiele

```
% blowfish::blowfish -hex -mode ecb -dir encrypt -key secret01 "hello,
    world!"
d0d8f27e7a374b9e2dbd9938dd04195a
```

```
set Key [blowfish::Init cbc $eight_bytes_key_data $eight_byte_iv]
append ciphertext [blowfish::Encrypt $Key $plaintext]
append ciphertext [blowfish::Encrypt $Key $additional_plaintext]
blowfish::Final $Key
```

Literatur

1. Schneier, B. "Applied Cryptography, 2. Auflage", 1996, ISBN 0-471-11709-9, Verlag John Wiley & Sons.

Autoren

Frank Pilhofer, Pat Thoyts

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *blowfish* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

cache

async

Stichwörter: async, asynchronous, cache, callback, synchronous

Name

cache::async – Asynchroner In-Memory-Cache

Übersicht

package require Tcl 8.5 9
package require cache::async ?**0.3.2**?
::cache::async *objectName commandprefix* ?*options...*?
objectName **get** *key donecmdprefix*
objectName **set** *key value*
objectName **unset** *key*
objectName **exists** *key*
objectName **clear** ?*key*?

Beschreibung

Dieses Paket stellt Objekte bereit, die Daten im Arbeitsspeicher zwischenspeichern und hinsichtlich Anfragen und Antworten asynchron arbeiten. Die Objekte sind unabhängig von Cache-Schlüsseln und -Werten, und unbekannte Methoden werden an den Anbieter der zwischengespeicherten Daten weitergeleitet. Diese beiden Eigenschaften erleichtern die Verwendung von Caches als Fassade für jeden beliebigen Datenanbieter.

API

Das Paket exportiert eine Klasse namens "**cache::async**", wie im Folgenden beschrieben.

::cache::async *objectName commandprefix ?options...?*

Der Befehl erstellt ein neues *Cache*-Objekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *objectName* lautet. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Objekt auszuführen.

Das *commandprefix* ist die Aktion, die ausgeführt werden soll, wenn ein Benutzer Daten aus dem Cache anfordert und der Cache den Schlüssel noch nicht kennt. Wenn der Befehl ausgeführt wird, erhält der *commandprefix* drei zusätzliche Argumente: die Zeichenkette **get**, den angeforderten Schlüssel und das Cache-Objekt selbst in Form seines Objektbefehls – in dieser Reihenfolge. Die Ausführung der Aktion erfolgt in einem Idle-Handler, wodurch sie von der ursprünglichen Anfrage entkoppelt wird.

Die einzige unterstützte Option ist

-full-async-results

Diese Option legt das Verhalten des Caches für den Fall fest, dass die angeforderten Schlüssel dem Cache zum Zeitpunkt der **get**-Anfrage bereits bekannt sind. Standardmäßig werden solche Anfragen ebenfalls asynchron beantwortet. Wird diese Option auf "**false**" gesetzt, wird der Cache gezwungen, synchron auf diese Anfragen zu reagieren, allerdings weiterhin über die angegebene Rückruffunktion.

Die durch die oben genannten Klassenbefehle erstellten Objektbefehle haben folgende Form:

objectName* get *key donecmdprefix

Diese Methode fordert die Daten für den *key* aus dem Cache an. Wenn die Daten noch nicht bekannt sind, wird das bei der Erstellung des Cache-Objekts festgelegte Befehlspräfix verwendet, um diese Informationen abzufragen.

Sobald die Informationen verfügbar sind oder werden, wird das *donecmdprefix* ausgeführt, um das Ergebnis an den Aufrufer zu übermitteln. Dieser Befehl wird entweder mit 2 oder 3 Argumenten aufgerufen, d. h.

1. Die Zeichenfolge "**set**", der *key* und der Wert.
2. Die Zeichenfolge "**unset**" und der *key*.

Diese beiden Möglichkeiten dienen dazu, entweder den Wert für den *key* anzugeben oder zu signalisieren, dass für den *key* kein Wert definiert ist. Letzteres unterscheidet sich von dem Fall, dass der Cache den *key* nicht kennt.

Bei einem Cache-Objekt, das auf vollständige Asynchronität konfiguriert ist (Standard), wird das *donecmdprefix* stets in einem Idle-Handler ausgeführt, wodurch es von der Anfrage entkoppelt wird. Andernfalls wird die Rückruffunktion synchron aufgerufen, wenn der *key* dem Cache zum Zeitpunkt des Aufrufs bekannt ist.

Ein weiterer wichtiger Aspekt des Verhaltens des Caches: Da er asynchron arbeitet, ist es möglich, dass mehrere **get**-Anfragen für denselben *key* gestellt werden, bevor dieser antworten kann. In diesem Fall sendet der Cache nur eine Datenanfrage an den Anbieter – und zwar für die erste dieser Anfragen –, setzt die anderen aus und benachrichtigt sie alle, wenn die Daten verfügbar sind.

objectName* set *key value

siehe unten ...

objectName* unset *key

Diese beiden Methoden stehen zur Verfügung, damit Nutzer des Caches dem Cache Schlüssel entweder mit einem *value* oder als undefiniert mitteilen können.

Es wird erwartet, dass der Datenanbieter (siehe *commandprefix* des Konstruktors) diese Methoden als Antwort auf Datenanfragen für unbekannte Schlüssel verwendet.

Hinweis: Dies entspricht der eigenen API des Caches gegenüber seinem Aufrufer: Der Cache ruft das *donecmd* von **get**-Anfragen auf, die an ihn selbst gerichtet wurden – entweder mit "set key value" oder "unset key" –, im Gegensatz zur Ausgabe von **get**-Anfragen an seinen eigenen Anbieter sendet, wobei er sich selbst anstelle des *donecmd* einsetzt und erwartet, entweder mit "set key Wert" oder "unset key" aufgerufen zu werden.

Dies bedeutet auch, dass diese Methoden das *donecmd* aller **get**-Anfragen aufrufen, die auf Informationen über den geänderten *key* warten.

objectName* exists *key

Diese Methode fragt den Cache nach Informationen zum *key* ab und gibt einen booleschen Wert zurück. Das Ergebnis lautet **true**, wenn der Schlüssel bekannt ist, andernfalls **false**.

***objectName* clear ?*key*?**

Diese Methode setzt den Status entweder des angegebenen *key* oder aller dem Cache bekannten Schlüssel zurück, sodass diese als unbekannt gelten. Dadurch wird erreicht, dass bei künftigen **get**-Anfragen die Informationen erneut vom Anbieter geladen werden müssen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *cache* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright


```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


clay

clay

Stichwörter: clay, TclOO, oo

Name

clay – Ein minimalistischer Rahmen für umfangreiche objektorientierte Projekte

Übersicht

package require Tcl 8.6 9
package require uuid
package require oo::dialect

proc **clay::PROC** *name arglist body* ?*ninja* ?

proc **clay::_ancestors** *resultvar class*

proc **clay::ancestors** ?*args*?

proc **clay::args_to_dict** ?*args*?

proc **clay::args_to_options** ?*args*?

proc **clay::dynamic_arguments** *ensemble method arglist* ?*args*?

proc **clay::dynamic_wrongargs_message** *arglist*

proc **clay::is_dict** *d*

proc **clay::is_null** *value*

proc **clay::leaf** ?*args*?

proc **clay::K** *a b*

proc **clay::noop** ?*args*?

proc **clay::cleanup**

proc **clay::object_create** *objname* ?*class* ?

proc **clay::object_rename** *object newname*

proc **clay::object_destroy** ?*args*?

proc **clay::path** ?*args*?

proc **clay::putb** ?*map*? *text*

proc **clay::script_path**

proc **clay::NSNormalize** *qualname*

proc **clay::uuid_generate** ?*args*?

proc **clay::uuid::generate_tcl_machinfo**

proc **clay::uuid::tostring** *uuid*

proc **clay::uuid::fromstring** *uuid*

proc **clay::uuid::equal** *left right*

proc **clay::uuid** *cmd* ?*args*?

proc **clay::tree::sanitize** *dict*

proc **clay::tree::_sanitizeb** *path varname dict*

proc **clay::tree::storage** *rawpath*

proc **clay::tree::dictset** *varname* ?*args*?

proc **clay::tree::dictmerge** *varname* ?*args*?

proc **clay::tree::merge** ?*args*?

proc **dictargs::proc** *name argspec body*

proc **dictargs::method** *name argspec body*
proc **clay::dialect::Push** *class*
proc **clay::dialect::Peek**
proc **clay::dialect::Pop**
proc **clay::dialect::create** *name ?parent ?*
proc **clay::dialect::NSNormalize** *namespace qualname*
proc **clay::dialect::DefineThunk** *target ?args?*
proc **clay::dialect::Canonical** *namespace NSpace class*
proc **clay::dialect::Define** *namespace class ?args?*
proc **clay::dialect::Aliases** *namespace ?args?*
proc **clay::dialect::SuperClass** *namespace ?args?*
proc **clay::dynamic_methods** *class*
proc **clay::dynamic_methods_class** *thisclass*
proc **clay::define::Array** *name ?values ?*
proc **clay::define::Delegate** *name info*
proc **clay::define::constructor** *arglist rawbody*
proc **clay::define::Class_Method** *name arglist body*
proc **clay::define::class_method** *name arglist body*
proc **clay::define::clay** *?args?*
proc **clay::define::destructor** *rawbody*
proc **clay::define::Dict** *name ?values ?*
proc **clay::define::Option** *name ?args?*
proc **clay::define::Method** *name argstyle argspec body*
proc **clay::define::Option_Class** *name ?args?*
proc **clay::define::Variable** *name ?default ?*
proc **clay::ensemble_methodbody** *ensemble einfo*
proc **clay::define::Ensemble** *rawmethod ?args?*
proc **clay::event::cancel** *self ?task **?*
proc **clay::event::generate** *self event ?args?*
proc **clay::event::nextid**
proc **clay::event::Notification_list** *self event ?stackvar ?*
proc **clay::event::notify** *rcpt sender event eventinfo*
proc **clay::event::process** *self handle script*
proc **clay::event::schedule** *self handle interval script*
proc **clay::event::subscribe** *self who event*
proc **clay::event::unsubscribe** *self ?args?*
proc **clay::singleton** *name script*

method **clay ancestors**
method **clay dump**
method **clay find** *path* ?**path...**?
method **clay get** *path* ?**path...**?
method **clay GET** *path* ?**path...**?
method **clay merge** *dict* ?**dict...**?
method **clay replace** *dictionary*
method **clay search** *path* ?**path...**?
method **clay set** *path* ?**path...**? *value*
method **clay ancestors**
method **clay cache** *path value*
method **clay cget** *field*
method **clay delegate** ?*stub*? ?*object*?
method **clay dump**
method **clay ensemble_map**
method **clay eval** *script*
method **clay evolve**
method **clay exists** *path* ?**path...**?
method **clay flush**
method **clay forward** *method object*
method **clay get** *path* ?**path...**?
method **clay leaf** *path* ?**path...**?
method **clay merge** *dict* ?**dict...**?
method **clay mixin** *class* ?**class...**?
method **clay mixinmap** ?*stub*? ?*classes*?
method **clay provenance** *path* ?**path...**?
method **clay replace** *dictionary*
method **clay search** *path valuevar isleafvar*
method **clay source** *filename*
method **clay set** *path* ?**path...**? *value*
method **InitializePublic**

Beschreibung

Clay führt sowohl für **oo::class** als auch für **oo::object** ein Ensemble namens "clay" ein. Dieses Ensemble verwaltet alle übergeordneten Interaktionen innerhalb des Rahmens. Clay speichert strukturierte Daten. Clan verwaltet die Weiterleitung der Methoden. Clay verfügt über Funktionen zur Verwaltung der komplexen Interaktionen, die durch Mixins entstehen.

Das zentrale Konzept besteht darin, dass sich im Inneren jedes Objekts und jeder Klasse (die ja

eigentlich ebenfalls Objekte sind) ein Dict namens "clay" befindet. Was in diesem Dict gespeichert ist, bleibt der Fantasie überlassen. Da dieses Dict jedoch über eine öffentliche Methode zugänglich gemacht wird, können wir strukturierte Daten zwischen Objekten, Klassen und Mixins austauschen.

Strukturierte Daten

Clay nutzt einen standardisierten Satz von Methodeninteraktionen und Introspektion, den TclOO bereits bereitstellt, um Suchvorgänge in Echtzeit durchzuführen. Suchvorgänge in Echtzeit bedeuten, dass die Daten stets aktuell sind, und wir vermeiden viele der Konflikte, die entstehen würden, wenn sich Objekte während des Betriebs mit denen anderer Klassen vermischen würden.

Die **clay**-Methoden verfügen sowohl für Klassen als auch für Objekte über eine "get"- und eine "set"-Methode. Bei Objekten durchsucht die "get"-Methode das lokale "clay"-Dict. Wenn das angeforderte Blatt nicht gefunden wird oder wenn die Abfrage auf einen Zweig bezieht, beginnt das System damit, die "clay"-Methoden aller Klassen abzufragen, die das Objekt implementieren, sowie aller Vorfahren dieser Klassen und aller Klassen, die in dieses Objekt "gemischt" wurden, einschließlich aller ihrer Vorfahren.

Als Zweige vorgesehene Einträge in einem Baum enden mit einem Verzeichnis-Shrug (/). Als Blätter vorgesehene Einträge bleiben unverändert. Dies dient dem Tool, das die Suchergebnisse erstellt, als Orientierungshilfe, um zu erkennen, welche Teile eines Dicts als Zweige und welche als Blätter vorgesehen sind. In einfachen Fällen kann die Kennzeichnung als Zweig ignoriert werden:

```
::oo::class create ::foo { }
::foo clay set property/ color blue
::foo clay set property/ shape round

set A [::foo new]
$A clay get property/
{color blue shape round}

$A clay set property/ shape square
$A clay get property/
{color blue shape square}
```

Wenn Sie jedoch anfangen, Textblöcke zu speichern, wird es unübersichtlich, zu erraten, welches Feld ein Dictionary ist und welches nicht:

```
::foo clay set description {A generic thing of designated color and
    shape}

$A clay get description
{A generic thing of designated color and shape}

Without a convention for discerning branches for leaves what should
    have been a value can be accidentally parsed as a dictionary, and
    merged with all of the other values that were never intended to be
```

```
    merge. Here is an example of it all going wrong:
::oo::class create ::foo { }
# Add description as a leaf
::foo clay set description  {A generic thing of designated color and
    shape}
# Add description as a branch
::foo clay set description/  {A generic thing of designated color and
    shape}

::oo::class create ::bar {
  superclass foo
}
# Add description as a leaf
::bar clay set description  {A drinking establishment of designated
    color and shape and size}
# Add description as a branch
::bar clay set description/  {A drinking establishment of designated
    color and shape and size}

set B [::bar new]
# As a leaf we get the value verbatim from he nearest ancestor
$B clay get description
  {A drinking establishment of designated color and shape and size}
# As a branch we get a recursive merge
$B clay get description/
{A drinking establishment of designated color and size thing of}
```

Clay-Dialekt

Clay basiert auf dem Modul oo::dialect aus Tcllib. Mit oo::dialect können Sie entweder Schlüsselwörter direkt zu Clay hinzufügen oder auf der Grundlage von Clay eine eigene Metaklasse und einen eigenen Satz von Schlüsselwörtern erstellen. Einzelheiten zu den Schlüsselwörtern und deren Funktionsweise finden Sie in den Funktionen des Namespace ::clay::define.

Methoden-Weiterleitung

Methoden-Weiterleitung: Manchmal ist es sinnvoll, über ein externes Objekt zu verfügen, das so aufgerufen werden kann, als wäre es eine Methode des Objekts. Clay bietet eine Ensemble-Methode namens "delegate" an, um diese Weiterleitung durchzuführen und um zu ermitteln, welche Methoden auf diese Weise delegiert sind. Alle delegierten Methoden sind durch HTML-ähnliche Tags (< >) gekennzeichnet.

```
::clay::define counter {
  Variable counter 0
  method incr {{howmuch 1}} {
    my variable counter
    incr counter $howmuch
```

```
  }
  method value {} {
    my variable counter
    return $counter
  }
  method reset {} {
    my variable counter
    set counter 0
  }
}
::clay::define example {
  variable buffer
  constructor {} {
    # Build a counter object
    set obj [namespace current]::counter
    ::counter create $obj
    # Delegate the counter
    my delegate <counter> $obj
  }
  method line {text} {
    my <counter> incr
    append buffer $text
  }
}

set A [example new]
$A line {Who's line is it anyway?}
$A <counter> value
1
```

Befehle

proc clay::PROC *name arglist body* ?*ninja* ?

Da viele Funktionen dieses Pakets möglicherweise als Befehle in zukünftige Tcl-Kerne aufgenommen oder von Paketen in binärer Form bereitgestellt werden, benötige ich eine deklarative Möglichkeit, um anzugeben: *Erstellen Sie diesen Befehl, falls noch keiner vorhanden ist*. Das Argument *ninja* ist ein Skript, das ausgeführt werden soll, wenn der Befehl über diesen Mechanismus erstellt wird.

proc clay::_ancestors *resultvar class*

siehe unten ...

proc clay::ancestors ?*args*?

siehe unten ...

proc clay::args_to_dict ?*args*?

siehe unten ...

proc clay::args_to_options ?*args*?

siehe unten ...

proc clay::dynamic_arguments *ensemble method arglist* ?*args*?
siehe unten ...

proc clay::dynamic_wrongargs_message *arglist*
siehe unten ...

proc clay::is_dict *d*
siehe unten ...

proc clay::is_null *value*
siehe unten ...

proc clay::leaf ?*args*?
siehe unten ...

proc clay::K *a b*
siehe unten ...

proc clay::noop ?*args*?
Führt einen Noop aus. Dies ist beim Prototyping nützlich, um Code-Blöcke zu kommentieren, ohne sie tatsächlich auskommentieren zu müssen. Außerdem dient es als praktische Standardoption für die Weiterleitung der Methoden, wenn noch kein Delegat zugewiesen wurde.

proc clay::cleanup
Verarbeitet den Prozess der Löschung der zu löschenden Objekte

proc clay::object_create *objname* ?*class* ?
siehe unten ...

proc clay::object_rename *object newname*
siehe unten ...

proc clay::object_destroy ?*args*?
Ein Objekt für die Löschung bei der nächsten Bereinigung markieren

proc clay::path ?*args*?
siehe unten ...

proc clay::putb ?*map*? *text*
Fügt einer Variable eine Textzeile hinzu. Wenden Sie optional eine Zeichenfolgenzuordnung an.

proc clay::script_path
siehe unten ...

proc clay::NSNormalize *qualname*
siehe unten ...

proc clay::uuid_generate ?*args*?
siehe unten ...

proc clay::uuid::generate_tcl_machinfo
siehe unten ...

proc clay::uuid::tostring *uuid*
siehe unten ...

proc clay::uuid::fromstring *uuid*
Konvertiert eine Zeichenfolgendarstellung einer UUID in ihr binary format.

proc clay::uuid::equal *left right*
Vergleicht zwei UUIDs auf Gleichheit.

proc clay::uuid *cmd* ?*args*?
UUID generieren -> Zeichenfolgenrepräsentation einer neuen UUID; UUID entspricht UUID1 und UUID2

proc clay::tree::sanitize *dict*
Gibt ein Dictionary aus, aus dem alle von **clay::tree::merge** hinzugefügten Einträge entfernt wurden.

proc clay::tree::_sanitizeb *path varname dict*
Hilfsfunktion für ::clay::tree::sanitize: Formatiert die Zeichenfolgendarstellung eines Dictionary-Elements innerhalb eines für Menschen lesbaren Zeilenstroms und ermittelt, ob die Funktion sich selbst mit weiterer Einrückung aufrufen muss, um einen Unterzweig als Ausdruck darzustellen

proc clay::tree::storage *rawpath*
Gibt den Pfad als Speicherpfad für clay::tree zurück, wobei alle Verzweigungsbegrenzer entfernt werden. Dieser Befehl teilt Argumente zudem auf, wenn sie ein "/" enthalten.

Beispiele:

```
> clay::tree::storage {foo bar baz bang}
foo bar baz bang
> clay::tree::storage {foo bar baz bang/}
foo bar baz bang
> clay::tree::storage {foo bar baz bang:}
foo bar baz bang:
> clay::tree::storage {foo/bar/baz bang:}
foo bar baz bang:
> clay::tree::storage {foo/bar/baz/bang}
foo bar baz bang
```

proc clay::tree::dictset *varname* ?*args*?
Legt ein Element mit einem rekursiven Dictionary fest, wobei alle Verzweigungen auf dem Weg zum Endelement markiert werden. Wenn der Wert im verschachtelten Dictionary nicht existiert, wird er als Blatt hinzugefügt. Wenn der Wert bereits als Zweig existiert, wird der übergebene Wert mit dem vorhandenen Wert zusammengeführt, sofern es sich um ein gültiges Dictionary handelt. Wenn der übergebene Wert kein gültiges Dictionary ist, überschreibt er den gespeicherten Wert und wird fortan als Blatt behandelt.

Beispiele:

```
> set r {}
> ::clay::tree::dictset r option color default Green
. {} option {. {} color {. {} default Green}}
> ::clay::tree::dictset r option {Something not dictlike}
. {} option {Something not dictlike}
# Note that if the value is not a dict, and you try to force it to
   be
# an error with be thrown on the merge
> ::clay::tree::dictset r option color default Blue
missing value to go with key
```

proc clay::tree::dictmerge *varname* ?*args*?
Eine rekursive Form der Zusammenführung von Wörterbüchern, die dazu dient, Variablen direkt an Ort und Stelle zu ändern.

Beispiele:

```
> set mydict {sub/ {sub/ {description {a block of text}}}}
> ::clay::tree::dictmerge mydict {sub/ {sub/ {field {another block
   of text}}}}]
> clay::tree::print $mydict
sub/ {
  sub/ {
    description {a block of text}
    field {another block of text}
  }
}
```

proc clay::tree::merge ?*args*?
Eine rekursive Methode zum Zusammenführen von Wörterbüchern

Eine Routine zum rekursiven Durchsuchen von Wörterbüchern und deren Zusammenführung, adaptiert von http://stevehavelka.com/tcl-dict-operation-nested-merge/

Beispiele:

```
> set mydict {sub/ {sub/ {description {a block of text}}}}
> set odict [clay::tree::merge $mydict {sub/ {sub/ {field {another
   block of text}}}}]
> clay::tree::print $odict
sub/ {
  sub/ {
    description {a block of text}
    field {another block of text}
  }
}
```

proc dictargs::proc *name argspec body*
"Benannte Prozeduren" als neuer Befehl

proc dictargs::method *name argspec body*
siehe unten ...

proc clay::dialect::Push *class*
siehe unten ...

proc clay::dialect::Peek
siehe unten ...

proc clay::dialect::Pop
siehe unten ...

proc clay::dialect::create *name* ?*parent* ?
Diese Prozedur generiert einen Namespace, eine "Mutter aller Klassen" sowie einen grundlegenden Satz von Richtlinien für diesen Dialekt.

proc clay::dialect::NSNormalize *namespace qualname*
Hilfsbefehle; nicht für den direkten Aufruf vorgesehen.

proc clay::dialect::DefineThunk *target ?args?*
siehe unten ...

proc clay::dialect::Canonical *namespace NSpace class*
siehe unten ...

proc clay::dialect::Define *namespace class ?args?*
Implementierung des Befehls "define" in den Sprachen

proc clay::dialect::Aliases *namespace ?args?*
siehe unten ...

proc clay::dialect::SuperClass *namespace ?args?*
siehe unten ...

proc clay::dynamic_methods *class*
siehe unten ...

proc clay::dynamic_methods_class *thisclass*
siehe unten ...

proc clay::define::Array *name ?values ?*
Neue OO-Schlüsselwörter für Ton

proc clay::define::Delegate *name info*
Eine Anmerkung, die angibt, dass Objekte dieser Klasse mit delegierten Methoden interagieren. Die Anmerkung ist als Dictionary konzipiert, und der einzige reservierte Schlüssel ist *description*, eine für Menschen lesbare Beschreibung.

proc clay::define::constructor *arglist rawbody*
siehe unten ...

proc clay::define::Class_Method *name arglist body*
Legt eine Methode für das Klassenobjekt selbst fest, anstatt für Objekte dieser Klasse

proc clay::define::class_method *name arglist body*
Und ein Alias für das neue Schlüsselwort "Class_Method"

proc clay::define::clay *?args?*
siehe unten ...

proc clay::define::destructor *rawbody*
siehe unten ...

proc clay::define::Dict *name ?values ?*
siehe unten ...

proc clay::define::Option *name ?args?*
Legt eine Option für die Klasse fest

proc clay::define::Method *name argstyle argspec body*
siehe unten ...

proc clay::define::Option_Class *name ?args?*
Definiert eine Klasse von Optionen. Alle Feld-/Wert-Paare werden von einer Option geerbt, die *name* als Klassenfeld festlegt.

proc clay::define::Variable *name ?default ?*
Dieses Schlüsselwort lässt sich auch wie folgt ausdrücken:

```
property variable NAME {default DEFAULT}
```

Variablen, die in der Eigenschaft "variable" registriert sind, werden ebenfalls initialisiert (sofern sie fehlen), wenn das Objekt über die Methode *morph* die Klasse wechselt.

proc clay::ensemble_methodbody *ensemble einfo*
Erstellt den Hauptteil der öffentlichen Dispatch-Methode eines Ensembles. "ensemble" ist der Name des Ensembles. "einfo" ist ein Dictionary mit Methoden für das Ensemble, und jeder Wert ist ein Skript, das beim Dispatch ausgeführt werden soll.

Beispiele:

```
::clay::ensemble_methodbody foo {
  bar {tailcall my Foo_bar {*}$args}
  baz {tailcall my Foo_baz {*}$args}
  clock {return [clock seconds]}
  default {puts "You gave me $method"}
}
```

proc clay::define::Ensemble *rawmethod* ?*args*?
siehe unten ...

proc clay::event::cancel *self* ?*task* *?
Ein geplantes Ereignis absagen

proc clay::event::generate *self event* ?*args*?
Ereignis generieren: Fügt einen Abonnementmechanismus für Objekte hinzu, um festzustellen, wer dieses Ereignis erhalten hat, und um Spam oder unendliche Rekursion zu verhindern

proc clay::event::nextid
siehe unten ...

proc clay::event::Notification_list *self event* ?*stackvar* ?
Wird rekursiv aufgerufen, um eine Liste der Personen zu erstellen, die Benachrichtigungen erhalten

proc clay::event::notify *rcpt sender event eventinfo*
Endgültige Zustellung an den vorgesehenen Empfänger als Objekt

proc clay::event::process *self handle script*
Ein Ereignisskript im globalen Namespace auswerten

proc clay::event::schedule *self handle interval script*
Ein Ereignis für einen späteren Zeitpunkt planen

proc clay::event::subscribe *self who event*
Ein Objekt auf ein Ereignis-Muster abonnieren

proc clay::event::unsubscribe *self* ?*args*?
Ein Objekt aus einem Ereignis-Muster abmelden

proc clay::singleton *name script*
Ein Objekt, das als eigene Klasse gedacht ist.

Klassen

Klasse clay::class

Methoden

method clay ancestors
Gibt diese Klasse und alle Vorfahren in Suchreihenfolge zurück.

method clay dump
Gibt einen vollständigen Auszug der Clay-Daten dieses Objekts zurück, jedoch ausschließlich die Clay-Daten dieses Objekts.

method clay find *path* ?path...?
Ruft einen Datenblock aus dem Clay-System ab. Wenn das letzte Element von *path* ein Zweig ist, gibt es eine rekursive Zusammenführung aller Daten dieses Objekts und seiner Klassen mit den Daten in diesem Zweig zurück. Wenn das letzte Element ein Blatt ist, wird dieses Objekt nach einem passenden Blatt durchsucht oder es werden alle zugehörigen Klassen nach einem passenden Blatt durchsucht und der erste gefundene Wert gegeben. Wird kein Wert gefunden, gibt es eine leere Zeichenkette zurück. Wird ein Zweig zurückgegeben, wird der oberste Eintrag weggelassen.

method clay get *path* ?path...?
Ruft einen Datenblock aus dem "Clay"-System der Klasse ab. Wird kein Wert gefunden, gibt es eine leere Zeichenkette zurück. Wird ein Zweig zurückgegeben, wird der oberste Eintrag weggelassen.

method clay GET *path* ?path...?
Ruft einen Datenblock aus dem "Clay"-System der Klasse ab. Wird kein Wert gefunden, gibt es eine leere Zeichenkette zurück.

method clay merge *dict* ?dict...?
Fügt die angegebenen Dictionarys rekursiv in den lokalen Clay-Speicher des Objekts ein.

method clay replace *dictionary*
Ersetzt den Inhalt des internen Clay-Speichers durch das angegebene Dictionary.

method clay search *path* ?path...?
Gibt den ersten übereinstimmenden Wert für den Pfad entweder in den Clay-Daten dieser Klasse oder in denen eines ihrer übergeordneten Elemente zurück

method clay set *path* ?path...? *value*
Fügt den Inhalt von **value** mit dem Speicherplatz des Objekts unter **path** zusammen.

Klasse clay::object

clay::object Diese Klasse wird von allen Klassen geerbt, die Optionen besitzen.

Methoden

method clay ancestors
Gibt die Klasse, zu der dieses Objekt gehört, alle in dieses Objekt gemischten Klassen sowie alle Vorfahren dieser Klassen in Suchreihenfolge zurück.

method clay cache *path value*
Speichert "VALUE" so, dass eine Abfrage nach "Pfad" in "SEARCH" diesen Wert stets gibt, bis der Cache geleert wird.

method clay cget *field*
Entnimmt einen Wert entweder aus der "clay"-Struktur des Objekts oder aus einer seiner Teilklassen, der mit dem Feldnamen übereinstimmt. Die Reihenfolge der Suche lautet wie

folgt:
1. Als Wert in der lokalen Dict-Variablen "config"
2. Als Wert in der lokalen Dict-Variablen "clay"
3. Als Blattknoten in einem beliebigen Vorfahren, der die Wurzel des "clay"-Baums bildet
4. Als Blattknoten in einem beliebigen Vorfahren mit dem Feldnamen **const** *field*
5. Als Blattknoten in einem beliebigen Vorfahren mit dem Feldnamen **option** *field* **default**

method clay delegate ?*stub*? ?*object*?
Introspektion oder Weiterleitung von Methoden. Ohne Argumente gibt die Methode eine Schlüssel-Wert-Liste aus Stubs und Objekten zurück. Mit nur dem Argument *stub* gibt die Methode das Objekt (falls vorhanden) zurück, das mit dem Stub verknüpft ist. Mit einem *stub* und einem *object* leitet dieser Befehl alle Aufrufe der Methode *stub* an das *object* weiter.

method clay dump
Gibt einen vollständigen Auszug der Clay-Daten dieses Objekts sowie die Daten aller rekursiv eingebundenen Klassen zurück.

method clay ensemble_map
Gibt ein Dictionary zurück, das die für dieses Objekt zusammenzustellenden Methodenensembles beschreibt

method clay eval *script*
Ein Skript im Namespace dieses Objekts wurde ausgewertet

method clay evolve
Löst die private Methode **InitializePublic** aus

method clay exists *path* ?path...?
Gibt den Wert 1 zurück, wenn *path* in den Clay-Daten des Objekts vorhanden ist. Werte größer als eins zeigen an, dass das Element in einer der Klassen vorhanden ist, aus denen das Objekt besteht. Der Wert Null bedeutet, dass der Pfad nicht gefunden werden konnte.

method clay flush
Löscht alle von der Clay-Implementierung erstellten Caches

method clay forward *method object*
Eine praktische Hülle für

```
oo::objdefine [self] forward {*}$args
```

method clay get *path* ?path...?
Ruft einen Datenblock aus dem Clay-System ab. Wenn das letzte Element von *path* ein Zweig (endet mit einem Schrägstrich /) ist, gibt es eine rekursive Zusammenführung aller Daten dieses Objekts und seiner Klassen mit den Daten in diesem Zweig zurück. Wenn das letzte Element ein Blatt ist, wird dieses Objekt oder es werden alle zugehörigen Klassen nach einem passenden Blatt durchsucht, und der erste gefundene Wert wird zurückgegeben. Wird kein Wert gefunden, gibt der Befehl eine leere Zeichenkette zurück.

method clay leaf *path* ?path...?
Ein modifizierter "get"-Befehl, der so angepasst ist, dass er ausschließlich Blattelemente abruft

method clay merge *dict* ?dict...?
Fügt die angegebenen Dictionarys rekursiv in den lokalen Clay-Speicher des Objekts ein.

method clay mixin *class* ?class...?

Führt [oo::objdefine [self] mixin] für dieses Objekt aus, wobei folgende zusätzliche Regeln gelten: Vor dem Aufruf wird für jede Klasse, die zuvor eingemischt wurde, aber nicht im neuen Ergebnis enthalten ist, das unter mixin/unmap-script registrierte Skript ausgeführt (sofern vorhanden). Für alle neuen Klassen, die vor diesem Aufruf noch nicht vorhanden waren, wird nach dem Aufruf des nativen TclOO-Mixins das unter mixin/map-script registrierte Skript ausgeführt (sofern vorhanden). Für alle Klassen, die nun vorhanden und eingemischt sind, wird das unter mixin/react-script registrierte Skript ausgeführt (sofern vorhanden).

method clay mixinmap ?*stub*? ?*classes*?
Wird kein Argument angegeben, gibt die Map der Stubs und Klassen zurück, die in das aktuelle Objekt eingemischt wurden. Wird nur ein Stub angegeben, gibt die Map der Klassen zurück, die in diesen Stub eingemischt wurden. Werden ein Stub und eine Klassenliste angegeben, werden die derzeit in diesem Stub enthaltenen Klassen durch die angegebenen Klassen ersetzt und das Verfahren "clay mixin" auf die neue Matrix der eingemischten Klassen angewendet.

method clay provenance *path* ?path...?
Gibt entweder **self** zurück, falls dieser Pfad im aktuellen Objekt vorhanden ist, oder die erste Klasse (sofern vorhanden) entlang des Clay-Suchpfads, die dieses Element enthält.

method clay replace *dictionary*
Ersetzt den Inhalt des internen Clay-Speichers durch das angegebene Dictionary.

method clay search *path valuevar isleafvar*
Gibt true zurück und setzt *valuevar* auf den Wert sowie *isleafvar* auf true oder false, je nachdem, ob *path* im Cache gefunden wurde.

method clay source *filename*
Wertet die angegebene Datei im Namespace des Objekts aus

method clay set *path* ?path...? *value*
Fügt den Inhalt von **value** mit dem "Clay"-Speicher des Objekts unter **path** zusammen.

method InitializePublic
Variablen instanziieren. Wird bei der Objekterstellung und während des "Clay"-Mixins aufgerufen.

Autoren

Sean Woods *mailto:*yoda@etoyoc.com

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *oo* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright

Copyright (c) 2018 Sean Woods <yoda@etoyoc.com>

clock

iso8601

Stichwörter: iso8601

Name

clock_iso8601 – Parsen von Datums- und Zeitangaben nach ISO 8601

Übersicht

package require Tcl 8.5 9
package require clock::iso8601 ?**0.2**?
::clock::iso8601 parse_date *date options...*
::clock::iso8601 parse_time *time options...*

Beschreibung

Dieses Paket bietet Funktionen zum Parsen von Datums- und Zeitangaben im ISO 8601-Format.

::clock::iso8601 parse_date *date options...*
: Dieser Befehl parst eine Datumszeichenkette nach ISO 8601 in einer unbekannten Variante und gibt das angegebene Datum bzw. die angegebene Uhrzeit in Sekunden seit der Epoche zurück.

  Zulässig sind die Optionen **-base**, **-gmt**, **-locale** und **-timezone** des eingebauten Befehls **clock scan**.

::clock::iso8601 parse_time *time options...*
: Dieser Befehl parst eine vollständige Zeitstempelzeichenkette nach ISO 8601 (Datum und Uhrzeit) in einer unbekannten Variante und gibt die angegebene Zeit in Sekunden seit der Epoche zurück.

  Zulässig sind die Optionen **-base**, **-gmt**, **-locale** und **-timezone** des eingebauten Befehls **clock scan**.

Datumsformate

Die Befehle akzeptieren die folgenden 23 Datumsformate:

```
(year)-(month)-(day)
(year)(month)(day)
(year)-(day in year)
(year)(day in year)
(year in century)-(month)-(day)
(year)-(month)          Day defaults to the 1st of the month
(year in century)(month)(day)
(year in century)-(day in year)
```

```
(year in century)(day in year)
--(month)-(day)         Year defaults to the current year
--(month)(day)          Year defaults to the current year
--(day in year)         Year defaults to the current year
---(day)                Year defaults to the current year, month to
    current month
(fiscal year)-W(week)-(wday)
(fiscal year)W(week)-(wday)
(fiscal year in century)-W(week)-(wday)
(fiscal year in century)W(week)(wday)
(fiscal year)-W(week)       Weekday defaults to monday
(fiscal year)W(week)        Weekday defaults to monday
-W(week)-(wday)         Year defaults to current fiscal year
-W(week)(wday)          Year defaults to current fiscal year
(wday)              Year defaults to current fiscal year, week to current week
(year)              Month defaults to january, day to 1st of the month
```

Die möglichen Bestandteile bzw. Felder in der obigen Darstellung und deren Bedeutungen sind:

year
: Jahr mit Jahrhundertangabe, 4 Ziffern

month
: Monat im Jahr, 2 Ziffern

day
: Tag des Monats, 2 Ziffern.

year in century
: Jahr ohne Jahrhundertangabe, 2 Ziffern

day in year
: Tag im Jahr, 3 Ziffern

fiscal year
: Wochenjahr nach ISO 8601, mit Jahrhundert, 4 Ziffern

fiscal year in century
: Wochenjahr nach ISO 8601, ohne Jahrhundert, 2 Ziffern

week
: Wochennummer nach ISO 8601

wday
: Wochentag, 1 Ziffer, Montag (1) bis Sonntag (7,0)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *clock::iso8601* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

rfc2822
Stichwörter: rfc2822

Name
clock_rfc2822 – Parsen von Datums- und Zeitangaben nach RFC 2822

Übersicht
package require Tcl 8.5 9
package require clock::rfc2822 ?**0.2**?
::clock::rfc2822 parse_date *date*

Beschreibung
Dieses Paket bietet Funktionen zum Parsen von Datumsangaben im RFC-2822-Format.

::clock::rfc2822 parse_date *date*
Dieser Befehl parst eine Datumszeichenkette nach RFC 2822 und gibt das angegebene Datum in Sekunden seit der Epoche zurück. Kann der Befehl das Datum nicht parsen, wird ein Fehler ausgelöst.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *clock::rfc2822* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

cmdline

cmdline
Stichwörter: cmdline, argument processing, argv, argv0, cmdline processing, command line processing

Name
cmdline – Prozeduren zur Verarbeitung von Befehlszeilen und Optionen.

Übersicht
package require Tcl 8.5 9
package require cmdline ?**1.5.3**?
::cmdline::getopt *argvVar optstring optVar valVar*

::cmdline::getKnownOpt *argvVar optstring optVar valVar*
::cmdline::getoptions *argvVar optlist* ?*usage*?
::cmdline::getKnownOptions *argvVar optlist* ?*usage*?
::cmdline::usage *optlist* ?*usage*?
::cmdline::getfiles *patterns quiet*
::cmdline::getArgv0

Beschreibung

Dieses Paket enthält Befehle zum Analysieren von Befehlszeilen und Optionen.

Behandlung von ::argv

Eine der am häufigsten verwendeten Variablen in diesem Paket ist **::argv**, die die Befehlszeile der aktuellen Anwendung enthält. Zu dieser Variablen gehört die Begleitvariable **::argc**, die zu Beginn der Anwendung auf die Anzahl der Elemente in **::argv** initialisiert wird.

Die Befehle in diesem Paket ändern den **::argc**-Begleiter *nicht*, wenn sie mit **::argv** aufgerufen werden. Es liegt in der Verantwortung des Aufrufers, den Wert konsistent zu halten, falls dies gewünscht oder erforderlich ist.

API

::cmdline::getopt *argvVar optstring optVar valVar*

Dieser Befehl funktioniert ähnlich wie die auf C basierende Standardfunktion **getopt**. Bei Angabe einer Optionszeichenfolge und eines Zeigers auf ein Array mit Argumenten verarbeitet dieser Befehl das *erste Argument* und gibt Informationen darüber zurück, wie weiter vorzugehen ist. Der Befehl gibt 1 zurück, wenn eine Option gefunden wurde, 0, wenn keine weiteren Optionen gefunden wurden, und -1, wenn ein Fehler aufgetreten ist.

argvVar enthält den Namen der Liste der zu verarbeitenden Argumente. Wenn Optionen gefunden werden, wird die Liste geändert und die verarbeiteten Argumente werden vom Anfang der Liste entfernt.

optstring enthält eine Liste der Befehlsoptionen, die die Anwendung akzeptiert. Wenn die Option auf ".arg" endet, verwendet der Befehl das nächste Argument als Argument für die Option oder extrahiert es aus dem aktuellen Argument, sofern es die Form "option=Wert" hat. Andernfalls ist die Option ein boolescher Wert, der bei Vorhandensein auf 1 gesetzt wird.

optVar bezieht sich auf die Variable, in der der Befehl die gefundene Option speichert (ohne das führende "-" und ohne die Erweiterung ".arg").

valVar bezieht sich auf die Variable, in der entweder der Wert für die angegebene Option bei Erfolg oder eine Fehlermeldung im Falle eines Fehlers gespeichert wird. Der gespeicherte Wert stammt bei .arg-Optionen aus der Befehlszeile, andernfalls ist der Wert 1.

::cmdline::getKnownOpt *argvVar optstring optVar valVar*

Ähnlich wie **::cmdline::getopt**, mit dem Unterschied, dass unbekannte Optionen in der Eingabe ignoriert werden.

::cmdline::getoptions *argvVar optlist* ?*usage*?

Verarbeitet den gesamten Satz an Befehlszeilenoptionen, die in der durch *argvVar* benannten

Listenvariablen enthalten sind, und setzt für nicht angegebene Optionen Standardwerte ein. Außerdem wird eine Fehlermeldung ausgegeben, in der die zulässigen Flags aufgelistet sind, falls ein falsches Flag angegeben wurde. Das optionale *usage*-argument enthält eine Zeichenkette, die vor der generierten Meldung eingefügt werden soll. Ist es nicht vorhanden, lautet der Standardwert "Optionen:".

argvVar enthält den Namen der Liste der zu verarbeitenden Argumente. Wenn Optionen gefunden werden, wird die Liste geändert und die verarbeiteten Argumente werden vom Anfang der Liste entfernt.

optlist enthält eine Liste von Listen, wobei jedes Element eine Option in der Form *flag default comment* legt.

Wenn *flag* auf ".arg" endet, wird der Wert aus der Befehlszeile übernommen. Andernfalls handelt es sich um einen booleschen Wert, der im Ergebnis erscheint, sofern er in der Befehlszeile vorhanden ist. Endet *flag* auf ".secret", wird es in der Verwendung nicht angezeigt.

Die Optionen **-?**, **-help** und **--** werden implizit berücksichtigt. Die ersten beiden brechen den Prozess der Optionenverarbeitung durch Auslösen eines Fehlers ab und erzwingen die Ausgabe der Hilfe, während die letzte die Optionenverarbeitung ohne Fehler abbricht und alle nachfolgenden Argumente der regulären Verarbeitung überlässt, selbst wenn sie mit einem Bindestrich beginnen.

Das Ergebnis des Befehls ist ein Dictionary, das alle Optionen ihren Werten zuordnet, unabhängig davon, ob es sich um benutzerdefinierte oder Standardwerte handelt.

::cmdline::getKnownOptions *argvVar optlist* ?*usage*?

Ähnlich wie **::cmdline::getoptions**, ignoriert jedoch alle unbekannten Optionen in der Eingabe.

::cmdline::usage *optlist* ?*usage*?

Erzeugt und gibt eine Fehlermeldung zurück, in der die zulässigen Flags aufgeführt sind. *optlist* ist wie bei **::cmdline::getoptions** definiert. Das optionale *usage*-argument enthält eine Zeichenkette, die vor der generierten Meldung eingefügt werden soll. Ist es nicht vorhanden, wird standardmäßig "Optionen:" verwendet.

::cmdline::getfiles *patterns quiet*

Anhand einer Liste von Datei-*Mustern* ermittelt dieser Befehl die Menge der gültigen Dateien. Unter Windows wird für jedes Argument eine Dateisuche durchgeführt. Unter Unix wird lediglich geprüft, ob die Datei existiert. Wenn ein Datei-Argument keine gültigen Dateien liefert, wird optional eine Warnung ausgegeben (set *quiet* auf "true" setzen).

Dieser Code verwendet außerdem den vollständigen Pfad für jede Datei. Wird dieser nicht angegeben, wird dem Dateinamen das aktuelle Verzeichnis vorangestellt. Dadurch wird sichergestellt, dass diese Dateien niemals mit Dateien in einer umschlossenen ZIP-Datei in Konflikt geraten. Der letzte Satz bezieht sich auf die Pro Tools.

::cmdline::getArgv0

Dieser Befehl gibt die "bereinigte" Version von *argv0* zurück. Dabei wird der führende Pfad

entfernt und die Erweiterung ".bin" gestrichen. Letzteres wird von den TclPro-Anwendungen verwendet, da diese von einem Shell-Skript umschlossen werden müssen.

Fehlercodes

Ab Version 1.5 verfügen alle vom Paket ausgelösten Fehler über einen korrekten **::errorCode**, der mit dem Tcl-Befehl **try** verwendet werden kann. Dieser Code enthält immer das Wort **CMDLINE** als erstes Element.

Beispiele

Cmdline::getoptions

Dieses Beispiel, das aus dem Paket **fileutil** stammt und leicht angepasst wurde, veranschaulicht die Verwendung von **cmdline::getoptions**. Zunächst wird eine Liste mit Optionen erstellt, anschließend wird die "args"-Liste zur Verarbeitung an **cmdline** übergeben. Danach werden verschiedene Optionen überprüft, um festzustellen, ob sie an das Skript übergeben wurden und welchen Wert sie haben.

```
        package require Tcl 8.5
        package require try         ;# Tcllib.
        package require cmdline 1.5 ;# First version with proper
    error-codes.

        # Notes:
        # - Tcl 8.6+ has 'try' as a builtin command and therefore does
    not
        #   need the 'try' package.
        # - Before Tcl 8.5 we cannot support 'try' and have to use
    'catch'.
        #   This then requires a dedicated test (if) on the contents of
        #   ::errorCode to separate the CMDLINE USAGE signal from
    actual errors.

        set options {
            {a          "set the atime only"}
            {m          "set the mtime only"}
            {c          "do not create non-existent files"}
            {r.arg  ""  "use time from ref_file"}
            {t.arg  -1  "use specified time"}
        }
        set usage ": MyCommandName \[options] filename ...\noptions:"

        try {
            array set params [::cmdline::getoptions argv $options
    $usage]

        # Note: argv is modified now. The recognized options are
        # removed from it, leaving the non-option arguments behind.
```

```
        } trap {CMDLINE USAGE} {msg o} {
            # Trap the usage signal, print the message, and exit the
    application.
            # Note: Other errors are not caught and passed through to
    higher levels!
        puts $msg
        exit 1
        }

        if {  $params(a) } { set set_atime "true" }
        set has_t [expr {$params(t) != -1}]
        set has_r [expr {[string length $params(r)] > 0}]
        if {$has_t && $has_r} {
            return -code error "Cannot specify both -r and -t"
        } elseif {$has_t} {
        ...
        }
```

Cmdline::getopt

Dieses Beispiel zeigt die Kernschleife von **cmdline::getoptions** aus dem vorherigen Beispiel. Es veranschaulicht, wie dabei **cmdline::get** verwendet wird, um die Optionen nacheinander zu verarbeiten.

```
    while {[set err [getopt argv $opts opt arg]]} {
    if {$err < 0} {
            set result(?) ""
            break
    }
    set result($opt) $arg
    }
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *cmdline* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

comm

comm

Stichwörter: comm, communication, ipc, message, remote communication, remote execution, rpc, secure, send, socket, ssl, tls

Name

comm – Eine Fernkommunikationsfunktion für Tcl (ab Version 8.5)

Übersicht

package require Tcl 8.5 9
package require comm ?**4.7.3**?
::comm::comm send ?*-async*? ?*-command callback*? *id cmd* ?*arg arg* ...?
::comm::comm self
::comm::comm interps
::comm::comm connect ?*id*?
::comm::comm new *chan* ?*name value* ...?
::comm::comm channels
::comm::comm config
::comm::comm config *name*
::comm::comm config ?*name value* ...?
::comm::comm shutdown *id*
::comm::comm abort
::comm::comm destroy
::comm::comm hook *event* ?**+**? ?*script*?
::comm::comm remoteid
::comm::comm_send
::comm::comm return_async
$future return ?*-code code*? ?*value*?
$future configure ?*-command cmdprefix*?
$future cget -command

Beschreibung

Der Befehl **comm** bietet eine Einrichtung zur entfernten Ausführung zwischen Interpretern, ähnlich wie **send(n)** von Tk, mit dem Unterschied, dass er für den Kommunikationspfad Sockets anstelle des X-Servers verwendet. Daher funktioniert **comm** mit mehreren Interpretern, läuft auf Windows- und Macintosh-Systemen und bietet Kontrolle über den Pfad der entfernten Ausführung.

Diese Befehle funktionieren genau wie **send** und **winfo interps**:

```
    ::comm::comm send ?-async? id cmd ?arg arg ...?
    ::comm::comm interps
```

Das ist alles, was Sie wirklich wissen müssen, um **comm** zu verwenden.

Befehle

Das Paket initialisiert **::comm::comm** als Standard-*chan*.

comm benennt Kommunikationsendpunkte mit einer *id*, die auf jedem Rechner eindeutig ist. Vor dem Senden von Befehlen wird die *id* eines anderen Interpreters benötigt. Im Gegensatz zu Tks

send kennt **comm** nicht implizit die *id*s aller Interpreter im System. Die folgenden vier Methoden bilden die grundlegende **comm**-Schnittstelle.

::comm::comm send ?-async? ?-command *callback*? *id cmd* ?*arg arg ...*?
Ruft den angegebenen Befehl im Interpreter mit dem Namen *id* auf. Der Befehl wartet auf das Ergebnis, und entfernte Fehler werden zurückgegeben, sofern nicht die Option **-async** oder **-command** angegeben ist. Ist **-async** angegeben, kehrt send sofort zurück, und es erfolgt keine weitere Benachrichtigung über das Ergebnis. Wird **-command** verwendet, legt *callback* einen Befehl fest, der beim Empfang des Ergebnisses aufgerufen wird. Diese Optionen schließen sich gegenseitig aus. Die Rückruffunktion erhält Argumente in der Form *-option value*, geeignet für **array set**. Die Optionen sind: *-id*, die Comm-ID des Interpreters, der den Befehl empfangen hat; *-serial*, eine eindeutige Seriennummer für jeden Befehl, der an einen bestimmten Comm-Interpreter gesendet wurde; *-chan*, der Name des Comm-Kanals; *-code*, der Ergebniscode des Befehls; *-errorcode*, der Fehlercode des Befehls, falls vorhanden; *-errorinfo*, die Fehlerinformation des Befehls, falls vorhanden; und *-result*, der Rückgabewert des Befehls. Geht die Verbindung verloren, bevor eine Antwort empfangen wird, wird die Rückruffunktion mit einer Meldung über den Verbindungsverlust aufgerufen, wobei -code gleich -1 ist. Bei Verwendung von **-command** gibt der Befehl die eindeutige Seriennummer des Befehls zurück.

::comm::comm self
Gibt die *id* für diesen Kanal zurück.

::comm::comm interps
Gibt eine Liste aller entfernten *id*s zurück, mit denen dieser Kanal verbunden ist. **comm** lernt eine neue entfernte *id* kennen, wenn zum ersten Mal ein Befehl an sie gesendet wird oder wenn eine entfernte *id* zum ersten Mal einen Befehl an diesen comm-Kanal sendet. **::comm::comm ids** ist ein Alias für diese Methode.

::comm::comm connect ?*id*?
Während **::comm::comm send** automatisch eine Verbindung zur angegebenen *id* herstellt, erzwingt dieser Befehl eine Verbindung zu einer entfernten *id*, ohne dass ein Befehl gesendet wird. Danach erscheint die entfernte *id* in **::comm::comm interps**.

Auswertungssemantik

Die Semantik der Auswertung von **::comm::comm send** ist so ausgelegt, dass sie *genau* mit der von **send** aus Tk übereinstimmt. Das bedeutet, dass **comm** die Argumente auf der entfernten Seite auswertet.

Sollten Sie feststellen, dass **::comm::comm send** bei einem bestimmten Befehl nicht funktioniert, versuchen Sie dasselbe mit send von Tk und prüfen Sie, ob das Ergebnis anders ausfällt. Sollte ein Problem vorliegen, melden Sie es bitte. So gab es beispielsweise einen Bericht, wonach dieser Befehl einen Fehler verursachte. Hinweis: Der entsprechende Befehl **send** verursacht ebenfalls denselben Fehler.

```
% ::comm::comm send id llength {a b c}
wrong # args: should be "llength list"
% send name llength {a b c}
wrong # args: should be "llength list"
```

Mit dem Hook **eval** (siehe unten) lässt sich von der doppelten Auswertungssemantik von **send**

auf eine einfache Auswertungssemantik umstellen.

Mehrere Kanäle

In jedem Tcl-Interpreter können mehrere **comm**-Kanäle (oder *Listener*) angelegt werden. Das gibt die Flexibilität, vollständige und eingeschränkte Kanäle anzulegen. So sind beispielsweise *Hook*-Skripte spezifisch für den Kanal, für den sie definiert wurden.

::comm::comm new *chan* ?*name value ...*?
: Erstellt einen neuen Kanal und einen Tcl-Befehl mit dem angegebenen Kanalnamen. Dieser neue Befehl steuert den neuen Kanal und akzeptiert dieselben Argumente wie **::comm::comm**. Alle verbleibenden Argumente werden an die Methode **config** übergeben. Der vollständig qualifizierte Name des Kanals wird zurückgegeben.

::comm::comm channels
: Listet alle in diesem Tcl-Interpreter angelegten Kanäle auf.

Die Standard-Konfigurationsparameter für einen neuen Kanal lauten:

```
    "-port 0 -local 1 -listen 0 -silent 0"
```

Der Standardkanal **::comm::comm** wird wie folgt angelegt:

```
    "::comm::comm new ::comm::comm -port 0 -local 1 -listen 1 -silent
    0"
```

Kanal-Konfiguration

Die Methode **config** verhält sich ähnlich wie **fconfigure**: Sie setzt oder erfragt Konfigurationsvariablen, die einem Kanal zugeordnet sind.

::comm::comm config
: siehe unten ...

::comm::comm config *name*
: siehe unten ...

::comm::comm config ?*name value...*?
: Ohne Argumente gibt **config** eine Liste aller Variablen und ihrer Werte zurück. Bei einem Argument gibt **config** den Wert eben dieses Arguments zurück. Bei einer geraden Anzahl von Argumenten werden die angegebenen Variablen auf die angegebenen Werte gesetzt.

Diese Konfigurationsvariablen können geändert werden (Beschreibungen dazu finden Sie an anderer Stelle in dieser Handbuchseite):

- **-listen** ?*0|1*?
- **-local** ?*0|1*?
- **-port** ?*port*?
- **-silent** ?*0|1*?
- **-socketcmd** ?*commandname*?
- **-interp** ?*interpreter*?
- **-events** ?*eventlist*?

Diese Konfigurationsvariablen sind schreibgeschützt:

- **-chan** *chan*
- **-serial** *n*

- **-socket** sock*ln*

Wenn **config** die Parameter eines bestehenden Kanals ändert (mit Ausnahme von **-interp** und **-events**), schließt es den lauschenden Socket und öffnet ihn neu. Eine automatisch zugewiesene Kanal-*id* ändert sich dabei. Das Erneuern des Sockets geschieht durch den Aufruf von **::comm::comm abort**, wodurch alle aktiven Sendevorgänge beendet werden.

ID-/Port-Zuordnungen

comm verwendet einen TCP-Port als Endpunkt-*id*. Die Methode **interps** (oder **ids**) listet lediglich alle TCP-Ports auf, mit denen der Kanal verbunden ist. Standardmäßig wird die *id* jedes Kanals vom Betriebssystem zufällig zugewiesen (beginnt jedoch in der Regel bei einem niedrigen Wert um 1024 und erhöht sich bei jeder Eröffnung eines neuen Sockets). Dieses Verhalten wird erreicht, indem der Konfigurationsoption **-port** der Wert 0 zugewiesen wird. Alternativ kann für einen bestimmten Kanal eine spezifische TCP-Portnummer angegeben werden. Als Sonderfall enthält comm Code, der mit **-port {}** einen hochnummerierten TCP-Port (>10000) belegt. Beachten Sie, dass ein Kanal nur dann erstellt und initialisiert wird, wenn der angegebene Port belegt werden kann.

Als Sonderfall gilt: Wenn der Kanal mit **-listen 0** konfiguriert ist, erstellt er keinen lauschenden Socket und verwendet für sich selbst die ID **0**. Ein solcher Kanal eignet sich nur für ausgehende Verbindungen (obwohl er, sobald eine Verbindung hergestellt ist, Sendeverkehr in beide Richtungen übertragen kann). Als weiterer Sonderfall gilt: Wenn der Kanal mit **-silent 0** konfiguriert ist, ignoriert die lauschende Seite Verbindungsversuche, bei denen die Protokollaushandlungsphase fehlgeschlagen ist, anstatt einen Fehler auszugeben.

Ausführungsumgebung

Ein Kommunikationskanal verwendet in seiner Standardkonfiguration den aktuellen Interpreter für die Ausführung aller empfangenen Skripte sowie der Skripte für Ereignisse, die den verschiedenen Hooks zugeordnet sind.

Diese unsichere Konfiguration kann vom Benutzer über die beiden Optionen **-interp** und **-events** geändert werden.

Wenn **-interp** gesetzt ist, werden alle empfangenen Skripte in dem als Wert der Option angegebenen Slave-Interpreter ausgeführt. Es wird vorausgesetzt, dass dieser Interpreter bereits vor der Konfiguration vorhanden ist. Das heißt, es liegt in der Verantwortung des Benutzers, diesen zu erstellen. Danach übernimmt jedoch der Kommunikationskanal den Besitz dieses Interpreters und zerstört ihn, wenn der Kommunikationskanal zerstört wird. Beachten Sie, dass eine Neukonfiguration des Kommunikationskanals auf einen anderen Interpreter oder auf die leere Zeichenkette den Besitz aufgibt, *ohne* den zuvor konfigurierten Interpreter zu zerstören. Die leere Zeichenkette hat eine besondere Bedeutung: Sie stellt das Standardverhalten wieder her, bei dem empfangene Skripte im aktuellen Interpreter ausgeführt werden.

Ebenfalls zu beachten ist, dass Antworten und Rückrufe (eine besondere Form der Antwort) *nicht* als empfangene Skripte gelten. Sie sind vertrauenswürdig, Teil der internen Mechanik von comm und werden daher stets im aktuellen Interpreter ausgeführt.

Selbst wenn ein Interpreter als Ausführungsumgebung für empfangene Skripte konfiguriert wurde, werden die mit den verschiedenen Hooks verbundenen Ereignisskripte standardmäßig weiterhin im aktuellen Interpreter ausgeführt. Um dies zu ändern, verwenden Sie die Option

-events, um eine Liste der Ereignisse anzugeben, deren Skripte ebenfalls im angegebenen Interpreter ausgeführt werden sollen. Der Inhalt dieser Option wird ignoriert, wenn der Kanal so konfiguriert ist, dass empfangene Skripte im aktuellen Interpreter ausgeführt werden.

Entfernte Interpreter

Standardmäßig ist jeder Kanal darauf beschränkt, nur Verbindungen vom lokalen System anzunehmen. Dies lässt sich mit der Konfigurationsoption **-local 0** aufheben. Bei solchen Kanälen hat der Parameter *id* die Form *{ id host }*.

WARNUNG: Der *host* muss stets in derselben Form angegeben werden (z. B. entweder als vollqualifizierter Domänenname, einfacher Hostname oder als IP-Adresse).

Verbindungen schließen

Diese Methoden geben Kontrolle über das Schließen von Verbindungen:

::comm::comm shutdown *id*
: Schließt die Verbindung zu *id* und bricht alle noch laufenden Befehle ab. Beachten Sie, dass nichts verhindert, dass die Verbindung durch einen anderen eingehenden oder ausgehenden Befehl sofort wieder geöffnet wird.

::comm::comm abort
: Ruft shutdown für alle offenen Verbindungen in diesem comm-Kanal auf.

::comm::comm destroy
: Bricht alle Verbindungen ab und zerstört dann diesen comm-Kanal selbst, einschließlich des Schließens des lauschenden Sockets. Spezieller Code erlaubt es, den Standardkanal **::comm::comm** so zu schließen, dass der Befehl **::comm::comm** nicht zerstört wird. Dabei wird der lauschende Socket geschlossen, wodurch sowohl eingehende als auch ausgehende Befehle auf dem Kanal verhindert werden. Diese Abfolge initialisiert den Standardkanal neu:

```
"::comm::comm destroy; ::comm::comm new ::comm::comm"
```

Wenn eine entfernte Verbindung verloren geht (weil die Gegenseite beendet wurde oder **shutdown** aufgerufen hat), kann **comm** eine Rückruffunktion der Anwendung aufrufen. Damit lässt sich beispielsweise ein Hilfsprozess aufräumen oder neu starten. Siehe die Rückruffunktion *lost* weiter unten.

Callbacks

Dies ist ein Mechanismus zum Setzen von Hooks für bestimmte Ereignisse:

::comm::comm hook *event* ?+? ?*script*?
: Verwendet eine Syntax ähnlich dem Tk-Befehl **bind**. Wird *script* ein **+** vorangestellt, wird das neue Skript angehängt. Ohne dieses ersetzt ein neues *script* jedes vorhandene Skript. Wird der Befehl ohne Skript aufgerufen, erfolgt keine Änderung. In allen Fällen gibt der Befehl das neue Hook-Skript zurück.

  Wenn ein *event* eintritt, wird das damit verknüpfte *script* ausgewertet, wobei die aufgeführten Variablen im Gültigkeitsbereich vorhanden und verfügbar sind. Der Rückgabecode (*nicht* der Rückgabewert) des Skripts wird üblicherweise verwendet, um zu entscheiden, wie nach dem Hook weiter verfahren werden soll.

  Zu den gängigen Variablen gehören:
  chan

der Name des comm-Kanals (und des Befehls)

id
die ID der betreffenden Gegenseite

fid
die Datei-ID für den Socket der Verbindung

Dies sind die definierten *events*:

connecting
Variablen: **chan**, **id**

Dieser Hook wird aufgerufen, bevor eine Verbindung zu der in *id* genannten Gegenseite hergestellt wird. Eine Fehlerrückgabe (über **error**) bricht den Verbindungsversuch mit dem Fehler ab. Beispiel:

```
% ::comm::comm hook connecting {
    if {[string match {*[02468]} $id]} {
        error "Can't connect to even ids"
    }
}
% ::comm::comm send 10000 puts ok
Connect to remote failed: Can't connect to even ids
%
```

connected
Variablen: **chan**, **fid**, **id**, **host** und **port**.

Dieser Hook wird unmittelbar nach dem Herstellen einer entfernten Verbindung zu *id* aufgerufen und ermöglicht eine beliebige Authentifizierung über den durch *fid* benannten Socket. Eine Fehlerrückgabe (über **error**) führt zum Schließen der Verbindung mit dem entsprechenden Fehler. *host* und *port* werden lediglich aus *id* extrahiert; eine Änderung dieser Werte hat jedoch keine Auswirkungen auf die Verbindung. Es ist zudem möglich, *fid* auszutauschen und zu ersetzen.

incoming
Variablen: **chan**, **fid**, **addr** und **remport**.

Dieser Hook wird beim Empfang einer eingehenden Verbindung aufgerufen und ermöglicht eine beliebige Authentifizierung über den mit *fid* benannten Socket. Eine Fehlerrückgabe (über **error**) führt zur Schließung der Verbindung mit dem entsprechenden Fehler. Beachten Sie, dass die Gegenstelle durch *remport* und *addr* benannt ist, die entfernte *id* aber noch unbekannt ist. Beispiel:

```
::comm::comm hook incoming {
    if {[string match 127.0.0.1 $addr]} {
        error "I don't talk to myself"
    }
}
```

eval

Variablen: **chan**, **id**, **cmd** und **buffer**.

Dieser Hook wird aufgerufen, nachdem ein vollständiges Skript von einer Gegenseite gesammelt wurde, jedoch *vor* seiner Auswertung. Dies ermöglicht die vollständige Kontrolle über die Verarbeitung eingehender Befehle. *cmd* enthält entweder **send** oder **async**. *buffer* enthält das auszuwertende Skript. Zum Zeitpunkt des Aufrufs des Hooks ist der Wert von *$chan remoteid* identisch mit dem Wert von *id*.

Durch Ändern von *buffer* kann der Hook das auszuwertende Skript ändern. Der Hook kann die Auswertung kurzschließen und bewirken, dass sofort ein Wert zurückgegeben wird, indem er **return** *result* (oder innerhalb einer Prozedur **return -code return** *result*) verwendet. Eine Fehlerrückgabe (über **error**) gibt ein Fehlerergebnis zurück, so als hätte das Skript den Fehler verursacht. Bei jeder anderen Rückgabe wird das Skript in *buffer* wie gewohnt ausgewertet. Aus Gründen der Kompatibilität mit Version 3.2 werden **break** und **return -code break** *result* unterstützt, die ähnlich wie **return {}** und **return -code return** *result* funktionieren.

Beispiele:
1. Erweiterung eines Befehls

```
% ::comm::comm send [::comm::comm self] pid
5013
% ::comm::comm hook eval {puts "going to execute $buffer"}
% ::comm::comm send [::comm::comm self] pid
going to execute pid
5013
```

2. Einen Befehl kurzschließen

```
% ::comm::comm hook eval {puts "would have executed $buffer";
return 0}
% ::comm::comm send [::comm::comm self] pid
would have executed pid
0
```

3. Ersetzen der doppelten Auswertungssemantik

```
% ::comm::comm send [::comm::comm self] llength {a b c}
wrong # args: should be "llength list"
% ::comm::comm hook eval {return [uplevel #0 $buffer]}
return [uplevel #0 $buffer]
% ::comm::comm send [::comm::comm self] llength {a b c}
3
```

4. Verwendung eines Slave-Interpreters

```
% interp create foo
% ::comm::comm hook eval {return [foo eval $buffer]}
% ::comm::comm send [::comm::comm self] set myvar 123
123
% set myvar
can't read "myvar": no such variable
% foo eval set myvar
123
```

5. Verwendung eines Slave-Interpreters (doppelte Auswertung)

```
% ::comm::comm hook eval {return [eval foo eval $buffer]}
```

6. Das auszuführende Skript unterlaufen

```
% ::comm::comm hook eval {
    switch -- $buffer {
        a {return A-OK}
        b {return B-OK}
        default {error "$buffer is a no-no"}
    }
}
% ::comm::comm send [::comm::comm self] pid
pid is a no-no
% ::comm::comm send [::comm::comm self] a
A-OK
```

reply

Variablen: **chan**, **id**, **buffer**, **ret** und **return()**.

Dieser Hook wird aufgerufen, nachdem ein vollständiges Antwortskript von einer Gegenseite gesammelt wurde, jedoch *vor* seiner Auswertung. Dies ermöglicht die vollständige Kontrolle über die Verarbeitung von Antworten auf gesendete Befehle. Der Antwort-*buffer* liegt in einer der folgenden Formen vor
- return result
- return -code code result
- return -code code -errorinfo info -errorcode ecode msg

Aus Sicherheitsgründen wird dies zerlegt. Das Rückgabeergebnis steht in *ret*, die Schalter von return stehen im Array return:
- *return(-code)*
- *return(-errorinfo)*
- *return(-errorcode)*

Jede dieser Variablen kann die leere Zeichenkette sein. Durch Ändern dieser vier Variablen lässt sich der Rückgabewert ändern, während das Ändern von *buffer* keine Auswirkungen hat.

callback

Variablen: **chan**, **id**, **buffer**, **ret** und **return()**.

Ähnlich wie *reply*, wird jedoch für Rückrufe verwendet.

lost

Variablen: **chan**, **id** und **reason**.

Dieser Hook wird aufgerufen, wenn die Verbindung zu **id** verloren geht. Der Rückgabewert (oder ein ausgelöster Fehler) wird ignoriert. *reason* ist eine erklärende Zeichenkette, die angibt, warum die Verbindung verloren ging. Beispiel:

```
    ::comm::comm hook lost {
        global myvar
        if {$myvar(id) == $id} {
            myfunc
            return
        }
    }
```

Nicht unterstützt

Diese Schnittstellen können sich in späteren Versionen ändern oder wegfallen.

::comm::comm remoteid

Gibt die *id* des Absenders des letzten auf diesem Kanal ausgeführten entfernten Befehls zurück. Wird diese Methode von einer Prozedur verwendet, die entfernt aufgerufen wird, muss sie aufgerufen werden, bevor Ereignisse verarbeitet werden. Andernfalls könnte ein anderer Befehl aufgerufen werden, der den Wert verändert.

::comm::comm_send

Durch den Aufruf dieser Prozedur werden die Tk-Befehle **send** und **winfo interps** durch diese Entsprechungen ersetzt, die **::comm::comm** verwenden.

```
    proc send {args} {
        eval ::comm::comm send $args
    }
    rename winfo tk_winfo
    proc winfo {cmd args} {
        if {![string match in* $cmd]} {
            return [eval [list tk_winfo $cmd] $args]
        }
        return [::comm::comm interps]
    }
```

Sicherheit

Ab Version 4.6 des Pakets wird die Option **-socketcmd** unterstützt, mit der der Benutzer eines comm-Kanals festlegen kann, welcher Befehl beim Öffnen eines Sockets verwendet werden soll. Es kann alles verwendet werden, was API-kompatibel mit dem eingebauten **::socket** (dem Standard) ist.

Die vorgesehene Hauptanwendung ist die Angabe des Befehls **tls::socket** (siehe Paket **tls**), um die Kommunikation abzusichern.

```
    # Load and initialize tls
    package require tls
    tls::init  -cafile /path/to/ca/cert -keyfile ...

    # Create secured comm channel
    ::comm::comm new SECURE -socketcmd tls::socket -listen 1
    ...
```

Die Abschnitte Ausführungsumgebung und Callbacks sind ebenfalls für die Sicherheit des Systems von Bedeutung, da sie Möglichkeiten bieten, die Ausführung auf eine bestimmte Umgebung zu beschränken, zusätzliche Authentifizierungen durchzuführen und Ähnliches.

Blockierungssemantik

Es gibt einen wesentlichen Unterschied zwischen **comm** und **send**. Beim Blockieren in einem synchronen entfernten Befehl verwendet **send** einen internen C-Hook (Tk_RestrictEvents) in die Ereignisschleife, um vorab nach send-bezogenen Ereignissen zu suchen und nur diese zu verarbeiten, ohne andere Ereignisse zu verarbeiten. Im Gegensatz dazu verwendet **comm** den Befehl **vwait** als Semaphor, um anzuzeigen, dass die Antwortnachricht eingetroffen ist. Der Unterschied besteht darin, dass ein synchroner **send** die Anwendung blockiert und die Verarbeitung aller Ereignisse (einschließlich fensterbezogener Ereignisse) verhindert, während ein synchroner **::comm::comm send** die Anwendung zwar blockiert, die Verarbeitung anderer Ereignisse jedoch weiterhin zulässt. Insbesondere werden **after idle**-Handler sofort ausgelöst, wenn comm blockiert.

Was lässt sich dagegen tun? Beachten Sie zunächst, dass dieses Verhalten bei jedem Code auftritt, der **vwait** verwendet, um zu blockieren und auf das Eintreten eines Ereignisses zu warten. Auf Kosten der Unterstützung mehrerer Kanäle könnte **comm** so geändert werden, dass es blockierende E/A am Socket durchführt, was eine send-ähnliche Blockierungssemantik ergibt. Die Unterstützung mehrerer Kanäle ist jedoch eine sehr nützliche Eigenschaft von comm, die als zu wichtig gilt, um darauf zu verzichten. Die verbleibenden Ansätze erfordern ein neues, in C geschriebenes ladbares Modul (was der Philosophie von **comm** etwas widerspricht). Eine Möglichkeit wäre die Erstellung einer modifizierten Version des Befehls **vwait**, die die Angabe der an Tcl_DoOneEvent übergebenen Ereignisflags ermöglicht. Bei **comm** würden lediglich die TCL_FILE_EVENTS verarbeitet werden. Ein weiterer Ansatz wäre die Implementierung eines Mechanismus ähnlich wie Tk_RestrictEvents, der jedoch auf die Tcl-Ereignisschleife angewendet wird (da **comm** kein Tk erfordert). Einer dieser Ansätze wird in einer zukünftigen Version von **comm** als optionale Komponente verfügbar sein.

Asynchrone Ergebniserzeugung

Standardmäßig ist das Ergebnis, das ein entfernt aufgerufener Befehl zurückgibt, das Ergebnis, das an den Aufrufer zurückgesendet wird. Das bedeutet, dass das Ergebnis synchron erzeugt wird und der Server, der den Aufruf bearbeitet, für die Dauer der Befehlsausführung blockiert ist.

Dies ist zwar vertretbar, solange auf dem Server nur kurz laufende Befehle ausgeführt werden; lang laufende Befehle, wie beispielsweise Datenbankabfragen, stellen jedoch ein Problem dar. Ein einzelner Befehl kann die Bearbeitung der Anfragen aller anderen Clients für einen beliebigen Zeitraum verhindern.

Vor Version 4.5 von comm bestand die einzige Lösung darin, den Serverbefehl so

umzuschreiben, dass er den eingebauten Tcl-Befehl **vwait** oder einen seiner Verwandten wie **tkwait** verwendet, um eine neue Ereignisschleife zu öffnen, die Anfragen verarbeitet, während der lang andauernde Vorgang ausgeführt wird. Dies birgt jedoch eigene Risiken, da dadurch sowohl ein Überlauf des Tcl-Stacks durch eine große Anzahl von Ereignisschleifen möglich ist als auch die Gefahr besteht, dass neuere Anfragen die Rückgabe älterer Anfragen blockieren, da die Ereignisschleifen in der Reihenfolge ihrer Erstellung abgebaut werden müssen.

Die richtige Lösung besteht darin, dass der aufgerufene Befehl **comm** mitteilt, dass er kein sofortiges, synchrones Ergebnis liefern kann oder will, dies jedoch zu einem späteren Zeitpunkt tun wird. Dann kann der Rahmen das Senden des eigentlichen Ergebnisses zurückstellen und die Verarbeitung der Anfragen in der Hauptereignisschleife fortsetzen. Keine Blockierung, keine Verschachtelung von Ereignisschleifen. Zu einem späteren Zeitpunkt liefert der lang andauernde Vorgang das Ergebnis über das Future-Objekt an comm, das es dann wie gewohnt an den Aufrufer weiterleitet.

Die für diese Lösung erforderliche Unterstützung wurde seit Version 4.5 in Form der neuen Methode **return_async** in comm integriert.

::comm::comm return_async

Dieser Befehl wird von einem entfernt aufgerufenen Skript verwendet, um dem comm-Kanal, der es aufgerufen hat, mitzuteilen, dass das an den Aufrufer zurückzusendende Ergebnis nicht synchron erzeugt wird. Wird dieser Befehl nicht aufgerufen, sendet comm standardmäßig das synchron erzeugte Ergebnis des Skripts selbst an den Aufrufer.

Das Ergebnis von **return_async** ist ein Objekt. An dieses Objekt, *future* genannt, muss das Ergebnis des Skripts übergeben werden, sobald es bereitsteht. Dann unternimmt es alles Nötige, um das Ergebnis an den Aufrufer des Skripts zu übermitteln, und zerstört sich anschließend selbst. Hat comm während der Berechnung des Ergebnisses die Verbindung zum Aufrufer verloren, versucht das Future nicht, das erhaltene Ergebnis zu übermitteln, sondern zerstört sich lediglich selbst. Das Future kann mit einem Befehl konfiguriert werden, der aufgerufen wird, wenn der Aufrufer verloren geht. Dies ermöglicht es dem Benutzer, einen vorzeitigen Abbruch des lang andauernden Vorgangs zu implementieren, sofern dieser dies unterstützt.

Ein Beispiel:

```
# Procedure invoked by remote clients to run database operations.
proc select {sql} {
    # Signal the async generation of the result

    set future [::comm::comm return_async]

    # Generate an async db operation and tell it where to deliver
    the result.

    set query [db query -command [list $future return] $sql]

    # Tell the database system which query to cancel if the
    connection
    # goes away while it is running.
```

```
    $future configure -command [list db cancel $query]

    # Note: The above will work without problem only if the async
    # query will nover run its completion callback immediately, but
    # only from the eventloop. Because otherwise the future we wish
    to
    # configure may already be gone. If that is possible use 'catch'
    # to prevent the error from propagating.
    return
}
```

Die API eines Future-Objekts lautet wie folgt:

$future return ?-code *code*? ?*value*?
: Mit dieser Methode teilen Sie dem Future mit, dass der lang andauernde Vorgang abgeschlossen ist. Als Argumente dienen ein optionaler Rückgabewert (Standardwert: die leere Zeichenkette) und der Tcl-Rückgabecode (Standardwert: OK).

  Das Future übermittelt diese Informationen an den Aufrufer, sofern die Verbindung in der Zwischenzeit nicht verloren ging, und zerstört sich anschließend selbst. Ist die Verbindung verloren gegangen, zerstört es sich lediglich selbst.

$future configure ?-command ?*cmdprefix*??
: siehe unten ...

$future cget -command
: Mit diesen Methoden kann der Benutzer einen Befehl abfragen und festlegen, der aufgerufen wird, wenn die Verbindung, zu der das Future gehört, verloren gegangen ist.

Kompatibilität

comm exportiert sich als Paket. Die Versionsnummer des Pakets hat das Format *Hauptversion. Nebenversion*, wobei sich die Hauptversion nur ändert, wenn eine nicht kompatible Änderung an der API oder am Protokoll vorgenommen wird. Kleinere Fehlerbehebungen und Änderungen wirken sich lediglich auf die Nebenversion aus. Um **comm** zu laden, wird in der Regel dieser Befehl verwendet:

```
    package require comm 3
```

Beachten Sie, dass auch ohne Versionsangabe (oder mit einer bestimmten Version) angefordert werden kann.

Die Versionshistorie von **comm** umfasst folgende Versionen:

4.6.3
: Ticket [ced0d60fc9] behoben. Korrekte Erkennung von EOF auf einem Socket hinzugefügt, der dann ordnungsgemäß geschlossen wird.

4.6.2
: Die Fehler 2972571 und 3066872 wurden behoben; im ersten Fall handelte es sich um eine fehlerhafte Erkennung einer maskierten geschweiften Klammer nach einem doppelten Backslash, im anderen Fall um ein blockierendes gets, das (im Nachhinein offensichtlich)

einen DoS-Angriff auf comm-Kanäle ermöglichte.

4.6.1

Die Implementierung von **comm::commCollect** wurde geändert, um das Verhalten von lindex vor Tcl 8 nachzubilden, d. h., es wurde die Möglichkeit geschaffen, das erste Wort einer Liste zu extrahieren, selbst wenn der gesamte Puffer keine wohlgeformte Liste ist. Ohne diese Änderung konnte das erste Wort nur extrahiert werden, wenn der gesamte Puffer eine wohlgeformte Liste war (seit Tcl 8), und in einer Situation mit sehr hoher Auslastung – d. h. wenn ein Server sehr schnell viele und/oder große Befehle sendet – kann dies unter Umständen nie geschehen, was schließlich zum Absturz des Empfängers führen kann, wenn ihm der Speicher ausgeht. Mit dieser Änderung ist der Empfänger stets in der Lage, das erste Wort zu verarbeiten, sobald es wohlgeformt ist, unabhängig von der Struktur des restlichen Puffers.

4.6

Es wurde die Option **-socketcmd** hinzugefügt, mit der Benutzer die Art und Weise, wie ein Socket geöffnet wird, überschreiben können. Der vorgesehene Hauptzweck besteht in der Angabe des Befehls **tls::socket** (siehe Paket **tls**) zur Sicherung der Kommunikation.

4.5.7

Die Behandlung bereits belegter Ports wurde geändert, um eine korrekte Fehlermeldung auszugeben.

4.5.6

Fehlerbehebung im Ersatz für **vwait**, robust gemacht gegen Variablennamen, die Leerzeichen enthalten.

4.5.5

Fehlerbehebung bei der Verarbeitung von Hooks, Tippfehler im Namen der Variablen.

4.5.4

Fehlerbehebung bei der Verarbeitung des von der Methode **send** empfangenen Ergebnisses. Ein *after idle unset result* wurde durch ein sofortiges **unset** ersetzt, wobei die Information in einer lokalen Variable gesichert wird.

Das **after idle** kann in einen durch fork erzeugten Kindprozess übergreifen, wenn zwischen seiner Einrichtung und dem fork keine Ereignisschleife liegt. Das kann den Kindprozess durcheinanderbringen, wenn die nächste Ereignisschleife das **vwait** im **send** von **comm** ist, einige Zeilen oberhalb des **after idle**, und der Kindprozess dieselbe Seriennummer für seine nächste Anfrage verwendet hat. In diesem Fall löscht das **after idle unset** des Elternprozesses genau das Array-Element, auf das der Kindprozess wartet, wodurch das **vwait** freigegeben wird und auf ein nun fehlendes Array-Element zugreift statt auf das erwartete Ergebnis.

4.5.3

Fehlerbehebungen in den Wrappern für die eingebauten Befehle **update** und **vwait**.

4.5.2

Fehlerbehebung im Wrapper für den eingebauten Befehl **update**.

4.5.1

Fehlerbehebungen bei der Verarbeitung von -interp für reguläre Skripte. Bei Skripten, die aus einer einzigen Anweisung in Form einer Liste bestehen, war die Verarbeitung des Puffers fehlerhaft. Ein fehlendes Argument beim neuen Befehl **commSendReply**, der mit Version 4.5

eingeführt wurde, wurde behoben. Dies hatte Auswirkungen auf die Fehlersuche.

4.5

Neue serverseitige Funktion. Der auf dem Server aufgerufene Befehl kann comm nun von der standardmäßigen synchronen Rückgabe seines Ergebnisses auf eine asynchrone (verzögerte) Rückgabe umstellen. Da snit zur Implementierung der von dieser Funktion verwendeten *future*-Objekte eingesetzt wird, benötigt comm ab dieser Version mindestens Tcl 8.3, um ausgeführt zu werden. Bitte lesen Sie den Abschnitt Asynchrone Ergebniserzeugung für weitere Details.

4.4.1

Fehlerbehebung bei der Ausführung von Hooks.

4.4

Fehlerbehebungen bei der Verarbeitung von -interp für reguläre Skripte und Hook-Skripte. Fehlerbehebungen bei der Bereinigung der Kanäle.

4.3.1

-interp und -events eingeführt, um die einfache Verwendung eines Slave-Interpreters zur Ausführung empfangener Skripte sowie von Skripten für Ereignisse zu ermöglichen.

4.3

Fehlerbehebungen sowie die Einführung von -silent, wodurch der Benutzer den Server bzw. die lauschende Seite dazu zwingen kann, Verbindungsversuche, bei denen die Protokollaushandlung fehlgeschlagen ist, stillschweigend zu ignorieren.

4.2

Fehlerbehebungen und, am wichtigsten, Umstellung auf UTF-8 als Standardkodierung für vollständige Internationalisierung (i18n) ohne Probleme.

4.1

Überarbeitung des internen Codes zur Entfernung des alten Pseudo-Objektmodells. Option send -command für eine asynchrone Rückruffunktion hinzugefügt.

4.0

Auf Wunsch von John LoVerso. Verbesserte Fehlerbehandlung bei asynchron aufgerufenen Befehlen.

3.7

In tcllib verschoben und in einen eigenen Namespace gelegt.

3.6

Ein Fehler beim Nachschlagen der remoteid für einen ausgeführten Befehl konnte auftreten, wenn die Verbindung geschlossen wurde, während mehrere asynchrone Sendevorgänge zur Ausführung in der Warteschlange standen.

3.5

Interne Änderung hinsichtlich der Verarbeitung von Antwortnachrichten aus einem **send**. Antwortnachrichten werden nun in den *value* dekodiert, der an **return** übergeben wird; anschließend wird mit diesem Wert eine neue return-Anweisung zusammengebaut. Bisher wurde der Rückgabecode von der Gegenseite als auszuwertender Befehl übergeben. Da sich das Wire-Protokoll nicht geändert hat, ist dies nach wie vor der Fall. Stattdessen dekodiert der Code zur Verarbeitung der Antwort die **reply**-Nachricht.

3.4

Es wurden weitere Kommentare im Quellcode hinzugefügt sowie die Konfigurationsvariablen in dieser Man-Seite dokumentiert. Ein Fehler wurde behoben, bei dem der Verlust der

Verbindung eine Fehlermeldung über eine Variable namens **pending** statt der Meldung über die verlorene Verbindung lieferte. **comm ids** ist nun ein Alias für **comm interps** (zuvor war es ein Alias für **comm chans**). Seit der Änderung des Methodenaufrufs in Version 3.0 wurden break und andere Ausnahmebedingungen von **comm send** nicht korrekt zurückgegeben. Dies wurde behoben, indem die zusätzliche Indirektionsebene in die interne Prozedur **commSend** entfernt wurde. Zudem wurde die Weitergabe der Variablen *errorCode* hinzugefügt. Dies bedeutet, dass diese Befehle nun genau so zurückgeben, wie sie es mit **send** tun würden:

```
comm send id break
catch {comm send id break}
comm send id expr 1 / 0
```

Es wurde ein neuer Hook für Antwortnachrichten hinzugefügt. Der Aufruf der Methode wurde überarbeitet, um die Verwendung von comm:*-Prozeduren zu vermeiden; dadurch konnte die Aufrufzeit um 40 % verkürzt werden. **comm config** wurde dokumentiert (da diese Handbuchseite noch das nicht mehr existierende **comm init** aufführte!)

3.3

Es wurden einige kleinere Fehler behoben und die Dokumentation überarbeitet. Es wurden einige Beispiele für Hooks hinzugefügt. Die Rückgabesemantik des Hooks **eval** wurde geändert.

3.2

Ein neues Wire-Protokoll, Version 3, wurde hinzugefügt. Es ist abwärtskompatibel mit Version 2, fügt aber einen Austausch der unterstützten Protokollversionen hinzu, um künftig eine Protokollaushandlung zu ermöglichen. Mehrere Fehler in der Hook-Implementierung wurden behoben. Ein neuer Abschnitt zur Blockierungssemantik wurde in die Handbuchseite aufgenommen.

3.1

Alle dokumentierten Hooks wurden implementiert. **commLostHook** wurde entfernt. Ein Fehler in **comm new** wurde behoben.

3.0

Dies ist eine neue Version von **comm** mit mehreren wesentlichen Änderungen. Es gibt eine neue Vorgehensweise zum Erstellen der Methoden, die unter dem Befehl **comm** verfügbar sind. Die Methode **comm init** wurde ausgemustert und durch **comm configure** ersetzt, das den Zugriff auf viele der klar definierten internen Variablen ermöglicht. Dadurch werden auch die für **comm new** verfügbaren Optionen verallgemeinert. Schließlich wird nun bei Herstellung einer Verbindung eine Protokollversion ausgetauscht. Dies ermöglicht zukünftige Änderungen am Wire-Protokoll. Derzeit ist die Protokollversion auf 2 festgelegt.

2.3

comm ids wurde in **comm channels** umbenannt. Die allgemeine Unterstützung für **comm hook** wurde vollständig implementiert, es existiert jedoch nur der Hook *lost*, der angepasst wurde, um der allgemeinen Hook-API zu entsprechen. **commLostHook** wird nicht mehr unterstützt (ersetzt durch **comm hook lost**) und **commLost** wurde entfernt.

2.2

Der Hook *died* wurde in *lost* umbenannt, um von **commLostHook** und einer frühen Implementierung von **comm lost hook** aufgerufen zu werden. Daher heißt **commDied** nun

commLost.

2.1
Die nicht unterstützte Methode **comm remoteid** wurde hinzugefügt.

2.0
comm wurde von Grund auf neu geschrieben (ist aber vollständig kompatibel mit Comm 1.0, ohne dass obTcl benötigt wird).

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Autoren

John LoVerso, John@LoVerso.Southborough.MA.US

http://www.opengroup.org/~loverso/tcl-tk/#comm

Lizenz

Bitte beachten Sie die Datei *comm.LICENSE*, die diesem Quellcode beiliegt, oder http://www.opengroup.org/www/dist_client/caubweb/COPYRIGHT.free.html.

Diese Lizenz für **comm**, die ab Version 3.2 gilt, ermöglicht die kostenlose Nutzung ohne Lizenzgebühren oder Tantiemen.

Bugs

- Schlägt die Initialisierung eines mit **::comm::comm new** erstellten Kanals fehl, sollte der Kanal zerstört werden. Derzeit verbleibt er in einem inkonsistenten Zustand.
- Es sollte eine Möglichkeit geben, einen Kanal bei einer Änderung der Konfiguration in den Ruhezustand zu versetzen.

Die folgenden Punkte lassen sich mit den vorhandenen Hooks umsetzen und werden hier als Erinnerung aufgeführt, um in einer zukünftigen Version einen Beispiel-Hook bereitzustellen.

- Einfachere Verwendung eines Slave-Interpreters für die eigentliche Ausführung der Befehle

(insbesondere im Modus "nicht lokal").
- Authentifizierung über eine Hostliste (wie xhost) oder ein "Magic Cookie" (wie xauth) im ersten Handshake.

Im Folgenden sind noch offene Aufgaben aufgeführt.

- Erkennung von Interpretern und Zuordnung von Namen zu Ports. Dies wird voraussichtlich in einem separaten, optionalen Nameserver liegen. (Siehe auch die unten aufgeführten verwandten Arbeiten.)
- Die Form *{id host}* so korrigieren, dass sie nicht von kanonischen Hostnamen abhängt. Dies erfordert Anpassungen an Tcl, um Hostnamen aufzulösen!

Diese Man-Seite ist umfangreicher als die Quelldatei.

Zur Verwendung älterer Versionen von Tcl

Tcl7.5 unter Windows weist einen Fehler auf, der dazu führt, dass der Interpreter hängen bleibt, wenn bei nicht-blockierenden Sockets das EOF erreicht wird. Dies lässt sich mit einem Befehl wie dem folgenden auslösen:

```
    "comm send $other exit"
```

Stellen Sie stets sicher, dass der Kanal inaktiv ist, bevor Sie ihn schließen oder beenden, oder verwenden Sie unter Windows mindestens Tcl7.6.

Tcl7.6 auf dem Mac weist mehrere Fehler auf. Es wird empfohlen, mindestens Tcl7.6p2 zu verwenden.

Tcl8.0 unter UNIX weist einen Socket-Fehler auf, der zum Absturz von Tcl führen kann. Es wird empfohlen, Tcl8.0p1 (oder Tcl7.6p2) zu verwenden.

Verwandte Arbeiten

Tcl-DP bietet eine RPC-basierte Schnittstelle zur entfernten Ausführung, ist jedoch eine kompilierte Tcl-Erweiterung. Siehe http://www.cs.cornell.edu/Info/Projects/zeno/Projects/Tcl-DP.html.

Michael Doyle miked@eolas.com hat Code, der die Tcl-DP-RPC-Schnittstelle unter Verwendung von Standard-Tcl-Sockets implementiert, ähnlich wie **comm**. Das DpTcl-Paket ist unter http://chiselapp.com/user/gwlester/repository/DpTcl verfügbar.

Andreas Kupries verwendet **comm** und hat im Rahmen seiner Pool-Bibliothek einen einfachen Nameserver entwickelt. Siehe http://www.purl.org/net/akupries/soft/pool/index.htm.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *comm* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright

```
Copyright (c) 1995-1998 The Open Group. All Rights Reserved.
Copyright (c) 2003-2004 ActiveState Corporation.
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

comm_wire

Stichwörter: comm_wire, comm, communication, ipc, message, remote communication, remote execution, rpc, socket

Name

comm_wire – Das Comm-Wire-Protokoll

Übersicht

package require comm

Beschreibung

Der Befehl **comm** bietet eine Einrichtung zur entfernten Ausführung zwischen Interpretern, ähnlich wie **send(n)** von Tk, mit dem Unterschied, dass er für den Kommunikationspfad Sockets anstelle des X-Servers verwendet. Daher funktioniert **comm** mit mehreren Interpretern, läuft auf Windows- und Macintosh-Systemen und bietet Kontrolle über den Pfad der entfernten Ausführung.

Dieses Dokument enthält eine Spezifikation der verschiedenen Versionen des Wire-Protokolls, das von comm intern für die Kommunikation zwischen seinen Endpunkten verwendet wird. Es ist für Anwender von **comm** nicht relevant, sondern richtet sich ausschließlich an Entwickler, die das Paket anpassen, eine kompatible Funktion in einer anderen Sprache schreiben oder eine andere Funktion auf Basis desselben Protokolls entwickeln möchten.

Wire-Protokoll Version 3

Grundschicht

Die grundlegende Kodierung für *alle* Daten ist UTF-8. Deshalb können Binärdaten, einschließlich des NULL-Zeichens, unverändert über die Leitung gesendet werden, ohne dass sie eigens verpackt (maskiert) werden müssen.

Grundlegende Nachrichtenebene

Auf der Grundschicht findet ein *nachrichtenorientierter* Datenaustausch statt. Die Gesamtheit aller in den Kanal geschriebenen Zeichen bildet eine Tcl-Liste, wobei jedes Element eine separate *Nachricht* darstellt, die wiederum selbst eine Liste ist. Die Nachrichten in der Gesamtliste werden durch EOL getrennt. Hinweis: EOL-Zeichen können auch innerhalb der Liste vorkommen. Sie lassen sich von den EOL-Zeichen, die Nachrichten voneinander trennen, dadurch unterscheiden, dass die Daten vom Anfang bis zu ihrer Position keine gültige Tcl-Liste bilden.

Das EOL wird durch das Zeilenvorschubzeichen signalisiert, d. h. **LF** oder, im Hexadezimalformat, **0x0a**. Dies entspricht der Unix-Konvention für Zeilenenden.

Als Liste besteht jede Nachricht aus *Wörtern*. Ihre Bedeutung hängt davon ab, wann die Nachricht im Rahmen des gesamten Austauschs gesendet wurde. Dies wird in den folgenden Abschnitten erläutert.

Verhandlungsnachrichten – erster Handshake

Das Befehlsprotokoll ist wie folgt definiert:

- Die erste Nachricht, die ein Client beim Aufbau der Verbindung an einen Server sendet, enthält zwei Wörter. Das erste Wort ist ebenfalls eine Liste und enthält die Versionen des Wire-Protokolls, die der Client akzeptieren kann, wobei die bevorzugte Version an erster Stelle steht. Das zweite Wort ist der TCP-Port, auf dem der Client auf Verbindungen zu sich selbst lauscht. Der Wert **0** signalisiert hier, dass der Client nicht auf Verbindungen lauscht, d. h., dass er ausschließlich zum Senden von Befehlen und nicht zum Empfangen von Befehlen dient.
- Die erste Nachricht, die der Server als Antwort auf die oben genannte Nachricht an den Client sendet, enthält nur ein Wort. Dieses Wort ist eine Liste, deren erstes Element die Zeichenkette **vers** ist und deren zweites Element die Version des Wire-Protokolls ist, die der Server aus den angebotenen Versionen ausgewählt hat.

Skript-/Befehlsnachrichten

Alle Nachrichten, die nach dem **ersten Handshake** folgen, bestehen aus drei Wörtern: einer Anweisung, einer Transaktions-ID und den Nutzdaten. Die gültigen Anweisungen sind nachstehend aufgeführt. Die Transaktions-IDs verwendet der Client, um eingehende Antworten den von ihm gesendeten Befehlsnachrichten zuzuordnen. Das bedeutet, dass ein Server die Transaktions-ID aus einer Befehlsnachricht in die Antwort kopieren muss, die er für diese Nachricht sendet.

send
: siehe unten ...

async
: siehe unten ...

command
: Die Nutzdaten sind das Tcl-Skript, das auf dem Server ausgeführt werden soll. Genauer ist es eine Liste mit den Skriptfragmenten. Der Server fügt diese Fragmente mit **concat** zusammen und bildet so das vollständige Skript, das serverseitig ausgeführt wird. Dies bildet die Semantik von Tcls "eval" nach. Meist ist es am besten, nur ein Wort in der Liste zu haben, nämlich eine Liste mit dem genauen Befehl.

  Beispiele:

```
    (a)     {send 1 {{array get tcl_platform}}}
    (b)     {send 1 {array get tcl_platform}}
    (c)     {send 1 {array {get tcl_platform}}}

    are all valid representations of the same command. They are
    generated via

    (a')    send {array get tcl_platform}
    (b')    send array get tcl_platform
    (c')    send array {get tcl_platform}

    respectively
```

Beachten Sie, dass (a), das durch (a') generiert wird, die übliche Form ist, wenn vom Client nur einzelne Befehle gesendet werden. Zum Beispiel mit **list** erstellt, wenn der Befehl variable Argumente enthält. Etwa so:

```
    send [list array get $the_variable]
```

Diese drei Anweisungen rufen das Skript jeweils serverseitig auf. Der Unterschied zwischen ihnen liegt in der Behandlung der Ergebniswerte und bestimmt somit, ob eine Antwort erwartet wird.

send
: Eine Antwort wird erwartet. Der Absender wartet auf das Ergebnis.

async
: Es wird keine Antwort erwartet; der Absender hat kein Interesse am Ergebnis und wartet auch nicht darauf.

command
: Eine Antwort wird erwartet, doch der Absender wartet nicht darauf. Er hat veranlasst, dass er eine prozessinterne Benachrichtigung erhält, wenn das Ergebnis vorliegt.

reply
: Wie die drei vorherigen Anweisungen, jedoch ist das Tcl-Skript in den Nutzdaten stark eingeschränkt. Es muss ein syntaktisch gültiger Tcl-Befehl **return** sein. Dieser enthält einen Ergebniscode, einen Wert, einen Fehlercode und Fehlerinformationen.

  Beispiele:

```
    {reply 1 {return -code 0 {}}}
    {reply 1 {return -code 0 {osVersion 2.4.21-99-default byteOrder
    littleEndian machine i686 platform unix os Linux user andreask
    wordSize 4}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *comm* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

Copyright
```
Copyright (c) 2005 Docs. Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

control

control
Stichwörter: control, assert, do, flow, no-op, structure

Name
Steuerung – Prozeduren für Kontrollflussstrukturen.

Übersicht
package require Tcl 8.5 9
package require control ?**0.1.4**?
control::control *command option* ?*arg arg* ...?
control::assert *expr* ?*arg arg* ...?
control::do *body* ?*option test*?
control::no-op ?*arg arg* ...?

Beschreibung
Das **control**-Paket bietet eine Vielzahl von Befehlen, die über die von Tcl bereitgestellten integrierten Strukturen hinaus zusätzliche Strukturen zur Steuerung des Programmablaufs bereitstellen. Dabei handelt es sich um Befehle, die in vielen Programmiersprachen als *Schlüsselwörter* oder als Teil der Sprache selbst betrachtet werden könnten. In Tcl sind Strukturen zur Steuerung des Programmablaufs jedoch genau wie alles andere auch einfach nur Befehle.

Befehle
control::control *command option ?arg arg ...?*
Der Befehl **control** dient als Konfigurationsbefehl zur Anpassung der anderen öffentlichen Befehle des control-Pakets. Das Argument *command* gibt den Befehl an, der angepasst werden soll. Die gültigen *Optionen* und nachfolgenden Argumente richten sich nach dem anzupassenden Befehl und sind in der Dokumentation zu diesem Befehl aufgeführt.

control::assert *expr ?arg arg ...?*
Wenn der Befehl **assert** deaktiviert ist, verhält er sich genau wie der Befehl **no-op**.

Wenn er aktiviert ist, führt der Befehl **assert** die Auswertung von *expr* als Ausdruck durch

(genauso wie **expr** sein Argument auswertet). Wenn sich bei der Auswertung herausstellt, dass *expr* kein gültiger boolescher Ausdruck ist (gemäß [**string is boolean -strict**]), wird ein Fehler ausgelöst. Wenn *expr* zu einem wahren booleschen Wert ausgewertet wird (wie von **if** erkannt), gibt **assert** eine leere Zeichenkette zurück. Andernfalls werden die verbleibenden Argumente von **assert** verwendet, um eine Meldungszeichenfolge zu bilden. Wenn keine Argumente vorhanden sind, lautet die Meldungszeichenfolge "assertion failed: $expr". Wenn Argumente vorhanden sind, werden diese mit **join** zu einer Meldungszeichenfolge zusammengefügt. Die Meldungszeichenfolge wird dann als Argument an einen Callback-Befehl angehängt, und der vervollständigte Callback-Befehl wird im globalen Namespace ausgewertet.

Der Befehl **assert** kann durch den Befehl **control** auf zwei Arten angepasst werden:

[**control::control assert enabled** ?*boolean*?] fragt ab oder legt fest, ob **control::assert** aktiviert ist. Wird der Befehl ohne ein *boolean*-Argument aufgerufen, gibt er einen booleschen Wert zurück, der angibt, ob der Befehl **control::assert** aktiviert ist. Wird der Befehl mit einem gültigen booleschen Wert als *boolean*-Argument aufgerufen, wird der Befehl **control::assert** entsprechend dem Argument aktiviert oder deaktiviert, und es wird eine leere Zeichenkette zurückgegeben.

[**control::control assert callback** ?*command*?] fragt die Rückruffunktion ab oder legt sie fest, die bei einem Assertion-Fehler von einem aktivierten **assert** aufgerufen wird. Wird die Funktion ohne ein *command*-Argument aufgerufen, gibt sie den aktuellen Befehl der Rückruffunktion zurück. Wird der Befehl mit einem *command*-Argument aufgerufen, wird dieses Argument zum neuen Callback-Befehl für den Fall eines Assertion-Fehlers. Hinweis: Ein Callback-Befehl für den Fall eines Assertion-Fehlers ist immer definiert, auch wenn **assert** deaktiviert ist. Der Standard-Callback-Befehl lautet [**return -code error**].

Hinweis: **control::assert** wurde so geschrieben, dass es in Kombination mit [**namespace import**] möglich ist, aktivierte **assert**-Befehle in einigen Namespaces und deaktivierte **assert**-Befehle in anderen Namespaces gleichzeitig zu verwenden. Diese Funktion ist nützlich, um die Fehlersuche modulweise unabhängig voneinander steuern zu können.

```
% package require control
% control::control assert enabled 1
% namespace eval one namespace import ::control::assert
% control::control assert enabled 0
% namespace eval two namespace import ::control::assert
% one::assert {1 == 0}
assertion failed: 1 == 0
% two::assert {1 == 0}
```

control::do *body* ?*option test*?

Der Befehl **do** wertet den Skriptkörper *body* wiederholt aus, *bis* der Ausdruck *test* wahr wird oder solange (*while*) *test* wahr ist, je nachdem, ob der Wert von *option**until** oder **while** lautet. Wenn *option* und *test* weggelassen werden, wird der Körper genau einmal ausgewertet. Nach normaler Ausführung gibt **do** eine leere Zeichenkette zurück. Ausnahmerückgabecodes (**break**, **continue**, **error** usw.) während der Auswertung von *body* werden genauso behandelt wie beim Befehl **while**, mit Ausnahme der unten unter

Einschränkungen genannten Fälle.

control::no-op ?*arg arg ...*?

Der Befehl **no-op** akzeptiert eine beliebige Anzahl von Argumenten und führt keine Aktion aus. Er gibt eine leere Zeichenkette zurück.

Einschränkungen

Einige der vom **control**-Paket bereitgestellten Befehle akzeptieren Argumente in Form von Skripten, die ausgewertet werden sollen. Aufgrund grundlegender Einschränkungen der Tcl-Befehle **catch** und **return** ist es diesen Befehlen nicht möglich, den Befehl [**return -code $code**] innerhalb eines dieser Skript-Argumente für einen beliebigen Wert von *$code* außer *ok* korrekt auszuwerten. In dieser Hinsicht sind die Befehle des **control**-Pakets im Vergleich zu den in Tcl integrierten Befehlen zur Steuerungsflusssteuerung (wie **if**, **while** usw.) sowie zu den Befehlen zur Steuerungsflusssteuerung, die von in C programmierten Paketen bereitgestellt werden können, eingeschränkt. Ein Beispiel für diesen Unterschied:

```
% package require control
% proc a {} {while 1 {return -code error a}}
% proc b {} {control::do {return -code error b} while 1}
% catch a
1
% catch b
0
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *control* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

coroutine

coro_auto

Stichwörter: coro_auto, after, channel, coroutine, events, exit, gets, global, green threads, read, threads, update, vwait

Name

coroutine::auto – Automatische Erkennung von Ereignissen und IO-Coroutinen

Übersicht

package require Tcl 8.6 9

package require coroutine::auto 1.3
package require coroutine 1.4

Beschreibung

Das Paket **coroutine::auto** bietet keine Befehle oder andere direkt sichtbare Funktionen. Es baut auf dem Paket **coroutine** auf und fängt verschiedene integrierte Befehle des Tcl-Kerns ab, um jeden Code, der diese verwendet, "coroutine-unabhängig" zu machen, d. h. so, dass er ohne Änderungen sowohl innerhalb als auch außerhalb einer Coroutine ausgeführt werden kann.

Die von diesem Paket betroffenen Befehle sind

- **after**
- **exit**
- **gets**
- **global**
- **puts**
- **read**
- **socket**
- **update**
- **vwait**

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *coroutine* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Coroutine

Copyright


```
Copyright (c) 2010-2014 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_coroutine

Stichwörter: tcllib_coroutine, after, channel, coroutine, events, exit, gets, global, green threads, read, threads, update, vwait

Name

Coroutine – Coroutinen-basierte Ereignis- und E/A-Verarbeitung

Übersicht

package require Tcl 8.6 9
package require coroutine 1.4

coroutine::util after *delay*
coroutine::util await *varname...*
coroutine::util create *arg...*
coroutine::util exit ?*status*?
coroutine::util gets *chan* ?*varname*?
coroutine::util gets_safety *chan limit varname*
coroutine::util global *varname...*
coroutine::util puts ?**-nonewline**? *channel string*
coroutine::util read -nonewline *chan* ?*n*?
coroutine::util socket ?*options...*? *host port*
coroutine::util update ?**idletasks**?
coroutine::util vwait *varname*

Beschreibung

Das **coroutine**-Paket bietet Coroutinen-fähige Implementierungen verschiedener ereignis- und kanalbezogener Befehle. Es kann in verschiedenen Modi ausgeführt werden:

1. Rufen Sie die Befehle über ihr Ensemble auf – in Code, der ausdrücklich für die Verwendung innerhalb von Coroutinen geschrieben wurde.
2. Importieren Sie die Befehle in einen Namespace, entweder direkt oder über **namespace path**. Dies ermöglicht die Verwendung innerhalb von Code, der an sich nicht coroutinenfähig ist und auf bestimmte Namespaces beschränkt ist.

Eine noch weitergehende Möglichkeit, Code "coroutinenunabhängig" zu gestalten als in Punkt 2 oben beschrieben, bietet das Paket coroutine::auto. Dieses fängt die entsprechenden integrierten Befehle ab und ändert deren Implementierung je nach Ausführungsumgebung, d. h. je nachdem, ob sie innerhalb oder außerhalb einer Coroutine ausgeführt werden.

API

Alle unten aufgeführten Befehle laufen synchron zur Coroutine ab, die sie aufruft, d. h., diese Coroutine wird so lange blockiert, bis das Ergebnis vorliegt. Die übergeordnete Ereignisschleife wird dadurch jedoch nicht blockiert.

coroutine::util after *delay*
: Dieser Befehl verzögert die Coroutine, die ihn aufruft, um *delay* Millisekunden.

coroutine::util await *varname...*
: Dieser Befehl ist eine Erweiterung des Befehls **coroutine::util vwait** (siehe unten), der auf einen Schreibvorgang in eine von mehreren benannten Namespace-Variablen wartet.

coroutine::util create *arg...*
: Dieser Befehl erstellt eine neue Coroutine mit einem automatisch zugewiesenen Namen und sorgt dafür, dass diese den durch die Argumente angegebenen Code ausführt.

coroutine::util exit ?*status*?
: Dieser Befehl beendet die aktuelle Coroutine, wodurch der Wert *status* zurückgegeben wird. Wurde kein Status festgelegt, gibt der Befehl den Standardwert *0* zurück.

coroutine::util gets *chan* ?*varname*?
: Dieser Befehl liest eine Zeile aus dem Kanal *chan* und gibt sie entweder als Ergebnis zurück oder schreibt sie – wenn ein *varname* festgelegt wurde – in die angegebene Variable und gibt die Anzahl der gelesenen Zeichen zurück.

coroutine::util gets_safety *chan limit varname*
Dieser Befehl liest eine Zeile aus dem Kanal *chan* bis zur Größe *limit* und speichert das Ergebnis in *varname*. Wird *limit* erreicht, bevor die festgelegte erste Zeilenumbruchstelle erreicht ist, wird ein Fehler ausgelöst. Der Befehl gibt die Anzahl der gelesenen Zeichen zurück.

coroutine::util global *varname...*
Dieser Befehl importiert die benannten globalen Variablen der Coroutine in den aktuellen Gültigkeitsbereich. Aus technischer Sicht befinden sich diese Variablen auf Ebene **#1** des Tcl-Stacks. Das heißt, es handelt sich nicht um reguläre globale Variablen im globalen Namespace, und jede Coroutine kann ihren eigenen Satz haben, unabhängig von allen anderen.

coroutine::util puts ?-nonewline? *channel string*
Dieser Befehl schreibt die Zeichenkette in den angegebenen *Kanal*. Im Gegensatz zur integrierten Funktion **puts** wartet dieser Befehl, bis der *Kanal* beschreibbar ist, bevor er tatsächlich darin schreibt.

coroutine::util read -nonewline *chan* ?*n*?
Dieser Befehl liest *n* Zeichen aus dem Kanal *chan* und gibt sie als Ergebnis zurück. Wenn *n* nicht festgelegt wird, liest der Befehl so lange aus dem Kanal, bis das EOF erreicht ist.

coroutine::util socket ?*options...*? *host port*
Dieser Befehl stellt eine Verbindung zum angegebenen Host und Port her und gibt zurück, sobald dies geschehen ist. Im Gegensatz zum integrierten Befehl führt er eine nicht-blockierende Verbindung im Hintergrund durch. Daher wird zwar die aufrufende Coroutine blockiert, die gesamte Anwendung wird jedoch nicht blockiert.

coroutine::util update ?idletasks?
Dieser Befehl bewirkt, dass die Coroutine, die ihn aufruft, ausstehende Ereignisse oder Leerlauf-Handler ausführt, bevor sie fortfährt.

coroutine::util vwait *varname*
Dieser Befehl bewirkt, dass die Coroutine, die ihn aufruft, auf einen Schreibvorgang in die benannte Namespace-Variable *varname* wartet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *coroutine* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Coroutine

Copyright

```
Copyright (c) 2010-2015 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

counter

counter

Stichwörter: counter, counting, histogram, statistics, tallying

Name

counter – Prozeduren für Zähler und Histogramme

Übersicht

package require Tcl 8.5 9
package require counter ?**2.0.6**?
::counter::init *tag args*
::counter::count *tag* ?*delta*? ?*instance*?
::counter::start *tag instance*
::counter::stop *tag instance*
::counter::get *tag args*
::counter::exists *tag*
::counter::names
::counter::histHtmlDisplay *tag args*
::counter::reset *tag args*

Beschreibung

Das Paket **counter** bietet eine Zählerfunktion und kann Statistiken und Histogramme für die erfassten Daten erstellen.

::counter::init *tag args*

Hiermit wird ein Zähler mit dem Namen *tag* definiert. Die *args* legen die Eigenschaften des Zählers fest. Die *args* sind

-group *name*

Hält einen gruppierten Zähler, bei dem der Name des Histogramm-Buckets an **::counter::count** übergeben wird.

-hist *bucketsize*

Sammelt den Zählerstand in Histogramm-Bucket mit der Größe *bucketsize*. Wenn es sich bei den Messwerten beispielsweise um Zeitwerte in Millisekunden handelt und *bucketsize* den Wert 10 hat, dann repräsentiert jedes Histogramm-Intervall Zeitwerte von 0 bis 10 ms, 10 bis 20 ms, 20 bis 30 ms und so weiter.

-hist2x *bucketsize*

Sammelt die Statistik in Histogramm-Bucket. Die Größe des ersten Buckets beträgt *bucketsize*, jeder weitere Bucket fasst Werte, deren Umfang doppelt so groß ist wie der des vorherigen Buckets. Wenn *bucketsize* beispielsweise 10 beträgt, so repräsentiert jedes Histogramm-Intervall Werte von 0 bis 10 ms, 10 bis 20 ms, 20 bis 40 ms, 40 bis 80 ms und so weiter.

-hist10x *bucketsize*

Sammelt die Statistik in Histogramm-Bucket. Die Größe des ersten Buckets beträgt *bucketsize*, jeder weitere Bucket umfasst Werte, die das Zehnfache der Größe des vorherigen Buckets betragen. Wenn beispielsweise *bucketsize* gleich 10 ist, repräsentiert

jedes Histogramm-Intervall Werte von 0 bis 10 ms, 10 bis 100 ms, 100 bis 1000 ms und so weiter.

-lastn *N*
Speichert die letzten *N* Werte des Zählers, um einen "gleitenden Durchschnitt" über die letzten *N* Werte zu ermitteln.

-timehist *secsPerMinute*
Hält ein zeitbasiertes Histogramm. Der Zählerstand wird entsprechend der aktuellen Uhrzeit in einen Histogramm-Bucket summiert. Es gibt 60 Minuten-Buckets, deren Größe durch *secsPerMinute* bestimmt wird; dieser Wert beträgt normalerweise 60, kann zu Tests jedoch auch kleiner sein. Jede "Stunde" (d. h. alle 60 "Minuten") wird der Inhalt der minutengenauen Buckets in den nächsten stündlichen Bucket summiert. Alle 24 "Stunden" wird der Inhalt der stündlichen Buckets in den nächsten täglichen Bucket summiert. Das "counter"-Paket hält alle zeitbasierten Histogramme synchron, sodass der erste vom Paket erfasste *secsPerMinute*-Wert für alle nachfolgenden zeitbasierten Histogramme verwendet wird.

::counter::count *tag* ?*delta*? ?*instance*?
Erhöht den durch *tag* identifizierten Zähler. Die Vorgabe ist 1; mit *delta* lässt sich um einen beliebigen ganzzahligen oder reellen Wert erhöhen. Jeder Zähler wird mit **::counter::init** deklariert, bevor er benutzt wird. Beim Typ **-group** wird der durch *instance* identifizierte Zähler erhöht.

::counter::start *tag instance*
Erfasst den Startzeitpunkt eines Intervalls. Der *tag* ist der Name des Zählers, der als wertbasiertes Histogramm "**-hist**" definiert ist. Die *instance* dient dazu, dieses Intervall von anderen Intervallen zu unterscheiden, die sich möglicherweise mit diesem überschneiden.

::counter::stop *tag instance*
Erfasst die Endzeit eines Intervalls. Die seit dem entsprechenden Aufruf von **::counter::start** für *instance* verstrichene Zeit wird in dem durch *tag* gekennzeichneten Histogramm erfasst.

::counter::get *tag args*
Gibt Statistiken zu einem durch *tag* identifizierten Zähler zurück. Die *args* legen fest, welcher Wert zurückgegeben werden soll:

-total
Gibt den Wert des Zählers zurück. Dies ist die Standardeinstellung, wenn *args* nicht angegeben wird.

-totalVar
Gibt den Namen der Variablen "total" zurück. Dies ist nützlich, um diese in Verbindung mit -textvariable in einem Tk-Widget festzulegen.

-N
Gibt die Anzahl der im Zähler gesammelten Messwerte zurück.

-avg
Gibt den Durchschnitt der im Zähler gesammelten Stichproben zurück.

-avgn
Gibt den Durchschnitt der letzten *N* erfassten Stichproben zurück. Der Wert *N* wird im Aufruf von **::counter::init** festgelegt.

-hist *bucket*

Wenn *bucket* festgelegt wird, gibt es den Wert in diesem Bucket des Histogramms zurück. Andernfalls gibt es das vollständige Histogramm im Array-Get-Format zurück, sortiert nach Buckets.

-histVar
Gibt den Namen der Histogramm-Array-Variablen zurück.

-histHour
Gibt das vollständige Stundenhistogramm im Array-Format zurück, sortiert nach Zeitfenstern.

-histHourVar
Gibt den Namen der Variablen für das Stundenhistogramm-Array zurück.

-histDay
Gibt das vollständige Tageshistogramm im Array-Format zurück, sortiert nach Buckets.

-histDayVar
Gibt den Namen der Variablen für das Tageshistogramm-Array zurück.

-resetDate
Gibt den Wert der Uhrsekunden zurück, der zum Zeitpunkt des letzten Zurücksetzens des Zählers aufgezeichnet wurde.

-all
Gibt ein Array zurück, das die Elemente des Arrays enthält, in dem der Zähler gespeichert ist. Dies umfasst die Gesamtsumme, die Anzahl der Stichproben (N) sowie etwaige typspezifische Informationen. Das Histogramm-Array ist darin nicht enthalten.

::counter::exists *tag*
Gibt den Wert 1 zurück, wenn der Zähler definiert ist.

::counter::names
Gibt eine Liste aller definierten Zähler zurück.

::counter::histHtmlDisplay *tag args*
Erstellt HTML-Code zur Darstellung eines Histogramms für einen Zähler. Die *args* steuern das Format der Darstellung. Es handelt sich dabei um folgende Parameter:

-title *string*
Beschriftung, die über dem Balkendiagramm angezeigt werden soll

-unit *unit*
Legt für die zeitbasierten Histogramme **minutes**, **hours** oder **days** fest. Bei wertbasierten Histogrammen wird die *unit* im Titel verwendet.

-images *url*
URL des Verzeichnisses "/images".

-gif *filename*
Bild für normale Histogrammbalken. Der *filename* ist relativ zum Verzeichnis "**-images**".

-ongif *filename*
Bild für den aktiven Histogrammbalken. Der *filename* ist relativ zum Verzeichnis "**-images** ".

-max *N*
Maximale Anzahl der anzuzeigenden wertbasierten Gruppen.

-height *N*

Pixelhöhe des höchsten Balkens.

-width *N*
Pixelbreite jedes Balkens.

-skip *N*
Zu überspringende Buckets bei der Beschriftung wertbasierter Histogramme.

-format *string*
Format, das zur Darstellung der Bezeichnungen von Buckets verwendet wird.

-text *boolean*
Wenn der Wert 1 ist, wird eine Textversion des Histogramms ausgegeben; andernfalls wird eine grafische Darstellung erstellt.

::counter::reset *tag args*
Setzt den Zähler mit dem Namen *tag* auf den Ausgangszustand zurück. Die *args* legen die neuen Eigenschaften des Zählers fest. Sie haben dieselbe Bedeutung wie bei **::counter::init** beschrieben.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *counter* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Datenstrukturen

crc

cksum
Stichwörter: cksum, checksum, crc, crc32, cyclic redundancy check, data integrity, security

Name
cksum – Berechnung einer mit cksum(1) kompatiblen Prüfsumme

Übersicht
package require Tcl 8.5 9
package require cksum ?**1.1.5**?
::crc::cksum ?*-format format*? ?*-chunksize size*? [*-channel chan* | *-filename file* | *string*]
::crc::CksumInit
::crc::CksumUpdate *token data*
::crc::CksumFinal *token*

Beschreibung
Dieses Paket enthält eine Tcl-Implementierung des cksum(1)-Algorithmus, die auf Informationen

basiert, die in der GNU-Implementierung dieses Programms als Teil des Pakets "GNU Textutils 2.0" bereitgestellt werden.

Befehle

::crc::cksum ?*-format format*? ?*-chunksize size*? [*-channel chan* | *-filename file* | *string*]

Der Befehl akzeptiert Zeichenfolgendaten, einen Kanal oder einen Dateinamen und gibt einen Prüfsummenwert zurück, der mithilfe des Algorithmus **cksum(1)** berechnet wurde. Das Ergebnis wird entweder gemäß dem angegebenen Bezeichner *format*(n) formatiert oder standardmäßig als vorzeichenlose Ganzzahl (%u) ausgegeben.

Optionen

-channel *name*

Gibt eine Prüfsumme für die aus einem Kanal gelesenen Daten zurück. Der Befehl liest so lange Daten aus dem Kanal, bis der Wert **eof** "wahr" ist. Wenn Sie während dieser Berechnung Ereignisse verarbeiten möchten, lesen Sie bitte den Abschnitt Programmierschnittstelle.

-filename *name*

Dies ist eine Komfortoption, die die angegebene Datei öffnet, die Kodierung auf "binär" setzt und anschließend so vorgeht, als wäre die Option *-channel* verwendet worden. Die Datei wird nach Abschluss des Vorgangs geschlossen.

-format *string*

Gibt die Prüfsumme unter Verwendung einer alternativen Formatvorlage zurück.

Programmierschnittstelle

Das Paket "cksum" implementiert die Prüfsumme mithilfe einer Kontextvariablen, der jederzeit zusätzliche Daten hinzugefügt werden können. Dies ist besonders nützlich in einer ereignisgesteuerten Umgebung, wie beispielsweise einer Tk-Anwendung oder einem Webserver-Paket. Die Daten, für die eine Prüfsumme berechnet werden soll, können während eines **fileevent**-Handlers schrittweise in einzelnen Blöcken verarbeitet werden. Dies kann die Interaktivität einer GUI-Anwendung verbessern und dazu beitragen, einen übermäßigen Speicherverbrauch zu vermeiden.

::crc::CksumInit

Eröffnet einen neuen cksum-Kontext. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss. Bei Bedarf kann ein optionaler Startwert festgelegt werden.

::crc::CksumUpdate *token data*

Fügt Daten zur durch das Token identifizierten Prüfsumme hinzu. Der Aufruf von *CksumUpdate $token* "*abcd*" entspricht dem Aufruf von *CksumUpdate $token* "*ab*", gefolgt von *CksumUpdate $token* "*cb*". Siehe Beispiele.

::crc::CksumFinal *token*

Gibt den Prüfsummenwert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, verliert das Token seine Gültigkeit. Das Ergebnis ist ein 32-Bit-Ganzzahlwert.

Beispiele

```
% crc::cksum "Hello, World!"
2609532967
```

```
% crc::cksum -format 0x%X "Hello, World!"
0x9B8A5027
```

```
% crc::cksum -file cksum.tcl
1828321145
```

```
% set tok [crc::CksumInit]
% crc::CksumUpdate $tok "Hello, "
% crc::CksumUpdate $tok "World!"
% crc::CksumFinal $tok
2609532967
```

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *crc* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright
```
Copyright (c) 2002, Pat Thoyts
```

crc16
Stichwörter: crc16, checksum, cksum, crc, crc32, cyclic redundancy check, data integrity, security

Name
crc16 – Durchführung einer 16-Bit-zyklischen Redundanzprüfung

Übersicht
package require Tcl 8.5 9
package require crc16 ?**1.1.5**?
::crc::crc16 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc16 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*

::crc::crc-ccitt ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc-ccitt ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::xmodem ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::xmodem ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::kermit ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::kermit ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::modbus ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::modbus ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::mcrf4xx ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::mcrf4xx ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::genibus ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::genibus ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::crc-x25 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc-x25 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::crc-sdlc ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc-sdlc ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::crc-usb ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::crc-usb ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::buypass ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::buypass ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::umts ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::umts ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::gsm ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::gsm ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::unknown2 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::unknown2 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::maxim ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::maxim ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::unknown3 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::unknown3 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::unknown4 ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::unknown4 ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*
::crc::cms ?*-format format*? ?*-seed value*? ?*-implementation procname*? **--** *message*
::crc::cms ?*-format format*? ?*-seed value*? ?*-implementation procname*? -filename *file*

Beschreibung

Dieses Paket bietet eine rein auf Tcl basierende Implementierung der CRC-Algorithmen, die auf den unter http://www.microconsultants.com/tips/crc/crc.txt bereitgestellten Informationen basiert. Es stehen zahlreiche Permutationen zur Berechnung von CRC-Prüfsummen zur Verfügung, und dieses Paket kann alle davon verarbeiten. Für die gängigsten Fälle sind Standardwerte voreingestellt.

Befehle

::crc::crc16 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::crc16 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename

file
siehe unten ...

::crc::crc-ccitt ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::crc-ccitt ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
siehe unten ...

::crc::xmodem ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::xmodem ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
siehe unten ...

::crc::kermit ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::kermit ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
siehe unten ...

::crc::modbus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::modbus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
siehe unten ...

::crc::mcrf4xx ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::mcrf4xx ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
siehe unten ...

::crc::genibus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::genibus ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
siehe unten ...

::crc::crc-x25 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::crc-x25 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
siehe unten ...

::crc::crc-sdlc ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::crc-sdlc ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::crc-usb ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::crc-usb ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::buypass ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::buypass ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::umts ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::umts ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::gsm ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::gsm ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::unknown2 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::unknown2 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::maxim ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::maxim ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::unknown3 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::unknown3 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*

siehe unten ...

::crc::unknown4 ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*

siehe unten ...

::crc::unknown4 ?-format *format*? ?-seed *value*? ?-implementation *procname*?

-filename *file*
siehe unten ...

::crc::cms ?-format *format*? ?-seed *value*? ?-implementation *procname*? -- *message*
siehe unten ...

::crc::cms ?-format *format*? ?-seed *value*? ?-implementation *procname*? -filename *file*
Der Befehl akzeptiert entweder Zeichenfolgendaten oder einen Dateinamen und gibt einen mithilfe des CRC-Algorithmus berechneten Prüfsummenwert zurück. Der verwendete Befehl legt das CRC-Polynom, den Startwert und die Bitreihenfolge für die gewünschte Standard-Prüfsummenberechnung fest. Das Ergebnis wird anhand des angegebenen Spezifikators *format*(n) formatiert oder standardmäßig als vorzeichenlose Ganzzahl (%u) ausgegeben.

Für den CRC-Algorithmus werden eine Reihe gängiger Polynome verwendet, von denen die am häufigsten verwendeten in diesem Paket enthalten sind. Der Einfachheit halber verfügt jedes dieser Polynome über einen Befehlsalias im crc-Namespace.

Es ist möglich, die CRC-32-Prüfsumme mithilfe dieses crc16-Pakets zu implementieren, da die Implementierung ausreichend generisch ist, um auf 32-Bit-Prüfsummen erweitert zu werden. Dies wurde beispielsweise bereits umgesetzt – allerdings ist dies nicht die schnellste Methode zur Implementierung dieses Algorithmus in Tcl, weshalb ein separates Paket namens **crc32** zur Verfügung steht.

Optionen

-filename *name*
Gibt eine Prüfsumme für den Dateiinhalt anstelle der Parameterdaten zurück.

-format *string*
Gibt die Prüfsumme unter Verwendung einer alternativen Formatvorlage zurück.

-seed *value*
Wählt einen alternativen Wert für die CRC-Berechnung aus. Der Standardwert ist 0 für die CRC16-Berechnung und 0xFFFF für die CCITT-Version. Dies kann nützlich sein, um den CRC für Datenstrukturen zu berechnen, ohne zuvor die gesamte Struktur in eine Zeichenkette umwandeln zu müssen. Der CRC des vorherigen Elements kann als Startwert für die Berechnung des CRC des nächsten Elements verwendet werden. Er dient außerdem dazu, eine Prüfsumme aus Fragmenten einer großen Nachricht (oder Datei) zu bilden.

-implementation *procname*
Dieser Hook wird bereitgestellt, damit Benutzer ihre eigene Implementierung bereitstellen können (beispielsweise eine in C kompilierte Erweiterung). Die angegebene Prozedur wird mit zwei Parametern aufgerufen. Der erste Parameter sind die Daten, für die eine Prüfsumme berechnet werden soll, und der zweite ist der Startwert. Als Ergebnis wird eine Ganzzahl erwartet.

Das Paket stellt einige Implementierungen von Standard-CRC-Polynomen für XMODEM, CCITT und die übliche CRC-16-Prüfsumme bereit. Der Einfachheit halber wurden zusätzliche Befehle bereitgestellt, die diese Implementierungen nutzen.

--

Beendet die Optionsverarbeitung. Beachten Sie, dass die Verwendung des Optionsbeendigungsflags bei der Verarbeitung von Daten aus Parametern wichtig ist. Wenn die Binärdaten einer der oben genannten Optionen ähneln, werden die Daten als Option interpretiert, sofern dieses Kennzeichen nicht enthalten ist. Verwenden Sie stets das Optionsbeendigungsflag -- vor der Angabe des Argumentes.

Beispiele

```
% crc::crc16 -- "Hello, World!"
64077
```

```
% crc::crc-ccitt -- "Hello, World!"
26586
```

```
% crc::crc16 -format 0x%X -- "Hello, World!"
0xFA4D
```

```
% crc::crc16 -file crc16.tcl
51675
```

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *crc* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright
```
Copyright (c) 2002, 2017, Pat Thoyts
```

crc32
Stichwörter: crc32, checksum, cksum, crc, cyclic redundancy check, data integrity, security

Name
crc32 – Durchführung einer 32-Bit-zyklischen Redundanzprüfung

Übersicht
package require Tcl 8.5 9
package require crc32 ?**1.3.4**?

::crc::crc32 ?*-format format*? ?*-seed value*? [*-channel chan* | *-filename file* | *message*]
::crc::Crc32Init ?*seed*?
::crc::Crc32Update *token data*
::crc::Crc32Final *token*

Beschreibung

Dieses Paket enthält eine Tcl-Implementierung des CRC-32-Algorithmus, die auf den unter http://www.naaccr.org/standard/crc32/ bereitgestellten Informationen basiertdocument.html. Wenn entweder das Paket **critcl** oder das Paket **Trf** verfügbar ist, kann intern eine kompilierte Version verwendet werden, um die Berechnung der Prüfsumme zu beschleunigen.

Befehle

::crc::crc32 ?-format *format*? ?-seed *value*? [*-channel chan* | *-filename file* | *message*]

Der Befehl akzeptiert entweder Zeichenfolgen, einen Kanal oder einen Dateinamen und gibt einen Prüfsummenwert zurück, der mithilfe des CRC-32-Algorithmus berechnet wurde. Das Ergebnis wird anhand des angegebenen Bezeichners *format*(n) formatiert. Standardmäßig wird der Wert als vorzeichenlose Ganzzahl zurückgegeben (Format %u).

Optionen

-channel *name*

Gibt eine Prüfsumme für die aus einem Kanal gelesenen Daten zurück. Der Befehl liest so lange Daten aus dem Kanal, bis der Wert **eof** "wahr" ist. Wenn Sie während dieser Berechnung Ereignisse verarbeiten möchten, lesen Sie bitte den Abschnitt Programmierschnittstelle.

-filename *name*

Dies ist eine Komfortoption, die die angegebene Datei öffnet, die Kodierung auf "binär" setzt und anschließend so vorgeht, als wäre die Option *-channel* verwendet worden. Die Datei wird nach Abschluss des Vorgangs geschlossen.

-format *string*

Gibt die Prüfsumme unter Verwendung einer alternativen Formatvorlage zurück.

-seed *value*

Wählt einen alternativen Wert für die CRC-Berechnung aus. Der Standardwert ist 0xffffffff. Dies kann nützlich sein, um den CRC für Datenstrukturen zu berechnen, ohne zuvor die gesamte Struktur in eine Zeichenkette umwandeln zu müssen. Der CRC des vorherigen Elements kann als Wert für die Berechnung des CRC des nächsten Elements verwendet werden. Hinweis: Der CRC32-Algorithmus beinhaltet einen abschließenden XOR-Schritt. Wenn eine inkrementelle Verarbeitung gewünscht ist, muss dieser Schritt rückgängig gemacht werden, bevor die Ausgabe des Algorithmus als Startwert für die weitere Verarbeitung verwendet wird. Eine einfachere Alternative ist die Verwendung der Programmierschnittstelle, die für diesen Betriebsmodus vorgesehen ist.

Programmierschnittstelle

Das CRC-32-Paket implementiert die Prüfsumme mithilfe einer Kontextvariablen, der jederzeit zusätzliche Daten hinzugefügt werden können. Dies ist besonders nützlich in einer ereignisgesteuerten Umgebung, wie beispielsweise einer Tk-Anwendung oder einem Webserver-Paket. Die Daten, für die eine Prüfsumme berechnet werden soll, können während

eines **fileevent**-Handlers schrittweise in einzelnen Blöcken verarbeitet werden. Dies kann die Interaktivität einer GUI-Anwendung verbessern und dazu beitragen, einen übermäßigen Speicherverbrauch zu vermeiden.

::crc::Crc32Init ?*seed*?
Eröffnet einen neuen CRC32-Kontext. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss. Bei Bedarf kann ein optionaler Startwert festgelegt werden.

::crc::Crc32Update *token data*
Fügt Daten zur durch das Token identifizierten Prüfsumme hinzu. Der Aufruf von *Crc32Update $token* "*abcd*" entspricht dem Aufruf von *Crc32Update $token* "*ab*", gefolgt von *Crc32Update $token* "*cb*". Siehe Beispiele.

::crc::Crc32Final *token*
Gibt den Prüfsummenwert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, verliert das Token seine Gültigkeit. Das Ergebnis ist ein 32-Bit-Ganzzahlwert.

Beispiele

```
% crc::crc32 "Hello, World!"
3964322768
```

```
% crc::crc32 -format 0x%X "Hello, World!"
0xEC4AC3D0
```

```
% crc::crc32 -file crc32.tcl
483919716
```

```
% set tok [crc::Crc32Init]
% crc::Crc32Update $tok "Hello, "
% crc::Crc32Update $tok "World!"
% crc::Crc32Final $tok
3964322768
```

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *crc* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2002, Pat Thoyts
```

sum

Stichwörter: sum, checksum, cksum, crc, crc32, cyclic redundancy check, data integrity, security

Name

sum – Berechnung einer "sum(1)"-kompatiblen Prüfsumme

Übersicht

package require Tcl 8.5 9
package require sum ?**1.1.3**?
::crc::sum ?*-bsd* | *-sysv*? ?*-format fmt*? ?*-chunksize size*? [*-filename file* | *-channel chan* | *string*]

Beschreibung

Dieses Paket enthält eine ausschließlich in Tcl geschriebene Implementierung des Befehls "sum(1)", der aus den Eingabedaten einen 16-Bit-Prüfsummenwert berechnet. Standardmäßig wird der BSD-Summenalgorithmus verwendet, es steht jedoch auch der SysV-Algorithmus zur Verfügung.

Befehle

::crc::sum ?*-bsd* | *-sysv*? ?*-format fmt*? ?*-chunksize size*? [*-filename file* | *-channel chan* | *string*]

Der Befehl akzeptiert Zeichenfolgen, einen Dateinamen oder einen Kanal als Eingabe und gibt einen Prüfsummenwert zurück, der mithilfe des Algorithmus **sum(1)** berechnet wurde. Das Ergebnis wird entweder gemäß dem angegebenen Bezeichner *format*(n) oder standardmäßig als vorzeichenlose Ganzzahl (%u) formatiert.

Optionen

-sysv

Der SysV-Algorithmus ist relativ einfach aufgebaut. Die Byte-Werte werden addiert, und ein eventueller Überlauf wird verworfen. Die niedrigsten 16 Bits werden als Prüfsumme zurückgegeben. Eingaben mit identischem Inhalt, aber unterschiedlicher Reihenfolge führen zum gleichen Ergebnis.

-bsd

Dieser Algorithmus ähnelt der SysV-Version, beinhaltet jedoch einen Schritt zur Bitrotation, wodurch eine Abhängigkeit von der Reihenfolge der Werte entsteht.

-filename *name*

Gibt eine Prüfsumme für den Dateiinhalt anstelle der Parameterdaten zurück.

-channel *chan*

Gibt eine Prüfsumme für den Inhalt des angegebenen Kanals zurück. Der Kanal muss zum Lesen geöffnet sein und sollte für die binäre Übersetzung konfiguriert sein. Der Kanal wird nach Abschluss des Vorgangs nicht geschlossen.

-chunksize *size*

Legt die Blockgröße fest, die beim Lesen von Daten aus Dateien oder Kanälen verwendet wird. Der Standardwert beträgt 4096.

-format *string*
Gibt die Prüfsumme unter Verwendung einer alternativen Formatvorlage zurück.

Beispiele

```
% crc::sum "Hello, World!"
37287
```

```
% crc::sum -format 0x%X "Hello, World!"
0x91A7
```

```
% crc::sum -file sum.tcl
13392
```

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *crc* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2002, Pat Thoyts <patthoyts@users.sourceforge.net>
```


cron

cron
Stichwörter: cron, odie

Name
cron – Tool zur Automatisierung der periodischen Ausführung von Befehlen

Übersicht
package require Tcl 8.6 9
package require cron ?**2.2**?
::cron::at ?*processname*? *timecode command*

::cron::cancel *processname*
::cron::every *processname frequency command*
::cron::in ?*processname*? *timecode command*
::cron::object_coroutine *object coroutine* ?*info*?
::cron::sleep *milliseconds*
::cron::task delete *process*
::cron::task exists *process*
::cron::task info *process*
::cron::task set *process field value* ?*field* ...? ?*value* ...?
::cron::wake ?*who*?
::cron::clock_step *milliseconds*
::cron::clock_delay *milliseconds*
::cron::clock_sleep *seconds* ?*offset*?
::cron::clock_set *newtime*

Beschreibung

Das Paket **cron** stellt eine Reihe von Pure-Tcl-Werkzeugen bereit, mit denen Programme Aufgaben so planen können, dass diese in regelmäßigen Abständen ausgeführt werden. Anstatt jede Aufgabe zu zwingen, einen eigenen Aufruf an die Ereignisschleife zu senden, ahmt das Cron-System das Cron-Dienstprogramm unter Unix nach: Eine Aufgabe prüft in regelmäßigen Abständen, ob etwas zu tun ist, und führt alle Befehle für einen bestimmten Zeitschritt auf einmal aus.

Änderungen in Version 2.0

Während Cron ursprünglich für Zeiträume von mehr als einer Sekunde konzipiert wurde, unterstützt die neueste Version nun eine Zeitauflösung bis auf die Millisekunde, was die Integration mit anderen zeitgesteuerten Ereignissen erleichtert. Version 2.0 ermöglicht zudem die korrekte Integration von Coroutinen und Objekten. Darüber hinaus wurde eine Funktion für eine externe (oder Skript-gesteuerte) Uhr hinzugefügt. Hinweis: Ereignisse im "vwait"-Stil mit einer externen Uhr funktionieren nicht besonders gut.

Befehle

::cron::at*?processname? timecode command*

Dieser Befehl registriert einen *command*, der zu dem durch *timecode* angegebenen Zeitpunkt aufgerufen werden soll. Wird *timecode* als Ganzzahl angegeben, wird davon ausgegangen, dass es sich um eine Unix-Zeit handelt. Alle anderen Eingaben werden von **clock scan** interpretiert und in Unix-Zeit umgewandelt. Diese Aufgabe kann durch nachfolgende Aufrufe der Paketbefehle geändert werden, indem auf *processname* verwiesen wird. Falls *processname* bereits existiert, wird er ersetzt. Wird *processname* nicht angegeben, wird ein Name generiert und vom Befehl zurückgegeben.

```
::cron::at start_coffee {Tomorrow at 9:00am}
    {remote::exec::coffeepot power on}
::cron::at shutdown_coffee {Tomorrow at 12:00pm}
    {remote::exec::coffeepot power off}
```

::cron::cancel *processname*

Dieser Befehl hebt die Registrierung des Prozesses *processname* auf und bricht alle

ausstehenden Befehle ab. Hinweis: "processname" kann ein Prozess sein, der entweder von **::cron::at** oder von **::cron::every** erstellt wurde.

```
::cron::cancel check_mail
```

::cron::every *processname frequency command*
Dieser Befehl registriert einen *command*, der im Intervall *frequency* aufgerufen werden soll. *frequency* wird in Sekunden angegeben. Diese Aufgabe kann durch nachfolgende Aufrufe der Paketbefehle geändert werden, indem auf *processname* verwiesen wird. Falls *processname* bereits existiert, wird er ersetzt.

```
::cron::every check_mail 900  ::imap_client::check_mail
::cron::every backup_db  3600 {::backup_procedure ::mydb}
```

::cron::in*?processname? timecode command*
Dieser Befehl registriert einen *command*, der nach einer durch *timecode* festgelegten Verzögerung aufgerufen werden soll. *timecode* wird als Ausdruck in Sekunden angegeben. Diese Aufgabe kann durch nachfolgende Aufrufe der Befehle dieses Pakets unter Bezugnahme auf *processname* geändert werden. Wenn *processname* bereits existiert, wird er ersetzt. Wird *processname* nicht angegeben, gibt der Befehl einen Namen zurück.

::cron::object_coroutine *object coroutine?info?*
Dieser Befehl registriert eine *coroutine*, die mit *object* verknüpft ist und unter Angabe der Parameter von *info* aufgerufen werden soll. Werden keine Parameter angegeben, wird davon ausgegangen, dass es sich bei der Coroutine um eine Leerlaufaufgabe handelt, die sich selbst beendet. *info* kann in jeder Form angegeben werden, die mit **::cron::task set** kompatibel ist.

::cron::sleep *milliseconds*
Wird dieser Befehl innerhalb einer Coroutine ausgeführt, registriert er die Coroutine für eine Callback-Funktion zum festgelegten Zeitpunkt und führt sofort ein "Yield" durch.

Wenn die Variable ::cron::time größer als 0 ist, erhöht dieser Befehl die interne Zeit schrittweise um jeweils 100 ms.

In allen anderen Fällen erzeugt dieser Befehl eine fiktive Variable, löst einen Nachruf aus und wartet mit "vwait" auf die Variable:

```
set eventid [incr ::cron::eventcount]
set var ::cron::event_#$eventid
set $var 0
::after $ms "set $var 1"
::vwait $var
::unset $var
```

Verwendung:

```
::cron::sleep 250
```

::cron::task delete *process*
Löscht den angegebenen Prozess *process*

::cron::task exists *process*

Gibt "true" zurück, wenn *process* bei cron registriert ist.

::cron::task info *process*
Gibt ein Dict zurück, das *process* beschreibt. Eine Beschreibung der Optionen finden Sie in **::cron::task set**.

::cron::task set *process field value?field...??value...?*
Wenn der *process* nicht existiert, wird er angelegt. Zu den Optionen gehören:

command
Wenn **coroutine** auf "black" gesetzt ist, handelt es sich um einen globalen Befehl, der diesen Prozess ausführt. Wenn **coroutine** nicht auf "black" gesetzt ist, ist dies der Befehl, der zum Erstellen oder erneuten Erstellen der Coroutine aufgerufen werden muss.

coroutine
Der Name der Coroutine (sofern vorhanden), die diesen Prozess implementiert.

frequency
Wenn der Wert -1 ist, wird dieser Prozess nach dem nächsten Ereignis beendet. Wenn der Wert 0 ist, soll dieser Prozess bei jedem Leerlaufereignis aufgerufen werden. Wenn der Wert positiv ist, soll dieser Prozess in regelmäßigen Abständen Ereignisse erzeugen. Die Frequenz ist eine ganze Zahl in Millisekunden, die den Abstand zwischen den Ereignissen angibt.

object
Das Objekt, das diesem Prozess oder dieser Coroutine zugeordnet ist.

scheduled
Wenn der Wert ungleich Null ist, ist dies die absolute Zeit (in Millisekunden) ab der Epoche, nach der dieser Prozess ein Ereignis auslöst. Wenn der Wert Null ist und der Wert von **frequency** ebenfalls Null ist, wird dieser Prozess bei jeder Leerlaufschleife aufgerufen.

running
Ein boolescher Flag. Ist er auf "true" gesetzt, bedeutet dies, dass der Prozess während der Ereignisschleife weder zurückgegeben hat noch die Kontrolle abgegeben hat und erst dann erneut aufgerufen wird, wenn dies geschieht.

::cron::wake*?who?*
Weckt den Cron-Prozess auf und sorgt dafür, dass dessen Ereignisschleife während des nächsten Leerlaufzyklus ausgeführt wird.

```
::cron::wake {I just did something important}
```

Es stehen mehrere Hilfsprogramme zur Verfügung, die intern in cron sowie zum Testen von cron verwendet werden, die jedoch im allgemeinen Fall nützlich sein können oder auch nicht.

::cron::clock_step *milliseconds*
Gibt eine absolute Uhrzeit relativ zur Epoche zurück, die auf den nächsten Rahmen zwischen einer Sekunde und der nächsten fällt, und zwar für den Wert von *milliseconds*

::cron::clock_delay *milliseconds*
Gibt eine absolute Uhrzeit relativ zur Epoche zurück, die auf den nächsten Rahmen zwischen einer Sekunde und den nächsten *milliseconds* in der Zukunft fällt.

::cron::clock_sleep *seconds?offset?*
Gibt eine absolute Uhrzeit relativ zur Epoche zurück, die genau *seconds* in der Zukunft liegt.

Wenn "offset" angegeben wird, kann dieser Wert positiv oder negativ sein und verschiebt die endgültige Zeit vor oder nach den Sekundenwechsel.

::cron::clock_set *newtime*
Stellt die interne Uhr für cron ein. Dieser Befehl verschiebt die Zeit in Schritten von 100 ms nach vorne und löst dabei Ereignisse aus, bis die interne Zeit den Wert *newtime* erreicht hat.

newtime wird in absoluten Millisekunden seit Beginn der Epoche als Ausdruck angegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *odie* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

System

Copyright


```
Copyright (c) 2016-2018 Sean Woods <yoda@etoyoc.com>
```


csv

csv

Stichwörter: csv, matrix, package, queue, tcllib

Name

csv – Prozeduren zur Verarbeitung von CSV-Daten.

Übersicht

package require Tcl 8.5 9
package require csv ?**0.8.1**?
::csv::iscomplete *data*
::csv::join *values* ?*sepChar*? ?*delChar*? ?*delMode*?
::csv::joinlist *values* ?*sepChar*? ?*delChar*? ?*delMode*?
::csv::joinmatrix *matrix* ?*sepChar*? ?*delChar*? ?*delMode*?
::csv::read2matrix ?**-alternate**? *chan m {sepChar ,} {expand none}*
::csv::read2queue ?**-alternate**? *chan q {sepChar ,}*
::csv::report *cmd matrix* ?*chan*?
::csv::split ?**-alternate**? *line* ?*sepChar*? ?*delChar*?
::csv::split2matrix ?**-alternate**? *m line {sepChar ,} {expand none}*
::csv::split2queue ?**-alternate**? *q line {sepChar ,}*
::csv::writematrix *m chan* ?*sepChar*? ?*delChar*?

::csv::writequeue *q chan* ?*sepChar*? ?*delChar*?

Beschreibung

Das **csv**-Paket bietet Befehle zur Bearbeitung von Daten im CSV-Format (CSV = Comma Separated Values).

Befehle

Folgende Befehle stehen zur Verfügung:

::csv::iscomplete *data*
Ein Prädikat, das prüft, ob das Argument *data* ein vollständiger CSV-Datensatz ist. Das Ergebnis ist ein boolescher Wert, der die Vollständigkeit der Daten angibt. Das Ergebnis ist "true", wenn die Daten vollständig sind.

::csv::join *values* ?*sepChar*? ?*delChar*? ?*delMode*?
Nimmt eine Liste von Werten entgegen und gibt eine Zeichenkette im CSV-Format zurück, die diese Werte enthält. Das Trennzeichen kann vom Aufrufer festgelegt werden, dies ist jedoch optional. Der Standardwert ist ",". Das Anführungszeichen bzw. das Begrenzungszeichen kann vom Aufrufer festgelegt werden, dies ist jedoch optional. Der Standardwert ist '"'. Standardmäßig ist der Anführungsmodus *delMode* auf "auto" gesetzt, sodass Werte nur bei Bedarf mit *delChar* umschlossen werden. Bei der Einstellung "always" werden die Werte stattdessen jedoch immer mit *delChar* umschlossen.

::csv::joinlist *values* ?*sepChar*? ?*delChar*? ?*delMode*?
Nimmt eine Liste von Listen mit Werten entgegen und gibt eine Zeichenkette im CSV-Format zurück, die diese Werte enthält. Das Trennzeichen kann vom Aufrufer festgelegt werden, dies ist jedoch optional. Der Standardwert ist ",". Das Anführungszeichen kann vom Aufrufer festgelegt werden, dies ist jedoch optional. Der Standardwert ist '"'. Standardmäßig ist der Anführungsmodus *delMode* auf "auto" gesetzt, sodass Werte nur bei Bedarf mit *delChar* umschlossen werden. Bei der Einstellung "always" werden die Werte stattdessen jedoch immer mit *delChar* umschlossen. Jedes Element der äußeren Liste wird als Datensatz betrachtet; diese werden im Ergebnis durch Zeilenumbrüche getrennt. Die Elemente jedes Datensatzes werden wie üblich formatiert (über **::csv::join**).

::csv::joinmatrix *matrix* ?*sepChar*? ?*delChar*? ?*delMode*?
Nimmt ein *Matrix*-Objekt entgegen, das der für das Paket struct::matrix festgelegten API entspricht, und gibt eine Zeichenkette im CSV-Format zurück, die diese Werte enthält. Das Trennzeichen kann vom Aufrufer festgelegt werden, dies ist jedoch optional. Der Standardwert ist ",". Das Anführungszeichen kann vom Aufrufer festgelegt werden, dies ist jedoch optional. Der Standardwert ist \'"'. Standardmäßig ist der Anführungsmodus *delMode* auf "auto" gesetzt, sodass Werte nur bei Bedarf mit *delChar* umschlossen werden. Wenn die Einstellung "always" gewählt ist, werden die Werte jedoch stattdessen immer von *delChar* umschlossen. Jede Zeile der Matrix wird als Datensatz betrachtet; diese sind im Ergebnis durch Zeilenumbrüche voneinander getrennt. Die Elemente jedes Datensatzes werden wie gewohnt formatiert (über **::csv::join**).

::csv::read2matrix ?-alternate? *chan m* {*sepChar*,} {*expand* none}
Ein Wrapper um **::csv::split2matrix** (siehe unten), der Zeilen im CSV-Format aus dem angegebenen Kanal liest (bis zum EOF) und sie der angegebenen Matrix hinzufügt. Eine Erläuterung des Arguments *expand* finden Sie unter **::csv::split2matrix**.

::csv::read2queue ?-alternate? *chan q* {*sepChar*,}

Ein Wrapper um **::csv::split2queue** (siehe unten), der Zeilen im CSV-Format aus dem angegebenen Kanal liest (bis zum EOF) und sie der angegebenen Warteschlange hinzufügt.

::csv::report *cmd matrix* ?*chan*?

Ein Befehl, der von den Matrixmethoden **format 2string** und **format 2chan** verwendet werden kann. Bei letzterer leitet dieser Befehl die Aufgabe an **::csv::writematrix** weiter. *cmd* muss entweder **printmatrix** oder **printmatrix2channel** lauten. Das Kanal-Argument *chan* muss bei letzterem vorhanden sein, bei ersterem darf es nicht vorhanden sein.

::csv::split ?-alternate? *line* ?*sepChar*? ?*delChar*?

Wandelt eine *Zeile* im CSV-Format in eine Liste der in dieser Zeile enthaltenen Werte um. Das Zeichen, mit dem die Werte voneinander getrennt werden, kann vom Aufrufer über *sepChar* festgelegt werden; dies ist jedoch optional. Der Standardwert ist ",". Das Anführungszeichen kann vom Aufrufer festgelegt werden; dies ist jedoch optional. Der Standardwert ist '"'.

Wenn die Option **-alternate** festgelegt ist, wird zum Parsen der Eingabe eine leicht abweichende Syntax verwendet. Diese Syntax wird weiter unten im Abschnitt Format erläutert.

::csv::split2matrix ?-alternate? *m line* {*sepChar*,} {*expand* none}

Entspricht **::csv::split**, fügt jedoch die resultierende Liste mithilfe der Methode **add row** als neue Zeile an die Matrix *m* an. Der über *expand* angegebene Erweiterungsmodus legt fest, wie der Befehl mit einer Matrix umgeht, die weniger Spalten hat als in *line* enthalten sind. Die zulässigen Modi sind:

none

Dies ist der Standardmodus. In diesem Modus muss der Aufrufer sicherstellen, dass die Matrix über genügend Spalten verfügt, um die gesamte Zeile aufzunehmen. Sind nicht genügend Spalten vorhanden, wird die Werteliste am Ende ohne Fehlermeldung gekürzt, damit sie in die Matrix passt.

empty

In diesem Modus erweitert der Befehl eine leere Matrix so, dass sie alle Spalten der angegebenen Zeile enthält, geht jedoch nicht darüber hinaus. Insgesamt hat dies zur Folge, dass die erste einer Reihe von Zeilen die Anzahl der Spalten in der Matrix bestimmt und alle folgenden Zeilen auf diese Größe gekürzt werden, als wäre der Modus **none** eingestellt.

auto

In diesem Modus erweitert der Befehl die Matrix nach Bedarf, um alle in *line* enthaltenen Spalten aufzunehmen. Das hat insgesamt zur Folge, dass die Matrix nach dem Hinzufügen einer Reihe von Zeilen über genügend Spalten verfügt, um alle Spalten der bisher längsten Zeile aufzunehmen.

::csv::split2queue ?-alternate? *q line* {*sepChar*,}

Genau wie bei **::csv::split**, jedoch wird die resultierende Liste mithilfe der Methode **put** als einzelnes Element an die Warteschlange *q* angehängt.

::csv::writematrix *m chan* ?*sepChar*? ?*delChar*?

Ein Wrapper um **::csv::join**, der alle Zeilen der Matrix *m* übernimmt und sie im CSV-Format in den Kanal *chan* schreibt.

::csv::writequeue *q chan* ?*sepChar*? ?*delChar*?

Ein Wrapper um **::csv::join**, der alle Elemente der Warteschlange *q* (vorausgesetzt, es handelt sich um Listen) übernimmt und sie im CSV-Format in den Kanal *chan* schreibt.

Format

Das Format regulärer CSV-Dateien wird wie folgt festgelegt:

1. Jeder Datensatz einer CSV-Datei (Comma-Separated Values, wie sie z. B. von Excel exportiert werden) besteht aus einer Reihe von ASCII-Werten, die durch "," getrennt sind. In anderen Sprachen kann das Trennzeichen jedoch auch ";" sein; dies spielt in diesem Fall jedoch keine Rolle, da die hier bereitgestellten Funktionen jedes beliebige Trennzeichen zulassen.
2. Genau dann, wenn ein Wert das Trennzeichen "," enthält, muss er (der Wert) in Anführungszeichen "" gesetzt werden. Enthält der Wert das Trennzeichen nicht, ist die Verwendung von Anführungszeichen optional.
3. Wenn ein Wert das Zeichen ", enthält, wird dieses Zeichen durch "" dargestellt.
4. Die Ausgabekette "" steht für den Wert ". Mit anderen Worten: Es wird davon ausgegangen, dass sie ausschließlich durch Regel 3 erzeugt wurde, d. h., dass der Wert nicht in Anführungszeichen gesetzt wurde.

Eine alternative Formatdefinition, die vor allem von MS-Produkten verwendet wird, legt fest, dass die Ausgabekette "" eine Darstellung der leeren Zeichenkette ist. Mit anderen Worten: Es wird davon ausgegangen, dass die Ausgabe durch Einfügen von Anführungszeichen in die leere Zeichenkette (d. h. Regel 2) und nicht durch Regel 3 erzeugt wurde. Dies ist der einzige Unterschied zwischen dem regulären und dem alternativen Format.

Das alternative Format wird durch die Angabe der Option **-alternate** bei den verschiedenen Split-Befehlen aktiviert.

Beispiel

Bei Verwendung des regulären Formats wird der Datensatz

```
123,"123,521.2","Mary says ""Hello, I am Mary""",""
```

wird in die einzelnen Elemente zerlegt

```
a) 123
b) 123,521.2
c) Mary says "Hello, I am Mary"
d) "
```

Bei Verwendung des alternativen Formats lautet das Ergebnis

```
a) 123
b) 123,521.2
c) Mary says "Hello, I am Mary"
d) (the empty string)
```

stattdessen. Wie Sie sehen, unterscheidet sich nur Punkt (d): Anstelle eines "." steht nun die leere Zeichenfolge.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden

oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *csv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

Copyright

```
Copyright (c) 2002-2015 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


debug

debug
Stichwörter: debug, log, narrative, trace

Name
debug – Debug-Erzählung – Kern

Übersicht
package require Tcl 8.5 9
package require debug ?**1.0.7**?
debug.tag *message* ?*level*?
debug 2array
debug define *tag*
debug header *text*
debug level *tag* ?*level*? ?*fd*?
debug names
debug off *tag*
debug on *tag*
debug parray *arrayvarname*
debug pdict *dict*
debug hexl *data* ?*prefix*?
debug nl
debug tab
debug prefix *tag* ?*text*?
debug setting (*tag level*) ... ?*fd*?
debug suffix *tag* ?*text*?
debug trailer *text*

Beschreibung
Die für die Fehlersuche relevanten Bereiche werden durch "Tags" dargestellt, für die unabhängig voneinander Relevanzstufen festgelegt werden können (eine ganze Zahl; je höher, desto

detaillierter).

API

debug.tag *message* ?*level*?

Für jedes bekannte Tag erstellt das Paket einen Befehl mit dieser Signatur, den der Benutzer anschließend verwenden kann, um die Debug-Beschreibung des Tags bereitzustellen. Die Beschreibung *message* wird als Tcl-Skript bereitgestellt, dessen Wert im Geltungsbereich des Aufrufers nur dann per **subst** eingesetzt wird, wenn die aktuelle Detailstufe für das *tag* der Detailstufe *level* des Aufrufs entspricht oder diese übersteigt. Dies ist nützlich, da man beliebig komplexe Beschreibungen im Code platzieren kann, ohne diese unnötigerweise auswerten zu müssen.

Siehe die Methoden **level** und **setting** zum Abfragen und Bearbeiten der aktuellen Detailebene für Tags.

Der tatsächlich ausgegebene Text besteht nicht nur aus der *message*, sondern auch aus globalen und tag-spezifischen Präfixen und Suffixen, sofern vorhanden, wobei jede Zeile der Nachricht die angegebenen Kopfzeilen und Nachspanne enthält.

Alle diese Teile sind **subst**-fähige Tcl-Skripte, die vor der Zusammenstellung einmal pro Nachricht ersetzt werden.

debug 2array

Diese Methode gibt ein Dictionary zurück, das die Namen aller dem Paket derzeit bekannten Debug-Tags ihrem Status und ihrer Protokollstufe zuordnet. Letztere werden in einem einzigen numerischen Wert kodiert, wobei eine negative Zahl ein inaktives Tag auf der durch den Absolutwert angegebenen Stufe bezeichnet und eine positive Zahl ein aktives Tag auf dieser Stufe.

Siehe auch die Methode **settings** weiter unten.

debug define *tag*

Diese Methode registriert den benannten *tag* im Paket. Wenn das Tag zuvor noch nicht bekannt gewesen ist, wird es in einen inaktiven Zustand platziert. Der Zustand eines bereits bekannten Tags bleibt unverändert.

Das Ergebnis der Methode ist die leere Zeichenkette.

debug header *text*

Diese Methode definiert ein globales **subst**-fähiges Tcl-Skript, das einen Text bereitstellt, der vor jeder Ausgabezeile gedruckt wird.

Beachten Sie, dass dies tagunabhängig ist.

Beachten Sie ferner, dass die Ersetzung der Kopfzeile nur einmal pro tatsächlich ausgegebener Meldung erfolgt, d. h., alle Zeilen derselben Meldung weisen denselben tatsächlichen Kopfzeilentext auf.

Das Ergebnis der Methode ist der angegebene Text.

debug level *tag* ?*level*? ?*fd*?
Diese Methode legt die Detailstufe *level* für das *tag* sowie den Kanal *fd* fest, in den die Beschreibung des Tags geschrieben werden soll. *level* ist ein ganzzahliger Wert >= 0, dessen Standardwert **1** lautet. Der Kanal ist standardmäßig auf **stderr** eingestellt.

Das Ergebnis der Methode ist die neue Detailstufe für das Tag.

debug names
Diese Methode gibt eine Liste zurück, die die Namen aller dem Paket derzeit bekannten Debug-Tags enthält.

debug off *tag*
Diese Methode registriert den benannten *tag* im Paket und setzt ihn auf inaktiv.

Das Ergebnis der Methode ist die leere Zeichenkette.

debug on *tag*
Diese Methode registriert den benannten *tag* als aktiv im Paket.

Das Ergebnis der Methode ist die leere Zeichenkette.

debug parray *arrayvarname*
Hierbei handelt es sich um eine Methode, die das benannte Array ähnlich wie der integrierte Befehl **parray** formatiert, mit dem Unterschied, dass sie die resultierende Zeichenkette zurückgibt, anstatt sie direkt in **stdout** zu schreiben.

Dadurch eignet sie sich für die Verwendung in Debug-Meldungen.

debug pdict *dict*
Hierbei handelt es sich um eine Methode, die das Dictionary ähnlich wie der integrierte Befehl **parray** für Arrays formatiert und die resultierende Zeichenkette zurückgibt.

Dadurch eignet sie sich für die Verwendung in Debug-Meldungen.

debug hexl *data* ?*prefix*?
Dies ist eine Hilfsmethode, die beliebige Daten als Hex-Dump formatiert und gibt die resultierende Zeichenkette zurück.

Dadurch eignet sie sich für die Verwendung in Debug-Meldungen.

Jeder Zeile des Dumps wird das Präfix *prefix* vorangestellt. Standardmäßig ist dieses Präfix eine leere Zeichenkette.

debug nl
Dies ist eine praktische Methode, um ein Zeilenvorschubzeichen (ASCII 0x0a) in eine Debug-Meldung einzufügen.

debug tab
Dies ist eine praktische Methode, um ein Tabulatorzeichen (ASCII 0x09) in eine Debug-Meldung einzufügen.

debug prefix *tag* ?*text*?
Diese Methode ähnelt der oben beschriebenen Methode **header** insofern, als sie ein **subst** -fähiges Tcl-Skript definiert, das zusätzlichen Text für Debug-Meldungen bereitstellt.

Im Gegensatz zu **header** wird der generierte Text der Meldung des Benutzers hinzugefügt, bevor diese in Zeilen aufgeteilt wird, wodurch es sich um eine Erweiterung auf Meldungsebene handelt.

Darüber hinaus ist das Skript tagabhängig.

Eine Ausnahme bildet das Skript für den Tag **::**, das auf alle Meldungen angewendet wird.

Wenn sowohl ein globales als auch ein tagabhängiges Präfix vorhanden sind, werden beide angewendet, wobei das globale Präfix vor dem tagabhängigen Präfix steht.

Beachten Sie, dass die Präfixersetzung nur einmal pro tatsächlich ausgegebener Meldung erfolgt.

Das Ergebnis der Methode ist die leere Zeichenkette.

Wenn das *tag* zum Zeitpunkt des Aufrufs noch nicht bekannt war, wird es registriert und als inaktiv markiert.

debug setting (*tag level*) ... ?*fd*?

Diese Methode ist eine Multi-Tag-Variante der oben genannten Methode **level**, in die zudem die Funktionalität der Methoden **on** und **off** integriert ist.

Jedes benannte *tag* wird auf die ihm folgende Detailstufe *level* gesetzt, wobei eine negative Stufe das Tag deaktiviert und eine positive Stufe es aktiviert.

Wenn auf das letzte Argument keine Stufe folgt, wird es nicht als Tag-Name behandelt, sondern als Kanal, in den alle benannten Tags ihre Meldungen ausgeben sollen.

Das Ergebnis der Methode ist die leere Zeichenkette.

debug suffix *tag* ?*text*?

Diese Methode ähnelt der unten aufgeführten Methode **trailer** insofern, als sie ein **subst** -fähiges Tcl-Skript definiert, das zusätzlichen Text für Debug-Meldungen bereitstellt.

Im Gegensatz zu **trailer** wird der generierte Text der Meldung des Benutzers hinzugefügt, bevor diese in Zeilen aufgeteilt wird, wodurch es sich um eine Erweiterung auf Meldungsebene handelt.

Darüber hinaus ist das Skript tagabhängig.

Als Ausnahme davon wird ein Skript für das Tag **::** auf alle Meldungen angewendet.

Wenn sowohl ein globales als auch ein tagabhängiges Suffix vorhanden sind, werden beide angewendet, wobei das globale Suffix nach dem tagabhängigen Suffix folgt.

Beachten Sie, dass die Ersetzung der Suffixe nur einmal pro tatsächlich ausgegebener

Meldung erfolgt.

Das Ergebnis der Methode ist die leere Zeichenkette.

Wenn das *tag* zum Zeitpunkt des Aufrufs noch nicht bekannt war, wird es registriert und als inaktiv markiert.

debug trailer *text*

Diese Methode definiert ein globales **subst**-fähiges Tcl-Skript, das einen Text bereitstellt, der nach jeder Ausgabezeile (jedoch vor dem Zeilenende) ausgegeben wird.

Hinweis: Dies ist tagunabhängig.

Hinweis: Die Ersetzung des Nachspanns erfolgt nur einmal pro tatsächlich ausgegebener Meldung, d. h., alle Zeilen derselben Meldung erhalten denselben tatsächlichen Nachspann.

Das Ergebnis der Methode ist der angegebene Text.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *debug* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Debugging, Ablaufverfolgung und Protokollierung

Copyright


```
Copyright (c) 200?, Colin McCormack, Wub Server Utilities
Copyright (c) 2012-2014, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


debug_caller

Stichwörter: debug_caller, debug, log, narrative, trace

Name

debug::caller – Debug-Erzählung – Aufrufer

Übersicht

package require Tcl 8.5 9
package require debug::caller ?**1.2**?
debug caller ?*args...*?

Beschreibung

API

debug **caller ?*args*...?**

Diese Methode ist nützlich, um in einem tag-spezifischen Präfix automatisch Aufruferinformationen für alle Verwendungen des Tags bereitzustellen. Oder in einer Nachricht, wenn nur bestimmte Plätze solche Details benötigen.

Darüber hinaus erkennt sie die verschiedenen internen Formen von Aufrufen für Methoden, die vom OO-System **snit** generiert werden, und schreibt diese zur besseren Lesbarkeit in ihre ursprüngliche Form um. Gleiches gilt für **TclOO**.

Wenn *args* festgelegt werden, werden diese als ganzzahlige Indizes von Befehlsargumenten behandelt, die in der Ausgabe *nicht* angezeigt werden sollen. Die betreffenden Argumente werden stattdessen durch * ersetzt. Der wichtigste vorgesehene Anwendungsfall hierfür ist der Ausschluss von Argumenten, von denen erwartet wird, dass sie große Tcl-Werte enthalten, d. h. lange Listen, große Dictionarys usw., um zu verhindern, dass sie die Erzählung überfrachten.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *debug* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Debugging, Ablaufverfolgung und Protokollierung

Copyright


```
Copyright (c) 2012-2015, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


debug_heartbeat

Stichwörter: debug_heartbeat, debug, heartbeat, log, narrative, trace

Name

debug::heartbeat – Debug-Erzählung – Heartbeat

Übersicht

package require Tcl 8.5 9
package require debug::heartbeat ?**1.0.2**?
package require debug ?**1**?
debug heartbeat ?*delta*?

Beschreibung

API

debug **heartbeat ?*delta*?**

Diese Methode aktiviert oder deaktiviert einen Heartbeat, mit dem die Ereignisschleife einer ereignisbasierten Tcl-Anwendung überwacht wird.

Sie reserviert für ihren Betrieb das Debug-Tag **heartbeat** und schreibt alle *delta* Millisekunden eine Meldung.

Ein *delta* <= 0 deaktiviert den Heartbeat.

Die vom Heartbeat erzeugte Meldung enthält einen Sequenzzähler und die Zeit in Millisekunden seit dem letzten Heartbeat, wodurch Einblicke in zeitliche Schwankungen und Abweichungen vom Nennwert *delta* gewonnen werden können.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *debug* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Debugging, Ablaufverfolgung und Protokollierung

Copyright


```
Copyright (c) 200?, Colin McCormack, Wub Server Utilities
Copyright (c) 2012, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


debug_timestamp

Stichwörter: debug_timestamp, debug, log, narrative, timestamps, trace

Name

debug::timestamp – Debug-Erzählung – Zeitstempel

Übersicht

package require Tcl 8.5 9
package require debug::timestamp ?**1.1**?
package require debug ?**1.0.7**?
debug timestamp

Beschreibung

API

debug **timestamp**

Diese Methode gibt Zeitangaben in Millisekunden seit einer Basislinie oder dem letzten Aufruf zurück und eignet sich daher besonders für ein tag-spezifisches Präfix, um automatisch Aufruferinformationen für alle Verwendungen des Tags bereitzustellen. Oder in einer Nachricht, wenn nur an bestimmten Plätzen solche Details benötigt werden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *debug* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Debugging, Ablaufverfolgung und Protokollierung

Copyright


```
Copyright (c) 200?, Colin McCormack, Wub Server Utilities
Copyright (c) 2012, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


defer

defer

Stichwörter: defer, cleanup, golang

Name

defer – Aufgeschobene Ausführung

Übersicht

package require Tcl 8.6 9
package require defer ?**1.1**?
::defer::defer ?*command*? ?*arg1*? ?*arg2*? ?*argN...*?
::defer::with *variableList script*
::defer::autowith *script*
::defer::cancel ?*id...*?

Beschreibung

Mit den **defer**-Befehlen kann ein Entwickler Aktionen so planen, dass sie im Rahmen der Beendigung des aktuellen Variablenbereichs ausgeführt werden. Dies ist besonders nützlich für Aufräumvorgänge. Da die aufgeschobenen Aktionen stets ausgeführt werden – und zwar immer in umgekehrter Reihenfolge zu der, in der die "defer"-Anweisungen selbst ausgeführt werden –,

kann der Programmierer die Bereinigung einer Ressource (beispielsweise eines Kanals) planen, sobald diese Ressource angefordert wird. Wenn die Prozedur oder das Lambda später beendet wird – sei es aufgrund eines Fehlers oder einer expliziten Rückkehr –, erfolgt die Bereinigung dieser Ressource stets.

Befehle

::defer::defer ?*command*? ?*arg1*? ?*arg2*? ?*argN...*?
Verschiebt die Ausführung eines Teils des Codes bis zum Ende des aktuellen Variablenbereichs. Jedes Argument wird mit den anderen verkettet, um das Skript zu bilden, das zum Zeitpunkt der Verschiebung ausgeführt wird. Es können mehrere defer-Anweisungen verwendet werden; diese werden in der Reihenfolge "Last-In, First-Out" ausgeführt. Der Wert ist ein Bezeichner, der später mit **defer::cancel** verwendet werden kann.

::defer::with *variableList script*
Verschiebt die Ausführung eines Skripts und kopiert dabei den aktuellen Wert einiger Variablen, deren Namen in *variableList* festgelegt sind, in das Skript. Das Skript verhält sich wie ein Lambda, wird jedoch auf derselben Ebene wie der Aufruf von **defer::with** ausgeführt. Der Wert wird wie bei **::defer::defer** zurückgegeben

::defer::autowith *script*
Entspricht **::defer::with**, verwendet jedoch alle lokalen Variablen in der Variablenliste.

::defer::cancel ?*id...*?
Bricht die Ausführung einer zurückgestellten Aktion ab. Das Argument *id* ist die Kennung, die von den Funktionen **::defer::defer**, **::defer::with** oder **::defer::autowith** gegeben wird. Es können beliebig viele Argumente angegeben werden; alle angegebenen IDs werden dabei abgebrochen.

Beispiele

```
package require defer 1
apply {{} {
    set fd [open /dev/null]
    defer::defer close $fd
}}
```

Literatur

1. siehe unten ...

Autoren

Roy Keane

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *defer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Hilfsprogramme

Copyright
```
Copyright (c) 2017, Roy Keene
```

des

des
Stichwörter: des, 3DES, block cipher, data integrity, encryption, security

Name
des – Implementierung der DES- und Triple-DES-Verschlüsselungsverfahren

Übersicht
package require Tcl 8.5 9
package require des 1.2
::DES::des ?*-mode [ecb|cbc|cfb|ofb]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-weak*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | *data*]
::DES::Init *mode keydata iv* ?*weak*?
::DES::Encrypt *Key data*
::DES::Decrypt *Key data*
::DES::Reset *Key iv*
::DES::Final *Key*

Beschreibung
Hierbei handelt es sich um eine Implementierung des Data Encryption Standard (DES) in Tcl, wie er vom US-amerikanischen National Institute of Standards and Technology (NIST) veröffentlicht wurde [1]. Diese Implementierung unterstützt zudem die Erweiterung "Triple DES" (3DES) des DES. DES ist eine 64-Bit-Blockverschlüsselung, die einen 56-Bit-Schlüssel verwendet. 3DES verwendet einen 168-Bit-Schlüssel. DES wurde inzwischen offiziell durch AES abgelöst, wird jedoch in vielen Protokollen weiterhin häufig verwendet.

Die tcllib-Implementierung von DES und 3DES basiert auf einer Implementierung von Mac Cody und steht als separater Download unter [2] zur Verfügung. Wer sich für die Einzelheiten der Weitergabe dieses Codes interessiert, wird auf die TclDES-Seiten verwiesen. Der tcllib-spezifische Code ist ein Wrapper für die TclDES-API, der für die DES-Verschlüsselung dieselbe API bereitstellt wie für andere Verschlüsselungsalgorithmen in der Bibliothek.

Befehle
::DES::des ?*-mode [ecb|cbc|cfb|ofb]*? ?*-dir [encrypt|decrypt]*? *-key keydata* ?*-iv vector*? ?*-hex*? ?*-weak*? ?*-out channel*? ?*-chunksize size*? [*-in channel* | *data*]

Wendet den Algorithmus **DES** entweder auf die durch das Argument bereitgestellten Daten oder auf die aus dem Kanal *-in* gelesenen Daten an. Wenn ein *-out*-Kanal angegeben wird, wird das Ergebnis in diesen Kanal geschrieben.

Die Option *-key* muss angegeben werden. Dieser Parameter erwartet eine binäre Zeichenkette mit einer Länge von 8 Byte und dient zur Erzeugung des Schlüsselschemas. Bei DES werden nur 56 Bit der Schlüsseldaten verwendet. Das höchstwertige Bit jedes Bytes wird verworfen.

Die Optionen *-mode* und *-dir* sind optional und sind standardmäßig auf den CBC-Modus bzw. encrypt gesetzt. Der Initialisierungsvektor *-iv* erwartet ein binäres Argument mit einer Länge von 8 Byte. Standardmäßig besteht dieser aus lauter Nullen. Weitere Informationen zu *-mode* und zur Verwendung des Initialisierungsvektors finden Sie unter Betriebsarten.

DES ist eine 64-Bit-Blockverschlüsselung. Das bedeutet, dass die Daten in Einheiten bereitgestellt werden müssen, die ein Vielfaches von 8 Bytes sind.

Programmierschnittstelle

Der interne Zustand wird in einer undurchsichtigen Struktur verwaltet, die von der Funktion **Init** gegeben wird. Im ECB-Modus wird der Zustand durch die Eingabe nicht beeinflusst; in anderen Modi wird jedoch ein gewisser eingabebedingter Zustand verwaltet, der durch den Aufruf der Funktion **Reset** mit einem neuen Wert für den Initialisierungsvektor zurückgesetzt werden kann.

::DES::Init *mode keydata iv ?weak?*

Erstellt ein neues DES-Schlüsselschema unter Verwendung der angegebenen Schlüsseldaten und des vorgegebenen Initialisierungsvektors. Der Initialisierungsvektor wird im ECB-Modus nicht verwendet, ist jedoch für andere Betriebsmodi von Bedeutung. Siehe Betriebsarten.

Es gibt eine kleine Anzahl von Schlüsseln, von denen bekannt ist, dass sie bei der Verwendung mit DES schwach sind. Standardmäßig wird ein Fehler ausgelöst, wenn ein solcher Schlüssel übergeben wird. Sollte es erforderlich sein, solche Schlüssel zu akzeptieren, kann der Parameter *weak* auf "true" gesetzt werden, um die Auslösung des Fehlers zu vermeiden.

::DES::Encrypt *Key data*

Verschlüsselt die bereitgestellten Daten mit einem vorbereiteten Schlüssel aus **Init**. Das Argument "data" sollte ein binäres Array sein, dessen Größe ein Vielfaches der DES-Blockgröße von 8 Byte beträgt. Das Ergebnis ist ein binäres Array, dessen Größe der der Eingabedaten entspricht.

::DES::Decrypt *Key data*

Hinweis: Der gleiche Schlüssel kann sowohl zum Verschlüsseln als auch zum Entschlüsseln von Daten verwendet werden, sofern der Initialisierungsvektor für den CBC-Modus entsprechend zurückgesetzt wird.

::DES::Reset *Key iv*

Setzt den Initialisierungsvektor zurück. Dies ermöglicht es dem Programmierer, einen Schlüssel wiederzuverwenden und den Aufwand für die Neugenerierung des Schlüsselschemas zu vermeiden, wenn dieselben Schlüsseldaten mehrfach verwendet werden.

::DES::Final *Key*

Diese Funktion sollte aufgerufen werden, um die mit *Key* verbundenen Ressourcen

freizugeben. Sobald diese Funktion aufgerufen wurde, darf der Schlüssel nicht mehr verwendet werden.

Betriebsarten

Electronic Code Book (ECB)

Der ECB-Modus ist der Grundmodus aller Blockchiffren. Jeder Block wird unabhängig verschlüsselt, sodass identischer Klartext bei der Verschlüsselung mit demselben Schlüssel zu identischem Ergebnis führt. Etwaige Verschlüsselungsfehler wirken sich zwar nur auf einen einzelnen Block aus, dieser ist jedoch anfällig für Angriffe mit bekanntem Klartext.

Cipher Block Chaining (CBC)

Im CBC-Modus wird die Ausgabe der Verschlüsselung des letzten Blocks zur Beeinflussung des aktuellen Blocks herangezogen. Für die Verarbeitung des ersten Blocks wird ein Initialisierungsvektor verwendet, dessen Größe der Chiffreblockgröße entspricht. Der Initialisierungsvektor sollte zufällig ausgewählt und als erster Block der Ausgabe übertragen werden. Fehler bei der Verschlüsselung wirken sich auf den aktuellen Block und den nächsten Block aus; danach korrigiert sich die Chiffre von selbst. CBC ist der am häufigsten verwendete Modus bei der Softwareverschlüsselung.

Cipher Feedback (CFB)

Der CFB-Modus kann verwendet werden, um Blockchiffren in Stromchiffren umzuwandeln. Im CFB-Modus wird der Initialisierungsvektor verschlüsselt und die Ausgabe anschließend mit dem Klartext-Strom XOR-verknüpft. Das Ergebnis wird dann als Initialisierungsvektor für die nächste Runde verwendet. Fehler wirken sich auf den aktuellen Block und den nächsten Block aus.

Output Feedback (OFB)

OFB ähnelt CFB, mit dem Unterschied, dass das Ergebnis der Verschlüsselung in die nächste Runde zurückgeführt wird und nicht der XOR-verwandelte Klartext. Dies bedeutet, dass sich Fehler nur auf einen einzelnen Block auswirken, die Verschlüsselung jedoch anfälliger für Angriffe ist.

Beispiele

```
% set ciphertext [DES::des -mode cbc -dir encrypt -key $secret
    $plaintext]
% set plaintext [DES::des -mode cbc -dir decrypt -key $secret
    $ciphertext]
```

```
set iv [string repeat \\0 8]
set Key [DES::Init cbc \\0\\1\\2\\3\\4\\5\\6\\7 $iv]
set ciphertext [DES::Encrypt $Key "somedata"]
append ciphertext [DES::Encrypt $Key "moredata"]
DES::Reset $Key $iv
set plaintext [DES::Decrypt $Key $ciphertext]
DES::Final $Key
```

Literatur

1. "Data Encryption Standard", Federal Information Processing Standards Publication 46-3, 1999, (http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf)

2. "TclDES: Tcl-Skripting auf höchstem Niveau" http://tcldes.sourceforge.net//

Autoren
Jochen C. Loewer, Mac Cody, Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *des* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```


tcldes
Stichwörter: tcldes, 3DES, DES, block cipher, data integrity, encryption, security

Name
tclDES – Implementierung der DES- und Triple-DES-Verschlüsselungsverfahren

Übersicht
package require Tcl 8.5 9
package require tclDES 1.1

Beschreibung
Das Paket "**tclDES**" ist ein Hilfspaket für "**des**".

Einzelheiten entnehmen Sie bitte der Dokumentation zu **des**.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *des* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```

tcldesjr

Stichwörter: tcldesjr, 3DES, DES, block cipher, data integrity, encryption, security

Name

tclDESjr – Implementierung der DES- und Triple-DES-Verschlüsselungsalgorithmen

Übersicht

package require Tcl 8.5 9
package require tclDESjr 1.1

Beschreibung

Das Paket "**tclDESjr**" ist ein Hilfspaket für "**des**".

Einzelheiten entnehmen Sie bitte der Dokumentation zu **des**.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *des* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```

dicttool

dicttool

Stichwörter: dicttool, dict

Name

dicttool – Dictionary-Tools

Übersicht

package require Tcl 8.5 9
package require dicttool ?**1.2**?
ladd *dictVar* ?*key value* ...?
ldelete *dictVar* ?*key value* ...?

dict getnull *key*
dict print *aDict*
dict is_dict *value*
dict rmerge *dict1 dict2 ?dict3 ...?*

Beschreibung

Dieses Paket erweitert den integrierten Befehl dict um mehrere neue Befehle. Darüber hinaus definiert das Paket auch einige praktische Listenbefehle. (Jeder Befehl wird nur dann hinzugefügt, wenn noch kein gleichnamiger Befehl vorhanden ist – für den Fall, dass einer dieser Befehle in den Kern aufgenommen wird.)

API

ladd *dictVar ?key value ...?*
: Dieser Befehl fügt jedes Paar von *Schlüssel-Wert*-Elementen zu *dictVar* hinzu, jedoch nur, wenn dieses Element noch nicht vorhanden ist (weder als Schlüssel noch als Wert). Dieser Befehl *muss* mit Paaren von *Schlüssel-Wert*-Elementen aufgerufen werden, damit *dictVar* als **dict** erhalten bleibt.

ldelete *dictVar ?key value ...?*
: Dieser Befehl löscht jedes Paar von *Schlüssel-Wert*-Elementen aus *dictVar*. Er führt keine Aktion durch, wenn das Paar nicht bereits in *dictVar* vorhanden ist. Dieser Befehl *muss* mit Paaren von *Schlüssel-Wert*-Elementen aufgerufen werden, um *dictVar* als **dict** zu erhalten.

  Verwenden Sie den Unterbefehl **dict remove** des integrierten Befehls dict, um Elemente allein anhand des Schlüssels zu entfernen.

dict getnull *key*
: Funktioniert wie **dict get**; wenn der Schlüssel *key* jedoch nicht vorhanden ist, gibt es eine leere Liste zurück, anstatt einen Fehler auszulösen.

  Eine Alternative zu diesem Befehl ist der integrierte Unterbefehl **dict getwithdefault** des Befehls dict *oder dessen Alias* **dict getdef**. Die Verwendung von **dict getdef** mit einer leeren Liste *{}* als Standardwert entspricht der Verwendung von **dict getnull**.

dict print *aDict*
: Gibt eine "übersichtlich formatierte" Zeichenfolgendarstellung von **dict** *aDict* zurück, wobei jeder verschachtelte Zweig in einer neuen Zeile steht und auf jeder Ebene um zwei Leerzeichen eingerückt ist.

dict is_dict *value*
: Gibt **1** (true) zurück, wenn *value* als **dict** interpretiert werden kann; andernfalls **0** (false).

  Dieser Befehl erzwingt *nicht*, dass eine vorhandene **dict**-Darstellung in eine andere interne Darstellung umgewandelt wird.

dict rmerge *dict1 dict2 ?dict3 ...?*
: Gibt ein **dict** zurück, das aus der rekursiven Verschmelzung aller **dict**-Argumente entsteht. Anders als der Unterbefehl **dict merge** des eingebauten Befehls dict steigt dieser Befehl in alle Ebenen des **dict** hinab. Beachten Sie, dass **dict**-Schlüssel, die auf einen Doppelpunkt enden, ein Blatt bezeichnen. Sie gelten als literaler Wert und werden nicht weiter hinab verfolgt.

```
set items [dict rmerge {
  option {color {default: green}}
} {
  option {fruit {default: mango}}
} {
  option {color {default: blue} fruit {widget: select values: {mango
    apple cherry grape}}}
}]
puts [dict print $items]
=>
option {
  color {
    default: blue
  }
  fruit {
    default: mango
    widget: select
    values: {mango apple cherry grape}
  }
}
```

Wenn oben der Unterbefehl "**dict merge**" des integrierten Befehls "dict](https://www.tcl-lang.org/man/tcl/TclCmd/dict.html)" verwendet worden wäre, wäre der Schlüssel-Wert-Eintrag "*default*" des Eintrags "*fruit*" verloren gegangen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *dict* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hilfsprogramme

Copyright


```
Copyright (c) 2017 Sean Woods <yoda@etoyoc.com>
```


dns

tcllib_dns

Stichwörter: tcllib_dns, DNS, domain name service, resolver, rfc 1034, rfc 1035, rfc 1886, rfc 7858

Name

dns – Tcl-Client für den Domain Name Service

Übersicht

package require Tcl 8.5 9
package require dns ?**1.6.1**?
::dns::resolve *query* ?*options*?
::dns::configure ?*options*?
::dns::name *token*
::dns::address *token*
::dns::cname *token*
::dns::result *token*
::dns::status *token*
::dns::error *token*
::dns::reset *token*
::dns::wait *token*
::dns::cleanup *token*
::dns::nameservers

Beschreibung

Das Paket "dns" stellt einen ausschließlich auf Tcl basierenden DNS-Client (Domain Name Service) bereit. Informationen zum DNS-Protokoll finden Sie unter (1) und (2); lesen Sie die Dokumentation zu resolver(3), um zu erfahren, wie die C-Bibliothek Domänennamen auflöst. Dieses Paket soll Tcl-Skripte vor Problemen schützen, die bei der Verwendung des System-Resolvers für langsame Namensserver auftreten können. Es kann von praktischem Nutzen sein oder auch nicht. Die Namensauflösung im Internet ist ein komplexes Unterfangen, und DNS ist nur ein Teil des Resolvers. Möglicherweise stellen Sie fest, dass Sie Hosts-Dateien, NIS oder WINS verwenden sollten, um nur einige andere Systeme zu nennen. Dieses Paket ist keine Ersetzung für den Resolver der C-Bibliothek – es implementiert jedoch die Namensauflösung über DNS. Das Paket erweitert zudem das Paket **uri** um die Unterstützung von DNS-URIs (4) der Form *dns:what.host.com* oder *dns://my.nameserver/what.host.com*. Der Befehl **dns::resolve** kann sowohl DNS-URIs als auch einfache Domänennamen als Abfrage verarbeiten.

Hinweis: Das Paket verwendet standardmäßig DNS über TCP-Verbindungen. Wenn Sie UDP verwenden möchten, müssen Sie das Paket **tcludp** installiert haben und über eine Version verfügen, die Binärdaten korrekt verarbeitet (> 1.0.4). Dieses ist unter http://tcludp.sourceforge.net/ verfügbar. Wenn das Paket **udp** vorhanden ist, wird standardmäßig UDP verwendet.

Hinweis: Das Paket unterstützt DNS über TLS (RFC 7858) für einen besseren Datenschutz bei DNS-Abfragen. Für die Nutzung dieser Funktion ist das TLS-Paket erforderlich.

Befehle

::dns::resolve *query* ?*options*?

Löst einen Domänennamen mithilfe des *DNS*-Protokolls auf. *query* ist der Domänenname, der abgefragt werden soll. Dabei sollte es sich entweder um einen vollqualifizierten Domänennamen oder um eine DNS-URI handeln.

-nameserver *hostname* or -server *hostname*

Legt einen alternativen Namen für den Nameserver für diese Anfrage fest.

-protocol *tcp|udp*
Legt das für diese Anfrage zu verwendende Netzwerkprotokoll fest. Mögliche Werte sind *tcp* oder *udp*.

-port *portnum*
Legt einen alternativen Port fest.

-search *domainlist*
siehe unten ...

-timeout *milliseconds*
Überschreibt die Standard-Zeitüberschreitung.

-type *TYPE*
Legt den Typ des DNS-Eintrags fest, der Sie interessiert. Gültige Werte sind A, NS, MD, MF, CNAME, SOA, MB, MG, MR, NULL, WKS, PTR, HINFO, MINFO, MX, TXT, SPF, SRV, AAAA, AXFR, MAILB, MAILA und *. Einzelheiten zu den Werten, die zurückgegeben werden, finden Sie in RFC 1035. Informationen zu SPF finden Sie unter http://spf.pobox.com/. Informationen zu AAAA-Einträgen finden Sie unter (3) und Einzelheiten zu SRV-Einträgen in RFC 2782.

-class *CLASS*
Legt die Klasse des Domainnamens fest. In der Regel ist dies "IN", es kann sich jedoch auch um "IN" für Internet-Domainnamen, "CS", "CH", "HS" oder "*" für eine beliebige Klasse handeln.

-recurse *boolean*
Setzt diesen Wert auf *false*, wenn Sie nicht möchten, dass der Nameserver Ihre Anfrage rekursiv bearbeitet. Standardmäßig ist der Wert auf *true* gesetzt.

-command *procname*
Legt eine Prozedur fest, die nach Abschluss der Anfrage aufgerufen werden soll. Der Prozedur wird das Token als einziges Argument übergeben.

-usetls *boolean*
Setzt den Wert auf *true*, um DNS über TLS zu verwenden. Dadurch wird die Verwendung von TCP erzwungen und der Standardport auf 853 geändert. Da eine Validierung der Zertifikate erforderlich ist, muss eine Quelle für Zertifikate einer vertrauenswürdigen Zertifizierungsstelle mithilfe von *-cafile* oder *-cadir* angegeben werden.

-cafile *filepath*
Gibt eine Datei an, die eine Sammlung vertrauenswürdiger Zertifikate von Zertifizierungsstellen enthält. Weitere Informationen finden Sie auf der Seite des Handbuchs zum Befehl **update-ca-certificates** oder in der Hilfe zur Option **-CAfile** von **openssl**.

-cadir *dirpath*
Gibt ein Verzeichnis an, das Zertifikate vertrauenswürdiger Zertifizierungsstellen enthält. Diese Angabe ist erforderlich, wenn **-cafile** nicht angegeben ist, damit die Validierung der Zertifikate funktioniert, wenn **-usetls** aktiviert ist. Informationen zur erforderlichen Struktur dieses Verzeichnisses finden Sie in der Dokumentation zu **openssl**.

::dns::configure ?*options*?
Der Befehl **::dns::configure** dient zur Konfiguration des DNS-Pakets. Der abzufragende

Server, das Protokoll und der Pfad für Domänen werden alle über diesen Befehl festgelegt. Werden keine Argumente angegeben, gibt der Befehl eine Liste aller aktuellen Einstellungen zurück. Wird nur ein Argument angegeben, muss es sich um den Namen einer Option handeln, und der Wert dieser Option wird zurückgegeben.

-nameserver *hostname*
Legt den Standard-Name des Nameservers fest, der für alle Abfragen verwendet werden soll. Die Standardeinstellung lautet *localhost*.

-protocol *tcp*|*udp*
Legt das standardmäßig zu verwendende Netzwerkprotokoll fest. Die Standardeinstellung ist *tcp*.

-port *portnum*
Legt den Standardport fest, der auf dem Nameserver verwendet werden soll. Der Standardwert ist 53.

-search *domainlist*
Legt die Liste für die Domänensuche fest. Diese wird derzeit nicht verwendet.

-timeout *milliseconds*
Legt den Standardwert für das Timeout bei DNS-Abfragen fest. Der Standardwert beträgt 30 Sekunden.

-loglevel *level*
Legt die Protokollstufe fest, die für die Ausgabe von Diagnosemeldungen aus diesem Paket verwendet wird. Die Standardeinstellung ist *warn*. Einzelheiten zu den verfügbaren Stufen finden Sie im Paket **log**.

-cafile *filepath*
Legt den Standard-Pfad für die Option "**-cafile**" auf "**dns::resolve**" fest.

-cadir *dirpath*
Legt den Standard-Verzeichnis-Pfad, der für die Option "**-cadir**" verwendet werden soll, auf "**dns::resolve**" fest.

::dns::name *token*
Gibt eine Liste aller Domainnamen zurück, die als Antwort auf Ihre Abfrage zurückgegeben wurden.

::dns::address *token*
Gibt eine Liste der Adressdatensätze zurück, die Ihrer Abfrage entsprechen.

::dns::cname *token*
Gibt eine Liste der kanonischen Namen zurück (in der Regel nur einen), die Ihrer Abfrage entsprechen.

::dns::result *token*
Gibt eine Liste aller dekodierten Antwortdatensätze zurück, die für Ihre Abfrage bereitgestellt wurden. Auf diese Weise können Sie auch bei ungewöhnlicheren Abfragetypen das Ergebnis extrahieren.

::dns::status *token*
Gibt das Statusflag zurück. Bei einer erfolgreich abgeschlossenen Abfrage lautet dieses *ok*. Es kann auch *error*, *timeout* oder *eof* lauten. Siehe auch **::dns::error**

::dns::error *token*
Gibt die Fehlermeldung zu Anfragen mit dem Status *error* zurück. Liegt keine Fehlermeldung

vor, kommt eine leere Zeichenkette zurück.

::dns::reset *token*
Eine DNS-Abfrage zurücksetzen oder abbrechen.

::dns::wait *token*
Wartet, bis eine DNS-Abfrage abgeschlossen ist, und gibt danach den Status zurück.

::dns::cleanup *token*
Entfernt alle Zustandsvariablen, die zu der Anfrage gehören.

::dns::nameservers
Versucht, eine Liste der derzeit für das System des Benutzers konfigurierten Nameserver zurückzugeben. Auf einem Unix-Rechner werden die in der Datei /etc/resolv.conf angegebenen Nameserver ausgewertet (sofern diese Datei vorhanden ist), während auf Windows-Systemen bestimmte Bereiche der Registrierung überprüft werden. Kann kein Nameserver gefunden werden, wird die Loopback-Adresse (127.0.0.1) als Standard verwendet.

Beispiele

```
% set tok [dns::resolve www.tcl.tk]
::dns::1
% dns::status $tok
ok
% dns::address $tok
199.175.6.239
% dns::name $tok
www.tcl.tk
% dns::cleanup $tok
```

Verwendung von DNS-URIs als Abfragen:

```
% set tok [dns::resolve "dns:tcl.tk;type=MX"]
% set tok [dns::resolve "dns://l.root-servers.net/www.tcl.tk"]
```

Adressrückwärtssuche:

```
% set tok [dns::resolve 127.0.0.1]
::dns::1
% dns::name $tok
localhost
% dns::cleanup $tok
```

Verwendung von DNS über TLS (RFC 7858):

```
% set tok [dns::resolve www.tcl.tk -nameserver
    dns-tls.bitwiseshift.net  -usetls 1 -cafile
    /etc/ssl/certs/ca-certificates.crt]
::dns::12
% dns::wait $tok
ok
% dns::address $tok
104.25.119.118 104.25.120.118
```

Literatur

1. Mockapetris, P., "Domain Names – Concepts and Facilities", RFC 1034, November 1987. (http://www.ietf.org/rfc/rfc1034.txt)
2. Mockapetris, P., "Domain Names – Implementation and Specification", RFC 1035, November 1087. (http://www.ietf.org/rfc/rfc1035.txt)
3. Thompson, S. und Huitema, C., "DNS-Erweiterungen zur Unterstützung von IP Version 6", RFC 1886, Dezember 1995. (http://www.ietf.org/rfc/rfc1886.txt)
4. Josefsson, S., "Domain Name System Uniform Resource Identifiers", Internet-Entwurf, Oktober 2003, (http://www.ietf.org/internet-drafts/draft-josefsson-dns-url-09.txt)
5. Gulbrandsen, A., Vixie, P. und Esibov, L., "Ein DNS-RR zur Festlegung des Standorts von Diensten (DNS SRV)", RFC 2782, Februar 2000, (http://www.ietf.org/rfc/rfc2782.txt)
6. Ohta, M. "Incremental Zone Transfer in DNS", RFC 1995, August 1996, (http://www.ietf.org/rfc/rfc1995.txt)
7. Hu, Z. et al. "Spezifikation für DNS über Transport Layer Security (TLS)", RFC 7858, Mai 2016, (http://www.ietf.org/rfc/rfc7858.txt)

Autoren

Pat Thoyts

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *dns* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

Copyright (c) 2002, Pat Thoyts

tcllib_ip

Stichwörter: tcllib_ip, internet address, ip, ipv4, ipv6, rfc 3513

Name
tcllib_ip – Bearbeitung von IPv4- und IPv6-Adressen

Übersicht
package require Tcl 8.5 9
package require ip ?**1.5.1**?
::ip::version *address*
::ip::is *class address*
::ip::equal *address address*
::ip::normalize *address*
::ip::contract *address*
::ip::distance *ipaddr1 ipaddr2*
::ip::nextIp *ipaddr* ?*offset*?
::ip::prefix *address*
::ip::type *address*
::ip::mask *address*
::ip::prefixToNative *prefix*
::ip::nativeToPrefix *nativeList*|*native* ?**-ipv4**?
::ip::intToString *number* ?**-ipv4**?
::ip::toInteger *ipaddr*
::ip::toHex *ipaddr*
::ip::maskToInt *ipmask*
::ip::broadcastAddress *prefix* ?**-ipv4**?
::ip::maskToLength *dottedMask*|*integerMask*|*hexMask* ?**-ipv4**?
::ip::lengthToMask *maskLength* ?**-ipv4**?
::ip::nextNet *ipaddr ipmask* ?*count*? ?**-ipv4**?
::ip::isOverlap *prefix prefix...*
::ip::isOverlapNative ?**-all**? ?**-inline**? ?**-ipv4**? *hexipaddr hexipmask hexiplist*
::ip::ipToLayer2Multicast *ipaddr*
::ip::ipHostFromPrefix *prefix* ?*-exclude prefixExcludeList*?
::ip::reduceToAggregates *prefixlist*
::ip::longestPrefixMatch *ipaddr prefixlist* ?**-ipv4**?
::ip::collapse *prefixlist*
::ip::subtract *prefixlist*

Beschreibung
Dieses Paket enthält eine Reihe von Befehlen, die beim Analysieren, Anzeigen und Vergleichen von Internetadressen helfen. Das Paket unterstützt sowohl IPv4- (1) als auch IPv6- (2) Adresstypen.

Befehle
::ip::version *address*
Gibt die Protokollversion der Adresse zurück (**4** oder **6**) oder **-1**, wenn es sich bei der Adresse weder um eine IPv4- noch um eine IPv6-Adresse handelt.

::ip::is *class address*
Gibt "true" zurück, wenn die Adresse zur angegebenen Protokollklasse gehört. Der

Klassenparameter kann entweder **ipv4** oder **ipv6** lauten. Dies entspricht im Wesentlichen dem Befehl **version** in Form eines booleschen Werts. Das Argument *class* kann auf **4** oder **6** verkürzt werden.

::ip::equal *address address*
Vergleicht zwei Adressangaben auf Gleichheit. Die Argumente werden normalisiert und das Adresspräfix ermittelt (sofern eine Maske angegeben wird). Die normalisierten Adressen werden anschließend Bit für Bit verglichen, und die Prozedur gibt "wahr" zurück, wenn sie übereinstimmen.

::ip::normalize *address*
Wandelt eine IPv4- oder IPv6-Adresse in eine vollständig ausgeschriebene Version um. Es gibt verschiedene Kurzschreibweisen für Internetadressen, bei denen überflüssige Teile oder Ziffern weggelassen werden. Diese Prozedur ist das Gegenteil von **contract**.

::ip::contract *address*
Wandelt die Internetadresse "**normalize**d" in eine kompaktere Form um, die für die Anzeige gegenüber den Benutzern geeignet ist.

::ip::distance *ipaddr1 ipaddr2*
Dieser Befehl berechnet den (ganzzahligen) Abstand zwischen der IPv4-Adresse *ipaddr1* und der IPv4-Adresse *ipaddr2*, d. h. "ipaddr2 – ipaddr1"

```
% ::ip::distance 1.1.1.1  1.1.1.5
4
```

::ip::nextIp *ipaddr* ?*offset*?
Dieser Befehl addiert die Ganzzahl *offset* zur IPv4-Adresse *ipaddr* und gibt die neue IPv4-Adresse zurück.

```
% ::ip::distance 1.1.1.1  4
1.1.1.5
```

::ip::prefix *address*
Gibt das Adresspräfix zurück, das durch Maskierung des Adressteils mit der Maske generiert wurde, sofern eine solche angegeben wurde. Liegt keine Maske vor, entspricht dies dem Aufruf von **normalize**.

::ip::type *address*
siehe unten ...

::ip::mask *address*
Wenn die angegebene Adresse eine Maske enthält, gibt sie diese zurück; andernfalls gibt sie eine leere Zeichenkette zurück.

::ip::prefixToNative *prefix*
Dieser Befehl konvertiert die Zeichenfolge *prefix* aus der Punktnotation (im Format /) in die native (hexadezimale) Form. Gibt eine Liste zurück, die zwei Elemente – "ipaddress" und "mask" – in dieser Reihenfolge in hexadezimaler Schreibweise enthält.

```
% ip::prefixToNative 1.1.1.0/24
0x01010100 0xffffff00
```

::ip::nativeToPrefix *nativeList*|*native* ?-ipv4?

Dieser Befehl wandelt die native (Hexadezimal-)Darstellung in die Punktnotation um. Er ist das Gegenstück zu **::ip::prefixToNative**.

list *nativeList* (in)
Liste mehrerer IP-Adressen in nativer Form. Die native Form ist eine Liste, wie sie von der Funktion **::ip::prefixToNative** zurückgegeben wird.

list *native* (in)
Eine Liste, wie sie von der Funktion **::ip::prefixToNative** zurückgegeben wird.

Der Befehl gibt eine Liste von Adressen in Punktnotation zurück, wenn er mit einer Liste von Adressen aufgerufen wurde. Andernfalls wird eine einzelne Adresse in Punktnotation zurückgegeben.

```
% ip::nativeToPrefix {0x01010100 0xffffff00} -ipv4
1.1.1.0/24
```

::ip::intToString *number* ?-ipv4?
Dieser Befehl wandelt eine als Ganzzahl festgelegte IP-Adresse in die Punktnotation um.

```
    ip::intToString 4294967295
    255.255.255.255
```

::ip::toInteger *ipaddr*
Dieser Befehl wandelt eine IP-Adresse in Punktnotation in eine Ganzzahl um.

```
% ::ip::toInteger 1.1.1.0
16843008
```

::ip::toHex *ipaddr*
Dieser Befehl wandelt eine IP-Adresse in Punktnotation in eine Hexadezimalzahl um.

```
% ::ip::toHex 1.1.1.0
0x01010100
```

::ip::maskToInt *ipmask*
Dieser Befehl wandelt eine IP-Maske entweder im Punktnotationsformat (255.255.255.0) oder im Längenformat (24) in eine Ganzzahl um.

```
::ip::maskToInt 24
4294967040
```

::ip::broadcastAddress *prefix* ?-ipv4?
Dieser Befehl gibt eine Broadcast-Adresse im Punktnotationsformat für das angegebene Routenpräfix *prefix* zurück, entweder in der Form "addr/mask" oder im nativen Format. Das Ergebnis liegt im Punktnotationsformat vor.

```
::ip::broadcastAddress 1.1.1.0/24
1.1.1.255

::ip::broadcastAddress {0x01010100 0xffffff00}
0x010101ff
```

::ip::maskToLength *dottedMask*|*integerMask*|*hexMask* ?-ipv4?
Dieser Befehl wandelt die Punkt- oder Ganzzahlform einer IP-Maske in die Form der Maskenlänge um.

```
::ip::maskToLength 0xffffff00 -ipv4
24

% ::ip::maskToLength 255.255.255.0
24
```

::ip::lengthToMask *maskLength* ?-ipv4?
Dieser Befehl wandelt eine IP-Maske im Format "Maskenlänge" in die Punktnotation um.

```
::ip::lengthToMask 24
255.255.255.0
```

::ip::nextNet *ipaddr ipmask* ?*count*? ?-ipv4?
Dieser Befehl gibt eine IP-Adresse an derselben Position im *count*-ten nachfolgenden Netzwerk zurück. Der Standardwert für *count* lautet **1**.

Die Adresse kann entweder als Ganzzahl oder in Punktnotation angegeben werden. Die Maske kann entweder als Ganzzahl, in Punktnotation oder in Form der Maskenlänge angegeben werden.

Das Ergebnis wird im Hexadezimalformat ausgegeben.

::ip::isOverlap *prefix prefix...*
Dieser Befehl prüft, ob sich die angegebenen IP-Präfixe überschneiden. Alle Argumente liegen im Format "Adresse/Maske" mit Punkttrennung vor. Alle Argumente nach dem ersten Präfix werden mit dem ersten Präfix verglichen. Das Ergebnis ist ein boolescher Wert. Er ist wahr, wenn bei einem der Präfixe eine Überschneidung festgestellt wurde.

```
% ::ip::isOverlap 1.1.1.0/24 2.1.0.1/32
0

::ip::isOverlap 1.1.1.0/24 2.1.0.1/32 1.1.1.1/32
1
```

::ip::isOverlapNative ?-all? ?-inline? ?-ipv4? *hexipaddr hexipmask hexiplist*
Dieser Befehl ähnelt dem Befehl **::ip::isOverlap**, allerdings liegen die Argumente in ihrer nativen Form vor, und die Form des Ergebnisses lässt sich vom Aufrufer besser steuern. Wenn die Option **-all** festgelegt wird, werden alle Adressen auf Überschneidungen überprüft, nicht nur bis zur ersten gefundenen Adresse. Wenn die Option **-inline** festgelegt wird, gibt der Befehl anstelle von Index-Werten das überlappende Präfix zurück.

Das Ergebnis des Befehls ist, abhängig von den angegebenen Optionen,
no options
Der Index der ersten gefundenen Überlappung oder 0, falls keine vorhanden ist.
-all
Eine Liste, die die Indizes aller gefundenen Überschneidungen enthält, oder eine leere

Liste, falls keine vorhanden sind.

-inline
Das erste überlappende Präfix oder, wenn kein solches vorhanden ist, eine leere Zeichenkette.

-all -inline
Eine Liste, die die Präfixe aller gefundenen Überschneidungen enthält, oder eine leere Liste, falls keine vorhanden sind.

```
% ::ip::isOverlapNative 0x01010100 0xffffff00 {{0x02010001
  0xffffffff}}
0

% ::ip::isOverlapNative 0x01010100 0xffffff00 {{0x02010001
  0xffffffff} {0x01010101 0xffffffff}}
2
```

::ip::ipToLayer2Multicast *ipaddr*
Dieser Befehl wandelt eine IPv4-Adresse in Punktnotation in eine Layer-2-Multicast-Adresse um, die ebenfalls in Punktnotation vorliegt.

```
% ::ip::ipToLayer2Multicast 224.0.0.2
01.00.5e.00.00.02
```

::ip::ipHostFromPrefix *prefix* ?-exclude *prefixExcludeList*?
Dieser Befehl gibt eine Hostadresse aus einem Präfix im Format "ipaddr/masklen" zurück und stellt dabei sicher, dass das Ergebnis keine Adresse ist, die in der *prefixExcludeList* enthalten ist. Das Ergebnis ist eine IP-Adresse im Punktnotationsformat.

```
%::ip::ipHostFromPrefix  1.1.1.5/24
1.1.1.1

%::ip::ipHostFromPrefix  1.1.1.1/32
1.1.1.1
```

::ip::reduceToAggregates *prefixlist*
Dieser Befehl ermittelt sich überschneidende Netzwerke und filtert die spezifischeren Netzwerke heraus. Die Präfixe liegen entweder im Addr/Mask-Format oder im nativen Format vor. Das Ergebnis ist eine Liste, die die sich nicht überschneidenden IP-Präfixe aus der Eingabe enthält.

```
% ::ip::reduceToAggregates {1.1.1.0/24 1.1.0.0/8  2.1.1.0/24
  1.1.1.1/32 }
1.0.0.0/8 2.1.1.0/24
```

::ip::longestPrefixMatch *ipaddr prefixlist* ?-ipv4?
Dieser Befehl ermittelt bei Angabe einer bestimmten Hostadresse das längste übereinstimmende Präfix aus einer Reihe von Präfixen. Die Präfixe in der Liste liegen entweder in nativer Form oder in der mit Punkten getrennten Form vor, während die

Hostadresse entweder im ipprefix-Format, in der mit Punkten getrennten Form oder als Ganzzahl vorliegt. Das Ergebnis ist das Präfix, das am genauesten mit der Hostadresse übereinstimmt.

```
% ::ip::longestPrefixMatch 1.1.1.1 {1.1.1.0/24 1.0.0.0/8
  2.1.1.0/24 1.1.1.0/28 }
1.1.1.0/28
```

::ip::collapse *prefixlist*

Dieser Befehl nimmt eine Liste von Präfixen entgegen und gibt eine Liste von Präfixen mit den größtmöglichen Subnetzmasken zurück, die die Eingabe abdecken, wodurch benachbarte Präfixe zu größeren Bereichen zusammengefasst werden.

Dies unterscheidet sich von **::ip::reduceToAggregates** darin, dass letzterer lediglich bestimmte Netzwerke aus einer Liste entfernt, wenn diese von anderen Elementen der Eingabe abgedeckt werden, während dieser Befehl Netzwerke aktiv zu größeren Bereichen zusammenfasst, wenn sie aneinandergrenzen.

```
% ::ip::collapse {1.2.2.0/24 1.2.3.0/24}
1.2.2.0/23
```

::ip::subtract *prefixlist*

Dieser Befehl erwartet eine Liste von Präfixen, von denen einige mit einem Bindestrich beginnen. Diese *negativen* Präfixe dienen dazu, Ausnahmen in den durch die anderen, *positiven* Präfixe beschriebenen Bereichen zu definieren. Das heißt, die negativen Präfixe werden von den positiven abgezogen, was zu einer umfangreicheren Liste von Beschreibungen führt, die die abgedeckten Bereiche ausschließlich als positive Werte beschreiben.

Beispiele

```
% ip::version ::1
6
% ip::version 127.0.0.1
4
```

```
% ip::normalize 127/8
127.0.0.0/8
% ip::contract 192.168.0.0
192.168
%
% ip::normalize fec0::1
fec0:0000:0000:0000:0000:0000:0000:0001
% ip::contract fec0:0000:0000:0000:0000:0000:0000:0001
fec0::1
```

```
% ip::equal 192.168.0.4/16 192.168.0.0/16
1
% ip::equal fec0::1/10 fec0::fe01/10
1
```

Literatur

1. Postel, J. "Internet Protocol". RFC 791, September 1981, (http://www.ietf.org/rfc/rfc791.txt)
2. Hinden, R. und Deering, S., "Internet Protocol Version 6 (IPv6) Addressing Architecture", RFC 3513, April 2003 (http://www.ietf.org/rfc/rfc3513.txt)

Autoren

Pat Thoyts

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *dns* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

```
Copyright (c) 2004, Pat Thoyts
Copyright (c) 2005 Aamer Akhter <aakhter@cisco.com>
```

docstrip

docstrip

Stichwörter: docstrip, .dtx, LaTeX, documentation, literate programming, source

Name

docstrip – Extraktion von Quellcode im Docstrip-Stil

Übersicht

package require Tcl 8.5 9
package require docstrip ?**1.3**?
docstrip::extract *text terminals* ?*option value* ...?
docstrip::sourcefrom *filename terminals* ?*option value* ...?

Beschreibung

Docstrip ist ein Tool, das zur Unterstützung einer Form der "Literate Programming" entwickelt

wurde. Es ist vor allem in der (La)TeX-Community verbreitet, wo es für nahezu alle Bereiche vom LaTeX-Kern aufwärts eingesetzt wird; es gibt jedoch keinen Grund, warum **docstrip** nicht auch für andere Arten von Software verwendet werden könnte.

Kurz gesagt besteht das Grundprinzip der literaten Programmierung darin, dass der Programmquellcode in erster Linie so geschrieben und strukturiert sein sollte, dass er den Entwicklern (und fortgeschrittenen Anwendern, die einen Blick "unter die Haube" werfen möchten) entgegenkommt, und nicht den Launen eines Compilers oder eines entsprechenden Verwenders des Quellcodes. Das bedeutet, dass literate Quelltexte oft eine Art "Übersetzung" in eine illiterate Form benötigen, die auch "dumme" Software verstehen kann. Das Tcl-Paket **docstrip** übernimmt diese Übersetzung.

Selbst für diejenigen, die sich nicht uneingeschränkt der Philosophie hinter der "Literate Programming" anschließen, kann **docstrip** insbesondere in folgenden Bereichen für mehr Klarheit sorgen:

- Programme, die nicht offensichtliche mathematische Konzepte nutzen
- Projekte, bei denen einzelne Codeabschnitte – möglicherweise in verschiedenen Sprachen – eng aufeinander abgestimmt werden müssen.

Zum einen, weil dadurch Zugriff auf weitaus leistungsfähigere Formatierungsfunktionen für Quellcode-Kommentare geboten wird, als dies im reinen Text möglich ist. Zum anderen, weil alle einzelnen Codeabschnitte nebeneinander in derselben Quelldatei zusammengefasst werden können.

Das funktioniert so, dass der Programmierer direkt nur eine oder mehrere "Master"-Quellcode-Dateien bearbeitet, aus denen **docstrip** dann die traditionelleren "Quell"-Dateien generiert, die Compiler oder ähnliche Programme erwarten würden. Die Master-Quelldateien enthalten in der Regel eine umfangreiche Dokumentation des Codes, manchmal sogar an Plätzen, an denen die Code-Nutzer keinerlei Kommentare zulassen würden. Die Etymologie von "docstrip" liegt darin, dass diese *doc*umentation *strip*ped away wurde (obwohl "Code-Extraktion" möglicherweise eine bessere Beschreibung wäre, da es stets darum ging, ausgewählte Teile der Master-Quelldatei zu kopieren, anstatt Text daraus zu löschen). Das Tcl-Paket **docstrip** enthält eine Neuimplementierung der grundlegenden Extraktionsfunktionalität aus dem Programm **docstrip** und ermöglicht es somit einem Tcl-Interpreter, die Master-Quelldateien direkt zu lesen und zu interpretieren.

Leser, die bisher noch nicht mit **docstrip** vertraut sind, aber mehr darüber erfahren möchten, können die folgenden Quellen zu Rate ziehen.

1. *The tclldoc package and class*, http://ctan.org/tex-archive/macros/latex/contrib/tclldoc/.
2. *The DocStrip utility*, http://ctan.org/tex-archive/macros/latex/base/docstrip.dtx.
3. *The doc and shortvrb Packages*, http://ctan.org/tex-archive/macros/latex/base/doc.dtx.
4. Kapitel 14 aus *The LaTeX Companion* (2. Auflage), Addison-Wesley, 2004; ISBN 0-201-36299-6.

Dateiformat

Die Grundeinheit, mit der **docstrip** arbeitet, sind die *lines* einer Master-Quelldatei. Die Extraktion besteht darin, einige dieser Zeilen auszuwählen, die vom Eingabetext in den Ausgabetext kopiert werden sollen. Die grundlegende Unterscheidung erfolgt zwischen *Codezeilen* (die kopiert werden und nicht mit einem Prozentzeichen beginnen) und *Kommentarzeilen* (die mit einem Prozentzeichen beginnen und nicht kopiert werden).

```
docstrip::extract [join {
  {% comment}
  {% more comment !"#$%&/(}
  {some command}
  { % blah $blah "Not a comment."}
  {% abc; this is comment}
  {# def; this is code}
  {ghi}
  {% jkl}
} \n] {}
```

gibt dieselbe Zeilenfolge zurück wie

```
join {
  {some command}
  { % blah $blah "Not a comment."}
  {# def; this is code}
  {ghi} ""
} \n
```

Für **docstrip** spielt es keine Rolle, welches Format für die Dokumentation in den Kommentarzeilen verwendet wird; um jedoch mehr zu erreichen als reine Textkommentare, wird in der Regel eine Markup-Sprache verwendet. Am häufigsten wird LaTeX verwendet, da es sich um einen sehr etablierten Standard handelt und zudem die beste Unterstützung für mathematische Formeln bietet; das **docstrip::util**-Paket bietet jedoch auch eine gewisse Unterstützung für *doctools*-like-Markup.

Neben den normalen Code- und Kommentarzeilen gibt es auch *Guard-Zeilen*, die mit den beiden Zeichen "%<" beginnen, sowie *Meta-Kommentarzeilen*, die mit den beiden Zeichen "\'%%" beginnen. Innerhalb der Guard-Zeilen wird zudem zwischen *Verbatim-Guard-Zeilen*, die mit "%<<" beginnen, und gewöhnlichen Guard-Zeilen unterschieden, bei denen auf "%<" kein weiteres "<" folgt. Die letztgenannte Kategorie ist bei weitem am häufigsten anzutreffen.

Gewöhnliche Schutzzeilen bedingen die Einbindung der von ihnen geschützten Codezeile(n) vom Wert eines booleschen Ausdrucks; der geschützte Block von Codezeilen wird nur dann eingebunden, wenn die Auswertung des Ausdrucks den Wert "wahr" ergibt. Die Syntax einer gewöhnlichen Schutzzeile lautet wie folgt:

```
'%' '<' STARSLASH EXPRESSION '>'
'%' '<' PLUSMINUS EXPRESSION '>' CODE
```

wobei

```
STARSLASH  ::=  '*' | '/'
PLUSMINUS  ::=  | '+' | '-'
EXPRESSION ::= SECONDARY | SECONDARY ',' EXPRESSION
             | SECONDARY '|' EXPRESSION
SECONDARY  ::= PRIMARY | PRIMARY '&' SECONDARY
PRIMARY    ::= TERMINAL | '!' PRIMARY | '(' EXPRESSION ')'
CODE       ::= { any character except end-of-line }
```

Das Komma und der senkrechte Strich stehen beide für "oder". Das Und-Zeichen steht für "und". Das Ausrufezeichen steht für "nicht". Ein TERMINAL kann eine beliebige nichtleere Zeichenfolge sein, die weder ">", "&", "|", ein Komma, "(" noch ")" enthält, obwohl das Handbuch **docstrip** etwas restriktiv ist und den ordnungsgemäßen Betrieb nur für Zeichenfolgen aus Buchstaben garantiert (obwohl selbst die LaTeX-Kernquellen in TERMINALs in großem Umfang auch Ziffern verwenden). Das zweite Argument von **docstrip::extract** ist die Liste jener TERMINALs, die als den Wert "true" habend gelten sollen; alle anderen TERMINALs gelten bei der Auswertung von Guard-Ausdrücken als "false".

Im Falle einer "%<*Ausdruck*>"-Abgrenzung umfassen die abgetrennten Zeilen alle Zeilen bis zur nächsten "%</*Ausdruck*>"-Abgrenzung mit demselben *Ausdruck (verglichen als Zeichenfolgen). Die durch solche "*"- und "/"-Begrenzungszeilen abgegrenzten Codeblöcke müssen korrekt verschachtelt sein.

```
   set text [join {
      {begin}
      {%<*foo>}
      {1}
      {%<*bar>}
      {2}
      {%</bar>}
      {%<*!bar>}
      {3}
      {%</!bar>}
      {4}
      {%</foo>}
      {5}
      {%<*bar>}
      {6}
      {%</bar>}
      {end}
   } \n]
   set res [docstrip::extract $text foo]
   append res [docstrip::extract $text {foo bar}]
   append res [docstrip::extract $text bar]
```

setzt $res auf das Ergebnis von

```
   join {
      {begin}
      {1}
      {3}
      {4}
      {5}
      {end}
      {begin}
      {1}
      {2}
```

```
      {4}
      {5}
      {6}
      {end}
      {begin}
      {5}
      {6}
      {end} ""
   } \n
```

In Guard-Zeilen ohne einen Modifikator "*", "/", "+" oder "-" nach dem "\'%<" gilt der Guard nur für den CODE, der auf das ">" in dieser einzelnen Zeile folgt. Ein Modifikator "+" entspricht keinem Modifikator. Ein Modifikator "-" verhält sich wie der Fall ohne Modifikator, jedoch wird der Ausdruck implizit negiert, d. h., der CODE einer Guard-Zeile mit "%<-" wird nur einbezogen, wenn die Auswertung den Wert "false" ergibt.

Metakommentarzeilen sind "Kommentarzeilen, die nicht entfernt werden sollen", sondern wie Codezeilen extrahiert werden; diese werden manchmal für Copyright-Hinweise und ähnliche Inhalte verwendet. Das Präfix "%%" wird jedoch nicht beibehalten, sondern durch den aktuellen Wert **-metaprefix** ersetzt, der üblicherweise auf ein Zeichen (oder eine Zeichenfolge) gesetzt wird, das in der Sprache des zu extrahierenden Codes als "Kommentar bis zum Zeilenende" dient.

```
   set text [join {
      {begin}
      {%<foo> foo}
      {%<+foo>plusfoo}
      {%<-foo>minusfoo}
      {middle}
      {%% some metacomment}
      {%<*foo>}
      {%%another metacomment}
      {%</foo>}
      {end}
   } \n]
   set res [docstrip::extract $text foo -metaprefix {# }]
   append res [docstrip::extract $text bar -metaprefix {#}]
```

setzt $res auf das Ergebnis von

```
   join {
      {begin}
      { foo}
      {plusfoo}
      {middle}
      {#  some metacomment}
      {# another metacomment}
      {end}
```

```
      {begin}
      {minusfoo}
      {middle}
      {# some metacomment}
      {end} ""
   } \n
```

Verbatim-Guards können verwendet werden, um die Interpretation eines Zeilenblocks als Code zu erzwingen, selbst wenn einige dieser Zeilen für "docstrip" wie Zeilen anderer Art aussehen. Ein Verbatim-Guard hat die Form "%<<*END-TAG*", und der Verbatim-Block wird durch die erste Zeile beendet, die genau "%*END-TAG*" lautet.

```
   set text [join {
      {begin}
      {%<*myblock>}
      {some stupid()}
      {   #computer<program>}
      {%<<QQQ-98765}
      {% These three lines are copied verbatim (including percents}
      {%% even if -metaprefix is something different than %%).}
      {%</myblock>}
      {%QQQ-98765}
      {   using*strange@programming<language>}
      {%</myblock>}
      {end}
   } \n]
   set res [docstrip::extract $text myblock -metaprefix {# }]
   append res [docstrip::extract $text {}]
```

setzt $res auf das Ergebnis von

```
   join {
      {begin}
      {some stupid()}
      {   #computer<program>}
      {% These three lines are copied verbatim (including percents}
      {%% even if -metaprefix is something different than %%).}
      {%</myblock>}
      {   using*strange@programming<language>}
      {end}
      {begin}
      {end} ""
   } \n
```

Der Prozess der Verarbeitung von Verbatim-Guards erfolgt auch innerhalb von Zeilenblöcken, die aufgrund eines äußeren Block-Guards nicht kopiert werden.

Ein letzter Aspekt der Syntax von **docstrip** ist, dass die Extraktion an einer Zeile endet, die genau "\endinput" lautet; dies wird häufig verwendet, um das Kopieren zufälliger Leerzeichen

am Dateiende zu vermeiden. Für den unwahrscheinlichen Fall, dass Sie eine solche Codezeile benötigen, können Sie diese mit einem Verbatim-Guard schützen.

Befehle

Das Paket definiert zwei Befehle.

docstrip::extract *text terminals ?option value ...?*

Der Befehl **extract** extrahiert den *text* und gibt die extrahierten Codezeilen als Zeichenkette zurück, wobei jede Zeile mit einem Zeilenumbruch abgeschlossen wird. *terminals* ist die Liste jener Terminale des Schutzausdrucks, deren Auswertung "wahr" sein soll. Die verfügbaren Optionen sind:

-annotate *lines*

Legt fest, dass auf jede extrahierte Zeile im Ergebnis die angegebene Anzahl von Anmerkungszeilen folgen soll. Der Standardwert ist 0. Anmerkungszeilen sind vor allem dann nützlich, wenn die extrahierten Zeilen einer weiteren Transformation unterzogen werden sollen. Eine erste Anmerkungszeile besteht aus einer Liste mit drei Elementen: Zeilentyp, bei der Extraktion entferntes Präfix und bei der Extraktion eingefügtes Präfix. Der Zeilentyp kann einer der folgenden sein: "V" (wörtlich), "M" (Metakommentar), "+" (+ oder keine Modifikator-Schutzzeile), "-" (– Modifikator-Schutzzeile), "." (normale Zeile). Eine zweite Anmerkungszeile ist die Nummer der Quellzeile. Eine dritte Anmerkungszeile ist der aktuelle Stapel der Block-Guards. Die Anforderung von mehr als drei Anmerkungszeilen wird derzeit nicht unterstützt.

-metaprefix *string*

Die Zeichenfolge, durch die das Präfix "%%" einer Metakommentarzeile ersetzt wird. Die Standardeinstellung lautet "%%". Bei Tcl-Code wäre dies in der Regel "#".

-onerror *keyword*

Legt fest, welche Maßnahmen ergriffen werden, wenn ein Formatfehler im zu verarbeitenden *text* erkannt wird. Die Einstellungsmöglichkeiten sind:

ignore

Ignoriert den Fehler und fährt fort, als wäre nichts geschehen.

puts

Schreibt eine Fehlermeldung nach **stderr** und setzt die Verarbeitung fort.

throw

Löst einen Fehler aus. Der Parameter **-errorcode** wird auf eine Liste gesetzt, deren erstes Element **DOCSTRIP** ist, deren zweites Element die Fehlerart angibt und deren drittes Element die Zeilennummer ist, in der der Fehler erkannt wurde. Dies ist die Standardeinstellung.

-trimlines *boolean*

Legt fest, ob *Leerzeichen* am Ende einer Zeile vor dem Prozess der Zeile entfernt werden sollen. Der Standardwert ist "true".

Es sei darauf hingewiesen, dass die *terminals* im Zusammenhang mit dem Programm **docstrip** häufig als "Optionen" bezeichnet werden, da diese festlegen, welche optionalen Codefragmente einbezogen werden sollen.

docstrip::sourcefrom *filename terminals ?option value ...?*

Der Befehl **sourcefrom** ist eine Docstripping-Emulation von **source**. Er öffnet die Datei

filename, liest sie ein, schließt sie, entfernt die Dokumentationsinformationen gemäß den Angaben unter *terminals* und führt die Auswertung des Ergebnisses im lokalen Kontext des Aufrufers durch; während dieser Zeit ist der Wert von **info script** der *filename*. Die Optionen werden an **fconfigure** übergeben, um die Datei vor dem Einlesen ihres Inhalts zu konfigurieren. **-metaprefix** wird auf "#" gesetzt, alle anderen **extract**-Optionen behalten ihre Standardwerte.

Dokumentaufbau

Das Dateiformat (wie oben beschrieben) bestimmt, ob eine Master-Quellcodedatei von **docstrip** korrekt verarbeitet werden kann; der Nutzen des Formats hängt jedoch nicht zuletzt auch davon ab, dass der Code und die Kommentarzeilen zusammen ein wohlgeformtes Dokument bilden.

Ein Dokumentformat, für das keine Tcl-fremde Software erforderlich ist, finden Sie unter dem Befehl **ddt2man** im Paket **docstrip::util**. Es wird empfohlen, Dateien, die dieses Dokumentformat verwenden, die Dateiendung "*.ddt*" zu geben, um sie von den traditionelleren, auf LaTeX basierenden "*.dtx*"-Dateien zu unterscheiden.

Master-Quelldateien mit der Erweiterung "*.dtx*" sind in der Regel so eingerichtet, dass sie direkt von **latex** gesetzt werden können, ohne dass weitere Dateien erforderlich sind. Dies wird dadurch erreicht, dass die Datei mit den folgenden Zeilen beginnt:

```
% \iffalse
%<*driver>
\documentclass{tclldoc}
\begin{document}
\DocInput{filename.dtx}
\end{document}
%</driver>
% \fi
```

oder eine Variante davon. Der Trick besteht darin, dass die Datei zweimal gelesen wird. Nach den üblichen LaTeX-Leseregeln gelten die ersten beiden Zeilen als Kommentare und werden daher ignoriert. Die dritte Zeile ist die Präambel des Dokuments, die vierte Zeile leitet den Dokumenttext ein und die sechste Zeile beendet das Dokument, sodass LaTeX an dieser Stelle stoppt – Nicht-Kommentare unterhalb dieser Stelle in der Datei unterliegen niemals den normalen LaTeX-Leseregeln. Zuvor wird jedoch der Befehl \DocInput in der fünften Zeile verarbeitet, der zwei Dinge bewirkt: Er ändert die Interpretation von "%" von "Kommentar" in "ignoriert" und gibt die im Argument angegebene Datei ein (bei der es sich normalerweise um den Namen der Datei handelt, in der sich der Befehl befindet). Erst bei diesem zweiten Durchlauf, bei dem die Datei gelesen wird, werden die darin enthaltenen Kommentare und der Code gesetzt.

Die Funktion von \iffalse ... \fi besteht darin, beim zweiten Durchlauf die Zeilen zwei bis sieben zu überspringen; dies ähnelt dem Konstrukt "if 0 { ... }" für Blockkommentare im Tcl-Code und ist hier unter anderem deshalb erforderlich, weil der Befehl \documentclass nur einmal ausgeführt werden darf. Die Funktion der -Schutzeinrichtungen besteht darin, zu verhindern, dass dieser kurze LaTeX-Code von **docstrip** extrahiert wird. Insgesamt führt dies dazu, dass die Datei sowohl als LaTeX-Dokument als auch als **docstrip**-Master-Quellcode-Datei fungieren kann.

Es ist nicht zwingend erforderlich, die Dokumentklasse "tclldoc" zu verwenden, doch diese bietet

eine Reihe von Funktionen, die für "*.dtx*"-Dateien mit Tcl-Code sehr praktisch sind. Weitere Informationen hierzu finden Sie in der oben genannten Literatur.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2003–2010 Lars Hellström <Lars dot Hellstrom at
    residenset dot net>
```


docstrip_util
Stichwörter: docstrip_util, .ddt, .dtx, LaTeX, Tcl module, catalogue, diff, docstrip, doctools, documentation, literate programming, module, package indexing, patch, source

Name
docstrip_util – Dienstprogramme im Zusammenhang mit Docstrip

Übersicht
package require Tcl 8.5 9
package require docstrip ?**1.3**?
package require docstrip::util ?**1.3.3**?
pkgProvide *name version terminals*
pkgIndex ?*terminal* ...?
fileoptions ?*option value* ...?
docstrip::util::index_from_catalogue *dir pattern* ?*option value* ...?
docstrip::util::modules_from_catalogue *target source* ?*option value* ...?
docstrip::util::classical_preamble *metaprefix message target* ?*source terminals* ...?
docstrip::util::classical_postamble *metaprefix message target* ?*source terminals* ...?
docstrip::util::packages_provided *text* ?*setup-script*?
docstrip::util::ddt2man *text*
docstrip::util::guards *subcmd text*
docstrip::util::patch *source-var terminals fromtext diff* ?*option value* ...?
docstrip::util::thefile *filename* ?*option value* ...?
docstrip::util::import_unidiff *diff-text* ?*warning-var*?

Beschreibung
Das Paket "**docstrip::util**" dient der Zusammenstellung verschiedener Hilfsprozeduren, die vor allem während der Installation oder der Entwicklungsphase nützlich sind. Es ist vom Basispaket getrennt, um einen Mehraufwand zu vermeiden, wenn dieses zur Kompilierung von "**source** "-Code verwendet wird.

Befehle zur Paketindizierung
Ähnlich wie bei rohen "*.tcl*"-Dateien können die Codezeilen in Docstrip-Quelldateien nach Paketdeklarationen durchsucht und entsprechende Indizes erstellt werden. Eine Komplikation besteht jedoch darin, dass man anhand der Codeblöcke selbst nicht erkennen kann, welche Teile zusammen ein funktionierendes Paket ergeben; normalerweise wäre diese Information in einer

begleitenden "*.ins*"-Datei zu finden, doch das Parsen einer solchen Datei ist keine leichte Aufgabe. Daher führt **docstrip::util** eine alternative Kodierung solcher Informationen in Form eines deklarativen Tcl-Skripts ein: den *Katalog* (des Inhalts einer Quelldatei).

Die folgenden Sonderbefehle stehen innerhalb eines Katalogs zur Verfügung:

pkgProvide *name version terminals*
: Legt fest, dass der Code für ein Paket mit dem Namen *name* und der Version *version* aus jenen Modulen in der Quelldatei besteht, die durch die Liste *terminals* der Terminale des Schutzausdrucks ausgewählt werden. Dieser Code sollte vorzugsweise keinen Befehl **package provide** für das Paket enthalten, da ein solcher durch die Mechanismen zum Laden des Pakets bereitgestellt wird.

pkgIndex ?*terminal* ...?
: Gibt an, dass sich der Code für ein Paket aus jenen Modulen in der Quelldatei zusammensetzt, die durch die aufgeführten *terminal*-Ausdrücke der Schutzklausel ausgewählt werden. Der Name und die Version dieses Pakets werden anhand der in diesem Code gefundenen Befehle **package** und **provide** ermittelt (daher muss ein solcher Befehl darin enthalten sein).

fileoptions ?*option value* ...?
: Legt die **fconfigure**-Optionen fest, die beim Einlesen der Quelle gelten sollen; bei reinen ASCII-Dateien kann dies in der Regel ignoriert werden. Wenn die Datei jedoch gemäß einer anderen **-encoding**-Einstellung interpretiert werden muss, kann dies auf diese Weise festgelegt werden. Der Befehl sollte normalerweise an erster Stelle im Katalog stehen, da er nur für die darauf folgenden Befehle wirksam ist.

Andere Tcl-Befehle werden ebenfalls unterstützt – ein Katalog wird durch Auswertung in einem sicheren Interpreter analysiert –, doch sind diese selten erforderlich. Um zukünftige Erweiterungen zu ermöglichen, werden unbekannte Befehle im Katalog stillschweigend ignoriert.

Um die Verteilung von Katalogen zusammen mit ihren Quelldateien zu vereinfachen, wird der Katalog *in der Quelldatei selbst* als Modul gespeichert, das durch das Terminal "**docstrip.tcl::catalogue**" ausgewählt wird. Dies unterstützt sowohl die Vorgehensweise, alle Katalogzeilen an einer Stelle zu sammeln, als auch die Vorgehensweise, jede Katalogzeile in unmittelbarer Nähe des Codes zu platzieren, den sie deklariert.

Die Platzierung von Katalogeinträgen neben dem Code, den sie deklarieren, könnte wie folgt aussehen

```
%    First there's the catalogue entry
%    \begin{tcl}
%<docstrip.tcl::catalogue>pkgProvide foo::bar 1.0 {foobar load}
%    \end{tcl}
%    second a metacomment used to include a copyright message
%    \begin{macrocode}
%<*foobar>
%% This file is placed in the public domain.
%    \end{macrocode}
%    third the package implementation
%    \begin{tcl}
namespace eval foo::bar {
   # ... some clever piece of Tcl code elided ...
```

```
%    \end{tcl}
%    which at some point may have variant code to make use of a
%    |load|able extension
%    \begin{tcl}
%<*load>
   load [file rootname [info script]][info sharedlibextension]
%</load>
%<*!load>
   # ... even more clever scripted counterpart of the extension
   # also elided ...
%</!load>
}
%</foobar>
%    \end{tcl}
%    and that's it!
```

Die entsprechende Konfiguration mit **pkgIndex** würde wie folgt lauten:

```
%    First there's the catalogue entry
%    \begin{tcl}
%<docstrip.tcl::catalogue>pkgIndex foobar load
%    \end{tcl}
%    second a metacomment used to include a copyright message
%    \begin{tcl}
%<*foobar>
%% This file is placed in the public domain.
%    \end{tcl}
%    third the package implementation
%    \begin{tcl}
package provide foo::bar 1.0
namespace eval foo::bar {
   # ... some clever piece of Tcl code elided ...
%    \end{tcl}
%    which at some point may have variant code to make use of a
%    |load|able extension
%    \begin{tcl}
%<*load>
   load [file rootname [info script]][info sharedlibextension]
%</load>
%<*!load>
   # ... even more clever scripted counterpart of the extension
   # also elided ...
%</!load>
}
%</foobar>
%    \end{tcl}
%    and that's it!
```

docstrip::util::index_from_catalogue *dir pattern ?option value ...?*

Dieser Befehl ist dem Standardbefehl **pkg_mkIndex** insofern ähnlich, als er Paketeinträge zu den Dateien "*pkgIndex.tcl*" hinzufügt. Der Unterschied besteht darin, dass er Quelldateien im **docstrip**-Format indexiert und nicht rohe "*.tcl*"-Dateien oder ladbare Bibliotheksdateien. Es werden nur Pakete berücksichtigt, die im Katalog einer Datei aufgeführt sind.

Das Argument *dir* ist das Verzeichnis, in dem nach Dateien gesucht werden soll (und dessen "*pkgIndex.tcl*"-Datei entsprechend angepasst werden sollte). Das Argument *pattern* ist ein Muster im Format **glob** für die zu durchsuchenden Dateien; ein typischer Wert wäre ***.dtx** or ***.{dtx,ddt}**. Die übrigen Argumente sind Option-Wert-Paare, wobei folgende Optionen unterstützt werden:

-recursein *dirpattern*
Wenn diese Option angegeben wird, wird die Operation **index_from_catalogue** in jedem Unterverzeichnis wiederholt, dessen Name mit dem Muster *dirpattern* übereinstimmt. **-recursein *** bewirkt, dass der gesamte Unterverzeichnisbaum mit *dir* als Stammverzeichnis in den Index aufgenommen wird.

-sourceconf *dictionary*
Gibt **fileoptions** an, wenn Sie die Dateikataloge lesen (sowie beim Lesen der Pakete, wenn der Katalog keinen Befehl "**fileoptions**" enthält). Standardmäßig ist das Feld leer. Dies ist vor allem dann nützlich, wenn sich die Kodierung Ihres Systems stark von der der Quelldatei unterscheidet (z. B. wenn es sich bei der einen um eine Zwei-Byte-Kodierung und bei der anderen um eine Ein-Byte-Kodierung handelt). **ascii** und **utf-8** unterscheiden sich in dieser Hinsicht nicht wesentlich.

-options *terminals*
Die *terminals* sind eine Liste von Terminals zusätzlich zu **docstrip.tcl::catalogue**, die beim Extrahieren des Katalogs als "wahr" markiert werden sollen. Standardmäßig ist diese Liste leer. Dies ermöglicht die Verwendung von "Variantenabschnitten" im Katalog selbst, z. B. um bestimmte Einträge mit einem zusätzlichen "experimentellen" Kennzeichen zu kennzeichnen und so zu verhindern, dass sie im Index erscheinen, es sei denn, dieser wird mit "experimentell" in der Liste **-options** generiert.

-report *boolean*
Wenn der *boolean* "true" ist, dann gibt der Programmcode einen textuellen, wahrscheinlich mehrzeiligen Bericht darüber zurück, was ausgeführt wurde. Der Standardwert ist "false"; in diesem Fall gibt es keinen bestimmten Rückgabewert.

-reportcmd *commandPrefix*
Jeder Eintrag im Bericht wird als zusätzliches Argument an das Befehlspräfix übergeben. Da "**index_from_catalogue**" in der Regel auf einer relativ hohen Ebene in Installationsskripten und dergleichen verwendet wird, ist der Standardwert für *commandPrefix* "**puts stdout**". Verwenden Sie **list**, um diese Funktion effektiv zu deaktivieren. Die Werte des Präfixes werden ignoriert.

Die generierten **package ifneeded**-Skripte enthalten einen **package require docstrip** -Befehl und einen docstrip::sourcefrom-Befehl. Wenn es sich bei dem Katalogeintrag um den Typ **pkgProvide** handelt, enthält das Skript **package ifneeded** zudem den Befehl **package provide**.

Hinweis: **index_from_catalogue** entfernt niemals etwas aus einer bestehenden "

pkgIndex.tcl"-Datei. Daher müssen Sie diese möglicherweise löschen (oder von **pkg_mkIndex** von Grund auf neu erstellen lassen), bevor Sie **index_from_catalogue** ausführen, um bestimmte Informationen, wie beispielsweise eine Versionsnummer für ein Paket, zu aktualisieren.

docstrip::util::modules_from_catalogue *target source* ?*option value* ...?

Dieser Befehl stellt eine Alternative zu **index_from_catalogue** dar, der anstelle von "*pkgIndex.tcl*"-Einträgen Tcl-Modul-Dateien ("*.tm*") erstellt. Da diese Aktion eher der klassischen Funktionsweise von **docstrip** entspricht, verfügt sie über Funktionen zum Einfügen von Vor- und Nachworten in die generierten Dateien.

Das Argument *source* ist der Name der Quelldatei, aus der "*.tm*"-Dateien generiert werden sollen. Das Argument *target* ist das Verzeichnis, das als Modulpfad gelten soll, d. h., dies ist der Pfad, an den die aus den Paketnamen abgeleiteten relativen Pfade angehängt werden. Die unterstützten Optionen sind:

-preamble *message*

Eine Meldung, die in die Präambel (den einleitenden Kommentarblock) der generierten Dateien eingefügt werden soll. Standardmäßig ist dies ein Leerzeichen. Es können mehrere Zeilen sein, die dann durch Zeilenumbrüche voneinander getrennt werden. Traditionell wird dies für Copyright-Hinweise oder Ähnliches verwendet, doch Metakommentarzeilen bieten hierfür eine Alternative.

-postamble *message*

Ähnlich wie **-preamble**, jedoch wird die Nachricht am Dateiende statt am Anfang eingefügt. Standardmäßig ist das Feld leer.

-sourceconf *dictionary*

Legt die **fileoptions** fest, die beim Einlesen des Katalogs der *source* verwendet werden (und auch beim Einlesen der Pakete, wenn der Katalog keinen Befehl **fileoptions** enthält). Standardmäßig ist das Feld leer. Dies ist vor allem dann nützlich, wenn sich die Kodierung Ihres Systems stark von der der Quelldatei unterscheidet (z. B. wenn es sich bei der einen um eine Zwei-Byte-Kodierung und bei der anderen um eine Ein-Byte-Kodierung handelt). **ascii** und **utf-8** unterscheiden sich in dieser Hinsicht nicht wesentlich.

-options *terminals*

Die *terminals* ist eine Liste von Terminals zusätzlich zu **docstrip.tcl::catalogue**, die beim Extrahieren des Katalogs als "wahr" behandelt werden sollen. Standardmäßig ist diese Liste leer. Dies ermöglicht die Verwendung von "Variantenabschnitten" im Katalog selbst, z. B. um bestimmte Einträge mit einer zusätzlichen "experimentellen" Schutzklausel abzusichern und somit zu verhindern, dass diese Pakete beitragen, es sei denn, diese werden mit "experimentell" in der Liste **-options** generiert.

-formatpreamble *commandPrefix*

Befehlspräfix, das zur eigentlichen Formatierung der Präambel verwendet wird. Es nimmt vier zusätzliche Argumente entgegen: *message*, *targetFilename*, *sourceFilename* und *terminalList* und gibt eine vollständig formatierte Präambel zurück. Standardmäßig wird **classical_preamble** mit einem *metaprefix* von "##" verwendet.

-formatpostamble *commandPrefix*

Befehlspräfix, das zur eigentlichen Formatierung des Postambels verwendet wird. Es nimmt vier zusätzliche Argumente entgegen: *message*, *targetFilename*, *sourceFilename* und *terminalList* und gibt ein vollständig formatiertes Postambel zurück. Standardmäßig wird

classical_postamble mit einem *metaprefix* von "##" verwendet.

-report *boolean*
Wenn der *boolean* "true" (was die Standardeinstellung ist), so gibt der Programmcode einen textuellen, wahrscheinlich mehrzeiligen Bericht darüber zurück, was ausgeführt wurde. Wenn er "false" ist, gibt es keinen bestimmten Wert.

-reportcmd *commandPrefix*
Jeder Eintrag im Bericht wird als zusätzliches Argument an dieses Befehlspräfix übergeben. Der Standardwert ist **list**, wodurch diese Funktion effektiv deaktiviert wird. Die Rückgabewerte des Präfixes werden ignoriert. Verwenden Sie beispielsweise "**puts stdout** ", um Berichtselemente sofort auf das Terminal auszugeben.

Eine bereits vorhandene Datei, die denselben Namen wie die zu erstellende Datei trägt, wird überschrieben.

docstrip::util::classical_preamble *metaprefix message target* ?*source terminals* ...?
Dieser Befehl gibt eine Einleitung im klassischen **docstrip**-Stil zurück

```
##
## This is `TARGET',
## generated by the docstrip::util package.
##
## The original source files were:
##
## SOURCE (with options: `foo,bar')
##
## Some message line 1
## line2
## line3
```

Wenn ...

```
docstrip::util::classical_preamble {##}\
  "\nSome message line 1\nline2\nline3" TARGET SOURCE {foo bar}
```

Der Befehl unterstützt Präambeln für Dateien, die aus mehreren Quellen generiert wurden, auch wenn **modules_from_catalogue** dies derzeit nicht benötigt.

docstrip::util::classical_postamble *metaprefix message target* ?*source terminals* ...?
Dieser Befehl gibt eine Postamble im klassischen **docstrip**-Stil zurück

```
## Some message line 1
## line2
## line3
##
## End of file `TARGET'.
```

Wenn ...

```
docstrip::util::classical_postamble {##}\
  "Some message line 1\nline2\nline3" TARGET SOURCE {foo bar}
```

Mit anderen Worten: Die Argumente *source* und *terminals* werden ignoriert, werden jedoch aus Gründen der Symmetrie gegenüber **classical_preamble** unterstützt.

docstrip::util::packages_provided *text* ?*setup-script*?

Dieser Befehl gibt eine Liste zurück, in der jedes Element mit geradem Index den Namen eines Pakets **provide**d von *text* enthält, wenn dieses als Tcl-Skript ausgewertet wird, und das darauf folgende Element mit ungeradem Index die entsprechende Version angibt. Er dient zur Indexierung von Paketen mit extrahiertem Code, ähnlich wie bei **pkg_mkIndex**.

Ein Unterschied zu **pkg_mkIndex** besteht darin, dass der *text* in einem sicheren Interpreter ausgewertet wird. Befehle vom Typ **package require** werden stillschweigend ignoriert, ebenso wie unbekannte Befehle (darunter **source** und **load**). Andere Fehler führen dazu, dass der Prozess des *text* abgebrochen wird; in diesem Fall werden nur jene Paketdeklarationen in den Rückgabewert aufgenommen, die vor dem Fehler angetroffen wurden.

Das Argument *setup-script* kann verwendet werden, um die Umgebung der Auswertung anzupassen, wenn der Code in *text* ganz besondere Anforderungen stellt. Das *setup-Skript* wird im lokalen Kontext der Prozedur **packages_provided** unmittelbar vor der Verarbeitung von *text* ausgewertet. Zu diesem Zeitpunkt wird der Name des Slave-Befehls für den sicheren Interpreter, der diese Verarbeitung durchführt, in der lokalen Variablen **c** gespeichert. Um beispielsweise den Inhalt des Arrays **::env** in den sicheren Interpreter zu kopieren, könnte man ein *setup-Skript* mit folgendem Inhalt verwenden:

```
  $c eval [list array set env [array get ::env]]
```

Befehle zur Quelldatenverarbeitung

Im Gegensatz zur vorherigen Befehlsgruppe, die mit dem Befehl docstrip::extract bestimmte Codezeilen extrahierte und diese anschließend weiterverarbeitete, werden die folgenden Befehle auf Text angewendet, der aus Zeilen aller Art besteht.

docstrip::util::ddt2man *text*

Der Befehl **ddt2man** formatiert *text* vom allgemeinen Format **docstrip** in das Format **doctools** "*.man*" (Tcl Markup Language für Manpages) um. Die verschiedenen Zeilentypen werden wie folgt behandelt:

comment and metacomment lines

Die Präfixe "%" und "%%" werden entfernt, der restliche Text bleibt unverändert.

empty lines

Diese werden unverändert beibehalten. (Das bedeutet im Endeffekt, dass sie nach einer Kommentarzeile als Kommentarzeilen und nach einer Codezeile als Codezeilen gezählt werden.)

code lines

Die Befehle **example_begin** und **example_end** werden am Anfang und am Ende jedes Blocks aufeinanderfolgender Codezeilen platziert. Klammern in einer Codezeile werden in die Befehle **lb** und **rb** umgewandelt.

verbatim guards

Diese werden wie gewohnt im Prozess verarbeitet, sodass sie im Ergebnis nicht erscheinen; jede Zeile in einem Verbatim-Block wird jedoch als Codezeile behandelt.

other guards
Diese werden als Codezeilen behandelt, mit der Ausnahme, dass die eigentliche Schutzklausel auf "**emph**" basiert.

Zum Zeitpunkt der Erstellung dieses Dokuments hat noch kein Projekt die **doctools**-Markups in den Master-Quelldateien verwendet, sodass noch keine Erfahrungen darüber vorliegen, was gut funktioniert. Eine Quelldatei könnte jedoch wie folgt aussehen

```
% [manpage_begin gcd n 1.0]
% [keywords divisor]
% [keywords math]
% [moddesc {Greatest Common Divisor}]
% [require gcd [opt 1.0]]
% [description]
%
% [list_begin definitions]
% [call [cmd gcd] [arg a] [arg b]]
%   The [cmd gcd] procedure takes two arguments [arg a] and [arg b]
    which
%   must be integers and returns their greatest common divisor.
proc gcd {a b} {
%   The first step is to take the absolute values of the arguments.
%   This relieves us of having to worry about how signs will be
    treated
%   by the remainder operation.
   set a [expr {abs($a)}]
   set b [expr {abs($b)}]
%   The next line does all of Euclid's algorithm! We can make do
%   without a temporary variable, since $a is substituted before the
%   [lb]set a $b[rb] and thus continues to hold a reference to the
%   "old" value of [var a].
   while {$b>0} { set b [expr { $a % [set a $b] }] }
%   In Tcl 8.3 we might want to use [cmd set] instead of [cmd return]
%   to get the slight advantage of byte-compilation.
%<tcl83>  set a
%<!tcl83>   return $a
}
% [list_end]
%
% [manpage_end]
```

Wenn der obige Text durch **docstrip::util::ddt2man** geleitet wird, ergibt sich eine syntaktisch korrekte **doctools**-Manpage, auch wenn deren Zweck etwas anders ist.

Es wird empfohlen, Master-Quellcode-Dateien mit **doctools**-Markup die Endung "*.ddt*" zu

geben; daher das "ddt" in **ddt2man**.

docstrip::util::guards *subcmd text*

Der Befehl **guards** gibt Informationen (vorwiegend statistischer Art) zu den gewöhnlichen "docstrip"-Schutzelementen zurück, die im *text* vorkommen. Der *subcmd* legt fest, welche Informationen zurückgegeben werden.

counts
Listet die Guard-Ausdrucks-Terminale mit deren Häufigkeiten auf. Das Format des Rückgabewerts ist ein Dictionary, das den Namen des Terminals der Anzahl seiner Vorkommen in der Datei zuordnet.

exprcount
Listet die Guard-Ausdrücke mit der jeweiligen Häufigkeit auf. Das Format des Rückgabewerts ist ein Dictionary, das den Ausdruck der Anzahl seiner Vorkommen in der Datei zuordnet.

exprerr
Listet die syntaktisch fehlerhaften Guard-Ausdrücke auf (z. B. nicht übereinstimmende Klammern oder ein fehlendes Terminal). Der Wert ist eine Liste, deren Elemente in keiner bestimmten Reihenfolge stehen.

expressions
Listet die Guard-Ausdrücke auf. Der Wert, der zurückgegeben wird, ist eine Liste, deren Elemente in keiner bestimmten Reihenfolge angeordnet sind.

exprmods
Listet die Guard-Ausdrücke mit Modifikatoren auf. Das Format des Werts ist ein Dictionary, bei dem jeder Index ein Guard-Ausdruck ist und jeder Eintrag eine Zeichenkette mit einem Zeichen für jede Guard-Zeile, die diesen Ausdruck enthält. Die Zeichen im Eintrag legen fest, welcher Modifikator in dieser Zeile verwendet wurde: +, -, *, / oder (für eine Schutzanweisung ohne Modifikator:) ein Leerzeichen. Dies ist die grundlegendste Form der von **guards** gesammelten Informationen.

names
Gibt die Terminale des Guard-Ausdrucks aus. Der Wert ist eine Liste, deren Elemente in keiner bestimmten Reihenfolge angeordnet sind.

rotten
Listet die fehlerhaften Guard-Zeilen auf (dabei sind jedoch Zeilen, in denen lediglich der Ausdruck fehlerhaft ist, nicht enthalten). Das Format des Werts, der zurückgegeben wird, ist ein Dictionary, das Zeilennummern ihrem Inhalt zuordnet.

docstrip::util::patch *source-var terminals fromtext diff ?option value ...?*

Dieser Befehl versucht, eine **diff**-Datei (beispielsweise einen eingereichten Patch), die für eine generierte Datei berechnet wurde, auf die **docstrip**-Quelldatei anzuwenden. Dies kann nützlich sein, wenn jemand eine generierte Datei bearbeitet und sie dabei fälschlicherweise für die Quelldatei gehalten hat. Dieser Befehl trifft keine spezifischen Annahmen für den Fall, dass es sich bei der generierten Datei um ein Tcl-Skript handelt.

patch setzt voraus, dass die zu patchen Quelldatei als Liste von Zeilen in einer Variablen gespeichert ist, und der Name dieser Variablen im Aufrufkontext wird in das Argument *source-var* übergeben. *terminals* ist die Liste der Terminals, die zum Extrahieren der gepatchten Datei verwendet werden. *diff* ist der tatsächlich anzuwendende Diff (in einem

Format, wie unten erläutert), und *fromtext* ist der Inhalt der Datei, die bei der Berechnung des Diffs als "Ausgangsdatei" diente. Mithilfe von Optionen lässt sich der Prozess weiter steuern.

Der Prozess funktioniert, indem die "Hunks" im *diff* von der generierten in die Quelldatei "übertragen" und anschließend auf die Elemente der *source-var* angewendet werden. Um diese Übertragung durchzuführen, muss festgelegt werden, wie Zeilen im *fromtext* den Elementen der *source-var* entsprechen, und genau hier kommen die *terminals* ins Spiel; Die Quelldatei wird zunächst unter den angegebenen *terminals*extract**extract** verarbeitet, und das Ergebnis dieser Verarbeitung wird anschließend mit dem *fromtext* abgeglichen. Dadurch entsteht eine Zuordnung, die die im *diff* angegebenen Zeilennummern in Elementnummern in *source-var* übersetzt – genau das ist für das Übertragen der Blöcke erforderlich.

Es gibt zwei Gründe, warum sowohl die *terminals* als auch der *fromtext* angegeben werden müssen. Erstens ist es sehr schwierig, den Überblick darüber zu behalten, wie viele Zeilen der Präambel auf andere Weise als durch das Kopieren von Zeilen aus Quelldateien bereitgestellt werden. Zweitens kann eine generierte Datei Material aus mehreren Quelldateien enthalten. Beides macht es unmöglich, vorherzusagen, welche Zeilennummer eine extrahierte Datei in der generierten Datei haben würde; daher sucht der Algorithmus zur Berechnung der Zeilennummernzuordnung stattdessen nach einem Zeilenblock im *fromtext*, der mit dem übereinstimmt, was aus der Quelle extrahiert werden kann. Dieser Abgleich wird durch die folgenden Optionen beeinflusst:

-matching *mode*
: Wie ähnlich müssen zwei Zeilen sein, damit sie übereinstimmen? Die unterstützten *mode* sind:

  exact
  : Zeilen müssen als Zeichenfolgen identisch sein. Dies ist die Standardeinstellung.

  anyspace
  : Alle Folgen von Leerzeichen werden vor dem Vergleich in einzelne Leerzeichen umgewandelt.

  nonspace
  : Wenn der Vergleich durchgeführt wird, werden ausschließlich Zeichen berücksichtigt, die keine Leerzeichen sind.

  none
  : Zwei beliebige Zeilen gelten als gleich.

-metaprefix *string*
: Der Wert "**-metaprefix**", der beim Extrahieren verwendet werden soll. Der Standardwert lautet "%%", doch bei Tcl-Code ist es wahrscheinlicher, dass für die generierte Datei "#" oder "##" verwendet wurde.

-trimlines *boolean*
: Der Wert "**-trimlines**", der beim Extrahieren verwendet werden soll. Der Standardwert ist "true".

Der Wert wird in Form eines "Unified Diff" gegeben und enthält ausschließlich jene Blöcke, die nicht oder nur teilweise übernommen wurden; ein Kommentar im Kopf jedes Blocks legt fest, welcher Fall vorliegt. In der Regel ist es erforderlich, sowohl den Wert von **patch** als auch den gepatchten Text selbst manuell zu überprüfen, da dieser Befehl Kommentarzeilen nicht an

den neuen Inhalt anpassen kann.

Ein Beispiel würde wie folgt aussehen:

```
set sourceL [split [docstrip::util::thefile from.dtx] \n]
set terminals {foo bar baz}
set fromtext [docstrip::util::thefile from.tcl]
set difftext [exec diff --unified from.tcl to.tcl]
set leftover [docstrip::util::patch sourceL $terminals $fromtext\
  [docstrip::util::import_unidiff $difftext] -metaprefix {#}]
set F [open to.dtx w]; puts $F [join $sourceL \n]; close $F
return $leftover
```

Hier wurde "*from.dtx*" als Vorlage für "*from.tcl*" verwendet, das jemand zu "*to.tcl*" umgeschrieben hat. Wir versuchen nun, eine Datei "*to.dtx*" zu erstellen, die als Vorlage für "*to.tcl*" dienen kann.

docstrip::util::thefile *filename* ?*option value* ...?
Der Befehl **thefile** öffnet die Datei *filename*, liest sie bis zum Ende, schließt sie und gibt den Inhalt zurück (wobei ein eventuell vorhandener abschließender Zeilenumbruch entfernt wird). Die Option-Wert-Paare werden an **fconfigure** übergeben, um den geöffneten Dateikanal zu konfigurieren, bevor Daten daraus gelesen werden.

docstrip::util::import_unidiff *diff-text* ?*warning-var*?
Dieser Befehl zerlegt einen Diff im unified-Format (**diff**-Schalter **-U** und **--unified**) in die Liste von Hunks, die **docstrip::util::patch** erwartet. *diff-text* ist der zu zerlegende Text. *warning-var* ist, falls angegeben, der Name einer Variablen im aufrufenden Kontext, an die Warnungen zu Zerlegungsproblemen mit **append** angehängt werden.

Das Ergebnis ist eine Liste von *hunks*. Jeder Hunk ist eine Liste aus fünf Elementen: *start1*, *end1*, *start2*, *end2*, *lines*. *start1* und *end1* sind die Zeilennummern der ersten und der letzten Zeile des Hunks in der Datei "from". *start2* und *end2* sind die entsprechenden Zeilennummern in der Datei "to". Die Zählung beginnt bei 1. *lines* enthält je Zeile des Hunks zwei Elemente: zuerst die Art der Zeile, dann ihren Inhalt. Die Art ist **-** für Zeilen nur in "from", **+** für Zeilen nur in "to" und **0** für Zeilen in beiden.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2003–2010 Lars Hellström <Lars dot Hellstrom at
    residenset dot net>
```


doctools

changelog
Stichwörter: changelog, doctools, emacs

Name

doctools::changelog – Verarbeitung von Text im Emacs-ChangeLog-Format

Übersicht

package require Tcl 8.5 9
package require textutil
package require doctools::changelog ?**1.2**?
::doctools::changelog::scan *text*
::doctools::changelog::flatten *entries*
::doctools::changelog::toDoctools *title module version entries*
::doctools::changelog::merge *entries...*

Beschreibung

Dieses Paket enthält Tcl-Befehle zur Verarbeitung und Neuformatierung von Text im "*ChangeLog* "-Format, das von **emacs** erzeugt wird.

API

::doctools::changelog::scan *text*

Der Befehl nimmt den *text* und parst ihn unter der Annahme, dass er ein von **emacs** erzeugtes ChangeLog enthält. Er gibt eine Datenstruktur zurück, die den Inhalt dieses ChangeLogs beschreibt.

Diese Datenstruktur ist eine Liste, in der jedes Element einen Eintrag im ChangeLog beschreibt. Jedes Element bzw. jeder Eintrag ist wiederum eine Liste aus drei Elementen, die in dieser Reihenfolge das Datum des Eintrags, dessen Autor und die abgegebenen Kommentare beschreiben. Das letzte Element jedes Eintrags, die Kommentare, ist eine Liste von Abschnitten. Jeder Abschnitt wird durch eine Liste beschrieben, die zwei Elemente enthält: eine Liste von Dateinamen und eine Zeichenkette mit dem eigentlichen Kommentar, der zu den Dateien des Abschnitts gehört.

```
{
{
    date
    author
    {
    {
        {file ...}
        commenttext
    }
    ...
    }
}
{...}
}
```

::doctools::changelog::flatten *entries*

Dieser Befehl wandelt eine Liste von Einträgen, wie sie oben von **change::scan** generiert wurde, in eine einfachere Liste aus Klartextblöcken um, die jeweils alle Informationen eines

einzelnen Eintrags enthalten.

::doctools::changelog::toDoctools ***title module version entries***
Dieser Befehl wandelt die vom Befehl **::doctools::changelog::scan** generierten, bereits geparsten *entries* des ChangeLog in ein Dokument im *doctools*-Format um und gibt dieses als Ergebnis des Befehls zurück.

Die anderen drei Argumente liefern die Informationen für den Kopf dieses Dokuments, die im ChangeLog selbst nicht enthalten sind.

::doctools::changelog::merge ***entries...***
Jedes Argument des Befehls wird als vorab geparstes Changelog angesehen, wie es vom Befehl **::doctools::changelog::scan** generiert wurde. Dieser Befehl führt sie alle zu einer einzigen Struktur zusammen und fasst mehrere Einträge mit demselben Datum und demselben Autor zu einem einzigen Eintrag zusammen. Die neue Struktur wird vom Befehl zurückgegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2003-2013 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

cvs

Stichwörter: cvs, changelog, cvs log, emacs, log

Name

doctools::cvs – Verarbeitung von Text im Format "cvs log"

Übersicht

package require Tcl 8.5 9
package require textutil
package require doctools::cvs ?**1.1**?
::doctools::cvs::scanLog *text evar cvar fvar*
::doctools::cvs::toChangeLog *evar cvar fvar*

Beschreibung

Dieses Paket enthält Tcl-Befehle zur Verarbeitung und Neuformatierung von Text in dem durch den Befehl **cvs log** erzeugten Format.

Die Befehle **::doctools::cvs::scanLog** und **::doctools::cvs::toChangeLog** stammen aus Code, der auf der Seite Tcl'ers Wiki zu finden ist. Siehe die Literatur am Ende der Seite.

API

::doctools::cvs::scanLog *text evar cvar fvar*

Der Befehl nimmt den *text* entgegen und wertet ihn unter der Annahme aus, dass er ein von **cvs log** generiertes CVS-Protokoll enthält. Die daraus resultierenden Informationen werden in den Variablen gespeichert, deren Namen über *evar*, *cvar* und *fvar* festgelegt wurden.

Bereits vorhandene Informationen in den referenzierten Variablen bleiben erhalten, sodass der Aufrufer Daten aus mehreren Protokollen in einer Datenbank zusammenführen kann.

varname *evar* (in)

Muss sich auf eine skalare Variable beziehen. Nach dem Aufruf enthält diese Variable eine Liste aller Einträge, die in der Protokolldatei gefunden wurden. Ein Eintrag wird durch die Kombination aus Datum und Autor identifiziert und kann sich auf mehrere physische Einträge verteilen, wobei jeder Eintrag einer berührten Datei entspricht.

Zu beachten ist, dass die Einträge in derselben Reihenfolge aufgelistet werden, in der sie im *text* gefunden wurden. Diese Reihenfolge ist nicht zwangsläufig nach Datum oder Autor sortiert.

Jedes Element der Liste ist eine Liste, die zwei Elemente enthält: das Datum des Eintrags und dessen Autor, in dieser Reihenfolge. Das Datum ist im Format **year**/**month**/**day** formatiert.

varname *cvar* (in)

Muss sich auf eine Array-Variable beziehen. Die Schlüssel sind Zeichenfolgen, die das Datum und den Autor der Protokolleinträge in dieser Reihenfolge enthalten und durch ein Komma getrennt sind.

Die Werte sind Listen mit Kommentaren zu dem jeweiligen Eintrag.

varname *fvar* (in)

Muss sich auf eine Array-Variable beziehen. Die Schlüssel sind Zeichenfolgen, die in dieser Reihenfolge das Datum, den Autor eines Protokolleintrags und einen Kommentar zu diesem Eintrag enthalten und durch Kommas getrennt sind.

Die Werte sind Listen der Dateien, die der Eintrag berührt.

::doctools::cvs::toChangeLog *evar cvar fvar*

Die drei Argumente für diesen Befehl entsprechen den letzten drei Argumenten des Befehls **::doctools::cvs::scanLog**. Dieser Befehl erwartet jedoch, dass diese mit Informationen zu einem oder mehreren Protokollen gefüllt sind. Er nimmt diese Informationen auf und wandelt sie in einen Text im Format eines ChangeLog um, wie er von **emacs** akzeptiert und generiert wird. Der erstellte Text wird als Ergebnis des Befehls zurückgegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2003-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx

Stichwörter: docidx, HTML, TMML, conversion, documentation, index, keyword index, latex, manpage, markdown, markup, nroff, wiki

Name

doctools::idx – docidx – Indizes verarbeiten

Übersicht

package require Tcl 8.5 9
package require doctools::idx ?**1.2.1**?
::doctools::idx::new *objectName* ?*-option value* ...?
::doctools::idx::help
::doctools::idx::search *path*
objectName method ?*arg arg* ...?
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**
objectName **destroy**
objectName **format** *text*
objectName **map** *symbolic actual*
objectName **parameters**
objectName **search** *path*
objectName **setparam** *name value*
objectName **warnings**

Beschreibung

Dieses Paket stellt eine Klasse zur Verfügung, mit der Objekte erstellt werden können, die in der Markup-Sprache *docidx* verfassten Text verarbeiten und in ein beliebiges Ausgabeformat X konvertieren können, für das eine *Formatierungs-Engine* verfügbar ist.

Leser, die sich für die Auszeichnungssprache selbst interessieren, sollten mit der *Einleitung in die Sprache docidx* beginnen und von dort aus zu den formalen Spezifikationen übergehen, d. h. zur *Syntax der Sprache docidx* und zur *Befehlsreferenz der Sprache docidx*.

Sollte der Leser hingegen eine eigene Formatierungs-Engine für ein bestimmtes Format schreiben wollen, d. h. als *Plugin-Entwickler* tätig sein, ist das Lesen und Verstehen der *docidx-Plugin-API-Referenz* unbedingt erforderlich, da dieses Dokument die Interaktion zwischen diesem Paket und seinen Plugins, d. h. den Formatierungs-Engines, im Detail festlegt.

Öffentliche API

Paketbefehle

::doctools::idx::new *objectName* ?-option *value* ...?

Dieser Befehl erstellt ein neues docidx-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Die Optionen und ihre Werte, die auf den Namen des Objekts folgen, dienen dazu, die Anfangskonfiguration des Objekts festzulegen.

::doctools::idx::help

Dies ist ein Komfortbefehl für Anwendungen, die ihren Benutzern eine kurze Beschreibung der verfügbaren Formatierungsbefehle und deren Bedeutung bereitstellen möchten. Er gibt eine Zeichenkette zurück, die einen Standard-Hilfetext enthält.

::doctools::idx::search *path*

Wann immer ein durch dieses Paket erstelltes Objekt den Namen eines Formats der Datei zuordnen muss, die den Code für dessen Formatierungs-Engine enthält, sucht es in einer Reihe von Verzeichnissen nach dieser Datei, die in einer Liste gespeichert sind. Weitere Erläuterungen finden Sie im Abschnitt Formatzuordnung.

Diese Liste enthält nicht nur drei Standardverzeichnisse, die vom Paket selbst deklariert werden, sondern kann auch vom Benutzer des Pakets erweitert werden. Dieser Befehl dient dazu. Wird ein *path* zu einem vorhandenen und lesbaren Verzeichnis angegeben, wird dieses Verzeichnis der Liste der zu durchsuchenden Verzeichnisse vorangestellt. Das bedeutet, dass der zuletzt hinzugefügte *path* später als erstes durchsucht wird.

Es wird ein Fehler ausgelöst, wenn der *path* entweder nicht existiert, kein Verzeichnis ist oder nicht lesbar ist.

Objektbefehl

Alle von **::doctools::idx::new** erstellten Befehle haben die folgende allgemeine Form und können verwendet werden, um verschiedene Operationen auf ihrem "docidx"-Konverterobjekt auszuführen.

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* configure**

Die Methode gibt eine Liste aller bekannten Optionen und deren aktuelle Werte zurück, wenn sie ohne Argumente aufgerufen wird.

objectName* configure *option

Die Methode verhält sich wie die Methode **cget**, wenn sie mit einem einzigen Argument aufgerufen wird, und gibt den Wert der durch dieses Argument festgelegten Option zurück.

***objectName* configure -option *value*...**

Die Methode konfiguriert die angegebenen **option**s des Objekts neu und setzt sie auf die zugehörigen *value*s, wenn sie mit einer geraden Anzahl von Argumenten (mindestens zwei) aufgerufen wird.

Die zulässigen Optionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* cget -option**

Diese Methode erwartet eine zulässige Konfigurationsoption als Argument und gibt den aktuellen Wert dieser Option für das Objekt zurück, für das die Methode aufgerufen wurde.

Die zulässigen Konfigurationsoptionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

objectName* format *text

Diese Methode verarbeitet den *text* mithilfe der konfigurierten Formatierungs-Engine und gibt die generierte Zeichenfolge als Ergebnis zurück. Es wird ein Fehler ausgelöst, wenn für das Objekt keine Konfiguration von **-format** vorhanden ist.

Die Methode geht davon aus, dass der *text* im *docidx*-Format vorliegt, wie im Begleitdokument *docidx_fmt* festgelegt. Andernfalls werden Fehler ausgelöst.

objectName* map *symbolic actual

Diese Methode fügt dem objektbezogenen Zuordnungsschema von *symbolic* Dateinamen zu den *actual* URIs einen Eintrag hinzu. Das Objekt speichert lediglich diese Zuordnung und stellt sie der konfigurierten Formatierungs-Engine über den Befehl **dt_fmap** zur Verfügung. Dieser Befehl wird ausführlicher in der *docidx-Plugin-API-Referenz* beschrieben, die die Interaktion zwischen den von diesem Paket erstellten Objekten und den Index-Formatierungs-Engines festlegt.

***objectName* parameters**

Diese Methode gibt eine Liste zurück, die die Namen aller von der konfigurierten Formatierungs-Engine bereitgestellten Parameter enthält. Sie gibt eine leere Liste zurück, wenn das Objekt noch nicht für ein bestimmtes Format konfiguriert ist.

objectName* search *path

Diese Methode erweitert die objektbezogene Liste der Pfade, die von den Indexformatierungs-Engines durchsucht werden. Siehe auch den Befehl **::doctools::idx::search** zur Erweiterung der paketbezogenen Pfadliste. Beachten Sie, dass der zuletzt eingegebene Pfad zuerst durchsucht wird. Weitere Einzelheiten finden Sie im Abschnitt Formatzuordnung.

objectName* setparam *name value

Diese Methode setzt den mit *name* bezeichneten Engine-Parameter auf den angegebenen *value*. Es wird ein Fehler ausgelöst, wenn das Objekt entweder noch nicht für ein bestimmtes Format konfiguriert ist oder wenn die Formatierungs-Engine für das konfigurierte Format keinen Parameter mit dem angegebenen *name* bereitstellt. Die Liste der von der konfigurierten Formatierungs-Engine bereitgestellten Parameter kann über die Methode **parameters** abgerufen werden.

***objectName* warnings**

Diese Methode gibt eine Liste zurück, die alle Warnungen enthält, die von der konfigurierten Formatierungs-Engine beim letzten Aufruf der Methode **format** erzeugt wurden.

Objekt-Konfiguration

Alle docidx-Objekte unterstützen die folgenden Konfigurationsoptionen:

-file *file*

Das Argument dieser Option wird im Objekt gespeichert und der konfigurierten Formatierungs-Engine über den Befehl **dt_file** zur Verfügung gestellt. Dieser Befehl wird im Begleitdokument *docidx_api* näher beschrieben, das die API zwischen dem Objekt und den Formatierungs-Engines festlegt.

Der Standardwert dieser Option ist die leere Zeichenkette.

Die konfigurierte Formatierungs-Engine sollte den Wert als Namen der Datei interpretieren, die das aktuell verarbeitete Dokument enthält.

-format *text*

Das Argument dieser Option legt das zu erzeugende Format fest und damit implizit auch die Formatierungs-Engine, die bei der Konvertierung von Text über die Methode **format** verwendet werden soll. Der Standardwert ist die leere Zeichenkette. Die Methode **format** kann nicht verwendet werden, wenn diese Option nicht mindestens einmal auf einen gültigen Wert gesetzt wurde.

Das Paket versucht umgehend, den angegebenen Namen einer Datei zuzuordnen, die den Code für eine Formatierungs-Engine enthält, welche dieses Format erzeugt. Sollte diese Zuordnung fehlschlagen, wird ein Fehler ausgelöst. In diesem Fall bleibt ein zuvor konfiguriertes Format unverändert.

Im Abschnitt Formatzuordnung wird detailliert erläutert, wie das Paket und das Objekt nach Engine-Implementierungen suchen.

Formatzuordnung

Das Paket und das Objekt führen den folgenden Algorithmus aus, wenn versucht wird, einen Formatnamen *foo* einer Datei zuzuordnen, die eine Implementierung einer Formatierungs-Engine für *foo* enthält:

1. Ist *foo* der Name einer vorhandenen Datei, so wird diese Datei direkt als Implementierung herangezogen.
2. Wenn dies nicht der Fall ist, wird die Liste der objektbezogenen Suchpfade durchsucht. Für jedes Verzeichnis in der Liste prüft das Paket, ob dieses Verzeichnis eine Datei "*idx.foo*" enthält. Ist dies der Fall, wird diese Datei als Implementierung herangezogen.

Hinweise: Diese Liste von Pfaden ist zunächst leer und kann über die Objektmethode **search** erweitert werden.

3. Ist dies nicht der Fall, wird die Liste der Paketpfade durchsucht. Für jedes Verzeichnis in der Liste prüft das Paket, ob dieses Verzeichnis eine Datei "*idx.foo*" enthält. Ist dies der Fall, wird diese Datei als Implementierung herangezogen.

   Diese Liste der Pfade kann über den Befehl **::doctools::idx::search** erweitert werden. Sie enthält zunächst einen Pfad, nämlich das Unterverzeichnis "*mpformats*" des Verzeichnisses, in dem sich das Paket selbst befindet. Mit anderen Worten: Wenn die Paketimplementierung "*docidx.tcl*" im Verzeichnis "*/usr/local/lib/tcllib/doctools*" installiert ist, wird standardmäßig das Verzeichnis "*/usr/local/lib/tcllib/doctools/mpformats*" nach Formatimplementierungen durchsucht.
4. Die Zuordnung schlägt fehl.

Vordefinierte Engines

Das Paket stellt vordefinierte Formatierungs-Engines für die folgenden Formate bereit. Einige der Formatierungs-Engines unterstützen Engine-Parameter. Diese werden ausdrücklich hervorgehoben.

html

Diese Engine erzeugt HTML-Markup zur Verarbeitung durch Webbrowser und Ähnliches. Diese Engine unterstützt drei Parameter:

footer

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den "body"-Abschnitt eines HTML-Dokuments sein. Der Standardwert ist die leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar vor dem Tag "" eingefügt und schließt damit den "body"-Abschnitt des generierten HTML-Codes ab.

Dies kann genutzt werden, um Standard-Fußzeilen-Markup in das generierte Dokument einzufügen.

header

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den "body"-Abschnitt eines HTML-Dokuments sein. Der Standardwert ist die leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar nach dem -Tag eingefügt und bildet den Beginn des Hauptteils des generierten HTML-Codes.

Dies kann genutzt werden, um Standard-Kopfzeilen-Markup in das generierte Dokument einzufügen.

meta

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den Kopfbereich eines HTML-Dokuments sein. Der Standardwert ist eine leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar nach dem -Tag eingefügt und bildet den Beginn des Kopfbereichs des generierten HTML-Dokuments.

Dies kann genutzt werden, um Standard-Metadaten-Markup in das generierte Dokument einzufügen, wie beispielsweise Verweise auf ein Stylesheet, Standard-Meta-Schlüsselwörter usw.

latex
: Diese Engine erzeugt eine Ausgabe, die für den Textprozessor **latex** aus der TeX-Welt geeignet ist.

list
: Diese Engine extrahiert Version, Abschnitt und Titel der Manpage aus dem Dokument. Damit lässt sich eine Verzeichnisliste für eine Reihe von Manpages erzeugen.

markdown
: Diese Engine generiert *Markdown*-Markup.

nroff
: Diese Engine erzeugt eine nroff-Ausgabe, die von **nroff** oder **groff** verarbeitet wird. Das Ergebnis sind Standard-Manpages, wie sie in der Unix-Welt bekannt sind.

null
: Diese Engine erzeugt keinerlei Ausgabe. Das kann genutzt werden, wenn lediglich eine Eingabe validiert werden soll.

tmml
: Diese Engine generiert TMML-Markup gemäß der Festlegung von Joe English. Die Tcl Manpage Markup Language ist eine Ableitung von XML.

wiki
: Diese Engine generiert Wiki-Markup, wie es Jean Claude Wipplers Anwendung **wikit** versteht.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2003-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_intro

Stichwörter: docidx_intro, index, keyword index, markup, semantic markup

Name

docidx_intro – Einleitung in docidx

Beschreibung

docidx (kurz für *documentation index*) bezeichnet eine Reihe miteinander verbundener, aber dennoch unterschiedlicher Komponenten, die zusammenwirken, um die Erstellung und

Transformation von Schlüsselwortindizes für Dokumentation zu vereinfachen. Dabei handelt es sich um

1. Eine auf Tcl basierende Sprache zur semantischen Auszeichnung eines Schlüsselwortindexes. Die Auszeichnung wird durch Tcl-Befehle dargestellt.
2. Ein Paket, das die Möglichkeit bietet, Texte zu lesen und zu transformieren, die in dieser Auszeichnungssprache verfasst sind. Es ist wichtig zu beachten, dass die eigentliche Transformation des Eingabetextes an Plugins weitergeleitet wird.
3. Eine API, die die Schnittstelle zwischen dem oben genannten Paket und einem Plugin beschreibt.

Welche der ausführlicheren Dokumente für den Leser dieser Einleitung relevant sind, hängt von seiner Rolle im Dokumentationsprozess ab.

1. Ein *Autor* von Dokumentationen muss die Auszeichnungssprache selbst verstehen. Ein Anfänger im Umgang mit docidx sollte zunächst die eher informell verfasste *Einleitung in die docidx-Sprache* lesen. Nachdem er diese verinnerlicht hat, sollte die formale Spezifikation der *docidx-Sprachsyntax* verständlich werden. Ein mit docidx erfahrener Autor benötigt möglicherweise nur gelegentlich die *docidx-Befehlsreferenz*, um sein Gedächtnis aufzufrischen.

   Während ein Dokument verfasst wird, kann die Anwendung **dtp** zur Validierung herangezogen werden; nach Fertigstellung führt sie zudem die Umwandlung in das gewählte System visueller Auszeichnung durch, sei es *roff, HTML, Klartext, Wiki usw. Die einfachere Anwendung **dtplite** verwendet docidx intern, wenn sie Verzeichnisse mit Dokumentation bearbeitet, und erzeugt für sie automatisch einen passenden Schlüsselwortindex.
2. Ein *Verarbeiter* von Dokumentation, die in der Auszeichnungssprache *docidx* verfasst ist, muss wissen, welche Werkzeuge zur Verfügung stehen.

   Das wichtigste Werkzeug ist die bereits erwähnte Anwendung **dtp**, die von Tcllib bereitgestellt wird. Die einfachere Anwendung **dtplite** zeigt dem Benutzer docidx nicht. Auf der untersten Ebene, die beiden Anwendungen gemeinsam ist, befindet sich jedoch das Paket **doctoools::idx**, das die grundlegenden Funktionen zum Lesen und Verarbeiten von Dateien bereitstellt, die Text im docidx-Format enthalten.
3. Nicht zuletzt müssen *Plugin-Entwickler* die Wechselwirkung zwischen dem Paket **doctools::idx** und dessen Plugins verstehen, wie in der *docidx-Plugin-API-Referenz* beschrieben.

Verwandte Formate

docidx steht nicht für sich allein, sondern hat zwei begleitende Formate. Diese heißen *doctoc* und *doctools* und dienen jeweils zur Auszeichnung von *Inhaltsverzeichnissen* bzw. allgemeiner Dokumentation. Sie werden in eigenen Dokumentensammlungen beschrieben, beginnend jeweils mit der *doctoc-Einleitung* und der *doctools-Einleitung*.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_cmdref
Stichwörter: docidx_lang_cmdref, docidx commands, docidx language, docidx markup, markup, semantic markup

Name
docidx_lang_cmdref – docidx-Sprachbefehlsreferenz

Übersicht
comment *plaintext*
include *filename*
index_begin *text title*
index_end
key *text*
lb
manpage *file text*
rb
url *url label*
vset *varname value*
vset *varname*

Beschreibung
Dieses Dokument legt sowohl die Namen als auch die Syntax aller Befehle fest, die zusammen die Markup-Sprache "docidx", Version 1, bilden. Da dieses Dokument als Nachschlagewerk gedacht ist, sind die Befehle in alphabetischer Reihenfolge aufgeführt, und die Beschreibungen sind relativ kurz gehalten. Anfängern wird empfohlen, zunächst die wesentlich informeller verfasste *Einleitung in die Sprache docidx* zu lesen.

Befehle
comment *plaintext*
Indexauszeichnung. Der Text des Arguments wird als Kommentar markiert, der außerhalb des eigentlichen Dokumenttextes steht. Die Hauptverwendung liegt bei freiformigem Text.

include *filename*
Vorlagen. Der Inhalt der genannten Datei wird als Text im docidx-Markup interpretiert und anstelle des Include-Befehls verarbeitet. Das Markup in der Datei muss in sich geschlossen sein. Ein Markup-Befehl kann die Dateigrenzen nicht überschreiten.

index_begin *text title*
Dokumentstruktur. Der Befehl zum Beginnen eines Indexes. Die Argumente sind eine Bezeichnung für die gesamte Gruppe von Dokumenten, auf die sich der Index bezieht (*text*), sowie der übergreifende Titeltext für den Index (*title*), ohne Markup.

Die Bezeichnung ist häufig der Name des Pakets (oder der Erweiterung), zu dem die Dokumente gehören.

index_end
Dokumentstruktur. Befehl zum Beenden eines Indexes. Alles, was in diesem Dokument nach diesem Befehl folgt, ist ein Fehler.

key *text*
Indexstruktur. Dieser Befehl fügt das Schlüsselwort *text* in den Index ein.

lb
Text. Der Befehl wird durch eine öffnende eckige Klammer ersetzt. Verwendung in freiformigem Text. Erforderlich, um zu verhindern, dass eine öffnende eckige Klammer als Beginn eines Markup-Befehls interpretiert wird. Die Verwendung ist auf die Argumente anderer Markup-Befehle beschränkt.

manpage *file text*
Indexstruktur. Dieser Befehl fügt dem Index ein Element hinzu, das auf ein Dokument verweist. Das Dokument wird über den symbolischen Namen *file* angegeben. Das Argument *text* dient zur Beschriftung des Verweises.

Symbolische Namen werden verwendet, um die Konvertierbarkeit dieses Formats in jedes beliebige Ausgabeformat zu gewährleisten. Der tatsächliche Name der Datei wird bei der Konvertierung der Eingabe durch die gewählte Formatierungs-Engine eingefügt. Dies erfolgt auf der Grundlage einer Zuordnung von symbolischen zu tatsächlichen Namen, die der Engine bereitgestellt wird.

rb
Text. Der Befehl wird durch eine schließende eckige Klammer ersetzt. Verwendung in freiformigem Text. Erforderlich, um zu verhindern, dass eine schließende eckige Klammer als Ende eines Markup-Befehls interpretiert wird. Die Verwendung ist auf die Argumente anderer Befehle beschränkt.

url *url label*
Indexstruktur. Dies ist der zweite Befehl zum Hinzufügen eines Elements zum Index. Um auf ein Dokument zu verweisen, wird jedoch kein symbolischer Name verwendet, sondern eine (möglicherweise formatspezifische) URL, die den genauen Ort des hier indizierten Dokuments beschreibt.

vset *varname value*
Vorlagen. In dieser Form setzt der Befehl die benannte Dokumentvariable auf den angegebenen *value*. Es wird keine Ausgabe erzeugt. Das heißt, der Befehl wird durch die leere Zeichenkette ersetzt.

vset *varname*
Vorlagen. In dieser Form wird der Befehl durch den Wert der benannten Dokumentvariablen ersetzt

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_faq

Stichwörter: docidx_lang_faq, docidx commands, docidx language, docidx markup, docidx syntax , examples, faq, markup, semantic markup

Name

docidx_lang_faq – docidx-Sprach-FAQ

Beschreibung

Überblick

Was ist dieses Dokument?

Dieses Dokument ist derzeit vor allem ein Platzhalter und soll mit häufig gestellten Fragen zur Auszeichnungssprache docidx und ihren Begleitern sowie mit den Antworten darauf gefüllt werden.

Bitte teilen Sie uns alle Fragen (und, falls möglich, Antworten) mit, die wir für dieses Dokument berücksichtigen sollten, wie im Abschnitt Fehler, Ideen, Rückmeldungen weiter unten erläutert.

Beispiele

Wo finde ich Beispiele für docidx?

Uns liegen keine direkten Beispiele für Dokumente vor, die mit dem docidx-Markup verfasst wurden. Der doctools-Prozessor **dtplite** erstellt jedoch einen Schlüsselwortindex, wenn er eine Reihe von Dokumenten verarbeitet, die im doctools-Markup verfasst sind. Die Zwischendatei dafür verwendet docidx-Auszeichnung und wird nicht gelöscht, wenn die Erzeugung abgeschlossen ist. Solche Dateien können daher als Beispiele dienen.

dtplite wird als Teil von Tcllib bereitgestellt; um es zu erhalten, benötigen Sie daher eines der Folgenden:

1. Ein Snapshot von Tcllib. Wie Sie einen solchen Snapshot abrufen können und welche Werkzeuge dafür erforderlich sind, wird unter *Development Snapshots*

[/wiki?name=Development+Snapshots] beschrieben.
2. Ein Tcllib-Release-Archiv. Diese sind auf der Seite *home* [/home] verfügbar.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_intro

Stichwörter: docidx_lang_intro, docidx commands, docidx language, docidx markup, docidx syntax, markup, semantic markup

Name

docidx_lang_intro – Einleitung in die Sprache "docidx"

Beschreibung

Dieses Dokument bietet anhand zahlreicher Beispiele eine informelle Einleitung in Version 1 der docidx-Auszeichnungssprache. Nach der Lektüre dieses Dokuments sollten Autoren in der Lage sein, die beiden Teile der formalen Spezifikation zu verstehen, nämlich die Spezifikation der *docidx-Sprachsyntax* und die *docidx-Befehlsreferenz*.

Grundlagen

Zwar ähnelt die *docidx-Markup-Sprache* im weitesten Sinne der *doctools-Markup-Sprache*, doch gibt es einen wesentlichen Unterschied. Ein Index besteht im Wesentlichen ausschließlich aus Markup-Befehlen, zwischen denen – abgesehen von Leerzeichen – kein Klartext eingestreut ist.

Jeder Markup-Befehl ist ein Tcl-Befehl, der von einem passenden Paar aus **[** und **]** umgeben ist. Innerhalb dieser Begrenzungszeichen gelten die üblichen Regeln für einen Tcl-Befehl hinsichtlich Wortanführungszeichen, verschachtelter Befehle, Fortsetzungszeilen usw. Das heißt:

```
    ... [key {markup language}] ...
```

```
  ... [manpage thefile \
          {file description}] ...
```

Grundlegender Aufbau

Das einfachste Dokument, das in docidx verfasst werden kann, ist

```
    [index_begin GROUPTITLE TITLE]
    [index_end]
```

Nicht besonders nützlich, aber gültig. Dies zeigt uns zudem, dass alle docidx-Dokumente aus nur einem Teil bestehen, in dem wir alle Schlüssel und deren Verweise auflisten werden.

Ein nützlicherer Index enthält zumindest Schlüsselwörter oder kurze "Schlüssel", d. h. die Phrasen, die indexiert wurden. Also:

```
[index_begin GROUPTITLE TITLE]
[key markup]
[key {semantic markup}]]
[key {docidx markup}]
[key {docidx language}]
[key {docidx commands}]
[index_end]
```

Im obigen Beispiel wird der Befehl key verwendet, um die Schlüsselwortphrasen zu definieren, die in den Index aufgenommen werden sollen.

Ein wirklich nützlicher Index listet jedoch nicht nur die Schlüsselwortphrasen auf, sondern enthält auch Verweise auf Dokumente, die mit den Schlüsselwörtern in Verbindung stehen. Hier ist ein fiktiver Index für alle Manpages im Modul *base64*:

```
[index_begin tcllib/base64 {De- & Encoding}]
[key base64]
[manpage base64]
[key encoding]
[manpage base64]
[manpage uuencode]
[manpage yencode]
[key uuencode]
[manpage uuencode]
[key yEnc]
[manpage yencode]
[key ydecode]
[manpage yencode]
[key yencode]
[manpage yencode]
[index_end]
```

Im obigen Beispiel wird der Befehl manpage verwendet, um Verweise auf Dokumente unter Verwendung symbolischer Namen einzufügen, wobei jeder Befehl dem unmittelbar vorangehenden Befehl key zugeordnet ist.

Der andere Befehl zum Einfügen von Verweisen lautet **url**. Im Gegensatz zu manpage verwendet er explizite (möglicherweise formatspezifische) URLs, um den Speicherort des referenzierten Dokuments anzugeben. Daher ist dieser Befehl für die Erstellung von Verweisen auf externe Dokumente vorgesehen, die auf keine andere Weise verarbeitet werden konnten.

Erweiterte Struktur

In allen bisherigen Beispielen waren wir hinsichtlich des Markups, das tatsächlich vor dem Befehl **index_begin** zum Öffnen des Dokuments verwendet werden darf, etwas ungenau.

Anstelle von reinen Leerzeichen sind auch die beiden Vorlagenbefehle include und vset zulässig, damit der Verfasser für den Index relevante Konfigurationseinstellungen festlegen und/oder importieren kann. Das heißt, es ist möglich, Folgendes zu schreiben:

```
[include FILE]
[vset VAR VALUE]
[index_begin GROUPTITLE TITLE]
...
[index_end]
```

Noch wichtiger ist, dass diese beiden Befehle überall dort zulässig sind, wo ein Markup-Befehl zulässig ist, unabhängig von jeglicher anderen Struktur.

```
[index_begin GROUPTITLE TITLE]
[include FILE]
[vset VAR VALUE]
...
[index_end]
```

Die einzige Einschränkung, die include beachten muss, besteht darin, dass der Inhalt der eingebundenen Datei an der Stelle der Einbindung gültig sein muss. Das heißt, eine Datei, die vor **index_begin** eingebunden wird, darf nur die Template-Befehle vset und include enthalten; eine Datei, die nach einem Schlüssel eingebunden wird, darf nur **manpage**- oder **url**-Verweise sowie andere Schlüssel enthalten, usw.

Escapes

Neben den bisher vorgestellten sechs Befehlen stehen zwei weitere zur Verfügung. Diese dienen jedoch nicht der Markierung der Indexstruktur, sondern dem Einfügen von Zeichen, nämlich **[** und **]**. Diese Befehle, nämlich lb bzw. rb, sind erforderlich, da unsere Verwendung von [und] zur Einrahmung von Markup-Befehlen es unmöglich macht, [und] direkt innerhalb des Textes zu verwenden.

Als Beispiel für ihre Verwendung dient der Quelltext des letzten Satzes im vorangegangenen Absatz, wobei einige Stellen hervorgehoben wurden.

```
  ...
  These commands, [cmd lb] and [cmd lb] respectively, are required
  because our use of [lb] and [rb] to bracket markup commands makes it
  impossible to directly use [lb] and [rb] within the text.
  ...
```

Weiterführende Literatur

Nachdem der Leser dieses Dokument nun verinnerlicht hat – wobei davon ausgegangen wird, dass es sich um einen *Verfasser* von Dokumentationen handelt –, sollte er ausreichend gerüstet sein, um auch die formale Spezifikation der *docidx-Sprachsyntax* zu verstehen. Von nun an dient die *docidx-Befehlsreferenz* zudem als detaillierte Spezifikation und Spickzettel für alle

verfügbaren Befehle und deren Syntax.

Um ein Dokument bereits während der Erstellung validieren zu können, empfiehlt es sich zudem, sich mit dem äußerst flexibel konfigurierbaren **dtp** von Tclapps vertraut zu machen.

Andererseits eignet sich docidx hervorragend für die automatische Generierung aus doctools-Dokumenten, und genau diesen Weg beschreitet das benutzerfreundliche und einfache **dtplite** von Tcllib, das im Hintergrund einen Index für eine Reihe von Dokumenten erstellt, ohne dass der Verfasser dies selbst tun muss.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_lang_syntax

Stichwörter: docidx_lang_syntax, docidx commands, docidx language, docidx markup, docidx syntax, markup, semantic markup

Name

docidx_lang_syntax – docidx-Sprachsyntax

Beschreibung

Dieses Dokument enthält die formale Spezifikation der Syntax der Markup-Sprache docidx, Version 1, in Backus-Naur-Form. Dieses Dokument ist als Nachschlagewerk gedacht und ergänzt die *docidx-Befehlsreferenz*. Anfängern wird empfohlen, zunächst die wesentlich informeller verfasste *Einleitung in die Sprache docidx* zu lesen, bevor sie versuchen, dieses Dokument oder die Befehlsreferenz zu verstehen.

Grundlagen

Im weitesten Sinne ist die *docidx-Auszeichnungssprache* wie SGML und ähnliche Sprachen. Ein in dieser Sprache verfasstes Dokument besteht in erster Linie aus Auszeichnungsbefehlen, wobei an einigen Stellen Text in das Dokument eingebettet ist.

Jeder Markup-Befehl ist lediglich ein Tcl-Befehl, der von einem passenden Paar aus "**[**" und "**]**" umgeben ist. Welche Befehle verfügbar sind und welche Argumente sie erfordern, d. h. ihre Syntax, wird in der *docidx-Befehlsreferenz für die Sprache* festgelegt.

In diesem Dokument legen wir zunächst das Lexem fest und legen anschließend die Syntax fest, d. h., wie sich Text und Markup-Befehle miteinander mischen lassen.

Lexikalische Definitionen

In den im folgenden Abschnitt aufgeführten Syntaxregeln

1. steht für den gesamten Text mit Ausnahme von Markup-Befehlen.
2. Jedes "XXX" steht für den Markup-Befehl [xxx] einschließlich seiner Argumente. Jeder Markup-Befehl ist ein Tcl-Befehl, der von einem passenden Paar aus **[** und **]** umgeben ist. Innerhalb dieser Begrenzungszeichen gelten die üblichen Regeln für einen Tcl-Befehl hinsichtlich der Quotierung von Wörtern, verschachtelter Befehle, Fortsetzungszeilen usw.
3. steht für sämtlichen Text, der ausschließlich aus Leerzeichen, Zeilenumbrüchen, Tabulatoren und dem Markup-Befehl comment besteht.

Syntax

Die hier aufgeführten Regeln legen lediglich die Syntax von docidx-Dokumenten fest. Die lexikalische Ebene der Sprache wurde im vorangegangenen Abschnitt behandelt.

Zur Syntax der (E)BNF selbst

1. Das Konstrukt { X } steht für null oder mehr Vorkommen von X.
2. Das Konstrukt [X] steht für das einmalige oder nullmalige Vorkommen von X.

Die Syntax lautet:

```
index     = defs
            INDEX_BEGIN
            [ contents ]
            INDEX_END
            { <WHITE> }

defs      = { INCLUDE | VSET | <WHITE> }
contents  = keyword { keyword }

keyword   = defs KEY ref { ref }
ref       = MANPAGE | URL | defs
```

Zuletzt eine Regel, die wir in der EBNF-Syntax nicht erfassen konnten, weil sie die Argumente der Markup-Befehle betrifft, etwas, das hier nicht modelliert ist.

1. Die Argumente aller Markup-Befehle müssen reiner Text sein, Text-Markup-Befehle oder beides. Als Text-Markup-Befehle gelten:
   1. lb,
   2. rb, oder
   3. vset (Form mit einem Argument).

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2007-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


docidx_plugin_apiref
Stichwörter: docidx_plugin_apiref, formatting engine, index, index formatter, keywords, markup, plugin, semantic markup

Name
docidx_plugin_apiref – API-Referenz für das docidx-Plugin

Übersicht
dt_fmap *symfname*
dt_format
dt_read *file*
dt_source *file*
ex_cappend *text*
ex_cget *varname*
ex_cis *cname*
ex_cname
ex_cpop *cname*
ex_cpush *cname*
ex_cset *varname value*
ex_lb ?*newbracket*?
ex_rb ?*newbracket*?
idx_initialize
idx_listvariables
idx_numpasses
idx_postprocess *text*
idx_setup *n*
idx_shutdown
idx_varset *varname text*
fmt_plain_text *text*

Beschreibung
Dieses Dokument richtet sich an *Plugin-Entwickler*, d. h. an Entwickler, die eine *Formatierungs-Engine* für Indizes für ein bestimmtes Ausgabeformat X erstellen möchten.

Es legt die Interaktion zwischen dem Paket **doctools::idx** und dessen Plugins fest, d. h. die Schnittstelle, an die sich jede Indexformatierungs-Engine halten muss.

Dieses Dokument befasst sich mit Version 1 der Schnittstelle.

Ein Leser, der sich hingegen eher für die Auszeichnungssprache selbst interessiert, sollte mit der *Einleitung in die Sprache docidx* beginnen und von dort aus zu den formalen Spezifikationen übergehen, d. h. zur *Syntax der Sprache docidx* und zur *Befehlsreferenz der Sprache docidx*.

Überblick

Die API für eine Indexformatierungs-Engine besteht aus zwei Hauptabschnitten.

Einerseits gibt es eine Reihe von Befehlen, mit denen das Plugin das Frontend abfragen kann. Diese Befehle werden vom Frontend bereitgestellt und in den Plugin-Interpreter eingebunden. Eine detaillierte Beschreibung finden Sie im Abschnitt Frontend-Befehle.

Andererseits muss das Plugin einen eigenen Satz von Befehlen bereitstellen, die dann vom Frontend bei der Verarbeitung der Eingaben in einer bestimmten Reihenfolge aufgerufen werden. Auch diese lassen sich in zwei Kategorien einteilen: Verwaltung und Formatierung. Eine detaillierte Beschreibung finden Sie im Abschnitt Plugin-Befehle und dessen Unterabschnitten.

Frontend-Befehle

In diesem Abschnitt wird die Reihe von Befehlen festgelegt, mit denen ein Plugin – auch als Indexformatierungs-Engine bezeichnet – das Frontend abfragen kann. Diese Befehle werden vom Frontend bereitgestellt und in den Plugin-Interpreter eingebunden.

Das heißt, eine Indexformatierungs-Engine kann davon ausgehen, dass alle folgenden Befehle vorhanden sind, wenn einer ihrer eigenen Befehle (wie im Abschnitt Plugin-Befehle festgelegt) ausgeführt wird.

Darüber hinaus kann davon ausgegangen werden, dass er uneingeschränkten Zugriff auf seinen eigenen sicheren Interpreter hat und somit weder die anderen Teile des Prozessors noch das Dateisystem beschädigen kann. Er kann den gesamten Prozess jedoch beenden oder hängen lassen, indem er ihn beendet oder eine Endlosschleife ausführt.

Um auf die importierten Befehle zurückzukommen: Alle Befehle mit dem Präfix *dt_* bieten eingeschränkten Zugriff auf bestimmte Teile des Frontends, während die Befehle mit dem Präfix *ex_* Zugriff auf den Zustand des Objekts **textutil::expander** gewähren, das die Hauptanalyse der Eingabe innerhalb des Frontends durchführt. Diese Befehle sollten nur unter ganz besonderen Umständen verwendet werden.

dt_fmap *symfname*
: Abfragebefehl. Er gibt den tatsächlichen Pfadnamen zurück, der in der Ausgabe anstelle des symbolischen Dateinamens *symfname* verwendet werden soll. Wird für *symfname* keine Zuordnung festgelegt, gibt er die unveränderte Eingabe zurück.

  Die erforderlichen Zuordnungen werden mit der Methode **map** eines Frontends eingerichtet, wie im Abschnitt **Objektmethoden** der Dokumentation zum Paket **doctools::idx** erläutert.

dt_format
: Abfragebefehl. Er gibt den Namen des Formats zurück, das der Index-Formatierungs-Engine zugeordnet ist.

dt_read *file*
: Kontrollierter Zugriff auf das Dateisystem. Gibt den Inhalt von *file* für beliebige Zwecke des Plugins zurück. Es können nur Dateien geladen werden, die sich entweder im selben

Verzeichnis wie die Datei mit der Engine oder in einem darunterliegenden Verzeichnis befinden. Der Versuch, eine Datei außerhalb dieses Verzeichnisses zu laden, führt zu einem Fehler.

dt_source *file*
Kontrollierter Zugriff auf das Dateisystem. Dieser Befehl ermöglicht es der Index-Formatierungs-Engine, zusätzlichen Tcl-Code zu laden, den sie möglicherweise benötigt. Es können nur Dateien geladen werden, die sich entweder im selben Verzeichnis wie die Datei mit der Engine oder in einem darunterliegenden Verzeichnis befinden. Der Versuch, eine Datei außerhalb dieses Verzeichnisses zu laden, führt zu einem Fehler.

ex_cappend *text*
Fügt im aktuellen Kontext eine Zeichenkette an die Ausgabe an. Dieser Befehl sollte von Makros oder Anwendungscode nur selten verwendet werden.

ex_cget *varname*
Ruft den Wert der im aktuellen Kontext definierten Variablen *varname* ab.

ex_cis *cname*
Ermittelt, ob der Name des aktuellen Kontexts *cname* lautet oder nicht.

ex_cname
Gibt den Namen des aktuellen Kontexts zurück.

ex_cpop *cname*
Entnimmt einen Kontext aus dem Kontextstapel und gibt die gesamte in diesem Kontext gesammelte Ausgabe zurück. Der Kontext muss den Namen *cname* tragen; andernfalls tritt ein Fehler auf.

ex_cpush *cname*
Fügt einen Kontext mit dem Namen *cname* auf den Kontextstapel ein. Der Kontext muss vor Abschluss der Erweiterung durch **cpop** vom Stapel entfernt werden, andernfalls tritt ein Fehler auf.

ex_cset *varname value*
Setzt die Variable *varname* im aktuellen Kontext auf den Wert *value*.

ex_lb ?*newbracket*?
Gibt den aktuellen Wert der linken Makroauswertungsklammer zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

ex_rb ?*newbracket*?
Gibt den aktuellen Wert der rechten Klammer der Makroauswertung zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

Plugin-Befehle

Das Plugin muss einen eigenen Satz von Befehlen bereitstellen, die dann vom Frontend bei der Verarbeitung der Eingaben in einer bestimmten Reihenfolge aufgerufen werden. Diese lassen sich in zwei Kategorien einteilen: Verwaltung und Formatierung. Die erwarteten Namen, Signaturen und Aufgaben dieser Befehle werden in den folgenden beiden Unterabschnitten näher beschrieben.

Verwaltungsbefehle

Die Verwaltungsbefehle, die ein Plugin bereitstellen muss, werden vom Frontend verwendet, um

1. das Plugin zu initialisieren und zu beenden
2. die Anzahl der Durchgänge zu ermitteln, die über die Eingabe laufen müssen
3. jeden Durchgang zu initialisieren und zu beenden
4. Engine-Parameter abzufragen und zu initialisieren.

Nachdem das Plugin geladen wurde und die Frontend-Befehle eingerichtet sind, werden die Befehle in der folgenden Reihenfolge aufgerufen:

```
    idx_numpasses -> n
    idx_listvariables -> vars

    idx_varset var1 value1
    idx_varset var2 value2
    ...
    idx_varset varK valueK
    idx_initialize
    idx_setup 1
    ...
    idx_setup 2
    ...
    ...
    idx_setup n
    ...
    idx_postprocess
    idx_shutdown
    ...
```

Das heißt, zunächst werden die Anzahl der Durchgänge und die Menge der verfügbaren Engine-Parameter festgelegt, woraufhin Aufrufe zur Einstellung der Parameter folgen. Dieser zweite Teil ist optional.

Nachdem das Plugin initialisiert wurde, wird die angegebene Anzahl von Durchgängen ausgeführt, das Endergebnis durchläuft einen globalen Nachbearbeitungsschritt und schließlich wird das Plugin wieder beendet. Darauf können weitere Konvertierungen folgen, wobei die Sequenz bei **idx_varset** neu gestartet wird.

In jedem Durchgang, d. h. nach dem Aufruf von **idx_setup**, verarbeitet das Frontend die Eingabe und ruft die Formatierungsbefehle auf, sobald Markup erkannt wird. Dies bedeutet, dass die Reihenfolge der Formatierungsbefehle durch die Grammatik der docidx-Markup-Sprache bestimmt wird, wie sie in der Spezifikation *Syntax der docidx-Sprache* festgelegt ist.

Eine andere Sichtweise auf die Abfolge ist folgende:

- Zunächst werden einige grundlegende Parameter festgelegt.
- Dann bildet alles, beginnend mit dem ersten "**idx_varset**" bis hin zu "**idx_shutdown**", einen *Lauf*, d. h. die Formatierung einer einzelnen Eingabe. Auf jeden Lauf können weitere folgen.
- In jeden Lauf sind ein oder mehrere *Durchgänge* eingebettet, die jeweils mit **idx_setup** beginnen und so lange andauern, bis entweder das nächste **idx_setup** oder **idx_postprocess**

erreicht ist.

Wenn zur Durchführung der Formatierung mehrere Durchgänge erforderlich sind, ist nur die Ausgabe des letzten Durchgangs relevant. Die Ausgabe aller vorhergehenden, vorbereitenden Durchgänge wird ignoriert.

Die Befehle, ihre Namen, Signaturen und Aufgaben lauten im Einzelnen wie folgt:

idx_initialize
: *Initialisierung/Beendigung*. Dieser Befehl wird zu Beginn jedes Konvertierungslaufs als erster Befehl dieses Laufs aufgerufen. Hinweis: Ein Lauf ist kein Durchgang, sondern kann aus mehreren Durchgängen bestehen. Er muss den allgemeinen Zustand des Plugins initialisieren, über die beim Laden durchgeführte Initialisierung hinaus. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

idx_listvariables
: *Initialisierung/Beendigung* und *Engine-Parameter*. Der zweite Befehl wird aufgerufen, nachdem der Plugin-Code geladen wurde, d. h. unmittelbar nach **idx_numpasses**. Er muss eine Liste zurückgeben, die die Namen der Parameter enthält, die das Frontend zur Konfiguration der Engine einstellen kann. Diese Liste kann leer sein.

idx_numpasses
: *Initialisierung/Beendigung* und *Durchgangsverwaltung*. Der erste Befehl, der nach dem Laden des Plugin-Codes aufgerufen wird. Vor diesem Befehl wird kein anderer Befehl der Engine aufgerufen. Er muss die Anzahl der Durchgänge zurückgeben, die diese Engine benötigt, um das Eingabedokument vollständig zu verarbeiten. Dieser Wert muss eine ganze Zahl sein, die größer oder gleich eins ist.

idx_postprocess *text*
: *Initialisierung/Beendigung*. Dieser Befehl wird unmittelbar nach dem letzten Durchgang eines Laufs aufgerufen. Sein Argument ist das Ergebnis der durch diesen Durchgang erzeugten Konvertierung. Er dient dazu, der Engine die Möglichkeit zu geben, globale Änderungen am erzeugten Dokument vorzunehmen. Ist für ein bestimmtes Format keine Nachbearbeitung erforderlich, muss der Befehl lediglich das Argument zurückgeben.

  Es wird erwartet, dass ein Wert zurückgegeben wird, nämlich das Endergebnis der Formatierung der Eingabe.

idx_setup *n*
: *Initialisierung/Beendigung* und *Durchgangsverwaltung*. Dieser Befehl wird zu Beginn jedes Durchgangs über die Eingabe in einem Lauf aufgerufen. Sein Argument ist die Nummer des Durchgangs, der gerade begonnen hat. Die Durchgänge werden ab **1** aufwärts gezählt. Der Befehl muss den internen Zustand des Plugins für diesen bestimmten Durchgang einrichten. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

idx_shutdown
: *Initialisierung/Beendigung*. Dieser Befehl wird am Ende jedes Konvertierungslaufs aufgerufen. Er ist der letzte Befehl, der in einem Lauf ausgeführt wird. Er muss alle laufspezifischen Zustände im Plugin bereinigen. Nach dem Aufruf muss sich die Engine in einem Zustand befinden, der den Start eines weiteren Laufs ermöglicht, ohne dass Informationen aus dem letzten Lauf in diesen neuen Lauf übergreifen. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

idx_varset *varname text*

Engine-Parameter. Dieser Befehl wird vom Frontend aufgerufen, um einen Engine-Parameter auf einen bestimmten Wert zu setzen. Der zu ändernde Parameter wird durch *varname* angegeben, der einzustellende Wert durch *text*.

Der Befehl muss einen Fehler auslösen, wenn ein unbekannter *varname* verwendet wird. Nur die von **idx_listvariables** zurückgegebenen Namen gelten als bekannt.

Die Werte aller Engine-Parameter müssen über Durchgänge und Läufe hinweg erhalten bleiben.

Formatierungsbefehle

Die Formatierungsbefehle müssen die Formatierung für das Ausgabeformat für alle Markup-Befehle der docidx-Markup-Sprache implementieren, mit Ausnahme von lb, rb, vset, include und comment. Diese Ausnahmen werden vom Frontend verarbeitet und sind für das Plugin nicht sichtbar. Im Gegenzug muss ein Befehl zur Formatierung von Klartext bereitgestellt werden, für etwas, das in der Eingabe überhaupt kein Markup enthält.

Das bedeutet, dass jeder der fünf in der *docidx-Sprachbefehlsreferenz* angegebenen Markup-Befehle, die nicht zu den oben aufgeführten Ausnahmen gehören, einen entsprechenden Formatierungsbefehl besitzt, der dieselben Argumente wie der Markup-Befehl akzeptiert und dessen Name dem Namen des Markup-Befehls mit dem vorangestellten Präfix *fmt_* entspricht.

Von allen Befehlen wird erwartet, dass sie ihre Eingabe gemäß der in der Befehlsreferenz festgelegten Semantik in irgendeiner Weise formatieren und den Teil davon, den sie für notwendig erachten, als Ergebnis zurückgeben, das der Ausgabe hinzugefügt wird.

Um eine im Wesentlichen doppelte Darstellung der Befehlsreferenz zu vermeiden, führen wir hier keinen der Befehle auf und verweisen den Leser lediglich auf die *Befehlsreferenz der docidx-Sprache*, in der deren Syntax und Beschreibung zu finden sind. Die einzige Ausnahme bildet der Formatierer für einfachen Text, für den es keinen entsprechenden Markup-Befehl gibt.

Die Reihenfolge, in der Formatierungsbefehle aufgerufen werden, ist nicht so streng festgelegt wie bei den Verwaltungsbefehlen, sondern richtet sich nach der Grammatik der docidx-Auszeichnungssprache, wie sie in der Spezifikation *Syntax der docidx-Sprache* festgelegt ist.

fmt_plain_text *text*

Kein zugehöriger Markup-Befehl.

Wird vom Frontend für jeden im Eingabetext vorkommenden Klartext aufgerufen. Er muss alle für Klartext erforderlichen speziellen Verarbeitungsschritte durchführen.

Als Ergebnis des Befehls wird der formatierte Text erwartet, der der Ausgabe hinzugefügt wird. Ist keine spezielle Verarbeitung erforderlich, muss er sein Argument einfach unverändert zurückgeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc

Stichwörter: doctoc, HTML, TMML, conversion, documentation, latex, manpage, markdown, markup, nroff, table of contents, toc, wiki

Name

doctools::toc – doctoc – Verarbeitung von Inhaltsverzeichnissen

Übersicht

package require Tcl 8.5 9
package require doctools::toc ?**1.3.1**?
::doctools::toc::new *objectName* ?*-option value* ...?
::doctools::toc::help
::doctools::toc::search *path*
objectName method ?*arg arg* ...?
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**
objectName **destroy**
objectName **format** *text*
objectName **map** *symbolic actual*
objectName **parameters**
objectName **search** *path*
objectName **setparam** *name value*
objectName **warnings**

Beschreibung

Dieses Paket stellt eine Klasse zur Verfügung, mit der Objekte erstellt werden können, die in der *doctoc*-Markup-Sprache verfassten Text verarbeiten und in ein beliebiges Ausgabeformat X konvertieren können, für das eine *Formatierungs-Engine* verfügbar ist.

Leser, die sich für die Auszeichnungssprache selbst interessieren, sollten mit der *Einleitung in die Sprache doctoc* beginnen und von dort aus zu den formalen Spezifikationen übergehen, d. h.

zur *Syntax der Sprache doctoc* und zur *Befehlsreferenz der Sprache doctoc*.

Sollte der Leser hingegen eine eigene Formatierungs-Engine für ein bestimmtes Format schreiben wollen, d. h. als *Plugin-Entwickler* tätig sein, ist das Lesen und Verstehen der *doctoc-Plugin-API-Referenz* unbedingt erforderlich, da dieses Dokument die Interaktion zwischen diesem Paket und seinen Plugins, d. h. den Formatierungs-Engines, im Detail festlegt.

Öffentliche API

Paketbefehle

::doctools::toc::new *objectName* ?-option *value* ...?

Dieser Befehl erstellt ein neues doctoc-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Die Optionen und ihre Werte, die auf den Namen des Objekts folgen, dienen dazu, die Anfangskonfiguration des Objekts festzulegen.

::doctools::toc::help

Dies ist ein Komfortbefehl für Anwendungen, die ihren Benutzern eine kurze Beschreibung der verfügbaren Formatierungsbefehle und deren Bedeutung bereitstellen möchten. Er gibt eine Zeichenkette zurück, die einen Standard-Hilfetext enthält.

::doctools::toc::search *path*

Wann immer ein durch dieses Paket erstelltes Objekt den Namen eines Formats der Datei zuordnen muss, die den Code für dessen Formatierungs-Engine enthält, sucht es in einer Reihe von Verzeichnissen nach dieser Datei, die in einer Liste gespeichert sind. Weitere Erläuterungen finden Sie im Abschnitt Formatzuordnung.

Diese Liste enthält nicht nur drei Standardverzeichnisse, die vom Paket selbst deklariert werden, sondern kann auch vom Benutzer des Pakets erweitert werden. Dieser Befehl dient dazu. Wird ein *path* zu einem vorhandenen und lesbaren Verzeichnis angegeben, wird dieses Verzeichnis der Liste der zu durchsuchenden Verzeichnisse vorangestellt. Das bedeutet, dass der zuletzt hinzugefügte *path* später als erstes durchsucht wird.

Es wird ein Fehler ausgelöst, wenn der *path* entweder nicht existiert, kein Verzeichnis ist oder nicht lesbar ist.

Objektbefehl

Alle von **::doctools::toc::new** erstellten Befehle haben die folgende allgemeine Form und können verwendet werden, um verschiedene Operationen auf dem zugehörigen "doctoc"-Konverterobjekt auszuführen.

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* configure**
Die Methode gibt eine Liste aller bekannten Optionen und deren aktuelle Werte zurück, wenn sie ohne Argumente aufgerufen wird.

objectName* configure *option
Die Methode verhält sich wie die Methode **cget**, wenn sie mit einem einzigen Argument aufgerufen wird, und gibt den Wert der durch dieses Argument festgelegten Option zurück.

***objectName* configure -option *value*...**
Die Methode konfiguriert die angegebenen **option**s des Objekts neu und setzt sie auf die zugehörigen *value*s, wenn sie mit einer geraden Anzahl von Argumenten (mindestens zwei) aufgerufen wird.

Die zulässigen Optionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* cget -option**
Diese Methode erwartet eine zulässige Konfigurationsoption als Argument und gibt den aktuellen Wert dieser Option für das Objekt zurück, für das die Methode aufgerufen wurde.

Die zulässigen Konfigurationsoptionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* destroy**
Diese Methode löscht das Objekt, für das sie aufgerufen wird.

objectName* format *text
Diese Methode verarbeitet den *text* mithilfe der konfigurierten Formatierungs-Engine und gibt die generierte Zeichenfolge als Ergebnis zurück. Es wird ein Fehler ausgelöst, wenn für das Objekt keine Konfiguration von **-format** vorhanden ist.

Die Methode geht davon aus, dass der *text* im *doctoc*-Format vorliegt, wie im Begleitdokument *doctoc_fmt* festgelegt. Andernfalls werden Fehler ausgelöst.

objectName* map *symbolic actual
Diese Methode fügt dem objektbezogenen Zuordnungsschema von *symbolic* Dateinamen zu den *actual* URIs einen Eintrag hinzu. Das Objekt speichert lediglich diese Zuordnung und stellt sie der konfigurierten Formatierungs-Engine über den Befehl **dt_fmap** zur Verfügung. Dieser Befehl wird ausführlicher in der *doctoc-Plugin-API-Referenz* beschrieben, die die Interaktion zwischen den von diesem Paket erstellten Objekten und den TOC-Formatierungs-Engines festlegt.

***objectName* parameters**
Diese Methode gibt eine Liste zurück, die die Namen aller von der konfigurierten Formatierungs-Engine bereitgestellten Parameter enthält. Sie gibt eine leere Liste zurück, wenn das Objekt noch nicht für ein bestimmtes Format konfiguriert ist.

objectName* search *path
Diese Methode erweitert die objektbezogene Liste der Pfade, die nach Formatierungs-Engines für Inhaltsverzeichnisse durchsucht werden. Siehe auch den Befehl **::doctools::toc::search** zur Erweiterung der paketbezogenen Pfadliste. Beachten Sie, dass der zuletzt eingegebene Pfad zuerst durchsucht wird. Weitere Einzelheiten finden Sie im Abschnitt Formatzuordnung.

objectName* setparam *name value
Diese Methode setzt den mit *name* bezeichneten Engine-Parameter auf den angegebenen *value*. Es wird ein Fehler ausgelöst, wenn das Objekt entweder noch nicht für ein bestimmtes

Format konfiguriert ist oder wenn die Formatierungs-Engine für das konfigurierte Format keinen Parameter mit dem angegebenen *name* bereitstellt. Die Liste der von der konfigurierten Formatierungs-Engine bereitgestellten Parameter kann über die Methode **parameters** abgerufen werden.

***objectName* warnings**

Diese Methode gibt eine Liste zurück, die alle Warnungen enthält, die von der konfigurierten Formatierungs-Engine beim letzten Aufruf der Methode **format** erzeugt wurden.

Objekt-Konfiguration

Alle doctoc-Objekte unterstützen die folgenden Konfigurationsoptionen:

-file *file*

Das Argument dieser Option wird im Objekt gespeichert und der konfigurierten Formatierungs-Engine über den Befehl **dt_file** zur Verfügung gestellt. Dieser Befehl wird im Begleitdokument *doctoc_api* näher beschrieben, das die API zwischen dem Objekt und den Formatierungs-Engines festlegt.

Der Standardwert dieser Option ist die leere Zeichenkette.

Die konfigurierte Formatierungs-Engine sollte den Wert als Namen der Datei interpretieren, die das aktuell verarbeitete Dokument enthält.

-format *text*

Das Argument dieser Option legt das zu erzeugende Format fest und damit implizit auch die Formatierungs-Engine, die bei der Konvertierung von Text über die Methode **format** verwendet werden soll. Der Standardwert ist die leere Zeichenkette. Die Methode **format** kann nicht verwendet werden, wenn diese Option nicht mindestens einmal auf einen gültigen Wert gesetzt wurde.

Das Paket versucht umgehend, den angegebenen Namen einer Datei zuzuordnen, die den Code für eine Formatierungs-Engine enthält, welche dieses Format erzeugt. Sollte diese Zuordnung fehlschlagen, wird ein Fehler ausgelöst. In diesem Fall bleibt ein zuvor konfiguriertes Format unverändert.

Im Abschnitt Formatzuordnung wird detailliert erläutert, wie das Paket und das Objekt nach Engine-Implementierungen suchen.

Formatzuordnung

Das Paket und das Objekt führen den folgenden Algorithmus aus, wenn versucht wird, einen Formatnamen *foo* einer Datei zuzuordnen, die eine Implementierung einer Formatierungs-Engine für *foo* enthält:

1. Ist *foo* der Name einer vorhandenen Datei, so wird diese Datei direkt als Implementierung herangezogen.
2. Wenn dies nicht der Fall ist, wird die Liste der objektbezogenen Suchpfade durchsucht. Für jedes Verzeichnis in der Liste prüft das Paket, ob dieses Verzeichnis eine Datei "*toc.foo*" enthält. Ist dies der Fall, wird diese Datei als Implementierung herangezogen.

   Hinweis: Diese Liste der Pfade ist zunächst leer und kann über die Objektmethode **search**

erweitert werden.

3. Ist dies nicht der Fall, wird die Liste der Paketpfade durchsucht. Für jedes Verzeichnis in der Liste prüft das Paket, ob dieses Verzeichnis eine Datei "*toc.foo*" enthält. Ist dies der Fall, wird diese Datei als Implementierung herangezogen.

   Diese Liste der Pfade kann über den Befehl **::doctools::toc::search** erweitert werden. Sie enthält zunächst einen Pfad, nämlich das Unterverzeichnis "*mpformats*" des Verzeichnisses, in dem sich das Paket selbst befindet. Mit anderen Worten: Wenn die Paketimplementierung "*doctoc.tcl*" im Verzeichnis "*/usr/local/lib/tcllib/doctools*" installiert ist, wird standardmäßig das Verzeichnis "*/usr/local/lib/tcllib/doctools/mpformats*" nach Formatimplementierungen durchsucht.
4. Die Zuordnung schlägt fehl.

Vordefinierte Engines

Das Paket stellt vordefinierte Formatierungs-Engines für die folgenden Formate bereit. Einige der Formatierungs-Engines unterstützen Engine-Parameter. Diese werden ausdrücklich hervorgehoben.

html

Diese Engine erzeugt HTML-Markup zur Verarbeitung durch Webbrowser und Ähnliches. Diese Engine unterstützt drei Parameter:

footer

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den "body"-Abschnitt eines HTML-Dokuments sein. Der Standardwert ist die leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar vor dem Tag "" eingefügt und schließt damit den "body"-Abschnitt des generierten HTML-Codes ab.

Dies kann genutzt werden, um Standard-Fußzeilen-Markup in das generierte Dokument einzufügen.

header

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den "body"-Abschnitt eines HTML-Dokuments sein. Der Standardwert ist die leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar nach dem -Tag eingefügt und bildet den Beginn des Hauptteils des generierten HTML-Codes.

Dies kann genutzt werden, um Standard-Kopfzeilen-Markup in das generierte Dokument einzufügen.

meta

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den Kopfbereich eines HTML-Dokuments sein. Der Standardwert ist eine leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar nach dem -Tag eingefügt und bildet den Beginn des Kopfbereichs des generierten HTML-Dokuments.

Dies kann genutzt werden, um Standard-Metadaten-Markup in das generierte Dokument einzufügen, wie beispielsweise Verweise auf ein Stylesheet, Standard-Meta-Schlüsselwörter usw.

latex

Diese Engine erzeugt eine Ausgabe, die für den Textprozessor **latex** aus der TeX-Welt geeignet ist.

list
Diese Engine extrahiert Version, Abschnitt und Titel der Manpage aus dem Dokument. Damit lässt sich eine Verzeichnisliste für eine Reihe von Manpages erzeugen.

markdown
Diese Engine generiert *Markdown*-Markup.

nroff
Diese Engine erzeugt eine nroff-Ausgabe, die von **nroff** oder **groff** verarbeitet wird. Das Ergebnis sind Standard-Manpages, wie sie in der Unix-Welt bekannt sind.

null
Diese Engine erzeugt keinerlei Ausgabe. Das kann genutzt werden, wenn lediglich eine Eingabe validiert werden soll.

tmml
Diese Engine generiert TMML-Markup gemäß der Festlegung von Joe English. Die Tcl Manpage Markup Language ist eine Ableitung von XML.

wiki
Diese Engine generiert Wiki-Markup, wie es Jean Claude Wipplers Anwendung **wikit** versteht.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2003-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_intro

Stichwörter: doctoc_intro, markup, semantic markup, table of contents, toc

Name

doctoc_intro – Einleitung in doctoc

Beschreibung

doctoc (Abkürzung für *documentation tables of contents*) bezeichnet eine Reihe miteinander verbundener, jedoch unterschiedlicher Komponenten, die zusammenwirken, um die Erstellung und Transformation von Inhaltsverzeichnissen für Dokumentationen zu vereinfachen. Dabei

handelt es sich um

1. Eine auf Tcl basierende Sprache für die semantische Auszeichnung eines Inhaltsverzeichnisses. Die Auszeichnung wird durch Tcl-Befehle dargestellt.
2. Ein Paket, das die Möglichkeit bietet, Texte zu lesen und zu transformieren, die in dieser Auszeichnungssprache verfasst sind. Es ist wichtig zu beachten, dass die eigentliche Transformation des Eingabetextes an Plugins weitergeleitet wird.
3. Eine API, die die Schnittstelle zwischen dem oben genannten Paket und einem Plugin beschreibt.

Welche der ausführlicheren Dokumente für den Leser dieser Einleitung relevant sind, hängt von seiner Rolle im Dokumentationsprozess ab.

1. Ein *Autor* von Dokumentationen muss die Auszeichnungssprache selbst verstehen. Ein Anfänger im Umgang mit doctoc sollte zunächst die eher informell verfasste *Einleitung in die doctoc-Sprache* lesen. Nachdem er diese verinnerlicht hat, sollte die formale Spezifikation der *doctoc-Syntax* verständlich werden. Ein mit doctoc erfahrener Autor benötigt möglicherweise nur gelegentlich die *Befehlsreferenz der doctoc-Sprache*, um sein Gedächtnis aufzufrischen.

   Während ein Dokument verfasst wird, kann die Anwendung **dtp** zur Prüfung verwendet werden, und nach der Fertigstellung führt sie auch die Umwandlung in das gewählte System visueller Auszeichnung durch, sei es *roff, HTML, Klartext, Wiki usw. Die einfachere Anwendung **dtplite** verwendet doctoc intern, wenn sie Verzeichnisse mit Dokumentation bearbeitet, und erzeugt für sie automatisch ein passendes Inhaltsverzeichnis.
2. Ein *Verarbeiter* von Dokumentation, die in der Auszeichnungssprache *doctoc* verfasst ist, muss wissen, welche Werkzeuge zur Verfügung stehen.

   Das wichtigste Werkzeug ist die bereits erwähnte Anwendung **dtp**, die von Tcllib bereitgestellt wird. Das einfachere **dtplite** zeigt dem Benutzer doctoc nicht. Auf der untersten Ebene, die beiden Anwendungen gemeinsam ist, befindet sich jedoch das Paket **doctoools::toc**, das die grundlegenden Funktionen zum Lesen und Verarbeiten von Dateien bereitstellt, die Text im doctoc-Format enthalten.
3. Zu guter Letzt müssen *Plugin-Entwickler* die Wechselwirkungen zwischen dem Paket **doctools::toc** und dessen Plugins verstehen, wie in der *doctoc-Plugin-API-Referenz* beschrieben.

Verwandte Formate

doctoc steht nicht für sich allein, sondern verfügt über zwei begleitende Formate. Diese heißen *docidx* und *doctools* und dienen jeweils der Auszeichnung von *Schlüsselwortindizes* bzw. allgemeiner Dokumentation. Sie werden in eigenen Dokumentenreihen beschrieben, beginnend jeweils mit der *docidx-Einleitung* und der *doctools-Einleitung*.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_cmdref
Stichwörter: doctoc_lang_cmdref, doctoc commands, doctoc language, doctoc markup, markup, semantic markup

Name
doctoc_lang_cmdref – doctoc-Sprachbefehlsreferenz

Übersicht
comment *plaintext*
division_end
division_start *text* ?*symfile*?
include *filename*
item *file text desc*
lb
rb
toc_begin *text title*
toc_end
vset *varname value*
vset *varname*

Beschreibung
Dieses Dokument legt sowohl die Namen als auch die Syntax aller Befehle fest, die zusammen die doctoc-Markup-Sprache, Version 1, bilden. Da dieses Dokument als Nachschlagewerk gedacht ist, sind die Befehle in alphabetischer Reihenfolge aufgeführt, und die Beschreibungen sind relativ kurz gehalten. Anfängern wird empfohlen, zunächst die wesentlich informeller verfasste *Einleitung in die doctoc-Sprache* zu lesen.

Befehle
comment *plaintext*
: Auszeichnung des Inhaltsverzeichnisses. Der Text des Arguments wird als Kommentar markiert, der außerhalb des eigentlichen Dokumenttextes steht. Die Hauptverwendung liegt bei freiformigem Text.

division_end
: Inhaltsverzeichnisstruktur. Dieser Befehl schließt den Abschnitt, der durch den letzten, ihm vorausgehenden und noch nicht geschlossenen **division_begin**-Befehl geöffnet wurde.

division_start *text* ?*symfile*?
: Inhaltsverzeichnisstruktur. Dieser Befehl eröffnet einen Abschnitt im Inhaltsverzeichnis. Sein

Gegenstück ist **division_end**. Gemeinsam ermöglichen sie es einem Benutzer, einem Inhaltsverzeichnis eine zusätzliche Struktur zu verleihen.

Der Titel des neuen Abschnitts wird durch das Argument *text* angegeben.

Wenn der symbolische Dateiname *symfile* vorhanden ist, soll der Abschnittstitel auf das referenzierte Dokument verweisen, sofern das Ausgabeformat Verweise unterstützt.

include *filename*

Vorlagen. Der Inhalt der genannten Datei wird als Text im doctoc-Markup interpretiert und anstelle des Include-Befehls verarbeitet. Das Markup in der Datei muss in sich geschlossen sein. Ein Markup-Befehl kann die Dateigrenzen nicht überschreiten.

item *file text desc*

Inhaltsverzeichnisstruktur. Dieser Befehl fügt dem Inhaltsverzeichnis ein einzelnes Element hinzu. Jedes dieser Elemente verweist auf ein Dokument. Das Dokument wird über den symbolischen Namen *file* angegeben. Das Argument *text* dient zur Beschriftung des Verweises, während *desc* einen kurzen beschreibenden Text zu diesem Dokument liefert.

Die symbolischen Namen werden verwendet, um die Konvertierbarkeit dieses Formats in jedes beliebige Ausgabeformat zu gewährleisten. Der tatsächliche Name der Datei wird bei der Konvertierung der Eingabe durch die gewählte Formatierungs-Engine eingefügt. Dies erfolgt auf der Grundlage einer Zuordnung von symbolischen zu tatsächlichen Namen, die der Engine bereitgestellt wird.

lb

Text. Der Befehl wird durch eine öffnende eckige Klammer ersetzt. Verwendung in freiformigem Text. Erforderlich, um zu verhindern, dass eine öffnende eckige Klammer als Beginn eines Markup-Befehls interpretiert wird. Die Verwendung ist auf die Argumente anderer Markup-Befehle beschränkt.

rb

Text. Der Befehl wird durch eine schließende eckige Klammer ersetzt. Verwendung in freiformigem Text. Erforderlich, um zu verhindern, dass eine schließende eckige Klammer als Ende eines Markup-Befehls interpretiert wird. Die Verwendung ist auf die Argumente anderer Befehle beschränkt.

toc_begin *text title*

Dokumentstruktur. Der Befehl zum Beginnen eines Inhaltsverzeichnisses. Die Argumente sind eine Bezeichnung für die gesamte Gruppe von Dokumenten, auf die sich das Inhaltsverzeichnis bezieht (*text*), sowie der übergreifende Titeltext für das Inhaltsverzeichnis (*title*), ohne Markup.

Die Bezeichnung ist häufig der Name des Pakets (oder der Erweiterung), zu dem die Dokumente gehören.

toc_end

Dokumentstruktur. Befehl zum Beenden eines Inhaltsverzeichnisses. Alles, was im Dokument nach diesem Befehl folgt, ist ein Fehler.

vset *varname value*

Vorlagen. In dieser Form setzt der Befehl die benannte Dokumentvariable auf den

angegebenen *value*. Es wird keine Ausgabe erzeugt. Das heißt, der Befehl wird durch die leere Zeichenkette ersetzt.

vset *varname*
Vorlagen. In dieser Form wird der Befehl durch den Wert der benannten Dokumentvariablen ersetzt

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_faq

Stichwörter: doctoc_lang_faq, doctoc commands, doctoc language, doctoc markup, doctoc syntax, examples, faq, markup, semantic markup

Name

doctoc_lang_faq – doctoc-Sprach-FAQ

Beschreibung

Überblick

Was ist dieses Dokument?

Dieses Dokument ist derzeit vor allem ein Platzhalter und soll mit häufig gestellten Fragen zur Auszeichnungssprache doctoc und ihren Begleitern sowie mit den Antworten darauf gefüllt werden.

Bitte teilen Sie uns alle Fragen (und, falls möglich, Antworten) mit, die wir für dieses Dokument berücksichtigen sollten, wie im Abschnitt Fehler, Ideen, Rückmeldungen weiter unten erläutert.

Beispiele

Wo finde ich Beispiele für doctoc?

Uns liegen keine direkten Beispiele für Dokumente vor, die mit dem doctoc-Markup verfasst wurden. Der doctools-Prozessor **dtplite** erstellt jedoch ein Inhaltsverzeichnis, wenn er eine Reihe von Dokumenten verarbeitet, die im doctools-Markup verfasst sind. Die Zwischendatei dafür verwendet doctoc-Auszeichnung und wird nicht gelöscht, wenn die Erzeugung

abgeschlossen ist. Solche Dateien können daher als Beispiele dienen.

dtplite wird als Teil von Tcllib bereitgestellt; um es zu erhalten, benötigen Sie daher eines der Folgenden:

1. Ein Snapshot von Tcllib. Wie Sie einen solchen Snapshot abrufen können und welche Werkzeuge dafür erforderlich sind, wird unter *Development Snapshots* [/wiki?name=Development+Snapshots] beschrieben.
2. Ein Tcllib-Release-Archiv. Diese sind auf der Seite *home* [/home] verfügbar.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_intro

Stichwörter: doctoc_lang_intro, doctoc commands, doctoc language, doctoc markup, doctoc syntax, markup, semantic markup

Name

doctoc_lang_intro – Einleitung in die Sprache "doctoc"

Beschreibung

Dieses Dokument bietet anhand zahlreicher Beispiele eine informelle Einleitung in die Version 1.1 der Markup-Sprache "doctoc". Nach der Lektüre dieses Dokuments sollten Autoren in der Lage sein, die beiden Teile der formalen Spezifikation zu verstehen, nämlich die Spezifikation der *doctoc-Sprachsyntax* und die *doctoc-Befehlsreferenz*.

Grundlagen

Zwar ähnelt die *doctoc-Auszeichnungssprache* im weitesten Sinne der *doctools-Auszeichnungssprache*, doch gibt es einen wesentlichen Unterschied. Ein Inhaltsverzeichnis besteht im Wesentlichen ausschließlich aus Auszeichnungsbefehlen, zwischen denen – abgesehen von Leerzeichen – kein einfacher Text eingestreut ist.

Jeder Markup-Befehl ist ein Tcl-Befehl, der von einem passenden Paar aus **[** und **]** umgeben ist. Innerhalb dieser Begrenzungszeichen gelten die üblichen Regeln für einen Tcl-Befehl hinsichtlich Wortanführungszeichen, verschachtelter Befehle, Fortsetzungszeilen usw. Das heißt:

```
    ... [division_start {Appendix 1}] ...
```

```
  ... [item thefile \
          label {file description}] ...
```

Grundlegender Aufbau

Das einfachste Dokument, das in doctoc verfasst werden kann, ist

```
    [toc_begin GROUPTITLE TITLE]
    [toc_end]
```

Dies zeigt uns zudem, dass alle doctoc-Dokumente aus nur einem Teil bestehen, in dem wir *Einträge* und *Abschnitte* auflisten werden.

Es steht frei, diese nach eigenem Ermessen zu kombinieren. Dies stellt eine Änderung gegenüber Version 1 der Sprache dar, die eine solche Kombination nicht zuließ, sondern lediglich die Verwendung entweder einer Reihe von Einträgen oder einer Reihe von Abschnitten erlaubte.

In den folgenden Abschnitten werden wir die Befehle für jede dieser beiden Möglichkeiten erläutern.

Einträge

Der Befehl item nimmt einen *Eintrag* in ein Inhaltsverzeichnis auf. Dabei handelt es sich im Wesentlichen um einen Verweis auf einen Abschnitt, Unterabschnitt usw. in dem Dokument oder der Dokumentensammlung, für die das Inhaltsverzeichnis bestimmt ist. Der Befehl benötigt drei Argumente: einen symbolischen Namen für die Datei, auf die sich der Eintrag bezieht, sowie zwei Textangaben zur Beschriftung des Eintrags und zur Beschreibung des referenzierten Abschnitts.

Es werden symbolische Namen verwendet, um die Konvertierbarkeit dieses Formats in jedes beliebige Ausgabeformat zu gewährleisten. Der tatsächliche Name einer Datei wird bei der Konvertierung der Eingabe von der gewählten Formatierungs-Engine eingefügt, basierend auf einer Zuordnung von symbolischen zu tatsächlichen Namen, die der Engine bereitgestellt werden.

Hier ein fiktives Beispiel für ein Inhaltsverzeichnis dieses Dokuments:

```
[toc_begin Doctoc {Language Introduction}]
[item 1 DESCRIPTION]
[item 1.1 {Basic structure}]
[item 1.2 Items]
[item 1.3 Divisions]
[item 2 {FURTHER READING}]
[toc_end]
```

Abschnitte

Im letzten Beispiel des vorangegangenen Abschnitts fällt auf, dass die referenzierten Abschnitte tatsächlich eine verschachtelte Struktur aufwiesen, was in den Bezeichnungen der Einträge zum Ausdruck kam, indem für alle unter Abschnitt 1 verschachtelten Abschnitte ein gemeinsames Präfix verwendet wurde.

Diese Art von Struktur lässt sich in der doctoc-Sprache durch die Verwendung von Abschnitten deutlicher hervorheben. Anstelle einer Reihe einfacher Einträge verwenden wir eine Reihe von Abschnitten für die wichtigsten Verweise und ordnen die verschachtelten Einträge dann innerhalb dieser an.

Selbstverständlich können wir anstelle der verschachtelten Einträge auch wieder Abschnitte verwenden und so beliebig tief verschachteln.

Ein Abschnitt wird durch zwei Befehle anstelle von einem gekennzeichnet: einen zum Starten des Abschnitts und einen zum Schließen des zuletzt geöffneten Abschnitts. Diese lauten:

division_start
: Dieser Befehl eröffnet einen neuen Abschnitt. Er erwartet ein oder zwei Argumente: den Titel des Abschnitts und den symbolischen Namen der Datei, auf die er verweist. Letzteres ist optional. Wenn der symbolische Dateiname angegeben ist, sollte der Abschnittstitel auf das referenzierte Dokument verweisen, sofern das Ausgabeformat Verweise unterstützt.

division_end
: Dieser Befehl schließt den zuletzt geöffneten und noch nicht geschlossenen Abschnitt.

Auf diese Weise können wir das letzte Beispiel wie folgt umformulieren:

```
[toc_begin Doctoc {Language Introduction}]
[division_start DESCRIPTION]
[item 1 {Basic structure}]
[item 2 Items]
[item 3 Divisions]
[division_end]
[division_start {FURTHER READING}]
[division_end]
[toc_end]
```

Oder, um eine tiefere Verschachtelung zu veranschaulichen

```
[toc_begin Doctoc {Language Introduction}]
[division_start DESCRIPTION]
[division_start {Basic structure}]
[item 1 Do]
[item 2 Re]
[division_end]
[division_start Items]
[item a Fi]
[item b Fo]
[item c Fa]
[division_end]
[division_start Divisions]
[item 1 Sub]
[item 1 Zero]
[division_end]
[division_end]
[division_start {FURTHER READING}]
[division_end]
[toc_end]
```

Es ist möglich, Einträge und Abschnitte beliebig zu kombinieren und leere Abschnitte einzufügen.

```
[toc_begin Doctoc {Language Introduction}]
[item 1 Do]
[division_start DESCRIPTION]
[division_start {Basic structure}]
[item 2 Re]
[division_end]
[item a Fi]
[division_start Items]
[item b Fo]
[item c Fa]
[division_end]
[division_start Divisions]
[division_end]
[division_end]
[division_start {FURTHER READING}]
[division_end]
[toc_end]
```

Erweiterte Struktur

In allen bisherigen Beispielen waren wir hinsichtlich des Markups, das tatsächlich vor dem Befehl **toc_begin** zum Öffnen des Dokuments verwendet werden darf, etwas ungenau.

Anstelle von reinen Leerzeichen sind auch die beiden Vorlagenbefehle include und vset zulässig, damit der Verfasser für das Inhaltsverzeichnis relevante Konfigurationseinstellungen festlegen und/oder importieren kann. Das heißt, es ist möglich, Folgendes zu schreiben:

```
[include FILE]
[vset VAR VALUE]
[toc_begin GROUPTITLE TITLE]
...
[toc_end]
```

Noch wichtiger ist, dass diese beiden Befehle überall dort zulässig sind, wo ein Markup-Befehl zulässig ist, unabhängig von jeglicher anderen Struktur.

```
[toc_begin GROUPTITLE TITLE]
[include FILE]
[vset VAR VALUE]
...
[toc_end]
```

Die einzige Einschränkung, die include beachten muss, besteht darin, dass der Inhalt der eingebundenen Datei an der Stelle der Einbindung gültig sein muss. Das heißt, eine Datei, die vor **toc_begin** eingebunden wird, darf nur die Vorlagenbefehle vset und include enthalten; eine Datei, die in einen Abschnitt eingebunden wird, darf nur Befehle für Elemente oder Abschnitte enthalten usw.

Escapes

Neben den bisher gezeigten sechs Befehlen stehen uns zwei weitere zur Verfügung. Ihre Funktion besteht jedoch nicht in der Markierung der Inhaltsverzeichnisstruktur, sondern im Einfügen von Zeichen, nämlich **[** und **]**. Diese Befehle, nämlich lb bzw. rb, sind erforderlich, da unsere Verwendung von [und] zur Einfassung von Markup-Befehlen die direkte Verwendung von [und] innerhalb des Textes unmöglich macht.

Als Beispiel für ihre Verwendung dient der Quelltext des letzten Satzes im vorangegangenen Absatz, wobei einige Stellen hervorgehoben wurden.

```
  ...
  These commands, [cmd lb] and [cmd lb] respectively, are required
  because our use of [lb] and [rb] to bracket markup commands makes it
  impossible to directly use [lb] and [rb] within the text.
  ...
```

Weiterführende Literatur

Nachdem der Leser dieses Dokument verinnerlicht hat – wobei davon ausgegangen wird, dass es sich um einen *Verfasser* von Dokumentationen handelt –, sollte er nun ausreichend gerüstet sein, um auch die formale *Syntaxspezifikation der Sprache doctoc* zu verstehen. Von nun an dient die *Befehlsreferenz der Sprache doctoc* zudem als detaillierte Spezifikation und Spickzettel für alle verfügbaren Befehle und deren Syntax.

Um ein Dokument bereits während der Erstellung validieren zu können, empfiehlt es sich zudem, sich mit dem äußerst flexibel konfigurierbaren **dtp** von Tclapps vertraut zu machen.

Andererseits eignet sich doctoc hervorragend für die automatische Generierung aus doctools-Dokumenten, und genau diesen Weg beschreitet das benutzerfreundliche und einfache **dtplite** von Tcllib, das hinter den Kulissen ein Inhaltsverzeichnis für eine Reihe von Dokumenten

erstellt, ohne dass der Verfasser dies selbst tun muss.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctoc_lang_syntax

Stichwörter: doctoc_lang_syntax, doctoc commands, doctoc language, doctoc markup, doctoc syntax, markup, semantic markup

Name

doctoc_lang_syntax – Syntax der doctoc-Sprache

Beschreibung

Dieses Dokument enthält die formale Spezifikation der Syntax der Markup-Sprache "doctoc", Version 1.1, in Backus-Naur-Form. Dieses Dokument ist als Nachschlagewerk gedacht und ergänzt die *doctoc-Befehlsreferenz*. Anfänger sollten zunächst die wesentlich informeller verfasste *Einleitung in die Sprache doctoc* lesen, bevor sie versuchen, dieses Dokument oder die Befehlsreferenz zu verstehen.

Grundlagen

Im weitesten Sinne ähnelt die *doctoc-Auszeichnungssprache* SGML und ähnlichen Sprachen. Ein in dieser Sprache verfasstes Dokument besteht in erster Linie aus Auszeichnungsbefehlen, wobei an einigen Stellen Text in das Dokument eingebettet ist.

Jeder Markup-Befehl ist lediglich ein Tcl-Befehl, der von einem passenden Paar aus "**[**" und "**]**" umgeben ist. Welche Befehle verfügbar sind und welche Argumente sie erfordern, d. h. ihre Syntax, wird in der *Befehlsreferenz der Sprache doctoc* festgelegt.

In diesem Dokument legen wir zunächst das Lexem fest und legen anschließend die Syntax fest, d. h., wie sich Text und Markup-Befehle miteinander mischen lassen.

Lexikalische Definitionen

In den im folgenden Abschnitt aufgeführten Syntaxregeln

1. steht für den gesamten Text mit Ausnahme von Markup-Befehlen.

2. Jedes "XXX" steht für den Markup-Befehl [xxx] einschließlich seiner Argumente. Jeder Markup-Befehl ist ein Tcl-Befehl, der von einem passenden Paar aus **[** und **]** umgeben ist. Innerhalb dieser Begrenzungszeichen gelten die üblichen Regeln für einen Tcl-Befehl hinsichtlich der Quotierung von Wörtern, verschachtelter Befehle, Fortsetzungszeilen usw.
3. steht für sämtlichen Text, der ausschließlich aus Leerzeichen, Zeilenumbrüchen, Tabulatoren und dem Markup-Befehl comment besteht.

Syntax

Die hier aufgeführten Regeln legen lediglich die Syntax von doctoc-Dokumenten fest. Die lexikalische Ebene der Sprache wurde im vorangegangenen Abschnitt behandelt.

Zur Syntax der (E)BNF selbst

1. Das Konstrukt { X } steht für null oder mehr Vorkommen von X.
2. Das Konstrukt [X] steht für das einmalige oder nullmalige Vorkommen von X.

Die Syntax lautet:

```
toc       = defs
            TOC_BEGIN
            contents
            TOC_END
            { <WHITE> }

defs      = { INCLUDE | VSET | <WHITE> }
contents  = { defs entry } [ defs ]

entry     = ITEM | division

division  = DIVISION_START
            contents
            DIVISION_END
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2007-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

doctoc_plugin_apiref

Stichwörter: doctoc_plugin_apiref, formatting engine, markup, plugin, semantic markup, table of contents, toc, toc formatter

Name

doctoc_plugin_apiref – API-Referenz für das doctoc-Plugin

Übersicht

dt_fmap *symfname*
dt_format
dt_read *file*
dt_source *file*
ex_cappend *text*
ex_cget *varname*
ex_cis *cname*
ex_cname
ex_cpop *cname*
ex_cpush *cname*
ex_cset *varname value*
ex_lb ?*newbracket*?
ex_rb ?*newbracket*?
toc_initialize
toc_listvariables
toc_numpasses
toc_postprocess *text*
toc_setup *n*
toc_shutdown
toc_varset *varname text*
fmt_plain_text *text*

Beschreibung

Dieses Dokument richtet sich an *Plugin-Entwickler*, d. h. an Entwickler, die eine *Formatierungs-Engine* für Inhaltsverzeichnisse für ein bestimmtes Ausgabeformat X erstellen möchten.

Es legt die Interaktion zwischen dem Paket **doctools::toc** und dessen Plugins fest, d. h. die Schnittstelle, an die sich jede Formatierungs-Engine für Inhaltsverzeichnisse halten muss.

Dieses Dokument befasst sich mit Version 1 der Schnittstelle.

Ein Leser, der sich hingegen eher für die Auszeichnungssprache selbst interessiert, sollte mit der *Einleitung in die Sprache doctoc* beginnen und von dort aus zu den formalen Spezifikationen übergehen, d. h. zur *Syntax der Sprache doctoc* und zur *Befehlsreferenz der Sprache doctoc*.

Überblick

Die API für eine Engine zur Formatierung von Inhaltsverzeichnissen besteht aus zwei Hauptabschnitten.

Einerseits gibt es eine Reihe von Befehlen, mit denen das Plugin das Frontend abfragen kann.

Diese Befehle werden vom Frontend bereitgestellt und in den Plugin-Interpreter eingebunden. Eine detaillierte Beschreibung finden Sie im Abschnitt Frontend-Befehle.

Andererseits muss das Plugin einen eigenen Satz von Befehlen bereitstellen, die dann vom Frontend bei der Verarbeitung der Eingaben in einer bestimmten Reihenfolge aufgerufen werden. Auch diese lassen sich in zwei Kategorien einteilen: Verwaltung und Formatierung. Eine detaillierte Beschreibung finden Sie im Abschnitt Plugin-Befehle und dessen Unterabschnitten.

Frontend-Befehle

In diesem Abschnitt wird die Menge der Befehle festgelegt, mit denen ein Plugin – auch als "TOC-Formatierungs-Engine" bezeichnet – das Frontend abfragen kann. Diese Befehle werden vom Frontend bereitgestellt und in den Plugin-Interpreter eingebunden.

Das heißt, eine TOC-Formatierungs-Engine kann davon ausgehen, dass alle folgenden Befehle vorhanden sind, wenn einer ihrer eigenen Befehle (wie im Abschnitt Plugin-Befehle festgelegt) ausgeführt wird.

Darüber hinaus kann davon ausgegangen werden, dass er uneingeschränkten Zugriff auf seinen eigenen sicheren Interpreter hat und somit weder die anderen Teile des Prozessors noch das Dateisystem beschädigen kann. Er kann den gesamten Prozess jedoch beenden oder hängen lassen, indem er ihn beendet oder eine Endlosschleife ausführt.

Um auf die importierten Befehle zurückzukommen: Alle Befehle mit dem Präfix *dt_* bieten eingeschränkten Zugriff auf bestimmte Teile des Frontends, während die Befehle mit dem Präfix *ex_* Zugriff auf den Zustand des Objekts **textutil::expander** gewähren, das die Hauptanalyse der Eingabe innerhalb des Frontends durchführt. Diese Befehle sollten nur unter ganz besonderen Umständen verwendet werden.

dt_fmap *symfname*
: Abfragebefehl. Er gibt den tatsächlichen Pfadnamen zurück, der in der Ausgabe anstelle des symbolischen Dateinamens *symfname* verwendet werden soll. Wird für *symfname* keine Zuordnung festgelegt, gibt er die unveränderte Eingabe zurück.

  Die erforderlichen Zuordnungen werden mit der Methode **map** eines Frontends eingerichtet, wie im Abschnitt **Objektmethoden** der Dokumentation zum Paket **doctools::toc** erläutert.

dt_format
: Abfragebefehl. Er gibt den Namen des Formats zurück, das der TOC-Formatierungs-Engine zugeordnet ist.

dt_read *file*
: Kontrollierter Zugriff auf das Dateisystem. Gibt den Inhalt von *file* für beliebige Zwecke des Plugins zurück. Es können nur Dateien geladen werden, die sich entweder im selben Verzeichnis wie die Datei mit der Engine oder in einem darunterliegenden Verzeichnis befinden. Der Versuch, eine Datei außerhalb dieses Verzeichnisses zu laden, führt zu einem Fehler.

dt_source *file*
: Kontrollierter Zugriff auf das Dateisystem. Dieser Befehl ermöglicht es der TOC-Formatierungs-Engine, zusätzlichen Tcl-Code zu laden, den sie möglicherweise benötigt. Es können nur Dateien geladen werden, die sich entweder im selben Verzeichnis wie die Datei mit der Engine oder in einem darunterliegenden Verzeichnis befinden. Der Versuch, eine Datei außerhalb dieses Verzeichnisses zu laden, führt zu einem Fehler.

ex_cappend *text*
Fügt im aktuellen Kontext eine Zeichenkette an die Ausgabe an. Dieser Befehl sollte von Makros oder Anwendungscode nur selten verwendet werden.

ex_cget *varname*
Ruft den Wert der im aktuellen Kontext definierten Variablen *varname* ab.

ex_cis *cname*
Ermittelt, ob der Name des aktuellen Kontexts *cname* lautet oder nicht.

ex_cname
Gibt den Namen des aktuellen Kontexts zurück.

ex_cpop *cname*
Entnimmt einen Kontext aus dem Kontextstapel und gibt die gesamte in diesem Kontext gesammelte Ausgabe zurück. Der Kontext muss den Namen *cname* tragen; andernfalls tritt ein Fehler auf.

ex_cpush *cname*
Fügt einen Kontext mit dem Namen *cname* auf den Kontextstapel ein. Der Kontext muss vor Abschluss der Erweiterung durch **cpop** vom Stapel entfernt werden, andernfalls tritt ein Fehler auf.

ex_cset *varname value*
Setzt die Variable *varname* im aktuellen Kontext auf den Wert *value*.

ex_lb ?*newbracket*?
Gibt den aktuellen Wert der linken Makroauswertungsklammer zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

ex_rb ?*newbracket*?
Gibt den aktuellen Wert der rechten Klammer der Makroauswertung zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

Plugin-Befehle

Das Plugin muss einen eigenen Satz von Befehlen bereitstellen, die dann vom Frontend bei der Verarbeitung der Eingaben in einer bestimmten Reihenfolge aufgerufen werden. Diese lassen sich in zwei Kategorien einteilen: Verwaltung und Formatierung. Die erwarteten Namen, Signaturen und Aufgaben dieser Befehle werden in den folgenden beiden Unterabschnitten näher beschrieben.

Verwaltungsbefehle

Die Verwaltungsbefehle, die ein Plugin bereitstellen muss, werden vom Frontend verwendet, um

1. das Plugin zu initialisieren und zu beenden
2. die Anzahl der Durchgänge zu ermitteln, die über die Eingabe laufen müssen
3. jeden Durchgang zu initialisieren und zu beenden
4. Engine-Parameter abzufragen und zu initialisieren.

Nachdem das Plugin geladen wurde und die Frontend-Befehle eingerichtet sind, werden die Befehle in der folgenden Reihenfolge aufgerufen:

```
    toc_numpasses -> n
    toc_listvariables -> vars

    toc_varset var1 value1
    toc_varset var2 value2
    ...
    toc_varset varK valueK
    toc_initialize
    toc_setup 1
    ...
    toc_setup 2
    ...
    ...
    toc_setup n
    ...
    toc_postprocess
    toc_shutdown
    ...
```

Das heißt, zunächst werden die Anzahl der Durchgänge und die Menge der verfügbaren Engine-Parameter festgelegt, woraufhin Aufrufe zur Einstellung der Parameter folgen. Dieser zweite Teil ist optional.

Nachdem das Plugin initialisiert wurde, wird die angegebene Anzahl von Durchgängen ausgeführt, das Endergebnis durchläuft einen globalen Nachbearbeitungsschritt und schließlich wird das Plugin wieder beendet. Darauf können weitere Konvertierungen folgen, wobei die Sequenz bei **toc_varset** neu gestartet wird.

In jedem Durchgang, d. h. nach dem Aufruf von **toc_setup**, verarbeitet das Frontend die Eingabe und ruft die Formatierungsbefehle auf, sobald Markup erkannt wird. Das bedeutet, dass die Reihenfolge der Formatierungsbefehle durch die Grammatik der doctoc-Markup-Sprache bestimmt wird, wie sie in der Spezifikation *Syntax der doctoc-Sprache* festgelegt ist.

Eine andere Sichtweise auf die Abfolge ist folgende:

- Zunächst werden einige grundlegende Parameter festgelegt.
- Dann bildet alles, beginnend mit dem ersten "**toc_varset**" bis hin zu "**toc_shutdown**", einen *Lauf*, d. h. die Formatierung einer einzelnen Eingabe. Auf jeden Lauf können weitere folgen.
- In jeden Lauf sind ein oder mehrere *Durchgänge* eingebettet, die jeweils mit **toc_setup** beginnen und so lange andauern, bis entweder das nächste **toc_setup** oder **toc_postprocess** erreicht ist.

  Wenn zur Durchführung der Formatierung mehrere Durchgänge erforderlich sind, ist nur die Ausgabe des letzten Durchgangs relevant. Die Ausgabe aller vorhergehenden, vorbereitenden Durchgänge wird ignoriert.

Die Befehle, ihre Namen, Signaturen und Aufgaben lauten im Einzelnen wie folgt:

toc_initialize
: *Initialisierung/Beendigung*. Dieser Befehl wird zu Beginn jedes Konvertierungslaufs als erster Befehl dieses Laufs aufgerufen. Hinweis: Ein Lauf ist kein Durchgang, sondern kann aus

mehreren Durchgängen bestehen. Er muss den allgemeinen Zustand des Plugins initialisieren, über die beim Laden durchgeführte Initialisierung hinaus. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

toc_listvariables
Initialisierung/Beendigung und *Engine-Parameter*. Der zweite Befehl wird aufgerufen, nachdem der Plugin-Code geladen wurde, d. h. unmittelbar nach **toc_numpasses**. Er muss eine Liste zurückgeben, die die Namen der Parameter enthält, die das Frontend zur Konfiguration der Engine einstellen kann. Diese Liste kann leer sein.

toc_numpasses
Initialisierung/Beendigung und *Durchgangsverwaltung*. Der erste Befehl, der nach dem Laden des Plugin-Codes aufgerufen wird. Vor diesem Befehl wird kein anderer Befehl der Engine aufgerufen. Er muss die Anzahl der Durchgänge zurückgeben, die diese Engine benötigt, um das Eingabedokument vollständig zu verarbeiten. Dieser Wert muss eine ganze Zahl sein, die größer oder gleich eins ist.

toc_postprocess *text*
Initialisierung/Beendigung. Dieser Befehl wird unmittelbar nach dem letzten Durchgang eines Laufs aufgerufen. Sein Argument ist das Ergebnis der durch diesen Durchgang erzeugten Konvertierung. Er dient dazu, der Engine die Möglichkeit zu geben, globale Änderungen am erzeugten Dokument vorzunehmen. Ist für ein bestimmtes Format keine Nachbearbeitung erforderlich, muss der Befehl lediglich das Argument zurückgeben.

Es wird erwartet, dass ein Wert zurückgegeben wird, nämlich das Endergebnis der Formatierung der Eingabe.

toc_setup *n*
Initialisierung/Beendigung und *Durchgangsverwaltung*. Dieser Befehl wird zu Beginn jedes Durchgangs über die Eingabe in einem Lauf aufgerufen. Sein Argument ist die Nummer des Durchgangs, der gerade begonnen hat. Die Durchgänge werden ab **1** aufwärts gezählt. Der Befehl muss den internen Zustand des Plugins für diesen bestimmten Durchgang einrichten. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

toc_shutdown
Initialisierung/Beendigung. Dieser Befehl wird am Ende jedes Konvertierungslaufs aufgerufen. Er ist der letzte Befehl, der in einem Lauf ausgeführt wird. Er muss alle laufspezifischen Zustände im Plugin bereinigen. Nach dem Aufruf muss sich die Engine in einem Zustand befinden, der den Start eines weiteren Laufs ermöglicht, ohne dass Informationen aus dem letzten Lauf in diesen neuen Lauf übergreifen. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

toc_varset *varname text*
Engine-Parameter. Dieser Befehl wird vom Frontend aufgerufen, um einen Engine-Parameter auf einen bestimmten Wert zu setzen. Der zu ändernde Parameter wird durch *varname* angegeben, der einzustellende Wert durch *text*.

Der Befehl muss einen Fehler auslösen, wenn ein unbekannter *varname* verwendet wird. Nur die von **toc_listvariables** zurückgegebenen Namen gelten als bekannt.

Die Werte aller Engine-Parameter müssen über Durchgänge und Läufe hinweg erhalten

bleiben.

Formatierungsbefehle

Die Formatierungsbefehle müssen die Formatierung für das Ausgabeformat für alle Markup-Befehle der doctoc-Markup-Sprache implementieren, mit Ausnahme von lb, rb, vset, include und comment. Diese Ausnahmen werden vom Frontend verarbeitet und sind für das Plugin nicht sichtbar. Im Gegenzug muss ein Befehl zur Formatierung von Klartext bereitgestellt werden, für etwas, das in der Eingabe überhaupt kein Markup enthält.

Das bedeutet, dass jeder der fünf Markup-Befehle, die in der *Befehlsreferenz der doctoc-Sprache* aufgeführt sind und nicht zu den oben genannten Ausnahmen gehören, einen entsprechenden Formatierungsbefehl besitzt, der dieselben Argumente wie der Markup-Befehl akzeptiert und dessen Name dem Namen des Markup-Befehls mit dem vorangestellten Präfix *fmt_* entspricht.

Von allen Befehlen wird erwartet, dass sie ihre Eingabe gemäß der in der Befehlsreferenz festgelegten Semantik in irgendeiner Weise formatieren und den Teil davon, den sie für notwendig erachten, als Ergebnis zurückgeben, das der Ausgabe hinzugefügt wird.

Um eine weitgehende Wiederholung der Befehlsreferenz zu vermeiden, führen wir hier keinen der Befehle auf, sondern verweisen den Leser lediglich auf die *Befehlsreferenz der doctoc-Sprache* hinsichtlich ihrer Syntax und Beschreibung. Die einzige Ausnahme bildet der Formatierer für einfachen Text, für den es keinen entsprechenden Markup-Befehl gibt.

Die Reihenfolge, in der Formatierungsbefehle aufgerufen werden, ist nicht so streng festgelegt wie bei den Verwaltungsbefehlen, sondern richtet sich nach der Grammatik der doctoc-Markup-Sprache, wie sie in der Spezifikation *Syntax der doctoc-Sprache* festgelegt ist.

fmt_plain_text *text*

Kein zugehöriger Markup-Befehl.

Wird vom Frontend für jeden im Eingabetext vorkommenden Klartext aufgerufen. Er muss alle für Klartext erforderlichen speziellen Verarbeitungsschritte durchführen.

Als Ergebnis des Befehls wird der formatierte Text erwartet, der der Ausgabe hinzugefügt wird. Ist keine spezielle Verarbeitung erforderlich, muss er sein Argument einfach unverändert zurückgeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools

Stichwörter: doctools, HTML, TMML, conversion, documentation, manpage, markdown, markup, nroff

Name

doctools – doctools – Prozess der Bearbeitung von Dokumenten

Übersicht

package require Tcl 8.5 9
package require doctools ?**1.6.1**?
::doctools::new *objectName* ?*option value...*?
::doctools::help
::doctools::search *path*
objectName method ?*arg arg* ...?
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**
objectName **destroy**
objectName **format** *text*
objectName **map** *symbolic actual*
objectName **parameters**
objectName **search** *path*
objectName **setparam** *name value*
objectName **warnings**

Beschreibung

Dieses Paket stellt eine Klasse zur Verfügung, mit der Objekte erstellt werden können, die in der *doctools*-Markup-Sprache verfassten Text verarbeiten und in ein beliebiges Ausgabeformat X konvertieren können, für das eine *Formatierungs-Engine* verfügbar ist.

Leser, die sich für die Auszeichnungssprache selbst interessieren, sollten mit der *Einleitung in die doctools-Sprache* beginnen und von dort aus zu den formalen Spezifikationen übergehen, d. h. zur *Syntax der doctools-Sprache* und zur *Befehlsreferenz der doctools-Sprache*.

Sollte der Leser hingegen eine eigene Formatierungs-Engine für ein bestimmtes Format schreiben wollen, d. h. als *Plugin-Entwickler* tätig sein, ist das Lesen und Verstehen der *doctools-Plugin-API-Referenz* unbedingt erforderlich, da dieses Dokument die Interaktion zwischen diesem Paket und seinen Plugins, d. h. den Formatierungs-Engines, im Detail festlegt.

Öffentliche API

Paketbefehle

::doctools::new *objectName* ?*option value*...?

Dieser Befehl erstellt ein neues doctools-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Die Optionen und ihre Werte, die auf den Namen des Objekts folgen, dienen dazu, die Anfangskonfiguration des Objekts festzulegen.

::doctools::help

Dies ist ein Komfortbefehl für Anwendungen, die ihren Benutzern eine kurze Beschreibung der verfügbaren Formatierungsbefehle und deren Bedeutung bereitstellen möchten. Er gibt eine Zeichenkette zurück, die einen Standard-Hilfetext enthält.

::doctools::search *path*

Wann immer ein durch dieses Paket erstelltes Objekt den Namen eines Formats der Datei zuordnen muss, die den Code für dessen Formatierungs-Engine enthält, sucht es in einer Reihe von Verzeichnissen nach dieser Datei, die in einer Liste gespeichert sind. Weitere Erläuterungen finden Sie im Abschnitt Formatzuordnung.

Diese Liste enthält nicht nur drei Standardverzeichnisse, die vom Paket selbst deklariert werden, sondern kann auch vom Benutzer des Pakets erweitert werden. Dieser Befehl dient dazu. Wird ein *path* zu einem vorhandenen und lesbaren Verzeichnis angegeben, wird dieses Verzeichnis der Liste der zu durchsuchenden Verzeichnisse vorangestellt. Das bedeutet, dass der zuletzt hinzugefügte *path* später als erstes durchsucht wird.

Es wird ein Fehler ausgelöst, wenn der *path* entweder nicht existiert, kein Verzeichnis ist oder nicht lesbar ist.

Objektbefehl

Alle von **::doctools::new** erstellten Befehle haben die folgende allgemeine Form und können verwendet werden, um verschiedene Operationen auf ihrem "doctools"-Konverterobjekt auszuführen.

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* configure**

Die Methode gibt eine Liste aller bekannten Optionen und deren aktuelle Werte zurück, wenn sie ohne Argumente aufgerufen wird.

objectName* configure *option

Die Methode verhält sich wie die Methode **cget**, wenn sie mit einem einzigen Argument aufgerufen wird, und gibt den Wert der durch dieses Argument festgelegten Option zurück.

***objectName* configure -option *value*...**

Die Methode konfiguriert die angegebenen **option**s des Objekts neu und setzt sie auf die zugehörigen *value*s, wenn sie mit einer geraden Anzahl von Argumenten (mindestens zwei) aufgerufen wird.

Die zulässigen Optionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* cget -option**
Diese Methode erwartet eine zulässige Konfigurationsoption als Argument und gibt den aktuellen Wert dieser Option für das Objekt zurück, für das die Methode aufgerufen wurde.

Die zulässigen Konfigurationsoptionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* destroy**
Diese Methode löscht das Objekt, für das sie aufgerufen wird.

objectName* format *text
Diese Methode lässt den *text* durch die konfigurierte Formatierungs-Engine laufen und gibt die generierte Zeichenkette als Ergebnis zurück. Es wird ein Fehler ausgelöst, wenn für das Objekt kein **-format** konfiguriert wurde.

Die Methode geht davon aus, dass der *text* im *doctools*-Format vorliegt, wie im Begleitdokument *doctools_fmt* festgelegt. Andernfalls werden Fehler ausgelöst.

objectName* map *symbolic actual
Diese Methode fügt dem objektbezogenen Zuordnungsschema von *symbolic* Dateinamen zu den *actual* URIs einen Eintrag hinzu. Das Objekt speichert lediglich diese Zuordnung und stellt sie der konfigurierten Formatierungs-Engine über den Befehl **dt_fmap** zur Verfügung. Dieser Befehl wird ausführlicher in der *doctools-Plugin-API-Referenz* beschrieben, die die Interaktion zwischen den von diesem Paket erstellten Objekten und den doctools-Formatierungs-Engines festlegt.

***objectName* parameters**
Diese Methode gibt eine Liste zurück, die die Namen aller von der konfigurierten Formatierungs-Engine bereitgestellten Parameter enthält. Sie gibt eine leere Liste zurück, wenn das Objekt noch nicht für ein bestimmtes Format konfiguriert ist.

objectName* search *path
Diese Methode erweitert die objektbezogene Liste der Pfade, die nach doctools-Formatierungs-Engines durchsucht werden. Siehe auch den Befehl **::doctools::search** zur Erweiterung der paketbezogenen Pfadliste. Beachten Sie, dass der zuletzt eingegebene Pfad zuerst durchsucht wird. Weitere Einzelheiten finden Sie im Abschnitt Formatzuordnung.

objectName* setparam *name value
Diese Methode setzt den mit *name* bezeichneten Engine-Parameter auf den angegebenen *value*. Es wird ein Fehler ausgelöst, wenn das Objekt entweder noch nicht für ein bestimmtes Format konfiguriert ist oder wenn die Formatierungs-Engine für das konfigurierte Format keinen Parameter mit dem angegebenen *name* bereitstellt. Die Liste der von der konfigurierten Formatierungs-Engine bereitgestellten Parameter kann über die Methode **parameters** abgerufen werden.

***objectName* warnings**
Diese Methode gibt eine Liste zurück, die alle Warnungen enthält, die von der konfigurierten Formatierungs-Engine beim letzten Aufruf der Methode **format** erzeugt wurden.

Objekt-Konfiguration

Alle doctools-Objekte unterstützen die folgenden Konfigurationsoptionen:

-file *file*
Das Argument dieser Option wird im Objekt gespeichert und der konfigurierten Formatierungs-Engine über die Befehle **dt_file** und **dt_mainfile** zur Verfügung gestellt. Diese Befehle werden im Begleitdokument *doctools_api* näher beschrieben, das die API zwischen dem Objekt und den Formatierungs-Engines festlegt.

Der Standardwert dieser Option ist die leere Zeichenkette.

Die konfigurierte Formatierungs-Engine sollte den Wert als Namen der Datei interpretieren, die das aktuell verarbeitete Dokument enthält.

-ibase *file*
Das Argument dieser Option wird im Objekt gespeichert und als Basispfad für die Auflösung relativer Include-Pfade verwendet. Ist diese Option nicht gesetzt (leere Zeichenkette), wird stattdessen der Wert von **-file** verwendet.

Hinweis: **-file** und **-ibase**, obwohl sie ähnlich aussehen, unterscheiden sich tatsächlich stark voneinander. Der Wert von **-file** wird von einigen Engines zur Generierung korrekter relativer Verweise zwischen Ausgabedokumenten (HTML) verwendet. Es handelt sich hierbei also um einen *Ziel*-Pfad. **-ibase** hingegen wird zur Auflösung relativer Einbindepfade verwendet und befasst sich somit mit *Quell*-Pfaden.

Der Wert dieser Option ist eine leere Zeichenkette.

-module *text*
Das Argument dieser Option wird im Objekt gespeichert und der konfigurierten Formatierungs-Engine über den Befehl **dt_module** zur Verfügung gestellt. Dieser Befehl wird im Begleitdokument *doctools_api* näher beschrieben, das die API zwischen dem Objekt und den Formatierungs-Engines festlegt.

Der Standardwert dieser Option ist die leere Zeichenkette.

Die konfigurierte Formatierungs-Engine sollte den Wert als Namen des Moduls interpretieren, zu dem die Datei gehört, die das aktuell verarbeitete Dokument enthält.

-format *text*
Das Argument dieser Option legt das zu erzeugende Format fest und damit implizit auch die Formatierungs-Engine, die bei der Konvertierung von Text über die Methode **format** verwendet werden soll. Der Standardwert ist die leere Zeichenkette. Die Methode **format** kann nicht verwendet werden, wenn diese Option nicht mindestens einmal auf einen gültigen Wert gesetzt wurde.

Das Paket versucht umgehend, den angegebenen Namen einer Datei zuzuordnen, die den Code für eine Formatierungs-Engine enthält, welche dieses Format erzeugt. Sollte diese Zuordnung fehlschlagen, wird ein Fehler ausgelöst. In diesem Fall bleibt ein zuvor konfiguriertes Format unverändert.

Im Abschnitt Formatzuordnung wird detailliert erläutert, wie das Paket und das Objekt nach

Engine-Implementierungen suchen.

-deprecated *boolean*
Diese Option ist ein boolesches Flag. Das Objekt erzeugt Warnungen, wenn dieses Flag gesetzt ist und der an die Methode **format** übergebene Text den veralteten Markup-Befehl strong enthält. Der Vorgabewert ist **FALSE**. Mit anderen Worten: Es werden keine Warnungen erzeugt.

-copyright *text*
Das Argument dieser Option wird im Objekt gespeichert und der konfigurierten Formatierungs-Engine über den Befehl **dt_copyright** zur Verfügung gestellt. Dieser Befehl wird im Begleitdokument *doctools_api* näher beschrieben, das die API zwischen dem Objekt und den Formatierungs-Engines festlegt.

Der Standardwert dieser Option ist die leere Zeichenkette.

Die konfigurierte Formatierungs-Engine sollte den Wert als Urheberrechtszuweisung für das aktuell verarbeitete Dokument oder das darin beschriebene Paket interpretieren.

Diese Information darf genau dann verwendet werden, wenn die Engine keine Urheberrechtszuweisungen im Dokument selbst finden kann. Solche werden durch den Formatierungsbefehl copyright festgelegt. Dieser Befehl wird im Begleitdokument *doctools_fmt* näher beschrieben, das das *doctools*-Format selbst festlegt.

Formatzuordnung

Das Paket und das Objekt führen den folgenden Algorithmus aus, wenn versucht wird, einen Formatnamen *foo* einer Datei zuzuordnen, die eine Implementierung einer Formatierungs-Engine für *foo* enthält:

1. Ist *foo* der Name einer vorhandenen Datei, so wird diese Datei direkt als Implementierung herangezogen.
2. Wenn dies nicht der Fall ist, wird die Liste der objektspezifischen Suchpfade durchsucht. Für jedes Verzeichnis in der Liste prüft das Paket, ob dieses Verzeichnis eine Datei "*fmt.foo*" enthält. Ist dies der Fall, wird diese Datei als Implementierung herangezogen.

   Hinweis: Diese Liste der Pfade ist zunächst leer und kann über die Objektmethode **search** erweitert werden.
3. Ist dies nicht der Fall, wird die Liste der Paketpfade durchsucht. Für jedes Verzeichnis in der Liste prüft das Paket, ob dieses Verzeichnis eine Datei "*fmt.foo*" enthält. Ist dies der Fall, wird diese Datei als Implementierung herangezogen.

   Diese Liste der Pfade kann über den Befehl **::doctools::search** erweitert werden. Sie enthält zunächst einen Pfad, nämlich das Unterverzeichnis "*mpformats*" des Verzeichnisses, in dem sich das Paket selbst befindet. Mit anderen Worten: Wenn die Paketimplementierung "*doctools.tcl*" im Verzeichnis "*/usr/local/lib/tcllib/doctools*" installiert ist, wird standardmäßig das Verzeichnis "*/usr/local/lib/tcllib/doctools/mpformats*" nach Formatimplementierungen durchsucht.
4. Die Zuordnung schlägt fehl.

Vordefinierte Engines

Das Paket stellt vordefinierte Engines für die folgenden Formate bereit. Einige dieser Engines unterstützen Parameter. Diese werden im Folgenden ebenfalls erläutert.

html

Diese Engine erzeugt HTML-Markup zur Verarbeitung durch Webbrowser und Ähnliches. Diese Engine unterstützt vier Parameter:

footer

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den "body"-Abschnitt eines HTML-Dokuments sein. Der Standardwert ist die leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar vor dem Tag "" eingefügt und schließt damit den "body"-Abschnitt des generierten HTML-Codes ab.

Dies kann genutzt werden, um Standard-Fußzeilen-Markup in das generierte Dokument einzufügen.

header

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den "body"-Abschnitt eines HTML-Dokuments sein. Der Standardwert ist die leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar nach dem -Tag eingefügt und bildet den Beginn des Hauptteils des generierten HTML-Codes.

Dies kann genutzt werden, um Standard-Kopfzeilen-Markup in das generierte Dokument einzufügen.

meta

Der Wert für diesen Parameter muss gültiges, in sich geschlossenes HTML-Markup für den Kopfbereich eines HTML-Dokuments sein. Der Standardwert ist eine leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar nach dem -Tag eingefügt und bildet den Beginn des Kopfbereichs des generierten HTML-Dokuments.

Dies kann genutzt werden, um Standard-Metadaten-Markup in das generierte Dokument einzufügen, wie beispielsweise Verweise auf ein Stylesheet, Standard-Meta-Schlüsselwörter usw.

xref

Der Wert für diesen Parameter muss eine Liste von Tripeln sein, die Querverweisinformationen enthalten. Diese Informationen werden von der Engine verwendet, um weitere Hyperlinks zu erstellen. Jedes Tripel ist eine Liste, die in dieser Reihenfolge ein Muster, einen symbolischen Dateinamen und eine Fragmentreferenz enthält. Wird ein Muster mehrfach angegeben, wird das zuletzt auftretende Muster verwendet.

Die Engine fragt bei bestimmten Befehlen die Xref-Datenbank ab und erstellt einen Link, wenn der betreffende Text mit einem der Muster übereinstimmt. Wird keine Übereinstimmung gefunden, wird kein Link erstellt. Der Link führt zu der im entsprechenden Tripel aufgeführten URI "**file#fragment**", nachdem der symbolische Name über "**dt_fmap**" in die tatsächliche URI umgewandelt wurde (siehe die *doctools-Plugin-API-Referenz*). Diese Zuordnung von Dateien zu URIs wurde durch Aufrufe der Methode **map** des doctools-Objekts erstellt (siehe Abschnitt Objektmethoden).

Die folgenden Formatierungsbefehle greifen auf die XRef-Datenbank zu:

cmd ***word***
: Der Befehl sucht in dieser Reihenfolge nach den Mustern "**sa,***word*" und "*word*". Wenn dies fehlschlägt, wird "*word*" in Kleinbuchstaben umgewandelt und der Vorgang erneut versucht.

syscmd ***word***
: Der Befehl sucht in dieser Reihenfolge nach den Mustern "**sa,***word*" und "*word*". Wenn dies fehlschlägt, wird "*word*" in Kleinbuchstaben umgewandelt und der Vorgang erneut versucht.

term ***word***
: Der Befehl sucht in dieser Reihenfolge nach den Mustern **kw,***word*, **sa,***word* und *word*. Wenn dies fehlschlägt, wird *word* in Kleinbuchstaben umgewandelt und der Vorgang erneut versucht.

package *word*
: Der Befehl sucht in dieser Reihenfolge nach den Mustern **sa,***word*, **kw,***word* und *word*. Wenn dies fehlschlägt, wird *word* in Kleinbuchstaben umgewandelt und der Vorgang erneut versucht.

see_also ***word...***
: Der Befehl sucht für jedes an den Befehl übergebene *word* in dieser Reihenfolge nach den Mustern **sa,***word* und *word*. Wenn dies fehlschlägt, wird *word* in Kleinbuchstaben umgewandelt und der Vorgang erneut versucht.

keywords ***word...***
: Der Befehl sucht für jedes an den Befehl übergebene *word* in dieser Reihenfolge nach den Mustern **kw,***word* und *word*. Wenn dies fehlschlägt, wird *word* in Kleinbuchstaben umgewandelt und der Vorgang erneut versucht.

latex
: Diese Engine erzeugt eine Ausgabe, die für den Textprozessor **latex** aus der TeX-Welt geeignet ist.

list
: Diese Engine extrahiert Version, Abschnitt und Titel der Manpage aus dem Dokument. Damit lässt sich eine Verzeichnisliste für eine Reihe von Manpages erzeugen.

nroff
: Diese Engine erzeugt eine nroff-Ausgabe, die von **nroff** oder **groff** verarbeitet wird. Das Ergebnis sind Standard-Manpages, wie sie in der Unix-Welt bekannt sind.

markdown
: Diese Engine generiert *Markdown*-Markup. Diese Engine unterstützt zwei Parameter:

  header
  : Der Wert für diesen Parameter muss gültiges, in sich geschlossenes Markdown-Markup für den Hauptteil eines Markdown-Dokuments sein. Der Standardwert ist die leere Zeichenkette. Der Wert wird in die generierte Ausgabe unmittelbar vor dem Inhaltsverzeichnis eingefügt.

    Dies kann genutzt werden, um Standard-Kopfzeilen-Markup in das generierte Dokument einzufügen.

  xref

Der Wert für diesen Parameter muss eine Liste von Tripeln sein, die Querverweisinformationen enthalten.

Die vollständigen Einzelheiten zur erwarteten Syntax und zur internen Verwendung durch die Engine sind oben für die *html*-Engine erläutert.

null
Diese Engine erzeugt keinerlei Ausgabe. Das kann genutzt werden, wenn lediglich eine Eingabe validiert werden soll.

tmml
Diese Engine generiert TMML-Markup gemäß der Festlegung von Joe English. Die Tcl Manpage Markup Language ist eine Ableitung von XML.

wiki
Diese Engine generiert Wiki-Markup, wie es Jean Claude Wipplers Anwendung **wikit** versteht.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2003-2024 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

doctools_intro

Stichwörter: doctools_intro, markup, semantic markup

Name

doctools_intro – Einleitung in doctools

Beschreibung

doctools (Abkürzung für *documentation tools*) bezeichnet eine Reihe miteinander verbundener, aber dennoch unterschiedlicher Komponenten, die zusammenwirken, um die Erstellung und Transformation von Dokumentationen zu vereinfachen. Dazu gehören

1. Eine auf Tcl basierende Sprache zur semantischen Auszeichnung von Text. Die Auszeichnung wird durch Tcl-Befehle dargestellt, die in den eigentlichen Text eingestreut sind.
2. Ein Paket, das die Möglichkeit bietet, Texte zu lesen und zu transformieren, die in dieser Auszeichnungssprache verfasst sind. Es ist wichtig zu beachten, dass die eigentliche

Transformation des Eingabetextes an Plugins weitergeleitet wird.
3. Eine API, die die Schnittstelle zwischen dem oben genannten Paket und einem Plugin beschreibt.

Welche der ausführlicheren Dokumente für den Leser dieser Einleitung relevant sind, hängt von seiner Rolle im Dokumentationsprozess ab.

1. Ein *Autor* von Dokumentationen muss die Auszeichnungssprache selbst verstehen. Ein Anfänger im Umgang mit doctools sollte zunächst die eher informell verfasste *Einleitung in die doctools-Sprache* lesen. Nachdem er diese verinnerlicht hat, sollte die formale Spezifikation der *doctools-Sprachsyntax* verständlich werden. Ein mit doctools erfahrener Autor benötigt möglicherweise nur gelegentlich die *Befehlsreferenz der doctools-Sprache*, um sein Gedächtnis aufzufrischen.

   Während ein Dokument verfasst wird, kann die Anwendung **dtplite** zu seiner Prüfung verwendet werden, und nach der Fertigstellung führt sie auch die Umwandlung in das gewählte System visueller Auszeichnung durch, sei es *roff, HTML, Klartext, Wiki usw.
2. Ein *Verarbeiter* von Dokumentation, die in der Auszeichnungssprache *doctools* verfasst wurde, muss wissen, welche Werkzeuge zur Verfügung stehen.

   Das wichtigste Werkzeug ist die bereits erwähnte Anwendung **dtplite**, die von Tcllib bereitgestellt wird. Ein leistungsfähigeres Werkzeug, was die Optionen und die Möglichkeit angeht, es zu konfigurieren, ist die Anwendung **dtp**, die von Tclapps bereitgestellt wird. Auf der untersten Ebene, die beiden Anwendungen gemeinsam ist, befindet sich jedoch das Paket **doctools**, das die grundlegenden Funktionen zum Lesen und Verarbeiten von Dateien bereitstellt, die Text im doctools-Format enthalten.
3. Nicht zuletzt müssen *Plugin-Entwickler* die Wechselwirkung zwischen dem Paket **doctools** und dessen Plugins verstehen, wie in der *doctools-Plugin-API-Referenz* beschrieben.

Verwandte Formate

doctools steht nicht für sich allein, sondern hat zwei begleitende Formate. Diese heißen *docidx* und *doctoc* und dienen jeweils der Auszeichnung von *Schlüsselwortindizes* bzw. *Inhaltsverzeichnissen*. Sie werden in eigenen Dokumentenreihen beschrieben, beginnend jeweils mit der *docidx-Einleitung* und der *doctoc-Einleitung*.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>

doctools_lang_cmdref

Stichwörter: doctools_lang_cmdref, doctools commands, doctools language, doctools markup, markup, semantic markup

Name

doctools_lang_cmdref – Referenz zu den Befehlen der Sprache "doctools"

Übersicht

arg *text*
arg_def *type name* ?*mode*?
bullet *(deprecated — use* **item)**
call *args*
category *text*
class *text*
cmd *text*
cmd_def *command*
comment *plaintext*
const *text*
copyright *text*
def *text*
description
enum
emph *text*
example *text*
example_begin
example_end
file *text*
fun *text*
image *name* ?*label*?
include *filename*
item
keywords *args*
lb
list_begin *what*
list_end
lst_item *text (deprecated — use* **def)**
manpage_begin *command section version*
manpage_end
method *text*
moddesc *text*
namespace *text*
nl *(deprecated — use* **para)**
opt *text*

opt_def *name* ?*arg*?
option *text*
package *text*
para
rb
require *package* ?*version...*?
section *name*
sectref *id* ?*text*?
sectref-external *text*
see_also *args*
strong *text (deprecated — use* **emph)**
subsection *name*
syscmd *text*
term *text*
titledesc *desc*
tkoption_def *name dbname dbclass*
type *text*
uri *text* ?*text*?
usage *args*
var *text*
vset *varname value*
vset *varname*
widget *text*

Beschreibung

Dieses Dokument legt sowohl die Namen als auch die Syntax aller Befehle fest, die zusammen die doctools-Markup-Sprache, Version 1, bilden. Da dieses Dokument als Nachschlagewerk gedacht ist, sind die Befehle in alphabetischer Reihenfolge aufgeführt, und die Beschreibungen sind relativ kurz gehalten. Anfängern wird empfohlen, zunächst die wesentlich informeller verfasste *Einleitung in die doctools-Sprache* zu lesen.

Befehle

arg *text*
: Textauszeichnung. Der Argumenttext wird als *Argument* eines Befehls markiert. Zu den Hauptanwendungsbereichen zählen die Hervorhebung von Befehlsargumenten in Freitext sowie die Argumentparameter der Auszeichnungsbefehle **call** und **usage**.

arg_def *type name* ?*mode*?
: Textstruktur. Listenelement. Argumentliste. Schließt automatisch das vorherige Listenelement. Legt den Daten-*type* des beschriebenen Arguments eines Befehls, dessen *name* und dessen E/A-*mode* fest. Letzterer ist optional.

bullet *(deprecated — use* item*)*
: *Veraltet*. Textstruktur. Listenelement. Aufzählung. Den kanonischen Befehl zum Öffnen eines Listenelements in einer Aufzählung finden Sie unter **item**.

call *args*
: Textstruktur. Listenelement. Definitionsliste. Schließt automatisch das vorherige Listenelement. Definiert den Begriff als Befehl und dessen Argumente. Das erste Argument ist

der Name des Befehls, der durch den folgenden Freitext beschrieben wird, und alle darauf folgenden Argumente sind Beschreibungen der Befehlsargumente. Es wird erwartet, dass die Argumente passend mit **arg**, **method**, **option** usw. gekennzeichnet sind und dass der Befehl selbst mit **cmd** gekennzeichnet ist. Es wird erwartet, dass der formatierte Begriff nicht nur an der entsprechenden Stelle ausgegeben wird, sondern je nach Ausgabeformat auch im Inhaltsverzeichnis des Dokuments oder in der Übersicht erscheint.

category *text*
Dokumentinformationen. Überall. Dieser Befehl registriert seine Argumente im Klartext als die Kategorie, zu der dieses Dokument gehört. Wird dieser Befehl mehrfach verwendet, wird der zuletzt angegebene Wert herangezogen.

class *text*
Textmarkierung. Das Argument wird als Name einer *Klasse* markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Klassennamen in Freitext.

cmd *text*
Textmarkierung. Der Text wird als Name eines *Tcl-Befehls* markiert. Der Text kann bereits andere Markierungen enthalten. Zu den Hauptanwendungsbereichen zählen die Hervorhebung von Befehlen in Freitext sowie die Befehlsparameter der Markierungsbefehle **call** und **usage**.

cmd_def *command*
Textstruktur. Listenelement. Befehlsliste. Schließt automatisch das vorherige Listenelement. Das Argument legt den Namen des *Tcl-Befehls* fest, der durch das Listenelement beschrieben werden soll. Es wird erwartet, dass dies in der Ausgabe so markiert wird, als wäre es mit **cmd** formatiert worden.

comment *plaintext*
Textmarkierung. Der Text wird als Kommentar markiert, der außerhalb des eigentlichen Dokumenttextes steht. Diese Funktion wird vor allem bei frei formatiertem Text verwendet.

const *text*
Textmarkierung. Das Argument wird als *konstanter* Wert markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Konstanten in Freitext.

copyright *text*
Dokumentinformationen. Überall. Der Befehl registriert das Argument im Klartext als Urheberrechtsabtretung für die Manpage. Bei mehrmaliger Ausführung werden die Abtretungen kumuliert.

def *text*
Textstruktur. Listenelement. Definitionsliste. Schließt automatisch das vorherige Listenelement. Der Argumenttext ist der Begriff, der durch das neue Listenelement definiert wird. Darauf kann Textauszeichnung angewendet werden.

description
Dokumentstruktur. Dieser Befehl trennt den Kopf vom Hauptteil des Dokuments. Er leitet implizit einen Abschnitt mit dem Namen "DESCRIPTION" ein (siehe Befehl **section**).

enum
Textstruktur. Listenelement. Nummerierte Liste. Schließt automatisch das vorherige Listenelement.

emph *text*
Textauszeichnung. Der Text wird als hervorgehoben formatiert. Diese Funktion dient in erster Linie dazu, beliebige Textstücke allgemein hervorzuheben, ohne diesen Stücken eine besondere Bedeutung beizumessen.

example *text*
Textstruktur, Textmarkierung. Dieser Befehl markiert sein Argument als *Beispiel*. Er dient in erster Linie der einfachen Einbettung von Beispielen in frei formatierten Text. Er sollte verwendet werden, wenn das Beispiel *keine* eigene spezielle Markierung benötigt. Andernfalls verwenden Sie eine Folge von **example_begin** ... **example_end**.

example_begin
Textstruktur. Dieser Befehl leitet ein Beispiel ein. Der gesamte Text bis zum nächsten **example_end** gehört zum Beispiel. Zeilenumbrüche, Leerzeichen und Tabulatoren müssen wortgetreu beibehalten werden. Beispiele dürfen nicht verschachtelt werden.

example_end
Textstruktur. Dieser Befehl schließt das Beispiel, das mit dem letzten Befehl "**example_begin** " begonnen wurde.

file *text*
Textmarkierung. Das Argument wird als *Datei* oder *Verzeichnis* markiert, d. h. im Allgemeinen als *Pfad*. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Pfaden in Freitext.

fun *text*
Textmarkierung. Das Argument wird als Name einer *Funktion* markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Funktionsnamen in Freitext.

image *name* ?*label*?
Textmarkierung. Das Argument ist der symbolische Name eines *Bildes* und wird durch das Bild selbst ersetzt, sofern das Backend eine passende Variante findet. Das zweite Argument, sofern vorhanden, wird als für Menschen lesbare Beschreibung des Bildes interpretiert und an geeigneter Stelle in die Ausgabe eingefügt, sofern dies vom Format unterstützt wird. Das HTML-Format kann diese Beschreibung beispielsweise in das *alt*-Attribut von Bildverweisen einfügen.

include *filename*
Templating. Der Inhalt der genannten Datei wird als Text im doctools-Markup interpretiert und anstelle des Include-Befehls verarbeitet. Das Markup in der Datei muss in sich geschlossen sein. Ein Markup-Befehl darf die Dateigrenzen nicht überschreiten.

item
Textstruktur. Listenelement. Aufzählung. Schließt automatisch das vorherige Listenelement.

keywords *args*
Dokumentinformationen. Überall. Dieser Befehl registriert alle seine Argumente im Klartext als Schlüsselwörter, die für dieses Dokument gelten. Jedes Argument ist ein einzelnes Schlüsselwort. Wird dieser Befehl mehrfach verwendet, werden alle Argumente kumuliert.

lb
Text. Der Befehl wird durch eine linke Klammer ersetzt. Verwendung in frei formatiertem Text. Erforderlich, um zu verhindern, dass eine linke Klammer als Beginn eines Markup-Befehls interpretiert wird.

list_begin *what*
Textstruktur. Dieser Befehl leitet eine Liste ein. Die genaue Art der Liste wird durch das Argument *what* des Befehls bestimmt. Dies legt wiederum fest, welche Befehle zum Einleiten der Listenelemente verwendet werden müssen. Listen können verschachtelt werden, d. h., es ist zulässig, innerhalb eines Listenelements eine neue Liste zu beginnen.

Die zulässigen Typen (und die ihnen zugeordneten Elementbefehle) sind:

arguments
arg_def.

commands
cmd_def.

definitions
def und **call**.

enumerated
enum

itemized
item

options
opt_def

tkoptions
tkoption_def

Darüber hinaus werden die folgenden Namen als Kurzformen für einige der regulären Typen erkannt:

args
Abkürzung für **arguments**.

cmds
Abkürzung für **commands**.

enum
Abkürzung für **enumerated**.

item
Abkürzung für **itemized**.

opts
Abkürzung für **options**.

Zuletzt werden die folgenden Namen zwar aus Gründen der Kompatibilität weiterhin erkannt, gelten aber ansonsten als *deprecated*.

arg
Veraltet. Siehe **arguments**.

bullet
Veraltet. Siehe **itemized**.

cmd
Veraltet. Siehe **commands**.

opt
Veraltet. Siehe **options**.

tkoption
Veraltet. Siehe **tkoptions**.

list_end
Textstruktur. Dieser Befehl schließt die Liste, die durch den letzten, ihm vorausgehenden **list_begin**-Befehl geöffnet wurde.

lst_item *text (deprecated — use* def*)*
Veraltet. Textstruktur. Listenelement. Definitionsliste. Der kanonische Befehl zum Öffnen eines allgemeinen Listenelements in einer Definitionsliste befindet sich unter **def**.

manpage_begin *command section version*
Dokumentstruktur. Der Befehl zum Beginnen einer Manpage. Die Argumente sind der Name des in der Manpage beschriebenen *command*, der *section* der Manpages, in der diese Manpage steht, sowie die *version* des Moduls, das den Befehl enthält. Alle Argumente müssen aus reinem Text bestehen, ohne Markup.

manpage_end
Dokumentstruktur. Befehl zum Beenden einer Manpage bzw. eines Dokuments. Alles, was im Dokument nach diesem Befehl folgt, ist ein Fehler.

method *text*
Textmarkierung. Der Argumenttext wird als Name einer *Objekt-Methode* markiert, d. h. als Unterbefehl eines Tcl-Befehls. Der Text kann bereits andere Markierungen enthalten. Zu den Hauptanwendungsbereichen zählen die Hervorhebung der Namen von Methoden in Freitext sowie die Befehlsparameter der Markierungsbefehle **call** und **usage**.

moddesc *text*
Dokumentinformationen. Kopf. Registriert das Argument im Klartext als Kurzbeschreibung des Moduls, in dem die Manpage steht.

namespace *text*
Textmarkierung. Der Text wird als Namespace-Name markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Namespace-Namen in Freitext.

nl *(deprecated — use* para*)*
Veraltet. Textstruktur. Den kanonischen Befehl zum Einfügen von Absatzumbrüchen in den Text finden Sie unter **para**.

opt *text*
Textauszeichnung. Der Argumenttext wird als *optional* markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung optionaler Argumente; siehe den Befehl arg **arg**.

opt_def *name* ?*arg*?
Textstruktur. Listenelement. Optionsliste. Schließt automatisch das vorherige Listenelement. Legt *name* und die Argumente der durch das Listenelement beschriebenen *option* fest. Es wird erwartet, dass der Name mit **option** markiert ist.

option *text*
Textmarkierung. Das Argument wird als *Option* markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Optionen, auch als Befehlsschalter bezeichnet, entweder in Freitext oder in den Argumenten der Befehle **call** und **usage**.

package *text*
Textauszeichnung. Das Argument wird als Name eines *package* markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Paketnamen in Freitext.

para
Textstruktur. Dieser Befehl unterteilt Freitext in Absätze. Jeder Befehl schließt den ihm vorausgehenden Absatz ab und leitet einen neuen Absatz für den darauf folgenden Text ein. Höherliegende Strukturformen sind Abschnitte und Unterabschnitte.

rb
Text. Der Befehl wird durch eine rechte Klammer ersetzt. Verwendung in frei formatiertem Text. Erforderlich, um zu verhindern, dass eine rechte Klammer als Ende eines Markup-Befehls interpretiert wird.

require *package* ?*version*...?
Dokumentinformationen. Kopf. Dieser Befehl registriert sein Argument *package* als Namen eines Pakets oder einer Anwendung, das bzw. die von dem beschriebenen Paket oder der beschriebenen Anwendung benötigt wird. Zudem kann eine Reihe von erforderlichen Versionen angegeben werden. Dieses Argument kann mit Markup versehen werden. Das übliche Markup lautet **opt**.

section *name*
Textstruktur. Dieser Befehl leitet einen neuen, benannten Dokumentabschnitt ein. Das Argument muss aus reinem Text bestehen. Er schließt implizit den letzten ihm vorausgehenden Absatz und eröffnet ebenfalls implizit den ersten Absatz des neuen Abschnitts.

sectref *id* ?*text*?
Textmarkierung. Formatiert einen Verweis auf den durch *id* gekennzeichneten Abschnitt. Wenn kein *text* festgelegt wird, wird in der Ausgabe der Titel des betreffenden Abschnitts verwendet; andernfalls wird *text* verwendet.

sectref-external *text*
Textmarkierung. Wie bei **sectref**, mit dem Unterschied, dass davon ausgegangen wird, dass sich der Abschnitt in einem anderen Dokument befindet und daher weder identifiziert werden muss noch eine Überprüfung auf Vorhandensein erfolgt. Es wird lediglich der zu formatierende Text benötigt.

see_also *args*
Dokumentinformationen. Überall. Der Befehl definiert direkte Querverweise auf andere Dokumente. Jedes Argument ist eine Beschriftung im Klartext, die das referenzierte Dokument identifiziert. Wird dieser Befehl mehrfach verwendet, werden alle Argumente kumuliert.

strong *text (deprecated — use* emph*)*
Veraltet. Textauszeichnung. Den kanonischen Befehl zum Hervorheben von Text finden Sie unter **emph**.

subsection *name*
Textstruktur. Dieser Befehl leitet einen neuen, benannten Unterabschnitt eines Abschnitts ein. Das Argument muss aus reinem Text bestehen. Er schließt implizit den letzten ihm vorausgehenden Absatz und eröffnet ebenfalls implizit den ersten Absatz des neuen Unterabschnitts.

syscmd *text*

Textmarkierung. Der Text wird als Name eines externen Befehls markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung externer Befehle in Freitext.

term *text*
Textmarkierung. Das Argument wird als unspezifische Terminologie markiert. Der Text ist möglicherweise bereits mit anderen Markierungen versehen. Die Hauptanwendung besteht in der Hervorhebung wichtiger Begriffe und Konzepte in Freitext.

titledesc *desc*
Dokumentinformationen. Kopf. Optional. Registriert das Klartext-Argument als Titel der Manpage. Der Vorgabewert entspricht dem durch **moddesc** registrierten Wert.

tkoption_def *name dbname dbclass*
Textstruktur. Listenelement. Widget-Optionsliste. Schließt das vorherige Listenelement automatisch. Legt den *name* der Option fest, wie er in Skripten verwendet wird, den Namen, der von der Optionsdatenbank (*dbname*) verwendet wird, sowie deren Klasse (*dbclass*), das heißt deren Typ. Es wird erwartet, dass der Name mit **option** markiert ist.

type *text*
Textmarkierung. Das Argument wird als Name eines *Datentyps* markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Datentypen in Freitext.

uri *text* ?*text*?
Textmarkierung. Das Argument wird als *URI* (d. h. als *Uniform Resource Identifier*) markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von URIs in Freitext. Das zweite Argument, sofern vorhanden, wird als die für Menschen lesbare Beschreibung des URIs interpretiert. Mit anderen Worten: als dessen Bezeichnung. Ohne explizite Bezeichnung gilt der URI selbst als seine eigene Bezeichnung.

usage *args*
Textmarkierung. Die vollständige Beschreibung finden Sie unter **call**; dieser Befehl ist syntaktisch identisch und stellt dieselben Anforderungen an die Markierung seiner Argumente. Im Gegensatz zu **call** ist es jedoch nicht zulässig, eine Ausgabe zu erzeugen, in der dieser Befehl im Text vorkommt. Der Befehl ist *still*. Der formatierte Text darf nur in einem anderen Abschnitt der Ausgabe erscheinen, beispielsweise in einem Inhaltsverzeichnis oder einer Übersicht, je nach Ausgabeformat.

var *text*
Textmarkierung. Das Argument wird als Name einer *Variablen* markiert. Der Text kann bereits andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Variablen in Freitext.

vset *varname value*
Vorlagen. In dieser Form setzt der Befehl die benannte Dokumentvariable auf den angegebenen *value*. Es wird keine Ausgabe erzeugt. Das heißt, der Befehl wird durch die leere Zeichenkette ersetzt.

vset *varname*
Vorlagen. In dieser Form wird der Befehl durch den Wert der benannten Dokumentvariablen ersetzt

widget *text*
Textmarkierung. Das Argument wird als Name eines *Widgets* markiert. Der Text kann bereits

andere Markierungen enthalten. Die Hauptanwendung besteht in der Hervorhebung von Widget-Namen in Freitext.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007-2010 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_lang_faq

Stichwörter: doctools_lang_faq, doctools commands, doctools language, doctools markup, doctools syntax, examples, faq, markup, semantic markup

Name

doctools_lang_faq – Häufig gestellte Fragen zur doctools-Sprache

Beschreibung

Überblick

Was ist dieses Dokument?

Dieses Dokument ist derzeit vor allem ein Platzhalter und soll mit häufig gestellten Fragen zur Auszeichnungssprache doctools und ihren Begleitern sowie mit den Antworten darauf gefüllt werden.

Bitte teilen Sie uns alle Fragen (und, falls möglich, Antworten) mit, die wir für dieses Dokument berücksichtigen sollten, wie im Abschnitt Fehler, Ideen, Rückmeldungen weiter unten erläutert.

Beispiele

Wo finde ich Beispiele für doctools?

Uns liegen keine direkten Beispiele für Dokumente vor, die mit dem doctools-Markup verfasst wurden. Der doctools-Prozessor **dtplite** erstellt jedoch ein Inhaltsverzeichnis, wenn er eine Reihe von Dokumenten verarbeitet, die im doctools-Markup verfasst sind. Die Zwischendatei dafür verwendet doctools-Auszeichnung und wird nicht gelöscht, wenn die Erzeugung abgeschlossen ist. Solche Dateien können daher als Beispiele dienen.

dtplite wird als Teil von Tcllib bereitgestellt; um es zu erhalten, benötigen Sie daher eines der Folgenden:

1. Ein Snapshot von Tcllib. Wie Sie einen solchen Snapshot abrufen können und welche Werkzeuge dafür erforderlich sind, wird unter *Development Snapshots* [/wiki?name=Development+Snapshots] beschrieben.
2. Ein Tcllib-Release-Archiv. Diese sind auf der Seite *home* [/home] verfügbar.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_lang_intro

Stichwörter: doctools_lang_intro, doctools commands, doctools language, doctools markup, doctools syntax, markup, semantic markup

Name

doctools_lang_intro – Einleitung in die Sprache "doctools"

Beschreibung

Dieses Dokument bietet anhand zahlreicher Beispiele eine informelle Einleitung in Version 1 der doctools-Markup-Sprache. Nach der Lektüre dieses Dokuments sollten Autoren in der Lage sein, die beiden Teile der formalen Spezifikation zu verstehen, nämlich die Spezifikation der *doctools-Sprachsyntax* und die *doctools-Befehlsreferenz*.

Grundlagen

Im weitesten Sinne ähnelt die *doctools-Auszeichnungssprache* eher LaTeX als SGML und ähnlichen Sprachen. Ein in dieser Sprache verfasstes Dokument besteht in erster Linie aus Text, in den Auszeichnungsbefehle eingebettet sind.

Jeder Markup-Befehl ist ein Tcl-Befehl, der von einem passenden Paar aus **[** und **]** umgeben ist. Innerhalb dieser Begrenzungszeichen gelten die üblichen Regeln für einen Tcl-Befehl hinsichtlich Wortanführungszeichen, verschachtelter Befehle, Fortsetzungszeilen usw. Das heißt:

```
  ... [list_begin enumerated] ...
```

```
  ... [call [cmd foo] \
          [arg bar]] ...
```

```
  ... [term {complex concept}] ...
```

```
  ... [opt "[arg key] [arg value]"] ...
```

Grundlegender Aufbau

Das einfachste Dokument, das mit doctools erstellt werden kann, ist

```
    [manpage_begin NAME SECTION VERSION]
[see_also doctools_intro]
[see_also doctools_lang_cmdref]
[see_also doctools_lang_faq]
[see_also doctools_lang_syntax]
[keywords {doctools commands}]
[keywords {doctools language}]
[keywords {doctools markup}]
[keywords {doctools syntax}]
[keywords markup]
[keywords {semantic markup}]
    [description]
    [vset CATEGORY doctools]
[include ../common-text/feedback.inc]
[manpage_end]
```

Dies zeigt uns zudem, dass alle doctools-Dokumente in zwei Teile unterteilt sind: den *Kopf* und den *Hauptteil*. Alles, was vor [description] steht, gehört zum Kopf, und alles, was danach folgt, gehört zum Hauptteil, wobei das gesamte Dokument von den beiden Befehlen **manpage_*** eingerahmt wird. Vor und nach diesen öffnenden und schließenden Befehlen befindet sich lediglich *Leerraum*.

Im weiteren Verlauf dieses Abschnitts werden wir uns ausschließlich mit dem Inhalt des Kopfs befassen; die Struktur des Hauptteils wird im Abschnitt Textstruktur behandelt.

Der Kopfbereich kann leer sein; andernfalls darf er lediglich eine beliebige Abfolge der vier sogenannten *Kopf*-Befehle sowie *Leerraum* enthalten. Diese Befehle lauten

- titledesc
- moddesc
- require
- copyright

Sie liefern über ihre Argumente zusätzliche Informationen zum Dokument, wie beispielsweise dessen Titel, den Titel der größeren Gruppe, zu der das Dokument gehört (falls zutreffend), die Anforderungen der dokumentierten Pakete (falls zutreffend) sowie Urheberrechtsvermerke. Alle diese Angaben können mehrfach vorkommen – oder auch gar nicht – und in beliebiger Reihenfolge verwendet werden. Bei titledesc und moddesc wird jedoch nur das letzte Vorkommen berücksichtigt. Bei den beiden anderen werden die angegebenen Informationen in der angegebenen Reihenfolge kumuliert. Normaler Text ist innerhalb des Kopfs nicht zulässig.

Vor diesem Hintergrund lautet ein weniger minimales Beispiel für ein Dokument:

```
[manpage_begin NAME SECTION VERSION]
[copyright {YEAR AUTHOR}]
[titledesc TITLE]
[moddesc   MODULE_TITLE]
[require   PACKAGE VERSION]
[require   PACKAGE]
[description]
[manpage_end]
```

Zu beachten ist, dass der Leerraum optional ist. Das Dokument

```
    [manpage_begin NAME SECTION VERSION]
    [copyright {YEAR AUTHOR}][titledesc TITLE][moddesc MODULE_TITLE]
    [require PACKAGE VERSION][require PACKAGE][description]
    [vset CATEGORY doctools]
[include ../common-text/feedback.inc]
[manpage_end]
```

hat dieselbe Bedeutung wie das vorherige Beispiel.

Wenn hingegen *Leerraum* vorhanden ist, besteht er nicht nur aus einer beliebigen Folge der Zeichen Leerzeichen, horizontaler und vertikaler Tabulator, Wagenrücklauf und Zeilenumbruch, sondern kann auch Kommentar-Markup in Form des Befehls comment enthalten.

```
[comment { ... }]
[manpage_begin NAME SECTION VERSION]
[copyright {YEAR AUTHOR}]
[titledesc TITLE]
[moddesc   MODULE_TITLE][comment { ... }]
[require   PACKAGE VERSION]
[require   PACKAGE]
[description]
[manpage_end]
[comment { ... }]
```

Erweiterte Struktur

In den einfachen Beispielen des letzten Abschnitts waren wir hinsichtlich des Markups, das tatsächlich vor dem Befehl **manpage_begin** zum Öffnen des Dokuments verwendet werden darf, etwas ungenau.

Anstelle von reinen Leerzeichen sind auch die beiden Vorlagenbefehle include und vset zulässig, damit der Verfasser dokumentrelevante Konfigurationseinstellungen festlegen und/oder importieren kann. Das heißt, es ist möglich, Folgendes zu schreiben:

```
[include FILE]
[vset VAR VALUE]
[manpage_begin NAME SECTION VERSION]
[description]
[manpage_end]
```

Noch wichtiger ist, dass diese beiden Befehle überall dort zulässig sind, wo ein Markup-Befehl zulässig ist, unabhängig von jeglicher anderen Struktur. Das heißt, beispielsweise auch im Kopfbereich.

```
[manpage_begin NAME SECTION VERSION]
[include FILE]
[vset VAR VALUE]
[description]
[manpage_end]
```

Die einzige Einschränkung, die include beachten muss, besteht darin, dass der Inhalt der eingebundenen Datei an der Stelle der Einbindung gültig sein muss. Das heißt, eine Datei, die vor **manpage_begin** eingebunden wird, darf nur die Template-Befehle vset und include enthalten; eine in den Kopf eingebundene Datei darf nur Kopfbefehle enthalten usw.

Textstruktur

Der Hauptteil des Dokuments besteht hauptsächlich aus Text, der möglicherweise in Abschnitte, Unterabschnitte und Absätze unterteilt ist, wobei bestimmte Teile zur Hervorhebung verschiedener semantischer Textkategorien markiert sind und durch die Verwendung von Beispielen und (verschachtelten) Listen eine zusätzliche Struktur entsteht.

In diesem Abschnitt werden die übergeordneten Strukturbefehle erläutert; alle weiteren Aspekte werden in den folgenden Abschnitten behandelt.

Die einfachste Möglichkeit, den Haupttext zu gliedern, ist die Einfügung von Absätzen. Der entsprechende Befehl lautet para. Jedes Auftreten dieses Befehls schließt den vorherigen Absatz und eröffnet automatisch den nächsten. Der erste Absatz wird am Anfang des Haupttextes automatisch durch den Befehl description eröffnet. Auf dieselbe Weise endet der letzte Absatz automatisch mit dem Befehl **manpage_end**.

```
[manpage_begin NAME SECTION VERSION]
[description]
 ...
[para]
 ...
[para]
 ...
[manpage_end]
```

Leere Absätze werden ignoriert.

Eine Struktur, die grober gegliedert ist als Absätze, sind Abschnitte, die es dem Verfasser ermöglichen, ein Dokument in größere, beschriftete Teile zu unterteilen. Der Befehl hierfür lautet **section**. Jedes Auftreten dieses Befehls schließt den vorherigen Abschnitt und eröffnet automatisch den nächsten, einschließlich dessen ersten Absatzes. Der erste Abschnitt wird am Anfang des Haupttextes automatisch durch den Befehl description eröffnet (dieser Abschnitt trägt die Bezeichnung "DESCRIPTION"). Auf dieselbe Weise endet der letzte Abschnitt automatisch mit dem Befehl **manpage_end**.

Leere Abschnitte werden *nicht* ignoriert. Es steht uns frei, innerhalb von Abschnitten Absätze zu verwenden oder darauf zu verzichten.

```
[manpage_begin NAME SECTION VERSION]
[description]
 ...
[section {Section A}]
 ...
[para]
 ...
[section {Section B}]
 ...
[manpage_end]
```

Zwischen Abschnitten und Absätzen befinden sich Unterabschnitte, um Abschnitte zu unterteilen. Der Befehl hierfür lautet subsection. Jedes Vorkommen dieses Befehls schließt den vorherigen Unterabschnitt und eröffnet automatisch den nächsten, einschließlich dessen ersten Absatzes. Ein Unterabschnitt wird automatisch am Anfang des Haupttextes durch den Befehl description sowie am Anfang jedes Abschnitts eröffnet. Ebenso endet der letzte Unterabschnitt automatisch mit dem Befehl **manpage_end**.

Leere Unterabschnitte werden *nicht* ignoriert. Es steht uns frei, innerhalb von Unterabschnitten Absätze zu verwenden oder darauf zu verzichten.

```
[manpage_begin NAME SECTION VERSION]
[description]
 ...
[section {Section A}]
 ...
[subsection {Sub 1}]
 ...
[para]
 ...
[subsection {Sub 2}]
 ...
[section {Section B}]
 ...
[manpage_end]
```

Textmarkierung

Nachdem wir uns mit der Gesamtstruktur befasst haben, die ein Autor dem Dokument auferlegen kann, wollen wir uns nun den Text innerhalb eines Absatzes genauer ansehen.

Zwar handelt es sich dabei meist lediglich um den schmucklosen Inhalt des Dokuments, doch gibt es Fälle, in denen wir bestimmte Teile davon als eine bestimmte Art von Sache hervorheben möchten, wie beispielsweise Befehlsargumente, Befehlsnamen, Konzepte, URIs usw.

Hierfür steht uns eine Reihe von Markup-Befehlen zur Verfügung, die den hervorzuhebenden Text als einziges Argument entgegennehmen. Es sei darauf hingewiesen, dass diese Befehle zwar in erster Linie zum Hervorheben von Teilen eines Absatzes dienen, jedoch auch zur Markierung der Argumente von Listenelement-Befehlen und anderer Markup-Befehle verwendet werden können.

Die uns zur Verfügung stehenden Befehle sind

arg
Sein Argument ist der Name eines Befehlsarguments.

class
Sein Argument ist der Name einer Klasse.

cmd
Sein Argument ist ein Befehlsname (Tcl-Befehl).

const
Sein Argument ist eine Konstante.

emph
Allgemeine, nicht-semantische Betonung.

file
Als Argument wird ein Dateiname bzw. ein Pfad übergeben.

fun
Sein Argument ist ein Funktionsname.

method
Sein Argument ist der Name einer Methode

namespace
Sein Argument ist der Name des Namespaces.

opt
Sein Argument ist ein optionales Syntaxelement.

option
Sein Argument ist ein Befehlszeilenschalter bzw. eine Widget-Option.

package
Sein Argument ist der Name eines Pakets.

sectref
Sein Argument ist der Titel eines Abschnitts oder Unterabschnitts, d. h. ein Verweis auf einen Abschnitt.

syscmd
Als Argument wird der Name eines Befehls (externer Systembefehl) übergeben.

term
Sein Argument ist ein Begriff oder eine allgemeine Bezeichnung.

type
Sein Argument ist ein Typname.

uri
Sein Argument ist ein Uniform Resource Identifier, d. h. ein externer Verweis. Mit einem zweiten Argument kann eine explizite Bezeichnung für den betreffenden Verweis angegeben werden.

usage
Die Argumente beschreiben die Syntax eines Tcl-Befehls.

var
Sein Argument ist eine Variable.

widget
Als Argument wird der Name eines Widgets übergeben.

Das Beispiel zur Veranschaulichung der Verwendung von Textmarkierungen ist ein Auszug aus der *doctools-Sprachbefehlsreferenz*, wobei einige Stellen hervorgehoben wurden. Es zeigt deren Verwendung innerhalb eines Textblocks, als Argumente eines Befehls für Listenelemente (call) sowie die Möglichkeit, diese zu verschachteln.

```
...
[call [cmd arg_def] [arg type] [arg name] [opt [arg mode]]]

Text structure. List element. Argument list. Automatically closes the
previous list element. Specifies the data-[arg type] of the described
argument of a command, its [arg name] and its i/o-[arg mode]. The
latter is optional.
...
```

Escapes

Neben den 20 Befehlen für einfache Auszeichnung, die im vorigen Abschnitt vorgestellt wurden, stehen zwei weitere zur Verfügung, bei denen es sich technisch gesehen ebenfalls um einfache Auszeichnung handelt. Ihre Funktion besteht jedoch nicht darin, Phrasen als bestimmte Arten von Dingen zu markieren, sondern Zeichen einzufügen, nämlich **[** und **]**. Diese Befehle, nämlich lb bzw. rb, sind erforderlich, da unsere Verwendung von [und] zur Einfassung von Markup-Befehlen die direkte Verwendung von [und] innerhalb des Textes unmöglich macht.

Als Beispiel für ihre Verwendung dient der Quelltext des letzten Satzes im vorangegangenen Absatz, wobei einige Stellen hervorgehoben wurden.

```
...
These commands, [cmd lb] and [cmd lb] respectively, are required
because our use of [lb] and [rb] to bracket markup commands makes it
impossible to directly use [lb] and [rb] within the text.
...
```

Querverweise

Die letzten beiden Befehle, die wir noch behandeln müssen, dienen der Deklaration von Querverweisen zwischen Dokumenten – sowohl expliziten als auch impliziten. Es handelt sich dabei um keywords und **see_also**. Beide akzeptieren eine beliebige Anzahl von Argumenten, die alle aus einfachem, unmarkiertem Text bestehen müssen. Das heißt, es ist nicht zulässig, Markup-Elemente darin zu verwenden. Beide Befehle können in einem Dokument mehrfach verwendet werden. In diesem Fall werden alle Argumente aller Vorkommen eines der beiden Befehle zu einer einzigen Menge zusammengefasst.

keywords
: Die Argumente dieses Befehls werden als Schlüsselwörter interpretiert, die das Dokument beschreiben. Ein Prozessor kann diese Informationen nutzen, um einen Index zu erstellen, der das betreffende Dokument indirekt mit allen Dokumenten verknüpft, die dieselben Schlüsselwörter enthalten.

see_also
: Die Argumente dieses Befehls werden als Verweise auf andere Dokumente interpretiert. Ein Prozessor kann sie als direkte Links zu diesen Dokumenten formatieren.

Alle Querverweis-Befehle können an beliebiger Stelle im Dokument zwischen **manpage_begin** und **manpage_end** vorkommen. Somit kann die Verfasserin entscheiden, ob sie diese am Anfang des Hauptteils oder an dessen Ende platzieren möchte, vielleicht in der Nähe der Stelle, an der ein Schlüsselwort im Hauptinhalt tatsächlich definiert wird, oder ob sie sie als Metadaten betrachtet, die im Kopf stehen sollten, usw.

Unser Beispiel zeigt den Quelltext der Querverweise dieses Dokuments, wobei einige Stellen hervorgehoben wurden. Sie befinden sich übrigens am Ende des Hauptteils.

```
...
[see_also doctools_intro]
[see_also doctools_lang_syntax]
[see_also doctools_lang_cmdref]
[keywords markup {semantic markup}]
[keywords {doctools markup} {doctools language}]
[keywords {doctools syntax} {doctools commands}]
[manpage_end]
```

Beispiele

Überall dort, wo wir einfachen Text eingeben können, können wir auch Beispiele eingeben. Für einfache Beispiele steht uns der Befehl **example** zur Verfügung, der ein einziges Argument erwartet, nämlich den Text des Arguments. Der Beispieltext darf keine Markups enthalten. Wenn wir Markups innerhalb eines Beispiels verwenden möchten, müssen wir stattdessen die Kombination aus den beiden Befehlen **example_begin** / **example_end** verwenden.

Der erste Befehl öffnet einen Beispielblock, der andere schließt ihn, und dazwischen können wir einfachen Text schreiben und alle üblichen Befehle zur Textauszeichnung verwenden. Zu beachten ist, dass Befehle zur Textstruktur nicht zulässig sind. Dies bedeutet auch, dass es nicht möglich ist, Beispiele und Listen innerhalb eines Beispiels einzubetten. Andererseits *können* wir innerhalb von Beispielblöcken Template-Befehle verwenden, um deren Inhalt aus einer Datei zu lesen (siehe Abschnitt Erweiterte Struktur).

Die Quelle für das allererste Beispiel in diesem Dokument (siehe Abschnitt Grundlagen), mit einigen Hervorhebungen, lautet:

```
[example {
  ... [list_begin enumerated] ...
}]
```

Bei Verwendung von **example_begin** / **example_end** würde dies wie folgt aussehen:

```
[example_begin]
  ... [list_begin enumerated] ...
[example_end]
```

Listen

Überall dort, wo wir einfachen Text schreiben können, können wir auch Listen schreiben. Die wichtigsten Befehle sind **list_begin** zum Starten einer Liste und **list_end** zum Schließen einer Liste. Der eröffnende Befehl nimmt ein Argument entgegen, das den Typ der begonnenen Liste festlegt; dies bestimmt wiederum, welche der acht vorhandenen Listenelement-Befehle

innerhalb der Liste zum Beginnen von Listenelementen zulässig sind.

Nach dem Eröffnungsbefehl ist nur Leerraum zulässig, bis der erste Listenelement-Befehl das erste Element der Liste eröffnet. Jedes Element ist eine reguläre Folge von Absätzen und wird entweder durch den nächsten Listenelement-Befehl oder durch das Ende der Liste geschlossen. Wird es durch einen Listenelement-Befehl geschlossen, eröffnet dieser Befehl automatisch das nächste Listenelement. Da ein Listenelement eine Folge von Absätzen ist, kann innerhalb eines Listenelements die gesamte reguläre Textauszeichnung verwendet werden, einschließlich Beispielen und weiterer Listen.

Die von **list_begin** unterstützten Listentypen und die zugehörigen Befehle für Listenelemente lauten wie folgt:

arguments
(**arg_def**) Hiermit wird eine *Argumentliste (Deklaration)* geöffnet. Es handelt sich um eine spezielle Form einer Begriffsdefinitionsliste, bei der der Begriff ein Argumentname ist, zusammen mit seinem Typ und seinem E/A-Modus.

commands
(**cmd_def**) Hiermit wird eine *Befehlsliste (Deklaration)* geöffnet. Es handelt sich um eine spezielle Form einer Begriffsdefinitionsliste, bei der der Begriff ein Befehlsname ist.

definitions
(def und call) Hiermit wird eine allgemeine *Liste der Begriffsdefinitionen* geöffnet. Die durch die Listenelemente definierten Begriffe werden über die Argumente der Listenelement-Befehle festgelegt; dabei handelt es sich entweder um allgemeine Begriffe, gegebenenfalls mit Markup (def), oder um Tcl-Befehle mit ihrer Syntax (call).

enumerated
(enum) Hiermit wird eine allgemeine *nummerierte Liste* geöffnet.

itemized
(item) Hiermit wird eine allgemeine *Auflistung* geöffnet.

options
(**opt_def**) Hiermit wird eine *Optionsliste (Deklaration)* geöffnet. Es handelt sich um eine spezielle Form einer Begriffsdefinitionsliste, bei der der Begriff ein Optionsname ist, gegebenenfalls zusammen mit den Argumenten der Option.

tkoptions
(**tkoption_def**) Hiermit wird eine *Widget-Optionsliste (Deklaration)* geöffnet. Es handelt sich um eine spezielle Form einer Begriffsdefinitionsliste, bei der der Begriff der Name einer Konfigurationsoption für ein Widget ist, mit seinem Namen und seiner Klasse in der Optionsdatenbank.

Unser Beispiel ist der Quelltext der Definitionsliste im vorigen Absatz, wobei der größte Teil des Inhalts in der Mitte entfernt wurde.

```
...
[list_begin definitions]
[def [const arg]]

([cmd arg_def]) This opens an argument (declaration) list. It is a
specialized form of a definition list where the term is an argument
```

```
name, with its type and i/o-mode.

[def [const itemized]]

([cmd item])
This opens a general itemized list.

...
[def [const tkoption]]

([cmd tkoption_def]) This opens a widget option (declaration) list.
  It
is a specialized form of a definition list where the term is the name
of a configuration option for a widget, with its name and class in
  the
option database.

[list_end]
...
```

Hinweis: Eine Liste darf nicht in einem (Unter-)Abschnitt beginnen und in einem anderen enden. Anders gesagt: Wechsel des (Unter-)Abschnitts sind innerhalb von Listen und Listenelementen nicht zulässig. Ein Beispiel für dieses *unzulässige* Konstrukt ist

```
...
[list_begin itemized]
[item]
...
[section {ILLEGAL WITHIN THE LIST}]
...
[list_end]
...
```

Weiterführende Literatur

Nachdem der Leser dieses Dokument verinnerlicht hat – wobei davon ausgegangen wird, dass es sich um einen *Verfasser* von Dokumentationen handelt –, sollte er nun ausreichend gerüstet sein, um auch die formale Spezifikation der *doctools-Sprachsyntax* zu verstehen. Von nun an dient die *doctools-Sprachbefehlsreferenz* zudem als detaillierte Spezifikation und Spickzettel für alle verfügbaren Befehle und deren Syntax.

Um ein Dokument bereits während der Erstellung validieren zu können, empfiehlt es sich zudem, sich mit einer der Anwendungen zur Verarbeitung und Konvertierung von doctools-Dokumenten vertraut zu machen, das heißt entweder mit dem benutzerfreundlichen und einfachen **dtplite** von Tcllib oder mit dem äußerst flexibel konfigurierbaren **dtp** von Tclapps.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_lang_syntax
Stichwörter: doctools_lang_syntax, doctools commands, doctools language, doctools markup, doctools syntax, markup, semantic markup

Name
doctools_lang_syntax – doctools-Sprachsyntax

Beschreibung
Dieses Dokument enthält die formale Spezifikation der Syntax der doctools-Markup-Sprache, Version 1, in Backus-Naur-Form. Dieses Dokument ist als Nachschlagewerk gedacht und ergänzt die *doctools-Befehlsreferenz*. Anfängern wird empfohlen, zunächst die wesentlich informeller verfasste *Einleitung in die doctools-Sprache* zu lesen, bevor sie versuchen, dieses Dokument oder die Befehlsreferenz zu verstehen.

Grundlagen
Im weitesten Sinne ähnelt die *doctools-Auszeichnungssprache* eher LaTeX als SGML und ähnlichen Sprachen. Ein in dieser Sprache verfasstes Dokument besteht in erster Linie aus Text, in den Auszeichnungsbefehle eingebettet sind.

Jeder Markup-Befehl ist lediglich ein Tcl-Befehl, der von einem passenden Paar aus "**[**" und "**]**" umgeben ist. Welche Befehle verfügbar sind und welche Argumente sie erfordern, d. h. ihre Syntax, wird in der *doctools-Befehlsreferenz* festgelegt.

In diesem Dokument legen wir zunächst das Lexem fest und legen anschließend die Syntax fest, d. h., wie sich Text und Markup-Befehle miteinander mischen lassen.

Lexikalische Definitionen
In den im folgenden Abschnitt aufgeführten Syntaxregeln

1. steht für den gesamten Text mit Ausnahme von Markup-Befehlen.
2. Jedes "XXX" steht für den Markup-Befehl [xxx] einschließlich seiner Argumente. Jeder Markup-Befehl ist ein Tcl-Befehl, der von einem passenden Paar aus **[** und **]** umgeben ist. Innerhalb dieser Begrenzungszeichen gelten die üblichen Regeln für einen Tcl-Befehl hinsichtlich der Quotierung von Wörtern, verschachtelter Befehle, Fortsetzungszeilen usw.
3. steht für sämtlichen Text, der ausschließlich aus Leerzeichen, Zeilenumbrüchen, Tabulatoren

und dem Markup-Befehl comment besteht.

Syntax

Die hier aufgeführten Regeln legen lediglich die Syntax von doctools-Dokumenten fest. Die lexikalische Ebene der Sprache wurde im vorangegangenen Abschnitt behandelt.

Zur Syntax der (E)BNF selbst

1. Das Konstrukt { X } steht für null oder mehr Vorkommen von X.
2. Das Konstrukt [X] steht für kein oder ein Vorkommen von X.
3. Das Konstrukt LIST_BEGIN steht für den Markup-Befehl **list_begin** mit **X** als Typ-Argument.

Die Syntax:

```
manpage = defs
          MANPAGE_BEGIN
          header
          DESCRIPTION
          body
          MANPAGE_END
          { <WHITE> }

defs    = { INCLUDE | VSET | <WHITE> }

header  = { TITLEDESC | MODDESC | COPYRIGHT | REQUIRE | defs | xref }

xref    = KEYWORDS | SEE_ALSO | CATEGORY

body    = paras { SECTION    sbody  }
sbody   = paras { SUBSECTION ssbody }
ssbody  = paras

paras   = tblock { (PARA | NL) tblock }

tblock  = { <TEXT> | defs | markup | xref | an_example | a_list }

markup  = ARG     | CLASS | CMD     | CONST     | EMPH    | FILE
        | FUN     | LB    | METHOD  | NAMESPACE | OPT     | OPTION
        | PACKAGE | RB    | SECTREF | STRONG    | SYSCMD  | TERM
        | TYPE    | URI   | USAGE   | VAR       | WIDGET

example = EXAMPLE
        | EXAMPLE_BEGIN extext EXAMPLE_END

extext  = { <TEXT> | defs | markup }

a_list  = LIST_BEGIN<arguments>   argd_list   LIST_END
        | LIST_BEGIN<commands>    cmdd_list   LIST_END
        | LIST_BEGIN<definitions> def_list    LIST_END
```

```
        | LIST_BEGIN<enumerated>  enum_list   LIST_END
        | LIST_BEGIN<itemized>    item_list   LIST_END
        | LIST_BEGIN<options>     optd_list   LIST_END
        | LIST_BEGIN<tkoptions>   tkoptd_list LIST_END

argd_list   = [ <WHITE> ] { ARG_DEF      paras }
cmdd_list   = [ <WHITE> ] { CMD_DEF      paras }
def_list    = [ <WHITE> ] { (DEF|CALL)   paras }
enum_list   = [ <WHITE> ] { ENUM         paras }
item_list   = [ <WHITE> ] { ITEM         paras }
optd_list   = [ <WHITE> ] { OPT_DEF      paras }
tkoptd_list = [ <WHITE> ] { TKOPTION_DEF paras }
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools_plugin_apiref

Stichwörter: doctools_plugin_apiref, document, formatter, formatting engine, manpage, markup, semantic markup

Name

doctools_plugin_apiref – API-Referenz für doctools-Plugins

Übersicht

dt_copyright
dt_file
dt_mainfile
dt_fileid
dt_fmap *symfname*
dt_format
dt_imgdata *key extensions*
dt_imgdst *key extensions*

dt_imgsrc *key extensions*
dt_lnesting
dt_module
dt_read *file*
dt_source *file*
dt_user
ex_cappend *text*
ex_cget *varname*
ex_cis *cname*
ex_cname
ex_cpop *cname*
ex_cpush *cname*
ex_cset *varname value*
ex_lb ?*newbracket*?
ex_rb ?*newbracket*?
fmt_initialize
fmt_listvariables
fmt_numpasses
fmt_postprocess *text*
fmt_setup *n*
fmt_shutdown
fmt_varset *varname text*
fmt_plain_text *text*

Beschreibung

Dieses Dokument richtet sich an *Plugin-Entwickler*, d. h. an Entwickler, die eine *Formatierungs-Engine* für doctools für ein bestimmtes Ausgabeformat X erstellen möchten.

Es legt die Interaktion zwischen dem Paket **doctools** und dessen Plugins fest, d. h. die Schnittstelle, an die sich jede doctools-Formatierungs-Engine halten muss.

Dieses Dokument befasst sich mit Version 1 der Schnittstelle.

Ein Leser, der sich hingegen eher für die Auszeichnungssprache selbst interessiert, sollte mit der *Einleitung in die doctools-Sprache* beginnen und von dort aus zu den formalen Spezifikationen übergehen, d. h. zur *Syntax der doctools-Sprache* und zur *Befehlsreferenz der doctools-Sprache*.

Überblick

Die API für eine doctools-Formatierungs-Engine besteht aus zwei Hauptabschnitten.

Einerseits gibt es eine Reihe von Befehlen, mit denen das Plugin das Frontend abfragen kann. Diese Befehle werden vom Frontend bereitgestellt und in den Plugin-Interpreter eingebunden. Eine detaillierte Beschreibung finden Sie im Abschnitt Frontend-Befehle.

Andererseits muss das Plugin einen eigenen Satz von Befehlen bereitstellen, die dann vom Frontend bei der Verarbeitung der Eingaben in einer bestimmten Reihenfolge aufgerufen werden. Auch diese lassen sich in zwei Kategorien einteilen: Verwaltung und Formatierung. Eine detaillierte Beschreibung finden Sie im Abschnitt Plugin-Befehle und dessen Unterabschnitten.

Frontend-Befehle

In diesem Abschnitt wird die Menge der Befehle festgelegt, mit denen ein Plugin – auch als "doctools-Formatierungs-Engine" bezeichnet – das Frontend abfragen kann. Diese Befehle werden vom Frontend bereitgestellt und in den Plugin-Interpreter eingebunden.

Das heißt, eine doctools-Formatierungs-Engine kann davon ausgehen, dass alle folgenden Befehle vorhanden sind, wenn einer ihrer eigenen Befehle (wie im Abschnitt Plugin-Befehle festgelegt) ausgeführt wird.

Darüber hinaus kann davon ausgegangen werden, dass er uneingeschränkten Zugriff auf seinen eigenen sicheren Interpreter hat und somit weder die anderen Teile des Prozessors noch das Dateisystem beschädigen kann. Er kann den gesamten Prozess jedoch beenden oder hängen lassen, indem er ihn beendet oder eine Endlosschleife ausführt.

Um auf die importierten Befehle zurückzukommen: Alle Befehle mit dem Präfix *dt_* bieten eingeschränkten Zugriff auf bestimmte Teile des Frontends, während die Befehle mit dem Präfix *ex_* Zugriff auf den Zustand des Objekts **textutil::expander** gewähren, das die Hauptanalyse der Eingabe innerhalb des Frontends durchführt. Diese Befehle sollten nur unter ganz besonderen Umständen verwendet werden.

dt_copyright
: Abfragebefehl. Er gibt eine Zeichenkette zurück, die die Copyright-Informationen enthält, mit denen der doctools-Prozessor konfiguriert wurde. Die entsprechende Option lautet **-copyright**.

dt_file
: Abfragebefehl. Er gibt den vollständigen Pfad der Datei zurück, die die derzeit von der Engine verarbeitete Eingabe enthält. Dabei kann es sich um eine eingebundene Datei handeln.

dt_mainfile
: Abfragebefehl. Er gibt den vollständigen Pfad der Toplevel-Datei zurück, die die derzeit von der Engine verarbeitete Eingabe enthält.

dt_fileid
: Abfragebefehl. Er gibt den Namen der Datei zurück, deren Eingabe gerade von der Engine verarbeitet wird, ohne Pfad und ohne Erweiterung.

dt_fmap *symfname*
: Abfragebefehl. Er gibt den tatsächlichen Pfadnamen zurück, der in der Ausgabe anstelle des symbolischen Dateinamens *symfname* verwendet werden soll. Wird für *symfname* keine Zuordnung festgelegt, gibt er die unveränderte Eingabe zurück.

  Die erforderlichen Zuordnungen werden mit der Methode **map** eines Frontends eingerichtet, wie im Abschnitt **Objektmethoden** der Dokumentation zum Paket **doctools** erläutert.

dt_format
: Abfragebefehl. Er gibt den Namen des Formats zurück, das der doctools-Formatierungs-Engine zugeordnet ist.

dt_imgdata *key extensions*
: Abfragebefehl. Zugriff auf die Bildzuordnung. Sucht nach einem Bild, das unter dem *key* gespeichert ist und die angegebene *extension* aufweist. Wird ein passendes Bild gefunden, werden dessen Daten als Ergebnis des Befehls zurückgegeben. Andernfalls wird eine leere Zeichenkette zurückgegeben.

dt_imgdst *key extensions*

Abfragebefehl. Zugriff auf die Bildzuordnung. Sucht nach einem Bild, das unter dem *key* gespeichert ist und die angegebene *extension* aufweist. Wird ein passendes Bild gefunden, wird dessen Zielpfad in der Ausgabe als Ergebnis des Befehls zurückgegeben. Andernfalls wird eine leere Zeichenkette zurückgegeben.

dt_imgsrc *key extensions*
Abfragebefehl. Zugriff auf die Bildzuordnung. Sucht nach einem Bild, das unter dem *key* gespeichert ist und die angegebene *extension* aufweist. Wird ein passendes Bild gefunden, wird dessen Ursprungspfad als Ergebnis des Befehls zurückgegeben. Andernfalls wird eine leere Zeichenkette zurückgegeben.

dt_lnesting
Abfragebefehl. Er gibt die Anzahl der derzeit geöffneten Listen zurück.

dt_module
Abfragebefehl. Er gibt den Namen des Moduls zurück, zu dem die aktuell verarbeitete Eingabe gehört.

dt_read *file*
Kontrollierter Zugriff auf das Dateisystem. Gibt den Inhalt von *file* für beliebige Zwecke des Plugins zurück. Es können nur Dateien geladen werden, die sich entweder im selben Verzeichnis wie die Datei mit der Engine oder in einem darunterliegenden Verzeichnis befinden. Der Versuch, eine Datei außerhalb dieses Verzeichnisses zu laden, führt zu einem Fehler.

dt_source *file*
Kontrollierter Zugriff auf das Dateisystem. Dieser Befehl ermöglicht es der doctools-Formatierungs-Engine, zusätzlichen Tcl-Code zu laden, den sie möglicherweise benötigt. Es können nur Dateien geladen werden, die sich entweder im selben Verzeichnis wie die Datei mit der Engine oder in einem darunterliegenden Verzeichnis befinden. Der Versuch, eine Datei außerhalb dieses Verzeichnisses zu laden, führt zu einem Fehler.

dt_user
Abfragebefehl. Er gibt den Namen des aktuellen Benutzers zurück, wie er dem Tcl-Interpreter bekannt ist, in dem sich das Frontend befindet, das die Formatierungs-Engine steuert.

ex_cappend *text*
Fügt im aktuellen Kontext eine Zeichenkette an die Ausgabe an. Dieser Befehl sollte von Makros oder Anwendungscode nur selten verwendet werden.

ex_cget *varname*
Ruft den Wert der im aktuellen Kontext definierten Variablen *varname* ab.

ex_cis *cname*
Ermittelt, ob der Name des aktuellen Kontexts *cname* lautet oder nicht.

ex_cname
Gibt den Namen des aktuellen Kontexts zurück.

ex_cpop *cname*
Entnimmt einen Kontext aus dem Kontextstapel und gibt die gesamte in diesem Kontext gesammelte Ausgabe zurück. Der Kontext muss den Namen *cname* tragen; andernfalls tritt ein Fehler auf.

ex_cpush *cname*
Fügt einen Kontext mit dem Namen *cname* auf den Kontextstapel ein. Der Kontext muss vor Abschluss der Erweiterung durch **cpop** vom Stapel entfernt werden, andernfalls tritt ein

Fehler auf.

ex_cset *varname value*
Setzt die Variable *varname* im aktuellen Kontext auf den Wert *value*.

ex_lb ?*newbracket*?
Gibt den aktuellen Wert der linken Makroauswertungsklammer zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

ex_rb ?*newbracket*?
Gibt den aktuellen Wert der rechten Klammer der Makroauswertung zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

Plugin-Befehle

Das Plugin muss einen eigenen Satz von Befehlen bereitstellen, die dann vom Frontend bei der Verarbeitung der Eingaben in einer bestimmten Reihenfolge aufgerufen werden. Diese lassen sich in zwei Kategorien einteilen: Verwaltung und Formatierung. Die erwarteten Namen, Signaturen und Aufgaben dieser Befehle werden in den folgenden beiden Unterabschnitten näher beschrieben.

Verwaltungsbefehle

Die Verwaltungsbefehle, die ein Plugin bereitstellen muss, werden vom Frontend verwendet, um

1. das Plugin zu initialisieren und zu beenden
2. die Anzahl der Durchgänge zu ermitteln, die über die Eingabe laufen müssen
3. jeden Durchgang zu initialisieren und zu beenden
4. Engine-Parameter abzufragen und zu initialisieren.

Nachdem das Plugin geladen wurde und die Frontend-Befehle eingerichtet sind, werden die Befehle in der folgenden Reihenfolge aufgerufen:

```
    fmt_numpasses -> n
    fmt_listvariables -> vars

    fmt_varset var1 value1
    fmt_varset var2 value2
    ...
    fmt_varset varK valueK
    fmt_initialize
    fmt_setup 1
    ...
    fmt_setup 2
    ...
    ...
    fmt_setup n
    ...
    fmt_postprocess
    fmt_shutdown
    ...
```

Das heißt, zunächst werden die Anzahl der Durchgänge und die Menge der verfügbaren Engine-Parameter festgelegt, woraufhin Aufrufe zur Einstellung der Parameter folgen. Dieser zweite Teil ist optional.

Nachdem das Plugin initialisiert wurde, wird die angegebene Anzahl von Durchgängen ausgeführt, das Endergebnis durchläuft einen globalen Nachbearbeitungsschritt und schließlich wird das Plugin wieder beendet. Darauf können weitere Konvertierungen folgen, wobei die Sequenz bei **fmt_varset** neu gestartet wird.

In jedem Durchgang, d. h. nach dem Aufruf von **fmt_setup**, verarbeitet das Frontend die Eingabe und ruft die Formatierungsbefehle auf, sobald Markup erkannt wird. Dies bedeutet, dass die Reihenfolge der Formatierungsbefehle durch die Grammatik der doctools-Markup-Sprache bestimmt wird, wie sie in der Spezifikation *Syntax der doctools-Sprache* festgelegt ist.

Eine andere Sichtweise auf die Abfolge ist folgende:

- Zunächst werden einige grundlegende Parameter festgelegt.
- Dann bildet alles, beginnend mit dem ersten "**fmt_varset**" bis hin zu "**fmt_shutdown**", einen *Lauf*, d. h. die Formatierung einer einzelnen Eingabe. Auf jeden Lauf können weitere folgen.
- In jeden Lauf sind ein oder mehrere *Durchgänge* eingebettet, die jeweils mit **fmt_setup** beginnen und so lange andauern, bis entweder das nächste **fmt_setup** oder **fmt_postprocess** erreicht ist.

  Wenn zur Durchführung der Formatierung mehrere Durchgänge erforderlich sind, ist nur die Ausgabe des letzten Durchgangs relevant. Die Ausgabe aller vorhergehenden, vorbereitenden Durchgänge wird ignoriert.

Die Befehle, ihre Namen, Signaturen und Aufgaben lauten im Einzelnen wie folgt:

fmt_initialize
: *Initialisierung/Beendigung*. Dieser Befehl wird zu Beginn jedes Konvertierungslaufs als erster Befehl dieses Laufs aufgerufen. Hinweis: Ein Lauf ist kein Durchgang, sondern kann aus

mehreren Durchgängen bestehen. Er muss den allgemeinen Zustand des Plugins initialisieren, über die beim Laden durchgeführte Initialisierung hinaus. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

fmt_listvariables
Initialisierung/Beendigung und *Engine-Parameter*. Der zweite Befehl wird aufgerufen, nachdem der Plugin-Code geladen wurde, d. h. unmittelbar nach **fmt_numpasses**. Er muss eine Liste zurückgeben, die die Namen der Parameter enthält, die das Frontend zur Konfiguration der Engine einstellen kann. Diese Liste kann leer sein.

fmt_numpasses
Initialisierung/Beendigung und *Durchgangsverwaltung*. Der erste Befehl, der nach dem Laden des Plugin-Codes aufgerufen wird. Vor diesem Befehl wird kein anderer Befehl der Engine aufgerufen. Er muss die Anzahl der Durchgänge zurückgeben, die diese Engine benötigt, um das Eingabedokument vollständig zu verarbeiten. Dieser Wert muss eine ganze Zahl sein, die größer oder gleich eins ist.

fmt_postprocess *text*
Initialisierung/Beendigung. Dieser Befehl wird unmittelbar nach dem letzten Durchgang eines Laufs aufgerufen. Sein Argument ist das Ergebnis der durch diesen Durchgang erzeugten Konvertierung. Er dient dazu, der Engine die Möglichkeit zu geben, globale Änderungen am erzeugten Dokument vorzunehmen. Ist für ein bestimmtes Format keine Nachbearbeitung erforderlich, muss der Befehl lediglich das Argument zurückgeben.

Es wird erwartet, dass ein Wert zurückgegeben wird, nämlich das Endergebnis der Formatierung der Eingabe.

fmt_setup *n*
Initialisierung/Beendigung und *Durchgangsverwaltung*. Dieser Befehl wird zu Beginn jedes Durchgangs über die Eingabe in einem Lauf aufgerufen. Sein Argument ist die Nummer des Durchgangs, der gerade begonnen hat. Die Durchgänge werden ab **1** aufwärts gezählt. Der Befehl muss den internen Zustand des Plugins für diesen bestimmten Durchgang einrichten. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

fmt_shutdown
Initialisierung/Beendigung. Dieser Befehl wird am Ende jedes Konvertierungslaufs aufgerufen. Er ist der letzte Befehl, der in einem Lauf ausgeführt wird. Er muss alle laufspezifischen Zustände im Plugin bereinigen. Nach dem Aufruf muss sich die Engine in einem Zustand befinden, der den Start eines weiteren Laufs ermöglicht, ohne dass Informationen aus dem letzten Lauf in diesen neuen Lauf übergreifen. Es wird kein Rückgabewert erwartet, und jeder zurückgegebene Wert wird ignoriert.

fmt_varset *varname text*
Engine-Parameter. Dieser Befehl wird vom Frontend aufgerufen, um einen Engine-Parameter auf einen bestimmten Wert zu setzen. Der zu ändernde Parameter wird durch *varname* angegeben, der einzustellende Wert durch *text*.

Der Befehl muss einen Fehler auslösen, wenn ein unbekannter *varname* verwendet wird. Nur die von **fmt_listvariables** zurückgegebenen Namen gelten als bekannt.

Die Werte aller Engine-Parameter müssen über Durchgänge und Läufe hinweg erhalten

bleiben.

Formatierungsbefehle

Die Formatierungsbefehle müssen die Formatierung für das Ausgabeformat für alle Markup-Befehle der doctools-Markup-Sprache implementieren, mit Ausnahme von lb, rb, vset, include und comment. Diese Ausnahmen werden vom Frontend verarbeitet und sind für das Plugin nicht sichtbar. Im Gegenzug muss ein Befehl zur Formatierung von Klartext bereitgestellt werden, für etwas, das in der Eingabe überhaupt kein Markup enthält.

Das bedeutet, dass jeder der 49 Markup-Befehle, die in der *doctools-Sprachbefehlsreferenz* aufgeführt sind und nicht zu den oben genannten Ausnahmen gehören, einen entsprechenden Formatierungsbefehl besitzt, der dieselben Argumente wie der Markup-Befehl akzeptiert und dessen Name dem Namen des Markup-Befehls mit dem vorangestellten Präfix *fmt_* entspricht.

Von allen Befehlen wird erwartet, dass sie ihre Eingabe gemäß der in der Befehlsreferenz festgelegten Semantik in irgendeiner Weise formatieren und den Teil davon, den sie für notwendig erachten, als Ergebnis zurückgeben, das der Ausgabe hinzugefügt wird.

Um eine im Wesentlichen doppelte Darstellung der Befehlsreferenz zu vermeiden, führen wir hier keinen der Befehle auf, sondern verweisen den Leser lediglich auf die *Befehlsreferenz der doctools-Sprache*, in der deren Signatur und Beschreibung zu finden sind. Die einzige Ausnahme bildet der Formatierer für einfachen Text, für den es keinen entsprechenden Markup-Befehl gibt.

Die Reihenfolge, in der Formatierungsbefehle aufgerufen werden, ist nicht so streng festgelegt wie bei den Verwaltungsbefehlen, sondern richtet sich nach der Grammatik der doctools-Markup-Sprache, wie sie in der Spezifikation *Syntax der doctools-Sprache* festgelegt ist.

fmt_plain_text *text*

Kein zugehöriger Markup-Befehl.

Wird vom Frontend für jeden im Eingabetext vorkommenden Klartext aufgerufen. Er muss alle für Klartext erforderlichen speziellen Verarbeitungsschritte durchführen.

Als Ergebnis des Befehls wird der formatierte Text erwartet, der der Ausgabe hinzugefügt wird. Ist keine spezielle Verarbeitung erforderlich, muss er sein Argument einfach unverändert zurückgeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2007-2010 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


mpexpand

Stichwörter: mpexpand, HTML, TMML, conversion, manpage, markup, nroff

Name

mpexpand – Markup-Prozessor

Übersicht

mpexpand ?*-module module*? *format infile*|- *outfile*|-
mpexpand.all ?*-verbose*? ?*module*?

Beschreibung

Diese Manpage beschreibt einen Prozessor bzw. Konverter für Manpages im doctools-Format, wie in **doctools_fmt** festgelegt. Der Prozessor basiert auf dem Paket **doctools**.

mpexpand ?-module *module*? *format infile*|- *outfile*|-

Der Prozessor nimmt drei Argumente entgegen: den Code, der beschreibt, welche Formatierung als Ausgabe erzeugt werden soll, die Datei, aus der die Auszeichnung gelesen wird, und die Datei, in die die erzeugte Ausgabe geschrieben wird. Wenn *infile* "**-**" ist, liest der Prozessor von **stdin**. Wenn *outfile* "**-**" ist, schreibt der Prozessor nach **stdout**.

Wenn die Option *-module* vorhanden ist, überschreibt ihr Wert die interne Definition des Modulnamens.

Die derzeit bekannten Ausgabeformate sind

nroff
: Der Prozessor erzeugt eine *roff-Ausgabe, das Standardformat für Unix-Manpages.

html
: Der Prozessor erzeugt HTML-Ausgabe zur Verwendung in Webbrowsern und zur Anzeige durch sie.

tmml
: Der Prozessor erzeugt TMML-Ausgaben, die Tcl Manpage Markup Language, eine Ableitung von XML.

latex
: Der Prozessor erzeugt eine LaTeX-Ausgabe.

wiki
: Der Prozessor erzeugt Wiki-Markup, wie es **wikit** versteht.

list
: Der Prozessor extrahiert die von **manpage_begin** bereitgestellten Informationen.

null
: Der Prozessor erzeugt keine Ausgabe.

mpexpand.all ?*-verbose*? ?*module*?

Dieser Befehl verwendet **mpexpand**, um alle möglichen Ausgabeformate für alle Manpages

im aktuellen Verzeichnis zu generieren. Die Manpages werden anhand der Erweiterung "*.man* " erkannt. Wenn *-verbose* angegeben ist, listet der Befehl seine Aktionen auf, bevor er sie ausführt.

Die *module*-Angabe wird an **mpexpand** übergeben.

Hinweise
Mögliche zukünftige Formate sind Klartext, PDF und PostScript.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2003 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools2base

html_cssdefaults
Stichwörter: html_cssdefaults, CSS, HTML, doctools, export, plugin, style

Name
doctools::html::cssdefaults – Standard-CSS-Stil für HTML-Export-Plugins

Übersicht
package require Tcl 8.5 9
package require doctools::html::cssdefaults ?**0.2**?
::doctools::html::cssdefaults::contents

Beschreibung
Dieses Paket stellt einen einzigen Befehl bereit, der den Zugriff auf den Text des Standard-CSS-Stils ermöglicht, der für das von den verschiedenen HTML-Export-Plugins erzeugte HTML-Markup verwendet werden soll.

Hierbei handelt es sich um ein internes Paket von doctools, das für *Export*-Plugins bestimmt ist,

d. h. für die Pakete, die doctools-bezogene Dokumentation in andere Formate konvertieren, insbesondere in *HTML*.

API

::doctools::html::cssdefaults::contents

Dieser Befehl gibt den Text des Standard-CSS-Stils zurück, der für das von den verschiedenen HTML-Export-Plugins generierte HTML-Markup verwendet werden soll.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nroff_manmacros

Stichwörter: nroff_manmacros, doctools, export, macros, man_macros, nroff, plugin

Name

doctools::nroff::man_macros – Definition des nroff-man-Makrosatzes für NROFF-Export-Plugins

Übersicht

package require Tcl 8.5 9
package require doctools::nroff::man_macros ?**0.2**?
::doctools::nroff::man_macros::contents

Beschreibung

Dieses Paket stellt einen einzigen Befehl bereit, der Zugriff auf die Definition des nroff-*man* -Makrosatzes gewährt, der für das von den verschiedenen NROFF-Export-Plugins generierte NROFF-Markup verwendet werden soll.

Hierbei handelt es sich um ein internes Paket von doctools, das für *Export*-Plugins bestimmt ist, d. h. für die Pakete, die doctools-bezogene Dokumentation in andere Formate konvertieren, insbesondere *nroff*.

API

::doctools::nroff::man_macros::contents

Dieser Befehl gibt den Text der Definition des nroff-*man*-Makrosatzes zurück, der für das von

den verschiedenen NROFF-Export-Plugins erzeugte NROFF-Markup verwendet werden soll.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcl_parse

Stichwörter: tcl_parse, Tcl syntax, command, doctools, parser, subst, word

Name

doctools::tcl::parse – Verarbeitung von Text im Format "subst -novariables"

Übersicht

package require Tcl 8.5 9
package require snit
package require fileutil
package require logger
package require struct::list
package require struct::stack
package require struct::set
package require treeql
package require doctools::tcl::parse ?**0.2**?
::doctools::tcl::parse text *tree text* ?*root*?
::doctools::tcl::parse file *tree path* ?*root*?

Beschreibung

Dieses Paket stellt Befehle zum Parsen von Text mit eingebetteten Tcl-Befehlen bereit, wie sie vom Tcl-Befehl "**subst -novariables**" verarbeitet werden. Das Ergebnis des Parsens ist ein abstrakter Syntaxbaum.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Parsern zur Verarbeitung der Markup-Sprachen *docidx*, *doctoc* und *doctools* verwendet wird.

API

::doctools::tcl::parse text *tree text* ?*root*?

Der Befehl nimmt den *text* und parst ihn unter der Annahme, dass er eine Zeichenkette enthält, die der eingebaute Tcl-Befehl **subst -novariables** annimmt. Andernfalls wird beim Parsen ein Fehler ausgelöst. Das für diese Fehler verwendete Format ist im Abschnitt Fehlerformat beschrieben.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück. Das eigentliche Ergebnis des Parsens wird in die Baumstruktur *tree* unter dem Knoten *root* eingetragen. Wird *root* nicht angegeben, wird die Wurzel von *tree* verwendet. *tree* muss vorhanden sein und der Befehl eines Baumobjekts sein, das dieselben Methoden unterstützt wie die vom Paket **struct::tree** erzeugten Bäume.

Im Fehlerfall bleibt *tree* in einem undefinierten Zustand.

::doctools::tcl::parse file *tree path* ?*root*?

Entspricht **text**, mit dem Unterschied, dass der zu analysierende Text aus der durch *path* angegebenen Datei gelesen wird.

Fehlerformat

Wenn der Parser ein Problem in der Eingabe feststellt, löst er einen Fehler in dem hier beschriebenen Format aus.

1. Die Meldung enthält den Grund für das Problem (unerwartetes Zeichen oder Ende der Eingabe), gegebenenfalls das betreffende Zeichen sowie die Zeile und Spalte, in der das Problem festgestellt wurde – und zwar in einer für Menschen lesbaren Form. Dieser Teil wird nicht näher dokumentiert, da sich sein Format nach unserem Ermessen ändern kann. Er ist für Menschen bestimmt, nicht für Maschinen.
2. Der Fehlercode enthält jedoch eine maschinenlesbare Darstellung des Problems in Form einer Liste mit fünf Elementen, die in der unten aufgeführten Reihenfolge Folgendes enthält:
   1. die konstante Zeichenkette **doctools::tcl::parse**
   2. die Ursache des Problems, eine von

      char
      : Unerwartetes Zeichen in der Eingabe

      eof
      : Unerwartetes Ende der Eingabe

1. Die Position des Problems, gemessen als Versatz vom Anfang der Eingabe, in Zeichen. Hinweis: Zeilenmarkierungen zählen als ein Zeichen.
2. Die Zeile, in der das Problem festgestellt wurde (gezählt ab 1 (eins)),
3. Die Spalte, in der das Problem festgestellt wurde (gezählt ab 0 (Null))

Baumstruktur

Nach der erfolgreichen Analyse einer Zeichenkette weist der generierte Baum folgende Struktur auf:

1. In den folgenden Punkten bezieht sich das Wort "Wurzel" auf den Knoten, der beim Aufruf von **text** oder **file** als Wurzel des Baums festgelegt wurde. Dabei kann es sich um die tatsächliche Wurzel des Baums handeln.
2. Bei allen folgenden Punkten wird zudem die Möglichkeit bereits vorhandener Attribute in den bestehenden Knoten außer Acht gelassen. Wenn Attribute mit denselben Namen wie die vom

Parser verwendeten Attribute vorhanden sind, werden die bereits vorhandenen Werte überschrieben. Attribute, deren Namen nicht mit den Attributen des Parsers kollidieren, bleiben unberührt.

3. Der Wurzelknoten weist keine Attribute auf.
4. Alle anderen Knoten verfügen über die folgenden Attribute

   type
   Der Wert ist eine Zeichenkette aus der Menge { Command, Text, Word }

   range
   Der Wert ist entweder leer oder eine Liste mit zwei Elementen, die ganzzahlige Werte enthält. Die Zahlen geben die Offsets des ersten und letzten Zeichens im Eingabetext des durch den Knoten beschriebenen Tokens an.

   line
   Der Wert ist eine ganze Zahl und gibt die Zeile in der Eingabe an, in der das durch den Knoten beschriebene Token endet. Die Zeilen werden ab 1 (**eins**) gezählt.

   col
   Der Wert ist eine ganze Zahl und gibt die Spalte in der Zeile der Eingabe an, in der das durch den Knoten beschriebene Token endet. Die Spalten werden ab 0 gezählt (**null**).

1. Die Kindknoten der Wurzel, sofern vorhanden, sind vom Typ Command und Text, in Halbalternation. Das bedeutet: Auf einen Text-Knoten muss ein Command-Knoten folgen, und auf einen Command-Knoten kann alles folgen, ein Text- oder ein weiterer Command-Knoten.
2. Die Kindknoten eines Command-Knotens, sofern vorhanden, sind vom Typ Command, Text und Word; sie beschreiben die Argumente des Befehls.
3. Die Kindknoten eines Word-Knotens, sofern vorhanden, sind vom Typ Command und Text, in Halbalternation. Das bedeutet: Auf einen Text-Knoten muss ein Command-Knoten folgen, und auf einen Command-Knoten kann alles folgen, ein Text- oder ein weiterer Command-Knoten.
4. Ein Word-Knoten ohne Kindknoten steht für die leere Zeichenkette.
5. Alle Text-Knoten sind Blätter des Baums.
6. Alle Blätter des Baums sind entweder Text- oder Command-Knoten. Word-Knoten können keine Blätter sein.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_msgcat

Stichwörter: tcllib_msgcat, catalog package, docidx, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat – Verwaltung der Nachrichtenkataloge für die verschiedenen Dokumentenparser

Übersicht

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat ?**0.2**?
::doctools::msgcat::init *prefix*

Beschreibung

Das Paket **doctools::msgcat** ist ein Unterstützungsmodul, das die Auswahl von Nachrichtenkatalogen für die verschiedenen Dokumentenverarbeitungspakete im doctools-System der Version 2 übernimmt. Als solches handelt es sich um ein internes Paket, mit dem ein normaler Benutzer (Entwickler) nicht in direkten Kontakt kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

Innerhalb der Systemarchitektur liegt dieses Paket unter den verschiedenen Parser-Paketen und wird von ihnen gemeinsam genutzt. Darunter liegen die Pakete, die die Nachrichtenkataloge für die verschiedenen Sprachen bereitstellen; sie sind jedoch keine expliziten Abhängigkeiten.

API

::doctools::msgcat::init ***prefix***

Der Befehl sucht die Nachrichtenkataloge für alle von **msgcat::mcpreferences** zurückgegebenen Sprachen und lädt sie, sofern sie gefunden werden konnten. Er gibt eine ganze Zahl zurück, die angibt, wie viele Pakete gefunden und geladen wurden.

Die Namen der Pakete, nach denen der Befehl sucht, haben die Form "doctools::msgcat::*prefix*::**langcode**", wobei *prefix* das Argument des Befehls ist und der Wert **langcode** aus dem Ergebnis von **msgcat::mcpreferences** stammt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools2idx

export_docidx
Stichwörter: export_docidx, docidx, doctools, export, index, serialization

Name
doctools::idx::export::docidx – docidx-Export-Plugin

Übersicht
package require Tcl 8.5 9
package require doctools::idx::export::docidx ?**0.2.2**?
export *serial configuration*

Beschreibung
Dieses Paket implementiert das doctools-Export-Plugin für Schlüsselwortindizes zur Erzeugung von docidx-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::export**, dem Exportmanager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur unter Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::export** und die darin bereitgestellten Exportmanager-Objekte.

API
Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Schlüsselwortindexes entgegen, wie sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes festgelegt ist und in *serial* enthalten ist, sowie die *configuration* (ein Dictionary), und erzeugt docidx-Markup, das den Index kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

[docidx]-Notation von Schlüsselwortindizes

Das docidx-Format für Schlüsselwortindizes, auch als *docidx-Markup-Sprache* bezeichnet, ist zu umfangreich, um in einem einzigen Abschnitt behandelt zu werden. Der interessierte Leser sollte mit dem Dokument beginnen

1. *Einleitung in die docidx-Sprache*

und von dort zu den formalen Spezifikationen weitergehen, d. h. den Dokumenten

1. *Syntax der docidx-Sprache* und
2. *Befehlsreferenz der docidx-Sprache*,

um die Sprache gründlich zu verstehen.

Konfiguration

Das docidx-Export-Plugin erkennt die folgenden Variablen der Konfiguration und passt sein Verhalten entsprechend der dort festgelegten Einstellungen an.

string *user*
: Diese Standard-Konfigurationsvariable enthält den Namen des Benutzers, der den Prozess ausführt, welcher das Export-Plugin aufgerufen hat. Das Plugin schreibt diese Angabe in den Herkunftskommentar am Anfang des erzeugten Dokuments.

string *file*
: Diese Standardkonfigurationsvariable enthält den Namen der Datei, aus der der Index stammt. Diese Variable kann ungesetzt sein oder die leere Zeichenkette enthalten. Das Plugin fügt diese Information, sofern sie definiert ist, d. h. gesetzt ist und nicht die leere Zeichenkette enthält, in den Herkunftskommentar am Anfang des generierten Dokuments ein.

boolean *newlines*
: Ist dieses Flag gesetzt, teilt das Plugin den generierten docidx-Code zeilenweise auf, wobei jeder Markup-Befehl in einer eigenen Zeile steht.

  Ist dieses Flag nicht gesetzt (Standardeinstellung), wird das gesamte Dokument in einer einzigen Zeile geschrieben, wobei der Abstand zwischen allen Elementen so gering wie möglich gehalten wird.

boolean *indented*
: Ist dieses Flag gesetzt, rückt das Plugin die Markup-Befehle entsprechend der Struktur der Indizes ein. Damit dies funktioniert, muss zudem **newlines** gesetzt sein. Dieser Effekt ist jedoch unabhängig vom Wert für **aligned**.

  Wenn dieses Flag nicht gesetzt ist (Standardeinstellung), wird die Ausgabe gemäß den Werten von **newlines** und **aligned** formatiert, und es erfolgt keine Einrückung.

boolean *aligned*
: Ist dieses Flag gesetzt, sorgt der Generator dafür, dass die Argumente für die Befehle **manpage** und **url** in einem Abschnitt mit Schlüsselwörtern vertikal ausgerichtet werden, um einen ansprechenden Tabelleneffekt zu erzielen. Damit dies funktioniert, muss zudem **newlines** gesetzt sein. Dieser Effekt ist jedoch unabhängig vom Wert für **indented**.

  Ist dieses Flag nicht gesetzt (Standardeinstellung), wird die Ausgabe gemäß den Werten von **newlines** und **indented** formatiert, und es erfolgt keine Ausrichtung.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **format** und **map** sowie ihre Werte.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung

Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_container

Stichwörter: idx_container, HTML, TMML, conversion, docidx markup, documentation, formatting, generation, index, json, keyword index, latex, manpage, markup, nroff, parsing, plugin, reference , tcler's wiki, text, url, wiki

Name

doctools::idx – Halten von Schlüsselwortindizes

Übersicht

package require doctools::idx ?**2.1**?
package require Tcl 8.5 9
package require doctools::idx::structure
package require snit
::doctools::idx *objectName*
objectName method ?*arg arg* ...?
objectName **destroy**
objectName **key add** *name*
objectName **key remove** *name*
objectName **key references** *name*

objectName **keys**
objectName **reference add** *type key name label*
objectName **reference remove** *name*
objectName **reference label** *name*
objectName **reference keys** *name*
objectName **reference type** *name*
objectName **references**
objectName **title**
objectName **title** *text*
objectName **label**
objectName **label** *text*
objectName **importer**
objectName **importer** *object*
objectName **exporter**
objectName **exporter** *object*
objectName **deserialize =** *data* ?*format*?
objectName **deserialize +=** *data* ?*format*?
objectName **serialize** ?*format*?

Beschreibung

Dieses Paket stellt eine Klasse zum Aufnehmen und programmatischen Bearbeiten von Schlüsselwortindizes bereit.

Dies ist eine der drei öffentlichen Säulen, auf denen die Verwaltung von Schlüsselwortindizes ruht. Die beiden anderen Säulen sind

1. *Export von Schlüsselwortindizes* sowie
2. *Import von Schlüsselwortindizes*

Informationen über Begriffe von Schlüsselwortindizes und ihre Bestandteile stehen im gleichnamigen Abschnitt. Informationen über die Datenstruktur, mit der Schlüsselwortindizes als Werte kodiert werden, stehen im Abschnitt Serialisierungsformat für Schlüsselwortindizes. Dies ist das einzige Format, das dieser Klasse direkt bekannt ist. Konvertierungen von und in jedes andere Format werden von Export- und Import-Manager-Objekten übernommen. Diese können einem Container zugeordnet werden, müssen es jedoch nicht; das ist lediglich eine Erleichterung.

Begriffe

1. Ein *Schlüsselwortindex* besteht aus einer (möglicherweise leeren) Menge von *Schlüsselwörtern*.
2. Jedes Schlüsselwort in der Menge wird durch seinen Namen identifiziert.
3. Jedes Schlüsselwort verfügt über eine (möglicherweise leere) Menge von *Referenzen*.
4. Eine Referenz kann mit mehr als einem Schlüsselwort verknüpft sein.
5. Eine Referenz, die nicht mit mindestens einem Schlüsselwort verknüpft ist, ist jedoch nicht möglich.
6. Jede Referenz wird durch ihr Ziel identifiziert, das je nach Referenztyp (**url** oder **manpage**) entweder als URL oder als symbolischer Dateiname angegeben wird.
7. Der Typ einer Referenz (URL oder manpage) hängt nur von der Referenz selbst ab und nicht von den Schlüsselwörtern, denen sie zugeordnet ist.

8. Zusätzlich zu einem Typ verfügt jede Referenz auch über eine beschreibende Bezeichnung. Diese Bezeichnung hängt ausschließlich von der Referenz selbst ab und nicht von den Schlüsselwörtern, denen sie zugeordnet ist.

Einige Hinweise

1. manpage-Referenzen sind für Verweise auf die Dokumente gedacht, für die der Index erstellt wurde. Ihr Ziel ist ein symbolischer Dateiname, der das Dokument identifiziert, und Export-Plugins können symbolische durch tatsächliche Dateinamen ersetzen, wenn das festgelegt ist.
2. URL-Referenzen hingegen sind für Verweise auf alles andere gedacht, etwa Websites. Ihr Ziel ist eine URL.
3. URL- und manpage-Referenzen teilen sich zwar einen Namensraum für ihre Bezeichner, das sollte jedoch kein Problem sein, denn manpage-Bezeichner sind symbolische Dateinamen und sollten daher nie wie URLs aussehen, die Bezeichner der URL-Referenzen.

API

Paketbefehle

::doctools::idx *objectName*

Dieser Befehl erstellt ein neues Container-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::doctools::idx** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

objectName* key add *name

Diese Methode fügt das Schlüsselwort *name* zum Index hinzu. Wenn das Schlüsselwort bereits vorhanden ist, erfolgt keine Aktion. Das Ergebnis der Methode ist die leere Zeichenkette.

objectName* key remove *name

Diese Methode entfernt das Schlüsselwort *name* aus dem Index. Wenn das Schlüsselwort bereits entfernt wurde, erfolgt keine Aktion. Alle Verweise, bei denen dieses Schlüsselwort die letzte Zuordnung war, werden ebenfalls entfernt. Das Ergebnis der Methode ist die leere Zeichenkette.

objectName* key references *name

Diese Methode gibt eine Liste zurück, die die Namen aller Referenzen enthält, die mit dem Schlüsselwort *name* verknüpft sind. Es wird ein Fehler ausgelöst, wenn das Schlüsselwort dem Index nicht bekannt ist. Die Reihenfolge der Referenzen in der Liste ist nicht definiert.

***objectName* keys**

Diese Methode gibt eine Liste zurück, die die Namen aller dem Index bekannten Schlüsselwörter enthält. Die Reihenfolge der Schlüsselwörter in der Liste ist nicht definiert.

objectName* reference add *type key name label

Diese Methode fügt die Referenz *name* zum Index hinzu und ordnet sie dem Schlüsselwort *key* zu. Die beiden anderen Argumente enthalten den *type* bzw. die *label* der Referenz. Der Typ muss mit den gespeicherten Informationen übereinstimmen, falls die Referenz bereits existiert; das heißt, diese Informationen sind unveränderlich, sobald die Referenz bekannt ist. Die einzige Möglichkeit, ihn zu ändern, besteht darin, die Referenz zu löschen und neu anzulegen. Die Bezeichnung hingegen wird automatisch auf den Wert des Arguments aktualisiert, wobei zuvor gespeicherte Informationen überschrieben werden. Sollte die Referenz bereits existieren, wird sie einfach mit dem *key* verknüpft. Sollte dies ebenfalls bereits der Fall sein, wird nichts unternommen, sondern lediglich die *label* auf den neuen Wert aktualisiert. Das Ergebnis der Methode ist die leere Zeichenkette.

Das *type*-Argument muss entweder **manpage** oder **url** lauten.

objectName* reference remove *name

Die Referenz *name* wird aus dem Index entfernt. Alle Verknüpfungen mit Schlüsselwörtern werden aufgelöst und die entsprechenden Referenzbezeichnungen entfernt. Das Ergebnis der Methode ist die leere Zeichenkette.

objectName* reference label *name

Diese Methode gibt die Bezeichnung zurück, die der Referenz *name* zugeordnet ist. Wird die Referenz nicht erkannt, wird ein Fehler ausgelöst.

objectName* reference keys *name

Diese Methode gibt eine Liste zurück, die die Namen aller Schlüsselwörter enthält, die mit der Referenz *name* verknüpft sind. Es wird ein Fehler ausgelöst, wenn die Referenz dem Index nicht bekannt ist. Die Reihenfolge der Schlüsselwörter in der Liste ist nicht definiert.

objectName* reference type *name

Diese Methode gibt den Typ der Referenz *name* zurück. Es wird ein Fehler ausgelöst, wenn die Referenz dem Index nicht bekannt ist.

***objectName* references**

Diese Methode gibt eine Liste zurück, die die Namen aller dem Index bekannten Referenzen enthält. Die Reihenfolge der Referenzen in der Liste ist nicht definiert.

***objectName* title**

Gibt den aktuell definierten Titel des Schlüsselwortindexes zurück.

objectName* title *text

Setzt den Titel des Schlüsselwortindexes auf *text* und gibt ihn als Ergebnis des Befehls zurück.

***objectName* label**

Gibt die aktuell definierte Bezeichnung des Schlüsselwortindexes zurück.

objectName* label *text

Setzt die Bezeichnung des Schlüsselwortindexes auf *text* und gibt diese als Ergebnis des Befehls zurück.

***objectName* importer**

Gibt das Import-Manager-Objekt zurück, das dem Container derzeit zugeordnet ist, sofern eines zugeordnet ist.

objectName* importer *object

Hängt das *object* als Import-Manager an den Container an und gibt es als Ergebnis des Befehls zurück. Das *object* geht *nicht* in den Besitz des Containers über. Die Löschung des Containers löscht das *object* also *nicht*.

Es wird erwartet, dass *object* eine Methode **import text** bereitstellt, die einen Text und einen Formatnamen entgegennimmt und die kanonische Serialisierung des im Text enthaltenen Schlüsselwortindexes zurückgibt, unter der Annahme des angegebenen Formats.

***objectName* exporter**

Gibt das Export-Manager-Objekt zurück, das dem Container derzeit zugeordnet ist, sofern eines zugeordnet ist.

objectName* exporter *object

Hängt das *object* als Export-Manager an den Container an und gibt es als Ergebnis des Befehls zurück. Das *object* geht *nicht* in den Besitz des Containers über. Die Löschung des Containers löscht das *object* also *nicht*.

Es wird erwartet, dass *object* eine Methode **export object** bereitstellt, die den Container und einen Formatnamen entgegennimmt und einen Text zurückgibt, der den im Container gespeicherten Schlüsselwortindex im angegebenen Format kodiert. Ferner wird erwartet, dass das *object* die Methode **serialize** des Containers verwendet, um die Serialisierung des Schlüsselwortindexes zu erhalten, aus der der Text erzeugt wird.

***objectName* deserialize = *data* ?*format*?**

Diese Methode ersetzt den Inhalt des Indexobjekts durch den in *data* enthaltenen Index. Wurde kein *format* angegeben, wird davon ausgegangen, dass es sich um die reguläre Serialisierung eines Schlüsselwortindexes handelt.

Andernfalls verwendet das Objekt den angehängten Import-Manager, um die Daten aus dem angegebenen Format in eine für das Objekt verarbeitbare Serialisierung zu konvertieren. In diesem Fall wird ein Fehler ausgelöst, wenn dem Container kein Import-Manager zugeordnet ist.

Das Ergebnis der Methode ist die leere Zeichenkette.

***objectName* deserialize += *data* ?*format*?**

Diese Methode verhält sich im Wesentlichen wie **deserialize =**, mit dem Unterschied, dass sie den Schlüsselwortindex in den *data* mit seinem Inhalt zusammenführt, anstatt ihn zu ersetzen. Die Methode löst einen Fehler aus, wenn eine Zusammenführung nicht möglich ist, d. h. wenn dadurch ein ungültiger Index entstehen würde. In diesem Fall bleibt der vorhandene Inhalt unverändert.

Das Ergebnis der Methode ist die leere Zeichenkette.

***objectName* serialize ?*format*?**

Diese Methode gibt den im Objekt enthaltenen Schlüsselwortindex zurück. Wenn kein *format* angegeben wird, entspricht das zurückgegebene Ergebnis der kanonischen Serialisierung des Inhalts.

Andernfalls verwendet das Objekt den angehängten Exportmanager, um die Daten in das angegebene Format zu konvertieren. In diesem Fall wird ein Fehler ausgelöst, wenn dem

Container kein Exportmanager zugeordnet ist.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung

Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export

Stichwörter: idx_export, HTML, conversion, docidx, documentation, export, formatting, generation, index, json, keyword index, manpage, markup, nroff, plugin, reference, tcler's wiki, text, url, wiki

Name

doctools::idx::export – Export von Schlüsselwortindizes

Übersicht

package require doctools::idx::export ?**0.2.2**?
package require Tcl 8.5 9
package require struct::map
package require doctools::idx::structure
package require snit
package require pluginmgr
::doctools::idx::export *objectName*
objectName method ?*arg arg* ...?
objectName **destroy**
objectName **export serial** *serial* ?*format*?

objectName **export object** *object* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*
export *serial configuration*

Beschreibung

Dieses Paket stellt eine Klasse zur Verwaltung der Plugins für den Export von Schlüsselwortindizes in andere Formate bereit, d. h. für deren Konvertierung beispielsweise in *docidx*, *HTML* usw.

Dies ist eine der drei öffentlichen Säulen, auf denen die Verwaltung von Schlüsselwortindizes ruht. Die beiden anderen Säulen sind

1. *Import von Schlüsselwortindizes* sowie
2. *Halten von Schlüsselwortindizes*

Informationen über Begriffe von Schlüsselwortindizes und ihre Bestandteile stehen im gleichnamigen Abschnitt. Informationen über die Datenstruktur, die die wesentliche Eingabe der von diesem Paket bereitgestellten Manager-Objekte ist, stehen im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

Das Plugin-System unserer Klasse basiert auf dem Paket **pluginmgr** und ist so konfiguriert, dass es nach Plugins sucht, die

1. die Umgebungsvariable **DOCTOOLS_IDX_EXPORT_PLUGINS**,
2. die Umgebungsvariable **DOCTOOLS_IDX_PLUGINS**,
3. die Umgebungsvariable **DOCTOOLS_PLUGINS**,
4. der Pfad "*~/.doctools/idx/export/plugin*"
5. der Pfad "*~/.doctools/idx/plugin*"
6. der Pfad "*~/.doctools/plugin*"
7. der Pfad "*~/.doctools/idx/export/plugins*"
8. der Pfad "*~/.doctools/idx/plugins*"
9. der Pfad "*~/.doctools/plugins*"
10. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\EXPORT\PLUGINS"
11. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\PLUGINS"
12. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS".

Die letzten drei werden nur verwendet, wenn das Paket auf einem Rechner mit dem Betriebssystem Windows(tm) ausgeführt wird.

Das gesamte System wird mit sechs vordefinierten Export-Plugins ausgeliefert, nämlich

docidx
: Weitere Informationen finden Sie unter *docidx-Export-Plugin*.

html
: Einzelheiten stehen in *html export plugin*.

json
: Einzelheiten stehen in *json export plugin*.

nroff

Einzelheiten stehen in *nroff export plugin*.

text
Einzelheiten stehen in *text export plugin*.

wiki
Weitere Informationen finden Sie unter *Wiki-Export-Plugin*.

Für Leser, die ein eigenes Export-Plugin für ein bestimmtes Format erstellen möchten – d. h. für *Plugin-Entwickler* –, ist es unbedingt erforderlich, den Abschnitt zu lesen und zu verstehen, der die Referenz zur Export-Plugin-API v2 enthält, da darin die Interaktion zwischen diesem Paket und seinen Plugins im Detail festgelegt wird.

Begriffe

1. Ein *Schlüsselwortindex* besteht aus einer (möglicherweise leeren) Menge von *Schlüsselwörtern*.
2. Jedes Schlüsselwort in der Menge wird durch seinen Namen identifiziert.
3. Jedes Schlüsselwort verfügt über eine (möglicherweise leere) Menge von *Referenzen*.
4. Eine Referenz kann mit mehr als einem Schlüsselwort verknüpft sein.
5. Eine Referenz, die nicht mit mindestens einem Schlüsselwort verknüpft ist, ist jedoch nicht möglich.
6. Jede Referenz wird durch ihr Ziel identifiziert, das je nach Referenztyp (**url** oder **manpage**) entweder als URL oder als symbolischer Dateiname angegeben wird.
7. Der Typ einer Referenz (URL oder manpage) hängt nur von der Referenz selbst ab und nicht von den Schlüsselwörtern, denen sie zugeordnet ist.
8. Zusätzlich zu einem Typ verfügt jede Referenz auch über eine beschreibende Bezeichnung. Diese Bezeichnung hängt ausschließlich von der Referenz selbst ab und nicht von den Schlüsselwörtern, denen sie zugeordnet ist.

Einige Hinweise

1. manpage-Referenzen sind für Verweise auf die Dokumente gedacht, für die der Index erstellt wurde. Ihr Ziel ist ein symbolischer Dateiname, der das Dokument identifiziert, und Export-Plugins können symbolische durch tatsächliche Dateinamen ersetzen, wenn das festgelegt ist.
2. URL-Referenzen hingegen sind für Verweise auf alles andere gedacht, etwa Websites. Ihr Ziel ist eine URL.
3. URL- und manpage-Referenzen teilen sich zwar einen Namensraum für ihre Bezeichner, das sollte jedoch kein Problem sein, denn manpage-Bezeichner sind symbolische Dateinamen und sollten daher nie wie URLs aussehen, die Bezeichner der URL-Referenzen.

API

Paketbefehle

::doctools::idx::export *objectName*
Dieser Befehl erstellt ein neues Export-Manager-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird unter dem aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::doctools::idx::export** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

***objectName* export serial *serial* ?*format*?**

Diese Methode nimmt die kanonische Serialisierung eines in *serial* gespeicherten Schlüsselwortindexes entgegen und konvertiert sie unter Verwendung des Export-Plugins für das jeweilige *format* in das angegebene *format*. Wird kein Plugin für das Format gefunden, wird ein Fehler ausgelöst. Die durch den Konvertierungsprozess erzeugte Zeichenkette wird als Ergebnis dieser Methode zurückgegeben.

Wenn kein Format festgelegt wird, verwendet die Methode standardmäßig das Format **docidx** .

Die Spezifikation dessen, was eine *kanonische* Serialisierung ist, finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

Das Plugin muss der im Abschnitt Referenz zur Export-Plugin-API v2 spezifizierten Schnittstelle entsprechen.

***objectName* export object *object* ?*format*?**

Diese Methode ist eine praktische Hülle um die im vorherigen Punkt beschriebene Methode **export serial**. Sie setzt voraus, dass *object* ein Objektbefehl ist, der eine **serialize**-Methode unterstützt, die die kanonische Serialisierung eines Schlüsselwortindexes zurückgibt. Sie ruft diese Methode auf, übergibt das Ergebnis an **export serial** und gibt die resultierende Zeichenkette als eigenes Ergebnis zurück.

***objectName* config names**

Diese Methode gibt eine Liste zurück, die die Namen aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config get**

Diese Methode gibt ein Dictionary zurück, das die Namen und Werte aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config set *name* ?*value*?**

Diese Methode setzt die Konfigurationsvariable *name* auf den angegebenen *value* und gibt den neuen Wert der Variablen zurück.

Wird kein Wert angegeben, gibt sie lediglich den aktuellen Wert zurück, ohne ihn zu ändern.

Hinweis: Der Benutzer kann zwar die vordefinierten Konfigurationsvariablen **user** und **format** setzen, dies hat jedoch keine Auswirkungen; diese Werte werden intern beim Aufruf eines

Import-Plugins überschrieben.

objectName* config unset *pattern...

Diese Methode löscht alle Konfigurationsvariablen, auf die die angegebenen *pattern*s passen. Wird kein Muster angegeben, löscht sie alle derzeit definierten Konfigurationsvariablen.

Referenz zur Export-Plugin-API v2

Dieses Paket nutzt Plugins, um die Unterstützung für alle Ausgabeformate zu verwalten, die über das Serialisierungsformat für Schlüsselwortindizes hinausgehen. Hier legen wir die API fest, über die die von diesem Paket erstellten Objekte mit ihren Plugins interagieren.

Ein Plugin für dieses Paket muss die nachfolgend aufgeführten Regeln einhalten:

1. Ein Plugin ist ein Paket.
2. Der Name eines Plugin-Pakets hat die Form doctools::idx::export::**FOO**, wobei **FOO** der Name des Formats ist, für das das Plugin eine Ausgabe generieren wird. Dieser Name ist zugleich das Argument, das den verschiedenen **export**-Methoden von Export-Manager-Objekten übergeben werden muss, um eine Zeichenkette mit der Kodierung des Schlüsselwortindexes in diesem Format zu erhalten.
3. Das Plugin kann davon ausgehen, dass das Paket **doctools::idx::export::plugin** vorhanden ist, was darauf hindeutet, dass es von einem echten Plugin-Manager aufgerufen wurde.
4. Ein Plugin muss einen Befehl bereitstellen, dessen Signatur wie unten dargestellt lautet.

   export *serial configuration*

   Immer wenn ein Exportmanager von **doctools::idx** eine Ausgabe für einen Index erstellen muss, wird dieser Befehl aufgerufen.

   string *serial*

   Dieses Argument enthält die *kanonische* Serialisierung des Indexes, für den die Ausgabe generiert werden soll. Die Definition dessen, was eine *kanonische* Serialisierung ist, finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

   dictionary *configuration*

   Dieses Argument enthält die bei der Erzeugung anzuwendende Konfiguration, als Dictionary von Variablennamen auf Werte.

   Die folgenden Konfigurationsvariablen haben eine vorgegebene Bedeutung, an die sich alle Plugins halten müssen, auch wenn sie diese Angabe nach eigenem Ermessen ignorieren können. Jede weitere Konfigurationsvariable, die ein Plugin erkennt, ist in der Handbuchseite dieses Plugins beschrieben.

   user

   Diese Variable soll den Namen des Benutzers enthalten, dem der Prozess gehört, der das Plugin aufruft.

   format

   Diese Variable soll den Namen des Formats enthalten, dessen Plugin aufgerufen wird.

   file

   Diese Variable soll, wenn sie vom Benutzer des Index-Objekts definiert wurde, den Namen der Eingabedatei enthalten, für die das Plugin seine Ausgabe generiert.

   map

   Diese Variable soll, wenn sie vom Benutzer des Indexobjekts definiert wurde, ein Dictionary enthalten, das die in den Referenzen vom Typ **manpage** verwendeten symbolischen Namen den tatsächlichen Pfaden (oder URLs) zuordnet. Ein Plugin muss

in der Lage sein, den Fall zu behandeln, dass für einen symbolischen Namen kein Eintrag in diesem Dictionary vorhanden ist.

1. Ein einzelner Nutzungszyklus eines Plugins besteht aus den Aufrufen des Befehls **export**. Dieser Aufruf muss das Plugin in einem Zustand hinterlassen, in dem ein weiterer Nutzungszyklus problemlos ausgeführt werden kann.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

url
Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung
Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_html

Stichwörter: idx_export_html, HTML, doctools, export, index, serialization

Name

doctools::idx::export::html – HTML-Export-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::idx::export::html ?**0.3**?
package require doctools::text
package require doctools::html
package require doctools::html::cssdefaults
export *serial configuration*

Beschreibung

Dieses Paket implementiert das doctools-Export-Plugin für Schlüsselwortindizes zur Erzeugung von HTML-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::export**, dem Exportmanager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur unter Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::export** und die darin bereitgestellten Exportmanager-Objekte.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Export-Plugin-API, Version 2.

export *serial configuration*

Dieser Befehl nimmt die kanonische Serialisierung eines Schlüsselwortindexes, wie im Abschnitt Serialisierungsformat für Schlüsselwortindizes festgelegt und in *serial* enthalten, sowie die *configuration* (ein Dictionary), und erzeugt HTML-Markup, das den Index kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Konfiguration

Das HTML-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

string *user*

Diese Standard-Konfigurationsvariable enthält den Namen des Benutzers, der den Prozess ausführt, welcher das Export-Plugin aufgerufen hat. Das Plugin schreibt diese Angabe in den Herkunftskommentar am Anfang des erzeugten Dokuments.

string *file*

Diese Standardkonfigurationsvariable enthält den Namen der Datei, aus der der Index stammt. Diese Variable kann ungesetzt sein oder die leere Zeichenkette enthalten. Das Plugin fügt diese Information, sofern sie definiert ist, d. h. gesetzt ist und nicht die leere Zeichenkette enthält, in den Herkunftskommentar am Anfang des generierten Dokuments ein.

dictionary *map*

Diese Standardkonfigurationsvariable enthält ein Dictionary, das die symbolischen Dateinamen in den Verweisen auf Handbuchseiten den tatsächlichen Dateinamen und/oder URLs zuordnet, die in der Ausgabe verwendet werden sollen.

URL-Verweise und symbolische Dateinamen ohne Zuordnung werden unverändert verwendet.

boolean *newlines*

Ist dieses Flag gesetzt, bricht das Plugin den erzeugten HTML-Code auf mehrere Zeilen um, mit jedem Markup-Befehl in einer eigenen Zeile.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird das ganze Dokument in eine einzige Zeile geschrieben, mit möglichst wenig Abstand zwischen allen Elementen.

boolean *indented*

Ist dieses Flag gesetzt, rückt das Plugin die Markup-Befehle entsprechend dem Aufbau der

Indizes ein. Damit das wirkt, ist auch **newlines** gesetzt.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird die Ausgabe gemäß dem Wert von **newlines** formatiert, und es wird nicht eingerückt.

string *meta*
Diese Variable soll ein HTML-Fragment aufnehmen (Vorgabe: leer). Das enthaltene Fragment wird in die erzeugte Ausgabe im Abschnitt des Dokuments eingefügt, unmittelbar nach dem Tag.

string *header*
Diese Variable soll ein HTML-Fragment aufnehmen (Vorgabe: leer). Das enthaltene Fragment wird in die erzeugte Ausgabe unmittelbar nach dem Titel-Tag im Hauptteil des Dokuments eingefügt, in der class.header 'ision.

string *footer*
Diese Variable soll ein HTML-Fragment aufnehmen (Vorgabe: leer). Das enthaltene Fragment wird in die erzeugte Ausgabe unmittelbar vor dem Tag eingefügt, in der class.footer 'ision.

dictionary *kwid*
Der Wert dieser Variablen (Standard: leer) ordnet Schlüsselwörter den Bezeichnern zu, die als deren Anker verwendet werden sollen. Jedes Schlüsselwort **FOO**, das nicht im Dictionary enthalten ist, verwendet **KW-FOO** als Anker, d. h. ihm wird die Zeichenkette **KW-** vorangestellt.

string *sepline*
Der Wert dieser Variablen ist die Zeichenkette für die Trennkommentare, die in die Ausgabe eingefügt werden, wenn die Ausgabe auf mehrere Zeilen umgebrochen und/oder eingerückt wird. Die Vorgabe besteht aus 60 Bindestrichen.

integer *kwidth*
Diese Variable gibt die Breite der vom Plugin generierten Spalte für Schlüsselwörter in der Haupttabelle an, ausgedrückt als Prozentsatz der Gesamtbreite der Tabelle. Es handelt sich um eine ganze Zahl im Bereich von 1 bis 99. Bei der Wahl eines Wertes außerhalb dieses Bereichs wechselt der Generator zurück zur Standardeinstellung von 35 Prozent.

string *dot*
Diese Variable enthält ein HTML-Fragment, das zwischen die Einträge der Navigationsleiste und die zu jedem Schlüsselwort gehörenden Referenzen eingefügt wird. Der Standardwert ist die HTML-Entität ·, d. h. das Aufzählungszeichen, auch bekannt als "Mittelpunkt", bzw. das Unicode-Zeichen 00B7.

string *class.main*
Diese Variable enthält den Klassennamen für die 'ivision des Hauptbereichs im erzeugten Dokument. Die Vorgabe ist **doctools**.

string *class.header*
Diese Variable enthält den Namen der Klasse für den Kopfbereich () des erzeugten Dokuments. Der Standardwert lautet **idx-header**. Dieser Bereich enthält den Dokumenttitel, den vom Benutzer angegebenen **header**, falls vorhanden, eine sichtbare Trennlinie sowie die Navigationsleiste für den schnellen Zugriff auf jeden Schlüsselwortabschnitt.

string *class.title*
Diese Variable enthält den Namen der Klasse für das Tag, das den Dokumenttitel umschließt. Der Standardwert lautet **idx-title**.

string *class.navsep*
Diese Variable enthält den Namen der Klasse für die Trennzeichen in den Kopf- und Fußzeilenabschnitten des generierten Dokuments. Der Standardwert lautet **idx-navsep**.

string *class.navbar*
Diese Variable enthält den Namen der Klasse für den Navigationsbereich (), der die Navigationsleiste des erzeugten Dokuments umschließt. Der Standardwert lautet **idx-kwnav**.

string *class.contents*
Diese Variable enthält den Namen der Klasse, die die Schlüsselwörter und deren Verweise im generierten Dokument enthält. Der Standardwert lautet **idx-contents**.

string *class.leader*
Diese Variable enthält den Namen der Klasse für die Anker, die das Plugin beim Wechsel von einem Abschnitt zum nächsten in die Schlüsselwort-Tabelle einfügt (jeder Abschnitt enthält alle Schlüsselwörter, die mit einem bestimmten Anfangszeichen beginnen). Der Standardwert lautet **idx-leader**.

string *class.row0*
Diese Variable enthält den Namen der Klasse, die zur Kennzeichnung der geraden Zeilen () der Tabelle mit Schlüsselwörtern verwendet wird. Der Standardwert lautet **idx-even**.

string *class.row1*
Diese Variable enthält den Namen der Klasse, die zur Kennzeichnung der ungeraden Zeilen () der Tabelle mit Schlüsselwörtern verwendet wird. Der Standardwert lautet **idx-odd**.

string *class.keyword*
Diese Variable enthält den Namen der Klasse, die zur Kennzeichnung der Schlüsselwortzelle/-Spalte () in der Tabelle der Schlüsselwörter des Dokuments verwendet wird. Der Standardwert lautet **idx-keyword**.

string *class.refs*
Diese Variable enthält den Namen der Klasse, die zur Kennzeichnung der Referenzzellen/-Spalten () in der Tabelle der Schlüsselwörter des Dokuments verwendet wird. Der Standardwert lautet **idx-refs**.

string *class.footer*
Diese Variable enthält den Namen der Klasse für den Fußbereich () des erzeugten Dokuments. Der Standardwert lautet **idx-footer**. Dieser Abschnitt enthält eine im Browser sichtbare Trennlinie sowie gegebenenfalls den vom Benutzer angegebenen Wert **footer**.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariable **format** und ihren Wert.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert.

Dieser Wert enthält den Inhalt des Index.

2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung

Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textformatierer-Plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_json
Stichwörter: idx_export_json, JSON, doctools, export, index, serialization

Name
doctools::idx::export::json – JSON-Export-Plugin

Übersicht
package require Tcl 8.5 9
package require doctools::idx::export::json ?**0.2**?
package require textutil::adjust
export *serial configuration*

Beschreibung
Dieses Paket implementiert das doctools-Export-Plugin für Schlüsselwortindizes zur Erzeugung von JSON-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::export**, dem Exportmanager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur unter Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::export** und die darin bereitgestellten Exportmanager-Objekte.

API
Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Schlüsselwortindexes, wie im Abschnitt Serialisierungsformat für Schlüsselwortindizes festgelegt und in *serial* enthalten, sowie die *configuration* (ein Dictionary), und erzeugt JSON-Markup, das den Index kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

JSON-Notation von Schlüsselwortindizes

Das für Schlüsselwortindizes verwendete JSON-Format ist eine direkte Umsetzung des Serialisierungsformat für Schlüsselwortindizes, bei der Tcl-Dictionaries als JSON-Objekte und Tcl-Listen als JSON-Arrays abgebildet werden. Beispielsweise entspricht die Tcl-Serialisierung

```
doctools::idx {
    label {Keyword Index}
    keywords {
        changelog  {changelog.man cvs.man}
        conversion {doctools.man docidx.man doctoc.man apps/dtplite.man
    mpexpand.man}
        cvs        cvs.man
    }
    references {
        apps/dtplite.man {manpage dtplite}
        changelog.man    {manpage doctools::changelog}
        cvs.man          {manpage doctools::cvs}
        docidx.man       {manpage doctools::idx}
        doctoc.man       {manpage doctools::toc}
        doctools.man     {manpage doctools}
        mpexpand.man     {manpage mpexpand}
    }
    title {}
}
```

der JSON-Zeichenkette

```
{
    "doctools::idx" : {
        "label"      : "Keyword Index",
        "keywords"   : {
            "changelog"  : ["changelog.man","cvs.man"],
            "conversion" :
    ["doctools.man","docidx.man","doctoc.man","apps\/dtplite.man","mpex
    pand.man"],
            "cvs"        : ["cvs.man"],
        },
        "references" : {
            "apps\/dtplite.man" : ["manpage","dtplite"],
            "changelog.man"     : ["manpage","doctools::changelog"],
            "cvs.man"           : ["manpage","doctools::cvs"],
            "docidx.man"        : ["manpage","doctools::idx"],
            "doctoc.man"        : ["manpage","doctools::toc"],
            "doctools.man"      : ["manpage","doctools"],
            "mpexpand.man"      : ["manpage","mpexpand"]
        },
        "title"      : ""
    }
}
```

Konfiguration

Das JSON-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

boolean *indented*
: Ist dieses Flag gesetzt, bricht das Plugin den erzeugten JSON-Code auf mehrere Zeilen um und rückt ihn entsprechend seinem inneren Aufbau ein, mit jedem Schlüssel eines Dictionarys in einer eigenen Zeile.

  Ist dieses Flag nicht gesetzt (die Vorgabe), wird das ganze JSON-Objekt in eine einzige Zeile geschrieben, mit möglichst wenig Abstand zwischen allen Elementen.

boolean *aligned*
: Ist dieses Flag gesetzt, sorgt der Generator dafür, dass die Werte der Schlüssel in einem Dictionary senkrecht zueinander ausgerichtet sind, mit dem Effekt einer Tabelle. Damit das wirkt, ist auch **indented** gesetzt.

  Ist dieses Flag nicht gesetzt (die Vorgabe), wird die Ausgabe gemäß dem Wert von **indented** formatiert, ohne die Werte der Dictionary-Schlüssel auszurichten.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **user**, **format**, **file** und **map** sowie ihre Werte.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um

Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung

Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender

Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_nroff

Stichwörter: idx_export_nroff, doctools, export, index, nroff, serialization

Name

doctools::idx::export::nroff – nroff-Export-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::idx::export::nroff ?**0.4**?
package require doctools::text
package require doctools::nroff::man_macros
export *serial configuration*

Beschreibung

Dieses Paket implementiert das doctools-Export-Plugin für Schlüsselwortindizes zur Erzeugung von nroff-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::export**, dem Exportmanager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur unter Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::export** und die darin bereitgestellten Exportmanager-Objekte.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Export-Plugin-API, Version 2.

export *serial configuration*

Dieser Befehl nimmt die kanonische Serialisierung eines Schlüsselwortindexes, wie im Abschnitt Serialisierungsformat für Schlüsselwortindizes festgelegt und in *serial* enthalten, sowie die *configuration* (ein Dictionary), und erzeugt nroff-Markup, das den Index kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Konfiguration

Das nroff-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

string *user*

Diese Standard-Konfigurationsvariable enthält den Namen des Benutzers, der den Prozess ausführt, welcher das Export-Plugin aufgerufen hat. Das Plugin schreibt diese Angabe in den Herkunftskommentar am Anfang des erzeugten Dokuments.

string *file*

Diese Standardkonfigurationsvariable enthält den Namen der Datei, aus der der Index stammt. Diese Variable kann ungesetzt sein oder die leere Zeichenkette enthalten. Das Plugin fügt diese Information, sofern sie definiert ist, d. h. gesetzt ist und nicht die leere Zeichenkette enthält, in den Herkunftskommentar am Anfang des generierten Dokuments ein.

boolean *inline*

Ist dieses Flag gesetzt (Standard), fügt das Plugin die Definitionen des *man*-Makrosatzes direkt in die Ausgabe.

Ist dieses Flag nicht gesetzt, fügt das Plugin einen Verweis auf die Definitionen des *man* -Makrosatzes in die Ausgabe, jedoch nicht die Makrodefinitionen selbst.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **format** und **map** sowie ihre Werte.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

title
Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

label
Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

keywords
Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

references
Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

manpage
Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

url
Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung
Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textformatierer-Plugin

Copyright
```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

idx_export_text
Stichwörter: idx_export_text, doctools, export, index, plain text, serialization

Name
doctools::idx::export::text – Klartext-Export-Plugin

Übersicht
package require Tcl 8.5 9
package require doctools::idx::export::text ?**0.3**?
package require doctools::text
export *serial configuration*

Beschreibung
Dieses Paket implementiert das doctools-Export-Plugin für Schlüsselwortindizes zur Erzeugung von Klartext-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::export**, dem Exportmanager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur unter Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::export** und die darin bereitgestellten Exportmanager-Objekte.

API
Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Schlüsselwortindexes, wie im Abschnitt Serialisierungsformat für Schlüsselwortindizes festgelegt und in *serial* enthalten, sowie die *configuration* (ein Dictionary), und erzeugt Klartext-Markup, das den Index kodiert. Die erzeugte Zeichenkette wird dann als Ergebnis des Befehls zurückgegeben.

Konfiguration
Das Text-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

dictionary *map*

Diese Standardkonfigurationsvariable enthält ein Dictionary, das die symbolischen Dateinamen in den Verweisen auf Manpages den tatsächlichen Dateinamen und/oder URLs zuordnet, die in der Ausgabe verwendet werden sollen.

URL-Verweise und symbolische Dateinamen ohne Zuordnung werden unverändert verwendet.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **user**, **file** und **format** sowie ihre Werte.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf

eines der Dokumente bezieht, für die der Index erstellt wurde.

url
Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung
Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_export_wiki

Stichwörter: idx_export_wiki, doctools, export, index, serialization, wiki

Name

doctools::idx::export::wiki – Wiki-Export-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::idx::export::wiki ?**0.3**?
package require doctools::text
export *serial configuration*

Beschreibung

Dieses Paket implementiert das doctools-Export-Plugin für Schlüsselwortindizes zur Erzeugung von Wiki-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::export**, dem Exportmanager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur unter Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::export** und die darin bereitgestellten Exportmanager-Objekte.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Export-Plugin-API, Version 2.

export *serial configuration*

Dieser Befehl nimmt die kanonische Serialisierung eines Schlüsselwortindexes, wie im Abschnitt Serialisierungsformat für Schlüsselwortindizes festgelegt und in *serial* enthalten, sowie die *configuration* (ein Dictionary), und erzeugt Wiki-Markup, das den Index kodiert. Die erzeugte Zeichenkette wird dann als Ergebnis des Befehls zurückgegeben.

Wiki-Markup

Die grundlegende Syntax des von diesem Plugin erzeugten Wiki-Markups ist unter http://wiki.tcl.tk/14 beschrieben.

Das Plugin geht über die klassische Markup-Syntax hinaus und generiert korrekte Überschriften sowie entweder eine Tabelle oder eine eingerückte Liste der Schlüsselwörter und ihrer Referenzen.

Konfiguration

Das Wiki-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

dictionary *map*

Diese Standardkonfigurationsvariable enthält ein Dictionary, das die symbolischen Dateinamen in den Verweisen auf Handbuchseiten den tatsächlichen Dateinamen und/oder URLs zuordnet, die in der Ausgabe verwendet werden sollen.

URL-Verweise und symbolische Dateinamen ohne Zuordnung werden unverändert verwendet.

enum *style*

Diese Variable erkennt zwei Werte als zulässig an: **list** (Standard) und **table**. Je nach Wert generiert das Plugin entweder eine listen- oder tabellenbasierte Wiki-Seite für den Index.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **user**, **file** und **format** sowie ihre Werte.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung

Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden –

werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_import

Stichwörter: idx_import, conversion, docidx, documentation, import, index, json, keyword index, manpage, markup, parsing, plugin, reference, url

Name

doctools::idx::import – Import von Schlüsselwortindizes

Übersicht

package require doctools::idx::import ?**0.2.2**?
package require Tcl 8.5 9
package require struct::map
package require doctools::idx::structure
package require snit
package require pluginmgr
::doctools::idx::import *objectName*
objectName method ?*arg arg* ...?
objectName **destroy**
objectName **import text** *text* ?*format*?
objectName **import file** *path* ?*format*?
objectName **import object text** *object text* ?*format*?
objectName **import object file** *object path* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*
objectName **includes**

objectName **include add** *path*
objectName **include remove** *path*
objectName **include clear**
IncludeFile *currentfile path*
import *text configuration*

Beschreibung

Dieses Paket stellt eine Klasse zur Verwaltung der Plugins für den Import von Schlüsselwortindizes aus anderen Formaten bereit, d. h. für deren Konvertierung aus beispielsweise *docidx*, *json* usw.

Dies ist eine der drei öffentlichen Säulen, auf denen die Verwaltung von Schlüsselwortindizes ruht. Die beiden anderen Säulen sind

1. *Export von Schlüsselwortindizes* sowie
2. *Halten von Schlüsselwortindizes*

Informationen über Begriffe von Schlüsselwortindizes und ihre Bestandteile stehen im gleichnamigen Abschnitt. Informationen über die Datenstruktur, die die wesentliche Ausgabe der von diesem Paket bereitgestellten Manager-Objekte ist, stehen im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

Das Plugin-System unserer Klasse basiert auf dem Paket **pluginmgr** und ist so konfiguriert, dass es nach Plugins sucht, die

1. die Umgebungsvariable **DOCTOOLS_IDX_IMPORT_PLUGINS**,
2. die Umgebungsvariable **DOCTOOLS_IDX_PLUGINS**,
3. die Umgebungsvariable **DOCTOOLS_PLUGINS**,
4. der Pfad "*~/.doctools/idx/import/plugin*"
5. der Pfad "*~/.doctools/idx/plugin*"
6. der Pfad "*~/.doctools/plugin*"
7. der Pfad "*~/.doctools/idx/import/plugins*"
8. der Pfad "*~/.doctools/idx/plugins*"
9. der Pfad "*~/.doctools/plugins*"
10. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\IMPORT\PLUGINS"
11. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\IDX\PLUGINS"
12. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS".

Die letzten drei werden nur verwendet, wenn das Paket auf einem Rechner mit dem Betriebssystem Windows(tm) ausgeführt wird.

Das gesamte System wird mit zwei vordefinierten Import-Plugins ausgeliefert, nämlich

docidx
Weitere Informationen finden Sie unter *docidx-Import-Plugin*.

json
Weitere Informationen finden Sie unter *JSON-Import-Plugin*.

Für Leser, die ein eigenes Import-Plugin für ein bestimmtes Format erstellen möchten – d. h. für *Plugin-Entwickler* –, ist es unbedingt erforderlich, den Abschnitt zu lesen und zu verstehen, der die Referenz zur Import-Plugin-API v2 enthält, da darin die Interaktion zwischen diesem Paket und seinen Plugins detailliert festgelegt wird.

Begriffe

1. Ein *Schlüsselwortindex* besteht aus einer (möglicherweise leeren) Menge von *Schlüsselwörtern*.
2. Jedes Schlüsselwort in der Menge wird durch seinen Namen identifiziert.
3. Jedes Schlüsselwort verfügt über eine (möglicherweise leere) Menge von *Referenzen*.
4. Eine Referenz kann mit mehr als einem Schlüsselwort verknüpft sein.
5. Eine Referenz, die nicht mit mindestens einem Schlüsselwort verknüpft ist, ist jedoch nicht möglich.
6. Jede Referenz wird durch ihr Ziel identifiziert, das je nach Referenztyp (**url** oder **manpage**) entweder als URL oder als symbolischer Dateiname angegeben wird.
7. Der Typ einer Referenz (URL oder manpage) hängt nur von der Referenz selbst ab und nicht von den Schlüsselwörtern, denen sie zugeordnet ist.
8. Zusätzlich zu einem Typ verfügt jede Referenz auch über eine beschreibende Bezeichnung. Diese Bezeichnung hängt ausschließlich von der Referenz selbst ab und nicht von den Schlüsselwörtern, denen sie zugeordnet ist.

Einige Hinweise

1. manpage-Referenzen sind für Verweise auf die Dokumente gedacht, für die der Index erstellt wurde. Ihr Ziel ist ein symbolischer Dateiname, der das Dokument identifiziert, und Export-Plugins können symbolische durch tatsächliche Dateinamen ersetzen, wenn das festgelegt ist.
2. URL-Referenzen hingegen sind für Verweise auf alles andere gedacht, etwa Websites. Ihr Ziel ist eine URL.
3. URL- und manpage-Referenzen teilen sich zwar einen Namensraum für ihre Bezeichner, das sollte jedoch kein Problem sein, denn manpage-Bezeichner sind symbolische Dateinamen und sollten daher nie wie URLs aussehen, die Bezeichner der URL-Referenzen.

API

Paketbefehle

::doctools::idx::import *objectName*

Dieser Befehl erstellt ein neues Import-Manager-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird unter dem aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::doctools::idx::import** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

***objectName* import text *text* ?*format*?**
Diese Methode nimmt den *text* und wandelt ihn mit dem Import-Plugin für das angegebene *format* in die kanonische Serialisierung eines Schlüsselwortindexes um. Wird kein Plugin für das Format gefunden, wird ein Fehler ausgelöst. Die durch den Konvertierungsprozess erzeugte Serialisierung wird als Ergebnis dieser Methode zurückgegeben.

Wenn kein Format festgelegt wird, verwendet die Methode standardmäßig **docidx**.

Die Spezifikation dessen, was eine *kanonische* Serialisierung ist, finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

Das Plugin muss der im Abschnitt Referenz zur Import-Plugin-API v2 spezifizierten Schnittstelle entsprechen.

***objectName* import file *path* ?*format*?**
Diese Methode ist eine praktische Hülle um die im vorherigen Punkt beschriebene Methode **import text**. Sie liest den Inhalt der angegebenen Datei in den Arbeitsspeicher ein, übergibt das Ergebnis an **import text** und gibt die daraus resultierende Serialisierung als eigenes Ergebnis zurück.

***objectName* import object text *object text* ?*format*?**
Diese Methode ist eine praktische Hülle um die im vorherigen Punkt beschriebene Methode **import text**. Sie setzt voraus, dass *object* ein Objektbefehl ist, der eine **deserialize**-Methode unterstützt, welche die kanonische Serialisierung eines Schlüsselwortindexes erwartet. Sie importiert den Text mithilfe von **import text** und übergibt die resultierende Serialisierung anschließend über **deserialize** an das *object*. Diese Methode gibt als Ergebnis die leere Zeichenkette zurück.

***objectName* import object file *object path* ?*format*?**
Diese Methode verhält sich wie **import object text**, mit dem Unterschied, dass sie den zu konvertierenden Text aus der angegebenen Datei liest, anstatt ihn als Argument zu erhalten.

***objectName* config names**
Diese Methode gibt eine Liste zurück, die die Namen aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config get**
Diese Methode gibt ein Dictionary zurück, das die Namen und Werte aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config set *name* ?*value*?**
Diese Methode setzt die Konfigurationsvariable *name* auf den angegebenen *value* und gibt den neuen Wert der Variablen zurück.

Wird kein Wert angegeben, gibt sie lediglich den aktuellen Wert zurück, ohne ihn zu ändern.

Hinweis: Der Benutzer kann zwar die vordefinierten Konfigurationsvariablen **user** und **format** setzen, dies hat jedoch keine Auswirkungen; diese Werte werden intern beim Aufruf eines Import-Plugins überschrieben.

***objectName* config unset *pattern*...**
Diese Methode löscht alle Konfigurationsvariablen, auf die die angegebenen *pattern*s passen.

Wird kein Muster angegeben, löscht sie alle derzeit definierten Konfigurationsvariablen.

***objectName* includes**
Diese Methode gibt eine Liste zurück, die die derzeit festgelegten Pfade enthält, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien verwendet werden sollen. Die Reihenfolge der Pfade in der Liste entspricht der Reihenfolge, in der sie verwendet werden – vom ersten bis zum letzten –, und entspricht zudem der Reihenfolge, in der sie dem Objekt hinzugefügt wurden.

objectName* include add *path
Diese Methode fügt den angegebenen *path* zur Liste der Pfade hinzu, in denen bei der Verarbeitung der Eingabe nach Include-Dateien gesucht wird. Der Pfad wird am Ende der Liste angehängt, sodass er erst nach allen zuvor hinzugefügten Pfaden durchsucht wird. Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Methode führt keine Aktion aus, wenn der Pfad bereits bekannt ist.

objectName* include remove *path
Diese Methode entfernt den angegebenen *path* aus der Liste der Pfade, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien herangezogen werden. Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Methode führt keine Aktion aus, wenn der Pfad nicht bekannt ist.

***objectName* include clear**
Diese Methode löscht die Liste der Pfade, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien herangezogen werden. Das Ergebnis des Befehls ist die leere Zeichenkette.

Referenz zur Import-Plugin-API v2

Dieses Paket nutzt Plugins, um die Unterstützung für alle Eingabeformate zu verwalten, die über das Serialisierungsformat für Schlüsselwortindizes hinausgehen. Hier legen wir die API fest, über die die von diesem Paket erstellten Objekte mit ihren Plugins interagieren.

Ein Plugin für dieses Paket muss die nachfolgend aufgeführten Regeln einhalten:

1. Ein Plugin ist ein Paket.
2. Der Name eines Plugin-Pakets hat die Form doctools::idx::import::**FOO**, wobei **FOO** der Name des Formats ist, das das Plugin einliest. Dieser Name ist zugleich das Argument, das den verschiedenen **import**-Methoden von Import-Manager-Objekten übergeben werden muss, um die kanonische Serialisierung eines Schlüsselwortindexes aus diesem Format zu erhalten.
3. Das Plugin kann davon ausgehen, dass das Paket **doctools::idx::export::plugin** vorhanden ist, was darauf hindeutet, dass es von einem echten Plugin-Manager aufgerufen wurde.
4. Das Plugin kann davon ausgehen, dass ein Befehl **IncludeFile** vorhanden ist, mit der Signatur **IncludeFile *currentfile path***
   Dieser Befehl muss vom Plugin aufgerufen werden, wenn es eine eingebundene Datei verarbeiten soll, wenn das Format ein solches Konzept vorsieht. Ein Beispiel für ein solches Format wäre *docidx*.

   Das Plugin muss die folgenden Argumente übergeben
   string *currentfile*
   Der Pfad der Datei, die es gerade verarbeitet. Das kann die leere Zeichenkette sein,

wenn keiner bekannt ist.

string *path*

Der Pfad der Include-Datei, wie ihn die gerade verarbeitete Include-Anweisung angibt.

```
Das Ergebnis des Befehls ist eine Liste mit fünf Elementen, die
Folgendes enthält:

1. Ein boolesches Flag, das Erfolg (**True**) oder Misserfolg
(**False**) des Vorgangs angibt.

2. Im Erfolgsfall den Inhalt der eingebundenen Datei, andernfalls
die leere Zeichenkette.

3. Der aufgelöste, d. h. der absolute Pfad der eingebundenen
Datei, sofern möglich, oder das unveränderte *path*-Argument. Dies
dient zur Anzeige in einer Fehlermeldung oder als
*currentfile*-Argument eines weiteren Aufrufs von **IncludeFile**,
falls diese Datei weitere Dateien enthält.

4. Im Erfolgsfall eine leere Zeichenkette und im Fehlerfall einen
Code, der den Grund dafür angibt, und zwar einen der folgenden:

    notfound
    : Die angegebene Datei konnte nicht gefunden werden.

    notread
    : Die angegebene Datei wurde gefunden, konnte jedoch nicht in
den Arbeitsspeicher geladen werden.


5. Eine leere Zeichenfolge im Falle eines Erfolgs oder eines
Fehlers mit dem Code **notfound** sowie eine zusätzliche
Fehlermeldung, die den Grund für einen Fehler mit dem Code
**notread** näher erläutert.
```

1. Ein Plugin muss einen Befehl bereitstellen, dessen Signatur wie unten dargestellt lautet.

   import *text configuration*

   Immer wenn ein Importmanager von **doctools::idx** Eingabedaten für einen Index auswerten muss, wird dieser Befehl aufgerufen.

   string *text*

   Dieses Argument enthält den Text, der den Index in dem Format kodiert, für das das Plugin bestimmt ist.

   dictionary *configuration*

   Dieses Argument enthält die beim Parsen anzuwendende Konfiguration, als Dictionary von Variablennamen auf Werte.

   Die folgenden Konfigurationsvariablen haben eine vorgegebene Bedeutung, an die sich

alle Plugins halten müssen, auch wenn sie diese Angabe nach eigenem Ermessen ignorieren können. Jede weitere Konfigurationsvariable, die ein Plugin erkennt, ist in der Handbuchseite dieses Plugins beschrieben.

user
Diese Variable soll den Namen des Benutzers enthalten, dem der Prozess gehört, der das Plugin aufruft.

format
Diese Variable soll den Namen des Formats enthalten, dessen Plugin aufgerufen wird.

1. Ein einzelner Nutzungszyklus eines Plugins besteht aus den Aufrufen des Befehls **import**. Dieser Aufruf muss das Plugin in einem Zustand hinterlassen, in dem ein weiterer Nutzungszyklus problemlos ausgeführt werden kann.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary

keywords bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,
   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.
   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung
Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_import_json

Stichwörter: idx_import_json, JSON, deserialization, doctools, import, index

Name

doctools::idx::import::json – JSON-Import-Plugin

Übersicht

package require Tcl 8.5 9

package require doctools::idx::import::json ?**0.2.1**?
package require doctools::idx::structure
package require json
import *string configuration*

Beschreibung

Dieses Paket implementiert das doctools-Import-Plugin für Schlüsselwortindizes zum Parsen von JSON-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::import**, dem Import-Manager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur auf Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::import** und die darin bereitgestellten Import-Manager-Objekte.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Import-Plugin-API, Version 2.

import *string configuration*

Dieser Befehl nimmt den *string* und parst ihn als JSON-Markup, das einen Schlüsselwortindex kodiert, im Kontext der angegebenen *configuration* (ein Dictionary). Das Ergebnis des Befehls ist die kanonische Serialisierung dieses Schlüsselwortindexes in der im Abschnitt Serialisierungsformat für Schlüsselwortindizes angegebenen Form.

JSON-Notation von Schlüsselwortindizes

Das für Schlüsselwortindizes verwendete JSON-Format ist eine direkte Umsetzung des Serialisierungsformat für Schlüsselwortindizes, bei der Tcl-Dictionaries als JSON-Objekte und Tcl-Listen als JSON-Arrays abgebildet werden. Beispielsweise entspricht die Tcl-Serialisierung

```
doctools::idx {
    label {Keyword Index}
    keywords {
        changelog  {changelog.man cvs.man}
        conversion {doctools.man docidx.man doctoc.man apps/dtplite.man
    mpexpand.man}
        cvs        cvs.man
    }
    references {
        apps/dtplite.man {manpage dtplite}
        changelog.man    {manpage doctools::changelog}
        cvs.man          {manpage doctools::cvs}
        docidx.man       {manpage doctools::idx}
        doctoc.man       {manpage doctools::toc}
        doctools.man     {manpage doctools}
        mpexpand.man     {manpage mpexpand}
    }
    title {}
}
```

der JSON-Zeichenkette

```
{
    "doctools::idx" : {
        "label"      : "Keyword Index",
        "keywords"   : {
            "changelog"  : ["changelog.man","cvs.man"],
            "conversion" :
    ["doctools.man","docidx.man","doctoc.man","apps\/dtplite.man","mpex
    pand.man"],
            "cvs"        : ["cvs.man"],
        },
        "references" : {
            "apps\/dtplite.man" : ["manpage","dtplite"],
            "changelog.man"     : ["manpage","doctools::changelog"],
            "cvs.man"           : ["manpage","doctools::cvs"],
            "docidx.man"        : ["manpage","doctools::idx"],
            "doctoc.man"        : ["manpage","doctools::toc"],
            "doctools.man"      : ["manpage","doctools"],
            "mpexpand.man"      : ["manpage","mpexpand"]
        },
        "title"      : ""
    }
}
```

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um

Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

   Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung

Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender

Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_introduction

Stichwörter: idx_introduction, conversion, formatting, index, keyword index, markup, parsing, plugin, semantic markup

Name

doctools2idx_introduction – DocTools – Schlüsselwortindizes

Beschreibung

docidx (Abkürzung für *Dokumentations-Indizes*) steht für eine Reihe verwandter und doch unterschiedlicher Einheiten, die gemeinsam die einfache Erstellung und Umformung von Schlüsselwortindizes für die Dokumentation ermöglichen.

Dies sind

1. Eine auf Tcl basierende Sprache zur semantischen Auszeichnung eines Schlüsselwortindexes. Die Auszeichnung erfolgt mittels Tcl-Befehlen. Anfänger sollten mit der *Einleitung in die docidx-Sprache* beginnen. Die formale Spezifikation ist auf zwei Dokumente aufgeteilt: Das eine befasst sich mit der *Syntax der docidx-Sprache*, das andere ist eine *Befehlsreferenz zur docidx-Sprache*.
2. Eine Reihe von Paketen zur programmatischen Bearbeitung von Schlüsselwortindizes im Arbeitsspeicher sowie zu deren Konvertierung zwischen verschiedenen Formaten, zum Lesen und Schreiben. Die oben genannte Auszeichnungssprache ist eines der Formate, die sowohl gelesen als auch geschrieben werden können.
3. Das System zur Konvertierung von Indizes basiert auf einem Plugin-Mechanismus; hierfür stehen zwei APIs zur Verfügung, die die Schnittstelle zwischen den oben genannten Paketen

und den Import-/Export-Plugins beschreiben.

Welche der ausführlicheren Dokumente für den Leser dieser Einleitung relevant sind, hängt von der jeweiligen Rolle im Dokumentationsprozess ab.

1. Ein *Autor* von Dokumentationen muss die Auszeichnungssprache selbst verstehen. Ein Anfänger im Umgang mit docidx sollte zunächst die eher informell verfasste *Einleitung in die docidx-Sprache* lesen. Ist diese aufgenommen, sollte die formale Spezifikation der *Syntax der docidx-Sprache* verständlich werden. Ein mit docidx vertrauter Autor braucht nur gelegentlich die *Befehlsreferenz der docidx-Sprache* zur Auffrischung.

   Während ein Dokument verfasst wird, kann die Anwendung **dtp** zur Validierung herangezogen werden; nach Fertigstellung führt sie zudem die Konvertierung in das gewählte visuelle Markup-System durch, sei es *roff, HTML, Klartext, Wiki usw. Die einfachere Anwendung **dtplite** nutzt docidx intern bei der Verwaltung von Dokumentationsverzeichnissen und generiert automatisch einen passenden Schlüsselwortindex.
2. Ein *Verarbeiter* von Dokumentation, die in der Markup-Sprache *docidx* verfasst ist, muss wissen, welche Werkzeuge zur Verfügung stehen.

   Das wichtigste Werkzeug ist die bereits erwähnte Anwendung **dtp**, die von Tcllib bereitgestellt wird. Die einfachere Anwendung **dtplite** macht *docidx* dem Benutzer nicht zugänglich. Auf der untersten Ebene, die beiden Anwendungen gemeinsam ist, finden wir jedoch die drei Pakete, die die grundlegenden Mittel zur Verarbeitung von Schlüsselwortindizes bereitstellen, d. h. den Import aus Textformaten, die programmatische Bearbeitung im Speicher und den Export in Textformate. Diese sind

   doctools::idx
   : Programmatische Bearbeitung von Schlüsselwortindizes im Speicher.

   doctools::idx::import
   : Import von Schlüsselwortindizes aus verschiedenen Textformaten. Die Liste der unterstützten Formate lässt sich durch Plugin-Pakete erweitern.

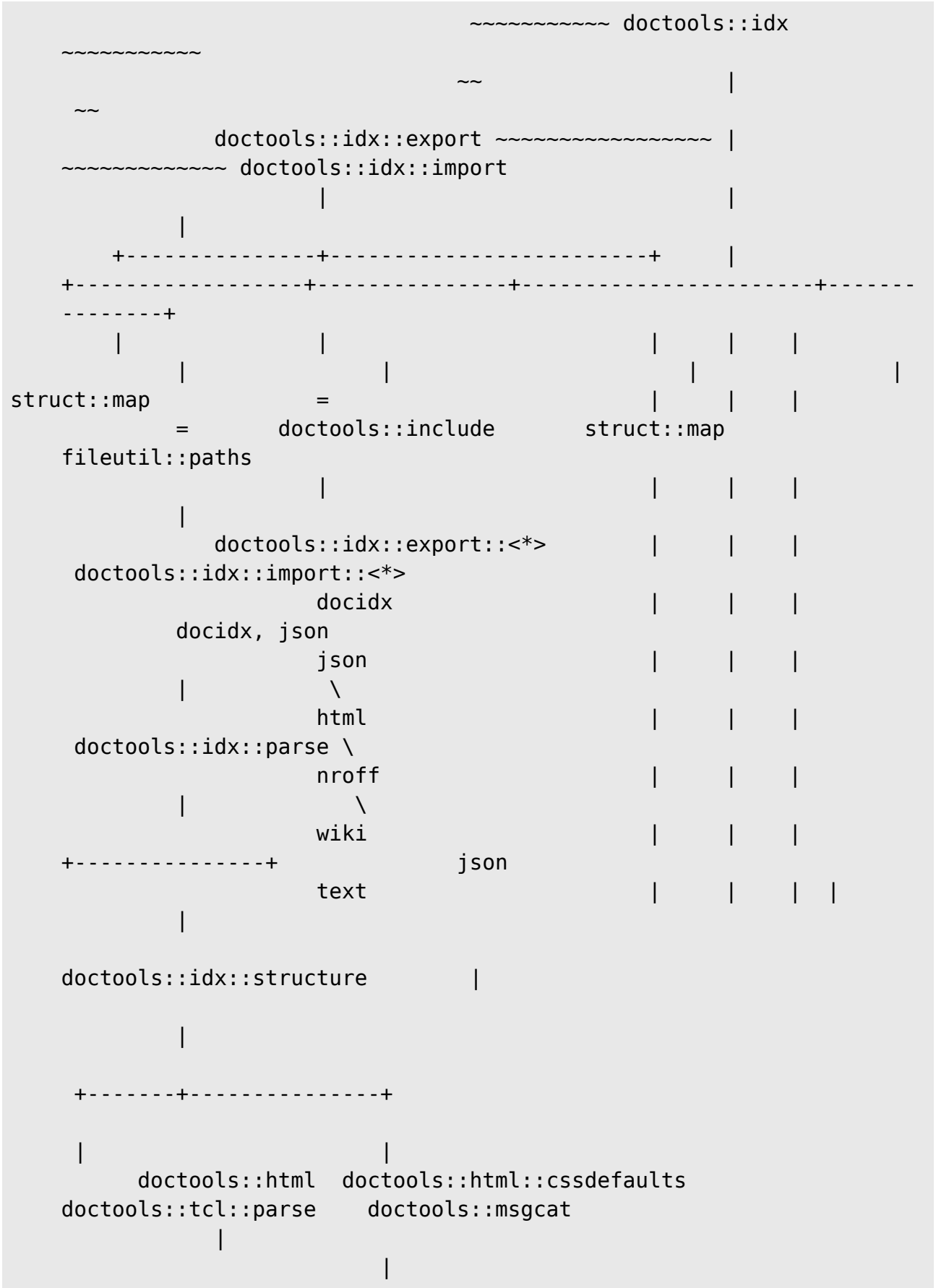
   doctools::idx::export
   : Export von Schlüsselwortindizes in verschiedene Textformate. Die Auswahl der unterstützten Formate lässt sich durch Plugin-Pakete erweitern.

1. Siehe auch den Abschnitt Paketübersicht für einen Überblick über die Abhängigkeiten zwischen diesen und anderen, unterstützenden Paketen.
2. Nicht zuletzt müssen *Plugin-Entwickler* das Zusammenspiel der Import- und Export-Pakete mit ihren Plugins verstehen. Diese APIs sind in der Dokumentation der beiden zuständigen Pakete beschrieben, also
   - **doctools::idx::import**
   - **doctools::idx::export**

Verwandte Formate

Das docidx-Format steht nicht für sich allein, sondern verfügt über zwei begleitende Formate. Diese heißen *doctoc* und *doctools* und sind jeweils für die Auszeichnung von *Inhaltsverzeichnissen* bzw. von allgemeiner Dokumentation vorgesehen. Sie werden in eigenen Dokumentensammlungen beschrieben, beginnend jeweils mit *DocTools – Inhaltsverzeichnisse* und *DocTools – Allgemein*.

Paketübersicht


~~~~~~~~~~~ doctools::idx
~~~~~~~~~~
~~
|
~~
doctools::idx::export ~~~~~~~~~~~~~~~~ |
~~~~~~~~~~~~ doctools::idx::import
|
|
|
+---------------+------------------------+
|
+-----------------+--------------+----------------------+-------
--------+
|
|
|
~~~~~~~~~~~~

```
          doctools::text  doctools::nroff::man_macros
                               =

                               |

                        doctools::msgcat::idx::<*>

                               c, en, de, fr

                               (fr == en for now)
        ~~      Interoperable objects, without actual package
    dependencies
        --      Package dependency, higher requires lower package
        =       Dynamic dependency through plugin system
        <*>     Multiple packages following the given form of naming.
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_c

Stichwörter: idx_msgcat_c, C, catalog package, docidx, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::idx::c – Nachrichtenkatalog für den docidx-Parser (C)

Übersicht

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::idx::c ?**0.1**?

Beschreibung

Das Paket **doctools::msgcat::idx::c** ist ein Unterstützungsmodul, das den Nachrichtenkatalog der Sprache C für den docidx-Parser im doctools-System der Version 2 bereitstellt. Als solches handelt es sich um ein internes Paket, mit dem ein normaler Benutzer (Entwickler) nicht direkt in Berührung kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

Innerhalb der Systemarchitektur liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System übernimmt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API
Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_de
Stichwörter: idx_msgcat_de, DE, catalog package, docidx, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name
doctools::msgcat::idx::de – Nachrichtenkatalog für den docidx-Parser (DE)

Übersicht
package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::idx::de ?**0.1**?

Beschreibung

Das Paket **doctools::msgcat::idx::de** ist ein Unterstützungsmodul, das den deutschsprachigen (DE) Nachrichtenkatalog für den docidx-Parser im doctools-System der Version 2 bereitstellt. Als solches handelt es sich um ein internes Paket, mit dem ein normaler Benutzer (Entwickler) nicht in direkten Kontakt kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

Innerhalb der Systemarchitektur liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System übernimmt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API

Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_en

Stichwörter: idx_msgcat_en, EN, catalog package, docidx, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::idx::en – Nachrichtenkatalog für den docidx-Parser (EN)

Übersicht

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::idx::en ?**0.1**?

Beschreibung

Das Paket **doctools::msgcat::idx::en** ist ein Unterstützungsmodul, das den englischsprachigen (EN) Nachrichtenkatalog für den docidx-Parser im doctools-System der Version 2 bereitstellt. Als solches handelt es sich um ein internes Paket, mit dem ein normaler Benutzer (Entwickler) nicht direkt in Berührung kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

Innerhalb der Systemarchitektur liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System übernimmt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API

Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_msgcat_fr

Stichwörter: idx_msgcat_fr, FR, catalog package, docidx, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::idx::fr – Nachrichtenkatalog für den docidx-Parser (FR)

Übersicht

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::idx::fr ?**0.1**?

Beschreibung

Das Paket **doctools::msgcat::idx::fr** ist ein Unterstützungsmodul, das den Nachrichtenkatalog in französischer Sprache (FR) für den docidx-Parser im doctools-System der Version 2 bereitstellt. Als solches handelt es sich um ein internes Paket, mit dem ein normaler Benutzer (Entwickler) nicht in direkten Kontakt kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

Innerhalb der Systemarchitektur liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System übernimmt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API

Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_parse

Stichwörter: idx_parse, docidx, doctools, lexer, parser

Name

doctools::idx::parse – Parsen von Text im docidx-Format

Übersicht

package require doctools::idx::parse ?**0.2**?
package require Tcl 8.5 9
package require doctools::idx::structure
package require doctools::msgcat

package require doctools::tcl::parse
package require fileutil
package require logger
package require snit
package require struct::list
package require struct::stack
::doctools::idx::parse text *text*
::doctools::idx::parse file *path*
::doctools::idx::parse includes
::doctools::idx::parse include add *path*
::doctools::idx::parse include remove *path*
::doctools::idx::parse include clear
::doctools::idx::parse vars
::doctools::idx::parse var set *name value*
::doctools::idx::parse var unset *name*
::doctools::idx::parse var clear ?*pattern*?

Beschreibung

Dieses Paket enthält Befehle zum Parsen von Text, der in der Markup-Sprache *docidx* verfasst ist, und zum Konvertieren dieses Textes in die kanonische Serialisierung des im Text kodierten Schlüsselwortindexes. Eine Beschreibung des Formats finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Paketen zur Verarbeitung von *docidx*-Dokumenten verwendet wird.

API

::doctools::idx::parse text *text*
: Der Befehl nimmt die in *text* enthaltene Zeichenkette entgegen und analysiert sie unter der Annahme, dass es sich um ein Dokument handelt, das mit der Markup-Sprache *docidx* verfasst wurde. Sollte sich diese Annahme als falsch erweisen, wird ein Fehler ausgelöst. Das Format dieser Fehler wird im Abschnitt Parsing-Fehler beschrieben.

  Wenn der Befehl erfolgreich ist, gibt er die kanonische Serialisierung des im Text kodierten Schlüsselwortindexes zurück. Eine Spezifikation dieses Formats finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

::doctools::idx::parse file *path*
: Entspricht **text**, mit dem Unterschied, dass der zu analysierende Text aus der durch *path* angegebenen Datei gelesen wird.

::doctools::idx::parse includes
: Diese Methode gibt die aktuelle Liste der Suchpfade zurück, die bei der Suche nach Include-Dateien verwendet werden.

::doctools::idx::parse include add *path*
: Diese Methode fügt den *path* zur Liste der Pfade hinzu, die bei der Suche nach einer Include-Datei durchsucht werden. Der Aufruf wird ignoriert, wenn der Pfad schon in der Liste steht. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::idx::parse include remove *path*

Diese Methode entfernt den *path* aus der Liste der Pfade, die bei der Suche nach einer Include-Datei durchsucht werden. Der Aufruf wird ignoriert, wenn der Pfad nicht in der Liste steht. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::idx::parse include clear
Diese Methode leert die Liste der Suchpfade für Include-Dateien.

::doctools::idx::parse vars
Diese Methode gibt ein Dictionary zurück, das die aktuellen vordefinierten Variablen enthält, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt.

::doctools::idx::parse var set *name value*
Diese Methode nimmt die Variable *name* in die Menge der vordefinierten Variablen auf, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt, und gibt ihr den angegebenen *value*. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::idx::parse var unset *name*
Diese Methode entfernt die Variable *name* aus der Menge der vordefinierten Variablen, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::idx::parse var clear ?*pattern*?
Diese Methode entfernt alle Variablen, auf die das *pattern* passt, aus der Menge der vordefinierten Variablen, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

Der Abgleich erfolgt mit **string match**, und das Vorgabemuster, wenn keines angegeben ist, ist *.

Parsing-Fehler

Das Format der Parsing-Fehlermeldungen, die bei Verstößen gegen die *docidx*-Markup-Syntax ausgegeben werden, ist für Menschen lesbar und nicht für die maschinelle Verarbeitung vorgesehen. Daher wird es nicht dokumentiert.

Allerdings ist der an die Meldung angehängte errorCode maschinenlesbar und hat das folgende Format:

1. Der Fehlercode ist eine Liste, wobei jedes Element einen einzelnen in der Eingabe gefundenen Fehler beschreibt. Die Liste enthält mindestens ein Element, möglicherweise auch mehrere.
2. Jedes Fehlerelement ist eine Liste, die sechs Zeichenfolgen enthält, welche einen Fehler detailliert beschreiben. Die Zeichenfolgen lauten
   1. Der Pfad der Datei, in der der Fehler aufgetreten ist. Dieser kann leer sein.
   2. Der Bereich des Tokens, an dem der Fehler gefunden wurde. Dieser Bereich ist eine Liste aus zwei Elementen mit dem Offset des ersten und des letzten Zeichens im Bereich, gezählt vom Anfang der Eingabe (der Datei). Offsets zählen ab null.
   3. Die Zeile, in der das erste Zeichen nach dem Fehler steht. Zeilen zählen ab eins.
   4. Die Spalte, in der das erste Zeichen nach dem Fehler steht. Spalten zählen ab null.
   5. Der Nachrichtencode des Fehlers. Dieser Wert kann als Argument von **msgcat::mc** dienen, um eine lokalisierte Fehlermeldung zu erhalten, sofern die Anwendung **doctools::msgcat::init** passend aufgerufen hat, um die nötigen Nachrichtenkataloge zu initialisieren (siehe Paket **doctools::msgcat**).
   6. Eine Auflistung der Details zum Fehler, wie beispielsweise der betroffene Markup-Befehl. Im

Falle des Meldungscodes **docidx/include/syntax** entspricht dieser Wert der Menge der in der eingebundenen Datei gefundenen Fehler, wobei das hier beschriebene Format verwendet wird.

[docidx]-Notation von Schlüsselwortindizes

Das docidx-Format für Schlüsselwortindizes, auch als *docidx-Markup-Sprache* bezeichnet, ist zu umfangreich, um in einem einzigen Abschnitt behandelt zu werden. Der interessierte Leser sollte mit dem Dokument beginnen

1. *Einleitung in die docidx-Sprache*

und von dort zu den formalen Spezifikationen weitergehen, d. h. den Dokumenten

1. *Syntax der docidx-Sprache* und
2. *Befehlsreferenz der docidx-Sprache*,

um die Sprache gründlich zu verstehen.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,
   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.
   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung
Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


idx_structure

Stichwörter: idx_structure, deserialization, docidx, doctools, serialization

Name

doctools::idx::structure – Docidx-Serialisierungsdienstprogramme

Übersicht

package require doctools::idx::structure ?**0.2**?
package require Tcl 8.5 9
package require logger
package require snit
::doctools::idx::structure verify *serial* ?*canonvar*?
::doctools::idx::structure verify-as-canonical *serial*
::doctools::idx::structure canonicalize *serial*
::doctools::idx::structure print *serial*
::doctools::idx::structure merge *seriala serialb*

Beschreibung

Dieses Paket stellt Befehle zur Verfügung, zur Bearbeitung der Serialisierungen von Schlüsselwortindizes, die vom doctools-System v2 verwaltet und im Abschnitt Serialisierungsformat für Schlüsselwortindizes beschrieben werden.

Hierbei handelt es sich um ein internes Paket von doctools, das von übergeordneten Paketen genutzt wird, die Schlüsselwortindizes verarbeiten und sie in verschiedene andere Formate und aus ihnen umwandeln, wie beispielsweise Dokumente, die mit *docidx*-Markup erstellt wurden.

API

::doctools::idx::structure verify *serial* ?*canonvar*?

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige *reguläre* Serialisierung eines Schlüsselwortindexes ist, und löst einen Fehler aus, wenn dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Wenn das Argument *canonvar* angegeben wird, wird es als Name einer Variablen im aufrufenden Kontext interpretiert. In diese Variable wird genau dann geschrieben, wenn *serial* eine gültige reguläre Serialisierung ist. Ihr Wert ist ein Boolescher Wert, wobei **True** angibt, dass die Serialisierung nicht nur gültig, sondern auch *kanonisch* ist. Bei einer gültigen, aber nicht kanonischen Serialisierung wird **False** geschrieben.

Informationen zur Spezifikation regulärer und kanonischer Schlüsselwortindex-Serialisierungen finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

::doctools::idx::structure verify-as-canonical *serial*

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige *kanonische* Serialisierung eines Schlüsselwortindexes darstellt, und löst einen Fehler aus, falls dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Informationen zur Spezifikation kanonischer Serialisierungen von Schlüsselwortindizes finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

::doctools::idx::structure canonicalize *serial*

Dieser Befehl geht davon aus, dass der Inhalt von *serial* eine gültige *reguläre* Serialisierung eines Schlüsselwortindexes ist, und löst einen Fehler aus, falls dies nicht der Fall ist.

Anschließend wandelt er die Eingabe in die *kanonische* Serialisierung des enthaltenen

Schlüsselwortindexes um und gibt diese als Ergebnis zurück. Wenn die Eingabe bereits kanonisch ist, wird sie unverändert zurückgegeben.

Informationen zur Spezifikation regulärer und kanonischer Index-Serialisierungen für Schlüsselwörter finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

::doctools::idx::structure print *serial*

Dieser Befehl geht davon aus, dass das Argument *serial* eine gültige reguläre Serialisierung eines Schlüsselwortindexes enthält, und gibt eine Zeichenkette zurück, die diesen Index in einer für Menschen lesbaren Form enthält.

Das genaue Format dieser Zeichenkette ist nicht festgelegt und darf nicht für die Analyse oder andere maschinelle Vorgänge herangezogen werden.

Informationen zur Spezifikation regulärer Schlüsselwortindex-Serialisierungen finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

::doctools::idx::structure merge *seriala serialb*

Dieser Befehl akzeptiert die regulären Serialisierungen zweier Schlüsselwortindizes und erstellt daraus deren Vereinigung. Das Ergebnis des Befehls ist die kanonische Serialisierung dieses vereinigten Schlüsselwortindexes.

Titel und Bezeichnung des resultierenden Indexes werden aus dem in *serialb* enthaltenen Index übernommen. Die Menge der Schlüsselwörter, Referenzen und ihrer Verbindungen ist die Vereinigung der Mengen der Schlüsselwörter und Referenzen der beiden Eingaben.

Informationen zur Spezifikation regulärer und kanonischer Schlüsselwortindex-Serialisierungen finden Sie im Abschnitt Serialisierungsformat für Schlüsselwortindizes.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title

   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label

Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

keywords
Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

references
Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,

manpage
Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.

url
Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung
Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

import_docidx

Stichwörter: import_docidx, deserialization, docidx, doctools, import, index

Name

doctools::idx::import::docidx – docidx-Import-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::idx::import::docidx ?**0.2.1**?
package require doctools::idx::parse
package require doctools::idx::structure
package require doctools::msgcat
package require doctools::tcl::parse
package require fileutil
package require logger
package require snit
package require struct::list
package require struct::set
package require struct::stack
package require struct::tree
package require treeql
import *string configuration*

Beschreibung

Dieses Paket implementiert das doctools-Import-Plugin für Schlüsselwortindizes zum Parsen von docidx-Markup.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Verwaltungspaketen zur Verwaltung von Schlüsselwortindizes verwendet wird, insbesondere von **doctools::idx::import**, dem Import-Manager.

Die Nutzung über einen herkömmlichen Interpreter ist zwar möglich, jedoch nur auf Umwegen, und wird nicht empfohlen. Die korrekte Vorgehensweise zur Nutzung dieser Funktionalität erfolgt über das Paket **doctools::idx::import** und die darin bereitgestellten Import-Manager-Objekte.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der docidx-Import-Plugin-API, Version 2.

import *string configuration*

Dieser Befehl nimmt den *string* und parst ihn im Kontext der angegebenen *configuration* (ein

Dictionary) als docidx-Markup, das einen Schlüsselwortindex kodiert. Das Ergebnis des Befehls ist die kanonische Serialisierung dieses Schlüsselwortindexes in der im Abschnitt Serialisierungsformat für Schlüsselwortindizes angegebenen Form.

[docidx]-Notation von Schlüsselwortindizes

Das docidx-Format für Schlüsselwortindizes, auch als *docidx-Markup-Sprache* bezeichnet, ist zu umfangreich, um in einem einzigen Abschnitt behandelt zu werden. Der interessierte Leser sollte mit dem Dokument beginnen

1. *Einleitung in die docidx-Sprache*

und von dort zu den formalen Spezifikationen weitergehen, d. h. den Dokumenten

1. *Syntax der docidx-Sprache* und
2. *Befehlsreferenz der docidx-Sprache*,

um die Sprache gründlich zu verstehen.

Serialisierungsformat für Schlüsselwortindizes

Hier legen wir das Format fest, das von den doctools-v2-Paketen verwendet wird, um Schlüsselwortindizes als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Schlüsselwortindex kann zwar mehr als eine reguläre Serialisierung aufweisen, doch genau eine davon ist *kanonisch* .

reguläre Serialisierung

1. Eine Index-Serialisierung ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **doctools::idx**, sowie dessen Wert. Dieser Wert enthält den Inhalt des Index.
2. Der Inhalt des Indexes besteht aus einem Tcl-Dictionary, das den Titel des Indexes, eine Bezeichnung sowie die Schlüsselwörter und Verweise enthält. Die entsprechenden Schlüssel und ihre Werte lauten

   title
   Der Wert ist eine Zeichenkette, die den Titel des Indexes enthält.

   label
   Der Wert ist eine Zeichenkette, die eine Bezeichnung für den Index enthält.

   keywords
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Schlüsselwörter als Schlüssel verwendet. Die zugehörigen Werte sind Listen, die die Bezeichner der Referenzen enthalten, die diesem bestimmten Schlüsselwort zugeordnet sind.

   Jeder in diesen Listen verwendete Referenzbezeichner muss als Schlüssel im Dictionary **references** vorhanden sein; dessen Definition steht im nächsten Punkt.

   references
   Der Wert ist ein Tcl-Dictionary, das die dem Index bekannten Bezeichner der Referenzen als Schlüssel verwendet. Die zugehörigen Werte sind Listen mit zwei Elementen, die in dieser Reihenfolge den Typ und die Bezeichnung der Referenz enthalten.

Jeder Schlüssel hier muss mindestens einem Schlüsselwort zugeordnet sein, d. h., er muss in mindestens einer der Referenzlisten vorkommen, die die Werte im Dictionary **keywords** bilden; die Definition hierfür steht im vorherigen Punkt.

1. Der *type* einer Referenz kann einen von zwei Werten annehmen,
   manpage
   Der Bezeichner der Referenz wird als symbolischer Dateiname interpretiert, der sich auf eines der Dokumente bezieht, für die der Index erstellt wurde.
   url
   Der Bezeichner der Referenz wird als URL interpretiert, die auf einen externen Ort verweist, beispielsweise eine Website usw.

kanonische Serialisierung
Die kanonische Serialisierung eines Schlüsselwortindexes hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen des Schlüsselwortindexes eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die für jedes Schlüsselwort des Indexes aufgeführten Referenzen – sofern vorhanden – werden in aufsteigender Wörterbuchordnung ihrer *Bezeichnungen* aufgelistet, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


doctools2toc

export_doctoc

Stichwörter: export_doctoc, doctoc, doctools, export, serialization, table of contents, toc

Name

doctools::toc::export::doctoc – Doctoc-Export-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::toc::export::doctoc ?**0.2.2**?
export *serial configuration*

Beschreibung

Dieses Paket implementiert das Export-Plugin von doctools für Inhaltsverzeichnisse, zur Erzeugung von doctoc-Markup.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::export**, den Export-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::export** und die Export-Manager-Objekte, die es bereitstellt.

API

Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Inhaltsverzeichnisses entgegen, wie im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt und in *serial* enthalten, dazu die *configuration*, ein Dictionary, und erzeugt doctoc-Markup, das das Inhaltsverzeichnis kodiert. Die erzeugte Zeichenkette wird als Ergebnis des Befehls zurückgegeben.

[doctoc]-Notation von Inhaltsverzeichnissen

Das doctoc-Format für Inhaltsverzeichnisse, auch *doctoc markup language* genannt, ist zu umfangreich für einen einzigen Abschnitt. Der interessierte Leser beginnt mit dem Dokument

1. *doctoc language introduction*

und geht von dort weiter zu den formalen Spezifikationen, also den Dokumenten

1. *doctoc language syntax* und
2. *doctoc language command reference*,

um die Sprache gründlich zu verstehen.

Konfiguration

Das doctoc-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

string *user*
: Diese Standard-Konfigurationsvariable enthält den Namen des Benutzers, der den Prozess ausführt, welcher das Export-Plugin aufgerufen hat. Das Plugin schreibt diese Angabe in den Herkunftskommentar am Anfang des erzeugten Dokuments.

string *file*
: Diese Standard-Konfigurationsvariable enthält den Namen der Datei, aus der das Inhaltsverzeichnis stammt. Diese Variable kann ungesetzt sein oder die leere Zeichenkette

enthalten. Das Plugin schreibt diese Angabe, wenn sie definiert ist, das heißt gesetzt und nicht die leere Zeichenkette, in den Herkunftskommentar am Anfang des erzeugten Dokuments.

boolean *newlines*
Ist dieses Flag gesetzt, bricht das Plugin den erzeugten doctoc-Code auf mehrere Zeilen um, mit jedem Markup-Befehl in einer eigenen Zeile.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird das ganze Dokument in eine einzige Zeile geschrieben, mit möglichst wenig Abstand zwischen allen Elementen.

boolean *indented*
Ist dieses Flag gesetzt, rückt das Plugin die Markup-Befehle entsprechend dem Aufbau der Inhaltsverzeichnisse ein. Damit das wirkt, ist auch **newlines** gesetzt. Diese Wirkung hängt nicht vom Wert von **aligned** ab.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird die Ausgabe gemäß dem Wert von **newlines** formatiert, und es wird nicht eingerückt.

boolean *aligned*
Ist dieses Flag gesetzt, sorgt der Generator dafür, dass die Argumente der Befehle **item** in einem Abschnitt senkrecht zueinander ausgerichtet sind, mit dem Effekt einer Tabelle. Damit das wirkt, ist auch **newlines** gesetzt. Diese Wirkung hängt nicht vom Wert von **indented** ab.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird die Ausgabe gemäß den Werten von **newlines** und **indented** formatiert, und es wird nichts ausgerichtet.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **format** und **map** sowie ihre Werte.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization
1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items

Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

reference
: Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

id
: Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

label
: Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
: Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
    : Der Wert ist eine Zeichenkette mit der Kennung des
Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
optional.

    **label**
    : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

    **items**
    : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
```

```
    in der Reihenfolge, in der sie angezeigt werden sollen. Diese
    Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


import_doctoc

Stichwörter: import_doctoc, deserialization, doctoc, doctools, import, table of contents, toc

Name

doctools::toc::import::doctoc – Doctoc-Import-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::toc::import::doctoc ?**0.2.1**?
package require doctools::toc::parse
package require doctools::toc::structure
package require doctools::msgcat
package require doctools::tcl::parse
package require fileutil

package require logger
package require snit
package require struct::list
package require struct::set
package require struct::stack
package require struct::tree
package require treeql
import *string configuration*

Beschreibung

Dieses Paket implementiert das Import-Plugin von doctools für Inhaltsverzeichnisse, zum Parsen von doctoc-Markup.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::import**, den Import-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::import** und die Import-Manager-Objekte, die es bereitstellt.

API

Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Import-Plugin-API, Version 2.

import *string configuration*

Dieser Befehl nimmt die *string* entgegen und parst sie als doctoc-Markup, das ein Inhaltsverzeichnis kodiert, im Kontext der angegebenen *configuration* (ein Dictionary). Das Ergebnis des Befehls ist die kanonische Serialisierung dieses Inhaltsverzeichnisses, in der Form aus dem Abschnitt Serialisierungsformat für Inhaltsverzeichnisse.

[doctoc]-Notation von Inhaltsverzeichnissen

Das doctoc-Format für Inhaltsverzeichnisse, auch *doctoc markup language* genannt, ist zu umfangreich für einen einzigen Abschnitt. Der interessierte Leser beginnt mit dem Dokument

1. *doctoc language introduction*

und geht von dort weiter zu den formalen Spezifikationen, also den Dokumenten

1. *doctoc language syntax* und
2. *doctoc language command reference*,

um die Sprache gründlich zu verstehen.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes

Tcl-Dictionary.

1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

   desc
   Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
```

```
    : Der Wert ist eine Zeichenkette mit der Kennung des
  Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
  optional.

    **label**
    : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
  die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
  ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
  der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

    **items**
    : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
  in der Reihenfolge, in der sie angezeigt werden sollen. Diese
  Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
  für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
  sich die rekursive Definition der Struktur: Abschnitte enthalten
  dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
  einschließlich weiterer Abschnitte.
```

canonical serialization

Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_container

Stichwörter: toc_container, HTML, TMML, conversion, doctoc markup, documentation, formatting,

generation, json, latex, markup, nroff, parsing, plugin, reference, table, table of contents, tcler's wiki, text, wiki

Name

doctools::toc – Aufnehmen von Inhaltsverzeichnissen

Übersicht

package require doctools::toc ?**2**?
package require Tcl 8.5 9
package require doctools::toc::structure
package require struct::tree
package require snit
::doctools::toc *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **+ reference** *id label docid desc*
objectName **+ division** *id label* ?*docid*?
objectName **remove** *id*
objectName **up** *id*
objectName **next** *id*
objectName **prev** *id*
objectName **child** *id label* ?*...*?
objectName **element** ?*...*?
objectName **children** *id*
objectName **type** *id*
objectName **full-label** *id*
objectName **elabel** *id* ?*newlabel*?
objectName **description** *id* ?*newdesc*?
objectName **document** *id* ?*newdocid*?
objectName **title**
objectName **title** *text*
objectName **label**
objectName **label** *text*
objectName **importer**
objectName **importer** *object*
objectName **exporter**
objectName **exporter** *object*
objectName **deserialize =** *data* ?*format*?
objectName **deserialize +=** *data* ?*format*?
objectName **serialize** ?*format*?

Beschreibung

Dieses Paket stellt eine Klasse bereit, die Inhaltsverzeichnisse aufnimmt und programmgesteuert bearbeitet.

Dies ist eine der drei öffentlichen Säulen, auf denen die Verwaltung von Inhaltsverzeichnissen beruht. Die beiden anderen Säulen sind

1. *Exportieren von Inhaltsverzeichnissen*, und
2. *Importieren von Inhaltsverzeichnissen*

Zu den Begriffe der Inhaltsverzeichnisse und ihrer Teile siehe den gleichnamigen Abschnitt. Zur Datenstruktur, mit der Inhaltsverzeichnisse als Werte kodiert werden, siehe den Abschnitt Serialisierungsformat für Inhaltsverzeichnisse. Das ist das einzige Format, das diese Klasse unmittelbar kennt. Umwandlungen von und in jedes andere Format übernehmen Export- und Import-Manager-Objekte. Die lassen sich einem Container zuordnen, müssen es aber nicht; das ist nur eine Bequemlichkeit.

Begriffe

1. Ein *Inhaltsverzeichnis* besteht aus einer (möglicherweise leeren) Liste von *Elementen*.
2. Jedes Element in der Liste wird durch seine Bezeichnung gekennzeichnet.
3. Jedes Element ist entweder ein *Verweis* oder ein *Abschnitt*.
4. Jeder Verweis hat ein zugehöriges Dokument, das durch eine symbolische Kennung identifiziert wird, sowie eine Textbeschreibung.
5. Jeder Abschnitt kann ein zugehöriges Dokument haben, das durch eine symbolische Kennung identifiziert wird.
6. Jeder Abschnitt besteht aus einer (möglicherweise leeren) Liste von *Elementen*, und jedes Element folgt den Regeln aus Punkt 2 und den darüber stehenden Punkten.

Einige Anmerkungen

1. Die genannten Regeln spannen einen Baum von Elementen auf, mit Verweisen als Blättern und Abschnitten als inneren Knoten, und jedes Element ist ein Eintrag im ganzen Inhaltsverzeichnis.
2. Die kennzeichnenden Bezeichnungen eines Elements E sind innerhalb seines Abschnitts (oder des Inhaltsverzeichnisses) eindeutig, und die volle Bezeichnung eines Elements E ist die Liste der Bezeichnungen aller Knoten auf dem eindeutigen Pfad von der Wurzel des Baums nach E, einschließlich E.

API

Paketbefehle

::doctools::toc *objectName*

Dieser Befehl erzeugt ein neues Container-Objekt mit einem zugehörigen Tcl-Befehl namens *objectName*. Dieser Befehl *object* ist in den Abschnitten Objektbefehl und Objektmethoden vollständig beschrieben. Der Objektbefehl entsteht im aktuellen Namespace, wenn *objectName* nicht vollqualifiziert ist, sonst im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::doctools::toc** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

objectName* + reference *id label docid desc

Diese Methode fügt dem Inhaltsverzeichnis ein neues Verweis-Element hinzu, unter dem Element, das über sein Handle *id* angegeben ist. Dieses Elternelement muss ein Abschnitt oder die Wurzel sein. Andernfalls wird ein Fehler ausgelöst. Das neue Element wird nach außen durch sein *label* identifiziert, das innerhalb des Elternelements eindeutig sein muss. Andernfalls wird ein Fehler ausgelöst.

Als Verweis bezieht es sich auf ein Dokument, das durch die symbolische Kennung *docid* identifiziert wird. Diese Kennung darf nicht die leere Zeichenkette sein, sonst wird ein Fehler ausgelöst. Neben der Bezeichnung hat das Element eine längere beschreibende Zeichenkette, übergeben in *desc*.

Das Ergebnis der Methode ist das Handle (id) des neuen Elements.

***objectName* + division *id label* ?*docid*?**

Diese Methode fügt dem Inhaltsverzeichnis ein neues Abschnitts-Element hinzu, unter dem Element, das über sein Handle *id* angegeben ist. Dieses Elternelement muss ein Abschnitt oder die Wurzel sein. Andernfalls wird ein Fehler ausgelöst. Das neue Element wird nach außen durch sein *label* identifiziert, das innerhalb des Elternelements eindeutig sein muss. Andernfalls wird ein Fehler ausgelöst.

Als Abschnitt kann es auf ein Dokument verweisen, das durch die symbolische Kennung *docid* identifiziert wird, muss das aber nicht.

Das Ergebnis der Methode ist das Handle (id) des neuen Elements.

objectName* remove *id

Diese Methode entfernt das durch das Handle *id* bestimmte Element aus dem Inhaltsverzeichnis. Ist das Element ein Abschnitt, werden auch alle seine Kinder entfernt, sofern vorhanden. Das Wurzelelement des Inhaltsverzeichnisses lässt sich dagegen nicht entfernen, nur seine Kinder.

Das Ergebnis der Methode ist die leere Zeichenkette.

objectName* up *id

Diese Methode gibt das Handle des Elternelements des durch sein Handle *id* bestimmten Elements zurück, oder die leere Zeichenkette, wenn *id* die Wurzel bezeichnet.

objectName* next *id

Diese Methode gibt das Handle des rechten Geschwisterelements des durch sein Handle *id* bestimmten Elements zurück, oder das Handle des Elternelements, wenn das Element kein rechtes Geschwisterelement hat, oder die leere Zeichenkette, wenn *id* die Wurzel bezeichnet.

objectName* prev *id

Diese Methode gibt das Handle des linken Geschwisterelements des durch sein Handle *id* bestimmten Elements zurück, oder das Handle des Elternelements, wenn das Element kein linkes Geschwisterelement hat, oder die leere Zeichenkette, wenn *id* die Wurzel bezeichnet.

***objectName* child *id label* ?*...*?**

Diese Methode gibt das Handle eines Kindes des durch sein Handle *id* bestimmten Elements zurück. Das Kind selbst ist durch eine Folge von Bezeichnungen bestimmt.

***objectName* element ?*...*?**
Diese Methode gibt das Handle des Elements zurück, das durch eine Folge von Bezeichnungen bestimmt ist, beginnend bei der Wurzel des Inhaltsverzeichnisses. Die Folge der Bezeichnungen darf leer sein; dann wird das Handle des Wurzelelements zurückgegeben.

objectName* children *id
Diese Methode gibt eine Liste mit den Handles aller Kinder des durch das Handle *id* bestimmten Elements zurück, vom ersten bis zum letzten, in dieser Reihenfolge.

objectName* type *id
Diese Methode gibt den Typ des Elements zurück, entweder **reference** oder **division**.

objectName* full-label *id
Diese Methode ist das Gegenstück zur Methode **element**. Sie wandelt das Handle *id* eines Elements in eine Liste von Bezeichnungen um, die das Element im ganzen Inhaltsverzeichnis vollständig bestimmen.

***objectName* elabel *id* ?*newlabel*?**
Diese Methode fragt die Bezeichnung des durch das Handle *id* bestimmten Elements ab und ändert sie gegebenenfalls. Ist das Argument *newlabel* vorhanden, wird die Bezeichnung auf diesen Wert gesetzt. In jedem Fall ist das Ergebnis der aktuelle Wert der Bezeichnung.

Wird die Bezeichnung geändert, muss die neue Bezeichnung innerhalb des enthaltenden Abschnitts eindeutig sein, sonst wird ein Fehler ausgelöst.

Bezieht sich *id* auf das Wurzelelement des Inhaltsverzeichnisses, ist diese Methode gleichbedeutend mit der Methode *label*, sie greift also auf die globale Bezeichnung des ganzen Inhaltsverzeichnisses zu.

***objectName* description *id* ?*newdesc*?**
Diese Methode fragt die Beschreibung des durch das Handle *id* bestimmten Elements ab und ändert sie gegebenenfalls. Ist das Argument *newdesc* vorhanden, wird die Beschreibung auf diesen Wert gesetzt. In jedem Fall ist das Ergebnis der aktuelle Wert der Beschreibung.

Das Element muss ein Verweis sein, sonst wird ein Fehler ausgelöst.

***objectName* document *id* ?*newdocid*?**
Diese Methode fragt den Dokumentverweis des durch das Handle *id* bestimmten Elements ab und ändert ihn gegebenenfalls. Ist das Argument *newdocid* vorhanden, wird der Dokumentverweis auf diesen Wert gesetzt. In jedem Fall ist das Ergebnis der aktuelle Wert des Dokumentverweises.

Den Verweis auf die leere Zeichenkette zu setzen hebt ihn auf und ist nur bei Abschnitten zulässig. Ist das Ergebnis umgekehrt die leere Zeichenkette, hat das Element keinen Dokumentverweis, und das kommt nur bei Abschnitten vor.

***objectName* title**
Gibt den aktuell festgelegten Titel des Inhaltsverzeichnisses zurück.

objectName* title *text
Setzt den Titel des Inhaltsverzeichnisses auf *text* und gibt ihn als Ergebnis des Befehls zurück.

***objectName* label**
Gibt die aktuell festgelegte Bezeichnung des Inhaltsverzeichnisses zurück.

objectName* label *text
Setzt die Bezeichnung des Inhaltsverzeichnisses auf *text* und gibt sie als Ergebnis des Befehls zurück.

***objectName* importer**
Gibt das Import-Manager-Objekt zurück, das dem Container derzeit zugeordnet ist, sofern eines zugeordnet ist.

objectName* importer *object
Ordnet das *object* dem Container als Import-Manager zu und gibt es als Ergebnis des Befehls zurück. Das *object* geht dabei *nicht* in den Besitz des Containers über. Die Zerstörung des Containers zerstört das *object* also *nicht*.

Erwartet wird, dass *object* eine Methode **import text** bereitstellt, die einen Text und einen Formatnamen entgegennimmt und die kanonische Serialisierung des im Text enthaltenen Inhaltsverzeichnisses zurückgibt, unter der Annahme des angegebenen Formats.

***objectName* exporter**
Gibt das Export-Manager-Objekt zurück, das dem Container derzeit zugeordnet ist, sofern eines zugeordnet ist.

objectName* exporter *object
Ordnet das *object* dem Container als Export-Manager zu und gibt es als Ergebnis des Befehls zurück. Das *object* geht dabei *nicht* in den Besitz des Containers über. Die Zerstörung des Containers zerstört das *object* also *nicht*.

Erwartet wird, dass *object* eine Methode **export object** bereitstellt, die den Container und einen Formatnamen entgegennimmt und einen Text zurückgibt, der das im Container gespeicherte Inhaltsverzeichnis im angegebenen Format kodiert. Weiter wird erwartet, dass das *object* die Methode **serialize** des Containers benutzt, um die Serialisierung des Inhaltsverzeichnisses zu holen, aus der der Text entsteht.

***objectName* deserialize = *data* ?*format*?**
Diese Methode ersetzt den Inhalt des Inhaltsverzeichnisobjekts durch das in *data* enthaltene Inhaltsverzeichnis. Wird kein *format* angegeben, gilt *data* als die reguläre Serialisierung eines Inhaltsverzeichnisses.

Andernfalls wandelt das Objekt die Daten über den zugeordneten Import-Manager aus dem angegebenen Format in eine Serialisierung um, die es verarbeiten kann. In diesem Fall wird ein Fehler ausgelöst, wenn dem Container kein Import-Manager zugeordnet ist.

Das Ergebnis der Methode ist die leere Zeichenkette.

***objectName* deserialize += *data* ?*format*?**
Diese Methode verhält sich im Wesentlichen wie **deserialize =**, führt das Inhaltsverzeichnis in *data* aber mit dem eigenen Inhalt zusammen, statt ihn zu ersetzen. Die Methode löst einen Fehler aus, wenn das Zusammenführen nicht möglich ist, also ein ungültiges Inhaltsverzeichnis ergäbe. Der vorhandene Inhalt bleibt dann unverändert.

Das Ergebnis der Methode ist die leere Zeichenkette.

***objectName* serialize ?*format*?**

Diese Methode gibt das im Objekt enthaltene Inhaltsverzeichnis zurück. Wird kein *format* angegeben, ist das Ergebnis die kanonische Serialisierung seines Inhalts.

Andernfalls wandelt das Objekt die Daten über den zugeordneten Export-Manager in das angegebene Format um. In diesem Fall wird ein Fehler ausgelöst, wenn dem Container kein Export-Manager zugeordnet ist.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge

(Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
    : Der Wert ist eine Zeichenkette mit der Kennung des
Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
optional.

    **label**
    : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

    **items**
    : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
in der Reihenfolge, in der sie angezeigt werden sollen. Diese
Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
sich die rekursive Definition der Struktur: Abschnitte enthalten
dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright
```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_export
Stichwörter: toc_export, HTML, conversion, doctoc, documentation, export, formatting, generation, json, manpage, markup, nroff, plugin, reference, table, table of contents, tcler's wiki, text, url, wiki

Name
doctools::toc::export – Exportieren von Inhaltsverzeichnissen

Übersicht
package require doctools::toc::export ?**0.2.2**?
package require Tcl 8.5 9
package require struct::map
package require doctools::toc::structure
package require snit
package require pluginmgr
::doctools::toc::export *objectName*
objectName method ?*arg arg* ...?
objectName **destroy**
objectName **export serial** *serial* ?*format*?
objectName **export object** *object* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*
export *serial configuration*

Beschreibung
Dieses Paket stellt eine Klasse zur Verwaltung der Plugins für den Export von Inhaltsverzeichnissen in andere Formate bereit, d. h. für deren Konvertierung beispielsweise in *doctoc*, *HTML* usw.

Dies ist eine der drei öffentlichen Säulen, auf denen die Verwaltung von Inhaltsverzeichnissen beruht. Die beiden anderen Säulen sind

1. *Importieren von Inhaltsverzeichnissen*, und

2. *Aufnehmen von Inhaltsverzeichnissen*

Zu den Begriffe der Inhaltsverzeichnisse und ihrer Teile siehe den gleichnamigen Abschnitt. Zur Datenstruktur, die den von diesem Paket bereitgestellten Manager-Objekten im Wesentlichen als Eingabe dient, siehe den Abschnitt Serialisierungsformat für Inhaltsverzeichnisse.

Das Plugin-System der Klasse beruht auf dem Paket **pluginmgr** und sucht Plugins über

1. die Umgebungsvariable **DOCTOOLS_TOC_EXPORT_PLUGINS**,
2. die Umgebungsvariable **DOCTOOLS_TOC_PLUGINS**,
3. die Umgebungsvariable **DOCTOOLS_PLUGINS**,
4. der Pfad "*~/.doctools/toc/export/plugin*"
5. der Pfad "*~/.doctools/toc/plugin*"
6. der Pfad "*~/.doctools/plugin*"
7. der Pfad "*~/.doctools/toc/export/plugins*"
8. der Pfad "*~/.doctools/toc/plugins*"
9. der Pfad "*~/.doctools/plugins*"
10. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\EXPORT\PLUGINS"
11. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\PLUGINS"
12. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS".

Die letzten drei werden nur verwendet, wenn das Paket auf einem Rechner mit dem Betriebssystem Windows(tm) ausgeführt wird.

Das gesamte System wird mit sechs vordefinierten Export-Plugins ausgeliefert, nämlich

doctoc
: Einzelheiten stehen in *doctoc export plugin*.

html
: Einzelheiten stehen in *html export plugin*.

json
: Einzelheiten stehen in *json export plugin*.

nroff
: Einzelheiten stehen in *nroff export plugin*.

text
: Einzelheiten stehen in *text export plugin*.

wiki
: Einzelheiten stehen in *wiki export plugin*.

Wer ein eigenes Export-Plugin für ein Format schreiben will, also ein *Plugin-Entwickler*, muss den Abschnitt Referenz zur Export-Plugin-API v2 lesen und verstehen. Er legt die Interaktion zwischen diesem Paket und seinen Plugins im Einzelnen fest.

Begriffe

1. Ein *Inhaltsverzeichnis* besteht aus einer (möglicherweise leeren) Liste von *Elementen*.
2. Jedes Element in der Liste wird durch seine Bezeichnung gekennzeichnet.
3. Jedes Element ist entweder ein *Verweis* oder ein *Abschnitt*.
4. Jeder Verweis hat ein zugehöriges Dokument, das durch eine symbolische Kennung identifiziert wird, sowie eine Textbeschreibung.
5. Jeder Abschnitt kann ein zugehöriges Dokument haben, das durch eine symbolische Kennung

identifiziert wird.
6. Jeder Abschnitt besteht aus einer (möglicherweise leeren) Liste von *Elementen*, und jedes Element folgt den Regeln aus Punkt 2 und den darüber stehenden Punkten.

Einige Anmerkungen

1. Die genannten Regeln spannen einen Baum von Elementen auf, mit Verweisen als Blättern und Abschnitten als inneren Knoten, und jedes Element ist ein Eintrag im ganzen Inhaltsverzeichnis.
2. Die kennzeichnenden Bezeichnungen eines Elements E sind innerhalb seines Abschnitts (oder des Inhaltsverzeichnisses) eindeutig, und die volle Bezeichnung eines Elements E ist die Liste der Bezeichnungen aller Knoten auf dem eindeutigen Pfad von der Wurzel des Baums nach E, einschließlich E.

API

Paketbefehle

::doctools::toc::export *objectName*

Dieser Befehl erzeugt ein neues Export-Manager-Objekt mit einem zugehörigen Tcl-Befehl namens *objectName*. Dieser Befehl *object* ist in den Abschnitten Objektbefehl und Objektmethoden vollständig beschrieben. Der Objektbefehl entsteht im aktuellen Namespace, wenn *objectName* nicht vollqualifiziert ist, sonst im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::doctools::toc::export** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

***objectName* export serial *serial* ?*format*?**

Diese Methode nimmt die kanonische Serialisierung eines Inhaltsverzeichnisses in *serial* und wandelt sie mit dem Export-Plugin für das angegebene *format* in dieses *format* um. Wird kein Plugin für das Format gefunden, wird ein Fehler ausgelöst. Die durch die Umwandlung erzeugte Zeichenkette wird als Ergebnis dieser Methode zurückgegeben.

Wird kein Format angegeben, verwendet die Methode **doctoc**.

Was eine *kanonische* Serialisierung ist, steht im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse.

Das Plugin muss der Schnittstelle im Abschnitt Referenz zur Export-Plugin-API v2 entsprechen.

***objectName* export object *object* ?*format*?**

Diese Methode ist eine bequeme Hülle um die im vorigen Punkt beschriebene Methode **export serial**. Sie erwartet, dass *object* ein Objektbefehl mit einer Methode **serialize** ist, die

die kanonische Serialisierung eines Inhaltsverzeichnisses zurückgibt. Sie ruft diese Methode auf, gibt das Ergebnis an **export serial** weiter und gibt die entstandene Zeichenkette als eigenes Ergebnis zurück.

***objectName* config names**
Diese Methode gibt eine Liste zurück, die die Namen aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config get**
Diese Methode gibt ein Dictionary zurück, das die Namen und Werte aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config set *name* ?*value*?**
Diese Methode setzt die Konfigurationsvariable *name* auf den angegebenen *value* und gibt den neuen Wert der Variablen zurück.

Wird kein Wert angegeben, gibt sie nur den aktuellen Wert zurück, ohne ihn zu ändern.

Der Benutzer kann die vordefinierten Konfigurationsvariablen **user** und **format** zwar setzen, das bleibt aber ohne Wirkung; diese Werte werden beim Aufruf eines Export-Plugins intern überschrieben.

***objectName* config unset *pattern*...**
Diese Methode löscht alle Konfigurationsvariablen, auf die die angegebenen *pattern*s passen. Wird kein Muster angegeben, löscht sie alle derzeit definierten Konfigurationsvariablen.

Referenz zur Export-Plugin-API v2

Plugins verwalten in diesem Paket die Unterstützung für jedes Ausgabeformat jenseits des Serialisierungsformat für Inhaltsverzeichnisse. Hier ist die API festgelegt, über die die von diesem Paket erzeugten Objekte mit ihren Plugins verkehren.

Ein Plugin für dieses Paket muss die nachfolgend aufgeführten Regeln einhalten:

1. Ein Plugin ist ein Paket.
2. Der Name eines Plugin-Pakets hat die Form doctools::toc::export::**FOO**, wobei **FOO** der Name des Formats ist, für das das Plugin die Ausgabe erzeugt. Dieser Name ist auch das Argument der verschiedenen Methoden **export** der Export-Manager-Objekte, um eine Zeichenkette zu erhalten, die ein Inhaltsverzeichnis in diesem Format kodiert.
3. Das Plugin kann davon ausgehen, dass das Paket **doctools::toc::export::plugin** vorhanden ist, was darauf hindeutet, dass es von einem echten Plugin-Manager aufgerufen wurde.
4. Ein Plugin muss einen Befehl bereitstellen, dessen Signatur wie unten dargestellt lautet.

   export *serial configuration*
   Immer wenn ein Export-Manager von **doctools::toc** eine Ausgabe für ein Inhaltsverzeichnis erzeugen muss, ruft er diesen Befehl auf.
   string *serial*
   Dieses Argument enthält die *kanonische* Serialisierung des Inhaltsverzeichnisses, für das die Ausgabe erzeugt wird. Was eine *kanonische* Serialisierung ist, steht im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse.
   dictionary *configuration*
   Dieses Argument enthält die bei der Erzeugung anzuwendende Konfiguration, als Dictionary von Variablennamen auf Werte.

Die folgenden Konfigurationsvariablen haben eine vorgegebene Bedeutung, an die sich alle Plugins halten müssen, auch wenn sie diese Angabe nach eigenem Ermessen ignorieren können. Jede weitere Konfigurationsvariable, die ein Plugin erkennt, ist in der Handbuchseite dieses Plugins beschrieben.

user
Diese Variable soll den Namen des Benutzers enthalten, dem der Prozess gehört, der das Plugin aufruft.

format
Diese Variable soll den Namen des Formats enthalten, dessen Plugin aufgerufen wird.

file
Diese Variable soll, wenn der Benutzer des Inhaltsverzeichnisobjekts sie definiert, den Namen der Eingabedatei enthalten, für die das Plugin seine Ausgabe erzeugt.

map
Diese Variable soll, wenn der Benutzer des Inhaltsverzeichnisobjekts sie definiert, ein Dictionary enthalten, das die in den Einträgen verwendeten symbolischen Dokumentkennungen auf tatsächliche Pfade (oder URLs) abbildet. Ein Plugin muss damit umgehen können, dass eine Dokumentkennung in dieser Abbildung keinen Eintrag hat.

1. Ein einzelner Nutzungszyklus eines Plugins besteht aus den Aufrufen des Befehls **export**. Dieser Aufruf muss das Plugin in einem Zustand hinterlassen, in dem ein weiterer Nutzungszyklus ohne Probleme ablaufen kann.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

reference
: Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

id
: Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

label
: Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
: Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
    : Der Wert ist eine Zeichenkette mit der Kennung des
Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
optional.

    **label**
    : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

    **items**
    : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
in der Reihenfolge, in der sie angezeigt werden sollen. Diese
Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
```

```
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_export_html

Stichwörter: toc_export_html, HTML, doctools, export, serialization, table of contents, toc

Name

doctools::toc::export::html – HTML-Export-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::toc::export::html ?**0.2**?
package require doctools::text
package require doctools::html
package require doctools::html::cssdefaults
export *serial configuration*

Beschreibung

Dieses Paket implementiert das Export-Plugin von doctools für Inhaltsverzeichnisse, zur

Erzeugung von HTML-Markup.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::export**, den Export-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::export** und die Export-Manager-Objekte, die es bereitstellt.

API

Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Inhaltsverzeichnisses entgegen, wie im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt und in *serial* enthalten, dazu die *configuration*, ein Dictionary, und erzeugt HTML-Markup, das das Inhaltsverzeichnis kodiert. Die erzeugte Zeichenkette wird als Ergebnis des Befehls zurückgegeben.

Konfiguration

Das HTML-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

string *user*
: Diese Standard-Konfigurationsvariable enthält den Namen des Benutzers, der den Prozess ausführt, welcher das Export-Plugin aufgerufen hat. Das Plugin schreibt diese Angabe in den Herkunftskommentar am Anfang des erzeugten Dokuments.

string *file*
: Diese Standard-Konfigurationsvariable enthält den Namen der Datei, aus der das Inhaltsverzeichnis stammt. Diese Variable kann ungesetzt sein oder die leere Zeichenkette enthalten. Das Plugin schreibt diese Angabe, wenn sie definiert ist, das heißt gesetzt und nicht die leere Zeichenkette, in den Herkunftskommentar am Anfang des erzeugten Dokuments.

dictionary *map*
: Diese Standard-Konfigurationsvariable enthält ein Dictionary, das die (symbolischen) Dokumentkennungen in Verweisen auf die tatsächlichen Dateinamen und/oder URLs abbildet, die in der Ausgabe verwendet werden.

  Dokumentkennungen ohne Abbildung bleiben unverändert.

boolean *newlines*
: Ist dieses Flag gesetzt, bricht das Plugin den erzeugten HTML-Code auf mehrere Zeilen um, mit jedem Markup-Befehl in einer eigenen Zeile.

  Ist dieses Flag nicht gesetzt (die Vorgabe), wird das ganze Dokument in eine einzige Zeile geschrieben, mit möglichst wenig Abstand zwischen allen Elementen.

boolean *indented*
: Ist dieses Flag gesetzt, rückt das Plugin die Markup-Befehle entsprechend dem Aufbau der Indizes ein. Damit das wirkt, ist auch **newlines** gesetzt.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird die Ausgabe gemäß dem Wert von **newlines** formatiert, und es wird nicht eingerückt.

string *meta*
Diese Variable soll ein HTML-Fragment aufnehmen (Vorgabe: leer). Das enthaltene Fragment wird in die erzeugte Ausgabe im Abschnitt des Dokuments eingefügt, unmittelbar nach dem Tag.

string *header*
Diese Variable soll ein HTML-Fragment aufnehmen (Vorgabe: leer). Das enthaltene Fragment wird in die erzeugte Ausgabe unmittelbar nach dem Titel-Tag im Hauptteil des Dokuments eingefügt, in der class.header 'ision.

string *footer*
Diese Variable soll ein HTML-Fragment aufnehmen (Vorgabe: leer). Das enthaltene Fragment wird in die erzeugte Ausgabe unmittelbar vor dem Tag eingefügt, in der class.footer 'ision.

dictionary *rid*
Der Wert dieser Variablen (Vorgabe: leer) bildet Verweise auf die Bezeichner ab, die als ihre Ankernamen dienen. Jeder Verweis **FOO**, der nicht im Dictionary steht, verwendet **REF-FOO** als Anker, also sich selbst mit vorangestellter Zeichenkette **REF-**.

string *sepline*
Der Wert dieser Variablen ist die Zeichenkette für die Trennkommentare, die in die Ausgabe eingefügt werden, wenn die Ausgabe auf mehrere Zeilen umgebrochen und/oder eingerückt wird. Die Vorgabe besteht aus 60 Bindestrichen.

string *class.main*
Diese Variable enthält den Klassennamen für die 'ivision des Hauptbereichs im erzeugten Dokument. Die Vorgabe ist **doctools**.

string *class.header*
Diese Variable enthält den Klassennamen für die 'ision des Kopfbereichs im erzeugten Dokument. Die Vorgabe ist **toc-header**. Dieser Abschnitt enthält den Dokumenttitel, den vom Benutzer angegebenen **header**, sofern vorhanden, und eine sichtbare Trennlinie.

string *class.title*
Diese Variable enthält den Klassennamen für das Tag, das den Dokumenttitel umschließt. Die Vorgabe ist **toc-title**.

string *class.navsep*
Diese Variable enthält den Klassennamen für die Trenner in den Kopf- und Fußabschnitten des erzeugten Dokuments. Die Vorgabe ist **toc-navsep**.

string *class.contents*
Diese Variable enthält den Klassennamen für das XXXXX, das im erzeugten Dokument die Schlüsselwörter und ihre Verweise aufnimmt. Die Vorgabe ist **toc-contents**.

string *class.ref*
Diese Variable enthält den Klassennamen für die Tabellenelemente, die Verweise auf andere Dokumente sind. Die Vorgabe ist **toc-ref**.

string *class.div*
Diese Variable enthält den Klassennamen für die Tabellenelemente, die Abschnitte sind. Die Vorgabe ist **toc-div**.

string *class.footer*

Diese Variable enthält den Klassennamen für die 'ision des Fußbereichs im erzeugten Dokument. Die Vorgabe ist **toc-footer**. Dieser Abschnitt enthält eine im Browser sichtbare Trennlinie und den vom Benutzer angegebenen **footer**, sofern vorhanden.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariable **format** und ihren Wert.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
    : Der Wert ist eine Zeichenkette mit der Kennung des
Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
optional.

    **label**
    : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

    **items**
    : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
in der Reihenfolge, in der sie angezeigt werden sollen. Diese
Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
sich die rekursive Definition der Struktur: Abschnitte enthalten
dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textformatierer-Plugin

Copyright
```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_export_json
Stichwörter: toc_export_json, JSON, doctools, export, serialization, table of contents, toc

Name
doctools::toc::export::json – JSON-Export-Plugin

Übersicht
package require Tcl 8.5 9
package require doctools::toc::export::json ?**0.2**?
package require textutil::adjust
export *serial configuration*

Beschreibung
Dieses Paket implementiert das Export-Plugin von doctools für Inhaltsverzeichnisse, zur Erzeugung von JSON-Markup.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::export**, den Export-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::export** und die Export-Manager-Objekte, die es bereitstellt.

API
Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Inhaltsverzeichnisses entgegen, wie im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt und in *serial* enthalten, dazu die *configuration*, ein Dictionary, und erzeugt JSON-Markup, das das Inhaltsverzeichnis kodiert. Die erzeugte Zeichenkette wird als Ergebnis des Befehls zurückgegeben.

JSON-Notation von Inhaltsverzeichnissen
Das für Inhaltsverzeichnisse verwendete JSON-Format ist eine direkte Abbildung des Serialisierungsformat für Inhaltsverzeichnisse, wobei Tcl-Dictionaries JSON-Objekten und Tcl-Listen JSON-Arrays entsprechen. Zum Beispiel ist die Tcl-Serialisierung

```
doctools::toc {
    items {
        {reference {
        desc {DocTools - Tables of Contents}
         id introduction.man
         label doctools::toc::introduction
    }}
    {division {
         id processing.man
         items {
             {reference {
             desc {doctoc serialization utilities}
             id structure.man
             label doctools::toc::structure
         }}
         {reference {
             desc {Parsing text in doctoc format}
             id parse.man
             label doctools::toc::parse
         }}
         }
             label Processing
        }}
    }
    label {Table of Contents}
    title TOC
}
```

entspricht der JSON-Zeichenkette

```
{
    "doctools::toc" : {
        "items" : [{
            "reference" : {
                "desc"  : "DocTools - Tables of Contents",
                "id"    : "introduction.man",
                "label" : "doctools::toc::introduction"
            }
        },{
            "division" : {
                "id"    : "processing.man",
                "items" : [{
                    "reference" : {
                        "desc"  : "doctoc serialization utilities",
                        "id"    : "structure.man",
                        "label" : "doctools::toc::structure"
                    }
```

```
                },{
                    "reference" : {
                        "desc"  : "Parsing text in doctoc format",
                        "id"    : "parse.man",
                        "label" : "doctools::toc::parse"
                    }
                }],
                "label" : "Processing"
            }
        }],
        "label" : "Table of Contents",
        "title" : "TOC"
    }
}
```

Konfiguration

Das JSON-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

boolean *indented*

Ist dieses Flag gesetzt, bricht das Plugin den erzeugten JSON-Code auf mehrere Zeilen um und rückt ihn entsprechend seinem inneren Aufbau ein, mit jedem Schlüssel eines Dictionarys in einer eigenen Zeile.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird das ganze JSON-Objekt in eine einzige Zeile geschrieben, mit möglichst wenig Abstand zwischen allen Elementen.

boolean *aligned*

Ist dieses Flag gesetzt, sorgt der Generator dafür, dass die Werte der Schlüssel in einem Dictionary senkrecht zueinander ausgerichtet sind, mit dem Effekt einer Tabelle. Damit das wirkt, ist auch **indented** gesetzt.

Ist dieses Flag nicht gesetzt (die Vorgabe), wird die Ausgabe gemäß dem Wert von **indented** formatiert, ohne die Werte der Dictionary-Schlüssel auszurichten.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **user**, **format**, **file** und **map** sowie ihre Werte.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser

Wert enthält den Inhalt des Inhaltsverzeichnisses.

2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

   desc
   Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
    : Der Wert ist eine Zeichenkette mit der Kennung des
Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
```

```
    optional.

        **label**
        : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
    die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
    ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
    der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

        **items**
        : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
    in der Reihenfolge, in der sie angezeigt werden sollen. Diese
    Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_export_nroff

Stichwörter: toc_export_nroff, doctools, export, nroff, serialization, table of contents, toc

Name

doctools::toc::export::nroff – nroff-Export-Plugin

Übersicht

package require Tcl 8.5 9
package require doctools::toc::export::nroff ?**0.3**?
package require doctools::text
package require doctools::nroff::man_macros
export *serial configuration*

Beschreibung

Dieses Paket implementiert das Export-Plugin von doctools für Inhaltsverzeichnisse, zur Erzeugung von nroff-Markup.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::export**, den Export-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::export** und die Export-Manager-Objekte, die es bereitstellt.

API

Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Inhaltsverzeichnisses entgegen, wie im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt und in *serial* enthalten, dazu die *configuration*, ein Dictionary, und erzeugt nroff-Markup, das das Inhaltsverzeichnis kodiert. Die erzeugte Zeichenkette wird als Ergebnis des Befehls zurückgegeben.

Konfiguration

Das nroff-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

string *user*
: Diese Standard-Konfigurationsvariable enthält den Namen des Benutzers, der den Prozess ausführt, welcher das Export-Plugin aufgerufen hat. Das Plugin schreibt diese Angabe in den Herkunftskommentar am Anfang des erzeugten Dokuments.

string *file*
: Diese Standard-Konfigurationsvariable enthält den Namen der Datei, aus der das Inhaltsverzeichnis stammt. Diese Variable kann ungesetzt sein oder die leere Zeichenkette enthalten. Das Plugin schreibt diese Angabe, wenn sie definiert ist, das heißt gesetzt und nicht die leere Zeichenkette, in den Herkunftskommentar am Anfang des erzeugten Dokuments.

boolean *inline*
: Ist dieses Flag gesetzt (Vorgabe), schreibt das Plugin die Definitionen des man-Makrosatzes direkt in die Ausgabe.

  Ist dieses Flag nicht gesetzt, schreibt das Plugin einen Verweis auf die Definitionen des

man-Makrosatzes in die Ausgabe, nicht aber die Makrodefinitionen selbst.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **format** und **map** sowie ihre Werte.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

   desc

Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
    **division**
    : Dieser Punkt beschreibt eine Gruppe von Einträgen im
    Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
    Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
    Bezeichnung für die Gruppe, einer optionalen Kennung eines
    Dokuments für die ganze Gruppe und der Liste der Einträge in der
    Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

        **id**
        : Der Wert ist eine Zeichenkette mit der Kennung des
    Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
    optional.

        **label**
        : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
    die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
    ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
    der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

        **items**
        : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
    in der Reihenfolge, in der sie angezeigt werden sollen. Diese
    Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textformatierer-Plugin

Copyright
```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_export_text
Stichwörter: toc_export_text, doctools, export, plain text, serialization, table of contents, toc

Name
doctools::toc::export::text – Klartext-Export-Plugin

Übersicht
package require Tcl 8.5 9
package require doctools::toc::export::text ?**0.2**?
package require doctools::text
export *serial configuration*

Beschreibung
Dieses Paket implementiert das Export-Plugin von doctools für Inhaltsverzeichnisse, zur Erzeugung von Klartext.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::export**, den Export-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::export** und die Export-Manager-Objekte, die es bereitstellt.

API
Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Export-Plugin-API, Version 2.

export *serial configuration*
Dieser Befehl nimmt die kanonische Serialisierung eines Inhaltsverzeichnisses entgegen, wie im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt und in *serial* enthalten, dazu die *configuration*, ein Dictionary, und erzeugt Klartext, der das Inhaltsverzeichnis kodiert. Die erzeugte Zeichenkette wird als Ergebnis des Befehls zurückgegeben.

Konfiguration
Das Text-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

dictionary *map*
Diese Standard-Konfigurationsvariable enthält ein Dictionary, das die (symbolischen)

Dokumentkennungen in Verweisen auf die tatsächlichen Dateinamen und/oder URLs abbildet, die in der Ausgabe verwendet werden.

Dokumentkennungen ohne Abbildung bleiben unverändert.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **user**, **file** und **format** sowie ihre Werte.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge

(Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
    : Der Wert ist eine Zeichenkette mit der Kennung des
Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
optional.

    **label**
    : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

    **items**
    : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
in der Reihenfolge, in der sie angezeigt werden sollen. Diese
Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
sich die rekursive Definition der Struktur: Abschnitte enthalten
dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textformatierer-Plugin

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_export_wiki
Stichwörter: toc_export_wiki, doctools, export, serialization, table of contents, toc, wiki

Name
doctools::toc::export::wiki – Wiki-Export-Plugin

Übersicht
package require Tcl 8.5 9
package require doctools::toc::export::wiki ?**0.2**?
package require doctools::text
export *serial configuration*

Beschreibung
Dieses Paket implementiert das Export-Plugin von doctools für Inhaltsverzeichnisse, zur Erzeugung von Wiki-Markup.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::export**, den Export-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::export** und die Export-Manager-Objekte, die es bereitstellt.

API
Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Export-Plugin-API, Version 2.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung eines Inhaltsverzeichnisses entgegen, wie im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt und in *serial* enthalten, dazu die *configuration*, ein Dictionary, und erzeugt Wiki-Markup, das das Inhaltsverzeichnis kodiert. Die erzeugte Zeichenkette wird als Ergebnis des Befehls zurückgegeben.

Wiki-Markup
Die grundlegende Syntax des von diesem Plugin erzeugten Wiki-Markups ist unter http://wiki.tcl.tk/14 beschrieben.

Das Plugin geht über das klassische Markup hinaus und erzeugt passende Überschriften und Einrückungen.

Konfiguration

Das Wiki-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten so, wie sie es vorgeben.

dictionary *map*
: Diese Standard-Konfigurationsvariable enthält ein Dictionary, das die (symbolischen) Dokumentkennungen in Verweisen auf die tatsächlichen Dateinamen und/oder URLs abbildet, die in der Ausgabe verwendet werden.

  Dokumentkennungen ohne Abbildung bleiben unverändert.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **user**, **file** und **format** sowie ihre Werte.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   : Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   : Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   : Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

     Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

     Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

     reference
     : Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren

Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

id
Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

label
Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
    **division**
    : Dieser Punkt beschreibt eine Gruppe von Einträgen im
    Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
    Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
    Bezeichnung für die Gruppe, einer optionalen Kennung eines
    Dokuments für die ganze Gruppe und der Liste der Einträge in der
    Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

        **id**
        : Der Wert ist eine Zeichenkette mit der Kennung des
    Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
    optional.

        **label**
        : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
    die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
    ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
    der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

        **items**
        : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
    in der Reihenfolge, in der sie angezeigt werden sollen. Diese
    Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_import

Stichwörter: toc_import, conversion, doctoc, documentation, import, json, manpage, markup, parsing, plugin, reference, table, table of contents, url

Name

doctools::toc::import – Importieren von Inhaltsverzeichnissen

Übersicht

package require doctools::toc::import ?**0.2.2**?
package require Tcl 8.5 9
package require struct::map
package require doctools::toc::structure
package require snit
package require pluginmgr
::doctools::toc::import *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **import text** *text* ?*format*?
objectName **import file** *path* ?*format*?
objectName **import object text** *object text* ?*format*?
objectName **import object file** *object path* ?*format*?
objectName **config names**
objectName **config get**
objectName **config set** *name* ?*value*?
objectName **config unset** *pattern...*

objectName **includes**
objectName **include add** *path*
objectName **include remove** *path*
objectName **include clear**
IncludeFile *currentfile path*
import *text configuration*

Beschreibung

Dieses Paket stellt eine Klasse zur Verwaltung der Plugins für den Import von Inhaltsverzeichnissen aus anderen Formaten bereit, d. h. für deren Konvertierung aus beispielsweise *doctoc*, *json* usw.

Dies ist eine der drei öffentlichen Säulen, auf denen die Verwaltung von Inhaltsverzeichnissen beruht. Die beiden anderen Säulen sind

1. *Exportieren von Inhaltsverzeichnissen*, und
2. *Aufnehmen von Inhaltsverzeichnissen*

Zu den Begriffe der Inhaltsverzeichnisse und ihrer Teile siehe den gleichnamigen Abschnitt. Zur Datenstruktur, die das wesentliche Ergebnis der von diesem Paket bereitgestellten Manager-Objekte ist, siehe den Abschnitt Serialisierungsformat für Inhaltsverzeichnisse.

Das Plugin-System der Klasse beruht auf dem Paket **pluginmgr** und sucht Plugins über

1. die Umgebungsvariable **DOCTOOLS_TOC_IMPORT_PLUGINS**,
2. die Umgebungsvariable **DOCTOOLS_TOC_PLUGINS**,
3. die Umgebungsvariable **DOCTOOLS_PLUGINS**,
4. der Pfad "*~/.doctools/toc/import/plugin*"
5. der Pfad "*~/.doctools/toc/plugin*"
6. der Pfad "*~/.doctools/plugin*"
7. der Pfad "*~/.doctools/toc/import/plugins*"
8. der Pfad "*~/.doctools/toc/plugins*"
9. der Pfad "*~/.doctools/plugins*"
10. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\IMPORT\PLUGINS"
11. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\TOC\PLUGINS"
12. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\DOCTOOLS\PLUGINS".

Die letzten drei werden nur verwendet, wenn das Paket auf einem Rechner mit dem Betriebssystem Windows(tm) ausgeführt wird.

Das gesamte System wird mit zwei vordefinierten Import-Plugins ausgeliefert, nämlich

doctoc
: Einzelheiten stehen in *doctoc import plugin*.

json
: Einzelheiten stehen in *json import plugin*.

Wer ein eigenes Import-Plugin für ein Format schreiben will, also ein *Plugin-Entwickler*, muss den Abschnitt Referenz zur Import-Plugin-API v2 lesen und verstehen. Er legt die Interaktion zwischen diesem Paket und seinen Plugins im Einzelnen fest.

Begriffe

1. Ein *Inhaltsverzeichnis* besteht aus einer (möglicherweise leeren) Liste von *Elementen*.
2. Jedes Element in der Liste wird durch seine Bezeichnung gekennzeichnet.
3. Jedes Element ist entweder ein *Verweis* oder ein *Abschnitt*.
4. Jeder Verweis hat ein zugehöriges Dokument, das durch eine symbolische Kennung identifiziert wird, sowie eine Textbeschreibung.
5. Jeder Abschnitt kann ein zugehöriges Dokument haben, das durch eine symbolische Kennung identifiziert wird.
6. Jeder Abschnitt besteht aus einer (möglicherweise leeren) Liste von *Elementen*, und jedes Element folgt den Regeln aus Punkt 2 und den darüber stehenden Punkten.

Einige Anmerkungen

1. Die genannten Regeln spannen einen Baum von Elementen auf, mit Verweisen als Blättern und Abschnitten als inneren Knoten, und jedes Element ist ein Eintrag im ganzen Inhaltsverzeichnis.
2. Die kennzeichnenden Bezeichnungen eines Elements E sind innerhalb seines Abschnitts (oder des Inhaltsverzeichnisses) eindeutig, und die volle Bezeichnung eines Elements E ist die Liste der Bezeichnungen aller Knoten auf dem eindeutigen Pfad von der Wurzel des Baums nach E, einschließlich E.

API

Paketbefehle

::doctools::toc::import *objectName*

Dieser Befehl erzeugt ein neues Import-Manager-Objekt mit einem zugehörigen Tcl-Befehl namens *objectName*. Dieser Befehl *object* ist in den Abschnitten Objektbefehl und Objektmethoden vollständig beschrieben. Der Objektbefehl entsteht im aktuellen Namespace, wenn *objectName* nicht vollqualifiziert ist, sonst im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::doctools::toc::import** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

***objectName* import text *text* ?*format*?**

Diese Methode nimmt den *text* und wandelt ihn mit dem Import-Plugin für das angegebene *format* in die kanonische Serialisierung eines Inhaltsverzeichnisses um. Wird kein Plugin für das Format gefunden, wird ein Fehler ausgelöst. Die durch die Umwandlung erzeugte Serialisierung wird als Ergebnis dieser Methode zurückgegeben.

Wird kein Format angegeben, verwendet die Methode **doctoc**.

Was eine *kanonische* Serialisierung ist, steht im Abschnitt Serialisierungsformat für

Inhaltsverzeichnisse.

Das Plugin muss der Schnittstelle im Abschnitt Referenz zur Import-Plugin-API v2 entsprechen.

***objectName* import file *path* ?*format*?**
Diese Methode ist eine praktische Hülle um die im vorherigen Punkt beschriebene Methode **import text**. Sie liest den Inhalt der angegebenen Datei in den Arbeitsspeicher ein, übergibt das Ergebnis an **import text** und gibt die daraus resultierende Serialisierung als eigenes Ergebnis zurück.

***objectName* import object text *object text* ?*format*?**
Diese Methode ist eine bequeme Hülle um die im vorigen Punkt beschriebene Methode **import text**. Sie erwartet, dass *object* ein Objektbefehl mit einer Methode **deserialize** ist, die die kanonische Serialisierung eines Inhaltsverzeichnisses erwartet. Sie importiert den Text mit **import text** und gibt die entstandene Serialisierung über **deserialize** an das *object* weiter. Diese Methode gibt die leere Zeichenkette als Ergebnis zurück.

***objectName* import object file *object path* ?*format*?**
Diese Methode verhält sich wie **import object text**, mit dem Unterschied, dass sie den zu konvertierenden Text aus der angegebenen Datei liest, anstatt ihn als Argument zu erhalten.

***objectName* config names**
Diese Methode gibt eine Liste zurück, die die Namen aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config get**
Diese Methode gibt ein Dictionary zurück, das die Namen und Werte aller dem Objekt derzeit bekannten Konfigurationsvariablen enthält.

***objectName* config set *name* ?*value*?**
Diese Methode setzt die Konfigurationsvariable *name* auf den angegebenen *value* und gibt den neuen Wert der Variablen zurück.

Wird kein Wert angegeben, gibt sie nur den aktuellen Wert zurück, ohne ihn zu ändern.

Der Benutzer kann die vordefinierten Konfigurationsvariablen **user** und **format** zwar setzen, das bleibt aber ohne Wirkung; diese Werte werden beim Aufruf eines Import-Plugins intern überschrieben.

***objectName* config unset *pattern*...**
Diese Methode löscht alle Konfigurationsvariablen, auf die die angegebenen *pattern*s passen. Wird kein Muster angegeben, löscht sie alle derzeit definierten Konfigurationsvariablen.

***objectName* includes**
Diese Methode gibt eine Liste zurück, die die derzeit festgelegten Pfade enthält, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien verwendet werden sollen. Die Reihenfolge der Pfade in der Liste entspricht der Reihenfolge, in der sie verwendet werden – vom ersten bis zum letzten –, und entspricht zudem der Reihenfolge, in der sie dem Objekt hinzugefügt wurden.

objectName* include add *path
Diese Methode fügt den angegebenen *path* zur Liste der Pfade hinzu, in denen bei der Verarbeitung der Eingabe nach Include-Dateien gesucht wird. Der Pfad wird am Ende der Liste angehängt, sodass er erst nach allen zuvor hinzugefügten Pfaden durchsucht wird. Das

Ergebnis des Befehls ist die leere Zeichenkette.

Die Methode führt keine Aktion aus, wenn der Pfad bereits bekannt ist.

objectName* include remove *path
Diese Methode entfernt den angegebenen *path* aus der Liste der Pfade, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien herangezogen werden. Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Methode führt keine Aktion aus, wenn der Pfad nicht bekannt ist.

***objectName* include clear**
Diese Methode löscht die Liste der Pfade, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien herangezogen werden. Das Ergebnis des Befehls ist die leere Zeichenkette.

Referenz zur Import-Plugin-API v2

Plugins verwalten in diesem Paket die Unterstützung für jedes Eingabeformat jenseits des Serialisierungsformat für Inhaltsverzeichnisse. Hier ist die API festgelegt, über die die von diesem Paket erzeugten Objekte mit ihren Plugins verkehren.

Ein Plugin für dieses Paket muss die nachfolgend aufgeführten Regeln einhalten:

1. Ein Plugin ist ein Paket.
2. Der Name eines Plugin-Pakets hat die Form doctools::toc::import::**FOO**, wobei **FOO** der Name des Formats ist, für das das Plugin die Ausgabe erzeugt. Dieser Name ist auch das Argument der verschiedenen Methoden **import** der Import-Manager-Objekte, um eine Zeichenkette zu erhalten, die ein Inhaltsverzeichnis in diesem Format kodiert.
3. Das Plugin kann davon ausgehen, dass das Paket **doctools::toc::export::plugin** vorhanden ist, was darauf hindeutet, dass es von einem echten Plugin-Manager aufgerufen wurde.
4. Das Plugin kann davon ausgehen, dass ein Befehl **IncludeFile** vorhanden ist, mit der Signatur
   IncludeFile *currentfile path*
   Diesen Befehl muss das Plugin aufrufen, wenn es eine eingebundene Datei verarbeiten muss, sofern das Format einen solchen Begriff hat. Ein Beispiel für ein solches Format ist *doctoc*.

   Das Plugin übergibt die folgenden Argumente
   string *currentfile*
   Der Pfad der Datei, die es gerade verarbeitet. Das kann die leere Zeichenkette sein, wenn keiner bekannt ist.
   string *path*
   Der Pfad der Include-Datei, wie ihn die gerade verarbeitete Include-Anweisung angibt.

```
Das Ergebnis des Befehls ist eine Liste mit fünf Elementen, die
Folgendes enthält:

1. Ein boolesches Flag, das Erfolg (**True**) oder Misserfolg
(**False**) des Vorgangs angibt.
```

```
    2. Im Erfolgsfall den Inhalt der eingebundenen Datei, andernfalls
    die leere Zeichenkette.

    3. Der aufgelöste, d. h. der absolute Pfad der eingebundenen
    Datei, sofern möglich, oder das unveränderte *path*-Argument. Dies
    dient zur Anzeige in einer Fehlermeldung oder als
    *currentfile*-Argument eines weiteren Aufrufs von **IncludeFile**,
    falls diese Datei weitere Dateien enthält.

    4. Im Erfolgsfall eine leere Zeichenkette und im Fehlerfall einen
    Code, der den Grund dafür angibt, und zwar einen der folgenden:

        notfound
        : Die angegebene Datei konnte nicht gefunden werden.

        notread
        : Die angegebene Datei wurde gefunden, konnte jedoch nicht in
    den Arbeitsspeicher geladen werden.


    5. Eine leere Zeichenfolge im Falle eines Erfolgs oder eines
    Fehlers mit dem Code **notfound** sowie eine zusätzliche
    Fehlermeldung, die den Grund für einen Fehler mit dem Code
    **notread** näher erläutert.
```

1. Ein Plugin muss einen Befehl bereitstellen, dessen Signatur wie unten dargestellt lautet.

import *text configuration*

Immer wenn ein Import-Manager von **doctools::toc** Eingabe für ein Inhaltsverzeichnis parsen muss, ruft er diesen Befehl auf.

string *text*

Dieses Argument enthält den Text, der das Inhaltsverzeichnis in dem Format kodiert, für das das Plugin zuständig ist.

dictionary *configuration*

Dieses Argument enthält die beim Parsen anzuwendende Konfiguration, als Dictionary von Variablennamen auf Werte.

Die folgenden Konfigurationsvariablen haben eine vorgegebene Bedeutung, an die sich alle Plugins halten müssen, auch wenn sie diese Angabe nach eigenem Ermessen ignorieren können. Jede weitere Konfigurationsvariable, die ein Plugin erkennt, ist in der Handbuchseite dieses Plugins beschrieben.

user

Diese Variable soll den Namen des Benutzers enthalten, dem der Prozess gehört, der das Plugin aufruft.

format

Diese Variable soll den Namen des Formats enthalten, dessen Plugin aufgerufen wird.

1. Ein einzelner Nutzungszyklus eines Plugins besteht aus den Aufrufen des Befehls **import**. Dieser Aufruf muss das Plugin in einem Zustand hinterlassen, in dem ein weiterer Nutzungszyklus problemlos ausgeführt werden kann.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

   desc

Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
    **division**
    : Dieser Punkt beschreibt eine Gruppe von Einträgen im
    Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
    Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
    Bezeichnung für die Gruppe, einer optionalen Kennung eines
    Dokuments für die ganze Gruppe und der Liste der Einträge in der
    Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

        **id**
        : Der Wert ist eine Zeichenkette mit der Kennung des
    Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
    optional.

        **label**
        : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
    die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
    ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
    der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

        **items**
        : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
    in der Reihenfolge, in der sie angezeigt werden sollen. Diese
    Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_import_json
Stichwörter: toc_import_json, JSON, deserialization, doctools, import, table of contents, toc

Name
doctools::toc::import::json – JSON-Import-Plugin

Übersicht
package require Tcl 8.5 9
package require doctools::toc::import::json ?**0.2.1**?
package require doctools::toc::structure
package require json
import *string configuration*

Beschreibung
Dieses Paket implementiert das Import-Plugin von doctools für Inhaltsverzeichnisse, zum Parsen von JSON-Markup.

Es ist ein internes Paket von doctools, für die übergeordneten Verwaltungspakete, die Inhaltsverzeichnisse bearbeiten, insbesondere **doctools::toc::import**, den Import-Manager.

Der Aufruf aus einem gewöhnlichen Interpreter ist möglich, aber nur mit Verrenkungen, und wird nicht empfohlen. Der vorgesehene Weg führt über das Paket **doctools::toc::import** und die Import-Manager-Objekte, die es bereitstellt.

API
Die von diesem Paket bereitgestellte API erfüllt die Spezifikation der doctoc-Import-Plugin-API, Version 2.

import *string configuration*
: Dieser Befehl nimmt die *string* entgegen und parst sie als JSON-Markup, das ein Inhaltsverzeichnis kodiert, im Kontext der angegebenen *configuration* (ein Dictionary). Das Ergebnis des Befehls ist die kanonische Serialisierung dieses Inhaltsverzeichnisses, in der Form aus dem Abschnitt Serialisierungsformat für Inhaltsverzeichnisse.

JSON-Notation von Inhaltsverzeichnissen
Das für Inhaltsverzeichnisse verwendete JSON-Format ist eine direkte Abbildung des Serialisierungsformat für Inhaltsverzeichnisse, wobei Tcl-Dictionaries JSON-Objekten und Tcl-Listen JSON-Arrays entsprechen. Zum Beispiel ist die Tcl-Serialisierung

```
doctools::toc {
    items {
        {reference {
        desc {DocTools - Tables of Contents}
         id introduction.man
         label doctools::toc::introduction
    }}
    {division {
         id processing.man
         items {
             {reference {
             desc {doctoc serialization utilities}
             id structure.man
             label doctools::toc::structure
         }}
         {reference {
             desc {Parsing text in doctoc format}
             id parse.man
             label doctools::toc::parse
         }}
         }
             label Processing
        }}
    }
    label {Table of Contents}
    title TOC
}
```

entspricht der JSON-Zeichenkette

```
{
    "doctools::toc" : {
        "items" : [{
            "reference" : {
                "desc"  : "DocTools - Tables of Contents",
                "id"    : "introduction.man",
                "label" : "doctools::toc::introduction"
            }
        },{
            "division" : {
                "id"    : "processing.man",
                "items" : [{
                    "reference" : {
                        "desc"  : "doctoc serialization utilities",
                        "id"    : "structure.man",
                        "label" : "doctools::toc::structure"
                    }
```

```
                },{
                    "reference" : {
                        "desc"  : "Parsing text in doctoc format",
                        "id"    : "parse.man",
                        "label" : "doctools::toc::parse"
                    }
                }],
                "label" : "Processing"
            }
        }],
        "label" : "Table of Contents",
        "title" : "TOC"
    }
}
```

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren

Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

id
Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

label
Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
    **division**
    : Dieser Punkt beschreibt eine Gruppe von Einträgen im
    Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
    Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
    Bezeichnung für die Gruppe, einer optionalen Kennung eines
    Dokuments für die ganze Gruppe und der Liste der Einträge in der
    Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

        **id**
        : Der Wert ist eine Zeichenkette mit der Kennung des
    Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
    optional.

        **label**
        : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
    die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
    ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
    der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

        **items**
        : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
    in der Reihenfolge, in der sie angezeigt werden sollen. Diese
    Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textformatierer-Plugin

Copyright


```
Copyright (c) 2009-2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_introduction

Stichwörter: toc_introduction, contents, conversion, formatting, markup, parsing, plugin, semantic markup, table of contents

Name

doctools2toc_introduction – DocTools – Inhaltsverzeichnisse

Beschreibung

doctoc (kurz für *documentation tables of contents*) bezeichnet eine Reihe verwandter, aber verschiedener Dinge, die zusammenwirken, damit sich Inhaltsverzeichnisse für Dokumentation leicht erzeugen und umwandeln lassen.

Dies sind

1. Eine auf Tcl beruhende Sprache für die semantische Auszeichnung eines Inhaltsverzeichnisses. Die Auszeichnung besteht aus Tcl-Befehlen. Anfänger beginnen mit der *doctoc language introduction*. Die formale Spezifikation ist auf zwei Dokumente verteilt, eines zur *doctoc language syntax*, das andere eine *doctoc language command reference*.
2. Eine Reihe von Paketen zur programmgesteuerten Bearbeitung von Inhaltsverzeichnissen im Arbeitsspeicher und zu ihrer Umwandlung zwischen verschiedenen Formaten, lesend und schreibend. Die genannte Auszeichnungssprache ist eines der Formate, die sich sowohl lesen als auch schreiben lassen.
3. Das System zur Umwandlung von Inhaltsverzeichnissen beruht auf einem Plugin-Mechanismus. Dafür gibt es zwei APIs, die die Schnittstelle zwischen den genannten Paketen und den Import- und Export-Plugins beschreiben.

Welche der ausführlicheren Dokumente für den Leser dieser Einleitung relevant sind, hängt von der Rolle im Dokumentationsprozess ab.

1. Ein *Autor* von Dokumentation muss die Auszeichnungssprache selbst verstehen. Wer mit doctoc anfängt, liest zuerst die eher informell geschriebene *doctoc language introduction*. Danach sollte die formale Spezifikation der *doctoc language syntax* verständlich sein. Wer doctoc schon kennt, braucht die *doctoc language command reference* nur noch gelegentlich zur Erinnerung.

   Während ein Dokument geschrieben wird, kann die Anwendung **dtp** es prüfen, und nach der Fertigstellung wandelt sie es auch in das gewählte System visueller Auszeichnung um, sei es *roff, HTML, Klartext, Wiki und so weiter. Die einfachere Anwendung **dtplite** verwendet doctoc intern, wenn sie Verzeichnisse mit Dokumentation bearbeitet, und erzeugt für sie automatisch ein passendes Inhaltsverzeichnis.
2. Ein *Verarbeiter* von Dokumentation in der Auszeichnungssprache *doctoc* muss wissen, welche Werkzeuge es gibt.

   Das wichtigste Werkzeug ist die genannte Anwendung **dtp** aus Tcllib. Die einfachere Anwendung **dtplite** zeigt dem Benutzer doctoc nicht. Auf der untersten Ebene, die beiden Anwendungen gemeinsam ist, liegen die drei Pakete mit den grundlegenden Mitteln für Inhaltsverzeichnisse, also Import aus Textformaten, programmgesteuerte Bearbeitung im Arbeitsspeicher und Export in Textformate. Das sind

   doctoools::toc
   : Programmgesteuerte Bearbeitung von Inhaltsverzeichnissen im Arbeitsspeicher.

   doctoools::toc::import
   : Import von Inhaltsverzeichnissen aus verschiedenen Textformaten. Die Menge der unterstützten Formate lässt sich durch Plugin-Pakete erweitern.

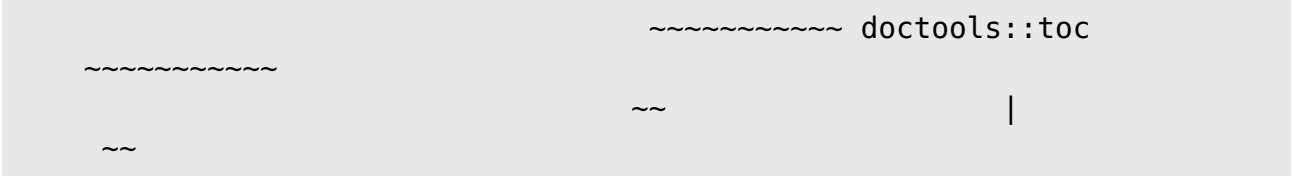
   doctoools::toc::export
   : Export von Inhaltsverzeichnissen in verschiedene Textformate. Die Menge der unterstützten Formate lässt sich durch Plugin-Pakete erweitern.

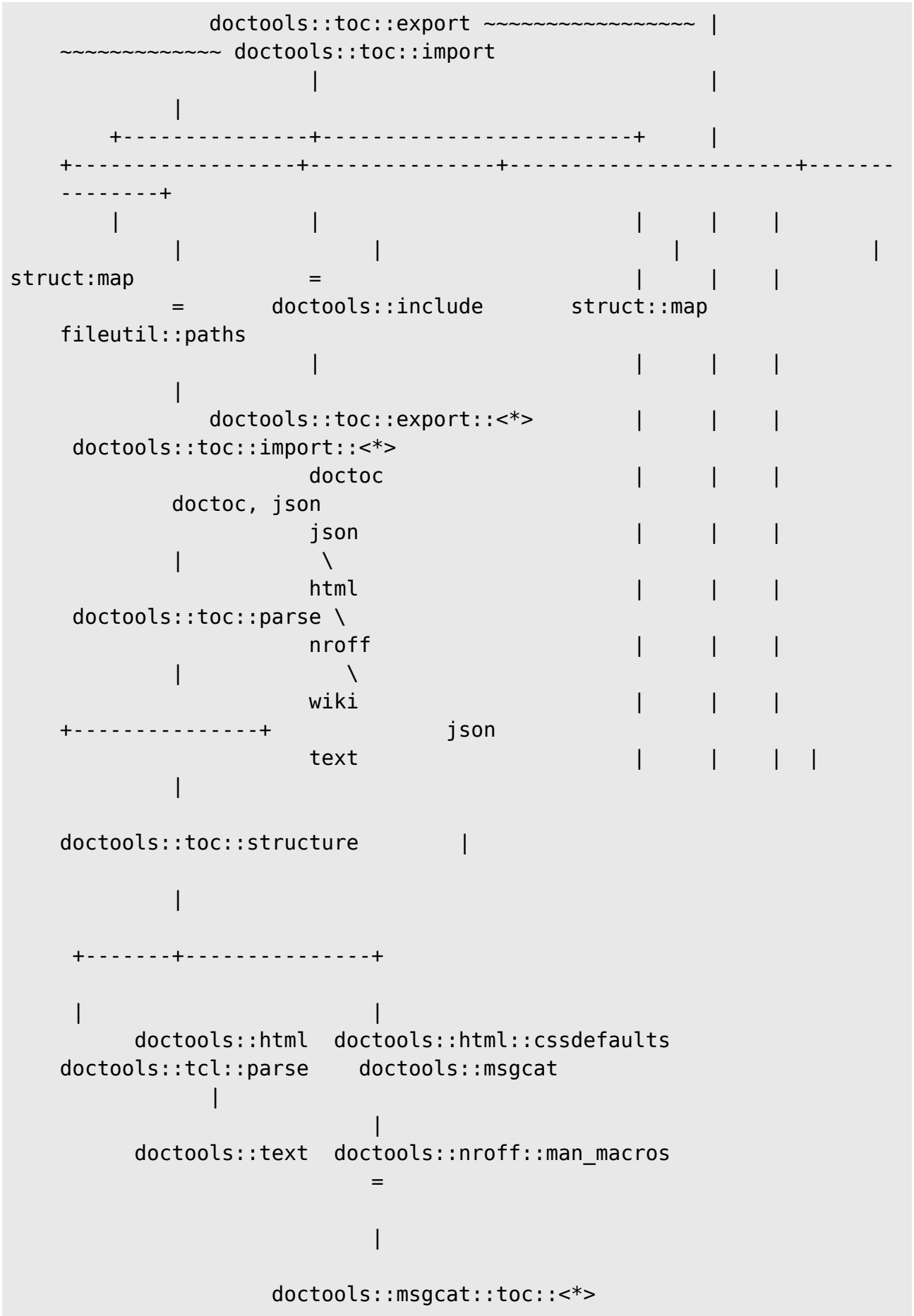
1. Siehe auch den Abschnitt Paketübersicht für einen Überblick über die Abhängigkeiten zwischen diesen und anderen, unterstützenden Paketen.
2. Nicht zuletzt müssen *Plugin-Entwickler* das Zusammenspiel der Import- und Export-Pakete mit ihren Plugins verstehen. Diese APIs sind in der Dokumentation der beiden zuständigen Pakete beschrieben, also
   - **doctoools::toc::import**
   - **doctoools::toc::export**

Verwandte Formate

Das doctoc-Format steht nicht allein, es hat zwei begleitende Formate. Sie heißen *docidx* und *doctools* und sind für die Auszeichnung von *Schlüsselwortindizes* beziehungsweise von allgemeiner Dokumentation bestimmt. Beschrieben sind sie in eigenen Dokumentgruppen, beginnend mit *DocTools - Keyword Indices* beziehungsweise *DocTools - General*.

Paketübersicht




doctools::toc::export ~~~~~~~~~~~~~~~~~ |
~~~~~~~~~~~~ doctools::toc::import
| |
| |
+---------------+-------------------------+ |
+-----------------+---------------+-----------------------+-------
--------+
| | | | | |
| | | | |
struct:map = | | |
= doctools::include struct::map
fileutil::paths
| | | |
| |
doctools::toc::export::<*> | | |
doctools::toc::import::<*>
doctoc | | |
doctoc, json
json | | |
| \
html | | |
doctools::toc::parse \
nroff | | |
| \
wiki | | |
+---------------+ json
text | | | |
|
doctools::toc::structure |
|
+-------+---------------+
| |
doctools::html doctools::html::cssdefaults
doctools::tcl::parse doctools::msgcat
|
|
doctools::text doctools::nroff::man_macros
=
|
doctools::msgcat::toc::<*>
~~~~~~~~~~~~

```
                          c, en, de, fr

                          (fr == en for now)
    ~~      Interoperable objects, without actual package
dependencies
    --      Package dependency, higher requires lower package
    =       Dynamic dependency through plugin system
    <*>     Multiple packages following the given form of naming.
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_msgcat_c

Stichwörter: toc_msgcat_c, C, catalog package, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::toc::c – Nachrichtenkatalog für den doctoc-Parser (C)

Übersicht

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::c ?**0.1**?

Beschreibung

Das Paket **doctools::msgcat::toc::c** ist ein Hilfsmodul und stellt den Nachrichtenkatalog der Sprache C für den doctoc-Parser im System doctools, Version 2, bereit. Es ist damit ein internes Paket, mit dem ein gewöhnlicher Benutzer (Entwickler) nicht in direkten Kontakt kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,

2. **doctools::toc**, oder
3. **doctools::idx**

In der Architektur des Systems liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System bereitstellt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API
Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_msgcat_de
Stichwörter: toc_msgcat_de, DE, catalog package, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name
doctools::msgcat::toc::de – Nachrichtenkatalog für den doctoc-Parser (DE)

Übersicht
package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::de ?**0.1**?

Beschreibung
Das Paket **doctools::msgcat::toc::de** ist ein Hilfsmodul und stellt den Nachrichtenkatalog der Sprache DE (deutsch) für den doctoc-Parser im System doctools, Version 2, bereit. Es ist damit ein internes Paket, mit dem ein gewöhnlicher Benutzer (Entwickler) nicht in direkten Kontakt kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

In der Architektur des Systems liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System bereitstellt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API

Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_msgcat_en

Stichwörter: toc_msgcat_en, EN, catalog package, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::toc::en – Nachrichtenkatalog für den doctoc-Parser (EN)

Übersicht

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::en ?**0.1**?

Beschreibung

Das Paket **doctools::msgcat::toc::en** ist ein Hilfsmodul und stellt den Nachrichtenkatalog der Sprache EN (englisch) für den doctoc-Parser im System doctools, Version 2, bereit. Es ist damit ein internes Paket, mit dem ein gewöhnlicher Benutzer (Entwickler) nicht in direkten Kontakt kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden

Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

In der Architektur des Systems liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System bereitstellt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API

Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_msgcat_fr

Stichwörter: toc_msgcat_fr, FR, catalog package, doctoc, doctools, i18n, internationalization, l10n, localization, message catalog, message package

Name

doctools::msgcat::toc::fr – Nachrichtenkatalog für den doctoc-Parser (FR)

Übersicht

package require Tcl 8.5 9
package require msgcat
package require doctools::msgcat::toc::fr ?**0.1**?

Beschreibung

Das Paket **doctools::msgcat::toc::fr** ist ein Hilfsmodul und stellt den Nachrichtenkatalog der Sprache FR (französisch) für den doctoc-Parser im System doctools, Version 2, bereit. Es ist damit ein internes Paket, mit dem ein gewöhnlicher Benutzer (Entwickler) nicht in direkten Kontakt kommen sollte.

Wenn Sie zu dieser Gruppe gehören, lesen Sie bitte die Dokumentation zu einem der folgenden Pakete:

1. **doctools::doc**,
2. **doctools::toc**, oder
3. **doctools::idx**

In der Architektur des Systems liegt dieses Paket unter dem Paket **doctools::msgcat**, das die allgemeine Verwaltung der Nachrichtenkataloge im System bereitstellt. *Hinweis*: Zwischen dem Manager-Paket und den Katalogpaketen besteht *keine* explizite Abhängigkeit. Der Katalog ist ein Plugin, das dynamisch ausgewählt und geladen wird.

API
Dieses Paket verfügt über keine exportierte API.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


toc_parse
Stichwörter: toc_parse, doctoc, doctools, lexer, parser

Name
doctools::toc::parse – Parsing von Text im doctoc-Format

Übersicht
package require doctools::toc::parse ?**0.2**?
package require Tcl 8.5 9
package require doctools::toc::structure
package require doctools::msgcat
package require doctools::tcl::parse
package require fileutil
package require logger
package require snit
package require struct::list

package require struct::stack
::doctools::toc::parse text *text*
::doctools::toc::parse file *path*
::doctools::toc::parse includes
::doctools::toc::parse include add *path*
::doctools::toc::parse include remove *path*
::doctools::toc::parse include clear
::doctools::toc::parse vars
::doctools::toc::parse var set *name value*
::doctools::toc::parse var unset *name*
::doctools::toc::parse var clear ?*pattern*?

Beschreibung

Dieses Paket stellt Befehle bereit, die in der Auszeichnungssprache *doctoc* geschriebenen Text parsen und in die kanonische Serialisierung des im Text kodierten Inhaltsverzeichnisses umwandeln. Das Format ist im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt.

Hierbei handelt es sich um ein internes Paket von doctools, das von den übergeordneten Paketen zur Verarbeitung von *doctoc*-Dokumenten verwendet wird.

API

::doctools::toc::parse text *text*
: Der Befehl nimmt die in *text* enthaltene Zeichenkette und parst sie unter der Annahme, dass sie ein mit der Auszeichnungssprache *doctoc* geschriebenes Dokument enthält. Erweist sich die Annahme als falsch, wird ein Fehler ausgelöst. Das Format dieser Fehler ist im Abschnitt Parsing-Fehler beschrieben.

  Bei Erfolg gibt der Befehl die kanonische Serialisierung des Inhaltsverzeichnisses zurück, das im Text kodiert war. Das Format ist im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt.

::doctools::toc::parse file *path*
: Entspricht **text**, mit dem Unterschied, dass der zu analysierende Text aus der durch *path* angegebenen Datei gelesen wird.

::doctools::toc::parse includes
: Diese Methode gibt die aktuelle Liste der Suchpfade zurück, die bei der Suche nach Include-Dateien verwendet werden.

::doctools::toc::parse include add *path*
: Diese Methode fügt den *path* zur Liste der Pfade hinzu, die bei der Suche nach einer Include-Datei durchsucht werden. Der Aufruf wird ignoriert, wenn der Pfad schon in der Liste steht. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::toc::parse include remove *path*
: Diese Methode entfernt den *path* aus der Liste der Pfade, die bei der Suche nach einer Include-Datei durchsucht werden. Der Aufruf wird ignoriert, wenn der Pfad nicht in der Liste steht. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::toc::parse include clear
: Diese Methode leert die Liste der Suchpfade für Include-Dateien.

::doctools::toc::parse vars
Diese Methode gibt ein Dictionary zurück, das die aktuellen vordefinierten Variablen enthält, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt.

::doctools::toc::parse var set *name value*
Diese Methode nimmt die Variable *name* in die Menge der vordefinierten Variablen auf, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt, und gibt ihr den angegebenen *value*. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::toc::parse var unset *name*
Diese Methode entfernt die Variable *name* aus der Menge der vordefinierten Variablen, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

::doctools::toc::parse var clear ?*pattern*?
Diese Methode entfernt alle Variablen, auf die das *pattern* passt, aus der Menge der vordefinierten Variablen, die der Auszeichnungsbefehl **vset** während der Verarbeitung kennt. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

Der Abgleich erfolgt mit **string match**, und das Vorgabemuster, wenn keines angegeben ist, ist *.

Parsing-Fehler

Das Format der Parsing-Fehlermeldungen, die bei Verstößen gegen die Auszeichnungssyntax von *doctoc* ausgelöst werden, ist für Menschen lesbar und nicht zur Verarbeitung durch Maschinen bestimmt. Deshalb ist es nicht dokumentiert.

Allerdings ist der an die Meldung angehängte errorCode maschinenlesbar und hat das folgende Format:

1. Der Fehlercode ist eine Liste, wobei jedes Element einen einzelnen in der Eingabe gefundenen Fehler beschreibt. Die Liste enthält mindestens ein Element, möglicherweise auch mehrere.
2. Jedes Fehlerelement ist eine Liste, die sechs Zeichenfolgen enthält, welche einen Fehler detailliert beschreiben. Die Zeichenfolgen lauten
   1. Der Pfad der Datei, in der der Fehler aufgetreten ist. Dieser kann leer sein.
   2. Der Bereich des Tokens, an dem der Fehler gefunden wurde. Dieser Bereich ist eine Liste aus zwei Elementen mit dem Offset des ersten und des letzten Zeichens im Bereich, gezählt vom Anfang der Eingabe (der Datei). Offsets zählen ab null.
   3. Die Zeile, in der das erste Zeichen nach dem Fehler steht. Zeilen zählen ab eins.
   4. Die Spalte, in der das erste Zeichen nach dem Fehler steht. Spalten zählen ab null.
   5. Der Nachrichtencode des Fehlers. Dieser Wert kann als Argument von **msgcat::mc** dienen, um eine lokalisierte Fehlermeldung zu erhalten, sofern die Anwendung **doctools::msgcat::init** passend aufgerufen hat, um die nötigen Nachrichtenkataloge zu initialisieren (siehe Paket **doctools::msgcat**).
   6. Eine Auflistung der Details zum Fehler, wie beispielsweise der betroffene Markup-Befehl. Im Falle des Fehlercodes **doctoc/include/syntax** entspricht dieser Wert der Menge der in der eingebundenen Datei gefundenen Fehler, wobei das hier beschriebene Format verwendet wird.

[doctoc]-Notation von Inhaltsverzeichnissen

Das doctoc-Format für Inhaltsverzeichnisse, auch *doctoc markup language* genannt, ist zu umfangreich für einen einzigen Abschnitt. Der interessierte Leser beginnt mit dem Dokument

1. *doctoc language introduction*

und geht von dort weiter zu den formalen Spezifikationen, also den Dokumenten

1. *doctoc language syntax* und
2. *doctoc language command reference*,

um die Sprache gründlich zu verstehen.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem Eintrag gehört.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese

Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
    **division**
    : Dieser Punkt beschreibt eine Gruppe von Einträgen im
    Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
    Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
    Bezeichnung für die Gruppe, einer optionalen Kennung eines
    Dokuments für die ganze Gruppe und der Liste der Einträge in der
    Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

        **id**
        : Der Wert ist eine Zeichenkette mit der Kennung des
    Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
    optional.

        **label**
        : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
    die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
    ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
    der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

        **items**
        : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
    in der Reihenfolge, in der sie angezeigt werden sollen. Diese
    Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
    für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
    sich die rekursive Definition der Struktur: Abschnitte enthalten
    dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
    einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dokumentationswerkzeuge

Copyright
```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

toc_structure
Stichwörter: toc_structure, deserialization, doctoc, doctools, serialization

Name
doctools::toc::structure – Doctoc-Serialisierungswerkzeuge

Übersicht
package require doctools::toc::structure ?**0.2**?
package require Tcl 8.5 9
package require logger
package require snit
::doctools::toc::structure verify *serial* ?*canonvar*?
::doctools::toc::structure verify-as-canonical *serial*
::doctools::toc::structure canonicalize *serial*
::doctools::toc::structure print *serial*
::doctools::toc::structure merge *seriala serialb*

Beschreibung
Dieses Paket stellt Befehle für die Arbeit mit den Serialisierungen von Inhaltsverzeichnissen bereit, wie sie das System doctools v2 verwaltet und wie sie im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt sind.

Es ist ein internes Paket von doctools, für die übergeordneten Pakete, die Inhaltsverzeichnisse bearbeiten und sie in verschiedene andere Formate umwandeln und daraus zurück, etwa Dokumente in der Auszeichnung *doctoc*.

API
::doctools::toc::structure verify *serial* ?*canonvar*?

Dieser Befehl prüft, ob der Inhalt von *serial* eine gültige *reguläre* Serialisierung eines Inhaltsverzeichnisses ist, und löst einen Fehler aus, wenn das nicht zutrifft. Das Ergebnis des Befehls ist die leere Zeichenkette.

Ist das Argument *canonvar* angegeben, gilt es als Name einer Variablen im aufrufenden Kontext. In diese Variable wird genau dann geschrieben, wenn *serial* eine gültige reguläre

Serialisierung ist. Ihr Wert ist ein boolescher Wert, **True**, wenn die Serialisierung nicht nur gültig, sondern auch *kanonisch* ist. **False** wird bei einer gültigen, aber nicht kanonischen Serialisierung geschrieben.

Reguläre und kanonische Serialisierungen sind im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt.

::doctools::toc::structure verify-as-canonical *serial*

Dieser Befehl prüft, ob der Inhalt von *serial* eine gültige *kanonische* Serialisierung eines Inhaltsverzeichnisses ist, und löst einen Fehler aus, wenn das nicht zutrifft. Das Ergebnis des Befehls ist die leere Zeichenkette.

Kanonische Serialisierungen sind im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt.

::doctools::toc::structure canonicalize *serial*

Dieser Befehl setzt voraus, dass der Inhalt von *serial* eine gültige *reguläre* Serialisierung eines Inhaltsverzeichnisses ist, und löst einen Fehler aus, wenn das nicht zutrifft.

Danach wandelt er die Eingabe in die *kanonische* Serialisierung des enthaltenen Inhaltsverzeichnisses um und gibt sie als Ergebnis zurück. Ist die Eingabe schon kanonisch, wird sie unverändert zurückgegeben.

Reguläre und kanonische Serialisierungen sind im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt.

::doctools::toc::structure print *serial*

Dieser Befehl setzt voraus, dass das Argument *serial* eine gültige reguläre Serialisierung eines Inhaltsverzeichnisses enthält, und gibt eine Zeichenkette zurück, die dieses Inhaltsverzeichnis in einer für Menschen lesbaren Form enthält.

Das genaue Format dieser Form ist nicht festgelegt und eignet sich nicht zum Parsen oder für andere maschinelle Verarbeitung.

Reguläre Serialisierungen sind im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt.

::doctools::toc::structure merge *seriala serialb*

Dieser Befehl nimmt die regulären Serialisierungen zweier Inhaltsverzeichnisse entgegen und bildet daraus ihre Vereinigung. Das Ergebnis des Befehls ist die kanonische Serialisierung dieses vereinigten Inhaltsverzeichnisses.

Titel und Bezeichnung des Ergebnisses stammen aus dem in *serialb* enthaltenen Inhaltsverzeichnis.

Das ganze Inhaltsverzeichnis und seine Abschnitte werden rekursiv auf dieselbe Weise zusammengeführt:

1. Alle Verweise, die in beiden Abschnitten vorkommen (an ihrer Bezeichnung erkannt), werden vereinigt; Dokumentkennungen und Beschreibungen stammen aus dem zweiten Inhaltsverzeichnis.
2. Alle Abschnitte, die in beiden Abschnitten vorkommen (an ihrer Bezeichnung erkannt),

werden vereinigt, mit der optionalen Dokumentkennung aus dem zweiten Inhaltsverzeichnis, falls dort eine steht, sonst aus dem ersten. Die Elemente im Abschnitt werden rekursiv mit demselben Algorithmus zusammengeführt, der in dieser Liste beschrieben ist.
3. Typkonflikte zwischen Elementen, also zwei Elemente mit derselben Bezeichnung, aber unterschiedlichem Typ, führen zu einem Fehler bei der Zusammenführung.
4. Alle Elemente, die im zweiten Abschnitt, aber nicht im ersten vorkommen, werden ans Ende der Elementliste im Ergebnis der Zusammenführung angehängt.

Reguläre und kanonische Serialisierungen sind im Abschnitt Serialisierungsformat für Inhaltsverzeichnisse festgelegt.

Serialisierungsformat für Inhaltsverzeichnisse

Hier ist das Format festgelegt, in dem die Pakete von doctools v2 Inhaltsverzeichnisse als unveränderliche Werte für den Transport, den Vergleich und Ähnliches serialisieren.

Es wird zwischen *regulären* und *kanonischen* Serialisierungen unterschieden. Ein Inhaltsverzeichnis kann mehr als eine reguläre Serialisierung haben, *kanonisch* ist aber genau eine.

regular serialization

1. Die Serialisierung eines beliebigen Inhaltsverzeichnisses ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält genau einen Schlüssel, **doctools::toc**, und dessen Wert. Dieser Wert enthält den Inhalt des Inhaltsverzeichnisses.
2. Der Inhalt des Inhaltsverzeichnisses ist ein Tcl-Dictionary mit dem Titel des Inhaltsverzeichnisses, einer Bezeichnung und seinen Elementen. Die maßgeblichen Schlüssel und ihre Werte sind

   title
   Der Wert ist eine Zeichenkette mit dem Titel des Inhaltsverzeichnisses.

   label
   Der Wert ist eine Zeichenkette mit einer Bezeichnung für das Inhaltsverzeichnis.

   items
   Der Wert ist eine Tcl-Liste mit den Elementen des Inhaltsverzeichnisses, in der Reihenfolge, in der sie angezeigt werden sollen.

   Jedes Element ist eine Tcl-Liste mit dem Typ des Eintrags und seiner Beschreibung, in dieser Reihenfolge. Anders gesagt ist es ein Tcl-Dictionary mit genau einem Schlüssel, dem Eintragstyp, abgebildet auf die Beschreibung des Eintrags.

   Die beiden zulässigen Eintragstypen und ihre Beschreibungen sind

   reference
   Dieser Punkt beschreibt einen einzelnen Eintrag im Inhaltsverzeichnis, der auf ein einzelnes Dokument verweist. Sein Wert ist dazu ein Tcl-Dictionary mit einer Kennung für das referenzierte Dokument, einer Bezeichnung und einer längeren Textbeschreibung, die dem Eintrag zugeordnet werden kann. Die maßgeblichen Schlüssel und ihre Werte sind

   id
   Der Wert ist eine Zeichenkette mit der Kennung des Dokuments, das zu dem

Eintrag gehört.

label
Der Wert ist eine Zeichenkette mit einer Bezeichnung für diesen Eintrag. Diese Zeichenkette kennzeichnet den Eintrag ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

desc
Der Wert ist eine Zeichenkette mit einer längeren Beschreibung dieses Eintrags.

```
**division**
: Dieser Punkt beschreibt eine Gruppe von Einträgen im
Inhaltsverzeichnis und erzeugt damit eine Hierarchie von
Einträgen. Sein Wert ist dazu ein Tcl-Dictionary mit einer
Bezeichnung für die Gruppe, einer optionalen Kennung eines
Dokuments für die ganze Gruppe und der Liste der Einträge in der
Gruppe. Die maßgeblichen Schlüssel und ihre Werte sind

    **id**
    : Der Wert ist eine Zeichenkette mit der Kennung des
Dokuments, das zur ganzen Gruppe gehört. Dieser Schlüssel ist
optional.

    **label**
    : Der Wert ist eine Zeichenkette mit einer Bezeichnung für
die Gruppe. Diese Zeichenkette kennzeichnet den Eintrag
ebenfalls, und keine zwei Einträge (Verweise und Abschnitte) in
der enthaltenden Liste dürfen dieselbe Bezeichnung haben.

    **items**
    : Der Wert ist eine Tcl-Liste mit den Elementen der Gruppe,
in der Reihenfolge, in der sie angezeigt werden sollen. Diese
Liste hat denselben Aufbau wie der Wert des Schlüssels **items**
für das ganze Inhaltsverzeichnis, siehe oben. Damit schließt
sich die rekursive Definition der Struktur: Abschnitte enthalten
dieselben Arten von Elementen wie das ganze Inhaltsverzeichnis,
einschließlich weiterer Abschnitte.
```

canonical serialization
Die kanonische Serialisierung eines Inhaltsverzeichnisses hat das im vorigen Punkt festgelegte Format und erfüllt zusätzlich die folgenden Bedingungen. Dadurch ist sie unter allen möglichen Serialisierungen dieses Inhaltsverzeichnisses eindeutig.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


dtplite

pkg_dtplite

Stichwörter: pkg_dtplite, HTML, TMML, conversion, docidx, doctoc, doctools, manpage, markup, nroff

Name

pkg_dtplite – Leichtgewichtiger DocTools-Markup-Prozessor

Übersicht

package require dtplite ?**1.3.2**?
dtplite::print-via *cmd*
dtplite::do *arguments*

Beschreibung

Das in diesem Dokument beschriebene Paket **dtplite** ist die Grundlage der Anwendung **dtplite**. Es ist eine schlanke Hülle um die verschiedenen Pakete von **doctools**.

Befehle

dtplite::print-via *cmd*

Leitet die Ausgaben des Pakets an das angegebene *cmd* um.

Das Ergebnis des Befehls ist die leere Zeichenkette.

dtplite::do *arguments*

Der Hauptbefehl nimmt eine *einzelne Liste* von *arguments* entgegen, verarbeitet sie und führt die angegebene Aktion aus.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Syntax innerhalb der Liste *arguments* ist im Abschnitt *COMMAND LINE* der Dokumentation der Anwendung **dtplite** erläutert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *doctools* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright


```
Copyright (c) 2004-2013 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


fileutil

fileutil

Stichwörter: fileutil, cat, file utilities, grep, temp file, test, touch, type

Name

fileutil – Prozeduren zur Implementierung einiger Dateidienste

Übersicht

package require Tcl 8.5 9
package require fileutil ?**1.16.4**?
::fileutil::lexnormalize *path*
::fileutil::fullnormalize *path*
::fileutil::test *path codes* ?*msgvar*? ?*label*?
::fileutil::cat (?*options*? *file*)...
::fileutil::writeFile ?*options*? *file data*
::fileutil::appendToFile ?*options*? *file data*
::fileutil::insertIntoFile ?*options*? *file at data*
::fileutil::removeFromFile ?*options*? *file at n*
::fileutil::replaceInFile ?*options*? *file at n data*
::fileutil::updateInPlace ?*options*? *file cmd*
::fileutil::fileType *filename*
::fileutil::find ?*basedir ?filtercmd*?
::fileutil::findByPattern *basedir* ?**-regexp|-glob**? ?**--**? *patterns*
::fileutil::foreachLine *var filename cmd*
::fileutil::grep *pattern* ?*files*?

::fileutil::install ?*-m mode*? *source destination*
::fileutil::stripN *path n*
::fileutil::stripPwd *path*
::fileutil::stripPath *prefix path*
::fileutil::jail *jail path*
::fileutil::touch ?**-a**? ?**-c**? ?**-m**? ?*-r ref_file*? ?*-t time*? *filename* ?*...*?
::fileutil::tempdir
::fileutil::tempdir *path*
::fileutil::tempdirReset
::fileutil::tempfile ?*prefix*?
::fileutil::maketempdir ?*-prefix str*? ?*-suffix str*? ?*-dir str*?
::fileutil::relative *base dst*
::fileutil::relativeUrl *base dst*

Beschreibung

Dieses Paket enthält Implementierungen von Standard-Unix-Dienstprogrammen.

::fileutil::lexnormalize *path*
Dieser Befehl führt eine rein lexikalische Normalisierung des *Pfads* durch und gibt den geänderten Pfad als Ergebnis zurück. Symbolische Links im Pfad werden *nicht* aufgelöst.

Beispiele:

```
fileutil::lexnormalize /foo/./bar
=> /foo/bar

fileutil::lexnormalize /foo/../bar
=> /bar
```

::fileutil::fullnormalize *path*
Dieser Befehl löst alle symbolischen Links im *Pfad* auf und gibt den geänderten Pfad als Ergebnis zurück. Im Gegensatz zum integrierten Befehl **file normalize** löst dieser Befehl auch einen symbolischen Link im letzten Element des Pfads auf.

::fileutil::test *path codes* ?*msgvar*? ?*label*?
Ein Befehl zum Testen verschiedener Eigenschaften eines *Pfads*. Die zu testenden Eigenschaften werden in *codes* angegeben, entweder als Liste von Schlüsselwörtern, die die Eigenschaften beschreiben, oder als Zeichenkette, in der jeder Buchstabe eine Abkürzung für eine zu testende Eigenschaft darstellt. Die erkannten Schlüsselwörter, Abkürzungen und die zugehörigen Eigenschaften sind in der folgenden Liste aufgeführt. Die Tests werden in der Reihenfolge ausgeführt, in der sie dem Befehl übergeben wurden.

Das Ergebnis des Befehls ist ein boolescher Wert. Er ist genau dann wahr, wenn der *Pfad* alle angegebenen Tests besteht. Falls der *Pfad* einen oder mehrere Tests nicht besteht, hinterlässt der erste fehlgeschlagene Test eine Meldung in der Variable, auf die *msgvar* verweist, wenn diese festgelegt ist. Der Meldung wird *label* vorangestellt, wenn dieses festgelegt ist. *Hinweis*: Die Variable, auf die *msgvar* verweist, wird überhaupt nicht verändert, wenn alle Tests bestanden werden.

***r*ead**
file readable

***w*rite**
file writable
***e*xists**
file exists
e*x*ec
file executable
***f*ile**
file isfile
***d*ir**
file isdirectory

::fileutil::cat (?*options*? *file*)...
Eine Tcl-Implementierung des UNIX-Befehls **cat**. Gibt den Inhalt der angegebenen Datei(en) zurück. Die Argumente sind die zu lesenden Dateien, dazwischen stehen Optionen zur Konfiguration des Prozesses. Sollten beim Lesen einer der Dateien Probleme auftreten, wird ein Fehler ausgegeben und es werden keine Daten zurückgegeben.

Die zulässigen Optionen sind **-encoding**, **-translation**, **-eofchar** und **--**. Mit Ausnahme der letzten Option nehmen alle Optionen einen einzelnen Wert als Argument entgegen, wie durch den Tcl-Befehl **fconfigure** festgelegt. Die Option **--** muss verwendet werden, um die Optionsverarbeitung vor einer Datei zu beenden, wenn der Name dieser Datei mit einem Bindestrich beginnt.

Jede Datei kann ihre eigenen Optionen vorangestellt haben, und für alles, was nicht direkt angegeben ist, werden die Standardwerte aus den Optionen der vorherigen Datei übernommen. Die erste Datei übernimmt die Systemstandardwerte für nicht angegebene Optionen.

::fileutil::writeFile ?*options*? *file data*
Der Befehl ersetzt den aktuellen Inhalt der angegebenen *Datei* durch *Daten*, wobei der Prozess durch die Optionen konfiguriert wird. Der Befehl akzeptiert dieselben Optionen wie **::fileutil::cat**. Die Angabe einer nicht vorhandenen Datei ist zulässig und bewirkt, dass der Befehl die Datei (sowie alle erforderlichen, aber fehlenden Verzeichnisse) erstellt.

::fileutil::appendToFile ?*options*? *file data*
Dieser Befehl verhält sich ähnlich wie **::fileutil::writeFile**, mit dem Unterschied, dass der bisherige Inhalt von *file* nicht ersetzt, sondern angehängt wird. Der Befehl akzeptiert dieselben Optionen wie **::fileutil::cat**.

::fileutil::insertIntoFile ?*options*? *file at data*
Dieser Befehl ähnelt **::fileutil::appendToFile**, mit dem Unterschied, dass die neuen Daten nicht am Ende angehängt, sondern an einer bestimmten Stelle innerhalb der Datei eingefügt werden. Außerdem muss bei diesem Befehl der Pfad zu einer vorhandenen Datei angegeben werden. Eine nicht vorhandene Datei wird nicht erstellt, sondern es wird stattdessen eine Fehlermeldung ausgegeben.

Die angegebene Position *at* muss eine ganze Zahl im Bereich von **0** bis [Dateigröße *Datei*] sein. **0** bewirkt das Einfügen der neuen Daten vor dem ersten Zeichen des vorhandenen

Inhalts, während [Dateigröße *Datei*] das Einfügen nach dem letzten Zeichen des vorhandenen Inhalts bewirkt, d. h. das Anhängen.

Der Befehl akzeptiert dieselben Optionen wie **::fileutil::cat**.

::fileutil::removeFromFile ?*options*? *file at n*

Dieser Befehl ist das Gegenstück zu **::fileutil::insertIntoFile** und entfernt *n* Zeichen aus der *Datei*, beginnend an der Position *at*. Die angegebene Position *at* muss eine ganze Zahl im Bereich von **0** bis [Dateigröße *Datei*] sein - *n*. Bei **0** beginnt das Entfernen der neuen Daten mit dem ersten Zeichen des vorhandenen Inhalts, während [Dateigröße *Datei*] - *n* das Entfernen des Endes des vorhandenen Inhalts bewirkt, d. h. das Kürzen der Datei.

Der Befehl akzeptiert dieselben Optionen wie **::fileutil::cat**.

::fileutil::replaceInFile ?*options*? *file at n data*

Dieser Befehl ist eine Kombination aus **::fileutil::removeFromFile** und **::fileutil::insertIntoFile**. Er entfernt zunächst den durch die Argumente *at* und *n* festgelegten Teil des Inhalts und fügt dann *data* an der angegebenen Stelle ein, wodurch der entfernte Inhalt effektiv durch *data* ersetzt wird. Alle durch **::fileutil::removeFromFile** und **::fileutil::insertIntoFile** auferlegten Constraints für *at* und *n* werden beachtet.

Der Befehl akzeptiert dieselben Optionen wie **::fileutil::cat**.

::fileutil::updateInPlace ?*options*? *file cmd*

Dieser Befehl kann als die allgemeine Kernfunktionalität von **::fileutil::replaceInFile** betrachtet werden. Er liest zunächst den Inhalt der angegebenen *Datei*, führt dann den Befehl *cmd* aus, wobei diese Daten angehängt werden, und schreibt schließlich das Ergebnis dieses Aufrufs als neuen Inhalt der Datei zurück.

Wenn der ausgeführte Befehl einen Fehler auslöst, wird die *Datei* nicht verändert.

Der Befehl akzeptiert dieselben Optionen wie **::fileutil::cat**.

::fileutil::fileType *filename*

Eine Implementierung des UNIX-Befehls **file**, die verschiedene Heuristiken verwendet, um den Dateityp zu ermitteln. Gibt eine Liste zurück, die so viele Typinformationen wie möglich über die Datei enthält, von den allgemeinsten (z. B. "binary" oder "text") bis zu den spezifischsten (z. B. "gif"). Der Rückgabewert für eine GIF-Datei wäre beispielsweise "binary graphic gif". Der Befehl erkennt die folgenden Dateitypen: Verzeichnis, leer, binär, Text, Skript (mit Interpreter), ausführbare ELF-Datei, ausführbare DOS-Datei, ausführbare NE-Datei, ausführbare PE-Datei, Grafik-GIF, Grafik-JPEG, Grafik-PNG, Grafik-TIFF, Grafik-Bitmap, HTML, XML (mit Doctype, wenn verfügbar), PGP-Nachricht, binäres PDF, Text-PS, Text-EPS, binäres Gravity_Wave_Data_Frame, komprimiertes Bzip, komprimiertes Gzip, komprimiertes Zip, komprimiertes Tar, Audio-Wave, Audio-MPEG und Link. Außerdem erkennt es "doctools", "doctoc" und "docidx"-Dokumentationsdateien sowie "tklib"-Diagramme.

::fileutil::find ?*basedir* ?*filtercmd*??

Eine Implementierung des Unix-Befehls **find**. Adaptiert aus dem Tcler-Wiki. Nimmt höchstens zwei Argumente entgegen: den Pfad zu dem Verzeichnis, in dem die Suche beginnen soll, und einen Befehl, mit dem die Auswertung des Interesses an jeder Datei durchgeführt wird. Der Pfad ist standardmäßig ".", d. h. das aktuelle Verzeichnis. Der Befehl ist standardmäßig die

leere Zeichenkette, was bedeutet, dass alle Dateien von Interesse sind. Der Befehl sorgt dafür, dass er sich bei zirkulären Verknüpfungsstrukturen *nicht* in Endlosschleifen verliert. Das Ergebnis des Befehls ist eine Liste, die die Pfade zu den relevanten Dateien enthält.

Der *filtercmd* wird, wenn er angegeben ist, als Befehlspräfix interpretiert, und ihm wird ein Argument hinzugefügt: der Name der Datei oder des Verzeichnisses, das find gerade prüft. Hinweis: Dieser Name ist *nicht* vollständig qualifiziert. Er muss mit dem Ergebnis von **pwd** verknüpft werden, um einen absoluten Dateinamen zu erhalten.

Das Ergebnis von *filtercmd* ist ein boolescher Wert, der angibt, ob die aktuelle Datei in die Liste der relevanten Dateien aufgenommen werden soll.

Beispiele:

```
# find .tcl files
package require fileutil
proc is_tcl {name} {return [string match *.tcl $name]}
set tcl_files [fileutil::find . is_tcl]
```

::fileutil::findByPattern *basedir* ?-regexp|-glob? ?--? *patterns*

Dieser Befehl basiert auf dem **TclX**-Befehl **recursive_glob**, mit dem Unterschied, dass er keine Rekursion über mehr als ein Verzeichnis gleichzeitig zulässt. Er nutzt intern **::fileutil::find** und ist daher in der Lage, symbolische Links zu verfolgen – was der **TclX** -Befehl nicht tut. Das erste Argument ist das Verzeichnis, in dem die Suche beginnen soll, das zweite Argument ist eine Liste von *Mustern*. Der Befehl gibt eine Liste aller Dateien zurück, die über *basedir* erreichbar sind und deren Namen mit mindestens einem der Muster übereinstimmen. Die Optionen vor der Musterliste legen den Abgleichstil fest, entweder reguläre Ausdrücke (regexp) oder Glob-Muster. Der Glob-Abgleich ist die Standardeinstellung, wenn keine Optionen angegeben werden. Die Verwendung der Option **--** beendet den Optionsprozess. Dies ermöglicht die Verwendung eines führenden "-" in den Mustern.

::fileutil::foreachLine *var filename cmd*

Der Befehl liest die Datei *Dateiname* ein und führt für jede Zeile in der Datei das Skript *cmd* aus. Während der Ausführung des Skripts wird die Variable *var* auf den Inhalt der aktuellen Zeile gesetzt. Der Wert dieses Befehls ist das Ergebnis des letzten Aufrufs des Skripts *cmd* oder die leere Zeichenkette, wenn die Datei leer war.

::fileutil::grep *pattern* ?*files*?

Implementierung von **grep**. Adaptiert aus dem Tcler-Wiki. Das erste Argument definiert das *Muster*, nach dem gesucht werden soll. Darauf folgt eine Liste der *Dateien*, die durchsucht werden sollen. Die Liste ist optional; fehlt sie, wird **stdin** verwendet. Das Ergebnis der Prozedur ist eine Liste, die die Treffer enthält. Jeder Treffer ist ein einzelnes Element der Liste und enthält den Dateinamen, die Nummer und den Inhalt der übereinstimmenden Zeile, getrennt durch Doppelpunkte.

::fileutil::install ?-m *mode*? *source destination*

Der Befehl **install** ähnelt in seiner Funktionsweise dem Befehl **install**, der auf vielen Unix-Systemen zu finden ist, oder dem Shell-Skript, das mit vielen Quellcode-Distributionen mitgeliefert wird (beispielsweise "unix/install-sh" in den Tcl-Quellcodes). Er kopiert *Quelle* – bei der es sich entweder um eine Datei oder ein Verzeichnis handeln kann – nach *Ziel*, das ein Verzeichnis sein sollte, es sei denn, *Quelle* ist ebenfalls eine einzelne Datei. Mit der Option

"-m" kann der Benutzer einen Modus im Unix-Stil festlegen (entweder oktal oder symbolisch – siehe **file attributes**).

::fileutil::stripN *path n*
Entfernt die ersten *n* Elemente aus dem angegebenen *Pfad* und gibt den geänderten Pfad zurück. Wenn *n* größer ist als die Anzahl der Komponenten in *Pfad*, wird eine leere Zeichenkette gegeben. Die Anzahl der Komponenten in einem bestimmten Pfad lässt sich ermitteln, indem Sie die Funktion **llength** auf die von **file split** zurückgegebene Liste anwendet.

::fileutil::stripPwd *path*
Genau dann, wenn sich der *Pfad* innerhalb des von [**pwd**] gegebenen Verzeichnisses (oder des aktuellen Arbeitsverzeichnisses selbst) befindet, wird er relativ zu diesem Verzeichnis umgewandelt. Mit anderen Worten: Das aktuelle Verzeichnis wird aus dem *Pfad* entfernt. Der gegebenenfalls modifizierte Pfad wird als Ergebnis des Befehls zurückgegeben. Wenn das aktuelle Verzeichnis selbst für *Pfad* festgelegt wurde, gibt der Befehl die Zeichenkette "**.**" zurück.

::fileutil::stripPath *prefix path*
Nur wenn sich der *Pfad* innerhalb des Verzeichnisses "*Präfix*" (oder im Präfixverzeichnis selbst) befindet, wird er relativ zu diesem Verzeichnis umgewandelt. Mit anderen Worten: Das Präfixverzeichnis wird aus dem *Pfad* entfernt. Der gegebenenfalls geänderte Pfad wird als Ergebnis des Befehls zurückgegeben. Wenn das Präfixverzeichnis selbst für *Pfad* festgelegt wurde, gibt der Befehl die Zeichenkette "**.**" zurück.

::fileutil::jail *jail path*
Dieser Befehl stellt sicher, dass der *Pfad* nicht aus dem Verzeichnis *jail* herausführt. Er gibt stets einen absoluten Pfad zurück, der aus *Pfad* abgeleitet wurde und sich innerhalb von *jail* befindet.

Ist *Pfad* ein absoluter Pfad und befindet er sich bereits innerhalb von *jail*, wird er unverändert zurückgegeben.

Ein absoluter Pfad außerhalb von *jail* wird um sein Stammverzeichnis gekürzt und dann in *jail* eingefügt, indem ihm dieses vorangestellt wird. Dasselbe geschieht, wenn *path* relativ ist, mit dem Unterschied, dass in diesem Fall nichts davon entfernt wird. Vor dem Hinzufügen des Präfixes *jail* wird *Pfad* lexikalisch normalisiert, um zu verhindern, dass der Aufrufer**..** -Segmente in *Pfad* verwendet, um aus dem *jail* zu entkommen.

::fileutil::touch ?-a? ?-c? ?-m? ?-r *ref_file*? ?-t *time*? *filename* ?...?
Implementierung von **touch**. Ändert die atime und mtime der angegebenen Dateien. Wenn **-c** gesetzt ist, werden keine Dateien erstellt, sofern diese nicht bereits vorhanden sind. Wenn **-r**, werden die atime- und mtime-Werte aus *ref_file* verwendet. Wenn **-t** verwendet wird, wird der ganzzahlige Wert *time* verwendet. Es ist unzulässig, sowohl **-r** als auch **-t** festzulegen. Wenn **-a** erreicht wird, wird nur die atime geändert. Wenn **-m** erreicht wird, wird nur die mtime geändert.

Dieser Befehl ist für Tcl-Versionen unter 8.3 nicht verfügbar.

::fileutil::tempdir
Der Befehl gibt den Pfad eines Verzeichnisses zurück, in dem der Aufrufer temporäre Dateien

ablegen kann, beispielsweise "*/tmp*" auf Unix-Systemen. Der Algorithmus, den wir zur Ermittlung des richtigen Verzeichnisses verwenden, lautet wie folgt:

1. Das Verzeichnis, das durch einen Aufruf von **::fileutil::tempdir** mit einem Argument festgelegt wurde. Wenn dieses vorhanden ist, wird ausschließlich dieses Verzeichnis geprüft, und keines der folgenden Elemente wird berücksichtigt.
2. Das Verzeichnis, das in der Variable TMPDIR angegeben ist.
3. Das Verzeichnis, das in der Variable TEMP angegeben ist.
4. Das Verzeichnis, das in der Umgebungsvariablen TMP angegeben ist.
5. Ein plattformspezifischer Speicherort:

   Windows
   "*C:\TEMP*", "*C:\TMP*", "*\TEMP*" und "*\TMP*" werden in dieser Reihenfolge geprüft.

   (classic) Macintosh
   Es wird die Variable TRASH_FOLDER verwendet. Dies ist höchstwahrscheinlich nicht korrekt.

   Unix
   Die Verzeichnisse "*/tmp*", "*/var/tmp*" und "*/usr/tmp*" werden in dieser Reihenfolge geprüft.

Der verwendete Algorithmus entspricht im Wesentlichen dem in der Python-Standardbibliothek verwendeten. Die einzige Ausnahme bildet der erste Punkt, nämlich die Möglichkeit, die Suche durch ein benutzerdefiniertes Verzeichnis zu überschreiben.

::fileutil::tempdir *path*
In diesem Modus legt der Befehl den *Pfad* als erstes und einziges Verzeichnis fest, das als temporäres Verzeichnis verwendet werden soll. Informationen zur Verwendung des Verzeichnisses "set" finden Sie im vorherigen Punkt. Der Befehl gibt die leere Zeichenkette zurück.

::fileutil::tempdirReset
Durch die Ausführung dieses Befehls werden die Informationen gelöscht, die beim letzten Aufruf von [**::fileutil::tempdir** *Pfad*] festgelegt wurden. Siehe auch den letzten Punkt.

::fileutil::tempfile ?*prefix*?
Der Befehl generiert einen Namen für eine temporäre Datei, in die geschrieben werden kann, sowie die zugehörige Datei. Der Dateiname ist eindeutig, die Datei ist beschreibbar und befindet sich im entsprechenden systemspezifischen temporären Verzeichnis. Der Name der Datei wird als Ergebnis des Befehls zurückgegeben.

Der Code stammt aus http://wiki.tcl.tk/772 und wird Igor Volobouev und einem anonymen Autor zugeschrieben.

::fileutil::maketempdir ?-prefix *str*? ?-suffix *str*? ?-dir *str*?
Der Befehl erstellt ein temporäres Verzeichnis, in das geschrieben werden kann. Der Name des Verzeichnisses ist eindeutig, das Verzeichnis ist beschreibbar und befindet sich im entsprechenden systemspezifischen temporären Verzeichnis. Der Name des Verzeichnisses wird als Ergebnis des Befehls zurückgegeben.

Mit den drei Optionen lässt sich das Verhalten des Befehls anpassen:
-prefix str

Der anfängliche, feste Teil des Verzeichnisnamens. Wenn nicht angegeben, lautet der Standardwert **tmp**.

-suffix str
Der feste Abschluss des Verzeichnisses. Wenn nichts angegeben wird, ist der Standardwert eine leere Zeichenkette.

-dir str
Das Verzeichnis, in dem das neue Verzeichnis platziert werden soll. Wenn nichts angegeben wird, wird standardmäßig das Ergebnis von **fileutil::tempdir** verwendet.

Der ursprüngliche Code hierfür wurde von Miguel Martinez Lopez bereitgestellt.

::fileutil::relative *base dst*
Dieser Befehl nimmt zwei Verzeichnispfade entgegen, die beide entweder absolut oder relativ sind, und berechnet den Pfad von *dst* relativ zu *base*. Dieser relative Pfad wird als Ergebnis des Befehls zurückgegeben. Wie aus dem vorigen Satz hervorgeht, kann der Befehl diese Beziehung zwischen den Argumenten nicht berechnen, wenn einer der Pfade absolut und der andere relativ ist.

Hinweis: Der Prozess durch diesen Befehl erfolgt rein lexikalisch. Symbolische Verknüpfungen werden *nicht* berücksichtigt.

::fileutil::relativeUrl *base dst*
Dieser Befehl nimmt zwei Dateipfade entgegen, die beide entweder absolut oder relativ sind, und berechnet den Pfad von *dst* relativ zu *base*, aus der Perspektive von *base* betrachtet. Dies entspricht dem Algorithmus, mit dem ein Browser einen relativen Link in der aktuell angezeigten Datei auflöst.

Der berechnete relative Pfad wird als Ergebnis des Befehls zurückgegeben. Wie aus dem vorangegangenen Satz hervorgeht, kann der Befehl diese Beziehung zwischen den Argumenten nicht berechnen, wenn einer der Pfade absolut und der andere relativ ist.

Hinweis: Die Verarbeitung durch diesen Befehl erfolgt rein lexikalisch. Symbolische Links werden *nicht* berücksichtigt.

Fehler und Inkompatibilitäten

1.14.9

In dieser Version wurde das fehlerhafte System von **fileutil::find** zur Verarbeitung von Symlinks durch ein korrekt funktionierendes System ersetzt, das alle zulässigen, nicht zyklischen Pfade unter einem Basis-Verzeichnis ordnungsgemäß auflistet.

Dies ist zwar korrekt, bedeutet jedoch, dass die Aufzählung bestimmter pathologischer Verzeichnishierarchien mit sich gegenseitig verweisenden Symlinks nun etwa O(n**2) Zeit in Anspruch nimmt, während der ursprüngliche fehlerhafte Code aufgrund seiner Fehlerhaftigkeit O(n) schaffte.

Ein konkretes Beispiel und Extremfall ist die "*/sys*"-Hierarchie unter Linux, in der einige hundert Geräte sowohl unter "*/sys/devices*" als auch unter "*/sys/class*" existieren, wobei die beiden Unterhierarchien miteinander verknüpft sind, was Millionen zulässige Pfade zur Aufzählung erzeugt. Die auf drei Geräte reduzierte Struktur sieht in etwa so aus:

```
    /sys/class/tty/tty0 --> ../../dev/tty0
    /sys/class/tty/tty1 --> ../../dev/tty1
    /sys/class/tty/tty2 --> ../../dev/tty1

    /sys/dev/tty0/bus
    /sys/dev/tty0/subsystem --> ../../class/tty
    /sys/dev/tty1/bus
    /sys/dev/tty1/subsystem --> ../../class/tty
    /sys/dev/tty2/bus
    /sys/dev/tty2/subsystem --> ../../class/tty
```

Der Befehl **fileutil::find** bietet derzeit keine Möglichkeit, dies zu umgehen. Wenn Sie mit einer solchen pathologischen Hierarchie umgehen müssen, empfiehlt es sich, auf das Paket **fileutil::traverse** und den darin enthaltenen gleichnamigen Befehl umzusteigen und anschließend die Option **-prefilter** zu verwenden, um zu verhindern, dass der Traverser symbolischen Links folgt, und zwar wie folgt:

```
    package require fileutil::traverse

    proc NoLinks {fileName} {
        if {[string equal [file type $fileName] link]} {
            return 0
        }
        return 1
    }

    fileutil::traverse T /sys/devices -prefilter NoLinks
    T foreach p {
        puts $p
    }
    T destroy
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

multi

Stichwörter: multi, copy, file utilities, move, multi-file, remove

Name
fileutil::multi – Bearbeitung mehrerer Dateien, Scatter/Gather, Standardobjekt

Übersicht
package require Tcl 8.5 9
package require fileutil::multi ?**0.2**?
package require fileutil::multi::op ?**0.5.4**?
package require wip ?**1.1**?
::fileutil::multi ?*word...*?

Beschreibung
Dieses Paket bietet einen einzigen Befehl, um Aktionen an mehreren Dateien durchzuführen, die anhand von Glob-Mustern ausgewählt wurden. Es handelt sich um eine dünne Schicht über dem Paket **fileutil::multi::op**, das Objekte für denselben Zweck bereitstellt. Dieses Paket erstellt lediglich ein einziges solches Objekt und leitet alle Dateibefhle an dieses weiter.

Im Mittelpunkt steht eine domänenspezifische Sprache, die die einfache Spezifizierung von Operationen zum Kopieren und/oder Verschieben und/oder Löschen mehrerer Dateien ermöglicht. Alternative Namen wären "Scatter/Gather-Prozessor" oder vielleicht sogar "Assembler". Die detaillierte Spezifikation dieser Sprache sowie Beispiele finden Sie in der Dokumentation zum Paket **fileutil::multi::op**.

Öffentliche API
Der Hauptbefehl des Pakets lautet:

::fileutil::multi ?*word*...?
: Dieser Befehl interpretiert die angegebenen Wörter als auszuführende Dateibefehle. Informationen zu den zulässigen Befehlen, ihrer Syntax und Semantik finden Sie im Abschnitt **FILE API** der Dokumentation zum Paket **fileutil::multi::op**.

  Das Ergebnis des Befehls ist das Ergebnis des zuletzt ausgeführten Dateibefehls.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

multiop
Stichwörter: multiop, copy, file utilities, move, multi-file, remove

Name

fileutil::multi::op – Bearbeitung mehrerer Dateien, Scatter/Gather

Übersicht

package require Tcl 8.5 9
package require fileutil::multi::op ?**0.5.4**?
package require wip ?**1.0**?
::fileutil::multi::op ?*opName*? ?*word...*?
opName *option* ?*arg arg* ...?
$opName do ?*word...*?
into *directory*
in *directory*
to *directory*
from *directory*
not *pattern*
for *pattern*
exclude *pattern*
but
except
as *name*
recursive
recursively
copy
move
remove
expand
invoke *cmdprefix*
reset
(
)
cd *directory*
up
for-windows
for-win
for-unix
the *pattern*
the-set *varname*
-> *varname*
strict
!strict
files
links
directories
dirs
all
state?

as?
excluded?
from?
into?
operation?
recursive?
strict?
type?

Beschreibung

Dieses Paket stellt Objekte bereit, mit denen sich Aktionen auf mehrere Dateien ausführen lassen, die anhand von Glob-Mustern ausgewählt wurden.

Im Mittelpunkt steht eine domänenspezifische Sprache, die die einfache Spezifizierung von Operationen zum Kopieren und/oder Verschieben und/oder Löschen mehrerer Dateien ermöglicht. Alternative Namen wären "Scatter/Gather-Prozessor" oder vielleicht sogar "Assembler".

API für Klassen

Der Hauptbefehl des Pakets lautet:

::fileutil::multi::op ?*opName*? ?*word*...?

Der Befehl erstellt ein neues Objekt für eine Operation mit mehreren Dateien, dem ein globaler Tcl-Befehl mit dem Namen *opName* zugeordnet ist. Mit diesem Befehl lassen sich die verschiedenen möglichen Dateioperationen aufrufen. Er hat die folgende allgemeine Form:
opName *option* ?*arg arg ...*?

Option und die *arg*s bestimmen das genaue Verhalten des Befehls.

Wenn die Zeichenfolge **%AUTO%** als *opName* verwendet wird, generiert das Paket selbstständig einen eindeutigen Name.

Wenn ein oder mehrere *Wörter* angegeben werden, werden diese als anfängliche Reihe von auszuführenden Dateibefehlen interpretiert. Das heißt, die Methode **do** des neu erstellten Objekts wird implizit aufgerufen, wobei die Wörter als Argumente verwendet werden.

Objekt-API

Für Objekte mit Mehrdateienbetrieb sind folgende Methoden möglich:

$opName do ?*word*...?

Diese Methode interpretiert die angegebenen Wörter als auszuführende Dateibefehle. Informationen zu den zulässigen Befehlen, ihrer Syntax und Semantik finden Sie im Abschnitt Datei-API.

Das Ergebnis der Methode ist das Ergebnis des zuletzt ausgeführten Dateibefehls.

Datei-API

Sowohl der Objektkonstruktor als auch die Methode **do** eine Liste von Wörtern und interpretieren diese als auszuführende Dateibefehle. Die Namen wurden so gewählt, dass Operationen als Sätze in einer nahezu natürlichen Sprache gebildet werden können. Die meisten Befehle

beeinflussen lediglich den Zustand des Objekts, d. h., sie liefern lediglich die Konfiguration, die von dem Befehl verwendet wird, der die eigentliche Aktion auslöst.

into *directory*
Legt das Zielverzeichnis für die Vorgänge fest.

in *directory*
Alias für **into**.

to *directory*
Alias für **into**.

from *directory*
Legt das Verzeichnis für die Vorgänge fest.

not *pattern*
Legt ein Glob-Muster für Pfade fest, die von der Operation ausgeschlossen werden sollen.

for *pattern*
Alias für **not**.

exclude *pattern*
Alias für **not**.

but
Hat keine eigenen Argumente, sondern durchsucht die Wortliste nach vorne und führt alle **not** -Befehle aus, die unmittelbar darauf folgen. Dies ermöglicht die Bildung von "aber nicht"- und "aber ausschließen"-Klauseln für eine natürlicher klingende Angabe ausgeschlossener Pfade.

except
Ein Semi-Alias für **but**. Hat keine eigenen Argumente, sucht jedoch in der Wortliste vorwärts und führt alle unmittelbar darauf folgenden **for**-Befehle aus. Dies ermöglicht die Bildung von "außer"-Klauseln für eine natürlicher klingende Angabe ausgeschlossener Pfade.

as *name*
Legt einen neuen Name für die erste Datei fest, die im Rahmen des aktuellen Vorgangs bearbeitet wird. Das heißt, für die Umbenennung einer einzelnen Datei während des Vorgangs.

recursive
Gibt an, dass die Dateierweiterung in der gesamten Verzeichnishierarchie und nicht nur im Verzeichnis selbst erfolgen soll.

recursively
Ein Alias für **recursive**.

copy
Zeigt an, dass es sich bei dem Vorgang um das Kopieren von Dateien vom Quell- in das Zielverzeichnis gemäß den angegebenen Einschluss- und Ausschlussmustern handelt.

move
Zeigt an, dass es sich bei dem Vorgang um das Verschieben von Dateien vom Quell- in das Zielverzeichnis gemäß den angegebenen Ein- und Ausschlussmustern handelt.

remove
Zeigt an, dass bei diesem Vorgang Dateien im Zielverzeichnis gemäß den angegebenen Ein- und Ausschlussmustern entfernt werden.

expand
Zeigt an, dass außer der Berechnung der Dateimenge anhand der Ein- und Ausschlussmuster

keine weitere Operation stattfindet. Diese Operation ist nicht verfügbar, wenn **the-set** verwendet wird.

invoke *cmdprefix*
Gibt an, dass das vom Benutzer angegebene Befehlspräfix *cmdprefix* die auszuführende Operation ist. Das Befehlspräfix wird auf globaler Ebene ausgeführt und erhält dabei das Quellverzeichnis, das Zielverzeichnis sowie die Dateigruppe (als Wörterbuchzuordnung von Quell- zu Zieldateien) in dieser Reihenfolge.

reset
Versetzt das Objekt in den Grundzustand, in dem alle Teile der Konfiguration Standardwerte aufweisen.

(
Speichert eine Kopie des aktuellen Zustands des Objekts auf einem Stapel.

)
Entnimmt den Zustand vom obersten Element des Zustandsstapels und stellt ihn wieder her, d. h., macht ihn zum neuen aktuellen Zustand des Objekts.

cd *directory*
Ändert das Zielverzeichnis in das Unterverzeichnis *verzeichnis* des aktuellen Ziels.

up
Ändert das Zielverzeichnis in das übergeordnete Verzeichnis des aktuellen Ziels.

for-windows
Prüft, ob Windows die aktuelle Plattform ist. Bricht den Prozess ab, wenn dies nicht der Fall ist.

for-win
Ein Alias für **for-windows**.

for-unix
Prüft, ob Unix die aktuelle Plattform ist. Bricht den Prozess ab, wenn dies nicht der Fall ist.

the *pattern*
Dieser Befehl legt die zu bearbeitenden Dateien anhand eines Glob-Musters fest und ist zugleich das aktive Element, d. h. der Befehl, der die angegebene Operation tatsächlich ausführt. Alle anderen Befehle ändern lediglich den Objektzustand, um die Operation vorzubereiten, führen aber keine weiteren Aktionen aus.

Um eine natürlichere Syntax zu ermöglichen, blickt dieser Befehl auch in die Liste der nachfolgenden Wörter vor und führt mehrere unmittelbar darauf folgende Befehle aus, bevor er seine eigenen Aktionen ausführt. Diese Befehle sind **as**, **but**, **exclude**, **except**, **from** und **into** (sowie Aliase). Auf diese Weise fungieren diese Befehle wie Qualifizierer und werden dennoch so wirksam, als wären sie vor diesem Befehl geschrieben worden.

Nachdem die Operation ausgeführt wurde, werden der Objektzustand, die Ausschlussmuster und der Aliasname, wenn sie festgelegt wurden, auf ihre Standardwerte (d. h. leer) zurückgesetzt, aber sonst nichts.

the-set *varname*
Ähnlich wie bei **the** wird die zu verwendende Dateimenge jedoch nicht implizit über ein Glob-Muster angegeben, sondern ist in der angegebenen Variable enthalten und wird von dort geladen. Die Operation **expand** steht bei Verwendung dieses Befehls nicht zur Verfügung.

-> *varname*

Speichert die Dateien aus der letzten Erweiterung in der angegebenen Variable.

strict
Die Dateierweiterung und die Definition des Zielverzeichnisses (**in** und Aliase) streng handhaben, d. h. Fehler bei fehlenden Verzeichnissen und leeren Erweiterungen melden.

!strict
Ergänzung zu **strict**. Ein fehlendes Zielverzeichnis oder eine leere Erweiterung werden nicht als Fehler gemeldet.

files
Die Suche auf Dateien beschränken. Standardmäßig werden alle Pfadtypen akzeptiert.

links
Die Suche auf symbolische Links beschränken. Standardmäßig werden alle Pfadtypen akzeptiert.

directories
Die Suche auf Verzeichnisse beschränken. Standardmäßig werden alle Arten von Pfaden akzeptiert.

dirs
Ein Alias für **directories**.

all
Alle Pfadtypen akzeptieren (Standard).

state?
Gibt den aktuellen Zustand des Objekts als Dictionary zurück. Die Schlüssel des Dictionary und ihre Bedeutungen lauten:

as
Letzte Einstellung vorgenommen von **as**.

excluded
Liste der derzeit bekannten Ausschlussmuster.

from
Aktuelles Verzeichnis für die Quellcodes, festgelegt durch **from**.

into
Aktuelles Verzeichnis für das Ziel, festgelegt durch **into** (und Aliase).

operation
Derzeit auszuführende Operation, festgelegt durch **copy**, **move**, **remove**, **expand** oder **invoke**.

recursive
Aktueller Rekursionsstatus. Wird durch **recursive** und**!recursive** gesetzt bzw. zurückgesetzt.

strict
Aktuelle Strenge. Wird mit **strict** und**!strict** gesetzt bzw. aufgehoben.

type
Begrenzer für den aktuellen PFADTYP. Einstellbar auf **files**, **directories**, **links** oder **all**.

as?
Gibt den aktuellen Aliasnamen zurück.

excluded?

Gibt die aktuell festgelegten Ausschlussmuster zurück.

from?
Gibt das aktuelle Quellverzeichnis zurück.

into?
Gibt das aktuelle Zielverzeichnis zurück.

operation?
Gibt die aktuell auszuführende Operation zurück.

recursive?
Gibt den aktuellen Rekursionsstatus zurück.

strict?
Gibt den aktuellen Strengegrad zurück.

type?
Gibt den aktuellen Pfadtyp-Begrenzer zurück.

Beispiele

In den folgenden Beispielen wird davon ausgegangen, dass die Variable **F** einen Verweis auf ein Objekt für eine Operation mit mehreren Dateien enthält.

```
$F do copy                      \
the  *.dll                    \
from c:/TDK/PrivateOpenSSL/bin \
to   [installdir_of tls]
```

```
$F do move      \
the  *       \
from /sources \
into /scratch  \
but not *.html

# Alternatively use 'except for *.html'.
```

```
$F do           \
move         \
the  index    \
from /sources  \
into /scratch   \
as   pkgIndex.tcl
```

```
$F do         \
remove     \
the *.txt  \
in /scratch
```

Beachten Sie, dass die Tatsache, dass die meisten Befehle lediglich den Zustand des Objekts verändern, es uns ermöglicht, neben fast natürlich klingenden Sätzen auch weitere Formulierungen als Spezifikationen zu verwenden. So lässt sich beispielsweise das zweite Beispiel in diesem Abschnitt wie folgt umformulieren:

```
    $F do            \
    from /sources \
    into /scratch  \
    but not *.html \
    move           \
    the  *
```

Und das Ergebnis ist nicht nur nach wie vor eine gültige Spezifikation, sondern bleibt sogar relativ gut lesbar.

Beachten Sie außerdem, dass die durch die Befehle **but**, **except** und **as** gesammelten Informationen automatisch zurückgesetzt werden, nachdem das zugehörige **the** ausgeführt wurde. Es wird jedoch kein anderer Status auf diese Weise zurückgesetzt, sodass der Benutzer Wiederholungen unveränderlicher Informationen vermeiden kann. Beispielsweise lassen sich die zweiten und dritten Beispiele dieses Abschnitts zusammenfassen und wie folgt umschreiben:

```
$F do                   \
    move                 \
    the  *                \
    from /sources          \
    into /scratch           \
    but not *.html not index \
    the  index                \
    as   pkgIndex.tcl
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

paths

Stichwörter: paths

Name

fileutil::paths – Suchpfad-Pools verwalten

Übersicht

package require Tcl 8.5 9
package require fileutil::paths ?**1.1**?
::fileutil::paths *poolName*

poolName method ?*arg arg* ...?
poolName **add** *path*
poolName **clear**
poolName **paths**
poolName **remove** *path*

Beschreibung

Stellt eine snit-Klasse bereit, deren Instanzen einen Pool von (Such-)Pfaden verwalten.

API

Der Hauptbefehl dient zur Erstellung von Suchpfad-Pools:

::fileutil::paths *poolName*
: Erstellt einen neuen, leeren Pool von Suchpfaden mit einem zugehörigen globalen Tcl-Befehl, dessen Name *poolName* lautet. Dieser kann verwendet werden, um verschiedene Operationen auf den Pool anzuwenden. Er hat die folgende allgemeine Form:
poolName method ?*arg arg ...*?
method und *arg*umente bestimmen das genaue Verhalten des Befehls.

Wenn *poolName* als **%AUTO%** festgelegt wird, wird vom Paket selbst ein eindeutiger Name generiert. Das Ergebnis des Befehls ist der vollqualifizierte Name des Instanzbefehls.

Für Pool-Objekte sind folgende Befehle möglich:

poolName* add *path
: Fügt den *Pfad* zum Pool hinzu. Es geschieht nichts, wenn der *Pfad* dem Pool bereits bekannt ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

***poolName* clear**
: Leert den gesamten Pool. Mit anderen Worten: Entfernt alle Pfade daraus. Das Ergebnis des Befehls ist die leere Zeichenkette.

***poolName* paths**
: Gibt die Liste aller dem Pool bekannten Pfade in der Reihenfolge zurück, in der sie hinzugefügt wurden.

poolName* remove *path
: Entfernt den *Pfad* aus dem Pool, wenn dieser dem Pool bekannt ist. Unbekannte Pfade werden ohne Fehlermeldung ignoriert. Das Ergebnis des Befehls ist die leere Zeichenkette.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

traverse

Stichwörter: traverse, directory traversal, traversal

Name

fileutil_traverse – Iteratives Durchlaufen von Verzeichnissen

Übersicht

package require Tcl 8.5 9
package require fileutil::traverse ?**0.7**?
package require fileutil
package require control
::fileutil::traverse ?*objectName*? *path* ?*option value...*?
$traverser command ?*arg arg* ...?
$traverser files
$traverser foreach *filevar script*
$traverser next *filevar*

Beschreibung

Dieses Paket stellt Objekte für die programmgesteuerte Durchquerung von Verzeichnishierarchien bereit. Der wichtigste vom Paket exportierte Befehl lautet:

::fileutil::traverse ?*objectName*? *path* ?*option value*...?

Der Befehl erstellt ein neues Traversal-Objekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *objectName* lautet. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Traverser auszuführen. Wenn die Zeichenkette **%AUTO%** als *objectName* verwendet wird, wird vom Paket selbst ein eindeutiger Name generiert.

Informationen zu den unterstützten Optionen finden Sie im Abschnitt Optionen. Beachten Sie, dass alle diese Optionen nur bei der Erstellung des Traversal-Objekts festgelegt werden können. Eine spätere Änderung ist nicht möglich und führt bei einem entsprechenden Versuch zu Fehlern.

Der Objektbefehl hat die folgende allgemeine Form:

$traverser command ?*arg arg ...*?

Der *Befehl* und seine *Argumente* bestimmen das genaue Verhalten des Objekts.

Für Traversierungsobjekte sind folgende Befehle möglich:

$traverser files

Diese Methode ist die oberste Ebene, die von Durchlaufobjekten bereitgestellt wird. Bei ihrer Ausführung gibt sie eine Liste zurück, die die Namen aller Dateien und Verzeichnisse enthält, die der aktuellen Konfiguration des Durchlaufobjekts entsprechen.

$traverser foreach *filevar script*

Die übergeordnete Methode files (siehe oben) basiert auf dieser Methode der mittleren Ebene. Bei ihrer Aufrufung ermittelt sie alle Dateien und Verzeichnisse, die der aktuellen Konfiguration entsprechen, und führt für jeden Pfad das *Skript* aus. Der aktuell betrachtete Pfad wird in der Variable gespeichert, die mit *filevar* benannt ist. Sowohl die Variable als auch das Skript befinden sich im Kontext des Aufrufers der Methode und werden dort ausgeführt. In der Methode files speichert das Skript die gefundenen Pfade lediglich in der Liste, die zurückgegeben wird.

$traverser next *filevar*

Dies ist die unterste Schnittstelle zum Traverser, auf der alle übergeordneten Methoden

aufbauen. Bei Aufruf gibt sie einen booleschen Wert zurück, der angibt, ob ein Pfad gefunden wurde, der der aktuellen Konfiguration entspricht (**True**), oder nicht (**False**). Wurde ein Pfad gefunden, wird dieser im Kontext des Aufrufers in der durch *filevar* benannten Variablen gespeichert.

Die **foreach**-Methode ruft diese Methode einfach in einer Schleife auf, bis sie **False** gibt. Diese Methode wird bereitgestellt, damit wir eine Verzeichnishierarchie auch schrittweise und ereignisgesteuert durchlaufen können.

Beachten Sie, dass der Durchlaufer symbolische Links verfolgt, es sei denn, dies würde dazu führen, dass er in einen Link-Zyklus gerät. Mit anderen Worten: Der Befehl sorgt dafür, dass er sich beim Auftreffen auf zirkuläre Link-Strukturen *nicht* in Endlosschleifen verliert. Beachten Sie, dass auch Links, denen nicht gefolgt wird, dennoch im Ergebnis erscheinen.

Optionen

-prefilter command_prefix
Diese Rückruffunktion wird für Verzeichnisse ausgeführt. Ihr Ergebnis bestimmt, ob der Traverser das Verzeichnis rekursiv durchläuft oder nicht. Standardmäßig werden alle Verzeichnisse rekursiv durchsucht. Die Rückruffunktion wird mit einem einzigen Argument aufgerufen, dem *absoluten* Pfad des Verzeichnisses, und muss einen booleschen Wert zurückgeben: **True**, wenn das Verzeichnis den Filter passiert, und **False**, wenn nicht.

-filter command_prefix
Diese Rückruffunktion wird für alle Pfade ausgeführt. Ihr Ergebnis bestimmt, ob der aktuelle Pfad ein gültiges Ergebnis ist, und wird von **next** zurückgegeben. Standardmäßig werden alle Pfade als gültig akzeptiert. Die Rückruffunktion wird mit einem einzigen Argument aufgerufen, dem zu prüfenden *absoluten* Pfad, und muss einen booleschen Wert zurückgeben: **True**, wenn der Pfad den Filter passiert, und **False**, wenn nicht.

-errorcmd command_prefix
Diese Rückruffunktion wird für alle Pfade ausgeführt, bei denen der Traverser Probleme hat. Dazu gehören beispielsweise Fälle, in denen er nicht in diese Pfade wechseln oder deren Status abrufen kann usw. Standardmäßig werden solche Probleme ignoriert. Die Rückruffunktion wird mit zwei Argumenten aufgerufen: dem *absoluten* Pfad, bei dem der Fehler aufgetreten ist, und der Fehlermeldung. Auch Fehler, die von den Filter-Callbacks ausgelöst werden, werden über diese Rückruffunktion behandelt. Fehler, die vom Fehler-Callback selbst ausgelöst werden, werden nicht abgefangen und ignoriert, sondern an den Aufrufer weitergeleitet, also an den, der **next** aufgerufen hat. Alle anderen Ergebnisse der Rückruffunktion werden ignoriert.

Fehler und Inkompatibilitäten

0.4.4
In dieser Version wurde das fehlerhafte System des Traversers zur Verarbeitung von Symlinks durch ein korrekt funktionierendes ersetzt, das alle zulässigen, nicht zyklischen Pfade unter einem Basis-Verzeichnis ordnungsgemäß auflistet.

Das ist zwar korrekt, bedeutet aber, dass die Aufzählung bestimmter pathologischer Verzeichnishierarchien mit sich gegenseitig verweisenden Symlinks nun etwa O(n**2) Zeit in Anspruch nimmt, während der ursprüngliche fehlerhafte Code aufgrund seiner Fehlerhaftigkeit

O(n) schaffte.

Ein konkretes Beispiel und Extremfall ist die "*/sys*"-Hierarchie unter Linux, wo einige hundert Geräte sowohl unter "*/sys/devices*" als auch unter "*/sys/class*" existieren, wobei die beiden Unterhierarchien miteinander verknüpft sind, was Millionen zulässiger Pfade zur Aufzählung erzeugt. Die auf drei Geräte reduzierte Struktur sieht in etwa so aus:

```
    /sys/class/tty/tty0 --> ../../dev/tty0
    /sys/class/tty/tty1 --> ../../dev/tty1
    /sys/class/tty/tty2 --> ../../dev/tty1

    /sys/dev/tty0/bus
    /sys/dev/tty0/subsystem --> ../../class/tty
    /sys/dev/tty1/bus
    /sys/dev/tty1/subsystem --> ../../class/tty
    /sys/dev/tty2/bus
    /sys/dev/tty2/subsystem --> ../../class/tty
```

Wenn Sie es mit einer solchen pathologischen Hierarchie zu tun hat, empfiehlt es sich, die Option **-prefilter** zu verwenden, um zu verhindern, dass der Traverser symbolische Links verfolgt, und zwar wie folgt:

```
    package require fileutil::traverse

    proc NoLinks {fileName} {
        if {[string equal [file type $fileName] link]} {
            return 0
        }
        return 1
    }

    fileutil::traverse T /sys/devices -prefilter NoLinks
    T foreach p {
        puts $p
    }
    T destroy
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

ftp

ftp
Stichwörter: ftp, internet, net, rfc 959

Name
ftp – Clientseitige TCL-Implementierung des FTP-Protokolls

Übersicht
package require Tcl 8.5 9
package require ftp ?**2.4.14**?
::ftp::Open *server user passwd* ?*options*?
::ftp::Close *handle*
::ftp::Cd *handle directory*
::ftp::Pwd *handle*
::ftp::Type *handle* ?**ascii|binary|tenex**?
::ftp::List *handle* ?*pattern*?
::ftp::NList *handle* ?*directory*?
::ftp::FileSize *handle file*
::ftp::ModTime *handle file*
::ftp::Delete *handle file*
::ftp::Rename *handle from to*
::ftp::Put *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?
::ftp::Append *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?
::ftp::Get *handle remote* ?(local | -variable *varname* | -channel *chan)?*
::ftp::Reget *handle remote* ?*local*? ?*from*? ?*to*?
::ftp::Newer *handle remote* ?*local*?
::ftp::MkDir *handle directory*
::ftp::RmDir *handle directory*
::ftp::Quote *handle arg1 arg2 ...*
::ftp::DisplayMsg *handle msg* ?*state*?

Beschreibung
Das Paket "ftp" stellt die Client-Seite des FTP-Protokolls bereit, wie es in RFC 959 (http://www.rfc-editor.org/rfc/rfc959.txt) festgelegt ist. Das Paket implementiert sowohl aktive (Standard) als auch passive FTP-Sitzungen.

Mit dem Befehl **::ftp::Open** wird eine neue FTP-Sitzung gestartet. Um eine bestehende FTP-Sitzung zu beenden, verwenden Sie **::ftp::Close**. Alle anderen Befehle dürfen ausschließlich innerhalb einer geöffneten FTP-Sitzung verwendet werden. Werden sie außerhalb dieses Kontexts eingesetzt, führen sie zu Fehlern. Das FTP-Paket enthält Befehle zur Bearbeitung von Dateien und Verzeichnissen auf Remote-Systemen. Um dieselben Vorgänge auf dem lokalen System durchzuführen, verwenden Sie in den Kern integrierte Befehle wie **cd** oder **file**.

Die Ausgabe des Pakets wird durch zwei Statusvariablen gesteuert: **::ftp::VERBOSE** und

::ftp::DEBUG. Wenn Sie **::ftp::VERBOSE** auf "1" setzen, wird das Paket dazu gezwungen, alle Antworten eines Remote-Servers anzuzeigen. Der Standardwert ist "0". Durch Setzen von **::ftp::DEBUG** auf "1" wird das Debugging aktiviert und das Paket dazu veranlasst, alle Rückgabecodes, Zustände, Zustandsänderungen und "echte" FTP-Befehle anzuzeigen. Der Standardwert ist "0".

Der Befehl **::ftp::DisplayMsg** dient dazu, die verschiedenen Meldungen aus der FTP-Sitzung anzuzeigen. Die Einstellung von **::ftp::VERBOSE** legt fest, ob dieser Befehl aufgerufen wird oder nicht. Die aktuelle Implementierung des Befehls nutzt das Paket **log** von tcllib, um die Meldungen an ihren endgültigen Speicherort zu schreiben. Dies bedeutet, dass das Verhalten von **::ftp::DisplayMsg** angepasst werden kann, ohne dessen Implementierung zu ändern. Für grundlegendere Änderungen ist es natürlich weiterhin möglich, die Implementierung durch die Anwendung zu überschreiben. Beachten Sie, dass die Standardimplementierung die Optionen **-output** bis **::ftp::Open** für einen sitzungsspezifischen Protokollbefehl berücksichtigt.

Achtung: Die Standardimplementierung protokolliert Fehlermeldungen wie alle anderen Meldungen auch. Wenn dieses Verhalten dahingehend geändert wird, dass stattdessen ein Fehler ausgelöst wird, ändert sich auch das Verhalten aller Befehle in der API. In einem solchen Fall geben sie keinen Fehlercode, wie unten beschrieben, zurück, sondern leiten den ausgelösten Fehler an den Aufrufer weiter.

API

::ftp::Open *server user passwd* ?*options*?

Dieser Befehl dient dazu, eine FTP-Sitzung zu starten, indem eine Steuerverbindung zum FTP-Server hergestellt wird. Für alle vom Aufrufer nicht angegebenen Optionen werden die Standardwerte verwendet.

Der Befehl nimmt als Parameter einen Hostnamen *server*, einen Benutzernamen *user* und ein Passwort *password* entgegen und gibt ein Sitzungs-Handle zurück, das eine ganze Zahl größer oder gleich "0" ist, wenn die Verbindung erfolgreich hergestellt wurde. Andernfalls wird "-1" zurückgegeben. Der Parameter *server* muss den Namen oder die Internetadresse (in Dezimalschreibweise mit Punkten) des FTP-Servers enthalten, mit dem eine Verbindung hergestellt werden soll. Die Parameter *user* und *passwd* müssen einen gültigen Benutzernamen und ein gültiges Passwort enthalten, um den Anmeldevorgang abzuschließen.

Die Optionen überschreiben einige Standardwerte oder legen spezielle Funktionen fest:

-blocksize *size*

Die Blockgröße wird bei der Datenübertragung verwendet. Es werden höchstens *size* Bytes auf einmal übertragen. Der Standardwert für diese Option beträgt 4096. Das Paket führt nach der Übertragung jedes Blocks die Auswertung des **-progress callback** für die jeweilige Sitzung durch.

-timeout *seconds*

Wenn *seconds* ungleich Null ist, legt **::ftp::Open** ein Timeout fest, das nach der angegebenen Anzahl von Sekunden eintritt. Der Standardwert beträgt 600.

-port *number*

Die Port-*number* legt einen alternativen Remote-Port auf dem FTP-Server fest, auf dem der FTP-Dienst läuft. Die meisten FTP-Dienste warten auf Anfragen für Verbindungen am Standardport 21. Manchmal, meist aus Gründen der Sicherheit, werden für FTP-Verbindungen andere Portnummern als 21 verwendet.

-mode *mode*
Die Option "*mode*" legt fest, ob eine Dateiübertragung im Modus **active** oder **passive** erfolgt. Im passiven Modus fordert der Client den FTP-Server auf, auf einem Datenport zu lauschen und auf die Verbindung zu warten, anstatt den Prozess selbst zu initiieren, wenn eine Datenübertragungsanforderung eingeht. Der passive Modus ist in der Regel eine Anforderung, wenn der Zugriff auf Server über eine Firewall erfolgt. Der Standardmodus ist **active**.

-progress *callback*
Die Rückruffunktion *callback* wird jedes Mal ausgeführt, wenn ein Datenblock übertragen wurde. Informationen zur Festlegung der Größe der übertragenen Blöcke finden Sie unter der Option **-blocksize**.

Bei der Auswertung von *callback* wird dem Skript ein Argument angehängt: die aktuell kumulierte Anzahl der bisher übertragenen Bytes.

-command *callback*
Durch die Angabe dieser Option wird die Verbindung in den asynchronen Modus versetzt. Der *callback* wird nach Abschluss eines Vorgangs ausgewertet. Während ein Vorgang ausgeführt wird, dürfen keine weiteren Vorgänge gestartet werden, bis eine Rückruffunktion für den aktuell ausgeführten Vorgang empfangen wurde.

Bei der Auswertung des *callback* werden dem Callback-Skript mehrere Argumente angehängt, nämlich das Schlüsselwort des abgeschlossenen Vorgangs sowie etwaige zusätzliche, für den Vorgang spezifische Argumente. Sollte während der Ausführung des Vorgangs ein Fehler aufgetreten sein, erhält die Rückruffunktion das Schlüsselwort **error**.

-output *callback*
Diese Option hat keinen Standardwert. Wird sie gesetzt, verwendet die Standardimplementierung von **::ftp::DisplayMsg** ihren Wert als Befehlspräfix, um alle internen Meldungen zu protokollieren. Die Rückruffunktion wird vor der Auswertung mit drei Argumenten angehängt: die ID der Sitzung, die Meldung selbst und der Verbindungsstatus, und zwar in dieser Reihenfolge.

::ftp::Close *handle*
Dieser Befehl beendet die angegebene FTP-Sitzung. Wenn gerade keine Dateiübertragung stattfindet, schließt der Server die Steuerverbindung sofort. Findet jedoch gerade eine Dateiübertragung statt, bleibt die Steuerverbindung offen, bis die Übertragung abgeschlossen ist. Sobald dies der Fall ist, schreibt der Server die Ergebnisantwort für die Übertragung in die Verbindung und schließt diese anschließend.

::ftp::Cd *handle directory*
Dieser Befehl ändert das aktuelle Arbeitsverzeichnis auf dem FTP-Server in ein festgelegtes Ziel-*directory*. Der Befehl gibt den Wert 1 zurück, wenn das aktuelle Arbeitsverzeichnis erfolgreich in das festgelegte Verzeichnis geändert wurde, oder den Wert 0, falls der Vorgang fehlschlägt. Das festgelegte Verzeichnis kann
- ein Unterverzeichnis des aktuellen Verzeichnisses,
- Zwei Punkte,**..** (als Kennzeichnung für das übergeordnete Verzeichnis des aktuellen Verzeichnisses)
- oder einen vollqualifizierten Pfad zu einem neuen Verzeichnis.

::ftp::Pwd *handle*
Dieser Befehl gibt den vollständigen Pfad des aktuellen Arbeitsverzeichnisses auf dem FTP-Server zurück oder im Fehlerfall eine leere Zeichenfolge.

::ftp::Type *handle* ?ascii|binary|tenex?
Dieser Befehl legt den FTP-Dateiübertragungstyp entweder auf **ascii**, **binary** oder **tenex** fest. Der Befehl gibt stets den aktuell eingestellten Typ zurück. Wird er ohne Angabe eines Typs aufgerufen, erfolgt keine Änderung.

Derzeit werden nur die Typen **ascii** und **binary** unterstützt. Es besteht eine vorläufige (Alpha-)Unterstützung für den Tenex-Modus. Der Typ **ascii** wird normalerweise verwendet, um Textdateien in ein Format zu konvertieren, das für Texteditoren auf der Plattform des Zielrechners geeignet ist. Dies betrifft hauptsächlich Zeilenendezeichen. Der Typ **binary** hingegen ermöglicht die unverfälschte Übertragung von Nicht-Text-Dateien, wie beispielsweise komprimierte Dateien, Bilder und ausführbare Dateien.

::ftp::List *handle* ?*pattern*?
Dieser Befehl gibt eine für Menschen lesbare Liste von Dateien zurück. Platzhalterausdrücke wie "**.tcl*" sind zulässig. Wenn *pattern* auf ein bestimmtes Verzeichnis verweist, gibt der Befehl den Inhalt dieses Verzeichnisses zurück. Ist *pattern* kein vollständig qualifizierter Pfadname, listet der Befehl Einträge relativ zum aktuellen Remote-Verzeichnis auf. Wird kein *pattern* festgelegt, gibt der Befehl den Inhalt des aktuellen Remote-Verzeichnisses zurück.

Die Auflistung enthält alle systemabhängigen Informationen, die der Server bereitstellt. Beispielsweise erzeugen die meisten UNIX-Systeme eine Ausgabe des Befehls **ls -l**. Der Befehl gibt die abgerufenen Informationen als TCL-Liste mit einem Element pro Eintrag zurück. Leere Zeilen und die "Total"-Zeilen von UNIX werden ignoriert und nicht in das von diesem Befehl ausgegebene Ergebnis aufgenommen.

Wenn der Befehl fehlschlägt, wird eine leere Liste zurückgegeben.

::ftp::NList *handle* ?*directory*?
Dieser Befehl verhält sich genauso wie der Befehl **::ftp::List**, mit dem Unterschied, dass er lediglich eine gekürzte Auflistung gibt. Das bedeutet, dass in einer sortierten Liste ausschließlich Dateinamen angezeigt werden.

::ftp::FileSize *handle file*
Dieser Befehl gibt die Größe der angegebenen *file* auf dem FTP-Server zurück. Wenn der Befehl fehlschlägt, wird eine leere Zeichenkette zurückgegeben.

ACHTUNG! Im ASCII-Modus funktioniert dieser Befehl nicht ordnungsgemäß und wird nicht von allen FTP-Server-Implementierungen unterstützt.

::ftp::ModTime *handle file*
Dieser Befehl ruft den Zeitpunkt der letzten Änderung der *file* auf dem FTP-Server als systemabhängigen Ganzzahlwert in Sekunden ab oder gibt eine leere Zeichenkette zurück, wenn ein Fehler aufgetreten ist. Verwenden Sie den integrierten Befehl **clock**, um den abgerufenen Wert in andere Formate zu konvertieren.

::ftp::Delete *handle file*
Dieser Befehl löscht die angegebene *file* auf dem FTP-Server. Der Befehl gibt den Wert 1

zurück, wenn die angegebene Datei erfolgreich gelöscht wurde, bzw. den Wert 0, wenn der Vorgang fehlgeschlagen ist.

::ftp::Rename *handle from to*

Dieser Befehl benennt die Datei *from* im aktuellen Verzeichnis des FTP-Servers in den angegebenen neuen Dateinamen *to* um. Dieser neue Dateiname darf nicht mit einem bereits vorhandenen Unterverzeichnis oder Dateinamen übereinstimmen. Der Befehl gibt den Wert 1 zurück, wenn die angegebene Datei erfolgreich umbenannt wurde, oder den Wert 0, wenn der Vorgang fehlgeschlagen ist.

::ftp::Put *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?

Dieser Befehl überträgt eine lokale Datei *local* in eine Remote-Datei *remote* auf dem FTP-Server. Wenn es sich bei den an den Befehl übergebenen Dateiparametern nicht um vollständig qualifizierte Pfadnamen handelt, verwendet der Befehl das aktuelle Verzeichnis auf dem lokalen und dem Remote-Host. Wenn der Name der Remote-Datei nicht angegeben ist, verwendet der Server den Namen der lokalen Datei als Namen der Remote-Datei. Der Befehl gibt 1 zurück, um eine erfolgreiche Übertragung anzuzeigen, und 0 im Falle eines Fehlers.

Wenn anstelle einer lokalen Datei **-data** *data* festgelegt wird, überträgt das System keine Datei, sondern die an den Befehl übergebenen *data*. In diesem Fall muss der Name der Remote-Datei festgelegt werden.

Wenn anstelle einer lokalen Datei "**-channel** *chan*" festgelegt wird, überträgt das System keine Datei, sondern liest den Inhalt des Kanals *chan* und schreibt diesen in die Remote-Datei. In diesem Fall muss der Name der Remote-Datei festgelegt werden. Nach der Übertragung wird *chan* geschlossen.

::ftp::Append *handle* (*local* | -data *data* | -channel *chan*) ?*remote*?

Dieser Befehl verhält sich wie **::ftp::Puts**, fügt die übertragenen Informationen jedoch an die Remote-Datei an. Wenn die Datei auf dem Server noch nicht existierte, wird sie angelegt.

::ftp::Get *handle remote* ?(*local* | -variable *varname* | -channel *chan*)?

Dieser Befehl ruft eine Remote-Datei *remote* auf dem FTP-Server ab und speichert deren Inhalt in der lokalen Datei *local*. Wenn die an den Befehl übergebenen Dateiparameter keine vollständig qualifizierten Pfadnamen sind, verwendet der Befehl das aktuelle Verzeichnis auf dem lokalen und dem Remote-Host. Wird der Name der lokalen Datei nicht angegeben, verwendet der Server den Namen der Remote-Datei als Namen der lokalen Datei. Der Befehl gibt 1 zurück, um eine erfolgreiche Übertragung anzuzeigen, und 0 im Falle eines Fehlers. Der Befehl löst einen Fehler aus, wenn das Verzeichnis, in dem die Datei *local* platziert werden soll, nicht existiert.

Wenn **-variable** *varname* festgelegt wird, speichert das System die abgerufenen Daten in der Variablen *varname* anstelle einer Datei.

Wenn **-channel** *chan* festgelegt wird, schreibt das System die abgerufenen Daten anstelle einer Datei in den Kanal *chan*. Das System schließt *chan* nach der Übertragung *nicht*; dies liegt in der Verantwortung des Aufrufers von **::ftp::Get**.

::ftp::Reget *handle remote* ?*local*? ?*from*? ?*to*?

Dieser Befehl verhält sich wie **::ftp::Get**, mit dem Unterschied, dass, wenn die lokale Datei

local vorhanden und kleiner als die Remote-Datei *remote* ist, davon ausgegangen wird, dass es sich bei der lokalen Datei um eine teilweise übertragene Kopie der Remote-Datei handelt, und die Übertragung an der Stelle fortgesetzt wird, an der der Fehler offenbar aufgetreten ist. Der Befehl löst einen Fehler aus, wenn das Verzeichnis, in dem die Datei *local* platziert werden soll, nicht existiert. Dieser Befehl ist nützlich bei der Übertragung sehr großer Dateien über Netzwerke, bei denen es häufig zu Verbindungsabbrüchen kommt.

Durch die Angabe der zusätzlichen Byte-Offsets *from* und *to* ändert der Befehl sein Verhalten und lädt genau den angegebenen Ausschnitt der Remote-Datei herunter. Dieser Modus ist nur möglich, wenn ein lokales Ziel explizit angegeben wird. Wird *to* weggelassen, wird bis zum Dateiende heruntergeladen.

::ftp::Newer *handle remote* ?*local*?

Dieser Befehl verhält sich wie **::ftp::Get**, mit dem Unterschied, dass er die Remote-Datei nur dann abruft, wenn der Änderungszeitstempel der Remote-Datei aktueller ist als der der Datei auf dem lokalen System. Existiert die Datei auf dem lokalen System nicht, wird die Remote-Datei als neuer angesehen. Der Befehl löst einen Fehler aus, wenn das Verzeichnis, in dem die Datei *local* abgelegt werden soll, nicht existiert.

::ftp::MkDir *handle directory*

Dieser Befehl erstellt das angegebene *directory* auf dem FTP-Server. Wenn der angegebene Pfad relativ ist, wird das neue Verzeichnis als Unterverzeichnis des aktuellen Arbeitsverzeichnisses angelegt. Andernfalls erhält das erstellte Verzeichnis den angegebenen Pfadnamen. Der Befehl gibt den Wert 1 zurück, um die erfolgreiche Erstellung des Verzeichnisses anzuzeigen, und den Wert 0 im Falle eines Fehlers.

::ftp::RmDir *handle directory*

Dieser Befehl löscht das angegebene Verzeichnis auf dem FTP-Server. Das Remote-Verzeichnis muss leer sein, andernfalls schlägt der Befehl fehl. Der Befehl gibt den Wert 1 zurück, um eine erfolgreiche Löschung des Verzeichnisses anzuzeigen, und den Wert 0 im Falle eines Fehlers.

::ftp::Quote *handle arg1 arg2...*

Dieser Befehl dient dazu, einen beliebigen FTP-Befehl an den Server zu senden. Er kann nicht zum Abrufen einer Verzeichnisauflistung oder zur Übertragung von Dateien verwendet werden. Er ist enthalten, damit eine Anwendung Befehle auf dem FTP-Server ausführen kann, die von diesem Paket nicht bereitgestellt werden. Die Argumente werden wörtlich, d. h. unverändert, an den Server gesendet.

Im Gegensatz zu den anderen Befehlen in diesem Paket wertet dieser Befehl die vom FTP-Server erhaltene Antwort nicht aus, sondern gibt sie unverändert an den Aufrufer zurück.

::ftp::DisplayMsg *handle msg* ?*state*?

Dieser Befehl wird vom Paket selbst verwendet, um die verschiedenen Meldungen aus den FTP-Sitzungen anzuzeigen. Das Paket selbst implementiert diesen Befehl sehr einfach, indem es die Meldungen in "**stdout**" schreibt (wenn "**::ftp::VERBOSE**" gesetzt wurde, siehe unten) und bei Fehlermeldungen TCL-Fehler auslöst. Es liegt in der Verantwortung der Anwendung, diesen Befehl bei Bedarf zu überschreiben. Der Autor empfiehlt die Verwendung einer Variablen für verschiedene Zustände, denen unterschiedliche Farben zugewiesen sind. Das Paket **log** ist hierfür nützlich.

::ftp::VERBOSE

Eine Variable, die die Ausgabe des Pakets steuert. Wenn Sie **::ftp::VERBOSE** auf "1" setzen, wird das Paket dazu veranlasst, alle Antworten eines Remote-Servers anzuzeigen. Der Standardwert ist "0".

::ftp::DEBUG
Eine Statusvariable, die die Ausgabe von ftp steuert. Wenn Sie **::ftp::DEBUG** auf "1" setzen, wird das Debugging aktiviert und das Paket wird dazu veranlasst, alle Rückgabecodes, Zustände, Zustandsänderungen und "echte" FTP-Befehle anzuzeigen. Der Standardwert ist "0".

Bugs
Die korrekte Ausführung vieler Befehle hängt vom ordnungsgemäßen Verhalten des Remote-Servers sowie von der Netzwerk- und Router-Konfiguration ab.

Ein in der Prozedur **::ftp::DisplayMsg** platzierter Update-Befehl kann zu hartnäckigen Fehlern oder Endlosschleifen führen. Die Lösung besteht darin, **update idletasks** anstelle von **update** zu verwenden.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ftp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

ftp_geturl
Stichwörter: ftp_geturl, ftp, internet, net, rfc 959

Name
ftp::geturl – URI-Handler für FTP-URLs

Übersicht
package require Tcl 8.5 9
package require ftp::geturl ?**0.2.3**?
::ftp::geturl *url*

Beschreibung
Dieses Paket stellt einen Befehl bereit, der die clientseitige Implementierung des *ftp*-Protokolls, das vom Paket **ftp** bereitgestellt wird, umsetzt, um das Abrufen von URLs unter Verwendung des *ftp*-Schemas zu ermöglichen.

API
::ftp::geturl *url*

Dieser Befehl kann vom generischen Befehl **::uri::geturl** (siehe Paket **uri**) verwendet werden, um den Inhalt von FTP-URLs abzurufen. Intern nutzt er die Befehle des Pakets **ftp**, um die Anfrage zu bearbeiten.

Der Inhalt einer *ftp*-URL ist wie folgt definiert:

file
Der Inhalt der angegebenen Datei selbst.

directory
Eine Auflistung des Inhalts des Verzeichnisses in Schlüssel-Wert-Notation, wobei der Dateiname den Schlüssel und die Attribute den zugehörigen Wert bilden.

link
Die Attribute des Links, einschließlich des Pfads, auf den er verweist.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ftp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

ftpd

ftpd

Stichwörter: ftpd, ftp, ftpserver, rfc 959, services

Name

ftpd – Tcl-basierte FTP-Server-Implementierung

Übersicht

package require Tcl 8.5 9
package require ftpd ?**1.4.1**?
::ftpd::server ?*myaddr*?
::ftpd::config ?*option value*? ?*option value* ...?
fsCmd **append** *path*
fsCmd **delete** *path channel*
fsCmd **dlist** *path style channel*
fsCmd **exists** *path*
fsCmd **mkdir** *path channel*
fsCmd **mtime** *path channel*
fsCmd **permissions** *path*
fsCmd **rename** *path newpath channel*

fsCmd **retr** *path*
fsCmd **rmdir** *path channel*
fsCmd **size** *path channel*
fsCmd **store** *path*

Beschreibung

Das Paket **ftpd** stellt eine einfache, ausschließlich auf Tcl basierende Serverbibliothek für das FTP-Protokoll bereit, wie es in RFC 959 (http://www.rfc-editor.org/rfc/rfc959.txt) festgelegt ist. Es lauscht auf dem üblichen FTP-Socket. Die meisten Serverfehler werden als Fehlermeldungen zurückgegeben, mit dem passenden Code daran. Da der Servercode des FTP-Daemons in der Ereignisschleife läuft, kann auf dem Server ein **bgerror** ausgelöst werden, wenn der Code im Modul Probleme macht.

Befehle

::ftpd::server ?*myaddr*?
Öffnet einen lauschenden Socket, der FTP-Verbindungen annimmt. *myaddr* ist ein optionales Argument. *myaddr* ist der Name in Domain-Schreibweise oder die numerische IP-Adresse der clientseitigen Netzschnittstelle, die für die Verbindung verwendet wird.

::ftpd::config ?*option value*? ?*option value ...*?
Der Wert ist immer der Name des Befehls, der als Rückruffunktion aufgerufen wird. Die Option legt fest, welche Rückruffunktion konfiguriert wird. Argumente und Rückgabewerte der einzelnen Rückruffunktionen stehen im Abschnitt Callbacks.

-authIpCmd *proc*
Rückruffunktion, die neue Verbindungen anhand der IP-Adresse der Gegenstelle authentifiziert.

-authUsrCmd *proc*
Rückruffunktion, die neue Verbindungen anhand des Benutzers authentifiziert, der sich anmeldet, und anhand des Benutzerpassworts.

-authFileCmd *proc*
Rückruffunktion zur Gewährung oder Verweigerung des Lese- und Schreibzugriffs eines Benutzers auf einen bestimmten Pfad oder eine bestimmte Datei.

-logCmd *proc*
Rückruffunktion für Log-Meldungen, die die FTP-Engine erzeugt.

-fsCmd *proc*
Rückruffunktion, die die Engine mit dem Dateisystem verbindet, auf dem sie arbeitet.

-closeCmd *proc*
Rückruffunktion, die aufgerufen wird, wenn eine Verbindung geschlossen wird. Dies ermöglicht es der einbettenden Anwendung, eigene Aufräumvorgänge durchzuführen.

-xferDoneCmd *proc*
Rückruffunktion für die Meldung, dass eine Übertragung abgeschlossen ist. Sie wird aufgerufen, sobald eine Datenübertragung zum oder vom Client abgeschlossen ist.

Callbacks

authIpCmd callback
authIpCmd erhält als Argument die IP-Adresse der Gegenstelle, die sich mit dem FTP-Server

verbinden will. Sie gibt 1 zurück, damit Benutzer von der angegebenen IP einen Anmeldeversuch unternehmen dürfen, und 0, um den Anmeldeversuch von dieser IP abzulehnen.

authUsrCmd callback

authUsrCmd erhält Benutzername und Passwort als seine zwei Argumente. Es gibt 1 zurück, um den Anmeldeversuch am ftpd anzunehmen, und 0, um ihn abzulehnen.

authFileCmd callback

authFileCmd erhält drei Argumente: den Benutzer (der gerade angemeldet ist), den Pfad oder Dateinamen, der gelesen oder geschrieben werden soll, und **read** oder **write**. Es gibt 1 zurück, damit der Pfad oder Dateiname gelesen oder geschrieben werden darf, und 0, um den Lese- oder Schreibversuch mit einem Berechtigungsfehlercode abzulehnen.

logCmd callback

logCmd erhält einen Schweregrad und eine Meldung als seine zwei Argumente. Die Schweregrade im Paket ftpd sind **note**, **debug** und **error**. logCmd gibt nichts zurück.

fsCmd callback

fsCmd erhält einen Unterbefehl, einen Dateinamen oder Pfad und je nach Unterbefehl weitere Argumente.

fsCmd unterstützt diese Unterbefehle:

fsCmd* append *path

Der Unterbefehl append erhält als Argument den Dateinamen, an den angehängt wird. Er gibt einen beschreibbaren Tcl-Kanal zurück.

fsCmd* delete *path channel

Der Unterbefehl "delete" erhält als zwei Argumente den Dateinamen der zu löschenden Datei und einen Kanal, in den geschrieben werden soll. Die angegebene Datei wird gelöscht, und die entsprechende FTP-Meldung wird in den Kanal geschrieben, der als zweites Argument übergeben wird. Der Unterbefehl "delete" gibt keinen Wert zurück.

fsCmd* dlist *path style channel

Der Unterbefehl dlist erhält drei Argumente: den Pfad, dessen Dateien aufzulisten sind, den Stil der Auflistung, entweder **nlst** oder **list**, und einen Kanal, auf den geschrieben wird. Die Dateien im angegebenen Pfad werden zeilenweise auf den angegebenen Kanal geschrieben. Ist der Stil **nlst**, wird nur der Name der Datei auf den Kanal geschrieben. Ist der Stil **list**, werden die Dateirechte, die Anzahl der Links auf die Datei, der Name des Benutzers, dem die Datei gehört, der Name der Gruppe, der die Datei gehört, die Größe der Datei in Byte, die Änderungszeit der Datei und der Dateiname in einem formatierten, durch Leerzeichen getrennten Format auf den Kanal geschrieben. Der Unterbefehl **dlist** gibt nichts zurück.

fsCmd* exists *path

Der Unterbefehl "exists" nimmt als einziges Argument den Namen einer Datei entgegen, deren Existenz überprüft werden soll. Der Unterbefehl "exists" gibt den Wert 1 zurück, wenn der angegebene Pfad existiert und es sich dabei nicht um ein Verzeichnis handelt.

fsCmd* mkdir *path channel

Der Unterbefehl "mkdir" erhält als zwei Argumente den Pfad eines zu erstellenden Verzeichnisses und einen Kanal, in den geschrieben werden soll. Der Unterbefehl "mkdir" erstellt das angegebene Verzeichnis, sofern dies erforderlich und möglich ist. Anschließend gibt der Unterbefehl "mkdir" die entsprechende Erfolgsmeldung oder Fehlermeldung an

den Kanal aus. Der Unterbefehl "mkdir" gibt keinen Wert zurück.

fsCmd* mtime *path channel
Der Unterbefehl "mtime" erhält als zwei Argumente den Pfad einer Datei, deren Änderungszeit überprüft werden soll, sowie einen Kanal. Wenn die Datei vorhanden ist, wird die mtime im korrekten FTP-Format an den Kanal ausgegeben; andernfalls werden eine entsprechende Fehlermeldung und ein Fehlercode an den Kanal ausgegeben. Der Unterbefehl "mtime" gibt keinen Wert zurück.

fsCmd* permissions *path
Der Unterbefehl "permissions" erwartet den Pfad einer Datei, deren Zugriffsrechte abgerufen werden sollen. Der Unterbefehl "permissions" gibt die oktalen Dateizugriffsrechte der angegebenen Datei zurück. Es wird vorausgesetzt, dass die Datei vorhanden ist.

fsCmd* rename *path newpath channel
Der Unterbefehl "rename" erhält als drei Argumente den Pfad der aktuellen Datei, den neuen Dateipfad und einen Kanal, in den geschrieben werden soll. Der Unterbefehl "rename" benennt die aktuelle Datei in den neuen Dateipfad um, wenn der Pfad zur neuen Datei existiert, und gibt anschließend die entsprechende Meldung über den Kanal aus. Existiert der neue Dateipfad nicht, wird die entsprechende Fehlermeldung über den Kanal ausgegeben. Der Unterbefehl "rename" gibt keinen Wert zurück.

fsCmd* retr *path
Der Unterbefehl "retr" erhält als einziges Argument den Pfad einer Datei, die gelesen werden soll. Der Unterbefehl "retr" gibt einen lesbaren Kanal zurück, über den die angegebene Datei gelesen werden kann.

fsCmd* rmdir *path channel
Der Unterbefehl "rmdir" erhält als zwei Argumente den Pfad eines zu löschenden Verzeichnisses und einen Kanal, in den geschrieben werden soll. Der Unterbefehl "rmdir" löscht das angegebene Verzeichnis (sofern möglich) und gibt die entsprechende Meldung an den Kanal aus (wobei es sich um eine Fehlermeldung handeln kann, falls das angegebene Verzeichnis nicht existiert oder nicht leer ist). Der Unterbefehl "rmdir" gibt keinen Wert zurück.

fsCmd* size *path channel
Der Unterbefehl "size" erhält als zwei Argumente den Pfad einer Datei, deren Größe (in Byte) ermittelt werden soll, sowie einen Kanal, in den die Ausgabe geschrieben werden soll. Der Unterbefehl "size" gibt den entsprechenden Code und die Größe der Datei aus, wenn es sich bei dem angegebenen Pfad um eine Datei handelt; andernfalls werden ein entsprechender Fehlercode und eine Fehlermeldung an den Kanal ausgegeben. Der Unterbefehl "size" gibt keinen Wert zurück.

fsCmd* store *path
Der Unterbefehl "store" erhält als einziges Argument den Pfad einer Datei, in die geschrieben werden soll. Der Unterbefehl "store" gibt einen beschreibbaren Kanal zurück.

closeCmd
Die Funktion **closeCmd** erhält beim Aufruf keine Argumente, und ein eventuell von ihr generierter Wert wird verworfen.

xferDoneCmd sock sock2 file bytes filename err

xferDoneCmd erhält beim Aufruf sechs Argumente. Das sind, in dieser Reihenfolge, das Kanal-Handle des Steuer-Sockets der Verbindung, das Kanal-Handle des Daten-Sockets der Übertragung (bereits geschlossen), das Handle des Kanals mit der übertragenen Datei, die Anzahl der übertragenen Bytes, der Pfad der übertragenen Datei und eine (möglicherweise leere) Fehlermeldung. Ein Rückgabewert wird verworfen.

Variablen

::ftpd::cwd
Das aktuelle Arbeitsverzeichnis einer Sitzung, wenn sich jemand zum ersten Mal mit dem FTPD verbindet oder der FTP-Befehl **REIN** eintrifft.

::ftpd::contact
Die E-Mail-Adresse der Person, die Kontakt für den FTP-Server ist. Diese Adresse wird als Teil der Antwort auf den Befehl **FTP HELP** ausgegeben.

::ftpd::port
Der Port, auf dem der FTP-Server lauschen soll. Ist der Port als Null angegeben, teilt das Betriebssystem einen ungenutzten Port als Server-Socket zu; danach enthält die Variable die zugeteilte Portnummer.

::ftpd::welcome
Die Meldung, die ausgegeben wird, wenn der Benutzer sich zum ersten Mal mit dem FTP-Server verbindet.

::ftpd::CurrentSocket
Für alle Rückruffunktionen und alle Dateisystembefehle zugänglich, die eine besondere Form der Rückruffunktion sind, und enthält das Handle des Socket-Kanals, der beim Aufruf der Rückruffunktion aktiv war.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ftpd* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

fumagic

cfront

Stichwörter: cfront, file recognition, file type, file utilities, mime, type

Name

fileutil::magic::cfront – Generatorkern für den Compiler von magic(5)-Dateien

Übersicht

package require Tcl 8.5 9
package require fileutil::magic::cfront ?**1.3.3**?
package require fileutil::magic::cgen ?**1.2.0**?
package require fileutil::magic::rt ?**1.2.0**?
package require struct::list
package require fileutil
::fileutil::magic::cfront::compile *path...*
::fileutil::magic::cfront::generate ?*compressed*?
boolean? -- *path...*
::fileutil::magic::cfront::install *path...*

Beschreibung

Dieses Paket stellt das Frontend eines Compilers bereit, der magic(5)-Dateien in Erkenner auf der Grundlage des Erkenner-Laufzeitpakets **fileutil::magic::rt** umwandelt. Zum Generator-Backend dieses Compilers siehe das Paket **fileutil::magic::cgen**.

Befehle

::fileutil::magic::cfront::compile *path*...
Dieser Befehl nimmt die Pfade einer oder mehrerer Dateien und Verzeichnisse und kompiliert alle diese Dateien sowie die Dateien in allen Verzeichnissen zu einem einzigen Analysator für alle in diesen Dateien angegebenen Dateitypen. Er gibt eine Liste zurück, deren erstes Element eine Liste von Dictionarys je Datei mit Analysatorskripten ist und deren zweites Element eine Liste von Analysatorbefehlen.

Alle Dateien müssen im Format von magic(5) vorliegen.

Das Ergebnis des Befehls ist ein Tcl-Skript, das den erzeugten Erkenner enthält.

::fileutil::magic::cfront::generate ?*compressed*
boolean? -- *path*... Dieser Befehl verhält sich hinsichtlich der angegebenen *path*-Argumente wie **::fileutil::magic::cfront::compile**, packt das entstehende Erkenner-Skript dann in eine Variable namens **named** und eine Prozedur namens **analyzer** und gibt das entstehende Skript zurück.

Das Argument *compressed* hat die Vorgabe **boolean true**, das heißt, Leerzeichen im zurückgegebenen Skript werden minimiert.

Das Argument -- ist immer erforderlich.

::fileutil::magic::cfront::install *path*...
Dieser Befehl verhält sich hinsichtlich der angegebenen Pfadargumente wie **::fileutil::magic::cfront::compile**. Er erzeugt mit **::fileutil::magic::cfront::generate** ein Erkenner-Skript und wertet dieses Skript im aktuellen Interpreter aus, damit der Befehl ::fileutil::magic::filetype es verwendet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden

oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil :: magic* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

cgen

Stichwörter: cgen, file recognition, file type, file utilities, mime, type

Name

fileutil::magic::cgen – Generatorkern für den Compiler von magic(5)-Dateien

Übersicht

package require Tcl 8.5 9
package require fileutil::magic::cgen ?**1.2.0**?
package require fileutil::magic::rt ?**1.2.0**?
package require struct::tree
package require struct::list
::fileutil::magic::cgen::2tree *script*
::fileutil::magic::cgen::treedump *tree*
::fileutil::magic::cgen::treegen *tree node*

Beschreibung

Dieses Paket stellt das Generator-Backend eines Compilers bereit, der magic(5)-Dateien in Erkenner auf der Grundlage des Erkenner-Laufzeitpakets **fileutil::magic::rt** umwandelt. Zum Compiler-Frontend, das diesen Generator verwendet, siehe das Paket **fileutil::magic::cfront**.

Befehle

::fileutil::magic::cgen::2tree *script*
: Dieser Befehl wandelt den durch das *script* angegebenen Erkenner in einen Baum um und gibt den Objektbefehl dieses Baums als Ergebnis zurück. Der Baum kommt aus dem Paket **struct::tree**.

  Das *script* hat das in magic(5) festgelegte Format.

::fileutil::magic::cgen::treedump *tree*
: Dieser Befehl nimmt einen von **::fileutil::magic::cgen::2tree** erzeugten *tree* und gibt eine Zeichenkette zurück, die den Baum für Menschen lesbar kodiert, als Hilfe bei der Fehlersuche.

::fileutil::magic::cgen::treegen *tree node*
: Dieser Befehl nimmt einen von **::fileutil::magic::cgen::2tree** erzeugten *tree* und gibt ein Tcl-Skript zurück, den Erkenner für die Dateitypen, die der am *node* gewurzelte Teilbaum darstellt. Das erzeugte Skript verwendet ausgiebig die Befehle des Erkenner-Laufzeitpakets **fileutil::magic::rt**.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil :: magic* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

filetypes

Stichwörter: filetypes, file recognition, file type, file utilities, type

Name

fileutil::magic::filetype – Prozeduren, die die Erkennung von Dateitypen implementieren

Übersicht

package require Tcl 8.6 9
package require fileutil::magic::filetype ?**2.0**?
::fileutil::magic::filetype *filename*

Beschreibung

Dieses Paket enthält einen Befehl zur Erkennung von Dateitypen in reinem Tcl.

Der Kern des Erkenners wurde aus einer "magic(5)"-Datei erzeugt, die die Prüfungen zur Erkennung von Dateien und die zugehörigen Dateitypen enthält.

Achtung! Dieser Erkenner ist groß, etwa 752 Kilobyte erzeugter Tcl-Code.

::fileutil::magic::filetype *filename*
: Dieser Befehl ähnelt dem Befehl **fileutil::fileType**.

  Er gibt eine Liste zurück, darin eine Liste von Beschreibungen, eine Liste von mimetype-Bestandteilen und eine Liste von Dateiendungen. Er gibt die leere Zeichenkette zurück, wenn der Dateiinhalt nicht erkannt wird.

Literatur

1. File(1) sources Diese Seite enthält die aktuellen Quellen des Befehls file, einschließlich der magic-Definitionen, die er verwendet. Letztere wurden von uns verwendet, um diesen Erkenner zu erzeugen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil :: magic* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

rtcore
Stichwörter: rtcore, file recognition, file type, file utilities, mime, type

Name
fileutil::magic::rt – Laufzeit-Kern für Engines zur Dateityperkennung, geschrieben in reinem Tcl

Übersicht
package require Tcl 8.5 9
package require fileutil::magic::rt ?**2.0**?
::fileutil::magic::rt::>
::fileutil::magic::rt::<
::fileutil::magic::rt::new *chan named analyze*
::fileutil::magic::rt::file_start *name*
::fileutil::magic::rt::emit *msg*
::fileutil::magic::rt::O *where*
::fileutil::magic::rt::R *where*
::fileutil::magic::rt::Nv *type offset compinvert comp expected*
::fileutil::magic::rt::N *type offset testinvert compinvert mod mand comp expected*
::fileutil::magic::rt::S *type offset testinvert mod mand comp val*
::fileutil::magic::rt::L *newlevel*
::fileutil::magic::rt::I *offset it ioi ioo iir io*
::fileutil::magic::rt::R *offset*
::fileutil::magic::rt::U *fileindex name*

Beschreibung
Dieses Paket stellt den Laufzeitkern für in reinem Tcl geschriebene Engines zur Dateityperkennung bereit und wird daher von allen anderen Paketen in diesem Modul verwendet, wie beispielsweise **fileutil::magic::filetype** und den beiden Compiler-Paketen **fileutil::magic::cgen** und **fileutil::magic::cfront**.

Befehle
::fileutil::magic::rt::>
Erhöht die Ebene und erledigt die zugehörige Verwaltung.

::fileutil::magic::rt::<
Verringert die Ebene und erledigt die zugehörige Verwaltung.

::fileutil::magic::rt::new *chan named analyze*
Erzeugt einen neuen Befehl, der bei jedem Aufruf eine Beschreibung der Datei zurückgibt und, wenn es keine weiteren Beschreibungen gibt, den Code *break*. *chan* ist der Kanal mit den zu beschreibenden Daten. Die Kanalkonfiguration wird danach nach Bedarf verwaltet. *named* ist ein Dictionary benannter Tests, wie es **fileutil::magic::cfront::compile** erzeugt. *analyze*

ist ein Befehlspräfix für eine Routine aus der Befehlsliste, die **fileutil::magic::cfront::compile** zurückgibt.

::fileutil::magic::rt::file_start *name*
Dieser Befehl markiert bei der Fehlersuche den Anfang einer magic-Datei. Er gibt die leere Zeichenkette zurück.

::fileutil::magic::rt::emit *msg*
Dieser Befehl fügt den Text *msg* dem Ergebnispuffer hinzu. Die Meldung kann die folgenden besonderen Zeichenfolgen enthalten. Sie werden durch gepufferte Werte ersetzt, bevor die Meldung dem Ergebnis hinzugefügt wird. Der Befehl gibt die leere Zeichenkette zurück.

\b
Diese Folge wird entfernt.

%s
Wird durch den zuletzt gepufferten Zeichenkettenwert ersetzt.

%ld
Wird durch den zuletzt gepufferten numerischen Wert ersetzt.

%d
Siehe oben.

${x:...?...}
Setzt eine Zeichenkette ein, wenn die Datei ausführbar ist, und sonst eine andere.

::fileutil::magic::rt::O *where*
Erzeugt einen Versatz aus *where*, relativ zum Cursor eine Ebene höher. Erzeugt einen Versatz aus *where*, relativ zum Versatz eine Ebene höher.

::fileutil::magic::rt::Nv *type offset compinvert comp expected*
Eine eingeschränkte Form von **::fileutile::magic::rt::N**, die nur auf Gleichheit prüft und bei der sich der Test nicht umkehren lässt.

::fileutil::magic::rt::N *type offset testinvert compinvert mod mand comp expected*
Ruft den numerischen Wert mit dem Typ *type* an der absoluten Position *offset* ab, vergleicht ihn mit *expected* unter Verwendung von *comp* als Vergleichsoperator und gibt das Ergebnis zurück.

Das Argument *comp* muss einer der Vergleichsoperatoren von Tcl sein.

```
    <comp> <fetched-and-masked-value> <comp> <expected>
```

Der spezielle Vergleichsoperator **x** signalisiert, dass kein Vergleich durchgeführt werden soll, oder, mit anderen Worten, dass der abgerufene Wert stets mit *val* übereinstimmt.

::fileutil::magic::rt::S *type offset testinvert mod mand comp val*
Wie **::fileutil::magic::rt::N**, nur dass Zeichenkettentypen geholt und verglichen werden, keine numerischen Daten.

::fileutil::magic::rt::L *newlevel*
Setzt die aktuelle Ebene im aufrufenden Kontext auf *newlevel*. Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::fileutil::magic::rt::I *offset it ioi ioo iir io*
Berechnet einen Versatz auf der Grundlage eines Anfangsversatzes und der angegebenen Modifikatoren.

::fileutil::magic::rt::R *offset*
Berechnet ausgehend von einem anfänglichen Versatz einen Versatz relativ zum Cursor auf der nächsthöheren Ebene. Der Cursor ist die Position in den Daten, die ein Zeichen nach den aus der Datei auf der nächsthöheren Ebene extrahierten Daten liegt.

::fileutil::magic::rt::U *fileindex name*
Fügt eine Ebene hinzu und verwendet ein benanntes Testskript.

Zahlentypen

byte
8-Bit-Ganzzahl

short
16-Bit-Ganzzahl, gespeichert in nativer Byte-Reihenfolge

beshort
siehe oben, gespeichert in Big-Endian

leshort
siehe oben, gespeichert in Small-/Little-Endian

long
32-Bit-Ganzzahl, gespeichert in nativer Byte-Reihenfolge

belong
siehe oben, gespeichert in Big-Endian

lelong
siehe oben, gespeichert in Small-/Little-Endian.

Alle oben genannten Typen gibt es auch vorzeichenlos. Die Typnamen sind dieselben, mit dem Zeichen "u" als Präfix.

date
32-Bit-Ganzzahl als Zeitstempel, gespeichert in nativer Byte-Reihenfolge

bedate
siehe oben, gespeichert in Big-Endian

ledate
siehe oben, gespeichert in Small-/Little-Endian

ldate
32-Bit-Ganzzahl als Zeitstempel, gespeichert in nativer Byte-Reihenfolge

beldate
siehe oben, gespeichert in Big-Endian

leldate
siehe oben, gespeichert in Small-/Little-Endian

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *fileutil :: magic* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

generator

generator
Stichwörter: generator, control structure, coroutine, filter, foldl, foldr, foreach, iterator, map, reduce, scanl

Name
generator – Prozeduren zum Erstellen und Verwenden von Generatoren.

Übersicht
package require Tcl 8.6 9
package require generator ?**0.3**?
generator define *name params body*
generator yield *arg* ?*args..*?
generator foreach *varList generator varList generator* ?*...*? *body*
generator next *generator* ?*varName..*?
generator exists *generator*
generator names
generator destroy ?*generator..*?
generator finally *cmd* ?*arg..*?
generator from *format value*
generator to *format generator*
generator map *function generator*
generator filter *predicate generator*
generator reduce *function zero generator*
generator foldl *function zero generator*
generator foldr *function zero generator*
generator all *predicate generator*
generator and *generator*
generator any *generator*
generator concat *generator* ?*generator..*?
generator concatMap *function generator*
generator drop *n generator*
generator dropWhile *predicate generator*
generator contains *element generator*
generator foldl1 *function generator*
generator foldli *function zero generator*
generator foldri *function zero generator*
generator head *generator*
generator tail *generator*

generator init *generator*
generator takeList *n generator*
generator take *n generator*
generator iterate *function init*
generator last *generator*
generator length *generator*
generator or *predicate generator*
generator product *generator*
generator repeat *n value..*
generator sum *generator*
generator takeWhile *predicate generator*
generator splitWhen *predicate generator*
generator scanl *function zero generator*

Beschreibung

Das Paket **generator** stellt Befehle zur Definition und Durchlaufung von Generatorausdrücken bereit. Ein *Generator* ist ein Befehl, der eine Folge von Werten gibt. Im Gegensatz zu einem gewöhnlichen Befehl, der eine Liste gibt, *gibt* ein Generator jedoch jeden Wert einzeln aus und wird anschließend unterbrochen, sodass nachfolgende Werte bei Bedarf abgerufen werden können. Auf diese Weise lassen sich Generatoren nutzen, um eine Reihe von Werten effizient zu durchlaufen, ohne alle Ergebnisse im Arbeitsspeicher generieren zu müssen. Generatoren können verwendet werden, um Elemente einer Datenstruktur oder Zeilen in der Ergebnismenge einer Datenbankabfrage zu durchlaufen, um Produzenten- und Konsumenten-Softwareentwürfe wie Parser und Tokenizer voneinander zu entkoppeln oder um ausgefeilte benutzerdefinierte Steuerungsstrategien wie die Backtracking-Suche zu implementieren. Generatoren verringern die Notwendigkeit, benutzerdefinierte Kontrollstrukturen zu implementieren, da viele dieser Strukturen als Generatoren umgestaltet werden können, was sowohl zu einer einfacheren Implementierung als auch zu einer standardisierteren Schnittstelle führt. Der Generatormechanismus baut auf dem Coroutinen-Mechanismus von Tcl 8.6 auf.

Das Paket exportiert einen einzigen Ensemble-Befehl: **generator**. Alle Funktionen werden als Unterbefehle dieses Befehls bereitgestellt. Die zentralen Unterbefehle des Pakets sind **define**, **yield** und **foreach**. Der Befehl **define** funktioniert wie der Tcl-Befehl **proc**, erstellt jedoch eine Generator-Prozedur, d. h. eine Prozedur, die bei Aufruf einen Generator gibt. Der Generator selbst ist ein Befehl, der mehrfach aufgerufen werden kann: Bei jedem Aufruf gibt er den nächsten Wert der generierten Folge zurück. Wenn die Folge erschöpft ist, gibt der Generatorbefehl eine leere Liste zurück und löscht sich anschließend selbst. Anstatt einen Generator manuell aufzurufen, bietet das Paket jedoch auch einen flexiblen Befehl **foreach**, der die Werte eines oder mehrerer Generatoren durchläuft. Diese Schleifenkonstruktion ahmt die Funktionalität des integrierten Tcl-Befehls **foreach** nach, einschließlich der Verarbeitung mehrerer Werte und der gleichzeitigen Durchlaufung mehrerer Generatoren. Das Schreiben eines Generators ist ebenfalls eine einfache Aufgabe, ähnlich wie das Schreiben einer normalen Prozedur: Verwenden Sie einfach den Befehl **define**, um den Generator zu definieren, und rufen Sie anschließend **yield** anstelle von **return** auf. Beispielsweise können wir einen Generator definieren, der die ganzen Zahlen in einem bestimmten Bereich durchläuft:

```
    generator define range {n m} {
        for {set i $n} {$i <= $m} {incr i} { generator yield $i }
    }
    generator foreach x [range 1 10] {
        puts "x = $x"
    }
```

Das obige Beispiel gibt, wie zu erwarten, die Zahlen von 1 bis 10 der Reihe nach aus. Der Unterschied zu einer normalen Schleife über eine Liste besteht darin, dass die Zahlen erst dann generiert werden, wenn sie benötigt werden. Wenn wir eine "break"-Anweisung in die Schleife einfügen, würden die verbleibenden Zahlen der Folge niemals generiert werden. Zur Veranschaulichung können wir einen Generator definieren, der die Folge der natürlichen Zahlen erzeugt: eine unendliche Reihe. Eine normale Prozedur würde beim Versuch, diese Reihe als Liste zu erzeugen, niemals zurückgeben. Durch die Verwendung eines Generators müssen wir nur jene Werte generieren, die tatsächlich verwendet werden:

```
    generator define nats {} {
        while 1 { generator yield [incr nat] }
    }
    generator foreach n [nats] {
        if {$n > 100} { break }
    }
```

Befehle

generator define *name params body*

Erstellt eine neue Generatorprozedur. Die Argumente des Befehls entsprechen denen von **proc**: ein *name*, eine Liste von Parametern und ein Hauptteil. Das Format der Parameterliste entspricht dem einer Prozedur. Insbesondere können Standardwerte und die "?args"-Syntax wie gewohnt verwendet werden. Bei jedem Aufruf der resultierenden Generator-Prozedur wird ein neuer Generator-Befehl (Koroutine) erstellt, der bei jedem Aufruf eine Liste von Werten liefert. Jedes Ergebnis eines Generators ist garantiert eine nicht leere Liste von Werten. Wenn ein Generator erschöpft ist, gibt er eine leere Liste zurück und löscht sich anschließend selbst, um Ressourcen freizugeben. Der Versuch, einen erschöpften Generator aufzurufen, führt zu einem Fehler, da der Befehl nicht mehr existiert.

generator yield *arg* ?*args..*?

Wird dieser Befehl bei der Definition eines Generators verwendet, gibt er die nächste Wertemenge an den Verbraucher zurück. Sobald der Befehl **yield** aufgerufen wurde, unterbricht der Generator seine Ausführung, damit der Verbraucher diesen Wert verarbeiten kann. Wenn der nächste Wert angefordert wird, setzt der Generator die Ausführung fort, als ob der Befehl "yield" gerade zurückgekehrt wäre, und kann die Verarbeitung fortsetzen, um das nächste Ergebnis auszugeben. Der Befehl **yield** muss mit mindestens einem Argument aufgerufen werden, kann jedoch auch mit mehreren Argumenten aufgerufen werden; in diesem Fall entspricht dies einem einmaligen Aufruf von **yield** für jedes Argument.

generator foreach *varList generator varList generator* ?...? *body*

Durchläuft einen oder mehrere Generatoren, weist den Variablen die nächsten Werte zu und führt anschließend den Schleifenkörper aus. Funktioniert ähnlich wie der integrierte Befehl

foreach, arbeitet jedoch mit Generatoren anstelle von Listen. Es können mehrere Generatoren parallel durchlaufen werden, und von einem einzelnen Generator können mehrere Ergebnisse gleichzeitig abgerufen werden. Wie beim integrierten Befehl **foreach** wird die Schleife so lange fortgesetzt, bis alle Generatoren durchlaufen wurden: Variablen für Generatoren, die vorzeitig erschöpft sind, werden auf die leere Zeichenkette gesetzt.

Der Befehl **foreach** bereinigt am Ende der Schleife automatisch alle Generatoren, unabhängig davon, ob die Schleife vorzeitig beendet wurde oder nicht. Dieses Verhalten dient der Benutzerfreundlichkeit, um in den üblichen Fällen eine explizite Bereinigung eines Generators zu vermeiden. Generatoren können jedoch vor dem Ende der Schleife zerstört werden; in diesem Fall läuft die Schleife wie gewohnt weiter, bis alle anderen Generatoren zerstört oder erschöpft sind.

Der Befehl **foreach** erstellt keinen Snapshot des Generators. Jegliche Änderungen am Zustand des Generators, die innerhalb der Schleife oder durch anderen Code vorgenommen werden, wirken sich auf den Zustand der Schleife aus. Insbesondere wenn der Code in der Schleife den Generator aufruft, um das nächste Element manuell abzurufen, wird dieses Element aus der Schleife ausgeschlossen, und die nächste Iteration setzt sich mit dem darauf folgenden Element fort. Es ist darauf zu achten, dass gleichzeitige Aktualisierungen von Generatoren vermieden werden, es sei denn, dieses Verhalten ist erforderlich (z. B. bei der Argumentverarbeitung).

generator next *generator* ?*varName..*?
Ruft die nächsten Werte manuell aus einem Generator ab. Für jede übergebene Variable wird ein Wert abgerufen und der entsprechenden Variablen zugewiesen. Wenn der Generator zu irgendeinem Zeitpunkt erschöpft ist, werden alle verbleibenden Variablen auf die leere Zeichenkette gesetzt.

generator exists *generator*
Gibt 1 zurück, wenn der Generator (noch) vorhanden ist, andernfalls 0.

generator names
Gibt eine Liste aller derzeit vorhandenen Generatorbefehle zurück.

generator destroy ?*generator..*?
Zerstört einen oder mehrere Generatoren und gibt alle damit verbundenen Ressourcen frei.

generator finally *cmd* ?*arg..*?
Wird dieser Befehl in der Definition einer Generator-Prozedur verwendet, sorgt er dafür, dass eine Ressource freigegeben wird, wenn der Generator zerstört wird – entweder explizit oder implizit, wenn der Generator erschöpft ist. Dieser Befehl kann wie ein **finally**-Block im Befehl **try** verwendet werden, mit dem Unterschied, dass er an den Lebenszyklus des Generators gebunden ist und nicht an einen bestimmten Geltungsbereich. Wenn wir beispielsweise einen Generator erstellen, um die Zeilen einer Textdatei zu durchlaufen, können wir mit **finally** sicherstellen, dass die Datei geschlossen wird, sobald der Generator zerstört wird:

```
    generator define lines file {
        set in [open $file]
        # Ensure file is always closed
        generator finally close $in
        while {[gets $in line] >= 0} {
```

```
            generator yield $line
        }
    }
    generator foreach line [lines /etc/passwd] {
        puts "[incr count]: $line"
        if {$count > 10} { break }
    }
    # File will be closed even on early exit
```

Wenn Sie einen Generator erstellen, der einen anderen Generator nutzt (wie beispielsweise die später definierten Standards **map** und **filter**), sollten Sie den Befehl **finally** verwenden, um sicherzustellen, dass dieser Generator zusammen mit seinem übergeordneten Generator zerstört wird. Der Generator **map** ist beispielsweise wie folgt definiert:

```
    generator define map {f xs} {
        generator finally generator destroy $xs
        generator foreach x $xs { generator yield [{*}$f $x] }
    }
```

generator from *format value*
Erstellt einen Generator aus einer Datenstruktur. Derzeit werden die Formate **list**, **dict** oder **string** unterstützt. Das Listenformat gibt jedes Element nacheinander zurück. Bei Dictionarys werden jeder Schlüssel und jeder Wert separat zurückgegeben. Schließlich werden Zeichenfolgen Zeichen für Zeichen zurückgegeben.

generator to *format generator*
Wandelt einen Generator in eine Datenstruktur um. Dies ist die umgekehrte Operation des Befehls **from** und unterstützt dieselben Datenstrukturen. Die beiden Operationen erfüllen die folgenden Identitätsgesetze (wobei **=** entsprechend interpretiert wird):

```
    [generator to $fmt [generator from $fmt $value]] = $value
    [generator from $fmt [generator to $fmt $gen]]   = $gen
```

Präludium

Die folgenden Befehle werden als Standardbibliothek mit Generator-Kombinatoren und Funktionen bereitgestellt, die praktische Operationen auf Generatoren ausführen. Die Funktionen in diesem Abschnitt orientieren sich lose an den entsprechenden Funktionen aus dem Haskell-Prelude. *Fehlersicht:* Die meisten Funktionen in diesem Prelude zerstören als Nebeneffekt alle ihnen übergebenen Generator-Argumente. Wenn Sie persistente Generatoren verwenden möchten, sehen Sie sich bitte die Streams-Bibliothek an.

generator map *function generator*
Wendet eine Funktion auf jedes Element eines Generators an und gibt einen neuen Generator mit den Ergebnissen zurück. Dies ist die klassische "map"-Funktion aus der funktionalen Programmierung, die hier auf Generatoren angewendet wird. So können wir beispielsweise mit dem folgenden Code alle Quadratzahlen generieren (wobei **nats** wie zuvor definiert ist):

```
proc square x { expr {$x * $x} }
generator foreach n [generator map square [nats]] {
    puts "n = $n"
    if {$n > 1000} { break }
}
```

generator filter *predicate generator*

Ein weiteres Juwel der funktionalen Programmierung. Dieser Befehl gibt einen Generator zurück, der nur jene Elemente aus dem Argumentgenerator liefert, die das Prädikat (eine boolesche Funktion) erfüllen. Hätten wir beispielsweise einen Generator namens **employees**, der einen Strom von Dictionarys gibt, die Personen darstellen, könnten wir mit einem einfachen Filter alle Personen herausfiltern, deren Gehalt über 100.000 Dollar (oder einer Währung Ihrer Wahl) liegt:

```
proc salary> {amount person} { expr {[dict get $person salary] >
$amount} }
set fat-cats [generator filter {salary> 100000} $employees]
```

generator reduce *function zero generator*

Dies ist die klassische "Left-Fold"-Operation. Dieser Befehl erwartet eine Funktion, einen Anfangswert und einen Generator von Werten. Für jedes Element im Generator wendet er die Funktion auf den aktuellen Akkumulatorwert (anfänglich das *zero*-Argument) und dieses Element an und verwendet anschließend das Ergebnis als neuen Akkumulatorwert. Dieser Prozess wird über den gesamten Generator hinweg (eager) wiederholt, und der endgültige Akkumulatorwert wird dann zurückgegeben. Betrachtet man die Funktion als binären Operator und das Null-Argument als das linke Identitätselement dieser Operation, so kann man den Befehl **reduce** als eine Art *Faltung* des Operators zwischen jedem aufeinanderfolgenden Wert-Paar im Generator in linksassoziativer Weise betrachten. Beispielsweise lässt sich die Summe einer Zahlenfolge berechnen, indem der Operator **+** zwischen den Zahlen angewendet wird, wobei 0 als Identitätselement dient:

```
# sum xs          = reduce + 0 xs
# sum [range 1 5] = reduce + 0 [range 1 5]
#                 = reduce + [+ 0 1] [range 2 5]
#                 = reduce + [+ 1 2] [range 3 5]
#                 = ...
#                 = reduce + [+ 10 5] <empty>
#                 = ((((0+1)+2)+3)+4)+5
#                 = 15
proc + {a b} { expr {$a + $b} }
proc sum gen { generator reduce + 0 $gen }
puts [sum [range 1 10]]
```

Die Operation **reduce** ist äußerst nützlich, und mit ihrer Hilfe lassen sich viele unterschiedliche Operationen definieren. So können wir beispielsweise mithilfe von Generatoren eine Fakultätsfunktion als Produkt eines Bereichs definieren. Diese Definition ist sowohl sehr übersichtlich als auch recht effizient (sowohl hinsichtlich des Speicherbedarfs als

auch der Laufzeit):

```
proc * {x y} { expr {$x * $y} }
proc prod gen { generator reduce * 0 $gen }
proc fac n { prod [range 1 $n] }
```

Obwohl die Operation **reduce** bei endlichen Generatoren effizient ist, sollte darauf geachtet werden, sie nicht auf einen unendlichen Generator anzuwenden, da dies zu einer Endlosschleife führen würde:

```
sum [nats]; # Never returns
```

generator foldl *function zero generator*
Dies ist ein Alias für den Befehl **reduce**.

generator foldr *function zero generator*
Dies ist die rechtsassoziative Version von **reduce**. Diese Operation ist im Allgemeinen ineffizient, da der gesamte Generator zunächst (als Liste) im Speicher ausgewertet werden muss, bevor die Reduktion beginnen kann. In einer Sprache mit sofortiger Auswertung wie Tcl ist diese Operation nur von begrenztem Nutzen und sollte nach Möglichkeit vermieden werden.

generator all *predicate generator*
Gibt "true" zurück, wenn alle Elemente des Generators das angegebene Prädikat erfüllen.

generator and *generator*
Gibt "true" zurück, wenn alle Elemente des Generators "true" sind (d. h., es wird die logische Konjunktion der Elemente gebildet).

generator any *generator*
Gibt "true" zurück, wenn eines der Elemente des Generators "true" ist (d. h. logische Disjunktion).

generator concat *generator* ?*generator..*?
Gibt einen Generator zurück, der aus der Verkettung aller Argument-Generatoren besteht.

generator concatMap *function generator*
Bei einer Funktion, die einen Wert auf eine Reihe von Werten abbildet, und einem Generator für Werte dieses Typs gibt die Funktion einen Generator zurück, der alle Ergebnisse in einer einzigen flachen Reihe enthält. Dies entspricht der Anwendung von **concat** auf das Ergebnis von **map**.

generator drop *n generator*
Entfernt die angegebene Anzahl von Elementen vom Anfang des Generators und gibt den resultierenden Generator zurück, aus dem diese Elemente entfernt wurden.

generator dropWhile *predicate generator*
Entfernt alle Elemente aus dem vorderen Teil des Generators, die das Prädikat erfüllen.

generator contains *element generator*
Gibt "true" zurück, wenn der Generator das angegebene Element enthält. Hinweis: Der Generator wird dadurch zerstört!

generator foldl1 *function generator*
Eine Version von **foldl**, die das Argument *zero* aus dem ersten Element des Generators entnimmt. Daher ist diese Funktion nur für nicht leere Generatoren gültig.

generator foldli *function zero generator*
Eine Version von **foldl**, die den ganzzahligen Index jedes Elements als erstes Argument an die Funktion übergibt. Das erste Element im Generator erhält an dieser Stelle den Index 0.

generator foldri *function zero generator*
Rechtsassoziative Version von **foldli**.

generator head *generator*
Gibt das erste Element des Generators zurück.

generator tail *generator*
Entfernt das erste Element des Generators und gibt den Rest zurück.

generator init *generator*
Gibt einen neuen Generator zurück, der aus allen Elementen des als Argument übergebenen Generators mit Ausnahme des letzten Elements besteht.

generator takeList *n generator*
Gibt die nächsten *n* Elemente des Generators als Liste zurück. Wenn im Generator nicht mehr genügend Elemente vorhanden sind, werden lediglich die verbleibenden Elemente zurückgegeben.

generator take *n generator*
Gibt die nächsten *n* Elemente des Generators als neuen Generator zurück. Der alte Generator wird gelöscht.

generator iterate *function init*
Gibt einen unendlichen Generator zurück, der durch wiederholte Anwendung der Funktion auf das Ausgangsargument gebildet wird. So lassen sich beispielsweise die Fibonacci-Zahlen wie folgt definieren:

```
    proc fst pair { lindex $pair 0 }
    proc snd pair { lindex $pair 1 }
    proc nextFib ab { list [snd $ab] [expr {[fst $ab] + [snd $ab]}] }
    proc fibs {} { generator map fst [generator iterate nextFib {0
    1}] }
```

generator last *generator*
Gibt das letzte Element des Generators zurück (sofern vorhanden).

generator length *generator*
Gibt die Länge des Generators zurück und löscht ihn dabei im Prozess.

generator or *predicate generator*
Gibt den Wert 1 zurück, wenn eines der Elemente des Generators das Prädikat erfüllt.

generator product *generator*
Gibt das Produkt der Zahlen in einem Generator zurück.

generator repeat *n value..*
Gibt einen Generator zurück, der aus *n* Kopien der angegebenen Elemente besteht. Mit dem Sonderwert *Inf* lässt sich eine unendliche Folge erzeugen.

generator sum *generator*
Gibt die Summe der Werte im Generator zurück.

generator takeWhile *predicate generator*
Gibt einen Generator der ersten Elemente des Argumentgenerators zurück, die das Prädikat

erfüllen.

generator splitWhen *predicate generator*
Teilt den Generator mithilfe des Prädikats zur Erkennung von Trennzeichen in Listen von Elementen auf. Die resultierenden Listen werden als Generator zurückgegeben. Elemente, die dem Trennzeichen-Prädikat entsprechen, werden verworfen. Um beispielsweise einen Generator unter Verwendung des Zeichenstrings "|" als Trennzeichen aufzuteilen:

```
    set xs [generator from list {a | b | c}]
    generator split {string equal "|"} $xs ;# returns a then b then c
```

generator scanl *function zero generator*
Ähnlich wie **foldl**, gibt jedoch einen Generator zurück, der alle Werte für das Argument "accumulator" enthält. Das letzte Element dieses Generators entspricht der Funktion **foldl**, die mit denselben Argumenten aufgerufen wurde.

Fehler, Ideen, Rückmeldungen
Bitte melden Sie Fehler in diesem Dokument oder im beschriebenen Paket an Neil Madden.

gpx

gpx
Stichwörter: gpx, gps

Name
gpx – Extrahiert Wegpunkte, Tracks und Routen aus GPX-Dateien

Übersicht
package require Tcl 8.5 9
package require gpx ?**1.1**?
::gpx::Create *gpxFilename* ?*rawXML*?
::gpx::Cleanup *token*
::gpx::GetGPXMetadata *token*
::gpx::GetWaypointCount *token*
::gpx::GetAllWaypoints *token*
::gpx::GetTrackCount *token*
::gpx::GetTrackMetadata *token whichTrack*
::gpx::GetTrackPoints *token whichTrack*
::gpx::GetRouteCount *token*
::gpx::GetRouteMetadata *token whichRoute*
::gpx::GetRoutePoints *token whichRoute*

Beschreibung
Dieses Modul parst eine GPX-Datei (GPS eXchange) und extrahiert daraus Wegpunkte, Tracks, Routen und Metadaten. Unterstützt werden die GPX-Versionen 1.0 und 1.1.

Befehle
::gpx::Create *gpxFilename* ?*rawXML*?

Der Befehl **::gpx::Create** ist der erste Befehl, der zur Verarbeitung von GPX-Daten aufgerufen wird. Er entnimmt die GPX-Daten dem Parameter *rawXML*, sofern vorhanden, sonst dem Inhalt von *gpxFilename*, und parst sie mit *tdom*. Er gibt einen Token-Wert zurück, den alle anderen Befehle verwenden.

::gpx::Cleanup *token*
Diese Prozedur räumt die mit *token* verbundenen Ressourcen auf. Es wird *dringend* empfohlen, diese Funktion aufzurufen, nachdem Sie die Arbeit mit einer bestimmten GPX-Datei abgeschlossen haben. Andernfalls wird der Speicher nicht freigegeben, und wenn Ihre App die Funktion **::gpx::Create** oft genug aufruft, könnte das Speicherleck zu Leistungseinbußen führen … oder Schlimmerem.

::gpx::GetGPXMetadata *token*
Diese Prozedur gibt ein Dictionary mit den Metadaten zurück, die zu den durch *token* identifizierten GPX-Daten gehören. Das Format des Metadaten-Dictionaries wird im Folgenden beschrieben, wobei die Schlüssel *version* und *creator* stets vorhanden sind.

::gpx::GetWaypointCount *token*
Diese Prozedur gibt die Anzahl der Wegpunkte zurück, die in den durch *token* identifizierten GPX-Daten definiert sind.

::gpx::GetAllWaypoints *token*
Diese Prozedur gibt eine Liste der Wegpunkte zurück, die in den durch *token* identifizierten GPX-Daten definiert sind. Das Format der einzelnen Wegpunkteinträge wird im Folgenden beschrieben.

::gpx::GetTrackCount *token*
Diese Prozedur gibt die Anzahl der in den durch *token* identifizierten GPX-Daten definierten Tracks zurück.

::gpx::GetTrackMetadata *token whichTrack*
Diese Prozedur gibt ein Dictionary mit den Metadaten zurück, die der Track-Nummer *whichTrack* (ab 1) in den durch *token* identifizierten GPX-Daten zugeordnet sind. Das Format des Metadaten-Dictionaries wird im Folgenden beschrieben.

::gpx::GetTrackPoints *token whichTrack*
Die Prozedur gibt eine Liste der Trackpunkte zurück, aus denen der Track mit der Nummer *whichTrack* (ab 1) in den durch *token* identifizierten GPX-Daten besteht. Das Format des Metadaten-Dictionaries wird im Folgenden beschrieben.

::gpx::GetRouteCount *token*
Diese Prozedur gibt die Anzahl der Routen zurück, die in den durch *token* identifizierten GPX-Daten definiert sind.

::gpx::GetRouteMetadata *token whichRoute*
Diese Prozedur gibt ein Dictionary mit den Metadaten zurück, die der Routennummer *whichRoute* (ab 1) in den durch *token* identifizierten GPX-Daten zugeordnet sind. Das Format des Metadaten-Dictionaries wird im Folgenden beschrieben.

::gpx::GetRoutePoints *token whichRoute*
Die Prozedur gibt eine Liste der Routenpunkte zurück, aus denen die Route mit der Nummer *whichRoute* (ab 1) in den durch *token* identifizierten GPX-Daten besteht. Das Format des Metadaten-Dictionaries wird im Folgenden beschrieben.

Datenstrukturen

metadata dictionary
Die Metadaten des GPX-Dokuments, eines Tracks, einer Route, eines Wegpunkts, eines Trackpunkts oder eines Routenpunkts werden in einem Dictionary zurückgegeben. Die Schlüssel dieses Dictionarys sind die jeweils vorhandenen optionalen GPX-Elemente. Der Wert jedes Schlüssels hängt vom GPX-Schema des jeweiligen Elements ab. So ist der Wert eines version-Schlüssels eine Zeichenkette, der eines link-Schlüssels ein untergeordnetes Dictionary mit den Schlüsseln *href* sowie optional *text* und *type*.

point item
Jeder Eintrag in einer Liste von Track- oder Routenpunkten besteht aus einer Liste mit drei Elementen: *latitude*, *longitude* und *metadata dictionary*. *latitude* und *longitude* sind Dezimalzahlen. Das Format des *metadata dictionary* ist oben beschrieben. Bei Punkten in einem Track sind in der Regel immer die Metadatenschlüssel ele (Höhe) und time (Zeit) vorhanden.

Beispiel

```
% set token [::gpx::Create myGpxFile.gpx]
% set version [dict get [::gpx::GetGPXMetadata $token] version]
% set trackCnt [::gpx::GetTrackCount $token]
% set firstPoint [lindex [::gpx::GetTrackPoints $token 1] 0]
% lassign $firstPoint lat lon ptMetadata
% puts "first point in the first track is at $lat, $lon"
% if {[dict exists $ptMetadata ele]} {
     puts "at elevation [dict get $ptMetadata ele] meters"
  }
% ::gpx::Cleanup $token
```

Literatur

1. GPX: the GPS Exchange Format (http://www.topografix.com/gpx.asp)
2. GPX 1.1 Schema Documentation (http://www.topografix.com/GPX/1/1/)
3. GPX 1.0 Developer's Manual (http://www.topografix.com/gpx_manual.asp)

Autoren

Keith Vetter

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *gpx* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dateiformate

Copyright

Copyright (c) 2010, Keith Vetter <kvetter@gmail.com>

grammar_aycock

aycock

Stichwörter: aycock, Earley, Horspool, parser, compiler, ambiguous, grammar, parsing, transducer

Name

grammar::aycock – Aycock-Horspool-Earley-Parsergenerator für Tcl

Übersicht

package require Tcl 8.5 9
package require grammar::aycock ?**1.1**?
::aycock::parser *grammar* ?**-verbose**?
parserName **parse** *symList valList* ?*clientData*?
parserName **destroy**
parserName **terminals**
parserName **nonterminals**
parserName **save**

Beschreibung

Das Paket **grammar::aycock** implementiert einen Parsergenerator für die Klasse von Parsern, die beschrieben ist in John Aycock und R. Nigel Horspool, Practical Earley Parsing, *The Computer Journal, 45*(6):620-630, 2002. http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.12.4254

Prozeduren

Das Paket **grammar::aycock** exportiert die folgende einzelne Prozedur:

::aycock::parser *grammar* ?-verbose?
: Erzeugt einen Parser für die angegebene *grammar* und gibt dessen Namen zurück. Wird das optionale Flag **-verbose** angegeben, gibt der Befehl ausführliche Informationen zum erzeugten Parser auf der Standardausgabe aus. Der zurückgegebene Parser ist ein Objekt, das die im Abschnitt Objektbefehl gezeigten Befehle akzeptiert.

Objektbefehl

***parserName* parse *symList valList* ?*clientData*?**
: Ruft einen von **::aycock::parser** zurückgegebenen Parser auf. *symList* ist eine Liste von Grammatiksymbolen, die die Terminale einer Eingabezeichenkette darstellen, und *valList* ist eine Liste ihrer semantischen Werte. Das Ergebnis ist der semantische Wert der gesamten Zeichenkette nach dem Parsen.

***parserName* destroy**
: Löscht einen von **::aycock::parser** erstellten Parser.

***parserName* terminals**
: Gibt eine Liste von Terminalsymbolen zurück, die im Argument *symList* des Objektbefehls

parse angegeben werden können.

***parserName* nonterminals**
Gibt eine Liste der nichtterminalen Symbole zurück, die in der Grammatik des Parsers definiert wurden.

***parserName* save**
Gibt ein Tcl-Skript zurück, das den Parser rekonstruiert, ohne dass zur Laufzeit der gesamte Mechanismus des Parsergenerators benötigt wird. Der rekonstruierte Parser stützt sich auf eine Reihe von Befehlen im Paket **grammar::aycock::runtime**, das ebenfalls automatisch geladen wird, wenn das Paket **grammar::aycock** geladen wird.

Beschreibung

Der Befehl **grammar::aycock::parser** akzeptiert eine Grammatik, die als Tcl-Liste ausgedrückt wird. Die Liste muss als Verkettung einer Reihe von *Regeln* aufgebaut sein. Jede *Regel* umfasst eine variable Anzahl von Elementen in der Liste:

- Der Name des nichtterminalen Symbols, auf das die Regel reduziert wird.
- Die literale Zeichenkette **::=**
- Null oder mehr Namen von terminalen oder nichtterminalen Symbolen, aus denen die rechte Seite der Regel besteht.
- Abschließend ein Tcl-Skript, das ausgeführt werden soll, wenn die Regel reduziert wird. Innerhalb des angegebenen Skripts enthält eine Variable namens _ eine Liste der semantischen Werte der Symbole auf der rechten Seite. Der vom Skript zurückgegebene Wert soll dem semantischen Wert der linken Seite entsprechen. Wenn der Parameter *clientData* an die Methode **parse** übergeben wurde, steht er in einer Variable namens **clientData** zur Verfügung. Es ist zulässig, dass das Skript eine leere Zeichenkette ist. In diesem Fall entspricht der semantische Wert der Regel dem semantischen Wert des ersten Symbols auf der rechten Seite. Wenn die rechte Seite ebenfalls leer ist, entspricht der semantische Wert der leeren Zeichenkette.

Die Analyse erfolgt mit einem Earley-Parser, der zwar hinsichtlich Geschwindigkeit und Speicherverbrauch nicht besonders effizient ist, jedoch effektiv mit mehrdeutigen Grammatiken umgeht. Aus diesem Grund eignet sich das Paket **grammar::aycock** möglicherweise am besten für die Verarbeitung natürlicher Sprache oder das Parsen außerordentlich komplexer Sprachen, in denen Mehrdeutigkeiten toleriert werden können.

Beispiel

Der folgende Code veranschaulicht einen einfachen Tischrechner, der als Operatoren ausschließlich **+**, ***** und Klammern zulässt. Er zeigt zudem das Format, in dem der lexikalische Analysator die Terminalsymbole dem Parser bereitstellen soll.

```
set p [aycock::parser {
    start ::= E {}
    E ::= E + T {expr {[lindex $_ 0] + [lindex $_ 2]}}
    E ::= T {}
    T ::= T * F {expr {[lindex $_ 0] * [lindex $_ 2]}}
    T ::= F {}
    F ::= NUMBER {}
    F ::= ( E ) {lindex $_ 1}
```

```
}]
puts [$p parse {(  NUMBER +  NUMBER )  *  ( NUMBER +  NUMBER ) }  {{}
    2      {} 3      {} {} {} 7     {} 1      {}}]
$p destroy
```

Wird das Beispiel ausgeführt, gibt es **40** aus.

Kategorie
Grammatiken und endliche Automaten

Copyright

```
Copyright (c) 2006 by Kevin B. Kenny <kennykb@acm.org>
Redistribution permitted under the terms of the Open Publication
    License <http://www.opencontent.org/openpub/>
```


grammar_fa

dacceptor
Stichwörter: dacceptor, acceptance, acceptor, automaton, finite automaton, grammar, parsing, regular expression, regular grammar, regular languages, state, transducer

Name
grammar::fa::dacceptor – Erstellen und Verwenden deterministischer Akzeptoren

Übersicht
package require Tcl 8.5 9
package require snit
package require struct::set
package require grammar::fa::dacceptor ?**0.1.2**?
::grammar::fa::dacceptor *daName fa* ?*-any any*?
daName *option* ?*arg arg* ...?
daName **destroy**
daName **accept?** *symbols*

Beschreibung
Dieses Paket stellt eine Klasse für Akzeptoren bereit, die aus deterministischen *endlichen Automaten* (DFA) aufgebaut sind. Akzeptoren sind Objekte, denen eine Folge von Symbolen übergeben werden kann und die angeben, ob der DFA, aus dem sie aufgebaut sind, diese Folge *akzeptieren* würde. Für die eigentliche Erstellung der DFAs, auf denen die Akzeptoren basieren, stehen die Pakete **grammar::fa** und **grammar::fa::op** zur Verfügung.

API
Das Paket exportiert die hier beschriebene API.

::grammar::fa::dacceptor *daName fa* ?-any *any*?
Erstellt einen neuen deterministischen Akzeptor mit einem zugehörigen globalen Tcl-Befehl, dessen Name *daName* lautet. Mit diesem Befehl können verschiedene Operationen auf dem

Akzeptor aufgerufen werden. Er hat die folgende allgemeine Form:

daName *option* ?*arg arg ...*?

Option und die *arg*s bestimmen das genaue Verhalten des Befehls. Weitere Erläuterungen finden Sie im Abschnitt Akzeptormethoden.

Der Akzeptor basiert auf dem deterministischen endlichen Automaten, der im Objekt *fa* gespeichert ist. Er bewahrt eine Kopie der relevanten Daten des FA in seinem eigenen Speicher auf, und zwar in einer Form, die für seine Zwecke leicht zu verwenden ist. Dies bedeutet auch, dass Änderungen, die nach der Erstellung des Akzeptors am *fa* vorgenommen werden, den Akzeptor *nicht* beeinflussen.

Wenn *any* festgelegt wurde, wandelt der Akzeptor alle Symbole in der Eingabe, die dem Basis-FA unbekannt sind, in dieses Symbol um, bevor er mit der Verarbeitung fortfährt.

Akzeptormethoden

Alle Akzeptoren stellen die folgenden Methoden zu ihrer Handhabung bereit:

***daName* destroy**

Löscht den Automaten einschließlich seines Speicherplatzes und des zugehörigen Befehls.

daName* accept? *symbols

Nimmt die Liste der *symbols* entgegen und prüft, ob der FA, auf dem der Akzeptor basiert, sie akzeptieren würde. Das Ergebnis ist ein boolescher Wert. **True** wird zurückgegeben, wenn die Symbole akzeptiert werden, andernfalls **False**. Zu beachten ist, dass ungültige Symbole in der Eingabe entweder in das Symbol *any* umgewandelt werden (falls es angegeben ist) oder dazu führen, dass der Akzeptanztest einfach fehlschlägt. Es werden keine Fehler ausgelöst. Die Methode verarbeitet nur genau das Präfix der Eingabe, das ausreicht, um die (Nicht-)Akzeptanz vollständig zu bestimmen.

Beispiele

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_fa* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

dexec

Stichwörter: dexec, automaton, execution, finite automaton, grammar, parsing, regular expression, regular grammar, regular languages, running, state, transducer

Name

grammar::fa::dexec – Ausführung deterministischer endlicher Automaten

Übersicht

package require Tcl 8.5 9
package require snit
package require grammar::fa::dexec ?**0.3**?
::grammar::fa::dexec *daName fa* ?*-any any*? ?*-command cmdprefix*?
daName *option* ?*arg arg* ...?
daName **destroy**
daName **put** *symbol*
daName **reset**
daName **state**
cmdprefix **error** *code message*
cmdprefix **final** *stateid*
cmdprefix **reset**
cmdprefix **state** *stateid*

Beschreibung

Dieses Paket stellt eine Klasse für Executors bereit, die auf deterministischen *endlichen Automaten* (DFA) basieren. Executors sind Objekte, denen eine Zeichenfolge stückweise übermittelt wird; sie führen Zustandsübergänge durch und melden zurück, sobald sie einen Endzustand erreichen oder einen Fehler in der Eingabe feststellen. Für die eigentliche Erstellung der DFAs, auf denen die Executors basieren, stehen die Pakete **grammar::fa** und **grammar::fa::op** zur Verfügung.

Die Objekte folgen einem Push-Modell. Symbole werden in den Executor übergeben, und wenn etwas Wichtiges geschieht – d. h. ein Fehler auftritt, ein Zustandsübergang stattfindet oder ein Endzustand erreicht wird –, wird dies über die Rückruffunktion gemeldet, die über die Option **-command** festgelegt wurde. Die Umstellung auf ein Pull-Modell, bei dem die Umgebung Nachrichten vom Objekt abruft und das Objekt über eine Rückruffunktion weitere Symbole anfordert, ist eine einfache Angelegenheit.

Hinweis: Die von **grammar::fa::dacceptor** bereitgestellten Akzeptorobjekte hätten auf der Grundlage der hier bereitgestellten Executors implementiert werden können, wurden jedoch aus Gründen der Leistungssteigerung nicht so umgesetzt (wir vermeiden dadurch eine Reihe von Aufrufen von Methoden und die dafür erforderliche Zeit).

API

Das Paket exportiert die hier beschriebene API.

::grammar::fa::dexec *daName fa* ?-any *any*? ?-command *cmdprefix*?
Erstellt einen neuen deterministischen Executor mit einem zugehörigen globalen Tcl-Befehl, dessen Name *daName* lautet. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Executor auszuführen. Er hat die folgende allgemeine Form:

daName *option* ?*arg arg ...*?
Option und die *arg*s bestimmen das genaue Verhalten des Befehls. Weitere Erläuterungen finden Sie im Abschnitt Executor-Methoden.

Der Executor basiert auf dem deterministischen endlichen Automaten, der im Objekt *fa* gespeichert ist. Er bewahrt eine Kopie der relevanten Daten des FA in seinem eigenen Speicher auf, und zwar in einer Form, die für seine Zwecke leicht zu verwenden ist. Dies bedeutet auch, dass Änderungen, die nach der Erstellung des Executors am *fa* vorgenommen werden, den Executor *nicht* beeinflussen.

Wenn *any* festgelegt wurde, wandelt der Executor alle Symbole in der Eingabe, die dem Basis-FA unbekannt sind, in dieses Symbol um, bevor er mit dem Prozess fortfährt.

Executor-Methoden

Alle Executors stellen die folgenden Methoden für deren Bearbeitung bereit:

***daName* destroy**
Löscht den Automaten einschließlich seines Speicherplatzes und des zugehörigen Befehls.

daName* put *symbol
Übernimmt den aktuellen Zustand des Executors und das *symbol* und führt den entsprechenden Zustandsübergang durch. Meldet über die Rückruffunktion des Befehls sowohl aufgetretene Fehler als auch das Erreichen eines Endzustands des zugrunde liegenden FA.

Wenn ein Fehler gemeldet wird, führen alle weiteren Aufrufe von **put** keine Aktion aus, bis die Fehlerbedingung durch einen Aufruf der Methode **reset** behoben wurde.

***daName* reset**
Setzt den Executor bedingungslos in den Startzustand der zugrunde liegenden FA. Dadurch werden zudem etwaige Fehlerzustände, auf die **put** möglicherweise gestoßen ist, zurückgesetzt.

***daName* state**
Gibt den aktuellen Zustand des zugrunde liegenden FA zurück. Dies ermöglicht eine Introspektion, ohne dass Daten aus der Rückruffunktion übergeben werden müssen.

Rückruffunktion des Executors

Die Rückruffunktion *cmdprefix*, die einem Executor über die Option **-command** übergeben wird, wird vom Objekt auf globaler Ebene unter Verwendung der unten beschriebenen Syntax ausgeführt. Beachten Sie, dass *cmdprefix* nicht einfach der Name eines Befehls ist, sondern ein vollständiges Befehlspräfix. Mit anderen Worten: Es kann zusätzlich zum Befehlswort weitere feste Argumentwörter enthalten.

cmdprefix* error *code message
Beim Executor ist ein Fehler aufgetreten, und *message* enthält einen für Menschen lesbaren Text, der die Art des Problems erläutert. Der *code* hingegen ist ein festgelegter, maschinenlesbarer Text. Die folgenden Fehlercodes können von Executor-Objekten generiert werden.
BADSYM

In der Eingabe wurde ein unbekanntes Symbol gefunden. Dies kann genau dann vorkommen, wenn kein Symbol "**-any**" festgelegt wurde.

BADTRANS
Der zugrunde liegende FA weist für die aktuelle Kombination aus Eingabesymbol und Zustand keinen Übergang auf. Mit anderen Worten: Der Executor war nicht in der Lage, für diese Kombination einen neuen Zustand zu berechnen.

cmdprefix* final *stateid
Der Executor ist in den Endzustand *stateid* übergegangen.

***cmdprefix* reset**
Der Executor wurde zurückgesetzt.

cmdprefix* state *stateid
Der Zustand des FA hat sich aufgrund eines Übergangs geändert. *stateid* ist der neue Zustand.

Beispiele

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_fa* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2007 Bogdan <rftghost@users.sourceforge.net>
```


fa

Stichwörter: fa, automaton, finite automaton, grammar, parsing, regular expression, regular grammar, regular languages, state, transducer

Name

grammar::fa – Erstellen und Bearbeiten endlicher Automaten

Übersicht

package require Tcl 8.5 9
package require snit 1.3
package require struct::list
package require struct::set

package require grammar::fa::op ?**0.3**?
package require grammar::fa ?**0.6**?
::grammar::fa *faName* ?=|*:=*|<--|*as*|*deserialize src*|*fromRegex re over*?
faName *option* ?*arg arg* ...?
faName **destroy**
faName **clear**
faName **=** *srcFA*
faName **-->** *dstFA*
faName **serialize**
faName **deserialize** *serialization*
faName **states**
faName **state add** *s1* ?*s2* ...?
faName **state delete** *s1* ?*s2* ...?
faName **state exists** *s*
faName **state rename** *s snew*
faName **startstates**
faName **start add** *s1* ?*s2* ...?
faName **start remove** *s1* ?*s2* ...?
faName **start?** *s*
faName **start?set** *stateset*
faName **finalstates**
faName **final add** *s1* ?*s2* ...?
faName **final remove** *s1* ?*s2* ...?
faName **final?** *s*
faName **final?set** *stateset*
faName **symbols**
faName **symbols@** *s* ?*d*?
faName **symbols@set** *stateset*
faName **symbol add** *sym1* ?*sym2* ...?
faName **symbol delete** *sym1* ?*sym2* ...?
faName **symbol rename** *sym newsym*
faName **symbol exists** *sym*
faName **next** *s sym* ?*--> next*?
faName **!next** *s sym* ?*--> next*?
faName **nextset** *stateset sym*
faName **is deterministic**
faName **is complete**
faName **is useful**
faName **is epsilon-free**
faName **reachable_states**
faName **unreachable_states**
faName **reachable** *s*
faName **useful_states**
faName **unuseful_states**
faName **useful** *s*
faName **epsilon_closure** *s*

faName **reverse**
faName **complete**
faName **remove_eps**
faName **trim** ?*what*?
faName **determinize** ?*mapvar*?
faName **minimize** ?*mapvar*?
faName **complement**
faName **kleene**
faName **optional**
faName **union** *fa* ?*mapvar*?
faName **intersect** *fa* ?*mapvar*?
faName **difference** *fa* ?*mapvar*?
faName **concatenate** *fa* ?*mapvar*?
faName **fromRegex** *regex* ?*over*?

Beschreibung

Dieses Paket stellt eine Containerklasse für *endliche Automaten* (kurz: FA) bereit. Es ermöglicht die schrittweise Definition des Automaten, dessen Bearbeitung sowie die Abfrage der Definition. Das Paket bietet zwar komplexe Operationen auf dem Automaten (über das Paket **grammar::fa::op**), verfügt jedoch nicht über die Möglichkeit, eine Definition für einen Symbolstrom auszuführen. Verwenden Sie hierfür die Pakete **grammar::fa::dacceptor** und **grammar::fa::dexec**. Ein weiteres damit verbundenes Paket ist **grammar::fa::compiler**. Es wandelt eine endliche Automatik (FA) in eine Ausführungsklasse um, in die die Definition der FA fest integriert ist. Die Ausgabe dieses Pakets lässt sich so konfigurieren, dass sie einer Vielzahl unterschiedlicher Implementierungssprachen und Paradigmen gerecht wird.

Weitere Informationen darüber, was ein endlicher Automat ist, finden Sie im Abschnitt Endliche Automaten.

API

Das Paket exportiert die hier beschriebene API.

::grammar::fa *faName* ?=|:=|<--|as|deserialize *src*|fromRegex *re* ?*over*??

Erstellt einen neuen endlichen Automaten mit einem zugehörigen globalen Tcl-Befehl, dessen Name *faName* lautet. Mit diesem Befehl lassen sich verschiedene Operationen auf dem Automaten ausführen. Er hat die folgende allgemeine Form:

faName *option* ?*arg arg ...*?

Option und die *arg*s bestimmen das genaue Verhalten des Befehls. Weitere Erläuterungen finden Sie im Abschnitt FA-Methoden. Der neue Automat ist leer, wenn kein *src* angegeben wird. Andernfalls enthält er eine Kopie der in *src* enthaltenen Definition. *src* muss bei allen Operatoren mit Ausnahme von **deserialize** und **fromRegex** eine FA-Objektreferenz sein. Der Operator **deserialize** erfordert stattdessen, dass *src* die Serialisierung eines FA ist, und **fromRegex** erwartet einen regulären Ausdruck in Form eines Syntaxbaums. Weitere Einzelheiten hierzu finden Sie unter ::grammar::fa::op::fromRegex.

FA-Methoden

Alle Automaten stellen die folgenden Methoden zu ihrer Steuerung bereit:

***faName* destroy**
Löscht den Automaten einschließlich seines Speicherplatzes und des zugehörigen Befehls.

***faName* clear**
Löscht die in *faName* enthaltene Definition des Automaten, zerstört das Objekt jedoch *nicht*.

faName = srcFA
Weist *faName* den Inhalt des in *srcFA* enthaltenen Automaten zu und überschreibt dabei etwaige bereits vorhandene Definitionen. Dies ist der Zuweisungsoperator für Automaten. Er kopiert den im FA-Objekt *srcFA* enthaltenen Automaten über die Automaten-Definition in *faName*. Der bisherige Inhalt von *faName* wird durch diesen Vorgang gelöscht.

Dieser Vorgang entspricht im Wesentlichen

```
faName deserialize [srcFA serialize]
```

faName --> dstFA
Dies ist der umgekehrte Zuweisungsoperator für Automaten. Er kopiert die im Objekt *faName* enthaltene Automatisierung in die Automatendefinition des Objekts *dstFA*. Der bisherige Inhalt von *dstFA* wird durch diesen Vorgang gelöscht.

Dieser Vorgang entspricht im Wesentlichen

```
dstFA deserialize [faName serialize]
```

***faName* serialize**
Diese Methode serialisiert den in *faName* gespeicherten Automaten. Mit anderen Worten: Sie gibt einen Tcl-*Wert* zurück, der diesen Automaten vollständig beschreibt. Dies ermöglicht beispielsweise die Übertragung von Automaten über beliebige Kanäle, Persistenz usw. Diese Methode bildet zudem die Grundlage sowohl für den Kopier-Konstruktor als auch für den Zuweisungsoperator.

Das Ergebnis dieser Methode muss über alle Implementierungen der Schnittstelle **grammar::fa** hinweg semantisch identisch sein. Dies ermöglicht es uns, Automaten zwischen verschiedenen Implementierungen derselben Schnittstelle zu kopieren.

Das Ergebnis ist eine Liste mit drei Elementen, die folgende Struktur aufweist:
1. Die Konstante "**grammar::fa**".
2. Eine Liste, die die Namen aller bekannten Eingabesymbole enthält. Die Reihenfolge der Elemente in dieser Liste spielt keine Rolle.
3. Der letzte Eintrag in der Liste ist ein Dictionary, wobei jedoch auch die Reihenfolge der Schlüssel von Bedeutung ist. Die Schlüssel sind die Zustände der serialisierten FA, und ihre Reihenfolge entspricht der Reihenfolge, in der die Zustände bei der Deserialisierung erstellt werden sollen. Dies ist wichtig, um die Reihenfolgebeziehung zwischen den Zuständen beizubehalten.

   Der Wert jedes Wörterbucheintrags ist eine Liste aus drei Elementen, die den Zustand detaillierter beschreiben.
   1. Ein boolescher Flag. Wenn sein Wert **true** lautet, handelt es sich um einen Startzustand; andernfalls ist dies nicht der Fall.
   2. Ein boolescher Flag. Wenn sein Wert "**true**" lautet, handelt es sich um einen Endzustand; andernfalls ist dies nicht der Fall.

3. Das letzte Element ist ein Dictionary, das die Übergänge für den Zustand beschreibt. Die Schlüssel sind Symbole (oder die leere Zeichenkette), und die Werte sind Mengen von Nachfolgezuständen.

Angenommen, es handelt sich um die folgende FA (die das Leben eines Lkw-Treibers auf sehr einfache Weise beschreibt:)

```
Drive -- yellow --> Brake -- red --> (Stop) -- red/yellow -->
Attention -- green --> Drive
(...) is the start state.
```

Eine mögliche Serialisierung lautet

```
grammar::fa \
{yellow red green red/yellow} \
{Drive     {0 0 {yellow     Brake}} \
 Brake     {0 0 {red        Stop}} \
 Stop      {1 0 {red/yellow Attention}} \
 Attention {0 0 {green      Drive}}}
```

Eine mögliche Variante, da mir die Reihenfolge der Erstellung hier nicht wichtig war

faName* deserialize *serialization
Dies ist die Ergänzung zu **serialize**. Sie ersetzt die Automatdefinition in *faName* durch den durch den Wert *serialization* beschriebenen Automaten. Der bisherige Inhalt von *faName* wird durch diesen Vorgang gelöscht.

***faName* states**
Gibt die Menge aller Zustände zurück, die *faName* bekannt sind.

***faName* state add *s1* ?*s2* ...?**
Fügt die Zustände *s1*, *s2* usw. zur FA-Definition in *faName* hinzu. Der Vorgang schlägt fehl, wenn einer der neuen Zustände bereits deklariert ist.

***faName* state delete *s1* ?*s2* ...?**
Löscht die Zustände *s1*, *s2* usw. sowie alle zugehörigen Informationen aus der FA-Definition in *faName*. Letzteres bedeutet, dass auch die Informationen zu eingehenden und ausgehenden Übergängen gelöscht werden. Wenn es sich bei dem gelöschten Zustand um einen Start- oder Endzustand handelt, werden diese Informationen ebenfalls ungültig gemacht. Der Vorgang schlägt fehl, wenn der Zustand *s* der FA nicht bekannt ist.

faName* state exists *s
Ein Prädikat. Es prüft, ob der Zustand *s* der FA in *faName* bekannt ist. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn der Zustand *s* bekannt ist, andernfalls auf **false**.

faName* state rename *s snew
Benennt den Zustand *s* in *snew* um. Der Vorgang schlägt fehl, wenn *s* kein bekannter Zustand ist. Der Vorgang schlägt ebenfalls fehl, wenn *snew* bereits als Zustand bekannt ist.

***faName* startstates**
Gibt die Menge der Zustände zurück, die als *start*-Zustände gekennzeichnet sind, auch als *initial*-Zustände bezeichnet. Was das bedeutet, steht unter Endliche Automaten.

***faName* start add *s1* ?*s2* ...?**
Kennzeichnet die Zustände *s1*, *s2* usw. im FA *faName* als *start* (auch *initial*).

***faName* start remove *s1* ?*s2* ...?**
Kennzeichnet die Zustände *s1*, *s2* usw. in der FA *faName* als *not start* (auch *not accepting*).

faName* start? *s
Ein Prädikat. Es testet, ob der Zustand *s* in der Finiten Automat *faName* den Wert *start* hat oder nicht. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn der Zustand *s* *start* ist, andernfalls auf **false**.

faName* start?set *stateset
Ein Prädikat. Es prüft, ob die Menge der Zustände *stateset* mindestens einen Startzustand enthält. Die Operation schlägt fehl, wenn die Menge ein Element enthält, das kein bekannter Zustand ist. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn in *stateset* ein Startzustand vorhanden ist, andernfalls auf **false**.

***faName* finalstates**
Gibt die Menge der Zustände zurück, die als *final*-Zustände gekennzeichnet sind, auch als *accepting*-Zustände bezeichnet. Was das bedeutet, steht unter Endliche Automaten.

***faName* final add *s1* ?*s2* ...?**
Kennzeichnet die Zustände *s1*, *s2* usw. im FA *faName* als *final* (auch *accepting*).

***faName* final remove *s1* ?*s2* ...?**
Kennzeichnet die Zustände *s1*, *s2* usw. in der FA *faName* als *not final* (auch *not accepting*).

faName* final? *s
Ein Prädikat. Es testet, ob der Zustand *s* in der FA *faName* *final* ist oder nicht. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn der Zustand *s* *final* ist, andernfalls auf **false**.

faName* final?set *stateset
Ein Prädikat. Es prüft, ob die Menge der Zustände *stateset* mindestens einen Endzustand enthält. Die Operation schlägt fehl, wenn die Menge ein Element enthält, das kein bekannter Zustand ist. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn in *stateset* ein Endzustand vorhanden ist, andernfalls auf **false**.

***faName* symbols**
Gibt die Menge aller Symbole zurück, die der FA *faName* bekannt sind.

***faName* symbols@ *s* ?*d*?**
Gibt die Menge aller Symbole zurück, für die der Zustand *s* Übergänge aufweist. Wenn das leere Symbol vorhanden ist, so weist *s* Epsilon-Übergänge auf. Wenn zwei Zustände festgelegt werden, so ist das Ergebnis die Menge der Symbole, für die Übergänge von *s* nach *t* bestehen. Diese Menge kann leer sein, wenn zwischen den beiden festgelegten Zuständen keine Übergänge bestehen.

faName* symbols@set *stateset
Gibt die Menge aller Symbole zurück, für die mindestens ein Zustand in der Zustandsmenge *stateset* Übergänge aufweist. Mit anderen Worten: die Vereinigung von [*faName* **symbols@ s**] für alle Zustände **s** in *stateset*. Wenn das leere Symbol vorhanden ist, so weist mindestens ein in *stateset* enthaltener Zustand Epsilon-Übergänge auf.

***faName* symbol add *sym1* ?*sym2* ...?**
Fügt die Symbole *sym1*, *sym2* usw. zur FA-Definition in *faName* hinzu. Der Vorgang schlägt fehl, wenn eines der Symbole bereits deklariert ist. Die leere Zeichenkette ist als Wert für die

Symbole nicht zulässig.

***faName* symbol delete *sym1* ?*sym2* ...?**
Löscht die Symbole *sym1*, *sym2* usw. sowie alle zugehörigen Informationen aus der FA-Definition in *faName*. Letzteres bedeutet, dass auch alle Übergänge, die diese Symbole verwenden, gelöscht werden. Der Vorgang schlägt fehl, wenn eines der Symbole der FA nicht bekannt ist.

faName* symbol rename *sym newsym
Benennt das Symbol *sym* in *newsym* um. Der Vorgang schlägt fehl, wenn *sym* kein bekanntes Symbol ist. Der Vorgang schlägt ebenfalls fehl, wenn *newsym* bereits als Symbol bekannt ist.

faName* symbol exists *sym
Ein Prädikat. Es prüft, ob das Symbol *sym* der FA in *faName* bekannt ist. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn das Symbol *sym* bekannt ist, andernfalls auf **false**.

***faName* next *s sym* ?--> *next*?**
Definiert Übergangsinformationen oder fragt sie ab.

Wenn *next* angegeben wird, fügt die Methode dem in *faName* enthaltenen FA einen Übergang vom Zustand *s* zum *successor*-Zustand *next* hinzu, der mit dem Symbol *sym* beschriftet ist. Der Vorgang schlägt fehl, wenn *s* oder *next* keine bekannten Zustände sind oder wenn *sym* kein bekanntes Symbol ist. Eine Ausnahme von Letzterem besteht darin, dass *sym* die leere Zeichenkette sein darf. In diesem Fall handelt es sich bei dem neuen Übergang um einen *Epsilon-Übergang*, der beim Durchlaufen keine Eingabe verbraucht. Die Operation schlägt ebenfalls fehl, wenn die Kombination aus (*s*, *sym* und *next*) bereits in der FA vorhanden ist.

Wurde *next* nicht angegeben, gibt die Methode die Menge der Zustände zurück, die von *s* aus über einen einzigen, mit dem Symbol *sym* beschrifteten Übergang erreicht werden können.

***faName*!next *s sym* ?--> *next*?**
Entfernt einen oder mehrere Übergänge aus der FA in *faName*.

Wenn *next* angegeben wurde, wird der einzelne Übergang vom Zustand *s* zum Zustand *next*, der mit dem Symbol *sym* beschriftet ist, aus der FA entfernt. Andernfalls werden *all* Übergänge entfernt, die vom Zustand *s* ausgehen und mit dem Symbol *sym* beschriftet sind.

Die Operation schlägt fehl, wenn *s* und/oder *next* nicht als Zustände bekannt sind. Sie schlägt ebenfalls fehl, wenn ein nicht leerer Wert für *sym* nicht als Symbol bekannt ist. Die leere Zeichenkette ist zulässig und ermöglicht das Entfernen von Epsilon-Übergängen.

faName* nextset *stateset sym
Gibt die Menge der Zustände zurück, die durch einen einzelnen Übergang erreicht werden können, der von einem Zustand aus der Menge *stateset* ausgeht und mit dem Symbol *sym* beschriftet ist.

Mit anderen Worten: Dies ist die Vereinigung von [*faName* next **s** *symbol*] für alle Zustände **s** in *stateset*.

***faName* is deterministic**
Ein Prädikat. Es prüft, ob es sich bei der FA in *faName* um eine deterministische FA handelt

oder nicht. Der Wert ist boolesch. Er wird auf **true** gesetzt, wenn die FA deterministisch ist, andernfalls auf **false**.

***faName* is complete**
Ein Prädikat. Es prüft, ob die FA in *faName* eine vollständige FA ist oder nicht. Eine FA ist vollständig, wenn sie mindestens einen Übergang pro Zustand und Symbol aufweist. Dies bedeutet auch, dass eine FA ohne Symbole oder Zustände ebenfalls vollständig ist. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn die FA vollständig ist, und andernfalls auf **false**.

Hinweis: Wenn eine FA Epsilon-Übergänge aufweist, können Übergänge über ein Symbol für einen Zustand S indirekt sein, d. h. nicht direkt an S, sondern an einen Zustand in der Epsilon-Hülle von S gekoppelt sein. Die Symbole für solche indirekten Übergänge werden bei der Berechnung der Vollständigkeit berücksichtigt.

***faName* is useful**
Ein Prädikat. Es testet, ob die FA in *faName* eine nützliche FA ist oder nicht. Eine FA gilt als nützlich, wenn alle Zustände *reachable* und *useful* sind. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn die FA nützlich ist, andernfalls auf **false**.

***faName* is epsilon-free**
Ein Prädikat. Es führt Tests durch, um zu prüfen, ob die FA in *faName* eine epsilonfreie FA ist oder nicht. Ein FA ist epsilonfrei, wenn es keine Epsilon-Übergänge aufweist. Diese Definition bedeutet, dass alle deterministischen FAs ebenfalls epsilonfrei sind und dass Epsilonfreiheit eine notwendige Voraussetzung für Deterministik ist. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn die FA epsilonfrei ist, und andernfalls auf **false**.

***faName* reachable_states**
Gibt die Menge der Zustände zurück, die von einem Startzustand aus über einen oder mehrere Übergänge erreichbar sind.

***faName* unreachable_states**
Gibt die Menge der Zustände zurück, die von keinem Startzustand aus durch eine beliebige Anzahl von Übergängen erreichbar sind. Dies ist

```
[faName states] - [faName reachable_states]
```

faName* reachable *s
Ein Prädikat. Es führt Tests durch, um zu prüfen, ob der Zustand *s* in der endlichen Automat *faName* von einem Startzustand aus über einen oder mehrere Übergänge erreicht werden kann. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn der Zustand erreichbar ist, und andernfalls auf **false**.

***faName* useful_states**
Gibt die Menge der Zustände zurück, die einen Endzustand über einen oder mehrere Übergänge erreichen können.

***faName* unuseful_states**
Gibt die Menge der Zustände zurück, die durch keine beliebige Anzahl von Übergängen einen Endzustand erreichen können. Dies ist

```
[faName states] - [faName useful_states]
```

faName* useful *s

Ein Prädikat. Es führt Tests durch, um zu prüfen, ob der Zustand *s* im endlichen Automaten *faName* über eine oder mehrere Übergänge einen Endzustand erreichen kann. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn der Zustand brauchbar ist, andernfalls auf **false**.

faName* epsilon_closure *s
Gibt die Menge der Zustände zurück, die vom Zustand *s* im FA *faName* über einen oder mehrere Epsilon-Übergänge erreichbar sind, d. h. Übergänge über das leere Symbol, also Übergänge, bei denen keine Eingabe verbraucht wird. Dies wird als *Epsilon-Abschluss* von *s* bezeichnet.

***faName* reverse**
siehe unten ...

***faName* complete**
siehe unten ...

***faName* remove_eps**
siehe unten ...

***faName* trim ?*what*?**
siehe unten ...

***faName* determinize ?*mapvar*?**
siehe unten ...

***faName* minimize ?*mapvar*?**
siehe unten ...

***faName* complement**
siehe unten ...

***faName* kleene**
siehe unten ...

***faName* optional**
siehe unten ...

***faName* union *fa* ?*mapvar*?**
siehe unten ...

***faName* intersect *fa* ?*mapvar*?**
siehe unten ...

***faName* difference *fa* ?*mapvar*?**
siehe unten ...

***faName* concatenate *fa* ?*mapvar*?**
siehe unten ...

***faName* fromRegex *regex* ?*over*?**
Diese Methoden ermöglichen komplexere Operationen mit der FA. Eine Beschreibung ihrer Funktionsweise finden Sie unter den gleichnamigen Befehlen im Paket **grammar::fa::op**.

Beispiele

Endliche Automaten

Für mathematisch Interessierte: Eine FA ist ein 5-Tupel (S, Sy, St, Fi, T), wobei

- S ist eine Menge von *states*,

- Sy ist eine Menge von *input symbols*,
- St ist eine Teilmenge von S, die Menge der *start*-Zustände, auch *initial*-Zustände genannt.
- Fi ist eine Teilmenge von S, die Menge der *final*-Zustände, auch *accepting* genannt.
- T ist eine Funktion von S x (Sy + epsilon) nach {S}, die *Übergangsfunktion*. Hier bezeichnet **epsilon** das leere Eingabesymbol und unterscheidet sich von allen Symbolen in Sy; und {S} ist die Menge der Teilmengen von S. Mit anderen Worten: T bildet eine Kombination aus Zustand und Eingabe (die leer sein kann) auf eine Menge von *successor states* ab.

In der Informatik wird ein FA meist als Graph dargestellt, in dem die Knoten die Zustände repräsentieren und die Kanten zwischen den Knoten die Übergangsfunktion kodieren: Für alle n in S' = T (s, sy) gibt es je eine Kante zwischen den Knoten, die s bzw. n darstellen, beschriftet mit sy. Die Start- und Akzeptanzzustände werden durch unterschiedliche visuelle Markierungen kodiert, das heißt, sie sind Attribute der Knoten.

FAs verarbeiten Symbolströme über Sy.

Von einem bestimmten FA sagt man, er *accept* einen endlichen Strom sy_1 sy_2 ... sy_n, wenn es im Graphen des FA einen Pfad gibt, der in einem Zustand in St beginnt und in einem Zustand in Fi endet und dessen Kanten die Beschriftungen sy_1, sy_2 usw. bis sy_n tragen. Die Menge aller von der FA akzeptierten Zeichenketten ist die *language* der FA. Eine wichtige Äquivalenz ist, dass die Menge der Sprachen, die ein FA akzeptieren kann, die Menge der *regular languages* ist.

Ein weiteres wichtiges Konzept sind deterministische FAs. Ein FA heißt *deterministic*, wenn es für jede Zeichenkette aus Eingabesymbolen genau einen Pfad im Graphen des FA gibt, der im Startzustand beginnt und dessen Kanten mit den Symbolen der Zeichenkette beschriftet sind. Auch wenn es so scheinen mag, dass nichtdeterministische FAs mehr Erkennungskraft haben, ist das nicht der Fall. Zu jedem nichtdeterministischen FA lässt sich ein deterministischer FA konstruieren, der dieselbe Sprache akzeptiert (--> Thompson's subset construction).

Eine der wichtigsten Anwendungen von FAs liegt im *parsing*, besonders in der *lexer*-Stufe (wo Symbole == Zeichen sind), aber das ist bei weitem nicht die einzige.

Zahlreiche Prozesse lassen sich als FA modellieren, wenn auch mit einer möglicherweise großen Menge von Zuständen. Für diese ist der Begriff der Akzeptanzzustände oft weniger oder gar nicht relevant. Stattdessen ist die Fähigkeit erforderlich, auf Zustandsänderungen in der FA zu reagieren, d. h. als Reaktion auf die Eingabe eine Ausgabe zu erzeugen. Das macht aus einem FA einen *finite transducer*, der eine zusätzliche Menge OSy von *output symbols* und eine zusätzliche *output function* O hat, die von "S x (Sy + epsilon)" auf "(Osy + epsilon)" abbildet, d. h. von einer Kombination aus Zustand und Eingabe – die möglicherweise leer ist – auf ein Ausgabesymbol oder nichts.

Für die Graphdarstellung bedeutet dies, dass Kanten zusätzlich mit dem Ausgabesymbol beschriftet werden, das geschrieben werden soll, wenn diese Kante beim Abgleichen der Eingabe durchlaufen wird. Hinweis: In einer Anwendung bedeutet das "Schreiben eines Ausgabesymbols" auch das "Ausführen von Code".

Transducer werden von diesem Paket nicht behandelt. Sie werden in Zukunft ein eigenes Paket erhalten.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *grammar_fa* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Grammatiken und endliche Automaten

Copyright

```
Copyright (c) 2004-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


faop
Stichwörter: faop, automaton, finite automaton, grammar, parsing, regular expression, regular grammar, regular languages, state, transducer

Name
grammar::fa::op – Operationen an endlichen Automaten

Übersicht
package require Tcl 8.5 9
package require snit
package require struct::list
package require struct::set
package require grammar::fa::op ?**0.4.2**?
::grammar::fa::op::constructor *cmd*
::grammar::fa::op::reverse *fa*
::grammar::fa::op::complete *fa* ?*sink*?
::grammar::fa::op::remove_eps *fa*
::grammar::fa::op::trim *fa* ?*what*?
::grammar::fa::op::determinize *fa* ?*mapvar*?
::grammar::fa::op::minimize *fa* ?*mapvar*?
::grammar::fa::op::complement *fa*
::grammar::fa::op::kleene *fa*
::grammar::fa::op::optional *fa*
::grammar::fa::op::union *fa fb* ?*mapvar*?
::grammar::fa::op::intersect *fa fb* ?*mapvar*?
::grammar::fa::op::difference *fa fb* ?*mapvar*?
::grammar::fa::op::concatenate *fa fb* ?*mapvar*?
::grammar::fa::op::fromRegex *fa regex* ?*over*?
::grammar::fa::op::toRegexp *fa*
::grammar::fa::op::toRegexp2 *fa*
::grammar::fa::op::toTclRegexp *regexp symdict*
::grammar::fa::op::simplifyRegexp *regexp*

Beschreibung

Dieses Paket bietet eine Reihe komplexer Operationen für endliche Automaten (kurz: FA), wie sie vom Paket **grammar::fa** bereitgestellt werden. Das Paket bietet weder die Möglichkeit, solche FAs zu erstellen und/oder zu bearbeiten, noch die Möglichkeit, einen FA für einen Symbolstrom auszuführen. Verwenden Sie hierfür die Pakete **grammar::fa** und **grammar::fa::interpreter**. Ein weiteres damit zusammenhängendes Paket ist **grammar::fa::compiler**, das einen FA in eine Executor-Klasse umwandelt, in die die Definitionen des FA fest integriert sind.

Weitere Informationen darüber, was ein endlicher Automat ist, finden Sie im Abschnitt *ENDLICHE AUTOMATEN* im Paket **grammar::fa**.

API

Das Paket exportiert die hier beschriebene API. Alle Befehle ändern ihr erstes Argument. Das heißt, welcher FA auch immer von ihnen berechnet wird, wird wieder darin gespeichert. Einige der Operationen konstruieren einen Automaten, dessen Zustände alle neu sind, jedoch mit den Zuständen des/der Quellautomaten in Beziehung stehen. Diese Operationen nehmen Namen von Variablen als optionale Argumente entgegen, in denen sie Zuordnungen speichern, die die Beziehung(en) beschreiben. Die Operationen lassen sich grob in Struktur- und Sprachoperationen unterteilen. Letztere werden anhand der Sprache definiert, die der/die Automat(en) akzeptiert/akzeptieren, während erstere anhand der strukturellen Eigenschaften des/der beteiligten Automaten definiert werden. Einige Operationen sind beides.
Strukturoperationen

::grammar::fa::op::constructor *cmd*

Dieser Befehl muss vom Benutzer des Pakets aufgerufen werden, bevor andere Operationen ausgeführt werden, um einen Befehl zu definieren, mit dem ein FA-Container-Objekt erstellt werden kann. Wird dies nicht getan, schlagen mehrere Operationen fehl, da sie keine internen und temporären Container erstellen können, um den Zustand und/oder Teilergebnisse zu speichern.

Jede Containerklasse, die dieses Paket für komplexe Operationen verwendet, sollte ihren eigenen Klassenbefehl als Konstruktor festlegen. Beispiele finden Sie im Paket **grammar::fa**.

::grammar::fa::op::reverse *fa*

Kehrt den *fa* um. Dazu wird die Richtung aller Übergänge umgekehrt und die Mengen der *start* - und *final*-Zustände werden vertauscht. Die Sprache von *fa* verändert sich auf unvorhersehbare Weise.

::grammar::fa::op::complete *fa* ?*sink*?

Vervollständigt den *fa*, tut aber nichts, wenn der *fa* bereits *complete* ist. Dies bedeutet, dass nur die erste einer Reihe mehrerer aufeinanderfolgender *complete*-Operationen auf *fa* eine Aktion ausführt. Die übrigen Operationen sind Nulloperationen.

Die Sprache von *fa* bleibt durch diese Operation unverändert.

Dies wird erreicht, indem ein einziger neuer Zustand, der *sink*, hinzugefügt wird, sowie Übergänge von allen anderen Zuständen zu diesem *sink* für alle Symbole, für die keine Übergänge vorhanden sind. Der *sink* selbst wird vollständig gemacht, indem Schleifenübergänge für alle Symbole hinzugefügt werden.

Zu beachten: Wenn ein FA über Epsilon-Übergänge verfügt, können Übergänge über ein Symbol für einen Zustand S indirekt sein, d. h. nicht direkt an S, sondern an einen Zustand in der Epsilon-Hülle von S gekoppelt. Die Symbole für solche indirekten Übergänge werden bei der Berechnung der Vollständigkeit eines Zustands berücksichtigt. Mit anderen Worten: Diese indirekt erreichbaren Symbole fehlen *nicht*.

Das Argument *sink* liefert den Namen für den neuen Zustand und darf, wenn angegeben, nicht bereits in der *fa* vorhanden sein. Wenn der Name nicht angegeben wird, benennt der Befehl den Zustand "sink**n**", wobei **n** so festgelegt wird, dass keine Kollisionen mit bestehenden Zuständen auftreten.

Hinweis: Der Sink-Zustand ist per Definition *nicht nützlich*. Mit anderen Worten: Zwar wird der FA dadurch vollständig, doch ist er im strengen Sinne *nicht nützlich*, da sie einen Zustand enthält, von dem aus kein Endzustand erreicht werden kann.

::grammar::fa::op::remove_eps *fa*

Entfernt alle Epsilon-Übergänge aus *fa* in einer Weise, dass die Sprache von *fa* unverändert bleibt. Es erfolgt jedoch keine Änderung, wenn *fa* bereits *epsilon-frei* ist. Dies bedeutet, dass nur die erste einer Reihe mehrerer aufeinanderfolgender remove_eps-Operationen auf *fa* eine Änderung bewirkt. Die übrigen Operationen sind Nulloperationen.

Hinweis: Diese Operation kann Zustände unerreichbar oder unbrauchbar machen. Diese Zustände entfernt die Operation nicht. Dafür ist **::grammar::fa::op::trim** zuständig.

::grammar::fa::op::trim *fa* ?*what*?

Entfernt unerwünschten Ballast aus *fa*. Die zulässigen Werte für *what* sind unten aufgeführt. Der Befehl verwendet standardmäßig den Wert**!reachable|!useful**, sofern kein spezifisches Argument angegeben wurde.

!reachable
: Entfernt alle Zustände, die von einem Startzustand aus nicht erreichbar sind.

!useful
: Entfernt alle Zustände, die nicht zu einem Endzustand führen können.

!reachable&!useful
: siehe unten ...

!(reachable|useful)
: Entfernt alle Zustände, die von einem Startzustand aus nicht erreichbar sind und von denen aus kein Endzustand erreicht werden kann.

!reachable|!useful
: siehe unten ...

!(reachable&useful)
: Entfernt alle Zustände, die von einem Startzustand aus nicht erreichbar sind oder von denen aus kein Endzustand erreicht werden kann.

::grammar::fa::op::determinize *fa* ?*mapvar*?

Macht den *fa* deterministisch, ohne die von dem *fa* akzeptierte Sprache zu ändern. Es erfolgt jedoch keine Aktion, wenn der *fa* bereits *deterministisch* ist. Dies bedeutet, dass nur die erste

einer Reihe mehrerer aufeinanderfolgender determinize-Operationen am *fa* eine Aktion ausführt. Die übrigen Operationen sind Nulloperationen.

Der Befehl speichert ein Dictionary, das die Beziehung zwischen den neuen Zuständen der resultierenden DFA und den Zuständen der Eingabe-NFA beschreibt, in *mapvar*, wenn dieses festgelegt wurde. Die Schlüssel des Dictionary sind die Handles für die Zustände der resultierenden DFA, die Werte sind Mengen von Zuständen aus der Eingabe-NFA.

Hinweis: Ein leeres Dictionary signalisiert, dass der Befehl den *fa* deterministisch machen konnte, ohne eine vollständige Teilmengenkonstruktion durchzuführen, sondern lediglich durch das Entfernen von Zuständen und das Umordnen von Übergängen (als Teil der Epsilon-Freiheit des FA).

Hinweis: Der Algorithmus kann die FA im technischen Sinne nicht deterministisch machen, wenn der FA keine Startzustände besitzt, da Determinismus erfordert, dass der FA genau einen Startzustand aufweist. In diesem Fall bemühen wir uns nach besten Kräften; der fehlende Startzustand ist dann die einzige Bedingung, die verhindert, dass das erzeugte Ergebnis *deterministisch* ist. Zu beachten ist außerdem, dass in diesem Fall die Möglichkeiten zum Entfernen von Zuständen aus dem FA ebenfalls stark eingeschränkt sind, da wir keine Zustände als unerreichbar deklarieren können.

::grammar::fa::op::minimize *fa* ?*mapvar*?

Erzeugt einen FA, der dieselbe Sprache wie *fa* akzeptiert, jedoch eine minimale Anzahl von Zuständen aufweist. Dazu wird die Methode von Brzozowski verwendet.

Der Befehl speichert ein Dictionary, das die Beziehung zwischen den neuen Zuständen der resultierenden minimalen FA und den Zuständen der Eingabe-FA beschreibt, in *mapvar*, wenn dies festgelegt wurde. Schlüssel des Dictionary sind die Handles für die Zustände der resultierenden minimalen FA, Werte sind Mengen von Zuständen aus der Eingabe-FA.

Hinweis: Ein leeres Dictionary signalisiert, dass der Befehl den *fa* minimieren konnte, ohne neue Zustände berechnen zu müssen. Dies sollte genau dann der Fall sein, wenn die Eingabe-FA bereits minimal war.

Hinweis: Wenn der Algorithmus keine Start- oder Endzustände hat, mit denen er arbeiten kann, ist das Ergebnis zwar technisch minimal, weist jedoch möglicherweise eine sehr unerwartete Struktur auf. Es ist zudem zu beachten, dass in diesem Fall die Möglichkeiten zum Entfernen von Zuständen aus dem FA erheblich eingeschränkt sind, da wir keine Zustände als unerreichbar deklarieren können.

Sprachoperationen Alle Operationen in diesem Abschnitt setzen voraus, dass alle Eingabe-FAs mindestens einen Start- und mindestens einen Endzustand aufweisen. Andernfalls ist die Sprache des FA nicht definiert, wodurch die Operation sinnlos wird (da sie auf definierte Weise auf den Sprachen der FAs operiert).

::grammar::fa::op::complement *fa*

Bildet das Komplement von *fa*. Das ist genau dann möglich, wenn *fa complete* und *deterministic* ist. Die resultierende FA akzeptiert die komplementäre Sprache von *fa*. Mit

anderen Worten: Alle Eingaben, die von *fa* nicht akzeptiert werden, werden von der resultierenden FA akzeptiert, und umgekehrt.

Der resultierende Automat verfügt über alle Zustände und Übergänge des Eingabeautomaten sowie über andere Endzustände.

::grammar::fa::op::kleene *fa*

Wendet den Kleene-Abschluss auf *fa* an. Der resultierende FA akzeptiert alle Zeichenfolgen **S**, für die wir eine natürliche Zahl **n** (einschließlich 0) und Zeichenfolgen **A1** ... **An** in der Sprache von *fa* finden können, sodass **S** die Verkettung von **A1** ... **An** ist. Mit anderen Worten: Die Sprache des Ergebnisses ist die unendliche Vereinigung aller Verkettungen endlicher Länge über der Sprache von *fa*.

Das Ergebnis enthält alle Zustände und Übergänge der Eingabe sowie neue Start- und Endzustände.

::grammar::fa::op::optional *fa*

Macht *fa* optional. Mit anderen Worten: Es berechnet den FA, der sowohl die Sprache von *fa* als auch das leere Wort (Epsilon) akzeptiert.

Das Ergebnis enthält alle Zustände und Übergänge der Eingabe sowie neue Start- und Endzustände.

::grammar::fa::op::union *fa fb* ?*mapvar*?

Kombiniert die endlichen Automaten *fa* und *fb* so, dass der resultierende endliche Automat die Vereinigung der Sprachen der beiden endlichen Automaten akzeptiert.

Das Ergebnis enthält alle Zustände und Übergänge der beiden Eingabe-Automaten sowie neue Start- und Endzustände. Alle Zustände von *fb*, die auch in *fa* vorhanden sind, werden umbenannt, und die *mapvar* enthält eine Zuordnung von den alten Zuständen von *fb* zu den neuen, sofern vorhanden.

Zu beachten ist, dass das Ergebnis nichtdeterministisch ist, selbst wenn die Eingaben deterministisch sind.

::grammar::fa::op::intersect *fa fb* ?*mapvar*?

Kombiniert die endlichen Automaten *fa* und *fb* so, dass der resultierende endliche Automat die Schnittmenge der Sprachen der beiden endlichen Automaten akzeptiert. Mit anderen Worten: Das Ergebnis akzeptiert ein Wort genau dann, wenn es sowohl von *fa* als auch von *fb* akzeptiert wird. Das Ergebnis ist nützlich, jedoch nicht zwangsläufig deterministisch oder minimal.

Der Befehl speichert ein Dictionary, das die Beziehung zwischen den neuen Zuständen der resultierenden FA und den Zustandspaaren der Eingabe-FAs beschreibt, in *mapvar*, wenn dieses festgelegt wurde. Schlüssel des Dictionary sind die Handles für die Zustände der resultierenden FA, Werte sind Zustandspaare aus den Eingabe-FAs. Paare werden durch Listen dargestellt. Das erste Element jedes Paares ist ein Zustand in *fa*, das zweite Element stammt aus *fb*.

::grammar::fa::op::difference *fa fb* ?*mapvar*?

Kombiniert die endlichen Automaten *fa* und *fb* derart, dass der resultierende endliche

Automat die Differenz der Sprachen der beiden endlichen Automaten akzeptiert. Mit anderen Worten: Das Ergebnis akzeptiert ein Wort genau dann, wenn das Wort von *fa* akzeptiert, von *fb* jedoch nicht akzeptiert wird. Dies lässt sich auch als Schnittmenge von *fa* mit dem Komplement von *fb* ausdrücken. Das Ergebnis ist zwar nützlich, aber nicht zwangsläufig deterministisch oder minimal.

Der Befehl speichert ein Dictionary, das die Beziehung zwischen den neuen Zuständen der resultierenden FA und den Zustandspaaren der Eingabe-FAs beschreibt, in *mapvar*, wenn dies festgelegt wurde. Schlüssel des Dictionary sind die Handles für die Zustände der resultierenden FA, Werte sind Zustands-Paare aus den Eingabe-FAs. Paare werden durch Listen dargestellt. Das erste Element jedes Paares ist ein Zustand in *fa*, das zweite Element stammt aus *fb*.

::grammar::fa::op::concatenate *fa fb* ?*mapvar*?

Kombiniert die endlichen Automaten *fa* und *fb* derart, dass der resultierende endliche Automat die Verkettung der Sprachen der beiden endlichen Automaten akzeptiert. Das heißt, ein Wort W wird von der resultierenden FA akzeptiert, wenn es zwei Wörter A und B gibt, die jeweils von *fa* bzw. *fb* akzeptiert werden, und W die Verkettung von A und B ist.

Die resultierende FA ist nichtdeterministisch.

::grammar::fa::op::fromRegex *fa regex* ?*over*?

Erzeugt einen nichtdeterministischen endlichen Automaten (FA), der dieselbe Sprache akzeptiert wie der reguläre Ausdruck *regex*. Wenn *over* festgelegt wird, wird dieser als die Menge der Symbole behandelt, über die der reguläre Ausdruck und der Automat definiert sind. Der Befehl berechnet die Menge aus den "S"-Konstruktoren in *regex*, wenn *over* nicht angegeben wurde. Diese Menge ist genau dann von Bedeutung, wenn der Komplementoperator "!" in *regex* verwendet wird, da sich die Komplementsprache eines FA je nach Symbolmenge erheblich unterscheidet.

Der reguläre Ausdruck wird durch eine verschachtelte Liste dargestellt, die einen Syntaxbaum bildet. Folgende Strukturen sind zulässig:

{S x}

Atomare reguläre Ausdrücke. Alle anderen Ausdrücke werden daraus gebildet. Akzeptiert das **S**ymbol "x".

{. A1 A2 ...}

Verkettungsoperator. Akzeptiert die Verkettung der regulären Ausdrücke **A1**, **A2** usw.

Hinweis: Dieser Operator akzeptiert null oder mehr Argumente. Bei null Argumenten ist die dargestellte Sprache *epsilon*, das leere Wort.

{| A1 A2 ...}

Auswahloperator, auch "Alternative" genannt. Akzeptiert alle Eingaben, die von mindestens einem der regulären Ausdrücke **A1**, **A2** usw. akzeptiert werden. Mit anderen Worten: die Vereinigung von **A1** und **A2**.

Hinweis: Dieser Operator akzeptiert null oder mehr Argumente. Bei null Argumenten ist die dargestellte Sprache die Sprache *empty*, die Sprache ohne Wörter.

{& A1 A2 ...}

Schnittoperator, logisches und. Akzeptiert alle Eingaben, die von allen regulären Ausdrücken **A1**, **A2** usw. akzeptiert werden. Mit anderen Worten: die Schnittmenge von **A1** und **A2**.

{? A}
Optionalitätsoperator. Akzeptiert das leere Wort sowie alle Elemente aus dem regulären Ausdruck **A**.

{* A}
Kleene-Abschluss. Akzeptiert das leere Wort sowie jede endliche Verkettung von Wörtern, die vom regulären Ausdruck **A** akzeptiert werden.

{+ A}
Positiver Kleene-Abschluss. Akzeptiert jede endliche Verkettung von Wörtern, die vom regulären Ausdruck **A** akzeptiert werden, jedoch nicht das leere Wort.

{! A}
Komplement-Operator. Akzeptiert jedes Wort, das vom regulären Ausdruck **A** nicht akzeptiert wird. Beachten Sie, dass das Komplement von der Symbolmenge abhängt, über die das Ergebnis laufen soll. Siehe dazu die Erläuterung zum Argument *over* weiter oben.

::grammar::fa::op::toRegexp *fa*
Dieser Befehl generiert und gibt einen regulären Ausdruck zurück, der dieselbe Sprache akzeptiert wie der endliche Automat *fa*. Der reguläre Ausdruck hat das oben beschriebene Format, wie es für **::grammar::fa::op::fromRegex** gilt.

::grammar::fa::op::toRegexp2 *fa*
Dieser Befehl verfügt über dieselbe Funktionalität wie **::grammar::fa::op::toRegexp**, verwendet jedoch einen anderen Algorithmus, um die generierten regulären Ausdrücke zu vereinfachen.

::grammar::fa::op::toTclRegexp *regexp symdict*
Dieser Befehl generiert und gibt einen regulären Ausdruck in Tcl-Syntax für den regulären Ausdruck *regexp* zurück, sofern dies möglich ist. *regexp* hat dasselbe Format, wie es von **::grammar::fa::op::fromRegex** erwartet wird.

Der Befehl schlägt fehl und löst einen Fehler aus, wenn *regexp* Komplementierungs- und Schnittoperationen enthält.

Das Argument *symdict* ist ein Dictionary, das Symbolnamen Paaren aus *syntaktischem Typ* und Tcl-Regexp zuordnet. Wenn ein in *regexp* vorkommendes Symbol nicht in diesem Dictionary aufgeführt ist, werden Ein-Zeichen-Symbole so interpretiert, dass sie sich selbst bezeichnen, während Mehrzeichen-Symbole als Name einer Zeichenklasse betrachtet werden.

::grammar::fa::op::simplifyRegexp *regexp*
Dieser Befehl vereinfacht einen regulären Ausdruck, indem er den folgenden Algorithmus zunächst auf den Hauptausdruck und anschließend rekursiv auf alle Teilausdrücke anwendet:

1. Wandeln Sie den Ausdruck in einen endlichen Automaten um.
2. Minimieren Sie den Automaten.
3. Wandeln Sie den Automaten wieder in einen regulären Ausdruck um.
4. Wählen Sie den kürzeren der beiden Ausdrücke aus: den ursprünglichen Ausdruck oder den Ausdruck aus dem vorherigen Schritt.

Beispiele

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_fa* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2004-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


grammar_me

gasm

Stichwörter: gasm, assembler, grammar, graph, parsing, tree, virtual machine

Name

grammar::me::cpu::gasm – ME-Assembler

Übersicht

package require grammar::me::cpu::gasm ?**0.2**?
::grammar::me::cpu::gasm::begin *g n* ?*mode*? ?*note*?
::grammar::me::cpu::gasm::done --> *t*
::grammar::me::cpu::gasm::state
::grammar::me::cpu::gasm::state! *s*
::grammar::me::cpu::gasm::lift *t dst* **=** *src*
::grammar::me::cpu::gasm::Inline *t node label*
::grammar::me::cpu::gasm::Cmd *cmd* ?*arg...*?
::grammar::me::cpu::gasm::Bra
::grammar::me::cpu::gasm::Nop *text*
::grammar::me::cpu::gasm::Note *text*
::grammar::me::cpu::gasm::Jmp *label*
::grammar::me::cpu::gasm::Exit
::grammar::me::cpu::gasm::Who *label*
::grammar::me::cpu::gasm::/Label *name*
::grammar::me::cpu::gasm::/Clear
::grammar::me::cpu::gasm::/Ok
::grammar::me::cpu::gasm::/Fail
::grammar::me::cpu::gasm::/At *name*

::grammar::me::cpu::gasm::/CloseLoop

Beschreibung

Dieses Paket stellt einen einfachen In-Memory-Assembler bereit. Es entstand ursprünglich als Unterstützungspaket für Pakete, die PEG- und andere Grammatiken in einen entsprechenden Matcher auf Basis der virtuellen ME-Maschine konvertieren, wie beispielsweise **page::compiler::peg::mecpu**. Obwohl es hinsichtlich der Befehle weitgehend agnostisch ist, können Anwender jeden beliebigen Befehlssatz wählen.

Das in der Erstellung befindliche Programm wird während der Assemblierung und der anschließenden Bearbeitung in einer Graphstruktur gespeichert (siehe Paket **struct::graph**), wobei Anweisungen durch Knoten dargestellt werden und der Ausführungsablauf zwischen den Anweisungen explizit in den Kanten zwischen diesen kodiert ist.

In diesem Modell erfolgt die Kodierung von Sprüngen nicht explizit, sondern sie ist implizit in den Bögen enthalten. Die Generierung expliziter Sprünge wird einem beliebigen Code überlassen, der die Graphstruktur in eine konventionellere Darstellung umwandelt. Das Gleiche gilt für Verzweigungen. Sie werden implizit durch alle Anweisungen kodiert, die zwei ausgehende Bögen aufweisen, während alle anderen Anweisungen nur einen ausgehenden Bogen haben. Ihre Bedingtheit wird dadurch gewährleistet, dass ihre ausgehenden Bögen mit Tags über die Bedingungen versehen werden, unter denen sie ausgeführt werden.

Zwar wird die Grafik, auf der der Assembler arbeitet, von außen bereitgestellt, d. h. sie ist extern, doch verwaltet er einen gewissen internen Zustand, und zwar:

1. Der Handle des Graphenknotens, auf den die meisten Assembler-Operationen angewendet werden können: der *anchor*.
2. Eine Zuordnung beliebiger Zeichenketten auf Befehle. Ein Befehl lässt sich während der Assemblierung *label* und später über dieses Label wieder abrufen.
3. Der beim Erstellen von Sprüngen zwischen Anweisungen zu verwendende Statuscode, der einer der folgenden Werte sein kann: **always**, **ok** oder **fail**.
4. Der aktuelle Betriebsmodus: entweder **halt**, **okfail** oder**!okfail**.
5. Der Name eines Knotens in einem Baum. Dieser sowie der oben genannte Betriebsmodus sind die Komponenten, die am stärksten von den Anforderungen eines Grammatik-Compilers beeinflusst werden, da sie bestimmte grundlegende Programmstrukturen voraussetzen (die über den Betriebsmodus ausgewählt werden) und den Graphen mit einem Baum verknüpfen, wie beispielsweise dem AST für die zu kompilierende Grammatik.

Definitionen

Da sowohl der Graph, auf dem der Assembler arbeitet, als auch der Baum, mit dem er verflochten ist, dem Assembler von außen bereitgestellt werden, ist es erforderlich, die von ihnen erwartete API festzulegen und die Strukturen zu beschreiben, die der Assembler in beiden Fällen erwartet und/oder erzeugt.

1. Jeder vom Assembler verwendete Objektbefehl muss die API bereitstellen, wie sie in der Dokumentation zum Paket **struct::graph** angegeben ist.
2. Jeder vom Assembler verwendete Objektbefehl für Baumobjekte muss die API bereitstellen, wie sie in der Dokumentation zum Paket **struct::tree** angegeben ist.
3. Jede vom Assembler in einem Graphen erzeugte Anweisung (Knoten) verfügt über mindestens zwei und höchstens drei Attribute:

instruction
Der Wert dieses Attributs ist der Name des Befehls. Die einzigen Namen, die derzeit vom Assembler definiert sind, sind die drei Pseudobefehle

NOP
Diese Anweisung hat keine Auswirkung. Nützlich für feste Rahmen-Knoten, unveränderliche Sprungziele und Ähnliches. Keine Argumente.

C
Ein .NOP, der das Einfügen beliebiger Kommentare in den Befehlsstrom ermöglicht, d. h. ein Kommentarknoten. Ein Argument: der Text des Kommentars.

BRA
Ein .NOP, der als explizit codierter bedingter Sprung dient. Keine Argumente.

```
    Wir behalten uns jedoch den Speicherplatz aller Befehle vor, deren
    Namen mit einem "." (Punkt) beginnen, für eine zukünftige
    Verwendung durch den Assembler.

**arguments**
: Der Wert dieses Attributs ist eine Liste von Zeichenketten, bei
    denen es sich um die Argumente des Befehls handelt. Der Inhalt
    hängt vom jeweiligen Befehl ab, und der Assembler kennt diese
    Argumente nicht und berücksichtigt sie auch nicht. Das bedeutet
    beispielsweise, dass er kein integriertes Wissen darüber besitzt,
    welcher Befehl welche Argumente benötigt, und daher keinerlei
    Überprüfung durchführt.

**expr**
: Dieses Attribut ist optional. Wenn es vorhanden ist, entspricht sein
    Wert dem Namen eines Knotens in der Baumstruktur, die mit dem
    Graphen verflochten ist.
```

1. Jeder Bogen zwischen zwei Befehlen weist ein Attribut auf:

condition
Der Wert dieses Attributs legt fest, unter welcher Bedingung die Ausführung diesen Bogen nimmt. Mögliche Werte sind **always**, **ok** und **fail**. Die erste Bedingung gilt für alle Bögen, die den einzigen ausgehenden Bogen einer Anweisung bilden. Die beiden anderen gelten für die beiden ausgehenden Bögen einer Anweisung, die implizit eine Verzweigung kodieren.

1. Ein Baumknoten, der dem Assembler zur Querverweisung übergeben wird, wird beschrieben und erhält die folgenden Attribute, von denen einige fest vorgegeben sind, andere hingegen vom Betriebsmodus abhängen. Alle Werte stellen Verweise auf Knoten im Befehlsgraphen dar. Einige Befehle erwarten bestimmte oder spezifische Kombinationen dieser Attribute.

gas::entry
Immer geschrieben.

gas::exit
Gilt für alle Modi mit Ausnahme von **okfail**.

gas::exit::ok
Verfasst für den Modus **okfail**.

gas::exit::fail
Verfasst für den Modus **okfail**.

API

::grammar::me::cpu::gasm::begin *g n* ?*mode*? ?*note*?

Dieser Befehl startet die Assemblierung einer Befehlsfolge und (re-)initialisiert den Zustand des Assemblers. Nach Abschluss der Befehlsfolge verwenden Sie bitte **::grammar::me::cpu::gasm::done**, um den Assembler abzuschließen.

Er arbeitet mit dem Graphen *g* im angegebenen *Modus* (Standard ist **okfail**). Im Rahmen der Initialisierung wird stets ein Standard-NOP-Befehl erstellt und mit dem Label "entry" versehen. Die Erstellung der übrigen Standardbefehle erfolgt gemäß *mode*-dependent:

halt
Eine "icf_halt"-Anweisung mit der Bezeichnung "exit/return".

!okfail
Ein "icf_ntreturn"-Befehl mit der Bezeichnung "exit/return".

okfail
Zwei .NOP-Befehle mit den Bezeichnungen "exit/ok" bzw. "exit/fail".

Die *note*, wenn sie festgelegt wird (Standard ist "nicht"), wird dem "entry"-Befehl .NOP zugewiesen.

Die Knotenreferenz *n* wird lediglich zur Verwendung durch **::grammar::me::cpu::gasm::done** gespeichert. Sie muss sich auf einen Knoten im Baum *t* -Argument dieses Befehls beziehen.

Nach Abschluss der Initialisierung wird der Befehl "entry" zum *anchor* und der Bedingungscode wird auf **always** gesetzt.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::done --> *t*

Dieser Befehl schließt die Erstellung einer Befehlsfolge ab und löscht anschließend den Status des Assemblers. *Hinweis*: Dadurch werden die erstellten Befehle *nicht* gelöscht. Sie stehen für zukünftige "begin/done"-Zyklen weiterhin zur Verfügung. Eine weitere Assemblierung ist erst nach einer Neuinitialisierung des Systems über **::grammar::me::cpu::gasm::begin** möglich.

Bevor der Zustand gelöscht wird, werden ausgewählte Verweise auf ausgewählte Befehle in die Attribute des Knotens *n* im Baum *t* geschrieben. Welche Befehle gespeichert werden, hängt vom *mode* ab. Sowohl *mode* als auch der Zielknoten *n* wurden beim Aufruf von **::grammar::me::cpu::gasm::begin** festgelegt.

Unabhängig vom Modus wird ein Verweis auf die Anweisung mit der Bezeichnung "entry" im Attribut **gas::entry** von *n* gespeichert. Der Verweis auf den Knoten *n* wird zudem im Attribut "expr" der Anweisung "entry" gespeichert. Darüber hinaus

halt
Ein Verweis auf die Anweisung mit der Bezeichnung "exit/return" wird im Attribut **gas::exit**

von *n* gespeichert.

okfail
Siehe **halt**.

!okfail
Der Verweis auf die beiden Anweisungen mit den Bezeichnungen "exit/ok" und "exit/fail" wird jeweils in den Attributen **gas::exit::ok** und **gas::exit::fail** von *n* gespeichert.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::state
Dieser Befehl gibt den aktuellen Status des Assemblers zurück. Sein Format ist nicht dokumentiert und gilt als paketintern.

::grammar::me::cpu::gasm::state! *s*
Dieser Befehl übernimmt einen serialisierten Assembler-Zustand *s*, wie er von **::grammar::me::cpu::gasm::state** gegeben wird, und macht ihn zum aktuellen Zustand des Assemblers.

Hinweis: Dadurch können Label-Definitionen überschrieben werden. Alle nicht in Konflikt stehenden Label-Definitionen aus dem vorherigen Zustand bleiben jedoch unberührt und werden mit *s* zusammengeführt.

Der Befehl gibt die leere Zeichenkette als Ergebnis zurück.

::grammar::me::cpu::gasm::lift *t dst = src*
Dieser Befehl wirkt auf den Baum *t* ein. Er kopiert den Inhalt der Attribute **gas::entry**, **gas::exit::ok** und **gas::exit::fail** vom Knoten *src* in den Knoten *dst*. Als Ergebnis gibt es die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::Inline *t node label*
Dieser Befehl verknüpft eine durch ein zuvor eingegebenes "begin/done"-Paar erstellte Befehlsfolge mit der aktuellen Befehlsfolge.

Zu diesem Zweck
1. liest die Befehlsverweise aus den Attributen **gas::entry**, **gas::exit::ok** und **gas::exit::fail** aus dem Knoten *n* des Baums *t* aus und stellt sie dem Assembler unter den Labels *label*/entry, *label*/exit::ok bzw. *label*/exit::fail zur Verfügung.
2. Erstellt einen Bogen vom *anchor* zum Knoten mit der Bezeichnung *label*/entry und versieht ihn mit dem aktuellen Bedingungscode.
3. Macht den Knoten mit der Bezeichnung *label*/exit/ok zum neuen *anchor*.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::Cmd *cmd* ?*arg*...?
Dies ist der grundlegende Befehl zum Hinzufügen von Anweisungen zum Diagramm. Er erstellt eine neue Anweisung vom Typ *cmd* mit den angegebenen Argumenten *arg*... Wenn *anchor* definiert wurde, wird zudem unter Verwendung des aktuellen Bedingungscodes ein Bogen vom *anchor* zur neuen Anweisung erstellt. Nach dem Aufruf wird die neue Anweisung zum *anchor* und der aktuelle Bedingungscode wird auf **always** gesetzt.

Der Befehl gibt die leere Zeichenkette als Ergebnis zurück.

::grammar::me::cpu::gasm::Bra
Dieser Befehl dient zum Erstellen einer .BRA-Pseudo-Anweisung. Er nutzt **::grammar::me::cpu::gasm::Cmd**, um die Anweisung tatsächlich zu erstellen, und übernimmt deren Verhalten.

::grammar::me::cpu::gasm::Nop *text*
Dies ist ein Komfort-Befehl zum Erstellen einer .NOP-Pseudo-Anweisung. Er nutzt **::grammar::me::cpu::gasm::Cmd**, um die Anweisung tatsächlich zu erstellen, und übernimmt deren Verhalten. Der *text* wird als erstes und einziges Argument der neuen Anweisung gespeichert.

::grammar::me::cpu::gasm::Note *text*
Dies ist ein Komfort-Befehl zum Erstellen einer .C-Pseudo-Anweisung, d. h. eines Kommentars. Er nutzt **::grammar::me::cpu::gasm::Cmd**, um die Anweisung tatsächlich zu erstellen, und übernimmt deren Verhalten. Der *text* wird als erstes und einziges Argument der neuen Anweisung gespeichert.

::grammar::me::cpu::gasm::Jmp *label*
Dieser Befehl erstellt einen Bogen vom *anchor* zur mit *label* gekennzeichneten Anweisung und versieht diese mit dem aktuellen Bedingungscode.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::Exit
Dieser Befehl erstellt je nach Betriebsmodus (siehe **::grammar::me::cpu::gasm::begin**) einen Bogen vom *anchor* zu einer der Exit-Anweisungen und versieht diesen mit dem aktuellen Bedingungscode.

Im Modus **okfail** wird je nach aktuellem Bedingungscode eine Verknüpfung zu der Anweisung "exit/ok" oder "exit/fail" hergestellt und diese mit dem aktuellen Bedingungscode versehen. In den beiden anderen Modi wird eine Verknüpfung zu der Anweisung "exit/return" hergestellt, und versieht diesen Befehl **always** mit dem Bedingungscode, unabhängig vom aktuellen Bedingungscode.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::Who *label*
Dieser Befehl gibt eine Referenz auf die mit *label* gekennzeichnete Anweisung zurück.

::grammar::me::cpu::gasm::/Label *name*
Dieser Befehl versieht den *anchor* mit der Bezeichnung *name*. *Hinweis*: Eine Anweisung kann mehr als eine Bezeichnung haben.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::/Clear
Dieser Befehl löscht den *anchor*, sodass dieser nicht mehr definiert ist, und setzt zudem den aktuellen Bedingungscode auf **always** zurück.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::/Ok
Dieser Befehl setzt den aktuellen Bedingungscode auf **ok**.

Der Befehl gibt als Ergebnis eine leere Zeichenkette zurück.

::grammar::me::cpu::gasm::/Fail
Dieser Befehl setzt den aktuellen Bedingungscode auf **fail**.

Der Befehl gibt als Ergebnis eine leere Zeichenkette zurück.

::grammar::me::cpu::gasm::/At *name*
Dieser Befehl setzt den *anchor* auf die mit *name* bezeichnete Anweisung und setzt zudem den aktuellen Bedingungscode auf **always** zurück.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::grammar::me::cpu::gasm::/CloseLoop
Dieser Befehl kennzeichnet den *anchor* als letzte Anweisung im Schleifenrumpf, indem er das Attribut **LOOP** erstellt.

Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_ast

Stichwörter: me_ast, AST, abstract syntax tree

Name

grammar::me_ast – Verschiedene Darstellungen von ASTs

Beschreibung

Dieses Dokument legt verschiedene Darstellungsformen für die *abstrakten Syntaxbäume* (kurz *AST*), die von Instanzen virtueller ME-Maschinen generiert werden, fest, unabhängig von der jeweiligen Variante. Bitte lesen Sie zunächst das Dokument **grammar::me_intro**, wenn Sie nicht wissen, was eine virtuelle ME-Maschine ist.

ASTs und alle von uns festgelegten Darstellungen unterscheiden zwischen zwei Arten von

Knoten, nämlich:

Terminal
: Terminalknoten beziehen sich auf die im Tokenstrom vorkommenden Terminalsymbole. Es sind stets Blattknoten. Terminalknoten haben niemals Kindknoten.

Nonterminal
: Nichtterminale Knoten stellen ein nichtterminales Symbol der bei der Syntaxanalyse verwendeten Grammatik dar. Sie können als Blattknoten und als innere Knoten des Baums vorkommen.

Beide Knotentypen enthalten grundlegende Bereichsinformationen, die dem Benutzer mitteilen, welche Teile der Eingabe von dem Knoten abgedeckt werden, indem sie die Position des ersten und letzten Tokens angeben, die innerhalb dieses Bereichs gefunden wurden. Die Positionen werden als nicht-negative ganzzahlige Offsets vom Anfang des Token-Stroms angegeben, wobei das erste im Strom gefundene Token am Offset 0 (Null) liegt.

Die Wurzel eines AST kann entweder ein Terminal- oder ein Nichtterminalknoten sein.

AST-Werte

Diese Darstellung von ASTs erfolgt als Tcl-Liste. Die Hauptliste stellt den Wurzelknoten des Baums dar, wobei die Darstellungen der Kindknoten darin verschachtelt sind.

Jeder Knoten wird durch eine einzelne Tcl-Liste dargestellt, die drei oder mehr Elemente enthält. Das erste Element ist entweder die leere Zeichenkette oder der Name eines nichtterminalen Symbols (der niemals die leere Zeichenkette ist). Das zweite und dritte Element geben dann die Positionen des ersten bzw. letzten Tokens an. Alle weiteren Elemente nach dem dritten sind die Darstellungen der Kindknoten, wobei der am weitesten links stehende Kindknoten zuerst kommt, also als viertes Element der Liste, die den Knoten darstellt.

AST-Objekte

In dieser Darstellung wird ein AST durch einen Tcl-Objektbefehl dargestellt, dessen API mit den Baumobjekten des Pakets **struct::tree** kompatibel ist. Das heißt, er muss mindestens alle von diesem Paket beschriebenen Methoden unterstützen und kann weitere unterstützen.

Aus diesem Grund werden die übrigen Spezifikationen unter Verwendung der Begriffe aus **struct::tree** verfasst.

Jeder Knoten des AST ist direkt einem Knoten im Baum-Objekt zugeordnet. Alle Daten jenseits der Kindknoten, also Knotentyp und Eingabepositionen, werden in Attributen des Knotens im Baumobjekt gespeichert. Diese sind:

type
: Der Typ des AST-Knotens. Die zulässigen Werte sind **terminal** und **nonterminal**.

range
: Die Positionen des ersten und letzten Tokens der Terminal-Daten in der vom Knoten abgedeckten Eingabe. Hierbei handelt es sich um eine Liste, die zwei Positionen enthält.

detail
: Dieses Attribut ist nur bei nichtterminalen Knoten vorhanden. Es enthält den Namen des im Knoten gespeicherten nichtterminalen Symbols.

Erweiterte AST-Objekte

Erweiterte AST-Objekte entsprechen AST-Objekten, enthalten jedoch zusätzliche Informationen.

detail
: Dieses Attribut ist nun an allen Knoten vorhanden. Sein Inhalt bleibt bei nichtterminalen Knoten unverändert. Bei terminalen Knoten enthält es eine Liste, die alle Token aus der Eingabe beschreibt, die von dem Knoten abgedeckt werden.

  Jedes Element der Liste enthält in dieser Reihenfolge den Tokennamen, das zugehörige Lexem-Attribut, die Zeilennummer und den Spaltenindex.

range_lc
: Dieses neue Attribut ist für alle Knoten definiert und enthält die Positionen aus dem Attribut *range*, umgerechnet in Zeilennummern und Spalten-Indizes. Die Zeilen werden ab 1 gezählt, die Spalten ab 0.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_cpu

Stichwörter: me_cpu, grammar, parsing, virtual machine

Name

grammar::me::cpu – Implementierung einer virtuellen Maschine II zur Analyse von Token-Strömen

Übersicht

package require Tcl 8.5 9
package require grammar::me::cpu ?**0.3**?
::grammar::me::cpu *meName matchcode*
meName option ?*arg arg* ...?
meName **lc** *location*
meName **tok** ?*from ?to*?
meName **pc** *state*
meName **iseof** *state*

meName **at** *state*
meName **cc** *state*
meName **sv**
meName **ok**
meName **error**
meName **lstk** *state*
meName **astk** *state*
meName **mstk** *state*
meName **estk** *state*
meName **rstk** *state*
meName **nc** *state*
meName **ast**
meName **halted**
meName **code**
meName **eof**
meName **put** *tok lex line col*
meName **putstring** *string lvar cvar*
meName **run** ?*n*?
meName **pull** *nextcmd*
meName **reset**
meName **destroy**

Beschreibung

Dieses Paket enthält eine Implementierung der virtuellen ME-Maschine. Bitte lesen Sie zunächst das Dokument **grammar::me_intro**, falls Sie nicht wissen, was eine virtuelle ME-Maschine ist.

Diese Implementierung bietet eine objektbasierte API, und die Maschinen sind nicht wirklich an Tcl gebunden. Eine C-Implementierung derselben API ist durchaus möglich.

Intern nutzt das Paket zur Erfüllung seiner Aufgaben tatsächlich die wertbasierten Befehle zur Maschinenmanipulation, wie sie vom Paket **grammar::me::cpu::core** bereitgestellt werden.

API

API für Klassen

Das Paket stellt direkt nur einen einzigen Befehl für die Erstellung von virtuellen ME-Maschinen bereit.

::grammar::me::cpu *meName matchcode*

Der Befehl erstellt ein neues ME-Maschinenobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *meName* lautet. Mit diesem Befehl können verschiedene Operationen auf der Maschine aufgerufen werden. Er hat die folgende allgemeine Form:

meName option ?*arg arg ...*?

Option und die *arg*s bestimmen das genaue Verhalten des Befehls.

Das Argument *matchcode* enthält die Abgleichanweisungen, die die Maschine beim Parsen des Eingabestroms ausführen muss. Bitte lesen Sie den Abschnitt **Darstellung des Match-Programms** der Dokumentation zum Paket **grammar::me::cpu::core**, um die Struktur dieses Werts zu erfahren.

Das Argument *tokmap*, das von der durch das Paket **grammar::me::tcl** bereitgestellten Implementierung verwendet wird, ist hier in den Match-Anweisungen verborgen und wird daher nicht benötigt.

Objekt-API

Alle Objekte virtueller Maschinen vom Typ "ME", die mit dem im Abschnitt API für Klassen angegebenen Klassenbefehl erstellt wurden, unterstützen die unten aufgeführten Methoden.

Die in diesem Paket enthaltenen Maschinen bieten Methoden für den Betrieb sowohl im Push- als auch im Pull-Style. "Push"-Modus bedeutet, dass Tokens bei ihrem Eintreffen in den Maschinenzustand geschoben werden und dort die weitere Ausführung auslösen, bis sie verbraucht sind. Mit anderen Worten: Dies ermöglicht es, die Maschine nach Belieben und beliebig oft anzuhalten und wieder aufzunehmen, den quasi-parallelen Betrieb mehrerer Maschinen sowie den Betrieb als Teil der Ereignisschleife.

meName* lc *location

Diese Methode wandelt die Position eines Tokens, das als Offset im Eingabestrom angegeben wird, in die zugehörige Zeilennummer und den Spaltenindex um. Das Ergebnis des Befehls ist eine Liste mit zwei Elementen, die diese beiden Werte in der im vorigen Satz genannten Reihenfolge enthält. Dies ermöglicht es übergeordneten Ebenen, die im Fehlerstatus und im generierten AST enthaltenen Positionsinformationen in besser lesbare Daten umzuwandeln.

Hinweis: Der Befehl kann keine Positionen umwandeln, die von der Maschine noch nicht erreicht wurden. Mit anderen Worten: Wenn die Maschine 7 Token gelesen hat, kann der Befehl die Offsets **0** bis **6** umwandeln, jedoch nichts darüber hinaus. Dies zeigt auch, dass es nicht möglich ist, Offsets zu konvertieren, die sich auf Positionen vor dem Beginn des Stroms beziehen.

***meName* tok ?*from* ?*to*??**

Diese Methode gibt eine Tcl-Liste zurück, die den Teil des Eingabestroms zwischen den Positionen *from* und *to* (beide inklusive) enthält. Wenn *to* nicht festgelegt wird, wird standardmäßig der Wert von *from* verwendet. Wenn *from* ebenfalls nicht festgelegt wird, wird der gesamte Eingabestrom zurückgegeben.

Jedes Element der zurückgegebenen Liste ist eine Liste aus vier Elementen: dem Token, dem zugehörigen Lexem, der Zeilennummer und dem Spaltenindex, in dieser Reihenfolge. Dieser Befehl unterliegt hinsichtlich seiner Positionsargumente denselben Einschränkungen wie die Methode **lc**.

meName* pc *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den aktuellen Wert des gespeicherten Programmzählers zurück.

meName* iseof *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den aktuellen Wert des gespeicherten EOF-Flags zurück.

meName* at *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt die aktuelle Position im Eingabestrom zurück.

meName* cc *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt das aktuelle Token zurück.

***meName* sv**

Dieser Befehl gibt den aktuellen semantischen Wert *SV* zurück, der in der Maschine gespeichert ist. Dabei handelt es sich um einen abstrakten Syntaxbaum, wie er im Dokument **grammar::me_ast**, Abschnitt **AST-WERTE**, beschrieben ist.

***meName* ok**

Diese Methode gibt den aktuellen Match-Status *OK* zurück.

***meName* error**

Diese Methode gibt den aktuellen Fehlerstatus *ER* zurück.

meName* lstk *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den Positionsstapel zurück.

meName* astk *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den AST-Stack zurück.

meName* mstk *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den AST-Marker-Stack zurück.

meName* estk *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den Fehlerstapel zurück.

meName* rstk *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den Rückgabestapel der Unterroutine zurück.

meName* nc *state

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den Cache für Nichtterminal-Übereinstimmungen als Dictionary zurück.

***meName* ast**

Diese Methode gibt den aktuellen obersten Eintrag des AST-Stapels *AS* zurück. Dabei handelt es sich um einen abstrakten Syntaxbaum, wie er im Dokument **grammar::me_ast**, Abschnitt **AST-WERTE**, festgelegt ist.

***meName* halted**

Diese Methode gibt einen booleschen Wert zurück, der dem Aufrufer mitteilt, ob die Engine die Ausführung angehalten hat oder nicht. "Halt" bedeutet, dass kein weiterer Abgleich mehr möglich ist und die über die andere Methode abgerufenen Informationen endgültig sind. Versuche, die Maschine mit **run** auszuführen, werden ignoriert, bis ein **reset** erfolgt.

***meName* code**

Diese Methode gibt die *code*-Information zurück, mit der das Objekt erzeugt wurde. Mit anderen Worten das Match-Programm, das die Maschine ausführt.

***meName* eof**

Diese Methode fügt am Ende des Eingabestroms eine Dateiende-Markierung ein. Dies signalisiert der Maschine, dass der aktuelle Inhalt der Eingabewarteschlange den letzten Teil der Eingabe darstellt und nichts mehr folgen wird. Versuche, weitere Zeichen in die

Warteschlange einzufügen, schlagen fehl.

meName* put *tok lex line col
Diese Methode fügt das Token *tok* am Ende des Eingabestroms hinzu, zusammen mit den zugehörigen Lexemdaten *lex* und den Zeilen- und Spaltenangaben *line* und *col*.

meName* putstring *string lvar cvar
Bei dieser Methode wird jedes einzelne Zeichen der *string* als Token vom ersten bis zum letzten Zeichen an das Ende des Eingabestroms angehängt. Die Lexeme sind leer, und die Zeilen- und Spaltenangaben werden auf der Grundlage der gefundenen Zeichen sowie der Daten in den Variablen *lvar* und *cvar* berechnet.

***meName* run ?*n*?**
Diese Methode bewirkt, dass die Maschine Abgleichanweisungen ausführt, bis entweder
- *n* Anweisungen wurden ausgeführt, oder
- ein Haltebefehl ausgeführt wurde oder
- Die Eingabewarteschlange ist leer und der Code fordert weitere Token zur Verarbeitung an.

Wenn kein Grenzwert *n* festgelegt wurde, werden nur die letzten beiden Bedingungen geprüft.

meName* pull *nextcmd
Diese Methode implementiert den Pull-Betrieb der Maschine. Sie bewirkt, dass die Maschine Match-Anweisungen ausführt, bis entweder eine Stoppanweisung erreicht wird oder das Befehlspräfix *nextcmd* keine weiteren Token mehr liefert.

Das Befehlspräfix *nextcmd* repräsentiert den Eingabestrom aus Zeichen und wird von der Maschine immer dann aufgerufen, wenn ein neues Zeichen aus dem Strom benötigt wird. Die Anweisung zur Handhabung hierfür lautet *ict_advance*. Die Rückruffunktion muss entweder die leere Liste oder eine Liste mit vier Elementen zurückgeben, die das Token, dessen Lexem-Attribut sowie dessen Position als Zeilennummer und Spalten-Index in dieser Reihenfolge enthalten. Die leere Liste signalisiert, dass das Ende des Eingabestroms erreicht wurde. Das Lexem-Attribut wird im Terminal-Cache gespeichert, von der Maschine jedoch ansonsten nicht verwendet.

Das Ende des Eingabestroms für diese Methode bedeutet nicht, dass die Methode **eof** für die Maschine als Ganzes aufgerufen wird. Indem dies vermieden wird und dennoch ein expliziter Aufruf der Methode verlangt wird, ist es möglich, während der Lebensdauer der Maschine Operationen im Push- und Pull-Stil zu kombinieren.

***meName* reset**
Diese Methode setzt die Maschine in ihren Ausgangszustand zurück und löscht dabei jeglichen bisherigen Zustand.

***meName* destroy**
Diese Methode löscht das Objekt und gibt alle von ihm belegten Ressourcen frei.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Grammatiken und endliche Automaten

Copyright

```
Copyright (c) 2005-2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_cpucore
Stichwörter: me_cpucore, grammar, parsing, virtual machine

Name
grammar::me::cpu::core – Manipulation des Zustands einer virtuellen ME-Maschine

Übersicht
package require Tcl 8.5 9
package require grammar::me::cpu::core ?**0.4**?
::grammar::me::cpu::core disasm *asm*
::grammar::me::cpu::core asm *asm*
::grammar::me::cpu::core new *asm*
::grammar::me::cpu::core lc *state location*
::grammar::me::cpu::core tok *state* ?*from ?to*?
::grammar::me::cpu::core pc *state*
::grammar::me::cpu::core iseof *state*
::grammar::me::cpu::core at *state*
::grammar::me::cpu::core cc *state*
::grammar::me::cpu::core sv *state*
::grammar::me::cpu::core ok *state*
::grammar::me::cpu::core error *state*
::grammar::me::cpu::core lstk *state*
::grammar::me::cpu::core astk *state*
::grammar::me::cpu::core mstk *state*
::grammar::me::cpu::core estk *state*
::grammar::me::cpu::core rstk *state*
::grammar::me::cpu::core nc *state*
::grammar::me::cpu::core ast *state*
::grammar::me::cpu::core halted *state*
::grammar::me::cpu::core code *state*
::grammar::me::cpu::core eof *statevar*
::grammar::me::cpu::core put *statevar tok lex line col*
::grammar::me::cpu::core run *statevar* ?*n*?

Beschreibung

Dieses Paket enthält eine Implementierung der virtuellen ME-Maschine. Bitte lesen Sie zunächst das Dokument **grammar::me_intro**, falls Sie nicht wissen, was eine virtuelle ME-Maschine ist.

Diese Implementierung stellt jede virtuelle ME-Maschine als Tcl-Wert dar und bietet Befehle zur Bearbeitung und Abfrage solcher Werte, um die Auswirkungen der Ausführung von Anweisungen, des Hinzufügens von Tokens, des Abrufens des Zustands usw. zu veranschaulichen.

Die Werte folgen vollständig dem Tcl-Paradigma, wonach jeder Wert eine Zeichenkette ist, ermöglichen jedoch gleichzeitig C-Implementierungen für einen geeigneten Tcl_ObjType, um alle wichtigen Daten in nativen Datenstrukturen zu speichern. Aus diesem Grund wird empfohlen, auf die Zustandswerte *ausschließlich* über die Befehle dieses Pakets zuzugreifen, um sicherzustellen, dass die interne Darstellung nicht verfälscht wird.

Die tatsächliche Struktur, die von allen Werten verwendet wird, wird im Abschnitt CPU-Zustand beschrieben.

API

Das Paket stellt direkt nur einen einzigen Befehl bereit, und der gesamte Funktionsumfang wird über dessen Methoden zur Verfügung gestellt.

::grammar::me::cpu::core disasm *asm*

Diese Methode gibt eine Liste zurück, die eine Disassemblierung der Match-Anweisungen im Format *asm* enthält. Das Format von *asm* ist im Abschnitt Darstellung des Match-Programms festgelegt.

Jedes Element des Ergebnisses enthält in dieser Reihenfolge das Befehlslabel, den Befehlsnamen und die Befehlsargumente. Das Label kann eine leere Zeichenkette sein. Sprungziele werden als Labels, Zeichenketten und Tokens ohne Kodierung dargestellt. Tokennamen werden mit ihrer numerischen ID vorangestellt, genau dann, wenn eine Token-Zuordnung definiert ist. Die beiden Komponenten werden durch einen Doppelpunkt getrennt.

::grammar::me::cpu::core asm *asm*

Diese Methode gibt Code in dem im Abschnitt Darstellung des Match-Programms angegebenen Format zurück, der aus dem ME-Assemblercode *asm* generiert wurde, der wiederum in dem Format vorliegt, wie er von der Methode **disasm** zurückgegeben wird.

::grammar::me::cpu::core new *asm*

Diese Methode erzeugt einen Wert für den Zustand einer virtuellen ME-Maschine in ihrem Ausgangszustand und gibt diesen als Ergebnis zurück.

Das Argument *matchcode* enthält eine Tcl-Darstellung der Match-Anweisungen, die die Maschine beim Parsen des Eingabestroms ausführen muss. Das Format ist im Abschnitt Darstellung des Match-Programms festgelegt.

Das Argument *tokmap*, das von der durch das Paket **grammar::me::tcl** bereitgestellten Implementierung verwendet wird, ist hier in den Match-Anweisungen verborgen und wird daher nicht benötigt.

::grammar::me::cpu::core lc *state location*

Diese Methode übernimmt den Wert des Status einer virtuellen ME-Maschine und wandelt

damit eine Position im Eingabestrom (als Offset) in eine Zeilennummer und einen Index für die Spalten um. Das Ergebnis der Methode ist eine Liste mit zwei Elementen, die diese beiden Werte in der im vorigen Satz genannten Reihenfolge enthalten.

Hinweis: Die Methode kann keine Positionen umwandeln, die die Maschine noch nicht aus dem Eingabestrom gelesen hat. Mit anderen Worten: Wenn die Maschine bisher 7 Zeichen gelesen hat, ist es möglich, die Offsets **0** bis **6** umzuwandeln, jedoch nichts darüber hinaus. Dies zeigt auch, dass es nicht möglich ist, Offsets zu konvertieren, die sich auf Stellen vor dem Beginn des Datenstroms beziehen.

Dieses Dienstprogramm ermöglicht es übergeordneten Ebenen, die im Fehlerstatus und im AST gefundenen Positions-Offsets in besser lesbare Daten umzuwandeln.

::grammar::me::cpu::core tok *state* ?*from* ?*to*??

Diese Methode übernimmt den Wert des Status einer virtuellen ME-Maschine und gibt eine Tcl-Liste zurück, die den Teil des Eingabestroms zwischen den Positionen *from* und *to* (beide inklusive) enthält. Wird *to* nicht festgelegt, wird standardmäßig der Wert von *from* verwendet. Wird *from* ebenfalls nicht festgelegt, wird der gesamte Eingabestrom zurückgegeben.

Diese Methode unterliegt hinsichtlich ihrer Positionsargumente denselben Einschränkungen wie die Methode **lc**.

::grammar::me::cpu::core pc *state*

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den aktuellen Wert des gespeicherten Programmzählers zurück.

::grammar::me::cpu::core iseof *state*

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den aktuellen Wert des gespeicherten EOF-Flags zurück.

::grammar::me::cpu::core at *state*

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt die aktuelle Position im Eingabestrom zurück.

::grammar::me::cpu::core cc *state*

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt das aktuelle Token zurück.

::grammar::me::cpu::core sv *state*

Diese Methode ruft den Statuswert einer virtuellen ME-Maschine ab und gibt den darin gespeicherten aktuellen semantischen Wert zurück. Dabei handelt es sich um einen abstrakten Syntaxbaum, wie er im Dokument **grammar::me_ast**, Abschnitt **AST-WERTE**, festgelegt ist.

::grammar::me::cpu::core ok *state*

Diese Methode ruft den Wert einer virtuellen ME-Maschine ab und gibt den darin gespeicherten Abgleichstatus zurück.

::grammar::me::cpu::core error *state*

Diese Methode ruft den Wert der virtuellen ME-Maschine ab und gibt den darin gespeicherten aktuellen Fehlerstatus zurück.

::grammar::me::cpu::core lstk *state*

Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den

Positionsstapel zurück.

::grammar::me::cpu::core astk *state*
Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den AST-Stack zurück.

::grammar::me::cpu::core mstk *state*
Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den AST-Marker-Stack zurück.

::grammar::me::cpu::core estk *state*
Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den Fehlerstapel zurück.

::grammar::me::cpu::core rstk *state*
Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den Rückgabestapel der Unterroutine zurück.

::grammar::me::cpu::core nc *state*
Diese Methode nimmt den Statuswert einer virtuellen ME-Maschine entgegen und gibt den Cache für Nichtterminal-Übereinstimmungen als Dictionary zurück.

::grammar::me::cpu::core ast *state*
Diese Methode ruft den Wert einer virtuellen ME-Maschine ab und gibt den abstrakten Syntaxbaum zurück, der sich derzeit an der Spitze des darin gespeicherten AST-Stapels befindet. Dabei handelt es sich um einen abstrakten Syntaxbaum gemäß der Spezifikation im Dokument **grammar::me_ast**, Abschnitt **AST-WERTE**.

::grammar::me::cpu::core halted *state*
Diese Methode ruft den Wert einer virtuellen ME-Maschine ab und gibt den darin gespeicherten aktuellen Haltezustand zurück, d. h., ob die Maschine angehalten wurde oder nicht.

::grammar::me::cpu::core code *state*
Diese Methode übernimmt den Wert der Status-Eigenschaft einer virtuellen ME-Maschine und gibt den darin gespeicherten Code zurück, d. h. die von der Maschine ausgeführten Befehle.

::grammar::me::cpu::core eof *statevar*
Diese Methode übernimmt den Wert des Status einer virtuellen ME-Maschine, wie er in der durch *statevar* bezeichneten Variable gespeichert ist, und ändert ihn so, dass das darin enthaltene EOF-Flag gesetzt wird. Dies signalisiert der Maschine, dass die in der Eingabewarteschlange befindlichen Token die letzten sind, die verarbeitet werden. Es werden keine weiteren mehr folgen.

::grammar::me::cpu::core put *statevar tok lex line col*
Diese Methode übernimmt den Wert einer virtuellen ME-Maschine, wie er in der durch *statevar* benannten Variable gespeichert ist, und modifiziert ihn so, dass das Token *tok* am Ende der Eingabewarteschlange angehängt wird, zusammen mit den zugehörigen Lexemdaten *lex* und den Zeilen- und Spaltenangaben *line*/*col*umn.

Der Vorgang schlägt mit einer Fehlermeldung fehl, wenn das EOF-Flag der Maschine über die Methode **eof** gesetzt wurde.

::grammar::me::cpu::core run *statevar* ?*n*?
Bei dieser Methode wird der Wert des Zustands einer virtuellen ME-Maschine, wie er in der durch *statevar* bezeichneten Variable gespeichert ist, übernommen, es wird eine Reihe von

Befehlen ausgeführt und der durch diese Änderungen entstandene Zustand wird wieder in der Variable gespeichert.

Die Ausführungsschleife läuft so lange, bis entweder
- *n* Anweisungen wurden ausgeführt, oder
- ein Haltebefehl ausgeführt wurde oder
- Die Eingabewarteschlange ist leer und der Code fordert weitere Token zur Verarbeitung an.

Wenn kein Grenzwert *n* festgelegt wurde, werden nur die letzten beiden Bedingungen geprüft.

Darstellung des Match-Programms

Ein Match-Programm wird durch eine verschachtelte Tcl-Liste dargestellt. Das erste Element, *asm*, ist eine Liste von Ganzzahlen, die die auszuführenden Befehle und deren Argumente enthalten. Das zweite Element, *pool*, ist eine Liste von Zeichenketten, auf die in den Befehlen verwiesen wird und die Fehlermeldungen, Token-Namen usw. enthalten. Das dritte Element, *tokmap*, enthält Informationen zur Reihenfolge der Token und ordnet deren Namen ihren numerischen Rängen zu. Dieses Element kann leer sein, wodurch beim Abgleich von Bereichen ein lexikografischer Vergleich erzwungen wird.

Alle ME-Befehle werden als Ganzzahlen kodiert, wobei die Zuordnung wie unten angegeben erfolgt. Einigen Befehlen – nämlich jenen, die Fehlermeldungen verarbeiten – wurde ein zusätzliches Argument zugewiesen, um diese Meldung explizit anzugeben, anstatt sie aus Token-Namen usw. zu konstruieren. Dadurch kann der Maschinenzustand statt der vollständigen Zeichenfolgen lediglich die Meldungs-IDs speichern.

Argumente für Sprungziele sind absolute Indizes innerhalb des *asm*-Elements, die sich auf den Befehl beziehen, zu dem gesprungen werden soll. Alle Zeichenfolgenargumente sind absolute Indizes innerhalb des *pool*-Elements. Zu suchende Token, Zeichen, Meldungen und Token- (bzw. Zeichen-)Klassen werden ebenfalls als Verweise auf das *pool*-Element kodiert.

1. "**ict_advance** *message*"
2. "**ict_match_token** *tok message*"
3. "**ict_match_tokrange** *tokbegin tokend message*"
4. "**ict_match_tokclass** *code message*"
5. "**inc_restore** *branchlabel nt*"
6. "**inc_save** *nt*"
7. "**icf_ntcall** *branchlabel*"
8. "**icf_ntreturn**"
9. "**iok_ok**"
10. "**iok_fail**"
11. "**iok_negate**"
12. "**icf_jalways** *branchlabel*"
13. "**icf_jok** *branchlabel*"
14. "**icf_jfail** *branchlabel*"
15. "**icf_halt**"
16. "**icl_push**"
17. "**icl_rewind**"
18. "**icl_pop**"
19. "**ier_push**"

20. "**ier_clear**"
21. "**ier_nonterminal** *message*"
22. "**ier_merge**"
23. "**isv_clear**"
24. "**isv_terminal**"
25. "**isv_nonterminal_leaf** *nt*"
26. "**isv_nonterminal_range** *nt*"
27. "**isv_nonterminal_reduce** *nt*"
28. "**ias_push**"
29. "**ias_mark**"
30. "**ias_mrewind**"
31. "**ias_mpop**"

CPU-Zustand

Ein Wert ist eine Liste, die die folgenden Elemente in der unten angegebenen Reihenfolge enthält:

1. *code*: Match-Anweisungen, siehe Darstellung des Match-Programms.
2. *pc*: Programmzähler, *int*.
3. *halt*: Halte-Flag, *boolean*.
4. *eof*: Eof-Flag, *boolean*
5. *tc*: Terminal-Cache und Eingabewarteschlange. Aufbau siehe unten.
6. *cl*: Aktuelle Position, *int*.
7. *ct*: Aktuelles Token, *string*.
8. *ok*: Übereinstimmungsstatus, *boolean*.
9. *sv*: Semantischer Wert, *list*.
10. *er*: Fehler-Status, *list*.
11. *ls*: Positionsstapel, *list*.
12. *as*: AST-Stack, *list*.
13. *ms*: AST-Marker-Stack, *list*.
14. *es*: Fehlerstapel, *list*.
15. *rs*: Rückgabestapel, *list*.
16. *nc*: Nichtterminal-Cache, *dictionary*.

tc, die Eingabewarteschlange mit den zur Verarbeitung anstehenden Tokens und der terminale Cache, der die bereits verarbeiteten Tokens enthält, bilden eine einheitliche Datenstruktur, die lediglich alle Tokens und deren Informationen enthält, wobei die aktuelle Position die bereits verarbeiteten Elemente von den noch anstehenden trennt. Jedes Element der Warteschlange/des Caches ist eine Liste, die das Token, dessen Lexem-Informationen, die Zeilennummer und den Index der Spalten in dieser Reihenfolge enthält.

Bei allen Stacks befindet sich das oberste Element am Ende; das Einfügen eines Elements entspricht also dem Anhängen an die Liste, die den Stack darstellt, und das Entfernen eines Elements entfernt das letzte Element.

er: Der Fehlerstatus ist entweder leer oder eine Liste mit zwei Elementen – einer Position in der Eingabe und einer Liste von Meldungen, die als Verweise auf das *pool*-Element des *code* kodiert sind.

nc: Der Nichtterminal-Cache ist nach Nichtterminal-Name und Position indiziert; jeder Wert ist

eine Liste mit vier Elementen, die die aktuelle Position, den Übereinstimmungsstatus, den semantischen Wert und den Fehlerstatus in dieser Reihenfolge enthält.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005-2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_intro

Stichwörter: me_intro, CFG, CFL, LL(k), PEG, TPDL, context-free grammar, context-free languages , expression, grammar, matching, parsing, parsing expression grammar, push down automaton, recursive descent, top-down parsing languages, transducer, virtual machine

Name

grammar::me_intro – Einleitung in virtuelle Maschinen zum Parsen von Token-Strömen

Beschreibung

Dieses Dokument ist eine Einleitung in die grundlegenden Mittel zum Parsen und/oder Abgleichen von *token*-Strömen und ein Überblick darüber. Zeichen sind eine häufig genutzte Möglichkeit im Token-Bereich.

Die Pakete selbst bieten allesamt Varianten einer *virtuellen Maschine* an, die *Match-Engine* genannt wird (kurz *ME*). Sie hat alle Mittel, die zum Abgleichen und Parsen eines Stroms nötig sind. Die Varianten werden entweder direkt gesteuert oder mit einem Match-Programm angepasst. Die virtuelle Maschine ist im Grunde ein Pushdown-Automat mit zusätzlichen Elementen für das Backtracking und/oder die Verarbeitung semantischer Daten sowie die Erstellung abstrakter Syntaxbäume (*AST*).

Aufgrund der großen Ähnlichkeit der konkreten Implementierungen der oben genannten virtuellen Maschine und der Datenstrukturen, die sie empfangen und erzeugen, werden diese gemeinsamen Teile in einem separaten Dokument festgelegt, auf das die Dokumentation der Pakete verweist, die die Maschine tatsächlich implementieren.

Die entsprechenden Dokumente sind:

grammar::me_vm
: Spezifikation der virtuellen Maschine.

grammar::me_ast

Spezifikation verschiedener Darstellungsformen für abstrakte Syntaxbäume.

grammar::me::util
Dienstprogramme.

grammar::me::tcl
Implementierung einer virtuellen ME-Maschine als Singleton, die hinsichtlich Kontrollfluss und Stacks an Tcl gebunden ist. Für Pull-Betrieb fest verdrahtet. Während der Verarbeitung nicht unterbrechbar.

grammar::me::cpu
Objektbasierte Implementierung einer virtuellen ME-Maschine mit explizitem Kontrollfluss und Stacks unter Verwendung von Bytecodes. Aussetzbar und fortsetzbar. Push-/Pull-Betrieb.

grammar::me::cpu::core
Kernfunktionalität zur Zustandsmanipulation und zur schrittweisen Ausführung, die in der Bytecode-basierten Implementierung von virtuellen ME-Maschinen zum Einsatz kommt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_tcl

Stichwörter: me_tcl, grammar, parsing, virtual machine

Name

grammar::me::tcl – Implementierung virtueller Maschinen I zur Analyse von Token-Strömen

Übersicht

package require Tcl 8.5 9
package require grammar::me::tcl ?**0.2**?
::grammar::me::tcl cmd *...*
::grammar::me::tcl init *nextcmd* ?*tokmap*?
::grammar::me::tcl lc *location*
::grammar::me::tcl tok *from* ?*to*?
::grammar::me::tcl tokens
::grammar::me::tcl sv

::grammar::me::tcl ast
::grammar::me::tcl astall
::grammar::me::tcl ctok
::grammar::me::tcl nc
::grammar::me::tcl next
::grammar::me::tcl ord
::grammar::me::tcl::ict_advance *message*
::grammar::me::tcl::ict_match_token *tok message*
::grammar::me::tcl::ict_match_tokrange *tokbegin tokend message*
::grammar::me::tcl::ict_match_tokclass *code message*
::grammar::me::tcl::inc_restore *nt*
::grammar::me::tcl::inc_save *nt startlocation*
::grammar::me::tcl::iok_ok
::grammar::me::tcl::iok_fail
::grammar::me::tcl::iok_negate
::grammar::me::tcl::icl_get
::grammar::me::tcl::icl_rewind *oldlocation*
::grammar::me::tcl::ier_get
::grammar::me::tcl::ier_clear
::grammar::me::tcl::ier_nonterminal *message location*
::grammar::me::tcl::ier_merge *olderror*
::grammar::me::tcl::isv_clear
::grammar::me::tcl::isv_terminal
::grammar::me::tcl::isv_nonterminal_leaf *nt startlocation*
::grammar::me::tcl::isv_nonterminal_range *nt startlocation*
::grammar::me::tcl::isv_nonterminal_reduce *nt startlocation* ?*marker*?
::grammar::me::tcl::ias_push
::grammar::me::tcl::ias_mark
::grammar::me::tcl::ias_pop2mark *marker*

Beschreibung

Dieses Paket enthält eine Implementierung der virtuellen ME-Maschine. Bitte lesen Sie zunächst das Dokument **grammar::me_intro**, falls Sie nicht wissen, was eine virtuelle ME-Maschine ist.

Diese Implementierung ist sehr stark an Tcl gebunden. Alle Stacks im Maschinenzustand werden über den Tcl-Stack verwaltet, der gesamte Kontrollfluss wird durch Tcl-Befehle gesteuert, und die übrigen Maschinenbefehle werden direkt auf Tcl-Befehle abgebildet. Insbesondere die Abgleichung von Nichtterminalsymbolen erfolgt ebenfalls über Tcl-Prozeduren, wodurch die Maschinenimplementierung im Wesentlichen um benutzerdefinierte Befehle erweitert wird.

Darüber hinaus verarbeitet die Implementierung nur eine einzige Maschine, die während der Ausführung nicht unterbrochen werden kann und fest auf den Pull-Betrieb ausgelegt ist. Das heißt, sie fordert jedes neue Token explizit über eine Rückruffunktion an und zieht es in ihren Zustand ein.

Ein zugehöriges Paket ist **grammar::peg::interp**, das einen generischen Interpreter/Parser für Parsing Expression Grammars (PEGs) bereitstellt, der auf dieser Implementierung der virtuellen ME-Maschine aufbaut.

API

Die in diesem Abschnitt dokumentierten Befehle implementieren keinen der Befehle der virtuellen ME-Maschine. Sie bieten Funktionen zur Initialisierung der Maschine und zum Abrufen wichtiger Informationen.

::grammar::me::tcl cmd*...*

Dies ist ein Ensemble-Befehl, der Zugriff auf die in diesem Abschnitt aufgeführten Befehle gewährt. Detaillierte Angaben finden Sie in den jeweiligen Methoden.

::grammar::me::tcl init *nextcmd* ?*tokmap*?

Dieser Befehl (re)initialisiert die Maschine. Er gibt die leere Zeichenkette zurück. Dieser Befehl muss vor jedem anderen Befehl dieses Pakets aufgerufen werden.

Das Befehlspräfix *nextcmd* steht für den Eingabestrom von Zeichen und wird von der Maschine aufgerufen, sobald ein neues Zeichen aus dem Strom benötigt wird. Die Anweisung zur Handhabung hierfür lautet *ict_advance*. Die Rückruffunktion muss entweder die leere Liste oder eine Liste mit vier Elementen zurückgeben, die das Token, dessen Lexem-Attribut sowie dessen Position als Zeilennummer und Index der Spalte in dieser Reihenfolge enthalten. Die leere Liste signalisiert, dass das Ende des Eingabestroms erreicht wurde. Das Lexem-Attribut wird im Terminal-Cache gespeichert, aber ansonsten von der Maschine nicht verwendet.

Das optionale Dictionary *tokmap* ordnet Token ganzzahligen Werten zu. Sofern vorhanden, legen die Zahlen eine Reihenfolge für die Token fest, die anschließend von *ict_match_tokrange* verwendet wird, um zu bestimmen, ob sich ein Token im angegebenen Bereich befindet oder nicht. Wird keine Token-Zuordnung festgelegt, wird stattdessen die lexikografische Reihenfolge der Tokennamen verwendet. Diese Wahl ist insbesondere dann sinnvoll, wenn Zeichen als Token verwendet werden.

::grammar::me::tcl lc *location*

Dieser Befehl wandelt die Position eines Tokens, das als Offset im Eingabestrom angegeben wird, in die zugehörige Zeilennummer und den Spaltenindex um. Das Ergebnis des Befehls ist eine Liste mit zwei Elementen, die diese beiden Werte in der im vorigen Satz genannten Reihenfolge enthält. Dies ermöglicht es übergeordneten Ebenen, die im Fehlerstatus und im generierten AST enthaltenen Positionsinformationen in besser lesbare Daten umzuwandeln.

Hinweis: Der Befehl kann keine Positionen umwandeln, die von der Maschine noch nicht erreicht wurden. Mit anderen Worten: Wenn die Maschine 7 Token gelesen hat, kann der Befehl die Offsets **0** bis **6** umwandeln, jedoch nichts darüber hinaus. Dies zeigt auch, dass es nicht möglich ist, Offsets zu konvertieren, die sich auf Positionen vor dem Beginn des Datenstroms beziehen.

Nach einem Aufruf von **init** wird der für die Konvertierung verwendete Zustand gelöscht, wodurch weitere Konvertierungen unmöglich werden, bis die Maschine erneut Token gelesen hat.

::grammar::me::tcl tok *from* ?*to*?

Dieser Befehl gibt eine Tcl-Liste zurück, die den Teil des Eingabestroms zwischen den Positionen *from* und *to* (beide inklusive) enthält. Wenn *to* nicht festgelegt ist, wird standardmäßig der Wert von *from* verwendet.

Jedes Element der zurückgegebenen Liste ist eine Liste aus vier Elementen: dem Token, dem zugehörigen Lexem, der Zeilennummer und dem Spaltenindex, in dieser Reihenfolge. Mit anderen Worten: Jedes Element hat dieselbe Struktur wie das Ergebnis der *nextcmd* -Rückruffunktion, die an **::grammar::me::tcl::init** übergeben wird.

Dieser Befehl unterwirft seine Positionsargumente denselben Einschränkungen wie **::grammar::me::tcl::lc**.

::grammar::me::tcl tokens
Dieser Befehl gibt die Anzahl der Token zurück, die der virtuellen ME-Maschine derzeit bekannt sind.

::grammar::me::tcl sv
Dieser Befehl gibt den aktuellen semantischen Wert *SV* zurück, der in der Maschine gespeichert ist. Dabei handelt es sich um einen abstrakten Syntaxbaum, wie er im Dokument **grammar::me_ast**, Abschnitt **AST-WERTE**, beschrieben ist.

::grammar::me::tcl ast
Diese Methode gibt den abstrakten Syntaxbaum zurück, der sich derzeit an der Spitze des AST-Stapels der virtuellen ME-Maschine befindet. Es handelt sich hierbei um einen abstrakten Syntaxbaum gemäß der Spezifikation im Dokument **grammar::me_ast**, Abschnitt **AST-WERTE**.

::grammar::me::tcl astall
Diese Methode gibt den gesamten Stapel der abstrakten Syntaxbäume zurück, die der virtuellen ME-Maschine derzeit bekannt sind. Jedes Element der zurückgegebenen Liste ist ein abstrakter Syntaxbaum, wie im Dokument **grammar::me_ast**, Abschnitt **AST-WERTE**, festgelegt. Das oberste Element des Stacks befindet sich am Ende der Liste.

::grammar::me::tcl ctok
Diese Methode gibt das aktuelle Token zurück, das von der virtuellen ME-Maschine verarbeitet wird.

::grammar::me::tcl nc
Diese Methode gibt den Inhalt des Nichtterminal-Caches als Dictionary zurück, das von "**symbol**,**location**" auf Übereinstimmungsinformationen abbildet.

::grammar::me::tcl next
Diese Methode gibt die Rückruffunktion für das nächste Token zurück, wie sie bei der Initialisierung der virtuellen ME-Maschine festgelegt wurde.

::grammar::me::tcl ord
Diese Methode gibt ein Dictionary zurück, das die bei der Initialisierung der virtuellen ME-Maschine festgelegte *tokmap* enthält. **::grammar::me::tcl::ok** Diese Variable enthält den aktuellen Übereinstimmungsstatus *OK*. Sie wird als Variable anstelle eines Befehls bereitgestellt, da dies den Zugriff auf diese Informationen beschleunigt; die Zugriffsgeschwindigkeit wird hier als sehr wichtig erachtet, da diese Informationen ständig zur Bestimmung des Kontrollflusses herangezogen werden.

Maschinenstatus

Bitte lesen Sie zunächst das Dokument **grammar::me_vm**, um sich über die Spezifikationen der virtuellen ME-Maschine und deren Zustand zu informieren.

Diese Implementierung verwaltet den in jenem Dokument beschriebenen Zustand, mit Ausnahme der Stacks – abgesehen vom AST-Stack. Mit anderen Worten: Der Positionsstack, der Fehlerstack, der Rückgabestack und der AST-Marker-Stack werden implizit über den Standard-Tcl-Gültigkeitsbereich verwaltet, d. h. über Tcl-Variablen in Prozeduren außerhalb dieser Implementierung.

Maschinenbefehle

Bitte lesen Sie zunächst das Dokument **grammar::me_vm**, um sich über die Spezifikationen der virtuellen ME-Maschine und ihres Befehlssatzes zu informieren.

Diese Implementierung ordnet alle Anweisungen Tcl-Befehlen im Namespace "::grammar::me::tcl" zu, mit Ausnahme der stapelbezogenen Befehle, der nichtterminalen Symbole und der Kontrollflussanweisungen. Hier werden lediglich die Befehle aufgelistet und gegebenenfalls die Unterschiede zu den angegebenen Befehlen erläutert. Informationen zu ihrer Semantik finden Sie in der oben genannten Spezifikation. Die Maschinenbefehle sind über den Ensemble-Befehl **::grammar::me::tcl** *nicht* erreichbar.

::grammar::me::tcl::ict_advance *message*
: Keine Änderungen.

::grammar::me::tcl::ict_match_token *tok message*
: Keine Änderungen.

::grammar::me::tcl::ict_match_tokrange *tokbegin tokend message*
: Genau dann, wenn bei der Initialisierung eine Token-Zuordnung festgelegt wurde, sind die Argumente die numerischen Darstellungen des kleinsten und des größten Tokens im Bereich. Andernfalls handelt es sich um die betreffenden Token selbst, und es wird ein lexikografischer Vergleich herangezogen.

::grammar::me::tcl::ict_match_tokclass *code message*
: Keine Änderungen.

::grammar::me::tcl::inc_restore *nt*
: Anstatt ein Verzweigungslabel entgegenzunehmen, gibt der Befehl einen booleschen Wert zurück. Das Ergebnis lautet **true** genau dann, wenn zwischengespeicherte Informationen gefunden wurden. Der Aufrufer muss die entsprechende Verzweigung vornehmen.

::grammar::me::tcl::inc_save *nt startlocation*
: Der Befehl nimmt die Startposition als zusätzliches Argument entgegen, da diese auf dem Tcl-Stack verwaltet wird und nicht im Maschinenzustand.

icf_ntcall *branchlabel*
: siehe unten ...

icf_ntreturn
: Diese beiden Anweisungen sind keinen Befehlen zugeordnet. Es handelt sich um Anweisungen zur Ablaufsteuerung, die in Tcl verarbeitet werden.

::grammar::me::tcl::iok_ok
: Keine Änderungen.

::grammar::me::tcl::iok_fail
: Keine Änderungen.

::grammar::me::tcl::iok_negate
: Keine Änderungen.

icf_jalways *branchlabel*
siehe unten ...

icf_jok *branchlabel*
siehe unten ...

icf_jfail *branchlabel*
siehe unten ...

icf_halt
Diese vier Anweisungen sind keinen Befehlen zugeordnet. Es handelt sich um Anweisungen zur Steuerung des Programmablaufs, die in Tcl verarbeitet werden.

::grammar::me::tcl::icl_get
Dieser Befehl gibt die aktuelle Position *CL* in der Eingabe zurück. Er ersetzt *icl_push*.

::grammar::me::tcl::icl_rewind *oldlocation*
Der Befehl nimmt die Position als Argument entgegen, da diese aus dem Tcl-Stack stammt und nicht aus dem Maschinenzustand.

icl_pop
Da sie nicht zugeordnet sind, werden die Stacks nicht vom Paket verwaltet.

::grammar::me::tcl::ier_get
Dieser Befehl gibt den aktuellen Fehlerstatus *ER* zurück. Er ersetzt *ier_push*.

::grammar::me::tcl::ier_clear
Keine Änderungen.

::grammar::me::tcl::ier_nonterminal *message location*
Der Befehl nimmt die Position als Argument entgegen, da diese aus dem Tcl-Stack stammt und nicht aus dem Maschinenzustand.

::grammar::me::tcl::ier_merge *olderror*
Der Befehl nimmt den zweiten Fehlerstatus, der zusammengeführt werden soll, als Argument entgegen, da dieser aus dem Tcl-Stack stammt und nicht aus dem Maschinenzustand.

::grammar::me::tcl::isv_clear
Keine Änderungen.

::grammar::me::tcl::isv_terminal
Keine Änderungen.

::grammar::me::tcl::isv_nonterminal_leaf *nt startlocation*
Der Befehl nimmt die Startposition als Argument entgegen, da diese aus dem Tcl-Stack stammt und nicht aus dem Maschinenzustand.

::grammar::me::tcl::isv_nonterminal_range *nt startlocation*
Der Befehl nimmt die Startposition als Argument entgegen, da diese aus dem Tcl-Stack stammt und nicht aus dem Maschinenzustand.

::grammar::me::tcl::isv_nonterminal_reduce *nt startlocation* ?*marker*?
Der Befehl nimmt die Startposition und den Marker als Argumente entgegen, da diese aus dem Tcl-Stack stammen und nicht aus dem Maschinenzustand.

::grammar::me::tcl::ias_push
Keine Änderungen.

::grammar::me::tcl::ias_mark
Dieser Befehl gibt einen Marker für den aktuellen Zustand des AST-Stapels *AS* zurück. Der Marker-Stapel wird nicht von der Maschine verwaltet.

::grammar::me::tcl::ias_pop2mark *marker*
Der Befehl übernimmt den Marker als Argument, da dieser aus dem Tcl-Stack stammt und nicht aus dem Maschinenzustand. Er ersetzt *ias_mpop*.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_util

Stichwörter: me_util, abstract syntax tree, syntax tree, tree

Name

grammar::me::util – AST-Dienstprogramme

Übersicht

package require Tcl 8.5 9
package require grammar::me::util ?**0.2**?
::grammar::me::util::ast2tree *ast tree* ?*root*?
::grammar::me::util::ast2etree *ast mcmd tree* ?*root*?
mcmd lc *location*
mcmd tok *from* ?*to*?
::grammar::me::util::tree2ast *tree* ?*root*?

Beschreibung

Dieses Paket stellt eine Reihe von Hilfsbefehlen zur Umwandlung zwischen den verschiedenen Darstellungen abstrakter Syntaxbäume bereit, wie sie im Dokument **grammar::me_ast** festgelegt sind.

::grammar::me::util::ast2tree *ast tree ?root?*
Dieser Befehl wandelt einen *ast* von der Wertdarstellung in die Objektdarstellung um. Alle Knoten im *ast* werden zu Knoten dieses *tree*, wobei die Wurzel des AST ein Kindknoten des Knotens *root* wird. Wird dieser Knoten nicht ausdrücklich angegeben, wird die Wurzel des Baums verwendet. Der bisherige Inhalt des Baums bleibt unberührt, er wird weder entfernt noch verändert, außer dass der angegebene Wurzelknoten einen neuen Kindknoten erhält.

::grammar::me::util::ast2etree *ast mcmd tree* ?*root*?

Dieser Befehl entspricht dem Befehl **::grammar::me::util::ast2tree**, mit dem Unterschied, dass das Ergebnis in der erweiterten Objektdarstellung des Eingabe-AST vorliegt. Die Quelle der erweiterten Informationen ist das Befehlspräfix *mcmd*. Es muss zwei Methoden verstehen, **lc** und **tok**, mit der unten festgelegten Semantik.

mcmd lc *location*

Nimmt die Position eines Tokens entgegen, die als Offset im Eingabestrom angegeben ist, und gibt eine Liste mit zwei Elementen zurück, die die zugehörige Zeilennummer und den Spaltenindex in dieser Reihenfolge enthält.

mcmd tok *from* ?*to*?

Nimmt eine oder zwei Positionen *from* und *to* als Offset im Eingabestrom entgegen und gibt eine Tcl-Liste zurück, die den angegebenen Teil des Eingabestroms enthält. Beide Positionen sind inklusive. Wenn *to* nicht angegeben ist, wird standardmäßig der Wert von *from* verwendet.

Jedes Element der zurückgegebenen Liste ist eine Liste, die das Token, das zugehörige Lexem, die Zeilennummer und den Spaltenindex in dieser Reihenfolge enthält.

Sowohl der Ensemble-Befehl ::grammar::me::tcl, der vom Paket **grammar::me::tcl** bereitgestellt wird, als auch der Objektbefehl, den das Paket **grammar::me::cpu** erzeugt, entsprechen der oben genannten Spezifikation.

::grammar::me::util::tree2ast *tree* ?*root*?

Dieser Befehl wandelt einen *ast* in (erweiterter) Objektdarstellung in einen Wert um und gibt ihn zurück. Ist ein Knoten *root* angegeben, wird der AST von diesem Knoten abwärts erzeugt. Andernfalls dient die Wurzel des Baumobjekts als Ausgangspunkt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


me_vm

Stichwörter: me_vm, grammar, parsing, virtual machine

Name

grammar::me_vm – Virtuelle Maschine zur Analyse von Token-Strömen

Beschreibung

Bitte lesen Sie zunächst das Dokument **grammar::me_intro**, um eine Einleitung in die verschiedenen Dokumente und deren Zusammenhänge zu erhalten.

Dieses Dokument legt eine virtuelle Maschine für das kontrollierte Abgleichen und Parsen von Token-Strömen fest, wobei ein *abstrakter Syntaxbaum* (kurz *AST*) erstellt wird, der die Struktur der Eingabe widerspiegelt. Zu den besonderen Merkmalen der Maschine zählen das Zwischenspeichern und die Wiederverwendung von Teilergebnissen, das Zwischenspeichern der erfassten Eingabe sowie die Möglichkeit, sowohl bei der Eingabe als auch bei der Erstellung des AST zurückzuspringen.

Diese Eigenschaften machen die angegebene virtuelle Maschine besonders nützlich für "Packrat"-Parser, die auf Parsing Expression Grammars basieren. Sie ist jedoch nicht auf diese Art von Parsern beschränkt. Auch herkömmliche LL- und LR-Parser lassen sich damit implementieren.

In den folgenden Abschnitten wird zunächst der abstrakte Zustand der virtuellen ME-Maschinen behandelt und anschließend deren Befehlssatz.

Maschinenstatus

Eine virtuelle ME-Maschine verwaltet den folgenden Zustand:

***Current token* CT**
: Das Token aus der Eingabe, das gerade von der Maschine verarbeitet wird.

  Diese Information wird von den im Abschnitt Abgleich von Terminalen definierten Anweisungen verwendet und verändert.

***Current location* CL**
: Die Position des *aktuellen Tokens* im Eingabestrom, ausgedrückt als Offset relativ zum Anfang des Stroms. Das erste Token befindet sich am Offset **0**.

  Diese Information wird implizit von den in den Abschnitten Abgleich von Terminalen und Abgleich von Nichtterminalen definierten Anweisungen verwendet und verändert; sie kann zudem durch die in Abschnitt Verwaltung der Eingabeposition definierten Anweisungen direkt abgefragt und verändert werden.

***Location stack* LS**
: Zusätzlich zu den oben genannten Elementen gibt es einen Stapel mit Positionen für das Zurückverfolgen. Positionen können auf den Stapel gelegt, von diesem entfernt und durch Festlegen der aktuellen Position entfernt werden.

  Diese Informationen werden implizit durch die in den Abschnitten Abgleich von Terminalen und Abgleich von Nichtterminalen definierten Anweisungen verwendet und verändert und können durch die im Abschnitt Verwaltung der Eingabeposition definierten Anweisungen direkt abgefragt und verändert werden.

***Match status* OK**
: Ein boolescher Wert, der das Ergebnis des letzten Versuchs zur Abgleichung der Eingabe

darstellt. Er wird auf **true** gesetzt, wenn dieser Versuch erfolgreich war, andernfalls auf **false**.

Diese Information wird durch die in den Abschnitten Abgleich von Terminalen, Abgleich von Nichtterminalen und Unbedingter Abgleich definierten Anweisungen beeinflusst. Sie wird durch die im Abschnitt Ablaufsteuerung definierten Anweisungen abgefragt.

***Semantic value* SV**

Der semantische Wert, der mit dem letzten Versuch, die Eingabe abzugleichen, verbunden ist (bzw. durch diesen erzeugt wird). Enthält entweder die leere Zeichenkette oder einen Knoten des aus der Eingabe konstruierten abstrakten Syntaxbaums.

Diese Information wird durch die in den Abschnitten Semantische Werte und AST-Stack-Verwaltung definierten Anweisungen beeinflusst.

***AST stack* AS**

Ein Stapel partieller abstrakter Syntaxbäume, die von der Maschine während des Abgleichs erstellt wurden.

Diese Informationen werden durch die in den Abschnitten Semantische Werte und AST-Stack-Verwaltung definierten Anweisungen beeinflusst.

***AST Marker stack* MS**

Zusätzlich zu den oben genannten Elementen gibt es einen "Stack der Stacks" für das Backtracking. Dabei handelt es sich eigentlich um einen Stack von Markern im AST-Stack, wodurch implizit zu einem bestimmten Zeitpunkt ein Snapshot des Zustands des AST-Stacks erstellt wird. Marker können auf den Stapel gesetzt, von ihm entfernt und dazu verwendet werden, den AST-Stapel auf einen früheren Zustand zurückzusetzen.

Diese Informationen werden durch die in den Abschnitten Semantische Werte und AST-Stack-Verwaltung definierten Anweisungen beeinflusst.

***Error status* ER**

Fehlerinformationen im Zusammenhang mit dem letzten Versuch, die Eingabe abzugleichen. Enthält entweder die leere Zeichenkette oder eine Liste mit zwei Elementen: eine Position in der Eingabe und eine Liste der damit verbundenen Fehlermeldungen, in dieser Reihenfolge.

Hinweis: Fehlerinformationen werden auch dann gesetzt, wenn der letzte Versuch, die Eingabe abzugleichen, erfolgreich war. Beispielsweise ist der Ausdruck *-operator (der nullmal oder mehrmals auf einen Teilausdruck passt) in einer Parsing-Ausdrucksgrammatik stets erfolgreich, selbst wenn weiter hinten in der Eingabe ein Problem auftritt und ein Rückwärtsablauf erforderlich wird. Solche Probleme dürfen bei der Fortsetzung des Abgleichs nicht außer Acht gelassen werden.

Diese Informationen werden durch die in den Abschnitten Abgleich von Terminalen, Abgleich von Nichtterminalen und Fehlerbehandlung definierten Anweisungen abgefragt und beeinflusst.

***Error stack* ES**

Zusätzlich zu den oben genannten Informationen wird ein Stapel mit Informationen zu Fehlern bereitgestellt, um bei der Rückverfolgung in Auswahlmöglichkeiten nach einem erfolglosen Abgleich die Zusammenführung aktueller und älterer Informationen zu Fehlern zu

ermöglichen.

Diese Informationen werden anhand der in den Abschnitten Abgleich von Terminalen, Abgleich von Nichtterminalen und Fehlerbehandlung definierten Anweisungen abgefragt und beeinflusst.

***Return stack* RS**
Ein Stapel von Programmzählern, d. h. Speicherorte im Code, die die virtuelle Maschine steuern, zur Verwaltung von Unterprogrammaufrufen, d. h. zum Abgleich von nichtterminalen Symbolen.

Diese Informationen werden durch die im Abschnitt Abgleich von Nichtterminalen definierten Befehle abgefragt und beeinflusst.

***Nonterminal cache* NC**
Ein Cache mit Maschinenzuständen (ein 4-Tupel, das eine Position im Eingabestrom, den Übereinstimmungsstatus *OK*, den semantischen Wert *SV* und den Fehlerstatus *ER* enthält), der anhand des Namens des nichtterminalen Symbols und der Position im Eingabestrom indiziert ist.

Die Schlüsselposition ist die Stelle, an der die Maschine den Versuch gestartet hat, das benannte nichtterminale Symbol abzugleichen, und die Position im Wert ist die Stelle, an der die Maschine nach Abschluss des Versuchs gelandet ist, unabhängig vom Erfolg des Versuchs.

Dieser Status wird durch die im Abschnitt Abgleich von Nichtterminalen definierten Anweisungen abgefragt und beeinflusst.

Maschinenbefehle

Nachdem der Maschinenzustand festgelegt wurde, lässt sich nun der Befehlssatz der virtuellen ME-Maschinen erläutern. Diese sind grob nach dem Maschinenzustand gruppiert, den sie beeinflussen und/oder abfragen.

Abgleich von Terminalen

Zunächst die Anweisungen zum Abgleichen von Tokens aus dem Eingabestrom und damit auch aller Terminalsymbole als Erweiterung.

Diese Befehle sind die einzigen, die ein neues Token aus dem Eingabestrom abrufen können. Es handelt sich hierbei um ein "können" und nicht um ein "werden", da die Befehle ein neues Token genau dann abrufen, wenn sich die aktuelle Position *CL* an der Spitze des Stroms befindet. Wenn die Maschine einen Rücklauf durchgeführt hat (siehe **icl_rewind**), holen die Befehle das zu vergleichende Token aus dem internen Cache.

ict_advance *message*
Diese Anweisung versucht, zum nächsten Token im Eingabestrom zu springen, d. h. zu dem Token, das auf die aktuelle Position *CL* folgt. Die Anweisung schlägt genau dann fehl, wenn das Ende des Eingabestroms erreicht ist, d. h., wenn kein nächstes Token vorhanden ist.

Der Erfolg bzw. Misserfolg der Anweisung wird im Übereinstimmungsstatus *OK* gespeichert. Im Falle eines Fehlers wird der Fehlerstatus *ER* auf die aktuelle Position gesetzt und die Meldung *message* hinterlegt. Im Erfolgsfall wird der Fehlerstatus *ER* gelöscht, das neue Token

wird zum aktuellen Token *CT* und die neue Position wird zur aktuellen Position *CL*.

Das Argument *message* ist ein Verweis auf die Zeichenkette, die bei Bedarf in den Fehlerstatus *ER* gesetzt werden soll.

ict_match_token *tok message*

Diese Anweisung führt Tests durch, um zu prüfen, ob das aktuelle Token *CT* mit dem Argument *tok* übereinstimmt, und speichert das Ergebnis im Übereinstimmungsstatus *OK*. Die Anweisung schlägt fehl, wenn das aktuelle Token nicht mit *tok* übereinstimmt.

Im Falle eines Fehlers wird der Fehlerstatus *ER* auf die aktuelle Position *CL* und die Meldung *message* gesetzt, und die aktuelle Position *CL* wird um ein Token zurückversetzt. Andernfalls, d. h. bei Erfolg, wird der Fehlerstatus *ER* gelöscht und die aktuelle Position *CL* bleibt unverändert.

ict_match_tokrange *tokbegin tokend message*

Diese Anweisung führt Tests durch, um zu prüfen, ob sich das aktuelle Token *CT* im Bereich der Token von *tokbegin* bis *tokend* (einschließlich) befindet, und speichert das Ergebnis im Übereinstimmungsstatus *OK*. Die Anweisung schlägt fehl, wenn sich das aktuelle Token nicht innerhalb dieses Bereichs befindet.

Im Falle eines Fehlers wird der Fehlerstatus *ER* auf die aktuelle Position *CL* und die Meldung *message* gesetzt, und die aktuelle Position *CL* wird um ein Token zurückversetzt. Andernfalls, d. h. bei Erfolg, wird der Fehlerstatus *ER* gelöscht und die aktuelle Position *CL* bleibt unverändert.

ict_match_tokclass *code message*

Diese Anweisung testet, ob das aktuelle Token *CT* zur Tokenklasse *code* gehört, und speichert das Ergebnis im Übereinstimmungsstatus *OK*. Die Anweisung schlägt fehl, wenn das aktuelle Token nicht zur angegebenen Klasse gehört.

Im Falle eines Fehlers wird der Fehlerstatus *ER* auf die aktuelle Position *CL* und die Meldung *message* gesetzt, und die aktuelle Position *CL* wird um ein Token zurückversetzt. Andernfalls, d. h. bei Erfolg, wird der Fehlerstatus *ER* gelöscht und die aktuelle Position *CL* bleibt unverändert.

Derzeit sind folgende Klassen zulässig:

alnum
: Ein Token wird akzeptiert, wenn es sich um ein alphabetisches Unicode-Zeichen oder eine Ziffer handelt.

alpha
: Ein Token wird akzeptiert, wenn es sich um ein alphabetisches Unicode-Zeichen handelt.

digit
: Ein Token wird akzeptiert, wenn es sich um ein Unicode-Ziffernzeichen handelt.

xdigit
: Ein Token wird akzeptiert, wenn es sich um ein hexadezimales Ziffernzeichen handelt.

punct
: Ein Token wird akzeptiert, wenn es sich um ein Unicode-Satzzeichen handelt.

space

Ein Token wird akzeptiert, wenn es sich um ein Unicode-Leerzeichen handelt.

Abgleich von Nichtterminalen

Die Anweisungen in diesem Abschnitt befassen sich mit der Zuordnung von Nichtterminal-Symbolen. Sie fragen den Nichtterminal-Cache *NC* nach gespeicherten Informationen ab und speichern diese Informationen im Cache.

Die Verwendung des Caches dient als Leistungsoptimierung für Backtracking-Parser und ermöglicht es ihnen, eine aufwendige erneute Überprüfung komplexer nichtterminaler Symbole zu vermeiden, sofern diese bereits zuvor angetroffen wurden.

inc_restore *branchlabel nt*

Diese Anweisung prüft, ob der Nichtterminal-Cache *NC* Informationen über das Nichtterminal-Symbol *nt* an der aktuellen Position *CL* enthält. Wenn dies der Fall ist, aktualisiert die Anweisung den Maschinenzustand (aktuelle Position *CL*, Übereinstimmungsstatus *OK*, semantischer Wert *SV* und Fehlerstatus *ER*) mit den gefundenen Informationen und setzt die Ausführung an der durch das *branchlabel* bezeichneten Anweisung fort. Die neue aktuelle Position *CL* ist das letzte Token, das mit dem Nichtterminal-Symbol übereinstimmt, d. h. zu diesem gehört.

Wurden keine Informationen gefunden, setzt der Befehl die Ausführung mit dem nächsten Befehl fort.

Zusammen mit **icf_ntcall** ist es möglich, Code für memoisiertes und nicht-memoisiertes Abgleichen von Nichtterminalsymbolen zu generieren, entweder als Unterprogrammaufrufe oder inline im Aufrufer.

inc_save *nt*

Dieser Befehl speichert den aktuellen Zustand der Maschine (aktuelle Position *CL*, Übereinstimmungsstatus *OK*, semantischer Wert *SV* und Fehlerstatus *ER*) im Nichtterminal-Cache *NC*. Zudem wird ein Eintrag aus dem Positionsstapel *LS* entfernt und als Startposition der Übereinstimmung gespeichert.

Es wird erwartet, dass dieser Befehl am Ende des Abgleichs eines Nichtterminalsymbols aufgerufen wird, wobei *nt* der Name des Nichtterminalsymbols ist, mit dem der Code gerade gearbeitet hat. Dadurch kann die Anweisung **inc_restore** die Daten überprüfen und abrufen, falls wir dieses nichtterminale Symbol während des Backtrackings erneut an derselben Stelle abgleichen müssen.

icf_ntcall *branchlabel*

Diese Anweisung ruft den Code zum Abgleichen des nichtterminalen Symbols *nt* als Unterprogramm auf. Zu diesem Zweck speichert sie den aktuellen Programmzähler *PC* auf dem Rückgabestapel *RS*, die aktuelle Position *CL* auf dem Positionsstapel *LS* und setzt die Ausführung anschließend an der Adresse *branchlabel* fort.

Das nächste passende **icf_ntreturn** bewirkt, dass die Ausführung mit dem Befehl fortgesetzt wird, der auf den Aufruf folgt.

icf_ntreturn

Dieser Befehl holt einen Eintrag vom Rückgabestapel *RS* ab, weist ihn dem Programmzähler

PC zu und setzt die Ausführung anschließend an der neuen Adresse fort.

Unbedingter Abgleich

Die Anweisungen in diesem Abschnitt beziehen sich auf die übrigen Abgleichoperatoren. Sie ändern den Abgleichstatus *OK* direkt und bedingungslos.

iok_ok
Dieser Befehl weist **true** den Abgleichstatus *OK* zu, was einen erfolgreichen Abgleich anzeigt.

iok_fail
Mit diesem Befehl wird der Abgleichstatus *OK* auf **false** gesetzt, was einen fehlgeschlagenen Abgleich anzeigt.

iok_negate
Diese Anweisung kehrt den Übereinstimmungsstatus *OK* um, wodurch ein Fehler zu einem Erfolg wird und umgekehrt.

Ablaufsteuerung

Die Anweisungen in diesem Abschnitt implementieren sowohl bedingte als auch unbedingte Kontrollflüsse. Die bedingten Sprünge fragen den Übereinstimmungsstatus *OK* ab.

icf_jalways *branchlabel*
Dieser Befehl setzt den Programmzähler *PC* auf die durch *branchlabel* angegebene Adresse und setzt die Ausführung von dort aus fort. Hierbei handelt es sich um einen bedingungslosen Sprung.

icf_jok *branchlabel*
Dieser Befehl setzt den Programmzähler *PC* auf die durch *branchlabel* angegebene Adresse. Dies geschieht genau dann, wenn der Übereinstimmungsstatus *OK* einen Erfolg anzeigt. Andernfalls wird die Ausführung einfach mit dem nächsten Befehl fortgesetzt. Hierbei handelt es sich um einen bedingten Sprung.

icf_jfail *branchlabel*
Dieser Befehl setzt den Programmzähler *PC* auf die durch *branchlabel* angegebene Adresse. Dies geschieht genau dann, wenn der Abgleichstatus *OK* einen Fehler anzeigt. Andernfalls wird die Ausführung einfach mit dem nächsten Befehl fortgesetzt. Hierbei handelt es sich um einen bedingten Sprung.

icf_halt
Dieser Befehl hält die Maschine an und unterbricht die weitere Ausführung.

Verwaltung der Eingabeposition

Die Anweisungen in diesem Abschnitt dienen dem Backtracking; sie manipulieren die aktuelle Position *CL* des Maschinenzustands. Sie ermöglichen es einem Benutzer der Maschine, Positionen in der Eingabe abzufragen und zu speichern sowie die aktuelle Position *CL* auf gespeicherte Positionen zurückzusetzen, wodurch sie zu einer der Komponenten gehören, die die Implementierung von Backtracking-Parsern ermöglichen.

icl_push
Dieser Befehl schiebt eine Kopie der aktuellen Position *CL* auf den Positionsstapel *LS*.

icl_rewind
Dieser Befehl entnimmt dem Positionsstapel *LS* einen Eintrag und versetzt anschließend die aktuelle Position *CL* zurück an diese Stelle in der Eingabe.

icl_pop
Dieser Befehl entnimmt dem Positionsstapel *LS* einen Eintrag und verwirft ihn.

Fehlerbehandlung

Die Anweisungen in diesem Abschnitt ermöglichen den Lese- und Schreibzugriff auf den Fehlerstatus *ER* der Maschine.

ier_push
Dieser Befehl schiebt eine Kopie des aktuellen Fehlerstatus *ER* auf den Fehlerstapel *ES*.

ier_clear
Dieser Befehl löscht den Fehlerstatus *ER*.

ier_nonterminal *message*
Dieser Befehl prüft, ob der Fehlerstatus *ER* einen Fehler enthält, dessen Position unmittelbar hinter der Position liegt, die im obersten Eintrag des Positionsstapels *LS* gefunden wurde. Wird kein solcher Fehler gefunden, geschieht nichts. Andernfalls wird der gefundene Fehler durch einen Fehler an der im Stapel gefundenen Position ersetzt, der die Meldung *message* aufweist.

ier_merge
Diese Anweisung entnimmt dem Fehlerstapel *ES* einen Eintrag, führt eine Zusammenführung mit dem aktuellen Fehlerstatus *ER* durch und speichert das Ergebnis dieser Zusammenführung als neuen Fehlerstatus *ER*.

Die Zusammenführung erfolgt wie folgt:

Ist einer der beiden Fehlerstatus-Einträge leer, wird der andere ausgewählt. Wenn beide Fehlerstatus nicht leer sind und sie sich auf unterschiedliche Stellen beziehen, wird der Fehlerstatus mit der weiter in der Eingabe liegenden Stelle ausgewählt. Wenn beide Fehlerstatus auf dieselbe Stelle verweisen, werden ihre Meldungen zusammengeführt (wobei Duplikate entfernt werden).

Semantische Werte

Die Anweisungen in diesem Abschnitt beeinflussen den semantischen Wert *SV*.

isv_clear
Dieser Befehl löscht den semantischen Wert *SV*.

isv_terminal
Diese Anweisung erzeugt einen Terminalknoten im AST für das aktuelle Token *CT*, macht ihn zum semantischen Wert *SV* und legt den Knoten außerdem auf den AST-Stapel *AS*.

isv_nonterminal_leaf *nt*
Diese Anweisung erstellt einen nichtterminalen AST-Knoten ohne Kindknoten für das Nichtterminal *nt* und macht ihn zum semantischen Wert *SV*.

Diese Anweisung sollte genau dann ausgeführt werden, wenn der Übereinstimmungsstatus *OK* einen Erfolg anzeigt. Im Falle eines Fehlers sollte **isv_clear** aufgerufen werden.

isv_nonterminal_range *nt*
Diese Anweisung erstellt einen nichtterminalen AST-Knoten für das Nichtterminal *nt* mit einem einzelnen Terminalknoten als Kindknoten und macht diesen AST zum semantischen Wert *SV*.

Der terminale Knoten bezieht sich auf die Eingabezeichenfolge von der Position, die sich ganz oben auf dem Positionsstapel *LS* befindet, bis zur aktuellen Position *CL* (beide einschließlich).

Diese Anweisung sollte genau dann ausgeführt werden, wenn der Übereinstimmungsstatus *OK* einen Erfolg anzeigt. Im Falle eines Fehlers sollte **isv_clear** aufgerufen werden.

isv_nonterminal_reduce *nt*
Diese Anweisung erstellt einen nichtterminalen AST-Knoten für das Nichtterminal *nt* und macht ihn zum semantischen Wert *SV*.

Alle Einträge auf dem AST-Stapel *AS*, die über dem Marker liegen, der im obersten Eintrag des AST-Marker-Stapels *MS* gefunden wurde, werden zu Kindern des neuen Knotens, wobei der Eintrag an der Stapelspitze zum Kind ganz rechts wird. Wenn der AST-Marker-Stapel *MS* leer ist, wird der gesamte Stapel verwendet. Der AST-Marker-Stapel *MS* bleibt unverändert.

Diese Anweisung sollte genau dann ausgeführt werden, wenn der Übereinstimmungsstatus *OK* einen Erfolg anzeigt. Im Falle eines Fehlers sollte **isv_clear** aufgerufen werden.

AST-Stack-Verwaltung

Die Anweisungen in diesem Abschnitt bearbeiten den AST-Stapel *AS* und den AST-Marker-Stapel *MS*.

ias_push
Diese Anweisung schiebt den semantischen Wert *SV* auf den AST-Stapel *AS*.

ias_mark
Dieser Befehl schiebt einen Marker für den aktuellen Zustand des AST-Stapels *AS* auf den AST-Marker-Stapel *MS*.

ias_mrewind
Dieser Befehl entnimmt dem AST-Marker-Stack *MS* einen Eintrag und fährt anschließend fort, Einträge aus dem AST-Stack *AS* zu entnehmen, bis der durch den entnommenen Marker dargestellte Zustand erneut erreicht ist. Es erfolgt keine Aktion, wenn der AST-Stack *AS* bereits kleiner ist als durch den entnommenen Marker angegeben.

ias_mpop
Dieser Befehl entnimmt dem AST-Marker-Stack *MS* einen Eintrag und verwirft ihn.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_me* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


grammar_peg

peg

Stichwörter: peg, LL(k), TDPL, context-free languages, expression, grammar, parsing, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

grammar::peg – Erstellen und Bearbeiten von Parsing Expression Grammars

Übersicht

package require Tcl 8.5 9
package require snit
package require grammar::peg ?**0.3**?
::grammar::peg *pegName* ?=|*:=*|<--|*as*|*deserialize src*?
pegName **destroy**
pegName **clear**
pegName **=** *srcPEG*
pegName **-->** *dstPEG*
pegName **serialize**
pegName **deserialize** *serialization*
pegName **is valid**
pegName **start** ?*pe*?
pegName **nonterminals**
pegName **nonterminal add** *nt pe*
pegName **nonterminal delete** *nt1* ?*nt2* ...?
pegName **nonterminal exists** *nt*
pegName **nonterminal rename** *nt ntnew*
pegName **nonterminal mode** *nt* ?*mode*?
pegName **nonterminal rule** *nt*
pegName **unknown nonterminals**

Beschreibung

Dieses Paket stellt eine Containerklasse für *Parsing Expression Grammars* (kurz: PEG) bereit. Es ermöglicht die schrittweise Definition der Grammatik, deren Bearbeitung sowie die Abfrage der Definition. Das Paket bietet weder komplexe Operationen auf der Grammatik an, noch verfügt es über die Fähigkeit, eine Grammatikdefinition für einen Symbolstrom auszuführen. Zwei mit diesem Paket verwandte Pakete sind **grammar::mengine** und **grammar::peg::interpreter**. Das erste davon definiert eine allgemeine virtuelle Maschine für das Abgleichen eines Zeichenstroms, und das zweite implementiert auf Basis dieser virtuellen Maschine einen Interpreter für Parsing-Ausdrücke.

Begriffe und Konzepte

PEGs ähneln kontextfreien Grammatiken, sind jedoch nicht mit diesen gleichzusetzen; in einigen Fällen sind PEGs strikt mächtiger als kontextfreie Grammatiken (es gibt PEGs für einige nicht-kontextfreie Sprachen). Die formale mathematische Definition von Parsing-Ausdrücken und Parsing Expression Grammars steht im Abschnitt Parsing-Expression-Grammatiken.

Kurz gesagt gibt es *terminale Symbole*, die die grundlegendsten Bausteine für *Sätze* darstellen, sowie *nichtterminale Symbole* mit zugehörigen *Parsing-Ausdrücken*, die die grammatikalische Struktur der Sätze definieren. Die beiden Symbolmengen unterscheiden sich voneinander und überschneiden sich nicht. Wenn von Symbolen die Rede ist, wird das Wort "Symbol" oft weggelassen. Die Vereinigung der Mengen der terminalen und nichtterminalen Symbole wird als Menge der *Symbole* bezeichnet.

Hier wird die Menge der *Terminalsymbole* nicht explizit verwaltet, sondern implizit als die Menge aller Zeichen definiert. Zu beachten ist, dass dies bedeutet, dass wir von Tcl die Fähigkeit übernehmen, den gesamten Unicode-Zeichensatz zu verarbeiten.

Ein Paar aus einem *Nichtterminal* und einem *Parsing-Ausdruck* wird auch als *grammatikalische Regel* oder kurz als *Regel* bezeichnet. Im Zusammenhang mit einer Regel wird das Nichtterminal häufig als linke Seite (LHS) und der Parsing-Ausdruck als rechte Seite (RHS) bezeichnet.

Der *Startausdruck* einer Grammatik ist ein Parsing-Ausdruck, aus dem alle Sätze, die in der durch die Grammatik festgelegten Sprache enthalten sind, *abgeleitet* werden. Um das Verständnis dieses Begriffs zu erleichtern, nehmen wir für einen Moment an, dass die rechte Seite jeder Regel sowie der Startausdruck entweder eine Folge von Symbolen oder eine Reihe alternativer Parsing-Ausdrücke sind. Im letzteren Fall kann die Regel als eine Menge von Regeln betrachtet werden, von denen jede eine Alternative für das Nichtterminal bereitstellt. Ein Parsing-Ausdruck A' ist nun eine Ableitung eines Parsing-Ausdrucks A, wenn wir eines der Nichtterminale N im Ausdruck und eine der alternativen Regeln R für N auswählen und dann das Nichtterminal in A durch die rechte Seite der ausgewählten Regel ersetzen. Hier wird deutlich, warum die Terminalsymbole so genannt werden. Sie lassen sich nicht weiter auflösen und beenden somit den Prozess der Ableitung neuer Ausdrücke. Ein Beispiel

```
    Rules
      (1)  A <- a B c
      (2a) B <- d B
      (2b) B <- e

    Some derivations, using starting expression A.

      A -/1/-> a B c -/2a/-> a d B c -/2b/-> a d e c
```

Ein abgeleiteter Ausdruck, der nur Terminale enthält, ist ein *Satz*. Die Menge aller Sätze, die aus dem Startausdruck abgeleitet werden können, ist die *Sprache* der Grammatik.

Einige Definitionen für Nichtterminale und Ausdrücke:

1. Ein Nichtterminal A heißt *reachable*, wenn sich aus dem Startausdruck ein Parsing-Ausdruck ableiten lässt, der A enthält.
2. Ein Nichtterminal A wird als *nützlich* bezeichnet, wenn es möglich ist, daraus einen Satz abzuleiten.

3. Ein Nichtterminal A wird als *rekursiv* bezeichnet, wenn es möglich ist, daraus einen Parsing-Ausdruck abzuleiten, der wiederum A enthält.
4. Die *FIRST-Menge* eines Nichtterminals A enthält alle Symbole, die als das Symbol ganz links in einem von A abgeleiteten Parsing-Ausdruck vorkommen können. Wenn die FIRST-Menge A selbst enthält, so wird dieses Nichtterminal als *linksrekursiv* bezeichnet.
5. Die *LAST-Menge* eines Nichtterminals A enthält alle Symbole, die als das Symbol ganz rechts in einem von A abgeleiteten Parsing-Ausdruck vorkommen können. Wenn die LAST-Menge A selbst enthält, so wird dieses Nichtterminal als *rechtsrekursiv* bezeichnet.
6. Die *FOLLOW-Menge* eines Nichtterminals A enthält alle Symbole, die in einem aus dem Startausdruck abgeleiteten Parsing-Ausdruck nach A auftreten können.
7. Ein Nichtterminal (oder ein Parsing-Ausdruck) wird als *nullfähig* bezeichnet, wenn sich daraus der leere Satz ableiten lässt.

Und auf der Grundlage der obigen Definitionen für Grammatiken gilt:

1. Eine Grammatik G ist genau dann *rekursiv*, wenn sie ein nichtterminales Symbol A enthält, das rekursiv ist. Die Begriffe *links-* und *rechtsrekursiv* sowie *nützlich* sind analog definiert.
2. Eine Grammatik ist *minimal*, wenn sie ausschließlich *erreichbare* und *nützliche* Nichtterminale enthält.
3. Eine Grammatik ist *wohlgeformt*, wenn sie nicht linksrekursiv ist. Solche Grammatiken sind zudem *vollständig*, was bedeutet, dass sie bei allen Eingabesätzen entweder erfolgreich sind oder fehlschlagen. Bei einer unvollständigen Grammatik hingegen gibt es Eingabesätze, bei denen der Versuch, diese mit der Grammatik abzugleichen, nicht beendet wird.
4. Da wir uns die Möglichkeit offenhalten möchten, eine Grammatik schrittweise in einem Container-Objekt aufzubauen, werden wir auf Phasen stoßen, in denen die rechte Seite einer oder mehrerer Regeln auf Symbole verweist, die dem Container noch nicht bekannt sind. Eine solche Grammatik bezeichnen wir als *ungültig*. Wir können den Begriff *unvollständig* nicht verwenden, da dieser Begriff bereits vergeben ist; siehe den letzten Punkt.

API der Container-Klasse

Das Paket exportiert die hier beschriebene API.

::grammar::peg *pegName* ?=|:=|<--|as|deserialize *src*?

Der Befehl erstellt ein neues Containerobjekt für eine Parsing-Ausdrucksgrammatik und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück. Die API, der der zurückgegebene Befehl folgt, wird im Abschnitt API für Container-Objekte beschrieben. Sie kann verwendet werden, um verschiedene Operationen auf den Container und die darin enthaltene Grammatik anzuwenden.

Der neue Container, d. h. die Grammatik, ist leer, wenn kein *src* angegeben wird. Andernfalls enthält er eine Kopie der in *src* enthaltenen Grammatik. *src* muss bei allen Operatoren mit Ausnahme von **deserialize** eine Referenz auf ein Container-Objekt sein. Der Operator **deserialize** verlangt stattdessen, dass *src* die Serialisierung einer Parsing Expression Grammar ist.

Eine leere Grammatik hat keine Nichtterminale, und der Startausdruck ist der leere Ausdruck, also Epsilon. Sie ist *gültig*, aber nicht *nützlich*.

API für Container-Objekte

Alle Grammatik-Container-Objekte stellen die folgenden Methoden zur Bearbeitung ihres Inhalts bereit:

***pegName* destroy**
Löscht die Grammatik, einschließlich ihres Speicherplatzes und des zugehörigen Befehls.

***pegName* clear**
Löscht die in *pegName* enthaltenen Definitionen der Grammatik, zerstört das Objekt jedoch *nicht*.

pegName = srcPEG
Weist *pegName* den Inhalt der in *srcPEG* enthaltenen Grammatik zu und überschreibt dabei etwaige bereits vorhandene Definitionen. Dies ist der Zuweisungsoperator für Grammatiken. Er kopiert die im Grammatikobjekt *srcPEG* enthaltene Grammatik über die Grammatikdefinition in *pegName*. Der bisherige Inhalt von *pegName* wird durch diesen Vorgang gelöscht.

Dieser Vorgang entspricht im Wesentlichen

```
    pegName deserialize [srcPEG serialize]
```

pegName --> dstPEG
Dies ist der Umkehrzuweisungsoperator für Grammatiken. Er kopiert die im Objekt *pegName* enthaltene Grammatik in die Grammatikdefinition des Objekts *dstPEG*. Der bisherige Inhalt von *dstPEG* wird durch diesen Vorgang gelöscht.

Dieser Vorgang entspricht im Wesentlichen

```
    dstPEG deserialize [pegName serialize]
```

***pegName* serialize**
Diese Methode serialisiert die in *pegName* gespeicherte Grammatik. Mit anderen Worten: Sie gibt einen Tcl-*value* zurück, der diese Grammatik vollständig beschreibt. Dies ermöglicht beispielsweise die Übertragung von Grammatiken über beliebige Kanäle, die Persistenz usw. Diese Methode bildet zudem die Grundlage sowohl für den Kopier-Konstruktor als auch für den Zuweisungsoperator.

Das Ergebnis dieser Methode muss über alle Implementierungen der Schnittstelle **grammar::peg** hinweg semantisch identisch sein. Dies ermöglicht es uns, Grammatiken zwischen verschiedenen Implementierungen derselben Schnittstelle zu kopieren.

Das Ergebnis ist eine Liste mit vier Elementen, die folgende Struktur aufweist:
1. Die Konstante "**grammar::peg**".
2. Ein Dictionary. Seine Schlüssel sind die Namen aller bekannten nichtterminalen Symbole, und die ihnen zugeordneten Werte sind die Parsing-Ausdrücke, die deren Satzstruktur beschreiben.
3. Ein Dictionary. Seine Schlüssel sind die Namen aller bekannten nichtterminalen Symbole, und die ihnen zugeordneten Werte liefern einem Matcher Hinweise auf die semantischen Werte, die durch das Symbol erzeugt werden.
4. Das letzte Element ist ein Parsing-Ausdruck, der *Startausdruck* der Grammatik.

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'
    Sign       <- '+' / '-'
    Number     <- Sign? Digit+
    Expression <- '(' Expression ')' / (Factor (MulOp Factor)*)
    MulOp      <- '*' / '/'
    Factor     <- Term (AddOp Term)*
    AddOp      <- '+'/'-'
    Term       <- Number
```

Eine mögliche Serialisierung lautet

```
    grammar::peg \
    {Expression {/ {x ( Expression )} {x Factor {* {x MulOp
    Factor}}}} \
     Factor     {x Term {* {x AddOp Term}}} \
     Term       Number \
     MulOp      {/ * /} \
     AddOp      {/ + -} \
     Number     {x {? Sign} {+ Digit}} \
     Sign       {/ + -} \
     Digit      {/ 0 1 2 3 4 5 6 7 8 9} \
    } \
    {Expression value     Factor     value \
     Term       value     MulOp      value \
     AddOp      value     Number     value \
     Sign       value     Digit      value \
    }
    Expression
```

Eine mögliche Variante, da die Reihenfolge der Nichtterminale im Dictionary keine Rolle spielt.

pegName* deserialize *serialization

Dies ist die Ergänzung zu **serialize**. Sie ersetzt die Grammatikdefinition in *pegName* durch die durch den Wert *serialization* beschriebene Grammatik. Der bisherige Inhalt von *pegName* wird durch diesen Vorgang gelöscht.

***pegName* is valid**

Ein Prädikat. Es prüft, ob die PEG in *pegName gültig* ist. Die Definition dieser Grammatikeigenschaft finden Sie im Abschnitt Begriffe und Konzepte. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn die PEG die geprüfte Eigenschaft aufweist, andernfalls auf **false**.

***pegName* start ?*pe*?**

Diese Methode definiert den *Startausdruck* der Grammatik. Sie ersetzt den zuvor definierten Startausdruck durch den Parsing-Ausdruck *pe*. Die Methode schlägt fehl und löst einen Fehler aus, wenn *pe* keinen gültigen Parsing-Ausdruck enthält, wie im Abschnitt Parsing-Ausdrücke festgelegt. In diesem Fall wird der bestehende Startausdruck nicht geändert. Die Methode gibt als Ergebnis die leere Zeichenkette zurück.

Wenn die Methode ohne Argument aufgerufen wird, gibt sie den aktuell definierten Startausdruck zurück.

***pegName* nonterminals**
Gibt die Menge aller der Grammatik bekannten nichtterminalen Symbole zurück.

pegName* nonterminal add *nt pe
Diese Methode fügt das Nichtterminal *nt* und den dazugehörigen Parsing-Ausdruck *pe* zur Menge der Nichtterminal-Symbole und Regeln der PEG hinzu, die im Objekt *pegName* enthalten sind. Die Methode schlägt fehl und löst einen Fehler aus, wenn entweder die Zeichenkette *nt* bereits als Symbol der Grammatik bekannt ist oder wenn *pe* keinen gültigen Parsing-Ausdruck enthält, wie im Abschnitt Parsing-Ausdrücke festgelegt. In diesem Fall wird die aktuelle Menge der nichtterminalen Symbole und Regeln nicht verändert. Die Methode gibt die leere Zeichenkette als Ergebnis zurück.

***pegName* nonterminal delete *nt1* ?*nt2* ...?**
Diese Methode entfernt die genannten Symbole *nt1* und *nt2* aus der Menge der nichtterminalen Symbole der PEG, die im Objekt *pegName* enthalten ist. Die Methode schlägt fehl und löst einen Fehler aus, wenn eine der Zeichenfolgen nicht als nichtterminales Symbol bekannt ist. In diesem Fall wird die aktuelle Menge der nichtterminalen Symbole nicht verändert. Die Methode gibt als Ergebnis die leere Zeichenkette zurück.

Die gespeicherte Grammatik wird ungültig, wenn die gelöschten Nichtterminale von der rechten Seite (RHS) noch bekannter Regeln referenziert werden.

pegName* nonterminal exists *nt
Ein Prädikat. Es führt Tests durch, um zu prüfen, ob das nichtterminale Symbol *nt* dem PEG in *pegName* bekannt ist. Das Ergebnis ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn das Symbol *nt* bekannt ist, andernfalls auf **false**.

pegName* nonterminal rename *nt ntnew
Diese Methode benennt das nichtterminale Symbol *nt* in *ntnew* um. Die Methode schlägt fehl und löst einen Fehler aus, wenn entweder *nt* nicht als nichtterminales Symbol bekannt ist oder wenn *ntnew* ein bekanntes Symbol ist. Die Methode gibt als Ergebnis die leere Zeichenkette zurück.

***pegName* nonterminal mode *nt* ?*mode*?**
Diese Methode gibt den mit dem Nichtterminal-Symbol *nt* verknüpften semantischen Modus zurück oder legt ihn fest. Wird kein *mode* angegeben, wird der aktuelle Modus des Nichtterminals zurückgegeben. Andernfalls wird der aktuelle Modus auf *mode* gesetzt. Die Methode schlägt fehl und löst einen Fehler aus, wenn *nt* nicht als Nichtterminal bekannt ist. Der vom Paket **grammar::peg::interpreter** implementierte Grammatik-Interpreter erkennt die folgenden Modi:

value
Der semantische Wert des Nichtterminals ist der abstrakte Syntaxbaum, der aus den ASTs der rechten Seite und einem Knoten für das Nichtterminal selbst gebildet wird.

match
Der semantische Wert des Nichtterminals ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für die von der rechten Seite (RHS) abgeglichene Zeichenkette besteht. Die von der rechten Seite (RHS) generierten ASTs werden verworfen.

leaf

Der semantische Wert des Nichtterminals ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal selbst besteht. Die von der rechten Seite (RHS) erzeugten ASTs werden verworfen.

discard
Das Nichtterminal hat keinen semantischen Wert. Die von der rechten Seite (RHS) generierten ASTs werden (ebenfalls) verworfen.

pegName* nonterminal rule *nt
Diese Methode gibt den Parsing-Ausdruck zurück, der dem Nichtterminal *nt* zugeordnet ist. Die Methode schlägt fehl und löst einen Fehler aus, wenn *nt* nicht als Nichtterminal bekannt ist.

***pegName* unknown nonterminals**
Diese Methode gibt eine Liste zurück, die die Namen aller nichtterminalen Symbole enthält, auf die auf der rechten Seite einer grammatikalischen Regel Bezug genommen wird, für deren Struktur jedoch keine Regel definiert ist. Mit anderen Worten: eine Liste der nichtterminalen Symbole, die die Grammatik ungültig machen. Die Grammatik ist gültig, wenn diese Liste leer ist.

Parsing-Ausdrücke

Verschiedene Methoden von PEG-Container-Objekten erwarten einen Parsing-Ausdruck als Argument oder geben einen solchen zurück. In diesem Abschnitt wird das Format solcher Parsing-Ausdrücke festgelegt.

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
2. Die Zeichenfolge "**alnum**" ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphanumerische Zeichen.
3. Die Zeichenfolge "**alpha**" ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Zeichen.
4. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
5. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
6. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.
7. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *ordered choice*, auch *prioritized choice*.
8. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
9. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
10. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
11. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das "*and lookahead*"-*Prädikat*.
12. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
13. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Beispiele für Parsing-Ausdrücke wurden bereits in der Beschreibung der Methode **serialize** aufgeführt.

Parsing-Expression-Grammatiken

Für mathematisch Interessierte: Ein PEG ist ein 4-Tupel (VN, VT, R, eS), wobei

- VN ist eine Menge von *nichtterminalen Symbolen*,
- VT ist eine Menge von *Terminalsymbolen*,
- R ist eine endliche Menge von Regeln, wobei jede Regel ein Paar (A, e) ist, wobei A in VN liegt und *e* ein *Parsing-Ausdruck* ist.
- eS ist ein Parsing-Ausdruck, der *Startausdruck*.

Weitere Bedingungen sind

- Die Schnittmenge von VN und VT ist leer.
- Für jedes A in VN gibt es genau ein Paar (A, e) in R. Mit anderen Worten: R ist eine Funktion von nichtterminalen Symbolen zu Parsing-Ausdrücken.

Parsing-Ausdrücke werden induktiv definiert über

- Die leere Zeichenkette (Epsilon) ist ein Parsing-Ausdruck.
- Ein Terminalsymbol *a* ist ein Parsing-Ausdruck.
- Ein nichtterminales Symbol *A* ist ein Parsing-Ausdruck.
- *e1e2* ist ein Parsing-Ausdruck für die Parsing-Ausdrücke *e1* und *e2*. Das heißt *Sequenz*.
- *e1*/*e2* ist ein Parsing-Ausdruck für die Parsing-Ausdrücke *e1* und *e2*. Dies wird als *geordnete Auswahl* bezeichnet.
- *e** ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e*. Dies wird als *Null-oder-mehr-Wiederholungen* bezeichnet, auch bekannt als *Kleene-Abschluss*.
- *e*+ ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e*. Dies wird als *eine oder mehrere Wiederholungen* bezeichnet, auch bekannt als *positiver Kleene-Abschluss*.
- !*e* ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e*. Dies wird als *Prädikat ohne Vorausschau* bezeichnet.
- &*e* ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e*. Dies wird als "*And-Lookahead-Prädikat*" bezeichnet.

PEGs werden verwendet, um eine grammatikalische Struktur für Symbolströme über VT zu definieren. Sie stellen eine moderne Weiterentwicklung älterer Formalismen dar, die von Alexander Birham entwickelt wurden. Diese Formalismen wurden als TS (TMG-Erkennungsschema) und gTS (verallgemeinertes TS) bezeichnet. Später wurden sie in TPDL (Top-Down-Parsing-Sprachen) und gTPDL (verallgemeinertes TPDL) umbenannt.

Sie lassen sich leicht durch rekursive Abstiegsparser mit Backtracking implementieren. Damit stehen sie in Verwandtschaft zu den kontextfreien LL(k)-Grammatiken.

Literatur

1. The Packrat Parsing and Parsing Expression Grammars Page, von Bryan Ford, Massachusetts Institute of Technology. Das ist die Haupteinstiegsseite zu PEGs und ihrer Umsetzung durch Packrat-Parser.
2. *Parsing Techniques - A Practical Guide* http://www.cs.vu.nl/~dick/PTAPG.html, ein Online-Buch, das eine klare, leicht verständliche und umfassende Darstellung zahlreicher verschiedener Parsing-Techniken mit ihren Zusammenhängen und Anwendungsmöglichkeiten bietet, einschließlich Techniken zur Fehlerbehebung.

3. Compilers and Compiler Generators, ein Online-Buch, das CoCo/R verwendet, einen Generator für rekursive Abstiegsparser.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_peg* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Grammatiken und endliche Automaten

Copyright


```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


peg_interp

Stichwörter: peg_interp, LL(k), TDPL, context-free languages, expression, grammar, matching, parsing, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer, virtual machine

Name

grammar::peg::interp – Interpreter zum Parsen von Ausdrucksgrammatiken

Übersicht

package require Tcl 8.5 9
package require grammar::mengine ?**0.1**?
package require grammar::peg::interp ?**0.1.2**?
::grammar::peg::interp::setup *peg*
::grammar::peg::interp::parse *nextcmd errorvar astvar*

Beschreibung

Dieses Paket stellt Befehle für den kontrollierten Abgleich eines Zeichenstroms anhand einer Parsing Expression Grammar sowie für die Erstellung eines abstrakten Syntaxbaums für den Strom und Teilausdrücke bereit.

Es baut auf der vom Paket **grammar::me::tcl** bereitgestellten virtuellen Maschine auf und interpretiert die ihm übergebene Parsing Expression Grammar direkt. Mit anderen Worten: Die Grammatik wird *nicht* vorkompiliert, sondern unverändert verwendet.

Die zu interpretierende Grammatik wird aus einem Container-Objekt entnommen, das der durch das Paket **grammar::peg::container** festgelegten Schnittstelle entspricht. Es werden lediglich die relevanten Teile in den Zustand dieses Pakets kopiert.

Zu beachten ist, dass das Paket genau eine Instanz des Interpreters bereitstellt. Um eine zweite

Grammatik zu interpretieren, muss man eine laufende Interpretation entweder abbrechen oder abschließen oder die Grammatiken in verschiedene Tcl-Interpreter legen.

Außerdem ist zu beachten, dass die Implementierung eine Pull-Verarbeitung der Eingabe voraussetzt. Der Interpreter ruft Zeichen aus dem Eingabestrom ab, sobald er sie benötigt. Für den Einsatz in einer Push-Umgebung, in der die Umgebung neue Zeichen übergibt, sobald sie eintreffen, muss die Engine in einen eigenen Thread gelegt werden.

Die Interpreter-API

Das Paket exportiert die folgende API

::grammar::peg::interp::setup *peg*

Dieser Befehl initialisiert den Interpreter neu. Er gibt die leere Zeichenkette zurück. Dieser Befehl muss vor jedem Abgleich als Erstes aufgerufen werden.

Sein Argument *peg* ist das Handle eines Objekts, das die zu interpretierende Parsing Expression Grammar enthält. Diese Grammatik muss gültig sein, andernfalls wird ein Fehler ausgelöst.

::grammar::peg::interp::parse *nextcmd errorvar astvar*

Dieser Befehl interpretiert die geladene Grammatik und versucht, sie mit dem Zeichenstrom abzugleichen, der durch das Befehlspräfix *nextcmd* dargestellt wird.

Das Befehlspräfix *nextcmd* steht für den Eingabezeichenstrom und wird vom Interpreter aufgerufen, sobald ein neues Zeichen aus dem Strom benötigt wird. Die Rückruffunktion muss entweder eine leere Liste zurückgeben oder eine Liste mit vier Elementen: das Token, sein Lexem-Attribut und seine Position als Zeilennummer und Spaltenindex, in dieser Reihenfolge. Die leere Liste signalisiert, dass das Ende des Eingabestroms erreicht wurde. Das Lexem-Attribut wird im Terminal-Cache gespeichert, von der Maschine aber sonst nicht verwendet.

Das Ergebnis des Befehls ist ein boolescher Wert, der angibt, ob der Abgleich erfolgreich war (**true**) oder nicht (**false**). Schlägt der Abgleich fehl, werden Fehlerinformationen in der Variablen gespeichert, auf die *errorvar* verweist. Die Variable, auf die *astvar* verweist, enthält stets den erzeugten abstrakten Syntaxbaum; im Fehlerfall ist er nur unvollständig und möglicherweise fehlerhaft.

Der abstrakte Syntaxbaum ist eine verschachtelte Liste, wie im Abschnitt **AST-WERTE** des Dokuments *grammar::me_ast* beschrieben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *grammar_peg* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Grammatiken und endliche Automaten

Copyright

```
Copyright (c) 2005-2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


hook

hook
Stichwörter: hook, callback, event, observer, producer, publisher, subject, subscriber, uevent

Name
hook – Hooks

Übersicht
package require Tcl 8.5 9
package require hook ?**0.3**?
hook bind ?*subject*? ?*hook*? ?*observer*? ?*cmdPrefix*?
hook call *subject hook* ?*args...*?
hook forget *object*
hook cget *option*
hook configure option *value* ...

Beschreibung
Dieses Paket stellt den Ensemble-Befehl **hook** bereit, der das Subject-Observer-Muster implementiert. Es ermöglicht *Subjekten* – bei denen es sich um *Module*, *Objekte*, *Widgets* usw. handeln kann –, *Hooks* synchron aufzurufen, die an eine beliebige Anzahl von Abonnenten, sogenannte *Beobachter*, gebunden sein können. Ein Subject kann eine beliebige Anzahl unterschiedlicher Hooks aufrufen, und eine beliebige Anzahl von Observern kann Rückruffunktionen an einen bestimmten Hook binden, der von einem bestimmten Subject aufgerufen wird. Hook-Bindungen können abgefragt und gelöscht werden.

Diese Man-Seite dient ausschließlich als Nachschlagewerk.

Begriffe

Einleitung
Tcl-Module senden Benachrichtigungen an andere Module in der Regel auf zwei Arten: über Tk-Ereignisse und über Callback-Optionen wie die Option **-yscrollcommand** des Text-Widgets. Tk-Ereignisse stehen nur in Tk zur Verfügung, und Callback-Optionen erfordern eine enge Kopplung zwischen den Modulen, die die Benachrichtigung senden und empfangen.

Eine lose Kopplung zwischen Sender und Empfänger ist jedoch häufig wünschenswert. Im Sinne des Model-View-Controller-Musters kann eine Ansicht einen Befehl (der auf einer Benutzereingabe beruht) an den Controller senden, der daraufhin das Modell aktualisiert. Das Modell kann dann einen Hook aufrufen, *den alle relevanten Ansichten abonnieren.* Das Model ist von den Views entkoppelt und muss tatsächlich nicht wissen, ob überhaupt Views existieren.

Derzeit verfügt Tcl/Tk über keinen Standardmechanismus zur Implementierung einer derartigen losen Kopplung. Dieses Paket definiert einen neuen Befehl, **hook**, der genau einen solchen Mechanismus implementiert.

Bindings

Der Befehl **hook** verwaltet eine Sammlung von Hook-Bindings. Ein Hook-Binding besteht aus vier Elementen:

1. Ein *subject*: Der Name der Entität, die den Hook aufrufen wird.
2. Der *hook* selbst. Ein Hook spiegelt in der Regel ein Ereignis im Leben des *subject* wider, das für andere Entitäten von Interesse sein könnte. Ein *hook* hat einen Namen und kann zudem Argumente haben. Hook-Namen sind beliebige Zeichenfolgen. Jedes *subject* muss die Namen und Argumente der Hooks dokumentieren, die es aufrufen kann.
3. Der Name des *observer*, der den *hook* vom *subject* empfangen möchte.
4. Ein Befehlspräfix, an das die *hook*-Argumente angehängt werden, wenn die Bindung ausgeführt wird.

Subjekte und Beobachter

Der Einfachheit halber werden in diesem Dokument Subjekte und Beobachter gemeinsam als *Objekte* bezeichnet, wobei keine Anforderungen hinsichtlich der tatsächlichen Implementierung dieser *Objekte* gestellt werden. Ein Objekt kann ein **TclOO**-, **Snit**- oder **XOTcl**-Objekt, ein Tcl-Befehl, ein Namespace, ein Modul, ein von einem anderen Objekt verwaltetes Pseudo-Objekt (so wie Tags vom Tk-Text-Widget verwaltet werden) oder einfach ein bekannter Name.

Die Namen von Subjekten und Beobachtern sind beliebige Zeichenfolgen; da "**hook**" jedoch möglicherweise auf Paketebene verwendet wird, sind Konventionen erforderlich, um Namenskonflikte zwischen Paketen zu vermeiden, die von verschiedenen Personen erstellt wurden.

Daher sollte jeder im Code auf Kern- oder Paketebene verwendete Subjekt- oder Beobachtername wie ein Tcl-Befehlsname aussehen und in einem Namespace definiert sein, der zu dem jeweiligen Paket gehört. Betrachten Sie beispielsweise einen Ensemble-Befehl **::foo**, der eine Reihe von Pseudo-Objekten erstellt und **hook** zum Versenden von Benachrichtigungen verwendet. Die Pseudo-Objekte tragen Namen, die keine Befehle sind, und existieren in einem eigenen Namespace, ähnlich wie Dateihandles. Um Namenskonflikte mit Subjekten zu vermeiden, die von anderen Paketen definiert wurden, sollten Benutzer von **hook** diese **::foo** -Handles Namen wie **::foo::1**, **::foo::2** und so weiter.

Da Objektnamen beliebige Zeichenfolgen sind, kann der Anwendungscode beliebige zusätzliche Konventionen verwenden, die sich aus den Anforderungen der Anwendung ergeben.

Literatur

Hook stellt die folgenden Befehle zur Verfügung:

hook bind ?*subject*? ?*hook*? ?*observer*? ?*cmdPrefix*?

Dieser Unterbefehl dient zum Erstellen, Aktualisieren, Löschen und Abfragen von Hook-Bindungen.

Wird er ohne Argumente aufgerufen, gibt er eine Liste der Subjekte mit Hooks zurück, an die derzeit Beobachter gebunden sind.

Wird er mit einem Argument – einem *subject* – aufgerufen, gibt er eine Liste der Hooks dieses

Subjekts zurück, an die derzeit Beobachter gebunden sind.

Wird er mit zwei Argumenten – einem *subject* und einem *hook* – aufgerufen, gibt er eine Liste der Beobachter zurück, die derzeit an dieses *subject* und diesen *hook* gebunden sind.

Wird er mit drei Argumenten – einem *subject*, einem *hook* und einem *observer* – aufgerufen, gibt er die eigentliche Bindung, das Befehlspräfix, das wenn der Hook aufgerufen wird, ausgeführt werden soll, oder die leere Zeichenkette zurück, falls keine solche Bindung vorhanden ist.

Wird er mit vier Argumenten aufgerufen, erstellt, aktualisiert oder löscht er eine Bindung. Ist *cmdPrefix* die leere Zeichenkette, werden alle bestehenden Bindungen für das *subject*, den *hook* und den *observer* gelöscht; nichts wird zurückgegeben. Andernfalls muss *cmdPrefix* ein Befehlspräfix sein, das so viele zusätzliche Argumente entgegennimmt, wie für *subject* und *hook* dokumentiert sind. Die Bindung wird hinzugefügt oder aktualisiert, und der Beobachter wird zurückgegeben.

Wenn der *observer* die leere Zeichenkette "", ist, wird ein neues Binding unter Verwendung eines automatisch generierten Beobachternamens der Form **::hook::ob<number>** erstellt. Der automatisch generierte Name wird zurückgegeben und kann wie gewohnt zum Abfragen, Aktualisieren und Löschen der Bindung verwendet werden. Werden stets automatische Beobachternamen verwendet, wird der Beobachtername faktisch zu einer eindeutigen Bindungs-ID.

Es ist möglich, **hook bind** aufzurufen, um eine Bindung an ein *Subjekt* und einen *hook* zu erstellen oder zu löschen, während man sich in einer Beobachterbindung für dasselbe *Subjekt* und denselben *hook* befindet. Die folgenden Regeln legen fest, was geschieht, wenn

```
    hook bind $s $h $o $binding
```

wird während der Ausführung von

```
    hook call $s $h
```

1. Eine Bindung wird nach ihrer Löschung niemals mehr aufgerufen.
2. Wenn eine Bindung aufgerufen wird, wird stets das zuletzt angegebene Befehlspräfix verwendet.
3. Die Menge der Beobachter, deren Bindings aufgerufen werden sollen, wird zu Beginn der Ausführung dieser Methode festgelegt und ändert sich danach nicht mehr, mit der Ausnahme, dass gelöschte Bindings nicht aufgerufen werden.

Insbesondere:

1. Wenn die Bindung von **$o** mit **$s** und **$h** gelöscht wird und die Bindung von **$o** während dieser Ausführung von
   ```
       hook call $s $h
   ```
2. Es wird nicht aufgerufen. (Hinweis: Sie ist möglicherweise bereits aufgerufen worden; und aller Wahrscheinlichkeit nach löscht sie sich gerade selbst.)
3. Wenn **$o** das Befehlspräfix ändert, das an **$s** und **$h** gebunden ist, und wenn die Bindung von **$o** während dieser Ausführung von

```
    hook call $s $h
```

4. Die neue Bindung wird wenn der Zeitpunkt gekommen ist aufgerufen. (Allerdings ist es wahrscheinlich wieder die Bindung "**$o**", die die Änderung vornimmt.)
5. Wenn ein neuer Beobachter an **$s** und **$h** gebunden wird, wird seine Bindung erst beim nächsten Aufruf von

```
    hook call $s $h
```

hook call *subject hook* ?*args*...?

Dieser Befehl wird aufgerufen, wenn das benannte *subject* den benannten *hook* aufrufen möchte. Alle relevanten Bindungen werden mit den angegebenen Argumenten im globalen Namespace aufgerufen. Hinweis: Die Bindungen werden synchron aufgerufen, bevor der Befehl zurückgibt; dadurch können die *args* Verweise auf Entitäten enthalten, die sofort nach dem Aufruf des Hooks bereinigt werden.

Die Reihenfolge, in der die Bindungen aufgerufen werden, ist nicht garantiert. Wenn die Reihenfolge unter den Beobachtern gewahrt bleiben soll, definieren Sie einen Beobachter und lassen Sie dessen Bindings die anderen Rückruffunktionen direkt in der richtigen Reihenfolge aufrufen.

Da der Mechanismus **hook** darauf ausgelegt ist, eine lose Kopplung zu unterstützen, wird davon ausgegangen, dass das *Subjekt* weder Kenntnis von den Beobachtern hat noch Erwartungen hinsichtlich der Rückgabewerte hegt. Dies hat eine Reihe von Konsequenzen:

1. **hook call** gibt die leere Zeichenkette zurück.
2. Normale Werte, die von Beobachter-Bindungen gegeben werden, werden ignoriert.
3. Fehler und andere Ausnahmewerte werden standardmäßig normal weitergegeben. Dies entspricht jedoch selten den Erwartungen, da die Subjekte in der Regel keine Kenntnis von den Beobachtern haben und daher nicht über die erforderlichen Kompetenzen verfügen, um mit deren Fehlern umzugehen. Da es sich hierbei um ein anwendungsspezifisches Problem handelt, werden Anwendungen in der Regel ein **-errorcommand** definieren wollen.

Wenn die Konfigurationsoption **-errorcommand** einen nicht leeren Wert aufweist, wird dieser Wert bei allen Fehlern und anderen Ausnahmerückgaben in Observer-Bindungen aufgerufen. Weitere Informationen zu Konfigurationsoptionen finden Sie unter **hook configure** weiter unten.

hook forget *object*

Dieser Befehl löscht alle vorhandenen Bindungen, in denen das genannte *object* entweder als *subject* oder als *observer* auftritt. Durch diese Methode gelöschte Bindungen werden nie wieder aufgerufen. Insbesondere gilt:

1. Wenn bei einem Aufruf von **hook call** ein Beobachter übersehen wird, werden etwaige noch nicht aufgerufene Bindungen, die dieser möglicherweise zum betreffenden Subjekt und Hook hatte, anschließend *nicht* aufgerufen.
2. Wenn ein Subjekt **$s** während eines Aufrufs von

```
hook call $s $h
```

3. vergessen wird, gibt **hook call** zurück, sobald die aktuelle Bindung zurückkehrt. Es werden keine weiteren Bindungen aufgerufen.

hook cget *option*
Dieser Befehl gibt den Wert einer der Konfigurationsoptionen des Befehls "**hook**" zurück.

hook configure option *value* ...
Dieser Befehl legt den Wert einer oder mehrerer Konfigurationsoptionen des Befehls "**hook**" fest:

-errorcommand *cmdPrefix*
Wenn der Wert dieser Option die leere Zeichenkette "", gibt, werden Fehler und andere Ausnahmen, die in Bindungsskripten auftreten, wie gewohnt weitergegeben. Andernfalls muss es sich um ein Befehlspräfix handeln, das drei zusätzliche Argumente entgegennimmt:
1. eine Liste mit vier Elementen {Thema, Aufhänger, Argumentliste, Beobachter},
2. die Ergebniszeichenfolge und
3. das Dictionary der Rückgabemöglichkeiten.

Anhand dieser Informationen kann **-errorcommand** entscheiden, ob der Fehler protokolliert, **interp bgerror** aufgerufen, die fehlerhafte Bindung gelöscht (wodurch verhindert wird, dass der Fehler ein zweites Mal auftritt) oder Ähnliches erfolgen soll.

-tracecommand *cmdPrefix*
Der Wert der Option sollte ein Befehlspräfix sein, das vier Argumente entgegennimmt:
1. a *subject*,
2. a *hook*,
3. eine Liste der Werte der Argumente des Hooks sowie
4. eine Liste der *Objekte*, für die der Hook aufgerufen wurde.

Der Befehl wird für jeden aufgerufenen Hook ausgeführt. Auf diese Weise kann die Anwendung die Ausführung der Hooks zu Debugging-Zwecken nachverfolgen.

Beispiel

Das Modul **::model** ruft den Hook als Reaktion auf Befehle auf, die die Daten des Modells ändern:

```
hook call ::model <Update>
```

Das Megawidget "**.view**" zeigt den Modellstatus an und muss über Modellaktualisierungen informiert werden. Daher abonniert es den Hook "" des Modells "::model".

```
hook bind ::model <Update> .view [list .view ModelUpdate]
```

Wenn **::model** den Hook aufruft, wird der Unterbefehl "ModelUpdate" von **.view** aufgerufen.

Später wird das Megawidget "**.view**" zerstört. In seinem Destruktor teilt es dem *hook* mit, dass es nicht mehr existiert:

```
hook forget .view
```

Alle Bindings, die **.view** betreffen, werden gelöscht.

Danksagungen

Hook wurde von William H. Duquette entwickelt und umgesetzt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *hook* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright


```
Copyright (c) 2010, by William H. Duquette
```


html

html

Stichwörter: html, checkbox, checkbutton, form, radiobutton, table

Name

html – Prozeduren zur Erstellung von HTML-Strukturen

Übersicht

package require Tcl 8.5 9
package require html ?**1.6**?
::html::author *author*
::html::bodyTag *args*
::html::cell *param value* ?*tag*?
::html::checkbox *name value*
::html::checkSet *key sep list*
::html::checkValue *name* ?*value*?
::html::closeTag
::html::default *key* ?*param*?
::html::description *description*
::html::end
::html::eval *arg* ?*args*?
::html::extractParam *param key* ?*varName*?
::html::font *args*
::html::for *start test next body*
::html::foreach *varlist1 list1* ?*varlist2 list2 ...*? *body*
::html::formValue *name* ?*defvalue*?
::html::getFormInfo *args*
::html::getTitle
::html::h *level string* ?*param*?

::html::h1 *string* ?*param*?
::html::h2 *string* ?*param*?
::html::h3 *string* ?*param*?
::html::h4 *string* ?*param*?
::html::h5 *string* ?*param*?
::html::h6 *string* ?*param*?
::html::hdrRow *args*
::html::head *title*
::html::headTag *string*
::html::html_entities *string*
::html::if *expr1 body1* ?*elseif expr2 body2 ...*? ?*else bodyN*?
::html::init ?*list*?
::html::keywords *args*
::html::mailto *email* ?*subject*?
::html::meta *args*
::html::meta_name *args*
::html::meta_equiv *args*
::html::meta_charset *charset*
::html::css *href*
::html::css-clear
::html::js *href*
::html::js-clear
::html::minorList *list* ?*ordered*?
::html::minorMenu *list* ?*sep*?
::html::nl2br *string*
::html::openTag *tag* ?*param*?
::html::paramRow *list* ?*rparam*? ?*cparam*?
::html::passwordInput ?*name*?
::html::passwordInputRow *label* ?*name*?
::html::quoteFormValue *value*
::html::radioSet *key sep list*
::html::radioValue *name value*
::html::refresh *seconds url*
::html::row *args*
::html::select *name param choices* ?*current*?
::html::selectPlain *name param choices* ?*current*?
::html::set *var val*
::html::submit *label* ?*name*? ?*title*?
::html::tableFromArray *arrname* ?*param*? ?*pat*?
::html::tableFromList *querylist* ?*param*?
::html::textarea *name* ?*param*? ?*current*?
::html::textInput *name value args*
::html::textInputRow *label name value args*
::html::varEmpty *name*
::html::while *test body*
::html::doctype *id*

::html::wrapTag *tag* ?*text*? ?*args*?

Beschreibung

Das Paket **html** stellt Befehle zur Verfügung, die HTML generieren. Diese Befehle geben in der Regel eine HTML-Zeichenkette als Ergebnis zurück. Insbesondere geben sie ihr Ergebnis nicht über **stdout** aus.

Der Befehl **::html::init** sollte frühzeitig aufgerufen werden, um das Modul zu initialisieren. Mit dieser Prozedur können Sie auch Standardwerte für HTML-Tag-Parameter festlegen.

::html::author *author*
: *Nur als Nebeneffekt*. Rufen Sie diese Funktion vor **::html::head** auf, um einen Autor für die Seite festzulegen. Der Autor wird in einem Hinweis im HEAD-Bereich vermerkt.

::html::bodyTag *args*
: Erzeugt ein *body*-Tag. Die Tag-Parameter werden aus *args* oder aus dem Text des *body*-Tags übernommen. Die Attribute sind mit **::html::init** definiert.

::html::cell *param value* ?*tag*?
: Erzeugt ein *td*- (oder *th*-)Tag, einen Wert und ein schließendes *td*- (oder *th*-)Tag. Die Tag-Parameter stammen aus *param* oder aus TD.*-Attributen, die mit **::html::init** definiert wurden. Hierbei wird **::html::font** verwendet, um ein Standard-*Schriftart*-Tag in die Tabellenzelle einzufügen. Das Argument *tag* ist standardmäßig auf "td" gesetzt.

::html::checkbox *name value*
: Erzeugt ein *Checkbox*-Formularelement mit dem angegebenen Name und Wert. Dabei wird **::html::checkValue** verwendet.

::html::checkSet *key sep list*
: Erstellt eine Reihe von *Checkbox*-Formularelementen und zugehörigen Beschriftungen. Die *Liste* sollte abwechselnd Beschriftungen und Werte enthalten. Dabei wird **::html::checkbox** verwendet. Alle *Checkbox*-Schaltflächen haben denselben *key* als Name. Das *sep* ist Text, der zur Trennung der Elemente verwendet wird.

::html::checkValue *name* ?*value*?
: Erzeugt den Eintrag "name=*name* value=*value*" für ein *Checkbox*-Formularelement. Wenn die CGI-Variable *name* den Wert *value* hat, wird dem Rückgabewert "SELECTED" hinzugefügt. Der Standardwert für *value* ist "1".

::html::closeTag
: Entnimmt ein Tag aus dem von **::html::openTag** erstellten Stapel und generiert das entsprechende schließende Tag (z. B.).

::html::default *key* ?*param*?
: Diese Prozedur wird von **::html::tagParam** verwendet, um den Namen und die Werteliste der Parameter für einen Tag zu generieren. Die Prozedur **::html::default** dient dazu, Standardwerte für diejenigen Elemente zu generieren, die noch nicht in *param* enthalten sind. Wenn der durch *key* identifizierte Wert mit einem Wert in *param* übereinstimmt, gibt diese Prozedur die leere Zeichenkette zurück. Andernfalls gibt sie eine Zeichenkette im Format "parameter=value" für ein durch *key* identifiziertes Formularelement zurück. Der *key* hat die Form "tag.parameter" (z. B. body.bgcolor). Verwenden Sie **::html::init**, um Standardwerte zu registrieren. *param* ist standardmäßig die leere Zeichenkette.

::html::description *description*
: *Nur als Nebeneffekt*. Rufen Sie diese Funktion vor **::html::head** auf, um ein *Meta*-Tag mit

einer Beschreibung für die Seite zu definieren. Dieses Tag wird später beim Aufruf von **::html::head** generiert.

::html::end
Alle offenen Tags aus dem Stapel entfernen und die entsprechenden schließenden HTML-Tags generieren (z. B.).

::html::eval *arg* ?*args*?
Diese Prozedur ähnelt dem integrierten Tcl-Befehl **eval**. Der einzige Unterschied besteht darin, dass sie "" gibt zurück, sodass sie aus einer HTML-Vorlagendatei aufgerufen werden kann, ohne dass unerwünschte Ergebnisse angehängt werden.

::html::extractParam *param key* ?*varName*?
Dies ist eine Parsing-Prozedur, die den Wert von *key* aus *param* extrahiert, wobei es sich um eine Liste im HTML-Stil vom Typ "name=quotedvalue" handelt. *varName* wird als Name einer Tcl-Variablen verwendet, deren Wert auf den in den Parametern gefundenen Wert gesetzt wird. Die Funktion gibt 1 zurück, wenn der Parameter in *param* gefunden wurde, andernfalls gibt sie 0 zurück. Wenn *varName* nicht angegeben ist, wird *key* als Variablenname verwendet.

::html::font *args*
Erzeugt ein Standard-*Schriftart*-Tag. Die Parameter für das Tag werden aus *args* und den mit **::html::init** definierten HTML-Standardwerten übernommen.

::html::for *start test next body*
Diese Prozedur ähnelt der integrierten Tcl-Kontrollstruktur **for**. Anstatt den Körper auszuwerten, gibt sie den durch "subst" ersetzten *Körper* zurück. Bei jeder Iteration der Schleife wird eine weitere Zeichenkette an den Wert angehängt.

::html::foreach *varlist1 list1 ?varlist2 list2 ...? body*
Diese Prozedur ähnelt der integrierten Tcl-Kontrollstruktur **foreach**. Anstatt die Auswertung des Körpers durchzuführen, gibt sie den durch "subst" bearbeiteten *Körper* zurück. Bei jeder Iteration der Schleife wird eine weitere Zeichenkette an den Wert angehängt.

::html::formValue *name* ?*defvalue*?
Gibt ein Name-Wert-Paar zurück, wobei der Wert – sofern vorhanden – anhand vorhandener CGI-Daten initialisiert wird. Das Ergebnis hat folgende Form:

```
name="fred" value="freds value"
```

::html::getFormInfo *args*
Erzeugt versteckte Felder, um Formularwerte zu erfassen. Wenn *args* leer ist, werden für alle CGI-Werte versteckte Felder erzeugt. Andernfalls ist *args* eine Liste von Zeichenfolgenmustern, die mit den Namen von Formularelementen übereinstimmen sollen.

::html::getTitle
Gibt die Titelzeichenfolge ohne das umgebende *title*-Tag zurück, das durch einen vorherigen Aufruf von **::html::title** gesetzt wurde.

::html::h *level string* ?*param*?
Erzeugt einen Überschriften-Tag (z. B. *h***level**). Der *string* wird in die Überschrift eingebettet, und *param* wird für die Tag-Parameter verwendet.

::html::h1 *string* ?*param*?
Erstellt ein *h1*-Tag. Siehe **::html::h**.

::html::h2 *string* ?*param*?

Erzeugt ein *h2*-Tag. Siehe **::html::h**.

::html::h3 *string* ?*param*?
Erzeugt ein *h3*-Tag. Siehe **::html::h**.

::html::h4 *string* ?*param*?
Erzeugt ein *h4*-Tag. Siehe **::html::h**.

::html::h5 *string* ?*param*?
Erstellt ein *h5*-Tag. Siehe **::html::h**.

::html::h6 *string* ?*param*?
Erzeugt ein *h6*-Tag. Siehe **::html::h**.

::html::hdrRow *args*
Erzeugt eine Tabellenzeile einschließlich der *tr*- und *th*-Tags. Jeder Wert in *args* wird in eine eigene Tabellenzelle eingefügt. Dabei wird **::html::cell** verwendet.

::html::head *title*
Erstellt den *head*-Abschnitt, der den *Titel* der Seite enthält. Wenn zuvor Aufrufe an **::html::author**, **::html::keywords**, **::html::description** oder **::html::meta** aufgerufen wurden, werden zusätzliche Tags in den *head*-Bereich eingefügt. Dadurch verbleibt ein offener *html*-Tag, der mit **::html::openTag** auf den Stack geschoben wurde.

::html::headTag *string*
Speichert einen Tag zur Einbindung in den von **::html::head** generierten *head*-Abschnitt. Der *String* umfasst den gesamten Inhalt des Tags mit Ausnahme der umschließenden spitzen Klammern < >.

::html::html_entities *string*
Dieser Befehl ersetzt alle Sonderzeichen in der *Zeichenkette* durch ihre HTML-Entitäten und gibt den geänderten Text zurück.

::html::if *expr1 body1* ?elseif *expr2 body2 ...*? ?else *bodyN*?
Diese Prozedur ähnelt der integrierten Tcl-Kontrollstruktur **if**. Anstatt den Hauptteil des tatsächlich ausgeführten Zweigs zu bewerten, gibt sie den durch "subst" ersetzten *Hauptteil* zurück. Hinweis: Die Syntax ist etwas restriktiver als die der integrierten Tcl-Kontrollstruktur **if** .

::html::init ?*list*?
::html::init akzeptiert eine Name-Wert-Liste im Tcl-Stil, die Werte für Elemente mit einem Namen der Form "tag.parameter" definiert. Ein Standardwert mit dem Schlüssel "body.bgcolor" definiert beispielsweise die Hintergrundfarbe für das *body*-Tag.

::html::keywords *args*
Nur als Nebeneffekt. Rufen Sie diese Funktion vor **::html::head** auf, um einen *Meta*-Tag mit Schlüsselwörtern für die Seite zu definieren. Der *Meta*-Tag wird in das Ergebnis von **::html::head** aufgenommen.

::html::mailto *email* ?*subject*?
Erstellt einen Hypertext-Link zu einer "mailto:"-URL.

::html::meta *args*
Name der Kompatibilität für **html::meta_name**.

::html::meta_name *args*
Nur als Nebeneffekt. Rufen Sie diese Funktion vor **::html::head** auf, um ein *meta*-Tag für die Seite zu definieren. Die Argumente (*args*) sind eine Liste aus Namen und Werten im Tcl-Stil,

die für die Attribute **name=** und **content=** des *meta*-Tags verwendet wird. Das *meta*-Tag wird in das Ergebnis von **::html::head** eingebunden.

::html::meta_equiv *args*
Nur als Nebeneffekt. Rufen Sie diese Funktion vor **::html::head** auf, um ein *meta*-Tag für die Seite zu definieren. Die Argumente (*args*) sind eine Liste aus Namen und Werten im Tcl-Stil, die für die Attribute **http-equiv=** und **content=** des *meta*-Tags verwendet wird. Das *meta* -Tag wird in das Ergebnis von **::html::head** aufgenommen.

::html::meta_charset *charset*
Nur als Nebeneffekt. Rufen Sie dies vor **::html::head** auf, um ein *meta*-Tag für die Seite zu definieren. Der *charset* wird mit dem Attribut **charset=** des *meta*-Tags verwendet. Der *meta* -Tag ist im Ergebnis von **::html::head** enthalten.

::html::css *href*
Nur als Nebeneffekt. Rufen Sie diesen Befehl vor **::html::head** auf, um ein *link*-Tag für ein verknüpftes CSS-Dokument zu definieren. Der Wert von *href* ist eine HTTP-URL zu einem CSS-Dokument. Das *link*-Tag wird in das Ergebnis von **::html::head** aufgenommen.

Mehrere Aufrufe dieses Befehls sind zulässig, wodurch die Verwendung mehrerer Verweise auf CSS-Dokumente ermöglicht wird. Mit anderen Worten: Die Argumente mehrerer Aufrufe werden kumuliert und überschreiben sich nicht gegenseitig.

::html::css-clear
Nur als Nebenwirkung. Rufen Sie diesen Befehl vor **::html::head** auf, um alle Verweise auf CSS-Dokumente zu löschen.

Mehrere Aufrufe dieses Befehls sind zulässig; nach dem ersten Aufruf in einer Folge, in der kein **::html::css** dazwischenliegt, hat dies jedoch keine weitere Auswirkung.

::html::js *href*
Nur als Nebenwirkung. Rufen Sie diesen Befehl vor **::html::head** auf, um ein *Skript*-Tag für ein verknüpftes JavaScript-Dokument zu definieren. Das *href* ist eine HTTP-URL zu einem JavaScript-Dokument. Das *script*-Tag wird in das Ergebnis von **::html::head** aufgenommen.

Mehrere Aufrufe dieses Befehls sind zulässig, wodurch die Verwendung mehrerer Verweise auf JavaScript-Dokumente ermöglicht wird. Mit anderen Worten: Die Argumente mehrerer Aufrufe werden kumuliert und überschreiben sich nicht gegenseitig.

::html::js-clear
Nur als Nebenwirkung. Rufen Sie diesen Befehl vor **::html::head** auf, um alle Verweise auf JavaScript-Dokumente zu löschen.

Mehrere Aufrufe dieses Befehls sind zulässig; nach dem ersten Aufruf in einer Folge, in der kein **::html::js** dazwischenliegt, hat der Befehl jedoch keine weitere Wirkung.

::html::minorList *list* ?*ordered*?
Erzeugt eine nummerierte oder unnummerierte Liste von Links. Der Name der *list* ist eine Liste im Tcl-Stil, bestehend aus Bezeichnungen und URLs für die Links. *ordered* ist ein boolescher Wert, mit dem zwischen einer nummerierten und einer unnummerierten Liste gewählt werden kann. Der Standardwert ist **false**.

::html::minorMenu *list* ?*sep*?

Erzeugt eine Reihe von Hypertext-Links. Die *list* ist eine Liste im Tcl-Stil, bestehend aus Namen und URLs für die Links. *sep* ist der Text, der zwischen den einzelnen Links eingefügt werden soll. Der Standardwert ist " | ".

::html::nl2br *string*
Dieser Befehl ersetzt alle Zeilenenden in der *Zeichenkette* durch ein *br*-Tag und gibt den geänderten Text zurück.

::html::openTag *tag* ?*param*?
Legt *tag* auf einen Stapel und erzeugt das Eröffnungstag für *tag*. Verwenden Sie **::html::closeTag**, um das Tag vom Stapel zu entfernen. Das zweite Argument liefert alle Tag-Argumente als Liste, deren Elemente im Format "**key**=**value**" formatiert sind.

::html::paramRow *list* ?*rparam*? ?*cparam*?
Erzeugt eine Tabellenzeile einschließlich der Tags *tr* und *td*. Jeder Wert in *list* wird in eine eigene Tabellenzelle eingefügt. Dabei wird **::html::cell** verwendet. Der Wert von *rparam* wird als Parameter für das *tr*-Tag verwendet. Der Wert von *cparam* wird als Parameter für die *td* -Tags an **::html::cell** übergeben.

::html::passwordInput ?*name*?
Erstellt ein *input*-Tag vom Typ *password*. Der *Name* lautet standardmäßig "password".

::html::passwordInputRow *label* ?*name*?
Formatiert eine Zeile der Tabelle, die eine Beschriftung und ein *input*-Tag vom Typ *password* enthält. Der *Name* lautet standardmäßig "password".

::html::quoteFormValue *value*
Setzt Sonderzeichen im Wert in Anführungszeichen, indem er sie durch HTML-Entitäten für Anführungszeichen, das Und-Zeichen und spitze Klammern ersetzt.

::html::radioSet *key sep list*
Erzeugt eine Reihe von *input*-Tags vom Typ *radio* und eine zugehörige Textbeschriftung. Alle Optionsfelder haben denselben *key* als Name. Der *sep* ist der Text, der zur Trennung der Elemente verwendet wird. Die *list* ist eine Liste aus Beschriftung und Wert im Tcl-Stil.

::html::radioValue *name value*
Erzeugt das Format "name=*name* value=*value*" für ein *Radio*-Formularelement. Wenn die CGI-Variable *name* den Wert *value* hat, wird dem Rückgabewert "SELECTED" hinzugefügt.

::html::refresh *seconds url*
Richtet einen *Meta*-Tag zum Aktualisieren ein. Rufen Sie diesen vor **::html::head** auf, und der HEAD-Abschnitt enthält einen *Meta*-Tag, der bewirkt, dass das Dokument in *seconds* Sekunden aktualisiert wird. Die *URL* ist optional. Wenn sie angegeben wird, legt sie eine neue Seite fest, die nach Ablauf des Aktualisierungsintervalls geladen werden soll.

::html::row *args*
Erzeugt eine Tabellenzeile einschließlich der Tags *tr* und *td*. Jeder Wert in *args* wird in eine eigene Tabellenzelle eingefügt. Dabei wird **::html::cell** verwendet. Über **::html::init** festgelegte Standardinformationen werden ignoriert.

::html::select *name param choices* ?*current*?
Erzeugt ein *select*-Formularelement und verschachtelte *option*-Tags. Die Attribute *name* und *param* werden zur Erzeugung des *select*-Tags verwendet. Die Liste *choices* ist eine Liste aus Namen und Werten im Tcl-Stil.

::html::selectPlain *name param choices* ?*current*?
Ähnlich wie **::html::select**, mit dem Unterschied, dass *choices* eine Tcl-Liste mit Werten ist,

die für die *Option*-Tags verwendet werden. Die Bezeichnung und der Wert für jede *Option* sind identisch.

::html::set *var val*
Diese Prozedur ähnelt dem integrierten Tcl-Befehl **set**. Der Hauptunterschied besteht darin, dass sie "" gibt, sodass sie aus einer HTML-Vorlagendatei aufgerufen werden kann, ohne dass unerwünschte Ergebnisse angehängt werden. Ein weiterer Unterschied ist, dass sie zwei Argumente benötigt.

::html::submit *label* ?*name*? ?*title*?
Erzeugt ein *input*-Tag vom Typ *submit*. Der Standardwert für *name* lautet "submit". Wenn eine nicht leere *title*-Zeichenkette festgelegt wird, erhält die Schaltfläche ein **title=**-Attribut mit diesem Wert.

::html::tableFromArray *arrname* ?*param*? ?*pat*?
Erzeugt eine zweispaltige *Tabelle* mit verschachtelten Zeilen zur Darstellung eines Tcl-Arrays. Die Tabelle erhält eine Überschrift, die dem Namen des Arrays entspricht, und jede generierte Zeile enthält ein Name-Wert-Paar. Die Array-Namen werden sortiert (**lsort** ohne spezielle Optionen). Das Argument *param* ist für das *table*-Tag vorgesehen und muss eine vorformatierte Zeichenkette enthalten. *pat* ist ein **string match**-Muster, mit dem die Array-Elemente ausgewählt werden, die in der Tabelle angezeigt werden sollen. Der Standardwert ist *, d. h., das gesamte Array wird angezeigt.

::html::tableFromList *querylist* ?*param*?
Erzeugt eine zweispaltige *Tabelle* mit verschachtelten Zeilen, um *querylist* anzuzeigen, bei dem es sich um ein Tcl-Dictionary handelt. Jede generierte Zeile enthält ein Name-Wert-Paar. Die Informationen werden in derselben Reihenfolge angezeigt, wie sie im Dictionary angegeben sind. Das Argument *param* ist für das *table*-Tag bestimmt und muss eine vorformatierte Zeichenkette enthalten.

::html::textarea *name* ?*param*? ?*current*?
Erzeugt ein *textarea*-Tag, das die aktuellen Werte umschließt.

::html::textInput *name value args*
Erzeugt ein *input*-Formular-Tag vom Typ *text*. Dabei wird **::html::formValue** verwendet. "args" bezeichnet alle zusätzlichen Tag-Attribute, die Sie in das *input*-Tag einfügen möchten.

::html::textInputRow *label name value args*
Erzeugt ein *input*-Formular-Tag vom Typ *text*, das als Zeile einer Tabelle mit einer zugehörigen Beschriftung formatiert ist. Bei "args" handelt es sich um beliebige zusätzliche Tag-Attribute, die Sie in das *input*-Tag einfügen möchten.

::html::varEmpty *name*
Diese Funktion gibt 1 zurück, wenn die angegebene Variable entweder nicht existiert oder als Wert die leere Zeichenkette hat.

::html::while *test body*
Diese Prozedur ähnelt der integrierten Tcl-Kontrollstruktur **while**. Anstatt die Auswertung des Körpers durchzuführen, gibt sie den durch "subst" ersetzten *Körper* zurück. Bei jeder Iteration der Schleife wird eine weitere Zeichenkette an den Wert angehängt.

::html::doctype *id*
Mit dieser Prozedur lässt sich die Standard-DOCTYPE-Deklarationszeichenfolge erstellen. Sie gibt die Standard-Deklarationszeichenfolge für die jeweilige ID zurück oder löst einen Fehler aus, wenn die ID unbekannt ist. Folgende IDs sind definiert:

1. HTML32
2. HTML40
3. HTML40T
4. HTML40F
5. HTML401
6. HTML401T
7. HTML401F
8. XHTML10S
9. XHTML10T
10. XHTML10F
11. XHTML11
12. XHTMLB

::html::wrapTag ***tag*** **?*text*? ?*args*?**
Eine Hilfsfunktion, um einen *Text* in ein Paar aus öffnendem und schließendem *Tag* einzufassen. Die Argumente (*args*) sind eine Liste aus Namen und Werten im Tcl-Stil, die dazu dient, dem öffnenden Tag Attribute und zugehörige Werte zuzuweisen. Das Ergebnis ist eine Zeichenkette, die aus dem öffnenden *Tag* zusammen mit den optionalen Attributen, dem optionalen Text und dem schließenden Tag besteht.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *html* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

CGI-Programmierung

htmlparse

htmlparse

Stichwörter: htmlparse, html, parsing, queue, tree

Name

htmlparse – Prozeduren zum Parsen von HTML-Zeichenketten

Übersicht

package require Tcl 8.5 9
package require struct::stack 1.3
package require cmdline 1.1
package require htmlparse ?**1.2.3**?
::htmlparse::parse ?*-cmd cmd*? ?*-vroot tag*? ?*-split n*? ?*-incvar var*? ?*-queue q*? *html*

::htmlparse::debugCallback ?*clientdata*? *tag slash param textBehindTheTag*
::htmlparse::mapEscapes *html*
::htmlparse::2tree *html tree*
::htmlparse::removeVisualFluff *tree*
::htmlparse::removeFormDefs *tree*

Beschreibung

Das **htmlparse**-Paket stellt Befehle bereit, mit denen Bibliotheken und Anwendungen HTML-Inhalte in einer Zeichenkette in eine Darstellung ihrer Wahl zerlegen können.

Folgende Befehle stehen zur Verfügung:

::htmlparse::parse ?-cmd *cmd*? ?-vroot *tag*? ?-split *n*? ?-incvar *var*? ?-queue *q*? *html*

Dieser Befehl ist der grundlegende Parser für HTML. Er nimmt eine HTML-Zeichenkette entgegen, analysiert sie und ruft für jedes gefundene Tag ein Befehlspräfix auf. Der HTML-Code muss nicht gültig sein, damit dieser Parser funktioniert. Es liegt in der Verantwortung des für jedes Tag aufgerufenen Befehls, dies zu überprüfen. Eine weitere Aufgabe des aufgerufenen Befehls ist die Verarbeitung von Tag-Attributen und Zeichenentitäten (Escape-Zeichen). Der Parser stellt dem aufgerufenen Befehl die nicht interpretierten Tag-Attribute zur Verfügung, um bei Ersterem zu helfen, und das Paket als Ganzes bietet einen Hilfsbefehl, **::htmlparse::mapEscapes**, zur Unterstützung bei der Behandlung von Letzterem. Der Parser ignoriert *tatsächlich* führende DOCTYPE-Deklarationen und alle gültigen HTML-Kommentare, auf die er stößt.

Alle Informationen, die über die HTML-Zeichenkette selbst hinausgehen, werden über Optionen festgelegt; diese werden im Folgenden erläutert.

Zum besseren Verständnis der Optionen folgen hier einige weitere Hintergrundinformationen zum Parser.

Er ist in der Lage, unvollständige Tags in der ihm übergebenen HTML-Zeichenkette zu erkennen. Unter normalen Umständen führt dies dazu, dass der Parser einen Fehler auslöst; wenn jedoch die Option *-incvar* verwendet wird, um eine globale (oder Namespace-)Variable festzulegen, speichert der Parser den unvollständigen Teil der Eingabe stattdessen in dieser Variablen. Dies ist eine große Hilfe bei der Verarbeitung von schrittweise eintreffendem HTML, da der Parser alles verarbeitet, was er kann, und die Verarbeitung des unvollständigen Teils aufschiebt, bis weitere Daten eingetroffen sind.

Ein weiteres Merkmal des Parsers sind seine zwei möglichen Betriebsmodi. Der Normalmodus wird aktiviert, wenn die Option *-queue* in der Befehlszeile, mit der der Parser aufgerufen wird, nicht vorhanden ist. Ist sie vorhanden, wechselt der Parser stattdessen in den inkrementellen Modus.

Der Hauptunterschied besteht darin, dass ein Parser im Normalmodus für jedes Tag, auf das er stößt, sofort das Befehlspräfix aufruft. Im inkrementellen Modus hingegen generiert der Parser eine Reihe von Skripten, die das Befehlspräfix für Gruppen von Tags in der HTML-Zeichenkette aufrufen, und speichert diese Skripte anschließend in der angegebenen Warteschlange. Es liegt dann in der Verantwortung des Aufrufers des Parsers, die Ausführung der Skripte in der Warteschlange sicherzustellen.

Hinweis: Das dem Parser übergebene Warteschlangen-Objekt muss dieselbe Schnittstelle

bereitstellen wie die in tcllib -> struct definierte Warteschlange. Das bedeutet beispielsweise, dass alle über dieses tcllib-Modul erstellten Warteschlangen hier sofort verwendet werden können. Die Warteschlange muss jedoch nicht unbedingt aus tcllib -> struct stammen, solange dieselbe Schnittstelle bereitgestellt wird.

In beiden Modi gibt der Parser eine leere Zeichenkette an den Aufrufer zurück.

Die Option *-split* kann einem Parser im inkrementellen Modus übergeben werden, um die Größe der von ihm erstellten Gruppen festzulegen. Mit anderen Worten: -split 5 bedeutet, dass jedes der generierten Skripte den Befehls-Präfix-Befehl für 5 aufeinanderfolgende Tags in der HTML-Zeichenkette aufruft. Ein Parser im Normalmodus ignoriert diese Option und ihren Wert.

Die Option *-vroot* legt ein virtuelles Stamm-Tag fest. Ein Parser im Normalmodus ruft den Befehl dafür unmittelbar vor und nach der Verarbeitung der TAGs im HTML-Text auf und simuliert so, dass die HTML-Zeichenkette von einer Kombination aus und umschlossen ist. Im inkrementellen Modus ist der Parser jedoch nicht in der Lage, das schließende virtuelle Stamm-Tag bereitzustellen, da er nie weiß, wann die Eingabe vollständig ist. In diesem Fall enthält das erste Skript, das bei jedem Aufruf des Parsers generiert wird, als ersten Befehl einen Aufruf des Befehlspräfixes für das virtuelle Stamm-Tag. Die folgenden Optionen stehen zur Verfügung:

-cmd *cmd*
: Der Befehl, der für jedes Tag in der HTML-Zeichenkette aufgerufen werden soll. Der Standardwert lautet *::htmlparse::debugCallback*.

-vroot *tag*
: Das virtuelle Root-Tag, das im Normalmodus um den HTML-Code herum eingefügt wird. Im inkrementellen Modus ist es das erste Tag in jedem vom Parser verarbeiteten Block, es gibt jedoch keine schließenden Tags. Der Standardwert ist *hmstart*.

-split *n*
: Die Größe der Gruppen, die von einem Parser im inkrementellen Modus erzeugt werden. Wird im normalen Modus ignoriert. Der Standardwert ist 10. Werte <= 0 sind nicht zulässig.

-incvar *var*
: Der Name der Variablen, in der unvollständiger HTML-Code gespeichert werden soll. Dies ist vor allem im inkrementellen Modus sinnvoll. Der Parser löst einen Fehler aus, wenn er unvollständigen HTML-Code erkennt und keinen Speicherort dafür hat. Dies ist im normalen Modus sinnvoll. Es werden nur unvollständige Tags erkannt, nicht jedoch fehlende Tags. Optional, Standardwert ist "keine Variable".

Schnittstelle zum Befehlspräfix
: Im Normalmodus ruft der Parser das Befehlspräfix auf, dem vier Argumente angehängt werden. Eine Beschreibung finden Sie unter **::htmlparse::debugCallback**.

  Im inkrementellen Modus rufen die generierten Skripte das Befehlspräfix jedoch mit fünf angehängten Argumenten auf. Die letzten vier davon sind dieselben wie oben erwähnt. Das erste ist eine Platzhalterzeichenfolge (**@win@**) für einen "clientdata"-Wert, der später während der eigentlichen Ausführung der generierten Skripte bereitgestellt wird. Dies könnte beispielsweise ein tk-Pfad für ein Fenster sein. Dies ermöglicht es dem Nutzer

dieses Pakets, HTML-Zeichenfolgen vorzuverarbeiten, ohne sie während der Analyse einer bestimmten Verbindung zuzuordnen. Diese Verbindung kann später erfolgen. Das bedeutet auch, dass es möglich ist, vorverarbeitetes HTML zwischenzuspeichern. Natürlich hindert nichts den Nutzer des Parsers daran, den Platzhalter durch eine leere Zeichenfolge zu ersetzen.

::htmlparse::debugCallback ?*clientdata*? *tag slash param textBehindTheTag*

Dieser Befehl ist die Standard-Callback-Funktion, die der Parser in **::htmlparse::parse** verwendet, wenn vom Benutzer kein eigener angegeben wurde. Er gibt lediglich seine Argumente an stdout aus. Diese Callback-Funktion kann sowohl im normalen als auch im inkrementellen Modus des aufrufenden Parsers verwendet werden. Mit anderen Worten: Er akzeptiert vier oder fünf Argumente. Die letzten vier Argumente werden im Folgenden beschrieben. Das optionale fünfte Argument enthält den Wert "clientdata", der vom Parser im inkrementellen Modus an die Rückruffunktion übergeben wird. Alle Rückruffunktionen müssen bei den letzten vier Argumenten der Signatur dieses Befehls entsprechen, und Rückruffunktionen, die beim inkrementellen Parsen verwendet werden, müssen bei den letzten fünf Argumenten dieser Signatur entsprechen.

Das erste Argument, *clientdata*, ist optional und nur vorhanden, wenn dieser Befehl von einem Parser im inkrementellen Modus aufgerufen wird. Es enthält alles, was der Benutzer dieses Pakets wünscht.

Das zweite Argument, *tag*, enthält den Namen des Tags, das gerade vom Parser verarbeitet wird.

Das dritte Argument, *slash*, ist entweder leer oder enthält ein Schrägstrichzeichen. Es ermöglicht der Rückruffunktion, zwischen öffnenden (slash ist leer) und schließenden Tags (slash enthält ein Schrägstrichzeichen) zu unterscheiden.

Das vierte Argument, *param*, enthält die nicht interpretierte Liste der Parameter für das Tag.

Das fünfte und letzte Argument, *textBehindTheTag*, enthält den Text, den der Parser hinter dem in *tag* genannten Tag gefunden hat.

::htmlparse::mapEscapes *html*

Dieser Befehl nimmt eine HTML-Zeichenkette entgegen, ersetzt alle Escape-Sequenzen durch ihre tatsächlichen Zeichen und gibt anschließend die resultierende Zeichenkette zurück. HTML-Zeichenketten, die keine Escape-Sequenzen enthalten, werden unverändert zurückgegeben.

::htmlparse::2tree *html tree*

Dieser Befehl ist ein Wrapper um **::htmlparse::parse**, der eine HTML-Zeichenkette (in *html*) entgegennimmt und diese in einen Baum umwandelt, der die logische Struktur des geparsten Dokuments enthält. Der Name des Baums wird dem Befehl als zweites Argument (*tree*) übergeben. Der Befehl generiert den Baum **nicht** selbst, sondern erwartet, dass der Aufrufer ihm einen bereits vorhandenen und leeren Baum übergibt. Er erwartet außerdem, dass das angegebene Baumobjekt dieselbe Schnittstelle wie das Baumobjekt in tcllib -> struct aufweist. Es muss nicht unbedingt aus tcllib -> struct stammen, muss aber dieselbe

Schnittstelle bereitstellen.

Die interne Rückruffunktion führt einige grundlegende Prüfungen der HTML-Gültigkeit durch und versucht, die grundlegendsten Fehler zu beheben. Der Befehl gibt den Inhalt seines zweiten Arguments zurück. Nebenwirkungen sind die Erstellung und Bearbeitung eines Baumobjekts.

Jeder Knoten im generierten Baum repräsentiert ein Tag in der Eingabe. Der Name des Tags wird im Attribut *type* des Knotens gespeichert. Alle mit dem Tag mitgelieferten HTML-Attribute werden unverändert im Attribut *data* des Tags gespeichert. Mit anderen Worten: Der Befehl analysiert HTML-Attribute *nicht* in ihre Namen und Werte.

Wenn ein Tag Text enthält, verfügt sein Knoten über untergeordnete Knoten vom Typ *PCDATA*, die diesen Text enthalten. Der Text wird im Attribut *data* dieser untergeordneten Knoten gespeichert.

::htmlparse::removeVisualFluff *tree*
Dieser Befehl durchläuft einen von **::htmlparse::2tree** generierten Baum und entfernt alle Knoten, die visuelle und keine strukturellen Tags darstellen. Der Zweck des Befehls besteht darin, die Navigation im Baum zu vereinfachen, ohne sich in visuellen Informationen zu verlieren, die für die Suche irrelevant sind. Sein einziges Argument ist der Name des Baums, der bereinigt werden soll.

::htmlparse::removeFormDefs *tree*
Genau wie **::htmlparse::removeVisualFluff** dient dieser Befehl dazu, die Größe des von **::htmlparse::2tree** generierten Baums zu reduzieren. Er entfernt alle Knoten, die Formulare und Formularelemente darstellen. Sein einziges Argument ist der Name des Baums, der verkleinert werden soll.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *htmlparse* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

http

autoproxy

Stichwörter: autoproxy, authentication, http, proxy

Name

autoproxy – Automatische Nutzung und Authentifizierung eines HTTP-Proxys

Übersicht

package require Tcl 8.5 9
package require http ?**2.0**?
package require autoproxy ?**1.8.1**?
::autoproxy::init
::autoproxy::cget *-option*
::autoproxy::configure ?*-option value*?
::autoproxy::tls_connect *args*
::autoproxy::tunnel_connect *args*
::autoproxy::tls_socket *args*

Beschreibung

Dieses Paket dient dazu, die Verwendung von HTTP-Proxy-Servern im Tcl-HTTP-Client-Code zu automatisieren. Es versucht, die Einstellungen für den Webzugriff aus den Standardverzeichnissen des Systems zu initialisieren, und kann bei Bedarf so konfiguriert werden, dass es die Authentifizierung mit dem Proxy aushandelt.

Unter Unix scheint es Standard zu sein, den lokalen HTTP-Proxy-Server über die Umgebungsvariablen http_proxy oder ftp_proxy zu identifizieren und die Domänen, die vom Proxy-Einsatz ausgeschlossen werden sollen, über no_proxy aufzulisten. Unter Windows können wir die Werte der Interneteinstellungen aus der Registrierung abrufen, um nahezu dieselben Informationen zu erhalten. Mit diesen Informationen können wir eine geeignete Filter-Prozedur für das Tcl-HTTP-Paket einrichten und die automatische Nutzung des Proxys veranlassen.

Es scheint eine Reihe von Möglichkeiten zu geben, wie die Variable http_proxy eingerichtet werden kann. Entweder als einfache Angabe von Host und Port oder – was häufiger der Fall ist – als URL; manchmal enthält die URL Authentifizierungsparameter, oder diese werden vom Benutzer abgefragt oder über http_proxy_user und http_proxy_pass bereitgestellt. Dieses Paket versucht, all diese Schemata zu unterstützen. Es versucht nach Kräften, die erforderlichen Parameter aus der Umgebung oder der Registrierung zu beziehen; sollte dies fehlschlagen, kann es neu konfiguriert werden.

TLS-Sicherheitshinweise

Hinweis Dieser Abschnitt gilt nur, wenn die TLS-Unterstützung durch das **TLS**-Paket bereitgestellt wird. Er gilt nicht, wenn **autoproxy** über die Konfigurationsoption **-tls_package** so konfiguriert wurde, dass ein anderes Paket verwendet wird, das dieselbe Funktion bereitstellen kann (z. B. **twapi**).

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele

Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Befehle

::autoproxy::init
Initialisiert das Autoproxy-Paket anhand der Systemressourcen. Unter Unix bedeutet dies, dass wir nach Umgebungsvariablen suchen. Unter Windows suchen wir nach denselben Umgebungsvariablen, prüfen aber zusätzlich die von Internet Explorer verwendeten Registrierungseinstellungen.

::autoproxy::cget *-option*
Einzelne Konfigurationsoptionen des Pakets abrufen. Siehe Optionen.

::autoproxy::configure ?-option *value*?
Konfiguriert das Autoproxy-Paket. Wenn Sie **configure** ohne Optionen aufrufen, gibt es eine Liste aller Optionsnamen und Werte zurück. Siehe Optionen.

::autoproxy::tls_connect *args*
Stellt über einen Proxy eine Verbindung zu einem sicheren Socket her. HTTP-Proxy-Server ermöglichen die Verwendung des HTTP-Befehls CONNECT, um über den Proxy eine Verbindung zum Zielrechner herzustellen. Diese Funktion verbirgt die Details. Zur Verwendung mit dem http-Paket siehe **tls_socket**.

Die *args*-Liste kann beliebige Optionen enthalten, die von dem jeweils verwendeten TLS-Paket unterstützt werden, muss jedoch mit dem Host und dem Port als letzten beiden Elementen enden.

::autoproxy::tunnel_connect *args*
Stellt über einen Proxy eine Verbindung zu einem Zielhost her. Dabei wird derselbe HTTP-Befehl "CONNECT" wie bei **tls_connect** verwendet, jedoch wird die Verbindung nach ihrem Aufbau nicht weiter gesichert.

Die *args*-Liste kann beliebige Optionen enthalten, die von dem jeweils verwendeten TLS-Paket unterstützt werden, muss jedoch mit dem Host und dem Port als letzten beiden Elementen enden.

Beachten Sie, dass viele Proxy-Server CONNECT-Aufrufe nur auf eine begrenzte Anzahl von Ports zulassen – in der Regel nur auf Port 443 (den sicheren HTTP-Port).

::autoproxy::tls_socket *args*
Diese Funktion dient dazu, einen proxy-fähigen Secure-Socket-Handler für das HTTPS-Protokoll zu registrieren. Sie darf nur in Verbindung mit dem Tcl-HTTP-Paket verwendet werden und sollte mit dem Befehl http::register registriert werden (siehe die folgenden Beispiele). Der

eigentliche Auftrag zur Herstellung der getunnelten Verbindung erfolgt über den Befehl tls_connect, der verwendet werden kann, wenn keine Registrierung über das HTTP-Paket erfolgt.

Optionen

-host hostname
siehe unten ...

-proxy_host hostname
Legt den Proxy-Hostnamen fest. Dieser wird normalerweise von **init** eingerichtet, kann aber auch hier konfiguriert werden.

-port number
siehe unten ...

-proxy_port number
Legt die Proxy-Portnummer fest. Diese wird normalerweise von **init** eingerichtet. Beispiel: **configure -port** *3128*

-no_proxy list
Sie können die von **init** eingerichtete **no_proxy**-Liste bearbeiten. Der Wert dieser Option ist eine Tcl-Liste von Zeichenketten, die mithilfe des Tcl-Befehls **string match** mit dem Host der HTTP-Anfrage abgeglichen werden. Daher sind Glob-Muster zulässig. Beispielsweise **configure -no_proxy** **.localdomain*

-authProc procedure
Mit dieser Option kann eine anwendungsdefinierte Prozedur festgelegt werden, die aufgerufen wird, wenn **configure -basic** ohne oder mit unzureichenden Authentifizierungsdaten aufgerufen wird. Dies kann genutzt werden, um dem Benutzer ein Dialogfeld anzuzeigen, in dem er zur Eingabe der zusätzlichen Informationen aufgefordert wird.

-basic
Die folgenden Optionen dienen zur Konfiguration der Parameter für das Basic-Authentifizierungsschema. Siehe Basisauthentifizierung. Um die aus einem früheren Aufruf dieser Funktion beibehaltenen Proxy-Authentifizierungsdaten zurückzusetzen, können entweder "--" oder keine zusätzlichen Parameter angegeben werden. Dadurch werden die vorhandenen Authentifizierungsdaten entfernt.

-tls_package packagename
Mit dieser Option kann das Tcl-Paket für die TLS-Unterstützung konfiguriert werden. Gültige Paketnamen sind **tls** (Standard) und **twapi**.

Basisauthentifizierung

"Basic" ist das einfachste und am häufigsten verwendete Authentifizierungsverfahren für HTTP-Proxys. Es wird in (1, Abschnitt 11) sowie in (2) beschrieben. Es bietet keinerlei Datenschutz und von seiner Verwendung sollte zugunsten sichererer Alternativen wie "Digest" abgeraten werden. Zur Durchführung der Basic-Authentifizierung kodiert der Client den Benutzernamen und das Passwort im Klartext, getrennt durch einen Doppelpunkt, mit der Base64-Kodierung. Diesem kodierten Text werden das Wort "Basic" und ein Leerzeichen vorangestellt.

Für dieses Schema stehen folgende Optionen zur Verfügung:

-username name

Der Benutzername, der für die Authentifizierung beim konfigurierten Proxy erforderlich ist.

-password password
Das für den angegebenen Benutzernamen erforderliche Passwort.

-realm realm
Diese Option wird von diesem Paket nicht verwendet, kann jedoch dazu dienen, Authentifizierungsdaten vom Benutzer abzufragen.

--
Der Indikator "End-of-Options" kann allein verwendet werden, um alle derzeit aktivierten Authentifizierungsdetails zu deaktivieren.

Beispiele

```
package require autoproxy
autoproxy::init
autoproxy::configure -basic -username ME -password SEKRET
set tok [http::geturl http://wiki.tcl.tk/]
http::data $tok
```

```
package require http
package require tls
package require autoproxy
autoproxy::init
http::register https 443 autoproxy::tls_socket
set tok [http::geturl https://www.example.com/]
```

Literatur

1. Berners-Lee, T., Fielding, R. und Frystyk, H. "Hypertext Transfer Protocol – HTTP/1.0", RFC 1945, Mai 1996, (http://www.rfc-editor.org/rfc/rfc1945.txt)
2. Franks, J. et al. "HTTP-Authentifizierung: Basic- und Digest-Zugriffsauthentifizierung", RFC 2617, Juni 1999 (http://www.rfc-editor.org/rfc/rfc2617.txt)

Bugs

Derzeit wird nur die Basisauthentifizierung (1) (2) unterstützt. Es ist geplant, in Zukunft auch die Unterstützung für Digest (2) und NTLM hinzuzufügen.

Autoren

Pat Thoyts

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *http :: autoproxy* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

httpd

httpd
Stichwörter: httpd, TclOO, WWW, http, httpserver, services

Name
httpd – Ein auf TclOO und Coroutinen basierender Webserver

Übersicht
package require Tcl 8.6 9
package require uuid
package require clay
package require coroutine
package require fileutil
package require fileutil::magic::filetype
package require websocket
package require mime
package require cron
package require uri
package require Markdown

method **ChannelCopy** *in out* ?*args*?

method **html_header** ?*title* ? ?*args*?

method **html_footer** ?*args*?

method **http_code_string** *code*

method **HttpHeaders** *sock* ?*debug* ?

method **HttpHeaders_Default**

method **HttpServerHeaders**

method **MimeParse** *mimetext*

method **Url_Decode** *data*

method **Url_PathCheck** *urlsuffix*

method **wait** *mode sock*

variable **ChannelRegister**

variable **reply**

variable **request**

delegate

method **constructor** *ServerObj* ?*args*?

method **destructor** ?*dictargs*?

method **ChannelRegister** ?*args*?

method **close**
method **Log_Dispatched**
method **dispatch** *newsock datastate*
method **Dispatch**
method **html_header** *title* ?*args*?
method **html_footer** ?*args*?
method **error** *code* ?*msg* ? ?*errorInfo* ?
method **content**
method **EncodeStatus** *status*
method **log** *type* ?*info* ?
method **CoroName**
method **DoOutput**
method **FormData**
method **PostData** *length*
method **Session_Load**
method **puts** *line*
method **RequestFind** *field*
method **request** *subcommand* ?*args*?
method **reply** *subcommand* ?*args*?
method **reset**
method **timeOutCheck**
method **timestamp**
variable **template**
variable **url_patterns**
method **constructor** *args* ?*port* **auto**? ?*myaddr* **127.0.0.1**? ?*string* **auto**? ?*name* **auto**? ?*doc_root* ? ?*reverse_dns* **0**? ?*configuration_file* ? ?*protocol* **HTTP/1.1**?
method **destructor** ?*dictargs*?
method **connect** *sock ip port*
method **ServerHeaders** *ip http_request mimetxt*
method **Connect** *uuid sock ip*
method **counter** *which*
method **CheckTimeout**
method **debug** ?*args*?
method **dispatch** *data*
method **Dispatch_Default** *reply*
method **Dispatch_Local** *data*
method **Headers_Local** *varname*

method **Headers_Process** *varname*
method **HostName** *ipaddr*
method **log** ?*args*?
method **plugin** *slot* ?*class* ?
method **port_listening**
method **PrefixNormalize** *prefix*
method **source** *filename*
method **start**
method **stop**
method **SubObject {} db**
method **SubObject {} default**
method **template** *page*
method **TemplateSearch** *page*
method **Thread_start**
method **Uuid_Generate**
method **Validate_Connection** *sock ip*
method **reset**
method **content**
method **Dispatch**
method **content**
method **FileName**
method **DirectoryListing** *local_file*
method **content**
method **Dispatch**
variable **exename**
method **CgiExec** *execname script arglist*
method **Cgi_Executable** *script*
method **proxy_channel**
method **proxy_path**
method **ProxyRequest** *chana chanb*
method **ProxyReply** *chana chanb* ?*args*?
method **Dispatch**
method **FileName**
method **proxy_channel**
method **ProxyRequest** *chana chanb*
method **ProxyReply** *chana chanb* ?*args*?
method **DirectoryListing** *local_file*

method **EncodeStatus** *status*

method **scgi_info**

method **proxy_channel**

method **ProxyRequest** *chana chanb*

method **ProxyReply** *chana chanb* ?*args*?

method **debug** ?*args*?

method **Connect** *uuid sock ip*

method **Dispatch_Dict** *data*

method **uri {} add** *vhosts patterns info*

method **uri {} direct** *vhosts patterns info body*

method **output**

method **DoOutput**

method **close**

method **local_memchan** *command* ?*args*?

method **Connect_Local** *uuid sock* ?*args*?

Beschreibung

Dieses Modul implementiert einen Webserver, der sich zur Einbindung in eine Anwendung eignet. Der Server ist objektorientiert und enthält alle Grundlagen, die für eine voll funktionsfähige Website erforderlich sind.

Minimalbeispiel

Zum Starten eines Webdienstes wird eine Klasse vom Typ **httpd::server** instanziiert und diesem Server werden eine oder mehrere zu bedienende URIs sowie von **httpd::reply** abgeleitete Klassen zur Generierung dieser URIs bereitgestellt.

```
package require httpd

oo::class create ::reply.hello {
  method content {} {
    my puts "<HTML><HEAD><TITLE>IRM Dispatch
    Server</TITLE></HEAD><BODY>"
    my puts "<h1>Hello World!</h1>"
    my puts </BODY></HTML>
  }
}
::httpd::server create HTTPD port 8015 myaddr 127.0.0.1 doc_root [file
    home]/htdocs
HTTPD plugin dispatch ::httpd::plugin.dict_dispatch
HTTPD uri add * /hello [list mixin reply.hello]
vwait forever
```

Das Grundmodul kann Dateien aus einem Dateisystem ausliefern. Dateien, deren Name auf **.tml** endet, werden im Stil von Tclhttpd substituiert:

```
<!-- hello.tml -->
[my html_header {Hello World!}]
Your Server is running.
<p>
The time is now [clock format [clock seconds]]
[my html_footer]
```

Ein vollständiges Beispiel für einen httpd-Server im Verzeichnis /examples der Tcllib. Hier wird zudem gezeigt, wie URIs über SCGI und HTTP-Proxys an andere Prozesse weitergeleitet werden können.

```
cd ~/tcl/sandbox/tcllib
tclsh examples/httpd.tcl
```

Klassen

Klasse httpd::mime

Eine Metaklasse für das MIME-Verarbeitungsverhalten über einen aktiven Socket hinweg

Methoden

method ChannelCopy *in out* ?*args*?
siehe unten ...

method html_header ?*title* ? ?*args*?
Gibt einen HTML-Block zurück

method html_footer ?*args*?
siehe unten ...

method http_code_string *code*
siehe unten ...

method HttpHeaders *sock* ?*debug* ?
siehe unten ...

method HttpHeaders_Default
siehe unten ...

method HttpServerHeaders
siehe unten ...

method MimeParse *mimetext*
Wandelt einen Block von MIME-kodiertem Text in eine Schlüssel-Wert-Liste um. Tritt eine Ausnahme auf, löst die Methode einen eigenen Aufruf der Methode **error** aus und ruft unverzüglich die Methode **output** auf, um einen Fehlercode zu generieren und die Verbindung zu schließen.

method Url_Decode *data*
Entfernt die HTTP-Kodierung aus einer Zeichenkette.

- Methode **Url_PathCheck** *urlsuffix*
- Methode **wait** *mode sock*

Klasse httpd::reply

ancestors: **httpd::mime**

Eine Klasse, die eine Anfrage durch den Prozess der Antwortgenerierung begleitet. Der mit der Antwort verbundene Socket ist jederzeit über die Variable *chan* verfügbar. Der Prozess der Antwortgenerierung beginnt damit, dass eine **httpd::server** ein **http::class**-Objekt erzeugt, eine Reihe von Verhaltensweisen einbindet und anschließend die **dispatch**-Methode des Antwortobjekts aufruft. Im Normalbetrieb führt die Methode **dispatch** folgende Schritte aus:

1. Ruft die Methode **reset** des Objekts auf, um die Standard-Header zu füllen.
2. Ruft die Methode **HttpHeaders** auf, um die MIME-Header aus dem Socket zu streamen
3. Ruft die Methode **request parse** auf, um den Strom der MIME-Header in ein Dictionary umzuwandeln, das über die Methode **request** gelesen werden kann.
4. Speichert den Rohdatenstrom der MIME-Header in der Variable *rawrequest* des Objekts.
5. Ruft die Methode **content** für das Objekt auf und löst einen Aufruf der Methode **error** aus, falls eine Ausnahme ausgelöst wird.
6. Ruft die Methode **output** des Objekts auf.

Entwickler können die Ausgabe über die Methode **puts** der Antwort in einen Puffer schreiben oder einfach die Variable *reply_body* des Objekts füllen. Die von der Methode **content** zurückgegebenen Informationen werden in keiner Weise interpretiert. Wenn eine Ausnahme ausgelöst wird (beispielsweise über den Befehl **error** in Tcl), erzeugt der Aufrufer automatisch eine Meldung 500 {Internal Error}. Eine typische Implementierung von **content** sieht wie folgt aus:

```
clay::define ::test::content.file {
   superclass ::httpd::content.file
   # Return a file
   # Note: this is using the content.file mixin which looks for the
   reply_file variable
   # and will auto-compute the Content-Type
   method content {} {
     my reset
    set doc_root [my request get DOCUMENT_ROOT]
    my variable reply_file
    set reply_file [file join $doc_root index.html]
   }
}
clay::define ::test::content.time {
  # return the current system time
   method content {} {
       my variable reply_body
    my reply set Content-Type text/plain
       set reply_body [clock seconds]
   }
}
clay::define ::test::content.echo {
   method content {} {
       my variable reply_body
    my reply set Content-Type [my request get CONTENT_TYPE]
       set reply_body [my PostData [my request get CONTENT_LENGTH]]
```

```
   }
}
clay::define ::test::content.form_handler {
   method content {} {
     set form [my FormData]
     my reply set Content-Type {text/html; charset=UTF-8}
    my puts [my html_header {My Dynamic Page}]
    my puts "<BODY>"
    my puts "You Sent<p>"
    my puts "<TABLE>"
    foreach {f v} $form {
      my puts "<TR><TH>$f</TH><TD><verbatim>$v</verbatim></TD>"
    }
    my puts "</TABLE><p>"
    my puts "Send some info:<p>"
    my puts "<FORM action=/[my request get REQUEST_PATH] method POST>"
    my puts "<TABLE>"
    foreach field {name rank serial_number} {
      set line "<TR><TH>$field</TH><TD><input name=\"$field\" "
      if {[dict exists $form $field]} {
        append line " value=\"[dict get $form $field]\""
      }
      append line " /></TD></TR>"
      my puts $line
    }
    my puts "</TABLE>"
    my puts [my html footer]
   }
}
```

Variable

variable ChannelRegister
siehe unten ...

variable reply
Ein Dictionary, das in die MIME-Header der Antwort umgewandelt wird

variable request
Ein Dictionary, das die durch SCGI durchgeführte Transformation der HTTP-Header für die Anfrage

Delegate enthält

delegate
Das Server-Objekt, das diese Antwort erzeugt hat

Methoden

method constructor *ServerObj* ?*args*?
siehe unten ...

method destructor ?*dictargs*?
Räumt beim Beenden auf.

method ChannelRegister ?*args*?
Registriert einen Kanal, der von der Methode **close** geschlossen werden soll.

method close
Schließt die von diesem Objekt geöffneten Kanäle.

method Log_Dispatched
Erfasst ein Dispatch-Ereignis.

method dispatch *newsock datastate*
Nimmt die Übergabe des Sockets *newsock* vom Server-Objekt entgegen und übergibt ihm den Zustand *datastate*. Felder, nach denen *datastate* insbesondere sucht, sind:

* **mixin** – eine Schlüssel-Wert-Liste mit Slots und Klassen, die vor dem Aufruf von **Dispatch** in das Objekt eingebunden werden sollen.

* **http** – eine Schlüssel-Wert-Liste mit Werten, die in das Ensemble *request* des Objekts eingefügt werden sollen.

Alle übrigen Felder werden an die Struktur **clay** des Objekts weitergereicht.

method Dispatch
siehe unten ...

method html_header *title* ?*args*?
siehe unten ...

method html_footer ?*args*?
siehe unten ...

method error *code* ?*msg* ? ?*errorInfo* ?
siehe unten ...

method content
REPLACE ME: Diese Methode ist der Kern Ihrer Anwendung. Sie schreibt über die Methode **puts** in den Ergebnispuffer und kann die Header über "clay put header_reply" anpassen.

method EncodeStatus *status*
Bildet aus der angegebenen Zeichenkette einen HTTP-Status-Header in Standardform.

method log *type* ?*info* ?
siehe unten ...

method CoroName
siehe unten ...

method DoOutput
Erzeugt die HTTP-Antwort, überträgt sie über *chan* und zerstört das Objekt.

method FormData
Bei GET-Anfragen wird der QUERY_DATA-Header in eine Schlüssel-Wert-Liste umgewandelt. Bei POST-Anfragen werden die POST-Daten gelesen und diese Informationen für "application/x-www-form-urlencoded"-POSTs in eine Schlüssel-Wert-Liste umgewandelt. Bei Multipart-Posts fasst es alle MIME-Header des Posts zu einer einzigen Schlüssel-Wert-Liste zusammen und stellt MIME_* Informationen bereit, wie sie vom Paket **mime** berechnet

werden, einschließlich des MIME_TOKEN, das wieder in das mime-Paket eingespeist werden kann, um den Inhalt auszulesen.

method PostData *length*
Liest *length* Bytes aus dem Socket *chan*, aber nur wenn die Anfrage ein POST oder PUSH ist. Andernfalls wird eine leere Zeichenkette zurückgegeben.

method Session_Load
Verwaltet Sitzungsdaten.

method puts *line*
Hängt den Wert von *string* an *reply_body* an und fügt ein abschließendes Zeilenumbruchzeichen hinzu.

method RequestFind *field*
siehe unten ...

method request *subcommand* ?*args*?
siehe unten ...

method reply *subcommand* ?*args*?
siehe unten ...

method reset
Löscht den Inhalt der Variablen *reply_body* und setzt alle Header in der Struktur **reply** auf die Standardwerte dieses Objekts zurück.

method timeOutCheck
Wird vom Objekt **http::server** aufgerufen, das diese Antwort ausgelöst hat. Es wird geprüft, ob beim Warten auf Daten oder beim Erstellen einer Antwort zu viel Zeit verstrichen ist; ist dies der Fall, wird ein Timeout-Fehler an die Anfrage zurückgegeben, das Objekt zerstört und der *chan*-Socket geschlossen.

method timestamp
Gibt die aktuelle Systemzeit im folgenden Format zurück:

```
%a, %d %b %Y %T %Z
```

Klasse httpd::server

ancestors: **httpd::mime**

Variable

- Variable **template**
- Variable **url_patterns**

Methoden

method constructor *args* ?*port* auto? ?*myaddr* 127.0.0.1? ?*string* auto? ?*name* auto? ?*doc_root* ? ?*reverse_dns* 0? ?*configuration_file* ? ?*protocol* HTTP/1.1?
siehe unten ...

method destructor ?*dictargs*?
siehe unten ...

method connect *sock ip port*
Antwortet auf einen offenen Socket. Diese Methode erzeugt eine Coroutine, die den Rest der Verbindung verwaltet. Die Abläufe der Coroutine steuert die Methode **Connect**.

method ServerHeaders *ip http_request mimetxt*
siehe unten ...

method Connect *uuid sock ip*
Diese Methode liest die HTTP-Header und greift anschließend auf die Methode **dispatch** zurück, um festzustellen, ob die Anfrage gültig ist und/oder welche Art von Antwort generiert werden soll. Im Normalfall wird ein Objekt der Klasse **::http::reply** erstellt und die Methode **dispatch** dieser Klasse aufgerufen. Dadurch wird die Kontrolle über den Socket an das Antwortobjekt übergeben. Das Antwortobjekt verwaltet den Rest der Transaktion, einschließlich des Schließens des Sockets.

method counter *which*
Erhöht einen internen Zähler.

method CheckTimeout
Prüft offene Verbindungen auf ein Timeout.

method debug ?*args*?
siehe unten ...

method dispatch *data*
Gibt zu einer Schlüssel-Wert-Liste mit Informationen eine Datenstruktur zurück, die beschreibt, wie der Server antworten soll.

method Dispatch_Default *reply*
Dispatch-Methode als letztes Mittel, bevor ein Fehler 404 NOT FOUND zurückgegeben wird. Standardmäßig wird in *DOCUMENT_ROOT* nach einer Datei gesucht, die zur Abfrage passt.

method Dispatch_Local *data*
Dispatch-Methode, die vor den von Plugins implementierten Methoden aufgerufen wird. Gibt sie ein nicht leeres Dictionary zurück, wird diese Struktur an die Antwort übergeben. Die Standardimplementierung ist leer.

method Headers_Local *varname*
Untersucht eine für eine Antwort vorgesehene Datenstruktur und passt sie gegebenenfalls an. Diese Methode wird vor den von Plugins implementierten Header-Methoden aufgerufen. Die Standardimplementierung ist leer.

method Headers_Process *varname*
Untersucht eine für eine Antwort vorgesehene Datenstruktur und passt sie gegebenenfalls an. Diese Methode wird von der Methode **plugin** dynamisch aufgebaut.

method HostName *ipaddr*
Wandelt eine IP-Adresse in einen Hostnamen um. Ist das Flag server/reverse_dns false, gibt die Methode nur die IP-Adresse zurück. Intern verwendet sie das Modul *dns* aus tcllib.

method log ?*args*?
Protokolliert ein Ereignis. Die Eingabe für args ist frei formulierbar. Die Methode ist dafür vorgesehen, vom Benutzer ersetzt zu werden, und tut bei einem http::server-Objekt im Auslieferungszustand nichts.

method plugin *slot* ?*class* ?
Bindet Verhalten aus einem Plugin ein. Diese Methode baut die Methoden **Dispatch** und **Headers** dynamisch neu auf. Für jedes Plugin sucht der Server unter *clay plugin/* nach diesen Einträgen:

load – ein Skript, das beim Aufruf der Methode **plugin** im Namespace des Servers ausgeführt

wird.

dispatch – ein Skript, das in die Methode **Dispatch** des Servers eingefügt wird.

headers – ein Skript, das in die Methode **Headers** des Servers eingefügt wird.

thread – ein Skript, das in die Methode **Thread_start** des Servers eingefügt wird.

method port_listening
Gibt den Port zurück, auf dem httpd tatsächlich auf Verbindungen wartet.

method PrefixNormalize *prefix*
Die Fassung im Auslieferungszustand entfernt abschließende "/" und "*" von einem Präfix. Die Methode kann ersetzt werden, um weitere für die Anwendung nötige Umformungen auszuführen.

method source *filename*
siehe unten ...

method start
Öffnet den Socket-Listener.

method stop
Schaltet den Socket-Listener ab und zerstört alle ausstehenden Antworten.

method SubObject {} db
siehe unten ...

method SubObject {} default
siehe unten ...

method template *page*
Gibt eine Vorlage für die Zeichenkette *page* zurück.

method TemplateSearch *page*
Sucht die Vorlage, die am besten zu *page* passt. Das kann lokale Dateisuchen, Strukturen im Arbeitsspeicher oder auch Datenbankabfragen umfassen. Die Fassung im Auslieferungszustand sucht nur nach Dateien mit der Erweiterung .tml oder .html im Verzeichnis ?doc_root?.

method Thread_start
Wird von der Methode **plugin** aufgebaut. Wird von der Methode **start** aufgerufen. Damit können Plugins Arbeits-Threads erzeugen.

method Uuid_Generate
Erzeugt eine GUUID. Damit erhält jede Anfrage eine eindeutige ID. Die Standardimplementierung lautet:

```
   return [::clay::uuid generate]
```

method Validate_Connection *sock ip*
Gibt bei einem Socket und einer IP-Adresse true zurück, wenn diese Verbindung beendet werden soll, und false, wenn sie fortgesetzt werden darf. Die Fassung im Auslieferungszustand gibt stets 0 zurück. Anwendungen können damit Sperrlisten umsetzen und den Zugriff anhand der IP-Adresse einschränken.

Klasse httpd::server::dispatch
ancestors: **httpd::server**

Stellt einen abwärtskompatiblen Alias bereit.

Klasse httpd::content.redirect
Methoden

- Methode **reset**
- Methode **content**

Klasse httpd::content.cache
Methoden

- Methode **Dispatch**

Klasse httpd::content.template
Methoden

- Methode **content**

Klasse httpd::content.file
Klasse zur Bereitstellung statischer Inhalte. Wenn diese Klasse verwendet wird, wird ihr vom Dispatcher ein lokaler Dateiname übergeben.

Methoden

- Methode **FileName**
- Methode **DirectoryListing** *local_file*
- Methode **content**
- Methode **Dispatch**

Klasse httpd::content.exec
Variable

- Variable **exename**

Methoden

- Methode **CgiExec** *execname script arglist*
- Methode **Cgi_Executable** *script*

Klasse httpd::content.proxy
ancestors: **httpd::content.exec**

Gibt Daten aus einem Proxy-Prozess zurück.

Methoden

- Methode **proxy_channel**
- Methode **proxy_path**
- Methode **ProxyRequest** *chana chanb*
- Methode **ProxyReply** *chana chanb* ?*args*?
- Methode **Dispatch**

Klasse httpd::content.cgi
ancestors: **httpd::content.proxy**

Methoden

method FileName
siehe unten ...

method proxy_channel
siehe unten ...

method ProxyRequest *chana chanb*
siehe unten ...

method ProxyReply *chana chanb* ?*args*?
siehe unten ...

method DirectoryListing *local_file*
Bei den meisten CGI-Anwendungen ist eine Verzeichnisauflistung verboten.

Klasse httpd::protocol.scgi

Gibt Daten aus einem SCGI-Prozess zurück.

Methoden

- Methode **EncodeStatus** *status*

Klasse httpd::content.scgi

ancestors: **httpd::content.proxy**

Methoden

- Methode **scgi_info**
- Methode **proxy_channel**
- Methode **ProxyRequest** *chana chanb*
- Methode **ProxyReply** *chana chanb* ?*args*?

Klasse httpd::server.scgi

ancestors: **httpd::server**

Fungiert als SCGI-Server.

Methoden

- Methode **debug** ?*args*?
- Methode **Connect** *uuid sock ip*

Klasse httpd::content.websocket

Wertet eine Verbindung zu einem WebSocket auf.

Klasse httpd::plugin

httpd-Plugin-Vorlage

Klasse httpd::plugin.dict_dispatch

Ein einfaches Plugin, das URLs anhand einer dict-Datenstruktur weiterleitet.

Methoden

method Dispatch_Dict *data*
Implementierung des Dispatchers

- Methode **uri {} add** *vhosts patterns info*

- Methode **uri {} direct** *vhosts patterns info body*

Klasse httpd::reply.memchan
ancestors: **httpd::reply**

Methoden

- Methode **output**
- Methode **DoOutput**
- Methode **close**

Klasse httpd::plugin.local_memchan
Methoden

method local_memchan *command* ?*args*?
siehe unten ...

method Connect_Local *uuid sock* ?*args*?
Eine abgewandelte Verbindungsmethode, die eine einfache GET-Anfrage an ein Objekt übergibt und die Daten direkt aus der Variablen reply_body des Objekts holt. Nötig, weil memchan bidirektional ist und sich anscheinend nicht ausdrücken lässt, dass der Server die eine Seite der Verbindung ist und die Antwort die andere.

Autoren
Sean Woods

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *network* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright

```
Copyright (c) 2018 Sean Woods <yoda@etoyoc.com>
```


ident

ident
Stichwörter: ident, identification, rfc 1413

Name
ident – Ident-Protokoll-Client

Übersicht

package require Tcl 8.5 9
package require ident ?**0.44**?
::ident::query *socket* ?*callback*?

Beschreibung

Das Paket **ident** enthält eine Client-Implementierung des ident-Protokolls, wie es in RFC 1413 (http://www.rfc-editor.org/rfc/rfc1413.txt) definiert ist.

::ident::query *socket* ?*callback*?

Dieser Befehl fragt den Ident-Daemon auf der Gegenstelle des angegebenen Sockets ab und gibt das Ergebnis der Abfrage als Dictionary zurück. Wenn Sie das Dictionary als Liste interpretieren, lautet der erste Schlüssel stets **resp-type** und kann einen der Werte **USERID**, **ERROR** und **FATAL** annehmen. Diese *Antworttypen* haben folgende Bedeutungen:

USERID

Dies weist auf eine erfolgreiche Antwort hin. Es werden zwei weitere Schlüssel mit den zugehörigen Werten zurückgegeben: **opsys** und **user-id**.

ERROR

Dies bedeutet, dass der Ident-Server einen Fehler zurückgegeben hat. Ein zweiter Schlüssel namens **error** ist vorhanden; sein Wert enthält das Feld **error-type** aus der Serverantwort.

FATAL

Schwerwiegende Fehler treten auf, wenn auf der Gegenstelle kein Ident-Server lauscht oder wenn der Ident-Server eine Antwort gibt, die nicht dem RFC entspricht. Eine detaillierte Fehlermeldung wird unter dem Schlüssel **error** zurückgegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ident* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2004 Reinhard Max <max@tclers.tk>
```


imap4

imap4

Stichwörter: imap4, email, imap, internet, mail, net, rfc3501, ssl, tls

Name

imap4 – clientseitige TCL-Implementierung des IMAP-Protokolls

Übersicht

package require Tcl 8.5 9
package require imap4 ?**0.5.5**?
::imap4::open *hostname* ?*port*?
::imap4::starttls *chan*
::imap4::login *chan user pass*
::imap4::folders *chan* ?*-inline*? ?*mboxref*? ?*mboxname*?
::imap4::select *chan* ?*mailbox*?
::imap4::examine *chan* ?*mailbox*?
::imap4::fetch *chan range* ?*-inline*? ?*attr ...*?
::imap4::noop *chan*
::imap4::check *chan*
::imap4::folderinfo *chan* ?*info*?
::imap4::msginfo *chan msgid* ?*info*? ?*defval*?
::imap4::mboxinfo *chan* ?*info*?
::imap4::isableto *chan* ?*capability*?
::imap4::create *chan mailbox*
::imap4::delete *chan mailbox*
::imap4::rename *chan oldname newname*
::imap4::subscribe *chan mailbox*
::imap4::unsubscribe *chan mailbox*
::imap4::search *chan expr* ?*...*?
::imap4::close *chan*
::imap4::cleanup *chan*
::imap4::debugmode *chan* ?*errormsg*?
::imap4::store *chan range data flaglist*
::imap4::expunge *chan*
::imap4::copy *chan msgid mailbox*
::imap4::logout *chan*

Beschreibung

Die Bibliothek **imap4** stellt die clientseitige Schnittstelle des *Internet Message Access Protocol* (IMAP) bereit, wobei Standard-Sockets oder eine sichere Verbindung über TLS/SSL verwendet werden. Das Paket ist vollständig in Tcl implementiert.

In diesem Dokument werden die Prozeduren beschrieben und deren Anwendung erläutert.

Prozeduren

Dieses Paket definiert die folgenden öffentlichen Prozeduren:

::imap4::open *hostname* ?*port*?

Öffnet eine neue IMAP-Verbindung und initialisiert den Handler; der IMAP-Kanal (Handler) wird zurückgegeben.

hostname – Mailserver

port – Verbindungsport, Standardwert ist 143.

Die Namespace-Variable **::imap4::use_ssl** kann verwendet werden, um eine sichere Verbindung über TLS/SSL herzustellen, sofern sie auf "true" gesetzt ist. In diesem Fall ist der Standard-Port für die Verbindung 993.

Hinweis: Für die Verbindung über SSL muss das Tcl-Modul *tls* bereits geladen sein, andernfalls wird ein Fehler ausgelöst.

```
    package require tls              ; # must be loaded for TLS/SSL
    set ::imap4::use_ssl 1           ; # request a secure connection
    set chan [::imap4::open $server] ; # default port is now 993
```

::imap4::starttls *chan*

Verwenden Sie diese Option, wenn Sie eine Verbindung zu einem ungesicherten Port herstellen sollen, der vor der Benutzeranmeldung in einen gesicherten Port umgeschaltet werden muss. Diese Funktion wird als *STARTTLS* bezeichnet.

::imap4::login *chan user pass*

Meldet sich mit dem Befehl "IMAP LOGIN" an; bei erfolgreicher Anmeldung gibt der Befehl den Wert 0 zurück.

chan – IMAP-Kanal

user – Benutzername

pass – Passwort

::imap4::folders *chan* ?*-inline*? ?*mboxref*? ?*mboxname*?

Ruft eine Liste der übereinstimmenden Ordner ab; bei Erfolg gibt es 0 zurück.

Für *mboxref* und *mboxname* sind Platzhalter wie "*" und "%" zulässig; mit dem Befehl **::imap4::folderinfo** können Ordnerinformationen abgerufen werden.

chan – IMAP-Kanal

mboxref – Postfachreferenz, Standardwert ist ""

mboxname – Postfachname, Standardwert ist "*"

Wenn **-inline** festgelegt wird, gibt es anstelle des Ergebniscodes eine kompakte Ordnerliste zurück. Alle Flags werden in Kleinbuchstaben umgewandelt und führende Sonderzeichen werden entfernt.

```
{{Arc08 noselect} {Arc08/Private {noinferiors unmarked}} {INBOX
    noinferiors}}
```

::imap4::select *chan* ?*mailbox*?

Wählt ein Postfach aus; bei Erfolg gibt es 0 zurück.

chan – IMAP-Kanal

mailbox – Pfad des Postfachs, Standardwert ist *INBOX*.

Vor der Auswahl/Überprüfung muss ein geöffnetes Postfach geschlossen werden – siehe: **::imap4::close**.

::imap4::examine *chan* ?*mailbox*?

"Untersucht" ein Postfach; entspricht der schreibgeschützten Version von **::imap4::select**.

chan – IMAP-Kanal

mailbox – Name oder Pfad der Mailbox; Standardwert ist *INBOX*.

Vor der Auswahl oder dem "Select"-Befehl muss eine geöffnete Mailbox geschlossen werden – siehe: **::imap4::close**.

::imap4::fetch *chan range* ?*-inline*? ?*attr ...*?

Attribute aus Nachrichten abrufen.

Die Attribute werden abgerufen und im internen Status gespeichert, der mit dem Befehl **::imap4::msginfo** abgerufen werden kann; bei Erfolg gibt er 0 zurück. Wenn **-inline** festgelegt wird, geben wir alle Datensätze als Liste in der im Argument *attr* definierten Reihenfolge zurück.

chan – IMAP-Kanal

range – Index der Nachrichten im Format *FROM*:*TO*

attr – abzurufende IMAP-Attribute

Hinweis: Wenn *FROM* weggelassen wird, wird die erste Nachricht angenommen; wenn *TO* weggelassen wird, wird die letzte Nachricht angenommen. Alle Indexbereiche beginnen bei 1.

::imap4::noop *chan*

Sendet den Befehl "NOOP" an den Server. Möglicherweise werden Informationen als unmarkierte Daten zurückgegeben.

chan – IMAP-Kanal

::imap4::check *chan*

Sendet den Befehl "CHECK" an den Server. Die Festplatte leeren.

chan – IMAP-Kanal

::imap4::folderinfo *chan* ?*info*?

Ruft Informationen zur zuletzt ausgewählten Ordnerliste ab. Wird das Argument *info* weggelassen oder ist es eine leere Zeichenkette, gibt der Befehl die vollständige Liste der für das Postfach verfügbaren Informationen zurück.

Endet der Name der gewünschten Information mit dem Zeichen ?, gibt der Befehl "true" zurück, wenn die Information verfügbar ist, andernfalls "false".

chan – IMAP-Kanal

info – Optionen für die abzurufende Ordnerliste

Derzeit unterstützte Optionen: *delim* – nur Hierarchie-Trennzeichen, *match* – Suchmuster für ref und mbox (siehe **::imap4::folders**), *names* – ausschließlich Liste der Ordnernamen, *flags* – Liste der Ordnernamen mit Flags im Format *{ {name {flags}} ... }* (Siehe auch kompaktes Format in der Funktion **::imap4::folders**).

```
{{Arc08 {{\NoSelect}}} {Arc08/Private {{\NoInferiors} {\UnMarked}}}
    {INBOX {\NoInferiors}}}
```

::imap4::msginfo *chan msgid* ?*info*? ?*defval*?

Ruft Informationen (die zuvor mit "fetch" erfasst wurden) zu einer bestimmten *msgid* ab. Wenn das Argument "info" weggelassen wird oder es eine leere Zeichenkette ist, gibt es die Liste der verfügbaren Informationsoptionen für die angegebene Nachricht zurück.

Wenn dem Namen der gewünschten Information ein "?"-Zeichen angehängt wird, gibt der Befehl "true" zurück, sofern die Information verfügbar ist, andernfalls "false".

chan – IMAP-Kanal

msgid – Nachrichtennummer

info – IMAP-Schlüsselwort zum Abrufen

defval – Standardwert, der gegeben wird, wenn "info" leer ist

Hinweis: Alle Nachrichten-Indexbereiche beginnen bei 1.

::imap4::mboxinfo *chan* ?*info*?

Ruft Informationen zum aktuell ausgewählten Postfach ab. Wird das Argument *info* weggelassen oder ist es eine leere Zeichenkette, wird die Liste der für das Postfach verfügbaren Informationsoptionen zurückgegeben.

Wird dem Namen der gewünschten Information ein "?"-Zeichen angehängt, gibt der Befehl "true" zurück, wenn die Information verfügbar ist, andernfalls "false".

chan – IMAP-Kanal

opt – abzurufende Postfachoption

Derzeit unterstützte Optionen: *EXISTS* (Anzahl der Nachrichten), *RECENT* (Anzahl der als "aktuell" markierten Nachrichten), *FLAGS*

In Verbindung mit OK: *PERMFLAGS*, *UIDNEXT*, *UIDVAL*, *UNSEEN*

Verschiedene Zustände: *CURRENT*, *FOUND*, *PERM*.

```
    ::imap4::select $chan INBOX
    puts "[::imap4::mboxinfo $chan exists] mails in INBOX"
```

::imap4::isableto *chan* ?*capability*?

Prüfung der Funktion. Die Funktion gibt 1 zurück, wenn die angeforderte Funktion unterstützt wird, andernfalls 0. Wird *capability* weggelassen, werden alle IMAP-Funktionscodes als Liste

zurückgegeben.

chan – IMAP-Kanal

info – IMAP-Schlüsselwort, das abgerufen werden soll

Hinweis: Verwenden Sie den Befehl "capability", um den Server zu befragen, falls dies vom Benutzer noch nicht geschehen ist.

::imap4::create *chan mailbox*

Erstellt ein neues Postfach.

chan – IMAP-Kanal

mailbox – Name des Postfachs

::imap4::delete *chan mailbox*

Ein neues Postfach löschen.

chan – IMAP-Kanal

mailbox – Name des Postfachs

::imap4::rename *chan oldname newname*

Ein neues Postfach umbenennen.

chan – IMAP-Kanal

mailbox – alter Name des Postfachs

mailbox – neuer Name des Postfachs

::imap4::subscribe *chan mailbox*

Ein neues Postfach abonnieren.

chan – IMAP-Kanal

mailbox – Name des Postfachs

::imap4::unsubscribe *chan mailbox*

Ein neues Postfach abmelden.

chan – IMAP-Kanal

mailbox – Name des Postfachs

::imap4::search *chan expr* ?*...*?

Sucht nach E-Mails, die den Suchkriterien entsprechen; bei Erfolg gibt es 0 zurück.

chan – IMAP-Kanal

expr – IMAP-Suchausdruck

Hinweise: Derzeit werden die folgenden Suchausdrücke unterstützt:

E-Mail-Header-Flags: alle E-Mail-Header-Einträge (die mit einem Doppelpunkt ":" enden), wie z. B. "From:", "Bcc:", ...

IMAP-Nachrichtensuchflags: ANSWERED, DELETED, DRAFT, FLAGGED, RECENT, SEEN, NEW, OLD, UNANSWERED, UNDELETED, UNDRAFT, UNFLAGGED, UNSEEN, ALL

IMAP-Header-Suchflags: BODY, CC, FROM, SUBJECT, TEXT, KEYWORD, BCC

Bedingte Suchflags für IMAP: SMALLER, LARGER, ON, SENTBEFORE, SENTON, SENTSINCE, SINCE, BEFORE (nicht implementiert), UID (nicht implementiert)

Logische Suchbedingungen: OR, NOT

```
::imap4::search $chan larger 4000 seen
puts "Found messages: [::imap4::mboxinfo $chan found]"
Found messages: 1 3 6 7 8 9 13 14 15 19 20
```

::imap4::close *chan*

Schließt das Postfach. Die Nachrichten im Ordner "\Deleted" werden endgültig gelöscht, und der Status wird auf "AUTH" zurückgegeben.

chan – IMAP-Kanal

::imap4::cleanup *chan*

Eine IMAP-Verbindung trennen und den belegten Speicherplatz freigeben. Das Postfach schließen. Die Nachrichten im Ordner "Gelöscht" werden endgültig entfernt, und der Status gibt zurück zu "AUTH".

chan – IMAP-Kanal

::imap4::debugmode *chan* ?*errormsg*?

Schaltet den Client in den Debug-Modus der Befehlszeile um.

Hierbei handelt es sich um einen reinen Entwicklermodus, der die Kontrolle an den Programmierer übergibt. Jede eingegebene Zeile wird unverändert an den Server gesendet (nach Hinzufügung der Anfrage-ID). Die Variable ::imap4::debug wird bei der Eingabe automatisch auf "1" gesetzt.

Es ist möglich, Tcl-Befehle auszuführen, die mit einem Schrägstrich beginnen.

chan – IMAP-Kanal

errormsg – optionale Fehlermeldung zur Anzeige

::imap4::store *chan range data flaglist*

Ändert die mit einer Nachricht im ausgewählten Postfach verknüpften Daten.

chan – IMAP-Kanal

range – Index der Nachricht im Format *FROM*:*TO*

flaglist – Flags, auf die sich *data* bezieht.

data – Die derzeit definierten *data*-Elemente, die gespeichert werden können, sind unten aufgeführt. *Hinweis*: Allen diesen Datentypen kann auch das Suffix ".SILENT" hinzugefügt werden, um die FETCH-Antwort ohne Tags zu unterdrücken.

FLAGS
Ersetzt die Flags für die Nachricht (mit Ausnahme von \Recent) durch die *flaglist*.

+FLAGS
Fügt die Flags in *flaglist* zu den bereits vorhandenen Flags für die Nachricht hinzu.

-FLAGS
Entfernt die Flags in *flaglist* aus den bestehenden Flags für die Nachricht.

Beispiel:

```
::imap4::store $chan $start_msgid:$end_msgid +FLAGS "Deleted"
```

::imap4::expunge *chan*
Entfernt dauerhaft alle Nachrichten, bei denen das Flag "\Deleted" gesetzt ist, aus dem aktuell ausgewählten Postfach, ohne dass die Verbindung geschlossen werden muss.

chan – IMAP-Kanal

::imap4::copy *chan msgid mailbox*
Kopiert die angegebene Nachricht (identifiziert durch ihre Nachrichtennummer) in das benannte Postfach, d. h. den IMAP-Ordner.

chan – IMAP-Kanal

msgid – Nachrichtennummer

mailbox – Name des Postfachs

::imap4::logout *chan*
Teilt dem Server mit, dass der Client die Verbindung beendet hat, und schließt die Netzwerkverbindung. Löscht Nachrichten mit dem Tag "\Deleted" endgültig.

Um sich erneut anzumelden, muss eine neue Verbindung hergestellt werden.

chan – IMAP-Kanal

Beispiele

```
set user myusername
set pass xtremescrt
set server imap.test.tld
set FOLDER INBOX
# Connect to server
set imap [::imap4::open $server]
::imap4::login $imap $user $pass
::imap4::select $imap $FOLDER
# Output all the information about that mailbox
```

```
    foreach info [::imap4::mboxinfo $imap] {
        puts "$info -> [::imap4::mboxinfo $imap $info]"
    }
    # fetch 3 records inline
    set fields {from: to: subject: size}
    foreach rec [::imap4::fetch $imap :3 -inline {*}$fields] {
        puts -nonewline "#[incr idx])"
        for {set j 0} {$j<[llength $fields]} {incr j} {
            puts "\t[lindex $fields $j] [lindex $rec $j]"
        }
    }

    # Show all the information available about the message ID 1
    puts "Available info about message 1: [::imap4::msginfo $imap 1]"

    # Use the capability stuff
    puts "Capabilities: [::imap4::isableto $imap]"
    puts "Is able to imap4rev1? [::imap4::isableto $imap imap4rev1]"

    # Cleanup
    ::imap4::cleanup $imap
```

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Literatur

Mark R. Crispin, "INTERNET MESSAGE ACCESS PROTOCOL – VERSION 4rev1", RFC 3501, März 2003, http://www.rfc-editor.org/rfc/rfc3501.txt

OpenSSL, http://www.openssl.org/

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *imap4* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste. Nur ein kleiner Teil von RFC 3501 ist umgesetzt.

Kategorie

Netzwerk

inifile

ini

Stichwörter: ini

Name

inifile – Auswerten von Windows-INI-Dateien

Übersicht

package require Tcl 8.5 9
package require inifile ?**0.3.3**?
::ini::open *file* ?*-encoding encoding*? ?*access*?
::ini::close *ini*
::ini::commit *ini*
::ini::revert *ini*
::ini::filename *ini*
::ini::sections *ini*
::ini::keys *ini section*
::ini::get *ini section*
::ini::exists *ini section* ?*key*?
::ini::value *ini section key* ?*default*?
::ini::set *ini section key value*
::ini::delete *ini section* ?*key*?
::ini::comment *ini section* ?*key*? ?*text*?
::ini::commentchar ?*char*?

Beschreibung

Dieses Paket bietet eine Schnittstelle zur einfachen Bearbeitung von Windows-INI-Dateien.

::ini::open *file* ?-encoding *encoding*? ?*access*?
Öffnet eine INI-Datei und gibt ein Handle zurück, das von anderen Befehlen verwendet wird. *access* entspricht der ersten (nicht POSIX-konformen) Form des Befehls **open**, mit der Ausnahme, dass der Modus **a** nicht unterstützt wird. Der Standardmodus ist **r+**.

Die Standard-*Kodierung* ist die Systemkodierung.

::ini::close *ini*
Schließt das angegebene Handle. Falls Änderungen vorgenommen, aber nicht mit **commit** gespeichert wurden, gehen diese verloren.

::ini::commit *ini*
Schreibt die Datei und alle Änderungen auf die Festplatte. Die Abschnitte werden in beliebiger Reihenfolge geschrieben. Die Schlüssel innerhalb eines Abschnitts werden in alphabetischer Reihenfolge geschrieben. Wenn die INI-Datei im schreibgeschützten Modus geöffnet wurde, wird ein Fehler ausgelöst.

::ini::revert *ini*
Setzt alle am inifile-Objekt vorgenommenen Änderungen auf den zuletzt festgeschriebenen Zustand zurück.

::ini::filename *ini*
Gibt den Namen der Datei zurück, mit der das *ini*-Objekt verknüpft ist.

::ini::sections *ini*
Gibt eine Liste aller Namen der vorhandenen Abschnitte im angegebenen Dateihandle zurück.

::ini::keys *ini section*
Gibt eine Liste aller Schlüsselnamen in dem angegebenen Abschnitt und der angegebenen Datei zurück.

::ini::get *ini section*
Gibt eine Liste von Schlüssel-Wert-Paaren zurück, die in dem angegebenen Abschnitt und der angegebenen Datei vorhanden sind.

::ini::exists *ini section* ?*key*?
Gibt einen booleschen Wert zurück, der angibt, ob der angegebene Abschnitt als Ganzes oder der angegebene Schlüssel innerhalb dieses Abschnitts vorhanden ist.

::ini::value *ini section key* ?*default*?
Gibt den Wert des angegebenen Schlüssels und Abschnitts zurück. Wenn angegeben, wird der Standardwert zurückgegeben, wenn der Schlüssel nicht existiert. Wenn der Schlüssel nicht existiert und kein Standardwert festgelegt wurde, wird ein Fehler ausgelöst.

::ini::set *ini section key value*
Legt den Wert des Schlüssels im angegebenen Abschnitt fest. Wenn der Abschnitt nicht existiert, wird ein neuer erstellt.

::ini::delete *ini section* ?*key*?
Entfernt den Schlüssel oder den gesamten Abschnitt samt aller darin enthaltenen Schlüssel. Ein Abschnitt wird nicht automatisch gelöscht, wenn er keine Schlüssel mehr enthält.

::ini::comment *ini section* ?*key*? ?*text*?
Liest und ändert Kommentare für Abschnitte und Schlüssel. Um einen Abschnitts-Kommentar zu schreiben, verwenden Sie eine leere Zeichenkette für den *Schlüssel*. Um alle Kommentare zu entfernen, verwenden Sie eine leere Zeichenkette für *text*. *text* kann aus einer Liste von Zeilen oder einer einzelnen Zeile bestehen. Alle in *text* enthaltenen Zeilenumbrüche werden korrekt verarbeitet. Kommentare können für nicht vorhandene Abschnitte oder Schlüssel geschrieben werden und führen nicht zu einem Fehler. Das Lesen eines Kommentars aus einem nicht vorhandenen Abschnitt oder Schlüssel gibt eine leere Zeichenkette zurück.

::ini::commentchar ?char?
Liest das Kommentarzeichen aus und legt es fest. Zeilen, die mit diesem Zeichen beginnen, werden als Kommentare behandelt. Wenn Kommentare ausgegeben werden, wird jedem Satz dieses Zeichen vorangestellt. Die Standardeinstellung ist **;**.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *inifile* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

interp

deleg_method
Stichwörter: deleg_method, comm, delegation, interpreter, method, snit

Name
deleg_method – Erstellung von Kommunikationsdelegaten (snit-Methoden)

Übersicht
package require Tcl 8.5 9
package require snit ?**1.1**?
package require interp::delegate::method ?**0.3**?
::interp::delegate::method ?**-async**? *name arguments comm id*

Beschreibung
Dieses Paket stellt einen einzigen Befehl zur Verwendung innerhalb einer Definition vom Typ **snit** (d. h. eigentlich **snit::macro**) bereit, zur bequemen Erstellung von Methoden, die die eigentliche Arbeit über einen vom Paket **comm** erstellten "Kanal" an einen entfernten Standort weiterleiten.

API
::interp::delegate::method ?-async? *name arguments comm id*
Dieser Befehl erstellt eine Methode mit dem Namen *name*. Alle Aufrufe dieser Methode leiten die eigentliche Arbeit an den Remote-Standort weiter, der durch den Kanal *comm* und die Endpunkt-ID *id* identifiziert wird.

Der Name der vom Delegator aufgerufenen Remote-Methode ist identisch mit dem Namen der Methode selbst.

Normalerweise marshallt die generierte Methode die *arguments* und gibt das Ergebnis der

Remote-Methode als ihr eigenes Ergebnis zurück. Wenn jedoch die Option **-async** festgelegt wurde, wartet die generierte Methode nicht auf ein Ergebnis und gibt es sofort zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *interp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

deleg_proc

Stichwörter: deleg_proc, comm, delegation, interpreter, procedure

Name

deleg_proc – Erstellung von Kommunikationsdelegaten (Prozeduren)

Übersicht

package require Tcl 8.5 9
package require interp::delegate::proc ?**0.3**?
::interp::delegate::proc ?**-async**? *name arguments comm id*

Beschreibung

Dieses Paket stellt einen einzigen Befehl zur Verfügung, mit dem sich Prozeduren bequem erstellen lassen, die die eigentliche Arbeit über einen vom Paket **comm** erstellten "Kanal" an einen entfernten Standort weiterleiten.

API

::interp::delegate::proc ?-async? *name arguments comm id*

Dieser Befehl erstellt eine Prozedur mit dem Namen *name* und gibt deren vollqualifizierten Namen zurück. Alle Aufrufe dieser Prozedur leiten die eigentliche Arbeit an den Remote-Standort weiter, der durch den Kommunikationskanal *comm* und die Endpunkt-ID *id* identifiziert wird.

Der Name der vom Delegator aufgerufenen Remote-Prozedur lautet [Namespace tail *name*]. Das heißt, die Namespace-Informationen werden aus dem Aufruf entfernt.

Normalerweise marshallt die generierte Prozedur die *arguments* und gibt das Ergebnis der

Remote-Prozedur als ihr eigenes Ergebnis zurück. Wenn jedoch die Option **-async** festgelegt wurde, wartet die generierte Prozedur nicht auf ein Ergebnis und gibt es sofort zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *interp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_interp

Stichwörter: tcllib_interp, alias, empty interpreter, interpreter, method, snit

Name

interp – Erstellung von Interp und Aliasing

Übersicht

package require Tcl 8.5 9
package require interp ?**0.1.3**?
::interp::createEmpty ?*path*?
::interp::snitLink *path methodlist*
::interp::snitDictLink *path methoddict*

Beschreibung

Dieses Paket bietet eine Reihe von Befehlen zur bequemen Erstellung von Tcl-Interpretern für eine stark eingeschränkte Ausführung.

API

::interp::createEmpty ?*path*?

Dieser Befehl erstellt einen leeren Tcl-Interpreter und gibt dessen Namen zurück. "Leer" bedeutet, dass der neue Interpreter weder Namespaces noch Befehle enthält. Er ist ausschließlich für die Erstellung von Aliasen nützlich.

Wenn ein *path* festgelegt wird, wird dieser als Name des neuen Interpreters verwendet.

::interp::snitLink *path methodlist*

Dieser Befehl setzt voraus, dass er innerhalb einer Methode eines snit-Objekts aufgerufen wurde und dass der Befehl **mymethod** verfügbar ist.

Er erweitert den durch *path* festgelegten Interpreter um Aliase für alle in der *methodlist* aufgeführten Methoden, wobei der Alias die Ausführung an die gleichnamige Methode des SNIT-Objekts weiterleitet, das diesen Befehl aufruft. Jedes Element von *methodlist* wird tatsächlich als Befehlspräfix interpretiert, wobei das erste Wort jedes Präfixes den Namen der Methode angibt, mit der eine Verknüpfung hergestellt werden soll.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::interp::snitDictLink *path methoddict*

Dieser Befehl verhält sich wie **::interp::snitLink**, mit dem Unterschied, dass er als Eingabe ein Dictionary verwendet, das Befehle Methoden zuordnet, und keine Liste von Methoden. Wie bei **::interp::snitLink** handelt es sich bei den Methodenverweisen tatsächlich um Befehlspräfixe. Dieser Befehl ermöglicht die Erstellung komplexerer Befehls-Methoden-Zuordnungen als **::interp::snitLink**.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *interp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


irc

irc

Stichwörter: irc, chat

Name

irc – IRC-Verbindung und -Schnittstelle erstellen.

Übersicht

package require Tcl 8.6 9
package require irc ?**0.8.0**?
::irc::config ?*key*? ?*value*?

::irc::connection
::irc::connections
net **registerevent** *event script*
net **getevent** *event script*
net **eventexists** *event script*
net **connect** *hostname* ?*port*?
net **config** ?*key*? ?*value*?
net **log** *level message*
net **logname**
net **connected**
net **sockname**
net **peername**
net **socket**
net **user** *username localhostname localdomainname userinfo*
net **nick** *nick*
net **ping** *target*
net **serverping**
net **join** *channel* ?*key*?
net **part** *channel* ?*message*?
net **quit** ?*message*?
net **privmsg** *target message*
net **notice** *target message*
net **ctcp** *target message*
net **kick** *channel target* ?*message*?
net **mode** *target args*
net **topic** *channel message*
net **invite** *channel target*
net **send** *text*
net **destroy**
who ?**address**?
action
target
additional
header
msg

Beschreibung

Dieses Paket stellt Low-Level-Befehle zur Verfügung, die das IRC-Protokoll (Internet Relay Chat) für die unmittelbare und interaktive Mehrbenutzer-Kommunikation nutzen.

::irc::config ?key? ?value?
: Setzt die Konfiguration ?key? auf ?value?. Die derzeit definierten Konfigurationsschlüssel sind die booleschen Flags **logger** und **debug**. **logger** bewirkt, dass **irc** das Logger-Paket zur Ausgabe von Fehlern verwendet. **debug** setzt **logger** voraus und gibt zusätzliche Debug-Ausgaben aus. Wenn weder ?key? noch ?value? angegeben wird, werden die aktuellen Werte zurückgegeben.

::irc::connection

Der Befehl erstellt ein neues Objekt zur Verwaltung einer IRC-Verbindung. Durch das Erstellen dieses IRC-Objekts wird die Netzwerkverbindung nicht automatisch hergestellt. Er gibt einen neuen Befehl im IRC-Namespace zurück, mit dem sich die neue IRC-Verbindung steuern lässt. Hinweis: Die alte Form des Befehls connection, bei der Hostname und Port als Argumente übergeben wurden, ist veraltet. Stattdessen legt **connect** diese Angaben fest.

::irc::connections
Gibt eine Liste aller aktuellen Verbindungen zurück, die mit **connection** erstellt wurden

Befehle pro Verbindung

In der folgenden Liste der verfügbaren Verbindungsmethoden steht *net* für einen Verbindungsbefehl, wie er von **::irc::connection** zurückgegeben wird.

net* registerevent *event script
Registriert einen Rückruf-Handler für das jeweilige Ereignis. Verfügbar sind die in RFC 1459 http://www.rfc-editor.org/rfc/rfc1459.txt beschriebenen Ereignisse. Darüber hinaus sind mehrere weitere Ereignisse definiert. **defaultcmd** fügt einen Befehl hinzu, der aufgerufen wird, wenn keine andere Rückruffunktion vorhanden ist. **EOF** wird aufgerufen, wenn die Verbindung ein Dateiende signalisiert. Die Ereignisse **defaultcmd**, **defaultnumeric**, **defaultevent** und **EOF** sind erforderlich. *script* wird im Namespace der Verbindung ausgeführt und kann dabei verschiedene Befehle nutzen (siehe Callback-Befehle weiter unten), die beim Parsen der Daten helfen.

net* getevent *event script
Gibt den aktuellen Ereignisbehandler zurück, sofern einer vorhanden ist. Andernfalls wird eine leere Zeichenkette zurückgegeben.

net* eventexists *event script
Gibt einen booleschen Wert zurück, der angibt, ob der Ereignisbehandler vorhanden ist.

***net* connect *hostname* ?port?**
Dadurch wird die Socket-Verbindung hergestellt. **::irc::connection** hat den Namespace und die zu verwendenden Befehle erstellt, jedoch den Socket noch nicht tatsächlich geöffnet. Dies geschieht hier. Hinweis: Bei der älteren Form von \'connect' mussten kein Hostname und kein Port angegeben werden; diese wurden mit 'connection' festgelegt. Diese Form ist veraltet.

***net* config ?key? ?value?**
Entspricht **::irc::config**, legt Optionen jedoch nur für die Verbindung *net* fest und ruft sie nur von dort ab.

net* log *level message
Wenn die Protokollierung durch **config** eingeschaltet ist, wird eine Protokoll-*message* auf der Stufe *level* geschrieben.

***net* logname**
Gibt den Namen der Logger-Instanz zurück, wenn die Protokollierung aktiviert ist.

***net* connected**
Gibt einen booleschen Wert zurück, der angibt, ob diese Verbindung zu einem Server besteht.

***net* sockname**
Gibt eine Liste mit drei Elementen zurück, bestehend aus der IP-Adresse, dem Hostnamen und dem Port der lokalen Seite der Verbindung, wenn derzeit eine Verbindung besteht.

***net* peername**
Gibt eine Liste mit drei Elementen zurück, bestehend aus der IP-Adresse, dem Hostnamen und

dem Port der Gegenstelle, wenn derzeit eine Verbindung besteht.

***net* socket**
Gibt den Tcl-Kanal für den von der Verbindung verwendeten Socket zurück.

net* user *username localhostname localdomainname userinfo
Sendet den Befehl USER an den Server. *username* ist der Benutzername, der angezeigt werden soll. *localhostname* ist der Host-Teil des Hostnamens, *localdomainname* der Domainname und *userinfo* eine kurze Beschreibung der Person. Das zweite und dritte Argument werden vom IRC-Server normalerweise ignoriert.

net* nick *nick
Der Befehl NICK. *nick* ist der Nickname, der für diese Verbindung verwendet werden soll.

net* ping *target
Sendet einen CTCP-PING an *target*.

***net* serverping**
Sendet einen PING an den Server.

***net* join *channel* ?*key*?**
channel ist der IRC-Kanal, dem beigetreten wird. IRC-Kanäle beginnen in der Regel mit einem Rautezeichen ("#") oder einem Und-Zeichen ("&").

***net* part *channel* ?*message*?**
Bewirkt, dass der Client den *channel* verlässt. Einige Netzwerke unterstützen möglicherweise das optionale Argument *message*

***net* quit ?*message*?**
Weist den IRC-Server an, die aktuelle Verbindung zu schließen. Das Paket verwendet einen generischen Vorgabewert, wenn keine *message* angegeben wurde.

net* privmsg *target message
Sendet *message* an *target*, wobei es sich entweder um einen Kanal oder um einen anderen Benutzer handeln kann; im letzteren Fall wird dessen Nickname verwendet.

net* notice *target message
Sendet ein **notice** mit der *message* an *target*, wobei es sich entweder um einen Kanal oder um einen anderen Benutzer handeln kann; im letzteren Fall wird dessen Nickname verwendet.

net* ctcp *target message
Sendet eine CTCP-Nachricht vom Typ *message* an *target*

***net* kick *channel target* ?*message*?**
Entfernt den Benutzer *target* mit einer *message* aus dem Kanal *channel*. Letztere kann weggelassen werden.

net* mode *target args
Legt den Modus *args* für das Ziel *target* fest. *target* kann ein Kanal, ein Kanalbenutzer oder der eigene Benutzer sein.

net* topic *channel message
Legt das Thema auf *channel* auf *message* fest; durch die Angabe einer leeren Zeichenfolge wird das Thema entfernt.

net* invite *channel target
Lädt *target* ein, dem Kanal *channel* beizutreten

net* send *text
Sendet *text* an den IRC-Server.

***net* destroy**
Löscht die Verbindung sowie den zugehörigen Namespace und die zugehörigen Informationen.

Callback-Befehle

Diese Befehle können innerhalb von Rückruffunktionen verwendet werden

who ?address?
Gibt den Nickname des Benutzers zurück, der einen Befehl ausgeführt hat. Das optionale Schlüsselwort **address** bewirkt, dass der Befehl den Benutzer im Format "username@address" zurückgibt.

action
Gibt die ausgeführte Aktion zurück, z. B. KICK, PRIVMSG, MODE usw. Dies ist in der Regel nicht sinnvoll, da Rückruffunktionen an ein bestimmtes Ereignis gebunden sind.

target
Gibt das Ziel eines bestimmten Befehls zurück, beispielsweise den Kanal oder den Benutzer, an den eine PRIVMSG gesendet wird.

additional
Gibt eine Liste aller zusätzlichen Argumente nach dem Ziel zurück.

header
Gibt den gesamten Ereignis-Header (alles bis zum:) als reguläre Liste zurück.

msg
Gibt den Meldungsteil des Befehls zurück (den Teil nach dem:).

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *irc* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

picoirc

Stichwörter: picoirc, chat, irc

Name

picoirc – Ein kleiner und einfacher, einbettbarer IRC-Client.

Übersicht

package require Tcl 8.6 9
package require picoirc ?**0.14.1**?
::picoirc::connect *callback nick* ?*password*? *url*

::picoirc::post *context channel message*

Beschreibung

Dieses Paket enthält einen universell einsetzbaren, minimalistischen IRC-Client, der sich zur Einbindung in andere Anwendungen eignet. Die gesamte Kommunikation mit der übergeordneten Anwendung erfolgt über eine von der Anwendung bereitgestellte Rückrufprozedur. Jede Verbindung verfügt über einen eigenen Zustand, sodass sich mehrere Server in einer einzigen Anwendungsinstanz anbinden lassen.

Zum Aufbau einer IRC-Verbindung wird **picoirc::connect** mit einer Rückrufprozedur, einem auf IRC zu verwendenden Nickname und der IRC-URL aufgerufen, die die Verbindung beschreibt. Zurückgegeben wird ein Variablenname, der den IRC-Verbindungskontext darstellt. Einzelheiten stehen unter Callback.

Dieses Paket ist ein relativ einfacher IRC-Client. Wenn mehr Funktionen nötig sind, kommt das Paket **irc** in Betracht.

Befehle

::picoirc::connect *callback nick ?password? url*

Stellt eine neue IRC-Verbindung zu dem durch *url* angegebenen Server her und meldet sich mit dem *nick* als Benutzername und optional mit dem *password* an. Wenn die *url* mit *ircs://* beginnt, wird eine TLS-Verbindung hergestellt. Der *callback* muss den Angaben unter Callback entsprechen. Zurückgegeben wird eine paketspezifische Variable, die beim Aufruf anderer Befehle in diesem Paket verwendet wird.

Hinweis: Für die Verbindung über TLS muss das Tcl-Modul *tls* bereits geladen sein, andernfalls wird ein Fehler ausgelöst.

```
# must be loaded for TLS
package require tls
# default arguments
tls::init -autoservername true -command workaround \
    -require 1 -cadir /etc/ssl/certs -tls1 0 -tls1.1 0
# avoid annoying bgerror, errors are already catched internally
proc workaround {state args} {
    if {$state == "verify"} {
        return [lindex $args 3]
    }
}
```

::picoirc::post *context channel message*

Dieser Befehl verarbeitet Benutzereingaben und sendet sie an den Server. Wenn *message* mehrzeilig ist, wird jede Zeile einzeln verarbeitet und gesendet. Eine Reihe von Befehlen wird erkannt, wenn ihnen ein Schrägstrich (/) vorangestellt ist. Solche Befehle werden in IRC-Befehlssequenzen umgewandelt und anschließend gesendet. Wenn *channel* leer ist, wird die gesamte Rohausgabe an den Server behandelt. Standardmäßig wird die *message* in den IRC-Socket geschrieben. Bevor dies geschieht, wird die Rückruffunktion mit "debug write" aufgerufen. Dadurch kann der Autor der Anwendung die rohen IRC-Daten prüfen und bei Bedarf den Fehlercode break zurückgeben, um die weitere Verarbeitung anzuhalten. Auf diese

Weise kann die Anwendung das standardmäßige Senden über die Rückruffunktion überschreiben.

Callback

Die Rückruffunktion muss wie folgt aussehen:

```
proc Callback {context state args} {
}
```

Dabei ist context der Name der IRC-Kontextvariablen (falls er an eine picoirc-Prozedur zurückgegeben werden muss). state ist einer der unten beschriebenen Zustände.

init
wird unmittelbar vor der Erstellung des Sockets aufgerufen

connect
wird aufgerufen, sobald die Verbindung hergestellt ist, bevor wir einem Kanal beitreten

close
wird aufgerufen, wenn der Socket geschlossen wird, bevor der Kontext gelöscht wird. Tritt vor dem Herstellen der Verbindung ein Fehler auf, ist das einzige Argument die Fehlermeldung des Sockets.

userlist *channel nicklist*
Wird aufgerufen, um die Anwendung über eine aktualisierte Benutzerliste zu informieren. Diese wird generiert, wenn die Ausgabe des IRC-Befehls "NAMES" erkannt wird. Das Paket sammelt die gesamte Ausgabe, die sich über mehrere Zeilen vom Server erstrecken kann, und ruft diese Rückruffunktion auf, wenn alle Zeilen empfangen wurden.

userinfo *nick info*
wird als Antwort auf einen WHOIS-Befehl aufgerufen. *nick* ist der Benutzer, an den sich der Befehl richtete. *info* ist das Dictionary mit detaillierten Angaben zu diesem Benutzer: name, host, channels und userinfo. userinfo enthält in der Regel Name und Version des IRC-Clients des Benutzers.

chat *target nick message type*
Wird aufgerufen, wenn eine Nachricht eintrifft. *target* ist die Identität, an die die Nachricht gerichtet war. Dies kann der angemeldete Nickname oder ein Kanalname sein. *nick* ist der Name des Absenders der Nachricht. *message* ist der Text der Nachricht. *type* wird auf "ACTION" gesetzt, wenn die Nachricht als CTCP-ACTION gesendet wurde. *type* wird auf "NOTICE" gesetzt, wenn die Nachricht als NOTICE-Befehl gesendet wurde; in diesem Fall ist *target* leer, wenn es mit dem Nickname des aktuellen Benutzers übereinstimmt, oder es lautet "*"; im letzteren Fall bedeutet ein leeres *target*, dass die Benachrichtigung vom Server stammt.

mode *nick target flags*
Wird aufgerufen, wenn sich der Modus eines Benutzers oder eines Kanals ändert. *nick* ist der Name des Benutzers, der die Änderung angefordert hat; dieser Parameter kann leer sein, wenn es sich um den Server handelt. *target* ist die Identität, deren Modus geändert wird. *flags* sind die Änderungen am Modus.

system *channel message*
wird aufgerufen, wenn eine Systemmeldung empfangen wird

topic *channel topic*

wird aufgerufen, wenn die Zeichenfolge des Kanalthemas erkannt wird. *topic* ist der Text des Kanalthemas.

traffic *action channel nick* ?*newnick*?

wird aufgerufen, wenn Benutzer beitreten, den Server verlassen oder ihren Namen ändern. *action* ist entweder entered, left oder nickchange, und *nick* ist der Benutzer, der die Aktion ausführt. *newnick* ist der neue Name, wenn *action* nickchange ist.

Hinweis: *channel* ist bei diesen Meldungen oft leer, da Nickname-Aktionen für den gesamten IRC-Server gelten. Die Nicknames aller verbundenen Kanäle sind selbst zu verwalten.

version

Diese Funktion wird aufgerufen, um eine Versionszeichenfolge abzufragen, mit der die interne Version überschrieben werden soll. Wenn sie implementiert ist, sollte sie eine durch Doppelpunkte getrennte Zeichenfolge in folgendem Format zurückgeben:

Appname:Appversion:LibraryVersion

Beispielsweise lautet der Vorgabewert

PicoIRC:[package provide picoirc]:Tcl [info patchlevel]

debug *type raw*

wird aufgerufen, wenn Daten entweder aus dem Netzwerk-Socket gelesen oder in diesen geschrieben werden. *type* wird beim Lesen von Daten auf **read** gesetzt und auf **write**, wenn die Daten geschrieben werden sollen. *raw* sind die unverarbeiteten IRC-Protokolldaten.

In beiden Fällen kann die Anwendung den Fehlercode break zurückgeben, um die weitere Verarbeitung der Rohdaten zu unterbrechen. Handelt es sich um eine Operation vom Typ **read**, verarbeitet das Paket diese Zeile nicht. Handelt es sich um die Operation **write**, sendet das Paket die Daten nicht. Diese Rückruffunktion ist für die Fehlersuche bei Protokollproblemen vorgesehen, könnte bei Bedarf aber auch dazu dienen, die gesamte Ein- und Ausgabe umzuleiten.

Kategorie

Netzwerk

javascript

javascript

Stichwörter: javascript, checkbox, html, selectionbox, submitbutton

Name

javascript – Prozeduren zur Erzeugung von HTML- und JavaScript-Strukturen.

Übersicht

package require Tcl 8.5 9
package require javascript ?**1.0.3**?
::javascript::makeSelectorWidget *id leftLabel leftValueList rightLabel rightValueList*

rightNameList ?*length*? ?*minWidth*?
::javascript::makeSubmitButton *name value*
::javascript::makeProtectedSubmitButton *name value msg*
::javascript::makeMasterButton *master value slavePattern boolean*
::javascript::makeParentCheckbox *parentName childName*
::javascript::makeChildCheckbox *parentName childName*

Beschreibung

Das Paket **::javascript** stellt Befehle bereit, die HTML- und JavaScript-Code erzeugen. Diese Befehle geben üblicherweise eine HTML-Zeichenkette zurück. Insbesondere schreiben sie ihr Ergebnis nicht nach **stdout**.

::javascript::makeSelectorWidget *id leftLabel leftValueList rightLabel rightValueList rightNameList* ?*length*? ?*minWidth*?

Erzeugt HTML-Code für ein Megawidget mit zwei Mehrfachauswahlen. Es besteht aus zwei nebeneinander stehenden Mehrfachauswahlfeldern, getrennt durch eine Schaltfläche mit Pfeil nach links und eine mit Pfeil nach rechts. Die Schaltfläche mit dem Pfeil nach rechts verschiebt alle im linken Feld ausgewählten Elemente in das rechte Feld. Die Schaltfläche mit dem Pfeil nach links verschiebt alle im rechten Feld ausgewählten Elemente in das linke Feld. Das Argument *id* ist das Suffix aller HTML-Objekte dieses Megawidgets. Das Argument *leftLabel* ist der Text über dem linken Auswahlfeld. Das Argument *leftValueList* enthält die Werte der Elemente im linken Auswahlfeld. Das Argument *leftNameList* enthält die Namen, die im linken Auswahlfeld erscheinen. Das Argument *rightLabel* ist der Text über dem rechten Auswahlfeld. Das Argument *rightValueList* enthält die Werte der Elemente im rechten Auswahlfeld. Das Argument *rightNameList* enthält die Namen, die im rechten Auswahlfeld erscheinen. Das Argument *length* (optional) legt fest, wie viele Elemente gezeigt werden, bevor eine senkrechte Bildlaufleiste hinzukommt; die Vorgabe ist 8. Das Argument *minWidth* (optional) ist die Anzahl der Leerzeichen für die Mindestbreite des Feldes; die Vorgabe ist 32.

::javascript::makeSubmitButton *name value*

Erzeugt eine HTML-Schaltfläche zum Absenden, die für jedes registrierte Mehrfachauswahlfeld ein verstecktes Feld zurücksetzt. Das Argument *name* ist der Name des HTML-Schaltflächenobjekts. Das Argument *value* ist die Beschriftung des HTML-Schaltflächenobjekts.

::javascript::makeProtectedSubmitButton *name value msg*

Erzeugt eine HTML-Schaltfläche zum Absenden, die den Benutzer vor dem Absenden des Formulars mit einer Rückfrage (Weiter/Abbrechen) warnt. Das Argument *name* ist der Name des HTML-Schaltflächenobjekts. Das Argument *value* ist die Beschriftung des HTML-Schaltflächenobjekts. Das Argument *msg* ist die Meldung, die beim Drücken der Schaltfläche erscheint.

::javascript::makeMasterButton *master value slavePattern boolean*

Erzeugt eine HTML-Schaltfläche, die ihre untergeordneten Kontrollkästchen auf den booleschen Wert setzt. Das Argument *master* ist der Name des übergeordneten HTML-Kontrollkästchens des Kindes. Das Argument *value* ist der Wert des Masters. Das Argument *slaves* ist der Name des zu erzeugenden untergeordneten HTML-Kontrollkästchens. Das Argument *boolean* ist der boolesche JavaScript-Wert für alle untergeordneten Kontrollkästchen; er muss "true" oder "false" sein.

::javascript::makeParentCheckbox *parentName childName*

Erzeugt ein HTML-Kontrollkästchen und koppelt seinen Wert an den des untergeordneten Kontrollkästchens. Ist das übergeordnete nicht markiert, wird das untergeordnete ebenfalls nicht markiert. Das Argument *parentName* ist der Name des zu erzeugenden übergeordneten HTML-Kontrollkästchens. Das Argument *childName* ist der Name des untergeordneten HTML-Kontrollkästchens.

::javascript::makeChildCheckbox *parentName childName*

Erzeugt ein HTML-Kontrollkästchen und koppelt seinen Wert an den des übergeordneten Kontrollkästchens. Ist das untergeordnete markiert, wird das übergeordnete ebenfalls markiert. Das Argument *parentName* ist der Name des übergeordneten HTML-Kontrollkästchens. Das Argument *childName* ist der Name des zu erzeugenden untergeordneten HTML-Kontrollkästchens.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *javascript* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

CGI-Programmierung

jpeg

jpeg

Stichwörter: jpeg, comment, exif, image, jfif, thumbnail

Name

jpeg – Abfrage und Bearbeitung von Metadaten bei JPEG-Dateien

Übersicht

package require Tcl 8.5 9
package require jpeg ?**0.7**?
::jpeg::isJPEG *file*
::jpeg::imageInfo *file*
::jpeg::dimensions *file*
::jpeg::getThumbnail *file*
::jpeg::getExif *file* ?*section*?
::jpeg::getExifFromChannel *channel* ?*section*?
::jpeg::formatExif *keys*
::jpeg::exifKeys
::jpeg::removeExif *file*
::jpeg::stripJPEG *file*
::jpeg::getComments *file*

::jpeg::addComment *file text...*
::jpeg::removeComments *file*
::jpeg::replaceComment *file text*
::jpeg::debug *file*
::jpeg::markers *channel*

Beschreibung

Dieses Paket enthält Befehle zum Abfragen und Bearbeiten von JPEG-Bildern. JPEG steht für *Joint Photography Experts Group* und ist ein Standard für die verlustbehaftete Komprimierung fotografischer Bilder. Die Spezifikation finden Sie unter *LINK_HERE*.

Befehle

::jpeg::isJPEG *file*
Gibt einen booleschen Wert zurück, der angibt, ob *file* ein JPEG-Bild ist.

::jpeg::imageInfo *file*
Gibt ein Dictionary mit den Schlüsseln **version**, **units**, **xdensity**, **ydensity**, **xthumb** und **ythumb** zurück. Die Werte sind die zugehörigen Eigenschaften des JPEG-Bildes in *file*. Löst einen Fehler aus, wenn *file* kein JPEG-Bild ist.

::jpeg::dimensions *file*
Gibt die Abmessungen der JPEG-Datei *file* als Liste der horizontalen und vertikalen Pixelanzahl zurück. Löst einen Fehler aus, wenn *file* kein JPEG-Bild ist.

::jpeg::getThumbnail *file*
Diese Prozedur gibt die binären Daten des Vorschaubildes zurück, wenn in *file* ein JPEG-Vorschaubild enthalten ist, andernfalls die leere Zeichenkette. Hinweis: Vorschaubilder können auch in anderen Formaten als JPEG eingebettet sein, auch wenn das unüblich ist. Der Befehl findet Vorschaubilder, die im JFXX- oder im EXIF-Segment der JPEG-Daten kodiert sind. Sind beide vorhanden, hat das EXIF-Vorschaubild Vorrang. Löst einen Fehler aus, wenn *file* kein JPEG-Bild ist.

```
set fh [open thumbnail.jpg w+]
fconfigure $fh -translation binary -encoding binary
puts -nonewline $fh [::jpeg::getThumbnail photo.jpg]
close $fh
```

::jpeg::getExif *file* ?*section*?
section muss entweder **main** oder **thumbnail** lauten. Der Standardwert ist **main**. Gibt ein Dictionary zurück, das die EXIF-Informationen für den angegebenen Abschnitt enthält. Beispiel:

```
set exif {
Make     Canon
Model    {Canon DIGITAL IXUS}
DateTime {2001:06:09 15:17:32}
}
```

Es wird ein Fehler ausgelöst, wenn *file* kein JPEG-Bild ist.

::jpeg::getExifFromChannel *channel* ?*section*?

Dieser Befehl entspricht **::jpeg::getExif**, verwendet aber einen bereits geöffneten Kanal. *channel* sollte ein positionierbarer Kanal sein (seek), und *section* ist wie bei **::jpeg::getExif** beschrieben.

Hinweis: Der JPEG-Parser erwartet, dass der Anfang des Kanals der Anfang der Bilddaten ist. Liegt das Bild in einem Container-Dateiformat, kann es nötig sein, die JPEG-Daten in einen temporären Container zu lesen: eine temporäre Datei oder einen Speicherkanal.

Achtung: Den Kanal nach der Verwendung zu schließen, ist Sache des Aufrufenden.

::jpeg::formatExif *keys*

Nimmt eine Liste von Schlüssel-Wert-Paaren entgegen, wie **getExif** sie zurückgibt, und bringt viele der Werte in eine besser lesbare Form. Es reicht ein einziges Schlüssel-Wert-Paar; die gesamten EXIF-Daten sind nicht nötig.

```
foreach {key val} [::jpeg::formatExif [::jpeg::getExif
photo.jpg]] {
    puts "$key: $val"
}
```

```
array set exif [::jpeg::getExif photo.jpg]
puts "max f-stop: [::jpeg::formatExif [list MaxAperture
$exif(MaxAperture)]]
```

::jpeg::exifKeys

Gibt eine Liste der EXIF-Schlüssel zurück, die derzeit verstanden werden. In den Daten von **getExif** können Schlüssel vorkommen, die nicht verstanden werden. Diese erscheinen im vierstelligen Hexadezimalformat.

::jpeg::removeExif *file*

Entfernt das Exif-Datensegment aus der angegebenen Datei und ersetzt es durch ein Standard-JFIF-Segment. Es wird ein Fehler ausgelöst, wenn *file* kein JPEG-Bild ist.

::jpeg::stripJPEG *file*

Entfernt alle Metadaten aus der JPEG-Datei, sodass nur das Bild übrig bleibt. Dazu gehören Kommentare, EXIF-Segmente, JFXX-Segmente und anwendungsspezifische Segmente. Es wird ein Fehler ausgelöst, wenn *file* kein JPEG-Bild ist.

::jpeg::getComments *file*

Gibt eine Liste aller JPEG-Kommentare in *file* zurück. Löst einen Fehler aus, wenn *file* kein gültiges JPEG-Bild ist.

::jpeg::addComment *file text...*

Fügt dem JPEG-Bild in *file* einen oder mehrere einfache *text*-Kommentare hinzu. Löst einen Fehler aus, wenn *file* kein gültiges JPEG-Bild ist.

::jpeg::removeComments *file*

Entfernt alle Kommentare aus der angegebenen Datei. Es wird ein Fehler ausgelöst, wenn *file* kein gültiges JPEG-Bild ist.

::jpeg::replaceComment *file text*

Ersetzt den ersten Kommentar in der Datei durch den neuen *text*. Das ist nur eine Kurzform für **::jpeg::removeComments** und **::jpeg::addComment**. Löst einen Fehler aus, wenn *file*

kein gültiges JPEG-Bild ist.

::jpeg::debug *file*
Gibt alle uns bekannten Informationen zur angegebenen Datei in einem übersichtlichen Format aus.

::jpeg::markers *channel*
Dies ist ein interner Hilfsbefehl, dokumentiert für fortgeschrittene Nutzer des Pakets. Das Argument *channel* ist ein geöffnetes Dateihandle am Anfang des ersten Markers (üblicherweise 2 Byte). Der Befehl gibt eine Liste mit einem Element je JFIF-Marker in der Datei zurück. Jedes Element ist eine Liste aus dem Namen des Markers, seinem Offset in der Datei und seiner Länge. Der Offset zeigt auf den Anfang der Daten des Abschnitts, nicht auf den Marker selbst. Die Länge ist die Länge der Daten vom angegebenen Offset bis zum Anfang des nächsten Markers.

Einschränkungen
Kann nur mit Dateien arbeiten; EXIF-Daten lassen sich nicht schreiben; GPS-EXIF-Daten werden nicht geparst; Makernote-Daten sind noch nicht implementiert

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *jpeg* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dateiformate

Copyright

```
Copyright (c) 2004-2005, Code: Aaron Faupell
    <afaupell@users.sourceforge.net>
Copyright (c) 2007, Code:  Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2004-2009, Doc:  Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2011, Code: Pat Thoyts <patthoyts@users.sourceforge.net>
```


json

json
Stichwörter: json, data exchange, exchange format, javascript

Name
json – JSON-Parser

Übersicht

package require Tcl 8.5 9
package require json ?**1.3.6**?
::json::json2dict *json_txt*
::json::many-json2dict *json_txt* ?*max*?

Beschreibung

Das **json**-Paket bietet eine einfache, ausschließlich auf Tcl basierende Bibliothek zum Parsen des JSON-Datenaustauschformats, wie es in RFC 4627 festgelegt ist. Beim Parsen von JSON bestehen gewisse Unklarheiten, da JSON Typinformationen enthält, die bei der Konvertierung durch dieses Modul nicht beibehalten werden. Außerdem werden JSON-**null**-Werte in das String-Literal "null" konvertiert (siehe Ticket). Das **json**-Paket gibt Daten entweder als Tcl-**dict** oder als **list** zurück, die wiederum verschachtelte **dict**s oder **list**s enthalten können. (Für die Verwendung ist entweder das **dict**-Paket oder Tcl 8.5 oder höher erforderlich.)

Eine alternative, ausschließlich auf Tcl basierende, typkonservierende Implementierung finden Sie im Modul typed-json.

Befehle

::json::json2dict *json_txt*

Gibt ein **dict** oder eine **list** zurück (die wiederum verschachtelte **dict**s und/oder **list**s enthalten können), das bzw. die das Ergebnis der Analyse des in *json_txt* übergebenen JSON-formatierten Textes ist.

Wenn *json_txt* mehr als eine JSON-Entität enthält, gibt es nur die erste zurück.

::json::many-json2dict *json_txt* ?*max*?

Gibt eine **list** zurück (die ihrerseits verschachtelte **dict**s und/oder **list**s enthält), die das Ergebnis der Analyse aller JSON-Entitäten (oder, wenn *max* festgelegt ist, der ersten *max* Entitäten) im JSON-formatierten Text ist, der in *json_txt* übergeben wurde.

Wenn *max* festgelegt ist, muss es sich um eine positive ganze Zahl **> 0** handeln, andernfalls löst der Befehl einen Fehler aus.

Wenn der JSON-formatierte Text verschachtelte Listen und Dicts enthält, gilt dies auch für das zurückgegebene Dict. In solchen Fällen kann der Zugriff auf einen Wert eine *Kombination* aus der Verwendung von dict get und lindex erfordern. (Siehe das zweite Beispiel.)

Beispiele

Ein Beispiel für ein in Tcl konvertiertes JSON-Array. Ein JSON-Array wird als einzelnes Element mit mehreren Unterelementen zurückgegeben.

```
[
    {
       "precision": "zip",
       "Latitude":  37.7668,
       "Longitude": -122.3959,
       "Address":   "",
       "City":      "SAN FRANCISCO",
```

```
       "State":     "CA",
       "Zip":       "94107",
       "Country":   "US"
    },
    {
       "precision": "zip",
       "Latitude":  37.371991,
       "Longitude": -122.026020,
       "Address":   "",
       "City":      "SUNNYVALE",
       "State":     "CA",
       "Zip":       "94085",
       "Country":   "US"
    }
]
=>
{Country US Latitude 37.7668 precision zip State CA City {SAN
    FRANCISCO} Address {} Zip 94107 Longitude -122.3959} {Country US
    Latitude 37.371991 precision zip State CA City SUNNYVALE Address
    {} Zip 94085 Longitude -122.026020}
```

Ein Beispiel für ein JSON-Dikt, das einen Eintrag enthält, dessen Wert eine Liste von Dicts ist.

```
{
  "items": [
     {
      "snippet": {
        "title": "In The Shade",
        "channelTitle": "Flox - Topic"
      },
      "statistics": {
        "viewCount": "274"
      }
    }
  ]
}
=>
items {{snippet {title {In The Shade} channelTitle {Flox - Topic}}
    statistics {viewCount 274}}}
```

Angenommen, der im JSON-Format vorliegende Text befindet sich in der Variable **jdata**, dann kann er in ein **dict** umgewandelt und die Elemente extrahiert werden.

```
set d [json::json2dict $jdata]
set title [dict get [lindex [dict get $d items] 0] snippet title]
=>
In The Shade
```

Ein Beispiel für ein in Tcl konvertiertes JSON-Objekt. Ein JSON-Objekt wird als Liste mit mehreren Elementen (ein "dict") zurückgegeben.

```
{
    "Image": {
        "Width":  800,
        "Height": 600,
        "Title":  "View from 15th Floor",
        "Thumbnail": {
            "Url":    "http://www.example.com/image/481989943",
            "Height": 125,
            "Width":  "100"
        },
        "IDs": [116, 943, 234, 38793]
    }
}
=>
Image {IDs {116 943 234 38793} Thumbnail {Width 100 Height 125 Url
    http://www.example.com/image/481989943} Width 800 Height 600 Title
    {View from 15th Floor}}
```

Verwandte Themen

Um JSON zu schreiben, anstatt es zu parsen, siehe das Paket **json::write**.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *json* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

CGI-Programmierung

Copyright

```
Copyright (c) 2006 ActiveState Software Inc.
Copyright (c) 2009 Thomas Maeder, Glue Software Engineering AG
```

json_write

Stichwörter: json_write, data exchange, exchange format, javascript, json

Name

json::write – JSON-Generierung

Übersicht

package require Tcl 8.5 9
package require json::write ?**1.0.5**?
::json::write indented
::json::write indented *flag*
::json::write aligned
::json::write aligned *flag*
::json::write string *s*
::json::write array *arg...*
::json::write array-strings *arg...*
::json::write object *key value...*
::json::write object-strings *key value...*

Beschreibung

Das **json::write** Das Paket bietet eine einfache, ausschließlich auf Tcl basierende Bibliothek zur Generierung von Text im JSON-Datenaustauschformat http://www.json.org/, wie in RFC 4627 http://www.ietf.org/rfc/rfc4627.txt festgelegt.

Befehle

::json::write indented
Diese Methode gibt den aktuellen Status der Einrückungseinstellung zurück.

::json::write indented *flag*
Diese Option und die Methode **aligned** legen das Layout des vom Paket generierten JSON fest.

Ist dieses *Flag* gesetzt (Standard), teilt das Paket den generierten JSON-Code in Zeilen auf und rückt ihn entsprechend seiner inneren Struktur ein, wobei jeder Schlüssel eines Objekts in einer eigenen Zeile steht.

Ist dieses Flag nicht gesetzt, wird das gesamte JSON-Objekt in einer einzigen Zeile geschrieben, wobei der Abstand zwischen allen Elementen minimal ist.

::json::write aligned
Diese Methode gibt den aktuellen Status der Einstellung für die Ausrichtung zurück.

::json::write aligned *flag*
Diese Option und die Methode **indented** legen das Layout des vom Paket generierten JSON fest.

Ist dieses *Flag* gesetzt (Standard), sorgt das Paket dafür, dass die Werte der Schlüssel in einem Objekt vertikal zueinander ausgerichtet sind, was einen ansprechenden Tabelleneffekt erzeugt. Damit dies funktioniert, muss auch **indented** gesetzt sein.

Ist dieses Flag nicht gesetzt, wird die Ausgabe entsprechend dem Wert von **indented** formatiert, ohne dass versucht wird, die Werte der Objektschlüssel zu richten.

::json::write string *s*
Diese Methode nimmt die Zeichenkette *s* entgegen und gibt sie als Ergebnis mit den für JSON korrekten Anführungszeichen zurück.

::json::write array *arg*...
Diese Methode nimmt eine Reihe von Argumenten im JSON-Format entgegen und gibt sie als ordnungsgemäß formatiertes JSON-Array zurück.

::json::write array-strings *arg*...
Dies ist eine Komfortvariante von **array**. Sie geht davon aus, dass alle Argumente einfache Zeichenketten sind, und formatiert sie als JSON-Zeichenketten, bevor sie an **array** übergeben werden.

::json::write object *key value*...
Diese Methode nimmt eine Reihe von Schlüssel-Wert-Argumenten entgegen, wobei die Werte bereits im JSON-Format vorliegen, und gibt sie als ordnungsgemäß formatiertes JSON-Objekt zurück, wobei die Schlüssel als JSON-Zeichenketten formatiert sind.

::json::write object-strings *key value*...
Dies ist eine Komfortvariante von **object**. Sie geht davon aus, dass alle *Wert*-Argumente einfache Zeichenketten sind, und formatiert sie als JSON-Zeichenketten, bevor sie an **object** übergeben werden.

Verwandte Themen
Um JSON zu parsen, anstatt es zu schreiben, siehe das Paket **json**.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *json* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
CGI-Programmierung

Copyright

```
Copyright (c) 2009-2013,2022 Andreas Kupries
    <andreas_kupries@sourceforge.net>
```


lambda

lambda
Stichwörter: lambda, anonymous procedure, callback, command prefix, currying, partial application, proc

Name
lambda – Hilfskommandos für anonyme Prozeduren

Übersicht

package require Tcl 8.5 9
package require lambda ?**1.1**?
::lambda *arguments body* ?*arg* ...?
::lambda@ *namespace arguments body* ?*arg* ...?

Beschreibung

Dieses Paket bietet zwei Komfortbefehle, die die Erstellung anonymer Befehle, d. h. Lambda-Ausdrücke, **proc**-like. Anstatt beispielsweise zu schreiben

```
set f {::apply {{x} {
   ....
}}}
```

mit seiner tiefen Verschachtelung von Klammern oder

```
set f [list ::apply {{x y} {
   ....
}} $value_for_x]
```

Um mit einem **list**-Befehl einige der Argumente einer partiellen Anwendung einzufügen, schreibe einfach

```
set f [lambda {x} {
   ....
}]
```

or

```
set f [lambda {x y} {
   ....
} $value_for_x]
```

Lambdas sind besonders nützlich für Befehle, die die Option **-command** akzeptieren.

API

::lambda *arguments body ?arg ...?*

Gibt einen anonymen Befehl zurück, der auf der Liste *arguments*, dem Skript *body* und etwaigen (optionalen) vordefinierten Argumentwerten *arg ...* basiert.

Bei der Ausführung wird der *body* in einem neuen Bereich der Prozedur direkt unterhalb des globalen Bereichs ausgeführt, wobei die Argumente auf die Werte gesetzt werden, die sowohl bei der Erstellung (alle *arg ...*-Argumente) als auch zum Zeitpunkt der Ausführung übergeben wurden.

::lambda@ *namespace arguments body ?arg ...?*

Gibt einen anonymen Befehl im Namespace *namespace* zurück, der auf der Liste *arguments*, dem Skript *body* und etwaigen (optionalen) vordefinierten Argumentwerten *arg ...* basiert.

Bei der Ausführung wird der *body* in einem neuen Prozedurbereich im Namespace *namespace* ausgeführt, wobei die Argumente auf die Werte gesetzt werden, die sowohl bei der Erstellung (alle *arg ...*-Argumente) als auch zum Zeitpunkt der Ausführung übergeben wurden.

Beispiele

Partielle Funktionsanwendung

Lambdas können für die partielle Funktionsanwendung (PFA) verwendet werden, obwohl dies aufgrund der **expr**-Kontexte in Tcl meist nicht erforderlich ist. Dennoch hier ein klassisches PFA-Beispiel:

```
proc make_adder addend { lambda {x y} { expr { $x + $y } } $addend }
set add5 [make_adder 5]
puts "[{*}$add5 92] [{*}$add5 17]"
=>
97 22
```

Mit dem Befehl **make_adder** wurde ein anonymer Befehl, d. h. ein Lambda, erstellt, der anschließend der Variable **add5** zugewiesen wurde. Um eine Variable als Befehl aufzurufen, muss ihr der Expansionsoperator **{*}** ("Splat") vorangestellt werden. Beachten Sie, dass diese Aufrufe nur *ein* Argument benötigen; das erste Argument wurde bei der Erstellung des Lambda festgelegt.

Verwendung von "regusb" mit "lambda"

Dieses Beispiel zeigt, wie Sie ein Lambda mit der Option **-command** des integrierten Befehls regsub verwenden.

```
# Simple URL-decoding: + → space; %XX → char
set encoded_url "www.tcl-lang.org%2FSet%20or%20%24%20usage%3F"
set url [regsub -all -command {(\+)|%([0-9A-Fa-f]{2})}  $encoded_url
    [lambda {_ plus hex} {
                if {$plus eq "+"} { return " " }
                scan $hex %x charCode
                format %c $charCode
            }]]
puts $url
=>
www.tcl-lang.org/Set or $ usage?
```

Beachten Sie, dass bei Verwendung der Option **-command** des integrierten Befehls regsub der Befehl selbst *hinter* der zu suchenden Zeichenfolge steht (an der Position des Arguments *subSpec*).

Verwendung von "lsort" mit Lambda

Dieses Beispiel zeigt, wie Sie mithilfe von Lambda-Ausdrücken Objekte sortieren, indem Sie die Option **-command** des integrierten Befehls lsort verwenden.

Zunächst eine minimale Definition einer Klasse **Id**, deren Objekte sortiert werden sollen.

```
oo::class create Id {
    variable Id
    variable Name
    constructor {id name} { set Id $id ; set Name $name }
    method id {} { return $Id }
```

```
    method name {} { return $Name }
    method to_string {} { return "$Id'$Name'" }
}
```

Anschließend wird eine Liste von **Id**s erstellt und dann mithilfe von Lambda-Ausdrücken auf zwei verschiedene Arten sortiert, um die gewünschten Sortierreihenfolgen zu erhalten.

```
set ids [list [Id new 3 Jane] [Id new 1 Bill] [Id new 4 Sam] [Id new 5
    Nell]]
#
puts -nonewline "Original order:     "
foreach id $ids { puts -nonewline "[$id to_string] " }
#
puts -nonewline "\nName,Id order:      "
foreach id [lsort -command [lambda {a b} {
        if {[set i [string compare [$a name] [$b name]]]} { return $i }
        if {[$a id] < [$b id]} { return -1 }
        if {[$a id] > [$b id]} { return 1 }
        return 0}
            ] $ids] {
    puts -nonewline "[$id to_string] "
}
#
puts -nonewline "\nId desc,Name order: "
foreach id [lsort -command [lambda {a b} {
        if {[$a id] < [$b id]} { return 1 }
        if {[$a id] > [$b id]} { return -1 }
        return [string compare [$a name] [$b name]]}
            ] $ids] {
    puts -nonewline "[$id to_string] "
}
puts ""
=>
Original order:     3'Jane' 1'Bill' 4'Sam' 5'Nell'
Name,Id order:      1'Bill' 3'Jane' 5'Nell' 4'Sam'
Id desc,Name order: 5'Nell' 4'Sam' 3'Jane' 1'Bill'
```

Beachten Sie, dass bei Verwendung der Option **-command** des integrierten Befehls lsort der Befehl selbst *vor* der zu sortierenden Liste steht.

Autoren

Andreas Kupries

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *lambda* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hilfsprogramme

Copyright

```
Copyright (c) 2011 Andreas Kupries, BSD licensed
```


lazyset

lazyset
Stichwörter: lazyset

Name
lazyset – Lazy Auswertung

Übersicht
package require Tcl 8.5 9
package require lazyset ?**1.1**?
::lazyset::variable ?*-array boolean*? ?*-appendArgs boolean*? *someVar commandPrefix*

Beschreibung
Das Paket "**lazyset**" bietet eine Möglichkeit, die Ausführung von Code so lange aufzuschieben, bis auf eine bestimmte Variable oder einen beliebigen Index eines Arrays verwiesen wird.

(Eine andere Art, die Ausführung aufzuschieben, ist eine Singleton-Klasse. Beim ersten Aufruf ihres Konstruktors, falls er überhaupt erfolgt, wird das Objekt erzeugt. Jeder weitere Konstruktoraufruf liefert das zuerst erzeugte Objekt zurück. Siehe oo::singleton.)

API
::lazyset::variable ?*-array boolean*? ?*-appendArgs boolean*? *someVar commandPrefix*

Sorgt dafür, dass der als *commandPrefix* angegebene Code ausgeführt wird, wenn die Variable, deren Name durch *someVar* festgelegt wird, zum ersten Mal gelesen wird.

Wenn das optionale Argument *-array boolean* auf "true" gelegt wird, wird die als *someVar* angegebene Variable als Array behandelt, und jeder Versuch, einen beliebigen Index des Arrays auszulesen, bewirkt, dass dieser Index beim Auslesen durch *commandPrefix* gesetzt wird.

Wenn das optionale Argument *-appendArgs boolean* als "false" festgelegt wird, werden der Name der Variable und die Unternamen nicht an das *commandPrefix* angehängt, bevor die Auswertung durchgeführt wird.

Wenn das Argument *-appendArgs boolean* nicht angegeben wird oder auf "true" gelegt wird,

werden dem *commandPrefix* 1 oder 2 zusätzliche Argumente angehängt. Wenn *-array boolean* als "true" festgelegt wird, werden 2 Argumente angehängt, die dem Namen der Variablen und dem Index entsprechen; andernfalls wird 1 Argument angehängt, das den Namen der Variablen enthält.

Der *commandPrefix*-Code wird im selben Geltungsbereich ausgeführt wie die Variable, die ausgelesen wird.

Beispiele

```
::lazyset::variable page {apply {{name} {
    package require http
    set token [http::geturl http://www.tcl-lang.org/]
    set data [http::data $token]
    return $data
}}}

puts $page
```

```
::lazyset::variable -array true page {apply {{name index} {
    package require http
    set token [http::geturl $index]
    set data [http::data $token]
    return $data
}}}

puts $page(http://www.tcl-lang.org/)
```

```
::lazyset::variable -appendArgs false simple {
    return -level 0 42
}

puts $simple
```

Autoren

Roy Keane

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *utility* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hilfsprogramme

Copyright

```
Copyright (c) 2018 Roy Keene
```

ldap

ldap

Stichwörter: ldap, directory access, internet, ldap client, protocol, rfc 2251, rfc 4511, x.500

Name

ldap – LDAP-Client

Übersicht

package require Tcl 8.5 9
package require ldap ?**1.10.2**?
::ldap::connect *host* ?*port*?
::ldap::tlsoptions reset
::ldap::tlsoptions ?*opt1 val1*? ?*opt2 val2*? ...
::ldap::secure_connect *host* ?*port*?
::ldap::secure_connect *host* ?*port*? ?*verify_cert*? ?*sni_servername*?
::ldap::disconnect *handle*
::ldap::starttls *handle*
::ldap::starttls *handle* ?*cafile*? ?*certfile*? ?*keyfile*? ?*verify_cert*? ?*sni_servername*?
::ldap::bind *handle* ?*name*? ?*password*?
::ldap::bindSASL *handle* ?*name*? ?*password*?
::ldap::unbind *handle*
::ldap::search *handle baseObject filterString attributes options*
::ldap::searchInit *handle baseObject filterString attributes options*
::ldap::searchNext *handle*
::ldap::searchEnd *handle*
::ldap::modify *handle dn attrValToReplace* ?*attrToDelete*? ?*attrValToAdd*?
::ldap::modifyMulti *handle dn attrValToReplace* ?*attrValToDelete*? ?*attrValToAdd*?
::ldap::add *handle dn attrValueTuples*
::ldap::addMulti *handle dn attrValueTuples*
::ldap::delete *handle dn*
::ldap::modifyDN *handle dn newrdn* ?*deleteOld*? ?*newSuperior*?
::ldap::info ip *handle*
::ldap::info bound *handle*
::ldap::info bounduser *handle*
::ldap::info connections
::ldap::info tls *handle*
::ldap::info tlsstatus *handle*
::ldap::info saslmechanisms *handle*
::ldap::info control *handle*
::ldap::info extensions *extensions*

::ldap::info whoami *handle*

Beschreibung

Das Paket **ldap** stellt eine ausschließlich auf Tcl basierende Client-Bibliothek für das LDAPv3-Protokoll bereit, wie es in RFC 4511 (http://www.rfc-editor.org/rfc/rfc4511.txt) festgelegt ist. Es funktioniert, indem es den Standard- (oder sicheren) LDAP-Socket auf dem Server öffnet und anschließend eine Tcl-API für den Zugriff auf die LDAP-Protokollbefehle bereitstellt. Alle Serverfehler werden als Tcl-Fehler zurückgegeben (ausgelöst), die mit dem Tcl-Befehl **catch** abgefangen werden müssen.

TLS-Sicherheitshinweise

Dieses Paket nutzt das Paket **TLS**, um die Sicherheit für **LDAPS**-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    ldap::tlsoptions -tls1 1 -ssl2 0 -ssl3 0 ;# forcibly activate
    support for the TLS1 protocol

    ... your own application code ...
```

Befehle

::ldap::connect ***host*** **?*port*?**

Stellt eine LDAPv3-Verbindung zum angegebenen *host* am angegebenen *port* her und gibt ein Token für die Verbindung zurück. Dieses Token ist das Argument *handle* für alle anderen Befehle. Wird kein *port* angegeben, wird standardmäßig **389** verwendet.

Der Befehl blockiert, bis die Verbindung hergestellt wurde oder der Verbindungsaufbau definitiv fehlgeschlagen ist.

::ldap::tlsoptions reset

Dieser Befehl setzt die TLS-Optionen auf die Standardwerte zurück. Er gibt die Menge der Optionen zurück. Die Verwendung dieses Befehls ist nicht kompatibel mit der obsoleten Form von **::ldap::secure_connect** und **::ldap_starttls**.

::ldap::tlsoptions ?*opt1 val1*? ?*opt2 val2*? ...

Dieser Befehl setzt eine oder mehrere Optionen auf einen Wert und kann mehrmals verwendet werden, um Optionen in mehreren Schritten hinzuzufügen. Eine vollständige Beschreibung der Optionen finden Sie in der Dokumentation zum Paket **tls**. Gültige Optionen und Werte sind:

-cadir directory

Gibt das Verzeichnis mit den CA-Zertifikaten an. Kein Standardwert.

-cafile file
Gibt die CA-Datei an. Kein Standardwert.

-cipher string
Gibt die zu verwendenden Cipher-Suites an. Kein Standardwert.

-dhparams file
Gibt eine Datei mit Diffie-Hellman-Parametern an. Kein Standardwert.

-request boolean
Fordert während des SSL-Handshakes ein Zertifikat von der Gegenstelle an. Standard: true.

-require boolean
Verlangt während des SSL-Handshakes ein gültiges Zertifikat von der Gegenstelle. Ist diese Option auf true gesetzt, muss auch -request auf true gesetzt sein. Standard: false

-servername host
Nur verfügbar, wenn die OpenSSL-Bibliothek, gegen die das TLS-Paket gelinkt ist, die TLS-Hostnamen-Erweiterung für 'Server Name Indication' (SNI) unterstützt. Benennt den logischen Host, mit dem kommuniziert wird und für den ein Zertifikat erwartet wird. Kein Standardwert.

-ssl2 bool
Verwendung von SSL v2 aktivieren. Standard: false

-ssl3 bool
Verwendung von SSL v3 aktivieren. Standard: false

-tls1 bool
Verwendung von TLS v1 aktivieren. Standard: true

-tls1.1 bool
Verwendung von TLS v1.1 aktivieren. Standard: true

-tls1.2 bool
Verwendung von TLS v1.2 aktivieren. Standard: true

Dieser Befehl gibt die aktuell eingestellten TLS-Optionen und -Werte zurück. Insbesondere können Sie diesen Befehl ohne Argumente verwenden, um die aktuell eingestellten Optionen abzurufen.

Die Verwendung dieses Befehls ist nicht kompatibel mit der obsoleten Form von **::ldap::secure_connect** und **::ldap_starttls** (siehe unten).

::ldap::secure_connect ***host*** **?*port*?**
Ähnlich wie **::ldap::connect**, mit dem Unterschied, dass die hergestellte Verbindung durch SSL gesichert ist. Der Port ist standardmäßig **636**. Dieser Befehl setzt die Verfügbarkeit des Pakets **TLS** voraus, einer SSL-Anbindung für Tcl. Ist **TLS** nicht verfügbar, schlägt dieser Befehl fehl.

TLS-Optionen werden mit **::ldap::tlsoptions** festgelegt.

Der Befehl blockiert, bis die Verbindung hergestellt wurde oder der Verbindungsaufbau definitiv fehlgeschlagen ist.

::ldap::secure_connect ***host*** **?*port*? ?*verify_cert*? ?*sni_servername*?**

Hinweis: Diese Form des Befehls ist veraltet, da TLS-Optionen mit einer Kombination aus Parametern dieses Befehls (*verify_cert* und *sni_servername*) und Argumenten für **::tls::init** (aus dem Paket **tls**) festgelegt werden mussten, etwa um Standardwerte für vertrauenswürdige Zertifikate festzulegen. Verwenden Sie vorzugsweise die oben genannte Form (ohne die Parameter *verify_cert* und *sni_servername*) und legen Sie die TLS-Optionen mit **::ldap::tlsoptions** fest.

Wenn *verify_cert* auf 1 gesetzt ist (Standardeinstellung), wird das Serverzertifikat anhand der bekannten Hosts überprüft. Wenn *sni_servername* gesetzt ist, wird der angegebene Hostname als Hostname für die Server Name Indication im TLS-Handshake verwendet.

Verwenden Sie **::tls::init**, um Standardwerte für vertrauenswürdige Zertifikate festzulegen.

TLS unterstützt verschiedene Protokollstufen. Gängig sind die Versionen 1.0, 1.1 und 1.2. Standardmäßig werden alle diese Versionen angeboten. Wenn Sie die zulässigen Protokolle anpassen müssen, können Sie die Liste ::ldap::tlsProtocols ändern (veraltet).

::ldap::disconnect *handle*
Schließt die LDAP-Verbindung, auf die sich das Token *handle* bezieht. Gibt als Ergebnis die leere Zeichenkette zurück.

::ldap::starttls *handle*
Startet die TLS-Aushandlung für die durch *handle* bezeichnete Verbindung, mit den TLS-Parametern, die mit **::ldap::tlsoptions** festgelegt wurden.

::ldap::starttls *handle* ?*cafile*? ?*certfile*? ?*keyfile*? ?*verify_cert*? ?*sni_servername*?
Hinweis: Diese Form des Befehls ist veraltet, da TLS-Optionen mit einer Kombination aus Parametern dieses Befehls (*cafile*, *certfile*, *keyfile*, *verify_cert* und *sni_servername*) und Argumenten für **::tls::init** (aus dem Paket **tls**) festgelegt werden mussten. Verwenden Sie vorzugsweise die oben genannte Form (ohne spezifische TLS-Argumente) und legen Sie die TLS-Optionen mit **::ldap::tlsoptions** fest.

Startet die TLS-Aushandlung für die durch *handle* bezeichnete Verbindung. Sie müssen mindestens das Argument *cafile* auf eine Datei mit vertrauenswürdigen Zertifikaten setzen, wenn *verify_cert* den Wert 1 hat, was der Standard ist. Mit *sni_servername* kann während des TLS-Handshakes ein anderer Hostname signalisiert werden. Die angekündigten Protokolle werden auf dieselbe Weise wie bei **::ldap::secure_connect** festgelegt. Sie können ein TLS-Client-Zertifikat mit den Optionen *certfile* und *keyfile* angeben.

::ldap::bind *handle* ?*name*? ?*password*?
Dieser Befehl authentifiziert die LDAP-Verbindung, auf die das Token in *handle* verweist, mit einem Benutzernamen und dem zugehörigen Passwort. Er blockiert, bis eine Antwort vom LDAP-Server eintrifft. Das Ergebnis ist die leere Zeichenkette. Sowohl *name* als auch *passwd* sind standardmäßig die leere Zeichenkette, wenn sie nicht angegeben werden. Indem Sie *name* und *passwd* weglassen, können Sie einen anonymen bind beim LDAP-Server durchführen. Sie können **::ldap::bind** erneut aufrufen, um sich mit anderen Anmeldedaten anzumelden.

::ldap::bindSASL *handle* ?*name*? ?*password*?
Dieser Befehl nutzt SASL-Authentifizierungsmechanismen, um einen mehrstufigen bind durchzuführen. Ansonsten entspricht er dem normalen **::ldap::bind**. Diese Funktion ist

derzeit experimentell und kann sich ändern. Einzelheiten zur Einrichtung und Verwendung von SASL mit OpenLDAP finden Sie in der Dokumentation zu **SASL** sowie in der Datei "*SASL.txt*" im tcllib-CVS-Repository.

::ldap::unbind *handle*

Dieser Befehl bittet den LDAP-Server, den letzten bind für die Verbindung freizugeben, auf die sich das Token in *handle* bezieht. Das *handle* ist nach dem unbind ungültig, da der Server die Verbindung schließt. Im Grunde ist dies also nur eine höflichere Form von disconnect.

::ldap::search *handle baseObject filterString attributes options*

Dieser Befehl führt unterhalb des *baseObject*-Baums eine LDAP-Suche mit einem komplexen LDAP-Suchausdruck *filterString* durch und gibt die angegebenen *attributes* aller passenden Objekte (DNs) zurück. Ist die Liste *attributes* leer, werden alle Attribute zurückgegeben. Der Befehl blockiert, bis er alle Ergebnisse empfangen hat. Die gültigen *options* entsprechen den für **::ldap::searchInit** aufgeführten Optionen.

Ein Beispiel für einen Suchausdruck lautet

```
    set filterString "|(cn=Linus*)(sn=Torvalds*)"
```

Der Rückgabewert des Befehls ist eine Liste verschachtelter Dictionaries. Die Schlüssel der ersten Ebene sind Objektbezeichner (DNs), die der zweiten Ebene Attributnamen. Mit anderen Worten, er hat folgende Form:

```
    {dn1 {attr1 {val11 val12 ...} attr2 {val21...} ...}} {dn2 {a1
    {v11 ...} ...}} ...
```

::ldap::searchInit *handle baseObject filterString attributes options*

Dieser Befehl leitet eine LDAP-Suche unterhalb des *baseObject*-Baums mit einem komplexen LDAP-Suchausdruck *filterString* ein. Die Suche holt die angegebenen *attributes* aller passenden Objekte (DNs). Der Befehl selbst startet lediglich die Suche; um die eigentlichen Ergebnisse abzurufen, verwenden Sie **::ldap::searchNext**. Eine Suche kann jederzeit mit **::ldap::searchEnd** abgebrochen werden. Dadurch wird dem Server durch das Senden einer ABANDON-Nachricht mitgeteilt, dass keine weiteren Ergebnisse gesendet werden sollen, und der interne Zustand der Suche wird bereinigt. Es kann zu einem bestimmten Zeitpunkt nur ein **::ldap::search** aktiv sein; dies schließt die Introspektionsbefehle **::ldap::info saslmechanisms**, **ldap::info control** und **ldap::info extensions** ein, die intern eine Suche ausführen. Fehlerantworten des Servers aufgrund falscher Argumente oder ähnlicher Ursachen werden mit dem ersten Aufruf von **::ldap::searchNext** zurückgegeben und sollten dort geprüft und behandelt werden. Ist die Liste der angeforderten *attributes* leer, werden alle Attribute zurückgegeben. Der Parameter *options* legt die bei der Suche zu verwendenden Optionen fest und hat das folgende Format:

```
    {-option1 value1 -option2 value2 ... }
```

Folgende Optionen stehen zur Verfügung:

-scope base one sub

Legt den Suchbereich auf **base**, **one** oder **sub** fest, für eine Suche nur im Basisobjekt, auf einer Ebene oder im ganzen Teilbaum. Standard ist **sub**.

-derefaliases never search find always

Legt fest, wie Aliase dereferenziert werden. Der Wert sollte **never**, **always**, **search** oder **find** sein, um festzulegen, dass Aliase nie dereferenziert werden, immer, beim Suchen, oder nur beim Auffinden des Basisobjekts der Suche. Standardmäßig werden Aliase nie dereferenziert.

-sizelimit num
Legt die maximale Anzahl der bei einer Suche zurückzugebenden Einträge fest. Wenn 0 angegeben wird, wird keine Begrenzung angewendet. Der Server kann eine konfigurationsabhängige Größenbeschränkung erzwingen, die unter dem mit dieser Option angegebenen Wert liegen kann. Der Standardwert ist 0, d. h. keine Begrenzung.

-timelimit seconds
Bittet den Server, für die Suche ein Zeitlimit von *seconds* Sekunden zu verwenden. Der Wert Null bedeutet, dass kein Limit gilt. Der Standardwert ist 0, d. h. kein Limit.

-attrsonly boolean
Ist der Wert 1, enthält das Suchergebnis nur die Attributnamen, nicht die Werte. Standardmäßig werden sowohl Attributnamen als auch Werte abgerufen.

-referencevar varname
Ist diese Option gesetzt, werden die LDAP-URIs der Suchergebnis-Verweise (search result references), sofern vorhanden, in der angegebenen Variable zurückgegeben. Der Aufrufer kann dann entscheiden, diesen Verweisen zu folgen und weitere LDAP-Server nach zusätzlichen Ergebnissen abzufragen.

::ldap::searchNext *handle*
Dieser Befehl gibt den nächsten Eintrag aus einer von **::ldap::searchInit** initiierten LDAP-Suche zurück. Er kehrt erst zurück, wenn ein neues Ergebnis empfangen wurde oder keine weiteren Ergebnisse mehr verfügbar sind, sorgt jedoch dafür, dass die Ereignisschleife aktiv bleibt. Der zurückgegebene Eintrag ist eine Liste mit zwei Elementen: Das erste ist der DN des Eintrags, das zweite ist die Liste der Attribute und Werte im folgenden Format:

```
    dn {attr1 {val11 val12 ...} attr2 {val21...} ...}
```

Der Befehl **::ldap::searchNext** gibt am Ende der Suche eine leere Liste zurück.

::ldap::searchEnd *handle*
Dieser Befehl beendet eine von **::ldap::searchInit** initiierte LDAP-Suche. Zudem wird der interne Zustand bereinigt, sodass eine neue Suche gestartet werden kann. Wenn der Client noch nicht alle Ergebnisse erhalten hat, sendet er eine ABANDON-Nachricht, um dem Server mitzuteilen, dass keine weiteren Ergebnisse für die vorherige Suche gesendet werden sollen.

::ldap::modify *handle dn attrValToReplace ?attrToDelete? ?attrValToAdd?*
Dieser Befehl ändert das Objekt *dn* auf dem LDAP-Server, mit dem wir über *handle* verbunden sind. Er ersetzt Attribute durch neue Werte, löscht Attribute und fügt neue Attribute mit neuen Werten hinzu. Alle Argumente sind Dictionaries, die Attributnamen auf Werte abbilden. Die optionalen Argumente sind standardmäßig das leere Dictionary, was bedeutet, dass weder Attribute gelöscht noch hinzugefügt werden.

dictionary *attrValToReplace* (in)
Wenn dieses Argument leer ist, werden keine Attribute geändert. Das Dictionary enthält die neuen Attribute und deren Werte. Diese *ersetzen alle* dem Objekt bekannten Attribute.

dictionary *attrToDelete* (in)

Wenn dieses Argument leer ist, werden keine Attribute gelöscht. Die Dictionary-Werte stellen Einschränkungen für die Löschung dar. Ein hier aufgeführtes Attribut wird genau dann gelöscht, wenn sein aktueller Wert auf dem Server mit dem im Dictionary angegebenen Wert übereinstimmt oder wenn der Wert im Dictionary die leere Zeichenkette ist.

dictionary *attrValToAdd* (in)
Wenn dieses Argument leer ist, werden keine Attribute hinzugefügt. Die Dictionary-Werte sind die Werte für die neuen Attribute.

Der Befehl blockiert, bis alle Änderungen abgeschlossen sind. Das Ergebnis ist die leere Zeichenkette.

::ldap::modifyMulti *handle dn attrValToReplace ?attrValToDelete? ?attrValToAdd?*
Dieser Befehl ändert das Objekt *dn* auf dem LDAP-Server, mit dem wir über *handle* verbunden sind. Er ersetzt Attribute durch neue Werte, löscht Attribute und fügt neue Attribute mit neuen Werten hinzu. Alle Argumente sind Listen im folgenden Format:

```
    attr1 {val11 val12 ...} attr2 {val21...} ...
```

wobei jede Werteliste leer sein kann, um alle Attribute zu löschen. Die optionalen Argumente sind standardmäßig leere Listen der zu löschenden und hinzuzufügenden Attribute.

list *attrValToReplace* (in)
Wenn dieses Argument leer ist, werden keine Attribute geändert. Das Dictionary enthält die neuen Attribute und deren Werte. Diese *ersetzen alle* dem Objekt bekannten Attribute.

list *attrValToDelete* (in)
Wenn dieses Argument leer ist, werden keine Attribute gelöscht. Wird kein Wert angegeben, wird die gesamte Wertemenge eines Attributs gelöscht.

list *attrValToAdd* (in)
Wenn dieses Argument leer ist, werden keine Attribute hinzugefügt.

Der Befehl blockiert, bis alle Änderungen abgeschlossen sind. Das Ergebnis ist die leere Zeichenkette.

::ldap::add *handle dn attrValueTuples*
Dieser Befehl erstellt ein neues Objekt unter Verwendung des angegebenen *dn*. Die Attribute des neuen Objekts werden auf die Werte in der Liste *attrValueTuples* gesetzt. Mehrwertige Attribute können mit mehreren Tupeln angegeben werden. Der Befehl blockiert, bis der Vorgang abgeschlossen ist. Das Ergebnis ist die leere Zeichenkette.

::ldap::addMulti *handle dn attrValueTuples*
Dieser Befehl ist der bevorzugte, um ein neues Objekt mit dem angegebenen *dn* zu erstellen. Die Attribute des neuen Objekts werden auf die Werte im Dictionary *attrValueTuples* gesetzt (dessen Schlüssel die Attributnamen sind). Jedes Tupel ist eine Liste mit mehreren Werten. Der Befehl blockiert, bis der Vorgang abgeschlossen ist. Das Ergebnis ist die leere Zeichenkette.

::ldap::delete *handle dn*
Dieser Befehl entfernt das durch *dn* festgelegte Objekt sowie alle seine Attribute vom Server. Der Befehl blockiert, bis der Vorgang abgeschlossen ist. Das Ergebnis ist die leere Zeichenkette.

::ldap::modifyDN *handle dn newrdn* ?*deleteOld*? ?*newSuperior*?
Dieser Befehl verschiebt oder kopiert das durch *dn* angegebene Objekt an eine neue Stelle im Objektbaum. Diese Stelle wird durch *newrdn*, eine *relative* Angabe, oder durch *newrdn* und *newSuperior*, eine *absolute* Angabe, festgelegt. Das optionale Argument *deleteOld* ist standardmäßig **true**, d. h. es handelt sich um ein Verschieben. Ist *deleteOld* nicht gesetzt, erstellt die Operation eine Kopie von *dn* an der neuen Stelle. Das optionale Argument *newSuperior* ist standardmäßig eine leere Zeichenkette, was bedeutet, dass das Objekt nicht in einen anderen Zweig des Baums verschoben werden darf. Wird dieses Argument angegeben, muss auch das Argument *deleteOld* angegeben werden. Der Befehl blockiert, bis der Vorgang abgeschlossen ist. Das Ergebnis ist die leere Zeichenkette.

::ldap::info ip *handle*
Dieser Befehl gibt die IP-Adresse des entfernten LDAP-Servers zurück, mit dem das Handle verbunden ist.

::ldap::info bound *handle*
Dieser Befehl gibt den Wert 1 zurück, wenn ein Handle erfolgreich ein **::ldap::bind** abgeschlossen hat. Wurde kein bind durchgeführt oder ist er fehlgeschlagen, wird der Wert 0 zurückgegeben.

::ldap::info bounduser *handle*
Dieser Befehl gibt den Benutzernamen zurück, der bei der bind-Operation verwendet wurde, sofern ein Handle erfolgreich ein **::ldap::bind** abgeschlossen hat. Wurde kein bind durchgeführt oder ist dieser fehlgeschlagen, wird eine leere Zeichenkette zurückgegeben.

::ldap::info connections
Dieser Befehl gibt alle derzeit vorhandenen LDAP-Verbindungs-Handles zurück.

::ldap::info tls *handle*
Dieser Befehl gibt den Wert 1 zurück, wenn die LDAP-Verbindung *handle* TLS/SSL über **ldap::secure_connect** verwendet oder **ldap::starttls** abgeschlossen hat, sonst 0.

::ldap::info tlsstatus *handle*
Dieser Befehl gibt den aktuellen Sicherheitsstatus eines TLS-gesicherten Kanals zurück. Das Ergebnis ist eine Liste von Schlüssel-Wert-Paaren, die die verbundene Gegenstelle beschreiben (Informationen zu den zurückgegebenen Werten finden Sie in der Dokumentation zum Paket **TLS**). Wenn die Verbindung nicht mit TLS gesichert ist, wird eine leere Liste zurückgegeben.

::ldap::info saslmechanisms *handle*
Gibt die vom Server angekündigten, unterstützten SASL-Mechanismen zurück. Nur gültig nach einem bind (anonym oder nicht).

::ldap::info control *handle*
Gibt die vom Server angekündigten, unterstützten Controls als Liste von OIDs zurück. Nur gültig nach einem bind. Dies ist derzeit experimentell und kann sich ändern.

::ldap::info extensions *extensions*
Gibt die unterstützten LDAP-Erweiterungen als Liste von OIDs zurück. Nur gültig nach einem bind. Dies ist derzeit experimentell und kann sich ändern.

::ldap::info whoami *handle*
Gibt die authzId für die aktuelle Verbindung zurück. Hiermit wird die Protokollerweiterung gemäß RFC 4532 implementiert.

Beispiele

Ein kleines Beispiel, das aus der mit diesem Code gelieferten Testanwendung entnommen wurde.

```
    package require ldap

    # Connect, bind, add a new object, modify it in various ways

    set handle [ldap::connect localhost 9009]

    set dn "cn=Manager, o=University of Michigan, c=US"
    set pw secret

    ldap::bind $handle $dn $pw

    set dn "cn=Test User,ou=People,o=University of Michigan,c=US"

    ldap::add $handle $dn {
    objectClass     OpenLDAPperson
    cn              {Test User}
    mail            test.user@google.com
    uid             testuid
    sn              User
    telephoneNumber +31415926535
    telephoneNumber +27182818285
    }

    set dn "cn=Another User,ou=People,o=University of Michigan,c=US"

    ldap::addMulti $handle $dn {
    objectClass     {OpenLDAPperson}
    cn              {{Anotther User}}
    mail            {test.user@google.com}
    uid             {testuid}
    sn              {User}
    telephoneNumber {+31415926535 +27182818285}
    }

    # Replace all attributes
    ldap::modify $handle $dn [list drink icetea uid JOLO]

    # Add some more
    ldap::modify $handle $dn {} {} [list drink water  drink
    orangeJuice pager "+1 313 555 7671"]

    # Delete
```

```
    ldap::modify $handle $dn {} [list drink water  pager ""]

    # Move
    ldap::modifyDN $handle $dn "cn=Tester"

    # Kill the test object, and shut the connection down.
    set dn "cn=Tester,ou=People,o=University of Michigan,c=US"
    ldap::delete $handle $dn

    ldap::unbind     $handle
    ldap::disconnect $handle
```

Und noch ein Beispiel: eine einfache Abfrage und die Verarbeitung der Ergebnisse.

```
    package require ldap
    set handle [ldap::connect ldap.acme.com 389]
    ldap::bind $handle
    set results [ldap::search $handle "o=acme,dc=com" "(uid=jdoe)" {}]
    foreach result $results {
    foreach {object attributes} $result break

    # The processing here is similar to what 'parray' does.
    # I.e. finding the longest attribute name and then
    # generating properly aligned output listing all attributes
    # and their values.

    set width 0
    set sortedAttribs {}
    foreach {type values} $attributes {
        if {[string length $type] > $width} {
        set width [string length $type]
        }
        lappend sortedAttribs [list $type $values]
    }

    puts "object='$object'"

    foreach sortedAttrib  $sortedAttribs {
        foreach {type values} $sortedAttrib break
        foreach value $values {
        regsub -all "\[\x01-\x1f\]" $value ? value
        puts [format "  %-${width}s %s" $type $value]
        }
    }
    puts ""
    }
    ldap::unbind $handle
    ldap::disconnect $handle
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ldap* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2004 Jochen Loewer <loewerj@web.de>
Copyright (c) 2006 Michael Schlenker <mic42@users.sourceforge.net>
```


ldapx

Stichwörter: ldapx, directory access, internet, ldap, ldap client, ldif, protocol, rfc 2251, rfc 2849

Name

ldapx – Erweiterte LDAP-Objektschnittstelle

Übersicht

package require Tcl 8.5 9
package require ldapx ?**1.3**?
e **reset**
e **dn** ?*newdn*?
e **rdn**
e **superior**
e **print**
se **isempty**
se **get** *attr*
se **get1** *attr*
se **set** *attr values*
se **set1** *attr value*
se **add** *attr values*
se **add1** *attr value*
se **del** *attr* ?*values*?
se **del1** *attr value*
se **getattr**
se **getall**
se **setall** *avpairs*
se **backup** ?*other*?
se **swap**
se **restore** ?*other*?
se **apply** *centry*
ce **change** ?*new*?
ce **diff** *new* ?*old*?
la **error** ?*newmsg*?
la **connect** *url* ?*binddn*? ?*bindpw*? ?*starttls*?
la **disconnect**
la **traverse** *base filter attrs entry body*
la **search** *base filter attrs*
la **read** *base filter entry ... entry*

la **commit** *entry ... entry*
li **channel** *chan*
li **error** ?*newmsg*?
li **read** *entry*
li **write** *entry*

Beschreibung

Das Paket **ldapx** bietet eine erweiterte Tcl-Schnittstelle zu LDAP-Verzeichnissen und LDIF-Dateien. Das Paket **ldapx** baut auf dem Paket **ldap** auf, um einen Zugriff auf LDAP auf niedriger Ebene zu ermöglichen.

Der LDAP-Zugriff ist kompatibel mit RFC 2251 (http://www.rfc-editor.org/rfc/rfc2251.txt). Der LDIF-Zugriff ist kompatibel mit RFC 2849 (http://www.rfc-editor.org/rfc/rfc2849.txt).

Überblick

Das Paket **ldapx** stellt Objekte zur Verfügung, die über eine benutzerfreundliche Programmierschnittstelle die Interaktion mit LDAP-Verzeichnissen und LDIF-Dateien ermöglichen. Es implementiert drei Klassen als **snit**::type.

Die erste Klasse, **entry**, dient zur Speicherung einzelner Einträge. Es stehen zwei verschiedene Formate zur Verfügung: Das erste ist das *Standard*-Format, das einen Eintrag so darstellt, wie er aus dem Verzeichnis gelesen wurde. Das zweite Format ist das *Änderungs*-Format, das die Unterschiede zwischen zwei Standardeinträgen speichert.

Bei diesen Einträgen muss eine Anwendung, die einen Eintrag in einem Verzeichnis ändern möchte, einen (Standard-)Eintrag aus dem Verzeichnis lesen, eine neue Kopie in einem neuen (Standard-)Eintrag erstellen, die neue Kopie ändern und anschließend die Unterschiede zwischen den beiden Einträgen in einem neuen (Änderungs-)Eintrag berechnen, der dann im Verzeichnis festgeschrieben werden kann.

Da solche Änderungen so häufig vorgenommen werden, können Standardeinträge eine eigene Kopie der Originaldaten enthalten. Bei Vorhandensein einer solchen Kopie liest die oben beschriebene Anwendung einen (Standard-)Eintrag aus dem Verzeichnis, sichert die Originaldaten, ändert den Eintrag und berechnet die Unterschiede zwischen dem Eintrag und seiner Sicherungskopie. Diese Unterschiede werden anschließend im Verzeichnis festgeschrieben.

Es werden Methoden bereitgestellt, um Differenzen zwischen zwei Einträgen zu berechnen, Differenzen auf einen Eintrag anzuwenden, um einen neuen Eintrag zu erhalten, sowie um Attribute in Standardeinträgen abzufragen oder zu setzen.

Die zweite Klasse ist die Klasse **ldap**. Sie stellt eine Methode **connect** bereit, die die Verbindung zum Verzeichnis herstellt und den bind durchführt, mit einheitlichem Zugriff auf LDAP und LDAPS über eine URL (ldap:// oder ldaps://). Die Kontrollstruktur **traverse** führt für jeden im Verzeichnis gefundenen Eintrag einen Rumpf aus. Die Methode **commit** wendet einige Änderungen (dargestellt als **entry**-Objekte) auf das Verzeichnis an. Da einige Attribute als UTF-8-Zeichenketten dargestellt werden, steuert die Option **-utf8**, welche Attribute konvertiert werden müssen und welche nicht.

Die letzte Klasse ist die Klasse **ldif**. Sie stellt eine Methode bereit, mit der ein Standard-Tcl-*Kanal* einem LDIF-Objekt zugeordnet werden kann. Anschließend lesen oder schreiben die Methoden **read** und **write** Einträge aus diesem Kanal bzw. in diesen Kanal. Diese Klasse kann Standard-

oder Änderungseinträge verwenden, je nach Typ der LDIF-Datei, die entweder Standardeinträge oder Änderungseinträge enthalten kann (jedoch nicht beides gleichzeitig). Die Option **-utf8** funktioniert genau wie bei der Klasse **ldap**.

Klasse Entry

Instanzdaten von Entry

Eine Instanz der Klasse **entry** enthält die folgenden Daten:

dn
: Dies ist der DN des Eintrags, der (in der LDAP-Terminologie) den RDN (relativer DN) und den Superior-Teil (übergeordneter Teil) umfasst.

format
: Das Format kann *uninitialized* (Eintrag noch nicht verwendet), *standard* oder *change* sein. Die meisten Methoden überprüfen das Format des Eintrags, das mit der Methode **reset** zurückgesetzt werden kann.

attrvals
: In einem *Standard*-Eintrag werden hier die Attribute und die zugehörigen Werte gespeichert. Viele Methoden ermöglichen den Zugriff auf diese Informationen. Attributnamen werden stets in Kleinbuchstaben umgewandelt.

backup
: In einem *Standard*-Eintrag kann die Sicherungskopie eine Kopie des DN sowie aller Attribute und Werte enthalten. Die Methoden **backup** und **restore** bearbeiten diese Daten, und die Methode **diff** kann diese Sicherungskopie verwenden.

change
: In einem *Änderungs*-Eintrag stellen diese Daten die Änderungen dar. Solche Änderungen werden durch spezielle Methoden wie **apply** oder **commit** verarbeitet. Das detaillierte Format sollte nicht direkt von Programmen verwendet werden.

  Intern werden Änderungen als Liste von Elementen dargestellt, wobei jedes Element eines der folgenden Formate aufweist (die den entsprechenden LDAP-Operationen entsprechen):

1. {**add** {attr1 {val1...valn} attr2 {...} ...}}

   Hinzufügen eines neuen Eintrags.
2. {**mod** {modop {attr1 ?val1...valn?} attr2 ...} {modop ...} ...}

   Änderung eines oder mehrerer Attribute und/oder Werte, wobei **modadd**, **moddel** oder **modrepl** sein kann (siehe die LDAP-Operation modify).
3. {**del**}

   Löschen eines alten Eintrags.
4. {**modrdn** newrdn deleteoldrdn ?newsuperior?}

   Umbenennung eines Eintrags.

Entry-Optionen

Diese Klasse definiert keine Option.

Methoden für alle Arten von Einträgen

***e* reset**

Diese Methode setzt den Eintrag in einen nicht initialisierten Zustand zurück.

***e* dn ?*newdn*?**
Diese Methode gibt den aktuellen DN des Eintrags zurück. Ist das optionale *newdn* angegeben, ersetzt es den aktuellen DN des Eintrags.

***e* rdn**
Diese Methode gibt den RDN-Teil des DN des Eintrags zurück.

***e* superior**
Diese Methode gibt den übergeordneten Teil des DN des Eintrags zurück.

***e* print**
Diese Methode gibt den Eintrag als druckfertige Zeichenkette zurück.

Methoden ausschließlich für Standardeinträge

Bei allen Methoden werden Attributnamen in Kleinbuchstaben umgewandelt.

***se* isempty**
Diese Methode gibt den Wert 1 zurück, wenn der Eintrag leer ist (d. h. keine Attribute enthält).

se* get *attr
Diese Methode gibt alle Werte des Attributs *attr* zurück oder eine leere Liste, wenn das Attribut nicht gefunden wird.

se* get1 *attr
Diese Methode gibt den ersten Wert des Attributs zurück.

se* set *attr values
Diese Methode legt die Werte (list *values*) des Attributs *attr* fest. Wenn die Liste leer ist, löscht diese Methode alle

se* set1 *attr value
Diese Methode setzt die Werte des Attributs *attr* auf den einzigen Wert *value*. Etwaige bisherige Werte werden durch den neuen Wert ersetzt.

se* add *attr values
Diese Methode fügt alle Elemente der list *values* den Werten des Attributs *attr* hinzu.

se* add1 *attr value
Diese Methode fügt dem Attribut *attr* einen einzelnen Wert hinzu, der durch den Parameter *value* vorgegeben wird.

***se* del *attr* ?*values*?**
Ist die optionale list *values* angegeben, löscht diese Methode alle angegebenen Werte aus dem Attribut *attr*. Ist das Argument *values* nicht angegeben, löscht diese Methode alle Werte.

se* del1 *attr value
Diese Methode löscht einen einzelnen Wert *value* aus dem Attribut *attr*.

***se* getattr**
Diese Methode gibt alle Attributnamen zurück.

***se* getall**
Diese Methode gibt alle Attribute und Werte des Eintrags zurück, zusammengefasst in einer Liste von Paaren.

se* setall *avpairs
Diese Methode legt alle Attribute und Werte auf einmal fest. Das Format des Arguments *avpairs* entspricht dem, das die Methode **getall** zurückgibt.

***se* backup ?*other*?**
Diese Methode speichert in einem anderen Standard-Eintragsobjekt *other* eine Kopie des aktuellen DN sowie der Attribute und Werte. Wenn das optionale Argument *other* nicht angegeben wird, erfolgt die Kopie im aktuellen Eintrag (an einer bestimmten Stelle, siehe Abschnitt Überblick).

***se* swap**
Diese Methode tauscht den aktuellen und den Sicherungskontext des Eintrags aus.

***se* restore ?*other*?**
Wenn das optionale Argument *other* angegeben wird, das dann ein *Standard*-Eintrag sein muss, stellt diese Methode den aktuellen Eintrag in den Eintrag *other* wieder her. Wenn das Argument *other* nicht angegeben wird, stellt diese Methode den aktuellen Eintrag aus seiner internen Sicherungskopie wieder her (siehe Abschnitt Überblick).

se* apply *centry
Diese Methode wendet die im Argument *centry* definierten Änderungen an, das ein *Änderungs*-Eintrag sein muss.

Methoden nur für Änderungseinträge

***ce* change ?*new*?**
Ist das optionale Argument *new* angegeben, ändert diese Methode die Änderungsliste (das genaue Format finden Sie im Unterabschnitt Instanzdaten von Entry). In beiden Fällen gibt sie die aktuelle Änderungsliste zurück. Warnung: Die von dieser Methode zurückgegebenen Werte sollten ausschließlich von speziellen Methoden wie **apply** oder **commit** verwendet werden.

***ce* diff *new* ?*old*?**
Diese Methode berechnet die Unterschiede zwischen den Einträgen *new* und *old* in Form einer Änderungsliste und speichert diese Liste im aktuellen *Änderungs*-Eintrag. Wenn das optionale Argument *old* nicht angegeben wird, wird der Unterschied anhand des Eintrags und seiner internen Sicherungskopie berechnet (siehe Abschnitt Überblick). Der Rückgabewert ist die berechnete Änderungsliste.

Beispiel für Entry

```
package require ldapx

#
# Create an entry and fill it as a standard entry with
# attributes and values
#
::ldapx::entry create e
e dn "uid=joe,ou=people,o=mycomp"
e set1 "uid"             "joe"
e set  "objectClass"     {person anotherObjectClass}
e set1 "givenName"       "Joe"
e set1 "sn"              "User"
e set  "telephoneNumber" {+31415926535 +2182818}
e set1 "anotherAttr"     "This is a beautiful day, isn't it?"
```

```
puts stdout "e\n[e print]"

#
# Create a second entry as a backup of the first, and
# make some changes on it.
# Entry is named automatically by snit.
#

set b [::ldapx::entry create %AUTO%]
e backup $b

puts stdout "$b\n[$b print]"

$b del  "anotherAttr"
$b del1 "objectClass" "anotherObjectClass"

#
# Create a change entry, a compute differences between first
# and second entry.
#

::ldapx::entry create c
c diff e $b

puts stdout "$c\n[$c print]"

#
# Apply changes to first entry. It should be the same as the
# second entry, now.
#

e apply c

::ldapx::entry create nc
nc diff e $b

puts stdout "nc\n[nc print]"

#
# Clean-up
#

e destroy
$b destroy
c destroy
```

```
    nc destroy
```

Klasse LDAP

LDAP-Instanzdaten

Eine Instanz der Klasse **ldap** enthält die folgenden Daten:

channel
: Dies ist der Kanal, den das Paket **ldap** für die Kommunikation mit dem LDAP-Server verwendet.

lastError
: Diese Variable enthält die Fehlermeldung, die in der letzten Methode der Klasse **ldap** aufgetreten ist (diese Zeichenkette wird in fast allen Methoden geändert). Mit der Methode **error** können Sie diese Meldung abrufen.

LDAP-Optionen

Die Optionen werden auf **ldap**-Instanzen mithilfe der Methode **configure** konfiguriert.

Die erste Option wird für TLS-Parameter verwendet:

-tlsoptions *list*
: Legt die TLS-Optionen fest, die beim Verbinden mit dem LDAP-Server verwendet werden (siehe die Methode **connect**). Eine Liste der gültigen Optionen finden Sie in der Dokumentation zum Paket **LDAP**.

  Der Standardwert ist **-request 1 -require 1 -ssl2 no -ssl3 no -tls1 yes -tls1.1 yes -tls1.2 yes**.

  Beispiel:

```
$l configure -tlsoptions {-request yes -require yes}
```

Bei Suchvorgängen wird eine Reihe von Optionen der Klasse **ldap** verwendet (Methoden **traverse**, **search** und **read**, siehe unten).

-scope base|one|sub
: Legt den Suchbereich der LDAP-Suche auf **base**, **one** oder **sub** fest, für eine Suche nur im Basisobjekt, auf einer Ebene oder im ganzen Teilbaum.

  Standard ist **sub**.

-derefaliases never|seach|find|always
: Legt fest, wie Aliase dereferenziert werden: **never** bedeutet, dass Aliase nie dereferenziert werden, **always**, dass Aliase immer dereferenziert werden, **search**, dass Aliase beim Suchen dereferenziert werden, oder **find**, dass Aliase nur beim Auffinden des Basisobjekts der Suche dereferenziert werden.

  Standard ist **never**.

-sizelimit integer
: Legt die maximale Anzahl der Einträge fest, die bei einer Suche abgerufen werden. Der Wert **0** bedeutet keine Begrenzung.

Standard ist **0**.

-timelimit integer
Legt das Zeitlimit für den Abschluss einer Suche fest. Der Wert **0** bedeutet keine Begrenzung.

Standard ist **0**.

-attrsonly 0|1
Legt fest, ob nur Attributnamen abgerufen werden sollen (Wert **1**). Standardmäßig (Wert **0**) werden auch Attributwerte abgerufen.

Standard ist **0**.

Die letzte Option wird verwendet, wenn Einträge abgerufen oder Änderungen im Verzeichnis festgeschrieben werden:

-utf8 pattern-yes pattern-no
Legt fest, welche Attributwerte in UTF-8 kodiert sind. Diese Angabe hängt vom LDAP-Schema der Anwendung ab, da einige Attribute, wie beispielsweise jpegPhoto, nicht in UTF-8 kodiert sind. Diese Option hat die Form einer Liste mit zwei regulären Ausdrücken, die für den Befehl **regexp** geeignet sind (mit den Ankerzeichen ^ und $). Der erste legt fest, welche Attributnamen in UTF-8 kodiert werden sollen, und der zweite wählt unter diesen die Attributnamen aus, die nicht in UTF-8 kodiert werden. So lässt sich sagen: alle Attribute konvertieren, außer jpegPhoto.

Die Standardeinstellung lautet {{.*} {}}, was bedeutet: Alle Attribute werden ausnahmslos konvertiert.

LDAP-Methoden

***la* error ?*newmsg*?**
Diese Methode gibt die Fehlermeldung zurück, die beim letzten Aufruf einer Methode der Klasse **ldap** aufgetreten ist. Wenn das optionale Argument *newmsg* übergeben wird, wird dieses zur letzten Fehlermeldung.

***la* connect *url* ?*binddn*? ?*bindpw*? ?*starttls*?**
Diese Methode stellt über die angegebene URL (die die Form *ldap://host:port* oder *ldaps://host:port* haben kann) eine Verbindung zum LDAP-Server her. Wenn ein optionales *binddn*-Argument zusammen mit dem *bindpw*-Argument angegeben wird, führt **connect** mit dem angegebenen DN und Passwort einen bind beim LDAP-Server durch.

Erhält das Argument *starttls* einen wahren Wert (**1**, **yes** usw.) und verwendet die URL das Schema *ldap://*, wird vor dem LDAP-bind eine TLS-Aushandlung auf der neu erstellten Verbindung gestartet. Standardwert: **no**.

Diese Methode gibt 1 zurück, wenn die Verbindung erfolgreich hergestellt wurde, oder 0, wenn ein Fehler aufgetreten ist (verwenden Sie die Methode **error**, um die Fehlermeldung abzurufen).

***la* disconnect**
Diese Methode trennt die Verbindung zum LDAP-Server (und führt gegebenenfalls ein unbind durch).

la* traverse *base filter attrs entry body

Diese Methode ist eine neue Kontrollstruktur. Sie durchsucht das LDAP-Verzeichnis ab dem angegebenen Base DN (Argument *base*) und wählt Einträge anhand des Arguments *filter* aus. Für jeden gefundenen Eintrag ruft diese Methode die durch das Argument *attrs* festgelegten Attribute ab (oder alle Attribute, wenn es sich um eine leere Liste handelt), speichert diese in der Instanz *entry* der Klasse **entry** und führt das durch das Argument *body* definierte Skript aus. Mit den Optionen lässt sich die Suche verfeinern.

Achtung: Wenn diese Methode verwendet wird, darf das Skript *body* keine weitere LDAP-Suche durchführen (Methoden **traverse**, **search** oder **read**).

la* search *base filter attrs

Diese Methode durchsucht das Verzeichnis auf dieselbe Weise wie die Methode **traverse**. Alle gefundenen Einträge werden in neu angelegten Instanzen der Klasse **entry** gespeichert, die als Liste zurückgegeben werden. Die neu angelegten Instanzen sollten zerstört werden, sobald sie nicht mehr benötigt werden.

la* read *base filter entry ... entry

Diese Methode liest einen oder mehrere Einträge unter Verwendung derselben Suchkriterien wie die Methoden **traverse** und **search**. Alle Attribute werden in den Einträgen gespeichert. Diese Methode bietet eine schnelle Möglichkeit, einige Einträge auszulesen. Sie gibt die Anzahl der im Verzeichnis gefundenen Einträge zurück (die größer sein kann als die Anzahl der ausgelesenen Einträge). Wird diese Methode ohne ein *entry*-Argument aufgerufen, gibt sie lediglich die Anzahl der gefundenen Einträge zurück, ohne Daten zu liefern.

la* commit *entry ... entry

Diese Methode schreibt die in den Argumenten *entry* gespeicherten Änderungen fest. Jedes *entry* kann entweder ein *Änderungs*-Eintrag oder ein *Standard*-Eintrag mit Sicherungskopie sein.

Hinweis: Künftig sollte diese Methode die von OpenLDAP 2.3 und höher bereitgestellte LDAP-Transaktionserweiterung verwenden.

LDAP-Beispiel

```
package require ldapx

#
# Connects to the LDAP directory using StartTLS
#

::ldapx::ldap create l
l configure -tlsoptions {-cadir /etc/ssl/certs -request yes
-require yes}
set url "ldap://server.mycomp.com"
if {! [l connect $url "cn=admin,o=mycomp" "mypasswd" yes]} then {
puts stderr "error: [l error]"
exit 1
}
```

```
#
# Search all entries matching some criterion
#

l configure -scope one
::ldapx::entry create e
set n 0
l traverse "ou=people,o=mycomp" "(sn=Joe*)" {sn givenName} e {
puts "dn: [e dn]"
puts "  sn:        [e get1 sn]"
puts "  givenName: [e get1 givenName]"
incr n
}
puts "$n entries found"
e destroy

#
# Add a telephone number to some entries
# Note this modification cannot be done in the "traverse"
operation.
#

set lent [l search "ou=people,o=mycomp" "(sn=Joe*)" {}]
::ldapx::entry create c
foreach e $lent {
$e backup
$e add1 "telephoneNumber" "+31415926535"
c diff $e
if {! [l commit c]} then {
    puts stderr "error: [l error]"
    exit 1
}
$e destroy
}
c destroy

l disconnect
l destroy
```

Klasse LDIF

LDIF-Instanzdaten

Eine Instanz der Klasse **ldif** enthält die folgenden Daten:

channel

Dies ist der Tcl-Kanal, über den der Inhalt von LDIF-Dateien gelesen oder gespeichert wird. Die

Zuordnung zwischen einer Instanz und einem Kanal erfolgt über die Methode **channel**. Es ist nicht erforderlich, diese Zuordnung aufzuheben, wenn der Vorgang mit der LDIF-Datei beendet ist.

format
LDIF-Dateien können entweder *Standard*-Einträge oder *Änderungs*-Einträge enthalten, jedoch nicht beides. Diese Variable enthält das erkannte Format der Datei (beim Lesen) bzw. das Format der in die Datei geschriebenen Einträge (beim Schreiben).

lastError
Diese Variable enthält die Fehlermeldung, die in der letzten Methode der Klasse **ldif** aufgetreten ist (diese Zeichenkette wird in fast allen Methoden geändert). Mit der Methode **error** können Sie diese Meldung abrufen.

version
Dies ist die Version der LDIF-Datei. Es wird ausschließlich Version 1 unterstützt: Die Methode **read** kann nur Dateien der Version 1 lesen, und die Methode **write** erstellt ausschließlich Dateien der Version 1.

LDIF-Optionen

Diese Klasse definiert zwei Optionen:

-ignore list-of-attributes
Diese Option dient dazu, bestimmte Attributnamen beim Lesen zu ignorieren. Um beispielsweise OpenLDAP-Replikdateien (replog) einzulesen, müssen die Attribute **replica** und **time** ignoriert werden, da sie nicht dem RFC 2849-Standard für LDIF-Dateien entsprechen.

Standardmäßig ist die Liste leer: Es wird kein Attribut ignoriert.

-utf8 pattern-yes pattern-no
Legt fest, welche Attributwerte in UTF-8 kodiert sind. Diese Angabe hängt vom LDAP-Schema der Anwendung ab, da einige Attribute, wie beispielsweise jpegPhoto, nicht in UTF-8 kodiert sind. Diese Option hat die Form einer Liste mit zwei regulären Ausdrücken, die für den Befehl **regexp** geeignet sind (mit den Ankerzeichen ^ und $). Der erste legt fest, welche Attributnamen in UTF-8 kodiert werden sollen, und der zweite wählt unter diesen die Attributnamen aus, die nicht in UTF-8 kodiert werden. So lässt sich sagen: alle Attribute konvertieren, außer jpegPhoto.

Die Standardeinstellung lautet `{{.*} {}}`, was bedeutet: Alle Attribute werden ausnahmslos konvertiert.

LDIF-Methoden

li* channel *chan
Diese Methode ordnet den Tcl-Kanal mit dem Namen *chan* der LDIF-Instanz zu. Dabei wird der Typ des LDIF-Objekts auf *uninitialized* zurückgesetzt.

***li* error ?*newmsg*?**
Diese Methode gibt die Fehlermeldung zurück, die beim letzten Aufruf einer Methode der Klasse **ldif** aufgetreten ist. Wenn das optionale Argument *newmsg* übergeben wird, wird dieses zur letzten Fehlermeldung.

li* read *entry

Diese Methode liest den nächsten Eintrag aus der LDIF-Datei und speichert ihn im Objekt *entry* der Klasse **entry**. Bei dem Eintrag kann es sich um einen *Standard*- oder einen *Änderungs*-Eintrag handeln.

li* write *entry

Diese Methode schreibt den im Argument *entry* angegebenen Eintrag in die LDIF-Datei.

LDIF-Beispiel

```
package require ldapx

# This examples reads a LDIF file containing entries,
# compare them to a LDAP directory, and writes on standard
# output an LDIF file containing changes to apply to the
# LDAP directory to match exactly the LDIF file.

::ldapx::ldif create liin
liin channel stdin

::ldapx::ldif create liout
liout channel stdout

::ldapx::ldap create la
if {! [la connect "ldap://server.mycomp.com"]} then {
puts stderr "error: [la error]"
exit 1
}
la configure -scope one

# Reads LDIF file

::ldapx::entry create e1
::ldapx::entry create e2
::ldapx::entry create c

while {[liin read e1] != 0} {
set base [e1 superior]
set id [e1 rdn]
if {[la read $base "($id)" e2] == 0} then {
    e2 reset
}

c diff e1 e2
if {[llength [c change]] != 0} then {
    liout write c
}
}
```

```
    la disconnect
    la destroy
    e1 destroy
    e2 destroy
    c destroy
    liout destroy
    liin destroy
```

Literatur

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ldap* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

```
Copyright (c) 2006-2018 Pierre David <pdav@users.sourceforge.net>
```

log

log

Stichwörter: log, log level, message, message level

Name

log – Prozeduren zur Protokollierung von Meldungen aus Bibliotheken und Anwendungen.

Übersicht

package require Tcl 8.5 9
package require log ?**1.5**?
::log::levels
::log::lv2longform *level*
::log::lv2color *level*
::log::lv2priority *level*
::log::lv2cmd *level*
::log::lv2channel *level*
::log::lvCompare *level1 level2*
::log::lvSuppress *level {suppress 1}*
::log::lvSuppressLE *level {suppress 1}*

::log::lvIsSuppressed *level*
::log::lvCmd *level cmd*
::log::lvCmdForall *cmd*
::log::lvChannel *level chan*
::log::lvChannelForall *chan*
::log::lvColor *level color*
::log::lvColorForall *color*
::log::log *level text*
::log::logarray *level arrayvar* ?*pattern*?
::log::loghex *level text data*
::log::logsubst *level msg*
::log::logMsg *text*
::log::logError *text*
::log::Puts *level text*

Beschreibung

Das **log**-Paket enthält Befehle, mit denen Bibliotheken und Anwendungen Informationen über ihren internen Ablauf und ihren Status gezielt protokollieren können.

Um das Paket zu verwenden, führen Sie einfach folgenden Befehl aus:

```
    package require log
    log::log notice "Some message"
```

Wie oben zu sehen ist, ist jede an die Protokollierungsfunktion übergebene Meldung mit einer *Stufe* verknüpft, die die Wichtigkeit der Meldung bestimmt. Der Benutzer kann dann auswählen, welche Stufen protokolliert werden sollen, welche Befehle für die Protokollierung der einzelnen Stufen verwendet werden sollen und in welchen Kanal die Meldung geschrieben werden soll. Im folgenden Beispiel ist die Protokollierung aller Meldungen der Stufe **debug** deaktiviert.

```
    package require log
    log::lvSuppress debug
    log::log debug "Unseen message" ; # No output
```

Standardmäßig werden alle Meldungen, die einer Fehlerstufe zugeordnet sind (**emergency**, **alert**, **critical** und **error**), an **stderr** geschrieben. Meldungen mit einem anderen Schweregrad werden an **stdout** ausgegeben. Im folgenden Beispiel wird das Log-Modul so umkonfiguriert, dass **debug**-Meldungen ebenfalls an **stderr** ausgegeben werden.

```
    package require log
    log::lvChannel debug stderr
    log::log debug "Written to stderr"
```

Jeder Meldungsgrad ist zudem mit einem Befehl verknüpft, der beim Protokollieren einer Meldung dieses Grades verwendet wird. Das oben beschriebene Verhalten beruht beispielsweise darauf, dass alle Meldungsgrade standardmäßig den Standardbefehl **::log::Puts** verwenden, um beliebige Meldungen zu protokollieren. Im folgenden Beispiel werden alle Meldungen der Stufe **notice** zur Protokollierung an den nicht standardmäßigen Befehl **toText** übergeben. Dadurch wird die Einstellung des Kanals für solche Meldungen deaktiviert, vorausgesetzt, dass **toText** diese nicht selbst verwendet.

```
    package require log
    log::lvCmd notice toText
    log::log notice "Handled by \"toText\""
```

Eine weitere von dieser Einrichtung gepflegte Datenbank ist eine Zuordnung von Nachrichtenebenen zu Farben. Die Informationen in dieser Datenbank haben *keinen* Einfluss auf das Verhalten des Moduls. Sie werden lediglich als Service bereitgestellt und in Erwartung der Nutzung dieser Funktion in der **tk**-based-Anwendung, die möglicherweise Meldungsprotokolle in Farben kennzeichnen möchte.

API

Folgende Befehle stehen zur Verfügung:

::log::levels
Gibt die Namen aller bekannten Ebenen in alphabetischer Reihenfolge zurück.

::log::lv2longform *level*
Wandelt jede eindeutige Abkürzung eines Levelnamens in den vollständigen Name des Levels um.

::log::lv2color *level*
Wandelt jeden Level-Name, einschließlich einzigartiger Abkürzungen, in die entsprechende Farbe um.

::log::lv2priority *level*
Wandelt jeden Level-Name, einschließlich einzigartiger Abkürzungen, in die entsprechende Priorität um.

::log::lv2cmd *level*
Wandelt jeden Level-Namen, einschließlich einzigartiger Abkürzungen, in das Befehlspräfix um, das zum Verfassen von Nachrichten mit diesem Level verwendet wird.

::log::lv2channel *level*
Wandelt jeden Level-Name, einschließlich einzigartiger Abkürzungen, in den Kanal um, den **::log::Puts** verwendet, um Nachrichten mit diesem Level zu schreiben.

::log::lvCompare *level1 level2*
Vergleicht zwei Ebenen (einschließlich eindeutiger Abkürzungen) hinsichtlich ihrer Priorität. Dieser Befehl kann über die Option -command von lsort verwendet werden. Das Ergebnis ist entweder -1, 0 oder 1 oder ein Fehler. Ein Ergebnis von -1 bedeutet, dass Ebene1 eine geringere Priorität hat als Ebene2. 0 bedeutet, dass beide Ebenen die gleiche Priorität haben. 1 bedeutet, dass Ebene1 eine höhere Priorität hat als Ebene2.

::log::lvSuppress *level* {*suppress* 1}
(De-)aktiviert die Ausgabe von Meldungen der festgelegten Stufe. Auch hier sind eindeutige Abkürzungen für die Stufe zulässig.

::log::lvSuppressLE *level* {*suppress* 1}
(De-)aktiviert die Ausgabe von Meldungen mit der festgelegten Stufe oder einer niedrigeren Priorität. Auch hier sind eindeutige Abkürzungen für die Stufe zulässig.

::log::lvIsSuppressed *level*
Fragt das Paket ab, ob die angegebene Stufe derzeit unterdrückt ist. Eindeutige Abkürzungen von Stufennamen sind zulässig.

::log::lvCmd *level cmd*

Legt für die angegebene Stufe fest, mit welchem Befehl die Nachrichten dieser Stufe geschrieben werden sollen. Eindeutige Abkürzungen der Stufennamen sind zulässig. Bei dem Befehl handelt es sich eigentlich um ein Befehlspräfix, und diese Funktion fügt vor dem Aufruf zwei Argumente an: die Stufe der Nachricht und die Nachricht selbst, in dieser Reihenfolge.

::log::lvCmdForall *cmd*
Legt für alle bekannten Ebenen fest, mit welchem Befehl die Nachrichten dieser Ebene geschrieben werden sollen. Der Befehl handelt eigentlich von einem Befehlspräfix, und diese Funktion fügt vor dem Aufruf zwei Argumente an: die Ebene der Nachricht und die Nachricht selbst, in dieser Reihenfolge.

::log::lvChannel *level chan*
Legt für die angegebene Ebene fest, in welchen Kanal **::log::Puts** (der Standardbefehl) die Meldungen dieser Ebene schreiben soll. Eindeutige Abkürzungen der Stufennamen sind zulässig. Der Befehl ist eigentlich ein Befehlspräfix, und diese Funktion fügt vor dem Aufruf zwei Argumente an: die Stufe der Nachricht und die Nachricht selbst, in dieser Reihenfolge.

::log::lvChannelForall *chan*
Legt für alle bekannten Ebenen fest, mit welchem Kanal **::log::Puts** (der Standardbefehl) die Meldungen dieser Ebene schreiben soll. Der Befehl ist eigentlich ein Befehlspräfix, und diese Funktion fügt vor dem Aufruf zwei Argumente an: die Ebene der Meldung und die Meldung selbst, in dieser Reihenfolge.

::log::lvColor *level color*
Legt für die angegebene Ebene die Farbe fest, die bei einem Aufruf von **::log::lv2color** für diese Ebene zurückgegeben wird. Eindeutige Abkürzungen der Ebenennamen sind zulässig.

::log::lvColorForall *color*
Legt für alle bekannten Ebenen die Farbe fest, die bei einem Aufruf von **::log::lv2color** zurückgegeben wird. Eindeutige Abkürzungen der Ebenennamen sind zulässig.

::log::log *level text*
Protokolliert eine Meldung gemäß den Vorgaben für Befehle, Kanäle und Unterdrückung. Mit anderen Worten: Der Befehl führt keine Aktion aus, wenn die angegebene Stufe unterdrückt ist. Ist sie nicht unterdrückt, wird die eigentliche Protokollierung an den angegebenen Befehl weitergeleitet. Ist für die Stufe kein Befehl angegeben, wird die Meldung nicht protokolliert. Der Standardbefehl **::log::Puts** schreibt die Meldung in den für die jeweilige Stufe angegebenen Kanal. Ist für die Stufe kein Kanal angegeben, wird die Meldung nicht protokolliert. Eindeutige Abkürzungen von Level-Namen sind zulässig. Fehler im eigentlichen Protokollierungsbefehl werden *nicht* abgefangen, sondern an den Aufrufer weitergeleitet, da sie auf Fehlkonfigurationen der Protokollierungsfunktion oder Fehler im Code des Aufrufers selbst hinweisen können.

::log::logarray *level arrayvar* ?*pattern*?
Ähnlich wie **::log::log**, protokolliert jedoch den Inhalt der angegebenen Array-Variablen *arrayvar*, gegebenenfalls beschränkt auf Einträge, die dem *Muster* entsprechen. Das Muster ist standardmäßig * (d. h. alle Einträge), sofern kein Muster angegeben wurde.

::log::loghex *level text data*
Ähnlich wie **::log::log**, geht jedoch davon aus, dass *data* Binärdaten enthält. Diese werden in eine gemischte Hex- und ASCII-Darstellung konvertiert, bevor sie in das Protokoll geschrieben werden.

::log::logsubst *level msg*

Ähnlich wie **::log::log**, jedoch kann *msg* Ersetzungen und Variablenverweise enthalten, die zunächst im Aufrufer-Gültigkeitsbereich ausgewertet werden. Der Zweck dieses Befehls besteht darin, im Fall ohne Protokollierung Overhead zu vermeiden, wenn die Erstellung der Protokollmeldung aufwendig ist. Etwaige Fehler bei der Ersetzung lösen bei der Befehlsausführung einen Fehler aus. Das folgende Beispiel zeigt eine XML-Textdarstellung, die nur im Debug-Modus generiert wird:

```
    log::logsubst debug {XML of node $node is '[$node toXml]'}
```

::log::logMsg *text*
Komfort-Wrapper für **::log::log**. Entspricht dem **::log::log info text**.

::log::logError *text*
Komfort-Wrapper für **::log::log**. Entspricht dem **::log::log error text**.

::log::Puts *level text*
Der Standard-Log-Befehl schreibt Meldungen und deren Schweregrade in vom Benutzer festgelegte Kanäle. Es wird davon ausgegangen, dass die Unterdrückungsprüfungen vom Aufrufer durchgeführt wurden. Es werden vollständige Schweregraddesignationen erwartet; Abkürzungen sind *nicht zulässig*.

Stufen

Das Paket definiert derzeit die folgenden Protokollstufen, wobei die Stufe mit der höchsten Priorität an erster Stelle steht.

- Notfall
- Warnung
- kritisch
- Fehler
- Fehler
- Hinweis
- Info
- debug

Hinweis: Standardmäßig werden alle Meldungen der Stufen **warning** bis hinunter zu **debug** unterdrückt. Dies geschieht bewusst, da (unserer Meinung nach) die Debugging-Ausgabe in den meisten Situationen unerwünscht ist. Die meisten Nutzer möchten eine solche Ausgabe nur dann sehen, wenn sie tatsächlich eine Anwendung debuggen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *log* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright


```
Copyright (c) 2001-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


logger

Stichwörter: logger, log, log level, service

Name

logger – System zur Steuerung der Protokollierung von Ereignissen.

Übersicht

package require Tcl 8.5 9
package require logger ?**0.9.5**?
logger::init *service*
logger::import ?**-all**? ?**-force**? ?*-prefix prefix*? ?*-namespace namespace*? *service*
logger::initNamespace *ns* ?*level*?
logger::services
logger::enable *level*
logger::disable *level*
logger::setlevel *level*
logger::levels
logger::servicecmd *service*
${log}::debug *message*
${log}::info *message*
${log}::notice *message*
${log}::warn *message*
${log}::error *message*
${log}::critical *message*
${log}::alert *message*
${log}::emergency *message*
${log}::setlevel *level*
${log}::enable *level*
${log}::disable *level*
${log}::lvlchangeproc *command*
${log}::lvlchangeproc
${log}::logproc *level*
${log}::logproc *level command*
${log}::logproc *level argname body*
${log}::services
${log}::servicename
${log}::currentloglevel
${log}::delproc *command*
${log}::delproc
${log}::delete
${log}::trace *command*
${log}::trace on

${log}::trace off
${log}::trace status ?*procName*? ?*...*?
${log}::trace add *procName* ?*...*?
${log}::trace add ?*-ns*? *nsName* ?*...*?
${log}::trace remove *procName* ?*...*?
${log}::trace remove ?*-ns*? *nsName* ?*...*?

Beschreibung

Das **logger**-Paket bietet ein flexibles System zur Protokollierung von Meldungen aus verschiedenen Diensten, nach Prioritätsstufen und mit unterschiedlichen Befehlen.

Um das Logger-Paket zu nutzen, gehen wir wie folgt vor:

```
    package require logger
    set log [logger::init myservice]
    ${log}::notice "Initialized myservice logging"

    ... code ...

    ${log}::notice "Ending myservice logging"
    ${log}::delete
```

Im obigen Code geschieht nach dem Laden des Pakets Folgendes:

logger::init *service*
: Initialisiert den Dienst *service* für die Protokollierung. Die Dienstnamen sind eigentlich Tcl-Namespace-Namen und werden daher durch "::" getrennt. Der Dienstname darf weder eine leere Zeichenkette noch ausschließlich aus ":" bestehen. Wenn ein Logger-Dienst initialisiert wird, "erbt" er Eigenschaften von seinen übergeordneten Diensten. Wenn es beispielsweise einen Dienst *foo* gäbe und wir **logger::init** *foo::bar* (um unter *foo* einen *bar* -Dienst anzulegen), würde *bar* die aktuelle Konfiguration des *foo*-Dienstes übernehmen, obwohl es natürlich auch möglich wäre, *bar* anschließend separat zu konfigurieren. Wenn ein Logger-Dienst initialisiert wird und der übergeordnete Dienst noch nicht existiert, wird dieser ebenfalls erstellt. Der neue Logger-Dienst wird mit dem Standard-Loglevel initialisiert, der mit **logger::setlevel** festgelegt wurde.

logger::import ?-all? ?-force? ?-prefix *prefix*? ?-namespace *namespace*? *service*
: Importiert die Befehle des Logger-Dienstes in den aktuellen Namespace. Ohne die Option **-all** werden nur die Befehle importiert, die den Protokollstufen entsprechen. Wird **-all** angegeben, werden alle Befehle im Format **${log}::cmd** importiert. Wenn der Import einen Befehl überschreiben würde, wird ein Fehler gegeben und kein Befehl importiert. Verwenden Sie die Option **-force**, um den Import zu erzwingen und vorhandene Befehle ohne Fehlermeldung zu überschreiben. Wenn die Option **-prefix** angegeben wird, werden die Befehle importiert, wobei ihren Namen das angegebene *Präfix* vorangestellt wird. Wenn die Option **-namespace** angegeben wird, werden die Befehle in den angegebenen Namespace importiert. Existiert der Namespace nicht, wird er angelegt. Wenn ein Namespace ohne vorangestelltes :: angegeben wird, wird er als untergeordneter Namespace des aktuellen Namespaces interpretiert.

logger::initNamespace *ns* ?*level*?
: Befehl zum Einrichten eines Namespace für die Protokollierung. Erstellt einen Logger-Dienst,

der nach dem Namespace *ns* benannt ist (das Präfix :: wird entfernt), importiert alle Log-Befehle in den Namespace und legt die Standard-Protokollierungsstufe fest – entweder wie durch *level* angegeben, oder übernommen von einem Dienst im übergeordneten Namespace, oder als festgelegter Standardwert **warn**.

logger::services
Gibt eine Liste aller verfügbaren Dienste zurück.

logger::enable *level*
Aktiviert global die Protokollierung auf und "über" der angegebenen Stufe. Die Stufen sind **debug**, **info**, **notice**, **warn**, **error**, **critical**, **alert**, **emergency**.

logger::disable *level*
Deaktiviert die Protokollierung global auf und "unterhalb" der angegebenen Stufe. Die Stufen sind die oben aufgeführten.

logger::setlevel *level*
Aktiviert die Protokollierung global auf und "über" der angegebenen Stufe. Die Stufen sind die oben aufgeführten. Dieser Befehl ändert die Standardprotokollierungsstufe für neue Logger, die mit **logger::init** erstellt werden.

logger::levels
Gibt eine Liste der verfügbaren Protokollstufen zurück (die auch oben unter **enable** aufgeführt sind).

logger::servicecmd *service*
Gibt das Token **${log}** zurück, das von **logger::init** für diesen Dienst erstellt wurde.

${log}::debug *message*
siehe unten ...

${log}::info *message*
siehe unten ...

${log}::notice *message*
siehe unten ...

${log}::warn *message*
siehe unten ...

${log}::error *message*
siehe unten ...

${log}::critical *message*
siehe unten ...

${log}::alert *message*
siehe unten ...

${log}::emergency *message*
Dies sind die Befehle, die aufgerufen werden, um tatsächlich eine Meldung zu einem Ereignis zu protokollieren. **${log}** ist die Variable, die aus **logger::init** abgeleitet wird.

${log}::setlevel *level*
Aktiviert die Protokollierung in dem Dienst, auf den **${log}** verweist, sowie in dessen untergeordneten Elementen ab der angegebenen Ebene und deaktivieren Sie die Protokollierung unterhalb dieser Ebene.

${log}::enable *level*
Aktiviert die Protokollierung in dem Dienst, auf den **${log}** verweist, sowie in dessen

untergeordneten Diensten ab der angegebenen Ebene. Beachten Sie, dass dadurch die Protokollierung unterhalb dieser Ebene *nicht* deaktiviert wird; daher sollten Sie stattdessen wahrscheinlich **setlevel** verwenden.

${log}::disable *level*
Deaktiviert die Protokollierung in dem Dienst, auf den **${log}** verweist, sowie in dessen untergeordneten Diensten auf der angegebenen Ebene und darunter. Hinweis: Dadurch wird die Protokollierung oberhalb dieser Ebene *nicht* aktiviert; daher sollten Sie stattdessen wahrscheinlich **setlevel** verwenden. Durch das Deaktivieren der Protokollstufe **emergency** wird die Protokollierung für den Dienst und seine untergeordneten Dienste ausgeschaltet.

${log}::lvlchangeproc *command*
siehe unten ...

${log}::lvlchangeproc
Legt das Skript so fest, dass es aufgerufen wird, wenn die betreffende Instanz ihre Protokollstufe ändert. Wird es ohne Befehl aufgerufen, gibt es den aktuell registrierten Befehl zurück. Dem Befehl werden zwei Argumente angehängt: die alte und die neue Protokollstufe. Die Rückruffunktion wird aufgerufen, nachdem alle Änderungen vorgenommen wurden. Sind untergeordnete Logger betroffen, werden deren Rückruffunktionen vor den Rückruffunktionen ihrer übergeordneten Logger aufgerufen.

```
proc lvlcallback {old new} {
    puts "Loglevel changed from $old to $new"
}
${log}::lvlchangeproc lvlcallback
```

${log}::logproc *level*
siehe unten ...

${log}::logproc *level command*
siehe unten ...

${log}::logproc *level argname body*
Dieser Befehl gibt es in drei Varianten – die dritte, ältere Variante ist veraltet und wird möglicherweise aus zukünftigen Versionen des Logger-Pakets entfernt. Die aktuell verwendete Version erwartet ein Argument: einen Befehl, der wenn die Stufe aufgerufen wird, ausgeführt werden soll. Der Callback-Befehl erwartet ein Argument: den zu protokollierenden Text. Wird **logproc** nur mit einer gültigen Stufe aufgerufen, gibt es den Namen des Befehls zurück, der derzeit als Callback-Befehl registriert ist. **logproc** legt fest, welcher Befehl die eigentliche Protokollierung für eine bestimmte Stufe durchführt. Das Logger-Paket enthält Standardbefehle für alle Protokollierungsstufen, doch mit **logproc** ist es möglich, diese durch benutzerdefinierten Code zu ersetzen. Auf diese Weise könnten Sie Ihre Protokolle über das Netzwerk an eine Datenbank oder an einen anderen Ort senden. Beispiel:

```
proc logtoserver {txt} {
    variable socket
    puts $socket "Notice: $txt"
}

${log}::logproc notice logtoserver
```

Trace-Protokolle unterscheiden sich geringfügig: Anstelle eines Arguments im Klartext wird dem logproc ein Dictionary übergeben, das aus den Schlüsselwörtern **enter** oder **leave** sowie einem weiteren Dictionary mit Details zum Trace besteht. Dazu gehören:
- **proc** – Name der Prozedur, die verfolgt wird.
- **level** – Die Stapelebene für den Aufruf einer Prozedur (aus **info level**).
- **script** – Der Name der Datei, in der die Prozedur definiert ist, oder eine leere Zeichenkette, wenn sie im interaktiven Modus definiert wurde.
- **caller** – Der Name der Prozedur, die die zu verfolgende Prozedur aufruft, oder eine leere Zeichenkette, wenn die Prozedur aus dem globalen Bereich (Stapelebene 0) aufgerufen wurde.
- **procargs** – Ein Dictionary, das aus den Namen der Argumente der Prozedur und den diesen Argumenten zugeordneten Werten besteht (nur bei **enter**-Traces).
- **status** – Der Tcl-Rückgabecode (z. B. **ok**, **continue** usw.) (nur bei **leave**).
- **result** – Der von der Prozedur zurückgegebene Wert (nur bei **leave**-Spuren).

${log}::services
Gibt eine Liste der registrierten Protokollierungsdienste zurück, die untergeordneten Dienste dieses Dienstes sind.

${log}::servicename
Gibt den Namen dieses Dienstes zurück.

${log}::currentloglevel
Gibt die derzeit aktivierte Protokollierungsstufe für diesen Dienst zurück. Wenn keine Protokollierung aktiviert ist, wird **none** zurückgegeben.

${log}::delproc *command*
siehe unten ...

${log}::delproc
Legt das Skript so fest, dass es aufgerufen wird, wenn die betreffende Protokollinstanz gelöscht wird. Wird es ohne Befehl aufgerufen, gibt es den aktuell registrierten Befehl zurück. Beispiel:

```
    ${log}::delproc [list closesock $logsock]
```

${log}::delete
Dieser Befehl löscht einen bestimmten Protokollierungsdienst sowie dessen untergeordnete Elemente. Sie müssen diesen Befehl aufrufen, um die von einem Dienst genutzten Ressourcen zu bereinigen.

${log}::trace *command*
Dieser Befehl steuert die Protokollierung von Ein- und Ausstiegsverfolgungen für bestimmte Prozeduren. Er dient dazu, die Verfolgung zu aktivieren und zu deaktivieren, den Verfolgungsstatus abzufragen und festzulegen, welche Prozeduren verfolgt werden sollen. Wenn die Verfolgung deaktiviert wird, werden die Verfolgungshandler deregistriert. Daher hat die Deaktivierung der Verfolgung keine Auswirkungen auf die Leistung einer Bibliothek, genau wie bei anderen Befehlen zur Protokollierungsstufe.

```
  proc tracecmd { dict } {
      puts $dict
  }

  set log [::logger::init example]
  ${log}::logproc trace tracecmd

  proc foo { args } {
      puts "In foo"
      bar 1
      return "foo_result"
  }

  proc bar { x } {
      puts "In bar"
      return "bar_result"
  }

  ${log}::trace add foo bar
  ${log}::trace on

  foo

# Output:
enter {proc ::foo level 1 script {} caller {} procargs {args {}}}
In foo
enter {proc ::bar level 2 script {} caller ::foo procargs {x 1}}
In bar
leave {proc ::bar level 2 script {} caller ::foo status ok result
    bar_result}
leave {proc ::foo level 1 script {} caller {} status ok result
    foo_result}
```

${log}::trace on
Aktiviert die Trace-Protokollierung für Prozeduren, die über den Befehl **trace add** registriert wurden. Dies entspricht dem Befehl **enable** für andere Protokollierungsstufen, ermöglicht jedoch die Trace-Protokollierung auf jeder beliebigen Stufe. Der Trace-Protokollierungsmechanismus nutzt die Ausführungs-Trace-Funktion von Tcl 8.4 und höher. Der Befehl **trace on** gibt einen Fehler zurück, wenn er von früheren Tcl-Versionen aus aufgerufen wird.

${log}::trace off
Deaktiviert die Trace-Protokollierung für Prozeduren, die über den Befehl **trace add** für die Trace-Protokollierung registriert wurden. Dies entspricht dem Befehl **disable** für andere Protokollierungsstufen, ermöglicht jedoch die Trace-Protokollierung auf jeder Stufe. Die Prozeduren werden nicht abgemeldet, sodass die Protokollierung für sie mit dem Befehl **trace on** wieder aktiviert werden kann. Die Trace-Registrierung verursacht keinen Mehraufwand,

wenn die Trace-Protokollierung deaktiviert ist.

${log}::trace status ?procName? ?...?
Dieser Befehl gibt eine Liste der derzeit für die Trace-Protokollierung registrierten Prozeduren zurück oder ein Flag, das angibt, ob für eine oder mehrere angegebene Prozeduren ein Trace registriert ist oder nicht.

${log}::trace add *procName* ?...?
siehe unten ...

${log}::trace add ?-ns? *nsName* ?...?
Dieser Befehl registriert eine oder mehrere Prozeduren für die Protokollierung von Ein- und Ausstiegsverläufen. Prozeduren können über eine Liste von Prozedurnamen oder Namespaces angegeben werden (in diesem Fall betrifft der Vorgang alle Prozeduren innerhalb des Namespaces). Standardmäßig wird jeder Name zunächst als Name einer Prozedur oder als Suchmuster im Glob-Stil interpretiert; wird er nicht gefunden, wird er als Name eines Namespaces interpretiert. Mit der Option *-ns* kann die Interpretation aller angegebenen Argumente als Namespace-Namen erzwungen werden. Prozeduren müssen definiert sein, bevor sie mit dem Befehl **trace add** zur Protokollierung registriert werden können. Alle Prozedur- oder Namespace-Namen bzw. -Muster, die mit keiner vorhandenen Prozedur übereinstimmen, werden stillschweigend ignoriert.

${log}::trace remove *procName* ?...?
siehe unten ...

${log}::trace remove ?-ns? *nsName* ?...?
Dieser Befehl hebt die Registrierung einer oder mehrerer Prozeduren auf, sodass für diese keine Trace-Protokollierung mehr durchgeführt wird. Dabei gelten dieselben Übereinstimmungsregeln wie beim Befehl **trace add**.

Umsetzung

Das Logger-Paket ist so implementiert, dass (unter Tcl 8.4 und neuer) deaktivierte Protokollierungsprozeduren optimiert werden. Sie werden auf eine Prozedur verweist, die keinen Körper hat und im Bytecode zu einem No-Op kompiliert wird. Dadurch sollte der Leistungseinbruch minimal sein. Wenn Sie wirklich alle Register ziehen wollen, können Sie das Token ${log} in Ihrem Code durch den tatsächlichen Namespace und Befehl ersetzen (${log}::warn wird zu ::logger::tree::myservice::warn), sodass keine Suche nach Variablen durchgeführt wird. Dadurch liegt die Leistung deaktivierter Logger-Befehle fast auf dem Niveau einer vollständigen Deaktivierung der Protokollierung.

Die "Objektorientierung" erfolgt über eine Hierarchie von Namespaces. Die Verwendung eines echten objektorientierten Systems wäre wahrscheinlich die bessere Vorgehensweise oder würde zumindest eine übersichtlichere Implementierung ermöglichen.

Die "Objektorientierung" des Dienstes wird mithilfe von Namespaces realisiert.

Logprocs und Callstack

Das Logger-Paket achtet besonders darauf, dass der Logproc nicht in den Aufrufstapel gelangt. Dadurch können Logprocs Code im Geltungsbereich des Aufrufers ausführen, indem sie "uplevel" verwenden, oder auf lokale Variablen zugreifen, indem sie "upvar" nutzen. Dies kann Traces mit allen üblichen Nebeneffekten auslösen.

```
     # Print caller and current vars in the calling proc
     proc log_local_var {txt} {
          set caller [info level -1]
          set vars [uplevel 1 info vars]
          foreach var [lsort $vars] {
             if {[uplevel 1 [list array exists $var]] == 1} {
                lappend val $var <Array>
             } else {
                lappend val $var [uplevel 1 [list set $var]]
             }
          }
          puts "$txt"
          puts "Caller: $caller"
          puts "Variables in callers scope:"
          foreach {var value} $val {
            puts "$var = $value"
          }
     }

     # install as logproc
     ${log}::logproc debug log_local_var
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *logger* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

loggerAppender

Stichwörter: loggerAppender, appender, logger

Name

logger::appender – Sammlung vordefinierter Appender für den Logger

Übersicht

package require Tcl 8.5 9
package require logger::appender ?**1.4**?
::logger::appender::console -level *level* **-service** *service* ?*options...*?
::logger::appender::colorConsole -level *level* **-service** *service* ?*options...*?

Beschreibung

Dieses Paket enthält eine vordefinierte Sammlung von Logger-Vorlagen.

::logger::appender::console -level *level* -service *service* ?*options*...?
siehe unten ...

-level level
Name der Ebene, der in der Prozedur zur Protokollierung als "Priorität" einzutragen ist.

-service service
Name des Dienstes, der in der Prozedur der Protokollierung als "Kategorie" anzugeben ist.

-appenderArgs appenderArgs
Alle weiteren Argumente für die Log-Prozedur in Listenform

-conversionPattern conversionPattern
Das zu verwendende Protokollmuster (siehe **logger::utils::createLogProc** für die zulässigen Ersetzungen).

-procName procName
Legt den Namen der erstellten Prozedur explizit fest.

-procNameVar procNameVar
Name der Variablen, die im aufrufenden Kontext gesetzt werden soll. Diese Variable enthält den Namen der Prozedur.

::logger::appender::colorConsole -level *level* -service *service* ?*options*...?
Eine Beschreibung der verfügbaren Optionen finden Sie unter **::logger::appender::colorConsole**.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *logger* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright


```
Copyright (c) 2005 Aamer Akhter <aakhter@cisco.com>
```


loggerUtils

Stichwörter: loggerUtils, appender, logger

Name

logger::utils – Dienstprogramme für den Logger

Übersicht

package require Tcl 8.5 9
package require logger::utils ?**1.3.2**?
::logger::utils::createFormatCmd *formatString*
::logger::utils::createLogProc -procName *procName* ?*options...*?
::logger::utils::applyAppender -appender *appenderType* ?*options...*?
::logger::utils::autoApplyAppender *command command-string log op args...*

Beschreibung

Dieses Paket fügt vorlagenbasierte *Appender* hinzu.

::logger::utils::createFormatCmd *formatString*

Dieser Befehl wandelt *formatString* in eine erweiterbare Befehlszeichenfolge um. Die folgenden Zeichenfolgen sind die bekannten Ersetzungen (aus log4perl), die in *formatString* vorkommen dürfen:

%c
Kategorie des Protokollierungsereignisses

%C
Vollqualifizierter Name des Protokollierungsereignisses

%d
Aktuelles Datum im Format JJJJ/MM/TT hh:mm:ss

%H
Hostname

%m
Zu protokollierende Meldung

%M
Methode, mit der das Protokoll-Ereignis ausgelöst wurde

%p
Priorität des Protokollierungsereignisses

%P
PID des aktuellen Prozesses

::logger::utils::createLogProc -procName *procName* ?*options*...?

Dieser Befehl ...

-procName procName
Der Name der zu erstellenden Prozedur.

-conversionPattern pattern
Die im *Muster* zulässigen Ersetzungen finden Sie unter **::logger::utils::createFormatCmd**.

-category category
Die Kategorie (Dienstleistung).

-priority priority
Die Priorität (Stufe).

-outputChannel channel
Kanal (standardmäßig stdout)

::logger::utils::applyAppender -appender *appenderType* ?*options*...?

Dieser Befehl erstellt einen Appender für die angegebenen Logger-Dienste. Wenn kein Dienst angegeben wird, wird der Appender als Standard-Appender für die angegebenen Ebenen hinzugefügt. Wenn keine Ebenen angegeben werden, werden alle Ebenen angenommen.

-service loggerservices

siehe unten ...

-serviceCmd loggerserviceCmds

Name der zu ändernden Logger-Instanz. **-serviceCmd** benötigt als Eingabe den Rückgabewert von logger::init.

-appender appenderType

Typ des zu verwendenden Appenders. Entweder **console** oder **colorConsole**.

-appenderArgs appenderArgs

Zusätzliche Argumente, die auf den Appender angewendet werden sollen. Das Argument der Option ist eine Liste aus Optionen und deren Argumenten.

Zum Beispiel

```
logger::utils::applyAppender -serviceCmd $log -appender console
    -appenderArgs {-conversionPattern {\[%M\] \[%p\] - %m}}
```

Es gelten die üblichen Tcl-Quoting-Regeln.

-levels levelList

Die Liste der Ebenen, auf die dieser Appender angewendet werden soll. Wird keine Liste angegeben, werden alle Ebenen berücksichtigt.

Beispiele:

```
% set log [logger::init testLog]
::logger::tree::testLog
% logger::utils::applyAppender -appender console -serviceCmd $log
% ${log}::error "this is an error"
[2005/08/22 10:14:13] [testLog] [global] [error] this is an error
```

::logger::utils::autoApplyAppender *command command-string log op args*...

Dieser Befehl ist dafür vorgesehen, über **trace leave** zu Aufrufen von logger::init hinzugefügt zu werden. Er prüft den vorkonfigurierten Status (über **::logger::utils::applyAppender**), um Appender für neu erstellte Logger-Instanzen automatisch anzulegen. Er gibt sein Argument *log* zurück.

Beispiele:

```
logger::utils::applyAppender -appender console
set log [logger::init applyAppender-3]
${log}::error "this is an error"
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *logger* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

Copyright

```
Copyright (c) 2005 Aamer Akhter <aakhter@cisco.com>
```


map

map_geocode_nominatim
Stichwörter: map_geocode_nominatim, geocoding, http, location, map, nominatim, server, url

Name
map::geocode::nominatim – Auflösung geografischer Namen mit einem Nominatim-Dienst

Übersicht
package require Tcl 8.6 9
package require http
package require json
package require uri
package require snit
package require map::geocode::nominatim ?**0.3**?
::map::geocode::nominatim *requestor* ?*-baseurl url*? ?*-callback callback*? ?*-error error callback*?
$cmdprefix *result*
$cmdprefix *errorstring*
requestor **search** *query*

Beschreibung
Dieses Paket stellt eine Klasse für den Zugriff auf Geokodierungsdienste bereit, die die *Nominatim*-Schnittstelle implementieren (siehe Literatur)

API
::map::geocode::nominatim *requestor* ?-baseurl *url*? ?-callback *callback*? ?-error *error callback*?

Erstellt ein Objekt für eine Geokodierungsanfrage (*requestor*), das seine Anfragen an den *Nominatim*-Server sendet.

Das Ergebnis des Befehls ist *name*.

Optionen

-baseurl *url*
Der Wert der Basis-URL des *Nominatim*-Dienstes. Der Standardwert ist der Dienst von *OpenStreetMap* unter http://nominatim.openstreetmap.org/search. Eine mögliche kostenlose Alternative finden Sie unter http://open.mapquestapi.com//nominatim/v1/search.

-callback *cmdprefix*
Ein Befehlspräfix, das aufgerufen werden soll, sobald Suchergebnisse vorliegen. Die Standardeinstellung, die aktiv ist, wenn bei der Objekterstellung keine Angabe erfolgte, besteht darin, das *result* (siehe unten) auf *stdout* auszugeben. Das Ergebnis des Befehlspräfixes wird ignoriert. Vom Befehlspräfix ausgelöste Fehler werden abgefangen und führen zur Aufrufung der Fehler-Rückruffunktion (siehe Option **-error** weiter unten), wobei die Fehlermeldung als Argument übergeben wird.

Die Signatur des Befehlspräfixes lautet:
$cmdprefix *result*
Das *result* ist eine Liste von Dictionaries, die jeweils einen Eintrag pro Treffer enthält. Jedes Dictionary enthält die folgenden Einträge:

place_id
Die Ortskennung (FIXME: Was ist das?)

licence
Die Datenlizenzangabe

osm_type
Der OSM-Typ des Standorts

osm_id
FIXME

boundingbox
Die Koordinaten des Begrenzungsrahmens (minimale und maximale Breite, minimale und maximale Länge)

lat
Der Breitengrad des Standorts

lon
Der Längengrad des Ortes

display_name
der für Menschen lesbare Name des Standorts

class
FIXME

type
FIXME

icon
FIXME

-error *cmdprefix*
Ein Befehlspräfix, das bei Auftreten von Fehlern aufgerufen wird. In der Regel handelt es sich dabei um HTTP-Fehler. Die Standardeinstellung, die aktiv ist, wenn bei der Objekterstellung keine Angabe erfolgte, besteht darin, den *errorstring* (siehe unten) an *stderr* auszugeben. Das Ergebnis des Befehlspräfixes wird ignoriert. Vom Befehlspräfix ausgelöste Fehler werden an

übergeordnete Ebenen weitergeleitet.

Die Signatur des Befehlspräfixes lautet:
- **$cmdprefix** *errorstring*

Methoden

requestor* search *query

Diese Methode gibt eine Liste von Dictionaries zurück, wobei jedes Element einem Treffer für die angegebene *query* entspricht.

Literatur

1. http://wiki.openstreetmap.org/wiki/Nominatim
2. http://open.mapquestapi.com/nominatim/

map_slippy

Stichwörter: map_slippy, geodesy, geography, latitute, location, longitude, map, slippy, zoom

Name

map::slippy – Gemeinsamer Code für auf Slippy basierende Kartenpakete

Übersicht

package require Tcl 8.6 9
package require map::slippy ?**0.10**?
::map slippy geo box 2point *zoom geobox*
::map slippy geo box center *geobox*
::map slippy geo box corners *geobox*
::map slippy geo box diameter *geobox*
::map slippy geo box dimensions *geobox*
::map slippy geo box fit *geobox canvdim zmax* ?*zmin*?
::map slippy geo box inside *geobox geo*
::map slippy geo box limit *geobox*
::map slippy geo box opposites *geobox*
::map slippy geo box perimeter *geobox*
::map slippy geo box valid *geobox*
::map slippy geo box valid-list *geoboxes*
::map slippy geo distance *geo1 geo2*
::map slippy geo distance* *closed geo...*
::map slippy geo distance-list *closed geo-list*
::map slippy geo limit *geo*
::map slippy geo bbox *geo...*
::map slippy geo bbox-list *geo-list*
::map slippy geo center *geo...*
::map slippy geo center-list *geo-list*
::map slippy geo diameter *geo...*
::map slippy geo diameter-list *geo-list*
::map slippy geo 2point *zoom geo*
::map slippy geo 2point* *zoom geo...*

::map slippy geo 2point-list *zoom geo-list*
::map slippy geo valid *geo*
::map slippy geo valid-list *geos*
::map slippy length *level*
::map slippy limit2 *x*
::map slippy limit3 *x*
::map slippy limit6 *x*
::map slippy point box 2geo *zoom pointbox*
::map slippy point box center *pointbox*
::map slippy point box corners *pointbox*
::map slippy point box diameter *pointbox*
::map slippy point box dimensions *pointbox*
::map slippy point box inside *pointbox point*
::map slippy point box opposites *pointbox*
::map slippy point box perimeter *pointbox*
::map slippy point distance *point1 point2*
::map slippy point distance* *closed point...*
::map slippy point distance-list *closed point-list*
::map slippy point bbox *point...*
::map slippy point bbox-list *point-list*
::map slippy point center *point...*
::map slippy point center-list *point-list*
::map slippy point diameter *point...*
::map slippy point diameter-list *point-list*
::map slippy point 2geo *zoom point*
::map slippy point 2geo* *zoom point...*
::map slippy point 2geo-list *zoom point-list*
::map slippy point simplify radial *threshold point-list*
::map slippy point simplify rdp *point-list*
::map slippy pretty-distance *x*
::map slippy tiles *level*
::map slippy tile size
::map slippy tile valid *zoom row column levels ?msgvar?*
::map slippy valid latitude *x*
::map slippy valid longitude *x*

Beschreibung

Dieses Paket bietet eine Reihe von Methoden, die Funktionen bereitstellen, die von allen Arten von auf "slippy" basierenden Kartenpaketen benötigt werden.

ACHTUNG, *Hinweis*: Version *0.9* hat keine Kompatibilität mit Version 0.7 und früheren Versionen. Die gesamte API wurde *grundlegend überarbeitet und geändert*.

Hinweis: Informationen zur Darstellung von Positionen in den verschiedenen Koordinatensystemen, die von den Befehlen dieses Pakets verwendet werden, finden Sie im Abschnitt Koordinatensysteme. In den Beschreibungen der Befehle werden diese Informationen nicht erneut aufgegriffen; es wird davon ausgegangen, dass sie bereits bekannt sind.

Koordinatensysteme

Die Befehle dieses Pakets arbeiten mit zwei unterschiedlichen Koordinatensystemen: geografischen Standorten und Punkten. Ersteres stellt Koordinaten für Orte auf der Erde dar, während letzteres für die Verwendung in Tk-*Canvas*-Widgets vorgesehen ist.

Geografisch

Geografische Standorte (kurz: *geo*) werden durch ein Paar aus *Breitengrad*- und *Längengrad* -Werten dargestellt, die jeweils in Grad angegeben werden, da es sich im Wesentlichen um Winkel handelt.

Der Längengrad "**Zero**" entspricht dem *Greenwich-Meridian*, wobei positive Werte nach *Osten* und negative Werte nach *Westen* verweisen, was einen Gesamtbereich von +/- 180 Grad ergibt. Hinweis: Die Längengrade +180 und -180 stellen denselben *Meridian* dar, der Greenwich gegenüberliegt.

Der Breitengrad **zero** entspricht dem *Äquator*, wobei positive Werte nach *Norden* und negative Werte nach *Süden* verweisen. Während der tatsächliche Bereich bei +/- 90 Grad liegt, erfordert die vom Paket verwendete Projektion, dass wir den Bereich auf etwa +/- **85.05112877983284** Grad begrenzen. Dies bedeutet, dass der Nord- und der Südpol nicht darstellbar sind und auf keiner Karte erscheinen.

Ein geografischer Standort wird durch eine Liste dargestellt, die zwei Werte enthält: den Breitengrad und den Längengrad des Standorts, und zwar in dieser Reihenfolge.

Ein geografischer Begrenzungsrahmen wird durch eine Liste dargestellt, die vier Werte enthält: den minimalen Breiten- und Längengrad des Rahmens sowie den maximalen Breiten- und Längengrad des Rahmens, und zwar in dieser Reihenfolge.

Geografische Standorte und Rahmen können mithilfe eines zusätzlichen Parameters, der *zoom* -Stufe, in Punkte und deren Rahmen umgewandelt werden. Dieser Parameter gibt die Größe der Karte auf dem Canvas an, auf dem die Koordinaten projiziert werden sollen.

Punkte

Punkte (kurz: *point*) werden durch ein Paar von *x*- und *y*-Werten dargestellt, die jeweils in Pixeln gemessen werden. Sie beziehen sich auf eine Position in einem Tk-*Canvas*-Widget. Da eine Karte in verschiedenen Vergrößerungsstufen angezeigt werden kann, hängen die genauen Pixel-Koordinaten eines geografischen Ortes von dieser *zoom*-Stufe ab.

Damit die folgende Erläuterung verständlich ist, sollten Sie beachten, dass eine in einem Tk-*Canvas*-Widget angezeigte Karte in gleich große quadratische *Kacheln* unterteilt ist.

Punktkoordinaten sind Kachelkoordinaten, die entsprechend der Größe dieser Kacheln skaliert sind. Dieses Paket verwendet Kacheln der Größe **256**; dies ist die Standardgröße, die von vielen Online-Servern verwendet wird, die Kartenkacheln verschiedener Art bereitstellen.

Ein Punkt wird durch eine Liste dargestellt, die die x- und y-Koordinaten des Ortes in dieser Reihenfolge enthält.

Ein Punktbegrenzungsrahmen wird durch eine Liste dargestellt, die vier Werte enthält: die minimalen x- und y-Koordinaten des Rahmens sowie die maximalen x- und y-Koordinaten des Rahmens, und zwar in dieser Reihenfolge.

Punktpositionen und Kästchen können mithilfe eines zusätzlichen Parameters, der *zoom*-Stufe, in geografische Positionen und die dazugehörigen Kästchen umgewandelt werden. Dieser

Parameter gibt die Größe der Karte im Canvas an, auf die die Koordinaten projiziert werden.

Kachelkoordinaten kommen in der API nur an einer Stelle vor, nämlich in der Signatur des Befehls **map slippy tile valid**. In allen anderen Fällen werden Punktkoordinaten verwendet.

Die Verwendung von Kachel-Koordinaten im Folgenden erleichtert es jedoch, die Struktur der Karte auf den verschiedenen *zoom*-Ebenen (Vergrößerungsfaktoren) zu erläutern.

Im Allgemeinen geben die ganzzahligen Anteile der Kachelkoordinaten die Zeilen- und Spaltennummer einer Kachel auf der Karte an, während die Bruchanteile angeben, wie weit innerhalb dieser Kachel sich der betreffende Ort befindet, wobei rein ganzzahlige Koordinaten (ohne Bruchanteil) die obere linke Ecke einer Kachel bezeichnen.

Der Nullpunkt der Karte befindet sich in der oberen linken Ecke, unabhängig vom Zoomfaktor, wobei größere Koordinaten nach rechts (Osten) und nach unten (Süden) verlaufen und kleinere Koordinaten nach links (Westen) und nach oben (Norden). Auch dies gilt unabhängig vom Zoomfaktor.

Negative Koordinaten sind nicht zulässig.

Bei der Zoomstufe **0** wird die gesamte Karte durch eine einzige Kachel dargestellt, wobei der geografische Nullpunkt bei den Kachel-Koordinaten bei 1/2, 1/2 liegt und der Bereich der Kachel-Koordinaten [0...1] beträgt.

Wenn man von der Zoomstufe **N** zur nächsttieferen (vergrößerten) Stufe wechselt (**N**+1) wird jede Kachel der Ebene **N** in ihre vier Quadranten unterteilt, die dann die Kacheln der Ebene **N**+1 bilden.

Das bedeutet, dass die Karte bei der Zoomstufe **N** (horizontal und vertikal) in **2^N** Zeilen und Spalten unterteilt wird, was insgesamt **4^N** Kacheln ergibt, wobei die Kachel-Koordinaten von **0** bis **2^N+1** reichen.

API

::map slippy geo box 2point *zoom geobox*
: Der Befehl wandelt den geografischen Bereich *geobox* für die angegebene *zoom*-Stufe in einen Punktbereich auf dem Canvas um und gibt diesen Bereich zurück.

::map slippy geo box center *geobox*
: Der Befehl gibt den Mittelpunkt des geografischen Bereichs *geobox* zurück.

::map slippy geo box corners *geobox*
: Dieser Befehl gibt eine Liste zurück, die die vier Eckpunkte enthält, die durch den geografischen Rahmen *geobox* definiert sind. Die vier Punkte sind in dieser Reihenfolge: oben links, unten links, oben rechts und unten rechts.

::map slippy geo box diameter *geobox*
: Der Befehl gibt den Durchmesser des geografischen Bereichs *geobox* in Metern zurück.

::map slippy geo box dimensions *geobox*
: Der Befehl gibt die Abmessungen des geografischen Rahmens *geobox* zurück, und zwar in dieser Reihenfolge: Breite und Höhe.

::map slippy geo box fit *geobox canvdim zmax* ?*zmin*?
: Dieser Befehl berechnet den Zoomfaktor so, dass die *geobox* in einen durch *canvdim* vorgegebenen Ansichtsbereich passt (eine Liste mit zwei Elementen, die die Breite und Höhe des genannten Ansichtsbereichs enthalten), und gibt diesen Wert zurück.

Der Zoomfaktor wird so angepasst, dass er innerhalb des Bereichs *zmin*…*zmax* liegt. Wenn *zmin* nicht angegeben wird, wird standardmäßig der Wert **0** verwendet.

::map slippy geo box inside *geobox geo*
Der Befehl testet, ob der geografische Standort *geo* innerhalb des geografischen Bereichs *geobox* liegt oder nicht. Wenn dies der Fall ist, gibt er **true** zurück, andernfalls **false**.

::map slippy geo box limit *geobox*
Dieser Befehl begrenzt den geografischen Bereich auf maximal 6 Dezimalstellen und gibt das Ergebnis zurück.

Bei geografischen Koordinaten entsprechen 6 Dezimalstellen in etwa einem Grid von 11,1 cm.

::map slippy geo box opposites *geobox*
Dieser Befehl gibt eine Liste zurück, die die beiden wichtigsten Eckpunkte enthält, die sich aus dem geografischen Rahmen *geobox* ergeben. Die beiden Punkte sind der obere linke und der untere rechte Punkt, in dieser Reihenfolge.

::map slippy geo box perimeter *geobox*
Der Befehl gibt den Umfang des geografischen Bereichs *geobox* in Metern zurück.

::map slippy geo box valid *geobox*
Dieser Befehl testet, ob das angegebene geografische Rechteck ausschließlich gültige Breiten- und Längengrade enthält. Er gibt den Wert **true** zurück, wenn das Rechteck gültig ist, andernfalls den Wert **false**.

::map slippy geo box valid-list *geoboxes*
Dieser Befehl prüft, ob die Liste der geografischen Felder ausschließlich gültige Breiten- und Längengrade enthält. Er gibt den Wert **true** zurück, wenn alle Felder gültig sind, andernfalls den Wert **false**.

::map slippy geo distance *geo1 geo2*
Dieser Befehl berechnet die Großkreisdistanz zwischen den beiden geografischen Standorten in Metern und gibt diesen Wert zurück.

Der Code basiert auf der Umsetzung der Haversine-Formel durch https://wiki.tcl-lang.org/page/geodesy.

::map slippy geo distance* *closed geo...*
Als Erweiterung von **map slippy geo distance** berechnet dieser Befehl die kumulative Entfernung entlang des durch die geordnete Menge geografischer Standorte festgelegten Pfads in Metern und gibt diese zurück.

Wenn der Pfad als *closed* gekennzeichnet ist (d. h. ein Polygon/eine Schleife), enthält das Ergebnis zudem die Entfernung zwischen dem letzten und dem ersten Element des Pfads, sodass das Ergebnis der Umfang der durch die Standorte beschriebenen Fläche entspricht.

::map slippy geo distance-list *closed geo-list*
Als Variante von **map slippy geo distance*** verwendet dieser Befehl den Pfad zur Berechnung der Länge als eine einzige Liste geografischer Standorte anstelle einer variierenden Anzahl von Argumenten.

::map slippy geo limit *geo*
Dieser Befehl begrenzt die geografische Position auf höchstens 6 Dezimalstellen und gibt das Ergebnis zurück.

Bei geografischen Koordinaten entsprechen 6 Dezimalstellen in etwa einem Grid von 11,1 cm.

::map slippy geo bbox *geo...*
siehe unten ...

::map slippy geo bbox-list *geo-list*
Diese beiden Befehle berechnen den Begrenzungsrahmen für die angegebene Menge geografischer Standorte und geben einen geografischen Rahmen zurück.

Wenn keine geografischen Standorte angegeben werden, lautet der Rahmen "**0 0 0 0**".

Die Standorte werden entweder als eine variable Anzahl von Argumenten oder als einzelne Liste angegeben.

::map slippy geo center *geo...*
siehe unten ...

::map slippy geo center-list *geo-list*
Diese beiden Befehle berechnen den Mittelpunkt des Begrenzungsrahmens für die angegebene Menge geografischer Standorte.

Wenn keine geografischen Standorte angegeben werden, lautet der Mittelpunkt "**0 0**".

Die Standorte werden entweder als eine variable Anzahl von Argumenten oder als einzelne Liste angegeben.

::map slippy geo diameter *geo...*
siehe unten ...

::map slippy geo diameter-list *geo-list*
Diese beiden Befehle berechnen den Durchmesser für die angegebene Menge geografischer Standorte in Metern. Der Durchmesser entspricht dem größten der paarweisen Abstände zwischen allen Standorten.

Werden weniger als zwei geografische Standorte angegeben, lautet der Durchmesser "**0**".

Die Standorte werden entweder als eine variable Anzahl von Argumenten oder als einzelne Liste festgelegt.

::map slippy geo 2point *zoom geo*
Dieser Befehl wandelt den geografischen Standort *geo* für die angegebene *zoom*-Stufe in einen Punkt auf dem Canvas um und gibt diesen Punkt zurück.

::map slippy geo 2point* *zoom geo...*
siehe unten ...

::map slippy geo 2point-list *zoom geo-list*
Diese beiden Befehle sind Erweiterungen von **map slippy geo 2point**, die eine Reihe von geografischen Standorten entweder als eine variable Anzahl von Argumenten oder als einzelne Liste entgegennehmen, diese alle gemäß der angegebenen *zoom*-Stufe in Punkte umwandeln und eine Liste der Ergebnisse gibt.

::map slippy geo valid *geo*
Dieser Befehl prüft, ob der angegebene geografische Standort ausschließlich gültige Breiten-

und Längengrade enthält. Er gibt den Wert **true** zurück, wenn der Standort gültig ist, andernfalls den Wert **false**.

::map slippy geo valid-list *geos*
Dieser Befehl prüft, ob die Liste der geografischen Standorte ausschließlich gültige Breiten- und Längengrade enthält. Er gibt den Wert **true** zurück, wenn alle Standorte gültig sind, andernfalls den Wert **false**.

::map slippy length *level*
Dieser Befehl gibt die Breite / Höhe einer auf "Slippy" basierenden Karte bei der angegebenen Zoom-*level* in Pixeln zurück. Im Wesentlichen entspricht dies dem Ergebnis von

```
expr { [tiles $level] * [tile size] }
```

::map slippy limit2 *x*
siehe unten ...

::map slippy limit3 *x*
siehe unten ...

::map slippy limit6 *x*
Dieser Befehl begrenzt den Wert auf höchstens 2, 3 oder 6 Dezimalstellen und gibt das Ergebnis zurück.

Bei geografischen Koordinaten entsprechen 6 Dezimalstellen in etwa einem Grid von 11,1 cm.

::map slippy point box 2geo *zoom pointbox*
Der Befehl wandelt den Punktbereich *pointbox* entsprechend der angegebenen *zoom*-Stufe in einen geografischen Bereich auf dem canvas um und gibt diesen Bereich zurück.

::map slippy point box center *pointbox*
Der Befehl gibt den Mittelpunkt der *pointbox* zurück.

::map slippy point box corners *pointbox*
Dieser Befehl gibt eine Liste zurück, die die vier Eckpunkte enthält, die durch den Punktkasten *pointbox* definiert sind. Die vier Punkte sind in dieser Reihenfolge: oben links, unten links, oben rechts und unten rechts.

::map slippy point box diameter *pointbox*
Der Befehl gibt den Durchmesser der *pointbox* in Pixeln zurück.

::map slippy point box dimensions *pointbox*
Der Befehl gibt die Abmessungen der *pointbox* – Breite und Höhe – in dieser Reihenfolge zurück.

::map slippy point box inside *pointbox point*
Der Befehl prüft, ob der *point* in der *pointbox* enthalten ist oder nicht. Wenn dies der Fall ist, gibt er **true** zurück, andernfalls **false**.

::map slippy point box opposites *pointbox*
Dieser Befehl gibt eine Liste zurück, die die beiden Haupteckpunkte enthält, die durch den Punktkasten *pointbox* definiert sind. Die beiden Punkte sind in dieser Reihenfolge der obere linke und der untere rechte Punkt.

::map slippy point box perimeter *pointbox*
Der Befehl gibt den Umfang der *pointbox* in Pixeln zurück.

::map slippy point distance *point1 point2*

Dieser Befehl berechnet den euklidischen Abstand zwischen den beiden Punkten in Pixeln und gibt diesen Wert zurück.

::map slippy point distance* *closed point...*
Als Erweiterung von **map slippy point distance** berechnet dieser Befehl die kumulative Entfernung entlang des durch die geordnete Punktmenge festgelegten Pfads und gibt diese zurück.

Wenn der Pfad als *closed* markiert ist (d. h. ein Polygon/eine Schleife), enthält das Ergebnis zudem die Entfernung zwischen dem letzten und dem ersten Element des Pfads, sodass das Ergebnis der Umfang der durch die Standorte beschriebenen Fläche entspricht.

::map slippy point distance-list *closed point-list*
Als Variante von **map slippy point distance*** verwendet dieser Befehl den Pfad zur Berechnung der Länge als einzelne Liste von Punkten anstelle einer variierenden Anzahl von Argumenten.

::map slippy point bbox *point...*
siehe unten ...

::map slippy point bbox-list *point-list*
Diese beiden Befehle berechnen den Begrenzungsrahmen für die angegebene Punktmenge und geben einen Punktkasten zurück.

Wenn keine Punkte angegeben werden, lautet der Kasten "**0 0 0 0**".

Die Positionen werden entweder als eine variable Anzahl von Argumenten oder als einzelne Liste angegeben.

::map slippy point center *point...*
siehe unten ...

::map slippy point center-list *point-list*
Diese beiden Befehle berechnen den Mittelpunkt des Begrenzungsrahmens für die angegebene Punktmenge.

Wenn keine Punkte angegeben werden, lautet der Mittelpunkt "**0 0**".

Die Positionen werden entweder als eine variable Anzahl von Argumenten oder als einzelne Liste angegeben.

::map slippy point diameter *point...*
siehe unten ...

::map slippy point diameter-list *point-list*
Dieser Befehl berechnet den Durchmesser der angegebenen Punktmenge in Pixeln. Der Durchmesser entspricht dem größten der paarweisen Abstände zwischen allen Positionen.

Werden weniger als zwei Punkte angegeben, lautet der Durchmesser "**0**".

Die Positionen werden entweder als eine variable Anzahl von Argumenten oder als einzelne Liste angegeben.

::map slippy point 2geo *zoom point*

Dieser Befehl wandelt den *point* auf dem canvas bei der angegebenen *zoom* in einen geografischen Standort um und gibt diesen Standort zurück.

::map slippy point 2geo* *zoom point...*
siehe unten ...

::map slippy point 2geo-list *zoom point-list*
Diese beiden Befehle sind Erweiterungen von **map slippy point 2geo**, die eine Reihe von Punkten entweder als eine variable Anzahl von Argumenten oder als eine einzige Liste entgegennehmen, diese alle entsprechend der angegebenen *zoom*-Stufe in geografische Standorte umwandeln und eine Liste der Ergebnisse gibt.

::map slippy point simplify radial *threshold point-list*
Dieser Befehl nimmt eine Punktfolge (als einzelne Liste) entgegen, vereinfacht diesen Pfad mithilfe des Algorithmus "*Radial Distance*" und gibt den vereinfachten Pfad als Liste zurück.

Im Wesentlichen behält der Algorithmus von benachbarten Punkten nur denjenigen bei, der näher an diesem ersten Punkt liegt als der festgelegte Schwellenwert, und verwirft die übrigen.

::map slippy point simplify rdp *point-list*
Dieser Befehl nimmt eine Punktgruppe (als einzelne Liste) entgegen, vereinfacht diese mithilfe des *nichtparametrischen Ramer-Douglas-Peucker*-Algorithmus und gibt den vereinfachten Pfad als Liste von Punkten zurück.

::map slippy pretty-distance *x*
Diese Methode formatiert die Entfernung *x* (in Metern) für die Anzeige und gibt die resultierende Zeichenfolge (einschließlich der gewählten Einheit) zurück.

Entfernungen im Subkilometerbereich werden auf zwei Dezimalstellen begrenzt, d. h. auf Zentimeter, während Entfernungen im Kilometerbereich auf drei Dezimalstellen begrenzt werden, d. h. auf Meter.

::map slippy tiles *level*
Dieser Befehl gibt die Breite / Höhe einer auf Slippy basierenden Karte bei der angegebenen Zoomstufe *level* zurück, in *Kacheln*.

::map slippy tile size
Dieser Befehl gibt die Breite / Höhe einer Kachel in einer auf "Slippy" basierenden Karte in Pixeln zurück.

::map slippy tile valid *zoom row column levels* ?*msgvar*?
Dieser Befehl prüft, ob die durch *zoom*, *row* und *column* beschriebene Kachel für eine auf "Slippy" basierende Karte mit dieser Anzahl an Zoom-*levels* gültig ist oder nicht. Das Ergebnis ist ein boolescher Wert: **true**, wenn die Kachel gültig ist, und **false** andernfalls. Im letzteren Fall wird eine Meldung in der durch *msgvar* benannten Variable hinterlassen, sofern diese festgelegt wurde.

::map slippy valid latitude *x*
Dieser Befehl testet, ob das Argument *x* ein gültiger Breitengradwert ist, und gibt das boolesche Ergebnis dieser Prüfung zurück. Das heißt, **true**, wenn der Wert gültig ist, und andernfalls **false**.

::map slippy valid longitude *x*
Dieser Befehl testet, ob das Argument *x* ein gültiger Längengradwert ist, und gibt das

boolesche Ergebnis dieser Prüfung zurück. Das heißt, **true**, wenn der Wert gültig ist, und **false** andernfalls.

Literatur

1. http://wiki.openstreetmap.org/wiki/Main_Page

map_slippy_cache

Stichwörter: map_slippy_cache, cache, filesystem, location, map, slippy, tile, zoom

Name

map::slippy::cache – Verwaltung eines Kache-Caches im lokalen Dateisystem

Übersicht

package require Tcl 8.6 9
package require Tk 8.6
package require map::slippy
package require map::slippy::cache ?**0.5**?
::map::slippy::cache *cacheName cachedir provider*
cacheName **valid** *tile* ?*msgvar*?
cacheName **exists** *tile*
cacheName **get** *tile donecmd*

Beschreibung

Dieses Paket stellt eine Klasse zur Verwaltung eines Cache mit Kacheln für "Slippy"-basierte Karten im lokalen Dateisystem bereit.

ACHTUNG, *Hinweis*: Version *0.5* hat keine abwärtskompatible Kompatibilität mit Version 0.3 und früheren Versionen. Dies ist darauf zurückzuführen, dass die internen Abläufe dieses Pakets auf die Verwendung der Version 0.8 von **map::slippy** umgestellt wurden.

API

::map::slippy::cache *cacheName cachedir provider*

Erstellt den Cache *cacheName* und konfiguriert ihn sowohl mit dem Pfad zu dem Verzeichnis, das die lokal zwischengespeicherten Kacheln enthält (*cachedir*), als auch mit dem Befehlspräfix, über das er angeforderte Kacheln abruft, die dem Cache selbst noch nicht bekannt sind (*provider*).

Das Ergebnis des Befehls lautet *cacheName*.

Methoden

***cacheName* valid *tile* ?*msgvar*?**

Diese Methode überprüft die Gültigkeit einer angegebenen *tile*-Kennung. Es handelt sich hierbei um eine Komfortfunktion für **::map::slippy tile valid**, die über dieselbe Schnittstelle verfügt.

cacheName* exists *tile

Diese Methode testet, ob der Cache die angegebene *tile* enthält oder nicht. Das Ergebnis ist ein boolescher Wert: **true**, wenn die Kachel bekannt ist, und andernfalls **false**. Die Kachel

wird durch eine Liste identifiziert, die drei Elemente enthält: Zoomstufe, Zeilennummer und Spaltennummer, in dieser Reihenfolge.

cacheName* get *tile donecmd

Dies ist die Hauptmethode des Caches, mit der das Bild für die angegebene *tile* aus dem Cache abgerufen wird. Die Kachel-Kennung ist eine Liste mit drei Elementen: der Zoomstufe sowie der Zeilen- und Spaltennummer der Kachel, in dieser Reihenfolge.

Der Befehl refix *donecmd* wird aufgerufen, wenn der Cache entweder das Bild für die Kachel kennt oder feststellt, dass kein Bild verfügbar sein wird. Er wird entweder mit 2 oder 3 Argumenten aufgerufen, d. h.
1. Die Zeichenfolge "**set**", die *tile* und das Bild.
2. Die Zeichenfolge "**unset**" und die *tile*.

Diese beiden Möglichkeiten dienen dazu, entweder das Bild für die *tile* anzugeben oder zu signalisieren, dass für die *tile* kein Bild definiert ist.

Wenn der Cache keine Informationen über die Kachel besitzt, ruft er den bei seiner Erstellung festgelegten Befehlspräfix *provider* auf und fügt drei Argumente hinzu: die Zeichenfolge **get**, die *tile* sowie eine Rückruffunktion in den Cache. Letzterer wird vom Provider aufgerufen, um entweder das Bild an den Cache zu übertragen oder zu signalisieren, dass für die Kachel kein Bild vorhanden ist.

Wenn mehrere Anfragen für dieselbe Kachel gestellt werden, wird nur eine einzige Anfrage an den Provider gesendet.

Literatur

1. http://wiki.openstreetmap.org/wiki/Main_Page

map_slippy_fetcher

Stichwörter: map_slippy_fetcher, http, location, map, server, slippy, tile, url, zoom

Name

map::slippy::fetcher – Zugriff auf einen Server, der Kacheln für auf Slippy basierende Karten bereitstellt

Übersicht

package require Tcl 8.6 9
package require Tk 8.6
package require map::slippy
package require map::slippy::fetcher ?**0.7**?
::map::slippy::fetcher *fetcherName levels url*
fetcherName **levels**
fetcherName **tileheight**
fetcherName **tilewidth**
fetcherName **get** *tile donecmd*

Beschreibung

Dieses Paket enthält eine Klasse für den Zugriff auf HTTP-Server, die Kacheln für Slippy-basierte Karten bereitstellen.

ACHTUNG, *Hinweis*: Die Version *0.7* hat keine abwärtskompatible Kompatibilität mit Version 0.5 und früheren Versionen. Dies ist darauf zurückzuführen, dass die internen Abläufe dieses Pakets auf die Verwendung der Version 0.8 von **map::slippy** umgestellt wurden.

API

::map::slippy::fetcher *fetcherName levels url*

Erstellt den Fetcher *fetcherName* und konfiguriert ihn mit der Anzahl der vom Kachelserver unterstützten Zoomstufen *levels* sowie der *url*, unter der er auf Kachelanfragen wartet.

Das Ergebnis des Befehls lautet *fetcherName*.

Methoden

***fetcherName* levels**

Diese Methode gibt die Anzahl der Zoomstufen zurück, die vom Fetcher-Objekt und dem von diesem aufgerufenen Kachelserver unterstützt werden.

***fetcherName* tileheight**

Diese Methode gibt die Höhe der ausgelieferten Kacheln in Pixeln zurück.

***fetcherName* tilewidth**

Diese Methode gibt die Breite der bereitgestellten Kacheln in Pixeln zurück.

fetcherName* get *tile donecmd

Dies ist die Hauptmethode des Fetchers, die das Bild für die angegebene *tile* abruft. Die Kachel-Kennung ist eine Liste mit drei Elementen: der Zoomstufe sowie den Zeilen- und Spaltennummern der Kachel, in dieser Reihenfolge.

Der Befehl "refix *donecmd*" wird aufgerufen, wenn der Fetcher entweder das Bild für die Kachel kennt oder feststellt, dass kein Bild mehr bereitgestellt wird. Er wird entweder mit 2 oder 3 Argumenten aufgerufen, d. h.
1. Die Zeichenfolge "**set**", die *tile* und das Bild.
2. Die Zeichenfolge "**unset**" und die *tile*.

Diese beiden Möglichkeiten dienen dazu, entweder das Bild für die *tile* anzugeben oder zu signalisieren, dass für die *tile* kein Bild definiert ist.

Literatur

1. http://wiki.openstreetmap.org/wiki/Main_Page

mapproj

mapproj

Stichwörter: mapproj, geodesy, map, projection

Name

mapproj – Routinen für Kartenprojektionen

Übersicht

package require Tcl ?**8.5 9**?
package require math::interpolate ?**1.1**?

package require math::special ?**0.2.2**?
package require mapproj ?**1.1**?
::mapproj::toPlateCarree *lambda_0 phi_0 lambda phi*
::mapproj::fromPlateCarree *lambda_0 phi_0 x y*
::mapproj::toCylindricalEqualArea *lambda_0 phi_0 lambda phi*
::mapproj::fromCylindricalEqualArea *lambda_0 phi_0 x y*
::mapproj::toMercator *lambda_0 phi_0 lambda phi*
::mapproj::fromMercator *lambda_0 phi_0 x y*
::mapproj::toMillerCylindrical *lambda_0 lambda phi*
::mapproj::fromMillerCylindrical *lambda_0 x y*
::mapproj::toSinusoidal *lambda_0 phi_0 lambda phi*
::mapproj::fromSinusoidal *lambda_0 phi_0 x y*
::mapproj::toMollweide *lambda_0 lambda phi*
::mapproj::fromMollweide *lambda_0 x y*
::mapproj::toEckertIV *lambda_0 lambda phi*
::mapproj::fromEckertIV *lambda_0 x y*
::mapproj::toEckertVI *lambda_0 lambda phi*
::mapproj::fromEckertVI *lambda_0 x y*
::mapproj::toRobinson *lambda_0 lambda phi*
::mapproj::fromRobinson *lambda_0 x y*
::mapproj::toCassini *lambda_0 phi_0 lambda phi*
::mapproj::fromCassini *lambda_0 phi_0 x y*
::mapproj::toPeirceQuincuncial *lambda_0 lambda phi*
::mapproj::fromPeirceQuincuncial *lambda_0 x y*
::mapproj::toOrthographic *lambda_0 phi_0 lambda phi*
::mapproj::fromOrthographic *lambda_0 phi_0 x y*
::mapproj::toStereographic *lambda_0 phi_0 lambda phi*
::mapproj::fromStereographic *lambda_0 phi_0 x y*
::mapproj::toGnomonic *lambda_0 phi_0 lambda phi*
::mapproj::fromGnomonic *lambda_0 phi_0 x y*
::mapproj::toAzimuthalEquidistant *lambda_0 phi_0 lambda phi*
::mapproj::fromAzimuthalEquidistant *lambda_0 phi_0 x y*
::mapproj::toLambertAzimuthalEqualArea *lambda_0 phi_0 lambda phi*
::mapproj::fromLambertAzimuthalEqualArea *lambda_0 phi_0 x y*
::mapproj::toHammer *lambda_0 lambda phi*
::mapproj::fromHammer *lambda_0 x y*
::mapproj::toConicEquidistant *lambda_0 phi_0 phi_1 phi_2 lambda phi*
::mapproj::fromConicEquidistant *lambda_0 phi_0 phi_1 phi_2 x y*
::mapproj::toAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
::mapproj::fromAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 x y*
::mapproj::toLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
::mapproj::fromLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 x y*
::mapproj::toLambertCylindricalEqualArea *lambda_0 phi_0 lambda phi*
::mapproj::fromLambertCylindricalEqualArea *lambda_0 phi_0 x y*
::mapproj::toBehrmann *lambda_0 phi_0 lambda phi*
::mapproj::fromBehrmann *lambda_0 phi_0 x y*

::mapproj::toTrystanEdwards *lambda_0 phi_0 lambda phi*
::mapproj::fromTrystanEdwards *lambda_0 phi_0 x y*
::mapproj::toHoboDyer *lambda_0 phi_0 lambda phi*
::mapproj::fromHoboDyer *lambda_0 phi_0 x y*
::mapproj::toGallPeters *lambda_0 phi_0 lambda phi*
::mapproj::fromGallPeters *lambda_0 phi_0 x y*
::mapproj::toBalthasart *lambda_0 phi_0 lambda phi*
::mapproj::fromBalthasart *lambda_0 phi_0 x y*

Beschreibung

Das Paket **mapproj** bietet eine Reihe von Prozeduren zur Umrechnung zwischen Weltkoordinaten (Breiten- und Längengrad) und Kartenkoordinaten bei verschiedenen Kartenprojektionen.

Befehle

Die folgenden Befehle dienen zur Umrechnung zwischen Weltkoordinaten und Kartenkoordinaten:

::mapproj::toPlateCarree *lambda_0 phi_0 lambda phi*
Konvertiert in die *Plate-Carrée*-Projektion (zylindrisch äquidistant).

::mapproj::fromPlateCarree *lambda_0 phi_0 x y*
Konvertiert aus der *Plate-Carrée*-Projektion (zylindrisch äquidistant).

::mapproj::toCylindricalEqualArea *lambda_0 phi_0 lambda phi*
Wandelt in die zylindrische flächentreue Projektion um.

::mapproj::fromCylindricalEqualArea *lambda_0 phi_0 x y*
Konvertiert aus der zylindrischen flächentreuen Projektion.

::mapproj::toMercator *lambda_0 phi_0 lambda phi*
Konvertiert in die Mercator-Projektion (zylindrisch-konform).

::mapproj::fromMercator *lambda_0 phi_0 x y*
Konvertiert aus der Mercator-Projektion (zylindrisch-konform).

::mapproj::toMillerCylindrical *lambda_0 lambda phi*
Konvertiert in die zylindrische Miller-Projektion.

::mapproj::fromMillerCylindrical *lambda_0 x y*
Konvertiert aus der zylindrischen Miller-Projektion.

::mapproj::toSinusoidal *lambda_0 phi_0 lambda phi*
Konvertiert in die sinusförmige (Sanson-Flamsteed-)Projektion.

::mapproj::fromSinusoidal *lambda_0 phi_0 x y*
Konvertiert aus der sinusförmigen (Sanson-Flamsteed-)Projektion.

::mapproj::toMollweide *lambda_0 lambda phi*
Konvertiert in die Mollweide-Projektion.

::mapproj::fromMollweide *lambda_0 x y*
Konvertiert aus der Mollweide-Projektion.

::mapproj::toEckertIV *lambda_0 lambda phi*
Konvertiert in die Eckert-IV-Projektion.

::mapproj::fromEckertIV *lambda_0 x y*

Konvertiert aus der Eckert-IV-Projektion.

::mapproj::toEckertVI ***lambda_0 lambda phi***
Konvertiert in die Eckert-VI-Projektion.

::mapproj::fromEckertVI ***lambda_0 x y***
Konvertiert aus der Eckert-VI-Projektion.

::mapproj::toRobinson ***lambda_0 lambda phi***
Wandelt in die Robinson-Projektion um.

::mapproj::fromRobinson ***lambda_0 x y***
Konvertiert aus der Robinson-Projektion.

::mapproj::toCassini ***lambda_0 phi_0 lambda phi***
Konvertiert in die Cassini-Projektion (transversal-zylindrisch-äquidistant).

::mapproj::fromCassini ***lambda_0 phi_0 x y***
Konvertiert aus der Cassini-Projektion (transversal-zylindrisch-äquidistant).

::mapproj::toPeirceQuincuncial ***lambda_0 lambda phi***
Konvertiert in die Peirce-Quincuncial-Projektion.

::mapproj::fromPeirceQuincuncial ***lambda_0 x y***
Konvertiert aus der Peirce-Quincuncial-Projektion.

::mapproj::toOrthographic ***lambda_0 phi_0 lambda phi***
Konvertiert in die orthografische Projektion.

::mapproj::fromOrthographic ***lambda_0 phi_0 x y***
Konvertiert aus der orthografischen Projektion.

::mapproj::toStereographic ***lambda_0 phi_0 lambda phi***
Wandelt in die stereografische (azimutal-konforme) Projektion um.

::mapproj::fromStereographic ***lambda_0 phi_0 x y***
Konvertiert aus der stereografischen (azimutal-konformen) Projektion.

::mapproj::toGnomonic ***lambda_0 phi_0 lambda phi***
Wandelt in die gnomonische Projektion um.

::mapproj::fromGnomonic ***lambda_0 phi_0 x y***
Konvertiert aus der gnomonischen Projektion.

::mapproj::toAzimuthalEquidistant ***lambda_0 phi_0 lambda phi***
Konvertiert in die azimutale äquidistante Projektion.

::mapproj::fromAzimuthalEquidistant ***lambda_0 phi_0 x y***
Konvertiert aus der azimutalen äquidistanten Projektion.

::mapproj::toLambertAzimuthalEqualArea ***lambda_0 phi_0 lambda phi***
Konvertiert in die azimutale flächentreue Lambert-Projektion.

::mapproj::fromLambertAzimuthalEqualArea ***lambda_0 phi_0 x y***
Konvertiert aus der azimutalen flächentreuen Lambert-Projektion.

::mapproj::toHammer ***lambda_0 lambda phi***
Konvertiert in die Hammer-Projektion.

::mapproj::fromHammer ***lambda_0 x y***
Konvertiert aus der Hammer-Projektion.

::mapproj::toConicEquidistant ***lambda_0 phi_0 phi_1 phi_2 lambda phi***
Konvertiert in die konische äquidistante Projektion.

::mapproj::fromConicEquidistant *lambda_0 phi_0 phi_1 phi_2 x y*
Konvertiert aus der konischen äquidistanten Projektion.

::mapproj::toAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
Konvertiert in die flächentreue konische Albers-Projektion.

::mapproj::fromAlbersEqualAreaConic *lambda_0 phi_0 phi_1 phi_2 x y*
Konvertiert aus der flächentreuen konischen Albers-Projektion.

::mapproj::toLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 lambda phi*
Konvertiert in die konforme konische Lambert-Projektion.

::mapproj::fromLambertConformalConic *lambda_0 phi_0 phi_1 phi_2 x y*
Konvertiert aus der konformen konischen Lambert-Projektion.

Unter den zylindrischen flächentreuen Projektionen gibt es mehrere benannte Standardparallelen:

::mapproj::toLambertCylindricalEqualArea *lambda_0 phi_0 lambda phi*
Wandelt in die zylindrische flächentreue Lambert-Projektion um. (Die Standardparallele ist der Äquator.)

::mapproj::fromLambertCylindricalEqualArea *lambda_0 phi_0 x y*
Konvertiert aus der zylindrischen flächentreuen Lambert-Projektion. (Die Standardparallele ist der Äquator.)

::mapproj::toBehrmann *lambda_0 phi_0 lambda phi*
Konvertiert in die zylindrische flächentreue Behrmann-Projektion. (Die Standardparallelen liegen bei 30 Grad Nord und Süd.)

::mapproj::fromBehrmann *lambda_0 phi_0 x y*
Konvertiert aus der zylindrischen flächentreuen Behrmann-Projektion. (Die Standardparallelen liegen bei 30 Grad Nord und Süd.)

::mapproj::toTrystanEdwards *lambda_0 phi_0 lambda phi*
Konvertiert in die zylindrische flächentreue Trystan-Edwards-Projektion. (Die Standardparallelen liegen bei 37,4 Grad Nord und Süd.)

::mapproj::fromTrystanEdwards *lambda_0 phi_0 x y*
Konvertiert aus der zylindrischen flächentreuen Trystan-Edwards-Projektion. (Die Standardparallelen liegen bei 37,4 Grad Nord und Süd.)

::mapproj::toHoboDyer *lambda_0 phi_0 lambda phi*
Konvertiert in die zylindrische flächentreue Hobo-Dyer-Projektion. (Die Standardparallelen liegen bei 37,5 Grad Nord und Süd.)

::mapproj::fromHoboDyer *lambda_0 phi_0 x y*
Konvertiert aus der zylindrischen flächentreuen Hobo-Dyer-Projektion. (Die Standardparallelen liegen bei 37,5 Grad Nord und Süd.)

::mapproj::toGallPeters *lambda_0 phi_0 lambda phi*
Konvertiert in die zylindrische flächentreue Gall-Peters-Projektion. (Die Standardparallelen liegen bei 45 Grad Nord und Süd.)

::mapproj::fromGallPeters *lambda_0 phi_0 x y*
Konvertiert aus der zylindrischen flächentreuen Gall-Peters-Projektion. (Die Standardparallelen liegen bei 45 Grad Nord und Süd.)

::mapproj::toBalthasart *lambda_0 phi_0 lambda phi*
Konvertiert in die zylindrische flächentreue Balthasart-Projektion. (Die Standardparallelen

liegen bei 50 Grad Nord und Süd.)

::mapproj::fromBalthasart *lambda_0 phi_0 x y*

Konvertiert aus der zylindrischen flächentreuen Balthasart-Projektion. (Die Standardparallelen liegen bei 50 Grad Nord und Süd.)

Argumente

Die Projektionsbefehle akzeptieren die folgenden Argumente:

lambda

Längengrad des zu projizierenden Punktes in Grad.

phi

Breitengrad des zu projizierenden Punktes in Grad.

lambda_0

Längengrad des Mittelpunkts des Kartenblatts in Grad. Bei vielen Projektionen ist dieser Wert auch der Referenzmeridian der Projektion.

phi_0

Breitengrad des Mittelpunkts des Kartenblatts in Grad. Bei azimutalen Projektionen ist dieser Wert auch der Breitengrad des Projektionsmittelpunkts.

phi_1

Breitengrad der ersten Referenzparallele, bei Projektionen, die Referenzparallele verwenden.

phi_2

Breitengrad der zweiten Referenzparallele, bei Projektionen, die Referenzparallele verwenden.

x

X-Koordinate eines Punktes auf der Karte, in Einheiten des Erdradius.

y

Y-Koordinate eines Punktes auf der Karte, in Einheiten des Erdradius.

Ergebnisse

Bei allen Prozeduren, deren Namen mit "to" beginnen, ist der Rückgabewert eine Liste aus einer *x*-Koordinate und einer *y*-Koordinate. Die Koordinaten beziehen sich auf den Mittelpunkt des zu zeichnenden Kartenblatts und werden in Erdradien am Referenzort auf der Karte gemessen. Bei allen Funktionen, deren Namen mit "from" beginnen, ist der Rückgabewert eine Liste aus Längen- und Breitengrad in Grad.

Auswahl einer Projektion

Dieses Paket bietet eine Vielzahl von Projektionen, da keine einzelne Projektion für alle Karten geeignet ist. In diesem Abschnitt soll eine Orientierungshilfe bei der Auswahl einer Projektion gegeben werden.

Zuerst ist die Art der Daten zu betrachten, die auf der Karte dargestellt werden sollen. Handelt es sich um *gerichtete* Daten (z. B. Winde, Meeresströmungen oder Magnetfelder), ist eine Projektion nötig, die Winkel beibehält; diese heißen *konforme* Projektionen. Zu den konformen Projektionen zählen die Mercator-Projektion, die azimutale flächentreue Albers-Projektion, die stereografische Projektion und die Peirce-Quincuncial-Projektion. Handelt es sich um *thematische* Daten, die Eigenschaften von Land oder Wasser beschreiben, wie Temperatur, Bevölkerungsdichte, Landnutzung oder demografische Daten, ist eine Projektion nötig, bei der die Flächen auf der Karte proportional zu den Flächen in der Wirklichkeit sind. Zu diesen

sogenannten *flächentreuen* Projektionen gehören die verschiedenen zylindrischen flächentreuen Projektionen, die sinusförmige Projektion, die azimutale flächentreue Lambert-Projektion, die flächentreue konische Albers-Projektion sowie mehrere Weltkartenprojektionen (zylindrische Miller-Projektion, Mollweide, Eckert IV, Eckert VI, Robinson und Hammer). Ist der entscheidende Faktor in den Daten die Entfernung von einem zentralen Punkt oder einer Linie (wie bei Flugstrecken), eignet sich eine *äquidistante* Projektion wie *Plate Carrée*, Cassini, die azimutale äquidistante oder die konische äquidistante Projektion am besten. Ist die Richtung von einem zentralen Punkt aus ein entscheidender Faktor (beispielsweise Flugrouten oder die Ausrichtung von Funkantennen), kommt fast sicher eine der azimutalen Projektionen in Frage. Geeignete Möglichkeiten sind die azimutale äquidistante, die azimutale flächentreue, die stereografische und möglicherweise die orthografische Projektion.

Als Nächstes ist zu betrachten, welchen Teil der Erde die Karte abdecken soll und welche allgemeine Form das betreffende Gebiet aufweist. Für Karten der gesamten Erde sind die zylindrischen flächentreuen Projektionen, die Eckert-IV- und -VI-Projektionen sowie die Mollweide-, Robinson- und Hammer-Projektionen insgesamt eine gute Wahl. Die Mercator-Projektion ist zwar traditionell, doch aufgrund der extremen Flächenverzerrungen in hohen Breiten ist sie eine schlechte Wahl, es sei denn, eine konforme Projektion ist erforderlich. Die Peirce-Projektion ist unter den konformen Projektionen die bessere Wahl, da sie Landformen weniger verzerrt. Die zylindrische Miller-Projektion ist ein Kompromiss, der Formen ähnlich der traditionellen Mercator-Projektion liefern soll, jedoch mit geringerer polarer Streckung. Die Peirce-Quincuncial-Projektion stellt alle Kontinente mit akzeptabler Verzerrung dar, wenn ein Referenzmeridian nahe +20 Grad gewählt wird. Die Robinson-Projektion liefert ansprechende Karten für Dinge wie politische Gliederungen, sollte bei der Darstellung wissenschaftlicher Daten jedoch vermieden werden, da andere Projektionen nützlichere geometrische Eigenschaften haben.

Soll die Karte eine Hemisphäre abdecken, eignen sich die stereografische, die azimutal-äquidistante, die Hammer- oder die Mollweide-Projektion; sie alle bilden die Hemisphäre als Kreis ab.

Soll die Karte ein großes Gebiet abdecken (mindestens einige hundert Kilometer Seitenlänge), jedoch weniger als eine Hemisphäre, stehen mehrere Möglichkeiten zur Auswahl. Azimutale Projektionen eignen sich in der Regel gut (stereografisch, azimutal äquidistant oder azimutal flächentreu nach Lambert, je nachdem, ob Formen, Entfernungen von einem Mittelpunkt oder Flächen von Bedeutung sind). Azimutale Projektionen (und möglicherweise die Cassini-Projektion) sind die einzigen wirklich guten Möglichkeiten für die Kartierung der Polarregionen.

Liegt das große Gebiet in einer der gemäßigten Zonen und ist es rund oder erstreckt es sich vorwiegend in Ost-West-Richtung, sind konische Projektionen eine gute Wahl. Die konforme konische Lambert-Projektion, die äquidistante konische Projektion oder die flächentreue konische Albers-Projektion kommen in Frage, je nachdem, ob Form, Entfernung oder Fläche die wichtigsten Parameter sind. Bei jeder dieser Projektionen sind die Referenzparallelen bei etwa 1/6 und 5/6 des darzustellenden Breitenbereichs zu wählen. So wirken beispielsweise Karten der 48 zusammenhängenden Bundesstaaten der Vereinigten Staaten mit Referenzparallelen von 28,5 und 45,5 Grad besonders ansprechend.

Ist das große Gebiet äquatorial und rund oder erstreckt es sich vorwiegend in Ost-West-Richtung, ist die Mercator-Projektion eine gute Wahl als konforme Projektion; die zylindrische flächentreue

Lambert-Projektion und die sinusförmige Projektion sind gute flächentreue Projektionen; und die *Plate Carrée* ist eine gute äquidistante Projektion.

Große Gebiete mit vorwiegend Nord-Süd-Ausrichtung, insbesondere solche, die den Äquator überspannen, brauchen andere Möglichkeiten. Die Cassini-Projektion ist eine gute Wahl für eine äquidistante Projektion (beispielsweise ergibt eine Cassini-Projektion mit einem Zentralmeridian von 80 Grad West eine ansprechende Karte der amerikanischen Kontinente). Als flächentreue Projektionen kommen die zylindrische flächentreue Projektion, die flächentreue konische Albers-Projektion, die sinusförmige Projektion, die Mollweide-Projektion und die Hammer-Projektion in Frage. Eine gute konforme Projektion für diesen Fall ist die transversale Mercator-Projektion, die leider noch nicht implementiert ist.

Bei kleinen Gebieten beginnt sich die Elliptizität der Erde auf den Kartenmaßstab auszuwirken. Dieses Paket versucht nicht, eine genaue Kartierung großmaßstäblicher topografischer Karten zu leisten. Sind geringfügige Maßstabsfehler in der Anwendung akzeptabel, sollte jede für große Gebiete geeignete Projektion auch für kleine Gebiete geeignet sein.

Es gibt einige Projektionen, die aufgrund ihrer besonderen Eigenschaften berücksichtigt werden. Die orthografische Projektion erzeugt Ansichten der Erde, wie sie aus dem Weltraum zu sehen ist. Die gnomonische Projektion erzeugt eine Karte, auf der alle Großkreise (die kürzeste Entfernung zwischen zwei Punkten auf der Erdoberfläche) als gerade Linien dargestellt werden. Diese Projektion ist zwar für die Navigationsplanung nützlich, weist jedoch extreme Verzerrungen hinsichtlich Form und Fläche auf und kann nur einen begrenzten Bereich der Erde darstellen (deutlich weniger als eine Hemisphäre).

Copyright


```
Copyright (c) 2007 Kevin B. Kenny <kennykb@acm.org>
```


markdown

markdown

Stichwörter: markdown

Name

markdown – Konvertiert Markdown-Text in HTML

Übersicht

package require Tcl 8.5 9
package require Markdown 1.2.4
package require textutil ?**0.8**?
::Markdown::convert *markdown*
::Markdown::register *langspec converter*
::Markdown::get_lang_counter
::Markdown::reset_lang_counter

Beschreibung

Das Paket **Markdown** bietet einen Befehl, um mit Markdown formatierten Text in HTML umzuwandeln.

::Markdown::convert *markdown*
Dieser Befehl nimmt einen Markdown-Textblock entgegen und gibt einen HTML-Block zurück.

Der Konverter unterstützt zwei Arten der Syntaxhervorhebung für umschlossene Code-Blöcke: Hervorhebung über einen registrierten Konverter (siehe **::Markdown::register**) oder reine JavaScript-Hervorhebung, z. B. über "highlight.js", wobei der im Markup verwendete Sprachbezeichner als CSS-Klasse des "code"-Elements im zurückgegebenen Markup festgelegt wird.

::Markdown::register *langspec converter*
Registriert einen sprachspezifischen Konverter zur Aufbereitung eines Codeblocks (z. B. Syntaxhervorhebung). Markdown unterstützt umschlossene Codeblöcke mit einer optionalen Sprachangabe (z. B. "tcl"). Wenn der Markdown-Parser einen solchen Code-Block verarbeitet und ein Konverter für die angegebene Sprachangabe registriert ist, wird der Konverter mit dem rohen Code-Block als Argument aufgerufen. Der Konverter soll als Ergebnis die Markup-Darstellung des Code-Blocks geben. Der angegebene Konverter kann ein beliebiger Tcl-Befehl sein; der rohe Textblock wird beim Aufruf als letztes Argument hinzugefügt.

::Markdown::get_lang_counter
Gibt ein Wörterbuch zurück, das den Sprachbezeichner und die Anzahl der Vorkommen in umschlossenen Codeblöcken enthält. Diese Funktion kann beispielsweise verwendet werden, um festzustellen, ob bestimmte CSS- oder JavaScript-Header für die Darstellung eingebunden werden sollen, ohne dass das gerenderte Ergebnis nachbearbeitet werden muss.

::Markdown::reset_lang_counter
Setzt die Sprachzähler zurück.

Unterstützte Markdown-Syntax

Dieser Markdown-Konverter unterstützt das ursprüngliche Markdown von Gruber und Swartz (weitere Informationen finden Sie auf deren syntax Seite):

- Absätze
- Header im ATX- und Setext-Stil
- Zitate
- Hervorhebung und starke Hervorhebung
- ungeordnete und geordnete Listen
- Inline-Links, Referenz-Links und automatische Links
- Inline- und Referenzbilder
- Inline-Code
- Code-Blöcke (mit vier Einrückungszeichen oder einem Tabulator)
- Inline-HTML
- Backslash-Escape-Zeichen
- horizontale Linien

Darüber hinaus wird die folgende erweiterte Markdown-Syntax unterstützt, die aus PHP Markdown Extra und GFM (Github Flavoured Markdown) stammt:

- Pipe-Tabellen
- eingeschlossene Code-Blöcke (mit optionalem Sprachbezeichner)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

math

bigfloat
Stichwörter: bigfloat, computations, floating-point, interval, math, multiprecision, tcl

Name
math::bigfloat – Gleitkommazahlen mit beliebiger Genauigkeit

Übersicht
package require Tcl 8.5 9
package require math::bigfloat ?**2.0.6**?
fromstr *number* ?*trailingZeros*?
tostr ?**-nosci**? *number*
fromdouble *double* ?*decimals*?
todouble *number*
isInt *number*
isFloat *number*
int2float *integer* ?*decimals*?
add *x y*
sub *x y*
mul *x y*
div *x y*
mod *x y*
abs *x*
opp *x*
pow *x n*
iszero *x*
equal *x y*
compare *x y*
sqrt *x*
log *x*
exp *x*
cos *x*
sin *x*
tan *x*

cotan *x*
acos *x*
asin *x*
atan *x*
cosh *x*
sinh *x*
tanh *x*
pi *n*
rad2deg *radians*
deg2rad *degrees*
round *x*
ceil *x*
floor *x*

Beschreibung

Das Paket "bigfloat" erweitert die Sprache Tcl um Funktionen für Gleitkommaarithmetik mit beliebiger Genauigkeit. Es ist für die Verwendung mit Tcl 8.5 konzipiert; für Tcl 8.4 wird jedoch eine frühere Version dieses Pakets bereitgestellt. Weitere Erläuterungen finden Sie im Abschnitt Wie sieht es mit Tcl 8.4 aus?. Vereinbarungsgemäß bezeichnen wir die in dieser Bibliothek verarbeiteten Zahlen als:

- BigFloat für Gleitkommazahlen beliebiger Länge.
- Ganzzahlen beliebiger Länge mit Vorzeichen, genau wie einfache Ganzzahlen seit Tcl 8.5.

Jeder BigFloat ist ein Intervall, nämlich [*m-d, m+d*], wobei *m* die Mantisse und *d* die Unsicherheit ist, die die Begrenzung der Genauigkeit dieser Zahl darstellt. Aus diesem Grund bezeichnen wir solche mathematischen Verfahren als *Intervallberechnungen*. Nehmen wir ein Beispiel aus der Physik: Wenn Sie eine Temperatur messen, sind nicht alle abgelesenen Ziffern *signifikant*. Manchmal können Sie einfach nicht allen Ziffern vertrauen – ganz zu schweigen davon, ob Double-Werte (Gleitkommazahlen) all diese Ziffern verarbeiten können. BigFloat löst dieses Problem – Sie können den erhaltenen Ziffern vertrauen – und bietet zudem die Möglichkeit, Zahlen mit beliebiger Genauigkeit zu speichern. BigFloats werden intern als Tcl-Listen dargestellt: Dieses Paket stellt eine Reihe von Prozeduren bereit, die auf diese interne Darstellung zugreifen, um:

- mathematische Operationen mit BigFloats und (optional) mit Ganzzahlen durchzuführen,
- BigFloats von ihrer internen Darstellung in Zeichenketten umzuwandeln und umgekehrt.

Einleitung

fromstr *number* ?*trailingZeros*?

Wandelt *number* in einen BigFloat um. Die Genauigkeit entspricht mindestens der Anzahl der in *number* angegebenen Ziffern. Wenn *number* ausschließlich Ziffern enthält und gegebenenfalls ein Minuszeichen vorhanden ist, wird es als Ganzzahl betrachtet. In diesem Fall erfolgt keinerlei Umwandlung.

trailingZeros – die Anzahl der Nullen, die am Ende der Gleitkommazahl angehängt werden sollen, um eine höhere Genauigkeit zu erzielen. Bei einer Ganzzahl ist dies nicht möglich.

```
# x and y are BigFloats : the first string contained a dot, and the
    second an e sign
set x [fromstr -1.000000]
set y [fromstr 2000e30]
# let's see how we get integers
set t 20000000000000
# the old way (package 1.2) is still supported for backwards
    compatibility :
set m [fromstr 10000000000]
# but we do not need fromstr for integers anymore
set n -39
# t, m and n are integers
```

Die letzte Ziffer von *number* gilt für die Prozedur als auf +/-1 genau. Beispielsweise entspricht 1.00 dem Intervall [0.99, 1.01] und 0.43 dem Intervall [0.42, 0.44]. Die Konstante Pi kann durch die Zahl "3.1415" angenähert werden. Diese Zeichenkette würde von **fromstr** als das Intervall [3.1414, 3.1416] aufgefasst. Wenn Sie also 1.0 als Double-Wert meinen, müssen Sie möglicherweise 1.000000 schreiben, um eine ausreichende Genauigkeit zu erzielen. Weitere Informationen zu diesem Thema finden Sie im Abschnitt Genauigkeit.

Beispiel:

```
set x [fromstr 1.0000000000]
# the next line does the same, but smarter
set y [fromstr 1. 10]
```

tostr ?-nosci? ***number***

Gibt einen BigFloat als Zeichenkette zurück, in der alle Ziffern exakt sind. *Alle exakten Ziffern* bedeutet, dass eine Rundung erfolgen kann, beispielsweise auf Null, wenn das Unsicherheitsintervall die tatsächlichen Ziffern nicht eindeutig angibt. *number* kann eine Ganzzahl sein, wodurch der Befehl genau das Eingabeargument zurückgibt. Mit der Option **-nosci** wird die Zahl nie in wissenschaftlicher Notation zurückgegeben, d. h. nicht als '3.4523e+5', sondern als '345230.'.

```
puts [tostr [fromstr 0.99999]] ;# 1.0000
puts [tostr [fromstr 1.00001]] ;# 1.0000
puts [tostr [fromstr 0.002]] ;# 0.e-2
```

Siehe hierzu den Abschnitt Genauigkeit. Siehe auch **iszero** zur Erkennung von Nullen, was bei der Durchführung einer Division hilfreich ist.

fromdouble ***double*** **?*decimals*?**

Konvertiert einen Double-Wert (einen einfachen Gleitkommawert) in einen BigFloat mit genau *decimals* Stellen. Ohne das Argument *decimals* verhält sich der Befehl wie **fromstr**. Wichtig ist hier eigentlich nur die Möglichkeit, BigFloats mit einer festen Anzahl von *decimals* Stellen zu erzeugen.

```
tostr [fromstr 1.111 4]
# returns : 1.111000 (3 zeros)
tostr [fromdouble 1.111 4]
# returns : 1.111
```

todouble *number*
Gibt zu einem BigFloat einen Double-Wert zurück, der in *expr* verwendet werden kann.

isInt *number*
Gibt 1 zurück, wenn *number* eine ganze Zahl ist, andernfalls 0.

isFloat *number*
Gibt 1 zurück, wenn *number* ein BigFloat ist, andernfalls 0.

int2float *integer* ?*decimals*?
Wandelt eine Ganzzahl in einen BigFloat mit *decimals* nachgestellten Nullen um. Die Standardanzahl und die minimale Anzahl von *decimals* beträgt 1. Wenn die Rückwandlung in eine Zeichenkette erfolgt, geht eine Dezimalstelle verloren:

```
set n 10
set x [int2float $n]; # like fromstr 10.0
puts [tostr $x]; # prints "10."
set x [int2float $n 3]; # like fromstr 10.000
puts [tostr $x]; # prints "10.00"
```

Arithmetik

add *x y*
siehe unten ...

sub *x y*
siehe unten ...

mul *x y*
Gibt die Summe, die Differenz und das Produkt von *x* mit *y* zurück. *x* – kann entweder ein BigFloat oder eine Ganzzahl sein; *y* – kann entweder ein BigFloat oder eine Ganzzahl sein. Sind beide Werte Ganzzahlen, verhalten sich diese Befehle wie **expr**.

div *x y*
siehe unten ...

mod *x y*
Gibt den Quotienten und den Rest der Division von *x* durch *y* zurück. Jedes Argument (*x* und *y*) kann entweder ein BigFloat oder eine Ganzzahl sein; eine Ganzzahl darf jedoch nicht durch einen BigFloat dividiert werden. Die Division durch Null führt zu einem Fehler.

abs *x*
Gibt den Betrag (Absolutwert) von *x* zurück

opp *x*
Gibt die Gegenzahl von *x* zurück (-*x*)

pow *x n*
Gibt *x* hoch *n* zurück. Dies funktioniert nur, wenn *n* eine ganze Zahl ist. *x* kann ein BigFloat oder eine ganze Zahl sein.

Vergleiche

iszero *x*

Gibt 1 zurück, wenn *x* Folgendes ist:
- ein BigFloat, der nahe genug an Null liegt, um den Fehler "Division durch Null" auszulösen.
- die ganze Zahl 0.

Hier sehen Sie, wie Zahlen, die nahe bei Null liegen, in Zeichenketten umgewandelt werden:

```
tostr [fromstr 0.001] ; # -> 0.e-2
tostr [fromstr 0.000000] ; # -> 0.e-5
tostr [fromstr -0.000001] ; # -> 0.e-5
tostr [fromstr 0.0] ; # -> 0.
tostr [fromstr 0.002] ; # -> 0.e-2

set a [fromstr 0.002] ; # uncertainty interval : 0.001, 0.003
tostr  $a ; # 0.e-2
iszero $a ; # false

set a [fromstr 0.001] ; # uncertainty interval : 0.000, 0.002
tostr  $a ; # 0.e-2
iszero $a ; # true
```

equal *x y*

Gibt 1 zurück, wenn *x* und *y* gleich sind, andernfalls 0.

compare *x y*

Gibt 0 zurück, wenn beide BigFloat-Argumente gleich sind, 1, wenn *x* größer als *y* ist, und -1, wenn *x* kleiner als *y* ist. Ein Vergleich zwischen einer Ganzzahl und einem BigFloat ist nicht möglich: Die Operanden müssen entweder beide BigFloats oder beide Ganzzahlen sein.

Analyse

sqrt *x*

siehe unten ...

log *x*

siehe unten ...

exp *x*

siehe unten ...

cos *x*

siehe unten ...

sin *x*

siehe unten ...

tan *x*

siehe unten ...

cotan *x*

siehe unten ...

acos *x*

siehe unten ...

asin *x*
siehe unten ...

atan *x*
siehe unten ...

cosh *x*
siehe unten ...

sinh *x*
siehe unten ...

tanh *x*
Die oben genannten Funktionen geben jeweils Folgendes zurück: Quadratwurzel, Logarithmus, Exponentialfunktion, Kosinus, Sinus, Tangens, Kotangens, Arkuskosinus, Arkussinus, Arkustangens, hyperbolischer Kosinus, hyperbolischer Sinus, hyperbolischer Tangens eines BigFloat *x*.

pi *n*
Gibt einen BigFloat zurück, der die Konstante Pi mit *n* Stellen nach dem Komma darstellt. *n* ist eine positive ganze Zahl.

rad2deg *radians*
siehe unten ...

deg2rad *degrees*
radians – Winkel im Bogenmaß (BigFloat)

degrees – Winkel in Grad (BigFloat)

Rechnet einen Winkel vom Bogenmaß in Grad um und *umgekehrt*.

Rundung

round *x*
siehe unten ...

ceil *x*
siehe unten ...

floor *x*
Die oben genannten Funktionen runden den BigFloat *x* wie die gleichnamige mathematische Funktion in *expr* und geben das Ergebnis als Ganzzahl zurück.

Genauigkeit

Wie wirken sich Umwandlungen auf die Genauigkeit aus?

- Wenn ein BigFloat aus einer Zeichenkette konvertiert wird, wird in der internen Darstellung die Unsicherheit auf der Ebene der letzten Ziffer als 1 gespeichert.
- Bei der Berechnung wird die Unsicherheit jedes Ergebnisses intern so realitätsnah wie möglich ermittelt, wodurch Speicherplatz eingespart wird.
- Wenn die Rückkonvertierung in eine Zeichenkette erfolgt, unterliegen die ausgegebenen Ziffern keiner Unsicherheit. Es erfolgt jedoch eine gewisse Rundung, da das Unterlassen einer solchen zu schwerwiegenden Problemen führen würde.

Unsicherheiten bleiben in der internen Darstellung der Zahl erhalten; es wird empfohlen, **tostr**

ausschließlich zur Ausgabe von Daten (auf dem Bildschirm oder in einer Datei) zu verwenden und **fromstr** NIEMALS mit dem Ergebnis von **tostr** aufzurufen. Es ist besser, Operanden stets in ihrer internen Darstellung zu belassen. Aufgrund der internen Funktionsweise dieser Bibliothek kann das Unsicherheitsintervall etwas breiter sein als erwartet, was jedoch keine falschen Ziffern verursachen sollte.

Nun stellen Sie sich vielleicht folgende Frage: Welche Genauigkeit erhalte ich nach dem Aufruf von add, sub, mul oder div? Zunächst erzeugen Sie eine Zahl aus ihrer Zeichenkettendarstellung, wobei nebenbei ihre Unsicherheit festgelegt wird:

```
set a [fromstr 1.230]
# $a belongs to [1.229, 1.231]
set a [fromstr 1.000]
# $a belongs to [0.999, 1.001]
# $a has a relative uncertainty of 0.1% : 0.001(the
    uncertainty)/1.000(the medium value)
```

Die Unsicherheit der Summe oder der Differenz zweier Zahlen entspricht der Summe ihrer jeweiligen Unsicherheiten.

```
set a [fromstr 1.230]
set b [fromstr 2.340]
set sum [add $a $b]]
# the result is : [3.568, 3.572] (the last digit is known with an
    uncertainty of 2)
tostr $sum ; # 3.57
```

Wenn wir jedoch beispielsweise eine ganze Zahl zu einem BigFloat addieren oder von diesem subtrahieren, bleibt die relative Unsicherheit des Ergebnisses unverändert. Daher ist es ratsam, ganze Zahlen nicht in BigFloats umzuwandeln:

```
set a [fromstr 0.999999999]
# now something dangerous
set b [fromstr 2.000]
# the result has only 3 digits
tostr [add $a $b]

# how to keep precision at its maximum
puts [tostr [add $a 2]]
```

Bei der Multiplikation und Division entspricht die relative Unsicherheit des Produkts bzw. des Quotienten der Summe der relativen Unsicherheiten der Operanden. Vorsicht bei der Division durch Null: Prüfen Sie jeden Divisor mit **iszero**.

```
set num [fromstr 4.00]
set denom [fromstr 0.01]

puts [iszero $denom];# true
set quotient [div $num $denom];# error : divide by zero
```

```
# opposites of our operands
puts [compare $num [opp $num]]; # 1
puts [compare $denom [opp $denom]]; # 0 !!!
# No suprise ! 0 and its opposite are the same...
```

Auswirkungen der Genauigkeit einer Zahl, die als gleich Null angesehen wird, auf die cos-Funktion:

```
puts [tostr [cos [fromstr 0. 10]]]; # -> 1.000000000
puts [tostr [cos [fromstr 0. 5]]]; # -> 1.0000
puts [tostr [cos [fromstr 0e-10]]]; # -> 1.000000000
puts [tostr [cos [fromstr 1e-10]]]; # -> 1.000000000
```

BigFloats mit unterschiedlichen internen Darstellungen können in dieselbe Zeichenkette konvertiert werden.

Bei den meisten Analysefunktionen (Kosinus, Quadratwurzel, Logarithmus usw.) ist es schwierig, die Genauigkeit des Ergebnisses zu bestimmen. Es scheint jedoch, dass der Genauigkeitsverlust des Ergebnisses in vielen Fällen nur eine oder zwei Ziffern beträgt. Es gibt einige Ausnahmen: zum Beispiel,

```
tostr [exp [fromstr 100.0 10]]
# returns : 2.688117142e+43 which has only 10 digits of precision,
    although the entry
# has 14 digits of precision.
```

Wie sieht es mit Tcl 8.4 aus?

Wenn Ihre Systemumgebung Tcl 8.5 nicht bereitstellt, aber 8.4 unterstützt, kann das Paket dennoch geladen werden; es wird dann auf *math::bigfloat* 1.2 zurückgeschaltet. Tatsächlich wird eine wichtige, in Tcl 8.5 eingeführte Funktion benötigt – die Fähigkeit, mit großen Zahlen (Bignums) umzugehen, was wir mit **expr** bewerkstelligen können. Vor Version 8.5 wurde diese Funktion von mehreren Paketen bereitgestellt, darunter das reine Tcl-Paket *math::bignum*, das von *tcllib* bereitgestellt wird. In diesem Fall müssen Sie lediglich wissen, dass die Argumente der hier erläuterten Befehle in ihrer internen Darstellung vorliegen müssen. Selbst bei Ganzzahlen müssen Sie also **fromstr** und **tostr** aufrufen, um sie zwischen Zeichenkette und interner Darstellung umzuwandeln.

```
#
# with Tcl 8.5
# ============
set a [pi 20]
# round returns an integer and 'everything is a string' applies to
    integers
# whatever big they are
puts [round [mul $a 10000000000]]
#
# the same with Tcl 8.4
# =====================
set a [pi 20]
```

```
# bignums (arbitrary length integers) need a conversion hook
set b [fromstr 10000000000]
# round returns a bignum:
# before printing it, we need to convert it with 'tostr'
puts [tostr [round [mul $a $b]]]
```

Namespaces und andere Pakete

Wir haben das Thema Namespaces bisher noch nicht behandelt, da wir davon ausgegangen sind, dass Sie öffentliche Befehle wie folgt in den globalen Namespace importiert haben:

```
namespace import ::math::bigfloat::*
```

Wenn es Ihnen besonders wichtig ist, Namenskonflikte zu vermeiden, bin ich der Ansicht, dass dies auf folgende Weise gelöst werden sollte:

```
package require math::bigfloat
# beware: namespace ensembles are not available in Tcl 8.4
namespace eval ::math::bigfloat {namespace ensemble create -command
    ::bigfloat}
# from now, the bigfloat command takes as subcommands all original
    math::bigfloat::* commands
set a [bigfloat sub [bigfloat fromstr 2.000] [bigfloat fromstr 0.530]]
puts [bigfloat tostr $a]
```

Beispiele

Raten Sie mal, was passiert, wenn Sie sich mit Astronomie beschäftigen. Hier ein Beispiel:

```
# convert acurrate angles with a millisecond-rated accuracy
proc degree-angle {degrees minutes seconds milliseconds} {
    set result 0
    set div 1
    foreach factor {1 1000 60 60} var [list $milliseconds $seconds
    $minutes $degrees] {
        # we convert each entry var into milliseconds
        set div [expr {$div*$factor}]
        incr result [expr {$var*$div}]
    }
    return [div [int2float $result] $div]
}
# load the package
package require math::bigfloat
namespace import ::math::bigfloat::*
# work with angles : a standard formula for navigation (taking
    bearings)
set angle1 [deg2rad [degree-angle 20 30 40   0]]
set angle2 [deg2rad [degree-angle 21  0 50 500]]
set opposite3 [deg2rad [degree-angle 51  0 50 500]]
```

```
set sinProduct [mul [sin $angle1] [sin $angle2]]
set cosProduct [mul [cos $angle1] [cos $angle2]]
set angle3 [asin [add [mul $sinProduct [cos $opposite3]] $cosProduct]]
puts "angle3 : [tostr [rad2deg $angle3]]"
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: bignum :: float* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright


```
Copyright (c) 2004-2008, by Stephane Arnold <stephanearnold at yahoo
    dot fr>
```


bignum

Stichwörter: bignum, bignums, math, multiprecision, tcl

Name

math::bignum – Ganzzahlen mit beliebiger Genauigkeit

Übersicht

package require Tcl ?**8.5 9**?
package require math::bignum ?**3.1.2**?
::math::bignum::fromstr *string* ?*radix*?
::math::bignum::tostr *bignum* ?*radix*?
::math::bignum::sign *bignum*
::math::bignum::abs *bignum*
::math::bignum::cmp *a b*
::math::bignum::iszero *bignum*
::math::bignum::lt *a b*
::math::bignum::le *a b*
::math::bignum::gt *a b*
::math::bignum::ge *a b*
::math::bignum::eq *a b*
::math::bignum::ne *a b*
::math::bignum::isodd *bignum*
::math::bignum::iseven *bignum*
::math::bignum::add *a b*

::math::bignum::sub *a b*
::math::bignum::mul *a b*
::math::bignum::divqr *a b*
::math::bignum::div *a b*
::math::bignum::rem *a b*
::math::bignum::mod *n m*
::math::bignum::pow *base exp*
::math::bignum::powm *base exp m*
::math::bignum::sqrt *bignum*
::math::bignum::rand *bits*
::math::bignum::lshift *bignum bits*
::math::bignum::rshift *bignum bits*
::math::bignum::bitand *a b*
::math::bignum::bitor *a b*
::math::bignum::bitxor *a b*
::math::bignum::setbit *bignumVar bit*
::math::bignum::clearbit *bignumVar bit*
::math::bignum::testbit *bignum bit*
::math::bignum::bits *bignum*

Beschreibung

Das Paket "bignum" erweitert die Programmiersprache Tcl um Funktionen für die Ganzzahlberechnung mit beliebiger Genauigkeit (auch als "Big Numbers" bezeichnet). "Big Numbers" werden intern als Tcl-Listen dargestellt: Dieses Paket bietet eine Reihe von Prozeduren, die auf diese interne Darstellung zugreifen, um:

- mathematische Operationen durchführen
- Bignums von der internen Darstellung in eine Zeichenkette mit der gewünschten Basis umwandeln und umgekehrt.

Die beiden Konstanten "0" und "1" werden jedoch automatisch in die interne Darstellung umgewandelt, um den Vergleich einer Zahl mit Null oder die Inkrementierung einer großen Zahl zu vereinfachen.

Die Bignum-Schnittstelle ist undurchsichtig, sodass Operationen mit Bignums, die nicht von Prozeduren dieses Pakets zurückgegeben (sondern manuell erstellt) werden, zu nicht spezifiziertem Verhalten führen können. Es ist unbedenklich, Bignums als reine Werte zu behandeln; daher besteht keine Notwendigkeit, ein Bignum freizugeben oder es über eine spezielle Operation zu duplizieren.

Beispiele

In diesem Abschnitt werden einige einfache Beispiele vorgestellt. Da diese Bibliothek lediglich dazu dient, mathematische Operationen durchzuführen, sind Beispiele möglicherweise der einfachste Weg, um den Umgang damit zu erlernen. Informationen zu den einzelnen Prozeduren finden Sie im Abschnitt "API" dieser Manpage.

```
    package require math::bignum

    # Multiplication of two bignums
    set a [::math::bignum::fromstr 88888881111111]
    set b [::math::bignum::fromstr 22222220000000]
    set c [::math::bignum::mul $a $b]
    puts [::math::bignum::tostr $c] ; # => will output
    1975308271604953086420000000
    set c [::math::bignum::sqrt $c]
    puts [::math::bignum::tostr $c] ; # => will output 44444440277777

    # From/To string conversion in different radix
    set a [::math::bignum::fromstr 11000101010101110010011111010111 2]
    puts [::math::bignum::tostr $a 16] ; # => will output 62ab93d7

    # Factorial example
    proc fact n {
        # fromstr is not needed for 0 and 1
        set z 1
        for {set i 2} {$i <= $n} {incr i} {
            set z [::math::bignum::mul $z [::math::bignum::fromstr $i]]
        }
        return $z
    }

    puts [::math::bignum::tostr [fact 100]]
```

API

::math::bignum::fromstr *string* ?*radix*?

Konvertiert *string* in eine Bignum-Zahl. Wird *radix* weggelassen oder ist es gleich Null, wird die Zeichenkette als Hexadezimalzahl interpretiert, wenn ihr *0x* vorangestellt ist, als Oktalzahl, wenn ihr *ox* vorangestellt ist, als Binärzahl, wenn ihr *bx* vorangestellt ist, und andernfalls als Zahl zur Basis 10. Wenn stattdessen das Argument *radix* im Bereich von 2 bis 36 festgelegt wird, wird die Zeichenkette *string* in der angegebenen Basis interpretiert. Beachten Sie, dass diese Konvertierung für zwei Konstanten nicht erforderlich ist: *0* und *1*. (siehe Beispiele)

::math::bignum::tostr *bignum* ?*radix*?

Konvertiert *bignum* in eine Zeichenkette, die die Zahl in der angegebenen Basis darstellt. Wird *radix* weggelassen, gilt als Standardwert 10.

::math::bignum::sign *bignum*

Gibt das Vorzeichen der Bignum-Zahl zurück. Die Prozedur gibt 0 zurück, wenn die Zahl positiv ist, und 1, wenn sie negativ ist.

::math::bignum::abs *bignum*

Gibt den Betrag (Absolutwert) der Bignum-Zahl zurück.

::math::bignum::cmp *a b*

Vergleicht die beiden Bignums a und b und gibt *0* zurück, wenn *a* == *b* gilt, *1*, wenn *a* > *b* gilt, und *-1*, wenn *a* < *b* gilt.

::math::bignum::iszero *bignum*
Gibt "true" zurück, wenn der Wert von *bignum* Null ist; andernfalls wird "false" zurückgegeben.

::math::bignum::lt *a b*
Gibt "true" zurück, wenn *a* < *b* ist, andernfalls "false".

::math::bignum::le *a b*
Gibt "true" zurück, wenn *a* <= *b* gilt, andernfalls "false".

::math::bignum::gt *a b*
Gibt "true" zurück, wenn *a* > *b* ist, andernfalls "false".

::math::bignum::ge *a b*
Gibt "true" zurück, wenn *a* >= *b* gilt, andernfalls "false".

::math::bignum::eq *a b*
Gibt "true" zurück, wenn *a* == *b* gilt, andernfalls "false".

::math::bignum::ne *a b*
Gibt "true" zurück, wenn *a!*= *b* gilt, andernfalls "false".

::math::bignum::isodd *bignum*
Gibt "true" zurück, wenn *bignum* ungerade ist.

::math::bignum::iseven *bignum*
Gibt "true" zurück, wenn *bignum* gerade ist.

::math::bignum::add *a b*
Gibt die Summe der beiden Bignums *a* und *b* zurück.

::math::bignum::sub *a b*
Gibt die Differenz der beiden Bignums *a* und *b* zurück.

::math::bignum::mul *a b*
Gibt das Produkt der beiden Bignums *a* und *b* zurück. Die Implementierung verwendet die Karatsuba-Multiplikation, wenn beide Zahlen größer als ein vorgegebener Schwellenwert sind; andernfalls kommt der direkte Algorithmus zum Einsatz.

::math::bignum::divqr *a b*
Gibt eine Liste mit zwei Elementen zurück, deren erstes Element den Quotienten der Division der beiden Bignums *a* und *b* enthält und deren zweites Element den Rest dieser Division enthält.

::math::bignum::div *a b*
Gibt den Quotienten der Division der beiden Bignums *a* und *b* zurück.

::math::bignum::rem *a b*
Gibt den Rest der Division der beiden Bignums *a* und *b* zurück.

::math::bignum::mod *n m*
Gibt *n* modulo *m* zurück. Diese Operation wird als modulare Reduktion bezeichnet.

::math::bignum::pow *base exp*
Gibt *base* hoch *exp* zurück.

::math::bignum::powm *base exp m*
Gibt *base* hoch *exp* modulo *m* zurück. Diese Funktion wird häufig im Bereich der Kryptografie verwendet.

::math::bignum::sqrt *bignum*
Gibt den ganzzahligen Teil der Quadratwurzel von *bignum* zurück.

::math::bignum::rand *bits*
Gibt eine Zufallszahl mit einer Länge von höchstens *bits* Bits zurück. Die zurückgegebene Zahl wird intern mithilfe der Tcl-Funktion *expr rand()* generiert und ist nicht geeignet, wenn eine nicht erratbare, kryptografisch sichere Zufallszahl benötigt wird.

::math::bignum::lshift *bignum bits*
Gibt das Ergebnis der Linksverschiebung der binären Darstellung von *bignum* um *bits* Stellen nach links zurück. Dies entspricht einer Multiplikation mit 2^*bits*, ist jedoch wesentlich schneller.

::math::bignum::rshift *bignum bits*
Gibt das Ergebnis einer Rechtsverschiebung der binären Darstellung von *bignum* um *bits* Stellen nach rechts zurück. Dies entspricht einer Division durch *2^bits*, ist jedoch wesentlich schneller.

::math::bignum::bitand *a b*
Gibt das Ergebnis einer bitweisen UND-Operation mit a und b zurück. Die Operation ist auf positive Zahlen beschränkt, einschließlich Null. Werden negative Zahlen als Argumente übergeben, ist das Ergebnis undefiniert.

::math::bignum::bitor *a b*
Gibt das Ergebnis einer bitweisen ODER-Operation zwischen a und b zurück. Die Operation ist auf positive Zahlen beschränkt, einschließlich Null. Werden negative Zahlen als Argumente übergeben, ist das Ergebnis undefiniert.

::math::bignum::bitxor *a b*
Gibt das Ergebnis einer bitweisen XOR-Operation zwischen a und b zurück. Die Operation ist auf positive Zahlen, einschließlich Null, beschränkt. Werden negative Zahlen als Argumente übergeben, ist das Ergebnis undefiniert.

::math::bignum::setbit *bignumVar bit*
Setzt das Bit an der Position *bit* in der in der Variablen *bignumVar* gespeicherten Bignum-Zahl auf 1. Bit 0 ist das niedrigstwertige Bit.

::math::bignum::clearbit *bignumVar bit*
Setzt das Bit an der Position *bit* in der in der Variablen *bignumVar* gespeicherten Bignum-Zahl auf 0. Bit 0 ist das niedrigstwertige Bit.

::math::bignum::testbit *bignum bit*
Gibt "true" zurück, wenn das Bit an der Position *bit* von *bignum* gesetzt ist; andernfalls wird "false" zurückgegeben. Liegt *bit* außerhalb des zulässigen Bereichs, wird es als auf Null gesetzt betrachtet.

::math::bignum::bits *bignum*
Gibt die Anzahl der Bits zurück, die zur Darstellung von bignum zur Basis 2 erforderlich sind.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: bignum* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

Copyright
```
Copyright (c) 2004 Salvatore Sanfilippo <antirez at invece dot org>
Copyright (c) 2004 Arjen Markus <arjenmarkus at users dot sourceforge
    dot net>
```

calculus
Stichwörter: calculus, differential equations, integration, math, roots

Name
math::calculus – Integralrechnung und gewöhnliche Differentialgleichungen

Übersicht
package require Tcl 8.5 9
package require math::calculus 1.1
::math::calculus::integral *begin end nosteps func*
::math::calculus::integralExpr *begin end nosteps expression*
::math::calculus::integral2D *xinterval yinterval func*
::math::calculus::integral2D_accurate *xinterval yinterval func*
::math::calculus::integral3D *xinterval yinterval zinterval func*
::math::calculus::integral3D_accurate *xinterval yinterval zinterval func*
::math::calculus::qk15 *xstart xend func nosteps*
::math::calculus::qk15_detailed *xstart xend func nosteps*
::math::calculus::eulerStep *t tstep xvec func*
::math::calculus::heunStep *t tstep xvec func*
::math::calculus::rungeKuttaStep *t tstep xvec func*
::math::calculus::boundaryValueSecondOrder *coeff_func force_func leftbnd rightbnd nostep*
::math::calculus::solveTriDiagonal *acoeff bcoeff ccoeff dvalue*
::math::calculus::newtonRaphson *func deriv initval*
::math::calculus::newtonRaphsonParameters *maxiter tolerance*
::math::calculus::regula_falsi *f xb xe eps*
::math::calculus::root_bisection *f xb xe eps*
::math::calculus::root_secant *f xb xe eps*
::math::calculus::root_brent *f xb xe eps*
::math::calculus::root_chandrupatla *f xb xe eps*

Beschreibung
Dieses Paket implementiert mehrere einfache mathematische Algorithmen:

- Die Integration einer Funktion über ein Intervall

- Die numerische Integration eines Systems gewöhnlicher Differentialgleichungen.
- Die Schätzung der Nullstelle(n) einer Gleichung in einer Variablen.

Das Paket ist vollständig in Tcl implementiert. Der Genauigkeit der Berechnungen wurde keine besondere Aufmerksamkeit gewidmet. Stattdessen wurden bekannte Algorithmen auf unkomplizierte Weise verwendet.

Dieses Dokument beschreibt die Prozeduren und erläutert deren Anwendung.

Prozeduren

Dieses Paket definiert die folgenden öffentlichen Prozeduren:

::math::calculus::integral *begin end nosteps func*

Bestimmt das Integral der angegebenen Funktion unter Verwendung der Simpson-Regel. Das Integrationsintervall lautet [*begin*, *end*]. Die übrigen Argumente lauten:

nosteps

Anzahl der Schritte, in die das Intervall unterteilt wird.

func

Zu integrierende Funktion. Sie sollte ein einziges Argument entgegennehmen.

::math::calculus::integralExpr *begin end nosteps expression*

Ähnlich wie die vorherige Prozedur berechnet diese das Integral des angegebenen Ausdrucks *expression* unter Verwendung der Simpson-Regel. Das Integrationsintervall lautet [*begin*, *end*]. Die übrigen Argumente sind:

nosteps

Anzahl der Schritte, in die das Intervall unterteilt wird.

expression

Zu integrierender Ausdruck. Er sollte die Variable "x" als einzige Variable verwenden (den "Integranden").

::math::calculus::integral2D *xinterval yinterval func*

siehe unten ...

::math::calculus::integral2D_accurate *xinterval yinterval func*

Die Befehle **integral2D** und **integral2D_accurate** berechnen das Integral einer Funktion zweier Variablen über das Rechteck, das durch die ersten beiden Argumente vorgegeben ist, bei denen es sich jeweils um eine Liste mit drei Elementen handelt: Anfang und Ende des Intervalls für die Variable sowie die Anzahl der Schritte.

Der Befehl **integral2D** wertet die Funktion im Mittelpunkt jedes Rechtecks aus, während der Befehl **integral2D_accurate** eine Vier-Punkte-Quadraturformel verwendet. Dies führt zu einer exakten Integration von Polynomen dritten Grades oder niedriger.

Die Funktion muss zwei Argumente entgegennehmen und den Wert der Funktion zurückgeben.

::math::calculus::integral3D *xinterval yinterval zinterval func*

siehe unten ...

::math::calculus::integral3D_accurate *xinterval yinterval zinterval func*

Die Befehle **integral3D** und **integral3D_accurate** entsprechen den Befehlen **integral2D**

und **integral3D_accurate** im dreidimensionalen Raum. Die Funktion *func* nimmt drei Argumente entgegen und wird über den durch drei Intervalle gegebenen Block im 3D-Raum integriert.

::math::calculus::qk15 *xstart xend func nosteps*

Bestimmt das Integral der angegebenen Funktion unter Verwendung der 15-Punkte-Quadraturregel nach Gauss-Kronrod. Der Rückgabewert ist die Schätzung des Integrals über das Intervall [*xstart*, *xend*]. Die übrigen Argumente sind:

func
Zu integrierende Funktion. Sie sollte ein einziges Argument entgegennehmen.

?nosteps?
Anzahl der Schritte, in die das Intervall unterteilt wird. Der Standardwert ist 1.

::math::calculus::qk15_detailed *xstart xend func nosteps*

Bestimmt das Integral der angegebenen Funktion unter Verwendung der 15-Punkte-Quadraturregel nach Gauss-Kronrod. Das Integrationsintervall lautet [*xstart*, *xend*]. Die Prozedur gibt eine Liste mit vier Werten zurück:
- Die Schätzung des Integrals über das angegebene Intervall (I).
- Eine Schätzung des absoluten Fehlers in I.
- Die Schätzung des Integrals des Betrags der Funktion über das Intervall.
- Die Schätzung des Integrals des Betrags der Funktion abzüglich ihres Mittelwerts über das Intervall.

Die übrigen Argumente lauten wie folgt:

func
Zu integrierende Funktion. Sie sollte ein einziges Argument entgegennehmen.

?nosteps?
Anzahl der Schritte, in die das Intervall unterteilt wird. Der Standardwert ist 1.

::math::calculus::eulerStep *t tstep xvec func*

Führt einen einzelnen Schritt bei der numerischen Integration eines Differentialgleichungssystems aus. Als Methode wird das Euler-Verfahren verwendet.

t
Wert der unabhängigen Variablen (in der Regel die Zeit) zu Beginn des Schritts.

tstep
Schrittweite für die unabhängige Variable.

xvec
Liste (Vektor) abhängiger Werte

func
Funktion von t und den abhängigen Werten, die eine Liste der Ableitungen der abhängigen Werte zurückgibt. (Die Längen von xvec und des Rückgabewerts von "func" müssen übereinstimmen.)

::math::calculus::heunStep *t tstep xvec func*

Führt einen einzelnen Schritt bei der numerischen Integration eines Differentialgleichungssystems aus. Als Methode wird das Heun-Verfahren verwendet.

t

Wert der unabhängigen Variablen (in der Regel die Zeit) zu Beginn des Schritts.

tstep
Schrittweite für die unabhängige Variable.

xvec
Liste (Vektor) abhängiger Werte

func
Funktion von t und den abhängigen Werten, die eine Liste der Ableitungen der abhängigen Werte zurückgibt. (Die Längen von xvec und des Rückgabewerts von "func" müssen übereinstimmen.)

::math::calculus::rungeKuttaStep *t tstep xvec func*

Führt einen einzelnen Schritt bei der numerischen Integration eines Differentialgleichungssystems aus. Als Methode wird die Runge-Kutta-Methode 4. Ordnung verwendet.

t
Wert der unabhängigen Variablen (in der Regel die Zeit) zu Beginn des Schritts.

tstep
Schrittweite für die unabhängige Variable.

xvec
Liste (Vektor) abhängiger Werte

func
Funktion von t und den abhängigen Werten, die eine Liste der Ableitungen der abhängigen Werte zurückgibt. (Die Längen von xvec und des Rückgabewerts von "func" müssen übereinstimmen.)

::math::calculus::boundaryValueSecondOrder *coeff_func force_func leftbnd rightbnd nostep*

Löst eine lineare Differentialgleichung zweiter Ordnung mit Randwerten an beiden Seiten. Die Gleichung muss folgende Form haben (die "konservative" Form):

```
d      dy     d
-- A(x)--  +  -- B(x)y + C(x)y  =  D(x)
dx     dx     dx
```

Normalerweise würde eine solche Gleichung wie folgt geschrieben werden:

```
    d2y        dy
a(x)---  + b(x)-- + c(x) y  =  D(x)
    dx2        dx
```

Die erste Form lässt sich leichter diskretisieren (durch Integration über ein endliches Volumen) als die zweite Form. Der Zusammenhang zwischen den beiden Formen ist recht einfach:

```
A(x)  =  a(x)
B(x)  =  b(x) - a'(x)
C(x)  =  c(x) - B'(x)  =  c(x) - b'(x) + a''(x)
```

Wegen der Differentiation ist es jedoch wesentlich einfacher, den Benutzer zu bitten, die Funktionen A, B und C direkt anzugeben.

coeff_func
Prozedur, die die drei Koeffizienten (A, B, C) der Gleichung zurückgibt und als einziges Argument die x-Koordinate entgegennimmt.

force_func
Prozedur, die die rechte Seite (D) als Funktion der x-Koordinate zurückgibt.

leftbnd
Eine Liste mit zwei Werten: die x-Koordinate der linken Grenze und der Wert an dieser Grenze.

rightbnd
Eine Liste mit zwei Werten: die x-Koordinate der rechten Grenze und der Wert an dieser Grenze.

nostep
Anzahl der Schritte, in die das Intervall unterteilt werden soll. Die Prozedur gibt eine Liste der x-Koordinaten und der Näherungswerte der Lösung zurück.

::math::calculus::solveTriDiagonal *acoeff bcoeff ccoeff dvalue*

Löst ein System linearer Gleichungen Ax = b, wobei A eine tridiagonale Matrix ist. Gibt die Lösung als Liste zurück.

acoeff
Liste der Werte auf der unteren Diagonalen

bcoeff
Liste der Werte auf der Hauptdiagonalen

ccoeff
Liste der Werte auf der oberen Diagonalen

dvalue
Liste der Werte auf der rechten Seite

::math::calculus::newtonRaphson *func deriv initval*

Bestimmt die Nullstelle einer Gleichung der Form

```
    func(x) = 0
```

unter Verwendung der Newton-Raphson-Methode. Die Prozedur benötigt die folgenden Argumente:

func
Prozedur, die den Wert der Funktion an der Stelle x zurückgibt

deriv
Prozedur, die die Ableitung der Funktion an der Stelle x zurückgibt

initval
Anfangswert für x

::math::calculus::newtonRaphsonParameters *maxiter tolerance*

Legt die numerischen Parameter für die Newton-Raphson-Methode fest:

maxiter

Maximale Anzahl der Iterationsschritte (Standardwert: 20)

tolerance
Relative Genauigkeit (Standardwert: 0.001)

::math::calculus::regula_falsi *f xb xe eps*
Gibt eine Schätzung der Nullstelle bzw. einer der Nullstellen der Funktion zurück, die im Intervall [xb,xe] liegt. Der Fehler dieser Schätzung liegt in der Größenordnung von eps*abs(xe-xb), wobei der tatsächliche Fehler geringfügig größer sein kann.

Die verwendete Methode ist die sogenannte *regula falsi* (*false position*). Es handelt sich um eine unkomplizierte Implementierung. Die Methode ist robust, setzt jedoch voraus, dass das Intervall eine Nullstelle oder zumindest eine ungerade Anzahl von Nullstellen einschließt, sodass der Wert der Funktion am Anfang ein anderes Vorzeichen hat als der Wert am Ende.

Im Gegensatz zum Newton-Raphson-Verfahren ist die Berechnung der Ableitung der Funktion nicht erforderlich.

command *f*
Name des Befehls, der die Funktion auswertet, deren Nullstelle zurückgegeben werden soll

float *xb*
Beginn des Intervalls, in dem die Nullstelle liegen soll

float *xe*
Ende des Intervalls

float *eps*
Zulässiger relativer Fehler (Standardwert: 1.0e-4)

::math::calculus::root_bisection *f xb xe eps*
Gibt eine Schätzung der Nullstelle bzw. einer der Nullstellen der Funktion zurück, die im Intervall [xb,xe] liegt. Der Fehler dieser Schätzung liegt in der Größenordnung von eps*abs(xe-xb); der tatsächliche Fehler kann geringfügig größer sein.

Als Methode wird die sogenannte *Bisektion* verwendet. Zu den Eigenschaften siehe die *regula falsi*-Prozedur.

command *f*
Name des Befehls, der die Funktion auswertet, deren Nullstelle zurückgegeben werden soll

float *xb*
Beginn des Intervalls, in dem die Nullstelle liegen soll

float *xe*
Ende des Intervalls

float *eps*
Zulässiger relativer Fehler (Standardwert: 1.0e-7)

::math::calculus::root_secant *f xb xe eps*
Gibt eine Schätzung der Nullstelle bzw. einer der Nullstellen der Funktion im Intervall [xb,xe] zurück. Der Fehler dieser Schätzung liegt in der Größenordnung von eps*abs(xe-xb); der tatsächliche Fehler kann geringfügig größer sein.

Als Methode wird die sogenannte *Sekantenmethode* verwendet. Zu den Eigenschaften: siehe die *regula falsi*-Prozedur. Hinweis: Diese Methode liefert nicht garantiert eine Nullstelle, ist

aber schnell.

command *f*
Name des Befehls, der die Funktion auswertet, deren Nullstelle zurückgegeben werden soll

float *xb*
Beginn des Intervalls, in dem die Nullstelle liegen soll

float *xe*
Ende des Intervalls

float *eps*
Zulässiger relativer Fehler (Standardwert: 1.0e-7)

::math::calculus::root_brent *f xb xe eps*
Gibt eine Schätzung der Nullstelle bzw. einer der Nullstellen der Funktion im Intervall [xb,xe] zurück. Der Fehler dieser Schätzung liegt in der Größenordnung von eps*abs(xe-xb); der tatsächliche Fehler kann geringfügig größer sein.

Als Methode wird die sogenannte *Brent*-Methode verwendet. Zu den Eigenschaften: siehe die *regula falsi*-Prozedur.

command *f*
Name des Befehls, der die Funktion auswertet, deren Nullstelle zurückgegeben werden soll

float *xb*
Beginn des Intervalls, in dem die Nullstelle liegen soll

float *xe*
Ende des Intervalls

float *eps*
Zulässiger relativer Fehler (Standardwert: 1.0e-7)

::math::calculus::root_chandrupatla *f xb xe eps*
Gibt eine Schätzung der Nullstelle bzw. einer der Nullstellen der Funktion im Intervall [xb,xe] zurück. Der Fehler dieser Schätzung liegt in der Größenordnung von eps*abs(xe-xb); der tatsächliche Fehler kann geringfügig größer sein.

Die verwendete Methode ist die sogenannte *Chandrupatla*-Methode. Sie ist eine Variante der Brent-Methode und gilt als schneller. Eine Erörterung finden Sie unter https://www.embeddedrelated.com/showarticle/855.php.

command *f*
Name des Befehls, der die Funktion auswertet, deren Nullstelle zurückgegeben werden soll

float *xb*
Beginn des Intervalls, in dem die Nullstelle liegen soll

float *xe*
Ende des Intervalls

float *eps*
Zulässiger relativer Fehler (Standardwert: 1.0e-7)

Notes:

Einige der oben genannten Prozeduren nehmen die *Namen* von Prozeduren als Argumente entgegen. Um Probleme mit der *Sichtbarkeit* dieser Prozeduren zu vermeiden, wird der vollqualifizierte Name dieser Prozeduren innerhalb der calculus-Routinen ermittelt. Für den

Benutzer hat dies nur eine Konsequenz: Die benannte Prozedur muss in der aufrufenden Prozedur sichtbar sein. Zum Beispiel:

```
    namespace eval ::mySpace {
       namespace export calcfunc
       proc calcfunc { x } { return $x }
    }
    #
    # Use a fully-qualified name
    #
    namespace eval ::myCalc {
       proc detIntegral { begin end } {
          return [integral $begin $end 100 ::mySpace::calcfunc]
       }
    }
    #
    # Import the name
    #
    namespace eval ::myCalc {
       namespace import ::mySpace::calcfunc
       proc detIntegral { begin end } {
          return [integral $begin $end 100 calcfunc]
       }
    }
```

Erweiterungen für das Randwertproblem zweiter Ordnung:

- Andere Arten von Randbedingungen (Nullgradient, Nullfluss)
- Eine weitere Schematisierung des Terms erster Ordnung (derzeit werden zentrale Differenzen verwendet, doch könnten auch stromaufwärts gerichtete Differenzen sinnvoll sein).
- Die verschiedenen Methoden zur Nullstellenbestimmung unterscheiden sich hinsichtlich ihrer Robustheit, Benutzerfreundlichkeit und Konvergenzgeschwindigkeit. Das Newton-Raphson-Verfahren ist zwar recht schnell (quadratische Konvergenz), liefert jedoch nicht garantiert ein Ergebnis. Die Methoden, die die Nullstelle in einem Intervall einschließen, erfordern zwei Startpunkte, liefern jedoch – mit Ausnahme des Sekantenverfahrens – garantiert eine gute Schätzung.

Beispiele

Lassen Sie uns einige einfache Beispiele betrachten:

Integrieren Sie x über das Intervall [0,100] (20 Schritte):

```
proc linear_func { x } { return $x }
puts "Integral: [::math::calculus::integral 0 100 20 linear_func]"
```

Bei einfachen Funktionen könnte die Alternative wie folgt lauten:

```
puts "Integral: [::math::calculus::integralExpr 0 100 20 {$x}]"
```

Vergessen Sie die geschweiften Klammern nicht!

Die Differentialgleichung für einen gedämpften Oszillator:

```
x'' + rx' + wx = 0
```

lässt sich in ein System von Gleichungen erster Ordnung zerlegen:

```
x' = y
y' = -ry - wx
```

Dann lässt sich dieses System mit folgendem Code lösen:

```
proc dampened_oscillator { t xvec } {
   set x  [lindex $xvec 0]
   set x1 [lindex $xvec 1]
   return [list $x1 [expr {-$x1-$x}]]
}

set xvec   { 1.0 0.0 }
set t      0.0
set tstep  0.1
for { set i 0 } { $i < 20 } { incr i } {
   set result [::math::calculus::eulerStep $t $tstep $xvec
    dampened_oscillator]
   puts "Result ($t): $result"
   set t      [expr {$t+$tstep}]
   set xvec   $result
}
```

Nehmen wir an, wir haben das Randwertproblem:

```
    Dy'' + ky = 0
    x = 0: y = 1
    x = L: y = 0
```

Dieses Randwertproblem könnte auf die Diffusion einer zerfallenden Substanz zurückzuführen sein.

Das lässt sich mit dem folgenden Fragment lösen:

```
   proc coeffs { x } { return [list $::Diff 0.0 $::decay] }
   proc force  { x } { return 0.0 }

   set Diff   1.0e-2
   set decay  0.0001
   set length 100.0

   set y [::math::calculus::boundaryValueSecondOrder \
      coeffs force {0.0 1.0} [list $length 0.0] 100]
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden

oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: calculus* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

Copyright

```
Copyright (c) 2002,2003,2004 Arjen Markus
```


changepoint
Stichwörter: changepoint, control, statistics

Name
math::changepoint – Methoden zur Erkennung von Änderungspunkten

Übersicht
package require Tcl 8.6 9
package require TclOO
package require math::statistics
package require math::changepoint ?**0.2**?
::math::changepoint::cusum-detect *data* ?*args*?
::math::changepoint::cusum-online ?*args*?
$cusumObj examine *value*
$cusumObj reset
::math::changepoint::binary-segmentation *data* ?*args*?

Beschreibung
Das Paket **math::changepoint** implementiert eine Reihe bekannter Methoden, um festzustellen, ob eine Datenreihe eine Verschiebung des Mittelwerts aufweist oder nicht. Hinweis: Diese Methoden zeigen lediglich an, ob eine Verschiebung des Mittelwerts wahrscheinlich ist. Aufgrund des stochastischen Charakters der zu analysierenden Daten sind falsch-positive Ergebnisse möglich. Die CUSUM-Methode ist sowohl in einer "Offline"- als auch in einer "Online"-Version implementiert, sodass sie entweder für eine vollständige Datenreihe oder zur Erkennung von Veränderungen in Daten verwendet werden kann, die nach und nach eintreffen. Die Implementierung basiert größtenteils auf folgenden Websites:

- https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm
- https://en.wikipedia.org/wiki/CUSUM

Im Wesentlichen wird die Abweichung der Daten von einem vorgegebenen Zielwert kumuliert, und wenn die Gesamtabweichung zu groß wird, wird ein Änderungspunkt gemeldet. Eine zweite Methode, die binäre Segmentierung, wird ausschließlich als "Offline"-Verfahren implementiert, da sie die Datenreihe als Ganzes untersuchen muss. In der hier enthaltenen Variante wurden

folgende Ansätze verwendet:

- Die Segmente, in die die Datenreihe unterteilt werden kann, sollten nicht zu kurz sein, da es sonst zu Segmenten kommen könnte, die nur aus einem einzigen Datenpunkt bestehen. Daher wird eine Mindestlänge festgelegt.
- Damit sich die Segmentierung lohnt, sollte ein Mindestgewinn bei der Reduzierung der Kostenfunktion (der Summe der quadrierten Abweichungen vom Mittelwert für jedes Segment) erzielt werden.

Dies steht möglicherweise nicht im Einklang mit den Beschreibungen der Methode in verschiedenen Veröffentlichungen, ist jedoch leicht verständlich und intuitiv. Eine Veröffentlichung, die weitere Informationen zur Methode im Allgemeinen liefert, ist "Selective review of offline change point detection methods" von Truong et al. https://arxiv.org/abs/1801.00718.

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren:

::math::changepoint::cusum-detect *data* ?args?

Untersucht eine vorgegebene Datenreihe und gibt die Position der ersten Änderung (falls vorhanden) zurück.

double *data*

Zu untersuchende Datenreihe

list *args*

Optionale Liste von Schlüssel-Wert-Paaren:

-target *value*

Der Zielwert (bzw. Mittelwert) für die Zeitreihe

-tolerance *value*

Die zulässige Standardabweichung

-kfactor *value*

Der Faktor, mit dem die Standardabweichung multipliziert werden soll (Standardwert ist 0,5, typischerweise zwischen 0,5 und 1,0)

-hfactor *value*

Der Faktor, der die Grenzen festlegt, innerhalb derer die "cusum"-Statistik akzeptiert wird (typischerweise 3,0–5,0, Standardwert 4,0)

::math::changepoint::cusum-online ?args?

Klasse zur Überprüfung der übergebenen Daten hinsichtlich der erwarteten Eigenschaften. Es müssen mindestens die Schlüsselwörter *-target* und *-tolerance* angegeben werden.

list *args*

Liste der Schlüssel-Wert-Paare:

-target *value*

Der Zielwert (bzw. Mittelwert) für die Zeitreihe

-tolerance *value*

Die zulässige Standardabweichung

-kfactor *value*

Der Faktor, mit dem die Standardabweichung multipliziert werden soll (Standardwert ist 0,5, typischerweise zwischen 0,5 und 1,0)

-hfactor *value*
Der Faktor, der die Grenzen festlegt, innerhalb derer die "cusum"-Statistik akzeptiert wird (typischerweise 3,0–5,0, Standardwert 4,0)

$cusumObj examine *value*
Übergibt einen Wert an das *cusum-online*-Objekt und prüft ihn. Bleibt die kumulative Summe mit diesem neuen Wert innerhalb der Grenzen, wird Null (0) zurückgegeben, andernfalls Eins (1).
double *value*
Der neue Wert

$cusumObj reset
Setzt die kumulative Summe zurück, sodass die Prüfung von vorne beginnen kann.
::math::changepoint::binary-segmentation *data* ?args?
Wendet die binäre Segmentierungsmethode rekursiv an, um Änderungspunkte zu ermitteln. Gibt eine Liste mit den Indizes potenzieller Änderungspunkte zurück.
list *data*
Zu untersuchende Daten
list *args*
Optionale Schlüssel-Wert-Paare:
-minlength *number*
Mindestanzahl an Punkten in jedem Segment (Standard: 5)
-threshold *value*
Faktor, der auf die Standardabweichung angewendet wird und als Schwellenwert dient, um die Änderung der Kostenfunktion als Verbesserung zu werten (Standardwert: 1,0)

Kategorie
Mathematik

Copyright
```
Copyright (c) 2020 by Arjen Markus
```

combinatorics
Stichwörter: combinatorics

Name
math::combinatorics – Kombinatorische Funktionen in der Tcl-Mathematikbibliothek

Übersicht
package require Tcl 8.5 9
package require math ?**1.2.6**?
package require Tcl 8.6 9
package require TclOO
package require math::combinatorics ?**2.1**?
::math::ln_Gamma *z*

::math::factorial *x*
::math::choose *n k*
::math::Beta *z w*
::math::combinatorics::permutations *n*
::math::combinatorics::variations *n k*
::math::combinatorics::combinations *n k*
::math::combinatorics::derangements *n*
::math::combinatorics::catalan *n*
::math::combinatorics::firstStirling *n m*
::math::combinatorics::secondStirling *n m*
::math::combinatorics::partitionP *n*
::math::combinatorics::list-permutations *n*
::math::combinatorics::list-variations *n k*
::math::combinatorics::list-combinations *n k*
::math::combinatorics::list-derangements *n*
::math::combinatorics::list-powerset *n*
::math::combinatorics::permutationObj new/create NAME *n*
$perm next
$perm reset
$perm setElements *elements*
$perm setElements
::math::combinatorics::combinationObj new/create NAME *n k*
$combin next
$combin reset
$combin setElements *elements*
$combin setElements

Beschreibung

Das Paket **math** enthält Implementierungen verschiedener Funktionen, die bei kombinatorischen Problemen nützlich sind. Das Paket **math::combinatorics** erweitert diese Sammlung um Funktionen, die auf Merkmalen von Tcl 8.6 beruhen. Hinweis: Die Bedeutung der Funktion "partitionP" sowie der Catalan- und Stirling-Zahlen wird auf der MathWorld-Website erläutert.

Befehle

::math::ln_Gamma *z*

Gibt den natürlichen Logarithmus der Gammafunktion für das Argument *z* zurück.

Die Gammafunktion ist definiert als das uneigentliche Integral von Null bis zum positiven Unendlichen von

```
t**(x-1)*exp(-t) dt
```

Die in der Tcl-Mathematikbibliothek verwendete Approximation stammt von Lanczos, *ISIAM J. Numerical Analysis, Serie B*, Band 1, S. 86. Für "**x** > 1" wird angegeben, dass der absolute Fehler des Ergebnisses kleiner als 5,5*10**-10 ist – das heißt, der resultierende Wert von Gamma, wenn

```
exp( ln_Gamma( x) )
```

berechnet wird, ist voraussichtlich auf mehr als neun signifikante Stellen genau.

::math::factorial *x*

Gibt die Fakultät des Arguments *x* zurück.

Für ganzzahlige Werte von *x* im Bereich 0 <= *x* <= 12 wird ein exaktes ganzzahliges Ergebnis zurückgegeben.

Für ganzzahlige Werte von *x* im Bereich 13 <= *x* <= 21 wird auf Rechnern mit IEEE-Gleitkommastandard ein exaktes Gleitkommaergebnis zurückgegeben.

Für ganze Zahlen *x* mit 22 <= *x* <= 170 ist das Ergebnis auf 1 ULP genau.

Für reelle Zahlen *x* mit *x* >= 0 wird das Ergebnis durch Berechnung von *Gamma(x+1)* unter Verwendung der Funktion **::math::ln_Gamma** approximiert; das Ergebnis wird voraussichtlich auf mehr als neun signifikante Stellen genau sein.

Es ist ein Fehler, *x* <= -1 oder *x* > 170 anzugeben oder einen Wert für *x*, der nicht numerisch ist.

::math::choose *n k*

Gibt den Binomialkoeffizienten *C(n, k)* zurück

```
C(n,k) = n! / k! (n-k)!
```

Wenn beide Parameter ganze Zahlen sind und das Ergebnis in 32 Bit passt, wird das Ergebnis auf eine ganze Zahl gerundet.

Ganzzahlige Ergebnisse sind bis mindestens *n* = 34 exakt. Gleitkommaergebnisse sind auf mehr als neun signifikante Stellen genau.

::math::Beta *z w*

Gibt die Beta-Funktion der Parameter *z* und *w* zurück.

```
Beta(z,w) = Beta(w,z) = Gamma(z) * Gamma(w) / Gamma(z+w)
```

Die Ergebnisse werden als Gleitkommazahl mit einer Genauigkeit von mehr als neun signifikanten Stellen zurückgegeben, sofern *w* und *z* beide mindestens 1 betragen.

::math::combinatorics::permutations *n*

Gibt die Anzahl der Permutationen von n Elementen zurück. Der Rückgabewert ist stets eine ganze Zahl; er unterliegt nicht der Beschränkung auf den Bereich von 32- oder 64-Bit-Ganzzahlen, da in Tcl 8.5 und späteren Versionen Ganzzahlen mit beliebiger Genauigkeit verwendet werden.

int *n*

Die Anzahl der Elemente, die permutiert werden sollen.

::math::combinatorics::variations *n k*

Gibt die Anzahl der Variationen von k Elementen zurück, die aus insgesamt n Elementen ausgewählt werden. Dabei wird die Reihenfolge der Elemente berücksichtigt.

int *n*

Die Anzahl der Elemente, aus denen eine Auswahl getroffen werden soll.

int *k*
Die Anzahl der Elemente, die in jeder Variation ausgewählt werden sollen.

::math::combinatorics::combinations *n k*
Gibt die Anzahl der Kombinationen von k Elementen zurück, die aus insgesamt n Elementen ausgewählt werden. Die Reihenfolge der Elemente spielt keine Rolle.

int *n*
Die Anzahl der Elemente, aus denen eine Auswahl getroffen werden soll.

int *k*
Die Anzahl der Elemente, die in jeder Kombination ausgewählt werden sollen.

::math::combinatorics::derangements *n*
Gibt die Anzahl der Derangements von n Elementen zurück. Ein Derangement ist eine Permutation, bei der jedes Element gegenüber seiner ursprünglichen Position verschoben ist.

int *n*
Die Anzahl der Elemente, die neu angeordnet werden sollen.

::math::combinatorics::catalan *n*
Gibt die n-te Catalan-Zahl zurück. Die Zahl n muss 1 oder größer sein. Diese Zahlen kommen in verschiedenen kombinatorischen Problemen vor.

int *n*
Der Index der Catalan-Zahl

::math::combinatorics::firstStirling *n m*
Berechnet eine Stirling-Zahl erster Art (mit Vorzeichen, m Zyklen in einer Permutation von n Elementen)

int *n*
Anzahl der Elemente

int *m*
Anzahl der Zyklen

::math::combinatorics::secondStirling *n m*
Berechnet eine Stirling-Zahl zweiter Art (m nichtleere Teilmengen aus n Elementen)

int *n*
Anzahl der Elemente

int *m*
Anzahl der Teilmengen

::math::combinatorics::partitionP *n*
Berechnet die Anzahl der Möglichkeiten, auf die eine ganze Zahl n als Summe positiver ganzer Zahlen dargestellt werden kann.

int *n*
Die betreffende Zahl

::math::combinatorics::list-permutations *n*
Gibt die Liste der Permutationen der Zahlen 0, ..., n-1 zurück.

int *n*
Die Anzahl der Elemente, die permutiert werden sollen.

::math::combinatorics::list-variations *n k*
Gibt die Liste der Variationen von k Zahlen zurück, die aus den Zahlen 0, ..., n-1 ausgewählt wurden. Dabei wird die Reihenfolge der Elemente berücksichtigt.
int *n*
Die Anzahl der Elemente, aus denen eine Auswahl getroffen werden soll.
int *k*
Die Anzahl der Elemente, die in jeder Variation ausgewählt werden sollen.

::math::combinatorics::list-combinations *n k*
Gibt die Liste der Kombinationen aus k Zahlen zurück, die aus den Zahlen 0, ..., n-1 ausgewählt wurden. Die Reihenfolge der Elemente wird dabei nicht berücksichtigt.
int *n*
Die Anzahl der Elemente, aus denen eine Auswahl getroffen werden soll.
int *k*
Die Anzahl der Elemente, die in jeder Kombination ausgewählt werden sollen.

::math::combinatorics::list-derangements *n*
Gibt die Liste der Derangements der Zahlen 0, ..., n-1 zurück.
int *n*
Die Anzahl der Elemente, die neu angeordnet werden sollen.

::math::combinatorics::list-powerset *n*
Gibt die Liste aller Teilmengen der Zahlen 0, ..., n-1 zurück.
int *n*
Die Anzahl der Elemente, die neu angeordnet werden sollen.

::math::combinatorics::permutationObj new/create NAME *n*
Erzeugt ein TclOO-Objekt, das Permutationen nacheinander zurückgibt. Wenn die letzte Permutation erreicht ist, wird eine leere Liste zurückgegeben.
int *n*
Die Anzahl der Elemente, die neu angeordnet werden sollen.

$perm next
Gibt die nächste Permutation von n Objekten zurück.
$perm reset
Setzt das Objekt zurück, sodass der Befehl *next* wieder die vollständige Liste liefert.
$perm setElements *elements*
Registriert eine Liste von Elementen, die mit dem Befehl *nextElements* permutiert werden sollen.
list *elements*
Die Liste mit n Elementen, die permutiert werden sollen.

$perm setElements
Gibt die nächste Permutation der registrierten Elemente zurück.

::math::combinatorics::combinationObj new/create NAME *n k*
Erzeugt ein TclOO-Objekt, das Kombinationen nacheinander zurückgibt. Wenn die letzte Kombination erreicht ist, wird eine leere Liste zurückgegeben.
int *n*
Die Anzahl der Elemente, die neu angeordnet werden sollen.

$combin next
Gibt die nächste Kombination aus n Objekten zurück.
$combin reset
Setzt das Objekt zurück, sodass der Befehl *next* wieder die vollständige Liste liefert.
$combin setElements *elements*
Registriert eine Liste von Elementen, die mit dem Befehl *nextElements* permutiert werden sollen.
list *elements*
Die Liste mit n Elementen, die permutiert werden sollen.

$combin setElements
Gibt die nächste Kombination der registrierten Elemente zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

constants

Stichwörter: constants, degrees, e, math, pi, radians

Name

math::constants – Mathematische und numerische Konstanten

Übersicht

package require Tcl ?**8.3**?
package require math::constants ?**1.0.4**?
::math::constants::constants *args*
::math::constants::print-constants *args*

Beschreibung

Dieses Paket definiert einige gängige mathematische und numerische Konstanten. Durch die Verwendung des Pakets erhalten Sie einheitliche Werte für Zahlen wie Pi und ln(10).

Es definiert zwei Befehle:

- einen zum Importieren der Konstanten
- einen, der anzeigt, welche Konstanten definiert sind und welche Werte sie tatsächlich haben.

Der Grund für dieses Paket liegt darin, dass Sie bei (mathematischen) Berechnungen häufig eine gute Näherung benötigen, beispielsweise für das Verhältnis von Grad zu Radiant. Sie können dies natürlich wie folgt definieren:

```
    variable radtodeg [expr {180.0/(4.0*atan(1.0))}]
```

und die Variable radtodeg verwenden, wann immer Sie die Umrechnung benötigen.

Dies hat zwei Nachteile:

- Sie müssen sich die richtige Formel oder den richtigen Wert merken, und das birgt die Gefahr von Fehlern.
- Insbesondere bei der Verwendung mathematischer Funktionen wie *atan* gehen Sie davon aus, dass diese korrekt implementiert wurden. Dies ist jedoch selten oder gar nicht der Fall, und je nach Plattform können geringfügige Unterschiede auftreten.

So können Sie dies mit dem Paket *math::constants* umsetzen:

```
    package require math::constants
    ::math::constants::constants radtodeg degtorad
```

wodurch im aufrufenden Namespace zwei Variablen erstellt werden: radtodeg und (deren Kehrwert) degtorad.

Zu den definierten Konstanten (deren Werte größtenteils aus mathematischen Tabellen stammen, deren Genauigkeit die üblicherweise verarbeitbare Genauigkeit übersteigt) gehören:

- Grundkonstanten wie Pi, e, Gamma (Eulersche Konstante)
- abgeleitete Werte wie ln(10) und sqrt(2)
- rein numerische Werte wie 1/3, die der Einfachheit halber und aufgrund der Tatsache aufgenommen wurden, dass bestimmte, scheinbar triviale Berechnungen wie:

  ```
      set value [expr {3.0*$onethird}]
  ```

- liefern *genau* den Wert, den Sie erwarten (sofern IEEE-Arithmetik verfügbar ist).

Die vollständige Liste der benannten Konstanten finden Sie im Abschnitt Konstanten.

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren:

::math::constants::constants *args*
: Importiert die Konstanten, deren Namen als Argumente angegeben sind

::math::constants::print-constants *args*
: Gibt die Konstanten, deren Namen als Argumente angegeben wurden, auf dem Bildschirm aus (Name, Wert und Beschreibung) oder, falls keine Argumente angegeben wurden, alle definierten Konstanten. Hierbei handelt es sich in erster Linie um eine Hilfsprozedur.

Konstanten

pi
: Verhältnis von Kreisumfang zu Durchmesser

e
Basis des natürlichen Logarithmus

ln10
Natürlicher Logarithmus von 10

phi
Der Goldene Schnitt

gamma
Eulersche Konstante

sqrt2
Quadratwurzel aus 2

thirdrt2
Kubikwurzel aus 2 (2 hoch 1/3)

sqrt3
Quadratwurzel aus 3

radtodeg
Umrechnung von Radiant in Grad

degtorad
Umrechnung von Grad in Radiant

onethird
Ein Drittel (0,3333....)

twothirds
Zwei Drittel (0,6666....)

onesixth
Ein Sechstel (0,1666....)

huge
(Ungefähr) die größte Zahl

tiny
(Ungefähr) die kleinste Zahl ungleich Null

eps
Die kleinste Zahl, für die gilt: 1+eps!= 1

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: constants* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright

Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>

decimal

Stichwörter: decimal, math, tcl

Name

math::decimal – Allgemeine Dezimalarithmetik

Übersicht

package require Tcl ?**8.5 9**?
package require math::decimal 1.0.3
::math::decimal::fromstr *string*
::math::decimal::tostr *decimal*
::math::decimal::setVariable *variable setting*
::math::decimal::add *a b*
::math::decimal::+ *a b*
::math::decimal::subtract *a b*
::math::decimal::- *a b*
::math::decimal::multiply *a b*
::math::decimal::* *a b*
::math::decimal::divide *a b*
::math::decimal::/ *a b*
::math::decimal::divideint *a b*
::math::decimal::remainder *a b*
::math::decimal::abs *decimal*
::math::decimal::compare *a b*
::math::decimal::max *a b*
::math::decimal::maxmag *a b*
::math::decimal::min *a b*
::math::decimal::minmag *a b*
::math::decimal::plus *a*
::math::decimal::minus *a*
::math::decimal::copynegate *a*
::math::decimal::copysign *a b*
::math::decimal::is-signed *decimal*
::math::decimal::is-zero *decimal*
::math::decimal::is-NaN *decimal*
::math::decimal::is-infinite *decimal*
::math::decimal::is-finite *decimal*
::math::decimal::fma *a b c*
::math::decimal::round_half_even *decimal digits*
::math::decimal::round_half_up *decimal digits*
::math::decimal::round_half_down *decimal digits*
::math::decimal::round_down *decimal digits*
::math::decimal::round_up *decimal digits*
::math::decimal::round_floor *decimal digits*

::math::decimal::round_ceiling *decimal digits*
::math::decimal::round_05up *decimal digits*

Beschreibung

Das "decimal"-Paket bietet Unterstützung für die Dezimalarithmetik sowohl für Gleitkommazahlen mit begrenzter Genauigkeit als auch für Gleitkommazahlen mit beliebiger Genauigkeit. Darüber hinaus wird die Ganzzahl-Arithmetik unterstützt.

Weitere Informationen sowie die Spezifikationen, von denen dieses Paket abhängt, finden Sie auf der allgemeinen Seite zur Dezimalarithmetik unter http://speleotrove.com/decimal. Dieses Paket bietet folgende Funktionen:

- Ein neuer Datentyp "decimal", der als Liste dargestellt wird, die Vorzeichen, Mantisse und Exponent enthält.
- Arithmetische Operationen mit diesen Dezimalzahlen, wie beispielsweise Addition, Subtraktion, Multiplikation usw.

Die Zahlen werden mithilfe der Operation ::math::decimal::fromstr in das Dezimalformat konvertiert.

Die Zahlen werden mithilfe der Operation ::math::decimal::tostr wieder in eine Zeichenkette zurückkonvertiert.

Beispiele

In diesem Abschnitt werden einige einfache Beispiele vorgestellt. Da der Zweck dieser Bibliothek darin besteht, dezimale Rechenoperationen durchzuführen, sind Beispiele möglicherweise der einfachste Weg, um den Umgang mit ihr zu erlernen und den Unterschied zwischen der Verwendung dieses Pakets und der weiteren Nutzung von expr zu erkennen. Informationen zu den einzelnen Prozeduren finden Sie im Abschnitt "API" dieser Man-Seite.

```
    package require math::decimal

    # Various operations on two numbers.
    # We first convert them to decimal format.
    set a [::math::decimal::fromstr 8.2]
    set b [::math::decimal::fromstr .2]

    # Then we perform our operations. Here we add
    set c [::math::decimal::+ $a $b]

    # Finally we convert back to string format for presentation to the
    user.
    puts [::math::decimal::tostr $c] ; # => will output 8.4

    # Other examples
    #
    # Subtraction
    set c [::math::decimal::- $a $b]
    puts [::math::decimal::tostr $c] ; # => will output 8.0
```

```
    # Why bother using this instead of simply expr?
    puts [expr {8.2 + .2}] ; # => will output 8.399999999999999
    puts [expr {8.2 - .2}] ; # => will output 7.999999999999999
    # See http://speleotrove.com/decimal to learn more about why this
    happens.
```

API

::math::decimal::fromstr *string*
Wandelt *string* in eine Dezimalzahl um.

::math::decimal::tostr *decimal*
Wandelt *decimal* in eine Zeichenkette um, die die Zahl im Dezimalsystem darstellt.

::math::decimal::setVariable *variable setting*
Setzt *variable* auf *setting*. Gültige Variablen sind:
- *rounding* – Beim Umskalieren zu verwendende Rundungsmethode. Zulässige Methoden sind round_half_even, round_half_up, round_half_down, round_down, round_up, round_floor, round_ceiling.
- *precision* – Maximale Anzahl der in der Mantisse zulässigen Ziffern.
- *extended* – 1 für den erweiterten Modus, 0 für den vereinfachten Modus.
- *maxExponent* – Maximaler Wert für den Exponenten. Der Standardwert beträgt 999.
- *minExponent* – Minimaler Wert für den Exponenten. Standardwert: -998.

::math::decimal::add *a b*
siehe unten ...

::math::decimal::+ *a b*
Gibt die Summe der beiden Dezimalzahlen *a* und *b* zurück.

::math::decimal::subtract *a b*
siehe unten ...

::math::decimal::- *a b*
Gibt die Differenz der beiden Dezimalzahlen *a* und *b* zurück.

::math::decimal::multiply *a b*
siehe unten ...

::math::decimal::* *a b*
Gibt das Produkt der beiden Dezimalzahlen *a* und *b* zurück.

::math::decimal::divide *a b*
siehe unten ...

::math::decimal::/ *a b*
Gibt den Quotienten der Division der beiden Dezimalzahlen *a* und *b* zurück.

::math::decimal::divideint *a b*
Gibt den ganzzahligen Teil des Quotienten der Division der Dezimalzahlen *a* und *b* zurück.

::math::decimal::remainder *a b*
Gibt den Rest der Division der beiden Dezimalzahlen *a* und *b* zurück.

::math::decimal::abs *decimal*
Gibt den Betrag (Absolutwert) der Dezimalzahl zurück.

::math::decimal::compare *a b*

Vergleicht die beiden Dezimalzahlen a und b und gibt *0* zurück, wenn *a* == *b* gilt, *1*, wenn *a* > *b* gilt, und *-1*, wenn *a* < *b* gilt.

::math::decimal::max *a b*
Vergleicht die beiden Dezimalzahlen a und b und gibt *a* zurück, wenn *a* >= *b* gilt, und *b*, wenn *a* < *b* gilt.

::math::decimal::maxmag *a b*
Vergleicht die beiden Dezimalzahlen a und b ohne Berücksichtigung ihrer Vorzeichen und gibt *a* zurück, wenn *abs(a)* >= *abs(b)* gilt, und *b*, wenn *abs(a)* < *abs(b)* gilt.

::math::decimal::min *a b*
Vergleicht die beiden Dezimalzahlen a und b und gibt *a* zurück, wenn *a* <= *b* gilt, und *b*, wenn *a* > *b* gilt.

::math::decimal::minmag *a b*
Vergleicht die beiden Dezimalzahlen a und b ohne Berücksichtigung ihrer Vorzeichen und gibt *a* zurück, wenn *abs(a)* <= *abs(b)* gilt, und *b*, wenn *abs(a)* > *abs(b)* gilt.

::math::decimal::plus *a*
Gibt das Ergebnis von *::math::decimal::+ 0 $a* zurück.

::math::decimal::minus *a*
Gibt das Ergebnis von *::math::decimal::- 0 $a* zurück.

::math::decimal::copynegate *a*
Gibt *a* mit umgekehrtem Vorzeichen zurück.

::math::decimal::copysign *a b*
Gibt *a* zurück, wobei das Vorzeichen auf das Vorzeichen von *b* gesetzt wird.

::math::decimal::is-signed *decimal*
Gibt das Vorzeichen der Dezimalzahl zurück. Die Prozedur gibt 0 zurück, wenn die Zahl positiv ist, und 1, wenn sie negativ ist.

::math::decimal::is-zero *decimal*
Gibt true zurück, wenn der *decimal*-Wert Null ist, andernfalls false.

::math::decimal::is-NaN *decimal*
Gibt true zurück, wenn der *decimal*-Wert NaN (keine Zahl) ist, andernfalls false.

::math::decimal::is-infinite *decimal*
Gibt true zurück, wenn der *decimal*-Wert unendlich ist, andernfalls false.

::math::decimal::is-finite *decimal*
Gibt true zurück, wenn der *decimal*-Wert endlich ist, andernfalls false.

::math::decimal::fma *a b c*
Gibt das Ergebnis zurück, das sich ergibt, wenn zunächst *a* mit *b* multipliziert und anschließend *c* addiert wird. Das Umskalieren erfolgt erst nach Abschluss aller Rechenoperationen. Daher kann das Ergebnis von dem abweichen, das sich bei getrennter Ausführung der einzelnen Rechenoperationen ergibt.

::math::decimal::round_half_even *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: Auf den nächstgelegenen Wert runden. Bei gleichem Abstand so runden, dass die letzte Ziffer gerade ist.

::math::decimal::round_half_up *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: Auf den

nächstgelegenen Wert runden. Bei gleichem Abstand aufrunden.

::math::decimal::round_half_down *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: Auf den nächstgelegenen Wert runden. Bei gleichem Abstand abrunden.

::math::decimal::round_down *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: In Richtung 0 runden. (Abschneiden)

::math::decimal::round_up *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: Von 0 weg runden

::math::decimal::round_floor *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: In Richtung -Infinity runden.

::math::decimal::round_ceiling *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: In Richtung Infinity runden

::math::decimal::round_05up *decimal digits*
Rundet *decimal* auf *digits* Dezimalstellen nach den folgenden Regeln: Bei 0 oder 5 von 0 weg runden. Wie round-up, mit dem Unterschied, dass nur dann aufgerundet wird, wenn die aufzurundende Ziffer 0 oder 5 ist, und dass das Ergebnis nach einem Überlauf dasselbe ist wie bei round-down.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *decimal* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright

```
Copyright (c) 2011 Mark Alston <mark at beernut dot com>
```

exact

Stichwörter: exact

Name

math::exact – Exakte reelle Arithmetik

Übersicht

package require Tcl 8.6 9
package require grammar::aycock 1.0

package require math::exact 1.0.2
::math::exact::exactexpr *expr*
number **ref**
number **unref**
number **asPrint** *precision*
number **asFloat** *precision*

Beschreibung

Der Befehl **exactexpr** im Paket **math::exact** ermöglicht exakte Berechnungen über den Bereich der berechenbaren reellen Zahlen. Dabei handelt es sich nicht um Berechnungen mit beliebiger Genauigkeit, sondern um exakte Berechnungen, bei denen Zahlen durch Algorithmen dargestellt werden, die aufeinanderfolgende Näherungswerte erzeugen. Am Ende einer Berechnung kann der Aufrufer eine bestimmte Genauigkeit für das Endergebnis anfordern, und Zwischenergebnisse werden mit der Genauigkeit berechnet, die zur Erfüllung dieser Anforderung erforderlich ist.

Prozeduren

Die folgende Prozedur ist der primäre Einstieg in das Paket **math::exact**.

::math::exact::exactexpr *expr*
: Nimmt einen mathematischen Ausdruck in Tcl-Syntax entgegen und gibt ein Objekt zurück, das das Programm zur Berechnung aufeinanderfolgender Näherungswerte für den Wert des Ausdrucks darstellt. Das Ergebnis wird als exakte reelle Zahl bezeichnet.

***number* ref**
: Erhöht die Referenzanzahl einer bestimmten exakten reellen Zahl.

***number* unref**
: Verringert die Referenzanzahl einer bestimmten exakten reellen Zahl und löscht die Zahl, wenn die Referenzanzahl Null ist.

number* asPrint *precision
: Formatiert die angegebene *number* für die Ausgabe mit der festgelegten *precision*. (Wie *precision* interpretiert wird, erfahren Sie weiter unten.) Zahlen, von denen bekannt ist, dass sie rational sind, werden als Brüche formatiert.

number* asFloat *precision
: Formatiert die angegebene *number* für die Ausgabe mit der festgelegten *precision*. (Wie *precision* interpretiert wird, erfahren Sie weiter unten.) Alle Zahlen werden im Gleitkomma-E-Format formatiert.

Parameter

expr
: Auszuwertender Ausdruck. Die Syntax für Ausdrücke entspricht der in Tcl, allerdings ist der Umfang der verfügbaren Operationen geringer. Weitere Einzelheiten finden Sie weiter unten im Abschnitt Ausdrücke.

number
: Das Objekt, das bei einem früheren Aufruf von **math::exact::exactexpr** zurückgegeben wurde

precision

Die gewünschte "Genauigkeit" des Ergebnisses. Die Genauigkeit entspricht (ungefähr) dem Betrag des binären Exponenten zuzüglich der Anzahl der Bits der binären Mantisse. Um beispielsweise Ergebnisse in IEEE-754-Gleitkommazahlen doppelter Genauigkeit zurückzugeben, sind 56 Bits zuzüglich des Exponenten erforderlich. Zahlen zwischen 1/2 und 2 erfordern eine Genauigkeit von 57; Zahlen zwischen 1/4 und 1/2 oder zwischen 2 und 4 erfordern 58; Zahlen zwischen 1/8 und 1/4 oder zwischen 4 und 8 erfordern 59; und so weiter.

Ausdrücke

Der Befehl **math::exact::exactexpr** akzeptiert Ausdrücke, die auf einer Teilmenge der Tcl-Syntax basieren. Die folgenden Komponenten dürfen in einem Ausdruck verwendet werden.

- Dezimalganzzahlen.
- Variablenreferenzen mit dem Dollarzeichen (**$**). Der Wert der Variablen muss das Ergebnis eines weiteren Aufrufs von **math::exact::exactexpr** sein. Die Referenzanzahl des Werts wird für jede Stelle, an der er im Ausdruck vorkommt, um eins erhöht.
- Der Potenzierungsoperator (**).
- Unäre Plus- (**+**) und Minus- (**-**) Operatoren.
- Die Operatoren für die Multiplikation (*) und die Division (/).
- Klammern zur Gruppierung.
- Funktionen. Die verfügbaren Funktionen finden Sie weiter unten im Abschnitt Funktionen.

Funktionen

Die folgenden Funktionen stehen zur Verwendung in exakten reellen Ausdrücken zur Verfügung.

acos(*x*)
Der Arkuskosinus von *x*. Das Ergebnis wird im Bogenmaß angegeben. Der Betrag von *x* muss kleiner als 1 sein.

acosh(*x*)
Der inverse hyperbolische Kosinus von *x*. *x* muss größer als 1 sein.

asin(*x*)
Der Arkussinus von *x*. Das Ergebnis wird im Bogenmaß angegeben. Der Betrag von *x* muss kleiner als 1 sein.

asinh(*x*)
Der inverse hyperbolische Sinus von *x*.

atan(*x*)
Der Arkustangens von *x*. Das Ergebnis wird im Bogenmaß angegeben.

atanh(*x*)
Der inverse hyperbolische Tangens von *x*. Der Betrag von *x* muss kleiner als 1 sein.

cos(*x*)
Der Kosinus von *x*. *x* wird im Bogenmaß angegeben.

cosh(*x*)
Der hyperbolische Kosinus von *x*.

e()
Die Basis der natürlichen Logarithmen = **2.71828...**

exp(*x*)
Die Exponentialfunktion von *x*.

log(*x*)
Der natürliche Logarithmus von *x*. *x* muss positiv sein.

pi()
Der Wert von Pi = **3.15159...**

sin(*x*)
Der Sinus von *x*. *x* wird im Bogenmaß angegeben.

sinh(*x*)
Der hyperbolische Sinus von *x*.

sqrt(*x*)
Die Quadratwurzel von *x*. *x* muss positiv sein.

tan(*x*)
Der Tangens von *x*. *x* wird im Bogenmaß angegeben.

tanh(*x*)
Der hyperbolische Tangens von *x*.

Zusammenfassung

Der Befehl **math::exact::exactexpr** stellt ein System bereit, das exakte Arithmetik über berechenbare reelle Zahlen durchführt, wobei die Zahlen als Algorithmen zur sukzessiven Annäherung dargestellt werden. Nachstehend finden Sie ein Beispiel, das die aus der Schule bekannte Lösungsformel für quadratische Gleichungen umsetzt.

```
namespace import math::exact::exactexpr
proc exactquad {a b c} {
    set d [[exactexpr {sqrt($b*$b - 4*$a*$c)}] ref]
    set r0 [[exactexpr {(-$b - $d) / (2 * $a)}] ref]
    set r1 [[exactexpr {(-$b + $d) / (2 * $a)}] ref]
    $d unref
    return [list $r0 $r1]
}

set a [[exactexpr 1] ref]
set b [[exactexpr 200] ref]
set c [[exactexpr {(-3/2) * 10**-12}] ref]
lassign [exactquad $a $b $c] r0 r1
$a unref; $b unref; $c unref
puts [list [$r0 asFloat 70] [$r1 asFloat 110]]
$r0 unref; $r1 unref
```

Das Programm gibt das Ergebnis aus:

```
-2.000000000000000075e2 7.499999999999999719e-15
```

Hinweis: Wenn die IEEE-754-Gleitkommadarstellung verwendet worden wäre, hätte ein katastrophaler Rundungsfehler zu einer kleineren Wurzel geführt, die um den Faktor zwei zu hoch ist:

```
-200.0 1.4210854715202004e-14
```

Die Aufrufe von **exactexpr** sollten weitgehend selbsterklärend sein. Die weiteren erwähnenswerten Befehle sind **ref** und **unref**. Der Aufrufer muss den Überblick über Verweise auf exakte Ausdrücke behalten – er muss **ref** jedes Mal aufrufen, wenn ein exakter Ausdruck in einer Variablen gespeichert wird, und **unref** jedes Mal, wenn die Variable den Gültigkeitsbereich verlässt oder überschrieben wird. Die Methode **asFloat** gibt Dezimalziffern aus, solange die angeforderte Genauigkeit dies zulässt. Sie wird beendet, wenn die angeforderte Genauigkeit zu einer Ungenauigkeit von mehr als einer Einheit in der niedrigstwertigen Ziffer führt.

Kategorie
Mathematik

Copyright

```
Copyright (c) 2015 Kevin B. Kenny <kennykb@acm.org>
Redistribution permitted under the terms of the Open Publication
    License <http://www.opencontent.org/openpub/>
```


figurate
Stichwörter: figurate, figurate numbers, mathematics

Name
math::figurate – Berechnung figurierter Zahlen

Übersicht
package require Tcl 8.6 9
package require math::figurate 1.0
::math::figurate::sum_sequence *n*
::math::figurate::sum_squares *n*
::math::figurate::sum_cubes *n*
::math::figurate::sum_4th_power *n*
::math::figurate::sum_5th_power *n*
::math::figurate::sum_6th_power *n*
::math::figurate::sum_7th_power *n*
::math::figurate::sum_8th_power *n*
::math::figurate::sum_9th_power *n*
::math::figurate::sum_10th_power *n*
::math::figurate::sum_sequence_odd *n*
::math::figurate::sum_squares_odd *n*
::math::figurate::sum_cubes_odd *n*
::math::figurate::sum_4th_power_odd *n*
::math::figurate::sum_5th_power_odd *n*
::math::figurate::sum_6th_power_odd *n*
::math::figurate::sum_7th_power_odd *n*
::math::figurate::sum_8th_power_odd *n*
::math::figurate::sum_9th_power_odd *n*
::math::figurate::sum_10th_power_odd *n*
::math::figurate::oblong *n*

::math::figurate::pronic *n*
::math::figurate::triangular *n*
::math::figurate::square *n*
::math::figurate::cubic *n*
::math::figurate::biquadratic *n*
::math::figurate::centeredTriangular *n*
::math::figurate::centeredSquare *n*
::math::figurate::centeredPentagonal *n*
::math::figurate::centeredHexagonal *n*
::math::figurate::centeredCube *n*
::math::figurate::decagonal *n*
::math::figurate::heptagonal *n*
::math::figurate::hexagonal *n*
::math::figurate::octagonal *n*
::math::figurate::octahedral *n*
::math::figurate::pentagonal *n*
::math::figurate::squarePyramidral *n*
::math::figurate::tetrahedral *n*
::math::figurate::pentatope *n*

Beschreibung

Summen von Zahlen, die einem bestimmten Muster folgen, werden als figurierte Zahlen bezeichnet. Ein einfaches Beispiel ist die Summe der ganzen Zahlen 1, 2, ... bis n. Sie können 1, 1+2=3, 1+2+3=6, ... Objekte in einem Dreieck anordnen, daher der Name Dreieckszahlen:

```
        *
        *   *
        *   *   *
        *   *   *   *
        . . .
```

Das Paket **math::figurate** umfasst eine Sammlung von Prozeduren zur Auswertung einer Vielzahl figurierter Zahlen. Zwar sind alle Formeln unkompliziert, doch die Details sind mitunter verwirrend. *Hinweis:* Die Prozeduren interpretieren Argumente kleiner als Null als "keine zu zählenden Objekte" und geben daher den Wert 0 zurück.

Prozeduren

Die Prozeduren lassen sich in einige Kategorien einteilen: Summen von ganzen Zahlen, die auf eine bestimmte Potenz erhoben werden, Summen von ungeraden ganzen Zahlen sowie allgemeine figurierte Zahlen, beispielsweise die Fünfeckzahlen.

::math::figurate::sum_sequence *n*

Gibt die Summe der ganzen Zahlen 1, 2, ..., n zurück.

int *n*

Die größte ganze Zahl in der Summe

::math::figurate::sum_squares *n*

Gibt die Summe der Quadrate 1**2, 2**2, ..., n**2 zurück.

int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_cubes *n*
Gibt die Summe der Kubikzahlen 1**3, 2**3, ..., n**3 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_4th_power *n*
Gibt die Summe der vierten Potenzen 1**4, 2**4, ..., n**4 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_5th_power *n*
Gibt die Summe der 5. Potenzen 1**5, 2**5, ..., n**5 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_6th_power *n*
Gibt die Summe der 6. Potenzen 1**6, 2**6, ..., n**6 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_7th_power *n*
Gibt die Summe der 7. Potenzen 1**7, 2**7, ..., n**7 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_8th_power *n*
Gibt die Summe der 8. Potenzen 1**8, 2**8, ..., n**8 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_9th_power *n*
Gibt die Summe der 9. Potenzen 1**9, 2**9, ..., n**9 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_10th_power *n*
Gibt die Summe der 10. Potenzen 1**10, 2**10, ..., n**10 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_sequence_odd *n*
Gibt die Summe der ungeraden ganzen Zahlen 1, 3, ..., 2n-1 zurück.
int *n*
Die größte ganze Zahl in der Summe

::math::figurate::sum_squares_odd *n*
Gibt die Summe der ungeraden Quadrate 1**2, 3**2, ..., (2n-1)**2 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_cubes_odd *n*
Gibt die Summe der ungeraden Kubikzahlen 1**3, 3**3, ..., (2n-1)**3 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_4th_power_odd *n*
Gibt die Summe der ungeraden 4. Potenzen 1**4, 2**4, ..., (2n-1)**4 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_5th_power_odd *n*
Gibt die Summe der ungeraden 5. Potenzen 1**5, 2**5, ..., (2n-1)**5 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_6th_power_odd *n*
Gibt die Summe der ungeraden 6. Potenzen 1**6, 2**6, ..., (2n-1)**6 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_7th_power_odd *n*
Gibt die Summe der ungeraden 7. Potenzen 1**7, 2**7, ..., (2n-1)**7 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_8th_power_odd *n*
Gibt die Summe der ungeraden 8. Potenzen 1**8, 2**8, ..., (2n-1)**8 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_9th_power_odd *n*
Gibt die Summe der ungeraden 9. Potenzen 1**9, 2**9, ..., (2n-1)**9 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::sum_10th_power_odd *n*
Gibt die Summe der ungeraden 10. Potenzen 1**10, 2**10, ..., (2n-1)**10 zurück.
int *n*
Die größte ganzzahlige Basis in der Summe

::math::figurate::oblong *n*
Gibt die n-te Rechteckzahl (das Doppelte der n-ten Dreieckszahl) zurück
int *n*

Erforderlicher Index

::math::figurate::pronic *n*
Gibt die n-te pronische Zahl zurück (Synonym für Rechteckzahl)
int *n*
Erforderlicher Index

::math::figurate::triangular *n*
Gibt die n-te Dreieckszahl zurück
int *n*
Erforderlicher Index

::math::figurate::square *n*
Gibt die n-te Quadratzahl zurück
int *n*
Erforderlicher Index

::math::figurate::cubic *n*
Gibt die n-te Kubikzahl zurück
int *n*
Erforderlicher Index

::math::figurate::biquadratic *n*
Gibt die n-te biquadratische Zahl (d. h. n**4) zurück.
int *n*
Erforderlicher Index

::math::figurate::centeredTriangular *n*
Gibt die n-te zentrierte Dreieckszahl zurück (die Elemente sind in konzentrischen Dreiecken angeordnet)
int *n*
Erforderlicher Index

::math::figurate::centeredSquare *n*
Gibt die n-te zentrierte Quadratzahl zurück (die Elemente sind in konzentrischen Quadraten angeordnet)
int *n*
Erforderlicher Index

::math::figurate::centeredPentagonal *n*
Gibt die n-te zentrierte Fünfeckzahl zurück (Elemente sind in konzentrischen Fünfecken angeordnet)
int *n*
Erforderlicher Index

::math::figurate::centeredHexagonal *n*
Gibt die n-te zentrierte Sechseckzahl zurück (Elemente sind in konzentrischen Sechsecken angeordnet)

int *n*
Erforderlicher Index

::math::figurate::centeredCube *n*
Gibt die n-te zentrierte Würfelzahl zurück (Elemente sind in konzentrischen Würfeln angeordnet)
int *n*
Erforderlicher Index

::math::figurate::decagonal *n*
Gibt die n-te Zehneckzahl zurück (Elemente, die in Zehnecken mit einem gemeinsamen Eckpunkt angeordnet sind)
int *n*
Erforderlicher Index

::math::figurate::heptagonal *n*
Gibt die n-te Siebeneckzahl zurück (Elemente, die in Siebenecken mit einem gemeinsamen Eckpunkt angeordnet sind)
int *n*
Erforderlicher Index

::math::figurate::hexagonal *n*
Gibt die n-te Sechseckzahl zurück (Elemente, die in Sechsecken mit einem gemeinsamen Eckpunkt angeordnet sind)
int *n*
Erforderlicher Index

::math::figurate::octagonal *n*
Gibt die n-te Achteckzahl zurück (Elemente, die in Achtecken mit einem gemeinsamen Eckpunkt angeordnet sind)
int *n*
Erforderlicher Index

::math::figurate::octahedral *n*
Gibt die n-te Oktaederzahl zurück (Elemente, die in Oktaedern mit einem gemeinsamen Mittelpunkt angeordnet sind)
int *n*
Erforderlicher Index

::math::figurate::pentagonal *n*
Gibt die n-te Fünfeckzahl zurück (Elemente, die in Fünfecken mit einem gemeinsamen Eckpunkt angeordnet sind)
int *n*
Erforderlicher Index

::math::figurate::squarePyramidral *n*
Gibt die n-te quadratische Pyramidalzahl zurück (Elemente sind in einer quadratischen Pyramide angeordnet)

int *n*
Erforderlicher Index

::math::figurate::tetrahedral *n*
Gibt die n-te Tetraederzahl zurück (Elemente, die in einer dreieckigen Pyramide angeordnet sind)
int *n*
Erforderlicher Index

::math::figurate::pentatope *n*
Gibt die n-te Pentatop-Zahl zurück (Elemente, die im vierdimensionalen Analogon einer dreieckigen Pyramide angeordnet sind)
int *n*
Erforderlicher Index

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: figurate* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

filtergen

Stichwörter: filtergen, digital, filtering

Name

math::filters – Digitale Filter

Übersicht

package require Tcl 8.6 9
package require TclOO
package require math::filters ?**0.4**?
::math::filters::filterButterworth *lowpass order samplefreq cutofffreq*
::math::filters::filter *coeffs data*
::math::filters::filterChebyshev *lowpass order samplefreq cutofffreq* ?*epsilon*?
::math::filters::filter *coeffs data*
::math::filters::filterObject new *coeffs yinit*
$filterObj filter *x*
$filterObj reset

Beschreibung

Das Paket **math::filters** implementiert digitale Filter, insbesondere Butterworth- und Tschebyscheff-Tiefpass- sowie Hochpassfilter. Die Prozeduren ermöglichen es, sowohl eine gesamte Datenreihe als auch einzelne Daten nacheinander zu filtern. Hinweis: https://en.wikipedia.org/wiki/Digital_filter und weitere Seiten auf Wikipedia beschreiben die Prinzipien und Entwurfsüberlegungen dieser digitalen Filter. Hinweis: Es handelt sich hierbei um eine Neuimplementierung im Vergleich zur Version 0.3, die auf GPL-lizenziertem Code basierte. Eine mögliche Inkompatibilität mit dieser Version besteht darin, dass die Ordnung nun gerade sein muss.

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren:

::math::filters::filterButterworth *lowpass order samplefreq cutofffreq*

Ermittelt die Koeffizienten für einen Butterworth-Filter einer vorgegebenen (geraden) Ordnung. Die Koeffizienten werden als verschachtelte Liste zurückgegeben, die für den Befehl oder die Methode *filter* geeignet ist,

bool *lowpass*
Erzeugt einen Tiefpassfilter (1) oder einen Hochpassfilter (0)

integer *order*
Die Ordnung des zu erzeugenden Filters – sie muss gerade sein

double *samplefreq*
Abtastfrequenz der Datenreihe

double *cutofffreq*
Grenzfrequenz des Filters (höchstens die Hälfte der Abtastfrequenz)

::math::filters::filter *coeffs data*

Filtert die gesamte Datenreihe anhand der Filterkoeffizienten.

list *coeffs*
Liste der Koeffizienten, wie sie von *filterButterworth* generiert wurde (oder eigentlich jede ähnliche Liste von Koeffizienten)

list *data*
Zu filternde Daten

::math::filters::filterChebyshev *lowpass order samplefreq cutofffreq ?epsilon?*

Ermittelt die Koeffizienten für einen Tschebyscheff-Filter einer vorgegebenen (geraden) Ordnung. Die Koeffizienten werden als verschachtelte Liste zurückgegeben, die für den Befehl oder die Methode *filter* geeignet ist. Für diese Art von Filtern kann ein zusätzlicher Parameter angegeben werden, der sogenannte Ripple-Faktor. Dieser steuert die Glätte des Verstärkungsfaktors.

bool *lowpass*
Erzeugt einen Tiefpassfilter (1) oder einen Hochpassfilter (0)

integer *order*
Die Ordnung des zu erzeugenden Filters – sie muss gerade sein

double *samplefreq*
Abtastfrequenz der Datenreihe

double *cutofffreq*
Grenzfrequenz des Filters (muss kleiner als die Hälfte der Abtastfrequenz sein)

double *epsilon*
Ripple-Faktor, Standardwert: 0,1

::math::filters::filter *coeffs data*
Filtert die gesamte Datenreihe anhand der Filterkoeffizienten.
list *coeffs*
Liste der Koeffizienten, wie sie von *filterButterworth* generiert wurde (oder eigentlich jede ähnliche Liste von Koeffizienten)
list *data*
Zu filternde Daten

::math::filters::filterObject new *coeffs yinit*
Erzeugt ein Filterobjekt. Die anfänglichen x-Werte werden mit Null angenommen. Die anfänglichen y-Werte können vorgegeben werden. Werden sie nicht angegeben, werden sie ebenfalls mit Null angenommen.
list *coeffs*
Liste der Koeffizienten, wie sie von *filterButterworth* generiert wurde (oder eigentlich jede ähnliche Liste von Koeffizienten)
list *yinit*
(Optional) Ausgangsdaten für das Filterergebnis.

$filterObj filter *x*
Filtert einen einzelnen Wert und gibt das Ergebnis zurück.
double *x*
Der zu filternde Wert

$filterObj reset
Setzt das Filterobjekt zurück (Neubeginn)

Kategorie
Mathematik

Copyright
```
Copyright (c) 2020 by Arjen Markus
```

fourier
Stichwörter: fourier, FFT, Fourier transform, complex numbers, mathematics

Name
math::fourier – Diskrete und schnelle Fourier-Transformationen

Übersicht
package require Tcl 8.5 9
package require math::fourier 1.0.3
::math::fourier::dft *in_data*
::math::fourier::inverse_dft *in_data*
::math::fourier::lowpass *cutoff in_data*

::math::fourier::highpass *cutoff in_data*

Beschreibung

Das Paket **math::fourier** verwendet zur Implementierung der diskreten Fourier-Transformation gegebenenfalls die schnelle Fourier-Transformation oder die gewöhnliche Transformation. Es enthält zudem einige einfache Filter-Prozeduren, die veranschaulichen, wie solche Filter implementiert werden können.

Ziel dieses Dokuments ist es, die implementierten Prozeduren zu beschreiben und einige Beispiele für ihre Verwendung zu geben. Da es zu den betreffenden Algorithmen umfangreiche Fachliteratur gibt, verweisen wir für weitere Erläuterungen auf einschlägige Lehrbücher. Wir verweisen zudem auf die ursprüngliche Wiki-Seite zu diesem Thema, auf der einige der Überlegungen hinter der aktuellen Implementierung beschrieben werden.

Allgemeine Informationen

Die beiden definierten Prozeduren der obersten Ebene lauten

- dft data-list
- inverse_dft data-list

Beide nehmen eine Liste von *komplexen Zahlen* entgegen und wenden auf diese Zahlenlisten jeweils eine diskrete Fourier-Transformation (DFT) bzw. deren Umkehrung an. Eine "komplexe Zahl" ist in diesem Fall entweder (i) ein Paar (eine Liste mit zwei Elementen) von Zahlen, das als Real- und Imaginärteil der komplexen Zahl interpretiert wird, oder (ii) eine einzelne Zahl, die als Realteil einer komplexen Zahl interpretiert wird, deren Imaginärteil Null ist. Der Rückgabewert hat stets das erste Format. (Die DFT liefert im Allgemeinen komplexe Ergebnisse, selbst wenn die Eingabe rein real ist.) Werden nacheinander erst die eine und dann die andere dieser Prozeduren auf eine Liste komplexer Zahlen angewendet, ergibt sich (abgesehen von Rundungsfehlern aufgrund der Gleitkommaarithmetik) wieder die ursprüngliche Zahlenliste.

Wenn die Eingabelänge N eine Zweierpotenz ist, verwenden diese Prozeduren den O(N log N)-Algorithmus der schnellen Fourier-Transformation. Wenn die Eingabelänge keine Zweierpotenz ist, wird die DFT stattdessen unter Verwendung des naiven quadratischen Algorithmus berechnet.

Einige Beispiele:

```
% dft {1 2 3 4}
{10 0.0} {-2.0 2.0} {-2 0.0} {-2.0 -2.0}
% inverse_dft {{10 0.0} {-2.0 2.0} {-2 0.0} {-2.0 -2.0}}
{1.0 0.0} {2.0 0.0} {3.0 0.0} {4.0 0.0}
% dft {1 2 3 4 5}
{15.0 0.0} {-2.5 3.44095480118} {-2.5 0.812299240582} {-2.5
-0.812299240582} {-2.5 -3.44095480118}
% inverse_dft {{15.0 0.0} {-2.5 3.44095480118} {-2.5
0.812299240582} {-2.5 -0.812299240582} {-2.5 -3.44095480118}}
{1.0 0.0} {2.0 8.881784197e-17} {3.0 4.4408920985e-17} {4.0
4.4408920985e-17} {5.0 -8.881784197e-17}
```

Im letztgenannten Fall wären die Imaginärteile <1e-16 bei exakter Arithmetik gleich Null gewesen, sind es hier jedoch aufgrund von Rundungsfehlern nicht.

Intern verwenden die Prozeduren ein flaches Listenformat, bei dem jedes Element mit einem geraden Index einen Realteil und jedes Element mit einem ungeraden Index einen Imaginärteil darstellt. Dies spiegelt sich in den Namen der Variablen durch die Präfixe "Re_" und "Im_" wider.

Das Paket enthält zwei einfache Filter. Sie haben ein analoges Gegenstück in einer einfachen elektronischen Schaltung, einem Widerstand und einem Kondensator in Reihe. Für die Verwendung dieser Filter ist das Paket **math::complexnumbers** erforderlich.

Prozeduren

Die öffentlichen Prozeduren zur Fourier-Transformation sind:

::math::fourier::dft *in_data*

Berechnet die *Fourier-Transformation* der angegebenen Liste komplexer Zahlen. Das Ergebnis ist eine Liste komplexer Zahlen, die die (komplexen) Amplituden der Fourier-Komponenten darstellen.

list *in_data*

Liste der Daten

::math::fourier::inverse_dft *in_data*

Ermittelt die *inverse Fourier-Transformation* der angegebenen Liste komplexer Zahlen (die als Amplituden interpretiert werden). Das Ergebnis ist eine Liste komplexer Zahlen, die die ursprünglichen (komplexen) Daten darstellen.

list *in_data*

Liste der Daten (Amplituden)

::math::fourier::lowpass *cutoff in_data*

Filtert die (komplexen) Amplituden so, dass hochfrequente Komponenten unterdrückt werden. Bei dem implementierten Filter handelt es sich um einen Tiefpassfilter erster Ordnung, das diskrete Äquivalent einer einfachen elektronischen Schaltung mit einem Widerstand und einem Kondensator.

float *cutoff*

Grenzfrequenz

list *in_data*

Liste der Daten (Amplituden)

::math::fourier::highpass *cutoff in_data*

Filtert die (komplexen) Amplituden so, dass niederfrequente Komponenten unterdrückt werden. Bei dem implementierten Filter handelt es sich um einen Tiefpassfilter erster Ordnung, das diskrete Äquivalent einer einfachen elektronischen Schaltung mit einem Widerstand und einem Kondensator.

float *cutoff*

Grenzfrequenz

list *in_data*

Liste der Daten (Amplituden)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *math :: fourier* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

fuzzy
Stichwörter: fuzzy, floating-point, math, rounding

Name
math::fuzzy – Unscharfer Vergleich von Gleitkommazahlen

Übersicht
package require Tcl ?**8.3**?
package require math::fuzzy ?**0.2**?
::math::fuzzy::teq *value1 value2*
::math::fuzzy::tne *value1 value2*
::math::fuzzy::tge *value1 value2*
::math::fuzzy::tle *value1 value2*
::math::fuzzy::tlt *value1 value2*
::math::fuzzy::tgt *value1 value2*
::math::fuzzy::tfloor *value*
::math::fuzzy::tceil *value*
::math::fuzzy::tround *value*
::math::fuzzy::troundn *value ndigits*

Beschreibung
Das Paket "Fuzzy" dient dazu, häufige Probleme mit Gleitkommazahlen auf systematische Weise zu lösen:

- Der Vergleich zweier Zahlen, die eigentlich identisch sein "sollten", wie beispielsweise 1.0 und 2.1/(1.2+0.9), führt nicht zwangsläufig zu dem intuitiv erwarteten Ergebnis.
- Das Runden einer Zahl, die genau zwischen zwei ganzen Zahlen liegt, kann zu seltsamen Fehlern führen, wie beispielsweise int(100.0*2.8)!= 28, sondern 27.

Das Fuzzy-Paket soll dabei helfen, diese Art von Problemen zu lösen, indem es "unscharfe" Vergleichsprozeduren für Gleitkommazahlen definiert. Dies geschieht durch die Berücksichtigung einer kleinen Toleranz, die automatisch bestimmt wird – die Toleranz beträgt das Dreifache des "Epsilon"-Werts, d. h. das Dreifache der kleinsten Zahl *eps*, bei der 1.0 und 1.0+$eps unterschieden werden können. In Tcl, das Gleitkommazahlen mit doppelter Genauigkeit verwendet, beträgt dieser Wert typischerweise 1.1e-16.

Prozeduren
Im Wesentlichen bietet das Paket die folgenden Prozeduren:

::math::fuzzy::teq *value1 value2*
Vergleicht zwei Gleitkommazahlen und gibt 1 zurück, wenn deren Werte innerhalb eines kleinen Bereichs liegen. Andernfalls wird 0 zurückgegeben.

::math::fuzzy::tne *value1 value2*
Gibt die Negation zurück, d. h., wenn die Differenz größer als die Toleranz ist, wird 1 zurückgegeben.

::math::fuzzy::tge *value1 value2*
Vergleicht zwei Gleitkommazahlen und gibt 1 zurück, wenn deren Werte entweder in einen kleinen Bereich fallen oder wenn die erste Zahl größer ist als die zweite. Andernfalls wird 0 zurückgegeben.

::math::fuzzy::tle *value1 value2*
Gibt den Wert 1 zurück, wenn die beiden Zahlen gemäß [teq] gleich sind oder wenn die erste kleiner als die zweite ist.

::math::fuzzy::tlt *value1 value2*
Gibt das Gegenteil von [tge] zurück.

::math::fuzzy::tgt *value1 value2*
Gibt das Gegenteil von [tle] zurück.

::math::fuzzy::tfloor *value*
Gibt die ganze Zahl zurück, die innerhalb einer genau definierten Toleranz kleiner oder gleich der angegebenen Gleitkommazahl ist.

::math::fuzzy::tceil *value*
Gibt die ganze Zahl zurück, die innerhalb einer genau definierten Toleranz größer oder gleich der angegebenen Gleitkommazahl ist.

::math::fuzzy::tround *value*
Rundet die Gleitkommazahl.

::math::fuzzy::troundn *value ndigits*
Rundet die Gleitkommazahl auf die festgelegte Anzahl von Dezimalstellen (Pro memorie).

Verwendung:

```
if { [teq $x $y] } { puts "x == y" }
if { [tne $x $y] } { puts "x != y" }
if { [tge $x $y] } { puts "x >= y" }
if { [tgt $x $y] } { puts "x > y" }
if { [tlt $x $y] } { puts "x < y" }
if { [tle $x $y] } { puts "x <= y" }

set fx      [tfloor $x]
set fc      [tceil  $x]
set rounded [tround $x]
set roundn  [troundn $x $nodigits]
```

Testfälle

Die Probleme, die bei Gleitkommazahlen auftreten können, werden anhand der Tests in der Datei "fuzzy.test" veranschaulicht:

- In mehreren Tests werden die üblichen Vergleichsoperatoren verwendet, und diese liefern

ausnahmslos keine nachvollziehbaren Ergebnisse.
- Ein Testfall verwendet [expr] ohne geschweifte Klammern ({ und }). Auch dieser schlägt fehl.

Daraus lässt sich schließen, dass jeder Ausdruck in geschweifte Klammern gesetzt werden sollte, da es andernfalls zu sehr unangenehmen Situationen kommen kann, wenn Genauigkeit erforderlich ist. Darüber hinaus werden Genauigkeit und Verständlichkeit der Ergebnisse durch die Verwendung dieser "toleranten" oder unscharfen Vergleiche verbessert.

Hinweis: Es gibt neben dem reinen Tcl-Paket auch eine C-basierte Version.

Literatur

Ursprüngliche Implementierung in Fortran durch Dr. H.D. Knoble (Penn State University).

P. E. Hagerty, "More on Fuzzy Floor and Ceiling", APL QUOTE QUAD 8(4):20–24, Juni 1978. Hinweis: Es hat fünf Jahre gedauert, bis TFLOOR=FL5 nach einem Begutachtungsverfahren veröffentlicht wurde.

L. M. Breed, "Definitions for Fuzzy Floor and Ceiling", APL QUOTE QUAD 8(3):16–23, März 1978.

D. Knuth, Art of Computer Programming, Band 1, Aufgabe 1.2.4–5.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: fuzzy* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

interpolate

Stichwörter: interpolate, interpolation, math, spatial interpolation

Name

math::interpolate – Interpolationsroutinen

Übersicht

package require Tcl ?**8.5 9**?
package require struct
package require math::interpolate ?**1.1.4**?
::math::interpolate::defineTable *name colnames values*
::math::interpolate::interp-1d-table *name xval*
::math::interpolate::interp-table *name xval yval*
::math::interpolate::interp-linear *xyvalues xval*
::math::interpolate::interp-lagrange *xyvalues xval*
::math::interpolate::prepare-cubic-splines *xcoord ycoord*
::math::interpolate::interp-cubic-splines *coeffs x*

::math::interpolate::interp-spatial *xyvalues coord*
::math::interpolate::interp-spatial-params *max_search power*
::math::interpolate::neville *xlist ylist x*

Beschreibung

Dieses Paket implementiert mehrere Interpolationsalgorithmen:

- Interpolation in einer Tabelle (eine oder zwei unabhängige Variablen); dies ist beispielsweise nützlich, wenn die Daten statisch sind, wie bei Tabellen mit statistischen Funktionen.
- Lineare Interpolation anhand eines vorgegebenen Datensatzes (als (x, y)-Paare organisiert).
- Lagrange-Interpolation. Diese ist vor allem von theoretischem Interesse, da keine Garantie hinsichtlich der Fehlergrenzen besteht. Eine mögliche Anwendung: Wenn Sie eine Gerade oder eine Parabel durch vorgegebene Punkte benötigen (die Werte werden berechnet, die Koeffizienten jedoch nicht zurückgegeben).

  Eine Variante ist die Neville-Methode, die ein besseres Verhalten und bessere Fehlergrenzen aufweist.
- Räumliche Interpolation unter Verwendung einer einfachen abstandsgewichteten Methode. Diese Prozedur lässt eine beliebige Anzahl räumlicher Dimensionen und eine beliebige Anzahl abhängiger Variablen zu.
- Eindimensionale Interpolation unter Verwendung kubischer Splines.

In diesem Dokument werden die Prozeduren beschrieben und deren Anwendung erläutert.

Inkompatibilität mit Version 1.0.3

Die Interpretation der Tabellen im Befehl **::math::interpolate::interpolate-1d-table** wurde geändert, um die Kompatibilität mit der Interpretation für die 2D-Interpolation im Befehl **::math::interpolate::interpolate-table** zu gewährleisten. Infolgedessen ist diese Version nicht kompatibel mit den früheren Versionen des Befehls (1.0.x).

Prozeduren

Das Interpolationspaket definiert die folgenden öffentlichen Prozeduren:

::math::interpolate::defineTable *name colnames values*

Definiert eine Tabelle mit einer oder zwei unabhängigen Variablen (die Unterscheidung ergibt sich implizit aus den Daten). Die Prozedur gibt den Namen der Tabelle zurück – dieser Name wird immer dann verwendet, wenn Sie die Werte interpolieren möchten. *Hinweis:* Diese Prozedur ist eine praktische Hülle für die Prozedur struct::matrix. Daher können Sie an jeder beliebigen Stelle in Ihrem Programm auf die Daten zugreifen.

string *name* (in)

Name der zu erstellenden Tabelle

list *colnames* (in)

Liste der Namen der Spalten

list *values* (in)

Liste der Werte (die Anzahl der Elemente sollte ein Vielfaches der Anzahl der Spalten sein. Weitere Informationen zur Interpretation der Daten finden Sie im Abschnitt Beispiele.

Die Werte müssen nach den unabhängigen Variablen sortiert sein.)

::math::interpolate::interp-1d-table *name xval*

Interpoliert in der eindimensionalen Tabelle "name" und gibt eine Liste von Werten zurück, jeweils einen für jede abhängige Spalte.

string *name* (in)

Name einer vorhandenen Tabelle

float *xval* (in)

Wert der unabhängigen *Zeilen*-Variablen

::math::interpolate::interp-table *name xval yval*

Interpoliert in der zweidimensionalen Tabelle "name" und gibt den interpolierten Wert zurück.

string *name* (in)

Name einer vorhandenen Tabelle

float *xval* (in)

Wert der unabhängigen *Zeilen*-Variablen

float *yval* (in)

Wert der unabhängigen *Spalten*-Variablen

::math::interpolate::interp-linear *xyvalues xval*

Interpoliert linear in der Liste der (x,y)-Paare und gibt den interpolierten Wert zurück.

list *xyvalues* (in)

Liste von (x, y)-Wertepaaren, sortiert nach steigendem x. Sie dienen als Knickpunkte einer stückweise linearen Funktion.

float *xval* (in)

Wert der unabhängigen Variablen, für den der Wert von y berechnet werden muss.

::math::interpolate::interp-lagrange *xyvalues xval*

Konstruiert anhand der Liste der (x,y)-Paare das eindeutige Polynom niedrigsten Grades, das durch alle Punkte verläuft, und gibt den interpolierten Wert zurück.

list *xyvalues* (in)

Liste der (x, y)-Wertepaare

float *xval* (in)

Wert der unabhängigen Variablen, für den der Wert von y berechnet werden muss.

::math::interpolate::prepare-cubic-splines *xcoord ycoord*

Gibt eine Liste von Koeffizienten zurück, mit denen die zweite Routine *interp-cubic-splines* die eigentliche Interpolation durchführt.

list *xcoord*

Liste der x-Koordinaten, an denen der Wert der zu interpolierenden Funktion bekannt ist. Die Koordinaten müssen streng aufsteigend sein. Es sind mindestens drei Punkte erforderlich.

list *ycoord*

Liste der y-Koordinaten (die Werte der Funktion bei den angegebenen x-Koordinaten).

::math::interpolate::interp-cubic-splines *coeffs x*

Gibt den interpolierten Wert an der Koordinate x zurück. Die Koeffizienten werden durch die Prozedur *prepare-cubic-splines* berechnet.

list *coeffs*
Liste der Koeffizienten, wie sie von prepare-cubic-splines zurückgegeben werden

float *x*
x-Koordinate, an der die Funktion geschätzt werden soll. Sie muss zwischen der ersten und der letzten x-Koordinate liegen, für die Werte angegeben wurden.

::math::interpolate::interp-spatial *xyvalues coord*
Verwendet eine einfache Interpolationsmethode mit Gewichten als Funktion des Kehrwerts des Abstands, um im 2D- und N-dimensionalen Raum zu interpolieren.

Die Liste xyvalues ist eine Liste von Listen:

```
{   {x1 y1 z1 {v11 v12 v13 v14}}
{x2 y2 z2 {v21 v22 v23 v24}}
...
}
```

Das letzte Element jeder inneren Liste ist entweder eine einzelne Zahl oder selbst eine Liste. Im letzteren Fall gibt die Methode einen Wert zurück, der eine Liste mit derselben Anzahl von Elementen enthält.

Die Methode wird durch den Suchradius und die Potenz des inversen Abstands beeinflusst.

list *xyvalues* (in)
Liste von Listen, wobei jede Unterliste eine Liste von Koordinaten und abhängigen Werten darstellt.

list *coord* (in)
Liste der Koordinaten, für die die Werte berechnet werden müssen

::math::interpolate::interp-spatial-params *max_search power*
Legt die Parameter für die räumliche Interpolation fest

float *max_search* (in)
Suchradius (Datenpunkte, die weiter entfernt liegen, werden ignoriert)

integer *power* (in)
Exponent für den Abstand (entweder 1 oder 2; Standardwert ist 2)

::math::interpolate::neville *xlist ylist x*
Führt eine Interpolation zwischen den tabellierten Werten einer Funktion durch, deren Abszissen *xlist* und deren Ordinaten *ylist* sind, um einen Schätzwert für den Wert der Funktion bei *x* zu ermitteln. Das Ergebnis ist eine Liste mit zwei Elementen; das erste Element ist der geschätzte Wert der Funktion, das zweite ist eine Schätzung des absoluten Fehlers des Ergebnisses. Es wird der Algorithmus von Neville für die Polynominterpolation verwendet. Hinweis: Bei einer großen Tabelle mit Werten wird ein Interpolationspolynom hohen Grades verwendet, was wahrscheinlich zu numerischen Instabilitäten führt; es ist ratsam, nur wenige tabellierte Werte in der Nähe der gewünschten Abszisse zu verwenden.

Beispiele

Beispiel für die Verwendung eindimensionaler Tabellen:

Angenommen, Sie haben mehrere tabellarisch aufgeführte Funktionen einer Variablen:

```
   x     y1     y2
 0.0    0.0    0.0
 1.0    1.0    1.0
 2.0    4.0    8.0
 3.0    9.0   27.0
 4.0   16.0   64.0
```

Um nun die Werte bei 0,5, 1,5, 2,5 und 3,5 zu berechnen, können Sie Folgendes verwenden:

```
  set table [::math::interpolate::defineTable table1  {x y1 y2} {   -
       1      2
                  0.0    0.0    0.0
                  1.0    1.0    1.0
                  2.0    4.0    8.0
                  3.0    9.0   27.0
                  4.0   16.0   64.0}]
  foreach x {0.5 1.5 2.5 3.5} {
      puts "$x: [::math::interpolate::interp-1d-table $table $x]"
  }
```

Bei eindimensionalen Tabellen wird die erste Zeile nicht verwendet. Bei zweidimensionalen Tabellen enthält die erste Zeile die Werte für die zweite unabhängige Variable.

Beispiel für die Verwendung kubischer Splines:

Angenommen, es liegen folgende Werte vor:

```
   x       y
 0.1     1.0
 0.3     2.1
 0.4     2.2
 0.8     4.11
 1.0     4.12
```

Um nun die Werte bei 0,1, 0,2, 0,3, ... 1,0 zu schätzen, können Sie Folgendes verwenden:

```
  set coeffs [::math::interpolate::prepare-cubic-splines  {0.1 0.3
   0.4 0.8  1.0}  {1.0 2.1 2.2 4.11 4.12}]
  foreach x {0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0} {
     puts "$x: [::math::interpolate::interp-cubic-splines $coeffs $x]"
  }
```

um die folgende Ausgabe zu erhalten:

```
0.1: 1.0
0.2: 1.68044117647
0.3: 2.1
0.4: 2.2
0.5: 3.11221507353
0.6: 4.25242647059
0.7: 5.41804227941
```

```
0.8: 4.11
0.9: 3.95675857843
1.0: 4.12
```

Wie Sie sehen, werden die Werte an den Abszissen exakt wiedergegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: interpolate* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright

```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
Copyright (c) 2004 Kevn B. Kenny <kennykb@users.sourceforge.net>
```

linalg

Stichwörter: linalg, least squares, linear algebra, linear equations, math, matrices, matrix, vectors

Name

math::linearalgebra – Lineare Algebra

Übersicht

package require Tcl ?**8.5 9**?
package require math::linearalgebra ?**1.1.7**?
::math::linearalgebra::mkVector *ndim value*
::math::linearalgebra::mkUnitVector *ndim ndir*
::math::linearalgebra::mkMatrix *nrows ncols value*
::math::linearalgebra::getrow *matrix row* ?*imin*? ?*imax*?
::math::linearalgebra::setrow *matrix row newvalues* ?*imin*? ?*imax*?
::math::linearalgebra::getcol *matrix col* ?*imin*? ?*imax*?
::math::linearalgebra::setcol *matrix col newvalues* ?*imin*? ?*imax*?
::math::linearalgebra::getelem *matrix row col*
::math::linearalgebra::setelem *matrix row* ?*col*? *newvalue*
::math::linearalgebra::swaprows *matrix irow1 irow2* ?*imin*? ?*imax*?
::math::linearalgebra::swapcols *matrix icol1 icol2* ?*imin*? ?*imax*?
::math::linearalgebra::show *obj* ?*format*? ?*rowsep*? ?*colsep*?
::math::linearalgebra::dim *obj*
::math::linearalgebra::shape *obj*

::math::linearalgebra::conforming *type obj1 obj2*
::math::linearalgebra::symmetric *matrix* ?*eps*?
::math::linearalgebra::norm *vector type*
::math::linearalgebra::norm_one *vector*
::math::linearalgebra::norm_two *vector*
::math::linearalgebra::norm_max *vector* ?*index*?
::math::linearalgebra::normMatrix *matrix type*
::math::linearalgebra::dotproduct *vect1 vect2*
::math::linearalgebra::unitLengthVector *vector*
::math::linearalgebra::normalizeStat *mv*
::math::linearalgebra::axpy *scale mv1 mv2*
::math::linearalgebra::add *mv1 mv2*
::math::linearalgebra::sub *mv1 mv2*
::math::linearalgebra::scale *scale mv*
::math::linearalgebra::rotate *c s vect1 vect2*
::math::linearalgebra::transpose *matrix*
::math::linearalgebra::matmul *mv1 mv2*
::math::linearalgebra::angle *vect1 vect2*
::math::linearalgebra::crossproduct *vect1 vect2*
::math::linearalgebra::matmul *mv1 mv2*
::math::linearalgebra::mkIdentity *size*
::math::linearalgebra::mkDiagonal *diag*
::math::linearalgebra::mkRandom *size*
::math::linearalgebra::mkTriangular *size* ?*uplo*? ?*value*?
::math::linearalgebra::mkHilbert *size*
::math::linearalgebra::mkDingdong *size*
::math::linearalgebra::mkOnes *size*
::math::linearalgebra::mkMoler *size*
::math::linearalgebra::mkFrank *size*
::math::linearalgebra::mkBorder *size*
::math::linearalgebra::mkWilkinsonW+ *size*
::math::linearalgebra::mkWilkinsonW- *size*
::math::linearalgebra::solveGauss *matrix bvect*
::math::linearalgebra::solvePGauss *matrix bvect*
::math::linearalgebra::solveTriangular *matrix bvect* ?*uplo*?
::math::linearalgebra::solveGaussBand *matrix bvect*
::math::linearalgebra::solveTriangularBand *matrix bvect*
::math::linearalgebra::determineSVD *A eps*
::math::linearalgebra::eigenvectorsSVD *A eps*
::math::linearalgebra::leastSquaresSVD *A y qmin eps*
::math::linearalgebra::choleski *matrix*
::math::linearalgebra::orthonormalizeColumns *matrix*
::math::linearalgebra::orthonormalizeRows *matrix*
::math::linearalgebra::dger *matrix alpha x y* ?*scope*?
::math::linearalgebra::dgetrf *matrix*
::math::linearalgebra::det *matrix*

::math::linearalgebra::largesteigen *matrix tolerance maxiter*
::math::linearalgebra::to_LA *mv*
::math::linearalgebra::from_LA *mv*

Beschreibung

Dieses Paket bietet sowohl Low-Level-Prozeduren als auch High-Level-Algorithmen zur Lösung von Problemen der linearen Algebra:

- robuste Lösung linearer Gleichungssysteme oder von Problemen der kleinsten Quadrate
- Bestimmung von Eigenvektoren und Eigenwerten symmetrischer Matrizen
- verschiedene Zerlegungen allgemeiner Matrizen oder Matrizen einer bestimmten Form
- (Eingeschränkte) Unterstützung für Matrizen in Bandspeicherung, einer gängigen Art dünn besetzter Matrizen.

Das Paket entstand als Neuimplementierung des LA-Pakets von Hume und aus dem Wunsch heraus, Low-Level-Prozeduren anzubieten, wie sie in der bekannten BLAS-Bibliothek zu finden sind. Matrizen werden als Listen von Listen implementiert und nicht als lineare Listen mit reservierten Elementen, wie im ursprünglichen LA-Paket, da sich herausgestellt hat, dass eine solche Implementierung tatsächlich schneller ist.

Es ist jedoch ratsam, die angebotenen Prozeduren wie *setrow* und *getrow* zu verwenden, anstatt sich explizit auf diese Darstellung zu verlassen: Auf diese Weise kann auf eine möglicherweise noch schnellere, kompilierte Implementierung umgestellt werden, die dieselbe API unterstützt.

Hinweis: Bei der Verwendung dieses Pakets in Kombination mit Tk kann es zu einem Namenskonflikt kommen, da sowohl dieses Paket als auch Tk einen Befehl *scale* definieren. Siehe den Abschnitt Namenskonflikt weiter unten.

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren (einige davon existieren als spezialisierte Prozeduren, siehe unten):

Constructing matrices and vectors

::math::linearalgebra::mkVector *ndim value*
Erzeugt einen Vektor mit ndim Elementen, von denen jedes den Wert *value* hat.
integer *ndim*
Dimension des Vektors (Anzahl der Komponenten)
double *value*
Zu verwendender einheitlicher Wert (Standard: 0.0)

::math::linearalgebra::mkUnitVector *ndim ndir*
Erzeugt einen Einheitsvektor im *ndim*-dimensionalen Raum entlang der *ndir*-ten Richtung.
integer *ndim*
Dimension des Vektors (Anzahl der Komponenten)
integer *ndir*
Richtung (0, ..., ndim-1)

::math::linearalgebra::mkMatrix *nrows ncols value*
Erzeugt eine Matrix mit *nrows* Zeilen und *ncols* Spalten. Alle Elemente haben den Wert *value*.
integer *nrows*

Anzahl der Zeilen

integer *ncols*
Anzahl der Spalten

double *value*
Zu verwendender einheitlicher Wert (Standard: 0.0)

::math::linearalgebra::getrow *matrix row* ?imin? ?imax?
Gibt eine einzelne Zeile einer Matrix als Liste zurück

list *matrix*
Die betreffende Matrix

integer *row*
Index der zurückzugebenden Zeile

integer *imin*
Kleinster Spaltenindex (Standard: 0)

integer *imax*
Größter Spaltenindex (Standard: ncols-1)

::math::linearalgebra::setrow *matrix row newvalues* ?imin? ?imax?
Setzt eine einzelne Zeile einer Matrix auf neue Werte (diese Liste muss dieselbe Anzahl an Elementen aufweisen wie die Anzahl der *Spalten* in der Matrix)

list *matrix*
Name der betreffenden Matrix

integer *row*
Index der zu aktualisierenden Zeile

list *newvalues*
Liste der neuen Werte für die Zeile

integer *imin*
Kleinster Spaltenindex (Standard: 0)

integer *imax*
Größter Spaltenindex (Standard: ncols-1)

::math::linearalgebra::getcol *matrix col* ?imin? ?imax?
Gibt eine einzelne Spalte einer Matrix als Liste zurück

list *matrix*
Die betreffende Matrix

integer *col*
Index der zurückzugebenden Spalte

integer *imin*
Kleinster Zeilenindex (Standard: 0)

integer *imax*
Größter Zeilenindex (Standard: nrows-1)

::math::linearalgebra::setcol *matrix col newvalues* ?imin? ?imax?
Setzt eine einzelne Spalte einer Matrix auf neue Werte (diese Liste muss dieselbe Anzahl an Elementen aufweisen wie die Anzahl der *Zeilen* in der Matrix)

list *matrix*
Name der betreffenden Matrix
integer *col*
Index der zu aktualisierenden Spalte
list *newvalues*
Liste der neuen Werte für die Spalte
integer *imin*
Kleinster Zeilenindex (Standard: 0)
integer *imax*
Größter Zeilenindex (Standard: nrows-1)

::math::linearalgebra::getelem *matrix row col*
Gibt ein einzelnes Element einer Matrix bzw. eines Vektors zurück
list *matrix*
Die betreffende Matrix oder der betreffende Vektor
integer *row*
Zeile des Elements
integer *col*
Spalte des Elements (bei Vektoren nicht vorhanden)

::math::linearalgebra::setelem *matrix row* ?col? *newvalue*
Setzt ein einzelnes Element einer Matrix (oder eines Vektors) auf einen neuen Wert
list *matrix*
Name der betreffenden Matrix
integer *row*
Zeile des Elements
integer *col*
Spalte des Elements (bei Vektoren nicht vorhanden)

::math::linearalgebra::swaprows *matrix irow1 irow2* ?imin? ?imax?
Vertauscht zwei Zeilen einer Matrix vollständig oder nur in einem ausgewählten Teil
list *matrix*
Name der betreffenden Matrix
integer *irow1*
Index der ersten Zeile
integer *irow2*
Index der zweiten Zeile
integer *imin*
Kleinster Spaltenindex (Standard: 0)
integer *imin*
Größter Spaltenindex (Standard: ncols-1)

::math::linearalgebra::swapcols *matrix icol1 icol2* ?imin? ?imax?
Vertauscht zwei Spalten einer Matrix vollständig oder nur in einem ausgewählten Teil
list *matrix*

Name der betreffenden Matrix

integer *irow1*
Index der ersten Spalte

integer *irow2*
Index der zweiten Spalte

integer *imin*
Kleinster Zeilenindex (Standard: 0)

integer *imin*
Größter Zeilenindex (Standard: nrows-1)

Querying matrices and vectors

::math::linearalgebra::show *obj* ?format? ?rowsep? ?colsep?
Gibt eine Zeichenkette zurück, die den Vektor oder die Matrix darstellt, um die Ausgabe zu vereinfachen. (Derzeit gibt es keine Möglichkeit, festgelegte Gruppen von Spalten auszugeben.)

list *obj*
Die betreffende Matrix oder der betreffende Vektor

string *format*
Format für die Ausgabe der Zahlen (Standard: %6.4f)

string *rowsep*
Zeichenkette, die zur Trennung der Zeilen verwendet werden soll (Standard: Zeilenumbruch)

string *colsep*
Zeichenkette zur Trennung der Spalten (Standard: Leerzeichen)

::math::linearalgebra::dim *obj*
Gibt die Anzahl der Dimensionen des Objekts zurück (entweder 0 für einen Skalar, 1 für einen Vektor und 2 für eine Matrix)

any *obj*
Skalar, Vektor oder Matrix

::math::linearalgebra::shape *obj*
Gibt die Anzahl der Elemente in jeder Dimension des Objekts zurück (entweder eine leere Liste für einen Skalar, eine einzelne Zahl für einen Vektor und eine Liste mit der Anzahl der Zeilen und Spalten für eine Matrix)

any *obj*
Skalar, Vektor oder Matrix

::math::linearalgebra::conforming *type obj1 obj2*
Prüft, ob zwei Objekte (Vektor oder Matrix) kompatible Formen aufweisen, d. h., ob sie in einer Operation wie der Addition oder der Matrixmultiplikation verwendet werden können.

string *type*
Art der Prüfung:
- "shape" – die beiden Objekte haben dieselbe Form (für alle elementweisen Operationen)
- "rows" – die beiden Objekte weisen die gleiche Anzahl von Zeilen auf (zur Verwendung als A und b in einem linearen Gleichungssystem $Ax = b$

- "matmul" – Das erste Objekt hat dieselbe Anzahl an Spalten wie das zweite Objekt an Zeilen. Nützlich für die Multiplikation von Matrizen untereinander oder von Matrizen mit Vektoren.

list *obj1*
Erster Vektor oder erste Matrix (linker Operand)

list *obj2*
Zweiter Vektor oder zweite Matrix (rechter Operand)

::math::linearalgebra::symmetric *matrix* ?eps?
Prüft, ob die angegebene (quadratische) Matrix symmetrisch ist. Das Argument "eps" gibt die Toleranz an.

list *matrix*
Zu prüfende Matrix

float *eps*
Toleranz zur Bestimmung der annähernden Gleichheit (Standardwert: 1.0e-8)

Basic operations

::math::linearalgebra::norm *vector type*
Gibt die Norm des angegebenen Vektors zurück. Das Argument "type" kann folgende Werte annehmen: 1, 2, inf oder max; dies entspricht jeweils der Summe der Absolutwerte, der gewöhnlichen euklidischen Norm oder der Max-Norm.

list *vector*
Vektor, Liste der Koeffizienten

string *type*
Normtyp (Standard: 2, die euklidische Norm)

::math::linearalgebra::norm_one *vector*
Gibt die L1-Norm des angegebenen Vektors zurück, d. h. die Summe der Absolutwerte

list *vector*
Vektor, Liste der Koeffizienten

::math::linearalgebra::norm_two *vector*
Gibt die L2-Norm des angegebenen Vektors zurück, d. h. die gewöhnliche euklidische Norm

list *vector*
Vektor, Liste der Koeffizienten

::math::linearalgebra::norm_max *vector* ?index?
Gibt die Linf-Norm des angegebenen Vektors zurück, d. h. den größten absoluten Koeffizienten

list *vector*
Vektor, Liste der Koeffizienten

integer *index*
(optional) Wenn der Wert ungleich Null ist, wird eine Liste aus dem Maximalwert und dem Index zurückgegeben, an dem dieser Maximalwert gefunden wurde. Wenn der Wert Null ist, wird der Maximalwert zurückgegeben.

::math::linearalgebra::normMatrix *matrix type*

Gibt die Norm der angegebenen Matrix zurück. Das Argument "type" kann folgende Werte annehmen: 1, 2, inf oder max; dies entspricht jeweils der Summe der Absolutwerte, der gewöhnlichen euklidischen Norm bzw. der Max-Norm.

list *matrix*
Matrix, Liste von Zeilenvektoren

string *type*
Normtyp (Standard: 2, die euklidische Norm)

::math::linearalgebra::dotproduct *vect1 vect2*
Berechnet das Skalarprodukt (innere Produkt) zweier Vektoren. Diese müssen dieselbe Form (Anzahl der Dimensionen) aufweisen.

list *vect1*
Erster Vektor, Liste der Koeffizienten

list *vect2*
Zweiter Vektor, Liste der Koeffizienten

::math::linearalgebra::unitLengthVector *vector*
Gibt einen Vektor in derselben Richtung mit der Länge 1 zurück.

list *vector*
Zu normalisierender Vektor

::math::linearalgebra::normalizeStat *mv*
Normalisiert die Matrix oder den Vektor im statistischen Sinne: Der Mittelwert der Elemente der Spalten des Ergebnisses beträgt Null und die Standardabweichung beträgt 1.

list *mv*
Vektor oder Matrix, die im oben genannten Sinne normalisiert werden soll

::math::linearalgebra::axpy *scale mv1 mv2*
Gibt einen Vektor oder eine Matrix zurück, der bzw. die aus einer "daxpy"-Operation resultiert, d. h.: berechnet a*x+y (wobei a ein Skalar und x sowie y Vektoren oder Matrizen gleicher Form sind) und gibt das Ergebnis zurück.

Spezielle Varianten sind: axpy_vect und axpy_mat (etwas schneller, jedoch ohne Überprüfung der Argumente)

double *scale*
Der Skalierungsfaktor für den ersten Vektor/die erste Matrix (a)

list *mv1*
Erster Vektor oder erste Matrix (x)

list *mv2*
Zweiter Vektor oder zweite Matrix (y)

::math::linearalgebra::add *mv1 mv2*
Gibt einen Vektor oder eine Matrix zurück, die der Summe der beiden Argumente (x+y) entspricht.

Spezielle Varianten sind: add_vect und add_mat (etwas schneller, jedoch ohne Überprüfung der Argumente)

list *mv1*

Erster Vektor oder erste Matrix (x)

list *mv2*

Zweiter Vektor oder zweite Matrix (y)

::math::linearalgebra::sub *mv1 mv2*

Gibt einen Vektor oder eine Matrix zurück, die der Differenz der beiden Argumente (x-y) entspricht.

Spezielle Varianten sind: sub_vect und sub_mat (etwas schneller, jedoch ohne Überprüfung der Argumente)

list *mv1*

Erster Vektor oder erste Matrix (x)

list *mv2*

Zweiter Vektor oder zweite Matrix (y)

::math::linearalgebra::scale *scale mv*

Skaliert einen Vektor oder eine Matrix und gibt das Ergebnis zurück, d. h.: berechnet a*x.

Spezielle Varianten sind: scale_vect und scale_mat (etwas schneller, jedoch ohne Überprüfung der Argumente)

double *scale*

Der Skalierungsfaktor für den Vektor/die Matrix (a)

list *mv*

Vektor oder Matrix (x)

::math::linearalgebra::rotate *c s vect1 vect2*

Wendet eine ebene Drehung auf zwei Vektoren an und gibt das Ergebnis als Liste aus zwei Vektoren zurück: c*x-s*y und s*x+c*y. In Algorithmen lassen sich der Kosinus und der Sinus des Winkels oft leicht bestimmen, daher ist es effizienter, diese Informationen direkt zu übergeben.

double *c*

Der Kosinus des Winkels

double *s*

Der Sinus des Winkels

list *vect1*

Erster Vektor (x)

list *vect2*

Zweiter Vektor (x)

::math::linearalgebra::transpose *matrix*

Transponiert eine Matrix

list *matrix*

Zu transponierende Matrix

::math::linearalgebra::matmul *mv1 mv2*

Multipliziert einen Vektor bzw. eine Matrix mit einem anderen Vektor bzw. einer anderen Matrix. Das Ergebnis ist eine Matrix, wenn sowohl x als auch y Matrizen sind oder beide Vektoren sind; in diesem Fall wird das "äußere Produkt" berechnet. Ist das eine ein Vektor und

das andere eine Matrix, dann ist das Ergebnis ein Vektor.

list *mv1*
Erster Vektor/erste Matrix (x)

list *mv2*
Zweiter Vektor/zweite Matrix (y)

::math::linearalgebra::angle *vect1 vect2*
Berechnet den Winkel zwischen zwei Vektoren (im Bogenmaß)

list *vect1*
Erster Vektor

list *vect2*
Zweiter Vektor

::math::linearalgebra::crossproduct *vect1 vect2*
Berechnet das Vektorprodukt zweier (dreidimensionaler) Vektoren

list *vect1*
Erster Vektor

list *vect2*
Zweiter Vektor

::math::linearalgebra::matmul *mv1 mv2*
Multipliziert einen Vektor bzw. eine Matrix mit einem anderen Vektor bzw. einer anderen Matrix. Das Ergebnis ist eine Matrix, wenn sowohl x als auch y Matrizen sind oder beide Vektoren sind; in diesem Fall wird das "äußere Produkt" berechnet. Ist das eine ein Vektor und das andere eine Matrix, dann ist das Ergebnis ein Vektor.

list *mv1*
Erster Vektor/erste Matrix (x)

list *mv2*
Zweiter Vektor/zweite Matrix (y)

Common matrices and test matrices

::math::linearalgebra::mkIdentity *size*
Erzeugt eine Einheitsmatrix der Dimension *size*.

integer *size*
Dimension der Matrix

::math::linearalgebra::mkDiagonal *diag*
Erzeugt eine Diagonalmatrix, deren Diagonalelemente die Elemente des Vektors *diag* sind.

list *diag*
Vektor, dessen Elemente für die Diagonale verwendet werden

::math::linearalgebra::mkRandom *size*
Erzeugt eine quadratische Matrix, deren Elemente gleichmäßig verteilte Zufallszahlen zwischen 0 und 1 sind und deren Dimension *size* beträgt.

integer *size*
Dimension der Matrix

::math::linearalgebra::mkTriangular *size* ?uplo? ?value?

Erzeugt eine Dreiecksmatrix, deren Elemente im oberen oder unteren Teil von Null verschieden sind, je nach dem Argument *uplo*.

integer *size*

Dimension der Matrix

string *uplo*

Füllt den oberen (U) oder den unteren Teil (L)

double *value*

Wert, mit dem die Matrix ausgefüllt werden soll

::math::linearalgebra::mkHilbert *size*

Erzeugt eine Hilbert-Matrix der Dimension *size*. Hilbert-Matrizen weisen bei Eigenwert- und Eigenvektorproblemen eine sehr schlechte Kondition auf. Daher eignen sie sich gut zum Testen der Genauigkeit von Algorithmen und Implementierungen.

integer *size*

Dimension der Matrix

::math::linearalgebra::mkDingdong *size*

Erzeugt eine "Dingdong"-Matrix der Dimension *size*. Dingdong-Matrizen werden zwar ungenau dargestellt, weisen jedoch die Eigenschaft auf, in Algorithmen wie der Gaußschen Elimination sehr stabil zu sein.

integer *size*

Dimension der Matrix

::math::linearalgebra::mkOnes *size*

Erzeugt eine quadratische Matrix der Dimension *size*, deren Einträge alle den Wert 1 haben.

integer *size*

Dimension der Matrix

::math::linearalgebra::mkMoler *size*

Erzeugt eine Moler-Matrix der Größe *size*. (Moler-Matrizen weisen eine sehr einfache Cholesky-Zerlegung auf. Sie besitzen einen kleinen Eigenwert und können Eliminationsmethoden für lineare Gleichungssysteme leicht stören.)

integer *size*

Dimension der Matrix

::math::linearalgebra::mkFrank *size*

Erzeugt eine Frank-Matrix der Größe *size*. (Frank-Matrizen sind recht gutartige Matrizen.)

integer *size*

Dimension der Matrix

::math::linearalgebra::mkBorder *size*

Erzeugt eine geränderte Matrix der Größe *size*. (Geränderte Matrizen weisen einen sehr niedrigen Rang auf und können bestimmte spezialisierte Algorithmen beeinträchtigen.)

integer *size*

Dimension der Matrix

::math::linearalgebra::mkWilkinsonW+ *size*
Erzeugt eine Wilkinson-Matrix W+ der Größe *size*. Diese Art von Matrix weist Paare von Eigenwerten auf, die sehr nahe beieinander liegen. In der Regel ist die Ordnung (Größe) ungerade.
integer *size*
Dimension der Matrix

::math::linearalgebra::mkWilkinsonW- *size*
Erzeugt eine Wilkinson-Matrix W- der Größe *size*. Diese Art von Matrix weist Paare von Eigenwerten mit entgegengesetzten Vorzeichen auf, wenn die Ordnung (Größe) ungerade ist.
integer *size*
Dimension der Matrix

Common algorithms

::math::linearalgebra::solveGauss *matrix bvect*
Löst ein lineares Gleichungssystem (Ax=b) mithilfe der Gaußschen Elimination. Gibt die Lösung (x) als Vektor oder Matrix derselben Form wie bvect zurück.
list *matrix*
Quadratische Matrix (Matrix A)
list *bvect*
Vektor oder Matrix, deren Spalten die einzelnen b-Vektoren sind

::math::linearalgebra::solvePGauss *matrix bvect*
Löst ein lineares Gleichungssystem (Ax=b) mithilfe der Gauß-Elimination mit partieller Pivotierung. Gibt die Lösung (x) als Vektor oder Matrix derselben Form wie bvect zurück.
list *matrix*
Quadratische Matrix (Matrix A)
list *bvect*
Vektor oder Matrix, deren Spalten die einzelnen b-Vektoren sind

::math::linearalgebra::solveTriangular *matrix bvect* ?uplo?
Löst ein lineares Gleichungssystem (Ax=b) durch Rückwärtseinsetzen. Es wird angenommen, dass die Matrix obere Dreiecksform aufweist.
list *matrix*
Unter- oder Oberdreiecksmatrix (Matrix A)
list *bvect*
Vektor oder Matrix, deren Spalten die einzelnen b-Vektoren sind
string *uplo*
Gibt an, ob es sich bei der Matrix um eine untere Dreiecksmatrix (L) oder eine obere Dreiecksmatrix (U) handelt. Der Standardwert ist "U".

::math::linearalgebra::solveGaussBand *matrix bvect*
Löst ein lineares Gleichungssystem (Ax=b) mithilfe der Gauß-Elimination, wobei die Matrix als Bandmatrix gespeichert ist (*vgl.* Speicherung). Gibt die Lösung (x) als Vektor oder Matrix derselben Form wie bvect zurück.
list *matrix*

Quadratische Matrix (Matrix A; in Bandform)

list *bvect*

Vektor oder Matrix, deren Spalten die einzelnen b-Vektoren sind

::math::linearalgebra::solveTriangularBand *matrix bvect*

Löst ein lineares Gleichungssystem (Ax=b) durch Rückwärtseinsetzen. Es wird angenommen, dass die Matrix eine obere Dreiecksmatrix ist und in Bandform gespeichert ist.

list *matrix*

Obere Dreiecksmatrix (Matrix A)

list *bvect*

Vektor oder Matrix, deren Spalten die einzelnen b-Vektoren sind

::math::linearalgebra::determineSVD *A eps*

Ermittelt die Singulärwertzerlegung einer Matrix: A = U S Vtrans. Gibt eine Liste zurück, die die Matrix U, den Vektor der Singulärwerte S und die Matrix V enthält.

list *A*

Zu zerlegende Matrix

float *eps*

Toleranz (Standardwert: 2.3e-16)

::math::linearalgebra::eigenvectorsSVD *A eps*

Ermittelt die Eigenvektoren und Eigenwerte einer reellen *symmetrischen* Matrix mithilfe der SVD. Gibt eine Liste zurück, die die Matrix der normierten Eigenvektoren und deren Eigenwerte enthält.

list *A*

Matrix, deren Eigenwerte bestimmt werden müssen

float *eps*

Toleranz (Standardwert: 2.3e-16)

::math::linearalgebra::leastSquaresSVD *A y qmin eps*

Ermittelt die Lösung für ein Problem der kleinsten Quadrate Ax ~ y mittels Singulärwertzerlegung. Das Ergebnis ist der Vektor x.

Hinweis: Wenn Sie der Matrix eine Spalte mit lauter Einsen hinzufügen, stellt diese Spalte eine Konstante dar, wie in: y = a*x1 + b*x2 + c. Soll der Achsenabschnitt Null sein, lassen Sie die Spalte einfach weg.

list *A*

Matrix der unabhängigen Variablen

list *y*

Liste der gemessenen Werte

float *qmin*

Kleinster zu berücksichtigender Singulärwert (Standardwert: 0.0)

float *eps*

Toleranz (Standardwert: 2.3e-16)

::math::linearalgebra::choleski *matrix*

Ermittelt die Cholesky-Zerlegung einer symmetrischen, positiv semidefiniten Matrix (diese

Bedingung wird nicht überprüft!). Das Ergebnis ist die untere Dreiecksmatrix L, für die gilt: L Lt = matrix.

list *matrix*
Zu zerlegende Matrix

::math::linearalgebra::orthonormalizeColumns *matrix*
Orthogonalisiert und normiert die *Spalten* der angegebenen Matrix mit dem modifizierten Gram-Schmidt-Verfahren und gibt das Ergebnis zurück.

list *matrix*
Matrix, deren Spalten orthonormiert werden sollen

::math::linearalgebra::orthonormalizeRows *matrix*
Orthogonalisiert und normiert die *Zeilen* der angegebenen Matrix mit dem modifizierten Gram-Schmidt-Verfahren und gibt das Ergebnis zurück.

list *matrix*
Matrix, deren Zeilen orthonormiert werden sollen

::math::linearalgebra::dger *matrix alpha x y* ?scope?
Führt die Rang-1-Operation A + alpha*x*y' direkt an der Matrix aus (das heißt: Die Matrix A wird verändert). Der Einfachheit halber wird die neue Matrix außerdem als Ergebnis zurückgegeben.

list *matrix*
Matrix, deren Zeilen angepasst werden müssen

double *alpha*
Skalierungsfaktor

list *x*
Ein Spaltenvektor

list *y*
Ein Spaltenvektor

list *scope*
Ohne Angabe wird die Operation auf alle Zeilen/Spalten von A angewendet; mit Angabe wird die Liste {imin imax jmin jmax} erwartet, wobei:
- *imin* Kleinster Zeilenindex
- *imax* Größter Zeilenindex
- *jmin* Kleinster Spaltenindex
- *jmax* Größter Spaltenindex

::math::linearalgebra::dgetrf *matrix*
Berechnet eine LU-Zerlegung einer allgemeinen Matrix mit partieller Pivotierung und Zeilenvertauschungen. Gibt den Permutationsvektor zurück.

Die Zerlegung hat die Form

```
P * A = L * U
```

wobei P eine Permutationsmatrix ist, L eine untere Dreiecksmatrix mit Einheitselementen auf der Diagonalen und U eine obere Dreiecksmatrix. Gibt den Permutationsvektor als Liste der

Länge n-1 zurück. Der letzte Eintrag der Permutation wird nicht gespeichert, da er implizit bekannt ist und den Wert n hat (die letzte Zeile wird mit keiner anderen Zeile vertauscht). Unter dem Index #i der Permutation ist der Index der Zeile #j gespeichert, die im Schritt #i mit der Zeile #i vertauscht wird. Das bedeutet, dass jeder Index der Permutation die Permutation im jeweiligen Schritt angibt, nicht die kumulierte Permutationsmatrix, die das Produkt der Permutationen ist.

list *matrix*

Bei der Eingabe: die zu faktorisierende Matrix. Bei der Ausgabe: die Faktoren L und U aus der Faktorisierung P*A = L*U; die Einheitsdiagonalelemente von L werden nicht gespeichert.

::math::linearalgebra::det *matrix*

Gibt die Determinante der angegebenen Matrix zurück, basierend auf der PA=LU-Zerlegung, d. h. der Gauß-Elimination mit partieller Pivotierung.

list *matrix*

Quadratische Matrix (Matrix A)

list *ipiv*

Die Pivots (optional). Wenn keine Pivots angegeben werden, wird eine PA=LU-Zerlegung durchgeführt. Wenn Pivots angegeben werden, gehen wir davon aus, dass diese die Pivots enthalten und dass die Matrix A die L- und U-Faktoren enthält, wie sie von dgterf bereitgestellt werden. b-Vektoren

::math::linearalgebra::largesteigen *matrix tolerance maxiter*

Gibt eine Liste zurück, die aus dem (dem Betrag nach) größten Eigenwert und dem zugehörigen Eigenvektor besteht. Verwendet die iterative Potenzmethode, wie sie in Algorithmus 7.3.3 von Golub & Van Loan beschrieben ist. Dieser Algorithmus wird hier für eine dichte Matrix verwendet (wird jedoch üblicherweise für dünn besetzte Matrizen eingesetzt).

list *matrix*

Quadratische Matrix (Matrix A)

double *tolerance*

Die relative Toleranz des Eigenwerts (Standardwert: 1.e-8).

integer *maxiter*

Die maximale Anzahl der Iterationen (Standardwert: 10).

Kompatibilität mit dem LA-Paket Zur Gewährleistung der Kompatibilität mit dem LA-Paket von Hume stehen zwei Prozeduren zur Verfügung:

::math::linearalgebra::to_LA *mv*

Wandelt einen Vektor oder eine Matrix in das vom ursprünglichen LA-Paket verwendete Format um.

list *mv*

Matrix oder Vektor

::math::linearalgebra::from_LA *mv*

Wandelt einen Vektor oder eine Matrix aus dem vom ursprünglichen LA-Paket verwendeten Format in das von der vorliegenden Implementierung verwendete Format um.

list *mv*

Matrix oder Vektor, wie sie im LA-Paket verwendet werden

Speicherung

Während die meisten Prozeduren davon ausgehen, dass die Matrizen in vollständiger Form vorliegen, gehen die Prozeduren *solveGaussBand* und *solveTriangularBand* davon aus, dass die Matrizen als *Bandmatrizen* gespeichert sind. Diese gängige Art "dünn besetzter" Matrizen steht in folgendem Zusammenhang mit gewöhnlichen Matrizen:

- "A" ist eine vollständige Matrix mit N Zeilen und M Spalten.
- "B" ist eine Bandmatrix mit m oberen und unteren Diagonalen und n Zeilen.
- "B" kann in einer gewöhnlichen Matrix mit (2m+1) Spalten (eine für jede Außendiagonale und die Hauptdiagonale) und n Zeilen gespeichert werden.
- Das Element i,j (i = -m,...,m; j =1,...,n) von "B" entspricht dem Element k,j von "A", wobei k = M+i-1 gilt und M mindestens (!) n beträgt, also die Anzahl der Zeilen in "B".
- Um das Element (i,j) der Matrix "B" zu setzen, verwenden Sie:

```
    setelem B $j [expr {$N+$i-1}] $value
```

(Hierfür gibt es bislang noch keine vereinfachte Prozedur.)

Anmerkungen zur Umsetzung

Es besteht ein Unterschied zwischen dem ursprünglichen LA-Paket von Hume und der aktuellen Implementierung. Während das LA-Paket eine lineare Liste verwendet, nutzt das aktuelle Paket Listen von Listen zur Darstellung von Matrizen. Es hat sich herausgestellt, dass die Algorithmen mit dieser Darstellung schneller und einfacher zu implementieren sind.

Das LA-Paket diente als Vorlage, und tatsächlich wurde beispielsweise die Implementierung des SVD-Algorithmus aus diesem Paket übernommen. Der Funktionsumfang wurde unter Verwendung von Ideen aus der bekannten BLAS-Bibliothek erweitert, und einige Algorithmen wurden auf der Grundlage der zweiten Auflage des Buches von J.C. Nash, "Compact Numerical Methods for Computers" (Adam Hilger, 1990), aktualisiert, das als Inspiration für das LA-Paket diente.

Um den Übergang zwischen den beiden Implementierungen zu erleichtern, stehen zwei Prozeduren zur Verfügung: *to_LA* und *from_LA*. Diese wurden oben beschrieben.

Noch zu tun

Dies und das: Die folgenden Algorithmen wurden noch nicht implementiert:

- determineQR
- certainlyPositive, diagonallyDominant

Namenskonflikt

Wenn Sie dieses Paket in einer Tk-fähigen Shell wie wish laden, dann führt der Befehl

```
namespace import ::math::linearalgebra
```

zu einer Fehlermeldung bezüglich "scale". Dies liegt daran, dass Tk alle seine Befehle im globalen Namespace definiert. Die Lösung besteht darin, die Befehle der linearen Algebra in einen Namespace zu importieren, der nicht der globale ist:

```
package require math::linearalgebra
namespace eval compute {
    namespace import ::math::linearalgebra::*
    ... use the linear algebra version of scale ...
}
```

Um den Befehl "scale" von Tk in demselben Namespace zu verwenden, können Sie ihn umbenennen:

```
namespace eval compute {
    rename ::scale scaleTk
    scaleTk .scale ...
}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: linearalgebra* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright


```
Copyright (c) 2004-2008 Arjen Markus
    <arjenmarkus@users.sourceforge.net>
Copyright (c) 2004 Ed Hume <http://www.hume.com/contact.us.htm>
Copyright (c) 2008 Michael Buadin <relaxkmike@users.sourceforge.net>
```


machineparameters

Stichwörter: machineparameters

Name

math::machineparameters – Berechnung der Maschinenparameter mit doppelter Genauigkeit.

Übersicht

package require Tcl 8.5 9
package require snit
package require math::machineparameters 0.2
machineparameters create *objectname* ?*options...*?
objectname **configure** ?*options...*?
objectname **cget** *opt*
objectname **destroy**

objectname **compute**
objectname **get** *key*
objectname **tostring**
objectname **print**

Beschreibung

Das Paket *math::machineparameters* ist das Tcl-Äquivalent zur LAPACK-Funktion DLAMCH. In Gleitkommasystemen wird eine Gleitkommazahl wie folgt dargestellt:

```
x = +/- d1 d2 ... dt basis^e
```

wobei die Ziffern die folgende Bedingung erfüllen:

```
0 <= di <= basis - 1, i = 1, t
```

gemäß der Konvention:

- t ist die Größe der Mantisse
- basis ist die Basis (die "Radix").

Die Methode **compute** berechnet alle Maschinenparameter. Anschließend kann die Methode **get** verwendet werden, um die einzelnen Parameter abzurufen. Die Methode **print** gibt einen Bericht auf der Standardausgabe aus.

Beispiel

Im folgenden Beispiel werden die Parameter eines Desktops unter Linux mit den folgenden Tcl-8.4.19-Eigenschaften berechnet:

```
% parray tcl_platform
tcl_platform(byteOrder) = littleEndian
tcl_platform(machine)   = i686
tcl_platform(os)        = Linux
tcl_platform(osVersion) = 2.6.24-19-generic
tcl_platform(platform)  = unix
tcl_platform(tip,268)   = 1
tcl_platform(tip,280)   = 1
tcl_platform(user)      = <username>
tcl_platform(wordSize)  = 4
```

Im folgenden Beispiel wird ein "machineparameters"-Objekt erstellt, dessen Eigenschaften berechnet und anschließend angezeigt.

```
     set pp [machineparameters create %AUTO%]
     $pp compute
     $pp print
     $pp destroy
```

Dies gibt Folgendes aus:

```
    Machine parameters
    Epsilon : 1.11022302463e-16
    Beta : 2
    Rounding : proper
    Mantissa : 53
    Maximum exponent : 1024
    Minimum exponent : -1021
    Overflow threshold : 8.98846567431e+307
    Underflow threshold : 2.22507385851e-308
```

Dies lässt sich gut mit den Ergebnissen von Lapack 3.1.1 vergleichen:

```
    Epsilon                      =   1.11022302462515654E-016
    Safe minimum                 =   2.22507385850720138E-308
    Base                         =    2.0000000000000000
    Precision                    =   2.22044604925031308E-016
    Number of digits in mantissa =    53.000000000000000
    Rounding mode                =   1.00000000000000000
    Minimum exponent             =   -1021.0000000000000
    Underflow threshold          =   2.22507385850720138E-308
    Largest exponent             =    1024.0000000000000
    Overflow threshold           =   1.79769313486231571E+308
    Reciprocal of safe minimum   =   4.49423283715578977E+307
```

Im folgenden Beispiel wird ein "machineparameters"-Objekt erstellt, die Eigenschaften berechnet und der Epsilon-Wert für die Maschine ermittelt.

```
    set pp [machineparameters create %AUTO%]
    $pp compute
    set eps [$pp get -epsilon]
    $pp destroy
```

Literatur

- "Algorithms to Reveal Properties of Floating-Point Arithmetic", Michael A. Malcolm, Stanford University, Communications of the ACM, Band 15, Ausgabe 11 (November 1972), Seiten: 949–951
- "More on Algorithms that Reveal Properties of Floating, Point Arithmetic Units", W. Morven Gentleman, University of Waterloo, Scott B. Marovich, Purdue University, Communications of the ACM, Band 17, Ausgabe 5 (Mai 1974), Seiten: 276–277

API für Klassen

machineparameters create *objectname* ?*options*...?

Der Befehl erstellt ein neues "machineparameters"-Objekt und gibt als Ergebnis den vollqualifizierten Namen des Objektbefehls zurück.

-verbose *verbose*

Setzen Sie diese Option auf 1, um die ausführliche Protokollierung zu aktivieren. Diese Option dient in erster Linie zu Debugging-Zwecken. Der Standardwert von *verbose* ist 0.

Objekt-API

***objectname* configure ?*options*...?**
Der Befehl konfiguriert die Optionen des Objekts *objectname*. Die Optionen entsprechen denen der statischen Methode **create**.

objectname* cget *opt
Gibt den Wert der Option mit dem Namen *opt* zurück. Die Optionen entsprechen denen der Methoden **create** und **configure**.

***objectname* destroy**
Zerstört das Objekt *objectname*.

***objectname* compute**
Berechnet die Maschinenparameter.

objectname* get *key
Gibt den Wert zurück, der dem angegebenen Schlüssel entspricht. Nachfolgend finden Sie eine Liste der verfügbaren Schlüssel.
- -epsilon: Kleinster Wert, für den 1+epsilon>1 falsch ist
- -rounding: Der auf der Maschine verwendete Rundungsmodus. Die Rundung erfolgt, wenn zur Darstellung der Zahl mehr als t Ziffern erforderlich wären. Mit dem aktuellen System lassen sich zwei Modi ermitteln: "chop" bedeutet, dass unabhängig vom Wert der Zahl nur t Ziffern beibehalten werden; "proper" bedeutet, dass ein anderer Rundungsmodus verwendet wird, sei es "auf die nächste Zahl runden", "aufrunden" oder "abrunden".
- -basis: Die Basis der Gleitkommadarstellung. Die Basis ist in der Regel 2, d. h. die binäre Darstellung (z. B. bei IEEE-754-Rechnern), doch einige Maschinen (wie beispielsweise HP-Taschenrechner) verwenden 10, 16 usw.
- -mantissa: Die Anzahl der Bits in der Mantisse
- -exponentmax: der größte positive Exponent, bevor ein Überlauf auftritt
- -exponentmin: Der größte negative Exponent, bevor es zu einem (allmählichen) Unterlauf kommt
- -vmax: Größter positiver Wert, bevor ein Überlauf auftritt
- -vmin: Größter negativer Wert, bevor ein (allmählicher) Unterlauf auftritt

***objectname* tostring**
Gibt einen Bericht zu den Maschinenparametern zurück.

***objectname* print**
Gibt die Maschinenparameter auf der Standardausgabe aus.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright

Copyright (c) 2008 Michael Baudin <michael.baudin@sourceforge.net>

math

Stichwörter: math, statistics

Name

math – Tcl-Mathematikbibliothek

Übersicht

package require Tcl 8.5 9
package require math ?**1.2.6**?
::math::cov *value value* ?*value* ...?
::math::integrate *list of xy value pairs*
::math::fibonacci *n*
::math::max *value* ?*value* ...?
::math::mean *value* ?*value* ...?
::math::min *value* ?*value* ...?
::math::product *value* ?*value* ...?
::math::random ?*value1*? ?*value2*?
::math::sigma *value value* ?*value* ...?
::math::stats *value value* ?*value* ...?
::math::sum *value* ?*value* ...?

Beschreibung

Das Paket **math** stellt mathematische Hilfsfunktionen bereit.

Neben einer Reihe von Grundbefehlen, die über das Paket *math* verfügbar sind, gibt es noch speziellere Pakete:

- **math::bigfloat** – Gleitkommaarithmetik mit beliebiger Genauigkeit
- **math::bignum** – Ganzzahlarithmetik mit beliebiger Genauigkeit
- **math::calculus::romberg** – Robuste Integrationsmethoden für Funktionen einer Variablen unter Verwendung der Romberg-Integration
- **math::calculus** – Integration von Funktionen, Lösen gewöhnlicher Differentialgleichungen
- **math::combinatorics** – Prozeduren für verschiedene kombinatorische Funktionen (beispielsweise die Gammafunktion und "k aus n")
- **math::complexnumbers** – Arithmetik mit komplexen Zahlen
- **math::constants** – Eine Sammlung bekannter mathematischer Konstanten wie Pi, E und der Goldene Schnitt
- **math::fourier** – Diskrete Fourier-Transformationen
- **math::fuzzy** – Unscharfe Vergleiche von Gleitkommazahlen
- **math::geometry** – Geometrische Berechnungen in 2D
- **math::interpolate** – Verschiedene Interpolationsmethoden
- **math::linearalgebra** – Paket zur linearen Algebra
- **math::optimize** – Optimierungsmethoden
- **math::polynomials** – Polynomarithmetik (einschließlich Familien klassischer Polynome)
- **math::rationalfunctions** – Arithmetik gebrochenrationaler Funktionen

- **math::roman** – Umgang mit römischen Ziffern (einschließlich Rechenoperationen)
- **math::special** – Approximationen spezieller Funktionen aus der mathematischen Physik
- **math::statistics** – Statistische Verfahren und Tests

Grundlegende Befehle

::math::cov *value value ?value ...?*
Gibt den Variationskoeffizienten von zwei oder mehr numerischen Werten in Prozent zurück.

::math::integrate *list of xy value pairs*
Gibt die Fläche unter einer "Kurve", die durch eine Menge von (x,y)-Paaren definiert ist, sowie die Fehlergrenze als Liste zurück.

::math::fibonacci *n*
Gibt die *n*-te Fibonacci-Zahl zurück.

::math::max *value ?value ...?*
Gibt das Maximum eines oder mehrerer numerischer Werte zurück.

::math::mean *value ?value ...?*
Gibt den Mittelwert oder den "Durchschnitt" eines oder mehrerer numerischer Werte zurück.

::math::min *value ?value ...?*
Gibt das Minimum eines oder mehrerer numerischer Werte zurück.

::math::product *value ?value ...?*
Gibt das Produkt aus einem oder mehreren numerischen Werten zurück.

::math::random ?*value1*? ?*value2*?
Gibt eine Zufallszahl zurück. Werden keine Argumente angegeben, ist die Zahl ein Gleitkommawert zwischen 0 und 1. Wird ein Argument angegeben, ist die Zahl ein ganzzahliger Wert zwischen 0 und *value1*. Werden zwei Argumente angegeben, ist die Zahl ein ganzzahliger Wert zwischen *value1* und *value2*.

::math::sigma *value value ?value ...?*
Gibt die Stichprobenstandardabweichung von zwei oder mehr numerischen Werten zurück.

::math::stats *value value ?value ...?*
Gibt den Mittelwert, die Standardabweichung und den Variationskoeffizienten (in Prozent) als Liste zurück.

::math::sum *value ?value ...?*
Gibt die Summe eines oder mehrerer numerischer Werte zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

math_geometry

Stichwörter: math_geometry, angle, distance, line, math, plane geometry, point

Name

math::geometry – Geometrische Berechnungen

Übersicht

package require Tcl ?**8.5 9**?
package require math::geometry ?**1.4.2**?
::math::geometry::+ *point1 point2*
::math::geometry::- *point1 point2*
::math::geometry::p *x y*
::math::geometry::distance *point1 point2*
::math::geometry::length *point*
::math::geometry::s* *factor point*
::math::geometry::direction *angle*
::math::geometry::h *length*
::math::geometry::v *length*
::math::geometry::between *point1 point2 s*
::math::geometry::octant *point*
::math::geometry::rect *nw se*
::math::geometry::nwse *rect*
::math::geometry::angle *line*
::math::geometry::angleBetween *vector1 vector2*
::math::geometry::inproduct *vector1 vector2*
::math::geometry::areaParallellogram *vector1 vector2*
::math::geometry::calculateDistanceToLine *P line*
::math::geometry::calculateDistanceToLineSegment *P linesegment*
::math::geometry::calculateDistanceToPolyline *P polyline*
::math::geometry::calculateDistanceToPolygon *P polygon*
::math::geometry::findClosestPointOnLine *P line*
::math::geometry::findClosestPointOnLineSegment *P linesegment*
::math::geometry::findClosestPointOnPolyline *P polyline*
::math::geometry::lengthOfPolyline *polyline*
::math::geometry::movePointInDirection *P direction dist*
::math::geometry::lineSegmentsIntersect *linesegment1 linesegment2*
::math::geometry::findLineSegmentIntersection *linesegment1 linesegment2*
::math::geometry::findLineIntersection *line1 line2*
::math::geometry::polylinesIntersect *polyline1 polyline2*
::math::geometry::polylinesBoundingIntersect *polyline1 polyline2 granularity*
::math::geometry::intervalsOverlap *y1 y2 y3 y4 strict*
::math::geometry::rectanglesOverlap *P1 P2 Q1 Q2 strict*
::math::geometry::bbox *polyline*
::math::geometry::overlapBBox *polyline1 polyline2* ?*strict*?
::math::geometry::pointInsideBBox *bbox point*
::math::geometry::cathetusPoint *pa pb cathetusLength* ?*location*?

::math::geometry::parallel *line offset ?orient?*
::math::geometry::unitVector *line*
::math::geometry::pointInsidePolygon *P polyline*
::math::geometry::pointInsidePolygonAlt *P polyline*
::math::geometry::rectangleInsidePolygon *P1 P2 polyline*
::math::geometry::areaPolygon *polygon*
::math::geometry::translate *vector polyline*
::math::geometry::rotate *angle polyline*
::math::geometry::rotateAbout *p angle polyline*
::math::geometry::reflect *angle polyline*
::math::geometry::degToRad *angle*
::math::geometry::radToDeg *angle*
::math::geometry::circle *centre radius*
::math::geometry::circleTwoPoints *point1 point2*
::math::geometry::pointInsideCircle *point circle*
::math::geometry::lineIntersectsCircle *line circle*
::math::geometry::lineSegmentIntersectsCircle *segment circle*
::math::geometry::intersectionLineWithCircle *line circle*
::math::geometry::intersectionCircleWithCircle *circle1 circle2*
::math::geometry::tangentLinesToCircle *point circle*
::math::geometry::intersectionPolylines *polyline1 polyline2 ?mode? ?granularity?*
::math::geometry::intersectionPolylineCircle *polyline circle ?mode? ?granularity?*
::math::geometry::polylineCutOrigin *polyline1 polyline2 ?granularity?*
::math::geometry::polylineCutEnd *polyline1 polyline2 ?granularity?*
::math::geometry::splitPolyline *polyline numberVertex*
::math::geometry::enrichPolyline *polyline accuracy*
::math::geometry::cleanupPolyline *polyline*

Beschreibung

Das Paket **math::geometry** ist eine Sammlung von Funktionen für Berechnungen und Bearbeitungen an zweidimensionalen geometrischen Objekten wie Punkten, Geraden und Polygonen.

Die geometrischen Objekte werden als einfache Listen von Koordinaten implementiert. So wird beispielsweise eine Gerade durch eine Liste von vier Zahlen definiert, nämlich die x- und y-Koordinaten eines ersten Punktes sowie die x- und y-Koordinaten eines zweiten Punktes auf dieser Geraden.

Hinweis: In Version 1.4.0 wurde eine Unstimmigkeit behoben – siehe https://core.tcl-lang.org/tcllib/tktview?name=fb4812f82b. Weitere Informationen finden Sie im Abschnitt Koordinatensystem

Die verschiedenen Objekttypen werden an der Anzahl der Koordinatenpaare und am Kontext, in dem sie verwendet werden, erkannt: Eine Liste mit vier Elementen kann als unendliche Gerade, als endliche Strecke, aber auch als Polylinie mit einem Segment oder als Punktmenge mit zwei Punkten betrachtet werden.

Derzeit werden folgende Objekttypen unterschieden:

- *point* (Punkt) – eine Liste mit zwei Koordinaten, die die x- bzw. die y-Koordinate darstellen.

- *line* (Gerade) – eine Liste mit vier Koordinaten, die als x- und y-Koordinaten zweier unterschiedlicher Punkte auf der Geraden interpretiert werden.
- *line segment* (Strecke) – eine Liste mit vier Koordinaten, die als x- und y-Koordinaten des ersten und des letzten Punktes der Strecke interpretiert werden.
- *polyline* (Polylinie) – eine Liste mit einer geraden Anzahl von Koordinaten, die als x- und y-Koordinaten einer geordneten Punktmenge interpretiert werden.
- *polygon* (Polygon) – ähnlich wie eine Polylinie, wobei jedoch implizit davon ausgegangen wird, dass die Polylinie geschlossen ist (wenn der erste und der letzte Punkt nicht zusammenfallen, wird das fehlende Segment automatisch hinzugefügt).
- *point set* (Punktmenge) – wiederum eine Liste mit einer geraden Anzahl von Koordinaten, wobei die Punkte jedoch ohne bestimmte Reihenfolge betrachtet werden.
- *circle* (Kreis) – eine Liste mit drei Zahlen, wobei die ersten beiden die Koordinaten des Mittelpunkts sind und die dritte der Radius ist.

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren:

::math::geometry::+ *point1 point2*
Berechnet die Summe der beiden als Punkte angegebenen Vektoren und gibt sie zurück. Das Ergebnis ist ebenfalls ein Vektor.

::math::geometry::- *point1 point2*
Berechnet die Differenz (point1 - point2) der beiden als Punkte angegebenen Vektoren und gibt sie zurück. Das Ergebnis ist ebenfalls ein Vektor.

::math::geometry::p *x y*
Erstellt einen Punkt aus seinen Koordinaten und gibt ihn als Ergebnis des Befehls zurück.

::math::geometry::distance *point1 point2*
Berechnet den Abstand zwischen den beiden Punkten und gibt ihn als Ergebnis des Befehls zurück. Dies entspricht im Wesentlichen

```
    math::geometry::length [math::geomtry::- point1 point2]
```

::math::geometry::length *point*
Berechnet die Länge des Vektors und gibt sie als Ergebnis des Befehls zurück.

::math::geometry::s* *factor point*
Skaliert den Vektor mit dem Faktor und gibt ihn als Ergebnis des Befehls zurück. Auch hierbei handelt es sich um einen Vektor.

::math::geometry::direction *angle*
Bei Angabe des Winkels in Grad berechnet dieser Befehl den Einheitsvektor, der in diese Richtung zeigt, und gibt ihn zurück. Der Vektor für angle == 0 zeigt nach rechts (Osten), der für angle == 90 nach oben (Norden).

::math::geometry::h *length*
Gibt einen horizontalen Vektor auf der X-Achse mit der festgelegten Länge zurück. Positive Längen weisen nach rechts (Osten).

::math::geometry::v *length*
Gibt einen vertikalen Vektor auf der Y-Achse mit der festgelegten Länge zurück. Positive Längen weisen nach unten (Süden).

::math::geometry::between *point1 point2 s*

Berechnet den Punkt, der sich im relativen Abstand *s* zwischen den beiden Punkten befindet, und gibt ihn als Ergebnis des Befehls zurück. Ein relativer Abstand von **0** liefert *point1*, der Abstand **1** liefert *point2*. Bei Abständen < 0 oder > 1 erfolgt eine Extrapolation entlang der Verbindungslinie zwischen den beiden Punkten.

::math::geometry::octant *point*

Berechnet den Oktanten des Kreises, in dem sich der Punkt befindet, und gibt ihn als Ergebnis des Befehls zurück. Die möglichen Ergebnisse sind

1. Osten
2. Nordosten
3. Norden
4. Nordwesten
5. Westen
6. Südwesten
7. Süden
8. Südosten

Jeder Oktant ist der Kreisbogen von +/- 22,5 Grad um die Himmelsrichtung, nach der der Oktant seinen Namen hat.

::math::geometry::rect *nw se*

Erstellt ein Rechteck aus seiner nordwestlichen und südöstlichen Ecke und gibt es als Ergebnis des Befehls zurück.

::math::geometry::nwse *rect*

Ermittelt die nordwestliche und die südöstliche Ecke des Rechtecks und gibt sie als Ergebnis des Befehls zurück (eine Liste mit zwei Elementen, die die Punkte in der angegebenen Reihenfolge enthält).

::math::geometry::angle *line*

Berechnet den Winkel von der positiven x-Achse zu einer gegebenen Geraden (nur in zwei Dimensionen).

list *line*

Koordinaten der Geraden

::math::geometry::angleBetween *vector1 vector2*

Berechnet den Winkel zwischen zwei Vektoren (in Grad)

list *vector1*

Erster Vektor

list *vector2*

Zweiter Vektor

::math::geometry::inproduct *vector1 vector2*

Berechnet das Skalarprodukt zweier Vektoren

list *vector1*

Erster Vektor

list *vector2*

Zweiter Vektor

::math::geometry::areaParallellogram *vector1 vector2*

Berechnet die Fläche des Parallelogramms, das die beiden Vektoren als Seiten hat

list *vector1*
Erster Vektor

list *vector2*
Zweiter Vektor

::math::geometry::calculateDistanceToLine *P line*
Berechnet den Abstand des Punktes P zur (unendlichen) Geraden und gibt das Ergebnis zurück

list *P*
Liste mit zwei Zahlen, den Koordinaten des Punktes

list *line*
Liste mit vier Zahlen, die Koordinaten zweier Punkte auf der Geraden

::math::geometry::calculateDistanceToLineSegment *P linesegment*
Berechnet den Abstand des Punktes P zur (endlichen) Strecke und gibt das Ergebnis zurück.

list *P*
Liste mit zwei Zahlen, den Koordinaten des Punktes

list *linesegment*
Liste mit vier Zahlen, den Koordinaten des ersten und des letzten Punktes der Strecke

::math::geometry::calculateDistanceToPolyline *P polyline*
Berechnet den Abstand des Punktes P zur Polylinie und gibt das Ergebnis zurück. Hinweis: Eine Polylinie muss nicht geschlossen sein.

list *P*
Liste mit zwei Zahlen, den Koordinaten des Punktes

list *polyline*
Liste von Zahlen, die die Koordinaten der Eckpunkte der Polylinie darstellen

::math::geometry::calculateDistanceToPolygon *P polygon*
Berechnet den Abstand des Punktes P zum Polygon und gibt das Ergebnis zurück. Wenn die Liste der Koordinaten nicht geschlossen ist (erster und letzter Punkt unterscheiden sich), wird sie automatisch geschlossen.

list *P*
Liste mit zwei Zahlen, den Koordinaten des Punktes

list *polygon*
Liste von Zahlen, den Koordinaten der Eckpunkte des Polygons

::math::geometry::findClosestPointOnLine *P line*
Gibt den Punkt auf einer Geraden zurück, der einem vorgegebenen Punkt am nächsten liegt.

list *P*
Liste mit zwei Zahlen, den Koordinaten des Punktes

list *line*
Liste mit vier Zahlen, die Koordinaten zweier Punkte auf der Geraden

::math::geometry::findClosestPointOnLineSegment *P linesegment*
Gibt den Punkt auf einer *Strecke* zurück, der einem vorgegebenen Punkt am nächsten liegt.

list *P*
Liste mit zwei Zahlen, den Koordinaten des Punktes

list *linesegment*
Liste mit vier Zahlen, dem ersten und dem letzten Punkt der Strecke

::math::geometry::findClosestPointOnPolyline *P polyline*
Gibt den Punkt auf einer Polylinie *polyline* zurück, der einem vorgegebenen Punkt am nächsten liegt.

list *P*
Liste mit zwei Zahlen, den Koordinaten des Punktes

list *polyline*
Liste von Zahlen, den Eckpunkten der Polylinie

::math::geometry::lengthOfPolyline *polyline*
Gibt die Länge der Polylinie *polyline* zurück (Hinweis: Sie wird nicht als Polygon betrachtet)

list *polyline*
Liste von Zahlen, den Eckpunkten der Polylinie

::math::geometry::movePointInDirection *P direction dist*
Verschiebt einen Punkt um einen gegebenen Abstand in eine gegebene Richtung und gibt die neuen Koordinaten zurück (nur in zwei Dimensionen).

list *P*
Koordinaten des zu verschiebenden Punktes

double *direction*
Richtung (in Grad; 0 ist nach rechts, 90 nach oben)

list *dist*
Entfernung, um die der Punkt verschoben werden soll

::math::geometry::lineSegmentsIntersect *linesegment1 linesegment2*
Prüft, ob sich zwei Strecken schneiden oder zusammenfallen. Gibt 1 zurück, wenn dies der Fall ist, andernfalls 0 (nur in zwei Dimensionen). Liegt ein Endpunkt einer Strecke auf der anderen Strecke (oder sehr nahe daran), gelten sie als sich schneidend.

list *linesegment1*
Erste Strecke

list *linesegment2*
Zweite Strecke

::math::geometry::findLineSegmentIntersection *linesegment1 linesegment2*
Ermittelt den Schnittpunkt zweier Strecken. Gibt die Koordinaten zurück oder die Schlüsselwörter "coincident" bzw. "none", wenn die Strecken zusammenfallen oder keine gemeinsamen Punkte haben (nur in zwei Dimensionen).

list *linesegment1*
Erste Strecke

list *linesegment2*
Zweite Strecke

::math::geometry::findLineIntersection *line1 line2*

Ermittelt den Schnittpunkt zweier (unendlicher) Geraden. Gibt die Koordinaten zurück oder die Schlüsselwörter "coincident" bzw. "none", wenn die Geraden zusammenfallen oder keine gemeinsamen Punkte haben (nur in zwei Dimensionen).

list *line1*
Erste Gerade

list *line2*
Zweite Gerade

Einzelheiten zum Algorithmus und zur zugrunde liegenden Mathematik finden Sie im Abschnitt Literatur.

::math::geometry::polylinesIntersect *polyline1 polyline2*
Prüft, ob sich zwei Polylinien schneiden oder nicht (nur in zwei Dimensionen).

list *polyline1*
Erste Polylinie

list *polyline2*
Zweite Polylinie

::math::geometry::polylinesBoundingIntersect *polyline1 polyline2 granularity*
Prüft, ob sich zwei Polylinien schneiden, wobei die Genauigkeit des Ergebnisses auf die angegebene Granularität beschränkt wird. Verwenden Sie diese Prozedur für eine schnellere, jedoch weniger genaue Schnittprüfung.

Funktionsweise:

Jede Polylinie wird in eine Reihe kleinerer Polylinien unterteilt, die jeweils aus *granularity* Punkten bestehen. Wenn sich die umschließenden Rechtecke eines Paares dieser kleineren Linien schneiden, gibt diese Prozedur 1 zurück, andernfalls 0.

list *polyline1*
Erste Polylinie

list *polyline2*
Zweite Polylinie

int *granularity*
Anzahl der Punkte in jedem Teil (<=1 bedeutet, dass jede Kante überprüft wird)

::math::geometry::intervalsOverlap *y1 y2 y3 y4 strict*
Prüft, ob sich zwei Intervalle überlappen.

double *y1,y2*
Beginn und Ende des ersten Intervalls

double *y3,y4*
Beginn und Ende des zweiten Intervalls

logical *strict*
Prüft auf strenge oder nicht strenge Überlappung

::math::geometry::rectanglesOverlap *P1 P2 Q1 Q2 strict*
Prüft, ob sich zwei Rechtecke überlappen.

list *P1*
obere linke Ecke des ersten Rechtecks

list *P2*
untere rechte Ecke des ersten Rechtecks
list *Q1*
obere linke Ecke des zweiten Rechtecks
list *Q2*
untere rechte Ecke des zweiten Rechtecks
list *strict*
Wahl zwischen strenger und nicht strenger Auslegung

::math::geometry::bbox *polyline*
Berechnet das umschließende Rechteck einer Polylinie. Gibt eine Liste mit vier Koordinaten zurück: die obere linke und die untere rechte Ecke des Rechtecks.
list *polyline*
Die zu untersuchende Polylinie

::math::geometry::overlapBBox *polyline1 polyline2* ?strict?
Prüft, ob sich die umschließenden Rechtecke zweier Polylinien überlappen oder nicht.

Argumente:
list *polyline1*
Die erste Polylinie
list *polyline1*
Die zweite Polylinie
int *strict*
Ob auf strenge Überlappung geprüft werden soll (1) oder ob sich die umschließenden Rechtecke berühren dürfen (0, Standard)

::math::geometry::pointInsideBBox *bbox point*
Prüft, ob sich der Punkt innerhalb oder auf dem Rand des umschließenden Rechtecks befindet oder nicht. Argumente:
list *bbox*
Das umschließende Rechteck als Liste von x/y-Koordinaten
list *point*
Der zu prüfende Punkt

::math::geometry::cathetusPoint *pa pb cathetusLength* ?location?
Gibt den dritten Punkt des rechtwinkligen Dreiecks zurück, das durch die beiden angegebenen Endpunkte der Hypotenuse definiert ist. Die Seite des Dreiecks vom Punkt A (oder B, wenn als Lage "b" angegeben ist) bis zum dritten Punkt hat die Länge der Kathete. Ist die Länge der Kathete größer als die Länge der Hypotenuse, wird eine leere Liste zurückgegeben.

Argumente:
list *pa*
Der Ausgangspunkt auf der Hypotenuse
list *pb*
Der Endpunkt auf der Hypotenuse

float *cathetusLength*
Die Länge der Kathete des Dreiecks

string *location*
Die Lage der angegebenen Kathete: "a" bedeutet, dass die angegebene Kathete den Punkt pa enthält (Standard), "b" bedeutet, dass die angegebene Kathete den Punkt pb enthält.

::math::geometry::parallel *line offset* ?orient?
Gibt eine zur angegebenen Geraden parallele Gerade im Abstand "offset" zurück. Die Ausrichtung wird durch die beiden Punkte bestimmt, die die Gerade definieren.

Argumente:
list *line*
Die angegebene Gerade

float *offset*
Der Abstand zur angegebenen Geraden

string *orient*
Ausrichtung der neuen Geraden relativ zur angegebenen Geraden (Standardwert: "right")

::math::geometry::unitVector *line*
Gibt einen Einheitsvektor zur angegebenen Geraden zurück, oder zur Richtung, wenn das Argument *line* ein einzelner Punkt ist (dann wird eine Gerade durch den Ursprung angenommen). Argumente:
list *line*
Die betreffende Gerade (oder ein einzelner Punkt, was eine Gerade durch den Ursprung impliziert)

::math::geometry::pointInsidePolygon *P polyline*
Stellt fest, ob sich ein Punkt vollständig innerhalb eines Polygons befindet. Wenn der Punkt das Polygon berührt, so befindet er sich nicht vollständig innerhalb des Polygons.
list *P*
Koordinaten des Punktes

list *polyline*
Die zu untersuchende Polylinie

::math::geometry::pointInsidePolygonAlt *P polyline*
Stellt fest, ob sich ein Punkt vollständig innerhalb eines Polygons befindet. Wenn der Punkt das Polygon berührt, so befindet er sich nicht vollständig innerhalb des Polygons. *Hinweis:* Diese alternative Prozedur nutzt dafür die sogenannte Windungszahl. Sie behandelt sich selbst schneidende Polygone auf "natürliche" Weise.
list *P*
Koordinaten des Punktes

list *polyline*
Die zu untersuchende Polylinie

::math::geometry::rectangleInsidePolygon *P1 P2 polyline*
Stellt fest, ob sich ein Rechteck vollständig innerhalb eines Polygons befindet. Berührt das Polygon das Rechteck, so befindet sich das Rechteck nicht vollständig innerhalb des Polygons.

list *P1*
Obere linke Ecke des Rechtecks
list *P2*
Untere rechte Ecke des Rechtecks
list *polygon*
Das betreffende Polygon

::math::geometry::areaPolygon *polygon*
Berechnet die Fläche eines Polygons.
list *polygon*
Das betreffende Polygon

::math::geometry::translate *vector polyline*
Verschiebt eine Polylinie um einen gegebenen Vektor
list *vector*
Verschiebungsvektor
list *polyline*
Die zu verschiebende Polylinie

::math::geometry::rotate *angle polyline*
Dreht eine Polylinie um einen gegebenen Winkel (in Grad) um den Ursprung
list *angle*
Winkel, um den die Polylinie gedreht werden soll (Grad)
list *polyline*
Die zu drehende Polylinie

::math::geometry::rotateAbout *p angle polyline*
Dreht eine Polylinie um einen gegebenen Punkt p und gibt die neue Polylinie zurück.

Argumente:
list *p*
Der Drehpunkt
float *angle*
Der Winkel, um den die Polylinie gedreht werden soll (Grad)
list *polyline*
Die zu drehende Polylinie

::math::geometry::reflect *angle polyline*
Spiegelt eine Polylinie an einer Geraden durch den Ursprung, die einen gegebenen Winkel (in Grad) mit der x-Achse bildet
list *angle*
Winkel der Spiegelachse (Grad)
list *polyline*
Die zu spiegelnde Polylinie

::math::geometry::degToRad *angle*
Rechnet von Grad in Radiant um

list *angle*
Winkel in Grad

::math::geometry::radToDeg *angle*
Rechnet von Radiant in Grad um
list *angle*
Winkel in Radiant

::math::geometry::circle *centre radius*
Hilfsprozedur, die einen Kreis aus einem Punkt und einem Radius erstellt.
list *centre*
Koordinaten des Kreismittelpunkts
list *radius*
Radius des Kreises

::math::geometry::circleTwoPoints *point1 point2*
Hilfsprozedur, die einen Kreis aus zwei Punkten auf seinem Umfang erstellt. Der Mittelpunkt ist der Punkt zwischen den beiden gegebenen Punkten, der Radius entspricht der Hälfte des Abstands zwischen diesen beiden Punkten.
list *point1*
Erster Punkt
list *point2*
Zweiter Punkt

::math::geometry::pointInsideCircle *point circle*
Stellt fest, ob der angegebene Punkt innerhalb des Kreises oder auf dem Kreisumfang liegt (1) oder außerhalb (0).
list *point*
Zu prüfender Punkt
list *circle*
Ein Kreis, der den Punkt enthalten kann oder auch nicht

::math::geometry::lineIntersectsCircle *line circle*
Stellt fest, ob die angegebene Gerade den Kreis schneidet oder ihn berührt (1) oder nicht (0).
list *line*
Zu prüfende Gerade
list *circle*
Kreis, der möglicherweise geschnitten wird oder auch nicht

::math::geometry::lineSegmentIntersectsCircle *segment circle*
Stellt fest, ob die angegebene Strecke den Kreis schneidet oder ihn berührt (1) oder nicht (0).
list *segment*
Zu prüfende Strecke
list *circle*
Kreis, der möglicherweise geschnitten wird oder auch nicht

::math::geometry::intersectionLineWithCircle *line circle*

Bestimmt die Punkte, an denen die angegebene Gerade den Kreis schneidet. Es kann null, ein oder zwei Punkte geben. (Wenn die Gerade den Kreis berührt oder ihm sehr nahe kommt, wird ein Punkt zurückgegeben. Zur Erkennung dieses Falls wird eine willkürlich gewählte Toleranz von 1.0e-10 mal dem Radius verwendet.)

list *line*
Zu prüfende Gerade

list *circle*
Kreis, der möglicherweise geschnitten wird oder auch nicht

::math::geometry::intersectionCircleWithCircle *circle1 circle2*

Bestimmt die Schnittpunkte der beiden angegebenen Kreise. Es kann null, ein oder zwei Schnittpunkte geben. (Wenn sich die beiden Kreise berühren oder sehr nahe kommen, wird ein Punkt zurückgegeben. Zur Erkennung dieses Falls wird eine willkürlich gewählte Toleranz von 1.0e-10 mal dem Mittelwert der Radien der beiden Kreise verwendet.)

list *circle1*
Erster Kreis

list *circle2*
Zweiter Kreis

::math::geometry::tangentLinesToCircle *point circle*

Ermittelt die Tangenten vom angegebenen Punkt an den Kreis. Es kann null, eine oder zwei Geraden geben. (Wenn der Punkt auf dem Kreisumfang oder sehr nahe am Kreis liegt, wird eine Gerade zurückgegeben. Zur Erkennung dieses Falls wird eine willkürlich gewählte Toleranz von 1.0e-10 mal dem Kreisradius verwendet.)

list *point*
Der betreffende Punkt

list *circle*
Kreis, für den die Tangenten bestimmt werden sollen

::math::geometry::intersectionPolylines *polyline1 polyline2* ?mode? ?granularity?

Gibt den ersten Punkt oder alle Punkte zurück, an denen sich die beiden Polylinien schneiden. Wenn die Anzahl der Punkte in den Polylinien groß ist, können Sie die Granularität nutzen, um schneller ein Näherungsergebnis zu erhalten.

Argumente:

list *polyline1*
Die erste Polylinie

list *polyline2*
Die zweite Polylinie

string *mode*
Ob nur der erste (Standard) oder alle Schnittpunkte ("all") zurückgegeben werden sollen

int *granularity*
Die Anzahl der Punkte, die bei der Suche nach Schnittpunkten übersprungen werden, zuzüglich 1 (ein Wert von 1 oder kleiner bedeutet, dass ein exaktes Ergebnis zurückgegeben wird)

::math::geometry::intersectionPolylineCircle *polyline circle* ?mode? ?granularity?

Gibt den ersten Punkt oder alle Punkte zurück, an denen die Polylinie den Kreis schneidet. Wenn die Anzahl der Punkte in der Polylinie groß ist, können Sie die Granularität nutzen, um schneller ein Näherungsergebnis zu erhalten.

Argumente:

list *polyline*
Die Polylinie, die den Kreis möglicherweise schneidet

list *circle*
Der betreffende Kreis

string *mode*
Ob nur der erste (Standard) oder alle Schnittpunkte ("all") zurückgegeben werden sollen

int *granularity*
Die Anzahl der Punkte, die bei der Suche nach Schnittpunkten übersprungen werden, zuzüglich 1 (ein Wert von 1 oder kleiner bedeutet, dass ein exaktes Ergebnis zurückgegeben wird)

::math::geometry::polylineCutOrigin *polyline1 polyline2* ?granularity?
Gibt den Abschnitt der ersten Polylinie vom Ursprung bis zum ersten Schnittpunkt mit der zweiten Polylinie zurück. Wenn die Anzahl der Punkte in der Polylinie groß ist, können Sie die Granularität nutzen, um schneller ein Näherungsergebnis zu erhalten.

Argumente:

list *polyline1*
Die erste Polylinie (von der ein Teil zurückgegeben werden soll)

list *polyline2*
Die zweite Polylinie

int *granularity*
Die Anzahl der Punkte, die bei der Suche nach Schnittpunkten übersprungen werden, zuzüglich 1 (ein Wert von 1 oder kleiner bedeutet, dass ein exaktes Ergebnis zurückgegeben wird)

::math::geometry::polylineCutEnd *polyline1 polyline2* ?granularity?
Gibt den Abschnitt der ersten Polylinie zurück, der vom letzten Schnittpunkt mit der zweiten Polylinie bis zum Ende reicht. Wenn die Anzahl der Punkte in der Polylinie groß ist, können Sie die Granularität nutzen, um schneller ein Näherungsergebnis zu erhalten.

Argumente:

list *polyline1*
Die erste Polylinie (von der ein Teil zurückgegeben werden soll)

list *polyline2*
Die zweite Polylinie

int *granularity*
Die Anzahl der Punkte, die bei der Suche nach Schnittpunkten übersprungen werden, zuzüglich 1 (ein Wert von 1 oder kleiner bedeutet, dass ein exaktes Ergebnis zurückgegeben wird)

::math::geometry::splitPolyline *polyline numberVertex*
Teilt die Polylinie in eine Reihe von Polylinien auf, wobei jede einzelne Polylinie

"numberVertex" Eckpunkte zwischen den beiden Endpunkten enthält.

Argumente:
list *polyline*
Die zu teilende Polylinie
int *numberVertex*
Die Anzahl der "inneren" Eckpunkte

::math::geometry::enrichPolyline *polyline accuracy*
Unterteilt jedes Segment einer Polylinie in mehrere kleinere Segmente und gibt das Ergebnis zurück.

Argumente:
list *polyline*
Die zu verfeinernde Polylinie
int *accuracy*
Die Anzahl der zu erstellenden Teilsegmente

::math::geometry::cleanupPolyline *polyline*
Entfernt doppelte benachbarte Eckpunkte und gibt das Ergebnis zurück.

Argumente:
list *polyline*
Die zu bereinigende Polylinie

Koordinatensystem

Das vom Paket verwendete Koordinatensystem ist das gewöhnliche kartesische Koordinatensystem, bei dem die positive x-Achse nach rechts und die positive y-Achse nach oben zeigt. Winkel und Richtungen werden in Bezug auf die positive x-Achse gegen den Uhrzeigersinn definiert, sodass ein Winkel von 90 Grad der Richtung der positiven y-Achse entspricht. Hinweis: Die Tk-Canvas-Koordinaten unterscheiden sich davon, da sich dort der Ursprung in der oberen linken Ecke des Fensters befindet. Bis einschließlich Version 1.3 verwendeten die Prozeduren direction und octant dieses Pakets diese Konvention uneinheitlich.

Literatur

1. Polygon Intersection
2. http://en.wikipedia.org/wiki/Line-line_intersection
3. http://local.wasp.uwa.edu.au/~pbourke/geometry/lineline2d/

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: geometry* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

Copyright

```
Copyright (c) 2001 by Ideogramic ApS and other parties
Copyright (c) 2010 by Andreas Kupries
Copyright (c) 2010 by Kevin Kenny
Copyright (c) 2018 by Arjen Markus
Copyright (c) 2020 by Manfred Rosenberger
```


numtheory
Stichwörter: numtheory, number theory, prime

Name
math::numtheory – Zahlentheorie

Übersicht
package require Tcl ?**8.5 9**?
package require math::numtheory ?**1.1.4**?
math::numtheory::isprime *N* ?*option value* ...?
math::numtheory::firstNprimes *N*
math::numtheory::primesLowerThan *N*
math::numtheory::primeFactors *N*
math::numtheory::primesLowerThan *N*
math::numtheory::primeFactors *N*
math::numtheory::uniquePrimeFactors *N*
math::numtheory::factors *N*
math::numtheory::totient *N*
math::numtheory::moebius *N*
math::numtheory::legendre *a p*
math::numtheory::jacobi *a b*
math::numtheory::gcd *m n*
math::numtheory::lcm *m n*
math::numtheory::numberPrimesGauss *N*
math::numtheory::numberPrimesLegendre *N*
math::numtheory::numberPrimesLegendreModified *N*
math::numtheory::differenceNumberPrimesLegendreModified *lower upper*
math::numtheory::listPrimePairs *lower upper step*
math::numtheory::listPrimeProgressions *lower upper step*

Beschreibung
Dieses Paket dient der Zusammenstellung verschiedener zahlentheoretischer Operationen, wobei der Schwerpunkt leicht auf Primzahlen liegt.

math::numtheory::isprime *N* ?*option value* ...?
Der Befehl **isprime** testet, ob die ganze Zahl *N* eine Primzahl ist, und gibt den booleschen Wert "true" zurück, wenn *N* eine Primzahl ist, sowie den booleschen Wert "false", wenn *N*

keine Primzahl ist. Die verwendete formale Definition von "Primzahl" entspricht der gängigen Definition, wonach die zu prüfende Zahl größer als 1 ist und nur triviale Teiler besitzt.

Genauer gesagt ist der Rückgabewert einer der Werte **0** (wenn *N* definitiv keine Primzahl ist), **1** (wenn *N* definitiv eine Primzahl ist) und **on** (wenn *N* wahrscheinlich eine Primzahl ist); die beiden letztgenannten Werte sind jeweils boolesche "true"-Werte. Der Fall, dass eine ganze Zahl als "wahrscheinlich prim" eingestuft werden kann, ergibt sich daraus, dass der in der Testimplementierung verwendete Miller-Rabin-Algorithmus grundsätzlich probabilistisch ist und bei ungünstigen Umständen möglicherweise nicht erkennt, dass eine Zahl tatsächlich zusammengesetzt ist. Mit Optionen lässt sich das Risiko solcher "falsch positiven Ergebnisse" im Test wählen. Für "kleine" *N* (was derzeit $N < 118670087467$ bedeutet) wird **1** zurückgegeben, da bekannt ist, dass hier keine falsch positiven Ergebnisse möglich sind.

Die einzige derzeit definierte Option lautet:

-randommr *repetitions*
: die festlegt, wie oft der Miller-Rabin-Test mit zufällig ausgewählten Basen wiederholt werden soll. Jede Wiederholung verringert die Wahrscheinlichkeit eines falsch-positiven Ergebnisses um mindestens den Faktor 4. Der Standardwert für *repetitions* beträgt 4.

Unbekannte Optionen werden stillschweigend ignoriert.

math::numtheory::firstNprimes *N*

Gibt die ersten N Primzahlen zurück

integer *N* (in)
: Anzahl der Primzahlen, die zurückgegeben werden

math::numtheory::primesLowerThan *N*

Gibt die Primzahlen zurück, die kleiner oder gleich N sind.

integer *N* (in)
: Größte zu berücksichtigende Zahl

math::numtheory::primeFactors *N*

Gibt eine Liste der Primfaktoren der Zahl N zurück.

integer *N* (in)
: Zu faktorisierende Zahl

math::numtheory::primesLowerThan *N*

Gibt die Primzahlen zurück, die kleiner oder gleich N sind.

integer *N* (in)
: Größte zu berücksichtigende Zahl

math::numtheory::primeFactors *N*

Gibt eine Liste der Primfaktoren der Zahl N zurück.

integer *N* (in)
: Zu faktorisierende Zahl

math::numtheory::uniquePrimeFactors *N*

Gibt eine Liste der *eindeutigen* Primfaktoren der Zahl N zurück.

integer *N* (in)

Zu faktorisierende Zahl

math::numtheory::factors *N*
Gibt eine Liste aller *eindeutigen* Teiler der Zahl N zurück, einschließlich 1 und N selbst.
integer *N* (in)
Zu faktorisierende Zahl

math::numtheory::totient *N*
Berechnet die Eulersche Phi-Funktion (Totientfunktion) für die Zahl N (Anzahl der zu N teilerfremden Zahlen).
integer *N* (in)
Die betreffende Zahl

math::numtheory::moebius *N*
Berechnet die Möbius-Funktion für die Zahl N
integer *N* (in)
Die betreffende Zahl

math::numtheory::legendre *a p*
Berechnet das Legendre-Symbol (a/p)
integer *a* (in)
Die obere Zahl im Symbol
integer *p* (in)
Die untere Zahl im Symbol (muss ungleich Null sein)

math::numtheory::jacobi *a b*
Berechnet das Jacobi-Symbol (a/b)
integer *a* (in)
Die obere Zahl im Symbol
integer *b* (in)
Die untere Zahl im Symbol (muss ungerade sein)

math::numtheory::gcd *m n*
Gibt den größten gemeinsamen Teiler von *m* und *n* zurück.
integer *m* (in)
Erste Zahl
integer *n* (in)
Zweite Zahl

math::numtheory::lcm *m n*
Gibt das kleinste gemeinsame Vielfache von *m* und *n* zurück.
integer *m* (in)
Erste Zahl
integer *n* (in)
Zweite Zahl

math::numtheory::numberPrimesGauss *N*

Schätzt die Anzahl der Primzahlen gemäß der Formel von Gauß.

integer *N* (in)
Die betreffende Zahl; sollte größer als 0 sein

math::numtheory::numberPrimesLegendre *N*
Schätzt die Anzahl der Primzahlen gemäß der Formel von Legendre.

integer *N* (in)
Die betreffende Zahl; sollte größer als 0 sein

math::numtheory::numberPrimesLegendreModified *N*
Schätzt die Anzahl der Primzahlen gemäß der modifizierten Formel von Legendre.

integer *N* (in)
Die betreffende Zahl; sollte größer als 0 sein

math::numtheory::differenceNumberPrimesLegendreModified *lower upper*
Schätzt die Anzahl der Primzahlen zwischen zwei Grenzen gemäß der modifizierten Formel von Legendre.

integer *lower* (in)
Untergrenze für die Primzahlen; sollte größer als 0 sein

integer *upper* (in)
Obergrenze für die Primzahlen; sollte größer als 0 sein

math::numtheory::listPrimePairs *lower upper step*
Gibt eine Liste von Primzahlpaaren zurück, die jeweils um den angegebenen Schritt voneinander abweichen.

integer *lower* (in)
Untergrenze für die Primzahlen; sollte größer als 0 sein

integer *upper* (in)
Obergrenze für die Primzahlen; sollte größer als die Untergrenze sein

integer *step* (in)
Schrittweite, um die sich die Primzahlen unterscheiden sollen; Standardwert ist 2

math::numtheory::listPrimeProgressions *lower upper step*
Gibt eine Liste von Listen mit Primzahlen zurück, die sich jeweils um den angegebenen Schritt vom vorherigen Wert unterscheiden.

integer *lower* (in)
Untergrenze für die Primzahlen; sollte größer als 0 sein

integer *upper* (in)
Obergrenze für die Primzahlen; sollte größer als die Untergrenze sein

integer *step* (in)
Schrittweite, um die sich die Primzahlen unterscheiden sollen; Standardwert ist 2

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: numtheory* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

Copyright
```
Copyright (c) 2010 Lars Hellström <Lars dot Hellstrom at residenset
    dot net>
```

optimize
Stichwörter: optimize, linear program, math, maximum, minimum, optimization

Name
math::optimize – Optimierungsroutinen

Übersicht
package require Tcl 8.5 9
package require math::optimize ?**1.0**?
::math::optimize::minimum *begin end func maxerr*
::math::optimize::maximum *begin end func maxerr*
::math::optimize::min_bound_1d *func begin end* ?*-relerror reltol*? ?*-abserror abstol*? ?*-maxiter maxiter*? ?*-trace traceflag*?
::math::optimize::min_unbound_1d *func begin end* ?*-relerror reltol*? ?*-abserror abstol*? ?*-maxiter maxiter*? ?*-trace traceflag*?
::math::optimize::solveLinearProgram *objective constraints*
::math::optimize::linearProgramMaximum *objective result*
::math::optimize::nelderMead *objective xVector* ?*-scale xScaleVector*? ?*-ftol epsilon*? ?*-maxiter count*? ??*-trace*? *flag?*

Beschreibung
Dieses Paket implementiert mehrere Optimierungsalgorithmen:

- Eine Funktion über ein gegebenes Intervall minimieren oder maximieren
- Ein lineares Programm lösen (eine lineare Funktion unter linearen Nebenbedingungen maximieren)
- Eine Funktion mehrerer Variablen minimieren, ausgehend von einer Anfangsschätzung für die Lage des Minimums.

Das Paket ist vollständig in Tcl implementiert. Der Genauigkeit der Berechnungen wurde keine besondere Aufmerksamkeit geschenkt. Stattdessen wurden die Algorithmen auf unkomplizierte Weise angewendet.

Dieses Dokument beschreibt die Prozeduren und erläutert deren Verwendung.

Prozeduren

Dieses Paket definiert die folgenden öffentlichen Prozeduren:

::math::optimize::minimum *begin end func maxerr*

Minimiert die angegebene (stetige) Funktion, indem die Werte im angegebenen Intervall untersucht werden. Die Prozedur ermittelt die Werte an beiden Enden sowie in der Mitte des Intervalls und bildet anschließend ein neues Intervall mit der Hälfte der Länge, das das Minimum enthält. Es wird keine Garantie dafür gegeben, dass das *globale* Minimum gefunden wird.

Die Prozedur gibt den "x"-Wert zurück, für den die Funktion ihr Minimum erreicht.

Diese Prozedur ist veraltet – verwenden Sie stattdessen min_bound_1d

begin – Anfang des Intervalls

end – Ende des Intervalls

func – Name der zu minimierenden Funktion (eine Prozedur, die ein Argument entgegennimmt).

maxerr – Maximaler relativer Fehler (Standardwert: 1.0e-4)

::math::optimize::maximum *begin end func maxerr*

Maximiert die angegebene (stetige) Funktion, indem die Werte im angegebenen Intervall untersucht werden. Die Prozedur ermittelt die Werte an beiden Enden sowie in der Mitte des Intervalls und bildet anschließend ein neues Intervall mit der Hälfte der Länge, das das Maximum enthält. Es wird keine Garantie dafür gegeben, dass das *globale* Maximum gefunden wird.

Die Prozedur gibt den "x"-Wert zurück, für den die Funktion ihr Maximum erreicht.

Diese Prozedur ist veraltet – verwenden Sie stattdessen max_bound_1d

begin – Anfang des Intervalls

end – Ende des Intervalls

func – Name der zu maximierenden Funktion (eine Prozedur, die ein Argument entgegennimmt).

maxerr – Maximaler relativer Fehler (Standardwert: 1.0e-4)

::math::optimize::min_bound_1d *func begin end* ?-relerror *reltol*? ?-abserror *abstol*? ?-maxiter *maxiter*? ?-trace *traceflag*?

Minimiert eine Funktion einer Variablen im angegebenen Intervall. Die Prozedur verwendet die Brent-Methode der parabolischen Interpolation, die durch Unterteilungen nach dem Goldenen Schnitt abgesichert ist, wenn die Interpolation nicht konvergiert. Es wird keine Garantie dafür gegeben, dass ein *globales* Minimum gefunden wird. Die auszuwertende Funktion *func* muss ein einzelner Tcl-Befehl sein; sie wird mit einer als letztes Argument angehängten Abszisse

ausgewertet.

x1 und *x2* sind die beiden Grenzen des Intervalls, in dem das Minimum gefunden werden soll. Sie müssen nicht in aufsteigender Reihenfolge stehen.

reltol ist, wenn es angegeben ist, die gewünschte Obergrenze für den relativen Fehler des Ergebnisses; der Standardwert ist 1.0e-7. Der angegebene Wert sollte niemals kleiner sein als die Quadratwurzel der Gleitkommagenauigkeit des Rechners, da sonst die Konvergenz nicht gewährleistet ist. *abstol* ist, sofern angegeben, die gewünschte Obergrenze für den absoluten Fehler des Ergebnisses; der Standardwert beträgt 1.0e-10. Bei kleinen Werten von *abstol* ist Vorsicht geboten, um Überlauf- bzw. Unterlaufbedingungen zu vermeiden; wenn erwartet wird, dass das Minimum bei einer kleinen, aber von Null verschiedenen Abszisse liegt, sollten Sie entweder eine Verschiebung der Funktion oder eine Änderung ihrer Längenskala in Betracht ziehen.

maxiter kann verwendet werden, um die Anzahl der durchzuführenden Funktionsauswertungen zu begrenzen; der Standardwert ist 100. Wenn der Befehl die Funktion mehr als *maxiter* Mal auswertet, gibt er einen Fehler an den Aufrufer zurück.

traceFlag ist ein boolescher Wert. Ist er wahr, gibt der Befehl bei jeder Auswertung der Funktion eine Meldung an die Standardausgabe aus, die die Abszisse und die Ordinate sowie einen Hinweis auf die gewählte Interpolationsart enthält. Der Standardwert ist 0 (keine Protokollierung).

::math::optimize::min_unbound_1d *func begin end* ?-relerror *reltol*? ?-abserror *abstol*? ?-maxiter *maxiter*? ?-trace *traceflag*?

Minimiert eine Funktion einer Variablen über die gesamte reelle Zahlenachse. Die Prozedur nutzt eine parabolische Extrapolation in Kombination mit einer Dilatation nach dem Goldenen Schnitt, um einen Bereich zu suchen, in dem ein Minimum vorliegt, gefolgt von Brents Methode der parabolischen Interpolation, die durch Unterteilungen nach dem Goldenen Schnitt abgesichert wird, wenn die Interpolation nicht konvergiert. Es wird keine Garantie dafür gegeben, dass ein *globales* Minimum gefunden wird. Die auszuwertende Funktion *func* muss ein einzelner Tcl-Befehl sein; sie wird mit einer als letztes Argument angehängten Abszisse ausgewertet.

x1 und *x2* sind zwei anfängliche Vermutungen, wo das Minimum liegen könnte. *x1* ist der Startpunkt für die Minimierung, und die Differenz zwischen *x2* und *x1* dient als Hinweis auf die charakteristische Längenskala des Problems.

reltol ist, wenn es angegeben ist, die gewünschte Obergrenze für den relativen Fehler des Ergebnisses; der Standardwert ist 1.0e-7. Der angegebene Wert sollte niemals kleiner sein als die Quadratwurzel der Gleitkommagenauigkeit des Rechners, da sonst die Konvergenz nicht gewährleistet ist. *abstol* ist, sofern angegeben, die gewünschte Obergrenze für den absoluten Fehler des Ergebnisses; der Standardwert beträgt 1.0e-10. Bei kleinen Werten für *abstol* ist Vorsicht geboten, um Überlauf- bzw. Unterlaufbedingungen zu vermeiden; wenn erwartet wird, dass das Minimum bei einer kleinen, aber von Null verschiedenen Abszisse liegt, sollten Sie entweder eine Verschiebung der Funktion oder eine Änderung ihrer Längenskala in

Betracht ziehen.

maxiter kann verwendet werden, um die Anzahl der durchzuführenden Funktionsauswertungen zu begrenzen; der Standardwert ist 100. Wenn der Befehl die Funktion mehr als *maxiter* Mal auswertet, gibt er einen Fehler an den Aufrufer zurück.

traceFlag ist ein boolescher Wert. Wenn er auf "true" gesetzt ist, gibt der Befehl bei jeder Auswertung einer Funktion eine Meldung an die Standardausgabe aus, die die Abszisse und die Ordinate sowie einen Hinweis auf die gewählte Interpolationsart enthält. Der Standardwert ist 0 (keine Protokollierung).

::math::optimize::solveLinearProgram *objective constraints*

Löst ein *lineares Programm* in Standardform mit einer einfachen Implementierung des Simplex-Algorithmus. (In der folgenden Erläuterung hat das lineare Programm N Nebenbedingungen und M Variablen.)

Die Prozedur gibt eine Liste mit M Werten zurück, bei denen die Zielfunktion ihr Maximum erreicht, oder ein einzelnes Schlüsselwort, wenn das lineare Programm unzulässig oder unbeschränkt ist (entweder "unfeasible" oder "unbounded").

objective – Die M Koeffizienten der Zielfunktion

constraints – Matrix aus Koeffizienten und Maximalwerten, die die linearen Nebenbedingungen darstellen. Erwartet wird eine Liste aus N Listen mit jeweils M+1 Zahlen: den M Koeffizienten und dem Maximalwert.

::math::optimize::linearProgramMaximum *objective result*

Hilfsfunktion, die den Maximalwert für die von der Prozedur solveLinearProgram gefundene Lösung zurückgibt.

objective – Die M Koeffizienten der Zielfunktion

result – Das von solveLinearProgram zurückgegebene Ergebnis

::math::optimize::nelderMead *objective xVector* ?-scale *xScaleVector*? ?-ftol *epsilon*? ?-maxiter *count*? ??-trace? *flag*?

Minimiert auf uneingeschränkte Weise eine Funktion mehrerer Variablen über den gesamten Raum. Die auszuwertende Funktion, *objective*, muss ein einzelner Tcl-Befehl sein. An diesen werden so viele Elemente angehängt, wie in der Anfangsschätzung für die Lage des Minimums enthalten sind, die als Tcl-Liste *xVector* übergeben wird.

xScaleVector ist eine anfängliche Schätzung des Problemmaßstabs; die ersten Auswertungen der Funktion erfolgen durch Variation der Koordinaten in *xVector* um die in *xScaleVector* angegebenen Beträge. Wenn *xScaleVector* nicht angegeben wird, werden die Koordinaten um den Faktor 1.0001 (sofern die Koordinate ungleich Null ist) oder um den konstanten Wert 0.0001 (sofern die Koordinate Null ist) variiert.

epsilon ist der gewünschte relative Fehler des am Minimum ausgewerteten Funktionswerts. Der Standardwert ist 1.0e-7, was in der Regel eine Genauigkeit von drei signifikanten Stellen

bei den x-Werten ergibt.

pp *count* ist eine Begrenzung der Anzahl der Durchläufe durch die Hauptschleife des Optimierers. Die Anzahl der Funktionsauswertungen kann ein Vielfaches dieser Zahl betragen. Wenn der Optimierer innerhalb von *maxiter* Iterationen kein Minimum mit einer Genauigkeit von *ftol* findet, gibt er seine aktuell beste Schätzung sowie einen Fehlerstatus zurück. Standardmäßig sind 500 Iterationen zulässig.

flag ist ein Flag, das, falls es wahr ist, bewirkt, dass bei jeder Auswertung der Zielfunktion eine Zeile an die Standardausgabe geschrieben wird, die die an die Funktion übergebenen Argumente und den zurückgegebenen Wert enthält. Der Standardwert ist "false".

Die Prozedur **nelderMead** gibt eine Liste abwechselnder Schlüsselwörter und Werte zurück, die für die Verwendung mit **array set** geeignet sind. Die Schlüsselwörter haben folgende Bedeutung:

x ist die ungefähre Position des Minimums.

y ist der Wert der Funktion am Punkt *x*.

yvec ist ein Vektor der besten N+1 erzielten Funktionswerte, wobei N die Dimension von *x* ist.

vertices ist eine Liste von Vektoren, die die Funktionsargumente angeben, die den Werten in *yvec* entsprechen.

nIter ist die Anzahl der Iterationen, die erforderlich sind, um Konvergenz zu erreichen oder zu scheitern.

status ist "ok", wenn der Vorgang erfolgreich war, oder "too-many-iterations", wenn die maximale Iterationsanzahl überschritten wurde.

nelderMead minimiert die angegebene Funktion unter Verwendung der Downhill-Simplex-Methode von Nelder und Mead. Diese Methode ist recht langsam – es sind wesentlich schnellere Methoden zur Minimierung bekannt –, hat jedoch den Vorteil, dass sie bei Problemen, bei denen das Minimum in einem Tal komplexer Topologie liegt, äußerst robust ist.

nelderMead kann gelegentlich an einem Punkt "hängenbleiben", an dem es keine weiteren Fortschritte mehr erzielen kann; es wird empfohlen, dass der Aufrufer die Funktion mindestens ein zweites Mal ausführt und dabei das vom vorherigen Aufruf gefundene Ergebnis als Anfangsschätzung übergibt. Der zweite Durchlauf ist in der Regel sehr schnell.

nelderMead kann in einigen Fällen für die Optimierung unter Nebenbedingungen verwendet werden. Fügen Sie dazu der Zielfunktion einen großen Wert hinzu, wenn sich die Parameter außerhalb des zulässigen Bereichs befinden. Um in diesem Modus effektiv zu arbeiten, erfordert **nelderMead**, dass der Anfangswert zulässig ist, und in der Regel, dass der

zulässige Bereich konvex ist.

Hinweise

Einige der oben genannten Prozeduren nehmen die *Namen* von Prozeduren als Argumente entgegen. Um Probleme mit der *Sichtbarkeit* dieser Prozeduren zu vermeiden, wird der vollqualifizierte Name dieser Prozeduren innerhalb der Optimierungsroutinen ermittelt. Für den Benutzer hat dies nur eine Konsequenz: Die benannte Prozedur muss in der aufrufenden Prozedur sichtbar sein. Zum Beispiel:

```
namespace eval ::mySpace {
   namespace export calcfunc
   proc calcfunc { x } { return $x }
}
#
# Use a fully-qualified name
#
namespace eval ::myCalc {
   puts [min_bound_1d ::myCalc::calcfunc $begin $end]
}
#
# Import the name
#
namespace eval ::myCalc {
   namespace import ::mySpace::calcfunc
   puts [min_bound_1d calcfunc $begin $end]
}
```

Die einfachen Prozeduren *minimum* und *maximum* sind veraltet: Die Alternativen sind wesentlich flexibler, robuster und erfordern weniger Funktionsauswertungen.

Beispiele

Lassen Sie uns einige einfache Beispiele betrachten:

Bestimmen Sie das Maximum von f(x) = x^3 exp(-3x) auf dem Intervall (0,10):

```
proc efunc { x } { expr {$x*$x*$x * exp(-3.0*$x)} }
puts "Maximum at: [::math::optimize::max_bound_1d efunc 0.0 10.0]"
```

Der maximal zulässige Fehler bestimmt die Anzahl der durchgeführten Schritte (bei jedem Schritt der Iteration wird das Intervall um den Faktor 1/2 verringert). Somit wird ein maximaler Fehler von 0.0001 in etwa 14 Schritten erreicht.

Ein Beispiel für ein *lineares Programm*:

Optimieren Sie den Ausdruck 3x+2y, wobei:

```
   x >= 0 and y >= 0 (implicit constraints, part of the
                     definition of linear programs)

   x + y   <= 1      (constraints specific to the problem)
   2x + 5y <= 10
```

Dieses Problem lässt sich wie folgt lösen:

```
   set solution [::math::optimize::solveLinearProgram  { 3.0   2.0 }
    { { 1.0   1.0   1.0 }
        { 2.0   5.0  10.0 } } ]
```

Hinweis: Eine Nebenbedingung wie:

```
   x + y >= 1
```

lässt sich so in die Standardform bringen:

```
   -x  -y <= -1
```

Die Theorie der linearen Programmierung ist Gegenstand zahlreicher Lehrbücher, und der hier implementierte Simplex-Algorithmus ist die bekannteste Methode zur Lösung dieser Art von Problemen, jedoch nicht die einzige.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: optimize* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright


```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
Copyright (c) 2004,2005 Kevn B. Kenny <kennykb@users.sourceforge.net>
```


pca

Stichwörter: pca, math, statistics, tcl

Name

math::PCA – Paket für die Hauptkomponentenanalyse

Übersicht

package require Tcl ?**8.6 9**?
package require math::linearalgebra 1.1
::math::PCA::createPCA *data* ?*args*?
$pca using ?number?|?-minproportion value?
$pca eigenvectors ?*option*?
$pca eigenvalues ?*option*?
$pca proportions ?*option*?
$pca approximate *observation*

$pca approximatOriginal
$pca scores *observation*
$pca distance *observation*
$pca qstatistic *observation* ?*option*?

Beschreibung

Das PCA-Paket bietet die Möglichkeit, eine Hauptkomponentenanalyse in Tcl durchzuführen, wobei eine objektorientierte Technik zum Einsatz kommt, die durch TclOO ermöglicht wird. Es definiert tatsächlich eine einzige öffentliche Methode, *::math::PCA::createPCA*, die auf der Grundlage der übergebenen Daten ein Objekt erstellt, um die eigentliche Analyse durchzuführen.

Die Methoden der mit diesem Befehl erstellten PCA-Objekte ermöglichen es, die Hauptkomponenten zu untersuchen, (neue) Beobachtungen unter Verwendung aller oder nur einer ausgewählten Anzahl von Komponenten zu approximieren sowie die Eigenschaften der Komponenten und die statistischen Merkmale der Approximationen zu untersuchen.

Das Paket orientiert sich am PCA-Beispiel, das im ursprünglichen Linearen-Algebra-Paket von Ed Hume bereitgestellt wurde.

Befehle

Das Paket *math::PCA* stellt einen öffentlichen Befehl bereit:

::math::PCA::createPCA *data* ?args?
Erzeugt ein neues Objekt auf der Grundlage der Daten, die über das Argument *data* übergeben werden. Die Hauptkomponenten können entweder auf Korrelationen oder auf Kovarianzen basieren. Alle Beobachtungen werden entsprechend dem Mittelwert und der Standardabweichung der Originaldaten normalisiert.

list *data*
- Eine Liste von Beobachtungen (siehe das Beispiel unten).

list *args*
- Eine Liste von Schlüssel-Wert-Paaren, die die Optionen definieren. Derzeit gibt es nur einen Schlüssel: *-covariances*. Dieser gibt an, ob Kovarianzen verwendet werden sollen (wenn der Wert 1 ist) oder stattdessen Korrelationen (Wert ist 0). Standardmäßig werden Korrelationen verwendet.

Das erstellte PCA-Objekt verfügt über die folgenden Methoden:

$pca using ?number?|?-minproportion value?
Legt die Anzahl der Komponenten fest, die bei der Analyse verwendet werden sollen (die Anzahl der beibehaltenen Komponenten). Gibt die Anzahl der Komponenten zurück, auch wenn kein Argument angegeben wird.

int *number*
- Die Anzahl der zu behaltenden Komponenten

double *value*
- Wählt die Anzahl der Komponenten anhand des minimalen Anteils der Variation aus, der durch diese Komponenten erhalten bleibt. Der Wert sollte zwischen 0 und 1 liegen.

$pca eigenvectors ?option?
Gibt die Eigenvektoren als Liste von Listen zurück.

string *option*
- Standardmäßig werden nur die *beibehaltenen* Komponenten zurückgegeben. Wenn alle Eigenvektoren benötigt werden, verwenden Sie die Option *-all*.

$pca eigenvalues ?option?
Gibt die Eigenwerte als Liste von Listen zurück.
string *option*
- Standardmäßig werden nur die Eigenwerte der *beibehaltenen* Komponenten zurückgegeben. Wenn alle Eigenwerte benötigt werden, verwenden Sie die Option *-all*.

$pca proportions ?option?
Gibt die Anteile für alle Komponenten zurück, d. h. den Anteil der Variation, den jede Komponente erklären kann.

$pca approximate *observation*
Gibt eine Näherung der Beobachtung auf der Grundlage der beibehaltenen Komponenten zurück
list *observation*
- Die Werte für die Beobachtung.

$pca approximatOriginal
Gibt unter Verwendung der beibehaltenen Komponenten eine Näherung der Originaldaten zurück. Es handelt sich um eine Komfortmethode, die auf den vollständigen Satz der Originaldaten angewendet wird.

$pca scores *observation*
Gibt die Scores (Komponentenwerte) pro beibehaltener Komponente für die angegebene Beobachtung zurück.
list *observation*
- Die Werte für die Beobachtung.

$pca distance *observation*
Gibt den Abstand zwischen der angegebenen Beobachtung und ihrer Näherung zurück. (Hinweis: Dieser Abstand basiert auf den normierten Vektoren.)
list *observation*
- Die Werte für die Beobachtung.

$pca qstatistic *observation* ?option?
Gibt die Q-Statistik – im Wesentlichen das Quadrat des Abstands – für die angegebene Beobachtung zurück.
list *observation*
- Die Werte für die Beobachtung.

string *option*
- Wenn die Beobachtung Teil der Originaldaten ist, möchten Sie möglicherweise die korrigierte Q-Statistik verwenden. Dies erreichen Sie mit der Option "-original".

Beispiel
TODO: NIST-Beispiel

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *PCA* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

polynomials

Stichwörter: polynomials, math, polynomial functions

Name

math::polynomials – Polynomfunktionen

Übersicht

package require Tcl ?**8.5**?
package require math::polynomials ?**1.0.2**?
::math::polynomials::polynomial *coeffs*
::math::polynomials::polynCmd *coeffs*
::math::polynomials::evalPolyn *polynomial x*
::math::polynomials::addPolyn *polyn1 polyn2*
::math::polynomials::subPolyn *polyn1 polyn2*
::math::polynomials::multPolyn *polyn1 polyn2*
::math::polynomials::divPolyn *polyn1 polyn2*
::math::polynomials::remainderPolyn *polyn1 polyn2*
::math::polynomials::derivPolyn *polyn*
::math::polynomials::primitivePolyn *polyn*
::math::polynomials::degreePolyn *polyn*
::math::polynomials::coeffPolyn *polyn index*
::math::polynomials::allCoeffsPolyn *polyn*

Beschreibung

Dieses Paket befasst sich mit Polynomfunktionen einer Variablen:

- die grundlegenden arithmetischen Operationen werden auf Polynome erweitert
- Berechnung der Ableitungen und Stammfunktionen dieser Funktionen
- Auswertung über eine allgemeine Prozedur oder über spezielle Prozeduren)

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren:

::math::polynomials::polynomial *coeffs*

Gibt eine (kodierte) Liste zurück, die das Polynom definiert. Ein Polynom

```
f(x) = a + b.x + c.x**2 + d.x**3
```

kann wie folgt definiert werden:

```
set f [::math::polynomials::polynomial [list $a $b $c $d]
```

list *coeffs*
Koeffizienten des Polynoms (in aufsteigender Reihenfolge)

::math::polynomials::polynCmd *coeffs*
Erzeugt eine neue Prozedur, die das Polynom auswertet. Der Name des Polynoms wird automatisch generiert. Dies ist nützlich, wenn Sie das Polynom mehrfach auswerten müssen, da die Prozedur aus einem einzigen [expr]-Befehl besteht.
list *coeffs*
Koeffizienten des Polynoms (in aufsteigender Reihenfolge) oder die vom Befehl *polynomial* zurückgegebene Polynomdefinition.

::math::polynomials::evalPolyn *polynomial x*
Wertet das Polynom an der Stelle x aus.
list *polynomial*
Die Definition des Polynoms (wie sie vom Befehl polynomial zurückgegeben wird).
float *x*
Die Koordinate, an der das Polynom ausgewertet werden soll

::math::polynomials::addPolyn *polyn1 polyn2*
Gibt ein neues Polynom zurück, das die Summe der beiden anderen ist.
list *polyn1*
Der erste Polynomoperand
list *polyn2*
Der zweite Polynomoperand

::math::polynomials::subPolyn *polyn1 polyn2*
Gibt ein neues Polynom zurück, das der Differenz der beiden anderen entspricht.
list *polyn1*
Der erste Polynomoperand
list *polyn2*
Der zweite Polynomoperand

::math::polynomials::multPolyn *polyn1 polyn2*
Gibt ein neues Polynom zurück, das das Produkt der beiden anderen Polynome ist. Wenn eines der Argumente ein Skalarwert ist, wird das andere Polynom lediglich skaliert.
list *polyn1*
Der erste Polynomoperand oder ein Skalar
list *polyn2*
Der zweite Polynomoperand oder ein Skalar

::math::polynomials::divPolyn *polyn1 polyn2*

Teilt das erste Polynom durch das zweite Polynom und gibt das Ergebnis zurück. Der Rest wird verworfen.

list *polyn1*
Der erste Polynomoperand

list *polyn2*
Der zweite Polynomoperand

::math::polynomials::remainderPolyn *polyn1 polyn2*
Teilt das erste Polynom durch das zweite Polynom und gibt den Rest zurück.

list *polyn1*
Der erste Polynomoperand

list *polyn2*
Der zweite Polynomoperand

::math::polynomials::derivPolyn *polyn*
Differenziert das Polynom und gibt das Ergebnis zurück.

list *polyn*
Das zu differenzierende Polynom

::math::polynomials::primitivePolyn *polyn*
Integriert das Polynom und gibt das Ergebnis zurück. Die Integrationskonstante wird auf Null gesetzt.

list *polyn*
Das zu integrierende Polynom

::math::polynomials::degreePolyn *polyn*
Gibt den Grad des Polynoms zurück.

list *polyn*
Das zu untersuchende Polynom

::math::polynomials::coeffPolyn *polyn index*
Gibt den Koeffizienten des Terms vom Grad index des Polynoms zurück.

list *polyn*
Das zu untersuchende Polynom

int *index*
Der Grad des Terms

::math::polynomials::allCoeffsPolyn *polyn*
Gibt die Koeffizienten des Polynoms (in aufsteigender Reihenfolge) zurück.

list *polyn*
Das betreffende Polynom

Anmerkungen zur Umsetzung

Die Implementierung zur Auswertung der Polynome an einem bestimmten Punkt nutzt die Horner-Regel, die numerische Stabilität und einen minimalen Aufwand an arithmetischen Operationen gewährleistet. Um zu erkennen, dass es sich bei einer Polynomdefinition tatsächlich um eine korrekte Definition handelt, besteht diese aus einer Liste mit zwei Elementen: dem

Schlüsselwort "POLYNOMIAL" und der Liste der Koeffizienten in absteigender Reihenfolge. Letzteres erleichtert die Umsetzung der Hornerschen Regel.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: polynomials* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright


```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```


probopt

Stichwörter: probopt, mathematics, optimisation, probabilistic calculations

Name

math::probopt – Probabilistische Optimierungsmethoden

Übersicht

package require Tcl 8.6 9
package require TclOO
package require math::probopt 1.1
::math::probopt::pso *function bounds args*
::math::probopt::sce *function bounds args*
::math::probopt::diffev *function bounds args*
::math::probopt::lipoMax *function bounds args*
::math::probopt::adaLipoMax *function bounds args*

Beschreibung

Der Zweck des Pakets **math::probopt** besteht darin, verschiedene Optimierungsalgorithmen bereitzustellen, die auf probabilistischen Verfahren basieren. Die Ergebnisse dieser Algorithmen können daher von einem Durchlauf zum nächsten variieren. Die Algorithmen sind allesamt bekannt und gut beschrieben, und ihre Befürworter behaupten im Allgemeinen, dass sie effizient und zuverlässig sind.

Da die meisten dieser Algorithmen über einen oder mehrere einstellbare Parameter oder sogar Varianten verfügen, bietet die jeweilige Schnittstelle Optionen zur Einstellung dieser Parameter bzw. zur Auswahl der Variante. Diese werden in Form von Schlüssel-Wert-Paaren angegeben, beispielsweise *-iterations 100*.

Dieses Handbuch enthält weder Empfehlungen zu diesen Algorithmen noch umfassende

Leitlinien für die Parameter. Hierzu verweisen wir auf Online-Artikel zu den betreffenden Algorithmen.

Ein paar Hinweise jedoch:

- Mit Ausnahme von LIPO sind die Algorithmen in der Lage, unregelmäßige (nicht glatte) und sogar diskontinuierliche Funktionen zu verarbeiten.
- Die Ergebnisse hängen vom Startwert (Seed) des Zufallszahlengenerators ab und sind wahrscheinlich nicht sehr genau, insbesondere wenn sich die Funktion in der Nähe des Optimums nur langsam ändert. Sie bieten jedoch einen guten Ausgangspunkt für einen deterministischen Algorithmus.

Die Sammlung umfasst die folgenden Algorithmen:

- PSO – Partikelschwarm-Optimierung
- SCE – Shuffled Complexes Evolution (Evolution gemischter Komplexe)
- DE – Differentialevolution
- LIPO – Lipschitz-Optimierung

Die verschiedenen Prozeduren verfügen über eine einheitliche Schnittstelle:

```
   set result [::math::probopt::algorithm function bounds args]
```

Die Argumente haben folgende Bedeutung:

- Das Argument *function* ist der Name der Prozedur, die die Funktion auswertet. Ihre Schnittstelle lautet:
  ```
      set value [function coords]
  ```
- wobei *coords* eine Liste von Koordinaten ist, an denen die Auswertung der Funktion durchgeführt werden soll. Sie muss den Funktionswert zurückgeben.
- Das Argument *bounds* ist eine Liste von Paaren aus Minimal- und Maximalwerten für jede Koordinate. Diese Liste bestimmt implizit die Dimension des Koordinatenraums, in dem das Optimum gesucht werden soll; beispielsweise können Sie für eine Funktion wie *x**2 + (y-1)**4* die Grenzen als *{{-1 1} {-1 1}}* festlegen, d. h. zwei Paare für die beiden Koordinaten.
- Der Rest (*args*) besteht aus null oder mehr Schlüssel-Wert-Paaren, die die Optionen festlegen. Welche Optionen von welchem Algorithmus unterstützt werden, ist im Folgenden dokumentiert.

Das Ergebnis der verschiedenen Optimierungsprozeduren ist ein Dictionary, das mindestens die folgenden Elemente enthält:

- *optimum-coordinates* ist eine Liste, die die Koordinaten des gefundenen Optimums enthält.
- *optimum-value* ist der Wert der Funktion an diesen Koordinaten.
- *evaluations* bezeichnet die Anzahl der Funktionsauswertungen.
- *best-values* ist eine Liste aufeinanderfolgender bester Werte, die im Rahmen der Iterationen ermittelt wurden.

Einzelheiten zu den Algorithmen

Das Paket enthält folgende Algorithmen:

::math::probopt::pso *function bounds args*

Der Algorithmus der "Partikelschwarm-Optimierung" basiert auf dem Prinzip, dass die potenziellen Optimumpunkte um den bisher besten Punkt herumschwärmen sollten, wobei Abweichungen zulässig sind, um Verbesserungen zu ermöglichen.

Er unterstützt die folgenden Optionen:
- *-swarmsize number*: Anzahl der zu berücksichtigenden Partikel (Standard: 50)
- *-vweight value*: Gewichtungsfaktor für die aktuelle "Geschwindigkeit" (0–1, Standardwert: 0,5)
- *-pweight value*: Gewicht für die beste Position des einzelnen Partikels (0–1, Standardwert: 0,3)
- *-gweight value*: Gewicht für die "beste" Gesamtposition pro Partikel (0–1, Standardwert: 0,3)
- *-type local/global*: Art der Optimierung
- *-neighbours number*: Größe der Nachbarschaft (Standardwert: 5, wird bei "local" verwendet)
- *-iterations number*: Maximale Anzahl von Iterationen
- *-tolerance value*: Absolute minimale Verbesserung des Minimalwerts

::math::probopt::sce *function bounds args*

Der Algorithmus "Shuffled Complex Evolution" ist eine Erweiterung des Nelder-Mead-Algorithmus, der mehrere Komplexe verwendet und diese neu anordnet, um das "globale" Optimum zu finden.

Er unterstützt die folgenden Optionen:
- *-complexes number*: Anzahl der zu berücksichtigenden Partikel (Standard: 2)
- *-mincomplexes number*: Mindestanzahl an Komplexen (Standard: 2; wird derzeit nicht verwendet)
- *-newpoints number*: Anzahl der neu zu generierenden Punkte (Standard: 1)
- *-shuffle number*: Anzahl der Iterationen, nach denen die Komplexe neu gemischt werden sollen (bei der Standardeinstellung 0 wird ein Wert anhand der Anzahl der Dimensionen berechnet)
- *-pointspercomplex number*: Anzahl der Punkte pro Komplex (wenn auf 0 gesetzt – dies ist die Standardeinstellung –, wird die Zahl anhand der Anzahl der Dimensionen berechnet)
- *-pointspersubcomplex number*: Anzahl der Punkte pro Teilkomplex (dient zur Auswahl der besten Punkte in jedem Komplex; bei der Standardeinstellung 0 wird ein Wert anhand der Anzahl der Dimensionen berechnet)
- *-iterations number*: Maximale Anzahl von Iterationen (Standard: 100)
- *-maxevaluations number*: Maximale Anzahl der Funktionsauswertungen (wenn diese Zahl erreicht ist, wird die Iteration abgebrochen. Standardwert: 1.000 Millionen)
- *-abstolerance value*: Absolute minimale Verbesserung des Minimalwerts (Standardwert: 0,0)
- *-reltolerance value*: Relative minimale Verbesserung des Minimalwerts (Standardwert: 0,001)

::math::probopt::diffev *function bounds args*

Der Algorithmus der "Differentialevolution" verwendet eine Reihe von Startpunkten, die anschließend anhand zufällig ausgewählter Punkte aktualisiert werden. Er ähnelt mehr oder weniger genetischen Algorithmen. Er wird durch zwei Parameter gesteuert, nämlich factor und lambda, wobei der erste die Aktualisierung anhand zufälliger Punkte und der zweite die Aktualisierung anhand des bisher besten gefundenen Punktes bestimmt.

Er unterstützt die folgenden Optionen:
- *-iterations number*: Maximale Anzahl von Iterationen (Standard: 100)
- *-number number*: Anzahl der zu verarbeitenden Punkte (bei der Standardeinstellung 0 wird diese aus der Anzahl der Dimensionen berechnet)
- *-factor value*: Gewichtung zufällig ausgewählter Punkte bei der Aktualisierung (0–1,

Standardwert: 0,6)
- *-lambda value*: Gewicht des bisher besten gefundenen Punktes bei der Aktualisierung (0–1, Standardwert: 0,0)
- *-crossover value*: Anteil der neuen Punkte, die beim Ersetzen der alten Punkte berücksichtigt werden sollen (0–1, Standardwert: 0,5)
- *-maxevaluations number*: Maximale Anzahl der Funktionsauswertungen (wenn diese Zahl erreicht ist, wird die Iteration abgebrochen. Standardwert: 1.000 Millionen)
- *-abstolerance value*: Absolute minimale Verbesserung des Minimalwerts (Standardwert: 0,0)
- *-reltolerance value*: Relative minimale Verbesserung des Minimalwerts (Standardwert: 0,001)

::math::probopt::lipoMax *function bounds args*

Der Algorithmus "Lipschitz-Optimierung" nutzt die "Lipschitz"-Eigenschaft der gegebenen Funktion, um ein *Maximum* innerhalb des vorgegebenen Begrenzungsrahmens zu finden. Es gibt zwei Varianten: *lipoMax* geht von einer festen Schätzung für den Lipschitz-Parameter aus.

Er unterstützt die folgenden Optionen:
- *-iterations number*: Anzahl der Iterationen (entspricht der tatsächlichen Anzahl der Funktionsauswertungen, Standardwert: 100)
- *-lipschitz value*: Schätzung des Lipschitz-Parameters (Standardwert: 10,0)

::math::probopt::adaLipoMax *function bounds args*

Der Algorithmus der "adaptiven Lipschitz-Optimierung" nutzt die "Lipschitz"-Eigenschaft der gegebenen Funktion, um ein *Maximum* innerhalb des vorgegebenen Begrenzungsrahmens zu finden. Die adaptive Variante nutzt tatsächlich zwei Phasen, um eine geeignete Schätzung für den Lipschitz-Parameter zu ermitteln. Dies wird durch den "Bernoulli"-Parameter gesteuert.

Wenn Sie eine große Anzahl von Iterationen festlegen, kann die Ausführung des Algorithmus sehr lange dauern, da er versucht, den Lipschitz-Parameter zu verfeinern, und die Wahrscheinlichkeit, eine bessere Schätzung zu finden, schnell abnimmt.

Er unterstützt die folgenden Optionen:
- *-iterations number*: Anzahl der Iterationen (entspricht der tatsächlichen Anzahl der Funktionsauswertungen, Standardwert: 100)
- *-bernoulli value*: Parameter für zufällige Entscheidungen (Exploration versus Exploitation, Standardwert: 0,1)

Literatur

Die verschiedenen Algorithmen wurden in Online-Veröffentlichungen beschrieben. Hier sind einige davon:

- *PSO*: Maurice Clerc, Standard Particle Swarm Optimisation (2012) https://hal.archives-ouvertes.fr/file/index/docid/764996/filename/SPSO_descriptions.pdf

  Alternativ: https://en.wikipedia.org/wiki/Particle_swarm_optimization
- *SCE*: Qingyuan Duan, Soroosh Sorooshian, Vijai K. Gupta, Optimal use of the SCE-UA global optimization method for calibrating watershed models (1994), Journal of Hydrology 158, S. 265–284

  https://www.researchgate.net/publication/223408756_Optimal_Use_of_the_SCE-UA_Global_Optimization_Me
- *DE*: Rainer Storn und Kenneth Price, Differential Evolution – A simple and efficient adaptive

scheme for global optimization over continuous spaces (1996)

http://www1.icsi.berkeley.edu/~storn/TR-95-012.pdf

- *LIPO*: Cedric Malherbe und Nicolas Vayatis, Global optimization of Lipschitz functions, (Juni 2017)

https://arxiv.org/pdf/1703.02628.pdf

Kategorie

Mathematik

qcomplex

Stichwörter: qcomplex, complex numbers, math

Name

math::complexnumbers – Einfaches Paket für komplexe Zahlen

Übersicht

package require Tcl 8.5 9
package require math::complexnumbers ?**1.0.3**?
::math::complexnumbers::+ *z1 z2*
::math::complexnumbers::- *z1 z2*
::math::complexnumbers::* *z1 z2*
::math::complexnumbers::/ *z1 z2*
::math::complexnumbers::conj *z1*
::math::complexnumbers::real *z1*
::math::complexnumbers::imag *z1*
::math::complexnumbers::mod *z1*
::math::complexnumbers::arg *z1*
::math::complexnumbers::complex *real imag*
::math::complexnumbers::tostring *z1*
::math::complexnumbers::exp *z1*
::math::complexnumbers::sin *z1*
::math::complexnumbers::cos *z1*
::math::complexnumbers::tan *z1*
::math::complexnumbers::log *z1*
::math::complexnumbers::sqrt *z1*
::math::complexnumbers::pow *z1 z2*

Beschreibung

Das mathematische Modul *complexnumbers* bietet eine unkomplizierte Implementierung komplexer Zahlen in reinem Tcl. Dahinter steht die Philosophie, dass der Benutzer sich bewusst ist, auf abstrakte Weise mit komplexen Zahlen zu arbeiten, und dabei eine möglichst hohe Leistung anstrebt, wie sie innerhalb der Grenzen eines Interpreters erreichbar ist.

Daher gehen die in diesem Paket definierten Prozeduren davon aus, dass die Argumente gültige (Darstellungen von) "komplexen Zahlen" sind, d. h. Listen aus zwei Zahlen, die den Real- und den Imaginärteil einer komplexen Zahl definieren (obwohl dies nur ein Detail ist: Verwenden Sie den Befehl *complex*, um eine gültige Zahl zu bilden.)

Die meisten Prozeduren implementieren die grundlegenden arithmetischen Operationen oder elementare Funktionen, während einige andere die Umwandlung zwischen verschiedenen Darstellungsformen vornehmen:

```
    set z [complex 0 1]
    puts "z = [tostring $z]"
    puts "z**2 = [* $z $z]
```

würde zu folgendem Ergebnis führen:

```
    z = i
    z**2 = -1
```

Verfügbare Prozeduren

Das Paket implementiert alle oder die meisten grundlegenden Operationen und elementaren Funktionen.

The arithmetic operations are:

::math::complexnumbers::+ *z1 z2*
Addiert die beiden Argumente und gibt die resultierende komplexe Zahl zurück
complex *z1* (in)
Erstes Argument in der Summierung
complex *z2* (in)
Zweites Argument in der Summierung

::math::complexnumbers::- *z1 z2*
Subtrahiert das zweite Argument vom ersten und gibt die resultierende komplexe Zahl zurück. Wenn nur ein Argument angegeben wird, wird das Negative von z1 zurückgegeben (d. h. -z1).
complex *z1* (in)
Erstes Argument bei der Subtraktion
complex *z2* (in)
Zweites Argument bei der Subtraktion (optional)

::math::complexnumbers::* *z1 z2*
Multipliziert die beiden Argumente und gibt die resultierende komplexe Zahl zurück
complex *z1* (in)
Erstes Argument bei der Multiplikation
complex *z2* (in)
Zweites Argument bei der Multiplikation

::math::complexnumbers::/ *z1 z2*
Teilt das erste Argument durch das zweite und gibt die resultierende komplexe Zahl zurück
complex *z1* (in)
Erstes Argument (Zähler) bei der Division
complex *z2* (in)
Zweites Argument (Nenner) bei der Division

::math::complexnumbers::conj *z1*
Gibt die konjugiert komplexe Zahl der angegebenen komplexen Zahl zurück
complex *z1* (in)
Die betreffende komplexe Zahl

Conversion/inquiry procedures:

::math::complexnumbers::real *z1*
Gibt den Realteil der angegebenen komplexen Zahl zurück
complex *z1* (in)
Die betreffende komplexe Zahl

::math::complexnumbers::imag *z1*
Gibt den Imaginärteil der angegebenen komplexen Zahl zurück
complex *z1* (in)
Die betreffende komplexe Zahl

::math::complexnumbers::mod *z1*
Gibt den Betrag (Modulus) der angegebenen komplexen Zahl zurück
complex *z1* (in)
Die betreffende komplexe Zahl

::math::complexnumbers::arg *z1*
Gibt das Argument ("Winkel" im Bogenmaß) der angegebenen komplexen Zahl zurück
complex *z1* (in)
Die betreffende komplexe Zahl

::math::complexnumbers::complex *real imag*
Bildet die komplexe Zahl "real + imag*i" und gibt sie zurück
float *real* (in)
Der Realteil der neuen komplexen Zahl
float *imag* (in)
Der Imaginärteil der neuen komplexen Zahl

::math::complexnumbers::tostring *z1*
Wandelt die komplexe Zahl in die Form "real + imag*i" um und gibt die Zeichenkette zurück
float *complex* (in)
Die umzuwandelnde komplexe Zahl

Elementary functions:

::math::complexnumbers::exp *z1*
Berechnet die Exponentialfunktion für das angegebene komplexe Argument und gibt das Ergebnis zurück
complex *z1* (in)
Das komplexe Argument der Funktion

::math::complexnumbers::sin *z1*
Berechnet die Sinusfunktion für das angegebene komplexe Argument und gibt das Ergebnis

zurück

complex *z1* (in)

Das komplexe Argument der Funktion

::math::complexnumbers::cos *z1*

Berechnet die Kosinusfunktion für das angegebene komplexe Argument und gibt das Ergebnis zurück

complex *z1* (in)

Das komplexe Argument der Funktion

::math::complexnumbers::tan *z1*

Berechnet die Tangensfunktion für das angegebene komplexe Argument und gibt das Ergebnis zurück

complex *z1* (in)

Das komplexe Argument der Funktion

::math::complexnumbers::log *z1*

Berechnet den (Hauptwert des) Logarithmus für das angegebene komplexe Argument und gibt das Ergebnis zurück

complex *z1* (in)

Das komplexe Argument der Funktion

::math::complexnumbers::sqrt *z1*

Berechnet den (Hauptwert der) Quadratwurzel für das angegebene komplexe Argument und gibt das Ergebnis zurück

complex *z1* (in)

Das komplexe Argument der Funktion

::math::complexnumbers::pow *z1 z2*

Berechnet "z1 hoch z2" und gibt das Ergebnis zurück

complex *z1* (in)

Die komplexe Zahl, die potenziert werden soll

complex *z2* (in)

Der zu verwendende komplexe Exponent

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: complexnumbers* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright

Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>

quasirandom

Stichwörter: quasirandom, mathematics, quasi-random

Name

math::quasirandom – Quasizufällige Punkte für Integrations- und Monte-Carlo-Methoden

Übersicht

package require Tcl 8.6 9
package require TclOO
package require math::quasirandom 1.1
::math::quasirandom::qrpoint create *NAME DIM* ?*ARGS*?
gen next
gen set-start *index*
gen set-evaluations *number*
gen integral *func minmax args*

Beschreibung

In vielen Anwendungsbereichen spielen Pseudozufallszahlen und Pseudozufallspunkte in einem (begrenzten) Stichprobenraum eine wichtige Rolle. Dies gilt beispielsweise für jede Art von Monte-Carlo-Simulation. Pseudozufallszahlen können jedoch zu zufällig sein, sodass eine große Anzahl von Datenpunkten erforderlich ist, um den Fehler oder die Schwankungen in den Ergebnissen auf den gewünschten Wert zu reduzieren.

Als Alternative können Quasizufallszahlen verwendet werden: Anstelle "völlig" willkürlicher Punkte werden Punkte erzeugt, die so vielfältig sind, dass sie den gesamten Stichprobenraum mehr oder weniger gleichmäßig abdecken. Folglich kann die Konvergenz gegen den Grenzwert wesentlich schneller erfolgen, wenn solche Quasizufallszahlen gut gewählt sind.

Das Paket definiert eine *Klasse* "qrpoint", die einen Befehl zur Erzeugung quasizufälliger Punkte in einer, zwei oder mehreren Dimensionen bereitstellt. Der Befehl kann entweder einzelne Punkte erzeugen, sodass diese in einem benutzerdefinierten Algorithmus verwendet werden können, oder diese Punkte zur Berechnung von Integralen von Funktionen nutzen, die über 1, 2 oder mehr Dimensionen definiert sind. Er enthält zudem mehrere weitere gängige Algorithmen. (Hinweis: Diese sind noch nicht implementiert.)

Ein besonderes Merkmal der Generatoren besteht darin, dass keine Einstellparameter erforderlich sind, was ihre Verwendung besonders einfach macht.

Befehle

Mithilfe der Klasse *qrpoint* wird ein Generator für quasizufällige Punkte erstellt:

::math::quasirandom::qrpoint create *NAME DIM* ?ARGS?

Dieser Befehl akzeptiert die folgenden Argumente:

string *NAME*

Der Name des zu erstellenden Befehls (alternativ erzeugt der Unterbefehl *new* einen eindeutigen Namen)

integer/string *DIM*
Die Anzahl der Dimensionen oder einer der Werte "circle", "disk", "sphere" oder "ball"

strings *ARGS*
Null oder mehr Schlüssel-Wert-Paare. Die unterstützten Optionen sind:
- *-start index*: Der Index für den nächsten zu erzeugenden Punkt (Standard: 1)
- *-evaluations number*: Die Anzahl der standardmäßig zu verwendenden Auswertungen (Standard: 100)

Die zurückgegebenen Punkte liegen im Hyperblock [0,1[^n (wobei n die Anzahl der Dimensionen ist) oder auf dem Einheitskreis, innerhalb der Einheitskreisscheibe, auf der Einheitssphäre (Kugeloberfläche) oder innerhalb der Einheitskugel.

Jeder Generator unterstützt die folgenden Unterbefehle:

gen next
Gibt die Koordinaten des nächsten quasizufälligen Punktes zurück

gen set-start *index*
Setzt den Index für den nächsten quasizufälligen Punkt neu. Dies ist nützlich, um zu steuern, welche Liste von Punkten zurückgegeben wird. Gibt den neuen oder den aktuellen Wert zurück, wenn kein Wert angegeben wurde.

gen set-evaluations *number*
Setzt die Standardanzahl der Auswertungen in zusammengesetzten Algorithmen neu. Hinweis: Die tatsächliche Anzahl ist das kleinste Vielfache von 4, das größer oder gleich der angegebenen Zahl ist. (Das Vielfache von 4 spielt bei der detaillierten Integrationsroutine eine Rolle.)

gen integral *func minmax args*
Berechnet das Integral der angegebenen Funktion über den Block (oder den Kreis, die Kugel usw.).

string *func*
Der Name der zu integrierenden Funktion

list *minmax*
Liste von Paaren aus Minimal- und Maximal-Koordinaten. Diese kann verwendet werden, um die quasizufälligen Koordinaten auf den gewünschten Hyperblock abzubilden.

Wenn es sich bei dem Raum um einen Kreis, eine Kreisscheibe usw. handelt, sollte dieses Argument ein einzelner Wert sein, nämlich der Radius. Der Kreis, die Kreisscheibe usw. ist auf den Ursprung zentriert. Wenn dies nicht den Anforderungen entspricht, sollte innerhalb der Funktion eine Transformation der Koordinaten vorgenommen werden.

strings *args*
Null oder mehr Schlüssel-Wert-Paare. Die folgenden Optionen werden unterstützt:
- *-evaluations number*: Die Anzahl der zu verwendenden Auswertungen. Wenn nicht anders angegeben, wird der Standardwert des Generator-Objekts verwendet.

Noch zu tun

Weitere Algorithmen und Varianten implementieren

Weitere Komponententests implementieren.

Vergleich mit Pseudozufallszahlen für die Integration.

Literatur
Es gibt verschiedene Algorithmen zur Erzeugung von Quasizufallszahlen. Die in diesem Paket erzeugten Generatoren basieren auf:
http://extremelearning.com.au/unreasonable-effectiveness-of-quasirandom-sequences/

Kategorie
Mathematik

rational_funcs
Stichwörter: rational_funcs, math, rational functions

Name
math::rationalfunctions – Gebrochenrationale Funktionen

Übersicht
package require Tcl ?**8.5 9**?
package require math::rationalfunctions ?**1.0.2**?
::math::rationalfunctions::rationalFunction *num den*
::math::rationalfunctions::ratioCmd *num den*
::math::rationalfunctions::evalRatio *rational x*
::math::rationalfunctions::addRatio *ratio1 ratio2*
::math::rationalfunctions::subRatio *ratio1 ratio2*
::math::rationalfunctions::multRatio *ratio1 ratio2*
::math::rationalfunctions::divRatio *ratio1 ratio2*
::math::rationalfunctions::derivPolyn *ratio*
::math::rationalfunctions::coeffsNumerator *ratio*
::math::rationalfunctions::coeffsDenominator *ratio*

Beschreibung
Dieses Paket befasst sich mit gebrochenrationalen Funktionen einer Variablen:

- die grundlegenden arithmetischen Operationen werden auf gebrochenrationale Funktionen erweitert
- Berechnung der Ableitungen dieser Funktionen
- Auswertung über eine allgemeine Prozedur oder über spezielle Prozeduren)

Prozeduren
Das Paket definiert die folgenden öffentlichen Prozeduren:

::math::rationalfunctions::rationalFunction *num den*
Gibt eine (kodierte) Liste zurück, die die gebrochenrationale Funktion definiert. Eine gebrochenrationale Funktion

```
            1 + x^3
f(x) = ------------
       1 + 2x + x^2
```

kann wie folgt definiert werden:

```
set f [::math::rationalfunctions::rationalFunction [list 1 0 0 1]
  [list 1 2 1]]
```

list *num*
Koeffizienten des Zählers der gebrochenrationalen Funktion (in aufsteigender Reihenfolge)

list *den*
Koeffizienten des Nenners der gebrochenrationalen Funktion (in aufsteigender Reihenfolge)

::math::rationalfunctions::ratioCmd *num den*
Erzeugt eine neue Prozedur, die die gebrochenrationale Funktion auswertet. Der Name der Funktion wird automatisch generiert. Dies ist nützlich, wenn Sie die Funktion mehrfach auswerten müssen, da die Prozedur aus einem einzigen [expr]-Befehl besteht.

list *num*
Koeffizienten des Zählers der gebrochenrationalen Funktion (in aufsteigender Reihenfolge)

list *den*
Koeffizienten des Nenners der gebrochenrationalen Funktion (in aufsteigender Reihenfolge)

::math::rationalfunctions::evalRatio *rational x*
Wertet die gebrochenrationale Funktion an der Stelle x aus.

list *rational*
Die Definition der gebrochenrationalen Funktion (wie sie vom Befehl rationalFunction zurückgegeben wird).

float *x*
Die Koordinate, an der die Funktion ausgewertet werden soll

::math::rationalfunctions::addRatio *ratio1 ratio2*
Gibt eine neue gebrochenrationale Funktion zurück, die die Summe der beiden anderen Funktionen darstellt.

list *ratio1*
Erster Operand (gebrochenrationale Funktion)

list *ratio2*
Zweiter Operand (gebrochenrationale Funktion)

::math::rationalfunctions::subRatio *ratio1 ratio2*
Gibt eine neue gebrochenrationale Funktion zurück, die der Differenz der beiden anderen entspricht.

list *ratio1*
Erster Operand (gebrochenrationale Funktion)

list *ratio2*
Zweiter Operand (gebrochenrationale Funktion)

::math::rationalfunctions::multRatio *ratio1 ratio2*
Gibt eine neue gebrochenrationale Funktion zurück, die das Produkt der beiden anderen Funktionen ist. Wenn eines der Argumente ein Skalarwert ist, wird die andere gebrochenrationale Funktion lediglich skaliert.

list *ratio1*
Erster Operand (gebrochenrationale Funktion oder Skalar)
list *ratio2*
Zweiter Operand (gebrochenrationale Funktion oder Skalar)

::math::rationalfunctions::divRatio *ratio1 ratio2*
Teilt die erste gebrochenrationale Funktion durch die zweite gebrochenrationale Funktion und gibt das Ergebnis zurück. Der Rest wird verworfen.
list *ratio1*
Erster Operand (gebrochenrationale Funktion)
list *ratio2*
Zweiter Operand (gebrochenrationale Funktion)

::math::rationalfunctions::derivPolyn *ratio*
Differenziert die gebrochenrationale Funktion und gibt das Ergebnis zurück.
list *ratio*
Die zu differenzierende gebrochenrationale Funktion

::math::rationalfunctions::coeffsNumerator *ratio*
Gibt die Koeffizienten des Zählers der gebrochenrationalen Funktion zurück.
list *ratio*
Die zu untersuchende gebrochenrationale Funktion

::math::rationalfunctions::coeffsDenominator *ratio*
Gibt die Koeffizienten des Nenners der gebrochenrationalen Funktion zurück.
list *ratio*
Die zu untersuchende gebrochenrationale Funktion

Anmerkungen zur Umsetzung

Die Implementierung der gebrochenrationalen Funktionen basiert auf dem Paket math::polynomials. Weitere Hinweise finden Sie in der Dokumentation zu diesem Paket.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: rationalfunctions* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright

```
Copyright (c) 2005 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

roman

Stichwörter: roman, conversion, integer, roman numeral

Name

math::roman – Werkzeuge zum Erstellen und Bearbeiten römischer Zahlen

Übersicht

package require Tcl 8.5 9
package require math::roman ?**1.1**?
::math::roman::toroman *i*
::math::roman::tointeger *r*
::math::roman::sort *list*
::math::roman::expr *args*

Beschreibung

::math::roman ist eine reine Tcl-Bibliothek zur Konvertierung zwischen Ganzzahlen und römischen Zahlen. Sie bietet zudem Hilfsfunktionen zum Sortieren römischer Zahlen und zum Rechnen mit ihnen.

Dieser Code stammt ursprünglich aus dem Tcler-Wiki unter http://wiki.tcl.tk/1823 und steht daher für jede Nutzung und jeden Zweck frei zur Verfügung. Ein herzliches Dankeschön an die findigen Köpfe, die diese cleveren Routinen entwickelt und sie großzügig der Tcl-Community zur Verfügung gestellt haben.

Obwohl diese Bibliothek unter Tcl 8.3 geschrieben und durch Tests geprüft wurde, gehe ich davon aus, dass sie unter allen 8.x-Versionen von Tcl funktionieren wird.

Befehle

::math::roman::toroman *i*
: Wandelt eine Ganzzahl in eine römische Zahl um. Das Ergebnis wird stets in Großbuchstaben ausgegeben. Der Wert Null wird in eine leere Zeichenkette umgewandelt.

::math::roman::tointeger *r*
: Wandelt eine römische Zahl in eine Ganzzahl um.

::math::roman::sort *list*
: Sortiert eine Liste römischer Zahlen von der kleinsten zur größten.

::math::roman::expr *args*
: Wertet einen Ausdruck aus, dessen Operanden ausschließlich römische Zahlen sind.

Von diesen Befehlen werden sowohl *toroman* als auch *tointeger* zur einfacheren Verwendung exportiert. Die beiden anderen werden nicht exportiert, da sie zu Konflikten mit bestehenden Tcl-Befehlen führen oder mit diesen verwechselt werden könnten.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: roman* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

Copyright

```
Copyright (c) 2005 Kenneth Green <kenneth.green@gmail.com>
```


romberg
Stichwörter: romberg

Name
math::calculus::romberg – Romberg-Integral

Übersicht
package require Tcl 8.5 9
package require math::calculus 0.6
::math::calculus::romberg *f a b* ?*-option value...*?
::math::calculus::romberg_infinity *f a b* ?*-option value...*?
::math::calculus::romberg_sqrtSingLower *f a b* ?*-option value...*?
::math::calculus::romberg_sqrtSingUpper *f a b* ?*-option value...*?
::math::calculus::romberg_powerLawLower *gamma f a b* ?*-option value...*?
::math::calculus::romberg_powerLawUpper *gamma f a b* ?*-option value...*?
::math::calculus::romberg_expLower *f a b* ?*-option value...*?
::math::calculus::romberg_expUpper *f a b* ?*-option value...*?

Beschreibung
Die Prozeduren **romberg** im Paket **math::calculus** führen die numerische Integration einer Funktion einer Variablen durch. Sie sind so konzipiert, dass sie "produktionsreif" sind, d. h., sie sind robust, präzise und hinsichtlich der Anzahl der Auswertungen angemessen effizient.

Prozeduren
Für die Romberg-Integration stehen folgende Prozeduren zur Verfügung:

::math::calculus::romberg *f a b ?-option value...?*
Integriert eine analytische Funktion über ein gegebenes Intervall.

::math::calculus::romberg_infinity *f a b ?-option value...?*
Integriert eine analytische Funktion über ein halbunendliches Intervall.

::math::calculus::romberg_sqrtSingLower *f a b ?-option value...?*
Integriert eine Funktion, von der erwartet wird, dass sie über ein Intervall hinweg analytisch ist, abgesehen vom Vorhandensein einer Singularität in Form einer inversen Quadratwurzel an der unteren Grenze.

::math::calculus::romberg_sqrtSingUpper *f a b ?-option value...?*
Integriert eine Funktion, von der erwartet wird, dass sie über ein Intervall hinweg analytisch ist, abgesehen vom Vorhandensein einer Singularität in Form einer inversen Quadratwurzel an der oberen Grenze.

::math::calculus::romberg_powerLawLower *gamma f a b* ?*-option value*...?
Integriert eine Funktion, von der erwartet wird, dass sie über ein Intervall hinweg analytisch ist, abgesehen vom Vorhandensein einer Potenzgesetz-Singularität an der unteren Grenze.

::math::calculus::romberg_powerLawUpper *gamma f a b* ?*-option value*...?
Integriert eine Funktion, von der erwartet wird, dass sie über ein Intervall hinweg analytisch ist, abgesehen vom Vorhandensein einer Potenzgesetz-Singularität an der oberen Grenze.

::math::calculus::romberg_expLower *f a b* ?*-option value*...?
Integriert eine exponentiell wachsende Funktion; die untere Grenze des Integrationsbereichs kann beliebig groß und negativ sein.

::math::calculus::romberg_expUpper *f a b* ?*-option value*...?
Integriert eine exponentiell abklingende Funktion; die Obergrenze des Integrationsbereichs kann beliebig groß sein.

Parameter

f
Zu integrierende Funktion. Diese muss als einzelner Tcl-Befehl ausgedrückt werden, an den ein einziges Argument angehängt wird, nämlich die Abszisse, bei der die Auswertung der Funktion erfolgen soll. Das erste Wort des Befehls wird vor jeder Auswertung mit **namespace which** im Geltungsbereich des Aufrufers verarbeitet. Aufgrund dieser Verarbeitung kann der Befehl lokal im aufrufenden Namespace definiert sein und muss nicht global sein.

a
Untergrenze des Integrationsbereichs.

b
Obergrenze des Integrationsbereichs. Für die Prozeduren **romberg_sqrtSingLower**, **romberg_sqrtSingUpper**, **romberg_powerLawLower**, **romberg_powerLawUpper**, **romberg_expLower** und **romberg_expUpper** muss die Untergrenze echt kleiner sein als die Obergrenze. Bei den übrigen Prozeduren können die Grenzen in beliebiger Reihenfolge angegeben werden.

gamma
Potenz, die für eine Singularität nach einem Potenzgesetz verwendet wird; Einzelheiten finden Sie im Abschnitt Uneigentliche Integrale.

Optionen

-abserror *epsilon*
Legt fest, dass der Integrationsvorgang höchstens so lange fortgesetzt wird, bis der geschätzte absolute Fehler des Integrals kleiner als *epsilon* ist. Der Fehler kann erheblich über- oder unterschätzt werden, wenn die Funktion (oder eine ihrer Ableitungen) Singularitäten enthält; Einzelheiten finden Sie im Abschnitt Uneigentliche Integrale. Der Standardwert beträgt 1.0e-08.

-relerror *epsilon*
Legt fest, dass der Integrationsvorgang höchstens so lange fortgesetzt wird, bis der geschätzte relative Fehler des Integrals kleiner als *epsilon* ist. Der Fehler kann erheblich über- oder unterschätzt werden, wenn die Funktion (oder eine ihrer Ableitungen) Singularitäten enthält; Einzelheiten finden Sie im Abschnitt Uneigentliche Integrale. Der Standardwert beträgt 1.0e-06.

-maxiter *m*
Legt fest, dass die Integration nach höchstens *n* Verdreifachungen der Anzahl der durchgeführten Auswertungen beendet wird. Mit anderen Worten: Bei einem Wert von *n* für **-maxiter** führt der Integrationsmechanismus höchstens 3**n* Auswertungen der Funktion durch. Der Standardwert beträgt 14, was einer Obergrenze von etwa 4,8 Millionen Auswertungen entspricht. (Gutartige Funktionen erfordern selten mehr als einige hundert Auswertungen.)

-degree *d*
Legt fest, dass bei der Romberg-Integration ein extrapolierendes Polynom vom Grad *d* verwendet werden soll; Einzelheiten finden Sie im Abschnitt Beschreibung. Der Standardwert ist 4. Der Wert darf höchstens *m*-1 betragen.

Beschreibung

Die Prozedur **romberg** führt eine Romberg-Integration unter Verwendung der modifizierten Mittelpunktsregel durch. Die Romberg-Integration ist ein iterativer Prozess. Im ersten Schritt wird die Funktion am Mittelpunkt des Integrationsbereichs ausgewertet, und der Wert wird mit der Breite des Intervalls multipliziert; das ergibt die gröbstmögliche Schätzung. Im zweiten Schritt wird das Intervall in drei Teile unterteilt, und die Funktion wird am Mittelpunkt jedes Teils ausgewertet; die Summe der Werte wird mit drei multipliziert. Im dritten Schritt werden neun Teile verwendet, im vierten siebenundzwanzig und so weiter, wobei sich die Anzahl der Unterteilungen bei jedem Schritt verdreifacht.

Sobald das Intervall mindestens *d* Mal unterteilt wurde, wird ein Polynom an die in den letzten *d* +1 Unterteilungen geschätzten Integrale angepasst. Die Integrale werden als Funktion des Quadrats der Breite der Teilintervalle betrachtet (jedes gute Lehrbuch der numerischen Mathematik behandelt diesen Prozess unter dem Begriff "Romberg-Integration"). Das Polynom wird auf eine Schrittweite von Null extrapoliert, wodurch ein Wert für das Integral sowie eine Schätzung des Fehlers berechnet werden.

Dieser Prozess verläuft nur dann einwandfrei, wenn die Funktion im Integrationsbereich analytisch ist; an beiden Enden des Bereichs dürfen hebbare Singularitäten auftreten, sofern der Grenzwert der Funktion (und aller ihrer Ableitungen) bei Annäherung an die Endpunkte existiert. Somit kann **romberg** verwendet werden, um eine Funktion wie f(x)=sin(x)/x über ein Intervall zu integrieren, das bei Null beginnt oder endet.

Beachten Sie, dass **romberg** entweder nicht konvergiert oder falsche Fehlerschätzungen zurückgibt, wenn die Funktion oder eine ihrer Ableitungen irgendwo im Integrationsbereich eine Singularität hat (mit Ausnahme des oben genannten Falls). Daher ist bei der Integration einer Funktion wie 1/(1-x**2) Vorsicht geboten, um die Stellen zu vermeiden, an denen die Ableitung singulär ist.

Uneigentliche Integrale

Die Romberg-Integration eignet sich auch zur Integration von Funktionen über halbunendliche Intervalle oder von Funktionen, die Singularitäten aufweisen. Der Trick besteht darin, einen Variablenwechsel vorzunehmen, um die Singularität zu beseitigen, und die Singularität an das eine oder andere Ende des Integrationsbereichs zu verlagern. Das Paket **math::calculus** stellt eine Reihe von **romberg**-Prozeduren zur Verfügung, um die häufigsten Fälle zu behandeln.

romberg_infinity

Integriert eine Funktion über ein halbunendliches Intervall; entweder *a* oder *b* kann unendlich sein. *a* und *b* müssen das gleiche Vorzeichen haben; wenn Sie über die Achse integrieren müssen, beispielsweise von einem negativen Wert bis zur positiven Unendlichkeit, verwenden Sie **romberg**, um vom negativen Wert bis zu einem kleinen positiven Wert zu integrieren, und anschließend **romberg_infinity**, um vom positiven Wert bis zur positiven Unendlichkeit zu integrieren. Die Prozedur **romberg_infinity** arbeitet mit dem Variablenwechsel u=1/x, sodass das Integral von a bis b über f(x) als das Integral von 1/a bis 1/b über f(1/u)/u**2 ausgewertet wird.

romberg_powerLawLower and romberg_powerLawUpper

Integriert eine Funktion, die entweder an der unteren oder an der oberen Grenze des Integrationsbereichs eine integrierbare Singularität in Form eines Potenzgesetzes aufweist (oder dort eine Ableitung mit einer Singularität in Form eines Potenzgesetzes besitzt). Diese Prozeduren erfordern einen ersten Parameter, *gamma*, der das Potenzgesetz angibt. Es wird angenommen, dass die Funktion oder ihre erste Ableitung wie (x-*a*)**(-*gamma*) oder (*b*-x)**(-*gamma*) divergiert. *gamma* muss größer als Null und kleiner als 1 sein.

Diese Prozeduren sind nicht nur bei der Integration von Funktionen nützlich, die an einem Ende des Integrationsbereichs gegen unendlich streben, sondern auch bei Funktionen, deren Ableitungen am Ende des Bereichs nicht existieren. Beispielsweise führt die Integration von f(x)=pow(x,0.25) mit dem Ursprung als einem Ende des Integrationsbereichs dazu, dass die Prozedur **romberg** den Fehler im Integral erheblich unterschätzt. Das Problem lässt sich beheben, indem man berücksichtigt, dass die erste Ableitung von f(x), f'(x)=x**(-3/4)/4, am Ursprung gegen unendlich geht. Die Integration mit **romberg_powerLawLower** mit *gamma* gleich 0.75 führt zu einer wesentlich geordneteren Konvergenz.

Diese Prozeduren arbeiten mit dem Variablenwechsel u=(x-a)**(1-gamma) (**romberg_powerLawLower**) bzw. u=(b-x)**(1-gamma) (**romberg_powerLawUpper**).

Zur Zusammenfassung der Bedeutung von gamma:
- Wenn f(x) ~ x**(-a) (0 < a < 1) gilt, verwenden Sie gamma = a
- Wenn f'(x) ~ x**(-b) (0 < b < 1) gilt, verwenden Sie gamma = b

romberg_sqrtSingLower and romberg_sqrtSingUpper

Diese Prozeduren verhalten sich im häufigen Fall *gamma*=0.5 identisch zu **romberg_powerLawLower** und **romberg_powerLawUpper**; das heißt, sie integrieren eine Funktion mit einer Singularität in Form einer inversen Quadratwurzel an einem Ende des Intervalls. Ihre Implementierung ist einfacher, da sie Quadratwurzeln anstelle beliebiger Potenzen verwenden.

romberg_expLower and romberg_expUpper

Diese Prozeduren dienen der Integration einer Funktion, die über ein halbunendliches Intervall exponentiell ansteigt oder abfällt. **romberg_expLower** behandelt exponentiell ansteigende Funktionen und ermöglicht es, die untere Integrationsgrenze als beliebig große negative Zahl festzulegen. **romberg_expUpper** behandelt exponentiell abklingende Funktionen und erlaubt, dass die obere Integrationsgrenze eine beliebig große positive Zahl ist. Die Funktionen nehmen den Variablenwechsel u=exp(-x) bzw. u=exp(x) vor.

Sonstige Variablenänderungen

Wenn Sie ein anderes uneigentliches Integral als die hier aufgeführten benötigen, lässt sich ein

Variablenwechsel mit nur wenigen Zeilen Tcl-Code umsetzen. Da der dafür erforderliche Tcl-Code etwas kompliziert ist, stellen wir Ihnen hier ein ausgearbeitetes Beispiel zur Verfügung.

Angenommen, die Funktion, die wir integrieren möchten, ist f(x)=exp(x)/sqrt(1-x*x) (keine sehr natürliche Funktion, aber ein gutes Beispiel), und wir möchten sie über das Intervall (-1,1) integrieren. Der Nenner wird an beiden Enden des Intervalls null. Wir möchten die Variable von x auf u umstellen, sodass dx/sqrt(1-x**2) zu du wird. Mit x=sin(u) ergibt sich dx=cos(u)*du und sqrt(1-x**2)=cos(u). Das Integral von f(x) von a bis b ist das Integral von f(sin(u))*cos(u) von asin(a) bis asin(b).

Wir können eine Funktion **g** definieren, die eine beliebige Funktion **f** sowie den Parameter u entgegennimmt und diesen neuen Integranden berechnet.

```
proc g { f u } {
    set x [expr { sin($u) }]
    set cmd $f; lappend cmd $x; set y [eval $cmd]
    return [expr { $y / cos($u) }]
}
```

Nun entspricht die Integration von **f** von *a* bis *b* der Integration von **g** von *asin(a)* bis *asin(b)*. Es ist etwas knifflig, **f** konsistent im Geltungsbereich des Aufrufers auszuwerten; die folgende Prozedur ermöglicht dies.

```
proc romberg_sine { f a b args } {
    set f [lreplace $f 0 0 [uplevel 1 [list namespace which [lindex $f
    0]]]]
    set f [list g $f]
    return [eval [linsert $args 0 romberg $f [expr { asin($a) }] [expr
    { asin($b) }]]]
}
```

Diese **romberg_sine**-Prozedur bewältigt jede Funktion, bei der sqrt(1-x*x) im Nenner steht. Unsere Beispielfunktion lautet f(x)=exp(x)/sqrt(1-x*x):

```
proc f { x } {
    expr { exp($x) / sqrt( 1. - $x*$x ) }
}
```

Die Integration erfolgt durch die Anwendung von **romberg_sine**, genau wie bei jeder anderen **romberg**-Prozedur:

```
foreach { value error } [romberg_sine f -1.0 1.0] break
puts [format "integral is %.6g +/- %.6g" $value $error]

integral is 3.97746 +/- 2.3557e-010
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: calculus* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

Copyright

```
Copyright (c) 2004 Kevin B. Kenny <kennykb@acm.org>. All rights
    reserved. Redistribution permitted under the terms of the Open
    Publication License <http://www.opencontent.org/openpub/>
```


special
Stichwörter: special, Bessel functions, error function, math, special functions

Name
math::special – Spezielle mathematische Funktionen

Übersicht
package require Tcl ?**8.5 9**?
package require math::special ?**0.5.3**?
::math::special::eulerNumber *index*
::math::special::bernoulliNumber *index*
::math::special::Beta *x y*
::math::special::incBeta *a b x*
::math::special::regIncBeta *a b x*
::math::special::Gamma *x*
::math::special::digamma *x*
::math::special::erf *x*
::math::special::erfc *x*
::math::special::invnorm *p*
::math::special::J0 *x*
::math::special::J1 *x*
::math::special::Jn *n x*
::math::special::J1/2 *x*
::math::special::J-1/2 *x*
::math::special::I_n *x*
::math::special::cn *u k*
::math::special::dn *u k*
::math::special::sn *u k*
::math::special::elliptic_K *k*
::math::special::elliptic_E *k*
::math::special::exponential_Ei *x*
::math::special::exponential_En *n x*
::math::special::exponential_li *x*
::math::special::exponential_Ci *x*

::math::special::exponential_Si *x*
::math::special::exponential_Chi *x*
::math::special::exponential_Shi *x*
::math::special::fresnel_C *x*
::math::special::fresnel_S *x*
::math::special::sinc *x*
::math::special::legendre *n*
::math::special::chebyshev *n*
::math::special::laguerre *alpha n*
::math::special::hermite *n*

Beschreibung

Dieses Paket implementiert mehrere sogenannte spezielle Funktionen, wie beispielsweise die Gammafunktion, die Besselfunktionen und dergleichen.

Jede Funktion wird durch eine Prozedur implementiert, die denselben Namen trägt (naja, zumindest annähernd):

- J0 für die Besselfunktion nullter Ordnung erster Art
- J1 für die Besselfunktion erster Ordnung erster Art
- Jn für die Besselfunktion n-ter Ordnung erster Art
- J1/2 für die Besselfunktion erster Art halber Ordnung
- J-1/2 für die Besselfunktion erster Art der Ordnung minus ein halb
- I_n für die modifizierte Besselfunktion erster Art der Ordnung n
- Gamma für die Gammafunktion, erf und erfc für die Fehlerfunktion und die komplementäre Fehlerfunktion
- fresnel_C und fresnel_S für die Fresnel-Integrale
- elliptic_K und elliptic_E (vollständige elliptische Integrale)
- exponent_Ei und andere Funktionen im Zusammenhang mit den sogenannten Exponentialintegralen
- legendre, hermite: einige der klassischen orthogonalen Polynome.

Überblick

In der folgenden Tabelle sind einige Eigenschaften der Funktionen dieses Pakets zusammengefasst: der Definitionsbereich für das Argument, die Werte für die Parameter sowie die Fehlergrenzen.

```
Family       | Function    | Domain x    | Parameter   | Error bound
-------------+-------------+-------------+-------------+--------------
Bessel       | J0, J1,     | all of R    | n = integer |   < 1.0e-8
             | Jn          |             |             |  (|x|<20,
    n<20)
Bessel       | J1/2, J-1/2,|  x > 0      | n = integer |   exact
Bessel       | I_n         | all of R    | n = integer |   < 1.0e-6
             |             |             |             |
Elliptic     | cn          | 0 <= x <= 1 |     --      |   < 1.0e-10
functions    | dn          | 0 <= x <= 1 |     --      |   < 1.0e-10
             | sn          | 0 <= x <= 1 |     --      |   < 1.0e-10
```

Elliptic	K	0 <= x < 1	--	< 1.0e-6
integrals	E	0 <= x < 1	--	< 1.0e-6
Error	erf		--	
functions	erfc			
Inverse	invnorm	0 < x < 1	--	< 1.2e-9
normal				
distribution				
Exponential	Ei	x != 0	--	< 1.0e-10
(relative)				
integrals	En	x > 0	--	as Ei
	li	x > 0	--	as Ei
	Chi	x > 0	--	< 1.0e-8
	Shi	x > 0	--	< 1.0e-8
	Ci	x > 0	--	< 2.0e-4
	Si	x > 0	--	< 2.0e-4
Fresnel	C	all of R	--	< 2.0e-3
integrals	S	all of R	--	< 2.0e-3
general	Beta	(see Gamma)	--	< 1.0e-9
	Gamma	x != 0,-1,	--	< 1.0e-9
		-2, ...		
	incBeta		a, b > 0	< 1.0e-9
	regIncBeta		a, b > 0	< 1.0e-9
	digamma	x != 0,-1		< 1.0e-9
		-2, ...		
	sinc	all of R	--	exact
orthogonal	Legendre	all of R	n = 0,1,...	exact
polynomials	Chebyshev	all of R	n = 0,1,...	exact
	Laguerre	all of R	n = 0,1,...	exact
			alpha el. R	
	Hermite	all of R	n = 0,1,...	exact

Hinweis: Einige der Fehlergrenzen sind geschätzt, da mit der implementierten Approximationsmethode keine "formalen" Grenzen verfügbar waren; andere gelten für die Hilfsfunktionen, die zur Schätzung der Hauptfunktionen verwendet werden.

Die folgenden bekannten Funktionen fehlen derzeit im Paket:

- Besselfunktionen zweiter Art (Y_n, K_n)
- Besselfunktionen beliebiger Ordnung (und somit auch die Airy-Funktionen)
- Tschebyscheff-Polynome zweiter Art (U_n)
- Die unvollständige Gammafunktion

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren:

::math::special::eulerNumber *index*

Gibt die Euler-Zahl mit dem Index *index* zurück (Hinweis: Es handelt sich hierbei um ganzzahlige Werte). Da die Größe dieser Zahlen sehr schnell zunimmt, steht nur eine begrenzte Anzahl zur Verfügung.

int *index*

Index der zurückzugebenden Zahl (muss zwischen 0 und 54 liegen)

::math::special::bernoulliNumber *index*

Gibt die Bernoulli-Zahl mit dem Index *index* zurück. Da die Größe dieser Zahlen sehr schnell zunimmt, steht nur eine begrenzte Anzahl zur Verfügung.

int *index*

Index der zurückzugebenden Zahl (muss zwischen 0 und 52 liegen)

::math::special::Beta *x y*

Berechnet die Betafunktion für die Argumente "x" und "y"

float *x*

Erstes Argument für die Betafunktion

float *y*

Zweites Argument für die Betafunktion

::math::special::incBeta *a b x*

Berechnet die unvollständige Betafunktion für das Argument "x" mit den Parametern "a" und "b"

float *a*

Erster Parameter für die unvollständige Betafunktion, a > 0

float *b*

Zweiter Parameter für die unvollständige Betafunktion, b > 0

float *x*

Argument für die unvollständige Betafunktion

::math::special::regIncBeta *a b x*

Berechnet die regularisierte unvollständige Betafunktion für das Argument "x" mit den Parametern "a" und "b"

float *a*

Erster Parameter für die unvollständige Betafunktion, a > 0

float *b*

Zweiter Parameter für die unvollständige Betafunktion, b > 0

float *x*

Argument für die regularisierte unvollständige Betafunktion

::math::special::Gamma *x*

Berechnet die Gammafunktion für das Argument "x"

float *x*

Argument der Gammafunktion

::math::special::digamma *x*
Berechnet die Digammafunktion (psi) für das Argument "x"
float *x*
Argument der Digammafunktion

::math::special::erf *x*
Berechnet die Fehlerfunktion für das Argument "x"
float *x*
Argument für die Fehlerfunktion

::math::special::erfc *x*
Berechnet die komplementäre Fehlerfunktion für das Argument "x"
float *x*
Argument für die komplementäre Fehlerfunktion

::math::special::invnorm *p*
Berechnet die Inverse der Normalverteilungsfunktion für das Argument "p"
float *p*
Argument für die inverse Normalverteilungsfunktion (p muss größer als 0 und kleiner als 1 sein)

::math::special::J0 *x*
Berechnet die Besselfunktion nullter Ordnung erster Art für das Argument "x"
float *x*
Argument der Besselfunktion

::math::special::J1 *x*
Berechnet die Besselfunktion erster Ordnung der ersten Art für das Argument "x"
float *x*
Argument der Besselfunktion

::math::special::Jn *n x*
Berechnet die Besselfunktion erster Art n-ter Ordnung für das Argument "x"
integer *n*
Ordnung der Besselfunktion
float *x*
Argument der Besselfunktion

::math::special::J1/2 *x*
Berechnet die Besselfunktion erster Art halber Ordnung für das Argument "x"
float *x*
Argument der Besselfunktion

::math::special::J-1/2 *x*
Berechnet die Besselfunktion erster Art der Ordnung minus ein halb für das Argument "x"
float *x*
Argument der Besselfunktion

::math::special::I_n *x*
Berechnet die modifizierte Besselfunktion erster Art der Ordnung n für das Argument "x"
int *x*
Positive ganzzahlige Ordnung der Funktion
float *x*
Argument der Funktion

::math::special::cn *u k*
Berechnet die elliptische Funktion *cn* für das Argument "u" und den Parameter "k".
float *u*
Argument der Funktion
float *k*
Parameter

::math::special::dn *u k*
Berechnet die elliptische Funktion *dn* für das Argument "u" und den Parameter "k".
float *u*
Argument der Funktion
float *k*
Parameter

::math::special::sn *u k*
Berechnet die elliptische Funktion *sn* für das Argument "u" und den Parameter "k".
float *u*
Argument der Funktion
float *k*
Parameter

::math::special::elliptic_K *k*
Berechnet das vollständige elliptische Integral erster Art für das Argument "k"
float *k*
Argument der Funktion

::math::special::elliptic_E *k*
Berechnet das vollständige elliptische Integral zweiter Art für das Argument "k"
float *k*
Argument der Funktion

::math::special::exponential_Ei *x*
Berechnet das Exponentialintegral zweiter Art für das Argument "x"
float *x*
Argument für die Funktion (x!= 0)

::math::special::exponential_En *n x*
Berechnet das Exponentialintegral erster Art für das Argument "x" und die Ordnung n
int *n*
Ordnung des Integrals (n >= 0)

float *x*
Argument der Funktion (x >= 0)

::math::special::exponential_li *x*
Berechnet das logarithmische Integral für das Argument "x"
float *x*
Argument der Funktion (x > 0)

::math::special::exponential_Ci *x*
Berechnet das Kosinusintegral für das Argument "x"
float *x*
Argument der Funktion (x > 0)

::math::special::exponential_Si *x*
Berechnet das Sinusintegral für das Argument "x"
float *x*
Argument der Funktion (x > 0)

::math::special::exponential_Chi *x*
Berechnet das Integral des hyperbolischen Kosinus für das Argument "x"
float *x*
Argument der Funktion (x > 0)

::math::special::exponential_Shi *x*
Berechnet das Integral des hyperbolischen Sinus für das Argument "x"
float *x*
Argument der Funktion (x > 0)

::math::special::fresnel_C *x*
Berechnet das Fresnel-Kosinus-Integral für das reelle Argument x
float *x*
Argument der Funktion

::math::special::fresnel_S *x*
Berechnet das Fresnel-Sinus-Integral für das reelle Argument x
float *x*
Argument der Funktion

::math::special::sinc *x*
Berechnet die Sinc-Funktion für das reelle Argument x
float *x*
Argument der Funktion

::math::special::legendre *n*
Gibt das Legendre-Polynom vom Grad n zurück (siehe Die orthogonalen Polynome)
int *n*
Grad des Polynoms

::math::special::chebyshev *n*
Gibt das Tschebyscheff-Polynom vom Grad n (erster Art) zurück
int *n*
Grad des Polynoms

::math::special::laguerre *alpha n*
Gibt das Laguerre-Polynom vom Grad n mit dem Parameter alpha zurück
float *alpha*
Parameter des Laguerre-Polynoms
int *n*
Grad des Polynoms

::math::special::hermite *n*
Gibt das Hermite-Polynom vom Grad n zurück
int *n*
Grad des Polynoms

Die orthogonalen Polynome

Für die Behandlung der klassischen Familien orthogonaler Polynome stützt sich das Paket auf das Paket *math::polynomials*. Um das Polynom an einer bestimmten Koordinate auszuwerten, verwenden Sie den Befehl *evalPolyn*:

```
set leg2 [::math::special::legendre 2]
puts "Value at x=$x: [::math::polynomials::evalPolyn $leg2 $x]"
```

Der Rückgabewert von *legendre* und den anderen Befehlen ist eigentlich die Definition des jeweiligen Polynoms, wie sie in jenem Paket verwendet wird.

Anmerkungen zur Umsetzung

Es sei angemerkt, dass die tatsächliche Implementierung von J0 und J1 auf einfachen Gaußschen Quadraturformeln beruht. Die (absolute) Genauigkeit der Ergebnisse liegt in der Größenordnung von 1.0e-4 oder besser. Der Hauptgrund für diese Art der Implementierung war, dass sie sich schnell umsetzen ließ (die Formeln sind einfach) und auch die Berechnungen schnell erfolgen.

Die Implementierung von J1/2 ist davon nicht betroffen: Diese Funktion lässt sich exakt als Zusammensetzung elementarer Funktionen ausdrücken.

Die Funktionen J0 und J1 sind diejenigen, denen Sie in der Praxis am häufigsten begegnen werden.

Die Berechnung von I_n basiert auf dem Miller-Algorithmus zur Berechnung der Minimalfunktion aus Rekursionsbeziehungen.

Die Berechnung der Gamma- und Betafunktionen stützt sich auf das Paket combinatorics, die der Fehlerfunktionen auf das Paket statistics.

Zur Berechnung der vollständigen elliptischen Integrale wird der AGM-Algorithmus verwendet.

Zahlreiche Informationen zu diesen Funktionen finden Sie in:

Abramowitz und Stegun: *Handbook of Mathematical Functions* (Dover, ISBN 486-61272-4)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: special* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright


```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```


statistics

Stichwörter: statistics, data analysis, mathematics

Name

math::statistics – Grundlegende statistische Funktionen und Prozeduren

Übersicht

package require Tcl 8.5 9
package require math::statistics 1.6.1
::math::statistics::mean *data*
::math::statistics::min *data*
::math::statistics::max *data*
::math::statistics::number *data*
::math::statistics::stdev *data*
::math::statistics::var *data*
::math::statistics::pstdev *data*
::math::statistics::pvar *data*
::math::statistics::median *data*
::math::statistics::basic-stats *data*
::math::statistics::histogram *limits values ?weights?*
::math::statistics::histogram-alt *limits values ?weights?*
::math::statistics::corr *data1 data2*
::math::statistics::interval-mean-stdev *data confidence*
::math::statistics::t-test-mean *data est_mean est_stdev alpha*
::math::statistics::test-normal *data significance*
::math::statistics::lillieforsFit *data*
::math::statistics::test-Duckworth *list1 list2 significance*
::math::statistics::test-anova-F *alpha args*
::math::statistics::test-Tukey-range *alpha args*
::math::statistics::test-Dunnett *alpha control args*

::math::statistics::quantiles *data confidence*
::math::statistics::quantiles *limits counts confidence*
::math::statistics::autocorr *data*
::math::statistics::crosscorr *data1 data2*
::math::statistics::mean-histogram-limits *mean stdev number*
::math::statistics::minmax-histogram-limits *min max number*
::math::statistics::linear-model *xdata ydata intercept*
::math::statistics::linear-residuals *xdata ydata intercept*
::math::statistics::test-2x2 *n11 n21 n12 n22*
::math::statistics::print-2x2 *n11 n21 n12 n22*
::math::statistics::control-xbar *data* ?*nsamples*?
::math::statistics::control-Rchart *data* ?*nsamples*?
::math::statistics::test-xbar *control data*
::math::statistics::test-Rchart *control data*
::math::statistics::test-Kruskal-Wallis *confidence args*
::math::statistics::analyse-Kruskal-Wallis *args*
::math::statistics::test-Levene *groups*
::math::statistics::test-Brown-Forsythe *groups*
::math::statistics::group-rank *args*
::math::statistics::test-Wilcoxon *sample_a sample_b*
::math::statistics::spearman-rank *sample_a sample_b*
::math::statistics::spearman-rank-extended *sample_a sample_b*
::math::statistics::kernel-density *data* opt *-option value ...*
::math::statistics::bootstrap *data sampleSize* ?*numberSamples*?
::math::statistics::wasserstein-distance *prob1 prob2*
::math::statistics::kl-divergence *prob1 prob2*
::math::statistics::logistic-model *xdata ydata*
::math::statistics::logistic-probability *coeffs x*
::math::statistics::tstat *dof* ?*alpha*?
::math::statistics::mv-wls *weights_and_values*
::math::statistics::mv-ols *values*
::math::statistics::pdf-normal *mean stdev value*
::math::statistics::pdf-lognormal *mean stdev value*
::math::statistics::pdf-exponential *mean value*
::math::statistics::pdf-uniform *xmin xmax value*
::math::statistics::pdf-triangular *xmin xmax value*
::math::statistics::pdf-symmetric-triangular *xmin xmax value*
::math::statistics::pdf-gamma *alpha beta value*
::math::statistics::pdf-poisson *mu k*
::math::statistics::pdf-chisquare *df value*
::math::statistics::pdf-student-t *df value*
::math::statistics::pdf-gamma *a b value*
::math::statistics::pdf-beta *a b value*
::math::statistics::pdf-weibull *scale shape value*
::math::statistics::pdf-gumbel *location scale value*
::math::statistics::pdf-pareto *scale shape value*

::math::statistics::pdf-cauchy *location scale value*
::math::statistics::pdf-laplace *location scale value*
::math::statistics::pdf-kumaraswamy *a b value*
::math::statistics::pdf-negative-binomial *r p value*
::math::statistics::cdf-normal *mean stdev value*
::math::statistics::cdf-lognormal *mean stdev value*
::math::statistics::cdf-exponential *mean value*
::math::statistics::cdf-uniform *xmin xmax value*
::math::statistics::cdf-triangular *xmin xmax value*
::math::statistics::cdf-symmetric-triangular *xmin xmax value*
::math::statistics::cdf-students-t *degrees value*
::math::statistics::cdf-gamma *alpha beta value*
::math::statistics::cdf-poisson *mu k*
::math::statistics::cdf-beta *a b value*
::math::statistics::cdf-weibull *scale shape value*
::math::statistics::cdf-gumbel *location scale value*
::math::statistics::cdf-pareto *scale shape value*
::math::statistics::cdf-cauchy *location scale value*
::math::statistics::cdf-F *nf1 nf2 value*
::math::statistics::cdf-laplace *location scale value*
::math::statistics::cdf-kumaraswamy *a b value*
::math::statistics::cdf-negative-binomial *r p value*
::math::statistics::empirical-distribution *values*
::math::statistics::random-normal *mean stdev number*
::math::statistics::random-lognormal *mean stdev number*
::math::statistics::random-exponential *mean number*
::math::statistics::random-uniform *xmin xmax number*
::math::statistics::random-triangular *xmin xmax number*
::math::statistics::random-symmetric-triangular *xmin xmax number*
::math::statistics::random-gamma *alpha beta number*
::math::statistics::random-poisson *mu number*
::math::statistics::random-chisquare *df number*
::math::statistics::random-student-t *df number*
::math::statistics::random-beta *a b number*
::math::statistics::random-weibull *scale shape number*
::math::statistics::random-gumbel *location scale number*
::math::statistics::random-pareto *scale shape number*
::math::statistics::random-cauchy *location scale number*
::math::statistics::random-laplace *location scale number*
::math::statistics::random-kumaraswamy *a b number*
::math::statistics::random-negative-binomial *r p number*
::math::statistics::histogram-uniform *xmin xmax limits number*
::math::statistics::incompleteGamma *x p* ?*tol*?
::math::statistics::incompleteBeta *a b x* ?*tol*?
::math::statistics::estimate-pareto *values*
::math::statistics::estimate-exponential *values*

::math::statistics::estimate-laplace *values*
::math::statistics::estimante-negative-binomial *r values*
::math::statistics::filter *varname data expression*
::math::statistics::map *varname data expression*
::math::statistics::samplescount *varname list expression*
::math::statistics::subdivide
::math::statistics::plot-scale *canvas xmin xmax ymin ymax*
::math::statistics::plot-xydata *canvas xdata ydata tag*
::math::statistics::plot-xyline *canvas xdata ydata tag*
::math::statistics::plot-tdata *canvas tdata tag*
::math::statistics::plot-tline *canvas tdata tag*
::math::statistics::plot-histogram *canvas counts limits tag*

Beschreibung

Das Paket **math::statistics** enthält Funktionen und Prozeduren für die grundlegende statistische Datenanalyse, wie zum Beispiel:

- Deskriptive statistische Kennzahlen (Mittelwert, Minimalwert, Maximalwert, Standardabweichung)
- Schätzungen der Verteilung in Form von Histogrammen und Quantilen
- Grundlegende Hypothesentests
- Wahrscheinlichkeitsdichten und Verteilungsfunktionen.

Das Paket soll bei der Entwicklung von Datenanalyseanwendungen oder bei Ad-hoc-Datenanalysen helfen; es ist selbst keine vollständige Anwendung und soll auch nicht mit vollwertigen (nicht-)kommerziellen Statistikpaketen konkurrieren.

Der Zweck dieses Dokuments besteht darin, die implementierten Prozeduren zu beschreiben und einige Beispiele für ihre Verwendung zu geben. Da es umfangreiche Literatur zu den verwendeten Algorithmen gibt, verweisen wir für weitere Erläuterungen auf einschlägige Lehrbücher. Das Paket enthält eine recht große Anzahl öffentlicher Prozeduren. Diese lassen sich in drei Gruppen unterteilen: allgemeine Prozeduren, Prozeduren, die sich mit bestimmten statistischen Verteilungen befassen, Listenprozeduren zur Auswahl oder Transformation von Daten sowie einfache Prozeduren zur grafischen Darstellung (diese erfordern Tk). *Hinweis:* Die zu analysierenden Daten sind stets in einer einfachen Liste enthalten. Fehlende Werte werden als leere Elemente der Liste dargestellt. *Hinweis:* Mit Version 1.0.1 wurde ein Fehler in den Prozeduren *pdf-lognormal*, *cdf-lognormal* und *random-lognormal* behoben. In früheren Versionen wurde das Argument für die Standardabweichung tatsächlich so verwendet, als handele es sich um die Varianz.

Allgemeine Prozeduren

Die allgemeinen statistischen Prozeduren sind:

::math::statistics::mean *data*
Ermittelt den Mittelwert (*mean*) der angegebenen Datenliste.
list *data*
- Liste der Daten

::math::statistics::min *data*
Ermittelt den *Minimalwert* der angegebenen Datenliste.

list *data*
- Liste der Daten

::math::statistics::max *data*
Ermittelt den *Maximalwert* der angegebenen Datenliste.
list *data*
- Liste der Daten

::math::statistics::number *data*
Ermittelt die Anzahl (*number*) der nicht fehlenden Daten in der angegebenen Liste
list *data*
- Liste der Daten

::math::statistics::stdev *data*
Ermittelt die *Stichprobenstandardabweichung* der Daten in der angegebenen Liste
list *data*
- Liste der Daten

::math::statistics::var *data*
Ermittelt die *Stichprobenvarianz* der Daten in der angegebenen Liste
list *data*
- Liste der Daten

::math::statistics::pstdev *data*
Ermittelt die *Standardabweichung der Grundgesamtheit* der Daten in der angegebenen Liste
list *data*
- Liste der Daten

::math::statistics::pvar *data*
Ermittelt die *Varianz der Grundgesamtheit* der Daten in der angegebenen Liste
list *data*
- Liste der Daten

::math::statistics::median *data*
Ermittelt den *Median* der Daten in der angegebenen Liste (beachten Sie, dass hierfür eine Sortierung der Daten erforderlich ist, was einen hohen Rechenaufwand verursachen kann)
list *data*
- Liste der Daten

::math::statistics::basic-stats *data*
Ermittelt eine Liste aller beschreibenden Kennzahlen: Mittelwert, Minimalwert, Maximalwert, Anzahl der Daten, Stichprobenstandardabweichung, Stichprobenvarianz, Standardabweichung der Grundgesamtheit und Varianz der Grundgesamtheit.

(Diese Routine wird immer aufgerufen, wenn eine oder alle der grundlegenden statistischen Kennzahlen benötigt werden. Daher werden alle Berechnungen durchgeführt und die betreffenden Werte zurückgegeben.)
list *data*

- Liste der Daten

::math::statistics::histogram *limits values* ?weights?

Ermittelt Histogrammdaten für die angegebene Datenliste. Gibt eine Liste zurück, die aus der Anzahl der Werte besteht, die in jedes Intervall fallen. (Das erste Intervall umfasst alle Werte, die kleiner als die erste Grenze sind; das letzte Intervall umfasst alle Werte, die größer als die letzte Grenze sind. Es gibt ein Intervall mehr als es Grenzwerte gibt.)

Optional können Sie Gewichte verwenden, um das Histogramm zu beeinflussen.

list *limits*

- Liste der Obergrenzen (in aufsteigender Reihenfolge) für die Intervalle des Histogramms.

list *values*

- Liste der Daten

list *weights*

- Liste der Gewichte, jeweils ein Gewicht pro Wert

::math::statistics::histogram-alt *limits values* ?weights?

Alternative Umsetzung der Histogramm-Prozedur: Das offene Ende der Intervalle liegt an der unteren Grenze statt an der oberen Grenze.

list *limits*

- Liste der Obergrenzen (in aufsteigender Reihenfolge) für die Intervalle des Histogramms.

list *values*

- Liste der Daten

list *weights*

- Liste der Gewichte, jeweils ein Gewicht pro Wert

::math::statistics::corr *data1 data2*

Ermittelt den Korrelationskoeffizienten zwischen zwei Datenreihen.

list *data1*

- Erste Liste mit Daten

list *data2*

- Zweite Liste mit Daten

::math::statistics::interval-mean-stdev *data confidence*

Gibt das Intervall zurück, das mit einem bestimmten Konfidenzniveau den Mittelwert enthält, sowie eines, das die Standardabweichung enthält (unter der Annahme einer Normalverteilung)

list *data*

- Liste der Rohdatenwerte (kleine Stichprobe)

float *confidence*

- Konfidenzniveau (beispielsweise 0,95 oder 0,99)

::math::statistics::t-test-mean *data est_mean est_stdev alpha*

Prüft, ob der Mittelwert einer Stichprobe mit einer bestimmten Wahrscheinlichkeit der geschätzten Normalverteilung entspricht. Gibt 1 zurück, wenn der Test erfolgreich ist, oder 0, wenn der Mittelwert wahrscheinlich nicht zur angegebenen Verteilung passt.

list *data*

- Liste der Rohdatenwerte (kleine Stichprobe)

float *est_mean*

- Geschätzter Mittelwert der Verteilung

float *est_stdev*

- Geschätzte Standardabweichung der Verteilung

float *alpha*

- Wahrscheinlichkeitsniveau (beispielsweise 0,95 oder 0,99)

::math::statistics::test-normal *data significance*

Prüft, ob die angegebenen Daten unter Berücksichtigung eines bestimmten Signifikanzniveaus einer Normalverteilung folgen. Gibt 1 zurück, wenn die Daten innerhalb des Signifikanzniveaus normalverteilt sind; andernfalls wird 0 zurückgegeben. Der zugrunde liegende Test ist der Lilliefors-Test. Geringere Werte der Signifikanz bedeuten eine strengere Prüfung.

list *data*

- Liste der Werte der Rohdaten

float *significance*

- Signifikanzniveau (entweder 0,01, 0,05, 0,10, 0,15 oder 0,20). Aus Gründen der Kompatibilität werden auch die Werte "1-significance", 0,80, 0,85, 0,90, 0,95 oder 0,99 akzeptiert.

Kompatibilitätsproblem: In der ursprünglichen Implementierung und Dokumentation wurde der Begriff "confidence" (Konfidenz) verwendet und ein Wert 1-significance benutzt (siehe Ticket 2812473fff). Dies wurde ab Version 0.9.3 korrigiert.

::math::statistics::lillieforsFit *data*

Gibt den Anpassungsgrad an eine Normalverteilung nach Lilliefors zurück. Je höher der Wert, desto wahrscheinlicher ist es, dass die Daten tatsächlich normalverteilt sind. Für den Test sind mindestens *fünf* Datenpunkte erforderlich.

list *data*

- Liste der Werte der Rohdaten

::math::statistics::test-Duckworth *list1 list2 significance*

Ermittelt anhand des Tukey-Duckworth-Tests, ob zwei Datensätze denselben Median aufweisen. Die Prozedur gibt 0 zurück, wenn die Mediane ungleich sind, 1, wenn sie gleich sind, und -1, wenn der Test nicht durchgeführt werden kann (der kleinste Wert muss in einem anderen Datensatz liegen als der größte Wert). # # Argumente: # list1 Werte des ersten Datensatzes # list2 Werte des zweiten Datensatzes # significance Signifikanzniveau (entweder 0,05, 0,01 oder 0,001) # # Gibt zurück: Prüft, ob die angegebenen Daten einer Normalverteilung mit einem bestimmten Signifikanzniveau folgen. Gibt 1 zurück, wenn die Daten innerhalb des Signifikanzniveaus normalverteilt sind, andernfalls 0. Der zugrunde liegende Test ist der Lilliefors-Test. Kleinere Werte des Signifikanzniveaus bedeuten eine strengere Prüfung.

list *list1*

- Erste Liste mit Daten

list *list2*

- Zweite Liste mit Daten

float *significance*
- Signifikanzniveau (entweder 0,05, 0,01 oder 0,001)

::math::statistics::test-anova-F *alpha args*

Ermittelt, ob zwei oder mehr Gruppen mit normalverteilten Daten dieselben Mittelwerte aufweisen. Die Prozedur gibt 0 zurück, wenn die Mittelwerte wahrscheinlich ungleich sind, und 1, wenn sie gleich sind. Hierbei handelt es sich um eine einfaktorielle Varianzanalyse (One-Way-ANOVA). Die Gruppen können auch in einer verschachtelten Liste gespeichert sein: Die Prozedur gibt für jedes Gruppenpaar eine Liste der Vergleichsergebnisse zurück. Jedes Element dieser Liste enthält: den Index der ersten Gruppe und den der zweiten Gruppe, die Angabe, ob die Mittelwerte wahrscheinlich unterschiedlich sind (1) oder nicht (0), sowie das Konfidenzintervall, auf dem die Schlussfolgerung basiert. Die Gruppen können auch in einer verschachtelten Liste gespeichert sein:

```
test-anova-F 0.05 $A $B $C
#
# Or equivalently:
#
test-anova-F 0.05 [list $A $B $C]
```

float *alpha*
- Signifikanzniveau

list *args*
- Zwei oder mehr zu prüfende Datengruppen

::math::statistics::test-Tukey-range *alpha args*

Stellt mithilfe des Tukey-Spannweitentests fest, ob zwei oder mehr Gruppen mit normalverteilten Daten dieselben Mittelwerte aufweisen. Dieser Test ergänzt den ANOVA-Test. Die Prozedur gibt eine Liste der Vergleichsergebnisse für jedes Gruppenpaar zurück. Jedes Element dieser Liste enthält: den Index der ersten Gruppe und den der zweiten Gruppe, die Angabe, ob die Mittelwerte wahrscheinlich unterschiedlich sind (1) oder nicht (0), sowie das Konfidenzintervall, auf dem die Schlussfolgerung basiert. Die Gruppen können ebenso wie beim ANOVA-Test auch in einer verschachtelten Liste gespeichert werden.

float *alpha*
- Signifikanzniveau – entweder 0,05 oder 0,01

list *args*
- Zwei oder mehr zu prüfende Datengruppen

::math::statistics::test-Dunnett *alpha control args*

Stellt mithilfe des Dunnett-Tests fest, ob eine oder mehrere Gruppen mit normalverteilten Daten denselben Mittelwert wie die Kontrollgruppe aufweisen. Dieser Test ergänzt den ANOVA-Test. Die Prozedur gibt eine Liste der Vergleichsergebnisse für jede Gruppe im Vergleich zur Kontrollgruppe zurück. Jedes Element dieser Liste enthält: die Angabe, ob die Mittelwerte wahrscheinlich unterschiedlich sind (1) oder nicht (0), sowie das Konfidenzintervall, auf dem diese Schlussfolgerung basiert. Die Gruppen können ebenso wie beim ANOVA-Test auch in einer verschachtelten Liste gespeichert werden.

Hinweis: Besondere Vorsicht ist geboten, wenn nur eine Gruppe zum Vergleich mit der

Kontrollgruppe vorhanden ist:

```
    test-Dunnett-F 0.05 $control [list $A]
```

Ansonsten wird die Gruppe A in Gruppen mit jeweils einem Element aufgeteilt – dies ist auf eine Mehrdeutigkeit zurückzuführen.

float *alpha*

- Signifikanzniveau – entweder 0,05 oder 0,01

list *args*

- Eine oder mehrere zu prüfende Datengruppen

::math::statistics::quantiles *data confidence*

Gibt die Quantile für einen bestimmten Datensatz zurück

list *data*

- Liste der Werte der Rohdaten

float *confidence*

- Konfidenzniveau (beispielsweise 0,95 oder 0,99) oder eine Liste von Konfidenzniveaus.

::math::statistics::quantiles *limits counts confidence*

Gibt die Quantile auf der Grundlage der Histogrammdaten zurück (Alternative zum Aufruf mit zwei Argumenten)

list *limits*

- Liste der Obergrenzen aus dem Histogramm

list *counts*

- Liste der Häufigkeiten für jedes Intervall im Histogramm

float *confidence*

- Konfidenzniveau (beispielsweise 0,95 oder 0,99) oder eine Liste von Konfidenzniveaus.

::math::statistics::autocorr *data*

Gibt die Autokorrelationsfunktion als Liste von Werten zurück (unter der Annahme gleichmäßiger Abstände zwischen den Stichproben, etwa die Hälfte der Anzahl der Rohdaten).

Die Korrelation wird so bestimmt, dass der erste Wert stets 1 ist und alle anderen Werte gleich oder kleiner als 1 sind. Die Anzahl der beteiligten Werte nimmt mit zunehmender "Zeit" (dem Index in der Liste der zurückgegebenen Werte) ab.

list *data*

- Rohdaten, für die die Autokorrelation bestimmt werden muss

::math::statistics::crosscorr *data1 data2*

Gibt die Kreuzkorrelationsfunktion als Liste von Werten zurück (unter der Annahme gleichmäßiger Abstände zwischen den Stichproben, etwa die Hälfte der Anzahl der Rohdaten).

Die Korrelation wird so berechnet, dass die Werte betragsmäßig niemals größer als 1 sein können. Die Anzahl der beteiligten Werte nimmt mit zunehmender "Zeit" (dem Index in der Liste der zurückgegebenen Werte) ab.

list *data1*

- Erste Liste mit Daten

list *data2*

- Zweite Liste mit Daten

::math::statistics::mean-histogram-limits *mean stdev number*

Ermittelt anhand von Mittelwert und Standardabweichung sinnvolle Grenzen für ein Histogramm. Hilfsfunktion – das Ergebnis eignet sich für die Histogrammfunktion.

float *mean*

- Mittelwert der Daten

float *stdev*

- Standardabweichung

int *number*

- Anzahl der zu generierenden Grenzwerte (Standardwert: 8)

::math::statistics::minmax-histogram-limits *min max number*

Ermittelt anhand eines Minimal- und Maximalwerts sinnvolle Grenzen für ein Histogramm.

Hilfsfunktion – das Ergebnis eignet sich für die Histogrammfunktion.

float *min*

- Erwarteter Mindestwert

float *max*

- Erwarteter Höchstwert

int *number*

- Anzahl der zu generierenden Grenzwerte (Standardwert: 8)

::math::statistics::linear-model *xdata ydata intercept*

Ermittelt die Koeffizienten für eine lineare Regression zwischen zwei Datenreihen (das Modell: Y = A + B*X). Gibt eine Liste von Parametern zurück, die die Anpassung beschreiben

list *xdata*

- Liste unabhängiger Daten

list *ydata*

- Liste der anzupassenden abhängigen Daten

boolean *intercept*

- (Optional) Achsenabschnitt berechnen (1, Standard) oder an eine Gerade durch den Ursprung anpassen (0).

Das Ergebnis besteht aus der folgenden Liste:
- (Schätzung des) Achsenabschnitts A
- (Schätzung der) Steigung B
- Standardabweichung von Y relativ zur Anpassung
- Korrelationskoeffizient R^2
- Anzahl der Freiheitsgrade df
- Standardfehler des Achsenabschnitts A
- Signifikanzniveau von A
- Standardfehler der Steigung B
- Signifikanzniveau von B

::math::statistics::linear-residuals *xdata ydata intercept*

Ermittelt die Differenz zwischen den Ist-Werten und den vom linearen Modell vorhergesagten

Werten.

Gibt eine Liste der Differenzen zwischen den Ist-Werten und den vorhergesagten Werten zurück.

list *xdata*
- Liste unabhängiger Daten

list *ydata*
- Liste der anzupassenden abhängigen Daten

boolean *intercept*
- (Optional) Achsenabschnitt berechnen (1, Standard) oder an eine Gerade durch den Ursprung anpassen (0)

::math::statistics::test-2x2 *n11 n21 n12 n22*

Ermittelt, ob sich zwei Stichproben, die jeweils einer Binomialverteilung folgen, signifikant voneinander unterscheiden oder nicht (was auf einen unterschiedlichen Parameter hindeuten würde).

Gibt den "Chi-Quadrat"-Wert zurück, anhand dessen die Signifikanz bestimmt werden kann.

int *n11*
- Anzahl der Ergebnisse mit dem ersten Wert aus der ersten Stichprobe.

int *n21*
- Anzahl der Ergebnisse mit dem ersten Wert aus der zweiten Stichprobe.

int *n12*
- Anzahl der Ergebnisse mit dem zweiten Wert aus der ersten Stichprobe.

int *n22*
- Anzahl der Ergebnisse mit dem zweiten Wert aus der zweiten Stichprobe.

::math::statistics::print-2x2 *n11 n21 n12 n22*

Ermittelt, ob sich zwei Stichproben, die jeweils einer Binomialverteilung folgen, signifikant voneinander unterscheiden oder nicht (was auf einen unterschiedlichen Parameter hindeuten würde).

Gibt einen kurzen Bericht zurück, der in einer interaktiven Sitzung nützlich ist.

int *n11*
- Anzahl der Ergebnisse mit dem ersten Wert aus der ersten Stichprobe.

int *n21*
- Anzahl der Ergebnisse mit dem ersten Wert aus der zweiten Stichprobe.

int *n12*
- Anzahl der Ergebnisse mit dem zweiten Wert aus der ersten Stichprobe.

int *n22*
- Anzahl der Ergebnisse mit dem zweiten Wert aus der zweiten Stichprobe.

::math::statistics::control-xbar *data* ?nsamples?

Ermittelt die Kontrollgrenzen für ein xbar-Diagramm. Die Anzahl der Daten in jeder Teilstichprobe beträgt standardmäßig 4. Es sind mindestens 20 Teilstichproben erforderlich.

Gibt den Mittelwert, die Untergrenze, die Obergrenze und die Anzahl der Daten pro Teilstichprobe zurück.

list *data*
- Liste der Beobachtungsdaten

int *nsamples*
- Anzahl der Daten pro Teilstichprobe

::math::statistics::control-Rchart *data* ?nsamples?

Ermittelt die Kontrollgrenzen für ein R-Diagramm. Die Anzahl der Daten in jeder Teilstichprobe (nsamples) ist standardmäßig auf 4 festgelegt. Es sind mindestens 20 Teilstichproben erforderlich.

Gibt die mittlere Spannweite, die untere Grenze, die obere Grenze und die Anzahl der Daten pro Teilstichprobe zurück.

list *data*
- Liste der Beobachtungsdaten

int *nsamples*
- Anzahl der Daten pro Teilstichprobe

::math::statistics::test-xbar *control data*

Ermittelt, ob die Daten die Kontrollgrenzen für das xbar-Diagramm überschreiten.

Gibt eine Liste der Teilstichproben (bzw. deren Indizes) zurück, die tatsächlich die Grenzen überschreiten.

list *control*
- Kontrollgrenzen, wie sie von der Prozedur "control-xbar" zurückgegeben werden

list *data*
- Liste der Beobachtungsdaten

::math::statistics::test-Rchart *control data*

Ermittelt, ob die Daten die Kontrollgrenzen für das R-Diagramm überschreiten.

Gibt eine Liste der Teilstichproben (bzw. deren Indizes) zurück, die tatsächlich die Grenzen überschreiten.

list *control*
- Kontrollgrenzen, wie sie von der Prozedur "control-Rchart" zurückgegeben werden

list *data*
- Liste der Beobachtungsdaten

::math::statistics::test-Kruskal-Wallis *confidence args*

Prüft mithilfe des Kruskal-Wallis-Tests, ob die Mediane der Grundgesamtheiten von zwei oder mehr Gruppen bei einem vorgegebenen Konfidenzniveau gleich sind.

float *confidence*
- Zu verwendendes Konfidenzniveau (0–1)

list *args*
- Zwei oder mehr Datenlisten

::math::statistics::analyse-Kruskal-Wallis *args*

Berechnet die statistischen Kennzahlen für den Kruskal-Wallis-Test. Gibt die Kruskal-Wallis-Statistik sowie die Wahrscheinlichkeit zurück, mit der dieser Wert auftreten

würde, wenn die Mediane der Grundgesamtheiten gleich wären.

list *args*

- Zwei oder mehr Datenlisten

::math::statistics::test-Levene *groups*

Berechnet die Levene-Statistik, um festzustellen, ob Gruppen von Daten dieselbe Varianz aufweisen (homoskedastisch sind) oder nicht. Die Daten sind in Gruppen organisiert. Bei dieser Version wird der Mittelwert der Daten als Maß zur Bestimmung der Abweichungen herangezogen. Die Statistik entspricht einer F-Statistik mit den Freiheitsgraden k-1 und N-k, wobei k die Anzahl der Gruppen und N die Gesamtzahl der Daten ist.

list *groups*

- Liste der Datengruppen

::math::statistics::test-Brown-Forsythe *groups*

Berechnet die Brown-Forsythe-Statistik, um festzustellen, ob Datengruppen dieselbe Varianz aufweisen (homoskedastisch sind) oder nicht. Ähnlich wie beim Levene-Test, allerdings wird bei dieser Variante der Median der Daten herangezogen.

list *groups*

- Liste der Datengruppen

::math::statistics::group-rank *args*

Bestimmt die Ränge der Daten aller Gruppen bezogen auf die gesamte Datenmenge. Gibt eine Liste zurück, die für jeden Datenpunkt die Gruppen-ID, den Wert und den Rang (im Falle von Gleichständen möglicherweise eine rationale Zahl) enthält.

list *args*

- Zwei oder mehr Datenlisten

::math::statistics::test-Wilcoxon *sample_a sample_b*

Berechnet die Wilcoxon-Teststatistik, um festzustellen, ob zwei Stichproben denselben Median aufweisen oder nicht. (Die Statistik kann als standardnormal angesehen werden, wenn beide Stichprobengrößen größer als 10 sind.) Gibt den Wert dieser Statistik zurück.

list *sample_a*

- Liste der Daten, aus denen sich die erste Stichprobe zusammensetzt

list *sample_b*

- Liste der Daten, aus denen sich die zweite Stichprobe zusammensetzt

::math::statistics::spearman-rank *sample_a sample_b*

Gibt als Alternative zum gewöhnlichen (Pearson-)Korrelationskoeffizienten die Spearman-Rangkorrelation zurück. Die beiden Stichproben sollten die gleiche Anzahl an Datenpunkten aufweisen.

list *sample_a*

- Erste Liste mit Daten

list *sample_b*

- Zweite Liste mit Daten

::math::statistics::spearman-rank-extended *sample_a sample_b*

Gibt als Alternative zum gewöhnlichen (Pearson-)Korrelationskoeffizienten die

Spearman-Rangkorrelation sowie zusätzliche Daten zurück. Die beiden Stichproben sollten die gleiche Anzahl an Datenpunkten aufweisen. Die Prozedur gibt den Korrelationskoeffizienten, die Anzahl der verwendeten Datenpaare und den z-Wert zurück, eine annähernd standardnormalverteilte Statistik, die die Signifikanz der Korrelation angibt.

list *sample_a*

- Erste Liste mit Daten

list *sample_b*

- Zweite Liste mit Daten

::math::statistics::kernel-density *data* opt *-option value* ...

Gibt die Dichtefunktion auf der Grundlage der Kernel-Dichte-Schätzung zurück. Die Prozedur wird durch eine kleine Anzahl von Optionen gesteuert, für die jeweils ein sinnvoller Standardwert vorgegeben ist.

Der Rückgabewert besteht aus drei Listen: den Mittelpunkten der Klassen, der zugehörigen Wahrscheinlichkeitsdichte sowie einer Liste von Berechnungsparametern (Anfang und Ende des Intervalls, Mittelwert und Standardabweichung sowie die verwendete Bandbreite). Die Berechnungsparameter können für weitere Analysen verwendet werden.

list *data*

- Die zu untersuchenden Daten

list *args*

- Option-Wert-Paare:

-weights *weights*

Gewichtung pro Datenpunkt (Standard: 1 für alle Daten)

-bandwidth *value*

Für die Schätzung zu verwendende Bandbreite (Standard: wird anhand der Standardabweichung ermittelt)

-number *value*

Anzahl der zurückzugebenden Klassen (Standard: 100)

-interval *{begin end}*

Anfang und Ende des Intervalls, für das die Dichte zurückgegeben wird (Standard: Mittelwert +/- 3*Standardabweichung)

-kernel *function*

Zu verwendender Kernel (einer von: gaussian, cosine, epanechnikov, uniform, triangular, biweight, logistic; Standard: gaussian)

::math::statistics::bootstrap *data sampleSize* ?numberSamples?

Erstellt eine Teilstichprobe oder mehrere Teilstichproben aus einer gegebenen Datenliste. Die Daten in den Stichproben werden aus dieser Liste ausgewählt – dabei können Wiederholungen auftreten. Gibt es nur eine Teilstichprobe, wird die Stichprobe selbst zurückgegeben (als Liste von "sampleSize" Werten), andernfalls eine Liste von Stichproben.

list *data*

Liste der zur Auswahl stehenden Werte

int *sampleSize*

Anzahl der Werte pro Stichprobe

int *numberSamples*

Anzahl der Stichproben (Standard: 1)

::math::statistics::wasserstein-distance *prob1 prob2*

Berechnet den Wasserstein-Abstand oder den Earth-Mover-Abstand für zwei Histogramme oder Wahrscheinlichkeitsdichten mit gleichmäßigen Intervallen. Die Histogramme müssen nicht so normiert sein, dass ihre Summe eins ergibt, sie müssen jedoch die gleiche Anzahl von Einträgen aufweisen.

Hinweis: Es wird davon ausgegangen, dass die Histogramme auf denselben äquidistanten Intervallen basieren. Da die Grenzen nicht übergeben werden, wird der Wert in der Länge der Intervalle ausgedrückt.

list *prob1*

Liste der Werte für das erste Histogramm/die erste Wahrscheinlichkeitsdichte

list *prob2*

Liste der Werte für das zweite Histogramm/die zweite Wahrscheinlichkeitsdichte

::math::statistics::kl-divergence *prob1 prob2*

Berechnet die Kullback-Leibler-Divergenz (KL-Divergenz) für zwei Histogramme oder Wahrscheinlichkeitsdichten mit gleichmäßigen Intervallen. Die Histogramme müssen nicht so normiert sein, dass ihre Summe eins ergibt, sie müssen jedoch die gleiche Anzahl von Einträgen aufweisen.

Hinweis: Es wird angenommen, dass die Histogramme auf denselben äquidistanten Intervallen basieren. Da die Grenzwerte nicht übergeben werden, wird der Wert in der Länge der Intervalle ausgedrückt.

Beachten Sie außerdem, dass die KL-Divergenz nicht symmetrisch ist und dass das zweite Histogramm an Stellen, an denen das erste Histogramm Werte ungleich Null aufweist, keine Nullen enthalten darf.

list *prob1*

Liste der Werte für das erste Histogramm/die erste Wahrscheinlichkeitsdichte

list *prob2*

Liste der Werte für das zweite Histogramm/die zweite Wahrscheinlichkeitsdichte

::math::statistics::logistic-model *xdata ydata*

Schätzt die Koeffizienten des logistischen Modells, das am besten zu den Daten passt. Die Daten bestehen aus unabhängigen x-Werten und dem Ergebnis 0 oder 1 für jeden der x-Werte. Das Ergebnis kann verwendet werden, um die Wahrscheinlichkeit zu schätzen, dass ein bestimmter x-Wert den Wert 1 ergibt.

list *xdata*

Liste der Werte, für die Erfolg (1) oder Misserfolg (0) bekannt ist

list *ydata*

Liste der Erfolge oder Misserfolge, die den einzelnen Werten in *xdata* entsprechen.

::math::statistics::logistic-probability *coeffs x*

Berechnet die Erfolgswahrscheinlichkeit für den Wert *x* aus den gegebenen Koeffizienten des logistischen Modells.

list *coeffs*

Liste der Koeffizienten, wie sie durch den Befehl **logistic-model** ermittelt wurden

float *x*

X-Wert, für den die Wahrscheinlichkeit zu bestimmen ist

Multivariate lineare Regression

Neben der linearen Regression mit einer einzigen unabhängigen Variable bietet das Statistikpaket zwei Prozeduren zur Durchführung einer linearen Regression nach der Methode der kleinsten Quadrate (OLS) und der gewichteten kleinsten Quadrate (WLS) mit mehreren Variablen. Diese wurden von Eric Kemp-Benedict entwickelt.

Zusätzlich zu diesen beiden bietet es eine Prozedur (tstat) zur Berechnung des Wertes der t-Statistik für die angegebene Anzahl von Freiheitsgraden, der erforderlich ist, um ein bestimmtes Signifikanzniveau nachzuweisen.

Hinweis: Diese Prozeduren setzen das Paket math::linearalgebra voraus.

Beschreibung der Prozeduren

::math::statistics::tstat *dof* ?alpha?

Gibt den Wert der t-Verteilung t* zurück, der folgende Bedingung erfüllt:

```
    P(t*)  =  1 - alpha/2
    P(-t*) =  alpha/2
```

für die Anzahl der Freiheitsgrade dof.

Bei einer Stichprobe normalverteilter Daten x mit einem Schätzwert xbar für den Mittelwert und sbar für die Standardabweichung lässt sich das Alpha-Konfidenzintervall für den Schätzwert des Mittelwerts wie folgt berechnen:

```
      ( xbar - t* sbar , xbar + t* sbar)
```

Die Rückgabewerte dieser Prozedur können mit einer geschätzten t-Statistik verglichen werden, um festzustellen, ob der geschätzte Wert eines Parameters sich bei dem angegebenen Konfidenzniveau signifikant von Null unterscheidet.

int *dof*

Anzahl der Freiheitsgrade

float *alpha*

Konfidenzniveau der t-Verteilung. Der Standardwert beträgt 0,05.

::math::statistics::mv-wls *weights_and_values*

Führt eine gewichtete lineare Regression nach der Methode der kleinsten Quadrate für die angegebenen Datenpunkte durch, wobei jedem Punkt ein Gewicht zugewiesen wird.

Das lineare Modell hat die Form

```
    y = b0 + b1 * x1 + b2 * x2 ... + bN * xN + error
```

und jeder Punkt erfüllt

```
    yi = b0 + b1 * xi1 + b2 * xi2 + ... + bN * xiN + Residual_i
```

Die Prozedur gibt eine Liste mit den folgenden Elementen zurück:
- Die R-Quadrat-Statistik
- Die bereinigte R-Quadrat-Statistik
- Eine Liste, die die geschätzten Koeffizienten b1, ..., bN und b0 enthält (Die Konstante b0 steht an letzter Stelle der Liste.)
- Eine Liste mit den Standardfehlern der Koeffizienten
- Eine Liste mit den 95-%-Konfidenzgrenzen der Koeffizienten, wobei jedes Grenzenpaar als Liste mit zwei Werten zurückgegeben wird

Argumente:

list *weights_and_values*

Eine Liste, bestehend aus: dem Gewicht für die erste Beobachtung, den Daten für die erste Beobachtung (als Unterliste), dem Gewicht für die zweite Beobachtung (als Unterliste) und so weiter. Die Unterlisten mit den Daten sind als Listen der Werte der abhängigen Variable y und der unabhängigen Variablen x1, x2 bis xN organisiert.

Beispiel für die Verwendung: Die Gewichtungsfaktoren sind recht einfach: 0,2 für negative Werte, um anzuzeigen, dass wir diesen Beobachtungen weniger Vertrauen schenken, und 1,0 für alle positiven Werte.

```
# Store the value of the unicode value for the "+/-" character
set pm "\u00B1"

# Provide some data
set data { 0.2 {  -.67  14.18  60.03 -7.5  }
           1.0 { 36.97  15.52  34.24 14.61 }
           0.2 {-29.57  21.85  83.36 -7.   }
           0.2 {-16.9   11.79  51.67 -6.56 }
           1.0 { 14.09  16.24  36.97 -12.84}
           1.0 { 31.52  20.93  45.99 -25.4 }
           1.0 { 24.05  20.69  50.27  17.27}
           1.0 { 22.23  16.91  45.07  -4.3 }
           1.0 { 40.79  20.49  38.92  -.73 }
           0.2 {-10.35  17.24  58.77  18.78}}

# Call the ols routine
set results [::math::statistics::mv-ols $data]

# Pretty-print the results
puts "R-squared: [lindex $results 0]"
puts "Adj R-squared: [lindex $results 1]"
puts "Coefficients $pm s.e. -- \[95% confidence interval\]:"
foreach val [lindex $results 2] se [lindex $results 3] bounds
    [lindex $results 4] {
    set lb [lindex $bounds 0]
    set ub [lindex $bounds 1]
    puts "   $val $pm $se -- \[$lb to $ub\]"
}
```

::math::statistics::mv-ols *values*

Führt eine lineare Regression nach der Methode der kleinsten Quadrate für die angegebenen Datenpunkte durch.

Diese Prozedur ruft lediglich ::mvlinreg::wls mit den Gewichten 1,0 auf und gibt dieselben Informationen zurück.

Beispiel für die Verwendung:

```
# Store the value of the unicode value for the "+/-" character
set pm "\u00B1"

# Provide some data
set data {{  -.67  14.18  60.03 -7.5  }
          { 36.97  15.52  34.24 14.61 }
          {-29.57  21.85  83.36 -7.   }
          {-16.9   11.79  51.67 -6.56 }
          { 14.09  16.24  36.97 -12.84}
```

```
          { 31.52  20.93  45.99 -25.4 }
          { 24.05  20.69  50.27  17.27}
          { 22.23  16.91  45.07  -4.3 }
          { 40.79  20.49  38.92  -.73 }
          {-10.35  17.24  58.77  18.78}}

# Call the ols routine
set results [::math::statistics::mv-ols $data]

# Pretty-print the results
puts "R-squared: [lindex $results 0]"
puts "Adj R-squared: [lindex $results 1]"
puts "Coefficients $pm s.e. -- \[95% confidence interval\]:"
foreach val [lindex $results 2] se [lindex $results 3] bounds [lindex
    $results 4] {
    set lb [lindex $bounds 0]
    set ub [lindex $bounds 1]
    puts "   $val $pm $se -- \[$lb to $ub\]"
}
```

Statistische Verteilungen

In der Fachliteratur findet sich eine Vielzahl von Wahrscheinlichkeitsverteilungen. Das Statistik-Paket unterstützt:

- Die Normalverteilung bzw. Gauß-Verteilung sowie die Lognormalverteilung
- Die Gleichverteilung – gleiche Wahrscheinlichkeit für alle Daten innerhalb eines bestimmten Intervalls
- Die Exponentialverteilung – nützlich als Modell für bestimmte Extremwertverteilungen.
- Die Gamma-Verteilung – basierend auf dem unvollständigen Gamma-Integral
- Die Beta-Verteilung
- Die Chi-Quadrat-Verteilung
- Die Studentsche t-Verteilung
- Die Poisson-Verteilung
- Die Pareto-Verteilung
- Die Gumbel-Verteilung
- Die Weibull-Verteilung
- Die Cauchy-Verteilung
- Die F-Verteilung (lediglich die kumulative Verteilungsfunktion)
- PM – Binomialverteilung.

Grundsätzlich gibt es für jede Verteilung Prozeduren für:

- Die Wahrscheinlichkeitsdichte (pdf-*)
- Die Verteilungsfunktion (cdf-*)
- Quantile für die angegebene Verteilung (quantiles-*)
- Histogramme für die angegebene Verteilung (histogram-*)
- Liste von Zufallswerten mit der angegebenen Verteilung (random-*)

Die folgenden Prozeduren wurden implementiert:

::math::statistics::pdf-normal *mean stdev value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes bei einer Normalverteilung mit gegebenem Mittelwert und gegebener Standardabweichung zurück.

float *mean*

- Mittelwert der Verteilung

float *stdev*

- Standardabweichung der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-lognormal *mean stdev value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Lognormalverteilung mit vorgegebenem Mittelwert und vorgegebener Standardabweichung zurück.

float *mean*

- Mittelwert der Verteilung

float *stdev*

- Standardabweichung der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-exponential *mean value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Exponentialverteilung mit einem vorgegebenen Mittelwert zurück.

float *mean*

- Mittelwert der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-uniform *xmin xmax value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes bei einer Gleichverteilung mit vorgegebenen Extremwerten zurück.

float *xmin*

- Mindestwert der Verteilung

float *xmin*

- Maximaler Wert der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-triangular *xmin xmax value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Dreiecksverteilung mit vorgegebenen Extremwerten zurück. Wenn das Argument min kleiner ist als das Argument max, so weisen kleinere Werte eine höhere Wahrscheinlichkeit auf und umgekehrt. Im ersten Fall hat die Wahrscheinlichkeitsdichtefunktion die Form *f(x) = 2(1-x)* und im anderen Fall die Form *f(x) = 2x*.

float *xmin*

- Mindestwert der Verteilung

float *xmin*
- Maximaler Wert der Verteilung

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-symmetric-triangular *xmin xmax value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine symmetrische Dreiecksverteilung mit vorgegebenen Extremwerten zurück.

float *xmin*
- Mindestwert der Verteilung

float *xmin*
- Maximaler Wert der Verteilung

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-gamma *alpha beta value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Gamma-Verteilung mit vorgegebenen Form- und Ratenparametern zurück

float *alpha*
- Formparameter

float *beta*
- Ratenparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-poisson *mu k*

Gibt die Wahrscheinlichkeit einer bestimmten Anzahl von Ereignissen im gleichen Intervall (k) für eine Poisson-Verteilung mit einem gegebenen Mittelwert (mu) zurück.

float *mu*
- Durchschnittliche Anzahl der Vorkommen

int *k*
- Anzahl der Vorkommen

::math::statistics::pdf-chisquare *df value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Chi-Quadrat-Verteilung mit vorgegebenen Freiheitsgraden zurück

float *df*
- Freiheitsgrade

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-student-t *df value*

Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Student-t-Verteilung mit den angegebenen Freiheitsgraden zurück

float *df*
- Freiheitsgrade

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-gamma *a b value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Gamma-Verteilung mit vorgegebenen Form- und Ratenparametern zurück

float *a*
- Formparameter

float *b*
- Ratenparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-beta *a b value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Beta-Verteilung mit vorgegebenen Formparametern zurück

float *a*
- Erster Formparameter

float *b*
- Zweiter Formparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-weibull *scale shape value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Weibull-Verteilung mit vorgegebenen Skalen- und Formparametern zurück

float *location*
- Skalenparameter

float *scale*
- Formparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-gumbel *location scale value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Gumbel-Verteilung mit vorgegebenen Lage- und Formparametern zurück

float *location*
- Lageparameter

float *scale*
- Formparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-pareto *scale shape value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Pareto-Verteilung mit vorgegebenen Skalen- und Formparametern zurück

float *scale*
- Skalenparameter

float *shape*
- Formparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-cauchy *location scale value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Cauchy-Verteilung mit vorgegebenen Lage- und Formparametern zurück. Hinweis: Die Cauchy-Verteilung hat keine endlichen Momente höherer Ordnung.

float *location*
- Lageparameter

float *scale*
- Formparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-laplace *location scale value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Laplace-Verteilung mit vorgegebenen Lage- und Formparametern zurück. Die Laplace-Verteilung setzt sich aus zwei Exponentialfunktionen zusammen, hat eine ausgeprägte Spitze und schwerere Ränder als die Normalverteilung.

float *location*
- Lageparameter (Mittelwert)

float *scale*
- Formparameter

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-kumaraswamy *a b value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine Kumaraswamy-Verteilung mit den vorgegebenen Parametern a und b zurück. Die Kumaraswamy-Verteilung ist mit der Beta-Verteilung verwandt, weist jedoch eine leicht zu berechnende kumulative Verteilungsfunktion auf.

float *a*
- Parameter a

float *b*
- Parameter b

float *value*
- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::pdf-negative-binomial *r p value*
Gibt die Wahrscheinlichkeit eines bestimmten Wertes für eine negative Binomialverteilung mit einer zulässigen Anzahl von Fehlschlägen und der Erfolgswahrscheinlichkeit zurück.

int *r*

- Zulässige Anzahl von Fehlschlägen (mindestens 1)

float *p*

- Erfolgswahrscheinlichkeit

int *value*

- Anzahl der Erfolge, für die die Wahrscheinlichkeit zurückgegeben werden soll

::math::statistics::cdf-normal *mean stdev value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Normalverteilung mit gegebenem Mittelwert und gegebener Standardabweichung zurück, d. h. die Wahrscheinlichkeit für Werte bis einschließlich des angegebenen Wertes.

float *mean*

- Mittelwert der Verteilung

float *stdev*

- Standardabweichung der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-lognormal *mean stdev value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Lognormalverteilung mit gegebenem Mittelwert und gegebener Standardabweichung zurück, d. h. die Wahrscheinlichkeit für Werte bis einschließlich des angegebenen Wertes.

float *mean*

- Mittelwert der Verteilung

float *stdev*

- Standardabweichung der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-exponential *mean value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Exponentialverteilung mit einem vorgegebenen Mittelwert zurück.

float *mean*

- Mittelwert der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-uniform *xmin xmax value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Gleichverteilung mit vorgegebenen Extremwerten zurück.

float *xmin*

- Mindestwert der Verteilung

float *xmin*

- Maximaler Wert der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-triangular *xmin xmax value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Dreiecksverteilung mit vorgegebenen Extremwerten zurück. Wenn xmin < xmax gilt, weisen kleinere Werte eine höhere Wahrscheinlichkeit auf und umgekehrt; siehe auch *pdf-triangular*

float *xmin*

- Mindestwert der Verteilung

float *xmin*

- Maximaler Wert der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-symmetric-triangular *xmin xmax value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine symmetrische Dreiecksverteilung mit vorgegebenen Extremwerten zurück.

float *xmin*

- Mindestwert der Verteilung

float *xmin*

- Maximaler Wert der Verteilung

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-students-t *degrees value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Student-t-Verteilung mit einer vorgegebenen Anzahl von Freiheitsgraden zurück.

int *degrees*

- Anzahl der Freiheitsgrade

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-gamma *alpha beta value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Gamma-Verteilung mit vorgegebenen Form- und Ratenparametern zurück.

float *alpha*

- Formparameter

float *beta*

- Ratenparameter

float *value*

- Wert, für den die kumulative Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-poisson *mu k*

Gibt die kumulative Wahrscheinlichkeit einer bestimmten Anzahl von Ereignissen im gleichen Intervall (k) für eine Poisson-Verteilung mit einem gegebenen Mittelwert (mu) zurück.

float *mu*

- Durchschnittliche Anzahl der Vorkommen

int *k*

- Anzahl der Vorkommen

::math::statistics::cdf-beta *a b value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Beta-Verteilung mit vorgegebenen Formparametern zurück

float *a*

- Erster Formparameter

float *b*

- Zweiter Formparameter

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-weibull *scale shape value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Weibull-Verteilung mit vorgegebenen Skalen- und Formparametern zurück.

float *scale*

- Skalenparameter

float *shape*

- Formparameter

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-gumbel *location scale value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Gumbel-Verteilung mit vorgegebenen Lage- und Skalenparametern zurück.

float *location*

- Lageparameter

float *scale*

- Skalenparameter

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-pareto *scale shape value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Pareto-Verteilung mit vorgegebenen Skalen- und Formparametern zurück

float *scale*

- Skalenparameter

float *shape*

- Formparameter

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-cauchy *location scale value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Cauchy-Verteilung mit vorgegebenen Lage- und Skalenparametern zurück.

float *location*

- Lageparameter

float *scale*

- Skalenparameter

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-F *nf1 nf2 value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine F-Verteilung mit nf1 und nf2 Freiheitsgraden zurück.

float *nf1*

- Freiheitsgrade für den Zähler

float *nf2*

- Freiheitsgrade für den Nenner

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-laplace *location scale value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Laplace-Verteilung mit vorgegebenen Lage- und Formparametern zurück. Die Laplace-Verteilung setzt sich aus zwei Exponentialfunktionen zusammen, hat eine ausgeprägte Spitze und schwerere Ränder als die Normalverteilung.

float *location*

- Lageparameter (Mittelwert)

float *scale*

- Formparameter

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-kumaraswamy *a b value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine Kumaraswamy-Verteilung mit den vorgegebenen Parametern a und b zurück. Die Kumaraswamy-Verteilung ist mit der Beta-Verteilung verwandt, weist jedoch eine leicht zu berechnende kumulative Verteilungsfunktion auf.

float *a*

- Parameter a

float *b*

- Parameter b

float *value*

- Wert, für den die Wahrscheinlichkeit ermittelt werden soll

::math::statistics::cdf-negative-binomial *r p value*

Gibt die kumulative Wahrscheinlichkeit eines bestimmten Wertes für eine negative Binomialverteilung mit einer zulässigen Anzahl von Fehlschlägen und der Erfolgswahrscheinlichkeit zurück.

int *r*

- Zulässige Anzahl von Fehlschlägen (mindestens 1)

float *p*

- Erfolgswahrscheinlichkeit

int *value*
- Höchste Anzahl an Erfolgen

::math::statistics::empirical-distribution *values*

Gibt eine Liste mit Werten und deren empirischer Wahrscheinlichkeit zurück. Die Werte sind in aufsteigender Reihenfolge sortiert. (Die Implementierung folgt der Beschreibung auf der entsprechenden Wikipedia-Seite.)

list *values*
- Liste der zu prüfenden Daten

::math::statistics::random-normal *mean stdev number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Normalverteilung mit dem angegebenen Mittelwert und der angegebenen Standardabweichung entsprechen.

float *mean*
- Mittelwert der Verteilung

float *stdev*
- Standardabweichung der Verteilung

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-lognormal *mean stdev number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Lognormalverteilung mit einem vorgegebenen Mittelwert und einer vorgegebenen Standardabweichung entsprechen.

float *mean*
- Mittelwert der Verteilung

float *stdev*
- Standardabweichung der Verteilung

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-exponential *mean number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Exponentialverteilung mit einem vorgegebenen Mittelwert entsprechen.

float *mean*
- Mittelwert der Verteilung

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-uniform *xmin xmax number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Gleichverteilung mit den angegebenen Extremwerten entsprechen.

float *xmin*
- Mindestwert der Verteilung

float *xmax*
- Maximaler Wert der Verteilung

int *number*

- Anzahl der zurückzugebenden Werte

::math::statistics::random-triangular *xmin xmax number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Dreiecksverteilung mit den angegebenen Extremwerten entsprechen. Wenn xmin < xmax gilt, weisen niedrigere Werte eine höhere Wahrscheinlichkeit auf und umgekehrt (siehe auch *pdf-triangular*).

float *xmin*
- Mindestwert der Verteilung

float *xmax*
- Maximaler Wert der Verteilung

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-symmetric-triangular *xmin xmax number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer symmetrischen Dreiecksverteilung mit den angegebenen Extremwerten entsprechen.

float *xmin*
- Mindestwert der Verteilung

float *xmax*
- Maximaler Wert der Verteilung

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-gamma *alpha beta number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Gamma-Verteilung mit den angegebenen Form- und Ratenparametern entsprechen.

float *alpha*
- Formparameter

float *beta*
- Ratenparameter

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-poisson *mu number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Poisson-Verteilung mit einem vorgegebenen Mittelwert entsprechen.

float *mu*
- Mittelwert der Verteilung

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-chisquare *df number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Chi-Quadrat-Verteilung mit den angegebenen Freiheitsgraden entsprechen.

float *df*
- Freiheitsgrade

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-student-t *df number*
Gibt eine Liste mit "number" Zufallswerten zurück, die einer Student-t-Verteilung mit den angegebenen Freiheitsgraden entsprechen.
float *df*
- Freiheitsgrade

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-beta *a b number*
Gibt eine Liste mit "number" Zufallswerten zurück, die einer Beta-Verteilung mit den angegebenen Formparametern entsprechen.
float *a*
- Erster Formparameter

float *b*
- Zweiter Formparameter

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-weibull *scale shape number*
Gibt eine Liste mit "number" Zufallswerten zurück, die einer Weibull-Verteilung mit den angegebenen Skalen- und Formparametern entsprechen.
float *scale*
- Skalenparameter

float *shape*
- Formparameter

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-gumbel *location scale number*
Gibt eine Liste mit "number" Zufallswerten zurück, die einer Gumbel-Verteilung mit den angegebenen Lage- und Skalenparametern entsprechen.
float *location*
- Lageparameter

float *scale*
- Skalenparameter

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-pareto *scale shape number*
Gibt eine Liste mit "number" Zufallswerten zurück, die einer Pareto-Verteilung mit den angegebenen Skalen- und Formparametern entsprechen.
float *scale*
- Skalenparameter

float *shape*
- Formparameter

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-cauchy *location scale number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Cauchy-Verteilung mit den angegebenen Lage- und Skalenparametern entsprechen.

float *location*
- Lageparameter

float *scale*
- Skalenparameter

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-laplace *location scale number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Laplace-Verteilung mit vorgegebenen Lage- und Formparametern entsprechen. Die Laplace-Verteilung setzt sich aus zwei Exponentialfunktionen zusammen, hat eine ausgeprägte Spitze und schwerere Ränder als die Normalverteilung.

float *location*
- Lageparameter (Mittelwert)

float *scale*
- Formparameter

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-kumaraswamy *a b number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer Kumaraswamy-Verteilung mit den vorgegebenen Parametern a und b entsprechen. Die Kumaraswamy-Verteilung ist mit der Beta-Verteilung verwandt, weist jedoch eine leicht zu berechnende kumulative Verteilungsfunktion auf.

float *a*
- Parameter a

float *b*
- Parameter b

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::random-negative-binomial *r p number*

Gibt eine Liste mit "number" Zufallswerten zurück, die einer negativen Binomialverteilung entsprechen.

int *r*
- Zulässige Anzahl von Fehlschlägen (mindestens 1)

float *p*
- Erfolgswahrscheinlichkeit

int *number*
- Anzahl der zurückzugebenden Werte

::math::statistics::histogram-uniform *xmin xmax limits number*

Gibt das erwartete Histogramm für eine Gleichverteilung zurück.

float *xmin*
- Mindestwert der Verteilung

float *xmax*
- Maximaler Wert der Verteilung

list *limits*
- Obergrenzen für die Klassen im Histogramm

int *number*
- Gesamtzahl der "Beobachtungen" im Histogramm

::math::statistics::incompleteGamma *x p* ?tol?

Wertet das unvollständige Gamma-Integral aus

```
                  1       / x               p-1
     P(p,x) =  --------    |   dt exp(-t) * t
               Gamma(p)  / 0
```

float *x*
- Wert von x (Grenzwert des Integrals)

float *p*
- Wert von p im Integranden

float *tol*
- Erforderliche Toleranz (Standardwert: 1.0e-9)

::math::statistics::incompleteBeta *a b x* ?tol?

Wertet das unvollständige Beta-Integral aus

float *a*
- Erster Formparameter

float *b*
- Zweiter Formparameter

float *x*
- Wert von x (Grenzwert des Integrals)

float *tol*
- Erforderliche Toleranz (Standardwert: 1.0e-9)

::math::statistics::estimate-pareto *values*

Schätzt die Parameter der Pareto-Verteilung, die den angegebenen Werten am nächsten kommt. Gibt die geschätzten Skalen- und Formparameter sowie den Standardfehler für den Formparameter zurück.

list *values*
- Liste von Werten, von denen angenommen wird, dass sie einer Pareto-Verteilung folgen

::math::statistics::estimate-exponential *values*

Schätzt den Parameter der Exponentialverteilung, der den angegebenen Werten am nächsten kommt. Gibt eine Schätzung des einen Parameters sowie des Standardfehlers zurück.

list *values*

- Liste von Werten, von denen angenommen wird, dass sie einer Exponentialverteilung folgen

::math::statistics::estimate-laplace *values*

Schätzt die Parameter der Laplace-Verteilung, die den angegebenen Werten am nächsten kommt. Gibt eine Schätzung des Lage- bzw. des Skalenparameters zurück, basierend auf der Maximum-Likelihood-Methode.

list *values*

- Liste von Werten, von denen angenommen wird, dass sie einer Exponentialverteilung folgen

::math::statistics::estimante-negative-binomial *r values*

Schätzt die Erfolgswahrscheinlichkeit für die negative Binomialverteilung, die den angegebenen Werten am nächsten kommt. Die zulässige Anzahl von Fehlschlägen muss angegeben werden.

int *r*

- Zulässige Anzahl von Fehlschlägen (mindestens 1)

int *number*

- Liste von Werten, von denen angenommen wird, dass sie einer negativen Binomialverteilung folgen.

ZU TUN: Es müssen noch weitere Beschreibungen der Funktionen hinzugefügt werden

Datenbearbeitung

Die Datenbearbeitungsprozeduren wirken auf Listen oder Listen von Listen:

::math::statistics::filter *varname data expression*

Gibt eine Liste zurück, die aus den Daten besteht, für die der logische Ausdruck wahr ist (dieser Befehl funktioniert analog zum Befehl **foreach**).

string *varname*

- Name der in dem Ausdruck verwendeten Variablen

list *data*

- Liste der Daten

string *expression*

- Logischer Ausdruck unter Verwendung des Namens der Variablen

::math::statistics::map *varname data expression*

Gibt eine Liste zurück, die aus den Daten besteht, die mittels des Ausdrucks transformiert wurden.

string *varname*

- Name der in dem Ausdruck verwendeten Variablen

list *data*

- Liste der Daten

string *expression*

- Ausdruck, der zur Transformation (Abbildung) der Daten verwendet werden soll

::math::statistics::samplescount *varname list expression*
Gibt eine Liste zurück, die aus den Anzahlen (*counts*) aller Daten in den Unterlisten des Arguments "list" besteht, für die der Ausdruck wahr ist.

string *varname*
- Name der in dem Ausdruck verwendeten Variablen

list *data*
- Liste von Unterlisten, die jeweils die Daten enthalten

string *expression*
- Logischer Ausdruck zur Überprüfung der Daten (Standardwert: "true").

::math::statistics::subdivide
Routine *PM* – noch nicht implementiert

Prozeduren zur grafischen Darstellung

Es stehen die folgenden einfachen Prozeduren zur grafischen Darstellung zur Verfügung:

::math::statistics::plot-scale *canvas xmin xmax ymin ymax*
Legt den Maßstab für eine Darstellung im angegebenen Canvas fest. Alle Darstellungsroutinen erwarten, dass diese Funktion als Erstes aufgerufen wird. Eine automatische Skalierung ist nicht vorgesehen.

widget *canvas*
- Zu verwendendes Canvas-Widget

float *xmin*
- Mindestwert für x

float *xmax*
- Maximaler x-Wert

float *ymin*
- Mindestwert für y

float *ymax*
- Maximaler y-Wert

::math::statistics::plot-xydata *canvas xdata ydata tag*
Erstellt ein einfaches XY-Diagramm im angegebenen Canvas – die Daten werden als Punktwolke dargestellt. Mit dem Tag lässt sich das Erscheinungsbild anpassen.

widget *canvas*
- Zu verwendendes Canvas-Widget

float *xdata*
- Reihe unabhängiger Daten

float *ydata*
- Reihe abhängiger Daten

string *tag*
- Tag, der den dargestellten Daten zugewiesen werden soll (Standardwert: xyplot)

::math::statistics::plot-xyline *canvas xdata ydata tag*
Erstellt ein einfaches XY-Diagramm im angegebenen Canvas – die Daten werden als Linie

durch die Datenpunkte dargestellt. Mit dem Tag lässt sich das Erscheinungsbild anpassen.

widget *canvas*
- Zu verwendendes Canvas-Widget

list *xdata*
- Reihe unabhängiger Daten

list *ydata*
- Reihe abhängiger Daten

string *tag*
- Tag, der den dargestellten Daten zugewiesen werden soll (Standardwert: xyplot)

::math::statistics::plot-tdata *canvas tdata tag*

Erstellt ein einfaches XY-Diagramm im angegebenen Canvas – die Daten werden als Punktwolke dargestellt. Die horizontale Koordinate entspricht dem Index. Mit dem Tag lässt sich das Erscheinungsbild anpassen. Diese Darstellungsart eignet sich beispielsweise für Autokorrelationsfunktionen oder zur Untersuchung des zeitabhängigen Verhaltens.

widget *canvas*
- Zu verwendendes Canvas-Widget

list *tdata*
- Reihe abhängiger Daten

string *tag*
- Tag, der den dargestellten Daten zugewiesen werden soll (Standardwert: xyplot)

::math::statistics::plot-tline *canvas tdata tag*

Erstellt ein einfaches XY-Diagramm im angegebenen Canvas – die Daten werden als Linie dargestellt. Eine Erläuterung finden Sie bei plot-tdata.

widget *canvas*
- Zu verwendendes Canvas-Widget

list *tdata*
- Reihe abhängiger Daten

string *tag*
- Tag, der den dargestellten Daten zugewiesen werden soll (Standardwert: xyplot)

::math::statistics::plot-histogram *canvas counts limits tag*

Erstellt ein einfaches Histogramm im angegebenen Canvas

widget *canvas*
- Zu verwendendes Canvas-Widget

list *counts*
- Reihe der Häufigkeiten je Klasse

list *limits*
- Reihe von Obergrenzen für die Klassen

string *tag*
- Tag, der den dargestellten Daten zugewiesen werden soll (Standardwert: xyplot)

Noch zu tun

Die folgenden Prozeduren müssen noch umgesetzt werden:

- F-test-stdev
- interval-mean-stdev
- histogram-normal
- histogram-exponential
- test-histogram
- test-corr
- quantiles-*
- fourier-coeffs
- fourier-residuals
- onepar-function-fit
- onepar-function-residuals
- plot-linear-model
- subdivide

Beispiele

Der folgende Code ist ein kleines Beispiel dafür, wie Sie einen Datensatz untersuchen können:

```
# Simple example:
# - Generate data (as a cheap way of getting some)
# - Perform statistical analysis to describe the data
#
package require math::statistics

#
# Two auxiliary procs
#
proc pause {time} {
   set wait 0
   after [expr {$time*1000}] {set ::wait 1}
   vwait wait
}

proc print-histogram {counts limits} {
   foreach count $counts limit $limits {
      if { $limit != {} } {
         puts [format "<%12.4g\t%d" $limit $count]
         set prev_limit $limit
      } else {
         puts [format ">%12.4g\t%d" $prev_limit $count]
      }
   }
}

#
# Our source of arbitrary data
#
proc generateData { data1 data2 } {
```

```
   upvar 1 $data1 _data1
   upvar 1 $data2 _data2

   set d1 0.0
   set d2 0.0
   for { set i 0 } { $i < 100 } { incr i } {
      set d1 [expr {10.0-2.0*cos(2.0*3.1415926*$i/24.0)+3.5*rand()}]
      set d2 [expr {0.7*$d2+0.3*$d1+0.7*rand()}]
      lappend _data1 $d1
      lappend _data2 $d2
   }
   return {}
}

#
# The analysis session
#
package require Tk
console show
canvas .plot1
canvas .plot2
pack   .plot1 .plot2 -fill both -side top

generateData data1 data2

puts "Basic statistics:"
set b1 [::math::statistics::basic-stats $data1]
set b2 [::math::statistics::basic-stats $data2]
foreach label {mean min max number stdev var} v1 $b1 v2 $b2 {
   puts "$label\t$v1\t$v2"
}
puts "Plot the data as function of \"time\" and against each other"
::math::statistics::plot-scale .plot1  0 100  0 20
::math::statistics::plot-scale .plot2  0 20   0 20
::math::statistics::plot-tline .plot1 $data1
::math::statistics::plot-tline .plot1 $data2
::math::statistics::plot-xydata .plot2 $data1 $data2

puts "Correlation coefficient:"
puts [::math::statistics::corr $data1 $data2]

pause 2
puts "Plot histograms"
.plot2 delete all
::math::statistics::plot-scale .plot2  0 20 0 100
set limits         [::math::statistics::minmax-histogram-limits 7 16]
```

```
set histogram_data [::math::statistics::histogram $limits $data1]
::math::statistics::plot-histogram .plot2 $histogram_data $limits

puts "First series:"
print-histogram $histogram_data $limits

pause 2
set limits         [::math::statistics::minmax-histogram-limits 0 15
    10]
set histogram_data [::math::statistics::histogram $limits $data2]
::math::statistics::plot-histogram .plot2 $histogram_data $limits d2
.plot2 itemconfigure d2 -fill red

puts "Second series:"
print-histogram $histogram_data $limits

puts "Autocorrelation function:"
set  autoc [::math::statistics::autocorr $data1]
puts [::math::statistics::map $autoc {[format "%.2f" $x]}]
puts "Cross-correlation function:"
set  crossc [::math::statistics::crosscorr $data1 $data2]
puts [::math::statistics::map $crossc {[format "%.2f" $x]}]

::math::statistics::plot-scale .plot1  0 100 -1  4
::math::statistics::plot-tline .plot1  $autoc "autoc"
::math::statistics::plot-tline .plot1  $crossc "crossc"
.plot1 itemconfigure autoc  -fill green
.plot1 itemconfigure crossc -fill yellow

puts "Quantiles: 0.1, 0.2, 0.5, 0.8, 0.9"
puts "First:  [::math::statistics::quantiles $data1 {0.1 0.2 0.5 0.8
    0.9}]"
puts "Second: [::math::statistics::quantiles $data2 {0.1 0.2 0.5 0.8
    0.9}]"
```

Wenn Sie dieses Beispiel ausführen, sollte Folgendes deutlich werden:

- Zwischen den beiden Zeitreihen besteht eine starke Korrelation, wie aus den Rohdaten und insbesondere aus den Korrelationsfunktionen hervorgeht.
- Beide Zeitreihen weisen eine signifikante periodische Komponente auf
- Die Histogramme sind für die Bestimmung der Eigenschaften der Zeitreihe nicht besonders hilfreich – sie zeigen den periodischen Charakter nicht auf.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: statistics* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Mathematik

symdiff
Stichwörter: symdiff

Name
math::calculus::symdiff – Symbolische Differentiation für Tcl

Übersicht
package require Tcl 8.5 9
package require grammar::aycock 1.0
package require math::calculus::symdiff 1.0.2
math::calculus::symdiff::symdiff *expression variable*
math::calculus::jacobian *variableDict*

Beschreibung
Das Paket **math::calculus::symdiff** bietet eine Funktion zur symbolischen Differenzierung von Tcl-Mathematik-Ausdrücken. Es ist nützlich, um Ableitungen für Pakete bereitzustellen, die entweder die Jacobi-Matrix einer Funktionsmenge benötigen oder die effizienter bzw. stabiler arbeiten, wenn die Jacobi-Matrix bereitgestellt wird.

Prozeduren
Das Paket **math::calculus::symdiff** exportiert die beiden Prozeduren:

math::calculus::symdiff::symdiff *expression variable*
Differenziert den angegebenen *expression* nach der angegebenen *variable*. (Eine Erläuterung der Teilmenge der Tcl-Mathematikausdrücke, die von **math::calculus::symdiff** akzeptiert werden, finden Sie im Abschnitt Ausdrücke weiter unten.) Das Ergebnis ist ein Tcl-Ausdruck, der die Ableitung berechnet. Meldet einen Fehler, wenn *expression* kein wohlgeformter Ausdruck ist oder nicht differenzierbar ist.

math::calculus::jacobian *variableDict*
Berechnet die Jacobi-Matrix eines Gleichungssystems. Das System wird durch das Dictionary *variableDict* angegeben, dessen Schlüssel die Namen der Variablen im System sind und dessen Werte Tcl-Ausdrücke sind, die die Werte dieser Variablen angeben. (Eine Erläuterung der Teilmenge der Tcl-Mathematikausdrücke, die von **math::calculus::symdiff** akzeptiert werden, finden Sie im Abschnitt Ausdrücke weiter unten. Das Ergebnis ist eine Liste von Listen: Das i-te Element der j-ten Unterliste ist die partielle Ableitung der i-ten Variablen nach der j-ten Variablen. Meldet einen Fehler, wenn einer der Ausdrücke nicht differenzierbar ist oder wenn *variableDict* kein wohlgeformtes Dictionary ist.

Ausdrücke

Das Paket **math::calculus::symdiff** akzeptiert nur eine kleine Teilmenge der Ausdrücke, die für Tcl-Befehle wie **expr** oder **if** zulässig sind. Konkret werden ausschließlich folgende Konstrukte akzeptiert:

- Gleitkomma-Konstanten wie beispielsweise **5** oder **3.14159e+00**.
- Verweise auf Tcl-Variablen mit $-Ersetzung. Die Namen der Variablen müssen aus alphanumerischen Zeichen und Unterstrichen bestehen: Die Schreibweise **${...}** wird nicht akzeptiert.
- Klammern.
- Die Operatoren **+**, **-**, *****, **/** und ******.
- Aufrufe der Funktionen acos, asin, atan, **atan2**, cos, cosh, exp, **hypot**, **log**, **log10**, pow, sin, sinh. sqrt, tan und tanh.

Befehlsersetzung, Backslash-Ersetzung und Argumenterweiterung werden nicht akzeptiert.

Beispiele

```
math::calculus::symdiff::symdiff {($a*$x+$b)*($c*$x+$d)} x
==> (($c * (($a * $x) + $b)) + ($a * (($c * $x) + $d)))
math::calculus::symdiff::jacobian {x {$a * $x + $b * $y}
                           y {$c * $x + $d * $y}}
==> {{$a} {$b}} {{$c} {$d}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: calculus* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright


```
Copyright (c) 2010 by Kevin B. Kenny <kennykb@acm.org>
Redistribution permitted under the terms of the Open Publication
    License <http://www.opencontent.org/openpub/>
```


trig

Stichwörter: trig, math, trigonometry

Name

math::trig – Trigonometrische und hyperbolische Funktionen

Übersicht

package require Tcl 8.5 9
package require math::trig 1.1
::math::trig::radian_reduced *angle*
::math::trig::degree_reduced *angle*

::math::trig::cosec *angle*
::math::trig::sec *angle*
::math::trig::cotan *angle*
::math::trig::acosec *value*
::math::trig::asec *value*
::math::trig::acotan *value*
::math::trig::cosech *value*
::math::trig::sech *value*
::math::trig::cotanh *value*
::math::trig::asinh *value*
::math::trig::acosh *value*
::math::trig::atanh *value*
::math::trig::acosech *value*
::math::trig::asech *value*
::math::trig::acotanh *value*
::math::trig::sind *angle*
::math::trig::cosd *angle*
::math::trig::tand *angle*
::math::trig::cosecd *angle*
::math::trig::secd *angle*
::math::trig::cotand *angle*

Beschreibung

Das Paket *math::trig* definiert eine Reihe von trigonometrischen und hyperbolischen Funktionen sowie deren Umkehrfunktionen. Darüber hinaus definiert es Varianten der trigonometrischen Funktionen, deren Argumente in Grad statt im Bogenmaß angegeben werden.

Zur Vereinfachung der Verwendung können diese Funktionen in den Namespace *tcl::mathfunc* importiert werden, sodass sie direkt im Befehl *expr* verwendet werden können.

Funktionen

Die Funktionen *radian_reduced* und *degree_reduced* geben einen reduzierten Winkel zurück, und zwar im Bogenmaß bzw. in Grad, in den Intervallen [0, 2pi) und [0, 360):

::math::trig::radian_reduced *angle*
Gibt den entsprechenden Winkel im Intervall [0, 2pi) zurück.
float *angle*
Winkel (im Bogenmaß)

::math::trig::degree_reduced *angle*
Gibt den entsprechenden Winkel im Intervall [0, 360) zurück.
float *angle*
Winkel (in Grad)

Zusätzlich zu den im Befehl *expr* definierten trigonometrischen Funktionen sind die folgenden definiert:

::math::trig::cosec *angle*
Berechnet den Kosekans des Winkels (1/cos(angle))

float *angle*
Winkel (im Bogenmaß)

::math::trig::sec *angle*
Berechnet den Sekans des Winkels (1/sin(angle))
float *angle*
Winkel (im Bogenmaß)

::math::trig::cotan *angle*
Berechnet den Kotangens des Winkels (1/tan(angle))
float *angle*
Winkel (im Bogenmaß)

Auch für diese Funktionen sind die Umkehrfunktionen definiert:

::math::trig::acosec *value*
Berechnet den Arkuskosekans des Wertes
float *value*
Wert des Arguments

::math::trig::asec *value*
Berechnet den Arkussekans des Wertes
float *value*
Wert des Arguments

::math::trig::acotan *value*
Berechnet den Arkuskotangens des Wertes
float *value*
Wert des Arguments

Es sind die folgenden hyperbolischen und inversen hyperbolischen Funktionen definiert:

::math::trig::cosech *value*
Berechnet den hyperbolischen Kosekans des Wertes (1/sinh(value))
float *value*
Wert des Arguments

::math::trig::sech *value*
Berechnet den hyperbolischen Sekans des Wertes (1/cosh(value))
float *value*
Wert des Arguments

::math::trig::cotanh *value*
Berechnet den hyperbolischen Kotangens des Wertes (1/tanh(value))
float *value*
Wert des Arguments

::math::trig::asinh *value*
Berechnet den inversen hyperbolischen Sinus des Wertes

float *value*
Wert des Arguments

::math::trig::acosh *value*
Berechnet den inversen hyperbolischen Kosinus des Wertes
float *value*
Wert des Arguments

::math::trig::atanh *value*
Berechnet den inversen hyperbolischen Tangens des Wertes
float *value*
Wert des Arguments

::math::trig::acosech *value*
Berechnet den inversen hyperbolischen Kosekans des Wertes
float *value*
Wert des Arguments

::math::trig::asech *value*
Berechnet den inversen hyperbolischen Sekans des Wertes
float *value*
Wert des Arguments

::math::trig::acotanh *value*
Berechnet den inversen hyperbolischen Kotangens des Wertes
float *value*
Wert des Arguments

Es sind die folgenden Versionen der gängigen trigonometrischen Funktionen und ihrer Umkehrfunktionen definiert:

::math::trig::sind *angle*
Berechnet den Sinus des Winkels (in Grad)
float *angle*
Winkel (in Grad)

::math::trig::cosd *angle*
Berechnet den Kosinus des Winkels (in Grad)
float *angle*
Winkel (im Bogenmaß)

::math::trig::tand *angle*
Berechnet den Kotangens des Winkels (in Grad)
float *angle*
Winkel (in Grad)

::math::trig::cosecd *angle*
Berechnet den Kosekans des Winkels (in Grad)
float *angle*

Winkel (in Grad)

::math::trig::secd *angle*
Berechnet den Sekans des Winkels (in Grad)
float *angle*
Winkel (in Grad)

::math::trig::cotand *angle*
Berechnet den Kotangens des Winkels (in Grad)
float *angle*
Winkel (in Grad)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *math :: trig* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Mathematik

Copyright

```
Copyright (c) 2018 Arjen Markus
```

md4

md4

Stichwörter: md4, hashing, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name

md4 – MD4-Message-Digest-Algorithmus

Übersicht

package require Tcl 8.5 9
package require md4 ?**1.0.8**?
::md4::md4 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::md4::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::md4::MD4Init
::md4::MD4Update *token data*
::md4::MD4Final *token*
::md4::HMACInit *key*
::md4::HMACUpdate *token data*

::md4::HMACFinal *token*

Beschreibung

Dieses Paket ist eine Implementierung des MD4-Message-Digest-Algorithmus in Tcl, wie er in RFC 1320 (1) und (2) beschrieben ist. Dieser Algorithmus nimmt eine beliebige Datenmenge entgegen und generiert aus der Eingabe einen 128-Bit-Message-Digest. Der MD4-Algorithmus ist schneller, aber potenziell weniger sicher als der verwandte MD5-Algorithmus (3).

Wenn Sie über **critcl** verfügen und das Paket **tcllibc** erstellt haben, erfolgt die Implementierung der Hash-Funktion durch kompilierten Code. Alternativ wird, wenn **cryptkit** verfügbar ist, dieses verwendet. Kann kein Paket gefunden werden, kommt die reine TCL-Implementierung zum Einsatz. Die Programmierschnittstelle bleibt in allen Fällen unverändert.

ACHTUNG Die Befehle in diesem Paket erwarten Binärdaten als Eingabe. Wird ein **-file** angegeben, wird dies durch die Befehle selbst sichergestellt, da sie die referenzierte Datei im Binärmodus öffnen. Wenn stattdessen Literal-Daten oder ein **-channel** übergeben werden, ist der *Aufrufer des Befehls* dafür verantwortlich, dies sicherzustellen. Der erforderliche Konvertierungsbefehl lautet **encoding convertto utf-8 $string**.

ACHTUNG, hier liegt ein *Tcl-9-Kompatibilitätsproblem* vor.

Tcl 8.x behandelt nicht-binäre Eingaben stillschweigend falsch, indem es diese intern auf die gewünschte Länge kürzt. Das heißt, indem nur das niedrige Byte eines höheren Unicode-Punkts verwendet wird. Es wird kein Fehler ausgelöst, das *Ergebnis ist einfach falsch*.

Tcl 9 löst stattdessen einen Fehler aus, d. h.

```
expected byte sequence but character <location> was ...
```

Befehle

::md4::md4 ?*-hex*? [*-channel channel* | *-file filename* | *string*]

Berechnet den MD4-Hash der in der Zeichenkette angegebenen Daten. Dieser wird standardmäßig als Binärzeichenkette zurückgegeben. Bei Angabe der Option *-hex* gibt es eine hexadezimal kodierte Version des Hashs zurück.

Die zu hashenden Daten können entweder als Zeichenfolgenargument für den Befehl "md4", als Dateiname oder als zuvor geöffneter Kanal angegeben werden. Wenn das Argument *-filename* angegeben wird, wird die Datei geöffnet, die Daten werden gelesen und gehasht, und die Datei wird geschlossen. Wenn das Argument *-channel* angegeben wird, werden Daten aus dem Kanal bis zum Dateiende gelesen. Der Kanal wird nicht geschlossen.

Es sollte nur eines der Argumente *-file*, *-channel* oder *string* angegeben werden.

::md4::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]

Berechnet einen HMAC-Digest (Hashed Message Authentication Digest) unter Verwendung des MD4-Digest-Algorithmus. HMACs sind in RFC 2104 (4) beschrieben und liefern einen MD4-Digest, der einen Schlüssel enthält. Alle Optionen außer *-key* entsprechen denen des Befehls **::md4::md4**.

Programmierschnittstelle

Für den Programmierer lässt sich der MD4-Hash als ein Behälter betrachten, in den man Daten

einfüllt. Ist die Eingabe abgeschlossen, wird ein Wert extrahiert, der aus den in den Behälter eingefüllten Daten abgeleitet wird. Die Programmierschnittstelle zum MD4-Hash arbeitet mit einem Token (das dem Behälter entspricht). **MD4Init** liefert ein Token, **MD4Update** nimmt so oft wie erforderlich weitere Daten entgegen. **MD4Final** gibt die Ressourcen frei und liefert den Hash-Wert. Ein entsprechender Funktionssatz stellt Ihnen einen schlüsselbasierten Digest (HMAC) zur Verfügung.

::md4::MD4Init
: Leitet einen neuen MD4-Hash ein. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss.

::md4::MD4Update *token data*
: Fügt Daten zu dem durch das Token identifizierten Hash hinzu. Der Aufruf von *MD4Update $token* "*abcd*" entspricht dem Aufruf von *MD4Update $token* "*ab*", gefolgt von *MD4Update $token* "*cb*". Siehe Beispiele.

::md4::MD4Final *token*
: Gibt den Hash-Wert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, verliert das Token seine Gültigkeit. Das Ergebnis ist eine binäre Zeichenkette mit einer Länge von 16 Byte, die den 128-Bit-MD4-Digest-Wert darstellt.

::md4::HMACInit *key*
: Dies entspricht dem Befehl **::md4::MD4Init**, mit dem Unterschied, dass hierfür der Schlüssel angegeben werden muss, der in den HMAC einbezogen werden soll.

::md4::HMACUpdate *token data*
: siehe unten ...

::md4::HMACFinal *token*
: Diese Befehle entsprechen den entsprechenden MD4-Befehlen.

Beispiele

```
% md4::md4 -hex "Tcl does MD4"
858da9b31f57648a032230447bd15f25
```

```
% md4::hmac -hex -key Sekret "Tcl does MD4"
c324088e5752872689caedf2a0464758
```

```
% set tok [md4::MD4Init]
::md4::1
% md4::MD4Update $tok "Tcl "
% md4::MD4Update $tok "does "
% md4::MD4Update $tok "MD4"
% md4::Hex [md4::MD4Final $tok]
858da9b31f57648a032230447bd15f25
```

Literatur

1. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
2. Rivest, R., "The MD4 message digest algorithm", in: A.J. Menezes und S.A. Vanstone (Hrsg.), Advances in Cryptology – CRYPTO '90 Proceedings, Seiten 303–311, Springer-Verlag, 1991.

3. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT und RSA Data Security, Inc., April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
4. Krawczyk, H., Bellare, M. und Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, Februar 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *md4* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

md5

md5

Stichwörter: md5, hashing, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name

md5 – MD5-Message-Digest-Algorithmus

Übersicht

package require Tcl 8.5 9
package require md5 ?**2.0.9**?
::md5::md5 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::md5::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::md5::MD5Init
::md5::MD5Update *token data*
::md5::MD5Final *token*
::md5::HMACInit *key*
::md5::HMACUpdate *token data*
::md5::HMACFinal *token*

Beschreibung

Dieses Paket ist eine Implementierung des MD5-Message-Digest-Algorithmus in Tcl, wie er in RFC 1321 (1) beschrieben ist. Dieser Algorithmus nimmt eine beliebige Datenmenge entgegen und erzeugt aus der Eingabe einen 128-Bit-Message-Digest. Der MD5-Algorithmus ist mit dem MD4-Algorithmus (2) verwandt, wurde jedoch gegen bestimmte Arten potenzieller Angriffe abgesichert. Bei neuen Anwendungen sollte MD5 gegenüber MD4 bevorzugt werden.

Wenn Sie **critcl** installiert und das **tcllibc**-Paket kompiliert haben, erfolgt die Implementierung der Hash-Funktion durch kompilierten Code. Alternativ wird, wenn Sie entweder **cryptkit** oder **Trf** installiert haben, eines dieser Programme zur Beschleunigung der Digest-Berechnung verwendet. Ist kein geeignetes kompiliertes Paket verfügbar, kommt die reine Tcl-Implementierung zum Einsatz. Die Programmierschnittstelle bleibt in allen Fällen unverändert.

Hinweis: Die vorherige Version dieses Pakets gab stets eine hexadezimal kodierte Zeichenkette zurück. Dies wurde geändert, um die Programmierschnittstelle zu vereinfachen und die Kompatibilität dieser Version mit anderen Implementierungen zu verbessern. Um die bisherige Funktionsweise zu erhalten, geben Sie entweder explizit die Paketversion 1 an oder verwenden Sie die Option *-hex* für den Befehl **md5**.

Achtung Die Befehle in diesem Paket erwarten Binärdaten als Eingabe. Wenn ein **-file** angegeben wird, wird dies von den Befehlen selbst sichergestellt, da sie die referenzierte Datei im Binärmodus öffnen. Wenn stattdessen Literal-Daten oder ein **-channel** übergeben werden, ist der *Aufrufer des Befehls* dafür verantwortlich, dies sicherzustellen. Die erforderliche Kodierung des Konvertierungsbefehls lautet **encoding convertto utf-8 $string**.

Achtung, hier liegt ein *Tcl-9-Kompatibilitätsproblem* vor.

Tcl 8.x behandelt nicht-binäre Eingaben stillschweigend falsch, indem es diese intern auf die gewünschte Länge kürzt. Das heißt, indem nur das niedrige Byte eines höheren Unicode-Punkts verwendet wird. Es wird kein Fehler ausgelöst, das *Ergebnis ist einfach falsch*.

Tcl 9 löst stattdessen einen Fehler aus, d. h.

```
expected byte sequence but character <location> was ...
```

Befehle

::md5::md5 ?*-hex*? [*-channel channel* | *-file filename* | *string*]

Berechnet den MD5-Hash der in der Zeichenkette angegebenen Binärdaten. Dieser wird standardmäßig als Binärzeichenkette zurückgegeben. Bei Angabe der Option *-hex* gibt es eine hexadezimal kodierte Version des Hashs zurück.

Die zu hashenden Daten können entweder als Zeichenfolgenargument für den Befehl **md5**, als Dateiname oder als zuvor geöffneter Kanal angegeben werden. Wenn das Argument *-filename* angegeben wird, wird die Datei geöffnet, die Daten werden gelesen und gehasht und die Datei wird geschlossen. Wenn das Argument *-channel* angegeben wird, werden Daten aus dem Kanal bis zum Dateiende gelesen. Der Kanal wird nicht geschlossen.

Es sollte nur eines der Argumente *-file*, *-channel* oder *string* angegeben werden.

::md5::hmac ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]

Berechnet einen HMAC-Digest (Hashed Message Authentication Digest) unter Verwendung des MD5-Digest-Algorithmus. HMACs sind in RFC 2104 (3) beschrieben und liefern einen MD5-Digest, der einen Schlüssel enthält. Alle Optionen außer *-key* entsprechen denen des Befehls **::md5::md5**.

Programmierschnittstelle

Für den Programmierer lässt sich der MD5-Hash als ein Behälter betrachten, in den Sie Daten

einfüllen. Wenn Sie fertig sind, extrahieren Sie einen Wert, der aus den in den Behälter eingefüllten Daten abgeleitet wird. Die Programmierschnittstelle zum MD5-Hash arbeitet mit einem Token (das dem Behälter entspricht). Sie rufen **MD5Init** auf, um ein Token zu erhalten, und ruft anschließend **MD5Update** so oft wie nötig auf, um Daten zum Hash hinzuzufügen. Um Ressourcen freizugeben und den Wert des Hash-Werts zu erhalten, rufen Sie dann **MD5Final** auf. Eine entsprechende Funktionsgruppe liefert einen schlüsselgebundenen Digest (HMAC).

::md5::MD5Init
: Erzeugt einen neuen MD5-Hash. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss.

::md5::MD5Update *token data*
: Fügt Daten zu dem durch das Token identifizierten Hash hinzu. Der Aufruf von *MD5Update $token* "*abcd*" entspricht dem Aufruf von *MD5Update $token* "*ab*", gefolgt von *MD5Update $token* "*cb*". Siehe Beispiele.

::md5::MD5Final *token*
: Gibt den Hash-Wert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, ist das Token ungültig. Das Ergebnis ist eine binäre Zeichenkette mit einer Länge von 16 Byte, die den 128-Bit-MD5-Wert darstellt.

::md5::HMACInit *key*
: Dies entspricht dem Befehl **::md5::MD5Init**, mit dem Unterschied, dass hierfür der Schlüssel angegeben werden muss, der in den HMAC einbezogen werden soll.

::md5::HMACUpdate *token data*
: siehe unten ...

::md5::HMACFinal *token*
: Diese Befehle entsprechen den entsprechenden MD5-Befehlen.

Beispiele

```
% md5::md5 -hex "Tcl does MD5"
8AAC1EE01E20BB347104FABB90310433
```

```
% md5::hmac -hex -key Sekret "Tcl does MD5"
35BBA244FD56D3EDF5F3C47474DACB5D
```

```
% set tok [md5::MD5Init]
::md5::1
% md5::MD5Update $tok "Tcl "
% md5::MD5Update $tok "does "
% md5::MD5Update $tok "MD5"
% md5::Hex [md5::MD5Final $tok]
8AAC1EE01E20BB347104FABB90310433
```

Literatur

1. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT und RSA Data Security, Inc., April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)

3. Krawczyk, H., Bellare, M. und Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, Februar 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *md5* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

md5crypt

md5crypt

Stichwörter: md5crypt, hashing, md5, message-digest, security

Name

md5crypt – MD5-basierte Passwortverschlüsselung

Übersicht

package require Tcl 8.5 9
package require md5 2.0
package require md5crypt ?**1.2.0**?
::md5crypt::md5crypt *password salt*
::md5crypt::aprcrypt *password salt*
::md5crypt::salt ?*length*?

Beschreibung

Dieses Paket enthält eine Implementierung des Passwortverschlüsselungsalgorithmus "MD5-crypt", der ursprünglich von FreeBSD entwickelt wurde und derzeit in vielen modernen Systemen als Ersatz für die Unix-Funktion "crypt(3)" verwendet wird. Eine Implementierung des eng verwandten Apache MD5-crypt ist ebenfalls verfügbar. Die Ausgabe dieser Befehle ist kompatibel mit der BSD- und OpenSSL-Implementierung von md5crypt sowie mit dem Programm "htpasswd" von Apache 2.

Befehle

::md5crypt::md5crypt *password salt*

Erzeugt einen BSD-kompatiblen, MD5-verschlüsselten Passwort-Hash aus dem Klartext-Passwort und einem zufälligen Salt (siehe SALT).

::md5crypt::aprcrypt *password salt*
Erzeugt einen Apache-kompatiblen, MD5-verschlüsselten Passwort-Hash aus dem Klartext-Passwort und einem zufälligen Salt (siehe SALT).

::md5crypt::salt ?*length*?
Generiert eine zufällige Salt-Zeichenkette, die für die Verwendung mit den Befehlen **md5crypt** und **aprcrypt** geeignet ist.

Salt

Das an eines der hier implementierten Verschlüsselungsverfahren übergebene Salt wird daraufhin überprüft, ob es mit der Magic-String des jeweiligen Verschlüsselungsverfahrens beginnt (entweder "1" für MD5-crypt oder "$apr1$" für Apache crypt). Wenn dies der Fall ist, wird dieser Teil entfernt. Die verbleibenden Zeichen bis zum nächsten $ und bis zu einer maximalen Länge von 8 Zeichen werden dann als Salt verwendet. Der Salt-Text sollte wahrscheinlich auf die Menge der alphanumerischen ASCII-Zeichen sowie "./" (Punkt und Schrägstrich) beschränkt werden – dies dient der Gewährleistung einer maximalen Kompatibilität mit dem Unix-Passwortdateiformat.

Wenn ein Passwort generiert wird, anstatt es aus einer Passwortdatei abzurufen, kann der Befehl **salt** verwendet werden, um ein zufälliges Salt zu generieren.

Beispiele

```
% md5crypt::md5crypt password 01234567
$1$01234567$b5lh2mHyD2PdJjFfALlEz1
```

```
% md5crypt::aprcrypt password 01234567
$apr1$01234567$IXBaQywhAhc0d75ZbaSDp/
```

```
% md5crypt::md5crypt password [md5crypt::salt]
$1$dFmvyRmO$T.V3OmzqeEf3hqJp2WFcb.
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *md5crypt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```

mime

mime

Stichwörter: mime, email, internet, mail, net, rfc 2045, rfc 2046, rfc 2049, rfc 821, rfc 822, smtp

Name

mime – Bearbeitung von MIME-Body-Teilen

Übersicht

package require Tcl 8.5 9
package require mime ?**1.7.2**?
::mime::initialize ?*-canonical type/subtype ?-param {key value}...*? ?*-encoding value*? ?*-header {key value}...*? (**-file** *name* | **-string** *value* | **-parts** *{token1 ... tokenN}*)
::mime::finalize *token* ?*-subordinates all | dynamic | none*?
::mime::getproperty *token* ?*property | -names*?
::mime::getheader *token* ?*key | -names*?
::mime::setheader *token key value* ?*-mode write | append | delete*?
::mime::getbody *token* ?**-decode**? ?*-command callback -blocksize octets*?
::mime::copymessage *token channel*
::mime::buildmessage *token*
::mime::parseaddress *string*
::mime::parsedatetime (*string* | **-now**) *property*
::mime::mapencoding *encoding_name*
::mime::reversemapencoding *charset_type*

Beschreibung

Das Bibliothekspaket **mime** stellt Befehle zum Erstellen und Bearbeiten von MIME-Body-Teilen bereit.

::mime::initialize ?-canonical *type/subtype* ?-param {*key value*}...? ?-encoding *value*? ?-header {*key value*}...?? (-file *name* | -string *value* | -parts {*token1 ... tokenN*})

Erstellt einen MIME-Teil und gibt ein Token zurück, das diesen repräsentiert.

- Wenn die Option **-canonical** vorhanden ist, liegt der Text in kanonischer (Roh-)Form vor und kann durch Abfrage der Option **-file**, **-string** oder **-parts** ermittelt.

  Darüber hinaus können sowohl die Optionen **-param** als auch **-header** Optionen nullmal oder mehrmals vorkommen, um **Content-Type**-Parameter (z. B. **charset**) bzw. Header-Schlüsselwörter und -Werte (z. B. **Content-Disposition**) festzulegen.

  Außerdem legt **-encoding**, sofern vorhanden, die **Content-Transfer-Encoding** beim Kopieren des Hauptteils fest.
- Wenn die Option **-canonical** nicht vorhanden ist, wird der in der Option **-file** oder **-string** enthaltene MIME-Teil analysiert, wobei gegebenenfalls dynamisch untergeordnete Elemente generiert werden.

::mime::finalize *token* ?-subordinates all | dynamic | none?

Löscht den durch *token* dargestellten MIME-Teil. Die Funktion gibt eine leere Zeichenkette zurück.

Wenn die Option **-subordinates** vorhanden ist, legt sie fest, welche untergeordneten

Elemente ebenfalls gelöscht werden sollen. Der Standardwert ist **dynamic**; dabei werden alle untergeordneten Elemente, die durch **::mime::initialize** erstellt wurden, zusammen mit dem sie enthaltenden Container gelöscht.

::mime::getproperty *token* ?*property* | -names?

Gibt eine Zeichenkette oder eine Liste von Zeichenketten zurück, die die Eigenschaften eines MIME-Teils enthalten. Wenn der Befehl mit dem Namen einer bestimmten Eigenschaft aufgerufen wird, gibt er den entsprechenden Wert zurück; wenn stattdessen **-names** gelegt wird, gibt er eine Liste aller Eigenschaften zurück; andernfalls gibt er ein serialisiertes Array aus Eigenschaften und Werten zurück.

Die möglichen Eigenschaften sind:

content
Der Typ/Subtyp, der den Inhalt beschreibt

encoding
Der "Content-Transfer-Encoding"-Header

params
Eine Liste der "Content-Type"-Parameter

parts
Eine Liste von Tokens für die untergeordneten Elemente des Teils. Diese Eigenschaft ist nur vorhanden, wenn der MIME-Teil untergeordnete Elemente enthält.

size
Die ungefähre Größe des Inhalts (unverschlüsselt)

::mime::getheader *token* ?*key* | -names?

Gibt den Header eines MIME-Teils als Dictionary mit möglicherweise redundanten Schlüsseln zurück.

Wird *key* angegeben, wird eine Liste der Werte mit übereinstimmenden Namen (ohne Berücksichtigung der Groß-/Kleinschreibung) zurückgegeben.

Wird **-names** angegeben, wird eine Liste aller Schlüssel zurückgegeben.

::mime::setheader *token key value* ?-mode write | append | delete?

Wenn **append** angegeben wird, wird ein neuer Header mit dem Name *key* erstellt und der Wert *value* hinzugefügt. Wenn **write** angegeben wird, werden alle vorhandenen Header, deren Name mit *key* übereinstimmt, gelöscht und anschließend ein neuer Header mit dem Name *key* und dem Wert *value* erstellt. Wird **delete** angegeben, werden alle vorhandenen Header, deren Name mit *key* übereinstimmt, gelöscht. Gibt eine Liste von Zeichenketten zurück, die den zuvor mit dem Schlüssel verknüpften Wert enthalten.

Der Wert für **-mode** ist einer der folgenden:

write
Der *Schlüssel*/*Wert* wird entweder angelegt oder überschrieben (Standard).

append
Dem *Schlüssel* wird ein neuer *Wert* angehängt (der bei Bedarf angelegt wird).

delete
Alle mit dem Schlüssel verknüpften Werte werden entfernt (der Parameter *value* wird ignoriert).

::mime::getbody *token* ?-decode? ?-command *callback* ?-blocksize *octets*??

Gibt eine Zeichenkette zurück, die den Hauptteil des durch *token* dargestellten MIME-Teils in kanonischer Form enthält.

Wenn die Option **-command** gesetzt ist, wird die Funktion wiederholt mit jeweils einem Fragment des Hauptteils wie folgt aufgerufen:

```
uplevel #0 $callback [list "data" $fragment]
```

(Die Option **-blocksize**, wenn vorhanden, legt die maximale Größe jedes an die Rückruffunktion übergebenen Fragments fest.)

Wenn das Ende des Hauptteils erreicht ist, wird die Rückruffunktion wie folgt aufgerufen:

```
    uplevel #0 $callback "end"
```

Alternativ wird bei Auftreten eines Fehlers die Rückruffunktion wie folgt aufgerufen:

```
    uplevel #0 $callback [list "error" reason]
```

Unabhängig davon wird der Wert des letzten Aufrufs der Rückruffunktion durch **::mime::getbody** nach oben weitergegeben.

Wenn die Option **-command** fehlt, gibt **::mime::getbody** einen Wert zurück, der eine Zeichenkette enthält, die den gesamten Textkörper des MIME-Teils enthält.

Wenn die Option **-decode** fehlt, wird der oben berechnete Wert zurückgegeben, ohne dass er verändert wird. Das bedeutet, dass er in dem für das Token angegebenen Zeichensatz vorliegt und nicht im üblichen UTF-8. Wenn die Option **-decode** jedoch vorhanden ist, verwendet der Befehl die mit dem Token verknüpften Zeichensatzinformationen, um die Zeichenkette vor der Rückgabe aus ihrer ursprünglichen Kodierung in UTF-8 zu konvertieren.

::mime::copymessage *token channel*

Kopiert den durch den Teil *token* dargestellten MIME-Inhalt in den angegebenen *Kanal*. Der Befehl wird synchron ausgeführt und nutzt "fileevent", damit asynchrone Vorgänge unabhängig davon fortgesetzt werden können. Er gibt eine leere Zeichenkette zurück.

::mime::buildmessage *token*

Gibt den durch *token* dargestellten MIME-Teil als Zeichenkette zurück. Dies ähnelt **::mime::copymessage**, nur dass die Daten als Rückgabekette zurückgegeben werden, anstatt in einen Kanal geschrieben zu werden.

::mime::parseaddress *string*

Nimmt eine Zeichenkette entgegen, die eine oder mehrere Adressangaben im 822-Format enthält, und gibt eine Liste serialisierter Arrays zurück, wobei jedes Element einer der im Argument festgelegten Adressen entspricht. Wenn die Zeichenkette mehr als eine Adresse enthält, werden diese durch Kommas getrennt.

Jedes serialisierte Array enthält die unten aufgeführten Eigenschaften. Beachten Sie, dass eine oder mehrere dieser Eigenschaften leer sein können.

address

local@domain

comment

Kommentar im Stil von 822

domain
der Domain-Teil (rhs)

error
nicht leer bei einem Parsing-Fehler

group
Mit dieser Adresse beginnt eine Gruppe

friendly
benutzerfreundliche Darstellung

local
der lokale Teil (lhs)

memberP
Diese Adresse gehört zu einer Gruppe.

phrase
der Satzteil

proper
Adressangabe im 822-Format

route
Routenspezifikation im 822-Stil (obsolet)

::mime::parsedatetime (*string* | -now) *property*

Nimmt eine Zeichenkette entgegen, die eine Datums- und Zeitangabe im 822-Format enthält, und gibt die angegebene Eigenschaft als serialisiertes Array zurück.

Die Liste der Eigenschaften und ihrer Wertebereiche lautet wie folgt:

hour
0.. 23

lmonth
Januar, Februar, ..., Dezember

lweekday
Sonntag, Montag, ... Samstag

mday
1.. 31

min
0.. 59

mon
1.. 12

month
Jan, Feb, ..., Dez

proper
Datums- und Zeitangabe im 822-Format

rclock
die seit damals bis heute verstrichenen Sekunden

sec
0.. 59

wday
0.. 6 (So.. Mo)

weekday
So, Mo, ..., Sa

yday
1.. 366

year
1900 ...

zone
-720.. 720 (Minuten östlich von GMT)

::mime::mapencoding *encoding_name*
Maps ordnet TCL-Kodierungen den für ihren MIME-Zeichensatztyp entsprechenden Name zu. Dies geschieht nur für Kodierungen, deren Zeichensatztypen bekannt waren. Die übrigen Kodierungen geben vorerst "" zurück.

::mime::reversemapencoding *charset_type*
Ordnet MIME-Zeichensatztypen den Tcl-Kodierungsnamen zu. Unbekannte Typen geben "" zurück.

Bekannte Fehler

Tcllib Bug #447037

Dieses Problem betrifft nur Nutzer, die Tcl und Mime auf einem 64-Bit-System verwenden. Die derzeit empfohlene Lösung für dieses Problem ist ein Upgrade auf Tcl Version 8.4. Diese Version bietet erweiterte 64-Bit-Unterstützung, und der Fehler tritt nicht mehr auf.

Das Problem hätte grundsätzlich gelöst werden können, indem für dieses Paket die Verwendung von Tcl 8.4 vorgeschrieben worden wäre. Wir haben uns jedoch gegen diese Lösung entschieden, da sie eine große Anzahl nicht betroffener Nutzer dazu zwingen würde, ihren Tcl-Interpreter ohne triftigen Grund zu aktualisieren.

Weitere Informationen finden Sie unter *Ticket 447037* Name [/tktview?Name=447037].

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *mime* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright

```
Copyright (c) 1999-2000 Marshall T. Rose
```

smtp

Stichwörter: smtp, email, internet, mail, mime, net, rfc 2554, rfc 2821, rfc 3207, rfc 821, rfc 822, tls

Name

smtp – Clientseitige TCL-Implementierung des SMTP-Protokolls

Übersicht

package require Tcl
package require mime ?**1.5.4**?
package require smtp ?**1.5.2**?
::smtp::sendmessage *token option...*

Beschreibung

Die Bibliothek **smtp** stellt die Client-Seite des Simple Mail Transfer Protocol (SMTP) bereit (1) (2).

::smtp::sendmessage *token option*...

Dieser Befehl sendet den durch *token* dargestellten MIME-Teil (siehe Paket **mime**) an einen SMTP-Server. *options* ist eine Liste von Optionen und den dazugehörigen Werten. Die folgenden Optionen werden unterstützt:

-servers
Eine Liste von SMTP-Servern. Der Standardwert ist **localhost**.

Wenn mehrere Server festgelegt sind, werden diese nacheinander ausprobiert. Hinweis: Die **-ports**-Einträge werden parallel zu den Servern durchlaufen. Wenn die Ports für die Anzahl der Server nicht ausreichen, wird der Standardport (siehe unten) verwendet. Wenn mehr Ports als Server vorhanden sind, werden die überzähligen Ports ignoriert.

-ports
Eine Liste von SMTP-Ports. Der Standardwert ist **25**.

Informationen zum Verhalten bei Angabe mehrerer Server und Ports finden Sie oben unter der Option **-servers**.

-client
Der Name, der als Hostname für die Verbindung zum Server verwendet werden soll. Standardmäßig ist dies entweder "localhost", wenn einer der Server "localhost" ist, oder die von **info hostname** zurückgegebene Zeichenfolge.

-queue
Gibt an, dass der SMTP-Server aufgefordert werden soll, die Nachricht zur späteren Verarbeitung in die Warteschlange zu stellen. Ein boolescher Wert.

-atleastone
Gibt an, dass der SMTP-Server mindestens einen geeigneten Empfänger finden muss, damit die Nachricht gesendet werden kann. Ein boolescher Wert.

-originator
Eine Zeichenkette, die eine Adressangabe im 822-Format enthält. Wenn diese vorhanden

ist, wird der Header nicht auf eine Absenderadresse überprüft.

-recipients
Eine Zeichenkette, die eine oder mehrere Adressangaben im 822-Format enthält. Wenn diese Zeichenkette vorhanden ist, wird der Header nicht auf Empfängeradressen überprüft. Enthält die Zeichenkette mehr als eine Adresse, werden diese durch Kommas getrennt.

-header
Eine Liste mit zwei Elementen: einem SMTP-Header und dem dazugehörigen Wert (die Option "-header" kann nullmal oder mehrmals vorkommen).

-usetls
Dieses Paket unterstützt standardmäßig die TLS-Erweiterung gemäß RFC 3207 (3), sofern das Paket "tls" verfügbar ist. Sie können diese Funktion mit dieser booleschen Option deaktivieren.

-tlsimport
Dieses boolesche Flag ist standardmäßig auf **false** gesetzt. Wenn dieses Flag gesetzt ist, importiert das Paket TLS bei einem erfolgreich geöffneten Kanal. Dies ist für Verbindungen erforderlich, die anstelle von **STARTTLS** eine native TLS-Aushandlung verwenden. Das **tls** -Paket wird bei Bedarf automatisch einbezogen.

-tlspolicy
Mit dieser Option können Sie einen Befehl festlegen, der aufgerufen werden soll, wenn während der TLS-Einrichtung ein Fehler auftritt. Der Befehl wird aufgerufen, wobei der SMTP-Code und die Diagnosemeldung angehängt werden. Der Befehl sollte "secure" oder "insecure" zurückgeben, wobei "insecure" dazu führt, dass das Paket über den unverschlüsselten Kanal weitergeleitet wird. Bei der Rückgabe von "secure" wird der Socket geschlossen und der nächste Server in der Liste **-servers** versucht.

-username
siehe unten ...

-password
Wenn Ihr SMTP-Server vor der Annahme von E-Mails eine Authentifizierung (RFC 2554 (4)) verlangt, können Sie **-username** und **-password** verwenden, um dem Server Ihre Authentifizierungsdaten zu übermitteln. Derzeit unterstützt dieses Paket die Authentifizierungsmethoden DIGEST-MD5, CRAM-MD5, LOGIN und PLAIN. Zunächst wird die sicherste Methode ausprobiert, anschließend werden die Methoden nacheinander durchprobiert, bis entweder die Autorisierung erfolgt ist oder keine Methoden mehr übrig sind. Beachten Sie, dass die Autorisierung, wenn der Server eine TLS-Verbindung zulässt, erst erfolgt, nachdem wir begonnen haben, den sicheren Kanal zu nutzen.

Bitte lesen Sie auch den Abschnitt Authentifizierung; dort werden die notwendigen Voraussetzungen, d. h. die zur Unterstützung dieser Optionen und der Authentifizierung erforderlichen Pakete, detailliert beschrieben.

Wenn die Option **-originator** nicht vorhanden ist, wird die Absenderadresse aus dem Feld **From** (oder **Resent-From**) übernommen; ebenso werden, wenn die Option **-recipients** nicht vorhanden ist, die Empfängeradressen aus den Feldern **To**, **cc** und **Bcc** (bzw. **Resent-To** usw.) übernommen. Beachten Sie, dass die von der Option **-header** bereitgestellten Header-Schlüssel/Werte (nicht die im MIME-Teil vorhandenen) herangezogen werden.

Unabhängig davon werden der ausgehenden Nachricht bei Bedarf Header-Schlüssel/Werte hinzugefügt, um sicherzustellen, dass eine gültige Nachricht im 822-Format versendet wird.

Der Befehl gibt eine Liste zurück, aus der hervorgeht, welche Empfänger für den SMTP-Server unzulässig waren. Jedes Element der Liste ist eine weitere Liste, die die Adresse, einen SMTP-Fehlercode und eine textuelle Fehlermeldung enthält. Je nach der Option **-atleastone** und den vorgesehenen Empfängern kann eine nicht leere Liste dennoch bedeuten, dass die Nachricht vom Server akzeptiert wurde.

Authentifizierung

Achtung: Die SMTP-Authentifizierung nutzt **SASL**. Das heißt, wenn der Benutzer eine Verbindung authentifizieren muss, d. h. die Optionen **-user** und **-password** verwenden muss (siehe oben), muss das **sasl**-Paket verfügbar sein, damit **smtp** es laden kann.

Dies ist eine weiche Abhängigkeit, da nicht jeder eine Authentifizierung benötigt und **sasl** von vielen kryptografischen (sicheren) Hash-Funktionen abhängt, d. h. von allen folgenden: **md5**, **otp**, **md4**, **sha1** und **ripemd160**.

Beispiel

```
proc send_simple_message {recipient email_server subject body} {
    package require smtp
    package require mime

    set token [mime::initialize -canonical text/plain \
    -string $body]
    mime::setheader $token Subject $subject
    smtp::sendmessage $token \
    -recipients $recipient -servers $email_server
    mime::finalize $token
}

send_simple_message someone@somewhere.com localhost \
    "This is the subject." "This is the message."
```

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Literatur

1. Jonathan B. Postel, "SIMPLE MAIL TRANSFER PROTOCOL", RFC 821, August 1982. (http://www.rfc-editor.org/rfc/rfc821.txt)
2. J. Klensin, "Simple Mail Transfer Protocol", RFC 2821, April 2001. (http://www.rfc-editor.org/rfc/rfc2821.txt)
3. P. Hoffman, "SMTP-Dienst-Erweiterung für sicheres SMTP über Transport Layer Security", RFC 3207, Februar 2002. (http://www.rfc-editor.org/rfc/rfc3207.txt)
4. J. Myers, "SMTP-Erweiterung für Authentifizierung", RFC 2554, März 1999. (http://www.rfc-editor.org/rfc/rfc2554.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *smtp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

```
Copyright (c) 1999-2000 Marshall T. Rose and others
```

mkdoc

mkdoc

Stichwörter: mkdoc

Name

mkdoc – Extrahiert Markdown-Kommentare aus dem Quellcode und konvertiert diese optional in HTML

Übersicht

package require Tcl 8.6 9
package require Markdown ?**1.2.2**?
package require yaml ?**0.4.2**?
package require mkdoc ?**0.7.2**?

package require hook
::mkdoc::mkdoc *infile outfile* ?*-css cssfile*?
::mkdoc::run *infile*

Beschreibung

Das Paket **mkdoc** stellt einen Befehl bereit, mit dem in den Code eingebettete Dokumentation extrahiert und diese Kommentare optional in HTML konvertiert werden können. Letzteres erfolgt unter Verwendung des Tcllib-Pakets **Markdown**. Jede Zeile der eingebetteten Dokumentation beginnt mit dem speziellen Kommentar-Marker **#'**.

::mkdoc::mkdoc *infile outfile* ?-css *cssfile*?

Der Befehl liest die angegebene *infile* und extrahiert die Codekommentare, die der Marker **#'** einleitet. Ist *outfile* eine Datei "*.html*" oder "*.htm*", wird das Markdown mit einem Vorgabestil oder mit dem angegebenen Stylesheet *cssfile* nach HTML umgewandelt.

Alle Argumente sind Pfade der Dateien, aus denen gelesen oder in die geschrieben wird.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die unterstützte Markdown-Syntax und ihre Erweiterungen stehen im Abschnitt Formatierung.

::mkdoc::run *infile*

Der Befehl liest die angegebene *infile*, extrahiert die eingebettete Dokumentation und führt dann den Inhalt des ersten Beispiels aus, also des in ``` gesetzten Blocks im Abschnitt **Example**.

Hier ist ein solches Beispiel, das der Tcl-Interpreter ausführt

```
    #' ## <a name="example">Example</a>
    #'
    #' ```
    #' puts "Hello mkdoc package"
    #' puts "I am in the example section"
    #' ```
```

GEFAHR, VORSICHT. Lässt sich die *infile* nicht öffnen, *beendet* der Befehl den ganzen Prozess.

Von der Verwendung dieses Befehls im allgemeinen Kontext wird abgeraten.

Beispiele

Das folgende Beispiel veranschaulicht die Konvertierung der Dokumentation, die in der Datei "*mkdoc.tcl*" selbst eingebettet ist:

```
    package require mkdoc
    # extracting the Markdown
    mkdoc::mkdoc mkdoc.tcl mkdoc.md
    # converting Markdown to HTML
    mkdoc::mkdoc mkdoc.md mkdoc.html
    # direct conversion without intermediate file
    mkdoc::mkdoc mkdoc.tcl mkdoc.html
```

Formatierung

Das Paket unterstützt die Syntax, die das Tcllib-Paket **Markdown** unterstützt.

Darüber hinaus unterstützt es eine Reihe einfacher YAML-Header, deren Informationen in entsprechende HTML-**meta**-Tags eingefügt werden. Die unterstützten Schlüssel sind

author
: Setzt den Autor des Dokuments. Die Vorgabe ist **NN**.

title
: Setzt den Dokumenttitel. Die Vorgabe ist **Documentation filename**.

date
: Setzt das Dokumentdatum. Die Vorgabe ist der aktuelle Tag.

css
: Setzt ein eigenes CSS-Stylesheet. Die Vorgabe ist das interne Stylesheet von mkdoc.

Hinweis: Im Ausgabemodus Markdown werden diese Kopfzeilen einfach in das Ergebnis durchgereicht. Das ist richtig, denn Verarbeiter wie **pandoc** können sie ebenfalls nutzen.

Die Syntax zeigt das Beispiel unten:

```
    #' ---
    #' title: mkdoc::mkdoc 0.7.1
    #' author: Detlef Groth, Schwielowsee, Germany
    #' date: 2022-04-17
    #' css: mini.css
    #' ---
    #'
```

Eine weitere Erweiterung gegenüber dem üblichen Markdown ist die Unterstützung von Includes auf einer einzigen Ebene.

Die Syntax zeigt das Beispiel unten:

```
    #' #include "path/to/include/file"
```

Hinweis, die doppelten Anführungszeichen um den Pfad gehören zur Syntax.

Vorsicht außerdem, dass relative Pfade relativ zum aktuellen Arbeitsverzeichnis aufgelöst werden und *nicht* relativ zum Ort der einbindenden Datei.

Fehler, Ideen, Rückmeldungen

Dieses Dokument und das darin beschriebene Paket enthalten zweifellos Fehler und andere Probleme. Bitte melden Sie diese dem Autor des Pakets. Bitte teilen Sie ihm auch Ihre Ideen für Verbesserungen am Paket oder an der Dokumentation mit.

Copyright am Code

Art der BSD-Lizenz:

Die folgenden Bedingungen gelten für alle Dateien, die mit der Software in Zusammenhang stehen, sofern in einzelnen Dateien nicht ausdrücklich etwas anderes angegeben ist.

Die Autoren erteilen hiermit die Erlaubnis, diese Software und ihre Dokumentation für jeden beliebigen Zweck zu nutzen, zu vervielfältigen, zu ändern, zu verbreiten und zu lizenzieren,

vorausgesetzt, dass die vorhandenen Copyright-Hinweise in allen Kopien beibehalten werden und dass dieser Hinweis bei jeder Weitergabe wörtlich übernommen wird. Für keine der zulässigen Verwendungszwecke ist eine schriftliche Vereinbarung, eine Lizenz oder die Zahlung von Lizenzgebühren erforderlich. Änderungen an dieser Software können durch ihre Autoren urheberrechtlich geschützt sein und müssen nicht den hier beschriebenen Lizenzbedingungen entsprechen, sofern die neuen Bedingungen auf der ersten Seite jeder Datei, für die sie gelten, deutlich angegeben sind.

In keinem Fall haften die Autoren oder Vertreiber gegenüber Dritten für direkte, indirekte, besondere, zufällige oder Folgeschäden, die sich aus der Nutzung dieser Software, ihrer Dokumentation oder daraus abgeleiteter Werke ergeben, selbst wenn die Autoren auf die Möglichkeit solcher Schäden hingewiesen wurden.

Die Autoren und Vertreiber schließen ausdrücklich jegliche Gewährleistung aus, einschließlich, aber nicht beschränkt auf die stillschweigenden Gewährleistungen der Marktgängigkeit, der Eignung für einen bestimmten Zweck und der Nichtverletzung von Rechten Dritter. Diese Software wird "wie besehen" bereitgestellt, und die Autoren und Vertreiber sind nicht verpflichtet, Wartung, Support, Updates, Erweiterungen oder Modifikationen bereitzustellen.

EINGESCHRÄNKTE RECHTE: Die Nutzung, Vervielfältigung oder Offenlegung durch die Regierung unterliegt den Einschränkungen, die in Unterabsatz (c) (1) (ii) der Klausel über Rechte an technischen Daten und Computersoftware gemäß DFARS 252.227-7013 und FAR 52.227-19 festgelegt sind.

Kategorie
Textverarbeitung

Copyright
```
Copyright (c) 2019-2024, Detlef Groth <detlef(at)dgroth(dot)de>
```

multiplexer

multiplexer
Stichwörter: multiplexer, chat

Name
multiplexer – Eins-zu-Viele-Kommunikation mit Sockets.

Übersicht
package require Tcl 8.5 9
package require logger
package require multiplexer ?**0.3**?
::multiplexer::create
${multiplexer_instance}::Init *port*
${multiplexer_instance}::Config *key value*
${multiplexer_instance}::AddFilter *cmdprefix*
cmdprefix *data chan clientaddress clientport*
${multiplexer_instance}::AddAccessFilter *cmdprefix*

cmdprefix *chan clientaddress clientport*
${multiplexer_instance}::AddExitFilter *cmdprefix*
cmdprefix *chan clientaddress clientport*

Beschreibung

Das Paket **multiplexer** bietet ein generisches System für die Eins-zu-Viele-Kommunikation unter Verwendung von Sockets. Stellen Sie sich beispielsweise ein Chat-System vor, bei dem ein Benutzer eine Nachricht sendet, die anschließend an alle anderen verbundenen Benutzer weitergeleitet wird.

Es ist möglich, dass verschiedene Multiplexer gleichzeitig laufen.

::multiplexer::create
Der Befehl **create** erstellt eine neue Multiplexer-"Instanz". Beispiele:

```
set mp [::multiplexer::create]
```

Diese Instanz lässt sich dann wie folgt bearbeiten:

```
${mp}::Init 35100
```

${multiplexer_instance}::Init *port*
Dadurch wird der Multiplexer gestartet, der auf dem angegebenen Port auf eingehende Daten wartet.

${multiplexer_instance}::Config *key value*
Verwenden Sie "**Config**", um die Multiplexer-Instanz zu konfigurieren. Zu den derzeit verfügbaren Konfigurationsoptionen gehören:

sendtoorigin
Ein boolescher Schalter. Wenn der Wert "**true**" ist, erhält der Absender eine Kopie der gesendeten Nachricht. Der Standardwert ist "**false**".

debuglevel
Legt die für die Multiplexer-Instanz zu verwendende Debug-Stufe fest, entsprechend den im Paket **logger** angegebenen Stufen (debug, info, notice, warn, error, critical).

${multiplexer_instance}::AddFilter *cmdprefix*
Befehl zum Hinzufügen eines Filters für Daten, die die Multiplexer-Instanz durchlaufen. Das registrierte *cmdprefix* wird aufgerufen, sobald Daten bei einer Multiplexer-Instanz eintreffen. Sind bei der Instanz mehrere Filter-Befehle registriert, werden diese in der Reihenfolge ihrer Registrierung aufgerufen, wobei jeder Filter das Ergebnis des vorhergehenden Filters als Argument erhält. Der erste Filter erhält die eingehenden Daten als Argument. Das vom letzten Filter zurückgegebene Ergebnis sind die Daten, die an alle Clients der Multiplexer-Instanz gesendet werden. Der Befehl wird wie folgt aufgerufen:

cmdprefix *data chan clientaddress clientport*
Nimmt die eingehenden *data* entgegen, verändert sie und gibt das Ergebnis zurück. Die letzten drei Argumente enthalten Informationen über den Client, der die Daten an den Filter gesendet hat: den Kanal, über den die Verbindung zum Client besteht, dessen IP-Adresse und dessen IP-Port.

${multiplexer_instance}::AddAccessFilter *cmdprefix*

Befehl zum Hinzufügen eines Zugriffsfilters. Das registrierte *cmdprefix* wird aufgerufen, wenn ein neuer Client-Socket versucht, eine Verbindung zur Multiplexer-Instanz herzustellen. Sind bei der Instanz mehrere Zugriffsfilter-Befehle registriert, werden diese in der Reihenfolge ihrer Registrierung aufgerufen. Gibt einer der aufgerufenen Befehle den Wert **-1** zurück, wird der Zugriff auf die Multiplexer-Instanz verweigert und der Client-Kanal sofort geschlossen. Bei jedem anderen Ergebnis wird dem Client der Zugriff auf die Multiplexer-Instanz gewährt. Das Befehlspräfix wird wie folgt aufgerufen:

cmdprefix *chan clientaddress clientport*

Die Argumente enthalten Informationen über den Client, der versucht, eine Verbindung zur Instanz herzustellen: den Kanal, über den wir mit dem Client verbunden sind, dessen IP-Adresse und dessen IP-Port.

${multiplexer_instance}::AddExitFilter *cmdprefix*

Fügt einen Filter hinzu, der ausgeführt wird, wenn ein Client-Socket einen EOF-Zustand erzeugt. Das registrierte *cmdprefix* wird aufgerufen, wenn ein Client-Socket des Multixers EOF signalisiert. Sind bei der Instanz mehrere Exit-Filter-Befehle registriert, werden diese in der Reihenfolge ihrer Registrierung aufgerufen. Von einem Exit-Filter ausgelöste Fehler werden ignoriert, jedoch protokolliert. Alle Ergebnisse, die von einem Exit-Filter gegeben werden, werden ignoriert. Der Befehlsvorhang wird wie folgt aufgerufen:

cmdprefix *chan clientaddress clientport*

Die Argumente enthalten Informationen über den Client, der das EOF signalisiert hat: den Kanal, über den wir mit dem Client verbunden sind, dessen IP-Adresse und dessen IP-Port.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *multiplexer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

namespacex

namespacex

Stichwörter: namespacex, extended namespace, info, namespace unknown, namespace utilities, state (de)serialization, unknown hooking, utilities

Name

namespacex – Befehle für Namespace-Dienstprogramme

Übersicht

package require Tcl 8.5 9

package require namespacex ?**0.4**?
::namespacex hook add ?*namespace*? *cmdprefix*
::namespacex hook proc ?*namespace*? *arguments body*
::namespacex hook on ?*namespace*? *guardcmdprefix actioncmdprefix*
::namespacex hook next *arg...*
::namespacex import fromns *cmdname* ?newname ...?
::namespacex info allchildren *namespace*
::namespacex info allvars *namespace*
::namespacex normalize *namespace*
::namespacex info vars *namespace* ?*pattern*?
::namespacex state get *namespace*
::namespacex state set *namespace dict*
::namespacex state drop *namespace*
::namespacex strip *prefix namespaces*

Beschreibung

Dieses Paket enthält eine Reihe von Hilfsprogrammen für die Arbeit mit Namespaces. Die Befehle lassen sich in vier Kategorien einteilen:

1. Hook-Befehle stellen eine Befehlskette bereit und bearbeiten diese, wodurch der einzelne reguläre **namespace unknown**-Handler ersetzt wird.
2. Ein Importbefehl ermöglicht es, beliebige Befehle aus einem anderen Namespace zu importieren.
3. Mit Informationsbefehlen können Variablen und untergeordnete Namespaces abgefragt werden.
4. Zustandsbefehle bieten die Möglichkeit, Werte von Variablen in einem Namespace zu serialisieren.

Befehle

::namespacex hook add ?*namespace*? *cmdprefix*

Fügt das *cmdprefix* zur Kette der Handler für unbekannte Befehle hinzu, die aufgerufen werden, wenn der *namespace* andernfalls seinen eigenen Handler für unbekannte Befehle aufrufen würde. Wenn *namespace* nicht angegeben ist, wird *cmdprefix* zur Kette der Handler für den Namensraum des Aufrufers hinzugefügt.

Die Kette der *cmdprefix*-Befehle wird in umgekehrter Reihenfolge ihrer Hinzufügung ausgeführt, d. h., der zuletzt hinzugefügte *cmdprefix* wird zuerst ausgeführt. Bei der Ausführung werden *cmdprefix* zusätzliche Argumente angehängt, wie dies bei jedem Handler für unbekannte Befehle eines Namespace der Fall wäre.

::namespacex hook proc ?*namespace*? *arguments body*

Fügt der Kette der Handler für den Namespace *namespace* eine anonyme Prozedur hinzu.

Wenn *namespace* nicht festgelegt wird, wird der Handler der Kette der Handler für den Namespace des Aufrufers hinzugefügt.

Die *arguments* und *body* werden wie beim Kernbefehl **proc** festgelegt.

::namespacex hook on ?*namespace*? *guardcmdprefix actioncmdprefix*

Fügt der Kette der Handler für den Namespace *namespace* eine geschützte Aktion hinzu.

Wenn *namespace* nicht festgelegt wird, wird der Handler der Kette der Handler für den Namespace des Aufrufers hinzugefügt.

Zunächst wird *guardcmdprefix* ausgeführt. Wenn dieser einen Wert gibt, der als "falsch" interpretiert werden kann, wird der nächste Handler für unbekannte Namenräume in der Kette ausgeführt. Andernfalls wird *actioncmdprefix* ausgeführt, und der Wert, der vom Handler zurückgegeben wird, entspricht dem Wert, der von *actioncmdprefix* zurückgegeben wird.

Bei der Ausführung werden sowohl *guardcmdprefix* als auch *actioncmdprefix* mit denselben zusätzlichen Argumenten versehen wie bei jedem Handler für unbekannte Namespaces.

::namespacex hook next *arg*...
Dieser Befehl steht Namespace-Hooks zur Verfügung, um den nächsten Hook in der Kette der Handler für den Namespace auszuführen.

::namespacex import fromns *cmdname ?newname ...?*
Importiert den Befehl *cmdname* aus dem Namespace *fromns* in den Namespace des Aufrufers. Der Befehl *cmdname* wird auch dann importiert, wenn der Namespace *fromns* den Befehl ursprünglich nicht exportiert hat.

Wird *newname* angegeben, wird der importierte Befehl unter diesem Namen bekannt sein. Andernfalls behält der Befehl seinen ursprünglichen Namen, wie er durch *cmdname* angegeben wurde.

Es können auch weitere Paare von *cmdname*-/*newname*-Argumenten angegeben werden.

::namespacex info allchildren *namespace*
Gibt eine Liste zurück, die die Namen aller untergeordneten Namespaces im angegebenen *namespace* und dessen untergeordneten Namespaces enthält. Die Namen sind alle vollständig qualifiziert.

::namespacex info allvars *namespace*
Gibt eine Liste zurück, die die Namen aller Variablen im angegebenen *namespace* und dessen untergeordneten Namespaces enthält. Die Namen sind alle relativ zum *namespace* angegeben und *nicht* vollständig qualifiziert.

::namespacex normalize *namespace*
Gibt den absoluten Namen von *namespace* zurück, der relativ zum Namensraum des Aufrufers aufgelöst wird, wobei alle nicht benötigten Doppelpunkte entfernt werden.

::namespacex info vars *namespace* ?*pattern*?
Gibt eine Liste zurück, die die Namen aller Variablen im angegebenen *namespace* enthält. Wenn das Argument *pattern* festgelegt wird, werden nur Variablen zurückgegeben, die mit *pattern* übereinstimmen. Die Übereinstimmung wird nach denselben Regeln ermittelt wie bei **string match**.

::namespacex state get *namespace*
Gibt ein Dictionary zurück, das die Namen und Werte aller Variablen im angegebenen *namespace* und dessen untergeordneten Namespaces enthält.

Beachten Sie, dass sich alle Namen relativ zum *namespace* beziehen und *nicht* vollständig qualifiziert sind.

::namespacex state set *namespace dict*
Nimmt ein Dictionary entgegen, das die Namen und Werte für eine Reihe von Variablen enthält, und ersetzt den aktuellen Zustand des angegebenen *namespace* und seiner untergeordneten Namespaces durch diesen Zustand. Das Ergebnis des Befehls ist die leere Zeichenkette.

::namespacex state drop *namespace*
Löscht alle Variablen im angegebenen *namespace* und dessen untergeordneten Namespaces. Das Ergebnis des Befehls ist die leere Zeichenkette.

::namespacex strip *prefix namespaces*
Jedes Element in *namespaces* muss der absolute, normalisierte Name eines untergeordneten Namespaces des Namespaces *prefix* sein. Gibt die entsprechende Liste der relativen Namen der untergeordneten Namespaces zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *namespacex* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright


```
Copyright (c) 200? Neil Madden (http://wiki.tcl.tk/12790)
Copyright (c) 200? Various (http://wiki.tcl.tk/1489)
Copyright (c) 2010 Documentation, Andreas Kupries
```


ncgi

ncgi

Stichwörter: ncgi, CGI, cookie, form, html

Name

ncgi – Prozeduren zur Bearbeitung von CGI-Werten.

Übersicht

package require Tcl 8.5 9
package require ncgi ?**1.4.6**?
::ncgi::cookie *cookie*
::ncgi::decode *str*
::ncgi::empty *name*
::ncgi::exists *name*
::ncgi::encode *string*

::ncgi::header ?*type*? *args*
::ncgi::import *cginame* ?*tclname*?
::ncgi::importAll *args*
::ncgi::importFile *cmd cginame* ?*filename*?
::ncgi::input ?*fakeinput*? ?*fakecookie*?
::ncgi::multipart *type query*
::ncgi::nvlist
::ncgi::names
::ncgi::parse
::ncgi::parseMimeValue *value*
::ncgi::query
::ncgi::redirect *url*
::ncgi::reset *query type*
::ncgi::setCookie *args*
::ncgi::setDefaultValue *key defvalue*
::ncgi::setDefaultValueList *key defvaluelist*
::ncgi::setValue *key value*
::ncgi::setValueList *key valuelist*
::ncgi::type
::ncgi::urlStub ?*url*?
::ncgi::value *key* ?*default*?
::ncgi::valueList *key* ?*default*?

Beschreibung

Das **ncgi**-Paket stellt Befehle zur Bearbeitung von CGI-Werten bereit. Dabei handelt es sich um Werte, die aus Webformularen stammen und entweder von CGI-Skripten oder von Seiten mit eingebettetem Tcl-Code verarbeitet werden. Verwenden Sie das **ncgi**-Paket, um diese Werte abzufragen, Cookies zu setzen und abzurufen sowie www-url-codierte Werte zu kodieren und zu decodieren.

Im einfachsten Fall ruft ein CGI-Skript zunächst **::ncgi::parse** und anschließend **::ncgi::value** auf, um verschiedene Werte aus dem Formular abzurufen. Wenn ein CGI-Wert wiederholt wird, sollten Sie **::ncgi::valueList** verwenden, um die vollständige Liste der Werte abzurufen.

Eine Alternative zu **::ncgi::parse** ist **::ncgi::input**, dessen Funktionsweise der Prozedur **cgi_input** von Don Libes ähnelt. **::ncgi::input** beschränkt wiederholte CGI-Werte so, dass ihre Name mit "List" enden. In diesem Fall gibt **::ncgi::value** die vollständige Liste der Werte zurück, während **::ncgi::input** Fehler auslöst, wenn es wiederholte Formularelemente ohne den richtigen Name findet.

Die Prozedur **::ncgi::reset** kann in TESTS und auf Webservern verwendet werden, um die Quelle der CGI-Werte zu initialisieren. Andernfalls werden die Werte aus der CGI-Umgebung eingelesen.

Im Folgenden wird der gesamte Ablauf der Prozeduren beschrieben.

::ncgi::cookie *cookie*
: Gibt eine Liste der Werte für *cookie* zurück, sofern vorhanden. Da möglicherweise mehrere Cookies mit demselben Namen vorhanden sind, gibt diese Prozedur eine Liste zurück.

::ncgi::decode *str*
: Dekodiert Zeichenketten in der www-url-encoding-Kodierung, bei der Sonderzeichen durch die

Zeichenfolge "%xx" dargestellt werden, wobei "xx" für den Zeichencode im Hexadezimalformat steht.

::ncgi::empty *name*
Gibt 1 zurück, wenn die CGI-Variable *name* nicht vorhanden ist oder den Wert "leere Zeichenkette" hat.

::ncgi::exists *name*
Der Wert wird als boolescher Wert zurückgegeben. Er gibt **0** zurück, wenn die CGI-Variable *name* nicht vorhanden ist, andernfalls **1**.

::ncgi::encode *string*
string im www-url-encoded-Format kodieren.

::ncgi::header ?*type*? *args*
Gibt den CGI-Header an die Standardausgabe aus. Dabei werden ein "Content-Type:"-Header sowie weitere Header basierend auf *args* ausgegeben, wobei *args* eine Liste aus Header-Namen und Header-Werten ist. Der Standardwert für *type* ist "text/html".

::ncgi::import *cginame* ?*tclname*?
Dadurch wird im aktuellen Gültigkeitsbereich eine Variable mit dem Wert der CGI-Variablen *cginame* angelegt. Der Name der Variablen lautet *tclname* oder *cginame*, falls *tclname* leer ist (Standard).

::ncgi::importAll *args*
Dadurch werden mehrere CGI-Variablen als Tcl-Variablen importiert. Wenn *args* leer ist, werden alle CGI-Werte importiert. Andernfalls wird jede in *args* aufgeführte CGI-Variable importiert.

::ncgi::importFile *cmd cginame* ?*filename*?
Hiermit werden Informationen zu einer hochgeladenen Datei aus einem Formular-Eingabefeld vom Typ **file** mit dem Name *cginame* bereitgestellt. *cmd* kann einer der folgenden Werte sein: **-server**, **-client**, **-type** oder **-data** lauten.

-client *cginame*
gibt den vom Client gesendeten Dateinamen zurück.

-type *cginame*
Gibt den MIME-Typ der hochgeladenen Datei zurück.

-data *cginame*
gibt den Inhalt der Datei zurück.

-server *cginame filename*
Schreibt den Inhalt der Datei in eine lokale temporäre Datei (oder den Dateinamen, wenn angegeben) und gibt den Namen der Datei zurück. Der Aufrufer ist dafür verantwortlich, diese Datei nach der Verwendung zu löschen.

::ncgi::input ?*fakeinput*? ?*fakecookie*?
Diese Funktion liest die CGI-Werte aus der Umgebung ein und dekodiert sie. Sie sorgt dafür, dass wiederholte Formularwerte am Ende ihres NAMENS den Zusatz "List" enthalten. Die CGI-Werte werden später mit der Prozedur **::ncgi::value** abgerufen.

::ncgi::multipart *type query*
Diese Prozedur analysiert eine *Abfrage* vom Typ "multipart/form-data". Sie wird von **::ncgi::nvlist** verwendet und normalerweise nicht direkt aufgerufen. Sie gibt eine abwechselnde Liste aus Namen und strukturierten Werten zurück. Jeder strukturierte Wert ist

wiederum eine Liste aus zwei Elementen. Das erste Element enthält Metadaten aus der "multipart/form-data"-Struktur. Das zweite Element ist der Formularwert. Wenn Sie **::ncgi::value** verwenden, erhalten Sie lediglich den Formularwert. Wenn Sie **::ncgi::valueList** verwenden, erhalten Sie den strukturierten Wert mit Metadaten und dem Wert.

Der *Typ* ist der gesamte Content-Type, einschließlich der Parameter wie *boundary*. Dies gibt eine Liste von Namen und Werten zurück, die die Multipart-Daten beschreiben. Die Werte sind eine verschachtelte Listenstruktur, die zunächst einige beschreibende Informationen und anschließend den eigentlichen Formularwert enthält. Die beschreibenden Informationen sind eine Liste von Header-Namen und Werten, die den Inhalt beschreiben.

::ncgi::nvlist
Dadurch geben Sie alle Abfragedaten als Liste aus Namen und Werten zurück. Bei "multipart/form-data" sind die Werte wie in **::ncgi::multipart** beschrieben strukturiert.

::ncgi::names
Dies gibt alle in den Abfragedaten gefundenen Namen als Liste zurück. **::ncgi::multipart**.

::ncgi::parse
Diese Prozedur liest die CGI-Werte aus der Umgebung ein und dekodiert sie. Die CGI-Werte werden später mit der Prozedur **::ncgi::value** abgerufen. Wenn ein CGI-Wert mehrfach vorkommt, sollten Sie **::ncgi::valueList** verwenden, um die vollständige Liste der Werte zu erhalten.

::ncgi::parseMimeValue *value*
Diese Funktion dekodiert den Content-Type und andere MIME-Header, die die Form "primärer Wert; param=val; p2=v2" haben. Sie gibt eine Liste zurück, deren erstes Element der primäre Wert ist und deren zweites Elemente eine Liste mit Parameternamen und -werten enthält.

::ncgi::query
Dies gibt die Rohdaten der Abfrage zurück.

::ncgi::redirect *url*
Erzeugt eine Antwort, die eine 302-Weiterleitung durch den Webserver auslöst. Die *url* ist die neue URL, auf die die Weiterleitung zielt. Die URL wird bei Bedarf mit dem aktuellen Server und dem aktuellen Verzeichnis ergänzt, um sie in eine vollständige URL umzuwandeln.

::ncgi::reset *query type*
Legt die Abfragedaten und den Content-Type für die aktuelle CGI-Sitzung fest. Dies wird von der Testsuite und von Webservern verwendet, um das ncgi-Modul so zu initialisieren, dass es nicht versucht, Daten aus der Standardeingabe zu lesen oder Variablen zu verwenden, um seine Daten zu beziehen. Wenn weder *query* noch *type* festgelegt sind, sucht das **ncgi**-Modul seine Daten in der Standard-CGI-Umgebung.

::ncgi::setCookie *args*
Legt einen Cookie-Wert fest, der als Teil der Antwort zurückgegeben wird. Dies muss erfolgen, bevor **::ncgi::header** oder **::ncgi::redirect** aufgerufen wird, damit das Cookie korrekt zurückgegeben wird. Die *args* sind eine Reihe von Flags und Werten:
- **-name** *name*
- **-value** *value*
- **-expires** *date*
- **-path** *path restriction*

- **-domain** *domain restriction*

::ncgi::setDefaultValue *key defvalue*
Legt einen CGI-Wert fest, wenn dieser noch nicht vorhanden ist. Dies wirkt sich auf zukünftige Aufrufe von **::ncgi::value** aus (jedoch nicht auf zukünftige Aufrufe von **::ncgi::nvlist**). Wenn der CGI-Wert bereits vorhanden ist, hat diese Prozedur keine Nebenwirkungen.

::ncgi::setDefaultValueList *key defvaluelist*
Ähnlich wie **::ncgi::setDefaultValue**, mit dem Unterschied, dass der Wert bereits eine Listenstruktur aufweist, um mehrere Kontrollkästchen oder eine Mehrfachauswahl darzustellen.

::ncgi::setValue *key value*
Legt einen CGI-Wert fest, der alle bereits in der CGI-Umgebung vorhandenen Werte überschreibt. Dies wirkt sich auf zukünftige Aufrufe von **::ncgi::value** aus (jedoch nicht auf zukünftige Aufrufe von **::ncgi::nvlist**).

::ncgi::setValueList *key valuelist*
Ähnlich wie **::ncgi::setValue**, mit dem Unterschied, dass der Wert bereits eine Listenstruktur aufweist, um mehrere Kontrollkästchen oder eine Mehrfachauswahl darzustellen.

::ncgi::type
Gibt den Content-Type der aktuellen CGI-Werte zurück.

::ncgi::urlStub ?*url*?
Gibt die aktuelle URL zurück, jedoch ohne Protokoll, Server und Port. Wenn *url* angegeben wird, legt dies die URL für die aktuelle Sitzung fest. Dieser Wert wird bei zukünftigen Aufrufen von **::ncgi::urlStub** zurückgegeben.

::ncgi::value *key* ?*default*?
Gibt den durch *key* identifizierten CGI-Wert zurück. Wenn der CGI-Wert nicht vorhanden ist, gibt es stattdessen den *Standardwert* zurück. Dieser Wert ist standardmäßig die leere Zeichenkette.

Wenn der Formularwert *key* wiederholt wird, gibt es zwei Fälle: Wurde **::ncgi::parse** aufgerufen, gibt **::ncgi::value** nur den ersten mit *key* verknüpften Wert zurück. Wenn **::ncgi::input** aufgerufen wird, gibt **::ncgi::value** einen Tcl-Wert zurück, und *key* muss auf "List" enden (z. B. "skuList"). Im Fall von "multipart/form-data" gibt diese Prozedur lediglich den Wert des Formularelements zurück. Wenn Sie die zu jedem Formularwert gehörenden Metadaten erhalten möchten, verwenden Sie **::ncgi::valueList**.

::ncgi::valueList *key* ?*default*?
Ähnlich wie **::ncgi::value**, gibt diese Prozedur jedoch immer eine Liste von Werten zurück (auch wenn nur ein Wert vorhanden ist). Im Fall von "multipart/form-data" gibt diese Prozedur eine Liste mit zwei Elementen zurück. Das erste Element besteht aus Metadaten in Form einer Parameter-Wert-Liste. Das zweite Element ist der Formularwert.

Beispiele

Eine Datei hochladen

```
HTML:
<html>
<form action="/cgi-bin/upload.cgi" method="POST"
    enctype="multipart/form-data">
Path: <input type="file" name="filedata"><br>
Name: <input type="text" name="filedesc"><br>
<input type="submit">
</form>
</html>

TCL: upload.cgi
#!/usr/local/bin/tclsh

::ncgi::parse
set filedata [::ncgi::value filedata]
set filedesc [::ncgi::value filedesc]

puts "<html> File uploaded at <a
    href=\"/images/$filedesc\">$filedesc</a> </html>"

set filename /www/images/$filedesc

set fh [open $filename w]
puts -nonewline $fh $filedata
close $fh
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ncgi* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

CGI-Programmierung

nettool

nettool

Stichwörter: nettool, odie

Name

nettool – Werkzeuge für vernetzte Anwendungen

Übersicht

package require Tcl 8.5 9
package require nettool ?**0.5.4**?
package require twapi 3.1
package require ip 0.2
package require platform 0.2
::cat *filename*
::nettool::allocate_port *startingport*
::nettool::arp_table
::nettool::broadcast_list
::nettool::claim_port *port* ?*protocol*?
::nettool::cpuinfo *args*
::nettool::find_port *startingport*
::nettool::hwid_list
::nettool::ip_list
::nettool::mac_list
::nettool::network_list
::nettool::port_busy *port*
::nettool::release_port *port* ?*protocol*?
::nettool::status
::nettool::user_data_root *appname*

Beschreibung

Das Paket **nettool** ist eine Sammlung von Werkzeugen in reinem Tcl. Sie führen gängige Netzwerkfunktionen über eine einheitliche Tcl-Schnittstelle aus, für die sonst verschiedene Pakete oder Aufrufe von exec nötig wären. Derzeit hat nettool Referenzimplementierungen für Windows, MacOSX und Linux (debian).

Befehle

::cat *filename*
: Gibt den Inhalt einer Datei als Ergebnis zurück.

::nettool::allocate_port *startingport*
: Versucht, *startingport* zu belegen. Ist der Port belegt, wird die Portnummer der Reihe nach erhöht, bis ein freier Port gefunden ist, und dieser Port wird belegt. Der Befehl nutzt eine eingebaute Datenbank bekannter Ports, damit er keinen Port zurückgibt, der häufig verwendet wird. (Beispiel: http (80))

::nettool::arp_table
: Gibt den Inhalt der ARP-Tabelle (Address Resolution Protocol) dieses Rechners zurück. Das Ergebnis ist eine Liste im Tcl-Format: *macid ipaddrlist* ...

::nettool::broadcast_list
: Gibt eine Liste der Broadcast-Adressen (geeignet für UDP-Multicast) zurück, denen dieser Computer zugeordnet ist.

::nettool::claim_port *port* ?*protocol*?
: Markiert *port* als belegt, wahlweise als **tcp** (Vorgabe) oder als **udp**.

::nettool::cpuinfo *args*
: Ohne Argumente gibt der Befehl eine Schlüssel-Wert-Liste zurück, die die CPU des aktuellen

Rechners beschreibt. Darin stehen die Zahl der Kerne oder Prozessoren für parallele Aufgaben, der eingebaute physische Arbeitsspeicher und die Prozessorfamilie.

Der genaue Inhalt hängt von der Plattform ab.

Unter Linux stammen die Angaben aus /proc/cpuinfo und /proc/meminfo.

Unter MacOSX stammen sie aus sysctl.

Unter Windows stammen sie aus TWAPI.

Mit Argumenten ist das Ergebnis eine Schlüssel-Wert-Liste, beschränkt auf die angeforderten Felder.

Die auf allen Plattformen üblichen Felder:

cpus
Anzahl der CPUs/Kerne/Ausführungseinheiten

speed
Taktfrequenz des/der Prozessoren in MHz

memory
Installierter Arbeitsspeicher (in MB)

vendor
Hersteller

::nettool::find_port *startingport*
Gibt *startingport* zurück, wenn dieser frei ist, sonst den nächsten freien Port nach *startingport*. Anders als **::nettool::allocate_port** belegt dieser Befehl den Port nicht.

Er nutzt eine eingebaute Datenbank bekannter Ports, damit er keinen Port zurückgibt, der häufig verwendet wird. (Beispiel: http (80))

::nettool::hwid_list
Gibt eine Liste hardwarespezifischer Kennungen dieses Rechners zurück. Quelle und Inhalt hängen von der Plattform ab.

Unter MacOSX kommen die Seriennummer der Hauptplatine und die MAC-IDs aller Netzwerkgeräte zurück.

Unter Windows kommen die Volume-Seriennummer von C und die MAC-IDs aller Netzwerkgeräte zurück.

Unter Linux kommen die MAC-IDs aller Netzwerkgeräte zurück.

::nettool::ip_list
Gibt eine Liste der diesem Rechner zugeordneten IP-Adressen zurück.

::nettool::mac_list
Gibt eine Liste der MAC-IDs der Netzwerkkarten dieses Rechners zurück. Die MAC-ID der primären Netzwerkkarte steht an erster Stelle.

::nettool::network_list
Gibt eine Liste der mit diesem Rechner verbundenen Netzwerke zurück. Die Netzwerke sind mit **ip::nativeToPrefix** formatiert.

::nettool::port_busy *port*
Gibt true zurück, wenn *port* belegt ist, sonst false.

::nettool::release_port *port* ?*protocol*?
Markiert *port* als nicht belegt, wahlweise als **tcp** (Vorgabe) oder als **udp**.

::nettool::status
Gibt eine Schlüssel-Wert-Liste zurück, die den Status des Rechners beschreibt. Die Ausgabe ist so angelegt, dass sie auf allen Plattformen mit der Ausgabe von **top** vergleichbar ist.

Übliche Felder sind:

load
Prozesse pro Verarbeitungseinheit

memory_total
Physischer Arbeitsspeicher insgesamt (MB)

memory_free
Ungenutzter physischer Arbeitsspeicher (MB)

::nettool::user_data_root *appname*
Gibt einen vollständigen Pfad zu einem Ordner zurück, in dem *appname* seine Daten ablegen soll. Der Befehl legt den Pfad nicht an, er berechnet ihn nur.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *odie* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

System

Copyright


```
Copyright (c) 2015-2018 Sean Woods <yoda@etoyoc.com>
```


nmea

nmea

Stichwörter: nmea, gps

Name

nmea – NMEA-Daten verarbeiten

Übersicht

package require Tcl 8.5 9
package require nmea ?**1.1.0**?

::nmea::input *sentence*
::nmea::open_port *port* ?*speed*?
::nmea::close_port
::nmea::configure_port *settings*
::nmea::open_file *file* ?*rate*?
::nmea::close_file
::nmea::do_line
::nmea::rate
::nmea::log ?*file*?
::nmea::checksum *data*
::nmea::write *sentence data*
::nmea::event *setence* ?*command*?

Beschreibung

Dieses Paket stellt eine einheitliche Schnittstelle bereit, um Software zu schreiben, die Eingaben nach dem NMEA-Standard empfängt. Daten kommen von COM-Schnittstellen, aus Dateien oder aus dem Programm. Es bildet Prüfsummen und protokolliert eingehende Daten. Nach dem Parsen gehen die Eingaben an vom Benutzer festgelegte Handler-Befehle. Einen Handler legt der Befehl **event** fest. Das Paket enthält keine GPS-spezifischen Funktionen. NMEA-Daten bestehen aus einem Satztyp und einer darauffolgenden Datenliste.

Befehle

::nmea::input *sentence*

Verarbeitet den übergebenen Satz und gibt ihn weiter. Enthält *sentence* keine Kommas, gilt er als Tcl-Liste. Sonst müssen es die üblichen, durch Kommas getrennten NMEA-Daten sein, mit optionaler Prüfsumme und vorangestelltem **$**.

```
nmea::input {$GPGSA,A,3,04,05,,09,12,,,24,,,,,2.5,1.3,2.1*39}
nmea::input [list GPGSA A 3 04 05  09 12 "" "" 24 "" "" ""  2.5 1.3
    2.1]
```

::nmea::open_port *port* ?speed?

Öffnet die angegebene COM-Schnittstelle und liest NMEA-Sätze, sobald welche anliegen. Die Geschwindigkeit ist 4800 bps oder *speed*.

::nmea::close_port

Schließt die COM-Verbindung, wenn eine offen ist.

::nmea::configure_port *settings*

Ändert die aktuellen Porteinstellungen. *settings* hat dasselbe Format wie fconfigure -mode.

::nmea::open_file *file* ?rate?

Öffnet die Datei *file* und liest NMEA-Sätze, einen je Zeile, im Abstand von *rate* Millisekunden. Das Dateiformat darf das vorangestellte **$** und die Prüfsumme weglassen. Ist *rate* <= 0 (die Vorgabe), werden Zeilen nur verarbeitet, wenn **do_line** aufgerufen wird.

::nmea::close_file

Schließt die offene Datei, falls eine vorhanden ist.

::nmea::do_line

Wenn derzeit eine Datei geöffnet ist, liest dieser Befehl eine einzelne Zeile daraus und

verarbeitet sie. Gibt die Anzahl der gelesenen Zeilen zurück.

::nmea::rate
Legt fest, in welchem Abstand in Millisekunden Zeilen aus der offenen Datei verarbeitet werden. Der Abstand gilt über Dateien hinweg; es muss keine Datei offen sein. 0 schaltet die automatische Zeilenverarbeitung aus.

::nmea::log ?file?
Startet oder beendet die Protokollierung der Eingabe. Ist ein Dateiname angegeben, werden alle an der offenen Schnittstelle empfangenen NMEA-Daten im Anhängemodus in *file* geschrieben. Ist *file* eine leere Zeichenkette, endet die Protokollierung. Ohne Datei gibt der Befehl einen booleschen Wert zurück, ob die Protokollierung gerade läuft. Daten, die **write** an die Schnittstelle schreibt, Daten aus Dateien und Eingaben über **input** werden nicht protokolliert.

::nmea::checksum *data*
Gibt die Prüfsumme der übergebenen Daten zurück.

::nmea::write *sentence data*
Wenn derzeit ein Port geöffnet ist, schreibt dieser Befehl den angegebenen Satz und die Daten im korrekten NMEA-Format mit Prüfsumme an den Port.

::nmea::event *setence* ?command?
Registriert eine Handler-Prozedur für einen gegebenen NMEA-*sentence*. Je Satz gibt es höchstens einen Handler; ein vorhandener wird ersetzt. Ohne Befehl gibt er den Namen des aktuellen Handlers für den gegebenen *setence* zurück, oder eine leere Zeichenkette, wenn keiner existiert. Neben den eingehenden Sätzen gibt es zwei eingebaute Typen, EOF und DEFAULT. Der Handler für den DEFAULT-*setence* läuft, wenn es für diesen Satz keinen eigenen Handler gibt. Der EOF-Handler läuft, wenn an der offenen Datei oder Schnittstelle das Dateiende erreicht ist.

Die Handler-Prozeduren, mit Ausnahme der eingebauten Typen, müssen genau ein Argument nehmen, eine Liste der Datenwerte. Der DEFAULT-Handler soll zwei Argumente haben, den Satztyp und die Datenwerte. Der EOF-Handler hat keine Argumente.

```
nmea::event gpgsa parse_sat_detail
nmea::event default handle_unknown

proc parse_sat_detail {data} {
    puts [lindex $data 1]
}

proc handle_unknown {name data} {
    puts "unknown data type $name"
}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nmea* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright
```
Copyright (c) 2006-2009, Aaron Faupell <afaupell@users.sourceforge.net>
```

nns

nns_auto
Stichwörter: nns_auto, automatic, client, name service, reconnect, restore

Name
nameserv::auto – Namensdienst-Funktion, Client-Erweiterung

Übersicht
package require Tcl 8.5 9
package require nameserv::auto ?**0.4**?
package require nameserv

Beschreibung
Zuerst ist das Dokument *Namensdienst-Funktion, Einleitung* zu lesen.

Dieses Paket stellt genau dieselbe API bereit wie das Paket **nameserv**, d. h. den regulären Namensdienst-Client. Es unterscheidet sich von diesem dadurch, dass es Maßnahmen ergreift, um sicherzustellen, dass länger gültige Daten – d. h. gebundene Namen sowie laufende und noch nicht abgeschlossene asynchrone Suchvorgänge – den Verlust der Verbindung zum Namensserver so weit wie möglich überstehen.

Das bedeutet, dass die gebundenen Namen sowie laufende und noch nicht abgeschlossene asynchrone Suchvorgänge clientseitig gespeichert und automatisch wieder an den Server übermittelt werden, wenn die Verbindung nach einem Verbindungsabbruch wiederhergestellt ist. Bei gebundenen Namen gibt es eine wichtige Einschränkung hinsichtlich einer solchen Wiederherstellung: Es ist möglich, dass ein Name dieses Clients während des Verbindungsabbruchs von einem anderen Client gebunden wurde. Solche Namen gehen vollständig verloren, und das Paket kann und wird den Nutzer bestenfalls darüber informieren.

API
Die für den Benutzer sichtbare API entspricht im Wesentlichen der API von **nameserv** und wird daher an dieser Stelle nicht näher beschrieben. Bitte lesen Sie die Dokumentation zu **nameserv** .

Die Unterschiede werden im Folgenden in den Abschnitten Optionen und Ereignisse erläutert.

Optionen
Dieses Paket unterstützt alle Optionen des Pakets **nameserv** sowie eine weitere Option. Mit

dieser zusätzlichen Option kann der Benutzer das Zeitintervall festlegen, in dem versucht wird, eine unterbrochene Verbindung wiederherzustellen.

-delay *milliseconds*
: Der Wert dieser Option ist eine ganze Zahl größer als 0, die das Intervall in Millisekunden festlegt, das zwischen den Versuchen zur Wiederherstellung einer unterbrochenen Verbindung gewartet werden soll. Der Standardwert lautet **1000**, d. h. eine Sekunde.

Ereignisse

Dieses Paket generiert alle Ereignisse des Pakets **nameserv** sowie zwei weitere. Beide Ereignisse werden für das Tag *nameserv* generiert.

lost-name
: Dieses Ereignis wird ausgelöst, wenn ein gebundener Name tatsächlich verloren gegangen ist, d. h. nach dem vorübergehenden Verlust der Verbindung zum Namensserver nicht wiederhergestellt werden konnte. Es weist darauf hin, dass ein anderer Client die Kontrolle über den Namen übernommen hat, während dieser Client nicht erreichbar war.

  Die Detailinformationen des Ereignisses bestehen aus einem Tcl-Dictionary, das zwei Schlüssel enthält: **name** und **data**. Ihre Werte enthalten alle Informationen über den verlorenen Namen.

re-connection
: Dieses Ereignis wird ausgelöst, wenn die Verbindung zum Server wiederhergestellt ist. Die gespeicherten Daten wurden zum Zeitpunkt der Auslösung des Ereignisses wiederhergestellt.

  Das Ereignis enthält keine Detailinformationen.

Design

Das Paket wird auf Basis des regulären Nameservice-Clients, d. h. des Pakets **nameserv**, implementiert. Es erkennt den Verbindungsverlust, indem es auf *lost-connection*-Ereignisse am Tag *nameserv* überwacht.

Es reagiert auf solche Ereignisse, indem es einen periodischen Timer startet und versucht, die Verbindung zum Server wiederherzustellen, sobald dieser Timer ausgelöst wird. Bei Erfolg wird der Timer abgebrochen, ein *re-connection*-Ereignis generiert, und das Paket gibt die gespeicherten gebundenen Namen und die kontinuierlichen Suchvorgänge erneut ein.

Ein weiterer Abbruch der Verbindung – sei es während oder nach der erneuten Eingabe der gespeicherten Informationen – führt lediglich dazu, dass der Timer neu gestartet wird und weitere Verbindungsversuche unternommen werden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright

```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nns_client
Stichwörter: nns_client, client, name service

Name
nameserv – Namensdienst-Funktion, Client

Übersicht
package require Tcl 8.5 9
package require nameserv ?**0.4.3**?
package require comm
package require logger
::nameserv::bind *name data*
::nameserv::release
::nameserv::search ?**-async|-continuous**? ?*pattern*?
::nameserv::protocol
::nameserv::server_protocol
::nameserv::server_features
::nameserv::cget -option
::nameserv::configure
::nameserv::configure -option
::nameserv::configure -option *value...*
$result destroy
$result filled
$result get *name*
$result names
$result size
$result getall ?*pattern*?

Beschreibung
Bitte lesen Sie zunächst *Namensdienst-Funktion, Einleitung*.

Dieses Paket stellt einen Client für die vom Paket **nameserv::server** implementierte Namensdienstfunktion bereit.

Dieser Dienst wurde auf der Grundlage des Pakets **comm** entwickelt und ist für dieses bestimmt. Er hat nichts mit dem Domain Name System des Internets zu tun. Wenn Sie nach einem Paket suchen, das sich damit befasst, sehen Sie sich bitte die Tcllib-Pakete **dns** und **resolv** an.

API
Das Paket exportiert acht Befehle, wie im Folgenden aufgeführt:

::nameserv::bind *name data*
Der Aufrufer dieses Befehls registriert den angegebenen *name* als seinen Namen im konfigurierten Namensdienst und ordnet ihm zusätzlich ein Stück *data* zu. Der Dienst führt mit diesen Informationen nichts weiter aus, als sie zu speichern und als Teil der Suchergebnisse bereitzustellen. Die Bedeutung liegt vollständig bei den Anwendungen, die den Namensdienst nutzen.

Eine allgemein sinnvolle Wahl wäre beispielsweise eine Kennung für einen Kommunikationsendpunkt, der vom Paket **comm** verwaltet wird. Wer den Namen abruft, kann sofort mit diesem Endpunkt, d. h. der registrierenden Anwendung, kommunizieren.

Von weiterer Bedeutung ist, dass sich ein Aufrufer unter mehr als einem Namen registrieren kann und jeder Name über eigene *data* verfügen kann.

Der Namensdienst – und somit auch dieser Befehl – löst einen Fehler aus, wenn der gewählte Name bereits registriert ist.

::nameserv::release
Durch den Aufruf dieses Befehls werden alle Namen (und die dazugehörigen Daten) freigegeben, die bei allen bisherigen Aufrufen von **::nameserv::bind** dieses Clients registriert wurden. Hinweis: Der Namensdienst führt diesen Befehl implizit aus, wenn die Verbindung zu diesem Client unterbrochen wird.

::nameserv::search ?-async|-continuous? ?*pattern*?
Dieser Befehl durchsucht den Namensdienst nach allen registrierten Namen, die dem angegebenen Glob-*pattern* entsprechen. Wird keins festgelegt, lautet das Standardmuster *, wodurch alles abgeglichen wird. Das Ergebnis des Befehls ist ein Dictionary, das die übereinstimmenden Namen den ihnen zur *bind*-Zeit zugeordneten Daten zuordnet.

Wenn die Option **-async** oder **-continuous** festgelegt wird, ändert sich das Ergebnis dieses Befehls und wird zum Tcl-Befehl eines Objekts, das das eigentliche Ergebnis enthält. Diese beiden Optionen werden genau dann unterstützt, wenn der Dienst, mit dem der Client verbunden ist, die Protokollfunktion *Search/Continuous* unterstützt.

Bei **-async** wird das Ergebnisobjekt asynchron mit den Einträgen gefüllt, die zum Zeitpunkt der Suche dem Muster entsprechen, und anschließend nicht mehr geändert. Die Option **-continuous** erweitert dieses Verhalten, indem sie den Dienst zusätzlich kontinuierlich auf das Hinzufügen und Entfernen von Einträgen überwacht, die dem Muster entsprechen, und den Inhalt des Objekts entsprechend aktualisiert.

Hinweis: Der Aufrufer ist dafür verantwortlich, das Objekt mit einer Rückruffunktion zu konfigurieren, damit er ordnungsgemäß benachrichtigt wird, wenn das aktuelle Ergebnis (oder weitere Änderungen) eintrifft.

Weitere Informationen zu diesem Objekt stehen im Abschnitt Asynchrone und kontinuierliche Suchvorgänge.

::nameserv::protocol
Dieser Befehl gibt die höchste Version des Namensdienstprotokolls zurück, die von dem Paket

unterstützt wird.

::nameserv::server_protocol
Dieser Befehl gibt die höchste Version des Namensdienstprotokolls zurück, die von dem Namensdienst unterstützt wird, mit dem der Client derzeit verbunden ist.

::nameserv::server_features
Dieser Befehl gibt eine Liste zurück, die die Namen der Funktionen des Namensdienstprotokolls enthält, die von dem Namensdienst unterstützt werden, mit dem der Client derzeit verbunden ist.

::nameserv::cget -option
Dieser Befehl gibt den aktuell konfigurierten Wert für das angegebene **-option** zurück. Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

::nameserv::configure
In dieser Form gibt der Befehl ein Dictionary mit allen unterstützten Optionen und deren aktuellen Werten zurück. Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

::nameserv::configure -option
In dieser Form ist der Befehl ein Alias für "**::nameserv::cget -option**". Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

::nameserv::configure -option *value*...
In dieser Form wird der Befehl verwendet, um eine oder mehrere der unterstützten Optionen zu konfigurieren. Es muss mindestens eine Option angegeben werden, und auf jede Option folgt ihr neuer Wert. Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

Diese Form kann nur verwendet werden, solange der Client noch keinen Kontakt zum Namensdienst hergestellt hat. Nach Herstellung des Kontakts ist eine Neukonfiguration nicht mehr möglich. Das bedeutet, dass diese Form des Befehls zur Initialisierung des Clients vor dessen Verwendung dient. Die Befehle, die einen Kontakt mit dem Namensdienst erzwingen, lauten
- **bind**
- **release**
- **search**
- **server_protocol**
- **server_features**

Verbindungsverwaltung

Der Client stellt automatisch eine Verbindung zum Dienst her, wenn einer der folgenden Befehle zum ersten Mal ausgeführt wird oder wenn einer der Befehle ausgeführt wird, nachdem die Verbindung unterbrochen wurde – unabhängig davon, wann dies geschehen ist.

- **bind**
- **release**
- **search**
- **server_protocol**
- **server_features**

Seit Version 0.2 des Clients wird ein Ereignis generiert, wenn die Verbindung abbricht, wodurch

höhere Schichten zusätzliche Maßnahmen ergreifen können. Dies erfolgt über das Support-Paket **uevent**. Dieses und alle anderen Pakete im Zusammenhang mit dem Namensdienst reservieren hiermit das uevent-Tag *nameserv*. Alle ihre Ereignisse werden an dieses Tag gesendet.

Ereignisse

Dieses Paket generiert nur ein Ereignis: *lost-connection*. Die zu diesem Ereignis bereitgestellten Detailinformationen sind ein Tcl-Dictionary. Der einzige im Dictionary enthaltene Schlüssel ist **reason**, und sein Wert ist eine Zeichenkette, die beschreibt, warum die Verbindung unterbrochen wurde. Diese Zeichenkette wird vom zugrunde liegenden Kommunikationspaket bereitgestellt, d. h. von **comm**.

Optionen

Die vom Client unterstützten Optionen dienen der Festlegung, welcher Namensdienst kontaktiert werden soll, d. h. des Standorts des Namensdienstes. Es handelt sich dabei um folgende Optionen:

-host *name*|*ipaddress*
: Diese Option legt fest, auf welchem Host der zu kontaktierende Namensdienst läuft, entweder über den *name* oder über die *ipaddress*. Die anfängliche Standardeinstellung lautet **localhost** , d. h., es wird erwartet, dass ein Namensdienst kontaktiert wird, der auf demselben Host läuft wie die Anwendung, die dieses Paket verwendet.

-port *number*
: Diese Option legt den Port fest, auf dem der zu kontaktierende Namensdienst auf eingehende Verbindungen wartet. Es muss sich um eine positive ganze Zahl (> 0) handeln, die nicht größer als 65536 (unsigned short) ist. Der anfängliche Standardwert ist die Zahl, die vom Befehl ::nameserv::common::port zurückgegeben wird, wie sie vom Paket **::nameserv::common** bereitgestellt wird.

Asynchrone und kontinuierliche Suchvorgänge

Asynchrone und kontinuierliche Suchvorgänge werden aufgerufen, indem entweder die Option **-async** oder **-continuous** als Argument für den Befehl **::nameserv::search** verwendet wird.

Hinweis: Diese beiden Optionen werden genau dann unterstützt, wenn der Dienst, mit dem der Client verbunden ist, die Protokollfunktion *Search/Continuous* unterstützt. Der vom Paket **::nameserv::server** bereitgestellte Dienst unterstützt dies seit Version 0.3.

Bei solchen Suchvorgängen ist das Ergebnis des Suchbefehls der Tcl-Befehl eines Objekts, das das eigentliche Ergebnis enthält. Die von diesen Objekten bereitgestellte API lautet wie folgt:

Optionen:
: siehe unten ...

-command *command_prefix*
: Diese Option ist zu setzen, wenn ein Nutzer des Ergebnisobjekts asynchrone Benachrichtigungen erhalten möchte, sobald das Suchergebnis oder Änderungen daran eintreffen.

Hinweis: Es ist zwar möglich, über die Methode **filled** auf das Eintreffen des ersten Suchergebnisses zu prüfen und nachfolgende Änderungen durch den Vergleich der Ausgabe der Methode **getall** mit einer gespeicherten Kopie zu erkennen; dies ist jedoch

nicht die empfohlene Vorgehensweise. Weitaus effizienter ist es, die Rückruffunktion **-command** zu setzen und die Benachrichtigungen bei ihrem Eintreffen zu verarbeiten.

Der *command_prefix* wird mit zwei Argumenten aufgerufen: dem Typ der Änderung und den Daten der Änderung. Der Typ ist entweder **add** oder **remove**, was jeweils auf neue bzw. gelöschte Daten hinweist. Die Daten der Änderung sind stets ein Dictionary, in dem die hinzugefügten bzw. entfernten Namen und die zugehörigen Daten aufgeführt sind.

Die erste für eine Suche gemeldete Änderung ist stets die Menge der zum Zeitpunkt der Suche übereinstimmenden Einträge.

Methoden:

siehe unten ...

$result destroy

Löscht das Objekt und beendet jegliche fortlaufende Überwachung des Dienstes, die für das Objekt möglicherweise aktiv war.

$result filled

Das Ergebnis ist ein boolescher Wert, der angibt, ob das Suchergebnis bereits vorliegt (**True**) oder nicht (**False**).

$result get *name*

Gibt die Daten zurück, die dem angegebenen *name* zur *bind*-Zeit zugeordnet sind.

$result names

Gibt eine Liste zurück, die alle Namen enthält, die dem Objekt zum Zeitpunkt des Aufrufs bekannt sind.

$result size

Gibt einen ganzzahligen Wert zurück, der die Größe des Ergebnisses zum Zeitpunkt des Aufrufs angibt.

$result getall ?*pattern*?

Gibt ein Dictionary zurück, das das Suchergebnis zum Zeitpunkt des Aufrufs enthält und die übereinstimmenden Namen den ihnen zur *bind*-Zeit zugeordneten Daten zuordnet.

Geschichte

0.3.1

Der SF-Fehler 1954771 wurde behoben.

0.3

Der Client wurde um die Möglichkeit erweitert, asynchrone und kontinuierliche Suchvorgänge durchzuführen.

0.2

Der Client wurde um die Funktion erweitert, Ereignisse zu generieren, wenn die Verbindung zum Namensdienst unterbrochen wird. Basierend auf dem Paket **uevent**.

0.1

Erste Implementierung des Clients.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden

oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright

```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nns_common
Stichwörter: nns_common, client, name service, server

Name
nameserv::common – Namensdienst-Funktion, gemeinsame Definitionen

Übersicht
package require Tcl 8.5 9
package require nameserv::common ?**0.2**?
::nameserv::common::port

Beschreibung
Bitte lesen Sie zunächst *Namensdienst-Funktion, Einleitung*.

Dieses Paket ist intern und für Benutzer nicht von Interesse. Es stellt die Befehle der Namensdienstfunktion bereit, die sowohl vom Client als auch vom Server genutzt werden und durch die Pakete **nameserv::server** und **nameserv** (der Client) implementiert werden.

Dieser Dienst wurde auf der Grundlage des Pakets **comm** entwickelt und ist für dieses bestimmt. Er hat nichts mit dem Domain Name System des Internets zu tun. Wenn Sie nach einem Paket suchen, das sich damit befasst, sehen Sie sich bitte die Tcllib-Pakete **dns** und **resolv** an.

API
Das Paket exportiert einen einzigen Befehl, wie im Folgenden festgelegt:

::nameserv::common::port
: Das vom Befehl zurückgegebene Ergebnis ist die Kennung des Standard-TCP/IP-Ports, auf dem ein Namensdienstserver auf eingehende Verbindungen wartet und zu dem ein Namensdienst-Client eine Verbindung herzustellen versucht.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright
```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

nns_intro
Stichwörter: nns_intro, client, name service, server

Name
nns_intro – Namensdienst-Funktion, Einleitung

Beschreibung
nns (Abkürzung für *nano nameservice*) ist eine für das Paket **comm** entwickelte Komponente, die diesem einen einfachen Namensdienst hinzufügt. Sie baut zudem auf **comm** auf und nutzt dieses für den Austausch von Nachrichten zwischen Client- und Server-Komponenten.

Diese Namensdienstfunktion hat nichts mit dem *Domain Name System* des Internets zu tun, das auch als *DNS* bekannt ist. Wenn Sie nach einem Paket suchen, das sich damit befasst, sehen Sie sich bitte eines der Pakete **dns** oder **resolv** an, die beide ebenfalls in Tcllib zu finden sind.

Tcllib stellt zwei Anwendungen und vier Pakete bereit, die zusammenarbeiten und auf verschiedenen Ebenen Zugriff auf die Funktion ermöglichen.

Anwendungen
Die Anwendung **nnsd** stellt einen einfachen Nameserver bereit, der von jedem an jedem beliebigen Ort auf seinem System nach eigenem Ermessen ausgeführt werden kann. Sie dient zudem als Beispiel für die Verwendung des serverseitigen Pakets **nameserv::server**.

Ergänzend zu diesem Server gibt es die Client-Anwendung **nns**. Eine mögliche, wenn auch nicht besonders sinnvolle Anwendung wäre die Eingabe von Name-Port-Bindungen in einen Server über ein Shell-Skript. Dies ist nicht sinnvoll, da Shell-Skripte in der Regel keinen auf **comm** basierenden Dienst bereitstellen.

Der einzige Fall, in dem dies einen gewissen Sinn ergeben würde, wäre ein Shell-Skript, das ein Tcl-Skript FOO umschließt, das **comm** verwendet, um den von FOO genutzten Listening-Port zu registrieren. Doch selbst in diesem Fall wäre es weitaus sinnvoller, FOO so zu erweitern, dass es den Namensdienst direkt nutzt. Wie das geht, kann **nns** sowohl als Beispiel als auch als Vorlage dienen. Darüber hinaus kann es ebenfalls nützlich sein, Nameservice-Abfragen aus Shell-Skripten heraus durchzuführen.

Die dritte Anwendung, **nnslog**, ist eine abgespeckte Version der Client-Anwendung **nns**. Sie ist darauf beschränkt, kontinuierlich nach allen Änderungen zu suchen und alle empfangenen Ereignisse auf stdout zu protokollieren.

Beide Clients nutzen das Paket nameserv::auto, um den Verlust und die Wiederherstellung der Verbindung zum Server automatisch zu handhaben.

Pakete

Die beiden wichtigsten Pakete zur Implementierung des Dienstes sind **nameserv** und **nameserv::server**, d. h. Client und Server. Letzteres verfügt über keine umfangreiche API, sondern lediglich über die notwendigen Funktionen zum Starten, Beenden und zur Konfiguration. Informationen zur Verwendung finden Sie in der Anwendung **nnsd**.

Der einfache Client im Paket **nameserv** stellt die eigentliche API bereit, mit der der Dienst verändert und abgefragt wird. Ein Beispiel für seine Verwendung ist die Anwendung **nns**.

Das zweite Client-Paket, nameserv::auto, hat API-Kompatibilität mit dem Basis-Client und bietet die zusätzliche Funktion, dass es Daten wie gebundene Namen automatisch wiederherstellt, wenn die Verbindung zum Namensdienst unterbrochen und anschließend wiederhergestellt wurde. Das heißt, es erkennt den Ausfall des Servers automatisch und gibt die Daten erneut ein, wenn der Server wieder verfügbar ist.

Das Paket **nameserv::common** ist für Benutzer nicht von Interesse. Es handelt sich um ein internes Paket, das Code und Definitionen enthält, die den Paketen **nameserv** und **nameserv::server** gemeinsam sind.

Alle Pakete verwenden das Paket **uevent** zur Meldung besonderer Umstände über Ereignisse und reservieren das uevent-Tag *nameserv* für ihre ausschließliche Nutzung. Alle ihre Ereignisse werden unter diesem Tag veröffentlicht.

Interne Abläufe

Das Dokument *Namensdienst-Funktion, Client/Server-Protokoll* legt das Protokoll fest, das von den Paketen **nameserv** und **nameserv::server** für die Kommunikation untereinander verwendet wird. Es ist für Nutzer weder der Pakete noch der Anwendungen von Belang.

Entwicklern, die die Namensdienstfunktion ändern und/oder erweitern oder einfach nur deren interne Funktionsweise verstehen möchten, wird jedoch dringend empfohlen, dieses Dokument zu lesen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

nns_protocol

Stichwörter: nns_protocol, comm, name service, protocol

Name

nameserv::protocol – Namensdienst-Funktion, Client/Server-Protokoll

Übersicht

Bind *name data*
Release
Search *pattern*
ProtocolVersion
ProtocolFeatures
Search/Continuous/Start *tag pattern*
Search/Continuous/Stop *tag*
Search/Continuous/Change *tag* **add|remove** *response*

Beschreibung

Die Pakete **nameserv::server**, **nameserv** und **nameserv::common** bieten eine einfache, ungeschützte Namensdienstfunktion für den Einsatz in kleinen, vertrauenswürdigen Umgebungen.

Bitte lesen Sie zunächst *Namensdienst-Funktion, Einleitung*.

Dieses Dokument enthält die Spezifikation des Netzwerkprotokolls, das von Client und Server zur Kommunikation untereinander verwendet wird, und ermöglicht so die Implementierung desselben Protokolls in anderen Sprachen.

Nano-Namensdienstprotokoll, Version 1

Dieses Protokoll definiert den grundlegenden Satz von Nachrichten, die von einem Namensdienst unterstützt werden müssen; dieser wird auch als *Core*-Funktion bezeichnet.

Grundschicht

Die grundlegende Kommunikation zwischen Client und Server erfolgt über das Protokoll zur Fernausführung, das durch das Tcl-Paket **comm** festgelegt wird. Das entsprechende Dokument, in dem das Übertragungsprotokoll beschrieben wird, finden Sie unter *comm_wire*.

Alle über dieses Protokoll ausgetauschten Skripte bestehen aus einzelnen Befehlen in Listenform und können daher als einfache Nachrichten und nicht als Tcl-Befehle interpretiert werden. Die im folgenden Abschnitt aufgeführten Befehle/Nachrichten sind die einzigen Befehle, die von der Serverseite verstanden werden. Befehls- und Variablenersetzungen sind innerhalb der Nachrichten nicht zulässig, d. h., Argumente müssen Literalwerte sein.

Das Protokoll ist synchron. Das heißt, auf jede gesendete Nachricht wird eine Antwort erwartet, die generiert werden muss. Alle Nachrichten werden vom Client gesendet. Der Server sendet keine Nachrichten, sondern lediglich Antworten auf Nachrichten.

Nachrichtenebene

Bind *name data*

Der Client sendet diese Nachricht, wenn er sich beim Dienst mit einem *name* und zugehörigen *data* registriert. Der Server muss eine Fehlerantwort senden, wenn der *name* bereits vergeben ist. Andernfalls muss die Antwort eine leere Zeichenkette sein.

Der Server muss mehrere Namen für denselben Client akzeptieren.

Release

Der Client sendet diese Nachricht, um alle Namen, unter denen er beim Dienst registriert ist, zu löschen. Die Antwort muss stets eine leere Zeichenkette sein.

Search *pattern*

Der Client sendet diese Nachricht, um den Dienst nach Namen zu durchsuchen, die dem Glob-*pattern* entsprechen. Die Antwort muss ein Dictionary sein, das die übereinstimmenden Namen als Schlüssel enthält und diese den Daten zuordnet, die zur **Bind**-Zeit mit ihnen verknüpft waren.

ProtocolVersion

Der Client sendet diese Nachricht, um beim Dienst die höchste Version des Namensdienstprotokolls abzufragen, die dieser unterstützt. Die Antwort muss eine positive ganze Zahl sein.

Server, die ausschließlich das *Nano Name Service Protocol Version 1* unterstützen, geben **1** zurück.

ProtocolFeatures

Der Client sendet diese Nachricht, um beim Dienst die von ihm unterstützten Funktionen des Namensdienstprotokolls abzufragen. Die Antwort muss eine Liste mit den Namen der Funktionen enthalten.

Server, die ausschließlich das *Nano Name Service Protocol Version 1* unterstützen, geben **{Core}** zurück.

Erweiterung des Nano-Namensdienstprotokolls: fortlaufende Suche

Dieses Protokoll definiert einen erweiterten Satz von Nachrichten, die von einem Namensdienst unterstützt werden sollen; diese Funktion wird auch als *Search/Continuous*-Funktion bezeichnet. Diese Funktion definiert zusätzliche Nachrichten zwischen Client und Server und entspricht ansonsten vollständig der Version 1 des Protokolls. Einzelheiten zu unseren Grundlagen finden Sie im letzten Abschnitt.

Ein Dienst, der diese Funktion unterstützt, muss den Namen der Funktion **Search/Continuous** in die Liste der Funktionen aufnehmen, die von der Nachricht *ProtocolFeatures* zurückgegeben wird.

Bei dieser Erweiterung ist das Protokoll asynchron. Für keine der Nachrichten in der Erweiterung wird eine direkte Antwort erwartet. Darüber hinaus beginnt der Server von sich aus mit dem Versenden von Nachrichten, anstatt lediglich auf Nachrichten zu antworten, und der Client muss in der Lage sein, diese Benachrichtigungen zu verarbeiten.

Search/Continuous/Start *tag pattern*

Der Client sendet diese Nachricht, um die Suche nach Namen, die dem Glob-*pattern*

entsprechen, im Dienst zu starten. Im Gegensatz zur regulären *Search*-Anfrage fordert diese den Server auf, die Datenbank kontinuierlich auf das Hinzufügen und Entfernen passender Einträge zu überwachen und den Client über alle derartigen Änderungen zu benachrichtigen. Die jeweilige Suche wird durch das *tag* identifiziert.

Es wird keine direkte Antwort erwartet; vielmehr erwarten die Clients, über Änderungen mittels expliziter *Search/Continuous/Result*-Nachrichten informiert zu werden, die vom Dienst generiert werden.

Ferner wird erwartet, dass die *tag*-Information unverändert an die *Search/Continuous/Result* -Nachrichten weitergegeben wird. Diese Kennzeichnung der Ergebnisse ermöglicht es den Clients, mehrere Suchvorgänge zu starten und zwischen den verschiedenen Ergebnissen zu unterscheiden.

Search/Continuous/Stop *tag*
Der Client sendet diese Nachricht, um die durch das *tag* gekennzeichnete fortlaufende Suche zu beenden.

Search/Continuous/Change *tag* add|remove *response*
Diese Nachricht wird vom Dienst an Clients mit aktiven fortlaufenden Suchen gesendet, um gefundene Änderungen zu übermitteln. Die erste derartige Nachricht für eine neue fortlaufende Suche muss den aktuellen Satz übereinstimmender Einträge enthalten.

Um dies sicherzustellen, muss ein Dienst eine **add**-Nachricht mit einem leeren *response*-Feld generieren, falls zu diesem Zeitpunkt keine übereinstimmenden Einträge vorhanden waren.

Die *response* muss ein Dictionary sein, das die übereinstimmenden Namen als Schlüssel enthält und diese den zum Zeitpunkt von **Bind** zugehörigen Daten zuordnet. Das Argument vor der *response* teilt dem Client mit, ob die Namen in der *response* zum Dienst hinzugefügt oder daraus entfernt wurden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

nns_server

Stichwörter: nns_server, name service, server

Name

nameserv::server – Namensdienst-Funktion, Server

Übersicht

package require Tcl 8.5 9
package require nameserv::server ?**0.3.3**?
package require comm
package require interp
package require logger
::nameserv::server::start
::nameserv::server::stop
::nameserv::server::active?
::nameserv::server::cget -option
::nameserv::server::configure
::nameserv::server::configure -option
::nameserv::server::configure -option *value...*

Beschreibung

Bitte lesen Sie zunächst *Namensdienst-Funktion, Einleitung*.

Dieses Paket enthält eine Implementierung der serverseitigen Komponente der Namensdienstfunktion, die vom Client abgefragt wird, der im Paket **nameserv** bereitgestellt wird. Alle vom Server benötigten Informationen werden im Arbeitsspeicher gehalten. Es gibt keinen persistenten Zustand.

Dieser Dienst wurde auf der Grundlage des Pakets **comm** entwickelt und ist für dieses bestimmt. Er hat nichts mit dem Domain Name System des Internets zu tun. Wenn Sie nach einem Paket suchen, das sich damit befasst, sehen Sie sich bitte die Tcllib-Pakete **dns** und **resolv** an.

Dieser Server unterstützt die Protokollfunktion *Core* sowie seit Version 0.3 auch die Funktion *Search/Continuous*.

API

Das Paket exportiert fünf Befehle, wie im Folgenden angegeben:

::nameserv::server::start
: Dieser Befehl startet den Server und sorgt dafür, dass er auf dem konfigurierten Port auf eingehende Verbindungen wartet. Von nun an können Clients eine Verbindung herstellen und Anfragen senden. Das Ergebnis des Befehls ist die leere Zeichenkette.

  Hinweis: Eingehende Anfragen werden nur dann bearbeitet, wenn die Anwendung, zu der der Server gehört, nach Ausführung dieses Befehls tatsächlich in eine Ereignisschleife wechselt.

::nameserv::server::stop
: Durch die Ausführung dieses Befehls wird der Server angehalten und alle darin gespeicherten Daten werden freigegeben. Bestehende Verbindungen werden beendet, und es werden keine neuen Verbindungen mehr angenommen. Das Ergebnis des Befehls ist die leere Zeichenkette.

::nameserv::server::active?
Dieser Befehl gibt einen booleschen Wert zurück, der den Status des Servers angibt. Das Ergebnis lautet **true**, wenn der Server aktiv ist, d. h. gestartet wurde, und andernfalls **false**.

::nameserv::server::cget -option
Dieser Befehl gibt den aktuell konfigurierten Wert für das angegebene **-option** zurück. Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

::nameserv::server::configure
In dieser Form gibt der Befehl ein Dictionary mit allen unterstützten Optionen und deren aktuellen Werten zurück. Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

::nameserv::server::configure -option
In dieser Form ist der Befehl ein Alias für "**::nameserv::server::cget -option**". Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

::nameserv::server::configure -option *value*...
In dieser Form dient der Befehl dazu, eine oder mehrere der unterstützten Optionen zu konfigurieren. Es muss mindestens eine Option angegeben werden, und auf jede Option folgt ihr neuer Wert. Die Liste der unterstützten Optionen und deren Bedeutung steht im Abschnitt Optionen.

Diese Form kann nur verwendet werden, wenn der Server nicht aktiv ist, d. h. noch nicht gestartet wurde oder bereits gestoppt wurde. Solange der Server aktiv ist, kann er nicht neu konfiguriert werden.

Optionen

Die vom Server unterstützten Optionen dienen der Festlegung des TCP-Ports, auf dem der Server auf Verbindungen wartet, sowie der Einstellung, ob nicht-lokale Verbindungen akzeptiert werden sollen oder nicht. Es handelt sich um folgende Optionen:

-localonly *bool*
Diese Option legt fest, ob nur lokale Verbindungen (-localonly 1) oder auch Remote-Verbindungen (-localonly 0) akzeptiert werden sollen. Standardmäßig werden nur lokale Verbindungen akzeptiert.

-port *number*
Diese Option legt den Port fest, auf dem der Namensdienst nach seinem Start auf eingehende Verbindungen wartet. Es muss sich um eine positive ganze Zahl (> 0) handeln, die nicht größer als 65536 (unsigned short) ist. Der anfängliche Standardwert ist die Zahl, die vom Befehl **::nameserv::server::common::port** zurückgegeben wird, wie sie vom Paket **::nameserv::server::common** bereitgestellt wird.

Geschichte

0.3
Der Server wurde um die Möglichkeit erweitert, asynchrone und kontinuierliche Suchvorgänge durchzuführen.

0.2
Der Name des Schalters "-local" wurde in "-localonly" geändert.

0.1

Erste Implementierung des Servers.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2007-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nntp

nntp

Stichwörter: nntp, news, nntpclient, rfc 1036, rfc 977

Name

nntp – Tcl-Client für das NNTP-Protokoll

Übersicht

package require Tcl 8.5 9
package require nntp ?**0.2.3**?
::nntp::nntp ?*host*? ?*port*? ?*nntpName*?
nntpName **method** ?*arg arg* ...?
nntpName **article** ?*msgid*?
nntpName **authinfo** ?*user*? ?*pass*?
nntpName **body** ?*msgid*?
nntpName **configure**
nntpName **configure** *option*
nntpName **configure** *option value* ...
nntpName **cget** *option*
nntpName **date**
nntpName **group** ?*group*?
nntpName **head** ?*msgid*?
nntpName **help**
nntpName **last**
nntpName **list**
nntpName **listgroup** ?*group*?

nntpName **mode_reader**
nntpName **newgroups** *since*
nntpName **newnews**
nntpName **newnews** *since*
nntpName **newnews** *group* ?*since*?
nntpName **next**
nntpName **post** *article*
nntpName **slave**
nntpName **stat** ?*msgid*?
nntpName **quit**
nntpName **xgtitle** ?*group_pattern*?
nntpName **xhdr** *field* ?*range*?
nntpName **xover** ?*range*?
nntpName **xpat** *field range* ?*pattern_list*?

Beschreibung

Das Paket **nntp** stellt eine einfache, ausschließlich auf Tcl basierende Client-Bibliothek für das NNTP-Protokoll bereit. Es öffnet den Standard-NNTP-Socket auf dem Server und stellt anschließend eine Tcl-API für den Zugriff auf die Befehle des NNTP-Protokolls bereit. Alle Serverfehler werden als Tcl-Fehler zurückgegeben (ausgelöst) und müssen mit dem Tcl-Befehl **catch** abgefangen werden.

Befehle

::nntp::nntp ?*host*? ?*port*? ?*nntpName*?

Der Befehl öffnet eine Socket-Verbindung zum angegebenen NNTP-Server und erzeugt ein neues nntp-Objekt mit einem zugehörigen globalen Tcl-Befehl namens *nntpName*. Mit diesem Befehl lassen sich die verschiedenen NNTP-Protokollbefehle der neuen Verbindung erreichen. Die Vorgabe für *port* ist "119", die Vorgabe für *host* ist "news". Diese Vorgaben lassen sich mit den Umgebungsvariablen **NNTPPORT** beziehungsweise **NNTPHOST** überschreiben.

Einige der Befehle dieses Pakets gehören nicht zu NNTP RFC 977 (http://www.rfc-editor.org/rfc/rfc977.txt) und stehen nicht auf allen NNTP-Servern zur Verfügung (oder sind dort nicht implementiert).

Der Zugriffsbefehl *nntpName* hat die folgende allgemeine Form:
***nntpName* method ?*arg arg ...*?**

Option und die *arg*s bestimmen das genaue Verhalten des Befehls.

***nntpName* article ?*msgid*?**

Fragt den Server nach dem Artikel *msgid* aus der aktuellen Gruppe. Der Artikel kommt als gültige Tcl-Liste zurück, darin die Kopfzeilen, dann eine Leerzeile, dann der Textkörper des Artikels. Jedes Element der Liste ist eine Zeile des Artikels.

***nntpName* authinfo ?*user*? ?*pass*?**

Sendet die Anmeldedaten (Benutzername und Passwort) an den Server.

***nntpName* body ?*msgid*?**

Fragt den Server nach dem Textkörper des Artikels *msgid* aus der aktuellen Gruppe. Der Textkörper des Artikels kommt als gültige Tcl-Liste zurück. Jedes Element der Liste ist eine Zeile des Textkörpers.

***nntpName* configure**
siehe unten ...

nntpName* configure *option
siehe unten ...

nntpName* configure *option value ...
siehe unten ...

nntpName* cget *option
Fragt Optionen des NNTP-Verbindungsobjekts ab und setzt sie. Derzeit wird nur eine Option unterstützt, **-binary**. Ist sie gesetzt, werden Artikel als Binärdaten statt als Text geholt. Betroffen sind nur die Methoden **article** und **body**.

Eine Anwendung ist das Herunterladen von Artikeln mit yEnc-kodierten Bildern. Trotz der Kodierung sind die Daten überwiegend binär, und das Holen als Text verfälscht sie.

Das Paket **yencode** liefert Kodierer und Dekodierer für solche Daten.

***nntpName* date**
Fragt den Server nach dem aktuellen Datum des Servers. Das Datum kommt im Format *YYYYMMDDHHMMSS* zurück.

***nntpName* group ?*group*?**
Setzt auf Wunsch die aktuelle Gruppe und holt Informationen über die gerade gewählte Gruppe. Gibt die geschätzte Anzahl der Artikel in der Gruppe zurück, gefolgt von der Nummer des ersten Artikels in der Gruppe, gefolgt vom letzten Artikel in der Gruppe, gefolgt vom Namen der Gruppe.

***nntpName* head ?*msgid*?**
Fragt den Server nach den Kopfzeilen des Artikels *msgid* aus der aktuellen Gruppe. Die Kopfzeilen des Artikels kommen als gültige Tcl-Liste zurück. Jedes Element der Liste ist eine Zeile der Kopfzeilen.

***nntpName* help**
Holt eine Liste der Befehle, die der News-Server unterstützt, mit dem gerade eine Verbindung besteht.

***nntpName* last**
Setzt den aktuellen Artikelzeiger so, dass er auf die vorherige Nachricht verweist (wenn vorhanden), und gibt die msgid dieser Nachricht zurück.

***nntpName* list**
Gibt eine Tcl-Liste gültiger Newsgroups und der zugehörigen Angaben zurück. Jede Newsgroup ist ein Element der Tcl-Liste in dieser Form:

```
      group last first p
```

Dabei ist der Name der Newsgroup, die Nummer des letzten bekannten Artikels, der derzeit in dieser Newsgroup liegt, die Nummer des ersten Artikels, der derzeit in der Newsgroup liegt, und entweder 'y' oder 'n', je nachdem, ob das Posten in dieser Newsgroup erlaubt ('y') oder verboten ('n') ist.

Die Felder und sind immer numerisch. Sie können führende Nullen haben. Fällt kleiner aus als, liegen derzeit keine Artikel in der Newsgroup.

***nntpName* listgroup ?*group*?**
Fragt den Server nach einer Liste aller Nachrichten (Nachrichtennummern) in der Gruppe, die das Argument *group* angibt, oder in der aktuellen Gruppe, wenn *group* nicht übergeben wurde.

***nntpName* mode_reader**
Fragt den Server nach seiner NNTP-Antwortzeichenfolge für 'MODE READER'.

nntpName* newgroups *since
Fragt den Server nach einer Liste aller neuen Newsgroups, die seit dem Zeitpunkt im Argument *since* angelegt wurden. *since* kann jede Zeitangabe sein, die **clock scan** versteht. Die Tcl-Liste der Newsgroups kommt in ähnlicher Form zurück wie die Gruppenliste des Befehls **nntpName list**. Jedes Element der Liste hat die Form:

```
group last first p
```

Dabei ist der Name der Newsgroup, die Nummer des letzten bekannten Artikels, der derzeit in dieser Newsgroup liegt, die Nummer des ersten Artikels, der derzeit in der Newsgroup liegt, und entweder 'y' oder 'n', je nachdem, ob das Posten in dieser Newsgroup erlaubt ('y') oder verboten ('n') ist.

***nntpName* newnews**
Fragt den Server nach einer Liste neuer Artikel, die am letzten Tag in der aktuellen Gruppe gepostet wurden.

nntpName* newnews *since
Fragt den Server nach einer Liste neuer Artikel, die seit dem Zeitpunkt im Argument *since* in der aktuellen Gruppe gepostet wurden. *since* kann jede Zeitangabe sein, die **clock scan** versteht.

***nntpName* newnews *group* ?*since*?**
Fragt den Server nach einer Liste neuer Artikel, die seit dem Zeitpunkt im Argument *since* in der durch *group* angegebenen Gruppe gepostet wurden (oder im vergangenen Tag, wenn kein Argument *since* übergeben wird). *since* kann jede Zeitangabe sein, die **clock scan** versteht.

***nntpName* next**
Setzt den aktuellen Artikelzeiger so, dass er auf die nächste Nachricht verweist (sofern vorhanden), und gibt die msgid dieser Nachricht zurück.

nntpName* post *article
Veröffentlicht einen Artikel in dem in RFC 1036 (http://www.rfc-editor.org/rfc/rfc1036.txt, Nachfolger von RFC 850) festgelegten Format in der aktuellen Newsgroup.

***nntpName* slave**
Kennzeichnet eine Verbindung als von einem Slave-NNTP-Server kommend. Das kann anzeigen, dass die Verbindung mehrere Personen bedient und Vorrang bekommen sollte. Die tatsächliche Verwendung hängt ganz von der Implementierung ab und kann von Server zu Server verschieden sein.

***nntpName* stat ?*msgid*?**
Der Befehl stat ähnelt dem Befehl article, nur dass kein Text zurückgegeben wird. Bei der Auswahl anhand der Nachrichtennummer innerhalb einer Gruppe setzt stat den aktuellen Artikelzeiger, ohne Text zu senden. Die zurückgegebene Bestätigung enthält die message-id, was von gewissem Wert sein kann. stat zur Auswahl anhand der message-id ist zulässig, aber

von zweifelhaftem Wert, denn eine Auswahl anhand der message-id ändert den "aktuellen Artikelzeiger" NICHT.

***nntpName* quit**
Schließt die Verbindung geordnet, nachdem ein NNTP-Befehl QUIT über den Socket gesendet wurde.

***nntpName* xgtitle ?*group_pattern*?**
Gibt eine Tcl-Liste zurück, in der jedes Element diese Form hat:

```
newsgroup description
```

Ist ein *group_pattern* angegeben, kommen nur Name und Beschreibung der Newsgroups zurück, die dem Muster entsprechen.

***nntpName* xhdr *field* ?*range*?**
Gibt den Wert des angegebenen Kopffeldes für die aktuelle Nachricht oder für eine Liste von Nachrichten der aktuellen Gruppe zurück. *field* ist der Titel eines Feldes in der Kopfzeile, etwa from, subject, date. Ist *range* nicht angegeben oder "", wird die aktuelle Nachricht abgefragt. Der Befehl gibt eine Liste von Elementen zurück, jedes in der Form:

```
    msgid value
```

msgid ist die Nummer der Nachricht, value der Wert des abgefragten Feldes. Das Argument *range* kann eine dieser Formen haben:

""
Die aktuelle Nachricht wird abgefragt.

msgid1-msgid2
Es werden alle Nachrichten zwischen *msgid1* und *msgid2* (einschließlich *msgid1* und *msgid2*) abgefragt.

msgid1 msgid2
Es werden alle Nachrichten zwischen *msgid1* und *msgid2* (einschließlich *msgid1* und *msgid2*) abgefragt.

***nntpName* xover ?*range*?**
Gibt Kopfinformationen für die aktuelle Nachricht oder für einen Bereich von Nachrichten der aktuellen Gruppe zurück. Die Angaben kommen als Tcl-Liste zurück, jedes Element in der Form:

```
    msgid subject from date idstring bodysize headersize xref
```

Ist *range* nicht angegeben oder "", wird die aktuelle Nachricht abgefragt. Das Argument *range* kann eine dieser Formen haben:

""
Die aktuelle Nachricht wird abgefragt.

msgid1-msgid2
Es werden alle Nachrichten zwischen *msgid1* und *msgid2* (einschließlich *msgid1* und *msgid2*) abgefragt.

msgid1 msgid2
Es werden alle Nachrichten zwischen *msgid1* und *msgid2* (einschließlich *msgid1* und

msgid2) abgefragt.

***nntpName* xpat *field range* ?*pattern_list*?**

Gibt den Wert des angegebenen Kopffeldes für eine bestimmte Nachricht oder für eine Liste von Nachrichten der aktuellen Gruppe zurück, deren Nachrichten zu den Mustern in pattern_list passen. *field* ist der Titel eines Feldes in der Kopfzeile, etwa from, subject, date. Die Angaben kommen als Tcl-Liste zurück, jedes Element in der Form:

```
msgid value
```

msgid ist die Nummer der Nachricht, value der Wert des abgefragten Feldes. Das Argument *range* kann eine dieser Formen haben:

msgid

Die durch *msgid* festgelegte Nachricht wird abgefragt.

msgid1-msgid2

Es werden alle Nachrichten zwischen *msgid1* und *msgid2* (einschließlich *msgid1* und *msgid2*) abgefragt.

msgid1 msgid2

Es werden alle Nachrichten zwischen *msgid1* und *msgid2* (einschließlich *msgid1* und *msgid2*) abgefragt.

Beispiel

Ein größeres Beispiel zum Versenden eines einzelnen Artikels.

```
package require nntp
set n [nntp::nntp NNTP_SERVER]
$n post "From: USER@DOMAIN.EXT (USER_FULL)
Path: COMPUTERNAME!USERNAME
Newsgroups: alt.test
Subject: Tcl test post -ignore
Message-ID: <[pid][clock seconds]
@COMPUTERNAME>
Date: [clock format [clock seconds] -format "%a, %d %
b %y %H:%M:%S GMT" -gmt true]

Test message body"
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nntp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

ntp

ntp_time
Stichwörter: ntp_time, NTP, SNTP, rfc 2030, rfc 868, time

Name
ntp_time – Tcl-Zeitdienst-Client

Übersicht
package require Tcl 8.5 9
package require time ?**1.2.2**?
::time::gettime ?*options*? *timeserver* ?*port*?
::time::getsntp ?*options*? *timeserver* ?*port*?
::time::configure ?*options*?
::time::cget *name*
::time::unixtime *token*
::time::status *token*
::time::error *token*
::time::reset *token* ?*reason*?
::time::wait *token*
::time::cleanup *token*

Beschreibung
Dieses Paket implementiert einen Client für das RFC 868 TIME-Protokoll (http://www.rfc-editor.org/rfc/rfc868.txt) sowie einen Minimal-Client für das RFC 2030 Simple Network Time Protocol (http://www.rfc-editor.org/rfc/rfc2030.txt). RFC 868 gibt die Zeit in Sekunden seit dem 1. Januar 1900 entweder an TCP- oder an UDP-Clients zurück. RFC 2030 liefert ebenfalls diese Zeit, dazu einen Bruchteil, den dieser Client nicht verwendet.

Befehle
::time::gettime ?*options*? *timeserver* ?*port*?
Holt die Zeit von *timeserver*. Hier dürfen die Optionen stehen, die der Befehl **configure** kennt. Der Befehl gibt ein Token zurück; die übrigen Befehle dieses Pakets brauchen es. Danach gibt **cleanup** alle Ressourcen frei. Der voreingestellte Port ist **37**.

::time::getsntp ?*options*? *timeserver* ?*port*?
Holt die Zeit von einem SNTP-Server. Dieser Befehl akzeptiert genau dieselben Argumente wie **::time::gettime**, nur ist der voreingestellte Port **123**. Das Ergebnis ist ein Token wie bei **::time::gettime** und wird genauso verwendet.

Es ist unwahrscheinlich, dass ein SNTP-Server über tcp antwortet; Sie brauchen dafür das Paket **tcludp** oder **ceptcl**. Lässt sich ein passendes Paket laden, wird standardmäßig udp verwendet.

::time::configure ?*options*?

Ohne Argumente gibt dieser Befehl alle aktuellen Konfigurationsoptionen und ihre Werte zurück. Sonst ist er mit Paaren aus Optionsname und Wert aufzurufen.

-protocol *number*
Legt das voreingestellte Netzwerkprotokoll fest. Voreingestellt ist udp, wenn das Paket tcludp verfügbar ist. Sonst wird tcp verwendet.

-port *number*
Legt den voreingestellten Port fest. RFC 868 verwendet Port **37**, RFC 2030 Port **123**.

-timeout *number*
Legt den voreingestellten Timeout in Millisekunden fest. Die Vorgabe ist 10 Sekunden.

-command *number*
Legt eine Befehlsprozedur fest, die bei einer Antwort läuft. Die Prozedur wird aufgerufen, das Zeit-Token hängt an der Argumentliste.

-loglevel *number*
Legt die Protokollierungsstufe fest. Die Vorgabe ist 'warning'.

::time::cget *name*
Gibt den aktuellen Wert der benannten Konfigurationsoption zurück.

::time::unixtime *token*
Formatiert die zurückgegebene Zeit für die Unix-Epoche. RFC 868 setzt den Zeitpunkt 0 auf den 1. Januar 1900, die Unix-Zeit setzt ihn auf den 1. Januar 1970. Dieser Befehl rechnet die Antwort in die Unix-Zeit um.

::time::status *token*
Gibt das Statusflag zurück. Nach einer erfolgreich abgeschlossenen Abfrage ist es *ok*. Es kann auch *error*, *timeout* oder *eof* sein. Siehe auch **::time::error**

::time::error *token*
Gibt die Fehlermeldung zu Anfragen mit dem Status *error* zurück. Liegt keine Fehlermeldung vor, kommt eine leere Zeichenkette zurück.

::time::reset *token?reason?*
Setzt die Abfrage zurück oder bricht sie ab. Optional lässt sich der Grund angeben, den der Befehl **error** festhält.

::time::wait *token*
Wartet, bis eine Abfrage abgeschlossen ist, und gibt danach den Status zurück.

::time::cleanup *token*
Entfernt alle Zustandsvariablen, die zu der Anfrage gehören.

```
% set tok [::time::gettime ntp2a.mcc.ac.uk]
% set t [::time::unixtime $tok]
% ::time::cleanup $tok
```

```
% set tok [::time::getsntp pool.ntp.org]
% set t [::time::unixtime $tok]
% ::time::cleanup $tok
```

```
proc on_time {token} {
   if {[time::status $token] eq "ok"} {
      puts [clock format [time::unixtime $token]]
   } else {
      puts [time::error $token]
   }
   time::cleanup $token
}
time::getsntp -command on_time pool.ntp.org
```

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ntp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright

```
Copyright (c) 2002, Pat Thoyts <patthoyts@users.sourceforge.net>
```


oauth

oauth
Stichwörter: oauth, RFC 2718, RFC 5849, twitter

Name
oauth – Basissignatur der OAuth-API

Übersicht
package require Tcl 8.5 9
package require oauth ?**1.0.4**?
::oauth::config
::oauth::config ?*options...*?
::oauth::header *baseURL* ?*postQuery*?
::oauth::query *baseURL* ?*postQuery*?

Beschreibung

Das Paket **oauth** stellt eine einfache, nur in Tcl geschriebene Bibliothek für die Kommunikation mit oauth-APIs bereit. Die aktuelle Version des Pakets unterstützt das OAuth-1.0-Protokoll, wie in RFC 5849 festgelegt.

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Befehle

::oauth::config
Wird dieser Befehl ohne Argumente aufgerufen, gibt er ein Dictionary zurück, das die aktuellen Werte aller Optionen enthält.

::oauth::config ?*options*...?
Mit Argumenten aufgerufen, Optionen gefolgt von ihren Werten, legt der Befehl verschiedene Parameter von Anwendung und Client fest und fragt sie ab, etwa den Proxy-Host und den User-Agent der HTTP-Anfragen. Die einzelnen Optionen stehen unten:

-accesstoken *string*
Dies ist das Token des Benutzers.

-accesstokensecret *string*
Dies ist das geheime Token des Benutzers.

-consumerkey *string*
Dies ist der öffentliche Token Ihrer App.

-consumersecret *string*
Dies ist das private Token Ihrer App.

-debug *bool*
Der Standardwert ist **off**. Wenn Sie die Option auf **on** setzen, wird die gerade erzeugte Basissignatur zusammen mit weiterer Debug-Ausgabe auf stdout geschrieben.

-oauthversion *version*
Dies ist die zu verwendende Version des OAuth-Protokolls. Derzeit wird ausschließlich **1.0**

unterstützt, die Standardversion.

-proxyhost *hostname*
Sie können einen Proxy-Host einrichten, über den die Verbindung zum API-Server von OAuth läuft.

-proxyport *port-number*
Portnummer Ihres Proxys.

-signmethod *method*
Die zu verwendende Signaturmethode. OAuth 1.0 unterstützt ausschließlich **HMAC-SHA1**, die Standardmethode.

-timeout *milliseconds*
Zeitlimit in Millisekunden für Ihre Abfrage. Der Standardwert ist **6000**, das heißt 6 Sekunden.

-urlencoding *encoding*
Die Kodierung, mit der **::http::formatQuery** die x-url-kodierten URLs erzeugt. Die Vorgabe ist **utf-8**, wie in RFC 2718 festgelegt.

::oauth::header *baseURL* ?*postQuery*?
Dieser Befehl erzeugt die Basissignatur. Mit passenden Einstellungen für die verschiedenen Tokens und Geheimnisse (siehe **::oauth::config**) ist das Ergebnis die Basis-Authentifizierungszeichenkette, die an den Server gesendet wird.

Sie müssen diese Prozedur nicht aufrufen, um die Abfrage zu erzeugen, denn **::oauth::query** (siehe unten) übernimmt das. Für die Fehlersuche ist der Aufruf dennoch nützlich.

url *baseURL*
Dieses Argument ist der URI-Pfad zum OAuth-API-Server. Wenn Sie eine GET-Anfrage senden wollen, geben Sie einen vollständigen Pfad an.

```
HTTP GET
::oauth::header {https://api.twitter.com/1.1/users/lookup.json?scre
    en_name=AbiertaMente}
```

url-encoded-string *postQuery*
Wenn Sie einen Header im POST-Format senden müssen, setzen Sie die Abfragezeichenkette in dieses Argument.

```
::oauth::header {https://api.twitter.com/1.1/friendships/create.jso
    n} {user_id=158812437&follow=true}
```

::oauth::query *baseURL* ?*postQuery*?
Diese Prozedur verwendet die Einstellungen von **::oauth::config**, erzeugt damit die Basisauthentifizierung und sendet anschließend den Befehl an die Server-API. Sie nimmt dieselben Argumente entgegen wie **::oauth::header**.

Das Ergebnis ist eine Liste aus zwei Elementen. Das erste Element ist ein Dictionary mit den HTTP-Header-Daten der Antwort. Damit können Sie beispielsweise den X-Rate-Limit von OAuth prüfen. Das zweite Element sind die Rohdaten, die der API-Server zurückgibt. Diese Zeichenkette ist in der Regel ein JSON-Objekt und lässt sich mit den Funktionen des Pakets **json** oder einem anderen JSON-Parser für Tcl weiter dekodieren.

Hier ein Beispiel, wie das bei Twitter abläuft. Vergessen Sie nicht, die Platzhalter-Tokens und -Schlüssel des Beispiels durch Ihre eigenen zu ersetzen, wenn Sie es ausprobieren.

```
% package require oauth
% package require json
% oauth::config -consumerkey {your_consumer_key} -consumersecret
    {your_consumer_key_secret} -accesstoken {your_access_token}
    -accesstokensecret {your_access_token_secret}

% set response [oauth::query
    https://api.twitter.com/1.1/users/lookup.json?screen_name=Abierta
    Mente]
% set jsondata [lindex $response 1]
% set data [json::json2dict $jsondata]
$ set data [lindex $data 0]
% dict for {key val} $data {puts "$key => $val"}
id => 158812437
id_str => 158812437
name => Un Librepensador
screen_name => AbiertaMente
location => Explico mis tuits ahí →
description => 160Caracteres para un SMS y contaba mi vida entera
    sin recortar vocales. Ahora en Twitter, podemos usar hasta 140 y
    a mí me sobrarían 20 para contaros todo lo q
url => http://t.co/SGs3k9odBn
entities => url {urls {{url http://t.co/SGs3k9odBn expanded_url
    http://librepensamiento.es display_url librepensamiento.es
    indices {0 22}}}} description {urls {}}
protected => false
followers_count => 72705
friends_count => 53099
listed_count => 258
created_at => Wed Jun 23 18:29:58 +0000 2010
favourites_count => 297
utc_offset => 7200
time_zone => Madrid
geo_enabled => false
verified => false
statuses_count => 8996
lang => es
status => created_at {Sun Oct 12 08:02:38 +0000 2014} id
    521209314087018496 id_str 521209314087018496 text
    {@thesamethanhim http://t.co/WFoXOAofCt} source {<a
    href="http://twitter.com" rel="nofollow">Twitter Web Client</a>}
    truncated false in_reply_to_status_id 521076457490350081
    in_reply_to_status_id_str 521076457490350081 in_reply_to_user_id
```

```
    2282730867 in_reply_to_user_id_str 2282730867
    in_reply_to_screen_name thesamethanhim geo null coordinates null
    place null contributors null retweet_count 0 favorite_count 0
    entities {hashtags {} symbols {} urls {{url
    http://t.co/WFoXOAofCt expanded_url
    http://www.elmundo.es/internacional/2014/03/05/53173dc1268e3e3f23
    8b458a.html display_url elmundo.es/internacional/… indices {16
    38}}} user_mentions {{screen_name thesamethanhim name Ἑλένη id
    2282730867 id_str 2282730867 indices {0 15}}}} favorited false
    retweeted false possibly_sensitive false lang und
contributors_enabled => false
is_translator => true
is_translation_enabled => false
profile_background_color => 709397
profile_background_image_url =>
    http://pbs.twimg.com/profile_background_images/704065051/9309c02a
    a2728bdf543505ddbd408e2e.jpeg
profile_background_image_url_https =>
    https://pbs.twimg.com/profile_background_images/704065051/9309c02
    aa2728bdf543505ddbd408e2e.jpeg
profile_background_tile => true
profile_image_url =>
    http://pbs.twimg.com/profile_images/2629816665/8035fb81919b840c5c
    c149755d3d7b0b_normal.jpeg
profile_image_url_https =>
    https://pbs.twimg.com/profile_images/2629816665/8035fb81919b840c5
    cc149755d3d7b0b_normal.jpeg
profile_banner_url =>
    https://pbs.twimg.com/profile_banners/158812437/1400828874
profile_link_color => FF3300
profile_sidebar_border_color => FFFFFF
profile_sidebar_fill_color => A0C5C7
profile_text_color => 333333
profile_use_background_image => true
default_profile => false
default_profile_image => false
following => true
follow_request_sent => false
notifications => false
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *oauth* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright
```
Copyright (c) 2014 Javi P. <hxm@eggdrop.es>
```

oometa

oometa
Stichwörter: oometa, TOOL, TclOO

Name
oometa – oo::meta Ein Datenregister für Klassen

Übersicht
oo::meta::info
oo::meta::info branchget ?*key*? ?*...*?
oo::meta::info branchset ?*key...*? *key value*
oo::meta::info dump *class*
oo::meta::info *class* **is** *type* ?*args*?
oo::meta::info *class* **merge** ?*dict*? ?*dict*? ?*...*?
oo::meta::info *class* **rebuild**
oo::meta::metadata *class*
oo::define meta
oo::class method meta
oo::object method meta
oo::object method meta cget ?*field*? ?*...*? *field*

Beschreibung
Das Paket "**oo::meta**" stellt einen Datenregistrierungsdienst für TclOO-Klassen bereit.

Verwendung
```
oo::class create animal {
  meta set biodata animal: 1
}
oo::class create mammal {
  superclass animal
  meta set biodata mammal: 1
}
oo::class create cat {
  superclass mammal
  meta set biodata diet: carnivore
```

```
}

cat create felix
puts [felix meta dump biodata]
> animal: 1 mammal: 1 diet: carnivore

felix meta set biodata likes: {birds mice}
puts [felix meta get biodata]
> animal: 1 mammal: 1 diet: carnivore likes: {bird mice}

# Modify a class
mammal meta set biodata metabolism: warm-blooded
puts [felix meta get biodata]
> animal: 1 mammal: 1 metabolism: warm-blooded diet: carnivore likes:
    {birds mice}

# Overwrite class info
felix meta set biodata mammal: yes
puts [felix meta get biodata]
> animal: 1 mammal: yes metabolism: warm-blooded diet: carnivore
    likes: {birds mice}
```

Konzept

Das Konzept hinter **oo::meta** besteht darin, dass jede Klasse einen Ausschnitt *lokaler* Daten beisteuert. Wenn **oo::meta::metadata** aufgerufen wird, durchläuft das System die von **oo::meta::ancestors** erzeugte lineare Abstammungsstruktur und fasst alle diese lokalen Daten für alle Vorfahren einer Klasse rekursiv zu einem einzigen Dict zusammen. Instanzen von oo::object können zudem Daten einer Klasse mit einem lokalen Dict kombinieren, das in der *meta*-Variablen gespeichert ist.

Befehle

oo::meta::info

oo::meta::info ist darauf ausgelegt, die Metadaten einer Klasse auf ähnliche Weise zu bearbeiten, wenn die einzelnen Elemente zu einem einzigen Dict zusammengefügt worden sind. Das System unterstützt alle Standard-Befehle für Dicts und bietet darüber hinaus folgende zusätzliche Funktionen:

oo::meta::info branchget ?*key*? ?...?

Gibt eine Dict-Darstellung des Elements an der Position *args* zurück, wobei jedoch alle abschließenden ":" aus den Feldnamen entfernt werden.

```
::oo::meta::info $myclass set option color {default: green widget:
    colorselect}
puts [::oo::meta::info $myclass get option color]
> {default: green widget: color}
puts [::oo::meta::info $myclass branchget option color]
> {default green widget color}
```

oo::meta::info branchset ?*key...*? *key value*
Führt *dict* mit allen anderen Informationen zusammen, die am Knoten ?*key...*? enthalten sind, und fügt allen Feldnamen ein abschließendes: hinzu.

```
::oo::meta::info $myclass branchset option color {default green
    widget colorselect}
puts [::oo::meta::info $myclass get option color]
> {default: green widget: color}
```

oo::meta::info dump *class*
Gibt den vollständigen Snapshot der Metadaten einer Klasse zurück, wie er von **oo::meta::metadata** erstellt wurde

oo::meta::info *class* is *type* ?*args*?
Gibt den booleschen Wert "true" oder "false" zurück, wenn das Element "?*args*?" mit **string is** *type value* übereinstimmen würde.

```
::oo::meta::info $myclass set constant mammal 1
puts [::oo::meta::info $myclass is true constant mammal]
> 1
```

oo::meta::info *class* merge ?*dict*? ?*dict*? ?*...*?
Führt alle Argumente zu einem einzigen Dict zusammen, das anschließend als neue lokale Darstellung für diese Klasse gespeichert wird.

oo::meta::info *class* rebuild
Zwingt das Metasystem dazu, alle zwischengespeicherten Darstellungen der Metadaten einer Klasse vor dem nächsten Zugriff auf **oo::meta::metadata** zu löschen

oo::meta::metadata *class*
Gibt ein Gesamtbild der Metadaten für *class* zurück, wobei deren *lokale* Daten mit den *lokalen* Daten ihrer Vorfahren kombiniert werden.

oo::define meta
Das Paket fügt den Befehl "**oo::define::meta**" ein, der es einer Klasse ermöglicht, während der Definition auf "**oo::meta::info**" zuzugreifen, ohne den Namen nachschlagen zu müssen.

```
oo::define myclass {
  meta set foo bar: baz
}
```

oo::class method meta
Das Paket fügt eine neue Methode "**meta**" in "**oo::class**" ein, die den Zugriff einer Instanz der Klasse auf "**oo::meta::info**" ermöglicht.

oo::object method meta
Das Paket fügt eine neue Methode **meta** in **oo::object** ein. **oo::object** kombiniert die Daten für seine Klasse (wie von **oo::meta::metadata** bereitgestellt) mit einer lokalen Variablen *meta*, um ein lokales Abbild der Metadaten zu erstellen. Diese Methode stellt die folgenden zusätzlichen Befehle bereit:

oo::object method meta cget ?*field*? ?*...*? *field*
Versucht, ein einzelnes Blatt zu finden und dessen Wert zurückzugeben. Bei der Suche nach

einer einzelnen Option ist dies schneller als **my meta getnull** ?*field*? ?*...*? *field*], da hier direkt eine Suche durchgeführt wird, anstatt zunächst das rekursive Merge-Produkt zwischen den Klassenmetadaten, der lokalen *meta*-Variablen zu erzeugen und erst danach die Suche durchzuführen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *tcloo* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

TclOO

Copyright


```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```


ooutil

ooutil

Stichwörter: ooutil, TclOO, callback, class methods, class variables, command prefix, currying, method reference, my method, singleton

Name

oo::util – Hilfsbefehle für TclOO

Übersicht

package require Tcl 8.5 9
package require TclOO
package require oo::util ?**1.2.3**?
mymethod *method* ?*arg...*?
classmethod *name arguments body*
classvariable ?*arg...*?
link *method...*
link {alias method}...
ooutil::singleton ?*arg...*?

Beschreibung

Dieses Paket bietet einen Komfortbefehl zur einfachen Festlegung von Instanzmethoden als Rückruffunktionen, wie beispielsweise Timer, Dateiereignisse, Tk-Bindungen usw.

Befehle

mymethod *method* ?*arg...*?

Dieser Befehl steht innerhalb von Instanzmethoden zur Verfügung. Er nimmt einen Methodennamen und gegebenenfalls Argumente für die Methode entgegen und gibt ein Befehlspräfix zurück, das bei seiner Ausführung die benannte Methode des Objekts, in dem wir uns gerade befinden, mit den übergebenen Argumenten sowie allen weiteren Argumenten aufruft, die zum Zeitpunkt des tatsächlichen Aufrufs bereitgestellt werden.

Hinweis: Der Befehl entspricht dem Befehl, der vom OO-Paket **snit** für denselben Zweck bereitgestellt wird, und ist nach diesem benannt.

classmethod *name arguments body*

Dieser Befehl steht innerhalb von Klassendefinitionen zur Verfügung. Er nimmt einen Namen für die Methode und gegebenenfalls Argumente für die Methode entgegen und erstellt eine Methode in der Klasse, die für Benutzer dieser Klasse sowie abgeleiteter Klassen verfügbar ist.

Hinweis: Der Befehl entspricht dem Befehl **typemethod**, der vom OO-Paket **snit** für denselben Zweck bereitgestellt wird.

Beispiel

```
oo::class create ActiveRecord {
    classmethod find args { puts "[self] called with arguments:
    $args" }
}
oo::class create Table {
    superclass ActiveRecord
}
puts [Table find foo bar]
# ======
# which will write
# ======
# ::Table called with arguments: foo bar
```

classvariable ?*arg*...?

Dieser Befehl steht innerhalb von Instanzmethoden zur Verfügung. Er nimmt eine Reihe von Variablennamen entgegen und macht diese im Gültigkeitsbereich der Methode verfügbar. Der ursprüngliche Gültigkeitsbereich der Variablen ist die Klasse (Instanz), zu der die Objektinstanz gehört. Mit anderen Worten: Die referenzierten Variablen werden von allen Instanzen ihrer Klasse gemeinsam genutzt.

Hinweis: Der Befehl entspricht in etwa dem Befehl **typevariable**, der vom OO-Paket **snit** für denselben Zweck bereitgestellt wird. Der Unterschied besteht darin, dass er nicht in der Klassendefinition selbst verwendet werden kann.

Beispiel:

```
% oo::class create Foo {
    method bar {z} {
        classvariable x y
        return [incr x $z],[incr y]
```

```
    }
}
::Foo
% Foo create a
::a
% Foo create b
::b
% a bar 2
2,1
% a bar 3
5,2
% b bar 7
12,3
% b bar -1
11,4
% a bar 0
11,5
```

link *method*...
siehe unten ...

link {*alias method*}...
Dieser Befehl steht innerhalb von Instanzmethoden zur Verfügung. Er nimmt eine Liste von Methodennamen und/oder Paaren aus Alias- und Methodennamen entgegen und macht die genannten Methoden für alle Instanzmethoden verfügbar, ohne dass der Befehl **my** erforderlich ist.

Der Aliasname, unter dem die Methode verfügbar wird, entspricht standardmäßig dem Methodennamen, sofern nicht ausdrücklich ein Alias-Methoden-Paar angegeben wurde.

Beispiele:

```
    link foo
    # The method foo is now directly accessible as foo instead of my
    foo.

    link {bar foo}
    # The method foo is now directly accessible as bar.

    link a b c
    # The methods a, b, and c all become directly acessible under
    their
    # own names.
```

Dieser Befehl wird voraussichtlich vor allem in Instanz-Konstruktoren verwendet werden – entweder aus Gründen der Einfachheit oder um bestimmte Methoden für die Verwendung in einer Mini-DSL einzurichten.

ooutil::singleton ?*arg*...?
Dieser Befehl ist eine Metaklasse, d. h. eine Variante des integrierten Befehls **oo::class**, die

sicherstellt, dass nur eine einzige Instanz der damit definierten Klassen erstellt wird.

Syntax und Ergebnisse entsprechen denen von **oo::class**.

Beispiel:

```
% oo::class create example {
   self mixin singleton
   method foo {} {self}
}
::example
% [example new] foo
::oo::Obj22
% [example new] foo
::oo::Obj22
```

Autoren
Donal Fellows, Andreas Kupries

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *oo::util* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hilfsprogramme

Copyright

```
Copyright (c) 2011-2015 Andreas Kupries, BSD licensed
```


otp

otp
Stichwörter: otp, hashing, message-digest, password, rfc 2289, security

Name
otp – Einmalpasswörter

Übersicht
package require Tcl 8.5 9
package require otp ?**1.1.0**?
::otp::otp-md4 ?*-hex*? ?*-words*? *-seed seed -count count data*
::otp::otp-md5 ?*-hex*? ?*-words*? *-seed seed -count count data*

::otp::otp-sha1 ?*-hex*? ?*-words*? *-seed seed -count count data*
::otp::otp-rmd160 ?*-hex*? ?*-words*? *-seed seed -count count data*

Beschreibung

Dieses Paket ist eine Implementierung des in RFC 2289 (1) beschriebenen Einmalpasswort-Systems in Tcl. Dieses System verwendet Message-Digest-Algorithmen, um eine Passphrase schrittweise zu hashen und so Einmalpasswörter zu erzeugen. Die resultierenden Daten werden dem Benutzer dann entweder als Hexadezimalziffern oder – durch Kodierung mit einem Dictionary mit 2048 Wörtern – bereitgestellt. Dieses System wird von OpenBSD für die Sicherheit der Anmeldung verwendet und kann als SASL-Mechanismus zur Authentifizierung von Benutzern eingesetzt werden.

In dieser Implementierung bieten wir Unterstützung für vier Algorithmen, die in der tcllib-Distribution enthalten sind: MD5 (2), MD4 (3), RIPE-MD160 (4) und SHA-1 (5).

Befehle

- **::otp::otp-md4** ?*-hex*? ?*-words*? *-seed seed -count count data*
- **::otp::otp-md5** ?*-hex*? ?*-words*? *-seed seed -count count data*
- **::otp::otp-sha1** ?*-hex*? ?*-words*? *-seed seed -count count data*
- **::otp::otp-rmd160** ?*-hex*? ?*-words*? *-seed seed -count count data*

Beispiele

```
% otp::otp-md5 -count 99 -seed host67821 "My Secret Pass Phrase"
(binary gibberish)
% otp::otp-md5 -words -count 99 -seed host67821 "My Secret Pass Phrase"
SOON ARAB BURG LIMB FILE WAD
% otp::otp-md5 -hex -count 99 -seed host67821 "My Secret Pass Phrase"
e249b58257c80087
```

Literatur

1. Haller, N. et al., "A One-Time Password System", RFC 2289, Februar 1998. http://www.rfc-editor.org/rfc/rfc2289.txt
2. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT und RSA Data Security, Inc., April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
3. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
4. H. Dobbertin, A. Bosselaers, B. Preneel, "RIPEMD-160, eine erweiterte Version von RIPEMD" http://www.esat.kuleuven.ac.be/~cosicart/pdf/AB-9601/AB-9601.pdf
5. "Secure Hash Standard", National Institute of Standards and Technology, US-Handelsministerium, April 1995. (http://www.itl.nist.gov/fipspubs/fip180-1.htm)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *otp* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```


page

page_intro
Stichwörter: page_intro, page, parser generator, text processing

Name
page_intro – Einleitung zu page

Beschreibung
page (kurz für *parser generator*) steht für eine Reihe verwandter Pakete, die bei der Erstellung von Parser-Generatoren helfen, und für andere Hilfsprogramme, die Textverarbeitung betreiben.

Sie dienen in erster Linie der Unterstützung der Tcllib-Anwendung **page**, wobei das Paket **page::pluginmgr** eine zentrale Rolle als Plugin-Verwaltung für die Anwendung spielt. Die übrigen Pakete führen Textverarbeitungsaufgaben auf niedriger Ebene sowie Hilfsaufgaben durch, die auf die Generierung von Parsern ausgerichtet sind, und **page** greift auf sie hauptsächlich über Plugins zu.

Die Pakete, die die Plugins implementieren, sind nicht wie gewöhnliche Pakete dokumentiert, da sie ohne umfangreiche Vorbereitungsmaßnahmen am Interpreter nicht in einen allgemeinen Interpreter wie tclsh geladen werden können. Diese Vorbereitungsmaßnahmen werden vom Plugin-Manager für sie vorgenommen.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Page-Parsergenerator

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

page_pluginmgr

Stichwörter: page_pluginmgr, page, parser generator, text processing

Name

page_pluginmgr – page-Plugin-Manager

Übersicht

package require page::pluginmgr ?**0.3**?
package require fileutil
::page::pluginmgr::reportvia *cmd*
::page::pluginmgr::report *level text* ?*from ?to*?
::page::pluginmgr::log *cmd*
::page::pluginmgr::configuration *name*
::page::pluginmgr::reader *name*
::page::pluginmgr::rconfigure *dict*
::page::pluginmgr::rtimeable
::page::pluginmgr::rtime
::page::pluginmgr::rgettime
::page::pluginmgr::rhelp
::page::pluginmgr::rlabel
::page::pluginmgr::read *read eof* ?*complete*?
read **num**
eof
done
::page::pluginmgr::writer *name*
::page::pluginmgr::wconfigure *dict*
::page::pluginmgr::wtimeable
::page::pluginmgr::wtime
::page::pluginmgr::wgettime
::page::pluginmgr::whelp
::page::pluginmgr::wlabel
::page::pluginmgr::write *chan data*
::page::pluginmgr::transform *name*
::page::pluginmgr::tconfigure *id dict*
::page::pluginmgr::ttimeable *id*
::page::pluginmgr::ttime *id*
::page::pluginmgr::tgettime *id*
::page::pluginmgr::thelp *id*
::page::pluginmgr::tlabel *id*
::page::pluginmgr::transform_do *id data*
page_cdefinition
page_rfeature *name*
page_rtime
page_rgettime
page_rlabel
page_rhelp

page_roptions
page_rconfigure *option value*
page_rrun
page_read *num*
page_read_done
page_eof
page_info *text* ?*from ?to*?
page_warning *text* ?*from ?to*?
page_error *text* ?*from ?to*?
page_log_info *text*
page_log_warning *text*
page_log_error *text*
page_wfeature
page_wtime
page_wgettime
page_wlabel
page_whelp
page_woptions
page_wconfigure *option value*
page_wrun *chan data*
page_info *text* ?*from ?to*?
page_warning *text* ?*from ?to*?
page_error *text* ?*from ?to*?
page_log_info *text*
page_log_warning *text*
page_log_error *text*
page_tfeature
page_ttime
page_tgettime
page_tlabel
page_thelp
page_toptions
page_tconfigure *option value*
page_trun *chan data*
page_info *text* ?*from ?to*?
page_warning *text* ?*from ?to*?
page_error *text* ?*from ?to*?
page_log_info *text*
page_log_warning *text*
page_log_error *text*

Beschreibung

Dieses Paket enthält den Plugin-Manager, der das Herzstück der Anwendung **page** bildet. Er verwaltet die verschiedenen Reader-, Writer-, Konfigurations- und Transformations-Plugins, die den Text tatsächlich verarbeiten (lesen, transformieren und schreiben).

Alle Plugins werden in eigens dafür vorbereitete Slave-Interpreter geladen. Obwohl sie mithilfe

von Paketen implementiert sind, benötigen sie diese spezielle Umgebung und können nicht in einem einfachen Interpreter wie tclsh verwendet werden. Aus diesem Grund werden sie im Abschnitt Vordefinierte Plugins nur allgemein beschrieben und nicht als reguläre Pakete dokumentiert. Es wird erwartet, dass sie den in den Abschnitten festgelegten APIs folgen.

1. API für Konfigurations-Plugins
2. API des Reader-Plugins
3. API des Writer-Plugins
4. **API DES TRANSFORM-PLUGINS**

je nach Typ.

API

::page::pluginmgr::reportvia *cmd*
: Dieser Befehl definiert die Rückruffunktion, die **::page::pluginmgr::report** (siehe unten) verwendet, um Eingabefehler und Warnungen zu melden. Standardmäßig werden solche Meldungen in den Standardfehlerkanal geschrieben.

::page::pluginmgr::report *level text* ?*from* ?*to*??
: Dieser Befehl meldet Eingabefehler und Warnungen. Standardmäßig werden solche Meldungen in die Standardfehlerausgabe geschrieben. Das lässt sich ändern, indem mit **::page::pluginmgr::reportvia** eine eigene Rückruffunktion gesetzt wird (siehe oben).

  Die Argumente *level* und *text* legen die Wichtigkeit der Meldung und die Meldung selbst fest. Die zulässigen Werte für Ersteres stehen im Paket **logger**.

  Die optionalen Argumente *from* und *to* kann der Aufrufer verwenden, um die Stelle oder den Bereich in der Eingabe anzugeben, an der das gemeldete Problem aufgetreten ist. Jedes davon ist eine Liste mit zwei Elementen, der Zeile und der Spalte in der Eingabe, in dieser Reihenfolge.

::page::pluginmgr::log *cmd*
: Dieser Befehl definiert eine Protokoll-Rückruffunktion, die geladene Plugins zum Melden interner Fehler, Warnungen und allgemeiner Informationen verwenden. Die leere Zeichenkette als Rückruffunktion schaltet die Protokollierung ab.

  Hinweis: Das *cmd* muss vom Paket **logger** erzeugt worden sein oder derselben API folgen.

  Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::page::pluginmgr::configuration *name*
: Dieser Befehl lädt das benannte Konfigurations-Plugin, ruft die darin kodierten Optionen ab und entlädt es anschließend sofort wieder.

  Ist *name* der Pfad zu einer Datei, wird zuerst versucht, diese Datei als Plugin zu laden; schlägt das fehl, wird die Datei geöffnet und ihr Inhalt als Liste von Optionen und deren Argumenten gelesen, getrennt durch Leerzeichen, Tabulatoren und Zeilenumbrüche, gegebenenfalls quotiert mit einfachen und doppelten Anführungszeichen.

  Die von Konfigurations-Plugins erwartete API steht im Abschnitt API für Konfigurations-Plugins.

Das Ergebnis des Befehls ist die Liste der abgerufenen Optionen.

::page::pluginmgr::reader *name*

Dieser Befehl lädt das angegebene Reader-Plugin und initialisiert es. Das Ergebnis des Befehls ist eine Liste der Optionen, die das Plugin versteht.

Es kann jeweils nur ein Reader-Plugin geladen werden. Das Laden eines weiteren Reader-Plugins führt dazu, dass das zuvor geladene Reader-Plugin deinitialisiert und entladen wird.

Informationen zur API, die von Reader-Plugins erwartet wird, finden Sie im Abschnitt API des Reader-Plugins.

::page::pluginmgr::rconfigure *dict*

Dieser Befehl konfiguriert das geladene Reader-Plugin. Die Optionen und ihre Werte werden als Tcl-Dictionary angegeben. Das Ergebnis des Befehls ist die leere Zeichenkette.

::page::pluginmgr::rtimeable

Dieser Befehl prüft, ob das geladene Reader-Plugin in der Lage ist, Zeitstatistiken zu erfassen. Das Ergebnis des Befehls ist ein boolescher Wert. Das Ergebnis lautet **true**, wenn das Plugin zeitlich erfasst werden kann, und andernfalls **false**.

::page::pluginmgr::rtime

Dieser Befehl aktiviert die Erfassung von Zeitstatistiken im geladenen Reader-Plugin.

::page::pluginmgr::rgettime

Dieser Befehl ruft die erfassten Zeitstatistiken des geladenen Reader-Plugins ab, nachdem dieses ausgeführt wurde.

::page::pluginmgr::rhelp

Dieser Befehl ruft die Hilfe-Zeichenkette des geladenen Reader-Plugins ab. Es wird erwartet, dass diese im *doctools*-Format vorliegt.

::page::pluginmgr::rlabel

Dieser Befehl ruft den für Menschen lesbaren Namen des geladenen Reader-Plugins ab.

::page::pluginmgr::read *read eof* ?*complete*?

Dieser Befehl ruft das geladene Reader-Plugin auf, um die Eingabe zu verarbeiten, und gibt die Ergebnisse des Plugins als eigenes Ergebnis zurück. Die Eingabe ist über die Rückruffunktionen *read* und *eof* erreichbar. Das optionale *done* kann abfangen, wann das Plugin die Verarbeitung abgeschlossen hat. Alle Argumente sind Befehlspräfixe.

Das Plugin ruft die Rückruffunktionen in den folgenden Situationen auf:

read num

wird aufgerufen, wenn Eingabe zum Verarbeiten benötigt wird, mit der Anzahl der angeforderten Zeichen oder Bytes. Als Ergebnis wird die Eingabe erwartet, die das Plugin benötigt.

eof

wird vom Plugin aufgerufen, um zu prüfen, ob die Eingabe das Ende des Datenstroms erreicht hat. Als Ergebnis wird ein boolescher Wert erwartet: **true**, wenn die Eingabe das EOF erreicht hat, und andernfalls **false**.

done

wird aufgerufen, wenn das Plugin die Verarbeitung der Eingabe abgeschlossen hat.

::page::pluginmgr::writer *name*
Dieser Befehl lädt das benannte Writer-Plugin und initialisiert es. Das Ergebnis des Befehls ist eine Liste der Optionen, die das Plugin versteht.

Es kann jeweils nur ein Writer-Plugin geladen werden. Das Laden eines weiteren Writer-Plugins führt dazu, dass das zuvor geladene Writer-Plugin deinitialisiert und entladen wird.

Die von Writer-Plugins erwartete API steht im Abschnitt API des Writer-Plugins.

::page::pluginmgr::wconfigure *dict*
Dieser Befehl konfiguriert das geladene Writer-Plugin. Die Optionen und ihre Werte werden als Tcl-Dictionary übergeben. Das Ergebnis des Befehls ist die leere Zeichenkette.

::page::pluginmgr::wtimeable
Dieser Befehl prüft, ob das geladene Writer-Plugin in der Lage ist, Ausführungszeiten zu messen. Das Ergebnis des Befehls ist ein boolescher Wert. Das Ergebnis lautet **true**, wenn das Plugin zeitlich gemessen werden kann, und andernfalls **false**.

::page::pluginmgr::wtime
Dieser Befehl aktiviert die Erfassung von Zeitstatistiken im geladenen Writer-Plugin.

::page::pluginmgr::wgettime
Dieser Befehl ruft die erfassten Zeitstatistiken des geladenen Writer-Plugins ab, nachdem dieses ausgeführt wurde.

::page::pluginmgr::whelp
Dieser Befehl ruft die Hilfe-Zeichenkette des geladenen Writer-Plugins ab. Es wird erwartet, dass diese im *doctools*-Format vorliegt.

::page::pluginmgr::wlabel
Dieser Befehl ruft den für Menschen lesbaren Namen des geladenen Writer-Plugins ab.

::page::pluginmgr::write *chan data*
Das geladene Writer-Plugin wird aufgerufen, um die Ausgabe zu erzeugen. Es erhält die *data*, aus denen die Ausgabe erzeugt werden soll, und das Tcl-Handle *chan* des Kanals, in den die erzeugte Ausgabe geschrieben wird. Der Befehl gibt als Ergebnis die leere Zeichenkette zurück.

::page::pluginmgr::transform *name*
Dieser Befehl lädt das angegebene Transformations-Plugin und initialisiert es. Das Ergebnis des Befehls ist eine Liste mit zwei Elementen, die in dieser Reihenfolge die Plugin-ID und eine Liste der vom Plugin unterstützten Optionen enthält.

Es können mehrere Transformations-Plugins geladen werden, die über Handles identifiziert werden.

Informationen zur API, die von Transformations-Plugins erwartet wird, finden Sie im Abschnitt **API DES TRANSFORM-PLUGINS**.

::page::pluginmgr::tconfigure *id dict*
Dieser Befehl konfiguriert das angegebene Transformations-Plugin. Die Optionen und ihre Werte werden als Tcl-Dictionary angegeben. Das Ergebnis des Befehls ist die leere

Zeichenkette.

::page::pluginmgr::ttimeable *id*
Dieser Befehl prüft, ob das angegebene Transformations-Plugin in der Lage ist, Zeitstatistiken zu erfassen. Das Ergebnis des Befehls ist ein boolescher Wert. Das Ergebnis lautet **true**, wenn das Plugin zeitlich erfasst werden kann, andernfalls **false**.

::page::pluginmgr::ttime *id*
Dieser Befehl aktiviert die Erfassung von Zeitstatistiken im angegebenen Transformations-Plugin.

::page::pluginmgr::tgettime *id*
Dieser Befehl ruft die erfassten Zeitstatistiken des angegebenen Transformations-Plugins nach dessen Ausführung ab.

::page::pluginmgr::thelp *id*
Dieser Befehl ruft die Hilfe-Zeichenkette des angegebenen Transformations-Plugins ab. Diese sollte im *doctools*-Format vorliegen.

::page::pluginmgr::tlabel *id*
Dieser Befehl ruft den für Menschen lesbaren Namen des identifizierten Transformations-Plugins ab.

::page::pluginmgr::transform_do *id data*
Das identifizierte Transformations-Plugin wird aufgerufen, um die angegebenen *data* zu verarbeiten. Das Ergebnis des Plugins wird als Ergebnis des Befehls zurückgegeben.

API für Konfigurations-Plugins

Von Konfigurations-Plugins wird erwartet, dass sie einen einzigen Befehl bereitstellen, der im Folgenden beschrieben wird.

page_cdefinition
Dieser Befehl eines Konfigurations-Plugins wird vom Plugin-Manager aufgerufen, um ihn auszuführen. Das Ergebnis muss eine Liste von Optionen und Werten sein, die verarbeitet werden sollen.

Konfigurations-Plugins erwarten nicht, dass die Umgebung spezielle Befehle bereitstellt.

Es wird davon ausgegangen, dass ein Konfigurations-Plugin mit der Bezeichnung **FOO** durch das Paket **page::config::FOO** implementiert wird.

Konfigurations-Plugins werden in einem Schritt geladen, ausgeführt und entladen; sie werden nicht im Speicher gehalten. Der Befehl hierfür lautet **::page::pluginmgr::configuration**.

API des Reader-Plugins

Von Reader-Plugins wird erwartet, dass sie die nachfolgend beschriebenen Befehle bereitstellen.

page_rfeature *name*
Dieser Befehl nimmt den *name* eines Merkmals entgegen und gibt ein boolesches Flag zurück, das angibt, ob das Merkmal vom Plugin unterstützt wird oder nicht. Das Ergebnis muss **true** lauten, wenn das Merkmal unterstützt wird, und andernfalls **false**.

Im Abschnitt Funktionen stehen die möglichen Merkmale, nach denen der Plugin-Manager fragt.

page_rtime

Dieser Befehl wird aufgerufen, um die Erfassung von Zeitstatistiken zu aktivieren.

page_rgettime
Dieser Befehl wird aufgerufen, um die erfassten Zeitstatistiken abzurufen.

page_rlabel
Dieser Befehl wird aufgerufen, um eine für Menschen lesbare Bezeichnung für das Plugin abzurufen.

page_rhelp
Dieser Befehl wird aufgerufen, um einen Hilfetext für ein Plugin abzurufen. Der Text sollte im *doctools*-Format vorliegen.

page_roptions
Dieser Befehl wird aufgerufen, um die vom Plugin unterstützten Optionen abzurufen.

page_rconfigure *option value*
Dieser Befehl wird aufgerufen, um das Plugin neu zu konfigurieren; dabei wird insbesondere die angegebene *option* auf den neuen *value* gesetzt.

page_rrun
Dieser Befehl wird aufgerufen, um den Eingabestrom gemäß der aktuellen Konfiguration des Plugins zu verarbeiten. Das Ergebnis des Befehls ist das Ergebnis der Verarbeitung.

Reader-Plugins erwarten, dass die Umgebung die folgenden speziellen Befehle bereitstellt.

page_read *num*
Dieser Befehl wird aufgerufen, um *num* Zeichen/Bytes aus der Eingabe zu lesen. Das Ergebnis muss aus *num* Zeichen/Bytes bestehen.

page_read_done
Dieser Befehl wird aufgerufen, um zu signalisieren, dass das Plugin die Verarbeitung der Eingabe abgeschlossen hat.

page_eof
Dieser Befehl wird aufgerufen, um zu prüfen, ob der Eingabestrom sein Ende erreicht hat. Das Ergebnis muss ein boolescher Wert sein: **true**, wenn das Ende der Eingabe erreicht ist, andernfalls **false**.

page_info *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer bestimmte Informationen anzuzeigen. Kann einen Ort oder einen Bereich in der Eingabe angeben. Jede Ortsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_warning *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer eine Warnung zu melden. Kann eine Stelle oder einen Bereich in der Eingabe angeben. Jede Positionsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_error *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer einen Fehler zu melden. Kann auf eine Stelle oder einen Bereich in der Eingabe hinweisen. Jede Positionsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_log_info *text*
Wird aufgerufen, um bestimmte interne Informationen zu melden.

page_log_warning *text*
Wird aufgerufen, um eine interne Warnung zu melden.

page_log_error *text*
Wird aufgerufen, um einen internen Fehler zu melden.

Es wird erwartet, dass ein Reader-Plugin mit der Bezeichnung **FOO** durch das Paket **page::reader::FOO** implementiert wird.

Reader-Plugins werden mit dem Befehl **::page::pluginmgr::reader** geladen. Es kann höchstens ein Reader-Plugin im Speicher verbleiben.

API des Writer-Plugins

Von Writer-Plugins wird erwartet, dass sie die folgenden Befehle bereitstellen, die im Folgenden beschrieben werden.

page_wfeature
Dieser Befehl nimmt den *name* eines Merkmals entgegen und gibt ein boolesches Flag zurück, das angibt, ob das Merkmal vom Plugin unterstützt wird oder nicht. Das Ergebnis muss **true** lauten, wenn das Merkmal unterstützt wird, und andernfalls **false**.

Im Abschnitt Funktionen stehen die möglichen Merkmale, nach denen der Plugin-Manager fragt.

page_wtime
Dieser Befehl wird aufgerufen, um die Erfassung von Zeitstatistiken zu aktivieren.

page_wgettime
Dieser Befehl wird aufgerufen, um die erfassten Zeitstatistiken abzurufen.

page_wlabel
Dieser Befehl wird aufgerufen, um eine für Menschen lesbare Bezeichnung für das Plugin abzurufen.

page_whelp
Dieser Befehl wird aufgerufen, um einen Hilfetext für ein Plugin abzurufen. Der Text sollte im *doctools*-Format vorliegen.

page_woptions
Dieser Befehl wird aufgerufen, um die vom Plugin unterstützten Optionen abzurufen.

page_wconfigure *option value*
Dieser Befehl wird aufgerufen, um das Plugin neu zu konfigurieren; dabei wird insbesondere die angegebene *option* auf den neuen *value* gesetzt.

page_wrun *chan data*
Dieser Befehl wird aufgerufen, um die angegebenen *data* zu verarbeiten und in den Ausgabestrom *chan* zu schreiben. Bei *chan* handelt es sich um ein Tcl-Kanal-Handle, das zum Schreiben geöffnet wurde. Das Ergebnis des Befehls ist die leere Zeichenkette.

Writer-Plugins erwarten, dass die Umgebung die folgenden speziellen Befehle bereitstellt.

page_info *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer bestimmte Informationen anzuzeigen. Kann einen Ort oder einen Bereich in der Eingabe angeben. Jede Ortsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_warning *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer eine Warnung zu melden. Kann eine Stelle oder einen Bereich in der Eingabe angeben. Jede Positionsangabe, sofern vorhanden, ist eine Liste mit

zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_error *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer einen Fehler zu melden. Kann auf eine Stelle oder einen Bereich in der Eingabe hinweisen. Jede Positionsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_log_info *text*
Wird aufgerufen, um bestimmte interne Informationen zu melden.

page_log_warning *text*
Wird aufgerufen, um eine interne Warnung zu melden.

page_log_error *text*
Wird aufgerufen, um einen internen Fehler zu melden.

Es wird erwartet, dass ein Writer-Plugin mit der Bezeichnung **FOO** durch das Paket **page::writer::FOO** implementiert wird.

Writer-Plugins werden mit dem Befehl **::page::pluginmgr::writer** geladen. Es kann höchstens ein Writer-Plugin im Speicher gehalten werden.

API des Transform-Plugins

Von Transformations-Plugins wird erwartet, dass sie die folgenden Befehle bereitstellen, die im Folgenden beschrieben werden.

page_tfeature
Dieser Befehl nimmt den *name* eines Merkmals entgegen und gibt ein boolesches Flag zurück, das angibt, ob das Merkmal vom Plugin unterstützt wird oder nicht. Das Ergebnis muss **true** lauten, wenn das Merkmal unterstützt wird, und andernfalls **false**.

Im Abschnitt Funktionen stehen die möglichen Merkmale, nach denen der Plugin-Manager fragt.

page_ttime
Dieser Befehl wird aufgerufen, um die Erfassung von Zeitstatistiken zu aktivieren.

page_tgettime
Dieser Befehl wird aufgerufen, um die erfassten Zeitstatistiken abzurufen.

page_tlabel
Dieser Befehl wird aufgerufen, um eine für Menschen lesbare Bezeichnung für das Plugin abzurufen.

page_thelp
Dieser Befehl wird aufgerufen, um einen Hilfetext für ein Plugin abzurufen. Der Text sollte im *doctools*-Format vorliegen.

page_toptions
Dieser Befehl wird aufgerufen, um die vom Plugin unterstützten Optionen abzurufen.

page_tconfigure *option value*
Dieser Befehl wird aufgerufen, um das Plugin neu zu konfigurieren; dabei wird insbesondere die angegebene *option* auf den neuen *value* gesetzt.

page_trun *chan data*
Dieser Befehl wird aufgerufen, um die angegebenen *data* zu verarbeiten und in den Ausgabestrom *chan* zu schreiben. Bei *chan* handelt es sich um ein Tcl-Kanal-Handle, das zum

Schreiben geöffnet wurde. Das Ergebnis des Befehls ist die leere Zeichenkette.

Transformations-Plugins erwarten, dass die Umgebung die folgenden speziellen Befehle bereitstellt.

page_info *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer bestimmte Informationen anzuzeigen. Kann einen Ort oder einen Bereich in der Eingabe angeben. Jede Ortsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_warning *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer eine Warnung zu melden. Kann eine Stelle oder einen Bereich in der Eingabe angeben. Jede Positionsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_error *text* ?*from* ?*to*??
Wird aufgerufen, um dem Benutzer einen Fehler zu melden. Kann auf eine Stelle oder einen Bereich in der Eingabe hinweisen. Jede Positionsangabe, sofern vorhanden, ist eine Liste mit zwei Elementen, die Zeilen- und Spaltennummern enthält.

page_log_info *text*
Wird aufgerufen, um bestimmte interne Informationen zu melden.

page_log_warning *text*
Wird aufgerufen, um eine interne Warnung zu melden.

page_log_error *text*
Wird aufgerufen, um einen internen Fehler zu melden.

Es wird erwartet, dass ein Transformations-Plugin **FOO** durch das Paket **page::transform::FOO** implementiert wird.

Transformations-Plugins werden mit dem Befehl **::page::pluginmgr::transform** geladen. Es können mehrere Transformations-Plugins im Speicher gehalten werden.

Vordefinierte Plugins

Die folgenden vordefinierten Plugins sind bekannt, d. h., sie werden vom Modul page bereitgestellt.

Configuration
siehe unten ...

peg
Gibt eine Reihe von Optionen zurück, mit denen die Anwendung **page** für die Verarbeitung einer PEG-Grammatik und die Generierung von ME-Code konfiguriert werden kann. Weitere Einzelheiten stehen in den Paketen **grammar_peg**, **grammar_me** und verwandten Paketen.

Reader
siehe unten ...

hb
Erwartet einen sogenannten *half-baked PEG container* als Eingabe und gibt den entsprechenden abstrakten Syntaxbaum zurück. Das Writer-Plugin *hb* erzeugt diese Art von Eingabe.

lemon

Erwartet eine Grammatikbeschreibung, wie sie vom LEMON-Parsergenerator von Richard Hipp verarbeitet wird, und gibt einen abstrakten Syntaxbaum dafür zurück.

peg
Erwartet eine Grammatikspezifikation in Form einer Parsing Expression Grammar (PEG) und gibt einen abstrakten Syntaxbaum dafür zurück.

ser
Erwartet als Eingabe die serialisierte Form einer Parsing Expression Grammar, wie sie das Paket **grammar::peg** erzeugt, wandelt sie in einen äquivalenten abstrakten Syntaxbaum um und gibt diesen zurück.

treeser
Erwartet als Eingabe die vom Paket **struct::tree** generierte serialisierte Form eines Baums und gibt diese nach der Validierung zurück.

Writer
siehe unten ...

hb
Erwartet als Eingabe einen abstrakten Syntaxbaum für eine Parsing Expression Grammar und schreibt ihn als sogenannten *half-baked PEG container* aus.

identity
Nimmt jede Eingabe entgegen und schreibt sie unverändert aus.

mecpu
Erwartet symbolischen Assembler-Code für die MatchEngine-CPU (siehe das Paket **grammar::me::cpu** und zugehörige Pakete) und gibt diesen als Tcl-Code für einen Parser aus.

me
Erwartet als Eingabe einen abstrakten Syntaxbaum für eine Parsing Expression Grammar und schreibt ihn als Tcl-Code für die MatchEngine aus (siehe das Paket **grammar::me** und verwandte Pakete), die Eingabe in dieser Grammatik parst.

null
Nimmt beliebige Eingaben entgegen und gibt nichts aus. Das logische Äquivalent zu /dev/null.

peg
Erwartet als Eingabe einen abstrakten Syntaxbaum für eine Parsing Expression Grammar und gibt diesen in Form einer kanonischen PEG aus, die vom Reader-Plugin *peg* gelesen werden kann.

ser
Erwartet als Eingabe einen abstrakten Syntaxbaum für eine Parsing Expression Grammar und gibt diesen als serialisierten PEG-Container aus, der vom Reader-Plugin *ser* gelesen werden kann.

tpc
Erwartet als Eingabe einen abstrakten Syntaxbaum für eine Parsing Expression Grammar und gibt diesen als Tcl-Code aus, der einen PEG-Container initialisiert, wie er vom Paket **grammar::peg** bereitgestellt wird.

tree
Nimmt einen beliebigen serialisierten Baum (gemäß dem Paket **struct::tree**) als Eingabe

entgegen und gibt ihn in einem generischen, eingerückten Format aus.

Transformation
siehe unten ...

mecpu
Nimmt einen abstrakten Syntaxbaum für eine Parsing Expression Grammar als Eingabe entgegen, generiert symbolischen Assembler-Code für die MatchEngine-CPU und gibt diesen als Ergebnis zurück (siehe das Paket **grammar::me::cpu** und zugehörige Pakete).

reachable
Nimmt einen abstrakten Syntaxbaum für eine Parsing Expression Grammar als Eingabe entgegen, führt eine Erreichbarkeitsanalyse durch und gibt den veränderten und annotierten Baum zurück.

realizable
Nimmt einen abstrakten Syntaxbaum für eine Parsing Expression Grammar als Eingabe entgegen, führt eine Realisierbarkeitsanalyse durch und gibt den modifizierten und mit Anmerkungen versehenen Baum zurück.

Funktionen

Der Plugin-Manager prüft die Plugins derzeit nur auf ein einziges Merkmal, **timeable**. Von einem Plugin, das dieses Merkmal unterstützt, wird angenommen, dass es auf Anforderung Zeitstatistiken erfassen kann.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Page-Parsergenerator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_flow

Stichwörter: page_util_flow, dataflow, graph walking, page, parser generator, text processing, tree walking

Name

page_util_flow – page-Dienstprogramm für Datenfluss und Baumdurchquerung

Übersicht

package require page::util::flow ?**0.2**?
package require snit
::page::util::flow *start flowvar nodevar script*
flow **visit** *node*
flow **visitl** *nodelist*
flow **visita** *node...*

Beschreibung

Dieses Paket stellt einen einzigen Hilfsbefehl bereit, der die auf dem Datenfluss beruhende Bearbeitung beliebiger Datenstrukturen, insbesondere abstrakter Syntaxbäume, erleichtert.

API

::page::util::flow *start flowvar nodevar script*

Dieser Befehl enthält die Kernlogik für das Durchlaufen einer beliebigen Datenstruktur, die sich in einzelne Teile zerlegen lässt. Beispiele für solche Strukturen sind Bäume und Graphen.

Der Befehl macht keinerlei Annahmen über die API der zu durchlaufenden Struktur, außer dass ihre Teile, hier Knoten genannt, durch Zeichenketten identifiziert werden. Diese Zeichenketten werden unverändert aus den Argumenten und dem Rumpf übernommen und unverändert an den Rumpf zurückgegeben.

Der Zugriff auf die eigentliche Datenstruktur und alle Entscheidungen, welche Knoten in welcher Reihenfolge besucht werden, werden an den Rumpf der Schleife weitergeleitet, also an das *script*.

Der Rumpf wird zuerst für die Knoten der über *start* angegebenen Startmenge aufgerufen und danach für die Knoten, deren Besuch der Rumpf angefordert hat. Der Befehl endet, wenn die Menge der zu besuchenden Knoten leer wird. Zu beachten ist, dass ein Knoten mehrmals besucht werden kann. Darüber hat der Rumpf die vollständige Kontrolle.

Der Rumpf wird im Kontext des Aufrufers aufgerufen. Die durch *nodevar* benannte Variable wird auf den aktuellen Knoten gesetzt, und die durch *flowvar* benannte Variable wird auf den Befehl des Flow-Objekts gesetzt, über den der Rumpf die als Nächstes zu besuchenden Knoten anfordern kann. Die API dieses Objekts beschreibt der nächste Abschnitt, Flow-API.

Zu beachten ist, dass der Befehl keine Zusagen über die Reihenfolge macht, in der Knoten besucht werden, außer dass die Knoten, deren Besuch die aktuelle Iteration angefordert hat, anschließend in irgendeiner Reihenfolge besucht werden.

Flow-API

In diesem Abschnitt wird die API beschrieben, die das Flow-Objekt dem Rumpf-Skript von **::page::util::flow** bereitstellt.

flow* visit *node

Der Aufruf dieser Methode fordert an, dass der Knoten *node* nach der aktuellen Iteration besucht wird.

flow* visitl *nodelist
Durch den Aufruf dieser Methode wird angefordert, dass alle in der Liste *nodelist* gefundenen Knoten nach der aktuellen Iteration besucht werden.

***flow* visita *node*...**
Dies ist die Variante der Methode **visitl** mit variabler Argumentanzahl; siehe oben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Page-Parsergenerator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_norm_lemon

Stichwörter: page_util_norm_lemon, graph walking, lemon, normalization, page, parser generator, text processing, tree walking

Name

page_util_norm_lemon – page-AST-Normalisierung, LEMON

Übersicht

package require page::util::norm_lemon ?**0.2**?
package require snit
::page::util::norm::lemon *tree*

Beschreibung

Dieses Paket stellt einen einzigen Hilfsbefehl zur Verfügung, der einen AST einer Lemon-Grammatik entgegennimmt und diesen auf verschiedene Weise normalisiert. Das Ergebnis wird als *normalisierter Lemon-Grammatikbaum* bezeichnet.

Hinweis: Dieses Paket kann nur innerhalb eines Plugins verwendet werden, das vom Paket **page::pluginmgr** verwaltet wird.

API

::page::util::norm::lemon *tree*
Dieser Befehl setzt voraus, dass das *tree*-Objekt eine Lemon-Grammatik enthält. Er normalisiert diesen Baum an Ort und Stelle. Das Ergebnis wird als *normalisierter*

Lemon-Grammatikbaum bezeichnet.

Die genauen durchgeführten Operationen werden vorerst nicht dokumentiert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Page-Parsergenerator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_norm_peg

Stichwörter: page_util_norm_peg, PEG, graph walking, normalization, page, parser generator, text processing, tree walking

Name

page_util_norm_peg – page-AST-Normalisierung, PEG

Übersicht

package require page::util::norm_peg ?**0.2**?
package require snit
::page::util::norm::peg *tree*

Beschreibung

Dieses Paket stellt einen einzigen Befehl zur Verfügung, der einen AST für eine Parsing Expression Grammar entgegennimmt und diesen auf verschiedene Weise normalisiert. Das Ergebnis wird als *normalisierter PE-Grammatikbaum* bezeichnet.

Hinweis: Dieses Paket kann nur innerhalb eines Plugins verwendet werden, das vom Paket **page::pluginmgr** verwaltet wird.

API

::page::util::norm::peg *tree*

Dieser Befehl geht davon aus, dass das Objekt *tree* eine Parsing Expression Grammar enthält. Er normalisiert diesen Baum an Ort und Stelle. Das Ergebnis wird als *normalisierter PE-Grammatikbaum* bezeichnet.

Die folgenden Operationen werden durchgeführt:

1. Die Daten aller Terminale werden in deren Großelternknoten gespeichert. Die Terminalknoten und ihre Elternknoten werden entfernt. Typinformationen werden verworfen.
2. Alle Knoten, die genau einen Kindknoten haben, sind irrelevant und werden entfernt, mit Ausnahme des Wurzelknotens. Der unmittelbare Kindknoten des Wurzelknotens ist ebenfalls irrelevant und wird ebenfalls entfernt.
3. Der Name der Grammatik wird von dem Baumknoten, in dem er gespeichert ist, in ein Attribut des Wurzelknotens verschoben, und der Baumknoten wird entfernt.

   Der Knoten, der den Startausdruck getrennt hält, wird als irrelevant entfernt, und der Wurzelknoten des Startausdrucks wird mit einem Marker-Attribut versehen; sein Handle wird zum schnellen Zugriff in einem Attribut des Wurzelknotens gespeichert.
4. Hinweisinformationen zu Nichtterminalen werden aus den Knoten in Attribute verschoben, und die nun überflüssigen Knoten werden gelöscht.

   Hinweis: Diese Transformation setzt das Entfernen aller Knoten mit genau einem Kindknoten voraus, weil dabei bereits alle "Attribute"-Knoten entfernt werden. Andernfalls müsste diese Transformation die Informationen in den Großelternknoten einfügen.

   Der vorgegebene Modus der Nichtterminale ist **value**.

   Wie bei der Definition der globalen Metadaten werden bestimmte Informationen aus den Knoten in Attribute verschoben, die nun überflüssigen Knoten gelöscht und die Wurzelknoten aller Definitionen mit Marker-Attributen versehen. Dadurch entsteht außerdem eine Zuordnung von Nichtterminalnamen zu ihren definierenden Knoten, die zum schnellen Nachschlagen in einem Attribut des Wurzelknotens gespeichert wird.

   Zuletzt wird der von einer Definition abgedeckte Bereich in der Eingabe berechnet. Die linke Grenze kommt vom Terminal für das Nichtterminalsymbol, das die Definition definiert. Die rechte Grenze kommt vom am weitesten rechts stehenden Kindknoten unter der Definition. Auch wenn dies noch kein Ausdrucksbaum ist, sind die Positionsdaten bereits stimmig.
5. Die verbleibenden Knoten unter allen Definitionen werden in echte Ausdrucksbäume umgewandelt. Zuerst Zeichenbereiche, danach unäre Operationen, Zeichen und Nichtterminale. Zuletzt wird der Baum durch Entfernen überflüssiger innerer Knoten abgeflacht.

   Die Reihenfolge ist entscheidend, um möglichst früh so viele Knoten wie möglich zu entfernen und späteren unnötigen Aufwand zu vermeiden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Page-Parsergenerator

Copyright

```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_peg
Stichwörter: page_util_peg, PEG, page, parser generator, parsing expression grammar, text processing, transformation

Name
page_util_peg – page-Hilfsprogramme für PEG-Transformationen

Übersicht
package require page::util::peg ?**0.2**?
package require snit
::page::util::peg::symbolNodeOf *tree node*
::page::util::peg::symbolOf *tree node*
::page::util::peg::updateUndefinedDueRemoval *tree*
::page::util::peg::flatten *treequery tree*
::page::util::peg::getWarnings *tree*
::page::util::peg::printWarnings *msg*
::page::util::peg::peOf *tree eroot*
::page::util::peg::printTclExpr *pe*

Beschreibung
Dieses Paket stellt einige gängige Operationen für PEG-Transformationen bereit. Als Eingabe wird ein *normalisierter PE-Grammatikbaum* vorausgesetzt; siehe dazu das Paket **page::util::norm::peg**, der gegebenenfalls um Attribute ergänzt werden kann, die aus Transformationen stammen, die nicht im Widerspruch zur Basisdefinition stehen.

API
::page::util::peg::symbolNodeOf *tree node*
Zu einem beliebigen Ausdrucksknoten *node* im AST *tree* ermittelt diese Funktion den Knoten (den Knoten selbst oder einen Vorfahren), der den Namen des Nichtterminals enthält, zu dem der Knoten gehört, und gibt dessen ID zurück. Das Ergebnis ist entweder die Wurzel des Baums (beim Startausdruck) oder ein Definitionsknoten.

::page::util::peg::symbolOf *tree node*
Wie **::page::util::peg::symbolNodeOf**, gibt jedoch anstelle des Knotens den Namen des Symbols zurück.

::page::util::peg::updateUndefinedDueRemoval *tree*
Das Entfernen von Knoten im AST *tree* kann dazu führen, dass Symbole eine oder mehrere Verwendungsstellen verlieren.

```
    A used by B and C,
    B is reachable,
    C is not,

    so A now loses the node in the expression for C calling it,
    or rather, not calling it anymore.
```

Dieser Befehl aktualisiert die Querverweise und welche Nichtterminale nun undefiniert sind.

::page::util::peg::flatten *treequery tree*
Dieser Befehl flacht verschachtelte Sequenz- und Auswahloperatoren im AST *tree* ab und verwendet das **treeql**-Objekt *treequery* erneut, um die Abfrage auszuführen, die ermittelt, welche Knoten entfernt werden.

::page::util::peg::getWarnings *tree*
Dieser Befehl prüft die Attribute des AST *tree* auf Probleme der Grammatik und stellt Warnungen zusammen. Sie hindern uns nicht daran, die Grammatik zu schreiben, stellen aber dennoch Probleme dar, auf die der Benutzer hingewiesen werden sollte.

Das Ergebnis des Befehls ist ein Dictionary, das die Namen der Nichtterminale ihren Warnungen zuordnet.

::page::util::peg::printWarnings *msg*
Das Argument des Befehls ist ein Dictionary, das Nichtterminalnamen ihren Warnungen zuordnet, wie es zum Beispiel der Befehl **::page::util::peg::getWarnings** erzeugt.

Die darin enthaltenen Warnungen werden formatiert und dann über den Protokollbefehl **page_info** ausgegeben. Dieser Befehl kann daher nur innerhalb eines Plugins verwendet werden, das vom Paket **page::pluginmgr** verwaltet wird.

::page::util::peg::peOf *tree eroot*
Dieser Befehl wandelt den Parsing-Ausdruck, der am Knoten *eroot* im AST *tree* beginnt, in eine verschachtelte Liste um. Die genaue Syntax dieser Liste wird durch das Paket **grammar::peg** festgelegt.

::page::util::peg::printTclExpr *pe*
Dieser Befehl wandelt den in der verschachtelten Liste *pe* enthaltenen Parsing-Ausdruck in eine Tcl-Zeichenkette um, die in ein Tcl-Skript eingefügt werden kann. Die genaue Syntax von *pe* entnehmen Sie bitte dem Paket **grammar::peg**.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Page-Parsergenerator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


page_util_quote

Stichwörter: page_util_quote, page, parser generator, quoting, text processing

Name

page_util_quote – page-Hilfsprogramme zur Zeichenquotierung

Übersicht

package require page::util::quote ?**0.2**?
package require snit
::page::util::quote::unquote *char*
::page::util::quote::quote'tcl *char*
::page::util::quote::quote'tclstr *char*
::page::util::quote::quote'tclcom *char*

Beschreibung

Dieses Paket stellt einige Hilfsbefehle bereit, die Zeichen in verschiedene Formen umwandeln.

API

::page::util::quote::unquote *char*
Ein Zeichen, wie es ein PEG-Prozessor in einem abstrakten Syntaxbaum speichert (siehe die Pakete **grammar::peg::interpreter**, **grammar::me** und ihre Beziehungen), also in einer quotierten Form, wird in das entsprechende Tcl-Zeichen umgewandelt. Das Zeichen wird als Ergebnis des Befehls zurückgegeben.

::page::util::quote::quote'tcl *char*
Dieser Befehl nimmt ein Tcl-Zeichen (interne Darstellung) entgegen und wandelt es in eine Zeichenkette um, die vom Tcl-Parser akzeptiert wird, das betreffende Zeichen wiederherstellt und im 7-Bit-ASCII-Format vorliegt. Die Zeichenkette wird als Ergebnis dieses Befehls zurückgegeben.

::page::util::quote::quote'tclstr *char*
Dieser Befehl nimmt ein Tcl-Zeichen (interne Darstellung) entgegen und wandelt es in eine Zeichenkette um, die der Tcl-Parser akzeptiert und die eine für Menschen lesbare Darstellung des betreffenden Zeichens erzeugt. Die Zeichenkette wird als Ergebnis dieses Befehls zurückgegeben.

Die Zeichenkette verwendet keine nicht druckbaren Zeichen. Sie kann Backslash-Quotierung verwenden. Zeichen aus dem oberen UTF-Bereich werden quotiert, um Probleme mit den nach wie vor verbreiteten ASCII-Terminals zu vermeiden. Es wird davon ausgegangen, dass die Zeichenkette in doppelten Anführungszeichen verwendet wird.

::page::util::quote::quote'tclcom *char*
Dieser Befehl nimmt ein Tcl-Zeichen (interne Darstellung) entgegen und wandelt es in eine

Zeichenkette um, die vom Tcl-Parser akzeptiert wird, wenn sie innerhalb eines Tcl-Kommentars verwendet wird. Die Zeichenkette wird als Ergebnis dieses Befehls zurückgegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *page* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Page-Parsergenerator

Copyright


```
Copyright (c) 2007 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pki

pki

Stichwörter: pki, cipher, data integrity, encryption, public key cipher, rsa, security

Name

pki – Implementierung der Public-Key-Verschlüsselung

Übersicht

package require Tcl 8.5 9
package require pki ?**0.22**?
::pki::encrypt ?*-binary*? ?*-hex*? ?*-pad*? ?*-nopad*? ?*-priv*? ?*-pub*? ?*--*? *input key*
::pki::decrypt ?*-binary*? ?*-hex*? ?*-unpad*? ?*-nounpad*? ?*-priv*? ?*-pub*? ?*--*? *input key*
::pki::sign *input key* ?*algo*?
::pki::verify *signedmessage plaintext key* ?*algo*?
::pki::key *key* ?*password*? ?*encodePem*?
::pki::pkcs::parse_key *key* ?*password*?
::pki::x509::parse_cert *cert*
::pki::rsa::generate *bitlength* ?*exponent*?
::pki::x509::verify_cert *cert trustedcerts* ?*intermediatecerts*?
::pki::x509::validate_cert *cert* ?*-sign_message dn_of_signer*? ?*-encrypt_message dn_of_signer* ? ?*-sign_cert dn_to_be_signed ca_depth*? ?*-ssl dn*?
::pki::pkcs::create_csr *keylist namelist* ?*encodePem*? ?*algo*?
::pki::pkcs::parse_csr *csr*
::pki::x509::create_cert *signreqlist cakeylist serial_number notBefore notAfter isCA extensions* ?*encodePem*? ?*algo*?

Beschreibung

Befehle

::pki::encrypt ?*-binary*? ?*-hex*? ?*-pad*? ?*-nopad*? ?*-priv*? ?*-pub*? ?--? *input key*

Verschlüsselt eine Nachricht mithilfe von PKI (voraussichtlich RSA).

Der Aufrufer muss entweder "**-priv**" für die Verschlüsselung mit dem privaten Schlüssel oder "**-pub**" für die Verschlüsselung mit dem öffentlichen Schlüssel festlegen. Die Standardoption sieht ein Padding und die Rückgabe im Hexadezimalformat vor. Es muss entweder "**-pub**" oder "**-priv**" festgelegt werden.

Die Option "**-hex**" bewirkt, dass die Daten als hexadezimale Zeichenkette mit Kodierung zurückgegeben werden, während die Option "**-binary**" bewirkt, dass die Daten als binäre Zeichenkette mit Kodierung zurückgegeben werden. Wenn diese Optionen mehrfach gelegt werden, wird die zuletzt gelegte Option verwendet.

Die Option **-pad** bewirkt, dass die Daten vor der Verschlüsselung gemäß PKCS#1 mit Padding aufgefüllt werden. Die Option **-nopad** unterbindet dieses Verhalten. Wenn diese Optionen mehrfach festgelegt werden, wird die zuletzt festgelegte Option verwendet.

Die zu verschlüsselnde Eingabe wird als *input* festgelegt.

Der Parameter *key*, der den zu verwendenden Schlüssel enthält, ist ein Wert, der von entweder **::pki::pkcs::parse_key**, **::pki::x509::parse_cert** oder **::pki::rsa::generate** gegeben wird.

Zuordnung zur OpenSSL-Anwendung **openssl**:
1. "openssl rsautl -encrypt" == "::pki::encrypt -binary -pub"
2. "openssl rsautl -sign" == "::pki::encrypt -binary -priv"

::pki::decrypt ?*-binary*? ?*-hex*? ?*-unpad*? ?*-nounpad*? ?*-priv*? ?*-pub*? ?--? *input key*

Entschlüsselt eine Nachricht mithilfe von PKI (wahrscheinlich RSA). Informationen zur Einstellung der Optionen finden Sie unter **::pki::encrypt**.

Zuordnung zur OpenSSL-Anwendung **openssl**:
1. "openssl rsautl -decrypt" == "::pki::decrypt -binary -priv"
2. "openssl rsautl -verify" == "::pki::decrypt -binary -pub"

::pki::sign *input key* ?*algo*?

Signiert die Nachricht *input* digital mit dem privaten *key*.

Wenn *algo* weggelassen wird, wird "sha1" angenommen. Mögliche Werte für *algo* sind "**md5**", "**sha1**", "**sha256**" und "**raw**". Die Angabe von "**raw**" für *algo* verhindert die Erstellung einer ASN.1-Struktur für die Kodierung des gewählten Hash-Algorithmus. *Achtung*: In diesem Fall muss der entsprechende **pkgi::verify** als Algorithmusinformation **with** aufgerufen werden. Umgekehrt legt die Angabe eines anderen Algorithmus als "**raw**" an dieser Stelle fest, dass der entsprechende Aufruf von **pkgi::verify** *ohne* Algorithmusangabe erfolgen muss.

Die *Eingabe* sollte der Klartext sein, auf den die Hash-Berechnung angewendet wird.

Der *key* sollte den privaten Schlüssel enthalten.

::pki::verify *signedmessage plaintext key ?algo?*
Überprüfen einer digitalen Signatur mithilfe eines öffentlichen *key*. Gibt "true" oder "false" zurück.

Achtung: Die Algorithmusinformation *algo* muss genau dann festgelegt werden, wenn die Funktion **pki::sign**, die die *signedmessage* erzeugt hat, mit dem Algorithmus "**raw**" aufgerufen wurde. Dadurch wurde die Erstellung der ASN.1-Struktur, die die Kodierung des gewählten Hash-Algorithmus enthält, verhindert. Umgekehrt dürfen Sie hier *keinen* Algorithmus angeben, wenn bei der Signierung bereits ein geeigneter Algorithmus festgelegt wurde.

::pki::key *key ?password? ?encodePem?*
Konvertiert eine Schlüsselstruktur in einen serialisierten, für andere Anwendungen geeigneten privaten Schlüssel mit der PEM- (Standard) oder DER-Kodierung. Bei RSA-Schlüsseln bedeutet dies PKCS#1.

::pki::pkcs::parse_key *key ?password?*
Konvertiert einen privaten *key* im PKCS#1-Format in einen verwendbaren Schlüssel, d. h. einen Schlüssel, der als Argument für **::pki::encrypt**, **::pki::decrypt**, **::pki::sign** und **::pki::verify** verwendet werden kann.

::pki::x509::parse_cert *cert*
Konvertiert ein X.509-Zertifikat in einen verwendbaren (öffentlichen) Schlüssel. Das zurückgegebene Dictionary kann als Argument für **::pki:encrypt**, **::pki::decrypt** und **::pki::verify** verwendet werden. Das Argument *cert* kann entweder mit PEM- oder DER-Kodierung kodiert sein. Zusätzlich zu den Informationen zum öffentlichen Schlüssel enthält das Dictionary die folgenden Schlüssel mit Zertifikatsinhalten, wie in RFC5280 definiert:
- **subject** enthält den Namen des Zertifikatsinhabers.
- **issuer** enthält den Namen der ausstellenden Zertifizierungsstelle.
- "**serial_number**" enthält die Seriennummer des Zertifikats.
- In "**notBefore**" ist das Startdatum der Gültigkeitsdauer des Zertifikats hinterlegt.
- In "**notAfter**" ist das Ablaufdatum der Zertifikatsgültigkeit hinterlegt.
- **version** liegt im X.509-Format vor.
- **extensions** enthält ein Dictionary mit den im Zertifikat enthaltenen Erweiterungen (siehe unten).

Das Wörterbuch enthält zusätzliche Einträge zum Thema Verschlüsselung. Diese sind für die Verwendung der oben genannten Befehle für kryptografische Operationen vorgesehen.

Der Schlüssel "**extensions**" im zurückgegebenen Wörterbuch enthält ein verschachteltes Dictionary, dessen Schlüssel den Namen (mit genau derselben Groß-/Kleinschreibung) in Certificate Extensions in RFC 5280. Das Format jedes Werts basiert ebenfalls auf den dort definierten ASN.1-Strukturen. Eine Veranschaulichung finden Sie im Abschnitt *Beispiele* [#section3].

::pki::rsa::generate *bitlength ?exponent?*
Erzeugt ein neues RSA-Schlüsselpaar, dessen Bestandteile als Argumente für **::pki::encrypt**, **::pki::decrypt**, **::pki::sign** und **::pki::verify** verwendet werden können.

Das Argument *bitlength* gibt die Länge des Moduls des öffentlichen Schlüssels an.

Das Argument *exponent* sollte im Allgemeinen nicht angegeben werden, es sei denn, Sie wissen genau, was Sie tun.

::pki::x509::verify_cert *cert trustedcerts* ?*intermediatecerts*?

Prüft, ob eine Vertrauensbeziehung zwischen dem im Argument *cert* angegebenen Zertifikat und einem der in der Zertifikatsliste des Arguments *trustedcerts* aufgeführten Zertifikate besteht. (Gegebenenfalls kann die Kette über nicht vertrauenswürdige Zertifikate verlaufen, die im Argument *intermediatecerts* aufgeführt sind; dies ist jedoch derzeit noch nicht implementiert.) Die in den Optionen *cert* und *trustedcerts* angegebenen Zertifikate sollten geparst werden (aus **::pki::x509::parse_cert**).

::pki::x509::validate_cert *cert* ?-sign_message *dn_of_signer*? ?-encrypt_message *dn_of_signer*? ?-sign_cert *dn_to_be_signed ca_depth*? ?-ssl *dn*?

Prüft, ob ein Zertifikat für eine bestimmte Verwendung gültig ist. Wenn mehrere Optionen festgelegt werden, müssen alle erfüllt sein, damit diese Prozedur den Wert "true" zurückgibt.

Derzeit ist nur die Option **-sign_cert** funktionsfähig. Die Argumente lauten *dn_to_be_signed* und *ca_depth*. *dn_to_be_signed* ist der Distinguished Name des Zertifikatsinhabers, um zu überprüfen, ob das im Argument *cert* angegebene Zertifikat signieren darf. Das Argument *ca_depth* dient dazu anzugeben, in welcher Tiefe die Überprüfung erfolgen soll. Bei einigen Zertifikaten ist begrenzt, wie weit in der Kette hinab sie zur Überprüfung eines bestimmten Zertifikats verwendet werden können.

::pki::pkcs::create_csr *keylist namelist* ?*encodePem*? ?*algo*?

Erstellt eine Zertifikatssignierungsanforderung (CSR) anhand eines Schlüsselpaars, das im Argument *keylist* angegeben ist.

Das Argument *namelist* ist eine Liste aus "Name"- und "Wert"-Paaren, die als angeforderter Distinguished Name in der CSR durch die Kodierung dargestellt werden sollen.

Die Option *encodePem* legt fest, ob das Ergebnis im PEM- oder im DER-Format kodiert werden soll. Der Wert "true" bewirkt, dass das Ergebnis im PEM-Format kodiert wird, während jeder andere Wert dazu führt, dass das Ergebnis im DER-Format kodiert wird. Die DER-Kodierung ist die Standardeinstellung.

Das Argument *algo* legt den Hash-Algorithmus fest, den wir zum Signieren dieser Zertifikatssignierungsanforderung verwenden sollen. Die Standardeinstellung ist "sha1". Weitere mögliche Werte sind "md5" und "sha256".

::pki::pkcs::parse_csr *csr*

Ein Zertifikatssignierungsersuchen analysieren. Das Argument *csr* kann entweder in der PEM- oder DER-Kodierung vorliegen. Der Befehl gibt ein Dictionary zurück, das die folgenden Schlüssel enthält:

- **subject** – enthält den Namen des Subjekts aus dem CSR.
- **type** – enthält den Namen des Algorithmus für den öffentlichen Schlüssel. Derzeit wird ausschließlich **rsa** unterstützt.
- **extensionRequest** – enthält ein Dictionary mit den Inhalten der **extensionRequest** HTTPS://datatracker.ietf.org/doc/html/rfc2986#page-5-Informationen im CSR. Dieses hat dieselbe Form wie das für das **extensions**-Dictionary in der Dokumentation zu **parse_cert**

beschriebene.

Möglicherweise sind im Dictionary weitere Schlüssel enthalten, die mit dem verwendeten Algorithmus für öffentliche Schlüssel in Zusammenhang stehen.

::pki::x509::create_cert *signreqlist cakeylist serial_number notBefore notAfter isCA extensions* ?*encodePem*? ?*algo*?

Unterzeichnet eine Signaturanforderung (in der Regel von **::pki::pkcs::create_csr** oder **::pki::pkcs::parse_csr**) mit einem Zertifikat einer Zertifizierungsstelle (CA).

Das Argument *signreqlist* sollte die geparste Signaturanforderung sein.

Das Argument *cakeylist* sollte das geparste CA-Zertifikat sein.

Das Argument *serial_number* sollte eine für dieses Zertifikat von dieser Zertifizierungsstelle eindeutige Seriennummer sein.

Die Argumente *notBefore* und *notAfter* sollten den Zeitpunkt vor bzw. nach dem das Zertifikat als ungültig gelten soll, enthalten. Die Zeitangabe sollte durch eine Kodierung in einem Format dargestellt werden, das von **clock format** akzeptiert wird (d. h. die Ergebnisse von **clock seconds** und **clock add**).

Das Argument *isCA* ist ein boolesches Argument, das angibt, ob es sich bei dem signierten Zertifikat um ein CA-Zertifikat handeln soll oder nicht. Wenn der Wert "true" angegeben wird, wird die Erweiterung "id-ce-basicConstraints" hinzugefügt, wobei die Argumente "critical" auf "true", "allowCA" auf "true" und "caDepth" auf -1 (unendlich) gesetzt sind.

Das Argument *extensions* ist eine Liste von Erweiterungen und deren Parametern, die für die Kodierung des erstellten Zertifikats verwendet werden sollen. Derzeit wird nur eine Erweiterung unterstützt ("id-ce-basicConstraints"). Sie akzeptiert drei Argumente: *critical*, *allowCA* und *caDepth*. Das Argument *critical* dieser Erweiterung (und jeder anderen Erweiterung) legt fest, ob der Validator das Zertifikat als ungültig zurückweisen soll, wenn er die Erweiterung nicht versteht (bei der Einstellung "true"), oder ob er die Erweiterung ignorieren soll (bei der Einstellung "false"). Das Argument *allowCA* dient dazu, als boolescher Wert festzulegen, ob wir als Zertifizierungsstelle (CA) verwendet werden dürfen oder nicht. Das Argument *caDepth* gibt an, wie viele untergeordnete Zertifizierungsstellen in der Hierarchie unter dieser Zertifizierungsstelle liegen dürfen. Ein Wert von "0" für das Argument *caDepth* bedeutet, dass diese Zertifizierungsstelle kein CA-Zertifikat signieren kann, ohne dass das Ergebnis ungültig wird. Ein Wert von "-1" bedeutet unbegrenzte Tiefe.

Beispiele

Im folgenden Beispiel wird mithilfe der TLS-Erweiterung ein Zertifikat von *www.example.com* abgerufen und dessen Inhalt ausgegeben.

```
% set so [tls::socket www.example.com 443]
sock00000229EB84E710
% tls::handshake $so
1
```

```
% set status [tls::status $so]
...output not shown...
% set cert_pem [dict get $status certificate]
...output not shown...
% set cert [::pki::x509::parse_cert $cert_pem]
...output not shown...
% dict get $cert subject
C=US, ST=California, L=Los Angeles, O=Internet Corporation for
Assigned Names and Numbers, CN=www.example.org
% dict get $cert issuer
C=US, O=DigiCert Inc, CN=DigiCert TLS RSA SHA256 2020 CA1
% clock format [dict get $cert notAfter]
Sun Dec 26 05:29:59 +0530 2021
% set extensions [dict get $cert extensions]
...output not shown...
% dict keys $extensions
authorityKeyIdentifier subjectKeyIdentifier subjectAltName
keyUsage extKeyUsage cRLDistributionPoints certificatePolicies
authorityInfoAccess id-ce-basicConstraints basicConstraints
1.3.6.1.4.1.11129.2.4.2
dict get $extensions basicConstraints
1 {0 -1}
% dict get $extensions keyUsage
1 {5 digitalSignature keyEncipherment}
% dict get $extensions extKeyUsage
0 {serverAuth clientAuth}
% dict get $extensions subjectAltName
0 {dNSName www.example.org dNSName example.com dNSName example.edu
dNSName example.net dNSName example.org dNSName www.example.com
dNSName www.example.edu dNSName www.example.net}
% dict get $extensions basicConstraints
1 {0 -1}
% dict get $extensions keyUsage
1 {5 digitalSignature keyEncipherment}
% dict get $extensions extKeyUsage
0 {serverAuth clientAuth}
```

Literatur

1. Profil für X.509-Zertifikate und Zertifikatssperrlisten (CRL) im Rahmen der öffentlichen Schlüsselinfrastruktur im Internet
2. Neue ASN.1-Module für die Public-Key-Infrastruktur unter Verwendung von X.509 (PKIX)
3. PKCS #10: Spezifikation der Syntax für Zertifikatsanfragen

Autoren

Roy Keene, Ashok P. Nadkarni

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *rsa* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright


```
Copyright (c) 2010, 2011, 2012, 2013, 2021 - 2024 Roy Keene, Andreas
    Kupries, Ashok P. Nadkarni
```


pluginmgr

pluginmgr

Stichwörter: pluginmgr, plugin management, plugin search

Name

pluginmgr – Ein Plugin verwalten

Übersicht

package require Tcl 8.5 9
package require pluginmgr ?**0.5**?
::pluginmgr *objectName* ?*option value...*?
::pluginmgr::paths *objectName name...*
objectName method ?*arg arg* ...?
objectName **clone**
objectName **configure**
objectName **configure** *option*
objectName **configure -option** *value...*
objectName **cget -option**
objectName **destroy**
objectName **do** *arg...*
objectName **interpreter**
objectName **plugin**
objectName **load** *string*
objectName **unload**
objectName **list**
objectName **path** *path*
objectName **paths**

Beschreibung

Dieses Paket stellt Befehle und Objekte für die allgemeine Verwaltung von Plugins bereit, die in eine Anwendung geladen werden können.

Um die Einführung eines weiteren Systems zur Lokalisierung von Tcl-Code zu vermeiden, baut das von diesem Paket bereitgestellte System auf dem regulären Paketverwaltungssystem auf. Jedes Plugin gilt als Paket, und ein einfacher Aufruf von **package require** genügt, um es zu finden und zu laden, sofern es vorhanden ist. Zusätzliche Pfade werden nur dann benötigt, wenn ein Plugin-Manager Teil einer verpackten Anwendung ist und nach Plugins suchen muss, die außerhalb dieser Anwendung vorhanden sind. Für diesen Fall stellt das Paket einen Befehl bereit, mit dem sich ein allgemeiner Satz solcher Pfade anhand der Namen des betreffenden Plugin-Managers und/oder der betreffenden Anwendung erstellen lässt.

Der wichtigste Beitrag dieses Pakets ist ein generischer Rahmen, der die einfache Deklaration von

1. Wie der Name eines Plugins in den Namen des Pakets übersetzt wird, das es implementiert, und umgekehrt.
2. Die Liste der Befehle, die ein Plugin als API bereitstellen muss, sowie die Liste komplexerer Prüfungen in Form von Code.
3. Die Liste der Befehle, die das Plugin von der Umgebung erwartet.

Das ermöglicht dann die einfache Erstellung von Plugin-Managern, die auf bestimmte Plugin-Typen einer Anwendung zugeschnitten sind.

Der gesamte Plugin-Code gilt als nicht vertrauenswürdig und wird stets in einem sicheren Interpreter ausgeführt. Der Interpreter ist hinreichend freigeschaltet, damit Plugins alle zusätzlich benötigten Pakete laden können.

Öffentliche API

Paketbefehle

::pluginmgr *objectName* ?*option value*...?

Dieser Befehl erstellt ein neues Plugin-Manager-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist, andernfalls im angegebenen Namespace.

Die Optionen und ihre Werte nach dem Namen des Objekts legen die Anfangskonfiguration des Manager-Objekts fest und geben die zutreffenden Plugins sowie deren API an.

::pluginmgr::paths *objectName name*...

Dieser Hilfsbefehl fügt dem angegebenen Objekt eine Reihe von Pfaden hinzu, basierend auf den angegebenen *name*s. Er sucht nach:

1. Die Umgebungsvariable *name*_PLUGINS. Ihr Inhalt wird als Liste von Paketpfaden interpretiert. Die Einträge müssen entweder durch**:** (unix) oder**;** (windows) getrennt sein.

   Der Name wird in Großbuchstaben umgewandelt.
2. Der Registrierungseintrag "HKEY_LOCAL_MACHINE\SOFTWARE*name*\PLUGINS". Sein Inhalt wird als Liste von Paketpfaden interpretiert. Die Einträge müssen durch**;** getrennt sein. Dieser Eintrag wird nur unter Windows (tm) berücksichtigt.

Die Groß- und Kleinschreibung der Buchstaben wird nicht geändert.

3. Der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE*name*\PLUGINS". Sein Inhalt wird als Liste von Paketpfaden interpretiert. Die Einträge müssen durch**;** getrennt sein. Dieser Eintrag wird nur unter Windows (tm) berücksichtigt.

   Die Groß- und Kleinschreibung der Buchstaben wird nicht geändert.
4. Das Verzeichnis "*~/.name*/plugin".
5. Das Verzeichnis "*~/.name*/plugins".

   Die Groß- und Kleinschreibung der Buchstaben wird nicht geändert.

Er fügt alle auf diese Weise gefundenen Pfade zur Liste der vom Objekt verwalteten Paketpfade hinzu.

Wenn *name* einen Namespace-Pfad enthält, wird jedes Element der Liste für jedes Präfix von *name* wiederholt, wobei:-Folgen in das passende Trennzeichen umgewandelt werden (Unterstrich für Umgebungsvariablen, Backslash für Registrierungseinträge und / für Verzeichnisse).

Beispiele:

```
    ::pluginmgr::paths ::obj docidx

    => env  DOCIDX_PLUGINS
       reg  HKEY_LOCAL_MACHINE\SOFTWARE\docidx\PLUGINS
       reg  HKEY_CURRENT_USER\SOFTWARE\docidx\PLUGINS
       path ~/.docidx/plugins

    ::pluginmgr::paths ::obj doctools::idx

    => env  DOCTOOLS_PLUGINS
       env  DOCTOOLS_IDX_PLUGINS
       reg  HKEY_LOCAL_MACHINE\SOFTWARE\doctools\PLUGINS
       reg  HKEY_LOCAL_MACHINE\SOFTWARE\doctools\idx\PLUGINS
       reg  HKEY_CURRENT_USER\SOFTWARE\doctools\PLUGINS
       reg  HKEY_CURRENT_USER\SOFTWARE\doctools\idx\PLUGINS
       path ~/.doctools/plugin
       path ~/.doctools/idx/plugin
```

Objektbefehl

Alle Befehle, die durch den Befehl **::pluginmgr** erstellt wurden (siehe Abschnitt Paketbefehle), haben die folgende allgemeine Form und können verwendet werden, um verschiedene Operationen auf ihrem Plugin-Manager-Objekt auszuführen.

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* clone**

Diese Methode erstellt ein neues Plugin-Verwaltungsobjekt und gibt den zugehörigen Objektbefehl zurück. Das erzeugte Objekt ist ein Klon des Objekts, auf das die Methode angewendet wurde. Das neue Objekt hat also dieselbe Konfiguration wie das aktuelle Objekt. Was den Zustand betrifft: Wenn im aktuellen Objekt ein Plugin geladen ist, werden dieses Plugin und der gesamte zugehörige Zustand in den erzeugten Klon verschoben, und das aktuelle Objekt wird in den Grundzustand zurückgesetzt (kein Plugin geladen). Auf diese Weise ist ein konfigurierter Plugin-Manager auch eine Fabrik für geladene Plugins.

***objectName* configure**

Die Methode gibt eine Liste aller bekannten Optionen und deren aktuelle Werte zurück, wenn sie ohne Argumente aufgerufen wird.

objectName* configure *option

Die Methode verhält sich wie die Methode **cget**, wenn sie mit einem einzigen Argument aufgerufen wird, und gibt den Wert der durch dieses Argument festgelegten Option zurück.

***objectName* configure -option *value*...**

Die Methode konfiguriert die angegebenen **option**s des Objekts neu und setzt sie auf die zugehörigen *value*s, wenn sie mit einer geraden Anzahl von Argumenten (mindestens zwei) aufgerufen wird.

Die zulässigen Optionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* cget -option**

Diese Methode erwartet eine zulässige Konfigurationsoption als Argument und gibt den aktuellen Wert dieser Option für das Objekt zurück, für das die Methode aufgerufen wurde.

Die zulässigen Konfigurationsoptionen sind im Abschnitt Objekt-Konfiguration beschrieben.

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

***objectName* do *arg*...**

Diese Methode interpretiert ihre Argumentliste als die Wörter eines Befehls und führt diesen Befehl im Ausführungskontext des Plugins aus. Das Ergebnis des ausgeführten Befehls wird zum Ergebnis der Methode. Der Aufruf schlägt mit einer Fehlermeldung fehl, wenn kein gültiges Plugin in das Manager-Objekt geladen wurde.

***objectName* interpreter**

Diese Methode gibt das Handle des sicheren Interpreters zurück, in den das aktuelle Plugin geladen ist. Eine leere Zeichenkette als Rückgabewert zeigt an, dass der Manager derzeit kein gültiges Plugin geladen hat.

***objectName* plugin**

Diese Methode gibt den Namen des aktuell geladenen Plugins zurück. Eine leere Zeichenkette als Rückgabewert zeigt an, dass der Manager derzeit kein gültiges Plugin geladen hat.

objectName* load *string

Diese Methode lädt ein benanntes Plugin, führt die Validierung des Plugins durch und initialisiert es im Manager-Objekt.

Der zum Auffinden und Laden des Plugins verwendete Algorithmus lautet wie folgt:

1. Wenn die *string* den Pfad zu einer vorhandenen Datei enthält, wird diese Datei als Implementierung des Plugins verwendet.
2. Andernfalls wird der Name des Plugins anhand des Werts der Option **-pattern** in einen

Paketnamen übersetzt und anschließend über die reguläre Paketverwaltung geladen.
3. Der Ladevorgang schlägt fehl.

Der Algorithmus zur Validierung und Initialisierung des geladenen Codes lautet wie folgt:
1. Wenn die Option **-api** nicht leer ist, werden Introspektionsbefehle verwendet, um sicherzustellen, dass das Plugin die aufgeführten Befehle bereitstellt.
2. Wenn die Option **-check** nicht leer ist, wird das angegebene Befehlspräfix aufgerufen.
3. Wenn einer der beiden oben genannten Schritte fehlschlägt, wird das Kandidaten-Plugin wieder entladen.
4. Ansonsten werden alle über die Option **-cmds** angegebenen Befehle im Plugin installiert.

Ein zuvor geladenes Plugin wird verworfen, jedoch nur, wenn das neue Plugin gefunden und erfolgreich validiert sowie initialisiert wurde. Dabei kommt es zu keiner gegenseitigen Beeinflussung von altem und neuem Plugin, da beide in getrennte sichere Interpreter gelegt werden.

***objectName* unload**
Diese Methode entlädt das aktuell geladene Plugin. Sie gibt die leere Zeichenkette zurück. Der Aufruf wird stillschweigend ignoriert, wenn überhaupt kein Plugin geladen ist.

***objectName* list**
Diese Methode nutzt den Inhalt der Option **-pattern**, um alle Pakete zu ermitteln, die als Plugins im Zuständigkeitsbereich dieses Manager-Objekts gelten können. Sie wandelt deren Namen in Plugin-Namen um und gibt eine Liste zurück, die diese enthält.

objectName* path *path
Diese Methode fügt den angegebenen *path* zur Liste der zusätzlichen Paketpfade hinzu, die bei der Suche nach einem Plugin berücksichtigt werden. Sie gibt die leere Zeichenkette zurück. Doppelte Pfade werden ignoriert, d. h. jeder Pfad wird nur einmal hinzugefügt. Pfade werden absolut gemacht, jedoch nicht normalisiert.

***objectName* paths**
Diese Methode gibt eine Liste zurück, die alle zusätzlichen Pfade enthält, die dem Plugin-Manager-Objekt seit dessen Erstellung hinzugefügt wurden.

Objekt-Konfiguration

Alle Plugin-Manager-Objekte unterstützen die folgenden Konfigurationsoptionen:

-pattern *string*
Der Wert dieser Option ist ein Glob-Muster, das genau einen '*'-Operator enthalten muss. Alle Pakete, deren Namen diesem Muster entsprechen, sind die vom Manager-Objekt erkannten Plugins. Umgekehrt ergibt das Ersetzen des '*'-Operators durch einen Plugin-Namen den Namen des Pakets, das dieses Plugin implementiert.

Diese Option hat keinen Vorgabewert, es sei denn, die Option **-name** wurde gesetzt. Sie muss gesetzt werden, bevor versucht wird, ein Plugin zu laden, entweder direkt oder über die Option **-name**.

-api *list*
Der Wert dieser Option ist eine Liste von Befehlsnamen, und jedes geladene Plugin muss diese Befehle bereitstellen. Namen, die nicht vollständig qualifiziert sind, gelten als im globalen Namespace verortet. Wenn die Liste leer ist, werden keine Erwartungen an das

Plugin gestellt. Der Standardwert ist die leere Liste.

-check *cmdprefix*
Der Wert dieser Option wird als Befehlspräfix interpretiert. Sein Zweck besteht darin, komplexe Prüfungen an einem geladenen Plugin-Paket durchzuführen, um dessen Gültigkeit zu überprüfen; diese Prüfungen gehen über eine einfache Liste der bereitgestellten Befehle hinaus.

Die Funktion wird mit dem Objektbefehl des Manager-Objekts als einzigem Argument aufgerufen und muss einen booleschen Wert zurückgeben. Ein Wert von **true** wird so interpretiert, dass das betreffende Plugin den Test bestanden hat. Der Aufruf erfolgt genau dann, wenn das betreffende Plugin bereits die über die Option **-api** festgelegte grundlegende API-Prüfung bestanden hat.

Der Standardwert ist eine leere Liste, was dazu führt, dass das Manager-Objekt den Aufruf unterdrückt und davon ausgeht, dass das Kandidaten-Plugin den Test bestanden hat.

-cmds *dict*
Der Wert dieser Option ist ein Dictionary. Es legt die Befehle fest, die dem Plugin zur Verfügung gestellt werden (als Schlüssel), sowie die vertrauenswürdigen Befehle in der Umgebung, die diese implementieren (als Werte). Die vertrauenswürdigen Befehle werden in dem durch die Option **-cmdip** angegebenen Interpreter ausgeführt. Der Standardwert ist ein leeres Dictionary.

-cmdip *ipspec*
Der Wert dieser Option ist der Pfad des Interpreters, in dem die dem Plugin übergebenen vertrauenswürdigen Befehle ausgeführt werden. Der Standardwert ist eine leere Zeichenkette, die sich auf den aktuellen Interpreter bezieht.

-setup *cmdprefix*
Der Wert dieser Option wird als Befehlspräfix interpretiert.

Er wird immer dann aufgerufen, wenn ein neuer sicherer Interpreter für ein Plugin erstellt wurde, jedoch noch bevor das Plugin geladen wird. Als einzige Argumente werden ihm der Objektbefehl des Managers und das Interpreter-Handle übergeben. Jeder Rückgabewert wird ignoriert.

Der Zweck besteht darin, einem Benutzer der Plugin-Verwaltung die Möglichkeit zu geben, Befehle, Pakete usw. zu definieren, die ein ausgewähltes Plugin beim Laden möglicherweise benötigt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pluginmgr* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

Copyright

```
Copyright (c) 2005 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


png

png
Stichwörter: png, comment, image, timestamp

Name
png – Abfrage und Bearbeitung von Metadaten bei PNG-Dateien

Übersicht
package require Tcl 8.5 9
package require crc32
package require png ?**0.4.1**?
::png::validate *file*
::png::isPNG *file*
::png::imageInfo *file*
::png::getTimestamp *file*
::png::setTimestamp *file time*
::png::getComments *file*
::png::removeComments *file*
::png::addComment *file keyword text*
::png::addComment *file keyword lang keyword2 text*
::png::getPixelDimension *file*
::png::image *file*
::png::write *file data*

Beschreibung
Dieses Paket enthält Befehle zum Abfragen und Bearbeiten von PNG-Bildern. PNG steht für *Portable Network Graphics* und wird unter http://www.libpng.org/pub/png/spec/1.2 festgelegt.

Befehle
::png::validate *file*

Gibt einen Wert zurück, der angibt, ob *file* eine gültige PNG-Datei ist. Geprüft werden die PNG-Signatur, die Prüfsumme jedes Chunks, das Vorhandensein eines Daten-Chunks, der erste Chunk auf den Header und der letzte Chunk auf den Abschluss. *Nicht* geprüft werden: die Gültigkeit der Werte in einem Chunk, mehrere Header-Chunks, nicht zusammenhängende Daten-Chunks und ein End-Chunk vor dem tatsächlichen Dateiende. Diese Prozedur kann viel Zeit brauchen.

Mögliche Rückgabewerte:

OK
Die Datei ist eine gültige PNG-Datei.

SIG
keine oder beschädigte PNG-Signatur.

BADLEN
beschädigte Chunk-Länge.

EOF
vorzeitiges Dateiende.

NOHDR
fehlender Header-Chunk.

CKSUM
CRC stimmt nicht überein.

NODATA
fehlende Daten-Chunks.

NOEND
fehlender Endmarker.

::png::isPNG *file*
Gibt einen booleschen Wert zurück, der angibt, ob die Datei *file* mit einer PNG-Signatur beginnt. Diese Überprüfung ist wesentlich schneller und weniger ressourcenintensiv als **::png::validate**, da nicht geprüft wird, ob die PNG-Daten gültig sind.

::png::imageInfo *file*
Gibt ein Dictionary mit den Schlüsseln **width**, **height**, **depth**, **color**, **compression**, **filter** und **interlace** zurück. Die Werte sind die zugehörigen Eigenschaften des PNG-Bildes in *file*. Löst einen Fehler aus, wenn *file* kein PNG-Bild ist oder die Prüfsumme des Headers ungültig ist. Zur Deutung der zurückgegebenen Eigenschaften siehe http://www.libpng.org/pub/png/spec/1.2/PNG-Chunks.html.

::png::getTimestamp *file*
Gibt die Epochenzeit zurück, wenn in dem in *file* enthaltenen PNG-Bild ein Zeitstempel-Chunk gefunden wird; andernfalls gibt es eine leere Zeichenkette zurück. Die Prüfsumme des Zeitstempel-Chunks wird nicht überprüft. Es wird ein Fehler ausgelöst, wenn *file* kein gültiges PNG-Bild ist.

::png::setTimestamp *file time*
Schreibt einen neuen Zeitstempel in *file*, entweder anstelle des bisherigen oder, wenn noch keiner da war, unmittelbar vor den Daten-Chunks. *time* ist die neue Zeit im GMT-Epochenformat. Löst einen Fehler aus, wenn *file* kein gültiges PNG-Bild ist.

::png::getComments *file*
Derzeit werden nur unkomprimierte Kommentare unterstützt. Die Prüfsummen der Kommentar-Chunks werden nicht geprüft. Gibt eine Liste zurück, in der jedes Element ein Kommentar ist. Jeder Kommentar ist selbst eine Liste. Die Liste eines Klartextkommentars besteht aus zwei Elementen: dem für Menschen lesbaren Schlüsselwort und den Textdaten. Ein Unicode-Kommentar (international) besteht aus vier Elementen: dem für Menschen lesbaren Schlüsselwort, der Sprachkennung, dem übersetzten Schlüsselwort und den Unicode-Textdaten. Löst einen Fehler aus, wenn *file* kein gültiges PNG-Bild ist.

::png::removeComments *file*

Entfernt alle Kommentare aus dem PNG-Bild in *file*. Vorsicht: Dafür wird Speicher in Höhe der Dateigröße abzüglich der Kommentare gebraucht, um das Zwischenergebnis zu halten. Löst einen Fehler aus, wenn *file* kein gültiges PNG-Bild ist.

::png::addComment *file keyword text*
Fügt dem PNG-Bild in *file* unmittelbar vor dem ersten Daten-Chunk einen einfachen *text* -Kommentar hinzu. Löst einen Fehler aus, wenn kein Daten-Chunk gefunden wird. *keyword* muss kürzer als 80 Zeichen sein, damit es der PNG-Spezifikation entspricht.

::png::addComment *file keyword lang keyword2 text*
Fügt dem PNG-Bild in *file* unmittelbar vor dem ersten Daten-Chunk einen Unicode-Kommentar (international) hinzu. Löst einen Fehler aus, wenn kein Daten-Chunk gefunden wird. *keyword* muss kürzer als 80 Zeichen sein, damit es der PNG-Spezifikation entspricht. *keyword2* ist das übersetzte *keyword*, in der durch die Sprachkennung *lang* bestimmten Sprache.

::png::getPixelDimension *file*
Gibt ein Dictionary mit den Schlüsseln **ppux**, **ppuy** und **unit** zurück, wenn die Angaben vorhanden sind. Andernfalls gibt es die leere Zeichenkette zurück.

Die Werte von **ppux** und **ppuy** sind die Pixel pro Einheit in X- bzw. Y-Richtung.

Zulässige Werte des Schlüssels **unit** sind **meter** und **unknown**. Im Fall **meter** ergibt sich der dpi-Wert durch Multiplikation mit dem Umrechnungsfaktor **0.0254**.

::png::image *file*
Gibt zu einer PNG-Datei das Bild im Format einer Liste von Scanlines zurück, wie es Tk_GetColor verwendet.

::png::write *file data*
Nimmt eine Liste von Scanlines im Format von Tk_GetColor entgegen und schreibt das repräsentierte Bild in *file*.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *png* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dateiformate

Copyright


```
Copyright (c) 2004, Code: Aaron Faupell
    <afaupell@users.sourceforge.net>
Copyright (c) 2004, Doc:  Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pop3

pop3

Stichwörter: pop3, email, mail, pop, rfc 1939, secure, ssl, tls

Name

pop3 – Tcl-Client für das POP3-E-Mail-Protokoll

Übersicht

package require Tcl 8.5 9
package require pop3 ?**1.11**?
::pop3::open ?*-msex 0|1*? ?*-retr-mode retr|list|slow*? ?*-socketcmd cmdprefix*? ?*-stls 0|1*? ?*-tls-callback stls-callback-command*? *host username password* ?*port*?
::pop3::config *chan*
::pop3::status *chan*
::pop3::last *chan*
::pop3::retrieve *chan startIndex* ?*endIndex*?
::pop3::delete *chan startIndex* ?*endIndex*?
::pop3::list *chan* ?*msg*?
::pop3::top *chan msg n*
::pop3::uidl *chan* ?*msg*?
::pop3::capa *chan*
::pop3::close *chan*

Beschreibung

Das Paket **pop3** stellt eine einfache, ausschließlich auf Tcl basierende Client-Bibliothek für das POP3-E-Mail-Protokoll bereit, wie es in RFC 1939 festgelegt ist. Es öffnet den üblichen POP3-Socket auf dem Server, übermittelt Benutzername und Passwort und stellt dann eine Tcl-API für die POP3-Protokollbefehle bereit. Alle Serverfehler werden als Tcl-Fehler zurückgegeben (ausgelöst) und müssen mit dem Tcl-Befehl **catch** abgefangen werden.

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

API

::pop3::open ?-msex 0|1? ?-retr-mode retr|list|slow? ?-socketcmd cmdprefix? ?-stls 0|1? ?-tls-callback stls-callback-command? *host username password* ?*port*?

Öffnet eine Socket-Verbindung zu dem durch *host* angegebenen Server und übermittelt *username* und *password* als Anmeldedaten an den Server. Die vorgegebene Portnummer ist **110** und lässt sich mit dem optionalen Argument *port* ändern. Der Rückgabewert ist ein Kanal, den alle anderen ::pop3-Funktionen verwenden.

Der Befehl erkennt fünf Optionen

-msex boolean

Diese Option teilt dem Paket mit, dass der Server ein MS-Exchange-Server ist (mit einigen Eigenheiten, die zu umgehen sind). Die Vorgabe ist **False**.

-retr-mode retr|list|slow

Der Abrufmodus legt fest, wie genau Nachrichten vom Server gelesen werden. Die zulässigen Werte sind **retr**, **list** und **slow**. Die Standardeinstellung ist **retr**. Weitere Informationen finden Sie unter **::pop3::retrieve**.

-socketcmd cmdprefix

Diese Option ermöglicht es dem Benutzer, die Verwendung des integrierten Befehls **socket** durch einen beliebigen API-kompatiblen Befehl zu überschreiben. Der vorgesehene Hauptzweck besteht darin, die neue Verbindung über SSL zu sichern, indem der Befehl **tls::socket** angegeben wird. Dieser Befehl wird ebenfalls speziell erkannt und ändert den Standardport der Verbindung auf **995**.

-stls boolean

Diese Option weist das Paket an, die Verbindung mit SSL oder TLS zu sichern. Sie führt STARTTLS aus, wie in IETF RFC 2595 beschrieben: zuerst eine normale, unverschlüsselte Verbindung, dann die Aushandlung einer SSLv3- oder TLSv1-Verbindung. Lässt sich die Verbindung nicht sichern, wird sie geschlossen und ein Fehler zurückgegeben.

-tls-callback stls-callback-command

Mit dieser Option lässt sich der **tls::callback** überschreiben, der beim SSL/TLS-Handshake von **-stls** verwendet wird. Zur Umsetzung dieser Rückruffunktion siehe das TLS-Handbuch.

::pop3::config *chan*

Gibt die Konfiguration der POP3-Verbindung, die durch das Kanal-Handle *chan* identifiziert wird, als serialisiertes Array zurück.

::pop3::status *chan*

Fragt den Server nach dem Status des Mail-Spools. Der Status kommt als Liste mit zwei Elementen zurück. Das erste ist die Anzahl der E-Mail-Nachrichten auf dem Server, das zweite die Größe (in Oktetten, Blöcken zu 8 Bit) des gesamten Mail-Spools.

::pop3::last *chan*

Fragt den Server nach der zuletzt aus dem Spool gelesenen E-Mail. Der Wert umfasst alle

Nachrichten, die irgendein Client des Anmeldekontos gelesen hat. Der E-Mail-Server muss den Befehl nicht unterstützen; dann gibt er möglicherweise 0 oder einen Fehler zurück.

::pop3::retrieve *chan startIndex ?endIndex?*

Ruft einen Bereich von Nachrichten vom Server ab. Ist *endIndex* nicht angegeben, wird nur eine Nachricht abgerufen. Der Rückgabewert ist eine Liste, die jede Nachricht als eigenes Element enthält. Zu *startIndex* und *endIndex* siehe unten.

Der Abrufmodus legt fest, wie die Nachrichten vom Server gelesen werden. Der Modus **retr** nimmt an, dass der Befehl RETR die Größe der Nachricht als Teil des Befehlsstatus liefert, und nutzt das, um die Nachricht effizient zu lesen. Im Modus **list** liefert RETR die Größe nicht, der Befehl LIST aber schon; damit wird die Nachrichtengröße vor dem eigentlichen Abruf geholt, der dann effizient erfolgen kann. Im letzten Modus, **slow**, lässt sich die Größe der Nachricht auf keine Weise ermitteln, und die Nachricht wird Zeile für Zeile vom Server gelesen.

Das System prüft den konfigurierten Modus und fällt auf die langsameren Modi zurück, wenn die genannten Annahmen nicht zutreffen.

::pop3::delete *chan startIndex ?endIndex?*

Löscht einen Bereich von Nachrichten auf dem Server. Ist *endIndex* nicht angegeben, wird nur eine Nachricht gelöscht. Die Indizes werden auf dem Server nicht neu geordnet. Wird Nachricht 1 gelöscht, ist die erste Nachricht in der Warteschlange Nachricht 2 (der Nachrichtenindex 1 ist nicht mehr gültig). Zu *startIndex* und *endIndex* siehe unten.

startIndex

Der *startIndex* kann entweder ein Index einer bestimmten Nachricht sein, beginnend mit dem Index 1, oder einen der folgenden Werte annehmen:

start

Dies ist ein logischer Wert für die erste Nachricht im Spool, der dem Wert 1 entspricht.

next

Die Nachricht unmittelbar nach der zuletzt gelesenen Nachricht, siehe **::pop3::last**.

end

Die neueste Nachricht im Spool (das Ende des Spools). Damit lässt sich nur die neueste Nachricht abrufen.

endIndex

Der Parameter *endIndex* ist optional und hat standardmäßig den Wert "-1", was bedeutet, dass nur die durch *startIndex* angegebene Nachricht abgerufen wird. Sofern er angegeben wird, kann es sich um den Index einer bestimmten Nachricht handeln, beginnend mit dem Index "1", oder er kann einen der folgenden Werte annehmen:

last

Bei dieser Nachricht handelt es sich um die zuletzt von einem POP3-Client gelesene Nachricht; siehe **::pop3::last**.

end

Die neueste Nachricht im Spool (das Ende des Spools).

::pop3::list *chan ?msg?*

Gibt die Scan-Auflistung des Postfachs zurück. Wenn der Parameter *msg* angegeben ist, wird nur die Auflistung für diese Nachricht zurückgegeben.

::pop3::top *chan msg n*
Optionaler POP3-Befehl, den nicht alle Server unterstützen. **::pop3::top** holt die Kopfzeilen der durch *msg* angegebenen Nachricht und *n* Zeilen aus dem Nachrichtentext.

::pop3::uidl *chan* ?*msg*?
Optionaler POP3-Befehl, den nicht alle Server unterstützen. **::pop3::uidl** gibt das UID-Listing des Postfachs zurück. Ist *msg* angegeben, kommt nur das Listing dieser Nachricht zurück.

::pop3::capa *chan*
Optionaler POP3-Befehl, den nicht alle Server unterstützen. **::pop3::capa** gibt eine Liste der Fähigkeiten des Servers zurück. TOP, SASL, UIDL, LOGIN-DELAY und STLS sind typische Fähigkeiten. Siehe IETF RFC 2449.

::pop3::close *chan*
Schließt die Verbindung geordnet, nachdem ein POP3-Befehl QUIT über den Socket gesendet wurde.

Sichere Mail-Übertragung

Eine POP3-Verbindung lässt sich mit SSL/TLS sichern, indem das Paket **TLS** vorausgesetzt wird und dann entweder die Option **-socketcmd** oder die Option **-stls** des Befehls **pop3::open** zum Einsatz kommt. Die erste Methode, die Option **-socketcmd**, erzwingt beim Öffnen der Verbindung den Befehl **tls::socket**. Das passt zu POP3-Servern, die nur SSL-Verbindungen erwarten. Sie lauschen in der Regel auf Port 995.

```
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create secured pop3 channel
    pop3::open -socketcmd tls::socket \
        $thehost $theuser $thepassword

    ...
```

Die zweite Methode, die Option **-stls**, verbindet sich mit dem üblichen POP3-Port und führt dann einen STARTTLS-Handshake aus. Das funktioniert nur bei POP3-Servern mit dieser Fähigkeit. Das Paket bestätigt, dass der Server STARTTLS unterstützt und der Handshake korrekt ausgeführt wurde, bevor die Authentifizierung folgt.

```
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create secured pop3 channel
    pop3::open -stls 1 \
        $thehost $theuser $thepassword

    ...
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *pop3* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

pop3d

pop3d
Stichwörter: pop3d, internet, network, pop3, protocol, rfc 1939, secure, ssl, tls

Name
pop3d – Implementierung eines Tcl-POP3-Servers

Übersicht
package require Tcl 8.5 9
package require pop3d ?**1.2.0**?
::pop3d::new ?*serverName*?
serverName *option* ?*arg arg* ...?
serverName **up**
serverName **down**
serverName **destroy** ?*mode*?
serverName **configure**
serverName **configure** *-option*
serverName **configure** *-option value*...
serverName **cget** *-option*
serverName **conn** list
serverName **conn** state *id*
authCmd **exists** *name*
authCmd **lookup** *name*
storageCmd **dele** *mbox msgList*
storageCmd **lock** *mbox*
storageCmd **unlock** *mbox*
storageCmd **size** *mbox* ?*msgId*?
storageCmd **stat** *mbox*
storageCmd **get** *mbox msgId*

Beschreibung
::pop3d::new ?*serverName*?
Dieser Befehl erstellt ein neues Serverobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *serverName* lautet.

Mit dem Befehl **serverName** können verschiedene Operationen auf dem Server aufgerufen werden. Er hat die folgende allgemeine Form:

serverName *option* ?*arg arg ...*?
Option und die *arg*s legen das genaue Verhalten des Befehls fest.

Ein POP3-Server kann auf jedem Port gestartet werden, für den der Aufrufer vom Betriebssystem die Berechtigung hat. Der Standardport ist 110, der Port aus dem in RFC 1939 (http://www.rfc-editor.org/rfc/rfc1939.txt) festgelegten Standard. Nach dem Erzeugen, Konfigurieren und Starten lauscht das Server-Objekt auf diesem Port, nimmt Verbindungen an und verarbeitet sie gemäß dem POP3-Protokoll.

Hinweis: Der von diesem Modul bereitgestellte Server verarbeitet von sich aus nur das Basisprotokoll. Für die höheren Ebenen der Benutzerauthentifizierung und die Verarbeitung der eigentlichen Postfachinhalte werden Rückruffunktionen aufgerufen.

Für Server-Objekte sind diese Befehle möglich:

***serverName* up**
Nach diesem Aufruf wartet der Server auf Verbindungen an seinem konfigurierten Port.

***serverName* down**
Nach diesem Aufruf wird der Server keine Verbindungen mehr annehmen. Bestehende Verbindungen sind davon nicht betroffen.

***serverName* destroy ?*mode*?**
Zerstört das Server-Objekt. Derzeit offene Verbindungen werden je nach gewähltem Modus behandelt. Die vorgesehenen Werte für *mode* sind:

kill
Zerstört den Server sofort und schließt alle derzeit offenen Verbindungen zwangsweise. Das ist der Standardmodus.

defer
Nimmt keine neuen Verbindungen mehr an und zerstört den Server erst, wenn die letzte der derzeit offenen Verbindungen geschlossen wurde.

***serverName* configure**
Gibt eine Liste zurück, die alle Optionen und deren aktuelle Werte in einem Format enthält, das für die Verwendung durch den Befehl **array set** geeignet ist. Die Optionen selbst werden im Abschnitt Optionen beschrieben.

serverName* configure *-option
Gibt den aktuellen Wert der angegebenen Option zurück. Dies ist ein Alias für die Methode **cget**. Die Optionen selbst werden im Abschnitt Optionen beschrieben.

serverName* configure *-option value...
Setzt die angegebene Option auf den angegebenen Wert. Die Optionen selbst werden im Abschnitt Optionen beschrieben.

serverName* cget *-option
Gibt den aktuellen Wert der angegebenen Option zurück. Die Optionen selbst werden im Abschnitt Optionen beschrieben.

***serverName* conn list**
Gibt eine Liste mit den Kennungen aller derzeit offenen Verbindungen zurück.

serverName* conn state *id
Gibt eine für [**array set**] geeignete Liste zurück, die den Status der Verbindung enthält, auf die *id* verweist.

Optionen

Für POP3-Serverobjekte stehen die folgenden Optionen zur Verfügung.

-port *port*
: Legt den *port* fest, auf dem nach neuen Verbindungen gelauscht wird. Voreinstellung ist 110. Diese Option ist etwas Besonderes. Ist *port* auf "0" gesetzt, wählt der Server, genauer das Betriebssystem, selbst einen freien Port. Eine Abfrage von **-port** gibt die Kennung dieses gewählten Ports zurück. Eine Änderung des Ports bei laufendem Server ändert weder den zurückgegebenen Wert noch den Port, auf dem der Server lauscht. Erst nach einem Neustart über einen Aufruf von **down** und anschließend **up** wirkt der neue Port. Erst dann ändert sich auch der Wert, den eine Abfrage von **-port** zurückgibt.

-auth *command*
: Definiert ein *command*-Präfix, das aufgerufen wird, wenn ein Benutzer authentifiziert werden muss. Ist kein solcher Befehl festgelegt, lehnt der Server alle Benutzer ab. Die Schnittstelle, die das Befehlspräfix bereitstellen muss, steht im Abschnitt Authentifizierung.

-storage *command*
: Definiert ein *command*-Präfix, das aufgerufen wird, wenn Postfachinhalte verarbeitet werden müssen. Ist kein solcher Befehl festgelegt, behauptet der Server, alle Postfächer seien leer. Die Schnittstelle, die das Befehlspräfix bereitstellen muss, steht im Abschnitt Postfächer.

-socket *command*
: Definiert ein *command*-Präfix, das zum Öffnen des Listening-Sockets aufgerufen wird. Damit kann der POP3-Server auf einem SSL-Socket lauschen, wie ihn das Paket **tls** bereitstellt; siehe den Befehl **tls::socket**.

Authentifizierung

Hier wird die Schnittstelle beschrieben, die die Authentifizierungs-Rückruffunktion bereitstellen muss, damit POP3-Server nach der Schnittstelle dieses Moduls sie nutzen können.

authCmd* exists *name
: Dieser Methode wird ein Benutzer*name* übergeben. Sie muss einen booleschen Wert zurückgeben, der angibt, ob der angegebene Benutzer existiert.

authCmd* lookup *name
: Dieser Methode wird ein Benutzer*name* übergeben. Sie muss eine Liste mit zwei Elementen zurückgeben, in dieser Reihenfolge das Passwort dieses Benutzers und eine Speicherreferenz.

  Die Speicherreferenz wird unverändert an die Speicher-Rückruffunktion übergeben; siehe die Abschnitte Optionen und Postfächer, dort die Option, die sie festlegt, bzw. die bereitzustellende Schnittstelle.

Postfächer

Hier wird die Schnittstelle beschrieben, die die Speicher-Rückruffunktion bereitstellen muss, damit POP3-Server nach der Schnittstelle dieses Moduls sie nutzen können. Das Argument *mbox* ist die Speicherreferenz, wie sie die Methode **lookup** des Authentifizierungsbefehls zurückgibt; siehe den Abschnitt Authentifizierung.

storageCmd* dele *mbox msgList
: Löscht die Nachrichten, deren numerische IDs in der *msgList* enthalten sind, aus dem über *mbox* festgelegten Postfach.

storageCmd* lock *mbox
: Diese Methode sperrt das angegebene Postfach für die Nutzung durch eine einzelne Verbindung zum Server. Das ist nötig, damit mehrere offene Verbindungen zum selben Postfach kein Chaos anrichten. Die ergänzende Methode ist **unlock**. Der Befehl gibt true zurück, wenn die Sperre gesetzt werden konnte, andernfalls false.

storageCmd* unlock *mbox
: Dies ist die ergänzende Methode zu **lock**; sie hebt die Sperre für das angegebene Postfach auf.

***storageCmd* size *mbox* ?*msgId*?**
: Ermittelt die Größe der über die ID in *msgId* angegebenen Nachricht in Byte und gibt diesen Wert zurück. Wurde keine Nachrichten-ID angegeben, gibt der Befehl die Größe des gesamten Maildrops zurück.

storageCmd* stat *mbox
: Ermittelt die Anzahl der Nachrichten in dem angegebenen Postfach und gibt diese Zahl zurück.

storageCmd* get *mbox msgId
: Gibt ein Handle für die angegebene Nachricht zurück. Dieses Handle ist ein MIME-Token, das der in der Dokumentation des Pakets **mime** beschriebenen Schnittstelle entspricht. Der POP3-Server nutzt die Funktionalität des MIME-Tokens, um die E-Mail an den Anforderer am anderen Ende einer POP3-Verbindung zu senden.

Sichere Mail-Übertragung

Die Option **-socket** (siehe Optionen) ermöglicht es den Benutzern des Pakets, die Art und Weise zu überschreiben, wie der Server seinen Listening-Socket öffnet. Die vorgesehene Hauptanwendung ist die Angabe des Befehls **tls::socket** (siehe Paket **tls**) zur Sicherung der Kommunikation.

```
package require tls
tls::init \
    ...

pop3d::new S -socket tls::socket
...
```

Literatur

1. RFC 1939
2. RFC 2449

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pop3d* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright

```
Copyright (c) 2002-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2005 Reinhard Max  <max@suse.de>
```


pop3d_dbox
Stichwörter: pop3d_dbox, internet, network, pop3, protocol, rfc 822

Name
pop3d::dbox – Einfache Postfachdatenbank für POP3D

Übersicht
package require Tcl 8.5 9
package require pop3d::dbox ?**1.0.3**?
::pop3d::dbox::new ?*dbName*?
dbName *option* ?*arg arg ...*?
dbName **destroy**
dbName **base** *base*
dbName **add** *mbox*
dbName **remove** *mbox*
dbName **move** *old new*
dbName **list**
dbName **exists** *mbox*
dbName **locked** *mbox*
dbName **lock** *mbox*
dbName **unlock** *mbox*
dbName **stat** *mbox*
dbName **size** *mbox* ?*msgId*?
dbName **dele** *mbox msgList*
storageCmd **get** *mbox msgId*

Beschreibung
Das Paket **pop3d::dbox** bietet einfache, grundlegende Funktionen zur Verwaltung von Postfächern. Jedes Postfach-Objekt verwaltet ein einzelnes Basisverzeichnis, dessen Unterverzeichnisse die verwalteten Postfächer sind. E-Mails in einem Postfach sind Dateien im Postfachverzeichnis; jede dieser Dateien enthält eine einzelne E-Mail, Header und Body, in einem zu RFC 822 (http://www.rfc-editor.org/rfc/rfc822.txt) konformen Format.

Jedes Postfach-Objekt, das der unten beschriebenen Schnittstelle folgt, kann zusammen mit dem POP3-Server-Kern aus dem Paket **pop3d** verwendet werden. Insbesondere lassen sich die von diesem Paket erzeugten Objekte direkt in der Speicher-Rückruffunktion von POP3-Servern verwenden, die derselben Schnittstelle folgen wie die vom Paket **pop3d** erzeugten Server.

::pop3d::dbox::new ?*dbName*?

Dieser Befehl erstellt ein neues Datenbankobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *dbName* lautet.

Mit dem Befehl **dbName** können verschiedene Operationen auf der Datenbank ausgeführt werden. Er hat die folgende allgemeine Form:

dbName *option* ?*arg arg ...*?
Option und die *arg*s legen das genaue Verhalten des Befehls fest.

Für Datenbankobjekte sind folgende Befehle möglich:

***dbName* destroy**
Löscht die Postfachdatenbank und alle temporären Daten. Das mit dem Objekt verknüpfte Verzeichnis wird nicht gelöscht.

dbName* base *base
Legt das Basisverzeichnis fest, in dem die zu verwaltenden Postfächer liegen. Wird diese Methode nicht aufgerufen, funktioniert keine der folgenden Methoden.

dbName* add *mbox
Fügt der Datenbank ein Postfach mit dem Namen *mbox* hinzu. Der Name muss eine gültige Pfad-Komponente sein.

dbName* remove *mbox
Entfernt das über *mbox* festgelegte Postfach sowie die darin enthaltenen E-Mails aus der Datenbank. Diese Methode schlägt fehl, wenn das festgelegte Postfach gesperrt ist.

dbName* move *old new
Ändert den Namen des Postfachs *old* in *new*.

***dbName* list**
Gibt eine Liste zurück, die die Namen aller Postfächer im Verzeichnis enthält, das der Datenbank zugeordnet ist.

dbName* exists *mbox
Gibt true zurück, wenn das Postfach mit dem Namen *mbox* in der Datenbank vorhanden ist, andernfalls false.

dbName* locked *mbox
Prüft, ob das über *mbox* angegebene Postfach derzeit gesperrt ist.

dbName* lock *mbox
Diese Methode sperrt das angegebene Postfach für die Nutzung durch eine einzelne Verbindung zum Server. Das ist nötig, damit mehrere offene Verbindungen zum selben Postfach kein Chaos anrichten. Die ergänzende Methode ist **unlock**. Der Befehl gibt true zurück, wenn die Sperre gesetzt werden konnte, andernfalls false.

dbName* unlock *mbox
Dies ist die ergänzende Methode zu **lock**; sie hebt die Sperre für das angegebene Postfach auf.

dbName* stat *mbox
Ermittelt die Anzahl der Nachrichten in dem angegebenen Postfach und gibt diese Zahl zurück. Diese Methode schlägt fehl, wenn das Postfach *mbox* nicht gesperrt ist.

***dbName* size *mbox* ?*msgId*?**
Ermittelt die Größe der durch die ID in *msgId* angegebenen Nachricht in Byte und gibt diesen Wert zurück. Der Befehl gibt die Größe des gesamten Maildrops zurück, wenn keine

Nachrichten-ID angegeben wurde. Falls angegeben, muss die *msgId* im Bereich "1 ... [*dbName* **stat**]" liegen, andernfalls schlägt der Aufruf fehl. Wenn **stat** vor diesem Aufruf nicht aufgerufen wurde, geht **size** davon aus, dass sich keine Nachrichten im Postfach befinden.

dbName* dele *mbox msgList

Löscht die Nachrichten, deren numerische IDs in der *msgList* enthalten sind, aus dem über *mbox* angegebenen Postfach. Die *msgList* darf nicht leer sein, da dieser Aufruf sonst fehlschlägt. Die numerischen IDs in *msgList* müssen im Bereich "1 ... [*dbName* **stat**]" liegen, andernfalls schlägt dieser Aufruf fehl. Wenn **stat** vor diesem Aufruf nicht aufgerufen wurde, geht **dele** davon aus, dass sich keine Nachrichten im Postfach befinden.

storageCmd* get *mbox msgId

Gibt ein Handle für die angegebene Nachricht zurück. Dieses Handle ist ein MIME-Token, das der in der Dokumentation des Pakets **mime** beschriebenen Schnittstelle entspricht. Das Token ist *schreibgeschützt*. Mit anderen Worten: Der Aufrufer darf mit dem Token alles tun, außer es zu ändern. Die *msgId* muss im Bereich "1 ... [*dbName* **stat**]" liegen, andernfalls schlägt dieser Aufruf fehl. Wenn **stat** vor diesem Aufruf nicht aufgerufen wurde, geht **get** davon aus, dass sich keine Nachrichten im Postfach befinden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pop3d* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pop3d_udb

Stichwörter: pop3d_udb, internet, network, pop3, protocol

Name

pop3d::udb – Einfache Benutzerdatenbank für pop3d

Übersicht

package require Tcl 8.5 9
package require pop3d::udb ?**1.0.1**?
::pop3d::udb::new ?*dbName*?
dbName *option* ?*arg arg* ...?
dbName **destroy**

dbName **add** *user pwd storage*
dbName **remove** *user*
dbName **rename** *user newName*
dbName **lookup** *user*
dbName **exists** *user*
dbName **who**
dbName **save** ?*file*?
dbName **read** *file*

Beschreibung

Das Paket **pop3d::udb** stellt einfache Datenbanken im Arbeitsspeicher bereit, die zusammen mit dem POP3-Server-Kern aus dem Paket **pop3d** verwendet werden können. Die Datenbanken verwenden die Namen der Benutzer als Schlüssel und ordnen ihnen Passwörter und Speicherreferenzen zu.

Die von diesem Paket erzeugten Objekte können direkt in der Authentifizierungs-Rückruffunktion von POP3-Servern verwendet werden, die derselben Schnittstelle folgen wie die vom Paket **pop3d** erzeugten Server.

::pop3d::udb::new ?*dbName*?
: Dieser Befehl erstellt ein neues Datenbankobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *dbName* lautet.

Mit dem Befehl **dbName** können verschiedene Operationen auf der Datenbank ausgeführt werden. Er hat die folgende allgemeine Form:

dbName *option* ?*arg arg ...*?
: *Option* und die *arg*s legen das genaue Verhalten des Befehls fest.

Für Datenbankobjekte sind folgende Befehle möglich:

***dbName* destroy**
: Löscht das Datenbankobjekt.

dbName* add *user pwd storage
: Fügt einen neuen Benutzer hinzu oder ändert die Daten eines vorhandenen Benutzers. Speichert das Passwort und die *storage*-Referenz für den angegebenen *user*.

dbName* remove *user
: Entfernt den angegebenen *user* aus der Datenbank.

dbName* rename *user newName
: Ändert den Namen des angegebenen *user* in *newName*.

dbName* lookup *user
: Sucht in der Datenbank nach dem angegebenen *user* und gibt eine Liste mit zwei Elementen zurück, in dieser Reihenfolge das zugehörige Passwort und die Speicherreferenz. Löst einen Fehler aus, wenn der Benutzer nicht gefunden wurde. Das ist die Schnittstelle, die die Authentifizierungs-Rückruffunktion des Pakets **pop3d** erwartet.

dbName* exists *user
: Gibt true zurück, wenn der angegebene *user* der Datenbank bekannt ist, andernfalls false.

***dbName* who**
: Gibt eine Liste der in der Datenbank bekannten Benutzer zurück.

***dbName* save ?*file*?**

Speichert den Inhalt der Datenbank in der angegebenen *file*. Ist keine Datei angegeben, wird der Pfad verwendet, der zuletzt bei einem Aufruf von *dbName* **read** benutzt wurde. Die erzeugte Datei kann mit der Methode **read** gelesen werden.

dbName* read *file

Liest die angegebene *file* und fügt die darin enthaltenen Benutzerdefinitionen zur Datenbank hinzu. Da die Datei tatsächlich per **source** ausgeführt wird, schützt ein sicherer Interpreter vor bösartigem Code. Dieser Interpreter kennt den Befehl **add**, mit dem Benutzer und die zugehörigen Daten in diese Datenbank aufgenommen werden. Der Befehl hat dieselbe Argumentsignatur wie die Methode **add**. Der Pfad der *file* wird intern gemerkt, damit er beim nächsten Aufruf von *dbName* **save** ohne Argument verwendet werden kann.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pop3d* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright


```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


practcl

practcl

Stichwörter: practcl

Name

practcl – Das Practcl-Modul

Übersicht

package require TclOO 1.0

proc **practcl::cat** *fname*

proc **practcl::docstrip** *text*

proc **putb** ?*map*? *text*

proc **Proc** *name arglist body*

proc **noop** ?*args*?

proc **practcl::debug** ?*args*?

proc **practcl::doexec** ?*args*?
proc **practcl::doexec_in** *path* ?*args*?
proc **practcl::dotclexec** ?*args*?
proc **practcl::domake** *path* ?*args*?
proc **practcl::domake.tcl** *path* ?*args*?
proc **practcl::fossil** *path* ?*args*?
proc **practcl::fossil_status** *dir*
proc **practcl::os**
proc **practcl::mkzip** *exename barekit vfspath*
proc **practcl::sort_dict** *list*
proc **practcl::local_os**
proc **practcl::config.tcl** *path*
proc **practcl::read_configuration** *path*
proc **practcl::tcllib_require** *pkg* ?*args*?
proc **practcl::platform::tcl_core_options** *os*
proc **practcl::platform::tk_core_options** *os*
proc **practcl::read_rc_file** *filename* ?*localdat* ?
proc **practcl::read_sh_subst** *line info*
proc **practcl::read_sh_file** *filename* ?*localdat* ?
proc **practcl::read_Config.sh** *filename*
proc **practcl::read_Makefile** *filename*
proc **practcl::cputs** *varname* ?*args*?
proc **practcl::tcl_to_c** *body*
proc **practcl::_tagblock** *text* ?*style* **tcl**? ?*note* ?
proc **practcl::de_shell** *data*
proc **practcl::grep** *pattern* ?*files* ?
proc **practcl::file_lexnormalize** *sp*
proc **practcl::file_relative** *base dst*
proc **practcl::findByPattern** *basedir patterns*
proc **practcl::log** *fname comment*
proc **practcl::_pkgindex_simpleIndex** *path*
proc **practcl::_pkgindex_directory** *path*
proc **practcl::_pkgindex_path_subdir** *path*
proc **practcl::pkgindex_path** ?*args*?
proc **practcl::installDir** *d1 d2*
proc **practcl::copyDir** *d1 d2* ?*toplevel* **1**?
proc **practcl::buildModule** *modpath*

proc **practcl::installModule** *modpath DEST*
proc **practcl::trigger** ?*args*?
proc **practcl::depends** ?*args*?
proc **practcl::target** *name info* ?*action* ?
method **constructor**
method **argspec** *argspec*
method **comment** *block*
method **keyword.Annotation** *resultvar commentblock type name body*
method **keyword.Class** *resultvar commentblock name body*
method **keyword.class** *resultvar commentblock name body*
method **keyword.Class_Method** *resultvar commentblock name* ?*args*?
method **keyword.method** *resultvar commentblock name* ?*args*?
method **keyword.proc** *commentblock name argspec*
method **reset**
method **Main**
method **section.method** *keyword method minfo*
method **section.annotation** *type name iinfo*
method **section.class** *class_name class_info*
method **section.command** *procinfo*
method **manpage** ?**header** *value*? ?**footer** *value*? ?**authors** *list*?
method **scan_text** *text*
method **scan_file** *filename*
method **_MorphPatterns**
method **define** *submethod* ?*args*?
method **graft** ?*args*?
method **initialize**
method **link** *command* ?*args*?
method **morph** *classname*
method **script** *script*
method **select**
method **source** *filename*
classmethod **select** *object*
method **config.sh**
method **BuildDir** *PWD*
method **MakeDir** *srcdir*
method **read_configuration**
method **build-cflags** *PROJECT DEFS namevar versionvar defsvar*

method **critcl** ?*args*?
method **Autoconf**
method **BuildDir** *PWD*
method **ConfigureOpts**
method **MakeDir** *srcdir*
method **make {} autodetect**
method **make {} clean**
method **make {} compile**
method **make {} install** *DEST*
method **build-compile-sources** *PROJECT COMPILE CPPCOMPILE INCLUDES*
method **build-Makefile** *path PROJECT*
method **build-library** *outfile PROJECT*
method **build-tclsh** *outfile PROJECT* ?*path* **auto**?
method **BuildDir** *PWD*
method **make {} autodetect**
method **make {} clean**
method **make {} compile**
method **make {} install** *DEST*
method **MakeDir** *srcdir*
method **NmakeOpts**
method **constructor** *module_object name info* ?*action_body* ?
method **do**
method **check**
method **output**
method **reset**
method **triggers**
method **constructor** *parent* ?*args*?
method **child** *method*
method **go**
method **cstructure** *name definition* ?*argdat* ?
method **include** *header*
method **include_dir** ?*args*?
method **include_directory** ?*args*?
method **c_header** *body*
method **c_code** *body*
method **c_function** *header body* ?*info* ?
method **c_tcloomethod** *name body* ?*arginfo* ?

method **cmethod** *name body* ?*arginfo* ?
method **c_tclproc_nspace** *nspace*
method **c_tclcmd** *name body* ?*arginfo* ?
method **c_tclproc_raw** *name body* ?*arginfo* ?
method **tcltype** *name argdat*
method **project-compile-products**
method **implement** *path*
method **initialize**
method **linktype**
method **generate-cfile-constant**
method **generate-cfile-header**
method **generate-cfile-tclapi**
method **generate-loader-module**
method **Collate_Source** *CWD*
method **select**
classmethod **select** *object*
method **code** *section body*
method **Collate_Source** *CWD*
method **project-compile-products**
method **generate-debug** ?*spaces* ?
method **generate-cfile-constant**
method **generate-cfile-public-structure**
method **generate-cfile-header**
method **generate-cfile-global**
method **generate-cfile-private-typedef**
method **generate-cfile-private-structure**
method **generate-cfile-functions**
method **generate-cfile-tclapi**
method **generate-hfile-public-define**
method **generate-hfile-public-macro**
method **generate-hfile-public-typedef**
method **generate-hfile-public-structure**
method **generate-hfile-public-headers**
method **generate-hfile-public-function**
method **generate-hfile-public-includes**
method **generate-hfile-public-verbatim**
method **generate-loader-external**

method **generate-loader-module**
method **generate-stub-function**
method **IncludeAdd** *headervar* ?*args*?
method **generate-tcl-loader**
method **generate-tcl-pre**
method **generate-tcl-post**
method **linktype**
method **Ofile** *filename*
method **project-static-packages**
method **toolset-include-directory**
method **target** *method* ?*args*?
method **project-compile-products**
method **generate-loader-module**
method **project-compile-products**
method **linker-products** *configdict*
method **initialize**
variable **make_object**
method **_MorphPatterns**
method **add** ?*args*?
method **install-headers** ?*args*?
method **make {} _preamble**
method **make {} pkginfo**
method **make {} objects**
method **make {} object** *name*
method **make {} reset**
method **make {} trigger** ?*args*?
method **make {} depends** ?*args*?
method **make {} filename** *name*
method **make {} target** *name Info body*
method **make {} todo**
method **make {} do**
method **child** *which*
method **generate-c**
method **generate-h**
method **generate-loader**
method **initialize**
method **implement** *path*

method **linktype**
method **_MorphPatterns**
method **constructor** ?*args*?
method **add_object** *object*
method **add_project** *pkg info* ?*oodefine* ?
method **add_tool** *pkg info* ?*oodefine* ?
method **build-tclcore**
method **child** *which*
method **linktype**
method **project** *pkg* ?*args*?
method **tclcore**
method **tkcore**
method **tool** *pkg* ?*args*?
method **clean** *PATH*
method **project-compile-products**
method **go**
method **generate-decls** *pkgname path*
method **implement** *path*
method **generate-make** *path*
method **linktype**
method **package-ifneeded** ?*args*?
method **shared_library** ?*filename* ?
method **static_library** ?*filename* ?
method **build-tclkit_main** *PROJECT PKG_OBJS*
method **Collate_Source** *CWD*
method **wrap** *PWD exename vfspath* ?*args*?
classmethod **Sandbox** *object*
classmethod **select** *object*
classmethod **claim_option**
classmethod **claim_object** *object*
classmethod **claim_path** *path*
method **scm_info**
method **DistroMixIn**
method **Sandbox**
method **SrcDir**
method **ScmTag**
method **ScmClone**

method **ScmUnpack**
method **ScmUpdate**
method **Unpack**
classmethod **claim_object** *object*
classmethod **claim_option**
classmethod **claim_path** *path*
method **ScmUnpack**
classmethod **claim_object** *obj*
classmethod **claim_option**
classmethod **claim_path** *path*
method **scm_info**
method **ScmClone**
method **ScmTag**
method **ScmUnpack**
method **ScmUpdate**
classmethod **claim_object** *obj*
classmethod **claim_option**
classmethod **claim_path** *path*
method **ScmTag**
method **ScmUnpack**
method **ScmUpdate**
method **_MorphPatterns**
method **BuildDir** *PWD*
method **child** *which*
method **compile**
method **go**
method **install** ?*args*?
method **linktype**
method **linker-products** *configdict*
method **linker-external** *configdict*
method **linker-extra** *configdict*
method **env-bootstrap**
method **env-exec**
method **env-install**
method **env-load**
method **env-present**
method **sources**

method **update**
method **unpack**
method **env-bootstrap**
method **env-present**
method **linktype**
method **env-bootstrap**
method **env-install**
method **env-present**
method **install** *DEST*
method **kettle** *path* ?*args*?
method **install** *DEST*
method **install** *DEST*
method **env-bootstrap**
method **env-install**
method **env-present**
method **install** *DEST*
method **install-module** *DEST* ?*args*?
method **env-bootstrap**
method **env-install**
method **install** *DEST*
method **install-module** *DEST* ?*args*?
method **clean**
method **env-install**
method **project-compile-products**
method **ComputeInstall**
method **go**
method **linker-products** *configdict*
method **project-static-packages**
method **BuildDir** *PWD*
method **compile**
method **Configure**
method **install** *DEST*
method **install** *DEST*
method **install** *DEST*
method **env-bootstrap**
method **env-present**
method **env-install**

method **go**

method **linktype**

Beschreibung

Das Modul "Practcl" ist ein Werkzeug zur Integration umfangreicher Module für C-API-Tcl-Code, der benutzerdefinierte Tcl-Typen und TclOO-Objekte erfordert.

Das Konzept von Practcl besteht darin, dass es sich um ein Paket aus einer einzigen Datei handelt, das jedes Tcl-basierte Projekt bei der Verteilung, Kompilierung, Verknüpfung, VFS-Vorbereitung, Erstellung der ausführbaren Datei und Installation unterstützen kann. Practcl ermöglicht es zudem, das Build-System eines Projekts von einem anderen Projekt aus aufzurufen, wodurch komplexe Projekte wie beispielsweise ein statisch verknüpftes Basekit relativ einfach erstellt werden können.

Practcl wird als einzelne Datei ausgeliefert und weist, abgesehen von einem Tcl 8.6-Interpreter, keine externen Abhängigkeiten auf.

Ein praxisorientiertes Projekt erstellen

Befehle

proc practcl::cat *fname*
: Eine Datei zusammenfügen

proc practcl::docstrip *text*
: Entfernt die globalen Kommentare aus dem Tcl-Code. Dies dient dazu, zu verhindern, dass die Kommentare zur Dokumentation Dateien überladen, die für die Verbreitung in einem maschinenlesbaren Format vorgesehen sind.

proc putb ?*map*? *text*
: Fügt einer Variable eine Textzeile hinzu. Wendet optional eine Zeichenfolgenzuordnung an.

proc Proc *name arglist body*
: Erstellt eine Prozedur, falls noch kein Befehl mit diesem Namen vorhanden ist

proc noop ?*args*?
: Ein Befehl, der nichts bewirkt. Eine praktische Möglichkeit, eine Anweisung zu negieren, ohne sie vollständig auskommentieren zu müssen. Außerdem ist er ein praktischer Anknüpfungspunkt für ein Objekt, das später benannt werden soll.

proc practcl::debug ?*args*?
: siehe unten ...

proc practcl::doexec ?*args*?
: Fügt eine statische Kopie von Tcl ein

proc practcl::doexec_in *path* ?*args*?
: siehe unten ...

proc practcl::dotclexec ?*args*?
: siehe unten ...

proc practcl::domake *path* ?*args*?
: siehe unten ...

proc practcl::domake.tcl *path* ?*args*?
: siehe unten ...

proc practcl::fossil *path* ?*args*?

siehe unten ...

proc practcl::fossil_status *dir*

siehe unten ...

proc practcl::os

siehe unten ...

proc practcl::mkzip *exename barekit vfspath*

Erstellt eine ZIP-Datei. Unter Tcl 8.6 wird dabei die native ZIP-Implementierung aufgerufen; bei älteren Interpretern wird ZIP über "exec" aufgerufen.

proc practcl::sort_dict *list*

Sortieren einer Schlüssel-Wert-Liste nach Dictionary-Reihenfolge. Dies ist erforderlich, da Tcl-Versionen vor 8.6 nicht über *lsort -stride 2*

proc practcl::local_os

Gibt ein Dictionary zurück, das das lokale Betriebssystem beschreibt. Zu den zurückgegebenen Feldern gehören:

- download – Dateisystempfad, in den Fossil-Repositorys und Quellcode-Tarballs für den aktuellen Benutzer heruntergeladen werden
- EXEEXT – Die Erweiterung, die ausführbaren Dateien zuweisen soll. (z. B. .exe unter Windows)
- fossil_mirror – Eine URI für einen Webserver im lokalen Netzwerk, der als Spiegel eines Fossil-Repositorys fungiert
- local_install – Pfad zum Dateisystem, in dem Pakete für die lokale Nutzung durch den aktuellen Benutzer installiert werden
- Präfix – Das Präfix, das dem Tcl-Kern/TEA bei der Installation in "local_install" in "./configure" angegeben wurde
- Sandbox – Der Speicherort, an dem dieses Projekt externe Projekte entpackt
- TEACUP_PROFILE – Der kanonische Name von ActiveState/Teacup für diese Plattform (d. h. win32-ix86 macosx10.5-i386-x86_84)
- TEACUP_OS – Das lokale Betriebssystem (Windows, macOS, OpenBSD usw.). Liefert dieselbe Antwort wie "tcl.m4", mit der Ausnahme, dass "macOS" anstelle von "Darwin" angegeben wird.
- TEA_PLATFORM – Die von "uname" zurückgegebene Plattform: -s-uname, -r (unter Unix) oder "windows" unter Windows
- TEACUP_ARCH – Die Prozessorarchitektur für das lokale Betriebssystem (d. h. ix86, x86_64)
- TEACUP_ARCH – Die Prozessorarchitektur für das lokale Betriebssystem (d. h. ix86, x86_64)
- teapot – Dateisystempfad, in den die Dateien des teapot-Pakets für den aktuellen Benutzer heruntergeladen werden
- userhome – Pfad zum Speichern lokalisierter Einstellungen, von Cache-Dateien aus Downloads usw. für den aktuellen Benutzer

Dieser Befehl nutzt eine Kombination aus lokalen Prüfungen mit "Exec", allen vorhandenen "tclConfig.sh"-Dateien, bereits berechneten autoconf-Daten sowie Daten, die aus einer Datei namens "practcl.rc" im Benutzerverzeichnis stammen. Der Speicherort des Benutzerverzeichnisses variiert je nach Plattform und Betriebssystem:

- Windows: ::env(LOCALAPPDATA)/Tcl
- MacOS: ~/Library/Application Support/Tcl
- Sonstiges: ~/tcl

proc practcl::config.tcl *path*
Ein transparenter Aufruf von ::practcl::read_configuration zur Wahrung der Abwärtskompatibilität mit älteren Versionen von Practcl

proc practcl::read_configuration *path*
Lokale Plattform ermitteln. Dieser Befehl sucht in dem angegebenen Pfad nach Daten, die von "autoconf" oder "autosetup" erfasst wurden, oder führt eigene Logiktests durch, falls keines der beiden Programme ausgeführt wurde. Das Vorhandensein einer Datei namens "config.site" an diesem Speicherort weist darauf hin, dass es sich bei diesem Projekt um eine Cross-Kompilierung handelt, und die in dieser Datei gespeicherten Daten werden für den Compiler und den Linker verwendet.

Dieser Befehl sucht in der folgenden Reihenfolge nach Informationen aus den folgenden Dateien:
- config.tcl – Eine Datei, die von autoconf/configure in neueren Versionen von TEA generiert wird und durch Kodierung als Tcl-Skript dargestellt ist.
- config.site – Eine Datei, die Informationen zum Cross-Compiler enthält und durch Kodierung als SH-Skript dargestellt wird
- ::env(VisualStudioVersion) – Unter Windows ein Wert der Umgebungsvariablen, der angibt, dass MS Visual Studio installiert ist

Dieser Befehl gibt ein Dictionary zurück, das alle aus den oben genannten Quellen bereinigten Daten enthält. Sofern keine weiteren Angaben gemacht werden, liefert dieser Befehl dieselbe Ausgabe wie ::practcl::local_os. In diesem Modus stellt dieser Befehl, wenn die Umgebungsvariable "VisualStudioVersion" vorhanden ist, eine Vorlage mit Feldern bereit, die für die Kompilierung unter Windows mit Microsoft Visual Studio geeignet sind. Das USEMSVC-Flag im Dictionary ist ein boolesches Flag, das angibt, ob dies tatsächlich der Fall ist.

proc practcl::tcllib_require *pkg* ?*args*?
Versucht, ein Paket zu laden, und ruft bei einem Fehlschlag "tcllib" ab.

proc practcl::platform::tcl_core_options *os*
Gibt die Zeichenkette zurück, die an ./configure übergeben werden soll, um den Tcl-Kern für das angegebene Betriebssystem zu kompilieren.
- Windows: --with-tzdata --with-encoding utf-8
- macosx: --enable-corefoundation=yes --enable-framework=no --with-tzdata --with-encoding utf-8
- Sonstiges: --with-tzdata --with-encoding utf-8

proc practcl::platform::tk_core_options *os*
siehe unten ...

proc practcl::read_rc_file *filename* ?*localdat* ?
Liest eine in einer Datei gespeicherte, stilisierte Schlüssel-Wert-Liste ein

proc practcl::read_sh_subst *line info*
Wandelt eine XXX.sh-Datei in eine Reihe von Tcl-Variablen um

proc practcl::read_sh_file *filename* ?*localdat* ?
siehe unten ...

proc practcl::read_Config.sh *filename*
Eine vereinfachte Version von "read_sh_file", die speziell auf das Abrufen von Daten aus der

Datei "(tcl|tk)Config.sh" zugeschnitten ist

proc practcl::read_Makefile *filename*
Eine vereinfachte Version von "read_sh_file", die speziell auf das Abrufen von Daten aus einer Makefile-Datei zugeschnitten ist

proc practcl::cputs *varname* ?*args*?
Argumente an einen Puffer anhängen. Der Befehl funktioniert ähnlich wie "puts", da bei jedem Aufruf zusätzlich ein Zeilenvorschub eingefügt wird. Im Gegensatz zu "puts" werden jedoch Leerzeichen im Zwischentext unterdrückt.

proc practcl::tcl_to_c *body*
siehe unten ...

proc practcl::_tagblock *text* ?*style* tcl? ?*note* ?
siehe unten ...

proc practcl::de_shell *data*
siehe unten ...

proc practcl::grep *pattern* ?*files* ?
Sucht nach dem Muster *pattern* in $files

proc practcl::file_lexnormalize *sp*
siehe unten ...

proc practcl::file_relative *base dst*
Berechnet einen relativen Pfad zwischen "base" und "dst".

Beispiele:

```
::practcl::file_relative ~/build/tcl/unix ~/build/tcl/library
> ../library
```

proc practcl::findByPattern *basedir patterns*
siehe unten ...

proc practcl::log *fname comment*
Ein Ereignis im Praxisprotokoll eintragen

proc practcl::_pkgindex_simpleIndex *path*
siehe unten ...

proc practcl::_pkgindex_directory *path*
Gibt "true" zurück, wenn die pkgindex-Datei eine andere Anweisung als "package ifneeded" enthält und/oder wenn ein "package ifneeded" eine DLL lädt

proc practcl::_pkgindex_path_subdir *path*
Hilfsfunktion für ::practcl::pkgindex_path

proc practcl::pkgindex_path ?*args*?
Alle angegebenen Pfade so indizieren, als würden sie im selben virtuellen Dateisystem enden

proc practcl::installDir *d1 d2*
Löscht den Inhalt von *d2* und kopiert anschließend den Inhalt von *d1* rekursiv in *d2*.

proc practcl::copyDir *d1 d2* ?*toplevel* 1?
Kopiert den Inhalt von *d1* rekursiv nach *d2*

proc practcl::buildModule *modpath*
siehe unten ...

proc practcl::installModule *modpath DEST*

Installiert ein Modul aus MODPATH in das angegebene Verzeichnis. Es wird davon ausgegangen, dass *dpath* der vollständig qualifizierte Pfad ist, in dem das Modul platziert werden soll. Alle bereits vorhandenen Dateien unter diesem Pfad werden gelöscht. Wenn es sich bei dem Pfad um einen symbolischen Link handelt, gibt der Prozess ohne Fehlermeldung und ohne weitere Aktion zurück. Enthält das Modul im Verzeichnis build/ Inhalte, die neuer sind als die .tcl-Dateien im Quellverzeichnis des Moduls, und existiert eine Datei build/build.tcl, wird diese Datei ausgeführt. Wenn das Quellverzeichnis eine Datei mit dem Namen "index.tcl" enthält, wird davon ausgegangen, dass es sich um ein Modul im TAO-Stil handelt, und das gesamte Verzeichnis (sowie alle Unterverzeichnisse) wird unverändert kopiert. Wenn keine Datei "index.tcl" vorhanden ist, werden alle .tcl-Dateien aus dem Modul-Quellverzeichnis kopiert, und es wird eine Datei "pkgIndex.tcl" generiert, sofern diese noch nicht existiert. Existiert im Quellverzeichnis ein Verzeichnis namens "htdocs", wird dieses Verzeichnis unverändert in das Zielverzeichnis kopiert.

proc practcl::trigger ?*args*?

Build-Ziele auslösen und Abhängigkeiten neu berechnen

Interne Abläufe:

```
::practcl::LOCAL make trigger {*}$args
foreach {name obj} [::practcl::LOCAL make objects] {
  set ::make($name) [$obj do]
}
```

proc practcl::depends ?*args*?

Berechnet, ob eine Abhängigkeit für eines der Argumente erfüllt oder neu erstellt werden muss.

Interne Abläufe:

```
::practcl::LOCAL make depends {*}$args
```

proc practcl::target *name info* ?*action* ?

Ein Build-Produkt deklarieren. Diese Prozedur ist lediglich eine Kurzform für *::practcl::LOCAL make task $name $info $action*

Durch die Registrierung eines Build-Produkts mit diesem Befehl wird ein Eintrag im globalen Array erstellt und ein Wert in das globale Array eingetragen.

Interne Details:

```
set obj [::practcl::LOCAL make task $name $info $action]
set ::make($name) 0
set filename [$obj define get filename]
if {$filename ne {}} {
  set ::target($name) $filename
}
```

Klassen

Klasse practcl::doctool

```
{ set authors {
   {John Doe} {jdoe@illustrious.edu}
   {Tom RichardHarry} {tomdickharry@illustrius.edu}
 }
 # Create the object
 ::practcl::doctool create AutoDoc
 set fout [open [file join $moddir module.tcl] w]
 foreach file [glob [file join $srcdir *.tcl]] {
   set content [::practcl::cat [file join $srcdir $file]]
    # Scan the file
    AutoDoc scan_text $content
    # Strip the comments from the distribution
    puts $fout [::practcl::docstrip $content]
 }
 # Write out the manual page
 set manout [open [file join $moddir module.man] w]
 dict set args header [string map $modmap [::practcl::cat [file join
    $srcdir manual.txt]]]
 dict set args footer [string map $modmap [::practcl::cat [file join
    $srcdir footer.txt]]]
 dict set args authors $authors
 puts $manout [AutoDoc manpage {*}$args]
 close $manout


}
```

Tool für Build-Skripte zur dynamischen Generierung von Dokumentationsdateien aus Kommentaren in Quellcode-Dateien

Methoden

method constructor

siehe unten ...

method argspec *argspec*

Verarbeitet eine Argumentliste zu einem Informationswörterbuch. Diese Methode versteht auch nicht-positionale Argumente, die in der Notation von Tipp 471 https://core.tcl-lang.org/tips/doc/trunk/tip/479.md ausgedrückt werden.

Die Ausgabe ist ein Dictionary, das alle Felder enthält und angibt, ob es sich bei den Feldern um **positional**, **mandatory**, and whether they have a **default** aufweisen.

Beispiele:

```
   my argspec {a b {c 10}}

   > a {positional 1 mandatory 1} b {positional 1 mandatory 1} c
    {positional 1 mandatory 0 default 10}
```

method comment *block*

Wandelt einen Kommentarblock in ein InformationsDictionary um. Wenn Zeilen im Kommentar mit einem einzelnen Wort beginnen, das mit einem Doppelpunkt endet, werden alle nachfolgenden Zeilen an ein Dictionary-Feld mit diesem Namen angehängt. Wenn keine Felder angegeben sind, wird der gesamte Text an das Feld **description** angehängt.

Beispiel:

```
my comment {Does something cool}
> description {Does something cool}

my comment {
title : Something really cool
author : Sean Woods
author : John Doe
description :
This does something really cool!
}
> description {This does something really cool!}
  title {Something really cool}
  author {Sean Woods
  John Doe}
```

method keyword.Annotation *resultvar commentblock type name body*

siehe unten ...

method keyword.Class *resultvar commentblock name body*

Verarbeitet einen oo::objdefine-Aufruf, der das Objekt der Klasse selbst ändert

method keyword.class *resultvar commentblock name body*

Verarbeitet die Anweisungen oo::define, clay::define usw.

method keyword.Class_Method *resultvar commentblock name ?args?*

Eine Anweisung für eine Methode der Klasse im Clay-Stil verarbeiten

method keyword.method *resultvar commentblock name ?args?*

Den Prozess für eine Anweisung für eine Objektmethode im TCLOO-Stil durchführen

method keyword.proc *commentblock name argspec*

Eine PROC-Anweisung ausführen

method reset

Den Zustand des Objekts und seiner eingebetteten Coroutine zurücksetzen

method Main

Hauptteil der eingebetteten Coroutine für das Objekt

method section.method *keyword method minfo*

Den Text der Handbuchseite für eine Methode oder Prozedur generieren

method section.annotation *type name iinfo*

siehe unten ...

method section.class *class_name class_info*

Den Text der Handbuchseite für eine Klasse generieren

method section.command *procinfo*

Erstellt den Text für die Handbuchseite im Abschnitt "Befehle"

method manpage ?header *value*? ?footer *value*? ?authors *list*?

Erstellt die Handbuchseite. Gibt den fertigen Text zurück, der zum Speichern in einer .man-Datei geeignet ist. Das Argument "header" ist ein Block aus Doctools-Text, der vor dem maschinell generierten Abschnitt eingefügt werden soll. "footer" ist ein Block aus Doctools-Text, der nach dem maschinell generierten Abschnitt eingefügt werden soll. "authors" ist eine Liste einzelner Autoren und E-Mail-Adressen im Format AUTOR E-MAIL ?AUTOR E-MAIL?...

method scan_text *text*

Einen Textblock scannen

method scan_file *filename*

Eine Textdatei einscannen

Klasse practcl::metaclass

Die Metaklasse für alle "practcl"-Objekte

Methoden

- Methode **_MorphPatterns**
- Methode **define** *submethod* ?*args*?
- Methode **graft** ?*args*?
- Methode **initialize**
- Methode **link** *command* ?*args*?
- Methode **morph** *classname*
- Methode **script** *script*
- Methode **select**
- Methode **source** *filename*

Klasse practcl::toolset

Eine Klasse ohne übergeordnete Klasse, die als Mixin konzipiert ist und eine Reihe von Verhaltensweisen im Zusammenhang mit dem Build definiert, die angepasst werden, je nachdem, ob gcc oder msvc als Zielkompilierer ausgewählt wird.

Klassenmethoden

classmethod select *object*

Führt die Auswahl für das Toolset-Mixin

Methods durch

method config.sh

Sucht eine Schlüssel-Wert-Liste, die dieses Projekt beschreibt, oder erstellt eine solche.

method BuildDir *PWD*

Ermittelt den Standort, an dem das Produkt hergestellt werden soll

method MakeDir *srcdir*

Gibt den Pfad zurück, an dem sich das Makefile relativ zu *srcdir* befindet. Bei dieser Implementierung ist MakeDir immer srcdir.

method read_configuration

Liest die Informationen zum Erstellungs-Prozess für dieses Paket. Bei dieser Implementierung werden Daten an den folgenden Speicherorten in der folgenden Reihenfolge gesucht:

config.tcl (von practcl generiert), PKGConfig.sh und das Makefile.

Wenn das Makefile herangezogen werden muss, aber nicht vorhanden ist, wird die Methode "Configure" aufgerufen.

method build-cflags *PROJECT DEFS namevar versionvar defsvar*
Methode DEFS Diese Methode füllt vier Variablen: name – Der Name des Pakets version – Die Version des Pakets defs – An den Compiler übergebene C-Flags includedir – Eine Liste von Pfaden, die dem Compiler zum Auffinden von Header-Dateien übergeben werden sollen

method critcl ?*args*?
Aufruf von "critcl" in einem externen Prozess

Klasse practcl::toolset.gcc

ancestors: **practcl::toolset**

Methoden

method Autoconf
siehe unten ...

method BuildDir *PWD*
siehe unten ...

method ConfigureOpts
siehe unten ...

method MakeDir *srcdir*
Ermittelt, in welchem Verzeichnis sich die Makefile-Vorlage befindet

method make {} autodetect
siehe unten ...

method make {} clean
siehe unten ...

method make {} compile
siehe unten ...

method make {} install *DEST*
siehe unten ...

method build-compile-sources *PROJECT COMPILE CPPCOMPILE INCLUDES*
siehe unten ...

method build-Makefile *path PROJECT*
siehe unten ...

method build-library *outfile PROJECT*
Erstellt eine statische oder dynamische Bibliothek

method build-tclsh *outfile PROJECT* ?*path* auto?
Erstellt eine statische ausführbare Datei

Klasse practcl::toolset.msvc

ancestors: **practcl::toolset**

Methoden

method BuildDir *PWD*
MSVC führt die Kompilierung stets im Quellverzeichnis durch

method make {} autodetect
Nichts tun

method make {} clean
siehe unten ...

method make {} compile
siehe unten ...

method make {} install *DEST*
siehe unten ...

method MakeDir *srcdir*
Ermittelt, in welchem Verzeichnis sich die Makefile-Vorlage befindet

- Methode **NmakeOpts**

Klasse practcl::make_obj

ancestors: **practcl::metaclass**

Ein Objekt, das im Rahmen des Build-Prozesses bereitgestellt wird. In der Regel handelt es sich dabei um eine Objektdatei, eine Header-Datei oder ein TCL-Skript, das auf irgendeine Weise kompiliert oder generiert werden muss.

Methoden

- Methode **constructor** *module_object name info* ?*action_body* ?
- Methode **do**
- Methode **check**
- Methode **output**
- Methode **reset**
- Methode **triggers**

Klasse practcl::object

ancestors: **practcl::metaclass**

Ein generisches "Practcl"-Objekt

Methoden

- Methode **constructor** *parent* ?*args*?
- Methode **child** *method*
- Methode **go**

Klasse practcl::dynamic

Dynamische Blöcke erzeugen keine eigenen .c-Dateien, sondern tragen zur Zusammenführung der Datei der Hauptbibliothek bei.

Methoden

method cstructure *name definition* ?*argdat* ?
Parser-Funktionen

method include *header*
siehe unten ...

method include_dir ?*args*?
siehe unten ...

method include_directory ?*args*?

siehe unten ...

method c_header *body*
siehe unten ...

method c_code *body*
siehe unten ...

method c_function *header body ?info ?*
siehe unten ...

method c_tcloomethod *name body ?arginfo ?*
siehe unten ...

method cmethod *name body ?arginfo ?*
Alias für den klassischen Namen

method c_tclproc_nspace *nspace*
siehe unten ...

method c_tclcmd *name body ?arginfo ?*
siehe unten ...

method c_tclproc_raw *name body ?arginfo ?*
Alias für den klassischen Namen

method tcltype *name argdat*
siehe unten ...

method project-compile-products
Wechselwirkungen zwischen Modulen

method implement *path*
siehe unten ...

method initialize
Praktische Grundlagen

method linktype
siehe unten ...

method generate-cfile-constant
siehe unten ...

method generate-cfile-header
siehe unten ...

method generate-cfile-tclapi
Erstellt Code, der Tcl-API-Aufrufe implementiert

method generate-loader-module
Erstellt Code, der ausgeführt wird, wenn das Paket/Modul im Interpreter initialisiert wird

method Collate_Source *CWD*
siehe unten ...

method select
Sobald sich ein Objekt als eine Form von "dynamisch" kennzeichnet, sollten Sie nicht länger versuchen, es in etwas anderes umzuwandeln.

Klasse practcl::product

Ein Ergebnis für das Build-System

Klassenmethoden

- classmethod **select** *object*

Methoden

method code *section body*
siehe unten ...

method Collate_Source *CWD*
siehe unten ...

method project-compile-products
siehe unten ...

method generate-debug ?*spaces* ?
siehe unten ...

method generate-cfile-constant
siehe unten ...

method generate-cfile-public-structure
Konstante statische Datenstrukturen mit Werten füllen

method generate-cfile-header
siehe unten ...

method generate-cfile-global
siehe unten ...

method generate-cfile-private-typedef
siehe unten ...

method generate-cfile-private-structure
siehe unten ...

method generate-cfile-functions
Erstellt Code, der Unterprogramme bereitstellt, die von Tcl-API-Methoden aufgerufen werden

method generate-cfile-tclapi
Erstellt Code, der Tcl-API-Aufrufe implementiert

method generate-hfile-public-define
siehe unten ...

method generate-hfile-public-macro
siehe unten ...

method generate-hfile-public-typedef
siehe unten ...

method generate-hfile-public-structure
siehe unten ...

method generate-hfile-public-headers
siehe unten ...

method generate-hfile-public-function
siehe unten ...

method generate-hfile-public-includes
siehe unten ...

method generate-hfile-public-verbatim
siehe unten ...

method generate-loader-external

siehe unten ...

method generate-loader-module
siehe unten ...

method generate-stub-function
siehe unten ...

method IncludeAdd *headervar* ?*args*?
siehe unten ...

method generate-tcl-loader
siehe unten ...

method generate-tcl-pre
Diese Methode generiert eine beliebige Tcl-Skriptdatei, die zur Vorinitialisierung der C-Bibliothek erforderlich ist.

method generate-tcl-post
siehe unten ...

method linktype
siehe unten ...

method Ofile *filename*
siehe unten ...

method project-static-packages
Vom Hauptprojekt aufgerufene Methoden

method toolset-include-directory
Vom Toolset aufgerufene Methoden

- Methode **target** *method* ?*args*?

Klasse practcl::product.cheader

ancestors: **practcl::product**

Ein Produkt, das aus einer C-Header-Datei generiert wurde. Das heißt: gar nichts.

Methoden

- Methode **project-compile-products**
- Methode **generate-loader-module**

Klasse practcl::product.csource

ancestors: **practcl::product**

Ein Produkt, das aus einer C-Quelldatei generiert wurde. In der Regel handelt es sich dabei um eine Objektdatei (.o).

Methoden

- Methode **project-compile-products**

Klasse practcl::product.clibrary

ancestors: **practcl::product**

Ein Produkt, das aus einer kompilierten C-Bibliothek generiert wird. In der Regel handelt es sich dabei um eine .a- oder eine .dylib-Datei; in komplexen Fällen kann es sich jedoch lediglich um ein Mittel handeln, mit dem ein Projekt den Quellcode eines anderen Projekts integriert.

Methoden

- Methode **linker-products** *configdict*

Klasse practcl::product.dynamic

ancestors: **practcl::dynamic practcl::product**

Ein Produkt, das aus C-Code generiert wird, der wiederum von "practcl" oder auf andere Weise erzeugt wurde. Diese C-Datei erzeugt möglicherweise eine eigene .o-Datei, möglicherweise aber auch nicht – je nachdem, ob sie für die Einbindung in eine Zusammenführung in Frage kommt.

Methoden

- Methode **initialize**

Klasse practcl::product.critcl

ancestors: **practcl::dynamic practcl::product**

Ein von critcl erstelltes Binärprodukt. Hinweis: Die Implementierung wurde noch nicht geschrieben; diese Klasse führt keine Funktionen aus.

Klasse practcl::module

ancestors: **practcl::object practcl::product.dynamic**

Letztendlich muss der gesamte C-Code in ein Modul geladen werden. Dabei handelt es sich entweder um eine dynamisch geladene Bibliothek, die eine Tcl-Erweiterung implementiert, oder um einen fest kompilierten Abschnitt einer benutzerdefinierten Shell bzw. Anwendung.

Variable

- Variable **make_object**

Methoden

method _MorphPatterns
siehe unten ...

method add ?*args*?
siehe unten ...

method install-headers ?*args*?
siehe unten ...

method make {} _preamble
siehe unten ...

method make {} pkginfo
siehe unten ...

method make {} objects
Gibt ein Dictionary mit allen Handles und den zugehörigen Objekten zurück

method make {} object *name*
Gibt das Objekt zurück, das mit dem Handle *name* verknüpft ist.

method make {} reset
Alle untergeordneten Objekte zurücksetzen

method make {} trigger ?*args*?
Wendet die "triggers"-Methode auf alle aufgeführten Handles an

method make {} depends ?*args*?

Führt die Prüfmethode für alle aufgeführten Handles durch

method make {} filename *name*
Gibt den Namen des Build-Produkts für das angegebene Handle zurück

method make {} target *name Info body*
siehe unten ...

method make {} todo
Gibt eine Liste von Handles für Objekte zurück, die bei der Methode "do" den Wert "true" zurückgeben

method make {} do
Für jede Zielübung wird die in der *action*-Definition angegebene Aktion ausgeführt, wenn die *do*-Methode den Wert "true" gibt.

method child *which*
siehe unten ...

method generate-c
Diese Methode generiert den Inhalt einer zusammengeführten .c-Datei, die den Loader für eine Reihe von Tools implementiert

method generate-h
Diese Methode generiert den Inhalt einer zusammengeführten .h-Datei, die die öffentliche API dieses Moduls beschreibt

- Methode **generate-loader**
- Methode **initialize**
- Methode **implement** *path*
- Methode **linktype**

Klasse practcl::project

ancestors: **practcl::module**

Ein Toplevel-Projekt, das eine Sammlung anderer Projekte darstellt

Methoden

method _MorphPatterns
siehe unten ...

method constructor ?*args*?
siehe unten ...

method add_object *object*
siehe unten ...

method add_project *pkg info* ?*oodefine* ?
siehe unten ...

method add_tool *pkg info* ?*oodefine* ?
siehe unten ...

method build-tclcore
Kompiliert den Tcl-Kern. Wenn die Definition *tk* wahr ist, wird auch der Tk-Kern kompiliert.

method child *which*
siehe unten ...

method linktype

siehe unten ...

method project *pkg* ?*args*?
Die Methoden eines Unterobjekts aufrufen

- Methode **tclcore**
- Methode **tkcore**
- Methode **tool** *pkg* ?*args*?

Klasse practcl::library

ancestors: **practcl::project**

Ein Toplevel-Projekt, das eine Bibliothek erstellt

Methoden

method clean *PATH*
siehe unten ...

method project-compile-products
siehe unten ...

method go
siehe unten ...

method generate-decls *pkgname path*
siehe unten ...

method implement *path*
siehe unten ...

method generate-make *path*
Abwärtskompatibler Aufruf

method linktype
siehe unten ...

method package-ifneeded ?*args*?
Erstellt ein "Paket ifneeded". Die Argumente sind eine Liste von Aliasen, auf die dieses Paket reagieren soll.

- Methode **shared_library** ?*filename* ?
- Methode **static_library** ?*filename* ?

Klasse practcl::tclkit

ancestors: **practcl::library**

Ein Toplevel-Projekt, das eine eigenständige ausführbare Datei erzeugt

Methoden

method build-tclkit_main *PROJECT PKG_OBJS*
siehe unten ...

method Collate_Source *CWD*
siehe unten ...

method wrap *PWD exename vfspath* ?*args*?
Eine ausführbare Datei verpacken

Klasse practcl::distribution

Eigenständige Klasse zur Verwaltung der Code-Verteilung. Diese Klasse ist dafür vorgesehen, in eine andere Klasse eingebunden zu werden (daher das Fehlen von Oberklassen).

Klassenmethoden

- classmethod **Sandbox** *object*
- classmethod **select** *object*
- Klassenmethode **claim_option**
- classmethod **claim_object** *object*
- classmethod **claim_path** *path*

Methoden

- Methode **scm_info**
- Methode **DistroMixIn**
- Methode **Sandbox**
- Methode **SrcDir**
- Methode **ScmTag**
- Methode **ScmClone**
- Methode **ScmUnpack**
- Methode **ScmUpdate**
- Methode **Unpack**

Klasse practcl::distribution.snapshot

ancestors: **practcl::distribution**

Eine Dateiverteilung aus einem ZIP-Archiv, einem Tarball oder einem anderen, nicht auf SCM basierenden Archivformat

Klassenmethoden

- classmethod **claim_object** *object*
- Klassenmethode **claim_option**
- classmethod **claim_path** *path*

Methoden

- Methode **ScmUnpack**

Klasse practcl::distribution.fossil

ancestors: **practcl::distribution**

Eine auf Fossil basierende Dateiverteilung

Klassenmethoden

classmethod claim_object *obj*
Überprüft die Metadaten auf Markierungen

classmethod claim_option
siehe unten ...

classmethod claim_path *path*
Überprüft, ob im Quellstamm Markierungen vorhanden sind

Methoden

method scm_info
siehe unten ...

method ScmClone
Den Quellcode klonen

- Methode **ScmTag**
- Methode **ScmUnpack**
- Methode **ScmUpdate**

Klasse practcl::distribution.git

ancestors: **practcl::distribution**

Eine auf Git basierende Dateiverteilung

Klassenmethoden

- classmethod **claim_object** *obj*
- Klassenmethode **claim_option**
- classmethod **claim_path** *path*

Methoden

- Methode **ScmTag**
- Methode **ScmUnpack**
- Methode **ScmUpdate**

Klasse practcl::subproject

ancestors: **practcl::module**

Ein Teilprojekt

Methoden

method _MorphPatterns
siehe unten ...
method BuildDir *PWD*
siehe unten ...
method child *which*
siehe unten ...
method compile
siehe unten ...
method go
siehe unten ...
method install ?*args*?
Projekt in das lokale Build-System integrieren
method linktype
siehe unten ...
method linker-products *configdict*
siehe unten ...
method linker-external *configdict*
siehe unten ...
method linker-extra *configdict*
siehe unten ...
method env-bootstrap

Methoden für Pakete/Tools, die heruntergeladen, gegebenenfalls kompiliert und intern von diesem Practcl-Prozess verwendet werden können. Laden Sie die Funktion in den Interpreter.

method env-exec
Gibt einen Dateipfad zurück, den "exec" aufrufen kann

method env-install
Installiert das Tool in der lokalen Umgebung

method env-load
Tun Sie alles Notwendige, um das Tool in die lokale Umgebung zu integrieren

method env-present
Prüft, ob das Werkzeug zum Laden verfügbar ist bzw. bereits geladen wurde

- Methode **sources**
- Methode **update**
- Methode **unpack**

Klasse practcl::subproject.source
ancestors: **practcl::subproject practcl::library**

Ein Projekt, bei dem das Kit den Quellcode selbst kompiliert und integriert

Methoden

- Methode **env-bootstrap**
- Methode **env-present**
- Methode **linktype**

Klasse practcl::subproject.teapot
ancestors: **practcl::subproject**

ein Schluck aus der Teekanne

Methoden

- Methode **env-bootstrap**
- Methode **env-install**
- Methode **env-present**
- Methode **install** *DEST*

Klasse practcl::subproject.kettle
ancestors: **practcl::subproject**

Methoden

- Methode **kettle** *path* ?*args*?
- Methode **install** *DEST*

Klasse practcl::subproject.critcl
ancestors: **practcl::subproject**

Methoden

- Methode **install** *DEST*

Klasse practcl::subproject.sak
ancestors: **practcl::subproject**

Methoden

- Methode **env-bootstrap**
- Methode **env-install**
- Methode **env-present**
- Methode **install** *DEST*
- Methode **install-module** *DEST* ?*args*?

Klasse practcl::subproject.practcl

ancestors: **practcl::subproject**

Methoden

- Methode **env-bootstrap**
- Methode **env-install**
- Methode **install** *DEST*
- Methode **install-module** *DEST* ?*args*?

Klasse practcl::subproject.binary

ancestors: **practcl::subproject**

Ein untergeordnetes Paket

Methoden

- Methode **clean**
- Methode **env-install**
- Methode **project-compile-products**
- Methode **ComputeInstall**
- Methode **go**
- Methode **linker-products** *configdict*
- Methode **project-static-packages**
- Methode **BuildDir** *PWD*
- Methode **compile**
- Methode **Configure**
- Methode **install** *DEST*

Klasse practcl::subproject.tea

ancestors: **practcl::subproject.binary**

Ein untergeordnetes TEA-basiertes Paket

Klasse practcl::subproject.library

ancestors: **practcl::subproject.binary practcl::library**

Eine von diesem Projekt erstellte untergeordnete C-Bibliothek

Methoden

- Methode **install** *DEST*

Klasse practcl::subproject.external

ancestors: **practcl::subproject.binary**

Eine untergeordnete externe C-Bibliothek

Methoden

- Methode **install** *DEST*

Klasse practcl::subproject.core

ancestors: **practcl::subproject.binary**

Methoden

- Methode **env-bootstrap**
- Methode **env-present**
- Methode **env-install**
- Methode **go**
- Methode **linktype**

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *practcl* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

TclOO

Copyright


```
Copyright (c) 2016-2018 Sean Woods <yoda@etoyoc.com>
```


processman

processman

Stichwörter: processman, odie

Name

processman – Tool zur Automatisierung der periodischen Rückruffunktion für Befehle

Übersicht

package require Tcl 8.5 9
package require twapi 3.1
package require cron 1.1
package require processman ?**0.8**?
::processman::find_exe *name*
::processman::kill *id*
::processman::kill_all
::processman::killexe *name*
::processman::onexit *id cmd*

::processman::priority *id level*
::processman::process_list
::processman::process_list *id*
::processman::spawn *id cmd args*

Beschreibung

Das Paket **processman** enthält eine Reihe von Pure-Tcl-Dienstprogrammen zur plattformunabhängigen Verwaltung von untergeordneten Prozessen.

Befehle

::processman::find_exe *name*
Sucht im Systempfad nach einer ausführbaren Datei mit dem Namen *name*. Fügen Sie unter Windows zusätzlich die Dateiendung .exe hinzu, wenn diese nicht angegeben ist.

::processman::kill *id*
Beendet den untergeordneten Prozess mit der ID *id*.

::processman::kill_all
Beendet alle von diesem Programm gestarteten Prozesse

::processman::killexe *name*
Beendet einen Prozess, der durch die ausführbare Datei identifiziert wird. Unter Unix löst dies einen "killall"-Befehl aus. Unter Windows wird "**twapi::get_process_ids**" verwendet, um einen Namen einer oder mehreren IDs zuzuordnen, die anschließend beendet werden.

::processman::onexit *id cmd*
Sorgt dafür, dass das Skript *cmd* ausgeführt wird, wenn dieses Programm feststellt, dass der Prozess *id* beendet wurde.

::processman::priority *id level*
Kennzeichnet den Prozess *id* mit der Prioritätsstufe *level*. Gültige Stufen: low, high, default.

Unter Unix wird der Prozess mit dem Befehl **nice** gekennzeichnet.

Unter Windows wird der Prozess über den Befehl **twapi::set_priority_class** geändert.

::processman::process_list
Gibt eine Liste der Prozesse zurück, die von diesem Programm ausgelöst wurden, sowie ein boolesches Flag, das angibt, ob der Prozess noch läuft.

::processman::process_list *id*
Gibt "true" zurück, wenn der Prozess *id* noch läuft, andernfalls "false".

::processman::spawn *id cmd args*
Startet einen Kindprozess, der durch *id* gekennzeichnet ist. *cmd* ist der Name des auszuführenden Befehls. *args* sind die Argumente, die an diesen Befehl übergeben werden sollen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *odie* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
System

Copyright

```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```


profiler

profiler
Stichwörter: profiler, performance, profile, speed

Name
profiler – Tcl-Quellcode-Profiler

Übersicht
package require Tcl 8.5 9
package require profiler ?**0.7**?
::profiler::init
::profiler::dump *pattern*
::profiler::print ?*pattern*?
::profiler::reset ?*pattern*?
::profiler::suspend ?*pattern*?
::profiler::resume ?*pattern*?
::profiler::new-disabled
::profiler::new-enabled
::profiler::sortFunctions *key*

Beschreibung
Das Paket **profiler** stellt einen einfachen Tcl-Quellcode-Profiler bereit. Es handelt sich um einen Profiler auf Funktionsebene; das heißt, er erfasst lediglich Informationen auf Funktionsebene, nicht jedoch die detaillierteren Informationen auf Zeilenebene. Er funktioniert durch die Neudefinition des Tcl-Befehls **proc**. Die Profilerstellung wird über den Befehl **::profiler::init** initiiert.

Befehle
::profiler::init
: Profilierung starten. Alle Prozeduren, die nach dem Aufruf dieses Befehls erstellt werden, werden profiliert. Um eine gesamte Anwendung zu profilieren, muss dieser Befehl vor allen anderen Befehlen aufgerufen werden.

::profiler::dump *pattern*
: Gibt Profilierungsinformationen für alle Funktionen aus, die dem Muster *pattern* entsprechen. Wenn kein Muster festgelegt wird, werden Informationen zu allen Funktionen zurückgegeben.

Das Ergebnis ist eine Liste von Schlüssel-Wert-Paaren, die Funktionsnamen Informationen zu der jeweiligen Funktion zuordnet. Die Informationen zu jeder Funktion bestehen wiederum aus einer Liste von Schlüssel-Wert-Paaren. Die verwendeten Schlüssel und ihre Werte lauten wie folgt:

totalCalls
Die Gesamtzahl der Aufrufe von *functionName*.

callerDist
Eine Liste von Schlüssel-Wert-Paaren, in der jede aufrufende Funktion, die *functionName* aufgerufen hat, der Anzahl ihrer Aufrufe von *functionName* zugeordnet wird.

compileTime
Die Laufzeit in Taktklicks der Funktion *functionName* bei ihrem ersten Aufruf.

totalRuntime
Die Summe der Laufzeiten aller Aufrufe von *functionName*.

averageRuntime
Durchschnittliche Laufzeit von *functionName*.

descendantTime
Summe der Zeit, die in den Nachfolgern von *functionName* verbracht wurde.

averageDescendantTime
Durchschnittliche Verweildauer in den untergeordneten Elementen von *functionName*.

::profiler::print ?*pattern*?
Gibt die Profiling-Informationen für alle Funktionen aus, die dem Muster *pattern* entsprechen. Wenn kein Muster angegeben wird, werden Informationen zu allen Funktionen angezeigt. Das Ergebnis ist eine für den Menschen lesbare Darstellung der Profiling-Informationen.

::profiler::reset ?*pattern*?
Die Profiling-Informationen für alle Funktionen, die dem Muster *pattern* entsprechen, werden zurückgesetzt. Wird kein Muster festgelegt, werden die Informationen für alle Funktionen zurückgesetzt.

::profiler::suspend ?*pattern*?
Das Profiling für alle Funktionen, die dem Muster *pattern* entsprechen, wird ausgesetzt. Wenn kein Muster angegeben wird, wird das Profiling für alle Funktionen ausgesetzt. Nach Ausführung dieses Befehls wird die Erfassung von Profiling-Informationen beendet. Bereits zuvor erfasste Profiling-Informationen werden jedoch nicht gelöscht. Verwenden Sie den Befehl "resume", um das Profiling wieder zu aktivieren.

::profiler::resume ?*pattern*?
Fährt die Profilerstellung für alle Funktionen fort, die dem Muster *pattern* entsprechen. Wenn kein Muster angegeben wird, wird die Profilerstellung für alle Funktionen fortgesetzt. Dieser Befehl sollte aufgerufen werden, nachdem der Profiler im Code angehalten wurde.

::profiler::new-disabled
Ändert den anfänglichen Profiling-Status für neue Prozeduren. Durch den Aufruf dieses Befehls wird das Profiling für alle Prozeduren deaktiviert, die nach diesem Befehl erstellt werden, bis der Befehl **new-enabled** aufgerufen wird. Aktivieren Sie das Profiling bestimmter Prozeduren über den Befehl **resume**.

::profiler::new-enabled
Ändert den anfänglichen Profiling-Status für neue Prozeduren. Durch den Aufruf dieses

Befehls wird das Profiling für alle Prozeduren aktiviert, die nach diesem Befehl erstellt werden, bis der Befehl **new-disabled** aufgerufen wird. Verhindern Sie das Profiling bestimmter Prozeduren mithilfe von **suspend**.

::profiler::sortFunctions ***key***
Gibt eine Liste von Funktionen zurück, sortiert nach einer bestimmten Profiling-Statistik. Unterstützte Werte für *key* sind: **calls**, **exclusiveTime**, **compileTime**, **nonCompileTime**, **totalRuntime**, **avgExclusiveTime** und **avgRuntime**. Das Rückgaberesultat ist eine Liste von Listen, wobei jede Unterliste aus einem Funktionsnamen und dem Wert von *key* für diese Funktion besteht.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *profiler* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

pt

pt_astree

Stichwörter: pt_astree, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::ast – Serialisierung abstrakter Syntaxbäume

Übersicht

package require Tcl 8.5 9
package require pt::ast ?**1.2**?
::pt::ast verify *serial* ?*canonvar*?
::pt::ast verify-as-canonical *serial*
::pt::ast canonicalize *serial*
::pt::ast print *serial*
::pt::ast bottomup *cmdprefix ast*
cmdprefix *ast*
::pt::ast topdown *cmdprefix pe*
::pt::ast equal *seriala serialb*
::pt::ast new0 *s loc* ?*child...*?
::pt::ast new *s start end* ?*child...*?

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt Befehle zur Verfügung, mit denen Sie die von den Parser-Tools verwalteten und im Abschnitt AST-Serialisierungsformat beschriebenen Serialisierungen abstrakter Syntaxbäume bearbeiten können.

Dies ist ein unterstützendes Paket in der Kernschicht der Parser-Tools.

IMAGE: arch_core_support

API

::pt::ast verify *serial* ?*canonvar*?

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige Serialisierung eines abstrakten Syntaxbaums ist, und löst einen Fehler aus, wenn dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Wenn das Argument *canonvar* angegeben wird, wird es als Name einer Variablen im aufrufenden Kontext interpretiert. In diese Variable wird genau dann geschrieben, wenn *serial* eine gültige reguläre Serialisierung ist. Ihr Wert ist ein Boolescher Wert, wobei **True** angibt, dass die Serialisierung nicht nur gültig, sondern auch *kanonisch* ist. Bei einer gültigen, aber nicht kanonischen Serialisierung wird **False** geschrieben.

Informationen zur Spezifikation von Serialisierungen finden Sie im Abschnitt AST-Serialisierungsformat.

::pt::ast verify-as-canonical *serial*

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige *kanonische* Serialisierung eines abstrakten Syntaxbaums ist, und löst einen Fehler aus, falls dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Informationen zur Spezifikation kanonischer Serialisierungen finden Sie im Abschnitt AST-Serialisierungsformat.

::pt::ast canonicalize *serial*

Dieser Befehl geht davon aus, dass der Inhalt von *serial* eine gültige *reguläre* Serialisierung einer abstrakten Syntax ist, und löst einen Fehler aus, wenn dies nicht der Fall ist.

Anschließend wandelt er die Eingabe in die *kanonische* Serialisierung des enthaltenen Baums um und gibt diese als Ergebnis zurück. Wenn die Eingabe bereits kanonisch ist, wird sie unverändert zurückgegeben.

Zur Spezifikation regulärer und kanonischer Serialisierungen siehe den Abschnitt AST-Serialisierungsformat.

::pt::ast print *serial*

Dieser Befehl geht davon aus, dass das Argument *serial* eine gültige Serialisierung eines abstrakten Syntaxbaums enthält, und gibt eine Zeichenkette zurück, die diesen Baum in einer für Menschen lesbaren Form enthält.

Das genaue Format dieser Darstellung ist nicht spezifiziert und darf nicht für die Analyse oder andere maschinelle Vorgänge herangezogen werden.

Informationen zur Spezifikation von Serialisierungen finden Sie im Abschnitt AST-Serialisierungsformat.

::pt::ast bottomup *cmdprefix ast*

Dieser Befehl durchläuft den abstrakten Syntaxbaum *ast* von unten nach oben bis zur Wurzel und ruft dabei für jeden Knoten das Befehlspräfix *cmdprefix* auf. Dies bedeutet, dass die Kindknoten eines Knotens N vor N selbst verarbeitet werden.

Das Befehlspräfix hat die Signatur

cmdprefix *ast*

Das heißt, er wird mit dem AST-Knoten aufgerufen, an dem sich der Durchlauf gerade befindet.

Das vom Befehlspräfix gegebene Ergebnis ersetzt *ast* in dem Knoten, dessen Kindknoten es war, wodurch Transformationen des Baums ermöglicht werden.

Dies bedeutet auch, dass für jeden inneren Knoten der Inhalt der Kindknoten den Ergebnissen der Befehlspräfixe entspricht, die für die Kindknoten dieses Knotens aufgerufen wurden.

::pt::ast topdown *cmdprefix pe*

Dieser Befehl durchläuft den abstrakten Syntaxbaum *ast* von der Wurzel bis zu den Blättern und ruft dabei für jeden Knoten das Befehlspräfix *cmdprefix* auf. Dies bedeutet, dass die Kindknoten eines Knotens N erst nach N verarbeitet werden.

Das Befehlspräfix hat dieselbe Signatur wie bei **bottomup** (siehe oben).

Das vom Befehlspräfix gegebene Ergebnis wird *ignoriert*.

::pt::ast equal *seriala serialb*

Dieser Befehl prüft die beiden abstrakten Syntaxbäume *seriala* und *serialb* auf strukturelle Übereinstimmung. Das Ergebnis des Befehls ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn die Bäume identisch sind, andernfalls auf **false**.

Der Zeichenkettenvergleich ist nur dann brauchbar, wenn davon ausgegangen werden kann, dass es sich bei den beiden Bäumen um reine Tcl-Listen handelt.

::pt::ast new0 *s loc ?child...?*

Dieser Befehl erstellt den AST für einen nichtterminalen Knoten, der sich auf das Symbol *s* an der Position *loc* in der Eingabe bezieht, sowie die Menge der Kindknoten *child* ..., von links nach rechts. Letztere kann leer sein. Der erzeugte Knoten wird als Ergebnis des Befehls zurückgegeben. Die Endposition ist *loc*-1, d. h. ein Zeichen vor dem Start. Dieser Knotentyp ist für Regeln möglich, die optionale Teile enthalten.

::pt::ast new *s start end ?child...?*

Dieser Befehl erstellt den AST für einen nichtterminalen Knoten, der sich auf das Symbol *s*

bezieht, das den Positionsbereich von *start* bis *end* in der Eingabe abdeckt, sowie die Menge der Kindknoten *child* ..., von links nach rechts. Letztere kann leer sein. Der erstellte Knoten wird als Ergebnis des Befehls zurückgegeben.

AST-Serialisierungsformat

An dieser Stelle legen wir das Format fest, das von den Parser-Tools verwendet wird, um abstrakte Syntaxbäume (ASTs) als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Jeder Knoten in einem AST repräsentiert ein nichtterminales Symbol einer Grammatik sowie den Bereich der Tokens/Zeichen in der Eingabe, den dieses Symbol abdeckt. ASTs enthalten keine terminalen Symbole, d. h. keine Tokens/Zeichen. Diese lassen sich anhand der Position eines Symbols aus der Eingabe rekonstruieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Baum kann zwar mehr als eine reguläre Serialisierung aufweisen, jedoch ist genau nur eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen AST entspricht der Serialisierung seines Wurzelknotens.
1. Die Serialisierung eines beliebigen Knotens ist eine Tcl-Liste, die mindestens drei Elemente enthält.
    1. Das erste Element ist der Name des im Knoten gespeicherten nichtterminalen Symbols.
    2. Das zweite und dritte Element geben die Positionen des ersten und letzten Tokens in dem Token-Strom an, den der Knoten repräsentiert (abdeckt).
        1. Positionen werden als nicht-negative ganzzahlige Offsets vom Anfang des Token-Stroms angegeben, wobei das erste im Strom gefundene Token am Offset 0 (Null) liegt.
        2. Die Endposition muss mindestens so groß sein wie die Startposition.

1. Alle Elemente nach den ersten drei sind die Kindknoten des Knotens, selbst wieder Knoten. Die Serialisierung eines Knotens ohne Kindknoten, also eines Blattknotens, hat daher genau drei Elemente. In der Liste steht der linke Kindknoten zuerst und der rechte zuletzt.

Kanonische Serialisierung

Die kanonische Serialisierung eines abstrakten Syntaxbaums hat das im vorigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Baums eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden Parsing Expression Grammar

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
```

```
    Term       <- Factor (MulOp Factor)*                          ;
    AddOp      <- '+'/'-'                                         ;
    Factor     <- '(' Expression ')' / Number                     ;
END;
```

und der Eingabezeichenkette

```
 120+5
```

Dann sollte ein Parser den folgenden abstrakten Syntaxbaum liefern (mit Ausnahme von Leerzeichen)

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

Oder, grafischer

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                          |             |
|                          +- Digit 2 2 | 0
|                                        |
Expression 0 4 --- Factor 0 4 -+------------------------------ AddOp 3
    3 | +
|                                        |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_cparam_config_critcl
Stichwörter: pt_cparam_config_critcl, EBNF, LL(k), PEG, TDPL, context-free languages, expression , grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name
pt::cparam::configuration::critcl – C/PARAM, vorgefertigte Konfiguration, Critcl

Übersicht
package require Tcl 8.5 9
package require pt::cparam::configuration::critcl ?**1.0.3**?
::pt::cparam::configuration::critcl def *name pkg version cmdprefix*

Beschreibung
Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket dient als Ergänzung zu pt::peg::to::cparam und soll die Verwendung dieses in hohem Maße konfigurierbaren Pakets durch die Bereitstellung einer vorgefertigten Konfiguration vereinfachen. Wenn diese Konfiguration angewendet wird, generiert das Paket pt::peg::to::cparam Pakete mit Parsern, die auf **critcl** basieren.

Es handelt sich um ein unterstützendes Paket in der Kernschicht der Parser-Tools.

IMAGE: arch_core_support

API
::pt::cparam::configuration::critcl def *name pkg version cmdprefix*

Der Befehl wendet die in diesem Paket bereitgestellte Konfiguration auf *cmdprefix* an, wodurch Parser auf Basis von **critcl** erstellt werden, deren Klasse *name* lautet, im Paket *pkg* mit *version*.

Die Verwendung eines Befehlspräfixes als API ermöglicht die Anwendung der Konfiguration nicht nur auf pt::peg::to::cparam (pt::peg::to::cparam configure), sondern auch auf Export-Manager-Instanzen und PEG-Container (**$export configuration set** bzw. **[$container exporter] configuration set**).

Oder auf jedes andere Befehlspräfix, das zwei Argumente – Option und Wert – akzeptiert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_cparam_config_tea

Stichwörter: pt_cparam_config_tea, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::cparam::configuration::tea – C/PARAM, vorgefertigte Konfiguration, TEA

Übersicht

package require Tcl 8.5 9
package require pt::cparam::configuration::tea ?**0.2**?
::pt::cparam::configuration::tea def *name pkg version cmdprefix*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket dient als Ergänzung zu pt::peg::to::cparam und soll die Verwendung dieses in hohem Maße konfigurierbaren Pakets durch die Bereitstellung einer vorgefertigten Konfiguration vereinfachen. Wenn diese Konfiguration angewendet wird, generiert das Paket pt::peg::to::cparam bloßen Parser-Code, der zur Einbindung in eine auf *TEA* basierende C-Erweiterung bereit ist.

Es handelt sich um ein unterstützendes Paket in der Kernschicht der Parser-Tools.

IMAGE: arch_core_support

API

::pt::cparam::configuration::tea def *name pkg version cmdprefix*

Der Befehl wendet die in diesem Paket bereitgestellte Konfiguration auf *cmdprefix* an, wodurch Parser auf Basis von **tea** erstellt werden, deren Klasse *name* lautet, im Paket *pkg* mit *version*.

Die Verwendung eines Befehlspräfixes als API ermöglicht die Anwendung der Konfiguration nicht nur auf pt::peg::to::cparam (pt::peg::to::cparam configure), sondern auch auf Export-Manager-Instanzen und PEG-Container (**$export configuration set** bzw. **[$container exporter] configuration set**).

Oder auf jedes andere Befehlspräfix, das zwei Argumente – Option und Wert – akzeptiert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_from_api

Stichwörter: pt_from_api, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_import_api – Import-API für Parser-Tools

Übersicht

package require Tcl 8.5 9
CONVERTER convert *text*
IncludeFile *currentfile path*
::import *text*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

In diesem Dokument werden zwei APIs beschrieben. Zunächst die API, die allen Paketen gemeinsam ist und der Konvertierung anderer Formate in Parsing Expression Grammars dient, und anschließend die API, die den Paketen gemeinsam ist, welche die Import-Plugins implementieren, die auf den Konvertierungspaketen aufbauen.

Die Zielgruppe sind Personen, die einen eigenen Konverter für eine bestimmte Eingabeart und/oder ein Import-Plugin für ihren eigenen oder einen anderen Konverter erstellen möchten.

Es befindet sich im Abschnitt "Import" der Kernschicht der Parser-Tools.

IMAGE: arch_core_import

Converter-API

Jeder (Grammatik-)Importkonverter muss die nachstehend aufgeführten Regeln befolgen:

1. Ein Konverter ist ein Paket. Sein Name ist beliebig, es wird jedoch empfohlen, ihn unter dem Namensraum **::pt::peg::from** anzulegen.
2. Das Paket stellt entweder einen einzelnen Tcl-Befehl bereit, der der unten beschriebenen Gliederung der API entspricht, oder einen Klassenbefehl, dessen Instanzen derselben API folgen. Die Befehle, die dieser API entsprechen, werden als *Konverterbefehle* bezeichnet.
3. Ein Konverter-Befehl muss die folgende einzelne Methode mit der angegebenen Signatur und Semantik bereitstellen. Konverter-Befehle dürfen weitere eigene Methoden bereitstellen, jedoch nicht weniger, und sie dürfen für die standardisierte Methode keine abweichende Semantik bereitstellen.

   CONVERTER convert *text*

   Diese Methode muss einen beliebigen *text* entgegennehmen, das heißt eine Parsing Expression Grammar in einem beliebigen Format. Das Ergebnis der Methode muss die kanonische Serialisierung einer Parsing Expression Grammar sein, wie im Abschnitt PEG-Serialisierungsformat festgelegt, also das Ergebnis des Einlesens und Umwandelns des Eingabetextes.

Plugin-API

Jedes (Grammatik-)Import-Plugin muss die nachstehend aufgeführten Regeln befolgen:

1. Ein Plugin ist ein Paket.
2. Der Name eines Plugin-Pakets hat die Form pt::peg::import::**FOO**, wobei **FOO** der Name des Formats ist, für das das Plugin Eingaben akzeptiert.
3. Das Plugin kann davon ausgehen, dass das Paket **pt::peg::import::plugin** vorhanden ist, was darauf hindeutet, dass es von einem echten Plugin-Manager aufgerufen wurde.

   Es wird empfohlen, dass ein Plugin das Vorhandensein dieses Pakets überprüft.
4. Das Plugin kann davon ausgehen, dass ein Befehl **IncludeFile** vorhanden ist, mit der Signatur

   IncludeFile *currentfile path*

   Dieser Befehl muss vom Plugin aufgerufen werden, wenn es eine eingebundene Datei verarbeiten muss, wenn das Format ein solches Konzept vorsieht.

   Das Plugin muss die folgenden Argumente übergeben

   string *currentfile*

   Der Pfad der Datei, die es gerade verarbeitet. Das kann die leere Zeichenkette sein, wenn keiner bekannt ist.

   string *path*

Der Pfad der Include-Datei, wie ihn die gerade verarbeitete Include-Anweisung angibt.

```
Das Ergebnis des Befehls ist eine Liste mit fünf Elementen, die
Folgendes enthält:

1. Ein boolesches Flag, das Erfolg (**True**) oder Misserfolg
(**False**) des Vorgangs angibt.

2. Im Erfolgsfall den Inhalt der eingebundenen Datei, andernfalls
die leere Zeichenkette.

3. Der aufgelöste, d. h. der absolute Pfad der eingebundenen
Datei, sofern möglich, oder das unveränderte *path*-Argument. Dies
dient zur Anzeige in einer Fehlermeldung oder als
*currentfile*-Argument eines weiteren Aufrufs von **IncludeFile**,
falls diese Datei weitere Dateien enthält.

4. Im Erfolgsfall eine leere Zeichenkette und im Fehlerfall einen
Code, der den Grund dafür angibt, und zwar einen der folgenden:

    notfound
    : Die angegebene Datei konnte nicht gefunden werden.

    notread
    : Die angegebene Datei wurde gefunden, konnte jedoch nicht in
den Arbeitsspeicher geladen werden.


5. Eine leere Zeichenfolge im Falle eines Erfolgs oder eines
Fehlers mit dem Code **notfound** sowie eine zusätzliche
Fehlermeldung, die den Grund für einen Fehler mit dem Code
**notread** näher erläutert.
```

1. Ein Plugin muss im globalen Namespace einen einzigen Befehl mit der unten dargestellten Signatur bereitstellen. Plugins dürfen weitere eigene Befehle bereitstellen, jedoch nicht weniger, und sie dürfen für den standardisierten Befehl keine abweichende Semantik vorgeben.

   ::import *text*

   Dieser Befehl muss einen Text entgegennehmen, der eine Parsing Expression Grammar in einem beliebigen Format enthält. Das Ergebnis des Befehls muss das Ergebnis des vom Plugin für die Eingabegrammatik aufgerufenen Konverters sein, d. h. die kanonische Serialisierung der in der Eingabe enthaltenen Parsing Expression Grammar.

   string *text*

   Dieses Argument enthält die Parsing Expression Grammar, für die die Serialisierung erzeugt werden soll. Die Spezifikation dessen, was eine *kanonische* Serialisierung ist, finden Sie im Abschnitt PEG-Serialisierungsformat.

1. Ein einzelner Nutzungszyklus eines Plugins besteht aus einem Aufruf des Befehls import. Dieser Aufruf muss das Plugin in einem Zustand hinterlassen, in dem ein weiterer Nutzungszyklus problemlos ausgeführt werden kann.

Verwendung

Ein Konverter wird so verwendet:

```
    # Get the converter (single command here, not class)
    package require the-converter-package

    # Perform the conversion
    set serial [theconverter convert $thegrammartext]

    ... process the result ...
```

Das Plugin **FOO** wird so verwendet:

```
    # Get an import plugin manager
    package require pt::peg::import
    pt::peg::import I

    # Run the plugin, and the converter inside.
    set serial [I import serial $thegrammartext FOO]

    ... process the result ...
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur

des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

mode
Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

value
Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf
Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung
Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
```

```
    Term       <- Factor (MulOp Factor)*                        ;
    AddOp      <- '+'/'-'                                       ;
    Factor     <- '(' Expression ')' / Number                   ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke

auf der Grundlage des eingebauten Tcl-Befehls **string is**.

4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, … ist das Ergebnis von [list / **e1 e2** …] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als

priorisierte Auswahl.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_introduction

Stichwörter: pt_introduction, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_introduction – Einleitung in die Parser-Tools

Übersicht

package require Tcl 8.5 9

Beschreibung

Willkommen bei den Parser-Tools, einem System zur Erstellung und Bearbeitung von Parsern und den ihnen zugrunde liegenden Grammatiken.

Welche Ziele haben Sie dazu bewegt, hierher zu kommen?

1. Möchten Sie einfach nur einen Parser für eine bestimmte Sprache erstellen?

   In diesem Fall werfen Sie doch einen Blick auf unsere Parser-Generator-Anwendung **pt** oder, wenn Sie etwas tiefer in die Materie einsteigen möchten, auf das zugrunde liegende Paket **pt::pgen**.
2. Möchten Sie mehr über die Architektur des Systems erfahren?

   Diese wird im folgenden Abschnitt Architektur von Parser-Tools beschrieben.
3. Interessieren Sie sich für die theoretischen Grundlagen, auf denen die Pakete und Werkzeuge basieren?

   Lesen Sie dazu die *Introduction to Parsing Expression Grammars*.

Architektur von Parser-Tools

Das System lässt sich grob in drei Schichten unterteilen, wie in der folgenden Abbildung dargestellt

IMAGE: architecture

Diese Schichten sind, von oben nach unten:

1. An oberster Stelle befinden sich die Anwendung und die Pakete, die die Pakete der darunterliegenden Schicht nutzen, um gängige Anwendungsfälle umzusetzen. Ein Beispiel ist das bereits erwähnte **pt::pgen**, das einen Parser-Generator bereitstellt.

   Die Liste der zu dieser Schicht gehörenden Pakete steht im Abschnitt Benutzerpakete.
2. In dieser Schicht befinden sich die Pakete, die den Kern der Funktionalität für das gesamte System bilden. Im Wesentlichen handelt es sich dabei um eine Reihe von Bausteinen, die sich auf unzählige Arten kombinieren lassen, ähnlich wie bei Lego (tm). Die Pakete der vorherigen Schicht sind "lediglich" vorgefertigte Kombinationen, um die wichtigsten Anwendungsfälle abzudecken.

   Die Liste der zu dieser Schicht gehörenden Pakete steht im Abschnitt Kernpakete

3. Zu guter Letzt gibt es die Schicht mit den Unterstützungspaketen, die allgemeine Funktionen bereitstellen, die nicht unbedingt in das Modul gehören.

   Die Liste der zu dieser Schicht gehörenden Pakete steht im Abschnitt Unterstützungspakete

Benutzerpakete

- **pt::pgen**

Kernpakete

Diese Schicht ist weiter in sechs Bereiche unterteilt: Speicherung, Import, Export, Transformation und Ausführung von Grammatiken sowie grammatikspezifische Unterstützungspakete.

Speicher

siehe unten ...

- **pt::peg::container**

Export

siehe unten ...

- **pt::peg::export**
- **pt::peg::export::container**
- **pt::peg::export::json**
- **pt::peg::export::peg**
- pt::peg::to::container
- pt::peg::to::json
- pt::peg::to::peg
- pt::peg::to::param
- pt::peg::to::tclparam
- pt::peg::to::cparam

Import

siehe unten ...

- **pt::peg::import**
- pt::peg::import::container
- **pt::peg::import::json**
- **pt::peg::import::peg**
- pt::peg::from::container
- pt::peg::from::json
- pt::peg::from::peg

Transformation

siehe unten ...

Ausführung

siehe unten ...

- **pt::peg::interp**
- **pt::rde**

Unterstützung

siehe unten ...

- **pt::tclparam::configuration::snit**

- **pt::tclparam::configuration::tcloo**
- **pt::cparam::configuration::critcl**
- **pt::ast**
- **pt::pe**
- **pt::peg**

Unterstützungspakete

- pt::peg::container::peg
- **text::write**
- **configuration**
- **paths**
- **char**

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_json_language

Stichwörter: pt_json_language, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::json_language – JSON-Austauschformat für Grammatiken

Übersicht

package require Tcl 8.5 9

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Das Format **json** für Parsing Expression Grammars wurde als Datenaustauschformat

geschrieben, das nicht an Tcl gebunden ist. Es wurde definiert, um den Austausch von Grammatiken mit PackRat/PEG-basierten Parsergeneratoren für andere Sprachen zu ermöglichen.

Dies wird durch die folgenden Regeln formal festgelegt:

1. Das JSON-Format eines jeden PEG ist ein JSON-Objekt.
2. Dieses Objekt enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
3. Der Inhalt der Grammatik ist ein JSON-Objekt, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein JSON-Objekt, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein JSON-Objekt. Die relevanten Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des Parsing-Ausdrucks enthält, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert ist ein JSON-Wert mit einem von drei Werten, die festlegen, wie ein Parser den durch das Symbol erzeugten semantischen Wert verarbeiten soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des
    Start-Parsing-Ausdrucks der Grammatik enthält, wie im Abschnitt
    [PE-Serialisierungsformat] beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Als Anmerkung für den fortgeschrittenen Leser: Dies entspricht im Wesentlichen der

Tcl-Serialisierung von PE-Grammatiken, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist, mit dem Unterschied, dass die Tcl-Dictionaries und -Listen dieses Formats auf JSON-Objekte und -Arrays abgebildet werden. Lediglich die Parsing-Ausdrücke selbst werden nicht weiter übersetzt, sondern als JSON-Zeichenketten beibehalten, die eine verschachtelte Tcl-Liste enthalten, und auch für das JSON gibt es kein Konzept der Kanonizität.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Eine JSON-Serialisierung dafür lautet

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
```

```
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

und eine Tcl-Serialisierung davon lautet

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

Die Ähnlichkeit des Letzteren mit JSON dürfte ziemlich offensichtlich sein.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter

allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke

als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes

Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_param

Stichwörter: pt_param, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer, virtual machine

Name

pt::param – Spezifikation der PackRat-Maschine

Übersicht

package require Tcl 8.5 9

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Willkommen bei der PackRat Machine (kurz: *PARAM*), einer virtuellen Maschine, die speziell auf die Unterstützung rekursiver Abstiegsparser, insbesondere von Packrat-Parsern, ausgerichtet ist. Zu diesem Zweck verfügt sie über Funktionen wie das Zwischenspeichern und die Wiederverwendung von Teilergebnissen, das Zwischenspeichern der gelesenen Eingabe sowie die Möglichkeit, sowohl bei der Eingabe als auch bei der Erstellung des AST zurückzuspringen.

Dieses Dokument legt den Architekturzustand und den Befehlssatz der Maschine fest.

Architekturzustand

Jede PARAM-Implementierung muss mindestens den folgenden Zustand verwalten:

***Input* (IN)**

Dies ist der Kanal, aus dem die zu verarbeitenden Zeichen gelesen werden.

Dieser Teil des Maschinenzustands wird von den im Abschnitt Eingabeverarbeitung definierten Befehlen verwendet und verändert.

***Aktuelles Zeichen* (CC)**

Das Zeichen aus der *Eingabe*, das gerade gegen seine möglichen Alternativen geprüft wird.

Dieser Teil des Maschinenzustands wird von den im Abschnitt Zeichenverarbeitung definierten Befehlen verwendet und verändert.

***Aktuelle Position* (CL)**

Die Position des *aktuellen Zeichens* in der *Eingabe*, ausgedrückt als Versatz relativ zum Anfang der Eingabe. Zeichen-Offsets werden ab **0** gezählt.

Dieser Teil des Maschinenzustands wird von den Befehlen verwendet und verändert, die in den Abschnitten Zeichenverarbeitung, Positionsverwaltung und Ausführung von Nichtterminalen definiert sind.

***Positionsstapel* (LS)**

Ein Stapel von Positionen in der *Eingabe*, gespeichert für mögliches Backtracking.

Dieser Teil des Maschinenzustands wird von den Befehlen verwendet und verändert, die in den Abschnitten Zeichenverarbeitung, Positionsverwaltung und Ausführung von Nichtterminalen definiert sind.

***Status* (ST)**

Der Status des letzten Versuchs, die *Eingabe* zu prüfen, der entweder Erfolg oder Fehlschlag anzeigt.

Dieser Teil des Maschinenzustands wird von den Befehlen verwendet und verändert, die in den Abschnitten Statussteuerung, Zeichenverarbeitung und Ausführung von Nichtterminalen definiert sind.

***Semantischer Wert* (SV)**

Der aktuelle semantische Wert, der entweder leer ist oder einen Knoten des aus der Eingabe konstruierten AST darstellt.

Dieser Teil des Maschinenzustands wird von den in den Abschnitten Aufbau von Werten und AST-Aufbau definierten Anweisungen verwendet und verändert.

***AST-Reduktionsstapel* (ARS)**

Der Stapel aus Teil-ASTs, der während des Prozesses der Verarbeitung von nichtterminalen Symbolen aufgebaut wird.

Dieser Teil des Maschinenzustands wird durch die in den Abschnitten Aufbau von Werten und AST-Aufbau definierten Befehle verwendet und verändert.

***AST-Stapel* (AS)**

Der Stapel der Reduktionsstapel, der für ein eventuelles Zurückverfolgen gespeichert wird.

Dieser Teil des Maschinenzustands wird von den Befehlen verwendet und verändert, die in den Abschnitten Aufbau von Werten und AST-Aufbau definiert sind.

***Fehlerstatus* (ER)**

Der aktuelle Fehlerzustand der Maschine. Dieser ist entweder leer oder auf ein Paar aus einer Position in der Eingabe und der Menge der Meldungen für diese Position gesetzt.

Hinweis: Dieser Teil des Maschinenzustands kann auch dann gesetzt werden, wenn der letzte Test des *aktuellen Zeichens* erfolgreich war. Beispielsweise ist der Operator * (er passt auf einen Teilausdruck null- oder mehrmals) in einer PEG immer erfolgreich, selbst wenn weiter hinten in der Eingabe ein Problem auftritt und zurückgesetzt werden muss. Solche Probleme dürfen beim Fortsetzen des Parsens nicht vergessen werden.

Dieser Teil des Maschinenzustands wird durch die in den Abschnitten Fehlerbehandlung, Zeichenverarbeitung und Ausführung von Nichtterminalen definierten Anweisungen verwendet und verändert.

***Fehlerstapel* (ES)**

Der Stapel der Fehler-Stati, der für das Backtracking gespeichert wird. Dies ermöglicht es der Maschine, aktuelle und ältere Fehler-Stati zusammenzuführen, wenn sie nach einem fehlgeschlagenen Abgleich ein Backtracking bei Auswahlmöglichkeiten durchführt.

Dieser Teil des Maschinenzustands wird von den Anweisungen verwendet und verändert, die in den Abschnitten Fehlerbehandlung, Zeichenverarbeitung und Ausführung von Nichtterminalen definiert sind.

***Nichtterminal-Cache* (NC)**

Ein Cache mit Maschinenzuständen, die durch Paare aus dem Namen eines nichtterminalen Symbols und einer Position in der Eingabe indiziert sind. Jedem Paar (N, L) ist ein 4-Tupel zugeordnet, das die Werte für CL, ST, SV und ER enthält, die zu verwenden sind, nachdem das Nichtterminal N ab der Position L geparst wurde. Es hilft Backtracking-Parsern bei der Leistung und ermöglicht es ihnen, ein aufwendiges erneutes Parsen komplexer Nichtterminale zu vermeiden, wenn diese an einer gegebenen Position schon einmal vorgekommen sind.

Die Schlüsselposition ist die Stelle, an der die Maschine den Versuch begann, das genannte nichtterminale Symbol abzugleichen, und die Position im gespeicherten 4-Tupel ist die Stelle, an der die Maschine nach Abschluss des Versuchs endete, unabhängig vom Erfolg des Versuchs.

Dieser Teil des Maschinenzustands wird von den im Abschnitt Ausführung von Nichtterminalen definierten Anweisungen verwendet und modifiziert.

***Terminal Cache* (TC)**

Ein Cache mit aus IN gelesenen Zeichen, deren Position in IN als Paar aus Zeile und Spalte angegeben ist, indiziert nach der Position in IN, diesmal als Zeichenversatz vom Anfang von IN. Es hilft Backtracking-Parsern, ein möglicherweise aufwendiges erneutes Lesen von Zeichen aus IN zu vermeiden, und ermöglicht sogar Backtracking, wenn IN nicht wahlfrei positionierbar ist.

Dieser Teil des Maschinenzustands wird von den im Abschnitt Eingabeverarbeitung definierten Befehlen verwendet und modifiziert.

Befehlssatz

Nachdem der Architekturzustand der Maschine festgelegt ist, ist es nun möglich, den Befehlssatz zu definieren, der auf diesen Zustand einwirkt und von jeder Implementierung des PARAM ausgeführt werden soll. Die 37 Befehle sind grob nach dem Zustand gruppiert, auf den sie während ihrer Ausführung einwirken und/oder den sie abfragen.

Eingabeverarbeitung

Die Befehle in diesem Abschnitt greifen hauptsächlich auf IN zu und holen die zu verarbeitenden Zeichen in die Maschine.

input_next *msg*

Diese Methode liest das nächste Zeichen, d. h. das Zeichen nach CL, aus IN. Bei Erfolg wird dieses Zeichen zu CC, CL wird um eins erhöht, ES wird gelöscht und der Vorgang wird in ST als erfolgreich vermerkt.

Der Vorgang kann das Zeichen aus IN lesen, falls das nächste Zeichen TC noch nicht bekannt ist. Bei Erfolg wird das neue Zeichen zusammen mit seiner Position (Zeile, Spalte) in TC gespeichert; ansonsten verhält sich die Operation wie oben beschrieben. Zukünftige Lesevorgänge von derselben Position – die aufgrund von Backtracking möglich sind – werden dann aus TC statt aus IN bedient.

Wenn hingegen das Ende von IN erreicht wurde, wird der Vorgang in ST als fehlgeschlagen vermerkt, CL bleibt unverändert, und das Paar aus CL und *msg* wird zum neuen ES.

Zeichenverarbeitung

Die Befehle in diesem Abschnitt greifen hauptsächlich auf CC zu und prüfen es anhand von Zeichenklassen, Zeichenbereichen und einzelnen Zeichen.

test_alnum

Diese Anweisung implementiert den speziellen PE-Operator "alnum", der prüft, ob CC in die gleichnamige Zeichenklasse fällt oder nicht.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_alpha

Diese Anweisung implementiert den speziellen PE-Operator "alpha", der prüft, ob CC in die gleichnamige Zeichenklasse fällt oder nicht.

Sowohl der Erfolg als auch das Scheitern der Prüfung werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (bei der die Kodierung als Blatt-Parsing-Ausdruck erfolgt) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_ascii

Diese Anweisung implementiert den speziellen PE-Operator "ascii", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgesetzt wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_char *char*

Diese Anweisung implementiert den Operator zur Zeichenübereinstimmung, d. h., sie prüft, ob CC gleich *char* ist.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST protokolliert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (bei der die Kodierung als Blatt-Parsing-Ausdruck erfolgt) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_ddigit

Diese Anweisung implementiert den speziellen PE-Operator "ddigit", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_digit

Diese Anweisung implementiert den speziellen PE-Operator "digit", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Prüfung werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (bei der die Kodierung als Blatt-Parsing-Ausdruck erfolgt) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_graph

Diese Anweisung implementiert den speziellen PE-Operator "graph", der prüft, ob CC in die gleichnamige Zeichenklasse fällt oder nicht.

Sowohl der Erfolg als auch das Scheitern der Prüfung werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_lower

Diese Anweisung implementiert den speziellen PE-Operator "lower", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (in Form eines Blatt-Parsing-Ausdrucks) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_print

Diese Anweisung implementiert den speziellen PE-Operator "print", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Prüfung werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_punct

Diese Anweisung implementiert den speziellen PE-Operator "punct", der prüft, ob CC in die gleichnamige Zeichenklasse fällt oder nicht.

Sowohl der Erfolg als auch das Scheitern der Prüfung werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (bei der die Kodierung als Blatt-Parsing-Ausdruck erfolgt) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_range *chars chare*

Diese Anweisung implementiert den Bereichsabgleichsoperator, d. h., sie prüft, ob CC in das Zeichenintervall fällt, das von den beiden Zeichen *chars* bis *chare* (beide eingeschlossen) gebildet wird.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST protokolliert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_space

Diese Anweisung implementiert den speziellen PE-Operator "space", der prüft, ob CC in die gleichnamige Zeichenklasse fällt oder nicht.

Sowohl der Erfolg als auch das Scheitern der Prüfung werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_upper

Diese Anweisung implementiert den speziellen PE-Operator "upper", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST vermerkt. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_wordchar
Diese Anweisung implementiert den speziellen PE-Operator "wordchar", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Tests werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (mit Kodierung als Blatt-Parsing-Ausdruck) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

test_xdigit
Diese Anweisung implementiert den speziellen PE-Operator "xdigit", der prüft, ob CC zur gleichnamigen Zeichenklasse gehört oder nicht.

Sowohl der Erfolg als auch das Scheitern der Prüfung werden direkt in ST gespeichert. Bei Erfolg wird ES gelöscht, während bei Scheitern das Paar aus CL und der erwarteten Eingabe (bei der die Kodierung als Blatt-Parsing-Ausdruck erfolgt) als neuer ES gesetzt wird und CL anschließend um ein Zeichen zurückgespult wird, um die Maschine auf einen weiteren Parsing-Versuch mit einer möglichen Alternative vorzubereiten.

Fehlerbehandlung

Die Anweisungen in diesem Abschnitt greifen hauptsächlich auf ER und ES zu.

error_clear
Dieser Befehl löscht ER.

error_push
Dieser Befehl erstellt eine Kopie von ER und legt diese auf ES ab.

error_pop_merge
Dieser Befehl nimmt den obersten Eintrag von ES und führt den darin enthaltenen Fehlerstatus mit ER zusammen; das Ergebnis wird der neue ER.

Die Zusammenführung unterliegt vier Regeln, wobei das Ergebnis der Zusammenführung
1. Leer, wenn beide Zustände leer sind.
2. Der nicht leere Zustand, wenn nur einer der beiden Zustände nicht leer ist.
3. Der Zustand mit der größeren Position, wenn die beiden Zustände verschiedene Positionen angeben.
4. Das Paar aus der beiden Zuständen gemeinsamen Position und der Vereinigungsmenge ihrer Meldungen, wenn die Zustände dieselbe Position haben.

error_nonterminal *symbol*
Dies ist ein bewachter Befehl. Er führt keine Aktion aus, wenn ES leer ist oder die Position in ES nicht unmittelbar hinter der zuletzt in LS gespeicherten Position liegt. Andernfalls legt er das Paar aus dieser Position und dem Nichtterminal *symbol* als neues ES fest.

Hinweis: Im obigen Zusammenhang bedeutet "unmittelbar danach" "diese Position plus eins" oder auch "die Position des nächsten Zeichens nach dieser Position".

Statussteuerung

Die Anweisungen in diesem Abschnitt greifen direkt auf ST zu.

status_ok
Dieser Befehl setzt ST auf **true** und protokolliert den Erfolg.

status_fail
Dieser Befehl setzt ST auf **false** und protokolliert dabei einen Fehler.

status_negate
Dieser Befehl kehrt den Wert von ST um und verwandelt damit einen Fehlschlag in einen Erfolg und umgekehrt.

Positionsverwaltung

Die Befehle in diesem Abschnitt greifen auf CL und LS zu.

loc_push
Dieser Befehl erstellt eine Kopie von CL und schiebt diese auf LS.

loc_pop_discard
Dieser Befehl nimmt die zuletzt gespeicherte Position von LS herunter.

loc_pop_rewind
Dieser Befehl nimmt die zuletzt gespeicherte Position von LS und stellt sie als CL wieder her.

Ausführung von Nichtterminalen

Die Befehle in diesem Abschnitt dienen dem Zugriff auf und der Bearbeitung von NC.

symbol_restore *symbol*
Dieser Befehl prüft, ob NC Daten für das Nichtterminal *symbol* an CL enthält oder nicht. Das Ergebnis des Befehls ist ein boolescher Flag, wobei **True** anzeigt, dass Daten im Cache gefunden wurden. In diesem Fall hat der Befehl den Architekturzustand der Maschine mit den zwischengespeicherten Informationen – nämlich CL, ST, ER und SV – weiter aktualisiert.

Die Methode, mit der das Ergebnis des Befehls in den Kontrollfluss transformiert wird, bleibt undefiniert und liegt in der Verantwortung der Implementierung.

symbol_save *symbol*
Dieser Befehl speichert die aktuellen Einstellungen von CL, ST, ER und SV in NC, wobei das Paar aus dem Nichtterminal *symbol* und der zuletzt in LS gespeicherten Position als Schlüssel verwendet wird.

Aufbau von Werten

Die Anweisungen in diesem Abschnitt dienen der Bearbeitung von SV.

value_clear
Dieser Befehl löscht SV.

value_leaf *symbol*
Diese Anweisung erstellt einen AST-Knoten für *symbol*, der den Bereich von IN ab dem ersten Zeichen nach der zuletzt in LS gespeicherten Position bis hin zu CL abdeckt, und speichert ihn

in SV. ...

value_reduce *symbol*
Diese Anweisung verhält sich im Allgemeinen wie **value_nonterminal_leaf**, mit dem Unterschied, dass sie alle auf ARS vorhandenen AST-Knoten – wenn vorhanden – übernimmt und sie zu Kindknoten des neuen Knotens macht, wobei der zuletzt auf ARS gespeicherte Knoten zum rechtesten bzw. letzten Kindknoten wird. Hinweis: ARS wird durch diesen Vorgang nicht verändert.

AST-Aufbau

Die Anweisungen in diesem Abschnitt dienen der Bearbeitung von ARS und AS.

ast_value_push
Dieser Befehl erstellt eine Kopie von SV und legt diese auf ARS ab.

ast_push
Dieser Befehl legt den aktuellen Zustand von ARS auf AS und löscht ARS anschließend.

ast_pop_rewind
Dieser Befehl nimmt den zuletzt auf AS gespeicherten Eintrag herunter und stellt ihn als neuen Zustand von ARS wieder her.

ast_pop_discard
Dieser Befehl nimmt den zuletzt auf AS gespeicherten Eintrag von AS herunter.

Ablaufsteuerung

Normalerweise würde dieser Abschnitt die Spezifikationen der Anweisungen zur Kontrollflusssteuerung des PARAM enthalten, d. h. (un)bedingte Sprünge und Ähnliches. Dieser Teil des PARAM wurde jedoch bewusst nicht spezifiziert. Dies ermöglicht es den Implementierungen, frei zu entscheiden, wie der Kontrollfluss umgesetzt werden soll.

Die Implementierung dieser Maschine in den Parser-Tools, d. h. im Paket **pt::rde**, ist nicht nur in Tcl programmiert, sondern stützt sich auch auf Tcl-Befehle, um die Ablaufsteuerung (Anweisungen) zu gewährleisten.

Wechselwirkung der Befehle mit dem Architekturzustand

```
Instruction     Inputs              Outputs
======================= ======================     ====================
ast_pop_discard     AS          ->  AS
ast_pop_rewind      AS          ->  AS, ARS
ast_push        ARS, AS         ->  AS
ast_value_push      SV, ARS         ->  ARS
======================= ======================     ====================
error_clear     -           ->  ER
error_nonterminal sym   ER, LS          ->  ER
error_pop_merge     ES, ER          ->  ER
error_push      ES, ER          ->  ES
======================= ======================     ====================
input_next msg      IN          ->  TC, CL, CC, ST, ER
======================= ======================     ====================
loc_pop_discard     LS          ->  LS
```

```
loc_pop_rewind      LS          ->  LS, CL
loc_push        CL, LS          ->  LS
======================= ======================     ====================
status_fail     -           ->  ST
status_negate       ST          ->  ST
status_ok       -           ->  ST
======================= ======================     ====================
symbol_restore sym  NC          ->  CL, ST, ER, SV
symbol_save    sym  CL, ST, ER, SV LS   ->  NC
======================= ======================     ====================
test_alnum          CC          ->  ST, ER
test_alpha      CC          ->  ST, ER
test_ascii      CC          ->  ST, ER
test_char char      CC          ->  ST, ER
test_ddigit     CC          ->  ST, ER
test_digit      CC          ->  ST, ER
test_graph      CC          ->  ST, ER
test_lower      CC          ->  ST, ER
test_print      CC          ->  ST, ER
test_punct      CC          ->  ST, ER
test_range chars chare  CC          ->  ST, ER
test_space      CC          ->  ST, ER
test_upper      CC          ->  ST, ER
test_wordchar       CC          ->  ST, ER
test_xdigit     CC          ->  ST, ER
======================= ======================     ====================
value_clear     -           ->  SV
value_leaf symbol   LS, CL          ->  SV
value_reduce symbol ARS, LS, CL     ->  SV
======================= ======================     ====================
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_parse_peg

Stichwörter: pt_parse_peg, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_parse_peg – Parser-Tools: PEG-Parser

Übersicht

package require Tcl 8.5 9
package require pt::parse::peg 1
pt::parse::peg ?*objectName*?
objectName **destroy**
objectName **parse** *chan*
objectName **parset** *text*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt eine Klasse bereit, deren Instanzen Parser für Parsing Expression Grammars in Textform sind.

API für Klassen

pt::parse::peg ?*objectName*?

Der Klassenbefehl erstellt Parser-Instanzen, d. h. Objekte. Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Wenn kein *objectName* angegeben wird, generiert die Klasse einen automatischen Namen und verwendet diesen. Wenn der *objectName* zwar angegeben wurde, jedoch nicht vollqualifiziert ist, wird die Instanz im aktuellen Namespace erstellt.

Instanz-API

Alle Parser-Instanzen stellen mindestens die unten aufgeführten Methoden bereit:

***objectName* destroy**

Diese Methode löscht die Parser-Instanz, gibt den gesamten belegten Speicher sowie alle anderen Ressourcen frei und löscht den Instanzbefehl.

Das Ergebnis des Befehls ist die leere Zeichenkette.

objectName* parse *chan

Bei dieser Methode wird der Parser mit dem Inhalt von *chan* als Eingabe ausgeführt (beginnend an der aktuellen Position im Kanal), bis kein weiteres Parsen mehr möglich ist,

entweder weil das Parsen abgeschlossen ist oder weil ein Syntaxfehler aufgetreten ist.

Zu beachten ist, dass die Parser-Tools auf Tcl 8.5+ basieren. Mit anderen Worten: Das Kanalargument ist nicht auf Dateien, Sockets usw. beschränkt. Uns steht die volle Leistungsfähigkeit von *reflektierten Kanälen* zur Verfügung.

Es sei außerdem darauf hingewiesen, dass der Parser die Zeichen bei Bedarf aus dem Eingabestrom abruft. Wenn ein durch dieses Paket erstellter Parser im Push-Modus, d. h. ereignisgesteuert betrieben werden muss, ist es erforderlich, auf Tcl 8.6+ umzusteigen und **coroutine::auto** zu verwenden, um ihn in eine Coroutine zu verpacken, wobei **read** entsprechend für den Push-Betrieb angepasst werden muss.

Nach erfolgreicher Ausführung gibt der Befehl einen abstrakten Syntaxbaum (AST) als Ergebnis zurück. Dieser AST hat die in Abschnitt **AST-SERIALISIERUNGSFORMAT** festgelegte Form. Als einfache verschachtelte Tcl-Liste kann er anschließend mit beliebigen Tcl-Befehlen des Benutzers weiterverarbeitet werden, um Transformationen, semantische Prüfungen usw. durchzuführen. Um dies zu erleichtern, bietet das Paket **pt::ast** eine Reihe von Hilfsbefehlen zur Validierung der Grundstruktur des Baums, zu dessen Ausgabe für Debugging-Zwecke sowie zum Durchlaufen des Baums entweder von unten nach oben oder von oben nach unten.

Wird ein Syntaxfehler festgestellt, löst der Befehl stattdessen einen Fehler aus. Dieser Fehler ist eine Tcl-Liste mit vier Elementen, die in der unten aufgeführten Reihenfolge Folgendes enthält:

1. Die Zeichenfolge **pt::rde**, die darauf hinweist, dass es sich um einen Parser-Laufzeitfehler handelt.
2. Die Position des Parsing-Fehlers, angegeben als Zeichenversatz vom Anfang der geparsten Eingabe.
3. Die Position des Parsing-Fehlers, nun als Liste mit zwei Elementen, die die Zeilennummer und die Spalte innerhalb der Zeile enthalten.
4. Eine Menge atomarer Parsing-Ausdrücke, welche die Zeichen und/oder Nichtterminale kodieren, die der Parser an der Fehlerstelle erwartet, aber nicht vorgefunden hat. Informationen zur Spezifikation atomarer Parsing-Ausdrücke finden Sie im Abschnitt **PE-SERIALISIERUNGSFORMAT**.

objectName* parset *text

Diese Methode führt den Parser aus, wobei die Zeichenfolge in *text* als Eingabe verwendet wird. In allen anderen Punkten verhält sie sich wie die oben gezeigte Methode **parse**.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_parser_api
Stichwörter: pt_parser_api, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name
pt_parser_api – Parser-API

Übersicht
package require Tcl 8.5 9
className ?*objectName*?
objectName **destroy**
objectName **parse** *chan*
objectName **parset** *text*

Beschreibung
Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Dokument beschreibt die API für den Zugriff auf die eigentliche Parsing-Funktionalität, die der Grammatik-Interpreter des Pakets **pt::peg::interp** und die von der Anwendung **pt** für die Ergebnisformate **critcl**, **snit** und **oo** erzeugten Parser gemeinsam haben.

Die Zielgruppe sind Personen, die einen Parser für eine von ihnen entwickelte Sprache erstellen und diesen Parser anschließend in einem Tcl-basierten Paket oder einer Anwendung verwenden möchten.

Es befindet sich in der Benutzerschicht der Parser-Tools.

IMAGE: arch_user_pkg

API für Klassen
className ?*objectName*?
Der Klassenbefehl erstellt Parser-Instanzen, d. h. Objekte. Das Ergebnis des Befehls ist der vollqualifizierte Name der Instanz.

Wenn kein *objectName* angegeben wird, generiert die Klasse einen automatischen Namen und verwendet diesen. Wenn der *objectName* zwar angegeben wurde, jedoch nicht vollqualifiziert ist, wird die Instanz im aktuellen Namespace erstellt.

Instanz-API

Alle Parser-Instanzen stellen mindestens die unten aufgeführten Methoden bereit:

***objectName* destroy**
Diese Methode löscht die Parser-Instanz, gibt den gesamten belegten Speicher sowie alle anderen Ressourcen frei und löscht den Instanzbefehl.

Das Ergebnis des Befehls ist die leere Zeichenkette.

objectName* parse *chan
Bei dieser Methode wird der Parser mit dem Inhalt von *chan* als Eingabe ausgeführt (beginnend an der aktuellen Position im Kanal), bis kein weiteres Parsen mehr möglich ist, entweder weil das Parsen abgeschlossen ist oder weil ein Syntaxfehler aufgetreten ist.

Zu beachten ist, dass die Parser-Tools auf Tcl 8.5+ basieren. Mit anderen Worten: Das Kanalargument ist nicht auf Dateien, Sockets usw. beschränkt. Uns steht die volle Leistungsfähigkeit von *reflektierten Kanälen* zur Verfügung.

Es sei außerdem darauf hingewiesen, dass der Parser die Zeichen bei Bedarf aus dem Eingabestrom abruft. Wenn ein durch dieses Paket erstellter Parser im Push-Modus, d. h. ereignisgesteuert betrieben werden muss, ist es erforderlich, auf Tcl 8.6+ umzusteigen und **coroutine::auto** zu verwenden, um ihn in eine Coroutine zu verpacken, wobei **read** entsprechend für den Push-Betrieb angepasst werden muss.

Nach erfolgreicher Ausführung gibt der Befehl einen abstrakten Syntaxbaum (AST) als Ergebnis zurück. Dieser AST hat die im Abschnitt AST-Serialisierungsformat angegebene Form. Als einfache, verschachtelte Tcl-Liste kann er anschließend mit beliebigen Tcl-Befehlen des Benutzers weiterverarbeitet werden, um Transformationen, semantische Prüfungen usw. durchzuführen. Um dies zu erleichtern, bietet das Paket **pt::ast** eine Reihe von Hilfsbefehlen zur Validierung der Grundstruktur des Baums, zu dessen Ausgabe für Debugging-Zwecke sowie zum Durchlaufen des Baums entweder von unten nach oben oder von oben nach unten.

Wenn ein Syntaxfehler auftritt, löst der Befehl stattdessen einen Fehler aus. Dieser Fehler ist eine Tcl-Liste mit vier Elementen, die in der unten aufgeführten Reihenfolge Folgendes enthält:

1. Die Zeichenfolge **pt::rde**, die darauf hinweist, dass es sich um einen Parser-Laufzeitfehler handelt.
2. Die Position des Parsing-Fehlers, angegeben als Zeichenversatz vom Anfang der geparsten Eingabe.
3. Die Position des Parsing-Fehlers, nun als Liste mit zwei Elementen, die die Zeilennummer und die Spalte innerhalb der Zeile enthalten.
4. Eine Menge atomarer Parsing-Ausdrücke, welche die Zeichen und/oder Nichtterminale kodieren, die der Parser an der Fehlerstelle erwartet, aber nicht vorgefunden hat. Informationen zur Spezifikation atomarer Parsing-Ausdrücke finden Sie im Abschnitt PE-Serialisierungsformat.

objectName* parset *text
Diese Methode führt den Parser aus, wobei die Zeichenfolge in *text* als Eingabe verwendet wird. In allen anderen Punkten verhält sie sich wie die oben gezeigte Methode **parse**.

Verwendung

Ein generierter Parser wird wie folgt verwendet

```
    package require the-parser-package ;# Generated by result-formats
    'critcl', 'snit' or 'oo' of 'pt'.
    set parser [the-parser-class]

    set ast [$parser parse $channel]
    ... process the abstract syntax tree ...
```

Wenn ein Grammatik-Interpreter zum Parsen verwendet wird, schleichen sich einige Abweichungen ein

```
    package require the-grammar-package ;# Generated by result-format
    'container' of 'pt'.
    set grammar [the-grammar-class]

    package require pt::peg::interp
    set parser [pt::peg::interp]

    $parser use $grammar

    set ast [$parser parse $channel]
    $parser destroy

    ... process the abstract syntax tree ...
```

AST-Serialisierungsformat

An dieser Stelle legen wir das Format fest, das von den Parser-Tools verwendet wird, um abstrakte Syntaxbäume (ASTs) als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Jeder Knoten in einem AST repräsentiert ein nichtterminales Symbol einer Grammatik sowie den Bereich der Tokens/Zeichen in der Eingabe, den dieses Symbol abdeckt. ASTs enthalten keine terminalen Symbole, d. h. keine Tokens/Zeichen. Diese lassen sich anhand der Position eines Symbols aus der Eingabe rekonstruieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Baum kann zwar mehr als eine reguläre Serialisierung aufweisen, jedoch ist genau nur eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen AST entspricht der Serialisierung seines Wurzelknotens.
1. Die Serialisierung eines beliebigen Knotens ist eine Tcl-Liste, die mindestens drei Elemente enthält.
    1. Das erste Element ist der Name des im Knoten gespeicherten nichtterminalen Symbols.
    2. Das zweite und dritte Element geben die Positionen des ersten und letzten Tokens in dem Token-Strom an, den der Knoten repräsentiert (abdeckt).
        1. Positionen werden als nicht-negative ganzzahlige Offsets vom Anfang des Token-Stroms angegeben, wobei das erste im Strom gefundene Token am Offset 0 (Null) liegt.
        2. Die Endposition muss mindestens so groß sein wie die Startposition.

1. Alle Elemente nach den ersten drei sind die Kindknoten des Knotens, selbst wieder Knoten. Die Serialisierung eines Knotens ohne Kindknoten, also eines Blattknotens, hat daher genau drei Elemente. In der Liste steht der linke Kindknoten zuerst und der rechte zuletzt.

Kanonische Serialisierung

Die kanonische Serialisierung eines abstrakten Syntaxbaums hat das im vorigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Baums eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden Parsing Expression Grammar

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

und der Eingabezeichenkette

```
 120+5
```

Dann sollte ein Parser den folgenden abstrakten Syntaxbaum liefern (mit Ausnahme von Leerzeichen)

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

Oder, grafischer

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                          |            |
|                          +- Digit 2 2 | 0
|                                       |
Expression 0 4 --- Factor 0 4 -+----------------------------- AddOp 3
    3 | +
|                                       |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_container

Stichwörter: pt_peg_container, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::container – PEG-Speicher

Übersicht

package require Tcl 8.5 9
package require snit
package require pt::peg::container ?**1**?
::pt::peg *objectName* ?=|:=|<--|*as*|*deserialize src*?
objectName **destroy**
objectName **clear**
objectName **importer**
objectName **importer** *object*
objectName **exporter**
objectName **exporter** *object*
objectName **=** *source*
objectName **-->** *destination*
objectName **serialize** ?*format*?
objectName **deserialize =** *data* ?*format*?
objectName **deserialize +=** *data* ?*format*?
objectName **start**
objectName **start** *pe*
objectName **nonterminals**
objectName **modes**
objectName **modes** *dict*
objectName **rules**
objectName **rules** *dict*
objectName **add** ?*nt...*?
objectName **remove** ?*nt...*?
objectName **exists** *nt*
objectName **rename** *ntold ntnew*
objectName **mode** *nt*
objectName **mode** *nt mode*
objectName **rule** *nt*
objectName **rule** *nt pe*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte

die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt eine Containerklasse für Parsing Expression Grammars bereit. Jede Instanz speichert eine einzelne Grammatik und erlaubt es, ihre Definition zu bearbeiten und abzufragen.

Es befindet sich im Abschnitt "Speicher" der Kernschicht der Parser-Tools und ist eine der drei Säulen, auf denen die Verwaltung von Parsing Expression Grammars beruht.

IMAGE: arch_core_container

Die beiden anderen Säulen sind, wie oben dargestellt,

1. *PEG-Import* sowie
2. *PEG-Export*.

Die folgenden Pakete stehen damit in Zusammenhang:

pt::rde
: Dieses Paket enthält eine Implementierung von PARAM, einer virtuellen Maschine zum Parsen eines Kanals, die speziell auf die Anforderungen bei der Verarbeitung von PEGs zugeschnitten ist.

pt::peg::interp
: Dieses Paket implementiert einen Interpreter für PEGs auf Basis der von **pt::peg::rde** bereitgestellten virtuellen Maschine.

API für Klassen

Das Paket exportiert die hier beschriebene API.

::pt::peg *objectName* ?=|:=|<--|as|deserialize *src*?
: Der Befehl erstellt ein neues Containerobjekt für eine Parsing Expression Grammar und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück. Die API dieses Objektbefehls wird im Abschnitt Objekt-API beschrieben. Sie kann verwendet werden, um verschiedene Operationen auf dem Objekt auszuführen.

  Der neue Container ist leer, wenn kein *src* angegeben wird. Andernfalls enthält er eine Kopie der in *src* enthaltenen Grammatik. Alle Operatoren mit Ausnahme von **deserialize** interpretieren *src* als Container-Objektbefehl. Der Operator **deserialize** interpretiert *src* stattdessen als die Serialisierung einer Parsing Expression Grammar, wie im Abschnitt PEG-Serialisierungsformat festgelegt.

  Eine leere Grammatik enthält keine nichtterminalen Symbole, und der Startausdruck ist der leere Ausdruck, d. h. Epsilon. Sie ist *gültig*, aber nicht *sinnvoll*.

Objekt-API

Alle von diesem Paket erstellten Objekte stellen die folgenden Methoden zur Bearbeitung und Abfrage ihres Inhalts bereit:

***objectName* destroy**
: Diese Methode zerstört das Objekt, gibt den gesamten belegten Speicher frei und löscht den zugehörigen Objektbefehl.

***objectName* clear**
: Diese Methode setzt das Objekt so zurück, dass es die leere Grammatik enthält. Das Objekt

selbst wird dabei *nicht* gelöscht.

***objectName* importer**
Diese Methode gibt das derzeit mit dem Container verknüpfte Import-Manager-Objekt zurück, sofern vorhanden.

objectName* importer *object
Diese Methode fügt das *object* als Importmanager dem Container hinzu und gibt es als Ergebnis des Befehls zurück. Beachten Sie, dass das *object nicht* in den Besitz des Containers übergeht. Das heißt, die Löschung des Containers führt *nicht* zur Löschung des *object*.

Es wird erwartet, dass *object* eine Methode namens **import text** bereitstellt, die einen Text und einen Formatnamen entgegennimmt und die kanonische Serialisierung des im Text enthaltenen Inhaltsverzeichnisses unter der Annahme des angegebenen Formats zurückgibt.

***objectName* exporter**
Diese Methode gibt das derzeit mit dem Container verknüpfte Exportmanager-Objekt zurück, sofern vorhanden.

objectName* exporter *object
Diese Methode fügt das *object* als Exportmanager dem Container hinzu und gibt es als Ergebnis des Befehls zurück. Beachten Sie, dass das *object nicht* in den Besitz des Containers übergeht. Das heißt, die Löschung des Containers führt *nicht* zur Löschung des *object*.

Es wird erwartet, dass das *object* eine Methode namens **export object** bereitstellt, die den Container und einen Formatnamen entgegennimmt und ein im Container gespeichertes, als Text kodiertes Inhaltsverzeichnis im angegebenen Format zurückgibt. Es wird ferner erwartet, dass das *object* die Methode **serialize** des Containers verwendet, um die Serialisierung des Inhaltsverzeichnisses zu holen, aus der der Text erzeugt werden soll.

objectName = source
Diese Methode weist uns den Inhalt des PEG-Objekts *source* zu und überschreibt dabei die bestehenden Definitionen. Dies ist der Zuweisungsoperator für Grammatiken.

Diese Operation entspricht im Grunde genommen

```
    objectName deserialize = [source serialize]
```

objectName --> destination
Diese Methode weist dem PEG-Objekt *destination* unsere Inhalte zu und überschreibt dabei die bestehenden Definitionen. Dies ist der umgekehrte Zuweisungsoperator für Grammatiken.

Diese Operation entspricht im Grunde genommen

```
    destination deserialize = [objectName serialize]
```

***objectName* serialize ?*format*?**
Diese Methode gibt unsere Grammatik in einer textuellen Form zurück, die für die Übertragung, die dauerhafte Speicherung usw. geeignet ist. Wenn kein *format* angegeben wird, entspricht das zurückgegebene Ergebnis der kanonischen Serialisierung der Grammatik, wie im Abschnitt PEG-Serialisierungsformat beschrieben.

Andernfalls verwendet das Objekt den angehängten Export-Manager, um die Daten in das

angegebene Format zu konvertieren. In diesem Fall schlägt die Methode mit einer Fehlermeldung fehl, wenn dem Container kein Export-Manager angehängt ist.

***objectName* deserialize = *data* ?*format*?**

Dies ist die ergänzende Methode zu **serialize**. Sie ersetzt die aktuelle Definition durch die in *data* enthaltene Grammatik. Wurde kein *format* angegeben, wird davon ausgegangen, dass es sich um die reguläre Serialisierung einer Grammatik handelt, wie im Abschnitt PEG-Serialisierungsformat beschrieben.

Andernfalls verwendet das Objekt den angehängten Import-Manager, um die Daten aus dem angegebenen Format in eine für das Objekt verarbeitbare Serialisierung zu konvertieren. In diesem Fall schlägt die Methode mit einer Fehlermeldung fehl, wenn dem Container kein Importmanager zugeordnet ist.

Das Ergebnis der Methode ist die leere Zeichenkette.

***objectName* deserialize += *data* ?*format*?**

Diese Methode verhält sich im Wesentlichen wie **deserialize =**, mit dem Unterschied, dass sie die Grammatik in *data* mit deren Inhalt zusammenführt, anstatt sie zu ersetzen. Die Methode bricht mit einer Fehlermeldung ab und lässt die Grammatik unverändert, wenn eine Zusammenführung nicht möglich ist, d. h. wenn dadurch eine ungültige Grammatik entstehen würde.

Das Ergebnis der Methode ist die leere Zeichenkette.

***objectName* start**

Diese Methode gibt den aktuellen Startausdruck der Grammatik zurück.

objectName* start *pe

Diese Methode definiert den *Startausdruck* der Grammatik. Sie ersetzt den aktuellen Startausdruck durch den Parsing-Ausdruck *pe* und gibt den neuen Startausdruck zurück.

Die Methode bricht mit einer Fehlermeldung ab und lässt die Grammatik unverändert, falls *pe* keinen gültigen Parsing-Ausdruck enthält, wie im Abschnitt PE-Serialisierungsformat festgelegt.

***objectName* nonterminals**

Diese Methode gibt die Menge aller der Grammatik bekannten nichtterminalen Symbole zurück.

***objectName* modes**

Diese Methode gibt ein Dictionary zurück, das die Menge aller der Grammatik bekannten nichtterminalen Symbole ihren semantischen Modi zuordnet.

objectName* modes *dict

Diese Methode nimmt ein Dictionary entgegen, das eine Menge von der Grammatik bekannter nichtterminaler Symbole ihren semantischen Modi zuordnet, und gibt das neue vollständige Dictionary der nichtterminalen Symbole zu semantischen Modi zurück.

Die Methode schlägt mit einem Fehler fehl, wenn eines der nichtterminalen Symbole im Dictionary der Grammatik unbekannt ist oder die leere Zeichenkette darstellt, d. h. ein ungültiges nichtterminales Symbol ist, oder wenn einer der gewählten *mode* nicht zu den

zulässigen Werten gehört.

***objectName* rules**

Diese Methode gibt ein Dictionary zurück, das die Menge aller der Grammatik bekannten nichtterminalen Symbole ihren Parsing-Ausdrücken (Rechtsseiten) zuordnet.

objectName* rules *dict

Diese Methode nimmt ein Dictionary entgegen, das eine Menge der der Grammatik bekannter nichtterminaler Symbole ihren Parsing-Ausdrücken (Rechtsseiten) zuordnet, und gibt das neue vollständige Dictionary der nichtterminalen Symbole zu Parsing-Ausdrücken zurück.

Die Methode schlägt mit einer Fehlermeldung fehl, wenn eines der nichtterminalen Symbole im Dictionary der Grammatik nicht bekannt ist, wenn die leere Zeichenkette – d. h. ein ungültiges nichtterminales Symbol – enthalten ist oder wenn einer der ausgewählten Parsing-Ausdrücke kein gültiger Parsing-Ausdruck im Sinne des Abschnitts PE-Serialisierungsformat ist.

***objectName* add ?*nt*...?**

Diese Methode fügt die nichtterminalen Symbole *nt* usw. zur Grammatik hinzu und definiert dafür den standardmäßigen semantischen Modus sowie den Ausdruck (**value** bzw. **epsilon**). Die Methode gibt als Ergebnis die leere Zeichenkette zurück.

Die Methode schlägt mit einer Fehlermeldung fehl und lässt die Grammatik unverändert, wenn eines der nichtterminalen Symbole entweder bereits in unserer Grammatik definiert ist oder der leeren Zeichenkette entspricht (ein ungültiges nichtterminales Symbol).

Die Methode führt keine Aktion aus, wenn kein Symbol als Argument festgelegt wurde.

***objectName* remove ?*nt*...?**

Diese Methode entfernt die genannten nichtterminalen Symbole *nt* usw. aus der Menge der unserer Grammatik bekannten nichtterminalen Symbole. Die Methode gibt als Ergebnis die leere Zeichenkette zurück.

Die Methode schlägt mit einer Fehlermeldung fehl und lässt die Grammatik unverändert, wenn eines der nichtterminalen Symbole der Grammatik nicht bekannt ist oder es sich um die leere Zeichenkette handelt, d. h. um ein ungültiges nichtterminales Symbol.

objectName* exists *nt

Diese Methode prüft, ob das nichtterminale Symbol *nt* in unserer Grammatik bekannt ist oder nicht. Der Wert ist boolesch. Er wird auf **true** gesetzt, wenn *nt* bekannt ist, andernfalls auf **false**.

Die Methode schlägt mit einer Fehlermeldung fehl, wenn *nt* die leere Zeichenkette ist, d. h. ein ungültiges nichtterminales Symbol.

objectName* rename *ntold ntnew

Diese Methode benennt das nichtterminale Symbol *ntold* in *ntnew* um. Die Methode gibt als Ergebnis die leere Zeichenkette zurück.

Die Methode schlägt mit einer Fehlermeldung fehl und lässt die Grammatik unverändert, wenn entweder *ntold* der Grammatik nicht bekannt ist, *ntnew* bereits bekannt ist oder eines

der beiden Symbole die leere Zeichenkette ist, d. h. ein ungültiges nichtterminales Symbol.

objectName* mode *nt

Diese Methode gibt den aktuellen semantischen Modus für das nichtterminale Symbol *nt* zurück.

Die Methode schlägt mit einer Fehlermeldung fehl, wenn *nt* in der Grammatik nicht bekannt ist oder es sich um die leere Zeichenkette handelt, d. h. um ein ungültiges nichtterminales Symbol.

objectName* mode *nt mode

Diese Methode setzt den semantischen Modus für das nichtterminale Symbol *nt* fest und gibt den neuen Modus zurück. Die Methode schlägt mit einer Fehlermeldung fehl, wenn *nt* der Grammatik nicht bekannt ist, es sich um die leere Zeichenkette handelt – d. h. um ein ungültiges nichtterminales Symbol – oder der gewählte *mode* nicht zu den zulässigen Werten gehört.

Die folgenden Modi sind zulässig:

value
: Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf
: Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
: Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

objectName* rule *nt

Diese Methode gibt den aktuellen Parsing-Ausdruck (rechte Seite) für das nichtterminale Symbol *nt* zurück.

Die Methode schlägt mit einer Fehlermeldung fehl, wenn *nt* der Grammatik nicht bekannt ist oder es sich um die leere Zeichenkette handelt, d. h. um ein ungültiges nichtterminales Symbol.

objectName* rule *nt pe

Diese Methode setzt den Parsing-Ausdruck (rechte Seite) des Nichtterminals *nt* auf *pe* und gibt den neuen Parsing-Ausdruck zurück.

Die Methode schlägt mit einer Fehlermeldung fehl, wenn *nt* der Grammatik nicht bekannt ist oder es sich um die leere Zeichenkette handelt, d. h. um ein ungültiges nichtterminales Symbol, oder wenn *pe* keinen gültigen Parsing-Ausdruck enthält, wie im Abschnitt PE-Serialisierungsformat festgelegt.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression

Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf

der Grundlage des eingebauten Tcl-Befehls **string is**.

13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_container_peg

Stichwörter: pt_peg_container_peg, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::container::peg – PEG-Speicher. Vorgefertigte Spezifikation einer PEG-Grammatik

Übersicht

package require Tcl 8.5 9
package require snit
package require pt::peg::container::peg ?**1**?
package require pt::peg::container

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt einen Untertyp von **pt::peg::container** bereit, der bereits mit einer Parsing Expression Grammar geladen ist, die ein Textformat für Parsing Expression Grammars beschreibt.

Der Untertyp stellt genau dieselbe API wie **pt::peg::container** bereit. Statt den Inhalt zu wiederholen, wird der Leser gebeten, das genannte Dokument zu lesen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_export

Stichwörter: pt_peg_export, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::export – PEG-Export

Übersicht

package require Tcl 8.5 9
package require snit
package require struct::map
package require pt::peg
package require pluginmgr
package require pt::peg::export ?**1.0.2**?
::pt::peg::export *objectName*
objectName method ?*arg arg* ...?
objectName **destroy**
objectName **export serial** *serial* ?*format*?
objectName **export object** *object* ?*format*?
objectName **configuration names**
objectName **configuration get**
objectName **configuration set** *name* ?*value*?
objectName **configuration unset** *pattern...*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt einen Manager für Parsing Expression Grammars bereit. Jede Instanz verwaltet eine Menge von Plugins für den Export dieser Grammatiken in andere Formate, also für ihre Umwandlung zum Beispiel nach *nroff*, *HTML* und so weiter.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und ist eine der drei Säulen, auf denen die Verwaltung von Parsing Expression Grammars beruht.

IMAGE: arch_core_export

Die beiden anderen Säulen sind, wie oben dargestellt,

1. *PEG-Import* sowie
2. *PEG-Speicherung*.

Informationen zur Datenstruktur, die den Hauptteil der Eingaben für die von diesem Paket bereitgestellten Manager-Objekte bildet, finden Sie im Abschnitt PEG-Serialisierungsformat.

Das Plugin-System dieser Klasse basiert auf dem Paket **pluginmgr** und ist so konfiguriert, dass es nach Plugins sucht, die

1. die Umgebungsvariable **GRAMMAR_PEG_EXPORT_PLUGINS**,
2. die Umgebungsvariable **GRAMMAR_PEG_PLUGINS**,
3. die Umgebungsvariable **GRAMMAR_PLUGINS**,
4. der Pfad "*~/.grammar/peg/export/plugin*"
5. der Pfad "*~/.grammar/peg/plugin*"
6. der Pfad "*~/.grammar/plugin*"
7. der Pfad "*~/.grammar/peg/export/plugins*"
8. der Pfad "*~/.grammar/peg/plugins*"
9. der Pfad "*~/.grammar/plugins*"
10. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\EXPORT\PLUGINS"
11. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\PLUGINS"
12. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PLUGINS".

Die letzten drei werden nur verwendet, wenn das Paket auf einem Rechner mit dem Betriebssystem Windows(tm) ausgeführt wird.

Das gesamte System wird mit drei vordefinierten Export-Plugins ausgeliefert, nämlich

container
: Weitere Informationen finden Sie unter *PEG Export Plugin. To CONTAINER format*.

json
: Weitere Informationen finden Sie unter *PEG Export Plugin. To JSON format*.

peg
: Weitere Informationen finden Sie unter *PEG Export Plugin. To PEG format*.

Für Leser, die ein eigenes Export-Plugin für ein bestimmtes Format erstellen möchten, d. h. *Plugin-Entwickler*, ist das Lesen und Verstehen der Spezifikation der *Parser Tools Export API* unerlässlich, da diese die Interaktion zwischen diesem Paket und seinen Plugins ausführlich dokumentiert.

API

Paketbefehle

::pt::peg::export *objectName*

Dieser Befehl erstellt ein neues Export-Manager-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird unter dem aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::pt::peg::export** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt, für das sie aufgerufen wird.

***objectName* export serial *serial* ?*format*?**

Diese Methode nimmt die kanonische Serialisierung einer in *serial* gespeicherten Parsing Expression Grammar und wandelt sie mit dem Export-Plugin für das angegebene *format* in dieses Format um. Wenn für das Format kein Plugin gefunden werden konnte, schlägt der Vorgang mit einer Fehlermeldung fehl. Die durch den Konvertierungsprozess erzeugte Zeichenkette wird als Ergebnis dieser Methode zurückgegeben.

Wenn kein *format* angegeben wird, verwendet die Methode standardmäßig **text**.

Die Spezifikation dessen, was eine *kanonische* Serialisierung ist, finden Sie im Abschnitt PEG-Serialisierungsformat.

Das Plugin muss der in der Spezifikation *Parser Tools Export API* dokumentierten Schnittstelle entsprechen.

***objectName* export object *object* ?*format*?**

Diese Methode ist eine praktische Hülle um die im vorherigen Punkt beschriebene Methode export serial. Sie setzt voraus, dass *object* ein Objektbefehl ist, der eine **serialize**-Methode unterstützt, welche die kanonische Serialisierung einer Parsing Expression Grammar zurückgibt. Sie ruft diese Methode auf, übergibt das Ergebnis an export serial und gibt die resultierende Zeichenkette als eigenes Ergebnis zurück.

***objectName* configuration names**

Diese Methode gibt eine Liste zurück, die die Namen aller dem Objekt derzeit bekannten Konfigurationsoptionen enthält.

***objectName* configuration get**

Diese Methode gibt ein Dictionary zurück, das die Namen und Werte aller dem Objekt derzeit bekannten Konfigurationsoptionen enthält.

***objectName* configuration set *name* ?*value*?**

Diese Methode setzt die Konfigurationsoption *name* auf den angegebenen *value* und gibt den neuen Wert der Option zurück.

Wird kein Wert angegeben, wird lediglich der aktuelle Wert zurückgegeben, ohne ihn zu

ändern.

Beachten Sie, dass diese Konfigurationsoptionen und ihre Werte bei der Durchführung des eigentlichen Exports lediglich an ein Plugin übergeben werden. Die Validierung erfolgt durch das Plugin, nicht durch den Manager.

***objectName* configuration unset *pattern*...**

Diese Methode hebt alle Konfigurationsoptionen auf, die den angegebenen *pattern*s entsprechen. Wird kein *pattern* angegeben, werden alle derzeit definierten Konfigurationsoptionen aufgehoben.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode

      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value

      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf

      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void

Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
```

```
                                    mode value}
        MulOp      {is {/ {t *} {t /}}
                                    mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                    mode value}
        Sign       {is {/ {t -} {t +}}
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.

1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden

Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_export_container

Stichwörter: pt_peg_export_container, CONTAINER, EBNF, LL(k), PEG, TDPL, context-free languages, export, expression, grammar, matching, parser, parsing expression, parsing expression grammar, plugin, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::export::container – PEG-Export-Plugin. CONTAINER-Format schreiben

Übersicht

package require Tcl 8.5 9
package require pt::peg::export::container ?**1**?
package require pt::peg::to::container

export *serial configuration*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert das Export-Plugin für Parsing Expression Grammars zur Erzeugung von CONTAINER-Markup.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und ist für die Verwendung durch **pt::peg::export**, den Exportmanager, vorgesehen, der zwischen diesem und der entsprechenden Kernkonvertierungsfunktionalität, die von pt::peg::to::container bereitgestellt wird, angesiedelt ist.

IMAGE: arch_core_eplugins

Zwar ist die direkte Verwendung dieses Pakets mit einem herkömmlichen Interpreter möglich, doch wird davon dringend abgeraten, da hierfür eine Reihe von Umwegen erforderlich ist, um die erwartete Umgebung bereitzustellen. Die richtige Vorgehensweise bei der Nutzung dieser Funktionalität hängt von der jeweiligen Situation ab:

1. In einer nicht vertrauenswürdigen Umgebung erfolgt der ordnungsgemäße Zugriff über das Paket **pt::peg::export** und die darin bereitgestellten Exportmanager-Objekte.
2. In einer vertrauenswürdigen Umgebung wird einfach das Paket pt::peg::to::container verwendet und direkt auf die zentralen Konvertierungsfunktionen zugegriffen.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Plugin-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

export *serial configuration*

Dieser Befehl nimmt die kanonische Serialisierung einer Parsing Expression Grammar entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, sowie die *configuration*, ein Dictionary, und generiert CONTAINER-Markup, das die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Konfiguration

Das CONTAINER-Export-Plugin erkennt die folgenden Konfigurationsvariablen und passt sein Verhalten entsprechend deren Vorgaben an.

enum *mode*

Der Wert dieser Konfigurationsvariablen legt fest, welche Methoden von **pt::peg**-Instanzen das Plugin zur Festlegung der Grammatik verwendet. Es gibt zwei zulässige Werte

bulk

In diesem Modus werden die Methoden **start**, **add**, **modes** und **rules** verwendet, um die Grammatik pauschal festzulegen, d. h. als eine Menge von nichtterminalen Symbolen sowie zwei Dictionarys, die die Symbole ihren semantischen Modi und Parsing-Ausdrücken zuordnen.

Dieser Modus ist der Standardmodus.

incremental
In diesem Modus werden die Methoden **start**, **add**, **mode** und **rule** verwendet, um die Grammatik stückweise festzulegen, wobei jedes Nichtterminal über einen eigenen Block von Definitionsbefehlen verfügt.

string *template*
Wenn diese Konfigurationsvariable gesetzt ist, wird davon ausgegangen, dass sie eine Zeichenkette enthält, in die der generierte Code und andere Konfigurationsdaten eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Stellen mit den folgenden Platzhaltern angegeben werden:
@user@
Ist durch den Wert der Konfigurationsvariablen **user** zu ersetzen.
@format@
Ist durch die Konstante **CONTAINER** zu ersetzen.
@file@
Ist durch den Wert der Konfigurationsvariablen **file** zu ersetzen.
@name@
Ist durch den Wert der Konfigurationsvariablen **name** zu ersetzen.
@mode@
Ist durch den Wert der Konfigurationsvariablen **mode** zu ersetzen.
@code@
Wird durch den generierten Code ersetzt.

Wenn diese Konfigurationsvariable nicht gesetzt oder leer ist, greift das Plugin auf eine Standardvorlage zurück, die als **@code@** definiert ist.

Hinweis: Dieses Plugin ignoriert je nach gewählter Vorlage die Standard-Konfigurationsvariablen **user**, **format**, **file** und ihre Werte.

Der Wert der Standardkonfigurationsvariablen **name** wird, wenn er festgelegt ist, als Name der Grammatik in der Ausgabe verwendet. Andernfalls greift das Plugin auf den Standardnamen **a_pe_grammar** zurück.

Grammatik-Container

Das Format **container** ist eine weitere Form, Parsing Expression Grammars zu beschreiben. Zwar sind Daten in diesem Format ausführbar, sie stellen jedoch keinen Parser für die Grammatik dar. Sie müssen stets in Verbindung mit dem Paket **pt::peg::interp**, einem Grammatik-Interpreter, verwendet werden.

Das Format stellt Grammatiken mittels einer **snit::type** dar, d. h. einer Klasse, deren Instanzen API-kompatibel zu den Instanzen des Pakets **pt::peg::container** sind und die mit der jeweiligen Grammatik vorbelegt sind.

Es gibt keine direkte formale Spezifikation, die über das oben Gesagte hinausgeht.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Eine mögliche Container-Serialisierung hierfür lautet

```
snit::type a_pe_grammar {
    constructor {} {
        install myg using pt::peg::container ${selfns}::G
        $myg start {n Expression}
        $myg add   AddOp Digit Expression Factor MulOp Number Sign Term
        $myg modes {
            AddOp      value
            Digit      value
            Expression value
            Factor     value
            MulOp      value
            Number     value
            Sign       value
            Term       value
        }
        $myg rules {
            AddOp      {/ {t -} {t +}}
            Digit      {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}
            Expression {/ {x {t \50} {n Expression} {t \51}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}}
            Factor     {x {n Term} {* {x {n AddOp} {n Term}}}}
            MulOp      {/ {t *} {t /}}
            Number     {x {? {n Sign}} {+ {n Digit}}}
            Sign       {/ {t -} {t +}}
            Term       {n Number}
        }
        return
    }

    component myg
    delegate method * to myg
}
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole

festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.

3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_export_json

Stichwörter: pt_peg_export_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, export,

expression, grammar, matching, parser, parsing expression, parsing expression grammar, plugin , push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::export::json – PEG-Export-Plugin. JSON-Format schreiben

Übersicht

package require Tcl 8.5 9
package require pt::peg::export::json ?**1**?
package require pt::peg::to::json
export *serial configuration*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert das Export-Plugin für Parsing Expression Grammars zur Erzeugung von JSON-Markup.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und ist für die Verwendung durch **pt::peg::export**, den Exportmanager, vorgesehen, der zwischen diesem und der entsprechenden Kernkonvertierungsfunktionalität, die von pt::peg::to::json bereitgestellt wird, angesiedelt ist.

IMAGE: arch_core_eplugins

Zwar ist die direkte Verwendung dieses Pakets mit einem herkömmlichen Interpreter möglich, doch wird davon dringend abgeraten, da hierfür eine Reihe von Umwegen erforderlich ist, um die erwartete Umgebung bereitzustellen. Die richtige Vorgehensweise bei der Nutzung dieser Funktionalität hängt von der jeweiligen Situation ab:

1. In einer nicht vertrauenswürdigen Umgebung erfolgt der ordnungsgemäße Zugriff über das Paket **pt::peg::export** und die darin bereitgestellten Exportmanager-Objekte.
2. In einer vertrauenswürdigen Umgebung wird einfach das Paket pt::peg::to::json verwendet und direkt auf die zentralen Konvertierungsfunktionen zugegriffen.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Plugin-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung einer Parsing Expression Grammar entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, sowie die *configuration*, ein Dictionary, und generiert eine JSON-Kodierung, die die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Konfiguration

Das JSON-Export-Plugin erkennt die folgenden Konfigurationsvariablen und ändert sein Verhalten

so, wie sie es vorgeben.

boolean *indented*
: Ist dieses Flag gesetzt, bricht das Plugin den erzeugten JSON-Code auf mehrere Zeilen um und rückt ihn entsprechend seinem inneren Aufbau ein, mit jedem Schlüssel eines Dictionarys in einer eigenen Zeile.

  Ist dieses Flag nicht gesetzt (die Vorgabe), wird das ganze JSON-Objekt in eine einzige Zeile geschrieben, mit möglichst wenig Abstand zwischen allen Elementen.

boolean *aligned*
: Ist dieses Flag gesetzt, sorgt der Generator dafür, dass die Werte der Schlüssel in einem Dictionary vertikal zueinander ausgerichtet sind, um einen ansprechenden Tabelleneffekt zu erzielen. Damit dies funktioniert, muss außerdem das Flag **indented** gesetzt sein.

  Wenn dieses Flag nicht gesetzt ist (Standardeinstellung), wird die Ausgabe gemäß dem Wert von **indented** formatiert, ohne dass versucht wird, die Werte der Dictionary-Schlüssel auszurichten.

Hinweis: Dieses Plugin ignoriert die Standard-Konfigurationsvariablen **user**, **format**, **file** und **name** sowie deren Werte.

JSON-Austauschformat für Grammatiken

Das Format **json** für Parsing Expression Grammars wurde als Datenaustauschformat geschrieben, das nicht an Tcl gebunden ist. Es wurde definiert, um den Austausch von Grammatiken mit PackRat/PEG-basierten Parsergeneratoren für andere Sprachen zu ermöglichen.

Dies wird durch die folgenden Regeln formal festgelegt:

1. Das JSON-Format eines jeden PEG ist ein JSON-Objekt.
2. Dieses Objekt enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
3. Der Inhalt der Grammatik ist ein JSON-Objekt, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   : Der Wert ist ein JSON-Objekt, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
     1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
     2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
     3. Der Wert für jedes Symbol ist selbst ein JSON-Objekt. Die relevanten Schlüssel und ihre Werte in diesem Dictionary lauten

        is
        : Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des Parsing-Ausdrucks enthält, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

        mode
        : Der Wert ist ein JSON-Wert mit einem von drei Werten, die festlegen, wie ein Parser den durch das Symbol erzeugten semantischen Wert verarbeiten soll.

          value

Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf
Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des
    Start-Parsing-Ausdrucks der Grammatik enthält, wie im Abschnitt
    [PE-Serialisierungsformat] beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Als Anmerkung für den fortgeschrittenen Leser: Dies entspricht im Wesentlichen der Tcl-Serialisierung von PE-Grammatiken, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist, mit dem Unterschied, dass die Tcl-Dictionaries und -Listen dieses Formats auf JSON-Objekte und -Arrays abgebildet werden. Lediglich die Parsing-Ausdrücke selbst werden nicht weiter übersetzt, sondern als JSON-Zeichenketten beibehalten, die eine verschachtelte Tcl-Liste enthalten, und auch für das JSON gibt es kein Konzept der Kanonizität.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Eine JSON-Serialisierung dafür lautet

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
```

```
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

und eine Tcl-Serialisierung davon lautet

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
```

```
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

Die Ähnlichkeit des Letzteren mit JSON dürfte ziemlich offensichtlich sein.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   : Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
     1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
     2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
     3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

        is
        : Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

        mode
        : Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

          value
          : Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

          leaf

Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung
Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.

3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_export_peg

Stichwörter: pt_peg_export_peg, EBNF, LL(k), PEG, TDPL, context-free languages, export,

expression, grammar, matching, parser, parsing expression, parsing expression grammar, plugin , push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::export::peg – PEG-Export-Plugin. PEG-Format schreiben

Übersicht

package require Tcl 8.5 9
package require pt::peg::export::peg ?**1**?
package require pt::peg::to::peg
export *serial configuration*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert das Export-Plugin für die Parsing Expression Grammar zur Erzeugung von PEG-Markup.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und ist für die Verwendung durch **pt::peg::export**, den Exportmanager, vorgesehen, der zwischen diesem und der entsprechenden Kernkonvertierungsfunktionalität, die von pt::peg::to::peg bereitgestellt wird, angesiedelt ist.

IMAGE: arch_core_eplugins

Zwar ist die direkte Verwendung dieses Pakets mit einem herkömmlichen Interpreter möglich, doch wird davon dringend abgeraten, da hierfür eine Reihe von Umwegen erforderlich ist, um die erwartete Umgebung bereitzustellen. Die richtige Vorgehensweise bei der Nutzung dieser Funktionalität hängt von der jeweiligen Situation ab:

1. In einer nicht vertrauenswürdigen Umgebung erfolgt der ordnungsgemäße Zugriff über das Paket **pt::peg::export** und die darin bereitgestellten Exportmanager-Objekte.
2. In einer vertrauenswürdigen Umgebung wird einfach das Paket pt::peg::to::peg verwendet und direkt auf die zentralen Konvertierungsfunktionen zugegriffen.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Plugin-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

export *serial configuration*
: Dieser Befehl nimmt die kanonische Serialisierung einer Parsing Expression Grammar entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, sowie die *configuration*, ein Dictionary, und generiert ein PEG-Markup, das die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Konfiguration

Das PEG-Export-Plugin erkennt die folgenden Variablen der Konfiguration und passt sein

Verhalten entsprechend der dort festgelegten Einstellungen an.

string *template*
Wenn diese Konfigurationsvariable gesetzt ist, wird davon ausgegangen, dass sie eine Zeichenkette enthält, in die der generierte Text und andere Konfigurationsdaten eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Stellen mit den folgenden Platzhaltern angegeben werden:

@user@
Ist durch den Wert der Konfigurationsvariablen **user** zu ersetzen.

@format@
Ist durch die Konstante **PEG** zu ersetzen.

@file@
Ist durch den Wert der Konfigurationsvariablen **file** zu ersetzen.

@name@
Ist durch den Wert der Konfigurationsvariablen **name** zu ersetzen.

@code@
Wird durch den generierten Text ersetzt.

Wenn diese Konfigurationsvariable nicht gesetzt oder leer ist, greift das Plugin auf eine Standardvorlage zurück, die als **@code@** definiert ist.

Hinweis: Dieses Plugin ignoriert je nach gewählter Vorlage die Standard-Konfigurationsvariablen **user**, **format**, **file** und ihre Werte.

Der Wert der Standardkonfigurationsvariablen **name** wird, wenn er festgelegt ist, als Name der Grammatik in der Ausgabe verwendet. Andernfalls greift das Plugin auf den Standardnamen **a_pe_grammar** zurück.

PEG-Spezifikationssprache

peg, eine Sprache zur Spezifikation von Parsing-Expression-Grammatiken, soll sowohl für Menschen lesbar als auch schreibbar sein, dabei jedoch streng genug, um eine maschinelle Verarbeitung zu ermöglichen. Wie jede Computersprache. Sie wurde definiert, um das Verfassen einer Grammatik-Spezifikation zu vereinfachen – etwas, wofür sich die anderen in den Parser-Tools vorhandenen Formate nicht eignen.

Sie wird formal durch die unten gezeigte Grammatik festgelegt, die in sich selbst geschrieben ist. Eine Einführung in die Sprache steht im *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
```

```
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'          ;
void:   OPENB           <- "["          ;
```

```
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Beispiel

Unser Beispiel legt die Grammatik eines einfachen Taschenrechners mit vier Grundrechenarten fest.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Mit höherwertigen Merkmalen der Notation, also den Zeichenklassen (vordefiniert und selbst definiert), lässt sich dieses Beispiel so umschreiben:

```
PEG calculator (Expression)
    Sign       <- [-+]                                         ;
    Number     <- Sign? <ddigit>+                              ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- [*/]                      ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- [-+]                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

mode

Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

value

Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf

Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void

Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
```

```
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes

alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein

Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_from_container

Stichwörter: pt_peg_from_container, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::from::container – PEG-Konvertierung. Aus dem CONTAINER-Format

Übersicht

package require Tcl 8.5 9

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket existiert nicht. Es besteht kein Bedarf dafür. Das CONTAINER-Format für Parsing Expression Grammars ist ein Stück Tcl-Code, das nach dem Einlesen mit source eine Klasse bereitstellt, deren Instanzen die zu importierende Grammatik bereits geladen haben. Man könnte es auch so betrachten: Die CONTAINER-Ausgabe ist ihr eigenes Importpaket.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_from_json

Stichwörter: pt_peg_from_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, conversion , expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::from::json – PEG-Konvertierung. JSON-Format lesen

Übersicht

package require Tcl 8.5 9
package require pt::peg::from::json ?**1.1**?
package require pt::peg
package require json
pt::peg::from::json convert *text*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter von JSON-Markup in Parsing Expression Grammars.

Es befindet sich im Abschnitt "Import" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::import** bereitgestellten Import-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Import-Plugin **pt::peg::import::json**, das zwischen Konverter und Import-Manager angesiedelt ist.

IMAGE: arch_core_iplugins

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Import API* zu finden ist.

pt::peg::from::json convert *text*
: Dieser Befehl nimmt die JSON-Markup-Kodierung einer Parsing Expression Grammar, die in *text* enthalten ist, und generiert die kanonische Serialisierung dieser Grammatik, wie in Abschnitt PEG-Serialisierungsformat festgelegt. Der erzeugte Wert wird anschließend als Ergebnis des Befehls zurückgegeben.

JSON-Austauschformat für Grammatiken

Das Format **json** für Parsing Expression Grammars wurde als Datenaustauschformat geschrieben, das nicht an Tcl gebunden ist. Es wurde definiert, um den Austausch von Grammatiken mit PackRat/PEG-basierten Parsergeneratoren für andere Sprachen zu ermöglichen.

Dies wird durch die folgenden Regeln formal festgelegt:

1. Das JSON-Format eines jeden PEG ist ein JSON-Objekt.
2. Dieses Objekt enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
3. Der Inhalt der Grammatik ist ein JSON-Objekt, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten
   rules
   : Der Wert ist ein JSON-Objekt, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
3. Der Wert für jedes Symbol ist selbst ein JSON-Objekt. Die relevanten Schlüssel und ihre Werte in diesem Dictionary lauten

   is
   Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des Parsing-Ausdrucks enthält, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

   mode
   Der Wert ist ein JSON-Wert mit einem von drei Werten, die festlegen, wie ein Parser den durch das Symbol erzeugten semantischen Wert verarbeiten soll.

   value
   Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

   leaf
   Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

   void
   Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des
    Start-Parsing-Ausdrucks der Grammatik enthält, wie im Abschnitt
    [PE-Serialisierungsformat] beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Als Anmerkung für den fortgeschrittenen Leser: Dies entspricht im Wesentlichen der Tcl-Serialisierung von PE-Grammatiken, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist, mit dem Unterschied, dass die Tcl-Dictionaries und -Listen dieses Formats auf JSON-Objekte und -Arrays abgebildet werden. Lediglich die Parsing-Ausdrücke selbst werden nicht weiter übersetzt, sondern als JSON-Zeichenketten beibehalten, die eine verschachtelte Tcl-Liste enthalten, und auch für das JSON gibt es kein Konzept der Kanonizität.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
```

```
    Expression <- Term (AddOp Term)*                                ;
    MulOp      <- '*' / '/'                                         ;
    Term       <- Factor (MulOp Factor)*                            ;
    AddOp      <- '+'/'-'                                           ;
    Factor     <- '(' Expression ')' / Number                       ;
END;
```

Eine JSON-Serialisierung dafür lautet

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
```

```
        "start" : "n Expression"
    }
}
```

und eine Tcl-Serialisierung davon lautet

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

Die Ähnlichkeit des Letzteren mit JSON dürfte ziemlich offensichtlich sein.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.

3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

   is
   Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

   mode
   Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

   value
   Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

   leaf
   Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

   void
   Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.

1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die

Terminalzeichenkette **x**.

17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright
```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_from_peg
Stichwörter: pt_peg_from_peg, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name
pt::peg::from::peg – PEG-Konvertierung. PEG-Format lesen

Übersicht
package require Tcl 8.5 9
package require pt::peg::from::peg ?**1.0.4**?
pt::peg::from::peg convert *text*

Beschreibung
Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter von PEG-Markup in Parsing Expression Grammars.

Es befindet sich im Abschnitt "Import" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::import** bereitgestellten Import-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Import-Plugin **pt::peg::import::peg**, das zwischen Konverter und Import-Manager angesiedelt ist.

IMAGE: arch_core_iplugins

API
Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Import API* zu finden ist.

pt::peg::from::peg convert *text*
Dieser Befehl nimmt die PEG-Markup-Kodierung einer Parsing Expression Grammar, die in *text* enthalten ist, und generiert die kanonische Serialisierung dieser Grammatik, wie in Abschnitt PEG-Serialisierungsformat festgelegt. Der erzeugte Wert wird anschließend als Ergebnis des Befehls zurückgegeben.

PEG-Spezifikationssprache

peg, eine Sprache zur Spezifikation von Parsing-Expression-Grammatiken, soll sowohl für Menschen lesbar als auch schreibbar sein, dabei jedoch streng genug, um eine maschinelle Verarbeitung zu ermöglichen. Wie jede Computersprache. Sie wurde definiert, um das Verfassen einer Grammatik-Spezifikation zu vereinfachen – etwas, wofür sich die anderen in den Parser-Tools vorhandenen Formate nicht eignen.

Sie wird formal durch die unten gezeigte Grammatik festgelegt, die in sich selbst geschrieben ist. Eine Einführung in die Sprache steht im *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
```

```
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
```

```
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Beispiel

Unser Beispiel legt die Grammatik eines einfachen Taschenrechners mit vier Grundrechenarten fest.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Mit höherwertigen Merkmalen der Notation, also den Zeichenklassen (vordefiniert und selbst definiert), lässt sich dieses Beispiel so umschreiben:

```
PEG calculator (Expression)
    Sign       <- [-+]                                         ;
    Number     <- Sign? <ddigit>+                              ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- [*/]                      ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- [-+]                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole

festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.

3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_import

Stichwörter: pt_peg_import, EBNF, LL(k), PEG, TDPL, context-free languages, expression,

grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::import – PEG-Import

Übersicht

package require Tcl 8.5 9
package require Tcl 8.5 9
package require snit
package require fileutil::paths
package require pt::peg
package require pluginmgr
package require pt::peg::import ?**1.0.2**?
::pt::peg::import *objectName*
objectName method ?*arg arg ...*?
objectName **destroy**
objectName **import text** *text* ?*format*?
objectName **import file** *path* ?*format*?
objectName **import object text** *object text* ?*format*?
objectName **import object file** *object path* ?*format*?
objectName **includes**
objectName **include add** *path*
objectName **include remove** *path*
objectName **include clear**

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt einen Manager für Parsing Expression Grammars bereit. Jede Instanz verwaltet eine Menge von Plugins für den Import dieser Grammatiken aus anderen Formaten, also für ihre Umwandlung zum Beispiel aus *peg*, *container*, *json* und so weiter.

Es befindet sich im Abschnitt "Import" der Kernschicht der Parser-Tools und ist eine der drei Säulen, auf denen die Verwaltung von Parsing Expression Grammars beruht.

IMAGE: arch_core_import

Die beiden anderen Säulen sind, wie oben dargestellt,

1. *PEG-Export* sowie
2. *PEG-Speicherung*.

Informationen zur Datenstruktur, die das wichtigste Ausgabeformat der von diesem Paket bereitgestellten Manager-Objekte darstellt, finden Sie im Abschnitt PEG-Serialisierungsformat.

Das Plugin-System unserer Klasse basiert auf dem Paket **pluginmgr** und ist so konfiguriert, dass es nach Plugins sucht, die

1. die Umgebungsvariable **GRAMMAR_PEG_IMPORT_PLUGINS**,

2. die Umgebungsvariable **GRAMMAR_PEG_PLUGINS**,
3. die Umgebungsvariable **GRAMMAR_PLUGINS**,
4. der Pfad "*~/.grammar/peg/import/plugin*"
5. der Pfad "*~/.grammar/peg/plugin*"
6. der Pfad "*~/.grammar/plugin*"
7. der Pfad "*~/.grammar/peg/import/plugins*"
8. der Pfad "*~/.grammar/peg/plugins*"
9. der Pfad "*~/.grammar/plugins*"
10. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\IMPORT\PLUGINS"
11. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PEG\PLUGINS"
12. der Registrierungseintrag "HKEY_CURRENT_USER\SOFTWARE\GRAMMAR\PLUGINS".

Die letzten drei werden nur verwendet, wenn das Paket auf einem Rechner mit dem Betriebssystem Windows(tm) ausgeführt wird.

Das gesamte System wird mit drei vordefinierten Import-Plugins ausgeliefert, nämlich

container
: Weitere Informationen finden Sie unter *PEG Import Plugin. From CONTAINER format*.

json
: Weitere Informationen finden Sie unter *PEG Import Plugin. From JSON format*.

peg
: Weitere Informationen finden Sie unter *PEG Import Plugin. From PEG format*.

Für Leser, die ein eigenes Import-Plugin für ein bestimmtes Format erstellen möchten, d. h. *Plugin-Entwickler*, ist das Lesen und Verstehen der Spezifikation der *Parser Tools Import API* unerlässlich, da diese die Interaktion zwischen diesem Paket und seinen Plugins ausführlich dokumentiert.

API

Paketbefehle

::pt::peg::import *objectName*
: Dieser Befehl erstellt ein neues Import-Manager-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Der Objektbefehl wird unter dem aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace.

Objektbefehl

Alle mit dem Befehl **::pt::peg::import** erstellten Objekte haben die folgende allgemeine Form:

objectName method ?*arg arg ...*?
: Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**
: Diese Methode löscht das Objekt, für das sie aufgerufen wird.

***objectName* import text *text* ?*format*?**
Diese Methode nimmt den *text* entgegen und konvertiert ihn unter Verwendung des Import-Plugins für das angegebene *format* in die kanonische Serialisierung einer Parsing Expression Grammar. Es wird ein Fehler ausgelöst, wenn für das Format kein Plugin gefunden werden konnte. Die durch den Konvertierungsprozess erzeugte Serialisierung wird als Ergebnis dieser Methode zurückgegeben.

Wenn kein *format* angegeben wird, verwendet die Methode standardmäßig **text**.

Die Spezifikation dessen, was eine *kanonische* Serialisierung ist, finden Sie im Abschnitt PEG-Serialisierungsformat.

Das Plugin muss der in der Spezifikation der *Parser Tools Import API* dokumentierten Schnittstelle entsprechen.

***objectName* import file *path* ?*format*?**
Diese Methode ist eine praktische Hülle um die im vorherigen Punkt beschriebene Methode import text. Sie liest den Inhalt der angegebenen Datei in den Arbeitsspeicher ein, übergibt das Ergebnis an import text und gibt die daraus resultierende Serialisierung als eigenes Ergebnis zurück.

***objectName* import object text *object text* ?*format*?**
Diese Methode ist eine praktische Hülle um die im vorherigen Punkt beschriebene Methode import text. Sie setzt voraus, dass *object* ein Objektbefehl ist, der eine **deserialize**-Methode unterstützt, welche die kanonische Serialisierung einer Parsing Expression Grammar erwartet. Sie importiert den Text mithilfe von import text und übergibt die resultierende Serialisierung anschließend über **deserialize** an *object*. Diese Methode gibt als Ergebnis die leere Zeichenkette zurück.

***objectName* import object file *object path* ?*format*?**
Diese Methode verhält sich wie **import object text**, mit dem Unterschied, dass sie den zu konvertierenden Text aus der angegebenen Datei liest, anstatt ihn als Argument zu erhalten.

***objectName* includes**
Diese Methode gibt eine Liste zurück, die die derzeit festgelegten Pfade enthält, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien verwendet werden sollen. Die Reihenfolge der Pfade in der Liste entspricht der Reihenfolge, in der sie verwendet werden – vom ersten bis zum letzten –, und entspricht zudem der Reihenfolge, in der sie dem Objekt hinzugefügt wurden.

objectName* include add *path
Diese Methode fügt den angegebenen *path* zur Liste der Pfade hinzu, in denen bei der Verarbeitung der Eingabe nach Include-Dateien gesucht wird. Der Pfad wird am Ende der Liste angehängt, sodass er erst nach allen zuvor hinzugefügten Pfaden durchsucht wird. Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Methode führt keine Aktion aus, wenn der Pfad bereits bekannt ist.

objectName* include remove *path
Diese Methode entfernt den angegebenen *path* aus der Liste der Pfade, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien herangezogen werden. Das

Ergebnis des Befehls ist die leere Zeichenkette.

Die Methode führt keine Aktion aus, wenn der Pfad nicht bekannt ist.

***objectName* include clear**

Diese Methode löscht die Liste der Pfade, die bei der Verarbeitung der Eingabe zur Suche nach Include-Dateien herangezogen werden. Das Ergebnis des Befehls ist die leere Zeichenkette.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode

      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value

      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf

      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void

      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
```

```
                                    mode value}
        Sign       {is {/ {t -} {t +}}
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der

Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt

festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_import_container

Stichwörter: pt_peg_import_container, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::import::container – PEG-Import-Plugin. Aus dem CONTAINER-Format

Übersicht

package require Tcl 8.5 9

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket existiert nicht. Es besteht kein Bedarf dafür. Das CONTAINER-Format für Parsing Expression Grammars ist ein Stück Tcl-Code, das nach dem Einlesen mit source eine Klasse bereitstellt, deren Instanzen die zu importierende Grammatik bereits geladen haben. Man könnte es auch so betrachten: Die CONTAINER-Ausgabe ist ihr eigenes Importpaket.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_import_json

Stichwörter: pt_peg_import_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, expression, grammar, import, matching, parser, parsing expression, parsing expression grammar, plugin, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::import::json – PEG-Import-Plugin. JSON-Format lesen

Übersicht

package require Tcl 8.5 9
package require pt::peg::import::json ?**1.1**?
package require pt::peg::to::json
import *text*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert das Import-Plugin für die Parsing Expression Grammar zur Verarbeitung von JSON-Markup.

Es befindet sich im Abschnitt "Import" der Kernschicht der Parser-Tools und ist für die Verwendung durch **pt::peg::import**, den Import-Manager, vorgesehen, der zwischen diesem und der entsprechenden Kernkonvertierungsfunktion, die von pt::peg::from::json bereitgestellt

wird, angesiedelt ist.

IMAGE: arch_core_iplugins

Zwar ist die direkte Verwendung dieses Pakets mit einem herkömmlichen Interpreter möglich, doch wird davon dringend abgeraten, da hierfür eine Reihe von Umwegen erforderlich ist, um die erwartete Umgebung bereitzustellen. Die richtige Vorgehensweise bei der Nutzung dieser Funktionalität hängt von der jeweiligen Situation ab:

1. In einer nicht vertrauenswürdigen Umgebung erfolgt der ordnungsgemäße Zugriff über das Paket **pt::peg::import** und die darin bereitgestellten Import-Manager-Objekte.
2. In einer vertrauenswürdigen Umgebung wird einfach das Paket pt::peg::from::json verwendet und direkt auf die zentralen Konvertierungsfunktionen zugegriffen.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Plugin-API, die in der Spezifikation *Parser Tools Import API* zu finden ist.

import *text*
: Dieser Befehl nimmt die JSON-Markup-Kodierung einer Parsing Expression Grammar, die in *text* enthalten ist, und generiert die kanonische Serialisierung dieser Grammatik, wie in Abschnitt PEG-Serialisierungsformat festgelegt. Der erzeugte Wert wird anschließend als Ergebnis des Befehls zurückgegeben.

JSON-Austauschformat für Grammatiken

Das Format **json** für Parsing Expression Grammars wurde als Datenaustauschformat geschrieben, das nicht an Tcl gebunden ist. Es wurde definiert, um den Austausch von Grammatiken mit PackRat/PEG-basierten Parsergeneratoren für andere Sprachen zu ermöglichen.

Dies wird durch die folgenden Regeln formal festgelegt:

1. Das JSON-Format eines jeden PEG ist ein JSON-Objekt.
2. Dieses Objekt enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
3. Der Inhalt der Grammatik ist ein JSON-Objekt, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   : Der Wert ist ein JSON-Objekt, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
     1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
     2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
     3. Der Wert für jedes Symbol ist selbst ein JSON-Objekt. Die relevanten Schlüssel und ihre Werte in diesem Dictionary lauten

        is
        : Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des Parsing-Ausdrucks enthält, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

        mode
        : Der Wert ist ein JSON-Wert mit einem von drei Werten, die festlegen, wie ein Parser den durch das Symbol erzeugten semantischen Wert verarbeiten soll.

value
Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf
Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des
    Start-Parsing-Ausdrucks der Grammatik enthält, wie im Abschnitt
    [PE-Serialisierungsformat] beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Als Anmerkung für den fortgeschrittenen Leser: Dies entspricht im Wesentlichen der Tcl-Serialisierung von PE-Grammatiken, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist, mit dem Unterschied, dass die Tcl-Dictionaries und -Listen dieses Formats auf JSON-Objekte und -Arrays abgebildet werden. Lediglich die Parsing-Ausdrücke selbst werden nicht weiter übersetzt, sondern als JSON-Zeichenketten beibehalten, die eine verschachtelte Tcl-Liste enthalten, und auch für das JSON gibt es kein Konzept der Kanonizität.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Eine JSON-Serialisierung dafür lautet

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
```

```
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

und eine Tcl-Serialisierung davon lautet

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                    mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                 mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
```

```
                                    mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                    mode value}
        MulOp      {is {/ {t *} {t /}}
                                    mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                    mode value}
        Sign       {is {/ {t -} {t +}}
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

Die Ähnlichkeit des Letzteren mit JSON dürfte ziemlich offensichtlich sein.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode

      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value

      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf
Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung
Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Hierbei handelt es sich um die *geordnete Auswahl*, auch bekannt als *priorisierte Auswahl*.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.

3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_import_peg

Stichwörter: pt_peg_import_peg, EBNF, LL(k), PEG, TDPL, context-free languages, expression,

grammar, import, matching, parser, parsing expression, parsing expression grammar, plugin, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::import::peg – PEG-Import-Plugin. Liest das PEG-Format

Übersicht

package require Tcl 8.5 9
package require pt::peg::import::peg ?**1.1**?
package require pt::peg::to::peg
import *text*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert das Import-Plugin für die Parsing-Expression-Grammatik (PEG), das PEG-Markup verarbeitet.

Es befindet sich im Abschnitt "Import" der Kernschicht der Parser-Tools und ist für die Verwendung durch **pt::peg::import**, den Import-Manager, vorgesehen, der zwischen diesem und der entsprechenden Kernkonvertierungsfunktion, die von pt::peg::from::peg bereitgestellt wird, angesiedelt ist.

Bild: arch_core_iplugins

Zwar ist die direkte Verwendung dieses Pakets mit einem herkömmlichen Interpreter möglich, doch wird davon dringend abgeraten, da hierfür eine Reihe von Umwegen erforderlich ist, um die erwartete Umgebung bereitzustellen. Die richtige Vorgehensweise bei der Nutzung dieser Funktionalität hängt von der jeweiligen Situation ab:

1. In einer nicht vertrauenswürdigen Umgebung erfolgt der ordnungsgemäße Zugriff über das Paket **pt::peg::import** und die darin bereitgestellten Import-Manager-Objekte.
2. In einer vertrauenswürdigen Umgebung genügt das Paket pt::peg::from::peg, der Zugriff auf die Kernumwandlung erfolgt direkt.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Plugin-API, die in der Spezifikation *Parser Tools Import API* zu finden ist.

import *text*
: Dieser Befehl nimmt die PEG-Markup-Kodierung einer Parsing Expression Grammar, die in *text* enthalten ist, und generiert die kanonische Serialisierung dieser Grammatik, wie in Abschnitt PEG-Serialisierungsformat festgelegt. Der erzeugte Wert wird anschließend als Ergebnis des Befehls zurückgegeben.

PEG-Spezifikationssprache

peg, eine Sprache zur Spezifikation von Parsing-Expression-Grammatiken, soll sowohl für Menschen lesbar als auch schreibbar sein, dabei jedoch streng genug, um eine maschinelle

Verarbeitung zu ermöglichen. Wie jede Computersprache. Sie wurde definiert, um das Verfassen einer Grammatik-Spezifikation zu vereinfachen – etwas, wofür sich die anderen in den Parser-Tools vorhandenen Formate nicht eignen.

Sie wird formal durch die unten gezeigte Grammatik festgelegt, die in sich selbst geschrieben ist. Eine Einführung in die Sprache steht im *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs
```

```
        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
```

```
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Beispiel

Unser Beispiel legt die Grammatik eines einfachen Taschenrechners mit vier Grundrechenarten fest.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Mit höherwertigen Merkmalen der Notation, also den Zeichenklassen (vordefiniert und selbst definiert), lässt sich dieses Beispiel so umschreiben:

```
PEG calculator (Expression)
    Sign       <- [-+]                                         ;
    Number     <- Sign? <ddigit>+                              ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- [*/]                      ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- [-+]                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf

der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_interp

Stichwörter: pt_peg_interp, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg::interp – Interpreter für Parsing Expression Grammars

Übersicht

package require Tcl 8.5 9
package require pt::peg::interp ?**1.0.2**?
package require pt::rde ?**1**?
package require snit
::pt::peg::interp *objectName grammar*
objectName **use** *grammar*
objectName **destroy**
objectName **parse** *chan*
objectName **parset** *text*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt eine Klasse bereit, deren Instanzen Packrat-Parser sind, die mithilfe einer Parsing Expression Grammar konfiguriert werden können. Die Grammatik wird direkt ausgeführt, d. h. interpretiert, wobei die zugrunde liegende Laufzeitumgebung vom Paket **pt::rde**

bereitgestellt wird und sich alles auf PARAM stützt.

Wie die zugehörige Laufzeitumgebung befindet sich auch dieses Paket im Abschnitt "Ausführung" der Kernschicht von Parser Tools.

Bild: arch_core_transform

Die interpretierte Grammatik wird aus einer Instanz von **pt::peg::container** kopiert oder aus einer beliebigen anderen Komponente, die dieselbe API bereitstellt, wie beispielsweise die von pt::peg::to::container erstellten Container-Klassen oder das zugehörige Export-Plugin **pt::peg::export::container**.

API für Klassen

Das Paket exportiert die hier beschriebene API.

::pt::peg::interp *objectName grammar*

Der Befehl erstellt ein neues Parser-Objekt und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück. Die API dieses Objektbefehls wird im Abschnitt Objekt-API beschrieben. Er kann verwendet werden, um verschiedene Operationen auf dem Objekt auszuführen.

Dieser neue Parser ist für die Ausführung eines leeren PEG konfiguriert. Zur Konfiguration für eine andere PEG dient die Methode **use** der Objekt-API.

Objekt-API

Alle durch dieses Paket erstellten Objekte stellen die folgenden Methoden bereit.

objectName* use *grammar

Diese Methode konfiguriert den Grammatik-Interpreter bzw. -Parser für die Ausführung der in *grammar* gespeicherten PEG, einem Objekt, das API-kompatibel mit Instanzen von **pt::peg::container** ist. Der Parser kopiert die relevanten Informationen der Grammatik und übernimmt das Objekt *nicht*.

Die Informationen einer zuvor verwendeten Grammatik werden überschrieben.

Das Ergebnis der Methode ist die leere Zeichenkette.

***objectName* destroy**

Diese Methode löscht die Parser-Instanz, gibt den gesamten belegten Speicher sowie alle anderen Ressourcen frei und löscht den Instanzbefehl.

Das Ergebnis des Befehls ist die leere Zeichenkette.

objectName* parse *chan

Bei dieser Methode wird der Parser mit dem Inhalt von *chan* als Eingabe ausgeführt (beginnend an der aktuellen Position im Kanal), bis eine weitere Analyse nicht mehr möglich ist – entweder, weil die Analyse abgeschlossen ist oder weil ein Syntaxfehler aufgetreten ist.

Die Parser-Tools beruhen auf Tcl 8.5+. Mit anderen Worten: Das Kanalargument ist nicht auf Dateien, Sockets usw. beschränkt. Uns steht die volle Leistungsfähigkeit von *reflektierten Kanälen* zur Verfügung.

Es sei außerdem darauf hingewiesen, dass der Parser die Zeichen bei Bedarf aus dem

Eingabestrom abruft. Wenn ein durch dieses Paket erstellter Parser im Push-Modus, d. h. ereignisgesteuert betrieben werden muss, ist es erforderlich, auf Tcl 8.6+ umzusteigen und **coroutine::auto** zu verwenden, um ihn in eine Coroutine zu verpacken, wobei **read** entsprechend für den Push-Betrieb angepasst werden muss.

Nach erfolgreicher Ausführung gibt der Befehl einen abstrakten Syntaxbaum (AST) als Ergebnis zurück. Dieser AST hat die im Abschnitt AST-Serialisierungsformat angegebene Form. Als einfache, verschachtelte Tcl-Liste kann er anschließend mit beliebigen Tcl-Befehlen des Benutzers weiterverarbeitet werden, um Transformationen, semantische Prüfungen usw. durchzuführen. Um dies zu erleichtern, bietet das Paket **pt::ast** eine Reihe von Hilfsbefehlen zur Validierung der Grundstruktur des Baums, zu dessen Ausgabe für Debugging-Zwecke sowie zum Durchlaufen des Baums entweder von unten nach oben oder von oben nach unten.

Wenn ein Syntaxfehler auftritt, löst der Befehl stattdessen einen Fehler aus. Dieser Fehler ist eine Tcl-Liste mit vier Elementen, die in der unten aufgeführten Reihenfolge Folgendes enthält:

1. Die Zeichenfolge **pt::rde**, die darauf hinweist, dass es sich um einen Parser-Laufzeitfehler handelt.
2. Die Position des Parsing-Fehlers, angegeben als Zeichenversatz vom Anfang der geparsten Eingabe.
3. Die Position des Parsing-Fehlers, nun als Liste mit zwei Elementen, die die Zeilennummer und die Spalte innerhalb der Zeile enthalten.
4. Eine Menge atomarer Parsing-Ausdrücke, welche die Zeichen und/oder Nichtterminale kodieren, die der Parser an der Fehlerstelle erwartet, aber nicht vorgefunden hat. Informationen zur Spezifikation atomarer Parsing-Ausdrücke finden Sie im Abschnitt PE-Serialisierungsformat.

objectName* parset *text

Diese Methode führt den Parser aus, wobei die Zeichenfolge in *text* als Eingabe verwendet wird. In allen anderen Punkten verhält sie sich wie die oben gezeigte Methode **parse**.

AST-Serialisierungsformat

An dieser Stelle legen wir das Format fest, das von den Parser-Tools verwendet wird, um abstrakte Syntaxbäume (ASTs) als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Jeder Knoten in einem AST repräsentiert ein nichtterminales Symbol einer Grammatik sowie den Bereich der Tokens/Zeichen in der Eingabe, den dieses Symbol abdeckt. ASTs enthalten keine terminalen Symbole, d. h. keine Tokens/Zeichen. Diese lassen sich anhand der Position eines Symbols aus der Eingabe rekonstruieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Ein Baum kann zwar mehr als eine reguläre Serialisierung aufweisen, doch ist genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen AST entspricht der Serialisierung seines Wurzelknotens.
1. Die Serialisierung eines beliebigen Knotens ist eine Tcl-Liste, die mindestens drei Elemente enthält.
    1. Das erste Element ist der Name des im Knoten gespeicherten nichtterminalen Symbols.
    2. Das zweite und dritte Element geben die Positionen des ersten und letzten Tokens in

dem Token-Strom an, den der Knoten repräsentiert (abdeckt).
1. Positionen werden als nicht-negative ganzzahlige Offsets vom Anfang des Token-Stroms angegeben, wobei das erste im Strom gefundene Token am Offset 0 (Null) liegt.
2. Die Endposition muss mindestens so groß sein wie die Startposition.

1. Alle Elemente nach den ersten drei sind die Kindknoten des Knotens, selbst wieder Knoten. Die Serialisierung eines Knotens ohne Kindknoten, also eines Blattknotens, hat daher genau drei Elemente. In der Liste steht der linke Kindknoten zuerst und der rechte zuletzt.

Kanonische Serialisierung

Die kanonische Serialisierung eines abstrakten Syntaxbaums hat das im vorigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Baums eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden Parsing Expression Grammar

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

und der Eingabezeichenkette

```
 120+5
```

Dann sollte ein Parser den folgenden abstrakten Syntaxbaum liefern (mit Ausnahme von Leerzeichen)

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
```

```
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

Oder, anschaulicher ausgedrückt

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                          |            |
|                          +- Digit 2 2 | 0
|                                       |
Expression 0 4 --- Factor 0 4 -+----------------------------- AddOp 3
    3 | +
|                                       |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine

eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein

Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_introduction

Stichwörter: pt_peg_introduction, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::pegrammar – Einleitung in Parsing Expression Grammars

Übersicht

package require Tcl 8.5 9

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Willkommen zur Einleitung in die *Parsing Expression Grammar*s (kurz: *PEG*), den von den Parser-Tools verwendeten Formalismus. Es wird vorausgesetzt, dass der Leser über Grundkenntnisse der Parsing-Theorie verfügt, d. h. über *kontextfreie Grammatiken* (kurz: *CFG*), *Sprachen* und zugehörige Begriffe wie *LL(k)*, *LR(k)*, *terminale* und *nichtterminale Symbole* usw. Es ist nicht unsere Absicht, solche grundlegenden Definitionen oder Begriffe wie *nützlich*, *erreichbar*, (links-/rechts-)*rekursiv*, *nullfähig*, First-/Last-/Follow-Mengen usw. noch einmal zusammenzufassen. Die Literatur am Ende nennt Quellen und Bücher, die diese Hintergründe liefern.

PEGs sind formal den CFGs sehr ähnlich, mit terminalen und nichtterminalen Symbolen, einem Startsymbol sowie Regeln, die die Struktur jedes nichtterminalen Symbols definieren. Der Hauptunterschied liegt in der Wahl (sic!) der *Auswahl*-Operatoren. Während CFGs eine *ungeordnete Auswahl* verwenden, um Alternativen darzustellen, nutzen PEGs eine *priorisierte Auswahl*. Das ist eine ausgefallene Art zu sagen, dass ein Parser zunächst die erste Alternative ausprobieren muss und die anderen Alternativen erst dann ausprobieren darf, wenn die erste fehlschlägt, und so weiter.

Auf der CFG-Seite ergeben sich daraus LL(k) und LR(k), um die Auswahl *deterministisch* mit begrenzter *Vorausschau* von k Terminalsymbolen zu gestalten, wobei LL im Wesentlichen eine *Top-Down*-Analyse, auch bekannt als *rekursiver Abstieg*, ist, und LR ein *Bottom-up*-Parsing, auch bekannt als *Shift-Reduce*-Parsing.

Auf der PEG-Seite können wir die Eingabe mittels rekursiver Abwärtsanalyse und *Backtracking* bei fehlgeschlagenen Auswahlen analysieren, wobei Letzteres unbegrenzter Vorausschau entspricht. Indem wir zusätzlich den Erfolg oder Misserfolg von Nichtterminalen an den jeweiligen Stellen, an denen sie geprüft wurden, protokollieren und diese Informationen nach dem Backtracking wiederverwenden, können wir den exponentiellen Anstieg der Laufzeit vermeiden, der üblicherweise mit Backtracking einhergeht, und die Analyse linear halten. Die Speicheranforderungen sind aufgrund dieses Caches natürlich höher, da wir Speicherplatz gegen Zeit eintauschen.

Dies ist das Grundkonzept hinter *Packrat-Parsern*.

Eine Einschränkung, die reine PEGs mit LL(k)-CFGs gemeinsam haben, besteht darin, dass *linksrekursive* Grammatiken nicht geparst werden können, da der zugehörige rekursive Abstiegsparser in eine unendliche Rekursion gerät. Diese Einschränkung wird in der Regel dadurch überwunden, dass reine PEGs um explizite Operatoren erweitert werden, um Wiederholungen – null oder mehr sowie eine oder mehr – festzulegen, oder, formal ausgedrückt, für den *Kleene-Abschluss* und den *positiven Kleene-Abschluss*. Genau dies leisten die Parser-Tools.

Eine weitere, speziell auf Parser Tools zugeschnittene Erweiterung ist eine Reihe von Operatoren, die mehr oder weniger direkt verschiedenen in Tcl integrierten Zeichenklassen zugeordnet sind,

d. h. den Klassen, auf die über **string is** zugegriffen werden kann.

Der weitere Teil dieses Dokuments umfasst die formale Definition von PEGs für mathematisch Interessierte sowie einen Anhang mit Literaturverweisen auf Stellen, die weitere Informationen speziell zu PEGs und zur Syntaxanalyse im Allgemeinen enthalten.

Formale Definition

Für mathematisch Interessierte: Eine Parsing Expression Grammar ist ein 4-Tupel (VN, VT, R, eS), wobei

- VN ist eine Menge von *nichtterminalen Symbolen*,
- VT ist eine Menge von *Terminalsymbolen*,
- R ist eine endliche Menge von Regeln, wobei jede Regel ein Paar (A, e) ist, wobei A in VN liegt und *e* ein *Parsing-Ausdruck* ist.
- eS ist ein Parsing-Ausdruck, der *Startausdruck*.

Weitere Bedingungen sind

- Die Schnittmenge von VN und VT ist leer.
- Für jedes A in VN gibt es genau ein Paar (A, e) in R. Mit anderen Worten: R ist eine Funktion von nichtterminalen Symbolen auf Parsing-Ausdrücke.

Parsing-Ausdrücke werden induktiv definiert über

- Die leere Zeichenkette (Epsilon) ist ein Parsing-Ausdruck.
- Ein Terminalsymbol *a* ist ein Parsing-Ausdruck.
- Ein nichtterminales Symbol *A* ist ein Parsing-Ausdruck.
- *e1e2* ist ein Parsing-Ausdruck für die Parsing-Ausdrücke *e1* und *e2*. Das heißt *Sequenz*.
- *e1*/*e2* ist ein Parsing-Ausdruck für die Parsing-Ausdrücke *e1* und *e2*. Dies wird als *geordnete Auswahl* bezeichnet.
- *e** ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e*. Dies wird als *Null-oder-mehr-Wiederholungen* bezeichnet, auch bekannt als *Kleene-Abschluss*.
- *e*+ ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e*. Dies wird als *eine oder mehrere Wiederholungen* bezeichnet, auch bekannt als *positiver Kleene-Abschluss*.
- !*e* ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e*. Das ist ein *Nicht-Vorausschau-Prädikat*.
- &*e* ist ein Parsing-Ausdruck für den Parsing-Ausdruck *e1*. Das ist ein *Und-Vorausschau-Prädikat*.

PEGs definieren eine grammatische Struktur für Symbolströme über VT. Sie sind eine moderne Formulierung älterer Formalismen, die Alexander Birham eingeführt hat. Diese Formalismen wurden als TS (TMG-Erkennungsschema) und gTS (verallgemeinertes TS) bezeichnet. Später wurden sie in TPDL (Top-Down-Parsing-Sprachen) und gTPDL (verallgemeinertes TPDL) umbenannt.

Sie lassen sich leicht durch rekursive Abstiegsparser mit Backtracking implementieren. Damit stehen sie in Verwandtschaft zu den kontextfreien LL(k)-Grammatiken.

Literatur

1. The Packrat Parsing and Parsing Expression Grammars Page, von Bryan Ford, Massachusetts Institute of Technology. Das ist die Haupteinstiegsseite zu PEGs und ihrer Umsetzung durch Packrat-Parser.
2. http://en.wikipedia.org/wiki/Parsing_expression_grammar Der Wikipedia-Eintrag zu Parsing Expression Grammars.

3. *Parsing Techniques - A Practical Guide* http://www.cs.vu.nl/~dick/PTAPG.html, ein Online-Buch, das eine klare, leicht verständliche und umfassende Darstellung zahlreicher verschiedener Parsing-Techniken mit ihren Zusammenhängen und Anwendungsmöglichkeiten bietet, einschließlich Techniken zur Fehlerbehebung.
4. Compilers and Compiler Generators, ein Online-Buch, das CoCo/R verwendet, einen Generator für rekursive Abstiegsparser.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_language

Stichwörter: pt_peg_language, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg_language – Tutorium zur PEG-Sprache

Übersicht

package require Tcl 8.5 9

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Willkommen zum Tutorial bzw. zur Einleitung in die PEG-Spezifikationssprache. Wenn Sie bereits mit der Sprache, die wir im Folgenden behandeln werden, vertraut sind und lediglich Ihr Gedächtnis auffrischen möchten, können Sie natürlich direkt zum oben genannten Abschnitt springen und dort die vollständige formale Spezifikation lesen.

Was ist das?

peg, eine Sprache zur Spezifikation von Parsing-Expression-Grammatiken, soll sowohl für Menschen lesbar als auch schreibbar sein, dabei jedoch streng genug, um eine maschinelle

Verarbeitung zu ermöglichen. Wie jede Computersprache. Sie wurde definiert, um das Verfassen einer Grammatik-Spezifikation zu vereinfachen – etwas, wofür sich die anderen in den Parser-Tools vorhandenen Formate nicht eignen.

Die Elemente der Sprache

Grundlegender Aufbau

Die allgemeine Gliederung einer textbasierten PEG sieht wie folgt aus:

```
PEG <<name>> (<<start-expression>>)
   <<rules>>
END;
```

Hinweis: Wir verwenden Text in doppelten spitzen Klammern als Platzhalter für noch nicht erläuterte Begriffe.

Namen

Namen werden hauptsächlich zur Identifizierung der nichtterminalen Symbole der Grammatik verwendet, d. h. der Elemente, die auf der linken Seite eines stehen. Eine Ausnahme bildet der Name, der nach dem Schlüsselwort **PEG** angegeben wird (siehe vorherigen Abschnitt); dabei handelt es sich um den Namen der gesamten Grammatik selbst.

Der Aufbau eines Namens ist einfach:

1. Es beginnt mit einem Buchstaben, einem Unterstrich oder einem Doppelpunkt, gefolgt von
2. null oder mehr Buchstaben, Ziffern, Unterstriche oder Doppelpunkte.

Oder, in der formalen Textnotation:

```
    ([_:] / <alpha>) ([_:] / <alnum>)*
```

Beispiele für Namen:

```
    Hello
    ::world
    _:submarine55_
```

Beispiele für Text, bei dem es sich *nicht* um Namen handelt:

```
    12
    .bogus
    0wrong
    @location
```

Regeln

Den Hauptteil des Textes einer Grammatikbeschreibung bilden die Regeln. Jede Regel definiert die Satzstruktur eines nichtterminalen Symbols. Ihre Grundstruktur lautet

```
     <<name>>  <-  <<expression>> ;
```

Mit wird das zu definierende nichtterminale Symbol festgelegt, während nach dem Pfeil (<-) dessen Struktur deklariert.

Hinweis: Jede Regel endet mit einem einzelnen Semikolon, auch die letzte. Das heißt, das

Semikolon dient als *Regelabschluss*, nicht als Trennzeichen.

Wir können so viele Regeln haben, wie wir möchten, solange wir jedes nichtterminale Symbol höchstens einmal definieren und für jedes nichtterminale Symbol, das in einem Ausdruck vorkommt – d. h. entweder im Startausdruck der Grammatik oder auf der rechten Seite einer Regel –, mindestens eine Regel vorsehen.

Ausdrücke

Die *Parsing*-Ausdrücke bilden den Kern jeder Spezifikation. Sie legen die Struktur des gesamten Dokuments (<>) sowie aller nichtterminalen Symbole fest.

Alle Ausdrücke setzen sich aus *atomaren Ausdrücken* und *Operatoren* zusammen, die diese miteinander verbinden. Es gibt Operatoren zur Auswahl zwischen Alternativen, zur Wiederholung von Teilen und für Vorausschau-Bedingungen. Es gibt jedoch keinen expliziten Operator für die Aneinanderreihung (auch als *Verkettung* bezeichnet) von Teilen. Diese wird durch die einfache Platzierung der Teile nebeneinander festgelegt.

Hier sind die Operatoren, geordnet nach ihrer Priorität (d. h. der Stärke der Bindung), vom höchsten zum niedrigsten Wert:

```
    # Binary operators.

    <<expression-1>>     <<expression-2>>  # sequence. parse 1, then 2.
    <<expression-1>>  /  <<expression-2>>  # alternative. try to parse
    1, and parse 2 if 1 failed to parse.

    # Prefix operators. Lookahead constraints. Same priority.

    & <<expression>>  # Parse expression, ok on successful parse.
    ! <<expression>>  # Ditto, except ok on failure to parse.

    # Suffix operators. Repetition. Same priority.

    <<expression>> ?  # Parse expression none, or once (repeat 0 or 1).
    <<expression>> *  # Parse expression zero or more times.
    <<expression>> +  # Parse expression one or more times.

    # Expression nesting

    ( <<expression>> ) # Put an expression in parens to change its
    priority.
```

Damit können wir nun den im Abschnitt **Namen** angegebenen formalen Ausdruck für Namen zerlegen:

```
    ([_:] / <alpha>) ([_:] / <alnum>)*
```

Es ist eine Sequenz aus zwei Teilen,

```
    [_:] / <alpha>
```

und

```
    ([_:] / <alnum>)*
```

Die Klammern um die Teile hielten deren innere Alternativen trotz der normalerweise höheren Priorität der Sequenz zusammen. Jeder der beiden Teile stellt eine Alternative dar, wobei der zweite Teil zusätzlich null- oder mehrmals wiederholt wird, sodass wir die folgenden drei atomaren Ausdrücke erhalten:

```
    [_:]
    <alpha>
    <alnum>
```

Und *atomare Ausdrücke* sind unser nächstes Thema. Sie lassen sich in drei Klassen einteilen:

1. Namen, d. h. nichtterminale Symbole,
2. Zeichenfolgenliterale und
3. Zeichenklassen.

Die Namen sind Ihnen bereits bekannt; falls nicht, lesen Sie bitte den Abschnitt Namen zur Auffrischung.

Zeichenfolgenliterale sind einfach. Sie werden entweder durch ein einfaches oder ein doppeltes Apostroph begrenzt (d. h., sie beginnen und enden damit), und zwischen den Begrenzungszeichen können beliebige Zeichen stehen, mit Ausnahme des Begrenzungszeichens selbst. Sie können auch leer sein. Beispiele für Zeichenfolgen sind

```
    ''
    ""
    'hello'
    "umbra"
    "'"
    '"'
```

Die beiden letzten Beispiele veranschaulichen, wie sich jedes Begrenzungszeichen in eine Zeichenfolge einfügen lässt.

Zum Schluss unter den atomaren Ausdrücken die Zeichenklassen. Es gibt eine Reihe vordefinierter Klassen, unten aufgeführt, und die Möglichkeit, eigene zu bilden. Die vordefinierten Klassen lauten:

```
    <alnum>    # Any unicode alphabet or digit character (string is
    alnum).
    <alpha>    # Any unicode alphabet character (string is alpha).
    <ascii>    # Any unicode character below codepoint 0x80 (string is
    ascii).
    <control>  # Any unicode control character (string is control).
    <ddigit>   # The digit characters [0-9].
    <digit>    # Any unicode digit character (string is digit).
    <graph>    # Any unicode printing character, except space (string
    is graph).
    <lower>    # Any unicode lower-case alphabet character (string is
```

```
    lower).
    <print>    # Any unicode printing character, incl. space (string
    is print).
    <punct>    # Any unicode punctuation character (string is punct).
    <space>    # Any unicode space character (string is space).
    <upper>    # Any unicode upper-case alphabet character (string is
    upper).
    <wordchar> # Any unicode word character (string is wordchar).
    <xdigit>   # The hexadecimal digit characters [0-9a-fA-F].
    .          # Any character, except end of input.
```

Und die Syntax für benutzerdefinierte Zeichenklassen lautet

```
    [ <<range>>* ]
```

wobei jeder Bereich entweder aus einem einzelnen Zeichen besteht oder die Form

```
   <<character>> - <character>>
```

Beispiele für Zeichenklassen, die in dieser Einleitung schon vorgekommen sind, sind

```
    [_:]
    [0-9]
    [0-9a-fA-F]
```

Wir sind mit den Ausdrücken fast fertig. Als Letztes müssen wir noch erläutern, wie die Zeichen in Klassen und Zeichenfolgenliteralen festgelegt werden.

Grundsätzlich stehen die Zeichen in der Eingabe für sich selbst; darüber hinaus gibt es verschiedene Arten von Escape-Syntax, um Steuerzeichen oder Zeichen darzustellen, die außerhalb der Kodierung liegen, in der der Text vorliegt.

Alle Escape-Sequenzen beginnen mit einem Backslash-Zeichen ("\", Unicode-Codepunkt 0x5C). Darauf folgt eine Folge von Oktalziffern, von "u" gefolgt von Hexadezimalziffern oder ein reguläres Zeichen aus einem festgelegten Satz für verschiedene Steuerzeichen. Einige Beispiele:

```
    \n \r \t \' \" \[ \] \\ #
    \000 up to \277         # octal escape, all ascii character,
    leading 0's can be removed.
    \u2CA7                  # hexadecimal escape, all unicode
    characters.
    #                       # Here 2ca7 <=> Koptic Small Letter Tau
```

Leerzeichen und Kommentare

Ein Thema, das bisher noch nicht angesprochen wurde, sind Leerzeichen und Kommentare.

Als Leerzeichen gelten alle Unicode-Leerzeichen, d. h. alle Zeichen der Klasse sowie Kommentare. Letztere sind Zeichenfolgen, die mit einem "#" (Hash, Unicode-Codepunkt 0x23) beginnen und am nächsten Zeilenende enden.

Zwischen allen syntaktischen Elementen einer Grammatikbeschreibung können Leerzeichen beliebig verwendet werden. Innerhalb von syntaktischen Elementen, wie Namen, Zeichenfolgenliteralen, vordefinierten Zeichenklassen usw., dürfen sie nicht verwendet werden.

Attribute von Nichtterminalen

Zum Schluss noch ein etwas fortgeschritteneres Thema. Im Abschnitt Regeln haben wir die Struktur einer Regel wie folgt dargestellt:

```
     <<name>>  <-  <<expression>> ;
```

Das stimmt nicht ganz. Es ist möglich, dem Nichtterminal in der Regel einen semantischen Modus zuzuordnen, indem man diesen vor dem Namen angibt und durch einen Doppelpunkt davon trennt, d. h. indem man schreibt:

```
    <<mode>> : <<name>>  <-  <<expression>> ;
```

Diese Schreibweise ist ebenfalls zulässig. Dieser Modus ist optional. Die bekannten Modi und ihre Bedeutungen lauten wie folgt:

value
: Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf
: Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
: Das Nichtterminal hat keinen semantischen Wert. Alle durch die rechte Seite des Symbols generierten ASTs werden (ebenfalls) verworfen.

Von diesen drei Modi können nur **leaf** und **void** direkt angegeben werden. **value** wird implizit durch das Fehlen eines Modus vor dem Nichtterminal festgelegt.

Mit all diesen Kenntnissen sollte es nun möglich sein, die formale Spezifikation der im nächsten Abschnitt dargestellten Textdarstellung – die in dieser Sprache verfasst ist – nicht nur zu lesen, sondern auch zu verstehen.

PEG-Spezifikationssprache

peg, eine Sprache zur Spezifikation von Parsing-Expression-Grammatiken, soll sowohl für Menschen lesbar als auch schreibbar sein, dabei jedoch streng genug, um eine maschinelle Verarbeitung zu ermöglichen. Wie jede Computersprache. Sie wurde definiert, um das Verfassen einer Grammatik-Spezifikation zu vereinfachen – etwas, wofür sich die anderen in den Parser-Tools vorhandenen Formate nicht eignen.

Sie wird formal durch die unten gezeigte Grammatik festgelegt, die in sich selbst geschrieben ist. Eine Einführung in die Sprache steht im *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;
```

```
        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;
```

```
void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
```

```
    -------
END;
```

Beispiel

Unser Beispiel legt die Grammatik eines einfachen Taschenrechners mit vier Grundrechenarten fest.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Mit höherwertigen Merkmalen der Notation, also den Zeichenklassen (vordefiniert und selbst definiert), lässt sich dieses Beispiel so umschreiben:

```
PEG calculator (Expression)
    Sign       <- [-+]                                         ;
    Number     <- Sign? <ddigit>+                              ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- [*/]                       ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- [-+]                       ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_op

Stichwörter: pt_peg_op, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt_peg_op – Parser-Tools: Operationen des PE-Grammatik-Dienstprogramms

Übersicht

package require Tcl 8.5 9
package require pt::peg::op ?**1.2.0**?
::peg::peg::op called *container*
::peg::peg::op dechain *container*
::peg::peg::op drop unreachable *container*
::peg::peg::op drop unrealizable *container*
::peg::peg::op flatten *container*
::peg::peg::op minimize *container*
::peg::peg::op modeopt *container*
::peg::peg::op reachable *container*
::peg::peg::op realizable *container*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket enthält eine Reihe von Dienstprogrammen, mit denen sich eine PE-Grammatik (Container) auf verschiedene Weise bearbeiten lässt.

API

::peg::peg::op called *container*

Dieser Befehl ermittelt die statische Aufrufstruktur für die nichtterminalen Symbole der im *container* gespeicherten Grammatik.

Das Ergebnis des Befehls ist ein Dictionary, das jedes Symbol den von ihm aufgerufenen Symbolen zuordnet. Die leere Zeichenkette dient als Schlüssel zur Darstellung des Startausdrucks der Grammatik.

Die Grammatik im Container wird nicht verändert.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op dechain *container*

Dieser Befehl vereinfacht alle Symbole, die lediglich auf ein anderes Symbol verweisen, indem er die rechte Seite des aufgerufenen Symbols in dessen Aufrufer einfügt. Dies funktioniert genau dann, wenn die Modi gemäß der nachstehenden Entscheidungstabelle korrekt übereinstimmen.

```
caller called | dechain | notes
--------------+---------+-----------------------
value  value  |  yes    |  value is passed
value  leaf   |  yes    |  value is passed
value  void   |  yes    |  caller is implied void
leaf   value  |  no     |  generated value was discarded, inlined
    would not. called may be implied void.
leaf   leaf   |  no     |  s.a.
leaf   void   |  no     |  s.a.
void   value  |  no     |  caller drops value, inlined would not.
void   leaf   |  no     |  s.a.
void   void   |  yes    |
```

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Grammatik im Container wird direkt geändert. Wenn dies nicht gewünscht ist, muss eine Kopie des ursprünglichen Containers verwendet werden.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op drop unreachable *container*

Dieser Befehl entfernt alle Symbole aus der Grammatik, die nicht **reachable** sind.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Grammatik im Container wird direkt geändert. Ist dies nicht erwünscht, muss eine Kopie des ursprünglichen Containers verwendet werden.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op drop unrealizable *container*

Dieser Befehl entfernt alle Symbole aus der Grammatik, die nicht **realizable** sind.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Grammatik im Container wird direkt geändert. Ist dies nicht erwünscht, muss eine Kopie des ursprünglichen Containers verwendet werden.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op flatten *container*

Dieser Befehl ebnet (siehe **pt::pe::op**) alle Ausdrücke in der Grammatik ein, d. h. den Startausdruck und die rechten Seiten aller nichtterminalen Symbole.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Grammatik im Container wird direkt geändert. Wenn dies nicht erwünscht ist, muss eine

Kopie des ursprünglichen Containers verwendet werden.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op minimize *container*

Dieser Befehl reduziert die bereitgestellte Grammatik, indem er die meisten anderen Methoden dieses Pakets anwendet.

Nach dem Einebnen der Ausdrücke entfernt er nicht erreichbare und nicht realisierbare Symbole, ebnet die Ausdrücke erneut ein und optimiert anschließend die Symbolmodi, bevor er Symbolketten so weit wie möglich zusammenfasst.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Grammatik im Container wird direkt verändert. Ist dies nicht erwünscht, muss eine Kopie des ursprünglichen Containers verwendet werden.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op modeopt *container*

Dieser Befehl optimiert die semantischen Modi von nichtterminalen Symbolen gemäß den beiden nachstehenden Regeln.

1. Wenn ein Symbol X mit dem Modus **value** keine anderen Symbole aufruft, d. h. ausschließlich Terminalsymbole in beliebiger Kombination verwendet, lässt sich dies durch Verwendung des Modus **leaf** einfacher darstellen.
2. Wenn ein Symbol X ausschließlich von Symbolen mit den Modi **leaf** oder **void** aufgerufen wird, sollte dieses Symbol ebenfalls den Modus **void** aufweisen, da jeder von ihm erzeugte AST ohnehin verworfen wird.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Die Grammatik im Container wird direkt geändert. Wenn dies nicht gewünscht ist, muss eine Kopie des ursprünglichen Containers verwendet werden.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op reachable *container*

Dieser Befehl ermittelt die Menge aller nichtterminalen Symbole, die vom Startausdruck der Grammatik aus erreichbar sind. Dabei handelt es sich im Wesentlichen um die transitive Hülle über **called** und die rechten Seiten der Symbole, beginnend mit dem Startausdruck.

Das Ergebnis des Befehls ist die Liste der erreichbaren Symbole.

Die Grammatik im Container wird nicht verändert.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

::peg::peg::op realizable *container*

Dieser Befehl berechnet die Menge aller nichtterminalen Symbole, die realisierbar sind, d. h. reine terminale Phrasen ableiten können. Dies geschieht iterativ, wobei zunächst für alle Symbole der Zustand "nicht realisierbar" gesetzt wird und anschließend alle realisierbaren Symbole aktualisiert sowie die Änderungen weiterverbreitet werden, bis sich nichts mehr ändert.

Das Ergebnis des Befehls ist die Liste der realisierbaren Symbole.

Die Grammatik im Container wird nicht verändert.

Die *container*-Instanz muss eine Methoden-API bereitstellen, wie sie vom Paket **pt::peg::container** bereitgestellt wird.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_peg_to_container

Stichwörter: pt_peg_to_container, CONTAINER, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::to::container – PEG-Konvertierung. Schreibt das Format CONTAINER

Übersicht

package require Tcl 8.5 9
package require pt::peg::to::container ?**1.1**?
package require pt::peg
package require text::write
package require char
pt::peg::to::container reset

pt::peg::to::container configure
pt::peg::to::container configure *option*
pt::peg::to::container configure *option value...*
pt::peg::to::container convert *serial*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter zur Umwandlung von Parsing Expression Grammars in Container-Markup.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::export** bereitgestellten Export-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Export-Plugin **pt::peg::export::container**, das zwischen Konverter und Export-Manager angesiedelt ist.

Bild: arch_core_eplugins

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

pt::peg::to::container reset
Dieser Befehl setzt die Konfiguration des Pakets auf die Standardeinstellungen zurück.

pt::peg::to::container configure
Dieser Befehl gibt ein Dictionary zurück, das die aktuelle Konfiguration des Pakets enthält.

pt::peg::to::container configure *option*
Dieser Befehl gibt den aktuellen Wert der angegebenen *option* des Pakets zurück. Die Liste der zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::container configure *option value...*
Dieser Befehl setzt die angegebenen *option*s des Pakets auf die angegebenen *value*s. Eine Übersicht über die zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::container convert *serial*
Dieser Befehl nimmt die kanonische Serialisierung einer Parsing-Expression-Grammatik entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, und generiert gemäß der aktuellen Paketkonfiguration eine Container-Kodierung, die die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Optionen

Der Konverter für das Container-Format erkennt die folgenden Optionen und passt sein Verhalten entsprechend der dort festgelegten Einstellung an.

-file string
Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-mode bulk|incremental
Der Wert dieser Option legt fest, welche Methoden von **pt::peg::container**-Instanzen zur Festlegung der Grammatik verwendet werden, d. h. zum Vorladen in den Container. Es gibt zwei zulässige Werte, die im Folgenden aufgeführt sind. Der Standardwert ist **bulk**.

bulk
In diesem Modus werden die Methoden **start**, **add**, **modes** und **rules** verwendet, um die Grammatik pauschal festzulegen, d. h. als eine Menge von nichtterminalen Symbolen sowie zwei Dictionarys, die die Symbole ihren semantischen Modi und Parsing-Ausdrücken zuordnen.

Dieser Modus ist der Standardmodus.

incremental
In diesem Modus werden die Methoden **start**, **add**, **mode** und **rule** verwendet, um die Grammatik stückweise festzulegen, wobei jedes Nichtterminal über einen eigenen Block von Definitionsbefehlen verfügt.

-template string
Der Wert dieser Option ist eine Zeichenkette, in die der generierte Code und die übrigen Konfigurationseinstellungen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mithilfe der unten aufgeführten Platzhalter angegeben werden. Der Standardwert lautet "**@code@**".

@user@
Ist durch den Wert der Option **-user** zu ersetzen.

@format@
Ist durch die Konstante **CONTAINER** zu ersetzen.

@file@
Ist durch den Wert der Option **-file** zu ersetzen.

@name@
Ist durch den Wert der Option **-name** zu ersetzen.

@mode@
Ist durch den Wert der Option **-mode** zu ersetzen.

@code@
Wird durch den generierten Code ersetzt.

Grammatik-Container

Das Format **container** ist eine weitere Form, Parsing Expression Grammars zu beschreiben. Zwar sind Daten in diesem Format ausführbar, sie stellen jedoch keinen Parser für die Grammatik dar. Sie müssen stets in Verbindung mit dem Paket **pt::peg::interp**, einem Grammatik-Interpreter, verwendet werden.

Das Format stellt Grammatiken mittels einer **snit::type** dar, d. h. einer Klasse, deren Instanzen API-kompatibel zu den Instanzen des Pakets **pt::peg::container** sind und die mit der jeweiligen Grammatik vorbelegt sind.

Es gibt keine direkte formale Spezifikation, die über das oben Gesagte hinausgeht.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Eine mögliche Container-Serialisierung hierfür lautet

```
snit::type a_pe_grammar {
    constructor {} {
        install myg using pt::peg::container ${selfns}::G
        $myg start {n Expression}
        $myg add   AddOp Digit Expression Factor MulOp Number Sign Term
        $myg modes {
            AddOp      value
            Digit      value
            Expression value
            Factor     value
            MulOp      value
            Number     value
            Sign       value
            Term       value
        }
        $myg rules {
            AddOp      {/ {t -} {t +}}
            Digit      {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}
            Expression {/ {x {t \50} {n Expression} {t \51}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}}
            Factor     {x {n Term} {* {x {n AddOp} {n Term}}}}
            MulOp      {/ {t *} {t /}}
            Number     {x {? {n Sign}} {+ {n Digit}}}
            Sign       {/ {t -} {t +}}
            Term       {n Number}
```

```
        }
        return
    }

    component myg
    delegate method * to myg
}
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode

      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value

      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf

      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void

      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
```

```
                                    mode value}
        Sign       {is {/ {t -} {t +}}
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der

Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch

sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.
1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_cparam

Stichwörter: pt_peg_to_cparam, CPARAM, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::to::cparam – PEG-Konvertierung. Schreibt das Format CPARAM

Übersicht

package require Tcl 8.5 9
package require pt::peg::to::cparam ?**1.1.3**?
pt::peg::to::cparam reset
pt::peg::to::cparam configure
pt::peg::to::cparam configure *option*
pt::peg::to::cparam configure *option value...*

pt::peg::to::cparam convert *serial*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter zur Umwandlung von Parsing Expression Grammars in CPARAM-Markup.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::export** bereitgestellten Export-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Export-Plugin **pt::peg::export::cparam**, das zwischen Konverter und Export-Manager angesiedelt ist.

Bild: arch_core_eplugins

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

pt::peg::to::cparam reset
: Dieser Befehl setzt die Konfiguration des Pakets auf die Standardeinstellungen zurück.

pt::peg::to::cparam configure
: Dieser Befehl gibt ein Dictionary zurück, das die aktuelle Konfiguration des Pakets enthält.

pt::peg::to::cparam configure *option*
: Dieser Befehl gibt den aktuellen Wert der angegebenen *option* des Pakets zurück. Die Liste der zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::cparam configure *option value*...
: Dieser Befehl setzt die angegebenen *option*s des Pakets auf die angegebenen *value*s. Eine Übersicht über die zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::cparam convert *serial*
: Dieser Befehl nimmt die kanonische Serialisierung einer Parsing-Expression-Grammatik entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, und generiert gemäß der aktuellen Paketkonfiguration ein CPARAM-Markup, das die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Optionen

Der Konverter in C-Code erkennt die folgenden Konfigurationsvariablen und passt sein Verhalten entsprechend deren Vorgaben an.

-file string
: Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
: Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der

Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-template string
Der Wert dieser Option ist eine Zeichenkette, in die der generierte Text und die übrigen Konfigurationseinstellungen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mithilfe der unten aufgeführten Platzhalter angegeben werden. Der Standardwert lautet "**@code@**".

@user@
Ist durch den Wert der Option **-user** zu ersetzen.

@format@
Ist durch die Konstante **C/PARAM** zu ersetzen.

@file@
Ist durch den Wert der Option **-file** zu ersetzen.

@name@
Ist durch den Wert der Option **-name** zu ersetzen.

@code@
Wird durch den generierten Tcl-Code ersetzt.

Die folgenden Optionen sind insofern besonders, als sie im generierten Code vorkommen und dort ebenfalls ersetzt werden.

@statedecl@
Ist durch den Wert der Option **state-decl** zu ersetzen.

@stateref@
Ist durch den Wert der Option **state-ref** zu ersetzen.

@strings@
Ist durch den Wert der Option **string-varname** zu ersetzen.

@self@
Ist durch den Wert der Option **self-command** zu ersetzen.

@def@
Ist durch den Wert der Option **fun-qualifier** zu ersetzen.

@ns@
Ist durch den Wert der Option **namespace** zu ersetzen.

@main@
Ist durch den Wert der Option **main** zu ersetzen.

@prelude@
Ist durch den Wert der Option **prelude** zu ersetzen.

-state-decl string
Eine C-Zeichenkette, die die Argumentdeklaration darstellt, die in den generierten Parsing-Funktionen verwendet werden soll, um auf den Parsing-Zustand zu verweisen. Im Wesentlichen handelt es sich dabei um den Typ und den Namen des Arguments. Der Standardwert ist die Zeichenkette **RDE_PARAM p**.

-state-ref string

Eine C-Zeichenkette mit dem Argumentnamen, mit dem die erzeugten Parsing-Funktionen auf den Parsing-Zustand verweisen. Der Standardwert ist die Zeichenkette **p**.

-self-command string
Eine C-Zeichenkette, die die Referenz angibt, die erforderlich ist, um die generierte Parser-Funktion (Methoden …) von einer anderen Parser-Funktion aus aufzurufen, entsprechend dem gewählten Rahmen (Template). Der Standardwert ist die leere Zeichenkette.

-fun-qualifier string
Eine C-Zeichenkette mit den Attributen der erzeugten Funktionen (Methoden …) gemäß dem gewählten Rahmen (Template). Der Standardwert lautet **static**.

-namespace string
Der Name des C-Namespaces, in dem sich die Parser-Funktionen (Methoden usw.) befinden sollen, oder ein allgemeines Präfix, das den Funktionsnamen vorangestellt werden soll. Der Standardwert ist die leere Zeichenkette.

-main string
Der Name der Hauptfunktion (Methode, …), die vom ausgewählten Rahmen (Template) aufgerufen werden soll, um mit der Analyse der Eingabe zu beginnen. Der Standardwert ist **__main**.

-string-varname string
Der Name der Variablen, die für die Tabelle der Zeichenfolgen verwendet wird, auf die der generierte Parser zurückgreift, d. h. Fehlermeldungen, Symbolnamen usw. Der Standardwert lautet **p_string**.

-prelude string
Ein Codeausschnitt, der am Anfang jeder generierten Parsing-Funktion eingefügt werden soll. Der Standardwert ist die leere Zeichenkette.

-indent integer
Die Anzahl der Zeichen, um die jede Zeile des generierten Codes eingerückt werden soll. Der Standardwert lautet **0**.

-comments boolean
Ein Schalter, der die Generierung von Code-Kommentaren steuert, die den ursprünglichen Parsing-Ausdruck enthalten, für den eine Parsing-Funktion vorgesehen ist. Der Standardwert lautet **on**.

Während die hohe Parametrisierbarkeit dieses Konverters – wie die Vielzahl der unterstützten Optionen zeigt – für fortgeschrittene Anwender einen Vorteil darstellt, da sie die Ausgabe des Konverters nach Bedarf anpassen können, wird ein unerfahrener Anwender wahrscheinlich den Wald vor lauter Bäumen nicht sehen.

Um diesen Anwendern zu helfen, wird ein Zusatzpaket bereitgestellt, das eine vorgefertigte Konfiguration enthält, mit der sich sofort einsatzbereite vollständige Parser generieren lassen. Es ist

pt::cparam::configuration::critcl
Die generierten Parser werden in einen auf **Critcl** basierenden Rahmen eingebettet.

C/PARAM-Code-Darstellung von Parsing-Expression-Grammatiken

Das Format **c** ist ausführbarer Code, nämlich ein Parser für die Grammatik. Die

Parser-Implementierung ist in C geschrieben und lässt sich mithilfe einer Vielzahl von Optionen an die Bedürfnisse der Benutzer anpassen.

Das Format **critcl** ist beispielsweise als vordefinierte Konfiguration dieser Optionen auf Basis des Generators für **c** implementiert.

Der Großteil eines solchen Rahmens muss über die Option **-template** festgelegt werden. Die zusätzlichen Optionen

- **-fun-qualifier** string
- **-main** string
- **-namespace** string
- **-prelude** string
- **-self-command** string
- **-state-decl** string
- **-state-ref** string
- **-string-varname** string

stellen Codeausschnitte bereit, die den Rahmen und den erzeugten Code verbinden. Die entsprechenden Platzhalter befinden sich im *generierten* Code. Darüber hinaus bieten die Optionen

- **-indent** integer
- **-comments** boolean

Ermöglicht die Anpassung der Code-Einrückung (Standard: keine) sowie die Festlegung, ob Kommentare generiert werden sollen, die die Parsing-Ausdrücke anzeigen, für die eine Funktion vorgesehen ist (Vorgabe on).

Beispiel

Auf ein Beispiel dieser Darstellung wird hier verzichtet. Es wäre für dieses Dokument viel zu umfangreich.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.

3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

   is
   Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

   mode
   Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

   value
   Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

   leaf
   Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

   void
   Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.

1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die

Terminalzeichenkette **x**.

17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_json
Stichwörter: pt_peg_to_json, EBNF, JSON, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name
pt::peg::to::json – PEG-Konvertierung. Schreibt das Format JSON

Übersicht
package require Tcl 8.5 9
package require pt::peg::to::json ?**1.1**?
package require pt::peg
package require json::write
pt::peg::to::json reset
pt::peg::to::json configure
pt::peg::to::json configure *option*
pt::peg::to::json configure *option value...*
pt::peg::to::json convert *serial*

Beschreibung
Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter, der Parsing Expression Grammars in JSON-Markup umwandelt.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::export** bereitgestellten Export-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Export-Plugin **pt::peg::export::json**, das zwischen Konverter und Export-Manager angesiedelt ist.

Bild: arch_core_eplugins

API
Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

pt::peg::to::json reset
Dieser Befehl setzt die Konfiguration des Pakets auf die Standardeinstellungen zurück.

pt::peg::to::json configure
Dieser Befehl gibt ein Dictionary zurück, das die aktuelle Konfiguration des Pakets enthält.

pt::peg::to::json configure *option*
Dieser Befehl gibt den aktuellen Wert der angegebenen *option* des Pakets zurück. Die Liste der zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::json configure *option value*...
Dieser Befehl setzt die angegebenen *option*s des Pakets auf die angegebenen *value*s. Eine Übersicht über die zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::json convert *serial*
Dieser Befehl nimmt die kanonische Serialisierung einer Parsing-Expression-Grammatik entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, und generiert gemäß der aktuellen Paketkonfiguration ein JSON-Markup, das die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Optionen

Der Konverter für das JSON-Grammatik-Austauschformat erkennt die folgenden Variablen der Konfiguration und passt sein Verhalten entsprechend ihrer Festlegung an.

-file string
Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-indented boolean
Wenn diese Option aktiviert ist, teilt das System das generierte JSON-Objekt in Zeilen auf und rückt es entsprechend seiner inneren Struktur ein, wobei jeder Schlüssel eines Dictionarys in einer eigenen Zeile steht.

Ist die Option nicht aktiviert (Standardeinstellung), wird das gesamte JSON-Objekt in einer einzigen Zeile ausgegeben, wobei der Abstand zwischen allen Elementen so gering wie möglich gehalten wird.

-aligned boolean
Wenn diese Option aktiviert ist, sorgt das System dafür, dass die Werte der Schlüssel in einem Dictionary vertikal zueinander ausgerichtet sind, um einen ansprechenden Effekt wie bei einer Tabelle zu erzielen. Damit dies funktioniert, muss zudem die Option **-indented** gesetzt sein.

Wenn die Option nicht gesetzt ist (Standardeinstellung), wird die Ausgabe gemäß dem Wert

von **indented** formatiert, ohne dass versucht wird, die Werte der Dictionary-Schlüssel auszurichten.

JSON-Austauschformat für Grammatiken

Das Format **json** für Parsing Expression Grammars wurde als Datenaustauschformat geschrieben, das nicht an Tcl gebunden ist. Es wurde definiert, um den Austausch von Grammatiken mit PackRat/PEG-basierten Parsergeneratoren für andere Sprachen zu ermöglichen.

Dies wird durch die folgenden Regeln formal festgelegt:

1. Das JSON-Format eines jeden PEG ist ein JSON-Objekt.
2. Dieses Objekt enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
3. Der Inhalt der Grammatik ist ein JSON-Objekt, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein JSON-Objekt, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein JSON-Objekt. Die relevanten Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des Parsing-Ausdrucks enthält, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode

      Der Wert ist ein JSON-Wert mit einem von drei Werten, die festlegen, wie ein Parser den durch das Symbol erzeugten semantischen Wert verarbeiten soll.

      value

      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf

      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void

      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert ist eine JSON-Zeichenkette, die die Tcl-Serialisierung des
    Start-Parsing-Ausdrucks der Grammatik enthält, wie im Abschnitt
    [PE-Serialisierungsformat] beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Als Anmerkung für den fortgeschrittenen Leser: Dies entspricht im Wesentlichen der Tcl-Serialisierung von PEGs, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist, nur dass die Tcl-Dictionaries und Listen dieses Formats auf JSON-Objekte und Arrays abgebildet werden. Lediglich die Parsing-Ausdrücke selbst werden nicht weiter übersetzt, sondern als JSON-Zeichenketten beibehalten, die eine verschachtelte Tcl-Liste enthalten, und auch für das JSON gibt es kein Konzept der Kanonizität.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Eine JSON-Serialisierung dafür lautet

```
{
    "pt::grammar::peg" : {
        "rules" : {
            "AddOp"     : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Digit"     : {
                "is"   : "\/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6}
    {t 7} {t 8} {t 9}",
                "mode" : "value"
            },
            "Expression" : {
                "is"   : "\/ {x {t (} {n Expression} {t )}} {x {n
    Factor} {* {x {n MulOp} {n Factor}}}}",
                "mode" : "value"
            },
            "Factor"    : {
                "is"   : "x {n Term} {* {x {n AddOp} {n Term}}}",
                "mode" : "value"
            },
```

```
            "MulOp"     : {
                "is"   : "\/ {t *} {t \/}",
                "mode" : "value"
            },
            "Number"    : {
                "is"   : "x {? {n Sign}} {+ {n Digit}}",
                "mode" : "value"
            },
            "Sign"      : {
                "is"   : "\/ {t -} {t +}",
                "mode" : "value"
            },
            "Term"      : {
                "is"   : "n Number",
                "mode" : "value"
            }
        },
        "start" : "n Expression"
    }
}
```

und eine Tcl-Serialisierung davon lautet

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

Die Ähnlichkeit des Letzteren mit JSON dürfte ziemlich offensichtlich sein.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole

festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein

Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_param

Stichwörter: pt_peg_to_param, EBNF, LL(k), PARAM, PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression,

parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::to::param – PEG-Konvertierung. Schreibt das Format PARAM

Übersicht

package require Tcl 8.5 9
package require pt::peg::to::param ?**1.1**?
package require pt::peg
package require pt::pe
pt::peg::to::param reset
pt::peg::to::param configure
pt::peg::to::param configure *option*
pt::peg::to::param configure *option value...*
pt::peg::to::param convert *serial*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter zur Umwandlung von Parsing Expression Grammars in PARAM-Markup.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::export** bereitgestellten Export-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Export-Plugin **pt::peg::export::param**, das zwischen Konverter und Export-Manager angesiedelt ist.

Bild: arch_core_eplugins

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

pt::peg::to::param reset
Dieser Befehl setzt die Konfiguration des Pakets auf die Standardeinstellungen zurück.

pt::peg::to::param configure
Dieser Befehl gibt ein Dictionary zurück, das die aktuelle Konfiguration des Pakets enthält.

pt::peg::to::param configure *option*
Dieser Befehl gibt den aktuellen Wert der angegebenen *option* des Pakets zurück. Die Liste der zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::param configure *option value...*
Dieser Befehl setzt die angegebenen *option*s des Pakets auf die angegebenen *value*s. Eine Übersicht über die zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::param convert *serial*
Dieser Befehl nimmt die kanonische Serialisierung einer Parsing-Expression-Grammatik

entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, und generiert gemäß der aktuellen Paketkonfiguration eine PARAM-Markup-Kodierung der Grammatik. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Optionen

Der Konverter für das PARAM-Markup erkennt die folgenden Konfigurationsvariablen und passt sein Verhalten entsprechend deren Vorgaben an.

-template string

Der Wert dieser Konfigurationsvariablen ist eine Zeichenkette, in die der generierte Text und die übrigen Konfigurationseinstellungen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mithilfe der unten aufgeführten Platzhalter angegeben werden. Der Standardwert lautet "**@code@**".

@user@
Ist durch den Wert der Konfigurationsvariablen **-user** zu ersetzen.

@format@
Ist durch die Konstante **PARAM** zu ersetzen.

@file@
Ist durch den Wert der Konfigurationsvariablen **-file** zu ersetzen.

@name@
Ist durch den Wert der Konfigurationsvariablen **-name** zu ersetzen.

@code@
Wird durch den generierten Text ersetzt.

-name string

Der Wert dieser Konfigurationsvariablen ist der Name der Grammatik, für die die Konvertierung durchgeführt wird. Der Standardwert lautet **a_pe_grammar**.

-user string

Der Wert dieser Konfigurationsvariablen ist der Name des Benutzers, für den die Konvertierung durchgeführt wird. Der Standardwert lautet **unknown**.

-file string

Der Wert dieser Konfigurationsvariablen ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die die Konvertierung durchgeführt wird. Der Standardwert lautet **unknown**.

PARAM-Code-Darstellung von Parsing-Expression-Grammatiken

Die PARAM-Code-Darstellung von Parsing-Ausdruck-Grammatiken ist ein Assembler-ähnlicher Text, der die in der *PackRat Machine Specification* dokumentierten Befehle der virtuellen Maschine verwendet, ergänzt um einige weitere Befehle zur Steuerung des Programmablaufs (jump ok, jump fail, call symbol, return).

Es ist nicht wirklich nützlich, außer vielleicht als Hilfsmittel, um zu veranschaulichen, wie eine Grammatik im Allgemeinen kompiliert wird, ohne sich von den Nebenaspekten eines Rahmens ablenken zu lassen – also ähnlich wie der unterstützende C- und Tcl-Code, der von den anderen von PARAM abgeleiteten Formaten generiert wird.

Es gibt keine direkte formale Spezifikation, die über das oben Gesagte hinausgeht.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Eine mögliche PARAM-Serialisierung hierfür lautet

```
# -*- text -*-
# Parsing Expression Grammar 'TEMPLATE'.
# Generated for unknown, from file 'TEST'

#
# Grammar Start Expression
#

<<MAIN>>:
         call              sym_Expression
         halt

#
# value Symbol 'AddOp'
#

sym_AddOp:
# /
#     '-'
#     '+'

         symbol_restore    AddOp
  found! jump              found_7
         loc_push

         call              choice_5

   fail! value_clear
     ok! value_leaf        AddOp
         symbol_save       AddOp
         error_nonterminal AddOp
```

```
         loc_pop_discard

found_7:
     ok! ast_value_push
         return

choice_5:
# /
#     '-'
#     '+'

         error_clear

         loc_push
         error_push

         input_next        "t -"
     ok! test_char         "-"

         error_pop_merge
     ok! jump              oknoast_4

         loc_pop_rewind
         loc_push
         error_push

         input_next        "t +"
     ok! test_char         "+"

         error_pop_merge
     ok! jump              oknoast_4

         loc_pop_rewind
         status_fail
         return

oknoast_4:
         loc_pop_discard
         return
#
# value Symbol 'Digit'
#

sym_Digit:
# /
#     '0'
```

```
#     '1'
#     '2'
#     '3'
#     '4'
#     '5'
#     '6'
#     '7'
#     '8'
#     '9'

         symbol_restore    Digit
  found! jump              found_22
         loc_push

         call              choice_20

   fail! value_clear
     ok! value_leaf        Digit
         symbol_save       Digit
         error_nonterminal Digit
         loc_pop_discard

found_22:
     ok! ast_value_push
         return

choice_20:
# /
#     '0'
#     '1'
#     '2'
#     '3'
#     '4'
#     '5'
#     '6'
#     '7'
#     '8'
#     '9'

         error_clear

         loc_push
         error_push

         input_next        "t 0"
     ok! test_char         "0"
```

```
    error_pop_merge
ok! jump              oknoast_19

    loc_pop_rewind
    loc_push
    error_push

    input_next        "t 1"
ok! test_char         "1"

    error_pop_merge
ok! jump              oknoast_19

    loc_pop_rewind
    loc_push
    error_push

    input_next        "t 2"
ok! test_char         "2"

    error_pop_merge
ok! jump              oknoast_19

    loc_pop_rewind
    loc_push
    error_push

    input_next        "t 3"
ok! test_char         "3"

    error_pop_merge
ok! jump              oknoast_19

    loc_pop_rewind
    loc_push
    error_push

    input_next        "t 4"
ok! test_char         "4"

    error_pop_merge
ok! jump              oknoast_19

    loc_pop_rewind
    loc_push
```

```
        error_push

        input_next        "t 5"
    ok! test_char         "5"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 6"
    ok! test_char         "6"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 7"
    ok! test_char         "7"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 8"
    ok! test_char         "8"

        error_pop_merge
    ok! jump              oknoast_19

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t 9"
    ok! test_char         "9"

        error_pop_merge
```

```
     ok! jump              oknoast_19

         loc_pop_rewind
         status_fail
         return

oknoast_19:
         loc_pop_discard
         return
#
# value Symbol 'Expression'
#

sym_Expression:
# /
#     x
#         '\('
#         (Expression)
#         '\)'
#     x
#         (Factor)
#         *
#             x
#                 (MulOp)
#                 (Factor)

         symbol_restore    Expression
  found! jump              found_46
         loc_push
         ast_push

         call              choice_44

   fail! value_clear
     ok! value_reduce      Expression
         symbol_save       Expression
         error_nonterminal Expression
         ast_pop_rewind
         loc_pop_discard

found_46:
     ok! ast_value_push
         return

choice_44:
# /
```

```
#     x
#         '\('
#         (Expression)
#         '\)'
#     x
#         (Factor)
#         *
#             x
#                 (MulOp)
#                 (Factor)

         error_clear

         ast_push
         loc_push
         error_push

         call              sequence_27

         error_pop_merge
     ok! jump              ok_43

         ast_pop_rewind
         loc_pop_rewind
         ast_push
         loc_push
         error_push

         call              sequence_40

         error_pop_merge
     ok! jump              ok_43

         ast_pop_rewind
         loc_pop_rewind
         status_fail
         return

ok_43:
         ast_pop_discard
         loc_pop_discard
         return

sequence_27:
# x
#     '\('
```

```
#     (Expression)
#     '\)'

         loc_push
         error_clear

         error_push

         input_next        "t ("
      ok! test_char        "("

         error_pop_merge
   fail! jump              failednoast_29
         ast_push
         error_push

         call              sym_Expression

         error_pop_merge
   fail! jump              failed_28
         error_push

         input_next        "t )"
      ok! test_char        ")"

         error_pop_merge
   fail! jump              failed_28

         ast_pop_discard
         loc_pop_discard
         return

failed_28:
         ast_pop_rewind

failednoast_29:
         loc_pop_rewind
         return

sequence_40:
# x
#     (Factor)
#     *
#         x
#             (MulOp)
#             (Factor)
```

```
         ast_push
         loc_push
         error_clear

         error_push

         call              sym_Factor

         error_pop_merge
   fail! jump              failed_41
         error_push

         call              kleene_37

         error_pop_merge
   fail! jump              failed_41

         ast_pop_discard
         loc_pop_discard
         return

failed_41:
         ast_pop_rewind
         loc_pop_rewind
         return

kleene_37:
# *
#     x
#         (MulOp)
#         (Factor)

         loc_push
         error_push

         call              sequence_34

         error_pop_merge
   fail! jump              failed_38
         loc_pop_discard
         jump              kleene_37

failed_38:
         loc_pop_rewind
         status_ok
```

```
         return

sequence_34:
# x
#     (MulOp)
#     (Factor)

         ast_push
         loc_push
         error_clear

         error_push

         call              sym_MulOp

         error_pop_merge
   fail! jump              failed_35
         error_push

         call              sym_Factor

         error_pop_merge
   fail! jump              failed_35

         ast_pop_discard
         loc_pop_discard
         return

failed_35:
         ast_pop_rewind
         loc_pop_rewind
         return
#
# value Symbol 'Factor'
#

sym_Factor:
# x
#     (Term)
#     *
#         x
#             (AddOp)
#             (Term)

         symbol_restore    Factor
  found! jump              found_60
```

```
         loc_push
         ast_push

         call              sequence_57

   fail! value_clear
     ok! value_reduce      Factor
         symbol_save       Factor
         error_nonterminal Factor
         ast_pop_rewind
         loc_pop_discard

found_60:
     ok! ast_value_push
         return

sequence_57:
# x
#     (Term)
#     *
#         x
#             (AddOp)
#             (Term)

         ast_push
         loc_push
         error_clear

         error_push

         call              sym_Term

         error_pop_merge
   fail! jump              failed_58
         error_push

         call              kleene_54

         error_pop_merge
   fail! jump              failed_58

         ast_pop_discard
         loc_pop_discard
         return

failed_58:
```

```
         ast_pop_rewind
         loc_pop_rewind
         return

kleene_54:
# *
#     x
#          (AddOp)
#          (Term)

         loc_push
         error_push

         call              sequence_51

         error_pop_merge
   fail! jump              failed_55
         loc_pop_discard
         jump              kleene_54

failed_55:
         loc_pop_rewind
         status_ok
         return

sequence_51:
# x
#     (AddOp)
#     (Term)

         ast_push
         loc_push
         error_clear

         error_push

         call              sym_AddOp

         error_pop_merge
   fail! jump              failed_52
         error_push

         call              sym_Term

         error_pop_merge
   fail! jump              failed_52
```

```
         ast_pop_discard
         loc_pop_discard
         return

failed_52:
         ast_pop_rewind
         loc_pop_rewind
         return
#
# value Symbol 'MulOp'
#

sym_MulOp:
# /
#     '*'
#     '/'

         symbol_restore    MulOp
  found! jump              found_67
         loc_push

         call              choice_65

   fail! value_clear
     ok! value_leaf        MulOp
         symbol_save       MulOp
         error_nonterminal MulOp
         loc_pop_discard

found_67:
     ok! ast_value_push
         return

choice_65:
# /
#     '*'
#     '/'

         error_clear

         loc_push
         error_push

         input_next        "t *"
     ok! test_char         "*"
```

```
        error_pop_merge
    ok! jump              oknoast_64

        loc_pop_rewind
        loc_push
        error_push

        input_next        "t /"
    ok! test_char         "/"

        error_pop_merge
    ok! jump              oknoast_64

        loc_pop_rewind
        status_fail
        return

oknoast_64:
        loc_pop_discard
        return
#
# value Symbol 'Number'
#

sym_Number:
# x
#     ?
#         (Sign)
#     +
#         (Digit)

        symbol_restore    Number
  found! jump             found_80
        loc_push
        ast_push

        call              sequence_77

   fail! value_clear
    ok! value_reduce      Number
        symbol_save       Number
        error_nonterminal Number
        ast_pop_rewind
        loc_pop_discard
```

```
found_80:
     ok! ast_value_push
         return

sequence_77:
# x
#     ?
#         (Sign)
#     +
#         (Digit)

         ast_push
         loc_push
         error_clear

         error_push

         call              optional_70

         error_pop_merge
   fail! jump              failed_78
         error_push

         call              poskleene_73

         error_pop_merge
   fail! jump              failed_78

         ast_pop_discard
         loc_pop_discard
         return

failed_78:
         ast_pop_rewind
         loc_pop_rewind
         return

optional_70:
# ?
#     (Sign)

         loc_push
         error_push

         call              sym_Sign
```

```
         error_pop_merge
   fail! loc_pop_rewind
     ok! loc_pop_discard
         status_ok
         return

poskleene_73:
# +
#     (Digit)

         loc_push

         call              sym_Digit

   fail! jump              failed_74

loop_75:
         loc_pop_discard
         loc_push
         error_push

         call              sym_Digit

         error_pop_merge
     ok! jump              loop_75
         status_ok

failed_74:
         loc_pop_rewind
         return
#
# value Symbol 'Sign'
#

sym_Sign:
# /
#     '-'
#     '+'

         symbol_restore    Sign
  found! jump              found_86
         loc_push

         call              choice_5

   fail! value_clear
```

```
     ok! value_leaf        Sign
         symbol_save       Sign
         error_nonterminal Sign
         loc_pop_discard

found_86:
     ok! ast_value_push
         return
#
# value Symbol 'Term'
#

sym_Term:
# (Number)

         symbol_restore    Term
  found! jump              found_89
         loc_push
         ast_push

         call              sym_Number

   fail! value_clear
     ok! value_reduce      Term
         symbol_save       Term
         error_nonterminal Term
         ast_pop_rewind
         loc_pop_discard

found_89:
     ok! ast_value_push
         return

#
#
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.

2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.

2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_peg
Stichwörter: pt_peg_to_peg, EBNF, LL(k), PEG, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name
pt::peg::to::peg – PEG-Konvertierung. Schreibt das Format PEG

Übersicht
package require Tcl 8.5 9
package require pt::peg::to::peg ?**1.0.3**?
package require pt::peg
package require pt::pe
package require text::write
pt::peg::to::peg reset
pt::peg::to::peg configure
pt::peg::to::peg configure *option*
pt::peg::to::peg configure *option value...*
pt::peg::to::peg convert *serial*

Beschreibung
Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter, der Parsing Expression Grammars in PEG-Markup umwandelt.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::export** bereitgestellten Export-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Export-Plugin **pt::peg::export::peg**, das zwischen Konverter und Export-Manager angesiedelt ist.

Bild: arch_core_eplugins

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

pt::peg::to::peg reset
Dieser Befehl setzt die Konfiguration des Pakets auf die Standardeinstellungen zurück.

pt::peg::to::peg configure
Dieser Befehl gibt ein Dictionary zurück, das die aktuelle Konfiguration des Pakets enthält.

pt::peg::to::peg configure *option*
Dieser Befehl gibt den aktuellen Wert der angegebenen *option* des Pakets zurück. Die Liste der zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::peg configure *option value*...
Dieser Befehl setzt die angegebenen *option*s des Pakets auf die angegebenen *value*s. Eine Übersicht über die zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::peg convert *serial*
Dieser Befehl nimmt die kanonische Serialisierung einer Parsing-Expression-Grammatik entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, und generiert gemäß der aktuellen Paketkonfiguration ein PEG-Markup, das die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Optionen

Der Konverter in die PEG-Sprache erkennt die folgenden Optionen und passt sein Verhalten entsprechend der dort festgelegten Vorgabe an.

-file string
Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string
Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-template string
Der Wert dieser Option ist eine Zeichenkette, in die der generierte Text und die Werte der anderen Optionen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mit den unten aufgeführten Platzhaltern angegeben werden. Der Standardwert lautet "**@code@**".

@user@
Ist durch den Wert der Option **-user** zu ersetzen.

@format@
Ist durch die Konstante **PEG** zu ersetzen.

@file@

Ist durch den Wert der Option **-file** zu ersetzen.

@name@
Ist durch den Wert der Option **-name** zu ersetzen.

@code@
Wird durch den generierten Text ersetzt.

PEG-Spezifikationssprache

peg, eine Sprache zur Spezifikation von Parsing-Expression-Grammatiken, soll sowohl für Menschen lesbar als auch schreibbar sein, dabei jedoch streng genug, um eine maschinelle Verarbeitung zu ermöglichen. Wie jede Computersprache. Sie wurde definiert, um das Verfassen einer Grammatik-Spezifikation zu vereinfachen – etwas, wofür sich die anderen in den Parser-Tools vorhandenen Formate nicht eignen.

Sie wird formal durch die unten gezeigte Grammatik festgelegt, die in sich selbst geschrieben ist. Eine Einführung in die Sprache steht im *PEG Language Tutorial*.

```
PEG pe_grammar_for_peg (Grammar)

    # ------------------------------------------------------------------
        # Syntactical constructs

        Grammar         <- WHITESPACE Header Definition* Final EOF ;

        Header          <- PEG Identifier StartExpr ;
        Definition      <- Attribute? Identifier IS Expression
    SEMICOLON ;
        Attribute       <- (VOID / LEAF) COLON ;
        Expression      <- Sequence (SLASH Sequence)* ;
        Sequence        <- Prefix+ ;
        Prefix          <- (AND / NOT)? Suffix ;
        Suffix          <- Primary (QUESTION / STAR / PLUS)? ;
        Primary         <- ALNUM / ALPHA / ASCII / CONTROL / DDIGIT /
    DIGIT
                        /  GRAPH / LOWER / PRINTABLE / PUNCT / SPACE /
    UPPER
                        /  WORDCHAR / XDIGIT
                        / Identifier
                        /  OPEN Expression CLOSE
                        /  Literal
                        /  Class
                        /  DOT
                        ;
        Literal         <- APOSTROPH  (!APOSTROPH  Char)* APOSTROPH
    WHITESPACE
                        /  DAPOSTROPH (!DAPOSTROPH Char)* DAPOSTROPH
    WHITESPACE ;
```

```
        Class           <- OPENB (!CLOSEB Range)* CLOSEB WHITESPACE ;
        Range           <- Char TO Char / Char ;

        StartExpr       <- OPEN Expression CLOSE ;
void:   Final           <- "END" WHITESPACE SEMICOLON WHITESPACE ;

        # ------------------------------------------------------------
    -------
        # Lexing constructs

        Identifier      <- Ident WHITESPACE ;
leaf:   Ident           <- ([_:] / <alpha>) ([_:] / <alnum>)* ;
        Char            <- CharSpecial / CharOctalFull / CharOctalPart
                        /  CharUnicode / CharUnescaped
                        ;

leaf:   CharSpecial     <- "\\" [nrt'"\[\]\\] ;
leaf:   CharOctalFull   <- "\\" [0-3][0-7][0-7] ;
leaf:   CharOctalPart   <- "\\" [0-7][0-7]? ;
leaf:   CharUnicode     <- "\\" 'u' HexDigit (HexDigit (HexDigit
    HexDigit?)?)? ;
leaf:   CharUnescaped   <- !"\\" . ;

void:   HexDigit        <- [0-9a-fA-F] ;

void:   TO              <- '-'           ;
void:   OPENB           <- "["           ;
void:   CLOSEB          <- "]"           ;
void:   APOSTROPH       <- "'"           ;
void:   DAPOSTROPH      <- '"'           ;
void:   PEG             <- "PEG" !([_:] / <alnum>) WHITESPACE ;
void:   IS              <- "<-"    WHITESPACE ;
leaf:   VOID            <- "void"  WHITESPACE ; # Implies that
    definition has no semantic value.
leaf:   LEAF            <- "leaf"  WHITESPACE ; # Implies that
    definition has no terminals.
void:   SEMICOLON       <- ";"     WHITESPACE ;
void:   COLON           <- ":"     WHITESPACE ;
void:   SLASH           <- "/"     WHITESPACE ;
leaf:   AND             <- "&"     WHITESPACE ;
leaf:   NOT             <- "!"     WHITESPACE ;
leaf:   QUESTION        <- "?"     WHITESPACE ;
leaf:   STAR            <- "*"     WHITESPACE ;
leaf:   PLUS            <- "+"     WHITESPACE ;
void:   OPEN            <- "("     WHITESPACE ;
void:   CLOSE           <- ")"     WHITESPACE ;
```

```
leaf:   DOT             <- "."     WHITESPACE ;

leaf:   ALNUM           <- "<alnum>"    WHITESPACE ;
leaf:   ALPHA           <- "<alpha>"    WHITESPACE ;
leaf:   ASCII           <- "<ascii>"    WHITESPACE ;
leaf:   CONTROL         <- "<control>"  WHITESPACE ;
leaf:   DDIGIT          <- "<ddigit>"   WHITESPACE ;
leaf:   DIGIT           <- "<digit>"    WHITESPACE ;
leaf:   GRAPH           <- "<graph>"    WHITESPACE ;
leaf:   LOWER           <- "<lower>"    WHITESPACE ;
leaf:   PRINTABLE       <- "<print>"    WHITESPACE ;
leaf:   PUNCT           <- "<punct>"    WHITESPACE ;
leaf:   SPACE           <- "<space>"    WHITESPACE ;
leaf:   UPPER           <- "<upper>"    WHITESPACE ;
leaf:   WORDCHAR        <- "<wordchar>" WHITESPACE ;
leaf:   XDIGIT          <- "<xdigit>"   WHITESPACE ;

void:   WHITESPACE      <- (" " / "\t" / EOL / COMMENT)* ;
void:   COMMENT         <- '#' (!EOL .)* EOL ;
void:   EOL             <- "\n\r" / "\n" / "\r" ;
void:   EOF             <- !. ;

        # ------------------------------------------------------------
    -------
END;
```

Beispiel

Unser Beispiel legt die Grammatik eines einfachen Taschenrechners mit vier Grundrechenarten fest.

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Mit höherwertigen Merkmalen der Notation, also den Zeichenklassen (vordefiniert und selbst definiert), lässt sich dieses Beispiel so umschreiben:

```
PEG calculator (Expression)
    Sign       <- [-+]                                  ;
    Number     <- Sign? <ddigit>+                       ;
    Expression <- Term (AddOp Term)*                    ;
    MulOp      <- [*/]                      ;
    Term       <- Factor (MulOp Factor)*                ;
    AddOp      <- [-+]                      ;
    Factor     <- '(' Expression ')' / Number           ;
END;
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode

      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value

      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf

      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void

Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
```

```
                                    mode value}
        MulOp       {is {/ {t *} {t /}}
                                    mode value}
        Number      {is {x {? {n Sign}} {+ {n Digit}}}
                                    mode value}
        Sign        {is {/ {t -} {t +}}
                                    mode value}
        Term        {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden

Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_peg_to_tclparam

Stichwörter: pt_peg_to_tclparam, EBNF, LL(k), PEG, TCLPARAM, TDPL, context-free languages, conversion, expression, format conversion, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, serialization, state, top-down parsing languages, transducer

Name

pt::peg::to::tclparam – PEG-Konvertierung. Schreibt das Format TCLPARAM

Übersicht

package require Tcl 8.5 9
package require pt::peg::to::tclparam ?**1.0.4**?
pt::peg::to::tclparam reset

pt::peg::to::tclparam configure
pt::peg::to::tclparam configure *option*
pt::peg::to::tclparam configure *option value...*
pt::peg::to::tclparam convert *serial*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket implementiert den Konverter zur Umwandlung von Parsing Expression Grammars in TCLPARAM-Markup.

Es befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools und kann entweder direkt mit den anderen Paketen dieser Schicht oder indirekt über den von **pt::peg::export** bereitgestellten Export-Manager verwendet werden. Letzteres ist für den Einsatz in nicht vertrauenswürdigen Umgebungen vorgesehen und erfolgt über das entsprechende Export-Plugin **pt::peg::export::tclparam**, das zwischen Konverter und Export-Manager angesiedelt ist.

Bild: arch_core_eplugins

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Converter-API, die in der Spezifikation *Parser Tools Export API* zu finden ist.

pt::peg::to::tclparam reset
Dieser Befehl setzt die Konfiguration des Pakets auf die Standardeinstellungen zurück.

pt::peg::to::tclparam configure
Dieser Befehl gibt ein Dictionary zurück, das die aktuelle Konfiguration des Pakets enthält.

pt::peg::to::tclparam configure *option*
Dieser Befehl gibt den aktuellen Wert der angegebenen *option* des Pakets zurück. Die Liste der zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::tclparam configure *option value...*
Dieser Befehl setzt die angegebenen *option*s des Pakets auf die angegebenen *value*s. Eine Übersicht über die zulässigen Optionen finden Sie im Abschnitt Optionen.

pt::peg::to::tclparam convert *serial*
Dieser Befehl nimmt die kanonische Serialisierung einer Parsing-Expression-Grammatik entgegen, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist, und generiert gemäß der aktuellen Paketkonfiguration ein TCLPARAM-Markup, das die Grammatik kodiert. Die erzeugte Zeichenkette wird anschließend als Ergebnis des Befehls zurückgegeben.

Optionen

Der Konverter für das Tcl/PARAM-Markup erkennt die folgenden Konfigurationsvariablen und passt sein Verhalten entsprechend deren Vorgaben an.

-template string
Der Wert dieser Konfigurationsvariablen ist eine Zeichenkette, in die der generierte Text und die übrigen Konfigurationseinstellungen eingefügt werden sollen. Es wird erwartet, dass die verschiedenen Speicherorte für Benutzerdaten mithilfe der unten aufgeführten Platzhalter

angegeben werden. Der Standardwert lautet "**@code@**".

@user@
Ist durch den Wert der Konfigurationsvariablen **-user** zu ersetzen.

@format@
Ist durch die Konstante **Tcl/PARAM** zu ersetzen.

@file@
Ist durch den Wert der Konfigurationsvariablen **-file** zu ersetzen.

@name@
Ist durch den Wert der Konfigurationsvariablen **-name** zu ersetzen.

@code@
Wird durch den generierten Tcl-Code ersetzt.

Die folgenden Konfigurationsvariablen sind insofern besonders, als sie im generierten Code vorkommen und dort ebenfalls ersetzt werden.

@runtime@
Ist durch den Wert der Konfigurationsvariablen **runtime-command** zu ersetzen.

@self@
Ist durch den Wert der Konfigurationsvariablen **self-command** zu ersetzen.

@def@
Ist durch den Wert der Konfigurationsvariablen **proc-command** zu ersetzen.

@ns@
Ist durch den Wert der Konfigurationsvariablen **namespace** zu ersetzen.

@main@
Ist durch den Wert der Konfigurationsvariablen **main** zu ersetzen.

@prelude@
Ist durch den Wert der Konfigurationsvariablen **prelude** zu ersetzen.

-name string
Der Wert dieser Konfigurationsvariablen ist der Name der Grammatik, für die die Konvertierung durchgeführt wird. Der Standardwert lautet **a_pe_grammar**.

-user string
Der Wert dieser Konfigurationsvariablen ist der Name des Benutzers, für den die Konvertierung durchgeführt wird. Der Standardwert lautet **unknown**.

-file string
Der Wert dieser Konfigurationsvariablen ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die die Konvertierung durchgeführt wird. Der Standardwert lautet **unknown**.

-runtime-command string
Eine Tcl-Zeichenkette, die den Tcl-Befehl oder einen Verweis darauf darstellt, der zum Aufruf der PARAM-Anweisung aus Parser-Prozeduren gemäß dem gewählten Rahmen (Template) verwendet wird. Der Standardwert ist die leere Zeichenkette.

-self-command string
Eine Tcl-Zeichenkette, die den Tcl-Befehl oder einen Verweis darauf darstellt, der verwendet wird, um die Parser-Prozeduren (Methoden ...) aus einer anderen Parser-Prozedur heraus aufzurufen, entsprechend dem gewählten Rahmen (Template). Der Standardwert ist die leere

Zeichenkette.

-proc-command string
Der Name des Tcl-Befehls, der zur Definition von Prozeduren (Methoden ...) verwendet wird, je nach gewähltem Rahmen (Template). Der Standardwert lautet **proc**.

-namespace string
Der Name des Namespaces, in dem sich die Parser-Prozeduren (Methoden, ...) befinden sollen, einschließlich des abschließenden "::", das zur Trennung vom eigentlichen Prozedurnamen erforderlich ist. Der Standardwert ist **::**.

-main string
Der Name der Hauptprozedur (Methode, ...), die vom ausgewählten Rahmen (Template) aufgerufen werden soll, um mit der Analyse der Eingabe zu beginnen. Der Standardwert lautet **__main**.

-prelude string
Ein Codeausschnitt, der am Anfang jedes generierten Parsing-Befehls eingefügt werden soll. Der Standardwert ist die leere Zeichenkette.

-indent integer
Die Anzahl der Zeichen, um die jede Zeile des generierten Codes eingerückt werden soll. Der Standardwert lautet **0**.

Die hohe Parametrierbarkeit dieses Konverters, die sich in der Vielzahl der unterstützten Optionen zeigt, ist zwar für fortgeschrittene Benutzer ein Vorteil, da sie die Ausgabe des Konverters nach Bedarf anpassen können, doch für Anfänger wird der Wald vor lauter Bäumen wahrscheinlich nicht mehr zu erkennen sein.

Um diesen Anwendern zu helfen, werden zwei Zusatzpakete bereitgestellt, die jeweils eine vorgefertigte Konfiguration enthalten, mit der sich sofort einsatzbereite vollständige Parser generieren lassen. Diese sind

pt::tclparam::configuration::snit
Die generierten Parser sind Klassen, die auf dem Paket **snit** basieren, d. h. auf snit::type.

pt::tclparam::configuration::tcloo
Die generierten Parser sind Klassen, die auf dem Paket **OO** basieren.

Tcl/PARAM-Code-Darstellung von Parsing-Expression-Grammatiken

Die Tcl/PARAM-Darstellung von Parsing Expression Grammars ist Tcl-Code, dessen Ausführung die Eingabe gemäß der Grammatik analysiert. Der Code basiert auf der in der *PackRat Machine Specification* dokumentierten virtuellen Maschine und verwendet deren Befehle sowie einige zusätzliche Befehle zur Steuerung des Kontrollflusses.

Hinweis: Der generierte Code für sich genommen ist nicht funktionsfähig. Er muss in einen Rahmen eingebettet werden, der Dienste wie den PARAM-Zustand, Implementierungen für die PARAM-Befehle usw. bereitstellt. Der Großteil eines solchen Rahmens muss über die Option **-template** festgelegt werden. Die zusätzlichen Optionen

- **-indent** integer
- **-main** string
- **-namespace** string
- **-prelude** string
- **-proc-command** string

- **-runtime-command** string
- **-self-command** string

stellen Codeausschnitte bereit, die den Rahmen und den erzeugten Code verbinden. Die entsprechenden Platzhalter befinden sich im *generierten* Code.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode

      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value

      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf

      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void

      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}               mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
```

```
                                    mode value}
        Sign       {is {/ {t -} {t +}}
                                    mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                    mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der

Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.

10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch

sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.
1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pegrammar

Stichwörter: pt_pegrammar, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::peg – Serialisierung der Parsing Expression Grammar

Übersicht

package require Tcl 8.5 9
package require pt::peg ?**1.1**?
package require pt::pe
::pt::peg verify *serial* ?*canonvar*?
::pt::peg verify-as-canonical *serial*
::pt::peg canonicalize *serial*
::pt::peg print *serial*

::pt::peg merge *seriala serialb*
::pt::peg equal *seriala serialb*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt Befehle zur Verfügung, für die Serialisierungen von Parsing Expression Grammars, wie die Parser-Tools sie verwalten und wie der Abschnitt PEG-Serialisierungsformat sie beschreibt.

Dies ist ein unterstützendes Paket in der Kernschicht der Parser-Tools.

Bild: arch_core_support

API

::pt::peg verify *serial* ?*canonvar*?

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige Serialisierung eines Parsing-Ausdrucks ist, und löst einen Fehler aus, wenn dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Wenn das Argument *canonvar* festgelegt wird, wird es als Name einer Variablen im aufrufenden Kontext interpretiert. In diese Variable wird genau dann geschrieben, wenn *serial* eine gültige reguläre Serialisierung ist. Ihr Wert ist ein Boolescher Wert, wobei **True** angibt, dass die Serialisierung nicht nur gültig, sondern auch *kanonisch* ist. Bei einer gültigen, aber nicht kanonischen Serialisierung wird **False** geschrieben.

Informationen zur Spezifikation von Serialisierungen finden Sie im Abschnitt PE-Serialisierungsformat.

::pt::peg verify-as-canonical *serial*

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige *kanonische* Serialisierung einer PEG darstellt, und löst einen Fehler aus, falls dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Informationen zur Spezifikation kanonischer Serialisierungen finden Sie im Abschnitt PEG-Serialisierungsformat.

::pt::peg canonicalize *serial*

Dieser Befehl geht davon aus, dass der Inhalt von *serial* eine gültige *reguläre* Serialisierung einer PEG ist, und löst einen Fehler aus, wenn dies nicht der Fall ist.

Anschließend wandelt er die Eingabe in die *kanonische* Serialisierung der enthaltenen PEG um und gibt diese als Ergebnis zurück. Ist die Eingabe bereits kanonisch, wird sie unverändert zurückgegeben.

Zur Spezifikation regulärer und kanonischer Serialisierungen siehe den Abschnitt PEG-Serialisierungsformat.

::pt::peg print *serial*

Dieser Befehl geht davon aus, dass das Argument *serial* eine gültige Serialisierung eines Parsing-Ausdrucks enthält, und gibt eine Zeichenkette zurück, die diesen PE in einer für Menschen lesbaren Form enthält.

Das genaue Format dieser Darstellung wird nicht festgelegt und darf nicht für das Parsen oder andere maschinengestützte Vorgänge herangezogen werden.

Informationen zur Spezifikation von Serialisierungen finden Sie im Abschnitt PEG-Serialisierungsformat.

::pt::peg merge *seriala serialb*

Dieser Befehl akzeptiert die regulären Serialisierungen zweier Grammatiken und erstellt daraus deren Vereinigung. Das Ergebnis des Befehls ist die kanonische Serialisierung dieser vereinigten Grammatik.

Ein Zusammenführungsfehler tritt auf, wenn für ein beliebiges nichtterminales Symbol S, das in beiden Eingabegrammatiken vorkommt, die beiden Eingabegrammatiken unterschiedliche semantische Modi festlegen.

Der semantische Modus jedes nichtterminalen Symbols S ist der semantische Modus von S in einer seiner Eingabegrammatiken. Die vorherige Regel stellte sicher, dass diese Werte für Symbole, die in beiden Grammatiken vorkommen, identisch sind.

Die rechte Seite jedes nichtterminalen Symbols S, das in beiden Eingabegrammatiken vorkommt, ist die Wahl zwischen den rechten Seiten von S in den Eingabegrammatiken, wobei der Parsing-Ausdruck von S in *seriala* an erster Stelle steht, es sei denn, beide Ausdrücke sind identisch. In diesem Fall wird der erste Ausdruck übernommen.

Die rechte Seite jedes nichtterminalen Symbols S, das nur in einer der Eingabegrammatiken vorkommt, ist die rechte Seite von S in der jeweiligen Eingabegrammatik.

Der Startausdruck der vereinheitlichten Grammatik ist die Auswahl zwischen den Startausdrücken der Eingabegrammatiken, wobei der Startausdruck von *seriala* an erster Stelle steht, es sei denn, beide Ausdrücke sind identisch. In diesem Fall wird der erste Ausdruck übernommen.

::pt::peg equal *seriala serialb*

Dieser Befehl prüft die beiden Grammatiken *seriala* und *serialb* auf strukturelle Übereinstimmung. Das Ergebnis des Befehls ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn die Ausdrücke identisch sind, andernfalls auf **false**.

Die Zeichenfolgenübereinstimmung ist nur dann anwendbar, wenn davon ausgegangen werden kann, dass es sich bei den beiden Grammatiken um reine Tcl-Listen und -Dictionaries handelt.

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules
   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.
   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is
      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

      mode
      Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

      value
      Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

      leaf
      Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

      void
      Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung

Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}              mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                mode value}
        MulOp      {is {/ {t *} {t /}}
                                mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                mode value}
        Sign       {is {/ {t -} {t +}}
                                mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf

der Grundlage des eingebauten Tcl-Befehls **string is**.

13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pexpr_op

Stichwörter: pt_pexpr_op, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::pe::op – Hilfsbefehle für Parsing-Ausdrücke

Übersicht

package require Tcl 8.5 9
package require pt::pe::op ?**1.0.2**?
package require pt::pe ?**1**?
package require struct::set
::pt::pe::op drop *dropset pe*
::pt::pe::op rename *nt ntnew pe*
::pt::pe::op called *pe*
::pt::pe::op flatten *pe*
::pt::pe::op fusechars *pe*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt zusätzliche Befehle zur Verfügung, um mit den Serialisierungen von Parsing-Ausdrücken zu arbeiten, die vom PEG und den zugehörigen Paketen verwaltet und im Abschnitt PE-Serialisierungsformat beschrieben werden.

Es handelt sich hierbei um ein internes Paket, das für übergeordnete Pakete bestimmt ist, welche sich mit PEGs befassen, deren Konvertierung in und aus verschiedenen anderen Formaten durchführen oder für andere Zwecke nutzen.

API

::pt::pe::op drop *dropset pe*
: Dieser Befehl entfernt alle Vorkommen der Nichtterminalsymbole aus der Menge *dropset* aus dem Parsing-Ausdruck *pe* und vereinfacht diesen. Dies kann dazu führen, dass der Ausdruck zu "epsilon" wird, d. h. dass er auf nichts mehr passt.

::pt::pe::op rename *nt ntnew pe*
: Dieser Befehl benennt alle Vorkommen des Nichtterminals *nt* im Parsing-Ausdruck *pe* in *ntnew* um.

::pt::pe::op called *pe*
: Dieser Befehl extrahiert die Menge aller nichtterminalen Symbole, die im Parsing-Ausdruck *pe* verwendet, d. h. "aufgerufen", werden.

::pt::pe::op flatten *pe*
: Dieser Befehl führt eine Transformation des Parsing-Ausdrucks durch, indem er in Sequenzen verschachtelte Sequenzen sowie in Auswahlmöglichkeiten verschachtelte Auswahlmöglichkeiten entfernt und die Kindknoten des verschachtelten Ausdrucks in den Elternknoten hebt. Zudem werden alle Sequenzen und Auswahlen mit nur einem Kindknoten entfernt, da diese überflüssig sind.

  Der resultierende Parsing-Ausdruck wird als Ergebnis des Befehls zurückgegeben.

::pt::pe::op fusechars *pe*
: Dieser Befehl transformiert den Parsing-Ausdruck, indem er benachbarte Terminale in Sequenzen sowie benachbarte Terminale und Bereiche in Auswahlmöglichkeiten zusammenführt; und setzt dabei höherstufige *strings* und *character classes* wieder zusammen.

  Der entstehende Pseudo-Parsing-Ausdruck ist das Ergebnis des Befehls und kann die Pseudo-Operatoren **str** für Zeichenfolgen, also strings, und **cl** für Zeichenauswahlen, also character classes, enthalten.

  Das Ergebnis wird als *Pseudo-Parsing-Ausdruck* bezeichnet, da es sich nicht mehr um einen echten Parsing-Ausdruck handelt und eine Überprüfung mit ::pt::peg verify fehlschlägt, wenn die neuen Pseudo-Operatoren im Ergebnis vorhanden sind; ansonsten weist es jedoch eine für einen Parsing-Ausdruck korrekte Struktur auf. Insbesondere werden die Befehle ::pt::peg bottomup und ::pt::peg topdown diese ohne Probleme verarbeiten.

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.
1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pexpression
Stichwörter: pt_pexpression, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name
pt::pe – Serialisierung von Parsing-Ausdrücken

Übersicht
package require Tcl 8.5 9
package require pt::pe ?**1.0.2**?
package require char
::pt::pe verify *serial* ?*canonvar*?
::pt::pe verify-as-canonical *serial*
::pt::pe canonicalize *serial*
::pt::pe print *serial*
::pt::pe bottomup *cmdprefix pe*
cmdprefix *pe op arguments*
::pt::pe topdown *cmdprefix pe*
::pt::pe equal *seriala serialb*
::pt::pe epsilon
::pt::pe dot
::pt::pe alnum
::pt::pe alpha
::pt::pe ascii
::pt::pe control
::pt::pe digit
::pt::pe graph
::pt::pe lower
::pt::pe print
::pt::pe punct
::pt::pe space
::pt::pe upper

::pt::pe wordchar
::pt::pe xdigit
::pt::pe ddigit
::pt::pe terminal *t*
::pt::pe range *ta tb*
::pt::pe nonterminal *nt*
::pt::pe choice *pe...*
::pt::pe sequence *pe...*
::pt::pe repeat0 *pe*
::pt::pe repeat1 *pe*
::pt::pe optional *pe*
::pt::pe ahead *pe*
::pt::pe notahead *pe*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt Befehle zur Verfügung, mit denen Sie die von den Parser-Tools verwalteten und im Abschnitt PE-Serialisierungsformat beschriebenen Serialisierungen von Parsing-Ausdrücken bearbeiten können.

Dies ist ein unterstützendes Paket in der Kernschicht der Parser-Tools.

Bild: arch_core_support

API

::pt::pe verify *serial* ?*canonvar*?

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige Serialisierung eines Parsing-Ausdrucks ist, und löst einen Fehler aus, wenn dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Wenn das Argument *canonvar* festgelegt wird, wird es als Name einer Variablen im aufrufenden Kontext interpretiert. In diese Variable wird genau dann geschrieben, wenn *serial* eine gültige reguläre Serialisierung ist. Ihr Wert ist ein Boolescher Wert, wobei **True** angibt, dass die Serialisierung nicht nur gültig, sondern auch *kanonisch* ist. Bei einer gültigen, aber nicht kanonischen Serialisierung wird **False** geschrieben.

Informationen zur Spezifikation von Serialisierungen finden Sie im Abschnitt PE-Serialisierungsformat.

::pt::pe verify-as-canonical *serial*

Dieser Befehl überprüft, ob der Inhalt von *serial* eine gültige *kanonische* Serialisierung eines Parsing-Ausdrucks ist, und löst einen Fehler aus, falls dies nicht der Fall ist. Das Ergebnis des Befehls ist die leere Zeichenkette.

Informationen zur Spezifikation kanonischer Serialisierungen finden Sie im Abschnitt PE-Serialisierungsformat.

::pt::pe canonicalize *serial*

Dieser Befehl geht davon aus, dass der Inhalt von *serial* eine gültige *reguläre* Serialisierung eines Parsing-Ausdrucks ist, und löst einen Fehler aus, falls dies nicht der Fall ist.

Anschließend wandelt er die Eingabe in die *kanonische* Serialisierung dieses Parsing-Ausdrucks um und gibt diese als Ergebnis zurück. Wenn die Eingabe bereits kanonisch ist, wird sie unverändert zurückgegeben.

Informationen zur Spezifikation regulärer und kanonischer Serialisierungen finden Sie im Abschnitt PE-Serialisierungsformat.

::pt::pe print *serial*

Dieser Befehl geht davon aus, dass das Argument *serial* eine gültige Serialisierung eines Parsing-Ausdrucks enthält, und gibt eine Zeichenkette zurück, die diesen PE in einer für Menschen lesbaren Form enthält.

Das genaue Format dieser Darstellung ist nicht spezifiziert und darf nicht für das Parsen oder andere maschinengestützte Vorgänge herangezogen werden.

Informationen zur Spezifikation von Serialisierungen finden Sie im Abschnitt PE-Serialisierungsformat.

::pt::pe bottomup *cmdprefix pe*

Dieser Befehl durchläuft den Parsing-Ausdruck *pe* von unten nach oben bis zur Wurzel und ruft dabei für jeden Teilausdruck das Befehlspräfix *cmdprefix* auf. Dies bedeutet, dass die Kindknoten eines Parsing-Ausdrucks PE vor PE verarbeitet werden.

Das Befehlspräfix hat die Signatur

cmdprefix *pe op arguments*

Das heißt, er wird mit dem Parsing-Ausdruck *pe*, an dem sich der Durchlauf gerade befindet, dem Operator *op* in diesem *pe* sowie den *arguments* des Operators aufgerufen.

Das vom Befehlspräfix zurückgegebene Ergebnis ersetzt *pe* in dem Parsing-Ausdruck, dessen Kindknoten er war, wodurch Transformationen des Ausdrucksbaums ermöglicht werden.

Dies bedeutet auch, dass für alle inneren Parsing-Ausdrücke der Inhalt von *arguments* den Ergebnissen des Befehlspräfixes entspricht, das für die Kindknoten dieses inneren Parsing-Ausdrucks aufgerufen wurde.

::pt::pe topdown *cmdprefix pe*

Dieser Befehl durchläuft den Parsing-Ausdruck *pe* von der Wurzel bis zu den Blättern und ruft dabei für jeden Teilausdruck das Befehlspräfix *cmdprefix* auf. Dies bedeutet, dass die Kindknoten eines Parsing-Ausdrucks PE nach PE verarbeitet werden.

Das Befehlspräfix hat dieselbe Signatur wie bei **bottomup** (siehe oben).

Das vom Befehlspräfix zurückgegebene Ergebnis wird *ignoriert*.

::pt::pe equal *seriala serialb*

Dieser Befehl prüft die beiden Parsing-Ausdrücke *seriala* und *serialb* auf strukturelle Übereinstimmung. Das Ergebnis des Befehls ist ein boolescher Wert. Er wird auf **true** gesetzt, wenn die Ausdrücke identisch sind, andernfalls auf **false**.

Die Zeichenfolgenübereinstimmung ist nur dann anwendbar, wenn davon ausgegangen werden kann, dass es sich bei den beiden Parsing-Ausdrücken um reine Tcl-Listen handelt.

::pt::pe epsilon
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für Epsilon.

::pt::pe dot
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für dot.

::pt::pe alnum
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für alnum.

::pt::pe alpha
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für alpha.

::pt::pe ascii
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für ascii.

::pt::pe control
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für control.

::pt::pe digit
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für digit.

::pt::pe graph
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für graph.

::pt::pe lower
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für lower.

::pt::pe print
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für print.

::pt::pe punct
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für punct.

::pt::pe space
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für space.

::pt::pe upper
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für upper.

::pt::pe wordchar
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für wordchar.

::pt::pe xdigit
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für xdigit.

::pt::pe ddigit
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für ddigit.

::pt::pe terminal *t*
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für das Terminalsymbol *t*.

::pt::pe range *ta tb*
Dieser Befehl erstellt den atomaren Parsing-Ausdruck für den Bereich der Terminalsymbole *ta* … *tb*.

::pt::pe nonterminal *nt*

Dieser Befehl erstellt den atomaren Parsing-Ausdruck für das nichtterminale Symbol *nt*.

::pt::pe choice *pe...*
Dieser Befehl erstellt den Parsing-Ausdruck für die geordnete, priorisierte Auswahl zwischen den Parsing-Ausdrücken der Argumente. Das erste Argument hat die höchste Priorität.

::pt::pe sequence *pe...*
Dieser Befehl erstellt den Parsing-Ausdruck für die Sequenz der Parsing-Ausdrücke in den Argumenten. Das erste Argument ist das erste Element der Sequenz.

::pt::pe repeat0 *pe*
Dieser Befehl erstellt den Parsing-Ausdruck für die Wiederholung des Parsing-Ausdrucks *pe*, null Mal oder öfter, auch Kleene-Abschluss genannt.

::pt::pe repeat1 *pe*
Dieser Befehl erstellt den Parsing-Ausdruck für die Wiederholung des Parsing-Ausdrucks *pe*, ein Mal oder öfter, auch positiver Kleene-Abschluss genannt.

::pt::pe optional *pe*
Dieser Befehl erstellt den Parsing-Ausdruck für die Optionalität des Parsing-Ausdrucks *pe*.

::pt::pe ahead *pe*
Dieser Befehl erstellt den Parsing-Ausdruck für die positive Vorausschau des Parsing-Ausdrucks *pe*.

::pt::pe notahead *pe*
Dieser Befehl erstellt den Parsing-Ausdruck für die negative Vorausschau des Parsing-Ausdrucks *pe*.

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung
siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der

Grundlage des eingebauten Tcl-Befehls **string is**.

6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list / **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein

Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_pgen

Stichwörter: pt_pgen, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton,

recursive descent, state, top-down parsing languages, transducer

Name

pt::pgen – Parser-Generator

Übersicht

package require Tcl 8.5 9
package require pt::pgen ?**1.2**?
::pt::pgen *inputformat text resultformat* ?*options...*?

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt einen Befehl bereit, der einen *Parser-Generator* implementiert, der Parsing Expression Grammars als Eingabe verwendet.

Es handelt sich um die Implementierung der Methode **generate** von **pt**, der *Parser Tools-Anwendung*.

Daher richtet sich dieses Dokument an Personen, die diesen Teil der Funktionalität von **pt** ändern und/oder erweitern möchten. Benutzer von **pt** werden hingegen auf die Manpage der Anwendung verwiesen, d. h. *Parser Tools Application*.

Es liegt in der Schicht der Benutzerpakete der Parser-Tools.

Bild: arch_user_pkg

API

::pt::pgen *inputformat text resultformat* ?*options...*?

Dieser Befehl nimmt die Parsing Expression Grammar in *text* (in dem durch *inputformat* festgelegten Format) entgegen und gibt dieselbe Grammatik im Format *resultformat* als Ergebnis des Befehls zurück.

Die beiden bekannten Eingabeformate sind **peg** und **json**. Eine Einleitung in diese Formate, einschließlich ihrer formalen Spezifikationen, finden Sie im *PEG Language Tutorial* und in *The JSON Grammar Exchange Format*. Die zur Analyse dieser Formate verwendeten Pakete sind

peg
pt::peg::from::peg

json
pt::peg::from::json

Auf der Ausgabeseite sind die bekannten Formate und die Pakete, die sie erzeugen:

c
pt::peg::to::cparam

container
pt::peg::to::container

critcl
pt::peg::to::cparam + **pt::cparam::configuration::critcl**

json

pt::peg::to::json

oo
pt::peg::to::tclparam + **pt::tclparam::configuration::tcloo**

peg
pt::peg::to::peg

snit
pt::peg::to::tclparam + **pt::tclparam::configuration::snit**

Die von den einzelnen Formaten unterstützten Optionen sind in den jeweiligen Paketen dokumentiert.

Beispiel

In diesem Abschnitt arbeiten wir ein vollständiges Beispiel durch, beginnend mit einer PEG-Grammatik und endend mit der Ausführung des daraus generierten Parsers auf einer Eingabe, wobei wir der in der folgenden Abbildung dargestellten Gliederung folgen:

Bild: flow

Unsere Grammatik, von der angenommen wird, dass sie in der Datei "*calculator.peg*" gespeichert ist, lautet wie folgt:

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
    Term       <- Factor (MulOp Factor)*                       ;
    AddOp      <- '+'/'-'                                      ;
    Factor     <- '(' Expression ')' / Number                  ;
END;
```

Daraus entsteht mit dem Skript "*gen*" ein auf **snit** beruhender Parser.

```
package require Tcl 8.5
package require fileutil
package require pt::pgen

lassign $argv name
set grammar [fileutil::cat $name.peg]
set pclass  [pt::pgen peg $gr snit -class $name -file  $name.peg -name
     $name]
fileutil::writeFile $name.tcl $pclass
exit 0
```

Der Aufruf lautet

```
 tclsh8.5 gen calculator
```

Damit liegen uns nun das Parser-Paket und die Klasse vor, die in die Datei "*calculator.tcl*"

geschrieben wurden. Unter der Annahme, dass dieses Paket anschließend ordnungsgemäß an einem Ort installiert wird, an dem Tcl es finden kann, können wir diese Klasse nun über ein Skript wie das folgende verwenden:

```
    package require calculator

    lassign $argv input
    set channel [open $input r]

    set parser [calculator]
    set ast [$parser parse $channel]
    $parser destroy
    close $channel

    ... now process the returned abstract syntax tree ...
```

wobei der in der Variable gespeicherte abstrakte Syntaxbaum wie folgt aussehen wird:

```
set ast {Expression 0 4
    {Factor 0 4
        {Term 0 2
            {Number 0 2
                {Digit 0 0}
                {Digit 1 1}
                {Digit 2 2}
            }
        }
        {AddOp 3 3}
        {Term 4 4
            {Number 4 4
                {Digit 4 4}
            }
        }
    }
}
```

unter der Annahme, dass die Eingabedatei und der Kanal den Text enthielten

```
 120+5
```

Eine anschaulichere Darstellung des Baums wäre

```
+- Digit 0 0 | 1
|            |
+- Term 0 2 --- Number 0 2 -+- Digit 1 1 | 2
|                          |            |
|                          +- Digit 2 2 | 0
|                                       |
Expression 0 4 --- Factor 0 4 -+----------------------------- AddOp 3
```

```
    3 | +
|                                       |
+- Term 4 4 --- Number 4 4 --- Digit 4 4 | 5
```

Unabhängig davon liegt es an dieser Stelle in der Verantwortung der Benutzerin, mit dem Baum zu arbeiten, um das von ihr angestrebte Ziel zu erreichen. Das heißt, ihn zu analysieren, durch eine Transformation umzuwandeln usw. Das Paket **pt::ast** dürfte dabei hilfreich sein, da es Befehle bereitstellt, mit denen sich solche AST-Strukturen auf verschiedene Weise durchlaufen lassen.

Zu beachten ist, dass die hier verwendeten Parser eine Datenstruktur zurückgeben, die die Struktur der Eingabe gemäß der dem Parser zugrunde liegenden Grammatik abbildet. Während des Parsing-Prozesses gibt es *keine* Rückruffunktionen, d. h. keine *Parsing-Aktionen*, wie sie bei den meisten anderen Parsern zu finden sind.

Kehren wir zum letzten Codeausschnitt zurück, in dem der Parser für eine bestimmte Eingabe ausgeführt wird: Beachten Sie, wie die Parser-Instanz der festgelegten *Parser-API* folgt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_rdengine

Stichwörter: pt_rdengine, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::rde – Laufzeitunterstützung für das Parsing, auf PARAM beruhend

Übersicht

package require Tcl 8.5 9
package require pt::rde ?**1.2**?
package require snit
package require struct::stack 1.6

package require pt::ast 1.2
::pt::rde *objectName*
objectName **destroy**
objectName **reset** *chan*
objectName **complete**
objectName **chan**
objectName **line**
objectName **column**
objectName **current**
objectName **location**
objectName **locations**
objectName **ok**
objectName **value**
objectName **error**
objectName **errors**
objectName **tokens** ?*from ?to*?
objectName **symbols**
objectName **known**
objectName **reducible**
objectName **asts**
objectName **ast**
objectName **position** *loc*
objectName **i_input_next** *msg*
objectName **i_test_alnum**
objectName **i_test_alpha**
objectName **i_test_ascii**
objectName **i_test_char** *char*
objectName **i_test_ddigit**
objectName **i_test_digit**
objectName **i_test_graph**
objectName **i_test_lower**
objectName **i_test_print**
objectName **i_test_punct**
objectName **i_test_range** *chars chare*
objectName **i_test_space**
objectName **i_test_upper**
objectName **i_test_wordchar**
objectName **i_test_xdigit**
objectName **i_error_clear**
objectName **i_error_push**
objectName **i_error_pop_merge**
objectName **i_error_nonterminal** *symbol*
objectName **i_status_ok**
objectName **i_status_fail**
objectName **i_status_negate**
objectName **i_loc_push**

objectName **i_loc_pop_discard**
objectName **i_loc_pop_rewind**
objectName **i:ok_loc_pop_rewind**
objectName **i_loc_pop_rewind/discard**
objectName **i_symbol_restore** *symbol*
objectName **i_symbol_save** *symbol*
objectName **i_value_clear**
objectName **i_value_clear/leaf**
objectName **i_value_clear/reduce**
objectName **i:ok_ast_value_push**
objectName **i_ast_push**
objectName **i_ast_pop_rewind**
objectName **i:fail_ast_pop_rewind**
objectName **i_ast_pop_rewind/discard**
objectName **i_ast_pop_discard**
objectName **i_ast_pop_discard/rewind**
objectName **i:ok_continue**
objectName **i:fail_continue**
objectName **i:fail_return**
objectName **i:ok_return**
objectName **si:void_state_push**
objectName **si:void2_state_push**
objectName **si:value_state_push**
objectName **si:void_state_merge**
objectName **si:void_state_merge_ok**
objectName **si:value_state_merge**
objectName **si:value_notahead_start**
objectName **si:void_notahead_exit**
objectName **si:value_notahead_exit**
objectName **si:kleene_abort**
objectName **si:kleene_close**
objectName **si:voidvoid_branch**
objectName **si:voidvalue_branch**
objectName **si:valuevoid_branch**
objectName **si:valuevalue_branch**
objectName **si:voidvoid_part**
objectName **si:voidvalue_part**
objectName **si:valuevalue_part**
objectName **si:value_symbol_start** *symbol*
objectName **si:value_void_symbol_start** *symbol*
objectName **si:void_symbol_start** *symbol*
objectName **si:void_void_symbol_start** *symbol*
objectName **si:reduce_symbol_end** *symbol*
objectName **si:void_leaf_symbol_end** *symbol*
objectName **si:value_leaf_symbol_end** *symbol*
objectName **si:value_clear_symbol_end** *symbol*

objectName **si:void_clear_symbol_end** *symbol*
objectName **si:next_char** *tok*
objectName **si:next_range** *toks toke*
objectName **si:next_alnum**
objectName **si:next_alpha**
objectName **si:next_ascii**
objectName **si:next_ddigit**
objectName **si:next_digit**
objectName **si:next_graph**
objectName **si:next_lower**
objectName **si:next_print**
objectName **si:next_punct**
objectName **si:next_space**
objectName **si:next_upper**
objectName **si:next_wordchar**
objectName **si:next_xdigit**

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket enthält eine Klasse, deren Instanzen die Laufzeitunterstützung für rekursive Abstiegsparser mit Backtracking bereitstellen, wie sie beispielsweise für die Ausführung von Parsing Expression Grammars erforderlich ist. Es implementiert die *PackRat Machine Specification*; daher ist die Lektüre dieses Dokuments *unabdingbar*, um sowohl diese Manpage als auch das Paket selbst zu verstehen. Die nachstehende Beschreibung enthält zahlreiche verkürzte Verweise auf die Anweisungen der PARAM und die verschiedenen Teile ihres architektonischen Zustands.

Das Paket befindet sich im Abschnitt "Ausführung" der Kernschicht der Parser-Tools.

Bild: arch_core_transform

Hinweis: Dieses Paket enthält nicht nur die Standard-Tcl-Implementierung, sondern auch einen Beschleuniger, d. h. eine auf Critcl basierende C-Implementierung.

API für Klassen

Das Paket exportiert die hier beschriebene API.

::pt::rde *objectName*

Der Befehl erstellt ein neues Laufzeitobjekt für einen rekursiven Abwärtsparser mit Backtracking und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück. Die API dieses Objektbefehls wird im Abschnitt Objekt-API beschrieben. Sie kann verwendet werden, um verschiedene Operationen auf dem Objekt auszuführen.

Objekt-API

Alle von diesem Paket erstellten Objekte stellen die folgenden 63 Methoden zur Bearbeitung und Abfrage ihres Zustands bereit, bei dem es sich im Wesentlichen um den architektonischen Zustand eines PARAM handelt.

Zuerst einige allgemeine Methoden und die Zugriffe auf den Zustand.

***objectName* destroy**
Diese Methode zerstört das Objekt, gibt den gesamten belegten Speicher frei und löscht den zugehörigen Objektbefehl.

objectName* reset *chan
Diese Methode setzt den Zustand der Laufzeitumgebung auf die Standardwerte zurück und bereitet sie so auf das Parsen der Zeichen im Kanal *chan* vor, der nun als IN dient.

Die Parser-Tools beruhen auf Tcl 8.5+. Mit anderen Worten: Das Kanal-Argument ist nicht auf Dateien, Sockets usw. beschränkt. Uns steht die volle Leistungsfähigkeit von *reflektierten Kanälen* zur Verfügung.

Es sei außerdem darauf hingewiesen, dass der Parser die Zeichen nach Bedarf aus dem Eingabestrom abruft. Wenn ein durch dieses Paket erstellter Parser im Push-Modus, d. h. ereignisgesteuert betrieben werden soll, ist es erforderlich, auf Tcl 8.6+ umzusteigen und **coroutine::auto** zu verwenden, um ihn in eine Coroutine zu verpacken, wobei **read** für den Push-Betrieb entsprechend angepasst werden muss.

***objectName* complete**
Diese Methode schließt die Analyse ab und gibt entweder den aus den Elementen von ARS erstellten AST zurück oder löst einen Fehler aus, der den aktuellen ER enthält.

***objectName* chan**
Diese Methode gibt das Handle des Kanals zurück, der IN ist.

***objectName* line**
Diese Methode gibt die Zeilennummer der Position zurück, an der sich IN derzeit befindet. Diese Zeilennummer muss aufgrund von Backtracking nicht mit der Zeilennummer von CL übereinstimmen.

***objectName* column**
Diese Methode gibt die Spalte zurück, an der sich IN derzeit befindet. Diese Spalte muss aufgrund von Backtracking nicht mit der Spalte von CL übereinstimmen.

***objectName* current**
Diese Methode gibt CC zurück.

***objectName* location**
Diese Methode gibt CL zurück.

***objectName* locations**
Diese Methode gibt die LS zurück. Der oberste Eintrag des Stapels ist das erste Element der zurückgegebenen Liste.

***objectName* ok**
Diese Methode gibt ST zurück.

***objectName* value**
Diese Methode gibt SV zurück.

***objectName* error**
Diese Methode gibt ER zurück. Dabei handelt es sich entweder um die leere Zeichenkette bei einem leeren ER oder um eine Liste mit zwei Elementen: die Stelle, auf die sich der Fehler bezieht, sowie eine Menge von Meldungen, die angeben, welche Symbole an dieser Stelle

erwartet wurden. Die Meldungen sind als einer der möglichen atomaren Parsing-Ausdrücke kodiert (Spezialoperatoren, Terminale, Bereichsoperatoren und Nichtterminale).

***objectName* errors**
Diese Methode gibt ES zurück. Der oberste Eintrag des Stapels ist das erste Element der zurückgegebenen Liste. Jeder Eintrag wird wie unter **error** beschrieben kodiert.

***objectName* tokens ?*from* ?*to*??**
Diese Methode gibt den Teil von TC zurück, der den Bereich der Positionen von IN abdeckt, beginnend bei *from* und endend bei *to*. Wenn *to* nicht festgelegt wird, wird es als identisch mit *from* angesehen. Wenn keines der beiden Argumente festgelegt wird, wird TC vollständig zurückgegeben.

Jedes Element in der zurückgegebenen Liste ist selbst eine Liste aus drei Elementen, die in dieser Reihenfolge das Zeichen an der angegebenen Position sowie die zugehörigen Zeilen- und Spaltennummern enthält.

***objectName* symbols**
Diese Methode gibt ein Dictionary zurück, das NC enthält. Die Schlüssel sind Listen mit zwei Elementen, die in dieser Reihenfolge das nichtterminale Symbol und die Position enthalten. Die Werte sind 4-Tupel, die in dieser Reihenfolge CL, ST, ER und SV enthalten. ER wird wie für die Methode **error** festgelegt kodiert.

***objectName* known**
Diese Methode gibt eine Liste zurück, die die Schlüssel von SC enthält. Diese werden auf dieselbe Weise kodiert wie bei der Methode **symbols**.

***objectName* reducible**
Diese Methode gibt ARS zurück. Der oberste Eintrag des Stapels ist das erste Element der zurückgegebenen Liste.

***objectName* asts**
Diese Methode gibt AS zurück. Der oberste Eintrag des Stapels ist das erste Element der zurückgegebenen Liste.

***objectName* ast**
Dies ist eine Hilfsmethode, die das oberste Element von ARS zurückgibt.

objectName* position *loc
Diese Methode gibt die Zeilen- und Spaltennummern für die angegebene Position von IN zurück, vorausgesetzt, diese Position wurde bereits während des Parsing-Prozesses erreicht.

Die folgenden Methoden implementieren alle PARAM-Anweisungen. Sie tragen alle das Präfix "i_".

Die Ablaufsteuerung erfolgt hauptsächlich über die in Tcl integrierten Befehle wie **if**, **while** usw. sowie über einige bedingte Varianten von PARAM-Anweisungen und Tcl-Befehlen. Das bedeutet, dass diese Befehlsvarianten nichts bewirken, wenn ihre Schutzbedingung nicht erfüllt ist. Sie sind an den Präfixen "i:ok_" und "i:fail_" zu erkennen, die den Wert angeben, den ST haben muss, damit der Befehl ausgeführt wird.

Die Befehle sind in derselben Reihenfolge aufgeführt, in der sie in der *PackRat Machine Specification* vorkommen, wobei die bedingten Varianten gegebenenfalls nach ihrer regulären Implementierung oder an deren Stelle aufgeführt sind.

objectName* i_input_next *msg

Diese Methode implementiert den PARAM-Befehl **input_next**.

***objectName* i_test_alnum**
Diese Methode implementiert den PARAM-Befehl **test_alnum**.

***objectName* i_test_alpha**
Diese Methode implementiert den PARAM-Befehl **test_alpha**.

***objectName* i_test_ascii**
Diese Methode implementiert den PARAM-Befehl **test_ascii**.

objectName* i_test_char *char
Diese Methode implementiert den PARAM-Befehl **test_char**.

***objectName* i_test_ddigit**
Diese Methode implementiert den PARAM-Befehl **test_ddigit**.

***objectName* i_test_digit**
Diese Methode implementiert den PARAM-Befehl **test_digit**.

***objectName* i_test_graph**
Diese Methode implementiert den PARAM-Befehl **test_graph**.

***objectName* i_test_lower**
Diese Methode implementiert den PARAM-Befehl **test_lower**.

***objectName* i_test_print**
Diese Methode implementiert den PARAM-Befehl **test_print**.

***objectName* i_test_punct**
Diese Methode implementiert den PARAM-Befehl **test_punct**.

objectName* i_test_range *chars chare
Diese Methode implementiert den PARAM-Befehl **test_range**.

***objectName* i_test_space**
Diese Methode implementiert den PARAM-Befehl **test_space**.

***objectName* i_test_upper**
Diese Methode implementiert den PARAM-Befehl **test_upper**.

***objectName* i_test_wordchar**
Diese Methode implementiert den PARAM-Befehl **test_wordchar**.

***objectName* i_test_xdigit**
Diese Methode implementiert den PARAM-Befehl **test_xdigit**.

***objectName* i_error_clear**
Diese Methode implementiert den PARAM-Befehl **error_clear**.

***objectName* i_error_push**
Diese Methode implementiert den PARAM-Befehl **error_push**.

***objectName* i_error_pop_merge**
Diese Methode implementiert den PARAM-Befehl **error_pop_merge**.

objectName* i_error_nonterminal *symbol
Diese Methode implementiert den PARAM-Befehl **error_nonterminal**.

***objectName* i_status_ok**
Diese Methode implementiert den PARAM-Befehl **status_ok**.

***objectName* i_status_fail**
Diese Methode implementiert den PARAM-Befehl **status_fail**.

***objectName* i_status_negate**
Diese Methode implementiert den PARAM-Befehl **status_negate**.

***objectName* i_loc_push**
Diese Methode implementiert den PARAM-Befehl **loc_push**.

***objectName* i_loc_pop_discard**
Diese Methode implementiert den PARAM-Befehl **loc_pop_discard**.

***objectName* i_loc_pop_rewind**
Diese Methode implementiert den PARAM-Befehl **loc_pop_rewind**.

***objectName* i:ok_loc_pop_rewind**
Diese geschützte Methode, eine Variante von **i_loc_pop_rewind**, wird nur ausgeführt, wenn "ST == ok" gilt.

***objectName* i_loc_pop_rewind/discard**
Diese Methode stellt eine praktische Kombination aus Ablaufsteuerung und den beiden PARAM-Anweisungen **loc_pop_rewind** und **loc_pop_discard** dar. Die erstgenannte wird bei "ST == fail" ausgeführt, die letztgenannte bei "ST == ok".

objectName* i_symbol_restore *symbol
Diese Methode implementiert den PARAM-Befehl **symbol_restore**.

Das boolesche Ergebnis der Prüfung wird als Ergebnis der Methode zurückgegeben und kann mit den üblichen Tcl-Befehlen zur Ablaufsteuerung verwendet werden.

objectName* i_symbol_save *symbol
Diese Methode implementiert den PARAM-Befehl **symbol_save**.

***objectName* i_value_clear**
Diese Methode implementiert den PARAM-Befehl **value_clear**.

***objectName* i_value_clear/leaf**
Diese Methode stellt eine praktische Kombination aus Ablaufsteuerung und den beiden PARAM-Anweisungen **value_clear** und **value_leaf** dar. Die erstgenannte wird bei "ST == fail" ausgeführt, die letztgenannte bei "ST == ok".

***objectName* i_value_clear/reduce**
Diese Methode stellt eine praktische Kombination aus Ablaufsteuerung und den beiden PARAM-Anweisungen **value_clear** und **value_reduce** dar. Die erstgenannte wird bei "ST == fail" ausgeführt, die letztgenannte bei "ST == ok".

***objectName* i:ok_ast_value_push**
Diese Methode implementiert eine geschützte Variante des PARAM-Befehls **ast_value_push**, die nur ausgeführt wird, wenn "ST == ok" gilt.

***objectName* i_ast_push**
Diese Methode implementiert den PARAM-Befehl **ast_push**.

***objectName* i_ast_pop_rewind**
Diese Methode implementiert den PARAM-Befehl **ast_pop_rewind**.

***objectName* i:fail_ast_pop_rewind**
Diese geschützte Methode, eine Variante von **i_ast_pop_rewind**, wird nur ausgeführt, wenn "ST == fail" gilt.

***objectName* i_ast_pop_rewind/discard**
Diese Methode stellt eine praktische Kombination aus Ablaufsteuerung und den beiden

PARAM-Anweisungen **ast_pop_rewind** und **ast_pop_discard** dar. Die erstgenannte wird bei "ST == fail" ausgeführt, die letztgenannte bei "ST == ok".

***objectName* i_ast_pop_discard**
Diese Methode implementiert den PARAM-Befehl **ast_pop_discard**.

***objectName* i_ast_pop_discard/rewind**
Diese Methode stellt eine praktische Kombination aus Ablaufsteuerung und den beiden PARAM-Anweisungen **ast_pop_discard** und **ast_pop_rewind** dar. Erstere wird bei "ST == fail" ausgeführt, letztere bei "ST == ok".

***objectName* i:ok_continue**
Diese geschützte Methode wird nur ausgeführt, wenn "ST == ok" gilt. Anschließend bricht sie die aktuelle Iteration der innersten Schleife in der aufrufenden Tcl-Prozedur ab.

***objectName* i:fail_continue**
Diese geschützte Methode wird nur ausgeführt, wenn "ST == fail" gilt. Anschließend bricht sie die aktuelle Iteration der innersten Schleife in der aufrufenden Tcl-Prozedur ab.

***objectName* i:fail_return**
Diese geschützte Methode wird nur ausgeführt, wenn "ST == fail" gilt. Anschließend bricht sie die aufrufende Tcl-Prozedur ab.

***objectName* i:ok_return**
Diese geschützte Methode wird nur ausgeführt, wenn "ST == ok" gilt. Anschließend bricht sie die aufrufende Tcl-Prozedur ab.

Die nächste Gruppe von Methoden sind *Superbefehle*, was bedeutet, dass jede von ihnen eine längere Folge von Befehlen implementiert, die üblicherweise in Parsern verwendet werden. Die kombinierten Befehle der vorherigen Gruppe, d. h. jene, deren Namen dem Muster "i_*/*" entsprechen, sind ebenfalls Superbefehle, wenn auch mit begrenztem Geltungsbereich, da sie zwei Befehle mit ihrem Kontrollfluss verarbeiten. Die folgende Gruppe hat einen wesentlich breiteren Anwendungsbereich und fasst bis zu sechs oder mehr PARAM-Befehle in einer einzigen Methode zusammen.

Hier lässt sich die Begründung für ihre Verwendung gut nachvollziehen:

1. Durch die Verwendung von weniger Anweisungen werden die generierten Parser kleiner, da die gemeinsamen Teile nun tatsächlich Teil der gemeinsamen Laufzeitumgebung sind und nicht mehr immer wieder explizit in den Code des Parsers geschrieben werden müssen.
2. Durch die Verwendung von weniger Befehlen wird zudem der mit Aufrufen an die Laufzeitumgebung verbundene Overhead reduziert, d. h. der Aufwand für die Vermittlung der Methodenaufrufe und die Einrichtung des Kontexts für Variablen.
3. Ein weiterer Effekt der Superbefehle besteht darin, dass auch deren interne Abläufe optimiert werden können, insbesondere hinsichtlich des Kontrollflusses und der Stapelnutzung, da die internen Abläufe der Laufzeitumgebung für alle in die Sequenz zusammengefassten Befehle zugänglich sind.

***objectName* si:void_state_push**
Diese Methode kombiniert

```
i_loc_push
i_error_clear
i_error_push
```

Parser verwenden es am Anfang von *void*-Sequenzen und bei Auswahlmöglichkeiten mit einem *void*-Anfangszweig.

***objectName* si:void2_state_push**

Diese Methode kombiniert

```
i_loc_push
i_error_clear
i_error_push
```

Parser verwenden es am Anfang von optionalen und wiederholten Ausdrücken.

***objectName* si:value_state_push**

Diese Methode kombiniert

```
i_ast_push
i_loc_push
i_error_clear
i_error_push
```

Parser verwenden ihn am Anfang von Sequenzen, die einen AST erzeugen, und bei Auswahlen mit einem Anfangszweig, der einen AST erzeugt.

***objectName* si:void_state_merge**

Diese Methode kombiniert

```
i_error_pop_merge
i_loc_pop_rewind/discard
```

Parser verwenden ihn am Ende von *void*-Sequenzen und Auswahlen, deren letzter Zweig *void* ist.

***objectName* si:void_state_merge_ok**

Diese Methode kombiniert

```
i_error_pop_merge
i_loc_pop_rewind/discard
i_status_ok
```

Parser verwenden es am Ende optionaler Ausdrücke

***objectName* si:value_state_merge**

Diese Methode kombiniert

```
i_error_pop_merge
i_ast_pop_rewind/discard
i_loc_pop_rewind/discard
```

Parser verwenden ihn am Ende von Sequenzen, die ASTs erzeugen, und von Auswahlen, deren letzter Zweig einen AST erzeugt.

***objectName* si:value_notahead_start**

Diese Methode kombiniert

```
i_loc_push
i_ast_push
```

Parser verwenden ihn am Anfang von Prädikaten mit negativer Vorausschau, die ASTs erzeugen.

***objectName* si:void_notahead_exit**

Diese Methode kombiniert

```
i_loc_pop_rewind
i_status_negate
```

Parser verwenden ihn am Ende von *void*-Prädikaten mit negativer Vorausschau.

***objectName* si:value_notahead_exit**

Diese Methode kombiniert

```
i_ast_pop_discard/rewind
i_loc_pop_rewind
i_status_negate
```

Parser verwenden ihn am Ende von Prädikaten mit negativer Vorausschau, die ASTs erzeugen.

***objectName* si:kleene_abort**

Diese Methode kombiniert

```
i_loc_pop_rewind/discard
i:fail_return
```

Parser verwenden dies, um eine positive Wiederholung zu beenden, wenn der erste, erforderliche Ausdruck fehlschlägt.

***objectName* si:kleene_close**

Diese Methode kombiniert

```
i_error_pop_merge
i_loc_pop_rewind/discard
i:fail_status_ok
i:fail_return
```

Parser verwenden es am Ende von Wiederholungen.

***objectName* si:voidvoid_branch**

Diese Methode kombiniert

```
i_error_pop_merge
i:ok_loc_pop_discard
i:ok_return
i_loc_rewind
i_error_push
```

Parser verwenden ihn beim Übergang zwischen den Zweigen einer Auswahl, wenn beide *void*

sind.

***objectName* si:voidvalue_branch**

Diese Methode kombiniert

```
i_error_pop_merge
i:ok_loc_pop_discard
i:ok_return
i_ast_push
i_loc_rewind
i_error_push
```

Parser verwenden ihn beim Übergang zwischen den Zweigen einer Auswahl, wenn der fehlgeschlagene Zweig *void* ist und der als Nächstes zu prüfende Zweig einen AST erzeugt.

***objectName* si:valuevoid_branch**

Diese Methode kombiniert

```
i_error_pop_merge
i_ast_pop_rewind/discard
i:ok_loc_pop_discard
i:ok_return
i_loc_rewind
i_error_push
```

Parser verwenden ihn beim Übergang zwischen den Zweigen einer Auswahl, wenn der fehlgeschlagene Zweig einen AST erzeugt und der als Nächstes zu prüfende Zweig *void* ist.

***objectName* si:valuevalue_branch**

Diese Methode kombiniert

```
i_error_pop_merge
i_ast_pop_discard
i:ok_loc_pop_discard
i:ok_return
i_ast_rewind
i_loc_rewind
i_error_push
```

Parser verwenden ihn beim Übergang zwischen den Zweigen einer Auswahl, wenn beide ASTs erzeugen.

***objectName* si:voidvoid_part**

Diese Methode kombiniert

```
i_error_pop_merge
i:fail_loc_pop_rewind
i:fail_return
i_error_push
```

Parser verwenden ihn beim Übergang zwischen Teilen einer Sequenz, wenn beide *void* sind.

***objectName* si:voidvalue_part**

Diese Methode kombiniert

```
i_error_pop_merge
i:fail_loc_pop_rewind
i:fail_return
i_ast_push
i_error_push
```

Parser verwenden ihn beim Übergang zwischen Teilen einer Sequenz, wenn der erfolgreich erkannte Teil *void* ist und danach ein AST erzeugt wird.

***objectName* si:valuevalue_part**

Diese Methode kombiniert

```
i_error_pop_merge
i:fail_ast_pop_rewind
i:fail_loc_pop_rewind
i:fail_return
i_error_push
```

Parser verwenden dies beim Übergang zwischen Teilen einer Sequenz, wenn beide Teile ASTs erzeugen.

objectName* si:value_symbol_start *symbol

Diese Methode kombiniert

```
if/found? i_symbol_restore $symbol
i:found:ok_ast_value_push
i:found_return
i_loc_push
i_ast_push
```

Parser verwenden es am Anfang eines nichtterminalen Symbols, das einen AST erzeugt, dessen rechte Seite möglicherweise ebenfalls einen AST erzeugt hat.

objectName* si:value_void_symbol_start *symbol

Diese Methode kombiniert

```
if/found? i_symbol_restore $symbol
i:found:ok_ast_value_push
i:found_return
i_loc_push
i_ast_push
```

Parser verwenden ihn am Anfang eines *void*-Nichtterminals, dessen rechte Seite einen AST erzeugen kann.

objectName* si:void_symbol_start *symbol

Diese Methode kombiniert

```
if/found? i_symbol_restore $symbol
i:found_return
i_loc_push
i_ast_push
```

Parser verwenden ihn am Anfang eines Nichtterminals, das einen AST erzeugt und dessen rechte Seite *void* ist.

objectName* si:void_void_symbol_start *symbol

Diese Methode kombiniert

```
if/found? i_symbol_restore $symbol
i:found_return
i_loc_push
```

Parser verwenden ihn am Anfang eines *void*-Nichtterminals, dessen rechte Seite ebenfalls *void* ist.

objectName* si:reduce_symbol_end *symbol

Diese Methode kombiniert

```
i_value_clear/reduce $symbol
i_symbol_save        $symbol
i_error_nonterminal  $symbol
i_ast_pop_rewind
i_loc_pop_discard
i:ok_ast_value_push
```

Parser verwenden ihn am Ende eines Nichtterminals, das einen AST erzeugt und den von der rechten Seite erzeugten AST als Kindknoten verwendet.

objectName* si:void_leaf_symbol_end *symbol

Diese Methode kombiniert

```
i_value_clear/leaf  $symbol
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
i:ok_ast_value_push
```

Parser verwenden ihn am Ende eines Nichtterminals, das einen AST erzeugt und dessen rechte Seite *void* ist.

objectName* si:value_leaf_symbol_end *symbol

Diese Methode kombiniert

```
i_value_clear/leaf  $symbol
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
i_ast_pop_rewind
i:ok_ast_value_push
```

Parser verwenden es am Ende eines nichtterminalen Symbols, das einen AST erzeugt, wobei der von der rechten Seite erzeugte AST verworfen wird.

objectName* si:value_clear_symbol_end *symbol

Diese Methode kombiniert

```
i_value_clear
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
i_ast_pop_rewind
```

Parser verwenden ihn am Ende eines *void*-Nichtterminals und verwerfen den von der rechten Seite erzeugten AST.

objectName* si:void_clear_symbol_end *symbol

Diese Methode kombiniert

```
i_value_clear
i_symbol_save       $symbol
i_error_nonterminal $symbol
i_loc_pop_discard
```

Parser verwenden ihn am Ende eines *void*-Nichtterminals, dessen rechte Seite ebenfalls *void* ist.

objectName* si:next_char *tok

siehe unten ...

objectName* si:next_range *toks toke

siehe unten ...

***objectName* si:next_alnum**

siehe unten ...

***objectName* si:next_alpha**

siehe unten ...

***objectName* si:next_ascii**

siehe unten ...

***objectName* si:next_ddigit**

siehe unten ...

***objectName* si:next_digit**

siehe unten ...

***objectName* si:next_graph**

siehe unten ...

***objectName* si:next_lower**
siehe unten ...

***objectName* si:next_print**
siehe unten ...

***objectName* si:next_punct**
siehe unten ...

***objectName* si:next_space**
siehe unten ...

***objectName* si:next_upper**
siehe unten ...

***objectName* si:next_wordchar**
siehe unten ...

***objectName* si:next_xdigit**
Diese Methoden vereinen alle

```
i_input_next $msg
i:fail_return
```

mit der entsprechenden Anweisung **i_test_xxx**. Parser verwenden diese zur Verarbeitung atomarer Ausdrücke.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_tclparam_config_nx

Stichwörter: pt_tclparam_config_nx, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::tclparam::configuration::nx – Tcl/PARAM, Vorgefertigte Konfiguration, NX

Übersicht

package require Tcl 8.5 9
package require pt::tclparam::configuration::nx ?**1.0.2**?
::pt::tclparam::configuration::nx def *name pkg version cmdprefix*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket dient als Ergänzung zu pt::peg::to::tclparam und soll die Verwendung dieses in hohem Maße konfigurierbaren Pakets durch die Bereitstellung einer vorgefertigten Konfiguration vereinfachen. Wenn diese Konfiguration angewendet wird, generiert das Paket pt::peg::to::tclparam Pakete mit Parsern, die auf **NX** basieren.

Es handelt sich um ein unterstützendes Paket in der Kernschicht der Parser-Tools.

Bild: arch_core_support

API

::pt::tclparam::configuration::nx def *name pkg version cmdprefix*

Der Befehl wendet die in diesem Paket bereitgestellte Konfiguration auf *cmdprefix* an, wodurch Parser auf Basis von **NX** erstellt werden, deren Klasse *name* lautet, im Paket *pkg* mit *version*.

Die Verwendung eines Befehlspräfixes als API ermöglicht die Anwendung der Konfiguration nicht nur auf pt::peg::to::tclparam (pt::peg::to::tclparam configure), sondern auch auf Export-Manager-Instanzen und PEG-Container (**$export configuration set** bzw. **[$container exporter] configuration set**).

Oder auf jedes andere Befehlspräfix, das zwei Argumente – Option und Wert – akzeptiert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_tclparam_config_snit

Stichwörter: pt_tclparam_config_snit, EBNF, LL(k), PEG, TDPL, context-free languages, expression , grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::tclparam::configuration::snit – Tcl/PARAM, Vorgefertigte Konfiguration, Snit

Übersicht

package require Tcl 8.5 9
package require pt::tclparam::configuration::snit ?**1.0.3**?
::pt::tclparam::configuration::snit def *name pkg version cmdprefix*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket dient als Ergänzung zu pt::peg::to::tclparam und soll die Verwendung dieses in hohem Maße konfigurierbaren Pakets durch die Bereitstellung einer vorgefertigten Konfiguration vereinfachen. Wenn diese Konfiguration angewendet wird, generiert das Paket pt::peg::to::tclparam Pakete mit Parsern, die auf **snit** basieren.

Es handelt sich um ein unterstützendes Paket in der Kernschicht der Parser-Tools.

Bild: arch_core_support

API

::pt::tclparam::configuration::snit def *name pkg version cmdprefix*

Der Befehl wendet die in diesem Paket bereitgestellte Konfiguration auf *cmdprefix* an, wodurch Parser auf Basis von **snit** erstellt werden, deren Klasse *name* lautet, im Paket *pkg* mit *version*.

Die Verwendung eines Befehlspräfixes als API ermöglicht die Anwendung der Konfiguration nicht nur auf pt::peg::to::tclparam (pt::peg::to::tclparam configure), sondern auch auf Export-Manager-Instanzen und PEG-Container (**$export configuration set** bzw. **[$container exporter] configuration set**).

Oder auf jedes andere Befehlspräfix, das zwei Argumente – Option und Wert – akzeptiert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_tclparam_config_tcloo
Stichwörter: pt_tclparam_config_tcloo, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name
pt::tclparam::configuration::tcloo – Tcl/PARAM, Vorgefertigte Konfiguration, Tcloo

Übersicht
package require Tcl 8.5 9
package require pt::tclparam::configuration::tcloo ?**1.0.5**?
::pt::tclparam::configuration::tcloo def *name pkg version cmdprefix*

Beschreibung
Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket dient als Ergänzung zu pt::peg::to::tclparam und soll die Verwendung dieses in hohem Maße konfigurierbaren Pakets durch die Bereitstellung einer vorgefertigten Konfiguration vereinfachen. Wenn diese Konfiguration angewendet wird, generiert das Paket pt::peg::to::tclparam Pakete mit Parsern, die auf **OO** basieren.

Es handelt sich um ein unterstützendes Paket in der Kernschicht der Parser-Tools.

Bild: arch_core_support

API
::pt::tclparam::configuration::tcloo def *name pkg version cmdprefix*

Der Befehl wendet die in diesem Paket bereitgestellte Konfiguration auf *cmdprefix* an, wodurch Parser auf Basis von **OO** erstellt werden, deren Klasse *name* lautet, im Paket *pkg* mit *version*.

Die Verwendung eines Befehlspräfixes als API ermöglicht die Anwendung der Konfiguration nicht nur auf pt::peg::to::tclparam (pt::peg::to::tclparam configure), sondern auch auf Export-Manager-Instanzen und PEG-Container (**$export configuration set** bzw. **[$container exporter] configuration set**).

Oder auf jedes andere Befehlspräfix, das zwei Argumente – Option und Wert – akzeptiert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


pt_to_api
Stichwörter: pt_to_api, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name
pt_export_api – Export-API für Parser-Tools

Übersicht
package require Tcl 8.5 9
CONVERTER reset
CONVERTER configure
CONVERTER configure *option*
CONVERTER configure *option value...*
CONVERTER convert *serial*
::export *serial configuration*

Beschreibung
Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

In diesem Dokument werden zwei APIs beschrieben. Zunächst die API, die allen Paketen gemeinsam ist und der Konvertierung von Parsing Expression Grammars in ein anderes Format dient, und anschließend die API, die den Paketen gemeinsam ist, welche die Export-Plugins implementieren, die auf den Konvertierungspaketen aufbauen.

Die Zielgruppe sind Personen, die einen eigenen Konverter für ein bestimmtes Ausgabeformat und/oder ein Export-Plugin für ihren eigenen oder einen anderen Konverter erstellen möchten.

Sie befindet sich im Abschnitt "Export" der Kernschicht der Parser-Tools.

Bild: arch_core_export

Converter-API

Jeder (Grammatik-)Exportkonverter muss die nachstehend aufgeführten Regeln befolgen:

1. Ein Konverter ist ein Paket. Sein Name ist beliebig, es wird jedoch empfohlen, ihn unter dem Namespace "**::pt::peg::to**" anzulegen.
2. Das Paket stellt entweder einen einzelnen Tcl-Befehl bereit, der der unten beschriebenen Gliederung der API entspricht, oder einen Klassenbefehl, dessen Instanzen derselben API folgen. Die Befehle, die dieser API entsprechen, werden als *Konverterbefehle* bezeichnet.
3. Ein Konverterbefehl muss die folgenden drei Methoden mit den angegebenen Signaturen und Semantiken bereitstellen. Konverterbefehle dürfen weitere eigene Methoden bereitstellen, jedoch nicht weniger, und sie dürfen für die standardisierten Methoden keine abweichende Semantik vorsehen.

   CONVERTER reset
   Diese Methode muss die Konfiguration des Konverters auf die Standardeinstellungen zurücksetzen. Das Ergebnis der Methode muss die leere Zeichenkette sein.

   CONVERTER configure
   Diese Methode muss in dieser Form ein Dictionary zurückgeben, das die aktuelle Konfiguration des Konverters enthält.

   CONVERTER configure *option*
   Diese Methode gibt in dieser Form den aktuellen Wert der angegebenen *option* des Konverters zurück.

   Bitte lesen Sie den Abschnitt Optionen, um sich über die Standard-Optionen zu informieren, die jeder Konverter akzeptieren muss. Alle weiteren Optionen, die von einem bestimmten Konverter akzeptiert werden, sind in dessen Manpage beschrieben.

   CONVERTER configure *option value*...
   Dieser Befehl setzt in dieser Form die angegebenen *option*s des Konverters auf die angegebenen *value*s.

   Bitte lesen Sie den Abschnitt Optionen, um sich über die Standard-Optionen zu informieren, die ein Konverter akzeptieren muss. Alle weiteren Optionen, die von einem bestimmten Konverter akzeptiert werden, sind in dessen Manpage beschrieben.

   CONVERTER convert *serial*
   Diese Methode muss die kanonische Serialisierung einer Parsing Expression Grammar akzeptieren, wie sie im Abschnitt PEG-Serialisierungsformat festgelegt ist und in *serial* enthalten ist. Das Ergebnis der Methode muss das Ergebnis der Konvertierung der Eingabegrammatik in das Format sein, für das der Konverter gemäß seiner Konfiguration vorgesehen ist.

Plugin-API

Jedes (Grammatik-)Export-Plugin muss die nachstehend aufgeführten Regeln einhalten:

1. Ein Plugin ist ein Paket.
2. Der Name eines Plugin-Pakets hat die Form pt::peg::export::**FOO**, wobei **FOO** der Name des Formats ist, für das das Plugin eine Ausgabe generiert.
3. Das Plugin kann davon ausgehen, dass das Paket **pt::peg::export::plugin** vorhanden ist, was darauf hindeutet, dass es von einem echten Plugin-Manager aufgerufen wurde.

Es wird empfohlen, dass ein Plugin das Vorhandensein dieses Pakets überprüft.

4. Ein Plugin muss im globalen Namespace einen einzigen Befehl mit der unten dargestellten Signatur bereitstellen. Plugins dürfen eigene Befehle hinzufügen, jedoch nicht weniger, und sie dürfen für den standardisierten Befehl keine abweichende Semantik vorgeben.

   ::export *serial configuration*

   Dieser Befehl muss die kanonische Serialisierung einer Parsing-Expression-Grammatik sowie die Konfiguration für den vom Plugin aufgerufenen Konverter entgegennehmen. Das Ergebnis des Befehls muss das Ergebnis des vom Plugin für die Eingabegrammatik und die Konfiguration aufgerufenen Konverters sein.

   string *serial*

   Dieses Argument enthält die *kanonische* Serialisierung der Parsing Expression Grammar, für die die Ausgabe erzeugt werden soll. Die Definition dessen, was eine *kanonische* Serialisierung ist, finden Sie im Abschnitt PEG-Serialisierungsformat.

   dictionary *configuration*

   Dieses Argument enthält die Konfiguration, mit der der Konverter vor seinem Aufruf eingerichtet werden soll, und zwar in Form eines Dictionary, das Optionen auf Werte abbildet.

   Bitte lesen Sie den Abschnitt Optionen, um sich über die Standard-Optionen zu informieren, die jeder Konverter – und somit auch jedes Plugin – akzeptieren muss. Alle weiteren Optionen, die von einem bestimmten Plugin akzeptiert werden, sind in dessen Manpage beschrieben.

1. Ein einzelner Nutzungszyklus eines Plugins besteht aus einem Aufruf des Befehls export. Dieser Aufruf muss das Plugin in einem Zustand hinterlassen, in dem ein weiterer Nutzungszyklus problemlos ausgeführt werden kann.

Optionen

Jeder Exportkonverter und jedes Plugin für einen Exportkonverter muss die nachstehend aufgeführten Optionen in seiner Methode **configure** akzeptieren. Konverter dürfen den Inhalt dieser Optionen bei der Durchführung einer Konvertierung ignorieren, dürfen sie jedoch nicht zurückweisen. Von Plugins wird erwartet, dass sie die ihnen übergebenen Optionen an den von ihnen aufgerufenen Konverter weiterleiten.

-file string

Der Wert dieser Option ist der Name der Datei oder einer anderen Einheit, aus der die Grammatik stammt und für die der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

-name string

Der Wert dieser Option ist der Name der Grammatik, die wir gerade verarbeiten. Der Standardwert lautet **a_pe_grammar**.

-user string

Der Wert dieser Option ist der Name des Benutzers, unter dem der Befehl ausgeführt wird. Der Standardwert lautet **unknown**.

Verwendung

Ein Konverter wird so verwendet:

```
    # Get the converter (single command here, not class)
    package require the-converter-package

    # Provide a configuration
    theconverter configure ...

    # Perform the conversion
    set result [theconverter convert $thegrammarserial]

    ... process the result ...
```

Das Plugin **FOO** wird so verwendet:

```
    # Get an export plugin manager
    package require pt::peg::export
    pt::peg::export E

    # Provide a configuration
    E configuration set ...

    # Run the plugin, and the converter inside.
    set result [E export serial $grammarserial FOO]

    ... process the result ...
```

PEG-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing Expression Grammars als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn eine PEG mehr als eine reguläre Serialisierung haben kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

1. Die Serialisierung eines beliebigen PEG ist ein verschachteltes Tcl-Dictionary.
1. Dieses Dictionary enthält einen einzigen Schlüssel, **pt::grammar::peg**, sowie dessen Wert. Dieser Wert enthält den Inhalt der Grammatik.
2. Der Inhalt der Grammatik besteht aus einem Tcl-Dictionary, das die Menge der nichtterminalen Symbole und den Startausdruck enthält. Die relevanten Schlüssel und ihre Werte lauten

   rules

   Der Wert ist ein Tcl-Dictionary, dessen Schlüssel die Namen der in der Grammatik bekannten nichtterminalen Symbole sind.

   1. Jedes nichtterminale Symbol darf nur einmal vorkommen.
   2. Die leere Zeichenkette ist kein zulässiges nichtterminales Symbol.
   3. Der Wert für jedes Symbol ist selbst ein Tcl-Dictionary. Die entsprechenden Schlüssel und ihre Werte in diesem Dictionary lauten

      is

      Der Wert entspricht der Serialisierung des Parsing-Ausdrucks, der die Satzstruktur

des Symbols beschreibt, wie im Abschnitt PE-Serialisierungsformat festgelegt.

mode
Der Wert kann einer von drei Werten sein, die festlegen, wie ein Parser mit dem durch das Symbol erzeugten semantischen Wert umgehen soll.

value
Der semantische Wert des nichtterminalen Symbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminale selbst besteht, dessen Kindknoten die ASTs der rechten Seite des Symbols sind.

leaf
Der semantische Wert des Nichtterminalsymbols ist ein abstrakter Syntaxbaum, der aus einem einzigen Knoten für das Nichtterminal besteht, der keine Kindknoten hat. Alle durch die rechte Seite des Symbols erzeugten ASTs werden verworfen.

void
Das Nichtterminal hat keinen semantischen Wert. Alle ASTs, die durch die rechte Seite des Symbols erzeugt werden, werden (ebenfalls) verworfen.

```
**start**
: Der Wert entspricht der Serialisierung des Start-Parsing-Ausdrucks
    der Grammatik, wie im Abschnitt [PE-Serialisierungsformat]
    beschrieben.
```

1. Die Terminalsymbole der Grammatik werden implizit als die Menge aller Terminalsymbole festgelegt, die im Startausdruck und auf der rechten Seite der Grammatikregeln verwendet werden.

Kanonische Serialisierung
Die kanonische Serialisierung einer Grammatik hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieser Grammatik eindeutig ist.

1. Die Schlüssel in allen verschachtelten Tcl-Dictionaries sind in aufsteigender Wörterbuchreihenfolge sortiert, wie sie der eingebaute Tcl-Befehl **lsort -increasing -dict** erzeugt.
2. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung eines Tcl-Dictionaries. Das heißt, sie enthält keine überflüssigen Leerzeichen.

Beispiel

Unter der Annahme der folgenden PEG für einfache mathematische Ausdrücke

```
PEG calculator (Expression)
    Digit      <- '0'/'1'/'2'/'3'/'4'/'5'/'6'/'7'/'8'/'9'       ;
    Sign       <- '-' / '+'                                    ;
    Number     <- Sign? Digit+                                 ;
    Expression <- Term (AddOp Term)*                           ;
    MulOp      <- '*' / '/'                                    ;
```

```
    Term       <- Factor (MulOp Factor)*                          ;
    AddOp      <- '+'/'-'                                         ;
    Factor     <- '(' Expression ')' / Number                     ;
END;
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
pt::grammar::peg {
    rules {
        AddOp      {is {/ {t -} {t +}}
                                  mode value}
        Digit      {is {/ {t 0} {t 1} {t 2} {t 3} {t 4} {t 5} {t 6} {t
    7} {t 8} {t 9}}                mode value}
        Expression {is {x {n Term} {* {x {n AddOp} {n Term}}}}
                                  mode value}
        Factor     {is {/ {x {t (} {n Expression} {t )}} {n Number}}
                                  mode value}
        MulOp      {is {/ {t *} {t /}}
                                  mode value}
        Number     {is {x {? {n Sign}} {+ {n Digit}}}
                                  mode value}
        Sign       {is {/ {t -} {t +}}
                                  mode value}
        Term       {is {x {n Factor} {* {x {n MulOp} {n Factor}}}}
                                  mode value}
    }
    start {n Expression}
}
```

PE-Serialisierungsformat

Hier legen wir das Format fest, das von den Parser-Tools verwendet wird, um Parsing-Ausdrücke als unveränderliche Werte für die Übertragung, den Vergleich usw. zu serialisieren.

Wir unterscheiden zwischen *regulären* und *kanonischen* Serialisierungen. Auch wenn ein Parsing-Ausdruck mehr als eine reguläre Serialisierung aufweisen kann, ist jeweils genau eine davon *kanonisch*.

Reguläre Serialisierung

siehe unten ...

Atomare Parsing-Ausdrücke

1. Die Zeichenfolge **epsilon** ist ein atomarer Parsing-Ausdruck. Sie passt auf die leere Zeichenfolge.
1. Die Zeichenfolge **dot** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Zeichen.
2. Die Zeichenfolge **alnum** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen und jede Unicode-Ziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
3. Die Zeichenfolge **alpha** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes alphabetische Unicode-Zeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke

auf der Grundlage des eingebauten Tcl-Befehls **string is**.

4. Die Zeichenfolge **ascii** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Zeichen unterhalb von U0080. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
5. Die Zeichenfolge **control** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Steuerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
6. Die Zeichenfolge **digit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Unicode-Ziffer. Das schließt auch Zeichen außerhalb des Bereichs [0..9] ein. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
7. Die Zeichenfolge **graph** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen außer dem Leerzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
8. Die Zeichenfolge **lower** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Kleinbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
9. Die Zeichenfolge **print** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes druckbare Unicode-Zeichen einschließlich des Leerzeichens. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
10. Die Zeichenfolge **punct** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Satzzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
11. Die Zeichenfolge **space** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Leerraumzeichen. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
12. Die Zeichenfolge **upper** ist ein atomarer Parsing-Ausdruck. Sie passt auf jeden Unicode-Großbuchstaben. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
13. Die Zeichenfolge **wordchar** ist ein atomarer Parsing-Ausdruck. Sie passt auf jedes Unicode-Wortzeichen. Das ist jedes alphanumerische Zeichen (siehe alnum) und jedes Verbindungssatzzeichen (zum Beispiel der Unterstrich). Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
14. Die Zeichenfolge **xdigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Hexadezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **string is**.
15. Die Zeichenfolge **ddigit** ist ein atomarer Parsing-Ausdruck. Sie passt auf jede Dezimalziffer. Das ist eine eigene Erweiterung der Parsing-Ausdrücke auf der Grundlage des eingebauten Tcl-Befehls **regexp**.
16. Der Ausdruck [list t **x**] ist ein atomarer Parsing-Ausdruck. Er passt auf die Terminalzeichenkette **x**.
17. Der Ausdruck [list n **A**] ist ein atomarer Parsing-Ausdruck. Er passt auf das Nichtterminal **A**.

Zusammengesetzte Parsing-Ausdrücke

1. Für die Parsing-Ausdrücke **e1**, **e2**, … ist das Ergebnis von [list / **e1 e2** …] ebenfalls ein Parsing-Ausdruck. Das ist die *geordnete Auswahl*, auch *priorisierte Auswahl* genannt.

1. Für die Parsing-Ausdrücke **e1**, **e2**, ... ist das Ergebnis von [list x **e1 e2** ...] ebenfalls ein Parsing-Ausdruck. Dies ist die *Sequenz*.
2. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list * **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *Kleene-Abschluss*, der null oder mehr Wiederholungen beschreibt.
3. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list + **e**] ebenfalls ein Parsing-Ausdruck. Dies ist der *positive Kleene-Abschluss*, der eine oder mehrere Wiederholungen beschreibt.
4. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list & **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *And-Lookahead-Prädikat*.
5. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list! **e**] ebenfalls ein Parsing-Ausdruck. Dies ist das *Not-Lookahead-Prädikat*.
6. Für einen Parsing-Ausdruck **e** ist das Ergebnis von [list ? **e**] ebenfalls ein Parsing-Ausdruck. Dies ist die *optionale Eingabe*.

Kanonische Serialisierung

Die kanonische Serialisierung eines Parsing-Ausdrucks hat das im vorherigen Punkt festgelegte Format und erfüllt darüber hinaus die nachstehenden Einschränkungen, wodurch sie unter allen möglichen Serialisierungen dieses Parsing-Ausdrucks eindeutig ist.

1. Die Zeichenfolgendarstellung des Werts entspricht der kanonischen Darstellung einer reinen Tcl-Liste. Das heißt, sie enthält keine überflüssigen Leerzeichen.
2. Terminale werden *nicht* als Bereiche kodiert (bei denen Anfang und Ende des Bereichs identisch sind).

Beispiel

Unter der Annahme, dass der auf der rechten Seite der Regel dargestellte Parsing-Ausdruck gilt

```
    Expression <- Term (AddOp Term)*
```

dann lautet seine kanonische Serialisierung (mit Ausnahme von Leerzeichen)

```
    {x {n Term} {* {x {n AddOp} {n Term}}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

pt_util

Stichwörter: pt_util, EBNF, LL(k), PEG, TDPL, context-free languages, expression, grammar, matching, parser, parsing expression, parsing expression grammar, push down automaton, recursive descent, state, top-down parsing languages, transducer

Name

pt::util – Allgemeine Hilfsbefehle

Übersicht

package require Tcl 8.5 9
package require pt::ast ?**1.2**?
::pt::util error2readable *error text*
::pt::util error2position *error text*
::pt::util error2text *error*

Beschreibung

Sind Sie ratlos? Haben Sie Schwierigkeiten, dieses Dokument zu verstehen? Dann lesen Sie bitte die Übersicht in der *Introduction to Parser Tools*. Dieses Dokument ist der Einstiegspunkt in das Gesamtsystem, zu dem das aktuelle Paket gehört.

Dieses Paket stellt allgemeine Hilfsbefehle bereit.

Dies ist ein unterstützendes Paket in der Kernschicht der Parser-Tools.

Bild: arch_core_support

API

::pt::util error2readable *error text*

Dieser Befehl nimmt die strukturierte Form eines Syntaxfehlers *error* – wie er von Parser-Laufzeitumgebungen ausgelöst wird – sowie den Eingabetext *text* entgegen, der diesen Fehler verursacht hat, und gibt eine Zeichenkette zurück, die den Fehler in einer für Menschen lesbaren Form beschreibt.

Der Eingabetext *text* wird benötigt, um die Zeichenposition des Fehlers in ein besser lesbares Zeilen-/Spaltenformat umzuwandeln und Auszüge aus dem Eingabetext rund um die Fehlerposition bereitzustellen.

::pt::util error2position *error text*

Dieser Befehl nimmt die strukturierte Form eines Syntaxfehlers *error*, wie Parser-Laufzeitumgebungen ihn auslösen, und den Eingabetext *text*, der den Fehler verursacht hat, und gibt eine Liste aus zwei Elementen zurück: Zeilennummer und Spaltenindex der Zeichenposition des Fehlers in der Eingabe, in dieser Reihenfolge.

::pt::util error2text *error*

Dieser Befehl nimmt die strukturierte Form eines Syntaxfehlers *error*, wie Parser-Laufzeitumgebungen ihn auslösen, und gibt eine Liste von Zeichenfolgen zurück. Jede beschreibt eine mögliche erwartete Eingabe in einer für Menschen lesbaren Form.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *pt* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Parser und Grammatiken

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


rc4

rc4

Stichwörter: rc4, arcfour, data integrity, encryption, security, stream cipher

Name

rc4 – Implementierung der RC4-Stromverschlüsselung

Übersicht

package require Tcl 8.5 9
package require rc4 ?**1.2.0**?
::rc4::rc4 ?*-hex*? *-key keyvalue* ?*-command lst*? ?*-out channel*? [*-in channel* | *-infile filename* | *string*]
::rc4::RC4Init *keydata*
::rc4::RC4 *Key data*
::rc4::RC4Final *Key*

Beschreibung

Dieses Paket ist eine Implementierung der von Ron Rivest von RSA Data Security Inc. entwickelten RC4-Stromverschlüsselung in Tcl. Die Verschlüsselung war ein Geschäftsgeheimnis von RSA, wurde jedoch rückentwickelt und 1994 im Internet veröffentlicht. Sie wird in einer Reihe von Netzwerkprotokollen zur Sicherung der Kommunikation eingesetzt. Um markenrechtlichen Beschränkungen zu umgehen, wird diese Verschlüsselung manchmal als ARCFOUR bezeichnet.

Befehle

::rc4::rc4 ?*-hex*? *-key keyvalue* ?*-command lst*? ?*-out channel*? [*-in channel* | *-infile filename* | *string*]

Wendet den RC4-Algorithmus entweder auf die durch das Argument bereitgestellten Daten

oder auf die aus dem Kanal *-in* gelesenen Daten an. Wird ein Kanal *-out* angegeben, wird das Ergebnis in diesen Kanal geschrieben. Bei Angabe der Option *-hex* gibt es eine hexadezimal kodierte Version des Ergebnisses zurück, wenn kein *-out*-Kanal verwendet wird.

Die zu verarbeitenden Daten können entweder als Zeichenfolgenargument für den Befehl rc4, als Dateiname oder als zuvor geöffneter Kanal angegeben werden. Wenn das Argument *-infile* angegeben wird, wird die Datei geöffnet, die Daten werden gelesen und verarbeitet, und die Datei wird geschlossen. Wenn das Argument *-in* angegeben wird, werden Daten aus dem Kanal bis zum Dateiende gelesen. Der Kanal wird nicht geschlossen. Wenn das Argument *-out* angegeben wird, wird das Verarbeitungsergebnis in diesen Kanal geschrieben.

Wenn *-command* angegeben wird, gibt der Befehl "rc4" nichts zurück. Stattdessen wird der angegebene Befehl aufgerufen, wobei die Ergebnisdaten von "rc4" als letzter Parameter angehängt werden. Dies ist besonders nützlich beim Lesen aus Tcl-Kanälen, da auf dem Kanal ein "fileevent" eingerichtet ist und die Daten in Blöcken verarbeitet werden.

Es sollte nur eines der Argumente *-infile*, *-in* oder *string* angegeben werden.

Programmierschnittstelle

::rc4::RC4Init *keydata*
Initialisiert einen neuen RC4-Schlüssel. Bei *keydata* handelt es sich um eine beliebige Menge an Binärdaten, die zur Initialisierung des internen Zustands der Verschlüsselungsfunktion verwendet wird.

::rc4::RC4 *Key data*
Verschlüsseln oder entschlüsseln Sie die Eingabedaten mit dem Schlüssel, den Sie durch den Aufruf von **RC4Init** erhalten haben.

::rc4::RC4Final *Key*
Diese Funktion sollte aufgerufen werden, um die mit *Key* verbundenen Ressourcen zu bereinigen. Sobald diese Funktion aufgerufen wurde, wird der Schlüssel gelöscht.

Beispiele

```
% set keydata [binary format H* 0123456789abcdef]
% rc4::rc4 -hex -key $keydata HelloWorld
3cf1ae8b7f1c670b612f
% rc4::rc4 -key $keydata [binary format H* 3cf1ae8b7f1c670b612f]
HelloWorld
```

```
set Key [rc4::RC4Init "key data"]
append ciphertext [rc4::RC4 $Key $plaintext]
append ciphertext [rc4::RC4 $Key $additional_plaintext]
rc4::RC4Final $Key
```

```
proc ::Finish {myState data} {
    DoStuffWith $myState $data
}
rc4::rc4 -in $socket -command [list ::Finish $ApplicationState]
```

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *rc4* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2003, Pat Thoyts <patthoyts@users.sourceforge.net>
```


rcs

rcs
Stichwörter: rcs, CVS, RCS patch, SCCS, diff -n format, patching, text conversion, text differences

Name
rcs – RCS-Dienstprogramme auf niedriger Ebene

Übersicht
package require Tcl 8.5 9
package require rcs ?**0.2**?
::rcs::text2dict *text*
::rcs::dict2text *dict*
::rcs::file2dict *filename*
::rcs::dict2file *filename dict*
::rcs::decodeRcsPatch *text*
::rcs::encodeRcsPatch *pcmds*
::rcs::applyRcsPatch *text pcmds*

Beschreibung
Das *Revision Control System*, kurz *RCS*, ist eine Sammlung von Anwendungen und zugehörigen Datenformaten, die es einem System ermöglichen, die Geschichte der Änderungen an einem Text dauerhaft zu speichern. Es bildet zusammen mit seinem Verwandten SCCS die Grundlage für viele andere Systeme dieser Art, wie beispielsweise *CVS* usw.

Dieses Paket implementiert *kein* RCS.

Es stellt lediglich eine Reihe von Befehlen auf niedriger Ebene bereit, die bei der Implementierung eines beliebigen Versionsverwaltungssystems nützlich sein dürften, nämlich:

1. Die Umwandlung von Texten in Datenstrukturen und zurück. Diese Strukturen erlauben, solche Texte leicht durch *Patches* zu ändern, also durch Folgen von Anweisungen, die einen Text in einen anderen umwandeln.
2. Sowie die Umwandlung eines bestimmten Patch-Formats, der sogenannten *RCS-Patches*, in Datenstrukturen und zurück, die ihre einfache Anwendung auf Texte erlauben.

Befehle

::rcs::text2dict *text*
: Wandelt das Argument *text* in ein Dictionary um, das denselben Text in indexierter Form enthält und darstellt, und gibt dieses Dictionary als Ergebnis zurück. Das Format des Ergebnisses beschreibt der Abschnitt Datenstruktur Text-Dict. Dieser Befehl gibt die *kanonische* Darstellung der Eingabe zurück.

::rcs::dict2text *dict*
: Dieser Befehl ist die ergänzende Operation zu **::rcs::text2dict**. Er wandelt ein Dictionary in der im Abschnitt Datenstruktur Text-Dict beschriebenen Form wieder in einen Text um und gibt diesen Text als Ergebnis zurück. Der Befehl akzeptiert auch nicht-kanonische Darstellungen des Textes als Eingabe.

::rcs::file2dict *filename*
: Dieser Befehl entspricht **::rcs::text2dict**, liest den umzuwandelnden Text aber aus der Datei mit dem Pfad *filename*. Die Datei muss vorhanden und lesbar sein.

::rcs::dict2file *filename dict*
: Dieser Befehl entspricht **::rcs::2dict2text**, speichert den entstehenden Text aber in der Datei mit dem Pfad *filename*. Die Datei wird angelegt, wenn sie noch nicht existiert, und muss beschreibbar sein. Das Ergebnis des Befehls ist die leere Zeichenkette.

::rcs::decodeRcsPatch *text*
: Wandelt das Argument *text* in eine Patch-Befehlsliste (PCL) um, wie im Abschnitt Liste der RCS-Patch-Befehle beschrieben, und gibt diese Liste als Ergebnis zurück. Der Eingabetext wird im *diff -n format* angenommen, auch als *RCS-Patch* bekannt, wie im Abschnitt RCS-Patch-Format beschrieben. Der Befehl ignoriert No-ops in der Eingabe; die entstehende PCL enthält also nur Anweisungen, die etwas bewirken.

::rcs::encodeRcsPatch *pcmds*
: Dieser Befehl ist die ergänzende Operation zu **::rcs::decodeRcsPatch**. Er wandelt eine Patch-Befehlsliste (PCL), wie im Abschnitt Liste der RCS-Patch-Befehle beschrieben, in einen Text im RCS-Patch-Format zurück und gibt diesen Text als Ergebnis zurück.

  Dieser Befehl und **::rcs::decodeRcsPatch** sind nicht genau komplementär, weil letzterer No-ops aus seiner Eingabe entfernt, die der Kodierer nicht wieder in den erzeugten RCS-Patch einsetzen kann. Das Ergebnis eines Dekodier-/Kodierschritts stimmt auf Zeichenebene daher möglicherweise nicht mit der ursprünglichen Eingabe überein, auf der funktionalen Ebene aber schon.

::rcs::applyRcsPatch *text pcmds*
: Wendet einen Patch in Form einer PCL auf einen als Dictionary gegebenen Text an und gibt den geänderten Text, wieder als Dictionary, als Ergebnis zurück.

  Für tatsächlichen Text wandeln **::rcs::text2dict** (oder ein gleichwertiger Befehl) und **::rcs::decodeRcsPatch** die Eingaben in Datenstrukturen um, die dieser Befehl annimmt.

Analog wandelt **::rcs::dict2text** (oder ein gleichwertiger Befehl) das Ergebnis dieses Befehls in tatsächlichen Text um.

Datenstruktur Text-Dict

Ein Text-Dictionary ist ein Dictionary, dessen Schlüssel ganze Zahlen und dessen Werte die zugehörigen Zeichenketten sind. Die Schlüssel sind die Zeilennummern eines Textes, die Werte der Text der jeweiligen Zeile. Ein Text kann viele Darstellungen als Dictionary haben, weil die Indexwerte für den Wiederaufbau nur richtig geordnet sein müssen; ihre genauen Werte spielen keine Rolle. Ebenso können die Zeichenketten mehrere physische Zeilen umfassen.

Der Text

```
Hello World,
how are you ?
Fine, and you ?
```

kann beispielsweise wie folgt dargestellt werden:

```
{{1 {Hello World,}} {2 {how are you ?}} {3 {Fine, and you ?}}}
```

oder

```
{{5 {Hello World,}} {8 {how are you ?}} {9 {Fine, and you ?}}}
```

oder

```
{{-1 {Hello World,
how are you ?}} {4 {Fine, and you ?}}}
```

Das erste Dictionary stellt die *kanonische* Darstellung des Textes dar, wobei die Zeilennummern bei **1** beginnen, in Schritten von **1** ansteigen und keine Lücken aufweisen, wobei jeder Wert genau einer physischen Zeile entspricht.

Alle Befehle, die aus Text ein Dictionary erzeugen, geben die kanonische Darstellung ihres Eingabetextes zurück. Die Befehle, die ein Dictionary annehmen und Text zurückgeben, akzeptieren in der Regel jede Darstellung, kanonisch oder nicht.

Die Anwendung eines Patches auf ein Text-Dictionary führt in der Regel dazu, dass das Dictionary nicht mehr kanonisch ist.

RCS-Patch-Format

Ein *Patch* ist im Allgemeinen eine Folge von Anweisungen, wie ein Eingabetext T in einen anderen Text T' umgewandelt wird, und ebenfalls in Textform kodiert.

Das von diesem Paket verstandene Textformat für Patches ist sehr einfach und unter den Namen *RCS-Patch* oder *diff -n format* bekannt.

Patches in diesem Format enthalten lediglich zwei verschiedene Befehle: zum Löschen von altem Text und zum Hinzufügen von neuem Text. Das Ersetzen eines Textes durch einen anderen wird als Kombination aus einer Löschung, gefolgt von einer Hinzufügung, behandelt.

Das Format ist zeilenorientiert, wobei jede Zeile entweder einen Befehl oder Textdaten enthält, die dem vorangehenden Befehl zugeordnet sind. Die erste Zeile eines *RCS-Patches* ist stets eine Befehlszeile.

Die Befehle lauten:

""
: Die Leerzeile ist ein Befehl, der keine Aktion auslöst.

"astart n"
: Eine Zeile, die mit dem Zeichen **a** beginnt, fügt Text zur Ausgabe hinzu. Darauf folgen **n** Zeilen Textdaten. Beim Anwenden des Patches werden die Daten genau zwischen den Zeilen **start** und **start**+1 eingefügt. Denselben Effekt hat das Anhängen der Daten an den vorhandenen Text der Zeile **start**. Eine nicht vorhandene Zeile **start** wird angelegt.

"dstart n"
: Eine Zeile, die mit dem Zeichen **d** beginnt, löscht Text aus der Ausgabe. Angewendet löscht sie **n** Zeilen Text; die erste gelöschte Zeile hat den Index **start**.

Die Zeilenindizes **start** beziehen sich immer auf den umzuwandelnden Text in seinem ursprünglichen Zustand, ohne die vorhergehenden Änderungen.

Die Anweisungen sind in der Reihenfolge anzuwenden, in der sie im Patch stehen, oder so, dass dasselbe Ergebnis entsteht wie bei der Anwendung in dieser Reihenfolge.

Dies ist das Format der Ergebnisse, die **::rcs::decodeRcsPatch** zurückgibt und die **::rcs::encodeRcsPatch** beziehungsweise **::rcs::appplyRcsPatch** annehmen. Der Dekodierer entfernt jedoch No-op-Befehle, und der Kodierer erzeugt keine No-ops; auf der Textebene sind sie daher nicht vollständig komplementär, nur auf der funktionalen Ebene.

Ein Beispiel für einen RCS-Patch ist

```
d1 2
d4 1
a4 2
The named is the mother of all things.

a11 3
They both may be called deep and profound.
Deeper and more profound,
The door of all subtleties!
```

Liste der RCS-Patch-Befehle

Patch-Befehlslisten (kurz: PCLs) sind die Datenstrukturen, die der Patch-Dekodierer erzeugt und die der Patch-Kodierer sowie der Befehl zum Anwenden des Patches annehmen. Sie stellen RCS-Patches als Tcl-Datenstrukturen dar.

Eine PCL ist eine Liste, in der jedes Element einen einzelnen Patch-Befehl darstellt, entweder eine Hinzufügung oder eine Löschung. Die Elemente sind selbst Listen, wobei das erste Element den Befehl festlegt und die übrigen Elemente die Argumente des Befehls darstellen.

a
: Dies ist die Anweisung zum Einfügen von Text. Sie hat zwei Argumente: den Index der Zeile, in die der Text eingefügt werden soll, und den einzufügenden Text – in dieser Reihenfolge.

d
: Dies ist die Anweisung zum Löschen von Text. Sie hat zwei Argumente, den Index der Zeile, an der das Löschen beginnt, und die Anzahl der zu löschenden Zeilen, in dieser Reihenfolge.

Dies ist das Format, das der Patch-Dekodierer zurückgibt und das der Patch-Kodierer und der Befehl zum Anwenden als Eingabe annehmen.

Unten steht ein Beispiel für einen Patch-Befehl; es stellt den RCS-Patch aus dem Abschnitt RCS-Patch-Format dar.

```
{{d 1 2} {d 4 1} {a 4 {The named is the mother of all things.

}} {a 11 {They both may be called deep and profound.
Deeper and more profound,
The door of all subtleties!}}}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *rcs* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

Copyright

```
Copyright (c) 2005, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2005, Colin McCormack <coldstore@users.sourceforge.net>
```

report

report

Stichwörter: report, matrix, table

Name

report – Berichtsobjekte erstellen und bearbeiten

Übersicht

package require Tcl 8.5 9
package require report ?**0.5**?
::report::report *reportName columns* ?*style style arg...*?
reportName *option* ?*arg arg ...*?
::report::defstyle *styleName arguments script*
::report::rmstyle *styleName*
::report::stylearguments *styleName*
::report::stylebody *styleName*

::report::styles
reportName **destroy**
reportName **templatecode disable|enable**
reportName **templatecode enabled**
reportName **templatecode get**
reportName **templatecode set** *templatedata*
reportName **tcaption** ?*size*?
reportName **bcaption** *size*
reportName **size** *column* ?*number|dyn*?
reportName **sizes** ?*size-list*?
reportName **pad** *column* ?*left|right|both padstring*?
reportName **justify** *column* ?**left|right|center**?
reportName **printmatrix** *matrix*
reportName **printmatrix2channel** *matrix chan*
reportName **columns**

Beschreibung

Dieses Paket stellt Berichtsobjekte bereit, die von den Formatierungsmethoden der Matrixobjekte verwendet werden können, um tabellarische Berichte der Matrix in verschiedenen Formen zu erstellen. Die hier definierten Berichtsobjekte unterteilen jeden Bericht in drei Regionen und zehn Klassen von *Zeilen* (verschiedene Trenn- und Datenzeilen). Ausführlichere Erläuterungen finden Sie im folgenden Abschnitt.

::report::report *reportName columns* ?style *style arg...*?
Erstellt ein neues Berichtsobjekt für einen Bericht mit *columns* Spalten, dem ein globaler Tcl-Befehl mit dem Namen *reportName* zugeordnet ist. Mit diesem Befehl lassen sich verschiedene Vorgänge der Konfiguration für den Bericht ausführen. Er hat die folgende allgemeine Form:

reportName *option* ?*arg arg ...*?
Die *Option* und die *arg* bestimmen das genaue Verhalten des Befehls. Weitere Erläuterungen finden Sie im Abschnitt Berichtsmethoden. Wenn kein **style** festgelegt wird, verwendet der Bericht den integrierten Stil **plain** als Standard-Konfiguration.

::report::defstyle *styleName arguments script*
Definiert den neuen Stil *styleName*. Weitere Informationen finden Sie im Abschnitt Styles.

::report::rmstyle *styleName*
Löscht den Stil *styleName*. Der Versuch, einen unbekannten oder integrierten Stil zu löschen, führt zu einer Fehlermeldung. Beachten Sie, dass dieser Befehl nicht überprüft, ob andere Stile von dem zu löschenden Stil abhängig sind. Das Löschen eines Stils, der noch von einem anderen Stil FOO verwendet wird, führt zu einem Laufzeitfehler, wenn FOO auf einen neu instanziierten Bericht angewendet wird.

::report::stylearguments *styleName*
Dieser Befehl gibt die Liste der Argumente zurück, die dem Stil *styleName* zugeordnet sind.

::report::stylebody *styleName*
Dieser Introspektionsbefehl gibt das Skript zurück, das dem Stil *styleName* zugeordnet ist.

::report::styles
Dieser Introspektionsbefehl gibt eine Liste zurück, die die Namen aller Stile enthält, die dem

Paket zum Zeitpunkt des Aufrufs bekannt sind. Die Reihenfolge der Namen in der Liste entspricht der Reihenfolge, in der die Styles erstellt wurden. Mit anderen Worten: Das erste Element ist der vordefinierte Stil "**plain**", gefolgt vom ersten vom Benutzer definierten Stil und so weiter.

Regionen

Die drei Bereiche sind die *obere Beschriftung*, der *Datenbereich* und die *untere Beschriftung*. Dabei handelt es sich, grob gesagt, um den Titel, die zu darstellenden Werte und eine Beschriftung am unteren Rand. Die Größe der Beschriftungsbereiche kann vom Benutzer als Anzahl der Zeilen festgelegt werden, die sie in der zu formatierenden Matrix einnehmen. Die Größe des Datenbereichs wird implizit festgelegt.

Zeilen

Vorlagen sind den zehn Linienklassen zugeordnet und legen die Formatierung für die jeweilige Linienart fest. Der Benutzer kann die Trennlinien nach Belieben aktivieren und deaktivieren, nicht jedoch die Datenzeilen. Ihre Verwendung richtet sich ausschließlich nach der Anzahl der Zeilen, die in den drei Bereichen enthalten sind. Datenzeilen und alle aktivierten Trennlinien müssen einer Vorlage zugeordnet sein.

Hinweis: Die Datenzeilen in einem Bericht und die Zeilen in der Matrix, aus der der Bericht erstellt wurde, weisen *keine* 1:1-Beziehung auf, wenn eine beliebige Zeile in der Matrix eine Höhe von mehr als eins aufweist.

Die verschiedenen Arten von Zeilen und die Codes, die von den Berichtsmethoden zur Kennzeichnung dieser Zeilen verwendet werden, sind:

top
: Die oberste Zeile eines Berichts. Sie trennt den Bericht von allem, was davor steht. Der Benutzer kann die Verwendung dieser Zeile nach Belieben aktivieren.

topdatasep
: Diese Zeile dient dazu, die Datenzeilen im oberen Beschriftungsbereich voneinander zu trennen, wenn dieser mehr als eine Zeile enthält und der Benutzer die Verwendung dieser Zeile aktiviert hat.

topcapsep
: Diese Zeile dient dazu, den oberen Beschriftungsbereich und den Datenbereich voneinander zu trennen, wenn der obere Beschriftungsbereich nicht leer ist und der Benutzer die Verwendung dieser Funktion aktiviert hat.

datasep
: Diese Zeile dient dazu, die Datenzeilen im Datenbereich voneinander zu trennen, wenn dieser mehr als eine Zeile enthält und der Benutzer diese Funktion aktiviert hat.

botcapsep
: Diese Zeile dient dazu, den Datenbereich und den Bereich für die untere Beschriftung voneinander zu trennen, wenn die untere Beschriftung nicht leer ist und der Benutzer deren Verwendung aktiviert hat.

botdatasep
: Diese Zeile dient dazu, die Datenzeilen im unteren Beschriftungsbereich voneinander zu trennen, wenn dieser mehr als eine Zeile enthält und der Benutzer die Verwendung dieser Zeile aktiviert hat.

bottom
Die unterste Zeile eines Berichts. Sie trennt den Bericht von allem, was danach folgt. Der Benutzer kann die Verwendung dieser Zeile nach Belieben aktivieren.

topdata
Diese Zeile legt das Format der Datenzeilen im oberen Kopfbereich des Berichts fest.

data
Diese Zeile legt das Format der Datenzeilen im Datenbereich des Berichts fest.

botdata
Diese Zeile legt das Format der Datenzeilen im unteren Beschriftungsbereich des Berichts fest.

Vorlagen

Jede Vorlage ist eine Liste von Zeichenfolgen, die zur Formatierung der Zeile verwendet werden, der sie zugeordnet ist. Bei einem Bericht, der die Spalten "**n**" enthält, muss eine Vorlage für eine Datenzeile "**n**+1" Elemente und eine Vorlage für eine Trennzeile "2*__n__+1" Elemente enthalten.

Die Elemente in einer Datenvorlage legen die Zeichenfolgen fest, die zur Trennung der Informationen über die Spalten verwendet werden. Zusammen mit den entsprechenden Elementen in den Trennzeichenvorlagen bilden sie die vertikalen Linien im Bericht.

Hinweis: Die entsprechenden Einträge in allen definierten Vorlagen müssen dieselbe Länge aufweisen. Dies wird vom Berichtsobjekt überprüft. Der erste Eintrag definiert die linke vertikale Linie und der letzte Eintrag die rechte vertikale Linie. Der Eintrag am Index **k** ("1", ..., "**n**-2") trennt die Informationen in den Spalten "**k**-1" und "**k**".

Die Einträge in einer Trennzeichenvorlage mit einem geraden Index ("0", "2", ...) legen die Trennzeichen für die Spalten fest. Der Eintrag am Index "2*__k__" ("0", "2", ..., "2*__n__") entspricht den Einträgen am Index "**k**" in den Datenvorlagen.

Die Einträge in einer Trennzeichenvorlage mit ungeraden Indizes ("1", "3", ...) legen die Zeichenfolgen fest, aus denen die horizontalen Linien in den Trennzeilen gebildet werden. Das Element am Index "2*__k__+1" ("1", "3", ..., "2*__n__+1") entspricht der Spalte "**k**". Bei der Erzeugung der horizontalen Linien werden die Elemente so lange dupliziert, bis sie mindestens die Länge ihrer Spalten erreichen, und anschließend auf die exakte Größe zugeschnitten.

Styles

Mithilfe von Styles kann der Benutzer dieses Pakets allgemeine Konfigurationen für Berichtsobjekte festlegen und diese später bei der eigentlichen Instanziierung der Berichtsobjekte verwenden. Sie werden als TCL-Skripte definiert, die bei ihrer Ausführung das Berichtsobjekt entsprechend der gewünschten Konfiguration einrichten.

Der Befehl zum Definieren von Stilen lautet **::report::defstyle**. Sein letztes Argument ist der TCL-Befehl **script**, der die eigentliche Neukonfiguration des Berichtsobjekts durchführt, um den gewünschten Stil zu erzielen.

In diesem Skript stehen die Namen aller zuvor definierten Stile als Befehle zur Verfügung, ebenso wie alle Befehle, die in einem sicheren Interpreter zu finden sind, sowie die Methoden der Konfiguration von Berichtsobjekten. Letztere wirken implizit auf das Objekt, das das Stilskript gerade ausführt. Die hier deklarierten **arguments** stehen im **script** als Variablen zur Verfügung. Beim Aufruf des Befehls eines zuvor deklarierten Stils müssen alle von diesem erwarteten

Argumente im Aufruf definiert werden.

Berichtsmethoden

Für Berichtsobjekte sind folgende Befehle möglich:

***reportName* destroy**
Löscht den Bericht einschließlich seines Speicherplatzes und des zugehörigen Befehls.

***reportName templatecode* disable|enable**
Aktiviert oder deaktiviert die Verwendung der Vorlage, auf die der *templatecode* verweist. Hier sind ausschließlich Codes für Trennlinien zulässig. Es ist nicht möglich, Datenzeilen zu aktivieren oder zu deaktivieren.

Durch die Aktivierung einer Vorlage überprüft der Bericht alle verwendeten Vorlagen auf Inkonsistenzen in den Definitionen der vertikalen Linien (siehe Abschnitt Vorlagen).

***reportName templatecode* enabled**
Gibt zurück, ob die durch *templatecode* angegebene Vorlage derzeit aktiviert ist oder nicht.

***reportName templatecode* get**
Gibt die Vorlage zurück, die derzeit dem durch den *templatecode* angegebenen Zeilentyp zugeordnet ist. Hier sind alle bekannten templatecodes zulässig.

reportName templatecode* set *templatedata
Legt die Vorlage fest, die mit der durch den *templatecode* bezeichneten Zeilenart verknüpft ist, und setzt sie auf den neuen Wert in *templatedata*. Informationen zu den Constraints hinsichtlich der Länge von Vorlagen finden Sie im Abschnitt Vorlagen.

***reportName* tcaption ?*size*?**
Legt die *size* des oberen Beschriftungsbereichs als die Anzahl der Zeilen fest, die er in der zu formatierenden Matrix einnimmt. Es sind nur Zahlen größer oder gleich Null zulässig. Wird keine *size* angegeben, gibt der Befehl stattdessen die aktuelle Größe zurück.

Wird die Größe der oberen Beschriftung auf einen Wert größer als Null gesetzt, wird die entsprechende Datenvorlage aktiviert und der Bericht überprüft alle verwendeten Vorlagen auf Inkonsistenzen in den Definitionen der vertikalen Linien (siehe Abschnitt Vorlagen).

reportName* bcaption *size
Legt die *size* des unteren Beschriftungsbereichs als die Anzahl der Zeilen fest, die er in der zu formatierenden Matrix einnimmt. Es sind nur Zahlen größer oder gleich Null zulässig. Wird keine *size* angegeben, gibt der Befehl stattdessen die aktuelle Größe zurück.

Wird die Größe der unteren Beschriftung auf einen Wert größer als Null gesetzt, wird die entsprechende Datenvorlage aktiviert und der Bericht überprüft alle verwendeten Vorlagen auf Inkonsistenzen in den Definitionen der vertikalen Linien (siehe Abschnitt Vorlagen).

***reportName* size *column* ?*number*|dyn?**
Legt die Breite der *column* in der Ausgabe fest. Der Wert **dyn** bedeutet, dass die von der zu formatierenden Matrix für die angegebene Spalte zurückgegebene Spaltenbreite verwendet werden soll. Die Formatierung der Spalte erfolgt dynamisch. Wird anstelle von **dyn** eine feste *number* verwendet, bedeutet dies, dass die Spalte eine Breite von genau dieser Anzahl an Zeichen hat (ohne Innenabstand). Hier sind nur Zahlen größer als Null zulässig.

Wird keine Größenangabe gemacht, gibt der Befehl stattdessen die aktuelle Größe der *column* zurück.

***reportName* sizes ?*size-list*?**

Mit dieser Methode kann der Benutzer die Breiten aller Spalten in einem einzigen Aufruf festlegen. Als Argument dient eine Liste, die die den Spalten zuzuordnenden Breiten enthält. Das erste Element wird der Spalte 0 zugeordnet, das nächste der Spalte 1 und so weiter.

Wird keine *Breitenliste* angegeben, gibt der Befehl stattdessen eine Liste zurück, die die aktuell eingestellten Breiten enthält.

***reportName* pad *column* ?left|right|both ?*padstring*??**

Mit dieser Methode kann der Benutzer einen Innenabstand auf der linken, rechten oder beiden Seiten einer *column* festlegen. Wenn das *padstring* nicht angegeben wird, wird standardmäßig ein einzelnes Leerzeichen verwendet. *Hinweis*: Eine alternative Möglichkeit, den Innenabstand festzulegen, besteht darin, in den Vorlagen vertikale Trennzeichen zu verwenden, die länger als ein Zeichen sind (siehe Abschnitt Vorlagen).

Wenn überhaupt keine Ausrichtungsangabe gemacht wird, gibt der Befehl stattdessen den aktuellen Zustand des Innenabstands der Spalten zurück. Dabei handelt es sich um eine Liste mit zwei Elementen, wobei das erste Element die linke Ausrichtung und das zweite die rechte Ausrichtung beschreibt.

***reportName* justify *column* ?left|right|center?**

Legt fest, wie die Werte der Zellen für eine *column* in den Bericht eingefügt werden, unter Berücksichtigung der angegebenen Spaltenbreite im Bericht.

Bei den Ausrichtungsarten **left** und **right** wird ein Wert, der kürzer als die Breite der Spalten ist, mit seiner benannten Kante an die entsprechende Kante der Spalten gebunden. Die andere Seite wird mit Leerzeichen aufgefüllt. Im Fall von **center** werden die Leerzeichen auf beiden Seiten des Wertes platziert, wobei die Anzahl der Leerzeichen auf der linken Seite höchstens um eins größer ist als die auf der rechten Seite.

Bei einem Wert, der länger ist als die Breite der Spalte, wird der Wert an der benannten Kante abgeschnitten. Dies bedeutet bei der Ausrichtung **left**, dass der *Endteil* (d. h. der Teil **right**) des Werts in der Ausgabe sichtbar gemacht wird. Bei "**center**" wird der Wert an beiden Seiten gekürzt, um in die Spalten zu passen, und die Anzahl der Zeichen, die auf der linken Seite des Werts gekürzt werden, ist höchstens um eins geringer als die Anzahl der Zeichen, die auf der rechten Seite gekürzt werden.

Wurde keine Ausrichtung festgelegt, gibt der Befehl stattdessen die aktuelle Ausrichtung der Spalten zurück.

reportName* printmatrix *matrix

Formatiert die *matrix* entsprechend der Konfiguration des Berichts und gibt die resultierende Zeichenfolge zurück. Die Matrix muss dieselbe Anzahl an Spalten wie der Bericht aufweisen. Die Matrix muss zudem über genügend Zeilen verfügen, damit sich die oberen und unteren Beschriftungsbereiche nicht überlappen. Der Datenbereich darf leer sein.

reportName* printmatrix2channel *matrix chan

Formatiert die *matrix* entsprechend der Konfiguration des Berichts und schreibt das Ergebnis

in den Kanal *chan*. Die Matrix muss dieselbe Anzahl an Spalten wie der Bericht aufweisen. Die Matrix muss zudem über genügend Zeilen verfügen, damit sich die oberen und unteren Beschriftungsbereiche nicht überlappen. Der Datenbereich darf leer sein.

***reportName* columns**
Gibt die Anzahl der Spalten im Bericht zurück.

Die Methoden **size**, **pad** und **justify** nehmen alle einen Index für eine Spalte als erstes Argument entgegen. Dieser Index darf alle Formen eines Indexes verwenden, die vom Befehl **lindex** akzeptiert werden. Der zulässige Bereich für Indizes lautet "0, ...,[**reportName** Spalten]-1".

Beispiele

Unsere Beispiele definieren einige allgemein nützliche Berichtsstyles.

Eine einfache Tabelle mit Rahmen um alle Informationen und vertikalen Trennlinien, jedoch ohne interne horizontale Trennlinien.

```
    ::report::defstyle simpletable {} {
    data    set [split "[string repeat "| "   [columns]]|"]
    top set [split "[string repeat "+ - " [columns]]+"]
    bottom  set [top get]
    top enable
    bottom  enable
    }
```

Eine Erweiterung des oben genannten **simpletable** mit einem Titelbereich.

```
    ::report::defstyle captionedtable {{n 1}} {
    simpletable
    topdata   set [data get]
    topcapsep set [top get]
    topcapsep enable
    tcaption $n
    }
```

Ausgehend von den obigen Definitionen folgt nun ein Beispiel, bei dem eine Matrix tatsächlich in einen tabellarischen Bericht umgewandelt wird. Dabei wird davon ausgegangen, dass die Matrix tatsächlich aussagekräftige Daten enthält.

```
    % ::struct::matrix m
    % # ... fill m with data, assume 5 columns
    % ::report::report r 5 style captionedtable 1
    % r printmatrix m
    +---+------------------+-------+-------+--------+
    |000|VERSIONS:         |2:8.4a3|1:8.4a3|1:8.4a3%|
    +---+------------------+-------+-------+--------+
    |001|CATCH return ok   |7      |13     |53.85   |
    |002|CATCH return error|68     |91     |74.73   |
    |003|CATCH no catch used|7     |14     |50.00   |
    |004|IF if true numeric |12    |33     |36.36   |
```

```
    |005|IF elseif          |15     |47     |31.91   |
    |   |true numeric       |       |       |        |
    +---+-------------------+-------+-------+--------+
    %
    % # alternate way of doing the above
    % m format 2string r
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *report* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright


```
Copyright (c) 2002-2014,2022 Andreas Kupries
    <andreas.kupries@gmail.com>
```


rest

rest

Stichwörter: rest

Name

rest – REST-Web-APIs definieren und inline oder asynchron aufrufen

Übersicht

package require Tcl 8.5 9
package require rest ?**1.7**?
::rest::simple *url query* ?*config*? ?*body*?
::rest::get *url query* ?*config*? ?*body*?
::rest::post *url query* ?*config*? ?*body*?
::rest::patch *url query* ?*config*? ?*body*?
::rest::head *url query* ?*config*? ?*body*?
::rest::put *url query* ?*config*? ?*body*?
::rest::delete *url query* ?*config*? ?*body*?
::rest::save *name file*
::rest::describe *name*
::rest::parameters *url* ?*key*?
::rest::parse_opts *static required optional words*

::rest::substitute *string var*
::rest::create_interface *name*

Beschreibung

Dieses Paket unterstützt zwei Arten der Nutzung: *einfache Aufrufe* und vollständige *Schnittstellen*. In einer *Schnittstelle* legen Sie eine Reihe von Regeln fest, woraufhin das Paket die Befehle erstellt, die den REST-Methoden entsprechen. Diese Befehle können zahlreiche Optionen umfassen, wie beispielsweise Eingabe- und Ausgabetransformationen sowie datentypspezifische Formatierungen. Dies führt zu einem übersichtlicheren und einfacheren Skript. Ein *einfacher Aufruf* hingegen lässt sich zwar einfacher und schneller implementieren, bietet jedoch weniger Funktionen. Er nimmt die URL und einige Optionen zum Befehl entgegen und gibt das Ergebnis direkt zurück. Jegliche Formatierung oder Überprüfung obliegt dem restlichen Skript.

Einfache Anwendung

Im einfachen Anwendungsfall führen Sie Aufrufe mithilfe der HTTP-Prozeduren durch und überprüfen oder verarbeiten die zurückgegebenen Daten anschließend selbst.

::rest::simple *url query* ?*config*? ?*body*?
siehe unten ...

::rest::get *url query* ?*config*? ?*body*?
siehe unten ...

::rest::post *url query* ?*config*? ?*body*?
siehe unten ...

::rest::patch *url query* ?*config*? ?*body*?
siehe unten ...

::rest::head *url query* ?*config*? ?*body*?
siehe unten ...

::rest::put *url query* ?*config*? ?*body*?
siehe unten ...

::rest::delete *url query* ?*config*? ?*body*?
Diese Befehle sind bis auf die verwendete HTTP-Methode alle gleichwertig. Wenn Sie **simple** verwenden, sollte die Methode als Option im *config*-Dictionary festgelegt werden. Ist dies nicht der Fall, wird standardmäßig **get** verwendet. Wenn ein *body* benötigt wird, muss das *config*-Dictionary vorhanden sein; es darf jedoch leer sein.

Das *config*-Dictionary unterstützt die folgenden Schlüssel
- **auth**
- **content-type**
- **cookie**
- **error-body**
- **format**
- **headers**
- **method**
- **timeout**

Zwei kurze Beispiele:

Beispiel 1, Yahoo-Chef:

```
    set appid APPID
    set search tcl
    set res [rest::get
    http://boss.yahooapis.com/ysearch/web/v1/$search [list appid
    $appid]]
    set res [rest::format_json $res]
```

Beispiel 2, Twitter:

```
    set url   http://twitter.com/statuses/update.json
    set query [list status $text]
    set res [rest::simple $url $query {
        method post
        auth   {basic user password}
        format json
    }]
```

Verwendung der Schnittstelle

Eine Schnittstelle zu einer REST-API besteht aus einer Reihe von Definitionen von REST-Aufrufen, die in einem Array enthalten sind. Der Name dieses Arrays wird zu einem Namespace, der die definierten Befehle enthält. Jeder Schlüssel des Arrays legt den Namen des Aufrufs fest, wobei die zugehörige Konfiguration aus einem Dictionary, d. h. Schlüssel-Wert-Paaren, besteht. Die zulässigen Schlüssel, d. h. die gültigen Konfigurationsoptionen, werden im Folgenden beschrieben. Nachdem Sie die Definitionen im Array erstellt haben, rufen Sie einfach **rest::create_interface** mit dem Array als Argument auf, um die gewünschten Befehle zu erstellen.

Beispiele: Yahoo Wetter:

```
   package require rest

   set yweather(forecast) {
      url      http://weather.yahooapis.com/forecastrss
      req_args { p: }
      opt_args { u: }
   }
   rest::create_interface yweather
   puts [yweather::forecast -p 94089]
```

::rest::save *name file*

Dieser Befehl speichert eine Kopie der dynamisch erstellten Prozeduren für alle in der Array-Variablen *name* angegebenen API-Aufrufe in der Datei *file*, um sie später laden zu können.

Das Ergebnis des Befehls ist die leere Zeichenkette

::rest::describe *name*

Dieser Befehl gibt eine Beschreibung aller in der Array-Variablen *name* angegebenen API-Aufrufe auf dem Kanal **stdout** aus.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::rest::parameters *url* ?*key*?

Dieser Befehl analysiert eine *url*-Abfragezeichenfolge und wandelt sie in ein Dictionary um; anschließend gibt er dieses Dictionary als Ergebnis zurück.

Wenn *key* festgelegt wird, gibt der Befehl nicht das gesamte Dictionary zurück, sondern lediglich den Wert des festgelegten *key*.

::rest::parse_opts *static required optional words*

Dieser Befehl implementiert einen benutzerdefinierten Parser für Befehlsoptionen.

dict *static*

Ein Verzeichnis von Optionen und deren Werten, die stets in der Ausgabe enthalten sind.

list *required*

Eine Liste von Optionen, die durch *words* angegeben werden müssen

list *optional*

Eine Liste von Optionen, die in den *words* vorkommen können, jedoch nicht erforderlich sind. Die Elemente müssen in einer der folgenden drei Formen vorliegen:

name

Die Option kann vorhanden sein oder auch nicht; es gibt keine Standardeinstellung.

name:

Sofern vorhanden, erfordert diese Option ein Argument.

name:value

Wenn dieser nicht vorhanden ist, verwenden Sie bitte **value** als Standardwert.

list *words*

Die Wörter, die in Optionen und Werte zerlegt werden sollen.

Das Ergebnis des Befehls ist eine Liste mit zwei Elementen. Das erste Element ist ein Dictionary, das die analysierten Optionen und deren Werte enthält. Das zweite Element ist eine Liste der verbleibenden Wörter.

::rest::substitute *string var*

Dieser Befehl nimmt eine *string* entgegen, ersetzt darin alle gefundenen Optionsbezeichner durch Werte und gibt die geänderte Zeichenkette als Ergebnis zurück.

Die zu ersetzenden Werte befinden sich in der Variablen *var*, die ein Dictionary enthalten soll, das die zu ersetzenden Optionsbezeichner ihren jeweiligen Werten zuordnet. *Hinweis*: Optionen-Bezeichner, für die in *var* kein Schlüssel vorhanden ist, werden durch die leere Zeichenkette ersetzt.

Die Bezeichner für Optionen in *string* müssen der Syntax **%...%** entsprechen, wobei**...** eine beliebige Kombination aus Kleinbuchstaben, Ziffern, Unterstrichen, Doppelpunkten und Bindestrichen enthalten darf.

::rest::create_interface *name*

Dieser Befehl erstellt Prozeduren für alle API-Aufrufe, die in der Array-Variablen *name*

angegeben sind.

Der Name dieses Arrays wird zu einem Namespace, der die definierten Befehle enthält. Jeder Schlüssel des Arrays legt den Namen des Aufrufs fest, wobei die zugehörige Konfiguration aus einem Dictionary besteht, d. h. aus Schlüssel-Wert-Paaren. Die zulässigen Schlüssel und ihre Bedeutungen lauten wie folgt:

url
: Der Wert dieser *required* Option muss das Ziel der HTTP-Anfrage sein.

description
: Der Wert dieser Option muss eine kurze Zeichenfolge sein, die den Aufruf beschreibt. Wenn kein Wert angegeben wird, gilt standardmäßig die leere Zeichenfolge. Wird ausschließlich von **::rest::describe** verwendet.

body
: Der Wert dieser Option gibt an, ob für den Anforderungstext des Aufrufs Argumente erforderlich sind oder nicht. Die zulässigen Werte sind nachstehend aufgeführt. Sofern nicht anders angegeben, lautet der Standardwert **optional**.

  none
  : Der Aufruf enthält keinen Anforderungstext; es darf kein Anforderungstext angegeben werden.

  optional
  : Ein Request-Body kann angegeben werden, ist jedoch nicht erforderlich.

  required
  : Es muss ein Request-Body angegeben werden.

  argument
  : Auf diesen Wert muss der Name einer Option folgen, wobei die gesamte Zeichenfolge als Liste behandelt wird. Der Request-Body wird als Wert dieser Option verwendet.

  mime_multipart
  : Ein Anfragetext muss angegeben werden und wird so interpretiert, dass jedes Argument einen Teil eines MIME/Multipart-Dokuments darstellt. Argumente müssen Listen mit zwei Elementen sein: einer Liste von Header-Schlüsseln und -Werten sowie dem MIME-Teiltext, und zwar in dieser Reihenfolge.

  mime_multipart/
  : Entspricht "mime_multipart", jedoch wird der Header "**Content-Type**" auf "**multipart/**" gesetzt.

method
: Der Wert dieser Option muss der Name der HTTP-Methode sein, die für die URL aufgerufen werden soll. Sofern nicht anders angegeben, ist die Standardeinstellung "GET". Zulässige Werte sind **GET**, **POST** und **PUT**, wobei die Groß- und Kleinschreibung keine Rolle spielt.

copy
: Wenn der Wert dieser Option angegeben ist, legt er den Namen eines zuvor definierten Aufrufs fest. Die Definition dieses Aufrufs wird in den aktuellen Aufruf übernommen, mit Ausnahme der Optionen, die vom aktuellen Aufruf selbst angegeben wurden.

unset
: Wenn der Wert dieser Option vorhanden ist, enthält er eine Liste der Optionen im aktuellen Aufruf. Diese Optionen werden aus den Definitionen entfernt. Verwenden Sie diese Option,

nachdem Sie eine bestehende Definition mit **copy** bearbeitet haben, um Optionen zu entfernen, anstatt deren Wert zu überschreiben.

headers
Angabe zusätzlicher Header-Felder. Der Wert dieser Option muss ein Dictionary sein, das die neuen Header-Felder und deren Werte enthält. Standardmäßig werden keine zusätzlichen Header hinzugefügt.

content-type
Der Wert dieser Option legt den Inhaltstyp für die Anfragedaten fest.

req_args
Der Wert dieser Option ist eine Liste, in der die erforderlichen Argumente des Aufrufs aufgeführt sind. Für Namen, die mit einem Doppelpunkt enden, muss ein Wert angegeben werden.

opt_args
Der Wert dieser Option ist eine Liste, in der die Argumente aufgeführt sind, die bei einem Aufruf vorhanden sein können, jedoch nicht erforderlich sind.

static_args
Der Wert dieser Option ist eine Liste, in der die Argumente aufgeführt sind, die stets gleich bleiben. Es macht keinen Sinn, den Benutzer damit zu belasten. Ein führender Bindestrich (**-**) ist zulässig, jedoch nicht erforderlich, um die Konsistenz mit der Befehlszeile zu wahren.

auth
Der Wert dieser Option legt fest, wie die Aufrufe authentifiziert werden sollen. Wird die Option nicht angegeben, findet keine Authentifizierung statt.

basic
Der Benutzer kann die *Basisauthentifizierung* konfigurieren, indem er die Prozedur **basic_auth** im Namespace "interface" überschreibt. Diese Prozedur benötigt zwei Argumente: den Benutzernamen und das Passwort, in dieser Reihenfolge.

bearer
Der Benutzer kann ein Bearer-Token als Authentifizierung konfigurieren. Der Wert entspricht dem Token, das an den HTTP-Autorisierungsheader übergeben wird.

sign
Der Wert muss tatsächlich eine Liste sein, deren zweites Element den Namen einer Prozedur enthält, die zur Signierung der Anfrage aufgerufen wird.

callback
Wenn diese Option vorhanden ist, wird die Methode als *asynchroner* Aufruf erstellt. Solche Aufrufe geben sofort den Wert des zugehörigen HTTP-Tokens zurück, anstatt das Ergebnis des Aufrufs zu geben. Die Ereignisschleife muss aktiv sein, um diese Option nutzen zu können.

Der Wert dieser Option wird als Befehlspräfix behandelt, das aufgerufen wird, sobald der HTTP-Aufruf abgeschlossen ist. Das Präfix erhält mindestens zwei zusätzliche Argumente: den Namen der aufrufenden Prozedur und den Status des Ergebnisses (entweder **OK** oder **ERROR**) in dieser Reihenfolge.

Im Falle von **OK** wird ein drittes Argument hinzugefügt: die mit dem Ergebnis verbundenen

Daten.

Nur wenn es sich bei **ERROR** um eine Weiterleitung handelt, wird die Zieladresse der Weiterleitung als Argument hinzugefügt. Wenn der Konfigurationsschlüssel **error-body** auf **true** gesetzt wird, werden die mit dem Ergebnis verbundenen Daten ebenfalls als Argument hinzugefügt.

Der HTTP-Anfrage-Header steht in dieser Prozedur über **upvar token token** zur Verfügung.

cookie
Der Wert dieser Option ist eine Liste von Cookies, die im HTTP-Header übermittelt werden sollen. Dies ist eine Abkürzung für die Option **headers**.

input_transform
Der Wert dieser Option ist ein Befehlspräfix oder ein Skript, mit dem vor dem Aufruf des Befehls eine Transformation der Abfrage durchgeführt wird. Eine Skripttransformation wird in eine automatisch generierte interne Prozedur eingebettet.

Wird kein Wert angegeben, erfolgt keine Transformation.

Der Befehl (Präfix) muss ein einzelnes Argument akzeptieren – nämlich die zu transformierende Abfrage (ein Dictionary) – und muss die modifizierte Abfrage (ebenfalls als Dictionary) als Ergebnis geben. Auf den Request-Body kann im Befehl zur Transformation über **upvar body body** zugegriffen werden.

format
siehe unten ...

result
Der Wert dieser Option legt das Format der zurückgegebenen Daten fest. Sofern nicht anders angegeben, lautet der Standardwert **auto**. Zulässige Werte sind:

auto
Automatische Erkennung zwischen **xml** und **json**.

discard
siehe unten ...

json
siehe unten ...

raw
siehe unten ...

rss
Dies ist als Sonderfall von **xml** formatiert.

- **tdom**
- **xml**

pre_transform
Der Wert dieser Option ist ein Befehlspräfix oder ein Skript, mit dem eine Transformation des Ergebnisses eines Aufrufs durchgeführt wird (*vor* der Anwendung der Ausgabetransformation gemäß **format**). Eine Skript-Transformation wird in eine

automatisch generierte interne Prozedur eingebettet.

Wird nichts angegeben, erfolgt keine Transformation.

Der Befehl (Präfix) muss ein einzelnes Argument – das zu transformierende Ergebnis – akzeptieren und das geänderte Ergebnis als Rückgabewert geben.

Der HTTP-Request-Header ist im Transformationsbefehl über **upvar token token** zugänglich.

post_transform
Der Wert dieser Option ist ein Befehlspräfix oder ein Skript, mit dem eine Transformation des Ergebnisses eines Aufrufs durchgeführt wird (*nach* der Anwendung der Ausgabetransformation gemäß **format**). Eine Skript-Transformation wird in eine automatisch generierte interne Prozedur eingebettet.

Wenn nichts festgelegt wird, erfolgt keine Transformation.

Der Befehl (Präfix) muss ein einzelnes Argument – das zu transformierende Ergebnis – akzeptieren und das geänderte Ergebnis als Rückgabewert geben.

Der HTTP-Request-Header ist im Transformationsbefehl über **upvar token token** zugänglich.

check_result
Der Wert dieser Option muss eine Liste aus zwei Ausdrücken sein, von denen jeder leer sein kann.

Der erste Ausdruck prüft die OK-Bedingung; er gibt den Wert **true** zurück, wenn das Ergebnis zufriedenstellend ist, und andernfalls den Wert **false**.

Der zweite Ausdruck stellt die ERROR-Bedingung dar; er gibt den Wert **false** zurück, sofern kein Fehler vorliegt; liegt ein Fehler vor, gibt er den Wert **true** zurück.

error_body
Der Wert dieser Option legt fest, ob bei Auftreten eines HTTP-Fehlers die Antwort gegeben wird oder nicht. Standardmäßig wird der Antworttext bei einem Fehler nicht gegeben.

Weitere Informationen finden Sie unter **callback** weiter oben.

timeout
Der Wert dieser Option legt fest, ob für den HTTP-Aufruf ein Timeout festgelegt werden soll. Standardmäßig ist kein Timeout festgelegt.

Der Timeout-Wert wird in Millisekunden angegeben.

Beispiele

Yahoo Geo:

```
set ygeo(parse) {
    url http://wherein.yahooapis.com/v1/document
    method post
    body { arg documentContent }
}
ygeo::parse "san jose ca"
# "san jose ca" will be interpreted as if it were specified as the
    -documentContent option
```

Google Docs:

```
set gdocs(upload) {
    url http://docs.google.com/feeds/default/private/full
    body mime_multipart
}
gdocs::upload [list {Content-Type application/atom+xml} $xml] [list
    {Content-Type image/jpeg} $filedata]
```

Köstlich:

```
set delicious(updated) {
    url https://api.del.icio.us/v1/posts/update
    auth basic
}

rest::create_interface flickr

flickr::basic_auth username password
```

Flickr:

```
set flickr(auth.getToken) {
   url http://api.flickr.com/services/rest/
   req_args { api_key: secret: }
   auth { sign do_signature }
}

rest::create_interface flickr

proc ::flickr::do_signature {query} {
    # perform some operations on the query here
    return $query
}
```

Enthalten

Das Paket bietet funktionsfähige, jedoch unvollständige Implementierungen für die folgenden Dienste:

- **del.icio.us**

- **facebook**
- **flickr**
- **twitter**
- **google calendar**
- **yahoo boss**
- **yahoo weather**

Bitte lesen Sie entweder die Implementierung des Pakets oder verwenden Sie nach dem Laden des Pakets **rest::describe**, um weitere Informationen zu erhalten.

Beachten Sie bitte auch die Entwicklerdokumentation auf den jeweiligen Websites.

TLS

Das Paket "**rest**" kann mit *https*-secured-Diensten verwendet werden, indem das TLS-Paket angefordert und anschließend bei dem Paket "**http**", auf dem es aufbaut, registriert wird. Beispiele

```
    package require tls
    http::register https 443 ::tls::socket
```

TLS-Sicherheitshinweise

Dieses Paket nutzt das TLS-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von TLS noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die TLS verwenden, gepatcht werden – nicht dieses Paket und auch nicht TLS selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie in den folgenden Beispielen gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *rest* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

ripemd

ripemd128

Stichwörter: ripemd128, RIPEMD, hashing, md4, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name

ripemd128 – RIPEMD-128-Message-Digest-Algorithmus

Übersicht

package require Tcl 8.5 9
package require ripemd128 ?**1.0.6**?
::ripemd::ripemd128 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::ripemd::hmac128 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::ripemd::RIPEMD128Init
::ripemd::RIPEMD128Update *token data*
::ripemd::RIPEMD128Final *token*
::ripemd::RIPEHMAC128Init *key*
::ripemd::RIPEHMAC128Update *token data*
::ripemd::RIPEHMAC128Final *token*

Beschreibung

Dieses Paket ist eine Implementierung des RIPEMD-128-Message-Digest-Algorithmus (1) in Tcl. Dieser Algorithmus nimmt eine beliebige Datenmenge entgegen und generiert aus der Eingabe einen 128-Bit-Message-Digest. Der RIPEMD-128-Algorithmus basiert auf dem MD4-Algorithmus (2, 4), wurde jedoch kryptografisch gegen die in MD4 festgestellten Schwachstellen abgesichert (4). RIPEMD-128 wurde als direkter Ersatz für MD4 und MD5 konzipiert (5). Wenn Sicherheit im Vordergrund steht, sollten RIPEMD-160 oder SHA-1 in Betracht gezogen werden.

Dieses Paket nutzt **Trf** zur Beschleunigung der Digest-Berechnung, wenn dieses verfügbar ist. Ist kein Beschleunigerpaket vorhanden, wird die reine Tcl-Implementierung verwendet.

ACHTUNG Die Befehle in diesem Paket erwarten Binärdaten als Eingabe. Wird ein **-file** angegeben, wird dies durch die Befehle selbst sichergestellt, da sie die referenzierte Datei im Binärmodus öffnen. Wenn stattdessen Literal-Daten oder ein **-channel** übergeben werden, ist der *Aufrufer des Befehls* dafür verantwortlich, dies sicherzustellen. Der erforderliche Konvertierungsbefehl lautet **encoding convertto utf-8 $string**.

ACHTUNG, hier liegt ein *Tcl-9-Kompatibilitätsproblem* vor.

Tcl 8.x behandelt nicht-binäre Eingaben stillschweigend falsch, indem es diese intern auf die gewünschte Länge kürzt. Das heißt, indem nur das niedrige Byte eines höheren Unicode-Punkts verwendet wird. Es wird kein Fehler ausgelöst, das *Ergebnis ist einfach falsch*.

Tcl 9 löst stattdessen einen Fehler aus, d. h.

```
expected byte sequence but character <location> was ...
```

Befehle

::ripemd::ripemd128 ?*-hex*? [*-channel channel* | *-file filename* | *string*]

Berechnet den RIPEMD-128-Digest der in der Zeichenkette angegebenen Daten. Dieser wird standardmäßig als Binärzeichenkette zurückgegeben. Bei Angabe der Option *-hex* gibt es eine hexadezimal kodierte Version des Digests zurück.

Die zu hashenden Daten können entweder als Zeichenfolgenargument für den Befehl ripemd128, als Dateiname oder als zuvor geöffneter Kanal angegeben werden. Wenn das Argument *-filename* angegeben wird, wird die Datei geöffnet, die Daten werden gelesen und gehasht, und die Datei wird geschlossen. Wenn das Argument *-channel* angegeben wird, werden Daten aus dem Kanal bis zum Dateiende gelesen. Der Kanal wird nicht geschlossen.

Es sollte nur eines der Argumente *-file*, *-channel* oder *string* angegeben werden.

::ripemd::hmac128 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]

Berechnet einen HMAC-Digest (Hashed Message Authentication Digest) unter Verwendung des RIPEMD-128-Digest-Algorithmus. HMACs sind in RFC 2104 (6) beschrieben und liefern einen RIPEMD-128-Digest, der einen Schlüssel enthält. Alle Optionen außer *-key* entsprechen denen des Befehls **::ripemd::ripemd128**.

Programmierschnittstelle

Für den Programmierer lassen sich Hash-Funktionen als ein Behälter betrachten, in den man Daten einfüllt. Ist die Eingabe abgeschlossen, wird ein Wert extrahiert, der sich eindeutig aus den Daten ableitet, die in den Behälter eingefüllt wurden. Die Programmierschnittstelle zum Hash arbeitet mit einem Token (das dem Behälter entspricht). **RIPEMD128Init** liefert ein Token, **RIPEMD128Update** nimmt so oft wie erforderlich weitere Daten entgegen. **RIPEMD128Final** gibt die Ressourcen frei und liefert den Hash-Wert. Ein entsprechender Funktionssatz liefert Ihnen einen schlüsselbasierten Digest (HMAC).

Wenn Sie über **critcl** verfügen und das Paket **tcllibc** erstellt haben, erfolgt die Implementierung der Hash-Funktion durch kompilierten Code. Sind hingegen sowohl die Trf- als auch die Memchan-Erweiterung verfügbar, werden diese verwendet. Andernfalls greift das Paket auf eine reine Tcl-Implementierung zurück. Die Programmierschnittstelle bleibt jedoch unverändert.

::ripemd::RIPEMD128Init

Erzeugt einen neuen RIPEMD-128-Hash. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss.

::ripemd::RIPEMD128Update *token data*

Fügt Daten zu dem durch das Token identifizierten Hash hinzu. Der Aufruf von *RIPEMD128Update $token* "*abcd*" entspricht dem Aufruf von *RIPEMD128Update $token* "*ab*", gefolgt von *RIPEMD128Update $token* "*cb*". Siehe Beispiele.

::ripemd::RIPEMD128Final *token*

Gibt den Hash-Wert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, verliert das Token seine Gültigkeit. Das Ergebnis ist eine binäre Zeichenkette mit einer Länge von 16 Byte, die den 128-Bit-RIPEMD-128-Wert darstellt.

::ripemd::RIPEHMAC128Init *key*

Dies entspricht dem Befehl **::ripemd::RIPEMD128Init**, mit dem Unterschied, dass hierfür der Schlüssel angegeben werden muss, der in den HMAC einbezogen werden soll.

::ripemd::RIPEHMAC128Update *token data*
siehe unten ...

::ripemd::RIPEHMAC128Final *token*
Diese Befehle entsprechen den entsprechenden Befehlen des RIPEMD128.

Beispiele

```
% ripemd::ripemd128 -hex "Tcl does RIPEMD-128"
3cab177bae65205d81e7978f63556c63
```

```
% ripemd::hmac128 -hex -key Sekret "Tcl does RIPEMD-128"
b359dc5971a05beea0be7b106b30e389
```

```
% set tok [ripemd::RIPEMD128Init]
::ripemd::1
% ripemd::RIPEMD128Update $tok "Tcl "
% ripemd::RIPEMD128Update $tok "does "
% ripemd::RIPEMD128Update $tok "RIPEMD-128"
% ripemd::Hex [ripemd::RIPEMD128Final $tok]
3cab177bae65205d81e7978f63556c63
```

Literatur

1. H. Dobbertin, A. Bosselaers, B. Preneel, "RIPEMD-160, eine erweiterte Version von RIPEMD" http://www.esat.kuleuven.ac.be/~cosicart/pdf/AB-9601/AB-9601.pdf
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Rivest, R., "The MD4 message digest algorithm", in: A.J. Menezes und S.A. Vanstone (Hrsg.), Advances in Cryptology – CRYPTO '90 Proceedings, Seiten 303–311, Springer-Verlag, 1991.
4. Dobbertin, H., "Kryptoanalyse von MD4", Journal of Cryptology, Band 11 (4), S. 253–271 (1998)
5. Rivest, R., "The MD5 Message-Digest Algorithm", RFC 1321, MIT und RSA Data Security, Inc., April 1992. (http://www.rfc-editor.org/rfc/rfc1321.txt)
6. Krawczyk, H., Bellare, M. und Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, Februar 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ripemd* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright


```
Copyright (c) 2004, Pat Thoyts <patthoyts@users.sourceforge.net>
```


ripemd160

Stichwörter: ripemd160, RIPEMD, hashing, md4, message-digest, rfc 1320, rfc 1321, rfc 2104, security

Name

ripemd160 – RIPEMD-160-Message-Digest-Algorithmus

Übersicht

package require Tcl 8.5 9
package require ripemd160 ?**1.0.7**?
::ripemd::ripemd160 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
::ripemd::hmac160 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
::ripemd::RIPEMD160Init
::ripemd::RIPEMD160Update *token data*
::ripemd::RIPEMD160Final *token*
::ripemd::RIPEHMAC160Init *key*
::ripemd::RIPEHMAC160Update *token data*
::ripemd::RIPEHMAC160Final *token*

Beschreibung

Dieses Paket ist eine Implementierung des RIPEMD-160-Message-Digest-Algorithmus (1) in Tcl. Dieser Algorithmus nimmt eine beliebige Datenmenge entgegen und generiert aus der Eingabe einen 160-Bit-Message-Digest. Der RIPEMD-160-Algorithmus basiert auf dem MD4-Algorithmus (2, 4), wurde jedoch kryptografisch gegen die in MD4 festgestellten Schwachstellen (4) abgesichert.

Dieses Paket nutzt **cryptkit** oder **Trf** zur Beschleunigung der Digest-Berechnung, wenn eines dieser Pakete verfügbar ist. Wenn kein Beschleunigerpaket vorhanden ist, wird die reine Tcl-Implementierung verwendet.

ACHTUNG Die Befehle in diesem Paket erwarten Binärdaten als Eingabe. Wird ein **-file** angegeben, wird dies durch die Befehle selbst sichergestellt, da sie die referenzierte Datei im Binärmodus öffnen. Wenn stattdessen Literal-Daten oder ein **-channel** übergeben werden, ist der *Aufrufer des Befehls* dafür verantwortlich, dies sicherzustellen. Der erforderliche Konvertierungsbefehl lautet **encoding convertto utf-8 $string**.

ACHTUNG, hier liegt ein *Tcl-9-Kompatibilitätsproblem* vor.

Tcl 8.x behandelt nicht-binäre Eingaben stillschweigend falsch, indem es diese intern auf die gewünschte Länge kürzt. Das heißt, indem nur das niedrige Byte eines höheren Unicode-Punkts verwendet wird. Es wird kein Fehler ausgelöst, das *Ergebnis ist einfach falsch*.

Tcl 9 löst stattdessen einen Fehler aus, d. h.

```
expected byte sequence but character <location> was ...
```

Befehle

::ripemd::ripemd160 ?*-hex*? [*-channel channel* | *-file filename* | *string*]
Berechnet den RIPEMD-160-Digest der in der Zeichenkette angegebenen Daten. Dieser wird standardmäßig als binäre Zeichenkette zurückgegeben. Bei Angabe der Option *-hex* gibt es eine hexadezimal kodierte Version des Digests zurück.

Die zu hashenden Daten können entweder als Zeichenfolgenargument des Befehls ripemd160, als Dateiname oder als zuvor geöffneter Kanal angegeben werden. Wenn das Argument *-filename* angegeben wird, wird die Datei geöffnet, die Daten werden gelesen und gehasht, und die Datei wird geschlossen. Wenn das Argument *-channel* angegeben wird, werden Daten aus dem Kanal bis zum Dateiende gelesen. Der Kanal wird nicht geschlossen.

Es sollte nur eines der Argumente *-file*, *-channel* oder *string* angegeben werden.

::ripemd::hmac160 ?*-hex*? *-key key* [*-channel channel* | *-file filename* | *string*]
Berechnet einen HMAC-Digest (Hashed Message Authentication Digest) unter Verwendung des RIPEMD-160-Digest-Algorithmus. HMACs sind in RFC 2104 (5) beschrieben und liefern einen RIPEMD-160-Digest, der einen Schlüssel enthält. Alle Optionen außer *-key* entsprechen denen des Befehls **::ripemd::ripemd160**.

Programmierschnittstelle

Für den Programmierer lassen sich Hash-Funktionen als ein Behälter betrachten, in den man Daten einfüllt. Ist die Eingabe abgeschlossen, wird ein Wert extrahiert, der sich eindeutig aus den Daten ableitet, die in den Behälter eingefüllt wurden. Die Programmierschnittstelle zum Hash arbeitet mit einem Token (das dem Behälter entspricht). **RIPEMD160Init** liefert ein Token, **RIPEMD160Update** nimmt so oft wie erforderlich weitere Daten entgegen. **RIPEMD160Final** gibt die Ressourcen frei und liefert den Hash-Wert. Ein entsprechender Funktionssatz liefert Ihnen einen schlüsselbasierten Digest (HMAC).

::ripemd::RIPEMD160Init
Startet einen neuen RIPEMD-160-Hash. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss.

::ripemd::RIPEMD160Update *token data*
Fügt Daten zu dem durch das Token identifizierten Hash hinzu. Der Aufruf von *RIPEMD160Update $token* "*abcd*" entspricht dem Aufruf von *RIPEMD160Update $token* "*ab*", gefolgt von *RIPEMD160Update $token* "*cb*". Siehe Beispiele.

::ripemd::RIPEMD160Final *token*
Gibt den Hash-Wert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, verliert das Token seine Gültigkeit. Das Ergebnis ist eine binäre Zeichenkette mit einer Länge von 16 Byte, die den 160-Bit-RIPEMD-160-Wert darstellt.

::ripemd::RIPEHMAC160Init *key*
Dies entspricht dem Befehl **::ripemd::RIPEMD160Init**, mit dem Unterschied, dass hierfür der Schlüssel angegeben werden muss, der in den HMAC einbezogen werden soll.

::ripemd::RIPEHMAC160Update *token data*
siehe unten ...

::ripemd::RIPEHMAC160Final *token*
Diese Befehle entsprechen den entsprechenden Befehlen des RIPEMD160.

Beispiele

```
% ripemd::ripemd160 -hex "Tcl does RIPEMD-160"
0829dea75a1a7074c702896723fe37763481a0a7
```

```
% ripemd::hmac160 -hex -key Sekret "Tcl does RIPEMD-160"
bf0c927231733686731dddb470b64a9c23f7f53b
```

```
% set tok [ripemd::RIPEMD160Init]
::ripemd::1
% ripemd::RIPEMD160Update $tok "Tcl "
% ripemd::RIPEMD160Update $tok "does "
% ripemd::RIPEMD160Update $tok "RIPEMD-160"
% ripemd::Hex [ripemd::RIPEMD160Final $tok]
0829dea75a1a7074c702896723fe37763481a0a7
```

Literatur

1. H. Dobbertin, A. Bosselaers, B. Preneel, "RIPEMD-160, eine erweiterte Version von RIPEMD" http://www.esat.kuleuven.ac.be/~cosicart/pdf/AB-9601/AB-9601.pdf
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Rivest, R., "The MD4 message digest algorithm", in: A.J. Menezes und S.A. Vanstone (Hrsg.), Advances in Cryptology – CRYPTO '90 Proceedings, Seiten 303–311, Springer-Verlag, 1991.
4. Dobbertin, H., "Kryptoanalyse von MD4", Journal of Cryptology, Band 11 (4), S. 253–271 (1998)
5. Krawczyk, H., Bellare, M. und Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, Februar 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *ripemd* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2004, Pat Thoyts <patthoyts@users.sourceforge.net>
```

sasl

gtoken

Stichwörter: gtoken, SASL, XGoogleToken, authentication

Name
SASL::XGoogleToken – Implementierung des SASL-NTLM-Mechanismus für Tcl

Übersicht
package require Tcl 8.5 9
package require SASL::XGoogleToken ?**1.0.2**?

Beschreibung
Dieses Paket stellt den Authentifizierungsmechanismus "XGoogleToken" für die Simple Authentication and Security Layer (SASL) bereit.

Bitte lesen Sie die Dokumentation zum Paket **sasl**, um weitere Einzelheiten zu erfahren.

TLS-Sicherheitshinweise
Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *sasl* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright
```
Copyright (c) 2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```

ntlm
Stichwörter: ntlm, SASL, authentication

Name
SASL::NTLM – Implementierung des SASL-NTLM-Mechanismus für Tcl

Übersicht
package require Tcl 8.5 9
package require SASL::NTLM ?**1.1.4**?

Beschreibung
Dieses Paket stellt den NTLM-Authentifizierungsmechanismus für das Simple Authentication and Security Layer (SASL) bereit.

Bitte lesen Sie die Dokumentation zum Paket **sasl**, um weitere Einzelheiten zu erfahren.

Literatur
1. Es liegt keine offizielle Spezifikation vor. Eine gute Beschreibung finden Sie jedoch unter http://davenport.sourceforge.net/ntlm.html.

Autoren
Pat Thoyts

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *sasl* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright
```
Copyright (c) 2005-2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```

sasl
Stichwörter: sasl, authentication

Name

SASL – Implementierung von SASL-Mechanismen für Tcl

Übersicht

package require Tcl 8.5 9
package require SASL ?**1.3.4**?
::SASL::new *option value* ?*...*?
::SASL::configure *option value* ?*...*?
::SASL::step *context challenge* ?*...*?
::SASL::response *context*
::SASL::reset *context*
::SASL::cleanup *context*
::SASL::mechanisms ?*type*? ?*minimum*?
::SASL::register *mechanism preference clientproc* ?*serverproc*?

Beschreibung

Das Simple Authentication and Security Layer (SASL) ist ein Rahmen zur Bereitstellung von Authentifizierung und Autorisierung für Kommunikationsprotokolle. Der SASL-Rahmen ist so aufgebaut, dass er die Aushandlung zwischen verschiedenen Authentifizierungsmechanismen ermöglicht. SASL kann bei der SMTP-, IMAP- und HTTP-Authentifizierung eingesetzt werden. Es wird zudem in XMPP, LDAP und BEEP verwendet. Unter Mechanismen finden Sie eine Übersicht über die mit tcllib bereitgestellten SASL-Mechanismen.

Der SASL-Rahmen arbeitet mit einem einfachen, mehrstufigen Challenge-Response-Mechanismus. Alle Mechanismen funktionieren auf dieselbe Weise, auch wenn die Anzahl der Schritte variieren kann. In dieser Implementierung muss eine Rückruffunktion bereitgestellt werden, über die der SASL-Rahmen die Benutzerdaten abruft. Einzelheiten zu dieser Prozedur finden Sie unter Callback-Prozedur.

Befehle

::SASL::new *option value ?...?*
: Erstellt einen neuen SASL-Kontext. Einzelheiten zu den möglichen Optionen dieses Befehls finden Sie unter Optionen. Für die meisten SASL-Prozeduren ist ein Kontext-Token erforderlich.

::SASL::configure *option value ?...?*
: Liest oder setzt eine Option des SASL-Kontexts. Weitere Einzelheiten finden Sie unter Optionen.

::SASL::step *context challenge ?...?*
: Dies ist die Kern-Prozedur für die Verwendung des SASL-Rahmens. Die Prozedur **step** sollte so lange aufgerufen werden, bis sie den Wert 0 gibt. Jeder Schritt nimmt eine Server-Challenge-Zeichenkette entgegen, die Antwort wird berechnet und im Kontext gespeichert. Jeder Mechanismus kann einen oder mehrere Schritte erfordern. Bei einigen Schritten ist möglicherweise keine Server-Challenge erforderlich; in diesem Fall sollte für diesen Parameter eine leere Zeichenfolge angegeben werden. Alle Mechanismen sollten eine anfänglich leere Challenge akzeptieren.

::SASL::response *context*
: Gibt die nächste Antwortzeichenfolge zurück, die an den Server gesendet werden soll.

::SASL::reset *context*

Initialisiert den SASL-Kontext neu. Dabei werden alle internen Zustände verworfen, und das Token kann wiederverwendet werden.

::SASL::cleanup *context*
Gibt alle mit dem SASL-Kontext verbundenen Ressourcen frei. Das Kontext-Token darf nach dem Aufruf dieser Prozedur nicht mehr verwendet werden.

::SASL::mechanisms ?*type*? ?*minimum*?
Gibt eine Liste aller verfügbaren SASL-Mechanismen zurück. Die Liste ist nach dem Wert der Präferenz des Mechanismus sortiert (siehe **register**), wobei die bevorzugten Mechanismen am Anfang der Liste stehen. Jeder Mechanismus mit einem Wert für die Präferenz unterhalb des *Mindestwerts* (der standardmäßig auf 0 gesetzt ist) wird aus der zurückgegebenen Liste entfernt. Dies ermöglicht die Festlegung einer Schwelle für die Sicherheit. Mechanismen mit einer Präferenz unter 25, die eine Authentifizierung übertragen, sind besonders anfällig für Abhörversuche und sollten nur bereitgestellt werden, wenn ein sicherer Kanal verwendet wird (z. B. TLS).

Der Parameter *type* kann entweder *client* oder *server* lauten und ist standardmäßig auf *client* gesetzt. Es werden nur Mechanismen zurückgegeben, deren Implementierung mit dem *type* übereinstimmt (dadurch können Server korrekt nur die Unterstützung für Mechanismen deklarieren, für die tatsächlich eine Server-Implementierung vorliegt).

::SASL::register *mechanism preference clientproc ?serverproc?*
Neue Mechanismen können dem Paket hinzugefügt werden, indem der Mechanismenname und die entsprechenden Implementierungsprozeduren registriert werden. Die Serverprozedur ist optional. Der Präferenzwert ist eine ganze Zahl, die zur Sortierung der vom Befehl **mechanisms** gegebenen Liste verwendet wird. Höhere Werte kennzeichnen einen bevorzugten Mechanismus. Wenn der Mechanismus bereits registriert ist, werden die gespeicherten Werte aktualisiert.

Optionen

-callback
Legt einen Befehl fest, der ausgewertet werden soll, wenn der SASL-Mechanismus Informationen über den Benutzer benötigt. Der Befehl wird mit dem aktuellen SASL-Kontext und einem Namen aufgerufen, der die gewünschten Informationen angibt. Siehe Beispiele.

-mechanism
Legt den zu verwendenden SASL-Mechanismus fest. Eine Liste der unterstützten Authentifizierungsmechanismen finden Sie unter **mechanisms**.

-service
Legt den Diensttyp für diesen Kontext fest. Einige Mechanismen nutzen diesen Parameter möglicherweise (z. B. DIGEST-MD5, GSSAPI und Kerberos). Wenn dieser Parameter nicht festgelegt wird, wird standardmäßig eine leere Zeichenkette verwendet. Wenn der Parameter **-type** auf "server" gesetzt ist, sollte diese Option auf eine gültige Dienstidentität gesetzt werden. Beispiele für gültige Dienstnamen sind smtp, ldap, beep und xmpp.

-server
Diese Option dient dazu, den Namen des Servers festzulegen, der bei SASL-Anfragen verwendet wird, wenn das System als SASL-Server fungiert.

-type

Der Kontexttyp kann entweder "client" oder "server" lauten. Standardmäßig wird die Anwendung als Client ausgeführt und reagiert auf Herausforderungen des Servers. Es können Mechanismen zur Unterstützung von serverseitigem SASL programmiert werden; durch Setzen dieser Option wird bei jedem **step** die nächste Herausforderung ausgegeben. Im Servermodus muss für jede eingehende Client-Verbindung ein neuer Kontext erstellt werden.

Callback-Prozedur

Wenn das SASL-Framework Benutzerdaten benötigt, ruft es die bei der Erstellung des Kontexts bereitgestellte Prozedur mit einem Argument auf, das die jeweils benötigte Information angibt.

In allen Fällen sollte eine einzige Antwortzeichenfolge gegeben werden.

login
Die Callback-Prozedur sollte die Autorisierungsidentität des Benutzers geben. Geben Sie eine leere Zeichenkette zurück, sofern diese nicht von der Authentifizierungsidentität abweicht. Lesen Sie [1], um mehr über die spezifische Bedeutung von Autorisierungs- und Authentifizierungsidentitäten innerhalb von SASL zu erfahren.

username
Die Callback-Prozedur sollte die Authentifizierungsidentität des Benutzers geben. Eine Erläuterung der genauen Bedeutung von Autorisierungs- und Authentifizierungsidentitäten im Rahmen von SASL finden Sie unter [1].

password
Die Callback-Prozedur sollte das Passwort geben, das mit der Authentifizierungsidentität übereinstimmt, wie sie innerhalb des aktuellen Realms verwendet wird.

Bei Servermechanismen sollte der Passwort-Callback stets mit der Authentifizierungsidentität und dem Realm als ersten beiden Parametern aufgerufen werden.

realm
Einige SASL-Mechanismen verwenden Realms zur Unterteilung von Authentifizierungsidentitäten. Die Realm-Zeichenkette ist protokollabhängig und entspricht häufig der aktuellen DNS-Domäne oder – im Falle des NTLM-Mechanismus – dem Windows NT-Domänennamen.

hostname
Gibt den Hostnamen des Clients zurück – in der Regel [info host].

Mechanismen

ANONYMOUS
Bei der Verwendung im Rahmen von FTP übermittelt dieser Mechanismus lediglich eine E-Mail-Adresse zur Authentifizierung. Der ANONYMOUS-Mechanismus wird in [2] festgelegt.

PLAIN
Dies ist der einfachste Mechanismus. Die Anmeldedaten der Benutzer werden im Klartext übertragen. Dieser Mechanismus sollte nur dann bereitgestellt werden, wenn eine verschlüsselte Verbindung besteht – in der Regel nachdem SSL oder TLS ausgehandelt wurde.

LOGIN
Der LOGIN-Mechanismus [1] übermittelt die Benutzerdaten in Base64-Kodierung. Dies ist nicht sicherer als PLAIN und sollte daher ebenfalls nicht ohne eine sichere Verbindung verwendet werden.

CRAM-MD5
Dieser Mechanismus verhindert, dass das Passwort des Benutzers im Klartext über das Netzwerk übertragen wird, indem das Passwort mit einem vom Server bereitgestellten Zufallswert (einem sogenannten "Nonce") gehasht wird. Ein Nachteil dieses Mechanismus besteht darin, dass der Server eine Datenbank mit Passwörtern im Klartext zum Vergleich führen muss. CRAM-MD5 wurde in [4] definiert.

DIGEST-MD5
Dieser Mechanismus stellt eine Verbesserung gegenüber dem CRAM-MD5-Mechanismus dar, da der Server keine Passwörter im Klartext speichern muss. Bei der Digest-Authentifizierung muss der Server den MD5-Digest des Benutzerpassworts speichern, was zur Erhöhung der Sicherheit des Systems beiträgt. Wie bei CRAM-MD5 wird das Passwort zusammen mit einem Server-Nonce und weiteren Daten gehasht, bevor es über das Netzwerk übertragen wird. Spezifiziert in [3].

OTP
OTP ist das in RFC 2289 [6] beschriebene Einmalpasswort-System. Dieser Mechanismus bietet Sicherheit gegen Replay-Angriffe und verhindert zudem, dass Passwörter oder Passwortäquivalente auf dem Server gespeichert werden. Über das Netzwerk wird lediglich ein Digest aus einem Seed und einer Passphrase übertragen. Erforderlich sind das Paket **otp** aus tcllib sowie eines oder mehrere der kryptografischen Digest-Pakete (md5 oder sha-1 werden am häufigsten verwendet).

NTLM
Es handelt sich hierbei um ein von Microsoft entwickeltes proprietäres Protokoll [5], das häufig zur Authentifizierung von Benutzern in einer Windows-Netzwerkumgebung verwendet wird. NTLM nutzt DES-Verschlüsselung und MD4-Hashwerte des Benutzerpassworts, um eine Verbindung zu authentifizieren. Da bei NTLM gewisse Schwachstellen festgestellt wurden, gibt es eine Reihe von Versionen des Protokolls. Da dieser Mechanismus zusätzliche Abhängigkeiten aufweist, wird er als separates Unterpaket bereitgestellt. Um diesen Mechanismus zu aktivieren, muss Ihre Anwendung das Paket SASL::NTLM laden.

X-GOOGLE-TOKEN
Hierbei handelt es sich um ein von Google entwickeltes proprietäres Protokoll, das zur Authentifizierung von Nutzern für den Google Talk-Dienst verwendet wird. Dieser Mechanismus führt zwei HTTP-Anfragen über einen SSL-Kanal durch und ist daher von der Verfügbarkeit der Pakete "tls" und "http" abhängig. Um diesen Mechanismus zu aktivieren, muss Ihre Anwendung das Paket "SASL::XGoogleToken" laden. Darüber hinaus wird empfohlen, das "autoproxy"-Paket zu verwenden, um HTTP-Proxys weitgehend mit hoher Transparenz zu handhaben.

SCRAM
Hierbei handelt es sich um ein in RFC 5802 [7] festgelegtes Protokoll. Um diesen Mechanismus zu aktivieren, muss Ihre Anwendung das Paket SASL::SCRAM laden.

Beispiele

Im Unterverzeichnis "examples" finden Sie ausführlichere Beispiele für die Verwendung von SASL mit Netzwerkprotokollen. Die folgenden Beispiele sollen Ihnen einen Eindruck davon vermitteln, wie die SASL-Befehle verwendet werden. In der Praxis sollte dies ereignisgesteuert erfolgen. Bei jedem Aufruf des Befehls **step** sollte die letzte Serverantwort als Argument des Befehls übergeben werden, damit der SASL-Mechanismus entsprechende Maßnahmen ergreifen

kann.

```
proc ClientCallback {context command args} {
    switch -exact -- $command {
        login    { return "" }
        username { return $::tcl_platform(user) }
        password { return "SecRet" }
        realm    { return "" }
        hostname { return [info host] }
        default  { return -code error unxpected }
    }
}

proc Demo {{mech PLAIN}} {
    set ctx [SASL::new -mechanism $mech -callback ClientCallback]
    set challenge ""
    while {1} {
        set more_steps [SASL::step $ctx challenge]
        puts "Send '[SASL::response $ctx]'"
        puts "Read server response into challenge var"
        if {!$more_steps} {break}
    }
    SASL::cleanup $ctx
}
```

Literatur

1. Myers, J. "Simple Authentication and Security Layer (SASL)", RFC 2222, Oktober 1997. (http://www.ietf.org/rfc/rfc2222.txt)
2. Newman, C. "Anonymous SASL Mechanism", RFC 2245, November 1997. (http://www.ietf.org/rfc/rfc2245.txt)
3. Leach, P., Newman, C. "Using Digest Authentication as a SASL Mechanism", RFC 2831, Mai 2000, (http://www.ietf.org/rfc/rfc2831.txt)
4. Klensin, J., Catoe, R. und Krumviede, P., "IMAP/POP AUTHorize-Erweiterung für Simple Challenge/Response" RFC 2195, September 1997. (http://www.ietf.org/rfc/rfc2195.txt)
5. Es liegt keine offizielle Spezifikation vor. Eine gute Beschreibung finden Sie jedoch unter http://davenport.sourceforge.net/ntlm.html.
6. Haller, N. et al., "A One-Time Password System", RFC 2289, Februar 1998, (http://www.ieft.org/rfc/rfc2289.txt)
7. Newman, C. et al., "Salted Challenge Response Authentication Mechanism (SCRAM) – SASL- und GSS-API-Mechanismen", RFC 5802, Juli 2010, (http://www.ieft.org/rfc/rfc5802.txt)

Autoren

Pat Thoyts

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *sasl* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright
```
Copyright (c) 2005-2006, Pat Thoyts <patthoyts@users.sourceforge.net>
```

scram
Stichwörter: scram, SASL, authentication

Name
SASL::SCRAM – Implementierung des SASL-SCRAM-Mechanismus für Tcl

Übersicht
package require Tcl 8.5 9
package require SASL::SCRAM ?**0.2**?

Beschreibung
Dieses Paket stellt den SCRAM-Authentifizierungsmechanismus für die Simple Authentication and Security Layer (SASL) bereit.

Bitte lesen Sie die Dokumentation zum Paket **sasl**, um weitere Einzelheiten zu erfahren.

Literatur
1. Newman, C. et al., "Salted Challenge Response Authentication Mechanism (SCRAM) – SASL- und GSS-API-Mechanismen", RFC 5802, Juli 2010, (http://www.ieft.org/rfc/rfc5802.txt)

Autoren
Sergei Golovan

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *sasl* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright

Copyright (c) 2013 Sergei Golovan <sgolovan@nes.ru>

sha1

sha1

Stichwörter: sha1, FIPS 180-1, hashing, message-digest, rfc 2104, security

Name

sha1 – SHA1-Message-Digest-Algorithmus

Übersicht

package require Tcl 8.5 9
package require sha1 ?**2.0.5**?
::sha1::sha1 ?**-hex|-bin**? [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha1::hmac *key string*
::sha1::hmac ?**-hex|-bin**? **-key key** [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha1::SHA1Init
::sha1::SHA1Update *token data*
::sha1::SHA1Final *token*
::sha1::HMACInit *key*
::sha1::HMACUpdate *token data*
::sha1::HMACFinal *token*

Beschreibung

Dieses Paket enthält eine Implementierung des SHA1-Message-Digest-Algorithmus in Tcl gemäß der Spezifikation in FIPS PUB 180-1 (1). Dieser Algorithmus nimmt eine Nachricht entgegen und generiert aus der Eingabe einen 160-Bit-Digest. Der SHA1-Algorithmus ist mit dem MD4-Algorithmus (2) verwandt, wurde jedoch hinsichtlich der Widerstandsfähigkeit gegen bestimmte Arten von kryptografischen Angriffen verbessert. In neuen Anwendungen sollte SHA1 gegenüber MD4 oder MD5 bevorzugt werden.

Dieses Paket enthält außerdem Unterstützung für die Erstellung verschlüsselter Nachrichten-Digests unter Verwendung des HMAC-Algorithmus aus RFC 2104 (3) mit SHA-1 als Nachrichten-Digest.

Achtung Die Befehle in diesem Paket erwarten Binärdaten als Eingabe. Wenn ein **-file** angegeben wird, wird dies von den Befehlen selbst sichergestellt, da sie die referenzierte Datei im Binärmodus öffnen. Wenn stattdessen Literal-Daten oder ein **-channel** übergeben werden, ist der *Aufrufer des Befehls* dafür verantwortlich, dies sicherzustellen. Die erforderliche Kodierung des Konvertierungsbefehls lautet **encoding convertto utf-8 $string**.

Achtung, hier liegt ein *Tcl-9-Kompatibilitätsproblem* vor.

Tcl 8.x behandelt nicht-binäre Eingaben stillschweigend falsch, indem es diese intern auf die gewünschte Länge kürzt. Das heißt, indem nur das niedrige Byte eines höheren Unicode-Punkts verwendet wird. Es wird kein Fehler ausgelöst, das *Ergebnis ist einfach falsch*.

Tcl 9 löst stattdessen einen Fehler aus, d. h.

```
expected byte sequence but character <location> was ...
```

Befehle

::sha1::sha1 ?-hex|-bin? [-channel channel | -file filename | ?--? *string*]
Der Befehl nimmt eine Nachricht entgegen und gibt den SHA1-Hash dieser Nachricht als hexadezimale Zeichenkette zurück. Sie können das Ergebnis als Binärdaten anfordern, indem Sie *-bin* angeben.

Die zu hashenden Daten können entweder als Zeichenfolgenargument für den Befehl **sha1**, als Dateiname oder als zuvor geöffneter Kanal angegeben werden. Wenn das Argument *-filename* angegeben wird, wird die Datei geöffnet, die Daten werden gelesen und gehasht und die Datei wird geschlossen. Wenn das Argument *-channel* angegeben wird, werden Daten aus dem Kanal bis zum Dateiende gelesen. Der Kanal wird nicht geschlossen. *Hinweis*: Die Verwendung der Kanal- oder Dateinamen-Optionen führt zur internen Nutzung von **vwait**. Um verschachtelte Ereignisschleifen in Tk- oder tclhttpd-Anwendungen zu vermeiden, sollten Sie die inkrementelle Programmier-API verwenden (siehe unten).

Es sollte nur eines der Argumente *-file*, *-channel* oder *string* angegeben werden.

Wenn der zu hashenden *string* mit einer Option verwechselt werden könnte (führender Bindestrich "-"), verwenden Sie davor die Option **--**, um den Options-Prozess zu beenden und die Interpretation als Zeichenkette zu erzwingen.

::sha1::hmac *key string*
siehe unten ...

::sha1::hmac ?-hex|-bin? -key key [-channel channel | -file filename | ?--? *string*]
Berechnet einen HMAC-Digest (Hashed Message Authentication Digest) unter Verwendung des SHA-1-Digest-Algorithmus. HMACs werden in RFC 2104 (3) beschrieben und liefern einen SHA-1-Digest, der einen Schlüssel enthält. Alle Optionen außer *-key* entsprechen denen des Befehls **::sha1::sha1**.

Wenn die zu hashende *Zeichenkette* mit einer Option verwechselt werden könnte (vorangestellter Bindestrich "-"), verwenden Sie davor die Option **--**, um den Options-Prozess zu beenden und die Interpretation als Zeichenkette zu erzwingen.

Programmierschnittstelle

Für den Programmierer lässt sich der SHA1-Hash als eine Art Behälter betrachten, in den Sie Daten einfüllen. Wenn Sie fertig sind, extrahieren Sie einen Wert, der aus den in den Behälter eingefüllten Daten abgeleitet wird. Die Programmierschnittstelle zum SHA1-Hash arbeitet mit einem Token (das dem Behälter entspricht). Sie rufen **SHA1Init** auf, um ein Token zu erhalten, und ruft anschließend **SHA1Update** so oft wie nötig auf, um Daten zum Hash hinzuzufügen. Um Ressourcen freizugeben und den Wert des Hash-Algorithmuses zu erhalten, rufen Sie dann **SHA1Final** auf. Ein entsprechender Funktionssatz liefert Ihnen einen schlüsselbasierten Digest (HMAC).

Wenn Sie **critcl** installiert und das **tcllibc**-Paket kompiliert haben, erfolgt die Implementierung der Hash-Funktion durch kompilierten Code. Falls dies nicht der Fall ist, können Sie das **Trf**-Paket

verwenden; andernfalls steht ein reines Tcl-Äquivalent zur Verfügung. Die Programmierschnittstelle bleibt in allen Fällen unverändert.

::sha1::SHA1Init
Erzeugt einen neuen SHA1-Hash. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss.

::sha1::SHA1Update *token data*
Fügt Daten zu dem durch das Token identifizierten Hash hinzu. Der Aufruf von *SHA1Update $token* "*abcd*" entspricht dem Aufruf von *SHA1Update $token* "*ab*", gefolgt von *SHA1Update $token* "*cb*". Siehe Beispiele.

::sha1::SHA1Final *token*
Gibt den Hash-Wert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, ist das Token ungültig. Das Ergebnis ist eine binäre Zeichenkette mit einer Länge von 20 Byte, die den 160-Bit-SHA1-Hash-Wert darstellt.

::sha1::HMACInit *key*
Dies entspricht dem Befehl **::sha1::SHA1Init**, mit dem Unterschied, dass hierfür der Schlüssel angegeben werden muss, der in den HMAC einbezogen werden soll.

::sha1::HMACUpdate *token data*
siehe unten ...

::sha1::HMACFinal *token*
Diese Befehle entsprechen den entsprechenden SHA1-Befehlen.

Beispiele

```
% sha1::sha1 "Tcl does SHA1"
285a6a91c45a9066bf39fcf24425796ef0b2a8bf
```

```
% sha1::hmac Sekret "Tcl does SHA1"
ae6251fa51b95b18cba2be95eb031d07475ff03c
```

```
% set tok [sha1::SHA1Init]
::sha1::1
% sha1::SHA1Update $tok "Tcl "
% sha1::SHA1Update $tok "does "
% sha1::SHA1Update $tok "SHA1"
% sha1::Hex [sha1::SHA1Final $tok]
285a6a91c45a9066bf39fcf24425796ef0b2a8bf
```

Literatur

1. "Secure Hash Standard", National Institute of Standards and Technology, US-Handelsministerium, April 1995. (http://www.itl.nist.gov/fipspubs/fip180-1.htm)
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Krawczyk, H., Bellare, M. und Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, Februar 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *sha1* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2005, Pat Thoyts <patthoyts@users.sourceforge.net>
```


sha256
Stichwörter: sha256, FIPS 180-1, hashing, message-digest, rfc 2104, security

Name
sha256 – SHA-256-Message-Digest-Algorithmus

Übersicht
package require Tcl 8.5 9
package require sha256 ?**1.0.6**?
::sha2::sha256 ?**-hex**|**-bin**? [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha2::sha224 ?**-hex**|**-bin**? [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha2::hmac *key string*
::sha2::hmac ?**-hex**|**-bin**? **-key key** [**-channel channel** | **-file filename** | ?**--**? *string*]
::sha2::SHA256Init
::sha2::SHA224Init
::sha2::SHA256Update *token data*
::sha2::SHA256Final *token*
::sha2::SHA224Final *token*
::sha2::HMACInit *key*
::sha2::HMACUpdate *token data*
::sha2::HMACFinal *token*

Beschreibung
Dieses Paket enthält eine Implementierung der Message-Digest-Algorithmen SHA256 und SHA224 in Tcl gemäß der Spezifikation in FIPS PUB 180-1 (1). Diese Algorithmen nehmen eine Nachricht entgegen und erzeugen aus der Eingabe einen 256-Bit- (bzw. 224-Bit-)Digest. Die SHA2-Algorithmen sind mit dem SHA1-Algorithmus verwandt.

Dieses Paket bietet außerdem Unterstützung für die Erstellung verschlüsselter Nachrichten-Digests unter Verwendung des HMAC-Algorithmus aus RFC 2104 (3) mit SHA256 als Nachrichten-Digest.

Achtung Die Befehle in diesem Paket erwarten Binärdaten als Eingabe. Wenn ein **-file**

angegeben wird, wird dies von den Befehlen selbst sichergestellt, da sie die referenzierte Datei im Binärmodus öffnen. Wenn stattdessen Literal-Daten oder ein **-channel** übergeben werden, ist der *Aufrufer des Befehls* dafür verantwortlich, dies sicherzustellen. Die erforderliche Kodierung des Konvertierungsbefehls lautet **encoding convertto utf-8 $string**.

Achtung, hier liegt ein *Tcl-9-Kompatibilitätsproblem* vor.

Tcl 8.x behandelt nicht-binäre Eingaben stillschweigend falsch, indem es diese intern auf die gewünschte Länge kürzt. Das heißt, indem nur das niedrige Byte eines höheren Unicode-Punkts verwendet wird. Es wird kein Fehler ausgelöst, das *Ergebnis ist einfach falsch*.

Tcl 9 löst stattdessen einen Fehler aus, d. h.

```
expected byte sequence but character <location> was ...
```

Befehle

::sha2::sha256 ?-hex|-bin? [-channel channel | -file filename | ?--? *string*]
Der Befehl nimmt eine Nachricht entgegen und gibt den SHA256-Digest dieser Nachricht als Hexadezimalzeichenfolge zurück. Sie können das Ergebnis als Binärdaten anfordern, indem Sie *-bin* angeben.

Die zu hashenden Daten können entweder als Zeichenfolgenargument für den Befehl **sha256** , als Dateiname oder als bereits geöffneter Kanal angegeben werden. Wenn das Argument *-filename* angegeben wird, wird die Datei geöffnet, die Daten werden gelesen und gehasht und die Datei wird geschlossen. Wenn das Argument *-channel* angegeben wird, werden Daten aus dem Kanal bis zum Dateiende gelesen. Der Kanal wird nicht geschlossen. *Hinweis*: Die Verwendung der Kanal- oder Dateinamenoptionen führt zur internen Nutzung von **vwait**. Um verschachtelte Ereignisschleifen in Tk- oder tclhttpd-Anwendungen zu vermeiden, sollten Sie die inkrementelle Programmier-API verwenden (siehe unten).

Es sollte nur eines der Argumente *-file*, *-channel* oder *string* angegeben werden.

Wenn die zu hashende *string* mit einer Option verwechselt werden könnte (führender Bindestrich "-"), verwenden Sie davor die Option **--**, um den Prozess der Optionen-Verarbeitung zu beenden und die Interpretation als Zeichenkette zu erzwingen.

::sha2::sha224 ?-hex|-bin? [-channel channel | -file filename | ?--? *string*]
Wie **::sha2::sha256**, nur dass hier der SHA224-Hash zurückgegeben wird.

::sha2::hmac *key string*
siehe unten ...

::sha2::hmac ?-hex|-bin? -key key [-channel channel | -file filename | ?--? *string*]
Berechnet einen HMAC-Digest (Hashed Message Authentication Digest) unter Verwendung des SHA256-Digest-Algorithmus. HMACs sind in RFC 2104 (3) beschrieben und liefern einen SHA256-Digest, der einen Schlüssel enthält. Alle Optionen außer *-key* entsprechen denen des Befehls **::sha2::sha256**.

Wenn die zu hashende *Zeichenkette* mit einer Option verwechselt werden könnte (führender Bindestrich "-"), setzen Sie die Option **--** davor, um den Options-Prozess zu beenden und die Interpretation als Zeichenkette zu erzwingen.

Programmierschnittstelle

Für den Programmierer lässt sich der SHA256-Hash als eine Art "Behälter" betrachten, in den Sie Daten einfüllen. Wenn Sie fertig sind, extrahieren Sie einen Wert, der aus den in den Behälter eingefüllten Daten abgeleitet wird. Die Programmierschnittstelle zum SHA256-Hash arbeitet mit einem Token (das dem Behälter entspricht). Sie rufen **SHA256Init** auf, um ein Token zu erhalten, und ruft anschließend so oft wie nötig **SHA256Update** auf, um Daten zum Hash hinzuzufügen. Um Ressourcen freizugeben und den Wert des Hashes zu erhalten, rufen Sie dann **SHA256Final** auf. Ein entsprechender Funktionssatz liefert Ihnen einen schlüsselgebundenen Digest (HMAC).

Wenn Sie **critcl** installiert und das **tcllibc**-Paket kompiliert haben, erfolgt die Implementierung der Hash-Funktion durch kompilierten Code. Andernfalls steht ein reines Tcl-Äquivalent zur Verfügung. Die Programmierschnittstelle bleibt in allen Fällen unverändert.

::sha2::SHA256Init
: siehe unten ...

::sha2::SHA224Init
: Erzeugt einen neuen SHA256/SHA224-Hash. Gibt eine Token-ID zurück, die für die verbleibenden Funktionen verwendet werden muss.

::sha2::SHA256Update *token data*
: Fügt Daten zu dem durch das Token identifizierten Hash hinzu. Der Aufruf von *SHA256Update $token* "*abcd*" entspricht dem Aufruf von *SHA256Update $token* "*ab*", gefolgt von *SHA256Update $token* "*cb*". Siehe Beispiele. Hinweis: Dieser Befehl wird sowohl für SHA256 als auch für SHA224 verwendet. Nur die Initialisierungs- und Abschlussbefehle der beiden Hash-Funktionen unterscheiden sich.

::sha2::SHA256Final *token*
: siehe unten ...

::sha2::SHA224Final *token*
: Gibt den Hash-Wert zurück und gibt alle von diesem Token belegten Ressourcen frei. Sobald dieser Befehl abgeschlossen ist, wird das Token ungültig. Das Ergebnis ist eine binäre Zeichenkette mit einer Länge von 32 bzw. 28 Byte, die den 256- bzw. 224-Bit-SHA256-/SHA224-Wert darstellt.

::sha2::HMACInit *key*
: Dies entspricht dem Befehl **::sha2::SHA256Init**, mit dem Unterschied, dass hierfür der Schlüssel angegeben werden muss, der in den HMAC einbezogen werden soll.

::sha2::HMACUpdate *token data*
: siehe unten ...

::sha2::HMACFinal *token*
: Diese Befehle entsprechen den entsprechenden SHA256-Befehlen.

Beispiele

```
% sha2::sha256 "Tcl does SHA256"
0b91043ee484abd83c3e4b08d6034d71b937026379f0f59bda6e625e6e214789
```

```
% sha2::hmac Sekret "Tcl does SHA256"
4f9352c64d655e8a36abe73e6163a9d7a54039877c1c92ec90b07d48d4e854e0
```

```
% set tok [sha2::SHA256Init]
::sha2::1
% sha2::SHA256Update $tok "Tcl "
% sha2::SHA256Update $tok "does "
% sha2::SHA256Update $tok "SHA256"
% sha2::Hex [sha2::SHA256Final $tok]
0b91043ee484abd83c3e4b08d6034d71b937026379f0f59bda6e625e6e214789
```

Literatur

1. "Secure Hash Standard", National Institute of Standards and Technology, US-Handelsministerium, April 1995. (http://www.itl.nist.gov/fipspubs/fip180-1.htm)
2. Rivest, R., "The MD4 Message Digest Algorithm", RFC 1320, MIT, April 1992. (http://www.rfc-editor.org/rfc/rfc1320.txt)
3. Krawczyk, H., Bellare, M. und Canetti, R. "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, Februar 1997. (http://www.rfc-editor.org/rfc/rfc2104.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *sha1* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2008, Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

simulation

annealing

Stichwörter: annealing, math, optimization, simulated annealing

Name

simulation::annealing – Simuliertes Annealing

Übersicht

package require Tcl ?**8.5 9**?
package require simulation::annealing 0.3
::simulation::annealing::getOption *keyword*
::simulation::annealing::hasOption *keyword*

::simulation::annealing::setOption *keyword value*
::simulation::annealing::findMinimum *args*
::simulation::annealing::findCombinatorialMinimum *args*

Beschreibung

Die Technik des *Simulated Annealing* bietet Methoden zur Schätzung des globalen Optimums einer Funktion. Sie wird auf der Wiki-Seite http://wiki.tcl.tk/... ausführlich beschrieben. Die Idee ist einfach:

- Punkte innerhalb eines vorgegebenen Suchraums zufällig auswählen
- Die Auswertung der zu optimierenden Funktion für jeden dieser Punkte durchführen und den Punkt auswählen, der den niedrigsten (oder höchsten) Wert aufweist, oder – in manchen Fällen – einen Punkt akzeptieren, der einen weniger optimalen Wert aufweist. Die Wahrscheinlichkeit, dass ein solcher nicht-optimaler Punkt akzeptiert wird, nimmt mit der Zeit ab.
- Die Akzeptanz von weniger optimalen Punkten bedeutet, dass die Methode nicht zwangsläufig in einem lokalen Optimum stecken bleibt und theoretisch in der Lage ist, das globale Optimum innerhalb des Suchraums zu finden.

Die Methode ähnelt dem Abkühlen von Material, daher der Name.

Das Paket *simulation::annealing* bietet den Befehl *findMinimum*:

```
    puts [::simulation::annealing::findMinimum  -trials 300
    -parameters {x -5.0 5.0 y -5.0 5.0}  -function
    {$x*$x+$y*$y+sin(10.0*$x)+4.0*cos(20.0*$y)}]
```

gibt den geschätzten Minimalwert der Funktion f(x,y) = *x**2+y**2+sin(10*x)+4*cos(20*y)* sowie die Werte von x und y aus, bei denen das Minimum erreicht wurde:

```
result -4.9112922923 x -0.181647676593 y 0.155743646974
```

Prozeduren

Das Paket definiert die folgenden Hilfsprozeduren:

::simulation::annealing::getOption *keyword*
Ruft den Wert einer Option ab, die als Teil des Befehls *findMinimum* angegeben wurde.
string *keyword*
Angegebenes Schlüsselwort (ohne vorangestelltes Minuszeichen)

::simulation::annealing::hasOption *keyword*
Gibt 1 zurück, wenn die Option vorhanden ist, andernfalls 0.
string *keyword*
Angegebenes Schlüsselwort (ohne vorangestelltes Minuszeichen)

::simulation::annealing::setOption *keyword value*
Setzt den Wert der angegebenen Option.
string *keyword*
Angegebenes Schlüsselwort (ohne vorangestelltes Minuszeichen)
string *value*
(Neuer) Wert der Option

Die wichtigsten Prozeduren sind *findMinimum* und *findCombinatorialMinimum*:

::simulation::annealing::findMinimum *args*
Ermittelt das Minimum einer Funktion mithilfe von Simulated Annealing. Die Funktion und die Parameter des Verfahrens werden über eine Liste von Schlüsselwort-Wert-Paaren vorgegeben.
int *n*
Liste von Schlüsselwort-Wert-Paaren, die während der Ausführung alle über den Befehl *getOption* abrufbar sind.

::simulation::annealing::findCombinatorialMinimum *args*
Ermittelt das Minimum einer Funktion diskreter Variablen mithilfe von Simulated Annealing. Die Funktion und die Parameter des Verfahrens werden über eine Liste von Schlüsselwort-Wert-Paaren vorgegeben.
int *n*
Liste von Schlüsselwort-Wert-Paaren, die während der Ausführung alle über den Befehl *getOption* abrufbar sind.

Der Befehl *findMinimum* enthält die folgenden vordefinierten Optionen:

- *-parameters list*: Tripel, die Parameter und Bereiche festlegen
- *-function expr*: Ausdruck, der die Funktion definiert
- *-code body*: Prozedurrumpf, der die Funktion definiert (hat Vorrang vor *-function*). Der Code soll die Variable "result" setzen
- *-init code*: Code, der zu Beginn ausgeführt wird *-final code*: Code, der am Ende ausgeführt wird *-trials n*: Anzahl der Versuche, bevor die Temperatur gesenkt wird *-reduce factor*: senkt die Temperatur um diesen Faktor (zwischen 0 und 1) *-initial-temp t*: Anfangstemperatur *-scale s*: Skala der Funktion (Größenordnung der Werte) *-estimate-scale y/n*: schätzt die Skala (nur wenn *-scale* fehlt) *-verbose y/n*: gibt ausführliche Fortschrittsinformationen in die Berichtsdatei aus (1) oder nicht (0) *-reportfile file*: geöffnete Datei für die Ausgabe (Voreinstellung stdout)

Alle weiteren Optionen lassen sich über die Prozedur getOption im Prozedurrumpf verwenden. Der Befehl *findCombinatorialMinimum* gibt die folgenden Optionen vor:

- *-number-params n*: Anzahl der binären Parameter (der Lösungsraum besteht aus Listen aus Einsen und Nullen). Dies ist eine erforderliche Option.
- *-initial-values*: Eine Liste aus Einsen und Nullen, die den Anfang der Suche bildet.

Die übrigen vordefinierten Optionen entsprechen denen von *findMinimum*.

Tipps

Die Prozedur *findMinimum* funktioniert, indem sie eine temporäre Prozedur erstellt, die die eigentliche Arbeit übernimmt. Sie durchläuft eine Schleife, bis sich der Punkt, der das geschätzte Optimum darstellt, innerhalb der vorgegebenen Anzahl von Versuchen nicht mehr ändert. Da die Temperatur immer weiter sinkt, verringert sich auch die Wahrscheinlichkeit, einen Punkt mit einem höheren Wert zu akzeptieren, sodass die Prozedur in der Praxis beendet wird.

Es ist möglich, über einen nicht rechteckigen Bereich zu optimieren, doch ist dabei gewisse Vorsicht geboten:

- Wenn der Punkt außerhalb des Untersuchungsbereichs liegt, können Sie einen sehr hohen Wert festlegen.
- Dies bedeutet, dass eine automatische Bestimmung eines Skalierungsfaktors nicht in Frage

kommt – die hohen Werte der Funktionen, die den Punkt in den Bereich hineinzwingen, würden die Schätzung verfälschen.

Hier sind Beispiele für die Suche nach einem Optimum innerhalb eines Kreises:

```
    puts [::simulation::annealing::findMinimum  -trials 3000  -reduce
    0.98  -parameters {x -5.0 5.0 y -5.0 5.0}  -code {
            if { hypot($x-5.0,$y-5.0) < 4.0 } {
                set result [expr
    {$x*$x+$y*$y+sin(10.0*$x)+4.0*cos(20.0*$y)}]
            } else {
                set result 1.0e100
            }
        }]
```

Die Methode ist theoretisch in der Lage, das globale Optimum zu ermitteln; häufig sind jedoch eine große Anzahl von Versuchen und eine langsame Absenkung der Temperatur erforderlich, um zuverlässige und reproduzierbare Schätzwerte zu erhalten.

Sie können die Option *-final* verwenden, um eine deterministische Optimierungsmethode anzuwenden, sobald Sie sicher sind, dass Sie sich in der Nähe des gewünschten Optimums befinden.

Die Prozedur *findCombinatorialMinimum* eignet sich für Fälle, in denen die Parameter die Werte 0 oder 1 annehmen (und es kann viele davon geben). Hier ein Beispiel:

- Wir haben eine Funktion, die ein absolutes Minimum erreicht, wenn die ersten zehn Zahlen 1 und die übrigen 0 sind:

```
proc cost {params} {
    set cost 0
    foreach p [lrange $params 0 9] {
        if { $p == 0 } {
            incr cost
        }
    }
    foreach p [lrange $params 10 end] {
        if { $p == 1 } {
            incr cost
        }
    }
    return $cost
}
```

- Wir möchten die Lösung finden, die dieses Minimum für verschiedene Längen des Lösungsvektors *params* liefert:

```
foreach n {100 1000 10000} {
    break
    puts "Problem size: $n"
    puts [::simulation::annealing::findCombinatorialMinimum  -trials
    300  -verbose 0  -number-params $n  -code {set result [cost
    $params]}]
}
```

- Mit zunehmender Vektorgröße steigt die Rechenzeit, doch die Prozedur endet, sobald eine Art Gleichgewicht erreicht ist. Für eine brauchbare Lösung können verschiedene Werte des Parameters trials ausprobiert werden. Außerdem sollte die zu minimierende Funktion von allen oder den meisten Parametern abhängen; ein Gegenbeispiel steht im Quellcode und sollte ausgeführt werden.

Kategorie
Mathematik

Copyright
```
Copyright (c) 2008 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

montecarlo
Stichwörter: montecarlo, math, montecarlo simulation, stochastic modelling

Name
simulation::montecarlo – Monte-Carlo-Simulationen

Übersicht
package require Tcl ?**8.5 9**?
package require simulation::montecarlo 0.2
package require simulation::random
package require math::statistics
::simulation::montecarlo::getOption *keyword*
::simulation::montecarlo::hasOption *keyword*
::simulation::montecarlo::setOption *keyword value*
::simulation::montecarlo::setTrialResult *values*
::simulation::montecarlo::setExpResult *values*
::simulation::montecarlo::getTrialResults
::simulation::montecarlo::getExpResult
::simulation::montecarlo::transposeData *values*
::simulation::montecarlo::integral2D *...*
::simulation::montecarlo::singleExperiment *args*

Beschreibung
Das Prinzip der *Monte-Carlo-Simulationen* ist im Grunde genommen einfach:

- Zufallswerte für einen oder mehrere Parameter erzeugen.
- Das Modell eines interessierenden Systems auswerten und die interessanten Ergebnisse für

jede Realisierung dieser Parameter festhalten.
- Nach einer geeigneten Anzahl solcher Versuche eine Gesamteigenschaft des Modells ableiten.

Denkbar ist ein Modell eines Rechnernetzes, ein Ökosystem oder überhaupt alles, was sich quantitativ beschreiben lässt und ein stochastisches Element enthält.

Das Paket *simulation::montecarlo* bietet einen grundlegenden Rahmen für ein solches Modellierungsverfahren:

```
#
# MC experiments:
# Determine the mean and median of a set of points and compare them
#
::simulation::montecarlo::singleExperiment -init {
    package require math::statistics

    set prng [::simulation::random::prng_Normal 0.0 1.0]
} -loop {
    set numbers {}
    for { set i 0 } { $i < [getOption samples] } { incr i } {
        lappend numbers [$prng]
    }
    set mean   [::math::statistics::mean $numbers]
    set median [::math::statistics::median $numbers] ;# ? Exists?
    setTrialResult [list $mean $median]
} -final {
    set result [getTrialResults]
    set means   {}
    set medians {}
    foreach r $result {
        foreach {m M} $r break
        lappend means   $m
        lappend medians $M
    }
    puts [getOption reportfile] "Correlation:
    [::math::statistics::corr $means $medians]"

} -trials 100 -samples 10 -verbose 1 -columns {Mean Median}
```

In diesem Beispiel soll untersucht werden, inwieweit der Medianwert und der Mittelwert einer zufälligen Zahlenmenge miteinander korrelieren. Manchmal ist der Medianwert ein robusteres Merkmal als der Mittelwert – insbesondere bei einer statistischen Verteilung mit "dicken" Schwänzen.

Prozeduren

Das Paket definiert die folgenden Hilfsprozeduren:

::simulation::montecarlo::getOption *keyword*

Ruft den Wert einer Option ab, die als Teil des Befehls *singleExperiment* angegeben wurde.

string *keyword*
Angegebenes Schlüsselwort (ohne vorangestelltes Minuszeichen)

::simulation::montecarlo::hasOption *keyword*
Gibt 1 zurück, wenn die Option vorhanden ist, andernfalls 0.
string *keyword*
Angegebenes Schlüsselwort (ohne vorangestelltes Minuszeichen)

::simulation::montecarlo::setOption *keyword value*
Setzt den Wert der angegebenen Option.
string *keyword*
Angegebenes Schlüsselwort (ohne vorangestelltes Minuszeichen)
string *value*
(Neuer) Wert der Option

::simulation::montecarlo::setTrialResult *values*
Speichert die Ergebnisse des Versuchs zur späteren Auswertung
list *values*
Liste der zu speichernden Werte

::simulation::montecarlo::setExpResult *values*
Setzt die Ergebnisse des gesamten Experiments (üblicherweise in der Abschlussphase).
list *values*
Liste der zu speichernden Werte

::simulation::montecarlo::getTrialResults
Ruft die Ergebnisse aller einzelnen Versuche zur Auswertung ab (üblicherweise in der Abschlussphase oder nach Abschluss des Befehls).

::simulation::montecarlo::getExpResult
Ruft die Ergebnisse des gesamten Experiments ab (üblicherweise in der Abschlussphase oder auch nach Ausführung des Befehls *singleExperiment*).

::simulation::montecarlo::transposeData *values*
Vertauscht Spalten und Zeilen einer Liste von Listen und gibt das Ergebnis zurück.
list *values*
Liste von Listen von Werten

Es gibt zwei Hauptprozeduren: *integral2D* und *singleExperiment*.

::simulation::montecarlo::integral2D*...*
Integriert eine Funktion über einen zweidimensionalen Bereich mit einem Monte-Carlo-Verfahren.

Argumente: PM

::simulation::montecarlo::singleExperiment *args*
Führt den Code über eine Anzahl von Versuchen aus und speichert die Ergebnisse. Die Iteration wird durch Parameter gesteuert, die über eine Liste von Schlüsselwort-Wert-Paaren vorgegeben werden.
int *n*

Liste von Schlüsselwort-Wert-Paaren, die während der Ausführung alle über den Befehl *getOption* abrufbar sind.

Der Befehl *singleExperiment* legt die folgenden Optionen vordefiniert fest:

- *-init code*: Code, der zu Beginn ausgeführt wird
- *-loop body*: Prozedurrumpf, der die wiederholt auszuführende Berechnung definiert. Der Code soll *setTrialResult* verwenden, um die Ergebnisse jedes Versuchs zu speichern (in der Regel eine Liste von Zahlen; die Interpretation bleibt der Implementierung überlassen). Hinweis: erforderliches Schlüsselwort.
- *-final code*: Code, der am Ende ausgeführt wird
- *-trials n*: Anzahl der Versuche im Experiment (erforderlich)
- *-reportfile file*: geöffnete Datei, an die die Ausgabe gehen soll (Voreinstellung: stdout)
- *-verbose*: schreibt die Zwischenergebnisse (1) oder nicht (0) (Voreinstellung: 0)
- *-analysis proc*: entweder "none" (keine automatische Analyse), standard (grundlegende Statistiken der Versuchsergebnisse und eine Korrelationsmatrix) oder der Name einer Prozedur, die die Analyse übernimmt.
- *-columns list*: Liste der Spaltennamen, nützlich für eine ausführliche Ausgabe und die Analyse.

Alle weiteren Optionen lassen sich über die Prozedur getOption im Prozedurrumpf verwenden.

Tipps

Die Prozedur *singleExperiment* erstellt eine temporäre Prozedur, die die eigentliche Arbeit übernimmt. Sie führt eine Schleife über die angegebene Anzahl von Versuchen durch.

Da eine temporäre Prozedur erstellt wird, bleiben die zu Beginn definierten lokalen Variablen innerhalb der Schleife bestehen.

Kategorie

Mathematik

Copyright

```
Copyright (c) 2008 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

simulation_random

Stichwörter: simulation_random, math, random numbers, simulation, statistical distribution

Name

simulation::random – Pseudozufallszahlengeneratoren

Übersicht

package require Tcl ?**8.5 9**?
package require simulation::random 0.4
::simulation::random::prng_Bernoulli *p*
::simulation::random::prng_Discrete *n*
::simulation::random::prng_Poisson *lambda*
::simulation::random::prng_Uniform *min max*
::simulation::random::prng_Triangular *min max*
::simulation::random::prng_SymmTriangular *min max*

::simulation::random::prng_Exponential *min mean*
::simulation::random::prng_Normal *mean stdev*
::simulation::random::prng_Pareto *min steep*
::simulation::random::prng_Gumbel *min f*
::simulation::random::prng_chiSquared *df*
::simulation::random::prng_Disk *rad*
::simulation::random::prng_Sphere *rad*
::simulation::random::prng_Ball *rad*
::simulation::random::prng_Rectangle *length width*
::simulation::random::prng_Block *length width depth*

Beschreibung

Dieses Paket enthält Befehle zur Erstellung von Pseudozufallszahlengeneratoren. Diese neuen Befehle bieten

- Zahlen, die normalverteilt, gleichmäßig verteilt oder nach einer Pareto- bzw. Gumbel-Verteilung verteilt sind usw.
- Koordinaten von Punkten, die gleichmäßig innerhalb einer Kugel oder eines Rechtecks verteilt sind.

Beispiel:

```
    set p [::simulation::random::prng_Normal -1.0 10.0]
```

Erzeugt einen neuen Befehl (dessen Name in der Variable "p" gespeichert ist), der normalverteilte Zahlen mit einem Mittelwert von -1,0 und einer Standardabweichung von 10,0 generiert.

Prozeduren

Das Paket definiert die folgenden öffentlichen Prozeduren für *diskrete* Verteilungen:

::simulation::random::prng_Bernoulli *p*
Erstellt einen Befehl (PRNG), der Zahlen mit einer Bernoulli-Verteilung erzeugt: Der Wert ist entweder 1 oder 0, mit der Wahrscheinlichkeit p, dass er 1 ist.
float *p*
Wahrscheinlichkeit, dass das Ergebnis 1 ist

::simulation::random::prng_Discrete *n*
Erstellt einen Befehl (PRNG), der die Zahlen 0 bis n-1 mit gleicher Wahrscheinlichkeit generiert.
int *n*
Anzahl der verschiedenen Werte (im Bereich von 0 bis n-1)

::simulation::random::prng_Poisson *lambda*
Erstellt einen Befehl (PRNG), der Zahlen gemäß der Poisson-Verteilung generiert.
float *lambda*
Mittlere Anzahl pro Zeitintervall

Das Paket definiert die folgenden öffentlichen Prozeduren für *kontinuierliche* Verteilungen:

::simulation::random::prng_Uniform *min max*

Erstellt einen Befehl (PRNG), der gleichmäßig verteilte Zahlen zwischen "min" und "max" generiert.

float *min*
Kleinste Zahl, die erzeugt wird

float *max*
Größte Zahl, die erzeugt wird

::simulation::random::prng_Triangular *min max*
Erstellt einen Befehl (PRNG), der Zahlen mit Dreiecksverteilung zwischen "min" und "max" generiert. Wenn das Argument "min" kleiner ist als das Argument "max", so haben kleinere Werte eine höhere Wahrscheinlichkeit und umgekehrt. Im ersten Fall hat die Wahrscheinlichkeitsdichtefunktion die Form *f(x) = 2(1-x)* und im anderen Fall die Form *f(x) = 2x*.

float *min*
Kleinste Zahl, die erzeugt wird

float *max*
Größte Zahl, die erzeugt wird

::simulation::random::prng_SymmTriangular *min max*
Erstellt einen Befehl (PRNG), der Zahlen erzeugt, die gemäß einem symmetrischen Dreieck um den Mittelwert von "min" und "max" verteilt sind.

float *min*
Kleinste Zahl, die erzeugt wird

float *max*
Größte Zahl, die erzeugt wird

::simulation::random::prng_Exponential *min mean*
Erstellt einen Befehl (PRNG), der exponentialverteilte Zahlen mit einem vorgegebenen Mindestwert und einem vorgegebenen Mittelwert erzeugt.

float *min*
Kleinste Zahl, die erzeugt wird

float *mean*
Mittelwert der Zahlen

::simulation::random::prng_Normal *mean stdev*
Erstellt einen Befehl (PRNG), der normalverteilte Zahlen mit einem vorgegebenen Mittelwert und einer vorgegebenen Standardabweichung erzeugt.

float *mean*
Mittelwert der Zahlen

float *stdev*
Standardabweichung

::simulation::random::prng_Pareto *min steep*
Erstellt einen Befehl (PRNG), der Zahlen gemäß der Pareto-Verteilung mit einem vorgegebenen Mindestwert und einer vorgegebenen Verteilungssteilheit erzeugt.

float *min*
Kleinste Zahl, die erzeugt wird

float *steep*
Steilheit der Verteilung

::simulation::random::prng_Gumbel *min f*
Erstellt einen Befehl (PRNG), der Zahlen gemäß der Gumbel-Verteilung mit einem vorgegebenen Mindestwert und einem vorgegebenen Skalierungsfaktor erzeugt. Die Wahrscheinlichkeitsdichtefunktion lautet:

```
P(v) = exp( -exp(f*(v-min)))
```

float *min*
Kleinste Zahl, die erzeugt wird
float *f*
Skalierungsfaktor für die Werte

::simulation::random::prng_chiSquared *df*
Erstellt einen Befehl (PRNG), der Zahlen gemäß der Chi-Quadrat-Verteilung mit df Freiheitsgraden erzeugt. Der Mittelwert ist 0 und die Standardabweichung ist 1.
float *df*
Freiheitsgrade

Das Paket definiert die folgenden öffentlichen Prozeduren für Zufalls-Punktmengen:

::simulation::random::prng_Disk *rad*
Erstellt einen Befehl (PRNG), der (x,y)-Koordinaten für Punkte erzeugt, die gleichmäßig über eine Scheibe mit gegebenem Radius verteilt sind.
float *rad*
Radius der Scheibe

::simulation::random::prng_Sphere *rad*
Erstellt einen Befehl (PRNG), der (x,y,z)-Koordinaten für Punkte erzeugt, die gleichmäßig über die Oberfläche einer Kugel mit gegebenem Radius verteilt sind.
float *rad*
Radius der Kugel

::simulation::random::prng_Ball *rad*
Erstellt einen Befehl (PRNG), der (x,y,z)-Koordinaten für Punkte erzeugt, die gleichmäßig innerhalb eines Balls mit gegebenem Radius verteilt sind.
float *rad*
Radius des Balls

::simulation::random::prng_Rectangle *length width*
Erstellt einen Befehl (PRNG), der (x,y)-Koordinaten für Punkte erzeugt, die gleichmäßig über ein Rechteck verteilt sind.
float *length*
Länge des Rechtecks (x-Richtung)
float *width*
Breite des Rechtecks (y-Richtung)

::simulation::random::prng_Block *length width depth*
Erstellt einen Befehl (PRNG), der (x,y,z)-Koordinaten für Punkte erzeugt, die gleichmäßig über einen Block verteilt sind

float *length*
Länge des Blocks (x-Richtung)

float *width*
Breite des Blocks (y-Richtung)

float *depth*
Tiefe des Blocks (z-Richtung)

Kategorie
Mathematik

Copyright
```
Copyright (c) 2004 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

smtpd

smtpd
Stichwörter: smtpd, rfc 2821, rfc 821, services, smtp, socket, vwait

Name
smtpd – Tcl-basierte SMTP-Server-Implementierung

Übersicht
package require Tcl 8.5 9
package require smtpd ?**1.6**?
::smtpd::start ?*myaddr*? ?*port*?
::smtpd::stop
::smptd::configure ?*option value*? ?*option value* ...?
::smtpd::cget ?*option*?

Beschreibung
Das Paket **smtpd** stellt eine einfache, ausschließlich auf Tcl basierende Serverbibliothek für das Simple Mail Transfer Protocol bereit, wie es in RFC 821 (http://www.rfc-editor.org/rfc/rfc821.txt) und RFC 2821 (http://www.rfc-editor.org/rfc/rfc2821.txt). Standardmäßig bindet der Server an die Standard-Netzwerkadresse und den Standard-SMTP-Port (25).

Dieses Paket wurde entwickelt, um Tests von Mail-User-Agent-Code von der Arbeitsstation eines Entwicklers aus zu ermöglichen. *Es wird nicht versucht, E-Mails an Ihr Postfach zuzustellen.* Stattdessen wird von den Nutzern dieses Pakets erwartet, dass sie eine Prozedur schreiben, die bei Eingang einer E-Mail aufgerufen wird. Sobald diese Prozedur zurückgibt, hat der Server nichts mehr mit der E-Mail zu tun.

Sicherheit
Auf Unix-Plattformen sind Root-Rechte erforderlich, um eine Bindung zum SMTP-Port

durchzuführen. Ich würde davon abraten, einen skriptbasierten Server als Root auszuführen, es sei denn, es gibt eine Methode, die Root-Rechte unmittelbar nach der Bindung des Sockets wieder abzulegen. Auf Windows-Plattformen sind keine Root- oder Administratorrechte erforderlich, um Sockets mit niedrigen Nummern zu binden. Allerdings ist die Sicherheit auf diesen Plattformen ohnehin gering.

Kurz gesagt: Dieser Code sollte wahrscheinlich nicht als dauerhaft laufender Mail Transfer Agent auf einem mit dem Internet verbundenen Server eingesetzt werden, auch wenn wir sorgfältig darauf achten, keine Benutzereingaben von außen zu bewerten. Es gibt viele andere, gründlich getestete und sicherheitsgeprüfte Programme, die als Mailserver für mit dem Internet verbundene Rechner verwendet werden können.

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
package require tls
tls::init -tls1 1 ;# forcibly activate support for the TLS1
protocol

... your own application code ...
```

Befehle

::smtpd::start ?*myaddr*? ?*port*?

Startet den Dienst, der auf *port* oder dem Standardport 25 lauscht. Wenn *myaddr* als Domänenname oder als numerische IP-Adresse im "Dotted-Quad"-Format angegeben wird, wird der Server-Socket an diese Netzwerkschnittstelle gebunden. Standardmäßig ist der Server an alle Netzwerkschnittstellen gebunden. Beispiel:

```
set sock [::smtpd::start [info hostname] 0]
```

wird an die Internetschnittstelle des Hosts auf dem ersten verfügbaren Port gebunden.

Derzeit unterstützt das Paket nur eine einzige Instanz eines SMTP-Servers. Dies könnte bei Bedarf geändert werden, was jedoch zur Folge hätte, dass das Paket etwas schwerer zu verstehen wäre. Wenn es einen triftigen Grund für den Betrieb mehrerer SMTP-Dienste gibt, wäre lediglich eine Anpassung des Arrays "**options**" und der Verwendung der Variablen "**::smtpd::stopped**" erforderlich.

Da der Servercode **fileevent**(n)-Handler zur Verarbeitung der Eingaben über Sockets verwendet, muss die Ereignisschleife laufen. Die Anwendung läuft dazu innerhalb von **wish**(1), oder **vwait**(n) wartet auf die Variable **::smtpd::stopped**, die gesetzt wird, wenn der Server anhält.

::smtpd::stop

Hält den Server an und gibt den lauschenden Socket frei. Wenn der Server nicht gestartet wurde, hat dieser Befehl keine Auswirkung. Die Variable **::smtpd::stopped** wird für die Verwendung mit **vwait**(n) gesetzt.

Beachten Sie, dass durch das Beenden des Servers keine derzeit aktiven Sitzungen getrennt werden, da diese über einen unabhängigen Kanal laufen. Nur durch das explizite Nachverfolgen und Schließen dieser Sitzungen oder durch das Beenden des Server-Prozesses werden alle laufenden Sitzungen beendet. Dies entspricht der üblichen Vorgehensweise bei Unix-Daemons, bei der der Server einen **fork**(2)-Befehl ausführt und die Client-Sitzung im untergeordneten Prozess fortgesetzt wird.

::smptd::configure ?*option value*? ?*option value...*?

Legt die Konfigurationsoptionen für den SMTP-Server fest. Bei den meisten Werten handelt es sich um den Namen einer Callback-Prozedur, die an verschiedenen Stellen im SMTP-Protokoll aufgerufen wird. Einzelheiten zu den Prozeduren finden Sie im Abschnitt Callbacks.

-banner *text*

Hinweis zu einer benutzerdefinierten Banner-Meldung. Das Standard-Banner lautet "tcllib smtpd 1.6". Beachten Sie, dass eine Änderung des Banners keinen Einfluss auf den Text in Klammern in der vollständigen Begrüßung, die Ausgabe des Status 220, die Serveradresse und den Zeitstempel hat.

-validate_host *proc*

Rückruffunktion zur Authentifizierung neuer Verbindungen anhand der IP-Adresse des Clients.

-validate_sender *proc*

Rückruffunktion zur Authentifizierung neuer Verbindungen anhand der E-Mail-Adresse des Absenders.

-validate_recipient *proc*

Rückruffunktion zur Validierung und Autorisierung einer E-Mail-Adresse des Empfängers

-deliverMIME *proc*

Rückruffunktion, die zur Zustellung von E-Mails als MIME-Token verwendet wird, das vom tcllib-Paket **mime** erstellt wurde.

-deliver *proc*

Rückruffunktion zur Zustellung von E-Mails. Diese Option hat keine Auswirkung, wenn die Option **-deliverMIME** gesetzt wurde.

::smtpd::cget ?*option*?

Wird keine *option* angegeben, gibt der Befehl eine Liste aller Optionen und ihrer aktuellen Werte zurück. Wird eine Option angegeben, gibt er den Wert dieser Option zurück.

Callbacks

validate_host callback

Diese Prozedur wird mit der IP-Adresse des Clients aufgerufen, sobald eine Verbindungsanfrage angenommen wurde und bevor Protokollbefehle verarbeitet werden. Wenn Sie einem bestimmten Host den Zugriff verweigern möchten, sollte diese Rückruffunktion einen Fehler geben. Beispiel:

```
proc validate_host {ipnum} {
   if {[string match "192.168.1.*" $ipnum]} {
      error "go away!"
   }
}
```

Wenn der Zugriff verweigert wird, erhält der Client eine Standardmeldung, die den Wortlaut Ihrer Fehlermeldung enthält, beispielsweise:

```
550 Access denied: I hate you.
```

Gemäß dem SMTP-Protokoll wird die Verbindung nicht geschlossen, sondern wir warten darauf, dass der Client einen QUIT-Befehl sendet. Alle anderen Befehle führen zu einem Fehler "**503 Bad Sequence**".

validate_sender callback

Die Rückruffunktion validate_sender wird während der Verarbeitung eines MAIL-Befehls mit der E-Mail-Adresse des Absenders aufgerufen, damit Sie E-Mails je nach dem angegebenen Absender annehmen oder ablehnen können. Um eine E-Mail abzulehnen, sollten Sie einen Fehler auslösen. Um beispielsweise E-Mails vom Benutzer "denied" abzulehnen:

```
proc validate_sender {address} {
   eval array set addr [mime::parseaddress $address]
   if {[string match "denied" $addr(local)]} {
        error "mailbox $addr(local) denied"
   }
   return
}
```

Der Inhalt einer Fehlermeldung wird nicht an den Client zurückgegeben.

validate_recipient callback

Die Rückruffunktion "validate_recipient" ähnelt der Rückruffunktion "validate_sender" und ermöglicht es Ihnen, während der Verarbeitung des RCPT-Befehls ein lokales Postfach zu überprüfen und E-Mails für eine lokale Benutzeradresse anzunehmen. Um E-Mails abzulehnen, lösen Sie wie oben beschrieben einen Fehler aus. Die Fehlermeldung wird ignoriert.

deliverMIME callback

Die Rückruffunktion "deliverMIME" wird aufgerufen, sobald eine E-Mail-Nachricht erfolgreich an den Server übermittelt wurde. Aus dem Absender, den Empfängern und den Daten wird ein MIME-Token erstellt, und die Prozedur des Benutzers wird mit diesem einzigen Argument aufgerufen. Wenn der Aufruf zurückgibt, wird das MIME-Token bereinigt; möchte der Benutzer die Daten daher aufbewahren, muss er eine Kopie anlegen.

```
proc deliverMIME {token} {
    set sender [lindex [mime::getheader $token From] 0]
    set recipients [lindex [mime::getheader $token To] 0]
    set mail "From $sender [clock format [clock seconds]]"
    append mail "\n" [mime::buildmessage $token]
    puts $mail
}
```

deliver callback

Der "deliver"-Callback wird aufgerufen, sobald eine E-Mail-Nachricht erfolgreich an den Server übermittelt wurde und die Option -deliverMIME nicht gesetzt ist. Die Prozedur wird mit dem Absender, einer Liste der Empfänger und dem Text der E-Mail als Liste von Zeilen aufgerufen. Beispiel:

```
proc deliver {sender recipients data} {
   set mail "From $sender  [clock format [clock seconds]]"
   append mail "\n" [join $data "\n"]
   puts "$mail"
}
```

Beachten Sie, dass der Befehl "DATA" eine Fehlermeldung gibt, wenn noch kein Absender oder Empfänger definiert wurde.

Variablen

::smtpd::stopped

Diese Variable wird während des Befehls **::smtpd::stop** auf **true** gesetzt, damit **vwait**(n) darauf warten kann.

Autoren

Verfasst von Pat Thoyts *mailto:patthoyts@users.sourceforge.net*.

Lizenz

Diese Software wird in der Hoffnung bereitgestellt, dass sie von Nutzen ist, jedoch OHNE JEGLICHE GEWÄHRLEISTUNG; dies schließt sogar die stillschweigende Gewährleistung der MARKTGÄNGIGKEIT oder der EIGNUNG FÜR EINEN BESTIMMTEN ZWECK ein. Weitere Einzelheiten entnehmen Sie bitte der Datei "*license.terms*".

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *smtpd* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

Copyright

```
Copyright (c) Pat Thoyts <patthoyts@users.sourceforge.net>
```


snit

snit
Stichwörter: snit, BWidget, C++, Incr Tcl, adaptors, class, mega widget, object, object oriented, type, widget, widget adaptors

Name
snit – Snit ist kein Incr Tcl

Übersicht
package require Tcl 8.5 9
package require snit ?**2.3.4**?
snit::type *name definition*
typevariable *name* ?**-array**? ?*value*?
typemethod *name arglist body*
typeconstructor *body*
variable *name* ?**-array**? ?*value*?
method *name arglist body*
option *namespec* ?*defaultValue*?
option *namespec* ?*options...*?
constructor *arglist body*
destructor *body*
proc *name args body*
delegate method *name* **to** *comp* ?*as target*?
delegate method *name* ?*to comp*? **using** *pattern*
delegate method * ?*to comp*? ?*using pattern*? ?*except exceptions*?
delegate option *namespec* **to** *comp*
delegate option *namespec* **to** *comp* **as** *target*
delegate option * **to** *comp*
delegate option * **to** *comp* **except** *exceptions*
component *comp* ?*-public method*? ?*-inherit flag*?
delegate typemethod *name* **to** *comp* ?*as target*?
delegate typemethod *name* ?*to comp*? **using** *pattern*
delegate typemethod * ?*to comp*? ?*using pattern*? ?*except exceptions*?
typecomponent *comp* ?*-public typemethod*? ?*-inherit flag*?
pragma ?*options...*?
expose *comp*
expose *comp* **as** *method*
onconfigure *name arglist body*
oncget *name body*

snit::widget *name definition*
widgetclass *name*
hulltype *type*
snit::widgetadaptor *name definition*
snit::typemethod *type name arglist body*
snit::method *type name arglist body*
snit::macro *name arglist body*
snit::compile *which type body*
$type *typemethod args...*
$type create *name* ?*option value* ...?
$type info typevars ?*pattern*?
$type info typemethods ?*pattern*?
$type info args *method*
$type info body *method*
$type info default *method aname varname*
$type info instances ?*pattern*?
$type destroy
$object *method args...*
$object configure ?*option*? ?*value*? ...
$object configurelist *optionlist*
$object cget *option*
$object destroy
$object info type
$object info vars ?*pattern*?
$object info typevars ?*pattern*?
$object info typemethods ?*pattern*?
$object info options ?*pattern*?
$object info methods ?*pattern*?
$object info args *method*
$object info body *method*
$object info default *method aname varname*
mymethod *name* ?*args...*?
mytypemethod *name* ?*args...*?
myproc *name* ?*args...*?
myvar *name*
mytypevar *name*
from *argvName option* ?*defvalue*?
install *compName* **using** *objType objName args...*
installhull using *widgetType args...*
installhull *name*
variable *name*
typevariable *name*
varname *name*
typevarname *name*
codename *name*
snit::boolean validate ?*value*?

snit::boolean *name*
snit::double validate ?*value*?
snit::double *name* ?*option value...*?
snit::enum validate ?*value*?
snit::enum *name* ?*option value...*?
snit::fpixels validate ?*value*?
snit::fpixels *name* ?*option value...*?
snit::integer validate ?*value*?
snit::integer *name* ?*option value...*?
snit::listtype validate ?*value*?
snit::listtype *name* ?*option value...*?
snit::pixels validate ?*value*?
snit::pixels *name* ?*option value...*?
snit::stringtype validate ?*value*?
snit::stringtype *name* ?*option value...*?
snit::window validate ?*value*?
snit::window *name*

Beschreibung

Snit ist ein reines Tcl-Objekt- und Megawidget-System. Es zeichnet sich unter den Tcl-Objektsystemen dadurch aus, dass es nicht auf Vererbung, sondern auf Weiterleitung basiert. Objektsysteme, die auf Vererbung basieren, erlauben es Ihnen lediglich, von Klassen zu erben, die mit demselben System definiert wurden, was einschränkend ist. In Tcl ist ein Objekt alles, was sich wie ein Objekt verhält; es sollte keine Rolle spielen, wie das Objekt implementiert wurde. Snit soll Ihnen dabei helfen, Anwendungen aus den vorhandenen Bausteinen zu erstellen; daher ist Snit so konzipiert, dass es jedes Objekt einbinden und darauf aufbauen kann, sei es ein handcodiertes Objekt, ein **Tk**-Widget, ein **Incr Tcl**-Objekt, ein **BWidget** oder fast alles andere.

Diese Manpage dient lediglich als Nachschlagewerk; eine leichter verständliche, eher als Tutorial gestaltete Einführung in die Konzepte von Snit finden Sie im beiliegenden Dokument snitfaq.

Snit-Versionen

Diese Manpage behandelt sowohl Snit 2.2 als auch Snit 1.3. Der wesentliche Unterschied zwischen den beiden Versionen besteht lediglich darin, dass Snit 2.2 Geschwindigkeitsoptimierungen enthält, die auf neuen Funktionen von Tcl 8.5 basieren; Snit 1.3 unterstützt Tcl 8.3, 8.4 und Tcl 8.5. Es gibt einige geringfügige Abweichungen; diese sind im Text der Manpage mit dem Vermerk "Inkompatibilität mit Snit 1.x" gekennzeichnet; sie werden zudem im snitfaq behandelt.

Referenz

Typ- und Widget-Definitionen

Snit stellt die folgenden Befehle zur Definition neuer Typen bereit:

snit::type *name definition*

Definiert einen neuen abstrakten Datentyp namens *name*. Ist *name* kein vollständig qualifizierter Befehlsname, wird davon ausgegangen, dass es sich um einen Namen in dem Namespace handelt, in dem der Befehl **snit::type** aufgerufen wurde (in der Regel der globale Namespace). Der Befehl gibt den vollqualifizierten Namen des neuen Typs zurück.

Der Typname ist dann ein Befehl, der unter anderem zum Anlegen von Objekten des neuen Typs dient.

Der *definition*-Block von **snit::type** ist ein Skript, das die folgenden Definitionen enthalten kann:

typevariable *name* ?-array? ?*value*?
: Definiert eine Typvariable namens *name* und optional mit dem Wert *value*. Typvariablen werden von allen Instanzen des Typs gemeinsam genutzt. Wenn die Option **-array** angegeben wird, sollte *value* ein Dictionary sein; er wird der Variablen mithilfe von **array set** zugewiesen.

typemethod *name arglist body*
: Definiert eine Typmethode – einen Unterbefehl des neuen Typbefehls – mit dem angegebenen Namen, der angegebenen Argumentliste und dem angegebenen Rumpf. Die *arglist* ist eine normale Tcl-Argumentliste und darf Standardargumente sowie das Argument **args** enthalten; sie darf jedoch nicht die Argumentnamen **type**, **self**, **selfns** oder **win** enthalten.

  Die Variable **type** wird im *body* automatisch auf den vollqualifizierten Namen des Typs gesetzt. Darüber hinaus sind Typvariablen im *body* jeder Typmethode automatisch sichtbar.

  Wenn *name* aus zwei oder mehr Tokens besteht, behandelt Snit ihn auf besondere Weise:

```
    typemethod {a b} {arg} { puts "Got $arg" }
```

  Diese Anweisung definiert implizit eine Typmethode namens **a**, die einen Unterbefehl **b** enthält. **b** wird wie folgt aufgerufen:

```
    $type a b "Hello, world!"
```

  a kann eine beliebige Anzahl von Unterbefehlen enthalten. Dadurch lässt sich eine hierarchische Befehlsstruktur definieren; weitere Beispiele finden Sie weiter unten unter **method**.

  Typmethoden können Befehle aus dem Namespace aufrufen, in dem der Typ definiert ist, ohne diese importieren zu müssen; wenn der Typname beispielsweise **::parentns::typename** lautet, dann können die Typmethoden dieses Typs **::parentns::someproc** genauso aufrufen wie **someproc**. *Inkompatibilität mit Snit 1.x:* Dies funktioniert in Snit 1.x nicht, da es von **namespace path** abhängt, einem neuen Befehl in Tcl 8.5.

  Inkompatibilität mit Snit 1.x: In Snit 1.x sind die folgenden beiden Aufrufe dieser Typmethode gleichwertig:

```
    $type a b "Hello, world!"
    $type {a b} "Hello, world!"
```

  In Snit 2.2 ist die zweite Form ungültig.

typeconstructor *body*
: Der Rumpf *body* des Typkonstruktors wird einmalig ausgeführt, wenn der Typ zum ersten Mal definiert wird; er wird in der Regel dazu verwendet, Typvariablen mit Array-Werten zu

initialisieren und Einträge zur Tk-Optionsdatenbank hinzuzufügen.

Die Variable **type** wird automatisch im *body* definiert und enthält den vollqualifizierten Namen des Typs. Darüber hinaus sind Typvariablen im *body* des Typkonstruktors automatisch sichtbar.

Ein Typ darf höchstens einen Typkonstruktor definieren.

Der Typkonstruktor kann Befehle aus dem Namespace aufrufen, in dem der Typ definiert ist, ohne diese importieren zu müssen; wenn der Typname beispielsweise **::parentns::typename** lautet, dann kann der Typkonstruktor **::parentns::someproc** ebenso aufrufen wie **someproc**. *Inkompatibilität mit Snit 1.x:* Dies funktioniert nicht in Snit 1.x, da es von **namespace path** abhängt, einem neuen Befehl in Tcl 8.5.

variable *name* ?-array? ?*value*?

Definiert eine Instanzvariable, d. h. eine private Variable, die jeder Instanz dieses Typs zugeordnet ist, sowie optional deren Anfangswert. Wenn die Option **-array** angegeben wird, sollte *value* ein Dictionary sein; dieses wird der Variablen mithilfe von **array set** zugewiesen.

method *name arglist body*

Definiert eine Instanzmethode, d. h. einen Unterbefehl jeder Instanz dieses Typs, mit dem angegebenen Namen, der Argumentliste und dem Rumpf. Die *arglist* ist eine normale Tcl-Argumentliste und kann Standardargumente sowie das Argument **args** enthalten.

Der Methode werden implizit außerdem die folgenden Argumente übergeben: **type**, das den vollqualifizierten Typnamen enthält; **self**, das den Namen des aktuellen Instanzbefehls enthält; **selfns**, das den Namen des privaten Namespaces der Instanz enthält; und **win**, das den ursprünglichen Instanznamen enthält. Folglich darf die *arglist* die Argumentnamen **type**, **self**, **selfns** oder **win** nicht enthalten.

Eine auf diese Weise definierte Instanzmethode wird als *lokal definiert* bezeichnet.

Typ- und Instanzvariablen sind automatisch in allen Instanzmethoden sichtbar. Verfügt der Typ über lokal definierte Optionen, ist auch das Array **options** sichtbar.

Wenn *name* aus zwei oder mehr Tokens besteht, behandelt Snit ihn auf besondere Weise:

```
    method {a b} {} { ... }
```

Diese Anweisung definiert implizit eine Methode namens **a**, die einen Unterbefehl namens **b** enthält. **b** wird wie folgt aufgerufen:

```
    $self a b "Hello, world!"
```

a kann eine beliebige Anzahl von Unterbefehlen enthalten. Dadurch lässt sich eine hierarchische Befehlsstruktur definieren:

```
% snit::type dog {
    method {tail wag}   {} {return "Wag, wag"}
    method {tail droop} {} {return "Droop, droop"}
}
::dog
% dog spot
::spot
% spot tail wag
Wag, wag
% spot tail droop
Droop, droop
%
```

Wir haben implizit eine Methode "tail" mit den Unterbefehlen "wag" und "droop" definiert. Folglich ist es ein Fehler, "tail" explizit zu definieren.

Methoden können Befehle aus dem Namespace aufrufen, in dem der Typ definiert ist, ohne diese importieren zu müssen; wenn der Typname beispielsweise **::parentns::typename** lautet, dann können die Methoden des Typs **::parentns::someproc** ebenso aufrufen wie **someproc**. *Inkompatibilität mit Snit 1.x:* Dies funktioniert in Snit 1.x nicht, da es von **namespace path** abhängt, einem neuen Befehl in Tcl 8.5.

Inkompatibilität mit Snit 1.x: In Snit 1.x sind die folgenden beiden Aufrufe dieser Methode gleichbedeutend:

```
    $self a b "Hello, world!"
    $self {a b} "Hello, world!"
```

In Snit 2.2 ist die zweite Form ungültig.

option *namespec* ?*defaultValue*?

siehe unten ...

option *namespec* ?*options...*?

Definiert eine Option für Instanzen dieses Typs und weist ihr optional einen Anfangswert zu. Der Anfangswert ist standardmäßig die leere Zeichenkette, wenn kein *defaultValue* festgelegt wird.

Eine auf diese Weise definierte Option wird als *lokal definiert* bezeichnet.

Der *namespec* ist eine Liste, die den Namen der Option, den Ressourcennamen und den Namen der Klasse definiert, z. B.:

```
    option {-font font Font} {Courier 12}
```

Der Name der Option muss mit einem Bindestrich beginnen und darf keine Großbuchstaben enthalten. Der Ressourcenname und der Klassenname sind optional; werden sie nicht angegeben, wird als Ressourcenname standardmäßig der Name der Option ohne Bindestrich verwendet, und als Klassenname standardmäßig der Ressourcenname mit großem Anfangsbuchstaben. Somit entspricht die folgende Anweisung dem vorherigen Beispiel:

```
    option -font {Courier 12}
```

Weitere Informationen zu Ressourcen- und Klassennamen finden Sie im Abschnitt Tk-Optionsdatenbank.

Optionen werden normalerweise mithilfe der Standard-Instanzmethoden **configure** und **cget** gesetzt und abgerufen; innerhalb des Instanzcodes (Methodenrümpfe usw.) stehen die Werte der Optionen über das Array **options** zur Verfügung:

```
    set myfont $options(-font)
```

Wenn der Typ Optionshandler definiert (z. B. **-configuremethod**), sollte er wahrscheinlich **configure** und **cget** verwenden, um auf seine Optionen zuzugreifen, damit subtile Fehler vermieden werden.

Die Anweisung **option** kann die folgenden Optionen enthalten:

-default *defvalue*
: Legt den Vorgabewert der Option fest; sonst ist der Vorgabewert der Option "".

-readonly *flag*
: *flag* kann ein beliebiger boolescher Wert sein, der von Tcl erkannt wird. Wenn *flag* wahr ist, ist die Option schreibgeschützt – sie kann nur bei der Erstellung mithilfe von **configure** oder **configurelist** gesetzt werden, d. h. im Konstruktor des Typs.

-type *type*
: Jede lokal definierte Option kann ihren Validierungstyp festlegen; dabei kann es sich entweder um den Namen eines Validierungstyps oder um eine Spezifikation für einen Validierungsuntertyp handeln.

  Beispielsweise kann eine Option festlegen, dass ihr Wert eine ganze Zahl sein muss, indem sie **snit::integer** als Validierungstyp angibt:

  ```
      option -number -type snit::integer
  ```

  Sie kann auch festlegen, dass ihr Wert eine ganze Zahl zwischen 1 und 10 ist, indem sie einen Validierungsuntertyp angibt:

  ```
      option -number -type {snit::integer -min 1 -max 10}
  ```

  Wenn für eine Option ein Validierungstyp oder -untertyp definiert wird, so wird dieser zur Validierung des Werts der Option herangezogen, sobald dieser durch die Methoden **configure** oder **configurelist** des Objekts geändert wird. Darüber hinaus werden die Werte aller dieser Optionen unmittelbar nach der Ausführung des Konstruktors automatisch validiert.

  Snit definiert eine Reihe von Validierungstypen und -untertypen, und es ist recht einfach, neue zu definieren. Eine vollständige Liste finden Sie im Abschnitt Validierungstypen, und im Abschnitt Validierungstypen definieren erfahren Sie, wie Sie eigene Typen definieren können.

-cgetmethod *methodName*
: Jede lokal definierte Option kann eine **-cgetmethod** definieren; diese wird aufgerufen,

wenn der Wert der Option mithilfe der Methode **cget** abgerufen wird. Was auch immer der Rumpf *body* der Methode zurückgibt, ist der Rückgabewert des Aufrufs von **cget**.

Die benannte Methode muss ein Argument entgegennehmen, nämlich den Namen der Option. Dieser Code entspricht beispielsweise der Standardbehandlung von **cget** durch Snit (ist jedoch langsamer):

```
option -font -cgetmethod GetOption
method GetOption {option} {
    return $options($option)
}
```

Beliebig viele Optionen können sich eine **-cgetmethod** teilen.

-configuremethod *methodName*

Jede lokal definierte Option kann eine **-configuremethod** definieren; diese wird aufgerufen, wenn der Wert der Option mithilfe der Methoden **configure** oder **configurelist** festgelegt wird. Es obliegt der benannten Methode, den Wert der Option zu speichern; mit anderen Worten: Der Wert wird nur dann im Array **options()** gespeichert, wenn die Methode ihn dort ablegt.

Die benannte Methode muss zwei Argumente entgegennehmen: den Namen der Option und ihren neuen Wert. Beispielsweise entspricht dieser Code der standardmäßigen Behandlung von **configure** durch Snit (ist jedoch langsamer als diese):

```
option -font -configuremethod SetOption
method SetOption {option value} {
    set options($option) $value
}
```

Beliebig viele Optionen können sich eine einzelne **-configuremethod** teilen.

-validatemethod *methodName*

Jede lokal definierte Option kann eine **-validatemethod** definieren; diese wird aufgerufen, wenn der Wert der Option mithilfe der Methoden **configure** oder **configurelist** gesetzt wird, unmittelbar vor der Methode **-configuremethod** (sofern vorhanden). Es ist Aufgabe der benannten Methode, den neuen Wert der Option zu validieren und einen Fehler auszulösen, falls der Wert ungültig ist.

Die benannte Methode muss zwei Argumente entgegennehmen: den Namen der Option und deren neuen Wert. Dieser Code überprüft beispielsweise, ob der Wert von **-flag** ein gültiger boolescher Wert ist:

```
option -font -validatemethod CheckBoolean
method CheckBoolean {option value} {
    if {![string is boolean -strict $value]} {
        error "option $option must have a boolean value."
    }
}
```

Beliebig viele Optionen können sich eine einzelne **-validatemethod** teilen.

constructor *arglist body*

Die Konstruktordefinition legt den Code *body* fest, der bei der Erstellung einer neuen Instanz ausgeführt werden soll. Die *arglist* ist eine normale Tcl-Argumentliste und kann Standardargumente sowie das Argument **args** enthalten.

Wie bei Methoden werden die Argumente **type**, **self**, **selfns** und **win** implizit definiert, und alle Typ- und Instanzvariablen sind in seinem *body* automatisch sichtbar.

Wenn *definition* den Konstruktor nicht explizit definiert, definiert Snit implizit einen. Wenn der Typ mindestens eine Option deklariert (sei es lokal oder durch Weiterleitung), wird der Standardkonstruktor wie folgt definiert:

```
constructor {args} {
    $self configurelist $args
}
```

Für das Standardverhalten eines Tk-Widgets sollte die Argumentliste, wie dargestellt, aus dem einzelnen Namen **args** bestehen.

Wenn *definition* weder einen Konstruktor noch Optionen definiert, wird der Standardkonstruktor wie folgt definiert:

```
constructor {} {}
```

Wie bei Methoden kann der Konstruktor Befehle aus dem Namespace aufrufen, in dem der Typ definiert ist, ohne diese importieren zu müssen; wenn der Typname beispielsweise **::parentns::typename** lautet, dann kann der Konstruktor **::parentns::someproc** genauso aufrufen wie **someproc**. *Inkompatibilität mit Snit 1.x:* Dies funktioniert nicht in Snit 1.x, da es von **namespace path** abhängt, einem neuen Befehl in Tcl 8.5.

destructor *body*

Der Destruktor dient dazu, alle Aktionen zu programmieren, die bei der Zerstörung einer Instanz des Typs ausgeführt werden müssen: typischerweise die Zerstörung aller im Konstruktor erzeugten Elemente.

Der Destruktor nimmt keine expliziten Argumente entgegen; wie bei den Methoden werden die Argumente **type**, **self**, **selfns** und **win** implizit definiert, und alle Typ- und Instanzvariablen sind in seinem *body* automatisch sichtbar. Wie bei Methoden kann der Destruktor Befehle aus dem Namespace aufrufen, in dem der Typ definiert ist, ohne diese importieren zu müssen; wenn der Typname beispielsweise **::parentns::typename** lautet, dann kann der Destruktor **::parentns::someproc** genauso aufrufen wie **someproc**. *Inkompatibilität mit Snit 1.x:* Dies funktioniert nicht in Snit 1.x, da es von **namespace path** abhängt, einem neuen Befehl in Tcl 8.5.

proc *name args body*

Definiert eine neue Tcl-Prozedur im Namespace des Typs.

Die definierte Prozedur unterscheidet sich von einer normalen Tcl-Prozedur dadurch, dass alle Typvariablen automatisch sichtbar sind. Die Prozedur kann zudem auf Instanzvariablen zugreifen, sofern ihr **selfns** (mit genau diesem Namen) als eines ihrer Argumente übergeben wird.

Obwohl sie für Prozeduren nicht implizit definiert sind, sollten die Argumentnamen **type**, **self** und **win** vermieden werden.

Wie bei Methoden und Typmethoden können Prozeduren Befehle aus dem Namespace aufrufen, in dem der Typ definiert ist, ohne diese importieren zu müssen; wenn der Typname beispielsweise **::parentns::typename** lautet, dann kann die Prozedur **::parentns::someproc** genauso aufrufen wie **someproc**. *Inkompatibilität mit Snit 1.x:* Dies funktioniert nicht in Snit 1.x, da es von **namespace path** abhängt, einem neuen Befehl in Tcl 8.5.

delegate method *name* to *comp* ?as *target*?

Die Methode *name* wird an die Komponente *comp* weitergeleitet. Das heißt, wenn die Methode *name* für eine Instanz dieses Typs aufgerufen wird, werden die Methode und ihre Argumente stattdessen an den Befehl der angegebenen Komponente übergeben. Das heißt, die folgende Anweisung

```
    delegate method wag to tail
```

ist ungefähr gleichwertig zu dieser ausdrücklich definierten Methode:

```
    method wag {args} {
        uplevel $tail wag $args
    }
```

Wie bei Methoden kann *name* aus mehreren Tokens bestehen; in diesem Fall wird davon ausgegangen, dass das letzte Token des Namens der Name der Methode der Komponente ist.

Mit der optionalen Klausel **as** können Sie den Namen der weitergeleiteten Methode angeben und gegebenenfalls einige Argumente hinzufügen:

```
    delegate method wagtail to tail as "wag briskly"
```

Eine Methode kann nicht zugleich lokal definiert und weitergeleitet sein.

Hinweis: Alle Formen von **delegate method** können sowohl an Instanzkomponenten als auch an Typkomponenten weitergeleitet werden.

delegate method *name* ?to *comp*? using *pattern*

In dieser Form der Anweisung **delegate** wird die Klausel **using** verwendet, um die genaue Form des Befehls festzulegen, an den die Methode *name* weitergeleitet wird. In dieser Form ist die **to**-Klausel optional, da der gewählte Befehl möglicherweise keine bestimmte Komponente beinhaltet.

Der Wert der Klausel **using** ist eine Liste, die einen oder alle der folgenden Ersetzungscodes enthalten kann; diese Codes werden durch den beschriebenen Wert ersetzt, um das Präfix des weitergeleiteten Befehls zu bilden. Hinweis: Die folgenden beiden Anweisungen sind gleichwertig:

```
    delegate method wag to tail
    delegate method wag to tail using "%c %m"
```

Jedes Element der Liste wird zu einem einzelnen Element des weitergeleiteten Befehls – es wird niemals erneut als Zeichenkette geparst.

Ersetzungen:

%%
Dies wird durch ein einzelnes "%" ersetzt. Um also die Zeichenfolge "%c" als Argument an den Befehl zu übergeben, würden Sie "%%c" schreiben.

%c
Das wird durch den Befehl der genannten Komponente ersetzt.

%m
Dies wird durch das letzte Token der Methode *name* ersetzt; wenn die Methode *name* aus einem Token besteht, entspricht dies **%M**.

%M
Dies wird durch die Methode *name* ersetzt; wenn *name* aus mehreren Tokens besteht, werden diese durch Leerzeichen verbunden.

%j
An deren Stelle tritt die Methode *name*; wenn *name* aus mehreren Tokens besteht, werden diese durch Unterstriche ("_") verbunden.

%t
Das wird durch den voll qualifizierten Typnamen ersetzt.

%n
Das wird durch den Namen des privaten Namespace der Instanz ersetzt.

%s
Das wird durch den Namen des Instanzbefehls ersetzt.

%w
Dies wird durch den ursprünglichen Namen des Instanzbefehls ersetzt; bei Snit-Widgets und Widget-Adaptern ist dies der Name des Tk-Fensters. Er bleibt unverändert, auch wenn der Instanzbefehl umbenannt wird.

delegate method * ?to *comp*? ?using *pattern*? ?except *exceptions*?
Die Form **delegate method *** leitet alle unbekannten Methodennamen an die angegebene Komponente *comp* weiter. Mit der Klausel **except** kann eine Liste von Ausnahmen *exceptions* festgelegt werden, d. h. Namen von Methoden, die nicht auf diese Weise weitergeleitet werden. Die Klausel **using** ist wie oben angegeben definiert. In dieser Form muss die Anweisung entweder die Klausel **to**, die Klausel **using** oder beide enthalten.

Tatsächlich kann das "*" eine Liste aus zwei oder mehr Tokens sein, deren letztes Element "*" ist, wie im folgenden Beispiel:

```
    delegate method {tail *} to tail
```

Dadurch wird implizit die Methode **tail** definiert, deren Unterbefehle an die Komponente **tail** weitergeleitet werden.

delegate option *namespec* to *comp*

siehe unten ...

delegate option *namespec* to *comp* as *target*

siehe unten ...

delegate option * to *comp*

siehe unten ...

delegate option * to *comp* except *exceptions*

Definiert eine weitergeleitete Option; *namespec* ist wie bei der Anweisung **option** definiert. Wenn die Instanzmethode **configure**, **configurelist** oder **cget** zum Setzen oder Abrufen des Werts der Option verwendet wird, wird der entsprechende Befehl **configure** oder **cget** auf die Komponente angewendet, als ob die Option mit der folgenden **-configuremethod** und **-cgetmethod** definiert worden wäre:

```
method ConfigureMethod {option value} {
    $comp configure $option $value
}

method CgetMethod {option} {
    return [$comp cget $option]
}
```

Hinweis: Weitergeleitete Optionen erscheinen niemals im Array **options**.

Wenn die Klausel **as** angegeben ist, wird anstelle von *name* der Optionsname *target* verwendet.

Die Form **delegate option *** leitet alle unbekannten Optionen an die angegebene Komponente *comp* weiter. Mit der Klausel **except** kann eine Liste von Ausnahmen *exceptions* festgelegt werden, d. h. Namen von Optionen, die nicht auf diese Weise weitergeleitet werden.

Warnung: Optionen können nur an eine Komponente weitergeleitet werden, wenn diese die Instanzmethoden **configure** und **cget** unterstützt.

Eine Option kann nicht gleichzeitig lokal definiert und weitergeleitet werden. TBD: Hier fortsetzen.

component *comp* ?-public *method*? ?-inherit *flag*?

Deklariert explizit eine Komponente namens *comp* und definiert automatisch die Instanzvariable der Komponente.

Wenn die Option **-public** angegeben wird, wird die Komponente durch die Definition einer *method* öffentlich gemacht, deren Unterbefehle an die Komponente weitergeleitet werden. So entspricht beispielsweise die Angabe von **-public mycomp** Folgendem:

```
component mycomp
delegate method {mymethod *} to mycomp
```

Wenn die Option **-inherit** festgelegt wird, muss *flag* ein boolescher Wert sein; ist *flag* wahr, werden alle unbekannten Methoden und Optionen an diese Komponente weitergeleitet. Der Name **-inherit** impliziert, dass Instanzen dieses neuen Typs in gewisser Weise die

Methoden und Optionen der Komponente erben. Das heißt, **-inherit yes** entspricht:

```
component mycomp
delegate option * to mycomp
delegate method * to mycomp
```

delegate typemethod *name* to *comp* ?as *target*?

Leitet die Typmethode *name* an die Typkomponente *comp* weiter. Das heißt, wenn die Typmethode *name* für diesen Typ aufgerufen wird, werden die Typmethode und ihre Argumente stattdessen an den Befehl der angegebenen Typkomponente übergeben. Das heißt, die folgende Anweisung

```
delegate typemethod lostdogs to pound
```

ist ungefähr gleichwertig zu dieser ausdrücklich definierten Methode:

```
typemethod lostdogs {args} {
    uplevel $pound lostdogs $args
}
```

Wie bei Typmethoden kann *name* aus mehreren Tokens bestehen; in diesem Fall wird davon ausgegangen, dass das letzte Token des Namens der Name der Methode der Komponente ist.

Mit der optionalen Klausel **as** können Sie den Namen der weitergeleiteten Methode angeben und gegebenenfalls einige Argumente hinzufügen:

```
delegate typemethod lostdogs to pound as "get lostdogs"
```

Eine Typmethode kann nicht gleichzeitig lokal definiert und weitergeleitet werden.

delegate typemethod *name* ?to *comp*? using *pattern*

In dieser Form der Anweisung **delegate** wird die Klausel **using** verwendet, um die genaue Form des Befehls festzulegen, an den die Typmethode *name* weitergeleitet wird. In dieser Form ist die **to**-Klausel optional, da der gewählte Befehl möglicherweise keine bestimmte Typkomponente beinhaltet.

Der Wert der Klausel **using** ist eine Liste, die einen oder alle der folgenden Ersetzungscodes enthalten kann; diese Codes werden durch den beschriebenen Wert ersetzt, um das Präfix des weitergeleiteten Befehls zu bilden. Hinweis: Die folgenden beiden Anweisungen sind gleichbedeutend:

```
delegate typemethod lostdogs to pound
delegate typemethod lostdogs to pound using "%c %m"
```

Jedes Element der Liste wird zu einem einzelnen Element des weitergeleiteten Befehls – es wird niemals erneut als Zeichenkette geparst.

Ersetzungen:

%%

Dies wird durch ein einzelnes "%" ersetzt. Um also die Zeichenfolge "%c" als Argument

an den Befehl zu übergeben, würden Sie "%%c" schreiben.

%c
Dies wird durch den Befehl der benannten Typkomponente ersetzt.

%m
Dies wird durch das letzte Token der Typmethode *name* ersetzt; wenn die Typmethode *name* aus einem Token besteht, entspricht dies **%M**.

%M
An deren Stelle tritt die Typmethode *name*; wenn *name* aus mehreren Tokens besteht, werden diese durch Leerzeichen verbunden.

%j
An deren Stelle tritt die Typmethode *name*; wenn *name* aus mehreren Tokens besteht, werden diese durch Unterstriche ("_") verbunden.

%t
Das wird durch den voll qualifizierten Typnamen ersetzt.

delegate typemethod * ?to *comp*? ?using *pattern*? ?except *exceptions*?
Die Form **delegate typemethod *** leitet alle unbekannten Typmethodennamen an die angegebene Typkomponente weiter. Mit der Klausel **except** kann eine Liste von Ausnahmen *exceptions* festgelegt werden, d. h. Namen von Typmethoden, die nicht auf diese Weise weitergeleitet werden. Die Klausel **using** ist wie oben angegeben definiert. In dieser Form muss die Anweisung entweder die Klausel **to**, die Klausel **using** oder beide enthalten.

Hinweis: Standardmäßig interpretiert Snit **$type foo** – wobei **foo** keine definierte Typmethode ist – als gleichbedeutend mit **$type create foo**, wobei **foo** der Name einer neuen Instanz des Typs ist. Wenn Sie **delegate typemethod *** verwenden, muss die Typmethode **create** stets explizit verwendet werden.

Das "*" kann eine Liste aus zwei oder mehr Tokens sein, deren letztes Element "*" ist, wie im folgenden Beispiel:

```
    delegate typemethod {tail *} to tail
```

Hiermit wird implizit die Typmethode **tail** definiert, deren Unterbefehle an die Typkomponente **tail** weitergeleitet werden.

typecomponent *comp* ?-public *typemethod*? ?-inherit *flag*?
Deklariert explizit eine Typkomponente namens *comp* und definiert automatisch die Typvariable der Komponente. Eine Typkomponente ist ein beliebiger Befehl, an den Typmethoden und Instanzmethoden weitergeleitet werden können; der Name des Befehls wird in einer Typvariablen gespeichert.

Wenn die Option **-public** angegeben wird, wird die Typkomponente durch die Definition einer *typemethod* öffentlich gemacht, deren Unterbefehle an die Typkomponente weitergeleitet werden; die Angabe von **-public mytypemethod** entspricht beispielsweise Folgendem:

```
    typecomponent mycomp
    delegate typemethod {mytypemethod *} to mycomp
```

Wenn die Option **-inherit** festgelegt wird, muss *flag* ein boolescher Wert sein; ist *flag* wahr, werden alle unbekannten Typmethoden an diese Typkomponente weitergeleitet. (Siehe den Hinweis zu "delegate typemethod *" weiter oben.) Der Name **-inherit** impliziert, dass dieser Typ in gewisser Weise das Verhalten der Typkomponente erbt. Das heißt, **-inherit yes** entspricht:

```
typecomponent mycomp
delegate typemethod * to mycomp
```

pragma ?*options...*?

Die Anweisung **pragma** ermöglicht die Steuerung der Art und Weise, wie Snit einen Typ generiert. Sie akzeptiert die folgenden Optionen; in jedem Fall muss *flag* ein von Tcl erkannter boolescher Wert sein, z. B. **0**, **1**, **yes**, **no** usw.

Indem Sie die Pragmas **-hastypeinfo**, **-hastypedestroy** und **-hasinstances** auf "false" setzen und entsprechende Typmethoden definieren, können Sie einen Ensemble-Befehl ohne jegliches unerwünschtes Verhalten erstellen.

-canreplace *flag*

Wenn der Wert "false" (Standard) eingestellt ist, erstellt Snit keine Instanz eines **snit::type**, deren Name mit dem eines bereits vorhandenen Befehls übereinstimmt; dadurch werden subtile Fehler vermieden. Durch Setzen dieses Pragmas auf "true" wird das Verhalten von Snit V0.93 und früheren Versionen wiederhergestellt.

-hastypeinfo *flag*

Ist dieser Wert auf "true" gesetzt (Standardeinstellung), verfügt der generierte Typ über eine Typmethode namens **info**, die zur Typintrospektion verwendet wird; die Typmethode **info** wird im Folgenden beschrieben. Ist dieser Wert auf "false" gesetzt, ist dies nicht der Fall.

-hastypedestroy *flag*

Ist dieser Wert auf "true" gesetzt (Standardeinstellung), verfügt der generierte Typ über eine Typmethode namens **destroy**, die dazu dient, den Typ und alle seine Instanzen zu löschen. Die Typmethode **destroy** wird im Folgenden beschrieben. Wenn dieser Wert auf "false" gesetzt ist, ist dies nicht der Fall.

-hastypemethods *flag*

Ist dieser Wert auf "true" gesetzt (Standardeinstellung), verfügt der Typbefehl des generierten Typs wie üblich über Unterbefehle (Typmethoden). Wenn der Wert "false" ist, dient der Typbefehl ausschließlich dazu, Instanzen des Typs zu erstellen; das erste Argument ist der Name der Instanz.

Dieses Pragma und **-hasinstances** können nicht beide auf "false" gesetzt werden.

-hasinstances *flag*

Wenn dieser Wert auf "true" gesetzt (Standardeinstellung) ist, verfügt der generierte Typ über eine Typmethode namens **create**, die zum Erstellen von Instanzen des Typs sowie für eine Reihe von instanzbezogenen Funktionen verwendet wird. Wenn der Wert auf "false" gesetzt ist, ist dies nicht der Fall.

Dieses Pragma und **-hastypemethods** können nicht beide auf "false" gesetzt werden.

-hasinfo *flag*

Ist dieser Wert auf "true" gesetzt (Standardeinstellung), verfügen Instanzen des generierten Typs über eine Instanzmethode namens **info**, die zur Instanzintrospektion dient; die Methode **info** wird weiter unten beschrieben. Wenn der Wert auf "false" gesetzt ist, ist dies nicht der Fall.

-simpledispatch *flag*

Dieses Pragma soll einfache, häufig verwendete abstrakte Datentypen (z. B. Stacks und Warteschlangen) effizienter machen.

Ist der Wert "false" (Standard), werden Instanzmethoden wie gewohnt aufgerufen. Ist er "true", wird stattdessen ein schnelleres Aufrufschema verwendet. Diese Geschwindigkeitssteigerung hat jedoch ihren Preis; bei **-simpledispatch yes** gelten folgende Einschränkungen:
- Methoden können nicht weitergeleitet werden.
- **uplevel** und **upvar** funktionieren nicht wie erwartet: Der Geltungsbereich des Aufrufers liegt zwei Ebenen höher statt einer.
- Die Methoden zur Behandlung von Optionen (**cget**, **configure** und **configurelist**) sind geringfügig langsamer.

expose *comp*

siehe unten ...

expose *comp* as *method*

Veraltet. Um die Komponente *comp* öffentlich zugänglich zu machen, verwenden Sie die Option **-public** von **component**.

onconfigure *name arglist body*

Veraltet. Definieren Sie stattdessen die Option **-configuremethod** von **option**.

Ab Version 0.95 werden die folgenden Definitionen,

```
option -myoption
onconfigure -myoption {value} {
    # Code to save the option's value
}
```

wie folgt umgesetzt:

```
option -myoption -configuremethod _configure-myoption
method _configure-myoption {_option value} {
    # Code to save the option's value
}
```

oncget *name body*

Veraltet. Definieren Sie stattdessen die Option **-cgetmethod** von **option**.

Ab Version 0.95 werden die folgenden Definitionen,

```
option -myoption
oncget -myoption {
    # Code to return the option's value
}
```

wie folgt umgesetzt:

```
option -myoption -cgetmethod _cget-myoption
method _cget-myoption {_option} {
    # Code to return the option's value
}
```

snit::widget *name definition*

Dieser Befehl definiert einen Snit-Megawidget-Typ mit dem angegebenen Namen *name*. *definition* wird wie bei **snit::type** angegeben. Ein **snit::widget** unterscheidet sich in folgenden Punkten von einem **snit::type**:

- Jede Instanz von **snit::widget** verfügt über eine automatisch erstellte Komponente namens **hull**, bei der es sich in der Regel um ein Tk-Frame-Widget handelt. Andere Widgets, die als Teil des Megawidgets erstellt werden, werden innerhalb dieses Widgets angelegt.

  Die Hull-Komponente wird zunächst mit dem angeforderten Widget-Namen erstellt; anschließend vollbringt Snit ein kleines Wunder, indem es die Hull-Komponente umbenennt und an ihrer Stelle einen eigenen Instanzbefehl einfügt. Der neue Name der Hull-Komponente wird in einer Instanzvariablen namens **hull** gespeichert.
- Der Name einer Instanz muss ein gültiger Tk-Fenstername sein, und das übergeordnete Fenster muss vorhanden sein.

Eine **snit::widget**-Definition kann alle Anweisungen enthalten, die in einer **snit::type** -Definition zulässig sind, und kann darüber hinaus Folgendes umfassen:

widgetclass *name*

Legt die Widget-Klasse des **snit::widget** auf *name* fest und überschreibt damit die Standardeinstellung. Weitere Informationen finden Sie im Abschnitt Tk-Optionsdatenbank.

hulltype *type*

Legt fest, welche Art von Widget als Hull des **snit::widget** verwendet wird. *type* kann **frame** (Standard), **toplevel** oder **labelframe** sein; die qualifizierten Entsprechungen davon, **tk::frame**, **tk::toplevel** und **tk::labelframe**; oder, wenn verfügbar, die entsprechenden Tile-Widgets: **ttk::frame**, **ttk::toplevel** und **ttk::labelframe**. In der Praxis kann jedes Widget, das die Option **-class** unterstützt, als Hull-Widget verwendet werden, indem sein Name mit **lappend** an die Variable **snit::hulltypes** angehängt wird.

snit::widgetadaptor *name definition*

Dieser Befehl definiert einen Snit-Megawidget-Typ mit dem angegebenen Namen. Er unterscheidet sich von **snit::widget** dadurch, dass die Komponente **hull** der Instanz nicht automatisch erstellt wird, sondern im Konstruktor angelegt und mithilfe des Befehls **installhull** installiert wird. Sobald der Hull installiert ist, wird sein Instanzbefehl wie bei normalen **snit::widget**-Instanzen umbenannt und ersetzt. Der ursprüngliche Befehl ist wiederum über die Instanzvariable **hull** zugänglich.

Hinweis: Es ist im Allgemeinen nicht möglich, die *Widget-Klasse* des Hull-Widgets eines **snit::widgetadaptor** zu ändern.

Informationen darüber, wie **snit::widgetadaptor**s mit der Optionsdatenbank interagieren,

finden Sie im Abschnitt Tk-Optionsdatenbank.

snit::typemethod *type name arglist body*
Definiert eine neue Typmethode (oder definiert eine bestehende Typmethode neu) für einen bereits vorhandenen Typ *type*.

snit::method *type name arglist body*
Definiert eine neue Instanzmethode (oder definiert eine vorhandene Instanzmethode neu) für einen bereits vorhandenen Typ *type*. Hinweis: Weitergeleitete Instanzmethoden können nicht neu definiert werden.

snit::macro *name arglist body*
Definiert ein Snit-Makro mit den angegebenen Werten *name*, *arglist* und *body*. Makros dienen dazu, neue Anweisungen für Typ- und Widget-Definitionen auf der Grundlage der in dieser Manpage definierten Anweisungen zu definieren.

Ein Makro ist lediglich eine Tcl-Prozedur, die im Slave-Interpreter definiert ist, der zum Kompilieren von Typ- und Widget-Definitionen verwendet wird. Somit haben Makros Zugriff auf alle Anweisungen zur Typ- und Widget-Definition. Weitere Einzelheiten finden Sie unter Makros und Metaprogrammierung.

Der Makroname *name* darf nicht mit einem Standard-Tcl-Befehl oder einer Snit-Typ- bzw. Widget-Definitionsanweisung übereinstimmen; so können Sie beispielsweise die Anweisungen **method** oder **delegate** nicht neu definieren, noch die Standard-Befehle **set**, **list** oder **string** .

snit::compile *which type body*
Snit definiert einen Typ, ein Widget oder einen Widgetadaptor, indem es die Definition in ein Tcl-Skript "kompiliert"; dieses Skript wird anschließend im Tcl-Interpreter ausgewertet, wodurch der neue Typ tatsächlich definiert wird.

Dieser Befehl macht den "Compiler" zugänglich. Für eine Definition *body* des benannten Typs *type*, wobei *which* **type**, **widget** oder **widgetadaptor** ist, gibt **snit::compile** eine Liste mit zwei Elementen zurück. Das erste Element ist der vollqualifizierte Typname; das zweite Element ist das Skript für die Definitionen.

snit::compile ist nützlich, wenn der von Snit generierte Code zusätzlich bearbeitet werden muss – etwa wenn er instrumentiert oder durch den TclDevKit-Compiler verarbeitet werden soll. Darüber hinaus könnte das zurückgegebene Skript in einer ".tcl"-Datei gespeichert und zur Definition des Typs als Teil einer Anwendung oder Bibliothek verwendet werden, wodurch der Kompilierungsaufwand beim Start der Anwendung eingespart wird. Hinweis: Zur Laufzeit muss dieselbe Version von Snit verwendet werden wie zur Kompilierungszeit.

Der Typbefehl

Eine Typ- oder Widget-Definition erzeugt einen Typbefehl, der zum Erstellen von Instanzen des Typs verwendet wird. Der Typbefehl hat folgende Form:

$type *typemethod args...*
Die *typemethod* kann eine beliebige der Standard-Typmethoden (z. B. **create**) oder eine beliebige in der Typdefinition definierte Typmethode sein. Die nachfolgenden *args* hängen von der jeweils gewählten *typemethod* ab.

Der Typbefehl wird meist zum Erstellen neuer Instanzen des Typs verwendet; daher wird die Methode **create** angenommen, wenn das erste Argument des Typbefehls keine gültige Typmethode angibt, es sei denn, die Typdefinition enthält **delegate typemethod *** oder das Pragma **-hasinstances** auf "false" gesetzt ist.

Wenn das Pragma **-hastypemethods** zudem auf "false" gesetzt ist, können Snit-Typbefehle auch ganz ohne Argumente aufgerufen werden; in diesem Fall erstellt der Typbefehl eine Instanz mit einem automatisch generierten Namen. Mit anderen Worten: Sofern das Pragma **-hastypemethods** auf "false" gesetzt ist und der Typ Instanzen besitzt, sind die folgenden Befehle gleichbedeutend:

```
snit::type dog { ... }

set mydog [dog create %AUTO%]
set mydog [dog %AUTO%]
set mydog [dog]
```

Bei Snit-Widgets funktioniert dies aus offensichtlichen Gründen nicht.

Inkompatibilität mit Snit 1.x: In Snit 1.x ist das oben beschriebene Verhalten unabhängig davon verfügbar, ob **-hastypemethods** auf "true" (Standard) oder "false" gesetzt ist.

Standard-Typmethoden

Zusätzlich zu den Typmethoden in der Definition des Typs verfügen alle Typ- und Widget-Befehle in der Regel über mindestens die folgenden Unterbefehle:

$type create *name* ?*option value* ...?

Erzeugt eine neue Instanz des Typs mit dem angegebenen *name* und ruft den Konstruktor des Typs auf.

Ist *name* bei **snit::type**s kein vollqualifizierter Befehlsname, gilt er als Name im Namespace, in dem der Aufruf von **snit::type** steht. Die Methode gibt den vollqualifizierten Instanznamen zurück.

Bei **snit::widget**s und **snit::widgetadaptor**s muss *name* ein gültiger Widget-Name sein; die Methode gibt den Widget-Namen zurück.

Solange *name* nicht mit dem Namen einer definierten Typmethode kollidiert, kann das Schlüsselwort **create** entfallen, es sei denn, die Typdefinition enthält **delegate typemethod *** oder das Pragma **-hasinstances** ist auf false gesetzt.

Enthält *name* die Zeichenkette **%AUTO%**, wird sie durch die Zeichenkette **$type$counter** ersetzt; dabei ist **$type** der Typname und **$counter** ein Zähler, der bei jeder Verwendung von **%AUTO%** für diesen Typ hochgezählt wird.

Standardmäßig sind die Argumente nach *name* eine Liste von *option*-Namen und ihren *value*s; der Konstruktor eines Typs kann jedoch eine andere Argumentliste festlegen.

Seit Snit V0.95 löst **create** einen Fehler aus, wenn *name* gleich dem Namen eines

bestehenden Befehls ist; für **snit::widget**s und **snit::widgetadaptor**s galt das schon immer. Das frühere Verhalten lässt sich mit dem Pragma **-canreplace** wiederherstellen.

$type info typevars ?*pattern*?
Gibt eine Liste der Typvariablen des Typs zurück (mit Ausnahme der Snit-internen Variablen); alle Variablennamen sind vollständig qualifiziert.

Wenn *pattern* angegeben wird, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die mit dem Muster übereinstimmen.

$type info typemethods ?*pattern*?
Gibt eine Liste der Namen der Typmethoden des Typs zurück. Wenn der Typ über hierarchische Typmethoden verfügt – unabhängig davon, ob diese lokal definiert oder weitergeleitet werden –, wird jeweils nur das erste Wort in die Liste aufgenommen.

Wenn die Typdefinition **delegate typemethod *** enthält, umfasst die Liste nur die Namen jener implizit weitergeleiteten Typmethoden, die mindestens einmal aufgerufen wurden und sich noch im Typmethoden-Cache befinden.

Wenn *pattern* angegeben wird, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die diesem Muster entsprechen.

$type info args *method*
Gibt eine Liste zurück, die die Namen der Argumente der Typmethode *method* in der Reihenfolge ihrer Angabe enthält. Diese Methode kann nicht auf weitergeleitete Typmethoden angewendet werden.

$type info body *method*
Gibt den Rumpf der Typmethode *method* zurück. Diese Methode kann nicht auf weitergeleitete Typmethoden angewendet werden.

$type info default *method aname varname*
Gibt einen booleschen Wert zurück, der angibt, ob das Argument *aname* der Typmethode *method* einen Standardwert hat (**true**) oder nicht (**false**). Wenn das Argument über einen Standardwert verfügt, wird dessen Wert in der Variablen *varname* abgelegt.

$type info instances ?*pattern*?
Gibt eine Liste der Instanzen des Typs zurück. Bei **snit::type** handelt es sich um eine Liste vollständig qualifizierter Instanznamen; bei **snit::widget** um eine Liste von Tk-Widget-Namen.

Wenn *pattern* angegeben wird, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die mit dem Muster übereinstimmen.

Inkompatibilität mit Snit 1.x: In Snit 1.x sind die vollständigen, aus mehreren Wörtern bestehenden Namen der hierarchischen Typmethoden im Rückgabewert enthalten.

$type destroy
Löscht die Instanzen des Typs, den Namespace des Typs und den Typbefehl selbst.

Der Instanzbefehl

Die Typmethode **create** eines Snit-Typs oder -Widgets erstellt Objekte dieses Typs; jedes Objekt

verfügt über einen eindeutigen Namen, der zugleich ein Tcl-Befehl ist. Dieser Befehl dient dem Zugriff auf die Methoden und Daten des Objekts und hat folgende Form:

$object *method args...*
: *method* kann eine beliebige der Standard-Instanzmethoden oder eine in der Typdefinition definierte Instanzmethode sein. Die nachfolgenden *args* hängen von der jeweils gewählten *method* ab.

Standard-Instanzmethoden

Zusätzlich zu etwaigen weitergeleiteten oder lokal definierten Instanzmethoden in der Typdefinition verfügen alle Snit-Objekte mindestens über die folgenden Unterbefehle:

$object configure ?*option*? ?*value*? ...
: Weist einer oder mehreren Optionen neue Werte zu. Wird die Methode mit einem Argument – einem Optionsnamen *option* – aufgerufen, gibt sie eine Liste zurück, die die Option beschreibt, wie es auch bei Tk-Widgets der Fall ist; wird sie ohne Argumente aufgerufen, gibt sie eine Liste von Listen zurück, die alle Optionen beschreiben, wie es auch bei Tk-Widgets der Fall ist.

  Warnung: Diese Informationen stehen für weitergeleitete Optionen nur dann zur Verfügung, wenn die Komponente, an die sie weitergeleitet werden, über eine Methode **configure** verfügt, die genau diese Art von Informationen zurückgibt.

  Hinweis: Snit definiert diese Methode nur, wenn der Typ über mindestens eine Option verfügt.

$object configurelist *optionlist*
: Ähnlich wie **configure**, nimmt jedoch ein Argument entgegen: eine Liste von Optionen und deren Werte. Sie ist vor allem im Konstruktor des Typs nützlich, kann jedoch überall verwendet werden.

  Hinweis: Snit definiert diese Methode nur, wenn der Typ mindestens eine Option aufweist.

$object cget *option*
: Gibt den Wert der Option zurück.

  Hinweis: Snit definiert diese Methode nur, wenn der Typ über mindestens eine Option verfügt.

$object destroy
: Löscht das Objekt, ruft dabei den **destructor** auf und gibt den gesamten zugehörigen Speicher frei.

  Hinweis: Die Methode **destroy** ist für **snit::widget**- oder **snit::widgetadaptor**-Objekte nicht definiert; deren Instanzen werden durch den Aufruf des Befehls **destroy** von **Tk** zerstört, genau wie normale Widgets.

$object info type
: Gibt den Typ der Instanz zurück.

$object info vars ?*pattern*?
: Gibt eine Liste der Instanzvariablen des Objekts zurück (mit Ausnahme der Snit-internen Variablen). Die Namen sind vollständig qualifiziert.

Wird *pattern* angegeben, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die mit dem Muster übereinstimmen.

$object info typevars ?*pattern*?

Gibt eine Liste der Typvariablen des Objekttyps zurück (mit Ausnahme der Snit-internen Variablen). Die Namen sind vollständig qualifiziert.

Wird *pattern* angegeben, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die mit dem Muster übereinstimmen.

$object info typemethods ?*pattern*?

Gibt eine Liste der Namen der Typmethoden des Typs zurück. Wenn der Typ über hierarchische Typmethoden verfügt – unabhängig davon, ob diese lokal definiert oder weitergeleitet werden –, wird jeweils nur das erste Wort in die Liste aufgenommen.

Wenn die Typdefinition **delegate typemethod *** enthält, umfasst die Liste nur die Namen jener implizit weitergeleiteten Typmethoden, die mindestens einmal aufgerufen wurden und sich noch im Typmethoden-Cache befinden.

Wenn *pattern* angegeben wird, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die mit dem Muster übereinstimmen.

Inkompatibilität mit Snit 1.x: In Snit 1.x sind die vollständigen, aus mehreren Wörtern bestehenden Namen hierarchischer Typmethoden im Rückgabewert enthalten.

$object info options ?*pattern*?

Gibt eine Liste der Namen der Optionen des Objekts zurück. Diese umfasst stets lokale Optionen sowie explizit weitergeleitete Optionen. Wenn unbekannte Optionen weitergeleitet werden und die Komponente, an die sie weitergeleitet werden, wie Tk-Widgets auf **$object configure** reagiert, so umfasst das Ergebnis alle möglichen unbekannten Optionen, die an die Komponente weitergeleitet werden können.

Wenn *pattern* angegeben wird, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die mit dem Muster übereinstimmen.

Beachten Sie, dass der Wert für verschiedene Instanzen desselben Typs unterschiedlich sein kann, wenn die Objekttypen der Komponenten von Instanz zu Instanz variieren können.

$object info methods ?*pattern*?

Gibt eine Liste der Namen der Methoden der Instanz zurück. Wenn der Typ über hierarchische Methoden verfügt – unabhängig davon, ob diese lokal definiert oder weitergeleitet werden –, wird jeweils nur das erste Wort jeder Methode in die Liste aufgenommen.

Wenn die Typdefinition **delegate method *** enthält, umfasst die Liste nur die Namen jener implizit weitergeleiteten Methoden, die mindestens einmal aufgerufen wurden und sich noch im Methodencache befinden.

Wenn *pattern* angegeben wird, wird es als **string match**-Muster verwendet; es werden nur Namen zurückgegeben, die mit dem Muster übereinstimmen.

Inkompatibilität mit Snit 1.x: In Snit 1.x sind die vollständigen, aus mehreren Wörtern bestehenden Namen hierarchischer Methoden im Rückgabewert enthalten.

$object info args *method*
Gibt eine Liste zurück, die die Namen der Argumente der Methode *method* der Instanz in der Reihenfolge ihrer Angabe enthält. Diese Methode kann nicht auf weitergeleitete Methoden angewendet werden.

$object info body *method*
Gibt den Rumpf der Methode *method* der Instanz zurück. Diese Methode kann nicht auf weitergeleitete Methoden angewendet werden.

$object info default *method aname varname*
Gibt einen booleschen Wert zurück, der angibt, ob das Argument *aname* der Methode *method* der Instanz einen Standardwert hat (**true**) oder nicht (**false**). Wenn das Argument über einen Standardwert verfügt, wird dessen Wert in der Variablen *varname* abgelegt.

Befehle zur Verwendung im Objektcode

Snit definiert die folgenden Befehle zur Verwendung in Ihrem Objektcode: Das heißt, zur Verwendung in Typmethoden, Instanzmethoden, Konstruktoren, Destruktoren, onconfigure-Handlern, oncget-Handlern und Prozeduren. Sie befinden sich nicht im Namespace ::snit::; stattdessen werden sie zusammen mit dem Typ erstellt und können ohne Qualifizierung verwendet werden.

mymethod *name* ?*args...*?
Der Befehl **mymethod** dient zum Formatieren von Befehlen, die als Rückruffunktion an andere Objekte übergeben werden sollen. Er gibt einen Befehl zurück, der bei seiner Ausführung die Methode *name* mit den angegebenen Argumenten sowie natürlich allen vom Aufrufer hinzugefügten Argumenten aufruft. Mit anderen Worten: Beide der folgenden Befehle bewirken, dass die Methode **dosomething** des Objekts aufgerufen wird, wenn die Schaltfläche **$button** gedrückt wird:

```
    $button configure -command [list $self dosomething myargument]

    $button configure -command [mymethod dosomething myargument]
```

Der wesentliche Unterschied zwischen den beiden besteht darin, dass die letztere Form auch dann noch funktioniert, wenn der Befehl des Objekts umbenannt wird.

mytypemethod *name* ?*args...*?
Der Befehl **mytypemethod** dient zum Formatieren von Befehlen, die als Rückruffunktion an andere Objekte übergeben werden sollen. Er gibt einen Befehl zurück, der bei seiner Ausführung die Typmethode *name* mit den angegebenen Argumenten aufruft – natürlich zusätzlich zu allen vom Aufrufer hinzugefügten Argumenten. Mit anderen Worten: Beide der folgenden Befehle bewirken, dass die Typmethode **dosomething** des Objekts aufgerufen wird, wenn **$button** gedrückt wird:

```
    $button configure -command [list $type dosomething myargument]

    $button configure -command [mytypemethod dosomething myargument]
```

Typbefehle können nicht umbenannt werden, sodass es in der Praxis kaum einen Unterschied zwischen den beiden Formen gibt. **mytypemethod** wird aus Gründen der Parallelität zu **mymethod** bereitgestellt.

myproc *name* ?*args...*?
Der Befehl **myproc** dient zum Formatieren von Befehlen, die als Rückruffunktion an andere Objekte übergeben werden sollen. Er gibt einen Befehl zurück, der bei seiner Ausführung die Typprozedur *name* mit den angegebenen Argumenten aufruft – natürlich zusätzlich zu allen vom Aufrufer hinzugefügten Argumenten. Mit anderen Worten: Beide der folgenden Befehle bewirken, dass die Prozedur **dosomething** des Objekts aufgerufen wird, wenn **$button** gedrückt wird:

```
    $button configure -command [list ${type}::dosomething myargument]

    $button configure -command [myproc dosomething myargument]
```

myvar *name*
Gibt bei Angabe eines Namens einer Instanzvariablen den vollqualifizierten Namen zurück. Verwenden Sie diesen Befehl, wenn Sie die Variable an ein anderes Objekt übergeben, z. B. als **-textvariable** an ein Tk-Label-Widget.

mytypevar *name*
Gibt bei Angabe eines Namens einer Typvariablen den vollständig qualifizierten Namen zurück. Verwenden Sie diesen Befehl, wenn Sie die Variable an ein anderes Objekt übergeben, z. B. als **-textvariable** an ein Tk-Label-Widget.

from *argvName option* ?*defvalue*?
Der Befehl **from** entnimmt einen Optionswert aus einer Liste von Optionen und deren Werten, wie sie beispielsweise an den **constructor** eines Typs übergeben wird. *argvName* muss der Name einer Variablen sein, die eine solche Liste enthält; *option* ist der Name der jeweiligen Option.

from sucht in der Optionsliste nach *option*. Wird sie gefunden, werden sie und ihr Wert aus der Liste entfernt, und der Wert wird zurückgegeben. Wenn *option* nicht in der Liste vorkommt, wird *defvalue* zurückgegeben. Handelt es sich um eine lokal definierte Option und ist *defvalue* nicht angegeben, wird stattdessen der in der Typdefinition festgelegte Standardwert der Option zurückgegeben.

install *compName* using *objType objName args...*
Erstellt ein neues Objekt vom Typ *objType* mit dem Namen *objName* und installiert es als Komponente *compName*, wie unter Komponenten und Weiterleitung beschrieben. Alle zusätzlichen *args...* werden zusammen mit dem Namen an den Befehl *objType* übergeben. Handelt es sich hierbei um einen **snit::type**, sind die folgenden beiden Befehle gleichbedeutend:

```
    install myComp using myObjType $self.myComp args...

    set myComp [myObjType $self.myComp args...]
```

Hinweis: Unabhängig davon, welche Methode verwendet wird, muss *compName* weiterhin in der Typdefinition unter Verwendung von **component** deklariert werden oder in mindestens

einer **delegate**-Anweisung referenziert werden.

Handelt es sich um ein **snit::widget** oder einen **snit::widgetadaptor** und wurden Optionen an die Komponente *compName* weitergeleitet, so erhalten diese Optionen Standardwerte aus der Tk-Optionsdatenbank. Beachten Sie, dass es keine Rolle spielt, ob es sich bei der zu installierenden Komponente um ein Widget handelt oder nicht. Weitere Informationen finden Sie im Abschnitt Tk-Optionsdatenbank.

install kann nicht zur Installation von Typkomponenten verwendet werden; weisen Sie stattdessen der Variablen der Typkomponente einfach den Befehlsnamen der Typkomponente zu.

installhull using *widgetType args...*
siehe unten ...

installhull *name*
Der Konstruktor eines **snit::widgetadaptor** muss ein Widget erstellen, das als Hull-Komponente des Objekts dient; das Widget wird mithilfe dieses Befehls als Hull-Komponente installiert. Beachten Sie, dass der Name des installierten Widgets **$win** lauten muss. Dieser Befehl hat zwei Formen.

Die erste Form legt den *widgetType* und die *args...* (d. h. die fest codierte Liste der Optionen) fest, die bei der Erstellung des Hulls verwendet werden sollen. In dieser Form erstellt **installhull** das Hull-Widget und initialisiert alle an den Hull weitergeleiteten Optionen aus der Tk-Optionsdatenbank.

In der zweiten Form wurde das Hull-Widget bereits erstellt; beachten Sie, dass dessen Name "$win" lauten muss. In diesem Fall wird die Tk-Optionsdatenbank *nicht* nach Optionen abgefragt, die an den Hull weitergeleitet wurden. Die längere Form wird bevorzugt; die kürzere Form ermöglicht es dem Programmierer jedoch, ein an anderer Stelle erstelltes Widget anzupassen, was mitunter nützlich ist. Beispielsweise kann sie verwendet werden, um ein "page"-Widget anzupassen, das von einem **BWidgets**-Notebook mit Registerkarten oder einem Seitenmanager-Widget erstellt wurde.

Weitere Informationen zu **snit::widgetadaptor**s und der Tk-Optionsdatenbank finden Sie unter Tk-Optionsdatenbank.

variable *name*
Normalerweise werden Instanzvariablen zusammen mit den Optionen, Methoden usw. in der Typdefinition definiert; solche Instanzvariablen sind automatisch in sämtlichem Instanzcode (z. B. in Methodenkörpern) sichtbar. Instanzcode kann jedoch den Befehl **variable** verwenden, um Instanzvariablen zu deklarieren, die nicht in der Typdefinition erscheinen, und um Variablen aus anderen Namespaces auf die übliche Weise in den Gültigkeitsbereich zu bringen.

Im Allgemeinen ist es am übersichtlichsten, alle Instanzvariablen in der Typdefinition zu definieren und auf deren Deklaration in Methoden usw. zu verzichten.

Hinweis: Es handelt sich hierbei um eine instanzspezifische Version des Standard-Tcl-Befehls **::variable**.

typevariable *name*
Normalerweise werden Typvariablen zusammen mit den Instanzvariablen in der Typdefinition definiert; solche Typvariablen sind automatisch im gesamten Code des Typs sichtbar. Typmethoden, Instanzmethoden und dergleichen können jedoch mit **typevariable** Typvariablen deklarieren, die nicht in der Typdefinition erscheinen.

Im Allgemeinen ist es am übersichtlichsten, alle Typvariablen in der Typdefinition zu deklarieren und auf deren Deklaration in Methoden, Typmethoden usw. zu verzichten.

varname *name*
Veraltet. Verwenden Sie stattdessen **myvar**.

Gibt bei Angabe eines Namens einer Instanzvariablen den vollqualifizierten Namen zurück. Verwenden Sie diesen Befehl, wenn Sie die Variable an ein anderes Objekt übergeben, z. B. als **-textvariable** an ein Tk-Label-Widget.

typevarname *name*
Veraltet. Verwenden Sie stattdessen **mytypevar**.

Gibt bei Angabe eines Typvariablennamens den vollqualifizierten Namen zurück. Verwenden Sie diesen Befehl, wenn Sie die Typvariable an ein anderes Objekt übergeben, z. B. als **-textvariable** an ein Tk-Label-Widget.

codename *name*
Veraltet. Verwenden Sie stattdessen **myproc**. Bei Angabe des Namens einer Prozedur (jedoch nicht einer Typ- oder Instanzmethode) gibt er den vollqualifizierten Befehlsnamen zurück, der sich zur Übergabe als Rückruffunktion eignet.

Komponenten und Weiterleitung

Wenn ein Objekt andere Objekte enthält – beispielsweise wenn eine Symbolleiste Schaltflächen enthält oder ein GUI-Objekt ein Objekt enthält, das auf eine Datenbank verweist –, wird das enthaltene Objekt als Komponente bezeichnet. Die übliche Vorgehensweise beim Umgang mit Komponentenobjekten, die zu einem Snit-Objekt gehören, besteht darin, diese mithilfe von **component** zu deklarieren, wodurch eine Komponenten-Instanzvariable erstellt wird. Im folgenden Beispiel enthält ein **dog**-Objekt ein **tail**-Objekt:

```
snit::type dog {
    component mytail

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    method wag {} {
        $mytail wag
    }
}
```

```
    snit::type tail {
        option -length 5
        option -partof
        method wag {} { return "Wag, wag, wag."}
    }
```

Da der Name des Objekts **tail** in einer Instanzvariablen gespeichert ist, ist er in jeder Methode leicht zugänglich.

Der Befehl **install** bietet eine alternative Möglichkeit, die Komponente zu erstellen und zu installieren:

```
    snit::type dog {
        component mytail

        constructor {args} {
            install mytail using tail %AUTO% -partof $self
            $self configurelist $args
        }

        method wag {} {
            $mytail wag
        }
    }
```

Bei **snit::type**s sind die beiden Methoden gleichwertig; bei den Typen **snit::widget** und **snit::widgetadaptor** initialisiert der Befehl **install** die Optionen des Widgets ordnungsgemäß, indem er die Tk-Optionsdatenbank abfragt.

In den obigen Beispielen ruft die Methode **wag** des Objekts **dog** lediglich die Methode **wag** der Komponente **tail** auf. In der OO-Fachsprache wird dies als Weiterleitung bezeichnet. Snit bietet eine einfachere Möglichkeit, dies zu bewerkstelligen:

```
    snit::type dog {
        delegate method wag to mytail

        constructor {args} {
            install mytail using tail %AUTO% -partof $self
            $self configurelist $args
        }
    }
```

Die Anweisung **delegate** in der Typdefinition definiert implizit die Instanzvariable **mytail**, die den Namen der Komponente enthält (obwohl es empfehlenswert ist, **component** zu verwenden, um sie explizit zu deklarieren); außerdem definiert sie die Methode **wag** des Objekts **dog** und leitet diese an die Komponente **mytail** weiter.

Auf Wunsch können alle ansonsten unbekannten Methoden an eine bestimmte Komponente weitergeleitet werden:

```
    snit::type dog {
    delegate method * to mytail

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    method bark { return "Bark, bark, bark!" }
    }
```

In diesem Fall wird ein **dog**-Objekt seine eigene **bark**-Methode verarbeiten; **wag** wird jedoch an **mytail** weitergeleitet. Jede andere Methode, die weder von **dog** noch von **tail** erkannt wird, löst einfach einen Fehler aus.

Die Optionsweiterleitung ähnelt der Weiterleitung der Methoden, mit Ausnahme der Interaktionen mit der Tk-Optionsdatenbank; dies wird im Abschnitt Tk-Optionsdatenbank beschrieben.

Typkomponenten und Weiterleitung

Die Beziehung zwischen Typkomponenten und Instanzkomponenten entspricht derjenigen zwischen Typvariablen und Instanzvariablen sowie derjenigen zwischen Typmethoden und Instanzmethoden. So wie eine Instanzkomponente eine Instanzvariable ist, die den Namen eines Befehls enthält, so ist eine Typkomponente eine Typvariable, die den Namen eines Befehls enthält. Im Wesentlichen ist eine Typkomponente eine Komponente, die von jeder Instanz des Typs gemeinsam genutzt wird.

So wie **delegate method** dazu verwendet werden kann, Methoden an Instanzkomponenten weiterzuleiten, wie unter Komponenten und Weiterleitung beschrieben, so kann **delegate typemethod** dazu verwendet werden, Typmethoden an Typkomponenten weiterzuleiten.

Hinweis: **delegate method** kann ab Snit 0.95 Methoden sowohl an Instanzkomponenten als auch an Typkomponenten weiterleiten.

Tk-Optionsdatenbank

In diesem Abschnitt wird beschrieben, wie Snit mit der Tk-Optionsdatenbank interagiert, wobei davon ausgegangen wird, dass der Leser über praktische Kenntnisse der Optionsdatenbank und ihrer Anwendungsmöglichkeiten verfügt. Das Buch *Practical Programming in Tcl and Tk* von Welch et al. bietet eine gute Einführung in die Optionsdatenbank, ebenso wie *Effective Tcl/Tk Programming*.

Snit ist so implementiert, dass es in den meisten Fällen im Hinblick auf die Optionsdatenbank einfach das Richtige tut, vorausgesetzt, der Widget-Entwickler hält sich an die Vorgaben von Snit. Im Hauptteil dieses Abschnitts wird ausführlich darauf eingegangen, welche Anforderungen Snit stellt. Im Folgenden finden Sie zur Orientierung eine kurze Zusammenfassung dieser Anforderungen.

- Wenn die Standard-Widget-Klasse des **snit::widget** nicht Ihren Vorstellungen entspricht, legen Sie sie explizit mithilfe von **widgetclass** in der Widget-Definition fest.
- Wenn Sie Optionen definieren oder weiterleiten, legen Sie die Namen der Ressourcen und Klassen ausdrücklich fest, wenn die Standardwerte nicht Ihren Vorstellungen entsprechen.

- Verwenden Sie **installhull using**, um den Hull von **snit::widgetadaptor**s zu installieren.
- Verwenden Sie **install**, um alle anderen Komponenten zu installieren.

Das Zusammenspiel von Tk-Widgets mit der Optionsdatenbank ist eine komplexe Angelegenheit; das Zusammenspiel von Snit mit der Optionsdatenbank ist dies in noch stärkerem Maße, und es lohnt sich, auf die Einzelheiten zu achten.

Festlegen der Widget-Klasse: Jedes Tk-Widget verfügt über eine Widget-Klasse. Bei Tk-Widgets entspricht der Name der Widget-Klasse lediglich dem Namen des Widget-Typs mit einem Großbuchstaben am Anfang, z. B. lautet die Widget-Klasse für **button**-Widgets "Button".

Ebenso ist die Widget-Klasse eines **snit::widget** standardmäßig der unqualifizierte Typname mit Großbuchstaben am Anfang. Beispielsweise lautet die Widget-Klasse von

```
    snit::widget ::mylibrary::scrolledText { ... }
```

"ScrolledText". Die Widget-Klasse kann zudem explizit mithilfe der Anweisung **widgetclass** innerhalb der Definition **snit::widget** festgelegt werden.

Jedes Widget kann als **hulltype** verwendet werden, sofern es die Option **-class** zum Ändern seines Widget-Klassennamens unterstützt. Siehe dazu die Erläuterung zum Befehl **hulltype** weiter oben. Der Benutzer kann dem Widget bei der Instanziierung die Option **-class** übergeben.

Die Widget-Klasse eines **snit::widgetadaptor** entspricht lediglich der Widget-Klasse seines Hull-Widgets; dies lässt sich nicht ändern, es sei denn, das Hull-Widget unterstützt **-class**; in diesem Fall ist es in der Regel sinnvoller, **snit::widget** anstelle von **snit::widgetadaptor** zu verwenden.

Festlegen der Ressourcen- und Klassennamen von Optionen: In Tk verfügt jede Option über drei Namen: den Namen der Option, den Namen der Ressource und den Namen der Klasse. Der Optionsname beginnt mit einem Bindestrich und besteht ausschließlich aus Kleinbuchstaben; er wird beim Erstellen von Widgets sowie bei den Befehlen **configure** und **cget** verwendet.

Die Ressourcen- und Klassennamen werden verwendet, um Standardwerte für Optionen durch Abfrage der Tk-Optionsdatenbank zu initialisieren. Der Ressourcenname besteht in der Regel lediglich aus dem Optionsnamen ohne Bindestrich, kann jedoch an Wortgrenzen Großbuchstaben enthalten; der Klassenname besteht in der Regel lediglich aus dem Ressourcennamen mit einem Großbuchstaben am Anfang, was jedoch nicht immer der Fall ist. Hier sind beispielsweise die Options-, Ressourcen- und Klassennamen für einige Optionen des **text**-Widgets:

```
    -background         background         Background
    -borderwidth        borderWidth        BorderWidth
    -insertborderwidth  insertBorderWidth  BorderWidth
    -padx               padX               Pad
```

Wie leicht zu erkennen ist, lassen sich die Namen der Ressourcen und Klassen manchmal aus dem Namen der Option ableiten, jedoch nicht immer.

Snit-Optionen verfügen ebenfalls über einen Ressourcennamen und einen Klassennamen. Standardmäßig folgen diese Namen der oben genannten Regel: Der Ressourcenname entspricht dem Optionsnamen ohne Bindestrich, und der Klassenname entspricht dem Ressourcennamen mit einem Großbuchstaben am Anfang. Dies gilt sowohl für lokal definierte Optionen als auch für explizit weitergeleitete Optionen:

```
    snit::widget mywidget {
        option -background
        delegate option -borderwidth to hull
        delegate option * to text
    # ...
    }
```

In diesem Fall lautet der Name der Widget-Klasse "Mywidget". Das Widget verfügt über die folgenden Optionen: **-background**, die lokal definiert ist, und **-borderwidth**, die explizit weitergeleitet wird; alle anderen Optionen werden an eine Komponente namens "text" weitergeleitet, bei der es sich wahrscheinlich um ein Tk-**text**-Widget handelt. Wenn dies der Fall ist, verfügt **mywidget** über dieselben Optionen wie ein **text**-Widget. Die Namen der Optionen, Ressourcen und Klassen lauten wie folgt:

```
    -background  background  Background
    -borderwidth borderwidth Borderwidth
    -padx        padX        Pad
```

Hinweis: Die lokal definierte Option **-background** hat zufällig dieselben drei Namen wie die Standard-Tk-Option **-background**; und **-pad**, die implizit an die Komponente **text** weitergeleitet wird, weist für **mywidget** dieselben drei Namen auf wie für das Widget **text**. **-borderwidth** hingegen weist andere Namen für Ressourcen und Klassen auf, da das interne Wort "width" nicht großgeschrieben wird. Aus Gründen der Konsistenz sollte dies jedoch geschehen; dies wird wie folgt umgesetzt:

```
    snit::widget mywidget {
    option -background
    delegate option {-borderwidth borderWidth} to hull
    delegate option * to text
    # ...
    }
```

Der Name der Klasse lautet standardmäßig "BorderWidth", wie zu erwarten.

Nehmen wir jedoch an, dass **mywidget** auch **-padx** und **-pady** an den Hull weiterleitet. In diesem Fall müssen sowohl der Name der Ressource als auch der Name der Klasse explizit festgelegt werden:

```
    snit::widget mywidget {
    option -background
    delegate option {-borderwidth borderWidth} to hull
    delegate option {-padx padX Pad} to hull
    delegate option {-pady padY Pad} to hull
    delegate option * to text
    # ...
    }
```

Abfrage der Optionsdatenbank: Wenn Sie Ihre Widget-Klasse und Namen der Optionen wie oben beschrieben festlegen, fragt Snit bei der Erstellung jeder Instanz die Optionsdatenbank ab und verhält sich dabei in der Regel korrekt. Im weiteren Verlauf dieses Abschnitts werden die

technischen Details näher erläutert.

Initialisierung lokal definierter Optionen: Wenn eine Instanz von snit::widget erstellt wird, werden ihre lokal definierten Optionen wie folgt initialisiert: Die Namen der Ressourcen und Klassen der einzelnen Optionen werden verwendet, um die Tk-Optionsdatenbank abzufragen. Wenn das Ergebnis nicht leer ist, wird es als Standardwert der Option verwendet; andernfalls kommt der in der Typdefinition fest codierte Standardwert zum Einsatz. In beiden Fällen kann der Standardwert vom Aufrufer überschrieben werden. Ein Beispiel:

```
    option add *Mywidget.texture pebbled

    snit::widget mywidget {
    option -texture smooth
    # ...
    }

    mywidget .mywidget -texture greasy
```

Hier würde **-texture** normalerweise standardmäßig auf "smooth" gesetzt werden, doch aufgrund des Eintrags, der der Optionsdatenbank hinzugefügt wurde, ist die Standardeinstellung "pebbled". Der Aufrufer hat die Standardeinstellung jedoch ausdrücklich überschrieben, sodass das neue Widget "greasy" sein wird.

Initialisierung von Optionen, die an den Hull weitergeleitet werden: Der Hull eines **snit::widget** ist ein Widget, und da seine Klasse festgelegt wurde, wird erwartet, dass er die Optionsdatenbank für sich selbst abfragt. Die einzige Ausnahme bilden Optionen, die unter einem anderen Namen an ihn weitergeleitet werden. Betrachten Sie den folgenden Code:

```
    option add *Mywidget.borderWidth 5
    option add *Mywidget.relief sunken
    option add *Mywidget.hullbackground red
    option add *Mywidget.background green

    snit::widget mywidget {
    delegate option -borderwidth to hull
    delegate option -hullbackground to hull as -background
    delegate option * to hull
    # ...
    }

    mywidget .mywidget

    set A [.mywidget cget -relief]
    set B [.mywidget cget -hullbackground]
    set C [.mywidget cget -background]
    set D [.mywidget cget -borderwidth]
```

Die Frage lautet: Welche Werte haben die Variablen A, B, C und D?

Der Wert von A ist "sunken". Der Hull ist ein Tk-Frame, dem die Widget-Klasse "Mywidget"

zugewiesen wurde; er fragt automatisch die Optionsdatenbank ab und übernimmt diesen Wert. Da die Option **-relief** implizit an den Hull weitergeleitet wird, unternimmt Snit nichts.

Der Wert von B lautet "red". Der Hull übernimmt automatisch den Wert "green" für seine Option **-background**, genau wie er zuvor den Wert für **-relief** übernommen hat. Snit weiß jedoch, dass **-hullbackground** der Option **-background** des Hulls zugeordnet ist; daher fragt es die Optionsdatenbank nach **-hullbackground** ab, erhält den Wert "red" und aktualisiert den Hull entsprechend.

Der Wert von C ist ebenfalls "red", da **-background** implizit an den Hull weitergeleitet wird; daher entspricht das Abrufen dieses Werts dem Abrufen von **-hullbackground**. Beachten Sie, dass dieser Fall ungewöhnlich ist; in der Praxis würde **-background** wahrscheinlich explizit an eine andere Komponente weitergeleitet werden.

Der Wert von D beträgt "5", jedoch nicht aus dem Grund, den Sie vermuten. Beachten Sie, dass gemäß der obigen Definition der Ressourcenname für **-borderwidth** standardmäßig "borderwidth" lautet, während der Eintrag in der Optionsdatenbank "borderWidth" lautet. Wie bei **-relief** übernimmt der Hull seine eigene Option **-borderwidth**, bevor Snit irgendetwas unternimmt. Da die Option unter ihrem eigenen Namen weitergeleitet wird, geht Snit davon aus, dass alles korrekt abgelaufen ist, und befasst sich nicht weiter damit.

Bei **snit::widgetadaptor**s stellt sich die Situation etwas anders dar. Widget-Adapter behalten die Widget-Klasse ihres Hulls bei, und der Hull wird nicht automatisch von Snit erstellt. Stattdessen muss **snit::widgetadaptor** in seinem Konstruktor **installhull** aufrufen. Dies geschieht üblicherweise wie folgt:

```
    snit::widgetadaptor mywidget {
    # ...
    constructor {args} {
        # ...
        installhull using text -foreground white
        #
    }
    #...
    }
```

In diesem Fall erstellt der Befehl **installhull** den Hull mithilfe eines Befehls wie dem folgenden:

```
    set hull [text $win -foreground white]
```

Der Hull ist ein **text**-Widget, daher lautet seine Widget-Klasse "Text". Genau wie bei **snit::widget**-Hulls geht Snit davon aus, dass er alle seine normalen Optionswerte automatisch übernimmt; Optionen, die unter einem anderen Namen weitergeleitet werden, werden auf dieselbe Weise aus der Optionsdatenbank initialisiert.

Initialisierung von Optionen, die an andere Komponenten weitergeleitet wurden: Nicht-Hull-Komponenten werden auf zwei Arten mit der Optionsdatenbank abgeglichen. Erstens bleibt ein Komponenten-Widget weiterhin ein Widget und wird daher wie gewohnt aus der Optionsdatenbank initialisiert. Zweitens wird die Optionsdatenbank nach allen an die Komponente weitergeleiteten Optionen abgefragt, und die Komponente wird entsprechend initialisiert – vorausgesetzt, sie wird mit dem Befehl **install** erstellt.

Bevor die Unterstützung für die Optionsdatenbank in Snit integriert wurde, bestand die übliche Vorgehensweise beim Erstellen einer Komponente darin, diese einfach im Konstruktor zu erstellen und ihren Befehlsnamen der Komponentenvariablen zuzuweisen:

```
    snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        set myComp [text $win.text -foreground black]
    }
    }
```

Der Nachteil dieser Methode besteht darin, dass Snit keine Möglichkeit hat, die Komponente ordnungsgemäß zu initialisieren. Daher wird nun folgender Ansatz verwendet:

```
    snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        install myComp using text $win.text -foreground black
    }
    }
```

Der Befehl **install** führt Folgendes aus:

- Erstellt eine Liste der Optionen, die explizit im Befehl **install** enthalten sind – in diesem Fall **-foreground**.
- Fragt die Optionsdatenbank nach allen Optionen, die explizit an die genannte Komponente weitergeleitet wurden.
- Erstellt die Komponente mithilfe des angegebenen Befehls, nachdem zuvor eine Liste von Optionen und Werten, die aus der Optionsdatenbank ausgelesen wurden, in den Befehl eingefügt wurde. Somit haben die explizit angegebenen Optionen (**-foreground**) Vorrang vor allen aus der Optionsdatenbank ausgelesenen Werten.
- Wenn die Widget-Definition Optionen implizit mithilfe von **delegate option *** an die Komponente weiterleitet, ruft Snit die Methode **configure** der neu erstellten Komponente auf, um eine Liste aller Optionen der Komponente zu erhalten. Daraus erstellt Snit eine Liste der Optionen, die implizit an die Komponente weitergeleitet wurden, jedoch nicht explizit im Befehl **install** enthalten waren. Für alle diese Optionen fragt Snit die Optionsdatenbank ab und konfiguriert die Komponente entsprechend.

Nicht-Widget-Komponenten: Die Optionsdatenbank wird für **snit::type**s niemals abgefragt, da eine Abfrage nur unter Angabe eines Tk-Widget-Namens möglich ist. **snit::widget**s können jedoch Nicht-Widget-Komponenten enthalten. Und wenn Optionen an diese Komponenten weitergeleitet werden und der Befehl **install** zur Installation dieser Komponenten verwendet wird, werden sie ebenso wie Widget-Komponenten anhand der Optionsdatenbank initialisiert.

Makros und Metaprogrammierung

Der Befehl **snit::macro** ermöglicht ein gewisses Maß an Metaprogrammierung mit Snit-Klassen. Nehmen wir beispielsweise an, Sie möchten Eigenschaften definieren: Instanzvariablen, die über Set- und Get-Methoden verfügen. Ihr Code könnte dann wie folgt aussehen:

```
    snit::type dog {
        variable mood happy

        method getmood {} {
            return $mood
        }

        method setmood {newmood} {
            set mood $newmood
        }
    }
```

Das sind neun Zeilen Text pro Eigenschaft. Oder Sie definieren das folgende **snit::macro**:

```
    snit::macro property {name initValue} {
        variable $name $initValue

        method get$name {} "return $name"

        method set$name {value} "set $name \$value"
    }
```

Hinweis: Es handelt sich bei einem **snit::macro** lediglich um eine normale Tcl-Prozedur, die im Slave-Interpreter definiert ist, der zum Kompilieren von Typ- und Widget-Definitionen dient; folglich hat sie Zugriff auf alle Befehle, die zur Definition von Typen und Widgets dienen.

Mit diesem neuen Makro können Sie eine Eigenschaft in einer einzigen Codezeile definieren:

```
    snit::type dog {
        property mood happy
    }
```

Innerhalb eines Makros beziehen sich die Befehle **variable** und **proc** auf die Snit-Typdefinitionsbefehle und nicht auf die Standard-Tcl-Befehle. Um die Standard-Tcl-Befehle zu erhalten, verwenden Sie **_variable** und **_proc**.

Da für die Kompilierung aller Snit-Typen und -Widgets in der Anwendung ein einziger Slave-Interpreter verwendet wird, besteht die Möglichkeit von Namenskonflikten bei Makros. Wenn Sie ein wiederverwendbares Paket mit Snit erstellen und dabei **snit::macro**s verwenden, definieren Sie diese bitte in Ihrem Paket-Namespace:

```
    snit::macro mypkg::property {name initValue} { ... }

    snit::type dog {
        mypkg::property mood happy
    }
```

Dadurch bleibt der globale Namespace für die Autoren von Anwendungen frei.

Validierungstypen

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art überprüft werden

können. So wird beispielsweise **snit::integer** verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    snit::integer validate 5     ;# Does nothing
    snit::integer validate 5.0   ;# Throws an error (not an integer!)
```

Die Methode **validate** gibt im Erfolgsfall stets den validierten Wert zurück und löst im Fehlerfall einen Fehler mit **-errorcode** INVALID aus.

Snit definiert eine Familie von Validierungstypen, die alle als **snit::type** implementiert sind. Sie können unverändert verwendet werden; darüber hinaus dienen ihre Instanzen als parametrisierte Untertypen. Beispielsweise ist eine Wahrscheinlichkeit eine Zahl zwischen 0,0 und 1,0 (einschließlich):

```
    snit::double probability -min 0.0 -max 1.0
```

Das obige Beispiel erstellt eine Instanz von **snit::double** – einen Validierungsuntertyp – namens **probability**, die zur Validierung von Wahrscheinlichkeitswerten verwendet werden kann:

```
    probability validate 0.5   ;# Does nothing
    probability validate 7.9   ;# Throws an error
```

Validierungsuntertypen können explizit definiert werden, wie im obigen Beispiel gezeigt; wenn bei einer lokal definierten Option **-type** angegeben wird, können sie auch dynamisch erstellt werden:

```
    snit::enum ::dog::breed -values {mutt retriever sheepdog}

    snit::type dog {
        # Define subtypes on the fly...
        option -breed -type {
            snit::enum -values {mutt retriever sheepdog}
        }

        # Or use predefined subtypes...
        option -breed -type ::dog::breed
    }
```

Jedes Objekt, das über eine Methode **validate** mit der oben beschriebenen Semantik verfügt, kann als Validierungstyp verwendet werden; Informationen zur Definition neuer Validierungstypen finden Sie im Abschnitt Validierungstypen definieren.

Snit definiert die folgenden Validierungstypen:

snit::boolean validate ?*value*?
siehe unten ...

snit::boolean *name*
Validiert boolesche Tcl-Werte: 1, 0, **on**, **off**, **yes**, **no**, **true**, **false**. Es ist möglich, Untertypen –

also Instanzen – von **snit::boolean** zu definieren, doch da der Typ keine Optionen hat, gibt es dafür keinen Grund.

snit::double validate ?*value*?
siehe unten ...

snit::double *name* ?*option value*...?
Validiert Gleitkommawerte. Mit den folgenden Optionen können Untertypen erstellt werden:

-min *min*
Legt eine untere Grenze für Gleitkommazahlen fest; ein Wert ist ungültig, wenn er streng kleiner als *min* ist.

-max *max*
Legt eine obere Grenze für Gleitkommazahlen fest; ein Wert ist ungültig, wenn er streng größer als *max* ist.

snit::enum validate ?*value*?
siehe unten ...

snit::enum *name* ?*option value*...?
Überprüft, ob ein Wert aus einer Aufzählungsliste stammt. Der Basistyp ist für sich genommen kaum von Nutzen, da nur Untertypen tatsächlich über eine Aufzählungsliste verfügen, anhand derer eine Validierung erfolgen kann. Untertypen können mit den folgenden Optionen erstellt werden:

-values *list*
Legt fest, welche Werte gültig sind. Ein Wert ist genau dann gültig, wenn er in der Liste enthalten ist.

snit::fpixels validate ?*value*?
siehe unten ...

snit::fpixels *name* ?*option value*...?
Nur für Tk-Programme. Validiert Bildschirmabstände in jeder der von **winfo fpixels** akzeptierten Formen. Untertypen können mit den folgenden Optionen erstellt werden:

-min *min*
Legt eine Untergrenze fest; ein Wert ist ungültig, wenn er streng kleiner als *min* ist. Die Grenze kann in jeder der von **winfo fpixels** akzeptierten Formen angegeben werden.

-max *max*
Legt eine Obergrenze fest; ein Wert ist ungültig, wenn er streng größer als *max* ist. Die Obergrenze kann in jeder der von **winfo fpixels** akzeptierten Formen angegeben werden.

snit::integer validate ?*value*?
siehe unten ...

snit::integer *name* ?*option value*...?
Validiert ganzzahlige Werte. Untertypen können mit den folgenden Optionen erstellt werden:

-min *min*
Legt eine ganzzahlige Untergrenze fest; ein Wert ist ungültig, wenn er streng kleiner als *min* ist.

-max *max*
Legt eine ganzzahlige Obergrenze fest; ein Wert ist ungültig, wenn er streng größer als *max* ist.

snit::listtype validate ?*value*?
siehe unten ...

snit::listtype *name* ?*option value*...?
Validiert Tcl-Listen. Mit den folgenden Optionen können Untertypen erstellt werden:

-minlen *min*
Legt eine Mindestlänge der Liste fest; der Wert ist ungültig, wenn die Liste weniger als *min* Elemente enthält. Der Standardwert ist 0.

-maxlen *max*
Legt eine maximale Listenlänge fest; der Wert ist ungültig, wenn die Liste mehr als *max* Elemente enthält.

-type *type*
Legt den Typ der Listenelemente fest; *type* muss der Name eines Validierungstyps oder -untertyps sein. Im folgenden Beispiel muss der Wert von **-numbers** eine Liste von Ganzzahlen sein.

```
    option -numbers -type {snit::listtype -type snit::integer}
```

Beachten Sie, dass diese Option die dynamische Definition neuer Validierungsuntertypen nicht unterstützt; das heißt, der folgende Code funktioniert (jedenfalls noch) nicht:

```
    option -numbers -type {
        snit::listtype -type {snit::integer -min 5}
    }
```

Definieren Sie den Untertyp stattdessen explizit:

```
    snit::integer gt4 -min 5

    snit::type mytype {
        option -numbers -type {snit::listtype -type gt4}
    }
```

snit::pixels validate ?*value*?
siehe unten ...

snit::pixels *name* ?*option value*...?
Nur für Tk-Programme. Validiert Bildschirmabstände in jeder der von **winfo pixels** akzeptierten Formen. Untertypen können mit den folgenden Optionen erstellt werden:

-min *min*
Legt eine Untergrenze fest; ein Wert ist ungültig, wenn er streng kleiner als *min* ist. Die Grenze kann in jeder der von **winfo pixels** akzeptierten Formen angegeben werden.

-max *max*
Legt eine Obergrenze fest; ein Wert ist ungültig, wenn er streng größer als *max* ist. Die Obergrenze kann in jeder der von **winfo pixels** akzeptierten Formen angegeben werden.

snit::stringtype validate ?*value*?
siehe unten ...

snit::stringtype *name* ?*option value*...?
Überprüft Tcl-Zeichenketten auf Gültigkeit. Der Basistyp ist für sich genommen kaum von Nutzen, da jeder Tcl-Wert gleichzeitig eine gültige Zeichenkette ist. Mit den folgenden Optionen können Untertypen erstellt werden:

-minlen *min*
Legt eine Mindestlänge für die Zeichenfolge fest; der Wert ist ungültig, wenn er weniger als *min* Zeichen enthält. Der Standardwert ist 0.

-maxlen *max*
Legt eine maximale Zeichenfolgenlänge fest; der Wert ist ungültig, wenn er mehr als *max* Zeichen enthält.

-glob *pattern*
Legt ein **string match**-Muster fest; der Wert ist ungültig, wenn er nicht mit dem Muster übereinstimmt.

-regexp *regexp*
Legt einen regulären Ausdruck fest; der Wert ist ungültig, wenn er nicht mit dem regulären Ausdruck übereinstimmt.

-nocase *flag*
Standardmäßig wird bei den Übereinstimmungen mit **-glob** und **-regexp** die Groß-/Kleinschreibung berücksichtigt. Wenn **-nocase** auf "true" gesetzt ist, erfolgt die Übereinstimmung sowohl bei **-glob** als auch bei **-regexp** ohne Berücksichtigung der Groß-/Kleinschreibung.

snit::window validate ?*value*?
siehe unten ...

snit::window *name*
Nur für Tk-Programme. Validiert Tk-Fensternamen. Der Wert muss dazu führen, dass **winfo exists** "true" zurückgibt; andernfalls ist der Wert ungültig. Es ist möglich, Untertypen – also Instanzen – von **snit::window** zu definieren, doch da der Typ derzeit keine Optionen hat, gibt es dafür keinen Grund.

Validierungstypen definieren

Es gibt drei Möglichkeiten, einen neuen Validierungstyp zu definieren: als Untertyp eines der Validierungstypen von Snit, als Validierungstyp-Befehl und als vollwertiger Validierungstyp, ähnlich den von Snit bereitgestellten. Die Definition von Untertypen der Validierungstypen von Snit wird weiter oben im Abschnitt Validierungstypen beschrieben.

Die nächsteinfachste Möglichkeit, einen neuen Validierungstyp zu erstellen, ist die Verwendung eines Validierungstyp-Befehls. Ein Validierungstyp ist lediglich ein Objekt, das über eine Methode namens **validate** verfügt; die Methode **validate** muss ein Argument – einen Wert – entgegennehmen, den Wert zurückgeben, wenn er gültig ist, und bei einem ungültigen Wert einen Fehler mit dem Code **-errorcode** INVALID auslösen. Dies lässt sich mit einem einfachen **proc** bewerkstelligen. So hätte beispielsweise der Validierungstyp **snit::boolean** auch wie folgt implementiert werden können:

```
    proc ::snit::boolean {"validate" value} {
        if {![string is boolean -strict $value]} {
            return -code error -errorcode INVALID  "invalid boolean
    \"$value\", should be one of: 1, 0, ..."
        }

        return $value
    }
```

Ein auf diese Weise definierter Validierungstyp kann natürlich keine Untertypen haben; für viele Anwendungen wird dies jedoch ausreichen.

Schließlich lässt sich ein vollwertiger Validierungstyp, der Untertypen haben kann, als **snit::type** definieren. Hier finden Sie ein Grundgerüst für den Einstieg:

```
    snit::type myinteger {
        # First, define any options you'd like to use to define
        # subtypes.  Give them defaults such that they won't take
        # effect if they aren't used, and marked them "read-only".
        # After all, you shouldn't be changing their values after
        # a subtype is defined.
        #
        # For example:

        option -min -default "" -readonly 1
        option -max -default "" -readonly 1

        # Next, define a "validate" type method which should do the
        # validation in the basic case.  This will allow the
        # type command to be used as a validation type.

        typemethod validate {value} {
            if {![string is integer -strict $value]} {
                return -code error -errorcode INVALID  "invalid value
    \"$value\", expected integer"
            }

            return $value
        }

        # Next, the constructor should validate the subtype options,
        # if any.  Since they are all readonly, we don't need to worry
        # about validating the options on change.

        constructor {args} {
            # FIRST, get the options
            $self configurelist $args
```

```
            # NEXT, validate them.

            # I'll leave this to your imagination.
        }

        # Next, define a "validate" instance method; its job is to
        # validate values for subtypes.

        method validate {value} {
            # First, call the type method to do the basic validation.
            $type validate $value

            # Now we know it's a valid integer.

            if {("" != $options(-min) && $value < $options(-min))  ||
                ("" != $options(-max) && $value > $options(-max))} {
                # It's out of range; format a detailed message about
                # the error, and throw it.

                set msg "...."

                return -code error -errorcode INVALID $msg
            }

            # Otherwise, if it's valid just return it.
            return $valid
        }
    }
```

Und nun verfügen Sie über einen Typ, der Untertypen haben kann.

Die Datei "validate.tcl" im Snit-Paket definiert alle Validierungstypen von Snit; dort finden Sie die vollständige Implementierung für **snit::integer** und die anderen Typen, die Sie als Beispiele für Ihre eigenen Typen verwenden können.

Einschränkungen

Sollten Sie Probleme haben, Fehler entdecken oder neue Ideen haben, sind Sie hiermit herzlich eingeladen, einen Bericht über Ihr Problem, Ihren Fehler oder Ihre Idee einzureichen, wie im Abschnitt Fehler, Ideen, Rückmeldungen weiter unten beschrieben.

Darüber hinaus möchten Sie sich vielleicht in die Snit-Mailingliste eintragen; weitere Informationen finden Sie unter http://www.wjduquette.com/snit.

Ein Bereich, den Sie besonders im Auge behalten sollten, ist die Verwendung von **snit::widgetadaptor** zur Anpassung von Megawidgets, die von anderen Megawidget-Paketen erstellt wurden; die korrekte Zerstörung der Widgets hängt von der Reihenfolge der -Bindings ab. Am sinnvollsten ist es, dies einfach zu unterlassen.

Bekannte Fehler

- Die von Snit 1.x zurückgegebenen Fehler-Stack-Traces sind äußerst unübersichtlich und enthalten in der Regel viel zu viele Informationen über die internen Abläufe von Snit. In Snit 2.2 wurden die Fehlermeldungen deutlich verbessert.
- Beachten Sie bitte auch die Projekt-Tracker, wie sie im Abschnitt Fehler, Ideen, Rückmeldungen weiter unten erläutert werden.

Geschichte

Im Zuge der Entwicklung von "Notebook" (siehe http://www.wjduquette.com/notebook), meiner auf Tcl basierenden persönlichen Notizbuchanwendung, stellte ich fest, dass ich diese als eine Sammlung von Objekten programmierte. Ich verwendete keinen bestimmten objektorientierten Rahmen; ich schrieb lediglich Objekte in reinem Tcl, wobei ich mich an die Richtlinien in meinem "Guide to Object Commands" (siehe http://www.wjduquette.com/tcl/objects.html) hielt und einige weitere Tricks anwandte, die ich mir seitdem angeeignet hatte. Und obwohl es gut funktionierte, wurde es aufgrund der Menge an Boilerplate-Code, die mit jedem neuen Objekt verbunden war, schnell mühsam.

Das war also der eine Punkt – Langeweile ist ein starker Motivationsfaktor. Der andere Punkt, der mir auffiel, war, dass ich Vererbung überhaupt nicht nutzte und sie mir auch nicht fehlte. Stattdessen setzte ich auf Weiterleitung: Objekte, die andere Objekte erzeugten und Methoden an diese weiterleiteten.

Und ich sagte mir: "Das wird langsam langweilig … es muss doch einen besseren Weg geben." Und eines Nachmittags begann ich aus einer Laune heraus, an Snit zu arbeiten, einem Objektsystem, das so funktioniert wie Tcl. Snit unterstützt zwar keine Vererbung, eignet sich jedoch hervorragend für die Weiterleitung und erleichtert die Erstellung von Megawidgets.

Wenn Sie Anmerkungen oder Vorschläge (oder Fehlermeldungen!) haben, zögern Sie bitte nicht, mir eine E-Mail an *will@wjduquette.com* zu senden. Darüber hinaus gibt es eine Snit-Mailingliste; weitere Informationen dazu finden Sie auf der Snit-Homepage (siehe http://www.wjduquette.com/snit).

Danksagungen

Snit wurde von Anfang an von William H. Duquette konzipiert und umgesetzt. Großer Dank gebührt jedoch den folgenden Personen, die Snit genutzt und mir wertvolles Feedback gegeben haben: Rolf Ade, Colin McCormack, Jose Nazario, Jeff Godfrey, Maurice Diamanti, Egon Pasztor, David S. Cargo, Tom Krehbiel, Michael Cleverly, Andreas Kupries, Marty Backe, Andy Goth, Jeff Hobbs, Brian Griffin, Donal Fellows, Miguel Sofer, Kenneth Green und Anton Kovalenko. Sollte ich jemanden vergessen haben, bitte ich um Entschuldigung; lassen Sie es mich wissen, und ich werde Ihren Namen der Liste hinzufügen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *snit* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

Copyright

```
Copyright (c) 2003-2009, by William H. Duquette
```


snitfaq
Stichwörter: snitfaq, BWidget, C++, Incr Tcl, adaptors, class, mega widget, object, object oriented, widget, widget adaptors

Name
snitfaq – Häufig gestellte Fragen zu Snit

Beschreibung

Überblick

Was ist dieses Dokument?
Dies ist eine ungewöhnliche FAQ-Liste, da nur wenige der darin aufgeführten Fragen tatsächlich häufig gestellt werden. Vielmehr handelt es sich hierbei um Fragen, die meiner Meinung nach ein Neuling bei Snit stellen sollte. Diese Datei stellt jedoch keine vollständige Referenz zu Snit dar; diese Informationen finden Sie auf der Seite **snit**.

Was ist "snit"?
Snit ist ein Rahmen zur Definition abstrakter Datentypen und Megawidgets in reinem Tcl. Der Name "Snit" steht für "Snit's Not Incr Tcl" und verdeutlicht, dass Snit bei der Definition von Objekten einen anderen Ansatz verfolgt als Incr Tcl, der bekannteste Objekt-Rahmen für Tcl. Hätte ich geahnt, dass Snit überhaupt populär werden würde, hätte ich mir wahrscheinlich einen anderen Namen ausgesucht.

Der Hauptzweck von Snit besteht darin, als *Objekt-Bindeglied* zu fungieren – also Ihnen dabei zu helfen, verschiedene Objekte aus unterschiedlichen Quellen zu Typen und Megawidgets mit übersichtlichen, benutzerfreundlichen Schnittstellen zusammenzustellen, damit Sie Ihre Anwendung einfacher erstellen können.

Bei Snit geht es nicht um theoretische Reinheit oder minimalistisches Design; es geht darum, Wirkungsvolles einfach und konsistent tun zu können, ohne darüber nachdenken zu müssen – sodass Sie sich ganz auf die Entwicklung Ihrer Anwendung konzentrieren können.

Bei Snit geht es nicht darum, Tausende von nahezu identischen, sorgfältig festgelegten, leichtgewichtigen Dingern zu implementieren – jedenfalls nicht als einzelne Snit-Objekte. Herkömmliche Tcl-Methoden sind wesentlich schneller und nicht wesentlich komplizierter. Bei Snit geht es jedoch *darum*, eine übersichtliche Schnittstelle zur Verwaltung einer Sammlung von Tausenden nahezu identischer, sorgfältig spezifizierter, schlanker "Dinger" zu implementieren (denken Sie beispielsweise an das Text-Widget und Text-Tags oder das Canvas-Widget und Canvas-Objekte). Mit Snit können Sie die Details darüber verbergen, wie genau diese "Dinger" gespeichert werden – sodass Sie diese ignorieren und sich ganz auf die Entwicklung Ihrer Anwendung konzentrieren können.

Snit ist weder eine Lebensphilosophie noch ein Wundermittel noch der Jungbrunnen. Es ist lediglich eine Methode, um Komplexität zu bewältigen – und einen Teil der Komplexität beim Bewältigen von Komplexität –, damit Sie sich auf die Entwicklung Ihrer Anwendung konzentrieren können.

Welche Tcl-Version benötigt snit?

Snit 1.3 benötigt Tcl 8.3 oder höher; Snit 2.2 benötigt Tcl 8.5 oder höher. Die Unterschiede zwischen Snit 1.3 und Snit 2.2 finden Sie im Abschnitt Snit-Versionen.

Wo kann ich Snit herunterladen?

Snit ist Teil von Tcllib, der Standardbibliothek von Tcl; möglicherweise verfügen Sie also bereits darüber. Es ist außerdem auf der Snit-Homepage unter http://www.wjduquette.com/snit verfügbar.

Welche Ziele verfolgt snit?

- Ein Snit-Objekt sollte mindestens ebenso effizient sein wie ein manuell programmiertes Tcl-Objekt (siehe http://www.wjduquette.com/tcl/objects.html).
- Die Tatsache, dass Snit bei der Implementierung eines Objekts verwendet wurde, sollte für die Klienten dieses Objekts nicht erkennbar (und unerheblich) sein.
- Snit sollte in der Lage sein, Objekte aus anderen Quellen, insbesondere Tk-Widgets, zu kapseln.
- Snit-Megawidgets sollten sich in ihrer Schnittstelle (soweit möglich) nicht von Tk-Widgets unterscheiden.
- Snit sollte "Tcl-ähnlich" sein – das heißt, es sollte nicht versuchen, C++, Smalltalk oder irgendetwas anderes nachzuahmen, sondern vielmehr Tcl selbst nachzuahmen.
- Es sollte eine einfache, benutzerfreundliche und leicht zu merkende Syntax aufweisen.

Inwiefern unterscheidet sich snit von anderen OO-Rahmen?

Snit ist unter den Tcl-Objektsystemen insofern einzigartig, als es nicht auf Vererbung, sondern auf Weiterleitung basiert. Objektsysteme, die auf Vererbung beruhen, erlauben es Ihnen lediglich, von Klassen zu erben, die mit demselben System definiert wurden, und das ist bedauerlich. In Tcl ist ein Objekt alles, was sich wie ein Objekt verhält; es sollte keine Rolle spielen, wie das Objekt implementiert wurde. Ich habe Snit entwickelt, um mir dabei zu helfen, Anwendungen aus den vorhandenen Bausteinen zu erstellen; daher ist Snit so konzipiert, dass es jedes Objekt einbinden und darauf aufbauen kann, sei es ein handcodiertes Objekt, ein Tk-Widget, ein Incr-Tcl-Objekt, ein BWidget oder fast alles andere.

Hinweis: Sie können den Effekt der Vererbung mithilfe von Komponenten und Weiterleitung erzielen – und dabei von allem erben, was wie ein Tcl-Objekt aussieht.

Was kann ich mit snit machen?

Mit Snit kann ein Programmierer:

- abstrakte Datentypen und Tk-Megawidgets erstellen.
- Instanzvariablen, Typvariablen und Optionen im Tk-Stil definieren.
- Konstruktoren, Destruktoren, Instanzmethoden, Typmethoden und Prozeduren definieren.
- einen Typ aus Komponententypen zusammensetzen. Instanzmethoden und Optionen können automatisch an die Komponententypen weitergeleitet werden.

Snit-Versionen

Welche Version von snit sollte ich verwenden?

Die aktuelle Snit-Distribution enthält zwei Versionen: Snit 1.3 und Snit 2.2. Der Grund dafür, dass beide Versionen enthalten sind, liegt darin, dass Snit 2.2 eine Reihe neuer Funktionen von Tcl 8.5 nutzt, um die Laufzeiteffizienz zu verbessern; als Nebeneffekt sind die Fehlermeldungen und Stack-Traces von Snit deutlich weniger hässlich geworden. Der Nachteil bei der Verwendung von Snit 2.2 besteht natürlich darin, dass Sie Tcl 8.5 als Zielplattform wählen müssen.

Snit 1.3 verfügt hingegen nicht über die Optimierungen von Snit 2.2, erfordert jedoch lediglich Tcl 8.3 oder höher.

Kurz gesagt: Wenn Sie Tcl 8.3 oder 8.4 als Zielplattform verwenden, sollten Sie Snit 1.3 nutzen. Wenn Sie Tcl 8.5 als Zielplattform in Betracht ziehen können, sollten Sie auf jeden Fall Snit 2.2 verwenden. Wenn Sie beide Versionen als Zielplattform nutzen wollen, können Sie entweder ausschließlich Snit 1.3 verwenden oder – sofern Ihr Code von den geringfügigen Inkompatibilitäten zwischen den beiden Versionen nicht betroffen ist – Snit 1.3 für Tcl 8.4 und Snit 2.2 für Tcl 8.5 einsetzen.

Wie wähle ich die Snit-Version aus, die ich verwenden möchte?

Um stets Snit 1.3 (oder eine neuere Version von Snit 1.x) zu verwenden, rufen Sie Snit wie folgt auf:

```
package require snit 1.3
```

Um stets Snit 2.2 (oder eine neuere Version von Snit 2.x) zu verwenden, geben Sie stattdessen Folgendes ein:

```
package require snit 2.2
```

Hinweis: Wenn Sie Snit 2.2 ausdrücklich anfordern, bricht Ihre Anwendung unter Tcl 8.4 ab, da Snit 2.2 für Tcl 8.4 nicht verfügbar ist.

Wenn Sie möchten, dass Ihre Anwendung stets die aktuellste verfügbare Version von Snit verwendet, legen Sie bitte keine Versionsnummer fest:

```
package require snit
```

Tcl sucht und lädt die aktuellste Version, die für die von Ihnen verwendete Tcl-Version verfügbar ist. Achten Sie in diesem Fall darauf, keine inkompatiblen Funktionen zu verwenden.

Inwiefern sind snit 1.3 und snit 2.2 nicht miteinander kompatibel?

Snit 2.2 ist soweit wie möglich als direkter Ersatz für Snit 1.3 gedacht. Leider waren einige Inkompatibilitäten unvermeidbar, da Snit 2.2 den neuen Mechanismus "**namespace ensemble**" von Tcl 8.5 zur Implementierung der Verteilung der Unterbefehle nutzt. Dieser Ansatz ist wesentlich schneller als der in Snit 1.3 verwendete Mechanismus und führt zudem zu deutlich besseren Fehlermeldungen; allerdings bringt er auch neue Einschränkungen für die Implementierung mit sich.

Es gibt vier konkrete Inkompatibilitäten zwischen Snit 1.3 und Snit 2.2.

- Snit 1.3 unterstützt die implizite Benennung von Objekten. Angenommen, Sie definieren einen neuen snit::type mit dem Namen **dog**. Sie können Instanzen von **dog** auf drei verschiedene Arten erstellen:

```
dog spot               ;# Explicit naming
set obj1 [dog %AUTO%]  ;# Automatic naming
set obj2 [dog]         ;# Implicit naming
```

- In Snit 2.2 werden Typbefehle mithilfe des Mechanismus **namespace ensemble** definiert; und **namespace ensemble** erlaubt es nicht, einen Ensemble-Befehl ohne Unterbefehl aufzurufen. Kurz gesagt: Bei Verwendung von **namespace ensemble** gibt es keine Möglichkeit, implizite Namen zu unterstützen.

  Es ist jedoch noch nicht alles verloren. Wenn der Typ keine Typmethoden besitzt, handelt es sich bei dem Typbefehl um einen einfachen Befehl und nicht um ein Ensemble, sodass **namespace ensemble** nicht verwendet wird. In diesem Fall ist implizite Benennung weiterhin möglich.

  Kurz gesagt: Sie können implizite Benennung nutzen, wenn Sie bereit sind, auf Typmethoden (einschließlich der Standard-Typmethoden wie **$type info**) zu verzichten. Verwenden Sie dazu das Pragma **-hastypemethods**:

```
pragma -hastypemethods 0
```

- Hierarchische Methoden und Typmethoden werden in Snit 2.2 unterschiedlich implementiert.

  Eine hierarchische Methode ist eine Instanzmethode, die Unterbefehle besitzt; diese Unterbefehle sind ihrerseits Methoden. Der Befehl **tag** des Tk-Text-Widgets und seine Unterbefehle sind Beispiele für hierarchische Methoden. Sie können solche Unterbefehle in Snit einfach implementieren, indem Sie mehrere Wörter in die Namen der Methoden einfügen:

```
method {tag configure} {tag args} { ... }

method {tag cget} {tag option} {...}
```

- Hier haben wir implizit eine Methode namens "**tag**" definiert, die zwei Unterbefehle enthält: "**configure**" und "**cget**".

  In Snit 1.3 konnten hierarchische Methoden auf zwei Arten aufgerufen werden:

```
$obj tag cget -myoption      ;# The good way
$obj {tag cget} -myoption    ;# The weird way
```

- Im zweiten Aufruf sehen wir, dass eine hierarchische Methode oder Typmethode schlichtweg eine solche ist, deren Name aus mehreren Wörtern besteht.

  In Snit 2.2 ist dies nicht mehr der Fall, und die "seltsame" Art, hierarchische Methoden und Typmethoden aufzurufen, funktioniert nicht mehr.
- Die dritte Inkompatibilität ergibt sich aus der zweiten. In Snit 1.3 waren hierarchische Methoden ebenfalls einfache Methoden, deren Name aus mehreren Wörtern bestand. Infolgedessen gab **$obj info methods** die vollständigen Namen aller hierarchischen Methoden zurück. Im obigen Beispiel würde die von **$obj info methods** zurückgegebene Liste **tag configure** und **tag cget** enthalten, jedoch nicht **tag**, da **tag** nur implizit definiert ist.

  In Snit 2.2 sind hierarchische Methoden und Typmethoden nicht mehr einfach solche, deren Name mehrere Wörter enthält; im obigen Beispiel würde die von **$obj info methods** zurückgegebene Liste **tag** enthalten, jedoch nicht **tag configure** oder **tag cget**.
- Die vierte Inkompatibilität ist auf eine neue Funktion zurückzuführen. Snit 2.2 verwendet den

neuen Befehl **namespace path**, sodass der Code eines Typs jeden im übergeordneten Namespace des Typs definierten Befehl ohne Qualifizierung oder Import aufrufen kann. Nehmen wir beispielsweise an, Sie haben ein Paket namens **mypackage**, das eine Reihe von Befehlen definiert, darunter einen Typ namens **::mypackage::mytype**. Dank **namespace path** kann der Code des Typs jeden der anderen Befehle aufrufen, die in **::mypackage::** definiert sind.

Dies ist äußerst praktisch. Es bedeutet jedoch auch, dass Befehle, die im übergeordneten Namespace "**::mypackage::**" definiert sind, den Zugriff des Typs auf gleichnamige Befehle im globalen Namespace blockieren können. Dies kann zu Fehlern führen. Beispielsweise enthält Tcllib einen Typ namens **::tie::std::file**. Der Code dieses Typs ruft den Standard-Befehl **file** auf. Bei der Ausführung mit Snit 2.2 kam es zu einem Fehler – der Befehl des Typs, **::tie::std::file**, ist selbst ein Befehl im übergeordneten Namespace des Typs; statt also den Standardbefehl "**file**" aufzurufen, rief der Typ sich selbst auf.

Gibt es weitere Unterschiede zwischen snit 1.x und snit 2.2?

Ja.

- Der Aufruf von Methoden erfolgt deutlich schneller.
- Viele Fehlermeldungen und Stack-Traces sind übersichtlicher.
- Das Pragma **-simpledispatch** ist veraltet und wird, falls angegeben, ignoriert. In Snit 1.x ersetzt **-simpledispatch** den Methodenaufruf durch einen schnelleren Mechanismus, wofür bestimmte Funktionen entfallen. Der Methodenaufruf in Snit 2.2 ist in allen Fällen noch schneller, sodass **-simpledispatch** nicht mehr benötigt wird.
- In Snit 2.2 kann der Code eines Typs (Methoden, Typmethoden, usw.) Befehle aus dem übergeordneten Namespace des Typs aufrufen, ohne diese zu qualifizieren oder zu importieren; das heißt, der Code des Typs **::parentns::mytype** kann **::parentns::someproc** einfach als **someproc** aufrufen.

  Dies ist äußerst nützlich, wenn ein Typ als Teil eines größeren Pakets definiert ist und sich den übergeordneten Namespace mit dem Rest des Pakets teilt; es bedeutet, dass der Typ andere, vom Paket definierte Befehle ohne zusätzlichen Aufwand aufrufen kann.

  Diese Funktion basiert auf dem neuen Tcl 8.5-Befehl **namespace path**, weshalb sie für V1.x nicht implementiert wurde. In V1.x-Code lässt sich ein ähnliches Ergebnis erzielen, indem man

  ```
  namespace import [namespace parent]::*
  ```

- in einem Typkonstruktor. Dies ist jedoch weniger nützlich, da dabei nur jene Befehle erfasst werden, die zum Zeitpunkt der Typdefinition bereits vom übergeordneten Namespace exportiert wurden.

Objekte

Was ist ein Objekt?

Eine vollständige Beschreibung der objektorientierten Programmierung würde den Rahmen dieser FAQ natürlich sprengen. Vereinfacht ausgedrückt ist ein Objekt eine Instanz eines abstrakten Datentyps – ein zusammenhängendes Bündel aus Code und Daten. Es gibt viele Möglichkeiten, Objekte in Tcl/Tk darzustellen; die bekanntesten Beispiele sind die Tk-Widgets.

Ein Tk-Widget ist ein Objekt; es wird durch einen Tcl-Befehl dargestellt. Die Methoden des Objekts sind Unterbefehle des Tcl-Befehls. Die Eigenschaften des Objekts sind Optionen, auf die

über die Methoden **configure** und **cget** zugegriffen wird. Snit verwendet dieselben Konventionen wie Tk-Widgets.

Was ist ein abstrakter Datentyp?

In der Informatik ist ein abstrakter Datentyp eine komplexe Datenstruktur zusammen mit einer Reihe von Operationen – ein Stapel, eine Warteschlange, ein binärer Baum usw. –, d. h., in moderner Terminologie ein Objekt. In Systemen, die eine Form der Vererbung beinhalten, wird üblicherweise das Wort *Klasse* anstelle von *abstraktem Datentyp* verwendet; da Snit jedoch Vererbung nicht im herkömmlichen Sinne implementiert, erscheint das ältere Wort passender. Manchmal wird dies als *objektbasierte* Programmierung im Gegensatz zur objektorientierten Programmierung bezeichnet. Hinweis: Sie können den Effekt der Vererbung mithilfe von Komponenten und Weiterleitung auf einfache Weise erzielen.

In Snit ist ein *Typ* – ebenso wie in Tk – ein Befehl, der Instanzen – also Objekte – erstellt, die zu diesem Typ gehören. Die meisten Typen definieren eine Reihe von *Optionen*, die bei der Erstellung festgelegt werden können und in der Regel später geändert werden können.

Darüber hinaus ist eine *Instanz* ebenfalls ein Tcl-Befehl – ein Befehl, der Zugriff auf die für diesen abstrakten Datentyp definierten Operationen gewährt. Üblicherweise werden die Operationen als Unterbefehle des Instanz-Befehls definiert. Um beispielsweise Text in ein Tk-Text-Widget einzufügen, verwenden Sie den Unterbefehl **insert** des Text-Widgets:

```
    # Create a text widget and insert some text in it.
    text .mytext -width 80 -height 24
    .mytext insert end "Howdy!"
```

In diesem Beispiel ist "**text**" der *Typ*-Befehl und "**.mytext**" der *Instanz*-Befehl.

In Snit werden Objekt-Unterbefehle im Allgemeinen als Instanzmethoden bezeichnet.

Welche Arten von abstrakten Datentypen bietet snit an?

Mit Snit können Sie drei Arten von abstrakten Datentypen definieren:

- snit::type
- snit::widget
- snit::widgetadaptor

Was ist ein snit::type?

Ein snit::type ist ein abstrakter Datentyp ohne grafische Benutzeroberfläche, z. B. ein Stapel oder eine Warteschlange. snit::types werden mit dem Befehl snit::type definiert. Wenn Sie beispielsweise ein Zwingerverwaltungssystem für einen Hundezüchter entwerfen würden, bräuchten Sie einen Typ "Hund".

```
% snit::type dog {
    # ...
}
::dog
%
```

Diese Definition legt einen neuen Befehl fest (in diesem Fall **::dog**), mit dem "Hund"-Objekte definiert werden können.

Eine Instanz von snit::type kann Instanzmethoden, Instanzvariablen, Optionen und Komponenten

besitzen. Der Typ selbst kann Typmethoden, Typvariablen, Typkomponenten und Prozeduren besitzen.

Was ist ein snit::widget? Die Kurzfassung

Ein snit::widget ist ein mit Snit erstelltes Tk-Megawidget; es ist einem snit::type sehr ähnlich. Siehe Widgets.

Was ist ein snit::widgetadaptor? Die Kurzfassung

Ein snit::widgetadaptor nutzt Snit, um einen bestehenden Widget-Typ (z. B. ein Tk-Label) zu umschließen und dessen Schnittstelle in geringerem oder größerem Umfang anzupassen. Es ist einem snit::widget sehr ähnlich. Siehe Widget-Adapter.

Wie erstelle ich eine Instanz von snit::type?

Sie erstellen eine Instanz von "snit::type", indem Sie den Namen der neuen Instanz an die create-Methode des Typs übergeben. Im folgenden Beispiel erstellen wir ein "**dog**"-Objekt mit dem Namen "**spot**".

```
% snit::type dog {
    # ....
}
::dog
% dog create spot
::spot
%
```

Im Allgemeinen kann der Name der Methode "**create**" weggelassen werden, solange der Name der Instanz nicht mit einer der definierten Typmethoden in Konflikt steht. (Siehe Typkomponenten für den Sonderfall, in dem dies nicht funktioniert.) Das folgende Beispiel ist daher identisch mit dem vorherigen Beispiel:

```
% snit::type dog {
    # ....
}
::dog
% dog spot
::spot
%
```

In diesem Dokument wird in der Regel die kürzere Form verwendet.

Wenn der Typ **dog** Optionen definiert, können diesen in der Regel bei der Erstellung Standardwerte zugewiesen werden:

```
% snit::type dog {
    option -breed mongrel
    option -color brown

    method bark {} { return "$self barks." }
}
::dog
```

```
% dog create spot -breed dalmation -color spotted
::spot
% spot cget -breed
dalmation
% spot cget -color
spotted
%
```

Nach der Erstellung bezeichnet der Instanzname nun einen neuen Tcl-Befehl, mit dem das Objekt gesteuert wird. Der folgende Code lässt beispielsweise den Hund bellen:

```
% spot bark
::spot barks.
%
```

Wie kann ich indirekt auf ein Objekt verweisen?

Manche Programmierer ziehen es vor, den Namen des Objekts in einer Variablen zu speichern und auf diese Weise darauf zu verweisen. Zum Beispiel:

```
% snit::type dog { ... }
::dog
% set d [dog spot -breed dalmation -color spotted]
::spot
% $d cget -breed
dalmation
% $d bark
::spot barks.
%
```

Falls Ihnen dieser Stil besser gefällt, ziehen Sie es vielleicht vor, dass Snit den Namen der Instanz automatisch generiert.

Wie kann ich den Namen des Objekts automatisch generieren?

Wenn Sie möchten, dass Snit einen Namen für ein Objekt generiert, verwenden Sie das Schlüsselwort **%AUTO%** als gewünschten Namen:

```
% snit::type dog { ... }
::dog
% set d [dog %AUTO%]
::dog2
% $d bark
::dog2 barks.
%
```

Das Schlüsselwort "**%AUTO%**" kann in eine längere Zeichenfolge eingebettet werden:

```
% set d [dog obj_%AUTO%]
::obj_dog4
% $d bark
::obj_dog4 barks.
%
```

Können Typen umbenannt werden?

Der Tcl-Befehl "**rename**" benennt andere Befehle um. Es ist in Tcl eine gängige Vorgehensweise, einen bestehenden Befehl zu ändern, indem man ihn umbenennt und einen neuen Befehl mit dem ursprünglichen Namen definiert; der neue Befehl ruft in der Regel den umbenannten Befehl auf.

Befehle vom Typ "snit::type" sollten jedoch niemals umbenannt werden; dies würde die Verbindung zwischen dem Typ und seinen Objekten unterbrechen.

Können Objekte umbenannt werden?

Der Tcl-Befehl "**rename**" benennt andere Befehle um. Es ist in Tcl eine gängige Vorgehensweise, einen bestehenden Befehl zu ändern, indem man ihn umbenennt und einen neuen Befehl mit dem ursprünglichen Namen definiert; der neue Befehl ruft in der Regel den umbenannten Befehl auf.

Alle Snit-Objekte (einschließlich *Widgets* und *Widgetadapter*) können umbenannt werden, doch diese Flexibilität hat einige Konsequenzen:

- In einer Instanzmethode enthält das implizite Argument **self** stets den aktuellen Namen des Objekts, sodass Instanzmethoden andere Instanzmethoden stets über **$self** aufrufen können.
- Wenn das Objekt umbenannt wird, ändert sich der Wert von **$self**. Verwenden Sie daher **$self** nicht für etwas, das bei einer Änderung von **$self** nicht mehr funktionieren würde. Übergeben Sie beispielsweise keinen Rückrufbefehl wie folgt an ein anderes Objekt:

```
    .btn configure -command [list $self ButtonPress]
```

- Es wird eine Fehlermeldung angezeigt, wenn **.btn** Ihren Befehl aufruft, nachdem Ihr Objekt umbenannt wurde.
- Stattdessen sollte Ihr Objekt seinen Rückrufbefehl wie folgt definieren:

```
    .btn configure -command [mymethod ButtonPress]
```

- Der Befehl mymethod gibt einen Code zurück, der die gewünschte Methode sicher aufruft; der Aufrufer der Rückruffunktion kann wie gewohnt zusätzliche Argumente an das Ende des Befehls anhängen.
- Jedes Objekt verfügt über einen privaten Namespace; der Name dieses Namespaces steht in Methodenkörpern usw. als Wert des impliziten Arguments **selfns** zur Verfügung. Dieser Wert bleibt während der gesamten Lebensdauer des Objekts konstant. Verwenden Sie **$selfns** anstelle von **$self**, wenn Sie ein eindeutiges Kennzeichen zur Identifizierung des Objekts benötigen.
- Wenn der Befehl einer "snit::widget"-Instanz umbenannt wird, bleibt der Name seines Tk-Fensters unverändert – und ist nach wie vor von großer Bedeutung. Folglich steht der Name des Tk-Fensters in Methoden als Wert des impliziten Arguments "**win**" zur Verfügung. Dieser Wert bleibt während der gesamten Lebensdauer des Objekts konstant. Beim Erstellen von untergeordneten Fenstern empfiehlt es sich, **$win.child** anstelle von **$self.child** als Namen

für das untergeordnete Fenster zu verwenden.

Wie kann ich ein Snit-Objekt löschen?

Jedes Snit-Objekt beliebigen Typs kann gelöscht werden, indem man es mithilfe des Tcl-Befehls **rename** in die leere Zeichenfolge umbenennt.

Snit-Megawidgets (d. h. Instanzen von snit::widget und snit::widgetadaptor) können wie jedes andere Widget zerstört werden: durch Verwendung des Tk-Befehls **destroy** auf das Widget selbst oder auf eines seiner übergeordneten Elemente in der Fensterhierarchie.

Jede Instanz von snit::type verfügt über eine Methode namens **destroy**:

```
% snit::type dog { ... }
::dog
% dog spot
::spot
% spot bark
::spot barks.
% spot destroy
% spot barks
invalid command name "spot"
%
```

Schließlich verfügt jeder Snit-Typ über eine Typmethode namens **destroy**; deren Aufruf löscht den Typ und alle seine Instanzen:

```
% snit::type dog { ... }
::dog
% dog spot
::spot
% spot bark
::spot barks.
% dog destroy
% spot bark
invalid command name "spot"
% dog fido
invalid command name "dog"
%
```

Instanzmethoden

Was ist eine Instanzmethode?

Eine Instanzmethode ist eine Prozedur, die einem bestimmten Objekt zugeordnet ist und als Unterbefehl des Befehls dieses Objekts aufgerufen wird. Sie hat uneingeschränkten Zugriff auf alle Typvariablen, Instanzvariablen usw. des Objekts.

Wie definiere ich eine Instanzmethode?

Instanzmethoden werden in der Typdefinition mithilfe der Anweisung **method** definiert. Betrachten Sie den folgenden Code, der möglicherweise dazu verwendet wird, Hunde zu einer Computersimulation hinzuzufügen:

```
% snit::type dog {
    method bark {} {
        return "$self barks."
    }

    method chase {thing} {
        return "$self chases $thing."
    }
}
::dog
%
```

Ein Hund kann bellen und er kann Dinge jagen.

Die Anweisung "**method**" sieht genauso aus wie ein normales Tcl-"proc", mit dem Unterschied, dass sie in einer "snit::type"-Definition vorkommt. Beachten Sie, dass jede Instanzmethode ein implizites Argument namens **self** erhält; dieses Argument enthält den Namen des Objekts. (Weitere Informationen zu impliziten Methodenargumenten finden Sie weiter unten.)

Wie ruft ein Client eine Instanzmethode auf?

Der Name der Methode wird zu einem Unterbefehl des Objekts. Lassen Sie uns beispielsweise einen simulierten Hund auf Herz und Nieren prüfen:

```
% dog spot
::spot
% spot bark
::spot barks.
% spot chase cat
::spot chases cat.
%
```

Wie ruft eine Instanzmethode eine andere Instanzmethode auf?

Wenn Methode A die Methode B für dasselbe Objekt aufrufen muss, tut sie dies genau wie ein Client: Sie ruft Methode B als Unterbefehl des Objekts selbst auf und verwendet dabei den im impliziten Argument **self** gespeicherten Objektnamen.

Nehmen wir zum Beispiel an, dass unsere Hunde niemals etwas jagen, ohne dabei zu bellen:

```
% snit::type dog {
    method bark {} {
        return "$self barks."
    }

    method chase {thing} {
        return "$self chases $thing.  [$self bark]"
    }
}
::dog
% dog spot
```

```
::spot
% spot bark
::spot barks.
% spot chase cat
::spot chases cat.  ::spot barks.
%
```

Gibt es Einschränkungen hinsichtlich der Namen von Instanzmethoden?

Nicht wirklich, solange Sie die Namen der Standard-Instanzmethoden vermeiden: **configure**, **configurelist**, **cget**, **destroy** und **info**. Außerdem definieren Namen von Methoden, die aus mehreren Wörtern bestehen, hierarchische Methoden.

Was ist eine hierarchische Methode?

Die Methoden eines Objekts sind Unterbefehle des Instanzbefehls des Objekts. Hierarchische Methoden erlauben es, dass die Methoden eines Objekts eigene Unterbefehle haben; diese können wiederum Unterbefehle haben und so weiter. So kann der Programmierer eine baumförmige Befehlsstruktur definieren, wie sie viele Tk-Widgets verwenden – die Unterbefehle der Methode **tag** des Tk-Widgets **text** sind hierarchische Methoden.

Wie definiere ich eine hierarchische Methode?

Definieren Sie Methoden, deren Namen aus mehreren Wörtern bestehen. Diese Wörter legen implizit die Hierarchie fest. Der folgende Code definiert beispielsweise eine Methode namens "**tag**" mit den Unterbefehlen "**cget**" und "**configure**":

```
snit::widget mytext {
    method {tag configure} {tag args} { ... }

    method {tag cget} {tag option} {...}
}
```

Hinweis: Es gibt keine explizite Definition für die Methode **tag**; sie ist implizit in den Definitionen von **tag configure** und **tag cget** enthalten. Wenn Sie in diesem Beispiel versuchen würden, "**tag**" explizit zu definieren, würden Sie eine Fehlermeldung erhalten.

Wie rufe ich hierarchische Methoden auf?

Als Unterbefehle von Unterbefehlen.

```
% mytext .text
.text
% .text tag configure redtext -foreground red -background black
% .text tag cget redtext -foreground
red
%
```

Wie kann ich eine Instanzmethode als "private" kennzeichnen?

Häufig ist es sinnvoll, private Methoden zu definieren, d. h. Instanzmethoden, die ausschließlich von anderen Methoden desselben Objekts aufgerufen werden sollen.

Snit implementiert keine Zugriffskontrolle für Instanzmethoden, sodass alle Methoden *de facto*

öffentlich sind. Üblicherweise beginnen die Namen öffentlicher Methoden jedoch mit einem Kleinbuchstaben, während die Namen privater Methoden mit einem Großbuchstaben beginnen.

Nehmen wir beispielsweise an, unsere simulierten Hunde bellen ausschließlich als Reaktion auf andere Reize; sie bellen niemals einfach nur zum Spaß. Daher wird aus der Methode "**bark**" die Methode "**Bark**", um anzuzeigen, dass es sich um eine private Methode handelt:

```
% snit::type dog {
    # Private by convention: begins with uppercase letter.
    method Bark {} {
        return "$self barks."
    }

    method chase {thing} {
        return "$self chases $thing. [$self Bark]"
    }
}
::dog
% dog fido
::fido
% fido chase cat
::fido chases cat. ::fido barks.
%
```

Gibt es Einschränkungen hinsichtlich der Argumente von Instanzmethoden?

Argumentlisten von Methoden werden genauso definiert wie normale Argumentlisten von Tcl-procs; insbesondere können sie Argumente mit Standardwerten und das Argument **args** enthalten.

Jede Methode verfügt jedoch zusätzlich zu den explizit definierten Argumenten über eine Reihe impliziter Argumente, die von Snit bereitgestellt werden. Die Namen dieser impliziten Argumente dürfen nicht zur Benennung expliziter Argumente verwendet werden.

Welche impliziten Argumente werden an jede Instanzmethode übergeben?

Die Argumente, die implizit an jede Methode übergeben werden, lauten **type**, **selfns**, **win** und **self**.

Was ist $type?

Das implizite Argument **type** enthält den vollqualifizierten Namen des Objekttyps:

```
% snit::type thing {
    method mytype {} {
        return $type
    }
}
::thing
% thing something
::something
% something mytype
::thing
%
```

Was ist $self?

Das implizite Argument **self** enthält den vollqualifizierten Namen des Objekts.

Wenn der Befehl des Objekts umbenannt wird, ändert sich "**$self**" bei nachfolgenden Aufrufen entsprechend. Daher sollten Sie in Ihrem Code nicht davon ausgehen, dass "**$self**" eine Konstante ist, es sei denn, Sie wissen mit Sicherheit, dass das Objekt niemals umbenannt wird.

```
% snit::type thing {
    method myself {} {
        return $self
    }
}
::thing
% thing mutt
::mutt
% mutt myself
::mutt
% rename mutt jeff
% jeff myself
::jeff
%
```

Was ist $selfns?

Jedes Snit-Objekt verfügt über einen privaten Namespace, in dem seine Instanzvariablen und Optionen gespeichert werden. Das implizite Argument **selfns** enthält den Namen dieses Namespaces; sein Wert ändert sich nie und bleibt während der gesamten Lebensdauer des Objekts konstant, selbst wenn sich der Name des Objekts ändert:

```
% snit::type thing {
    method myNameSpace {} {
        return $selfns
    }
}
::thing
% thing jeff
```

```
::jeff
% jeff myNameSpace
::thing::Snit_inst3
% rename jeff mutt
% mutt myNameSpace
::thing::Snit_inst3
%
```

Das obige Beispiel verdeutlicht, wie Snit den privaten Namespace einer Instanz benennt; Sie sollten jedoch keinen Code schreiben, der von dieser spezifischen Namenskonvention abhängt, da sich diese in zukünftigen Versionen ändern könnte.

Was ist $win?

Das implizite Argument **win** ist für alle Snit-Methoden definiert, obwohl es eigentlich nur für die Methoden von Widgets und Widget-Adaptern sinnvoll ist (siehe Widgets und Widget-Adapter). **$win** ist schlichtweg der ursprüngliche Name des Objekts, unabhängig davon, ob es umbenannt wurde oder nicht. Bei Widgets und Widget-Adaptern ist es daher auch der Name eines Tk-Fensters.

Wenn ein snit::widgetadaptor verwendet wird, um die Schnittstelle eines Widgets oder Megawidgets anzupassen, muss es den ursprünglichen Befehl des Widgets umbenennen und durch einen eigenen ersetzen.

Wenn Sie also **win** verwenden, wann immer der Name des Tk-Fensters benötigt wird, bedeutet dies, dass ein snit::widget oder snit::widgetadaptor durch einen snit::widgetadaptor angepasst werden kann. Weitere Informationen finden Sie unter Widgets.

Wie übergebe ich eine Instanzmethode als Rückruffunktion?

Das hängt vom Kontext ab.

Angenommen, ich habe in meiner Anwendung ein **dog**-Objekt mit dem Namen **fido**, und ich möchte, dass "**fido**" bellt, wenn eine Tk-Schaltfläche namens "**.bark**" gedrückt wird. In diesem Fall erstelle ich den Rückrufbefehl wie gewohnt unter Verwendung von "**list**":

```
    button .bark -text "Bark!" -command [list fido bark]
```

Im typischen Tcl-Stil verwenden wir eine Rückruffunktion, um zwei unabhängige Komponenten miteinander zu verknüpfen. Angenommen jedoch, das Objekt **dog** verfügt über eine grafische Benutzeroberfläche und besitzt die Schaltfläche selbst? In diesem Fall muss **dog** eine seiner eigenen Instanzmethoden an die Schaltfläche übergeben, die es besitzt. Die naheliegende Vorgehensweise ist folgende:

```
% snit::widget dog {
    constructor {args} {
        #...
        button $win.barkbtn -text "Bark!" -command [list $self bark]
        #...
    }
}
::dog
%
```

(Hinweis: In diesem Beispiel wird aus unserem **dog** ein snit::widget, da es über GUI-Funktionen verfügt. Weitere Informationen finden Sie unter Widgets.) Wenn wir also ein **dog** mit dem Namen **.spot** erstellen, wird dadurch eine Tk-Schaltfläche mit dem Namen **.spot.barkbtn** erstellt; beim Drücken ruft die Schaltfläche **$self bark** auf.

Nun, das wird funktionieren – vorausgesetzt, **.spot** wird niemals umbenannt. Aber ist das Umbenennen von Widgets nicht ungewöhnlich? In der Tat – es sei denn, **.spot** ist die Hull-Komponente eines snit::widgetadaptor. Wenn dies der Fall ist, wird es umbenannt, und "**.spot**" wird zum Namen des Objekts "snit::widgetadaptor". Wenn die Schaltfläche gedrückt wird, wird der Befehl **$self bark** von snit::widgetadaptor verarbeitet, was möglicherweise das Richtige bewirkt – oder auch nicht.

Es gibt eine sicherere Vorgehensweise, und die sieht folgendermaßen aus:

```
% snit::widget dog {
    constructor {args} {
        #...
        button $win.barkbtn -text "Bark!" -command [mymethod bark]
        #...
    }
}
::dog
%
```

Der Befehl mymethod akzeptiert eine beliebige Anzahl von Argumenten und kann wie **list** zum Erstellen eines Rückrufbefehls verwendet werden; der einzige Unterschied besteht darin, dass mymethod eine Form des Befehls zurückgibt, die sich auch dann nicht ändert, wenn sich der Name der Instanz ändert.

Andererseits möchten Sie vielleicht lieber zulassen, dass ein Widget-Adapter eine Methode überschreibt, sodass Ihr umbenanntes Widget die Methode des Widget-Adapters anstelle seiner eigenen aufruft. In diesem Fall wird die Verwendung von **[list $self bark]** das gewünschte Ergebnis erzielen … allerdings handelt es sich hierbei um eine Technik, die nur unter sorgfältig kontrollierten Umständen angewendet werden sollte.

Wie leite ich Instanzmethoden an eine Komponente weiter?

Siehe Weiterleitung.

Instanzvariablen

Was ist eine Instanzvariable?

Eine Instanzvariable ist eine private Variable, die einem bestimmten Snit-Objekt zugeordnet ist. Instanzvariablen können Skalare oder Arrays sein.

Wie wird eine skalare Instanzvariable definiert?

Skalare Instanzvariablen werden in der Typdefinition mithilfe der Anweisung variable definiert. Sie können ihnen einfach einen Namen geben oder sie mit einem Wert initialisieren:

```
snit::type mytype {
    # Define variable "greeting" and initialize it with "Howdy!"
    variable greeting "Howdy!"
}
```

Wie wird eine Array-Instanzvariable definiert?

Array-Instanzvariablen werden ebenfalls in der Typdefinition mithilfe des Befehls variable definiert. Sie können diese gleichzeitig initialisieren, indem Sie die Option **-array** angeben:

```
snit::type mytype {
    # Define array variable "greetings"
    variable greetings -array {
        formal "Good Evening"
        casual "Howdy!"
    }
}
```

Was geschieht, wenn ich eine Instanzvariable nicht initialisiere?

Variablen existieren erst dann wirklich, wenn ihnen Werte zugewiesen wurden. Wenn Sie eine Variable bei ihrer Definition nicht initialisieren, müssen Sie darauf achten, ihr einen Wert zuzuweisen (beispielsweise im Konstruktor oder in einer Methode), bevor Sie darauf zugreifen.

Gibt es Einschränkungen hinsichtlich der Namen von Instanzvariablen?

Nur ein paar.

Erstens verfügt jedes Snit-Objekt über eine integrierte Instanzvariable namens **options**, die niemals neu definiert werden sollte.

Zweitens sind alle Namen, die mit "Snit_" beginnen, für die Verwendung durch den internen Code von Snit reserviert.

Drittens dürften Instanzvariablennamen, die das Namespace-Trennzeichen enthalten (**::**), große Verwirrung stiften.

Muss ich meine Instanzvariablen in meinen Methoden deklarieren?

Nein. Sobald Sie eine Instanzvariable in der Typdefinition definiert haben, kann diese in jedem Instanzcode (Instanzmethoden, Konstruktor und Destruktor) ohne separate Deklaration verwendet werden. Dies weicht von der üblichen Tcl-Praxis ab, bei der alle nicht-lokalen Variablen in einer Prozedur deklariert werden müssen.

Die Tatsache, dass alle Instanzvariablen implizit im gesamten Instanzcode verfügbar sind, geht mit einem Geschwindigkeitsverlust einher. Auch wenn Sie die Variablen in Ihrem Code nicht explizit deklarieren müssen, muss Snit sie dennoch deklarieren, und das kostet Zeit. Wenn Sie

zehn Instanzvariablen haben, muss eine Methode, die keine davon verwendet, dennoch den Deklarationsaufwand für alle zehn tragen. In den meisten Fällen sind die zusätzlichen Laufzeitkosten vernachlässigbar. In Extremfällen möchten Sie sie möglicherweise vermeiden; dafür gibt es zwei Möglichkeiten.

Die erste Möglichkeit besteht darin, eine einzige Instanzvariable – ein Array – zu definieren und alle Ihre Instanzdaten in diesem Array zu speichern. Auf diese Weise entsteht Ihnen nur der Deklarationsaufwand für eine einzige Variable – und diese benötigen Sie wahrscheinlich ohnehin die meiste Zeit. Diese Methode versagt jedoch, wenn Ihre Instanzvariablen mehrere Arrays enthalten; in Tcl 8.5 könnte Ihnen der Befehl **dict** jedoch Abhilfe schaffen.

Die zweite Methode besteht darin, Ihre Instanzvariablen explizit in Ihrem Instanzcode zu deklarieren, sie jedoch *nicht* in die Typdefinition aufzunehmen:

```
snit::type dog {
    constructor {} {
        variable mood

        set mood happy
    }

    method setmood {newMood} {
        variable mood

        set mood $newMood
    }

    method getmood {} {
        variable mood

        return $mood
    }
}
```

Auf diese Weise können Sie sicherstellen, dass in jeder Methode nur die erforderlichen Variablen enthalten sind – allerdings auf Kosten von längerem Code und Laufzeitfehlern, falls Sie vergessen sollten, eine benötigte Variable zu deklarieren.

Wie übergebe ich den Namen einer Instanzvariablen an ein anderes Objekt?

In Tk ist es üblich, einem Widget einen Namen einer Variablen zu übergeben; beispielsweise verfügen Tk-Label-Widgets über die Option "**-textvariable**", die den Namen der Variablen angibt, die den Text des Widgets enthält. Dadurch kann das Programm den Wert des Labels aktualisieren, indem es der Variable einfach einen neuen Wert zuweist.

Wenn Sie den Namen der Instanzvariablen naiv an das Label-Widget übergeben, wird Sie das Ergebnis verwirren; Tk geht davon aus, dass der Name eine globale Variable bezeichnet. Stattdessen müssen Sie einen vollqualifizierten Variablennamen angeben. Innerhalb einer Instanzmethode oder eines Konstruktors können Sie den Namen der Variablen mithilfe des Befehls myvar vollständig qualifizieren:

```
snit::widget mywidget {
    variable labeltext ""

    constructor {args} {
        # ...

        label $win.label -textvariable [myvar labeltext]

        # ...
    }
}
```

Wie mache ich eine Instanzvariable öffentlich?

In der Praxis tun Sie das nicht. Stattdessen implementieren Sie öffentliche Variablen als Optionen. Alternativ können Sie Instanzmethoden schreiben, um den Wert der Variablen festzulegen und abzurufen.

Optionen

Was ist eine Option?

Die Optionen eines Typs entsprechen dem, was in anderen objektorientierten Sprachen als öffentliche Membervariablen oder Eigenschaften bezeichnet wird: Es handelt sich um Werte, die von den Clients eines Objekts abgerufen und (in der Regel) gesetzt werden können.

Die Implementierung von Optionen in Snit folgt ziemlich genau dem Tk-Modell, mit der Ausnahme, dass snit::type-Objekte in der Regel nicht mit der Tk-Optionsdatenbank interagieren; Objekte vom Typ snit::widget und snit::widgetadaptor hingegen tun dies stets.

Wie definiere ich eine Option?

Optionen werden in der Typdefinition mithilfe der Anweisung **option** definiert. Betrachten Sie den folgenden Typ, der in einer Anwendung verwendet werden soll, die eine Liste von Hunden für eine Tierhandlung verwaltet:

```
snit::type dog {
    option -breed -default mongrel
    option -color -default brown
    option -akc   -default 0
    option -shots -default 0
}
```

Demnach weist ein Hund vier wesentliche Merkmale auf: eine Rasse, eine Farbe, ein Kennzeichen, das angibt, ob er beim American Kennel Club als reinrassig registriert ist, sowie ein weiteres Kennzeichen, das angibt, ob er geimpft wurde. Der Standardhund ist offenbar ein brauner Mischling.

Beim Definieren einer Option können Sie eine Reihe von Optionen angeben; ist **-default** die einzige, können Sie das Wort **-default** wie folgt weglassen:

```
snit::type dog {
    option -breed mongrel
    option -color brown
    option -akc   0
    option -shots 0
}
```

Wenn kein Wert für "**-default**" festgelegt wird, ist der Standardwert der Option eine leere Zeichenkette (siehe jedoch Tk-Optionsdatenbank).

In der Snit-Manpage werden Optionen wie diese als "lokal definierte" Optionen bezeichnet.

Wie kann ein Client bei der Erstellung eines Objekts Optionen festlegen?

Es ist üblich, dass der Client bei der Erstellung eines Objekts nach dem Namen des Objekts eine beliebige Anzahl von Optionen und deren Werte übergeben kann. So kann beispielsweise der in der vorherigen Antwort definierte Befehl "**::dog**" nun zum Erstellen einzelner Hunde verwendet werden. Bei der Erstellung können einzelne oder alle Optionen festgelegt werden.

```
% dog spot -breed beagle -color "mottled" -akc 1 -shots 1
::spot
% dog fido -shots 1
::fido
%
```

::spot ist also ein reinrassiger Beagle; **::fido** ist ein typischer Mischling, doch seine Besitzer kümmern sich offensichtlich gut um ihn, da er geimpft ist.

Hinweis: Wenn der Typ einen Konstruktor definiert, kann er eine andere Syntax zur Objekterstellung festlegen. Weitere Informationen finden Sie unter Konstruktoren.

Wie kann ein Client den Wert einer Option abrufen?

Rufen Sie die Werte der Optionen mithilfe der Methode **cget** ab:

```
% spot cget -color
mottled
% fido cget -breed
mongrel
%
```

Wie kann ein Client Optionen nach dem Erzeugen des Objekts setzen?

Mit der Instanzmethode **configure** können beliebig viele Optionen gleichzeitig festgelegt werden. Nehmen wir an, bei genauerer Betrachtung stellt sich heraus, dass ::fido kein brauner Mischling ist, sondern ein seltener Arctic Boar Hound mit einer wunderschönen graubraunen Farbe:

```
% fido configure -color dun -breed "Arctic Boar Hound"
% fido cget -color
dun
% fido cget -breed
Arctic Boar Hound
```

Alternativ akzeptiert die Methode **configurelist** eine Liste mit Optionen und Werten; gelegentlich ist dies praktischer:

```
% set features [list -color dun -breed "Arctic Boar Hound"]
-color dun -breed {Arctic Boar Hound}
% fido configurelist $features
% fido cget -color
dun
% fido cget -breed
Arctic Boar Hound
%
```

In Tcl 8.5 kann das Schlüsselwort * in diesem Fall zusammen mit **configure** verwendet werden:

```
% set features [list -color dun -breed "Arctic Boar Hound"]
-color dun -breed {Arctic Boar Hound}
% fido configure {*}$features
% fido cget -color
dun
% fido cget -breed
Arctic Boar Hound
%
```

Die Ergebnisse sind dieselben.

Wie sollte eine Instanzmethode auf einen Wert einer Option zugreifen?

Es gibt zwei Möglichkeiten, wie eine Instanzmethode den Wert einer Option festlegen und abrufen kann. Eine davon besteht darin, die Methoden **configure** und **cget** zu verwenden, wie im Folgenden dargestellt.

```
% snit::type dog {
    option -weight 10

    method gainWeight {} {
        set wt [$self cget -weight]
        incr wt
        $self configure -weight $wt
    }
}
::dog
% dog fido
::fido
% fido cget -weight
10
% fido gainWeight
% fido cget -weight
11
%
```

Alternativ stellt Snit eine integrierte Array-Instanzvariable namens **options** zur Verfügung. Die Indizes entsprechen den Namen der Optionen; die Werte entsprechen den Werten der Optionen. Die Methode **gainWeight** lässt sich somit wie folgt umschreiben:

```
    method gainWeight {} {
        incr options(-weight)
    }
```

Wie Sie sehen können, erfordert die Verwendung der Variablen **options** deutlich weniger Tipparbeit und ist die übliche Vorgehensweise. Sollten Sie jedoch **-configuremethod** oder **-cgetmethod** (in den folgenden Antworten beschrieben) verwenden, möchten Sie möglicherweise dennoch die Methoden **configure** und **cget** verwenden, um sicherzustellen, dass alle von Ihnen implementierten speziellen Verarbeitungsschritte auch tatsächlich ausgeführt werden. Wenn die Option zudem an eine Komponente weitergeleitet wurde, sind **configure** und **cget** die einzige Möglichkeit, darauf zuzugreifen, ohne direkt auf die Komponente zuzugreifen. Weitere Informationen finden Sie unter Weiterleitung.

Wie kann ich eine Option auf "Nur-Lesen" setzen?

Definieren Sie die Option mit **-readonly yes**.

Angenommen, Sie verfügen über eine Option, die festlegt, wie Instanzen Ihres Typs erstellt werden; diese muss zum Zeitpunkt der Erstellung festgelegt werden und ist danach unveränderlich. Beispielsweise ändert ein Hund niemals seine Rasse; er kann geimpft sein oder auch nicht, und falls nicht, kann er die Impfungen zu einem späteren Zeitpunkt erhalten. **-breed** sollte schreibgeschützt sein, **-shots** hingegen nicht.

```
% snit::type dog {
    option -breed -default mongrel -readonly yes
    option -shots -default no
}
::dog
% dog fido -breed retriever
::fido
% fido configure -shots yes
% fido configure -breed terrier
option -breed can only be set at instance creation
%
```

Wie kann ich Zugriffe auf den Wert einer Option erfassen?

Definieren Sie für die Option eine **-cgetmethod**.

Was ist eine -cgetmethod?

"**-cgetmethod**" ist eine Methode, die immer dann aufgerufen wird, wenn der Wert der zugehörigen Option über die Instanzmethode "**cget**" abgefragt wird. Der Handler kann den Wert der Option berechnen, ihn aus einer Datenbank abrufen oder jede andere von Ihnen gewünschte Aktion ausführen.

So würde das Standardverhalten aussehen, wenn es unter Verwendung von **-cgetmethod** geschrieben würde:

```
snit::type dog {
    option -color -default brown -cgetmethod GetOption

    method GetOption {option} {
        return $options($option)
    }
}
```

Es kann jede Instanzmethode verwendet werden, sofern sie ein Argument akzeptiert, nämlich den Namen der Option, deren Wert abgerufen werden soll.

Wie kann ich Änderungen am Wert einer Option erfassen?

Definieren Sie für die Option eine **-configuremethod**.

Was ist eine -configuremethod?

"**-configuremethod**" ist eine Methode, die immer dann aufgerufen wird, wenn der zugehörigen Option über die Instanzmethoden "**configure**" oder "**configurelist**" ein neuer Wert zugewiesen wird. Die Methode kann den Wert an ein anderes Objekt weiterleiten, ihn in einer Datenbank speichern oder jede andere von Ihnen gewünschte Aktion ausführen.

So würde das Standard-Verhalten der Konfiguration aussehen, wenn es unter Verwendung von **-configuremethod** geschrieben würde:

```
snit::type dog {
    option -color -default brown -configuremethod SetOption

    method SetOption {option value} {
        set options($option) $value
    }
}
```

Es kann jede Instanzmethode verwendet werden, sofern sie zwei Argumente entgegennimmt: den Namen der Option und den neuen Wert.

Hinweis: Wenn Ihre Methode den Wert nicht im Array "**options**" speichert, wird das Array "**options**" nicht aktualisiert.

Wie kann ich den Wert einer Option validieren?

Definieren Sie eine **-validatemethod**.

Was ist eine -validatemethod?

"**-validatemethod**" ist eine Methode, die immer dann aufgerufen wird, wenn der zugehörigen Option über die Instanzmethoden "**configure**" oder "**configurelist**" ein neuer Wert zugewiesen wird. Es ist Aufgabe der Methode, zu prüfen, ob der neue Wert gültig ist, und andernfalls einen Fehler auszulösen. Die Methode **-validatemethod** wird, sofern vorhanden, aufgerufen, bevor der Wert im Array **options** gespeichert wird; insbesondere wird sie vor der Methode **-configuremethod** aufgerufen, sofern diese vorhanden ist.

Nehmen wir beispielsweise an, eine Option nimmt stets einen booleschen Wert an. Sie können wie folgt sicherstellen, dass der Wert tatsächlich ein gültiger boolescher Wert ist:

```
% snit::type dog {
    option -shots -default no -validatemethod BooleanOption

    method BooleanOption {option value} {
        if {![string is boolean -strict $value]} {
            error "expected a boolean value, got \"$value\""
        }
    }
}
::dog
% dog fido
% fido configure -shots yes
% fido configure -shots NotABooleanValue
expected a boolean value, got "NotABooleanValue"
%
```

Hinweis: Dieselbe **-validatemethod** kann zur Validierung einer beliebigen Anzahl von booleschen Optionen verwendet werden.

Jede Methode kann als **-validatemethod** fungieren, sofern sie zwei Argumente entgegennimmt: den Namen der Option und den neuen Wert der Option.

Typvariablen

Was ist eine Typvariable?

Eine Typvariable ist eine private Variable, die einem Snit-Typ und nicht einer bestimmten Instanz dieses Typs zugeordnet ist. In C++ und Java wird für dasselbe Konzept der Begriff *statische Membervariable* (*static member variable*) verwendet. Typvariablen können Skalare oder Arrays sein.

Wie wird eine skalare Typvariable definiert?

Skalare Typvariablen werden in der Typdefinition mithilfe der Anweisung typevariable definiert. Sie können ihnen einfach einen Namen geben oder sie mit einem Wert initialisieren:

```
snit::type mytype {
    # Define variable "greeting" and initialize it with "Howdy!"
    typevariable greeting "Howdy!"
}
```

Jedes Objekt vom Typ **mytype** hat nun Zugriff auf eine einzelne Variable namens **greeting**.

Wie wird eine Typvariable mit Array-Wert definiert?

Typvariablen mit Array-Wert werden ebenfalls mit dem Befehl typevariable definiert; um sie zu initialisieren, fügen Sie die Option **-array** hinzu:

```
snit::type mytype {
    # Define typearray variable "greetings"
    typevariable greetings -array {
        formal "Good Evening"
```

```
        casual "Howdy!"
    }
}
```

Was geschieht, wenn ich eine Typvariable nicht initialisiere?

Variablen existieren erst dann wirklich, wenn ihnen Werte zugewiesen werden. Wenn Sie eine Variable bei ihrer Definition nicht initialisieren, müssen Sie darauf achten, ihr (beispielsweise im Typkonstruktor) einen Wert zuzuweisen, bevor Sie darauf verweisen.

Gibt es Einschränkungen hinsichtlich der Namen von Typvariablen?

Für Namen von Typvariablen gelten dieselben Einschränkungen wie für Namen von Instanzvariablen.

Muss ich meine Typvariablen in meinen Methoden deklarieren?

Nein. Sobald Sie eine Typvariable in der Typdefinition definiert haben, kann diese in Instanzmethoden oder Typmethoden ohne Deklaration verwendet werden. Dies weicht von der üblichen Tcl-Praxis ab, bei der alle nicht-lokalen Variablen in einer Prozedur deklariert werden müssen.

Typvariablen unterliegen denselben Kompromissen zwischen Geschwindigkeit und Lesbarkeit wie Instanzvariablen; siehe Muss ich meine Instanzvariablen in meinen Methoden deklarieren?

Wie übergebe ich den Namen einer Typvariablen an ein anderes Objekt?

In Tk ist es üblich, einem Widget einen Namen einer Variablen zu übergeben; beispielsweise verfügen Tk-Label-Widgets über die Option "**-textvariable**", die den Namen der Variablen angibt, die den Text des Widgets enthält. Dadurch kann das Programm den Wert des Labels aktualisieren, indem es der Variable einfach einen neuen Wert zuweist.

Wenn Sie dem Label-Widget naiv einen Typvariablennamen übergeben, wird Sie das Ergebnis verwirren; Tk geht davon aus, dass der Name eine globale Variable bezeichnet. Stattdessen müssen Sie einen vollständig qualifizierten Variablennamen angeben. Innerhalb einer Instanzmethode oder eines Konstruktors können Sie den Namen der Typvariablen mithilfe des Befehls mytypevar vollständig qualifizieren:

```
snit::widget mywidget {
    typevariable labeltext ""

    constructor {args} {
        # ...

        label $win.label -textvariable [mytypevar labeltext]

        # ...
    }
}
```

Wie mache ich eine Typvariable öffentlich?

Dafür gibt es zwei Möglichkeiten. Die bevorzugte Vorgehensweise besteht darin, zwei Typmethoden zu schreiben, um den Wert der Typvariablen festzulegen und abzufragen.

Typvariablen werden im Namespace des Typs gespeichert, der denselben Namen wie der Typ selbst trägt. Daher können Sie den Namen der Typvariablen auch in Ihrer Dokumentation veröffentlichen, damit Clients direkt darauf zugreifen können. Zum Beispiel:

```
snit::type mytype {
    typevariable myvariable
}

set ::mytype::myvariable "New Value"
```

Typmethoden

Was ist eine Typmethode?

Eine Typmethode ist eine Prozedur, die nicht mit einer bestimmten Instanz des Typs, sondern mit dem Typ selbst verknüpft ist und als Unterbefehl des Typbefehls aufgerufen wird.

Wie definiere ich eine Typmethode?

Typmethoden werden in der Typdefinition mithilfe der Anweisung **typemethod** definiert:

```
snit::type dog {
    # List of pedigreed dogs
    typevariable pedigreed

    typemethod pedigreedDogs {} {
        return $pedigreed
    }
}
```

Angenommen, der Typ "**dog**" verwaltet eine Liste mit den Namen der Hunde, die über einen Stammbaum verfügen. Die Typmethode **pedigreedDogs** gibt diese Liste zurück.

Die Anweisung "**typemethod**" sieht genauso aus wie eine normale Tcl-Anweisung "proc", mit dem Unterschied, dass sie in einer snit::type-Definition vorkommt. Beachten Sie, dass jede Typmethode ein implizites Argument namens **type** erhält, das den vollqualifizierten Typnamen enthält.

Wie ruft ein Client eine Typmethode auf?

Der Name der Typmethode wird zu einem Unterbefehl des Typbefehls. Nehmen wir beispielsweise an, dass der Konstruktor jeden Rassehund zur Liste von **pedigreedDogs** hinzufügt,

```
snit::type dog {
    option -pedigreed 0

    # List of pedigreed dogs
    typevariable pedigreed

    typemethod pedigreedDogs {} {
        return $pedigreed
```

```
    }

    # ...
}

dog spot -pedigreed 1
dog fido

foreach dog [dog pedigreedDogs] { ... }
```

Gibt es Einschränkungen hinsichtlich der Namen von Typmethoden?

Nicht wirklich, solange Sie die Namen der Standard-Typmethoden vermeiden: **create**, **destroy** und **info**.

Wie kann ich eine Typmethode als privat deklarieren?

Manchmal ist es sinnvoll, private Typmethoden zu definieren, d. h. Typmethoden, die ausschließlich von anderen Typ- oder Instanzmethoden desselben Objekts aufgerufen werden sollen.

Snit sieht keine Zugriffskontrolle für Typmethoden vor; gemäß Konvention beginnen die Namen öffentlicher Methoden mit einem Kleinbuchstaben, während die Namen privater Methoden mit einem Großbuchstaben beginnen.

Alternativ kann ein Snit proc als private Typmethode verwendet werden; siehe Prozeduren.

Gibt es Einschränkungen hinsichtlich der Argumente von Typmethoden?

Argumentlisten für Methoden werden genau wie normale Argumentlisten für Tcl-Prozeduren definiert; insbesondere können sie Argumente mit Standardwerten sowie das Argument **args** enthalten.

Jede Typmethode wird jedoch mit einem impliziten Argument namens **type** aufgerufen, das den Namen des Typbefehls enthält. Darüber hinaus sollten Typmethoden gemäß der Konvention die Verwendung von Namen vermeiden, die implizit für Instanzmethoden definiert sind.

Wie ruft eine Instanz- oder Typmethode eine Typmethode auf?

Wenn eine Instanz- oder Typmethode eine Typmethode aufrufen muss, sollte sie hierfür **$type** verwenden:

```
snit::type dog {

    typemethod pedigreedDogs {} { ... }

    typemethod printPedigrees {} {
        foreach obj [$type pedigreedDogs] { ... }
    }
}
```

Wie übergebe ich eine Typmethode als Rückruffunktion?

In Tcl ist es üblich, einen Codeausschnitt an ein anderes Objekt zu übergeben, damit dieses ihn

später aufruft. Da Typen nicht umbenannt werden können, können Sie einfach den Typnamen verwenden oder – wenn die Rückruffunktion aus einer Typmethode heraus registriert wird – **type** . Nehmen wir beispielsweise an, wir möchten eine Liste von Rassehunden ausgeben, wenn eine Tk-Schaltfläche gedrückt wird:

```
button .btn -text "Pedigrees" -command [list dog printPedigrees]
pack .btn
```

Alternativ können Sie aus einer Methode oder einer Typmethode den Befehl mytypemethod verwenden, genauso wie Sie mymethod verwenden würden, um einen Rückrufbefehl für Instanzmethoden zu definieren.

Können Typmethoden hierarchisch sein?

Ja, Sie können hierarchische Typmethoden auf genau dieselbe Weise definieren wie hierarchische Instanzmethoden. Weitere Informationen finden Sie unter Instanzmethoden.

Prozeduren

Was ist eine Prozedur?

Ein "Snit proc" ist im Grunde nichts anderes als eine Tcl-Prozedur, die innerhalb des Namespaces des Typs definiert ist. Sie können Prozeduren für privaten Code verwenden, der nicht mit einer bestimmten Instanz in Verbindung steht.

Wie definiere ich eine Prozedur?

Prozeduren werden definiert, indem eine proc-Anweisung in die Typdefinition aufgenommen wird:

```
snit::type mytype {
    # Pops and returns the first item from the list stored in the
    # listvar, updating the listvar
   proc pop {listvar} { ... }

   # ...
}
```

Gibt es Einschränkungen hinsichtlich der Namen von Prozeduren?

Es kann jeder beliebige Name verwendet werden, solange er nicht mit "**Snit_**" beginnt; Namen, die mit "**Snit_**" beginnen, sind für die eigene Verwendung durch Snit reserviert. Der umsichtige Programmierer wird jedoch proc-Namen (**set**, **list**, **if** usw.) vermeiden, die Standardbefehle von Tcl verdecken würden.

Da procs privat sind, sollten ihre Namen gemäß der Konvention mit einem Großbuchstaben beginnen; da es jedoch in der Regel keine öffentlichen procs im Namespace des Typs gibt, spielt dies letztlich keine große Rolle.

Wie ruft eine Methode eine Prozedur auf?

Genau wie beim Aufruf eines beliebigen Tcl-Befehls. Zum Beispiel:

```
snit::type mytype {
    # Pops and returns the first item from the list stored in the
    # listvar, updating the listvar
    proc pop {listvar} { ... }

    variable requestQueue {}

    # Get one request from the queue and process it.
    method processRequest {} {
        set req [pop requestQueue]
    }
}
```

Wie kann ich eine Prozedur als Rückruffunktion an ein anderes Objekt übergeben?

Der Befehl myproc gibt einen Rückrufbefehl für die proc zurück, genau wie mymethod dies für eine Methode tut.

Typkonstruktoren

Was ist ein Typkonstruktor?

Ein Typkonstruktor ist ein Codeabschnitt, der den Typ als Ganzes initialisiert, ähnlich wie ein statischer Initialisierer in C++. Der Codeabschnitt eines Typkonstruktors wird einmalig bei der Definition des Typs ausgeführt und danach nie wieder.

Ein Typ kann höchstens einen Typkonstruktor haben.

Wie definiere ich einen Typkonstruktor?

Ein Typkonstruktor wird durch die Verwendung der Anweisung typeconstructor in der Typdefinition definiert. Nehmen wir beispielsweise an, der Typ verwendet eine Typvariable mit Array-Wert als Nachschlagetabelle, und die Werte im Array müssen beim Programmstart berechnet werden.

```
% snit::type mytype {
    typevariable lookupTable

    typeconstructor {
        array set lookupTable {key value...}
    }
}
```

Konstruktoren

Was ist ein Konstruktor?

In der objektorientierten Programmierung ist der Konstruktor eines Objekts dafür zuständig, das Objekt bei seiner Erstellung vollständig zu initialisieren. Der Konstruktor erhält die Liste der Optionen, die an die Methode **create** des Befehls snit::type übergeben wurden, und kann anschließend beliebige Aktionen ausführen. Dazu können beispielsweise die Berechnung von

Werten für Instanzvariablen, das Einlesen von Daten aus Dateien, das Anlegen weiterer Objekte, die Aktualisierung von Typ- und Instanzvariablen sowie weitere Vorgänge gehören.

Der Rückgabewert des Konstruktors wird ignoriert (es sei denn, es handelt sich natürlich um einen Fehler).

Wie definiere ich einen Konstruktor?

Ein Konstruktor wird mithilfe der Anweisung constructor in der Typdefinition definiert. Angenommen, es soll eine Liste aller Rassehunde geführt werden. Diese Liste kann in einer Typvariablen verwaltet und über eine Typmethode abgerufen werden. Immer wenn ein Hund angelegt wird, kann er sich selbst in die Liste eintragen – vorausgesetzt, er ist beim American Kennel Club registriert.

```
% snit::type dog {
    option -akc 0

    typevariable akcList {}

    constructor {args} {
        $self configurelist $args

        if {$options(-akc)} {
            lappend akcList $self
        }
    }

    typemethod akclist {} {
        return $akcList
    }
}
::dog
% dog spot -akc 1
::spot
% dog fido
::fido
% dog akclist
::spot
%
```

Was bewirkt der Standard-Konstruktor?

Wenn Sie keinen Konstruktor explizit angeben, wird der Standardkonstruktor verwendet, der mit dem hier gezeigten, explizit definierten Konstruktor identisch ist:

```
snit::type dog {
    constructor {args} {
        $self configurelist $args
    }
}
```

Wenn der Konstruktor aufgerufen wird, wird **args** auf die Liste der Argumente gesetzt, die dem Namen des Objekts folgen. Der Konstruktor darf diese Liste nach eigenem Ermessen interpretieren; üblicherweise wird davon ausgegangen, dass es sich um eine Liste von Namen und Werten für Optionen handelt, wie im obigen Beispiel gezeigt. Wenn Sie lediglich die Werte für Optionen speichern möchten, sollten Sie die Methode **configurelist** verwenden, wie dargestellt.

Kann ich für den Konstruktor andere Argumente auswählen?

Ja, das ist möglich. Nehmen wir beispielsweise an, wir möchten sicherstellen, dass die Rasse für jeden Hund zum Zeitpunkt der Erstellung ausdrücklich angegeben wird und danach nicht mehr geändert werden kann. Eine Möglichkeit, dies zu erreichen, ist folgende:

```
% snit::type dog {
    variable breed

    option -color brown
    option -akc 0

    constructor {theBreed args} {
        set breed $theBreed
        $self configurelist $args
    }

    method breed {} { return $breed }
}
::dog
% dog spot dalmatian -color spotted -akc 1
::spot
% spot breed
dalmatian
```

Der Nachteil besteht darin, dass diese Syntax nicht den Standards entspricht und die Kompatibilität Ihres neuen Typs mit dem Code anderer Entwickler einschränken könnte. Snit geht beispielsweise davon aus, dass es Komponenten mithilfe der Standard-Erstellungssyntax erstellen kann.

Gibt es Einschränkungen hinsichtlich der Argumente des Konstruktors?

Für Argumentlisten von Konstruktoren gelten dieselben Einschränkungen wie für Argumentlisten von Instanzmethoden. Sie weisen dieselben impliziten Argumente auf und können Standardwerte sowie das Argument **args** enthalten.

Gibt es beim Schreiben des Konstruktors etwas Besonderes zu beachten?

Ja. Das Schreiben des Konstruktors kann schwierig sein, wenn Sie Optionen an Komponenten weiterleiten, und es gibt spezifische Probleme im Zusammenhang mit snit::widgets und snit::widgetadaptors. Siehe Weiterleitung, Widgets, Widget-Adapter und Tk-Optionsdatenbank.

Destruktoren

Was ist ein Destruktor?

Ein Destruktor ist eine besondere Art von Methode, die aufgerufen wird, wenn ein Objekt zerstört wird. Er ist dafür zuständig, alle notwendigen Aufräumarbeiten durchzuführen, wenn das Objekt entfernt wird: Komponenten zu zerstören, Dateien zu schließen und so weiter.

Wie definiere ich einen Destruktor?

Destruktoren werden mithilfe der Anweisung destructor in der Typdefinition definiert.

Nehmen wir an, wir führen eine Liste mit Rassehunden; dann möchten wir Hunde aus dieser Liste entfernen, wenn sie zerstört werden.

```
snit::type dog {
    option -akc 0

    typevariable akcList {}

    constructor {args} {
        $self configurelist $args

        if {$options(-akc)} {
            lappend akcList $self
        }
    }

    destructor {
        set ndx [lsearch $akcList $self]

        if {$ndx != -1} {
            set akcList [lreplace $akcList $ndx $ndx]
        }
    }

    typemethod akclist {} {
        return $akcList
    }
}
```

Gibt es Einschränkungen hinsichtlich der Argumente von Destruktoren?

Ja; ein Destruktor hat keine expliziten Argumente.

Welche impliziten Argumente werden an den Destruktor übergeben?

Der Destruktor erhält dieselben impliziten Argumente, die an **INSTANZMETHODEN** übergeben werden: **type**, **selfns**, **win** und **self**.

Müssen Komponenten explizit zerstört werden?

Ja und nein.

Alle Tk-Widgets, die von einem snit::widget oder snit::widgetadaptor erstellt wurden, werden von Tk automatisch zerstört, wenn das Megawidget zerstört wird – dies entspricht dem üblichen

Tk-Verhalten (die Zerstörung eines übergeordneten Widgets führt zur Zerstörung des gesamten Baums).

Komponenten normaler **snit::types** hingegen werden niemals automatisch zerstört, ebenso wenig wie Nicht-Widget-Komponenten von Snit-Megawidgets. Wenn Ihr Objekt diese in seinem Konstruktor erstellt, sollte es sie in der Regel in seinem Destruktor wieder löschen.

Gibt es etwas Besonderes beim Schreiben eines Destruktors zu beachten?

Ja. Wenn der Konstruktor eines Objekts einen Fehler auslöst, wird der Destruktor des Objekts zur Bereinigung aufgerufen; dies bedeutet, dass das Objekt zum Zeitpunkt des Aufrufs des Destruktors möglicherweise noch nicht vollständig instanziiert ist. Dies kann dazu führen, dass der Destruktor einen eigenen Fehler auslöst; das Ergebnis ist in der Regel irreführend, verwirrend und wenig hilfreich. Daher ist es wichtig, Ihren Destruktor so zu schreiben, dass er ausfallsicher ist.

Beispielsweise könnte ein **dog** eine **tail**-Komponente erstellen; diese Komponente muss dann gelöscht werden. Angenommen jedoch, bei der Verarbeitung der Erstellungsoptionen tritt ein Fehler auf – dann wird der Destruktor aufgerufen, und es gibt kein **tail**, das zerstört werden könnte. Die einfachste Lösung besteht in der Regel darin, etwaige Fehler beim Zerstören von Komponenten abzufangen und zu ignorieren.

```
snit::type dog {
    component tail

    constructor {args} {
        $self configurelist $args

        set tail [tail %AUTO%]
    }

    destructor {
        catch {$tail destroy}
    }
}
```

Komponenten

Was ist eine Komponente?

Häufig erstellt und verwaltet ein Objekt eine Reihe weiterer Objekte. Ein Snit-Megawidget erstellt beispielsweise oft eine Reihe von Tk-Widgets. Diese Objekte sind Teil des Hauptobjekts; es setzt sich aus ihnen zusammen, weshalb sie als Komponenten des Objekts bezeichnet werden.

Snit verwendet **COMPONENT** jedoch auch in einer genaueren Bedeutung. Die Komponenten eines Snit-Objekts sind jene Objekte, an die Methoden oder Optionen weitergeleitet werden können. (Weitere Informationen zur Weiterleitung finden Sie unter Weiterleitung.)

Wie deklariere ich eine Komponente?

Zunächst müssen Sie festlegen, welche Rolle eine Komponente innerhalb Ihres Objekts spielt, und dieser Rolle einen Namen geben. Anschließend deklarieren Sie die Komponente unter Verwendung ihres Rollennamens und der Anweisung component. Die Anweisung component

deklariert eine *Instanzvariable*, in der der Name des Befehls der Komponente gespeichert wird, wenn die Komponente angelegt wird.

Nehmen wir beispielsweise an, Ihr Objekt "**dog**" erstellt ein Objekt "**tail**" (zweifellos, um damit besser herumwedeln zu können):

```
snit::type dog {
    component mytail

    constructor {args} {
        # Create and save the component's command
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    method wag {} {
        $mytail wag
    }
}
```

Wie hier zu sehen ist, spielt es keine Rolle, wie der tatsächliche Name des Objekts "**tail**" lautet; das Objekt "**dog**" verweist über den Komponentennamen darauf.

Das obige Beispiel zeigt eine Möglichkeit, die Methode **wag** an die Komponente **mytail** weiterzuleiten; eine einfachere Vorgehensweise finden Sie unter Weiterleitung.

Wie wird eine Komponente benannt?

Eine Komponente hat zwei Namen. Der erste Name ist der der Komponentenvariablen; dieser steht für die Rolle, die das Komponentenobjekt innerhalb des Snit-Objekts spielt. Dies ist der eigentliche Komponentenname und der Name, mit dem im Snit-Code auf die Komponente Bezug genommen wird. Der zweite Name ist der Name des eigentlichen Komponentenobjekts, das vom Konstruktor des Snit-Objekts erstellt wurde. Dieser zweite Name ist stets ein Tcl-Befehlsname und wird als Objektname der Komponente bezeichnet.

Im Beispiel der vorherigen Frage lautet der Name der Komponente "**mytail**"; der Objektname der Komponente **mytail** wird von Snit automatisch gewählt, da beim Erstellen des Komponentenobjekts **%AUTO%** verwendet wurde.

Gibt es Einschränkungen hinsichtlich der Namen von Komponenten?

Ja. snit::widget- und snit::widgetadaptor-Objekte verfügen über eine spezielle Komponente, die **hull**-Komponente; daher sollte der Name **hull** für keinen anderen Zweck verwendet werden.

Da es sich bei den Namen der Komponenten im Übrigen tatsächlich um Namen von Instanzvariablen handelt, müssen sie den Regeln für Instanzvariablen entsprechen.

Was ist eine eigene Komponente?

Eine *eigene* Komponente ist eine Komponente, deren Objektbefehl in seiner Lebensdauer von dem snit::type oder snit::widget gesteuert wird.

Wie oben bereits erwähnt, ist eine Komponente ein Objekt, an das unser Objekt Methoden oder Optionen weiterleiten kann. Nach dieser Definition erstellt unser Objekt in der Regel seine eigenen Komponentenobjekte, was jedoch nicht zwingend erforderlich ist. Betrachten Sie

folgendes Beispiel: Ein Hundeobjekt verfügt über eine Schwanzkomponente; die Schwanzkomponente weiß jedoch, dass sie Teil des Hundes ist:

```
snit::type dog {
    component mytail

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }

    destructor {
        catch {$mytail destroy}
    }

    delegate method wagtail to mytail as wag

    method bark {} {
        return "$self barked."
    }
}

 snit::type tail {
     component mydog
     option -partof -readonly yes

     constructor {args} {
         $self configurelist $args
         set mydog $options(-partof)
     }

     method wag {} {
         return "Wag, wag."
     }

     method pull {} {
         $mydog bark
     }
 }
```

Wenn Sie also einen Hund auffordern, mit dem Schwanz zu wedeln, weist er seinen Schwanz an, zu wedeln; und wenn Sie am Schwanz des Hundes ziehen, weist der Schwanz den Hund an, zu bellen. In diesem Szenario ist der Schwanz eine Komponente des Hundes und der Hund eine Komponente des Schwanzes, doch der Hund besitzt den Schwanz und nicht umgekehrt.

Was bewirkt der Befehl "install"?

Der Befehl install erstellt mithilfe eines angegebenen Befehls eine eigene Komponente und weist

das Ergebnis der Instanzvariablen der Komponente zu. Beispiel:

```
snit::type dog {
    component mytail

    constructor {args} {
        # set mytail [tail %AUTO% -partof $self]
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }
}
```

Im Code eines snit::type entspricht der oben gezeigte Befehl install dem Befehl **set mytail**, der auskommentiert ist. Im Code von "snit::widget" oder "snit::widgetadaptor" hingegen fragt der Befehl install zusätzlich die Tk-Optionsdatenbank ab und initialisiert die Optionen der neuen Komponente entsprechend. Aus Gründen der Konsistenz empfiehlt es sich, sich anzugewöhnen, für alle eigenen Komponenten den Befehl install zu verwenden.

Müssen eigene Komponenten im Konstruktor angelegt werden?

Nein, nicht unbedingt. Tatsächlich gibt es keinen Grund, warum ein Objekt eine Komponente im Laufe seiner eigenen Lebensdauer nicht mehrfach zerstören und neu erstellen sollte.

Gibt es Einschränkungen hinsichtlich der Namen von Komponentenobjekten?

Ja.

Komponentenobjekte, bei denen es sich um Tk-Widgets oder Megawidgets handelt, müssen gültige Tk-Fensternamen aufweisen.

Komponentenobjekte, die keine Widgets oder Megawidgets sind, müssen über vollständig qualifizierte Befehlsnamen verfügen, d. h. Namen, die den vollständigen Namespace des Befehls enthalten. Beachten Sie, dass Snit Objekte stets mit vollständig qualifizierten Namen erstellt.

Außerdem müssen die Objektnamen der Komponenten, die sich im Besitz Ihres Objekts befinden, eindeutig sein. Für Widget-Komponenten stellt dies kein Problem dar, da Widget-Namen stets eindeutig sind; betrachten Sie jedoch den folgenden Code:

```
snit::type tail { ... }

snit::type dog {
    delegate method wag to mytail

    constructor {} {
        install mytail using tail mytail
    }
}
```

Dieser Code verwendet den Namen der Komponente "**mytail**" als Namen des Objekts der Komponente. Dies ist nicht sinnvoll, und zwar aus folgendem Grund: Der Code einer Snit-Instanz wird im Namespace des Snit-Typs ausgeführt. In diesem Fall wird die Komponente "**mytail**" im Namespace "**::dog::**" erstellt und erhält somit den Namen "**::dog::mytail**".

Nehmen wir nun an, Sie erstellen zwei Hunde. Beide Hunde werden versuchen, einen Schwanz

namens **::dog::mytail** zu erstellen. Der erste wird dabei erfolgreich sein, der zweite hingegen nicht, da Snit die Erstellung eines Objekts nicht zulässt, wenn dessen Name bereits ein Befehl ist. Hier sind zwei Möglichkeiten, diese Situation zu vermeiden:

Erstens: Wenn es sich bei dem Komponententyp um "snit::type" handelt, können Sie "**%AUTO%** " als Namen festlegen und haben so die Gewissheit, einen eindeutigen Namen zu erhalten. Dies ist die sicherste Vorgehensweise:

```
    install mytail using tail %AUTO%
```

Wenn der Komponententyp nicht "snit::type" lautet, können Sie die Komponente im Namespace der Instanz des Objekts anlegen:

```
    install mytail using tail ${selfns}::mytail
```

Achten Sie darauf, einen eindeutigen Namen innerhalb des Namespaces der Instanz zu wählen.

Muss ich meine eigenen Komponenten zerstören?

Das hängt davon ab. Wenn ein übergeordnetes Widget zerstört wird, werden alle untergeordneten Widgets automatisch zerstört. Wenn Ihr Objekt also ein snit::widget oder snit::widgetadaptor ist, müssen Sie keine Komponenten-Widgets löschen, da diese in der Regel Kinder oder Nachkommen Ihres Megawidgets sind.

Sollte Ihr Objekt jedoch eine Instanz von snit::type sein, wird keine seiner eigenen Komponenten automatisch zerstört; ebenso wenig werden Nicht-Widget-Komponenten eines snit::widget automatisch zerstört. Alle derartigen eigenen Komponenten müssen explizit zerstört werden, andernfalls werden sie überhaupt nicht zerstört.

Kann ich den Objektbefehl einer Komponente als Teil meiner Schnittstelle verfügbar machen?

Ja, und es gibt zwei Möglichkeiten, dies zu bewerkstelligen. Die geeignetste Vorgehensweise ist in der Regel die Weiterleitung (siehe Weiterleitung). Durch Weiterleitung können Sie die von Ihnen festgelegten Optionen und Methoden an bestimmte Komponenten weitergeben. Dadurch werden die Komponenten für die Nutzer Ihres Typs effektiv verborgen, was eine gute Kapselung gewährleistet.

Es gibt jedoch Fälle, in denen es angebracht – und obendrein einfacher – ist, die gesamte Komponente einfach zum Teil der öffentlichen Schnittstelle Ihres Typs zu machen.

Wie kann ich den Objektbefehl einer Komponente verfügbar machen?

Geben Sie bei der Deklaration der Komponente die Option "**-public**" der Anweisung "component " an. Der Wert dieser Option ist der Name einer Methode, die an den Objektbefehl Ihrer Komponente weitergeleitet wird.

Nehmen wir beispielsweise an, Sie haben ein Combobox-Megawidget erstellt, das ein listbox-Widget enthält, und Sie möchten die gesamte Schnittstelle der Listbox öffentlich machen. Das können Sie wie folgt bewerkstelligen:

```
snit::widget combobox {
     component listbox -public listbox

     constructor {args} {
```

```
          install listbox using listbox $win.listbox ....
     }
}

combobox .mycombo
.mycombo listbox configure -width 30
```

Ihr Combobox-Widget "**.mycombo**" verfügt nun über die Methode "**listbox**", die dieselben Unterbefehle wie das listbox-Widget selbst enthält. Der obige Code legt somit die Breite der Listbox-Komponente auf 30 fest.

In der Regel wählen Sie als Namen der Methode denselben Namen wie den der Komponente; Sie können die Methode jedoch beliebig benennen.

Typkomponenten

Was ist eine Typkomponente?

Eine Typkomponente ist eine Komponente, die nicht zu einer bestimmten Instanz des Typs, sondern zum Typ selbst gehört. Die Beziehung zwischen Komponenten und Typkomponenten entspricht der Beziehung zwischen Instanzvariablen und Typvariablen. Sowohl Instanzmethoden als auch Typmethoden können an Typkomponenten weitergeleitet werden.

Sobald Sie Komponenten und Weiterleitung verstanden haben, sind Typkomponenten im Grunde nichts Neues.

Wie deklariere ich eine Typkomponente?

Deklarieren Sie eine Typkomponente mithilfe der Anweisung typecomponent. Sie akzeptiert dieselben Optionen (**-inherit** und **-public**) wie die Anweisung component und definiert eine Typvariable, die den Objektbefehl der Typkomponente enthält.

Angenommen, in Ihrem Modell gibt es viele Hunde, aber nur einen Tierarzt. Sie könnten den Tierarzt als Typkomponente definieren.

```
snit::type veterinarian { ... }

snit::type dog {
    typecomponent vet

    # ...
}
```

Wie installiere ich eine Typkomponente?

Verwenden Sie einfach den Befehl **set**, um den Objektbefehl der Komponente der Typkomponente zuzuweisen. Da Typen (auch snit::widget-Typen) keine Widgets sind und ohnehin keine Optionen besitzen, werden die zusätzlichen Funktionen des Befehls install nicht benötigt.

In der Regel installieren Sie Typkomponenten im Typkonstruktor, wie hier gezeigt:

```
snit::type veterinarian { ... }

snit::type dog {
    typecomponent vet

    typeconstructor {
        set vet [veterinarian %AUTO%]
    }
}
```

Gibt es Einschränkungen hinsichtlich der Namen von Typkomponenten?

Ja, genau wie bei Instanzvariablen, Typvariablen und normalen Komponenten.

Weiterleitung

Was ist Weiterleitung?

Einfach ausgedrückt bedeutet Weiterleitung, dass Sie eine Ihnen übertragene Aufgabe an einen Ihrer Assistenten weitergeben. (Sie haben doch Assistenten, oder?) Snit-Objekte können dasselbe tun. Das folgende Beispiel zeigt eine Möglichkeit, wie das Objekt "**dog**" seine Methode "**wag**" und seine Option "**-taillength**" an seine Komponente "**tail**" weiterleiten kann.

```
snit::type dog {
    variable mytail

    option -taillength -configuremethod SetTailOption -cgetmethod
    GetTailOption

    method SetTailOption {option value} {
         $mytail configure $option $value
    }

    method GetTailOption {option} {
         $mytail cget $option
    }

    method wag {} {
        $mytail wag
    }

    constructor {args} {
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }

}
```

Dies ist zwar der umständliche Weg, verdeutlicht jedoch, worum es bei der Weiterleitung eigentlich geht. Die einfache Vorgehensweise finden Sie in den folgenden Antworten.

Hinweis: Der Konstruktor ruft die Methode **configurelist** erst auf, **nachdem** er sein **tail** angelegt hat; andernfalls würde ein Fehler auftreten, wenn "**-taillength**" in der Liste von "**args**" enthalten wäre.

Wie leite ich eine Methode an ein Komponentenobjekt weiter?

Da die Weiterleitung so häufig vorkommt, macht Snit sie Ihnen leicht. Jede Methode kann an jede Komponente oder Typkomponente weitergeleitet werden, indem eine einzige delegate -Anweisung in die Typdefinition aufgenommen wird. (Weitere Informationen zu Komponentennamen finden Sie unter Komponenten und Typkomponenten.)

Hier ist beispielsweise eine viel bessere Möglichkeit, die Methode "**wag**" des Objekts "**dog**" weiterzuleiten:

```
% snit::type dog {
    delegate method wag to mytail

    constructor {} {
        install mytail using tail %AUTO%
    }
}
::dog
% snit::type tail {
    method wag {} { return "Wag, wag, wag."}
}
::tail
% dog spot
::spot
% spot wag
Wag, wag, wag.
```

Dieser Code hat dieselbe Wirkung wie der unter der vorherigen Frage gezeigte Code: Wenn die Methode **wag** eines **dog** aufgerufen wird, werden der Aufruf und seine Argumente automatisch an das Objekt **tail** weitergeleitet.

Hinweis: Wenn eine Komponente in einer delegate-Anweisung erwähnt wird, wird die Instanzvariable der Komponente implizit definiert. Es empfiehlt sich jedoch weiterhin, diese mithilfe der component-Anweisung explizit zu deklarieren.

Hinweis: Sie können den Namen einer Methode entweder mit der Anweisung **method** oder mit delegate definieren; Sie können nicht beides tun.

Kann ich an eine Methode mit anderem Namen weiterleiten?

Angenommen, Sie möchten die Methode "**wagtail**" von "**dog**" an die Methode "**wag**" von "**tail**" weiterleiten. Schließlich wedeln Sie mit dem Schwanz, nicht mit dem Hund. Das ist ganz einfach:

```
snit::type dog {
    delegate method wagtail to mytail as wag

    constructor {args} {
        install mytail using tail %AUTO% -partof $self
```

```
        $self configurelist $args
    }
}
```

Kann ich an eine Methode mit zusätzlichen Argumenten weiterleiten?

Angenommen, die Methode **wag** von **tail** nimmt als Argument entgegen, wie oft der Schwanz wedeln soll. Sie möchten die Methode "**wagtail**" von "**dog**" an die Methode "**wag**" von "**tail**" weiterleiten, wobei Sie festlegen möchten, dass der Schwanz genau dreimal wedeln soll. Auch dies ist ganz einfach:

```
snit::type dog {
    delegate method wagtail to mytail as {wag 3}
    # ...
}

snit::type tail {
    method wag {count} {
        return [string repeat "Wag " $count]
    }
    # ...
}
```

Kann ich eine Methode an etwas anderes als ein Objekt weiterleiten?

Bei der normalen Weiterleitung einer Methode wird davon ausgegangen, dass Sie eine Methode (einen Unterbefehl eines Objektbefehls) an eine Methode eines anderen Objekts (einen Unterbefehl eines anderen Objektbefehls) weiterleiten. Allerdings folgen nicht alle Tcl-Objekte den Tk-Konventionen, und nicht alles, an das Sie eine Methode weiterleiten möchten, ist zwangsläufig ein Objekt. Daher ermöglicht Snit es Ihnen, mithilfe der Klausel **using** der Anweisung delegate eine Methode ganz einfach an nahezu alles weiterzuleiten, was Sie möchten.

Angenommen, Ihre Hundesimulation speichert Hunde in einer Datenbank, wobei jeder Hund als einzelner Datensatz vorliegt. Die von Ihnen verwendete Datenbank-API stellt eine Reihe von Befehlen zur Verwaltung von Datensätzen bereit; jeder davon nimmt die Datensatz-ID (eine von Ihnen gewählte Zeichenfolge) als erstes Argument entgegen. Beispielsweise speichert der Befehl "**saverec**" einen Datensatz. Wenn Sie die Datensatz-ID als Namen des Hundeobjekts verwenden, können Sie die Methode **save** des Hundes wie folgt an den Befehl **saverec** weiterleiten:

```
snit::type dog {
    delegate method save using {saverec %s}
}
```

Wenn die Methode "**save**" aufgerufen wird, wird "**%s**" durch den Namen der Instanz ersetzt; etwaige zusätzliche Argumente werden dem resultierenden Befehl angehängt.

Die Klausel **using** versteht eine Reihe weiterer %-Ersetzungen; neben dem Namen der Instanz können Sie den Namen der Methode (**%m**), den Namen des Typs (**%t**), den Namespace der Instanz (**%n**), den Tk-Fensternamen (**%w**) sowie – wenn in der Anweisung delegate ein Name

einer Komponente oder einer Typkomponente angegeben wurde – den Objektbefehl der Komponente (**%c**) einsetzen.

Wie leite ich eine Methode an ein Typkomponenten-Objekt weiter?

Genau so, wie Sie es bei einer Komponente tun würden. Die Anweisung delegate method akzeptiert in ihrer Klausel **to** sowohl Namen von Komponenten als auch Namen von Typkomponenten.

Wie leite ich eine Typmethode an ein Typkomponenten-Objekt weiter?

Verwenden Sie die Anweisung delegate typemethod. Sie funktioniert wie delegate method, weist jedoch folgende Unterschiede auf: Erstens definiert sie eine Typmethode anstelle einer Instanzmethode; zweitens ignoriert die Klausel **using** die %-Ersetzungen **%s**, **%n** und **%w**.

Selbstverständlich können Sie eine Typmethode nicht an eine Instanzkomponente weiterleiten … Snit wüsste nicht, welche Instanz sie erhalten soll.

Wie leite ich eine Option an ein Komponentenobjekt weiter?

Die erste Frage in diesem Abschnitt (siehe Weiterleitung) zeigt eine Möglichkeit, eine Option an eine Komponente weiterzuleiten; da dieses Muster jedoch häufig vorkommt, erleichtert Snit die Umsetzung. Beispielsweise verfügt jedes **tail**-Objekt über eine **-length**-Option; wir möchten dem Ersteller eines **dog**-Objekts ermöglichen, die Länge des Schwanzes festzulegen. Dies können wir wie folgt umsetzen:

```
% snit::type dog {
    delegate option -length to mytail

    constructor {args} {
        install mytail using tail %AUTO% -partof $self
        $self configurelist $args
    }
}
::dog
% snit::type tail {
    option -partof
    option -length 5
}
::tail
% dog spot -length 7
::spot
% spot cget -length
7
```

Dies führt zu nahezu demselben Ergebnis wie die unter der ersten Frage in diesem Abschnitt gezeigten Methoden **-configuremethod** und **-cgetmethod**: Immer wenn die Option **-length** eines **dog**-Objekts gesetzt oder abgerufen wird, wird die Option des zugrunde liegenden **tail** -Objekts ebenfalls gesetzt bzw. abgerufen.

Hinweis: Sie können einen Namen für eine Option entweder mit der Anweisung **option** oder mit delegate definieren; Sie können nicht beides tun.

Kann ich an eine Option mit anderem Namen weiterleiten?

In der vorherigen Antwort haben wir die Option "**-length**" von "**dog**" an dessen "**tail**" weitergeleitet. Das ist natürlich falsch. Der Hund hat eine Länge, und der Schwanz hat eine Länge, und diese unterscheiden sich voneinander. Was wir eigentlich tun möchten, ist, dem **dog** eine **-taillength**-Option zu geben, diese jedoch an die Option "**-length**" von "**tail**" weiterzuleiten:

```
snit::type dog {
    delegate option -taillength to mytail as -length

    constructor {args} {
        set mytail [tail %AUTO% -partof $self]
        $self configurelist $args
    }
}
```

Wie leite ich jede unbekannte Methode oder Option an ein Komponentenobjekt weiter?

Es kann vorkommen, dass ein Snit-Objekt den Großteil seines Verhaltens von einer seiner Komponenten bezieht. Dies ist beispielsweise häufig bei **snit::widgetadaptors** der Fall, wo wir das Verhalten eines bestehenden Widgets geringfügig anpassen möchten. Um unser Beispiel mit **dog** fortzusetzen, nehmen wir jedoch an, dass wir einen snit::type namens **animal** haben, der eine Vielzahl von Tierverhalten – Fortbewegung, Fressen, Schlafen und so weiter – implementiert. Wir möchten, dass unsere **dog**-Objekte dieselben Verhaltensweisen erben und gleichzeitig eigene, hundeähnliche Verhaltensweisen hinzufügen. So können wir einem **dog** eigene Methoden und Optionen zuweisen und gleichzeitig alle anderen Methoden und Optionen an seine **animal**-Komponente weiterleiten:

```
snit::type dog {
    delegate option * to animal
    delegate method * to animal

    option -akc 0

    constructor {args} {
        install animal using animal %AUTO% -name $self
        $self configurelist $args
    }

    method wag {} {
        return "$self wags its tail"
    }
}
```

Das war's. Ein **dog** ist nun ein **animal**, das über eine **-akc**-Option verfügt und mit dem Schwanz wedeln (**wag**) kann.

Hinweis: Wir müssen nicht die vollständige Liste der Namen der Methoden oder der Optionen

angeben, die **animal** erhalten wird. Es übernimmt alles, was **dog** nicht erkennt – und erkennt es das ebenfalls nicht, löst es einfach einen Fehler aus, genau wie es sein soll.

Sie können zudem alle unbekannten Typmethoden mithilfe von **delegate typemethod *** an eine Typkomponente weiterleiten.

Wie leite ich alle Methoden oder Optionen bis auf bestimmte an eine Komponente weiter?

In der vorherigen Antwort haben wir erklärt, dass jedes **dog** ein **animal** ist, indem alle unbekannten Methoden und Optionen an die Komponente **animal** weitergeleitet werden. Was aber, wenn der Typ **animal** einige Methoden oder Optionen enthält, die wir unterdrücken möchten?

Eine Lösung besteht darin, alle Optionen und Methoden explizit weiterzuleiten und auf die Annehmlichkeiten von **delegate method *** und **delegate option *** zu verzichten. Wenn wir jedoch nur einige wenige Optionen oder Methoden unterdrücken möchten, gibt es einen einfacheren Weg:

```
snit::type dog {
    delegate option * to animal except -numlegs
    delegate method * to animal except {fly climb}

    # ...

    constructor {args} {
        install animal using animal %AUTO% -name $self -numlegs 4
        $self configurelist $args
    }

    # ...
}
```

Hunde haben vier Beine; daher legen wir dies bei der Erstellung der Komponente **animal** ausdrücklich fest und schließen **-numlegs** ausdrücklich aus der Menge der weitergeleiteten Optionen aus. Ebenso können Hunde weder **fly** noch **climb** ausführen; daher schließen wir diese **animal**-Methoden wie dargestellt aus.

Kann eine hierarchische Methode weitergeleitet werden?

Ja; legen Sie einfach mehrere Wörter im Namen der weitergeleiteten Methode an:

```
snit::type tail {
    method wag {} {return "Wag, wag"}
    method droop {} {return "Droop, droop"}
}

snit::type dog {
    delegate method {tail wag} to mytail
    delegate method {tail droop} to mytail
```

```
    # ...

    constructor {args} {
        install mytail using tail %AUTO%
        $self configurelist $args
    }

    # ...
}
```

Auch nicht erkannte hierarchische Methoden können weitergeleitet werden; der folgende Code leitet alle Unterbefehle der Methode "tail" an die Komponente "mytail" weiter:

```
snit::type dog {
    delegate method {tail *} to mytail

    # ...
}
```

Widgets

Was ist ein snit::widget?

Ein snit::widget ist die Snit-Version dessen, was Tcl-Programmierer üblicherweise als *Megawidget* bezeichnen: ein widgetähnliches Objekt, das in der Regel aus einem oder mehreren Tk-Widgets besteht, die alle in einem Tk-Frame enthalten sind.

Ein snit::widget ist ebenfalls eine besondere Art von snit::type. Nahezu alle Angaben in dieser FAQ-Liste, die sich auf **snit::types** beziehen, gelten auch für **snit::widgets**.

Wie definiere ich ein snit::widget?

snit::widgets werden mit dem Befehl snit::widget definiert, ebenso wie **snit::types** mit dem Befehl snit::type definiert werden.

Der Hauptteil der Definition kann dieselben Arten von Anweisungen enthalten sowie einige weitere, auf die im Folgenden eingegangen wird.

Inwiefern unterscheiden sich snit::widgets von snit::types?

- Der Name einer Instanz von snit::type kann ein beliebiger gültiger Tcl-Befehlsname in einem beliebigen Namespace sein. Der Name einer Instanz von snit::widget muss ein gültiger Tk-Widget-Name sein, und das übergeordnete Widget muss bereits vorhanden sein.
- Eine Instanz von snit::type kann durch Aufruf ihrer Methode **destroy** zerstört werden. Instanzen von snit::widget verfügen über keine "destroy"-Methode; verwenden Sie stattdessen den Tk-Befehl **destroy**.
- Jede Instanz eines snit::widget verfügt über eine vordefinierte Komponente, die als **hull** -Komponente bezeichnet wird. Der Hull ist in der Regel ein Tk-Widget vom Typ "**frame**" oder "**toplevel**"; alle anderen Widgets, die als Teil des "snit::widget" erstellt wurden, sind in der Regel im Hull enthalten.
- Bei snit::widgets können die Optionen Standardwerte aus der Tk-Optionsdatenbank erhalten.

Was ist die Hull-Komponente?

Snit kann kein Tk-Widget-Objekt erstellen; dies ist nur Tk möglich. Daher muss jede Instanz von snit::widget um ein echtes Tk-Widget herumgelegt werden; dieses Tk-Widget wird als *Hull-Komponente* bezeichnet. Snit setzt das von Ihnen definierte Verhalten (Methoden, Optionen usw.) gewissermaßen huckepack auf die Hull-Komponente, sodass sich das Ganze wie ein Standard-Tk-Widget verhält.

Bei snit::widgets muss die Hull-Komponente ein Tk-Widget sein, das die Option **-class** definiert.

snit::widgetadaptors unterscheiden sich von snit::widgets vor allem dadurch, dass jede Art von Widget als Hull-Komponente verwendet werden kann; siehe Widget-Adapter.

Wie lege ich den Hull-Typ eines snit::widget fest?

Die Hull-Komponente eines snit::widget ist in der Regel ein Tk-Widget vom Typ **frame**; es kann sich jedoch um jedes beliebige Tk-Widget handeln, das die Option **-class** definiert. Sie können den gewünschten Hull-Typ explizit festlegen, indem Sie den Befehl hulltype in die Widget-Definition aufnehmen:

```
snit::widget mytoplevel {
    hulltype toplevel

    # ...
}
```

Wenn der Befehl "hulltype" nicht vorkommt, ist der Hull ein "**frame**".

Standardmäßig erkennt Snit die folgenden Hull-Typen: die Tk-Widgets **frame**, **labelframe**, **toplevel** sowie die Tile-Widgets **ttk::frame**, **ttk::labelframe** und **ttk::toplevel**. Um die Verwendung eines anderen Widget-Typs als Hull-Typ zu ermöglichen, können Sie den Widget-Befehl mit **lappend** an die Variable **snit::hulltypes** anhängen (immer vorausgesetzt, das Widget definiert die Option **-class**. Nehmen wir beispielsweise an, Tk erhält einen neuen Widget-Typ namens **prettyframe**:

```
lappend snit::hulltypes prettyframe

snit::widget mywidget {
    hulltype prettyframe

    # ...
}
```

Wie soll ich Widgets benennen, die Komponenten eines snit::widget sind?

Jedes Widget – sei es ein echtes Tk-Widget oder ein Snit-Megawidget – muss über einen gültigen Tk-Fensternamen verfügen. Wenn ein snit::widget erstmals erstellt wird, ist sein Instanzname, **self**, ein Tk-Fenstername; wird das snit::widget jedoch von einem snit::widgetadaptor als Hull-Komponente verwendet, ändert sich sein Instanzname. Aus diesem Grund wird jeder Methode, jedem Konstruktor, jedem Destruktor usw. von snit::widget ein weiteres implizites Argument übergeben, nämlich **win**, der Fenstername des Megawidgets. Alle Kindfenster sollten mit "**win**" als Wurzel benannt werden.

Nehmen wir also an, Sie erstellen ein Toolbar-Widget, einen Frame, der aus mehreren

nebeneinander angeordneten Schaltflächen besteht. Das könnte in etwa so aussehen:

```
snit::widget toolbar {
    delegate option * to hull

    constructor {args} {
        button $win.open -text Open -command [mymethod open]
        button $win.save -text Save -command [mymethod save]

        # ....

        $self configurelist $args

    }
}
```

Siehe auch die Frage zur Umbenennung von Objekten weiter oben in dieser Datei.

Widget-Adapter

Was ist ein snit::widgetadaptor?

Ein snit::widgetadaptor ist eine Art von snit::widget. Während der Hull eines snit::widget automatisch erstellt wird und stets ein Tk-Frame ist, kann ein snit::widgetadaptor auf einem beliebigen Tk-Widget – oder auf einem beliebigen Snit-Megawidget oder sogar (mit etwas Glück) auf Megawidgets basieren, die mithilfe eines anderen Pakets definiert wurden.

Es wird als *Widget-Adapter* bezeichnet, da es Ihnen ermöglicht, ein vorhandenes Widget zu übernehmen und dessen Verhalten anzupassen.

Wie definiere ich ein snit::widgetadaptor?

Verwenden Sie den Befehl snit::widgetadaptor. Die Definition für snit::widgetadaptor entspricht genau der für snit::type oder snit::widget, mit der Ausnahme, dass der Konstruktor die Hull-Komponente erstellen und installieren muss.

Der folgende Code erstellt beispielsweise ein schreibgeschütztes Text-Widget, indem die Methoden **insert** und **delete** auf einfache Weise in No-Ops umgewandelt werden. Anschließend definieren wir neue Methoden, **ins** und **del**, die als **insert** und **delete** an die Hull-Komponente weitergeleitet werden. Auf diese Weise haben wir das Text-Widget angepasst und ihm ein neues Verhalten verliehen, wobei es im Grunde genommen weiterhin ein Text-Widget bleibt.

```
::snit::widgetadaptor rotext {

    constructor {args} {
        # Create the text widget; turn off its insert cursor
        installhull using text -insertwidth 0

        # Apply any options passed at creation time.
        $self configurelist $args
    }
```

```
    # Disable the text widget's insert and delete methods, to
    # make this readonly.
    method insert {args} {}
    method delete {args} {}

    # Enable ins and del as synonyms, so the program can insert and
    # delete.
    delegate method ins to hull as insert
    delegate method del to hull as delete

    # Pass all other methods and options to the real text widget, so
    # that the remaining behavior is as expected.
    delegate method * to hull
    delegate option * to hull
}
```

Der wichtigste Teil befindet sich im Konstruktor. Während snit::widget den Hull für Sie erstellt, ist snit::widgetadaptor dazu nicht in der Lage – es weiß nicht, welche Art von Widget Sie wünschen. Daher erstellt der Konstruktor zunächst die Hull-Komponente (in diesem Fall ein Tk-Text-Widget) und installiert diese anschließend mit dem Befehl installhull.

Hinweis: Es gibt keinen Instanzbefehl, bis Sie durch die Installation einer Hull-Komponente einen solchen erstellen. Jeder Versuch, Methoden an **$self** zu übergeben, bevor installhull aufgerufen wird, schlägt fehl.

Kann ich ein Widget anpassen, das an anderer Stelle im Programm erstellt wurde?

Ja.

Manchmal kann es praktisch sein, ein bereits vorhandenes Widget anzupassen, anstatt ein eigenes zu erstellen. Beispielsweise verwaltet das Bwidget "**PagesManager**" eine Reihe von "**frame**"-Widgets, von denen jeweils nur eines sichtbar ist. Die Anwendung entscheidet, welches "**frame**" sichtbar ist. Alle diese **frame**-Instanzen werden vom **PagesManager** selbst mithilfe seiner **add**-Methode erstellt. Es ist praktisch, diese Frames so anzupassen, dass sie die gewünschten Funktionen erfüllen.

In einem solchen Fall existiert das Tk-Widget bereits, wenn "snit::widgetadaptor" erstellt wird. Snit stellt zu diesem Zweck eine alternative Form des Befehls "installhull" bereit:

```
snit::widgetadaptor pageadaptor {
    constructor {args} {
        # The widget already exists; just install it.
        installhull $win

        # ...
    }
}
```

Kann ich ein anderes Megawidget anpassen?

Möglicherweise. Wenn es sich bei dem anderen Megawidget um ein snit::widget oder snit::widgetadaptor handelt, dann ja. Ist dies nicht der Fall, dann wiederum möglicherweise. Sie müssen es ausprobieren und sehen, ob es funktioniert. Am ehesten werden Sie Probleme mit der Zerstörung von Widgets haben – Sie müssen sicherstellen, dass Ihr Megawidget-Code das Ereignis erhält, bevor das von Ihnen angepasste Megawidget dieses Ereignis erhält.

Tk-Optionsdatenbank

Was ist die Tk-Optionsdatenbank?

Die Tk-Optionsdatenbank ist eine Datenbank mit Standardwerten für Optionen, die von Tk selbst verwaltet wird; jede Tk-Anwendung verfügt über eine solche Datenbank. Das Konzept der Optionsdatenbank leitet sich von der sogenannten X-Windows-Ressourcendatenbank ab; die Optionsdatenbank ist jedoch in jeder Tk-Implementierung verfügbar, auch in solchen, die das X-Windows-System nicht verwenden (z. B. Microsoft Windows).

Eine ausführliche Beschreibung der Tk-Optionsdatenbank würde den Rahmen dieses Dokuments sprengen; sowohl *Practical Programming in Tcl and Tk* von Welch, Jones und Hobbs als auch *Effective Tcl/Tk Programming* von Harrison und McClennan enthalten gute Einführungen zu diesem Thema.

Snit ist so implementiert, dass es in den meisten Fällen im Hinblick auf die Optionsdatenbank einfach das Richtige tut, vorausgesetzt, der Widget-Entwickler hält sich an die Vorgaben von Snit. Im Hauptteil dieses Abschnitts wird ausführlich darauf eingegangen, welche Anforderungen Snit stellt. Im Folgenden finden Sie zur Orientierung eine kurze Zusammenfassung dieser Anforderungen.

- Wenn die Standard-Widget-Klasse des Widgets nicht Ihren Vorstellungen entspricht, legen Sie diese explizit mithilfe der Anweisung widgetclass in der Widget-Definition fest.
- Wenn Sie Optionen definieren oder weiterleiten, legen Sie die Namen der Ressourcen und Klassen bei Bedarf ausdrücklich fest.
- Verwenden Sie den Befehl installhull using, um den Hull für snit::widgetadaptors zu erstellen und zu installieren.
- Verwenden Sie den Befehl install, um alle Komponenten, bei denen es sich um Widgets handelt, zu erstellen und zu installieren.
- Verwenden Sie den Befehl install, um Komponenten zu erstellen und zu installieren, bei denen es sich nicht um Widgets handelt, falls Sie möchten, dass diese Optionswerte aus der Optionsdatenbank erhalten.

Das Zusammenspiel von Tk-Widgets mit der Optionsdatenbank ist komplex; das Zusammenspiel von Snit mit der Optionsdatenbank ist dies noch mehr und erfordert daher besondere Sorgfalt.

Verwenden snit::types die Tk-Optionsdatenbank?

Nein, das tun sie nicht; für die Abfrage der Optionsdatenbank ist ein Tk-Fenstername erforderlich, und snit::types verfügen nicht über einen solchen.

Wenn Sie eine Instanz von snit::type als Komponente von snit::widget oder snit::widgetadaptor anlegen, und wenn Optionen an die Komponente weitergeleitet werden und Sie diese mit install erstellen und installieren, dann fragt das Megawidget die Optionsdatenbank im Auftrag von snit::type ab. Dies entspricht möglicherweise Ihren Vorstellungen, möglicherweise aber auch nicht; seien Sie daher bitte vorsichtig.

Wie lautet die Widget-Klasse meines snit::widget?

Jedes Tk-Widget verfügt über eine "Widget-Klasse": einen Namen, der beim Hinzufügen von Optionseinstellungen zur Datenbank verwendet wird. Bei Tk-Widgets entspricht die Widget-Klasse dem Namen des Widget-Befehls mit einem Großbuchstaben am Anfang. Beispielsweise lautet die Widget-Klasse des Tk-Widgets "**button**" "**Button**".

Ebenso ist die Widget-Klasse eines snit::widget standardmäßig der unqualifizierte Typname mit großem Anfangsbuchstaben. Beispielsweise lautet die Widget-Klasse von

```
snit::widget ::mylibrary::scrolledText { ... }
```

is **ScrolledText**.

Die Widget-Klasse kann auch explizit mithilfe der Anweisung widgetclass innerhalb der Definition von snit::widget festgelegt werden:

```
snit::widget ::mylibrary::scrolledText {
    widgetclass Text

    # ...
}
```

Die obige Definition besagt, dass ein "**scrolledText**"-Megawidget dieselbe Widget-Klasse aufweist wie ein gewöhnliches "**text**"-Widget. Ob dies eine gute Idee ist oder nicht, hängt davon ab, wie der Rest des Megawidgets definiert ist und wie dessen Optionen weitergeleitet werden.

Wie lautet die Widget-Klasse meines snit::widgetadaptor?

Die Widget-Klasse eines snit::widgetadaptor entspricht lediglich der Widget-Klasse seines Hull-Widgets; Snit hat hierauf keinen Einfluss.

Hinweis: Die Widget-Klasse kann nur für die Widgets **frame** und **toplevel** geändert werden; aus diesem Grund sind dies die zulässigen Hull-Typen für snit::widgets.

Versuchen Sie, snit::widgetadaptors nur zu verwenden, um geringfügige Änderungen am Verhalten eines anderen Widgets vorzunehmen. In diesem Fall ist es in der Regel ohnehin nicht sinnvoll, die Klasse des Widgets zu ändern.

Was sind Ressourcen- und Klassennamen von Optionen?

Jede Tk-Widget-Option hat drei Namen: den Optionsnamen, den Ressourcennamen und den Namen der Klasse. Der Optionsname beginnt mit einem Bindestrich und besteht ausschließlich aus Kleinbuchstaben; er wird beim Erstellen von Widgets sowie bei den Befehlen **configure** und **cget** verwendet.

Die Ressourcen- und Klassennamen werden verwendet, um Standardwerte für Optionen durch Abfrage der Optionsdatenbank zu initialisieren. Der Ressourcenname besteht in der Regel lediglich aus dem Optionsnamen ohne Bindestrich, kann jedoch an Wortgrenzen Großbuchstaben enthalten; der Klassenname besteht in der Regel lediglich aus dem Ressourcennamen mit einem Großbuchstaben am Anfang, was jedoch nicht immer der Fall ist. Hier sind beispielsweise die Namen der Optionen, Ressourcen und Klassen für verschiedene Optionen des Tk-Widgets **text**:

```
    -background         background         Background
    -borderwidth        borderWidth        BorderWidth
    -insertborderwidth  insertBorderWidth  BorderWidth
    -padx               padX               Pad
```

Wie leicht zu erkennen ist, lassen sich die Namen der Ressourcen und Klassen manchmal aus dem Namen der Option ableiten, jedoch nicht immer.

Wie lauten die Ressourcen- und Klassennamen für die Optionen meines Megawidgets?

Bei Optionen, die implizit mithilfe von **delegate option *** an eine Komponente weitergeleitet werden, entsprechen die Ressourcen- und Klassennamen genau den von der Komponente definierten Bezeichnungen. Die Methode **configure** gibt diese Namen zusammen mit den Standard- und aktuellen Werten der Option zurück:

```
% snit::widget mytext {
    delegate option * to text

    constructor {args} {
        install text using text .text
        # ...
    }

    # ...
}
::mytext
% mytext .text
.text
% .text configure -padx
-padx padX Pad 1 1
%
```

Bei allen anderen Optionen (unabhängig davon, ob sie lokal definiert oder explizit weitergeleitet wurden) können die Namen der Ressourcen und Klassen explizit festgelegt werden oder es kann zugelassen werden, dass sie Standardwerte annehmen.

Standardmäßig entspricht der Name der Ressource lediglich dem Namen der Option ohne Bindestrich; der Name der Klasse entspricht hingegen dem Namen der Option mit einem Großbuchstaben am Anfang. Nehmen wir beispielsweise an, wir leiten explizit "-padx" weiter:

```
% snit::widget mytext {
    option -myvalue 5

    delegate option -padx to text
    delegate option * to text

    constructor {args} {
        install text using text .text
```

```
        # ...
    }

    # ...
}
::mytext
% mytext .text
.text
% .text configure -myvalue
-myvalue myvalue Myvalue 5 5
% .text configure -padx
-padx padx Padx 1 1
%
```

Hier werden die Ressourcen- und Klassennamen anhand der Standardregeln ausgewählt. Oftmals reichen diese Regeln aus, doch im Fall von "-padx" würden wir es höchstwahrscheinlich vorziehen, wenn die Ressourcen- und Klassennamen der Option mit denen der integrierten Tk-Widgets übereinstimmen. Dies lässt sich leicht bewerkstelligen:

```
% snit::widget mytext {
    delegate option {-padx padX Pad} to text

    # ...
}
::mytext
% mytext .text
.text
% .text configure -padx
-padx padX Pad 1 1
%
```

Wie initialisiert snit die lokal definierten Optionen meines Megawidgets?

Die Optionsdatenbank wird für jede der lokal definierten Optionen des Megawidgets unter Verwendung des Ressourcen- und Klassennamens der Option abgefragt. Wenn das Ergebnis nicht "" ist, ersetzt es den in der Widget-Definition angegebenen Standardwert. In beiden Fällen kann der Standardwert vom Aufrufer überschrieben werden. Zum Beispiel:

```
option add *Mywidget.texture pebbled

snit::widget mywidget {
    option -texture smooth
    # ...
}

mywidget .mywidget -texture greasy
```

Hier würde **-texture** normalerweise standardmäßig auf "smooth" gesetzt werden, doch aufgrund des Eintrags, der der Optionsdatenbank hinzugefügt wurde, ist die Standardeinstellung

"pebbled". Der Aufrufer hat die Standardeinstellung jedoch ausdrücklich überschrieben, sodass das neue Widget "greasy" sein wird.

Wie initialisiert snit weitergeleitete Optionen?

Das hängt davon ab, ob die Optionen an den Hull oder an eine andere Komponente weitergeleitet werden.

Wie initialisiert snit Optionen, die an den Hull weitergeleitet werden?

Der Hull eines snit::widget ist ein Widget, und da seine Klasse festgelegt wurde, wird erwartet, dass er die Optionsdatenbank für sich selbst abfragt. Die einzige Ausnahme betrifft Optionen, die an ihn unter einem anderen Namen weitergeleitet werden. Betrachten Sie den folgenden Code:

```
option add *Mywidget.borderWidth 5
option add *Mywidget.relief sunken
option add *Mywidget.hullbackground red
option add *Mywidget.background green

snit::widget mywidget {
    delegate option -borderwidth to hull
    delegate option -hullbackground to hull as -background
    delegate option * to hull
    # ...
}

mywidget .mywidget

set A [.mywidget cget -relief]
set B [.mywidget cget -hullbackground]
set C [.mywidget cget -background]
set D [.mywidget cget -borderwidth]
```

Die Frage lautet: Welche Werte haben die Variablen A, B, C und D?

Der Wert von A ist "sunken". Der Hull ist ein Tk-Frame, dem die Widget-Klasse **Mywidget** zugewiesen wurde; er fragt automatisch die Optionsdatenbank ab und übernimmt diesen Wert. Da die Option **-relief** implizit an den Hull weitergeleitet wird, unternimmt Snit nichts.

Der Wert von B lautet "red". Der Hull übernimmt automatisch den Wert "green" für seine Option **-background**, genau wie er zuvor den Wert für **-relief** übernommen hat. Snit weiß jedoch, dass **-hullbackground** der Option **-background** des Hulls zugeordnet ist; daher fragt es die Optionsdatenbank nach **-hullbackground** ab, erhält den Wert "red" und aktualisiert den Hull entsprechend.

Der Wert von C ist ebenfalls "red", da **-background** implizit an den Hull weitergeleitet wird; daher entspricht das Abrufen dieses Werts dem Abrufen von **-hullbackground**. Beachten Sie, dass dieser Fall ungewöhnlich ist; die Option **-background** hätte wahrscheinlich mithilfe der Klausel **except** der delegate-Anweisung ausgeschlossen oder (was wahrscheinlicher ist) an eine andere Komponente weitergeleitet werden sollen.

Der Wert von D lautet "5", jedoch nicht aus dem Grund, den Sie vermuten. Hinweis: Gemäß der

obigen Definition lautet der Ressourcenname für **-borderwidth** standardmäßig **borderwidth**, während der Eintrag in der Optionsdatenbank **borderWidth** lautet, entsprechend der standardmäßigen Tk-Namenskonvention für diese Option. Wie bei **-relief** übernimmt der Hull seine eigene Option **-borderwidth**, bevor Snit irgendetwas unternimmt. Da die Option unter ihrem eigenen Namen weitergeleitet wird, geht Snit davon aus, dass alles korrekt abgelaufen ist, und befasst sich nicht weiter damit. Um Verwechslungen zu vermeiden, hätte die Option **-borderwidth** wie folgt weitergeleitet werden sollen:

```
    delegate option {-borderwidth borderWidth BorderWidth} to hull
```

Bei snit::widgetadaptors stellt sich die Situation etwas anders dar. Widget-Adapter behalten die Widget-Klasse ihres Hulls bei, und der Hull wird nicht automatisch von Snit erstellt. Stattdessen muss snit::widgetadaptor in seinem Konstruktor installhull aufrufen. Dies geschieht üblicherweise wie folgt:

```
snit::widgetadaptor mywidget {
    # ...
    constructor {args} {
        # ...
        installhull using text -foreground white
        # ...
    }
    # ...
}
```

In diesem Fall erstellt der Befehl installhull den Hull mithilfe eines Befehls wie dem folgenden:

```
    set hull [text $win -foreground white]
```

Der Hull ist ein **text**-Widget, daher lautet seine Widget-Klasse **Text**. Genau wie bei snit::widget -Hulls geht Snit davon aus, dass er alle normalen Werte für Optionen automatisch übernimmt, ohne dass ein Eingreifen seitens Snit erforderlich ist. Optionen, die unter einem anderen Namen weitergeleitet werden, werden auf dieselbe Weise wie oben beschrieben aus der Optionsdatenbank initialisiert.

In früheren Versionen von Snit wurde erwartet, dass snit::widgetadaptors den Befehl installhull wie folgt aufrufen:

```
    installhull [text $win -foreground white]
```

Diese Form funktioniert weiterhin – aber Snit fragt dann die Optionsdatenbank nicht wie oben beschrieben ab.

Wie initialisiert snit Optionen, die an andere Komponenten weitergeleitet werden?

Bei Hull-Komponenten geht Snit davon aus, dass Tk den Großteil der Arbeit automatisch erledigt. Bei anderen Komponenten ist die Sache etwas komplizierter, da diese zweimal mit der Optionsdatenbank abgeglichen werden.

Ein Komponenten-Widget bleibt weiterhin ein Widget und wird daher wie gewohnt aus der Optionsdatenbank initialisiert. Ein **text**-Widget bleibt ein **text**-Widget, unabhängig davon, ob es Bestandteil eines Megawidgets ist oder nicht, und wird als solches erstellt.

Anschließend wird die Optionsdatenbank nach allen an die Komponente weitergeleiteten Optionen abgefragt, und die Komponente wird entsprechend initialisiert – vorausgesetzt, sie wird mit dem Befehl install erstellt.

Bevor die Unterstützung für die Optionsdatenbank in Snit integriert wurde, bestand die übliche Vorgehensweise beim Erstellen einer Komponente darin, diese einfach im Konstruktor zu erstellen und ihren Befehlsnamen der Komponentenvariablen zuzuweisen:

```
snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        set myComp [text $win.text -foreground black]
    }
}
```

Der Nachteil dieser Methode besteht darin, dass Snit keine Möglichkeit hat, die Komponente ordnungsgemäß zu initialisieren. Daher wird nun folgender Ansatz verwendet:

```
snit::widget mywidget {
    delegate option -background to myComp

    constructor {args} {
        install myComp using text $win.text -foreground black
    }
}
```

Der Befehl install führt Folgendes aus:

- Erstellt eine Liste der Optionen, die explizit im Befehl install enthalten sind – in diesem Fall **-foreground**.
- Fragt die Optionsdatenbank nach allen Optionen, die explizit an die genannte Komponente weitergeleitet wurden.
- Erstellt die Komponente mithilfe des angegebenen Befehls, nachdem eine Liste von Optionen und Werten, die aus der Optionsdatenbank ausgelesen wurden, in diesen Befehl eingefügt wurde. Somit haben die explizit angegebenen Optionen (wie **-foreground**) Vorrang vor allen aus der Optionsdatenbank ausgelesenen Werten.
- Wenn die Widget-Definition Optionen implizit mithilfe von **delegate option *** an die Komponente weiterleitet, ruft Snit die Methode **configure** der neu erstellten Komponente auf, um eine Liste aller Optionen der Komponente zu erhalten. Daraus erstellt Snit eine Liste der implizit an die Komponente weitergeleiteten Optionen, die nicht explizit im Befehl install enthalten waren. Für alle diese Optionen fragt Snit die Optionsdatenbank ab und konfiguriert die Komponente entsprechend.

Sie müssen all dies nicht unbedingt wissen; verwenden Sie einfach install, um Ihre Komponenten zu installieren, und Snit wird versuchen, das Richtige zu tun.

Was geschieht, wenn ich ein Nicht-Widget als Komponente eines Widgets installiere?

Ein snit::type fragt niemals die Optionsdatenbank ab. Ein snit::widget kann jedoch Komponenten

enthalten, die keine Widgets sind. Wenn Optionen an diese Komponenten weitergeleitet werden und der Befehl install zur Installation dieser Komponenten verwendet wird, werden sie ebenso wie Widget-Komponenten aus der Optionsdatenbank initialisiert.

Bei der Verwendung innerhalb eines Megawidgets geht install jedoch davon aus, dass die erstellte Komponente eine weitgehend standardmäßige, widgetähnliche Erstellungssyntax verwendet. Wenn dies nicht der Fall ist, verwenden Sie install bitte nicht.

Ensemble-Befehle

Was ist ein Ensemble-Befehl?

Ein Ensemble-Befehl ist ein Befehl mit Unterbefehlen. Snit-Objekte sind allesamt Ensemble-Befehle; der Begriff bezieht sich jedoch in der Regel eher auf Befehle wie die Standard-Tcl-Befehle **string**, **file** und **clock**. In gewisser Weise handelt es sich hierbei um Singleton-Objekte – es gibt jeweils nur eine Instanz davon.

Wie kann ich mit snit einen Ensemble-Befehl erstellen?

Es gibt zwei Möglichkeiten – als snit::type oder als Instanz eines snit::type.

Wie kann ich einen Ensemble-Befehl unter Verwendung einer Instanz von snit::type erstellen?

Definieren Sie einen Typ, dessen Instanzmethoden die Unterbefehle Ihres Ensemble-Befehls sind. Erstellen Sie anschließend eine Instanz dieses Typs mit dem gewünschten Namen.

Der folgende Code nutzt beispielsweise Weiterleitung, um eine Entsprechung zum Standardbefehl **string** zu erstellen:

```
snit::type ::mynamespace::mystringtype {
    delegate method * to stringhandler

    constructor {} {
        set stringhandler string
    }
}

::mynamespace::mystringtype mystring
```

Wir definieren den Typ in einem Namespace, sodass der Typbefehl verborgen ist; anschließend erstellen wir eine einzelne Instanz mit dem gewünschten Namen – in diesem Fall **mystring**.

Diese Methode weist zwei Nachteile auf. Erstens bleibt der Typbefehl ungenutzt herumliegen. Noch schwerwiegender ist jedoch, dass Ihr brandneuer Ensemble-Befehl die Unterbefehle "**info**" und "**destroy**" enthält, für die Sie wahrscheinlich keine Verwendung haben. Lesen Sie jedoch weiter.

Wie kann ich mithilfe von snit::type einen Ensemble-Befehl erstellen?

Definieren Sie einen Typ, dessen Typmethoden die Unterbefehle Ihres Ensemble-Befehls sind.

Der folgende Code nutzt beispielsweise Weiterleitung, um eine Entsprechung zum Standardbefehl **string** zu erstellen:

```
snit::type mystring {
    delegate typemethod * to stringhandler

    typeconstructor {
        set stringhandler string
    }
}
```

Nun ist der Typbefehl selbst Ihr Ensemble-Befehl.

Diese Methode hat nur einen Nachteil, und obwohl dieser erheblich ist, lässt er sich dennoch überwinden. Ihr neuer Ensemble-Befehl wird die Unterbefehle **create**, **info** und **destroy** enthalten, die Sie nicht wünschen. Und was noch schlimmer ist: Da die Methode **create** implizit sein kann, werden Benutzer Ihres Befehls versehentlich Instanzen Ihres Typs **mystring** erstellen, sollten sie einen der Unterbefehle falsch schreiben. Der Befehl wird erfolgreich sein – beim ersten Mal –, aber nicht das gewünschte Ergebnis liefern. Das ist sehr schlecht.

Die Abhilfe besteht darin, einige Pragmas festzulegen, wie hier gezeigt:

```
snit::type mystring {
    pragma -hastypeinfo    no
    pragma -hastypedestroy no
    pragma -hasinstances   no

    delegate typemethod * to stringhandler

    typeconstructor {
        set stringhandler string
    }
}
```

Hier haben wir die Anweisung pragma verwendet, um Snit mitzuteilen, dass wir weder die Typmethode **info** noch die Typmethode **destroy** wünschen und dass unser Typ keine Instanzen hat; dadurch werden die Typmethode **create** und der gesamte zugehörige Code entfernt. Infolgedessen verhält sich unser Ensemble-Befehl wie vorgesehen, ohne unerwartete Unterbefehle.

Pragmas

Was ist ein Pragma?

Ein Pragma ist eine Option, die Sie in Ihren Typdefinitionen festlegen können und die Einfluss darauf hat, wie der Typ definiert wird und wie er nach seiner Definition funktioniert.

Wie setze ich ein Pragma?

Verwenden Sie die Anweisung pragma. Jedes Pragma ist eine Option mit einem Wert; bei jeder Verwendung der Anweisung pragma können Sie eine oder mehrere davon festlegen.

Wie werde ich die Typmethode "info" los?

Setzen Sie das Pragma **-hastypeinfo** auf **no**:

```
snit::type dog {
    pragma -hastypeinfo no
    # ...
}
```

Snit wird davon absehen, die Typmethode **info** zu definieren.

Wie werde ich die Typmethode "destroy" los?

Setzen Sie das Pragma **-hastypedestroy** auf **no**:

```
snit::type dog {
    pragma -hastypedestroy no
    # ...
}
```

Snit wird davon absehen, die Typmethode **destroy** zu definieren.

Wie werde ich die Typmethode "create" los?

Setzen Sie das Pragma **-hasinstances** auf **no**:

```
snit::type dog {
    pragma -hasinstances no
    # ...
}
```

Snit verzichtet darauf, die Typmethode **create** zu definieren; wenn Sie den Typbefehl mit einem unbekannten Methodennamen aufrufen, erhalten Sie eine Fehlermeldung anstelle einer neuen Instanz des Typs.

Dies ist nützlich, wenn Sie einen snit::type verwenden möchten, um einen Ensemble-Befehl zu definieren, anstatt einen Typ mit Instanzen.

Die Pragmas **-hastypemethods** und **-hasinstances** können nicht beide falsch sein (sonst bliebe nichts mehr übrig).

Wie werde ich Typmethoden ganz los?

Die Typbefehle normaler Tk-Widgets haben keine Unterbefehle; sie dienen lediglich dazu, Widgets zu erstellen – in der Sprache von Snit: Der Typbefehl ruft direkt die Typmethode **create** auf. Um dasselbe Verhalten in Snit zu erzielen, setzen Sie das Pragma **-hastypemethods** auf **no**:

```
snit::type dog {
    pragma -hastypemethods no
    #...
}

# Creates ::spot
dog spot

# Tries to create an instance called ::create
dog create spot
```

Die Pragmas **-hastypemethods** und **-hasinstances** können nicht beide falsch sein (sonst bliebe nichts mehr übrig).

Warum kann ich kein Objekt erstellen, das ein altes Objekt mit demselben Namen ersetzt?

Bis zur Version 0.95 von Snit konnten Sie für eine Instanz von snit::type einen beliebigen Namen verwenden, selbst wenn dieser Name bereits von einem anderen Objekt oder Befehl verwendet wurde. Sie konnten beispielsweise Folgendes tun:

```
snit::type dog { ... }

dog proc
```

Sie haben nun einen neuen Hund mit dem Namen "proc", was wahrscheinlich nicht ganz das ist, was Sie eigentlich beabsichtigt hatten. Infolgedessen gibt Snit nun eine Fehlermeldung aus, wenn der von Ihnen gewählte Name der Instanz mit einem bereits vorhandenen Befehl übereinstimmt. Um das alte Verhalten wiederherzustellen, setzen Sie das Pragma **-canreplace** auf **yes**:

```
snit::type dog {
    pragma -canreplace yes
    # ...
}
```

Wie mache ich meinen einfachen Typ schneller?

In Snit 1.x können Sie das Pragma **-simpledispatch** auf **yes** setzen.

Der Methodenaufruf in Snit 1.x ist sowohl flexibel als auch schnell, doch diese Flexibilität hat ihren Preis. Wenn Ihr Typ diese Flexibilität nicht benötigt, können Sie mithilfe des Pragmas **-simpledispatch** einen einfacheren Aufrufmechanismus einsetzen, der deutlich schneller läuft. Die Einschränkungen lauten wie folgt:

- Methoden können nicht weitergeleitet werden.
- **uplevel** und **upvar** funktionieren nicht wie erwartet: Der Geltungsbereich des Aufrufers liegt zwei Ebenen höher statt einer.
- Die Methoden zur Verarbeitung von Optionen (**cget**, **configure** und **configurelist**) sind geringfügig langsamer.

In Snit 2.2 ist das Pragma **-simpledispatch** veraltet und wird ignoriert; jeder Methodenaufruf in Snit 2.2 ist schneller als **-simpledispatch** in Snit 1.x.

Makros

Was ist ein Makro?

Ein Snit-Makro ist nichts anderes als eine Tcl-Prozedur, die im Tcl-Interpreter definiert ist, der zum Kompilieren von Snit-Typdefinitionen verwendet wird.

Wozu sind Makros gut?

Mit Snit-Makros können Sie eine neue Syntax für Typdefinitionen festlegen und die bedingte Kompilierung unterstützen.

Wie führe ich eine bedingte Kompilierung durch?

Angenommen, Sie möchten, dass Ihr Typ eine schnelle C-Erweiterung verwendet, wenn diese verfügbar ist; andernfalls greifen Sie auf eine langsamere Tcl-Implementierung zurück. Sie möchten im ersten Fall einen Satz von Methoden definieren und im zweiten Fall einen anderen. Doch wie kann Ihre Typdefinition erkennen, ob die schnelle C-Erweiterung verfügbar ist oder nicht?

Das ist ganz einfach. Definieren Sie außerhalb einer Typdefinition ein Makro, das 1 zurückgibt, wenn die Erweiterung verfügbar ist, und andernfalls 0:

```
if {$gotFastExtension} {
    snit::macro fastcode {} {return 1}
} else {
    snit::macro fastcode {} {return 0}
}
```

Verwenden Sie anschließend Ihr Makro in Ihrer Typdefinition:

```
snit::type dog {

    if {[fastcode]} {
        # Fast methods
        method bark {} {...}
        method wagtail {} {...}
    } else {
        # Slow methods
        method bark {} {...}
        method wagtail {} {...}
    }
}
```

Wie definiere ich eine neue Syntax für Typdefinitionen?

Verwenden Sie ein Makro. Beispielsweise soll die Option **-background** Ihres snit::widget auf eine Reihe von Komponenten-Widgets übertragen werden. Sie könnten dies wie folgt umsetzen:

```
snit::widget mywidget {
    option -background -default white -configuremethod
    PropagateBackground

    method PropagateBackground {option value} {
        $comp1 configure $option $value
        $comp2 configure $option $value
        $comp3 configure $option $value
    }
}
```

Für eine einzige Option ist dies in Ordnung; wenn Sie jedoch mehrere Optionen haben, wird es mühsam und fehleranfällig. Verpacken Sie es daher in ein Makro:

```
snit::macro propagate {option "to" components} {
    option $option -configuremethod Propagate$option

    set body "\n"

    foreach comp $components {
        append body "\$$comp configure $option \$value\n"
    }

    method Propagate$option {option value} $body
}
```

Dann können Sie es wie folgt verwenden:

```
snit::widget mywidget {
    option -background default -white
    option -foreground default -black

    propagate -background to {comp1 comp2 comp3}
    propagate -foreground to {comp1 comp2 comp3}
}
```

Gibt es Einschränkungen hinsichtlich der Namen von Makros?

Ja, die gibt es. Sie dürfen keine Standard-Tcl-Befehle oder Snit-Typdefinitionsanweisungen neu definieren. Sie können jeden beliebigen anderen Befehlsnamen verwenden, einschließlich des Namens eines zuvor definierten Makros.

Wenn Sie Snit-Makros in Ihrer Anwendung verwenden, können Sie diese wie oben gezeigt im globalen Namespace benennen. Wenn Sie sie jedoch verwenden, um Typen oder Widgets zu definieren, die von anderen genutzt werden sollen, sollten Sie Ihre Makros im selben Namespace wie Ihre Typen oder Widgets definieren. Auf diese Weise kommt es zu keinen Konflikten mit den Makros anderer Entwickler.

Wenn mein raffiniertes snit::widget beispielsweise **::mylib::mywidget** heißt, dann sollte ich mein Makro "**propagate**" als "**::mylib::propagate**" definieren:

```
snit::macro mylib::propagate {option "to" components} { ... }

snit::widget ::mylib::mywidget {
    option -background default -white
    option -foreground default -black

    mylib::propagate -background to {comp1 comp2 comp3}
    mylib::propagate -foreground to {comp1 comp2 comp3}
}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *snit* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

Copyright

```
Copyright (c) 2003-2006, by William H. Duquette
```


soundex

soundex
Stichwörter: soundex, knuth, text comparison, text likeness

Name
soundex – Soundex

Übersicht
package require Tcl 8.5 9
package require soundex ?**1.1**?
::soundex::knuth *string*

Beschreibung
Dieses Paket enthält Soundex-Algorithmen, die den Vergleich von Wörtern auf der Grundlage ihrer phonetischen Ähnlichkeit ermöglichen.

Derzeit steht lediglich ein Algorithmus von Knuth zur Verfügung, der auf englische Namen und Wörter abgestimmt ist.

::soundex::knuth *string*
Berechnet den Soundex-Code der Eingabe *string* mit Knuths Algorithmus und gibt ihn als Ergebnis des Befehls zurück.

Beispiele
```
% ::soundex::knuth Knuth
K530
```

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *soundex* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) ????, Algorithm: Donald E. Knuth
Copyright (c) 2003, Documentation: Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 1998, Tcl port: Evan Rempel <erempel@uvic.ca>
```


stooop

stooop
Stichwörter: stooop, C++, class, object, object oriented

Name
stooop – Objektorientierte Erweiterung.

Übersicht
package require Tcl 8.5 9
package require stooop ?**4.4.2**?
::stooop::class *name body*
::stooop::new *class* ?*arg arg ...*?
::stooop::delete *object* ?*object ...*?
::stooop::virtual proc *name {this ?arg arg ...?}* ?*body*?
::stooop::classof *object*
::stooop::new *object*
::stooop::printObjects ?*pattern*?
::stooop::record
::stooop::report ?*pattern*?

Beschreibung
Dieses Paket stellt Befehle zur Verfügung, mit denen sich Tcl auf objektorientierte Weise erweitern lässt, wobei eine vertraute, C++-ähnliche Syntax und ein entsprechendes Verhalten zum Einsatz kommen. Stooop führt lediglich einige wenige neue Befehle ein: **class**, **new**, **delete**, **virtual** und **classof**. Zusammen mit einigen Programmierkonventionen ist das im Grunde alles, was Sie wissen müssen, um Stooop zu verwenden. Stooop soll so einfach wie möglich zu bedienen sein.

Dieses Handbuch ist sehr prägnant und dient als kurze Übersicht für den Programmierer, der zu diesem Zeitpunkt bereits die ausführliche HTML-Dokumentation *stooop_man.html* gelesen haben sollte.

::stooop::class *name body*
Dieser Befehl erzeugt eine Klasse. Der Klassenrumpf, inhaltlich einem Tcl-Namespace vergleichbar (eine Klasse ist tatsächlich auch einer), enthält Definitionen von

Member-Prozeduren. Member-Prozeduren lassen sich auch außerhalb des Klassenrumpfs definieren, indem ihrem Namen das Präfix **class::** vorangestellt wird, so wie bei Namespace-Prozeduren.

proc *class* {this ?*arg arg ...*?} ?*base* {?*arg arg ...*?} ...? *body*

Dies ist die Konstruktor-Prozedur für die Klasse. Sie wird nach einem Aufruf von **new** für die Klasse aufgerufen. Sie muss denselben Namen wie die Klasse tragen und ein erstes Argument mit dem Namen **this** enthalten. Vor dem eigentlichen Prozedurkörper sind beliebig viele Angaben zu Basisklassen zulässig, einschließlich der an deren Konstruktor zu übergebenden Argumente.

proc ~*class* {this} *body*

Dies ist die Destruktor-Prozedur der Klasse. Sie wird nach einem Aufruf von **delete** aufgerufen. Ihr Name muss aus einem einzelnen Zeichen **~** gefolgt vom Klassennamen bestehen (wie in C++). Sie muss ein einziges Argument namens **this** haben.

proc *name* {this ?*arg arg ...*?} *body*

Es handelt sich hierbei um eine Member-Prozedur der Klasse, da ihr erstes Argument den Namen **this** trägt. Sie ermöglicht innerhalb der Prozedur einen einfachen Zugriff auf die Member-Daten des Objekts, auf das **this** verweist. Beispiel:

```
set ($this,data) 0
```

proc *name* {?*arg arg ...*?} *body*

Es handelt sich hierbei um eine statische (im Sinne von C++) Member-Prozedur der Klasse, da ihr erstes Argument nicht den Namen "**this**" trägt. Auf statische (globale) Klassendaten kann wie folgt zugegriffen werden:

```
set (data) 0
```

proc *class* {this copy} *body*

Dies ist die optionale Kopierprozedur für die Klasse. Sie muss denselben Namen wie die Klasse tragen und genau zwei Argumente mit den Namen **this** und **copy** aufweisen. Sie wird nach einem Aufruf von **new** für ein vorhandenes Objekt der Klasse aufgerufen.

::stooop::new *class* ?*arg arg ...*?

Dieser Befehl erzeugt ein Objekt. Das erste Argument ist der Klassenname, danach folgen die Argumente, die der Konstruktor der entsprechenden Klasse benötigt. Zurückgegeben wird eine eindeutige Kennung für das soeben erzeugte Objekt.

::stooop::delete *object* ?*object ...*?

Dieser Befehl dient zum Löschen eines oder mehrerer Objekte. Er benötigt ein oder mehrere Objektkennungen als Argument(e).

::stooop::virtual proc *name* {this ?*arg arg ...*?} ?*body*?

Der Spezifizierer **virtual** kann bei Member-Prozeduren verwendet werden, um dynamische Bindung zu erreichen. Eine Prozedur einer Basisklasse kann dann in der abgeleiteten Klasse bzw. den abgeleiteten Klassen neu definiert (überladen) werden. Wird die Prozedur der Basisklasse für ein Objekt aufgerufen, wird tatsächlich die Prozedur der abgeleiteten Klasse aufgerufen, sofern sie vorhanden ist. Hat die Prozedur der Basisklasse keinen Prozedurrumpf, gilt sie als rein virtuell, und es wird stets die Prozedur der abgeleiteten Klasse aufgerufen.

::stooop::classof *object*

Dieser Befehl gibt die Klasse des vorhandenen Objekts zurück, das als einzelner Parameter übergeben wurde.

::stooop::new *object*

Dieser Befehl erzeugt ein Objekt durch Kopieren eines vorhandenen Objekts. Dabei wird der Kopierkonstruktor der entsprechenden Klasse aufgerufen, sofern er vorhanden ist; andernfalls werden die Daten-Member des kopierten Objekts einfach kopiert.

Fehlersuche

Umgebungsvariablen

siehe unten ...

STOOOPCHECKDATA

Steht diese Variable auf einem beliebigen wahren Wert, prüft stooop Zugriffe auf ungültige Member- oder Klassendaten.

STOOOPCHECKPROCEDURES

Steht diese Variable auf einem beliebigen wahren Wert, prüft stooop ungültige Argumente von Member-Prozeduren und die Instanziierung reiner Schnittstellenklassen.

STOOOPCHECKALL

Steht diese Variable auf einem beliebigen wahren Wert, schaltet stooop sowohl die Prüfung der Prozeduren als auch die der Daten-Member ein.

STOOOPCHECKOBJECTS

Steht diese Variable auf einem beliebigen wahren Wert, schaltet stooop die Objektprüfung ein. Danach stehen diese Prozeduren des stooop-Namespaces für die Fehlersuche bereit: **printObjects**, **record** und **report**.

STOOOPTRACEPROCEDURES

Steht diese Umgebungsvariable auf **stdout**, **stderr** oder einem Dateinamen, wird die Prozedurverfolgung aktiviert. Die stooop-Bibliothek gibt dann für jeden Aufruf einer Member-Prozedur eine Zeile Informationstext auf dem angegebenen Kanal aus.

STOOOPTRACEPROCEDURESFORMAT

Legt das Ausgabeformat der Prozedurverfolgung fest. Voreinstellung ist **"class: %C, procedure: %p, object: %O, arguments: %a"**.

STOOOPTRACEDATA

Steht diese Umgebungsvariable auf **stdout**, **stderr** oder einem Dateinamen, wird die Datenverfolgung aktiviert. Die stooop-Bibliothek gibt dann für jeden Zugriff auf Member-Daten eine Zeile Informationstext auf dem angegebenen Kanal aus.

STOOOPTRACEDATAFORMAT

Legt das Ausgabeformat der Datenverfolgung fest. Voreinstellung ist **"class: %C, procedure: %p, array: %A, object: %O, member: %m, operation: %o, value: %v"**.

STOOOPTRACEDATAOPERATIONS

Bei der Datenverfolgung werden standardmäßig alle Lese-, Schreib- und Unset-Zugriffe gemeldet. Die Variable kann auf eine beliebige Kombination der Buchstaben **r**, **w** und **u** gesetzt werden, um gezielter zu verfolgen (siehe die Tcl-Handbuchseite zu **trace**).

STOOOPTRACEALL

Steht diese Umgebungsvariable auf **stdout**, **stderr** oder einem Dateinamen, werden Prozedur- und Datenverfolgung zusammen aktiviert.

::stooop::printObjects ?*pattern*?
Gibt eine geordnete Liste der vorhandenen Objekte in der Reihenfolge ihrer Erstellung aus, wobei das älteste Objekt an erster Stelle steht. Jede Ausgabezeile enthält den Klassennamen, die Objektkennung und die Prozedur, in der die Erstellung erfolgte. Das optionale Musterargument (wie im Tcl-Befehl **string match**) kann verwendet werden, um die Ausgabe auf übereinstimmende Klassennamen zu beschränken.

::stooop::record
Beim Aufruf wird eine Momentaufnahme aller vorhandenen stooop-Objekte genommen. Mit report lässt sich später feststellen, welche Objekte in der Zwischenzeit erzeugt oder gelöscht wurden.

::stooop::report ?*pattern*?
Gibt die Objekte aus, die seit dem letzten Aufruf von **::stooop::record** erzeugt oder gelöscht wurden. Ist das Argument pattern angegeben, beschränkt es die Ausgabe auf passende Klassennamen.

Beispiele
Die vollständige HTML-Dokumentation finden Sie in der Datei *stooop_man.html*.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *stooop* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

switched
Stichwörter: switched, C++, class, object, object oriented

Name
switched – Verwaltung von Schaltern und Optionen.

Übersicht
package require Tcl 8.5 9
package require switched ?**2.2.2**?
complete *this*
options *this*
set-option *this value*

Beschreibung
Die Klasse "**switched**" dient als Basisklasse für Benutzerklassen mit Prozeduren für Schalter und Optionen. Sie bietet Funktionen zur Verwaltung von Optionen über eine einfache

Schnittstelle.

Beispiel:

```
set vehicle [new car -length 4.5 -width 2 -power 100 -fuel diesel]
puts "my car was running on [switched::cget $vehicle -fuel]"
switched::configure $vehicle -power 40 -fuel electricity
puts "but is now running on clean [switched::cget $vehicle -fuel]"
```

Wie Sie vielleicht bereits vermutet haben, leitet sich die Klasse "**car**" natürlich von der Klasse "**switched**" ab. Sehen wir uns einmal an, wie die Funktionsweise dieser abgeleiteten Klasse aussieht:

```
class car {
    proc car {this args} switched {$args} {
        # car specific initialization code here
        switched::complete $this
    }
    ...
}
```

Der Konstruktor der Klasse switched nimmt die optionalen Paare aus Konfigurationsoption und Wert als Parameter entgegen. Die Switched-Schicht verwaltet die Switched-Optionen dann vollständig: Sie prüft ihre Gültigkeit, speichert ihre Werte und stellt eine klare Schnittstelle zu den Konfigurationsprozeduren der Benutzerschicht bereit.

Dem Programmierer stehen diese Member der Klasse **switched** zur Verfügung:

complete *this*
: Diese Prozedur teilt der Switched-Schicht mit, dass das Objekt der abgeleiteten Klasse (in den Beispielen ein Auto) vollständig aufgebaut ist. Dann erfolgt die Erstkonfiguration des Switched-Objekts mit den Standardwerten der Optionen (siehe Prozedur **options**), gegebenenfalls überschrieben durch Werte aus der Konstruktion, die beim Aufruf des Operators **new** übergeben wurden. Diese Prozedur darf nur einmal aufgerufen werden, üblicherweise am Ende oder kurz vor dem Ende des Konstruktors der abgeleiteten Klasse. (*Hinweis*: auch das Daten-Member **complete** weiter unten in diesem Abschnitt).

options *this*
: Diese Prozedur muss die Konfigurationsbeschreibung für *alle* Optionen zurückgeben, die das Switched-Objekt akzeptiert. Sie ist eine rein virtuelle Member-Prozedur, ihre Implementierung in der abgeleiteten Klasse ist daher *obligatorisch*. Die Prozedur muss eine Liste von Listen zurückgeben. Jede Liste gehört zu einer einzelnen Option und besteht aus dem Schalternamen, dem Standardwert der Option und einem optionalen Anfangswert. Zum Beispiel:

  ```
  class car {
      ...
      proc options {this} {
          return [list [list -fuel petrol petrol] [list -length {} {}]
      [list -power {} {}] [list -width {} {}] ]
      }
  ```

```
    proc set-fuel {this value} {
        ...
    }
    ...
}
```

In diesem Fall werden vier Optionen festgelegt: **fuel**, **length**, **power** und **width**. Die Standard- und Anfangswerte für die Option **fuel** sind identisch und auf **petrol** festgelegt. Bei den übrigen Optionen sind alle Werte leer.

Für jede Option muss in der abgeleiteten Klasse eine entsprechende Prozedur **set-option** definiert sein. Da beispielsweise eine Option **fuel** definiert wurde, gibt es in der Klasse car eine Prozedur **set-fuel**. Die Parameter sind stets der Objektbezeichner (es handelt sich nicht um eine statische, sondern um eine dynamisch definierte virtuelle Prozedur), gefolgt vom neuen Wert der Option. Eine Prozedur **set-option** wird nur aufgerufen, wenn der neue Wert vom aktuellen abweicht (ein Cache zur Beschleunigung) oder wenn in der Prozedur **options** für diese Option kein Anfangswert gesetzt ist.

Weicht in dieser Prozedur der Anfangswert vom Standardwert ab oder wird er weggelassen, wird die Erstkonfiguration erzwungen, und die entsprechende Prozedur **set-option** wird von der Switched-Prozedur **complete** am Ende des Konstruktors der abgeleiteten Klasse aufgerufen. Zum Beispiel:

```
class car {
    ...
    proc options {this} {
        return [list [list -fuel petrol] [list -length {} {}] [list
    -power 100 50] [list -width {} {}] ]
    }
    ...
}
```

In diesem Fall wird die Konfiguration der Optionen **fuel** und **power** erzwungen, das heißt, die entsprechenden Prozeduren **set-option** werden beim Erzeugen des Switched-Objekts aufgerufen (siehe die Beschreibung der Prozeduren **set-option** weiter unten).

Da für **fuel** kein Anfangswert vorhanden ist, wird **set-fuel** mit dem Standardwert (**petrol**) als Argument aufgerufen. Bei **power** wird **set-power** mit dem Anfangswert als Argument (**50**) aufgerufen, weil der Anfangswert vom Standardwert abweicht.

Bei den übrigen Optionen werden die entsprechenden Prozeduren **set-option** nicht aufgerufen, weil die Anfangswerte (die letzten Elemente der Optionslisten) mit den Standardwerten übereinstimmen. Der Programmierer muss dafür sorgen, dass die Anfangswerte der Optionen korrekt sind.

set-option *this value*

Diese Prozeduren lassen sich als dynamische virtuelle Funktionen betrachten. Für jede unterstützte Option muss es eine Implementierung geben, wie sie die Prozedur **options** zurückgibt. Zum Beispiel:

```
class car {
    ...
    proc options {this} {
        return [list ...
            [list -width {} {}] ]
    }
    ...
    proc set-width {this value} {
        ...
    }
    ...
}
```

Da die Option **-width** in der Prozedur **options** aufgeführt wurde, gibt es eine Implementierung von **set-width**, die die Breite des Autos setzen würde (und zum Beispiel das Aussehen einer grafischen Darstellung anpassen würde).

Wird ein unterstütztes **option** in die von **options** zurückgegebene Liste aufgenommen, kann die entsprechende Prozedur **set-option** aufgerufen werden, sobald das Switched-Objekt vollständig ist; das ist der Fall, wenn die Prozedur **complete** der Switched-Ebene aufgerufen wird. Zum Beispiel:

```
class car {
    proc car {this args} switched {args} {
        ...
        switched::complete $this
   }
    ...
    proc options {this} {
        return [list [list -fuel petrol] [list -length 4.5] [list
    -power 350] [list -width 1.8] ]
    }
    proc set-fuel {this value} {
        ...
    }
    proc set-length {this value} {
        ...
    }
    proc set-power {this value} {
        ...
    }
    proc set-width {this value} {
        ...
    }
}

new car
```

In diesem Fall wird mit **new car** ein neues Auto ohne Optionen erzeugt. Dadurch wird der Konstruktor von car aufgerufen, der nach der vollständigen Initialisierung der car-Schicht wiederum die Prozedur **complete** der Switched-Ebene aufruft. Da zu diesem Zeitpunkt in keiner Optionsliste der Prozedur **options** Anfangswerte stehen, wird **set-fuel** mit dem Standardwert **petrol** als Parameter aufgerufen, danach **set-length** mit **4.5**, **set-power** mit **350** und schließlich **set-width** mit **1.8**. So lassen sich die Prozeduren **set-option** beim Debuggen prüfen; anschließend werden die Anfangswerte der Optionen eingetragen.

Die Switched-Schicht prüft, ob eine Option gültig ist (also in der Prozedur **options** aufgeführt ist), prüft aber nicht die Gültigkeit des an **set-option** übergebenen Werts. Die Prozedur soll einen Fehler auslösen (zum Beispiel mit dem Tcl-Befehl **error**), wenn der Wert ungültig ist.

Die Switched-Schicht merkt sich außerdem die aktuellen Werte der Optionen, sodass **set-option** nur aufgerufen wird, wenn der als Parameter übergebene Wert vom aktuellen Wert abweicht (siehe die Beschreibung der Daten-Member **-option**).

-option

Das Daten-Member **-option** hält den aktuellen Wert der Option. Für jede in der Prozedur **options** aufgeführte Option gibt es eines. Es ist ein nur lesbarer Wert, den die Switched-Schicht bei einer Änderung der Option zum Vergleich heranzieht. In der von switched abgeleiteten Schicht wird es selten verwendet, außer in wenigen Fällen wie dem folgenden Beispiel:

```
...
proc car::options {this} {
    return {
        ...
        {-manufacturer {} {}}
        ...
    }
}

proc car::set-manufacturer {this value} {}

proc car::printData {this} {
    puts "manufacturer: $switched::($this,-manufacturer)"
    ...
}
```

In diesem Fall wird der Name des Herstellers auf der Switched-Schicht gespeichert (deshalb hat die Prozedur **set-manufacturer** nichts zu tun) und später in der Prozedur **printData** gelesen.

complete

Das Daten-Member **complete** (nicht zu verwechseln mit der Prozedur **complete**) ist ein boolescher Wert. Sein Anfangswert ist **false**, und er wird ganz am Ende der Switched-Prozedur **complete** auf **true** gesetzt. Es ist nützlich, wenn bestimmte Optionen nur bei der Erzeugung und nicht dynamisch gesetzt werden sollen, wie das folgende Beispiel zeigt:

```
proc car::set-width {this value} {
    if {$switched::($this,complete)} {
        error {option -width cannot be set dynamically}
    }
    ...
}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *stooop* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

string

token

Stichwörter: token, lexing, regex, string, tokenization

Name

string::token – Iterative Lexem-Analyse auf Basis regulärer Ausdrücke

Übersicht

package require Tcl 8.5 9
package require string::token ?**1.1**?
package require fileutil
::string token text *lex string*
::string token file *lex path*
::string token chomp *lex startVar string resultVar*

Beschreibung

Dieses Paket enthält Befehle für die auf regulären Ausdrücken basierende Lexem-Analyse (Tokenisierung) von Zeichenketten.

API

::string token text *lex string*

Dieser Befehl nimmt ein Dictionary *lex* entgegen, das reguläre Ausdrücke ("regexps") auf Bezeichnungen abbilden muss, und zerlegt die Zeichenkette *string* gemäß diesem Dictionary in Token.

Der Befehl testet jeden regulären Ausdruck im *lex*-Dictionary der Reihe nach. Da Tcl-**dict**s in Einfügungsreihenfolge sortiert sind, wird empfohlen, die regulären Ausdrücke vom spezifischsten zum allgemeinsten zu legen.

Der Befehl gibt eine Liste von Tokens zurück. Jedes Token ist eine Liste mit drei Elementen, die die Bezeichnung (aus dem *lex*-**dict**) sowie den Start- und Endindex der Übereinstimmung in der *string* enthält.

Der Befehl löst einen Fehler aus, wenn er nicht in der Lage ist, die gesamte Zeichenkette zu tokenisieren.

Bei zeilenorientierten Lexern lassen sich durch zeilenweises Lexing eine feinere Fehlerbehandlung und die Verarbeitung großer Dateien erreichen.

::string token file *lex path*

Dieser Befehl ist eine praktische Hülle um **::string token text**, die das Tokenisieren ganzer Dateien vereinfacht.

Hinweis: Dieser Befehl lädt die gesamte Datei in den Arbeitsspeicher, bevor er mit der Verarbeitung beginnt.

::string token chomp *lex startVar string resultVar*

Dieser Befehl steht zur Verfügung, damit Benutzer eigene Lexer schreiben können. Beispielsweise, um je nach internem Zustand im Verlauf des Lexings unterschiedliche Lexing-Dictionaries anzuwenden.

(Die Befehle **::string token text** und **::string token file** nutzen diesen Befehl im Hintergrund.)

Dieser Befehl benötigt ein Dictionary *lex*, das reguläre Ausdrücke auf Bezeichnungen abbilden muss, eine Variable *startVar*, die die Indexposition enthält, an der die Lexem-Analyse in der Eingabe *string* beginnen soll, sowie eine Ergebnisvariable *resultVar*, an die jedes erfolgreich analysierte Lexem angehängt wird.

Der Befehl gibt einen Statuscode zurück, der den Zustand des Lexers angibt. Diese sind

0
: Es wurde kein Token gefunden.

1
: Token gefunden.

2
: Das Ende der Zeichenkette ist erreicht.

Wenn ein Token erkannt wurde (Status **1**), aktualisiert der Befehl den Index in *startVar* auf das Zeichen, das unmittelbar *auf* das lexikalisch analysierte Token folgt. Außerdem wird an *resultVar* das lexikalisch analysierte Token angehängt, das selbst eine Liste mit drei Elementen ist, die die Bezeichnung (aus dem *lex*-**dict**) sowie den Start- und End-Index der Übereinstimmung in der *Zeichenkette* enthält.

Wenn ein Token nicht erkannt wurde (Status **0** oder **2**), werden weder *startVar* noch *resultVar* aktualisiert.

Wie bereits erwähnt, testet der Befehl jeden regulären Ausdruck im *lex*-Dictionary der Reihe nach. Da Tcl-**dict**s in der Reihenfolge ihrer Einfügung sortiert sind, wird empfohlen, die regulären Ausdrücke vom spezifischsten zum allgemeinsten anzugeben.

Beachten Sie außerdem, dass allen Regex-Mustern implizit das Constraint-Escape-Zeichen **\A** vorangestellt wird, um sicherzustellen, dass eine Übereinstimmung genau an dem in *startVar* angegebenen Index beginnt.

Beispiele

Dieses Beispiel zeigt, wie Sie mit dem Befehl **string token text** ein einfaches zeilenorientiertes *.ini*-Dateiformat lexikalisch analysiert. (Informationen zur umfassenden Bearbeitung von *.ini* -Dateien finden Sie unter inifile.)

```
const INI_EG "; example.ini
Filename = test.db

invalid
\[Database\]
# remote data
Port: 143
IP: 192.0.2.62

junk
"

proc main {} {
    set groups [lex_ini $::INI_EG]
    puts General:
    dict for {key value} [dict get $groups General] { puts "\t$key:
    $value" }
    puts Database:
    dict for {key value} [dict get $groups Database] { puts "\t$key:
    $value" }
}

proc lex_ini ini_text {
    const INI_LEX [dict create {[#;].*$} COMMENT {\[[^]]+\]} GROUP
    {\w+\s*[:=].+$} ENTRY]
    set group General
    set linenum 0
    foreach line [split $ini_text \n] {
        incr linenum
        if {[string trim $line] eq ""} { continue }
        try {
            set token [string token text $INI_LEX $line]
            if {[llength $token]} { set token [lindex $token 0] }
            lassign $token label i j
```

```
            switch $label {
                COMMENT { continue }
                GROUP {
                    set group [string range $line $i+1 $j-1]
                    dict set groups $group {}
                }
                ENTRY {
                    set entry [string range $line $i $j]
                    set k [lindex [regexp -inline -indices {[=:]}
    $entry] 0 0]
                    set key [string trim [string range $entry 0 $k-1]]
                    set value [string trim [string range $entry $k+1
    end]]
                    dict set groups $group $key $value
                }
            }
        } on error err {
            puts "Error: line $linenum: $err"
        }
    }
    return $groups
}

main
=>
Error: line 4: Unexpected character 'i' at offset 0
Error: line 10: Unexpected character 'j' at offset 0
General:
    Filename: test.db
Database:
    Port: 143
    IP: 192.0.2.62
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

token_shell

Stichwörter: token_shell, bash, lexing, parsing, shell, string, tokenization

Name

string::token::shell – Analyse der Shell-Befehlszeile

Übersicht

package require Tcl 8.5 9
package require string::token::shell ?**1.3**?
package require string::token ?**1**?
package require fileutil
::string token shell ?**-indices**? ?**-partial**? ?--? *string*

Beschreibung

Dieses Paket enthält einen Befehl, der eine Textzeile mithilfe von grundlegenden **sh**-syntax in eine Liste von Wörtern zerlegt.

API

::string token shell ?-indices? ?-partial? ?--? *string*

Standardmäßig gibt der Befehl eine Liste von Wörtern aus der Eingabe *string* zurück. Die Wörter werden gemäß der grundlegenden sh(1p)-Syntax aus *string* aufgeteilt.

Es wird ein Fehler ausgelöst, wenn die Eingabe nicht der erwarteten Syntax entspricht.

Das Verhalten des Befehls kann durch Angabe einer der beiden Optionen **-indices** und **-partial** geändert werden.

--

Wenn diese Option festgelegt ist, wird die Auswertung an dieser Stelle beendet. Diese Option ist erforderlich, wenn die Eingabe *string* mit einem Bindestrich beginnen kann. Mit anderen Worten: Dies ist so gut wie zwingend erforderlich, wenn *string* eine Benutzereingabe ist.

-indices

Wenn dieser Befehl festgelegt wird, gibt er eine Liste von Listen mit jeweils 4 Elementen zurück. Jede dieser Listen enthält den Typ des Wortes, dessen Start- und Endindex im *string* sowie das Wort selbst (aus dem *string* kopiert).

Bei jeder beliebigen Liste mit vier Elementen stimmt die Länge des Wortes, die anhand der Differenz zwischen den Indizes berechnet wird, möglicherweise nicht mit der Länge des Wortes im letzten Element überein. Dies liegt daran, dass die Indizes Trennzeichen, Anführungszeichen innerhalb des Wortes usw. enthalten, während diese im Wort im letzten Element entfernt wurden.

Die möglichen Token-Typen sind

PLAIN

Einfaches Wort, nicht in Anführungszeichen gesetzt.

D:QUOTED

Das Wort wurde durch doppelte Anführungszeichen begrenzt.

S:QUOTED

Das Wort wurde durch einfache Anführungszeichen begrenzt.

D:QUOTED:PART
siehe unten ...

S:QUOTED:PART
Wie die vorherigen Typen, jedoch fehlt dem Wort das schließende Anführungszeichen, d. h., es ist unvollständig. Diese Token-Typen können nur auftreten, wenn die Option **-partial** festgelegt wurde, und auch dann nur für das letzte Wort des Ergebnisses. Wenn die Option **-partial** *nicht* angegeben ist, führt ein unvollständiges letztes Wort stattdessen dazu, dass der Befehl einen Fehler ausgibt.

-partial
Wenn diese Option festgelegt ist, akzeptiert der Parser ein unvollständiges Wort in Anführungszeichen (d. h. mit einem öffnenden Anführungszeichen, aber ohne passendes schließendes Anführungszeichen) am Ende der *Zeichenkette*, ohne einen Fehler auszugeben.

Die von diesem Befehl akzeptierte grundlegende Shell-Syntax besteht aus Wörtern ohne Anführungszeichen sowie Wörtern in einfachen und doppelten Anführungszeichen, die durch Leerzeichen voneinander getrennt sind. Leerzeichen am Anfang und am Ende sind ebenfalls zulässig und werden, sofern vorhanden, entfernt.

Shell-Variablen in ihren verschiedenen Formen werden *nicht* erkannt, ebenso wenig wie Sub-Shells.

Hier sind die anerkannten Wortformen:

single-quoted word
Ein in einfache Anführungszeichen gesetztes Wort beginnt mit einem einfachen Anführungszeichen, d. h. **'** (ASCII 39; U+0027), gefolgt von null oder mehr Unicode-Zeichen (mit Ausnahme des einfachen Anführungszeichens) und endet mit einem einfachen Anführungszeichen.

Auf das Wort muss entweder das Ende der *Zeichenkette* oder ein Leerzeichen folgen. Ein Wort darf nicht direkt auf ein anderes Wort folgen, ohne dass etwas dazwischen steht.

double-quoted word
Ein in doppelte Anführungszeichen gesetztes Wort beginnt mit einem doppelten Anführungszeichen, d. h. **"** (ASCII 34; U+0022), gefolgt von null oder mehr Unicode-Zeichen (mit Ausnahme des doppelten Anführungszeichens) und endet mit einem doppelten Anführungszeichen.

Anders als bei Wörtern in einfachen Anführungszeichen können doppelte Anführungszeichen in Wörtern in doppelten Anführungszeichen eingebettet werden, indem sie mit einem Backslash **** (ASCII 92; U+005C) maskiert werden, z. B. **"...\"..."**. Ebenso kann jeder literale Backslash durch eine Backslash-Escape-Sequenz eingebettet werden, z. B. **"...\\..."**.

unquoted word
Nicht in Anführungszeichen gesetzte Wörter werden nicht durch Anführungszeichen begrenzt und dürfen daher keine Leerzeichen oder einfache Anführungszeichen enthalten.

Doppelte Anführungszeichen und Backslashes können jedoch in nicht in Anführungszeichen gesetzte Wörter eingefügt werden, indem sie wie bei Wörtern in doppelten Anführungszeichen mit einem Escape-Zeichen versehen werden.

whitespace
Ein Leerzeichen ist jedes Unicode-Leerzeichen, d. h. jedes Zeichen, das string is space den Wert **1** (wahr) gibt oder das dem regulären Ausdruck **\\s** entspricht.

Leerzeichen *können* vor dem ersten Wort und nach dem letzten Wort vorkommen. Leerzeichen *müssen* zwischen benachbarten Wörtern vorkommen.

Beispiele

Ein Beispiel für eine Befehlszeile, die in ihre einzelnen Wörter zerlegt wurde.

```
const CMDLINE "grep --include=*.{tcl,tm,tk} -rl * | grep -v \"The
    Cloud\""
puts "CMDLINE=«$CMDLINE»"
foreach word [string token shell $CMDLINE] {
    puts "word=«$word»"
}
=>
CMDLINE=«grep --include=*.{tcl,tm,tk} -rl * | grep -v "The Cloud"»
word=«grep»
word=«--include=*.{tcl,tm,tk}»
word=«-rl»
word=«*»
word=«|»
word=«grep»
word=«-v»
word=«The Cloud»
```

Ein Beispiel für dieselbe Befehlszeile wie oben, zerlegt in ihre einzelnen Wörter, mit Tokens und Zeichenfolgen-Indizes.

```
foreach element [string token shell -indices $CMDLINE] {
    lassign $element token i j word
    puts "token=$token i=$i j=$j word=«$word»"
}
=>
token=PLAIN i=0 j=3 word=«grep»
token=PLAIN i=5 j=27 word=«--include=*.{tcl,tm,tk}»
token=PLAIN i=29 j=31 word=«-rl»
token=PLAIN i=33 j=33 word=«*»
token=PLAIN i=35 j=35 word=«|»
token=PLAIN i=37 j=40 word=«grep»
token=PLAIN i=42 j=43 word=«-v»
token=D:QUOTED i=45 j=55 word=«The Cloud»
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

stringprep

stringprep

Stichwörter: stringprep, unicode

Name

stringprep – Implementierung von stringprep

Übersicht

package require Tcl 8.5 9
package require stringprep 1.0.3
::stringprep::register *profile* ?*-mapping list*? ?*-normalization form*? ?*-prohibited list*? ?*-prohibitedList list*? ?*-prohibitedCommand command*? ?*-prohibitedBidi boolean*?
::stringprep::stringprep *profile string*
::stringprep::compare *profile string1 string2*

Beschreibung

Hierbei handelt es sich um eine Tcl-Implementierung der "Preparation of Internationalized Strings" ("stringprep"). Sie unterstützt die Definition von stringprep-Profilen und deren Verwendung zur Aufbereitung von Unicode-Zeichenketten für Vergleiche, wie in RFC 3454 definiert.

Hinweis: RFC 3454 (2002) wurde durch RFC 7564 (2015) ersetzt, der wiederum durch RFC 8264 (2017) ersetzt wurde.

Dieses Paket wird derzeit nicht gepflegt; siehe ticket.

API

Wenn als einzige Zeichenaufbereitung die Unicode-Normalisierung nötig ist, reichen die Befehle des Pakets **unicode** aus.

::stringprep::register ***profile*** **?*-mapping list*? ?*-normalization form*? ?*-prohibited list*? ?*-prohibitedList list*? ?*-prohibitedCommand command*? ?*-prohibitedBidi boolean*?**

Registriert ein **stringprep**-Profil mit dem Namen *profile*, das die angegebenen Optionen anwendet.

-mapping ***list***

Diese Option legt die Zuordnungstabellen **stringprep** fest. Es handelt sich dabei um die Tabellen in Anhang B von RFC 3454. Die üblichen Listenwerte sind {B.1 B.2} oder {B.1 B.3}, wobei B.1 Zeichen enthält, die üblicherweise auf nichts abgebildet werden, B.2 in Profilen mit der Unicode-Normalisierungsform KC verwendet wird und B.3 die Groß-/Kleinschreibungsfaltung festlegt. Der Standardwert ist eine leere Liste, was bedeutet, dass keine Zuordnung erfolgt.

-normalization *form*
Diese Option legt fest, welche Normalisierungsform angewendet werden soll. Die *form* ist eine Zeichenkette, die einen der folgenden Werte annehmen kann: "D", "C", "KD" oder "KC". Formal unterstützt RFC 3454 entweder keine Normalisierung oder "KC". Die Standardeinstellung ist keine Normalisierung.

-prohibited *list*
Diese Option legt eine Liste von RFC 3454-Tabellen mit unzulässigen Zeichen fest. Beispielsweise lautet die Liste aus RFC 3491 wie folgt: {A.1 C.1.2 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8 C.9}. Formal erlaubt RFC 3454 entweder das Verbieten aller Tabellen von C.3 bis C.9 oder keiner davon. Standardmäßig wird keine dieser Tabellen verboten.

-prohibitedList *list*
Diese Option legt eine Liste verbotener Zeichen als Unicode-Codepunkte fest. Um beispielsweise die Zeichen **" & ' /: < > @** zu verbieten, ist eine Liste ihrer Codepunkte anzugeben: {0x22 0x26 0x27 0x2F 0x3A 0x3C 0x3E 0x40}. Standardmäßig ist die Liste leer. (Wenn diese Zeichen festgelegt werden, gelten sie zusätzlich zu den durch die Option *-prohibited* verbotenen Zeichen.)

-prohibitedCommand *command*
Diese Option legt einen *command* fest, der mit einem Unicode-Codepunkt aufgerufen wird, sobald ein Zeichen zugeordnet und normalisiert wird. Wenn der *command* **1** (true) zurückgibt, wird das Zeichen als unzulässig behandelt. Ein Rückgabewert von **0** (false) gewährleistet die normale Verarbeitung. Diese Option ist eine nützliche Alternative dazu, der Option *-prohibitedList* eine sehr umfangreiche Liste zu übergeben. Standardmäßig ist kein Befehl festgelegt.

-prohibitedBidi *boolean*
Wenn diese Option festgelegt und mit einem Wert ungleich Null übergeben wird (z. B. **1**; true), werden die in Abschnitt 6 von RFC 3454 definierten Regeln zur bidirektionalen Zeichenverarbeitung verwendet. Standardmäßig werden die bidirektionalen Regeln nicht verwendet.

::stringprep::stringprep *profile string*
Gibt eine möglicherweise geänderte Kopie von *string* basierend auf dem angegebenen *profile* zurück; andernfalls wird ein Fehler ausgelöst.

Mögliche Fehler sind: *invalid_profile* – das angegebene *profile* ist nicht definiert; *prohibited_character* – die angegebene *string* enthält ein unzulässiges Zeichen; *prohibited_bidi* – die angegebene *string* enthält eine unzulässige bidirektionale Zeichenfolge.

::stringprep::compare *profile string1 string2*
Führt einen Vergleich der beiden angegebenen Zeichenfolgen durch, indem das Ergebnis der jeweiligen Zeichenfolgenvorbereitung gemäß dem angegebenen *profile* verglichen wird. Gibt **0** zurück, wenn die Zeichenfolgen (gemäß dem angegebenen *profile*) gleich sind, oder **-1**,

wenn *string1* lexikografisch kleiner ist als *string2*, bzw. **1**, wenn *string1* lexikografisch größer ist als *string2*.

Beispiele

Dieses Beispiel zeigt, wie sich mit **stringprep** Zeichenfolgen vergleichen lassen.

```
::stringprep::register basicprep -mapping {B.1 B.2} -normalization KC
# Three poor UTF-8 byte sequences (to illustrate stringprep, not to
    use in practice):
const EACUTE [encoding convertfrom utf-8 \x65\xCC\x81]
const ANGSTROM [encoding convertfrom utf-8 \xE2\x84\xAB]
const ODOTS [encoding convertfrom utf-8 \x6F\xCC\x88]
const A "Caf${EACUTE} ${ANGSTROM}ngstr${ODOTS}m"
# Best to either use a UTF-8-savvy editor:
const B "Café Ångström"
# Or to use Tcl's Unicode escapes:
const C "Caf\u00E9 \u212Bngstr\u00F6m"

puts "A '$A' hex: [binary encode hex [encoding convertto utf-8 $A]]"
puts "B '$B' hex: [binary encode hex [encoding convertto utf-8 $B]]"
puts "C '$C' hex: [binary encode hex [encoding convertto utf-8 $C]]"
puts "string compare \$A \$B → [expr {[string compare $A $B] ?
    "unequal" : "equal"}]"
puts "string compare \$A \$C → [expr {[string compare $A $C] ?
    "unequal" : "equal"}]"
puts "stringprep::compare basicprep \$A \$B → [expr
    {[stringprep::compare basicprep $A $B] ? "unequal" : "equal"}]"
puts "stringprep::compare basicprep \$A \$C → [expr
    {[stringprep::compare basicprep $A $C] ? "unequal" : "equal"}]"
=>
A 'Cafe´ Ångstro¨m' hex: 43616665cc8120e284ab6e677374726fcc886d
B 'Café Ångström' hex: 436166c3a920c3856e67737472c3b66d
C 'Café Ångström' hex: 436166c3a920e284ab6e67737472c3b66d
string compare $A $B → unequal
string compare $A $C → unequal
stringprep::compare basicprep $A $B → equal
stringprep::compare basicprep $A $C → equal
```

Ältere Beispiele

Definition des Nameprep-Profils (siehe RFC 3491).

```
::stringprep::register nameprep  -mapping {B.1 B.2}  -normalization KC
     -prohibited {A.1 C.1.2 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8 C.9}
    -prohibitedBidi 1
```

Definitionen der Profile Nodeprep und resourceprep (siehe RFC 3920).

```
::stringprep::register nodeprep  -mapping {B.1 B.2}  -normalization KC
     -prohibited {A.1 C.1.1 C.1.2 C.2.1 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8
    C.9}  -prohibitedList {0x22 0x26 0x27 0x2f 0x3a 0x3c 0x3e 0x40}
    -prohibitedBidi 1

::stringprep::register resourceprep  -mapping {B.1}  -normalization KC
     -prohibited {A.1 C.1.2 C.2.1 C.2.2 C.3 C.4 C.5 C.6 C.7 C.8 C.9}
    -prohibitedBidi 1
```

Autoren
Sergei Golovan

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *stringprep* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright

```
Copyright (c) 2007-2009, Sergei Golovan <sgolovan@nes.ru>
```


stringprep_data
Stichwörter: stringprep_data, stringprep, unicode

Name
stringprep::data – stringprep-Datentabellen, generiert, intern

Übersicht
package require Tcl 8.5 9
package require stringprep::data 1.0.3

Beschreibung
Das Paket **stringprep::data** ist ein Hilfspaket für **stringprep** und stellt diesem die Datentabellen bereit, die es für seine Funktionen braucht. Es ist ein *internes* Paket, auf das nicht eigenständig zugegriffen werden sollte. Deshalb hat es auch keine öffentlich dokumentierte API. Seine Implementierung wird durch ein Skript erzeugt.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *stringprep* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright


```
Copyright (c) 2007-2009, Sergei Golovan <sgolovan@nes.ru>
```


unicode

Stichwörter: unicode, normalization

Name

unicode – Implementierung der Unicode-Normalisierung

Übersicht

package require Tcl 8.5 9
package require unicode 1.1.1
::unicode::normalizeS *form string*
::unicode::fromstring *string*
::unicode::tostring *codepointsList*
::unicode::normalize *form codepointsList*

Beschreibung

Dies ist eine Tcl-Implementierung der Unicode-Normalisierungsbefehle.

Hinweis: Das Paket **stringprep** kann sich als nützlich erweisen, wenn die Normalisierung allein nicht ausreicht (z. B. bei Zeichenfolgenvergleichen).

API

Das Argument *form* in den unten aufgeführten Befehlen muss eine Zeichenkette sein, die einen der folgenden Werte annimmt: **D** (kanonische Zerlegung), **C** (kanonische Zerlegung, gefolgt von kanonischer Zusammensetzung), **KD** (Kompatibilitätszerlegung) oder **KC** (Kompatibilitätszerlegung, gefolgt von kanonischer Zusammensetzung).

::unicode::normalizeS *form string*
: Gibt eine möglicherweise geänderte Kopie von *string* zurück, die gemäß der angegebenen *form* normalisiert wurde.

  Dies ist eine Kurzform für: *::unicode::tostring* [*unicode::normalize $form* [*::unicode::fromstring $string*]].

::unicode::fromstring *string*
: Gibt eine Liste von ganzzahligen Unicode-Codepunkten zurück, die die Zeichen in *string* darstellen.

  (Diese ganzzahligen Codepunkte werden in **unicode** für die interne Darstellung von Zeichenfolgen verwendet.)

::unicode::tostring *codepointsList*
: Gibt eine Zeichenkette zurück, deren Zeichen den in *codepointsList* enthaltenen Ganzzahlen

entsprechen.

Jede Ganzzahl, die ein gültiger Unicode-Codepunkt ist, wird in der zurückgegebenen Zeichenkette durch das entsprechende Unicode-Zeichen dargestellt. Jede Ganzzahl, die kein gültiger Unicode-Codepunkt ist, wird durch das Unicode-Ersatzzeichen (?), U+FFFD, dargestellt, es sei denn, sie ist ausreichend groß; in diesem Fall wird ein Fehler ausgelöst.

::unicode::normalize *form codepointsList*

Gibt eine möglicherweise geänderte Kopie der *codepointsList* aus ganzen Zahlen zurück (die alle gültige Unicode-Codepunkte sein müssen), die gemäß der angegebenen Normalisierungsform *form* normalisiert wurde.

Beispiele

Einige Unicode-Zeichen verfügen in einer Unicode-Kodierung wie beispielsweise UTF-8 über mehr als eine Darstellung. Normalerweise stellt dies *innerhalb* einer Programmiersprache kein Problem dar, da die Sprache für ein bestimmtes Zeichen in der Regel stets dieselbe Darstellung verwendet. Bei Daten, die aus Dateien oder anderen externen Quellen eingelesen werden, könnte jedoch jede der möglichen Darstellungen verwendet werden. In diesen Fällen kann daher eine Normalisierung erforderlich sein.

Hier ist ein Beispiel für das Problem:

```
set a "Å"
set b "\u212B"
set c [encoding convertfrom utf-8 \xC3\x85]
set d [encoding convertfrom utf-8 \xE2\x84\xAB]
set e [encoding convertfrom utf-8 \x41\xCC\x8A]
set s [join [list $a $b $c $d $e] ""]
puts "s=$s a=b→[expr {$a eq $b}] a=c→[expr {$a eq $c}] a=d→[expr {$a
    eq $d}] a=e→[expr {$a eq $e}] b=c→[expr {$a eq $d}] b=d→[expr {$a
    eq $d}] b=e→[expr {$a eq $e}] c=d→[expr {$a eq $d}] c=e→[expr {$a
    eq $e}]"
=>
s=ÅÅÅÅA˚ a=b→0 a=c→1 a=d→0 a=e→0 b=c→0 b=d→0 b=e→0 c=d→0 c=e→0
```

Die obige Ausgabe zeigt, dass alle Zeichen zwar dieselben sind, jedoch unterschiedliche UTF-8-Darstellungen aufweisen und daher *nicht* als gleich verglichen werden.

Die Lösung besteht darin, die Daten zu normalisieren. Zum Beispiel:

```
set t [unicode::normalizeS KC $s]
set a [string index $t 0]
set b [string index $t 1]
set c [string index $t 2]
set d [string index $t 3]
set e [string index $t 4]
puts "t=$t a=b→[expr {$a eq $b}] a=c→[expr {$a eq $c}] a=d→[expr {$a
    eq $d}] a=e→[expr {$a eq $e}] b=c→[expr {$a eq $d}] b=d→[expr {$a
    eq $d}] b=e→[expr {$a eq $e}] c=d→[expr {$a eq $d}] c=e→[expr {$a
    eq $e}]"
=>
t=ÅÅÅÅÅ a=b→1 a=c→1 a=d→1 a=e→1 b=c→1 b=d→1 b=e→1 c=d→1 c=e→1
```

Nach der Normalisierung weisen alle Zeichen dieselbe Darstellung auf und werden korrekt als gleichwertig verglichen.

Ältere Beispiele

```
% ::unicode::fromstring "\u0410\u0411\u0412\u0413"
1040 1041 1042 1043
% ::unicode::tostring {49 50 51 52 53}
12345
%
```

```
% ::unicode::normalize D {7692 775}
68 803 775
% ::unicode::normalizeS KD "\u1d2c"
A
%
```

Literatur

1. "Unicode Standard Annex #15: Unicode Normalization Forms", (http://unicode.org/reports/tr15/)

Autoren

Sergei Golovan

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *stringprep* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright

Copyright (c) 2007, Sergei Golovan <sgolovan@nes.ru>

unicode_data

Stichwörter: unicode_data, stringprep, unicode

Name

unicode::data – unicode-Datentabellen, generiert, intern

Übersicht

package require Tcl 8.5 9
package require unicode::data 1.1.1

Beschreibung

Das Paket **unicode::data** ist ein Hilfspaket für **unicode** und stellt diesem die Datentabellen bereit, die es für seine Funktionen braucht. Es ist ein *internes* Paket, auf das nicht eigenständig zugegriffen werden sollte. Deshalb hat es auch keine öffentlich dokumentierte API. Seine Implementierung wird durch ein Skript erzeugt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *stringprep* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright

Copyright (c) 2007, Sergei Golovan <sgolovan@nes.ru>

struct

disjointset

Stichwörter: disjointset, disjoint set, equivalence class, find, merge find, partition, partitioned set , union

Name

struct::disjointset – Datenstruktur "Disjunkte Menge"

Übersicht

package require Tcl 8.6 9
package require struct::disjointset ?**1.2**?
::struct::disjointset *aDisjointSet*
aDisjointSet **add-element** *element*
aDisjointSet **add-partition** *elements*

aDisjointSet **partitions**
aDisjointSet **num-partitions**
aDisjointSet **equal** *a b*
aDisjointSet **merge** *a b*
aDisjointSet **find** *element*
aDisjointSet **exemplars**
aDisjointSet **find-exemplar** *element*
aDisjointSet **destroy**

Beschreibung

Dieses Paket enthält einen Befehl zur Verarbeitung von disjunkten Mengen. Alternative Namen für diese Art von Datenstruktur sind *union-find* und *merge-find*.

Ein häufiger Anwendungsfall für gewöhnliche Mengen ist die Beantwortung der Frage: "Enthält die Menge S das Element E?". Die Antwort ist ein einfacher Boolescher Wert.

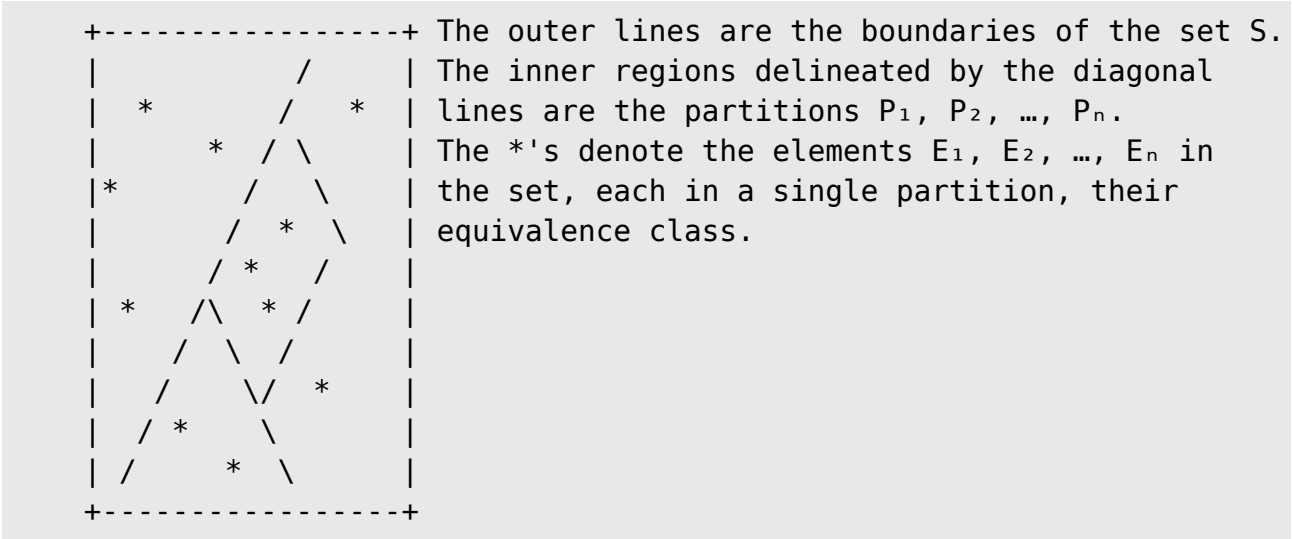
Ein häufiger Anwendungsfall für disjunkte Mengen ist die Beantwortung der Frage: "Welche der Mengen S_1, S_2, …, S_n enthält das Element E?". Die Antwort ist eine Menge – oder nichts, falls keine der Mengen E enthält.

Ein weiterer häufiger Anwendungsfall für disjunkte Mengen ist die Möglichkeit, zwei Mengen schnell zu einer einzigen zusammenzuführen, wobei die resultierende Menge weiterhin eine schnelle Suche nach Elementen ermöglicht. Daher der Begriff *Merge-Find*.

Der Grund, warum diese Datenstruktur als *disjunkte Menge* bezeichnet wird, liegt darin, dass sie als *eine einzige* Menge mit *Partitionen* betrachtet werden kann. Mit anderen Worten: Eine disjunkte Menge ist:

- eine endliche *Menge* S, die enthält
- eine Reihe von *Elementen* E_1, E_2, …, E_n, gruppiert in
- eine Menge von *Partitionen* P_1, P_2, …, P_n. Der letztgenannte Begriff trifft zu, da die Schnittmenge jedes Paares P_i, P_j von Partitionen leer ist, d. h., $\emptyset = P_i \cap P_j$, wobei die Menge selbst der Vereinigung aller Partitionen entspricht, d. h., $S = P_1 \cup P_2 \cup \ldots \cup P_n$.

Ein alternativer Name für die *Partitionen* wäre *Äquivalenzklassen*, wobei alle Elemente derselben Klasse als gleich angesehen werden. Hier ist eine grafische Darstellung der oben aufgeführten Konzepte:



API

Das Paket stellt einen einzigen Befehl bereit, **::struct::disjointset**, der alle seine Funktionen über Methoden bereitstellt.

::struct::disjointset *aDisjointSet*

Erstellt ein neues Objekt vom Typ "disjoint set" mit einem zugehörigen globalen Tcl-Befehl namens *aDisjointSet*. Dieser Befehl kann verwendet werden, um Methoden für disjunkte Mengen aufzurufen, die alle die folgende allgemeine Form haben:

aDisjointSet method ?*arg ...*?

Die *Methode* und die *Argumente* legen fest, welche Operation ausgeführt werden soll.

Methoden

aDisjointSet* add-element *element

Erstellt eine neue Partition im *aDisjointSet* und fügt das einzelne *Element* zur neuen Partition hinzu. Der Befehl gewährleistet die Integrität der disjunkten Menge, d. h., wenn *element* bereits in der disjunkten Menge enthalten ist – unabhängig davon, in welcher Partition –, löst diese Methode einen Fehler aus.

Diese Methode gibt die leere Zeichenkette zurück.

Diese Methode läuft in konstanter Zeit, *O(1)*.

aDisjointSet* add-partition *elements

Erstellt eine neue Partition im *aDisjointSet* und fügt alle angegebenen *Elemente* zur neuen Partition hinzu. Der Befehl gewährleistet die Integrität der disjunkten Menge, d. h., wenn eines der Elemente in *Elemente* bereits in der disjunkten Menge enthalten ist – unabhängig davon, in welcher Partition –, löst diese Methode einen Fehler aus.

Diese Methode gibt die leere Zeichenkette zurück.

Die Laufzeit dieser Methode ist proportional zur Größe von *Elementen*, *O(N)*, wobei *N* die Anzahl der *Elemente* ist.

***aDisjointSet* partitions**

Gibt die Partitionen des *aDisjointSet* als Liste von Listen zurück. Die äußeren Listen sind die Partitionen, die inneren Listen enthalten die Elemente der jeweiligen Partition.

Diese Methode läuft in $O(N \times \alpha(N))$ Zeit, wobei *N* die Anzahl der Elemente in der disjunkten Menge ist und die Funktion α die inverse Ackermann-Funktion ist.

***aDisjointSet* num-partitions**

Gibt die Anzahl der Partitionen im *aDisjointSet* zurück.

Diese Methode läuft in konstanter Zeit.

aDisjointSet* equal *a b

Gibt **1** (wahr) zurück, wenn die Elemente *a* und *b* in der *aDisjointSet* in derselben Partition liegen; andernfalls gibt es **0** (falsch) zurück.

Wenn eines oder beide Argumente überhaupt nicht in der disjunkten Menge enthalten sind, löst diese Methode einen Fehler aus.

Diese Methode läuft in amortisierter Zeit *O(α(N))*, wobei *N* die Anzahl der Elemente in der größeren Partition ist und die Funktion *α* die inverse Ackermann-Funktion ist.

aDisjointSet* merge *a b

Ermittelt die Partitionen im *aDisjointSet*, in denen die Elemente *a* und *b* enthalten sind, und führt diese Partitionen zu einer einzigen Partition zusammen. Wenn die beiden Elemente bereits in derselben Partition enthalten waren, bleibt die disjunkte Menge unverändert.

Wenn eines oder beide Argumente überhaupt nicht in der disjunkten Menge sind, löst diese Methode einen Fehler aus.

Diese Methode gibt die leere Zeichenkette zurück.

Diese Methode läuft in amortisierter Zeit *O(α(N))*, wobei *N* die Anzahl der Elemente in der größeren der zusammenzuführenden Partitionen ist und die *α*-Funktion die inverse Ackermann-Funktion ist. Die Laufzeit im schlimmsten Fall beträgt *O(N)*.

aDisjointSet* find *element

Gibt eine Liste der Elemente der Partition des *aDisjointSet* zurück, die das angegebene *element* enthält; oder eine leere Zeichenkette, wenn *element* nicht in der disjunkten Menge enthalten ist.

Diese Methode läuft in *O(N × α(N))* Zeit, wobei *N* die Gesamtzahl der Elemente in der disjunkten Menge ist und die Funktion *α* die inverse Ackermann-Funktion ist. Siehe **find-exemplar** für eine schnellere Methode, wenn lediglich eine eindeutige Partitionskennung benötigt wird und keine Liste ihrer Elemente.

***aDisjointSet* exemplars**

Gibt eine Liste zurück, die für jede Partition in *aDisjointSet* ein Exemplar – ein beliebig ausgewähltes Element – enthält.

Hinweis: Die einzige Operation, die das für eine bestimmte Partition ausgewählte Exemplar ändert, ist **merge**.

Diese Methode läuft in *O(N × α(N))* Zeit, wobei *N* die Gesamtzahl der Elemente in der disjunkten Menge ist und die Funktion *α* die inverse Ackermann-Funktion ist.

aDisjointSet* find-exemplar *element

Gibt ein Exemplar – ein beliebig ausgewähltes Element – der Partition des *aDisjointSet* zurück, die das angegebene *element* enthält.

Befindet sich *element* überhaupt nicht in der disjunkten Menge, löst diese Methode einen Fehler aus.

Hinweis: Die einzige Operation, die das für eine bestimmte Partition ausgewählte Exemplar verändert, ist **merge**.

Diese Methode läuft in *O(α(N))* Zeit, wobei *N* die Anzahl der Elemente in der Partition ist, die *element* enthält, und die *α*-Funktion die inverse Ackermann-Funktion ist.

***aDisjointSet* destroy**

Löscht das *aDisjointSet*-Objekt, einschließlich des globalen Befehls *aDisjointSet*, sowie den gesamten damit verbundenen Speicher.

Beispiele

In den Beispielen werden die Ergebnisse mithilfe der folgenden **proc**-Prozedur dargestellt:

```
proc dump_disjointset name {
    puts "$name:"
    foreach partition [$name partitions] {
        puts -nonewline "  partition "
        set sep "{"
        foreach element $partition {
            puts -nonewline $sep$element
            set sep " "
        }
        puts "}"
    }
}
```

In diesen Beispielen wird eine neue disjunkte Menge namens *colors* erstellt und es werden eine Partition mit dem einzigen Element *black* sowie eine weitere Partition mit den Elementen *cyan*, *magenta* und *yellow* hinzugefügt.

```
struct::disjointset colors
colors add-element black
puts "colors num-partitions=[colors num-partitions]"
colors add-partition {cyan magenta yellow}
puts "colors num-partitions=[colors num-partitions]"
dump_disjointset colors
=>
colors num-partitions=1
colors num-partitions=2
colors:
  partition {black}
  partition {cyan magenta yellow}
```

In diesen Beispielen werden einige Funktionen der Disjoint-Set-API anhand des oben erstellten Disjoint-Sets *colors* vorgestellt.

```
puts "colors equal black yellow=[colors equal black yellow]"
puts "colors equal cyan yellow=[colors equal cyan yellow]"
puts "colors equal cyan magenta=[colors equal cyan magenta]"
colors merge black cyan
puts "colors equal black yellow=[colors equal black yellow]"
```

```
dump_disjointset colors
=>
colors equal black yellow=0
colors equal cyan yellow=1
colors equal cyan magenta=1
colors merge black cyan
colors equal black yellow=1
colors:
  partition {black cyan magenta yellow}
```

In diesem Beispiel werden einige Funktionen der Disjoint-Set-API anhand des oben erstellten Disjoint-Sets *colors* weiter untersucht.

```
colors add-partition {red green blue}
dump_disjointset colors
puts "[colors find-exemplar blue] [colors find-exemplar yellow]"
colors destroy
=>
colors:
  partition {black cyan magenta yellow}
  partition {red green blue}
red cyan
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: disjointset* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

graph

Stichwörter: graph, adjacent, arc, cgraph, degree, edge, loop, neighbour, node, serialization, subgraph, vertex

Name

struct::graph – Gerichtete Graphobjekte erzeugen und bearbeiten

Übersicht

package require Tcl 8.5 9
package require struct::graph ?**2.4.4**?
package require struct::list ?**1.8.6**?
package require struct::set ?**2.2.4**?

::struct::graph ?*aGraph*? ?=|:=|*as*|*deserialize source*?
aGraph **=** *sourceGraph*
aGraph **-->** *destGraph*
aGraph **append** *key value*
aGraph **deserialize** *serialization*
aGraph **destroy**
aGraph **arc append** *arc key value*
aGraph **arc attr** *key*
aGraph **arc attr** *key* **-arcs** *list*
aGraph **arc attr** *key* **-glob** *globpattern*
aGraph **arc attr** *key* **-regexp** *repattern*
aGraph **arc delete** *arc* ?*arc ...*?
aGraph **arc exists** *arc*
aGraph **arc flip** *arc*
aGraph **arc get** *arc key*
aGraph **arc getall** *arc* ?*glob*?
aGraph **arc getunweighted**
aGraph **arc getweight** *arc*
aGraph **arc keys** *arc* ?*glob*?
aGraph **arc keyexists** *arc key*
aGraph **arc insert** *start end* ?*name*?
aGraph **arc lappend** *arc key value*
aGraph **arc rename** *arc newname*
aGraph **arc set** *arc key* ?*value*?
aGraph **arc setunweighted** ?*weight*?
aGraph **arc setweight** *arc weight*
aGraph **arc unsetweight** *arc*
aGraph **arc hasweight** *arc*
aGraph **arc source** *arc*
aGraph **arc target** *arc*
aGraph **arc nodes** *arc*
aGraph **arc move-source** *arc newsource*
aGraph **arc move-target** *arc newtarget*
aGraph **arc move** *arc newsource newtarget*
aGraph **arc unset** *arc key*
aGraph **arc weights**
aGraph **arcs** ?*-key key -value value*? ?*-filter cmdprefix*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding node ?node*? ...?
aGraph **lappend** *key value*
aGraph **node append** *node key value*
aGraph **node attr** *key*
aGraph **node attr** *key* **-nodes** *list*
aGraph **node attr** *key* **-glob** *globpattern*
aGraph **node attr** *key* **-regexp** *repattern*
aGraph **node degree** ?*-in*|*-out*? *node*
aGraph **node delete** *node* ?*node...*?

aGraph **node exists** *node*
aGraph **node get** *node key*
aGraph **node getall** *node* ?*glob*?
aGraph **node keys** *node* ?*glob*?
aGraph **node keyexists** *node key*
aGraph **node insert** ?*node...*?
aGraph **node lappend** *node key value*
aGraph **node opposite** *node arc*
aGraph **node rename** *node newname*
aGraph **node set** *node key* ?*value*?
aGraph **node unset** *node key*
aGraph **nodes** ?*-key key*? ?*-value value*? ?*-filter cmdprefix*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding node node...*?
aGraph **get** *key*
aGraph **getall** ?*pattern*?
aGraph **keys** ?*pattern*?
aGraph **keyexists** *key*
aGraph **serialize** ?*node...*?
aGraph **set** *key* ?*value*?
aGraph **swap** *node1 node2*
aGraph **unset** *key*
aGraph **walk** *node* ?*-type type*? ?*-order order*? ?*-dir direction*? -command *cmd*

Beschreibung

Ein gerichteter Graph ist eine Datenstruktur mit zwei Mengen von Elementen, *Knoten* und *Bögen* (auch *Kanten* genannt), sowie einer Beziehung, die Knoten und Bögen verbindet.

Jeder Bogen verbindet zwei Knoten. Einer heißt *Quelle*, der andere *Ziel*. Das gibt dem Bogen eine Richtung: Er führt von der Quelle zum Ziel. Sind Quelle und Ziel derselbe Knoten, heißt der Bogen eine *Schleife*.

Ist ein Knoten Quelle oder Ziel eines Bogens, heißen Knoten und Bogen *benachbart*. Sind zwei Knoten über einen oder mehrere Bögen verbunden, heißen auch die Knoten *benachbart*.

Ein Knoten kann Quelle beliebig vieler *ausgehender Bögen* und Ziel beliebig vieler *eingehender Bögen* sein. Die Zahl der *ausgehenden Bögen* ist der *Ausgangsgrad* des Knotens, die Zahl der *eingehenden Bögen* der *Eingangsgrad*.

Das Paket **struct::graph** verwaltet Graphen aus Knoten, Bögen und ihren Beziehungen. Zusätzlich lassen sich beliebig viele benannte *Attribute* mit dem Graphen selbst und mit jedem seiner Knoten und Bögen verbinden.

Befehle für eine große Zahl üblicher Graphalgorithmen stehen im Paket **struct::graph::ops**.

Hinweis: Die Hauptversion von **struct::graph** ist auf Version 2 gewechselt, weil sich die API des Moduls inkompatibel geändert hat. Wer Version 1 nutzt, liest den Abschnitt Änderungen für v2. Dort steht die vollständige Liste der Änderungen, einschließlich der Inkompatibilitäten mit Version 1.

Ab Version 2.2 gibt es eine C-Implementierung auf critcl-Basis. Sie verlangt Tcl 8.4 oder neuer.

API

::struct::graph ?*aGraph*? ?=|:=|as|deserialize *source*?

Dieser Befehl erzeugt ein neues Graphobjekt mit einem zugehörigen globalen Tcl-Befehl. Ist *aGraph* angegeben, trägt der globale Befehl diesen Namen; sonst erzeugt das Paket einen eindeutigen Namen.

Ist eine *source* angegeben, wird der neue Graph daraus initialisiert. Bei den Operatoren **=**,**:=** und **as** ist das Argument *source* der Name eines anderen Graphobjekts, und der Zuweisungsoperator **=** wird ausgeführt. Beim Operator **deserialize** gilt *source* als serialisiertes Graphobjekt, und **deserialize** wird ausgeführt.

Zum Beispiel:

```
    ::struct::graph graphA = graphB
```

entspricht

```
    ::struct::graph graphA
    graphA = graphB
```

und

```
    ::struct::graph graphA deserialize $graphB
```

entspricht

```
    ::struct::graph graphA
    graphA deserialize $graphB
```

Sobald das Graphobjekt und der zugehörige Befehl erzeugt sind, dient der Befehl den verschiedenen Graphoperationen. Sie haben alle dieselbe allgemeine Form:

aGraph method ?*arg ...*?

Die *method* und die *arg*s legen fest, welche Operation ausgeführt wird.

Methoden

aGraph* = *sourceGraph

Der Operator **=** ist der *Zuweisungs*operator für Graphobjekte. Er kopiert den Graphen aus dem Graphobjekt *sourceGraph* über die Graphdaten in *aGraph*. Der bisherige Inhalt von *aGraph* wird dabei gelöscht.

Die Operation entspricht praktisch

```
    aGraph deserialize [sourceGraph serialize]
```

Die Operation setzt voraus, dass *sourceGraph* eine Methode **serialize** bereitstellt, die eine gültige Graphserialisierung zurückgibt.

aGraph* --> *destGraph

Der Operator **-->** ist der Operator der *umgekehrten Zuweisung* für Graphobjekte. Er kopiert den Graphen aus dem Graphobjekt *aGraph* über die Graphdaten im Objekt *destGraph*. Der

bisherige Inhalt von *destGraph* wird dabei gelöscht.

Die Operation entspricht praktisch

```
    destGraph deserialize [aGraph serialize]
```

Die Operation setzt voraus, dass *destGraph* eine Methode **deserialize** bereitstellt, die eine Graphserialisierung entgegennimmt, und dass *aGraph* eine Methode **serialize** bereitstellt, die eine gültige Graphserialisierung zurückgibt.

aGraph* append *key value
Hängt *value* an das Graphattribut zum angegebenen *key* an. Gibt den neuen Wert des Graphattributs zu *key* zurück.

aGraph* deserialize *serialization
Das ist das Gegenstück zur Methode **serialize**. Sie ersetzt die Graphdaten in *aGraph* durch den Graphen, den der Wert *serialization* beschreibt. Der bisherige Inhalt von *aGraph* wird dabei gelöscht.

***aGraph* destroy**
Zerstört den Graphen, einschließlich seines Speichers und des zugehörigen Befehls.

aGraph* arc append *arc key value
Hängt *value* an das Attribut des angegebenen *arc* zum angegebenen *key* an. Gibt den neuen Wert des Attributs von *arc* zu *key* zurück.

aGraph* arc attr *key
siehe unten ...

aGraph* arc attr *key* -arcs *list
siehe unten ...

aGraph* arc attr *key* -glob *globpattern
siehe unten ...

aGraph* arc attr *key* -regexp *repattern
Sind keine Optionen angegeben, gibt die Methode **arc attr** ein **dict** zurück, dessen Schlüssel die Namen *jedes* Bogens im Graphen *aGraph* sind. Zu jedem Schlüssel dieses **dict** ist der zugehörige Wert der Wert des Bogenattributs zum angegebenen *key*. Bögen ohne diesen *key* fehlen im Ergebnis-**dict**.

Ist eine der Optionen angegeben, entsteht praktisch dasselbe Ergebnis-**dict** wie ohne Optionen, gefiltert auf die Bögen, die der Einschränkung der Option entsprechen.

Die möglichen Einschränkungen sind:

-arcs
Der Wert muss eine Liste von Bogennamen sein. Nur die in dieser Liste genannten Bögen werden nach dem Attribut *key* durchsucht.

-glob
Der Wert ist ein Glob-Muster. Nur die Bögen im Graphen, deren Namen diesem Muster entsprechen, werden nach dem Attribut *key* durchsucht.

-regexp
Der Wert ist ein regulärer Ausdruck. Nur die Bögen im Graphen, deren Namen diesem Muster entsprechen, werden nach dem Attribut *key* durchsucht.

***aGraph* arc delete *arc* ?*arc* ...?**
Entfernt die angegebenen Bögen aus dem Graphen.

Existiert einer der Bögen nicht, wird ein Fehler ausgelöst.

aGraph* arc exists *arc
Gibt **1** (wahr) zurück, wenn der angegebene *arc* im Graphen vorhanden ist, sonst **0** (falsch).

aGraph* arc flip *arc
Kehrt die Richtung von *arc* um, das heißt, Quell- und Zielknoten des Bogens werden vertauscht.

aGraph* arc get *arc key
Gibt den Wert zu *key* von *arc* zurück.

Existiert *arc* nicht oder existiert *key* nicht, wird ein Fehler ausgelöst.

***aGraph* arc getall *arc* ?*glob*?**
Gibt ein Dictionary (geeignet für [**array set**]) der Attributschlüssel und -werte von *arc* zurück.

Ist ein *glob*-Muster angegeben, werden nur die Attribute aufgenommen, deren Schlüssel passen.

***aGraph* arc getunweighted**
Gibt eine Liste der Namen aller Bögen des Graphen zurück, denen *kein* Gewicht zugeordnet ist.

aGraph* arc getweight *arc
Gibt das *arc* zugeordnete Gewicht zurück.

Existiert *arc* nicht oder hat er kein Gewicht, wird ein Fehler ausgelöst.

***aGraph* arc keys *arc* ?*glob*?**
Gibt eine Liste der Schlüssel von *arc* zurück.

Ist *glob* angegeben, gehören nur die Attribute zur Liste, deren Namen zum Glob passen.

aGraph* arc keyexists *arc key
Gibt **1** (wahr) zurück, wenn *arc* den angegebenen *key* hat, sonst **0** (falsch).

***aGraph* arc insert *start end* ?*name*?**
Fügt einen Bogen in den Graphen ein, der am Knoten *start* beginnt und am Knoten *end* endet. Der neue Bogen heißt *name*, wenn das angegeben ist; sonst wird ein eindeutiger, vom System erzeugter Name der Form *arcx* verwendet.

aGraph* arc lappend *arc key value
Hängt den angegebenen *value* (als Liste) an den Wert zu *key* von *arc* an und gibt den neuen Wert von *key* von *arc* zurück.

aGraph* arc rename *arc newname
Benennt *arc* in *newname* um und gibt *newname* zurück.

Löst einen Fehler aus, wenn der angegebene *arc* nicht existiert oder bereits ein Bogen namens *newname* existiert.

***aGraph* arc set *arc key* ?*value*?**
Liest oder setzt den Wert des *arc* zugeordneten *key*.

Ist *value nicht* angegeben, wird der Wert des *arc* zugeordneten *key* zurückgegeben. Ist ein *value* angegeben, wird dieser *value* dem angegebenen *key* zugewiesen, und der Wert des *arc* zugeordneten *key* wird zurückgegeben.

Einem Bogen können beliebig viele Schlüssel-Wert-Paare zugeordnet sein.

***aGraph* arc setunweighted ?*weight*?**
Setzt das Gewicht aller Bögen ohne Gewicht auf *weight*. Ist kein *weight* angegeben, ist die Vorgabe **0**. Gibt die leere Zeichenkette zurück.

aGraph* arc setweight *arc weight
Setzt das Gewicht von *arc* auf *weight* und gibt *weight* zurück.

aGraph* arc unsetweight *arc
Entfernt das Gewicht von *arc*, falls vorhanden; sonst geschieht nichts. Gibt die leere Zeichenkette zurück.

aGraph* arc hasweight *arc
Gibt **1** (wahr) zurück, wenn *arc* ein Gewicht hat, sonst **0** (falsch).

aGraph* arc source *arc
Gibt den Knoten zurück, an dem der angegebene *arc* beginnt.

aGraph* arc target *arc
Gibt den Knoten zurück, an dem der angegebene *arc* endet.

aGraph* arc nodes *arc
Gibt eine zweielementige Liste der Knoten zurück, an denen der angegebene *arc* beginnt und endet.

aGraph* arc move-source *arc newsource
Setzt den Quellknoten von *arc* auf *newsource*. Das lässt sich als Drehen des Bogens um seinen Zielknoten beschreiben.

aGraph* arc move-target *arc newtarget
Setzt den Zielknoten von *arc* auf *newtarget*. Das lässt sich als Drehen des Bogens um seinen Quellknoten beschreiben.

aGraph* arc move *arc newsource newtarget
Setzt Quell- und Zielknoten von *arc* auf *newsource* beziehungsweise *newtarget*.

aGraph* arc unset *arc key
Entfernt *key* von *arc* und den zugehörigen Wert. Es geschieht nichts, wenn *key* nicht existiert.

***aGraph* arc weights**
Gibt ein **dict** zurück, in dem jeder Schlüssel der Name eines Bogens mit Gewicht ist und jeder zugehörige Wert das Gewicht dieses Bogens.

***aGraph* arcs ?-key *key* ?-value *value*?? ?-filter *cmdprefix*?**
?-in|-out|-adj|-inner|-embedding *node* ?node? ...?
Gibt eine Liste der Bögen des Graphen zurück.

Ist keine Einschränkung angegeben, steht jeder Bogen in der Liste.

Einschränkungen

Einschränkungen können begrenzen, welche Bögen in die Liste kommen: anhand der Knoten, die der Bogen verbindet, anhand der dem Bogen zugeordneten Schlüsselwerte oder anhand

von beidem. Ein allgemeiner Filterbefehl ist ebenfalls möglich. Einschränkungen, die verbundene Knoten betreffen, nehmen eine variable Zahl von Knoten als Argument, angegeben nach dem Namen der Einschränkung selbst.

Höchstens eine Einschränkung **-key** darf angegeben werden, und wenn sie vorhanden ist, höchstens eine Einschränkung **-value**. **key** (und, falls vorhanden, **-value**) wird, wenn angegeben, *als Zweites* angewandt. (Werden zwei oder mehr Optionen **-key** verwendet oder wird **-value** ohne zugehörigen **key** verwendet, wird ein Fehler ausgelöst.)

-key *key*
: Gibt nur Bögen zurück, die den zugeordneten *key* haben.

-value *value*
: Gibt nur Bögen zurück, die den zugeordneten *key* haben (der *zwingend* mit der Option **-key** anzugeben ist), *und* bei denen der Wert von *key* gleich *value* ist.

Höchstens ein Befehl **-filter** darf angegeben werden, und wenn er vorhanden ist, wird er *zuletzt* angewandt. (Werden zwei oder mehr Optionen **-filter** verwendet, wird ein Fehler ausgelöst.)

-filter *cmdprefix*
: Gibt nur Bögen zurück, für die das angegebene *cmdprefix* **1** (wahr) zurückgibt.

  Für jeden in Frage kommenden Bogen wird *cmdprefix* im Kontext des Aufrufers ausgeführt. Es wird mit zwei Argumenten aufgerufen: dem Namen des Graphobjekts und dem Namen des in Frage kommenden Bogens. Der Befehl *cmdprefix* soll entweder **1** (wahr; den Bogen in der zurückzugebenden Liste behalten) oder **0** (falsch; den Bogen aus der zurückzugebenden Liste entfernen) zurückgeben.

Höchstens eine Einschränkung darf mit einem von **-in**, **-out**, **-adj**, **-inner** oder **-embedding** angegeben werden. Ist eine davon vorhanden, wird sie *zuerst* angewandt. (Werden zwei oder mehr dieser Optionen verwendet, wird ein Fehler ausgelöst.)

-in
: Gibt nur Bögen zurück, deren Ziel einer der angegebenen Knoten ist. Mit anderen Worten: die Vereinigung aller eingehenden Bögen der angegebenen Knoten.

-out
: Gibt nur Bögen zurück, deren Quelle einer der angegebenen Knoten ist. Mit anderen Worten: die Vereinigung aller ausgehenden Bögen der angegebenen Knoten.

-adj
: Gibt nur Bögen zurück, die zu mindestens einem der angegebenen Knoten benachbart sind. Mit anderen Worten: die Vereinigung aller eingehenden *und* ausgehenden Bögen der angegebenen Knoten (also die Vereinigung von **-in** und **-out**).

-inner
: Gibt nur Bögen zurück, die zu zwei der angegebenen Knoten benachbart sind. Das ist die Menge der Bögen im Teilgraphen, den die angegebenen Knoten bilden.

-embedding
: Gibt nur Bögen zurück, die zu genau einem der angegebenen Knoten benachbart sind. Das ist die Menge der Bögen, die den Teilgraphen der angegebenen Knoten mit dem Rest des Graphen verbinden.

Hinweis: Nach der Angabe einer der obigen Optionen wird jedes Wort mit führendem Bindestrich, das keine gültige Option ist, als Knotenname behandelt und nicht als ungültige Option. Das gilt, bis entweder eine gültige Option die Knotenliste beendet oder das Ende des Befehls erreicht ist, je nachdem, was zuerst eintritt.

aGraph* lappend *key value
Hängt *value* (als Liste) an das Graphattribut zum angegebenen *key* an und gibt das Attribut mit dem neuen Wert von *key* zurück.

aGraph* node append *node key value
Hängt *value* an das Attribut von *node* zum angegebenen *key* an und gibt das Attribut mit dem neuen Wert von *key* zurück.

aGraph* node attr *key
siehe unten ...

aGraph* node attr *key* -nodes *list
siehe unten ...

aGraph* node attr *key* -glob *globpattern
siehe unten ...

aGraph* node attr *key* -regexp *repattern
Gibt ein **dict** zurück, dessen Schlüssel Knotennamen sind und dessen zugehörige Werte die Attributwerte dieser Knoten zum angegebenen *key* sind.

Das zurückgegebene **dict** enthält alle Knoten des Graphen, die den angegebenen *key* haben, *oder*, wenn eine optionale Einschränkung angegeben ist, die Teilmenge dieser Knoten, die der Einschränkung entsprechen. Knoten, die den angegebenen *key nicht* haben oder die Einschränkung *nicht* erfüllen (falls angegeben), fehlen im zurückgegebenen **dict**.

Die möglichen Einschränkungen sind:

-nodes
Nur die Knoten des Graphen, die in der angegebenen *list* stehen und den angegebenen *key* haben, werden in das Ergebnis-**dict** aufgenommen.

-glob
Nur die Knoten des Graphen, deren Namen zum angegebenen Glob-Muster *globpattern* passen und den angegebenen *key* haben, werden in das Ergebnis-**dict** aufgenommen.

-regexp
Nur die Knoten des Graphen, deren Namen zum angegebenen regulären Ausdruck *repattern* passen und den angegebenen *key* haben, werden in das Ergebnis-**dict** aufgenommen.

aGraph* node degree ?-in|-out? *node
Gibt die Zahl der zum angegebenen *node* benachbarten Bögen zurück.

Ist die Einschränkung **-in** angegeben, werden nur die eingehenden Bögen gezählt; ist **-out** angegeben, nur die ausgehenden. Ist keine Einschränkung angegeben, werden alle eingehenden und ausgehenden Bögen gezählt.

***aGraph* node delete *node* ?*node*...?**
Löscht die angegebenen Knoten aus dem Graphen *und außerdem* alle Bögen dieser Knoten (damit keine unverbundenen Bögen bleiben).

aGraph* node exists *node
Gibt **1** (wahr) zurück, wenn der angegebene *node* im Graphen vorhanden ist, sonst **0** (falsch).

aGraph* node get *node key
Gibt den *node* zugeordneten Wert zu *key* zurück.

Löst einen Fehler aus, wenn *node* den angegebenen *key* nicht hat.

***aGraph* node getall *node* ?*glob*?**
Gibt ein **dict** (geeignet für [**array set**]) für *node* zurück. Die Schlüssel des **dict** sind Attributnamen (also ihre Schlüssel), die Werte die zugehörigen Attributwerte.

Ist *glob* angegeben, werden nur die Attribute in das zurückgegebene Dictionary aufgenommen, deren Namen (also Schlüssel) zu diesem Glob-Muster passen.

***aGraph* node keys *node* ?*glob*?**
Gibt eine Liste der Attributschlüssel von *node* zurück.

Ist *glob* angegeben, werden nur die Attribute in die zurückgegebene Liste aufgenommen, deren Namen (also Schlüssel) zu diesem Glob-Muster passen.

aGraph* node keyexists *node key
Gibt **1** (wahr) zurück, wenn *node* den angegebenen *key* hat, sonst **0** (falsch).

***aGraph* node insert ?*node*...?**
Fügt einen oder mehrere Knoten in den Graphen ein und gibt eine Liste der eingefügten Knoten zurück. Diese neuen Knoten sind mit keinem Bogen verbunden.

Ist kein Knoten angegeben, wird ein einzelner neuer Knoten eingefügt und zurückgegeben. Der neue Knoten erhält einen eindeutigen, vom System erzeugten Namen der Form *nodex*.

aGraph* node lappend *node key value
Hängt *value* (als Liste) an das Attribut von *node* zum angegebenen *key* an und gibt das Attribut mit dem neuen Wert von *key* zurück.

aGraph* node opposite *node arc
Gibt den Knoten am anderen Ende des angegebenen *arc* zurück. Dieser Bogen muss zum angegebenen *node* benachbart sein.

aGraph* node rename *node newname
Benennt *node* in *newname* um und gibt den neuen Namen des Knotens zurück.

Ein Fehler wird ausgelöst, wenn der Knoten nicht existiert oder bereits ein Knoten namens *newname* existiert.

***aGraph* node set *node key* ?*value*?**
Liest oder setzt das Attribut von *node*, das der angegebene *key* bezeichnet.

Ist *value* nicht angegeben, gibt dieser Befehl den aktuellen Wert von *key* zurück. Ist *value* angegeben, weist dieser Befehl *value* dem *key* von *node* zu.

Jedem Knoten können beliebig viele Schlüsselwerte zugeordnet sein.

aGraph* node unset *node key
Entfernt *key* von *node* und den zugehörigen Wert aus den Attributen von *node*. Es geschieht

nichts, wenn *node* den angegebenen *key* nicht hat.

***aGraph* nodes ?-key *key*? ?-value *value*? ?-filter *cmdprefix*? ?-in|-out|-adj|-inner|-embedding *node node*...?**

Gibt eine Liste der Knoten des Graphen zurück. Jeder Knoten steht in der Liste, solange keine Einschränkungen angegeben sind.

Einschränkungen können die Knoten in der Liste anhand der dem Knoten zugeordneten Schlüsselwerte oder anhand benachbarter Knoten begrenzen. Einschränkungen, die benachbarte Knoten betreffen, nehmen eine Liste von Knoten als Argument, angegeben nach dem Namen der Einschränkung selbst.

Die Einschränkungen entsprechen im Wesentlichen denen der Methode **arcs**. Die genaue Bedeutung ändert sich etwas, weil die Einschränkungen hier auf Knoten statt auf Bögen wirken; das allgemeine Verhalten bleibt dasselbe, besonders bei Wörtern mit führendem Bindestrich in Knotenlisten.

Einschränkungen

-in
Gibt die *eingehenden Nachbarn* zurück, also eine Liste aller Knoten mit mindestens einem ausgehenden Bogen, der in einem Knoten der angegebenen Knotenmenge endet. (Anders ausgedrückt: die Menge der Quellknoten der **-in**-Bögen dieser Knotenmenge.)

-out
Gibt die *ausgehenden Nachbarn* zurück, also eine Liste aller Knoten mit mindestens einem eingehenden Bogen, der in einem Knoten der angegebenen Knotenmenge beginnt. (Anders ausgedrückt: die Menge der Zielknoten der **-out**-Bögen dieser Knotenmenge.)

-adj
Gibt die *Nachbarn* zurück, also die Vereinigung der von **-in** und **-out** zurückgegebenen Knoten.

-inner
Gibt die Menge der Nachbarn zurück (siehe **-adj** oben), die auch in der angegebenen Knotenmenge liegen, also den Schnitt der von **-adj** zurückgegebenen Nachbarn mit der Knotenmenge.

-embedding
Gibt die Menge der Nachbarn zurück (siehe **-adj** oben), die *nicht* in der Knotenmenge liegen, also die Differenz zwischen den von **-adj** zurückgegebenen Nachbarn und der Knotenmenge.

-key *key*
Gibt nur Knoten zurück, die den angegebenen *key* haben.

-value *value*
Gibt nur Knoten zurück, die den angegebenen *key* haben und deren zugehöriger Wert *value* ist.

Diese Einschränkung ist nur zusammen mit **-key** verwendbar.

-filter *cmdprefix*
Gibt nur die Knoten zurück, für die der Befehl *cmdprefix* **1** (wahr) zurückgibt.

Der Befehl *cmdprefix* wird mit zwei Argumenten aufgerufen, dem Namen des Graphobjekts

und dem Namen des betreffenden Knotens. Er wird im Kontext des Aufrufers ausgeführt und muss einen booleschen Wert zurückgeben, **0** (falsch; den Knoten nicht aufnehmen) oder **1** (wahr; den Knoten aufnehmen).

aGraph* get *key
Gibt den dem *key* des Graphen zugeordneten Wert zurück.

Löst einen Fehler aus, wenn der Graph den angegebenen *key* nicht hat.

Siehe die Methode **set**.

***aGraph* getall ?*pattern*?**
Gibt ein Dictionary (geeignet für [**array set**]) mit allen Schlüsseln des Graphen und ihren zugehörigen Werten zurück.

Ist *pattern* angegeben, gehören nur die Attribute zum zurückgegebenen Dictionary, deren Namen zum Muster passen. Das Muster ist ein **glob**-Muster.

***aGraph* keys ?*pattern*?**
Gibt eine Liste aller Schlüssel des Graphen zurück.

Ist *pattern* angegeben, gehören nur die Attribute zur zurückgegebenen Liste, deren Namen zum Muster passen. Das Muster ist ein **glob**-Muster.

aGraph* keyexists *key
Gibt **1** (wahr) zurück, wenn der Graph den angegebenen *key* hat, sonst **0** (falsch).

***aGraph* serialize ?*node*...?**
Gibt den Graphen (oder einen Teilgraphen) als einzelnen Tcl-Wert serialisiert zurück. Mit der Methode **deserialize** lässt sich der Wert wieder in einen Graphen zurückverwandeln.

Serialisiert wird der gesamte Graph, es sei denn, ein oder mehrere *node*s sind angegeben; dann wird nur der von diesen Knoten aufgespannte Teilgraph serialisiert.

Ein serialisierter Graph ist ein Tcl-Wert und lässt sich wie jede Tcl-Zeichenkette kopieren, zum Beispiel an einen Kanal senden oder in eine Datei schreiben.

Diese Methode ist auch die Grundlage für Kopierkonstruktor und Zuweisungsoperator des Graphen.

Die Implementierung der Methode serialize jeder Graphschnittstelle muss semantisch gleichwertige Serialisierungswerte erzeugen, damit diese Werte zwischen Implementierungen austauschbar sind.

Der serialisierte Wert ist eine Liste von Tripeln plus einem **dict** am Ende. Das heißt *[llength $serial] % 3 == 1*. Gültige Werte sind unter anderem 1, 4, 7, ...

Das letzte Element der Liste ist ein **dict** mit den Attributen des gesamten Graphen.

Alle vorangehenden Elemente sind Tripel, eines je Knoten, jeweils bestehend aus:
1. Der Name des Knotens.
2. Ein **dict** mit den Attributen des Knotens.
3. Eine Liste aller Bögen, die an diesem Knoten beginnen (siehe unten).

Jedes Element der Bogenliste eines Knotens ist selbst eine Liste aus drei oder vier Elementen, und zwar:

1. Der Name des Bogens.
2. Ein Verweis auf den Zielknoten des Bogens. Dieser Verweis ist der ganzzahlige Index des Zielknotens in der Hauptserialisierungsliste. (Diese Ganzzahl ist immer >= *0* und < *[llength $serial]* und ein Vielfaches von drei.)

   Hinweis: Der inneren Konsistenz wegen verlangt die Serialisierung, dass jeder in einem Graphen verwendete Bogenname eindeutig ist, ob am selben Knoten oder an einem anderen.
3. Ein **dict** mit den Attributen des Bogens.
4. Ein optionales Bogengewicht. Fehlt es, hat dieser Bogen kein zugeordnetes Gewicht.

   Hinweis: Diese Angabe ist neu gegenüber der Serialisierung von **graph** 2.3 und früher. Optionale Bogengewichte im neuen Format erhalten einen Teil der Kompatibilität mit dem alten. Insbesondere hat jeder Graph, der *keine* Gewichte verwendet, eine Serialisierung, die das ältere Graphpaket deserialisieren kann. Werden dagegen Gewichte verwendet, können ältere Pakete nicht deserialisieren.

Für alle Attributdictionaries ist jeder Schlüssel ein Attributname, und der dem Schlüssel zugeordnete Wert enthält den Wert des Attributs.

Hinweis: Die Reihenfolge der Knoten in der Serialisierung und der Bögen innerhalb jedes Knotens hat keine Bedeutung. Zwei semantisch gleichwertige Graphen können also verschiedene Serialisierungen haben.

```
# A possible serialization for the graph structure
#
#        d -----> %2
#       /         ^ \
#      /          /   \
#     /          b     \
#    /          /       \
#  %1 <- a - %0          e
#    ^          \\      /
#     \\         c     /
#      \\         \\  /
#       \\         v v
#        f ------ %3
# is
#
# %3 {} {{f 6 {}}} %0 {} {{a 6 {}} {b 9 {}} {c 0 {}}} %1 {} {{d
9 {}}} %2 {} {{e 0 {}}} {}
#
# This assumes that the graph has neither attribute data nor
weighted arcs.
```

***aGraph* set *key* ?*value*?**

Liest oder setzt den Wert zum angegebenen Graph-*key*.

Ist kein *value* angegeben, wird der *key* zugeordnete Wert zurückgegeben. Ist *value* angegeben, wird *key* auf den angegebenen *value* gesetzt und *value* zurückgegeben.

Löst einen Fehler aus, wenn kein *value* angegeben ist (also beim Lesen) und der Graph den angegebenen *key* nicht hat.

Einem Graphen können beliebig viele Schlüsselwerte zugeordnet sein. Siehe auch die Methode **get**.

aGraph* swap *node1 node2

Vertauscht die Positionen von *node1* und *node2* im Graphen.

aGraph* unset *key

Entfernt *key* des Graphen und den zugehörigen Wert. Es geschieht nichts, wenn *key* nicht existiert.

aGraph* walk *node* ?-type *type*? ?-order *order*? ?-dir *direction*? -command *cmd

Führt einen Breiten- oder Tiefendurchlauf des Graphen aus, beginnend bei *node* und in Richtung **forward** (Richtung der ausgehenden Bögen) oder **backward** (Gegenrichtung der eingehenden Bögen).

Ist der Typ des Durchlaufs angegeben, muss er **dfs** (Tiefe zuerst; die Vorgabe) oder **bfs** (Breite zuerst) sein.

Ist die Ordnung angegeben, muss sie **pre** (Vorordnung; die Vorgabe), **post** (Nachordnung) oder **both** sein.

Vorordnung heißt, ein Knoten wird besucht, bevor einer seiner Nachbarn besucht wird (festgelegt durch *direction*, siehe unten). Nachordnung heißt, ein Knoten wird besucht, nachdem seine Nachbarn besucht wurden. Bei beiden Ordnungen wird ein Knoten vor *und* nach seinen Nachbarn besucht. Der Breitendurchlauf lässt sich *nur* mit der Vorordnung kombinieren; nur der Tiefendurchlauf ist mit Vorordnung, Nachordnung oder beiden Ordnungen verwendbar.

Ist die Richtung angegeben, muss sie **backward** (Gegenrichtung der eingehenden Bögen) oder **forward** (Richtung der ausgehenden Bögen) sein.

Im Verlauf des Durchlaufs wird der Befehl *cmd* an jedem Knoten ausgewertet und erhält drei Argumente. Die Argumente sind der Modus des Aufrufs (**enter** oder **leave**), der Graph selbst (also *aGraph*) und der Name des aktuellen Knotens. Bei einem Durchlauf in Vorordnung werden alle Knoten betreten (Modus **enter** wird an *cmd* übergeben); bei einem Durchlauf in Nachordnung werden alle Knoten verlassen (Modus **leave** wird übergeben). Bei beiden Ordnungen ist der erste Besuch eines Knotens der Eintritt (Modus **enter**) und der zweite Besuch der Austritt (Modus **leave**).

Änderungen für v2

Änderungen für v2.4

Das Serialisierungsformat der Methode **serialize** hat sich geändert, um die (optionale) Verwendung von Gewichten abzubilden. Einzelheiten stehen in der Beschreibung der Methode.

Änderungen für v2.x

Die folgenden nennenswerten Änderungen sind eingetreten:

1. Die API für den Zugriff auf Attribute und ihre Werte ist vereinfacht.

   Alle Funktionen zum Vorgabeattribut "data" sind entfernt. Dieses Vorgabeattribut gibt es nicht mehr. Jeder Zugriff auf Attribute muss den Namen des Attributs angeben. Diese rückwärts *inkompatible* Änderung hat die Signaturen aller Methoden mit Attributen vereinfacht.

   Insbesondere ist die Option **-key** entfallen. Die Beschreibung der neuen APIs steht bei den Bogen- und Knotenmethoden **get**, **getall**, **set**, **unset**, **append**, **lappend**, **keyexists** und **keys**.
2. Die Methoden **keys** und **getall** nehmen jetzt ein optionales Glob-Muster als Argument. Ist es angegeben, beschränken sie die zurückgegebenen Attributdaten auf die Schlüssel, die zum Muster passen.
3. Bögen und Knoten lassen sich jetzt umbenennen. Siehe die Beschreibung der Methoden **arc rename** und **node rename**.
4. Die Struktur ist um APIs zur Serialisierung und Deserialisierung von Graphen erweitert und um eine Reihe von Operationen auf diesen Methoden (zum Beispiel Graphzuweisung, Kopierkonstruktion).

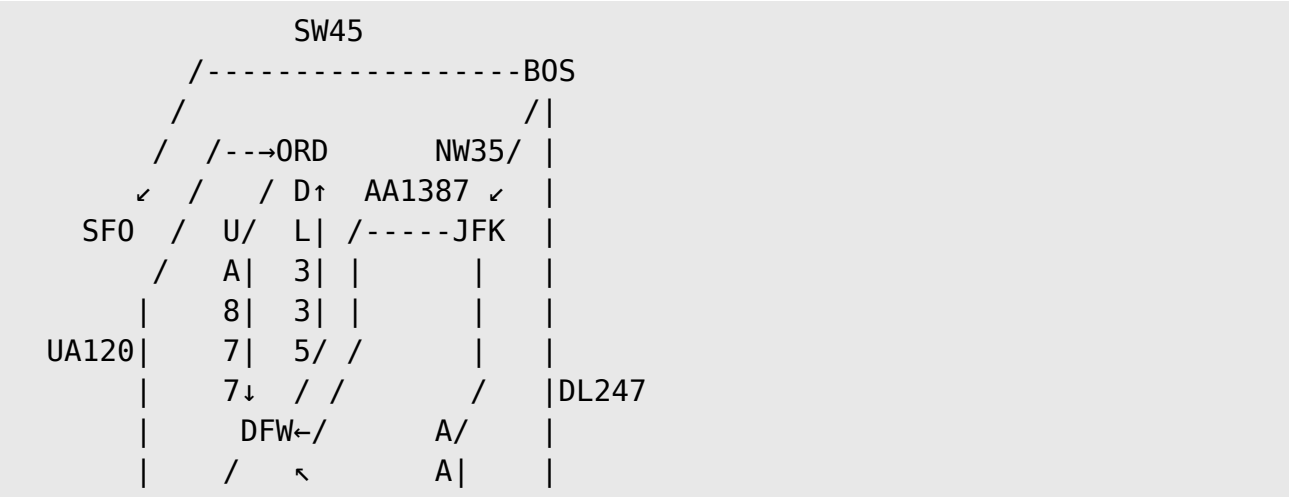
   Siehe die Beschreibung der Methoden **serialize**, **deserialize**, **=** und **-->** sowie die Beschreibung der Erzeugung von Graphobjekten.

   Serialisierung und Deserialisierung bedeuten, dass sich Graphen wie jede Tcl-Zeichenkette kopieren lassen, zum Beispiel an einen Kanal senden oder in eine Datei schreiben.
5. Sowohl **arc** als auch **node** haben jetzt eine Methode **attr**, die auf Attributdaten zugreift, unabhängig von den Beziehungen zwischen Bögen und Knoten.
6. Sowohl **arcs** als auch **nodes** können jetzt Einschränkungen angeben, die die zurückgegebenen Bögen oder Knoten filtern; Einzelheiten stehen in ihrer Beschreibung.

Beispiele

Das folgende Beispiel zeigt die Grundzüge der Verwendung des Pakets **struct::graph**.

Eine Abbildung des im Beispiel verwendeten Graphen:



```
       |   /AA49 \      9|   /
      LAX←/       \     0|  /
        ↖     AA523\    3↓ ↙
         \          \--MIA
          \-----------/
            AA411
```

Der Code, der den oben gezeigten Graphen darstellt, zusammen mit Gewichten (Flugstrecken in Kilometern).

```
proc visit_airport {mode graph airport} {
    puts " [expr {$mode eq "enter" ? "→" : "←"}] $graph $airport"
}

try { ;# Adapted from "Data Structures and Algorithms in Java"
    ::struct::graph flights
    flights node insert SFO
    flights node insert LAX
    flights node insert ORD
    flights node insert DFW
    flights node insert JFK
    flights node insert BOS
    flights node insert MIA
    flights arc insert JFK SFO SW45
    flights arc setweight SW45 4152
    flights arc insert JFK DFW AA1387
    flights arc setweight AA1387 2235
    flights arc insert JFK MIA AA903
    flights arc setweight AA903 1756
    flights arc insert BOS JFK NW35
    flights arc setweight NW35 300
    flights arc insert BOS MIA DL247
    flights arc setweight DL247 2027
    flights arc insert MIA DFW AA523
    flights arc setweight AA523 1802
    flights arc insert MIA LAX AA411
    flights arc setweight AA411 3763
    flights arc insert LAX ORD UA120
    flights arc setweight UA120 2802
    flights arc insert DFW LAX AA49
    flights arc setweight AA49 1983
    flights arc insert DFW ORD DL335
    flights arc setweight DL335 1290
    flights arc insert ORD DFW UA877
    flights arc setweight UA877 1290
    puts "flights walk ORD:"
    puts [flights walk ORD -order both -command visit_airport]
```

```
    puts "All flights: [lsort -dictionary [flights arcs]]"
    puts "Chicago flights: [flights arcs -embedding ORD]"
    puts "Chicago direct connections: [flights nodes -embedding ORD]"
} finally {
    flights destroy
}
=>
flights walk ORD:
 → flights ORD
 → flights DFW
 → flights LAX
 ← flights LAX
 ← flights DFW
 ← flights ORD

All flights: AA49 AA411 AA523 AA903 AA1387 DL247 DL335 NW35 SW45 UA120
    UA877
Chicago flights: DL335 UA120 UA877
Chicago direct connections: DFW LAX
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: graph* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright

```
Copyright (c) 2002-2009,2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

graph1

Stichwörter: graph1, cgraph, graph

Name

struct::graph_v1 – Gerichtete Graphobjekte erzeugen und bearbeiten

Übersicht

package require Tcl 8.5 9
package require struct::graph ?**1.2.2**?

graphName *option* ?*arg arg* ...?
graphName **destroy**
graphName **arc append** *arc* ?*-key key*? *value*
graphName **arc delete** *arc* ?*arc* ...?
graphName **arc exists** *arc*
graphName **arc get** *arc* ?*-key key*?
graphName **arc getall** *arc*
graphName **arc keys** *arc*
graphName **arc keyexists** *arc* ?*-key key*?
graphName **arc insert** *start end* ?*child*?
graphName **arc lappend** *arc* ?*-key key*? *value*
graphName **arc set** *arc* ?*-key key*? ?*value*?
graphName **arc source** *arc*
graphName **arc target** *arc*
graphName **arc unset** *arc* ?*-key key*?
graphName **arcs** ?*-key key*? ?*-value value*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding nodelist*?
graphName **node append** *node* ?*-key key*? *value*
graphName **node degree** ?*-in*|*-out*? *node*
graphName **node delete** *node* ?*node* ...?
graphName **node exists** *node*
graphName **node get** *node* ?*-key key*?
graphName **node getall** *node*
graphName **node keys** *node*
graphName **node keyexists** *node* ?*-key key*?
graphName **node insert** ?*child*?
graphName **node lappend** *node* ?*-key key*? *value*
graphName **node opposite** *node arc*
graphName **node set** *node* ?*-key key*? ?*value*?
graphName **node unset** *node* ?*-key key*?
graphName **nodes** ?*-key key*? ?*-value value*? ?*-in*|*-out*|*-adj*|*-inner*|*-embedding nodelist*?
graphName **get** ?*-key key*?
graphName **getall**
graphName **keys**
graphName **keyexists** ?*-key key*?
graphName **set** ?*-key key*? ?*value*?
graphName **swap** *node1 node2*
graphName **unset** ?*-key key*?
graphName **walk** *node* ?*-order order*? ?*-type type*? ?*-dir direction*? -command *cmd*

Beschreibung

Dieses Paket ist abgelöst durch **struct::graph** *Version 2.*

Der Befehl ::struct::graph erzeugt ein neues Graphobjekt mit einem zugehörigen globalen Tcl-Befehl namens *graphName*. Mit diesem Befehl lassen sich verschiedene Operationen auf dem Graphen ausführen. Er hat die folgende allgemeine Form:

graphName *option* ?*arg arg ...*?
Option und die *args* legen das genaue Verhalten des Befehls fest.

Ein gerichteter Graph ist eine Struktur mit zwei Mengen von Elementen, *Knoten* und *Bögen*, zusammen mit einer Beziehung ("Verbundenheit"), die Knoten und Bögen eine allgemeine Struktur gibt.

Jeder Bogen verbindet zwei Knoten. Einer heißt *Quelle*, der andere *Ziel*. Das gibt dem Bogen eine Richtung: Er führt von der Quelle zum Ziel. Quelle und Ziel eines Bogens dürfen derselbe Knoten sein. Ein solcher Bogen heißt *Schleife*. Ist ein Knoten Quelle oder Ziel eines Bogens, heißen beide *benachbart*. Das setzt sich in eine Beziehung zwischen Knoten fort: Sind zwei Knoten über mindestens einen Bogen verbunden, heißen auch sie *benachbart*.

Jeder Knoten kann Quelle und Ziel beliebig vieler Bögen sein. Die ersten heißen die *ausgehenden Bögen* des Knotens, die zweiten die *eingehenden Bögen* des Knotens. Die Zahl der Kanten in der jeweiligen Menge heißt der *Eingangs-* beziehungsweise der *Ausgangsgrad* des Knotens.

Neben den Beziehungen zwischen Knoten und Bögen erlaubt diese Graphimplementierung, jedem Knoten und jedem Bogen beliebig viele Schlüsselwerte zuzuordnen.

Für Graphobjekte sind die folgenden Befehle möglich:

***graphName* destroy**
Zerstört den Graphen, einschließlich seines Speichers und des zugehörigen Befehls.

graphName* arc append *arc* ?-key *key*? *value
Hängt einen *value* an einen der *arc* zugeordneten Schlüsselwerte an. Ist kein *key* angegeben, gilt der Schlüssel **data**.

***graphName* arc delete *arc* ?*arc* ...?**
Entfernt die angegebenen Bögen aus dem Graphen.

graphName* arc exists *arc
Gibt wahr zurück, wenn der angegebene *arc* im Graphen vorhanden ist.

***graphName* arc get *arc* ?-key *key*?**
Gibt den Wert zum Schlüssel *key* für *arc* zurück. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**.

graphName* arc getall *arc
Gibt eine serialisierte Liste von Schlüssel-Wert-Paaren (geeignet für [**array set**]) für *arc* zurück.

graphName* arc keys *arc
Gibt eine Liste der Schlüssel von *arc* zurück.

***graphName* arc keyexists *arc* ?-key *key*?**
Gibt wahr zurück, wenn der angegebene *key* für *arc* existiert. Ist kein *key* angegeben, gilt der Schlüssel **data**.

***graphName* arc insert *start end* ?*child*?**
Fügt einen Bogen namens *child* in den Graphen ein, der am Knoten *start* beginnt und am Knoten *end* endet. Ist der Name des neuen Bogens nicht angegeben, erzeugt das System einen eindeutigen Namen der Form *arcx*.

graphName* arc lappend *arc* ?-key *key*? *value
Hängt einen *value* (als Liste) an einen der *arc* zugeordneten Schlüsselwerte an. Ist kein *key* angegeben, gilt der Schlüssel **data**.

***graphName* arc set *arc* ?-key *key*? ?*value*?**
Setzt oder liest einen der einem Bogen zugeordneten Schlüsselwerte. Ist kein Schlüssel

angegeben, gilt der Schlüssel **data**. Jedem Bogen, der einem Graphen hinzugefügt wird, wird für den Schlüssel **data** automatisch die leere Zeichenkette zugewiesen. Einem Bogen können beliebig viele Schlüsselwerte zugeordnet sein. Ist *value* nicht angegeben, gibt dieser Befehl den aktuellen Wert des Schlüssels zurück; ist *value* angegeben, weist dieser Befehl diesen Wert dem Schlüssel zu.

graphName* arc source *arc
Gibt den Knoten zurück, an dem der angegebene *arc* beginnt.

graphName* arc target *arc
Gibt den Knoten zurück, an dem der angegebene *arc* endet.

***graphName* arc unset *arc* ?-key *key*?**
Entfernt einen Schlüsselwert vom Bogen *arc*. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**.

***graphName* arcs ?-key *key*? ?-value *value*? ?-in|-out|-adj|-inner|-embedding *nodelist*?**
Gibt eine Liste der Bögen im Graphen zurück. Ist keine Einschränkung angegeben, enthält die Liste alle Bögen. Einschränkungen können die Liste der zurückgegebenen Bögen anhand der Knoten begrenzen, die der Bogen verbindet, anhand der dem Bogen zugeordneten Schlüsselwerte oder anhand von beidem. Einschränkungen, die verbundene Knoten betreffen, haben eine Liste von Knoten als Argument, angegeben nach dem Namen der Einschränkung selbst.

-in
Gibt eine Liste aller Bögen zurück, deren Ziel einer der Knoten in *nodelist* ist.

-out
Gibt eine Liste aller Bögen zurück, deren Quelle einer der Knoten in *nodelist* ist.

-adj
Gibt eine Liste aller Bögen zurück, die zu mindestens einem der Knoten in *nodelist* benachbart sind. Das ist die Vereinigung der von **-in** und **-out** zurückgegebenen Knoten.

-inner
Gibt eine Liste aller Bögen zurück, die zu zwei der Knoten in *nodelist* benachbart sind. Das ist die Menge der Bögen im Teilgraphen, den die angegebenen Knoten aufspannen.

-embedding
Gibt eine Liste aller Bögen zurück, die zu genau einem der Knoten in *nodelist* benachbart sind. Das ist die Menge der Bögen, die den von den angegebenen Knoten aufgespannten Teilgraphen mit dem Rest des Graphen verbinden.

-key *key*
Beschränkt die Liste der zurückgegebenen Bögen auf die Bögen, die einen zugeordneten Schlüssel *key* haben.

-value *value*
Diese Einschränkung ist nur zusammen mit **-key** verwendbar. Sie beschränkt die Liste der zurückgegebenen Bögen auf die Bögen, deren zugehöriger Schlüssel *key* den Wert *value* hat.

Die Einschränkungen durch **-in**, **-out**, **-adj**, **-inner** oder **-embedded** werden zuerst angewandt. Mehr als eine davon anzugeben, ist unzulässig. Zuletzt werden die Einschränkungen über **-key** (und **-value**) angewandt. Mehr als einen **-key** (und **-value**) anzugeben, ist unzulässig.

graphName* node append *node* ?-key *key*? *value
Hängt einen *value* an einen der *node* zugeordneten Schlüsselwerte an. Ist kein *key* angegeben, gilt der Schlüssel **data**.

graphName* node degree ?-in|-out? *node
Gibt die Zahl der zum angegebenen *node* benachbarten Bögen zurück. Ist eine der Einschränkungen **-in** oder **-out** angegeben, werden nur die eingehenden beziehungsweise ausgehenden Bögen gezählt.

***graphName* node delete *node* ?*node* ...?**
Entfernt die angegebenen Knoten aus dem Graphen. Alle Bögen dieser Knoten werden ebenfalls entfernt, damit keine unverbundenen Bögen bleiben.

graphName* node exists *node
Gibt wahr zurück, wenn der angegebene *node* im Graphen vorhanden ist.

***graphName* node get *node* ?-key *key*?**
Gibt den Wert zum Schlüssel *key* für *node* zurück. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**.

graphName* node getall *node
Gibt eine serialisierte Liste von Schlüssel-Wert-Paaren (geeignet für [**array set**]) für *node* zurück.

graphName* node keys *node
Gibt eine Liste der Schlüssel von *node* zurück.

***graphName* node keyexists *node* ?-key *key*?**
Gibt wahr zurück, wenn der angegebene *key* für *node* existiert. Ist kein *key* angegeben, gilt der Schlüssel **data**.

***graphName* node insert ?*child*?**
Fügt einen Knoten namens *child* in den Graphen ein. Der Knoten hat keine Bögen. Ist der Name des neuen Knotens nicht angegeben, erzeugt das System einen eindeutigen Namen der Form *nodex*.

graphName* node lappend *node* ?-key *key*? *value
Hängt einen *value* (als Liste) an einen der *node* zugeordneten Schlüsselwerte an. Ist kein *key* angegeben, gilt der Schlüssel **data**.

graphName* node opposite *node arc
Gibt den Knoten am anderen Ende des angegebenen *arc* zurück. Dieser Bogen muss zum angegebenen *node* benachbart sein.

***graphName* node set *node* ?-key *key*? ?*value*?**
Setzt oder liest einen der einem Knoten zugeordneten Schlüsselwerte. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**. Jedem Knoten, der einem Graphen hinzugefügt wird, wird für den Schlüssel **data** automatisch die leere Zeichenkette zugewiesen. Ein Knoten kann beliebig viele zugeordnete Schlüsselwerte haben. Ist *value* nicht angegeben, gibt dieser Befehl den aktuellen Wert des Schlüssels zurück; ist *value* angegeben, weist dieser Befehl diesen Wert dem Schlüssel zu.

***graphName* node unset *node* ?-key *key*?**
Entfernt einen Schlüsselwert vom Knoten *node*. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**.

***graphName* nodes ?-key *key*? ?-value *value*? ?-in|-out|-adj|-inner|-embedding *nodelist*?**

Gibt eine Liste der Knoten im Graphen zurück. Einschränkungen können die Liste der zurückgegebenen Knoten anhand benachbarter Knoten oder anhand der dem Knoten zugeordneten Schlüsselwerte begrenzen. Einschränkungen, die benachbarte Knoten betreffen, haben eine Liste von Knoten als Argument, angegeben nach dem Namen der Einschränkung selbst.

Die möglichen Einschränkungen sind dieselben wie bei der Methode **arcs**. Die zurückzugebende Knotenmenge wird als Vereinigung aller Quell- und Zielknoten aller Bögen berechnet, die die für **arcs** festgelegte Einschränkung erfüllen.

***graphName* get ?-key *key*?**
Gibt den Wert zum Schlüssel *key* für den Graphen zurück. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**.

***graphName* getall**
Gibt eine serialisierte Liste von Schlüssel-Wert-Paaren (geeignet für [**array set**]) für den gesamten Graphen zurück.

***graphName* keys**
Gibt eine Liste der Schlüssel für den gesamten Graphen zurück.

***graphName* keyexists ?-key *key*?**
Gibt wahr zurück, wenn der angegebene *key* für den gesamten Graphen existiert. Ist kein *key* angegeben, gilt der Schlüssel **data**.

***graphName* set ?-key *key*? ?*value*?**
Setzt oder liest einen der einem Graphen zugeordneten Schlüsselwerte. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**. Jedem Graphen wird für den Schlüssel **data** automatisch die leere Zeichenkette zugewiesen. Ein Graph kann beliebig viele zugeordnete Schlüsselwerte haben. Ist *value* nicht angegeben, gibt dieser Befehl den aktuellen Wert des Schlüssels zurück; ist *value* angegeben, weist dieser Befehl diesen Wert dem Schlüssel zu.

graphName* swap *node1 node2
Vertauscht die Position von *node1* und *node2* im Graphen.

***graphName* unset ?-key *key*?**
Entfernt einen Schlüsselwert vom Graphen. Ist kein Schlüssel angegeben, gilt der Schlüssel **data**.

graphName* walk *node* ?-order *order*? ?-type *type*? ?-dir *direction*? -command *cmd
Führt einen Breiten- oder Tiefendurchlauf des Graphen aus, beginnend beim Knoten *node*, entweder in Richtung der ausgehenden Bögen oder entgegengesetzt zu den eingehenden Bögen.

Den Typ des Durchlaufs, Breite zuerst oder Tiefe zuerst, legt der Wert von *type* fest; **bfs** bedeutet Breite zuerst, **dfs** bedeutet Tiefe zuerst. Tiefe zuerst ist die Vorgabe.

Die Ordnung des Durchlaufs, Vorordnung, Nachordnung oder beide Ordnungen, legt der Wert von *order* fest; **pre** bedeutet Vorordnung, **post** bedeutet Nachordnung, **both** bedeutet beide Ordnungen. Vorordnung ist die Vorgabe. Vorordnung heißt, ein Knoten wird besucht, bevor einer seiner Nachbarn besucht wird (festgelegt durch *direction*, siehe unten). Nachordnung heißt, ein Elternknoten wird besucht, nachdem seine Nachbarn besucht wurden. Bei beiden Ordnungen wird ein Knoten vor *und* nach seinen Nachbarn besucht. Die Kombination eines

Breitendurchlaufs mit Nachordnung oder beiden Ordnungen ist unzulässig.

Die Richtung des Durchlaufs legt der Wert von *dir* fest; **backward** bedeutet die Gegenrichtung der eingehenden Bögen, **forward** die Richtung der ausgehenden Bögen.

Im Verlauf des Durchlaufs wird der Befehl *cmd* an jedem Knoten ausgewertet, und der Modus des Aufrufs (**enter** oder **leave**) sowie die Werte *graphName* und der Name des aktuellen Knotens werden angehängt. Bei einem Durchlauf in Vorordnung werden alle Knoten betreten (**enter**), bei einem Durchlauf in Nachordnung werden alle Knoten verlassen. Bei beiden Ordnungen wird der Knoten beim ersten Besuch betreten (**enter**) und beim zweiten Besuch verlassen (**leave**).

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: graph* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright


```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


graphops

Stichwörter: graphops, adjacency list, adjacency matrix, adjacent, approximation algorithm, arc, articulation point, augmenting network, augmenting path, bfs, bipartite, blocking flow, bridge, complete graph, connected component, cut edge, cut vertex, degree, degree constrained spanning tree, diameter, dijkstra, distance, eccentricity, edge, flow network, graph, heuristic, independent set, isthmus, level graph, local searching, loop, matching, max cut, maximum flow, minimal spanning tree, minimum cost flow, minimum degree spanning tree, minimum diameter spanning tree, neighbour, node, radius, residual graph, shortest path, squared graph, strongly connected component, subgraph, travelling salesman, vertex, vertex cover

Name

struct::graph::op – Operationen für gerichtete und ungerichtete Graphobjekte

Übersicht

package require Tcl 8.6 9
package require struct::graph::op ?**0.11.4**?
struct::graph::op::toAdjacencyMatrix *g*
struct::graph::op::toAdjacencyList *g* ?*options...*?

struct::graph::op::kruskal *g*
struct::graph::op::prim *g*
struct::graph::op::isBipartite? *g* ?*partitions*?
struct::graph::op::tarjan *g*
struct::graph::op::connectedComponents *g*
struct::graph::op::connectedComponentOf *g node*
struct::graph::op::isConnected? *g*
struct::graph::op::isCutVertex? *g node*
struct::graph::op::isBridge? *g arc*
struct::graph::op::isEulerian? *g* ?*tourvar*?
struct::graph::op::isSemiEulerian? *g* ?*pathvar*?
struct::graph::op::dijkstra *g start_node* ?*options...*?
struct::graph::op::distance *g start_node end_node* ?*-arcmode mode*?
struct::graph::op::eccentricity *g node* ?*-arcmode mode*?
struct::graph::op::radius *g* ?*-arcmode mode*?
struct::graph::op::diameter *g* ?*-arcmode mode*?
struct::graph::op::BellmanFord *g startnode*
struct::graph::op::Johnsons *g* ?*options...*?
struct::graph::op::FloydWarshall *g*
struct::graph::op::MetricTravellingSalesman *g*
struct::graph::op::Christofides *g*
struct::graph::op::GreedyMaxMatching *g*
struct::graph::op::MaxCut *g U V*
struct::graph::op::UnweightedKCenter *g k*
struct::graph::op::WeightedKCenter *g nodeWeights W*
struct::graph::op::GreedyMaxIndependentSet *g*
struct::graph::op::GreedyWeightedMaxIndependentSet *g nodeWeights*
struct::graph::op::VerticesCover *g*
struct::graph::op::EdmondsKarp *g s t*
struct::graph::op::BusackerGowen *g desiredFlow s t*
struct::graph::op::ShortestsPathsByBFS *g s outputFormat*
struct::graph::op::BFS *g s* ?*outputFormat...*?
struct::graph::op::MinimumDiameterSpanningTree *g*
struct::graph::op::MinimumDegreeSpanningTree *g*
struct::graph::op::MaximumFlowByDinic *g s t blockingFlowAlg*
struct::graph::op::BlockingFlowByDinic *g s t*
struct::graph::op::BlockingFlowByMKM *g s t*
struct::graph::op::createResidualGraph *g f*
struct::graph::op::createAugmentingNetwork *g f path*
struct::graph::op::createLevelGraph *Gf s*
struct::graph::op::TSPLocalSearching *g C*
struct::graph::op::TSPLocalSearching3Approx *g C*
struct::graph::op::createSquaredGraph *g*
struct::graph::op::createCompleteGraph *g originalEdges*

Beschreibung

Dieses Paket, **struct::graph::op**, ergänzt das Paket **struct::graph**. Es stellt eine Reihe üblicher Operationen und Algorithmen für gerichtete oder ungerichtete Graphen bereit.

Die Operationen dieses Pakets lassen sich auf *jedes* Graphobjekt anwenden, das dieselbe API bereitstellt wie die von **struct::graph** erzeugten Objekte. Dieses Paket hängt daher nicht unmittelbar von **struct::graph** ab.

Operationen

struct::graph::op::toAdjacencyMatrix *g*

Gibt die Adjazenzmatrix des Graphen *g* zurück.

Die Elemente der äußeren Liste sind die Zeilen der Matrix, die inneren Elemente die Spaltenwerte jeder Zeile. Die Matrix hat **n**+1 Zeilen und Spalten. Die erste Zeile und die erste Spalte (Index 0) enthalten den Namen des Knotens, zu dem die Zeile beziehungsweise Spalte gehört. Alle übrigen Elemente sind boolesche Werte, **1** (wahr), wenn zwischen den beiden Knoten der jeweiligen Zeile und Spalte ein Bogen besteht, sonst **0** (falsch).

Die Matrix ist symmetrisch. Sie bildet die Richtung der Bögen *nicht* ab, nur ihr Vorhandensein oder Fehlen zwischen Knoten. Parallele Bögen in *g* kann sie ebenfalls nicht darstellen.

struct::graph::op::toAdjacencyList *g* ?*options*...?

Gibt eine *Adjazenzliste* des Graphen *g* zurück. Jedes Listenelement ist ein **dict**, dessen Schlüssel ein Knoten ist und dessen Wert eine Liste der benachbarten Knoten ist. Mit **-weights** ist jedes Element der Werteliste selbst eine zweielementige Liste aus Knoten und Gewicht.

Argumente:

siehe unten ...

Graphobjekt *g*

Ein Graph, der in eine *Adjazenzliste* umgewandelt wird.

Optionen:

siehe unten ...

-directed

In der Vorgabe wird der Graph *g* wie ein *ungerichteter Graph* behandelt. Mit dieser Option behandelt der Befehl *g* als gerichteten Graphen.

-weights

In der Vorgabe werden die Gewichtsangaben von *g* (falls vorhanden) ignoriert. Mit dieser Option nimmt der Befehl vorhandene Gewichtsangaben in das Ergebnis auf.

Die Option setzt voraus, dass jeder Bogen in *g* ein zugeordnetes Gewicht hat; sonst wird ein Fehler ausgelöst.

struct::graph::op::kruskal *g*

Gibt eine Liste von Bogennamen im Graphen *g* zurück, bestimmt durch den Algorithmus von Kruskal.

In einem zusammenhängenden Graphen sind das die Bögen, die einen minimalen Spannbaum (MST) aufspannen. In einem unzusammenhängenden Graphen sind das die

Bögen eines minimalen Spannwaldes (selbst aus einem MST je zusammenhängender Komponente zusammengesetzt, ohne Komponenten aus einem einzelnen Knoten).

Die Zeitkomplexität des Algorithmus ist *O(E*log E)* oder *O(E*log V)*, wobei *V* die Zahl der Knoten und *E* die Zahl der Kanten in *g* ist. Der Algorithmus von Prim kann andere Ergebnisse liefern, wenn mehr als ein MST möglich ist.

Welche Knoten ein zurückgegebener Bogen verbindet, liefert die Methode **arc nodes** des Graphen für jeden Bogen.

Der Befehl löst einen Fehler aus, wenn irgendein Bogen von *g* kein zugeordnetes Gewicht hat.

struct::graph::op::prim *g*

Gibt eine Liste von Bogennamen im Graphen *g* zurück, bestimmt durch den Algorithmus von Prim.

In einem zusammenhängenden Graphen sind das die Bögen, die einen minimalen Spannbaum (MST) aufspannen. In einem unzusammenhängenden Graphen sind das die Bögen eines minimalen Spannwaldes (selbst aus einem MST je zusammenhängender Komponente zusammengesetzt, ohne Komponenten aus einem einzelnen Knoten).

Die Zeitkomplexität liegt zwischen *O(E+V*log V)* und *O(V*V)*, je nach Implementierung (Fibonacci-Heap und Adjazenzliste gegenüber Adjazenzmatrix), wobei *V* die Zahl der Knoten und *E* die Zahl der Kanten in *g* ist. Der Algorithmus von Kruskal kann andere Ergebnisse liefern, wenn mehr als ein MST möglich ist.

Welche Knoten ein zurückgegebener Bogen verbindet, liefert die Methode **arc nodes** des Graphen für jeden Bogen.

Der Befehl löst einen Fehler aus, wenn irgendein Bogen von *g* kein zugeordnetes Gewicht hat.

struct::graph::op::isBipartite? *g* ?*partitions*?

Gibt **1** (wahr) zurück, wenn der Graph *g* bipartit ist, sonst **0** (falsch).

Ist der optionale Variablenname *partitions* angegeben *und* ist *g* bipartit, wird *partitions* angelegt, falls die Variable nicht existiert, und auf die Liste der beiden Partitionen gesetzt. Ist *g* nicht bipartit, wird *partitions* nicht angelegt, wenn die Variable nicht existiert, und nicht geändert, wenn sie existiert.

struct::graph::op::tarjan *g*

Gibt eine Liste von Mengen zurück. Jede Menge enthält die Knoten einer *stark zusammenhängenden* Komponente (SCC) von *g*. Die Vereinigung aller SCCs überdeckt den gesamten Graphen, und keine zwei SCCs schneiden sich.

Enthalten *alle* entstandenen SCCs nur einen Knoten, ist *g azyklisch*. Enthält das Ergebnis nur eine SCC mit allen Knoten von *g*, ist der Graph *stark zusammenhängend*.

Hinweis: Das Paket **struct::set** muss geladen sein, bevor diese Funktion verwendet wird, denn sie gibt eine Menge des Typs **struct::set** zurück.

struct::graph::op::connectedComponents *g*
Gibt eine Liste von Mengen zurück. Jede Menge enthält die Knoten einer *zusammenhängenden* Komponente (CC) von *g*. Die Vereinigung aller CCs überdeckt den gesamten Graphen, und keine zwei CCs schneiden sich.

Enthält das Ergebnis nur eine CC mit allen Knoten von *g*, ist der Graph *zusammenhängend*.

Hinweis: Das Paket **struct::set** muss geladen sein, bevor diese Funktion verwendet wird, denn sie gibt eine Menge des Typs **struct::set** zurück.

struct::graph::op::connectedComponentOf *g node*
Gibt eine Liste der *zusammenhängenden* Komponenten (CCs) von *g* zurück, die den Knoten *node* enthalten.

Der Befehl löst einen Fehler aus, wenn *node* kein Knoten von *g* ist.

struct::graph::op::isConnected? *g*
Gibt **1** (wahr) zurück, wenn der Graph *g zusammenhängend* ist, sonst **0** (falsch).

struct::graph::op::isCutVertex? *g node*
Gibt **1** (wahr) zurück, wenn *node* von *g* ein *Schnittknoten* ist (*auch Artikulationspunkt genannt*), sonst **0** (falsch).

Der Befehl löst einen Fehler aus, wenn *node* kein Knoten von *g* ist.

struct::graph::op::isBridge? *g arc*
Gibt **1** (wahr) zurück, wenn *arc* von *g* eine *Brücke* ist (*auch Schnittkante oder Isthmus genannt*), sonst **0** (falsch).

Der Befehl löst einen Fehler aus, wenn *arc* kein Bogen von *g* ist.

struct::graph::op::isEulerian? *g* ?*tourvar*?
Gibt **1** (wahr) zurück, wenn der Graph *g eulersch* ist, sonst **0** (falsch).

Ist der Graph eulersch und ist *tourvar* angegeben, wird eine Eulertour berechnet und in der benannten Variablen gespeichert. Die Tour ist eine Liste der durchlaufenen Bögen, in der Reihenfolge des Durchlaufs.

struct::graph::op::isSemiEulerian? *g* ?*pathvar*?
Gibt **1** (wahr) zurück, wenn der Graph *g halb-eulersch* ist, sonst **0** (falsch).

Ist der Graph halb-eulersch und ist *pathvar* angegeben, wird ein Eulerweg berechnet und in der benannten Variablen gespeichert. Der Weg ist eine Liste der durchlaufenen Bögen, in der Reihenfolge des Durchlaufs.

struct::graph::op::dijkstra *g start_node* ?*options*...?
Gibt ein **dict** zurück, dessen Schlüssel Knoten sind und dessen Werte von den Optionen abhängen. In der Vorgabe werden ein ungerichteter Durchlauf und das Baumformat verwendet. Das Ergebnis ist dann ein **dict**, dessen Werte Listen von Knoten sind. Der Durchlauf beruht auf den Abständen im gewichteten Graphen *g* vom *start_node* zu allen anderen Knoten des Graphen.

Zwei Optionen werden unterstützt:

-arcmode mode
mode kann **undirected** (die Vorgabe) oder **directed** sein. Bei **directed** werden alle Bögen von der Quelle zum Ziel durchlaufen. Bei **undirected** werden alle Bögen in beide Richtungen durchlaufen.

-outputformat format
format kann **tree** (die Vorgabe) oder **distances** sein. In beiden Fällen ist das Ergebnis ein **dict**, dessen Schlüssel die Namen aller Knoten von *g* sind. Ist *format* **tree**, ist jeder Wert der Weg vom Knoten zum *start_node* (eine Liste von Knoten, ohne *start_node* selbst). Ist *format* **distances**, ist jeder Wert der Abstand vom Knoten zum *start_node*. *Hinweis:* Mit *format* **distances** sind Abstände **0** oder **Inf** möglich.

struct::graph::op::distance *g start_node end_node* ?-arcmode *mode*?
Gibt den Abstand im Graphen *g* von *start_node* nach *end_node* bei ungerichtetem Durchlauf zurück (also *-arcmode* **undirected**). Für einen gerichteten Durchlauf verwenden Sie *-arcmode* **directed**. (Siehe die Option **-arcmode** von **struct::graph::op::dijkstra**.)

struct::graph::op::eccentricity *g node* ?-arcmode *mode*?
Gibt die *Exzentrizität* des angegebenen *node* im Graphen *g* bei ungerichtetem Durchlauf zurück (also *-arcmode* **undirected**). Für einen gerichteten Durchlauf verwenden Sie *-arcmode* **directed**. (Siehe die Option **-arcmode** von **struct::graph::op::dijkstra**.)

Die (un)gerichtete *Exzentrizität* eines Knotens ist der größte (un)gerichtete Abstand zwischen diesem Knoten und irgendeinem anderen Knoten im Graphen.

struct::graph::op::radius *g* ?-arcmode *mode*?
Gibt den *Radius* des Graphen *g* bei ungerichtetem Durchlauf zurück (also *-arcmode* **undirected**). Für einen gerichteten Durchlauf verwenden Sie *-arcmode* **directed**. (Siehe die Option **-arcmode** von **struct::graph::op::dijkstra**.)

Der (un)gerichtete *Radius* eines Graphen ist die kleinste (un)gerichtete *Exzentrizität* aller Knoten im Graphen.

struct::graph::op::diameter *g* ?-arcmode *mode*?
Gibt den *Durchmesser* des Graphen *g* bei ungerichtetem Durchlauf zurück (also *-arcmode* **undirected**). Für einen gerichteten Durchlauf verwenden Sie *-arcmode* **directed**. (Siehe die Option **-arcmode** von **struct::graph::op::dijkstra**.)

Der (un)gerichtete *Durchmesser* eines Graphen ist die größte (un)gerichtete *Exzentrizität* aller Knoten im Graphen.

struct::graph::op::BellmanFord *g startnode*
Sucht **kürzeste Wege** zwischen einem gewählten Knoten und allen anderen Knoten im Graphen *g*. Grundlage ist das Relaxationsverfahren. Im Vergleich zu **struct::graph::op::dijkstra** setzt der Algorithmus nicht voraus, dass alle Kantengewichte im Eingabegraphen *g* positiv sind.

Diese Allgemeinheit setzt die Komplexität des Algorithmus auf *O(V*E)*, wobei *V* die Zahl der Knoten und *E* die Zahl der Kanten in *g* ist.

Argumente:
siehe unten ...

Graph object *g* (input)
Gerichteter, zusammenhängender und kantengewichteter Graph *g* ohne negative Kreise (Kreise mit negativer Gewichtsumme bedeuten, dass es keinen kürzesten Weg gibt, weil das Gesamtgewicht bei jedem Durchlauf des Kreises sinkt). Negative Kantengewichte sind zulässig.

Node *startnode* (input)
Der Knoten, für den alle kürzesten Wege zu jedem anderen Knoten in *g* gesucht werden.

Ergebnis:
Dictionary, das für jeden Knoten (Schlüssel) die Abstände zu jedem anderen Knoten in *g* enthält.

Hinweis: Findet der Algorithmus einen negativen Kreis, gibt er eine Fehlermeldung zurück.

struct::graph::op::Johnsons *g* ?*options*...?
Sucht **kürzeste Wege** zwischen allen Knotenpaaren im Graphen. Für dünne Graphen asymptotisch schneller als der Algorithmus **struct::graph::op::FloydWarshall**. Der Algorithmus von Johnson verwendet **struct::graph::op::BellmanFord** und **struct::graph::op::dijkstra** als Unterprozeduren.

Zeitkomplexität: *O(n**2*log(n) +n*m)*, wobei *n* die Zahl der Knoten und *m* die Zahl der Kanten in *g* ist.

Argumente:
siehe unten ...

Graph object *g* (input)
Gerichteter Graph *g*, kantengewichtet und ohne Kreise mit negativer Gewichtsumme (solche Kreise bedeuten, dass es keinen kürzesten Weg gibt, weil das Gesamtgewicht bei jedem Durchlauf des Kreises sinkt). Negative Kantengewichte sind zulässig.

Optionen:
siehe unten ...

-filter
Gibt nur vorhandene Abstände zurück und schneidet alle Werte *Inf* für nicht vorhandene Verbindungen zwischen Knotenpaaren ab.

Ergebnis:
Dictionary mit den Abständen zwischen allen Knotenpaaren.

struct::graph::op::FloydWarshall *g*
Sucht **kürzeste Wege** zwischen allen Knotenpaaren in gewichteten Graphen.

Zeitkomplexität: *O(V^3)*, wobei *V* die Zahl der Knoten ist.

Speicherkomplexität: *O(V^2)*.

Argumente:
siehe unten ...

Graph object *g* (input)
Gerichteter und gewichteter Graph *g*.

Ergebnis:

Dictionary mit den kürzesten Abständen von jedem Knoten zu jedem Knoten.

Hinweis: Der Algorithmus findet die Lösungen dynamisch. Er vergleicht alle möglichen Wege durch den Graphen zwischen jedem Knotenpaar. Der Graph darf keinen Kreis mit negativer Gewichtsumme besitzen (solche Kreise bedeuten, dass es keinen kürzesten Weg gibt, weil das Gesamtgewicht bei jedem Durchlauf des Kreises sinkt).

Umgekehrt lässt sich der Algorithmus verwenden, um solche Kreise zu finden: Ist irgendein vom Algorithmus gefundener kürzester Abstand für Knoten *v* und *u* negativ (wenn *v* derselbe Knoten wie *u* ist), gehört dieser Knoten sicher zu mindestens einem negativen Kreis.

struct::graph::op::MetricTravellingSalesman *g*

Algorithmus für eine metrische Variante beim Problem des Handlungsreisenden. Das *TSP-Problem* ist *NP-vollständig*, es gibt also keinen effizienten Algorithmus, der es löst. Gierige Verfahren werden mit wachsender Knotenmenge extrem langsam.

Argumente:

siehe unten ...

Graph object *g* (input)

Ungerichteter, gewichteter Graph *g*.

Ergebnis:

Näherungslösung eines minimalen *Hamilton-Kreises*, eines geschlossenen Weges, der alle Knoten besucht, jeden genau einmal.

Hinweis: **Es ist ein 2-Approximationsalgorithmus.**

struct::graph::op::Christofides *g*

Ein weiterer Algorithmus für das **metrische** *TSP-Problem*. Die Implementierung von Christofides verwendet ein *Maximum-Matching*, um einen besseren Approximationsfaktor zu erreichen.

Argumente:

siehe unten ...

Graph Object *g* (input)

Ungerichteter, gewichteter Graph *g*.

Ergebnis:

Näherungslösung eines minimalen *Hamilton-Kreises*, eines geschlossenen Weges, der alle Knoten besucht, jeden genau einmal.

Hinweis: **Es ist ein 3/2-Approximationsalgorithmus.**

struct::graph::op::GreedyMaxMatching *g*

Das Verfahren *Greedy Max Matching* findet ein **maximales Matching** (kein Maximum-Matching) für den gegebenen Graphen *g*. Es nimmt Kanten in die Lösung auf, beginnend bei den Kanten mit den niedrigsten Kosten.

Argumente:

siehe unten ...

Graph Object *g* (input)

Ungerichteter Graph *g*.

Ergebnis:

Menge von Kanten, das maximale Matching für den Graphen *g*.

struct::graph::op::MaxCut *g U V*

Algorithmus für das **Maximum-Schnitt-Problem**.

Argumente:

siehe unten ...

Graph Object *g* (input)

Der zu schneidende Graph.

List *U* (output)

Variable, die die erste Knotenmenge (Schnitt) der Lösung speichert.

List *V* (output)

Variable, die die zweite Knotenmenge (Schnitt) der Lösung speichert.

Ergebnis:

Der Algorithmus gibt die Zahl der Kanten zwischen den beiden gefundenen Knotenmengen zurück.

Hinweis: MaxCut ist ein **2-Approximationsalgorithmus.**

struct::graph::op::UnweightedKCenter *g k*

Approximationsalgorithmus für das K-Zentrum-Problem.

Argumente:

siehe unten ...

Graph Object *g* (input)

Ungerichteter vollständiger Graph *g*, der die Dreiecksungleichung erfüllt.

Integer *k* (input)

Positive Ganzzahl, die festlegt, wie viele Knoten im *k-Zentrum* enthalten sind.

Ergebnis:

Menge von Knoten, das *k*-Zentrum des Graphen *g*.

Hinweis: UnweightedKCenter ist ein **2-Approximationsalgorithmus.**

struct::graph::op::WeightedKCenter *g nodeWeights W*

Approximationsalgorithmus für eine gewichtete Variante beim K-Zentrum-Problem.

Argumente:

siehe unten ...

Graph Object *g* (input)

Ungerichteter vollständiger Graph *g*, der die Dreiecksungleichung erfüllt.

Integer *W* (input)

Positive Ganzzahl, die das höchstmögliche Gewicht des vom Algorithmus gefundenen *k-Zentrums* festlegt.

List *nodeWeights* (input)

Liste der Knoten und ihrer Gewichte im Graphen *g*.

Ergebnis:

Menge von Knoten, die der Algorithmus als Lösung gefunden hat.

Hinweis: WeightedKCenter ist ein **3-Approximationsalgorithmus.**

struct::graph::op::GreedyMaxIndependentSet *g*

Eine *maximale unabhängige Menge* ist eine *unabhängige Menge*, bei der das Hinzufügen jedes weiteren Knotens die Menge zwingt, eine Kante zu enthalten.

Der Algorithmus gibt für den Eingabegraphen *g* eine Menge von Knoten (Liste) zurück, die in der vom Algorithmus gefundenen maximalen unabhängigen Menge enthalten sind.

struct::graph::op::GreedyWeightedMaxIndependentSet *g nodeWeights*

Gewichtete Variante der *maximalen unabhängigen Menge*. Als Argument nimmt sie nicht nur den Graphen *g*, sondern auch eine Menge von Gewichten für alle Knoten in *g*.

Hinweis: Lesen Sie dazu auch die Beschreibung der *maximalen unabhängigen Menge*.

struct::graph::op::VerticesCover *g*

Eine *Knotenüberdeckung* ist eine Menge von Knoten, sodass jede Kante des Graphen zu mindestens einem Knoten der Menge inzident ist. Dieser 2-Approximationsalgorithmus sucht eine minimale *Knotenüberdeckung*, ein klassisches Optimierungsproblem der Informatik und ein typisches Beispiel eines *NP-schweren* Optimierungsproblems mit Approximationsalgorithmus. Für den Eingabegraphen *g* gibt der Algorithmus die Menge der Kanten (Liste) zurück, die die vom Algorithmus gefundene Knotenüberdeckung ist.

struct::graph::op::EdmondsKarp *g s t*

Verbesserter Algorithmus von Ford und Fulkerson. Er berechnet den **maximalen Fluss** im gegebenen Flussnetz *g*.

Argumente:

siehe unten ...

Graph Object *g* (input)

Gewichteter und gerichteter Graph. Jede Kante soll ein ganzzahliges Attribut haben, das als maximaler Durchsatz gilt, den diese Verbindung (Kante) tragen kann.

Node *s* (input)

Der Knoten, der die Quelle des Graphen *g* ist.

Node *t* (input)

Der Knoten, der die Senke des Graphen *g* ist.

Ergebnis:

Die Prozedur gibt das Dictionary mit den Durchsätzen aller Kanten zurück. Zu jedem Schlüssel (die Kante zwischen den Knoten *u* und *v* in der Form *list u v*) gibt es einen Wert, den Durchsatz dieses Schlüssels. Kanten, deren Durchsatz 0 ist, werden nicht zurückgegeben (das entspricht einer fehlenden Verbindung im Flussnetz zwischen den durch eine solche Kante verbundenen Knoten).

Die Grundidee des Algorithmus ist, die kürzesten Erweiterungspfade in *g* zu finden, solange es sie gibt, und für jeden Pfad die Kantengewichte entlang dieses Pfades mit dem größtmöglichen Durchsatz zu aktualisieren. Der endgültige (maximale) Fluss ist gefunden, wenn es keinen weiteren Erweiterungspfad von der Quelle zur Senke gibt.

Hinweis: Komplexität des Algorithmus: *O(V*E)*, wobei *V* die Zahl der Knoten und *E* die Zahl der Kanten in *g* ist.

struct::graph::op::BusackerGowen *g desiredFlow s t*

Der Algorithmus findet eine Lösung für das **Problem des kostenminimalen Flusses**. Gesucht ist ein Fluss, dessen Höchstwert *desiredFlow* sein kann, vom Quellknoten *s* zum Senkenknoten *t* im gegebenen Flussnetz *g*. Dieses Netz hat neben den Durchsätzen an den Kanten auch nichtnegative Kosten auf jeder Kante, die Kosten der Nutzung dieser Kante beim Leiten des Flusses über sie (zum Beispiel Treibstoffverbrauch, Zeit oder ein anderes nutzungsabhängiges Maß).

Argumente:

siehe unten ...

Graph Object *g* (input)

Flussnetz (gerichteter Graph). Jede Kante im Graphen soll zwei ganzzahlige Attribute haben: *cost* und *throughput*.

Integer *desiredFlow* (input)

Höchstwert des Flusses für dieses Netz.

Node *s* (input)

Der Quellknoten des Graphen *g*.

Node *t* (input)

Der Senkenknoten des Graphen *g*.

Ergebnis:

Dictionary mit den Werten der genutzten Durchsätze für jede Kante (Schlüssel), vom Algorithmus gefunden.

Hinweis: Komplexität des Algorithmus: *O(V**2*desiredFlow)*, wobei *V* die Zahl der Knoten in *g* ist.

struct::graph::op::ShortestsPathsByBFS *g s outputFormat*

Algorithmus für kürzeste Wege mit der BFS-Methode. Im Vergleich zu **struct::graph::op::dijkstra** kann er mit negativen Kantengewichten arbeiten. Negative Kreise sind nicht zulässig. Für dünne Graphen ist der Algorithmus besser als Dijkstra, es gibt aber pathologische Fälle (die in der Praxis gewöhnlich nicht auftreten), in denen die Zeitkomplexität mit der Knotenzahl exponentiell wächst.

Argumente:

siehe unten ...

Graph Object *g* (input)

Eingabegraph.

Node *s* (input)

Quellknoten, für den alle Abstände zu jedem anderen Knoten in *g* berechnet werden.

Optionen und Ergebnis:

siehe unten ...

distances

Ist das gewählte *outputFormat* **distances**, gibt die Prozedur ein Dictionary mit den Abständen zwischen dem Quellknoten *s* und jedem anderen Knoten in *g* zurück.

paths

Ist das gewählte *outputFormat* **paths**, gibt die Prozedur ein Dictionary zurück, das für jeden Knoten *v* eine Liste von Knoten enthält, den Weg zwischen dem Quellknoten *s* und dem Knoten *v*.

struct::graph::op::BFS *g s* ?*outputFormat*...?

Breitensuche. Der Algorithmus erzeugt den BFS-Baum. Speicher- und Zeitkomplexität: *O(V + E)*, wobei *V* die Zahl der Knoten und *E* die Zahl der Kanten ist.

Argumente:

siehe unten ...

Graph Object *g* (input)

Eingabegraph.

Node *s* (input)

Quellknoten für die BFS-Prozedur.

Optionen und Ergebnis:

siehe unten ...

graph

Ist das gewählte **outputFormat graph**, gibt die Prozedur eine Graphstruktur (**struct::graph**) zurück, die dem vom Algorithmus gefundenen BFS-Baum entspricht.

tree

Ist das gewählte **outputFormat tree**, gibt die Prozedur eine Baumstruktur (**struct::tree**) zurück, die dem vom Algorithmus gefundenen BFS-Baum entspricht.

struct::graph::op::MinimumDiameterSpanningTree *g*

Ziel ist, für den Eingabegraphen *g* den *Spannbaum* mit dem kleinsten *Durchmesser* zu finden.

Die Grundidee ist, *BFS* über alle Knoten in *g* laufen zu lassen. Ist der Durchmesser *d* des Baumes ungerade, ist der von *BFS* gelieferte Baum minimal (bezogen auf den Durchmesser). Ist der Durchmesser *d* gerade, kann der optimale Baum einen minimalen *Durchmesser* von *d* oder *d-1* haben.

Dann baut der Algorithmus den von *BFS* gelieferten Baum neu auf, indem er einen Knoten zwischen dem Wurzelknoten und dem Kindknoten (den Kindknoten) der Wurzel einfügt, sodass der mit diesem Kindknoten als Wurzel gebildete Teilbaum der größte ist (die größte Höhe hat). Im nächsten Schritt wird auf diesem neu aufgebauten Baum erneut *BFS* ausgeführt, mit dem neuen Knoten als Wurzel. Hat sich die Höhe des Baumes nicht geändert, ist eine bessere Lösung gefunden.

Für den Eingabegraphen *g* gibt der Algorithmus die Graphstruktur (**struct::graph**) zurück, die der vom Algorithmus gefundene Spannbaum mit minimalem Durchmesser ist.

struct::graph::op::MinimumDegreeSpanningTree *g*

Der Algorithmus findet für den Eingabegraphen *g* einen Spannbaum *T* mit dem kleinstmöglichen Grad. Das Problem ist *NP-schwer*, der Algorithmus ist daher ein Approximationsalgorithmus.

Sei *V* die Knotenmenge von *g* und *W* eine beliebige Teilmenge von *V*. Sei außerdem *OPT* die optimale Lösung und *ALG* die vom Algorithmus für *g* gefundene Lösung.

Es lässt sich beweisen, dass die gefundene Lösung folgende Ungleichung erfüllen muss:

*((|W| + k - 1) / |W|) <= ALG <= 2*OPT + log2(n) + 1*.

Argumente:

siehe unten ...

Graph Object *g* (input)

Ungerichteter schlichter Graph.

Ergebnis:

Der Algorithmus gibt eine Graphstruktur zurück, die dem vom Algorithmus gefundenen Spannbaum *T* entspricht.

struct::graph::op::MaximumFlowByDinic *g s t blockingFlowAlg*

Der Algorithmus findet den **maximalen Fluss** für das durch *g* dargestellte Flussnetz. Er beruht auf Verfahren zum Finden blockierender Flüsse. Die unterschiedlichen Komplexitäten passen dadurch besser zu verschiedenen Graphen.

Argumente:

siehe unten ...

Graph Object *g* (input)

Gerichteter Graph *g*, der das Flussnetz darstellt. Jede Kante soll das Attribut *throughput* mit einem ganzzahligen Wert haben.

Node *s* (input)

Der Quellknoten des Flussnetzes *g*.

Node *t* (input)

Der Senkenknoten des Flussnetzes *g*.

Optionen:

siehe unten ...

dinic

Die Prozedur findet den maximalen Fluss für das Flussnetz *g* mit dem Algorithmus von Dinic (**struct::graph::op::BlockingFlowByDinic**) für die Berechnung des blockierenden Flusses.

mkm

Die Prozedur findet den maximalen Fluss für das Flussnetz *g* mit dem Algorithmus von Malhotra, Kumar und Maheshwari (**struct::graph::op::BlockingFlowByMKM**) für die Berechnung des blockierenden Flusses.

Ergebnis:

Der Algorithmus gibt ein Dictionary zurück, das für jede Kante (Schlüssel) im Netz *g* den Flusswert enthält.

Hinweis: **struct::graph::op::BlockingFlowByDinic** hat die Komplexität *O(m*n^2)* und **struct::graph::op::BlockingFlowByMKM** die Komplexität *O(n^3)*, wobei *n* die Zahl der Knoten und *m* die Zahl der Kanten im Flussnetz *g* ist.

struct::graph::op::BlockingFlowByDinic *g s t*

Der Algorithmus findet für das gegebene Netz *g* mit Quelle *s* und Senke *t* einen **blockierenden Fluss**, aus dem sich ein *maximaler Fluss* für dieses Netz *g* gewinnen lässt.

Argumente:

siehe unten ...

Graph Object *g* (input)

Gerichteter Graph *g*, der das Flussnetz darstellt. Jede Kante soll das Attribut *throughput*

mit einem ganzzahligen Wert haben.

Node *s* (input)
Der Quellknoten des Flussnetzes *g*.

Node *t* (input)
Der Senkenknoten des Flussnetzes *g*.

Ergebnis:
Der Algorithmus gibt ein Dictionary zurück, das für jede Kante (Schlüssel) im Netz *g* den Wert des blockierenden Flusses enthält.

Hinweis: Die Komplexität des Algorithmus ist *O(n*m)*, wobei *n* die Zahl der Knoten und *m* die Zahl der Kanten im Flussnetz *g* ist.

struct::graph::op::BlockingFlowByMKM *g s t*
Der Algorithmus findet für das gegebene Netz *g* mit Quelle *s* und Senke *t* einen **blockierenden Fluss**, aus dem sich ein *maximaler Fluss* für dieses *Netz g* gewinnen lässt.
Argumente:
siehe unten ...
Graph Object *g* (input)
Gerichteter Graph *g*, der das Flussnetz darstellt. Jede Kante soll das Attribut *throughput* mit einem ganzzahligen Wert haben.

Node *s* (input)
Der Quellknoten des Flussnetzes *g*.

Node *t* (input)
Der Senkenknoten des Flussnetzes *g*.

Ergebnis:
Der Algorithmus gibt ein Dictionary zurück, das für jede Kante (Schlüssel) im Netz *g* den Wert des blockierenden Flusses enthält.

Hinweis: Die Komplexität des Algorithmus ist *O(n^2)*, wobei *n* die Zahl der Knoten im Flussnetz *g* ist.

struct::graph::op::createResidualGraph *g f*
Die Prozedur erzeugt einen *Residualgraphen* (oder ein **Residualnetz**) für das Netz *g* und den gegebenen Fluss *f*.
Argumente:
siehe unten ...
Graph Object *g* (input)
Flussnetz (gerichteter Graph, in dem jede Kante das Attribut *throughput* gesetzt hat).

dictionary *f* (input)
Die aktuellen Flüsse im Flussnetz *g*.

Ergebnis:
Die Prozedur gibt eine Graphstruktur zurück, die ein aus dem Eingabeflussnetz *g* erzeugter *Residualgraph* ist.

struct::graph::op::createAugmentingNetwork *g f path*

Die Prozedur erzeugt ein **Erweiterungsnetz** für ein gegebenes Residualnetz *g*, einen Fluss *f* und einen Erweiterungspfad *path*.

Argumente:

siehe unten ...

Graph Object *g* (input)

Residualnetz (gerichteter Graph), in dem für jede Kante zwei Attribute gesetzt sind: throughput und cost.

Dictionary *f* (input)

Dictionary, das für jede Kante (Schlüssel) den aktuellen Flusswert auf dieser Kante enthält.

List *path* (input)

Erweiterungspfad, eine Menge von Kanten (Liste), für die die Netzänderung erzeugt wird.

Ergebnis:

Der Algorithmus gibt eine Graphstruktur zurück, die das geänderte Erweiterungsnetz enthält.

struct::graph::op::createLevelGraph *Gf s*

Für den gegebenen Residualgraphen *Gf* findet die Prozedur den **Schichtengraphen**.

Argumente:

siehe unten ...

Graph Object *Gf* (input)

Residualnetz, in dem jede Kante ihr Attribut *throughput* auf einen bestimmten Wert gesetzt hat.

Node *s* (input)

Der Quellknoten des Residualnetzes *Gf*.

Ergebnis:

Die Prozedur gibt einen aus dem Eingabe-*Residualnetz* erzeugten *Schichtengraphen* zurück.

struct::graph::op::TSPLocalSearching *g C*

Der Algorithmus ist eine *Heuristik der lokalen Suche* für das *Problem des Handlungsreisenden* . Für eine Lösung des *TSP-Problems* prüft er, ob sich eine bessere Lösung finden lässt. Da *TSP* ein bekanntes NP-vollständiges Problem ist, ist der Algorithmus ein Approximationsalgorithmus (mit Approximationsfaktor 2).

Argumente:

siehe unten ...

Graph Object *g* (input)

Ungerichteter und vollständiger Graph, bei dem auf jeder einzelnen Kante das Attribut "weight" gesetzt ist.

List *C* (input)

Eine Liste von Kanten, die ein *Hamilton-Kreis* sind, die Lösung des *TSP-Problems* für den Graphen *g*.

Ergebnis:

Der Algorithmus gibt die beste Lösung für das *TSP-Problem* zurück, die er finden konnte.

Hinweis: Die Lösung hängt von der Wahl des Anfangskreises *C* ab. Ein besserer Kreis garantiert nicht, dass eine bessere Lösung gefunden wird. Die Praxis zeigt aber, dass der Startkreis eine möglichst kleine Gewichtsumme haben sollte.

struct::graph::op::TSPLocalSearching3Approx *g C*

Der Algorithmus ist eine *Heuristik der lokalen Suche* für das *Problem des Handlungsreisenden* . Für eine Lösung des *TSP-Problems* prüft er, ob sich eine bessere Lösung finden lässt. Da *TSP* ein bekanntes NP-vollständiges Problem ist, ist der Algorithmus ein Approximationsalgorithmus (mit Approximationsfaktor 3).

Argumente:

siehe unten ...

Graph Object *g* (input)

Ungerichteter und vollständiger Graph, bei dem auf jeder einzelnen Kante das Attribut "weight" gesetzt ist.

List *C* (input)

Eine Liste von Kanten, die ein *Hamilton-Kreis* sind, die Lösung des *TSP-Problems* für den Graphen *g*.

Ergebnis:

Der Algorithmus gibt die beste Lösung für das *TSP-Problem* zurück, die er finden konnte.

Hinweis: In der Praxis erweist sich der 3-Approximationsalgorithmus als deutlich wirksamer als die 2-Approximation, er hat aber den schlechteren Approximationsfaktor. Weitere Heuristiken der lokalen Suche (zum Beispiel die 4-Approximation) bringen nicht genug Gewinn, um den schlechteren Approximationsfaktor auszugleichen. Verwendet werden daher vor allem die 2- und die 3-Approximation.

struct::graph::op::createSquaredGraph *g*

Ein X-Quadrat-Graph hat dieselbe Knotenmenge wie der Eingabegraph *g*, aber eine andere Kantenmenge. Der X-Quadrat-Graph hat die Kante *(u,v)* genau dann, wenn der Abstand zwischen den Knoten *u* und *v* nicht größer als X ist und *u!= v*.

Die Prozedur gibt für den Eingabegraphen *g* dessen Zweiquadrat-Graphen zurück.

Hinweis: Die Abstände bei der Wahl der neuen Kantenmenge zählen die Zahl der Kanten, nicht die Summe der Kantengewichte.

struct::graph::op::createCompleteGraph *g originalEdges*

Für den Eingabegraphen *g* fügt die Prozedur fehlende Bögen hinzu, sodass ein *vollständiger Graph* entsteht. In der Variablen *originalEdges* hält sie außerdem die Menge der Bögen, die *g* vor dieser Operation besaß.

Hintergrund und Begriffe

Kürzeste Wege

Definition (*single-pair shortest path problem*):

Formal: Gegeben ein gewichteter Graph (sei *V* die Knotenmenge und *E* eine Kantenmenge) und ein Knoten *v* aus *V*. Gesucht ist ein Weg *P* von *v* zu einem *v'* aus V, sodass die Summe der

Kantengewichte entlang des Weges unter allen Wegen von v nach v' minimal ist.

Verallgemeinerungen:

- *Das Kürzester-Weg-Problem mit einer Quelle*, bei dem kürzeste Wege von einem Quellknoten v zu allen anderen Knoten im Graphen zu finden sind.
- *Das Kürzester-Weg-Problem mit einem Ziel*, bei dem kürzeste Wege von allen Knoten im Graphen zu einem einzelnen Zielknoten v zu finden sind. Das lässt sich auf das Problem mit einer Quelle zurückführen, indem die Kanten im Graphen umgekehrt werden.
- *Das Kürzester-Weg-Problem für alle Paare*, bei dem kürzeste Wege zwischen jedem Knotenpaar v, v' im Graphen zu finden sind.

Hinweis: Das Ergebnis des *Kürzester-Weg-Problems* kann ein *Kürzester-Weg-Baum* sein, ein Teilgraph eines gegebenen (möglicherweise gewichteten) Graphen, so konstruiert, dass der Abstand zwischen einem gewählten Wurzelknoten und allen anderen Knoten minimal ist. Es ist ein Baum, weil sich bei zwei Wegen zwischen dem Wurzelknoten und einem Knoten v (also einem Kreis) die letzte Kante des längeren Weges löschen lässt, ohne den Abstand vom Wurzelknoten zu irgendeinem Knoten im Teilgraphen zu vergrößern.

Problem des Handlungsreisenden

Definition:

Für einen kantengewichteten Graphen (die Kantengewichte sollen positiv sein) ist das Ziel, den Kreis zu finden, der jeden Knoten im Graphen genau einmal besucht (*Hamilton-Kreis*).

Verallgemeinerungen:

- *Metrisches TSP* – Eine sehr natürliche Einschränkung des *TSP* verlangt, dass die Abstände zwischen den Städten eine *Metrik* bilden, also die *Dreiecksungleichung* erfüllen. Für beliebige drei Städte *A*, *B* und *C* darf der Abstand zwischen *A* und *C* höchstens der Abstand von *A* nach *B* plus der Abstand von *B* nach *C* sein. Die meisten natürlichen Instanzen des *TSP* erfüllen diese Einschränkung.
- *Euklidisches TSP* – Das euklidische TSP, oder *planare TSP*, ist das *TSP*, bei dem der Abstand der gewöhnliche *euklidische Abstand* ist. Das *euklidische TSP* ist ein Sonderfall des *TSP* mit *Dreiecksungleichung*, denn Abstände in der Ebene erfüllen die Dreiecksungleichung. Es scheint aber leichter zu sein als das allgemeine *TSP* mit *Dreiecksungleichung*. Zum Beispiel ist *der minimale Spannbaum* des zu einer Instanz des *euklidischen TSP* gehörigen Graphen ein *euklidischer minimaler Spannbaum* und lässt sich für *n* Punkte in erwarteter Zeit *O(n log n)* berechnen (deutlich weniger als die Kantenzahl). Dadurch arbeitet der einfache *2-Approximationsalgorithmus* für das TSP mit Dreiecksungleichung von oben schneller.
- *Asymmetrisches TSP* – In den meisten Fällen ist der Abstand zwischen zwei Knoten im *TSP* -Netz in beiden Richtungen gleich. Der Fall, in dem der Abstand von *A* nach *B* nicht gleich dem Abstand von *B* nach *A* ist, heißt *asymmetrisches TSP*. Eine praktische Anwendung eines *asymmetrischen TSP* ist die Routenoptimierung mit straßengenauer Wegeführung (asymmetrisch durch Einbahnstraßen, Verbindungsrampen und Autobahnen).

Zuordnungsproblem

Definition:

Gegeben ein Graph *G = (V,E)*. Ein Matching oder eine *kantenunabhängige Menge M* in *G* ist eine Menge paarweise nicht benachbarter Kanten, das heißt, keine zwei Kanten teilen einen gemeinsamen Knoten. Ein Knoten ist *zugeordnet*, wenn er zu einer Kante im *Matching M*

inzident ist. Sonst ist der Knoten *nicht zugeordnet*.

Verallgemeinerungen:

- *Maximales Matching* – ein Matching *M* eines Graphen G mit der Eigenschaft, dass *M* kein *Matching* mehr ist, wenn irgendeine nicht in *M* liegende Kante zu *M* hinzugefügt wird. *M* ist also maximal, wenn es keine echte Teilmenge eines anderen *Matchings* im Graphen G ist. Anders gesagt: Ein *Matching M* eines Graphen G ist maximal, wenn jede Kante in G einen nichtleeren Schnitt mit mindestens einer Kante in *M* hat.
- *Maximum-Matching* – ein Matching, das die größtmögliche Zahl von Kanten enthält. Es kann viele *Maximum-Matchings* geben. Die *Matchingzahl* eines Graphen G ist die Größe eines *Maximum-Matchings*. Jedes *Maximum-Matching* ist *maximal*, aber nicht jedes *maximale Matching* ist ein *Maximum-Matching*.
- *Perfektes Matching* – ein Matching, das alle Knoten des Graphen zuordnet. Jeder Knoten des Graphen ist also zu genau einer Kante des Matchings inzident. Jedes *perfekte Matching* ist *maximum* und damit *maximal*. In einem Teil der Literatur heißt es *vollständiges Matching*. Ein *perfektes Matching* ist auch eine *Kantenüberdeckung minimaler Größe*. Außerdem ist die Größe eines *Maximum-Matchings* nicht größer als die Größe einer *minimalen Kantenüberdeckung*.
- *Fast perfektes Matching* – ein Matching, in dem genau ein Knoten nicht zugeordnet ist. Das ist nur möglich, wenn der Graph eine ungerade Knotenzahl hat, und ein solches *Matching* muss *maximum* sein. Gibt es für jeden Knoten eines Graphen ein fast perfektes Matching, das nur diesen Knoten auslässt, heißt der Graph auch *faktorkritisch*.

Verwandte Begriffe:

- *Alternierender Weg* – zu einem Matching *M* ist ein *alternierender Weg* ein Weg, dessen Kanten abwechselnd zum Matching und nicht zum Matching gehören.
- *Erweiterungspfad* – zu einem Matching *M* ist ein *Erweiterungspfad* ein *alternierender Weg*, der an freien (nicht zugeordneten) Knoten beginnt und endet.

Schnittprobleme

Definition:

Ein *Schnitt* ist eine Partition der Knoten eines Graphen in zwei *disjunkte Teilmengen*. Die *Schnittmenge* des *Schnitts* ist die Menge der Kanten, deren Endpunkte in verschiedenen Teilmengen der Partition liegen. Kanten kreuzen den Schnitt, wenn sie in seiner *Schnittmenge* liegen.

Formal:

- Ein *Schnitt C = (S,T)* ist eine Partition von *V* eines Graphen *G = (V, E)*.
- Ein *s-t-Schnitt C = (S,T)* eines *Flussnetzes N = (V, E)* ist ein Schnitt von *N*, bei dem *s* in *S* und *t* in *T* liegt, wobei *s* und *t* die *Quelle* beziehungsweise die *Senke* von *N* sind.
- Die *Schnittmenge* eines *Schnitts C = (S,T)* ist diejenige Kantenmenge des Graphen *G = (V, E)* , bei der jede Kante *(u, v)* die Bedingung erfüllt, dass *u* in *S* und *v* in *T* liegt.

In einem *ungewichteten ungerichteten* Graphen ist die Größe oder das Gewicht eines Schnitts die Zahl der Kanten, die den Schnitt kreuzen. In einem *gewichteten Graphen* ist derselbe Begriff die Summe der Gewichte der Kanten, die den Schnitt kreuzen.

In einem *Flussnetz* ist ein *s-t-Schnitt* ein Schnitt, der *Quelle* und *Senke* in verschiedenen Teilmengen verlangt, und seine *Schnittmenge* besteht nur aus Kanten, die von der Seite der

Quelle zur Seite der *Senke* führen. Die Kapazität eines *s-t-Schnitts* ist die Summe der Kapazitäten aller Kanten in der *Schnittmenge*.

Der *Schnitt* eines Graphen kann manchmal seine *Schnittmenge* meinen statt der Partition.

Verallgemeinerungen:

- *Minimaler Schnitt* – Ein Schnitt ist minimal, wenn die Größe des Schnitts nicht größer ist als die Größe jedes anderen Schnitts.
- *Maximaler Schnitt* – Ein Schnitt ist maximal, wenn die Größe des Schnitts nicht kleiner ist als die Größe jedes anderen Schnitts.
- *Dünnster Schnitt* – Das *Problem des dünnsten Schnitts* ist, die Knoten so in zwei Teile zu teilen, dass das Verhältnis aus der Zahl der Kanten über den Schnitt und der Zahl der Knoten in der kleineren Hälfte der Partition minimal wird.

K-Zentrum-Problem

Definitionen:

siehe unten ...

Ungewichtetes K-Zentrum

Für jede Menge *S* (eine Teilmenge von *V*) und einen Knoten *v* sei *connect(v,S)* die Kosten der billigsten Kante, die *v* mit irgendeinem Knoten in *S* verbindet. Ziel ist ein *S* mit |*S*| = *k*, für das *max_v{connect(v,S)}* möglichst klein ist.

Anders gesagt lässt sich das zum Beispiel verwenden, um die besten Standorte in der Stadt (Knoten des Eingabegraphen) für k Gebäude zu finden, sodass diese Gebäude allen anderen Orten der Stadt möglichst nah sind.

Gewichtetes K-Zentrum

Die Variante des *ungewichteten k-Zentrum-Problems*. Der Graph ist kantengewichtet, und die Knoten des Eingabegraphen *g* haben ebenfalls Gewichte. Außerdem gibt es die Einschränkung *W*. Ziel ist eine Knotenmenge *S* (eine Teilmenge von *V*), deren Gesamtgewicht nicht größer als *W* ist und für die die Funktion *max_v { min_u { cost(u,v) }}* den kleinstmöglichen Wert hat (*v* ist ein Knoten in *V* und *u* ein Knoten in *S*).

Flussprobleme

Definitionen:

- *das maximale Flussproblem* – Ziel ist ein zulässiger Fluss durch ein Flussnetz mit einer Quelle und einer Senke, der maximal ist. Das *maximale Flussproblem* ist ein Sonderfall komplexerer Netzflussprobleme, etwa des *Zirkulationsproblems*. Der Höchstwert eines *s-t-Flusses* ist gleich der minimalen Kapazität eines *s-t-Schnitts* im Netz, wie der *Satz vom maximalen Fluss und minimalen Schnitt* sagt.

```
Formaler für ein Flussnetz *G = (V,E)*, in dem für jede Kante *(u,
    v)* ihr Durchsatz *c(u,v)* festgelegt ist. Als *Fluss* *F*
    definieren wir eine Menge nichtnegativer ganzzahliger Attribute
    *f(u,v)* auf den Kanten, die folgende Bedingungen erfüllen:

1. für jede Kante *(u, v)* in *G* soll diese Bedingung erfüllt sein:
         0 <= f(u,v) <= c(u,v)
```

```
2. Das Netz *G* hat einen Quellknoten *s*, sodass der Fluss *F*
    gleich der Summe des ausgehenden Flusses vermindert um die Summe
    des eingehenden Flusses an diesem Quellknoten *s* ist.

3. Das Netz *G* hat einen Senkenknoten *t*, sodass der Wert *-F*
    gleich der Summe des eingehenden Flusses vermindert um die Summe
    des ausgehenden Flusses an diesem Senkenknoten *t* ist.

4. Für jeden Knoten, der weder *Quelle* noch *Senke* ist, sollen die
    Summe des eingehenden Flusses und die Summe des ausgehenden
    Flusses gleich sein.
```

- *das Problem des kostenminimalen Flusses* – Ziel ist der billigstmögliche Weg, eine bestimmte Flussmenge durch ein *Flussnetz* zu schicken.
- *blockierender Fluss* – einen *blockierenden Fluss* für ein *Residualnetz Gf* nennen wir einen Fluss *b* in *Gf*, für den gilt:
  1. Jeder Weg von der *Senke* zur *Quelle* ist der kürzeste Weg in *Gf*.
  2. Jeder kürzeste Weg in *Gf* enthält eine Kante mit voll ausgeschöpftem Durchsatz in *Gf+b*.
- *Residualnetz* – für ein Flussnetz *G* und einen Fluss *f* wird das *Residualnetz* aus den Kanten aufgebaut, die einen größeren Fluss senden können. Es enthält nur die Kanten, die einen Fluss größer als 0 senden können.
- *Schichtennetz* – es hat dieselbe Knotenmenge wie der *Residualgraph*, aber nur die Kanten *(u,v)* aus *Gf*, für die die Gleichung *distance(s,u)+1 = distance(s,v)* gilt.
- *Erweiterungsnetz* – eine Änderung des *Residualnetzes* unter Berücksichtigung der neuen Flusswerte. Die Struktur bleibt unverändert, die Durchsätze und Kosten an den Kanten sind andere.

Approximationsalgorithmus

k-Approximationsalgorithmus:

Ein Algorithmus ist eine k-Approximation, wenn für *ALG* (die vom Algorithmus zurückgegebene Lösung) und *OPT* (die optimale Lösung) diese Ungleichung gilt:
- für Minimierungsprobleme: *ALG/OPT <= k*
- für Maximierungsprobleme: *OPT/ALG <= k*

Literatur

1. Adjacency matrix
2. Adjacency list
3. Kruskal's algorithm
4. Prim's algorithm
5. Bipartite graph
6. Strongly connected components
7. Tarjan's strongly connected components algorithm
8. Cut vertex
9. Bridge

10. Bellman-Ford's algorithm
11. Johnson's algorithm
12. Floyd-Warshall's algorithm
13. Travelling Salesman Problem
14. Christofides Algorithm
15. Max Cut
16. Matching
17. Max Independent Set
18. Vertex Cover
19. Ford-Fulkerson's algorithm
20. Maximum Flow problem
21. Busacker-Gowen's algorithm
22. Dinic's algorithm
23. K-Center problem
24. BFS
25. Minimum Degree Spanning Tree
26. Approximation algorithm

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: graph* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright


```
Copyright (c) 2008 Alejandro Paz <vidriloco@gmail.com>
Copyright (c) 2008 (docs) Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2009 Michal Antoniewski <antoniewski.m@gmail.com>
```


matrix

Stichwörter: matrix

Name

struct::matrix – Matrixobjekte erzeugen und bearbeiten

Übersicht

package require Tcl 8.5 9
package require struct::matrix ?**2.2**?
::struct::matrix ?*aMatrix*? ?=|*:=*|*as*|*deserialize source*?

aMatrix *option* ?*arg arg ...*?
aMatrix **=** *sourceMatrix*
aMatrix **-->** *destMatrix*
aMatrix **add column** ?*values*?
aMatrix **add columns** *n*
aMatrix **add row** ?*values*?
aMatrix **add rows** *n*
aMatrix **cells**
aMatrix **cellsize** *column row*
aMatrix **columns**
aMatrix **columnwidth** *column*
aMatrix **delete column** *column*
aMatrix **delete columns** *n*
aMatrix **delete row** *row*
aMatrix **delete rows** *n*
aMatrix **deserialize** *serialization*
aMatrix **destroy**
aMatrix **format 2string** ?*report*?
aMatrix **format 2chan** ??*report*? *channel?*
aMatrix **get cell** *column row*
aMatrix **get column** *column*
aMatrix **get rect** *column_tl row_tl column_br row_br*
aMatrix **get row** *row*
aMatrix **insert column** *column* ?*values*?
aMatrix **insert row** *row* ?*values*?
aMatrix **link** ?*-transpose*? *arrayvar*
aMatrix **links**
aMatrix **rowheight** *row*
aMatrix **rows**
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **all** *pattern*
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **column** *column pattern*
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **row** *row pattern*
aMatrix **search** ?*-nocase*? ?*-exact*|*-glob*|*-regexp*? **rect** *column_tl row_tl column_br row_br pattern*
aMatrix **serialize** ?*column_tl row_tl column_br row_br*?
aMatrix **set cell** *column row value*
aMatrix **set column** *column values*
aMatrix **set rect** *column row values*
aMatrix **set row** *row values*
aMatrix **sort columns** ?**-increasing|-decreasing**? *row*
aMatrix **sort rows** ?**-increasing|-decreasing**? *column*
aMatrix **swap columns** *column_a column_b*
aMatrix **swap rows** *row_a row_b*
aMatrix **transpose**
aMatrix **unlink** *arrayvar*

Beschreibung

Dieses Paket stellt einen Befehl zum Erzeugen von Matrizen bereit und Unterbefehle, mit denen sich erzeugte Matrizen bearbeiten lassen.

Eine Matrix ist eine rechteckige Anordnung von Zellen in Zeilen und Spalten. Jede Zelle enthält einen einzelnen Wert beliebiger Art. Jede Zelle lässt sich über ein Paar bei **0** beginnender ganzer Zahlen ansprechen, in der Form {column, row}.

Der besondere nichtnumerische Index **end** spricht je nach Zusammenhang die letzte Zeile oder die letzte Spalte einer Matrix an. Indizes der Form **end**-**number** werden vom Ende der Zeile oder Spalte gezählt, genau wie bei gewöhnlichen Tcl-Listen.

Ein Index außerhalb des gültigen Bereichs löst einen Fehler aus.

Vorgabe ist, dass jede Matrix leer erzeugt wird. Neue Zeilen und Spalten lassen sich nach Bedarf hinzufügen.

Eine Matrix kann eine Array-Sicht auf ihren Inhalt bereitstellen. Damit kann zum Beispiel *tkTable* die Matrix in die Anzeige einbinden.

Der Hauptbefehl dieses Pakets ist:

::struct::matrix ?*aMatrix*? ?=|:=|as|deserialize *source*?

Dieser Befehl erzeugt ein neues Matrixobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *aMatrix* ist. Mit diesem Befehl lassen sich verschiedene Operationen auf der Matrix ausführen. Er hat die folgende allgemeine Form:

aMatrix *option* ?*arg arg ...*?

option und die *arg*s legen fest, was der Befehl tut.

Ist *aMatrix* nicht angegeben, erzeugt das Paket selbst einen eindeutigen Namen. Ist eine *source* angegeben, wird die neue Matrix daraus initialisiert. Bei den Operatoren **=**,**:=** und **as** gilt das Argument *source* als Name eines anderen Matrixobjekts, und der Zuweisungsoperator **=** wird ausgeführt. Bei **deserialize** ist *source* ein serialisiertes Matrixobjekt, und **deserialize** wird ausgeführt.

Mit anderen Worten

```
    ::struct::matrix mymatrix = b
```

entspricht

```
    ::struct::matrix mymatrix
    mymatrix = b
```

und

```
    ::struct::matrix mymatrix deserialize $b
```

entspricht

```
    ::struct::matrix mymatrix
    mymatrix deserialize $b
```

Matrixobjekte stellen die folgenden Befehle bereit:

aMatrix* = *sourceMatrix
Der Operator **=** ist der *Zuweisungs*operator für Matrixobjekte. (Dieser Operator lässt sich als **=**,**:=** oder **as** schreiben.) Er kopiert die Matrix aus dem Matrixobjekt *sourceMatrix* über die Matrixdaten in *aMatrix*. Der bisherige Inhalt von *aMatrix* wird bei dieser Operation gelöscht.

Die Operation entspricht praktisch

```
    aMatrix deserialize [sourceMatrix serialize]
```

aMatrix* --> *destMatrix
Der Operator **-->** ist der Operator der *umgekehrten Zuweisung* für Matrixobjekte. Er kopiert die Matrix aus dem Matrixobjekt *aMatrix* über die Matrixdaten im Objekt *destMatrix*. Der bisherige Inhalt von *destMatrix* wird bei dieser Operation gelöscht.

Die Operation entspricht praktisch

```
    destMatrix deserialize [aMatrix serialize]
```

***aMatrix* add column ?*values*?**
Erweitert die Matrix um eine Spalte hinter der letzten Spalte.

Ist kein *values* angegeben, werden die neuen Zellen auf die leere Zeichenkette gesetzt. Sind *values angegeben*, wendet diese Methode praktisch **set column** (siehe unten) auf die neu hinzugefügte Spalte an.

aMatrix* add columns *n
Erweitert die Matrix um *n* Spalten hinter der letzten Spalte und setzt alle neuen Zellen auf die leere Zeichenkette.

Ist *n* <= *0*, wird ein Fehler ausgelöst.

***aMatrix* add row ?*values*?**
Erweitert die Matrix um eine Zeile hinter der letzten Zeile.

Ist kein *values* angegeben, werden die neuen Zellen auf die leere Zeichenkette gesetzt. Sind *values angegeben*, wendet diese Methode praktisch **set row** (siehe unten) auf die neu hinzugefügte Zeile an.

aMatrix* add rows *n
Erweitert die Matrix um *n* Zeilen hinter der letzten Zeile und setzt alle neuen Zellen auf die leere Zeichenkette.

Ist *n* <= *0*, wird ein Fehler ausgelöst.

***aMatrix* cells**
Gibt die Zahl der Zellen zurück, die die Matrix derzeit verwaltet. Das ist das Produkt aus **rows** und **columns**.

aMatrix* cellsize *column row
Gibt die Länge der Zeichenkettendarstellung des Werts zurück, den die angesprochene Zelle derzeit enthält.

***aMatrix* columns**
Gibt die Zahl der Spalten zurück, die die Matrix derzeit verwaltet.

aMatrix* columnwidth *column
Gibt die Länge der längsten Zeichenkettendarstellung aller Werte zurück, die die Zellen der angesprochenen Spalte derzeit enthalten, sofern diese alle nur eine Zeile umfassen. Bei Zellwerten über mehrere Zeilen geht die Länge ihrer längsten Zeile in die Berechnung ein.

Hinweis: Der Befehl erkennt ANSI-Farbsteuersequenzen und nimmt sie aus der Breite einer Zeile heraus, weil sie logisch die Breite null haben.

Hinweis: Der Befehl erkennt asiatische Zeichen doppelter Breite und berücksichtigt sie.

Mit anderen Worten ist die zurückgegebene Länge die Zahl der Terminalspalten, die die Zeichenkette belegt, und nicht die Zahl der Zeichen.

aMatrix* delete column *column
Löscht die angegebene Spalte aus der Matrix und rückt alle Spalten mit höherem Index um einen Index nach unten.

aMatrix* delete columns *n
Löscht *n* Spalten von rechts aus der Matrix. Der Wert von *n* muss die Bedingung "0 < *n* < [**aMatrix columns**]" erfüllen.

aMatrix* delete row *row
Löscht die angegebene Zeile aus der Matrix und rückt alle Zeilen mit höherem Index um einen Index nach unten.

aMatrix* delete rows *n
Löscht *n* Zeilen von unten aus der Matrix. Der Wert von *n* muss die Bedingung "0 < *n* < [**aMatrix rows**]" erfüllen.

aMatrix* deserialize *serialization
Das ist das Gegenstück zu **serialize**. Sie ersetzt die Matrixdaten in *aMatrix* durch die Matrix, die der Wert *serialization* beschreibt. Der bisherige Inhalt von *aMatrix* wird bei dieser Operation gelöscht.

***aMatrix* destroy**
Zerstört die Matrix, einschließlich ihres Speichers und des zugehörigen Befehls.

***aMatrix* format 2string ?*report*?**
Formatiert die Matrix mit dem angegebenen Berichtsobjekt und gibt die Zeichenkette mit dem Ergebnis dieser Operation zurück. Der Bericht muss die Methode **printmatrix** unterstützen. Ist kein *report* angegeben, verwendet das System eine interne Berichtsdefinition, um die Matrix zu formatieren.

***aMatrix* format 2chan ??*report*? *channel*?**
Formatiert die Matrix mit dem angegebenen Berichtsobjekt und schreibt die Zeichenkette mit dem Ergebnis dieser Operation in den Kanal. Der Bericht muss die Methode **printmatrix2channel** unterstützen. Ist kein *report* angegeben, verwendet das System eine interne Berichtsdefinition, um die Matrix zu formatieren. Ist kein *channel* angegeben, verwendet das System **stdout**.

aMatrix* get cell *column row
Gibt den Wert zurück, den die durch Zeilen- und Spaltenindex bestimmte Zelle derzeit enthält.

aMatrix* get column *column

Gibt eine Liste mit den Werten aus allen Zellen der durch den Index bestimmten Spalte zurück. Der Inhalt der Zelle in Zeile 0 steht als erstes Element dieser Liste.

aMatrix* get rect *column_tl row_tl column_br row_br

Gibt eine Liste von Listen mit Zellwerten zurück. Die im Ergebnis gespeicherten Werte stammen aus der Teilmatrix, deren Zellen links oben und rechts unten durch *column_tl, row_tl* bzw. *column_br, row_br* bestimmt sind. Dabei müssen die folgenden Gleichungen gelten: "*column_tl* <= *column_br*" und "*row_tl* <= *row_br*". Das Ergebnis ist so aufgebaut: Die äußere Liste ist die Liste der Zeilen, ihre Elemente sind Listen, die eine einzelne Zeile darstellen. Die Zeile mit dem kleinsten Index ist das erste Element der äußeren Liste. Die Elemente der Zeilenlisten stellen die ausgewählten Zellwerte dar. Die Zelle mit dem kleinsten Index ist das erste Element jeder Zeilenliste.

aMatrix* get row *row

Gibt eine Liste mit den Werten aus allen Zellen der durch den Index bestimmten Zeile zurück. Der Inhalt der Zelle in Spalte 0 steht als erstes Element dieser Liste.

***aMatrix* insert column *column* ?*values*?**

Erweitert die Matrix um eine Spalte und verhält sich dann wie **set column** (siehe unten) auf dieser neuen Spalte, falls *values* angegeben wurden. Ohne *values* werden die neuen Zellen auf die leere Zeichenkette gesetzt. Die neue Spalte wird unmittelbar vor der Spalte mit dem angegebenen Index eingefügt. Das heißt: Ist *column* kleiner oder gleich null, wird die neue Spalte am Anfang der Matrix eingefügt, vor der ersten Spalte. Hat *column* den Wert **end**, oder ist er größer oder gleich der Zahl der Spalten in der Matrix, wird die neue Spalte an die Matrix angehängt, hinter die letzte Spalte. Die alte Spalte am gewählten Index und alle Spalten mit höherem Index rücken um einen Index nach oben.

***aMatrix* insert row *row* ?*values*?**

Erweitert die Matrix um eine Zeile und verhält sich dann wie **set row** (siehe unten) auf dieser neuen Zeile, falls *values* angegeben wurden. Ohne *values* werden die neuen Zellen auf die leere Zeichenkette gesetzt. Die neue Zeile wird unmittelbar vor der Zeile mit dem angegebenen Index eingefügt. Das heißt: Ist *row* kleiner oder gleich null, wird die neue Zeile am Anfang der Matrix eingefügt, vor der ersten Zeile. Hat *row* den Wert **end**, oder ist er größer oder gleich der Zahl der Zeilen in der Matrix, wird die neue Zeile an die Matrix angehängt, hinter die letzte Zeile. Die alte Zeile an diesem Index und alle Zeilen mit höherem Index rücken um einen Index nach oben.

aMatrix* link ?-transpose? *arrayvar

Verknüpft die Matrix mit der angegebenen Array-Variable. Der Inhalt aller Zellen der Matrix liegt damit auch im Array, und alle Änderungen an der Matrix werden ebenfalls dorthin übertragen. Der Inhalt der Zelle *(column,row)* wird im Array unter dem Schlüssel *column,row* gespeichert. Ist die Option **-transpose** angegeben, wird stattdessen der Schlüssel *row,column* verwendet. Die Matrix lässt sich mit mehr als einem Array verknüpfen. Die Verknüpfung geht in beide Richtungen, Änderungen am Array werden also auch in der Matrix gespiegelt.

***aMatrix* links**

Gibt eine Liste mit den Namen aller Array-Variablen zurück, mit denen die Matrix durch einen Aufruf der Methode **link** verknüpft wurde.

aMatrix* rowheight *row

Gibt die Höhe der angegebenen Zeile in Textzeilen zurück. Das ist die größte Zahl von

Textzeilen, die eine Zelle unter allen Zellen der Zeile einnimmt.

***aMatrix* rows**
Gibt die Zahl der Zeilen zurück, die die Matrix derzeit verwaltet.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? all *pattern
Durchsucht die ganze Matrix nach Zellen, die auf das *pattern* passen, und gibt eine Liste aller Treffer zurück. Jeder Eintrag dieser Liste ist selbst eine Liste und enthält den Spalten- und den Zeilenindex der passenden Zelle, in dieser Reihenfolge. Die Ergebnisse sind zuerst nach der Spalte und dann nach der Zeile geordnet, beides aufsteigend. Treffer links und oben in der Matrix stehen damit vor Treffern rechts und unten.

Die Art des Musters (Zeichenkette, Glob, regulärer Ausdruck) bestimmt die Option nach dem Schlüsselwort **search**. Fehlt die Option, gilt **-exact**.

Ist die Option **-nocase** angegeben, unterscheidet die Suche nicht zwischen Groß- und Kleinschreibung.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? column *column pattern
Wie **search all**, die Suche ist aber auf die angegebene Spalte beschränkt.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? row *row pattern
Wie **search all**, die Suche ist aber auf die angegebene Zeile beschränkt.

aMatrix* search ?-nocase? ?-exact|-glob|-regexp? rect *column_tl row_tl column_br row_br pattern
Wie **search all**, die Suche ist aber auf den angegebenen rechteckigen Bereich der Matrix beschränkt.

***aMatrix* serialize ?*column_tl row_tl column_br row_br*?**
Diese Methode serialisiert die Teilmatrix, die die Rechteckangabe aufspannt. Sie gibt also einen Tcl-*Wert* zurück, der diese Matrix vollständig beschreibt. Ist kein Rechteck angegeben, wird die ganze Matrix serialisiert. So lassen sich Matrixobjekte (oder Teile davon) zum Beispiel über beliebige Kanäle übertragen, dauerhaft ablegen und so weiter. Diese Methode ist auch die Grundlage für den Kopierkonstruktor und den Zuweisungsoperator.

Das Ergebnis dieser Methode muss über alle Implementierungen der Matrixschnittstelle semantisch gleich sein. Erst dadurch lassen sich Matrixdaten zwischen verschiedenen Implementierungen derselben Schnittstelle kopieren.

Das Ergebnis ist eine Liste mit genau drei Elementen.

Die ersten beiden Elemente der Liste geben die Zahl der Zeilen und der Spalten der Matrix an, in dieser Reihenfolge. Die Werte sind ganze Zahlen größer oder gleich null.

Das letzte Element der Liste enthält die Werte der serialisierten Matrixzellen, in der Form eines Wertes, wie ihn **get rect** zurückgibt. Leere Zellen rechts und unten in der Matrix dürfen in diesem Wert jedoch fehlen, weil die Größenangabe in der Serialisierung dem Empfänger erlaubt, trotz der fehlenden Werte eine Matrix der richtigen Größe zu erzeugen.

```
    # A possible serialization for the matrix structure
    #
    # | a b d g |
    # | c e     |
```

```
    # | f       |
    #
    # is
    #
    # 3 4 {{a b d g} {c e} {f}}
```

aMatrix* set cell *column row value
Setzt den Wert in der durch Zeilen- und Spaltenindex bestimmten Zelle auf die Daten im dritten Argument.

aMatrix* set column *column values
Setzt die Werte in den durch den Spaltenindex bestimmten Zellen auf die Elemente der Liste, die als drittes Argument übergeben wird. Jedes Element der Liste wird einer Zelle zugewiesen, das erste Element der Zelle in Zeile 0 und dann nach oben. Enthält die Liste weniger Werte, als Zeilen vorhanden sind, werden die übrigen Zeilen auf die leere Zeichenkette gesetzt. Enthält die Liste mehr Werte, als Zeilen vorhanden sind, werden die überzähligen Elemente ignoriert. Die Matrix wird durch diese Operation nicht erweitert.

aMatrix* set rect *column row values
Nimmt eine Liste von Listen mit Zellwerten und schreibt sie in die Teilmatrix, deren Zelle links oben durch die beiden Indizes bestimmt ist. Sind die Teillisten der äußeren Liste nicht gleich lang, werden die kürzeren Teillisten mit leeren Zeichenketten auf die Länge der längsten Teilliste aufgefüllt. Reicht die durch die Zelle links oben und die Zahl der Zeilen und Spalten in *values* bestimmte Teilmatrix über die Matrix hinaus, die geändert wird, werden die überstehenden Teile der Werte ignoriert, also praktisch abgeschnitten. Dieser Unterbefehl erwartet seine Eingabe in dem Format, das **get rect** zurückgibt.

aMatrix* set row *row values
Setzt die Werte in den durch den Zeilenindex bestimmten Zellen auf die Elemente der Liste, die als drittes Argument übergeben wird. Jedes Element der Liste wird einer Zelle zugewiesen, das erste Element der Zelle in Spalte 0 und dann nach oben. Enthält die Liste weniger Werte, als Spalten vorhanden sind, werden die übrigen Spalten auf die leere Zeichenkette gesetzt. Enthält die Liste mehr Werte, als Spalten vorhanden sind, werden die überzähligen Elemente ignoriert. Die Matrix wird durch diese Operation nicht erweitert.

aMatrix* sort columns ?-increasing|-decreasing? *row
Sortiert die Spalten der Matrix und verwendet die Daten in der angegebenen *row* als Sortierschlüssel. Die Optionen **-increasing** und **-decreasing** haben dieselbe Bedeutung wie bei **lsort**. Fehlt die Option, wird **-increasing** angenommen.

aMatrix* sort rows ?-increasing|-decreasing? *column
Sortiert die Zeilen der Matrix und verwendet die Daten in der angegebenen *column* als Sortierschlüssel. Die Optionen **-increasing** und **-decreasing** haben dieselbe Bedeutung wie bei **lsort**. Fehlt die Option, wird **-increasing** angenommen.

aMatrix* swap columns *column_a column_b
Vertauscht den Inhalt der beiden angegebenen Spalten.

aMatrix* swap rows *row_a row_b
Vertauscht den Inhalt der beiden angegebenen Zeilen.

***aMatrix* transpose**
Transponiert den Inhalt der Matrix, vertauscht also Zeilen gegen Spalten und umgekehrt.

aMatrix* unlink *arrayvar

Hebt die Verknüpfung zwischen der Matrix und der angegebenen Array-Variable auf, falls eine besteht.

Beispiele

Die Beispiele unten setzen eine Matrix M von 5x5 voraus, deren erste Zeile die Werte 1 bis 5 enthält, mit 1 in der Zelle links oben. Jede weitere Zeile enthält den Inhalt der Zeile darüber, um eine Zelle nach rechts gedreht.

```
% M get rect 0 0 4 4
{{1 2 3 4 5} {5 1 2 3 4} {4 5 1 2 3} {3 4 5 1 2} {2 3 4 5 1}}
```

```
% M set rect 1 1 {{0 0 0} {0 0 0} {0 0 0}}
% M get rect 0 0 4 4
{{1 2 3 4 5} {5 0 0 0 4} {4 0 0 0 3} {3 0 0 0 2} {2 3 4 5 1}}
```

Sind die Stildefinitionen aus dem Beispielabschnitt der Manpage zum Paket **report** jetzt in den Interpreter geladen, folgt ein Beispiel, das eine Matrix als tabellarischen Bericht formatiert. Der Code, der die Matrix mit Daten füllt, ist nicht gezeigt. enthält nützliche Daten.

```
    % ::struct::matrix m
    % # ... fill m with data, assume 5 columns
    % ::report::report r 5 style captionedtable 1
    % m format 2string r
    +---+------------------+-------+-------+--------+
    |000|VERSIONS:         |2:8.4a3|1:8.4a3|1:8.4a3%|
    +---+------------------+-------+-------+--------+
    |001|CATCH return ok   |7      |13     |53.85   |
    |002|CATCH return error|68     |91     |74.73   |
    |003|CATCH no catch used|7     |14     |50.00   |
    |004|IF if true numeric|12     |33     |36.36   |
    |005|IF elseif         |15     |47     |31.91   |
    |   |true numeric      |       |       |        |
    +---+------------------+-------+-------+--------+
    %
    % # alternate way of doing the above
    % r printmatrix m
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: matrix* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Datenstrukturen

Copyright

```
Copyright (c) 2002-2013,2019,2022 Andreas Kupries
    <andreas.kupries@gmail.com>
```


pool
Stichwörter: pool, discrete items, finite, struct

Name
struct::pool – Pool-Objekte (aus einzelnen Elementen) anlegen und bearbeiten

Übersicht
package require Tcl 8.5 9
package require struct::pool ?**1.2.4**?
::struct::pool ?*poolName*? ?*maxsize*?
poolName *option* ?*arg arg* ...?
poolName **add** *itemName1* ?*itemName2 itemName3* ...?
poolName **clear** ?**-force**?
poolName **destroy** ?**-force**?
poolName **info** *type* ?*arg*?
poolName **maxsize** ?*maxsize*?
poolName **release** *itemName*
poolName **remove** *itemName* ?**-force**?
poolName **request** itemVar ?*options*?

Beschreibung
Dieses Paket stellt Pool-Objekte bereit, mit denen sich endliche Sammlungen diskreter Elemente verwalten lassen.

::struct::pool ?*poolName*? ?*maxsize*?
Erstellt ein neues Pool-Objekt. Wird kein *poolName* angegeben, erhält der neue Pool den Namen "pool**X**", wobei X eine positive ganze Zahl ist. Das optionale zweite Argument *maxsize* muss eine positive ganze Zahl sein, die die maximale Größe des Pools angibt; dies ist die maximale Anzahl von Elementen, die der Pool enthalten darf. Der Standardwert hierfür lautet **10**.

Dem Pool-Objekt ist ein globaler Tcl-Befehl zugeordnet, dessen Name *poolName* lautet. Mit diesem Befehl lassen sich verschiedene Konfigurationsoperationen auf dem Pool aufrufen. Er hat die folgende allgemeine Form:

poolName *option* ?*arg arg ...*?
Option und die *arg*s bestimmen das genaue Verhalten des Befehls. Eine detaillierte Auflistung der Optionen und ihrer Funktionsweise finden Sie im Abschnitt Pool-Objektbefehl .

Pools und Zuteilung

Der Befehl pool und der von ihm erzeugte Pool-Objektbefehl dienen dazu, Pools aus einzelnen Elementen zu verwalten. Beispiele für einen Pool aus einzelnen Elementen sind:

- die Plätze in einem Kino, Theater, Zug usw., die Besucher bzw. Reisende reservieren können;
- die dynamischen IP-Adressen, die ein Internetdienstanbieter an seine Kunden vergeben kann;
- die Fahrzeugflotte eines Autovermieters, die von Kunden gemietet werden kann;
- die Klassenräume in einem Schulgebäude, die eingeplant werden müssen;
- die Datenbankverbindungen, die den Client-Threads einer Webserver-Anwendung zur Verfügung stehen;
- die Bücher in einer Bibliothek, die die Kunden ausleihen können;
- usw. …

Der gemeinsame Nenner dieser Beispiele besteht darin, dass es eine mehr oder weniger feste Anzahl von Elementen (Sitzplätze, IP-Adressen, Autos, …) gibt, die mehr oder weniger regelmäßig zugewiesen werden sollen. Ein Element kann jeweils nur einmal zugewiesen sein. Ein zugewiesenes Element muss freigegeben werden, bevor es erneut zugewiesen werden kann. Während laufend Elemente eines Pools zugewiesen und freigegeben werden, bleibt die Gesamtzahl der Elemente im Pool konstant.

Festzuhalten, welche Elemente zugewiesen sind und an wen, ist der Zweck des Befehls pool und der ihm untergeordneten Befehle.

Pool-Fachsprache: Wenn wir sagen, dass ein Element *zugewiesen* ist, bedeutet dies, dass das Element *belegt*, *im Besitz* oder *besetzt* ist; es steht nicht mehr zur Verfügung. Wenn ein Element *frei* ist, ist es *verfügbar*. Ein Element freizugeben (deallocate) heißt dasselbe wie es freizusetzen oder zurückzugeben. Von der Person oder Stelle, der das Element zugeteilt wurde, sagt man, sie besitze das Element.

Elemente

Diskrete Elemente

Der Befehl **pool** ist *ausschließlich für diskrete Elemente* vorgesehen. Beachten Sie, dass es Pools gibt, bei denen die Zuweisung auf nicht-diskreter Basis erfolgt, beispielsweise beim Computerspeicher. Es gibt zudem Pools, aus denen die zugeteilten Anteile voraussichtlich nicht zurückgegeben werden, wie beispielsweise ein Wohltätigkeitsfonds oder eine Suppenschüssel, aus der Sie eine Portion erhalten können. Schließlich gibt es sogar Pools, aus denen niemals etwas zugeteilt oder zurückgegeben wird, wie beispielsweise ein Schwimmbecken oder eine Klärgrube.

Eindeutige Elementnamen

Ein Pool kann keine doppelten Namen für Elemente verarbeiten. Daher müssen die Elemente in einem Pool eindeutige Namen haben.

Gleichwertigkeit der Elemente

Aus Sicht des Verwalters eines Pools sind die Elemente gleichwertig. Dem Verwalter eines Pools ist es gleichgültig, welche Entität bzw. Person ein bestimmtes Element belegt. Kunden können jedoch aufgrund einer ihnen bekannten Eigenschaft des Elements Präferenzen für ein bestimmtes Element haben.

Präferenzen

Ein künftiger Besitzer bevorzugt möglicherweise ein bestimmtes Element. Eine Zuweisung nach

Präferenz wird unterstützt (siehe die Option **-prefer** des Unterbefehls request). Eine Präferenz für ein bestimmtes Element ergibt sich meist daraus, dass sich die Merkmale der Elemente unterscheiden. Beachten Sie, dass die Pool-Befehle selbst nicht dafür ausgelegt sind, solche Elementeigenschaften zu verwalten. Wenn Elementeigenschaften in einer Anwendung eine Rolle spielen, sollten sie separat verwaltet werden.

Pool-Objektbefehl

Die folgenden Unterbefehle und die dazugehörigen Argumente stehen für jeden Pool-Objektbefehl zur Verfügung.

***poolName* add *itemName1* ?*itemName2 itemName3 ...*?**

Dieser Befehl fügt die in der Befehlszeile angegebenen Elemente dem Pool hinzu. Sollten in der Befehlszeile doppelte Elementnamen vorkommen, wird ein Fehler ausgelöst. Sollten eines oder mehrere der Elemente bereits im Pool vorhanden sein, wird dies ebenfalls als Fehler gewertet.

***poolName* clear ?-force?**

Entfernt alle Elemente aus dem Pool. Sollten zum Zeitpunkt der Ausführung des Befehls bereits Elemente zugewiesen sein, wird ein Fehler ausgelöst. Dieses Verhalten kann über das Argument **-force** geändert werden. Wird dieses Argument in der Befehlszeile angegeben, wird der Pool unabhängig vom Zuweisungsstatus seiner Elemente geleert.

***poolName* destroy ?-force?**

Löscht die Pool-Datenstruktur, alle zugehörigen Variablen sowie den zugehörigen Pool-Objektbefehl. Standardmäßig prüft der Befehl, ob noch Elemente zugewiesen sind, und löst in diesem Fall einen Fehler aus. Dieses Verhalten kann über das Argument **-force** geändert werden. Wenn dieses Argument in der Befehlszeile angegeben wird, wird die Pool-Datenstruktur unabhängig vom Zuweisungsstatus ihrer Elemente zerstört.

***poolName* info *type* ?*arg*?**

Gibt verschiedene Informationen zum Pool für die weitere programmatische Verwendung zurück. Das Argument *type* gibt die Art der angeforderten Informationen an. Lediglich der Typ **allocID** verwendet ein zusätzliches Argument.

allocID *itemName*

gibt die allocID des Elements mit dem Namen *itemName* zurück. Freie Elemente haben die Zuweisungs-ID **-1**.

allitems

gibt eine Liste aller Elemente im Pool zurück.

allocstate

Gibt eine Liste von Schlüssel-Wert-Paaren zurück; die Schlüssel sind die Elemente, die Werte die zugehörigen Zuweisungs-IDs. Freie Elemente haben die Zuweisungs-ID **-1**.

cursize

gibt die aktuelle Poolgröße zurück, d. h. die Anzahl der Elemente im Pool.

freeitems

gibt eine Liste von Elementen zurück, die derzeit nicht zugewiesen sind.

maxsize

gibt die maximale Größe des Pools zurück.

***poolName* maxsize ?*maxsize*?**

Legt die maximale Größe des Pools fest oder fragt diese ab, je nachdem, ob das Argument *maxsize* angegeben wird oder nicht. Wird *maxsize* angegeben, wird die maximale Größe des Pools auf diesen Wert gesetzt. Wird kein Argument angegeben, gibt der Befehl die aktuelle maximale Größe des Pools zurück. In dieser Variante ist der Befehl ein Alias für:

poolName info maxsize.

Das Argument *maxsize* muss eine positive ganze Zahl sein.

poolName* release *itemName
Gibt das zuvor zugewiesene Element mit dem Namen *itemName* frei. Es wird ein Fehler ausgelöst, wenn das Element zum Zeitpunkt der Befehlsausführung nicht zugewiesen war.

***poolName* remove *itemName* ?-force?**
Entfernt das Element mit dem Namen *itemName* aus dem Pool. War das Element zum Zeitpunkt der Ausführung des Befehls zugewiesen, wird ein Fehler ausgelöst. Dieses Verhalten kann über das optionale Argument **-force** geändert werden. Wird dieses Argument in der Befehlszeile angegeben, wird das Element unabhängig von seinem Zuweisungsstatus entfernt.

***poolName* request itemVar ?options?**
Bearbeitet eine Anforderung eines Elements und berücksichtigt dabei eine mögliche Präferenz für ein bestimmtes Element. Je nach Verfügbarkeit der Elemente gibt es zwei mögliche Ergebnisse:

1. Die Anforderung wird erfüllt, ein Element wird zugewiesen, und die Variable, deren Name mit dem Argument *itemVar* übergeben wird, wird auf den Namen des zugewiesenen Elements gesetzt. Der Befehl gibt den Wert 1 zurück.
2. Die Anforderung wird abgelehnt. Es wird kein Element zugewiesen. Die Variable mit dem Namen itemVar wird nicht gesetzt. Versuche, *itemVar* zu lesen, können einen Fehler auslösen, wenn die Variable vor der Anforderung nicht definiert war. Der Befehl gibt den Wert 0 zurück.

Die Werte, die dieser Befehl zurückgibt, sind zur Überprüfung vorgesehen. Die folgenden Beispiele zeigen, wie dies geschieht. Wird der Wert nicht überprüft, kann dies zu fehlerhaftem Verhalten führen. Wird über die Option **-prefer** (siehe unten) keine Präferenz für ein bestimmtes Element angegeben, werden alle Anforderungen erfüllt, solange Elemente verfügbar sind.

Folgende Optionen werden unterstützt:

-allocID *allocID*
Wird die Anforderung erfüllt, wird ein Element der durch allocID bezeichneten Stelle zugewiesen. Wird der Zuweisungsstatus eines Elements abgefragt, wird diese Zuweisungs-ID zurückgegeben. Fehlt die Option **-allocID**, wird das Element an **dummyID** vergeben und gehört dann diesem. Zuweisungs-IDs können beliebige Werte sein, außer dem Wert -1, der für freie Elemente reserviert ist.

-prefer *preferredItem*
Diese Option ändert die Zuweisungsstrategie wie folgt: Wenn das Element mit dem Namen *preferredItem* zum Zeitpunkt der Ausführung des Befehls nicht zugewiesen ist, wird die Anforderung erfüllt (Rückgabewert ist 1). Wenn das Element zum Zeitpunkt der Ausführung des Befehls bereits zugewiesen war, wird die Anforderung abgelehnt (Rückgabewert ist 0).

Beispiele

Es werden zwei Beispiele vorgestellt. Das erste Beispiel simuliert eine schrittweise interaktive tclsh-Sitzung, in der jeder Schritt erläutert wird. Das zweite Beispiel veranschaulicht die Verwendung in einer Serveranwendung, die mit einer Backend-Anwendung kommuniziert.

Beispiel 1

Dieses Beispiel zeigt eine interaktive tclsh-Sitzung, die den Fall der Fahrzeugflotte einer Autovermietung behandelt. In zehn Schritten wird die Verwendung chronologisch erläutert, von der Erzeugung des Pools über die wichtigsten Phasen seiner Nutzung bis zur abschließenden Zerstörung.

Randbemerkung:

In diesem Beispiel werden Markennamen zur Kennzeichnung der verschiedenen Elemente verwendet. Ein Markenname könnte jedoch als Eigenschaft eines Elements betrachtet werden. Da der Befehl pool nicht für die Verwaltung von Eigenschaften von Elementen ausgelegt ist, müssen diese separat verwaltet werden. In diesem Fall sollten die Elemente neutralere Namen wie car1, car2, car3 usw. erhalten, und eine separate Datenbank oder ein Array sollte die den Auto-Bezeichnungen zugeordneten Markennamen enthalten.

```
 1. Load the package into an interpreter
 % package require pool
 0.1

 2. Create a pool object called `CarPool' with a maximum size of
55 items (cars):
 % pool CarPool 55
 CarPool

 4. Add items to the pool:
 % CarPool add Toyota Trabant Chrysler1 Chrysler2 Volkswagen

 5. Somebody crashed the Toyota. Remove it from the pool as
follows:
 % CarPool remove Toyota

 6. Acquired a new car for the pool. Add it as follows:
 % CarPool add Nissan

 7. Check whether the pool was adjusted correctly:
 % CarPool info allitems
 Trabant Chrysler1 Chrysler2 Volkswagen Nissan
```

Hier wird die interaktive Sitzung kurz unterbrochen, um die Verwendung des Unterbefehls request in einem Programm zu zeigen:

```
    # Mrs. Swift needs a car. She doesn't have a preference for a
    # particular car. We'll issue a request on her behalf as follows:
    if { [CarPool request car -allocID "Mrs. Swift"] }  {
        # request was honoured, process the variable `car'
        puts "$car has been allocated to [CarPool info allocID $car]."
    } else {
        # request was denied
         puts "No car available."
    }
```

Beachten Sie, wie der Befehl **if** den vom Unterbefehl **request** zurückgegebenen Wert verwendet.

```
    # Suppose Mr. Wiggly has a preference for the Trabant:
    if { [CarPool request car -allocID "Mr. Wiggly" -prefer Trabant]
   }  {
        # request was honoured, process the variable `car'
        puts "$car has been allocated to [CarPool info allocID $car]."
    } else {
        # request was denied
         puts "The Trabant was not available."
    }
```

Weiter mit der interaktiven Sitzung:

```
    8. When the car is returned then you can render it available by:
    % CarPool release Trabant

    9. When done, you delete the pool.
    % CarPool destroy
    Couldn't destroy `CarPool' because some items are still allocated.

    Oops, forgot that Mrs. Swift still occupies a car.

    10. We force the destruction of the pool as follows:
    % CarPool destroy -force
```

Beispiel 2

Dieses Beispiel beschreibt den Fall, aus dem sich der Bedarf des Autors an einer Poolverwaltung ergab. Es handelt sich um ein Beispiel für eine Serveranwendung, die Anfragen von Client-Anwendungen entgegennimmt. Die Client-Anfragen werden an eine Backend-Anwendung weitergereicht, bevor das Ergebnis an die Client-Anwendung zurückgeht. In vielen Fällen gibt es mehrere gleichwertige Instanzen von Backend-Anwendungen, an die eine Client-Anfrage weitergeleitet werden kann. Die Dateideskriptoren, die die Kanäle zu diesen Backend-Instanzen identifizieren, bilden einen Verbindungspool. Eine bestimmte Verbindung kann jeweils nur einer einzigen Client-Anfrage zugewiesen werden.

```
    # Create the pool of connections (pipes)
    set maxpipes 10
    pool Pipes $maxpipes
    for {set i 0} {$i < $maxpipes} {incr i} {
        set fd [open "|backendApplication" w+]
        Pipes add $fd
    }

    # A client request comes in. The request is identified as
   `clientX'.
    # Dispatch it onto an instance of a back-end application
    if { [Pipes request fd -allocID clientX] } {
        # a connection was allocated
        # communicate to the back-end application via the variable
   `fd'
        puts $fd "someInstruction"
        # ...... etc.
    } else {
        # all connections are currently occupied
        # store the client request in a queue for later processing,
        # or return a 'Server busy' message to the client.
    }
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: pool* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright

Copyright (c) 2002, Erik Leunissen <e.leunissen@hccnet.nl>

prioqueue

Stichwörter: prioqueue, ordered list, priority queue

Name

struct::prioqueue – Prioqueue-Objekte anlegen und bearbeiten

Übersicht

package require Tcl 8.5 9
package require struct::prioqueue ?**1.5**?
::struct::prioqueue ?**-ascii|-dictionary|-integer|-real**? ?*prioqueueName*?
prioqueueName **option** ?*arg arg* ...?
prioqueueName **clear**
prioqueueName **remove** *item*
prioqueueName **destroy**
prioqueueName **get** ?*count*?
prioqueueName **peek** ?*count*?
prioqueueName **peekpriority** ?*count*?
prioqueueName **put** *item prio* ?*item prio* ...?
prioqueueName **size**

Beschreibung

Dieses Paket implementiert eine einfache Prioritätswarteschlange unter Verwendung verschachtelter Tcl-Listen.

Der Befehl **::struct::prioqueue** erstellt eine neue Prioritätswarteschlange mit dem Standard-Prioritätsschlüsseltyp *-integer*. Das bedeutet, dass Schlüssel, die dem Unterbefehl **put** übergeben werden, diesen Typ aufweisen müssen.

Dadurch wird auch die Prioritätsreihenfolge festgelegt. Bei den Schlüsseltypen *-ascii* und *-dictionary* werden die Daten in aufsteigender Reihenfolge sortiert (wie bei **lsort** *-increasing*), während bei *-integer* und *-real* die Daten in absteigender Reihenfolge sortiert werden (wie bei **lsort** *-decreasing*).

Die Namen der Prioqueues unterliegen keinen Einschränkungen, können jedoch als Optionen interpretiert werden, wenn kein Prioritätstyp angegeben wird.

::struct::prioqueue ?-ascii|-dictionary|-integer|-real? ?*prioqueueName*?
Der Befehl **::struct::prioqueue** erzeugt ein neues Prioqueue-Objekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *prioqueueName* lautet. Mit diesem Befehl lassen sich verschiedene Operationen auf der Prioqueue aufrufen. Er hat die folgende allgemeine Form:

***prioqueueName* option ?*arg arg ...*?**
option und die *arg*s legen das genaue Verhalten des Befehls fest. Für Prioqueue-Objekte sind folgende Befehle möglich:

***prioqueueName* clear**
Entfernt alle Elemente aus der Prioqueue.

prioqueueName* remove *item
Entfernt das angegebene Element aus dieser Prioritätswarteschlange.

***prioqueueName* destroy**
Zerstört die Prioqueue einschließlich ihres Speichers und des zugehörigen Befehls.

***prioqueueName* get ?*count*?**
Gibt die ersten *count* Elemente der Prioritätswarteschlange zurück (jedoch nicht deren Prioritäten) und entfernt sie aus der Prioritätswarteschlange. Wird *count* nicht angegeben, beträgt der Standardwert 1. Ist *count* gleich 1, ist das Ergebnis eine einfache Zeichenkette; andernfalls ist es eine Liste. Falls angegeben, muss es größer oder gleich 1 sein. Befinden sich keine oder zu wenige Elemente in der Prioritätswarteschlange, löst dieser Befehl einen Fehler aus.

***prioqueueName* peek ?*count*?**
Gibt die ersten *count* Elemente der Prioritätswarteschlange zurück (jedoch nicht deren Prioritäten), ohne sie aus der Prioritätswarteschlange zu entfernen. Wird *count* nicht angegeben, beträgt der Standardwert 1. Ist *count* gleich 1, ist das Ergebnis eine einfache Zeichenkette; andernfalls ist es eine Liste. Falls angegeben, muss *count* größer oder gleich 1 sein. Befinden sich keine oder zu wenige Elemente in der Warteschlange, löst dieser Befehl einen Fehler aus.

***prioqueueName* peekpriority ?*count*?**
Gibt die Prioritätsschlüssel der ersten *count* Elemente zurück, ohne die Elemente aus der Prioritätswarteschlange zu entfernen. Wird *count* nicht angegeben, beträgt der Standardwert 1. Ist *count* gleich 1, ist das Ergebnis eine einfache Zeichenkette; andernfalls ist es eine Liste. Falls angegeben, muss *count* größer oder gleich 1 sein. Befinden sich keine oder zu wenige Elemente in der Warteschlange, löst dieser Befehl einen Fehler aus.

***prioqueueName* put *item prio* ?*item prio ...*?**
Fügt das angegebene *item* oder die angegebenen Elemente in die Prioqueue ein. *prio* muss ein gültiger Prioritätsschlüssel für diesen Prioqueue-Typ sein; andernfalls wird ein Fehler ausgelöst und es wird kein Element hinzugefügt. Elemente werden nach ihrem Prioritätsrang eingefügt. Elemente gleicher Priorität bleiben in der Reihenfolge, in der sie hinzugefügt wurden.

***prioqueueName* size**
Gibt die Anzahl der Elemente in der Prioritätswarteschlange zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: prioqueue* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright

```
Copyright (c) 2003 Michael Schlenker <mic42@users.sourceforge.net>
```

queue

Stichwörter: queue, graph, list, matrix, pool, prioqueue, record, set, skiplist, stack, tree

Name

struct::queue – Warteschlangenobjekte anlegen und bearbeiten

Übersicht

package require Tcl 8.5 9

package require struct::queue ?**1.4.6**?
queueName **option** ?*arg arg* ...?
queueName **clear**
queueName **destroy**
queueName **get** ?*count*?
queueName **peek** ?*count*?
queueName **put** *item* ?*item* ...?
queueName **unget** *item*
queueName **size**

Beschreibung

Der Namespace **::struct** enthält Befehle zur Verarbeitung endlicher Warteschlangen.

Er exportiert einen einzigen Befehl: **::struct::queue**. Alle hier bereitgestellten Funktionen sind über einen Unterbefehl dieses Befehls aufrufbar.

Hinweis: Ab Version 1.4.1 dieses Pakets steht eine auf critcl basierende C-Implementierung zur Verfügung. Diese Implementierung erfordert jedoch Tcl 8.4.

Der Befehl **::struct::queue** erzeugt ein neues Warteschlangenobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *queueName* lautet. Mit diesem Befehl lassen sich verschiedene Operationen auf der Warteschlange aufrufen. Er hat die folgende allgemeine Form:

***queueName* option ?*arg arg ...*?**
Option und die *arg*s legen das genaue Verhalten des Befehls fest. Für Warteschlangenobjekte sind folgende Befehle möglich:

***queueName* clear**
Entfernt alle Elemente aus der Warteschlange.

***queueName* destroy**
Zerstört die Warteschlange einschließlich ihres Speichers und des zugehörigen Befehls.

***queueName* get ?*count*?**
Gibt die ersten *count* Elemente der Warteschlange zurück und entfernt sie aus der Warteschlange. Wird *count* nicht angegeben, beträgt der Standardwert 1. Ist *count* gleich 1, ist das Ergebnis eine einfache Zeichenkette; andernfalls ist es eine Liste. Falls angegeben, muss es größer oder gleich 1 sein. Befinden sich nicht genügend Elemente in der Warteschlange, um die Anforderung zu erfüllen, löst dieser Befehl einen Fehler aus.

***queueName* peek ?*count*?**
Gibt die ersten *count* Elemente der Warteschlange zurück, ohne sie aus der Warteschlange zu entfernen. Wird *count* nicht angegeben, beträgt der Standardwert 1. Ist *count* gleich 1, ist das Ergebnis eine einfache Zeichenkette; andernfalls ist es eine Liste. Falls angegeben, muss es größer oder gleich 1 sein. Befinden sich nicht genügend Elemente in der Warteschlange, um die Anforderung zu erfüllen, löst dieser Befehl einen Fehler aus.

***queueName* put *item* ?*item ...*?**
Fügt das angegebene *item* oder die angegebenen Elemente in die Warteschlange ein. Wird mehr als ein *item* angegeben, werden die Elemente in der Reihenfolge hinzugefügt, in der sie aufgeführt sind.

queueName* unget *item
Fügt das *item* am Anfang der Warteschlange ein, d. h. vor alle anderen Elemente, die sich

bereits in der Warteschlange befinden. Damit bildet dieser Vorgang das Gegenstück zur Methode **get**.

***queueName* size**
Gibt die Anzahl der Elemente in der Warteschlange zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: queue* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

record

Stichwörter: record, data structures, struct

Name

struct::record – Datensätze definieren und anlegen (ähnlich wie "C"-Strukturen)

Übersicht

package require Tcl 8.5 9
package require struct::record ?**1.2.4**?
record define *recordName recordMembers* ?*instanceName1 instanceName2* ...?
record show *record*
record show *instances recordName*
record show *members recordName*
record show *values instanceName*
record exists *record recordName*
record exists *instance instanceName*
record delete *record recordName*
record delete *instance instanceName*
instanceName **cget** -member
instanceName **cget** -member1 -member2
instanceName **cget**
instanceName **configure**
instanceName
instanceName **configure** -member *value*
instanceName **configure** -member1 *value1* -member2 *value2*
recordName **instanceName|#auto** ?*-member1 value1 -member2 value2* ...?
instanceName **cget** ?*-member1 -member2* ...?
instanceName **configure** ?*-member1 value1 -member2 value2* ...?

Beschreibung

Das Paket **::struct::record** bietet eine Möglichkeit, Variablen zu einer einzigen Datenstruktur zusammenzufassen, ähnlich einer *C*-Struktur. Die Elemente eines Datensatzes können Variablen oder andere Datensätze sein. Ein Datensatz darf jedoch keine zirkulären Datensätze enthalten, d. h. Datensätze, die denselben Datensatz als Element enthalten.

Dieses Paket wurde so strukturiert, dass es der Funktionsweise von Tk-Objekten sehr ähnlich ist. Jede Datensatzdefinition erzeugt ein Datensatzobjekt, das diese Definition kapselt. Anschließend kann dieses Datensatzobjekt Instanzen dieses Datensatzes erstellen. Diese Instanzen können dann mit den Methoden **cget** und **configure** bearbeitet werden.

Das Paket enthält nur einen Befehl auf oberster Ebene, jedoch mehrere Unterbefehle (siehe unten). Es beachtet außerdem den Namespace, in dem der Datensatz definiert wurde; daher sind die zurückgegebenen Objekte vollqualifiziert.

Hinweis: Wer Tcl 9 verwendet, findet die eingebaute objektorientierte Funktionalität von Tcl möglicherweise bequemer und vielseitiger als dieses Paket.

record define *recordName recordMembers ?instanceName1 instanceName2 ...?*
: Definiert einen Datensatz. *recordName* ist der Name des Datensatzes und wird zudem als Objektbefehl verwendet. Dieser Objektbefehl dient dazu, Instanzen der Datensatzdefinition zu erstellen. Die *recordMembers* sind die Elemente des Datensatzes, aus denen sich die Datensatzdefinition zusammensetzt. Dabei handelt es sich um Variablen und andere Datensätze. Werden optionale *instanceName*-Argumente angegeben, wird nach der Erstellung der Definition für jeden *instanceName* eine Instanz generiert.

record show *record*
: Gibt eine Liste der definierten Datensätze zurück.

record show *instances recordName*
: Gibt die Instanzen zurück, die von *recordName* instanziiert wurden.

record show *members recordName*
: Gibt die für den Datensatz *recordName* definierten Elemente zurück. Das Format entspricht dem Format, in dem die Datensätze definiert wurden.

record show *values instanceName*
: Gibt eine Liste der Werte zurück, die für die Instanz *instanceName* festgelegt sind. Die Ausgabe besteht aus einer Liste von Schlüssel-Wert-Paaren. Falls verschachtelte Datensätze vorhanden sind, bilden die Werte dieser verschachtelten Datensätze ihrerseits eine Liste.

record exists *record recordName*
: Prüft, ob ein Datensatz (*record*) mit dem Namen *recordName* existiert.

record exists *instance instanceName*
: Prüft, ob eine Instanz (*instance*) mit dem Namen *instanceName* existiert.

record delete *record recordName*
: Löscht *recordName* sowie alle Instanzen von *recordName*. Existiert der Datensatz nicht, wird ein Fehler zurückgegeben.

record delete *instance instanceName*
: Löscht die *instance* mit dem Namen *instanceName*. Existiert die Instanz nicht, wird ein Fehler zurückgegeben. Beachten Sie, dass dabei auch alle verschachtelten Instanzen rekursiv gelöscht werden.

Datensatzelemente

Die Elemente eines Datensatzes können entweder Variablen oder andere Datensätze sein. Ein und derselbe Datensatz darf jedoch nicht in sich selbst verschachtelt werden (zyklisch). Um einen verschachtelten Datensatz zu definieren, müssen Sie das Schlüsselwort **record** zusammen mit dem Namen des Datensatzes und dem Namen der Instanz dieses verschachtelten Datensatzes (innerhalb des Containers) angeben. Das sieht zum Beispiel so aus:

```
# this is the nested record
record define mynestedrecord {
    nest1
    nest2
}

# This is the main record
record define myrecord {
    mem1
    mem2
    {record mynestedrecord mem3}
}
```

Sie können den Elementen eines Datensatzes auch Standard- oder Anfangswerte zuweisen, indem Sie den Eintrag für das jeweilige Element in geschweifte Klammern setzen:

```
record define myrecord {
    mem1
    {mem2 5}
}
```

Alle Instanzen, die anhand dieser Datensatzdefinition angelegt werden, haben anfangs den Wert **5** für das Element *mem2*. Ist kein Standardwert angegeben, ist der Wert die leere Zeichenkette.

Werte abrufen

Um den Wert eines Elements abzurufen, gibt es mehrere Möglichkeiten.

instanceName* cget *-member

In dieser Form gibt die eingebaute Instanzmethode **cget** den Wert des angegebenen *member* zurück. Beachten Sie den führenden Bindestrich.

Um auf ein verschachteltes Element zuzugreifen, verwenden Sie die *Punktnotation*:

```
instanceName cget -mem3.nest1
```

instanceName* cget *-member1 -member2

In dieser Form gibt die eingebaute Instanzmethode **cget** eine Liste zurück, die die Werte beider angegebener Elemente in der Reihenfolge ihrer Angabe enthält.

***instanceName* cget**

siehe unten ...

***instanceName* configure**

siehe unten ...

instanceName

Diese Formen sind alle gleichwertig. Sie geben ein Dictionary zurück, das alle Elemente und die zugehörigen Werte enthält.

Werte festlegen

Um den Wert eines Elements festzulegen, gibt es mehrere Möglichkeiten.

instanceName* configure *-member value

In dieser Form setzt die eingebaute Instanzmethode **configure** das angegebene *member* auf den angegebenen *value*. Beachten Sie den führenden Bindestrich.

Um auf ein verschachteltes Element zuzugreifen, verwenden Sie die *Punktnotation*:

```
instanceName configure -mem3.nest1 value
```

instanceName* configure *-member1 value1 -member2 value2

In dieser Form setzt die eingebaute Instanzmethode **configure** alle angegebenen Elemente auf die zugehörigen Werte.

Zugriff über Alias

In der ursprünglichen Implementierung erfolgte der Zugriff mithilfe der Punktnotation, ähnlich wie beim Zugriff auf *C*-Strukturen. Es herrschte jedoch Einigkeit darüber, die Schnittstelle stärker an Tcl anzulehnen, was durchaus sinnvoll war. Der ursprüngliche Alias-Zugriff ist jedoch weiterhin vorhanden. Für manche könnte er sich als hilfreich erweisen.

Grundsätzlich wird für jedes Element jeder Instanz ein Alias erstellt. Dieser Alias dient dazu, Werte für dieses Element abzurufen und festzulegen. Ein Beispiel mit den oben definierten Datensätzen verdeutlicht das:

```
% # Create an instance first
% myrecord inst1
::inst1

% # To get a member of an instance, just use the alias. It behaves
% # like a Tcl command:
% inst1.mem1

% # To set a member via the alias, just include a value. And optionally
% # the equal sign - syntactic sugar.
% inst1.mem1 = 5
5

% inst1.mem1
5

% # For nested records, just continue with the dot notation.
% # note, no equal sign.
% inst1.mem3.nest1 10
10
```

```
% inst1.mem3.nest1
10

% # just the instance by itself gives all member/values pairs for that
% # instance
% inst1
-mem1 5 -mem2 {} -mem3 {-nest1 10 -nest2 {}}

% # and to get all members within the nested record
% inst1.mem3
-nest1 10 -nest2 {}
```

Der Befehl record

Für jeden Datensatzbefehl stehen die folgenden Unterbefehle und die zugehörigen Argumente zur Verfügung:

***recordName instanceName*|#auto ?*-member1 value1 -member2 value2 ...*?**

Mithilfe des Objektbefehls *recordName*, der aus der Datensatzdefinition erstellt wurde, können Instanzen der Datensatzdefinition angelegt werden. Sobald eine Instanz angelegt ist, erbt sie die Elemente der Datensatzdefinition – ganz ähnlich wie bei Objekten. Beim Anlegen der Instanz wird außerdem ein Objektbefehl für die Instanz mit dem Namen *instanceName* erzeugt.

Dieser Objektbefehl dient dem Zugriff auf die Datenelemente der Instanz. Während der Instanziierung können zwar Werte für diese Instanz angegeben werden; am Ende müssen jedoch *alle* Werte angegeben werden, und zwar als Schlüssel-Wert-Paare, wie bei der Methode **configure**. Verschachtelte Datensätze müssen im Listenformat vorliegen.

Anstelle von *instanceName* kann auch **#auto** angegeben werden. Wird **#auto** verwendet, wird der Name der Instanz automatisch erzeugt und hat die Form **recordNameN**, wobei **N** eine erzeugte eindeutige ganze Zahl ist (beginnend bei 0).

Instanzbefehl

Die folgenden Unterbefehle und die dazugehörigen Argumente stehen für jeden Instanzbefehl eines Datensatzes zur Verfügung:

***instanceName* cget ?*-member1 -member2 ...*?**

Jede Instanz verfügt über die Methode **cget**. Sie funktioniert sehr ähnlich wie der Befehl **cget** von Tk-Widgets. Sie fragt die Werte der Elemente dieser bestimmten Instanz ab. Werden keine Argumente angegeben, gibt sie ein Dictionary zurück.

***instanceName* configure ?*-member1 value1 -member2 value2 ...*?**

Jede Instanz verfügt über die Methode **configure**. Sie funktioniert sehr ähnlich wie der Befehl **configure** von Tk-Widgets. Sie setzt die Werte der angegebenen Elemente dieser Instanz. Werden keine Argumente angegeben, gibt sie eine Dictionary-Liste zurück.

Beispiele

Zwei Beispiele zeigen anschaulich, wie dieses Paket verwendet wird.

Beispiel 1 – Kontaktdaten

Das wohl offensichtlichste Beispiel wäre die Speicherung von Kontaktdaten wie Adressen, Telefonnummern, Anmerkungen usw. Da eine Person mehrere Telefonnummern, mehrere E-Mail-Adressen usw. haben kann, werden wir diese mithilfe verschachtelter Datensätze definieren. Als Erstes definieren wir also die verschachtelten Datensätze:

```
##
##  This is an interactive example, to see what is returned by
##  each command as well.
##

% namespace import ::struct::record::*

% # define a nested record. Notice that country has default 'USA'.
% record define locations {
    street
    street2
    city
    state
    zipcode
    {country USA}
    phone
}
::locations
% # Define the main record. Notice that it uses the location record
    twice.
% record define contacts {
    first
    middle
    last
    {record locations home}
    {record locations work}
}
::contacts
% # Create an instance for the contacts record.
% contacts cont1
::cont1
% # Display some introspection values
% record show records
::contacts ::locations
% #
% record show values cont1
-first {} -middle {} -last {} -home {-street {} -street2 {} -city {}
    -state {} -zipcode {} -country USA -phone {}} -work {-street {}
    -street2 {} -city {} -state {} -zipcode {} -country USA -phone {}}
% #
```

```
% record show instances contacts
::cont1
% #
% cont1 config
-first {} -middle {} -last {} -home {-street {} -street2 {} -city {}
    -state {} -zipcode {} -country USA -phone {}} -work {-street {}
    -street2 {} -city {} -state {} -zipcode {} -country USA -phone {}}
% #
% cont1 cget
-first {} -middle {} -last {} -home {-street {} -street2 {} -city {}
    -state {} -zipcode {} -country USA -phone {}} -work {-street {}
    -street2 {} -city {} -state {} -zipcode {} -country USA -phone {}}
% # copy one record to another record
% record define contacts2 [record show members contacts]
::contacts2
% record show members contacts2
first middle last {record locations home} {record locations work}
% record show members contacts
first middle last {record locations home} {record locations work}
%
```

Beispiel 2 – verkettete Liste

Das folgende Beispiel veranschaulicht lediglich eine einfache verkettete Liste

```
% # define a very simple record for linked list
% record define linkedlist {
    value
    next
}
::linkedlist
% linkedlist lstart
::lstart
% lstart config -value 1 -next [linkedlist #auto]
% [lstart cget -next] config -value 2 -next [linkedlist #auto]
% [[lstart cget -next] cget -next] config -value 3 -next "end"
% set next lstart
lstart
% while 1 {
    lappend values [$next cget -value]
    set next [$next cget -next]
    if {[string match "end" $next]} break
}
% puts "$values"
1 2 3
% # cleanup linked list
% # We could just use delete record linkedlist also
```

```
% foreach I [record show instances linkedlist] {
    record delete instance $I
}
% record show instances linkedlist
%
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: record* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright

Copyright (c) 2002, Brett Schwarz <brett_schwarz@yahoo.com>

skiplist

Stichwörter: skiplist

Name

struct::skiplist – Skiplisten anlegen und bearbeiten

Übersicht

package require Tcl 8.5 9
package require struct::skiplist ?**1.4**?
skiplistName *option* ?*arg arg* ...?
skiplistName **delete** *node* ?*node...*?
skiplistName **destroy**
skiplistName **insert** *key value*
skiplistName **search** *node* ?*-key key*?
skiplistName **size**
skiplistName **walk** *cmd*

Beschreibung

Der Befehl **::struct::skiplist** erzeugt ein neues Skiplist-Objekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *skiplistName* lautet. Mit diesem Befehl lassen sich verschiedene Operationen auf der Skiplist aufrufen. Er hat die folgende allgemeine Form:

skiplistName *option* ?*arg arg ...*?
Option und die *args* bestimmen das genaue Verhalten des Befehls.

Skiplisten sind eine Alternative zu Binärbäumen als Datenstruktur. Mit ihnen lassen sich

geordnete Listen über beliebige Folgen von Einfügungen und Löschungen hinweg pflegen. Skiplisten erreichen durch Zufall eine probabilistische Balancierung; daher sind die Algorithmen zum Einfügen und Löschen in Skiplisten wesentlich einfacher und schneller als die für Binärbäume.

Mehr zu Skiplisten finden Sie bei Pugh, William. *Skip lists: a probabilistic alternative to balanced trees* In: Communications of the ACM, Juni 1990, 33(6) 668–676.

Derzeit kann der Schlüssel entweder eine Zahl oder eine Zeichenkette sein, und Vergleiche werden mit dem eingebauten Größer-als-Operator durchgeführt. Für Skiplist-Objekte sind folgende Befehle möglich:

***skiplistName* delete *node* ?*node*...?**
Entfernt die angegebenen Knoten aus der Skiplist.

***skiplistName* destroy**
Zerstört die Skiplist einschließlich ihres Speichers und des zugehörigen Befehls.

skiplistName* insert *key value
Fügt einen Knoten mit dem angegebenen *key* und *value* in die Skiplist ein. Gibt es bereits einen Knoten mit diesem Schlüssel, wird dessen Wert aktualisiert und seine Knotenebene zurückgegeben. Andernfalls wird ein neuer Knoten angelegt und 0 zurückgegeben.

***skiplistName* search *node* ?-key *key*?**
Sucht einen bestimmten Schlüssel in der Skiplist. Wird er nicht gefunden, wird 0 zurückgegeben. Wird er gefunden, wird eine Liste aus zwei Elementen zurückgegeben: 1, gefolgt vom Wert des Knotens.

***skiplistName* size**
Gibt die Anzahl der Knoten in der Skiplist zurück.

skiplistName* walk *cmd
Durchläuft die Skiplist vom ersten Knoten bis zum letzten. An jedem Knoten wird der Befehl *cmd* ausgewertet, mit Schlüssel und Wert des aktuellen Knotens als angehängten Argumenten.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: skiplist* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright

```
Copyright (c) 2000 Keith Vetter
```

stack

Stichwörter: stack, graph, matrix, queue, tree

Name

struct::stack – Stapelobjekte anlegen und bearbeiten

Übersicht

package require Tcl 8.5 9
package require struct::stack ?**1.5.4**?
stackName **option** ?*arg arg* ...?
stackName **clear**
stackName **destroy**
stackName **get**
stackName **getr**
stackName **peek** ?*count*?
stackName **peekr** ?*count*?
stackName **trim** ?*newsize*?
stackName **trim*** ?*newsize*?
stackName **pop** ?*count*?
stackName **push** *item* ?*item...*?
stackName **size**

Beschreibung

Der Namespace **::struct** enthält Befehle zur Verarbeitung endlicher Stapel.

Er exportiert einen einzigen Befehl: **::struct::stack**. Alle hier bereitgestellten Funktionen sind über einen Unterbefehl dieses Befehls aufrufbar.

Hinweis: Ab Version 1.3.3 dieses Pakets steht eine auf critcl basierende C-Implementierung zur Verfügung. Diese Implementierung erfordert jedoch Tcl 8.4.

Der Befehl **::struct::stack** erzeugt ein neues Stapelobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *stackName* lautet. Mit diesem Befehl lassen sich verschiedene Operationen auf dem Stapel aufrufen. Er hat die folgende allgemeine Form:

***stackName* option ?*arg arg ...*?**
Option und die *arg*s legen das genaue Verhalten des Befehls fest. Für Stapelobjekte sind folgende Befehle möglich:

***stackName* clear**
Entfernt alle Elemente vom Stapel.

***stackName* destroy**
Zerstört den Stapel einschließlich seines Speichers und des zugehörigen Befehls.

***stackName* get**
Gibt den gesamten Inhalt des Stapels als Liste zurück, ohne die Elemente vom Stapel zu entfernen.

***stackName* getr**
Eine Variante von **get**, die den Inhalt in umgekehrter Reihenfolge zurückgibt.

***stackName* peek ?*count*?**
Gibt die obersten *count* Elemente des Stapels zurück, ohne sie vom Stapel zu entfernen. Wird *count* nicht angegeben, beträgt der Standardwert 1. Ist *count* gleich 1, ist das Ergebnis eine

einfache Zeichenkette; andernfalls ist es eine Liste. Falls angegeben, muss *count* größer oder gleich 1 sein. Befinden sich nicht genügend Elemente auf dem Stapel, um die Anforderung zu erfüllen, löst dieser Befehl einen Fehler aus.

***stackName* peekr ?*count*?**
Eine Variante von **peek**, die die Elemente in umgekehrter Reihenfolge zurückgibt.

***stackName* trim ?*newsize*?**
Verkleinert den Stapel so, dass er höchstens *newsize* Elemente enthält, und gibt eine Liste zurück, die die entfernten Elemente enthält. Nichts geschieht, wenn der Stapel bereits die angegebene Größe hat oder kleiner ist. In diesem Fall ist das Ergebnis eine leere Liste.

***stackName* trim* ?*newsize*?**
Eine Variante von **trim**, die die Verkleinerung durchführt, die entfernten Elemente jedoch nicht zurückgibt.

***stackName* pop ?*count*?**
Gibt die obersten *count* Elemente des Stapels zurück und entfernt sie vom Stapel. Wird *count* nicht angegeben, beträgt der Standardwert 1. Ist *count* gleich 1, ist das Ergebnis eine einfache Zeichenkette; andernfalls ist es eine Liste. Falls angegeben, muss *count* größer oder gleich 1 sein. Befinden sich nicht genügend Elemente auf dem Stapel, um die Anforderung zu erfüllen, löst dieser Befehl einen Fehler aus.

***stackName* push *item* ?*item...*?**
Legt das angegebene *item* bzw. die angegebenen Elemente auf den Stapel. Wird mehr als ein *item* angegeben, werden die Elemente in der Reihenfolge ihrer Auflistung auf den Stapel gelegt.

***stackName* size**
Gibt die Anzahl der Elemente auf dem Stapel zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: stack* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

struct_list

Stichwörter: struct_list, Fisher-Yates, assign, common, comparison, diff, differential, equal, equality, filter, first permutation, flatten, folding, full outer join, generate permutations, inner join, join, left outer join, list, longest common subsequence, map, next permutation, outer join, permutation, reduce, repeating, repetition, reshuffle, reverse, right outer join, shuffle, subsequence, swapping

Name

struct::list – Prozeduren zur Bearbeitung von Listen

Übersicht

package require Tcl 8.5 9
package require struct::list ?**1.9**?
::struct::list longestCommonSubsequence *sequence1 sequence2* ?*maxOccurs*?
::struct::list longestCommonSubsequence2 *sequence1 sequence2* ?*maxOccurs*?
::struct::list lcsInvert *lcsData len1 len2*
::struct::list lcsInvert2 *lcs1 lcs2 len1 len2*
::struct::list lcsInvertMerge *lcsData len1 len2*
::struct::list lcsInvertMerge2 *lcs1 lcs2 len1 len2*
::struct::list reverse *sequence*
::struct::list shuffle *list*
::struct::list assign *sequence varname* ?*varname?...*?
::struct::list flatten ?**-full**? ?**--**? *sequence*
::struct::list map *sequence cmdprefix*
::struct::list mapfor *var sequence script*
::struct::list filter *sequence cmdprefix*
::struct::list filterfor *var sequence expr*
::struct::list split *sequence cmdprefix* ?*passVar failVar*?
::struct::list fold *sequence initialvalue cmdprefix*
::struct::list shift *listvar*
::struct::list iota *n*
::struct::list equal ?**-simple**? ?**--**? *a b*
::struct::list repeat *size element1* ?*element2 element3...*?
::struct::list repeatn *value size...*
::struct::list dbJoin ?**-inner|-left|-right|-full**? ?*-keys varname*? {keycol table}...
::struct::list dbJoinKeyed ?**-inner|-left|-right|-full**? ?*-keys varname*? *table...*
::struct::list swap *listvar i j*
::struct::list firstperm *list*
::struct::list nextperm *perm*
::struct::list permutations *list*
::struct::list foreachperm *var list body*

Beschreibung

Der Namespace **::struct::list** enthält mehrere nützliche Befehle zur Verarbeitung von Tcl-Listen. Im Allgemeinen implementieren diese Befehle Algorithmen, die komplexer oder spezialisierter sind als die von Tcl selbst bereitgestellten.

Er exportiert nur einen einzigen Befehl: **struct::list**. Alle hier bereitgestellten Funktionen sind über einen Unterbefehl dieses Befehls erreichbar.

Befehle

::struct::list longestCommonSubsequence *sequence1 sequence2* ?*maxOccurs*?

Gibt eine Liste von Indizes in *sequence1* zurück und eine zugehörige Liste, in der jedes Element der Index des übereinstimmenden Werts in *sequence2* ist, nach dem Algorithmus der längsten gemeinsamen Teilfolge. Wenn *maxOccurs* angegeben ist, beschränkt sich die

gemeinsame Teilfolge auf Elemente, die in *sequence2* höchstens *maxOccurs* Mal vorkommen.

::struct::list longestCommonSubsequence2 *sequence1 sequence2 ?maxOccurs?*

Gibt die längste gemeinsame Teilfolge der Elemente in den beiden Listen *sequence1* und *sequence2* zurück. Ist *maxOccurs* angegeben, ist das Ergebnis nur eine Näherung: Zunächst wird die längste gemeinsame Folge nur jener Elemente ermittelt, die in *sequence2* höchstens *maxOccurs* Mal vorkommen; mit diesem Ergebnis werden dann die beiden Listen aneinander ausgerichtet, und für die Teillisten zwischen den beiden Elementen werden die längsten gemeinsamen Teilfolgen bestimmt.

Das Ergebnis entspricht dem von **longestCommonSubsequence**.

::struct::list lcsInvert *lcsData len1 len2*

Nimmt eine Beschreibung einer längsten gemeinsamen Teilfolge (*lcsData*) entgegen, invertiert sie und gibt das Ergebnis zurück. Invertieren heißt hier: Die Eingabe beschreibt, welche Teile der beiden Sequenzen identisch sind, die Ausgabe beschreibt stattdessen die Unterschiede.

Damit das Ergebnis vollständig bestimmt ist, müssen die Längen der beiden Sequenzen bekannt sein; sie werden über *len1* und *len2* angegeben.

Das Ergebnis ist eine Liste, in der jedes Element einen Block der Unterschiede zwischen den beiden Sequenzen beschreibt. Diese Beschreibung ist eine Liste aus drei Elementen: einem Typ und zwei Indexpaaren in *sequence1* bzw. *sequence2*, in dieser Reihenfolge. Der Typ kann einen von drei Werten annehmen:

added

Beschreibt eine Ergänzung. Das heißt, Elemente, die in *sequence1* fehlen, sind in *sequence2* zu finden. Das Indexpaar in *sequence1* beschreibt, an welcher Stelle in *sequence1* der hinzugefügte Bereich erwartet worden wäre. Der erste Index bezieht sich auf das Element unmittelbar vor dem hinzugefügten Bereich, und der zweite Index bezieht sich auf das Element unmittelbar nach dem hinzugefügten Bereich. Das Indexpaar in *sequence2* beschreibt den Bereich der Elemente, der dort hinzugefügt wurde. Der erste Index bezieht sich auf das erste Element in diesem Bereich, und der zweite Index bezieht sich auf das letzte Element in diesem Bereich.

deleted

Beschreibt eine Löschung. Das heißt, Elemente, die in *sequence1* enthalten sind, fehlen in *sequence2*. Das Indexpaar in *sequence1* beschreibt den Bereich der Elemente, der gelöscht wurde. Der erste Index bezieht sich auf das erste Element in diesem Bereich, und der zweite Index bezieht sich auf das letzte Element in diesem Bereich. Das Indexpaar in *sequence2* beschreibt, an welcher Stelle in *sequence2* der gelöschte Bereich erwartet worden wäre. Der erste Index bezieht sich auf das Element unmittelbar vor dem gelöschten Bereich, und der zweite Index bezieht sich auf das Element unmittelbar nach dem gelöschten Bereich.

changed

Beschreibt eine allgemeine Änderung. Das heißt, ein Bereich von Elementen in *sequence1* wurde durch einen anderen Bereich von Elementen in *sequence2* ersetzt. Das Indexpaar in *sequence1* beschreibt den Bereich der Elemente, der ersetzt wurde. Der erste Index bezieht sich auf das erste Element des Bereichs, und der zweite Index bezieht sich auf das letzte Element des Bereichs. Das Indexpaar in *sequence2* beschreibt den Bereich der

Elemente, der den ursprünglichen Bereich ersetzt. Auch hier bezieht sich der erste Index auf das erste Element des Bereichs, und der zweite Index bezieht sich auf das letzte Element des Bereichs.

```
sequence 1 = {a b r a c a d a b r a}
lcs 1      =   {1 2   4 5     8 9 10}
lcs 2      =   {0 1   3 4     5 6 7}
sequence 2 =   {b r i c a     b r a c}

Inversion  = {{deleted  {0  0} {-1 0}}
              {changed  {3  3}  {2 2}}
              {deleted  {6  7}  {4 5}}
              {added   {10 11}  {8 8}}}
```

Hinweise:
- Ein Index **-1** in einem *deleted*-Block bezieht sich auf die Stelle unmittelbar vor dem ersten Element der zweiten Sequenz.
- Ebenso verweist ein Index in einem *added*-Block, der gleich der Länge der ersten Sequenz ist, auf die Stelle unmittelbar hinter dem Ende der Sequenz.

::struct::list lcsInvert2 ***lcs1 lcs2 len1 len2***

Ähnlich wie **lcsInvert**. Anstatt das Ergebnis eines Aufrufs von **longestCommonSubsequence** direkt zu übernehmen, erwartet dieser Unterbefehl die Indizes der beiden Sequenzen in zwei separaten Listen.

::struct::list lcsInvertMerge ***lcsData len1 len2***

Ähnlich wie **lcsInvert**. Gibt im Wesentlichen dieselbe Struktur wie dieser Befehl zurück, nur dass sie auch Blöcke vom Typ **unchanged** enthalten kann.

Diese neuen Blöcke beschreiben die Teile, die zwischen den beiden Sequenzen unverändert geblieben sind. Das bedeutet, dass das Ergebnis dieses Befehls sowohl die geänderten als auch die unveränderten Teile der beiden Sequenzen in einer Struktur beschreibt.

```
sequence 1 = {a b r a c a d a b r a}
lcs 1      =   {1 2   4 5     8 9 10}
lcs 2      =   {0 1   3 4     5 6 7}
sequence 2 =   {b r i c a     b r a c}

Inversion/Merge  = {{deleted   {0  0} {-1 0}}
                    {unchanged {1  2}  {0 1}}
                    {changed   {3  3}  {2 2}}
                    {unchanged {4  5}  {3 4}}
                    {deleted   {6  7}  {4 5}}
                    {unchanged {8 10}  {5 7}}
                    {added    {10 11}  {8 8}}}
```

::struct::list lcsInvertMerge2 ***lcs1 lcs2 len1 len2***

Ähnlich wie **lcsInvertMerge**. Anstatt das Ergebnis eines Aufrufs von **longestCommonSubsequence** direkt zu übernehmen, erwartet dieser Unterbefehl die

Indizes für die beiden Sequenzen in zwei separaten Listen.

::struct::list reverse *sequence*

Der Unterbefehl nimmt eine einzelne *sequence* als Argument entgegen und gibt eine neue Sequenz zurück, die die Elemente der Eingabesequenz in umgekehrter Reihenfolge enthält.

::struct::list shuffle *list*

Der Unterbefehl nimmt eine *list* entgegen und gibt eine Kopie dieser Liste zurück, deren Elemente in zufälliger Reihenfolge angeordnet sind. Jede mögliche Reihenfolge der Elemente wird mit gleicher Wahrscheinlichkeit erzeugt. Intern wird der Fisher-Yates-Mischalgorithmus verwendet.

::struct::list assign *sequence varname ?varname?...*

Der Unterbefehl weist die ersten **n** Elemente der Eingabe *sequence* der Variablen oder den Variablen zu, deren Namen nach der Sequenz aufgeführt sind; **n** ist die Anzahl der angegebenen Variablen.

Sind mehr Variablen angegeben, als *sequence* Elemente hat, erhalten die überzähligen Variablen die leere Zeichenkette.

Enthält *sequence* mehr Elemente, als Variablennamen angegeben sind, gibt der Unterbefehl eine Liste der nicht zugewiesenen Elemente zurück. Andernfalls wird eine leere Liste zurückgegeben.

```
tclsh> ::struct::list assign {a b c d e} foo bar
c d e
tclsh> set foo
a
tclsh> set bar
b
```

::struct::list flatten ?-full? ?--? *sequence*

Der Unterbefehl nimmt eine einzelne *sequence* entgegen und gibt eine neue Sequenz zurück, in der eine Verschachtelungsebene der Eingabesequenz entfernt ist. Mit anderen Worten: Die Unterlisten in der Eingabesequenz werden durch ihre Elemente ersetzt.

Ist die Option **-full** angegeben, entfernt der Unterbefehl jede Verschachtelung, die er findet.

```
tclsh> ::struct::list flatten {1 2 3 {4 5} {6 7} {{8 9}} 10}
1 2 3 4 5 6 7 {8 9} 10
tclsh> ::struct::list flatten -full {1 2 3 {4 5} {6 7} {{8 9}}
10}
1 2 3 4 5 6 7 8 9 10
```

::struct::list map *sequence cmdprefix*

Der Unterbefehl nimmt eine zu bearbeitende *sequence* sowie ein Befehlspräfix (*cmdprefix*) entgegen, das eine Operation angibt, wendet das Befehlspräfix auf jedes Element der Sequenz an und gibt die Sequenz der Ergebnisse zurück.

Das Befehlspräfix wird mit einem einzelnen angehängten Wort ausgewertet. Die Auswertung erfolgt im Kontext des Aufrufers des Unterbefehls.

```
tclsh> # squaring all elements in a list

tclsh> proc sqr {x} {expr {$x*$x}}
tclsh> ::struct::list map {1 2 3 4 5} sqr
1 4 9 16 25

tclsh> # Retrieving the second column from a matrix
tclsh> # given as list of lists.

tclsh> proc projection {n list} {::lindex $list $n}
tclsh> ::struct::list map {{a b c} {1 2 3} {d f g}} {projection
1}
b 2 f
```

::struct::list mapfor *var sequence script*

Der Unterbefehl nimmt eine zu bearbeitende *sequence* sowie ein Tcl-*script* entgegen, wendet das Skript auf jedes Element der Sequenz an und gibt eine Sequenz zurück, die aus den Ergebnissen dieser Anwendung besteht.

Das Skript wird unverändert ausgewertet und hat über die angegebene Iterationsvariable *var* Zugriff auf das aktuelle Listenelement. Die Auswertung erfolgt im Kontext des Aufrufers des Unterbefehls.

```
tclsh> # squaring all elements in a list

tclsh> ::struct::list mapfor x {1 2 3 4 5} {
expr {$x * $x}
}
1 4 9 16 25

tclsh> # Retrieving the second column from a matrix
tclsh> # given as list of lists.

tclsh> ::struct::list mapfor x {{a b c} {1 2 3} {d f g}} {
lindex $x 1
}
b 2 f
```

::struct::list filter *sequence cmdprefix*

Der Unterbefehl nimmt eine zu bearbeitende *sequence* sowie ein Befehlspräfix (*cmdprefix*) entgegen, das eine Operation angibt. Er wendet das Befehlspräfix auf jedes Element der Sequenz an und gibt eine Sequenz aus allen Elementen von *sequence* zurück, für die das Befehlspräfix **true** geliefert hat. Mit anderen Worten: Dieser Befehl filtert alle Elemente der Eingabe *sequence* heraus, die den durch *cmdprefix* dargestellten Test nicht bestehen, und gibt die übrigen Elemente zurück.

Das Befehlspräfix wird mit einem einzelnen angehängten Wort ausgewertet. Die Auswertung erfolgt im Kontext des Aufrufers des Unterbefehls.

```
tclsh> # removing all odd numbers from the input

tclsh> proc even {x} {expr {($x % 2) == 0}}
tclsh> ::struct::list filter {1 2 3 4 5} even
2 4
```

Hinweis: **filter** ist eine spezielle Anwendung von **fold**, bei der das Ergebnis je nach Testergebnis um das aktuelle Element erweitert wird oder nicht.

::struct::list filterfor *var sequence expr*

Der Unterbefehl nimmt eine zu bearbeitende *sequence* sowie einen Tcl-Ausdruck (*expr*) entgegen, der eine Bedingung angibt. Er wendet die Bedingung auf jedes Element der Sequenz an und gibt eine Sequenz aus allen Elementen von *sequence* zurück, für die der Ausdruck **true** geliefert hat. Mit anderen Worten: Dieser Befehl filtert alle Elemente der Eingabesequenz *sequence* heraus, die die durch die Bedingung *expr* dargestellte Prüfung nicht bestehen, und gibt die verbleibenden Elemente zurück.

Der Ausdruck wird unverändert ausgewertet und hat über die angegebene Iterationsvariable *var* Zugriff auf das aktuelle Listenelement. Die Auswertung erfolgt im Kontext des Aufrufers des Unterbefehls.

```
tclsh> # removing all odd numbers from the input

tclsh> ::struct::list filterfor x {1 2 3 4 5} {($x % 2) == 0}
2 4
```

::struct::list split *sequence cmdprefix* ?*passVar failVar*?

Dies ist eine Variante der Methode **filter**, siehe oben. Statt nur die Elemente zurückzugeben, die den Test bestehen, liefert er Listen sowohl der Elemente, die den Test bestehen, als auch derer, die ihn nicht bestehen.

Sind keine Variablennamen angegeben, ist das Ergebnis des Befehls eine Liste aus der Liste der bestehenden und der Liste der nicht bestehenden Elemente, in dieser Reihenfolge. Andernfalls werden die Listen der Elemente, die den Test bestanden haben, und der Elemente, die den Test nicht bestanden haben, in den beiden angegebenen Variablen gespeichert, und das Ergebnis ist eine Liste, die zwei Zahlen enthält: die Anzahl der Elemente, die den Test bestanden haben, und die Anzahl der Elemente, die den Test nicht bestanden haben, in dieser Reihenfolge.

Die Schnittstelle zum Test entspricht der von **filter** verwendeten.

::struct::list fold *sequence initialvalue cmdprefix*

Der Unterbefehl nimmt eine zu bearbeitende *sequence*, eine beliebige Zeichenkette *initial value* sowie ein Befehlspräfix (*cmdprefix*) entgegen, das eine Operation angibt.

Das Befehlspräfix wird mit zwei angehängten Wörtern ausgewertet. Das zweite dieser Wörter ist stets ein Element der Sequenz. Die Auswertung erfolgt im Kontext des Aufrufers des Unterbefehls.

Anschließend wird die Sequenz durch wiederholte Anwendung des Befehlspräfixes auf einen

einzigen Wert reduziert, und dieser Wert wird zurückgegeben. Diese Reduktion erfolgt durch

1
Anwendung des Befehls auf den Anfangswert und das erste Element der Liste.

2
Anwendung des Befehls auf das Ergebnis des letzten Aufrufs und das zweite Element der Liste.

...
siehe unten ...

i
Anwendung des Befehls auf das Ergebnis des letzten Aufrufs und das Element mit der Nummer **i** in der Liste.

...
siehe unten ...

end
Anwendung des Befehls auf das Ergebnis des letzten Aufrufs und das letzte Element der Liste. Das Ergebnis dieses Aufrufs wird als Ergebnis des Unterbefehls zurückgegeben.

```
tclsh> # summing the elements in a list.
tclsh> proc + {a b} {expr {$a + $b}}
tclsh> ::struct::list fold {1 2 3 4 5} 0 +
15
```

::struct::list shift *listvar*
Der Unterbefehl nimmt die in der durch *listvar* benannten Variablen enthaltene Liste und verschiebt sie um ein Element nach unten. Nach dem Aufruf enthält *listvar* eine Liste mit dem zweiten bis letzten Element der Eingabeliste. Das erste Element der Liste wird als Ergebnis des Befehls zurückgegeben. Bei einer leeren Liste geschieht nichts.

::struct::list iota *n*
Der Unterbefehl gibt eine Liste zurück, die die ganzzahligen Werte im Bereich **[0,n)** enthält. Das Element am Index **i** der Liste enthält die Zahl **i**.

Für "*n* == **0**" wird eine leere Liste zurückgegeben.

::struct::list equal ?-simple? ?--? *a b*
Der Unterbefehl vergleicht die beiden Listen *a* und *b* auf Gleichheit. Mit anderen Worten: Sie müssen dieselbe Länge aufweisen und dieselben Elemente in derselben Reihenfolge enthalten.

Standardmäßig werden Gleichheitsprüfungen rekursiv auf Elemente von Listen angewendet, die selbst Listen sind. Dieses Verhalten wird deaktiviert, wenn die Option **-simple** festgelegt wird.

Die Option **--** beendet die Optionsverarbeitung, auch wenn das folgende Argument wie eine Option aussieht.

Als Ergebnis des Befehls wird ein boolescher Wert zurückgegeben. Dieser Wert lautet **true**, wenn die beiden Listen gleich sind, und andernfalls **false**.

::struct::list repeat *size element1 ?element2 element3...?*

Der Unterbefehl erstellt eine Liste der Länge "*size* * *Anzahl der Elemente*", indem er die Folge der Elemente *element1 element2... size*-mal wiederholt. *size* muss eine positive ganze Zahl sein, *element***n** kann ein beliebiger Tcl-Wert sein. Beachten Sie, dass **repeat 1 arg ...** dasselbe ist wie **list arg ...**, nur dass bei **repeat** das *arg* erforderlich ist.

Beispiele:

```
tclsh> ::struct::list repeat 3 a
a a a
tclsh> ::struct::list repeat 3 [::struct::list repeat 3 0]
{0 0 0} {0 0 0} {0 0 0}
tclsh> ::struct::list repeat 3 a b c
a b c a b c a b c
tclsh> ::struct::list repeat 3 [::struct::list repeat 2 a] b c
{a a} b c {a a} b c {a a} b c
```

::struct::list repeatn *value size...*

Der Unterbefehl erzeugt eine (verschachtelte) Liste, die an allen Positionen *value* enthält. Die genaue Größe und die Verschachtelungstiefe werden durch die *size*-Argumente bestimmt, die alle ganze Zahlen größer oder gleich null sein müssen.

Ein einzelnes *size*-Argument, das eine Liste aus mehr als einem Element ist, wird so behandelt, als wären mehrere *size*-Argumente angegeben.

Ist nur ein *size*-Argument vorhanden, ist die zurückgegebene Liste nicht verschachtelt, hat die Länge *size* und enthält an allen Positionen *value*. Sind mehrere *size*-Argumente vorhanden, ist die zurückgegebene Liste verschachtelt und hat die Länge, die durch das zuletzt angegebene *size*-Argument festgelegt wird. Die Elemente dieser Liste werden als Ergebnis von **Repeat** für dieselben Argumente definiert, jedoch ohne den letzten *size*-Wert.

Ist kein *size*-Argument vorhanden, wird eine leere Liste zurückgegeben.

```
tclsh> ::struct::list repeatn  0 3 4
{0 0 0} {0 0 0} {0 0 0} {0 0 0}
tclsh> ::struct::list repeatn  0 {3 4}
{0 0 0} {0 0 0} {0 0 0} {0 0 0}
tclsh> ::struct::list repeatn  0 {3 4 5}
{{0 0 0} {0 0 0} {0 0 0} {0 0 0}} {{0 0 0} {0 0 0} {0 0 0} {0 0
0}} {{0 0 0} {0 0 0} {0 0 0} {0 0 0}} {{0 0 0} {0 0 0} {0 0 0}
{0 0 0}} {{0 0 0} {0 0 0} {0 0 0} {0 0 0}}
```

::struct::list dbJoin ?-inner|-left|-right|-full? ?-keys *varname*? {*keycol table*}...

Die Methode führt eine Verknüpfung von Tabellen gemäß der relationalen Algebra durch. Die Ausführung einer der möglichen Outer-Join-Operationen wird durch das Vorhandensein einer der Optionen **-left**, **-right** oder **-full** ausgelöst. Ist keine dieser Optionen vorhanden, wird ein regulärer Inner Join durchgeführt. Dies kann auch durch die Angabe von **-inner** ausgelöst werden. Die verschiedenen möglichen Verknüpfungsoperationen werden im Abschnitt Tabellenverknüpfung ausführlich erläutert.

Wenn die Option **-keys** vorhanden ist, ist ihr Argument der Name einer Variablen, in der die vollständige Liste der gefundenen Schlüssel gespeichert wird. Je nach der genauen Beschaffenheit der Eingabetabelle und dem Verknüpfungsmodus enthält die Ausgabetabelle standardmäßig möglicherweise nicht alle Schlüssel. In einem solchen Fall kann der Aufrufer eine Variable für diese Informationen deklarieren und diese dann selbst in die Ausgabetabelle einfügen, da er über mehr Informationen zur Platzierung verfügt als dieser Befehl.

Nun bleibt noch das Format der Argumente zu erläutern.

Die *keycol*-Argumente sind die Indizes der Spalten in den Tabellen, die die für die Verknüpfung zu verwendenden Schlüsselwerte enthalten. Jedes Argument bezieht sich auf die unmittelbar darauf folgende Tabelle. Die Spalten werden ab **0** gezählt, wobei diese Nummer die erste Spalte bezeichnet. Die Tabelle, zu der der Spaltenindex gehört, muss mindestens *keycol*+1 Spalten aufweisen. Bei einer geringeren Anzahl wird ein Fehler ausgelöst.

Die *table*-Argumente stellen eine Tabelle oder Matrix aus Zeilen und Spalten mit Werten dar. Wir verwenden dieselbe Darstellung, wie sie von den Methoden **get rect** und **set rect** von **matrix**-Objekten generiert und verarbeitet wird. Mit anderen Worten: Jedes Argument ist eine Liste, die die gesamte Matrix darstellt. Ihre Elemente sind ebenfalls Listen, von denen jede eine einzelne Zeile der Matrix darstellt. Die Elemente der Zeilenlisten sind die Werte der Spalten.

Die aus der Verknüpfungsoperation resultierende Tabelle wird als Ergebnis des Befehls zurückgegeben. Wir verwenden dieselbe Darstellung wie oben für die Eingabe-*table*s beschrieben.

::struct::list dbJoinKeyed ?-inner|-left|-right|-full? ?-keys *varname*? *table*...

Die mit dieser Methode durchgeführten Vorgänge entsprechen den oben für **dbJoin** beschriebenen. Der einzige Unterschied besteht in der Angabe der zu verwendenden Schlüssel. Anstatt hier von der Tabelle getrennte Spaltenindizes zu verwenden, werden die Schlüssel innerhalb der Tabelle selbst bereitgestellt. Die Zeilenelemente in jeder *table* sind keine Listen von Spaltenwerten, sondern eine Liste mit zwei Elementen, wobei das zweite Element die reguläre Liste der Spaltenwerte und das erste Element der zu verwendende Schlüssel ist.

::struct::list swap *listvar i j*

Der Unterbefehl tauscht die Elemente an den Indizes *i* und *j* in der Liste aus, die in der durch *listvar* bezeichneten Variablen gespeichert ist. Die Liste wird an Ort und Stelle geändert und zudem als Ergebnis des Unterbefehls zurückgegeben.

::struct::list firstperm *list*

Dieser Unterbefehl gibt die lexikografisch erste Permutation der Eingabeliste *list* zurück.

::struct::list nextperm *perm*

Dieser Unterbefehl akzeptiert eine Permutation einer Menge von Elementen (die durch *perm* bereitgestellt wird) und gibt die nächste Permutation in lexikografischer Reihenfolge zurück.

Der hier verwendete Algorithmus stammt von Donal E. Knuth; Einzelheiten finden Sie im Abschnitt Literatur.

::struct::list permutations *list*

Dieser Unterbefehl gibt eine Liste zurück, die alle Permutationen der Eingabeliste *list* in lexikografischer Reihenfolge enthält.

::struct::list foreachperm *var list body*

Dieser Unterbefehl führt das Skript *body* einmal für jede Permutation der angegebenen *list* aus. Die Permutationen werden in lexikografischer Reihenfolge durchlaufen, und die Variable *var* wird auf die Permutation gesetzt, für die *body* gerade ausgeführt wird. Das Ergebnis des Schleifenbefehls ist die leere Zeichenkette.

Längste gemeinsame Teilfolge und Dateivergleich

Der Unterbefehl **longestCommonSubsequence** bildet den Kern eines flexiblen Systems zur Durchführung von Differenzvergleichen von Dateien, ähnlich der Funktionalität, die der Unix-Befehl **diff** bietet. Zwar ist diese Prozedur für viele Aufgaben des Dateivergleichs recht schnell, doch verschlechtert sich ihre Leistung erheblich, wenn *sequence2* viele gleiche Elemente enthält (wie beispielsweise bei der Verwendung dieser Prozedur zum Vergleich zweier Dateien, bei denen ein Viertel der Zeilen leer ist). Dieser Nachteil ist dem verwendeten Algorithmus inhärent (Einzelheiten finden Sie in der Literatur).

Ein Ansatz zur Bewältigung des Leistungsproblems, der sich in der Praxis mitunter als wirksam erweist, besteht darin, Elemente, die häufiger als eine bestimmte Anzahl von Malen vorkommen, willkürlich auszuschließen. Diese Anzahl wird als Parameter *maxOccurs* angegeben. Wenn häufig vorkommende Zeilen auf diese Weise ausgeschlossen werden, erscheinen sie nicht in der berechneten gemeinsamen Teilfolge; das Ergebnis ist dann die längste gemeinsame Teilfolge aus selten vorkommenden Elementen. Die Prozedur **longestCommonSubsequence2** implementiert diese Heuristik. Sie fungiert als Wrapper um **longestCommonSubsequence**; sie berechnet die längste gemeinsame Teilfolge aus selten vorkommenden Elementen und unterteilt anschließend die Teilfolgen, die zwischen den Übereinstimmungen liegen, um die tatsächliche längste gemeinsame Teilfolge anzunähern.

Tabellenverknüpfung

Dies ist eine Operation aus der relationalen Algebra für relationale Datenbanken.

Am einfachsten lässt sich der reguläre Inner Join so verstehen, dass zunächst das kartesische Produkt aller beteiligten Tabellen gebildet wird und anschließend in der resultierenden Tabelle nur jene Zeilen beibehalten werden, bei denen die Werte in den festgelegten Schlüsselspalten miteinander übereinstimmen.

Eine naive Umsetzung dieser Beschreibung, d. h. wie oben beschrieben, führt zu einem *riesigen* Zwischenergebnis. Um dies zu vermeiden, werden das kartesische Produkt und die Filterung der Zeilen gleichzeitig durchgeführt. Erforderlich ist eine schnelle Methode, um festzustellen, ob ein Schlüssel in einer Tabelle vorhanden ist. In einer echten Datenbank erfolgt dies über Indizes. Hier verwenden wir intern Arrays.

Ein *Outer*-Join ist eine Erweiterung des Inner-Joins für zwei Tabellen. Es gibt drei Varianten von Outer-Joins, die als *Left*-, *Right*- und *Full*-Outer-Joins bezeichnet werden. Ihr Ergebnis enthält stets alle Zeilen eines Inner-Joins sowie einige zusätzliche Zeilen.

1. Beim "Left Outer Join" handelt es sich bei den zusätzlichen Zeilen um alle Zeilen der linken Tabelle, für die es in der rechten Tabelle keinen Schlüssel gibt. Diese werden mit einer leeren Zeile der rechten Tabelle verknüpft, um sie in das Ergebnis einzubinden.
2. Beim "Right Outer Join" sind die zusätzlichen Zeilen alle Zeilen aus der rechten Tabelle, für die

es in der linken Tabelle keinen Schlüssel gibt. Sie werden mit einer leeren Zeile der linken Tabelle verknüpft, um sie in das Ergebnis einzufügen.
3. Der "Full Outer Join" vereint sowohl den "Left Outer Join" als auch den "Right Outer Join". Mit anderen Worten: Die zusätzlichen Zeilen entsprechen der Kombination der für den "Left Outer Join" und den "Right Outer Join" definierten Zeilen.

Wir erweitern alle Verknüpfungen von zwei auf **n** Tabellen (**n** > 2), indem wir folgenden Befehl ausführen:

```
    (...((table1 join table2) join table3) ...) join tableN
```

Beispiele für alle Verknüpfungen:

```
    Inner Join

    {0 foo}              {0 bagel}    {0 foo   0 bagel}
    {1 snarf} inner join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}             {3 driver}

    Left Outer Join

    {0 foo}                   {0 bagel}    {0 foo   0 bagel}
    {1 snarf} left outer join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}                  {3 driver}   {2 blue  {} {}}

    Right Outer Join

    {0 foo}                    {0 bagel}    {0 foo   0 bagel}
    {1 snarf} right outer join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}                   {3 driver}   {{} {}   3 driver}

    Full Outer Join

    {0 foo}                   {0 bagel}    {0 foo   0 bagel}
    {1 snarf} full outer join {1 snatz}  = {1 snarf 1 snatz}
    {2 blue}                  {3 driver}   {2 blue  {} {}}
                                           {{} {}   3 driver}
```

Literatur

1. J. W. Hunt und M. D. McIlroy, "An algorithm for differential file comparison", Comp. Sci. Tech. Rep. #41, Bell Telephone Laboratories (1976). Verfügbar im Internet auf der persönlichen Website des zweiten Autors: http://www.cs.dartmouth.edu/~doug/
2. Donald E. Knuth, "Fascicle 2b of 'The Art of Computer Programming' volume 4". Im Internet verfügbar auf der persönlichen Website des Autors: http://www-cs-faculty.stanford.edu/~knuth/fasc2b.ps.gz.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden

oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: list* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Datenstrukturen

Copyright

```
Copyright (c) 2003-2005 by Kevin B. Kenny. All rights reserved
Copyright (c) 2003-2012 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


struct_map
Stichwörter: struct_map

Name
struct::map – Schlüssel-Wert-Zuordnungen verwalten

Übersicht
package require struct::map ?**1**?
::struct::map *mapName*
mapName method ?*arg arg* ...?
mapName **get**
mapName **names**
mapName **set** *name* ?*value*?
mapName **unset** ?*pattern...*?

Beschreibung
Stellt eine snit-Klasse bereit, deren Instanzen eine Schlüssel-Wert-Map verwalten. Mit anderen Worten: eine Objekthülle um Tcl-Arrays.

API
Der Hauptbefehl erzeugt Maps:

::struct::map *mapName*
Erzeugt eine neue, leere Map mit einem zugehörigen globalen Tcl-Befehl, dessen Name *mapName* lautet. Mit ihm lassen sich verschiedene Operationen auf der Map aufrufen. Er hat die folgende allgemeine Form:
mapName method ?*arg arg ...*?
method und *arg*umente bestimmen das genaue Verhalten des Befehls.

Wird als *mapName* **%AUTO%** angegeben, erzeugt das Paket selbst einen eindeutigen Namen. Das Ergebnis des Befehls ist der vollqualifizierte Name des Instanzbefehls.

Für Map-Objekte sind folgende Befehle möglich:

***mapName* get**
Gibt die gesamte Map als Tcl-Dictionary zurück.

***mapName* names**
Gibt die Liste aller der Map bekannten Schlüssel in beliebiger Reihenfolge zurück.

***mapName* set *name* ?*value*?**
Setzt den Schlüssel *name* auf den angegebenen *value*, sofern ein Wert angegeben ist. Gibt den Wert für den Schlüssel zurück. Löst einen Fehler aus, wenn der Schlüssel unbekannt ist.

***mapName* unset ?*pattern*...?**
Entfernt alle Schlüssel aus der Map, die auf mindestens eines der glob-Muster *pattern* passen. Wird kein Muster angegeben, werden alle Schlüssel entfernt. Mit anderen Worten: Das Standardmuster lautet *. Das Ergebnis des Befehls ist die leere Zeichenkette.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: list* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

struct_set

Stichwörter: struct_set, cardinality, difference, emptiness, exclusion, inclusion, intersection, membership, set, symmetric difference, union

Name

struct::set – Befehle zur Bearbeitung von Mengen

Übersicht

package require Tcl 8.5 9
package require struct::set ?**2.2.4**?
::struct::set empty *set*
::struct::set size *set*
::struct::set contains *set value*
::struct::set union ?*set1...*?
::struct::set intersect ?*set1...*?
::struct::set difference *set1 set2*
::struct::set symdiff *set1 set2*
::struct::set intersect3 *set1 set2*
::struct::set equal *set1 set2*
::struct::set include *setVar value*
::struct::set exclude *setVar value*
::struct::set add *setVar set*
::struct::set subtract *setVar set*

::struct::set subsetof *A B*

Beschreibung

Der Namespace **::struct::set** enthält mehrere nützliche Befehle zur Verarbeitung endlicher Mengen.

Er exportiert nur einen einzigen Befehl: **struct::set**. Der gesamte Funktionsumfang ist über die Unterbefehle dieses Befehls verfügbar.

Eine neue leere Menge kann mit dem Befehl **::struct::set add** *setVar {}* erstellt werden.

Hinweis: Ab Version 2.2 dieses Pakets wird, sofern verfügbar, eine auf critcl basierende C-Implementierung verwendet, und zwar mit Tcl 8.4 oder höher.

Befehle

Im Folgenden sind *set*, *set1* und *set2* **set**-Werte, und *setVar* ist der Name einer Variablen, die ein **set** enthält.

::struct::set empty *set*
Gibt **1** (wahr) zurück, wenn *set* leer ist, sonst **0** (falsch).

::struct::set size *set*
Gibt die Kardinalität von *set* zurück, d. h. die Anzahl der Elemente in *set*. Sie kann null sein.

::struct::set contains *set value*
Gibt **1** (wahr) zurück, wenn *set* das Element *value* enthält, sonst **0** (falsch).

::struct::set union ?*set1*...?
Gibt ein **set** zurück, das die Vereinigung der angegebenen **set**s ist, d. h. *set1* ∪ *set2* ∪ .... Das resultierende **set** enthält jeden Wert aus jedem angegebenen **set**, ohne Duplikate.

::struct::set intersect ?*set1*...?
Gibt ein **set** zurück, das die Schnittmenge der angegebenen **set**s ist, d. h. *set1* ∩ *set2* ∩ .... Das resultierende **set** enthält jeden Wert, den *alle* angegebenen **set**s gemeinsam haben, ohne Duplikate.

::struct::set difference *set1 set2*
Gibt ein **set** zurück, das die Differenz von *set1* und *set2* ist, d. h. *set1* – *set2*. Das resultierende **set** enthält jeden Wert, der in *set1* enthalten ist, aber *nicht* in *set2* vorkommt.

::struct::set symdiff *set1 set2*
Gibt ein **set** zurück, das die symmetrische Differenz von *set1* und *set2* ist, d. h. *set1* Δ *set2*. Das resultierende **set** enthält jeden Wert, der in *set1 oder* in *set2* enthalten ist, jedoch *nicht* in beiden.

::struct::set intersect3 *set1 set2*
Liefert eine Kombination der Methoden **intersect** und **difference**.

Der Rückgabewert ist eine Liste mit drei Elementen, die in dieser Reihenfolge "*set1* ∩ *set2*", "*set1* – *set2*" und "*set2* – *set1*" enthält. Mit anderen Worten: die Schnittmenge von *set1* und *set2* sowie deren Differenzen.

::struct::set equal *set1 set2*
Gibt **1** (wahr) zurück, wenn die beiden **set**s genau dieselben Werte enthalten, sonst **0** (falsch).

::struct::set include *setVar value*

Fügt das Element *value* zum **set** mit dem Namen *setVar* hinzu und legt *setVar* an, falls es noch nicht existiert. Ist *value* bereits im **set** *setVar*, geschieht nichts, und das ist kein Fehler. Gibt nichts zurück. Dieser Befehl kann zum Erstellen neuer **set**s verwendet werden.

::struct::set exclude *setVar value*
Entfernt das Element *value* aus dem **set** mit dem Namen *setVar*. Ist *value* nicht in *setVar* enthalten, tut der Befehl nichts, und das ist kein Fehler. Gibt nichts zurück.

::struct::set add *setVar set*
Fügt jedes Element aus dem **set**-Wert *set* zum **set** namens *setVar* hinzu, ohne Duplikate. Das **set** *setVar* wird angelegt, falls es noch nicht existiert. Gibt nichts zurück.

Verwenden Sie **::struct::set add** *setVar {}*, um eine neue leere Menge zu erstellen. Verwenden Sie **::struct::set include**, um einzelne Werte hinzuzufügen.

::struct::set subtract *setVar set*
Entfernt jedes Element des **set**-Werts *set* aus dem **set** namens *setVar*. Gibt nichts zurück. Verwenden Sie **::struct::set exclude**, um einzelne Werte zu entfernen.

::struct::set subsetof *A B*
Gibt **1** (wahr) zurück, wenn $A \subseteq B$ gilt, d. h., wenn **set** *A* eine echte Teilmenge von **set** *B* oder gleich **set** *B* ist, sonst **0** (falsch).

Beispiele

Ein neues **set** von Grund auf anlegen und füllen. Das **set earth_metals** wird im ersten Schleifendurchlauf angelegt und in den folgenden Durchläufen ergänzt.

```
foreach element {Be Mg Ca Sr Ba Ra} {
    struct::set include earth_metals $element
}
```

Ein **set** als Vereinigung einiger vorhandener **set**s bilden.

```
set metals [struct::set union $alkali_metals $earth_metals]
```

Die Eigenschaften eines **set** abfragen.

```
if {[struct::set empty $metals]} {
    puts "empty"
} else {
    puts "[struct::set size $metals] elements"
}
```

Literatur

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: set* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Datenstrukturen

Copyright

```
Copyright (c) 2004-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


struct_tree
Stichwörter: struct_tree, breadth-first, depth-first, in-order, node, post-order, pre-order, serialization, tree

Name
struct::tree – Baumobjekte anlegen und bearbeiten

Übersicht
package require Tcl 8.5 9
package require struct::tree ?**2.1.1**?
package require struct::list ?**1.5**?
::struct::tree ?*treeName*? ?=|*:=*|*as*|*deserialize source*?
treeName option ?*arg arg* ...?
::struct::tree::prune
treeName **=** *sourcetree*
treeName **-->** *desttree*
treeName **ancestors** *node*
treeName **append** *node key value*
treeName **attr** *key*
treeName **attr** *key* **-nodes** *list*
treeName **attr** *key* **-glob** *globpattern*
treeName **attr** *key* **-regexp** *repattern*
treeName **children** ?**-all**? *node* ?*filter cmdprefix*?
treeName **cut** *node*
treeName **delete** *node* ?*node* ...?
treeName **depth** *node*
treeName **descendants** *node* ?*filter cmdprefix*?
treeName **deserialize** *serialization*
treeName **destroy**
treeName **exists** *node*
treeName **get** *node key*
treeName **getall** *node* ?*pattern*?
treeName **keys** *node* ?*pattern*?
treeName **keyexists** *node key*
treeName **index** *node*
treeName **insert** *parent index* ?*child* ?*child* ...?
treeName **isleaf** *node*

treeName **lappend** *node key value*
treeName **leaves**
treeName **move** *parent index node* ?*node* ...?
treeName **next** *node*
treeName **numchildren** *node*
treeName **nodes**
treeName **parent** *node*
treeName **previous** *node*
treeName **rename** *node newname*
treeName **rootname**
treeName **serialize** ?*node*?
treeName **set** *node key* ?*value*?
treeName **size** ?*node*?
treeName **splice** *parent from* ?*to*? ?*child*?
treeName **swap** *node1 node2*
treeName **unset** *node key*
treeName **walk** *node* ?*-order order*? ?*-type type*? *loopvar script*
treeName **walkproc** *node* ?*-order order*? ?*-type type*? *cmdprefix*

Beschreibung

Ein Baum ist eine Sammlung benannter Elemente, Knoten genannt, von denen einer als Wurzel ausgezeichnet ist, zusammen mit einer Beziehung ("Elternschaft"), die den Knoten eine hierarchische Struktur gibt. (Data Structures and Algorithms; Aho, Hopcroft und Ullman; Addison-Wesley, 1987). Neben der Verwaltung der Knotenbeziehungen erlaubt diese Baumimplementierung, jedem Knoten beliebig viele Werte unter Schlüsseln zuzuordnen.

Die Namen der Elemente können beliebige Zeichenketten sein.

Ein Baum ähnelt somit einem Array, weist jedoch drei wesentliche Unterschiede auf:

1. Der Zugriff auf Bäume erfolgt über einen Objektbefehl, während der Zugriff auf Arrays über Variablen erfolgt. (Das bedeutet, dass Bäume nicht lokal zu einer Prozedur sein können.)
2. Bäume weisen eine hierarchische Struktur auf, während ein Array lediglich eine ungeordnete Sammlung ist.
3. Jeder Knoten eines Baums verfügt über eine eigene Sammlung von Attributen und Werten. Dies entspricht einem Array, bei dem jeder Wert ein Dictionary ist.

Hinweis: Die Hauptversion des Pakets **struct** ist wegen inkompatibler Änderungen an der API dieses Moduls auf 2.0 gewechselt. Die vollständige Liste aller Änderungen, inkompatibler und anderer, finden Sie im Abschnitt Änderungen in Version 2.0.

API

Tree-Klasse-API

Die wichtigsten Befehle des Pakets lauten:

::struct::tree ?*treeName*? ?=|:=|as|deserialize *source*?

Der Befehl erzeugt ein neues Baumobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *treeName* lautet. Mit diesem Befehl lassen sich verschiedene Operationen auf dem Baum aufrufen. Er hat die folgende allgemeine Form:
treeName option ?*arg arg ...*?

Option und die *arg*s bestimmen das genaue Verhalten des Befehls.

Ist *treeName* nicht angegeben, erzeugt das Paket selbst einen eindeutigen Namen. Ist eine *source* angegeben, wird der neue Baum daraus initialisiert. Bei den Operatoren **=**,**:=** und **as** wird *source* als Name eines anderen Baumobjekts interpretiert, und der Zuweisungsoperator **=** wird ausgeführt. Bei **deserialize** ist *source* ein serialisiertes Baumobjekt, und **deserialize** wird ausgeführt.

Mit anderen Worten:

```
    ::struct::tree mytree = b
```

entspricht

```
    ::struct::tree mytree
    mytree = b
```

und

```
    ::struct::tree mytree deserialize $b
```

entspricht

```
    ::struct::tree mytree
    mytree deserialize $b
```

::struct::tree::prune
Dieser Befehl steht außerhalb der Baummethoden, da er selbst keine Baummethode ist. Er wirkt jedoch eng mit der Methode **walk** zusammen. Im walk-Skript verwendet, bewirkt er, dass der Durchlauf die Kindknoten des aktuellen Knotens überspringt. Dieser Befehl kann nicht mit den Durchlaufarten verwendet werden, die Kindknoten vor ihrem Elternknoten besuchen, d. h. **post** und **in**. Zulässig sind nur die Durchlaufreihenfolgen **pre** und **both**. Passen Befehl und gewählte Durchlaufreihenfolge nicht zusammen, wird ein Fehler ausgelöst.

API für Baumobjekte

Zuvor zwei allgemeine Anmerkungen:

1. Der Wurzelknoten des Baums kann an den meisten Stellen verwendet werden, an denen ein Knoten verlangt wird. Der Standardname des Wurzelknotens lautet "root", dieser kann jedoch mit der Methode **rename** geändert werden (siehe unten). Wie auch immer der Wurzelknoten des Baums gerade heißt, sein Name lässt sich mit der Methode **rootname** abfragen.
2. Die Methode **insert** ist die einzige Möglichkeit, neue Knoten anzulegen; sie werden automatisch einem Elternknoten zugeordnet. Ein Baumobjekt kann außer dem Wurzelknoten keine Knoten ohne Elternknoten enthalten.

Und nun die Methoden, die die von diesem Paket erzeugten Baumobjekte unterstützen:

treeName = sourcetree
Dies ist der Zuweisungsoperator für Baumobjekte. Er kopiert den im Baumobjekt *sourcetree* enthaltenen Baum über die Baumdaten in *treeName*. Der bisherige Inhalt von *treeName* wird durch diesen Vorgang gelöscht.

Dieser Vorgang entspricht im Wesentlichen

```
    treeName deserialize [sourcetree serialize]
```

treeName* --> *desttree
Dies ist der umgekehrte Zuweisungsoperator für Baumobjekte. Er kopiert den im Baumobjekt *treeName* enthaltenen Baum über die Baumdaten im Objekt *desttree*. Der bisherige Inhalt von *desttree* wird durch diesen Vorgang gelöscht.

Dieser Vorgang entspricht im Wesentlichen

```
    desttree deserialize [treeName serialize]
```

treeName* ancestors *node
Diese Methode erweitert die Methode **parent** und gibt eine Liste aller Vorfahren des angegebenen *node* zurück. Der unmittelbare Vorfahr, also der Elternknoten, ist das erste Element dieser Liste, dessen Elternknoten das zweite Element und so weiter, bis der Wurzelknoten erreicht ist, der somit das letzte Element der zurückgegebenen Liste bildet.

treeName* append *node key value
Hängt *value* an einen der Werte an, die einem Knoten unter einem Schlüssel zugeordnet sind. Gibt den neuen Wert des Attributs *key* zurück.

treeName* attr *key
siehe unten ...

treeName* attr *key* -nodes *list
siehe unten ...

treeName* attr *key* -glob *globpattern
siehe unten ...

treeName* attr *key* -regexp *repattern
Diese Methode ruft den Wert des Attributs mit dem Namen *key* für alle Knoten im Baum ab, die der über eine der möglichen Optionen festgelegten Einschränkung entsprechen und das angegebene Attribut besitzen.

Das Ergebnis ist ein Dictionary, das Knotennamen den Werten des Attributs *key* an diesen Knoten zuordnet. Knoten, die das Attribut *key* nicht besitzen oder eine angegebene Einschränkung nicht erfüllen, werden im Ergebnis nicht aufgeführt.

Die möglichen Einschränkungen sind:

-nodes
Der Wert ist eine Liste von Knoten. Nur die in dieser Liste aufgeführten Knoten werden nach dem Attribut durchsucht.

-glob
Der Wert ist ein Glob-Muster. Nur die Knoten im Baum, deren Namen diesem Muster entsprechen, werden nach dem Attribut durchsucht.

-regexp
Der Wert ist ein regulärer Ausdruck. Nur die Knoten im Baum, deren Namen diesem Muster entsprechen, werden nach dem Attribut durchsucht.

***treeName* children ?-all? *node* ?filter *cmdprefix*?**
Gibt eine Liste der Kindknoten von *node* zurück. Ist die Option **-all** angegeben, enthält das

Ergebnis nicht nur die direkten Kindknoten, sondern auch deren Kindknoten usw. Ist ein Filterbefehl angegeben, enthält das Endergebnis nur die Knoten, die den Test bestehen. Der Befehl in *cmdprefix* wird mit zwei Argumenten aufgerufen: dem Namen des Baumobjekts und dem Namen des betreffenden Knotens. Er wird im Kontext des Aufrufers ausgeführt und muss einen booleschen Wert zurückgeben. Knoten, für die der Befehl **false** zurückgibt, werden aus der Ergebnisliste entfernt, bevor diese an den Aufrufer zurückgegeben wird.

Einige Beispiele:

```
mytree insert root end 0 ; mytree set 0 volume 30
mytree insert root end 1
mytree insert root end 2
mytree insert 0    end 3
mytree insert 0    end 4
mytree insert 4    end 5 ; mytree set 5 volume 50
mytree insert 4    end 6

proc vol {t n} {
$t keyexists $n volume
}
proc vgt40 {t n} {
if {![$t keyexists $n volume]} {return 0}
expr {[$t get $n volume] > 40}
}

tclsh> lsort [mytree children -all root filter vol]
0 5

tclsh> lsort [mytree children -all root filter vgt40]
5

tclsh> lsort [mytree children root filter vol]
0

tclsh> puts ([lsort [mytree children root filter vgt40]])
()
```

treeName* cut *node

Entfernt den durch *node* festgelegten Knoten aus dem Baum, jedoch nicht dessen Kinder. Die Kinder von *node* werden zu Kindern des Elternknotens von *node*, und zwar an dem Index, an dem *node* stand.

***treeName* delete *node* ?*node* ...?**

Entfernt die angegebenen Knoten aus dem Baum. Alle Kindknoten dieser Knoten werden ebenfalls entfernt, damit keine verwaisten Knoten zurückbleiben.

treeName* depth *node

Gibt die Anzahl der Schritte vom Knoten *node* zum Wurzelknoten zurück.

***treeName* descendants *node* ?filter *cmdprefix*?**

Diese Methode erweitert die Methode **children** und gibt eine Liste zurück, die alle Knoten enthält, die von *node* abstammen und den Filter erfüllen, wenn ein solcher angegeben wurde.

Das ist dasselbe wie "*treeName* **children -all**". **descendants** ist vorzuziehen; "children -all" wird irgendwann als veraltet erklärt.

treeName* deserialize *serialization
Das Gegenstück zu **serialize**. Ersetzt die Baumdaten in *treeName* durch den Baum, den der Wert *serialization* beschreibt. Der bisherige Inhalt von *treeName* wird durch diesen Vorgang gelöscht.

***treeName* destroy**
Zerstört den Baum einschließlich seines Speichers und des zugehörigen Befehls.

treeName* exists *node
Gibt true zurück, wenn der angegebene Knoten im Baum existiert.

treeName* get *node key
Gibt den Wert zurück, der dem Schlüssel *key* für den Knoten *node* zugeordnet ist.

***treeName* getall *node* ?*pattern*?**
Gibt ein Dictionary zurück (geeignet für die Verwendung mit [**array set**]), das die Attributdaten von *node* enthält. Ist das glob-Muster *pattern* angegeben, sind nur die Attribute, deren Namen mit dem Muster übereinstimmen, Bestandteil des Dictionarys.

***treeName* keys *node* ?*pattern*?**
Gibt eine Liste der Schlüssel von *node* zurück. Ist *pattern* angegeben, enthält die zurückgegebene Liste nur die Attribute, deren Namen auf das Muster passen. Das Muster ist ein **glob**-Muster.

treeName* keyexists *node key
Gibt true zurück, wenn der angegebene *key* für *node* existiert.

treeName* index *node
Gibt den Index von *node* in der Kinderliste seines Elternknotens zurück. Hat ein Knoten beispielsweise *nodeFoo*, *nodeBar* und *nodeBaz* in dieser Reihenfolge als Kinder, beträgt der Index von *nodeBar* 1.

***treeName* insert *parent index* ?*child* ?*child* ...??**
Fügt einen oder mehrere Knoten als Kinder des Knotens *parent* in den Baum ein. Die Knoten werden in der Reihenfolge hinzugefügt, in der sie angegeben werden. Wenn *parent* gleich **root** ist, bezieht sich dies auf die Wurzel des Baums. Die neuen Knoten werden an der durch *index* angegebenen Position in die Kinderliste des Knotens *parent* eingefügt. *index* kann **end** sein; dann werden die neuen Knoten hinter dem derzeit letzten Kind eingefügt. Indizes in der Form "end-**n**" werden ebenfalls akzeptiert.

Existieren angegebene Kinder bereits in *treeName*, werden diese Knoten von ihrer bisherigen Stelle an die von diesem Befehl angegebene neue Stelle verschoben.

Wird kein *child* angegeben, wird ein einzelner Knoten hinzugefügt und ein Name für den neuen Knoten generiert. Der generierte Name hat die Form *node***x**, wobei **x** eine Zahl ist. Angegebene Namen dürfen weder Leerraum noch Doppelpunkte (":") enthalten.

Das Ergebnis dieses Befehls ist eine Liste der hinzugefügten Knoten.

treeName* isleaf *node
Gibt true zurück, wenn *node* ein Blatt des Baums ist (d. h., wenn *node* keine Kinder hat), sonst false.

treeName* lappend *node key value
Hängt *value* (als Liste) an einen der Werte an, die *node* unter einem Schlüssel zugeordnet sind. Gibt den neuen Wert des Attributs *key* zurück.

***treeName* leaves**
Gibt eine Liste aller Blattknoten des Baums zurück.

***treeName* move *parent index node* ?*node* ...?**
Macht die angegebenen Knoten zu Kindern von *parent* und fügt sie an dem durch *index* angegebenen Index in die Kinderliste des Elternknotens ein. Beachten Sie, dass der Befehl alle Knoten aus dem Baum herausnimmt, bevor er sie unter dem neuen Elternknoten einfügt, und dass er die Einfügeposition nach dem Herausnehmen und vor dem erneuten Einfügen bestimmt. Das ist wichtig, wenn ein oder mehrere Knoten an einen anderen Index verschoben werden sollen, ohne dass sich ihr Elternknoten ändert.

treeName* next *node
Gibt den rechten Geschwisterknoten von *node* zurück oder die leere Zeichenkette, wenn *node* das letzte Kind seines Elternknotens war.

treeName* numchildren *node
Gibt die Anzahl der direkten Kinder von *node* zurück.

***treeName* nodes**
Gibt eine Liste aller Knoten des Baums zurück.

treeName* parent *node
Gibt den Elternknoten von *node* zurück.

treeName* previous *node
Gibt den linken Geschwisterknoten von *node* zurück oder die leere Zeichenkette, wenn *node* das erste Kind seines Elternknotens war.

treeName* rename *node newname
Benennt den Knoten *node* in *newname* um. Es wird ein Fehler ausgelöst, wenn der Knoten entweder nicht existiert oder bereits ein Knoten mit dem Namen *newname* vorhanden ist. Das Ergebnis des Befehls ist der neue Name des Knotens.

***treeName* rootname**
Gibt den Namen des Wurzelknotens des Baums zurück.

***treeName* serialize ?*node*?**
Diese Methode serialisiert den Teilbaum, der bei *node* beginnt. Mit anderen Worten: Sie gibt einen Tcl-*value* zurück, der den bei *node* beginnenden Baum vollständig beschreibt. Dies ermöglicht beispielsweise die Übertragung von Baumobjekten (oder Teilen davon) über beliebige Kanäle, die dauerhafte Speicherung usw. Diese Methode ist zudem die Grundlage für Kopierkonstruktor und Zuweisungsoperator.

Das Ergebnis dieser Methode muss über alle Implementierungen der Baumschnittstelle hinweg semantisch identisch sein. Erst das erlaubt es, Baumdaten zwischen verschiedenen Implementierungen derselben Schnittstelle zu kopieren.

Das Ergebnis ist eine Liste, die ein Vielfaches von drei Elementen enthält. Sie ähnelt einem serialisierten Array, nur dass auf jeden Schlüssel zwei Werte folgen. Die Schlüssel sind die

Namen der Knoten im serialisierten Baum. Die beiden Werte sind, in dieser Reihenfolge, ein Verweis auf den Elternknoten und die Attributdaten.

Für den Wurzelknoten des Baums ist der Verweis auf den Elternknoten die leere Zeichenkette. Bei allen anderen Knoten ist es der Index des Elternknotens in der Liste. Es sind also ganze Zahlen größer oder gleich null, kleiner als die Länge der Liste und Vielfache von drei. Die Reihenfolge der Knoten in der Liste ist insofern wichtig, als aus ihr die Kinderlisten jedes Knotens rekonstruiert werden. Die Kinder eines Knotens müssen in der Serialisierung in derselben Reihenfolge stehen wie bei ihrem Elternknoten im Baum.

Die Attributdaten eines Knotens sind ein Dictionary, d. h. eine Liste mit gerader Länge, die ein serialisiertes Array enthält. Bei einem Knoten ohne Attributdaten ist das Dictionary die leere Liste.

Hinweis: Die aktuelle Implementierung liefert den Wurzelknoten als erstes Element der Liste, gefolgt von seinen Kindern und deren Kindern in Tiefensuche-Reihenfolge; das gilt aber nicht zwangsläufig für andere Implementierungen. Die einzige Information, auf die sich ein Leser der serialisierten Daten hinsichtlich der Struktur des Baums verlassen kann, ist, dass der Wurzelknoten durch die leere Zeichenkette für die Elternreferenz gekennzeichnet ist, dass alle anderen Knoten über den Index in der Liste auf ihren Elternknoten verweisen und dass die Kinder in derselben Reihenfolge auftreten wie in ihrem Elternknoten.

```
A possible serialization for the tree structure

            +- d
      +- a -+
root -+- b  +- e
      +- c
is

{root {} {} a 0 {} d 3 {} e 3 {} b 0 {} c 0 {}}

The above assumes that none of the nodes have attributes.
```

***treeName* set *node key* ?*value*?**

Setzt oder liest einen der Werte, die einem Knoten unter einem Schlüssel zugeordnet sind. Einem Knoten können beliebig viele solcher Werte zugeordnet sein. Wenn *value* nicht angegeben wird, gibt dieser Befehl den aktuell dem Schlüssel zugewiesenen Wert zurück; wenn *value* angegeben wird, weist dieser Befehl dem Schlüssel diesen Wert zu und gibt ihn zurück.

***treeName* size ?*node*?**

Gibt die Anzahl der Nachkommen des Knotens *node* zurück; ist kein Knoten angegeben, wird **root** angenommen.

***treeName* splice *parent from* ?*to*? ?*child*?**

Fügt einen Knoten mit dem Namen *child* als Kind des Knotens *parent* in den Baum ein. Ist *parent* gleich **root**, ist die Wurzel des Baums gemeint. Der neue Knoten wird an dem durch *from* angegebenen Index in die Kinderliste des Elternknotens eingefügt. Die Kinder von *parent* , die im Bereich der Indizes *from* und *to* liegen, werden zu Kindern von *child* gemacht. Ist *to*

nicht angegeben, ist der Standardwert **end**. Wenn für *child* kein Name angegeben wird, wird für den neuen Knoten ein Name generiert. Der generierte Name hat die Form *node***x**, wobei **x** eine Zahl ist. Der Befehl gibt den Namen des neuen Knotens zurück.

Die Argumente *from* und *to* sind reguläre Listenindizes, d. h. auch die Form "end-**n**" wird akzeptiert.

treeName* swap *node1 node2

Vertauscht die Positionen von *node1* und *node2* im Baum.

treeName* unset *node key

Entfernt einen Wert unter einem Schlüssel vom Knoten *node*. Existiert *key* nicht, tut die Methode nichts.

treeName* walk *node* ?-order *order*? ?-type *type*? *loopvar script

Durchläuft den Baum in Breiten- oder Tiefensuche, beginnend am Knoten *node*. Die Art des Durchlaufs – Breitensuche oder Tiefensuche – wird durch den Wert von *type* bestimmt; **bfs** steht für die Breitensuche, **dfs** für die Tiefensuche. Die Tiefensuche ist die Standardeinstellung. Die Reihenfolge der Durchquerung – Pre-Order, Post-Order, Both-Order oder In-Order – wird durch den Wert von *order* bestimmt; **pre** steht für "Pre-Order", **post** für "Post-Order", **both** für "Both-Order" und **in** für "In-Order". "Pre-Order" ist die Standardeinstellung.

"Pre-Order"-Durchlauf bedeutet, dass ein Elternknoten vor allen seinen Kindern besucht wird. Beispielsweise besucht eine Breitensuche, die an der Wurzel beginnt, zunächst die Wurzel, gefolgt von allen Kindern der Wurzel und anschließend von allen Enkelknoten der Wurzel. "Post-Order"-Durchlauf bedeutet, dass ein Elternknoten nach allen seinen Kindern besucht wird. "Both-Order"-Durchlauf bedeutet, dass ein Elternknoten sowohl vor *als auch* nach allen seinen Kindern besucht wird. "In-Order"-Durchlauf bedeutet, dass ein Elternknoten nach seinem ersten Kind und vor dem zweiten besucht wird. Dies ist eine Verallgemeinerung des "In-Order"-Durchlaufs für Binärbäume und führt zu dem gewünschten Ergebnis, wenn ein Binärbaum durchlaufen wird. Die Kombination einer Breitensuche mit einer In-Order-Durchquerung ist unzulässig.

Im Verlauf des Durchlaufs wird *script* an jedem Knoten ausgewertet. Die Auswertung erfolgt im Kontext des Aufrufers der Methode. Die Schleifenvariablen stehen in *loopvar*. Ist nur eine Variable angegeben, wird sie auf die ID des Knotens gesetzt. Sind zwei Variablen angegeben, d. h. *loopvar* ist eine echte Liste, wird die erste Variable auf die am Knoten ausgeführte Aktion gesetzt und die andere auf die ID des Knotens selbst. Alle Schleifenvariablen werden im Kontext des Aufrufers angelegt.

Es gibt drei mögliche Aktionen: **enter**, **leave** oder **visit**. **enter**-Aktionen finden bei Pre-Order-Durchläufen statt; **leave**-Aktionen finden bei Post-Order-Durchläufen statt; **visit** -Aktionen finden bei In-Order-Durchläufen statt. Bei einem "Both-Order"-Durchlauf wird der Befehl für jeden Knoten zweimal ausgewertet; die Aktion lautet **enter** für die erste Auswertung und **leave** für die zweite.

Hinweis: Die Aktion **enter** für einen Knoten wird immer ausgeführt, bevor der Walker die Kinder dieses Knotens untersucht. Das bedeutet, dass Änderungen, die *script* an den Kindern des Knotens vornimmt, sich unmittelbar auf den Walker und die von ihm ausgeführten Schritte auswirken.

Jede andere Manipulation, beispielsweise an Knoten, die in der Baumstruktur höher liegen (d. h. bereits besucht wurden), oder beim Verlassen des Knotens, führt zu undefinierten Ergebnissen. Diese können erfolgreich sein, zu einem Fehler führen, stillschweigend ein falsches Ergebnis berechnen oder irgendetwas dazwischen.

Abschließend eine kleine Tabelle, die die Zusammenhänge zwischen den verschiedenen Optionen und den möglichen Aktionen veranschaulicht.

```
order       type    actions         notes
-----       ----    -----           -----
pre         dfs     enter           parent before children
post        dfs     leave           parent after children
in          dfs     visit           parent between first and second
   child.
both        dfs     enter, leave    parent before and after children
-----       ----    -----           -----
pre         bfs     enter           parent before children
post        bfs     leave           parent after children
in          bfs             -- illegal --
both        bfs     enter, leave    parent before and after children
-----       ----    -----           -----
```

Beachten Sie den Befehl **::struct::tree::prune**. Mit diesem Befehl im walk-Skript lässt sich erzwingen, dass der Durchlauf die Kindknoten des aktuellen Knotens überspringt. Ist die Durchlaufreihenfolge **post** oder **in**, löst er einen Fehler aus, da diese Modi die Kinder vor ihrem Elternknoten besuchen und das Beschneiden dann sinnlos ist.

treeName **walkproc** ***node*** **?-order** ***order*****?** **?-type** ***type*****?** ***cmdprefix***

Diese Methode entspricht in allen wesentlichen Punkten der Methode **walk**, mit Ausnahme der Schnittstelle zum Benutzercode. Diese Methode ruft ein Befehlspräfix mit drei zusätzlichen Argumenten (Baum, Knoten und Aktion) auf, anstatt ein Skript auszuwerten und den Knoten über eine Schleifenvariable zu übergeben.

Änderungen in Version 2.0

Die folgenden nennenswerten Änderungen sind eingetreten:

1. Die API für den Zugriff auf Attribute und deren Werte wurde vereinfacht.

   Alle Funktionen im Zusammenhang mit dem Standardattribut "data" wurden entfernt. Dieses Standardattribut existiert nicht mehr. Bei allen Zugriffen auf Attribute muss der Name des jeweiligen Attributs angegeben werden. Diese *inkompatible* Änderung ermöglichte es uns, die Signatur aller Methoden, die mit Attributen arbeiten, zu vereinfachen.

   Insbesondere das Flag **-key** ist nicht mehr erforderlich; mehr noch, seine Verwendung ist nun untersagt. Bitte lesen Sie die Dokumentation zu den Methoden **set**, **get**, **getall**, **unset**, **append**, **lappend**, **keyexists** und **keys**, um eine Beschreibung der neuen APIs zu erhalten.
2. Die Methoden **keys** und **getall** akzeptieren nun ein optionales Muster-Argument und geben nur noch Attributdaten für Schlüssel zurück, die diesem Muster entsprechen.
3. Knoten können nun umbenannt werden. Weitere Informationen finden Sie in der Dokumentation zur Methode **rename**.

4. Die Struktur wurde um APIs für die Serialisierung und Deserialisierung von Baumobjekten sowie um eine Reihe darauf basierender Operationen (Baumzuweisung, Kopierkonstruktion) erweitert.

   Bitte lesen Sie die Dokumentation zu den Methoden **serialize**, **deserialize**, **=** und **-->** sowie die Dokumentation zum Erzeugen von Baumobjekten.

   Über das Kopieren ganzer Baumobjekte hinaus ermöglichen diese neuen APIs auch die Übertragung von Baumobjekten über beliebige Kanäle und ein einfaches dauerhaftes Speichern.
5. Die Walker-API wurde verschlankt und stärker an den Befehl **foreach** angeglichen. Im Einzelnen:
   - Die überflüssige Option **-command** wurde entfernt.
   - Ebenso die Platzhalter. Statt der Platzhalter sind zwei Schleifenvariablen anzugeben, die Knoten und Aktion aufnehmen.
   - Das alte Befehlsargument ist jetzt als Skript dokumentiert, was es auch früher schon war.
   - Dass **enter**-Aktionen aufgerufen werden, bevor der Walker die Kinder eines Knotens betrachtet, ist nun dokumentiert. Mit anderen Worten: Es ist nun offiziell zulässig, die Kinderliste eines Knotens unter *diesen* Umständen zu manipulieren. Es wurde klargestellt, dass Änderungen unter allen anderen Umständen zu undefinierten Ergebnissen führen können – von der stillschweigenden Berechnung eines falschen Ergebnisses bis hin zum Auftreten eines Fehlers.

1. Es wurde eine neue Methode, **attr**, hinzugefügt, die die Abfrage und den Abruf von Attributdaten unabhängig von der Knotenbeziehung ermöglicht.
2. Die Methode **children** wurde um die Möglichkeit erweitert, anhand eines beliebigen Filterkriteriums unter den Kindern des Knotens auszuwählen. Eine weitere Erweiterung besteht darin, dass nicht nur die direkten Kinder des Knotens, sondern der gesamte darunterliegende Baum berücksichtigt werden kann.

Beispiele

Das folgende Beispiel zeigt das Anlegen neuer Knoten:

```
mytree insert root end 0   ; # Create node 0, as child of the root
mytree insert root end 1 2 ; # Ditto nodes 1 & 2
mytree insert 0    end 3   ; # Now create node 3 as child of node 0
mytree insert 0    end     ; # Create another child of 0, with a
#                            generated name. The name is returned
#                            as the result of the command.
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: tree* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Datenstrukturen

Copyright

```
Copyright (c) 2002-2004,2012 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


struct_tree1
Stichwörter: struct_tree1, tree

Name
struct::tree_v1 – Baumobjekte anlegen und bearbeiten

Übersicht
package require Tcl 8.5 9
package require struct::tree ?**1.2.3**?
treeName option ?*arg arg* ...?
treeName **append** *node* ?*-key key*? *value*
treeName **children** *node*
treeName **cut** *node*
treeName **delete** *node* ?*node* ...?
treeName **depth** *node*
treeName **destroy**
treeName **exists** *node*
treeName **get** *node* ?*-key key*?
treeName **getall** *node*
treeName **keys** *node*
treeName **keyexists** *node* ?*-key key*?
treeName **index** *node*
treeName **insert** *parent index* ?*child* ?*child* ...?
treeName **isleaf** *node*
treeName **lappend** *node* ?*-key key*? *value*
treeName **move** *parent index node* ?*node* ...?
treeName **next** *node*
treeName **numchildren** *node*
treeName **parent** *node*
treeName **previous** *node*
treeName **set** *node* ?*-key key*? ?*value*?
treeName **size** ?*node*?
treeName **splice** *parent from* ?*to*? ?*child*?
treeName **swap** *node1 node2*
treeName **unset** *node* ?*-key key*?
treeName **walk** *node* ?*-order order*? ?*-type type*? **-command** *cmd*

Beschreibung
Der Befehl ::struct::tree erzeugt ein neues Baumobjekt mit einem zugehörigen globalen

Tcl-Befehl, dessen Name *treeName* lautet. Mit diesem Befehl lassen sich verschiedene Operationen auf dem Baum aufrufen. Er hat die folgende allgemeine Form:

treeName option ?*arg arg ...*?
Option und die *arg*s bestimmen das genaue Verhalten des Befehls.

Ein Baum ist eine Sammlung benannter Elemente, Knoten genannt, von denen einer als Wurzel ausgezeichnet ist, zusammen mit einer Beziehung ("Elternschaft"), die den Knoten eine hierarchische Struktur gibt. (Data Structures and Algorithms; Aho, Hopcroft und Ullman; Addison-Wesley, 1987). Neben der Verwaltung der Knotenbeziehungen erlaubt diese Baumimplementierung, jedem Knoten beliebig viele Werte unter Schlüsseln zuzuordnen.

Die Namen der Elemente können beliebige Zeichenketten sein.

Ein Baum ähnelt somit einem Array, weist jedoch drei wesentliche Unterschiede auf:

1. Der Zugriff auf Bäume erfolgt über einen Objektbefehl, während der Zugriff auf Arrays über Variablen erfolgt. (Das bedeutet, dass Bäume nicht lokal zu einer Prozedur sein können.)
2. Bäume weisen eine hierarchische Struktur auf, während ein Array lediglich eine ungeordnete Sammlung ist.
3. Jeder Knoten eines Baums verfügt über eine eigene Sammlung von Attributen und Werten. Dies entspricht einem Array, bei dem jeder Wert ein Dictionary ist.

Für Baumobjekte sind folgende Befehle möglich:

treeName* append *node* ?-key *key*? *value
Hängt *value* an einen der Werte an, die einem Knoten unter einem Schlüssel zugeordnet sind. Ist kein *key* angegeben, wird der Schlüssel **data** angenommen.

treeName* children *node
Gibt eine Liste der Kindknoten von *node* zurück.

treeName* cut *node
Entfernt den durch *node* festgelegten Knoten aus dem Baum, jedoch nicht dessen Kinder. Die Kinder von *node* werden zu Kindern des Elternknotens von *node*, und zwar an dem Index, an dem *node* stand.

***treeName* delete *node* ?*node* ...?**
Entfernt die angegebenen Knoten aus dem Baum. Alle Kindknoten dieser Knoten werden ebenfalls entfernt, damit keine verwaisten Knoten zurückbleiben.

treeName* depth *node
Gibt die Anzahl der Schritte vom Knoten *node* zum Wurzelknoten zurück.

***treeName* destroy**
Zerstört den Baum einschließlich seines Speichers und des zugehörigen Befehls.

treeName* exists *node
Gibt true zurück, wenn der angegebene Knoten im Baum existiert.

***treeName* get *node* ?-key *key*?**
Gibt den Wert zurück, der dem Schlüssel *key* für den Knoten *node* zugeordnet ist. Ist kein Schlüssel angegeben, wird der Schlüssel **data** angenommen.

treeName* getall *node
Gibt eine serialisierte Liste von Schlüssel-Wert-Paaren (geeignet für [**array set**]) für *node* zurück.

treeName* keys *node

Gibt eine Liste der Schlüssel von *node* zurück.

***treeName* keyexists *node* ?-key *key*?**
Gibt wahr zurück, wenn der angegebene *key* für *node* existiert. Ist kein *key* angegeben, gilt der Schlüssel **data**.

treeName* index *node
Gibt den Index von *node* in der Kinderliste seines Elternknotens zurück. Hat ein Knoten beispielsweise *nodeFoo*, *nodeBar* und *nodeBaz* in dieser Reihenfolge als Kinder, beträgt der Index von *nodeBar* 1.

***treeName* insert *parent index* ?*child* ?*child* ...??**
Fügt einen oder mehrere Knoten als Kinder des Knotens *parent* in den Baum ein. Die Knoten werden in der Reihenfolge hinzugefügt, in der sie angegeben werden. Ist *parent* gleich **root**, bezieht sich dies auf die Wurzel des Baums. Die neuen Knoten werden an dem durch *index* angegebenen Index in die Kinderliste des Knotens *parent* eingefügt. *index* kann **end** sein; dann werden die neuen Knoten hinter dem derzeit letzten Kind eingefügt.

Existieren angegebene Kinder bereits in *treeName*, werden diese Knoten von ihrer bisherigen Stelle an die von diesem Befehl angegebene neue Stelle verschoben.

Wird kein *child* angegeben, wird ein einzelner Knoten hinzugefügt, und für den neuen Knoten wird ein Name generiert. Der generierte Name hat die Form *node***x**, wobei **x** eine Zahl ist. Angegebene Namen dürfen weder Leerraum noch Doppelpunkte (":") enthalten.

Das Ergebnis dieses Befehls ist eine Liste der hinzugefügten Knoten.

treeName* isleaf *node
Gibt true zurück, wenn *node* ein Blatt des Baums ist (d. h., wenn *node* keine Kinder hat), sonst false.

treeName* lappend *node* ?-key *key*? *value
Hängt einen *value* (als Liste) an einen der *node* zugeordneten Schlüsselwerte an. Ist kein *key* angegeben, gilt der Schlüssel **data**.

***treeName* move *parent index node* ?*node* ...?**
Macht die angegebenen Knoten zu Kindern von *parent* und fügt sie an dem durch *index* angegebenen Index in die Kinderliste des Elternknotens ein. Beachten Sie, dass der Befehl alle Knoten aus dem Baum herausnimmt, bevor er sie unter dem neuen Elternknoten einfügt, und dass er die Einfügeposition nach dem Herausnehmen und vor dem erneuten Einfügen bestimmt. Das ist wichtig, wenn ein oder mehrere Knoten an einen anderen Index verschoben werden sollen, ohne dass sich ihr Elternknoten ändert.

treeName* next *node
Gibt den rechten Geschwisterknoten von *node* zurück oder die leere Zeichenkette, wenn *node* das letzte Kind seines Elternknotens war.

treeName* numchildren *node
Gibt die Anzahl der direkten Kinder von *node* zurück.

treeName* parent *node
Gibt den Elternknoten von *node* zurück.

treeName* previous *node
Gibt den linken Geschwisterknoten von *node* zurück oder die leere Zeichenkette, wenn *node*

das erste Kind seines Elternknotens war.

***treeName* set *node* ?-key *key*? ?*value*?**

Setzt oder liest einen der Werte, die einem Knoten unter einem Schlüssel zugeordnet sind. Ist kein Schlüssel angegeben, wird der Schlüssel **data** angenommen. Bei jedem Knoten, der einem Baum hinzugefügt wird, erhält der Schlüssel **data** automatisch den Wert "". Einem Knoten können beliebig viele solcher Werte zugeordnet sein. Ist *value* nicht angegeben, gibt dieser Befehl den aktuell dem Schlüssel zugewiesenen Wert zurück; ist *value* angegeben, weist dieser Befehl dem Schlüssel diesen Wert zu.

***treeName* size ?*node*?**

Gibt die Anzahl der Nachkommen des Knotens *node* zurück; ist kein Knoten angegeben, wird **root** angenommen.

***treeName* splice *parent from* ?*to*? ?*child*?**

Fügt einen Knoten mit dem Namen *child* als Kind des Knotens *parent* in den Baum ein. Ist *parent* gleich **root**, ist die Wurzel des Baums gemeint. Der neue Knoten wird an dem durch *from* angegebenen Index in die Kinderliste des Elternknotens eingefügt. Die Kinder von *parent* , die im Bereich der Indizes *from* und *to* liegen, werden zu Kindern von *child* gemacht. Ist *to* nicht angegeben, ist der Standardwert **end**. Wenn für *child* kein Name angegeben wird, wird für den neuen Knoten ein Name generiert. Der generierte Name hat die Form *node***x**, wobei **x** eine Zahl ist. Der Befehl gibt den Namen des neuen Knotens zurück.

treeName* swap *node1 node2

Vertauscht die Positionen von *node1* und *node2* im Baum.

***treeName* unset *node* ?-key *key*?**

Entfernt einen Wert unter einem Schlüssel vom Knoten *node*. Ist kein Schlüssel angegeben, wird der Schlüssel **data** angenommen.

treeName* walk *node* ?-order *order*? ?-type *type*? -command *cmd

Durchläuft den Baum in Breiten- oder Tiefensuche, beginnend am Knoten *node*. Die Art des Durchlaufs – Breitensuche oder Tiefensuche – wird durch den Wert von *type* bestimmt; **bfs** steht für die Breitensuche, **dfs** für die Tiefensuche. Die Tiefensuche ist die Standardeinstellung. Die Reihenfolge der Durchquerung – Pre-Order, Post-Order, Both-Order oder In-Order – wird durch den Wert von *order* bestimmt; **pre** steht für "Pre-Order", **post** für "Post-Order", **both** für "Both-Order" und **in** für "In-Order". "Pre-Order" ist die Standardeinstellung.

"Pre-Order"-Durchlauf bedeutet, dass ein Elternknoten vor allen seinen Kindern besucht wird. Beispielsweise besucht eine Breitensuche, die an der Wurzel beginnt, zunächst die Wurzel, dann alle Kinder der Wurzel und anschließend alle Enkel der Wurzel. "Post-Order"-Durchlauf bedeutet, dass ein Elternknoten nach allen seinen Kindern besucht wird. "Both-Order"-Durchlauf bedeutet, dass ein Elternknoten sowohl vor *als auch* nach allen seinen Kindern besucht wird. Bei der In-Order-Durchquerung wird ein Elternknoten nach seinem ersten Kind und vor dem zweiten besucht. Dies ist eine Verallgemeinerung der In-Order-Durchquerung für Binärbäume und führt zu dem gewünschten Ergebnis, wenn ein Binärbaum durchlaufen wird. Die Kombination einer Breitensuche mit der In-Order-Durchquerung ist unzulässig.

Im Verlauf der Durchquerung wird der Befehl *cmd* an jedem Knoten ausgewertet. Vor der Auswertung wird in *cmd* eine Prozentersetzung vorgenommen, genau wie in einem **bind** -Skript. Die folgenden Ersetzungen werden erkannt:

%%
Fügt ein wörtliches Zeichen % ein.

%t
Name des Baumobjekts.

%n
Name des aktuellen Knotens.

%a
Name der stattfindenden Aktion; entweder **enter**, **leave** oder **visit**. **enter**-Aktionen finden bei Pre-Order-Durchläufen statt; **leave**-Aktionen bei Post-Order-Durchläufen; **visit**-Aktionen bei In-Order-Durchläufen. Bei einem "Both-Order"-Durchlauf wird der Befehl für jeden Knoten zweimal ausgewertet; die Aktion lautet **enter** für die erste Auswertung und **leave** für die zweite.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *struct :: tree* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Datenstrukturen

Copyright


```
Copyright (c) 2002 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tar

tar

Stichwörter: tar, archive, tape archive

Name

tar – Erstellen, Entpacken und Bearbeiten von Tar-Dateien

Übersicht

package require Tcl 8.5 9
package require tar ?**0.15**?
::tar::contents *tarball* ?**-chan**? ?**-gzip**?
::tar::stat *tarball* ?*file*? ?**-chan**? ?**-gzip**?
::tar::untar *tarball args*
::tar::get *tarball fileName* ?**-chan**? ?**-gzip**?
::tar::create *tarball files args*

::tar::add *tarball files args*
::tar::remove *tarball files*

Beschreibung

Hinweis: Ab Version 0.8 unterstützen die Befehle des tar-Readers (contents, stats, get, untar) die GNU-LongName-Erweiterung (Header-Typ \'L') für lange Pfade.

Vorsicht!

Bei allen Befehlen gilt bei Verwendung von **-chan** ...

1. Es wird davon ausgegangen, dass der Kanal zum Lesen geöffnet und für binäre Eingaben konfiguriert wurde.
2. Es wird davon ausgegangen, dass sich die Kanalposition am Anfang einer gültigen TAR-Datei befindet.
3. Die Befehle *ändern* die Kanalposition, während sie ihre Aufgabe ausführen.
4. Die Befehle schließen den Kanal *nicht*.
5. Mit anderen Worten: Die Befehle hinterlassen den Kanal in einem Zustand, der höchstwahrscheinlich für die Verwendung durch weitere **tar**-Befehle ungeeignet ist. Wenn Sie dies dennoch tut, führt dies höchstwahrscheinlich zu Fehlern, fehlerhaften Daten usw. usw.
6. Es liegt in der Verantwortung des Nutzers, den Kanal wieder auf eine geeignete Position einzustellen.
7. Wenn eine Kanal-Transformation verwendet wird, die grundsätzlich nicht durchsuchbar ist, beispielsweise **gunzip**, obliegt es dem Benutzer, (a) die Transformation aufzuheben, bevor er den Kanal an eine geeignete Position zurückspringt, und (b) sie anschließend wieder anzuwenden.

Zur Unterstützung der gzip-Komprimierung:

1. Es werden Fehler ausgegeben, wenn versucht wird, aus komprimierten tar-Archiven zu lesen, während keine Komprimierungsunterstützung (d. h. **::zlib**) verfügbar ist.
2. Beim Versuch, ein unkomprimiertes tar-Archiv zu lesen, werden Fehler ausgegeben, wenn der Benutzer eine Komprimierung angefordert hat (**-gzip**).

   Beim Versuch, ein komprimiertes tar-Archiv zu lesen, werden keine Fehler ausgegeben, wenn keine Komprimierung angefordert wurde und diese unterstützt wird. In diesem Fall aktivieren die Befehle automatisch den Code, der die Komprimierung verarbeitet.
3. Beim Versuch, komprimierte TAR-Archive zu bearbeiten, werden Fehler ausgegeben. Siehe die Befehle **tar::add** und **tar::remove**. Dies wird nicht unterstützt.
4. Die Erstellung komprimierter TAR-Archive wird jedoch unterstützt, da das Archiv dabei sequenziell geschrieben wird, was eine Streaming-Komprimierung ermöglicht.

Befehle

::tar::contents *tarball* ?-chan? ?-gzip?

Gibt eine Liste der in *tarball* enthaltenen Dateien zurück. Die Reihenfolge ist nicht sortiert und hängt davon ab, in welcher Reihenfolge die Dateien im Archiv gespeichert wurden.

Wenn die Option **-chan** vorhanden ist, wird *tarball* als offener Kanal interpretiert. Es wird davon ausgegangen, dass der Kanal zum Lesen geöffnet und für binäre Eingaben konfiguriert wurde. Der Befehl schließt den Kanal *nicht*.

::tar::stat *tarball* ?file? ?-chan? ?-gzip?

Gibt ein verschachteltes Wörterbuch zurück, das Informationen zur genannten "Datei" in *tarball* enthält, oder zu allen Dateien, wenn keine angegeben wurde. Die oberste Ebene besteht aus Paaren aus Dateinamen und Informationen. Die Informationen bestehen aus einem Dict mit den Schlüsseln "**mode uid gid size mtime type linkname uname gname devmajor devminor**".

```
% ::tar::stat tarball.tar
foo.jpg {mode 0644 uid 1000 gid 0 size 7580 mtime 811903867 type
    file linkname {} uname user gname wheel devmajor 0 devminor 0}
```

Wenn die Option **-chan** vorhanden ist, wird *tarball* als offener Kanal interpretiert. Es wird davon ausgegangen, dass der Kanal zum Lesen geöffnet und für binäre Eingaben konfiguriert wurde. Der Befehl schließt den Kanal *nicht*.

::tar::untar *tarball args*

Entpackt den *Tarball*. *-file* und *-glob* beschränken das Entpacken auf Dateien, die genau mit dem angegebenen Argument übereinstimmen oder dessen Muster erfüllen. Es wird kein Fehler ausgelöst, wenn keine Dateien übereinstimmen. Gibt eine Liste der extrahierten Dateinamen und die Dateigröße zurück. Bei nicht regulären Dateien ist die Größe null. Führende Pfadtrennzeichen werden entfernt, sodass Pfade immer relativ sind.

-dir dirName
: Verzeichnis, in das extrahiert werden soll. Verwendet **pwd**, wenn kein Verzeichnis angegeben wurde.

-file fileName
: Entpackt nur die Datei mit diesem Name. Der Name wird mit dem im Archiv gespeicherten vollständigen Pfad einschließlich der Verzeichnisse abgeglichen.

-glob pattern
: Es werden nur Dateien extrahiert, die diesem "Glob"-Muster entsprechen. Das Muster wird mit dem vollständigen, im Archiv gespeicherten Pfad abgeglichen.

-nooverwrite
: Bereits vorhandene Dateien nicht überschreiben.

-nomtime
: Behält den Zeitpunkt der Dateiänderung als aktuellen Zeitpunkt bei, anstatt ihn auf den Wert im Archiv zu setzen.

-noperms
: Unter Unix sollten Sie die Dateiberechtigungen auf die aktuelle umask belassen, anstatt sie auf die im Archiv enthaltenen Werte festzulegen.

-chan
: Wenn diese Option angegeben ist, wird *tarball* als geöffneter Kanal interpretiert. Es wird davon ausgegangen, dass der Kanal zum Lesen geöffnet und für binäre Eingaben konfiguriert wurde. Der Befehl schließt den Kanal *nicht*.

```
% foreach {file size} [::tar::untar tarball.tar -glob *.jpg] {
puts "Extracted $file ($size bytes)"
}
```

::tar::get *tarball fileName* ?-chan? ?-gzip?

Gibt den Inhalt von *Dateiname* aus dem *Tarball* zurück.

```
% set readme [::tar::get tarball.tar doc/README] {
% puts $readme
}
```

Wenn die Option **-chan** vorhanden ist, wird *tarball* als offener Kanal interpretiert. Es wird davon ausgegangen, dass der Kanal zum Lesen geöffnet und für binäre Eingaben konfiguriert wurde. Der Befehl schließt den Kanal *nicht*.

Es wird ein Fehler ausgelöst, wenn der Dateiname im tar-Archiv nicht gefunden wird.

::tar::create *tarball files args*

Erstellt eine neue TAR-Datei, die die *Dateien* enthält. *Dateien* muss als einzelnes Argument festgelegt werden, das eine gültige Liste von Dateinamen darstellt.

-dereference

Normalerweise speichert **create** Links als tatsächliche Verweise auf eine Datei, die im Archiv vorhanden sein kann oder auch nicht. Wenn Sie diese Option legen, wird stattdessen die Datei gespeichert, auf die der Link verweist.

-chan

Wenn diese Option angegeben ist, wird *tarball* als offener Kanal interpretiert. Es wird davon ausgegangen, dass der Kanal zum Schreiben geöffnet und für binäre Ausgabe konfiguriert wurde. Der Befehl schließt den Kanal *nicht*.

```
% ::tar::create new.tar [glob -nocomplain file*]
% ::tar::contents new.tar
file1 file2 file3
```

::tar::add *tarball files args*

Fügt *Dateien* am Ende des vorhandenen *Tarballs* an. *Dateien* müssen als einzelnes Argument angegeben werden, das eine gültige Liste von Dateinamen darstellt.

-dereference

Normalerweise speichert **add** Links als tatsächliche Links, die auf eine Datei verweisen, die im Archiv vorhanden sein kann oder auch nicht. Wenn Sie diese Option legen, wird stattdessen die Datei gespeichert, auf die der Link verweist.

-prefix string

Normalerweise speichert **add** Dateien unter genau dem Namen, der als Argument angegeben wurde. Durch die Angabe von "-prefix" wird jedem Namen die *Zeichenkette* vorangestellt.

-quick

Die einzige sichere Methode, die Position im *Tarball* zu finden, an der neue Dateien hinzugefügt werden können, besteht darin, ihn von Anfang an zu lesen. Wenn der *Tarball* jedoch mit einer "Blocksize" von 1 geschrieben wurde (wie es bei diesem Paket der Fall ist), können Sie diese Position alternativ auch durch Suchen vom Ende aus ermitteln. Die Option "-quick" weist **add** an, Letzteres zu tun.

::tar::remove *tarball files*

Entfernt *Dateien* aus dem *Tarball*. Es wird kein Fehler ausgegeben, wenn die Datei im Tarball nicht vorhanden ist. Es sind Schreibrechte für das Verzeichnis sowie freier Speicherplatz in einer Größe erforderlich, die mindestens der Größe des Tarballs entspricht.

```
% ::tar::remove new.tar {file2 file3}
% ::tar::contents new.tar
file3
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *tar* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dateiformate

tepam

tepam_argument_dialogbox

Stichwörter: tepam_argument_dialogbox, data entry form, parameter entry form

Name

tepam::argument_dialogbox – TEPAM argument_dialogbox, Referenzhandbuch

Übersicht

package require Tcl 8.5 9
package require Tk 8.3
package require tepam ?**0.5**?
tepam::argument_dialogbox *item_name item_attributes* ?*item_name item_attributes*? ?...?
tepam::argument_dialogbox *{item_name item_attributes ?item_name item_attributes? ?...?}*

Beschreibung

Aufruf der Argument-Dialogbox

Der TEPAM-Befehl **argument_dialogbox** ist ein flexibler und einfach zu verwendender Generator für Dateneingabeformulare, der zur Verfügung steht, wenn Tk geladen wurde. Jedes Eingabefeld eines Formulars wird über ein *Dateneingabeelement* definiert, das an **argument_dialogbox** in zwei Formaten übergeben werden kann:

tepam::argument_dialogbox *item_name item_attributes ?item_name item_attributes? ?...?*

Bei diesem ersten Format wird jedes *Dateneingabeelement* über ein Paar aus zwei Argumenten definiert. Das erste Argument ist der *Elementname*, der das im Formular zu

verwendende Eingabe-Widget festlegt. Das zweite Argument, die sogenannten *Elementattribute*, legt die Variable fest, die dem Dateneingabeelement zugewiesen wird, sowie eventuelle Formatierungs- und Kontextinformationen.

Der Befehl **argument_dialogbox** gibt **ok** zurück, wenn die eingegebenen Daten (über die Schaltfläche *OK*) bestätigt und durch eine Datenprüfung validiert wurden. Wenn die eingegebenen Daten verworfen werden (über die Schaltfläche *Cancel*), gibt **argument_dialogbox cancel** zurück.

Ein kleines Beispiel veranschaulicht, wie **argument_dialogbox** eingesetzt werden kann:

```
set DialogResult [tepam::argument_dialogbox \
   -title "Itinerary selection" \
   -file {-label "Itinerary report" -variable report_file} \
   -frame {-label "Itinerary start"} \
      -comment {-text "Specify your itinerary start location"} \
      -entry {-label "City" -variable start_city -type string} \
      -entry {-label "Street" -variable start_street -type string
    -optional 1} \
      -entry {-label "Street number" -variable start_street_nbr
    -type integer -optional 1} \
   -frame {-label "Itinerary destination"} \
      -comment {-text "Specify your itinerary destination"} \
      -entry {-label "City" -variable dest_city -type string} \
      -entry {-label "Street" -variable dest_street -type string
    -optional 1} \
      -entry {-label "Street number" -variable dest_street_nbr -type
    integer -optional 1} \
   -frame {} \
   -checkbutton {-label "Don't use highways" -variable no_highway}]
```

In diesem Beispiel wird eine Dialogbox mit dem Titel *Itinerary selection* geöffnet. Ein erstes Eingabe-Widget in dieser Box ermöglicht die Auswahl einer Berichtsdatei. Es folgen zwei Rahmen, in denen jeweils der Start- und der Zielort der Route festgelegt werden. Jeder dieser Orte, die mit einem Kommentar beschrieben sind, verfügt über drei Eingabe-Widgets zur Angabe der Stadt, der Straße und der Hausnummer. Unterhalb des zweiten Rahmens befindet sich ein Checkbutton, mit dem festgelegt werden kann, ob eventuelle Autobahnen ignoriert werden sollen.

tepam::argument_dialogbox {*item_name item_attributes ?item_name item_attributes? ?...?*}

Manchmal ist es einfacher, alle Definitionen der Dateneingabeelemente in Form einer einzigen Liste an **argument_dialogbox** zu übergeben und nicht als einzelne Argumente. Das zweite von **argument_dialogbox** unterstützte Format entspricht genau dem ersten, mit dem Unterschied, dass alle Elementdefinitionen in einer einzigen Liste zusammengefasst sind, die an **argument_dialogbox** übergeben wird. Das vorangegangene Beispiel lässt sich daher auch wie folgt schreiben:

```
set DialogResult [tepam::argument_dialogbox {
   -title "Itinerary selection"
   -file {-label "Itinerary report" -variable report_file}
   ...
   -checkbutton {-label "Don't use highways" -variable no_highway} }]
```

Die Befehle **argument_dialogbox** sowie **procedure** werden aus dem Namespace tepam exportiert. Um diese Befehle ohne das Namespace-Präfix **tepam::** zu verwenden, reicht es aus, sie in den Haupt-Namespace zu importieren:

```
namespace import tepam::*

set DialogResult [argument_dialogbox \
   -title "Itinerary selection"
   ...
```

In den folgenden Unterabschnitten werden die verschiedenen Typen von Argumentelementen erläutert, die **argument_dialogbox** akzeptiert, eingeteilt in drei Gruppen. Im weiteren Verlauf dieses Dokuments wird das erste Format zur Definition von Dateneingabeelementen verwendet, wobei dieses Format jederzeit in das zweite Format umgewandelt werden kann, indem alle Argumente in eine einzige Liste zusammengefasst werden, die anschließend an **argument_dialogbox** übergeben wird.

Elemente zur Definition des Kontexts

Mit der ersten Elementgruppe lassen sich einige Kontextaspekte einer Argument-Dialogbox festlegen. Diese Elemente nehmen eine einfache Zeichenkette als Elementattribut entgegen:

```
tepam::argument_dialogbox \
   -<argument_name> string \
   ...
```

Die folgenden Elemente werden dieser Gruppe zugeordnet:

-title *string*
Der Fenstertitel der Dialogbox, der standardmäßig *Dialog* lautet, kann über das Element *-title* geändert werden:

```
tepam::argument_dialogbox \
   -title "System configuration" \
   ...
```

-window *string*
Die Argument-Dialogbox verwendet standardmäßig *.dialog* als Toplevel-Fenster des Dialogs. Dieser Pfad kann über das Element *-window* geändert werden:

```
tepam::argument_dialogbox \
   -window .dialog \
   ...
```

-parent *string*

Wird ein Elternfenster festgelegt, wird die Argument-Dialogbox neben diesem angezeigt. Ohne explizite Festlegung eines Elternfensters gilt das Toplevel-Fenster als Elternfenster.

```
tepam::argument_dialogbox \
   -parent .my_appl \
   ...
```

-context *string*

Wenn ein Kontext definiert ist, wird der Zustand der Dialogbox – d. h. die eingegebenen Daten sowie die Fenstergröße und -position – beim nächsten Aufruf der Argument-Dialogbox wiederhergestellt. Durch die Zuweisung eines Kontexts kann der Zustand der Dialogbox in ihrem Kontext gespeichert werden, um zwischen verschiedenen Verwendungen der Argument-Dialogbox zu unterscheiden.

```
tepam::argument_dialogbox \
   -context destination_definitions \
   ...
```

Formatierungs- und Anzeigeoptionen

Insbesondere bei großen, komplexen Formularen ist es wichtig, dass die verschiedenen Dateneingabe-Widgets grafisch übersichtlich angeordnet und mit Erläuterungen versehen sind, um dem Benutzer einen sofortigen und klaren Überblick zu verschaffen. Es stehen einige Elemente zur Verfügung, mit denen sich die Dialogboxen strukturieren und mit Erläuterungen versehen lassen.

Die Elemente dieser Klassifizierungsgruppe erfordern als Elementattribute eine Liste mit Definitionen, die ihrerseits Attribut-Name-Wert-Paare enthält:

```
tepam::argument_dialogbox \
   ...
   -<argument_name> {
      ?-<attribute_name> <attribute_value>?
      ?-<attribute_name> <attribute_value>?
      ?...?
   }
   ...
```

Die folgenden Elemente werden dieser Gruppe zugeordnet:

-frame *list*

Das Element *-frame* ermöglicht es, alle nachfolgenden Eingabe-Widgets in einen beschrifteten Rahmen zu packen, bis ein nächstes Rahmenelement definiert wird oder bis das letzte Eingabe-Widget definiert wurde. Es erkennt die folgenden Attribute innerhalb der Attributliste des Elements:

-label *string*

Mit der Anweisung *-label* kann eine optionale Rahmenbeschriftung festgelegt werden.

Beispiel:

```
tepam::argument_dialogbox \
   ...
   -frame {-label "Destination address"}
   ...
```

Um einen offenen Rahmen zu schließen, ohne einen neuen zu öffnen, muss der Anweisung *-frame* eine leere Liste übergeben werden.

```
tepam::argument_dialogbox \
   ...
   -frame {}
   ...
```

-sep [const {{}}]
Eingabe-Widgets können mit der Anweisung *-sep* getrennt werden, wofür keine zusätzlichen Definitionen erforderlich sind. Die zugehörige Definitionsliste muss vorhanden sein, ihr Inhalt wird jedoch ignoriert.

```
tepam::argument_dialogbox \
   ...
   -sep {}
   ...
```

-comment *string*
Kommentare und Beschreibungen können über das Attribut *-text* des Elements *-comment* hinzugefügt werden. Beachten Sie, dass jedes Eingabe-Widget selbst ebenfalls ein *-text* -Attribut für Kommentare und Beschreibungen enthalten kann. Das Element *-comment* ermöglicht es jedoch beispielsweise, eine Beschreibung zwischen zwei Rahmen einzufügen.

```
tepam::argument_dialogbox \
   ...
   -comment {-text "Specify bellow the destination address"}
   ...
```

-yscroll 0|1|auto
Mit diesem Attribut lässt sich eine eventuell vorhandene vertikale Bildlaufleiste steuern. Durch die Einstellung auf **0** wird die Bildlaufleiste dauerhaft deaktiviert, durch die Einstellung auf **1** wird sie aktiviert. Standardmäßig ist der Wert auf **auto** gesetzt. Die Bildlaufleiste wird in diesem Modus nur dann aktiviert, wenn die vertikale Größe des Dateneingabeformulars 66 % der Bildschirmhöhe überschreitet.

```
tepam::argument_dialogbox \
   ...
   -yscroll auto
   ...
```

Globale benutzerdefinierte Validierung der Daten

Mit dieser Elementgruppe lassen sich globale benutzerdefinierte Prüfungen festlegen, um die

eingegebenen Daten zu validieren.

-validatecommand *script*
Benutzerdefinierte Datenprüfungen können über Validierungsbefehle durchgeführt werden, die mit dem Element *-validatecommand* definiert werden. Beispiel:

```
tepam::argument_dialogbox \
   -entry {-label "Your comment" -variable YourCom} \
   -validatecommand {IllegalWordDetector $YourCom}
```

Der Validierungsbefehl wird im Kontext der aufrufenden Prozedur ausgeführt, sobald alle grundlegenden Datenprüfungen durchgeführt und die Datenvariablen zugewiesen wurden. Der Zugriff auf alle Daten erfolgt über die Datenvariablen. Beachten Sie, dass es auch ein Eingabe-Widget-spezifisches Attribut *-validatecommand* gibt, mit dem benutzerdefinierte Prüfungen für bestimmte Dateneingaben festgelegt werden können.

Das Attribut *-validatecommand* kann wiederholt werden, um mehrere benutzerdefinierte Prüfungen festzulegen.

-validatecommand_error_text *string*
Mit diesem Element lässt sich die Standardfehlermeldung für eine globale benutzerdefinierte Validierung des Arguments (definiert durch *-validatecommand*) überschreiben. Außerdem kann dieses Attribut wiederholt werden, falls mehrere Prüfungen deklariert sind.

-validatecommand2 *script*
siehe unten ...

-validatecommand2_error_text *string*
Diese Elemente sind hauptsächlich für den internen Gebrauch in TEPAM bestimmt.

Sie entsprechen jeweils den Elementen *-validatecommand* und *-validatecommand_error_text*. Die Validierung der Daten erfolgt jedoch nicht auf der nächsthöheren Stapelebene, sondern zwei Stapelebenen höher.

Elemente für Dateneingabe-Widgets

Dateneingabe-Widgets werden mithilfe der Widget-Elemente erstellt. Diese Elemente erfordern als Elementattribute eine Liste der Definitionen, die wiederum Attribut-Name-Wert-Paare enthält:

```
tepam::argument_dialogbox \
   ...
   -<argument_name> {
      ?-<attribute_name> <attribute_value>?
      ?-<attribute_name> <attribute_value>?
      ?...?
   }
   ...
```

Die Attributliste kann verschiedene Attribute enthalten, um ein Eingabe-Widget zu beschreiben und zu kommentieren sowie dessen eingegebenen Wert einzuschränken. Alle Eingabe-Widgets akzeptieren einen gemeinsamen Satz von Attributen, die im Abschnitt Attribute der Eingabe-Widget-Elemente beschrieben sind.

TEPAM definiert eine umfangreiche Sammlung von Eingabe-Widgets. Wenn Bedarf besteht, kann diese Sammlung um weitere anwendungsspezifische Eingabe-Widgets erweitert werden (siehe Anwendungsspezifische Eingabe-Widgets):

-entry *list*

Das Element *-entry* erzeugt das einfachste, aber universellste Dateneingabe-Widget. Es ermöglicht die Eingabe beliebiger Daten in Form von einzeiligen Zeichenfolgen.

```
tepam::argument_dialogbox \
   -entry {-label Name -variable Entry}
```

-text *list*

Das Element *-text* erzeugt ein Widget für mehrzeilige Texteingaben. Die Höhe des Widgets kann über das Attribut *-height* festgelegt werden.

```
tepam::argument_dialogbox \
   -text {-label Name -variable Text -height 5}
```

-checkbox *list*

Mit dem Element *-checkbox* wird eine Gruppe von Checkboxen erstellt. Die Anzahl der Checkboxen und ihre Optionswerte werden über eine dem Attribut *-choices* zugewiesene Liste oder über eine mit dem Attribut *-choicevariable* deklarierte Variable festgelegt:

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} -default italic}
```

Wenn die Beschriftungen der Checkboxen von den Optionswerten abweichen sollen, können diese Beschriftungen mit dem Attribut *-choicelabels* definiert werden:

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} \
              -choicelabels {Bold Italic Underline} \
              -default italic}
```

Im Gegensatz zu einer Radiobox-Gruppe ermöglicht eine Checkbox-Gruppe die gleichzeitige Auswahl mehrerer Auswahloptionen. Die Auswahl wird aus diesem Grund in der definierten Variablen in Form einer Liste gespeichert, auch wenn nur eine Auswahloption ausgewählt wurde.

-radiobox *list*

Mit dem Element *-radiobox* wird eine Gruppe von Radioboxen angelegt. Die Anzahl der Radioboxen und ihre Optionswerte werden über eine dem Attribut *-choices* zugewiesene Liste oder über eine mit dem Attribut *-choicevariable* deklarierte Variable festgelegt.

Im Gegensatz zu einer Checkbox-Gruppe erlaubt eine Radiobox-Gruppe nur die Auswahl einer einzigen Auswahloption. Der ausgewählte Optionswert wird direkt – und nicht in Form einer Liste – in der definierten Variable gespeichert.

```
tepam::argument_dialogbox \
   -radiobox {-label "Text adjustment" -variable Adjustment \
              -choices {left center right} -default left}
```

Wenn sich die Beschriftungen der Radioboxen von den Optionswerten unterscheiden sollen, können diese Beschriftungen mit dem Attribut *-choicelabels* definiert werden:

```
tepam::argument_dialogbox \
   -radiobox {-label "Text adjustment" -variable Adjustment \
              -choices {left center right} \
              -choicelabels {Left Center Right} -default left}
```

-checkbutton *list*

Das Eingabe-Widget *-checkbutton* ermöglicht das Aktivieren oder Deaktivieren einer einzelnen Auswahloption. Der in die Variable geschriebene Wert lautet entweder **0**, wenn der Checkbutton nicht aktiviert wurde, oder **1**, wenn er aktiviert wurde. Ein eventuell angegebener Vorgabewert muss ebenfalls entweder **0** oder **1** lauten.

```
tepam::argument_dialogbox \
   -checkbutton {-label Capitalize -variable Capitalize -default 1}
```

Für die Verwaltung von Auswahllisten stehen verschiedene Arten von List- und Comboboxen zur Verfügung.

-combobox *list*

Die Combobox ist eine Kombination aus einem normalen Eingabe-Widget und einer Dropdown-Liste. Die Combobox ermöglicht die Auswahl eines einzelnen Elements aus dieser Dropdown-Liste. Die Liste der verfügbaren Elemente kann entweder als Liste für das Attribut *-choices* bereitgestellt werden oder über eine Variable, die mit dem Attribut *-choicevariable* festgelegt wird.

```
tepam::argument_dialogbox \
   -combobox {-label "Text size" -variable Size -choices {8 9 10 12
    15 18} -default 12}
```

Und hier ist ein Beispiel für die Verwendung einer Variablen zur Definition der Auswahlliste:

```
set TextSizes {8 9 10 12 15 18}
tepam::argument_dialogbox \
   -combobox {-label "Text size" -variable Size -choicevariable
    TextSizes -default 12}
```

-listbox *list*

Im Gegensatz zur Combobox wird die Listbox beim Eingabe-Widget *listbox* stets angezeigt. Es kann jeweils nur ein Element ausgewählt werden, es sei denn, das Attribut *-multiple_selection* ist gesetzt. Die Höhe der Listbox kann über das Attribut *-height* festgelegt werden. Wenn die Höhe nicht explizit definiert wird, passt sich die Höhe der Listbox automatisch an die Größe der Argument-Dialogbox an. Im ersten Beispiel wird eine Variable verwendet, um die verfügbaren Auswahlmöglichkeiten zu definieren:

```
set set AvailableSizes
for {set k 0} {$k<16} {incr k} {lappend AvailableSizes [expr 1<<$k]}

tepam::argument_dialogbox \
   -listbox {-label "Distance" -variable Distance \
             -choicevariable AvailableSizes -default 6 -height 5}
```

Hier ein Beispiel für die Auswahl mehrerer Elemente. Beachten Sie, dass auch die Vorgabeauswahl mehrere Elemente enthalten kann:

```
tepam::argument_dialogbox \
   -listbox {-label "Text styles" -variable Styles \
             -choices {bold italic underline overstrike} \
             -choicelabels {Bold Italic Underline Overstrike} \
             -default {bold underline} -multiple_selection 1 \
             -height 3}
```

-disjointlistbox *list*

Anstelle einer normalen Listbox muss eine disjunkte Listbox verwendet werden, wenn die Reihenfolge der Auswahl von Bedeutung ist. Das Eingabe-Widget für disjunkte Listboxen besteht eigentlich aus zwei Listboxen: einer zur Auswahl der Elemente und einer zur Anzeige der ausgewählten Elemente in der gewählten Reihenfolge.

Disjunkte Listboxen ermöglichen stets die Auswahl mehrerer Elemente. Mit Ausnahme des Attributs *-multiple_selection* akzeptieren disjunkte Listboxen dieselben Attribute wie normale Listboxen, z. B. *-height, -choices, -choicevariable, -default*.

```
tepam::argument_dialogbox \
   -disjointlistbox {-label "Preferred scripting languages"
    -variable Languages \
             -comment "Please select your preferred languages in the
    order" \
             -choices {JavaScript Lisp Lua Octave PHP Perl Python
    Ruby Scheme Tcl} \
             -default {Tcl Perl Python}}
```

Die Datei- und Verzeichnisauswahl bilden eine weitere Gruppe von Dateneingabe-Widgets. Ein Absatz im Abschnitt Attribute der Eingabe-Widget-Elemente erläutert die widgetspezifischen Attribute, mit denen die gewünschten Dateitypen, das aktive Verzeichnis usw. festgelegt werden können.

-file *list*

Das Element *-file* erstellt eine Gruppe, die aus einem Eingabe-Widget und einer Schaltfläche besteht, über die ein Dateibrowser geöffnet werden kann. Für dieses Eingabefeld wird automatisch der Datentyp *file* ausgewählt, wenn kein Datentyp explizit über das Attribut *-type* definiert wurde.

```
tepam::argument_dialogbox \
   -file {-label "Image file" -variable ImageF \
          -filetypes {{"GIF" {*.gif}} {"JPG" {*.jpg}}} \
          -initialfile "picture.gif"}
```

-existingfile *list*
Das Element *-existingfile* erstellt eine Gruppe, die aus einem Eingabe-Widget sowie einer Schaltfläche besteht, über die ein Browser geöffnet werden kann, um eine vorhandene Datei auszuwählen. Der Datentyp *existingfile* wird für dieses Eingabefeld automatisch ausgewählt, wenn kein Datentyp explizit über das Attribut *-type* definiert wurde.

```
tepam::argument_dialogbox \
   -existingfile {-label "Image file" -variable ImageF \
                  -filetypes {{"GIF" {*.gif}} {"JPG" {*.jpg}}} \
                  -initialfile "picture.gif"}
```

-directory *list*
Das Element *-directory* erstellt eine Gruppe, die aus einem Eingabe-Widget und einer Schaltfläche besteht, über die ein Verzeichnisbrowser geöffnet werden kann. Für dieses Eingabefeld wird automatisch der Datentyp *directory* ausgewählt, wenn kein Datentyp explizit über das Attribut *-type* definiert wurde.

```
tepam::argument_dialogbox \
   -directory {-label "Report directory" -variable ReportDir}
```

-existingdirectory *list*
Das Element *-existingdirectory* erstellt eine Gruppe, die aus einem Eingabe-Widget sowie einer Schaltfläche besteht, über die ein Browser geöffnet werden kann, um ein vorhandenes Verzeichnis auszuwählen. Der Datentyp *existingdirectory* wird für dieses Eingabefeld automatisch ausgewählt, wenn kein Datentyp explizit über das Attribut *-type* definiert wurde.

```
tepam::argument_dialogbox \
   -existingdirectory {-label "Report directory" -variable ReportDir}
```

Schließlich gibt es noch eine letzte Gruppe mit einigen weiteren speziellen Dateneingabe-Widgets.

-color *list*
Die Farbauswahl besteht aus einem Eingabe-Widget sowie einer Schaltfläche, über die ein Farbauswahlfenster geöffnet werden kann. Für diesen Eingabe-Widget-Typ wird automatisch der Datentyp *color* ausgewählt, wenn kein Datentyp explizit über das Attribut *-type* definiert wurde.

```
tepam::argument_dialogbox \
   -color {-label "Background color" -variable Color -default red}
```

-font *list*
Die Schriftartauswahl besteht aus einem Eingabe-Widget sowie einer Schaltfläche, über die ein Schriftarten-Browser geöffnet werden kann. Für diesen Eingabe-Widget-Typ wird

automatisch der Datentyp *font* ausgewählt, sofern kein Datentyp explizit über das Attribut *-type* definiert wurde. Das Eingabe-Widget zeigt einen Beispieltext im Format der ausgewählten Schriftart an.

Der Schriftarten-Browser ermöglicht standardmäßig die Auswahl der vom Tk-Befehl **font families** bereitgestellten Schriftfamilien sowie einer angemessenen Auswahl verschiedener Schriftgrößen zwischen 6 und 40 Punkt. Über die Attribute *-font_families* bzw. *-font_sizes* können jeweils andere Sätze von Schriftfamilien und Schriftgrößen festgelegt werden.

Wenn über das Attribut *-default* keine Vorgabeschriftart angegeben wird, wird die Standardschriftart des Label-Widgets, das die ausgewählte Schriftart anzeigt, als vorausgewählte Schriftart verwendet. Gehört die Schriftfamilie dieses Label-Widgets nicht zu den verfügbaren Familien, wird die erste verfügbare Familie als Vorgabe verwendet. Wenn die Schriftgröße dieses Label-Widgets nicht zu den verfügbaren Größen gehört, wird die nächstgelegene verfügbare Größe als Standardgröße ausgewählt.

```
tepam::argument_dialogbox \
   -font {-label "Font" -variable Font \
          -font_sizes {8 10 12 16} \
          -default {Arial 20 italic}}
```

Attribute der Eingabe-Widget-Elemente

Alle Eingabe-Widget-Elemente akzeptieren die folgenden Attribute:

-text *string*

Etwaige Beschreibungen und Kommentare, die mit dem Attribut *-text* festgelegt werden, werden oberhalb des Eingabe-Widgets angezeigt.

```
tepam::argument_dialogbox \
   -entry {-text "Please enter your name bellow" -variable Name}
```

-label *string*

Das Attribut label erstellt links neben dem Eingabe-Widget eine Beschriftung, wobei die angegebene Zeichenfolge als Beschriftungstext verwendet wird:

```
tepam::argument_dialogbox \
   -entry {-label Name -variable Name}
```

-variable *string*

Alle Eingabe-Widgets erfordern die Angabe einer Variablen. Nachdem die eingegebenen Informationen über die Schaltfläche "OK" bestätigt wurden, werden die Daten des Eingabe-Widgets in den definierten Variablen gespeichert.

```
tepam::argument_dialogbox \
   -existingdirectory {-label "Report directory" -variable ReportDir}
```

-default *string*

Über das Attribut *-default* können eventuelle Vorgabedaten für die Eingabe-Widgets bereitgestellt werden. Der Vorgabewert wird überschrieben, wenn eine Argument-Dialogbox mit einem definierten Kontext erneut aufgerufen wird. In diesem Fall wird der bei einem

vorherigen Aufruf bestätigte Wert als Vorgabewert verwendet.

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} -default italic}
```

-optional 0|1

Daten können mit dem Attribut *-optional* als optional oder obligatorisch gekennzeichnet werden, wobei dieses Attribut entweder **0** (obligatorisch) oder **1** (optional) als Attributdaten erfordert.

Falls ein Eintrag optional ist und keine Daten eingegeben wurden – d. h. der Eintrag enthält eine leere Zeichenkette –, wird der Eintrag als undefiniert betrachtet und die zugewiesene Variable wird nicht definiert.

```
tepam::argument_dialogbox \
   -entry {-label "City" -variable start_city -type string} \
   -entry {-label "Street" -variable start_street -type string
    -optional 0} \
   -entry {-label "Street number" -variable start_street_nbr -type
    integer -optional 1} \
```

-type *string*

Wenn der Datentyp mit dem Attribut *-type* definiert wird, führt die Argument-Dialogbox nach der Bestätigung der eingegebenen Werte und vor dem Schließen der Dialogbox automatisch eine Datentypprüfung durch. Wenn ein typinkompatibler Wert festgestellt wird, erscheint ein Fehlermeldungsfenster, und der Benutzer kann den Wert korrigieren.

Die Argument-Dialogbox akzeptiert alle Typen, die im TEPAM-Paket festgelegt wurden und die auch von tepam::procedure verwendet werden (siehe das *Referenzhandbuch zu tepam::procedure*).

Einige Eingabe-Widgets wie die Datei- und Verzeichnis-Widgets sowie die Farb- und Schriftart-Widgets legen den Standarddatentyp automatisch fest, wenn kein Typ explizit mit dem Attribut *-type* angegeben wurde.

```
tepam::argument_dialogbox \
   -entry {-label "Street number" -variable start_street_nbr -type
    integer} \
```

-range *string*

Werte können mit dem Attribut *-range* eingeschränkt werden. Der gültige Bereich wird durch eine Liste definiert, die den kleinsten und den größten gültigen Wert enthält.

Das Attribut *-range* darf nur für numerische Argumente wie Ganzzahlen und Gleitkommazahlen verwendet werden.

```
tepam::argument_dialogbox \
   -entry {-label Month -variable Month -type integer -range {1 12}}
```

-validatecommand *string*

Benutzerdefinierte Validierungen von Argumentwerten können über spezifische Validierungsbefehle durchgeführt werden, die mit dem Attribut *-validatecommand* definiert sind. Der angegebene Validierungsbefehl kann ein vollständiges Skript sein, in dem das Muster *%P* als Platzhalter für den zu validierenden Wert dient.

```
tepam::argument_dialogbox \
   -entry {-label "Your comment" -variable YourCom \
           -validatecommand "IllegalWordDetector %P"}
```

Zwar dient dieses benutzerdefinierte Attribut zur Validierung eines bestimmten Arguments, doch gibt es auch ein globales Attribut zur Datenvalidierung (*-validatecommand*), mit dem Validierungen durchgeführt werden können, die mehrere Argumente umfassen.

-validatecommand_error_text *string*
Mit diesem Attribut lässt sich die Standardfehlermeldung für eine benutzerdefinierte Validierung des Arguments (definiert durch *-validatecommand*) überschreiben.

Einige weitere Attribute werden sowohl von den List- und Comboboxen als auch von den Radio- und Checkbuttons unterstützt.

-choices *string*
Auswahllisten können direkt mit dem Attribut *-choices* definiert werden. Diese Art der Definition von Auswahllisten eignet sich besonders für kleinere, feste Auswahlen.

```
tepam::argument_dialogbox \
   -listbox {-label "Text styles" -variable Styles \
             -choices {bold italic underline} -default underline
```

-choicelabels *string (only check and radio buttons)*
Wenn die Beschriftungen der Check- und Radioboxen von den Optionswerten abweichen sollen, können diese mit dem Attribut *-choicelabels* definiert werden:

```
tepam::argument_dialogbox \
   -checkbox {-label "Font sytle" -variable FontStyle \
              -choices {bold italic underline} \
              -choicelabels {Bold Italic Underline}
```

-choicevariable *string*
Eine weitere Möglichkeit zur Definition der Auswahllisten ist die Verwendung des Attributs *-choicevariable*. Diese Methode zur Definition von Auswahllisten eignet sich besonders für umfangreiche und gegebenenfalls variable Auswahlen.

```
set TextSizes {8 9 10 12 15 18}
tepam::argument_dialogbox \
   -combobox {-label "Text size" -variable Size -choicevariable
    TextSizes}
```

-multiple_selection 0|1
Das Listbox-Element (**-listbox**) erlaubt standardmäßig die Auswahl von nur einem Element. Durch Setzen des Attributs *-multiple_selection* auf **1** können mehrere Elemente ausgewählt werden.

```
tepam::argument_dialogbox \
   -listbox {-label "Text styles" -variable Styles \
             -choices {bold italic underline} -default underline \
             -multiple_selection 1 -height 3}
```

Die Widgets zur Datei- und Verzeichnisauswahl unterstützen einige zusätzliche Attribute.

-filetypes *string*
Das Dateityp-Attribut wird von den Elementen **-file** und **-existingfile** verwendet, um die Dateiendungen festzulegen, nach denen der Dateibrowser sucht.

```
tepam::argument_dialogbox \
   -file {-label "Image file" -variable ImageF \
          -filetypes {{"GIF" {*.gif}} {"JPG" {*.jpg}}}}
```

-initialfile *string*
Die von den Dateibrowsern der Widgets **-file** und **-existingfile** verwendete Ausgangsdatei ist standardmäßig die mit dem Attribut *-default* definierte Datei, es sei denn, es wird eine Datei mit dem Attribut *-initialfile* angegeben.

```
tepam::argument_dialogbox \
   -file {-variable ImageF -initialfile "picture.gif"}
```

-activedir *string*
Das Attribut *-activedir* überschreibt das standardmäßige aktive Suchverzeichnis, das von den Dateibrowsern aller Datei- und Verzeichnis-Eingabe-Widgets verwendet wird. Das standardmäßige aktive Suchverzeichnis wird durch das Verzeichnis einer angegebenen Startdatei (*-initialfile*) definiert, sofern eine solche definiert ist; andernfalls durch das Verzeichnis der Vorgabedatei bzw. des Vorgabeverzeichnisses, das mit dem Attribut *-default* festgelegt wird.

```
tepam::argument_dialogbox \
   -file "-variable ImageF -activedir $pwd"
```

Schließlich gibt es noch ein letztes Attribut, das von einigen Widgets unterstützt wird:

-height *string*
Alle Widgets, die eine Auswahlliste enthalten (**-listbox**, **-disjointlistbox**, **-font**) sowie das mehrzeilige Widget **-text** akzeptieren das Attribut *-height*, das die Anzahl der angezeigten Zeilen der Auswahllisten festlegt.

```
tepam::argument_dialogbox \
   -listbox {-label "Text size" -variable Size \
             -choices {8 9 10 12 15 18} -default 12 -height 3}
```

Wenn keine Höhe explizit angegeben wurde, wird die Höhe des Widgets dynamisch an die Größe der Argument-Dialogbox angepasst.

Anwendungsspezifische Eingabe-Widgets

Ein anwendungsspezifisches Eingabe-Widget kann der Argument-Dialogbox zur Verfügung

gestellt werden, indem eine entsprechende Prozedur zum Namespace tepam hinzugefügt wird. Diese Prozedur verfügt über drei Argumente: Das erste ist der Pfad zum Widget, das zweite ein Unterbefehl und das dritte Argument dient verschiedenen Zwecken:

```
proc tepam::ad_form(<WidgetName>) {W Command {Par ""}} {
   upvar Option Option; # if required
   variable argument_dialogbox; # if required
   switch $Command {
      "create" <CreateCommandSequence>
      "set_choice" <SetChoiceCommandSequence>
      "set" <SetCommandv>
      "get" <GetCommandSequence>
   }
}
```

Argument_dialogbox übernimmt die Verwaltung der Attribute *-label* und *-text* für alle Eingabe-Widgets. Für jedes Dateneingabe-Widget erstellt er einen Rahmen, in den die Komponenten des Dateneingabe-Widgets eingefügt werden können. Der Pfad zu diesem Rahmen wird über das Argument *W* angegeben.

Die Prozedur des Eingabe-Widgets muss drei obligatorische und einen optionalen Befehl unterstützen, die über das Argument *Command* ausgewählt werden:

create
: Die Eingabe-Widget-Prozedur wird ein erstes Mal mit dem Befehl *create* aufgerufen, um das Dateneingabe-Widget zu erstellen.

  Die von **argument_dialogbox** für die Eingabe-Widgets bereitgestellten Rahmen sind standardmäßig nur in X-Richtung ausdehnbar. Um sie auch in Y-Richtung ausdehnbar zu machen, beispielsweise für ausdehnbare Listboxen, muss beim Erstellen eines Eingabe-Widgets der Befehl **ad_form(make_expandable) $W** ausgeführt werden.

set_choice
: Die Prozedur des Eingabe-Widgets wird nur dann mit dem Befehl *set_choice* aufgerufen, wenn das Attribut *-choices* oder *-choicevariable* festgelegt wurde. Der Befehl ist daher nur für List- und Comboboxen relevant.

  Die Liste der verfügbaren Auswahlmöglichkeiten, die entweder mit dem Attribut *-choices* oder *-choicevariable* festgelegt wird, wird über das Argument *Par* an die Eingabe-Widget-Prozedur übergeben. Diese Liste kann zur Initialisierung einer Liste der verfügbaren Auswahlmöglichkeiten verwendet werden.

set
: Wenn ein Vorgabewert entweder über das Attribut *-default* oder über einen vorangehenden Aufruf definiert wird, wird die Prozedur des Eingabe-Widgets mit dem Befehl *set* aufgerufen. Das Argument *Par* enthält in diesem Fall den Wert, auf den das Eingabe-Widget initialisiert werden soll.

get
: Der Befehl *get* der Eingabe-Widget-Prozedur muss den aktuellen Wert des Eingabe-Widgets zurückgeben.

Eventuell angegebene Attribute der Eingabe-Widget-Elemente stehen über die Array-Variable **Option** der aufrufenden Prozedur zur Verfügung. Auf diese Variable kann innerhalb der Eingabe-Widget-Prozedur über den Befehl **upvar** zugegriffen werden.

Möglicherweise müssen bestimmte Informationen in einer Variablen gespeichert werden. Zu diesem Zweck muss die Array-Variable **argument_dialogbox** zusammen mit Array-Indizes verwendet werden, die mit "*$W,*" beginnen, z. B. *argument_dialogbox($W,values)*.

Beispiele für Eingabe-Widget-Prozeduren werden direkt in der Quelldatei des TEPAM-Pakets bereitgestellt, die die Standard-Eingabe-Widget-Prozeduren festlegt. Die einfachste Prozedur ist die für das grundlegende *entry*-Widget:

```
proc tepam::ad_form(entry) {W Command {Par ""}} {
   switch $Command {
      "create" {pack [entry \$W.entry] -fill x \
                        -expand yes -pady 4 -side left}
      "set" {\$W.entry insert 0 $Par}
      "get" {return [\$W.entry get]}
   }
}
```

Es ist zudem möglich, auf eine bereits vorhandene Eingabe-Widget-Prozedur zurückzugreifen, um eine neue, spezifischere abzuleiten. Das *radiobox*-Widget wird beispielsweise verwendet, um ein neues Eingabe-Widget zu erstellen, das die Auswahl zwischen *left*, *center* oder *right* ermöglicht. Das ursprüngliche *radiobox*-Widget wird unmittelbar nach dem *create*-Befehl mit dem *set_choice*-Befehl aufgerufen, um die feste Liste der Auswahloptionen festzulegen.

```
proc tepam::ad_form(rcl) {W Command {Par ""}} {
   set Res [ad_form(radiobox) $W $Command $Par]
   if {$Command=="create"} {
      ad_form(radiobox) $W set_choice {left center right}
   }
   return $Res
}
```

Bitte sehen Sie in der Quelldatei des TEPAM-Pakets nach, um weitere und komplexere Beispiele für Prozeduren des Eingabe-Widgets zu finden.

Variablen

Der Befehl **argument_dialogbox** verwendet zwei Variablen innerhalb des Namespaces **::tepam**:

argument_dialogbox
: Anwendungsspezifische Eingabe-Widget-Prozeduren können diese Array-Variable nutzen, um ihre eigenen Daten zu speichern, wobei sie den an die Prozedur übergebenen Widget-Pfad als Index verwenden, z. B. *argument_dialogbox($W,)*.

last_parameters
: Diese Array-Variable wird von einer Argument-Dialogbox nur dann verwendet, wenn deren Kontext über das Attribut *-context* festgelegt wurde. Die Position und Größe der Argument-Dialogbox sowie die darin eingegebenen Daten werden in dieser Variablen

gespeichert, wenn die Daten bestätigt und das Formular geschlossen werden. Dadurch kann das Formular seinen vorherigen Zustand wiederherstellen, sobald es erneut aufgerufen wird.

Um die gespeicherten Parameter nicht nur in der aktuellen Anwendungssitzung, sondern auch in einer anderen wiederzuverwenden, reicht es aus, den Inhalt der Array-Variablen **last_parameter** in einer Konfigurationsdatei zu speichern, die beim nächsten Start der Anwendung geladen wird.

Kategorie
Eingabeformular für Argumente, Megawidget

Copyright

```
Copyright (c) 2009-2013, Andreas Drollinger
```


tepam_doc_gen
Stichwörter: tepam_doc_gen, automatic documentation, documentation, procedure documentation

Name
tepam::doc_gen – TEPAM DOC-Generierung, Referenzhandbuch

Übersicht
package require Tcl 8.5 9
package require tepam 0.5
package require tepam::doc_gen ?**0.1.3**?
tepam::doc_gen::generate ?*-format format*? ?*-style style*? ?*-header_footer*? ?*-dest_file dest_file*? *name*
tepam::doc_gen::patch ?*-format format*? ?*-style style*? ?*-search_pattern search_pattern*? ?*-src_string src_string* | *-src_file src_file*? ?*-dest_file dest_file*? ?*name*?

Beschreibung
Dieses Paket erstellt Dokumentationen zu TEPAM-Prozeduren (Prozeduren, die mit tepam::procedure deklariert wurden). Die Dokumente werden im klassischen UNIX-Dokumentstil unter Verwendung der folgenden Dokumentabschnitte erstellt: Name, Synopsis, Description, Arguments und Example. **TEPAM Doc Gen** bietet Unterstützung für verschiedene Dokumentformate. Bei Bedarf kann die Unterstützung für weitere Formate hinzugefügt werden.

Das Paket **TEPAM Doc Gen** stellt die folgenden Befehle bereit:

tepam::doc_gen::generate ?-format *format*? ?-style *style*? ?-header_footer? ?-dest_file *dest_file*? *name*

Dieser Befehl generiert die Dokumentation für eine angegebene Prozedur (*name*) in einem der unterstützten Formate (TXT, HTML, POD (Perl Doc), DT (TclLib DocTool)) oder in einem benutzerdefinierten Format. Das Format wird über ?format? angegeben. Das Flag ?-header_footer? fügt der Dokumentation Dateikopf und -fuß hinzu. Wenn ?dest_file? angegeben ist, wird die Dokumentation in einer Datei gespeichert (Dateikopf und -fuß werden in diesem Fall automatisch hinzugefügt) und der Dateiname zurückgegeben. Andernfalls gibt **generate** die Dokumentationszeichenfolge zurück.

tepam::doc_gen::patch ?-format *format*? ?-style *style*? ?-search_pattern *search_pattern*? ?-src_string *src_string* | -src_file *src_file*? ?-dest_file *dest_file*? ?name?

Dieser Befehl fügt Prozedurdokumentationen an den Stellen in ein bestehendes Masterdokument ein, die durch Einfügeplatzhalter gekennzeichnet sind, welche dem Muster ?search_pattern? entsprechen. Das vorhandene Masterdokument wird entweder als Daten über das Argument (?src_string?) oder über eine Datei (?src_file?) bereitgestellt. Das endgültige Dokument wird von **patch** zurückgegeben, wenn keine Zieldatei definiert ist (?dest_file?). Andernfalls wird das Dokument in der angegebenen Datei gespeichert, und die Anzahl der Einfügeplatzhalter, die erfolgreich verarbeitet werden konnten, wird zurückgegeben.

Standardmäßig werden alle Einfügeplatzhalter des Masterdokuments verarbeitet. Durch die Angabe des Arguments ?name? wird das Einfügen der Dokumentation auf eine bestimmte Prozedur beschränkt.

Argumente

?-format *format*?

Legt das Dokumentationsformat fest. **TEPAM Doc Gen** unterstützt die folgenden Formate:
- TXT – Textformat (Standard)
- HTML
- POD – Perl Plain Old Documentation Format (PerlPOD)
- DT – TclLib DocTool-Format

Im Abschnitt Unterstützung für neue Dokumentformate hinzufügen wird erläutert, wie die Unterstützung für weitere Formate hinzugefügt werden kann.

?-style *style*?

Die Dokumentation wird standardmäßig im Tcl-Stil generiert (z. B. **command arg1 arg2 ...**). Eine Dokumentation im C-Stil kann generiert werden, indem Sie dieses Argument auf "C" setzen (z. B. **command(arg1,arg2,...)**).

?-dest_file *dest_file*?

Wenn ?dest_file? definiert ist, wird die Dokumentation in die angegebene Zieldatei geschrieben. Andernfalls geben die Befehle **generate** und **patch** die Dokumentationszeichenfolge zurück.

***name* / ?name?**

Dies ist der Name der Prozedur, für die die Dokumentation erstellt werden soll. Für **generate** ist es ein obligatorisches Argument, für **patch** ein optionales.

?-header_footer?

Generate fügt der generierten Prozedurdokumentation Dateikopf und -fuß nur dann hinzu, wenn eine Datei generiert wird. Mit dem Flag ?-header_footer? werden Kopf und Fuß auch dann generiert, wenn **generate** die Dokumentation als Zeichenkette zurückgibt.

?-src_string *src_string* | -src_file *src_file*?

Patch fügt Prozedurdokumentationen in ein bestehendes Dokument ein, das entweder als Zeichenfolge an das Argument (?src_string?) oder als Datei (?src_file?) übergeben wird. Eines dieser beiden Argumente muss festgelegt werden.

?-search_pattern *search_pattern*?

Das Argument ?search_pattern? definiert den in einem Dokument verwendeten

Einfügeplatzhalter für die Dokumentation. Es handelt sich um einen regulären Ausdruck, der von **regexp** akzeptiert wird, und er muss einen in Klammern gesetzten Teilausdruck enthalten, der den Namen der Prozedur angibt, für die die Dokumentation eingefügt werden soll.

Das Standardmuster für den Einfügeplatzhalter lautet *\{!(.*?)!\}*, was bedeutet, dass der Name der Prozedur zwischen *{!* und*!}* eingebettet wird. Der Abschnitt Beispiele enthält ein Beispiel für ein benutzerdefiniertes Einfügeplatzhaltermuster.

Vordefinierte Dokumentformate

TEPAM Doc Gen definiert die folgenden Dokumentformate vor:

Txt – Textformat

Die Dokumentation wird in einem einfachen Textformat erstellt, wenn dieses Format ausgewählt wird. Das Format kann über die folgende Variable angepasst werden:

tepam::doc_gen::Option(TXT,MaxLineLength)
Vorgabewert: 80

Diese Variable legt die Grenze für den Zeilenumbruch (Zeichenposition) fest.

HTML – HTML-Format

TEPAM Doc Gen generiert CSS-formatierte HTML-Dateien. Die HTML-Dokumentation kann über die folgende Variable angepasst werden:

tepam::doc_gen::Option(HTML,CssFile)
Vorgabewert: "*tepam_doc_stylesheet.css*"

Diese Variable legt die CSS-Stylesheet-Datei fest, auf die in den generierten HTML-Dateien verwiesen wird.

Das CSS-Stylesheet kann angepasst werden, um die Formatierung der Dokumentation zu ändern. Ein guter Ausgangspunkt für die Erstellung eines angepassten CSS-Stylesheets ist die CSS-Datei, die dem Beispiel bzw. der Demo von **TEPAM Doc Gen** beiliegt. Die HTML-Dokumentation verwendet die folgenden CSS-Klassen-Styles:

- *h1.tepam_page_title* – Titel der Seite. Wird von **generate** nur verwendet, wenn eine Datei erstellt wird oder wenn Kopf und Fuß generiert werden (Flag ?-header_footer? gesetzt).
- *div.tepam_command_help* – Dokumentationscontainer. Die gesamte Dokumentation der Prozeduren befindet sich in diesem Container.
- *p.tepam_section_title* – Abschnittstitel (z. B. *Name*, *Synopsis*, *Description*, …)
- *p.tepam_sub_section_title* – Titel eines Unterabschnitts (dient zur Trennung der Dokumentation mehrerer Unterprozeduren)
- *p.tepam_name* – Abschnitt "Name"
- *p.tepam_synopsis* – Abschnitt "Synopsis"
- *p.tepam_description* – Einzelner Beschreibungsabsatz
- *ul.tepam_description_list* – Element einer HTML-Aufzählungsliste (ungeordnete Liste) innerhalb des Beschreibungsabschnitts
- *dt.tepam_argument* – Ein Element einer HTML-Beschreibungsliste, das zur Auflistung der

Argumente der Prozeduren dient
- *p.tepam_argument_description* – Absatz zur Beschreibung des Arguments
- *p.tepam_argument_attribute* – Zeile mit einem Argumentattribut
- *pre.tepam_example* – Beispielabschnitt

Pod – Perl-Dokumentationsformat

Die Dokumentation wird im Perl Plain Old Documentation-Format (PerlPOD) erstellt, wenn dieses Format ausgewählt wurde.

Dt – tcllib doctools-Format

Die Dokumentation wird im Tcllib-DocTools-Format erstellt, wenn dieses Format ausgewählt ist.

Unterstützung für neue Dokumentformate hinzufügen

Die Unterstützung für ein neues Dokumentformat kann hinzugefügt werden, indem im Namespace **tepam::doc_gen** eine Reihe von Prozeduren definiert wird, die die verschiedenen Komponenten des Dokuments generieren.

Das folgende Dokumentationslisting enthält Token, die sich auf die verschiedenen Prozeduren zur Dokumentenerstellung beziehen:

```
     <01>
<03> <20s>   NAME<20e>
     <30s>       message_box - Displays text in a message box<30e>
     <20s>   SYNOPSYS<20e>
     <40s>       message_box [-mtype <mtype>] <text><40e>
     <20s>   DESCRIPTION<20e>
     <21s>     message_box<21e>
     <54s>       message_box [-mtype <mtype>] <text><54e>
     <50s>       This procedure allows displaying a text in an
   message box. The following
                 message types are supported:<50e>
<51> <53s>       * Info<53e>
     <53s>       * Warning<53e>
     <53s>       * Error<53e>
     <52>
     <50s>       If the text parameter is use multiple times the
   different texts are
                 concatenated to create the message text.<50e>
     <20s>   ARGUMENTS<20e>
<60> <62s>       [-mtype <mtype>]<62e>
<63> <65s>          Message type<65e>
     <66s>          Default: "Warning"<66e>
     <66s>          Multiple: yes<66e>
     <66s>          Choices: Info, Warning, Error<66e>
     <64>
     <62s>       <text><62e>
<63> <65s>          One or multiple text lines to display<65e>
     <66s>          Type: string<66e>
```

```
     <66s>          Multiple: yes<66e>
     <64><61>
     <20s>   EXAMPLE<20e>
<70> <72s>       message_box "Please save first the document"<72e>
     <73s>       -> 1<73e>
     <71><04>
     <02>
```

Es gibt zwei Arten von Prozeduren zur Dokumentenerstellung:

Prozeduren zur Inhaltserzeugung (z. B. <40s>...<40e>)
Diese Prozeduren generieren Dokumentinhalt auf der Grundlage des Textes, der als Prozedurargument übergeben wird. Die obige Auflistung enthält zwei Token für diese Prozeduren, die den Anfang und das Ende des generierten Inhalts kennzeichnen.

Prozeduren zur Steuerungserzeugung (z. B. <03>)
Diese Prozeduren erzeugen Steuerkonstrukte, beispielsweise den Prolog- und Epilog-Code für Listen, Abschnitte usw. Diese Prozeduren haben keine Argumente.

Die folgende Gruppe von Prozeduren muss definiert werden, um ein neues Dokumentformat zu unterstützen:

***01* - gen($Format,Header) {*Text*}**
Wird nur aufgerufen, wenn **doc_gen** eine Datei erzeugt oder mit dem Flag ?-header_footer? aufgerufen wird. Die Prozedur erstellt den Dateikopf. Der übergebene Parameter ist der Name der Prozedur, für die die Dokumentation erstellt werden soll.

***02* - gen($Format,Footer) {*Text*}**
Wird nur aufgerufen, wenn **doc_gen** eine Datei erzeugt oder mit dem Flag ?-header_footer? aufgerufen wird. Die Prozedur erstellt den Dateifuß.

***03* - gen($Format,Begin) {}**
Erzeugt den Prolog (Präambel) der Dokumentation

***04* - gen($Format,End) {}**
Erzeugt den Epilog der Dokumentation

***20* - gen($Format,SectionTitle) {*Text*}**
Erzeugt eine Abschnittsüberschrift (z. B. *Name*, *Synopsis*, *Description*, ...). Der Rohtext der Überschrift wird als Parameter übergeben.

***21* - gen($Format,SubSectionTitle) {*Text*}**
Erzeugt eine Überschrift für einen Unterabschnitt. Unterabschnitte werden verwendet, wenn für mehrere Unterbefehle eine einzige Dokumentation erstellt wird, um diese voneinander abzugrenzen. Der Rohtext der Überschrift wird als Parameter übergeben.

***30* - gen($Format,Name) {*Text*}**
Erzeugt den Abschnitt "Name" (ohne Titel). Der Rohtext des Abschnitts wird als Parameter übergeben.

***40* - gen($Format,Synopsis) {*Text*}**
Erzeugt den Abschnitt "Synopsis" (ohne Titel). Der als Parameter übergebene Abschnittstext ist vorformatiert (die Argumentzeichenfolgen werden von **gen($Format,ArgumentString)** generiert).

***50* - gen($Format,Description) {*Text*}**

Erzeugt einen Beschreibungsabsatz. Der Rohtext des Absatzes wird als Parameter übergeben.

***51* - gen($Format,DescriptionListBegin) {}**
Erzeugt den Prolog einer Aufzählungsliste (ungeordneten Liste) innerhalb des Beschreibungsabschnitts. Dieser Prolog ist in der Regel der Startcode einer Listenstruktur.

***52* - gen($Format,DescriptionListEnd) {}**
Erzeugt den Epilog einer Aufzählungsliste (ungeordneten Liste) innerhalb des Beschreibungsabschnitts. Dieser Epilog ist in der Regel der Abschlusscode einer Listenstruktur.

***53* - gen($Format,DescriptionListItem) {*Text*}**
Erzeugt einen Texteintrag in einer ungeordneten Beschreibungsliste (Aufzählungsliste). Der Rohtext des Eintrags wird als Parameter übergeben.

***54* - gen($Format,DescriptionSynopsis) {*Text*}**
Erzeugt die Synopsis-Zeile am Anfang des Beschreibungsabschnitts. Der Befehl kann eine leere Zeichenkette zurückgeben, wenn an dieser Stelle keine Synopsis-Zeile erforderlich ist.

Einige Formate (z. B. Tcl DocTools) erfordern, dass die Synopsis-Zeile im Beschreibungsabschnitt definiert wird, um daraus automatisch den Synopsis-Abschnitt zu erstellen. Der als Parameter übergebene Abschnittstext ist vorformatiert (die Argumentzeichenfolgen werden von **gen($Format,ArgumentString)** generiert).

***60* - gen($Format,ArgumentListBegin) {}**
Erzeugt den Prolog der Argumentliste (Definitionsliste ohne Aufzählungszeichen). Dieser Prolog ist in der Regel der Startcode einer Definitionsliste.

***61* - gen($Format,ArgumentListEnd) {}**
Erzeugt den Epilog der Argumentliste. Dieser Epilog ist in der Regel die Abschlusszeichenfolge einer Listenstruktur.

***62* - gen($Format,ArgumentListItem) {Name IsOptional IsNamed Type}**
Erzeugt eine Argumentzeile innerhalb der Argumentliste. Dieser Befehl kann auf **gen($Format,ArgumentDetailBegin)** zurückgreifen, da die Parameter identisch sind.

***63* - gen($Format,ArgumentDetailBegin) {}**
Erzeugt den Prolog (Präambel) der Argumentdetails.

***64* - gen($Format,ArgumentDetailEnd) {}**
Erzeugt den Epilog der Argumentdetails

***65* - gen($Format,ArgumentDescription) {*Text*}**
Erzeugt die Argumentbeschreibung (ein Absatz).

***66* - gen($Format,ArgumentAttribute) {*Text*}**
Erzeugt die Zeichenkette für ein einzelnes Argumentattribut. Der Befehl wird für jedes Attribut einzeln aufgerufen.

***70* - gen($Format,ExampleBegin) {}**
Erzeugt den Prolog (Präambel) des Beispielabschnitts

***71* - gen($Format,ExampleEnd) {}**
Erzeugt den Epilog des Beispielabschnitts

***72* - gen($Format,ExampleCommandLine) {*Text*}**
Erzeugt im Beispielabschnitt eine einzelne Befehlszeile. Der Befehl wird für jede einzelne Befehlszeile aufgerufen.

***73* - gen($Format,ExampleResultLine) {*Text*}**

Erzeugt eine Befehlsergebniszeile

***80* - gen($Format,ArgumentString) {Name IsOptional IsNamed Type}**

Erzeugt den Teil der Befehlszeile oder der Synopsis, der sich auf ein bestimmtes Argument bezieht. Die erzeugte Zeichenkette muss angeben, ob ein Argument optional und benannt ist und ob es sich um ein Flag handelt.

Dieser Prozedur werden die folgenden Parameter übergeben:

Name

Name des Arguments

IsOptional

Falls wahr (=**1**), ist das Argument optional, was durch die generierte Zeichenfolge kenntlich gemacht werden sollte (beispielsweise durch Einfügen des Arguments in Klammern {[]} oder in Fragezeichen '?'):

```
gen(TXT,ArgumentString) mtype 1 0 string -> "[mtype]"
```

IsNamed

Wenn dieser Wert wahr ist (=**1**), handelt es sich bei einem Argument um ein benanntes Argument (Option). Die generierte Zeichenfolge sollte in diesem Fall den Namen des Arguments bzw. der Option enthalten, gefolgt vom Argument selbst:

```
gen(TXT,ArgumentString) mtype 0 1 string -> "-mtype <mtype>"
```

Benannte Argumente können ebenfalls optional sein:

```
gen(TXT,ArgumentString) mtype 1 1 string -> "[-mtype <mtype>]"
```

Type

Gibt den Typ des Arguments an. Wenn der Typ auf **none** gesetzt ist, handelt es sich bei dem Argument um ein Flag, das durch die generierte Zeichenfolge gekennzeichnet werden muss. Beispiel:

```
gen(TXT,ArgumentString) close 1 1 none -> "[-close]"
```

Beispiele

Tepam::doc_gen::generate

Das Paket **TEPAM Doc Gen** lässt sich erkunden, indem die Dokumentation des Befehls **tepam::doc_gen::generate** erzeugt wird. Das folgende Beispiel erzeugt das Dokument im Textformat (Standardformat):

```
tepam::doc_gen::generate tepam::doc_gen::generate
```

Das folgende Beispiel erzeugt die Dokumentation im HTML-Format:

```
tepam::doc_gen::generate -format HTML tepam::doc_gen::generate
```

Das Flag ?header_footer? fügt zusätzlich Dateikopf und -fuß hinzu:

```
tepam::doc_gen::generate -format HTML -header_footer
    tepam::doc_gen::generate
```

Die Dokumentation kann direkt in einer Datei gespeichert werden. Dateikopf und -fuß werden auf diese Weise automatisch generiert:

```
tepam::doc_gen::generate -format HTML -dest_file doc_gen.html
    tepam::doc_gen::generate
```

Die generierte HTML-Datei verweist auf eine CSS-Stylesheet-Datei (Vorgabewert: tepam_doc_stylesheet.css). Um die HTML-Datei korrekt anzuzeigen, muss diese CSS-Stylesheet-Datei in das Verzeichnis der generierten HTML-Datei kopiert werden.

Das Tcl-DOC-Tools-Format kann als Zwischenformat zur Erzeugung anderer Formate, beispielsweise HTML, verwendet werden:

```
# Generate the documentation in Tcl Doc Tool format
set dt [tepam::doc_gen::generate -format DT -header_footer
    tepam::doc_gen::generate]

# Create a new doc tools object (HTML format)
package require doctools
::doctools::new myDoc -format html

# Open the HTML file, and write the HTML formatted documentation
set fHtml [open doc_gen.dt.html w]
puts $fHtml [myDoc format $dt]
close $fHtml
```

Tepam::doc_gen::patch

Während **generate** nur eine begrenzte Anzahl von Möglichkeiten zur Anpassung der Dokumentstruktur bietet, gewährt **patch** mehr Flexibilität. So können beispielsweise mehrere Dokumentationen für unterschiedliche Prozeduren sowie Metainformationen hinzugefügt werden.

Die folgende Auflistung veranschaulicht die Funktionsweise des Befehls **patch**. Zunächst wird eine Zeichenkette mit einem HTML-Masterdokument definiert, die zwei Platzhalter für Prozedurdokumentation enthält (*{**}*). Diese Platzhalter werden von **patch** durch die generierte Dokumentation der betreffenden Prozeduren ersetzt. Da vom Standard abweichende Platzhalter verwendet werden, wird **patch** mit einer expliziten Platzhaltermusterdefinition (Argument *search_pattern*) aufgerufen.

```
# Define the HTML master document
set HtmlMasterDoc {\
<html>
  <head>
    <title>tepam::doc_gen</title>
    <link rel="stylesheet" href="tepam_doc_stylesheet.css">
    <meta content="documentation" name="keywords"></meta>
```

```
  </head>
  <body>
    <h1>tepam::doc_gen</h1>
      <h2>Generate</h2>
{*tepam::doc_gen::generate*}
      <h2>Patch</h2>
{*tepam::doc_gen::patch*}
  </body>
<html>\
}

# Patch the master document: This will replace the placeholders by the
# procedure documentation divisions:
tepam::doc_gen::patch -format HTML -search_pattern {\{\*(.*?)\*\}} \
                      -src_string $HtmlMasterDoc -dest_file
    tepam_doc_gen.html
```

Kategorie
Dokumentationswerkzeuge

Copyright

```
Copyright (c) 2013, Andreas Drollinger
```


tepam_introduction
Stichwörter: tepam_introduction, argument integrity, argument validation, arguments, entry mask, parameter entry form, procedure, subcommand

Name
tepam – Eine Einleitung in TEPAM, den erweiterten Manager für Prozeduren und Argumente von Tcl

Beschreibung
Dieses Dokument bietet eine informelle Einleitung in TEPAM, den "Enhanced Procedure and Argument Manager" von Tcl. Ausführliche Informationen zum TEPAM-Paket finden Sie in den Referenzhandbüchern *tepam::procedure* und *tepam::argument_dialogbox*.

Überblick
Dieses Paket bietet eine neue Syntax für die Deklaration von Tcl-Prozeduren, die die Implementierung von Unterbefehlen für Prozeduren sowie den Umgang mit den verschiedenen Arten von Prozedurargumenten – wie Flags oder Schalter, Optionen, unbenannte Argumente, optionale und obligatorische Optionen und Argumente, Vorgabewerte usw. – vereinfacht. Prozedurdeklarationen können um detaillierte Informationen zur Prozedur und ihren Argumenten ergänzt werden. Diese Informationen werden für folgende Zwecke verwendet:

Zunächst wird vor den Körper einer mit TEPAM deklarierten Prozedur eine Präambel gesetzt. Diese Präambel ruft einen Argumentmanager auf, der anhand der bereitgestellten Informationen

die Gültigkeit der Argumenttypen und -werte überprüft, bevor der Prozedurkörper ausgeführt wird. Außerdem werden diese Informationen verwendet, um auf Wunsch Hilfe- und Verwendungstexte zu generieren oder um klare Fehlermeldungen auszugeben, falls die Validierung der Argumente fehlschlägt. Die Informationen ermöglichen zudem die automatische Erstellung grafischer Formulare, die eine interaktive Definition aller Argumente ermöglichen, wenn eine Prozedur interaktiv aufgerufen wird. Und schließlich tragen die zusätzlichen Informationen dazu bei, die Deklaration einer Prozedur und aller ihrer Argumente auf übersichtliche Weise selbst zu kommentieren.

Der grafische Formulargenerator, der die erforderlichen Formulare zur Argumentspezifikation für die interaktiven Aufrufe einer Prozedur erstellt, steht auch für andere Zwecke als die Spezifikation von Prozedurargumenten zur Verfügung. Er ermöglicht die effiziente Erstellung komplexer Formulare zur Parametereingabe, die unabhängig von der neuen Methode zur Definition von Prozeduren in TEPAM verwendet werden können.

Hier ein kurzer Überblick über alle wichtigen Funktionen von TEPAM:

- Neue Syntax für selbstdokumentierende Prozedurdeklarationen: Die zusätzlichen Informationen, die für eine korrekte Deklaration einer Prozedur erforderlich sind, müssen nicht mehr durch zusätzliche Anweisungen bereitgestellt werden, sondern können in einer natürlichen Syntax direkt in den Prozedurkopf eingefügt werden.
- Einfache Methode zur Definition von Unterbefehlen: Ein Unterbefehl wird wie eine Prozedur deklariert, und zwar einfach mit einem Namen der Prozedur, der sich aus einem Basisnamen und einem daran anschließenden Namen des Unterbefehls zusammensetzt. Unter-Unterbefehle werden auf identische Weise erstellt, indem einfach Namen der Prozeduren verwendet werden, die aus drei Wörtern bestehen.
- Flexible Verwendung von Flags (Schaltern), Optionen (benannte Argumente) und unbenannten Argumenten. Optionsnamen werden optional automatisch vervollständigt.
- Unterstützung für Vorgabewerte, obligatorische bzw. optionale Optionen und Argumente, Auswahllisten, Wertebereiche sowie mehrfach verwendbare Optionen und Argumente.
- Wahl zwischen dem Aufrufstil *benannte Argumente zuerst, unbenannte Argumente danach* (typisch für Tcl-Befehle) und dem Aufrufstil *unbenannte Argumente zuerst, benannte Argumente danach* (typisch für Tk-Befehle).
- Falls der Stil *benannte Argumente zuerst, unbenannte Argumente danach* (Tcl) ausgewählt ist: Klare Trennung zwischen Optionen und Argumenten durch das Flag "--". Auf die unbenannten Argumente kann optional als Optionen (benannte Argumente) zugegriffen werden.
- Automatische Typ- und Wertprüfung vor der Ausführung des Prozedurkörpers unter Berücksichtigung von Validierungsbereichen, Auswahllisten und benutzerdefinierten Validierungsbefehlen. Bei Bedarf Generierung einer eindeutigen Fehlermeldung.
- Es gibt zahlreiche vordefinierte Typen (integer, boolean, double, color, file, font, ...). Weitere anwendungsspezifische Typen lassen sich problemlos hinzufügen.
- Automatische Generierung von Hilfe- und Verwendungstexten, wenn eine Prozedur mit dem Flag *-help* aufgerufen wird.
- Automatische Erstellung eines interaktiven Formulars zur Definition der Argumente, falls eine Prozedur mit dem Flag *-interactive* aufgerufen wird.
- Prozeduraufrufe können protokolliert werden, was nützlich ist, um bei interaktiv aufgerufenen Prozeduren die Befehlszeilen des Aufrufs zu erhalten.
- Leistungsstarke und codeeffiziente Generierung komplexer Formulare zur Definition von

Parametern.

Prozedurdeklaration

Die Syntax für die Deklaration einer Prozedur in TEPAM ist einfach und selbsterklärend. Anstatt eine Prozedur mit dem Tcl-Schlüsselwort **proc** zu deklarieren, wird eine Prozedur mit dem TEPAM-Befehl **procedure** deklariert, der wie **proc** ebenfalls drei Argumente benötigt: den Namen der Prozedur, den Prozedurkopf und den Prozedurkörper.

Im folgenden Beispiel wird der Unterbefehl **message** der Prozedur **display** deklariert. Dieser Befehl verfügt über mehrere benannte und unbenannte Argumente:

```
tepam::procedure {display message} {
   -return            -
   -short_description "Displays a simple message box"
   -description       "This procedure allows displaying a configurable
    message box."
   -args {
      {-mtype -default Warning -choices {Info Warning Error}
    -description "Message type"}
      {-font -type font -default {Arial 10 italic} -description
    "Message text font"}
      {-level -type integer -optional -range {1 10} -description
    "Message level"}
      {-fg -type color -default black -description "Message color"}
      {-bg -type color -optional -description "Background color"}
      {-no_border -type none -description "Use a splash window style
    (no border)"}
      {-log_file -type file -optional -description "Optional message
    log file"}
      {text -type string -multiple -description "Multiple text lines
    to display"}
   }
} {
   puts "display message:"
   foreach var {mtype font level fg bg no_border log_file text} {
      if {[info exists $var]} {
         puts  "  $var=[set $var]"
      }
   }
}
```

Ein Aufruf einer auf diese Weise deklarierten Prozedur ruft zunächst den TEPAM-Argumentmanager auf, bevor der Prozedurkörper ausgeführt wird. Der Argumentmanager analysiert die übergebenen Argumente, validiert sie, ergänzt sie gegebenenfalls um Vorgabewerte und stellt sie schließlich dem Prozedurkörper als lokale Variablen zur Verfügung. Falls ein Argument fehlt oder einen falschen Typ aufweist, generiert der Argumentmanager eine Fehlermeldung, die den Grund für den Fehler erläutert.

Wie das obige Beispiel zeigt, akzeptiert der TEPAM-Befehl **procedure** Unterbefehlsdefinitionen

als Prozedurnamen und ermöglicht es, weitaus mehr Informationen als nur die Argumentliste im Prozedurkopf zu definieren. Der Prozedurkörper hingegen ist bei einem mit **proc** deklarierten Befehl identisch mit dem eines mit **procedure** deklarierten Befehls.

Der Prozedurkopf ermöglicht es, zusätzlich zu den Argumenten einige Prozedurattribute zu definieren, wie beispielsweise eine Beschreibung, Angaben zum Rückgabewert usw. Diese Informationen werden im Wesentlichen für die automatische Generierung umfassender Hilfe- und Verwendungstexte verwendet.

Eine Liste von Anweisungen zur Definition von Argumenten, die dem Argument *-args* zugewiesen wird, definiert die Argumente der Prozedur. Jede Anweisung zur Definition eines Arguments beginnt mit dem Namen des Arguments, auf den optional einige Argumentattribute folgen.

Es lassen sich drei Arten von Argumenten definieren: unbenannte Argumente, benannte Argumente und Flags. Die Unterscheidung zwischen benannten und unbenannten Argumenten erfolgt anhand des ersten Zeichens des Argumentnamens, das bei benannten Argumenten einfach "-" lautet. Ein Flag ist definiert als ein benanntes Argument vom Typ *none*.

Benannte und unbenannte Argumente sind obligatorisch, es sei denn, sie werden mit dem Flag *-optional* deklariert oder es wurde für sie ein Vorgabewert mit der Option *-default* festgelegt. Benannte Argumente und das letzte unbenannte Argument können das Attribut *-multiple* aufweisen, was bedeutet, dass sie mehrfach definiert werden können. Der erwartete Datentyp des Arguments wird mit der Option *-type* festgelegt. TEPAM definiert eine große Auswahl an Standarddatentypen, die problemlos durch anwendungsspezifische Datentypen ergänzt werden können.

Die Reihenfolge der Argumentdeklarationen ist nur für unbenannte Argumente von Bedeutung, die standardmäßig nach den benannten Argumenten ausgewertet werden (Tcl-Stil). Mit einer Variable lässt sich dieses Verhalten so ändern, dass unbenannte Argumente vor den benannten Argumenten ausgewertet werden (Tk-Stil).

Hilfe zur Prozedur

Die deklarierte Prozedur kann einfach mit der Option *-help* aufgerufen werden, um Informationen über die Verwendung der Prozedur und ihrer Argumente abzurufen:

```
display message -help
  ->
NAME
      display message - Displays a simple message box
SYNOPSYS
      display message
            [-mtype <mtype>] :
               Message type, default: "Warning", choices: {Info
    Warning Error}
            [-font <font>] :
               Message text font, type: font, default: Arial 10 italic
            [-level <level>] :
               Message level, type: integer, range: 1..10
            [-fg <fg>] :
               Message color, type: color, default: black
```

```
            [-bg <bg>] :
               Background color, type: color
            [-no_border ] :
               Use a splash window style (no border)
            [-log_file <log_file>] :
               Optional message log file, type: file
            <text> :
               Multiple text lines to display, type: string
DESCRIPTION
      This procedure allows displaying a configurable message box.
```

Prozedurenaufruf

Die angegebene Prozedur kann auf vielfältige Weise aufgerufen werden. Das folgende Listing zeigt einige gültige Prozeduraufrufe:

```
display message "The document hasn't yet been saved!"
-> display message:
     mtype=Warning
     font=Arial 10 italic
     fg=black
     no_border=0
     text={The document hasn't yet been saved!}

display message -fg red -bg black "Please save first the document"
-> display message:
     mtype=Warning
     font=Arial 10 italic
     fg=red
     bg=black
     no_border=0
     text={Please save first the document}

display message -mtype Error -no_border "Why is here no border?"
-> display message:
     mtype=Error
     font=Arial 10 italic
     fg=black
     no_border=1
     text={Why is here no border?}

display message -font {Courier 12} -level 10 \
   "Is there enough space?" "Reduce otherwise the font size!"
-> display message:
     mtype=Warning
     font=Courier 12
     level=10
```

```
    fg=black
    no_border=0
    text={Is there enough space?} {Reduce otherwise the font size!}
```

Die folgenden Zeilen veranschaulichen, wie fehlerhafte Argumente erkannt werden. Das obligatorische Argument *text* fehlt beim ersten Prozeduraufruf:

```
display message -font {Courier 12}
  -> display message: Required argument is missing: text
```

Es werden nur bekannte Argumente akzeptiert:

```
display message -category warning Hello
  -> display message: Argument '-category' not known
```

Argumenttypen werden automatisch überprüft, und es wird eine Fehlermeldung ausgegeben, falls der Wert des Arguments nicht den erwarteten Typ aufweist:

```
display message -fg MyColor "Hello"
  -> display message: Argument 'fg' requires type 'color'.  Provided
    value: 'MyColor'
```

Auswahllisten müssen eingehalten werden ...

```
display message -mtype Fatal Hello
  -> display message: Argument (mtype) has to be one of the  following
    elements: Info, Warning, Error
```

... sowie gültige Wertebereiche:

```
display message -level 12 Hello
  -> display message: Argument (level) has to be between 1 and 10
```

Interaktive Prozedurenaufrufe

Die intuitivste Art, die Prozedur aufzurufen, ist die Verwendung eines Formulars, in dem alle Argumente interaktiv festgelegt werden können. Dieses Formular wird automatisch generiert, wenn die deklarierte Prozedur mit dem Flag *-interactive* aufgerufen wird. Um diese Funktion nutzen zu können, muss die Tk-Bibliothek geladen sein.

```
display message -interactive
```

Das generierte Formular enthält für jedes Argument ein Dateneingabe-Widget, das an den jeweiligen Argumenttyp angepasst ist. Zur Festlegung von Flags werden Checkbuttons verwendet, für kleine Auswahllisten Radioboxen, für größere Auswahllisten disjunkte Listboxen, und Dateien, Verzeichnisse, Schriftarten und Farben lassen sich mit eigenen Auswahldialogen (Browsern) auswählen.

Nachdem die angegebenen Argumentdaten über eine "OK"-Schaltfläche bestätigt wurden, werden die eingegebenen Daten zunächst validiert, bevor die angegebenen Argumente in lokale Variablen umgewandelt und der Prozedurkörper ausgeführt wird. Sollten die eingegebenen Daten ungültig sein, erscheint eine Meldung, und der Benutzer kann diese korrigieren, bis sie gültig sind.

Die Prozeduraufrufe können optional in einer Variablen protokolliert werden. Dies ist beispielsweise nützlich, um die Befehlszeilen von interaktiv aufgerufenen Prozeduren abzurufen.

Flexible Argument-Dialogbox

Der Formulargenerator, der im vorangegangenen Beispiel die Argument-Dialogbox für den interaktiven Aufruf einer Prozedur erstellt, steht auch für andere Zwecke als die Definition von Prozedurargumenten zur Verfügung. Wenn Tk geladen wurde, stellt TEPAM eine Argument-Dialogbox bereit, mit dem sich komplexe Parameterdefinitionsformulare auf sehr effiziente Weise erstellen lassen.

Das folgende Beispiel soll veranschaulichen, wie einfach es ist, komplexe Eingabeformulare zu erstellen. Es erstellt eine Eingabemaske, über die eine zu kopierende Datei sowie ein Zielverzeichnis angegeben werden können, und enthält zudem eine Checkbox, mit der festgelegt werden kann, ob eine eventuell bereits vorhandene Datei überschrieben werden darf. Zur Auswahl von Dateien und Verzeichnissen stehen benutzerfreundliche Browser zur Verfügung. Und schließlich bietet das Formular auch die Möglichkeit, die Auswahl zu bestätigen oder abzulehnen. Hier ist der Codeausschnitt, der all dies umsetzt:

```
tepam::argument_dialogbox \
   -existingfile {-label "Source file" -variable SourceFile} \
   -existingdirectory {-label "Destination folder" -variable DestDir} \
   -checkbutton {-label "Overwrite existing file" -variable Overwrite}
```

Der Befehl **argument_dialogbox** gibt **ok** zurück, wenn die eingegebenen Daten validiert wurden. Er gibt **cancel** zurück, wenn die Dateneingabe abgebrochen wurde. Nach der Validierung der eingegebenen Daten definiert **argument_dialogbox** alle angegebenen Variablen mit den eingegebenen Daten im aufrufenden Kontext.

Der Befehl **argument_dialogbox** benötigt für jede Variable, die er verarbeiten soll, ein Argumentpaar. Das erste Argument definiert den Typ des Eingabe-Widgets, mit dem der Wert der Variable ausgewählt wird, und das zweite Argument ist eine Liste von Attributen, die sich auf die Variable und das Eingabe-Widget beziehen.

Es stehen zahlreiche Arten von Eingabe-Widgets zur Verfügung: Neben den einfachen generischen Eingabefeldern gibt es verschiedene Arten von Listen- und Kombinationsfeldern, Browser für vorhandene und neue Dateien und Verzeichnisse, Check- und Radioboxen und -buttons sowie Farb- und Schriftartenauswahlfelder. Bei Bedarf können zusätzliche Eingabe-Widgets definiert werden.

Die Attributliste enthält Paare aus Attributnamen und Attributdaten. Das primäre Attribut ist *-variable*, mit dem die Variable im aufrufenden Kontext festgelegt wird, in der die eingegebenen Daten gespeichert werden sollen. Ein weiteres häufig verwendetes Attribut ist *-label*, mit dem dem Dateneingabe-Widget eine Beschriftung hinzugefügt werden kann. Es stehen weitere Attribute zur Verfügung, mit denen Vorgabewerte, die erwarteten Datentypen, gültige Datenbereiche usw. festgelegt werden können.

Das folgende Beispiel für eine komplexere Argument-Dialogbox bietet einen guten Überblick über die verschiedenen verfügbaren Eingabe-Widget-Typen und Parameterattribute. Das Beispiel enthält zudem einige Formatierungsanweisungen wie *-frame* und *-sep*, die es ermöglichen, die verschiedenen Eingabe-Widgets in Rahmen und Abschnitten zu organisieren:

```
set ChoiceList {"Choice 1" "Choice 2" "Choice 3" "Choice 4" "Choice 5"
    "Choice 6"}

set Result [tepam::argument_dialogbox \
   -title "System configuration" \
   -context test_1 \
   -frame {-label "Entries"} \
      -entry {-label Entry1 -variable Entry1} \
      -entry {-label Entry2 -variable Entry2 -default "my default"} \
   -frame {-label "Listbox & combobox"} \
      -listbox {-label "Listbox, single selection" -variable Listbox1 \
                -choices {1 2 3 4 5 6 7 8} -default 1 -height 3} \
      -listbox {-label "Listbox, multiple selection" -variable Listbox2
                -choicevariable ChoiceList -default {"Choice 2"
    "Choice 3"}
                -multiple_selection 1 -height 3} \
      -disjointlistbox {-label "Disjoined listbox" -variable
    DisJntListbox
                         -choicevariable ChoiceList \
                         -default {"Choice 3" "Choice 5"} -height 3} \
      -combobox {-label "Combobox" -variable Combobox \
                 -choices {1 2 3 4 5 6 7 8} -default 3} \
   -frame {-label "Checkbox, radiobox and checkbutton"} \
      -checkbox {-label Checkbox -variable Checkbox
                 -choices {bold italic underline} -choicelabels {Bold
    Italic Underline} \
                 -default italic} \
      -radiobox {-label Radiobox -variable Radiobox
                 -choices {bold italic underline} -choicelabels {Bold
    Italic Underline} \
                 -default underline} \
      -checkbutton {-label CheckButton -variable Checkbutton -default
    1} \
   -frame {-label "Files & directories"} \
      -existingfile {-label "Input file" -variable InputFile} \
      -file {-label "Output file" -variable OutputFile} \
      -sep {} \
      -existingdirectory {-label "Input directory" -variable
    InputDirectory} \
      -directory {-label "Output irectory" -variable OutputDirectory} \
   -frame {-label "Colors and fonts"} \
      -color {-label "Background color" -variable Color -default red} \
      -sep {} \
      -font {-label "Font" -variable Font -default {Courier 12
    italic}}]
```

Der Befehl **argument_dialogbox** belegt alle angegebenen Variablen mit den eingegebenen Daten und gibt **ok** zurück, wenn die Daten über die Schaltfläche "OK" bestätigt wurden. Wenn die Dateneingabe mit der Schaltfläche "Cancel" abgebrochen wird, gibt **argument_dialogbox cancel** zurück.

```
if {$Result=="cancel"} {
   puts "Canceled"
} else { # $Result=="ok"
   puts "Arguments: "
   foreach Var {
      Entry1 Entry2
      Listbox1 Listbox2 DisJntListbox
      Combobox Checkbox Radiobox Checkbutton
      InputFile OutputFile InputDirectory OutputDirectory
      Color Font
   } {
      puts "  $Var: '[set $Var]'"
   }
}
-> Arguments:
   Entry1: 'Hello, this is a trial'
   Entry2: 'my default'
   Listbox1: '1'
   Listbox2: '{Choice 2} {Choice 3}'
   DisJntListbox: '{Choice 3} {Choice 5}'
   Combobox: '3'
   Checkbox: 'italic'
   Radiobox: 'underline'
   Checkbutton: '1'
   InputFile: 'c:\tepam\in.txt'
   OutputFile: 'c:\tepam\out.txt'
   InputDirectory: 'c:\tepam\input'
   OutputDirectory: 'c:\tepam\output'
   Color: 'red'
   Font: 'Courier 12 italic'
```

Kategorie

Prozeduren, Argumente, Parameter, Optionen

Copyright


```
Copyright (c) 2009-2013, Andreas Drollinger
```


tepam_procedure

Stichwörter: tepam_procedure, argument integrity, argument validation, arguments, procedure, subcommand

Name

tepam::procedure – TEPAM-Prozedur, Referenzhandbuch

Übersicht

package require Tcl 8.5 9
package require tepam ?**0.5**?
tepam::procedure *name attributes body*

Beschreibung

Dieses Paket bietet eine alternative Möglichkeit, Tcl-Prozeduren zu deklarieren und deren Argumente zu verwalten. Die Deklaration einer Prozedur mit TEPAM bietet gegenüber dem Tcl-Standardbefehl **proc** zahlreiche Vorteile: TEPAM ermöglicht es, innerhalb der Prozedurdeklaration alle Informationen anzugeben, die zur Erstellung umfassender Dokumentationen und Hilfeunterstützung erforderlich sind. Diese Informationen werden zudem von einem automatisch aufgerufenen Argumentprüfer verwendet, der die übergebenen Prozedurargumente validiert, bevor der Prozedurkörper ausgeführt wird. Schließlich kann eine Prozedur interaktiv aufgerufen werden, wodurch ein grafisches Formular geöffnet wird, in dem die Argumente der Prozedur festgelegt werden können.

TEPAM vereinfacht zudem den Umgang mit den verschiedenen Argumenttypen, wie beispielsweise den *benannten Argumenten* (oft auch als *Optionen* bezeichnet) und den *unbenannten Argumenten*. TEPAM unterstützt sowohl den Stil *benannte zuerst, unbenannte danach* (typischer Tcl-Befehlsstil) als auch den Stil *unbenannte zuerst, benannte danach* (typischer Tk-Befehlsstil). TEPAM kümmert sich um Vorgabewerte für Argumente, optionale Argumente, mehrfach anwendbare Argumente usw. und macht es überflüssig, die Gültigkeit der Argumente in den Prozedurkörpern zu prüfen.

Einen informellen Überblick über alle Funktionen zur Deklaration und zum Aufruf von TEPAM-Prozeduren sowie eine kurze Einführung in TEPAM bietet *tepam(n)*.

Begriffsbestimmungen

Die genaue Bedeutung einiger in diesem Dokument verwendeter Begriffe wird im Folgenden kurz erläutert, um Unklarheiten und Missverständnisse zu vermeiden.

Subcommand

Die Verwendung von Unterbefehlen ist in der Programmiersprache Tcl weit verbreitet. Mehrere Befehle sind in einem einzigen Hauptbefehl zusammengefasst und können über das erste Argument ausgewählt werden.

Der Befehl **string** ist ein Beispiel für einen solchen Befehl, der unter anderem Unterbefehle zur Überprüfung der Länge einer Zeichenkette, zum Vergleich von Zeichenketten, zum Extrahieren von Teilzeichenketten usw. implementiert:

```
string length string
string compare string string
string range string first last
...
```

TEPAM bietet einen Rahmen, der die einfache Implementierung solcher Unterbefehle in Form von Tcl-Prozeduren ermöglicht. Er erlaubt nicht nur die Definition einer ersten Ebene von

Unterbefehlen, sondern auch höherer Ebenen von Unterbefehlen. Die Klassenprüfung des Befehls **string** könnte als eigenständige Unter-Unterbefehle des Befehls **string** implementiert werden:

```
string is alnum string
string is integer string
string is double string
...
```

Prozedurattribut
TEPAM ermöglicht es, einer deklarierten Prozedur verschiedene Arten von Attributen zuzuweisen. Einige dieser Attribute dienen *lediglich* zu Dokumentationszwecken, während andere Attribute festlegen, wie die Prozedur aufgerufen werden muss. Auch die Prozedurargumente werden in Form eines Prozedurattributs definiert.

Argument
TEPAM verwendet den Begriff *Argument* für die Parameter einer Prozedur.

Im folgenden Beispiel wird der Unterbefehl **string compare** mit mehreren Argumenten aufgerufen:

```
string compare -nocase -length 3 "emphasized" "emphasised"
```

In den folgenden Absätzen werden diese verschiedenen Argumenttypen erörtert.

Benanntes Argument
Einige Parameter, wie beispielsweise *-length 3* des Unterbefehls **string compare**, müssen als Paare aus Argumentnamen und Argumentwerten angegeben werden. Dieser Parametertyp wird häufig auch als *Option* bezeichnet.

TEPAM verwendet den Begriff *benanntes Argument* sowohl für solche Optionen als auch für die Flags (siehe nächster Punkt).

Flag, switch
Ein weiterer Parametertyp ist das *Flag* oder der *Schalter*. Flags werden einfach dadurch übergeben, dass ihr Name mit vorangestelltem Zeichen "-" angegeben wird. Das *-nocase* aus dem vorherigen Beispiel **string compare** ist ein solches Flag.

Flags werden von TEPAM als eine spezielle Form von *benannten Argumenten* betrachtet.

Unbenanntes Argument
Für die übrigen Parameter, d. h. diejenigen, bei denen der Name des Arguments nicht angegeben werden muss, verwendet TEPAM den Begriff *unbenanntes Argument*. Das vorangegangene Beispiel **string compare** verwendet für die beiden übergebenen Zeichenfolgen zwei *unbenannte Argumente*.

Argumentattribut
TEPAM ermöglicht es, den Zweck jedes Prozedurarguments mithilfe von *Argumentattributen* zu beschreiben. Während einige davon lediglich der Dokumentation der Argumente dienen, werden die meisten Attribute von einem Argumentmanager verwendet, um die bei einem Prozeduraufruf übergebenen Argumente zu steuern und zu validieren. Argumentattribute dienen dazu, Vorgabewerte, Parameterklassen (integer, xdigit, font, …), Validierungslisten für

Auswahlmöglichkeiten, Wertebereiche usw. festzulegen.

Benannte Argumente zuerst, unbenannte Argumente danach

Der Befehl **string compare** aus dem vorherigen Beispiel erfordert, dass die *benannten Argumente* (Optionen, Flags) zuerst angegeben werden. Die beiden obligatorischen (unbenannten) Argumente müssen als letztes Argument angegeben werden.

```
string compare -nocase -length 3 Water $Text
```

Dies ist der übliche Tcl-Stil (es gibt Ausnahmen), der in der TEPAM-Dokumentation als Stil *benannte Argumente zuerst, unbenannte Argumente danach* bezeichnet wird.

Unbenannte Argumente zuerst, benannte Argumente danach

Im Gegensatz zu den meisten Tcl-Befehlen verwendet Tk in der Regel (auch hier gibt es Ausnahmen) eine andere Aufrufweise, bei der die *unbenannten Argumente* zuerst angegeben werden müssen, bevor die *benannten Argumente* folgen:

```
pack .ent1 .ent2 -fill x -expand yes -side left
```

Dieser Stil wird in der TEPAM-Dokumentation als Stil *unbenannte Argumente zuerst, benannte Argumente danach* bezeichnet.

Prozedurdeklaration

Mit TEPAM können neue Tcl-Prozeduren mithilfe des Befehls **tepam::procedure** deklariert werden, der – ähnlich wie der Standard-Tcl-Befehl **proc** – ebenfalls drei Argumente aufweist:

tepam::procedure *name attributes body*

siehe unten ...

Die Syntax für die Deklaration von TEPAM-Prozeduren wird anhand des folgenden Beispiels veranschaulicht:

```
tepam::procedure {display message} {
   -short_description
      "Displays a simple message box"
   -description
      "This procedure allows displaying a configurable\
       message box. The default message type that is\
       created is a warning, but also errors and info can\
       be generated.
       The procedure accepts multiple text lines."
   -example
      {display message -mtype Warning "Save first your job"}
   -args {
      {-mtype -choices {Info Warning Error} \
              -default Warning -description "Message type"}
      {text   -type string -multiple \
              -description "Multiple text lines to display"}
   }
} {
```

```
    puts "Message type: $mtype"
    puts "Message: $text"
}
```

Die drei Argumente von **procedure** lauten:

name

Der Prozedurname lässt sich sehr flexibel verwenden. Prozedurnamen können Namespace-Qualifizierer enthalten. Durch die Angabe einer aus zwei Elementen bestehenden Namensliste als Prozedurnamen wird ein Unterbefehl einer Prozedur deklariert. Es ist sogar möglich, Unter-Unterbefehle einer Prozedur zu deklarieren, indem Namenslisten mit drei Elementen angegeben werden.

Hier sind einige gültige Prozedurdeklarationen mit verschiedenen Prozedurnamen (die Argumente für Attribute und Körper sind der Einfachheit halber leer):

```
# Simple procedure name:
tepam::procedure display_message {} {}

# Procedure declared in the main namespace:
tepam::procedure ::display_message {} {}

# Procedure in the namespace ::ns:
tepam::procedure ::ns::display_message {} {}

# Declaration of the subcommand message of the procedure display:
tepam::procedure {display message} {} {}
```

attributes

Alle Prozedurattribute werden in Form einer Optionsliste bereitgestellt, die Paare aus Optionsnamen und Optionswerten enthält. Das obige Beispiel weist als Prozedurattribute eine Kurzbeschreibung und eine normale Beschreibung auf, doch auch die Prozedurargumente werden in Form eines Prozedurattributs definiert.

Die meisten Prozedurattribute dienen der Dokumentation. Einige von ihnen beeinflussen jedoch auch die Art und Weise, wie die Prozedur aufgerufen werden kann. Der Abschnitt Prozedur-Attribute behandelt die verfügbaren Prozedurattribute im Detail.

Die Prozedurargumente werden in Form eines speziellen Prozedurattributs definiert. Die meisten der in der Argumentdefinition angegebenen Informationen dienen nicht nur zu Dokumentationszwecken. Diese Informationen werden vielmehr vom TEPAM-Argumentmanager verwendet, um die verschiedenen Formen von Argumenten, die bei Aufrufen von Prozeduren übergeben werden, zu verarbeiten und zu validieren. Der Abschnitt Deklaration von Argumenten behandelt alle Attribute der Argumentdefinition im Detail.

body

Dies ist der normale Prozedurkörper. Die deklarierten Argumente stehen dem Prozedurkörper in Form von Variablen zur Verfügung.

Der Prozedurkörper wird nur ausgeführt, wenn die übergebene Argumentgruppe vom

TEPAM-Argumentmanager validiert werden konnte.

```
tepam::procedure {display_message} {
   -args {
      {-mtype -default Warning -choices {Warning Error}}
      {text -type string}
   }
} {
   puts "Message type: $mtype"
   puts "Message: $text"
}
```

Die Befehle **procedure** sowie **argument_dialogbox** werden aus dem Namespace tepam exportiert. Um diese Befehle ohne das Namespace-Präfix **tepam::** zu verwenden, reicht es aus, sie in den Haupt-Namespace zu importieren:

```
namespace import tepam::*

procedure {display_message} {
   -args {
      ...
```

Prozedur-Attribute

Die erste Gruppe von Attributen beeinflusst das Verhalten der deklarierten Prozedur:

-named_arguments_first 0|1
Dieses Attribut definiert den Aufrufstil einer Prozedur. TEPAM verwendet standardmäßig den Stil *benannte Argumente zuerst, unbenannte Argumente danach* (Tcl). Dieses Standardverhalten kann global geändert werden, indem die Variable **tepam::named_arguments_first** auf **0** gesetzt wird. Dieser globale Aufrufstil kann für eine einzelne Prozedur mithilfe des Attributs *-named_arguments_first* individuell geändert werden.

-auto_argument_name_completion 0|1
Die deklarierten Prozeduren versuchen standardmäßig automatisch, eventuell abgekürzte Argumentnamen den definierten Argumentnamen zuzuordnen. Dieses Standardverhalten kann global geändert werden, indem die Variable **tepam::auto_argument_name_completion** auf **0** gesetzt wird. Diese globale Einstellung der automatischen Namenvervollständigung für Argumente kann für eine Prozedur individuell über das Prozedurattribut *-auto_argument_name_completion* geändert werden.

-interactive_display_format extended|short
Eine mit dem TEPAM-Befehl **procedure** deklarierte Prozedur kann stets mit der Option **-interactive** aufgerufen werden. Auf diese Weise wird ein grafisches Formular generiert, in dem alle Werte der Prozedurargumente angegeben werden können. Für diese interaktiven Formulare gibt es zwei Anzeigemodi. Während der Modus *extended* eher für Prozeduren mit wenigen Argumenten geeignet ist, eignet sich die Form **short** besser für Prozeduren mit sehr vielen Argumenten.

Die Wahl zwischen der Verwendung kurzer oder erweiterter Formulare kann global über die Variable **tepam::interactive_display_format** konfiguriert werden. Diese globale Einstellung

lässt sich anschließend für eine einzelne Prozedur mithilfe des Prozedurattributs *-interactive_display_format* individuell ändern.

-args *list*
Die Prozedurargumente werden über das Attribut *-args* deklariert. Ein Argument wird über eine Liste definiert, deren erstes Element der Name des Arguments ist, gefolgt von eventuellen Argumentattributen. Alle diese Argumentdefinitionslisten werden ihrerseits in eine globale Liste zusammengefasst, die dem Attribut *-args* zugewiesen wird.

Die Syntax der Argumentdefinition wird im folgenden Unterabschnitt näher beschrieben.

Die folgenden Attribute ermöglichen die Festlegung benutzerdefinierter Argumentprüfungen sowie benutzerdefinierter Fehlermeldungen für den Fall, dass diese Prüfungen fehlschlagen:

-validatecommand *script*
Benutzerdefinierte Argumentvalidierungen können über spezifische Validierungsbefehle durchgeführt werden, die mit dem Attribut *-validatecommand* definiert sind.

Beispiel für die Deklaration eines Validierungsbefehls:

```
tepam::procedure {display_message} {
   -args {
      {text -type string -description "Message text"} }
   -validatecommand {IllegalWordDetector $text}
} {
}
```

Der Validierungsbefehl wird im Kontext des deklarierten Prozedurkörpers ausgeführt. Der Zugriff auf die verschiedenen Argumentwerte erfolgt über die Argumentnamen. Hinweis: Es gibt außerdem ein Argumentattribut *-validatecommand*, mit dem benutzerdefinierte Prüfungen für bestimmte Argumente deklariert werden können.

Das Attribut *-validatecommand* kann wiederholt werden, um mehrere benutzerdefinierte Prüfungen zu deklarieren.

-validatecommand_error_text *string*
Mit diesem Attribut lässt sich die Standardfehlermeldung für eine benutzerdefinierte Validierung eines Arguments (definiert durch *-validatecommand*) überschreiben. Außerdem kann dieses Attribut wiederholt werden, falls mehrere Validierungen von Argumenten deklariert sind.

Mit dem folgenden Attribut lassen sich die Protokollierungseinstellungen für eine einzelne Prozedur steuern:

-command_log 0|1|"interactive"
Dieses Argument konfiguriert die Protokollierung der Prozeduraufrufe in der Listenvariablen **tepam::ProcedureCallLogList**. Wird dieses Argument in einer Prozedurdeklaration nicht definiert, wird die durch die Variable **tepam::command_log** festgelegte Standardkonfiguration verwendet.

Wird dieses Argument auf **0** gesetzt, wird jede Protokollierung von Prozeduraufrufen abgeschaltet; bei **1** werden alle Prozeduraufrufe protokolliert, und bei **interactive** nur die

Prozeduren, die interaktiv aufgerufen werden (Prozeduren, die mit dem Flag **-interactive** aufgerufen werden).

Die nächste Gruppe von Prozedurattributen dient ausschließlich der Dokumentation und der Erstellung von Hilfetexten:

-category *string*
Einer Prozedur kann zu Dokumentationszwecken eine Kategorie zugewiesen werden. Als Kategorie wird jede beliebige Zeichenfolge akzeptiert.

-short_description *string*
Die Kurzbeschreibung einer Prozedur wird sowohl in der Zusammenfassung der Dokumentation einer generierten Liste von Prozeduren als auch im Abschnitt NAME einer generierten Handbuchseite zur Prozedur verwendet.

-description *string*
Die einer Prozedur zugewiesene (vollständige) Beschreibung wird zur Erstellung des Benutzerhandbuchs und der Hilfeseiten verwendet.

-return *string*
Das Attribut *-return* ermöglicht die Festlegung des erwarteten Rückgabewerts einer Prozedur (zu Dokumentationszwecken).

-example *string*
Ein Hilfetext oder eine Seite des Handbuchs zu einer Prozedur kann mithilfe des Attributs *-example* gegebenenfalls um Beispiele ergänzt werden.

Deklaration von Argumenten

Das folgende Beispiel veranschaulicht die Struktur, die für die Definitionen der Argumente im Rahmen einer Deklaration einer Prozedur verwendet wird:

```
tepam::procedure {display_message} {
   -args {
      {-mtype -default Warning -choices {Info Warning Error}
    -description "Message type"}
      {-font -type font -default {Arial 10 italic} -description
    "Message text font"}
      {-level -type integer -optional -range {1 10} -description
    "Message level"}
      {-fg -type color -optional -description "Message color"}
      {-log_file -type file -optional -description "Optional message
    log file"}
      {text -type string -multiple -description "Multiple text lines
    to display"}
   }
} {
}
```

Jedes der Prozedurargumente wird mit einer Liste deklariert, deren erstes Element der Name des Arguments ist, gefolgt von eventuellen Attributen. Die Syntax der Argumentdefinition lässt sich wie folgt formalisieren:

```
tepam::procedure <name> {
   -args {
      {<argument_name_1> <arg_attr_name_1a> <arg_attr_value_1a>
    <arg_attr_name_1b> <arg_attr_value_1b> ...}
      {<argument_name_2> <arg_attr_name_2a> <arg_attr_value_2a>
    <arg_attr_name_2b> <arg_attr_value_2b> ...}
      ...
   }
} <body>
```

Die Argumentnamen und Attribute müssen wie folgt verwendet werden:

Argumentname (>)

Der angegebene Argumentname legt fest, ob es sich bei dem Argument um ein *unbenanntes Argument* oder ein *benanntes Argument* handelt. Darüber hinaus kann ein Argumentname auch leer sein, um einen Argumentkommentar zu kennzeichnen, oder mit # beginnen, um einen Abschnittskommentar zu kennzeichnen.

""

Dies ist die einfachste Form eines Argumentnamens: Ein Argument, dessen Name nicht mit "-" beginnt, ist ein *unbenanntes Argument*. Der bei einem Prozeduraufruf übergebene Parameter wird einer Variable mit dem Namen zugewiesen.

```
tepam::procedure {print_string} {
   -args {
      {text -type string -description "This is an unnamed
    argument"}
   }
} {
   puts $text
}

print_string "Hello"
 -> Hello
```

"-"

Ein Argument, dessen Name mit "-" beginnt, ist ein *benanntes Argument* (auch als *Option* bezeichnet). Der bei einem Prozeduraufruf übergebene Parameter wird einer Variable mit dem Namen (nicht -) zugewiesen.

```
tepam::procedure {print_string} {
   -args {
      {-text -type string -description "This is a named argument"}
   }
} {
   puts $text
}

print_string -text "Hello"
 -> Hello
```

"--"

Dieses Flag ermöglicht es, das Ende der benannten Argumente und den Beginn der unbenannten Argumente eindeutig festzulegen, falls der Stil *benannte Argumente zuerst, unbenannte Argumente danach (Tcl)* ausgewählt wurde.

Wenn der Stil *unbenannte Argumente zuerst, benannte Argumente danach (Tk)* ausgewählt ist, wird dieses Flag ignoriert, sofern die unbenannten Argumente bereits geparst wurden. Andernfalls wird es dem entsprechenden unbenannten Argument zugewiesen.

"-" or ""

Ein leerer Argumentname (entweder '-' oder '') leitet einen Kommentar für die folgenden Argumente ein.

```
tepam::procedure {print_time} {
   -interactive_display_format short
   -args {
      {hours -type integer -description "Hour"}
      {minutes -type integer -description "Minute"}

      {- The following arguments are optional:}
      {seconds -type integer -default 0 -description "Seconds"}
      {milliseconds -type integer -default 0 -description
    "Milliseconds"}
   }
} {
   puts "${hour}h${minutes}:[expr $seconds+0.001*$milliseconds]"
}
```

Argumentkommentare werden im Wesentlichen in den grafischen Formularen zur Argumentdefinition verwendet, die erstellt werden, wenn eine Prozedur interaktiv aufgerufen wird.

"#*"

Eine Argumentdefinitionsliste, die mit "#" beginnt, wird als Abschnittskommentar gewertet. Die Argumentdefinitionsliste wird um die "#"-Zeichen gekürzt, und die verbleibende Zeichenfolge wird als Abschnittskommentar verwendet.

Abschnittskommentare können dazu dienen, den Code der Argumentdefinitionen visuell zu

strukturieren. Abschnittskommentare werden zudem zur Strukturierung der generierten Hilfetexte und der interaktiven Formulare zur Argumentdefinition verwendet.

```
tepam::procedure {complex_multiply} {
   -description "This function perform a complex multiplication"
   -args {
      {#### First complex number ####}
      {-r0 -type double -description "First number real part"}
      {-i0 -type double -description "First number imaginary part"}

      {#### Second complex number ####}
      {-r1 -type double -description "Second number real part"}
      {-i1 -type double -description "Second number imaginary
    part"}
   }
} {
   return [expr $r0*$r1 - $i0*$i1]
}
```

Argumentattribute (> >)

Die folgenden Argumentattribute werden unterstützt:

-description *string*

Das Argumentattribut description wird zu Dokumentationszwecken verwendet. Interaktive Formulare zur Definition von Argumenten nutzen dieses Attribut, um Erläuterungen zu einem Argument bereitzustellen.

-type *type*

Das Argumentattribut type ermöglicht es, das Argument entweder einem vordefinierten Datentyp oder einem anwendungsspezifischen Datentyp zuzuweisen. Die bei einem Prozeduraufruf übergebenen Argumentwerte werden automatisch auf Übereinstimmung mit dem definierten Argumenttyp überprüft.

Der Abschnitt Argumenttypen enthält eine Liste vordefinierter Datentypen und erläutert, wie anwendungsspezifische Typen festgelegt werden können.

Der Argumenttyp *none* hat eine besondere Bedeutung. Ein Argument vom Typ *none* wird als *Flag* behandelt. Ein Flag ist stets optional, und die zugehörige Variable enthält den logischen Wert **1**, wenn das Flag beim Prozeduraufruf angegeben wurde, andernfalls den Wert **0**.

-default *value*

Eventuelle Vorgabewerte können über das Argumentattribut -default definiert werden. Argumente mit Vorgabewerten sind automatisch optionale Argumente.

-optional|-mandatory

Argumente sind standardmäßig obligatorisch, sofern kein Vorgabewert definiert ist. Das Flag *-optional* macht ein Argument zu einem optionalen Argument.

Wird ein optionales Argument bei einem Prozeduraufruf nicht definiert, wird die

entsprechende Variable nicht definiert. Das Flag *-mandatory* ist das Gegenteil von *-optional* . Dieses Flag dient lediglich der Vollständigkeit, da ein Argument standardmäßig ohnehin obligatorisch ist.

-multiple
Argumente mit dem Attribut *-multiple* können während eines Prozeduraufrufs mehrfach definiert werden. Die Werte, die während eines Prozeduraufrufs für ein solches Argument übergeben werden, werden in einer Listenvariablen gespeichert. Dies gilt auch dann, wenn ein solches Argument während eines Prozeduraufrufs nur einmal definiert wird.

Das Attribut *-multiple* kann sowohl unbenannten als auch benannten Argumenten zugewiesen werden. Bei einem benannten Argument muss das Paar aus Argumentname und Wert für jeden übergebenen Wert wiederholt werden. Falls das Argument mit dem Attribut *-multiple* ein unbenanntes Argument ist, muss dieses das absolut letzte aller unbenannten Argumente sein.

-choices *list*
Über das Attribut *-choices* kann einem Argument eine mögliche Menge gültiger Argumentwerte zugewiesen werden. Der bei einem Prozeduraufruf übergebene Argumentwert wird anhand der angegebenen Auswahlwerte überprüft.

-choicelabels *list*
Jedem Auswahlwert kann mithilfe des Attributs *-choicelabels* eine kurze Beschreibung zugewiesen werden. Diese Beschreibungen werden in den generierten Hilfetexten sowie als Beschriftungen der Radio- und Checkboxen bei den interaktiven Aufrufen verwendet.

Das Attribut *-choicelabels* ist optional; sollte es jedoch definiert werden, muss seine Liste dieselbe Länge aufweisen wie die Argumentliste von *-choices*.

-range *{double double}*
Eine weitere Einschränkung des Arguments kann mit dem Attribut *-range* definiert werden. Der gültige Bereich wird durch eine Liste festgelegt, die den kleinsten und den größten zulässigen Wert enthält. Das Attribut *-range* darf ausschließlich für numerische Argumente wie Ganzzahlen und Gleitkommazahlen verwendet werden.

-validatecommand *script*
Benutzerdefinierte Validierungen von Argumentwerten können über spezifische Validierungsbefehle durchgeführt werden, die mit dem Attribut *-validatecommand* definiert sind. Der angegebene Validierungsbefehl kann ein vollständiges Skript sein, in dem das Muster *%P* durch den zu validierenden Wert ersetzt wird.

Beispiel für die Deklaration eines Validierungsbefehls:

```
tepam::procedure {display_message} {
   -args {
      {text -type string -description "Message text" \
            -validatecommand {IllegalWordDetector %P}}
} {
}
```

Zwar dient dieses benutzerdefinierte Attribut zur Validierung eines bestimmten Arguments, doch gibt es auch ein globales Attribut *-validatecommand*, mit dem Validierungen

durchgeführt werden können, die mehrere Argumente umfassen.

-validatecommand_error_text *string*
Mit diesem Attribut lässt sich die Standardfehlermeldung für eine benutzerdefinierte Validierung eines Arguments (definiert durch *-validatecommand*) überschreiben.

-widget *string*
Die Widgets, mit denen die verschiedenen Argumente bei einem interaktiven Aufruf einer Prozedur definiert werden können, werden in der Regel automatisch entsprechend dem Argumenttyp ausgewählt. Mit dem Attribut *-widget* lässt sich für ein Argument explizit ein bestimmter Widget-Typ festlegen.

-auxargs *list*
Wird eine Prozedur interaktiv aufgerufen, können dem interaktiven Formular zur Argumentdefinition zusätzliche Argumentattribute über das Attribut *-auxargs* übergeben werden, das selbst eine Liste aus Attributnamen- und Attributwertpaaren darstellt:

```
-auxargs {-<arg_attr_name_1a> <arg_attr_value_1a> \
          -<arg_attr_name_1b> <arg_attr_value_1b>
          ...
}
```

Wenn eine Prozedur beispielsweise einen Dateinamen als Argument entgegennimmt, kann es sinnvoll sein, den erforderlichen Dateityp für das interaktive Argumentdefinitionsformular festzulegen. Diese Information kann über das Attribut *-auxargs* an das Argumentdefinitionsformular übermittelt werden:

```
tepam::procedure LoadPicture {
   -args {
      {FileName -type existingfile -description "Picture file" \
                -auxargs {-filetypes {{"GIF" {*.gif}} {"JPG"
    {*.jpg}} }}}
   }
} {
}
```

-auxargs_commands *script*
Wenn die Attribute der Hilfsargumente nicht statisch sind, sondern dynamisch anpassbar sein müssen, ermöglicht das Attribut *-auxargs_commands* deren Definition über Befehle, die während eines Prozeduraufrufs ausgeführt werden. Dem Attribut *-auxargs_commands* muss eine Liste von Paaren aus Hilfsattributnamen und Befehlen übergeben werden. Die übergebenen Befehle werden im Kontext der aufrufenden Prozedur ausgeführt.

```
-auxargs_commands {-<arg_attr_name_1a> <arg_attr_command_1a> \
                   -<arg_attr_name_1b> <arg_attr_command_1b>
                   ...
}
```

Variablen

Mehrere im Namespace **::tepam** definierte Variablen beeinflussen die Funktionsweise der Prozeduren, die mit dem TEPAM-Befehl **procedure** deklariert wurden.

named_arguments_first

Diese Variable legt den allgemeinen Aufrufstil der Prozeduren fest. Sie ist standardmäßig auf **1** gesetzt, wodurch der Stil *benannte Argumente zuerst, unbenannte Argumente danach* (Tcl-Stil) ausgewählt wird.

Durch Setzen dieser Variable auf **0** wird der Stil *unbenannte Argumente zuerst, benannte Argumente danach* global ausgewählt (Tk-Stil):

```
set tepam::named_arguments_first 0
```

Während diese Variable den allgemeinen Aufrufstil festlegt, kann das Prozedurattribut *-named_arguments_first* diesen Stil für jede deklarierte Prozedur individuell anpassen.

auto_argument_name_completion

Diese Variable steuert den allgemeinen Modus für den automatischen Abgleich von Argumentnamen. Standardmäßig ist sie auf **1** gesetzt, was bedeutet, dass die aufgerufenen Prozeduren versuchen, eventuell abgekürzte Argumentnamen mit den deklarierten Argumentnamen abzugleichen.

Durch Setzen dieser Variable auf **0** wird der Modus für den automatischen Abgleich der Argumentnamen deaktiviert:

```
set tepam::auto_argument_name_completion 0
```

Während diese Variable den allgemeinen Abgleichmodus festlegt, kann das Prozedurattribut *-auto_argument_name_completion* diesen Modus für jede deklarierte Prozedur individuell anpassen.

interactive_display_format

Eine über den TEPAM-Befehl **procedure** deklarierte Prozedur kann stets mit dem Schalter **-interactive** aufgerufen werden. Auf diese Weise wird ein grafisches Formular generiert, das die interaktive Eingabe aller Prozedurargumente ermöglicht.

Für diese interaktiven Formulare stehen zwei Anzeigemodi zur Verfügung. Der Modus *extended*, der als Standardmodus dient, eignet sich eher für Prozeduren mit wenigen Argumenten. Die Form **short** ist besser für Prozeduren mit sehr vielen Argumenten geeignet:

```
set tepam::interactive_display_format "short"
```

Die Entscheidung, ob kurze oder erweiterte Formulare verwendet werden sollen, kann über die Variable **interactive_display_format** global konfiguriert werden. Diese globale Einstellung kann für eine Prozedur mithilfe des Prozedurattributs *-interactive_display_format* individuell geändert werden.

help_line_length

Mit dieser Variable lässt sich die maximale Zeilenlänge festlegen, die vom Prozedur-Hilfetextgenerator verwendet wird. Die Standardlänge, die auf 80 (Zeichen) festgelegt ist, lässt sich problemlos an die Anforderungen einer Anwendung anpassen:

```
set tepam::help_line_length 120
```

Da diese Variable direkt bei der Erstellung des Hilfetextes verwendet wird, kann ihr Wert kontinuierlich an den aktuellen Bedarf angepasst werden.

command_log

Prozeduraufrufe können in der Listenvariablen **tepam::ProcedureCallLogList** protokolliert werden. Die Variable **tepam::command_log** steuert die Standardprotokollierungseinstellungen für alle Prozeduren. Folgende Konfigurationen werden unterstützt:
- *0*: Deaktiviert die Protokollierung aller Prozeduraufrufe
- *1*: Aktiviert die Protokollierung aller Prozeduraufrufe
- "*interactive*": Protokolliert alle interaktiv aufgerufenen Prozeduren (d. h. Prozeduren, die mit dem Flag -interactive aufgerufen werden). Dies ist die Standardkonfiguration.

Diese Standardkonfiguration für die Protokollierung kann für jede Prozedur individuell über das Attribut *-command_log* geändert werden.

Argumenttypen

TEPAM bietet eine umfassende Auswahl an Argumenttypen für Prozeduren. Diese lassen sich bei Bedarf problemlos durch anwendungsspezifische Typen ergänzen.

Vordefinierte Argumenttypen

Zur Erinnerung: Jedem angegebenen Argument einer Prozedur kann ein Typ zugewiesen werden:

```
tepam::procedure {warning} {
   -args {
      {-font -type font -default {Arial 10 italic}}
      {-severity_level -type integer -optional -range {1 10}}
      {-fg -type color -optional -description "Message color"}
      {text -type string -multiple -description "Multiple text lines
    to display"}
   }
} {
   ...
}
```

Es gibt einige *Spezialtypen*, die die erste Kategorie vordefinierter Argumenttypen bilden:

- **none**

Ein *Flag*, auch *Schalter* genannt, ist als benanntes Argument definiert, das den Typ **none** hat. Flags sind stets optional, und der Vorgabewert der zugewiesenen Variable wird auf **0** gesetzt. Im Gegensatz zu den (normalen) benannten Argumenten muss für ein Flag kein Wert für das Argument angegeben werden.

Da für ein Flag kein Wert angegeben werden muss, wird für diesen Argumenttyp auch keine Datenprüfung durchgeführt.

```
tepam::procedure flag_test {
   -args {
      {-flag -type none -description "This is a flag"}
   }
```

```
} {
   puts $flag
}

flag_test
-> 0

flag_test -flag
-> 1
```

- **string**

  String ist ein generischer Argumentdatentyp. Einem Argument vom Typ string kann eine beliebige Zeichenkette übergeben werden, weshalb keine Datentypüberprüfungen durchgeführt werden. Der Typ string ermöglicht die Definition einzeiliger Zeichenketten bei interaktiven Aufrufen von Prozeduren.
- **text**

  Text entspricht **string**, mit dem einzigen Unterschied, dass es die Eingabe mehrzeiliger Zeichenfolgen bei interaktiven Aufrufen von Prozeduren ermöglicht.
- **{}**

  {} Ein leerer Argumenttyp (**blank**) bezeichnet einen undefinierten Argumenttyp. Dies ist der Standardargumenttyp, der verwendet wird, wenn kein Typ explizit angegeben wurde. Ein Argument vom Typ **blank** verhält sich genauso wie ein Argument vom Typ **string**, d. h., es werden keine Datenprüfungen für Argumente durchgeführt. Der einzige Unterschied besteht darin, dass der Datentyp **string** in der generierten Hilfedokumentation erwähnt wird, während dies beim Typ **blank** nicht der Fall ist.

Von TEPAM werden mehrere *numerische Typen* definiert. Die Typvalidierungsprozeduren verwenden die Befehle **string is -strict**, um die Gültigkeit der übergebenen Argumente zu überprüfen, wodurch sichergestellt wird, dass keine leeren Zeichenfolgen als Argumentwert akzeptiert werden. Der Typvalidierungsausdruck für die numerischen Typen und die Argumenttypen, auf die dieser Ausdruck angewendet wird, lauten wie folgt:

```
string is <type_to_check> -strict <argument_value>
```

- *boolean*
- *integer*
- *double*

Leere Zeichenfolgen werden als Argumentwert für alle alphanumerischen Argumenttypen akzeptiert. Die Argumenttypen, die in diese Kategorie fallen, sowie der für sie verwendete Validierungsausdruck lauten wie folgt:

```
string is <type_to_check> <argument_value>
```

- *alnum*
- *alpha*
- *ascii*
- *control*
- *digit*

- *graph*
- *lower*
- *print*
- *punct*
- *space*
- *upper*
- *wordchar*
- *xdigit*

Zusätzlich zu den mit den Befehlen **string is** überprüften Datentypen legt TEPAM einige weitere nützliche Datentypen fest:

- *char*

  Jede Zeichenkette mit einer Länge von 1 Zeichen entspricht dem Typ *character*. Die Typüberprüfung erfolgt anhand des folgenden Ausdrucks:

  ```
  expr [string length <argument_value>]==1
  ```

- *color*

  Alle Zeichenfolgen, die von Tk als Farbe akzeptiert werden, gelten als gültiges Farbargument. Beachten Sie, dass das Tk-Paket geladen sein muss, um den Typ *color* verwenden zu können. TEPAM verwendet den folgenden Befehl zur Validierung des Farbtyps:

  ```
  expr ![catch {winfo rgb . <argument_value>}]
  ```

- *font*

  Alle Zeichenfolgen, die von Tk als Schriftart akzeptiert werden, gelten als gültiges Schriftart-Argument. Beachten Sie, dass das Tk-Paket geladen sein muss, um den Typ *font* verwenden zu können. TEPAM verwendet den folgenden Befehl zur Validierung des Schriftarttyps:

  ```
  expr ![catch {font measure <argument_value> ""}]
  ```

- *file*

  Alle Zeichenfolgen, die keines der folgenden Zeichen enthalten, gelten als gültige Dateinamen: * ? " < >. Es ist nicht erforderlich, dass die Datei und das Verzeichnis, in dem sie sich befindet, vorhanden sind. Zeichenfolgen der Länge Null gelten nicht als gültige Dateinamen.

  Zur Validierung der Dateinamen wird der folgende Ausdruck verwendet:

  ```
  expr [string length <argument_value>]>0 && ![regexp {[\"*?<>:]}
      <argument_value>]
  ```

- *existingfile*

  Das Argument ist gültig, wenn es mit einer vorhandenen Datei übereinstimmt. Zur Validierung der Argumente dieses Typs wird folgende Prüfung durchgeführt:

  ```
  file exists <argument_value>
  ```

- *directory*

  Ein Argument vom Typ directory wird auf genau dieselbe Weise validiert wie die Argumente vom Typ file.

- *existingdirectory*

Das Argument ist gültig, wenn es mit einem vorhandenen Verzeichnis übereinstimmt. Zur Validierung der Argumente dieses Typs wird folgende Prüfung durchgeführt:

```
file isdirectory <argument_value>
```

Anwendungsspezifische Argumenttypen definieren

Um die Unterstützung für einen neuen anwendungsspezifischen Argumenttyp hinzuzufügen, muss lediglich im Namespace tepam eine Validierungsfunktion **Validation()** hinzugefügt werden. Diese Funktion erwartet ein Argument. Sie muss **1** zurückgeben, wenn das übergebene Argument dem entsprechenden Datentyp entspricht, andernfalls **0**.

Der Abschnitt mit den Validierungsbefehlen im Paket *tepam.tcl* enthält genügend Beispiele für Validierungsfunktionen, da dort die Funktionen für die Standard-TEPAM-Typen implementiert sind.

Der folgende zusätzliche Codeausschnitt zeigt die Validierungsfunktion für einen benutzerdefinierten Argumenttyp, der Werte mit einer Zeichenfolgenlänge von genau 2 erfordert:

```
proc tepam::Validate(two_char) {v} {expr {[string length $v]==2}}
```

Prozeduren-Aufrufe

Hilfe

Jede Prozedur kann mit dem Flag *-help* aufgerufen werden. Die Prozedur gibt dann einen generierten Hilfetext an *stdout* aus und kehrt anschließend zurück, ohne weitere Aktionen auszuführen.

Für die erste in Prozeduren-Aufrufe deklarierte Prozedur würden die Hilfeanfrage und der ausgegebene Hilfetext wie folgt lauten:

```
display message -help
->
NAME
      display message - Displays a simple message box
SYNOPSIS
      display message
            [-mtype <mtype>]
               Message type, default: "Warning", choices: {Info,
    Warning, Error}
            <text>
               Multiple text lines to display, type: string
DESCRIPTION
      This procedure allows displaying a configurable message box. The
    default
      message type that is created is a warning, but also errors and
    info can
      be generated.
      The procedure accepts multiple text lines.
EXAMPLE
      display message -mtype Warning "Save first your job"
```

Der Argumentmanager prüft, ob das zuletzt übergebene Argument *-help* lautet, und generiert in diesem Fall die angeforderte Hilfemeldung. Somit wird auch im folgenden Beispiel die Hilfemeldung ausgegeben:

```
display message -mtype Info "It is 7:00" -help
```

Andererseits führt der folgende Aufruf zu einem Fehler:

```
display message -help -mtype Info "It is 7:00"
->
display message: Argument '-help' not known
```

Interaktiver Aufruf einer Prozedur

Wenn Tk geladen wurde, kann eine Prozedur mit dem Flag *-interactive* aufgerufen werden, um ein grafisches Formular zu öffnen, in dem alle Prozedurargumente interaktiv angegeben werden können. Das folgende Beispiel stellt sicher, dass die Tk-Bibliothek geladen ist, und zeigt die Befehlszeile zum interaktiven Aufruf der unter **PROZEDUREN-AUFRUFE** deklarierten Prozedur:

```
package require Tk
display message -interactive
```

Auch das Flag *-interactive* muss an der letzten Argumentposition stehen, wie es auch für das Flag *-help* erforderlich ist. Argumente, die vor dem Flag *-interactive* definiert sind, werden ignoriert. Das folgende Beispiel ist daher ebenfalls ein gültiger interaktiver Aufruf einer Prozedur:

```
display message -mtype Info "It is 7:00" -interactive
```

Unbenannte Argumente

Unbenannte Argumente werden der aufgerufenen Prozedur in der Regel als einfache Parameter übergeben. Bei dieser Form des Prozeduraufrufs müssen die übergebenen Argumente streng der Reihenfolge der angegebenen Argumente entsprechen. Dem letzten Argument können mehrere Parameter zugewiesen werden, sofern dieses über das Attribut *-multiple* verfügt. Somit gilt für die folgende deklarierte Prozedur ...

```
tepam::procedure {display_message} {
   -args {
      {mtype -choices {Info Warning Error}}
      {text -type string -multiple}
   }
} {
   puts "$mtype: [join $text]"
}
```

... kann beispielsweise auf folgende Weise aufgerufen werden:

```
display_message Info "It is PM 7:00."
-> Info: It is PM 7:00.

display_message Info "It is PM 7:00." "You should go home."
-> Info: It is PM 7:00. You should go home.
```

Das Schöne daran ist, dass unbenannte Argumente auch als benannte Argumente übergeben werden können, was beispielsweise dann praktisch sein kann, wenn einem Benutzer die genaue Reihenfolge der Argumente nicht bekannt ist:

```
display_message -mtype Info -text "It is PM 7:00."
-> Info: It is PM 7:00.

display_message -text "It is PM 7:00." -mtype Info
-> Info: It is PM 7:00.

display_message -mtype Info -text "It is PM 7:00." -text "You should
    go home."
-> Info: It is PM 7:00. You should go home.

display_message -text "It is PM 7:00." -text "You should go home."
    -mtype Info
-> Info: It is PM 7:00. You should go home.
```

Benannte Argumente

Benannte Argumente müssen einer Prozedur in Form von Parameterpaaren übergeben werden, die sich aus den Argumentnamen und den Argumentwerten zusammensetzen. Die Reihenfolge, in der sie bei einem Prozeduraufruf übergeben werden, ist unerheblich und muss nicht mit der Reihenfolge in der Argumentspezifikation übereinstimmen.

Die folgende deklarierte Prozedur ...

```
tepam::procedure {display_message} {
   -args {
      {-mtype -choices {Info Warning Error}}
      {-text -type string -multiple}
   }
} {
   puts "$mtype: [join $text]"
}
```

... kann auf folgende Weise aufgerufen werden:

```
display_message -mtype Info -text "It is PM 7:00."
-> Info: It is PM 7:00.

display_message -text "It is PM 7:00." -mtype Info
-> Info: It is PM 7:00.

display_message -mtype Info -text "It is PM 7:00." -text "You should
    go home."
-> Info: It is PM 7:00. You should go home.

display_message -text "It is PM 7:00." -text "You should go home."
    -mtype Info
-> Info: It is PM 7:00. You should go home.
```

Auch benannte Argumente, die nicht über das Attribut *-multiple* verfügen, können mehrfach angegeben werden. In einem solchen Fall wird nur das zuletzt angegebene Argument berücksichtigt:

```
display_message -mtype Info -text "It is PM 7:00." -mtype Warning
-> Warning: It is PM 7:00.
```

Unbenannte Argumente zuerst, benannte Argumente danach (Tk-Stil)

Eine Prozedur, die definiert wurde, während die Variable **tepam::named_arguments_first** auf 1 gesetzt war, oder bei der das Prozedurattribut *-named_arguments_first* auf 1 gesetzt war, muss im Tcl-Stil aufgerufen werden. Die folgende Deklaration einer Prozedur wird in diesem Abschnitt verwendet, um die Bedeutung dieser Aufrufweise zu veranschaulichen:

```
set tepam::named_arguments_first 1
tepam::procedure my_proc {
   -args {
      {-n1 -default ""}
      {-n2 -default ""}
      {u1 -default ""}
      {u2 -default ""}
   }
} {
   puts "n1:'$n1', n2:'$n2', u1:'$u1', u2:'$u2'"
}
```

Die unbenannten Argumente werden am Ende des Prozeduraufrufs hinter den benannten Argumenten platziert:

```
my_proc -n1 N1 -n2 N2 U1 U2
-> n1:'N1', n2:'N2', u1:'U1', u2:'U2'
```

Der Argumentparser betrachtet das erste Argument, das nicht mit dem Zeichen "-" beginnt, sowie alle nachfolgenden Argumente als unbenannte Argumente:

```
my_proc U1 U2
-> n1:'', n2:'', u1:'U1', u2:'U2'
```

Benannte Argumente können mehrfach definiert werden. Wenn das benannte Argument über das Attribut *-multiply* verfügt, werden alle Argumentwerte in einer Liste zusammengefasst. Andernfalls wird nur der zuletzt angegebene Wert beibehalten:

```
my_proc -n1 N1 -n2 N2 -n1 M1 U1 U2
-> n1:'M1', n2:'N2', u1:'U1', u2:'U2'
```

Der Name des ersten unbenannten Arguments darf daher nicht mit dem Zeichen "-" beginnen. Andernfalls wird das unbenannte Argument als Name eines anderen benannten Arguments interpretiert. Dies ist insbesondere dann von Bedeutung, wenn das erste unbenannte Argument durch eine Variable angegeben wird, die beliebige Zeichenfolgen enthalten kann:

```
my_proc -n1 N1 -n2 N2 "->" "<-"
-> my_proc: Argument '->' not known

set U1 "->"
my_proc -n1 N1 -n2 N2 $U1 U2
my_proc: Argument '->' not known
```

Das Flag "--" ermöglicht es, die unbenannten Argumente eindeutig von den benannten Argumenten zu trennen. Alle Daten nach dem Flag "--" werden als unbenanntes Argument betrachtet:

```
my_proc -n1 N1 -n2 N2 -- "->" "<-"
-> n1:'N1', n2:'N2', u1:'->', u2:'<-'

set U1 "->"
my_proc -n1 N1 -n2 N2 -- $U1 U2
-> n1:'N1', n2:'N2', u1:'->', u2:'<-'
```

Benannte Argumente zuerst, unbenannte Argumente danach (Tcl-Stil)

Der Tk-Aufrufstil wird gewählt, wenn eine Prozedur definiert wird, während die Variable **tepam::named_arguments_first** auf 0 gesetzt ist, oder wenn das Prozedurattribut *-named_arguments_first* auf 0 gesetzt wurde. Zur Veranschaulichung dieser Aufrufart wird in diesem Abschnitt die folgende Prozedur verwendet:

```
set tepam::named_arguments_first 0
tepam::procedure my_proc {
   -args {
      {-n1 -default ""}
      {-n2 -default ""}
      {u1}
      {u2 -default "" -multiple}
   }
} {
   puts "n1:'$n1', n2:'$n2', u1:'$u1', u2:'$u2'"
}
```

In diesem Fall müssen die unbenannten Argumente zuerst angegeben werden. Die benannten Argumente werden anschließend angegeben:

```
my_proc U1 U2 -n1 N1 -n2 N2
-> n1:'N1', n1:'N1', u1:'U1', u2:'U2'
```

Der Argumentparser weist jedem definierten, unbenannten Argument einen Wert zu, bevor er zum Einlesen der benannten Argumente übergeht. Dieses Standardverhalten ändert sich geringfügig, wenn es unbenannte Argumente gibt, die optional sind oder mehrere Werte annehmen können.

Einem unbenannten Argument, das optional ist (das entweder das Attribut *-optional* besitzt oder einen Vorgabewert hat), wird nur dann ein Wert zugewiesen, wenn der Wert nicht mit dem Zeichen "-" beginnt oder wenn keine benannten Argumente definiert sind. Ein Wert, der mit "-" beginnt, wird andernfalls als Name eines benannten Arguments betrachtet.

Einem Argument, das das Attribut *-multiple* aufweist, werden Werte zugewiesen, solange der Parameterwert nicht mit dem Zeichen "-" beginnt.

Werte, die mit dem Zeichen "-" beginnen, können daher keinen optionalen, unbenannten Argumenten zugewiesen werden, was die Verwendung des Tcl-Prozeduraufrufstils einschränkt. Der Tk-Stil ist in manchen Fällen möglicherweise vorzuziehen, da er eine eindeutige Trennung der benannten von den unbenannten Argumenten mittels des Flags "--" ermöglicht.

Lassen Sie uns nun etwas weniger theoretisch untersuchen, auf welche Weise die zuvor definierte Prozedur aufgerufen werden kann: Im ersten Beispiel wird die Prozedur ohne

Parameter aufgerufen, was zu einem Fehler führt, da *u1* ein obligatorisches Argument ist:

```
my_proc
-> my_proc: Required argument is missing: u1
```

Der Prozeduraufruf ist gültig, wenn für *u1* ein Parameter angegeben wird:

```
my_proc U1
-> n1:'', n2:'', u1:'U1', u2:''
```

Wenn weitere Parameter angegeben werden, die nicht mit dem Zeichen "-" beginnen, werden diese den unbenannten Argumenten zugeordnet. *U2* erhält drei dieser Parameter, da es mehrere Werte akzeptiert:

```
my_proc U1 U2 U3 U4
-> n1:'', n2:'', u1:'U1', u2:'U2 U3 U4'
```

Sobald ein Parameter mit "-" beginnt und alle unbenannten Argumente zugewiesen wurden, versucht der Argumentmanager, den Parameter als Namen eines benannten Arguments zu interpretieren. Der Prozeduraufruf schlägt fehl, wenn einem unbenannten Argument ein Wert zugewiesen wird, der mit "-" beginnt:

```
my_proc U1 U2 U3 U4 -U5
-> my_proc: Argument '-U5' not known
```

Die Zuweisung eines Parameters zu einem benannten Argument schlägt fehl, wenn undefinierte, unbenannte (nicht optionale) Argumente vorhanden sind. Die Namensangabe wird in diesem Fall einfach als Parameterwert betrachtet, der dem *nächsten* unbenannten Argument zugewiesen wird. Dies war im folgenden Beispiel sicherlich nicht beabsichtigt:

```
my_proc -n1 N1
-> n1:'', n2:'', u1:'-n1', u2:'N1'
```

Die Situation ist völlig anders, wenn allen obligatorischen, unbenannten Argumenten bereits Werte zugewiesen wurden. Ein Parameter, der mit dem Zeichen "-" beginnt, wird in diesem Fall als Namensbezeichner für ein benanntes Argument betrachtet:

```
my_proc U1 -n1 N1
-> n1:'N1', n2:'', u1:'U1', u2:''
```

Hinter den benannten Argumenten sind keine unbenannten Argumente zulässig:

```
my_proc U1 -n1 N1 U2
-> my_proc: Argument 'U2' is not an option
```

Das Flag "--" hat keine besondere Bedeutung, wenn nicht allen obligatorischen Argumenten ein Wert zugewiesen wurde. Dieses Flag wird einfach einem der unbenannten Argumente zugeordnet:

```
my_proc -- -n1 N1
-> n1:'N1', n2:'', u1:'--', u2:''
```

Das Flag "--" wird jedoch einfach ignoriert, wenn der Argumentparser bereits mit der Verarbeitung der benannten Argumente begonnen hat:

```
my_proc U1 -- -n1 N1
-> n1:'N1', n2:'', u1:'U1', u2:''

my_proc U1 -n1 N1 -- -n2 N2
-> n1:'N1', n2:'N2', u1:'U1', u2:''
```

Roh-Argumentliste

Manchmal kann es erforderlich sein, dass der Prozedurkörper auf die gesamte Liste der bei einem Prozeduraufruf übergebenen Argumente zugreifen kann. Dies ist über die Variable **args** möglich, die stets die unverarbeitete Argumentliste enthält:

```
tepam::procedure {display_message} {
   -args {
      {-mtype -choices {Warning Error} -default Warning}
      {text -type string -multiple}

   }
} {
   puts "args: $args"
}
display_message -mtype Warning "It is 7:00"
-> args: -mtype Warning {It is 7:00}
```

Kategorie

Prozeduren, Argumente, Parameter, Optionen

Copyright

```
Copyright (c) 2009-2013, Andreas Drollinger
```

term

ansi_cattr

Stichwörter: ansi_cattr, ansi, attribute control, color control, control, terminal

Name

term::ansi::code::attr – ANSI-Attributsequenzen

Übersicht

package require Tcl 8.5 9
package require term::ansi::code ?**0.2**?
package require term::ansi::code::attr ?**0.2**?
::term::ansi::code::attr::names
::term::ansi::code::attr::import ?*ns*? ?*arg...*?
::term::ansi::code::attr::fgblack
::term::ansi::code::attr::fgred
::term::ansi::code::attr::fggreen

::term::ansi::code::attr::fgyellow
::term::ansi::code::attr::fgblue
::term::ansi::code::attr::fgmagenta
::term::ansi::code::attr::fgcyan
::term::ansi::code::attr::fgwhite
::term::ansi::code::attr::fgdefault
::term::ansi::code::attr::bgblack
::term::ansi::code::attr::bgred
::term::ansi::code::attr::bggreen
::term::ansi::code::attr::bgyellow
::term::ansi::code::attr::bgblue
::term::ansi::code::attr::bgmagenta
::term::ansi::code::attr::bgcyan
::term::ansi::code::attr::bgwhite
::term::ansi::code::attr::bgdefault
::term::ansi::code::attr::bold
::term::ansi::code::attr::dim
::term::ansi::code::attr::italic
::term::ansi::code::attr::underline
::term::ansi::code::attr::blink
::term::ansi::code::attr::revers
::term::ansi::code::attr::hidden
::term::ansi::code::attr::strike
::term::ansi::code::attr::nobold
::term::ansi::code::attr::noitalic
::term::ansi::code::attr::nounderline
::term::ansi::code::attr::noblink
::term::ansi::code::attr::norevers
::term::ansi::code::attr::nohidden
::term::ansi::code::attr::nostrike
::term::ansi::code::attr::reset

Beschreibung

Dieses Paket stellt symbolische Namen für die ANSI-Attribut-Steuercodes bereit. Für jeden Steuercode wird ein einzelner Befehl bereitgestellt, der diesen Code als Ergebnis zurückgibt. Keiner der Befehle dieses Pakets schreibt in einen Kanal; dies wird von Paketen höherer Ebene übernommen, wie beispielsweise **term::ansi::send**.

API

Introspektion

::term::ansi::code::attr::names
: Dieser Befehl dient der Introspektion. Er gibt als Ergebnis eine Liste zurück, die die Namen aller Attributbefehle enthält.

::term::ansi::code::attr::import ?*ns*? ?*arg*...?
: Dieser Befehl importiert einige oder alle Attributbefehle in den Namespace *ns*. Standardmäßig ist dies der Namespace *attr*. Beachten Sie, dass es sich hierbei um einen relativen

Namespace handelt, wodurch der importierte Befehl in einem Unternamespace des aktuellen Namespaces abgelegt wird. Standardmäßig werden alle Befehle importiert; dies kann jedoch eingeschränkt werden, indem die Namen der gewünschten Befehle nach dem Namespace-Argument aufgeführt werden.

Attribute

::term::ansi::code::attr::fgblack
Setzt die Textfarbe auf *Black*.

::term::ansi::code::attr::fgred
Setzt die Textfarbe auf *Red*.

::term::ansi::code::attr::fggreen
Setzt die Textfarbe auf *Green*.

::term::ansi::code::attr::fgyellow
Setzt die Textfarbe auf *Yellow*.

::term::ansi::code::attr::fgblue
Setzt die Textfarbe auf *Blue*.

::term::ansi::code::attr::fgmagenta
Setzt die Textfarbe auf *Magenta*.

::term::ansi::code::attr::fgcyan
Setzt die Textfarbe auf *Cyan*.

::term::ansi::code::attr::fgwhite
Setzt die Textfarbe auf *White*.

::term::ansi::code::attr::fgdefault
Setzt die Standardtextfarbe (*Black*).

::term::ansi::code::attr::bgblack
Setzt den Hintergrund auf *Black*.

::term::ansi::code::attr::bgred
Setzt den Hintergrund auf *Red*.

::term::ansi::code::attr::bggreen
Setzt den Hintergrund auf *Green*.

::term::ansi::code::attr::bgyellow
Setzt den Hintergrund auf *Yellow*.

::term::ansi::code::attr::bgblue
Setzt den Hintergrund auf *Blue*.

::term::ansi::code::attr::bgmagenta
Setzt den Hintergrund auf *Magenta*.

::term::ansi::code::attr::bgcyan
Setzt den Hintergrund auf *Cyan*.

::term::ansi::code::attr::bgwhite
Setzt den Hintergrund auf *White*.

::term::ansi::code::attr::bgdefault
Standardhintergrund festlegen (transparent).

::term::ansi::code::attr::bold
Weiter so!

::term::ansi::code::attr::dim
Schaltet die abgeschwächte Darstellung ein.

::term::ansi::code::attr::italic
Kursivschrift aktivieren.

::term::ansi::code::attr::underline
Unterstrich aktivieren.

::term::ansi::code::attr::blink
Bitte blinken Sie weiter.

::term::ansi::code::attr::revers
Rückwärtsgang einlegen.

::term::ansi::code::attr::hidden
Ausgeblendet.

::term::ansi::code::attr::strike
Durchstreichen aktiviert.

::term::ansi::code::attr::nobold
Fettdruck deaktivieren.

::term::ansi::code::attr::noitalic
Kursivschrift beenden.

::term::ansi::code::attr::nounderline
Unterstrich aus.

::term::ansi::code::attr::noblink
Blinken ausschalten.

::term::ansi::code::attr::norevers
Bitte fahren Sie rückwärts weg.

::term::ansi::code::attr::nohidden
Ausgeblendet.

::term::ansi::code::attr::nostrike
Durchstreichung deaktivieren.

::term::ansi::code::attr::reset
Setzt alle Attribute auf ihre Standardwerte zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

ansi_cctrl

Stichwörter: ansi_cctrl, ansi, attribute control, color control, control, terminal

Name

term::ansi::code::ctrl – ANSI-Steuersequenzen

Übersicht

package require Tcl 8.5 9
package require term::ansi::code ?**0.2**?
package require term::ansi::code::ctrl ?**0.3**?
::term::ansi::code::ctrl::names
::term::ansi::code::ctrl::import ?*ns*? ?*arg...*?
::term::ansi::code::ctrl::eeol
::term::ansi::code::ctrl::esol
::term::ansi::code::ctrl::el
::term::ansi::code::ctrl::ed
::term::ansi::code::ctrl::eu
::term::ansi::code::ctrl::es
::term::ansi::code::ctrl::sd
::term::ansi::code::ctrl::su
::term::ansi::code::ctrl::ch
::term::ansi::code::ctrl::sc
::term::ansi::code::ctrl::rc
::term::ansi::code::ctrl::sca
::term::ansi::code::ctrl::rca
::term::ansi::code::ctrl::st
::term::ansi::code::ctrl::ct
::term::ansi::code::ctrl::cat
::term::ansi::code::ctrl::qdc
::term::ansi::code::ctrl::qds
::term::ansi::code::ctrl::qcp
::term::ansi::code::ctrl::rd
::term::ansi::code::ctrl::elw
::term::ansi::code::ctrl::dlw
::term::ansi::code::ctrl::eg
::term::ansi::code::ctrl::lg
::term::ansi::code::ctrl::scs0 *tag*
::term::ansi::code::ctrl::scs1 *tag*
::term::ansi::code::ctrl::sda *arg...*
::term::ansi::code::ctrl::sda_fgblack
::term::ansi::code::ctrl::sda_fgred
::term::ansi::code::ctrl::sda_fggreen
::term::ansi::code::ctrl::sda_fgyellow

::term::ansi::code::ctrl::sda_fgblue
::term::ansi::code::ctrl::sda_fgmagenta
::term::ansi::code::ctrl::sda_fgcyan
::term::ansi::code::ctrl::sda_fgwhite
::term::ansi::code::ctrl::sda_fgdefault
::term::ansi::code::ctrl::sda_bgblack
::term::ansi::code::ctrl::sda_bgred
::term::ansi::code::ctrl::sda_bggreen
::term::ansi::code::ctrl::sda_bgyellow
::term::ansi::code::ctrl::sda_bgblue
::term::ansi::code::ctrl::sda_bgmagenta
::term::ansi::code::ctrl::sda_bgcyan
::term::ansi::code::ctrl::sda_bgwhite
::term::ansi::code::ctrl::sda_bgdefault
::term::ansi::code::ctrl::sda_bold
::term::ansi::code::ctrl::sda_dim
::term::ansi::code::ctrl::sda_italic
::term::ansi::code::ctrl::sda_underline
::term::ansi::code::ctrl::sda_blink
::term::ansi::code::ctrl::sda_revers
::term::ansi::code::ctrl::sda_hidden
::term::ansi::code::ctrl::sda_strike
::term::ansi::code::ctrl::sda_nobold
::term::ansi::code::ctrl::sda_noitalic
::term::ansi::code::ctrl::sda_nounderline
::term::ansi::code::ctrl::sda_noblink
::term::ansi::code::ctrl::sda_norevers
::term::ansi::code::ctrl::sda_nohidden
::term::ansi::code::ctrl::sda_nostrike
::term::ansi::code::ctrl::sda_reset
::term::ansi::send::fcp *row col*
::term::ansi::code::ctrl::cu ?*n*?
::term::ansi::code::ctrl::cd ?*n*?
::term::ansi::code::ctrl::cf ?*n*?
::term::ansi::code::ctrl::cb ?*n*?
::term::ansi::code::ctrl::ss ?*s e*?
::term::ansi::code::ctrl::skd *code str*
::term::ansi::code::ctrl::title *str*
::term::ansi::code::ctrl::gron
::term::ansi::code::ctrl::groff
::term::ansi::code::ctrl::tlc
::term::ansi::code::ctrl::trc
::term::ansi::code::ctrl::brc
::term::ansi::code::ctrl::blc
::term::ansi::code::ctrl::ltj
::term::ansi::code::ctrl::ttj

::term::ansi::code::ctrl::rtj
::term::ansi::code::ctrl::btj
::term::ansi::code::ctrl::fwj
::term::ansi::code::ctrl::hl
::term::ansi::code::ctrl::vl
::term::ansi::code::ctrl::groptim *str*
::term::ansi::code::ctrl::clear
::term::ansi::code::ctrl::init
::term::ansi::code::ctrl::showat *row col text*

Beschreibung

Dieses Paket stellt symbolische Namen für die ANSI-Steuersequenzen bereit. Für jede Sequenz wird ein einzelner Befehl bereitgestellt, der die Sequenz als Ergebnis zurückgibt. Keiner der Befehle dieses Pakets schreibt in einen Kanal; dies wird von Paketen höherer Ebene übernommen, wie beispielsweise **term::ansi::send**.

API

Introspektion

::term::ansi::code::ctrl::names
Dieser Befehl dient der Introspektion. Er gibt als Ergebnis eine Liste zurück, die die Namen aller Attributbefehle enthält.

::term::ansi::code::ctrl::import ?*ns*? ?*arg*...?
Dieser Befehl importiert einige oder alle Attributbefehle in den Namespace *ns*. Standardmäßig ist dies der Namespace *ctrl*. Beachten Sie, dass es sich hierbei um einen relativen Namensraumnamen handelt, wodurch der importierte Befehl in einen Unternamensraum des aktuellen Namensraums eingefügt wird. Standardmäßig werden alle Befehle importiert; dies kann jedoch eingeschränkt werden, indem die Namen der gewünschten Befehle nach dem Namensraum-Argument aufgeführt werden.

Sequenzen

::term::ansi::code::ctrl::eeol
Löschen bis zum Zeilenende

::term::ansi::code::ctrl::esol
Löschen bis zum Zeilenanfang

::term::ansi::code::ctrl::el
(Aktuelle) Zeile löschen

::term::ansi::code::ctrl::ed
Nach unten löschen (bis zum Ende)

::term::ansi::code::ctrl::eu
Nach oben (zurück zum Anfang)

::term::ansi::code::ctrl::es
Bildschirm löschen

::term::ansi::code::ctrl::sd
Bitte scrollen Sie nach unten

::term::ansi::code::ctrl::su

Nach oben scrollen

::term::ansi::code::ctrl::ch
Cursor Startseite

::term::ansi::code::ctrl::sc
Cursor speichern

::term::ansi::code::ctrl::rc
Cursor wiederherstellen (Speicherung rückgängig machen)

::term::ansi::code::ctrl::sca
Cursor + Attribute speichern

::term::ansi::code::ctrl::rca
Cursor und Attribute wiederherstellen

::term::ansi::code::ctrl::st
Tabulator setzen (@ aktuelle Position)

::term::ansi::code::ctrl::ct
Registerkarte löschen (an der aktuellen Position)

::term::ansi::code::ctrl::cat
Alle Registerkarten löschen

::term::ansi::code::ctrl::qdc
Gerätecode abfragen

::term::ansi::code::ctrl::qds
Gerätestatus abfragen

::term::ansi::code::ctrl::qcp
Cursorposition abfragen

::term::ansi::code::ctrl::rd
Gerät zurücksetzen

::term::ansi::code::ctrl::elw
Zeilenumbruch aktivieren

::term::ansi::code::ctrl::dlw
Zeilenumbruch deaktivieren

::term::ansi::code::ctrl::eg
Grafikmodus aufrufen

::term::ansi::code::ctrl::lg
Grafikmodus beenden

::term::ansi::code::ctrl::scs0 ***tag***
Standardzeichensatz festlegen

::term::ansi::code::ctrl::scs1 ***tag***
Legt den alternativen Zeichensatz fest.

Legt fest, welcher Zeichensatz für die Standardschriftart (scs0) oder die alternative Schriftart (scs1) verwendet wird. Dies hat keinen Einfluss darauf, ob die Standard- oder die alternative Schriftart verwendet wird, sondern betrifft lediglich deren Definitionen.

Die zulässigen Tags und ihre Bedeutungen lauten wie folgt:

A
Zeichensatz Vereinigtes Königreich

B
ASCII-Zeichensatz

0
Spezielle Grafiken

1
Standardzeichensatz des alternativen Zeichen-ROM

2
Sondergrafik des alternativen Zeichen-ROM

::term::ansi::code::ctrl::sda *arg*...
Anzeigeattribute festlegen. Die Argumente sind die Codesequenzen für die möglichen Attribute, wie sie vom Paket **term::ansi::code::attr** bereitgestellt werden. Der Einfachheit halber bietet dieses Paket zudem zusätzliche Befehle, mit denen jeweils ein einzelnes spezifisches Attribut festgelegt werden kann.

::term::ansi::code::ctrl::sda_fgblack
Setzt die Textfarbe auf *Black*.

::term::ansi::code::ctrl::sda_fgred
Setzt die Textfarbe auf *Red*.

::term::ansi::code::ctrl::sda_fggreen
Setzt die Textfarbe auf *Green*.

::term::ansi::code::ctrl::sda_fgyellow
Setzt die Textfarbe auf *Yellow*.

::term::ansi::code::ctrl::sda_fgblue
Setzt die Textfarbe auf *Blue*.

::term::ansi::code::ctrl::sda_fgmagenta
Setzt die Textfarbe auf *Magenta*.

::term::ansi::code::ctrl::sda_fgcyan
Setzt die Textfarbe auf *Cyan*.

::term::ansi::code::ctrl::sda_fgwhite
Setzt die Textfarbe auf *White*.

::term::ansi::code::ctrl::sda_fgdefault
Setzt die Standardtextfarbe (*Black*).

::term::ansi::code::ctrl::sda_bgblack
Setzt den Hintergrund auf *Black*.

::term::ansi::code::ctrl::sda_bgred
Setzt den Hintergrund auf *Red*.

::term::ansi::code::ctrl::sda_bggreen
Setzt den Hintergrund auf *Green*.

::term::ansi::code::ctrl::sda_bgyellow
Setzt den Hintergrund auf *Yellow*.

::term::ansi::code::ctrl::sda_bgblue
Setzt den Hintergrund auf *Blue*.

::term::ansi::code::ctrl::sda_bgmagenta
Setzt den Hintergrund auf *Magenta*.

::term::ansi::code::ctrl::sda_bgcyan
Setzt den Hintergrund auf *Cyan*.

::term::ansi::code::ctrl::sda_bgwhite
Setzt den Hintergrund auf *White*.

::term::ansi::code::ctrl::sda_bgdefault
Standardhintergrund festlegen (transparent).

::term::ansi::code::ctrl::sda_bold
Weiter so!

::term::ansi::code::ctrl::sda_dim
Schaltet die abgeschwächte Darstellung ein.

::term::ansi::code::ctrl::sda_italic
Kursivschrift aktivieren.

::term::ansi::code::ctrl::sda_underline
Unterstrich aktivieren.

::term::ansi::code::ctrl::sda_blink
Bitte blinken Sie weiter.

::term::ansi::code::ctrl::sda_revers
Rückwärtsgang einlegen.

::term::ansi::code::ctrl::sda_hidden
Ausgeblendet.

::term::ansi::code::ctrl::sda_strike
Durchstreichen aktiviert.

::term::ansi::code::ctrl::sda_nobold
Fettdruck deaktivieren.

::term::ansi::code::ctrl::sda_noitalic
Kursivschrift beenden.

::term::ansi::code::ctrl::sda_nounderline
Unterstrich aus.

::term::ansi::code::ctrl::sda_noblink
Blinken ausschalten.

::term::ansi::code::ctrl::sda_norevers
Bitte fahren Sie rückwärts weg.

::term::ansi::code::ctrl::sda_nohidden
Ausgeblendet.

::term::ansi::code::ctrl::sda_nostrike
Durchstreichung deaktivieren.

::term::ansi::code::ctrl::sda_reset
Setzt alle Attribute auf ihre Standardwerte zurück.

::term::ansi::send::fcp *row col*
Cursorposition erzwingen (auch bekannt als "Gehe zu").

::term::ansi::code::ctrl::cu ?*n*?
Cursor nach oben. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::code::ctrl::cd ?*n*?

Cursor nach unten. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::code::ctrl::cf ?*n*?
Cursor Weiterleitung. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::code::ctrl::cb ?*n*?
Cursor rückwärts. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::code::ctrl::ss ?*s e*?
Bildschirm scrollen (gesamter Bildschirm oder zwischen den Zeilen "Start" und "Ende", einschließlich).

::term::ansi::code::ctrl::skd *code str*
Schlüsseldefinition festlegen.

::term::ansi::code::ctrl::title *str*
Legt den Titel des Terminals fest.

::term::ansi::code::ctrl::gron
Wechselt zu Zeichen- und Kastengrafik, das heißt zur alternativen Schriftart.

::term::ansi::code::ctrl::groff
Wechselt zu normalen Zeichen, das heißt zur Standardschriftart.

::term::ansi::code::ctrl::tlc
Zeichengrafik, obere linke Ecke.

::term::ansi::code::ctrl::trc
Zeichengrafik, obere rechte Ecke.

::term::ansi::code::ctrl::brc
Zeichengrafik, rechte untere Ecke.

::term::ansi::code::ctrl::blc
Zeichengrafik, linke untere Ecke.

::term::ansi::code::ctrl::ltj
Zeichengrafik, T-Kreuzung links.

::term::ansi::code::ctrl::ttj
Zeichengrafik, obere T-Kreuzung.

::term::ansi::code::ctrl::rtj
Zeichengrafik, rechte T-Kreuzung.

::term::ansi::code::ctrl::btj
Zeichengrafik, T-Kreuzung unten.

::term::ansi::code::ctrl::fwj
Zeichengrafiken, Vierwegekreuzung.

::term::ansi::code::ctrl::hl
Zeichengrafik, horizontale Linie.

::term::ansi::code::ctrl::vl
Zeichengrafik, vertikale Linie.

::term::ansi::code::ctrl::groptim *str*
Zeichengrafiken optimieren. Die oben genannten Generatorbefehle erzeugen viel zu viele überflüssige Befehle zum Wechseln in den Grafikmodus und zurück. Dieser Befehl entfernt alle nicht benötigten Wechsel. Zu diesem Zweck erkennt er auch, welche Zeichen in beiden Modi gleich aussehen, um Zeichenfolgen zu verarbeiten, die außerhalb dieses Pakets erstellt wurden.

::term::ansi::code::ctrl::clear
Bildschirm löschen. Im Wesentlichen eine Abfolge von "CursorHome" und "EraseDown".

::term::ansi::code::ctrl::init
Initialisiert die Standard- und Alternativschriftarten auf ASCII und Box-Grafiken.

::term::ansi::code::ctrl::showat *row col text*
Formatiert den Textblock für die Anzeige an der angegebenen Stelle.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright


```
Copyright (c) 2006-2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ansi_cmacros

Stichwörter: ansi_cmacros, ansi, control, frame, menu, terminal

Name

term::ansi::code::macros – Makro-Sequenzen

Übersicht

package require Tcl 8.5 9
package require textutil::repeat
package require textutil::tabify
package require term::ansi::code::macros ?**0.2**?
::term::ansi::code::macros::names
::term::ansi::code::macros::import ?*ns*? ?*arg...*?
::term::ansi::code::macros::menu *menu*
::term::ansi::code::macros::frame *string*

Beschreibung

Dieses Paket bietet übergeordnete Steuersequenzen für komplexere Formen.

API

Introspektion

::term::ansi::code::macros::names

Dieser Befehl dient der Introspektion. Er gibt als Ergebnis eine Liste zurück, die die Namen aller Attributbefehle enthält.

::term::ansi::code::macros::import ?*ns*? ?*arg*...?
Dieser Befehl importiert einige oder alle Attributbefehle in den Namespace *ns*. Standardmäßig ist dies der Namespace *macros*. Beachten Sie, dass es sich hierbei um einen relativen Namespace handelt, wodurch der importierte Befehl in einem Kind-Namespace des aktuellen Namespaces landet. Standardmäßig werden alle Befehle importiert; dies kann jedoch eingeschränkt werden, indem die Namen der gewünschten Befehle nach dem Namespace-Argument aufgeführt werden.

Sequenzen

::term::ansi::code::macros::menu *menu*
Die Beschreibung eines Menüs wird in einen formatierten rechteckigen Textblock umgewandelt, wobei die Menübefehlszeichen durch fetten roten Text hervorgehoben werden. Das Ergebnis wird als Ergebnis des Befehls zurückgegeben.

Die Beschreibung, *menu*, ist ein Dictionary, das Menübezeichnungen den entsprechenden Befehlszeichen zuordnet.

::term::ansi::code::macros::frame *string*
Der in der Zeichenkette enthaltene Textabschnitt wird nach der Normalisierung interner Tabulatoren am rechten Rand mit Leerzeichen aufgefüllt und anschließend in einen Rahmen aus Box-Grafiken eingefügt. Das Ergebnis wird als Ergebnis des Befehls zurückgegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ansi_code

Stichwörter: ansi_code, control, declare, define, terminal

Name

term::ansi::code – Hilfsmittel für Steuersequenzen

Übersicht

package require Tcl 8.5 9
package require term::ansi::code ?**0.2**?
::term::ansi::code::esc *str*
::term::ansi::code::escb *str*
::term::ansi::code::define *name escape code*
::term::ansi::code::const *name code*

Beschreibung

Dieses Paket enthält Befehle, mit denen sich Steuersequenzen auf einfache Weise definieren lassen.

::term::ansi::code::esc *str*
Dieser Befehl gibt die Argumentzeichenfolge zurück, der das ANSI-Escape-Zeichen "\033" vorangestellt ist.

::term::ansi::code::escb *str*
Dieser Befehl gibt die Argumentzeichenfolge zurück, der eine gängige ANSI-Escape-Sequenz, nämlich "\033[", vorangestellt ist.

::term::ansi::code::define *name escape code*
Dieser Befehl definiert eine Prozedur *name*, die die Steuersequenz *code* zurückgibt, beginnend mit der angegebenen Escape-Sequenz, entweder **esc** oder **escb**.

::term::ansi::code::const *name code*
Dieser Befehl definiert eine Prozedur *name*, die die Steuersequenz *code* zurückgibt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ansi_ctrlu

Stichwörter: ansi_ctrlu, ansi, columns, control, cooked, input mode, lines, raw, rows, terminal

Name

term::ansi::ctrl::unix – Steuerungsvorgänge und Abfragen

Übersicht

package require Tcl 8.5 9
package require term::ansi::ctrl::unix ?**0.1.2**?
::term::ansi::ctrl::unix::import ?*ns*? ?*arg...*?
::term::ansi::ctrl::unix::raw
::term::ansi::ctrl::unix::cooked
::term::ansi::ctrl::unix::columns
::term::ansi::ctrl::unix::rows

Beschreibung

WARNUNG: Dieses Paket ist Unix-spezifisch und hängt von der Verfügbarkeit zweier Unix-Systembefehle zur Terminalsteuerung ab, nämlich **stty** und **tput**, die beide im **$PATH** zu finden sein müssen. Fehlt einer dieser beiden Befehle, schlägt das Laden des Pakets fehl.

Das Paket stellt Befehle bereit, mit denen die Standardeingabe des aktuellen Prozesses zwischen den Eingabemodi *raw* und *cooked* umgeschaltet werden kann und mit denen die Größe von Terminals, d. h. die Anzahl der verfügbaren Spalten und Zeilen, abgefragt werden kann.

API

Introspektion

::term::ansi::ctrl::unix::import ?*ns*? ?*arg*...?

Dieser Befehl importiert einige oder alle Attributbefehle in den Namespace *ns*. Standardmäßig ist dies der Namespace *ctrl*. Beachten Sie, dass es sich hierbei um einen relativen Namensraumnamen handelt, wodurch der importierte Befehl in einen Unternamensraum des aktuellen Namensraums eingefügt wird. Standardmäßig werden alle Befehle importiert; dies kann jedoch eingeschränkt werden, indem die Namen der gewünschten Befehle nach dem Namensraum-Argument aufgeführt werden.

Operationen

::term::ansi::ctrl::unix::raw

Dieser Befehl schaltet die Standardeingabe des aktuellen Prozesses in den *raw*-Eingabemodus um. Das bedeutet, dass fortan alle vom Benutzer eingegebenen Zeichen sofort an die Anwendung gemeldet werden, anstatt im Betriebssystempuffer zu verbleiben, bis die Enter-/Return-Taste gedrückt wird.

::term::ansi::ctrl::unix::cooked

Dieser Befehl schaltet die Standardeingabe des aktuellen Prozesses in den *cooked* -Eingabemodus um. Das bedeutet, dass fortan alle vom Benutzer eingegebenen Zeichen zur Bearbeitung in den Betriebssystem-Puffern gespeichert werden, bis die Enter-/Return-Taste gedrückt wird.

::term::ansi::ctrl::unix::columns

Dieser Befehl fragt das an die Standardeingabe angeschlossene Terminal ab, wie viele Spalten für die Anzeige zur Verfügung stehen.

::term::ansi::ctrl::unix::rows

Dieser Befehl fragt das an die Standardeingabe angeschlossene Terminal nach der Anzahl der Zeilen, die zur Anzeige zur Verfügung stehen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright


```
Copyright (c) 2006-2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ansi_send

Stichwörter: ansi_send, character output, control, terminal

Name

term::ansi::send – Ausgabe von ANSI-Steuersequenzen an Terminals

Übersicht

package require Tcl 8.5 9
package require term::ansi::send ?**0.2**?
::term::ansi::send::import ?*ns*? *...*
::term::ansi::send::eeol
::term::ansi::send::esol
::term::ansi::send::el
::term::ansi::send::ed
::term::ansi::send::eu
::term::ansi::send::es
::term::ansi::send::sd
::term::ansi::send::su
::term::ansi::send::ch
::term::ansi::send::sc
::term::ansi::send::rc
::term::ansi::send::sca
::term::ansi::send::rca
::term::ansi::send::st
::term::ansi::send::ct
::term::ansi::send::cat
::term::ansi::send::qdc
::term::ansi::send::qds
::term::ansi::send::qcp

::term::ansi::send::rd
::term::ansi::send::elw
::term::ansi::send::dlw
::term::ansi::send::eg
::term::ansi::send::lg
::term::ansi::send::scs0 *tag*
::term::ansi::send::scs1 *tag*
::term::ansi::send::sda *arg...*
::term::ansi::send::sda_fgblack
::term::ansi::send::sda_fgred
::term::ansi::send::sda_fggreen
::term::ansi::send::sda_fgyellow
::term::ansi::send::sda_fgblue
::term::ansi::send::sda_fgmagenta
::term::ansi::send::sda_fgcyan
::term::ansi::send::sda_fgwhite
::term::ansi::send::sda_fgdefault
::term::ansi::send::sda_bgblack
::term::ansi::send::sda_bgred
::term::ansi::send::sda_bggreen
::term::ansi::send::sda_bgyellow
::term::ansi::send::sda_bgblue
::term::ansi::send::sda_bgmagenta
::term::ansi::send::sda_bgcyan
::term::ansi::send::sda_bgwhite
::term::ansi::send::sda_bgdefault
::term::ansi::send::sda_bold
::term::ansi::send::sda_dim
::term::ansi::send::sda_italic
::term::ansi::send::sda_underline
::term::ansi::send::sda_blink
::term::ansi::send::sda_revers
::term::ansi::send::sda_hidden
::term::ansi::send::sda_strike
::term::ansi::send::sda_nobold
::term::ansi::send::sda_noitalic
::term::ansi::send::sda_nounderline
::term::ansi::send::sda_noblink
::term::ansi::send::sda_norevers
::term::ansi::send::sda_nohidden
::term::ansi::send::sda_nostrike
::term::ansi::send::sda_reset
::term::ansi::send::fcp *row col*
::term::ansi::send::cu ?*n*?
::term::ansi::send::cd ?*n*?
::term::ansi::send::cf ?*n*?

::term::ansi::send::cb ?*n*?
::term::ansi::send::ss ?*s e*?
::term::ansi::send::skd *code str*
::term::ansi::send::title *str*
::term::ansi::send::gron
::term::ansi::send::groff
::term::ansi::send::tlc
::term::ansi::send::trc
::term::ansi::send::brc
::term::ansi::send::blc
::term::ansi::send::ltj
::term::ansi::send::ttj
::term::ansi::send::rtj
::term::ansi::send::btj
::term::ansi::send::fwj
::term::ansi::send::hl
::term::ansi::send::vl
::term::ansi::send::groptim *str*
::term::ansi::send::clear
::term::ansi::send::init
::term::ansi::send::showat *row col text*

Beschreibung

Dieses Paket stellt Befehle zum Senden von ANSI-Terminalsteuerungssequenzen an ein Terminal bereit. Alle Befehle gibt es in zwei Varianten: eine zum Senden an einen beliebigen Kanal und eine zum Senden an *stdout*.

Die Befehle werden mithilfe der vom Paket **term::ansi::code::ctrl** bereitgestellten Steuersequenzen definiert. Sie verfügen über dieselben Argumente wie die Befehle, auf denen sie basieren, mit Ausnahme der Variante für die Ausgabe an einen beliebigen Kanal. Ihr erstes Argument ist stets ein Kanal-Handle, gefolgt von den ursprünglichen Argumenten. Im Folgenden werden wir nur die Variante auflisten, die an *stdout* sendet.

::term::ansi::send::import ?*ns*?*...*
: Importiert die Befehle dieses Pakets in den Namespace *ns*. Wird kein Namespace festgelegt, wird standardmäßig *send* verwendet. Hinweis: Es handelt sich bei diesem Standardwert um einen relativen Namespace-Namen, d. h., der eigentliche Namespace wird unter dem aktuellen Namespace angelegt.

  Standardmäßig werden alle Befehle importiert; dies kann jedoch auf bestimmte Befehle beschränkt werden, indem diese hinter dem zu importierenden Namespace aufgeführt werden.

::term::ansi::send::eeol
: Löschen bis zum Zeilenende.

::term::ansi::send::esol
: Löschen bis zum Zeilenanfang.

::term::ansi::send::el

(Aktuelle) Zeile löschen.

::term::ansi::send::ed
Nach unten löschen (bis zum Ende).

::term::ansi::send::eu
Nach oben (zum Anfang).

::term::ansi::send::es
Bildschirm löschen.

::term::ansi::send::sd
Bitte scrollen Sie nach unten.

::term::ansi::send::su
Bitte scrollen Sie nach oben.

::term::ansi::send::ch
Cursor auf "Startseite".

::term::ansi::send::sc
Cursor speichern. Hinweis: Es kann jeweils nur eine gespeicherte Position verwaltet werden. Es handelt sich hierbei nicht um einen unbegrenzten Stapel. Ein Speichern vor dem Wiederherstellen überschreibt die gespeicherten Daten.

::term::ansi::send::rc
Cursor wiederherstellen (Speicherung rückgängig machen).

::term::ansi::send::sca
Cursor + Attribute speichern.

::term::ansi::send::rca
Cursor und Attribute wiederherstellen.

::term::ansi::send::st
Tabulator setzen (@ aktuelle Position).

::term::ansi::send::ct
Registerkarte löschen (an der aktuellen Position).

::term::ansi::send::cat
Alle Registerkarten löschen.

::term::ansi::send::qdc
Gerätecode abfragen.

::term::ansi::send::qds
Gerätestatus abfragen.

::term::ansi::send::qcp
Position des Abfragecursors abfragen.

::term::ansi::send::rd
Gerät zurücksetzen.

::term::ansi::send::elw
Zeilenumbruch aktivieren.

::term::ansi::send::dlw
Zeilenumbruch deaktivieren.

::term::ansi::send::eg
Wechselt in den Grafikmodus.

::term::ansi::send::lg
Grafikmodus beenden.

::term::ansi::send::scs0 *tag*
siehe unten ...

::term::ansi::send::scs1 *tag*
Wählt den Zeichensatz.

Legt fest, welcher Zeichensatz für die Standardschriftart (scs0) und die alternative Schriftart (scs1) verwendet wird. Dies hat keinen Einfluss darauf, ob die Standard- oder die alternative Schriftart verwendet wird, sondern betrifft lediglich deren Definitionen.

Die zulässigen Tags und ihre Bedeutungen lauten wie folgt:

A
Zeichensatz Vereinigtes Königreich

B
ASCII-Zeichensatz

0
Spezielle Grafiken

1
Standardzeichensatz des alternativen Zeichen-ROM

2
Sondergrafik des alternativen Zeichen-ROM

::term::ansi::send::sda *arg*...
Anzeigeattribute festlegen. Die Argumente sind die Codesequenzen für die möglichen Attribute, wie sie vom Paket **term::ansi::code::attr** bereitgestellt werden. Der Einfachheit halber bietet dieses Paket zudem zusätzliche Befehle, mit denen jeweils ein einzelnes spezifisches Attribut festgelegt werden kann.

::term::ansi::send::sda_fgblack
Setzt die Textfarbe auf *Black*.

::term::ansi::send::sda_fgred
Setzt die Textfarbe auf *Red*.

::term::ansi::send::sda_fggreen
Setzt die Textfarbe auf *Green*.

::term::ansi::send::sda_fgyellow
Setzt die Textfarbe auf *Yellow*.

::term::ansi::send::sda_fgblue
Setzt die Textfarbe auf *Blue*.

::term::ansi::send::sda_fgmagenta
Setzt die Textfarbe auf *Magenta*.

::term::ansi::send::sda_fgcyan
Setzt die Textfarbe auf *Cyan*.

::term::ansi::send::sda_fgwhite
Setzt die Textfarbe auf *White*.

::term::ansi::send::sda_fgdefault
Setzt die Standardtextfarbe (*Black*).

::term::ansi::send::sda_bgblack
Setzt den Hintergrund auf *Black*.

::term::ansi::send::sda_bgred
Setzt den Hintergrund auf *Red*.

::term::ansi::send::sda_bggreen
Setzt den Hintergrund auf *Green*.

::term::ansi::send::sda_bgyellow
Setzt den Hintergrund auf *Yellow*.

::term::ansi::send::sda_bgblue
Setzt den Hintergrund auf *Blue*.

::term::ansi::send::sda_bgmagenta
Setzt den Hintergrund auf *Magenta*.

::term::ansi::send::sda_bgcyan
Setzt den Hintergrund auf *Cyan*.

::term::ansi::send::sda_bgwhite
Setzt den Hintergrund auf *White*.

::term::ansi::send::sda_bgdefault
Standardhintergrund festlegen (transparent).

::term::ansi::send::sda_bold
Weiter so!

::term::ansi::send::sda_dim
Schaltet die abgeschwächte Darstellung ein.

::term::ansi::send::sda_italic
Kursivschrift aktivieren.

::term::ansi::send::sda_underline
Unterstrich aktivieren.

::term::ansi::send::sda_blink
Bitte blinken Sie weiter.

::term::ansi::send::sda_revers
Rückwärtsgang einlegen.

::term::ansi::send::sda_hidden
Ausgeblendet.

::term::ansi::send::sda_strike
Durchstreichen aktiviert.

::term::ansi::send::sda_nobold
Fettdruck deaktivieren.

::term::ansi::send::sda_noitalic
Kursivschrift beenden.

::term::ansi::send::sda_nounderline
Unterstrich aus.

::term::ansi::send::sda_noblink
Blinken ausschalten.

::term::ansi::send::sda_norevers

Bitte fahren Sie rückwärts weg.

::term::ansi::send::sda_nohidden
Ausgeblendet.

::term::ansi::send::sda_nostrike
Durchstreichung deaktivieren.

::term::ansi::send::sda_reset
Setzt alle Attribute auf ihre Standardwerte zurück.

::term::ansi::send::fcp *row col*
Cursorposition erzwingen (auch bekannt als "Gehe zu").

::term::ansi::send::cu ?*n*?
Cursor nach oben. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::send::cd ?*n*?
Cursor nach unten. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::send::cf ?*n*?
Cursor Weiterleitung. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::send::cb ?*n*?
Cursor rückwärts. *n* ist standardmäßig auf 1 gesetzt.

::term::ansi::send::ss ?*s e*?
Bildschirm scrollen (gesamter Bildschirm oder zwischen den Zeilen "Start" und "Ende", einschließlich).

::term::ansi::send::skd *code str*
Schlüsseldefinition festlegen.

::term::ansi::send::title *str*
Legt den Titel des Terminals fest.

::term::ansi::send::gron
Wechselt zu Zeichen- und Kastengrafik, das heißt zur alternativen Schriftart.

::term::ansi::send::groff
Wechselt zu normalen Zeichen, das heißt zur Standardschriftart.

::term::ansi::send::tlc
Zeichengrafik, obere linke Ecke.

::term::ansi::send::trc
Zeichengrafik, obere rechte Ecke.

::term::ansi::send::brc
Zeichengrafik, rechte untere Ecke.

::term::ansi::send::blc
Zeichengrafik, linke untere Ecke.

::term::ansi::send::ltj
Zeichengrafik, T-Kreuzung links.

::term::ansi::send::ttj
Zeichengrafik, obere T-Kreuzung.

::term::ansi::send::rtj
Zeichengrafik, rechte T-Kreuzung.

::term::ansi::send::btj

Zeichengrafik, T-Kreuzung unten.

::term::ansi::send::fwj
Zeichengrafiken, Vierwegekreuzung.

::term::ansi::send::hl
Zeichengrafik, horizontale Linie.

::term::ansi::send::vl
Zeichengrafik, vertikale Linie.

::term::ansi::send::groptim *str*
Zeichengrafiken optimieren. Die oben genannten Generatorbefehle erzeugen viel zu viele überflüssige Befehle zum Wechseln in den Grafikmodus und zurück. Dieser Befehl entfernt alle nicht benötigten Wechsel. Zu diesem Zweck erkennt er auch, welche Zeichen in beiden Modi gleich aussehen, um Zeichenfolgen zu verarbeiten, die außerhalb dieses Pakets erstellt wurden.

::term::ansi::send::clear
Bildschirm löschen. Im Wesentlichen eine Abfolge von "CursorHome" und "EraseDown".

::term::ansi::send::init
Initialisiert die Standard- und Alternativschriftarten auf ASCII und Box-Grafiken.

::term::ansi::send::showat *row col text*
Zeigt den Textblock an der angegebenen Stelle an.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


imenu

Stichwörter: imenu, control, menu, terminal, text display

Name

term::interact::menu – Terminal-Widget, Menü

Übersicht

package require Tcl 8.5 9
package require term::interact::menu ?**0.2**?

term::interact::menu *object dict ?options...?*
object **interact**
object **done**
object **clear**
object **configure**
object **configure** *option*
object **configure** *option value...*
object **cget** *option*

Beschreibung

Dieses Paket enthält eine Klasse zur Erstellung eines einfachen Menü-Steuerelements.

API für Klassen

Das Paket exportiert einen einzigen Befehl, den Befehl "class", der die Erstellung von Menüinstanzen ermöglicht. Seine API lautet:

term::interact::menu *object dict ?options...?*

Dieser Befehl erstellt ein neues Menüobjekt mit dem Namen *object*, initialisiert es und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück.

Das Argument ist das anzuzeigende Menü, gegebenenfalls gefolgt von Konfigurationsoptionen und deren Werten. Die Optionen werden im Abschnitt Konfiguration erläutert. Das Menü ist ein Dictionary, das Bezeichnungen symbolischen Aktionscodes zuordnet.

Objekt-API

Die durch den Befehl "class" erstellten Objekte stellen die nachfolgend aufgeführten Methoden bereit:

***object* interact**

Zeigt das Menü an der konfigurierten Stelle auf dem Bildschirm an und beginnt mit der Interaktion damit. Dabei wird eine eigene Ereignisschleife zur Verarbeitung eingehender Zeichen eröffnet. Die Methode gibt zurück, wenn die Interaktion abgeschlossen ist. Eine Beschreibung der möglichen Interaktionen finden Sie im Abschnitt Interaktion.

Die Methode gibt am Ende der Interaktion die symbolische Aktion des vom Benutzer ausgewählten Menüelements zurück.

***object* done**

Diese Methode kann von benutzerdefinierten Aktionen verwendet werden, um die Interaktion mit dem Objekt zu beenden.

***object* clear**

Diese Methode kann von benutzerdefinierten Aktionen verwendet werden, um das Menü vom Terminal zu entfernen.

***object* configure**

siehe unten ...

object* configure *option

siehe unten ...

object* configure *option value...

siehe unten ...

object* cget *option
Standardmethoden zum Abrufen und Konfigurieren der Menüoptionen.

Konfiguration

Eine Menüinstanz erkennt die folgenden Optionen:

-in chan
Legt den Kanal fest, aus dem Zeichenfolgen gelesen werden sollen. Der Standardwert lautet **stdin**.

-out chan
Legt den Kanal fest, in den der Menüinhalt geschrieben werden soll. Der Standardwert lautet **stdout**.

-column int
Legt die Spalte des Terminals fest, in der der linke Rand der Menüanzeige erscheinen soll. Der Standardwert ist 0, d. h. die Spalte ganz links.

-line int
Legt die Zeile des Terminals fest, in der der obere Rand der Menüanzeige erscheinen soll. Der Standardwert ist 0, d. h. die oberste Zeile.

-height int
Legt die maximale Anzahl der Textzeilen fest, die auf dem Bildschirm angezeigt werden sollen. Der Standardwert beträgt 25.

-actions dict
Legt ein Dictionary fest, das zusätzliche Aktionen enthält, wobei Zeichenfolgen als Schlüssel verwendet werden. Hinweis: Diese Zeichenfolgen können die im Abschnitt Interaktion beschriebenen fest programmierten Zeichenfolgen nicht überschreiben.

-hilitleft int
siehe unten ...

-hilitright int
Standardmäßig wird der gesamte ausgewählte Menüeintrag in der inversen Darstellung hervorgehoben. Sind diese beiden Optionen jedoch vorhanden, beschränken sie die inverse Darstellung auf den angegebenen Teilbereich des Eintrags.

-framed bool
Standardmäßig wird das Menü so dargestellt, dass von den grafischen Elementen der Zeichenbox nur die Kopf- und Fußzeile verwendet werden. Wenn dieses Kennzeichen gesetzt ist, wird das Menü vollständig von einer Box umschlossen.

Interaktion

Ein Menüobjekt erkennt die unten aufgeführten Steuersequenzen und verhält sich wie beschrieben. Der Benutzer kann über die Konfiguration weitere Steuersequenzen festlegen, auf die reagiert werden soll, kann diese Standardeinstellungen jedoch nicht überschreiben.

Cursor Up
Die Auswahl wird um einen Eintrag nach oben verschoben, es sei denn, der erste Eintrag des Menüs ist bereits ausgewählt.

Cursor Down

Die Auswahl wird um einen Eintrag nach unten verschoben, es sei denn, der letzte Eintrag des Menüs ist bereits ausgewählt.

Enter/Return
Die Interaktion mit dem Objekt wurde beendet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ipager

Stichwörter: ipager, control, pager, terminal, text display

Name

term::interact::pager – Terminal-Widget, Paging

Übersicht

package require Tcl 8.5 9
package require term::interact::pager ?**0.2**?
term::interact::pager *object text* ?*options...*?
object **interact**
object **done**
object **clear**
object **text** *text*
object **configure**
object **configure** *option*
object **configure** *option value...*
object **cget** *option*

Beschreibung

Dieses Paket enthält eine Klasse zur Erstellung einer einfachen Textanzeige mit Seitenumschaltung.

API für Klassen

Das Paket exportiert einen einzigen Befehl, den Befehl "class", der die Erstellung von

Pager-Instanzen ermöglicht. Seine API lautet wie folgt:

term::interact::pager *object text ?options...?*
Dieser Befehl erstellt ein neues Pager-Objekt mit dem Namen *object*, initialisiert es und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück.

Das Argument ist der anzuzeigende Text, dem gegebenenfalls Konfigurationsoptionen und deren Werte folgen. Die Optionen werden im Abschnitt Konfiguration erläutert.

Objekt-API

Die durch den Befehl "class" erstellten Objekte stellen die nachfolgend aufgeführten Methoden bereit:

***object* interact**
Zeigt den Pager an der konfigurierten Stelle auf dem Bildschirm an und beginnt mit der Interaktion. Dadurch wird eine eigene Ereignisschleife für die Verarbeitung eingehender Zeichen eröffnet. Die Methode gibt zurück, wenn die Interaktion abgeschlossen ist. Die möglichen Interaktionen stehen im Abschnitt Interaktion.

***object* done**
Diese Methode kann von benutzerdefinierten Aktionen verwendet werden, um die Interaktion mit dem Objekt zu beenden.

***object* clear**
Diese Methode kann von benutzerdefinierten Aktionen verwendet werden, um den Pager vom Terminal zu entfernen.

object* text *text
Mit dieser Methode lässt sich der vom Pager angezeigte Text ändern. Der Pager setzt die Anzeige zurück, sodass die erste Zeile des Textes oben erscheint.

***object* configure**
siehe unten ...

object* configure *option
siehe unten ...

object* configure *option value...
siehe unten ...

object* cget *option
Standardmethoden zum Abrufen und Konfigurieren der Optionen des Pagers.

Konfiguration

Eine Pager-Instanz erkennt die folgenden Optionen:

-in chan
Legt den Kanal fest, aus dem Zeichenfolgen gelesen werden sollen. Der Standardwert lautet **stdin**.

-out chan
Legt den Kanal fest, in den der Inhalt des Pagers geschrieben werden soll. Der Standardwert ist **stdout**.

-column int
Legt die Spalte des Terminals fest, in der der linke Rand der Pager-Anzeige erscheinen soll.

Der Standardwert ist 0, d. h. die Spalte ganz links.

-line int
Legt die Zeile des Terminals fest, in der der obere Rand der Pager-Anzeige erscheinen soll. Der Standardwert ist 0, d. h. die oberste Zeile.

-height int
Legt die maximale Anzahl der Textzeilen fest, die auf dem Bildschirm angezeigt werden sollen. Der Standardwert beträgt 25.

-actions dict
Legt ein Dictionary fest, das zusätzliche Aktionen enthält, wobei Zeichenfolgen als Schlüssel verwendet werden. Hinweis: Diese Zeichenfolgen können die im Abschnitt Interaktion beschriebenen fest programmierten Zeichenfolgen nicht überschreiben.

Interaktion

Ein Pager-Objekt erkennt die unten aufgeführten Steuersequenzen und verhält sich wie beschrieben. Der Benutzer kann über die Konfiguration weitere Steuersequenzen festlegen, auf die reagiert werden soll, kann diese Standardeinstellungen jedoch nicht überschreiben.

Cursor Up
Der Text wird um eine Zeile nach unten gescrollt, wodurch oben eine weitere Zeile sichtbar wird. Der Pager reagiert nicht, wenn die erste Zeile des Textes bereits angezeigt wird.

Cursor Down
Der Text wird um eine Zeile nach oben gescrollt, wodurch am unteren Rand eine weitere Zeile sichtbar wird. Der Pager reagiert nicht, wenn die letzte Zeile des Textes bereits angezeigt wird.

Page Up
Der Text wird um eine Seite nach unten gescrollt. Der Pager reagiert nicht, wenn die erste Zeile des Textes bereits angezeigt wird.

Page Down
Der Text wird um eine Seite nach oben gescrollt. Der Pager reagiert nicht, wenn die letzte Zeile des Textes bereits angezeigt wird.

Enter/Return
Die Interaktion mit dem Objekt wurde beendet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

receive

Stichwörter: receive, character input, control, get character, listener, receiver, terminal

Name

term::receive – Allgemeine Eingaben von Terminals

Übersicht

package require Tcl 8.5 9
package require term::receive ?**0.2**?
::term::receive::getch ?*chan*?
::term::receive::listen *cmd* ?*chan*?
cmd **process** *string*
cmd **eof**
::term::receive::unlisten ?*chan*?

Beschreibung

Dieses Paket stellt die grundlegendsten Befehle zum Empfangen von Zeichen auf einem Terminal bereit. Im Wesentlichen handelt es sich dabei um praktische Wrapper für die integrierten Befehle **read** und **fileevent**.

::term::receive::getch ?*chan*?

Dieser Befehl liest ein einzelnes Zeichen aus dem Kanal mit dem Handle *chan* und gibt es als Ergebnis des Befehls zurück.

Wird *chan* nicht angegeben, wird standardmäßig **stdin** verwendet.

Es liegt in der Verantwortung des Aufrufenden, sicherzustellen, dass der Kanal einzelne Zeichen bereitstellen kann. Unter Unix kann dies beispielsweise mithilfe des Befehls aus dem Paket **term::ansi::ctrl::unix** erfolgen.

::term::receive::listen *cmd* ?*chan*?

Dieser Befehl richtet einen fileevent-Listener für den Kanal mit dem Handle *chan* ein und ruft das Befehlspräfix *cmd* auf, sobald Zeichen empfangen wurden oder das EOF erreicht wurde.

Wenn *chan* nicht angegeben wird, wird standardmäßig **stdin** verwendet.

Die Signatur des Befehlspräfixes lautet

cmd* process *string

Diese Methode wird aufgerufen, wenn Zeichen empfangen wurden, und *string* enthält diese zur weiteren Verarbeitung.

***cmd* eof**

Diese Methode wird aufgerufen, wenn am Kanal, den wir überwachen, das EOF erreicht wurde. Es handelt sich dabei um den letzten Aufruf, den die Rückruffunktion empfängt.

::term::receive::unlisten ?*chan*?

Dieser Befehl deaktiviert den fileevent-Listener für den Kanal mit dem Handle *chan*.

Wird *chan* nicht angegeben, wird standardmäßig **stdin** verwendet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Terminalsteuerung

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


term

Stichwörter: term, control, terminal

Name

term – Allgemeine Terminalsteuerung

Übersicht

package require Tcl 8.5 9
package require term ?**0.2**?

Beschreibung

Es ist vorgesehen, dass dieses Paket allgemeine Befehle zur Terminalsteuerung auf hoher Ebene bereitstellt, ähnlich wie ncurses oder vergleichbare Pakete. Derzeit ist jedoch noch nichts implementiert. Das heißt, dieses Paket ist leer. Es kann zwar geladen werden, bietet jedoch keine Funktionen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Terminalsteuerung

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


term_bind
Stichwörter: term_bind, character input, control, dispatcher, listener, receiver, terminal

Name
term::receive::bind – Tastaturzuordnung von Terminals

Übersicht
package require Tcl 8.5 9
package require term::receive::bind ?**0.2**?
term::receive::bind *object* ?*map*?
object **map** *str cmd*
object **default** *cmd*
object **listen** ?*chan*?
object **unlisten** ?*chan*?
object **reset**
object **next** *char*
object **process** *str*
object **eof**

Beschreibung
Dieses Paket stellt eine Klasse zur Erstellung einfacher Dispatcher bereit, die Zeichenfolgen zu Aktionen zuordnen. Intern ist jeder Dispatcher im Wesentlichen ein deterministischer endlicher Automat mit Baumstruktur.

API für Klassen
Das Paket exportiert einen einzigen Befehl, den Befehl "class", der die Erstellung von Dispatcher-Instanzen ermöglicht. Seine API lautet wie folgt:

term::receive::bind *object* ?*map*?
Dieser Befehl erstellt ein neues Dispatcher-Objekt mit dem Namen *object*, initialisiert es und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück.

Das Argument ist ein Dictionary, das Zeichenketten, das heißt Zeichenfolgen, den Befehlspräfixen zuordnet, die aufgerufen werden sollen, wenn die Zeichenfolge im Eingabestrom gefunden wird.

Objekt-API
Die durch den Befehl "class" erstellten Objekte stellen die nachfolgend aufgeführten Methoden bereit:

object* map *str cmd

Diese Methode fügt eine zusätzliche Zuordnung von der Zeichenkette *str* zur Aktion *cmd* hinzu. Die Zuordnung wird sofort wirksam, sobald sich der Prozessor in einem Präfix von *str* befindet, oder beim nächsten Reset-Vorgang. Die Aktion ist ein Befehlspräfix und wird mit einem angehängten Argument aufgerufen, nämlich der Zeichenfolge, die den Aufruf auslöst. Sie wird im globalen Namespace ausgeführt.

object* default *cmd
Diese Methode definiert eine Standardaktion *cmd*, die immer dann aufgerufen wird, wenn eine unbekannte Zeichenfolge auftritt. Der Befehlspräfix wird genauso behandelt wie die reguläre Aktion, die über die Methode **map** definiert wurde.

***object* listen ?*chan*?**
Diese Methode richtet einen fileevent-Listener für den Kanal mit dem Handle *chan* ein und ruft das Dispatcher-Objekt auf, sobald Zeichen empfangen wurden oder das EOF erreicht wurde.

Wird *chan* nicht angegeben, wird standardmäßig **stdin** verwendet.

***object* unlisten ?*chan*?**
Diese Methode entfernt den fileevent-Listener für den Kanal mit dem Handle *chan*.

Wenn *chan* nicht festgelegt wird, wird standardmäßig **stdin** verwendet.

***object* reset**
Diese Methode setzt den Zeichenprozessor auf den Anfang des Baums zurück.

object* next *char
Diese Methode bewirkt, dass der Zeichenprozessor das Zeichen *c* verarbeitet. Dies kann lediglich zu einem Wechsel des internen Zustands führen oder eine zugehörige Aktion für eine erkannte Zeichenfolge auslösen.

object* process *str
Diese Methode bewirkt, dass der Zeichenprozessor die Zeichenfolge *str* verarbeitet, dabei den internen Zustand weiterführt und bei Bedarf Aktionen auslöst. Hierbei handelt es sich um eine Rückruffunktion für **listen**.

***object* eof**
Diese Methode bewirkt, dass der Zeichenprozessor ein EOF in der Eingabe verarbeitet. Derzeit hat dies keine Auswirkung. Es handelt sich hierbei um eine Rückruffunktion für **listen**.

Hinweise

Aufgrund der Einfachheit des DFA ist es nicht möglich, eine Zeichenfolge zu erkennen, deren Präfix eine andere bereits erkannte Zeichenfolge ist.

Mit anderen Worten: Die Menge der erkannten Zeichenfolgen muss einen *Präfixcode* bilden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Terminalsteuerung

Copyright

```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


term_send
Stichwörter: term_send, character output, control, terminal

Name
term::send – Allgemeine Ausgabe an Terminals

Übersicht
package require Tcl 8.5 9
package require term::send ?**0.2**?
::term::send::wrch *chan str*
::term::send::wr *str*

Beschreibung
Dieses Paket stellt die grundlegendsten Befehle zum Senden von Zeichen an ein Terminal bereit. Im Wesentlichen handelt es sich dabei um praktische Wrapper für den integrierten Befehl **puts**.

::term::send::wrch *chan str*
Sendet den Text *str* an den durch das Handle *chan* festgelegten Kanal. Im Gegensatz zum integrierten Befehl **puts** beendet dieser Befehl die Zeichenfolge nicht mit einem Zeilenendezeichen. Außerdem erzwingt er ein Leeren der internen Tcl- und Betriebssystem-Puffer, um sicherzustellen, dass die Zeichen sofort verarbeitet werden.

::term::send::wr *str*
Dieser Befehl entspricht dem Befehl **::term::send::wrch**, mit dem Unterschied, dass der Zielkanal fest auf *stdout* festgelegt ist.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *term* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Terminalsteuerung

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


textutil

adjust

Stichwörter: adjust, TeX, adjusting, formatting, hyphenation, indenting, justification, paragraph, string, undenting

Name

textutil::adjust – Prozeduren zum Anpassen, Einrücken und Ausrücken von Absätzen

Übersicht

package require Tcl 8.5 9
package require textutil::adjust ?**0.7.4**?
::textutil::adjust::adjust *string* ?*option value* ...?
::textutil::adjust::readPatterns *filename*
::textutil::adjust::listPredefined
::textutil::adjust::getPredefined *filename*
::textutil::adjust::indent *string prefix* ?*skip*?
::textutil::adjust::undent *string*

Beschreibung

Das Paket **textutil::adjust** enthält Befehle zum Einrücken, Ausrücken und Blocksatz des in einer Zeichenkette enthaltenen Textes.

API

::textutil::adjust::adjust *string ?option value ...?*

Gibt eine Kopie von *string* zurück, die entsprechend den angegebenen Optionen geändert wurde.

Alle Zeilenumbrüche im ursprünglichen *string* werden als Leerzeichen behandelt, d. h., der Eingabe-*string* wird als ein einziger Textabsatz behandelt. Wenn die resultierende Zeichenkette kürzer ist als **-length**, enthält sie keine Zeilenumbrüche. Andernfalls enthält das Ergebnis so viele Zeilenumbrüche, wie für die gewünschte Ausrichtung erforderlich sind.

Zusammen mit **::textutil::adjust::indent** ist es möglich, korrekt umgebrochene Absätze mit beliebigen Einrückungen zu erstellen.

Im Hintergrund werden jeder Zeilenumbruch, jedes Tabulatorzeichen oder jede Folge von zwei oder mehr Leerzeichen als ein einziges Leerzeichen behandelt, und einzelne Leerzeichen werden verwendet, um die Wörter einer Zeile voneinander zu trennen. Dies sind die *tatsächlichen* Zeilen des Algorithmus. Jede *tatsächliche* Zeile wird in eine *logische* Zeile platziert, die genau **-length** Zeichen breit ist (auch wenn die *tatsächliche* Zeile kürzer ist). Jedes Wort in einer *echten* Zeile wird durch ein einzelnes Leerzeichen getrennt, außer wenn die Option **-justify plain** verwendet wird; in diesem Fall werden Wörter durch ein oder mehrere Leerzeichen getrennt, je nach Bedarf, um die *echte* Zeile auszufüllen.

Standardmäßig werden nachgestellte Leerzeichen (einschließlich Tabulatoren, die zunächst durch Leerzeichen ersetzt werden) entfernt. Siehe die Option **-full** weiter unten.

Die folgenden Optionen können nach dem Parameter *string* angegeben werden. Sie steuern, wie der Befehl jede *tatsächliche* Zeile in die entsprechende *logische* Zeile platziert.

-full *boolean*
: Wenn die Einstellung **0** (false; Standard) gewählt ist, werden alle abschließenden Leerzeichen aus jeder Zeile entfernt, bevor die Zeichenkette zurückgegeben wird. Wenn die Einstellung **1** (true) gewählt ist, bleiben alle abschließenden Leerzeichen am Ende jeder Zeile erhalten.

-hyphenate *boolean*
: Bei der Einstellung **0** (falsch; Standard) erfolgt keine Silbentrennung. Bei der Einstellung **1** (wahr) versucht der Befehl, das letzte Wort jeder Zeile gegebenenfalls zu trennen.

  Wenn **-hyphenate** auf **1** gesetzt ist, *müssen* die Silbentrennungsmuster durch Aufruf des Befehls **::textutil::adjust::readPatterns** Der Befehl geladen worden sein, *bevor* **::textutil::adjust::adjust** aufgerufen wird; andernfalls löst **::textutil::adjust::adjust** einen Fehler aus.

-justify center|left|plain|right
: Legt die Ausrichtung der zurückgegebenen Zeichenkette auf eine der folgenden Optionen fest: **left** (Standard), **center**, **plain** (blockiert) oder **right**. Die Ausrichtung bedeutet, dass jede Zeile in der zurückgegebenen Zeichenkette (möglicherweise mit Ausnahme der letzten Zeile) entsprechend der angegebenen Ausrichtung ausgerichtet wird.

  Wenn die Ausrichtung auf **plain** (ausgerichtet) gesetzt ist und die Anzahl der druckbaren Zeichen in der letzten Zeile weniger als 90 % von **-length** beträgt, wird die letzte Zeile **left** ausgerichtet, um den Innenabstand dieser Zeile zu vermeiden, wenn sie zu kurz ist. Die einzelnen Werte haben folgende Bedeutung:

  center
  : Jede *tatsächliche* Zeile wird in der *logischen* Zeile zentriert, indem zwischen der *tatsächlichen* Zeile und der *logischen* Zeile ein Innenabstand mit Leerzeichen eingehalten wird, der bei Bedarf am Anfang und am Ende die *tatsächliche* Zeile auffüllt. Wenn die letzte Zeile weniger als 90 % von **-length** ausmacht, wird sie **left** statt **center** ausgerichtet. (siehe auch die Option **-full**.)

  left
  : Jede *tatsächliche* Zeile wird links von der *logischen* Zeile platziert, ohne vorangestellte Leerzeichen. Wenn **-full** den Wert **1** (wahr) hat, wird die *tatsächliche* Zeile am Ende mit Leerzeichen zum Innenabstand aufgefüllt, sodass sie **-length** Zeichen breit ist.

  plain
  : Jede Zeile ist vollständig ausgerichtet. Das bedeutet, dass die Wörter jeder *tatsächlichen* Zeile in der *logischen* Zeile platziert werden, getrennt durch ein oder mehrere Leerzeichen, je nach Bedarf, um die Zeile vollständig auszufüllen, ohne dass am Anfang oder Ende Leerzeichen stehen.

  right
  : Jede *tatsächliche* Zeile wird rechts neben der *logischen* Zeile platziert, ohne Leerzeichen am Ende, und bei Bedarf mit Innenabstand am Anfang aufgefüllt, damit die *tatsächliche* Zeile eine Breite von **-length** Zeichen hat.

-length *integer*
Legt die *logische* Zeilenlänge auf *integer* fest. *integer* muss eine positive ganze Zahl > *0* sein. Der Standardwert ist **72**.

-strictlength *boolean*
Wenn die Einstellung **0** (falsch; Standard) gewählt ist, darf eine Zeile die angegebene Länge **-length** überschreiten, wenn ein einzelnes Wort länger als **-length** ist. Wenn die Einstellung **1** (wahr) gewählt ist, werden Wörter, die länger als **-length** sind, so geteilt, dass keine Zeile den festgelegten Wert **-length** überschreitet.

::textutil::adjust::readPatterns *filename*
Liest die Silbentrennungsmuster aus der Datei *Dateiname* ein.

Dieser Befehl wird nur benötigt, wenn der Befehl **::textutil::adjust::adjust** zusammen mit **-hyphenate 1** (wahr) verwendet wird. In diesem Fall *muss* der Aufruf von **::textutil::adjust::readPatterns** *vor* dem Aufruf von **::textutil::adjust::adjust** erfolgen.

Das Paket **textutil::adjust** enthält eine Reihe vordefinierter Musterdateien. Verwenden Sie den Befehl **::textutil::adjust::listPredefined**, um deren Namen aufzulisten.

::textutil::adjust::listPredefined
Gibt eine Liste der Namen der von diesem Paket bereitgestellten Silbentrennungsdateien zurück.

::textutil::adjust::getPredefined *filename*
Gibt den vollständigen Pfad und den Dateinamen der Silbentrennungsdatei *filename* zurück. *Hinweis*: *filename muss* unbedingt einer der Namen sein, die von **::textutil::adjust::listPredefined** zurückgegeben werden.

::textutil::adjust::indent *string prefix* ?*skip*?
Gibt eine modifizierte Kopie von *string* zurück, wobei jeder nicht übersprungenen Zeile das Präfix *prefix* sowie Leerzeichen (mit Ausnahme des Zeilenumbruchs) vorangestellt und am Ende abgeschnitten werden. Standardmäßig werden keine Zeilen übersprungen, sodass allen Zeilen das Präfix vorangestellt wird.

Die Option **skip** legt fest, wie viele Zeilen am Anfang der Zeichenkette übersprungen werden sollen; der Standardwert ist **0**, in diesem Fall wird keine Zeile übersprungen und alle Zeilen erhalten das Präfix. Wenn **skip** auf einen Wert > *0* gesetzt wird, werden entsprechend viele Zeilen am Anfang der Zeichenkette übersprungen, d. h. nicht mit einem Präfix versehen, während alle nachfolgenden Zeilen mit einem Präfix versehen werden. Wird **skip** auf **1** gesetzt, entsteht ein herkömmlicher hängender Einzug.

In Verbindung mit **::textutil::adjust::adjust** ist es möglich, korrekt umgebrochene Absätze mit beliebigen Einzügen zu erstellen.

::textutil::adjust::undent *string*
Gibt eine geänderte Kopie von *string* zurück, bei der aus jeder Zeile ein etwaiges gemeinsames Präfix aus Leerzeichen entfernt wurde.

Leere Zeilen und Zeilen, die ausschließlich aus Leerzeichen bestehen, werden bei der Berechnung des Präfixes ignoriert, und alle diese Zeilen werden als leere Zeilen

zurückgegeben.

Beachten Sie, dass der Name dieses Befehls **undent** *lautet und nicht unindent.*

Beispiele

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::adjust** in der Praxis.

Die folgenden **proc** und **const** werden in den nachstehenden Beispielen verwendet.

```
proc replace_spc {s {sub ▲}} { regsub -all { } $s $sub }
const LINES "Four\nsiamese cats\nheard barks\nand bolted."
```

Der Befehl **replace_spc** ersetzt jedes Leerzeichen in der übergebenen Zeichenkette durch das Symbol ▲. Die Konstante **LINES** liefert Text, mit dem die Beispiele arbeiten können.

Begründung

Das folgende Beispiel zeigt alle verfügbaren Ausrichtungsoptionen. Leerzeichen wurden sichtbar gemacht, indem sie mithilfe von **-full 1** beibehalten und durch das Symbol ▲ ersetzt wurden.

Normalerweise würde **-full** *nicht* verwendet werden, und es gäbe *keine* Leerzeichen am Zeilenende.

```
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    left]
puts "======= left =======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    center]
puts "====== center ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    plain]
puts "======= plain ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    right]
puts "======= right ======\n[replace_spc $lines]"
=>
======= left =======
Four▲siamese▲cats▲▲▲
heard▲barks▲and▲▲▲▲▲
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

====== center ======
▲▲Four▲siamese▲cats▲
▲▲▲heard▲barks▲and▲▲
▲▲▲▲▲▲▲bolted.▲▲▲▲▲▲

======= plain ======
Four▲siamese▲▲▲▲cats
heard▲▲▲barks▲▲▲▲and
```

```
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

======= right ======
▲▲▲Four▲siamese▲cats
▲▲▲▲▲heard▲barks▲and
▲▲▲▲▲▲▲▲▲▲▲▲▲bolted.
```

Einrücken und Ausrücken

In den folgenden Beispielen wurden führende Leerzeichen sichtbar gemacht, indem anstelle von beispielsweise zwei Leerzeichen eine Einrückung von ·· verwendet wurde, und andere Leerzeichen wurden sichtbar gemacht, indem sie durch das Symbol ▲ ersetzt wurden. Es gibt keine nachgestellten Leerzeichen, da **textutil::adjust::indent** diese entfernt.

```
set lines [textutil::adjust::adjust $LINES -length 20 -justify left]
puts "======== left =========\n[replace_spc $lines]"
set indented [textutil::adjust::indent $lines "··"]
puts "======== indent =======\n[replace_spc $indented]"
# undent only unindents spaces
set indented [regsub -all · $indented " "]
set unindented [textutil::adjust::undent $indented]
puts "======== undent =======\n[replace_spc $unindented]"
set hanging [textutil::adjust::indent $lines "··" 1]
puts "==== hanging indent ===\n[replace_spc $hanging]"
=>
======== left =========
Four▲siamese▲cats
heard▲barks▲and
bolted.
======== indent =======
··Four▲siamese▲cats
··heard▲barks▲and
··bolted.
======== undent =======
Four▲siamese▲cats
heard▲barks▲and
bolted.
==== hanging indent ===
Four▲siamese▲cats
··heard▲barks▲and
··bolted.
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

expander
Stichwörter: expander, string, template processing, text expansion

Name
textutil::expander – Prozeduren zur Bearbeitung von Vorlagen und zum Erweitern von Text.

Übersicht
package require Tcl 8.5 9
package require textutil::expander ?**1.3.2**?
::textutil::expander *expanderName*
expanderName **cappend** *text*
expanderName **cget** *varname*
expanderName **cis** *cname*
expanderName **cname**
expanderName **cpop** *cname*
expanderName **ctopandclear**
expanderName **cpush** *cname*
expanderName **cset** *varname value*
expanderName **cvar** *varname*
expanderName **errmode** *newErrmode*
expanderName **evalcmd** ?*newEvalCmd*?
expanderName **expand** *string* ?*brackets*?
expanderName **lb** ?*newbracket*?
expanderName **rb** ?*newbracket*?
expanderName **reset**
expanderName **setbrackets** *lbrack rbrack*
expanderName **textcmd** ?*newTextCmd*?
expanderName **where**

Beschreibung
Das Paket **textutil::expander** bietet eine Funktion zur Erstellung von Zeichenfolgenvorlagen, die leistungsfähiger und flexibler ist als der integrierte Befehl subst.

Im Abschnitt Anleitung erfahren Sie mehr über das Paket, und im Abschnitt API finden Sie alle Einzelheiten.

Expander vs. subst
Der Tcl-Befehl **subst** wird häufig zur Unterstützung einer Art von Vorlagenverarbeitung verwendet. Bei einer Zeichenkette mit eingebetteten Variablen oder Funktionsaufrufen fügt **subst** die Werte der Variablen und Funktionen ein und gibt die neue Zeichenkette zurück:

```
% set greeting "Howdy"
Howdy
% proc place {} {return "World"}
% subst {$greeting, [place]!}
Howdy, World!
%
```

Durch die Definition einer geeigneten Reihe von Tcl-Befehlen lässt sich mit **subst** eine Markup-Sprache implementieren, die HTML ähnelt.

Der Befehl **subst** ist zwar effizient, weist jedoch bei dieser Art der Vorlagenverarbeitung drei Nachteile auf:

- Es gibt keine Möglichkeit, den Klartext zwischen zwei eingebetteten Tcl-Befehlen zu identifizieren und zu verarbeiten; das erschwert den kontextsensitiven Umgang mit Klartext.
- Eingebettete Befehle müssen zwingend zwischen **[** und **]** stehen; in besonderen Fällen ist es praktisch, verschiedene Klammern wählen zu können. Wer Seiten erstellt, die eine große Menge an Tcl-Code-Beispielen enthalten, wird möglicherweise lieber **<<** und **>>** als Begrenzer für den eingebetteten Code verwenden.
- Es gibt keine einfache Möglichkeit, inkrementelle Eingaben zu verarbeiten, wie Sie es beispielsweise beim Einlesen von Daten aus einem Socket tun möchten.

Derzeit löst "expander" die ersten beiden Probleme.

Im folgenden Abschnitt wird die API von "expander" beschrieben; darauf folgen die Abschnitte zum Tutorial, siehe Anleitung.

API

Das Paket **textutil::expander** enthält nur einen Befehl, der im Folgenden beschrieben wird. Der Rest dieses Abschnitts befasst sich mit einer Beschreibung der Methoden für die durch diesen Befehl erstellten Expander-Objekte.

::textutil::expander *expanderName*

Der Befehl erstellt ein neues Expander-Objekt mit einem zugehörigen Tcl-Befehl, dessen Name *expanderName* lautet. Mit diesem Befehl können verschiedene Methoden aufgerufen werden. Wenn *expanderName* nicht vollständig qualifiziert ist, wird er als relativ zum aktuellen Namespace interpretiert. Die Methodenaufrufe haben die folgende allgemeine Form:

```
expanderName method ?arg ...?
```

Expander-Objekte unterstützen die folgenden Methoden:

expanderName* cappend *text

Fügt im aktuellen Kontext eine Zeichenkette an die Ausgabe an. Dieser Befehl sollte von Makros oder Anwendungscode nur selten verwendet werden.

expanderName* cget *varname

Gibt den Wert der im aktuellen Kontext definierten Variablen *varname* zurück.

expanderName* cis *cname

Gibt **1** (wahr) zurück, wenn der Name des aktuellen Kontexts *cname* lautet; andernfalls gibt es **0** (falsch) zurück. *cname*.

***expanderName* cname**
Gibt den Namen des aktuellen Kontexts zurück.

expanderName* cpop *cname
Entnimmt einen Kontext aus dem Kontextstapel und gibt die gesamte in diesem Kontext gesammelte Ausgabe zurück. Der Kontext muss *cname* heißen, andernfalls wird ein Fehler ausgelöst.

***expanderName* ctopandclear**
Gibt die aktuell im obersten Kontext erfasste Ausgabe zurück und leert diesen Puffer. Dies entspricht in etwa einer Kombination aus **cpop** und anschließendem **cpush**, mit dem Unterschied, dass hier der interne Zustand (Klammern) beibehalten wird.

expanderName* cpush *cname
Fügt einen Kontext namens *cname* auf den Kontextstapel ein. Der Kontext muss vor Ende der Erweiterung mit **cpop** vom Stapel entfernt werden, andernfalls wird ein Fehler ausgelöst.

expanderName* cset *varname value
Setzt die Variable *varname* im aktuellen Kontext auf den Wert *value*.

expanderName* cvar *varname
Gibt den internen Variablennamen der Kontextvariablen *varname* zurück; dadurch kann die Variable an Befehle wie **lappend** übergeben werden.

expanderName* errmode *newErrmode
Legt den Fehlermodus für die Makroauswertung auf einen der Werte **nothing**, **macro**, **error** oder **fail** fest; der Standardwert ist **fail**. Der Wert bestimmt, wie der Expander reagiert, wenn bei der Auswertung eines Makros ein Fehler festgestellt wird.

fail
Der Fehler wird wie üblich weitergeleitet (d. h., er wird ausgelöst) und kann von der Anwendung abgefangen oder ignoriert werden.

error
Das Makro wird zu einer detaillierten Fehlermeldung erweitert, und die Erweiterung wird fortgesetzt.

macro
Das Makro wird zu sich selbst erweitert; das heißt, es wird unverändert an die Ausgabe weitergegeben.

nothing
Das Makro wird zu einer leeren Zeichenkette erweitert und somit praktisch ignoriert.

***expanderName* evalcmd ?*newEvalCmd*?**
Gibt den aktuellen Befehl für die Auswertung zurück, der standardmäßig **uplevel #0** lautet. Wenn festgelegt, wird *newEvalCmd* für die spätere Verwendung gespeichert und anschließend zurückgegeben; es muss sich um einen Tcl-Befehl handeln, der ein zusätzliches Argument erwartet: das auszuwertende Makro.

***expanderName* expand *string* ?*brackets*?**
Erweitert die Eingabezeichenkette, ersetzt darin enthaltene Makros durch ihre expandierten Werte und gibt die expandierte Zeichenkette zurück.

Hinweis: Diese Methode während ihrer Ausführung einen neuen (leeren) Kontext auf den Kontextstapel schiebt und ihn bei der Rückkehr wieder entfernt.

Wenn *brackets* angegeben wird, muss es sich um eine Liste aus zwei Zeichenketten handeln; die Elemente werden ausschließlich für diese Erweiterung als linke und rechte Klammersequenz für die Makroerweiterung verwendet.

***expanderName* lb ?*newbracket*?**
Gibt den aktuellen Wert der linken Makroauswertungsklammer zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

***expanderName* rb ?*newbracket*?**
Gibt den aktuellen Wert der rechten Klammer der Makroauswertung zurück; dies dient zur Verwendung als oder innerhalb eines Makros, wenn die Klammer in den Ausgabetext aufgenommen werden soll. Wenn *newbracket* festgelegt wird, wird dies zur neuen Klammer und zurückgegeben.

***expanderName* reset**
Setzt alle Expander-Einstellungen auf ihre Ausgangswerte zurück. Es ist mit unerwarteten Ergebnissen zu rechnen, wenn dieser Befehl innerhalb eines Aufrufs von **expand** aufgerufen wird.

expanderName* setbrackets *lbrack rbrack
Legt die linken und rechten Klammern für die Makroauswertung fest. Dieser Befehl ist zur Verwendung als oder innerhalb eines Makros vorgesehen oder dient dazu, die Klammerdefinitionen dauerhaft zu ändern. Standardmäßig lauten die Klammern **[** und **]**, es kann jedoch jede nicht leere Zeichenkette verwendet werden; zum Beispiel **<** und **>** oder **(*** und ***)** oder sogar **Hello,** und **World!**.

***expanderName* textcmd ?*newTextCmd*?**
Gibt den aktuellen Befehl zur Verarbeitung von Klartext zurück, der standardmäßig die leere Zeichenkette ist, was *identity* bedeutet. Wenn angegeben, wird *newTextCmd* für die spätere Verwendung gespeichert und anschließend zurückgegeben; es muss sich um einen Tcl-Befehl handeln, der ein zusätzliches Argument erwartet: den zu verarbeitenden Text. Das Expander-Objekt verwendet diesen Befehl für jeden antreffenden Klartext, wodurch der Benutzer des Objekts die Möglichkeit erhält, den gesamten Klartext auf eine einheitliche Weise zu verarbeiten, bevor er in die Ausgabe geschrieben wird. Das Objekt erwartet, dass der Befehl den verarbeiteten Klartext gibt.

Hinweis: Die Kombination "**textcmd** *plaintext*" wird zur tatsächlichen Auswertung durch den *evalcmd* geleitet. Mit anderen Worten: Der *textcmd* wird als spezielles Makro behandelt, das implizit den gesamten Klartext in der Vorlage umschließt.

***expanderName* where**
Gibt eine Liste mit drei Elementen zurück, die die aktuelle Zeichenposition, Zeile und Spalte enthält, die der Expander bei der Verarbeitung der aktuellen Eingabezeichenfolge erreicht hat.

Anleitung

Grundlagen

Erstellen Sie zunächst ein Expander-Objekt:

```
% package require textutil::expander
1.2
% ::textutil::expander myexp
::myexp
%
```

Das erstellte **::myexp**-Objekt kann verwendet werden, um Textzeichenfolgen zu erweitern, die eingebettete Tcl-Befehle enthalten. Standardmäßig werden eingebettete Befehle durch eckige Klammern begrenzt. Beachten Sie, dass der Expander nicht versucht, Variablen zu interpolieren, da Variablen von eingebetteten Befehlen referenziert werden können:

```
% set greeting "Howdy"
Howdy
% proc place {} {return "World"}
% ::myexp expand {[set greeting], [place]!}
Howdy, World!
%
```

Makros einbetten

Ein Expander-Makro ist einfach ein in eine Textzeichenfolge eingebettetes Tcl-Skript. Expander führt das Skript im globalen Kontext aus und ersetzt es durch die resultierende Zeichenfolge. Zum Beispiel:

```
% set greetings {Howdy Hi "What's up"}
Howdy Hi "What's up"
% ::myexp expand {There are many ways to say "Hello, World!":
[set result {}
foreach greeting $greetings {
append result "$greeting, World!\\n"
}
set result]
And that's just a small sample!}
There are many ways to say "Hello, World!":
Howdy, World!
Hi, World!
What's up, World!

And that's just a small sample!
%
```

Makrobefehle schreiben

In der Regel werden *Makrobefehle* zur Erstellung einer Auszeichnungssprache verwendet. Ein Makrobefehl ist lediglich ein Tcl-Befehl, der eine Ausgabekette gibt. Mit dem Befehl "expand" lässt sich beispielsweise eine generische Dokumentauszeichnungssprache implementieren, die auf HTML oder ein beliebiges anderes Ausgabeformat angepasst werden kann:

```
% proc bold {} {return "<b>"}
% proc /bold {} {return "</b>"}
% ::myexp expand {Some of this text is in [bold]boldface[/bold]}
Some of this text is in <b>boldface</b>
%
```

Die oben genannten Definitionen von **bold** und **/bold** geben HTML zurück, doch solche Befehle können beliebig komplex gestaltet werden; sie könnten beispielsweise je nach gewünschtem Ausgabeformat entscheiden, was zurückgegeben wird.

Expansionsklammern ändern

Standardmäßig werden eingebettete Makros in eckige Klammern, **[** und **]**, gesetzt. Wenn eckige Klammern in der Ausgabe enthalten sein müssen, können in der Eingabe die Befehle **lb** und **rb** verwendet werden. Alternativ oder wenn eckige Klammern aus einem anderen Grund unerwünscht sind, können die Klammern für die Makroauswertung durch ein beliebiges Paar nichtleerer Zeichenfolgen ersetzt werden.

Die Methode **setbrackets** ändert die Klammern für alle nachfolgenden Erweiterungen, die auf das aktuelle Erweiterungsobjekt angewendet werden. Sie können beispielsweise Pseudo-HTML schreiben, indem Sie sie in **<** und **>** ändern:

```
% ::myexp setbrackets < >
% ::myexp expand {<bold>This is boldface</bold>}
<b>This is boldface</b>
```

Alternativ können Sie die Expansionsklammern vorübergehend ändern, indem Sie die gewünschten Klammern an die Methode **expand** übergeben:

```
% ::myexp setbrackets "\\[" "\\]"
% ::myexp expand {<bold>This is boldface</bold>} {< >}
<b>This is boldface</b>
%
```

Individuelle Makroauswertung

Standardmäßig werden Makros mit dem Tcl-Befehl **uplevel #0** ausgewertet, sodass der eingebettete Code im globalen Kontext ausgeführt wird. Die Anwendung kann über die Methode **evalcmd** einen anderen Befehl für die Auswertung bereitstellen; dies ermöglicht es der Anwendung beispielsweise, einen sicheren Interpreter zu verwenden oder sogar etwas anderes als Tcl-Code auszuwerten. Es gibt jedoch eine Einschränkung: Um als gültig erkannt zu werden, muss ein Makro den Wert **1** (wahr) geben, wenn es an den Tcl-Unterbefehl info complete übergeben wird.

Beispielsweise erfolgt bei jedem Makro eine Auswertung, bei der der Makrotext selbst zurückgegeben wird.

```
proc identity {macro} {return $macro}
::myexp evalcmd identity
```

Verwendung des Kontextstapels

Oft ist es wünschenswert, ein Makro-Paar zu definieren, das in irgendeiner Weise auf den

Klartext zwischen den beiden Makros einwirkt. Betrachten wir eine Reihe von Makros zum Hinzufügen von Fußnoten zu einer Seite: Sie hätten etwa Folgendes implementieren können:

```
    Dr. Pangloss, however, thinks that this is the best of all
    possible worlds.[footnote "See Candide, by Voltaire"]
```

Das Makro **footnote** würde dieser Fußnote vermutlich eine Nummer zuweisen und den Text speichern, um ihn später zu formatieren. Diese Lösung ist jedoch unschön, wenn der Fußnotentext lang ist oder zusätzliche Markups enthalten soll. Ziehen Sie stattdessen Folgendes in Betracht:

```
    Dr. Pangloss, however, thinks that this is the best of all
    possible worlds.[footnote]See [bookTitle "Candide"], by
    [authorsName "Voltaire"], for more information.[/footnote]
```

Hier steht der Fußnotentext zwischen den Makros **footnote** und **/footnote**, setzt sich in einer zweiten Zeile fort und enthält selbst mehrere Makros. Das ist sowohl übersichtlicher als auch flexibler; mit den bisher vorgestellten Funktionen gibt es jedoch keine einfache Möglichkeit, dies umzusetzen. Dafür ist der Kontextstapel gedacht.

Jede Makro-Erweiterung findet in einem bestimmten Kontext statt. Hier fügt das Makro **footnote** einen neuen Kontext auf den Kontextstapel. Anschließend wird der gesamte erweiterte Text in diesen neuen Kontext eingefügt. **/footnote** ruft ihn ab, indem es den Kontext vom Stapel nimmt. Hier ist eine grobe Implementierung dieser beiden Makros:

```
    proc footnote {} {
        ::myexp cpush footnote
    }

    proc /footnote {} {
        set footnoteText [::myexp cpop footnote]

        # Save the footnote text, and return an appropriate footnote
        # number and link.
    }
```

Die Methode **cpush** schiebt einen neuen Kontext auf den Stack; das Argument ist der Name des Kontexts. Dies kann eine beliebige Zeichenkette sein, ist jedoch in der Regel der Name des Makros selbst. Anschließend überprüft **cpop**, ob der aktuelle Kontext den erwarteten Name trägt, entfernt ihn vom Stack und gibt den gesammelten Text zurück.

Expand bietet mehrere weitere Werkzeuge im Zusammenhang mit dem Kontextstapel. Angenommen, das erste Makro in einem Kontextpaar nimmt Argumente entgegen oder berechnet Werte, die das zweite Makro in diesem Paar benötigt. Nach dem Aufruf von **cpush** kann das erste Makro eine oder mehrere Kontextvariablen definieren; das zweite Makro kann deren Werte jederzeit vor dem Aufruf von **cpop** abrufen. Nehmen wir beispielsweise an, das Dokument müsse die Fußnotennummer explizit angeben:

```
    proc footnote {footnoteNumber} {
        ::myexp cpush footnote
        ::myexp csave num $footnoteNumber
        # Return an appropriate link
    }

    proc /footnote {} {
        set footnoteNumber [::myexp cget num]
        set footnoteText [::myexp cpop footnote]

        # Save the footnote text and its footnoteNumber for future
        # output.
    }
```

Manchmal kann es wünschenswert sein, Makros zu definieren, die nur innerhalb eines bestimmten Kontextpaares gültig sind; solche Makros sollten mithilfe von **cis** oder **cname** überprüfen, ob sie nur im richtigen Kontext aufgerufen werden.

Geschichte

Das **expander**-Paket wurde von William H. Duquette geschrieben; es handelt sich um eine Neuauflage des zentralen Algorithmus des Makroverarbeitungsprogramms "expand".

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dokumentationswerkzeuge

Copyright

Copyright (c) William H. Duquette https://github.com/wjduquette

patch

Stichwörter: patch, diff -ruN, diff, unified format, fossil, git, unified format diff

Name

textutil::patch – Anwendung von Uni-Diff-Patches auf Verzeichnisbäume

Übersicht

package require Tcl 8.5 9
package require textutil::patch ?**0.2**?

::textutil::patch::apply *basedirectory striplevel patch reportcmd*
{*}reportcmd apply *filename*
{*}reportcmd fail *filename hunk expected seen*
{*}reportcmd fail-already *filename hunk*

Beschreibung

Dieses Paket stellt einen einzigen Befehl bereit, der einen Patch im unified diff format auf die Dateien in einer Verzeichnisstruktur anwendet.

API

::textutil::patch::apply *basedirectory striplevel patch reportcmd*

Wendet den *Patch* (d. h. den Text des eigentlichen Patches, nicht einen Dateinamen) unter Verwendung des angegebenen *striplevel* auf die Dateien im *basedirectory* an und meldet Aktionen durch Aufrufe des Befehlspräfixes *reportcmd*. Gibt eine leere Zeichenkette zurück.

Das Argument *striplevel* entspricht dem Befehl **-p** (oder **--strip**) der Option patch mit dem Wert *striplevel*.

Löst einen Fehler aus, wenn *patch* nicht geparst werden kann oder wenn an den Dateien im *basedirectory* keine Änderungen vorgenommen werden.

Aktionen, die während des Patchvorgangs auftreten, einschließlich der Unmöglichkeit, einen Hunk anzuwenden, werden über das Befehlspräfix *reportcmd* gemeldet. Dateien mit Problemen bleiben unverändert, während Dateien ohne Probleme – unabhängig davon, ob sie vor oder nach den problematischen Dateien verarbeitet werden – gemäß dem *patch* geändert werden.

Jeder Aufruf des Befehlspräfixes kann eine von drei möglichen Formen annehmen:

{*}reportcmd apply *filename*
: Dieser Aufruf erfolgt zu Beginn jeder Patch-Anwendung, wobei der *Dateiname* der zu patchenden Datei übergeben wird. Es wird versucht, alle für den *Dateinamen* gesammelten Hunks anzuwenden.

{*}reportcmd fail *filename hunk expected seen*
: Dieser Aufruf erfolgt jedes Mal, wenn die Anwendung eines "Hunks" fehlschlägt. Dem Befehl werden der gerade verarbeitete *Dateiname*, die *Hunk*-Nummer, der *erwartete* Text sowie der tatsächlich *erkannte* Text übergeben.

{*}reportcmd fail-already *filename hunk*
: Dieser Aufruf erfolgt jedes Mal, wenn die Anwendung eines "Hunks" fehlschlägt, weil die angegebene *Hunk*-Nummer *bereits* auf den *Dateinamen* angewendet wurde.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

repeat
Stichwörter: repeat, blanks, repetition, string

Name
textutil::repeat – Prozeduren zum Wiederholen von Zeichenfolgen.

Übersicht
package require Tcl 8.5 9
package require textutil::repeat ?**0.8**?
::textutil::repeat::strRepeat *text count*
::textutil::repeat::blank *count*

Beschreibung
Das Paket **textutil::repeat** enthält Befehle zum Erzeugen von Zeichenfolgen, indem eine Zeichenfolge mehrfach wiederholt wird.

API
::textutil::repeat::strRepeat *text count*
Gibt eine Zeichenkette zurück, die den *text count*-mal wiederholt enthält.

Die Wiederholungen werden ohne Trennzeichen miteinander verbunden. Ist der Wert von *count* <= *0*, gibt der Befehl eine leere Zeichenkette zurück.

Hinweis: Wenn der Tcl-Kern, in den dieses Paket geladen wird, den Befehl **string repeat** bereitstellt, wird dieser Befehl aus Gründen der maximalen Geschwindigkeit in dessen Begriffen implementiert. Andernfalls wird eine schnelle Tcl-Implementierung verwendet.

::textutil::repeat::blank *count*
Gibt eine Zeichenkette zurück, die *count* Leerzeichen enthält.

Dies dient als Erleichterung für den Aufruf von **::textutil::repeat::strRepeat** mit **" "** als erstem Argument.

Beispiele
Dieses kurze Beispiel veranschaulicht die Befehle des Pakets **::textutil::repeat** in der Praxis.

```
puts [::textutil::repeat::strRepeat = 22]
puts =[::textutil::repeat::blank 20]=
=>
======================
=                     =
```

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

tabify
Stichwörter: tabify, formatting, string, tabstops

Name
textutil::tabify – Prozeduren zum (Ent-)Tabulieren von Zeichenketten

Übersicht
package require Tcl 8.5 9
package require textutil::tabify ?**0.8**?
::textutil::tabify::tabify *string* ?*count*?
::textutil::tabify::tabify2 *string* ?*count*?
::textutil::tabify::untabify *string* ?*count*?
::textutil::tabify::untabify2 *string* ?*count*?

Beschreibung
Das Paket **textutil::tabify** enthält Befehle zum Ersetzen von Tabulatoren durch Leerzeichen oder von Leerzeichen durch Tabulatoren in Zeichenketten.

API
::textutil::tabify::tabify *string* ?*count*?
Gibt eine Kopie von *string* zurück, wobei jede Teilzeichenfolge aus *count* Leerzeichen durch ein horizontales Tabulatorzeichen ersetzt wird. Der Standardwert für *count* ist 8.

::textutil::tabify::tabify2 *string* ?*count*?
Gibt eine Kopie von *string* zurück, in der Teilzeichenfolgen mit *count oder weniger* Leerzeichen durch einen horizontalen Tabulator ersetzt wurden, wobei ein Texteditor-ähnlicher Algorithmus verwendet wird. Der Standardwert für *count* ist 8.

Der von diesem Befehl verwendete Algorithmus behandelt jede Zeile in *string* so, als hätte sie fiktive Tabulatorpositionen alle *count* Spalten. (Wenn also *count* **8** ist, dann bei Spalten **8**, **16**, **24**, ...) Nur Folgen von *zwei oder mehr* Leerzeichen, die unmittelbar vor einem fiktiven Tabulator-Stopp auftreten, werden durch einen Tabulator ersetzt.

::textutil::tabify::untabify *string* ?*count*?
Gibt eine Kopie von *string* zurück, in der jeder horizontale Tabulator durch *count* Leerzeichen ersetzt wurde. Der Standardwert für *count* ist 8.

::textutil::tabify::untabify2 *string* ?*count*?
Gibt eine Kopie von *string* zurück, wobei horizontale Tabulatoren durch *count oder weniger* Leerzeichen ersetzt werden – nach einem Prinzip, das dem eines Texteditors ähnelt. Der

Standardwert für *count* ist 8.

Dieser Befehl ist das Gegenstück zu **::textutil::tabify::tabify2**. Der von diesem Befehl verwendete Algorithmus behandelt jede Zeile in *string* so, als hätte sie fiktive Tabulatorpositionen alle *count* Spalten. (Wenn also *count* **8** ist, werden an den Spalten **8**, **16**, **24**, ...) wird anstelle einer blinden Ersetzung jedes horizontalen Tabulatorzeichens durch *count* Leerzeichen jedes horizontale Tabulatorzeichen an einer fiktiven Tabulatorposition durch so viele Leerzeichen ersetzt, dass die nächste fiktive Tabulatorposition erreicht wird.

Es gibt jedoch eine Asymmetrie: Ein Tabulatorzeichen kann durch ein einzelnes Leerzeichen ersetzt werden, aber nicht umgekehrt.

Beispiele

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::tabify** in der Praxis.

In den Beispielen wird ein kleiner Hilfsbefehl verwendet, um Leerzeichen und Tabulatoren deutlich voneinander abzugrenzen und sichtbar zu machen:

```
proc replace_tab_spc s { regsub -all { } [regsub -all \t $s →] ▲ }
```

Dieses Beispiel veranschaulicht die Verwendung des Befehls **::textutil::tabify::tabify**.

```
const LINES1 "    if \{\$x\} \{\n        puts 1\n    \}"
puts "=== original ===\n[replace_tab_spc $LINES1]\n"
set tabbed [::textutil::tabify::tabify $LINES1]
puts "=== 8spc→tab ===\n[replace_tab_spc $tabbed]\n"
set tabbed [::textutil::tabify::tabify $LINES1 4]
puts "=== 4spc→tab ===\n[replace_tab_spc $tabbed]\n"
=>
=== original ===
▲▲▲▲if▲{$x}▲{
▲▲▲▲▲▲▲▲puts▲1
▲▲▲▲}

=== 8spc→tab ===
▲▲▲▲if▲{$x}▲{
→puts▲1
▲▲▲▲}

=== 4spc→tab ===
→if▲{$x}▲{
→→puts▲1
→}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

textutil
Stichwörter: textutil, TeX, formatting, hyphenation, indenting, paragraph, regular expression, string, trimming

Name
textutil – Prozeduren zur Bearbeitung von Texten und Zeichenketten.

Übersicht
package require Tcl 8.5 9
package require textutil ?**0.10**?
::textutil::adjust::adjust *string* ?*option value* ...?
::textutil::adjust::readPatterns *filename*
::textutil::adjust::listPredefined
::textutil::adjust::getPredefined *filename*
::textutil::adjust::indent *string prefix* ?*skip*?
::textutil::adjust::undent *string*
::textutil::repeat::strRepeat *text count*
::textutil::repeat::blank *count*
::textutil::split::splitn *string* ?*size*?
::textutil::split::splitx *string* ?*regexp*?
::textutil::string::chop *string*
::textutil::string::tail *string*
::textutil::string::cap *string*
::textutil::string::capEachWord *string*
::textutil::string::uncap *string*
::textutil::string::longestCommonPrefixList *list*
::textutil::string::longestCommonPrefix ?*string...*?
::textutil::tabify::tabify *string* ?*count*?
::textutil::tabify::tabify2 *string* ?*count*?
::textutil::tabify::untabify *string* ?*count*?
::textutil::tabify::untabify2 *string* ?*count*?
::textutil::trim::trim *string* ?*regexp*?
::textutil::trim::trimleft *string* ?*regexp*?
::textutil::trim::trimright *string* ?*regexp*?
::textutil::trim::trimPrefix *string prefix*
::textutil::trim::trimEmptyHeading *string*

Beschreibung

Das **textutil**-Paket enthält Befehle zur Bearbeitung von Zeichenketten. Dieses Paket ist ein Bündel, das die Befehle der sechs unten aufgeführten Unterpakete enthält, die sich alle im **textutil**-Namespace befinden.

- **textutil::adjust**
- **textutil::repeat**
- **textutil::split**
- **textutil::string**
- **textutil::tabify**
- **textutil::trim**

Das Bundle ist *veraltet* und wird in einer zukünftigen Tcllib-Version entfernt. Anstatt dieses Pakets zu verwenden, sollten Sie je nach Bedarf bestimmte Unterpakete mit "package require" einbinden.

Der Einfachheit halber werden alle oben aufgeführten Befehle des **textutil**-Namespaces im folgenden Abschnitt API beschrieben, zusätzlich zu ihrer Dokumentation in den Unterpaketen selbst.

Hinweis: Die folgenden Unterpakete gehören ebenfalls zum **textutil**-Namespace, sind jedoch nicht Teil des Bundles; sie müssen daher bei Bedarf explizit über package require angefordert werden.

- **textutil::expander**
- **textutil::patch**
- **textutil::wcswidth**

Die oben aufgeführten Befehle des Namespaces **textutil** werden ausschließlich in den jeweiligen Unterpaketen beschrieben.

API

::textutil::adjust::adjust *string* ?*option value ...*?

Gibt eine Kopie von *string* zurück, die entsprechend den angegebenen Optionen geändert wurde.

Alle Zeilenumbrüche im ursprünglichen *string* werden als Leerzeichen behandelt, d. h., der Eingabe-*string* wird als ein einziger Textabsatz behandelt. Wenn die resultierende Zeichenkette kürzer ist als **-length**, enthält sie keine Zeilenumbrüche. Andernfalls enthält das Ergebnis so viele Zeilenumbrüche, wie für die gewünschte Ausrichtung erforderlich sind.

Zusammen mit **::textutil::adjust::indent** ist es möglich, korrekt umgebrochene Absätze mit beliebigen Einrückungen zu erstellen.

Im Hintergrund werden jeder Zeilenumbruch, jedes Tabulatorzeichen oder jede Folge von zwei oder mehr Leerzeichen als ein einziges Leerzeichen behandelt, und einzelne Leerzeichen werden verwendet, um die Wörter einer Zeile voneinander zu trennen. Dies sind die *tatsächlichen* Zeilen des Algorithmus. Jede *tatsächliche* Zeile wird in eine *logische* Zeile platziert, die genau **-length** Zeichen breit ist (auch wenn die *tatsächliche* Zeile kürzer ist). Jedes Wort in einer *echten* Zeile wird durch ein einzelnes Leerzeichen getrennt, außer wenn die Option **-justify plain** verwendet wird; in diesem Fall werden Wörter durch ein oder mehrere Leerzeichen getrennt, je nach Bedarf, um die *echte* Zeile auszufüllen.

Standardmäßig werden nachgestellte Leerzeichen (einschließlich Tabulatoren, die zunächst durch Leerzeichen ersetzt werden) entfernt. Siehe die Option **-full** weiter unten.

Die folgenden Optionen können nach dem Parameter *string* angegeben werden. Sie steuern, wie der Befehl jede *tatsächliche* Zeile in die entsprechende *logische* Zeile platziert.

-full *boolean*
: Wenn die Einstellung **0** (false; Standard) gewählt ist, werden alle abschließenden Leerzeichen aus jeder Zeile entfernt, bevor die Zeichenkette zurückgegeben wird. Wenn die Einstellung **1** (true) gewählt ist, bleiben alle abschließenden Leerzeichen am Ende jeder Zeile erhalten.

-hyphenate *boolean*
: Bei der Einstellung **0** (falsch; Standard) erfolgt keine Silbentrennung. Bei der Einstellung **1** (wahr) versucht der Befehl, das letzte Wort jeder Zeile gegebenenfalls zu trennen.

  Wenn **-hyphenate** auf **1** gesetzt ist, *müssen* die Silbentrennungsmuster durch Aufruf des Befehls **::textutil::adjust::readPatterns** Der Befehl geladen worden sein, *bevor* **::textutil::adjust::adjust** aufgerufen wird; andernfalls löst **::textutil::adjust::adjust** einen Fehler aus.

-justify center|left|plain|right
: Legt die Ausrichtung der zurückgegebenen Zeichenkette auf eine der folgenden Optionen fest: **left** (Standard), **center**, **plain** (blockiert) oder **right**. Die Ausrichtung bedeutet, dass jede Zeile in der zurückgegebenen Zeichenkette (möglicherweise mit Ausnahme der letzten Zeile) entsprechend der angegebenen Ausrichtung ausgerichtet wird.

  Wenn die Ausrichtung auf **plain** (ausgerichtet) gesetzt ist und die Anzahl der druckbaren Zeichen in der letzten Zeile weniger als 90 % von **-length** beträgt, wird die letzte Zeile **left** ausgerichtet, um den Innenabstand dieser Zeile zu vermeiden, wenn sie zu kurz ist. Die einzelnen Werte haben folgende Bedeutung:

  center
  : Jede *tatsächliche* Zeile wird in der *logischen* Zeile zentriert, indem zwischen der *tatsächlichen* Zeile und der *logischen* Zeile ein Innenabstand mit Leerzeichen eingehalten wird, der bei Bedarf am Anfang und am Ende die *tatsächliche* Zeile auffüllt. Wenn die letzte Zeile weniger als 90 % von **-length** ausmacht, wird sie **left** statt **center** ausgerichtet. (siehe auch die Option **-full**.)

  left
  : Jede *tatsächliche* Zeile wird links von der *logischen* Zeile platziert, ohne vorangestellte Leerzeichen. Wenn **-full** den Wert **1** (wahr) hat, wird die *tatsächliche* Zeile am Ende mit Leerzeichen zum Innenabstand aufgefüllt, sodass sie **-length** Zeichen breit ist.

  plain
  : Jede Zeile ist vollständig ausgerichtet. Das bedeutet, dass die Wörter jeder *tatsächlichen* Zeile in der *logischen* Zeile platziert werden, getrennt durch ein oder mehrere Leerzeichen, je nach Bedarf, um die Zeile vollständig auszufüllen, ohne dass am Anfang oder Ende Leerzeichen stehen.

  right
  : Jede *tatsächliche* Zeile wird rechts neben der *logischen* Zeile platziert, ohne Leerzeichen am Ende, und bei Bedarf mit Innenabstand am Anfang aufgefüllt, damit die *tatsächliche* Zeile eine Breite von **-length** Zeichen hat.

-length *integer*
Legt die *logische* Zeilenlänge auf *integer* fest. *integer* muss eine positive ganze Zahl > *0* sein. Der Standardwert ist **72**.

-strictlength *boolean*
Wenn die Einstellung **0** (falsch; Standard) gewählt ist, darf eine Zeile die angegebene Länge **-length** überschreiten, wenn ein einzelnes Wort länger als **-length** ist. Wenn die Einstellung **1** (wahr) gewählt ist, werden Wörter, die länger als **-length** sind, so geteilt, dass keine Zeile den festgelegten Wert **-length** überschreitet.

::textutil::adjust::readPatterns *filename*
Liest die Silbentrennungsmuster aus der Datei *Dateiname* ein.

Dieser Befehl wird nur benötigt, wenn der Befehl **::textutil::adjust::adjust** zusammen mit **-hyphenate 1** (wahr) verwendet wird. In diesem Fall *muss* der Aufruf von **::textutil::adjust::readPatterns** *vor* dem Aufruf von **::textutil::adjust::adjust** erfolgen.

Das Paket **textutil::adjust** enthält eine Reihe vordefinierter Musterdateien. Verwenden Sie den Befehl **::textutil::adjust::listPredefined**, um deren Namen aufzulisten.

::textutil::adjust::listPredefined
Gibt eine Liste der Namen der von diesem Paket bereitgestellten Silbentrennungsdateien zurück.

::textutil::adjust::getPredefined *filename*
Gibt den vollständigen Pfad und den Dateinamen der Silbentrennungsdatei *filename* zurück. *Hinweis*: *filename muss* unbedingt einer der Namen sein, die von **::textutil::adjust::listPredefined** zurückgegeben werden.

::textutil::adjust::indent *string prefix* ?*skip*?
Gibt eine modifizierte Kopie von *string* zurück, wobei jeder nicht übersprungenen Zeile das Präfix *prefix* sowie Leerzeichen (mit Ausnahme des Zeilenumbruchs) vorangestellt und am Ende abgeschnitten werden. Standardmäßig werden keine Zeilen übersprungen, sodass allen Zeilen das Präfix vorangestellt wird.

Die Option **skip** legt fest, wie viele Zeilen am Anfang der Zeichenkette übersprungen werden sollen; der Standardwert ist **0**, in diesem Fall wird keine Zeile übersprungen und alle Zeilen erhalten das Präfix. Wenn **skip** auf einen Wert > *0* gesetzt wird, werden entsprechend viele Zeilen am Anfang der Zeichenkette übersprungen, d. h. nicht mit einem Präfix versehen, während alle nachfolgenden Zeilen mit einem Präfix versehen werden. Wird **skip** auf **1** gesetzt, entsteht ein herkömmlicher hängender Einzug.

In Verbindung mit **::textutil::adjust::adjust** ist es möglich, korrekt umgebrochene Absätze mit beliebigen Einzügen zu erstellen.

::textutil::adjust::undent *string*
Gibt eine geänderte Kopie von *string* zurück, bei der aus jeder Zeile ein etwaiges gemeinsames Präfix aus Leerzeichen entfernt wurde.

Leere Zeilen und Zeilen, die ausschließlich aus Leerzeichen bestehen, werden bei der Berechnung des Präfixes ignoriert, und alle diese Zeilen werden als leere Zeilen

zurückgegeben.

Beachten Sie, dass der Name dieses Befehls **undent** *lautet und nicht unindent.*

::textutil::repeat::strRepeat *text count*
Gibt eine Zeichenkette zurück, die den *text count*-mal wiederholt enthält.

Die Wiederholungen werden ohne Trennzeichen miteinander verbunden. Ist der Wert von *count* <= *0*, gibt der Befehl eine leere Zeichenkette zurück.

Hinweis: Wenn der Tcl-Kern, in den dieses Paket geladen wird, den Befehl **string repeat** bereitstellt, wird dieser Befehl aus Gründen der maximalen Geschwindigkeit in dessen Begriffen implementiert. Andernfalls wird eine schnelle Tcl-Implementierung verwendet.

::textutil::repeat::blank *count*
Gibt eine Zeichenkette zurück, die *count* Leerzeichen enthält.

Dies dient als Erleichterung für den Aufruf von **::textutil::repeat::strRepeat** mit **" "** als erstem Argument.

::textutil::split::splitn *string* ?*size*?
Gibt eine Liste von Zeichenketten zurück, die aufeinanderfolgende Teilzeichenfolgen von *string* enthalten, die jeweils *size* Zeichen lang sind – mit Ausnahme der letzten, die kürzer ist, wenn die Länge von *string* kein genaues Vielfaches von *size* ist. Der Standardwert für *size* ist **1**, d. h., es wird eine Liste der Zeichen von *string* zurückgegeben; dies entspricht dem Aufruf des integrierten Befehls split mit dem zweiten Argument **""**.

Wenn *string* leer ist, gibt es eine leere Liste zurück. Wenn *size* <= *0*, wird ein Fehler ausgelöst.

::textutil::split::splitx *string* ?*regexp*?
Gibt eine Liste von Zeichenketten zurück, die aufeinanderfolgende Teilzeichenfolgen von *string* enthalten, die gemäß dem regulären Ausdruck *regexp* aufgeteilt wurden, dessen Standardwert *[\t \r\n]+* ist (Tabulatoren, Leerzeichen, Zeilenumbrüche).

Hinweis: Wenn *regexp* erfassende Klammern enthält, werden die erfassten Gruppen als zusätzliche Elemente in die Ergebnisliste aufgenommen.

Ist *string* leer, gibt es eine leere Liste zurück. Wenn *regexp* leer ist, wird *string* an jedem Zeichen aufgeteilt; dies entspricht dem Aufruf des integrierten Befehls split mit dem zweiten Argument **""**.

::textutil::string::chop *string*
Gibt eine Kopie von *string* zurück, bei der das letzte Zeichen entfernt wurde.

::textutil::string::tail *string*
Gibt eine Kopie von *string* zurück, bei der das erste Zeichen entfernt wurde.

::textutil::string::cap *string*
Gibt eine Kopie von *string* zurück, bei der das erste Zeichen großgeschrieben ist.

::textutil::string::capEachWord *string*

Gibt eine Kopie von *string* zurück, bei der das erste Zeichen jedes Wortes großgeschrieben ist.

::textutil::string::uncap *string*

Gibt eine Kopie von *string* zurück, wobei das erste Zeichen in Kleinbuchstaben umgewandelt wird.

::textutil::string::longestCommonPrefixList *list*

Gibt das längste gemeinsame Präfix der Zeichenketten in der angegebenen *Liste* zurück.

Wird kein Argument angegeben, gibt es die leere Zeichenkette zurück.

::textutil::string::longestCommonPrefix ?*string*...?

Gibt das längste gemeinsame Präfix der Zeichenfolgenargumente *string*, ... zurück.

Wird eine einzelne Zeichenfolge angegeben, wird diese zurückgegeben, da sie ihr eigenes längstes gemeinsames Präfix ist. Wird kein Argument angegeben, wird die leere Zeichenfolge zurückgegeben.

::textutil::tabify::tabify *string* ?*count*?

Gibt eine Kopie von *string* zurück, wobei jede Teilzeichenfolge aus *count* Leerzeichen durch ein horizontales Tabulatorzeichen ersetzt wird. Der Standardwert für *count* ist 8.

::textutil::tabify::tabify2 *string* ?*count*?

Gibt eine Kopie von *string* zurück, in der Teilzeichenfolgen mit *count oder weniger* Leerzeichen durch einen horizontalen Tabulator ersetzt wurden, wobei ein Texteditor-ähnlicher Algorithmus verwendet wird. Der Standardwert für *count* ist 8.

Der von diesem Befehl verwendete Algorithmus behandelt jede Zeile in *string* so, als hätte sie fiktive Tabulatorpositionen alle *count* Spalten. (Wenn also *count* **8** ist, dann bei Spalten **8**, **16**, **24**, ...) Nur Folgen von *zwei oder mehr* Leerzeichen, die unmittelbar vor einem fiktiven Tabulator-Stopp auftreten, werden durch einen Tabulator ersetzt.

::textutil::tabify::untabify *string* ?*count*?

Gibt eine Kopie von *string* zurück, in der jeder horizontale Tabulator durch *count* Leerzeichen ersetzt wurde. Der Standardwert für *count* ist 8.

::textutil::tabify::untabify2 *string* ?*count*?

Gibt eine Kopie von *string* zurück, wobei horizontale Tabulatoren durch *count oder weniger* Leerzeichen ersetzt werden – nach einem Prinzip, das dem eines Texteditors ähnelt. Der Standardwert für *count* ist 8.

Dieser Befehl ist das Gegenstück zu **::textutil::tabify::tabify2**. Der von diesem Befehl verwendete Algorithmus behandelt jede Zeile in *string* so, als hätte sie fiktive Tabulatorpositionen alle *count* Spalten. (Wenn also *count* **8** ist, werden an den Spalten **8**, **16**, **24**, ...) wird anstelle einer blinden Ersetzung jedes horizontalen Tabulatorzeichens durch *count* Leerzeichen jedes horizontale Tabulatorzeichen an einer fiktiven Tabulatorposition durch so viele Leerzeichen ersetzt, dass die nächste fiktive Tabulatorposition erreicht wird.

Es gibt jedoch eine Asymmetrie: Ein Tabulatorzeichen kann durch ein einzelnes Leerzeichen ersetzt werden, aber nicht umgekehrt.

::textutil::trim::trim *string* ?*regexp*?

Gibt eine Kopie von *string* zurück, wobei alle am Anfang oder Ende stehenden

Teilzeichenfolgen, die mit *regexp* übereinstimmen, aus *string* entfernt werden, sofern dieser keine Zeilenumbrüche enthält, bzw. aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält. Der Standardwert für *regexp* ist *[\t]+*. Dieser Standardwert entfernt führende und abschließende Leerzeichen und Tabulatoren aus *string*, wenn dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, wenn dieser Zeilenumbrüche enthält.

::textutil::trim::trimleft *string* ?*regexp*?
Gibt eine Kopie von *string* zurück, wobei alle führenden Teilzeichenfolgen, die mit *regexp* übereinstimmen, aus *string* entfernt werden, sofern dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält. Der Standardwert für *regexp* ist *[\t]+*. Dieser Standardwert entfernt führende Leerzeichen und Tabulatoren aus *string*, wenn dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, wenn dieser Zeilenumbrüche enthält.

::textutil::trim::trimright *string* ?*regexp*?
Gibt eine Kopie von *string* zurück, wobei alle am Ende stehenden Teilzeichenfolgen, die mit *regexp* übereinstimmen, aus *string* entfernt werden, sofern dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält. Der Standardwert für *regexp* ist *[\t]+*. Dieser Standardwert entfernt abschließende Leerzeichen und Tabulatoren aus *string*, sofern dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält.

::textutil::trim::trimPrefix *string prefix*
Gibt eine Kopie von *string* zurück, wobei *prefix* am Anfang entfernt wird, sofern es vorhanden ist; andernfalls wird *string* unverändert zurückgegeben.

Vergleichen Sie diesen Befehl mit den Unterbefehlen **trim**, **trimleft** und **trimright** des integrierten Befehls string, die alle Zeichen aus einer als Zeichenkette übergebenen Zeichenmenge entfernen.

::textutil::trim::trimEmptyHeading *string*
Gibt eine Kopie von *string* zurück, aus der alle führenden Leerzeilen (einschließlich Zeilen, die ausschließlich Leerzeichen enthalten) entfernt wurden.

Beispiele

Anpassen

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::adjust** in der Praxis.

Die folgenden **proc** und **const** werden in den nachstehenden Beispielen verwendet.

```
proc replace_spc {s {sub ▲}} { regsub -all { } $s $sub }
const LINES "Four\nsiamese cats\nheard barks\nand bolted."
```

Der Befehl **replace_spc** ersetzt jedes Leerzeichen in der übergebenen Zeichenkette durch das Symbol ▲. Die Konstante **LINES** liefert Text, mit dem die Beispiele arbeiten können.

Begründung

Das folgende Beispiel zeigt alle verfügbaren Ausrichtungsoptionen. Leerzeichen wurden sichtbar gemacht, indem sie mithilfe von **-full 1** beibehalten und durch das Symbol ▲ ersetzt wurden.

Normalerweise würde **-full** *nicht* verwendet werden, und es gäbe *keine* Leerzeichen am Zeilenende.

```
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    left]
puts "======= left =======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    center]
puts "====== center ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    plain]
puts "======= plain ======\n[replace_spc $lines]"
set lines [textutil::adjust::adjust $LINES -length 20 -full 1 -justify
    right]
puts "======= right ======\n[replace_spc $lines]"
=>
======= left =======
Four▲siamese▲cats▲▲▲
heard▲barks▲and▲▲▲▲▲
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

====== center ======
▲▲Four▲siamese▲cats▲
▲▲▲heard▲barks▲and▲▲
▲▲▲▲▲▲▲bolted.▲▲▲▲▲▲

======= plain ======
Four▲siamese▲▲▲▲cats
heard▲▲▲barks▲▲▲▲and
bolted.▲▲▲▲▲▲▲▲▲▲▲▲▲

======= right ======
▲▲▲Four▲siamese▲cats
▲▲▲▲▲heard▲barks▲and
▲▲▲▲▲▲▲▲▲▲▲▲▲bolted.
```

Einrücken und Ausrücken

In den folgenden Beispielen wurden führende Leerzeichen sichtbar gemacht, indem anstelle von beispielsweise zwei Leerzeichen eine Einrückung von ·· verwendet wurde, und andere Leerzeichen wurden sichtbar gemacht, indem sie durch das Symbol ▲ ersetzt wurden. Es gibt keine nachgestellten Leerzeichen, da **textutil::adjust::indent** diese entfernt.

```
set lines [textutil::adjust::adjust $LINES -length 20 -justify left]
puts "======== left =========\n[replace_spc $lines]"
set indented [textutil::adjust::indent $lines "··"]
puts "======== indent =======\n[replace_spc $indented]"
# undent only unindents spaces
```

```
set indented [regsub -all · $indented " "]
set unindented [textutil::adjust::undent $indented]
puts "======== undent =======\n[replace_spc $unindented]"
set hanging [textutil::adjust::indent $lines "··" 1]
puts "==== hanging indent ===\n[replace_spc $hanging]"
=>
======== left =========
Four▴siamese▴cats
heard▴barks▴and
bolted.
======== indent =======
··Four▴siamese▴cats
··heard▴barks▴and
··bolted.
======== undent =======
Four▴siamese▴cats
heard▴barks▴and
bolted.
==== hanging indent ===
Four▴siamese▴cats
··heard▴barks▴and
··bolted.
```

Wiederholen

Dieses kurze Beispiel veranschaulicht die Befehle des Pakets **::textutil::repeat** in der Praxis.

```
puts [::textutil::repeat::strRepeat = 22]
puts =[::textutil::repeat::blank 20]=
=>
======================
=                     =
```

Split

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::split** in der Praxis.

Um die Auswirkungen der Beispiele besser sichtbar zu machen, wird jedes Element der zurückgegebenen Listen mit **⟨** und **⟩** in Klammern gesetzt. Bei Editoren, die UTF-8 nicht unterstützen, können diese durch **\u276C** und **\u276D** ersetzt werden.

Dieses erste Beispiel zeigt, wie Sie Zeichenfolgen nach ihrer Länge aufteilt.

```
const LINE1 "α and 𝕫"
set chars [::textutil::split::splitn $LINE1] ;# splits into characters
puts ⟨[join $chars "⟩ ⟨"]⟩
set chars [split $LINE1 ""] ;# splits into characters
puts ⟨[join $chars "⟩ ⟨"]⟩
set chunks [::textutil::split::splitn $LINE1 3]
```

```
puts ⟨[join $chunks "⟩ ⟨"]⟩
=>
⟨α⟩ ⟨ ⟩ ⟨a⟩ ⟨n⟩ ⟨d⟩ ⟨ ⟩ ⟨ℤ⟩
⟨α⟩ ⟨ ⟩ ⟨a⟩ ⟨n⟩ ⟨d⟩ ⟨ ⟩ ⟨ℤ⟩
⟨α a⟩ ⟨nd ⟩ ⟨ℤ⟩
```

Dieses zweite Beispiel zeigt, wie Sie Zeichenfolgen mithilfe regulärer Ausdrücke aufteilt.

```
const LINE2 "a, be , cat, done , eagle"
set chunks [::textutil::split::splitx $LINE2] ;# splits on [\t \r\n]+
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*,\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*(,)\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
=>
⟨a,⟩ ⟨be⟩ ⟨,⟩ ⟨cat,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
⟨a⟩ ⟨be⟩ ⟨cat⟩ ⟨done⟩ ⟨eagle⟩
⟨a⟩ ⟨,⟩ ⟨be⟩ ⟨,⟩ ⟨cat⟩ ⟨,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
```

Zeichenkette

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::string** in der Praxis.

Dieses erste Beispiel zeigt alle Befehle, die auf eine einzelne Zeichenkette angewendet werden können.

```
const LINE "has the σαβ cogs"
puts ""$LINE" const"
set line [::textutil::string::chop $LINE]
puts ""$line" chop"
set line [::textutil::string::tail $line]
puts ""$line" tail"
set line [::textutil::string::cap $line]
puts ""$line" cap"
set line [::textutil::string::capEachWord $line]
puts ""$line" capEachWord"
set line [::textutil::string::uncap $line]
puts ""$line" uncap"
=>
"has the σαβ cogs" const
"has the σαβ cog" chop
"as the σαβ cog" tail
"As the σαβ cog" cap
"As The Σαβ Cog" capEachWord
"as The Σαβ Cog" uncap
```

Dieses Beispiel zeigt die Befehle für das längste gemeinsame Präfix, die derzeit verwendet werden.

```
const LIST [list handbag handcuff handful handle handy]
puts "[::textutil::string::longestCommonPrefixList $LIST]"
puts "[::textutil::string::longestCommonPrefix king queen knave]"
puts "[::textutil::string::longestCommonPrefix king kin kind kinks]"
=>
"hand"
""
"kin"
```

Dieses Beispiel zeigt, wie Sie mit dem Befehl **::textutil::string::longestCommonPrefixList** den gemeinsamen Pfad aus einer Liste von Pfaden ermittelt.

```
proc longestCommonPath paths {
    const SEP [file separator]
    set path [::textutil::string::longestCommonPrefixList $paths]
    if {[string index $path end] ne $SEP} {
        if {[set j [string last $SEP $path]] > -1} {
            set path [string range $path 0 $j-1]
        }
    }
    if {$path ne $SEP} { set path [string trimright $path $SEP] }
    return $path
}

const PATHS [list /home/sally/music /home/sally/museums
    /home/sally/musings]
puts [::textutil::string::longestCommonPrefixList $PATHS]
puts [longestCommonPath $PATHS]
puts [longestCommonPath [list /bin / /sbin]]
=>
/home/sally/mus
/home/sally
/
```

Tabify

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::tabify** in der Praxis.

In den Beispielen wird ein kleiner Hilfsbefehl verwendet, um Leerzeichen und Tabulatoren deutlich voneinander abzugrenzen und sichtbar zu machen:

```
proc replace_tab_spc s { regsub -all { } [regsub -all \t $s →] ▲ }
```

Dieses Beispiel veranschaulicht die Verwendung des Befehls **::textutil::tabify::tabify**.

```
const LINES1 "    if \{\$x\} \{\n        puts 1\n    \}"
puts "=== original ===\n[replace_tab_spc $LINES1]\n"
set tabbed [::textutil::tabify::tabify $LINES1]
puts "=== 8spc→tab ===\n[replace_tab_spc $tabbed]\n"
```

```
set tabbed [::textutil::tabify::tabify $LINES1 4]
puts "=== 4spc→tab ===\n[replace_tab_spc $tabbed]\n"
=>
=== original ===
▲▲▲▲if▲{$x}▲{
▲▲▲▲▲▲▲▲puts▲1
▲▲▲▲}

=== 8spc→tab ===
▲▲▲▲if▲{$x}▲{
→puts▲1
▲▲▲▲}

=== 4spc→tab ===
→if▲{$x}▲{
→→puts▲1
→}
```

Trim

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::trim** in der Praxis.

Die folgende **proc** wird in einigen der nachfolgenden Beispiele verwendet.

```
proc replace_ws s { regsub -all { } [regsub -all \t [regsub -all \n $s
    ⏎] →] ▲ }
```

Diese Beispiele veranschaulichen die Verwendung von **::textutil::trim::trimEmptyHeading** und **::textutil::trim::trim**.

```
const LINES1 "\t\n Alpha\t\n Beta \t\n Gamma \t\n"
puts "== const ============================\n[replace_ws $LINES1]"
set line [::textutil::trim::trimEmptyHeading $LINES1]
puts "== textutil::trim::trimEmptyHeading ==\n[replace_ws $line]"
set line [string trimleft $LINES1]
puts "== string trimleft ==================\n[replace_ws $line]"
set line [::textutil::trim::trim $LINES1]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
→⏎▲Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== textutil::trim::trimEmptyHeading ==
▲Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== string trimleft ==================
Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== textutil::trim::trim ==============
⏎Alpha⏎Beta⏎Gamma⏎
```

Beachten Sie den subtilen Unterschied im Verhalten zwischen

::textutil::trim::trimEmptyHeading und **string trimleft**.

Dieses Beispiel veranschaulicht die Verwendung der integrierten Funktion **string trim** in Kombination mit **::textutil::trim::trim**.

```
const LINES2 [string trim "\t\n Delta\t\n Epsilon \t\n Zeta \t\n"]
puts "== const ============================\n[replace_ws $LINES2]"
set line [::textutil::trim::trim $LINES2]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
Delta→⏎▲Epsilon▲→⏎▲Zeta
== textutil::trim::trim ==============
Delta⏎Epsilon⏎Zeta
```

In diesen Beispielen wird die integrierte Funktion **string trim** mit **::textutil::trim::trimPrefix** verglichen.

```
const PATH /home/homer
puts "const='$PATH'"
puts "string trim \$PATH /home='[string trim $PATH /home]'"
puts "string trim \$PATH /home='[string trim $PATH ehmo/]'"
puts "::textutil::trim::trimPrefix \$PATH
    /home='[::textutil::trim::trimPrefix $PATH /home]'"
const LINE mimic
puts "string trim \$LINE mic='[string trim $LINE mic]'"
puts "::textutil::trim::trimPrefix \$LINE
    mic='[::textutil::trim::trimPrefix $LINE mic]'"
=>
const='/home/homer'
string trim $PATH /home='r'
string trim $PATH /home='r'
::textutil::trim::trimPrefix $PATH /home='/homer'
string trim $LINE mic=''
::textutil::trim::trimPrefix $LINE mic='mimic'
```

Bei den **PATH**-Beispielen entfernt der Befehl **string trim** alle einzelnen Zeichen, die in der Zeichenkette "/home" enthalten sind, d. h. "/", "h", "o", "m", "e". Die Reihenfolge der Zeichen in der Zeichenkette spielt keine Rolle, wie das Beispiel "ehmo/" zeigt. Vergleichen Sie dies mit **::textutil::trim::trimPrefix**, das die Zeichenkette "/home" wörtlich trimmt.

Bei den **LINE**-Beispielen entfernt der Befehl **string trim** alle einzelnen Zeichen, die in der Zeichenkette "mic" enthalten sind, d. h. "m", "i" und "c". Vergleichen Sie dies mit **::textutil::trim::trimPrefix**, das die Literalzeichenfolge "mic" trimmt; da die **LINE** jedoch nicht mit "mic" beginnt, wird nichts getrimmt und die Zeichenfolge wird unverändert gegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

textutil_split

Stichwörter: textutil_split, regular expression, split, string

Name

textutil::split – Prozeduren zum Aufteilen von Texten

Übersicht

package require Tcl 8.5 9
package require textutil::split ?**0.9**?
::textutil::split::splitn *string* ?*size*?
::textutil::split::splitx *string* ?*regexp*?

Beschreibung

Das Paket **textutil::split** enthält Befehle, mit denen Zeichenfolgen nach Länge oder anhand regulärer Ausdrücke in Teilzeichenfolgen aufgeteilt werden können.

Um an einem bestimmten Zeichen oder einer Zeichenfolge zu trennen, verwenden Sie den integrierten Befehl split. Um eine Zeichenfolge in ihre einzelnen Zeichen zu zerlegen, verwenden Sie den integrierten Befehl split mit dem zweiten Argument **""**.

API

::textutil::split::splitn *string* ?*size*?

Gibt eine Liste von Zeichenketten zurück, die aufeinanderfolgende Teilzeichenfolgen von *string* enthalten, die jeweils *size* Zeichen lang sind – mit Ausnahme der letzten, die kürzer ist, wenn die Länge von *string* kein genaues Vielfaches von *size* ist. Der Standardwert für *size* ist **1**, d. h., es wird eine Liste der Zeichen von *string* zurückgegeben; dies entspricht dem Aufruf des integrierten Befehls split mit dem zweiten Argument **""**.

Wenn *string* leer ist, gibt es eine leere Liste zurück. Wenn *size* <= *0*, wird ein Fehler ausgelöst.

::textutil::split::splitx *string* ?*regexp*?

Gibt eine Liste von Zeichenketten zurück, die aufeinanderfolgende Teilzeichenfolgen von *string* enthalten, die gemäß dem regulären Ausdruck *regexp* aufgeteilt wurden, dessen Standardwert *[\t \r\n]+* ist (Tabulatoren, Leerzeichen, Zeilenumbrüche).

Hinweis: Wenn *regexp* erfassende Klammern enthält, werden die erfassten Gruppen als zusätzliche Elemente in die Ergebnisliste aufgenommen.

Ist *string* leer, gibt es eine leere Liste zurück. Wenn *regexp* leer ist, wird *string* an jedem

Zeichen aufgeteilt; dies entspricht dem Aufruf des integrierten Befehls split mit dem zweiten Argument **""**.

Beispiele

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::split** in der Praxis.

Um die Auswirkungen der Beispiele besser sichtbar zu machen, wird jedes Element der zurückgegebenen Listen mit **⟨** und **⟩** in Klammern gesetzt. Bei Editoren, die UTF-8 nicht unterstützen, können diese durch **\u276C** und **\u276D** ersetzt werden.

Dieses erste Beispiel zeigt, wie Sie Zeichenfolgen nach ihrer Länge aufteilt.

```
const LINE1 "α and ℤ"
set chars [::textutil::split::splitn $LINE1] ;# splits into characters
puts ⟨[join $chars "⟩ ⟨"]⟩
set chars [split $LINE1 ""] ;# splits into characters
puts ⟨[join $chars "⟩ ⟨"]⟩
set chunks [::textutil::split::splitn $LINE1 3]
puts ⟨[join $chunks "⟩ ⟨"]⟩
=>
⟨α⟩ ⟨ ⟩ ⟨a⟩ ⟨n⟩ ⟨d⟩ ⟨ ⟩ ⟨ℤ⟩
⟨α⟩ ⟨ ⟩ ⟨a⟩ ⟨n⟩ ⟨d⟩ ⟨ ⟩ ⟨ℤ⟩
⟨α a⟩ ⟨nd ⟩ ⟨ℤ⟩
```

Dieses zweite Beispiel zeigt, wie Sie Zeichenfolgen mithilfe regulärer Ausdrücke aufteilt.

```
const LINE2 "a, be , cat, done , eagle"
set chunks [::textutil::split::splitx $LINE2] ;# splits on [\t \r\n]+
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*,\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
set chunks [::textutil::split::splitx $LINE2 {\s*(,)\s*}]
puts ⟨[join $chunks "⟩ ⟨"]⟩
=>
⟨a,⟩ ⟨be⟩ ⟨,⟩ ⟨cat,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
⟨a⟩ ⟨be⟩ ⟨cat⟩ ⟨done⟩ ⟨eagle⟩
⟨a⟩ ⟨,⟩ ⟨be⟩ ⟨,⟩ ⟨cat⟩ ⟨,⟩ ⟨done⟩ ⟨,⟩ ⟨eagle⟩
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Textverarbeitung

textutil_string
Stichwörter: textutil_string, capitalize, chop, common prefix, formatting, prefix, string, uncapitalize

Name
textutil::string – Prozeduren zur Bearbeitung von Texten und Zeichenfolgen.

Übersicht
package require Tcl 8.5 9
package require textutil::string ?**0.9**?
::textutil::string::chop *string*
::textutil::string::tail *string*
::textutil::string::cap *string*
::textutil::string::capEachWord *string*
::textutil::string::uncap *string*
::textutil::string::longestCommonPrefixList *list*
::textutil::string::longestCommonPrefix ?*string...*?

Beschreibung
Das Paket **textutil::string** bietet verschiedene Befehle zur Zeichenfolgenbearbeitung.

Wie Sie aus einer Liste von Pfaden einen gemeinsamen Pfad ermittelt, erfahren Sie im letzten der Beispiele.

API
::textutil::string::chop *string*
Gibt eine Kopie von *string* zurück, bei der das letzte Zeichen entfernt wurde.

::textutil::string::tail *string*
Gibt eine Kopie von *string* zurück, bei der das erste Zeichen entfernt wurde.

::textutil::string::cap *string*
Gibt eine Kopie von *string* zurück, bei der das erste Zeichen großgeschrieben ist.

::textutil::string::capEachWord *string*
Gibt eine Kopie von *string* zurück, bei der das erste Zeichen jedes Wortes großgeschrieben ist.

::textutil::string::uncap *string*
Gibt eine Kopie von *string* zurück, wobei das erste Zeichen in Kleinbuchstaben umgewandelt wird.

::textutil::string::longestCommonPrefixList *list*
Gibt das längste gemeinsame Präfix der Zeichenketten in der angegebenen *Liste* zurück.

Wird kein Argument angegeben, gibt es die leere Zeichenkette zurück.

::textutil::string::longestCommonPrefix ?*string*...?
Gibt das längste gemeinsame Präfix der Zeichenfolgenargumente *string*, ... zurück.

Wird eine einzelne Zeichenfolge angegeben, wird diese zurückgegeben, da sie ihr eigenes längstes gemeinsames Präfix ist. Wird kein Argument angegeben, wird die leere Zeichenfolge zurückgegeben.

Beispiele

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::string** in der Praxis.

Dieses erste Beispiel zeigt alle Befehle, die auf eine einzelne Zeichenkette angewendet werden können.

```
const LINE "has the σαβ cogs"
puts ""$LINE" const"
set line [::textutil::string::chop $LINE]
puts ""$line" chop"
set line [::textutil::string::tail $line]
puts ""$line" tail"
set line [::textutil::string::cap $line]
puts ""$line" cap"
set line [::textutil::string::capEachWord $line]
puts ""$line" capEachWord"
set line [::textutil::string::uncap $line]
puts ""$line" uncap"
=>
"has the σαβ cogs" const
"has the σαβ cog" chop
"as the σαβ cog" tail
"As the σαβ cog" cap
"As The Σαβ Cog" capEachWord
"as The Σαβ Cog" uncap
```

Dieses Beispiel zeigt die Befehle für das längste gemeinsame Präfix, die derzeit verwendet werden.

```
const LIST [list handbag handcuff handful handle handy]
puts "[::textutil::string::longestCommonPrefixList $LIST]"
puts "[::textutil::string::longestCommonPrefix king queen knave]"
puts "[::textutil::string::longestCommonPrefix king kin kind kinks]"
=>
"hand"
""
"kin"
```

Dieses Beispiel zeigt, wie Sie mit dem Befehl **::textutil::string::longestCommonPrefixList** den gemeinsamen Pfad aus einer Liste von Pfaden ermittelt.

```
proc longestCommonPath paths {
    const SEP [file separator]
    set path [::textutil::string::longestCommonPrefixList $paths]
    if {[string index $path end] ne $SEP} {
        if {[set j [string last $SEP $path]] > -1} {
            set path [string range $path 0 $j-1]
        }
    }
    if {$path ne $SEP} { set path [string trimright $path $SEP] }
    return $path
}

const PATHS [list /home/sally/music /home/sally/museums
    /home/sally/musings]
puts [::textutil::string::longestCommonPrefixList $PATHS]
puts [longestCommonPath $PATHS]
puts [longestCommonPath [list /bin / /sbin]]
=>
/home/sally/mus
/home/sally
/
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

trim

Stichwörter: trim, prefix, regular expression, string, trimming

Name

textutil::trim – Prozeduren zum Kürzen von Zeichenketten

Übersicht

package require Tcl 8.5 9
package require textutil::trim ?**0.8**?
::textutil::trim::trim *string* ?*regexp*?
::textutil::trim::trimleft *string* ?*regexp*?
::textutil::trim::trimright *string* ?*regexp*?

::textutil::trim::trimPrefix *string prefix*
::textutil::trim::trimEmptyHeading *string*

Beschreibung

Das Paket **textutil::trim** enthält Befehle, mit denen Zeichenfolgen auf verschiedene Weise gekürzt werden können, unter anderem mithilfe beliebiger regulärer Ausdrücke.

Um Zeichenfolgen von bestimmten Zeichengruppen (z. B. Leerzeichen) zu entfernen, verwenden Sie die integrierten Unterbefehle **trim**, **trimleft** und **trimright** des Befehls string.

API

::textutil::trim::trim *string* ?*regexp*?
: Gibt eine Kopie von *string* zurück, wobei alle am Anfang oder Ende stehenden Teilzeichenfolgen, die mit *regexp* übereinstimmen, aus *string* entfernt werden, sofern dieser keine Zeilenumbrüche enthält, bzw. aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält. Der Standardwert für *regexp* ist *[\t]+*. Dieser Standardwert entfernt führende und abschließende Leerzeichen und Tabulatoren aus *string*, wenn dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, wenn dieser Zeilenumbrüche enthält.

::textutil::trim::trimleft *string* ?*regexp*?
: Gibt eine Kopie von *string* zurück, wobei alle führenden Teilzeichenfolgen, die mit *regexp* übereinstimmen, aus *string* entfernt werden, sofern dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält. Der Standardwert für *regexp* ist *[\t]+*. Dieser Standardwert entfernt führende Leerzeichen und Tabulatoren aus *string*, wenn dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, wenn dieser Zeilenumbrüche enthält.

::textutil::trim::trimright *string* ?*regexp*?
: Gibt eine Kopie von *string* zurück, wobei alle am Ende stehenden Teilzeichenfolgen, die mit *regexp* übereinstimmen, aus *string* entfernt werden, sofern dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält. Der Standardwert für *regexp* ist *[\t]+*. Dieser Standardwert entfernt abschließende Leerzeichen und Tabulatoren aus *string*, sofern dieser keine Zeilenumbrüche enthält, oder aus *jeder Zeile* innerhalb von *string*, sofern dieser Zeilenumbrüche enthält.

::textutil::trim::trimPrefix *string prefix*
: Gibt eine Kopie von *string* zurück, wobei *prefix* am Anfang entfernt wird, sofern es vorhanden ist; andernfalls wird *string* unverändert zurückgegeben.

  Vergleichen Sie diesen Befehl mit den Unterbefehlen **trim**, **trimleft** und **trimright** des integrierten Befehls string, die alle Zeichen aus einer als Zeichenkette übergebenen Zeichenmenge entfernen.

::textutil::trim::trimEmptyHeading *string*
: Gibt eine Kopie von *string* zurück, aus der alle führenden Leerzeilen (einschließlich Zeilen, die ausschließlich Leerzeichen enthalten) entfernt wurden.

Beispiele

Die folgenden Beispiele veranschaulichen einige Befehle des Pakets **::textutil::trim** in der Praxis.

Die folgende **proc** wird in einigen der nachfolgenden Beispiele verwendet.

```
proc replace_ws s { regsub -all { } [regsub -all \t [regsub -all \n $s
    ⏎] →] ▲ }
```

Diese Beispiele veranschaulichen die Verwendung von **::textutil::trim::trimEmptyHeading** und **::textutil::trim::trim**.

```
const LINES1 "\t\n Alpha\t\n Beta \t\n Gamma \t\n"
puts "== const ============================\n[replace_ws $LINES1]"
set line [::textutil::trim::trimEmptyHeading $LINES1]
puts "== textutil::trim::trimEmptyHeading ==\n[replace_ws $line]"
set line [string trimleft $LINES1]
puts "== string trimleft ==================\n[replace_ws $line]"
set line [::textutil::trim::trim $LINES1]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
→⏎▲Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== textutil::trim::trimEmptyHeading ==
▲Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== string trimleft ==================
Alpha→⏎▲Beta▲→⏎▲Gamma▲→⏎
== textutil::trim::trim ==============
⏎Alpha⏎Beta⏎Gamma⏎
```

Beachten Sie den subtilen Unterschied im Verhalten zwischen **::textutil::trim::trimEmptyHeading** und **string trimleft**.

Dieses Beispiel veranschaulicht die Verwendung der integrierten Funktion **string trim** in Kombination mit **::textutil::trim::trim**.

```
const LINES2 [string trim "\t\n Delta\t\n Epsilon \t\n Zeta \t\n"]
puts "== const ============================\n[replace_ws $LINES2]"
set line [::textutil::trim::trim $LINES2]
puts "== textutil::trim::trim ==============\n[replace_ws $line]"
=>
== const ============================
Delta→⏎▲Epsilon▲→⏎▲Zeta
== textutil::trim::trim ==============
Delta⏎Epsilon⏎Zeta
```

In diesen Beispielen wird die integrierte Funktion **string trim** mit **::textutil::trim::trimPrefix** verglichen.

```
const PATH /home/homer
puts "const='$PATH'"
puts "string trim \$PATH /home='[string trim $PATH /home]'"
puts "string trim \$PATH /home='[string trim $PATH ehmo/]'"
puts "::textutil::trim::trimPrefix \$PATH
```

```
    /home='[::textutil::trim::trimPrefix $PATH /home]'"
const LINE mimic
puts "string trim \$LINE mic='[string trim $LINE mic]'"
puts "::textutil::trim::trimPrefix \$LINE
    mic='[::textutil::trim::trimPrefix $LINE mic]'"
=>
const='/home/homer'
string trim $PATH /home='r'
string trim $PATH /home='r'
::textutil::trim::trimPrefix $PATH /home='/homer'
string trim $LINE mic=''
::textutil::trim::trimPrefix $LINE mic='mimic'
```

Bei den **PATH**-Beispielen entfernt der Befehl **string trim** alle einzelnen Zeichen, die in der Zeichenkette "/home" enthalten sind, d. h. "/", "h", "o", "m", "e". Die Reihenfolge der Zeichen in der Zeichenkette spielt keine Rolle, wie das Beispiel "ehmo/" zeigt. Vergleichen Sie dies mit **::textutil::trim::trimPrefix**, das die Zeichenkette "/home" wörtlich trimmt.

Bei den **LINE**-Beispielen entfernt der Befehl **string trim** alle einzelnen Zeichen, die in der Zeichenkette "mic" enthalten sind, d. h. "m", "i" und "c". Vergleichen Sie dies mit **::textutil::trim::trimPrefix**, das die Literalzeichenfolge "mic" trimmt; da die **LINE** jedoch nicht mit "mic" beginnt, wird nichts getrimmt und die Zeichenfolge wird unverändert gegeben.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

wcswidth

Stichwörter: wcswidth, character type, character width, double-wide character, prefix, regular expression, string

Name

textutil::wcswidth – Prozeduren zur Berechnung der Endbreite von Zeichenketten

Übersicht

package require Tcl 8.5 9
package require textutil::wcswidth ?**35.3**?
::textutil::wcswidth *string*
::textutil::wcswidth_char *codepoint*

::textutil::wcswidth_type *codepoint*

Beschreibung

Das Paket **textutil::wcswidth** enthält Befehle, die Zeichentypen, Zeichenbreiten und Zeichenfolgenbreiten im Zusammenhang mit der Ausgabe auf dem Terminal (der Konsole) anzeigen.

Die Daten dieses Pakets und damit auch die zugrunde liegende Funktionalität basieren auf der Unicode-Datenbankdatei http://www.unicode.org/Public/UCD/latest/ucd/EastAsianWidth.txt.

API

Die Befehle dieses Pakets berücksichtigen Zeichen doppelter Breite aus den verschiedenen asiatischen und anderen Skripten.

::textutil::wcswidth *string*
: Gibt die Anzahl der Zeichenzellen zurück, die für die Ausgabe der angegebenen *Zeichenkette* auf einem Terminal benötigt werden.

::textutil::wcswidth_char *codepoint*
: Gibt die Anzahl der Zeichenzellen zurück, die für das angegebene Unicode-Zeichen mit dem *Codepoint* bei der Ausgabe auf einem Terminal benötigt werden.

  Wichtig: Das Unicode-Zeichen mit dem *Codepoint* muss als Ganzzahl festgelegt werden.

::textutil::wcswidth_type *codepoint*
: Gibt den Zeichentyp des angegebenen Unicode-Zeichens mit dem *Codepunkt* zurück.

  Der Wert ist eine Zeichenkette, die eine der folgenden Konstanten enthält: **A**, **F**, **H**, **N**, **Na** oder **W**, wie in http://www.unicode.org/Public/UCD/latest/ucd/EastAsianWidth.txt festgelegt.

  Wichtig: Das Unicode-Zeichen *codepoint* muss als Ganzzahl festgelegt werden.

Beispiele

Dieses Beispiel zeigt alle drei Befehle in der Praxis.

```
const LINE "Δ÷☕M☃𝕫"
puts "string length  = [string length $LINE]"
puts "terminal width = [textutil::wcswidth $LINE]"
foreach char [split $LINE ""] {
    set cp [scan $char %c]
    set pad [string repeat " " [expr {2 - [textutil::wcswidth_char
    $cp]}]]
    puts [format "%-2s%sU+%04X %s" $char $pad $cp
    [textutil::wcswidth_type $cp]]
}
=>
string length  = 6
terminal width = 8
Δ  U+0394 A
÷  U+00F7 A
```

```
☕ U+2615 W
M  U+004D N
☔ U+2614 W
ℤ  U+2124 N
```

Auf einem Terminal, das eine Monospace-Schriftart verwendet, ist die oben gezeigte Ausgabe *perfekt* ausgerichtet; jegliches "Wackeln", das auf dieser Seite zu sehen ist, ist ein Artefakt.

Hinweis: Die Ausgabe wäre *nicht* ausgerichtet, wenn **pad** auf **" "** gesetzt wäre, d. h. wenn die Zeichenbreiten nicht berücksichtigt würden, z. B.:

```
string length  = 6
terminal width = 8
Δ  U+0394 A
÷  U+00F7 A
☕  U+2615 W
M  U+004D N
☔  U+2614 W
ℤ  U+2124 N
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *textutil* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Textverarbeitung

tie

tie

Stichwörter: tie, array, database, file, metakit, persistence, untie

Name

tie – Persistenz von Arrays

Übersicht

package require Tcl 8.5 9
package require tie ?**1.3**?
::tie::tie *arrayvarname options... dstype dsname...*
::tie::untie *arrayvarname* ?*token*?
::tie::info ties *arrayvarname*
::tie::info types

::tie::info type *dstype*
::tie::register *dsclasscmd* **as** *dstype*
dsclasscmd *objname* ?*dsname...*?
ds destroy
ds names
ds size
ds get
ds set *dict*
ds unset ?*pattern*?
ds setv *index value*
ds unsetv *index*
ds getv *index*

Beschreibung

Das Paket **tie** bietet einen Rahmen für die Erstellung persistenter Tcl-Array-Variablen. Zu beachten ist, dass der bereitgestellte Mechanismus so allgemein gehalten ist, dass er auch für die Verteilung der Inhalte von Tcl-Arrays auf mehrere Threads und Prozesse, d. h. für die Kommunikation, genutzt werden kann.

Dies – Persistenz und Kommunikation – wird erreicht, indem eine Tcl-Array-Variable an eine *Datenquelle gebunden* wird. Beispiele für Datenquellen sind andere Tcl-Arrays und Dateien.

Beachten Sie, dass eine einzelne Tcl-Array-Variable mit mehr als einer *Datenquelle* verknüpft werden kann. Gerade diese Funktion ermöglicht es, den Rahmen auch für die Kommunikation zu nutzen. Verknüpfen Sie einfach mehrere Tcl-Arrays in verschiedenen Client-Prozessen mit einem Tcl-Array auf einem Server, und alle Änderungen an einem dieser Arrays werden an alle anderen weitergegeben. Natürlich sind auch weniger zentralisierte Varianten davon möglich.

Verwendung von Ties

Tie-API

In diesem Abschnitt wird die grundlegende API beschrieben, mit der Verknüpfungen zwischen Tcl-Array-Variablen und Datenquellen hergestellt und aufgehoben werden. Diese Schnittstelle ist die einzige, mit der sich ein Gelegenheitsnutzer befassen muss. Die folgenden Abschnitte über die verschiedenen internen Schnittstellen können getrost übersprungen werden.

::tie::tie *arrayvarname options... dstype dsname...*

Dieser Befehl stellt eine Verknüpfung zwischen dem Tcl-Array her, dessen Name durch das Argument *arrayvarname* angegeben wird, und der *Datenquelle*, die durch das Argument *dstype* und die Reihe von *dsname*-Argumenten identifiziert wird. Alle Änderungen, die nach der Ausführung dieses Befehls am Tcl-Array vorgenommen werden, werden zur sicheren Aufbewahrung (oder zur Verteilung) in der *Datenquelle* gespeichert.

Das Ergebnis des Befehls ist stets ein Token, das die neue Verknüpfung identifiziert. Dieses Token kann später verwendet werden, um diese spezifische Verknüpfung aufzuheben.

varname *arrayvarname* (in)

Der Name der Tcl-Array-Variablen, mit der die neue Verbindung hergestellt werden soll.

name|command *dstype* (in)

Dieses Argument legt den Typ der *Datenquelle* fest, auf die Sie zugreifen möchten. Der *dstype* kann einer der folgenden Werte sein: **log**, **array**, **remotearray**, **file**, **growfile** oder

dsource; darüber hinaus kann der Programmierer weitere Datenquellentypen registrieren. Auf jeden *dstype* folgen ein oder mehrere Argumente, die die *Datenquelle* identifizieren, an die das Array gebunden werden soll.

string *dsname* (in)
Die Reihe von *dsname*-Argumenten, die auf den *dstype* folgen, identifiziert die *Datenquelle* , mit der wir eine Verbindung herstellen möchten, und muss für den gewählten Typ geeignet sein.

Der Befehl unterstützt eine Reihe weiterer Optionen, die den Prozess der Einrichtung der Verbindung zwischen dem Tcl-Array und der *Datenquelle* steuern.

-open
Das Tcl-Array für die neue Verknüpfung wird aus der *Datenquelle geladen*, und der zuvor vorhandene Inhalt des Tcl-Arrays wird gelöscht. Es wird darauf geachtet, den bisherigen Inhalt *nicht* zu löschen, falls die Erstellung der Verknüpfung fehlschlägt.

Diese Option und die Option **-save** schließen sich gegenseitig aus. Wenn weder diese noch die Option **-save** angegeben sind, wird diese Option als Standard angenommen.

-save
Das Tcl-Array für die neue Verknüpfung wird in der *Datenquelle gespeichert*, und der zuvor vorhandene Inhalt der *Datenquelle* wird gelöscht.

Diese Option und die Option **-open** schließen sich gegenseitig aus. Wenn weder diese Option noch die Option **-open** festgelegt wird, wird standardmäßig die Option **-open** angenommen.

-merge
Die Verwendung dieser Option verhindert das Löschen bereits vorhandener Inhalte und führt stattdessen eine Zusammenführung der Daten durch. Sie kann in Verbindung mit **-open** oder **-save** festgelegt werden. Diese Optionen legen fest, wie mit Daten verfahren wird, die sowohl im Tcl-Array als auch in der *Datenquelle* vorhanden sind, d. h. mit Duplikaten.

Wenn **-open** verwendet wird, haben die Daten in der *Datenquelle* Vorrang. Mit anderen Worten: Bei Duplikaten werden die Daten aus der *Datenquelle* in das Tcl-Array geladen.

Bei Verwendung mit **-save** haben die Daten im Tcl-Array Vorrang. Mit anderen Worten: Bei Duplikaten werden die Daten aus dem Tcl-Array in die *Datenquelle* gespeichert.

::tie::untie *arrayvarname* ?*token*?
Dieser Befehl löst eine oder mehrere Verknüpfungen auf, die mit dem durch *arrayvarname* benannten Tcl-Array verbunden sind. Wird kein *token* angegeben, werden alle Verknüpfungen zu diesem Tcl-Array aufgelöst. Andernfalls wird nur die Verbindung entfernt, für die das Token steht, sofern diese tatsächlich mit dem Array verbunden ist. Der Versuch, eine bestimmte Verbindung zu entfernen, die nicht zu dem angegebenen Array gehört, führt zu einem Fehler.

Zu beachten ist, dass durch das Aufheben einer Verbindung zwar die paketinternen Verwaltungsinformationen zerstört werden, die *Datenquelle*, die von der Verbindung verwaltet wurde, jedoch nicht verändert, sondern lediglich geschlossen wird.

Nachdem der Befehl zurückgibt, werden keine der am Array vorgenommenen Änderungen mehr in der *Datenquelle* gespeichert.

Das Ergebnis des Befehls ist eine leere Zeichenkette.

varname *arrayname* (in)
Der Name einer Tcl-Array-Variablen, die möglicherweise Verknüpfungen aufweist.

handle *token* (in)
Ein Handle, das eine bestimmte Verbindung repräsentiert. Dieses Argument ist optional.

::tie::info ties *arrayvarname*
Dieser Befehl gibt eine Liste der "Ties" zurück, die der durch *arrayvarname* bezeichneten Tcl-Array-Variablen zugeordnet sind. Die Ergebnisliste ist leer, wenn der Variablen keine "Ties" zugeordnet sind.

::tie::info types
Dieser Befehl gibt ein Dictionary zurück, das die registrierten Typen sowie die ihnen zugeordneten Klassenbefehle enthält.

::tie::info type *dstype*
Dieser Befehl gibt den vollständig aufgelösten Befehl für eine Klasse mit dem Namen des Typs zurück. Das bedeutet, dass der Befehl einer Kette von Typdefinitionen bis zu deren Ende folgt.

Standarddatenquellentypen

Dieses Paket stellt die folgenden sechs Typen als Beispiele und Standarddatenquellen bereit.

log
Diese *Datenquelle* verwaltet weder tatsächliche Daten noch sorgt sie für deren dauerhafte Speicherung. Sie akzeptiert keine identifizierenden Argumente. Alle Änderungen werden lediglich in **stdout** protokolliert.

array
Diese *Datenquelle* verwendet ein reguläres Tcl-Array als Quelle für die persistenten Daten. Sie akzeptiert ein einziges identifizierendes Argument, nämlich den Namen dieses Tcl-Arrays. Alle Änderungen werden in dieses Array übernommen.

remotearray
Diese *Datenquelle* ähnelt **array**. Der Unterschied besteht darin, dass auf das Tcl-Array, auf das wir spiegeln, nicht direkt zugegriffen werden kann, sondern nur über einen Befehl vom Typ **send**.

Er akzeptiert drei identifizierende Argumente: den Namen des anderen Tcl-Arrays, den Befehls-Präfix für den **send**-ähnlichen Zugriffs-Befehl sowie eine Kennung für die Remote-Entität, die das Array hostet – in dieser Reihenfolge. Alle Änderungen werden über das Befehlspräfix auf dieses Array gespiegelt. Alle Befehle werden im Kontext des globalen Namespaces ausgeführt.

"**send**-ähnlich" bedeutet, dass das Befehlspräfix die Syntax und Semantik von **send** aufweisen muss. Das heißt, es handelt sich um einen Kanal, über den wir beliebige Befehle an eine andere Instanz senden können. Die *Datenquelle* des Remote-Arrays verwendet jedoch ausschließlich die Befehle **set**, **unset**, **array exists**, **array names**, **array set** und **array get** , um Werte im Remote-Array abzurufen und festzulegen.

Das Befehlspräfix und die Entitäts-ID sind voneinander getrennt, damit die Datenquelle beim

Zusammenstellen der eigentlichen Befehle Optionen wie **-async** verwenden kann.

Beispiele für Befehlspräfixe, aufgeführt zusammen mit der ID der Remote-Entität, ohne Optionen. Tatsächlich ist nur der Teil vor der ID das Befehlspräfix:

send *tkname*
: Das Tcl-Array befindet sich in einem entfernten Interpreter und wird über die X-Kommunikation von Tk aufgerufen.

comm::comm send *hostportid*
: Das Tcl-Array befindet sich in einem Remote-Interpreter und wird über einen Socket aufgerufen.

thread::send *threadid*
: Das Tcl-Array befindet sich in einem Remote-Interpreter in einem anderen Thread dieses Prozesses.

file
: Diese *Datenquelle* verwendet eine einzelne Datei als Quelle für die persistenten Daten. Sie akzeptiert ein einziges identifizierendes Argument: den Pfad zu dieser Datei. Die Datei muss sowohl lesbar als auch beschreibbar sein. Sie darf noch nicht vorhanden sein; in diesem Fall legt die *Datenquelle* sie an. Nur in diesem (und ausschließlich in diesem) Fall muss das Verzeichnis, in dem sich die Datei befindet, ebenfalls vorhanden und beschreibbar sein.

  Alle Änderungen werden in der Datei als korrekte Tcl-Befehle gespeichert, wobei pro Operation ein Befehl steht. Mit anderen Worten: Die Datei enthält stets ein korrektes Tcl-Skript.

  Existiert die Datei bereits, wenn die sie verwendende "tie"-Verbindung eingerichtet wird, wird sie komprimiert, d. h., überflüssige Operationen werden entfernt, sofern das darin gespeicherte Operationenprotokoll entweder mindestens eine Operation enthält, die das gesamte Array löscht, oder mindestens das 1,5-Fache an Operationen im Vergleich zu den Einträgen im geladenen Array aufweist.

growfile
: Diese *Datenquelle* ähnelt **file** hinsichtlich des Speichermediums für die Array-Daten und ihrer Konfiguration. Im Gegensatz zu ersterer geht sie jedoch davon aus und stellt sicher, dass das verknüpfte Array niemals schrumpft. Das heißt: Das Anlegen neuer Array-Einträge sowie die Änderung bestehender Einträge ist zulässig, das Löschen von Einträgen hingegen nicht und führt dazu, dass die Datenquelle Fehler meldet.

  Diese Einschränkung ermöglicht es uns, sowohl das Dateiformat als auch den Zugriff auf die Datei radikal zu vereinfachen. Zum einen wird die Datei nur einmal gelesen, und der interne Cache kann nicht ungültig gemacht werden. Zweitens beschränkt sich das Schreiben von Daten auf ein einfaches Anhängen, und es ist kein Komprimierungsschritt erforderlich. Das Format des Inhalts ist die Zeichenfolgendarstellung eines Dictionarys, das am Ende unbegrenzt schrittweise erweitert werden kann.

dsource
: Diese *Datenquelle* verwendet ein explizit angegebenes *Datenquellenobjekt* als Quelle für die persistenten Daten. Sie akzeptiert ein einziges identifizierendes Argument, das Befehlspräfix,

d. h. den Objektbefehl.

Um diesen Typ zu verwenden, müssen Sie wissen, wie der Rahmen Verknüpfungen verwaltet und was **data source objects** sind.

Alle Änderungen werden an das angegebene Objekt weitergeleitet.

Neue Datenquellen anlegen

Dieser Abschnitt ist für den gelegentlichen Nutzer von Ties nicht von Interesse. Nur Entwickler, die neue Datenquellen erstellen möchten, müssen die hier bereitgestellten Informationen kennen.

Datenquellen-Objekte

Alle Verknüpfungen werden intern durch ein In-Memory-Objekt dargestellt, das als Vermittler zwischen dem Verknüpfungs-Rahmen und der jeweiligen *Datenquelle* – wie beispielsweise einem Array, einer Datei usw. – fungiert. Dies ist das *Datenquellen-Objekt*.

Die Klasse "**data source class**" ist *keine* generische Klasse, sondern spezifisch für den Typ der *Datenquelle*. Um eine neue *Datenquelle* zu erstellen, müssen wir eine solche Klasse schreiben und diese anschließend als neuen Typ beim Rahmen registrieren.

In den folgenden Unterabschnitten werden die verschiedenen APIs beschrieben, denen ein **data source class** und die von ihm generierten Objekte entsprechen müssen, um die Kompatibilität mit dem tie-Rahmen zu gewährleisten.

Datenquellenobjekte werden normalerweise automatisch vom Rahmen erstellt und gelöscht, wenn eine Verknüpfung angelegt oder entfernt wird. Diese Verwaltung kann durch die Verwendung des Typs "dsource" explizit umgangen werden. Die *Datenquelle* für diesen Typ ist selbst ein *Datenquellenobjekt*, und dieses Objekt befindet sich außerhalb des Geltungsbereichs des Tie-Rahmens und wird von diesem nicht verwaltet. Mit anderen Worten: Dieser Typ ermöglicht die Erstellung von Ties, die mit bereits vorhandenen *Datenquellenobjekten* kommunizieren, und diese Objekte bleiben auch nach dem Entfernen der Ties, die sie nutzen, erhalten.

Eine neue Datenquellenklasse registrieren

Nachdem ein **data source class** geschrieben wurde, muss es als neuer Typ im Rahmen registriert werden.

::tie::register *dsclasscmd* as *dstype*

Durch die Verwendung dieses Befehls speichert das Tie-Framework den Befehl für die Klasse *dsclasscmd* eines **data source class** unter dem Namen *dstype*.

Nach dem Aufruf akzeptiert das Argument *dstype* des Basisbenutzerbefehls **::tie::tie** *dstype* als Typnamen und wandelt diesen intern in den entsprechenden Befehl für die Klasse um, um **data source objects** für die neue *Datenquelle* anzulegen.

Datenquellenklasse

Jede Datenquellenklasse wird durch einen einzelnen Befehl dargestellt, der auch als *Klassenbefehl* oder *Objektbefehl* bezeichnet wird. Seine Syntax lautet

dsclasscmd *objname* ?*dsname*...?

Das erste Argument des Befehls "class" ist der Name des zu erstellenden *Datenquellenobjekts*. Der Rahmen liefert stets die Zeichenfolge **%AUTO%**, um anzuzeigen, dass der Befehl "class" nicht nur das Objekt, sondern auch den Objektnamen generieren muss.

Darauf folgt eine Reihe von Argumenten, die die Datenquelle identifizieren, für die das neue Objekt bestimmt ist. Dabei handelt es sich um dieselben *dsname*-Argumente, die auch dem grundlegenden Benutzerbefehl **::tie::tie** übergeben werden. Ihre tatsächliche Bedeutung hängt von der *Datenquellenklasse* ab.

Das Ergebnis des Befehls "class" muss der vollqualifizierte Name des neuen *Datenquellenobjekts* sein, d. h. der Name des *Objektbefehls*. Die Schnittstelle, der dieser Befehl folgen muss, wird im Abschnitt API für Datenquellenobjekte beschrieben.

API für Datenquellenobjekte

Bitte lesen Sie zunächst den Abschnitt Datenquellenklasse, um zu erfahren, wie Sie neue *Objektbefehle* erstellen können.

Jeder *Objektbefehl* für ein *Datenquellen*-Objekt muss mindestens die unten aufgeführten Methoden bereitstellen, um eine ordnungsgemäße Zusammenarbeit mit dem "tie"-Rahmen zu gewährleisten. Beachten Sie, dass die Namen der meisten Methoden mit den Unterbefehlen des integrierten Befehls **array** übereinstimmen.

ds destroy
Diese Methode wird aufgerufen, wenn das Objekt **ds** gelöscht wird. Es muss nun alle seine internen Ressourcen freigeben, die mit der externen Datenquelle verknüpft sind.

ds names
Dieser Befehl muss eine Liste geben, die die Namen aller Schlüssel enthält, die in der *Datenquelle* gefunden wurden, mit der das Objekt kommuniziert. Dies entspricht **array names**.

ds size
Dieser Befehl muss eine Ganzzahl geben, die die Anzahl der Schlüssel festlegt, die in der *Datenquelle* gefunden wurden, mit der das Objekt kommuniziert. Dies entspricht **array size**.

ds get
Dieser Befehl muss ein Dictionary geben, das die Daten enthält, die in der *Datenquelle* gefunden wurden, mit der das Objekt kommuniziert. Dies entspricht **array get**.

ds set *dict*
Dieser Befehl nimmt ein Dictionary entgegen und fügt dessen Inhalt der Datenquelle hinzu, mit der das Objekt kommuniziert. Dies entspricht dem Befehl **array set**.

ds unset ?*pattern*?
Dieser Befehl nimmt ein Muster entgegen und entfernt alle Elemente aus der *Datenquelle*, deren Schlüssel diesem Muster entsprechen. Wird kein Muster angegeben, wird standardmäßig "*****" verwendet, was zur Entfernung aller Elemente führt. Dies entspricht nahezu dem Befehl "**array unset**".

ds setv *index value*
Dieser Befehl muss den *value* in der *Datenquelle*, mit der das Objekt kommuniziert, unter dem Schlüssel *index* speichern.

Das Ergebnis des Befehls wird ignoriert. Wird ein Fehler ausgelöst, wird dieser als Fehler der Set-Operation angezeigt, die den Methodenaufruf verursacht hat.

ds unsetv *index*
Dieser Befehl muss den Wert unter dem Schlüssel *index* aus der *Datenquelle* entfernen, mit der das Objekt kommuniziert.

Das Ergebnis des Befehls wird ignoriert. Wird ein Fehler ausgelöst, wird dieser als Fehler der Löschoperation angezeigt, die den Methodenaufruf verursacht hat.

ds getv *index*
Dieser Befehl muss den Wert für den Schlüssel *index* in der *Datenquelle* zurückgeben, mit der das Objekt kommuniziert.

Und hier finden Sie eine kleine Tabelle, in der die Methoden der *Datenquelle* mit den regulären Tcl-Befehlen für den Zugriff auf ein Array verglichen werden.

```
        Regular Tcl             Data source
        -----------             -----------
        array names a           ds names
        array size  a           ds size
        array get   a           ds get
        array set   a dict      ds set   dict
        array unset a pattern   ds unset ?pattern?
        -----------             -----------
        set a($idx) $val        ds setv   idx val
        unset a($idx)           ds unsetv idx
        $a($idx)                ds getv   idx
        -----------             -----------
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *tie* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright


```
Copyright (c) 2004-2021 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tie_std

Stichwörter: tie_std, array, database, file, metakit, persistence, tie, untie

Name
tie – Array-Persistenz, Standarddatenquellen

Übersicht
package require Tcl 8.5 9
package require tie::std::log ?**1.2**?
package require tie::std::array ?**1.2**?
package require tie::std::rarray ?**1.2**?
package require tie::std::file ?**1.2**?
package require tie::std::growfile ?**1.2**?
package require tie::std::dsource ?**1.2**?

Beschreibung
Bei den in diesem Dokument als Anforderungen aufgeführten Paketen handelt es sich um interne Pakete, die die Standarddatenquellen des Pakets **tie** bereitstellen, wie im Abschnitt *STANDARD-DATENQUELLENTYPEN* der Dokumentation zu **tie** beschrieben.

Sie werden automatisch von **tie** geladen und registriert, wenn dieses selbst aufgerufen wird; daher ist es nicht erforderlich, sie separat anzufordern, obwohl dies möglich ist.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *tie* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Programmierwerkzeuge

Copyright

```
Copyright (c) 2008-2021 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tiff

tiff
Stichwörter: tiff, image, tif

Name
tiff – Lesen und Schreiben von TIFF-Dateien sowie Abfragen und Bearbeitung von Metadaten

Übersicht

package require Tcl 8.5 9
package require tiff ?**0.2.3**?
::tiff::isTIFF *file*
::tiff::byteOrder *file*
::tiff::numImages *file*
::tiff::dimensions *file* ?*image*?
::tiff::imageInfo *file* ?*image*?
::tiff::entries *file* ?*image*?
::tiff::getEntry *file entry* ?*image*?
::tiff::addEntry *file entry* ?*image*?
::tiff::deleteEntry *file entry* ?*image*?
::tiff::getImage *file* ?*image*?
::tiff::writeImage *image file* ?*entry*?
::tiff::nametotag *names*
::tiff::tagtoname *tags*
::tiff::debug *file*

Beschreibung

Dieses Paket enthält Befehle zum Abfragen, Ändern, Lesen und Schreiben von TIFF-Bildern. TIFF steht für *Tagged Image File Format* und ist ein Standard für die verlustfreie Speicherung von fotografischen Bildern und zugehörigen Metadaten. Die Spezifikation finden Sie unter http://partners.adobe.com/public/developer/tiff/index.html.

In einer einzelnen TIFF-Datei können mehrere Bilder gespeichert sein. Die Argumente ?*image*? der Funktionen in diesem Paket dienen dem Zugriff auf andere Bilder als das erste. Die Daten in einem TIFF-Bild werden als eine Reihe von Tags mit numerischem Wert gespeichert, dargestellt entweder im vierstelligen Hexadezimalformat oder als Zeichenkettenname. Eine Übersicht definierter Tags und ihrer Bedeutung steht unter http://www.awaresystems.be/imaging/tiff/tifftags.html

Befehle

::tiff::isTIFF *file*
: Gibt einen booleschen Wert zurück, der angibt, ob *file* ein TIFF-Bild ist.

::tiff::byteOrder *file*
: Gibt entweder **big** oder **little** zurück. Löst einen Fehler aus, wenn *file* kein TIFF-Bild ist.

::tiff::numImages *file*
: Gibt die Anzahl der Bilder in *file* zurück. Löst einen Fehler aus, wenn *file* kein TIFF-Bild ist.

::tiff::dimensions *file* ?image?
: Gibt die Abmessungen des Bildes mit der Nummer ?*image*? in *file* als Liste der horizontalen und vertikalen Pixelanzahl zurück. Löst einen Fehler aus, wenn *file* kein TIFF-Bild ist.

::tiff::imageInfo *file* ?image?
: Gibt ein Dictionary mit den Schlüsseln **ImageWidth**, **ImageLength**, **BitsPerSample**, **Compression**, **PhotometricInterpretation**, **ImageDescription**, **Orientation**, **XResolution**, **YResolution**, **ResolutionUnit**, **DateTime**, **Artist** und **HostComputer** zurück. Die Werte sind die zugehörigen Eigenschaften des TIFF-Bildes ?*image*? in *file*. Die

Werte können leer sein, wenn das zugehörige Tag in der Datei fehlt.

```
puts [::tiff::imageInfo photo.tif]

ImageWidth 686 ImageLength 1024 BitsPerSample {8 8 8}
Compression 1
PhotometricInterpretation 2 ImageDescription {} Orientation 1
XResolution 170.667 YResolution 170.667 ResolutionUnit 2
DateTime {2005:12:28 19:44:45}
Artist {} HostComputer {}
```

An diesen Tags ist nichts Besonderes. Die Prozedur ruft lediglich **getEntry** mit gängigen Einträgen auf. Löst einen Fehler aus, wenn *file* kein TIFF-Bild ist.

::tiff::entries *file* ?image?

Gibt eine Liste aller Einträge in *file* und ?*image*? im Hexadezimalformat zurück. Löst einen Fehler aus, wenn *file* kein TIFF-Bild ist.

::tiff::getEntry *file entry* ?image?

Gibt den Wert von *entry* aus dem Bild ?*image*? in der TIFF-Datei *file* zurück. *entry* kann eine Liste mehrerer Einträge sein. Fehlt ein Eintrag, wird eine leere Zeichenkette zurückgegeben.

```
set data [::tiff::getEntry photo.tif {0131 0132}]
puts "file was written at [lindex $data 0] with software [lindex
$data 1]"
```

Es wird ein Fehler ausgelöst, wenn *file* kein TIFF-Bild ist.

::tiff::addEntry *file entry* ?image?

Fügt die angegebenen Einträge dem durch ?*image*? benannten Bild hinzu (Vorgabe 0) oder optional **all**. *entry* muss eine Liste sein, deren Elemente jeweils eine Liste aus Tag, Typ und Wert sind. Ist ein Tag schon vorhanden, wird es überschrieben.

```
::tiff::addEntry photo.tif {{010e 2 "an example photo"} {013b 2
"Aaron F"}}
```

Die Datentypen sind wie folgt definiert.

1
BYTE (8-Bit-Ganzzahl ohne Vorzeichen)

2
ASCII

3
SHORT (16-Bit-Ganzzahl ohne Vorzeichen)

4
LONG (32-Bit-Ganzzahl ohne Vorzeichen)

5
RATIONAL

6
SBYTE (vorzeichenbehaftetes 8-Bit-Byte)

7
UNDEFINED (nicht interpretierte Binärdaten)
8
SSHORT (vorzeichenbehaftete 16-Bit-Ganzzahl)
9
SLONG (vorzeichenbehaftete 32-Bit-Ganzzahl)
10
SRATIONAL
11
FLOAT (32-Bit-Gleitkommazahl)
12
DOUBLE (64-Bit-Gleitkommazahl)

Es wird ein Fehler ausgelöst, wenn *file* kein TIFF-Bild ist.

::tiff::deleteEntry *file entry* ?image?
Löscht die angegebenen Einträge aus dem durch ?*image*? benannten Bild (Vorgabe 0) oder optional aus **all**. Löst einen Fehler aus, wenn *file* kein TIFF-Bild ist.

::tiff::getImage *file* ?image?
Gibt den Namen eines Tk-Bildes zurück, das das Bild am Index ?*image*? aus *file* enthält. Löst einen Fehler aus, wenn *file* kein TIFF-Bild ist oder das Bild ein nicht unterstütztes Format hat. Unterstützt werden unkomprimiertes 24-Bit-RGB und unkomprimierte 8-Bit-Palette.

::tiff::writeImage *image file* ?entry?
Schreibt den Inhalt des Tk-Bildes *image* in eine TIFF-Datei *file*. Die Dateien werden unkomprimiert mit 24 Bit und Big-Endian-Byte-Reihenfolge geschrieben. Zusätzliche Einträge für das Bild können angegeben werden, im selben Format wie bei **tiff::addEntry**.

::tiff::nametotag *names*
Gibt eine Liste zurück, in der *names* von der Zeichenkette ins vierstellige Format übersetzt sind. Vierstellige Namen in der Eingabe werden unverändert durchgereicht. Zeichenketten ohne definierten Tag-Namen lösen einen Fehler aus.

::tiff::tagtoname *tags*
Gibt eine Liste zurück, in der *tags* vom vierstelligen Format in Zeichenketten übersetzt sind. Hat ein Tag keinen definierten Namen, wird es unverändert durchgereicht.

::tiff::debug *file*
Gibt alle uns bekannten Informationen zur angegebenen Datei in einem übersichtlichen Format aus.

Variablen

Für die Zuordnung von vierstelligen Tag-Namen zu Zeichenfolgen-Namen wird das Array ::tiff::tiff_tags verwendet. Für die umgekehrte Zuordnung wird das Array ::tiff::tiff_sgat verwendet.

Einschränkungen

1. EXIF-IFD kann nicht geschrieben werden
2. Lesen ist auf unkomprimiertes 8-Bit-RGB und unkomprimierte 8-Bit-Palettenbilder beschränkt
3. Schreiben ist auf unkomprimiertes 8-Bit-RGB beschränkt

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *tiff* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dateiformate

Copyright


```
Copyright (c) 2005-2006, Aaron Faupell <afaupell@users.sourceforge.net>
```


tool

meta

Stichwörter: meta, TclOO, callback, class methods, class variables, command prefix, currying, method reference, my method, singleton

Name

oo::util – Hilfsbefehle für TclOO

Übersicht

package require Tcl 8.5 9
package require TclOO
package require oo::util ?**1.2.2**?
mymethod *method* ?*arg...*?
classmethod *name arguments body*
classvariable ?*arg...*?
link *method...*
link {alias method}...
ooutil::singleton ?*arg...*?

Beschreibung

Dieses Paket bietet einen Komfortbefehl zur einfachen Festlegung von Instanzmethoden als Rückruffunktionen, wie beispielsweise Timer, Dateiereignisse, Tk-Bindungen usw.

Befehle

mymethod *method* ?*arg*...?

Dieser Befehl steht innerhalb von Instanzmethoden zur Verfügung. Er nimmt einen Methodennamen und gegebenenfalls Argumente für die Methode entgegen und gibt ein Befehlspräfix zurück, das bei seiner Ausführung die benannte Methode des Objekts, in dem wir uns gerade befinden, mit den übergebenen Argumenten sowie allen weiteren Argumenten

aufruft, die zum Zeitpunkt des tatsächlichen Aufrufs bereitgestellt werden.

Hinweis: Der Befehl entspricht dem Befehl, der vom OO-Paket **snit** für denselben Zweck bereitgestellt wird, und ist nach diesem benannt.

classmethod *name arguments body*

Dieser Befehl steht innerhalb von Klassendefinitionen zur Verfügung. Er nimmt einen Namen für die Methode und gegebenenfalls Argumente für die Methode entgegen und erstellt eine Methode in der Klasse, die für Benutzer dieser Klasse sowie abgeleiteter Klassen verfügbar ist.

Hinweis: Der Befehl entspricht dem Befehl **typemethod**, der vom OO-Paket **snit** für denselben Zweck bereitgestellt wird.

Beispiel

```
oo::class create ActiveRecord {
    classmethod find args { puts "[self] called with arguments:
    $args" }
}
oo::class create Table {
    superclass ActiveRecord
}
puts [Table find foo bar]
# ======
# which will write
# ======
# ::Table called with arguments: foo bar
```

classvariable ?*arg*...?

Dieser Befehl steht innerhalb von Instanzmethoden zur Verfügung. Er nimmt eine Reihe von Variablennamen entgegen und macht diese im Gültigkeitsbereich der Methode verfügbar. Der ursprüngliche Gültigkeitsbereich der Variablen ist die Klasse (Instanz), zu der die Objektinstanz gehört. Mit anderen Worten: Die referenzierten Variablen werden von allen Instanzen ihrer Klasse gemeinsam genutzt.

Hinweis: Der Befehl entspricht in etwa dem Befehl **typevariable**, der vom OO-Paket **snit** für denselben Zweck bereitgestellt wird. Der Unterschied besteht darin, dass er nicht in der Klassendefinition selbst verwendet werden kann.

Beispiel:

```
% oo::class create Foo {
    method bar {z} {
        classvariable x y
        return [incr x $z],[incr y]
    }
}
::Foo
% Foo create a
```

```
::a
% Foo create b
::b
% a bar 2
2,1
% a bar 3
5,2
% b bar 7
12,3
% b bar -1
11,4
% a bar 0
11,5
```

link *method*...
siehe unten ...

link {*alias method*}...
Dieser Befehl steht innerhalb von Instanzmethoden zur Verfügung. Er nimmt eine Liste von Methodennamen und/oder Paaren aus Alias- und Methodennamen entgegen und macht die genannten Methoden für alle Instanzmethoden verfügbar, ohne dass der Befehl **my** erforderlich ist.

Der Aliasname, unter dem die Methode verfügbar wird, entspricht standardmäßig dem Methodennamen, sofern nicht ausdrücklich ein Alias-Methoden-Paar angegeben wurde.

Beispiele:

```
    link foo
    # The method foo is now directly accessible as foo instead of my
    foo.

    link {bar foo}
    # The method foo is now directly accessible as bar.

    link a b c
    # The methods a, b, and c all become directly acessible under
    their
    # own names.
```

Dieser Befehl wird voraussichtlich vor allem in Instanz-Konstruktoren verwendet werden – entweder aus Gründen der Einfachheit oder um bestimmte Methoden für die Verwendung in einer Mini-DSL einzurichten.

ooutil::singleton ?*arg*...?
Dieser Befehl ist eine Metaklasse, d. h. eine Variante des integrierten Befehls **oo::class**, die sicherstellt, dass nur eine einzige Instanz der damit definierten Klassen erstellt wird.

Syntax und Ergebnisse entsprechen denen von **oo::class**.

Beispiel:

```
% oo::class create example {
   self mixin singleton
   method foo {} {self}
}
::example
% [example new] foo
::oo::Obj22
% [example new] foo
::oo::Obj22
```

Autoren
Donal Fellows, Andreas Kupries

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *oo::util* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Hilfsprogramme

Copyright

```
Copyright (c) 2011-2015 Andreas Kupries, BSD licensed
```


tool
Stichwörter: tool, TclOO, framework

Name
tool – TclOO-Bibliothek (TOOL)-Rahmen

Übersicht
package require Tcl 8.6 9
package require sha1
package require dicttool
package require oo::meta
package require oo::dialect
object **cget** *option*
object **configure** ?*keyvaluelist*?
object **configure** *field value* ?*field*? ?*value*? ?*...*?
object **configurelist** ?*keyvaluelist*?
object **forward** *stub forward*

object **graft** *stub forward*
object **InitializePublic**
object **Eval_Script** ?*script*?
object **Option_Default** *field*

tool::define **class_method** *arglist body*

tool::define **array** *name contents*

tool::define **array_ensemble** *methodname varname* ?cases?

tool::define **dict_ensemble** *methodname varname* ?cases?

tool::define **method** *methodname arglist body*

tool::define **option** *name dictopts*

tool::define **property** ?branch? *field value*

tool::define **variable** *name value*

Beschreibung

Dieses Modul implementiert den Tcl Object Oriented Library-Rahmen, kurz *TOOL*. Es ist als universell einsetzbarer Rahmen konzipiert, der eigenständig genutzt werden kann und leicht erweiterbar ist.

TOOL definiert eine Metaklasse, die der Beschreibung TclOO mehrere zusätzliche Schlüsselwörter, Standardverhalten für die zugehörigen Objekte sowie eine Top-Down-Integration mit den vom Paket **oo::meta** bereitgestellten Funktionen bereitstellt.

Die Metaklasse "TOOL" wurde mit dem Paket "**oo::dialect**" erstellt und kann daher als Grundlage für weitere Metaklassen dienen. Als Metaklasse verfügt "TOOL" über eine eigene "class"-Klasse, eine eigene "Objekt"-Klasse und definiert einen Namespace.

```
package require tool

# tool::class workds just like oo::class
tool::class create myclass {
}

# tool::define works just like oo::define
tool::define myclass method noop {} {}

# tool::define and tool::class understand additional keywords
tool::define myclass array_ensemble mysettings mysettings {}

# And tool interoperates with oo::define
oo::define myclass method do_something {} { return something }

# TOOL and TclOO objects are interchangeable
oo::class create myooclass {
  superclass myclass
}
```

In mehreren Handbuchseiten werden bestimmte Schlüsselwörter und Methoden ausführlicher

behandelt.

- **tool::array_ensemble**
- **tool::dict_ensemble**
- **tool::method_ensemble**
- **tool::object**
- **tool::option_handling**

Schlüsselwörter

TOOL fügt neue Schlüsselwörter für Klassendefinitionen hinzu (oder ändert bestehende). Die neuen Schlüsselwörter stehen jedoch nur in Aufrufen von *tool::class create* oder *tool::define* zur Verfügung.

tool::define class_method *arglist body*
: Definiert eine Methode für das Klassenobjekt selbst. Anders als bei **self method** wird diese Methode an die Nachfahren der Klasse weitergegeben.

tool::define array *name contents*
: Deklariert eine Variable *name*, die für Objekte dieser Klasse sowie für alle Objekte von Nachfahren dieser Klasse als Array angelegt und mit *contents* gefüllt wird.

tool::define array_ensemble *methodname varname* ?cases?
: Deklariert ein Methoden-Ensemble *methodname*, das den Zugriff auf die Variable *varname* steuert. *cases* ist eine Schlüssel-Wert-Liste aus Methodennamen und Rümpfen, die über die Standardvorlage gelegt werden. Siehe **tool::array_ensemble**.

  Ein Methodenname ist reserviert: **initialize**. **initialize** deklariert die Anfangswerte, mit denen das Array gefüllt wird, als Schlüssel-Wert-Liste; daraus wird keine Methode des Ensembles.

tool::define dict_ensemble *methodname varname* ?cases?
: Deklariert ein Methoden-Ensemble *methodname*, das den Zugriff auf die Variable *varname* steuert. *cases* ist eine Schlüssel-Wert-Liste aus Methodennamen und Rümpfen, die über die Standardvorlage gelegt werden. Siehe tool::dict_ensemble.

  Ein Methodenname ist reserviert: **initialize**. **initialize** deklariert die Anfangswerte, mit denen das Array gefüllt wird, als Schlüssel-Wert-Liste; daraus wird keine Methode des Ensembles.

tool::define method *methodname arglist body*
: Enthält *methodname* die Zeichenfolge ::, gilt die Methode als Teil eines Methoden-Ensembles. Siehe **tool::method_ensembles**. Andernfalls verhält sich dieser Befehl genau wie der Standardbefehl **oo::define method**.

tool::define option *name dictopts*
: Deklariert eine Option. *dictopts* ist eine Schlüssel-Wert-Liste mit den Parametern der Option. Siehe **tool::option_handling**.

```
tool::class create myclass {
  option color {
    post-command: {puts [list %self%'s %field% is now %value%]}
    default: green
  }
}
```

```
myclass create foo
foo configure color purple
> foo's color is now purple
```

tool::define property ?branch? *field value*
Definiert ein neues Blatt im Metadatenbaum der Klasse. Ohne *branch* erscheint das Blatt im Abschnitt *const* und ist entweder über die Methode **property** des Objekts oder über **oo::meta::info** *class* **get const** *field* erreichbar:

tool::define variable *name value*
Deklariert eine Variable *name*, die für Objekte dieser Klasse sowie für alle Objekte von Nachfahren dieser Klasse mit dem Wert *value* initialisiert wird.

Öffentliche Objektmethoden

Das TOOL-Objekt, die "Mutter aller Klassen", definiert mehrere Methoden, um ein einheitliches Verhalten im gesamten Rahmen sicherzustellen.

object* cget *option
Gibt den Wert der Option *option* dieses Objekts zurück. Wenn für diese Klasse der Wert **property options_strict** auf "true" gesetzt ist, führt der Aufruf einer Option, die nicht mit dem Schlüsselwort **option** deklariert wurde, zu einem Fehler. In allen anderen Fällen gibt das Objekt den Wert zurück, wenn er im Array *options* des Objekts vorhanden ist. Wenn er nicht vorhanden ist, versucht das Objekt, eine Eigenschaft mit demselben Namen abzurufen.

***object* configure ?keyvaluelist?**
siehe unten ...

***object* configure *field value* ?field? ?value? ?...?**
Dieser Befehl fügt dem Array *options* der Objekte neue Werte hinzu, gemäß den in den Beschreibungen der Optionen festgelegten Regeln. Einzelheiten finden Sie unter **tool::option_handling**. **configure** entfernt führende Minuszeichen ("-") aus Feldnamen, wodurch eine quasi-abwärtskompatible Funktionsweise zu tk-Optionen gewährleistet wird.

***object* configurelist ?keyvaluelist?**
Dieser Befehl fügt dem Array *options* der Objekte neue Werte hinzu, gemäß den in den Beschreibungen der Optionen festgelegten Regeln. Dieser Befehl führt Validierungs- und alternative Speicherregeln durch. Er löst keine Trigger-Regeln aus. Weitere Informationen finden Sie unter **tool::option_handling**.

object* forward *stub forward
Eine Weiterleitung zu **oo:objdefine [self] forward**

object* graft *stub forward
Weiterleitet die Methode an das durch *forward* angegebene Objekt oder den angegebenen Befehl

```
tool::object create A
tool::object create B
A graft buddy B
A configure color red
B configure color blue
A cget color
```

```
> red
A <buddy> cget color
> blue
```

Private Objektmethoden

***object* InitializePublic**

Überprüft die Metadaten der Klasse, um sicherzustellen, dass jedes Array, jede Option und jede Variable, die deklariert, aber nicht initialisiert wurde, mit dem Standardwert initialisiert wird. Diese Methode wird vom Konstruktor und von der "morph"-Methode aufgerufen. Ein mehrfacher Aufruf ist unbedenklich.

***object* Eval_Script ?script?**

Führt einen Textblock innerhalb des Namespaces des Objekts aus. Zeilen, die mit einem # beginnen, werden als Kommentare ignoriert. Befehle, die mit :: beginnen, werden als Aufruf eines globalen Befehls interpretiert. Alle anderen Tcl-Befehle, denen das Präfix "my" fehlt, erhalten dieses Präfix, damit das Skript interne Methoden ausführen kann. Diese Methode ist für Skripte der Konfiguration vorgesehen, in denen die Objektmethoden eine domänenspezifische Sprache interpretieren.

```
tool::class myclass {
  constructor script {
    my Eval_Script $script
  }
  method node {nodename info} {
    my variable node
    dict set node $nodename $info
  }
  method get {args} {
    my variable node
    return [dict get $node $args]
  }
}
myclass create movies {
  # This block of code is executed by the object
  node {The Day the Earth Stood Still} {
    date: 1952
    characters: {GORT Klatoo}
  }
}
movies get {The Day the Earth Stood Still} date:
> 1952
```

object* Option_Default *field

Berechnet den Standardwert für eine Option. Siehe **tool::option_handling**.

Autoren

Sean Woods

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *tcloo* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

TclOO

Copyright


```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```


tool_dict_ensemble

Stichwörter: tool_dict_ensemble, TOOL, TclOO

Name

tool::dict_ensemble – Dictionary-Tools

Übersicht

package require tool ?**0.4.2**?
object **ensemble add** *field*

Beschreibung

Der Befehl **dict_ensemble** ist ein Schlüsselwort, das durch tool hinzugefügt wurde. Er definiert eine öffentliche Variable (die als Dict gespeichert ist) sowie eine Zugriffsfunktion, mit der die in diesem Dict gespeicherten Werte bearbeitet und abgerufen werden können.

object ensemble* add *field

] *value Wert ...*] Fügt Elemente zu einer Liste hinzu, die über das *field* "leaf" des von diesem Ensemble verwalteten Dicts gepflegt wird. Deklariert eine Variable *name*, die als Array initialisiert wird und mit *contents* für Objekte dieser Klasse sowie für alle Objekte von Klassen, die von dieser Klasse abgeleitet sind, gefüllt wird.

Autoren

Sean Woods

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *tool* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Hilfsprogramme

Copyright

```
Copyright (c) 2015 Sean Woods <yoda@etoyoc.com>
```


transfer

connect
Stichwörter: connect, active, channel, connection, passive, secure, ssl, tls, transfer

Name
transfer::connect – Einrichtung der Verbindung

Übersicht
package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::connect ?**0.3**?
transfer::connect *objectName* ?*options...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **connect** *command*

Beschreibung
Dieses Paket stellt Objekte bereit, die über ausreichende Informationen verfügen, um entweder aktiv eine Verbindung zu einem Gegenstück herzustellen oder passiv auf eine Verbindung von diesem Gegenstück zu warten. Das heißt, jedes durch dieses Paket erstellte Objekt befindet sich stets in einem von zwei sich ergänzenden Modi, die als *aktiv* (das Objekt initiiert die Verbindung) und *passiv* (das Objekt empfängt die Verbindung) bezeichnet werden.

Von den beiden Objekten in einem Verbindungspaar muss eines für den *aktiven* Modus konfiguriert werden, das andere muss dann für den *passiven* Modus konfiguriert werden. Dadurch wird festgelegt, welcher der beiden Partner eine Verbindung zu wem herstellt (der *aktive* zum anderen) bzw. wer auf wen wartet (der *passive* auf den anderen). Hinweis: Dies ist völlig unabhängig von der Richtung einer Datenübertragung über die Verbindung, nachdem diese hergestellt wurde. Ein aktives Objekt kann nach dem Aufbau der Verbindung entweder Daten senden oder empfangen. Ebenso kann das passive Objekt dasselbe tun, nachdem das Warten auf seinen Partner beendet ist.

API

Paketbefehle
transfer::connect *objectName* ?*options*...?

Dieser Befehl erstellt ein neues Verbindungsobjekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und

Objektmethoden ausführlich erläutert. Die unterstützten *options* werden im Abschnitt Optionen erläutert.

Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace. Der vollständig qualifizierte Name des Objektbefehls wird als Ergebnis des Befehls zurückgegeben.

Objektbefehl

Alle mit dem Befehl **::transfer::connect** erstellten Objekte haben die folgende allgemeine Form:

***objectName* method ?*arg arg ...*?**

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt. Bei einem *aktiven* Objekt ist dies unbedenklich, sobald eine Verbindung hergestellt wurde, da die Rückruffunktion zur Fertigstellung synchron abläuft. Bei einem *passiven* Objekt, das derzeit darauf wartet, dass sein Partner die Verbindung herstellt, ist dies jedoch nicht sicher und führt später zu Fehlern, wenn der Verbindungsaufbau abgeschlossen ist und versucht wird, auf die nun fehlenden Datenstrukturen des zerstörten Objekts zuzugreifen.

objectName* connect *command

Diese Methode leitet den Verbindungsaufbau gemäß der Konfiguration des Objekts ein. Wenn die Verbindung hergestellt ist, wird die Rückruffunktion *command* mit einem zusätzlichen Argument aufgerufen, nämlich dem Kanal-Handle des Sockets, über den Daten übertragen werden können.

Das genaue Verhalten der Methode hängt vom konfigurierten Modus ab.

active

Der Verbindungsaufbau erfolgt synchron. Das Objekt wartet, bis die Verbindung hergestellt ist. Die Methode gibt als Ergebnis die leere Zeichenkette zurück.

passive

Der Verbindungsaufbau erfolgt asynchron. Die Methode gibt sofort zurück, sobald ein lauschenden Socket eingerichtet wurde. Die Verbindung wird im Hintergrund hergestellt. Die Methode gibt die Portnummer des lauschenden Sockets zurück, die vom Aufrufer verwendet werden kann. Eine wichtige Anwendung besteht darin, diese Information an die Gegenstelle weiterzugeben, damit diese weiß, mit welchem Port sie eine Verbindung herstellen muss.

Dies ist notwendig, da das Objekt möglicherweise für den Port **0** konfiguriert wurde, wodurch das Betriebssystem den tatsächlichen Port auswählen kann, auf dem es auf Verbindungen wartet.

Der Listening-Port wird sofort geschlossen, wenn die Verbindung durch die Gegenstelle hergestellt wurde, um das Zeitfenster zu minimieren, in dem sich auch Dritte mit dem Port verbinden könnten. Dennoch wird empfohlen, zusätzliche Maßnahmen im Protokoll

außerhalb des "connect"- und "transfer"-Objekts zu ergreifen, um sicherzustellen, dass eine Verbindung nicht mit einem nicht identifizierten/nicht autorisierten Partner hergestellt wird. Eine Möglichkeit hierfür ist die Verwendung von SSL/TLS. Informationen dazu finden Sie unter der Option **-socketcmd** und im Abschnitt Sichere Verbindungen.

Optionen

Verbindungsobjekte unterstützen die nachfolgend aufgeführten Optionen.

-mode *mode*

Diese Option legt den Modus fest, in dem sich das Objekt befindet. Sie ist optional und ist standardmäßig auf den Modus **active** eingestellt. Die beiden möglichen Modi sind:

active

In diesem Modus sind die beiden Optionen **-host** und **-port** relevant und legen den Host sowie den TCP-Port fest, mit dem das Objekt eine Verbindung herstellen muss. Der Host wird entweder durch einen Namen oder eine IP-Adresse angegeben.

passive

In diesem Modus ist die Option **-host** ohne Bedeutung und wird ignoriert, falls sie konfiguriert sein sollte. Die einzige Option, die das Objekt benötigt, ist **-port**; diese legt den TCP-Port fest, an dem der lauschenden Socket geöffnet wird, um auf die Verbindung vom Partner zu warten.

-host *hostname-or-ipaddr*

Diese Option legt den Host fest, mit dem im *aktiven* Modus eine Verbindung hergestellt werden soll, entweder anhand des Namens oder der IP-Adresse. Ein für den *passiven* Modus konfiguriertes Objekt ignoriert diese Option.

-port *int*

Im *aktiven* Modus legt diese Option den Port fest, mit dem das Objekt voraussichtlich eine Verbindung herstellen wird. Im *passiven* Modus hingegen handelt es sich um den Port, an dem das Objekt den lauschenden Socket erstellt, um auf eine Verbindung zu warten. Der Standardwert ist **0**, wodurch das Betriebssystem den tatsächlichen Port für den lauschenden Socket selbst auswählen kann.

-socketcmd *command*

Mit dieser Option kann der Benutzer festlegen, welcher Befehl zum Öffnen eines Sockets verwendet werden soll. Standardmäßig wird der integrierte Befehl **::socket** verwendet. Jeder mit diesem Befehl kompatible Befehl ist zulässig.

Die vorgesehene Hauptanwendung ist die Angabe von **tls::socket**. Das heißt, diese Option ermöglicht die Einrichtung sicherer Übertragungskanäle, ohne dass dieses Paket explizit vom Paket **tls** abhängig gemacht wird.

Siehe auch den Abschnitt Sichere Verbindungen.

-encoding encodingname

siehe unten ...

-eofchar eofspec

siehe unten ...

-translation transspec

Diese Optionen entsprechen denen, die vom integrierten Befehl **fconfigure** erkannt werden. Sie legen die Konfiguration fest, die für den Kanal zwischen den beiden Partnern nach dessen Aufbau, jedoch vor dem Aufruf der Rückruffunktion, eingestellt werden soll (siehe Methode **connect**).

Sichere Verbindungen

Eine Möglichkeit, die von Objekten dieses Pakets hergestellten Verbindungen zu sichern, besteht darin, das Paket **tls** vorzuschreiben und anschließend die Option **-socketcmd** so zu konfigurieren, dass die Verwendung des Befehls **tls::socket** zum Öffnen des Sockets erzwungen wird.

```
# Load and initialize tls
package require tls
tls::init -cafile /path/to/ca/cert -keyfile ...

# Create a connector with secure socket setup,
transfer::connect C -socketcmd tls::socket ...
...
```

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
package require tls
tls::init -tls1 1 ;# forcibly activate support for the TLS1
protocol

... your own application code ...
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *transfer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Transfermodul

Copyright

```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


copyops
Stichwörter: copyops, channel, copy, transfer

Name
transfer::copy – Grundlagen der Datenübertragung

Übersicht
package require Tcl 8.5 9
package require transfer::copy ?**0.3**?
transfer::copy::do chan|string *data outchannel* ?*options...*?
transfer::copy::chan *channel outchannel* ?*options...*?
transfer::copy::string *string outchannel* ?*options...*?
transfer::copy::doChan *channel outchannel optvar*
transfer::copy::doString *string outchannel optvar*
transfer::copy::options *outchannel optionlist optvar*

Beschreibung
Dieses Paket stellt eine Reihe von Befehlen für die asynchrone Verarbeitung von Informationen bereit, die entweder in einer Zeichenkette oder in einem Kanal enthalten sind. Der Hauptvorteil dieses Pakets besteht darin, dass die hier enthaltenen Befehle eine benutzerfreundlichere Rückruffunktion bieten als der integrierte Befehl **fcopy**, wodurch die Nutzung dieser Funktionen einfacher ist als die des integrierten Befehls.

API
transfer::copy::do chan|string *data outchannel* ?*options*...?
Dieser Befehl überträgt die in *data* enthaltenen Informationen gemäß den *outchannel* an den *options*. Der Typ der in *data* enthaltenen Informationen wird durch das erste Argument bestimmt.

Die für diesen Befehl verfügbaren Optionen entsprechen denen des Befehls **transfer::copy::options** und werden dort erläutert.

chan
Das Argument *data* enthält das Handle eines Kanals, und die tatsächlich zu übertragenden Informationen werden aus diesem Kanal gelesen.

string
Das Argument *data* enthält eine Zeichenkette, und dies sind die zu übertragenden Informationen.

transfer::copy::chan *channel outchannel* ?*options*...?
Dieser Befehl ist eine kürzere und direktere Form des Befehls **transfer::copy::do chan**.

transfer::copy::string *string outchannel* ?*options*...?
Dieser Befehl ist eine kürzere und direktere Form des Befehls **transfer::copy::do string**.

transfer::copy::doChan *channel outchannel optvar*
Dieser Befehl ist eine alternative Form von **transfer::copy::chan**, die ihre Optionen nicht aus einer Argumentliste variabler Länge, sondern aus der Array-Variablen mit dem Namen *optvar* ausliest.

transfer::copy::doString *string outchannel optvar*
Dieser Befehl ist eine alternative Form von **transfer::copy::string**, der seine Optionen nicht aus einer Argumentliste variabler Länge, sondern aus der durch *optvar* benannten Array-Variablen ausliest.

transfer::copy::options *outchannel optionlist optvar*
Dieser Befehl ist der Optionsprozessor, der von allen oben genannten Befehlen verwendet wird, welche ihre Optionen aus einer Argumentliste variabler Länge auslesen. Er liest zunächst die Standardeinstellungen aus dem Kanal-Handle *outchannel* aus, verarbeitet anschließend die Liste der Optionen in *optionlist* und speichert schließlich die Ergebnisse in der durch *optvar* benannten Array-Variablen. Der Inhalt dieser Variablen liegt in einem Format vor, das von allen oben genannten Befehlen, die ihre Optionen aus einer Array-Variablen lesen, direkt verstanden wird.

Die erkannten Optionen sind:

-blocksize *int*
Diese Option legt die Größe der Blöcke fest, die in einem Vorgang übertragen werden sollen. Sie ist optional und nimmt standardmäßig den Wert von **-buffersize** an, wie er für den Ausgabekanal konfiguriert ist.

Falls angegeben, muss ihr Wert eine ganze Zahl größer als Null sein.

-command *commandprefix*
Diese Option legt die Rückruffunktion für den Abschluss des Vorgangs fest. Diese Option muss angegeben werden. Andernfalls oder wenn als Argument eine leere Liste angegeben wurde, wird ein Fehler ausgelöst.

Ihr Wert muss ein Befehlspräfix sein, d. h. eine Liste, deren erstes Wort der auszuführende Befehl ist, gefolgt von Wörtern, die feste Argumente enthalten. Wenn die Rückruffunktion aufgerufen wird, werden dem Präfix ein oder zwei zusätzliche Argumente angehängt. Das erste Argument ist die Anzahl der übertragenen Bytes. Das optionale zweite Argument ist eine Fehlermeldung und wird genau dann hinzugefügt, wenn während der Übertragung ein Fehler aufgetreten ist.

-progress *commandprefix*
Diese Option legt die Fortschritts-Rückruffunktion für den Vorgang fest. Sie ist optional und ist standardmäßig auf eine leere Liste gesetzt. Diese letzte Möglichkeit signalisiert, dass keine Rückmeldung angefordert wurde, und deaktiviert diese Funktion.

Ihr Wert muss ein Befehlspräfix sein; siehe oben, **-command**, für eine ausführlichere Erläuterung. Wenn die Rückruffunktion aufgerufen wird, wird dem Präfix ein einziges

zusätzliches Argument angehängt. Dieses Argument gibt die Anzahl der Bytes an, die bisher übertragen wurden.

-size *int*
Diese Option legt die Anzahl der Bytes fest, die aus den Eingabedaten gelesen und übertragen werden sollen. Sie ist optional und standardmäßig auf "Alles übertragen" eingestellt. Ihr Wert muss eine ganze Zahl sein; jeder Wert kleiner als Null hat dieselbe Bedeutung, d. h., es werden alle verfügbaren Daten übertragen. Jeder andere Wert entspricht der Anzahl der zu übertragenden Bytes.

Alle Übertragungsbefehle lösen einen Fehler aus, wenn der Benutzer versucht, mehr Daten zu übertragen, als in der Eingabe verfügbar sind. Dies geschieht sofort, noch bevor die Übertragung tatsächlich beginnt, sofern es sich bei der Eingabe um eine Zeichenkette handelt. Andernfalls, d. h. bei einem Kanal als Eingabe, wird der Fehler in dem Moment ausgelöst, in dem die Unterlaufbedingung tatsächlich erkannt wird.

-encoding *encodingname*
siehe unten ...

-eofchar *eofspec*
siehe unten ...

-translation *transspec*
Diese Optionen entsprechen denen, die vom integrierten Befehl **fconfigure** erkannt werden, und legen die Einstellungen für den Ausgabekanal fest, die während der Übertragung – und ausschließlich während dieser – aktiv sein sollen. Das heißt, die Einstellungen des Ausgabekanals vor der Übertragung werden gespeichert und am Ende einer Übertragung wiederhergestellt, unabhängig davon, ob diese erfolgreich war oder fehlgeschlagen ist. Keine dieser Optionen ist erforderlich; sofern sie nicht angegeben werden, gelten standardmäßig die Einstellungen des Ausgabekanals.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *transfer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Transfermodul

Copyright


```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ddest

Stichwörter: ddest, channel, copy, data destination, transfer

Name
transfer::data::destination – Datenziel

Übersicht
package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::data::destination ?**0.3**?
transfer::data::destination *objectName* ?*options...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **put** *chunk*
objectName **done**
objectName **valid** *msgvar*
objectName **receive** *channel done*

Beschreibung
Dieses Paket enthält Objekte, die in erster Linie das Ziel einer Datenübertragung beschreiben. Sie sind zudem in der Lage, den Empfang von Informationen aus einem Kanal an das beschriebene Ziel zu initiieren.

API
transfer::data::destination *objectName* ?*options*...?
Dieser Befehl erstellt ein neues Datenzielobjekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Die unterstützten *options* werden im Abschnitt Optionen erläutert.

Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace. Der vollständig qualifizierte Name des Objektbefehls wird als Ergebnis des Befehls zurückgegeben.

Objektbefehl
Alle mit dem Befehl **::transfer::data::destination** erstellten Objekte haben die folgende allgemeine Form:

***objectName* method ?*arg arg ...*?**
Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden
***objectName* destroy**
Diese Methode löscht das Objekt. Wenn dies geschieht, während das Objekt gerade mit dem Empfang von Informationen aus einem Kanal beschäftigt ist, führt dies später zu Fehlern, sobald der Empfang abgeschlossen ist und versucht wird, auf die nun fehlenden Datenstrukturen des gelöschten Objekts zuzugreifen.

objectName* put *chunk

Die Hauptrezeptormethode. Speichert den empfangenen *chunk* am konfigurierten Speicherort. Sie muss für jedes empfangene Datenelement aufgerufen werden.

***objectName* done**
Die Sekundärrezeptor-Methode. Schließt den Empfänger ab. Sie muss aufgerufen werden, wenn der Empfangskanal "EOF" signalisiert. Danach können weder diese Methode noch die Methode **put** mehr aufgerufen werden.

objectName* valid *msgvar
Diese Methode überprüft die Konfiguration des Objekts auf Gültigkeit. Sie gibt einen booleschen Wert zurück, dessen Wert "**True**" lautet, wenn das Objekt gültig ist, und andernfalls "**False**". Im letzteren Fall wird die Variable, deren Name in *msgvar* gespeichert ist, auf eine Fehlermeldung gesetzt, die das bei der Konfiguration festgestellte Problem beschreibt. Andernfalls bleibt diese Variable unverändert.

objectName* receive *channel done
Diese Methode leitet den Empfang von Daten aus dem angegebenen *channel* ein. Die empfangenen Daten werden über Aufrufe der Methoden **put** und **done** im konfigurierten Ziel gespeichert. Wenn der Empfang abgeschlossen ist, wird das Befehlspräfix *done* aufgerufen, wobei die Anzahl der empfangenen Zeichen als einziges zusätzliches Argument angehängt wird.

Optionen

Alle Datenziele unterstützen die unten aufgeführten Optionen. Zu beachten ist, dass alle Optionen halb-exklusiv sind und jeweils einen anderen Zieltyp sowie zugehörige Informationen festlegen. Werden diese Optionen mehrmals angegeben, wird die zuletzt angegebene Option zur tatsächlichen Konfiguration des Objekts herangezogen.

-channel *handle*
Diese Option legt fest, dass das Ziel der Daten ein Kanal ist, und das zugehörige Argument ist das Handle des Kanals, in den die empfangenen Daten geschrieben werden sollen.

-file *path*
Diese Option legt fest, dass das Ziel der Daten eine Datei ist, und das zugehörige Argument ist der Pfad der Datei, in die die empfangenen Daten geschrieben werden sollen.

-variable *varname*
Diese Option legt fest, dass das Ziel der Daten eine Variable ist, und das zugehörige Argument enthält den Namen der Variable, in die die empfangenen Daten geschrieben werden sollen. Es wird davon ausgegangen, dass es sich um eine globale Variable oder eine Variable im Namespace handelt, die im globalen Namespace verankert ist.

-progress *command*
Wenn diese Option festgelegt ist, wird damit ein Befehl definiert, der für jeden empfangenen Byte-Block aufgerufen wird, sodass der Benutzer den Fortschritt des Datenempfangs überwachen kann. Die Rückruffunktion wird stets mit einem zusätzlichen Argument aufgerufen, nämlich der Anzahl der bisher empfangenen Bytes.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *transfer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Transfermodul

Copyright

```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


dsource
Stichwörter: dsource, channel, copy, data source, transfer

Name
transfer::data::source – Datenquelle

Übersicht
package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::copy ?**0.3**?
package require transfer::data::source ?**0.3**?
transfer::data::source *objectName* ?*options...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **type**
objectName **data**
objectName **size**
objectName **valid** *msgvar*
objectName **transmit** *channel blocksize done*

Beschreibung
Dieses Paket enthält Objekte, die in erster Linie die Herkunft bestimmter zu übertragender Daten beschreiben. Sie sind zudem in der Lage, mithilfe des Basis-Pakets **transfer::copy** die Übertragung der beschriebenen Informationen an einen Kanal zu initiieren.

API

Paketbefehle
transfer::data::source *objectName* ?*options*...?

Dieser Befehl erstellt ein neues Datenquellenobjekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Die unterstützten *options* werden im Abschnitt Optionen erläutert.

Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig

qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace. Der vollständig qualifizierte Name des Objektbefehls wird als Ergebnis des Befehls zurückgegeben.

Objektbefehl

Alle mit dem Befehl **::transfer::data::source** erstellten Objekte haben die folgende allgemeine Form:

***objectName* method ?*arg arg ...*?**

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt. Dies ist unbedenklich, solange eine vom Objekt initiierte Übertragung aktiv ist, da alle für die Übertragung erforderlichen Daten bereits kopiert wurden und nach Abschluss der Übertragung kein Zugriff mehr auf das initiierende Objekt erfolgt. Das heißt, die Übertragung ist vollständig vom Quellobjekt selbst getrennt.

***objectName* type**

Diese Methode gibt eine Zeichenkette zurück, die den Typ der Daten beschreibt, auf die sich das Objekt bezieht. Die möglichen Werte und ihre Bedeutungen lauten:

undefined

Es wurden entweder gar keine Daten angegeben oder diese waren unvollständig. Das Objekt kennt den Typ nicht.

string

Die zu übertragenden Daten sind in einer Zeichenkette enthalten.

channel

Die zu übertragenden Daten sind in einem Kanal enthalten.

***objectName* data**

Diese Methode gibt je nach Typ der Daten, auf die sich das Objekt bezieht, einen Wert zurück, über den auf die Daten zugegriffen werden kann. Die Methode löst einen Fehler aus, wenn der Typ **undefined** ist. Beim Typ **string** gibt es als Rückgabewert die Daten selbst, während beim Typ **channel** das Rückgabewert das Handle des Kanals ist, der die Daten enthält.

***objectName* size**

Diese Methode gibt einen Wert zurück, der vom Typ der Daten, auf die sich das Objekt bezieht, sowie von der Größe dieser Daten abhängt. Die Methode löst einen Fehler aus, wenn der Typ "**undefined**" ist. Die Rückgabe eines negativen Werts signalisiert, dass das Objekt die absolute Größe nicht im Voraus ermitteln kann (wie beispielsweise bei Daten in einem Kanal).

objectName* valid *msgvar

Diese Methode überprüft die Konfiguration des Objekts auf Gültigkeit. Sie gibt einen booleschen Wert zurück, dessen Wert "**True**" lautet, wenn das Objekt gültig ist, und andernfalls "**False**". Im letzteren Fall wird die Variable, deren Name in *msgvar* gespeichert ist, auf eine Fehlermeldung gesetzt, die das bei der Konfiguration festgestellte Problem beschreibt. Andernfalls bleibt diese Variable unverändert.

objectName* transmit *channel blocksize done

Diese Methode leitet eine Übertragung der referenzierten Daten an den angegebenen

channel ein. Wenn die Übertragung abgeschlossen ist, wird das Befehlspräfix *done* aufgerufen, gemäß den Regeln für die Option **-command** des Befehls transfer::copy::do im Paket **transfer::copy**. Die Angabe *blocksize* legt die Größe der Blöcke fest, die in einem Durchgang übertragen werden sollen. Siehe dazu die Option **-blocksize** des Befehls transfer::copy::do im Paket **transfer::copy**.

Optionen

Alle Datenquellen unterstützen die unten aufgeführten Optionen. Zu beachten ist, dass die ersten vier Optionen sich teilweise gegenseitig ausschließen, da jede eine andere Art von Datenquelle und zugehörigen Inhalt festlegt. Werden diese Optionen mehrmals angegeben, wird die zuletzt angegebene Option zur tatsächlichen Konfiguration des Objekts herangezogen.

-string *text*
Diese Option legt fest, dass die Datenquelle eine direkte Zeichenkette ist und das zugehörige Argument die betreffende Zeichenkette enthält.

-channel *handle*
Diese Option legt fest, dass die Datenquelle ein Kanal ist, und das zugehörige Argument ist das Handle des Kanals, der die Daten enthält.

-file *path*
Diese Option legt fest, dass die Daten aus einer Datei stammen, und das zugehörige Argument ist der Pfad zu der Datei, die die Daten enthält.

-variable *varname*
Diese Option legt fest, dass die Datenquelle eine in einer Variablen gespeicherte Zeichenkette ist und das zugehörige Argument den Namen der betreffenden Variablen enthält. Es wird davon ausgegangen, dass es sich um eine globale Variable oder eine Variable im Namespace handelt, die im globalen Namespace verankert ist.

-size *int*
Diese Option legt die Größe der Datenübertragung fest. Sie ist optional und hat standardmäßig den Wert -1. Dieser Wert sowie jeder andere Wert kleiner als Null bewirkt, dass alle Daten aus der Quelle übertragen werden.

-progress *command*
Wenn diese Option festgelegt ist, wird damit ein Befehl definiert, der für jeden übertragenen Byte-Block aufgerufen wird, sodass der Benutzer den Fortschritt der Datenübertragung überwachen kann. Die Rückruffunktion wird stets mit einem zusätzlichen Argument aufgerufen, nämlich der Anzahl der bisher übertragenen Bytes.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *transfer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Transfermodul

Copyright


```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


receiver

Stichwörter: receiver, channel, copy, data destination, secure, ssl, tls, transfer

Name

transfer::receiver – Datenquelle

Übersicht

package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::data::destination ?**0.3**?
package require transfer::connect ?**0.3**?
package require transfer::receiver ?**0.3**?
transfer::receiver *object* ?*options...*?
transfer::receiver stream channel *chan host port* ?*arg...*?
transfer::receiver stream file *path host port* ?*arg...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **start**
objectName **busy**

Beschreibung

Dieses Paket fasst Datenziele und den Verbindungsaufbau in einem gemeinsamen Objekt zusammen, um über einen Socket eingehende Informationen zu empfangen. Diese Objekte unterstützen alle Optionen der Objekte, die von den Paketen transfer::data::destination und transfer::connect erstellt wurden.

API

Paketbefehle

transfer::receiver *object* ?*options*...?

Dieser Befehl erstellt ein neues Empfängerobjekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Die unterstützten *options* werden im Abschnitt Optionen erläutert.

Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace. Der vollständig qualifizierte Name des Objektbefehls wird als Ergebnis des Befehls zurückgegeben.

transfer::receiver stream channel *chan host port* ?*arg*...?

Diese Methode erstellt eine "Fire-and-Forget"-Übertragung für die Daten, die von der Quelle unter Host/Port (Details siehe unten) stammen und in den Kanal *chan* geschrieben werden,

beginnend an der aktuellen Suchposition. Der Kanal ist so konfiguriert, dass er für die Übertragung binäre Übersetzung und Kodierung verwendet. Der Kanal wird nach Abschluss der Übertragung *nicht* geschlossen. Dies wird der Rückruffunktion überlassen.

Werden sowohl *host* als auch *port* angegeben, wird eine **active**-Verbindung zur Datenquelle hergestellt. Wenn nur ein *port* festgelegt wird (wobei *host* die leere Zeichenkette ist), wird stattdessen eine **passive**-Verbindung hergestellt, d. h., der Empfänger wartet dann auf eine Verbindung durch den Sender.

Alle Argumente nach dem Port werden als Optionen behandelt und dienen zur Konfiguration des internen Empfängerobjekts. Eine Liste der unterstützten Optionen und deren Bedeutung finden Sie im Abschnitt Optionen. *Hinweis*: Die Signatur des für die **-command** -Rückruffunktion angegebenen Befehlspräfixes unterscheidet sich von der Signatur für dieselbe Option des Empfängerobjekts. Diese Rückruffunktion erhält lediglich die Anzahl der Bytes und die übertragene Datenmenge sowie gegebenenfalls eine Fehlermeldung. Es wird kein Verweis auf das intern verwendete Empfängerobjekt hergestellt.

Das vom Befehl zurückgegebene Ergebnis ist die leere Zeichenkette, wenn der Befehl so eingestellt wurde, dass eine *aktive* Verbindung hergestellt wird, und andernfalls der Port, auf dem das interne Empfängerobjekt lauscht, d. h. wenn es so konfiguriert ist, dass es *passiv* eine Verbindung herstellt. Siehe auch das Paket transfer::connect und die Beschreibung der Methode **connect**, um zu erfahren, woher dieses Verhalten stammt.

transfer::receiver stream file *path host port* ?*arg*...?

Diese Methode entspricht der Methode **stream channel**, mit dem Unterschied, dass die empfangenen Daten in den Pfad *path* geschrieben werden und dabei den bisherigen Inhalt ersetzen.

Objektbefehl

Alle mit dem Befehl **::transfer::receiver** erstellten Objekte haben die folgende allgemeine Form:

***objectName* method ?*arg arg ...*?**

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt. Wenn Sie dies tun, während ein Empfang noch läuft, kommt es später zu Fehlern, sobald der Empfang abgeschlossen ist und versucht wird, auf die nun fehlenden Datenstrukturen des gelöschten Objekts zuzugreifen.

***objectName* start**

Diese Methode leitet den Datenempfang ein, indem sie zunächst die Verbindung herstellt und anschließend die empfangenen Informationen in das Ziel kopiert. Die Methode löst einen Fehler aus, wenn bereits ein Empfang läuft oder noch nicht abgeschlossen ist. Das heißt, es ist nicht möglich, zwei Empfänge parallel auszuführen, sondern nur nacheinander. Fehler werden ebenfalls ausgelöst, wenn die Konfiguration des Datenziels ungültig ist oder wenn keine Rückruffunktion für den Abschluss angegeben wurde.

Das von der Methode zurückgegebene Ergebnis ist die leere Zeichenkette für ein Objekt, das für eine *aktive* Verbindung konfiguriert ist, und andernfalls der Port, auf dem das Objekt auf Verbindungen wartet, d. h. wenn es für eine *passive* Verbindung konfiguriert ist. Siehe auch das Paket transfer::connect und die Beschreibung der Methode **connect**, um zu erfahren, woher dieses Verhalten stammt.

***objectName* busy**

Diese Methode gibt einen booleschen Wert zurück, der angibt, ob gerade ein Empfang stattfindet (**True**) oder nicht (**False**).

Optionen

Alle Empfängerobjekte unterstützen die Vereinigung der Optionen, die von ihren "connect"- und "data destination"-Komponenten unterstützt werden, sowie eine eigene Option. Siehe auch die Dokumentation zu den Paketen transfer::data::destination und transfer::connect.

-command *cmdprefix*

Diese Option legt den Befehl fest, der aufgerufen werden soll, wenn der Empfang der Informationen abgeschlossen ist. Die diesem Befehl übergebenen Argumente entsprechen denen, die an den Abschluss-Callback des Befehls transfer::copy::do übergeben werden, der vom Paket **transfer::copy** bereitgestellt wird.

-mode *mode*

Diese Option legt den Modus fest, in dem sich das Objekt befindet. Sie ist optional und ist standardmäßig auf den Modus **active** eingestellt. Die beiden möglichen Modi sind:

active

In diesem Modus sind die beiden Optionen **-host** und **-port** relevant und legen den Host sowie den TCP-Port fest, mit dem das Objekt eine Verbindung herstellen muss. Der Host wird entweder durch einen Namen oder eine IP-Adresse angegeben.

passive

In diesem Modus ist die Option **-host** ohne Bedeutung und wird ignoriert, falls sie konfiguriert sein sollte. Die einzige Option, die das Objekt benötigt, ist **-port**; diese legt den TCP-Port fest, an dem der lauschenden Socket geöffnet wird, um auf die Verbindung vom Partner zu warten.

-host *hostname-or-ipaddr*

Diese Option legt den Host fest, mit dem im *aktiven* Modus eine Verbindung hergestellt werden soll, entweder anhand des Namens oder der IP-Adresse. Ein für den *passiven* Modus konfiguriertes Objekt ignoriert diese Option.

-port *int*

Im *aktiven* Modus legt diese Option den Port fest, mit dem das Objekt voraussichtlich eine Verbindung herstellen wird. Im *passiven* Modus hingegen handelt es sich um den Port, an dem das Objekt den lauschenden Socket erstellt, um auf eine Verbindung zu warten. Der Standardwert ist **0**, wodurch das Betriebssystem den tatsächlichen Port für den lauschenden Socket selbst auswählen kann.

-socketcmd *command*

Mit dieser Option kann der Benutzer festlegen, welcher Befehl zum Öffnen eines Sockets verwendet werden soll. Standardmäßig wird der integrierte Befehl **::socket** verwendet. Jeder mit diesem Befehl kompatible Befehl ist zulässig.

Die vorgesehene Hauptanwendung ist die Angabe von **tls::socket**. Das heißt, diese Option ermöglicht die Einrichtung sicherer Übertragungskanäle, ohne dass dieses Paket explizit vom Paket **tls** abhängig gemacht wird.

Siehe auch den Abschnitt Sichere Verbindungen.

-encoding encodingname
siehe unten ...

-eofchar eofspec
siehe unten ...

-translation transspec
Diese Optionen entsprechen denen, die vom integrierten Befehl **fconfigure** erkannt werden. Sie legen die Konfiguration fest, die für den Kanal zwischen den beiden Partnern nach dessen Aufbau, jedoch vor dem Aufruf der Rückruffunktion, eingestellt werden soll (siehe Methode **connect**).

-channel *handle*
Diese Option legt fest, dass das Ziel der Daten ein Kanal ist, und das zugehörige Argument ist das Handle des Kanals, in den die empfangenen Daten geschrieben werden sollen.

-file *path*
Diese Option legt fest, dass das Ziel der Daten eine Datei ist, und das zugehörige Argument ist der Pfad der Datei, in die die empfangenen Daten geschrieben werden sollen.

-variable *varname*
Diese Option legt fest, dass das Ziel der Daten eine Variable ist, und das zugehörige Argument enthält den Namen der Variable, in die die empfangenen Daten geschrieben werden sollen. Es wird davon ausgegangen, dass es sich um eine globale Variable oder eine Variable im Namespace handelt, die im globalen Namespace verankert ist.

-progress *command*
Wenn diese Option festgelegt ist, wird damit ein Befehl definiert, der für jeden empfangenen Byte-Block aufgerufen wird, sodass der Benutzer den Fortschritt des Datenempfangs überwachen kann. Die Rückruffunktion wird stets mit einem zusätzlichen Argument aufgerufen, nämlich der Anzahl der bisher empfangenen Bytes.

Sichere Verbindungen

Eine Möglichkeit, die von Objekten dieses Pakets hergestellten Verbindungen zu sichern, besteht darin, das Paket **tls** vorzuschreiben und anschließend die Option **-socketcmd** so zu konfigurieren, dass die Verwendung des Befehls **tls::socket** zum Öffnen des Sockets erzwungen wird.

```
    # Load and initialize tls
    package require tls
    tls::init -cafile /path/to/ca/cert -keyfile ...

    # Create a connector with secure socket setup,
    transfer::receiver R -socketcmd tls::socket ...
    ...
```

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *transfer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Transfermodul

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tqueue

Stichwörter: tqueue, channel, copy, queue, transfer

Name

transfer::copy::queue – In der Warteschlange befindliche Überweisungen

Übersicht

package require Tcl 8.5 9
package require snit ?**1.0**?

package require struct::queue ?**1.4**?
package require transfer::copy ?**0.3**?
package require transfer::copy::queue ?**0.2**?
transfer::copy::queue *objectName outchannel* ?*options...*?
objectName **method** ?*arg arg ...*?
objectName **destroy**
objectName **busy**
objectName **pending**
objectName **put** *request*

Beschreibung

Dieses Paket stellt Objekte bereit, die Übertragungsanforderungen für einen einzelnen Kanal mithilfe einer FIFO-Warteschlange serialisieren. Die gesammelten Anforderungen werden in der Reihenfolge ihres Eingangs ausgeführt, wobei in der Regel die erste Anforderung, die ein freies Objekt erreicht, die Ausführung einleitet. Neue Anforderungen können hinzugefügt werden, während das Objekt aktiv ist, und werden zurückgestellt, bis alle zuvor eingegangenen Anforderungen erfolgreich abgeschlossen wurden.

Wenn eine Anfrage einen Übertragungsfehler verursacht, wird die Ausführung angehalten, und alle nachfolgenden Anfragen werden nicht bearbeitet. Derzeit bedeutet dies, dass deren Rückruffunktionen zur Abschlussbestätigung überhaupt nicht ausgelöst werden.

HINWEIS: Ich bin mir nicht ganz sicher, ob es sinnvoll ist, die Rückruffunktionen zur Fertigstellung der nicht bearbeiteten Anfragen nach einem Fehler, der das Warteschlangenobjekt stoppt, nicht auszulösen. Dies zwingt den Nutzer der Warteschlange dazu, sich ebenfalls an die Rückruffunktionen zu erinnern und diese auszuführen. Denn andernfalls würde alles im System, was auf eine Benachrichtigung über den Status einer Anfrage angewiesen ist, in der Schwebe bleiben. Ich komme langsam zu der Überzeugung, dass es sinnvoller ist, die entsprechenden Abschluss-Callbacks mit einer Fehlermeldung über den Abbruch der Warteschlange und einer übertragenen Datenmenge von 0 Byte auszulösen.

Alle Transferanfragen haben folgende Form:

```
    {type data options...}
```

wobei *type* zu {**chan**, **string**} gehört und *data* die zu übertragenden Informationen festlegt. Bei **chan** handelt es sich bei *data* um das Handle des Kanals, der die tatsächlich zu übertragenden Informationen enthält, während *data* bei **string** direkt die zu übertragenden Informationen enthält. Die *options* sind eine Liste dieser Optionen und ihrer Werte und entsprechen denen, die von den Low-Level-Kopieroperationen des Pakets **transfer::copy** akzeptiert werden. Beachten Sie, wie sich die Anfrage allein dadurch, dass man ihr "transfer::copy::do" voranstellt und ein Kanal-Handle zwischen *data* und *options* einfügt, auf einfache Weise von einer reinen Datenstruktur in einen Befehl verwandelt, dessen Auswertung die Anfrage ausführt.

API

Paketbefehle

transfer::copy::queue *objectName outchannel* ?*options*...?

Dieser Befehl erstellt ein neues Warteschlangenobjekt für die Verwaltung des Kanals *outchannel* mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*

-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Die unterstützten *options* werden im Abschnitt Optionen erläutert.

Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace. Der vollständig qualifizierte Name des Objektbefehls wird als Ergebnis des Befehls zurückgegeben.

Objektbefehl

Alle mit dem Befehl **::transfer::copy::queue** erstellten Objekte haben die folgende allgemeine Form:

***objectName* method ?*arg arg ...*?**

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt. Wenn Sie dies tun, während das Objekt gerade beschäftigt ist, kommt es später zu Fehlern, wenn die aktuell ausgeführte Anfrage abgeschlossen ist und versucht, auf die nun fehlenden Datenstrukturen des gelöschten Objekts zuzugreifen.

***objectName* busy**

Diese Methode gibt einen booleschen Wert zurück, der angibt, ob das Objekt derzeit eine Anfrage bearbeitet (d. h. *busy*, Wert **True**) oder nicht (d. h. *im Leerlauf*, Wert **False**).

***objectName* pending**

Diese Methode gibt die Anzahl der Anfragen zurück, die derzeit in der Warteschlange auf ihre Ausführung warten. Eine Anfrage, die gerade bearbeitet wird, wird nicht als wartend gezählt.

objectName* put *request

Diese Methode fügt die Übertragung *request* in die Warteschlange der ausstehenden Anforderungen des Objekts ein. Wenn das Objekt *idle* ist, wird es *busy* und bearbeitet die Anforderung unverzüglich. Andernfalls wird die Bearbeitung der neuen Anforderung zurückgestellt, bis alle vorhergehenden Anforderungen bearbeitet wurden.

Optionen

Die einzige bekannte Option ist **-on-status-change**. Sie ist optional und ist standardmäßig auf die leere Liste gesetzt, wodurch die Meldung von Statusänderungen deaktiviert wird. Andernfalls ist ihr Argument ein Befehlspräfix, das immer dann aufgerufen wird, wenn sich der interne Status des Objekts ändert. Die Rückruffunktion wird mit zwei zusätzlichen Argumenten aufgerufen, nämlich den Ergebnissen der Methoden **pending** und **busy**, in dieser Reihenfolge. Auf diese Weise kann jeder Benutzer beispielsweise leicht erkennen, wenn das Objekt alle ausstehenden Anfragen verarbeitet hat.

Verwendung

Eine mögliche Anwendung dieses Pakets und dieser Klasse liegt in einem HTTP-1.1-Server, der die Ergebnisse verwaltet, die auf die Übertragung an den Client warten.

Beachten Sie, dass das System in dieser Anwendung zudem eine zusätzliche Datenstruktur benötigt, die noch ausstehende Ergebnisse erfasst, da diese möglicherweise in einer anderen Reihenfolge zurückkommen als die Anfragen des Clients, und diese in der richtigen Reihenfolge

an die eigentliche Warteschlange übergibt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *transfer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Transfermodul

Copyright


```
Copyright (c) 2006 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


transmitter

Stichwörter: transmitter, channel, copy, data source, secure, ssl, tls, transfer

Name

transfer::transmitter – Datenquelle

Übersicht

package require Tcl 8.5 9
package require snit ?**1.0**?
package require transfer::copy ?**0.3**?
package require transfer::data::source ?**0.3**?
package require transfer::connect ?**0.3**?
package require transfer::transmitter ?**0.3**?
transfer::transmitter *objectName* ?*options...*?
transfer::transmitter stream channel *chan host port* ?*arg...*?
transfer::transmitter stream file *path host port* ?*arg...*?
objectName **method** ?*arg arg* ...?
objectName **destroy**
objectName **start**
objectName **busy**

Beschreibung

Dieses Paket fasst Datenquellen und Verbindungseinstellungen in einem gemeinsamen Objekt zusammen, um Informationen über einen Socket zu übertragen. Diese Objekte unterstützen alle Optionen der Objekte, die von den Paketen transfer::data::source und transfer::connect erstellt wurden.

API

Paketbefehle

transfer::transmitter *objectName* ?*options*...?

Dieser Befehl erstellt ein neues Senderobjekt mit einem zugehörigen Tcl-Befehl, dessen Name *objectName* lautet. Dieser *object*-Befehl wird in den Abschnitten Objektbefehl und Objektmethoden ausführlich erläutert. Die unterstützten *options* werden im Abschnitt Optionen erläutert.

Der Objektbefehl wird im aktuellen Namespace erstellt, wenn *objectName* nicht vollständig qualifiziert ist; andernfalls erfolgt die Erstellung im angegebenen Namespace. Der vollständig qualifizierte Name des Objektbefehls wird als Ergebnis des Befehls zurückgegeben.

transfer::transmitter stream channel *chan host port* ?*arg*...?

Diese Methode erstellt eine "Fire-and-Forget"-Übertragung für die im Kanal *chan* enthaltenen Daten, beginnend an der aktuellen Suchposition. Der Kanal ist so konfiguriert, dass für die Übertragung eine binäre Übersetzung und Kodierung verwendet wird. Der Kanal wird automatisch geschlossen, wenn die Übertragung abgeschlossen ist.

Wenn sowohl *host* als auch *port* angegeben werden, wird eine **active**-Verbindung zum Ziel hergestellt. Wenn nur ein *port* festgelegt wird (wobei *host* die leere Zeichenkette ist), wird stattdessen eine **passive**-Verbindung hergestellt.

Alle Argumente nach dem Port werden als Optionen behandelt und dienen zur Konfiguration des internen Senderobjekts. Eine Liste der unterstützten Optionen und deren Bedeutung finden Sie im Abschnitt Optionen. *Beachten* Sie jedoch, dass sich die Signatur des für die **-command**-Rückruffunktion angegebene Befehlspräfixes von der Signatur derselben Option des Senderobjekts unterscheidet. Diese Rückruffunktion wird lediglich mit der Anzahl der Bytes und der übertragenen Datenmenge sowie gegebenenfalls einer Fehlermeldung übergeben. Es wird kein Verweis auf das intern verwendete Senderobjekt hergestellt.

Das vom Befehl zurückgegebene Ergebnis ist die leere Zeichenkette, wenn dieser so eingestellt war, dass eine *aktive* Verbindung hergestellt wird, und andernfalls der Port, auf dem das interne Senderobjekt auf Verbindungen wartet, d. h. wenn es so konfiguriert ist, dass es *passiv* eine Verbindung herstellt. Siehe auch das Paket transfer::connect und die Beschreibung der Methode **connect**, um zu erfahren, woher dieses Verhalten stammt.

transfer::transmitter stream file *path host port* ?*arg*...?

Diese Methode entspricht der Methode **stream channel**, mit dem Unterschied, dass die in der Datei mit dem Pfad *path* enthaltenen Daten übertragen werden.

Objektbefehl

Alle mit dem Befehl **::transfer::transmitter** erstellten Objekte haben die folgende allgemeine Form:

***objectName* method ?*arg arg ...*?**

Die Methode **method** und ihre *arg*umente bestimmen das genaue Verhalten des Befehls. Detaillierte Angaben finden Sie im Abschnitt Objektmethoden.

Objektmethoden

***objectName* destroy**

Diese Methode löscht das Objekt. Wenn Sie dies tun, während eine Übertragung läuft, führt dies später zu Fehlern, sobald die Übertragung abgeschlossen ist und versucht wird, auf die nun fehlenden Datenstrukturen des gelöschten Objekts zuzugreifen.

***objectName* start**

Diese Methode leitet die Datenübertragung ein, indem sie zunächst die Verbindung herstellt und anschließend die Informationen kopiert. Die Methode löst einen Fehler aus, wenn bereits eine Übertragung im Gange ist oder noch andauert. Das heißt, es ist nicht möglich, zwei Übertragungen parallel für ein einzelnes Objekt durchzuführen, sondern nur nacheinander. Für parallele Übertragungen sind mehrere Senderobjekte erforderlich, eines pro Übertragung. Fehler werden ebenfalls ausgelöst, wenn die Konfiguration der Datenquelle ungültig ist oder wenn keine Rückruffunktion für den Abschluss festgelegt wurde.

Das von der Methode zurückgegebene Ergebnis ist die leere Zeichenkette für ein Objekt, das für den Aufbau einer *aktiven* Verbindung konfiguriert ist, und andernfalls der Port, auf dem das Objekt auf Verbindungen wartet, d. h. wenn es für eine *passive* Verbindung konfiguriert ist. Siehe auch das Paket transfer::connect und die Beschreibung der Methode **connect**, um zu erfahren, woher dieses Verhalten stammt.

***objectName* busy**

Diese Methode gibt einen booleschen Wert zurück, der angibt, ob gerade eine Übertragung stattfindet (**True**) oder nicht (**False**).

Optionen

Alle Senderobjekte unterstützen die Vereinigung der Optionen, die von ihren "connect"- und "data source"-Komponenten unterstützt werden, sowie zwei eigene Optionen. Siehe auch die Dokumentation zu den Paketen transfer::data::source und transfer::connect.

-blocksize *int*

Diese Option legt die Größe der Chunks fest, die in einem Block übertragen werden sollen. Die Verwendung ist optional; der Standardwert lautet **1024**.

-command *cmdprefix*

Diese Option legt den Befehl fest, der aufgerufen werden soll, wenn die Übertragung der Informationen abgeschlossen ist. Die diesem Befehl übergebenen Argumente entsprechen denen, die an die Rückruffunktion des Befehls transfer::copy::do übergeben werden, die vom Paket **transfer::copy** bereitgestellt wird.

-mode *mode*

Diese Option legt den Modus fest, in dem sich das Objekt befindet. Sie ist optional und ist standardmäßig auf den Modus **active** eingestellt. Die beiden möglichen Modi sind:

active

In diesem Modus sind die beiden Optionen **-host** und **-port** relevant und legen den Host sowie den TCP-Port fest, mit dem das Objekt eine Verbindung herstellen muss. Der Host wird entweder durch einen Namen oder eine IP-Adresse angegeben.

passive

In diesem Modus ist die Option **-host** ohne Bedeutung und wird ignoriert, falls sie konfiguriert sein sollte. Die einzige Option, die das Objekt benötigt, ist **-port**; diese legt den TCP-Port fest, an dem der lauschenden Socket geöffnet wird, um auf die Verbindung vom Partner zu warten.

-host *hostname-or-ipaddr*
Diese Option legt den Host fest, mit dem im *aktiven* Modus eine Verbindung hergestellt werden soll, entweder anhand des Namens oder der IP-Adresse. Ein für den *passiven* Modus konfiguriertes Objekt ignoriert diese Option.

-port *int*
Im *aktiven* Modus legt diese Option den Port fest, mit dem das Objekt voraussichtlich eine Verbindung herstellen wird. Im *passiven* Modus hingegen handelt es sich um den Port, an dem das Objekt den lauschenden Socket erstellt, um auf eine Verbindung zu warten. Der Standardwert ist **0**, wodurch das Betriebssystem den tatsächlichen Port für den lauschenden Socket selbst auswählen kann.

-socketcmd *command*
Mit dieser Option kann der Benutzer festlegen, welcher Befehl zum Öffnen eines Sockets verwendet werden soll. Standardmäßig wird der integrierte Befehl **::socket** verwendet. Jeder mit diesem Befehl kompatible Befehl ist zulässig.

Die vorgesehene Hauptanwendung ist die Angabe von **tls::socket**. Das heißt, diese Option ermöglicht die Einrichtung sicherer Übertragungskanäle, ohne dass dieses Paket explizit vom Paket **tls** abhängig gemacht wird.

Siehe auch den Abschnitt Sichere Verbindungen.

-encoding encodingname
siehe unten ...

-eofchar eofspec
siehe unten ...

-translation transspec
Diese Optionen entsprechen denen, die vom integrierten Befehl **fconfigure** erkannt werden. Sie legen die Konfiguration fest, die für den Kanal zwischen den beiden Partnern nach dessen Aufbau, jedoch vor dem Aufruf der Rückruffunktion, eingestellt werden soll (siehe Methode **connect**).

-string *text*
Diese Option legt fest, dass die Datenquelle eine direkte Zeichenkette ist und das zugehörige Argument die betreffende Zeichenkette enthält.

-channel *handle*
Diese Option legt fest, dass die Datenquelle ein Kanal ist, und das zugehörige Argument ist das Handle des Kanals, der die Daten enthält.

-file *path*
Diese Option legt fest, dass die Daten aus einer Datei stammen, und das zugehörige Argument ist der Pfad zu der Datei, die die Daten enthält.

-variable *varname*
Diese Option legt fest, dass die Datenquelle eine in einer Variablen gespeicherte Zeichenkette ist und das zugehörige Argument den Namen der betreffenden Variablen enthält. Es wird davon ausgegangen, dass es sich um eine globale Variable oder eine Variable im Namespace handelt, die im globalen Namespace verankert ist.

-size *int*
Diese Option legt die Größe der Datenübertragung fest. Sie ist optional und hat

standardmäßig den Wert -1. Dieser Wert sowie jeder andere Wert kleiner als Null bewirkt, dass alle Daten aus der Quelle übertragen werden.

-progress *command*
Wenn diese Option festgelegt ist, wird damit ein Befehl definiert, der für jeden übertragenen Byte-Block aufgerufen wird, sodass der Benutzer den Fortschritt der Datenübertragung überwachen kann. Die Rückruffunktion wird stets mit einem zusätzlichen Argument aufgerufen, nämlich der Anzahl der bisher übertragenen Bytes.

Sichere Verbindungen

Eine Möglichkeit, die von Objekten dieses Pakets hergestellten Verbindungen zu sichern, besteht darin, das Paket **tls** vorzuschreiben und anschließend die Option **-socketcmd** so zu konfigurieren, dass die Verwendung des Befehls **tls::socket** zum Öffnen des Sockets erzwungen wird.

```
# Load and initialize tls
package require tls
tls::init -cafile /path/to/ca/cert -keyfile ...

# Create a connector with secure socket setup,
transfer::transmitter T -socketcmd tls::socket ...
...
```

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
package require tls
tls::init -tls1 1 ;# forcibly activate support for the TLS1
protocol

... your own application code ...
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *transfer* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Transfermodul

Copyright
```
Copyright (c) 2006-2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

treeql

treeql
Stichwörter: treeql, Cost, DOM, XPath, XSLT, structured queries, tree, tree query language

Name
treeql – Abfrage von Baumobjekten

Übersicht
package require Tcl 8.5 9
package require snit
package require struct::list
package require struct::set
package require treeql ?**1.3.2**?
treeql *objectname* **-tree** *tree* ?*-query query*? ?*-nodes nodes*? ?*args...*?
qo **query** *args...*
qo **result**
qo **discard**

Beschreibung
Dieses Paket stellt Objekte bereit, mit denen Baumobjekte abgefragt und durch Transformationen verändert werden können, wobei die API der vom Paket **struct::tree** erstellten Baumobjekte befolgt wird.

Die hier verwendete Abfrage- und Bearbeitungssprache für Baumstrukturen, TreeQL, ist von Cost inspiriert (weitere Informationen finden Sie im Abschnitt Literatur).

Das Paket **treeql** ist eine relativ schlanke Abfragefunktion für baumstrukturierte Datentypen. Es implementiert eine geordnete Menge von Knoten (im Grunde eine Liste), die durch die Anwendung von TreeQL-Operatoren auf jeden Knoten nacheinander generiert und durch einen Filter gefiltert werden.

API

Treeql-Klassen-API
Der Befehl **treeql** ist vom Typ **snit**::type und implementiert die Treeql-Abfragesprache. Das

bedeutet, dass er der API für Klassenbefehle folgt, wie sie im Paket **snit** festgelegt ist. Seine allgemeine Syntax lautet

treeql *objectname* -tree *tree* ?-query *query*? ?-nodes *nodes*? ?*args*...?

Der Befehl erstellt ein neues Baumabfrageobjekt und gibt den vollqualifizierten Namen des Objektbefehls als Ergebnis zurück. Die API, der der zurückgegebene Befehl folgt, wird im Abschnitt Treeql-Objekt-API beschrieben.

Jedes Abfrageobjekt ist mit einem einzelnen *tree*-Objekt verknüpft. Dies ist das Objekt, auf das alle Abfragen angewendet werden.

Wenn die Option **-nodes** angegeben wurde, wird ihr Argument als Liste von Knoten behandelt. Diese Liste wird zur Initialisierung der Knotenmenge verwendet. Standardmäßig ist dies eine leere Liste.

Wenn die Option **-query** angegeben wurde, wird ihr Argument als Objekt interpretiert, nämlich als die *Elternabfrage* dieser Abfrage. Standardmäßig ist dies das Objekt selbst. Alle Abfragen werden in der Umgebung dieses Objekts interpretiert.

Alle Argumente, die auf die Optionen folgen, werden als Abfrage behandelt und sofort ausgeführt, nachdem die *Knotenmenge* initialisiert wurde. Dabei wird dieselbe Syntax für die Abfrage verwendet wie bei der Methode **query**.

Die hierfür verfügbaren TreeQL-Operationen werden im Abschnitt über Die Baumabfragesprache erläutert. In diesem Abschnitt wird auch der oben verwendete Begriff *Knotenmenge* erklärt.

Treeql-Objekt-API

Da **treeql** in **snit** implementiert wurde, stehen dem Benutzer alle Standardmethoden der auf **snit** basierenden Basisklassen zur Verfügung und werden daher hier nicht aufgeführt. Bitte lesen Sie die Dokumentation zu **snit**, um zu erfahren, um welche es sich dabei handelt und welche Funktionen sie bieten.

Die vom Paket **treeql** bereitgestellten Methoden werden im Folgenden aufgeführt und erläutert.

***qo* query *args*...**

Diese Methode interpretiert ihre Argumente als eine Reihe von TreeQL-Operatoren und wertet diese von links nach rechts (d. h. vom ersten bis zum letzten) aus. Beachten Sie, dass der erste Operator die dem Objekt aktuell bekannten *Knotenmenge* verwendet, um seine Aktionen auszuführen. Mit anderen Worten: Die *Knotenmenge* wird vor der Ausführung der Abfrage *nicht* gelöscht oder anderweitig verändert. Dies ermöglicht es dem Benutzer, mehrere Abfragen nacheinander auszuführen, wobei jede Abfrage die Ergebnisse der vorherigen nutzt. Jegliche Initialisierung muss von der jeweiligen Abfrage selbst unter Verwendung von TreeQL-Operatoren vorgenommen werden. Das Ergebnis der Methode ist die *Knotenmenge* nach Ausführung des letzten Operators der Abfrage.

Hinweis: Nicht abgefangene Fehler belassen die *Knotenmenge* des Objekts in einem Zwischenzustand, der den TreeQL-Operatoren entspricht, die vor dem Auftreten des Fehlers erfolgreich ausgeführt wurden.

Im Einzelnen bedeutet dies Folgendes:

1. Das erste Argument wird als Name eines Abfrageoperators interpretiert; anschließend wird die Anzahl der von diesem Operator benötigten Argumente ermittelt und aus den unmittelbar folgenden Argumenten entnommen.

   Aus diesem Grund dürfen Operatoren keine optionalen Argumente haben; alle Argumente müssen wie definiert vorhanden sein. Wird dies nicht beachtet, führt dies zumindest zu Verwirrung beim Interpreter, verursacht jedoch höchstwahrscheinlich Fehler.
2. Der Operator wird auf die aktuelle Knotenmenge angewendet, wodurch eine neue Knotenmenge entsteht und/oder das Baum-Objekt, mit dem das Abfrageobjekt verbunden ist, verändert wird.
3. Die verwendeten Argumente (d. h. der Name des Operators und die Argumente) werden aus der Liste der Argumente für Methoden entfernt, und anschließend wird der gesamte Prozess ab Schritt [1] wiederholt, bis die Liste der Argumente leer ist oder ein Fehler aufgetreten ist.

```
# q is the query object.

q query root children get data

# The above query
# - Resets the node set to the root node - root
# - Adds the children of root to the set - children
# - Replaces the node set with the       - get data
#   values for the attribute 'data',
#   for all nodes in the set which
#   have such an attribute.
# - And returns this information.

# Below we can see the same query, but rewritten
# to show the structure as it is seen by the query
# interpreter.

q query \
    root \
    children \
    get data
```

Die für diese Sprache verfügbaren Operatoren der TreeQL-Sprache werden im Abschnitt über Die Baumabfragesprache erläutert. In diesem Abschnitt wird auch der oben verwendete Begriff *Knotenmenge* erläutert.

***qo* result**

Diese Methode gibt eine Liste zurück, die die aktuelle Knotenmenge enthält.

***qo* discard**

Diese Methode gibt die aktuelle Knotenmenge zurück (wie die Methode **result**), löscht jedoch zusätzlich das Abfrageobjekt (*qo*). Dies ist nützlich beim Erstellen und Verwenden von Unterabfragen (%AUTO%-Objekte werden unmittelbar nach der Verwendung gelöscht).

Die Baumabfragesprache

In diesem und den folgenden Abschnitten wird die von den Abfrageobjekten dieses Pakets verwendete Baumabfragesprache im Detail festgelegt.

Zunächst erläutern wir die allgemeinen Konzepte, die der Sprache zugrunde liegen und für deren Verständnis erforderlich sind. Anschließend folgen die Spezifikationen für alle verfügbaren Abfrageoperatoren. Diese lassen sich in acht Kategorien einteilen, wobei jede Kategorie einen eigenen Abschnitt bildet.

1. Treeql-Konzepte
2. Strukturgeneratoren
3. Attributfilter
4. Attribut-Mutatoren
5. Zugriffsmethoden für Attributzeichenfolgen
6. Unterabfragen
7. Operatoren für Knotenmengen
8. Iteratoren für Knotenmengen
9. Unterstützung für typisierte Knoten

Treeql-Konzepte

Das wichtigste Konzept, das es zu verstehen gilt, ist das der *Knotenmenge*. Jedes Abfrageobjekt verwaltet genau eine solche *Knotenmenge*, und im Wesentlichen nutzen alle Operatoren diese sowohl als Eingabe als auch für ihr Ergebnis. Diese Struktur enthält lediglich die Referenzen aller Knoten, die für das Abfrageobjekt derzeit von Interesse sind. Die Bezeichnung *Menge* ist insofern etwas irreführend, da

1. Ein Knoten (Handle) kann in der Struktur mehrmals vorkommen, und
2. Auch die Reihenfolge der Knoten in der Struktur ist von Bedeutung. Wann immer ein Operator alle Knoten in der Knotenmenge verarbeitet, erfolgt dies in der Reihenfolge, in der sie in der Struktur vorkommen.

Was das mögliche mehrfache Vorkommen eines Knotens betrifft, betrachten Sie eine Knotenmenge, die zwei Knoten A und B enthält, die beide den Knoten P als ihren unmittelbaren Elternknoten haben. Durch die Anwendung des TreeQL-Operators "parent" wird P dann zweimal zur neuen Knotenmenge hinzugefügt, einmal pro Knoten, dessen Elternknoten er war. Das heißt, die neue Knotenmenge lautet dann {P P}.

Strukturgeneratoren

Alle Operatoren für Baumstrukturen lokalisieren Knoten im Baum anhand einer strukturellen Beziehung zu den derzeit in der Menge enthaltenen Knoten und ersetzen anschließend die aktuelle Knotenmenge durch die Menge der gefundenen Knoten. Knoten, die eine solche Beziehung mehrfach erfüllen, werden so oft zum Ergebnis hinzugefügt, wie sie die Beziehung erfüllen.

Es ist wichtig zu beachten, dass die gefundenen Knoten während der Verarbeitung der Knotenmenge in einem separaten Speicherbereich gesammelt werden und erst dann zur aktuellen Knotenmenge hinzugefügt werden (oder diese ersetzen), wenn die aktuelle Knotenmenge vollständig verarbeitet wurde. Mit anderen Worten: Die neuen Knoten werden vom Operator *nicht* ebenfalls verarbeitet und haben keinen Einfluss auf die Iteration.

Wenn die Beschreibung eines Operators erfolgt, wird die Variable **N** verwendet, um auf einen

beliebigen Knoten in der Knotenmenge zu verweisen.

ancestors
Ersetzt die aktuelle Knotenmenge durch die Vorfahren aller Knoten **N** in der Knotenmenge, sofern **N** einen Elternknoten hat. Mit anderen Worten: Knoten ohne Elternknoten fließen nicht in die neue Knotenmenge ein. Mit anderen Worten: Es werden alle Knoten auf dem Pfad vom Knoten **N** zur Wurzel in dieser Reihenfolge (Wurzel zuletzt) für alle Knoten **N** in der Knotenmenge verwendet. Dies schließt die Wurzel ein, jedoch nicht den Knoten selbst.

rootpath
Ersetzt die aktuelle Knotenmenge durch die Vorfahren aller Knoten **N** in der Knotenmenge, sofern **N** einen Elternknoten besitzt. Mit anderen Worten: Knoten ohne Elternknoten fließen nicht in die neue Knotenmenge ein. Im Gegensatz zum Operator **ancestors** werden die Knoten jedoch in umgekehrter Reihenfolge hinzugefügt, d. h. der Wurzelknoten zuerst.

parent
Ersetzt die aktuelle Knotenmenge durch den Elternknoten von **N** für alle Knoten **N** in der Knotenmenge, sofern **N** einen Elternknoten hat. Mit anderen Worten: Knoten ohne Elternknoten fließen nicht in die neue Knotenmenge ein.

children
Ersetzt die aktuelle Knotenmenge durch die unmittelbaren Kindknoten des Knotens **N** für alle Knoten **N** in der Knotenmenge, sofern **N** Kindknoten besitzt. Mit anderen Worten: Knoten ohne Kindknoten fließen nicht in die neue Knotenmenge ein.

left
Ersetzt die aktuelle Knotenmenge durch den vorherigen bzw. linken Geschwisterknoten für alle Knoten **N** in der Knotenmenge, sofern **N** Geschwisterknoten auf der linken Seite hat. Mit anderen Worten: Knoten ohne Geschwisterknoten auf der linken Seite fließen nicht in die neue Knotenmenge ein.

right
Ersetzt die aktuelle Knotenmenge für alle Knoten **N** in der Knotenmenge durch den nächsten/rechten Geschwisterknoten, sofern **N** Geschwisterknoten auf der rechten Seite hat. Mit anderen Worten: Knoten ohne Geschwisterknoten auf der rechten Seite fließen nicht in die neue Knotenmenge ein.

prev
Ersetzt die aktuelle Knotenmenge durch alle vorherigen/linken Geschwisterknoten des Knotens **N** für alle Knoten **N** in der Knotenmenge, sofern **N** Geschwisterknoten auf der linken Seite besitzt. Mit anderen Worten: Knoten ohne linke Geschwister werden ignoriert. Der linke Geschwisterknoten, der direkt an den Knoten angrenzt, wird zuerst hinzugefügt, der am weitesten links liegende Geschwisterknoten zuletzt (umgekehrte Baumreihenfolge).

esib
Ersetzt die aktuelle Knotenmenge durch alle vorherigen/linken Geschwisterknoten des Knotens **N** für alle Knoten **N** in der Knotenmenge, sofern **N** Geschwisterknoten auf der linken Seite besitzt. Mit anderen Worten: Knoten ohne linke Geschwister werden ignoriert. Der am weitesten links liegende Geschwisterknoten wird zuerst hinzugefügt, der an den Knoten angrenzende linke Geschwisterknoten zuletzt (in Baumreihenfolge).

Der Name der Methode ist eine Abkürzung für *Earlier SIBling*.

next

Ersetzt die aktuelle Knotenmenge durch alle nächsten/rechten Geschwisterknoten des Knotens **N** für alle Knoten **N** in der Knotenmenge, sofern **N** Geschwisterknoten auf der rechten Seite besitzt. Mit anderen Worten: Knoten ohne Geschwister rechts tragen nicht zur neuen Knotenmenge bei. Das rechts angrenzende Geschwisterknoten des Knotens wird zuerst hinzugefügt, das ganz rechts liegende Geschwisterknoten zuletzt (in Baumreihenfolge).

root
Ersetzt die aktuelle Knotenmenge durch eine Knotenmenge, die einen einzigen Knoten enthält, nämlich die Wurzel des Baums.

tree
Ersetzt die aktuelle Knotenmenge durch eine Knotenmenge, die alle im Baum gefundenen Knoten enthält. Die Knoten werden in der Vorreihenfolge hinzugefügt (zuerst die Elternknoten, dann die Kindknoten, wobei letztere von links nach rechts und vom ersten bis zum letzten Knoten aufgeführt werden).

descendants
Ersetzt die aktuelle Knotenmenge durch die Knoten aller Teilbäume, deren Wurzel der Knoten **N** ist, und zwar für alle Knoten **N** in der Knotenmenge, sofern **N** Kindknoten besitzt. Mit anderen Worten: Knoten ohne Kindknoten fließen nicht in die neue Knotenmenge ein.

Dies entspricht dem Operator **children**, umfasst jedoch auch die Kindknoten der Kindknoten, d. h. alle *echten Nachkommen*. "Mit Wurzel bei **N**" bedeutet, dass **N** selbst nicht zur neuen Menge hinzugefügt wird, was auch durch den Begriff *echte Nachkommen* impliziert wird.

subtree
Ähnlich wie der Operator **descendants**, umfasst jedoch zusätzlich den Knoten **N**. Mit anderen Worten:

Ersetzt die aktuelle Knotenmenge durch die Knoten des Teilbaums des Knotens **N** für alle Knoten **N** in der Knotenmenge, sofern **N** Kindknoten besitzt. Mit anderen Worten: Knoten ohne Kindknoten fließen nicht in die neue Knotenmenge ein. Das entspricht also dem Operator **children**, umfasst jedoch auch die Kindknoten der Kindknoten usw. "Von **N**" bedeutet, dass **N** selbst zur neuen Menge hinzugefügt wird.

forward
Ersetzt die aktuelle Knotenmenge durch die Knoten in den Teilbäumen, deren Wurzelknoten die rechten Geschwisterknoten des Knotens **N** sind, und zwar für alle Knoten **N** in der Knotenmenge, sofern **N** rechte Geschwister hat und diese wiederum Kindknoten besitzen. Mit anderen Worten: Knoten ohne rechte Geschwister sowie rechte Geschwister ohne Kindknoten werden ignoriert.

Dies entspricht der Operatorfolge

```
next descendants
```

later
Dies ist ein Alias für den Operator **forward**.

backward
Ersetzt die aktuelle Knotenmenge durch die Knoten der abgeflachten vorherigen Teilbäume in umgekehrter Baumstruktur.

Dies entspricht nahezu der Operatorfolge

```
prev descendants
```

Der einzige Unterschied besteht darin, dass hierbei die Knoten in umgekehrter Reihenfolge verwendet werden.

earlier
Ersetzt die aktuelle Knotenmenge durch die Knoten der abgeflachten vorherigen Teilbäume in der Reihenfolge des Baums.

Dies entspricht der Operatorfolge

```
prev subtree
```

Attributfilter

Diese Operatoren filtern die Knotenmenge anhand von Attributen der Knoten und deren Eigenschaften. Filtern bedeutet, dass alle Knoten, die die Kriterien nicht erfüllen, aus der Knotenmenge entfernt werden. Mit anderen Worten: Die Knotenmenge wird durch die Menge der Knoten ersetzt, die die Filterkriterien erfüllen.

hasatt *attr*
Reduziert die Knotenmenge auf jene Knoten, die ein Attribut mit dem Namen *attr* besitzen.

withatt *attr value*
Reduziert die Knotenmenge auf jene Knoten, die ein Attribut mit dem Namen *attr* besitzen und bei denen der Wert dieses Attributs gleich *value* ist (der Operator "==" ist **string equal -nocase**).

withatt! *attr val*
Dies entspricht **withatt**, allerdings müssen alle Knoten in der Knotenmenge über das Attribut verfügen, und der Operator "==" lautet **string equal**, d. h. nicht **-nocase**. Der Operator führt zu einem Fehler, wenn die Knoten das Attribut nicht besitzen.

attof *attr vals*
Reduziert die Knotenmenge auf jene Knoten, die ein Attribut namens *attr* besitzen und bei denen der Wert dieses Attributs in der Liste *vals* der zulässigen Werte enthalten ist. Der hier verwendete Operator für das Enthaltensein führt eine Glob-Übereinstimmung durch (wobei der Attributwert als Muster dient) und ignoriert die Groß-/Kleinschreibung des Attributwerts, *jedoch nicht* bei den Elementen von *vals*.

attmatch *attr match*
Genau wie bei **withatt**, jedoch wird bei **string match** der Operator "==" verwendet, und *match* ist das Muster, auf das geprüft wird.

Hinweis: *match* wird als unvollständige Argument*liste* für **string match** interpretiert. Das bedeutet, dass es als Liste interpretiert wird, die das Muster enthält, und dem Musterelement können Optionen vorangestellt werden, die von **string match** verstanden werden, wie beispielsweise **-nocase**. Dies ist besonders wichtig, falls das Muster Leerzeichen enthält. Es muss in eine Liste eingeschlossen werden, damit der Interpreter es korrekt interpretieren kann.

Attribut-Mutatoren

Diese Operatoren ändern die Attribute der Knoten innerhalb des zugrunde liegenden Baums. Mit anderen Worten: Alle diese Operatoren haben *Nebenwirkungen*.

set *attr val*
: Setzt das Attribut *attr* für alle Knoten **N** in der Knotenmenge auf den Wert *val*. Der Operator schlägt fehl, wenn ein Knoten kein Attribut mit dem Namen *attr* besitzt. Der Baum verbleibt in einem teilweise geänderten Zustand.

unset *attr*
: Hebt das Attribut *attr* für alle Knoten **N** in der Knotenmenge auf. Der Operator schlägt fehl, wenn ein Knoten kein Attribut mit dem Namen *attr* besitzt. Der Baum verbleibt in einem teilweise geänderten Zustand.

Zugriffsmethoden für Attributzeichenfolgen

Diese Operatoren rufen die Werte der Knotenattribute aus dem zugrunde liegenden Baum ab. Die erfassten Ergebnisse werden in der Knotenmenge gespeichert, sind jedoch keine tatsächlichen Knoten.

Mit anderen Worten: Sie definieren die Semantik der vom Abfrageobjekt gespeicherten Knotenmenge neu, sodass diese nach ihrer Ausführung auch Daten enthält, die keine Knoten sind.

Der Interpreter wird beendet, sobald er die Verarbeitung eines dieser Operatoren abgeschlossen hat, wobei alle nachfolgenden Abfrageelemente stillschweigend verworfen werden. Dies bedeutet auch, dass unsere Ausführungen zur Pflege einer Knotenmenge nicht ganz zutreffend sind. Es handelt sich zwar um eine Knotenmenge, solange der Interpreter Befehle verarbeitet, doch kann sie am Ende der Abfrageverarbeitung als Wertmenge für Attribute belassen werden.

string *op attr*
: Wendet den Zeichenfolgenoperator *op* auf das Attribut *attr* für alle Knoten **N** in der Knotenmenge an, sammelt die Ergebnisse dieser Anwendung und legt sie in der Knotenmenge ab.

  Der Operator schlägt fehl, wenn ein Knoten kein Attribut namens *attr* besitzt.

  Das Argument *op* wird als teilweise Argumentliste für den integrierten Befehl **string** interpretiert. Sein erstes Wort muss einer der von **string** unterstützten Unterbefehle sein. Darauf müssen alle für den Unterbefehl erforderlichen Argumente folgen, mit Ausnahme des letzten. Dieses letzte Argument wird durch den Wert des Attributs bereitgestellt.

get *pattern*
: Für alle Knoten **N** in der Knotenmenge ermittelt der Operator zunächst alle Attribute, deren Namen dem Glob-*pattern* entsprechen, anschließend die Werte dieser Attribute und ersetzt schließlich die Knotenmenge durch die Liste dieser Attributwerte.

attlist
: Dies ist eine praktische Kurzdefinition für den Operator **getvals** *. Mit anderen Worten: Sie ersetzt die Knotenmenge durch eine Liste der Werte für alle Attribute aller Knoten **N** in der Knotenmenge.

attrs *glob*
: Ersetzt die aktuelle Knotenmenge durch eine Liste von Attributlisten, wobei für jeden Knoten

N in der Knotenmenge eine eigene Attributliste vorhanden ist.

attval *attname*
Reduziert die aktuelle Knotenmenge mithilfe des Operators **hasatt** und ersetzt sie anschließend durch eine Liste, die die Werte des Attributs *attname* für alle Knoten **N** in der Knotenmenge enthält.

Unterabfragen

Unterabfragen liefern Knotenmengen, die anschließend dazu verwendet werden, die aktuelle Knotenmenge zu erweitern, zu verkleinern oder zu ersetzen.

andq *query*
Ersetzt die Knotenmenge durch die Schnittmenge der durch die Unterabfrage *query* erzeugten Knotenmenge und der aktuellen Knotenmenge.

Bei der Ausführung der Unterabfrage wird die aktuelle Knotenmenge als eigene Ausgangsknotenmenge verwendet.

orq *query*
Ersetzt die Knotenmenge durch die Vereinigung der durch die Unterabfrage *query* erzeugten Knotenmenge und der aktuellen Knotenmenge. Doppelte Knoten werden entfernt.

Bei der Ausführung der Unterabfrage wird die aktuelle Knotenmenge als eigene Ausgangsknotenmenge verwendet.

notq *query*
Ersetzt die Knotenmenge durch die Menge der Knoten, die durch die Unterabfrage *query* generiert wurden und die ebenfalls nicht in der aktuellen Knotenmenge enthalten sind. Mit anderen Worten: Es handelt sich um die Differenz zwischen der aktuellen Knotenmenge und der durch die Unterabfrage generierten Knotenmenge.

Bei der Ausführung der Unterabfrage wird die aktuelle Knotenmenge als eigene Ausgangsknotenmenge verwendet.

Operatoren für Knotenmengen

Diese Operatoren ändern die Knotenmenge direkt, ohne auf den Baum zurückzugreifen.

unique
Entfernt doppelte Knoten aus der Knotenmenge unter Beibehaltung der Reihenfolge. Mit anderen Worten: Das früheste Vorkommen eines Knoten-Handles bleibt erhalten, alle weiteren Vorkommen werden entfernt.

select
Ersetzt die aktuelle Knotenmenge durch eine Knotenmenge, die ausschließlich den ersten Knoten der aktuellen Knotenmenge enthält

transform *query var body*
Zunächst interpretiert es die Unterabfrage *query* und verwendet dabei die aktuelle Knotenmenge als Ausgangsknotenmenge. Anschließend durchläuft es das Ergebnis dieser Abfrage, wobei es den Handle jedes Knotens an die in *var* benannte Variable bindet und das Skript *body* ausführt. Die gesammelten Ergebnisse dieser Ausführungen bilden die neue Knotenmenge, die die aktuelle ersetzt.

Das Skript *body* wird im Kontext des Aufrufers ausgeführt.

map *var body*

Durchläuft die aktuelle Knotenmenge, führt dabei die Bindung des Handles jedes Knotens an die in *var* angegebene Variable durch und führt das Skript *body* aus. Die gesammelten Ergebnisse dieser Ausführungen bilden die neue Knotenmenge, die die aktuelle ersetzt.

Das Skript *body* wird im Kontext des Aufrufers ausgeführt.

quote *val*

Fügt den Literalwert *val* an die aktuelle Knotenmenge an.

replace *val*

Ersetzt die aktuelle Knotenmenge durch den Literalwert *val*.

Iteratoren für Knotenmengen

foreach *query var body*

Es wertet die Unterabfrage *query* aus und führt anschließend auf den Knoten in der durch diese Abfrage erstellten Knotenmenge eine Operation durch, die dem Operator **over** entspricht. Die aktuelle Knotenmenge wird nicht verändert, außer durch Nebenwirkungen des Skripts *body*.

Das Skript *body* wird im Kontext des Aufrufers ausgeführt.

with *query body*

Der Interpreter wertet die *query* aus und führt anschließend das Skript *body* auf der durch die Abfrage generierten Knotenmenge aus. Zuletzt wird die aktuelle Knotenmenge in den Zustand zurückversetzt, in dem sie sich vor der Ausführung der Abfrage befand.

Das Skript *body* wird im Kontext des Aufrufers ausgeführt.

over *var body*

Führt das Skript *body* für jeden Knoten in der Knotenmenge aus, wobei die durch *var* benannte Variable auf den Namen des aktuellen Knotens gesetzt wird. Das Skript *body* wird im Kontext des Aufrufers ausgeführt.

Dies entspricht dem eingebauten Befehl **foreach**, wobei die Knotenmenge als Quelle der zu durchlaufenden Liste dient.

Die Ergebnisse der Ausführung von *body* werden ignoriert.

delete

Löscht alle Knoten, die in der aktuellen Knotenmenge enthalten sind, aus dem Baum.

Unterstützung für typisierte Knoten

Diese Filter und Zugriffsfunktionen setzen das Vorhandensein eines Attributs namens **@type** voraus und stellen Kurzformen dar, die sich für Cost-artige Baumabfragen, die Bearbeitung von HTML-Bäumen und Ähnliches eignen.

nodetype

Gibt den Knotentyp der Knoten zurück. Zugriffsmethode für Attributzeichenfolgen. Dies

entspricht

```
get @type
```

oftype *t*
Reduziert die Knotenmenge auf jene Knoten, deren Typ mit *t* übereinstimmt, wobei die Groß-/Kleinschreibung ignoriert wird.

nottype *t*
Reduziert die Knotenmenge auf jene Knoten, deren Typ nicht *t* entspricht, wobei die Groß-/Kleinschreibung ignoriert wird.

oftypes *attrs*
Reduziert die Menge auf Knoten, deren @type ein Element der Liste *attrs* von Typen ist. Der Wert von @type wird als Glob-Muster verwendet, wobei die Groß- und Kleinschreibung beachtet wird.

Beispiele
... Zu erledigen ...

Literatur
1. COST im Tcler-Wiki.
2. TreeQL im Tcler-Wiki. Bitte diskutieren Sie dieses Paket dort.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *treeql* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Datenstrukturen

Copyright

```
Copyright (c) 2004 Colin McCormack <coldstore@users.sourceforge.net>
Copyright (c) 2004 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


try

tcllib_fhome
Stichwörter: tcllib_fhome

Name
file::home – file home – Das Home-Verzeichnis des aktuellen oder eines anderen Benutzers gibt

zurück

Übersicht

package require Tcl 8.x
package require fhome ?**1**?
file home ?*user*?

Beschreibung

Dieses Paket bietet eine Paket-Implementierung des Tcl-9-Unterbefehls file home (TIP 602) für Tcl 8.x.

file home ?*user*?
Gibt das Home-Verzeichnis des aktuellen Benutzers oder, wenn angegeben, des angegebenen *Benutzers* zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *file* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hilfsprogramme

Copyright


```
Copyright (c) 2024 Andreas Kupries, BSD licensed
```


tcllib_throw

Stichwörter: tcllib_throw, error, return, throw

Name

throw - throw - Löst eine Fehlerausnahme mit einer Meldung aus

Übersicht

package require Tcl 8.5 9
package require throw ?**1.1**?
::throw *error_code error_message*

Beschreibung

Dieses Paket bietet eine Paket-Implementierung des Befehls **throw** aus Tcl 8.6 (TIP 329) für Tcl 8.5. Der Code wurde direkt aus der Revision ? von Tcl 8.6 übernommen, als "try/finally" noch als Tcl-Prozedur statt in C implementiert war.

::throw *error_code error_message*

"throw" dient lediglich dazu, die Reihenfolge der Argumente des Befehls "error" neu zu ordnen. Es löst einen Fehler mit dem angegebenen Fehlercode und der angegebenen Fehlermeldung aus.

Beispiele

```
throw {MYERROR CODE} "My error message"
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *try* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hilfsprogramme

Copyright


```
Copyright (c) 2015 Miguel Martínez López, BSD licensed
```


tcllib_try

Stichwörter: tcllib_try, cleanup, error, exception, final, resource management

Name

try – try – Fehler und Ausnahmen abfangen und verarbeiten

Übersicht

package require Tcl 8.5 9
package require try ?**1.1**?
::try *body* ?*handler...*? ?*finally script*?

Beschreibung

Dieses Paket bietet eine Implementierung mit Kompatibilität zum Tcl 8.6-Befehl **try**/**finally** (TIP 329) für Tcl 8.5. Der Code stammt direkt aus der Tcl-8.6-Revision ?, als **try**/**finally** als Tcl-Prozedur statt in C implementiert wurde.

::try *body* ?*handler...*? ?finally *script*?

Dieser Befehl führt das Skript *body* aus und ruft – je nachdem, wie das Skript endet (normales Beenden, Fehler oder ein anderes außergewöhnliches Ergebnis) – ein Handler-Skript auf, um den jeweiligen Fall zu behandeln. Sobald all dies geschehen ist und eine **finally**-Klausel vorhanden ist, wird das darin enthaltene *Skript* ausgeführt, und das Ergebnis des Handlers (oder des *body*, falls kein Handler gefunden wurde) darf weiterverbreitet werden. Hinweise: Die **finally**-Klausel wird auch dann verarbeitet, wenn ein Fehler auftritt, und zwar unabhängig davon, welcher *Handler* gegebenenfalls verwendet wird.

Die *Handler*-Klauseln haben jeweils einen Ausdruck aus mehreren Wörtern und müssen eine der folgenden Formen haben:

on *code variableList script*
: Diese Klausel passt, wenn die Auswertung von *body* mit dem Ausnahmecode *code* endete. *code* darf eine Ganzzahl sein oder eines der Wörter **ok**, **error**, **return**, **break** oder **continue**. Diese Wörter entsprechen den Ganzzahlen 0 bis 4.

trap *pattern variableList script*
: Diese Klausel trifft zu, wenn die Auswertung von *body* zu einem Fehler geführt hat und das Präfix von **-errorcode** aus dem Dictionary des Interpreters mit dem Muster übereinstimmt. Die Anzahl der aus **-errorcode** entnommenen Präfixwörter entspricht der Listenlänge von *Muster*, und Leerzeichen zwischen den Wörtern werden sowohl in **-errorcode** als auch in *Muster* vor dem Vergleich normalisiert.

  Das Wort *variableList* in jedem *Handler* wird stets als Liste von Variablennamen interpretiert. Wenn das erste Wort der Liste vorhanden und nicht leer ist, benennt es eine Variable, in die das Ergebnis der Auswertung von *body* (aus dem Haupt-**try**) gespeichert wird; diese enthält die für Menschen lesbare Form etwaiger Fehler. Wenn das zweite Wort der Liste vorhanden und nicht leer ist, benennt es eine Variable, in die das Options Dictionary des Interpreters zum Zeitpunkt der Beendigung der Ausführung von *body* platziert wird.

  Das Wort *script* in jedem *handler* wird ebenfalls immer gleich interpretiert: als Tcl-Skript, das ausgewertet wird, wenn die Klausel übereinstimmt. Ist *script* ein Literal **-** und ist der *handler* nicht der letzte, wird stattdessen das *script* des folgenden *handler* aufgerufen (genau wie beim **switch**-Befehl).

  Beachte, dass *handler*-Klauseln der Reihe nach abgeglichen werden und dass immer die erste übereinstimmende ausgewählt wird. Es wird höchstens eine *Handler*-Klausel ausgewählt. Folglich überdeckt ein **on error** alle nachfolgenden **trap**-Anweisungen im **try**. Hinweis: **on error** ist gleichbedeutend mit **trap {}**.

  Wenn während der Auswertung entweder der *Handler*- oder der **finally**-Klausel eine Ausnahme auftritt (d. h. ein beliebiges Ergebnis, das nicht **ok** ist), wird das Dictionary der ursprünglichen Ausnahme unter dem Schlüssel **-during**.

Beispiele

Eine Datei in jedem Fall schließen:

```
set f [open /some/file/name a]
try {
    puts \$f "some message"
    # ...
} finally {
    close \$f
}
```

Verschiedene Gründe behandeln, warum eine Datei nicht zum Lesen geöffnet werden kann:

```
try {
    set f [open /some/file/name]
} trap {POSIX EISDIR} {} {
    puts "failed to open /some/file/name: it's a directory"
} trap {POSIX ENOENT} {} {
    puts "failed to open /some/file/name: it doesn't exist"
}
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *try* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hilfsprogramme

Copyright


```
Copyright (c) 2008 Donal K. Fellows, BSD licensed
```


udpcluster

udpcluster

Stichwörter: udpcluster, name service, server

Name

udpcluster – UDP-Peer-to-Peer-Cluster

Übersicht

package require Tcl 8.5 9
package require udpcluster ?**0.3.4**?
package require ip
package require nettool
package require comm
package require interp
package require dicttool
package require cron

Beschreibung

Dieses Paket ist eine schlanke Alternative zu Zeroconf. Es nutzt UDP-Pakete, um eine Tabelle der Dienste zu füllen, die jeder Knoten in einem lokalen Netzwerk anbietet. Jeder Teilnehmer sendet per Broadcast eine Schlüssel-Wert-Liste als Klartext in UTF-8. Darin steht, welche Ports offen sind

und welche Protokolle an diesen Ports erwartet werden. Entwickler können darüber hinaus beliebige weitere Schlüssel-Wert-Paare hinzufügen.

Verwendung von udpcluster.

Rufen Sie für jeden Dienst, den Sie veröffentlichen möchten, folgenden Befehl auf:

```
cluster::publish echo@[cluster::macid] {port 10000 protocol echo}
```

Um abzufragen, welche Dienste im lokalen Netzwerk verfügbar sind:

```
set results [cluster::search PATTERN]
# And inside that result...
echo@LOCALMACID {
   port 10000
   protocol echo
}
```

So heben Sie die Veröffentlichung eines Dienstes auf:

```
cluster::unpublish echo@[cluster::macid]
```

Historische Hinweise:

Dieses Werkzeug war ursprünglich als nns::cluster bekannt. Im Laufe der Entwicklung wurde klar, dass es mit keiner der anderen Einrichtungen in NNS zusammenarbeitete.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *nameserv* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

Copyright

```
Copyright (c) 2016-2018 Sean Woods <yoda@etoyoc.com>
```

uev

uevent

Stichwörter: uevent, bind, event, generate event, hook, unbind

Name

uevent – Benutzerereignisse

Übersicht

package require Tcl 8.5 9
package require uevent ?**0.3.2**?
package require logger
::uevent::bind *tag event command*
command *tag event details*
::uevent::unbind *token*
::uevent::generate *tag event* ?*details*?
::uevent::list
::uevent::list *tag*
::uevent::list *tag event*
::uevent::watch::tag::add *pattern command*
{*}command bound *tag*
{*}command unbound *tag*
::uevent::watch::tag::remove *token*
::uevent::watch::event::add *tag_pattern event_pattern command*
{*}command bound *tag event*
{*}command unbound *tag event*
::uevent::watch::event::remove *token*

Beschreibung

Dieses Paket bietet eine allgemeine Funktion zur Verarbeitung von Benutzerereignissen. Es ermöglicht die Zuordnung beliebiger Befehle zu beliebigen Ereignissen auf beliebigen Tags, das Aufheben von Bindungen sowie die Erzeugung von Ereignissen.

Der wesentliche Unterschied zum im Tcl/Tk-Kern integrierten Ereignissystem besteht darin, dass dieses lediglich virtuelle Ereignisse erzeugen kann, und zwar ausschließlich für Widgets. Es ist nicht möglich, die integrierten Funktionen zu nutzen, um Ereignisse an beliebige (Pseudo-)Objekte zu binden, und es ist auch nicht in der Lage, Ereignisse für solche Objekte zu erzeugen.

Hier können wir davon ausgehen, dass jedes betreffende Objekt durch ein eigenes Tag dargestellt wird. Dies ist möglich, da wir beliebige Tags zulassen.

Weitere Unterschiede:

1. Das Paket nutzt zur Abwicklung aller Vorgänge lediglich eine zweistufige Hierarchie – Tags und Ereignisse –, während das Tcl/Tk-System drei Ebenen verwendet, nämlich Objekte, Tags und Ereignisse, wobei zwischen Objekten und Tags eine n:m-Beziehung besteht.
2. Dieses Paket löst alle zugewiesenen Befehle für eine Kombination aus Tag und Ereignis aus, wobei diese voneinander unabhängig sind. Ein zugewiesener Befehl kann den Ereignisverarbeitungskern nicht dazu zwingen, die Verarbeitung eines nachfolgenden Befehls abzubrechen.

API

Das Paket exportiert acht Befehle, wie im Folgenden angegeben. Beachten Sie, dass wenn das Paket innerhalb von Tcl 8.5+ verwendet wird, alle übergeordneten Befehle Ensembles sind, d. h., die Trennzeichen "::" können durch Leerzeichen ersetzt werden.

::uevent::bind *tag event command*

Mit diesem Befehl wird das Präfix *command* so registriert, dass es ausgelöst wird, wenn das *event* für das *tag* eintritt. Das Ergebnis des Befehls ist ein undurchsichtiges Token, das die Bindung repräsentiert. Bitte beachten Sie: Wird dieselbe Kombination aus <*tag*,*event*, *command*> mehrfach verwendet, gibt der Befehl bei jedem Aufruf dasselbe Token zurück.

Die Signatur des Präfixes *command* lautet
- **command** *tag event details*

wobei *details* das bzw. die Argumente des Ereignisses enthält. Der Inhalt ist ereignisspezifisch und muss zwischen dem eigentlichen Erzeuger des Ereignisses und dem Empfänger vereinbart werden. Dieses Paket überträgt lediglich die Informationen und führt darüber hinaus keine weitere Verarbeitung durch.

::uevent::unbind *token*

Dieser Befehl hebt die durch das *token* dargestellte Ereignisbindung auf. Das Token muss das Ergebnis eines Aufrufs von **::uevent::bind** sein. Das Ergebnis des Befehls ist die leere Zeichenkette.

::uevent::generate *tag event* ?*details*?

Dieser Befehl generiert ein *event* für das *tag* und löst damit alle Befehle aus, die dieser Kombination zugeordnet sind. Das Argument *details* wird einfach unverändert an alle Ereignisbehandler weitergeleitet. Es liegt in der Verantwortung des Codes, der das Ereignis generiert und verarbeitet, eine Vereinbarung über das Format und den Inhalt der darin enthaltenen Informationen zu treffen. Das Ergebnis des Befehls ist die leere Zeichenkette.

Beachten Sie, dass alle zugeordneten Befehle unabhängig voneinander ausgelöst werden. Die Ereignisbehandler dürfen keine bestimmte Reihenfolge voraussetzen. Sie werden zudem *nicht* synchron mit dem Aufruf dieses Befehls aufgerufen, sondern lediglich in die Ereigniswarteschlange zur Verarbeitung gestellt, wenn das System zur Ereignisschleife zurückkehrt.

Das Erzeugen eines Ereignisses für ein unbekanntes Tag oder für eine <*tag*,*event* >-Kombination, der keine Befehle zugeordnet sind, ist zulässig; solche Aufrufe werden ignoriert.

::uevent::list

In dieser Form gibt der Befehl eine Liste zurück, die die Namen aller Tags enthält, denen Ereignisse mit zugeordneten Befehlen zugeordnet sind.

::uevent::list *tag*

In diesem Format gibt der Befehl eine Liste zurück, die die Namen aller Ereignisse für das *tag* enthält, denen Befehle zugeordnet sind. Die Angabe eines unbekannten Tags, d. h. eines Tags ohne Ereignis und ohne Befehle, führt dazu, dass der Befehl eine Fehlermeldung ausgibt.

::uevent::list *tag event*

In diesem Format gibt der Befehl eine Liste zurück, die alle Befehle enthält, die dem *event* für das *tag* zugeordnet sind. Die Angabe eines unbekannten Tags oder eines unbekannten Ereignisses führt dazu, dass der Befehl eine Fehlermeldung ausgibt.

::uevent::watch::tag::add *pattern command*

Dieser Befehl richtet eine Art umgekehrter Ereignisse ein. Diese Ereignisse werden ausgelöst – d. h., das Präfix *command* wird aufgerufen –, wenn Beobachter an bestimmte Tags gebunden

oder von diesen getrennt werden.

Beachten Sie, dass das Präfix *command* pro Tag nur zweimal aufgerufen wird: erstens, wenn der erste Befehl an ein beliebiges Ereignis des Tags gebunden wird, und zweitens, wenn der letzte an das Tag gebundene Befehl entfernt wird.

Die Signatur des Präfixes *command* lautet
- **{*}command bound** *tag*
- **{*}command unbound** *tag*

Das Ergebnis des Befehls ist ein Token, das den Watcher repräsentiert.

::uevent::watch::tag::remove *token*
Dieser Befehl entfernt einen Beobachter für (Un-)Bind-Ereignisse bei Tags.

Das Ergebnis des Befehls ist die leere Zeichenkette.

::uevent::watch::event::add *tag_pattern event_pattern command*
Dieser Befehl richtet eine Art "umgekehrter Ereignisse" ein. Dabei werden Ereignisse generiert – d. h., das Präfix *command* wird aufgerufen –, wenn Beobachter bestimmte Kombinationen aus Tags und Ereignissen binden oder die Bindung dazu aufheben.

Beachten Sie, dass das Befehlspräfix pro Tag-/Ereigniskombination nur zweimal aufgerufen wird: erstens, wenn der erste Befehl daran gebunden wird, und zweitens, wenn der letzte daran gebundene Befehl entfernt wird.

Die Signatur des *command*-Präfixes lautet
- **{*}command bound** *tag event*
- **{*}command unbound** *tag event*

Das Ergebnis des Befehls ist ein Token, das den Watcher repräsentiert.

::uevent::watch::event::remove *token*
Dieser Befehl entfernt einen Beobachter für (Un-)Bind-Ereignisse bei Tag-/Ereignis-Kombinationen.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *uevent* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright

```
Copyright (c) 2007-2012 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

uevent_onidle

Stichwörter: uevent_onidle, callback, deferal, event, idle, merge, on-idle

Name

uevent::onidle – Antrag auf Zusammenlegung und Verschiebung in die Leerlaufzeit

Übersicht

package require Tcl 8.5 9
package require uevent::onidle ?**0.2**?
package require logger
::uevent::onidle *objectName commandprefix*
objectName **request**

Beschreibung

Dieses Paket stellt Objekte bereit, mit denen mehrere Anfragen für eine Aktion zusammengefasst und die Aktion ausgeführt werden können, sobald das System (die Ereignisschleife) nicht mehr ausgelastet ist. Die auszuführende Aktion wird bei der Erstellung des Objekts konfiguriert.

API

Das Paket exportiert eine Klasse namens "**uevent::onidle**", wie im Folgenden beschrieben.

::uevent::onidle *objectName commandprefix*

Der Befehl erstellt ein neues *onidle*-Objekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *objectName* lautet. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Objekt auszuführen.

Das *commandprefix* ist die Aktion, die ausgeführt werden soll, wenn die Ereignisschleife im Leerlauf ist und der Benutzer dies über die Methode **request** angefordert hat (siehe unten).

Die durch die oben genannten Klassenbefehle erstellten Objektbefehle haben folgende Form:

***objectName* request**

Diese Methode fordert die Ausführung des bei der Erstellung von *objectName* festgelegten Befehlspräfixes an, sobald die Ereignisschleife das nächste Mal inaktiv ist. Mehrere Anfragen werden zusammengefasst und führen zu nur einer einzigen Ausführung des Befehlspräfixes.

Beispiele

Beispiele für diese Art der Verzögerung finden sich in den (C-Level-)Implementierungen aller Tk-Widgets, in denen Geometrieberechnungen und das Neuzeichnen von Fenstern auf diese Weise verzögert werden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *uevent* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright


```
Copyright (c) 2008 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


units

units

Stichwörter: units, angle, constants, conversion, distance, radians, unit

Name

units – Einheitenumrechnung

Übersicht

package require Tcl 8.5 9
package require units ?**2.2.3**?
::units::convert *value targetUnits*
::units::reduce *unitString*
::units::new *name baseUnits*

Beschreibung

Diese Bibliothek bietet eine Funktion zur Umwandlung verschiedener wissenschaftlicher und technischer Kurzschreibweisen in Gleitkommazahlen. Dadurch können Anwendungsentwickler Werte mit unterschiedlichen Einheiten problemlos in einheitlich skalierte Zahlen umwandeln.

Die Funktion zur Einheitenumrechnung ist zudem in der Lage, zwischen kompatiblen Einheiten umzurechnen. Wenn beispielsweise eine Anwendung einen Wert in *Ohm* (Widerstand) erwartet und der Benutzer die Einheiten *Milliwebers/Femtocoulomb* legt, wird dies von der Umrechnungsroutine entsprechend verarbeitet. Bei dem Versuch einer falschen Umrechnung wird eine Fehlermeldung ausgegeben.

Werte werden mittels Dimensionsanalyse von einem Einheitensystem in ein anderes umgerechnet. Sowohl die Ausgangseinheiten als auch die Zieleinheiten werden in Elementareinheiten und einen Skalierungsfaktor zerlegt. Die Einheiten werden auf Kompatibilität geprüft, und die Skalierungsfaktoren werden durch Multiplikation und Division angewendet. Diese Methode ist äußerst flexibel und sehr robust.

Neue Einheiten und neue Einheitenabkürzungen können anhand bestehender Einheiten und Abkürzungen definiert werden. Es ist auch möglich, eine neue Grundeinheit zu definieren, obwohl dies wahrscheinlich nicht erforderlich sein wird. Neue Einheiten werden in der Regel definiert, um Nicht-SI-Maßsysteme zu berücksichtigen, beispielsweise die Definition der Einheit *Zoll* als *2,54 cm*.

Befehle

::units::convert *value targetUnits*

Wandelt die Zeichenfolge *value* in eine Gleitkommazahl um, die auf die angegebenen *targetUnits* skaliert wird. Die Zeichenfolge *value* kann eine Zahl und Einheiten enthalten. Wenn Einheiten festgelegt werden, müssen diese mit den *targetUnits* kompatibel sein. Werden für den *value* keine Einheiten festgelegt, wird er auf die Zieleinheiten skaliert. Beispielsweise:

```
% ::units::convert "2.3 miles" km
3.7014912
% ::units::convert 300m/s miles/hour
671.080887616
% ::units::convert "1.0 m kg/s^2" newton
1.0
% ::units::convert 1.0 millimeter
1000.0
```

::units::reduce *unitString*

Gibt eine Einheitenzeichenfolge zurück, die aus einem Skalierungsfaktor und einer durch Leerzeichen getrennten Liste sortierter und reduzierter Grundeinheiten besteht. Die reduzierte Einheitenzeichenfolge kann einen Schrägstrich enthalten (der durch Leerzeichen von den umgebenden Grundeinheiten getrennt ist), der angibt, dass sich die verbleibenden Untereinheiten im Nenner befinden. Löst einen Fehler aus, wenn die *unitString* ungültig ist.

```
% ::units::reduce pascal
1000.0 gram / meter second second
```

::units::new *name baseUnits*

Erstellt eine neue Einheitenumrechnung mit dem festgelegten Namen. Der neue Name *name* darf ausschließlich aus Buchstaben (Groß- oder Kleinbuchstaben) bestehen. Die Zeichenfolge *baseUnits* kann aus jeder gültigen Zeichenfolge für eine Einheitenumrechnung bestehen, einschließlich konstanter Faktoren, Zähler- und Nenneranteile, Einheiten mit Vorsätzen und Exponenten. Die *baseUnits* dürfen eine beliebige Anzahl von Untereinheiten enthalten, müssen sich jedoch auf primitive Einheiten zurückführen lassen. *BaseUnits* könnte auch die Zeichenfolge *-primitive* sein, um eine neue Art von Größe darzustellen, die sich nicht aus anderen Einheiten ableiten lässt. Dies würden Sie jedoch wahrscheinlich nur tun, wenn Sie eine neue universelle Eigenschaft entdeckt hätten.

```
% ::units::new furlong "220 yards"
% ::units::new fortnight "14 days"
% ::units::convert 100m/s furlongs/fortnight
601288.475303
```

Format der Einheitenzeichenfolge

Das Format für Werte und Einheitenzeichenfolgen ist sehr flexibel. Es ist möglich, praktisch jede Kombination aus Einheiten, Präfixen und Potenzen zu definieren. Gültige Einheitenzeichenfolgen müssen diesen Regeln entsprechen.

- Eine Einheitenfolge besteht aus einem optionalen Skalierungsfaktor, gefolgt von null oder mehr Untereinheiten. Der Skalierungsfaktor muss eine gültige Gleitkommazahl sein und kann von den Untereinheiten getrennt sein oder auch nicht. Der Skalierungsfaktor kann negativ sein.
- Untereinheiten werden durch ein oder mehrere Trennzeichen voneinander getrennt; dabei handelt es sich um Leerzeichen (" "), Bindestrich ("-"), Sternchen ("*") und Schrägstrich ("/"). Sicher, beschweren Sie sich ruhig darüber, dass das Minuszeichen ("-") zur Darstellung der Multiplikation verwendet wird. Das ist mathematisch gesehen einfach nicht korrekt, und eigentlich sollten wir von jedem die Anforderung stellen, das Sternchen ("*") zur Trennung aller Einheiten zu verwenden. Letztendlich sind jedoch komplexe Einheitenfolgen wie *m-kg/s^2* angenehm lesbar.
- Das Schrägstrich-Trennzeichen ("/") zeigt an, dass sich die nachfolgenden Untereinheiten im Nenner befinden. Es darf höchstens ein Schrägstrich-Trennzeichen vorhanden sein.
- Untereinheiten können Skalierungsfaktoren mit Gleitkommazahlen sein, müssen jedoch – mit Ausnahme des führenden Skalierungsfaktors – von gültigen Trennzeichen umgeben sein. Skalierungsfaktoren von Untereinheiten dürfen nicht negativ sein. (Beachten Sie, dass der Bindestrich ein Einheitentrennzeichen ist.)
- Untereinheiten können gültige Einheiten oder Abkürzungen sein. Sie können ein Präfix enthalten. Sie können die Pluralendungen "s" oder "es" enthalten. Sie können außerdem eine Potenzangabe enthalten, die durch ein Zirkumflex ("^") gekennzeichnet ist, gefolgt von einer ganzen Zahl, die hinter dem Namen der Einheit (oder dem Plural-Suffix, falls vorhanden) steht. Negative Exponenten sind nicht zulässig. (Beachten Sie, dass der Bindestrich als Einheitstrennzeichen dient.)

Beispiele für gültige Einheitenangaben

```
Unit String              Reduced Unit String
-----------------------------------------------------------
meter                    1.0 meter
kilometer                1000.0 meter
km                       1000.0 meter
km/s                     1000.0 meter / second
/microsecond             1000000.0 / second
/us                      1000000.0 / second
kg-m/s^2                 1000.0 gram meter / second second
30second                 30.0 second
30 second                30.0 second
30 seconds               30.0 second
200*meter/20.5*second    9.75609756098 meter / second
```

SI-Einheiten

Die SI-Standardeinheiten sind gemäß der *NIST Special Publication 330* vordefiniert. Die Standardeinheiten sowohl für die SI-Basiseinheiten (Tabelle 1) als auch für die abgeleiteten SI-Einheiten mit speziellen Namen (Tabellen 3a und 3b) sind hier als Referenz enthalten. Jeder Name und jede Abkürzung einer Standardeinheit sind in diesem Paket enthalten.

SI-Basiseinheiten

Quantity	Unit Name	Abbr.
Length	meter	m
Mass	kilogram	kg
Time	second	s
Current	ampere	A
Temperature	kelvin	K
Amount	mole	mol
Luminous Intensity	candela	cd

Abgeleitete SI-Einheiten mit besonderen Namen

Quantity	Unit Name	Abbr.	Units	Base Units
plane angle	radian	rad	m/m	m/m
solid angle	steradian	sr	m^2/m^2	m^2/m^2
frequency	hertz	Hz		/s
force	newton	N		m-kg/s^2
pressure	pascal	Pa	N/m^2	kg/m-s^2
energy, work	joule	J	N-m	m^2-kg/s^2
power, radiant flux	watt	W	J/s	m^2-kg/s^3
electric charge	coulomb	C		s-A
electric potential	volt	V	W/A	m^2-kg/s^3-A
capacitance	farad	F	C/V	s^4-A^2/m^2-kg
electric resistance	ohm		V/A	m^2-kg/s^3-A^2
electric conductance	siemens	S	A/V	s^3-A^2/m^2-kg
magnetic flux	weber	Wb	V-s	m^2-kg/s^2-A
magnetic flux density	tesla	T	Wb/m^2	kg/s^2-A
inductance	henry	H	Wb/A	m^2-kg/s^2-A^2
luminous flux	lumen	lm		cd-sr
illuminance	lux	lx	lm/m^2	cd-sr/m^2
activity (of a radionuclide)	becquerel	Bq		/s
absorbed dose	gray	Gy	J/kg	m^2/s^2
dose equivalent	sievert	Sv	J/kg	m^2/s^2

Hinweis: Die SI-Einheit Kilogramm wird tatsächlich als Gramm umgesetzt, da 1e-6 Kilogramm = 1 Milligramm und nicht 1 Mikrokilogramm entspricht. Die Abkürzung für den elektrischen Widerstand (Ohm), nämlich das Omega-Zeichen, wird nicht unterstützt.

Außerdem ist zu beachten: Es besteht keine Unterstützung für Temperaturen in Celsius oder Fahrenheit. Die Routinen zur Einheitenumrechnung können Werte lediglich durch Multiplikation und Division skalieren, sodass es nicht möglich ist, thermodynamische Temperaturen (Kelvin) in absolute Grad Celsius oder Fahrenheit umzurechnen. Die Umrechnung thermodynamischer Größen, wie beispielsweise der thermischen Ausdehnung (pro Temperatureinheit), lässt sich jedoch problemlos in die Bibliothek der Einheiten integrieren.

SI-Einheiten können ein Vorzeichen für ein Vielfaches oder ein Bruchteil einer Einheit tragen. Das Vorzeichen oder dessen Abkürzung sollte ohne Leerzeichen vor der Einheit stehen.

Zusammengesetzte Vorzeichen sind nicht zulässig, und ein Vorzeichen sollte niemals allein verwendet werden. Diese Vorzeichen sind in Tabelle 5 der *Special Publication 330* definiert.

SI-Präfixe

```
Prefix Name     Abbr.   Factor
----------------------------------------
yotta           Y       1e24
zetta           Z       1e21
exa             E       1e18
peta            P       1e15
tera            T       1e12
giga            G       1e9
mega            M       1e6
kilo            k       1e3
hecto           h       1e2
deka            da      1e1
deca                    1e1

deci            d       1e-1
centi           c       1e-2
milli           m       1e-3
micro           u       1e-6
nano            n       1e-9
pico            p       1e-12
femto           f       1e-15
atto            a       1e-18
zepto           z       1e-21
yocto           y       1e-24
```

Hinweis: Wir definieren das gleiche Präfix sowohl in der US-amerikanischen Schreibweise ("deka") als auch in der nicht-US-amerikanischen Schreibweise ("deca"). Hinweis: Wir nehmen die Freiheit, die Schreibweise von "micro" mit einem "u" anstelle des griechischen Buchstabens Mu zuzulassen.

In der Praxis werden häufig Nicht-SI-Einheiten verwendet. Anhang B.8 der *NIST Special Publication 811* listet zahlreiche Umrechnungsfaktoren für Nicht-SI-Einheiten auf. Es ist nicht möglich, alle möglichen Definitionen der Einheiten in dieses Paket aufzunehmen. In einigen Fällen gibt es für eine bestimmte Einheit je nach Kontext viele verschiedene Umrechnungsfaktoren. (Der Anhang listet über 40 Umrechnungen für britische Wärmeeinheiten auf!) Anwendungsspezifische Umrechnungen können jederzeit mit dem Befehl **new** hinzugefügt werden, doch einige bekannte und häufig verwendete Umrechnungen sind bereits in diesem Paket enthalten.

Nicht-SI-Einheiten

```
Unit Name            Abbr.    Base Units
-----------------------------------------------
angstrom                      1.0E-10 m
astronomicalUnit     AU       1.495979E11 m
atmosphere                    1.01325E5 Pa
bar                           1.0E5 Pa
calorie                       4.1868 J
curie                         3.7E10 Bq
day                           8.64E4 s
degree                        1.745329E-2 rad
erg                           1.0E-7 J
faraday                       9.648531 C
fermi                         1.0E-15 m
foot                 ft       3.048E-1 m
gauss                         1.0E-4 T
gilbert                       7.957747E-1 A
grain                gr       6.479891E-5 kg
hectare              ha       1.0E4 m^2
hour                 h        3.6E3 s
inch                 in       2.54E-2 m
lightYear                     9.46073E15 m
liter                L        1.0E-3 m^3
maxwell              Mx       1.0E-8 Wb
mho                           1.0 S
micron                        1.0E-6 m
mil                           2.54E-5 m
mile                 mi       1.609344E3 m
minute               min      6.0E1 s
parsec               pc       3.085E16 m
pica                          4.233333E-3 m
pound                lb       4.535924E-1 kg
revolution                    6.283185 rad
revolutionPerMinute  rpm      1.047198E-1 rad/s
yard                 yd       9.144E-1 m
year                          3.1536E7 s
```

Größen und abgeleitete Einheiten mit besonderen Namen

Dieses Paket zur Einheitenumrechnung beschränkt sich ausschließlich auf die Umrechnung, den Vergleich und die Skalierung von Einheiten. Dieses Paket berücksichtigt keine der Größennamen, weder für Basiseinheiten noch für abgeleitete Einheiten. Eine ähnliche Implementierung oder eine Erweiterung in einer typisierten oder objektorientierten Sprache könnte benutzerdefinierte Typen für die Größen einführen. Die Überprüfung des Größentyps könnte beispielsweise dazu dienen, sicherzustellen, dass alle *Längen*-Werte ordnungsgemäß auf *Meter* reduziert werden oder dass alle *Geschwindigkeits*-Werte ordnungsgemäß auf *Meter/Sekunde* reduziert werden.

Es wurde eine C-Implementierung dieses Pakets erstellt, die in Verbindung mit dem "Simplified Wrapper Interface Generator" (http://www.swig.org/) funktioniert. Dieses Paket (units.i) nutzt das

Typmap-System von SWIG, um Skript-Größenangaben automatisch in Gleitkommagrößen umzuwandeln. Funktionsargumente werden als Größentypen festgelegt (z. B. *typedef float Length*), und die Ziel-Einheiten (wie sie vom C-Anwendungscode erwartet werden) werden in einem assoziativen Array festgelegt. Für jeden Größentyp sind zudem Standardeinheiten definiert, die auf alle einheitenlosen Größenangaben angewendet werden.

Ein mit einer Mengen-Typprüfung erweitertes Einheitensystem könnte von der Einbeziehung weiterer abgeleiteter Typen profitieren, die in Form spezieller Einheiten zum Ausdruck gebracht werden, wie in Tabelle 2 der *NIST-Veröffentlichung 330* dargestellt. Die Größe *Fläche* könnte beispielsweise als Einheiten definiert werden, die sich ordnungsgemäß auf *Meter²* zurückführen lassen, obwohl der Nutzen der Definition einer Einheit mit dem Namen *Quadratmeter* fraglich ist.

Literatur

Die Namen der Einheiten, Abkürzungen und Werte stammen aus den Veröffentlichungen der Technologieverwaltung des US-Handelsministeriums, des National Institute of Standards and Technology (NIST), in *NIST Special Publication 330: Das Internationale Einheitensystem (SI)* und *NIST Special Publication 811: Leitfaden zur Anwendung des Internationalen Einheitensystems (SI)*. Beide Veröffentlichungen sind (Stand: Dezember 2000) unter http://physics.nist.gov/cuu/Reference/contents.html erhältlich.

Die Ideen hinter der Umsetzung dieses Pakets basieren zum Teil auf Code, der 1993 von Adrian Mariano geschrieben wurde und der eine Dimensionsanalyse von Einheitenzeichenfolgen unter Verwendung von Tabellen fester Größe aus C-Strukturen durchführte. Nachdem Adrians Code Ende der 1990er Jahre verschollen war, tauchte er im Programm "GNU Units" unter http://www.gnu.org/software/units/ wieder auf.

Autoren

Robert W. Techentin

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *units* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright


```
Copyright (c) 2000-2005 Mayo Foundation
```


uri

uri

Stichwörter: uri, fetching information, file, ftp, gopher, http, https, ldap, mailto, news, prospero, rfc 1630, rfc 2255, rfc 2396, rfc 3986, url, wais, www

Name
uri – URI-Dienstprogramme

Übersicht
package require Tcl 8.5 9
package require uri ?**1.2.8**?
uri::setQuirkOption *option* ?*value*?
uri::split *url* ?*defaultscheme*?
uri::join ?*key value* ...?
uri::resolve *base url*
uri::isrelative *url*
uri::geturl *url* ?*options...*?
uri::canonicalize *uri*
uri::register *schemeList script*

Beschreibung
Dieses Paket erfüllt zwei Aufgaben.

Erstens bietet es eine Reihe von Befehlen zur Bearbeitung von URLs/URIs und zum Abrufen der darin festgelegten Daten. Zum Abrufen von Daten analysiert dieses Paket die angeforderte URL/URI und leitet sie anschließend an das entsprechende Paket (**http**, **ftp**, ...) weiter, um die Daten tatsächlich abzurufen. Derzeit sind diese Befehle für die Schemata *http*, *https*, *ftp*, *mailto*, *news*, *ldap*, *ldaps* und *file* definiert. Das Paket **uri::urn** fügt das Schema *urn* hinzu.

Zweitens stellt es reguläre Ausdrücke für eine Reihe von **registered** URL-/URI-Schemas bereit. Zu den registrierten Schemata gehören derzeit *ftp*, *ldap*, *ldaps*, *file*, *http*, *https*, *gopher*, *mailto*, *news*, *wais* und *prospero*. Das Paket **uri::urn** fügt das Schema *urn* hinzu.

Die Befehle des Pakets entsprechen RFC 3986 (https://www.rfc-editor.org/rfc/rfc3986.txt), mit Ausnahme einer Lücke, die sich aus RFC 1630 ergibt und in RFC 3986, Abschnitte 5.2.2 und 5.4.2, beschrieben ist. Diese Lücke erlaubt es, dass eine relative URI ein Schema enthält, wenn dieses mit dem Schema der Basis-URI übereinstimmt, anhand derer sie aufgelöst wird. RFC 3986 empfiehlt, diese Verwendung zu vermeiden.

Befehle
uri::setQuirkOption *option* ?*value*?
: **uri::setQuirkOption** ist ein Zugriffsbefehl für eine Reihe von "Quirk-Optionen". Der Befehl hat dieselbe Semantik wie der Befehl **set**: Wird er mit einem Argument aufgerufen, liest er einen vorhandenen Wert aus; mit zwei Argumenten schreibt er einen neuen Wert. Der Wert einer "Quirk-Option" ist boolesch: Der Wert **false** fordert die Konformität mit RFC 3986, während **true** die Verwendung der Quirk verlangt. Eine Erläuterung der verschiedenen Optionen und ihres Zwecks finden Sie im Abschnitt Quirk-Optionen.

uri::split *url* ?*defaultscheme*?
: **uri::split** nimmt eine *URL* entgegen, dekodiert sie und gibt anschließend eine Liste von Schlüssel-Wert-Paaren zurück, die für **array set** geeignet sind und die Bestandteile der *URL* enthalten. Wenn das Schema in der *URL* fehlt, wird standardmäßig der Wert von *defaultscheme* verwendet, wenn dieser festgelegt wurde, andernfalls *http*. Derzeit werden die Schemata *http*, *https*, *ftp*, *mailto*, *news*, *ldap*, *ldaps* und *file* vom Paket selbst unterstützt.

Informationen dazu, wie dieser Bereich erweitert werden kann, finden Sie im Abschnitt Erweiterung.

Die Menge der Bestandteile einer URL (= die Menge der Schlüssel im zurückgegebenen Dictionary) hängt vom Schema der URL ab. Der einzige Schlüssel, der daher immer vorhanden ist, ist **scheme**. Für die folgenden Schemata sind die Bestandteile und ihre Schlüssel bekannt:

ftp
: **user**, **pwd**, **host**, **port**, **path**, **type**, **pbare**. Das Feld "pbare" ist optional.

http(s)
: **user**, **pwd**, **host**, **port**, **path**, **query**, **fragment**, **pbare**. Das "pbare" ist optional.

file
: **path**, **host**. Der Host ist optional.

mailto
: **user**, **host**. Der Host ist optional.

ldap(s)
: **host**, **port**, **dn**, **attrs**, **scope**, **filter**, **extensions**

news
: Entweder **message-id** oder **newsgroup-name**.

Eine Erläuterung des booleschen Werts **pbare** finden Sie unter den Optionen *NoInitialSlash* und *NoExtraKeys* in Quirk-Optionen.

Die Bestandteile werden als Ausschnitte des Arguments *url* zurückgegeben, ohne Entfernung der Prozent-Kodierung ("URL-Kodierung") oder anderer Anpassungen. Insbesondere unter Windows® ist der **path** in *file* kein gültiger lokaler Dateiname. Weitere Informationen finden Sie unter Beispiele.

uri::join ?*key value*?...

uri::join nimmt eine Liste von Schlüssel-Wert-Paaren entgegen (die beispielsweise von **uri::split** generiert wurden) und gibt die kanonische URL zurück, die diese darstellen. Derzeit werden die Schemata *http*, *https*, *ftp*, *mailto*, *news*, *ldap*, *ldaps* und *file* vom Paket selbst unterstützt. Informationen dazu, wie dieser Bereich erweitert werden kann, finden Sie im Abschnitt Erweiterung.

Die Argumente müssen Ausschnitte einer gültigen URL sein, mit Prozent-Kodierung ("URL-Kodierung") und allen anderen erforderlichen Anpassungen. Insbesondere ist unter Windows der **path** im Schema *file* kein gültiger lokaler Dateiname. Weitere Informationen finden Sie unter Beispiele.

uri::resolve *base url*

uri::resolve löst die festgelegte *URL* relativ zu *base* gemäß RFC 3986 auf. Mit anderen Worten: Eine nicht-relative *url* wird unverändert zurückgegeben, während bei einer relativen *url* die fehlenden Teile aus *base* entnommen und dieser vorangestellt werden. Das Ergebnis dieser Operation wird zurückgegeben. Bei einer leeren *url* ist das Ergebnis *base* ohne deren URI-Fragment (falls vorhanden). Der Befehl ist für die Schemata *http*, *https*, *ftp* und *file* verfügbar.

uri::isrelative *url*

uri::isrelative ermittelt, ob die angegebene *URL* absolut oder relativ ist. Der Befehl steht für

URLs mit beliebigen Schemata zur Verfügung.

uri::geturl *url* ?*options*...?

uri::geturl dekodiert die angegebene *URL* und leitet die Anfrage anschließend an das Paket weiter, das für das in der URL gefundene Schema geeignet ist. Der Befehl geht davon aus, dass das Paket zur Verarbeitung des angegebenen Schemas entweder denselben Namen wie das Schema selbst (einschließlich möglicher Groß- und Kleinschreibung) gefolgt von **::geturl** trägt oder, falls dies nicht der Fall ist, denselben Namen wie das Schema selbst (einschließlich möglicher Groß- und Kleinschreibung) hat. Er geht ferner davon aus, dass das geladene Paket eine Funktion **geturl**-command im Namespace bereitstellt, der denselben Namen wie das Paket selbst trägt. Dieser Befehl wird mit der angegebenen *URL* und allen angegebenen *Optionen* aufgerufen. Derzeit verarbeitet **geturl** selbst keine Optionen.

Hinweis: file-URLs bilden eine Ausnahme von der oben beschriebenen Regel. Sie werden intern verarbeitet.

Es ist nicht möglich, die Ergebnisse des Befehls festzulegen. Sie hängen von der Funktion **geturl**-command für das Schema ab, an das die Anfrage weitergeleitet wurde.

uri::canonicalize *uri*

uri::canonicalize gibt die kanonische Form einer URI zurück. Die kanonische Form einer URI ist eine Form, in der relative Pfade, d. h. "." und "..", aufgelöst wurden. Der Befehl ist für alle URI-Schemas verfügbar, für die die Befehle **uri::split** und **uri::join** vorhanden sind. Der Befehl gibt eine kanonisierte URI zurück, wenn das URI-Schema eine **path**-Komponente enthält (d. h. *http*, *https*, *ftp* und *file*). Bei Schemata, die über die Befehle **uri::split** und **uri::join** verfügen, aber keine **path**-Komponente haben (d. h. *mailto*, *news*, *ldap* und *ldaps*), gibt der Befehl die *uri* unverändert zurück.

uri::register *schemeList script*

uri::register registriert das erste Element von *schemeList* als neues Schema und die übrigen Elemente als Aliase für dieses Schema. Es erstellt den Namespace für das Schema und führt das *Skript* im neuen Namespace aus. Das Skript muss Variablen deklarieren, die für das Schema relevante reguläre Ausdrücke enthalten. Zumindest die Variable **schemepart** muss deklariert werden, da diese dazu dient, die Variablen zu erweitern, die die registrierten Schemata verwalten.

Programme

Zusätzlich zu den oben genannten Befehlen bietet dieses Paket reguläre Ausdrücke zur Erkennung von URLs für eine Reihe von URL-Schemas.

Für jedes unterstützte Schema wird innerhalb des Namensraums *uri* ein Namespace bereitgestellt, der denselben Namen wie das Schema selbst trägt. Dieser enthält die Variable **url** , deren Inhalt ein regulärer Ausdruck zur Erkennung von URLs dieses Schemas ist. Weitere Variablen können reguläre Ausdrücke für Teile von URLs dieses Schemas enthalten.

Die Variable **uri::schemes** enthält eine Liste aller registrierten Schemata. Derzeit sind dies *ftp*, *ldap*, *ldaps*, *file*, *http*, *https*, *gopher*, *mailto*, *news*, *wais* und *prospero*.

Erweiterung

Die Erweiterung der von **uri::split** und **uri::join** unterstützten Schemata ist einfach, da beide

Befehle die Anfrage nicht selbst bearbeiten, sondern sie anhand des Schemas der URL an einen anderen Befehl im *uri*-Namespace weiterleiten.

uri::split und **uri::join** rufen **Split[string totitle]** bzw. **Join[string totitle]** auf.

Die Bereitstellung der Befehle "split" und "join" reicht aus, um die Befehle **uri::canonicalize** und **uri::geturl** zu erweitern (wobei Letzteres von der Verfügbarkeit eines geeigneten Pakets mit einem **geturl**-Befehl abhängt). Um hingegen den Befehl **uri::resolve** auf ein neues Schema zu erweitern, muss der Befehl selbst geändert werden.

Um die Palette der Schemata zu erweitern, für die Musterinformationen verfügbar sind, verwenden Sie den Befehl **uri::register**.

Ein Beispiel für ein Paket, das sowohl Befehle als auch Musterinformationen für ein neues Schema bereitstellt, ist **uri::urn**, das das Schema *urn* hinzufügt.

Quirk-Optionen

Der Wert einer "Quirk-Option" ist ein boolescher Wert: Der Wert **false** fordert die Konformität mit RFC 3986, während **true** die Verwendung des Quirks anfordert. Verwenden Sie den Befehl **uri::setQuirkOption**, um auf die Werte der Quirk-Optionen zuzugreifen.

Quirk-Optionen sind sowohl nützlich, um bei Änderungen an der Befehlsspezifikation Abwärtskompatibilität zu gewährleisten, als auch, um nützliche Funktionen hinzuzufügen, die nicht in den RFC-Spezifikationen enthalten sind. Derzeit sind folgende Quirk-Optionen definiert:

NoInitialSlash

Diese "Besonderheiten"-Option betrifft das führende Zeichen vom **path** (sofern nicht leer) in den Schemata *http*, *https* und *ftp*.

RFC 3986 definiert, dass **path** in einer absoluten URI mit einem führenden "/" beginnen muss, es sei denn, der Wert von **path** ist die leere Zeichenkette. Für das Schema *file* folgen alle Versionen des Pakets **uri** dieser Regel. Die Quirk-Option *NoInitialSlash* gilt nicht für das Schema *file*.

Für die Schemata *http*, *https* und *ftp* definieren Versionen von **uri** vor 1.2.7, dass **path** *KEIN* führendes "/" enthält. Wenn die Quirk-Option *NoInitialSlash* auf **true** gesetzt ist (Standardeinstellung), wird dieses Verhalten auch in Version 1.2.7 angewendet. Um stattdessen die in RFC 3986 definierten Werte für **path** zu verwenden, setzen Sie diese Quirk-Option auf **false**.

Diese Einstellung hat keinen Einfluss auf die Konformität mit RFC 3986. Wenn *NoInitialSlash* auf **true** gesetzt ist, kann der Wert von **path** in den Schemata *http*, *https* oder *ftp* nicht zwischen URIs unterscheiden, bei denen der vollständige "RFC 3986-Pfad" jeweils die leere Zeichenkette "" oder einen einzelnen Schrägstrich "/" ist. Die fehlenden Informationen werden in einem zusätzlichen **uri::split**-Schlüssel **pbare** erfasst.

Der boolesche Wert **pbare** wird definiert, wenn die Besonderheiten *NoInitialSlash* und *NoExtraKeys* die Werte **true** bzw. **false** haben. In diesem Fall gilt: Ist der Wert von **path** die leere Zeichenkette "", ist **pbare true**, wenn der vollständige "RFC 3986-Pfad" "" ist, und **pbare** ist **false**, wenn der vollständige "RFC 3986-Pfad" "/" ist.

Die Verwendung der Quirk-Option *NoInitialSlash* ist eine Frage der persönlichen Vorliebe.

NoExtraKeys

Diese Quirk-Option ermöglicht vollständige Abwärtskompatibilität mit Versionen von **uri** vor 1.2.7, indem der oben beschriebene **uri::split**-Schlüssel **pbare** weggelassen wird (siehe Quirk-Option *NoInitialSlash*). Das Ergebnis ist eine größere Abwärtskompatibilität des Befehls **uri::split** Befehl, jedoch die Unfähigkeit, zwischen URIs zu unterscheiden, bei denen der vollständige "RFC 3986-Pfad" jeweils die leere Zeichenkette "" oder einen einzelnen Schrägstrich "/" ist – d. h. eine geringfügige Abweichung vom RFC 3986.

Wenn die Quirk-Option *NoExtraKeys* auf **false** gesetzt (Standard), gibt der Befehl **uri::split** einen zusätzlichen Schlüssel **pbare** zurück, und die Befehle entsprechen RFC 3986. Wenn die Quirk-Option *NoExtraKeys* auf **true** gesetzt ist, wird der Schlüssel **pbare** nicht definiert, und es besteht keine vollständige Konformität mit RFC 3986.

Die Verwendung der Quirk-Option *NoExtraKeys* wird *NICHT* empfohlen, da sie bei der Einstellung auf **true** die Konformität mit RFC 3986 beeinträchtigt. Die Option ist nur aus Gründen der Kompatibilität enthalten, um mit Code kompatibel zu sein, der für frühere Versionen von **uri** geschrieben wurde und Werte für **path** ohne führendes "/" benötigt, *UND AUCH* unerwartete Schlüssel in den Ergebnissen von **uri::split** nicht tolerieren kann.

HostAsDriveLetter

Wenn Sie das Schema *file* auf der Windows-Plattform verarbeitet, verwenden Versionen von **uri** vor 1.2.7 das Feld **host** zur Darstellung eines Windows-Laufwerksbuchstabens und des darauf folgenden Doppelpunkts sowie das Feld **path** zur Darstellung des Dateinamenspfads nach dem Doppelpunkt. Solche URIs sind ungültig und werden von keinem RFC anerkannt. Wenn die Option *HostAsDriveLetter* auf **true** gesetzt ist, wird dieses Verhalten auch in Version 1.2.7 verwendet. Um unter Windows *file*-URIs zu verwenden, die dem RFC 3986 entsprechen, setzen Sie diese Option auf **false** (Standard).

Die Verwendung dieser Quirk-Option wird *NICHT* empfohlen, da bei der Einstellung **true** die **uri**-Befehle ungültige URIs erwarten und erzeugen. Die Option ist nur aus Gründen der Kompatibilität mit älterem Code enthalten.

RemoveDoubleSlashes

Wenn eine URI durch **uri::canonicalize** kanonisiert wird, wird ihr **path** durch Entfernen der Segmente "." und ".." normalisiert. RFC 3986 schreibt die Entfernung leerer Segmente "" nicht vor (d. h. die Zusammenführung doppelter Schrägstriche, was ein Merkmal der Normalisierung von Dateinamen, nicht jedoch der **path**-Normalisierung von URIs ist): Er behandelt URIs mit überzähligen Schrägstrichen so, als würden sie auf unterschiedliche Ressourcen verweisen. Wenn die Quirk-Option *RemoveDoubleSlashes* auf **true** gesetzt ist (Standardeinstellung), werden leere Segmente aus dem **path** entfernt. Um die Entfernung zu verhindern und damit RFC 3986 zu entsprechen, setzen Sie diese Quirk-Option auf **false**.

Die Verwendung dieser Besonderheiten-Option ist eine Frage der Präferenz. Ein URI mit doppelten Schrägstrichen im Pfad wurde höchstwahrscheinlich versehentlich generiert, ganz sicher dann, wenn er eine direkte Zuordnung zu einer Datei auf einem Server aufweist. In manchen Fällen kann es besser sein, den URI zu bereinigen; in anderen Fällen ist es besser, den URI beizubehalten und den möglichen Fehler vom Server behandeln zu lassen.

Abwärtskompatibilität

Um sich so ähnlich wie möglich wie Versionen von **uri** vor 1.2.7 zu verhalten, legen Sie die folgenden Quirk-Optionen fest:

- **uri::setQuirkOption** *NoInitialSlash* 1
- **uri::setQuirkOption** *NoExtraKeys* 1
- **uri::setQuirkOption** *HostAsDriveLetter* 1
- **uri::setQuirkOption** *RemoveDoubleSlashes* 0

In Code, der die Rückgabe eines zusätzlichen Schlüssels **pbare** durch **uri::split** tolerieren kann, setzen Sie

- **uri::setQuirkOption** *NoExtraKeys* 0

um eine bessere Übereinstimmung mit RFC 3986 zu erreichen.

Neue Designs

Für neue Projekte werden folgende Einstellungen empfohlen:

- **uri::setQuirkOption** *NoInitialSlash* 0
- **uri::setQuirkOption** *NoExtraKeys* 0
- **uri::setQuirkOption** *HostAsDriveLetter* 0
- **uri::setQuirkOption** *RemoveDoubleSlashes* 0|1

Standardwerte

Die Standardwerte für das Paket **uri** in Version 1.2.7 sollen einen Kompromiss zwischen Abwärtskompatibilität und verbesserten Funktionen darstellen. In zukünftigen Versionen des Pakets **uri** können andere Werte gewählt werden.

- **uri::setQuirkOption** *NoInitialSlash* 1
- **uri::setQuirkOption** *NoExtraKeys* 0
- **uri::setQuirkOption** *HostAsDriveLetter* 0
- **uri::setQuirkOption** *RemoveDoubleSlashes* 1

Beispiele

Ein lokaler Windows®-Dateiname wie "**C:\Other Files\startup.txt**" ist nicht geeignet, als **path** -Element einer URI im Schema *file* verwendet zu werden.

Der Tcl-Befehl **file normalize** wandelt die Backslashes in Schrägstriche um. Um einen gültigen **path** für das Scheme-*file* zu erzeugen, muss dem normalisierten Dateinamen "**/**" vorangestellt werden, gefolgt von allen Zeichen, die nicht dem Klammerausdruck **regexp** entsprechen

```
    [a-zA-Z0-9$_.+!*'(,)?:@&=-]
```

muss durch prozentkodierte Kodierung dargestellt werden.

Das Ergebnis in diesem Beispiel lautet "**/C:/Other%20Files/startup.txt**", was ein gültiger Wert für **path** ist.

```
% uri::join path /C:/Other%20Files/startup.txt scheme file

file:///C:/Other%20Files/startup.txt

% uri::split file:///C:/Other%20Files/startup.txt

path /C:/Other%20Files/startup.txt scheme file
```

Auf UNIX®-Systemen beginnen Dateinamen mit "/", was gleichzeitig als Verzeichnisrennzeichen dient. Die einzige Maßnahme, die erforderlich ist, um einen Dateinamen in einen gültigen **path** umzuwandeln, ist die Prozent-Kodierung.

Danksagungen

Originalcode (reguläre Ausdrücke) von Andreas Kupries. Modularisierung durch Steve Ball, ebenso die Funktionen "split", "join" und "resolve". Konformität mit RFC 3986 durch Keith Nash.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *uri* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

urn-scheme

Stichwörter: urn-scheme, rfc 2141, uri, url, urn

Name

uri_urn – URI-Dienstprogramme, URN-Schema

Übersicht

package require Tcl 8.5 9
package require uri::urn ?**1.0.4**?
uri::urn::quote *url*
uri::urn::unquote *url*

Beschreibung

Dieses Paket stellt zwei Befehle zum Ein- und Ausklemmen der unzulässigen Zeichen für URLs unter Verwendung des *urn*-Schemas bereit, registriert das Schema beim Paket **uri**, und stellt interne Hilfsfunktionen bereit, die automatisch von den Befehlen uri::split und uri::join des Pakets **uri** verwendet werden, um URLs nach dem *urn*-Schema zu verarbeiten.

Befehle

uri::urn::quote *url*
Dieser Befehl setzt die Zeichen, die gemäß dem *urn*-Schema (gemäß RFC 2141, Abschnitt 2.2) in der *url* nicht zulässig sind, in Anführungszeichen und gibt die geänderte URL als Ergebnis zurück.

uri::urn::unquote *url*
Dieser Befehl führt das Gegenteil von **::uri::urn::quote** aus. Er nimmt eine *urn*-URL entgegen, entfernt die Anführungszeichen um alle unzulässigen Zeichen und gibt die geänderten URLs als Ergebnis zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *uri* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Netzwerk

uuid

uuid

Stichwörter: uuid, GUID

Name

uuid – Erzeugung und Vergleich von UUIDs

Übersicht

package require Tcl 8.5 9
package require uuid ?**1.0.9**?
::uuid::uuid generate
::uuid::uuid equal *id1 id2*

Beschreibung

Dieses Paket stellt einen Generator für universell eindeutige Identifikatoren (UUID) bereit, die auch als global eindeutige Identifikatoren (GUID) bezeichnet werden. Diese Implementierung folgt dem Spezifikationsentwurf aus (1), obwohl es sich dabei eigentlich um einen abgelaufenen Entwurf handelt.

Befehle

::uuid::uuid generate
Erzeugt eine UUID vom Typ 4, indem eine bestimmte Anzahl von Bits verschiedener Daten, darunter die Uhrzeit und der Hostname, mit einem MD5-Hash-Algorithmus verarbeitet wird.

Gibt die Zeichenfolgenrepräsentation der neuen UUID zurück.

::uuid::uuid equal *id1 id2*

Vergleicht zwei UUIDs und gibt "true" zurück, wenn beide Argumente dieselbe UUID sind.

Beispiele

```
% uuid::uuid generate
b12dc22c-5c36-41d2-57da-e29d0ef5839c
```

Literatur

1. Paul J. Leach, "UUIDs und GUIDs", Februar 1998. (http://www.opengroup.org/dce/info/draft-leach-uuids-guids-01.txt)

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *uuid* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Hashwerte, Prüfsummen und Verschlüsselung

Copyright

```
Copyright (c) 2004, Pat Thoyts <patthoyts@users.sourceforge.net>
```

valtype

cc_amex

Stichwörter: cc_amex, AMEX, American Express, Checking, Testing, Type checking, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name

valtype::creditcard::amex – Validierung einer AMEX-Kreditkartennummer

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::amex ?**1.1**?
valtype::creditcard::amex validate *value*
valtype::creditcard::amex checkdigit *value*

Beschreibung

Dieses Paket implementiert einen SNIT-Validierungstyp für eine AMEX-Kreditkartennummer.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::creditcard::amex validate .... ;# Does nothing
    valtype::creditcard::amex validate .... ;# Throws an error (bad
    American Expresss creditcard number).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::creditcard::amex validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::creditcard::amex checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **CREDITCARD AMEX**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode**-Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID CREDITCARD AMEX CHARACTER
: Der Wert der Eingabe enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine AMEX-Kreditkartennummer handelt.

INVALID CREDITCARD AMEX CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich

kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID CREDITCARD AMEX LENGTH
Der eingegebene Wert hat nicht die richtige Länge, um eine AMEX-Kreditkartennummer zu sein.

INVALID CREDITCARD AMEX PREFIX
Der eingegebene Wert beginnt nicht mit dem/den "magischen Wert(en)", der/die erforderlich ist/sind, damit es sich um eine AMEX-Kreditkartennummer handelt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


cc_discover

Stichwörter: cc_discover, Checking, Discover, Testing, Type checking, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name

valtype::creditcard::discover – Validierung der Discover-Kreditkartennummer

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::discover ?**1.1**?
valtype::creditcard::discover validate *value*
valtype::creditcard::discover checkdigit *value*

Beschreibung

Dieses Paket implementiert einen Snit-Validierungstyp für eine Discover-Kreditkartennummer.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::creditcard::discover validate .... ;# Does nothing
    valtype::creditcard::discover validate .... ;# Throws an error
    (bad Discover creditcard number).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::creditcard::discover validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::creditcard::discover checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **CREDITCARD DISCOVER**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode**-Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID CREDITCARD DISCOVER CHARACTER
: Der Wert der Eingabe enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine Discover-Kreditkartennummer handelt.

INVALID CREDITCARD DISCOVER CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID CREDITCARD DISCOVER LENGTH
: Der eingegebene Wert hat nicht die richtige Länge, um eine Discover-Kreditkartennummer zu sein.

INVALID CREDITCARD DISCOVER PREFIX

Der eingegebene Wert beginnt nicht mit dem/den "magischen Wert(en)", der/die erforderlich ist/sind, damit es sich um eine Discover-Kreditkartennummer handelt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

cc_mastercard

Stichwörter: cc_mastercard, Checking, MasterCard, Testing, Type checking, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name

valtype::creditcard::mastercard – Validierung der Mastercard-Kreditkartennummer

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::mastercard ?**1.1**?
valtype::creditcard::mastercard validate *value*
valtype::creditcard::mastercard checkdigit *value*

Beschreibung

Dieses Paket implementiert einen SNIT-Validierungstyp für eine Mastercard-Kreditkartennummer.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::creditcard::mastercard validate .... ;# Does nothing
    valtype::creditcard::mastercard validate .... ;# Throws an error
    (bad Mastercard creditcard number).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::creditcard::mastercard validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::creditcard::mastercard checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **CREDITCARD MASTERCARD**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode**-Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID CREDITCARD MASTERCARD CHARACTER
: Der Wert der Eingabe enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine Mastercard-Kreditkartennummer handelt.

INVALID CREDITCARD MASTERCARD CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID CREDITCARD MASTERCARD LENGTH
: Der eingegebene Wert hat nicht die richtige Länge, um eine Mastercard-Kreditkartennummer zu sein.

INVALID CREDITCARD MASTERCARD PREFIX
: Der eingegebene Wert beginnt nicht mit den für eine Mastercard-Kreditkartennummer erforderlichen Kennzahlen.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


cc_visa

Stichwörter: cc_visa, Checking, Testing, Type checking, VISA, Validation, Value checking, bank, card for credit, credit card, finance, isA

Name

valtype::creditcard::visa – Validierung der VISA-Kreditkartennummer

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::creditcard::visa ?**1.1**?
valtype::creditcard::visa validate *value*
valtype::creditcard::visa checkdigit *value*

Beschreibung

Dieses Paket implementiert einen SNIT-Validierungstyp für eine VISA-Kreditkartennummer.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::creditcard::visa validate .... ;# Does nothing
    valtype::creditcard::visa validate .... ;# Throws an error (bad
    VISA creditcard number).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::creditcard::visa validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::creditcard::visa checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **CREDITCARD VISA**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode**-Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID CREDITCARD VISA CHARACTER
: Der Wert der Eingabe enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine VISA-Kreditkartennummer handelt.

INVALID CREDITCARD VISA CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID CREDITCARD VISA LENGTH
: Der eingegebene Wert hat nicht die richtige Länge für eine VISA-Kreditkartennummer.

INVALID CREDITCARD VISA PREFIX
: Der eingegebene Wert beginnt nicht mit dem/den "magischen Wert(en)", der/die erforderlich ist/sind, damit es sich um eine VISA-Kreditkartennummer handelt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das

Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Validierung, Typprüfung

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


ean13
Stichwörter: ean13, Checking, EAN, European Article Number, International Article Number, Testing, Type checking, Validation, Value checking, isA

Name
valtype::gs1::ean13 – Validierung für EAN-13

Übersicht
package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::gs1::ean13 ?**1.1**?
valtype::gs1::ean13 validate *value*
valtype::gs1::ean13 checkdigit *value*

Beschreibung
Dieses Paket implementiert einen SNIT-Validierungstyp für einen EAN-13-Code.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::gs1::ean13 validate .... ;# Does nothing
    valtype::gs1::ean13 validate .... ;# Throws an error (bad EAN13).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API
Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::gs1::ean13 validate *value*
Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form –

zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::gs1::ean13 checkdigit *value*
Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **EAN13**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID EAN13 CHARACTER
Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um einen EAN-13-Code handelt.

INVALID EAN13 CHECK-DIGIT
Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID EAN13 LENGTH
Der eingegebene Wert hat nicht die richtige Länge für einen EAN-13-Code.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

iban

Stichwörter: iban, Checking, International Bank Account Number, Testing, Type checking,

Validation, Value checking, bank, finance, isA

Name
valtype::iban – Validierung der IBAN

Übersicht
package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::iban ?**1.8**?
valtype::iban validate *value*
valtype::iban checkdigit *value*

Beschreibung
Dieses Paket implementiert einen SNIT-Validierungstyp für eine IBAN.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::iban validate .... ;# Does nothing
    valtype::iban validate .... ;# Throws an error (bad International
    Bank Account Number).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API
Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::iban validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::iban checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes
Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **IBAN**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID IBAN CHARACTER
: Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine IBAN handelt.

INVALID IBAN CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


imei

Stichwörter: imei, Checking, International Mobile Equipment Identity, Testing, Type checking, Validation, Value checking, cell-phone, isA, mobile phone, phone

Name

valtype::imei – Validierung für IMEI

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::imei ?**1.1**?
valtype::imei validate *value*
valtype::imei checkdigit *value*

Beschreibung

Dieses Paket implementiert einen Snit-Validierungstyp für eine IMEI.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::imei validate .... ;# Does nothing
    valtype::imei validate .... ;# Throws an error (bad International
    Mobile Equipment Identity (IMEI) number).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::imei validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::imei checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **IMEI**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID IMEI CHARACTER
: Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine IMEI handelt.

INVALID IMEI CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich

kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID IMEI LENGTH
Der eingegebene Wert hat nicht die richtige Länge, um eine IMEI zu sein.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


isbn

Stichwörter: isbn, Book Number, Checking, EAN, EAN13, European Article Number, International Article Number, International Standard Book Number, Testing, Type checking, Validation, Value checking, isA

Name

valtype::isbn – Validierung der ISBN

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::isbn ?**1.1**?
valtype::isbn validate *value*
valtype::isbn checkdigit *value*
valtype::isbn 13of *value*

Beschreibung

Dieses Paket implementiert einen SNIT-Validierungstyp für eine ISBN.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss

jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::isbn validate .... ;# Does nothing
    valtype::isbn validate .... ;# Throws an error (bad ISBN).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::isbn validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::isbn checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

valtype::isbn 13of *value*
: Diese Methode erwartet eine 10-stellige ISBN im alten Format und gibt die kanonische, moderne 13-stellige ISBN zurück. Dies wird von **validate** verwendet, um die Eingabe zu kanonisieren, sodass alle Teile des Systems nach der Validierung davon ausgehen können, mit modernen 13-stelligen ISBNs zu arbeiten.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **ISBN**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID ISBN CHARACTER
: Der Wert der Eingabe enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine ISBN handelt.

INVALID ISBN CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID ISBN LENGTH
: Der eingegebene Wert hat nicht die richtige Länge für eine ISBN.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Validierung, Typprüfung

Copyright
```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

luhn
Stichwörter: luhn, Checking, Testing, Type checking, Validation, Value checking, isA

Name
valtype::luhn – Validierung einer reinen Zahlenfolge mit einer LUHN-Prüfziffer

Übersicht
package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn ?**1.1**?
valtype::luhn validate *value*
valtype::luhn checkdigit *value*

Beschreibung
Dieses Paket implementiert einen Snit-Validierungstyp für eine einfache Zahl mit einer LUHN-Prüfziffer.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::luhn validate .... ;# Does nothing
    valtype::luhn validate .... ;# Throws an error (bad luhn
    checkdigit).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen

Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::luhn validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::luhn checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **LUHN**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID LUHN CHARACTER
: Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine reine Zahl mit einer LUHN-Prüfziffer handelt.

INVALID LUHN CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

luhn5

Stichwörter: luhn5, Checking, Testing, Type checking, Validation, Value checking, isA, luhn, luhn-5

Name

valtype::luhn5 – Validierung einer reinen Zahl mit einer LUHN5-Prüfziffer

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::luhn5 ?**1.1**?
valtype::luhn5 validate *value*
valtype::luhn5 checkdigit *value*

Beschreibung

Dieses Paket implementiert einen Snit-Validierungstyp für eine einfache Zahl mit einer LUHN5-Prüfziffer.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::luhn5 validate .... ;# Does nothing
    valtype::luhn5 validate .... ;# Throws an error (bad luhn5
    checkdigit).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::luhn5 validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::luhn5 checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird

keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **LUHN5**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID LUHN5 CHARACTER
: Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine reine Zahl mit einer LUHN5-Prüfziffer handelt.

INVALID LUHN5 CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


usnpi

Stichwörter: usnpi, Checking, NPI, National Provider Identifier, Testing, Type checking, US-NPI, Validation, Value checking, isA, medicare

Name

valtype::usnpi – Validierung für USNPI

Übersicht

package require Tcl 8.5 9

package require snit 2
package require valtype::common
package require valtype::luhn
package require valtype::usnpi ?**1.1**?
valtype::usnpi validate *value*
valtype::usnpi checkdigit *value*

Beschreibung

Dieses Paket implementiert einen Snit-Validierungstyp für eine USNPI.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::usnpi validate .... ;# Does nothing
    valtype::usnpi validate .... ;# Throws an error (bad US National
    Provider Identifier (US-NPI) number).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::usnpi validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::usnpi checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **USNPI**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID USNPI CHARACTER
Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine USNPI handelt.

INVALID USNPI CHECK-DIGIT
Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID USNPI LENGTH
Der Wert hat eine falsche Länge, um eine USNPI zu sein.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


valtype_common

Stichwörter: valtype_common, Checking, Testing, Type checking, Validation, Value checking, isA

Name

valtype::common – Validierung, allgemeiner Code

Übersicht

package require Tcl 8.5 9
package require valtype::common ?**1.1**?
valtype::common::reject *code text*
valtype::common::badchar *code* ?*text*?
valtype::common::badcheck *code* ?*text*?
valtype::common::badlength *code lengths* ?*text*?
valtype::common::badprefix *code prefixes* ?*text*?

Beschreibung

Dieses Paket implementiert eine Reihe gängiger Befehle, die von den Validierungstypen in diesem Modul verwendet werden. Diese Befehle kapseln im Wesentlichen die Auslösung von Validierungsfehlern und stellen sicher, dass ein korrekter **-errorcode** verwendet wird. Siehe

Abschnitt Fehlercodes.

API

valtype::common::reject *code text*
: Der Hauptbefehl dieses Pakets löst einen **INVALID**-Fehler mit dem angegebenen *text* aus. Das erste Argument ist eine Liste von Codes, die den **INVALID** um detaillierte Informationen erweitern.

valtype::common::badchar *code* ?*text*?
: Dieser Befehl löst einen **INVALID CHAR**-Fehler mit dem angegebenen *text* aus. Das erste Argument ist eine Liste von Codes, die weitere Details enthalten. Diese werden zwischen den Codes **INVALID** und **CHARACTER** eingefügt.

valtype::common::badcheck *code* ?*text*?
: Dieser Befehl löst einen **INVALID CHECK-DIGIT**-Fehler mit dem angegebenen *text* aus, sofern vorhanden, ergänzt um den Standardtext "Die Prüfziffer ist falsch". Das erste Argument ist eine Liste von Codes, die Details liefern. Diese werden zwischen den Codes **INVALID** und **CHECK_DIGIT** eingefügt.

valtype::common::badlength *code lengths* ?*text*?
: Dieser Befehl löst einen **INVALID LENGTH**-Fehler mit dem angegebenen *text* aus, sofern vorhanden, ergänzt um den Standardtext "falsche Länge, erwartet wurden ... Zeichen". Das erste Argument ist eine Liste von Codes, die Details liefern. Diese werden zwischen den Codes **INVALID** und **LENGTH** eingefügt. Das Argument *lengths* ist eine Liste der erwarteten Eingabelängen, d. h. dies sind die gültigen Längen.

valtype::common::badprefix *code prefixes* ?*text*?
: Dieser Befehl löst einen **INVALID PREFIX**-Fehler mit dem angegebenen *text* aus, sofern vorhanden, ergänzt um den Standardtext "Falsches Präfix, erwartet wurde ...". Das erste Argument ist eine Liste von Codes, die weitere Details liefern. Diese werden zwischen den Codes **INVALID** und **PREFIX** eingefügt. Das Argument *prefixes* ist eine Liste der erwarteten Eingabeprefixe, d. h., dies sind die gültigen Präfixe.

Fehlercodes

Die von den Befehlen dieses Pakets ausgelösten Fehler verwenden durchweg die Codes **-errorcode** und **INVALID**, um die von ihnen dargestellten Fehler bei der Validierung der Eingaben von pakketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der den Typ selbst beschreibt. Dieser wird vom Aufrufer bereitgestellt. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode** -Fehler, die von diesem Paket ausgelöst werden können, ist unten aufgeführt, wobei **<>** als Platzhalter sowohl für die vom Aufrufer bereitgestellten Typinformationen als auch für die Typbeschreibung dient.

INVALID <> CHARACTER
: Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um einen **<>** handelt.

INVALID <> CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen

Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

INVALID <> LENGTH
Der Wert hat eine falsche Länge, um als **<>** zu gelten.

INVALID <> PREFIX
Der Eingabewert beginnt nicht mit dem/den magischen Wert(en), der/die erforderlich ist/sind, damit es sich um einen **<>** handelt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Validierung, Typprüfung

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


verhoeff

Stichwörter: verhoeff, Checking, Testing, Type checking, Validation, Value checking, isA

Name

valtype::verhoeff – Validierung einer reinen Zahl mit einer VERHOEFF-Prüfziffer

Übersicht

package require Tcl 8.5 9
package require snit 2
package require valtype::common
package require valtype::verhoeff ?**1.1**?
valtype::verhoeff validate *value*
valtype::verhoeff checkdigit *value*

Beschreibung

Dieses Paket implementiert einen Snit-Validierungstyp für eine einfache Zahl mit einer Verhoeff-Prüfziffer.

Ein Validierungstyp ist ein Objekt, mit dem Tcl-Werte einer bestimmten Art validiert werden können. So wird beispielsweise **snit::integer**, ein vom Paket **snit** definierter Validierungstyp, verwendet, um zu überprüfen, ob ein Tcl-Wert eine Ganzzahl ist.

Jeder Validierungstyp verfügt über eine Methode namens **validate**, die zur Durchführung der

Validierung dient. Diese Methode muss ein einziges Argument entgegennehmen, nämlich den zu validierenden Wert; außerdem darf sie nichts unternehmen, wenn der Wert gültig ist, muss jedoch einen Fehler auslösen, wenn der Wert ungültig ist:

```
    valtype::verhoeff validate .... ;# Does nothing
    valtype::verhoeff validate .... ;# Throws an error (bad verhoeff
    checkdigit).
```

Die Methode **validate** gibt bei Erfolg stets den validierten Wert zurück und löst bei einem Fehler den Fehlercode **-errorcode** INVALID aus, möglicherweise zusammen mit zusätzlichen Elementen, die weitere Details zum Problem liefern.

API

Die von diesem Paket bereitgestellte API entspricht der Spezifikation der Snit-Validierungstypen, wie sie in der Dokumentation von *Snit's Not Incr Tcl* zu finden ist.

valtype::verhoeff validate *value*
: Diese Methode überprüft den *value* und gibt ihn – gegebenenfalls in kanonischer Form – zurück, wenn er die Validierung besteht. Wenn der Wert die Validierung nicht besteht, wird ein Fehler ausgelöst.

valtype::verhoeff checkdigit *value*
: Diese Methode berechnet eine Prüfziffer für den *value*. Zuvor wird dieser – mit Ausnahme der Prüfziffer – einer Validierung unterzogen. Wenn der Wert die Validierung nicht besteht, wird keine Prüfziffer berechnet, sondern stattdessen ein Fehler ausgelöst.

Fehlercodes

Wie in der Paketbeschreibung angegeben, verwenden die Fehler, die von den Befehlen dieses Pakets als Reaktion auf Fehler bei der Eingabevalidierung ausgelöst werden, den Code **-errorcode** INVALID, um sich von paketinternen Fehlern zu unterscheiden.

Um detailliertere Informationen darüber zu liefern, warum die Validierung fehlgeschlagen ist, geht der Fehlercode **-errorCode** tatsächlich noch einen Schritt weiter. Zunächst enthält er einen Code, der die Art des Fehlers selbst angibt. In diesem Fall lautet dieser **VERHOEFF**. Darauf folgen Werte, die den Grund für den Fehler näher erläutern. Die vollständige Liste der **-errorCode**-Fehler, die von diesem Paket ausgelöst werden können, lautet wie folgt:

INVALID VERHOEFF CHARACTER
: Der Eingabewert enthielt ein oder mehrere unzulässige Zeichen, d. h. Zeichen, die in der Eingabe nicht vorkommen dürfen, damit es sich um eine reine Zahl mit einer VERHOEFF-Prüfziffer handelt.

INVALID VERHOEFF CHECK-DIGIT
: Die Prüfziffer des eingegebenen Werts ist falsch. Dies deutet in der Regel auf einen Eingabefehler hin, beispielsweise vertauschte oder fehlende Ziffern usw. Selbstverständlich kann es sich bei der Eingabe auch um eine regelrechte Fälschung handeln.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *valtype* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Validierung, Typprüfung

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


virtchannel_base

cat
Stichwörter: cat, concatenation channel, reflected channel, tip 219, virtual channel

Name
tcl::chan::cat – Verkettungskanal

Übersicht
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::core ?**1.1**?
package require tcl::chan::cat ?**1.0.4**?
::tcl::chan::cat *chan...*

Beschreibung
Das Paket **tcl::chan::cat** stellt einen Befehl zur Erstellung von Verkettungskanälen bereit. Dabei handelt es sich um nicht durchsuchbare Kanäle, die eine Liste untergeordneter Kanäle enthalten, deren Inhalte sie der Reihe nach geben, bis alle erschöpft sind. Auf diese Weise stellt der Kanal die Verkettung der Inhalte aller untergeordneten Kanäle dar.

Hinweis: Die erstellten Kanäle übernehmen die Verantwortung für die Kanäle, aus denen sie aufgebaut wurden. Sobald sie einen ihrer Kanäle vollständig ausgeschöpft haben, wird dieser geschlossen. Ebenso führt das Schließen des "cat"-Kanals dazu, dass alle ihm noch untergeordneten Kanäle geschlossen werden.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::core**".

Die Ereignisbehandlung wird an den derzeit aktiven Kanal weitergeleitet.

API
::tcl::chan::cat *chan*...
Dieser Befehl erstellt den Verkettungskanal unter Verwendung aller angegebenen Kanäle und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


facade

Stichwörter: facade, concatenation channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::facade – Fassadenkanal

Übersicht

package require Tcl 8.5 9
package require TclOO
package require logger
package require tcl::chan::core ?**1.1**?
package require tcl::chan::facade ?**1.1**?
::tcl::chan::facade *chan*

Beschreibung

Das Paket **tcl::chan::facade** stellt einen Befehl zur Verfügung, mit dem Fassaden für andere Kanäle erstellt werden können. Dabei handelt es sich um Kanäle, die einen einzigen untergeordneten Kanal besitzen und alle Vorgänge an diesen weiterleiten.

Fassaden dienen in erster Linie der Fehlerbehebung bei Aktionen auf einem Kanal. Zwar könnten die meisten Informationen durch eine virtuelle Kanal-Transformation nachverfolgt werden, doch hat diese keinen Zugriff auf die ereignisbezogenen Operationen; zudem sind sie nur in Tcl 8.6 verfügbar.

Daher handelt es sich hierbei um einen Kanal, der mit Tcl 8.5 genutzt werden kann und Zugriff auf alle Vorgänge innerhalb eines Kanals bietet.

Die auf dem Kanal abgefangenen Aktionen werden über das Paket **logger** protokolliert.

Darüber hinaus bieten die Fassaden die folgenden zusätzlichen Konfigurationsoptionen für Kanäle:

-self

Das TclOO-Objekt, das die Fassade verwaltet.

-fd
Der Griff des untergeordneten, d. h. umwickelten Kanals.

-used
Der Zeitpunkt, zu dem der verpackte Kanal zuletzt von der Fassade gelesen oder beschrieben wurde, gemäß **clock milliseconds**. Der Wert **0** bedeutet, dass auf den untergeordneten Kanal bisher noch gar nicht zugegriffen wurde.

-created
Der Zeitpunkt der Erstellung der Fassade gemäß **clock milliseconds**.

-user
Ein frei definierbarer Wert, der den Nutzer der Fassade und den damit verbundenen Kanal identifiziert.

Von diesen ist lediglich die Option **-user** beschreibbar.

API

::tcl::chan::facade *chan*
Dieser Befehl erstellt den Fassadenkanal um den angegebenen Kanal *chan* herum und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


halfpipe

Stichwörter: halfpipe, callbacks, fifo, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::halfpipe – In-Memory-Kanal, eine Hälfte eines FIFO2

Übersicht

package require Tcl 8.5 9
package require TclOO

package require tcl::chan::events ?**1.1**?
package require tcl::chan::halfpipe ?**1.0.2**?
::tcl::chan::halfpipe ?*-option value...*?
objectCmd **put** *bytes*

Beschreibung

Das Paket "**tcl::chan::halfpipe**" stellt einen Befehl bereit, mit dem eine Hälfte eines "**tcl::chan::fifo2**"-Paares erstellt wird. Das Schreiben in einen solchen Kanal löst eine Reihe von Rückruffunktionen aus, die die Daten anschließend verarbeiten. Dies ähnelt einem Kanal-Handler, verfügt jedoch über eine wesentlich einfachere API.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::halfpipe ?-option *value*...?
: Dieser Befehl erstellt einen Halfpipe-Kanal und konfiguriert ihn mit den Rückruffunktionen, die ausgeführt werden sollen, wenn der Kanal geschlossen wird, Daten in ihn geschrieben wurden oder er leer ist. Eine Liste der Optionen und der zugehörigen Semantik finden Sie im Abschnitt Optionen. Das Ergebnis des Befehls ist eine Liste mit zwei Elementen: dem Handle des neuen Kanals und dem Objektbefehl des Kanalhandlers, in dieser Reihenfolge. Letzteres wird dem Aufrufer übergeben, um ihm Zugriff auf die Methode **put** zum Hinzufügen von Daten zum Kanal zu gewähren.

  Zwei Halfpipes mit etwas Verbindungslogik in den Rückruffunktionen ergeben zusammen ein **tcl::chan::fifo2**.

objectCmd* put *bytes
: Diese Methode des Kanal-Handler-Objekts fügt die Daten *bytes* in den Kanal ein, sodass sie von dort gelesen werden können.

Optionen

-close-command cmdprefix
: Diese Rückruffunktion wird aufgerufen, wenn der Kanal geschlossen wird. Es wird ein einziges Argument übergeben: das Handle des Kanals, der geschlossen wird. Das Ergebnis der Rückruffunktion wird ignoriert.

-write-command cmdprefix
: Diese Rückruffunktion wird aufgerufen, wenn Daten in den Kanal geschrieben werden. Es werden zwei Argumente übergeben: das Handle des Kanals, in den geschrieben wird, und die geschriebenen Daten. Das Ergebnis der Rückruffunktion wird ignoriert.

-empty-command cmdprefix
: Diese Rückruffunktion wird aufgerufen, wenn der Kanal keine Daten mehr zum Lesen enthält. Es wird ein einziges Argument übergeben: das Handle des Kanals.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Kanäle

Copyright

```
Copyright (c) 2009, 2019 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


nullzero
Stichwörter: nullzero, /dev/null, /dev/zero, null, reflected channel, tip 219, virtual channel, zero

Name
tcl::chan::nullzero – Kombination aus Null- und Zero-Kanälen

Übersicht
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::nullzero ?**1.1**?
::tcl::chan::nullzero

Beschreibung
Das Paket **tcl::chan::nullzero** enthält einen Befehl zum Erstellen von Kanälen, bei denen es sich um eine Kombination aus Null- und Null-Geräten handelt. Diese vergessen sofort alles, was an sie geschrieben wird, und geben beim Auslesen einen unendlichen Strom von Nullzeichen zurück.

Die entsprechenden Pakete lauten **tcl::chan::null** und **tcl::chan::zero**.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API
::tcl::chan::nullzero
Dieser Befehl erstellt einen neuen Null-Null-Kanal und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links

in der zweiten Navigationsleiste.

Kategorie
Kanäle

Copyright
```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

randseed
Stichwörter: randseed, /dev/random, merge, random, reflected channel, seed, tip 219, virtual channel

Name
tcl::randomseed – Dienstprogramme für zufällige Kanäle

Übersicht
package require Tcl 8.5 9
package require TclOO
package require tcl::randomseed ?**1.1**?
::tcl::randomseed
::tcl::combine *seed1 seed2*

Beschreibung
Das Paket "**tcl::randomseed**" enthält einige Befehle, die bei der Initialisierung von **tcl::chan::random**-Kanälen helfen.

API
::tcl::randomseed
Dieser Befehl gibt eine Liste von Startwerten zurück, die als Startwertargument für Zufallskanäle geeignet sind. Die Zahlen werden aus der Prozess-ID, der aktuellen Uhrzeit und dem Tcl-Zufallszahlengenerator abgeleitet.

::tcl::combine *seed1 seed2*
Dieser Befehl nimmt zwei Startlisten entgegen und führt sie zu einer einzigen Liste zusammen, indem er sie elementweise mit dem XOR-Operator modulo 256 verknüpft. Wenn die Listen nicht gleich lang sind, wird die kürzere der beiden mit Nullen aufgefüllt, bevor sie mit der anderen zusammengeführt wird.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


std
Stichwörter: std, reflected channel, standard io, stdin, stdout, tip 219, virtual channel

Name
tcl::chan::std – Standards-E/A, Zusammenführung von stdin und stdout

Übersicht
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::core ?**1.1**?
package require tcl::chan::std ?**1.2**?
::tcl::chan::std

Beschreibung
Das Paket **tcl::chan::std** stellt einen Befehl bereit, mit dem ein Standardkanal erstellt wird, der stdin und stdout zu einem einzigen les- und schreibbaren Kanal zusammenfasst. Das Ergebnis lässt sich nicht wie die ursprünglichen Standardkanäle durchsuchbar machen.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::core**".

API
::tcl::chan::std
Dieser Befehl erstellt den Kanal "std" und gibt dessen Handle zurück.

Der Kanal wird nur einmal, beim ersten Aufruf, erstellt, und alle nachfolgenden Aufrufe geben lediglich dieses Handle zurück.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Kanäle

Copyright

```
Copyright (c) 2011 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_fifo

Stichwörter: tcllib_fifo, fifo, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::fifo – In-Memory-FIFO-Kanal

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::fifo ?**1.1**?
::tcl::chan::fifo

Beschreibung

Das Paket **tcl::chan::fifo** stellt einen Befehl zur Verfügung, mit dem Kanäle erstellt werden können, die ausschließlich im Arbeitsspeicher existieren. Der Zugriff erfolgt nach dem FIFO-Prinzip, d. h., die Daten werden in der Reihenfolge aus dem Kanal ausgelesen, in der sie dort geschrieben wurden. Dies entspricht den FIFO-Kanälen des Pakets **Memchan**, mit dem Unterschied, dass dieses in reinem Tcl und nicht in C geschrieben ist. Andererseits ist ist **Memchan** mit Tcl 8.4 und früheren Versionen kompatibel, während dieses Paket Tcl 8.5 oder höher sowie **TclOO** erfordert.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::fifo

Dieser Befehl erstellt einen neuen FIFO-Kanal und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_fifo2

Stichwörter: tcllib_fifo2, connected fifos, fifo, in-memory channel, inter-thread communication, reflected channel, tip 219, virtual channel

Name

tcl::chan::fifo2 – Im Speicher verbundene FIFO-Kanäle

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::halfpipe ?**1.1**?
package require tcl::chan::fifo2 ?**1.1**?
::tcl::chan::fifo2

Beschreibung

Das Paket **tcl::chan::fifo2** stellt einen Befehl bereit, mit dem Paare von Kanälen erstellt werden können, die ausschließlich im Speicher existieren und nach dem FIFO-Prinzip miteinander verbunden sind. Was in die eine Hälfte des Paares geschrieben wird, kann in derselben Reihenfolge aus der anderen Hälfte gelesen werden. Eine besondere Anwendungsmöglichkeit hierfür ist die Kommunikation zwischen Threads, wobei eine Hälfte des Paares in den Thread verschoben wird, mit dem kommuniziert werden soll. Dies entspricht den "fifo2"-Kanälen, die vom Paket **Memchan** bereitgestellt werden, mit dem Unterschied, dass dies in reinem Tcl und nicht in C geschrieben ist. Andererseits ist ist **Memchan** mit Tcl 8.4 und früheren Versionen kompatibel, während dieses Paket Tcl 8.5 oder höher sowie **TclOO** erfordert.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::fifo2

Dieser Befehl erstellt ein neues verbundenes Paar von FIFO-Kanälen und gibt deren Handles als Liste mit zwei Elementen zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_memchan
Stichwörter: tcllib_memchan, in-memory channel, reflected channel, tip 219, virtual channel

Name
tcl::chan::memchan – In-Memory-Kanal

Übersicht
package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::memchan ?**1.0.5**?
::tcl::chan::memchan

Beschreibung
Das Paket **tcl::chan::memchan** stellt einen Befehl zur Verfügung, mit dem Kanäle erstellt werden können, die ausschließlich im Arbeitsspeicher liegen. Diese ermöglichen einen Direktzugriff, d. h., sie sind durchsuchbar. Dies entspricht den memchan-Kanälen des Pakets **Memchan**, mit dem Unterschied, dass dieses in reinem Tcl und nicht in C geschrieben ist. Andererseits ist ist **Memchan** mit Tcl 8.4 und früheren Versionen kompatibel, während dieses Paket Tcl 8.5 oder höher sowie **TclOO** erfordert.

Die entsprechenden Pakete lauten **tcl::chan::string** und **tcl::chan::variable**.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API
::tcl::chan::memchan
Dieser Befehl erstellt einen neuen Memchan-Kanal und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen
Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright

```
Copyright (c) 2009-2017 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_null

Stichwörter: tcllib_null, /dev/null, null, reflected channel, tip 219, virtual channel

Name

tcl::chan::null – Nullkanal

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::null ?**1.1**?
::tcl::chan::null

Beschreibung

Das Paket **tcl::chan::null** stellt einen Befehl zur Erstellung von Nullkanälen bereit, d. h. von schreibgeschützten Kanälen, die alles, was in sie geschrieben wird, sofort wieder vergessen. Dies entspricht den Nullkanälen des Pakets **Memchan**, mit dem Unterschied, dass dieses in reinem Tcl und nicht in C geschrieben ist. Andererseits ist ist **Memchan** mit Tcl 8.4 und früheren Versionen kompatibel, während dieses Paket Tcl 8.5 oder höher sowie **TclOO** erfordert.

Die entsprechenden Pakete lauten **tcl::chan::zero** und **tcl::chan::nullzero**.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::null

Dieser Befehl erstellt einen neuen Nullkanal und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_random

Stichwörter: tcllib_random, /dev/random, random, reflected channel, tip 219, virtual channel

Name

tcl::chan::random – Zufälliger Kanal

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::random ?**1.1**?
::tcl::chan::random *seed*

Beschreibung

Das Paket **tcl::chan::random** stellt einen Befehl zur Verfügung, mit dem zufällige Kanäle erstellt werden können, d. h. schreibgeschützte Kanäle, die beim Auslesen einen unendlichen Strom pseudozufälliger Zeichen geben. Dies ähnelt den Zufallskanälen des Pakets **Memchan**, mit dem Unterschied, dass dieses in reinem Tcl und nicht in C geschrieben ist und zudem einen wesentlich einfacheren Generator verwendet. Andererseits ist **Memchan** mit Tcl 8.4 und früheren Versionen kompatibel, während dieses Paket Tcl 8.5 oder höher sowie TclOO erfordert.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::random *seed*

Dieser Befehl erstellt einen neuen Zufallskanal und gibt dessen Handle zurück. Der Startwert ist eine Liste von Ganzzahlen, die zur Initialisierung des internen Rückkopplungs-Schieberegisters des Generators verwendet wird.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_string

Stichwörter: tcllib_string, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::string – Schreibgeschützter In-Memory-Kanal

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::string ?**1.0.4**?
::tcl::chan::string *content*

Beschreibung

Das Paket **tcl::chan::string** stellt einen Befehl zur Verfügung, mit dem Kanäle erstellt werden können, die ausschließlich im Arbeitsspeicher existieren. Diese ermöglichen einen Direktzugriff, d. h., sie sind durchsuchbar. Im Gegensatz zu den von **tcl::chan::memchan** erstellten Kanälen sind sie jedoch schreibgeschützt; ihr Inhalt wird zum Zeitpunkt der Erstellung bereitgestellt und ist danach unveränderlich.

Die entsprechenden Pakete lauten **tcl::chan::memchan** und **tcl::chan::variable**.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::string *content*

Dieser Befehl erstellt einen neuen Zeichenfolgenkanal und gibt dessen Handle zurück. Der Kanal ermöglicht einen zufälligen, schreibgeschützten Zugriff auf die Zeichenfolge *content*.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_variable

Stichwörter: tcllib_variable, in-memory channel, reflected channel, tip 219, virtual channel

Name

tcl::chan::variable – In-Memory-Kanal unter Verwendung einer Variablen zur Speicherung

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::variable ?**1.0.5**?
::tcl::chan::variable *varname*

Beschreibung

Das Paket **tcl::chan::variable** stellt einen Befehl zur Verfügung, mit dem Kanäle erstellt werden können, die ausschließlich im Arbeitsspeicher liegen. Diese ermöglichen einen Direktzugriff, d. h., sie sind durchsuchbar. Im Gegensatz zu den von **tcl::chan::memchan** erstellten Kanälen werden die Daten jedoch nicht im Kanal selbst verborgen, sondern in einer zugehörigen Variable gespeichert, die zum Zeitpunkt der Erstellung festgelegt wird.

Die entsprechenden Pakete lauten **tcl::chan::memchan** und **tcl::chan::string**.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::variable *varname*

Dieser Befehl erstellt einen neuen Variablenkanal und gibt dessen Handle zurück. Der Inhalt des Kanals wird in der zugehörigen Namespace-Variablen *varname* gespeichert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

tcllib_zero

Stichwörter: tcllib_zero, /dev/zero, reflected channel, tip 219, virtual channel, zero

Name

tcl::chan::zero – Zero Kanal

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::zero ?**1.1**?
::tcl::chan::zero

Beschreibung

Das Paket **tcl::chan::zero** stellt einen Befehl zur Erstellung von Nullkanälen bereit, d. h. von schreibgeschützten Kanälen, die beim Lesen einen unendlichen Strom von Nullzeichen geben. Dies entspricht den Nullkanälen des Pakets **Memchan**, mit dem Unterschied, dass dieses in reinem Tcl und nicht in C geschrieben ist. Andererseits ist ist **Memchan** mit Tcl 8.4 und früheren Versionen kompatibel, während dieses Paket Tcl 8.5 oder höher sowie TclOO erfordert.

Die entsprechenden Pakete lauten **tcl::chan::null** und **tcl::chan::nullzero**.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::zero
: Dieser Befehl erstellt einen neuen Nullkanal und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

textwindow

Stichwörter: textwindow, Tk, reflected channel, text widget, tip 219, virtual channel

Name

tcl::chan::textwindow – Textfenster-Kanal

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::events ?**1.1**?
package require tcl::chan::textwindow ?**1.1**?
::tcl::chan::textwindow *widget*

Beschreibung

Das Paket **tcl::chan::textwindow** stellt einen Befehl bereit, mit dem schreibgeschützte Kanäle erstellt werden können, die mit Text-Widgets verbunden sind. Alles, was in den Kanal geschrieben wird, wird im zugehörigen Widget angezeigt.

Die interne Klasse "**TclOO**", die den Kanal-Handler implementiert, ist eine Unterklasse des Rahmens "**tcl::chan::events**".

API

::tcl::chan::textwindow *widget*

Dieser Befehl erstellt einen neuen Textfenster-Kanal und gibt dessen Handle zurück. Daten, die in diesen Kanal geschrieben werden, werden im zugehörigen *widget* angezeigt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


virtchannel_core

core

Stichwörter: core, reflected channel, tip 219, virtual channel

Name

tcl::chan::core – Grundlegende Unterstützung für reflektierte/virtuelle Kanäle

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::core ?**1.1**?
::tcl::chan::core *objectName*
objectName **initialize** *thechannel mode*
objectName **finalize** *thechannel*
objectName **destroy**

Beschreibung

Das Paket "**tcl::chan::core**" enthält die Klasse "**TclOO**", die allgemeine Funktionen bereitstellt, die praktisch jeder reflektierte oder virtuelle Kanal benötigt (Initialisierung, Finalisierung).

Diese Klasse ist dafür vorgesehen, entweder als Oberklasse einer konkreten Kanal-Klasse verwendet oder in eine solche Klasse eingemischt zu werden.

API für Klassen

::tcl::chan::core *objectName*

Dieser Befehl erstellt ein neues Channel-Core-Objekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *objectName* lautet. Mit diesem Befehl können verschiedene Operationen auf dem Objekt aufgerufen werden, wie im Abschnitt zur Instanz-API beschrieben.

Instanz-API

Die API der Channel-Core-Instanzen stellt lediglich zwei Methoden bereit, die beide den Befehlen des Channel-Handlers entsprechen (siehe dazu TIP 219). Sie sind dafür vorgesehen, von der Objektinstanz aus aufgerufen zu werden, in die der Channel-Core integriert wurde.

objectName* initialize *thechannel mode

Diese Methode implementiert das Standardverhalten für die Methode **initialize** von Kanal-Handlern. Mithilfe von Introspektion ermittelt sie die von der Instanz unterstützten Handler-Methoden und gibt eine Liste mit deren Namen zurück, wie es die Unterstützung für reflektierte Kanäle im Tcl-Kern erwartet.

Zudem speichert sie das Kanal-Handle in einer Instanzvariablen, damit Unterklassen darauf zugreifen können.

objectName* finalize *thechannel

Diese Methode implementiert das Standardverhalten für die Methode **finalize** der Kanalhandler. Sie löscht sich lediglich selbst.

***objectName* destroy**

Durch das Löschen der Kanal-Kerninstanz wird der Kanal geschlossen, für den sie initialisiert wurde; siehe die Methode **initialize**. Wird die Instanz im Rahmen eines Aufrufs von **finalize** gelöscht, geschieht dies nicht, da davon ausgegangen wird, dass der Kanal von Tcl gelöscht wird.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


events

Stichwörter: events, event management, reflected channel, tip 219, virtual channel

Name

tcl::chan::events – Ereignisunterstützung für reflektierte/virtuelle Kanäle

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::chan::core ?**1.1**?
package require tcl::chan::events ?**1.1**?
::tcl::chan::events *objectName*
objectName **finalize** *thechannel*
objectName **watch** *thechannel eventmask*
objectName **allow** *eventname...*
objectName **disallow** *eventname...*

Beschreibung

Das Paket "**tcl::chan::events**" enthält die Klasse "**TclOO**", die ein allgemeines Verhalten implementiert, das von nahezu jedem reflektierten oder virtuellen Kanal benötigt wird, der ereignisgesteuerte E/A unterstützt. Es handelt sich um eine Unterklasse von "**tcl::chan::core**", die deren gesamtes Verhalten übernimmt.

Diese Klasse ist dafür vorgesehen, entweder als Oberklasse einer konkreten Kanal-Klasse verwendet oder in eine solche Klasse eingemischt zu werden.

API für Klassen

::tcl::chan::events *objectName*

Dieser Befehl erstellt ein neues Kanalereignis-Kernobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *objectName* lautet. Mit diesem Befehl können verschiedene

Operationen auf dem Objekt aufgerufen werden, wie im Abschnitt zur Instanz-API beschrieben.

Instanz-API

Die API der Channel-Event-Core-Instanzen stellt lediglich vier Methoden bereit, von denen zwei den Befehlen des Ereignisbehandlers entsprechen (siehe dazu TIP 219) und die anderen beiden von Unterklassen zur Steuerung der Ereignisgenerierung verwendet werden. Erstere erwarten, dass sie von der Objekt-Instanz aufgerufen werden, in die der Kanal-Ereignis-Kern integriert wurde.

objectName* finalize *thechannel

Diese Methode implementiert das Standardverhalten für die Methode "**finalize**" von Kanal-Handlern. Sie überschreibt das von "**tcl::chan::core**" geerbte Verhalten und deaktiviert darüber hinaus jegliche Ereignisgenerierung, bevor sie sich selbst beendet.

objectName* watch *thechannel eventmask

Diese Methode implementiert das Standardverhalten für die Methode **watch** der Kanalhandler. Sie wird vom E/A-System aufgerufen, sobald sich das Interesse an einem Ereignis ändert, und aktualisiert den Instanzzustand, um die Erzeugung der (nicht) relevanten Ereignisse zu aktivieren bzw. zu unterdrücken.

objectName* allow *eventname...

siehe unten ...

objectName* disallow *eventname...

Diese beiden Methoden werden in Unterklassen exportiert, sodass deren Instanzen ihrem Ereigniskern mitteilen können, welche Ereignisse der von ihnen implementierte Kanal erzeugen darf (zulassen) oder nicht erzeugen darf (verbieten). Zusammen mit den Informationen über die vom Tcl-E/A-System angeforderten Ereignisse, die über die Methode **watch** eingehen, kann der Ereigniskern feststellen, welche Ereignisse er erzeugen soll (bzw. nicht erzeugen soll), und entsprechend handeln.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

transformcore

Stichwörter: transformcore, reflected channel, tip 219, virtual channel

Name

tcl::transform::core – Grundlegende Unterstützung für reflektierte/virtuelle Kanal-Transformationen

Übersicht

package require Tcl 8.5 9
package require TclOO
package require tcl::transform::core ?**1.1**?
::tcl::transform::core *objectName*
objectName **initialize** *thechannel mode*
objectName **finalize** *thechannel*
objectName **destroy**

Beschreibung

Das Paket "**tcl::transform::core**" enthält die Klasse "**TclOO**", die das allgemeine Verhalten implementiert, das für nahezu jede reflektierte oder virtuelle Transformation des Kanals erforderlich ist (Initialisierung, Finalisierung).

Diese Klasse ist dafür vorgesehen, entweder als Oberklasse einer konkreten Kanal-Klasse verwendet oder in eine solche Klasse eingemischt zu werden.

API für Klassen

::tcl::transform::core *objectName*

Dieser Befehl erstellt ein neues Transformationskernobjekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *objectName* lautet. Mit diesem Befehl können verschiedene Operationen auf das Objekt angewendet werden, wie im Abschnitt zur Instanz-API beschrieben.

Instanz-API

Die API der Transform-Core-Instanzen stellt lediglich zwei Methoden bereit, die beide den Befehlen des Transform-Handlers entsprechen (siehe dazu TIP 230). Sie sind dafür vorgesehen, von der Objektinstanz aus aufgerufen zu werden, in die der Transform-Core integriert wurde.

objectName* initialize *thechannel mode

Diese Methode implementiert das Standardverhalten für die Methode **initialize** der Transformations-Handler. Mithilfe von Introspektion ermittelt sie die von der Instanz unterstützten Handler-Methoden und gibt eine Liste mit deren Namen zurück, wie es die Unterstützung für reflektierte Transformationen im Tcl-Kern erwartet.

Zudem speichert sie das Kanal-Handle in einer Instanzvariablen, damit Unterklassen darauf zugreifen können.

objectName* finalize *thechannel

Diese Methode implementiert das Standardverhalten für die Methode **finalize** der Kanalhandler. Sie löscht sich lediglich selbst.

***objectName* destroy**

Durch das Löschen der Transformationskern-Instanz wird der Kanal geschlossen, für den die Transformation initialisiert wurde; siehe dazu die Methode **initialize**. Wird die Instanz im Rahmen eines Aufrufs von **finalize** gelöscht, geschieht dies nicht, da davon ausgegangen wird, dass der Kanal und die Transformation von Tcl gelöscht werden.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


virtchannel_transform

adler32

Stichwörter: adler32, channel transformation, checksum, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::adler32 – Adler32-Transformation

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::adler32 ?**1.1**?
::tcl::transform::adler32 *chan* **-option** *value...*

Beschreibung

Das Paket **tcl::transform::adler32** stellt einen Befehl zur Verfügung, der eine Kanal-Transformation erstellt, welche die gelesenen und geschriebenen Bytes unverändert durchlässt (wie **tcl::transform::identity**), jedoch zusätzlich kontinuierlich die Adler-32-Prüfsummen der Daten berechnet, die es in jeder Richtung verarbeitet hat, und diese in Tcl-Variablen speichert, die bei der Erstellung festgelegt wurden.

Zu diesem Modul gehörende Transformationen sind **tcl::transform::counter**, **tcl::transform::crc32**, **tcl::transform::identity** und **tcl::transform::observe**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine

Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::adler32 *chan* -option *value*...

Dieser Befehl erstellt eine Adler32-Prüfsummentransformation auf dem Kanal *chan* und gibt dessen Handle zurück. Die zulässigen Optionen sind

-read-variable varname

Der Wert der Option ist der Name einer globalen Variablen oder einer Variablen im Namespace; dies ist der Speicherort, an dem die Transformation die Adler-32-Prüfsumme der aus dem Kanal gelesenen Daten speichern muss.

Wird kein Wert angegeben oder eine leere Zeichenkette verwendet, wird die Prüfsumme der Lesedirektion nicht gespeichert.

-write-variable varname

Der Wert der Option ist der Name einer globalen Variablen oder einer Variablen im Namespace; dies ist der Speicherort, an dem die Transformation die Adler-32-Prüfsumme der in den Kanal geschriebenen Daten speichern muss.

Wird kein Wert angegeben oder eine leere Zeichenkette verwendet, wird die Prüfsumme der Schreibrichtung nicht gespeichert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


hex

Stichwörter: hex, channel transformation, hexadecimal, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::hex – Transformation der Hexadezimal-Kodierung

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::hex ?**1.1**?
::tcl::transform::hex *chan*

Beschreibung

Das Paket **tcl::transform::hex** enthält einen Befehl, mit dem eine Kanaltransformation erstellt wird, die die in den Kanal geschriebenen Daten hexadezimal codiert und die aus dem Kanal gelesenen Daten wieder decodiert.

Eine damit verbundene Transformation in diesem Modul ist **tcl::transform::base64**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::hex *chan*

Dieser Befehl erstellt eine Hex-Transformation auf dem Kanal *chan* und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


identity

Stichwörter: identity, channel transformation, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::identity – Transformation der Identität

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::identity ?**1.1**?

::tcl::transform::identity *chan*

Beschreibung

Das Paket **tcl::transform::identity** enthält einen Befehl zur Erstellung einer Identitätskanal-Transformation, die nichts anderes tut, als die gelesenen und geschriebenen Bytes unverändert durchzulassen. In einer Anwendung ist dies zwar nicht wirklich nützlich, doch als prototypische Beobachter-Transformation dient der Code als nützlicher Ausgangspunkt für alle anderen Beobachter, die möglicherweise geschrieben werden sollen.

Die in diesem Modul enthaltenen Transformationen, die aus dem Code von "Identity" abgeleitet wurden, lauten **tcl::transform::adler32**, **tcl::transform::counter**, **tcl::transform::crc32** und **tcl::transform::observe**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::identity *chan*

Dieser Befehl erstellt eine Identitätstransformation auf dem Kanal *chan* und gibt deren Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


limitsize

Stichwörter: limitsize, channel transformation, reflected channel, size limit, tip 230, transformation, virtual channel

Name

tcl::transform::limitsize – Eingabebeschränkung

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?

package require tcl::transform::limitsize ?**1.1**?
::tcl::transform::limitsize *chan max*

Beschreibung

Das Paket **tcl::transform::limitsize** enthält einen Befehl zur Erstellung einer Kanaltransformation, die die Anzahl der Zeichen begrenzt, die aus dem Kanal gelesen werden können. Ein Generator für ein künstliches EOF.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::limitsize *chan max*

Dieser Befehl erstellt eine Transformation zur Größenbegrenzung auf dem Kanal *chan* und gibt deren Handle zurück.

max ist die Anzahl der Bytes, die aus dem Kanal gelesen werden können, bevor die Transformation ein EOF signalisiert. Hinweis: Durch das Entfernen der Transformation wird auch das von ihr generierte EOF gelöscht.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


observe

Stichwörter: observe, channel transformation, observer, reflected channel, stream copy, tip 230, transformation, virtual channel

Name

tcl::transform::observe – Beobachter-Transformation, Stream-Kopie

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::observe ?**1.1**?

::tcl::transform::observe *chan logw logr*

Beschreibung

Das Paket **tcl::transform::observer** stellt einen Befehl zur Verfügung, mit dem eine Kanal-Transformation erstellt wird, die die gelesenen und geschriebenen Bytes unverändert weiterleitet (wie bei **tcl::transform::identity**), jedoch zusätzlich die in jeder Richtung empfangenen Daten in Kanäle kopiert, die bei der Erstellung festgelegt wurden.

Zu diesem Modul gehörende Transformationen sind **tcl::transform::adler32**, **tcl::transform::counter**, **tcl::transform::crc32** und **tcl::transform::identity**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::observe *chan logw logr*

Dieser Befehl erstellt eine Beobachtertransformation auf dem Kanal *chan* und gibt deren Handle zurück. Die Kanal-Handles *logr* und *logw* dienen als Ziel für die Kopie der Daten.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


rot

Stichwörter: rot, caesar cipher, channel transformation, cipher, decryption, encryption, reflected channel, rot13, tip 230, transformation, virtual channel

Name

tcl::transform::rot – Rot-Verschlüsselung

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::rot ?**1.1**?
::tcl::transform::rot *chan key*

Beschreibung

Das Paket **tcl::transform::rot** enthält einen Befehl zum Erstellen einer Kanal-Transformation, die eine einfache Verschlüsselung (beim Schreiben) und Entschlüsselung (beim Lesen) der Buchstaben durchführt. Der verwendete Algorithmus ist die Caesar-Chiffre, wobei es sich bei "rot13" um eine spezielle Variante davon handelt.

Eine damit verbundene Transformation in diesem Modul ist **tcl::transform::otp**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::rot *chan key*

Dieser Befehl erstellt eine Rotationsverschlüsselungstransformation für den Kanal *chan* und gibt dessen Handle zurück.

Der Parameter "*key*" legt fest, um wie viele Stellen die Zeichen im Alphabet gedreht werden, und wird auf den Bereich "0...25" umgewandelt.

Beachten Sie, dass diese Transformation nur Bytes in den Bereichen ASCII 65...90 sowie 97...122, d. h. die Groß- und Kleinbuchstaben "A...Z" und "a...z". Alle anderen Bytes werden unverändert weitergeleitet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


spacer

Stichwörter: spacer, channel transformation, reflected channel, spacing, tip 230, transformation, virtual channel

Name

tcl::transform::spacer – Einsetzen und Entnehmen von Zwischenstücken

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::spacer ?**1.1**?
::tcl::transform::spacer *chan n* ?*space*?

Beschreibung

Das Paket **tcl::transform::spacer** enthält einen Befehl zum Erstellen einer Kanal-Transformation, die den in den Kanal geschriebenen Daten Leerzeichen hinzufügt und diese Leerzeichen aus den aus dem Kanal gelesenen Daten entfernt.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::spacer *chan n ?space?*

Dieser Befehl erstellt eine Spacer-Transformation auf dem Kanal *chan* und gibt deren Handle zurück.

Die Zeichenfolge *space* wird alle *n* geschriebenen Datenbytes eingefügt; auf der Leseseite erfolgt der Vorgang in umgekehrter Reihenfolge, wobei die Abstände entfernt werden. Wenn *space* nicht festgelegt wird, wird standardmäßig ein einzelnes Leerzeichen (ASCII 32) verwendet.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


tcllib_zlib

Stichwörter: tcllib_zlib, channel transformation, compression, decompression, reflected channel, tip 230, tip 234, transformation, virtual channel, zlib

Name

tcl::transform::zlib – (De-)Komprimierung mit zlib

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::zlib ?**1.1**?
::tcl::transform::zlib *chan* ?*level*?

Beschreibung

Das Paket **tcl::transform::zlib** enthält einen Befehl, mit dem eine Kanal-Transformation erstellt wird, die die geschriebenen Daten mit zlib komprimiert und beim Lesen wieder dekomprimiert.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::zlib *chan* ?*level*?

Dieser Befehl erstellt eine zlib-Komprimierungstransformation für den Kanal *chan* und gibt dessen Handle zurück.

Der Parameter *level* legt fest, wie intensiv die Komprimierung erfolgen soll, wobei die Werte von **0** bis **9** reichen; der Standardwert ist **4**.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


vt_base64

Stichwörter: vt_base64, base64, channel transformation, reflected channel, tip 230, tip 317, transformation, virtual channel

Name

tcl::transform::base64 – Transformation der Base64-Kodierung

Übersicht

package require Tcl 8.6 9

package require tcl::transform::core ?**1.1**?
package require tcl::transform::base64 ?**1.1**?
::tcl::transform::base64 *chan*

Beschreibung

Das Paket "**tcl::transform::base64**" enthält einen Befehl, mit dem eine Kanal-Transformation erstellt wird, die die in den Kanal geschriebenen Daten in Base64-Code kodiert und die aus dem Kanal gelesenen Daten wieder entschlüsselt.

Eine damit verbundene Transformation in diesem Modul ist **tcl::transform::hex**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::base64 ***chan***

Dieser Befehl erstellt eine Base64-Transformation auf dem Kanal *chan* und gibt dessen Handle zurück.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


vt_counter

Stichwörter: vt_counter, channel transformation, counter, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::counter – Transformation

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::counter ?**1.1**?
::tcl::transform::counter *chan* **-option** *value...*

Beschreibung

Das Paket **tcl::transform::counterr** stellt einen Befehl bereit, der eine Kanal-Transformation erstellt, welche die gelesenen und geschriebenen Bytes unverändert weiterleitet (wie **tcl::transform::identity**), jedoch zusätzlich die Bytes zählt, die in jeder Richtung verarbeitet wurden, und diese Zählwerte in Tcl-Variablen speichert, die bei der Erstellung festgelegt werden.

Zu diesem Modul gehörende Transformationen sind **tcl::transform::adler32**, **tcl::transform::crc32**, **tcl::transform::identity** und **tcl::transform::observe**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::counter *chan* -option *value*...

Dieser Befehl erstellt eine Zählertransformation auf dem Kanal *chan* und gibt deren Handle zurück. Die zulässigen Optionen sind

-read-variable varname

Der Wert der Option ist der Name einer globalen Variablen oder einer Variablen im Namespace; dies ist der Speicherort, an dem die Transformation die Byteanzahl der aus dem Kanal gelesenen Daten speichern muss.

Wird kein Wert angegeben oder eine leere Zeichenkette verwendet, wird der Zähler der Lese-Richtung nicht gespeichert.

-write-variable varname

Der Wert der Option ist der Name einer globalen Variablen oder einer Variablen im Namespace; dies ist der Speicherort, an dem die Transformation die Byteanzahl der in den Kanal geschriebenen Daten speichern muss.

Wird kein Wert angegeben oder eine leere Zeichenkette verwendet, wird der Zähler der Schreibrichtung nicht gespeichert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

vt_crc32

Stichwörter: vt_crc32, channel transformation, checksum, crc32, reflected channel, tip 230, transformation, virtual channel

Name

tcl::transform::crc32 – CRC32-Transformation

Übersicht

package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::crc32 ?**1.1**?
::tcl::transform::crc32 *chan* **-option** *value...*

Beschreibung

Das Paket **tcl::transform::crc32** stellt einen Befehl zur Verfügung, der eine Kanal-Transformation erstellt, welche die gelesenen und geschriebenen Bytes unverändert durchlässt (wie **tcl::transform::identity**), jedoch zusätzlich kontinuierlich die CRC32-Prüfsummen der Daten berechnet, die es in jeder Richtung verarbeitet hat, und diese in Tcl-Variablen speichert, die bei der Erstellung festgelegt wurden. Bei der betreffenden Prüfsumme handelt es sich um die CRC32-Prüfsumme von zlib.

Zu diesem Modul gehörende Transformationen sind **tcl::transform::adler32**, **tcl::transform::counter**, **tcl::transform::identity** und **tcl::transform::observe**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API

::tcl::transform::crc32 *chan* -option *value*...

Dieser Befehl erstellt eine CRC32-Prüfsummen-Transformation für den Kanal *chan* und gibt dessen Handle zurück. Die zulässigen Optionen sind

-read-variable varname

Der Wert der Option ist der Name einer globalen Variablen oder einer Variablen im Namespace; dies ist der Speicherort, an dem die Transformation die CRC32-Prüfsumme der aus dem Kanal gelesenen Daten speichern muss.

Wird kein Wert angegeben oder eine leere Zeichenkette verwendet, wird die Prüfsumme der Lesedirektion nicht gespeichert.

-write-variable varname

Der Wert der Option ist der Name einer globalen Variablen oder einer Variablen im Namespace; dies ist der Speicherort, an dem die Transformation die CRC32-Prüfsumme der in den Kanal geschriebenen Daten speichern muss.

Wird kein Wert angegeben oder eine leere Zeichenkette verwendet, wird die Prüfsumme der Schreibrichtung nicht gespeichert.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Kanäle

Copyright

```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


vt_otp
Stichwörter: vt_otp, channel transformation, cipher, decryption, encryption, one time pad, otp, reflected channel, tip 230, transformation, virtual channel, xor

Name
tcl::transform::otp – Verschlüsselung mittels Einmal-Pad

Übersicht
package require Tcl 8.6 9
package require tcl::transform::core ?**1.1**?
package require tcl::transform::otp ?**1.1**?
::tcl::transform::otp *chan keychanw keychanr*

Beschreibung
Das Paket **tcl::transform::otp** enthält einen Befehl zum Erstellen einer Kanal-Transformation, die extern bereitgestellte Einmal-Chiffrierschlüssel verwendet, um die Verschlüsselung (beim Schreiben) und die Entschlüsselung (beim Lesen) durchzuführen.

Eine damit verbundene Transformation in diesem Modul ist **tcl::transform::rot**.

Die interne Klasse "**TclOO**", die den Handler für Transformationen implementiert, ist eine Unterklasse des Rahmens "**tcl::transform::core**".

API
::tcl::transform::otp *chan keychanw keychanr*

Dieser Befehl erstellt eine auf dem Kanal *chan* basierende Verschlüsselungstransformation nach dem One-Time-Pad-Prinzip und gibt deren Handle zurück.

Die beiden Kanäle *keychanw* und *keychanr* enthalten die One-Time-Pads für die Schreib- bzw. Leserichtung. Ihre Inhalte werden gelesen und mit den Bytes, die in den Kanal geschrieben bzw. aus ihm gelesen werden, XOR-verknüpft.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *virtchannel* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Kanäle

Copyright


```
Copyright (c) 2009 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


websocket

websocket

Stichwörter: websocket, http, internet, net, rfc 6455

Name

websocket – Tcl-Implementierung des WebSocket-Protokolls

Übersicht

package require Tcl 8.6 9
package require http 2.7
package require logger
package require sha1
package require base64
package require websocket ?**1.6**?
::websocket::open *url handler* ?*options*?
::websocket::send *sock type* ?*msg*? ?*final*?
::websocket::server *sock*
::websocket::live *sock path cb* ?*proto*?
::websocket::test *srvSock cliSock path* ?*hdrs*? ?*qry*?
::websocket::upgrade *sock*
::websocket::takeover *sock handler* ?*server*?
::websocket::conninfo *sock what*
::websocket::find ?*host*? ?*port*?
::websocket::configure *sock args*
::websocket::loglevel ?*loglvl*?
::websocket::close *sock* ?*code*? ?*reason*?

Beschreibung

HINWEIS: DIESE DOKUMENTATION BEFINDET SICH NOCH IN DER ENTWICKLUNG...

Die WebSocket-Bibliothek ist eine reine Tcl-Implementierung der WebSocket-Spezifikation, die sowohl die Anforderungen von Clients als auch von Servern abdeckt. WebSockets bieten eine Möglichkeit, eine normale HTTP-Verbindung zu einer langlebigen und kontinuierlichen binären oder textbasierten Kommunikation zwischen Client und Server auszubauen. Die Bibliothek bietet eine hochrangige Schnittstelle zum Empfangen und Senden von Daten gemäß RFC 6455 (Version 13 des Protokolls) und entlastet die Aufrufer von allen notwendigen Aufgaben der Protokollaufteilung und -wiederzusammensetzung. Sie implementiert die im Standard festgelegte Ping-Funktion sowie die entsprechenden Steuerungsmöglichkeiten. Pings werden vom Server initiiert und gewährleisten die Aufrechterhaltung der Verbindung über Heimnetzwerke (NAT) hinweg. Die Bibliothek verfügt über eine Reihe von Introspektionsfunktionen, um den aktuellen Status der Verbindung abzufragen, aber bei Bedarf auch Benachrichtigungen über eingehende Pings zu empfangen. Schließlich enthält die Bibliothek eine Reihe von Hilfsprozeduren, um die Aktualisierung des Handshakes in bestehenden Webservern zu erleichtern.

Im Mittelpunkt der Bibliothek steht die Prozedur **websocket::takeover**, die einen regulären Socket übernimmt und ihn als WebSocket behandelt, wodurch alle erforderlichen Protokoll-Framing-, Paketierungs- und Wiederzusammensetzungsvorgänge auf Servern und Clients durchgeführt werden. Die Prozedur nimmt zudem einen Handler entgegen – einen Befehl, der jedes Mal aufgerufen wird, wenn ein (möglicherweise wieder zusammengesetztes) Paket von der Gegenstelle zur Zustellung an den ursprünglichen Aufrufer bereitsteht. Obwohl der Befehl **websocket::takeover** vom Paket exportiert wird, wird er in Anwendungen nur selten aufgerufen, da das Paket andere Befehle bereitstellt, die speziell auf die Anforderungen von Clients und Servern zugeschnitten sind.

In der Regel stellen Clients eine Verbindung zu einem Remote-Server her, indem sie dem Befehl **websocket::open** eine WebSocket-URL (mit den Schemata *ws:* oder *wss:*) sowie den oben beschriebenen Handler übergeben. Die Prozedur zum Öffnen der Verbindung ist eine Hülle um die neuesten Implementierungen von http::geturl: Es sorgt dafür, dass der innerhalb der HTTP-Bibliothek erstellte Socket zur Wiederverwendung offen bleibt, entfernt ihn jedoch aus der (internen) Zuordnung bekannter Sockets, um ihn für eigene Zwecke zu nutzen.

Die Server registrieren sich zunächst mithilfe des Befehls **::websocket::server** und richten mithilfe des Befehls **::websocket::live** eine Reihe von Handlern für Pfade ein. Anschließend sollten sie bei jeder eingehenden Client-Verbindung die eingehende Anfrage mit dem Befehl **::websocket::test** prüfen, um festzustellen, ob es sich um eine Upgrade-Anfrage handelt, und mit dem Befehl **::websocket::upgrade** den abschließenden Handshake durchführen, um die Socket-Verbindung unter die Kontrolle der WebSocket-Bibliothek und ihrer zentralen Prozedur zu stellen.

Neben diesen Hauptbefehlen bietet das Paket eine Reihe von Befehlen zur Introspektion und für grundlegende Operationen an den WebSockets, die es verwaltet. Da WebSocket-Verbindungen von langer Dauer sind, erfolgt der Großteil der weiteren Kommunikation mit der Bibliothek über Rückruffunktionen, d. h. Befehle, die immer dann ausgelöst werden, wenn wichtige Ereignisse innerhalb der Bibliothek eintreten, vor allem jedoch, wenn Daten über einen WebSocket empfangen wurden.

Callbacks

Eine Reihe von Befehlen der Bibliothek nimmt den Befehl "handler" als Argument entgegen; dabei handelt es sich um einen Befehl, der bei Empfang von Daten, aber auch bei wichtigen Ereignissen innerhalb der Bibliothek oder bei Ereignissen, die durch vom Gegenüber gesendete Steuermeldungen ausgelöst werden, aufgerufen wird. Bei jeder ausgeführten Rückruffunktion werden die folgenden Argumente angehängt:

sock
Die Kennung des WebSockets, wie sie beispielsweise von **::websocket::open** gegeben wird

type
Ein Textfeld, das den Inhalt des Ereignisses oder der Meldung beschreibt, kann einen der folgenden Werte annehmen

text
Vollständiger SMS-Text

binary
Vollständige Binärnachricht

ping
Eingehende Ping-Nachricht

pong
Antwort auf eine eingehende Ping-Nachricht

connect
Benachrichtigung über eine erfolgreiche Verbindung zum Server

disconnect
Trennung von der Gegenstelle

close
Verbindung wird geschlossen

timeout
Meldung über ein Zeitlimit bei der Verbindung

error
Meldung eines Fehlerzustands

msg
Enthält die Daten der Nachricht, sofern dies relevant ist, d. h. wenn der *type* **text**, **binary**, **ping** oder **pong** lautet und sofern Daten vorhanden sind.

API

::websocket::open *url handler* ?*options*?
Dieser Befehl wird in Clients verwendet, um eine WebSocket-Verbindung zu einem entfernten, WebSocket-fähigen HTTP-Server herzustellen. Die als Argument in *url* angegebene URL sollte mit "ws:" oder "wss:" beginnen, da dies die WebSocket-Entsprechungen von "http:" und "https:" sind. Der *handler* ist ein Befehl, der bei Datenempfang oder bei wichtigen Ereignissen während der Lebensdauer des WebSockets aufgerufen wird. **::websocket::open** gibt einen Socket zurück, der sowohl als Kennung des WebSockets als auch des physischen Low-Level-Sockets zum Server dient. Dieser Socket kann in einer Reihe weiterer Befehle zur Introspektion oder zur Steuerung des Verhaltens der Bibliothek verwendet werden. Da es sich im Wesentlichen um einen Wrapper um den Befehl **::http::geturl** handelt, bietet dieser Befehl größtenteils denselben Satz von durch Bindestriche getrennten Optionen wie

::http::geturl. Im Folgenden sind die Optionen dokumentiert, die sich von **::http::geturl** unterscheiden und spezifisch für die WebSocket-Bibliothek sind.

-headers
Diese Option wird unterstützt, wobei zu beachten ist, dass in der Bibliothek intern eine Reihe von Headern automatisch hinzugefügt wird, um mit dem Server den Handshake für die Umwandlung des Sockets von einem regulären HTTP-Socket in einen WebSocket durchführen zu können.

-validate
Diese Option wird nicht unterstützt, da sie für WebSockets keinen wirklichen Sinn ergibt.

-handler
Diese Option wird intern von der WebSocket-Bibliothek verwendet und kann nicht genutzt werden.

-command
Diese Option wird intern von der WebSocket-Bibliothek verwendet und kann nicht genutzt werden.

-protocol
Diese Option legt eine Liste von Anwendungsprotokollen fest, über die ein Handshake mit dem Server hergestellt werden soll. Diese Protokolle können dem Server dabei helfen, anwendungsspezifische Funktionen auszulösen. Die Option http::geturl -protocol wird intern von der WebSocket-Bibliothek verwendet und kann nicht genutzt werden.

-timeout
Diese Option wird unterstützt, wird jedoch als Teil der Bibliothek implementiert, um eine Reihe abschließender Bereinigungen zu ermöglichen.

::websocket::send *sock type* ?*msg*? ?*final*?
Dieser Befehl sendet ein Fragment oder eine Steuermeldung an die Gegenstelle des durch *sock* identifizierten WebSockets. Der in *type* angegebene Nachrichtentyp kann entweder eine Ganzzahl gemäß der Spezifikation oder (vorzugsweise) eine der folgenden Zeichenfolgen sein, bei denen Groß- und Kleinschreibung keine Rolle spielt: "text", "binary" oder "ping". Der Inhalt der an die Gegenstelle zu sendenden Nachricht ist in *msg* enthalten, und die Fragmentierung der Nachricht wird durch die Einstellung des Arguments *final* auf einen Wert ungleich "true" ermöglicht, wobei zu beachten ist, dass der Typ jedes Fragments dann identisch sein muss. Der Befehl gibt die Anzahl der tatsächlich gesendeten Bytes zurück oder -1 im Fehlerfall. Schwerwiegende Fehler, beispielsweise wenn *sock* keinen bekannten WebSocket identifiziert oder wenn die Verbindung noch nicht stabil ist, führen zu Fehlern, die abgefangen werden müssen.

::websocket::server *sock*
Dieser Befehl registriert den (Accept-)Socket *sock* als Kennung für einen HTTP-Server, der WebSockets unterstützt. Die Pfade, auf denen dieser Server auf eingehende Verbindungen wartet, sollten mithilfe von **::websocket::live** deklariert werden.

::websocket::live *sock path cb* ?*proto*?
Diese Prozedur registriert Rückruffunktionen, die auf einem mit **::websocket::server** registrierten, WebSocket-kompatiblen Server ausgeführt werden, sobald ein Client eine Verbindung zu einem übereinstimmenden Pfad und Protokoll herstellt. *sock* ist der lauschenden Socket des WebSocket-kompatiblen Servers, der mit **::websocket::server**

deklariert wurde. *path* ist ein Pfad im Glob-Stil, der mit der Client-Anfrage abgeglichen werden soll, wann immer diese eingeht. *cb* ist der Befehl für die Rückruffunktion (siehe "Callbacks"). *proto* ist ein Protokollnamens-Matcher im Glob-Stil.

::websocket::test *srvSock cliSock path* ?*hdrs*? ?*qry*?

Diese Prozedur prüft, ob es sich bei der Verbindung eines ankommenden Clients über den Socket *cliSock* und den Pfad *path* um die Eröffnung eines WebSocket-Streams auf einem bekannten Server *srvSock* handelt. Die eingehende Anfrage wird nicht sofort verarbeitet, sondern es wird zunächst ein (temporärer) Kontext für die eingehende Verbindung erstellt. Dies ermöglicht es dem Servercode, bei Bedarf eine Reihe von Aktionen durchzuführen, bevor die WebSocket-Stream-Verbindung aktiv wird. Die Analyse erfolgt durch Auswertung des Inhalts der Header *hdrs*, die eine Dictionary-Liste der HTTP-Header der eingehenden Client-Verbindung enthalten sollten. Der Befehl gibt **1** zurück, wenn es sich um eine eingehende WebSocket-Upgrade-Anfrage handelt, andernfalls **0**.

::websocket::upgrade *sock*

Rüstet den Socket *sock*, der von **::websocket::test** als WebSocket-Verbindungsanfrage eingestuft wurde, auf einen echten WebSocket auf, wie er von dieser Bibliothek erkannt wird. Infolgedessen wird der erforderliche Handshake für die Verbindung an den Client gesendet, und der Befehl sorgt dafür, dass während der Laufzeit des WebSockets entsprechende Rückruffunktionen ausgeführt werden, insbesondere unter Verwendung der in **::websocket::live** beschriebenen Spezifikationen.

::websocket::takeover *sock handler* ?*server*?

Übernimmt den bereits geöffneten Socket *sock*, um das Senden und Empfangen von WebSocket-Frames auf Basis dieses Sockets zu implementieren. Die Prozedur sorgt dafür, dass *handler* zurückgerufen wird, sobald Nachrichten, Steuermeldungen oder andere wichtige interne Ereignisse empfangen werden oder eintreten. *server* ist standardmäßig auf **0** gesetzt und kann auf **1** (oder einen booleschen Wert, der zu "true" ausgewertet wird) gesetzt werden, um festzulegen, dass es sich um einen WebSocket innerhalb eines Servers handelt. Abgesehen von protokollspezifischen Besonderheiten sollten Server ihre Clients in regelmäßigen Abständen anpingen, um die Verbindung stets offen zu halten. Wenn *server* auf "true" gesetzt ist, sorgt die Bibliothek dafür, dass diese Pings automatisch gesendet werden.

::websocket::conninfo *sock what*

Bietet Aufrufern einige Funktionen zur Selbstauswertung, um bestimmte halbinterne Informationen über eine bestehende WebSocket-Verbindung abzurufen. Je nach Wert des Arguments *what* gibt die Prozedur die folgenden Informationen zurück:

peername

Name (bevorzugt) oder IP-Adresse der Gegenstelle.

sockname

oder **name** Name oder IP-Adresse des lokalen Endgeräts.

closed

1, wenn die Verbindung geschlossen ist, andernfalls **0**

client

1, wenn es sich um eine Client-WebSocket-Verbindung handelt, andernfalls **0**

server

1, wenn es sich um eine Server-WebSocket-Verbindung handelt, andernfalls **0**

type

server, wenn es sich bei der Verbindung um einen Server-WebSocket handelt, andernfalls **client**.

handler
Der dem WebSocket zugeordnete Handler-Befehl

state
Der Status des WebSockets, der einer der folgenden sein kann:

CONNECTING
Die Verbindung zur Gegenstelle wird gerade hergestellt.

CONNECTED
Die Verbindung zum Gegenüber ist hergestellt.

CLOSED
Die Verbindung wurde geschlossen.

::websocket::find ?*host*? ?*port*?
Durchsucht die bestehenden WebSocket-Verbindungen nach solchen, die den als Parameter übergebenen Filtern für Hostnamen und Portnummern entsprechen. Bei dieser Suche wird die Gegenstelle berücksichtigt, und das Argument *host* wird sowohl mit dem Hostnamen (sofern verfügbar) als auch mit der IP-Adresse der Gegenstelle abgeglichen. Sowohl das Argument *host* als auch das Argument *port* sind Filter für den String-Abgleich im Glob-Stil und sind standardmäßig auf * gesetzt, d. h., sie finden Übereinstimmungen mit jedem Host und/oder jeder Portnummer.

::websocket::configure *sock args*
Dieser Befehl akzeptiert eine Reihe von durch Bindestriche eingeleiteten Optionen (sowie deren Werte), um das Verhalten einer bestehenden WebSocket-Verbindung zu konfigurieren. Die folgenden Optionen werden erkannt (sie können auf den kleinsten gemeinsamen Nenner verkürzt werden):

-keepalive
Dies ist die Anzahl der Sekunden zwischen den einzelnen Keepalive-Pings, die über die Verbindung gesendet werden. Eine Null oder eine negative Zahl deaktiviert diese Funktion effektiv. Auf Servern ist der Standardwert für **-keepalive** auf 30 Sekunden festgelegt, während auf Clients keine Pings initiiert werden.

-ping
Dies ist der Text, der bei den automatisierten Pings verwendet wird. Standardmäßig ist dieser Text eine leere Zeichenkette, was zu einem leeren Ping führt.

::websocket::loglevel ?*loglvl*?
Legt die Protokollierungsstufe der Bibliothek fest oder rufen Sie diese ab; die Standardstufe ist error. Die Protokollierung basiert auf dem Logger-Modul, und die Bibliothek verwendet die folgenden Stufen: **debug**, **info**, **notice**, **warn** und **error**. Wird diese Prozedur ohne Argument aufgerufen, gibt sie lediglich die aktuelle Protokollierungsstufe zurück. Andernfalls sollte *loglvl* die gewünschte Protokollierungsstufe enthalten.

::websocket::close *sock* ?*code*? ?*reason*?
Schließt eine WebSocket-Verbindung, die direkt oder indirekt an **::websocket::takeover** übergeben wurde, ordnungsgemäß. Die Prozedur sendet optional den *code* und den beschreibenden *reason* als Teil des Schließungs-Handshakes. Es stehen geeignete

Standardwerte zur Verfügung, sodass für eine Reihe bekannter Codes entsprechende Gründe zurückgesendet werden. Es werden nur die ersten 125 Zeichen einer Begründungszeichenfolge beibehalten und als Teil des Handshakes gesendet. Die bekannten Codes lauten:

1000
Normalerweise geschlossen (der Standard-*code*, wenn kein Code angegeben wurde).

1001
Endpunkt wird abgeschafft

1002
Protokollfehler

1003
Inkompatibler Datentyp empfangen

1006
Fehlverschluss

1007
Die empfangenen Daten stimmen nicht mit dem Typ überein

1008
Verstoß gegen die Richtlinien

1009
Die empfangene Nachricht ist zu groß

1010
Fehlende Erweiterung

1011
Unerwarteter Zustand

1015
TLS-Handshake-Fehler

Beispiele

Das folgende Skript stellt einen Client dar, der eine WebSocket-Verbindung zu einem Echo-Dienst herstellt, 400 ms wartet, um sicherzustellen, dass die Verbindung tatsächlich hergestellt wurde, und eine einzelne Textnachricht sendet, die vom Echo-Dienst zurückgesendet werden soll. In einem realen Beispiel würde wahrscheinlich die Rückruffunktion "**connect**" verwendet werden, um festzustellen, wann die Verbindung zum Remote-Server hergestellt wurde, und erst zu diesem Zeitpunkt würden Daten gesendet werden. Abschließend schließt das Skript die Verbindung.

```
package require websocket
::websocket::loglevel debug
proc handler { sock type msg } {
    switch -glob -nocase -- $type {
    co* {
        puts "Connected on $sock"
    }
    te* {
        puts "RECEIVED: $msg"
```

```
    }
    cl* -
    dis* {
    }
    }

}
proc test { sock } {
    puts "[::websocket::conninfo $sock type] from
    [::websocket::conninfo $sock sockname] to [::websocket::conninfo
    $sock peername]"

    ::websocket::send $sock text "Testing, testing..."
    after 2000 ::websocket::close $sock
}
set sock [::websocket::open ws://ws.ifelse.io/ handler]
after 400 test $sock
vwait forever
```

Beispiele für Code für einen WebSocket-Server finden Sie im Tcllib-Verzeichnis "examples/websocket".

TLS-Sicherheitshinweise

Dieses Paket nutzt das **TLS**-Paket, um die Sicherheit für **https**-URLs und andere Socket-Verbindungen zu gewährleisten.

Entscheidungen wie die Auswahl der zu unterstützenden Protokolle und der zu verwendenden Verschlüsselungsalgorithmen fallen jedoch weder in den Zuständigkeitsbereich von **TLS** noch in den des Pakets. Solche Entscheidungen liegen in der Verantwortung der jeweiligen Anwendung, die das Paket nutzt, und werden wahrscheinlich auch von den Servern beeinflusst, mit denen die Anwendung kommuniziert.

Angesichts des kürzlich von Google entdeckten POODLE-Angriffs werden beispielsweise viele Server die Unterstützung für das SSLv3-Protokoll deaktivieren. Um dieser Änderung Rechnung zu tragen, müssen die Anwendungen, die **TLS** verwenden, gepatcht werden – nicht dieses Paket und auch nicht **TLS** selbst. Ein solcher Patch kann so einfach sein wie die allgemeine Aktivierung der **tls1**-Unterstützung, wie im folgenden Beispiel gezeigt.

```
    package require tls
    tls::init -tls1 1 ;# forcibly activate support for the TLS1
    protocol

    ... your own application code ...
```

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *websocket* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Netzwerk

wip

wip
Stichwörter: wip, interpreter, list, word

Name
wip – Wort-Interpreter

Übersicht
package require Tcl 8.5 9
package require wip ?**2.3**?
package require snit ?**1.4**?
package require struct::set
::wip *wipName engine arg...*
def *name*
def *name method_prefix*
wipName *option* ?*arg arg* ...?
wip::dsl ?*suffix*?
wipName **def** *name* ?*method_prefix*?
wipName **defl** *names*
wipName **defd** *dict*
wipName **deflva** *name...*
wipName **defdva** (*name method_prefix*)...
wipName **undefl** *names*
wipName **undefva** *name...*
wipName **unknown** *cmdprefix*
wipName **runl** *wordlist*
wipName **run** *word...*
wipName **run_next**
wipName **run_next_while** *acceptable*
wipName **run_next_until** *rejected*
wipName **run_next_if** *acceptable*
wipName **run_next_ifnot** *rejected*
wipName **next**
wipName **peek**
wipName **peekall**
wipName **insertl** *at wordlist*
wipName **replacel** *wordlist*

wipName **pushl** *wordlist*
wipName **addl** *wordlist*
wipName **insert** *at word...*
wipName **replace** *word...*
wipName **push** *word...*
wipName **add** *word...*

Beschreibung

Dieses Paket stellt einen Mikro-Interpreter für Listen von Wörtern bereit. Darauf aufbauende domänenspezifische Sprachen haben ein wenig ein Forth-Gefühl, wobei der Eingabestrom in Wörter zerlegt wird und jede weitere Strukturierung der jeweiligen Sprache überlassen bleibt. Im Wesentlichen liegt hier nur die zentrale Dispatch-Schleife vor und keinerlei tatsächliche Befehle, sodass es sich nur um ein Forth-Gefühl handelt und nicht um ein echtes Forth.

Die Idee basiert auf dem Prozessor "**treeql**" von Colin McCormack, der so modifiziert wurde, dass er weniger Standardcode in den Implementierungen der Befehle erfordert – was jedoch wahrscheinlich zu Lasten der Ausführungsgeschwindigkeit geht. Zudem ist auch die Schnittstelle zwischen Prozessorkern und Befehlen komplexer.

Allgemeines Verhalten

Wortinterpreter verfügen über eine Zuordnung zwischen den Namen der Sprachbefehle, die sie erkennen sollen, und den Methoden in der engine, die dafür aufgerufen werden sollen, sowie gegebenenfalls über feste Argumente für diese Methoden. Diese Zuordnung ist weitgehend statisch; es ist jedoch möglich, sie während der Ausführung einer Wortliste (= Programm) zu ändern.

Zum Zeitpunkt der Definition eines Sprachbefehls nutzt der Wort-Interpreter die Introspektionsfunktionen von **snit**, um die Anzahl der Argumente zu ermitteln, die von der Methode der *engine* erwartet werden, und zusammen mit der Anzahl der festen Argumente, die im Methodenpräfix der Zuordnung angegeben sind, ist dann bekannt, wie viele Argumente der Sprachbefehl erwartet. Dies ist die *Arität* des Befehls. Methoden mit variabler Argumentanzahl (d. h. mit dem letzten Argument namens *args*) sind *nicht* zulässig und führen dazu, dass der Wort-Interpreter bereits bei der Definition einen Fehler auslöst.

Die Fähigkeiten von **snit** wurden genannt; das *engine*-Objekt kann jedoch auf beliebige Weise geschrieben sein, solange es die Methode **info args** versteht, die einen Methodennamen entgegennimmt und die Liste der Argumente dieser Methode zurückgibt.

Bei der Ausführung einer Wortliste (auch Programm genannt) wird das erste Wort stets als Name eines Sprachbefehls genommen, die folgenden Wörter als dessen Argumente, entsprechend der *Arität* des Befehls. Befehls- und Argumentwörter werden aus der Liste entfernt, und anschließend wird die zugehörige Methode der *engine* mit den Argumentwörtern ausgeführt. Der Vorgang wiederholt sich danach mit dem dann ersten Wort der Liste.

Die Methoden, die die Sprachbefehle implementieren, haben möglicherweise uneingeschränkten Zugriff auf die Wortliste und dürfen sie nach eigenem Ermessen bearbeiten.

1. Das bedeutet beispielsweise, dass wir Methoden mit variabler Argumentanzahl zwar nicht direkt spezifizieren können, diese jedoch nach ihren festen Argumenten weitere Wörter verarbeiten können, bevor sie zur Ausführungsschleife zurückkehren. Dies kann durch ihre festen Argumente gesteuert werden.

2. Eine weitere Möglichkeit besteht darin, die Methode **run_next** und ihre Varianten zu verwenden, um Befehle auszuführen, die auf den aktuellen Befehl folgen, und so die Ausführungsreihenfolge zu ändern.
3. Die Ausführung lässt sich durch die Verwendung der Zugriffsmethoden des Programms weiter anpassen; diese ermöglichen es einer Befehlsimplementierung, die verbleibende Liste von Wörtern zu ändern (Wörter einfügen, ersetzen, voranstellen oder anhängen), ohne diese sofort auszuführen.
4. Zuletzt speichern und stellen die grundlegenden **run**-Methoden bei ihrer Verwendung eine vorhandene Wortliste wieder her und ermöglichen so die rekursive Verwendung innerhalb von Befehlsimplementierungen.

API für Klassen

Der Hauptbefehl des Pakets lautet:

::wip *wipName engine arg...*

Der Befehl erstellt ein neues Wort-Interpreter-Objekt mit einem zugehörigen globalen Tcl-Befehl, dessen Name *wipName* lautet. Wurde jedoch die Zeichenfolge **%AUTO%** als Objektname verwendet, erzeugt das Paket einen eigenen eindeutigen Namen für das Objekt.

Die *engine* ist das Objekt, an das der Wort-Interpreter alle erkannten Befehle weiterleitet, und die *arg*umente sind eine Wortliste, die eine anfängliche Zuordnung von Sprachwörtern zu Methoden der *engine* definiert.

Die erkannte Sprache dieser Wortliste ist

def *name*

Definiert *name* als Befehl der Sprache, der einer gleichnamigen Methode der *engine* zugeordnet wird.

def *name method_prefix*

Definiert *name* als Befehl der Sprache, der der im *method_prefix* genannten Methode der *engine* zugeordnet wird.

Der zurückgegebene Befehl kann verwendet werden, um verschiedene Operationen auf dem Objekt auszuführen. Er hat die folgende allgemeine Form:

wipName *option* ?*arg arg ...*?

Option und die *arg*s bestimmen das genaue Verhalten des Befehls.

Das Paket exportiert darüber hinaus den Befehl:

wip::dsl ?*suffix*?

Dieser Befehl ist für die Verwendung innerhalb von snit-Typen vorgesehen, die einen oder mehrere wip-Interpreter als Komponente nutzen. Die Verwendung innerhalb der Typdefinition installiert den Großteil des Standardcodes, der zum Einrichten und Nutzen eines Wort-Interpreters nötig ist.

Es wird eine Komponente namens *wip* sowie eine Methode **wip_setup** zu deren Initialisierung installiert. Diese Methode muss innerhalb des Konstruktors des Typs aufgerufen werden, der den Wort-Interpreter verwendet. Darüber hinaus wird eine Reihe von Prozeduren installiert, die die Objekt-API des Wort-Interpreters den Methoden des Typs direkt zur Verfügung stellen, ohne dass die Komponente angegeben werden muss.

Hinweis: Dadurch wird die zu interpretierende Sprache nicht installiert und kann auch nicht installiert werden, d. h. die Zuordnung von Wörtern zu Methoden der *engine*.

Es ist möglich, mehrere Wort-Interpreter-Komponenten innerhalb eines Typs zu instanziieren, indem unterschiedliche Suffixe als Argumente für den Befehl verwendet werden. In diesem Fall ändert sich der Name der Komponente zu \'wip_**$suffix**', der Einrichtungsbefehl lautet \'wip_**$suffix**_setup' und alle Prozeduren erhalten ebenfalls das Suffix \'_**$suffix**'.

Objekt-API

Für Wortinterpreter-Objekte sind folgende Befehle möglich:

***wipName* def *name* ?*method_prefix*?**
Definiert einen Sprachbefehl *name* und ordnet ihn der Methode zu, die im *method_prefix* der *engine* genannt ist. Wenn der Name des *method_prefix* nicht angegeben ist, ist er einfach der Name des Sprachbefehls.

wipName* defl *names
Definiert eine Reihe von Sprachbefehlen, die über die Liste der *names* festgelegt werden und die alle den gleichnamigen Methoden der *engine* zugeordnet sind.

wipName* defd *dict
Definiert eine Reihe von Sprachbefehlen, die über das Dictionary *dict* mit Namen und Methodenpräfixen festgelegt werden.

***wipName* deflva *name*...**
Wie die Methode **defl**, die Liste der Namen wird jedoch über mehrere Argumente angegeben.

***wipName* defdva (*name method_prefix*)...**
Wie die Methode **defd**, das Dictionary der Namen und Methodenpräfixe wird jedoch über mehrere Argumente angegeben.

wipName* undefl *names
Entfernt die angegebene Reihe von Sprachbefehlen aus der Zuordnung.

***wipName* undefva *name*...**
Wie die Methode **undefl**, die Liste der Namen wird jedoch über mehrere Argumente angegeben.

wipName* unknown *cmdprefix
Legt den Handler für unbekannte Wörter auf *cmdprefix* fest. Dieses Befehlspräfix nimmt ein Argument entgegen, das aktuelle Wort, und löst entweder einen Fehler aus oder gibt das Ergebnis der Ausführung des Wortes zurück, wie es der Handler definiert. Der Standard-Handler löst lediglich einen Fehler aus.

wipName* runl *wordlist
Behandelt die Wortliste in *wordlist* als Programm und führt die enthaltenen Befehle nacheinander aus. Das Ergebnis des zuletzt ausgeführten Befehls wird als Ergebnis dieses Befehls zurückgegeben.

Die *wordlist* wird im Objekt gespeichert, damit die anderen *run*-Methoden sowie die allgemeinen Programm-Zugriffsmethoden (siehe unten) darauf zugreifen können. Eine zuvor gespeicherte Wortliste wird während der Ausführung dieser Methode gesichert und vor der Rückgabe wiederhergestellt. Das ermöglicht die rekursive Ausführung von Wortlisten

innerhalb von Wortlisten.

wipName* run *word...
Wie die Methode **runl**, die Liste der auszuführenden Wörter wird jedoch über mehrere Argumente angegeben.

***wipName* run_next**
Low-Level-Methode. Ermittelt das nächste Wort in der Wortliste sowie dessen Argumente und führt es anschließend aus. Das Ergebnis des ausgeführten Worts ist das Ergebnis dieser Methode.

Sie steht zur Verwendung innerhalb von Befehlsimplementierungen zur Verfügung. Die Methoden **run** und **runl** nutzen sie, um Wörter so lange auszuführen, bis ihre Wortliste erschöpft ist.

wipName* run_next_while *acceptable
Low-Level-Methode. Ruft die Methode **run_next** auf, solange das nächste Wort zur Menge der *acceptable* Wörter gehört und das Programm nicht leer ist. Das Ergebnis des zuletzt ausgeführten Befehls wird als Ergebnis dieses Befehls zurückgegeben.

Diese Methode steht zur Verwendung innerhalb von Befehlsimplementierungen zur Verfügung, um die Ausführungsreihenfolge zu ändern.

wipName* run_next_until *rejected
Low-Level-Methode. Ruft die Methode **run_next** auf, bis das nächste Wort zur Menge der *rejected* Wörter gehört und das Programm nicht leer ist. Das Ergebnis des zuletzt ausgeführten Befehls wird als Ergebnis dieses Befehls zurückgegeben.

Diese Methode steht zur Verwendung innerhalb von Befehlsimplementierungen zur Verfügung, um die Ausführungsreihenfolge zu ändern.

wipName* run_next_if *acceptable
Low-Level-Methode. Ruft die Methode **run_next** auf, wenn das nächste Wort zur Menge der *acceptable* Wörter gehört und das Programm nicht leer ist. Das Ergebnis des zuletzt ausgeführten Befehls wird als Ergebnis dieses Befehls zurückgegeben.

Diese Methode steht zur Verwendung innerhalb von Befehlsimplementierungen zur Verfügung, um die Ausführungsreihenfolge zu ändern.

wipName* run_next_ifnot *rejected
Low-Level-Methode. Ruft die Methode **run_next** auf, wenn das nächste Wort nicht zur Menge der *rejected* Wörter gehört und das Programm nicht leer ist. Das Ergebnis des zuletzt ausgeführten Befehls wird als Ergebnis dieses Befehls zurückgegeben.

Diese Methode steht zur Verwendung innerhalb von Befehlsimplementierungen zur Verfügung, um die Ausführungsreihenfolge zu ändern.

***wipName* next**
Gibt das nächste Wort im Programm zurück. Das Wort wird dabei ebenfalls entfernt.

***wipName* peek**
Gibt das nächste Wort im Programm zurück, ohne es zu entfernen.

***wipName* peekall**

Gibt das verbleibende Programm vollständig zurück.

wipName* insertl *at wordlist
Grundlegende Programm-Zugriffsmethode. Fügt die angegebene *wordlist* in das Programm ein, unmittelbar vor dem Wort an der Position *at*. Die Positionen werden ab **zero** gezählt.

wipName* replacel *wordlist
Grundlegende Programm-Zugriffsmethode. Ersetzt das gesamte gespeicherte Programm durch die angegebene *wordlist*.

wipName* pushl *wordlist
Programm-Zugriffsmethode. Die angegebene *wordlist* wird an den Anfang des verbleibenden Programms gesetzt. Entspricht

```
$wip insertl 0 $wordlist
```

wipName* addl *wordlist
Programm-Zugriffsmethode. Die angegebene *wordlist* wird an das Ende des verbleibenden Programms angehängt. Entspricht

```
$wip insertl end $wordlist
```

wipName* insert *at word...
Ähnlich wie bei der Methode **insertl**, mit dem Unterschied, dass die Wörter über mehrere Argumente festgelegt werden.

wipName* replace *word...
Wie die Methode **setl**, die Wörter werden jedoch über mehrere Argumente angegeben.

wipName* push *word...
Ähnlich wie bei der Methode **pushl**, mit dem Unterschied, dass die Wörter über mehrere Argumente festgelegt werden.

wipName* add *word...
Ähnlich wie bei der Methode **addl**, mit dem Unterschied, dass die Wörter über mehrere Argumente festgelegt werden.

Beispiele

Noch keine Beispiele.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *wip* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Programmierwerkzeuge

Copyright


```
Copyright (c) 2007-2010 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


yaml

huddle

Stichwörter: huddle, data exchange, exchange format, json, parsing, text processing, yaml

Name

huddle – Erstellen und Bearbeiten eines "huddle"-Objekts

Übersicht

package require Tcl 8.5 9
package require huddle ?**0.5**?
huddle create *key value* ?*key value* ...?
huddle list ?*value value* ...?
huddle number *number*
huddle string *string*
huddle boolean *expression to evaluate as true or false*
huddle true
huddle false
huddle null
huddle get *object key* ?*key* ...?
huddle gets *object key* ?*key* ...?
huddle set *objectVar key* ?*key* ...? *value*
huddle remove *object key* ?*key* ...?
huddle combine *object1 object2* ?*object3* ...?
huddle equal *object1 object2*
huddle append *objectVar key value* ?*key value* ...?
huddle append *objectVar value* ?*value* ...?
huddle keys *object*
huddle llength *object*
huddle type *object* ?*key key...*?
huddle strip *object*
huddle jsondump *object* ?*offset*? ?*newline*? ?*begin_offset*?
huddle compile *spec data*
huddle isHuddle *object*
huddle checkHuddle *object*
huddle to_node *object* ?*tag*?
huddle wrap *tag src*
huddle call *tag command args*
huddle addType *callback*
callback *command* ?*args*?
setting

get_sub *src key*
strip *src*
set *src key value*
remove *src key value*

Beschreibung

Huddle stellt ein generisches, Tcl-basiertes Serialisierungs- bzw. Zwischenformat bereit. Derzeit wird jeder Knoten in ein Tag mit einfachen Typinformationen eingeschlossen.

Wenn die Huddle-Notation in andere Serialisierungsformate wie JSON oder YAML konvertiert wird, werden diese Typinformationen verwendet, um die richtige Notation auszuwählen. Und wenn die Huddle-Notation von JSON/YAML/... in Huddle konvertiert wird, kann deren Notation verwendet werden, um den richtigen Huddle-Typ auszuwählen.

Auf diese Weise kann "Huddle" als gemeinsames Vermittlungsformat dienen.

```
huddle-format: >
  {HUDDLE {huddle-node}}
huddle-node: >
  {tag content}
each content of tag means:
  s: (content is a) string
  L: list, each sub node is a huddle-node
  D: dict, each sub node is a huddle-node
confirmed:
  - JSON
  - YAML(generally, but cannot discribe YAML-tags)
limitation:
  - cannot discribe aliases from a node to other node.
```

Das **huddle**-Paket gibt Daten als Tcl-**dict** zurück. Für die Verwendung ist entweder das **dict** -Paket oder Tcl 8.5 erforderlich.

Befehle

huddle create *key value ?key value ...?*
Erstellt ein "Huddle"-Objekt als Dict. Es kann weitere "Huddle"-Objekte enthalten.

huddle list ?*value value ...*?
Erstellt ein "Huddle"-Objekt als Liste. Es kann andere "Huddle"-Objekte enthalten.

huddle number *number*
Erstellt ein "Huddle"-Objekt als Zahl.

huddle string *string*
Erstellt ein "Huddle"-Objekt als Zeichenfolge.

huddle boolean *expression to evaluate as true or false*
Erstellt ein Huddle-Objekt als Booleschen Wert, der einen Ausdruck als "wahr" oder "falsch" auswertet –

huddle true
Erstellt ein "Huddle"-Objekt mit dem booleschen Wert "true".

huddle false

Erstellt ein "Huddle"-Objekt mit dem booleschen Wert "false".

huddle null

Erstellt ein "Huddle"-Objekt als Null.

huddle get *object key ?key ...?*

Fast dasselbe wie **dict get**. Ruft ein Unterobjekt aus dem Huddle-Objekt ab. *key* kann als Index der Huddle-Liste verwendet werden.

huddle gets *object key ?key ...?*

Holt ein Unterobjekt aus dem "Huddle"-Objekt, ohne Zusatzdaten.

huddle set *objectVar key ?key ...? value*

Fast dasselbe wie **dict set**. Legt ein Unterobjekt des Huddle-Objekts fest. *key* kann als Index der Huddle-Liste verwendet werden.

huddle remove *object key ?key ...?*

Fast dasselbe wie **dict remove**. Entfernt ein Unterobjekt aus dem Huddle-Objekt. *key* kann als Index der Huddle-Liste verwendet werden.

huddle combine *object1 object2 ?object3 ...?*

Zusammenführen der angegebenen Huddle-Objekte.

```
% set aa [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}
% set bb [huddle create a k l m]
HUDDLE {D {a {s k} l {s m}}}
% huddle combine $aa $bb
HUDDLE {D {a {s k} c {s d} l {s m}}}
```

huddle equal *object1 object2*

Vergleicht zwei Huddle-Objekte rekursiv. Gibt 1 zurück, wenn sie gleich sind, andernfalls 0.

```
% set aa [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}
% set bb [huddle create c d a b]
HUDDLE {D {c {s d} a {s b}}}
% huddle equal $aa $bb
1
```

huddle append *objectVar key value ?key value ...?*

siehe unten ...

huddle append *objectVar value ?value ...?*

Unterelemente anhängen. Bei Dictionaries werden Schlüssel und Wert angegeben, bei Listen die Werte.

```
% set aa [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}
% huddle append aa a k l m
HUDDLE {D {a {s k} c {s d} l {s m}}}
% set bb [huddle list i j k l]
HUDDLE {L {{s i} {s j} {s k} {s l}}}
% huddle append bb g h i
HUDDLE {L {{s i} {s j} {s k} {s l} {s g} {s h} {s i}}}
```

huddle keys *object*
Genau wie bei **dict keys**.

huddle llength *object*
Entspricht **llength**.

huddle type *object* ?*key key...*?
Gibt den Elementtyp zurück, der durch die Schlüssel festgelegt ist. Wenn "key" nicht angegeben ist, gibt es den Typ des Wurzelknotens zurück.

string
Der Knoten ist eine TCL-Zeichenkette.

dict
Der Knoten ist ein Dict.

list
Der Knoten ist eine Liste.

number
Der Knoten ist eine Zahl.

boolean
Der Knoten ist ein boolescher Wert.

null
Der Knoten ist ein Nullwert.

```
% huddle type {HUDDLE {s str}}
string
% huddle type {HUDDLE {L {{s a} {s b} {s c}}}}
list
% huddle type {HUDDLE {D {aa {s b} cc {s d}}}} cc
string
```

huddle strip *object*
Alle Tags wurden entfernt. In normale Tcl-Listen/Dictionaries konvertiert.

huddle jsondump *object* ?*offset*? ?*newline*? ?*begin_offset*?
Einen JSON-Stream aus dem Huddle-Objekt ausgeben.

offset ""
Beginn-Offset als Leerzeichen " ".

```
# normal output has some indents. some strings are escaped.
% huddle jsondump {HUDDLE {L {{L {{s i} {s baa} {s \\k} {L {{s 1.0}
    {s true} {s /g} {s h}}} {L {{s g}}}}} {s t}}}}
[
  [
    "i",
    "baa",
    "\\k",
    [
      1.0,
      true,
      "\/g",
      "h"
    ],
    ["g"]
  ],
  "t"
]
# stripped output
% huddle jsondump {HUDDLE {D {dd {D {bb {D {a {s baa} c {s {d
a}}}} cc {D {g {s h}}}}} ee {D {i {s j} k {s 1} j {s { m\a}}}}}}} ""
    ""
{"dd": {"bb": {"a": "baa","c": "d\na"},"cc": {"g": "h"}},"ee": {"i":
    "j","k": 1,"j": " m\\a"}}
```

huddle compile *spec data*

Erstellt ein "Huddle"-Objekt aus einfachen Tcl-Werten. *spec* ist wie folgt definiert:

string
: Die Daten sind einfach eine Zeichenkette.

list
: "data" ist eine TCL-Liste aus Zeichenketten

dict
: "data" ist ein TCL-Dict aus Zeichenketten

list list
: "data" ist eine TCL-Liste von Listen

list dict
: "data" ist eine TCL-Liste aus Dictionaries

dict xx list
: "data" ist ein TCL-Dict, bei dem der Wert des Schlüssels "xx" eine TCL-Liste ist.

dict * list
: "data" ist ein TCL-Dict aus Listen; "*data*" sind einfache TCL-Werte.

```
% huddle compile {dict * list} {a {1 2 3} b {4 5}}
HUDDLE {D {a {L {{s 1} {s 2} {s 3}}} b {L {{s 4} {s 5}}}}}
% huddle compile {dict * {list {dict d list}}} {a {{c 1} {d {2 2 2}
    e 3}} b {{f 4 g 5}}}
HUDDLE {D {a {L {{D {c {s 1}}} {D {d {L {{s 2} {s 2} {s 2}}} e {s
    3}}}}} b {L {{D {f {s 4} g {s 5}}}}}}}
```

huddle isHuddle *object*
Wenn *Objekt* ein "Huddle" ist, gibt es 1 zurück. Andernfalls gibt es 0 zurück.

huddle checkHuddle *object*
Wenn *Objekt* kein Huddle ist, wird ein Fehler ausgelöst.

huddle to_node *object* ?*tag*?
Für Typ-Callbacks.

Wenn *object* ein "Huddle" ist, gibt es den Stammknoten zurück. Andernfalls gibt es **[list s $object]** zurück.

```
% huddle to_node str
s str
% huddle to_node str !!str
!!str str
% huddle to_node {HUDDLE {s str}}
s str
% huddle to_node {HUDDLE {l {a b c}}}
l {a b c}
```

huddle wrap *tag src*
für Typ-Callbacks.

Erstellt ein Huddle-Objekt aus *src* mit dem angegebenen *Tag*.

```
% huddle wrap "" str
HUDDLE str
% huddle wrap s str
HUDDLE {s str}
```

huddle call *tag command args*
für Typ-Callbacks.

Befehl wird an das Standard-*Tag* -callback weitergeleitet

huddle addType *callback*
Fügt der Huddle-Bibliothek einen benutzerdefinierten Typ/Tag hinzu. Siehe "Zusätzlicher Typ".
callback
Name der Rückruffunktion für den zusätzlichen Typ.

Rückruffunktion eingeben

Die Definition von "Callback" für den Typ "user".

callback *command* ?*args*?

siehe unten ...

command
huddle-Unterbefehl, der für die Antwort über die Rückruffunktion benötigt wird.

args
Argumente des Unterbefehls. Die Anzahl der Argumente ist je nach Unterbefehl unterschiedlich.

Die Rückruffunktion sollte die folgenden Unterbefehle zurückgeben.

setting
gibt lediglich ein festes Wörterbuch mit den Typinformationen zum Setzen des Benutzer-Tags zurück.

type typename
Typname des Typs

method {method1 method2 method3 ...}
Methodenliste als "huddle"-Unterbefehl. Anschließend können Sie **[huddle method1 ...]** aufrufen

tag {tag1 child/parent tag2 child/parent ...}
Tag-Liste für "huddle-node" als Wörterbuch. Wenn der Typ untergeordnete Knoten hat, verwende "parent", andernfalls "child".

get_sub *src key*
gibt einen durch *key* festgelegten Unterknoten zurück.

src
Ein Knoteninhalt in einem Huddle-Objekt.

strip *src*
Gibt den Inhalt des bereinigten Knotens zurück. Wenn der Typ untergeordnete Knoten hat, muss jeder Knoten bereinigt werden.

set *src key value*
Legt einen Unterknoten aus dem markierten Inhalt fest und gibt "self" zurück.

remove *src key value*
Entfernt einen Unterknoten aus dem "tagged-content" und gibt "self" zurück.

strip muss für alle Typen definiert sein. **get_sub** muss für Container-Typen definiert sein. **set/remove** sollten definiert sein, falls Sie sie aufrufen.

```
# callback sample for my-dict
proc my_dict_setting {command args} {
    switch -- $command {
        setting { ; # type definition
            return {
                type dict
                method {create keys}
                tag {d child D parent}
                constructor create
                str s
            }
```

```
        # type:   the type-name
        # method: add methods to huddle's subcommand.
        #          "get_sub/strip/set/remove/equal/append" called
by huddle module.
        #          "strip" must be defined at all types.
        #          "get_sub" must be defined at container types.
        #          "set/remove/equal/append" shuould be defined,
if you call them.
        # tag:    tag definition("child/parent" word is maybe
obsoleted)
    }
    get_sub { ; # get a sub-node specified by "key" from the
tagged-content
        foreach {src key} $args break
        return [dict get $src $key]
    }
    strip { ; # strip from the tagged-content
        foreach {src nop} $args break
        foreach {key val} $src {
            lappend result $key [huddle strip $val]
        }
        return $result
    }
    set { ; # set a sub-node from the tagged-content
        foreach {src key value} $args break
        dict set src $key $value
        return $src
    }
    remove { ; # remove a sub-node from the tagged-content
        foreach {src key value} $args break
        return [dict remove $src $key]
    }
    equal { ; # check equal for each node
        foreach {src1 src2} $args break
        if {[llength $src1] != [llength $src2]} {return 0}
        foreach {key1 val1} $src1 {
            if {![dict exists $src2 $key1]} {return 0}
            if {![huddle _equal_subs $val1 [dict get $src2
$key1]]} {return 0}
        }
        return 1
    }
    append { ; # append nodes
        foreach {str src list} $args break
        if {[llength $list] % 2} {error {wrong # args: should be
"huddle append objvar ?key value ...?"}}
```

```
            set resultL $src
            foreach {key value} $list {
                if {$str ne ""} {
                    lappend resultL $key [huddle to_node $value $str]
                } else {
                    lappend resultL $key $value
                }
            }
            return [eval dict create $resultL]
        }
        create { ; # $args: all arguments after "huddle create"
            if {[llength $args] % 2} {error {wrong # args: should be
    "huddle create ?key value ...?"}}
            set resultL {}
            foreach {key value} $args {
                lappend resultL $key [huddle to_node $value]
            }
            return [huddle wrap D $resultL]
        }
        keys {
            foreach {src nop} $args break
            return [dict keys [lindex [lindex $src 1] 1]]
        }
        default {
            error "$command is not callback for dict"
        }
    }
}
```

```
# inheritance sample from default dict-callback
proc ::yaml::_huddle_mapping {command args} {
    switch -- $command {
        setting { ; # type definition
            return {
                type dict
                method {mapping}
                tag {!!map parent}
                constructor mapping
                str !!str
            }
        }
        mapping { ; # $args: all arguments after "huddle mapping"
            if {[llength $args] % 2} {error {wrong # args: should be
    "huddle mapping ?key value ...?"}}
            set resultL {}
            foreach {key value} $args {
```

```
                lappend resultL $key [huddle to_node $value !!str]
            }
            return [huddle wrap !!map $resultL]
        }
        default { ; # devolving to default dict-callback
            return [huddle call D $command $args]
        }
    }
}
```

So fügen Sie einen Typ hinzu

Sie können Huddle-Knotentypen hinzufügen, z. B. ::struct::tree. Definieren Sie dazu zunächst eine Callback-Prozedur für einen zusätzlichen getaggten Typ. Die Prozedur erhält die Argumente *command* und ?*args*?. Sie enthält einige Switch-Abschnitte.

Außerdem wird der Unterbefehl "addType" aufgerufen.

```
huddle addType my_dict_setting
```

Arbeitsbeispiel

```
# create as a dict
% set bb [huddle create a b c d]
HUDDLE {D {a {s b} c {s d}}}

# create as a list
% set cc [huddle list e f g h]
HUDDLE {L {{s e} {s f} {s g} {s h}}}
% set bbcc [huddle create bb $bb cc $cc]
HUDDLE {D {bb {D {a {s b} c {s d}}} cc {L {{s e} {s f} {s g} {s h}}}}}
% set folding [huddle list $bbcc p [huddle list q r] s]
HUDDLE {L {{D {bb {D {a {s b} c {s d}}} cc {L {{s e} {s f} {s g} {s
    h}}}}} {s p} {L {{s q} {s r}}} {s s}}}

# normal Tcl's notation
% huddle strip $folding
{bb {a b c d} cc {e f g h}} p {q r} s

# get a sub node
% huddle get $folding 0 bb
HUDDLE {D {a {s b} c {s d}}}
% huddle gets $folding 0 bb
a b c d

# overwrite a node
% huddle set folding 0 bb c kkk
HUDDLE {L {{D {bb {D {a {s b} c {s kkk}}} cc {L {{s e} {s f} {s g} {s
```

```
    h}}}}} {s p} {L {{s q} {s r}}} {s s}}}

# remove a node
% huddle remove $folding 2 1
HUDDLE {L {{D {bb {D {a {s b} c {s kkk}}} cc {L {{s e} {s f} {s g} {s
    h}}}}} {s p} {L {{s q}}} {s s}}}
% huddle strip $folding
{bb {a b c kkk} cc {e f g h}} p {q r} s

# dump as a JSON stream
% huddle jsondump $folding
[
  {
    "bb": {
      "a": "b",
      "c": "kkk"
    },
    "cc": [
      "e",
      "f",
      "g",
      "h"
    ]
  },
  "p",
  [
    "q",
    "r"
  ],
  "s"
]
```

Einschränkungen

Wird gerade gedruckt.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *huddle* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright

```
Copyright (c) 2008-2011 KATO Kanryu <kanryu6@users.sourceforge.net>
Copyright (c) 2015 Miguel Martínez López <aplicacionamedida@gmail.com>
```

yaml

Stichwörter: yaml, data exchange, huddle, parsing, text processing

Name

yaml – YAML-Format-Encoder/Decoder für die Kodierung und Dekodierung

Übersicht

package require Tcl 8.5 9
package require yaml ?**0.4.2**?
::yaml::yaml2dict ?*options*? *txt*
::yaml::yaml2huddle ?*options*? *txt*
::yaml::setOption ?*options*?
::yaml::dict2yaml *dict* ?*indent*? ?*wordwrap*?
::yaml::list2yaml *list* ?*indent*? ?*wordwrap*?
::yaml::huddle2yaml *huddle* ?*indent*? ?*wordwrap*?

Beschreibung

Das **yaml**-Paket bietet eine einfache, ausschließlich auf Tcl basierende Bibliothek zum Parsen des YAML-Datenaustauschformats http://www.yaml.org/, wie es unter http://www.yaml.org/spec/1.1/ festgelegt ist.

Das **yaml**-Paket gibt Daten als Tcl-**dict** zurück. Für die Verwendung ist entweder das **dict**-Paket oder Tcl 8.5 erforderlich.

Befehle

::yaml::yaml2dict ?*options*? *txt*
siehe unten ...

::yaml::yaml2huddle ?*options*? *txt*
YAML-formatierten Text *txt* in ein Tcl-Dict/Huddle parsen und den Wert geben.

-file
txt ist der Dateiname eines YAML-Streams.

-stream
txt ist lediglich ein YAML-Stream.

-types *list*
Die *Liste* ist eine Typenliste für die YAML-Skalar-Typen (z. B.!!str!!timestamp!!integer!!true ...).

```
-types {timestamp integer null true false}
```

In diesem Fall, wenn eine Zeichenfolge mit "timestamp" übereinstimmte, wurde sie in den TCL-internen Zeitstempel umgewandelt (z. B. "2001-12-15T02:59:43.1Z" => 1008385183).

-m:true *param*
Der *param* besteht aus zwei Elementen der Liste, die den Wert "true" haben und als Zeichenfolgen betrachtet werden.

```
-m:true {1 {true on + yes y}}
```

In diesem Fall wurde die in YAML Stream gefundene Zeichenfolge "yes" automatisch in 1 umgewandelt.

-m:false *param*
Der *param* besteht aus zwei Elementen der Liste, die den Wert "false" haben und als Zeichenfolgen betrachtet werden.

```
-m:false {0 {false off - no n}}
```

-m:null *param*
Der *param* besteht aus zwei Elementen der Liste, die den Wert "null" haben und als Zeichenfolgen betrachtet werden.

```
-m:null {"" {null nil "" ~}}
```

-validate
Experiment, alt: Der Ausgabestrom enthält in jedem Knoten das YAML-Tag.

```
% puts [::yaml::load -validate {[aaa, bbb]}]
=>
!!seq {{!!str aaa} {!!str bbb}}
```

::yaml::setOption ?*options*?
Ändert die impliziten Optionen für die Bibliothek. Die Parameter entsprechen nun denen von **::yaml::yaml2dict**. Die Argumente von **::yaml::yaml2dict** haben Vorrang vor dieser Einstellung.

::yaml::dict2yaml *dict* ?*indent*? ?*wordwrap*?
siehe unten ...

::yaml::list2yaml *list* ?*indent*? ?*wordwrap*?
siehe unten ...

::yaml::huddle2yaml *huddle* ?*indent*? ?*wordwrap*?
Ein dict-/list-/huddle-Objekt in einen YAML-Stream umwandeln.

indent
Einrückung jedes Blockknotens in Leerzeichen. Derzeit beträgt der Standardwert 2.

wordwrap
Zeilenumbruch für YAML-Stream. Derzeit ist der Standardwert 40.

Beispiele

Ein Beispiel für einen in Tcl konvertierten YAML-Stream. Ein YAML-Stream wird als einzelnes Element mit mehreren Unterelementen zurückgegeben.

```
{
--- !<tag:clarkevans.com,2002:invoice>
invoice: 34843
date   : 2001-01-23
```

```
bill-to: &id001
    given  : Chris
    family : Dumars
    address:
        lines: |
            458 Walkman Dr.
            Suite #292
        city    : Royal Oak
        state   : MI
        postal  : 48046
ship-to: *id001
product:
    - sku         : BL394D
      quantity    : 4
      description : Basketball
      price       : 450.00
    - sku         : BL4438H
      quantity    : 1
      description : Super Hoop
      price       : 2392.00
tax  : 251.42
total: 4443.52
comments:
    Late afternoon is best.
    Backup contact is Nancy
    Billsmer @ 338-4338.
}
=>
invoice 34843 date 2001-01-23 bill-to {given Chris family Dumars
    address {lines {458 Walkman Dr.
Suite #292
} city {Royal Oak} state MI postal 48046}} ship-to {given Chris family
    Dumars address {lines {458 Walkman Dr.
Suite #292
} city {Royal Oak} state MI postal 48046}} product {{sku BL394D
    quantity 4 description Basketball price 450.00} {sku BL4438H
    quantity 1 description {Super Hoop} price 2392.00}} tax 251.42
    total 4443.52 comments {Late afternoon is best. Backup contact is
    Nancy Billsmer @ 338-4338.}
```

Ein Beispiel für ein in Tcl konvertiertes YAML-Objekt. Ein YAML-Objekt wird als Liste mit mehreren Elementen (ein Dict) zurückgegeben.

```
{
---
- [name        , hr, avg  ]
- [Mark McGwire, 65, 0.278]
```

```
- [Sammy Sosa  , 63, 0.288]
-
  Mark McGwire: {hr: 65, avg: 0.278}
  Sammy Sosa: { hr: 63, avg: 0.288}
}
=>
{name hr avg} {{Mark McGwire} 65 0.278} {{Sammy Sosa} 63 0.288} {{Mark
    McGwire} {hr 65 avg 0.278} {Sammy Sosa} {hr 63 avg 0.288}}
```

Einschränkungen

Der Tag-Parser ist noch nicht implementiert. Derzeit werden Tags einfach ignoriert.

Nur Anker => Reihenfolge der Aliase. Rückverweise auf Aliase werden nicht unterstützt.

Zu viele Klammern oder zu wenige Klammern.

Der Zeichensatz für Zeilenvorschübe ist unzureichend. Bitte verwenden Sie als Zeilenumbrüche ausschließlich "\n".

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *yaml* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Copyright


```
Copyright (c) 2008 KATO Kanryu <kanryu6@users.sourceforge.net>
```


zip

decode

Stichwörter: decode, decompression, zip

Name

zipfile::decode – Zugriff auf ZIP-Archive

Übersicht

package require Tcl 8.5 9
package require fileutil::decode 0.2.1
package require Trf
package require zlibtcl
package require zipfile::decode ?**0.11.0**?
::zipfile::decode::archive
::zipfile::decode::close

::zipfile::decode::comment *adict*
::zipfile::decode::content *archive*
::zipfile::decode::copyfile *adict path dst*
::zipfile::decode::files *adict*
::zipfile::decode::filetime *adict path*
::zipfile::decode::getfile *zdict path*
::zipfile::decode::hasfile *adict path*
::zipfile::decode::filesize *zdict path*
::zipfile::decode::filecomment *zdict path*
::zipfile::decode::iszip *archive*
::zipfile::decode::open *archive*
::zipfile::decode::unzip *adict dstdir*
::zipfile::decode::unzipfile *archive dstdir*

Beschreibung

Hinweis: Die Pakete "Trf" und "zlibtcl" sind nicht erforderlich, wenn TCL 8.6 verfügbar ist. Dieses Paket enthält Befehle zum Entpacken von ZIP-Archiven und zum Zugriff auf deren Inhalt.

API

::zipfile::decode::archive
: Dieser Befehl dekodiert die zuletzt geöffnete (und noch nicht geschlossene) ZIP-Archivdatei. Das Ergebnis des Befehls ist ein Dictionary, das den Inhalt des Archivs beschreibt. Die Struktur dieses Dictionarys ist nicht öffentlich. Der Zugriff sollte ordnungsgemäß über die bereitgestellten Zugriffsbefehle dieses Pakets erfolgen.

::zipfile::decode::close
: Dieser Befehl gibt alle Zustandsdaten frei, die mit dem letzten Aufruf von **::zipfile::decode::open** verbunden sind. Das Ergebnis des Befehls ist die leere Zeichenkette.

::zipfile::decode::comment *adict*
: Dieser Befehl nimmt ein Dictionary entgegen, das die derzeit geöffnete ZIP-Archivdatei beschreibt, wie sie von **::zipfile::decode::archive** zurückgegeben wird, und gibt den globalen Kommentar des Archivs zurück.

::zipfile::decode::content *archive*
: Dies ist ein Komfortbefehl, der die angegebene ZIP-*Archiv*-Datei entpackt und als Ergebnis die Liste der darin enthaltenen Pfade gibt.

::zipfile::decode::copyfile *adict path dst*
: Dieser Befehl nimmt ein Dictionary entgegen, das die aktuell geöffnete ZIP-Archivdatei beschreibt, wie sie von **::zipfile::decode::archive** zurückgegeben wird, und kopiert den entpackten Inhalt der Datei mit dem Pfad *path* im Archiv in die Datei *dst*. Wenn die Datei im Archiv nicht gefunden wird, wird ein Fehler ausgelöst.

::zipfile::decode::files *adict*
: Dieser Befehl nimmt ein Dictionary entgegen, das die aktuell geöffnete ZIP-Archivdatei beschreibt, wie sie von **::zipfile::decode::archive** zurückgegeben wird, und gibt die Liste der im Archiv enthaltenen Dateien zurück.

::zipfile::decode::filetime *adict path*
: Dieser Befehl nimmt ein Dictionary entgegen, das die derzeit geöffnete ZIP-Archivdatei

beschreibt, wie sie von **::zipfile::decode::archive** zurückgegeben wird, und gibt den Zeitstempel der Datei *Pfad* im Archiv zurück.

::zipfile::decode::getfile *zdict path*
Dieser Befehl nimmt ein Dictionary entgegen, das die derzeit geöffnete ZIP-Archivdatei beschreibt, wie sie von **::zipfile::decode::archive** zurückgegeben wird, und gibt den entpackten Inhalt der Datei mit dem Pfad *path* im Archiv zurück. Wenn die Datei im Archiv nicht gefunden wird, wird ein Fehler ausgelöst.

::zipfile::decode::hasfile *adict path*
Dieser Befehl nimmt ein Dictionary entgegen, das die aktuell geöffnete ZIP-Archivdatei beschreibt, wie es **::zipfile::decode::archive** gibt, und prüft, ob der angegebene *Pfad* im Archiv gefunden wird. Das Ergebnis des Befehls ist ein boolescher Wert: **true**, wenn der Pfad gefunden wird, andernfalls **false**.

::zipfile::decode::filesize *zdict path*
Dieser Befehl nimmt ein Dictionary entgegen, das die aktuell geöffnete ZIP-Archivdatei beschreibt, wie sie von **::zipfile::decode::archive** zurückgegeben wird, und gibt die entpackte Größe der Datei *Pfad* im Archiv zurück. Wenn die Datei im Archiv nicht gefunden wird, wird ein Fehler ausgelöst.

::zipfile::decode::filecomment *zdict path*
Dieser Befehl nimmt ein Dictionary entgegen, das die derzeit geöffnete ZIP-Archivdatei beschreibt, wie sie von **::zipfile::decode::archive** zurückgegeben wird, und gibt den dateispezifischen Kommentar der Datei *pfad* im Archiv zurück. Wenn die Datei im Archiv nicht gefunden wird, wird ein Fehler ausgelöst.

::zipfile::decode::iszip *archive*
Dieser Befehl nimmt den Pfad einer vermeintlichen ZIP-*Archiv*-Datei entgegen und gibt als Ergebnis ein boolesches Flag zurück, das angibt, ob es sich tatsächlich um ein ZIP-Archiv handelt (**true**) oder nicht (**false**).

::zipfile::decode::open *archive*
Dieser Befehl nimmt den Pfad einer ZIP-*Archiv*-Datei entgegen und bereitet diese für die Entpackung vor. Das Ergebnis des Befehls ist die leere Zeichenkette. Alle wichtigen Informationen werden im globalen Status gespeichert. Werden mehrere "open"-Aufrufe nacheinander ausgeführt, steht den anderen Befehlen nur der Status des letzten Aufrufs zur Verfügung.

::zipfile::decode::unzip *adict dstdir*
Dieser Befehl nimmt ein Dictionary entgegen, das die derzeit geöffnete ZIP-Archivdatei beschreibt, wie es **::zipfile::decode::archive** zurückgibt, und entpackt das Archiv in das angegebene Zielverzeichnis *dstdir*. Das Ergebnis des Befehls ist die leere Zeichenkette.

::zipfile::decode::unzipfile *archive dstdir*
Dies ist ein Komfort-Befehl, der die angegebene ZIP-*Archiv*-Datei in das angegebene Ziel-Verzeichnis *dstdir* entpackt.

Das Ergebnis des Befehls ist die leere Zeichenkette.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie

bitte die Tcllib-Tracker und geben Sie *zipfile* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie
Dateien

Copyright

```
Copyright (c) 2008-2022 Andreas Kupries
```


encode
Stichwörter: encode, compression, zip

Name
zipfile::encode – Erstellung von ZIP-Archiven

Übersicht
package require Tcl 8.5 9
package require logger
package require Trf
package require crc32
package require snit
package require zlibtcl
package require fileutil
package require zipfile::encode ?**0.5.2**?
::zipfile::encode *objectName*
comment: *text*
file: *dst owned src* ?*noCompress*?
write *archive*

Beschreibung
Hinweis: Die Pakete "Trf" und "zlibtcl" sind für Tcl Version 8.6 oder höher nicht erforderlich.
Dieses Paket stellt eine Klasse zur Erstellung von ZIP-Archiven bereit.

API für Klassen
::zipfile::encode *objectName*

Der Befehl "class" erstellt Encoder-Instanzen, d. h. Objekte. Das Ergebnis des Befehls ist der vollqualifizierte Name des Instanzbefehls.

Das Argument *objectName* ist erforderlich. Wenn *objectName* auf **%AUTO%** gesetzt ist, generiert die Klasse einen automatischen Namen im aktuellen Namespace und verwendet diesen. Ist der Wert von *objectName* nicht vollständig qualifiziert, wird der Befehl für die Instanz relativ zum aktuellen Namespace erstellt.

Instanz-API

comment: *text*

Diese Methode legt den Text des globalen Kommentars für das Archiv fest. Das Ergebnis der Methode ist die leere Zeichenkette. Werden mehrere Aufrufe dieser Methode für denselben Encoder durchgeführt, haben die Daten des letzten Aufrufs Vorrang vor allen vorherigen Texten.

file: *dst owned src ?noCompress?*

Diese Methode fügt dem Archiv eine neue Datei hinzu. Der Inhalt der Datei befindet sich im Dateisystem unter *src* und wird im Archiv unter dem Pfad *dst* gespeichert. Wenn die Datei als *Eigentum* des Archivs deklariert ist, wird die Originaldatei beim Erstellen und Schreiben des Archivs gelöscht. Wenn *noCompress* auf **true** gesetzt ist, wird die Datei beim Schreiben nicht komprimiert. Andernfalls (Standardeinstellung) wird die Datei komprimiert, sofern dies vorteilhaft ist. Das Ergebnis der Methode ist eine leere Zeichenkette.

write *archive*

Diese Methode nimmt den globalen Kommentar und alle hinzugefügten Dateien, kodiert sie als Zip-Archiv und speichert das Ergebnis unter dem Pfad *archive* im Dateisystem. Alle hinzugefügten Dateien, die dem Archiv gehören, werden an dieser Stelle gelöscht. Zur Reihenfolge: Die Dateien werden dem Archiv in derselben Reihenfolge hinzugefügt, in der sie über **file:** angegeben wurden. *Hinweis:* Dieses Verhalten gilt ab Version 0.4. Vor 0.4 war keine bestimmte Reihenfolge dokumentiert. Sortiert wurde lexikografisch. Die Änderung unterstützt **zip**-basierte Dateiformate, die eine bestimmte Dateireihenfolge im Archiv verlangen, zum Beispiel "*.epub*".

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *zipfile* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dateien

Copyright


```
Copyright (c) 2008-2009 Andreas Kupries
```


mkzip

Stichwörter: mkzip, decompression, zip

Name

zipfile::mkzip – ZIP-Archiv erstellen

Übersicht

package require Tcl 8.6 9
package require zipfile::mkzip ?**1.2.4**?
::zipfile::mkzip::mkzip *zipfile* ?**-zipkit**? ?*-runtime prefix*? ?*-comment string*? ?*-directory rootpath*? ?*-prepend-base-directory bool*? ?*-exclude exclude*? ?**--**? ?*path...*?

Beschreibung

Dieses Paket nutzt die zlib-Funktionen in Tcl 8.6, um ZIP-Archive zu erstellen.

API

::zipfile::mkzip::mkzip *zipfile* ?-zipkit? ?-runtime *prefix*? ?-comment *string*? ?-directory *rootpath*? ?-prepend-base-directory *bool*? ?-exclude *exclude*? ?--? ?*path*...?

Aus http://wiki.tcl.tk/15158

Dieser Befehl erstellt ein ZIP-Archiv aus einer Verzeichnisstruktur und nutzt dabei ausschließlich die Kernfunktionen von Tcl 8.6. Die resultierende ZIP-Datei sollte mit anderen **zip**-Programmen kompatibel sein – mit der möglichen Ausnahme der Unicode-Unterstützung. Die durch diesen Befehl erzeugten Dateien verwenden die UTF-8-Kodierung für alle Dateinamen und Kommentare, und es wurde insbesondere unter Windows festgestellt, dass **info-zip** und die in Windows integrierte ZIP-Ansicht diesen Teil der ZIP-Dateispezifikation nur unzureichend unterstützen. Das Programm **7-Zip** zeigt UTF-8-Dateinamen jedoch korrekt an, und das Paket **vfs::zip** verwendet diese natürlich.

Wenn Sie

```
::mkzip::mkzip mystuff.tm -zipkit -directory mystuff.vfs
```

Es packt Ihren virtuellen Dateisystembaum "*mystuff.vfs/*" in ein ZIP-Archiv mit einem geeigneten Header, sodass Sie es unter Unix als ausführbar kennzeichnen können und es mit tclkit laufen sollte. Alternativ kannst du es auch mit **tclsh** oder **wish** 8.6 ausführen, wenn du möchtest.

Um den Header der ausführbaren Datei zu ändern, legen Sie das **-runtime** "*preface*" fest, wobei "preface" eine Datei ist, deren Inhalt Sie voranstellen möchten. Unter Windows können Sie beispielsweise ein selbstextrahierendes ZIP-Archiv erstellen, indem Sie

```
mkzip mystuff.exe -directory mystuff.vfs -runtime unzipsfx.exe
```

Die Datei "*unzipsfx.exe*" ist der selbstextrahierende Stub von Info-Zip.

Zulässige Optionen:

-runtime path
: Diese Option legt eine Datei fest, die als Präfix für das eigentliche ZIP-Archiv verwendet werden soll. Wenn angegeben, wird **-zipkit** ignoriert.

-zipkit
: Weist den Befehl an, ein Präfix zu generieren, das das Archiv zu einem ZIP-basierten Starkit macht. Wird ignoriert, wenn **-runtime** vorhanden ist.

-comment string
: Diese Option legt einen globalen Kommentar fest, der in das generierte Archiv eingefügt werden soll.

-directory path
Diese Option legt das Verzeichnis fest, das in das generierte Archiv placed werden soll. Wenn angegeben, werden alle *Pfad*-Argumente *ignoriert*.

-prepend-base-directory bool
Wenn diese Option aktiviert ist, wird allen Pfaden, die dem Archiv hinzugefügt werden, der Endteil des Pfads **-directory** vorangestellt.

Die Option wird ignoriert, wenn die Option **-directory** nicht angegeben ist.

Standardmäßig ist diese Option auf **false** gesetzt.

-exclude list
Diese Option legt eine Liste von Wildcard-Mustern fest. Alle Pfade, die mit mindestens einem dieser Muster übereinstimmen, werden nicht in das erzeugte Archiv platziert. Die Standardeinstellung dieser Option lautet

```
    CVS/* */CVS/* *~ ".#*" "*/.#*"
```

--
Diese Option markiert das Ende der Optionen und sorgt dafür, dass alle nachfolgenden Wörter als Argumente verarbeitet werden, auch wenn sie mit einem Bindestrich beginnen.

Zulässige Argumente:
path *path*
Jeder Pfad ist ein Verzeichnis oder eine Datei, die in das generierte Archiv platziert werden soll. Hinweis: Diese werden ignoriert, wenn die Option **-directory** festgelegt wird.

Fehler, Ideen, Rückmeldungen

Wenn Sie Fehler in diesem Dokument oder Fehler und Probleme im beschriebenen Paket finden oder Verbesserungen für die Dokumentation oder das Paket vorschlagen möchten, nutzen Sie bitte die Tcllib-Tracker und geben Sie *zipfile* als Kategorie an.

Schlagen Sie Codeänderungen bitte als *Unified Diffs* vor, also als Ausgabe von **diff -u**.

Anhänge sind eingebetteten Patches ausdrücklich vorzuziehen. Anhänge lassen sich über das Formular **Edit** des Tickets direkt nach dem Anlegen hinzufügen, mit der Schaltfläche ganz links in der zweiten Navigationsleiste.

Kategorie

Dateien

Copyright

```
Copyright (c) 2009 Pat Thoyts
```

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen.

Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung entstand mit Unterstützung von DeepL und KI-Werkzeugen und wurde von Hand durchgesehen und überarbeitet.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Lizenzbedingungen license

This software is copyrighted by Ajuba Solutions and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Stichwortverzeichnis

.

.ddt 266
.dtx 46, 258, 266

/

/dev/null 1883, 1889
/dev/random 1884, 1890
/dev/zero 1883, 1893

3

3DES 240, 243, 244

A

abstract syntax tree 577, 597
acceptance 550
acceptor 550
active 1796
adaptors 1411, 1452
adjacency list 1563
adjacency matrix 1563
adjacent 1541, 1563
adjust 1736
adjusting 1736
adler32 1899
aes 78
after 205, 206
alias 666
amazon 80
ambiguous 548
American Express 1855
AMEX 1855
angle 832, 1840
annealing 1395
anonymous procedure 684
ansi 1703, 1707, 1714, 1716
ansi_cattr 1703
ansi_cctrl 1707
ansi_cmacros 1714
ansi_code 1715
ansi_ctrlu 1716
ansi_send 1718
appender 725, 726
application 23, 25, 26
approximation algorithm 1563
arc 1541, 1563

arcfour 1352
archive 1643
argument integrity 1671, 1679
argument processing 178
argument validation 1671, 1679
arguments 1671, 1679
argv 178
argv0 178
array 1775, 1783
articulation point 1563
ascii85 123
asn 108
assembler 571
assert 203
assign 1612
AST 577
async 159
asynchronous 159
attribute control 1703, 1707
augmenting network 1563
augmenting path 1563
authentication 634, 1381, 1382, 1388
automatic 967
automatic documentation 1663
automaton 550, 552, 554, 564
autoproxy 634
aycock 548

B

bank 1855, 1857, 1859, 1861, 1865
base32 117, 119, 121
base32core 119
base32hex 121
base64 125, 1907
bash 1526
bee 134
bench 140
bench language 145, 146, 148
bench_intro 145
bench_lang_intro 146
bench_lang_spec 148
bench_read 150
bench_wcsv 152
bench_wtext 153
benchmark 140, 145, 146, 148, 150, 152, 153
ber 108
Bessel functions 881

bfs 1563
bibliography 154
bibtex 154
bigfloat 751
bignum 760
bignums 760
bind 1835
bipartite 1563
BitTorrent 134
blanks 1750
block cipher 78, 156, 240, 243, 244
blocking flow 1563
blowfish 156
Book Number 1868
breadth-first 1628
bridge 1563
BWidget 1411, 1452

C

C 415, 484
C++ 1411, 1452, 1514, 1517
cache 159, 740
caesar cipher 1904
calculus 765
callback 159, 617, 684, 998, 1787, 1839
callbacks 1881
capitalize 1768
card for credit 1855, 1857, 1859, 1861
cardinality 1625
cat 498, 1879
catalog package 368, 415, 416, 417, 418, 484, 485, 486, 487
catalogue 266
cc_amex 1855
cc_discover 1857
cc_mastercard 1859
cc_visa 1861
cell-phone 1866
cer 108
CFG 590
CFL 590
cfront 530
cgen 532
CGI 956
cgraph 1541, 1557
changelog 276, 278
changepoint 774
channel 205, 206, 1796, 1800, 1803, 1805, 1808, 1812, 1815

channel transformation 1899, 1900, 1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1910, 1911
character input 1731, 1733
character output 1718, 1735
character type 1773
character width 1773
chat 667, 671, 951
checkbox 622, 674
checkbutton 622
Checking 1855, 1857, 1859, 1861, 1863, 1864, 1866, 1868, 1870, 1872, 1873, 1875, 1877
checksum 212, 214, 219, 222, 1899, 1910
chop 1768
cipher 1023, 1904, 1911
cksum 212, 214, 219, 222
class 1411, 1452, 1514, 1517
class methods 998, 1787
class variables 998, 1787
clay 161
cleanup 238, 1832
client 23, 26, 967, 969, 974, 975
cloud 80
cmdline 178
cmdline processing 178
color control 1703, 1707
columns 1716
combinatorics 776
comm 183, 200, 664, 665, 977
comm_wire 200
command 365
command line processing 178
command prefix 684, 998, 1787
comment 676, 1035
common 1612
common prefix 1768
communication 183, 200
comparison 1612
compiler 548
complete graph 1563
complex numbers 800, 866
compression 1906, 1943
computations 751
concatenation channel 1879, 1880
connect 1796
connected component 1563
connected fifos 1887

connection 1796
constants 781, 1840
CONTAINER 1145, 1236
contents 481
context-free grammar 590
context-free languages 34, 590, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1153, 1162, 1172, 1181, 1189, 1197, 1198, 1206, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
control 203, 774, 1703, 1707, 1714, 1715, 1716, 1718, 1725, 1728, 1731, 1732, 1733, 1735
control structure 537
conversion 15, 46, 280, 301, 324, 362, 372, 378, 401, 412, 436, 444, 469, 481, 497, 875, 1172, 1181, 1236, 1244, 1253, 1262, 1286, 1295, 1840
cooked 1716
cookie 956
copy 507, 1800, 1803, 1805, 1808, 1812, 1815
copyops 1800
core 1894
coro_auto 205
coroutine 205, 206, 537
Cost 1820
counter 209, 1908
counting 209
CPARAM 1244
crc 212, 214, 219, 222
crc16 214
crc32 212, 214, 219, 222, 1910
credit card 1855, 1857, 1859, 1861
cron 223
cryptography 156
CSS 363
csv 150, 152, 227
currying 684, 998, 1787
cut edge 1563
cut vertex 1563
CVS 1354
cvs 278
cvs log 278
cyclic redundancy check 212, 214, 219, 222

D

dacceptor 550
data analysis 889
data destination 1803, 1808

data entry form 1647
data exchange 679, 682, 1926, 1937
data integrity 78, 212, 214, 219, 222, 240, 243, 244, 1023, 1352
data source 1805, 1815
data structures 1602
database 1775, 1783
dataflow 1015
ddest 1803
DE 416, 485
debug 231, 235, 236, 237
debug_caller 235
debug_heartbeat 236
debug_timestamp 237
decimal 784
declare 1715
decode 1940
decompression 1906, 1940, 1944
decryption 1904, 1911
defer 238
deferal 1839
define 1715
degree 1541, 1563
degree constrained spanning tree 1563
degrees 781
deleg_method 664
deleg_proc 665
delegation 664, 665
depth-first 1628
der 108
DES 243, 244
des 240
deserialization 408, 423, 427, 433, 477, 493
dexec 552
diameter 1563
dict 244
dicttool 244
diff 266, 1612, 1748
diff -n format 1354
diff -ruN 1748
difference 1625
differential 1612
differential equations 765
digital 798
dijkstra 1563
directory access 690, 701
directory traversal 515

Discover 1857
discrete items 1592
disjoint set 1536
disjointset 1536
dispatcher 1733
distance 832, 1563, 1840
DNS 246
do 203
docidx 15, 280, 368, 369, 378, 401, 415, 416, 417, 418, 419, 423, 427, 497
docidx commands 287, 289, 290, 293
docidx language 287, 289, 290, 293
docidx markup 287, 289, 290, 293, 372
docidx syntax 289, 290, 293
docidx_intro 285
docidx_lang_cmdref 287
docidx_lang_faq 289
docidx_lang_intro 290
docidx_lang_syntax 293
docidx_plugin_apiref 295
docstrip 46, 258, 266
docstrip_util 266
doctoc 15, 301, 368, 429, 433, 444, 469, 484, 485, 486, 487, 488, 493, 497
doctoc commands 308, 310, 311, 316
doctoc language 308, 310, 311, 316
doctoc markup 308, 310, 311, 316, 436
doctoc syntax 310, 311, 316
doctoc_intro 306
doctoc_lang_cmdref 308
doctoc_lang_faq 310
doctoc_lang_intro 311
doctoc_lang_syntax 316
doctoc_plugin_apiref 318
doctools 15, 266, 276, 324, 363, 364, 365, 368, 369, 384, 389, 393, 396, 398, 408, 415, 416, 417, 418, 419, 423, 427, 429, 433, 450, 455, 459, 463, 466, 477, 484, 485, 486, 487, 488, 493, 497
doctools commands 333, 341, 342, 352
doctools language 333, 341, 342, 352
doctools markup 333, 341, 342, 352
doctools syntax 341, 342, 352
doctools_intro 331
doctools_lang_cmdref 333
doctools_lang_faq 341
doctools_lang_intro 342

doctools_lang_syntax 352
doctools_plugin_apiref 354
document 354
documentation 46, 258, 266, 280, 301, 324, 372, 378, 401, 436, 444, 469, 1663
DOM 1820
dom 105
domain name service 246
double-wide character 1773
dsource 1805
dtplite 15

E

e 781
EAN 1863, 1868
EAN13 1868
ean13 1863
Earley 548
EBNF 34, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1153, 1162, 1172, 1181, 1189, 1197, 1198, 1206, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
eccentricity 1563
edge 1541, 1563
emacs 276, 278
email 653, 940, 945, 1038
emptiness 1625
empty interpreter 666
EN 417, 486
encode 1943
encoding 123, 125, 128, 131
encryption 78, 156, 240, 243, 244, 1023, 1352, 1904, 1911
entry mask 1671
equal 1612
equality 1612
equivalence class 1536
error 1831, 1832
error function 881
European Article Number 1863, 1868
event 617, 1835, 1839
event management 1896
events 205, 206, 1896
exact 788
examples 146, 289, 310, 341
exception 1832
exchange format 679, 682, 1926

exclusion 1625
execution 552
exif 676
exit 205, 206
expander 1741
export 363, 364, 369, 378, 384, 389, 393, 396, 398, 429, 444, 450, 455, 459, 463, 466, 1145, 1153, 1162
export_docidx 369
export_doctoc 429
expression 34, 590, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1189, 1197, 1198, 1206, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
extended namespace 953

F

fa 554
facade 1880
faop 564
faq 289, 310, 341
fetching information 1846
FFT 800
fifo 1881, 1886, 1887
figurate 792
figurate numbers 792
file 1775, 1783, 1846
file recognition 530, 532, 533, 534
file type 530, 532, 533, 534
file utilities 498, 507, 530, 532, 533, 534
filesystem 740
filetypes 533
fileutil 498
filter 537, 1612
filtergen 798
filtering 798
final 1832
finance 1855, 1857, 1859, 1861, 1865
find 1536
finite 1592
finite automaton 550, 552, 554, 564
FIPS 180-1 1389, 1392
first permutation 1612
Fisher-Yates 1612
flatten 1612
floating-point 751, 803
flow 203

flow network 1563
folding 1612
foldl 537
foldr 537
foreach 537
form 622, 956
format conversion 1172, 1181, 1236, 1244, 1253, 1262, 1286, 1295
formatter 354
formatting 150, 152, 153, 372, 378, 412, 436, 444, 481, 1736, 1751, 1753, 1768
formatting engine 295, 318, 354
fossil 1748
fourier 800
Fourier transform 800
FR 418, 487
frame 1714
framework 1790
ftp 519, 525, 526, 1846
ftp_geturl 525
ftpd 526
ftpserver 526
full outer join 1612
fuzzy 803

G

gasm 571
generate event 1835
generate permutations 1612
generation 372, 378, 437, 444
generator 537
geocoding 729
geodesy 731, 742
geography 731
get character 1731
gets 205, 206
git 1748
global 205, 206
golang 238
gopher 1846
gps 545, 964
gpx 545
grammar 34, 548, 550, 552, 554, 564, 571, 579, 584, 590, 591, 598, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351

graph 571, 1541, 1557, 1563, 1600, 1611
graph walking 1015, 1017, 1018
graph1 1557
graphops 1563
green threads 205, 206
grep 498
gtoken 1381
GUID 1854

H

halfpipe 1881
hashing 932, 935, 938, 1001, 1375, 1378, 1389, 1392
heartbeat 236
heuristic 1563
hex 121, 1900
hexadecimal 1900
histogram 209
hook 617, 1835
Horspool 548
HTML 15, 280, 301, 324, 362, 363, 372, 378, 384, 436, 444, 450, 497
html 622, 630, 674, 956
html_cssdefaults 363
htmlparse 630
http 634, 639, 729, 741, 1846, 1912
httpd 639
https 1846
httpserver 639
huddle 1926, 1937
human readable 150, 153
hyphenation 1736, 1753

I

i18n 368, 415, 416, 417, 418, 484, 485, 486, 487
iban 1864
ident 652
identification 652
identity 1901
idle 1839
idx_container 372
idx_export 378
idx_export_html 384
idx_export_json 389
idx_export_nroff 393
idx_export_text 396
idx_export_wiki 398
idx_import 401

idx_import_json 408
idx_introduction 412
idx_msgcat_c 415
idx_msgcat_de 416
idx_msgcat_en 417
idx_msgcat_fr 418
idx_parse 419
idx_structure 423
image 676, 1035, 1783
imap 653
imap4 653
imei 1866
imenu 1725
import 401, 408, 427, 433, 469, 477, 1198, 1207
import_docidx 427
import_doctoc 433
in-memory channel 1881, 1886, 1887, 1888, 1891, 1892
in-order 1628
inclusion 1625
Incr Tcl 1411, 1452
indenting 1736, 1753
independent set 1563
index 280, 285, 295, 369, 372, 378, 384, 389, 393, 396, 398, 401, 408, 412, 427
index formatter 295
info 953
ini 662
inner join 1612
input mode 1716
integer 875
integration 765
inter-thread communication 1887
International Article Number 1863, 1868
International Bank Account Number 1864
International Mobile Equipment Identity 1866
International Standard Book Number 1868
internationalization 368, 415, 416, 417, 418, 484, 485, 486, 487
internet 108, 519, 525, 653, 690, 701, 940, 945, 1042, 1046, 1048, 1912
internet address 251
interpolate 805
interpolation 805
interpreter 664, 665, 666, 1920
intersection 1625
interval 751
ip 251

ipager 1728
ipc 183, 200
ipv4 251
ipv6 251
irc 667, 671
isA 1855, 1857, 1859, 1861, 1863, 1865, 1866, 1868, 1870, 1872, 1873, 1875, 1877
isbn 1868
iso8601 176
isthmus 1563
iterator 537

J

javascript 674, 679, 682
jfif 676
join 1612
jpeg 676
JSON 389, 408, 455, 477, 1153, 1172, 1198, 1253
json 372, 378, 401, 437, 444, 469, 679, 682, 1926
json_write 682
justification 1736

K

keyword index 280, 285, 372, 378, 401, 412
keywords 295
knuth 1513

L

l10n 368, 415, 416, 417, 418, 484, 485, 486, 487
lambda 684
LaTeX 46, 258, 266
latex 280, 301, 372, 437
latitute 731
lazyset 688
ldap 690, 701, 1846
ldap client 690, 701
ldapx 701
ldif 701
least squares 810
left outer join 1612
lemon 1017
level graph 1563
lexer 419, 488
lexing 1522, 1526
limitsize 1902
linalg 810
line 832
linear algebra 810

linear equations 810
linear program 850
lines 1716
list 1600, 1612, 1920
listener 1731, 1733
literate programming 46, 258, 266
LL(k) 34, 590, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1153, 1162, 1172, 1181, 1189, 1197, 1198, 1206, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
local searching 1563
localization 368, 415, 416, 417, 418, 484, 485, 486, 487
location 729, 731, 740, 741
log 231, 235, 236, 237, 278, 713, 718
log level 713, 718
logger 718, 725, 726
loggerAppender 725
loggerUtils 726
longest common subsequence 1612
longitude 731
loop 1541, 1563
luhn 1870, 1872
luhn-5 1872
luhn5 1872

M

machineparameters 826
macros 364
mail 653, 940, 945, 1038
mailto 1846
man_macros 364
manpage 15, 280, 301, 324, 354, 362, 372, 378, 401, 444, 469, 497
map 537, 729, 731, 740, 741, 742, 1612
map_geocode_nominatim 729
map_slippy 731
map_slippy_cache 740
map_slippy_fetcher 741
mapproj 742
markdown 280, 301, 324, 749
markup 15, 46, 280, 285, 287, 289, 290, 293, 295, 301, 306, 308, 310, 311, 316, 318, 324, 331, 333, 341, 342, 352, 354, 362, 372, 378, 401, 412, 437, 444, 469, 481, 497
MasterCard 1859
matching 34, 590, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172,

1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351, 1563
math 751, 760, 765, 781, 784, 803, 805, 810, 830, 832, 850, 856, 859, 866, 872, 881, 928, 1395, 1399, 1402
math_geometry 832
mathematics 792, 800, 862, 870, 889
matrices 810
matrix 227, 810, 1358, 1583, 1600, 1611
max cut 1563
maximum 850
maximum flow 1563
md4 932, 1375, 1378
md5 935, 938
md5crypt 938
me_ast 577
me_cpu 579
me_cpucore 584
me_intro 590
me_tcl 591
me_util 597
me_vm 598
medicare 1873
mega widget 1411, 1452
membership 1625
menu 1714, 1725
merge 1839, 1884
merge find 1536
merging 140
message 183, 200, 713
message catalog 368, 415, 416, 417, 418, 484, 485, 486, 487
message level 713
message package 368, 415, 416, 417, 418, 484, 485, 486, 487
message-digest 932, 935, 938, 1001, 1375, 1378, 1389, 1392
meta 1787
metakit 1775, 1783
method 664, 666
method reference 998, 1787
mime 530, 532, 534, 940, 945
minimal spanning tree 1563
minimum 850
minimum cost flow 1563
minimum degree spanning tree 1563
minimum diameter spanning tree 1563
mkdoc 21, 948
mkzip 1944

mobile phone 1866
module 266
montecarlo 1399
montecarlo simulation 1399
move 507
mpexpand 362
multi 507
multi-file 507
multiop 507
multiplexer 951
multiprecision 751, 760
my method 998, 1787

N

name service 23, 25, 26, 967, 969, 974, 975, 977, 980, 1834
namespace unknown 953
namespace utilities 953
namespacex 953
narrative 231, 235, 236, 237
National Provider Identifier 1873
ncgi 956
neighbour 1541, 1563
net 519, 525, 653, 940, 945, 1912
nettool 961
network 1042, 1046, 1048
news 982, 1846
next permutation 1612
nmea 964
nns 23
nns_auto 967
nns_client 969
nns_common 974
nns_intro 975
nns_protocol 977
nns_server 980
nnsd 25
nnslog 26
nntp 982
nntpclient 982
no-op 203
node 1541, 1563, 1628
nominatim 729
normalization 140, 1017, 1018, 1533
NPI 1873
nroff 15, 280, 301, 324, 362, 364, 372, 378, 393, 437, 444, 459, 497
nroff_manmacros 364

ntlm 1382
NTP 988
ntp_time 988
null 1883, 1889
nullzero 1883
number theory 846
numtheory 846

O

oauth 990
object 1411, 1452, 1514, 1517
object oriented 1411, 1452, 1514, 1517
observe 1903
observer 617, 1903
odie 223, 961, 1079
on-idle 1839
one time pad 1911
oo 161
oometa 995
ooutil 998
optimisation 862
optimization 850, 1395
optimize 850
ordered list 1598
otp 1001, 1911
outer join 1612

P

package 227
package indexing 266
page 27, 1003, 1004, 1015, 1017, 1018, 1020, 1022
page_intro 1003
page_pluginmgr 1004
page_util_flow 1015
page_util_norm_lemon 1017
page_util_norm_peg 1018
page_util_peg 1020
page_util_quote 1022
pager 1728
paragraph 1736, 1753
PARAM 1262
parameter entry form 1647, 1671
parser 34, 105, 365, 419, 488, 548, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343,

1351
parser generator 27, 1003, 1004, 1015, 1017, 1018, 1020, 1022
parsing 150, 154, 372, 401, 412, 437, 469, 481, 548, 550, 552, 554, 564, 571, 579, 584, 590, 591, 598, 607, 615, 630, 1526, 1926, 1937
parsing expression 34, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
parsing expression grammar 34, 590, 607, 615, 1020, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1263, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
partial application 684
partition 1536
partitioned set 1536
passive 1796
password 1001
patch 266, 1748
patching 1354
paths 514
pca 856
PEG 34, 590, 1018, 1020, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1153, 1162, 1172, 1181, 1189, 1197, 1198, 1206, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
peg 607
peg_interp 615
performance 140, 145, 146, 148, 150, 152, 153, 1081
permutation 1612
persistence 1775, 1783
phone 1866
pi 781
picoirc 671
pkg_dtplite 497
pki 1023
plain text 396, 463
plane geometry 832
plugin 295, 318, 363, 364, 372, 378, 401, 412, 437, 444, 469, 481, 1145, 1154, 1163, 1198, 1207
plugin management 1029
plugin search 1029
pluginmgr 1029

png 1035
point 832
polynomial functions 859
polynomials 859
pool 1592, 1600
pop 1038
pop3 1038, 1042, 1046, 1048
pop3d 1042
pop3d_dbox 1046
pop3d_udb 1048
post-order 1628
practcl 1050
pre-order 1628
prefix 1768, 1770, 1773
prime 846
prioqueue 1598, 1600
priority queue 1598
probabilistic calculations 862
probopt 862
proc 684
procedure 665, 1671, 1679
procedure documentation 1663
processman 1079
producer 617
profile 1081
profiler 1081
projection 742
prospero 1846
protocol 108, 690, 701, 977, 1042, 1046, 1048
proxy 634
pt 34
pt_astree 1083
pt_cparam_config_critcl 1088
pt_cparam_config_tea 1089
pt_from_api 1090
pt_introduction 1098
pt_json_language 1100
pt_param 1108
pt_parse_peg 1118
pt_parser_api 1120
pt_peg_container 1127
pt_peg_container_peg 1137
pt_peg_export 1138
pt_peg_export_container 1145
pt_peg_export_json 1153
pt_peg_export_peg 1162

pt_peg_from_container 1172
pt_peg_from_json 1172
pt_peg_from_peg 1181
pt_peg_import 1189
pt_peg_import_container 1197
pt_peg_import_json 1198
pt_peg_import_peg 1206
pt_peg_interp 1215
pt_peg_introduction 1221
pt_peg_language 1224
pt_peg_op 1233
pt_peg_to_container 1236
pt_peg_to_cparam 1244
pt_peg_to_json 1253
pt_peg_to_param 1262
pt_peg_to_peg 1286
pt_peg_to_tclparam 1295
pt_pegrammar 1303
pt_pexpr_op 1310
pt_pexpression 1314
pt_pgen 1320
pt_rdengine 1324
pt_tclparam_config_nx 1339
pt_tclparam_config_snit 1341
pt_tclparam_config_tcloo 1342
pt_to_api 1343
pt_util 1351
public key cipher 1023
publisher 617
push down automaton 34, 590, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1263, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351

Q

qcomplex 866
quasi-random 870
quasirandom 870
queue 227, 630, 1600, 1611, 1812
quoting 1022

R

radians 781, 1840
radiobutton 622
radius 1563
random 1884, 1890

random numbers 1402
randseed 1884
rational functions 872
rational_funcs 872
raw 1716
rc4 1352
rcs 1354
RCS patch 1354
read 205, 206
reading 150
receive 1731
receiver 1731, 1733, 1808
reconnect 967
record 1600, 1602
recursive descent 34, 590, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1263, 1286, 1295, 1303, 1310, 1314, 1321, 1324, 1339, 1341, 1342, 1343, 1351
reduce 537, 1612
reference 372, 378, 401, 437, 444, 469
reflected channel 1879, 1880, 1881, 1883, 1884, 1885, 1886, 1887, 1888, 1889, 1890, 1891, 1892, 1893, 1894, 1896, 1898, 1899, 1900, 1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1910, 1911
regex 1522
regular expression 550, 552, 554, 564, 1753, 1766, 1770, 1773
regular grammar 550, 552, 554, 564
regular languages 550, 552, 554, 564
remote communication 183, 200
remote execution 183, 200
remove 507
repeat 1750
repeating 1612
repetition 1612, 1750
report 1358
reshuffle 1612
residual graph 1563
resolver 246
resource management 1832
rest 1365
restore 967
return 1831
reverse 1612
rfc 821 940, 945, 1406
rfc 822 940, 945, 1046

rfc 868 988
rfc 959 519, 525, 526
rfc 977 982
rfc 1034 246
rfc 1035 246
rfc 1036 982
rfc 1320 932, 935, 1375, 1378
rfc 1321 932, 935, 1375, 1378
rfc 1413 652
rfc 1630 1846
rfc 1886 246
rfc 1939 1038, 1042
rfc 2030 988
rfc 2045 940
rfc 2046 940
rfc 2049 940
rfc 2104 932, 935, 1375, 1378, 1389, 1392
rfc 2141 1853
rfc 2251 690, 701
rfc 2255 1846
rfc 2289 1001
rfc 2396 1846
rfc 2554 945
RFC 2718 990
rfc 2821 945, 1406
rfc 2849 701
rfc 3207 945
rfc 3513 251
rfc 3986 1846
rfc 4511 690
RFC 5849 990
rfc 6455 1912
rfc 7858 246
rfc2822 178
rfc3501 653
rfc3548 117
rfc4648 117, 121
right outer join 1612
RIPEMD 1375, 1378
ripemd128 1375
ripemd160 1378
roman 875
roman numeral 875
romberg 876
roots 765
rot 1904

rot13 1904
rounding 803
rows 1716
rpc 183, 200
rsa 1023
rtcore 534
running 552

S

S3 80
SASL 1381, 1382, 1388
sasl 1382
scanl 537
SCCS 1354
scram 1388
secure 183, 1038, 1042, 1796, 1808, 1815
security 78, 156, 212, 214, 219, 222, 240, 243, 244, 932, 935, 938, 1001, 1023, 1352, 1375, 1378, 1389, 1392
seed 1884
selectionbox 674
semantic markup 285, 287, 289, 290, 293, 295, 306, 308, 310, 311, 316, 318, 331, 333, 341, 342, 352, 354, 412, 481
send 183
serialization 134, 369, 384, 389, 393, 396, 398, 423, 429, 450, 455, 459, 463, 466, 493, 1145, 1154, 1163, 1172, 1181, 1198, 1207, 1236, 1244, 1253, 1263, 1286, 1295, 1541, 1628
server 25, 729, 741, 974, 975, 980, 1834
service 718
services 526, 639, 1406
set 1600, 1625
sha1 1389
sha256 1392
shell 1526
shortest path 1563
shuffle 1612
simulated annealing 1395
simulation 1402
simulation_random 1402
singleton 998, 1787
size limit 1902
skiplist 1600, 1609
slippy 731, 740, 741
smtp 940, 945, 1406
smtpd 1406
snit 664, 666, 1411
snitfaq 1452
SNTP 988

socket 183, 200, 1406
soundex 1513
source 46, 258, 266
spacer 1905
spacing 1905
spatial interpolation 805
special 881
special functions 881
specification 148
speed 1081
split 1766
squared graph 1563
ssl 183, 653, 1038, 1042, 1796, 1808, 1815
stack 1600, 1611
standard io 1885
state 34, 550, 552, 554, 564, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1263, 1286, 1295, 1303, 1310, 1314, 1321, 1324, 1339, 1341, 1342, 1343, 1351
state (de)serialization 953
statistical distribution 1402
statistics 209, 774, 830, 856, 889
std 1885
stdin 1885
stdout 1885
stochastic modelling 1399
stooop 1514
stream cipher 1352
stream copy 1903
string 1522, 1526, 1736, 1741, 1750, 1751, 1753, 1766, 1768, 1770, 1773
stringprep 1529, 1532, 1536
stringprep_data 1532
strongly connected component 1563
struct 1592, 1602
struct_list 1612
struct_map 1624
struct_set 1625
struct_tree 1628
struct_tree1 1639
structure 203
structured queries 1820
style 363
subcommand 1671, 1679

subgraph 1541, 1563
subject 617
submitbutton 674
subscriber 617
subsequence 1612
subst 365
sum 222
swapping 1612
switched 1517
symdiff 927
symmetric difference 1625
synchronous 159
syntax tree 597

T

tabify 1751
table 437, 444, 469, 622, 1358
table of contents 301, 306, 318, 429, 433, 437, 444, 450, 455, 459, 463, 466, 469, 477, 481
tabstops 1751
tallying 209
tape archive 1643
tar 1643
tcl 751, 760, 784, 856
Tcl module 266
Tcl syntax 365
tcl_community_communication 49
tcl_parse 365
tcldes 243
tcldesjr 244
tcldocstrip 46
tcler's wiki 372, 378, 437, 444
tcllib 227
tcllib_coroutine 206
tcllib_devguide 52
tcllib_dns 246
tcllib_fhome 1830
tcllib_fifo 1886
tcllib_fifo2 1887
tcllib_installer 68
tcllib_interp 666
tcllib_ip 251
tcllib_license 74
tcllib_memchan 1888
tcllib_msgcat 368
tcllib_null 1889
tcllib_random 1890

tcllib_releasemgr 75
tcllib_sources 76
tcllib_string 1891
tcllib_throw 1831
tcllib_try 1832
tcllib_variable 1892
tcllib_zero 1893
tcllib_zlib 1906
TclOO 161, 639, 995, 998, 1787, 1790, 1795
TCLPARAM 1295
TDPL 34, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1153, 1162, 1172, 1181, 1189, 1197, 1198, 1206, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1262, 1286, 1295, 1303, 1310, 1314, 1320, 1324, 1339, 1341, 1342, 1343, 1351
temp file 498
template processing 1741
tepam_argument_dialogbox 1647
tepam_doc_gen 1663
tepam_introduction 1671
tepam_procedure 1679
term 1732
term_bind 1733
term_send 1735
terminal 1703, 1707, 1714, 1715, 1716, 1718, 1725, 1728, 1731, 1732, 1733, 1735
test 498
Testing 1855, 1857, 1859, 1861, 1863, 1864, 1866, 1868, 1870, 1872, 1873, 1875, 1877
testing 140, 145, 146, 148, 150, 152, 153
TeX 1736, 1753
text 150, 153, 372, 378, 437, 444
text comparison 1513
text conversion 1354
text differences 1354
text display 1725, 1728
text expansion 1741
text likeness 1513
text processing 27, 154, 1003, 1004, 1015, 1017, 1018, 1020, 1022, 1926, 1937
text widget 1894
textutil 1753
textutil_split 1766
textutil_string 1768
textwindow 1894
threads 205, 206

throw 1831
thumbnail 676
tie 1775, 1783
tie_std 1783
tif 1783
tiff 1783
tile 740, 741
time 988
timestamp 1035
timestamps 237
tip 219 1879, 1880, 1881, 1883, 1884, 1885, 1886, 1887, 1888, 1889, 1890, 1891, 1892, 1893, 1894, 1896, 1898
tip 230 1899, 1900, 1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1910, 1911
tip 234 1906
tip 317 1907
Tk 1894
tls 183, 653, 945, 1038, 1042, 1796, 1808, 1815
TMML 15, 280, 301, 324, 362, 372, 436, 497
toc 301, 306, 318, 429, 433, 450, 455, 459, 463, 466, 477
toc formatter 318
toc_container 436
toc_export 444
toc_export_html 450
toc_export_json 455
toc_export_nroff 459
toc_export_text 463
toc_export_wiki 466
toc_import 469
toc_import_json 477
toc_introduction 481
toc_msgcat_c 484
toc_msgcat_de 485
toc_msgcat_en 486
toc_msgcat_fr 487
toc_parse 488
toc_structure 493
token 1522
token_shell 1526
tokenization 1522, 1526
TOOL 995, 1795
tool 1790
tool_dict_ensemble 1795
top-down parsing languages 34, 590, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224,

1233, 1236, 1244, 1253, 1263, 1286, 1295, 1303, 1310, 1314, 1321, 1324, 1339, 1341, 1342, 1343, 1351
torrent 134
touch 498
TPDL 590
tqueue 1812
trace 231, 235, 236, 237
transducer 34, 548, 550, 552, 554, 564, 590, 607, 615, 1083, 1088, 1089, 1090, 1098, 1100, 1108, 1118, 1120, 1127, 1137, 1138, 1145, 1154, 1163, 1172, 1181, 1190, 1197, 1198, 1207, 1215, 1221, 1224, 1233, 1236, 1244, 1253, 1263, 1286, 1295, 1303, 1310, 1314, 1321, 1324, 1339, 1341, 1342, 1343, 1351
transfer 1796, 1800, 1803, 1805, 1808, 1812, 1815
transformation 1020, 1899, 1900, 1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1910, 1911
transformcore 1898
transmitter 1815
travelling salesman 1563
traversal 515
traverse 515
tree 571, 597, 630, 1600, 1611, 1628, 1639, 1820
tree query language 1820
tree walking 1015, 1017, 1018
treeql 1820
trig 928
trigonometry 928
trim 1770
trimming 1753, 1770
twitter 990
type 498, 530, 532, 533, 534, 1411
Type checking 1855, 1857, 1859, 1861, 1863, 1864, 1866, 1868, 1870, 1872, 1873, 1875, 1877

U

udpcluster 1834
uevent 617, 1835
uevent_onidle 1839
unbind 1835
uncapitalize 1768
undenting 1736
unicode 1529, 1532, 1533, 1536
unicode_data 1536
unified format 1748
unified format diff 1748
union 1536, 1625
unit 1840

units 1840
unknown hooking 953
untie 1775, 1783
update 205, 206
uri 1846, 1853
url 372, 378, 401, 444, 469, 729, 741, 1846, 1853
urn 1853
urn-scheme 1853
US-NPI 1873
usnpi 1873
utilities 953
uuencode 128
uuid 1854

V

Validation 1855, 1857, 1859, 1861, 1863, 1865, 1866, 1868, 1870, 1872, 1873, 1875, 1877
valtype_common 1875
Value checking 1855, 1857, 1859, 1861, 1863, 1865, 1866, 1868, 1870, 1872, 1873, 1875, 1877
vectors 810
verhoeff 1877
vertex 1541, 1563
vertex cover 1563
virtual channel 1879, 1880, 1881, 1883, 1884, 1885, 1886, 1887, 1888, 1889, 1890, 1891, 1892, 1893, 1894, 1896, 1898, 1899, 1900, 1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1910, 1911
virtual machine 571, 579, 584, 590, 591, 598, 615, 1108
VISA 1861
vt_base64 1907
vt_counter 1908
vt_crc32 1910
vt_otp 1911
vwait 205, 206, 1406

W

wais 1846
wcswidth 1773
websocket 1912
widget 1411, 1452
widget adaptors 1411, 1452
wiki 280, 301, 372, 378, 398, 437, 444, 466
wip 1920
word 365, 1920
WWW 639
www 1846

X

x.208 108
x.209 108
x.500 690
XGoogleToken 1381
xml 105
xor 1911
XPath 1820
XSLT 1820
xsxp 105

Y

yaml 1926, 1937
ydecode 131
yEnc 131
yencode 131

Z

zero 1883, 1893
zip 1940, 1943, 1944
zlib 1906
zoom 731, 740, 741